WRF 源码精读

——《WRF 源码导论》配套源码文库

作者：Jim Zhao

单位：兰州大学大气科学学院

2026 年 10 月

Contents

code_00 — 《WRF 源码精读》文库总纲（README） **1**
一、文库是什么 . . . 1
二、双轨读法 . . . 1
三、篇目总表 . . . 1
四、体例约定 . . . 2
五、工具链 . . . 2
六、阅读路径建议 . . . 2

code_01 — 模型常数层（share/module_model_constants.F：全书共用常数的唯一权威定义） **3**
一、版本指纹 . . . 3
二、定位 . . . 3
三、全文（逐段注记） . . . 3
四、改造入口 . . . 5
五、上下游链接 . . . 5

code_05 — RRTMG 长波辐射方案（module_ra_rrtmg_lw.F：McICA 随机云与 k 分布气体吸收） **7**
一、版本指纹 . . . 7
二、定位 . . . 7
三、文件头：parkind 与 parrrtm . . . 8
四、WRF 入口 RRTMG_LWRAD . . . 10
五、方案核心 rrtmg_lw（全文） . . . 15
六、McICA 云光学 cldprmc（全文） . . . 20
七、气体吸收 taumol 与波段样本 taugb1 . . . 23
八、传输求解 rtrnmc（头段） . . . 26
九、休眠与未录单元注记 . . . 28
十、改造入口 . . . 28
十一、上下游链接 . . . 28

code_06 — RRTMG 短波辐射方案（module_ra_rrtmg_sw.F：两流传输与 delta-Eddington 标定） **29**
一、版本指纹 . . . 29
二、定位 . . . 29
三、文件头：parrrsw . . . 30
四、方案核心 rrtmg_sw（全文） . . . 31
五、WRF 入口 RRTMG_SWRAD（三段） . . . 39
六、云光学 cldprmc_sw（尾段：delta-M 标定） . . . 42
七、气体吸收 taumol_sw 与波段样本 taumol16 . . . 43
八、两流传输 spcvmc_sw（头段） . . . 45
九、ECMWF 六类气溶胶 swaerpr（全文） . . . 46
十、改造入口 . . . 47
十一、上下游链接 . . . 47

文库 code_07a：module_mp_thompson.F 上半——模块声明、初始化与列驱动 **49**
07a.0 版本指纹与阅读说明 . . . 49
07a.1 一句话定位与调用链 . . . 49
07a.2 文件头注释原文与精译 . . . 49
07a.3 USE 依赖与模块级开关 . . . 50

07a.4 模块级常数逐组解读 . . . 51
07a.4.1 密度、云滴数与气溶胶缺省剖面 . . . 51
07a.4.2 谱形参数：N(D) 的 “形状” 由谁定 . . . 52
07a.4.3 质量与落速幂律：把谱 “接上” 物理 . . . 52
07a.4.4 收集效率、数值下限与冰核化常数 . . . 53
07a.4.5 气体常数与潜热 . . . 54
07a.4.6 查找表体系与预计算矩阵 . . . 54
07a.5 子程序总表 . . . 56
07a.6 `thompson_init` 逐段精讲 . . . 57
07a.6.1 签名与 “PRESENT 判型” . . . 57
07a.6.2 气溶胶缺省剖面 . . . 57
07a.6.3 查找表分配与常数预计算 . . . 59
07a.6.4 粒径 bin 与建表调用链 . . . 61
07a.7 `mp_gt_driver` 逐段精讲 . . . 64
07a.7.1 签名与实参契约 . . . 64
07a.7.2 提列与霰参数反推 . . . 65
07a.7.3 主实体调用与降水回写 . . . 66
07a.7.4 回写、负值治理与诊断接口 . . . 67
07a.8 `mp_thompson` 主实体逐段精讲 . . . 69
07a.8.0 签名与局部变量 . . . 69
07a.8.1 倾向数组清零 . . . 71
07a.8.2 列装入与谱参数化：从 (q, N) 到 (N_0, λ) . . . 72
07a.8.3 热力学环境场 . . . 74
07a.8.4 雪矩量与谱参数（雨/霰） . . . 75
07a.8.5 暖雨过程 . . . 77
07a.8.6 冰相过程准备：查表索引与淞附效率 . . . 78
07a.8.7 雪/霰淞附与雨-雪/雨-霰碰并 . . . 80
07a.8.8 冰核化三来源 . . . 81
07a.8.9 冰晶/雪/霰的凝华与转化 . . . 83
07a.8.10 融化分支 . . . 84
07a.8.11 守恒五保险 . . . 85
07a.8.12 倾向装配与潜热 . . . 86
07a.8.13 TAU+1 预更新 . . . 89
07a.8.14 饱和调整（凝结/蒸发） . . . 91
07a.8.15 雨的蒸发 . . . 92
07a.8.16 末速度与子步长判定 . . . 94
07a.8.17 沉降积分与瞬时相变 . . . 96
07a.8.18 最终回写 . . . 98
07a.9 改造入口（→ 教材 ch20/ch21 案索引） . . . 99
07a.10 上下游链接 . . . 99

文库 code_07b：module_mp_thompson.F 下半——建表族、物理函数族与诊断接口 101
07b.0 版本指纹与阅读说明 . . . 101
07b.1 建表族：离线算什么、在线省什么 . . . 101
07b.2.1 `qr_acr_qg`：雨-霰碰撞并合五维表 . . . 101
07b.2.2 `qr_acr_qs`：雨-雪碰撞四维表（12 张小表） . . . 104
07b.2.3 `freezeH2O`：Bigg 冻结表与尘埃订正 . . . 107
07b.2.4 `qi_aut_qs`：冰晶 → 雪自动转化表 . . . 109
07b.2.5 `table_Efrw` / `table_Efsw`：碰并效率表 . . . 109
07b.2.6 `Eff_aero`：气溶胶清除效率（运行时函数） . . . 111
07b.2.7 `table_dropEvap`：蒸发的累积分布表 . . . 111
07b.2.8 `table_ccnAct` 与 `activ_ncloud`：气溶胶活化 . . . 112
07b.3 不完全 Γ 函数族（Numerical Recipes） . . . 114
07b.4 饱和混合比函数：RSLF / RSIF . . . 115
07b.5 冰核化物理函数：iceDeMott / iceKoop / delta_p . . . 116
07b.6 辐射与雷达接口 . . . 118
07b.7 Bolton (1980) 热力学函数族 . . . 123

07b.8 改造入口（→ 教材 ch20/ch21 案索引） . . . 124
07b.9 上下游链接 . . . 125

code_08 — 尘排放简易方案（module_dust_emis.F + module_data_gocart_dust.F：bulk_dust_emis 与 GOCART 五 _bins 数据） 127
一、版本指纹 . . . 127
二、定位 . . . 127
三、头部与数据底座 . . . 127
四、主体与改造入口 . . . 128
五、上下游链接 . . . 129

文库 code_09a：module_bl_mynnedmf.F90 上半——主入口、闭合初始化、混合长与湍流闭合 131
09a.0 版本指纹与阅读说明 . . . 131
09a.1 一句话定位与调用链 . . . 131
09a.2 文件头注释原文与精译 . . . 131
09a.3 模块头与调试开关 . . . 133
09a.4 主入口 mynnedmf 逐段精讲 . . . 133
09a.4.1 签名与实参契约 . . . 133
09a.4.2 编译期配置选项 . . . 136
09a.4.3 调试、清零与首次初始化 . . . 136
09a.4.4 每步主流程：列构造与 Monin–Obukhov 参数 . . . 138
09a.4.5 表面相似函数分支 . . . 140
09a.4.6 次网格云与质量通量调用 . . . 140
09a.4.7 湍流闭合与二阶矩预测 . . . 142
09a.4.8 倾向求解与收尾 . . . 143
09a.5 mym_initialize：闭合的迭代初始化 . . . 144
09a.6 mym_level2：Level 2 稳定函数与有效静力稳定度 . . . 146
09a.7 mym_length：主混合长的三套配方 . . . 149
09a.8 boulac_length0 / boulac_length：气块抬升距离法 . . . 154
09a.9 mym_turbulence：从 L2 到 L3 的扩散系数 . . . 157
09a.10 mym_predict：Crank–Nicolson 三对角预测 . . . 162
09a.11 mym_condensation：次网格云 PDF 三选 . . . 167
09a.12 改造入口（→ 教材 ch20/ch21 案索引） . . . 172
09a.13 上下游链接 . . . 172

文库 code_09b：module_bl_mynnedmf.F90 下半——倾向驱动、质量通量与辅助函数族 173
09b.0 版本指纹与阅读说明 . . . 173
09b.1 mynn_tendencies：九变量倾向的三对角装配 . . . 173
09b.2 moisture_check：负值修复与柱水量守恒 . . . 189
09b.3 mynn_mix_chem / mynn_mix_scalars：化学与标量的边界层混合 . . . 190
09b.4 retrieve_exchange_coeffs 与三对角求解器三兄弟 . . . 193
09b.5 GET_PBLH：混合型边界层高度诊断 . . . 194
09b.6 DMP_mf：八羽谱积分上升流（EDMF 核心） . . . 196
09b.7 condensation_edmf 三件：羽流的饱和调整 . . . 209
09b.8 ddmp_mf：辐射冷却驱动的下沉流 . . . 211
09b.9 SCALE_AWARE：灰区网格自适应函数 . . . 217
09b.10 blend 饱和函数族：与 Thompson 同源的 “相位混合” . . . 218
09b.11 phim / phih：Puhales (2020) 新相似函数 . . . 220
09b.12 topdown_cloudrad：云顶辐射冷却的解析 TKE 源 . . . 221
09b.13 改造入口（→ 教材 ch20/ch21 案索引） . . . 223
09b.14 上下游链接 . . . 223

code_10 — MYNN 表面层方案（MYNN-SFC 四件套 + 双运行线） 225
一、版本指纹 . . . 225
二、定位 . . . 225
三、WRF 运行线：常数桥接层与 wrapper . . . 226
3.1 常数桥接层 module_sf_mynnsfc_common（全文） . . . 226
3.2 wrapper 文件头：新选项命名与文献 . . . 227

3.3 mynnsfc_init：查表初始化 . . . 228
3.4 mynnsfc_driver：逐点分派 . . . 228
四、CCPP 单柱线主讲：module_sf_mynnsfc . . . 232
4.1 模块头与选项文档 . . . 232
4.2 SFCLAY_mynn：三维包装（全文） . . . 233
4.3 SFCLAY1D_mynn：物理核心（十段走读） . . . 237
4.4 z/L 迭代器与初猜 . . . 252
4.5 相似性函数与粗糙度参数化库 . . . 254
4.6 查表系统：psi_init 与两套表函数 . . . 264
五、三伴生件 C 级注记 . . . 267
5.1 module_sf_mynnsfc_land.F90（1749 行，B 级不展开） . . . 267
5.2 module_sf_mynnsfc_water.F90（2104 行，B 级不展开） . . . 268
5.3 module_sf_mynnsfc_ice.F90（1552 行，B 级不展开） . . . 269
六、改造入口与验收 . . . 270
6.1 选项编号对照（改造前必查） . . . 270
6.2 高频改造点 . . . 270
七、上下游链接 . . . 270

code_11 — New Tiedtke 积云参数化的 WRF 外壳（module_cu_ntiedtke.F：CCPP 式三段桥接） 271
一、版本指纹 . . . 271
二、定位 . . . 271
三、文件头与模块声明 . . . 272
四、主入口 cu_ntiedtke_driver（全文） . . . 273
五、ntiedtkeinit（全文） . . . 275
六、cu_ntiedtke_pre_run（全文） . . . 276
七、cu_ntiedtke_post_run（全文） . . . 278
八、physics_mmm 实体对照（注记） . . . 278
九、改造入口 . . . 279
十、上下游链接 . . . 279

code_12a — Noah LSM 本体 · 上半（module_sf_noahlsm.F：主入口 SFLX 与蒸发/热力线） 281
一、版本指纹 . . . 281
二、定位 . . . 281
三、文件头：MPAS/WRF 双框架与三张参数表 . . . 282
四、主入口 SFLX（全文） . . . 283
五、雪反照率 ALCALC 与冠层阻抗 CANRES . . . 291
六、雪导热 CSNOW 与蒸发线 DEVAP、DEVAP_hydro、EVAPO、FAC2MIT . . . 294
七、冻土液态水 FRH2O . . . 297
八、土壤热传导右端 HRT 与推进 HSTEP . . . 299
九、无雪路径 NOPAC . . . 302
十、潜势蒸发 PENMAN . . . 305
十一、改造入口 . . . 306
十二、上下游链接 . . . 307

code_12b — Noah LSM 本体 · 下半（module_sf_noahlsm.F：参数查表、隐式求解器与雪线） 309
一、版本指纹 . . . 309
二、定位 . . . 309
三、参数查表 REDPRM . . . 310
四、三对角求解器 ROSR12 . . . 312
五、土壤热推进外壳 SHFLX . . . 313
六、土壤水推进 SMFLX . . . 314
七、雪盖分数 SNFRAC 与相变源 SNKSRC . . . 316
八、雪盖路径 SNOPAC（全文） . . . 318
九、雪物理三件套 SNOWPACK、SNOWZ0、SNOW_NEW . . . 322
十、土壤水求解器 SRT . . . 324
十一、土壤水推进 SSTEP 与水力传导 WDFCND . . . 327
十二、温度插值三件 TBND、TDFCND、TMPAVG . . . 329

十三、蒸腾 TRANSP . . . 332
十四、老版地表交换系数 SFCDIF_off . . . 333
十五、改造入口 . . . 336
十六、上下游链接 . . . 336

code_13 — Noah 驱动外壳（module_sf_noahdrv.F：格点循环、海冰/冰川分派与 TBL 读取） 337
一、版本指纹 . . . 337
二、定位 . . . 337
三、文件头：依赖清单 . . . 338
四、lsm 主入口（B 级节选） . . . 338
4.1 分层设置与首步初始化 . . . 338
4.2 格点预处理：单位换算与强迫量 . . . 339
4.3 城市前处理与单层 UCM 的灌溉/绿洲挂点 . . . 340
4.4 ICE 三分支：海冰、普通陆地、冰川 . . . 341
4.5 状态回写、fasdas 皮肤温度与能量残差 . . . 343
4.6 单层 UCM 桥接与 FRC 加权 . . . 344
4.7 诊断累积 . . . 344
4.8 BEP/BEM 聚合 . . . 345
五、LSMINIT：状态初始化与 TBL 读取入口 . . . 346
六、SOIL_VEG_GEN_PARM：三张 TBL 的读取 . . . 347
七、mosaic 双件（C 级注记） . . . 348
7.1 lsm_mosaic 接口索引 . . . 348
7.2 lsm_mosaic_init . . . 349
八、改造入口 . . . 349
九、上下游链接 . . . 350

code_14a — BEP/BEM 共享底座 · 上半（module_sf_bep_bem.F：主入口与柱求解器） 351
一、版本指纹 . . . 351
二、定位 . . . 351
三、文件头：维度常数与物理常数 . . . 352
四、主入口 BEP_BEM（全文） . . . 353
五、柱求解器 BEP1D（全文） . . . 363
六、几何与材料参数 param（全文） . . . 370
七、插值与平均单元 . . . 373
八、辐射调制与表面温度 . . . 374
九、源项组装 buildings（全文） . . . 378
十、改造入口 . . . 381
十一、上下游链接 . . . 381

code_14b — BEP/BEM 共享底座 · 下半（module_sf_bep_bem.F：辐射、导热、通量与初始化单元） 383
一、版本指纹 . . . 383
二、城市网格 → 模式网格 . . . 383
三、峡谷辐射 . . . 386
四、导热与水分求解器 . . . 392
五、通量参数化 . . . 395
六、初始化与视野因子 . . . 401
七、辐射诊断与窗户光学 . . . 406
八、改造入口 . . . 411
九、上下游链接 . . . 411

code_15 — BEP 单独版（module_sf_bep.F 全文精读） 413
一、版本指纹 . . . 413
二、定位 . . . 413
三、文件头 . . . 413
四、主入口 BEP（全文） . . . 414
五、柱求解器 BEP1D（全文） . . . 419
六、与底座同源的单元（差异注记） . . . 422
七、改造入口 . . . 445

八、上下游链接 . . . 446

code_16 — BEM 建筑能量模型（module_sf_bem.F 全文精读） 447
一、版本指纹 . . . 447
二、定位 . . . 447
三、文件头：BEM 物理常数 . . . 447
四、主入口 BEM（全文） . . . 448
五、PV 单元 . . . 455
六、导热求解器族 . . . 456
七、室内辐射与对流 . . . 461
八、室内源与调节 . . . 468
九、改造入口（论文 2 对接） . . . 473
十、上下游链接 . . . 473

code_17 — 单层城市冠层模式 SLUCM（module_sf_urban.F：主入口选段与参数表） 475
一、版本指纹 . . . 475
二、定位 . . . 475
三、文件头：参数表存储与三套扩展开关 . . . 476
四、主入口选段五处 . . . 477
4.1 Macdonald 峡谷几何与 NUDAPT SVF . . . 477
4.2 屋顶温度：二分迭代与双方案分派 . . . 478
4.3 绿屋顶段（urban-nbs） . . . 479
4.4 峡谷交换与 CH_SCHEME . . . 479
4.5 收尾：多次反射、树冠与 TC/QC 加权 . . . 479
五、特色单元三件 . . . 481
六、改造入口 . . . 482
七、上下游链接 . . . 482

code_20 — 辐射驱动器（module_radiation_driver.F：节拍门控、云量三案与气溶胶管线） 483
一、版本指纹 . . . 483
二、定位 . . . 483
三、节拍门控与太阳几何 . . . 484
四、云量三案（namelist icloud） . . . 487
五、臭氧与气溶胶三通道 . . . 493
六、方案分派：长波 . . . 495
七、方案分派：短波 . . . 496
八、收尾：FARMs、状态恢复与通量累积 . . . 498
九、地形阴影 pre_radiation_driver . . . 502
十、短波插值器（swint_opt=1/2） . . . 504
十一、单元总表（21 伴生单元） . . . 505
十二、改造入口 . . . 505
十三、上下游链接 . . . 506

文库 code_21：module_microphysics_driver.F——微物理方案的总装配面 507
21.0 版本指纹与阅读说明 . . . 507
21.1 一句话定位与调用链 . . . 507
21.2 模块头 . . . 507
21.3 USE 普查：44 项依赖的分组解读 . . . 508
21.4 网格结构与变量定义注释 . . . 509
21.5 实参契约：约 400 个实参的分组地图 . . . 510
21.6 前置处理：从进入函数到 tile 循环 . . . 514
21.7 分发骨架：`micro_select` 与 KESSLER 样板 . . . 516
21.8 Thompson 三 CASE：8 号、28 号、38 号 . . . 517
21.9 SELECT 收尾与闪电诊断 . . . 520
21.10 扰动子程序对 . . . 520
21.11 改造入口：加一个新微物理方案的完整清单 . . . 521
21.12 上下游链接 . . . 522

code_22 — PBL 驱动器（module_pbl_driver.F 全 CASE 走读） 523
一、版本指纹 . . . 523
二、定位 . . . 523
三、主入口 pbl_driver . . . 524
3.1 USE 清单与方案点名册 . . . 524
3.2 局部变量与方案前置旗标 . . . 525
3.3 触发节拍与 PBL 时间步 . . . 525
3.4 状态保存、倾向清零与 a/b/sf/vl 预备 . . . 527
3.5 方案分派 CASE 全走读 . . . 528
3.6 CASE 之外的四个附加模块 . . . 538
四、SPP 与通用扩散辅助件 . . . 541
五、改造入口 . . . 546
六、上下游链接 . . . 546

code_23 — 积云参数化公共调度器（module_cumulus_driver.F：时间门控与全 CASE 分派） 547
一、版本指纹 . . . 547
二、定位 . . . 547
三、USE 链与方案名单 . . . 548
四、本地初始化与缺省值 . . . 549
五、四测试门控：CUDT 的真实语义 . . . 549
六、前期倾向组装：G3 与 NTIEDTKE 专属 . . . 550
七、tile 循环与 MSKF 气溶胶插值 . . . 551
八、CASE 分派 • 上：KF 家族、BMJ、MSKF、GD . . . 551
九、CASE 分派 • 中：SAS 三兄弟、G3、GFL . . . 553
十、CASE 分派 • 下：CAMZM、新旧 Tiedtke、KSAS/NSAS、KFCUP . . . 555
十一、收尾：G3 平滑、PRATEC 与激活时刻回写 . . . 558
十二、气溶胶插值两单元（全文） . . . 558
十三、改造入口 . . . 561
十四、上下游链接 . . . 561

code_24 — 表面驱动器（module_surface_driver.F 两族 CASE 选段） 563
一、版本指纹 . . . 563
二、定位 . . . 563
三、前处理：耦合、灌溉、SST 与海冰 . . . 563
四、面层方案族 sfclay_select（全 CASE） . . . 566
五、陆面方案族 sfc_select：SLAB 与 Noah 代表段 . . . 570
六、收尾：海洋/湖泊/诊断限幅/人为热 . . . 575
七、MYNN 海冰 wrapper（全文） . . . 577
八、改造入口 . . . 580
九、上下游链接 . . . 580

code_30 — 大步长动力工具箱（dyn_em/module_big_step_utilities_em.F：状态方程、phy_prep 与 rhs_ph） 581
一、版本指纹 . . . 581
二、定位 . . . 581
三、质量场：calc_mu_uv . . . 582
四、状态诊断：calc_alt、calc_p_rho_phi、calc_php . . . 583
五、物理翻译官 phy_prep（全文） . . . 587
六、位势方程右端 rhs_ph（头段） . . . 590
七、未录单元注记 . . . 591
八、改造入口 . . . 592
九、上下游链接 . . . 592

code_31 — 非时间分裂物理总装·上（dyn_em/module_first_rk_step_part1.F：辐射 → 陆面 →PBL→ 积云 → 浅对流 →FDDA 调用序） 593
一、版本指纹 . . . 593
二、定位 . . . 593
三、模块头与 USE 链 . . . 594
四、倾向清零与状态换算 . . . 595
五、辐射段：pre_radiation_driver + radiation_driver . . . 596
六、陆面段：surface_driver . . . 599
七、边界层段：pbl_driver . . . 605
八、火段：ifire 三选 . . . 608
九、积云段：cumulus_driver + convtrans_prep . . . 609
十、浅对流与 SCM 强迫 . . . 611
十一、格点 nudging 收尾 . . . 612
十二、改造入口 . . . 613
十三、上下游链接 . . . 613

code_32 — 非时间分裂物理总装·下（dyn_em/module_first_rk_step_part2.F：倾向记账、TKE 混合与随机扰动） 615
一、版本指纹 . . . 615
二、定位 . . . 615
三、USE 链 . . . 616
四、随机参数化五连 . . . 617
五、倾向记账预备 calculate_phy_tend . . . 619
六、扩散度量与 Km/Kh . . . 620
七、条件 HALO 与点阵地图 . . . 622
八、总账合并 update_phy_ten . . . 623
九、PLANET 彩蛋 damptop . . . 624
十、观测 nudging 与 use_theta_m . . . 625
十一、垂直扩散后水平扩散 . . . 626
十二、改造入口 . . . 627
十三、上下游链接 . . . 627

code_40 — real 初始化的基态构造（dyn_em/module_initialize_real.F 选段：基态气压/温度/位势与 500 hPa 调整） 629
一、版本指纹 . . . 629
二、定位 . . . 629
三、调用点：znw 与 c1..c4 系数的生成 . . . 630
四、基态构造五段 . . . 631
五、未录单元注记（42 单元分组） . . . 634
六、改造入口 . . . 634
七、上下游链接 . . . 634

code_41 — 垂直坐标系数生成（dyn_em/nest_init_utils.F：compute_eta 与 compute_vcoord_1d_coeffs） 637
一、版本指纹 . . . 637
二、定位 . . . 637
三、compute_vcoord_1d_coeffs（全文） . . . 638
四、compute_eta（头段） . . . 640
五、未录单元注记 . . . 641
六、改造入口 . . . 642
七、上下游链接 . . . 642

code_50 — solve_em 主循环导航页（dyn_em/solve_em.F：十四站的地图与调用点索引） 643
一、版本指纹 . . . 643
二、定位 . . . 643
三、主循环地图 . . . 644
四、调用点行号索引（4.8.0 实测） . . . 644
五、与文库各篇的对照 . . . 644
六、上下游链接 . . . 645

code_60 — 短波气溶胶光学预处理器（module_ra_aerosol.F：AOD@550 展开成方案波段） 647
一、版本指纹 . . . 647
二、定位 . . . 647
三、头部注记与两个主入口头段 . . . 647
四、单元接口索引（10 单元） . . . 649
五、改造入口 . . . 650
六、上下游链接 . . . 650

code_61 — Noah 的海冰与冰川件（module_sf_noah_seaice_drv.F + module_sf_noah_seaice.F + module_sf_noahlsm_glacial_only.F：C 级接口索引） 651
一、版本指纹 . . . 651
二、定位 . . . 651
三、三件头部 . . . 651
四、同源单元差异注记（不重录） . . . 653
五、改造入口 . . . 653
六、上下游链接 . . . 653

code_62 — 次级驱动器三件头部（shallowcu_driver + module_fr_fire_driver_wrf.F + fddagd_driver：C 级接口索引） 655
一、版本指纹 . . . 655
二、定位 . . . 655
三、独立浅对流 shallowcu_driver . . . 655
四、火模式 WRF 包装层 module_fr_fire_driver_wrf.F（全文） . . . 656
五、格点 nudging fddagd_driver . . . 658
六、改造入口 . . . 658
七、上下游链接 . . . 658

code_00 — 《WRF 源码精读》文库总纲（README）

> 版本锁定：WRF 4.8.0（`source/WRFv4.8.0/WRFV4.8.0/`）；所有代码块由 `workspace/scripts/code_extract.py` 从源码逐位抽取并附 md5 指纹（acceptance.py 全量校验）。2026-10-12 建成，**29/29 篇全部交付，文库齐账**。

一、文库是什么

与教材（`doc\chNN_*.md`）**硬切**：教材回答「为什么这样写、物理对应、方案脉络」（节选 +导读），文库回答「这段代码在做什么、每个变量是谁、完整上下文」（保真源码 +行段中文讲解，独立可读）。两向导航：教材各章「延伸阅读」指向文库篇，文库每篇开头给「谁在调它」与教材锚点。

二、双轨读法

- **依赖拓扑序**（默认编号）：被依赖者在前——读任何一篇时它用到的下层概念已读过。L0 底座（01）→ L1 方案本体（05–17）→ L2 驱动器（20–24）→ L3 动力枢纽（30–32）→ L4 初始化（40–41）→ L5 内核导航（50）→ C 段外围（60–62）。
- **教材章序**：ch10 坐标 →code_41/30；ch12 辐射 →code_20/05/06/60；ch13 微物理 →code_21/07a/07b/08；ch14 PBL→code_22/09a/09b/10；ch15 积云 →code_23/11/62；ch16 陆面 →code_24/13/12a/12b/61；ch17 城市 →code_14a/14b/15/16；ch21 内核 →code_50/31/32/30；ch20 改造 → 各篇「改造入口」节。

三、篇目总表

编号	文件	级	交付	教材锚点
01	code_01_module_model_constants.md	A 轻	✓	app_a/ch11
05	code_05_module_ra_rrtmg_lw.md	B	✓	ch12
06	code_06_module_ra_rrtmg_sw.md	B	✓	ch12
07	code_07a/07b_module_mp_thompson.md	A	✓ 拆两	ch13
08	code_08_module_dust_emis.md	C	✓	ch13/ch20
09	code_09a/09b_module_bl_mynnedmf.md	A	✓ 拆两	ch14
10	code_10_module_sf_mynnsfc.md	B	✓	ch14
11	code_11_module_cu_ntiedtke.md	A	✓	ch15
12	code_12a/12b_module_sf_noahlsm.md	A	✓ 拆两	ch16
13	code_13_module_sf_noahdrv.md	B	✓	ch16
14	code_14a/14b_module_sf_bep_bem.md	A	✓ 拆两	ch17
15	code_15_module_sf_bep.md	A	✓	ch17
16	code_16_module_sf_bem.md	A	✓	ch17
17	code_17_module_sf_urban.md	B	✓	ch17
20	code_20_module_radiation_driver.md	B	✓	ch12
21	code_21_module_microphysics_driver.md	B	✓	ch13
22	code_22_module_pbl_driver.md	B	✓	ch14
23	code_23_module_cumulus_driver.md	B	✓	ch15
24	code_24_module_surface_driver.md	B	✓	ch16/17
30	code_30_module_big_step_utilities_em.md	B	✓	ch21/ch10
31	code_31_module_first_rk_step_part1.md	B	✓	ch21
32	code_32_module_first_rk_step_part2.md	B	✓	ch21

编号	文件	级	交付	教材锚点
40	code_40_module_initialize_real.md	B	✓	ch07/ch10
41	code_41_nest_init_utils.md	B	✓	ch10
50	code_50_solve_em.md	C	✓	ch21
60	code_60_module_ra_aerosol.md	C	✓	ch11/12
61	code_61_noah_seaice_glacial.md	C	✓	ch16
62	code_62_shallowcu_fire_fdda.md	C	✓	ch15/ch21

不收录（plan § 三取舍）：phys/noahmp/（248 文件独立库）、module_dm/IO/通用工具、TEMPO 与 physics_mmm 目录（physics_mmm 的 cu_ntiedtke 实体已按对照注记收进 code_11 § 八）、module_ra_cam*（CAM 辐射旧系）。

四、体例约定

1. **八段模板**：版本指纹 → 定位 → 文件头 → 逐段保真块 +讲解 → 单元总表 → 改造入口 → 上下游链接。
2. **保真红线**：代码一律脚本抽取（显示层 =块首 `!file:` 标记 + 逐行 `L{05d}` 行号前缀），md5 对原始内容；acceptance.py 自动全量。
3. **讲解规范**：中文全角引号；源码符号不译；书公式只给指针（「书 (6.8.14) → ch14 §14.8」式）；mermaid 边标签禁半角括号与斜杠。
4. **A 级 >5000 行拆 _a/_b**；B 级同源单元用「差异注记 +回指」不重录。

五、工具链

- workspace/scripts/code_extract.py（v1.2，扫描器栈式配对支持 F90 CONTAINS 嵌套）：list/full/sub/lines/verify/assemble；多源占位符语法为双花括号内「CODE 前缀 + 冒号 + WRF 根相对路径 + 冒号 + full 或子程序名或行段」。
- workspace/scripts/acceptance.py：全量指纹校验（glob doc/code/code_*.md，按指纹定位源、剥离显示层比对）。
- workspace/scripts/fp_files.py：篇首指纹表；check_quotes.py：全角引号检查。
- 骨架（可再组装的源稿）在 workspace/code_library/skel_code_*.md。

六、阅读路径建议

1. **系统学 WRF 内核**：01 → 30 → 50（地图）→ 31 → 32 → 41 → 40，配教材 ch21/ch10。
2. **改一个物理方案**：先读对应驱动器（20/21/22/23/24）的 CASE 分派，再读本体的方案篇，最后照「改造入口」动手，配 ch20。
3. **只跑个例不改码**：40/41（初值从哪来）+ 13/24（陆面/海冰分支）+ 21（降水从哪来）。
4. **城市热岛专题**（论文 2 路线）：14a→14b→16→15→12a/12b（背景场）→31 的 surface 段 →ch17/ch19。

code_01 — 模型常数层（share/module_model_constants.F：全书共用常数的唯一权威定义）

一、版本指纹

源文件	行数	md5（整文件，按字节）
share/module_model_constants.F	159	60664d7df258a193d01cbbd1eeec51db

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级轻量全文（159 行全收）。与书附录 C 的逐常数偏差对照已在教材 app_a_symbols_constants.md 建表，本篇只补模块内口径注记，不重复偏差表。

二、定位

一句话：**WRF 全模式的常数底座**——动力核、五大物理驱动器、初始化件全部从这一个模块取常数（文库内 code_05/06/07/09/10/12/14/20/21/23 等所有篇目的 USE 常数最终汇到这里）；无任何外部依赖，天然是依赖拓扑的 01 号。

谁在调它：几乎一切物理与动力模块（grep `USE module_model_constants` 全树数百处）；教材锚点：app_a_symbols_constants.md（三列对照表）、ch11_composition_moisture_aerosol.md（常数物理背景）。

三、全文（逐段注记）

```
!file: share/module_model_constants.F    L1-L159
L00001  !WRF:MODEL_LAYER:CONSTANTS
L00002  !
L00003
L00004   MODULE module_model_constants
L00005
L00006     !  2. Following are constants for use in defining real number bounds.
L00007
L00008     !  A really small number.
L00009
L00010     REAL    , PARAMETER :: epsilon         = 1.E-15
L00011
L00012     !  4. Following is information related to the physical constants.
L00013
L00014     !  These are the physical constants used within the model.
L00015
L00016  ! JM NOTE -- can we name this grav instead?
L00017     REAL    , PARAMETER :: g = 9.81  ! acceleration due to gravity (m {s}^-2)
L00018
L00019     REAL    , PARAMETER :: r_d          = 287.       ! gas constant of dry air (J deg^-1 kg^-1)
L00020     REAL    , PARAMETER :: cp           = 7.*r_d/2.  !
L00021
L00022     REAL    , PARAMETER :: r_v          = 461.6      ! gas constant for water vapor (J deg^-1 kg^-1)
L00023     REAL    , PARAMETER :: cv           = cp-r_d     ! Specific heat of air at contant volume (J deg^-1 kg^-1)
L00024     REAL    , PARAMETER :: cpv          = 4.*r_v
L00025     REAL    , PARAMETER :: cvv          = cpv-r_v    !
L00026     REAL    , PARAMETER :: cvpm         = -cv/cp
L00027     REAL    , PARAMETER :: cliq         = 4190.      ! specific heat of liquid water at 0^oC
L00028     REAL    , PARAMETER :: cice         = 2106.      ! specific heat of ice at 0^oC
L00029     REAL    , PARAMETER :: psat         = 610.78
L00030     REAL    , PARAMETER :: rcv          = r_d/cv     !
L00031     REAL    , PARAMETER :: rcp          = r_d/cp
L00032     REAL    , PARAMETER :: rovg         = r_d/g
L00033     REAL    , PARAMETER :: c2           = cp * rcv
L00034     real    , parameter :: mwdry        = 28.966     ! molecular weight of dry air (g/mole)
L00035
L00036     REAL    , PARAMETER :: p1000mb      = 100000.    ! pressure at 1000 hPa (pa)
```

```
L00037     REAL    , PARAMETER :: t0          = 300.       ! base state tempertaure (K)
L00038     REAL    , PARAMETER :: p0          = p1000mb    ! base state surface pressure (pa)
L00039     REAL    , PARAMETER :: cpovcv      = cp/(cp-r_d)
L00040     REAL    , PARAMETER :: cvovcp      = 1./cpovcv
L00041     REAL    , PARAMETER :: rvovrd      = r_v/r_d
L00042
L00043     REAL    , PARAMETER :: reradius    = 1./6370.0e03  ! reciprocal of earth radius (m^-1)
L00044
L00045     REAL    , PARAMETER :: asselin     = .025
L00046  !   REAL    , PARAMETER :: asselin      = .0
L00047     REAL    , PARAMETER :: cb          = 25.
L00048
L00049     REAL    , PARAMETER :: XLV0        = 3.15E6       !  constant defined for calculation of latent heating
L00050     REAL    , PARAMETER :: XLV1        = 2370.        !  constant defined for calculation of latent heating
L00051     REAL    , PARAMETER :: XLS0        = 2.905E6      !  constant defined for calculation of latent heating
L00052     REAL    , PARAMETER :: XLS1        = 259.532      !  constant defined for calculation of latent heating
L00053
L00054     REAL    , PARAMETER :: XLS         = 2.85E6      ! latent heat of sublimation of water at 0^oC (J kg^-1)
L00055     REAL    , PARAMETER :: XLV         = 2.5E6       ! latent heat of vaporization of water at 0^oC (J kg^-1)
L00056     REAL    , PARAMETER :: XLF         = 3.50E5      ! latent heat of fusion of water at 0^oC (J kg^-1)
L00057
L00058     REAL    , PARAMETER :: rhowater    = 1000.       ! density of liquid water at 0^oC (kg m^-3)
L00059     REAL    , PARAMETER :: rhosnow     = 100.        ! density of snow (kg m^-3)
L00060     REAL    , PARAMETER :: rhoair0     = 1.28        ! density of dry air at 0^oC and 1000mb pressure (kg m^-3)
L00061
L00062     REAL    , PARAMETER :: RE_QC_BG    = 2.49E-6     ! effective radius of cloud for background (m)
L00063     REAL    , PARAMETER :: RE_QI_BG    = 4.99E-6     ! effective radius of ice for background (m)
L00064     REAL    , PARAMETER :: RE_QS_BG    = 9.99E-6     ! effective radius of snow for background (m)
L00065     REAL    , PARAMETER :: RE_QC_MAX   =  50.E-6     ! max effective radius of cloud allowed
L00066     REAL    , PARAMETER :: RE_QI_MAX   = 125.E-6     ! max effective radius of ice allowed
L00067     REAL    , PARAMETER :: RE_QS_MAX   = 999.E-6     ! max effective radius of snow allowed
L00068  !
L00069  ! Now namelist-specified parameter: ccn_conc - RAS
L00070  !   REAL    , PARAMETER :: n_ccn0       = 1.0E8
L00071  !
L00072     REAL    , PARAMETER :: piconst     = 3.14159265358979323846264338    ! constant of PI
L00073     REAL    , PARAMETER :: DEGRAD      = piconst/180.                    ! radians per degree
L00074     REAL    , PARAMETER :: DPD         = 360./365.
L00075
L00076     REAL    , PARAMETER ::  SVP1=0.6112      ! constant for saturation vapor pressure calculation (dimensionless)
L00077     REAL    , PARAMETER ::  SVP2=17.67       ! constant for saturation vapor pressure calculation (dimensionless)
L00078     REAL    , PARAMETER ::  SVP3=29.65       ! constant for saturation vapor pressure calculation (K)
L00079     REAL    , PARAMETER ::  SVPT0=273.15     ! constant for saturation vapor pressure calculation (K)
L00080     REAL    , PARAMETER ::  EP_1=R_v/R_d-1.  !  constant for virtual temperature (r_v/r_d - 1) (dimensionless)
L00081     REAL    , PARAMETER ::  EP_2=R_d/R_v     ! constant for specific humidity calculation (dimensionless)
L00082     REAL    , PARAMETER ::  KARMAN=0.4               ! von Karman constant
L00083     REAL    , PARAMETER ::  EOMEG=7.2921E-5          ! angular velocity of rotation (rad^-1)
L00084     REAL    , PARAMETER ::  STBOLT=5.67051E-8        ! Stefan-Boltzmann constant (W m^-2 deg^-4)
L00085
L00086     REAL    , PARAMETER ::  prandtl = 1./3.0   ! prandtl's mixing length (m)
L00087                                                ! constants for w-damping option
L00088     REAL    , PARAMETER ::  w_alpha = 0.3      ! strength m/s/s
L00089     REAL    , PARAMETER ::  w_beta  = 1.0      ! activation cfl number
L00090
L00091         REAL , PARAMETER ::  pq0=379.90516     !
L00092         REAL , PARAMETER ::  epsq2=0.2         ! initial TKE for camuw PBL scheme (m2 s^-2)
L00093         REAL , PARAMETER ::  a2=17.2693882
L00094         REAL , PARAMETER ::  a3=273.16
L00095         REAL , PARAMETER ::  a4=35.86
L00096         REAL , PARAMETER ::  epsq=1.e-12      ! threshold specified for SPECIFIC HUMIDITY calculation in BMJ cumulus
↪  scheme (kg kg^-1)
L00097         REAL , PARAMETER ::  p608=rvovrd-1.
L00098         REAL , PARAMETER ::  climit=1.e-20
L00099         REAL , PARAMETER ::  cm1=2937.4
L00100         REAL , PARAMETER ::  cm2=4.9283
L00101         REAL , PARAMETER ::  cm3=23.5518
L00102  !       REAL , PARAMETER ::  defc=8.0
L00103  !       REAL , PARAMETER ::  defm=32.0
L00104         REAL , PARAMETER ::  defc=0.0
L00105         REAL , PARAMETER ::  defm=99999.0
L00106         REAL , PARAMETER ::  epsfc=1./1.05
L00107         REAL , PARAMETER ::  epswet=0.0
L00108         REAL , PARAMETER ::  fcdif=1./3.
L00109         REAL , PARAMETER ::  fcm=0.00003
L00110         REAL , PARAMETER ::  gma=-r_d*(1.-rcp)*0.5
L00111         REAL , PARAMETER ::  p400=40000.0
L00112         REAL , PARAMETER ::  phitp=15000.0
L00113         REAL , PARAMETER ::  pi2=2.*3.1415926, pi1=3.1415926
L00114         REAL , PARAMETER ::  plbtm=105000.0
L00115         REAL , PARAMETER ::  plomd=64200.0
L00116         REAL , PARAMETER ::  pmdhi=35000.0
L00117         REAL , PARAMETER ::  q2ini=0.50
L00118         REAL , PARAMETER ::  rfcp=0.25/cp
L00119         REAL , PARAMETER ::  rhcrit_land=0.75
L00120         REAL , PARAMETER ::  rhcrit_sea=0.80
L00121         REAL , PARAMETER ::  rlag=14.8125
L00122         REAL , PARAMETER ::  rlx=0.90
L00123         REAL , PARAMETER ::  scq2=50.0
L00124         REAL , PARAMETER ::  slopht=0.001
L00125         REAL , PARAMETER ::  tlc=2.*0.703972477
L00126         REAL , PARAMETER ::  wa=0.15
L00127         REAL , PARAMETER ::  wght=0.35
L00128         REAL , PARAMETER ::  wpc=0.075
L00129         REAL , PARAMETER ::  z0land=0.10    ! surface roughness length over land (m)
L00130         REAL , PARAMETER ::  z0max=0.008    !  maximum roughness length (m)
L00131         REAL , PARAMETER ::  z0sea=0.001   ! roughness length over ocean (m)
L00132
L00133
L00134     !  Earth
L00135
L00136     !  The value for P2SI *must* be set to 1.0 for Earth
L00137     !  Although, now we may not need this declaration here (see above)
L00138     !REAL    , PARAMETER :: P2SI         = 1.0
L00139
L00140     !  Orbital constants:
L00141
L00142     INTEGER , PARAMETER :: PLANET_YEAR = 365   ! number of days in a calendar year
L00143     REAL , PARAMETER :: OBLIQUITY = 23.5       ! solar obliquity (degree)
L00144     REAL , PARAMETER :: ECCENTRICITY = 0.014   ! Orbital eccentricity
```

```
L00145     REAL , PARAMETER :: SEMIMAJORAXIS = 1.0    ! Ratio of semi-major axis of planet / semi-major axis of earth
L00146     REAL , PARAMETER :: zero_date = 0.0        ! Time of perihelion passage
L00147     REAL , PARAMETER :: EQUINOX_FRACTION= 0.0  ! Fraction into the year (from perhelion) of the occurrence of the
↪  Northern Spring Equinox
L00148
L00149  ! 2012103
L00150  #if (EM_CORE == 1)
L00151  ! for calls to set_tiles
L00152     INTEGER, PARAMETER :: ZONE_SOLVE_EM = 1
L00153     INTEGER, PARAMETER :: ZONE_SFS = 2
L00154  #endif
L00155
L00156   CONTAINS
L00157     SUBROUTINE init_module_model_constants
L00158     END SUBROUTINE init_module_model_constants
L00159   END MODULE module_model_constants
```

讲解（按段走读，行号见显示层前缀）：① **实数界与重力**——epsilon=1e-15；**g=9.81**（书 9.80665 的圆整，JM 留言「can we name this grav instead」——与局部变量 g 重名的老问题）。② **气体与比热群**——r_d=287（书 287.05）；**cp=7·r_d/2=1004.5**（干空气定压比热的理论关系式，非独立常数）；r_v=461.6；cv=cp−r_d=717.5；**cpv=4·r_v=1846.4**（水汽比热同为理论关系）；cvv、cvpm=−cv/cp（位温方程里的负 cv/cp 项，微物理饱和调整用）；cliq=4190/cice=2106（0°C 液水/冰比热，与 noahlsm 自带的 4218/2106 微差——**两套口径并存**，见 code_12a § 三）；psat=610.78 Pa（0°C 饱和水汽压）；rcv/rcp/rovg/c2、mwdry=28.966。③ **基态群**——p1000mb=100000 Pa（**名实之差：变量名写 1000mb，值是 1e5 Pa**）；t0=300（基态温度，注意不是 0°C 的 273.15）；p0=p1000mb；cpovcv/cvovcp/rvovrd。④ **几何与滤波**——reradius=1/6370e3（地球半径倒数）；**asselin=0.025**（Asselin-Robert 时间滤波系数，动力核用；0 的备选注释 =关滤波实验）；cb=25。⑤ **旧 eta 系潜热线性式四件**——XLV0/XLV1/XLS0/XLS1（BMJ 等 Janjic 谱系方案的 Lv(T)/Ls(T) 线性式系数，与书 (4.x) 多项式口径不同——**教材 13.9.2 的 Lv(T) 三口径议题在此再加一条**）；⑥**0°C 潜热三常数**——XLS=2.85e6/XLV=2.5e6/XLF=3.50e5（书 2.834e6/2.5e6/3.336e5 的 WRF 圆整，noahlsm 用 2.83e6/2.501e6/3.335e5——又是并存口径）。⑦ **密度三件**——rhowater=1000/rhosnow=100/rhoair0=1.28。⑧ **辐射有效半径六件**——RE_QC/QI/QS_BG（2.49/4.99/9.99 μm 的背景值）与 MAX（50/125/999 μm 上限）——微物理不供有效半径时辐射云光学的默认。⑨ **角度与历法**——piconst（18 位精度）、DEGRAD、**DPD=360/365（度/儒略日）**——radconst 的太阳黄经就吃它。⑩ **饱和水汽压与虚拟温**——SVP1=0.6112/SVP2=17.67/SVP3=29.65（Tetens/Bolton 系；cal_cldfra1 的 Murray 1966 系数 17.2693882/35.86 是另一套，两套并存）；SVPT0=273.15；EP_1/EP_2；KARMAN=0.4；EOMEG=7.2921e-5（地球自转角速度）；STBOLT=5.67051e-8（Stefan-Boltzmann）。⑪ **边界层杂项**——prandtl=1/3、w_alpha=0.3/w_beta=1.0（垂直速度阻尼选项）。⑫**BMJ/eta 遗产系数群**（pq0 至 z0sea 约 40 个）——pq0=379.90516、a2/a3/a4（饱和比湿 Tetens）、epsq（BMJ 比湿阈值）、cm1..3、**defc/defm 被从 8.0/32.0 改成 0.0/99999.0（原对流触发尺度判据被关闭）**、rhcrit_land/sea=0.75/0.80、z0land=0.10/z0max=0.008/z0sea=0.001（注意 z0max 名实：是**最小粗糙度**口径还是最大——注释写 maximum，值 8mm 比 land 10cm 小，实为旧 eta 的 z0 低限，勿按名取义）。⑬ **行星常数群**——PLANET_YEAR=365、OBLIQUITY=23.5（与 radconst 的 OBECL 同值不同名）、**ECCENTRICITY=0.014（定义了但 radconst 不用它——radconst 走 Paltridge-Platt 谐波式）**、SEMIMAJORAXIS、EQUINOX_FRACTION——行星通用框架的遗迹（火星 WRF 用同一模块）。⑭**tile 分区常数**——ZONE_SOLVE_EM/ZONE_SFS（set_tiles 的分区编号，仅 EM_CORE）。⑮ 空壳 init 子程序——框架要求每个模块有 init 入口的仪式性实现。

四、改造入口

1. **常数敏感性实验**（g/r_d/cp 微调）只动本文件——全模式生效，注意 cp=7·r_d/2 的联动关系不是独立参数。
2. **Asselin 滤波开关**：asselin=0 备选注释。
3. **双口径并存清单**（改常数前先查）：cliq/cice vs noahlsm 自带；XLV/XLS/XLF vs LSUBC/LSUBS/LSUBF 系；SVP 群 vs Murray 群 vs noahlsm 的 ELCP 口径——**只改本文件不影响自带副本**（code_10/12a 已标注的同名拷贝问题同源）。
4. **行星框架**：OBLIQUITY/ECCENTRICITY 群——理想行星实验（如火星）的入口，但 radconst 的太阳几何不走这组。

五、上下游链接

- 消费方：文库全部方案篇（code_05/06/07/09/10/12/14/20/21/22/23）；动力核导航：`code_50_solve_em.md`
- 教材正文：`app_a_symbols_constants.md`（偏差对照权威表）、ch11_composition_moisture_aerosol.md

code_05 — RRTMG 长波辐射方案（module_ra_rrtmg_lw.F：McICA 随机云与 k 分布气体吸收）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_ra_rrtmg_lw.F	14633	d778343f8e829c3880d4aa2b11c089a4

- 抽取工具：workspace/scripts/code_extract.py v1.2（v1.3 起扫描器支持 CONTAINS 嵌套，taumol 内的 taugb1..16 才能配对），2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：主链五件（RRTMG_LWRAD 选段 2 段、rrtmg_lw 全文、cldprmc 全文、taumol 头段、taugb1 全文）+ rtrnmc 头段 + 文件头两模块；未录 60 余单元给分组总表（§ 九）。教材锚点：ch12_radiation_family.md（波段结构/文献/ghg 与 o3input 已精读，本篇给完整源码上下文）。

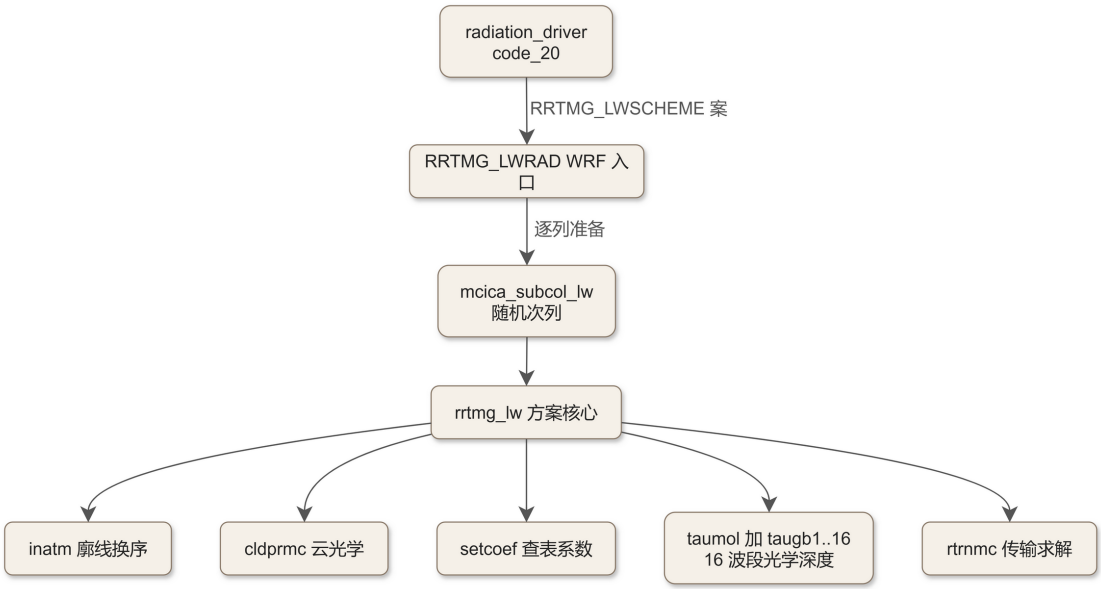
二、定位

一句话：**AER 的 RRTMG 长波方案**——RRTM 的 GCM 快速版：16 波段 k 分布（k-distribution）+ g 点从 256 缩减到 140 + **McICA（Monte-Carlo Independent Column Approximation，Pincus et al. 2003）随机次格云**；WRF 侧入口 RRTMG_LWRAD 做单位换算与逐列准备，核心 rrtmg_lw 按 inatm→cldprmc→setcoef→taumol→rtrnmc 五步流水。

谁在调它：radiation_driver 的 CASE (RRTMG_LWSCHEME)（`code_20_module_radiation_driver.md` § 六）。**模块级依赖方向：**module_ra_rrtmg_sw.F 的 parrrsw 首行就是 `use parkind`——**SW 文件不自带 parkind，从本文件导入**（SW 必须晚于 LW 编译，plan § 三「sw 依赖 lw」的落实处）。

85 单元分组总表（扫描器输出）：

组	单元	行段	说明
主链	rrtmg_lw / inatm	L10694–L11522	方案核心 / 廓线换序换算
WRF 入口	RRTMG_LWRAD / INIRAD / rrtmg_lwinit / rrtmg_lwlookuptable	L11570–L13075	逐列准备与调核心
MCICA 群	mixbits/twist/nextState/temper/initialize_scalar/…/getRandomNumber_2D	L1743–L2022	KISSVEC 随机数流（Pincus）
MCICA 群	mcica_subcol_lw / generate_stochastic_clouds / kissvec	L2091–L2729	随机次列生成
云光学	cldprmc	L2764–L3025	McICA 云光学深度
传输	rtrnmc	L3085–L3522	两向传输 +扩散角积分
廓线准备	setcoef / lwatmref / lwavplank	L3556–L4790	查表系数/参考大气/普朗克
气体吸收	taumol（含 taugb1..taugb16）	L4824–L7942	16 波段光学深度与普朗克分数
初始化	rrtmg_lw_ini / lwdatinit / lwcmbdat / O3DATA	L7976–L12978	k 系数装填与 g 点缩减
g 点缩减	cmbgb1..cmbgb16	L8320–L9971	256→140 逐波段重排
非 McICA 云	lwcldpr	L9974–L10612	**休眠**（imca=1 固定走 cldprmc）
k 系数	lw_kgb01..lw_kgb16	L13094–L14539	逐波段 k 分布系数（纯数据）
工具	relcalc / reicalc	L14542–L14630	有效半径查表



三、文件头：parkind 与 parrrtm

```
!file: phys/module_ra_rrtmg_lw.F    L1-L157
L00001  !MODULE module_ra_rrtmg_lw
L00002
L00003        module parkind
L00004  !     implicit none
L00005        save
L00006
L00007  !------------------------------------------------------------------
L00008  ! rrtmg kinds
L00009  ! Define integer and real kinds for various types.
L00010  !
L00011  ! Initial version: MJIacono, AER, jun2006
L00012  ! Revised: MJIacono, AER, aug2008
L00013  !------------------------------------------------------------------
L00014
L00015  !
L00016  !     integer kinds
L00017  !     -------------
L00018  !
L00019  !     integer, parameter :: kind_ib = selected_int_kind(13)  ! 8 byte integer
L00020  !     integer, parameter :: kind_im = selected_int_kind(6)   ! 4 byte integer
L00021        integer, parameter :: kind_ib = kind(1)
L00022        integer, parameter :: kind_im = kind(1)
L00023        integer, parameter :: kind_in = kind(1)               ! native integer
L00024
L00025  !
L00026  !     real kinds
L00027  !     ----------
L00028  !
L00029  !      integer, parameter :: kind_rb = selected_real_kind(12) ! 8 byte real
L00030  !      integer, parameter :: kind_rm = selected_real_kind(6)  ! 4 byte real
L00031  !      integer, parameter :: kind_rn = kind(1.0)              ! native real
L00032
L00033  #if 0
L00034  ! Modified for WRF:
L00035  #ifdef DOUBLE_PRECISION
L00036        integer, parameter :: kind_rb = selected_real_kind(12) ! 8 byte real
L00037  #endif
L00038  #ifndef DOUBLE_PRECISION
L00039        integer, parameter :: kind_rb = selected_real_kind(6)  ! 4 byte real
L00040  #endif
L00041  #else
L00042         integer, parameter :: kind_rb = kind(1.0)              ! native real
L00043  #endif
L00044
L00045        end module parkind
L00046
L00047        module parrrtm
L00048
L00049        use parkind ,only : im => kind_im
L00050
L00051  !     implicit none
L00052        save
L00053
L00054  !------------------------------------------------------------------
L00055  ! rrtmg_lw main parameters
L00056  !
L00057  ! Initial version:  JJMorcrette, ECMWF, Jul 1998
L00058  ! Revised: MJIacono, AER, Jun 2006
L00059  ! Revised: MJIacono, AER, Aug 2007
L00060  ! Revised: MJIacono, AER, Aug 2008
L00061  !------------------------------------------------------------------
L00062
L00063  !  name     type     purpose
L00064  ! -----  :  ----    : ----------------------------------------------
L00065  ! mxlay  :  integer: maximum number of layers
L00066  ! mg     :  integer: number of original g-intervals per spectral band
```

```
! nbndlw :  integer: number of spectral bands
! maxxsec:  integer: maximum number of cross-section molecules
!                    (e.g. cfcs)
! maxinpx:  integer:
! ngptlw :  integer: total number of reduced g-intervals for rrtmg_lw
! ngNN   :  integer: number of reduced g-intervals per spectral band
! ngsNN  :  integer: cumulative number of g-intervals per band
!-----------------------------------------------------------------------

      integer(kind=im), parameter :: mxlay  = 203
      integer(kind=im), parameter :: mg     = 16
      integer(kind=im), parameter :: nbndlw = 16
      integer(kind=im), parameter :: maxxsec= 4
      integer(kind=im), parameter :: mxmol  = 38
      integer(kind=im), parameter :: maxinpx= 38
      integer(kind=im), parameter :: nmol   = 7
! Use for 140 g-point model
      integer(kind=im), parameter :: ngptlw = 140
! Use for 256 g-point model
!      integer(kind=im), parameter :: ngptlw = 256

! Use for 140 g-point model
      integer(kind=im), parameter :: ng1  = 10
      integer(kind=im), parameter :: ng2  = 12
      integer(kind=im), parameter :: ng3  = 16
      integer(kind=im), parameter :: ng4  = 14
      integer(kind=im), parameter :: ng5  = 16
      integer(kind=im), parameter :: ng6  = 8
      integer(kind=im), parameter :: ng7  = 12
      integer(kind=im), parameter :: ng8  = 8
      integer(kind=im), parameter :: ng9  = 12
      integer(kind=im), parameter :: ng10 = 6
      integer(kind=im), parameter :: ng11 = 8
      integer(kind=im), parameter :: ng12 = 8
      integer(kind=im), parameter :: ng13 = 4
      integer(kind=im), parameter :: ng14 = 2
      integer(kind=im), parameter :: ng15 = 2
      integer(kind=im), parameter :: ng16 = 2

      integer(kind=im), parameter :: ngs1  = 10
      integer(kind=im), parameter :: ngs2  = 22
      integer(kind=im), parameter :: ngs3  = 38
      integer(kind=im), parameter :: ngs4  = 52
      integer(kind=im), parameter :: ngs5  = 68
      integer(kind=im), parameter :: ngs6  = 76
      integer(kind=im), parameter :: ngs7  = 88
      integer(kind=im), parameter :: ngs8  = 96
      integer(kind=im), parameter :: ngs9  = 108
      integer(kind=im), parameter :: ngs10 = 114
      integer(kind=im), parameter :: ngs11 = 122
      integer(kind=im), parameter :: ngs12 = 130
      integer(kind=im), parameter :: ngs13 = 134
      integer(kind=im), parameter :: ngs14 = 136
      integer(kind=im), parameter :: ngs15 = 138

! Use for 256 g-point model
!      integer(kind=im), parameter :: ng1  = 16
!      integer(kind=im), parameter :: ng2  = 16
!      integer(kind=im), parameter :: ng3  = 16
!      integer(kind=im), parameter :: ng4  = 16
!      integer(kind=im), parameter :: ng5  = 16
!      integer(kind=im), parameter :: ng6  = 16
!      integer(kind=im), parameter :: ng7  = 16
!      integer(kind=im), parameter :: ng8  = 16
!      integer(kind=im), parameter :: ng9  = 16
!      integer(kind=im), parameter :: ng10 = 16
!      integer(kind=im), parameter :: ng11 = 16
!      integer(kind=im), parameter :: ng12 = 16
!      integer(kind=im), parameter :: ng13 = 16
!      integer(kind=im), parameter :: ng14 = 16
!      integer(kind=im), parameter :: ng15 = 16
!      integer(kind=im), parameter :: ng16 = 16

!      integer(kind=im), parameter :: ngs1  = 16
!      integer(kind=im), parameter :: ngs2  = 32
!      integer(kind=im), parameter :: ngs3  = 48
!      integer(kind=im), parameter :: ngs4  = 64
!      integer(kind=im), parameter :: ngs5  = 80
!      integer(kind=im), parameter :: ngs6  = 96
!      integer(kind=im), parameter :: ngs7  = 112
!      integer(kind=im), parameter :: ngs8  = 128
!      integer(kind=im), parameter :: ngs9  = 144
!      integer(kind=im), parameter :: ngs10 = 160
!      integer(kind=im), parameter :: ngs11 = 176
!      integer(kind=im), parameter :: ngs12 = 192
!      integer(kind=im), parameter :: ngs13 = 208
!      integer(kind=im), parameter :: ngs14 = 224
!      integer(kind=im), parameter :: ngs15 = 240
!      integer(kind=im), parameter :: ngs16 = 256

      end module parrrtm
```

讲解：①**parkind** 模块：kind_ib/im/in 全取 kind(1) 原生整型；kind_rb 被改成 kind(1.0) **原生单精度**——原版 selected_real_kind(12) 双精度开关被 WRF 用 `#if 0` 关掉（性能换精度的全局决定，改回双精度要动这里且 sw 文件共用同一模块）；②**parrrtm**：mxlay=203 层上限、nbndlw=16 波段、mg=16（每波段原 g 点数）、**ngptlw=140**（缩减后 g 点总数，256 的备选注释保留）、ng1..ng16 每波段缩减后 g 点数（10/12/16/14/16/8/12/8/12/6/8/8/4/2/2/2，合计 140）、ngs1..ngs15 累积表——k 分布的「谱-统计」结构全部在这张参数表上。

四、WRF 入口 RRTMG_LWRAD

```
!file: phys/module_ra_rrtmg_lw.F    L11570-L11901
L11570     SUBROUTINE RRTMG_LWRAD(                                        &
L11571                         rthratenlw,                                 &
L11572                         rthratenlwc,                                &
L11573                         lwupt, lwuptc, lwuptcln, lwdnt, lwdntc, lwdntcln, &
L11574                         lwupb, lwupbc, lwupbcln, lwdnb, lwdnbc, lwdnbcln, &
L11575  !                      lwupflx, lwupflxc, lwdnflx, lwdnflxc,       &
L11576                         glw, olr, lwcf, emiss,                      &
L11577                         p8w, p3d, pi3d,                             &
L11578                         dz8w, tsk, t3d, t8w, rho3d, r, g,           &
L11579                         icloud, warm_rain, cldfra3d,                &
L11580                         cldovrlp,idcor,xlat,                        &
L11581                         lradius,iradius,                            &
L11582                         is_cammgmp_used,                            &
L11583                         f_ice_phy, f_rain_phy,                      &
L11584                         xland, xice, snow,                          &
L11585                         qv3d, qc3d, qr3d,                           &
L11586                         qi3d, qs3d, qg3d,                           &
L11587                         o3input, o33d,                              &
L11588                         f_qv, f_qc, f_qr, f_qi, f_qs, f_qg,         &
L11589                         re_cloud, re_ice, re_snow,                  &  ! G. Thompson
L11590                         has_reqc, has_reqi, has_reqs,               &  ! G. Thompson
L11591                         tauaerlw1,tauaerlw2,tauaerlw3,tauaerlw4,    & ! czhao
L11592                         tauaerlw5,tauaerlw6,tauaerlw7,tauaerlw8,    & ! czhao
L11593                         tauaerlw9,tauaerlw10,tauaerlw11,tauaerlw12,   & ! czhao
L11594                         tauaerlw13,tauaerlw14,tauaerlw15,tauaerlw16,   & ! czhao
L11595                         aer_ra_feedback,                            & !czhao
L11596  !jdfcz                 progn,prescribe,                            & !czhao
L11597                         progn,calc_clean_atm_diag,                  & !czhao
L11598                         qndrop3d,f_qndrop,                          & !czhao
L11599  !ccc added for time varying gases.
L11600                         yr,julian,ghg_input,                        &
L11601  !ccc
L11602                         mp_physics,                                 &
L11603                         ids,ide, jds,jde, kds,kde,                  &
L11604                         ims,ime, jms,jme, kms,kme,                  &
L11605                         its,ite, jts,jte, kts,kte,                  &
L11606                         lwupflx, lwupflxc, lwdnflx, lwdnflxc        &
L11607                                                                     )
L11608  !-------------------------------------------------------------------
L11609  !ccc To use clWRF time varying trace gases
L11610     USE MODULE_RA_CLWRF_SUPPORT, ONLY : read_CAMgases
L11611
L11612     IMPLICIT NONE
L11613  !-------------------------------------------------------------------
L11614     LOGICAL, INTENT(IN )      ::        warm_rain
L11615     LOGICAL, INTENT(IN )      ::   is_CAMMGMP_used ! Added for CAM5 RRTMG<->CAMMGMP
L11616  !
L11617     INTEGER, INTENT(IN )      ::        ids,ide, jds,jde, kds,kde, &
L11618                                         ims,ime, jms,jme, kms,kme, &
L11619                                         its,ite, jts,jte, kts,kte
L11620
L11621     INTEGER, INTENT(IN )      ::        ICLOUD, GHG_INPUT
L11622     INTEGER, INTENT(IN )      ::        MP_PHYSICS
L11623  !
L11624     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )                 , &
L11625           INTENT(IN   ) ::                                    dz8w, &
L11626                                                                t3d, &
L11627                                                                t8w, &
L11628                                                                p8w, &
L11629                                                                p3d, &
L11630                                                               pi3d, &
L11631                                                              rho3d
L11632
L11633     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )                 , &
L11634           INTENT(INOUT)  ::                                RTHRATENLW, &
L11635                                                           RTHRATENLWC
L11636
L11637     REAL, DIMENSION( ims:ime, jms:jme )                          , &
L11638           INTENT(INOUT)  ::                                     GLW, &
L11639                                                                 OLR, &
L11640                                                                LWCF
L11641
L11642     REAL, DIMENSION( ims:ime, jms:jme )                          , &
L11643           INTENT(IN   )  ::                                   EMISS, &
L11644                                                                 TSK
L11645
L11646     REAL, INTENT(IN  )   ::                                     R,G
L11647
L11648     REAL, DIMENSION( ims:ime, jms:jme )                          , &
L11649           INTENT(IN   )  ::                                   XLAND, &
L11650                                                                XICE, &
L11651                                                                SNOW
L11652  ! Added for time-varying trace gases.
L11653     INTEGER, INTENT(IN    ) ::                                    yr
L11654     REAL, INTENT(IN    ) ::                                  julian
L11655
L11656  !
L11657  ! Optional
L11658  !
L11659     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )                 , &
L11660           OPTIONAL                                               , &
L11661           INTENT(IN   ) ::                                       &
L11662                                                          CLDFRA3D, &
L11663                                                           LRADIUS, &
L11664                                                           IRADIUS, &
L11665
L11666                                                              QV3D, &
L11667                                                              QC3D, &
L11668                                                              QR3D, &
L11669                                                              QI3D, &
L11670                                                              QS3D, &
L11671                                                              QG3D, &
L11672                                                           QNDROP3D
L11673
```

```
!..Added by G. Thompson to couple cloud physics effective radii.
   REAL, DIMENSION(ims:ime,kms:kme,jms:jme), INTENT(IN)::         &
                                                        re_cloud, &
                                                          re_ice, &
                                                          re_snow
   INTEGER, INTENT(IN):: has_reqc, has_reqi, has_reqs

   real pi,third,relconst,lwpmin,rhoh2o

   REAL, DIMENSION( ims:ime, kms:kme, jms:jme )                 , &
         OPTIONAL                                               , &
         INTENT(IN   ) ::                                         &
                                                    F_ICE_PHY, &
                                                   F_RAIN_PHY

   LOGICAL, OPTIONAL, INTENT(IN)   ::                            &
                                    F_QV,F_QC,F_QR,F_QI,F_QS,F_QG,F_QNDROP
! Optional
   REAL, DIMENSION( ims:ime, kms:kme, jms:jme ), OPTIONAL ,      &
         INTENT(IN    ) :: tauaerlw1,tauaerlw2,tauaerlw3,tauaerlw4, & ! czhao
                           tauaerlw5,tauaerlw6,tauaerlw7,tauaerlw8, & ! czhao
                           tauaerlw9,tauaerlw10,tauaerlw11,tauaerlw12, & ! czhao
                           tauaerlw13,tauaerlw14,tauaerlw15,tauaerlw16

   INTEGER,    INTENT(IN  ), OPTIONAL   ::       aer_ra_feedback
!jdfcz   INTEGER,    INTENT(IN  ), OPTIONAL   ::       progn,prescribe
   INTEGER,    INTENT(IN  ), OPTIONAL   ::       progn
   INTEGER,    INTENT(IN  )             ::       calc_clean_atm_diag

!  Ozone
   REAL, DIMENSION( ims:ime, kms:kme, jms:jme )                 , &
         INTENT(INOUT) :: O33D
   INTEGER, INTENT(IN ) :: o3input

      real, parameter :: thresh=1.e-9
      real slope
      character(len=200) :: msg


! Top of atmosphere and surface longwave fluxes (W m-2)
   REAL, DIMENSION( ims:ime, jms:jme ),                          &
         OPTIONAL, INTENT(INOUT) ::                              &
                     LWUPT,LWUPTC,LWUPTCLN,LWDNT,LWDNTC,LWDNTCLN,&
                     LWUPB,LWUPBC,LWUPBCLN,LWDNB,LWDNBC,LWDNBCLN

! Layer longwave fluxes (including extra layer above model top)
! Vertical ordering is from bottom to top (W m-2)
   REAL, DIMENSION( ims:ime, kms:kme+2, jms:jme ),               &
         OPTIONAL, INTENT(OUT) ::                                &
                             LWUPFLX,LWUPFLXC,LWDNFLX,LWDNFLXC

!  LOCAL VARS

   REAL, DIMENSION( kts:kte+1 ) ::                                 Pw1D, &
                                                                   Tw1D

   REAL, DIMENSION( kts:kte ) ::                                 TTEN1D, &
                                                               CLDFRA1D, &
                                                                   DZ1D, &
                                                                    P1D, &
                                                                    T1D, &
                                                                   QV1D, &
                                                                   QC1D, &
                                                                   QR1D, &
                                                                   QI1D, &
                                                                  RHO1D, &
                                                                   QS1D, &
                                                                   QG1D, &
                                                                   O31D, &
                                                                 qndrop1d
!BSF: From eq. (5) on p. 2434 in McFarquhar & Heymsfield (1996)
       real, parameter :: re_50C=1250.0/9.917, re_40C=1250.0/9.337, &
                          re_30C=1250.0/9.208, re_20C=1250.0/9.387


! Added local arrays for RRTMG
    integer ::                                                     ncol, &
                                                                   nlay, &
                                                                   icld, &
                                                              cldovrlp, &
                                                                 idcor, &
                                                                juldat, &
                                                               inflglw, &
                                                              iceflglw, &
                                                              liqflglw
! Latitude
    real, dimension( ims:ime,jms:jme ), intent(in) ::              xlat     ! (latitude for cldovrlp=4 or 5)
    real                                           ::               lat

! Dimension with extra layer from model top to TOA
    real, dimension( 1, kts:nlayers+1 )  ::                        plev, &
                                                                   tlev
    real, dimension( 1, kts:nlayers )  ::                          play, &
                                                                   tlay, &
                                                                 h2ovmr, &
                                                                  o3vmr, &
                                                                 co2vmr, &
                                                                  o2vmr, &
                                                                 ch4vmr, &
                                                                 n2ovmr, &
                                                                cfc11vmr, &
                                                                cfc12vmr, &
                                                                cfc22vmr, &
                                                                 ccl4vmr
    real, dimension( kts:nlayers )  ::                             o3mmr
! Add height of each layer for exponential-random cloud overlap
! This will be derived below from the dz in each layer
    real, dimension( 1, kts:nlayers )  ::                            hgt
    real ::                                                       dzsum
! For old cloud property specification for rrtm_lw
```

```
L11784      real, dimension( kts:kte )  ::                          clwp, &
L11785                                                              ciwp, &
L11786                                                              cswp, &
L11787                                                              plwp, &
L11788                                                              piwp
L11789  ! Surface emissivity (for 16 LW spectral bands)
L11790      real, dimension( 1, nbndlw )  ::                        emis
L11791  ! Dimension with extra layer from model top to TOA,
L11792  ! though no clouds are allowed in extra layer
L11793      real, dimension( 1, kts:nlayers )  ::                  clwpth, &
L11794                                                             ciwpth, &
L11795                                                             cswpth, &
L11796                                                                rel, &
L11797                                                                rei, &
L11798                                                                res, &
L11799                                                            cldfrac, &
L11800                                                            relqmcl, &
L11801                                                            reicmcl, &
L11802                                                            resnmcl
L11803      real, dimension( nbndlw, 1, kts:nlayers )  ::         taucld
L11804      real, dimension( ngptlw, 1, kts:nlayers )  ::         cldfmcl, &
L11805                                                            clwpmcl, &
L11806                                                            ciwpmcl, &
L11807                                                            cswpmcl, &
L11808                                                            taucmcl
L11809      real, dimension( 1, kts:nlayers, nbndlw )  ::            tauaer
L11810
L11811  ! Output arrays contain extra layer from model top to TOA
L11812      real, dimension( 1, kts:nlayers+1 )  ::                    uflx, &
L11813                                                                 dflx, &
L11814                                                               uflxc, &
L11815                                                               dflxc, &
L11816                                                            uflxcln, &
L11817                                                            dflxcln
L11818
L11819      real, dimension( 1, kts:nlayers )  ::                      hr, &
L11820                                                                 hrc
L11821
L11822      real, dimension ( 1 ) ::                                  tsfc, &
L11823                                                                  ps
L11824      real ::                                                     ro, &
L11825                                                                  dz
L11826      real:: snow_mass_factor
L11827
L11828  !..We can use message interface regardless of what options are running,
L11829  !.. so let us ask for it here.
L11830        CHARACTER(LEN=256)                          :: message
L11831        LOGICAL, EXTERNAL                           :: wrf_dm_on_monitor
L11832
L11833  !ccc To add time-varying trace gases (CO2, N2O and CH4). Read the conc.  from file
L11834  ! then interpolate to date of run.
L11835        REAL(8)                                     :: co2, n2o, ch4, cfc11, cfc12
L11836
L11837  ! Set trace gas volume mixing ratios, 2005 values, IPCC (2007)
L11838  ! cfc-22 (169 ppt)
L11839      real :: cfc22
L11840      data cfc22 / 0.169e-9 /
L11841  ! ccl4 (93 ppt)
L11842      real :: ccl4
L11843      data ccl4 / 0.093e-9 /
L11844  ! Set oxygen volume mixing ratio (for o2mmr=0.23143)
L11845      real :: o2
L11846      data o2 / 0.209488 /
L11847
L11848      integer :: iplon, irng, permuteseed
L11849      integer :: nb
L11850
L11851  ! For old cloud property specification for rrtm_lw
L11852  ! Cloud and precipitation absorption coefficients
L11853      real :: abcw,abice,abrn,absn
L11854      data abcw /0.144/
L11855      data abice /0.0735/
L11856      data abrn /0.330e-3/
L11857      data absn /2.34e-3/
L11858
L11859  ! Molecular weights and ratios for converting mmr to vmr units
L11860  !    real :: amd       ! Effective molecular weight of dry air (g/mol)
L11861  !    real :: amw       ! Molecular weight of water vapor (g/mol)
L11862  !    real :: amo       ! Molecular weight of ozone (g/mol)
L11863  !    real :: amo2      ! Molecular weight of oxygen (g/mol)
L11864  ! Atomic weights for conversion from mass to volume mixing ratios
L11865  !    data amd   /  28.9660   /
L11866  !    data amw   /  18.0160   /
L11867  !    data amo   /  47.9998   /
L11868  !    data amo2  /  31.9999   /
L11869
L11870      real :: amdw     ! Molecular weight of dry air / water vapor
L11871      real :: amdo     ! Molecular weight of dry air / ozone
L11872      real :: amdo2    ! Molecular weight of dry air / oxygen
L11873      data amdw /  1.607793 /
L11874      data amdo /  0.603461 /
L11875      data amdo2 / 0.905190 /
L11876
L11877  !!
L11878      real, dimension( 1, 1:kte-kts+1 )  :: pdel         ! Layer pressure thickness (mb)
L11879
L11880      real, dimension(1, 1:kte-kts+1) ::   cicewp, &     ! in-cloud cloud ice water path
L11881                                           cliqwp, &     ! in-cloud cloud liquid water path
L11882                                           csnowp, &     ! in-cloud snow water path
L11883                                            reliq, &     ! effective drop radius (microns)
L11884                                            reice        ! effective ice crystal size (microns)
L11885      real, dimension(1, 1:kte-kts+1):: recloud1d, &
L11886                                         reice1d, &
L11887                                        resnow1d
L11888
L11889      real :: gliqwp, gicewp, gsnowp, gravmks
L11890
L11891  !
L11892  !    REAL   ::  TSFC,GLW0,OLR0,EMISS0,FP
```

```
L11893
L11894      real, dimension (1) :: landfrac, landm, snowh, icefrac
L11895
L11896      integer :: pcols, pver
L11897
L11898  !
L11899      INTEGER :: i,j,K, idx_rei
L11900      REAL :: corr
L11901      LOGICAL :: predicate
```

讲解：① 签名即功能清单：RTHRATENLW/C 晴空倾向、LWUPT/LWDNT/LWUPB/LWDNB 及其 C/CLN 三套通量诊断、LWUPFLX 系逐层通量（kte+2 维——模型顶外垫一层到 TOA）、tauaerlw1..16 十六个独立数组（WRF_CHEM 气溶胶反馈的逐波段 AOD）、ghg_input 时变气体、o3input/O33D、re_cloud/ice/snow 微物理直供有效半径、cldovrlp/idcor 云重叠选项；② 关键局部常数：**时变气体 2005 年 IPCC 默认值**（cfc22=0.169e-9、ccl4=0.093e-9、o2=0.209488；CO2/CH4/N2O 由 read_CAMgases 读文件）、mmr→vmr 分子量比（amdw=1.607793 等）；③ 老 rrtm 云吸收系数 abcw=0.144/abice=0.0735/abrn=0.330e-3/absn=2.34e-3（INFLAG 老口径备胎）；④**McFarquhar–Heymsfield (1996) 式 (5) 的冰有效半径对照点** re_50C=1250/9.917 等——温度分层查 rei 表的锚点。

```
!file: phys/module_ra_rrtmg_lw.F    L12598-L12834
L12598  ! Limit upper bound of reice for Fu ice parameterization and convert
L12599  ! from effective radius to generalized effective size (*1.0315; Fu, 1996)
L12600              if (iceflglw .eq. 3) then
L12601                 do k = kts, kte
L12602                    reice(ncol,k) = reice(ncol,k) * 1.0315
L12603                    reice(ncol,k) = min(140.0,reice(ncol,k))
L12604                 end do
L12605              endif
L12606  !if CAMMGMP is used, use output from CAMMGMP
L12607              if(is_CAMMGMP_used) then
L12608                 do k = kts, kte
L12609                    if ( qi1d(k) .gt. 1.e-20 .or. qs1d(k) .gt. 1.e-20) then
L12610                       reice(ncol,k) = iradius(i,k,j)
L12611                    else
L12612                       reice(ncol,k) = 25.
L12613                    end if
L12614                    reice(ncol,k) = max(5., min(140.0,reice(ncol,k)))
L12615                    if ( qc1d(k) .gt. 1.e-20) then
L12616                       reliq(ncol,k) = lradius(i,k,j)
L12617                    else
L12618                       reliq(ncol,k) = 10.
L12619                    end if
L12620                    reliq(ncol,k) = max(2.5, min(60.0,reliq(ncol,k)))
L12621                 enddo
L12622              endif
L12623
L12624
L12625  ! Set cloud physical property arrays
L12626              do k = kts, kte
L12627                 clwpth(ncol,k) = cliqwp(ncol,k)
L12628                 ciwpth(ncol,k) = cicewp(ncol,k)
L12629                 rel(ncol,k) = reliq(ncol,k)
L12630                 rei(ncol,k) = reice(ncol,k)
L12631              enddo
L12632
L12633  !Mukul
L12634              if (inflglw .eq. 5) then
L12635                 do k = kts, kte
L12636                    cswpth(ncol,k) = csnowp(ncol,k)
L12637                    res(ncol,k) = resnow1d(ncol,k)
L12638                 end do
L12639              else
L12640                 do k = kts, kte
L12641                    cswpth(ncol,k) = 0.
L12642                    res(ncol,k) = 10.
L12643                 end do
L12644              endif
L12645
L12646  ! Zero out cloud optical properties here; not used when passing physical properties
L12647  ! to radiation and taucld is calculated in radiation
L12648              do k = kts, kte
L12649                 do nb = 1, nbndlw
L12650                    taucld(nb,ncol,k) = 0.0
L12651                 enddo
L12652              enddo
L12653           endif
L12654
L12655  ! No clouds are allowed in the extra layer from model top to TOA
L12656           ! Steven Cavallo: Edited out for buffer adjustment below
L12657           if ( 1 == 0 ) then
L12658
L12659
L12660           clwpth(ncol,kte+1) = 0.
L12661           ciwpth(ncol,kte+1) = 0.
L12662           cswpth(ncol,kte+1) = 0.
L12663           rel(ncol,kte+1) = 10.
L12664           rei(ncol,kte+1) = 10.
L12665           res(ncol,kte+1) = 10.
L12666           cldfrac(ncol,kte+1) = 0.
L12667           do nb = 1, nbndlw
L12668              taucld(nb,ncol,kte+1) = 0.
L12669           enddo
L12670
L12671           endif
L12672
L12673           ! Buffer adjustment. Steven Cavallo December 2010
L12674           do k=kte+1,nlayers
L12675              clwpth(ncol,k) = 0.
L12676              ciwpth(ncol,k) = 0.
L12677              cswpth(ncol,k) = 0.
```

```
L12678             rel(ncol,k) = 10.
L12679             rei(ncol,k) = 10.
L12680             res(ncol,k) = 10.
L12681             cldfrac(ncol,k) = 0.
L12682             do nb = 1,nbndlw
L12683                taucld(nb,ncol,k) = 0.
L12684             enddo
L12685          enddo
L12686
L12687          iplon = 1
L12688          irng = 0
L12689          permuteseed = 150
L12690
L12691  ! Sub-column generator for McICA
L12692          lat=xlat(i,j) !retrieve scalar latitude for column calculation
L12693          call mcica_subcol_lw(iplon, ncol, nlay, icld, permuteseed, irng, play, &
L12694                       cldfrac, ciwpth, clwpth, cswpth, rei, rel, res, taucld, &
L12695                       hgt, idcor, juldat, lat, &
L12696                       cldfmcl, ciwpmcl, clwpmcl, cswpmcl, reicmcl, relqmcl, resnmcl, taucmcl)
L12697
L12698  !-----------------------------------------------------------------------
L12699  ! Aerosol optical depth, single scattering albedo and asymmetry parameter -czhao 03/2010
L12700  !-----------------------------------------------------------------------
L12701  ! Aerosol optical depth by layer for each RRTMG longwave band
L12702  ! No aerosols in layer above model top (kte+1)
L12703  ! Steven Cavallo: Upper bound of loop changed to nlayers from kte+1
L12704  !        do nb = 1, nbndlw
L12705  !           do k = kts, kte+1
L12706  !              tauaer(ncol,k,nb) = 0.
L12707  !           enddo
L12708  !        enddo
L12709
L12710  ! ... Aerosol effects. Added aerosol feedbacks from Chem , 03/2010 -czhao
L12711  !
L12712        do nb = 1, nbndlw
L12713        do k = kts,nlayers
L12714           tauaer(ncol,k,nb) = 0.
L12715        end do
L12716        end do
L12717
L12718  #if ( WRF_CHEM == 1 )
L12719     IF ( AER_RA_FEEDBACK == 1) then
L12720  !     do nb = 1, nbndlw
L12721        do k = kts,kte      !wig
L12722          if(tauaerlw1(i,k,j).gt.thresh .and. tauaerlw16(i,k,j).gt.thresh) then
L12723            tauaer(ncol,k,1)=tauaerlw1(i,k,j)
L12724            tauaer(ncol,k,2)=tauaerlw2(i,k,j)
L12725            tauaer(ncol,k,3)=tauaerlw3(i,k,j)
L12726            tauaer(ncol,k,4)=tauaerlw4(i,k,j)
L12727            tauaer(ncol,k,5)=tauaerlw5(i,k,j)
L12728            tauaer(ncol,k,6)=tauaerlw6(i,k,j)
L12729            tauaer(ncol,k,7)=tauaerlw7(i,k,j)
L12730            tauaer(ncol,k,8)=tauaerlw8(i,k,j)
L12731            tauaer(ncol,k,9)=tauaerlw9(i,k,j)
L12732            tauaer(ncol,k,10)=tauaerlw10(i,k,j)
L12733            tauaer(ncol,k,11)=tauaerlw11(i,k,j)
L12734            tauaer(ncol,k,12)=tauaerlw12(i,k,j)
L12735            tauaer(ncol,k,13)=tauaerlw13(i,k,j)
L12736            tauaer(ncol,k,14)=tauaerlw14(i,k,j)
L12737            tauaer(ncol,k,15)=tauaerlw15(i,k,j)
L12738            tauaer(ncol,k,16)=tauaerlw16(i,k,j)
L12739          endif
L12740        enddo ! k
L12741  !     end do ! nb
L12742
L12743  !wig beg
L12744        do nb = 1, nbndlw
L12745           slope = 0.  !use slope as a sum holder
L12746           do k = kts,kte
L12747              slope = slope + tauaer(ncol,k,nb)
L12748           end do
L12749           if( slope < 0. ) then
L12750              write(msg,'("ERROR: Negative total lw optical depth of ",f8.2," at point i,j,nb=",3i5)') slope,i,j,nb
L12751              call wrf_error_fatal(msg)
L12752           else if( slope > 5. ) then
L12753              call wrf_message("--------------------------")
L12754              write(msg,'("WARNING: Large total lw optical depth of ",f8.2," at point i,j,nb=",3i5)') slope,i,j,nb
L12755              call wrf_message(msg)
L12756
L12757              call wrf_message("Diagnostics 1: k, tauaerlw1, tauaerlw16")
L12758              do k=kts,kte
L12759                 write(msg,'(i4,2f8.2)') k, tauaerlw1(i,k,j), tauaerlw16(i,k,j)
L12760                 call wrf_message(msg)
L12761              end do
L12762              call wrf_message("--------------------------")
L12763           endif
L12764        enddo  ! nb
L12765        endif  ! aer_ra_feedback
L12766  #endif
L12767
L12768  ! Call RRTMG longwave radiation model
L12769           call rrtmg_lw &
L12770              (ncol    ,nlay    ,icld    , &
L12771               play    ,plev    ,tlay    ,tlev    ,tsfc    , &
L12772               h2ovmr  ,o3vmr   ,co2vmr  ,ch4vmr  ,n2ovmr  ,o2vmr , &
L12773               cfc11vmr,cfc12vmr,cfc22vmr,ccl4vmr ,emis    , &
L12774               inflglw ,iceflglw,liqflglw,cldfmcl , &
L12775               taucmcl ,ciwpmcl ,clwpmcl ,cswpmcl, reicmcl ,relqmcl ,resnmcl , &
L12776               tauaer  , &
L12777               uflx    ,dflx    ,hr      ,uflxc   ,dflxc,  hrc, &
L12778               uflxcln ,dflxcln, calc_clean_atm_diag )
L12779
L12780  ! Output downard surface flux, and outgoing longwave flux and cloud forcing
L12781  ! at the top of atmosphere (W/m2)
L12782           glw(i,j) = dflx(1,1)
L12783  !         olr(i,j) = uflx(1,kte+2)
L12784  !         lwcf(i,j) = uflxc(1,kte+2) - uflx(1,kte+2)
L12785  ! Steven Cavallo: Changed OLR to be valid at the top of atmosphere instead
```

```
L12786  ! of top of model.  Dec 2010.
L12787           olr(i,j) = uflx(1,nlayers+1)
L12788           lwcf(i,j) = uflxc(1,nlayers+1) - uflx(1,nlayers+1)
L12789
L12790           if (present(lwupt)) then
L12791  ! Output up and down toa fluxes for total and clear sky
L12792  ! nlayers+1 represents value at 0 mb
L12793              lwupt(i,j)     = uflx(1,nlayers+1)
L12794              lwuptc(i,j)    = uflxc(1,nlayers+1)
L12795              lwdnt(i,j)     = dflx(1,nlayers+1)
L12796              lwdntc(i,j)    = dflxc(1,nlayers+1)
L12797  ! Output up and down surface fluxes for total and clear sky
L12798              lwupb(i,j)     = uflx(1,1)
L12799              lwupbc(i,j)    = uflxc(1,1)
L12800              lwdnb(i,j)     = dflx(1,1)
L12801              lwdnbc(i,j)    = dflxc(1,1)
L12802              if(calc_clean_atm_diag .gt. 0)then
L12803  ! Output up and down toa fluxes for clean sky
L12804                  lwuptcln(i,j)  = uflxcln(1,nlayers+1)
L12805                  lwdntcln(i,j)  = dflxcln(1,nlayers+1)
L12806  ! Output up and down surface fluxes for clean sky
L12807                  lwupbcln(i,j)  = uflxcln(1,1)
L12808                  lwdnbcln(i,j)  = dflxcln(1,1)
L12809              end if
L12810           endif
L12811
L12812  ! Output up and down layer fluxes for total and clear sky.
L12813  ! Vertical ordering is from bottom to top in units of W m-2.
L12814           if ( present (lwupflx) ) then
L12815           do k=kts,kte+2
L12816              lwupflx(i,k,j)  = uflx(1,k)
L12817              lwupflxc(i,k,j) = uflxc(1,k)
L12818              lwdnflx(i,k,j)  = dflx(1,k)
L12819              lwdnflxc(i,k,j) = dflxc(1,k)
L12820           enddo
L12821           endif
L12822
L12823  ! Output heating rate tendency; convert heating rate from K/d to K/s
L12824  ! Heating rate arrays are ordered vertically from bottom to top here.
L12825           do k=kts,kte
L12826              tten1d(k) = hr(ncol,k)/86400.
L12827              rthratenlw(i,k,j) = tten1d(k)/pi3d(i,k,j)
L12828              tten1d(k) = hrc(ncol,k)/86400.
L12829              rthratenlwc(i,k,j) = tten1d(k)/pi3d(i,k,j)
L12830           enddo
L12831
L12832  !
L12833        end do i_loop
L12834     end do j_loop
```

讲解：主入口尾段五件事：①**Fu (1996) 口径换算**——iceflglw=3 时冰有效半径乘 1.0315 变广义有效尺寸 dge 并钳位 [5,140] μm；CAMMGMP 在场时直接取 iradius/lradius（无冰/水时 25/10 μm 兜底）；②inflglw=5 的雪路径（cswpth/res）只有新版开、否则清零；③ **模型顶外垫层不允许有云**（Steven Cavallo 2010-12 的 buffer adjustment——kte+1 到 nlayers 全清，云/降水只进模式层）；④**mcica_subcol_lw 调用**：iplon=1（WRF 逐列单列调）、permuteseed=150、irng=0（KISSVEC 随机数）——McICA 的次列展开在这里完成后才调 rrtmg_lw；⑤WRF_CHEM 的 tauaerlw1..16 装载（负总光学深度 FATAL、超 5 警告打印诊断）；⑥**rrtmg_lw 调用与回写**：GLW=dflx(1,1)（地面下行长波）、**OLR 取 uflx(1,nlayers+1)——TOA 而非模型顶**（Cavallo 修正，注释保留老口径）；加热率换算 hr/86400 转 K/s、再除 pi3d 转位温倾向——radiation_driver 的 RTHRATENLW=RTHRATEN 语义在此成形。

五、方案核心 rrtmg_lw（全文）

```
!file: phys/module_ra_rrtmg_lw.F    L10694-L11183    (subroutine rrtmg_lw)
L10694        subroutine rrtmg_lw &
L10695             (ncol    ,nlay    ,icld    , &
L10696              play    ,plev    ,tlay    ,tlev    ,tsfc    , &
L10697              h2ovmr  ,o3vmr   ,co2vmr  ,ch4vmr  ,n2ovmr  ,o2vmr , &
L10698              cfc11vmr,cfc12vmr,cfc22vmr,ccl4vmr ,emis    , &
L10699              inflglw ,iceflglw,liqflglw,cldfmcl , &
L10700              taucmcl ,ciwpmcl ,clwpmcl , cswpmcl ,reicmcl ,relqmcl , resnmcl , &
L10701              tauaer  , &
L10702              uflx    ,dflx    ,hr      ,uflxc   ,dflxc,  hrc, &
L10703              uflxcln ,dflxcln, calc_clean_atm_diag )
L10704
L10705  ! -------- Description --------
L10706
L10707  ! This program is the driver subroutine for RRTMG_LW, the AER LW radiation
L10708  ! model for application to GCMs, that has been adapted from RRTM_LW for
L10709  ! improved efficiency.
L10710  !
L10711  ! NOTE: The call to RRTMG_LW_INI should be moved to the GCM initialization
L10712  !  area, since this has to be called only once.
L10713  !
L10714  ! This routine:
L10715  !    a) calls INATM to read in the atmospheric profile from GCM;
L10716  !       all layering in RRTMG is ordered from surface to toa.
L10717  !    b) calls CLDPRMC to set cloud optical depth for McICA based
L10718  !       on input cloud properties
L10719  !    c) calls SETCOEF to calculate various quantities needed for
L10720  !       the radiative transfer algorithm
L10721  !    d) calls TAUMOL to calculate gaseous optical depths for each
L10722  !       of the 16 spectral bands
```

```
!    e) calls RTRNMC (for both clear and cloudy profiles) to perform the
!       radiative transfer calculation using McICA, the Monte-Carlo
!       Independent Column Approximation, to represent sub-grid scale
!       cloud variability
!    f) passes the necessary fluxes and cooling rates back to GCM
!
! Two modes of operation are possible:
!     The mode is chosen by using either rrtmg_lw.nomcica.f90 (to not use
!     McICA) or rrtmg_lw.f90 (to use McICA) to interface with a GCM.
!
!    1) Standard, single forward model calculation (imca = 0)
!    2) Monte Carlo Independent Column Approximation (McICA, Pincus et al.,
!       JC, 2003) method is applied to the forward model calculation (imca = 1)
!
! This call to RRTMG_LW must be preceeded by a call to the module
!     mcica_subcol_gen_lw.f90 to run the McICA sub-column cloud generator,
!     which will provide the cloud physical or cloud optical properties
!     on the RRTMG quadrature point (ngpt) dimension.
!     Two random number generators are available for use when imca = 1.
!     This is chosen by setting flag irnd on input to mcica_subcol_gen_lw.
!     1) KISSVEC (irnd = 0)
!     2) Mersenne-Twister (irnd = 1)
!
! Two methods of cloud property input are possible:
!     Cloud properties can be input in one of two ways (controlled by input
!     flags inflglw, iceflglw, and liqflglw; see text file rrtmg_lw_instructions
!     and subroutine rrtmg_lw_cldprop.f90 for further details):
!
!    1) Input cloud fraction and cloud optical depth directly (inflglw = 0)
!    2) Input cloud fraction and cloud physical properties (inflglw = 1 or 2);
!       cloud optical properties are calculated by cldprop or cldprmc based
!       on input settings of iceflglw and liqflglw.  Ice particle size provided
!       must be appropriately defined for the ice parameterization selected.
!
! One method of aerosol property input is possible:
!     Aerosol properties can be input in only one way (controlled by input
!     flag iaer; see text file rrtmg_lw_instructions for further details):
!
!    1) Input aerosol optical depth directly by layer and spectral band (iaer=10);
!       band average optical depth at the mid-point of each spectral band.
!       RRTMG_LW currently treats only aerosol absorption;
!       scattering capability is not presently available.
!
!
! ------- Modifications -------
!
! This version of RRTMG_LW has been modified from RRTM_LW to use a reduced
! set of g-points for application to GCMs.
!
!-- Original version (derived from RRTM_LW), reduction of g-points, other
!   revisions for use with GCMs.
!     1999: M. J. Iacono, AER, Inc.
!-- Adapted for use with NCAR/CAM.
!     May 2004: M. J. Iacono, AER, Inc.
!-- Revised to add McICA capability.
!     Nov 2005: M. J. Iacono, AER, Inc.
!-- Conversion to F90 formatting for consistency with rrtmg_sw.
!     Feb 2007: M. J. Iacono, AER, Inc.
!-- Modifications to formatting to use assumed-shape arrays.
!     Aug 2007: M. J. Iacono, AER, Inc.
!-- Modified to add longwave aerosol absorption.
!     Apr 2008: M. J. Iacono, AER, Inc.

! --------- Modules ----------

      use parrrtm, only : nbndlw, ngptlw, maxxsec, mxmol
      use rrlw_con, only: fluxfac, heatfac, oneminus, pi
      use rrlw_wvn, only: ng, ngb, nspa, nspb, wavenum1, wavenum2, delwave

! ------- Declarations -------

! ----- Input -----
      integer(kind=im), intent(in) :: ncol            ! Number of horizontal columns
      integer(kind=im), intent(in) :: nlay            ! Number of model layers
      integer(kind=im), intent(inout) :: icld         ! Cloud overlap method
                                                      !    0: Clear only
                                                      !    1: Random
                                                      !    2: Maximum/random
                                                      !    3: Maximum
                                                      !    4: Exponential
                                                      !    5: Exponential/random
      real(kind=rb), intent(in) :: play(:,:)          ! Layer pressures (hPa, mb)
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: plev(:,:)          ! Interface pressures (hPa, mb)
                                                      !    Dimensions: (ncol,nlay+1)
      real(kind=rb), intent(in) :: tlay(:,:)          ! Layer temperatures (K)
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: tlev(:,:)          ! Interface temperatures (K)
                                                      !    Dimensions: (ncol,nlay+1)
      real(kind=rb), intent(in) :: tsfc(:)            ! Surface temperature (K)
                                                      !    Dimensions: (ncol)
      real(kind=rb), intent(in) :: h2ovmr(:,:)        ! H2O volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: o3vmr(:,:)         ! O3 volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: co2vmr(:,:)        ! CO2 volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: ch4vmr(:,:)        ! Methane volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: n2ovmr(:,:)        ! Nitrous oxide volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: o2vmr(:,:)         ! Oxygen volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: cfc11vmr(:,:)      ! CFC11 volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: cfc12vmr(:,:)      ! CFC12 volume mixing ratio
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: cfc22vmr(:,:)      ! CFC22 volume mixing ratio
```

```
L10831                                                  !    Dimensions: (ncol,nlay)
L10832        real(kind=rb), intent(in) :: ccl4vmr(:,:) ! CCL4 volume mixing ratio
L10833                                                  !    Dimensions: (ncol,nlay)
L10834        real(kind=rb), intent(in) :: emis(:,:)    ! Surface emissivity
L10835                                                  !    Dimensions: (ncol,nbndlw)
L10836
L10837        integer(kind=im), intent(in) :: inflglw   ! Flag for cloud optical properties
L10838        integer(kind=im), intent(in) :: iceflglw  ! Flag for ice particle specification
L10839        integer(kind=im), intent(in) :: liqflglw  ! Flag for liquid droplet specification
L10840
L10841        real(kind=rb), intent(in) :: cldfmcl(:,:,:) ! Cloud fraction
L10842                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10843        real(kind=rb), intent(in) :: ciwpmcl(:,:,:) ! In-cloud ice water path (g/m2)
L10844                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10845        real(kind=rb), intent(in) :: clwpmcl(:,:,:) ! In-cloud liquid water path (g/m2)
L10846                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10847        real(kind=rb), intent(in) :: cswpmcl(:,:,:) ! In-cloud snow water path (g/m2)
L10848                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10849        real(kind=rb), intent(in) :: reicmcl(:,:) ! Cloud ice particle effective size (microns)
L10850                                                  !    Dimensions: (ncol,nlay)
L10851                                                  ! specific definition of reicmcl depends on setting of iceflglw:
L10852                                                  ! iceflglw = 0: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L10853                                                  !              r_ec must be >= 10.0 microns
L10854                                                  ! iceflglw = 1: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L10855                                                  !              r_ec range is limited to 13.0 to 130.0 microns
L10856                                                  ! iceflglw = 2: ice effective radius, r_k, (Key, Streamer Ref.
↪  Manual, 1996)
L10857                                                  !              r_k range is limited to 5.0 to 131.0 microns
L10858                                                  ! iceflglw = 3: generalized effective size, dge, (Fu, 1996),
L10859                                                  !              dge range is limited to 5.0 to 140.0 microns
L10860                                                  !              [dge = 1.0315 * r_ec]
L10861        real(kind=rb), intent(in) :: relqmcl(:,:) ! Cloud water drop effective radius (microns)
L10862                                                  !    Dimensions: (ncol,nlay)
L10863        real(kind=rb), intent(in) :: resnmcl(:,:) ! Snow effective radius (microns)
L10864                                                  !    Dimensions: (ncol,nlay)
L10865        real(kind=rb), intent(in) :: taucmcl(:,:,:) ! In-cloud optical depth
L10866                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10867  !      real(kind=rb), intent(in) :: ssacmcl(:,:,:) ! In-cloud single scattering albedo
L10868                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10869                                                  !   for future expansion
L10870                                                  !   lw scattering not yet available
L10871  !      real(kind=rb), intent(in) :: asmcmcl(:,:,:) ! In-cloud asymmetry parameter
L10872                                                  !    Dimensions: (ngptlw,ncol,nlay)
L10873                                                  !   for future expansion
L10874                                                  !   lw scattering not yet available
L10875        real(kind=rb), intent(in) :: tauaer(:,:,:) ! aerosol optical depth
L10876                                                  !   at mid-point of LW spectral bands
L10877                                                  !    Dimensions: (ncol,nlay,nbndlw)
L10878  !      real(kind=rb), intent(in) :: ssaaer(:,:,:) ! aerosol single scattering albedo
L10879                                                  !    Dimensions: (ncol,nlay,nbndlw)
L10880                                                  !   for future expansion
L10881                                                  !   (lw aerosols/scattering not yet available)
L10882  !      real(kind=rb), intent(in) :: asmaer(:,:,:) ! aerosol asymmetry parameter
L10883                                                  !    Dimensions: (ncol,nlay,nbndlw)
L10884                                                  !   for future expansion
L10885                                                  !   (lw aerosols/scattering not yet available)
L10886        integer, intent(in) :: calc_clean_atm_diag ! Control for clean air diagnositic calls for WRF-Chem
L10887
L10888  ! ----- Output -----
L10889
L10890        real(kind=rb), intent(out) :: uflx(:,:)   ! Total sky longwave upward flux (W/m2)
L10891                                                  !    Dimensions: (ncol,nlay+1)
L10892        real(kind=rb), intent(out) :: dflx(:,:)   ! Total sky longwave downward flux (W/m2)
L10893                                                  !    Dimensions: (ncol,nlay+1)
L10894        real(kind=rb), intent(out) :: hr(:,:)     ! Total sky longwave radiative heating rate (K/d)
L10895                                                  !    Dimensions: (ncol,nlay)
L10896        real(kind=rb), intent(out) :: uflxc(:,:)  ! Clear sky longwave upward flux (W/m2)
L10897                                                  !    Dimensions: (ncol,nlay+1)
L10898        real(kind=rb), intent(out) :: dflxc(:,:)  ! Clear sky longwave downward flux (W/m2)
L10899                                                  !    Dimensions: (ncol,nlay+1)
L10900        real(kind=rb), intent(out) :: hrc(:,:)    ! Clear sky longwave radiative heating rate (K/d)
L10901                                                  !    Dimensions: (ncol,nlay)
L10902        real(kind=rb), intent(out) :: uflxcln(:,:) ! Clean sky longwave upward flux (W/m2)
L10903                                                  !    Dimensions: (ncol,nlay+1)
L10904        real(kind=rb), intent(out) :: dflxcln(:,:) ! Clean sky longwave downward flux (W/m2)
L10905                                                  !    Dimensions: (ncol,nlay+1)
L10906
L10907  ! ----- Local -----
L10908
L10909  ! Control
L10910        integer(kind=im) :: nlayers             ! total number of layers
L10911        integer(kind=im) :: istart              ! beginning band of calculation
L10912        integer(kind=im) :: iend                ! ending band of calculation
L10913        integer(kind=im) :: iout                ! output option flag (inactive)
L10914        integer(kind=im) :: iaer                ! aerosol option flag
L10915        integer(kind=im) :: iplon               ! column loop index
L10916        integer(kind=im) :: imca                ! flag for mcica [0=off, 1=on]
L10917        integer(kind=im) :: ims                 ! value for changing mcica permute seed
L10918        integer(kind=im) :: k                   ! layer loop index
L10919        integer(kind=im) :: ig                  ! g-point loop index
L10920
L10921  ! Atmosphere
L10922        real(kind=rb) :: pavel(nlay+1)          ! layer pressures (mb)
L10923        real(kind=rb) :: tavel(nlay+1)          ! layer temperatures (K)
L10924        real(kind=rb) :: pz(0:nlay+1)           ! level (interface) pressures (hPa, mb)
L10925        real(kind=rb) :: tz(0:nlay+1)           ! level (interface) temperatures (K)
L10926        real(kind=rb) :: tbound                 ! surface temperature (K)
L10927        real(kind=rb) :: coldry(nlay+1)         ! dry air column density (mol/cm2)
L10928        real(kind=rb) :: wbrodl(nlay+1)         ! broadening gas column density (mol/cm2)
L10929        real(kind=rb) :: wkl(mxmol,nlay+1)      ! molecular amounts (mol/cm-2)
L10930        real(kind=rb) :: wx(maxxsec,nlay+1)     ! cross-section amounts (mol/cm-2)
L10931        real(kind=rb) :: pwvcm                  ! precipitable water vapor (cm)
L10932        real(kind=rb) :: semiss(nbndlw)         ! lw surface emissivity
L10933        real(kind=rb) :: fracs(nlay+1,ngptlw)   !
```

```
L10934        real(kind=rb) :: taug(nlay+1,ngptlw)    ! gaseous optical depths
L10935        real(kind=rb) :: taut(nlay+1,ngptlw)    ! gaseous + aerosol optical depths
L10936
L10937        real(kind=rb) :: taua(nlay+1,nbndlw)    ! aerosol optical depth
L10938  !      real(kind=rb) :: ssaa(nlay+1,nbndlw)   ! aerosol single scattering albedo
L10939                                                !   for future expansion
L10940                                                !   (lw aerosols/scattering not yet available)
L10941  !      real(kind=rb) :: asma(nlay+1,nbndlw)   ! aerosol asymmetry parameter
L10942                                                !   for future expansion
L10943                                                !   (lw aerosols/scattering not yet available)
L10944
L10945  ! Atmosphere - setcoef
L10946        integer(kind=im) :: laytrop             ! tropopause layer index
L10947        integer(kind=im) :: jp(nlay+1)          ! lookup table index
L10948        integer(kind=im) :: jt(nlay+1)          ! lookup table index
L10949        integer(kind=im) :: jt1(nlay+1)         ! lookup table index
L10950        real(kind=rb) :: planklay(nlay+1,nbndlw)!
L10951        real(kind=rb) :: planklev(0:nlay+1,nbndlw)!
L10952        real(kind=rb) :: plankbnd(nbndlw)       !
L10953
L10954        real(kind=rb) :: colh2o(nlay+1)         ! column amount (h2o)
L10955        real(kind=rb) :: colco2(nlay+1)         ! column amount (co2)
L10956        real(kind=rb) :: colo3(nlay+1)          ! column amount (o3)
L10957        real(kind=rb) :: coln2o(nlay+1)         ! column amount (n2o)
L10958        real(kind=rb) :: colco(nlay+1)          ! column amount (co)
L10959        real(kind=rb) :: colch4(nlay+1)         ! column amount (ch4)
L10960        real(kind=rb) :: colo2(nlay+1)          ! column amount (o2)
L10961        real(kind=rb) :: colbrd(nlay+1)         ! column amount (broadening gases)
L10962
L10963        integer(kind=im) :: indself(nlay+1)
L10964        integer(kind=im) :: indfor(nlay+1)
L10965        real(kind=rb) :: selffac(nlay+1)
L10966        real(kind=rb) :: selffrac(nlay+1)
L10967        real(kind=rb) :: forfac(nlay+1)
L10968        real(kind=rb) :: forfrac(nlay+1)
L10969
L10970        integer(kind=im) :: indminor(nlay+1)
L10971        real(kind=rb) :: minorfrac(nlay+1)
L10972        real(kind=rb) :: scaleminor(nlay+1)
L10973        real(kind=rb) :: scaleminorn2(nlay+1)
L10974
L10975        real(kind=rb) :: &                      !
L10976                         fac00(nlay+1), fac01(nlay+1), &
L10977                         fac10(nlay+1), fac11(nlay+1)
L10978        real(kind=rb) :: &                      !
L10979                         rat_h2oco2(nlay+1),rat_h2oco2_1(nlay+1), &
L10980                         rat_h2oo3(nlay+1),rat_h2oo3_1(nlay+1), &
L10981                         rat_h2on2o(nlay+1),rat_h2on2o_1(nlay+1), &
L10982                         rat_h2och4(nlay+1),rat_h2och4_1(nlay+1), &
L10983                         rat_n2oco2(nlay+1),rat_n2oco2_1(nlay+1), &
L10984                         rat_o3co2(nlay+1),rat_o3co2_1(nlay+1)
L10985
L10986  ! Atmosphere/clouds - cldprop
L10987        integer(kind=im) :: ncbands             ! number of cloud spectral bands
L10988        integer(kind=im) :: inflag              ! flag for cloud property method
L10989        integer(kind=im) :: iceflag             ! flag for ice cloud properties
L10990        integer(kind=im) :: liqflag             ! flag for liquid cloud properties
L10991
L10992  ! Atmosphere/clouds - cldprmc [mcica]
L10993        real(kind=rb) :: cldfmc(ngptlw,nlay+1)  ! cloud fraction [mcica]
L10994        real(kind=rb) :: ciwpmc(ngptlw,nlay+1)  ! in-cloud ice water path [mcica]
L10995        real(kind=rb) :: clwpmc(ngptlw,nlay+1)  ! in-cloud liquid water path [mcica]
L10996        real(kind=rb) :: cswpmc(ngptlw,nlay+1)  ! in-cloud snow path [mcica]
L10997        real(kind=rb) :: relqmc(nlay+1)         ! liquid particle effective radius (microns)
L10998        real(kind=rb) :: reicmc(nlay+1)         ! ice particle effective size (microns)
L10999        real(kind=rb) :: resnmc(nlay+1)         ! snow particle effective size (microns)
L11000        real(kind=rb) :: taucmc(ngptlw,nlay+1)  ! in-cloud optical depth [mcica]
L11001  !      real(kind=rb) :: ssacmc(ngptlw,nlay+1) ! in-cloud single scattering albedo [mcica]
L11002                                                !   for future expansion
L11003                                                !   (lw scattering not yet available)
L11004  !      real(kind=rb) :: asmcmc(ngptlw,nlay+1) ! in-cloud asymmetry parameter [mcica]
L11005                                                !   for future expansion
L11006                                                !   (lw scattering not yet available)
L11007
L11008  ! Output
L11009        real(kind=rb) :: totuflux(0:nlay+1)     ! upward longwave flux (w/m2)
L11010        real(kind=rb) :: totdflux(0:nlay+1)     ! downward longwave flux (w/m2)
L11011        real(kind=rb) :: fnet(0:nlay+1)         ! net longwave flux (w/m2)
L11012        real(kind=rb) :: htr(0:nlay+1)          ! longwave heating rate (k/day)
L11013        real(kind=rb) :: totuclfl(0:nlay+1)     ! clear sky upward longwave flux (w/m2)
L11014        real(kind=rb) :: totdclfl(0:nlay+1)     ! clear sky downward longwave flux (w/m2)
L11015        real(kind=rb) :: fnetc(0:nlay+1)        ! clear sky net longwave flux (w/m2)
L11016        real(kind=rb) :: htrc(0:nlay+1)         ! clear sky longwave heating rate (k/day)
L11017        real(kind=rb) :: totuclnlfl(0:nlay+1)   ! clean sky upward longwave flux (w/m2)
L11018        real(kind=rb) :: totdclnlfl(0:nlay+1)   ! clean sky downward longwave flux (w/m2)
L11019        real(kind=rb) :: fnetcln(0:nlay+1)      ! clean sky net longwave flux (w/m2)
L11020        real(kind=rb) :: htrcln(0:nlay+1)       ! clean sky longwave heating rate (k/day)
L11021
L11022  !
L11023  ! Initializations
L11024
L11025  !jm not thread safe      oneminus = 1._rb - 1.e-6_rb
L11026  !jm not thread safe      pi = 2._rb * asin(1._rb)
L11027  !jm not thread safe      fluxfac = pi * 2.e4_rb                 ! orig:   fluxfac = pi * 2.d4
L11028        istart = 1
L11029        iend = 16
L11030        iout = 0
L11031        ims = 1
L11032
L11033  ! Set imca to select calculation type:
L11034  !  imca = 0, use standard forward model calculation
L11035  !  imca = 1, use McICA for Monte Carlo treatment of sub-grid cloud variability
L11036
L11037  ! *** This version uses McICA (imca = 1) ***
L11038
L11039  ! Set icld to select of clear or cloud calculation and cloud overlap method
L11040  ! icld = 0, clear only
```

```
! icld = 1, with clouds using random cloud overlap
! icld = 2, with clouds using maximum/random cloud overlap
! icld = 3, with clouds using maximum cloud overlap (McICA only)
! icld = 4, with clouds using exponential cloud overlap (McICA only)
! icld = 5, with clouds using exponential/random cloud overlap (McICA only)

! Set iaer to select aerosol option
! iaer = 0, no aerosols
! icld = 10, input total aerosol optical depth (tauaer) directly
      iaer = 10

! Call model and data initialization, compute lookup tables, perform
! reduction of g-points from 256 to 140 for input absorption coefficient
! data and other arrays.
!
! In a GCM this call should be placed in the model initialization
! area, since this has to be called only once.
!      call rrtmg_lw_ini(cpdair)

!  This is the main longitude/column loop within RRTMG.
      do iplon = 1, ncol

!  Prepare atmospheric profile from GCM for use in RRTMG, and define
!  other input parameters.

         call inatm (iplon, nlay, icld, iaer, &
              play, plev, tlay, tlev, tsfc, h2ovmr, &
              o3vmr, co2vmr, ch4vmr, n2ovmr, o2vmr, cfc11vmr, cfc12vmr, &
              cfc22vmr, ccl4vmr, emis, inflglw, iceflglw, liqflglw, &
              cldfmcl, taucmcl, ciwpmcl, clwpmcl, cswpmcl, reicmcl, relqmcl, resnmcl, tauaer, &
              nlayers, pavel, pz, tavel, tz, tbound, semiss, coldry, &
              wkl, wbrodl, wx, pwvcm, inflag, iceflag, liqflag, &
              cldfmc, taucmc, ciwpmc, clwpmc, cswpmc, reicmc, relqmc, resnmc, taua)

!  For cloudy atmosphere, use cldprop to set cloud optical properties based on
!  input cloud physical properties.  Select method based on choices described
!  in cldprop.  Cloud fraction, water path, liquid droplet and ice particle
!  effective radius must be passed into cldprop.  Cloud fraction and cloud
!  optical depth are transferred to rrtmg_lw arrays in cldprop.

         call cldprmc(nlayers, inflag, iceflag, liqflag, cldfmc, ciwpmc, &
                      clwpmc, cswpmc, reicmc, relqmc, resnmc, ncbands, taucmc)

! Calculate information needed by the radiative transfer routine
! that is specific to this atmosphere, especially some of the
! coefficients and indices needed to compute the optical depths
! by interpolating data from stored reference atmospheres.

         call setcoef(nlayers, istart, pavel, tavel, tz, tbound, semiss, &
                      coldry, wkl, wbrodl, &
                      laytrop, jp, jt, jt1, planklay, planklev, plankbnd, &
                      colh2o, colco2, colo3, coln2o, colco, colch4, colo2, &
                      colbrd, fac00, fac01, fac10, fac11, &
                      rat_h2oco2, rat_h2oco2_1, rat_h2oo3, rat_h2oo3_1, &
                      rat_h2on2o, rat_h2on2o_1, rat_h2och4, rat_h2och4_1, &
                      rat_n2oco2, rat_n2oco2_1, rat_o3co2, rat_o3co2_1, &
                      selffac, selffrac, indself, forfac, forfrac, indfor, &
                      minorfrac, scaleminor, scaleminorn2, indminor)

!  Calculate the gaseous optical depths and Planck fractions for
!  each longwave spectral band.

         call taumol(nlayers, pavel, wx, coldry, &
                     laytrop, jp, jt, jt1, planklay, planklev, plankbnd, &
                     colh2o, colco2, colo3, coln2o, colco, colch4, colo2, &
                     colbrd, fac00, fac01, fac10, fac11, &
                     rat_h2oco2, rat_h2oco2_1, rat_h2oo3, rat_h2oo3_1, &
                     rat_h2on2o, rat_h2on2o_1, rat_h2och4, rat_h2och4_1, &
                     rat_n2oco2, rat_n2oco2_1, rat_o3co2, rat_o3co2_1, &
                     selffac, selffrac, indself, forfac, forfrac, indfor, &
                     minorfrac, scaleminor, scaleminorn2, indminor, &
                     fracs, taug)


! Combine gaseous and aerosol optical depths, if aerosol active
         if (iaer .eq. 0) then
            do k = 1, nlayers
               do ig = 1, ngptlw
                  taut(k,ig) = taug(k,ig)
               enddo
            enddo
         elseif (iaer .eq. 10) then
            do k = 1, nlayers
               do ig = 1, ngptlw
                  taut(k,ig) = taug(k,ig) + taua(k,ngb(ig))
               enddo
            enddo
         endif

! Call the radiative transfer routine.
! Either routine can be called to do clear sky calculation.  If clouds
! are present, then select routine based on cloud overlap assumption
! to be used.  Clear sky calculation is done simultaneously.
! For McICA, RTRNMC is called for clear and cloudy calculations.

#if (WRF_CHEM == 1)
        ! Call the radiative transfer routine for "clean" sky first,
        ! passing taug rather than taut so we have no aerosol influence.
        ! We will keep totuclnlfl, totdclnlfl, fnetcln, and htrcln,
        ! and then overwrite the rest with the second call to rtrnmc.
         if(calc_clean_atm_diag .gt. 0)then
             call rtrnmc(nlayers, istart, iend, iout, pz, semiss, ncbands, &
                         cldfmc, taucmc, planklay, planklev, plankbnd, &
                         pwvcm, fracs, taug, &
                         totuclnlfl, totdclnlfl, fnetcln, htrcln, &
                         totuclfl, totdclfl, fnetc, htrc )
         else
            do k = 0, nlayers
                totuclnlfl(k) = 0.0
```

```
L11150                 totdclnlfl(k) = 0.0
L11151              end do
L11152           end if
L11153  #else
L11154           do k = 0, nlayers
L11155              totuclnlfl(k) = 0.0
L11156              totdclnlfl(k) = 0.0
L11157           end do
L11158  #endif
L11159           call rtrnmc(nlayers, istart, iend, iout, pz, semiss, ncbands, &
L11160                       cldfmc, taucmc, planklay, planklev, plankbnd, &
L11161                       pwvcm, fracs, taut, &
L11162                       totuflux, totdflux, fnet, htr, &
L11163                       totuclfl, totdclfl, fnetc, htrc )
L11164
L11165  !  Transfer up and down fluxes and heating rate to output arrays.
L11166  !  Vertical indexing goes from bottom to top; reverse here for GCM if necessary.
L11167
L11168           do k = 0, nlayers
L11169              uflx(iplon,k+1) = totuflux(k)
L11170              dflx(iplon,k+1) = totdflux(k)
L11171              uflxc(iplon,k+1) = totuclfl(k)
L11172              dflxc(iplon,k+1) = totdclfl(k)
L11173              uflxcln(iplon,k+1) = totuclnlfl(k)
L11174              dflxcln(iplon,k+1) = totdclnlfl(k)
L11175           enddo
L11176           do k = 0, nlayers-1
L11177              hr(iplon,k+1) = htr(k)
L11178              hrc(iplon,k+1) = htrc(k)
L11179           enddo
L11180
L11181        enddo
L11182
L11183        end subroutine rrtmg_lw
```

讲解：AER 原版核心的全景注释（L10705–L11021）本身就是文献：**icld 云重叠六选**（0 晴空/1 随机/2 最大随机/3 最大/4 指数/5 指数随机——McICA 下才有 3-5）、iceflglw 四选（0/1 Ebert-Curry 1992、2 Key STREAMER、3 Fu 1996 广义有效尺寸）、liqflglw（0 CCM3、1 Hu-Stamnes 1993）、LW 气溶胶**只吸收不散射**（ssaaer/asma 注释「for future expansion」）。代码体：imca=1 固定 McICA；iaer=10 固定直接输入逐波段 AOD；逐列循环五步——inatm（WRF 自上而下 →RRTMG 自下而上换序、算 coldry/wkl 分子柱量）、cldprmc（物理属性 → 光学深度）、setcoef（laytrop 对流层顶判定 + jp/jt/jt1 查表索引 + fac00..11 双线性插值系数 + 水汽自/外连续谱自变量）、taumol（16 波段并行：taug 光学深度 + fracs 普朗克分数）、rtrnmc（**调两次**——WRF_CHEM 时先传 taug 无气溶胶算 clean 版、再传 taut=taug+taua 算 total 版）；输出回排 uflx/dflx/hr 三组 ×（total/clear/clean）。

六、McICA 云光学 cldprmc（全文）

```
!file: phys/module_ra_rrtmg_lw.F    L2764-L3025    (subroutine cldprmc)
L02764        subroutine cldprmc(nlayers, inflag, iceflag, liqflag, cldfmc, &
L02765                           ciwpmc, clwpmc, cswpmc, reicmc, relqmc, resnmc, ncbands, taucmc)
L02766  ! ----------------------------------------------------------------------------
L02767
L02768  ! Purpose:  Compute the cloud optical depth(s) for each cloudy layer.
L02769
L02770  ! ------- Input -------
L02771
L02772        integer(kind=im), intent(in) :: nlayers         ! total number of layers
L02773        integer(kind=im), intent(in) :: inflag          ! see definitions
L02774        integer(kind=im), intent(in) :: iceflag         ! see definitions
L02775        integer(kind=im), intent(in) :: liqflag         ! see definitions
L02776
L02777        real(kind=rb), intent(in) :: cldfmc(:,:)        ! cloud fraction [mcica]
L02778                                                        !    Dimensions: (ngptlw,nlayers)
L02779        real(kind=rb), intent(in) :: ciwpmc(:,:)        ! cloud ice water path [mcica]
L02780                                                        !    Dimensions: (ngptlw,nlayers)
L02781        real(kind=rb), intent(in) :: clwpmc(:,:)        ! cloud liquid water path [mcica]
L02782                                                        !    Dimensions: (ngptlw,nlayers)
L02783        real(kind=rb), intent(in) :: cswpmc(:,:)        ! cloud snow path [mcica]
L02784                                                        !    Dimensions: (ngptlw,nlayers)
L02785        real(kind=rb), intent(in) :: relqmc(:)          ! liquid particle effective radius (microns)
L02786                                                        !    Dimensions: (nlayers)
L02787        real(kind=rb), intent(in) :: reicmc(:)          ! ice particle effective radius (microns)
L02788                                                        !    Dimensions: (nlayers)
L02789        real(kind=rb), intent(in) :: resnmc(:)          ! snow particle effective radius (microns)
L02790                                                        !    Dimensions: (nlayers)
L02791                                                        ! specific definition of reicmc depends on setting of iceflag:
L02792                                                        ! iceflag = 0: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L02793                                                        !             r_ec must be >= 10.0 microns
L02794                                                        ! iceflag = 1: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L02795                                                        !             r_ec range is limited to 13.0 to 130.0 microns
L02796                                                        ! iceflag = 2: ice effective radius, r_k, (Key, Streamer Ref.
↪  Manual, 1996)
L02797                                                        !             r_k range is limited to 5.0 to 131.0 microns
L02798                                                        ! iceflag = 3: generalized effective size, dge, (Fu, 1996),
L02799                                                        !             dge range is limited to 5.0 to 140.0 microns
L02800                                                        !             [dge = 1.0315 * r_ec]
L02801
L02802  ! ------- Output -------
L02803
L02804        integer(kind=im), intent(out) :: ncbands        ! number of cloud spectral bands
L02805        real(kind=rb), intent(inout) :: taucmc(:,:)     ! cloud optical depth [mcica]
L02806                                                        !    Dimensions: (ngptlw,nlayers)
```

```
L02807
L02808  ! ------- Local -------
L02809
L02810        integer(kind=im) :: lay                         ! Layer index
L02811        integer(kind=im) :: ib                          ! spectral band index
L02812        integer(kind=im) :: ig                          ! g-point interval index
L02813        integer(kind=im) :: index
L02814        integer(kind=im) :: icb(nbndlw)
L02815
L02816        real(kind=rb) :: abscoice(ngptlw)               ! ice absorption coefficients
L02817        real(kind=rb) :: abscoliq(ngptlw)               ! liquid absorption coefficients
L02818        real(kind=rb) :: abscosno(ngptlw)               ! snow absorption coefficients
L02819        real(kind=rb) :: cwp                            ! cloud water path
L02820        real(kind=rb) :: radice                         ! cloud ice effective size (microns)
L02821        real(kind=rb) :: factor                         !
L02822        real(kind=rb) :: fint                           !
L02823        real(kind=rb) :: radliq                         ! cloud liquid droplet radius (microns)
L02824        real(kind=rb) :: radsno                         ! cloud snow effective size (microns)
L02825        real(kind=rb), parameter :: eps = 1.e-6_rb      ! epsilon
L02826        real(kind=rb), parameter :: cldmin = 1.e-20_rb  ! minimum value for cloud quantities
L02827        character*80 errmess
L02828
L02829  ! ------- Definitions -------
L02830
L02831  !     Explanation of the method for each value of INFLAG.  Values of
L02832  !     0 or 1 for INFLAG do not distingish being liquid and ice clouds.
L02833  !     INFLAG = 2 does distinguish between liquid and ice clouds, and
L02834  !     requires further user input to specify the method to be used to
L02835  !     compute the aborption due to each.
L02836  !     INFLAG = 0:  For each cloudy layer, the cloud fraction and (gray)
L02837  !                  optical depth are input.
L02838  !     INFLAG = 1:  For each cloudy layer, the cloud fraction and cloud
L02839  !                  water path (g/m2) are input.  The (gray) cloud optical
L02840  !                  depth is computed as in CCM2.
L02841  !     INFLAG = 2:  For each cloudy layer, the cloud fraction, cloud
L02842  !                  water path (g/m2), and cloud ice fraction are input.
L02843  !       ICEFLAG = 0:  The ice effective radius (microns) is input and the
L02844  !                     optical depths due to ice clouds are computed as in CCM3.
L02845  !       ICEFLAG = 1:  The ice effective radius (microns) is input and the
L02846  !                     optical depths due to ice clouds are computed as in
L02847  !                     Ebert and Curry, JGR, 97, 3831-3836 (1992).  The
L02848  !                     spectral regions in this work have been matched with
L02849  !                     the spectral bands in RRTM to as great an extent
L02850  !                     as possible:
L02851  !                     E&C 1      IB = 5      RRTM bands 9-16
L02852  !                     E&C 2      IB = 4      RRTM bands 6-8
L02853  !                     E&C 3      IB = 3      RRTM bands 3-5
L02854  !                     E&C 4      IB = 2      RRTM band 2
L02855  !                     E&C 5      IB = 1      RRTM band 1
L02856  !       ICEFLAG = 2:  The ice effective radius (microns) is input and the
L02857  !                     optical properties due to ice clouds are computed from
L02858  !                     the optical properties stored in the RT code,
L02859  !                     STREAMER v3.0 (Reference: Key. J., Streamer
L02860  !                     User's Guide, Cooperative Institute for
L02861  !                     Meteorological Satellite Studies, 2001, 96 pp.).
L02862  !                     Valid range of values for re are between 5.0 and
L02863  !                     131.0 micron.
L02864  !       ICEFLAG = 3: The ice generalized effective size (dge) is input
L02865  !                    and the optical properties, are calculated as in
L02866  !                    Q. Fu, J. Climate, (1998). Q. Fu provided high resolution
L02867  !                    tables which were appropriately averaged for the
L02868  !                    bands in RRTM_LW.  Linear interpolation is used to
L02869  !                    get the coefficients from the stored tables.
L02870  !                    Valid range of values for dge are between 5.0 and
L02871  !                    140.0 micron.
L02872  !       LIQFLAG = 0:  The optical depths due to water clouds are computed as
L02873  !                     in CCM3.
L02874  !       LIQFLAG = 1:  The water droplet effective radius (microns) is input
L02875  !                     and the optical depths due to water clouds are computed
L02876  !                     as in Hu and Stamnes, J., Clim., 6, 728-742, (1993).
L02877  !                     The values for absorption coefficients appropriate for
L02878  !                     the spectral bands in RRTM have been obtained for a
L02879  !                     range of effective radii by an averaging procedure
L02880  !                     based on the work of J. Pinto (private communication).
L02881  !                     Linear interpolation is used to get the absorption
L02882  !                     coefficients for the input effective radius.
L02883
L02884        data icb /1,2,3,3,3,4,4,4,5, 5, 5, 5, 5, 5, 5, 5/
L02885
L02886  !jm not thread safe      hvrclc = '$Revision: 1.8 $'
L02887
L02888        ncbands = 1
L02889
L02890  ! This initialization is done in rrtmg_lw_subcol.F90.
L02891  !      do lay = 1, nlayers
L02892  !         do ig = 1, ngptlw
L02893  !            taucmc(ig,lay) = 0.0_rb
L02894  !         enddo
L02895  !      enddo
L02896
L02897  ! Main layer loop
L02898        do lay = 1, nlayers
L02899
L02900          do ig = 1, ngptlw
L02901            cwp = ciwpmc(ig,lay) + clwpmc(ig,lay) + cswpmc(ig,lay)
L02902            if (cldfmc(ig,lay) .ge. cldmin .and. &
L02903               (cwp .ge. cldmin .or. taucmc(ig,lay) .ge. cldmin)) then
L02904
L02905  ! Ice clouds and water clouds combined.
L02906              if (inflag .eq. 0) then
L02907  ! Cloud optical depth already defined in taucmc, return to main program
L02908                return
L02909
L02910              elseif(inflag .eq. 1) then
L02911                  stop 'INFLAG = 1 OPTION NOT AVAILABLE WITH MCICA'
L02912  !               cwp = ciwpmc(ig,lay) + clwpmc(ig,lay)
L02913  !               taucmc(ig,lay) = abscld1 * cwp
L02914
L02915  ! Separate treatement of ice clouds and water clouds.
```

```
L02916              elseif(inflag .ge. 2) then
L02917                 radice = reicmc(lay)
L02918
L02919  ! Calculation of absorption coefficients due to ice clouds.
L02920                 if ((ciwpmc(ig,lay)+cswpmc(ig,lay)) .eq. 0.0_rb) then
L02921                    abscoice(ig) = 0.0_rb
L02922                    abscosno(ig) = 0.0_rb
L02923
L02924                 elseif (iceflag .eq. 0) then
L02925                    if (radice .lt. 10.0_rb) stop 'ICE RADIUS TOO SMALL'
L02926                    abscoice(ig) = absice0(1) + absice0(2)/radice
L02927                    abscosno(ig) = 0.0_rb
L02928
L02929                 elseif (iceflag .eq. 1) then
L02930                    if (radice .lt. 13.0_rb .or. radice .gt. 130._rb) stop &
L02931                        'ICE RADIUS OUT OF BOUNDS'
L02932                    ncbands = 5
L02933                    ib = icb(ngb(ig))
L02934                    abscoice(ig) = absice1(1,ib) + absice1(2,ib)/radice
L02935                    abscosno(ig) = 0.0_rb
L02936
L02937  ! For iceflag=2 option, ice particle effective radius is limited to 5.0 to 131.0 microns
L02938
L02939                 elseif (iceflag .eq. 2) then
L02940                    if (radice .lt. 5.0_rb .or. radice .gt. 131.0_rb) stop 'ICE RADIUS OUT OF BOUNDS'
L02941                       ncbands = 16
L02942                       factor = (radice - 2._rb)/3._rb
L02943                       index = int(factor)
L02944                       if (index .eq. 43) index = 42
L02945                       fint = factor - float(index)
L02946                       ib = ngb(ig)
L02947                       abscoice(ig) = &
L02948                           absice2(index,ib) + fint * &
L02949                           (absice2(index+1,ib) - (absice2(index,ib)))
L02950                       abscosno(ig) = 0.0_rb
L02951
L02952  ! For iceflag=3 option, ice particle generalized effective size is limited to 5.0 to 140.0 microns
L02953
L02954                 elseif (iceflag .ge. 3) then
L02955                    if (radice .lt. 5.0_rb .or. radice .gt. 140.0_rb) then
L02956                          write(errmess,'(A,i5,i5,f8.2,f8.2)' )          &
L02957                 'ERROR: ICE GENERALIZED EFFECTIVE SIZE OUT OF BOUNDS'   &
L02958                 ,ig, lay, ciwpmc(ig,lay), radice
L02959                          call wrf_error_fatal(errmess)
L02960                       end if
L02961                       ncbands = 16
L02962                       factor = (radice - 2._rb)/3._rb
L02963                       index = int(factor)
L02964                       if (index .eq. 46) index = 45
L02965                       fint = factor - float(index)
L02966                       ib = ngb(ig)
L02967                       abscoice(ig) = &
L02968                           absice3(index,ib) + fint * &
L02969                           (absice3(index+1,ib) - (absice3(index,ib)))
L02970                       abscosno(ig) = 0.0_rb
L02971
L02972                 endif
L02973
L02974  !..Incorporate additional effects due to snow.
L02975                 if (cswpmc(ig,lay).gt.0.0_rb .and. iceflag .eq. 5) then
L02976                    radsno = resnmc(lay)
L02977                    if (radsno .lt. 5.0_rb .or. radsno .gt. 140.0_rb) then
L02978                          write(errmess,'(A,i5,i5,f8.2,f8.2)' )          &
L02979                 'ERROR: SNOW GENERALIZED EFFECTIVE SIZE OUT OF BOUNDS'   &
L02980                 ,ig, lay, cswpmc(ig,lay), radsno
L02981                          call wrf_error_fatal(errmess)
L02982                       end if
L02983                       ncbands = 16
L02984                       factor = (radsno - 2._rb)/3._rb
L02985                       index = int(factor)
L02986                       if (index .eq. 46) index = 45
L02987                       fint = factor - float(index)
L02988                       ib = ngb(ig)
L02989                       abscosno(ig) = &
L02990                           absice3(index,ib) + fint * &
L02991                           (absice3(index+1,ib) - (absice3(index,ib)))
L02992                 endif
L02993
L02994
L02995  ! Calculation of absorption coefficients due to water clouds.
L02996                 if (clwpmc(ig,lay) .eq. 0.0_rb) then
L02997                    abscoliq(ig) = 0.0_rb
L02998
L02999                 elseif (liqflag .eq. 0) then
L03000                     abscoliq(ig) = absliq0
L03001
L03002                 elseif (liqflag .eq. 1) then
L03003                    radliq = relqmc(lay)
L03004                    if (radliq .lt. 2.5_rb .or. radliq .gt. 60._rb) stop &
L03005                        'LIQUID EFFECTIVE RADIUS OUT OF BOUNDS'
L03006                    index = int(radliq - 1.5_rb)
L03007                    if (index .eq. 0) index = 1
L03008                    if (index .eq. 58) index = 57
L03009                    fint = radliq - 1.5_rb - float(index)
L03010                    ib = ngb(ig)
L03011                    abscoliq(ig) = &
L03012                          absliq1(index,ib) + fint * &
L03013                          (absliq1(index+1,ib) - (absliq1(index,ib)))
L03014                 endif
L03015
L03016                 taucmc(ig,lay) = ciwpmc(ig,lay) * abscoice(ig) + &
L03017                                  clwpmc(ig,lay) * abscoliq(ig) + &
L03018                                  cswpmc(ig,lay) * abscosno(ig)
L03019
L03020              endif
L03021           endif
L03022           enddo
```

```
L03023        enddo
L03024
L03025        end subroutine cldprmc
```

讲解：云光学深度组装，ngptlw×nlayers 的 g 点维循环：①inflag=0 直接返回（光学深度外部给定——WRF 主线不走）；inflag=1 直接 stop（MCICA 不支持）；**inflag≥2 分冰/水/雪三源**：冰按 iceflag 四选——0 CCM3 单参数 absice0=(0.005,1.0)/re（≥10 μm）、1 Ebert-Curry 五谱段 icb 映射（ngb→E&C 谱区）、2 Key/STREAMER 43 档表插值（5–131 μm）、3 Fu 1998 高分辨率表 45 档插值（5–140 μm，越界 wrf_error_fatal）；雪附加项只在 iceflag=5（WRF 扩展口径）生效；水按 liqflag——0 常数 absliq0、1 Hu-Stamnes 58 档插值（2.5–60 μm）；②**taucmc=ciwp · abscoice+clwp · abscoliq+cswp · abscosnow**——冰/水/雪各自水路 × 质量吸收系数线性叠加；③ncbands 记录实际用到几个波段（E&C=5、Key/Fu=16——rtrnmc 的云循环界）。stop 语句是老代码风格（越界直接停机），iceflag=3 分支被 WRF 改成 wrf_error_fatal。

七、气体吸收 taumol 与波段样本 taugb1

```
!file: phys/module_ra_rrtmg_lw.F    L4824-L5068
L04824        subroutine taumol(nlayers, pavel, wx, coldry, &
L04825                          laytrop, jp, jt, jt1, planklay, planklev, plankbnd, &
L04826                          colh2o, colco2, colo3, coln2o, colco, colch4, colo2, &
L04827                          colbrd, fac00, fac01, fac10, fac11, &
L04828                          rat_h2oco2, rat_h2oco2_1, rat_h2oo3, rat_h2oo3_1, &
L04829                          rat_h2on2o, rat_h2on2o_1, rat_h2och4, rat_h2och4_1, &
L04830                          rat_n2oco2, rat_n2oco2_1, rat_o3co2, rat_o3co2_1, &
L04831                          selffac, selffrac, indself, forfac, forfrac, indfor, &
L04832                          minorfrac, scaleminor, scaleminorn2, indminor, &
L04833                          fracs, taug)
L04834  !----------------------------------------------------------------------------
L04835
L04836  ! ****************************************************************************
L04837  ! *                                                                          *
L04838  ! *                  Optical depths developed for the                        *
L04839  ! *                                                                          *
L04840  ! *                RAPID RADIATIVE TRANSFER MODEL (RRTM)                     *
L04841  ! *                                                                          *
L04842  ! *                                                                          *
L04843  ! *            ATMOSPHERIC AND ENVIRONMENTAL RESEARCH, INC.                  *
L04844  ! *                        131 HARTWELL AVENUE                               *
L04845  ! *                        LEXINGTON, MA 02421                               *
L04846  ! *                                                                          *
L04847  ! *                                                                          *
L04848  ! *                           ELI J. MLAWER                                  *
L04849  ! *                         JENNIFER DELAMERE                                *
L04850  ! *                         STEVEN J. TAUBMAN                                *
L04851  ! *                         SHEPARD A. CLOUGH                                *
L04852  ! *                                                                          *
L04853  ! *                                                                          *
L04854  ! *                                                                          *
L04855  ! *                                                                          *
L04856  ! *                       email:  mlawer@aer.com                             *
L04857  ! *                       email:  jdelamer@aer.com                           *
L04858  ! *                                                                          *
L04859  ! *        The authors wish to acknowledge the contributions of the          *
L04860  ! *        following people:  Karen Cady-Pereira, Patrick D. Brown,          *
L04861  ! *        Michael J. Iacono, Ronald E. Farren, Luke Chen, Robert Bergstrom. *
L04862  ! *                                                                          *
L04863  ! ****************************************************************************
L04864  ! *                                                                          *
L04865  ! *  Revision for g-point reduction: Michael J. Iacono, AER, Inc.            *
L04866  ! *                                                                          *
L04867  ! ****************************************************************************
L04868  ! *     TAUMOL                                                               *
L04869  ! *                                                                          *
L04870  ! *     This file contains the subroutines TAUGBn (where n goes from         *
L04871  ! *     1 to 16).  TAUGBn calculates the optical depths and Planck fractions *
L04872  ! *     per g-value and layer for band n.                                    *
L04873  ! *                                                                          *
L04874  ! *  Output:  optical depths (unitless)                                      *
L04875  ! *           fractions needed to compute Planck functions at every layer    *
L04876  ! *               and g-value                                                *
L04877  ! *                                                                          *
L04878  ! *     COMMON /TAUGCOM/  TAUG(MXLAY,MG)                                     *
L04879  ! *     COMMON /PLANKG/   FRACS(MXLAY,MG)                                    *
L04880  ! *                                                                          *
L04881  ! *  Input                                                                   *
L04882  ! *                                                                          *
L04883  ! *     COMMON /FEATURES/ NG(NBANDS),NSPA(NBANDS),NSPB(NBANDS)               *
L04884  ! *     COMMON /PRECISE/  ONEMINUS                                           *
L04885  ! *     COMMON /PROFILE/  NLAYERS,PAVEL(MXLAY),TAVEL(MXLAY),                 *
L04886  ! *     &                 PZ(0:MXLAY),TZ(0:MXLAY)                            *
L04887  ! *     COMMON /PROFDATA/ LAYTROP,                                           *
L04888  ! *    &                  COLH2O(MXLAY),COLCO2(MXLAY),COLO3(MXLAY),          *
L04889  ! *    &                  COLN2O(MXLAY),COLCO(MXLAY),COLCH4(MXLAY),          *
L04890  ! *    &                  COLO2(MXLAY)                                       *
L04891  ! *     COMMON /INTFAC/   FAC00(MXLAY),FAC01(MXLAY),                         *
L04892  ! *    &                  FAC10(MXLAY),FAC11(MXLAY)                          *
L04893  ! *     COMMON /INTIND/   JP(MXLAY),JT(MXLAY),JT1(MXLAY)                     *
L04894  ! *     COMMON /SELF/     SELFFAC(MXLAY), SELFFRAC(MXLAY), INDSELF(MXLAY)    *
L04895  ! *                                                                          *
L04896  ! *     Description:                                                         *
L04897  ! *     NG(IBAND) - number of g-values in band IBAND                         *
L04898  ! *     NSPA(IBAND) - for the lower atmosphere, the number of reference      *
L04899  ! *                   atmospheres that are stored for band IBAND per         *
L04900  ! *                   pressure level and temperature.  Each of these         *
L04901  ! *                   atmospheres has different relative amounts of the      *
```

```
L04902  ! *                  key species for the band (i.e. different binary          *
L04903  ! *                  species parameters).                                     *
L04904  ! *     NSPB(IBAND) - same for upper atmosphere                               *
L04905  ! *     ONEMINUS - since problems are caused in some cases by interpolation    *
L04906  ! *                parameters equal to or greater than 1, for these cases     *
L04907  ! *                these parameters are set to this value, slightly < 1.      *
L04908  ! *     PAVEL - layer pressures (mb)                                          *
L04909  ! *     TAVEL - layer temperatures (degrees K)                                *
L04910  ! *     PZ - level pressures (mb)                                             *
L04911  ! *     TZ - level temperatures (degrees K)                                   *
L04912  ! *     LAYTROP - layer at which switch is made from one combination of       *
L04913  ! *               key species to another                                      *
L04914  ! *     COLH2O, COLCO2, COLO3, COLN2O, COLCH4 - column amounts of water       *
L04915  ! *               vapor,carbon dioxide, ozone, nitrous ozide, methane,        *
L04916  ! *               respectively (molecules/cm**2)                              *
L04917  ! *     FACij(LAY) - for layer LAY, these are factors that are needed to      *
L04918  ! *                  compute the interpolation factors that multiply the      *
L04919  ! *                  appropriate reference k-values.  A value of 0 (1) for    *
L04920  ! *                  i,j indicates that the corresponding factor multiplies   *
L04921  ! *                  reference k-value for the lower (higher) of the two      *
L04922  ! *                  appropriate temperatures, and altitudes, respectively.   *
L04923  ! *     JP - the index of the lower (in altitude) of the two appropriate      *
L04924  ! *          reference pressure levels needed for interpolation               *
L04925  ! *     JT, JT1 - the indices of the lower of the two appropriate reference   *
L04926  ! *               temperatures needed for interpolation (for pressure         *
L04927  ! *               levels JP and JP+1, respectively)                           *
L04928  ! *     SELFFAC - scale factor needed for water vapor self-continuum, equals  *
L04929  ! *               (water vapor density)/(atmospheric density at 296K and      *
L04930  ! *               1013 mb)                                                    *
L04931  ! *     SELFFRAC - factor needed for temperature interpolation of reference   *
L04932  ! *                water vapor self-continuum data                            *
L04933  ! *     INDSELF - index of the lower of the two appropriate reference         *
L04934  ! *               temperatures needed for the self-continuum interpolation    *
L04935  ! *     FORFAC  - scale factor needed for water vapor foreign-continuum.      *
L04936  ! *     FORFRAC - factor needed for temperature interpolation of reference    *
L04937  ! *               water vapor foreign-continuum data                          *
L04938  ! *     INDFOR  - index of the lower of the two appropriate reference         *
L04939  ! *               temperatures needed for the foreign-continuum interpolation *
L04940  ! *                                                                           *
L04941  ! *  Data input                                                              *
L04942  ! *     COMMON /Kn/ KA(NSPA(n),5,13,MG), KB(NSPB(n),5,13:59,MG), SELFREF(10,MG),*
L04943  ! *                 FORREF(4,MG), KA_M'MGAS', KB_M'MGAS'                      *
L04944  ! *        (note:  n is the band number,'MGAS' is the species name of the minor *
L04945  ! *         gas)                                                              *
L04946  ! *                                                                           *
L04947  ! *     Description:                                                          *
L04948  ! *     KA - k-values for low reference atmospheres (key-species only)        *
L04949  ! *          (units: cm**2/molecule)                                          *
L04950  ! *     KB - k-values for high reference atmospheres (key-species only)       *
L04951  ! *          (units: cm**2/molecule)                                          *
L04952  ! *     KA_M'MGAS' - k-values for low reference atmosphere minor species      *
L04953  ! *          (units: cm**2/molecule)                                          *
L04954  ! *     KB_M'MGAS' - k-values for high reference atmosphere minor species     *
L04955  ! *          (units: cm**2/molecule)                                          *
L04956  ! *     SELFREF - k-values for water vapor self-continuum for reference       *
L04957  ! *               atmospheres (used below LAYTROP)                            *
L04958  ! *               (units: cm**2/molecule)                                     *
L04959  ! *     FORREF  - k-values for water vapor foreign-continuum for reference    *
L04960  ! *               atmospheres (used below/above LAYTROP)                      *
L04961  ! *               (units: cm**2/molecule)                                     *
L04962  ! *                                                                           *
L04963  ! *     DIMENSION ABSA(65*NSPA(n),MG), ABSB(235*NSPB(n),MG)                   *
L04964  ! *     EQUIVALENCE (KA,ABSA),(KB,ABSB)                                       *
L04965  ! *                                                                           *
L04966  !***************************************************************************
L04967
L04968  ! ------- Declarations -------
L04969
L04970  ! ----- Input -----
L04971        integer(kind=im), intent(in) :: nlayers         ! total number of layers
L04972        real(kind=rb), intent(in) :: pavel(:)           ! layer pressures (mb)
L04973                                                        !    Dimensions: (nlayers)
L04974        real(kind=rb), intent(in) :: wx(:,:)            ! cross-section amounts (mol/cm2)
L04975                                                        !    Dimensions: (maxxsec,nlayers)
L04976        real(kind=rb), intent(in) :: coldry(:)          ! column amount (dry air)
L04977                                                        !    Dimensions: (nlayers)
L04978
L04979        integer(kind=im), intent(in) :: laytrop         ! tropopause layer index
L04980        integer(kind=im), intent(in) :: jp(:)           !
L04981                                                        !    Dimensions: (nlayers)
L04982        integer(kind=im), intent(in) :: jt(:)           !
L04983                                                        !    Dimensions: (nlayers)
L04984        integer(kind=im), intent(in) :: jt1(:)          !
L04985                                                        !    Dimensions: (nlayers)
L04986        real(kind=rb), intent(in) :: planklay(:,:)      !
L04987                                                        !    Dimensions: (nlayers,nbndlw)
L04988        real(kind=rb), intent(in) :: planklev(0:,:)     !
L04989                                                        !    Dimensions: (nlayers,nbndlw)
L04990        real(kind=rb), intent(in) :: plankbnd(:)        !
L04991                                                        !    Dimensions: (nbndlw)
L04992
L04993        real(kind=rb), intent(in) :: colh2o(:)          ! column amount (h2o)
L04994                                                        !    Dimensions: (nlayers)
L04995        real(kind=rb), intent(in) :: colco2(:)          ! column amount (co2)
L04996                                                        !    Dimensions: (nlayers)
L04997        real(kind=rb), intent(in) :: colo3(:)           ! column amount (o3)
L04998                                                        !    Dimensions: (nlayers)
L04999        real(kind=rb), intent(in) :: coln2o(:)          ! column amount (n2o)
L05000                                                        !    Dimensions: (nlayers)
L05001        real(kind=rb), intent(in) :: colco(:)           ! column amount (co)
L05002                                                        !    Dimensions: (nlayers)
L05003        real(kind=rb), intent(in) :: colch4(:)          ! column amount (ch4)
L05004                                                        !    Dimensions: (nlayers)
L05005        real(kind=rb), intent(in) :: colo2(:)           ! column amount (o2)
L05006                                                        !    Dimensions: (nlayers)
L05007        real(kind=rb), intent(in) :: colbrd(:)          ! column amount (broadening gases)
L05008                                                        !    Dimensions: (nlayers)
```

```
L05009
L05010        integer(kind=im), intent(in) :: indself(:)
L05011                                                    !    Dimensions: (nlayers)
L05012        integer(kind=im), intent(in) :: indfor(:)
L05013                                                    !    Dimensions: (nlayers)
L05014        real(kind=rb), intent(in) :: selffac(:)
L05015                                                    !    Dimensions: (nlayers)
L05016        real(kind=rb), intent(in) :: selffrac(:)
L05017                                                    !    Dimensions: (nlayers)
L05018        real(kind=rb), intent(in) :: forfac(:)
L05019                                                    !    Dimensions: (nlayers)
L05020        real(kind=rb), intent(in) :: forfrac(:)
L05021                                                    !    Dimensions: (nlayers)
L05022
L05023        integer(kind=im), intent(in) :: indminor(:)
L05024                                                    !    Dimensions: (nlayers)
L05025        real(kind=rb), intent(in) :: minorfrac(:)
L05026                                                    !    Dimensions: (nlayers)
L05027        real(kind=rb), intent(in) :: scaleminor(:)
L05028                                                    !    Dimensions: (nlayers)
L05029        real(kind=rb), intent(in) :: scaleminorn2(:)
L05030                                                    !    Dimensions: (nlayers)
L05031
L05032        real(kind=rb), intent(in) :: &              !
L05033                       fac00(:), fac01(:), &        !    Dimensions: (nlayers)
L05034                       fac10(:), fac11(:)
L05035        real(kind=rb), intent(in) :: &              !
L05036                       rat_h2oco2(:),rat_h2oco2_1(:), &
L05037                       rat_h2oo3(:),rat_h2oo3_1(:), & !    Dimensions: (nlayers)
L05038                       rat_h2on2o(:),rat_h2on2o_1(:), &
L05039                       rat_h2och4(:),rat_h2och4_1(:), &
L05040                       rat_n2oco2(:),rat_n2oco2_1(:), &
L05041                       rat_o3co2(:),rat_o3co2_1(:)
L05042
L05043  ! ----- Output -----
L05044        real(kind=rb), intent(out) :: fracs(:,:)        ! planck fractions
L05045                                                    !    Dimensions: (nlayers,ngptlw)
L05046        real(kind=rb), intent(out) :: taug(:,:)         ! gaseous optical depth
L05047                                                    !    Dimensions: (nlayers,ngptlw)
L05048
L05049  !jm not thread safe      hvrtau = '$Revision: 1.7 $'
L05050
L05051  ! Calculate gaseous optical depth and planck fractions for each spectral band.
L05052
L05053        call taugb1
L05054        call taugb2
L05055        call taugb3
L05056        call taugb4
L05057        call taugb5
L05058        call taugb6
L05059        call taugb7
L05060        call taugb8
L05061        call taugb9
L05062        call taugb10
L05063        call taugb11
L05064        call taugb12
L05065        call taugb13
L05066        call taugb14
L05067        call taugb15
L05068        call taugb16
```

讲解：taumol 的巨型版权头（Mlawer/Delamere/Taubman/Clough，AER）把 k 分布方法的全部数据结构讲清楚了：①**KA/KB**——低/高层参考大气的 k 值表（cm²/molecule），维数 (NSPA(n),5,13,MG)——5 个温度档 ×13 压力比档 ×MG 个 g 点；**KA_M'GAS'**——次要气体（本波段主吸收体之外的）专用表；**SELFREF/FORREF**——水汽自/外连续谱参考表；② 插值自变量：JP（参考压力层索引）、JT/JT1（温度索引）、FAC00..11（温度 × 压力四角双线性权重）；**LAYTROP 对流层顶分层**——上下大气用不同主要气体组合（低层 h2o+co2 等、高层 h2o+o3 等）；③ **次要气体比率 rat_h2oco2 等 12 个**——binary species parameter（二参数法）插值键；④ 代码体只有 16 行：逐波段调 taugb1..16（CONTAINS 内部子程序，共享宿主作用域）。

```
!file: phys/module_ra_rrtmg_lw.F    L5073-L5166    (subroutine taugb1)
L05073        subroutine taugb1
L05074  !----------------------------------------------------------------------------
L05075
L05076  ! ------- Modifications -------
L05077  !  Written by Eli J. Mlawer, Atmospheric & Environmental Research.
L05078  !  Revised by Michael J. Iacono, Atmospheric & Environmental Research.
L05079  !
L05080  !     band 1:  10-350 cm-1 (low key - h2o; low minor - n2)
L05081  !                          (high key - h2o; high minor - n2)
L05082  !
L05083  !     note: previous versions of rrtm band 1:
L05084  !           10-250 cm-1 (low - h2o; high - h2o)
L05085  !----------------------------------------------------------------------------
L05086
L05087  ! ------- Modules -------
L05088
L05089        use parrrtm, only : ng1
L05090        use rrlw_kg01, only : fracrefa, fracrefb, absa, ka, absb, kb, &
L05091                              ka_mn2, kb_mn2, selfref, forref
L05092
L05093  ! ------- Declarations -------
L05094
L05095  ! Local
L05096        integer(kind=im) :: lay, ind0, ind1, inds, indf, indm, ig
L05097        real(kind=rb) :: pp, corradj, scalen2, tauself, taufor, taun2
L05098
L05099
L05100  ! Minor gas mapping levels:
L05101  !     lower - n2, p = 142.5490 mbar, t = 215.70 k
```

```
L05102  !     upper - n2, p = 142.5490 mbar, t = 215.70 k
L05103
L05104  ! Compute the optical depth by interpolating in ln(pressure) and
L05105  ! temperature.  Below laytrop, the water vapor self-continuum and
L05106  ! foreign continuum is interpolated (in temperature) separately.
L05107
L05108  ! Lower atmosphere loop
L05109        do lay = 1, laytrop
L05110
L05111           ind0 = ((jp(lay)-1)*5+(jt(lay)-1))*nspa(1) + 1
L05112           ind1 = (jp(lay)*5+(jt1(lay)-1))*nspa(1) + 1
L05113           inds = indself(lay)
L05114           indf = indfor(lay)
L05115           indm = indminor(lay)
L05116           pp = pavel(lay)
L05117           corradj =  1.
L05118           if (pp .lt. 250._rb) then
L05119              corradj = 1._rb - 0.15_rb * (250._rb-pp) / 154.4_rb
L05120           endif
L05121
L05122           scalen2 = colbrd(lay) * scaleminorn2(lay)
L05123           do ig = 1, ng1
L05124              tauself = selffac(lay) * (selfref(inds,ig) + selffrac(lay) * &
L05125                   (selfref(inds+1,ig) - selfref(inds,ig)))
L05126              taufor =  forfac(lay) * (forref(indf,ig) + forfrac(lay) * &
L05127                   (forref(indf+1,ig) -  forref(indf,ig)))
L05128              taun2 = scalen2*(ka_mn2(indm,ig) + &
L05129                   minorfrac(lay) * (ka_mn2(indm+1,ig) - ka_mn2(indm,ig)))
L05130              taug(lay,ig) = corradj * (colh2o(lay) * &
L05131                  (fac00(lay) * absa(ind0,ig) + &
L05132                   fac10(lay) * absa(ind0+1,ig) + &
L05133                   fac01(lay) * absa(ind1,ig) + &
L05134                   fac11(lay) * absa(ind1+1,ig)) &
L05135                   + tauself + taufor + taun2)
L05136               fracs(lay,ig) = fracrefa(ig)
L05137           enddo
L05138        enddo
L05139
L05140  ! Upper atmosphere loop
L05141        do lay = laytrop+1, nlayers
L05142
L05143           ind0 = ((jp(lay)-13)*5+(jt(lay)-1))*nspb(1) + 1
L05144           ind1 = ((jp(lay)-12)*5+(jt1(lay)-1))*nspb(1) + 1
L05145           indf = indfor(lay)
L05146           indm = indminor(lay)
L05147           pp = pavel(lay)
L05148           corradj =  1._rb - 0.15_rb * (pp / 95.6_rb)
L05149
L05150           scalen2 = colbrd(lay) * scaleminorn2(lay)
L05151           do ig = 1, ng1
L05152              taufor = forfac(lay) * (forref(indf,ig) + &
L05153                   forfrac(lay) * (forref(indf+1,ig) - forref(indf,ig)))
L05154              taun2 = scalen2*(kb_mn2(indm,ig) + &
L05155                   minorfrac(lay) * (kb_mn2(indm+1,ig) - kb_mn2(indm,ig)))
L05156              taug(lay,ig) = corradj * (colh2o(lay) * &
L05157                  (fac00(lay) * absb(ind0,ig) + &
L05158                   fac10(lay) * absb(ind0+1,ig) + &
L05159                   fac01(lay) * absb(ind1,ig) + &
L05160                   fac11(lay) * absb(ind1+1,ig)) &
L05161                   + taufor + taun2)
L05162              fracs(lay,ig) = fracrefb(ig)
L05163           enddo
L05164        enddo
L05165
L05166        end subroutine taugb1
```

讲解：波段 1（10–350 cm^{-1} 远红外转动带；旧版 10–250 cm^{-1}）样本，低/高大气两个循环：① 低层 ind0=((jp−1)•5+(jt−1))•nspa(1)+1——**压力×温度 →k 表展平索引**；四角插值 taug=colh2o•(fac00•absa(ind0)+fac10•absa(ind0+1)+fac01•absa(ind1)+fac11•absa(ind1+1))；② 自/外连续谱：tauself=selffac•(selfref 温度插值)、taufor=forfac•(forref 插值)——selffac 是水汽密度比（自展宽随水汽 ^{2} 增长）；③ **次要气体 N_2**：taun2=scalen2•(ka_mn2 插值)，scalen2=colbrd•scaleminorn2（展宽气体柱量）；④**corradj 辐射修正**——250 hPa 以下线性衰减（低层）、上层 corradj=1−0.15•(p/95.6)（pp/95.6 线性）——band 1 的经验修正因子；⑤fracs=fracrefa（普朗克分数本波段是常数，复杂波段里才插值）。taugb2..16 与此同构、仅气体组合与表维不同——读一知十六。

八、传输求解 rtrnmc（头段）

```
!file: phys/module_ra_rrtmg_lw.F    L3085-L3200
L03085        subroutine rtrnmc(nlayers, istart, iend, iout, pz, semiss, ncbands, &
L03086                          cldfmc, taucmc, planklay, planklev, plankbnd, &
L03087                          pwvcm, fracs, taut, &
L03088                          totuflux, totdflux, fnet, htr, &
L03089                          totuclfl, totdclfl, fnetc, htrc )
L03090  !----------------------------------------------------------------------------
L03091  !
L03092  !  Original version:   E. J. Mlawer, et al. RRTM_V3.0
L03093  !  Revision for GCMs:  Michael J. Iacono; October, 2002
L03094  !  Revision for F90:  Michael J. Iacono; June, 2006
L03095  !
L03096  !  This program calculates the upward fluxes, downward fluxes, and
L03097  !  heating rates for an arbitrary clear or cloudy atmosphere.  The input
L03098  !  to this program is the atmospheric profile, all Planck function
L03099  !  information, and the cloud fraction by layer.  A variable diffusivity
L03100  !  angle (SECDIFF) is used for the angle integration.  Bands 2-3 and 5-9
L03101  !  use a value for SECDIFF that varies from 1.50 to 1.80 as a function of
```

```
L03102  !  the column water vapor, and other bands use a value of 1.66.  The Gaussian
L03103  !  weight appropriate to this angle (WTDIFF=0.5) is applied here.  Note that
L03104  !  use of the emissivity angle for the flux integration can cause errors of
L03105  !  1 to 4 W/m2 within cloudy layers.
L03106  !  Clouds are treated with the McICA stochastic approach and maximum-random
L03107  !  cloud overlap.
L03108  !**************************************************************************
L03109
L03110  ! ------- Declarations -------
L03111
L03112  ! ----- Input -----
L03113        integer(kind=im), intent(in) :: nlayers         ! total number of layers
L03114        integer(kind=im), intent(in) :: istart          ! beginning band of calculation
L03115        integer(kind=im), intent(in) :: iend            ! ending band of calculation
L03116        integer(kind=im), intent(in) :: iout            ! output option flag
L03117
L03118  ! Atmosphere
L03119        real(kind=rb), intent(in) :: pz(0:)             ! level (interface) pressures (hPa, mb)
L03120                                                        !    Dimensions: (0:nlayers)
L03121        real(kind=rb), intent(in) :: pwvcm              ! precipitable water vapor (cm)
L03122        real(kind=rb), intent(in) :: semiss(:)          ! lw surface emissivity
L03123                                                        !    Dimensions: (nbndlw)
L03124        real(kind=rb), intent(in) :: planklay(:,:)      !
L03125                                                        !    Dimensions: (nlayers,nbndlw)
L03126        real(kind=rb), intent(in) :: planklev(0:,:)     !
L03127                                                        !    Dimensions: (0:nlayers,nbndlw)
L03128        real(kind=rb), intent(in) :: plankbnd(:)        !
L03129                                                        !    Dimensions: (nbndlw)
L03130        real(kind=rb), intent(in) :: fracs(:,:)         !
L03131                                                        !    Dimensions: (nlayers,ngptw)
L03132        real(kind=rb), intent(in) :: taut(:,:)          ! gaseous + aerosol optical depths
L03133                                                        !    Dimensions: (nlayers,ngptlw)
L03134
L03135  ! Clouds
L03136        integer(kind=im), intent(in) :: ncbands         ! number of cloud spectral bands
L03137        real(kind=rb), intent(in) :: cldfmc(:,:)        ! layer cloud fraction [mcica]
L03138                                                        !    Dimensions: (ngptlw,nlayers)
L03139        real(kind=rb), intent(in) :: taucmc(:,:)        ! layer cloud optical depth [mcica]
L03140                                                        !    Dimensions: (ngptlw,nlayers)
L03141
L03142  ! ----- Output -----
L03143        real(kind=rb), intent(out) :: totuflux(0:)      ! upward longwave flux (w/m2)
L03144                                                        !    Dimensions: (0:nlayers)
L03145        real(kind=rb), intent(out) :: totdflux(0:)      ! downward longwave flux (w/m2)
L03146                                                        !    Dimensions: (0:nlayers)
L03147        real(kind=rb), intent(out) :: fnet(0:)          ! net longwave flux (w/m2)
L03148                                                        !    Dimensions: (0:nlayers)
L03149        real(kind=rb), intent(out) :: htr(0:)           ! longwave heating rate (k/day)
L03150                                                        !    Dimensions: (0:nlayers)
L03151        real(kind=rb), intent(out) :: totuclfl(0:)      ! clear sky upward longwave flux (w/m2)
L03152                                                        !    Dimensions: (0:nlayers)
L03153        real(kind=rb), intent(out) :: totdclfl(0:)      ! clear sky downward longwave flux (w/m2)
L03154                                                        !    Dimensions: (0:nlayers)
L03155        real(kind=rb), intent(out) :: fnetc(0:)         ! clear sky net longwave flux (w/m2)
L03156                                                        !    Dimensions: (0:nlayers)
L03157        real(kind=rb), intent(out) :: htrc(0:)          ! clear sky longwave heating rate (k/day)
L03158                                                        !    Dimensions: (0:nlayers)
L03159
L03160  ! ----- Local -----
L03161  ! Declarations for radiative transfer
L03162        real(kind=rb) :: abscld(nlayers,ngptlw)
L03163        real(kind=rb) :: atot(nlayers)
L03164        real(kind=rb) :: atrans(nlayers)
L03165        real(kind=rb) :: bbugas(nlayers)
L03166        real(kind=rb) :: bbutot(nlayers)
L03167        real(kind=rb) :: clrurad(0:nlayers)
L03168        real(kind=rb) :: clrdrad(0:nlayers)
L03169        real(kind=rb) :: efclfrac(nlayers,ngptlw)
L03170        real(kind=rb) :: uflux(0:nlayers)
L03171        real(kind=rb) :: dflux(0:nlayers)
L03172        real(kind=rb) :: urad(0:nlayers)
L03173        real(kind=rb) :: drad(0:nlayers)
L03174        real(kind=rb) :: uclfl(0:nlayers)
L03175        real(kind=rb) :: dclfl(0:nlayers)
L03176        real(kind=rb) :: odcld(nlayers,ngptlw)
L03177
L03178
L03179        real(kind=rb) :: secdiff(nbndlw)                ! secant of diffusivity angle
L03180        real(kind=rb) :: transcld, radld, radclrd, plfrac, blay, dplankup, dplankdn
L03181        real(kind=rb) :: odepth, odtot, odepth_rec, odtot_rec, gassrc
L03182        real(kind=rb) :: tblind, tfactot, bbd, bbdtot, tfacgas, transc, tausfac
L03183        real(kind=rb) :: rad0, reflect, radlu, radclru
L03184
L03185        integer(kind=im) :: icldlyr(nlayers)                ! flag for cloud in layer
L03186        integer(kind=im) :: ibnd, ib, iband, lay, lev, l, ig  ! loop indices
L03187        integer(kind=im) :: igc                             ! g-point interval counter
L03188        integer(kind=im) :: iclddn                          ! flag for cloud in down path
L03189        integer(kind=im) :: ittot, itgas, itr               ! lookup table indices
L03190
L03191  ! ------- Definitions -------
L03192  ! input
L03193  !    nlayers                     ! number of model layers
L03194  !    ngptlw                      ! total number of g-point subintervals
L03195  !    nbndlw                      ! number of longwave spectral bands
L03196  !    ncbands                     ! number of spectral bands for clouds
L03197  !    secdiff                     ! diffusivity angle
L03198  !    wtdiff                      ! weight for radiance to flux conversion
L03199  !    pavel                       ! layer pressures (mb)
L03200  !    pz                          ! level (interface) pressures (mb)
```

讲解：Mlawer 原版注释点出长波传输的全部要点：① **变扩散角**——波段 2-3 与 5-9 的 secdiff 随柱水汽量在 1.50–1.80 间变化（水汽多 → 有效辐射角度更垂直），其余波段 1.66；辐射率 → 通量转换权重 wtdiff=0.5；**注释自认误差 1–4 W/m²**（云层内用发射率角积分通量的近似代价）；②McICA 随机云 + 最大随机重叠——云的次格变化用随机抽样表达，单次调用只抽样一次（这是 LWUPT 等

通量带随机噪声的根源，教材 ch12 的「随机噪声通量」即此）；③ 局部量清单：urad/drad 上/下行辐射率、uclfl/dclfl 晴空、clrurad/clrdrad、efclfrac 有效云分数、odcld 云光学深度。

九、休眠与未录单元注记

1. **lwcldpr（L9974–L10612）**：非 McICA 路径的云光学（icld 老式灰色光学深度）——本版本 imca=1 恒定，**从 rrtmg_lw 主链不可达**；其头部数据表（absice0/1/2/3、absliq0/1）倒是被 cldprmc 共享（module rrlw_cld）。
2. **inatm（L11186–L11522）**：WRF 廓线 →RRTMG 廓线的换序（自上而下 → 自下而上）与单位换算（mmr→vmr 用 amdw/amdo 系、比湿 → 体积混合比、气压 Pa→hPa），并算 coldry（干空气柱密度 mol/cm²）与 wkl 分子柱量——k 分布的插值自变量全在这里备齐。
3. **setcoef（L3556–L3921）**：对流层顶判定（laytrop）、jp/jt/jt1 查表索引、fac 四角权重、普朗克函数（planklay/planklev/plankbnd）、水汽连续谱自变量、12 个次要气体比率——taumol 的全部插值自变量。
4. **lw_kgb01..16（纯系数数据）与 cmbgb1..16（g 点缩减重排）**：k 分布的两层数据底座；rrtmg_lw_ini/lwdatinit/lwcmbdat 在初始化时装填——改谱或加气体的入口在这三件 +对应 kgb 表。
5. **O3DATA（L12868–L12978）**：o3input 的臭氧气候场读取；**relcalc/reicalc（L14542–L14630）**：云水/云冰有效半径查表（Ebert-Curry/Key/Fu 三口径的半径-温度表）。

十、改造入口

1. **温室气体**：ghg_input 分支 + read_CAMgases（clWRF 时变气体）——CO2/CH4/N2O 浓度敏感性实验入口；2005 默认值在 RRTMG_LWRAD data 段。
2. **冰云参数化**：iceflglw=0/1/2/3 四选（Ebert-Curry/Key/Fu）——云辐射敏感性标准实验；WRF namelist 无此开关，改 RRTMG_LWRAD 内的 iceflglw 赋值。
3. **McICA 随机种子**：permuteseed=150/irng=0——随机噪声通量的统计实验（多样本集合）入口。
4. **云重叠**：icld 0–5（namelist cldovrlp 传入）——指数重叠 4/5 是重叠敏感性研究的前线。
5. **气溶胶反馈**：tauaerlw1..16 装载段（WRF_CHEM）——负值/大值守卫是改造时的现成护栏。
6. **双精度恢复**：parkind 的 kind_rb（`#if 0` 段）——注意 SW 共用该模块。

十一、上下游链接

- 驱动层：`code_20_module_radiation_driver.md`（CASE RRTMG_LWSCHEME、ghg/o3/气溶胶管线）
- 同族短波：`code_06_module_ra_rrtmg_sw.md`（共用本文件 parkind；parrrtsw/rrsw_kg16..29 结构平行）
- 气溶胶光学：`code_60_module_ra_aerosol.md`（SW 侧专用；LW 气溶胶只走 WRF_CHEM 通道）
- 教材正文：ch12_radiation_family.md

code_06 — RRTMG 短波辐射方案（module_ra_rrtmg_sw.F：两流传输与 delta-Eddington 标定）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_ra_rrtmg_sw.F	12786	2497d05912baf2e3a0e9c43bd874dd48

- 抽取工具：workspace/scripts/code_extract.py v1.2（扫描器支持 CONTAINS 嵌套），2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：主链五件（rrtmg_sw 全文、RRTMG_SWRAD 三段、cldprmc_sw 尾段、taumol_sw 头段 +taumol16、spcvmc_sw 头段）+ swaerpr 全文；62 单元的分组总表见 § 九。教材锚点：ch12_radiation_family.md。

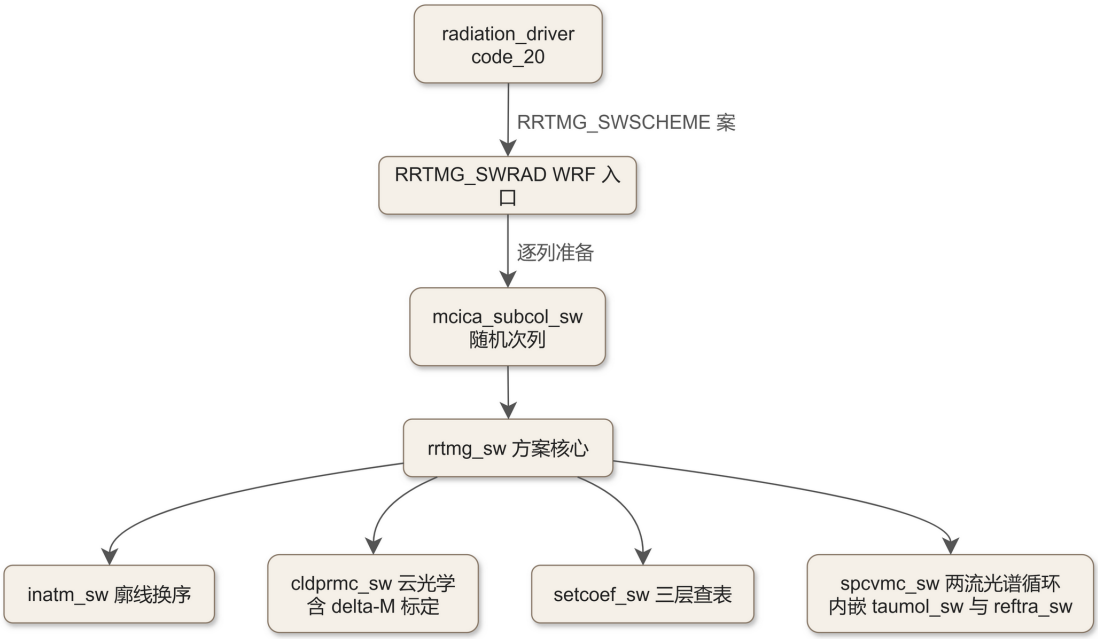
二、定位

一句话：**AER 的 RRTMG 短波方案**——与 code_05 长波同谱系：14 波段 k 分布 + g 点 224 缩减到 112 + McICA 随机次格云，但传输换成 **Barker 两流**（delta-Eddington 标定），且短波有散射——云与气溶胶都要给单次散射反照率与不对称因子。**模块级依赖：本文件不带 `parkind`，`use parkind` 从 module_ra_rrtmg_lw.F 导入**（LW 必须先编译）。

谁在调它：radiation_driver 的 CASE (RRTMG_SWSCHEME)（`code_20_module_radiation_driver.md` § 七），且先经 calc_aerosol_rrtmg_sw（`code_60_module_ra_aerosol.md`）备好 14 波段气溶胶光学。

62 单元分组总表（扫描器输出）：

组	单元	行段	说明
方案核心	rrtmg_sw / inatm_sw	L8856–L10005	核心编排 / 廓线换序
WRF 入口	RRTMG_SWRAD / rrtmg_swinit / rrtmg_swlookuptable	L10034–L11694	逐列准备与调核心
MCICA 群	mcica_subcol_sw / generate_stochastic_clouds_sw / kissvec	L1395–L2040	随机次列（与 LW 同源）
云光学	cldprmc_sw	L2077–L2498	含散射三参数 + delta-M 标定
传输	reftra_sw / spcvmc_sw / vrtqdr_sw	L2530–L2810, L8196–L8774, L8035–L8159	快速传输 / 两流光谱循环 / 垂直求积
廓线准备	setcoef_sw / swatmref	L2843–L3156	三层结构（tropopause/switch/low）
气溶胶	swaerpr	L5032–L5132	ECMWF 六类谱带光学
气体吸收	taumol_sw（含 taumol16..taumol29）	L3190–L4653	14 波段光学深度
初始化	rrtmg_sw_ini / swdatinit / swcmbdat	L4684–L5029	k 系数装填与 g 点缩减
非 McICA 云	swcldpr	L6181–L8004	**休眠**（同 LW 的 lwcldpr）
g 点缩减	cmbgb16s..cmbgb29	L5135–L6178	224→112 逐波段重排
k 系数	sw_kgb16..sw_kgb29	L11714–L12781	逐波段 k 分布系数（纯数据）



三、文件头：parrrsw

```
!file: phys/module_ra_rrtmg_sw.F    L1-L117
L00001  !!MODULE module_ra_rrtmg_sw
L00002
L00003        module parrrsw
L00004
L00005        use parkind ,only : im => kind_im, rb => kind_rb
L00006
L00007  !     implicit none
L00008        save
L00009
L00010  !------------------------------------------------------------------
L00011  ! rrtmg_sw main parameters
L00012  !
L00013  ! Initial version:  JJMorcrette, ECMWF, jul1998
L00014  ! Revised: MJIacono, AER, jun2006
L00015  ! Revised: MJIacono, AER, aug2008
L00016  !------------------------------------------------------------------
L00017
L00018  !  name     type     purpose
L00019  ! -----  :  ----    : ----------------------------------------------
L00020  ! mxlay  :  integer: maximum number of layers
L00021  ! mg     :  integer: number of original g-intervals per spectral band
L00022  ! nbndsw :  integer: number of spectral bands
L00023  ! naerec :  integer: number of aerosols (iaer=6, ecmwf aerosol option)
L00024  ! ngptsw :  integer: total number of reduced g-intervals for rrtmg_lw
L00025  ! ngNN   :  integer: number of reduced g-intervals per spectral band
L00026  ! ngsNN  :  integer: cumulative number of g-intervals per band
L00027  !------------------------------------------------------------------
L00028
L00029        integer(kind=im), parameter :: mxlay  = 203    !jplay, klev
L00030        integer(kind=im), parameter :: mg     = 16     !jpg
L00031        integer(kind=im), parameter :: nbndsw = 14     !jpsw, ksw
L00032        integer(kind=im), parameter :: naerec  = 6     !jpaer
L00033        integer(kind=im), parameter :: mxmol  = 38
L00034        integer(kind=im), parameter :: nstr   = 2
L00035        integer(kind=im), parameter :: nmol   = 7
L00036  ! Use for 112 g-point model
L00037        integer(kind=im), parameter :: ngptsw = 112    !jpgpt
L00038  ! Use for 224 g-point model
L00039  !      integer(kind=im), parameter :: ngptsw = 224   !jpgpt
L00040
L00041  ! may need to rename these - from v2.6
L00042        integer(kind=im), parameter :: jpband   = 29
L00043        integer(kind=im), parameter :: jpb1     = 16   !istart
L00044        integer(kind=im), parameter :: jpb2     = 29   !iend
L00045
L00046        integer(kind=im), parameter :: jmcmu    = 32
L00047        integer(kind=im), parameter :: jmumu    = 32
L00048        integer(kind=im), parameter :: jmphi    = 3
L00049        integer(kind=im), parameter :: jmxang   = 4
L00050        integer(kind=im), parameter :: jmxstr   = 16
L00051
L00052  ! Use for 112 g-point model
L00053        integer(kind=im), parameter :: ng16 = 6
L00054        integer(kind=im), parameter :: ng17 = 12
L00055        integer(kind=im), parameter :: ng18 = 8
L00056        integer(kind=im), parameter :: ng19 = 8
L00057        integer(kind=im), parameter :: ng20 = 10
L00058        integer(kind=im), parameter :: ng21 = 10
L00059        integer(kind=im), parameter :: ng22 = 2
L00060        integer(kind=im), parameter :: ng23 = 10
L00061        integer(kind=im), parameter :: ng24 = 8
L00062        integer(kind=im), parameter :: ng25 = 6
```

```
L00063        integer(kind=im), parameter :: ng26 = 6
L00064        integer(kind=im), parameter :: ng27 = 8
L00065        integer(kind=im), parameter :: ng28 = 6
L00066        integer(kind=im), parameter :: ng29 = 12
L00067
L00068        integer(kind=im), parameter :: ngs16 = 6
L00069        integer(kind=im), parameter :: ngs17 = 18
L00070        integer(kind=im), parameter :: ngs18 = 26
L00071        integer(kind=im), parameter :: ngs19 = 34
L00072        integer(kind=im), parameter :: ngs20 = 44
L00073        integer(kind=im), parameter :: ngs21 = 54
L00074        integer(kind=im), parameter :: ngs22 = 56
L00075        integer(kind=im), parameter :: ngs23 = 66
L00076        integer(kind=im), parameter :: ngs24 = 74
L00077        integer(kind=im), parameter :: ngs25 = 80
L00078        integer(kind=im), parameter :: ngs26 = 86
L00079        integer(kind=im), parameter :: ngs27 = 94
L00080        integer(kind=im), parameter :: ngs28 = 100
L00081        integer(kind=im), parameter :: ngs29 = 112
L00082
L00083  ! Use for 224 g-point model
L00084  !      integer(kind=im), parameter :: ng16 = 16
L00085  !      integer(kind=im), parameter :: ng17 = 16
L00086  !      integer(kind=im), parameter :: ng18 = 16
L00087  !      integer(kind=im), parameter :: ng19 = 16
L00088  !      integer(kind=im), parameter :: ng20 = 16
L00089  !      integer(kind=im), parameter :: ng21 = 16
L00090  !      integer(kind=im), parameter :: ng22 = 16
L00091  !      integer(kind=im), parameter :: ng23 = 16
L00092  !      integer(kind=im), parameter :: ng24 = 16
L00093  !      integer(kind=im), parameter :: ng25 = 16
L00094  !      integer(kind=im), parameter :: ng26 = 16
L00095  !      integer(kind=im), parameter :: ng27 = 16
L00096  !      integer(kind=im), parameter :: ng28 = 16
L00097  !      integer(kind=im), parameter :: ng29 = 16
L00098
L00099  !      integer(kind=im), parameter :: ngs16 = 16
L00100  !      integer(kind=im), parameter :: ngs17 = 32
L00101  !      integer(kind=im), parameter :: ngs18 = 48
L00102  !      integer(kind=im), parameter :: ngs19 = 64
L00103  !      integer(kind=im), parameter :: ngs20 = 80
L00104  !      integer(kind=im), parameter :: ngs21 = 96
L00105  !      integer(kind=im), parameter :: ngs22 = 112
L00106  !      integer(kind=im), parameter :: ngs23 = 128
L00107  !      integer(kind=im), parameter :: ngs24 = 144
L00108  !      integer(kind=im), parameter :: ngs25 = 160
L00109  !      integer(kind=im), parameter :: ngs26 = 176
L00110  !      integer(kind=im), parameter :: ngs27 = 192
L00111  !      integer(kind=im), parameter :: ngs28 = 208
L00112  !      integer(kind=im), parameter :: ngs29 = 224
L00113
L00114  ! Source function solar constant
L00115        real(kind=rb), parameter :: rrsw_scon = 1.36822e+03     ! W/m2
L00116
L00117        end module parrrsw
```

讲解：①**parrrsw 首行 use parkind**——本文件不定义 parkind，从 module_ra_rrtmg_lw.F 导入（编译顺序耦合）；②nbndsw=14 波段、波段编号 16–29（jpb1=16 起算——短波的「第 1 波段」全局编号是 16，k 系数表 sw_kgb16..29 与 taumol16..29 沿用全局号）、**ngptsw=112**（224 备选注释）、ng16..ng29 每波段 g 点数（合计 112）、ngs 累积表；③jmcmu/jmumu=32×32 角积分网格、jmphi=3（两流的离散角度参数，reftra 用）；④**rrsw_scon=1368.22 W/m²**——RRTMG 内置太阳常数（solvar 缩放的基准），而 radiation_driver 的 radconst 用 1370 • ECCFAC——**两套太阳常数口径并存**（1368.22 vs 1370，教材 ch12 的 Qd 口径讨论在此落地）。

四、方案核心 rrtmg_sw（全文）

```
!file: phys/module_ra_rrtmg_sw.F    L8856-L9622    (subroutine rrtmg_sw)
L08856        subroutine rrtmg_sw &
L08857             (ncol    ,nlay    ,icld    , &
L08858              play    ,plev    ,tlay    ,tlev    ,tsfc   , &
L08859              h2ovmr , o3vmr   ,co2vmr  ,ch4vmr  ,n2ovmr ,o2vmr , &
L08860              asdir   ,asdif   ,aldir   ,aldif   , &
L08861              coszen  ,adjes   ,dyofyr  ,scon    , &
L08862              inflgsw ,iceflgsw,liqflgsw,cldfmcl , &
L08863              taucmcl ,ssacmcl ,asmcmcl ,fsfcmcl , &
L08864              ciwpmcl ,clwpmcl ,cswpmcl ,reicmcl ,relqmcl ,resnmcl, &
L08865              tauaer  ,ssaaer  ,asmaer  ,ecaer   , &
L08866              swuflx  ,swdflx  ,swhr    ,swuflxc ,swdflxc ,swhrc, swuflxcln ,swdflxcln , aer_opt,  &
L08867  ! --------- Add the following four compenants for ssib shortwave down radiation ---!
L08868  ! -------------------      by Zhenxin 2011-06-20      --------------------------------!
L08869              sibvisdir, sibvisdif, sibnirdir, sibnirdif,         &
L08870  ! ----------------------  End,  Zhenxin 2011-06-20    --------------------------------!
L08871              swdkdir,swdkdif,                                    & ! jararias, 2013/08/10
L08872              swdkdirc,                                           & ! PAJ
L08873              calc_clean_atm_diag,                                &
L08874              sw_zbbcddir, sw_dirdflux, sw_difdflux               & ! WRF-CMAQ twoway coupled model
L08875                                                                    )
L08876
L08877
L08878  ! ------- Description -------
L08879
L08880  ! This program is the driver for RRTMG_SW, the AER SW radiation model for
L08881  !  application to GCMs, that has been adapted from RRTM_SW for improved
L08882  !  efficiency and to provide fractional cloudiness and cloud overlap
L08883  !  capability using McICA.
L08884  !
```

```
! Note: The call to RRTMG_SW_INI should be moved to the GCM initialization
!  area, since this has to be called only once.
!
! This routine
!    b) calls INATM_SW to read in the atmospheric profile;
!       all layering in RRTMG is ordered from surface to toa.
!    c) calls CLDPRMC_SW to set cloud optical depth for McICA based
!       on input cloud properties
!    d) calls SETCOEF_SW to calculate various quantities needed for
!       the radiative transfer algorithm
!    e) calls SPCVMC to call the two-stream model that in turn
!       calls TAUMOL to calculate gaseous optical depths for each
!       of the 16 spectral bands and to perform the radiative transfer
!       using McICA, the Monte-Carlo Independent Column Approximation,
!       to represent sub-grid scale cloud variability
!    f) passes the calculated fluxes and cooling rates back to GCM
!
! Two modes of operation are possible:
!     The mode is chosen by using either rrtmg_sw.nomcica.f90 (to not use
!     McICA) or rrtmg_sw.f90 (to use McICA) to interface with a GCM.
!
!    1) Standard, single forward model calculation (imca = 0); this is
!       valid only for clear sky or fully overcast clouds
!    2) Monte Carlo Independent Column Approximation (McICA, Pincus et al.,
!       JC, 2003) method is applied to the forward model calculation (imca = 1)
!       This method is valid for clear sky or partial cloud conditions.
!
! This call to RRTMG_SW must be preceeded by a call to the module
!     mcica_subcol_gen_sw.f90 to run the McICA sub-column cloud generator,
!     which will provide the cloud physical or cloud optical properties
!     on the RRTMG quadrature point (ngptsw) dimension.
!
! Two methods of cloud property input are possible:
!     Cloud properties can be input in one of two ways (controlled by input
!     flags inflag, iceflag and liqflag; see text file rrtmg_sw_instructions
!     and subroutine rrtmg_sw_cldprop.f90 for further details):
!
!    1) Input cloud fraction, cloud optical depth, single scattering albedo
!       and asymmetry parameter directly (inflgsw = 0)
!    2) Input cloud fraction and cloud physical properties: ice fracion,
!       ice and liquid particle sizes (inflgsw = 1 or 2);
!       cloud optical properties are calculated by cldprop or cldprmc based
!       on input settings of iceflgsw and liqflgsw
!
! Two methods of aerosol property input are possible:
!     Aerosol properties can be input in one of two ways (controlled by input
!     flag iaer, see text file rrtmg_sw_instructions for further details):
!
!    1) Input aerosol optical depth, single scattering albedo and asymmetry
!       parameter directly by layer and spectral band (iaer=10)
!    2) Input aerosol optical depth and 0.55 micron directly by layer and use
!       one or more of six ECMWF aerosol types (iaer=6)
!
!
! ------- Modifications -------
!
! This version of RRTMG_SW has been modified from RRTM_SW to use a reduced
! set of g-point intervals and a two-stream model for application to GCMs.
!
!-- Original version (derived from RRTM_SW)
!     2002: AER. Inc.
!-- Conversion to F90 formatting; addition of 2-stream radiative transfer
!     Feb 2003: J.-J. Morcrette, ECMWF
!-- Additional modifications for GCM application
!     Aug 2003: M. J. Iacono, AER Inc.
!-- Total number of g-points reduced from 224 to 112.  Original
!   set of 224 can be restored by exchanging code in module parrrsw.f90
!   and in file rrtmg_sw_init.f90.
!     Apr 2004: M. J. Iacono, AER, Inc.
!-- Modifications to include output for direct and diffuse
!   downward fluxes.  There are output as "true" fluxes without
!   any delta scaling applied.  Code can be commented to exclude
!   this calculation in source file rrtmg_sw_spcvrt.f90.
!     Jan 2005: E. J. Mlawer, M. J. Iacono, AER, Inc.
!-- Revised to add McICA capability.
!     Nov 2005: M. J. Iacono, AER, Inc.
!-- Reformatted for consistency with rrtmg_lw.
!     Feb 2007: M. J. Iacono, AER, Inc.
!-- Modifications to formatting to use assumed-shape arrays.
!     Aug 2007: M. J. Iacono, AER, Inc.

! --------- Modules ---------

      use parrrsw, only : nbndsw, ngptsw, naerec, nstr, nmol, mxmol, &
                          jpband, jpb1, jpb2
      use rrsw_aer, only : rsrtaua, rsrpiza, rsrasya
      use rrsw_con, only : heatfac, oneminus, pi
      use rrsw_wvn, only : wavenum1, wavenum2

! ------- Declarations

! ----- Input -----

      integer(kind=im), intent(in) :: ncol            ! Number of horizontal columns
      integer(kind=im), intent(in) :: nlay            ! Number of model layers
      integer(kind=im), intent(inout) :: icld         ! Cloud overlap method
                                                      !    0: Clear only
                                                      !    1: Random
                                                      !    2: Maximum/random
                                                      !    3: Maximum
                                                      !    4: Exponential
                                                      !    5: Exponential/random
      real(kind=rb), intent(in) :: play(:,:)          ! Layer pressures (hPa, mb)
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: plev(:,:)          ! Interface pressures (hPa, mb)
                                                      !    Dimensions: (ncol,nlay+1)
      real(kind=rb), intent(in) :: tlay(:,:)          ! Layer temperatures (K)
                                                      !    Dimensions: (ncol,nlay)
      real(kind=rb), intent(in) :: tlev(:,:)          ! Interface temperatures (K)
```

```
L08994                                                    !    Dimensions: (ncol,nlay+1)
L08995         real(kind=rb), intent(in) :: tsfc(:)       ! Surface temperature (K)
L08996                                                    !    Dimensions: (ncol)
L08997         real(kind=rb), intent(in) :: h2ovmr(:,:)   ! H2O volume mixing ratio
L08998                                                    !    Dimensions: (ncol,nlay)
L08999         real(kind=rb), intent(in) :: o3vmr(:,:)    ! O3 volume mixing ratio
L09000                                                    !    Dimensions: (ncol,nlay)
L09001         real(kind=rb), intent(in) :: co2vmr(:,:)   ! CO2 volume mixing ratio
L09002                                                    !    Dimensions: (ncol,nlay)
L09003         real(kind=rb), intent(in) :: ch4vmr(:,:)   ! Methane volume mixing ratio
L09004                                                    !    Dimensions: (ncol,nlay)
L09005         real(kind=rb), intent(in) :: n2ovmr(:,:)   ! Nitrous oxide volume mixing ratio
L09006                                                    !    Dimensions: (ncol,nlay)
L09007         real(kind=rb), intent(in) :: o2vmr(:,:)    ! Oxygen volume mixing ratio
L09008                                                    !    Dimensions: (ncol,nlay)
L09009         real(kind=rb), intent(in) :: asdir(:)      ! UV/vis surface albedo direct rad
L09010                                                    !    Dimensions: (ncol)
L09011         real(kind=rb), intent(in) :: aldir(:)      ! Near-IR surface albedo direct rad
L09012                                                    !    Dimensions: (ncol)
L09013         real(kind=rb), intent(in) :: asdif(:)      ! UV/vis surface albedo: diffuse rad
L09014                                                    !    Dimensions: (ncol)
L09015         real(kind=rb), intent(in) :: aldif(:)      ! Near-IR surface albedo: diffuse rad
L09016                                                    !    Dimensions: (ncol)
L09017
L09018         integer(kind=im), intent(in) :: dyofyr     ! Day of the year (used to get Earth/Sun
L09019                                                    !  distance if adjflx not provided)
L09020         real(kind=rb), intent(in) :: adjes         ! Flux adjustment for Earth/Sun distance
L09021         real(kind=rb), intent(in) :: coszen(:)     ! Cosine of solar zenith angle
L09022                                                    !    Dimensions: (ncol)
L09023         real(kind=rb), intent(in) :: scon          ! Solar constant (W/m2)
L09024
L09025         integer(kind=im), intent(in) :: inflgsw    ! Flag for cloud optical properties
L09026         integer(kind=im), intent(in) :: iceflgsw   ! Flag for ice particle specification
L09027         integer(kind=im), intent(in) :: liqflgsw   ! Flag for liquid droplet specification
L09028
L09029         real(kind=rb), intent(in) :: cldfmcl(:,:,:)  ! Cloud fraction
L09030                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09031         real(kind=rb), intent(in) :: taucmcl(:,:,:)  ! In-cloud optical depth
L09032                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09033         real(kind=rb), intent(in) :: ssacmcl(:,:,:)  ! In-cloud single scattering albedo
L09034                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09035         real(kind=rb), intent(in) :: asmcmcl(:,:,:)  ! In-cloud asymmetry parameter
L09036                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09037         real(kind=rb), intent(in) :: fsfcmcl(:,:,:)  ! In-cloud forward scattering fraction
L09038                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09039         real(kind=rb), intent(in) :: ciwpmcl(:,:,:)  ! In-cloud ice water path (g/m2)
L09040                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09041         real(kind=rb), intent(in) :: clwpmcl(:,:,:)  ! In-cloud liquid water path (g/m2)
L09042                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09043         real(kind=rb), intent(in) :: cswpmcl(:,:,:)  ! In-cloud snow water path (g/m2)
L09044                                                      !    Dimensions: (ngptsw,ncol,nlay)
L09045         real(kind=rb), intent(in) :: reicmcl(:,:)    ! Cloud ice effective radius (microns)
L09046                                                      !    Dimensions: (ncol,nlay)
L09047                                                      ! specific definition of reicmcl depends on setting of iceflglw:
L09048                                                      ! iceflglw = 0: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L09049                                                      !              r_ec must be >= 10.0 microns
L09050                                                      ! iceflglw = 1: ice effective radius, r_ec, (Ebert and Curry,
↪  1992),
L09051                                                      !              r_ec range is limited to 13.0 to 130.0 microns
L09052                                                      ! iceflglw = 2: ice effective radius, r_k, (Key, Streamer Ref.
↪  Manual, 1996)
L09053                                                      !              r_k range is limited to 5.0 to 131.0 microns
L09054                                                      ! iceflglw = 3: generalized effective size, dge, (Fu, 1996),
L09055                                                      !              dge range is limited to 5.0 to 140.0 microns
L09056                                                      !              [dge = 1.0315 * r_ec]
L09057         real(kind=rb), intent(in) :: relqmcl(:,:)    ! Cloud water drop effective radius (microns)
L09058                                                      !    Dimensions: (ncol,nlay)
L09059         real(kind=rb), intent(in) :: resnmcl(:,:)    ! Cloud snow effective radius (microns)
L09060                                                      !    Dimensions: (ncol,nlay)
L09061         real(kind=rb), intent(in) :: tauaer(:,:,:)   ! Aerosol optical depth (iaer=10 only)
L09062                                                      !    Dimensions: (ncol,nlay,nbndsw)
L09063                                                      ! (non-delta scaled)
L09064         real(kind=rb), intent(in) :: ssaaer(:,:,:)   ! Aerosol single scattering albedo (iaer=10 only)
L09065                                                      !    Dimensions: (ncol,nlay,nbndsw)
L09066                                                      ! (non-delta scaled)
L09067         real(kind=rb), intent(in) :: asmaer(:,:,:)   ! Aerosol asymmetry parameter (iaer=10 only)
L09068                                                      !    Dimensions: (ncol,nlay,nbndsw)
L09069                                                      ! (non-delta scaled)
L09070         real(kind=rb), intent(in) :: ecaer(:,:,:)    ! Aerosol optical depth at 0.55 micron (iaer=6 only)
L09071                                                      !    Dimensions: (ncol,nlay,naerec)
L09072                                                      ! (non-delta scaled)
L09073         integer, intent(in)       :: calc_clean_atm_diag! Control for clean air diagnositic calls for WRF-Chem
L09074
L09075  ! ----- Output -----
L09076
L09077         real(kind=rb), intent(out) :: swuflx(:,:)    ! Total sky shortwave upward flux (W/m2)
L09078                                                      !    Dimensions: (ncol,nlay+1)
L09079         real(kind=rb), intent(out) :: swdflx(:,:)    ! Total sky shortwave downward flux (W/m2)
L09080                                                      !    Dimensions: (ncol,nlay+1)
L09081         real(kind=rb), intent(out) :: sibvisdir(:,:) ! visible direct downward flux  (W/m2)
L09082                                                      !    Dimensions: (ncol,nlay+1) Zhenxin (2011/06/20)
L09083         real(kind=rb), intent(out) :: sibvisdif(:,:) ! visible diffusion downward flux  (W/m2)
L09084                                                      !    Dimensions: (ncol,nlay+1) Zhenxin (2011/06/20)
L09085         real(kind=rb), intent(out) :: sibnirdir(:,:) ! Near IR direct downward flux  (W/m2)
L09086                                                      !    Dimensions: (ncol,nlay+1)  Zhenxin (2011/06/20)
L09087         real(kind=rb), intent(out) :: sibnirdif(:,:) ! Near IR diffusion downward flux  (W/m2)
L09088                                                      !    Dimensions: (ncol,nlay+1) Zhenxin (2011/06/20)
L09089         real(kind=rb), intent(out) :: swhr(:,:)      ! Total sky shortwave radiative heating rate (K/d)
L09090                                                      !    Dimensions: (ncol,nlay)
L09091         real(kind=rb), intent(out) :: swuflxc(:,:)   ! Clear sky shortwave upward flux (W/m2)
L09092                                                      !    Dimensions: (ncol,nlay+1)
L09093         real(kind=rb), intent(out) :: swdflxc(:,:)   ! Clear sky shortwave downward flux (W/m2)
L09094                                                      !    Dimensions: (ncol,nlay+1)
L09095         real(kind=rb), intent(out) :: swhrc(:,:)     ! Clear sky shortwave radiative heating rate (K/d)
L09096                                                      !    Dimensions: (ncol,nlay)
```

```
L09097        real(kind=rb), intent(out) :: swuflxcln(:,:)    ! Clean sky shortwave upward flux (W/m2)
L09098                                                        !    Dimensions: (ncol,nlay+1)
L09099        real(kind=rb), intent(out) :: swdflxcln(:,:)    ! Clean sky shortwave downward flux (W/m2)
L09100                                                        !    Dimensions: (ncol,nlay+1)
L09101
L09102        integer, intent(in)        :: aer_opt
L09103        real(kind=rb), intent(out) :: &
L09104          swdkdir(:,:), & ! Total shortwave downward direct flux (W/m2),  Dimensions: (ncol,nlay) jararias, 2013/08/10
L09105          swdkdif(:,:), & ! Total shortwave downward diffuse flux (W/m2), Dimensions: (ncol,nlay) jararias, 2013/08/10
L09106          swdkdirc(:,:)   ! Total shortwave downward direct flux clear sky (W/m2),  Dimensions: (ncol,nlay)
L09107
L09108        real, intent(out) :: sw_zbbcddir,   &  ! WRF-CMAQ twoway coupled model
L09109                             sw_dirdflux,   &  ! WRF-CMAQ twoway coupled model
L09110                             sw_difdflux       ! WRF-CMAQ twoway coupled model
L09111
L09112
L09113
L09114
L09115
L09116  ! ----- Local -----
L09117
L09118  ! Control
L09119        integer(kind=im) :: nlayers             ! total number of layers
L09120        integer(kind=im) :: istart              ! beginning band of calculation
L09121        integer(kind=im) :: iend                ! ending band of calculation
L09122        integer(kind=im) :: icpr                ! cldprop/cldprmc use flag
L09123        integer(kind=im) :: iout                ! output option flag (inactive)
L09124        integer(kind=im) :: iaer                ! aerosol option flag
L09125        integer(kind=im) :: idelm               ! delta-m scaling flag (inactive)
L09126        integer(kind=im) :: isccos              ! instrumental cosine response flag (inactive)
L09127        integer(kind=im) :: iplon               ! column loop index
L09128        integer(kind=im) :: i                   ! layer loop index                       ! jk
L09129        integer(kind=im) :: ib                  ! band loop index                        ! jsw
L09130        integer(kind=im) :: ia, ig              ! indices
L09131        integer(kind=im) :: k                   ! layer loop index
L09132        integer(kind=im) :: ims                 ! value for changing mcica permute seed
L09133        integer(kind=im) :: imca                ! flag for mcica [0=off, 1=on]
L09134
L09135        real(kind=rb) :: zepsec, zepzen         ! epsilon
L09136        real(kind=rb) :: zdpgcp                 ! flux to heating conversion ratio
L09137
L09138  ! Atmosphere
L09139        real(kind=rb) :: pavel(nlay+1)          ! layer pressures (mb)
L09140        real(kind=rb) :: tavel(nlay+1)          ! layer temperatures (K)
L09141        real(kind=rb) :: pz(0:nlay+1)           ! level (interface) pressures (hPa, mb)
L09142        real(kind=rb) :: tz(0:nlay+1)           ! level (interface) temperatures (K)
L09143        real(kind=rb) :: tbound                 ! surface temperature (K)
L09144        real(kind=rb) :: pdp(nlay+1)            ! layer pressure thickness (hPa, mb)
L09145        real(kind=rb) :: coldry(nlay+1)         ! dry air column amount
L09146        real(kind=rb) :: wkl(mxmol,nlay+1)      ! molecular amounts (mol/cm-2)
L09147
L09148  !      real(kind=rb) :: earth_sun              ! function for Earth/Sun distance factor
L09149        real(kind=rb) :: cossza                 ! Cosine of solar zenith angle
L09150        real(kind=rb) :: adjflux(jpband)        ! adjustment for current Earth/Sun distance
L09151        real(kind=rb) :: solvar(jpband)         ! solar constant scaling factor from rrtmg_sw
L09152                                                !  default value of 1368.22 Wm-2 at 1 AU
L09153        real(kind=rb) :: albdir(nbndsw)         ! surface albedo, direct          ! zalbp
L09154        real(kind=rb) :: albdif(nbndsw)         ! surface albedo, diffuse         ! zalbd
L09155
L09156        real(kind=rb) :: taua(nlay+1,nbndsw)    ! Aerosol optical depth
L09157        real(kind=rb) :: ssaa(nlay+1,nbndsw)    ! Aerosol single scattering albedo
L09158        real(kind=rb) :: asma(nlay+1,nbndsw)    ! Aerosol asymmetry parameter
L09159
L09160  ! Atmosphere - setcoef
L09161        integer(kind=im) :: laytrop             ! tropopause layer index
L09162        integer(kind=im) :: layswtch            ! tropopause layer index
L09163        integer(kind=im) :: laylow              ! tropopause layer index
L09164        integer(kind=im) :: jp(nlay+1)          !
L09165        integer(kind=im) :: jt(nlay+1)          !
L09166        integer(kind=im) :: jt1(nlay+1)         !
L09167
L09168        real(kind=rb) :: colh2o(nlay+1)         ! column amount (h2o)
L09169        real(kind=rb) :: colco2(nlay+1)         ! column amount (co2)
L09170        real(kind=rb) :: colo3(nlay+1)          ! column amount (o3)
L09171        real(kind=rb) :: coln2o(nlay+1)         ! column amount (n2o)
L09172        real(kind=rb) :: colch4(nlay+1)         ! column amount (ch4)
L09173        real(kind=rb) :: colo2(nlay+1)          ! column amount (o2)
L09174        real(kind=rb) :: colmol(nlay+1)         ! column amount
L09175        real(kind=rb) :: co2mult(nlay+1)        ! column amount
L09176
L09177        integer(kind=im) :: indself(nlay+1)
L09178        integer(kind=im) :: indfor(nlay+1)
L09179        real(kind=rb) :: selffac(nlay+1)
L09180        real(kind=rb) :: selffrac(nlay+1)
L09181        real(kind=rb) :: forfac(nlay+1)
L09182        real(kind=rb) :: forfrac(nlay+1)
L09183
L09184        real(kind=rb) :: &                      !
L09185                            fac00(nlay+1), fac01(nlay+1), &
L09186                            fac10(nlay+1), fac11(nlay+1)
L09187
L09188  ! Atmosphere/clouds - cldprop
L09189        integer(kind=im) :: ncbands             ! number of cloud spectral bands
L09190        integer(kind=im) :: inflag              ! flag for cloud property method
L09191        integer(kind=im) :: iceflag             ! flag for ice cloud properties
L09192        integer(kind=im) :: liqflag             ! flag for liquid cloud properties
L09193
L09194  !      real(kind=rb) :: cldfrac(nlay+1)        ! layer cloud fraction
L09195  !      real(kind=rb) :: tauc(nlay+1)           ! in-cloud optical depth (non-delta scaled)
L09196  !      real(kind=rb) :: ssac(nlay+1)           ! in-cloud single scattering albedo (non-delta scaled)
L09197  !      real(kind=rb) :: asmc(nlay+1)           ! in-cloud asymmetry parameter (non-delta scaled)
L09198  !      real(kind=rb) :: fsfc(nlay+1)           ! in-cloud forward scattering fraction (non-delta scaled)
L09199  !      real(kind=rb) :: ciwp(nlay+1)           ! in-cloud ice water path
L09200  !      real(kind=rb) :: clwp(nlay+1)           ! in-cloud liquid water path
L09201  !      real(kind=rb) :: rei(nlay+1)            ! cloud ice particle size
L09202  !      real(kind=rb) :: rel(nlay+1)            ! cloud liquid particle size
L09203
```

```
!      real(kind=rb) :: taucloud(nlay+1,jpband)  ! in-cloud optical depth
!      real(kind=rb) :: taucldorig(nlay+1,jpband)! in-cloud optical depth (non-delta scaled)
!      real(kind=rb) :: ssacloud(nlay+1,jpband)  ! in-cloud single scattering albedo
!      real(kind=rb) :: asmcloud(nlay+1,jpband)  ! in-cloud asymmetry parameter

! Atmosphere/clouds - cldprmc [mcica]
      real(kind=rb) :: cldfmc(ngptsw,nlay+1)    ! cloud fraction [mcica]
      real(kind=rb) :: ciwpmc(ngptsw,nlay+1)    ! in-cloud ice water path [mcica]
      real(kind=rb) :: clwpmc(ngptsw,nlay+1)    ! in-cloud liquid water path [mcica]
      real(kind=rb) :: cswpmc(ngptsw,nlay+1)    ! in-cloud snow water path [mcica]
      real(kind=rb) :: relqmc(nlay+1)           ! liquid particle effective radius (microns)
      real(kind=rb) :: reicmc(nlay+1)           ! ice particle effective size (microns)
      real(kind=rb) :: resnmc(nlay+1)           ! snow particle effective size (microns)
      real(kind=rb) :: taucmc(ngptsw,nlay+1)    ! in-cloud optical depth [mcica]
      real(kind=rb) :: taormc(ngptsw,nlay+1)    ! unscaled in-cloud optical depth [mcica]
      real(kind=rb) :: ssacmc(ngptsw,nlay+1)    ! in-cloud single scattering albedo [mcica]
      real(kind=rb) :: asmcmc(ngptsw,nlay+1)    ! in-cloud asymmetry parameter [mcica]
      real(kind=rb) :: fsfcmc(ngptsw,nlay+1)    ! in-cloud forward scattering fraction [mcica]

! Atmosphere/clouds/aerosol - spcvrt,spcvmc
      real(kind=rb) :: ztauc(nlay+1,nbndsw)     ! cloud optical depth
      real(kind=rb) :: ztaucorig(nlay+1,nbndsw) ! unscaled cloud optical depth
      real(kind=rb) :: zasyc(nlay+1,nbndsw)     ! cloud asymmetry parameter
                                                !  (first moment of phase function)
      real(kind=rb) :: zomgc(nlay+1,nbndsw)     ! cloud single scattering albedo
      real(kind=rb) :: ztaua(nlay+1,nbndsw)     ! total aerosol optical depth
      real(kind=rb) :: ztauacln(nlay+1,nbndsw)  ! dummy total aerosol optical depth for clean case (=zero)
      real(kind=rb) :: zasya(nlay+1,nbndsw)     ! total aerosol asymmetry parameter
      real(kind=rb) :: zomga(nlay+1,nbndsw)     ! total aerosol single scattering albedo

      real(kind=rb) :: zcldfmc(nlay+1,ngptsw)   ! cloud fraction [mcica]
      real(kind=rb) :: ztaucmc(nlay+1,ngptsw)   ! cloud optical depth [mcica]
      real(kind=rb) :: ztaormc(nlay+1,ngptsw)   ! unscaled cloud optical depth [mcica]
      real(kind=rb) :: zasycmc(nlay+1,ngptsw)   ! cloud asymmetry parameter [mcica]
      real(kind=rb) :: zomgcmc(nlay+1,ngptsw)   ! cloud single scattering albedo [mcica]

      real(kind=rb) :: zbbfu(nlay+2)           ! temporary upward shortwave flux (w/m2)
      real(kind=rb) :: zbbfd(nlay+2)           ! temporary downward shortwave flux (w/m2)
      real(kind=rb) :: zbbcu(nlay+2)           ! temporary clear sky upward shortwave flux (w/m2)
      real(kind=rb) :: zbbcd(nlay+2)           ! temporary clear sky downward shortwave flux (w/m2)
      real(kind=rb) :: zbbfddir(nlay+2)        ! temporary downward direct shortwave flux (w/m2)
      real(kind=rb) :: zbbcddir(nlay+2)        ! temporary clear sky downward direct shortwave flux (w/m2)
      real(kind=rb) :: zuvfd(nlay+2)           ! temporary UV downward shortwave flux (w/m2)
      real(kind=rb) :: zuvcd(nlay+2)           ! temporary clear sky UV downward shortwave flux (w/m2)
      real(kind=rb) :: zuvfddir(nlay+2)        ! temporary UV downward direct shortwave flux (w/m2)
      real(kind=rb) :: zuvcddir(nlay+2)        ! temporary clear sky UV downward direct shortwave flux (w/m2)
      real(kind=rb) :: znifd(nlay+2)           ! temporary near-IR downward shortwave flux (w/m2)
      real(kind=rb) :: znicd(nlay+2)           ! temporary clear sky near-IR downward shortwave flux (w/m2)
      real(kind=rb) :: znifddir(nlay+2)        ! temporary near-IR downward direct shortwave flux (w/m2)
      real(kind=rb) :: znicddir(nlay+2)        ! temporary clear sky near-IR downward direct shortwave flux (w/m2)
      real(kind=rb) :: zbbclnu(nlay+2)         ! temporary clean sky upward shortwave flux (w/m2)
      real(kind=rb) :: zbbclnd(nlay+2)         ! temporary clean sky downward shortwave flux (w/m2)
      real(kind=rb) :: zbbclnddir(nlay+2)      ! temporary clean sky downward direct shortwave flux (w/m2)
      real(kind=rb) :: zuvclnd(nlay+2)         ! temporary clean sky UV downward shortwave flux (w/m2)
      real(kind=rb) :: zuvclnddir(nlay+2)      ! temporary clean sky UV downward direct shortwave flux (w/m2)
      real(kind=rb) :: zniclnd(nlay+2)         ! temporary clean sky near-IR downward shortwave flux (w/m2)
      real(kind=rb) :: zniclnddir(nlay+2)      ! temporary clean sky near-IR downward direct shortwave flux (w/m2)

! Optional output fields
      real(kind=rb) :: swnflx(nlay+2)          ! Total sky shortwave net flux (W/m2)
      real(kind=rb) :: swnflxc(nlay+2)         ! Clear sky shortwave net flux (W/m2)
      real(kind=rb) :: dirdflux(nlay+2)        ! Direct downward shortwave surface flux
      real(kind=rb) :: difdflux(nlay+2)        ! Diffuse downward shortwave surface flux
      real(kind=rb) :: uvdflx(nlay+2)          ! Total sky downward shortwave flux, UV/vis
      real(kind=rb) :: nidflx(nlay+2)          ! Total sky downward shortwave flux, near-IR
      real(kind=rb) :: dirdnuv(nlay+2)         ! Direct downward shortwave flux, UV/vis
      real(kind=rb) :: difdnuv(nlay+2)         ! Diffuse downward shortwave flux, UV/vis
      real(kind=rb) :: dirdnir(nlay+2)         ! Direct downward shortwave flux, near-IR
      real(kind=rb) :: difdnir(nlay+2)         ! Diffuse downward shortwave flux, near-IR

! Output - inactive
!      real(kind=rb) :: zuvfu(nlay+2)          ! temporary upward UV shortwave flux (w/m2)
!      real(kind=rb) :: zuvfd(nlay+2)          ! temporary downward UV shortwave flux (w/m2)
!      real(kind=rb) :: zuvcu(nlay+2)          ! temporary clear sky upward UV shortwave flux (w/m2)
!      real(kind=rb) :: zuvcd(nlay+2)          ! temporary clear sky downward UV shortwave flux (w/m2)
!      real(kind=rb) :: zvsfu(nlay+2)          ! temporary upward visible shortwave flux (w/m2)
!      real(kind=rb) :: zvsfd(nlay+2)          ! temporary downward visible shortwave flux (w/m2)
!      real(kind=rb) :: zvscu(nlay+2)          ! temporary clear sky upward visible shortwave flux (w/m2)
!      real(kind=rb) :: zvscd(nlay+2)          ! temporary clear sky downward visible shortwave flux (w/m2)
!      real(kind=rb) :: znifu(nlay+2)          ! temporary upward near-IR shortwave flux (w/m2)
!      real(kind=rb) :: znifd(nlay+2)          ! temporary downward near-IR shortwave flux (w/m2)
!      real(kind=rb) :: znicu(nlay+2)          ! temporary clear sky upward near-IR shortwave flux (w/m2)
!      real(kind=rb) :: znicd(nlay+2)          ! temporary clear sky downward near-IR shortwave flux (w/m2)


! Initializations

      iout = 0 !BSINGH(PNNL) initializing iout to zero(Might be wrong!) as this variable is never initialized but used
↪  in spcvmc_sw
      zepsec = 1.e-06_rb
      zepzen = 1.e-10_rb
!jm not thread safe      oneminus = 1.0_rb - zepsec
!jm not thread safe      pi = 2._rb * asin(1._rb)

      istart = jpb1
      iend = jpb2
      icpr = 0
      ims = 2

! In a GCM with or without McICA, set nlon to the longitude dimension
!
! Set imca to select calculation type:
!  imca = 0, use standard forward model calculation (clear and overcast only)
!  imca = 1, use McICA for Monte Carlo treatment of sub-grid cloud variability
!            (clear, overcast or partial cloud conditions)

! *** This version uses McICA (imca = 1) ***
```

```
L09310
L09311  ! Set icld to select of clear or cloud calculation and cloud
L09312  ! overlap method (read by subroutine readprof from input file INPUT_RRTM):
L09313  ! icld = 0, clear only
L09314  ! icld = 1, with clouds using random cloud overlap (McICA only)
L09315  ! icld = 2, with clouds using maximum/random cloud overlap (McICA only)
L09316  ! icld = 3, with clouds using maximum cloud overlap (McICA only)
L09317  ! icld = 4, with clouds using exponential cloud overlap (McICA only)
L09318  ! icld = 5, with clouds using exponential/random cloud overlap (McICA only)
L09319
L09320  ! Set iaer to select aerosol option
L09321  ! iaer = 0, no aerosols
L09322  ! iaer = 6, use six ECMWF aerosol types
L09323  !          input aerosol optical depth at 0.55 microns for each aerosol type (ecaer)
L09324  ! iaer = 10, input total aerosol optical depth, single scattering albedo
L09325  !           and asymmetry parameter (tauaer, ssaaer, asmaer) directly
L09326        if ( aer_opt.eq.0 .or. aer_opt.eq.2 .or. aer_opt.eq.3) then
L09327        iaer = 10
L09328        else if ( aer_opt .eq. 1 ) then
L09329        iaer = 6
L09330        endif
L09331
L09332  ! Call model and data initialization, compute lookup tables, perform
L09333  ! reduction of g-points from 224 to 112 for input absorption
L09334  ! coefficient data and other arrays.
L09335  !
L09336  ! In a GCM this call should be placed in the model initialization
L09337  ! area, since this has to be called only once.
L09338  !      call rrtmg_sw_ini(cpdair)
L09339
L09340  ! This is the main longitude/column loop in RRTMG.
L09341  ! Modify to loop over all columns (nlon) or over daylight columns
L09342
L09343        do iplon = 1, ncol
L09344
L09345  ! Prepare atmosphere profile from GCM for use in RRTMG, and define
L09346  ! other input parameters
L09347
L09348           call inatm_sw (iplon, nlay, icld, iaer, &
L09349                play, plev, tlay, tlev, tsfc, h2ovmr, &
L09350                o3vmr, co2vmr, ch4vmr, n2ovmr, o2vmr, &
L09351                adjes, dyofyr, scon, inflgsw, iceflgsw, liqflgsw, &
L09352                cldfmcl, taucmcl, ssacmcl, asmcmcl, fsfcmcl, ciwpmcl, clwpmcl, cswpmcl, &
L09353                reicmcl, relqmcl, resnmcl, tauaer, ssaaer, asmaer, &
L09354                nlayers, pavel, pz, pdp, tavel, tz, tbound, coldry, wkl, &
L09355                adjflux, solvar, inflag, iceflag, liqflag, cldfmc, taucmc, &
L09356                ssacmc, asmcmc, fsfcmc, ciwpmc, clwpmc, cswpmc, reicmc, relqmc, resnmc, &
L09357                taua, ssaa, asma)
L09358
L09359  !  For cloudy atmosphere, use cldprop to set cloud optical properties based on
L09360  !  input cloud physical properties.  Select method based on choices described
L09361  !  in cldprop.  Cloud fraction, water path, liquid droplet and ice particle
L09362  !  effective radius must be passed in cldprop.  Cloud fraction and cloud
L09363  !  optical properties are transferred to rrtmg_sw arrays in cldprop.
L09364
L09365           call cldprmc_sw(nlayers, inflag, iceflag, liqflag, cldfmc, &
L09366                           ciwpmc, clwpmc, cswpmc, reicmc, relqmc, resnmc, &
L09367                           taormc, taucmc, ssacmc, asmcmc, fsfcmc)
L09368           icpr = 1
L09369
L09370  ! Calculate coefficients for the temperature and pressure dependence of the
L09371  ! molecular absorption coefficients by interpolating data from stored
L09372  ! reference atmospheres.
L09373
L09374           call setcoef_sw(nlayers, pavel, tavel, pz, tz, tbound, coldry, wkl, &
L09375                           laytrop, layswtch, laylow, jp, jt, jt1, &
L09376                           co2mult, colch4, colco2, colh2o, colmol, coln2o, &
L09377                           colo2, colo3, fac00, fac01, fac10, fac11, &
L09378                           selffac, selffrac, indself, forfac, forfrac, indfor)
L09379
L09380
L09381  ! Cosine of the solar zenith angle
L09382  !  Prevent using value of zero; ideally, SW model is not called from host model when sun
L09383  !  is below horizon
L09384
L09385           cossza = coszen(iplon)
L09386           if (cossza .le. zepzen) cossza = zepzen
L09387
L09388  ! Transfer albedo, cloud and aerosol properties into arrays for 2-stream radiative transfer
L09389
L09390  ! Surface albedo
L09391  !  Near-IR bands 16-24 and 29 (1-9 and 14), 820-16000 cm-1, 0.625-12.195 microns
L09392           do ib=1,9
L09393              albdir(ib) = aldir(iplon)
L09394              albdif(ib) = aldif(iplon)
L09395           enddo
L09396           albdir(nbndsw) = aldir(iplon)
L09397           albdif(nbndsw) = aldif(iplon)
L09398  !  UV/visible bands 25-28 (10-13), 16000-50000 cm-1, 0.200-0.625 micron
L09399           do ib=10,13
L09400              albdir(ib) = asdir(iplon)
L09401              albdif(ib) = asdif(iplon)
L09402           enddo
L09403
L09404
L09405  ! Clouds
L09406           if (icld.eq.0) then
L09407
L09408              zcldfmc(:,:) = 0._rb
L09409              ztaucmc(:,:) = 0._rb
L09410              ztaormc(:,:) = 0._rb
L09411              zasycmc(:,:) = 0._rb
L09412              zomgcmc(:,:) = 1._rb
L09413
L09414           elseif (icld.ge.1) then
L09415              do i=1,nlayers
L09416                 do ig=1,ngptsw
L09417                    zcldfmc(i,ig) = cldfmc(ig,i)
L09418                    ztaucmc(i,ig) = taucmc(ig,i)
```

```
L09419                     ztaormc(i,ig) = taormc(ig,i)
L09420                     zasycmc(i,ig) = asmcmc(ig,i)
L09421                     zomgcmc(i,ig) = ssacmc(ig,i)
L09422                  enddo
L09423               enddo
L09424
L09425            endif
L09426
L09427  ! Aerosol
L09428  ! IAER = 0: no aerosols
L09429            if (iaer.eq.0) then
L09430
L09431               ztaua(:,:) = 0._rb
L09432               zasya(:,:) = 0._rb
L09433               zomga(:,:) = 1._rb
L09434
L09435  ! IAER = 6: Use ECMWF six aerosol types. See rrsw_aer.f90 for details.
L09436  ! Input aerosol optical thickness at 0.55 micron for each aerosol type (ecaer),
L09437  ! or set manually here for each aerosol and layer.
L09438            elseif (iaer.eq.6) then
L09439
L09440  !             do i = 1, nlayers
L09441  !                do ia = 1, naerec
L09442  !                   ecaer(iplon,i,ia) = 1.0e-15_rb
L09443  !                enddo
L09444  !             enddo
L09445
L09446               do ib = 1, nbndsw
L09447                  do i = 1, nlayers
L09448                     ztaua(i,ib) = 0._rb
L09449                     zasya(i,ib) = 0._rb
L09450                     zomga(i,ib) = 0._rb
L09451                     do ia = 1, naerec
L09452                        ztaua(i,ib) = ztaua(i,ib) + rsrtaua(ib,ia) * ecaer(iplon,i,ia)
L09453                        zomga(i,ib) = zomga(i,ib) + rsrtaua(ib,ia) * ecaer(iplon,i,ia) * &
L09454                                      rsrpiza(ib,ia)
L09455                        zasya(i,ib) = zasya(i,ib) + rsrtaua(ib,ia) * ecaer(iplon,i,ia) * &
L09456                                      rsrpiza(ib,ia) * rsrasya(ib,ia)
L09457                     enddo
L09458                     if (zomga(i,ib) /= 0._rb) then
L09459                        zasya(i,ib) = zasya(i,ib) / zomga(i,ib)
L09460                     endif
L09461                     if (ztaua(i,ib) /= 0._rb) then
L09462                        zomga(i,ib) = zomga(i,ib) / ztaua(i,ib)
L09463                     endif
L09464                  enddo
L09465               enddo
L09466
L09467  ! IAER=10: Direct specification of aerosol optical properties from GCM
L09468            elseif (iaer.eq.10) then
L09469
L09470               do ib = 1 ,nbndsw
L09471                  do i = 1 ,nlayers
L09472                     ztaua(i,ib) = taua(i,ib)
L09473                     ztauacln(i,ib) = 0.0
L09474                     zasya(i,ib) = asma(i,ib)
L09475                     zomga(i,ib) = ssaa(i,ib)
L09476                  enddo
L09477               enddo
L09478
L09479            endif
L09480
L09481
L09482  ! Call the 2-stream radiation transfer model
L09483
L09484            do i=1,nlayers+1
L09485               zbbcu(i) = 0._rb
L09486               zbbcd(i) = 0._rb
L09487               zbbfu(i) = 0._rb
L09488               zbbfd(i) = 0._rb
L09489               zbbcddir(i) = 0._rb
L09490               zbbfddir(i) = 0._rb
L09491               zuvcd(i) = 0._rb
L09492               zuvfd(i) = 0._rb
L09493               zuvcddir(i) = 0._rb
L09494               zuvfddir(i) = 0._rb
L09495               znicd(i) = 0._rb
L09496               znifd(i) = 0._rb
L09497               znicddir(i) = 0._rb
L09498               znifddir(i) = 0._rb
L09499            enddo
L09500
L09501            call spcvmc_sw &
L09502                (nlayers, istart, iend, icpr, iout, &
L09503                 pavel, tavel, pz, tz, tbound, albdif, albdir, &
L09504                 zcldfmc, ztaucmc, zasycmc, zomgcmc, ztaormc, &
L09505                 ztaua, zasya, zomga, cossza, coldry, wkl, adjflux, &
L09506                 laytrop, layswtch, laylow, jp, jt, jt1, &
L09507                 co2mult, colch4, colco2, colh2o, colmol, coln2o, colo2, colo3, &
L09508                 fac00, fac01, fac10, fac11, &
L09509                 selffac, selffrac, indself, forfac, forfrac, indfor, &
L09510                 zbbfd, zbbfu, zbbcd, zbbcu, zuvfd, zuvcd, znifd, znicd, &
L09511                 zbbfddir, zbbcddir, zuvfddir, zuvcddir, znifddir, znicddir)
L09512
L09513  ! Transfer up and down, clear and total sky fluxes to output arrays.
L09514  ! Vertical indexing goes from bottom to top; reverse here for GCM if necessary.
L09515
L09516            do i = 1, nlayers+1
L09517               swuflxc(iplon,i) = zbbcu(i)
L09518               swdflxc(iplon,i) = zbbcd(i)
L09519               swuflx(iplon,i) = zbbfu(i)
L09520               swdflx(iplon,i) = zbbfd(i)
L09521               uvdflx(i) = zuvfd(i)
L09522               nidflx(i) = znifd(i)
L09523
L09524  !  Direct/diffuse fluxes
L09525               dirdflux(i) = zbbfddir(i)
L09526               difdflux(i) = swdflx(iplon,i) - dirdflux(i)
```

```
L09527              swdkdir(iplon,i) = dirdflux(i) ! all-sky direct flux   jararias, 2013/08/10
L09528              swdkdif(iplon,i) = difdflux(i) ! all-sky diffuse flux  jararias, 2013/08/10
L09529              swdkdirc(iplon,i) = zbbcddir(i) ! PAJ: clear-sky direct flux
L09530
L09531  !  UV/visible direct/diffuse fluxes
L09532              dirdnuv(i) = zuvfddir(i)
L09533              difdnuv(i) = zuvfd(i) - dirdnuv(i)
L09534  !  ------- Zhenxin add vis/uv downwards dir or dif here --!
L09535              sibvisdir(iplon,i) = dirdnuv(i)
L09536              sibvisdif(iplon,i) = difdnuv(i)
L09537  !  ----- End of Zhenxin addition  ------------!
L09538  !  Near-IR direct/diffuse fluxes
L09539              dirdnir(i) = znifddir(i)
L09540              difdnir(i) = znifd(i) - dirdnir(i)
L09541  !  ---------Zhenxin add nir downwards dir and dif here --!
L09542              sibnirdir(iplon,i) = dirdnir(i)
L09543              sibnirdif(iplon,i) = difdnir(i)
L09544  !  --------    End of Zhenxin addition 2011-05  ---------!
L09545           enddo
L09546
L09547  !  Total and clear sky net fluxes
L09548           do i = 1, nlayers+1
L09549              swnflxc(i) = swdflxc(iplon,i) - swuflxc(iplon,i)
L09550              swnflx(i) = swdflx(iplon,i) - swuflx(iplon,i)
L09551           enddo
L09552
L09553  !  Total and clear sky heating rates
L09554           do i = 1, nlayers
L09555              zdpgcp = heatfac / pdp(i)
L09556              swhrc(iplon,i) = (swnflxc(i+1) - swnflxc(i)) * zdpgcp
L09557              swhr(iplon,i) = (swnflx(i+1) - swnflx(i)) * zdpgcp
L09558           enddo
L09559           swhrc(iplon,nlayers) = 0._rb
L09560           swhr(iplon,nlayers) = 0._rb
L09561
L09562  #if (WRF_CHEM == 1)
L09563           !  Repeat call to 2-stream radiation model using "clean sky"
L09564           !  variables and aerosol tau set to 0
L09565           if(calc_clean_atm_diag .gt. 0)then
L09566              do i=1,nlayers+1
L09567                  zbbcu(i) = 0._rb
L09568                  zbbcd(i) = 0._rb
L09569                  zbbclnu(i) = 0._rb
L09570                  zbbclnd(i) = 0._rb
L09571                  zbbcddir(i) = 0._rb
L09572                  zbbclnddir(i) = 0._rb
L09573                  zuvcd(i) = 0._rb
L09574                  zuvclnd(i) = 0._rb
L09575                  zuvcddir(i) = 0._rb
L09576                  zuvclnddir(i) = 0._rb
L09577                  znicd(i) = 0._rb
L09578                  zniclnd(i) = 0._rb
L09579                  znicddir(i) = 0._rb
L09580                  zniclnddir(i) = 0._rb
L09581               enddo
L09582
L09583               call spcvmc_sw &
L09584                   (nlayers, istart, iend, icpr, iout, &
L09585                    pavel, tavel, pz, tz, tbound, albdif, albdir, &
L09586                    zcldfmc, ztaucmc, zasycmc, zomgcmc, ztaormc, &
L09587                    ztauacln, zasya, zomga, cossza, coldry, wkl, adjflux, &
L09588                    laytrop, layswtch, laylow, jp, jt, jt1, &
L09589                    co2mult, colch4, colco2, colh2o, colmol, coln2o, colo2, colo3, &
L09590                    fac00, fac01, fac10, fac11, &
L09591                    selffac, selffrac, indself, forfac, forfrac, indfor, &
L09592                    zbbclnd, zbbclnu, zbbcd, zbbcu, zuvclnd, zuvcd, zniclnd, znicd, &
L09593                    zbbclnddir, zbbcddir, zuvclnddir, zuvcddir, zniclnddir, znicddir)
L09594
L09595              do i = 1, nlayers+1
L09596                 swuflxcln(iplon,i) = zbbclnu(i)
L09597                 swdflxcln(iplon,i) = zbbclnd(i)
L09598              enddo
L09599           else
L09600              do i = 1, nlayers+1
L09601                 swuflxcln(iplon,i) = 0.0
L09602                 swdflxcln(iplon,i) = 0.0
L09603              enddo
L09604           end if
L09605
L09606  #else
L09607           do i = 1, nlayers+1
L09608              swuflxcln(iplon,i) = 0.0
L09609              swdflxcln(iplon,i) = 0.0
L09610           enddo
L09611
L09612  #endif
L09613  ! End longitude loop
L09614        enddo
L09615
L09616  ! begin WRF-CMAQ twoway coupled model block
L09617        sw_zbbcddir = zbbcddir(1)
L09618        sw_dirdflux = dirdflux(1)
L09619        sw_difdflux = difdflux(1)
L09620  ! end WRF-CMAQ twoway coupled model block
L09621
L09622        end subroutine rrtmg_sw
```

讲解：与 rrtmg_lw 平行但三处本质不同：① **散射三参数**——云给 taucmcl/ssacmcl/asmcmcl/fsfcmcl 四件（光学深度/SSA/不对称因子/前向散射分数）、气溶胶给 tauaer/ssaaer/asmaer + ecaer（iaer=6 的 ECMWF 六类 0.55 μm AOD）；② **两流传输**——调 spcvmc_sw 而非 rtrnmc（Barker 两流 + McICA，见 § 八）；③ **反照率谱分解**（L9390–9402）：近红外波段 1–9 与 14（即全局 16–24、29）用 aldir/aldif，UV/可见波段 10–13（全局 25–28）用 asdir/asdif——**四分量反照率输入的谱段归属在这里定死**（SSiB/太阳能应用的依据）。其余：imca=1 McICA；**iaer 由 aer_opt 映射**（0/2/3→10 直接输入

三参数、1→6 ECMWF 六类）；iaer=6 分支的光学量加权混合公式 ztaua=Σrsrtaua • ecaer、zomga/zasya 归一化；加热率 zdpgcp=heatfac/pdp（气压厚度倒数）；WRF_CHEM 下**第二次 spcvmc_sw 调用**（ztauacln=0 的 clean 版）；直散分解 zbbfddir → swdkdir/swdkdif/swdkdirc 与 SSiB 四分量 sibvisdir/sibvisdif/sibnirdir/sibnirdif（Zhenxin 2011）。

五、WRF 入口 RRTMG_SWRAD（三段）

```
!file: phys/module_ra_rrtmg_sw.F    L10550-L10621
L10550  !-----CALCULATE SHORT WAVE RADIATION
L10551  !
L10552  ! Read time-varying trace gases concentrations and interpolate them to run date
L10553     IF ( GHG_INPUT .EQ. 1 ) THEN
L10554        CALL read_CAMgases(yr,julian,.false.,"RRTMG",co2,n2o,ch4,cfc11,cfc12)
L10555        IF ( wrf_dm_on_monitor() ) THEN
L10556          WRITE(message,*)'RRTMG SW CLWRF interpolated GHG values year:',yr,' julian day:',julian
L10557          call wrf_debug( 1, message)
L10558          WRITE(message,*)'  co2vmr: ',co2,' n2ovmr:',n2o,' ch4vmr:',ch4,' cfc11vmr:',cfc11,' cfc12vmr:',cfc12
L10559          call wrf_debug( 1, message)
L10560        END IF
L10561     ELSE
L10562  ! Set trace gas volume mixing ratios, 2005 values, IPCC (2007)
L10563  ! Annual function for co2 in WRF v4.2
L10564        co2 = (280. + 90.*exp(0.02*(yr-2000)))*1.e-6
L10565  !     co2 = 379.e-6
L10566        ch4 = 1774.e-9
L10567        n2o = 319.e-9
L10568     END IF
L10569
L10570  ! All fields are ordered vertically from bottom to top
L10571  ! Pressures are in mb
L10572
L10573  ! latitude loop
L10574    j_loop: do j = jts,jte
L10575
L10576  ! longitude loop
L10577       i_loop: do i = its,ite
L10578           rho1d(kts:kte)=rho3d(i,kts:kte,j) ! BUG FIX (SGT): this was uninitialized
L10579  !
L10580  ! Do shortwave by default, deactivate below if sun below horizon
L10581           dorrsw = .true.
L10582
L10583  ! Cosine solar zenith angle for current time step
L10584  !
L10585            ! jararias, 14/08/2013
L10586            coszr(i,j)=xcoszen(i,j)
L10587            coszrs=xcoszen(i,j)
L10588
L10589  ! Set flag to prevent shortwave calculation when sun below horizon
L10590           if (coszrs.le.0.0) dorrsw = .false.
L10591  ! Perform shortwave calculation if sun above horizon
L10592           if (dorrsw) then
L10593
L10594           do k=kts,kte+1
L10595              Pw1D(K) = p8w(I,K,J)/100.
L10596              Tw1D(K) = t8w(I,K,J)
L10597           enddo
L10598
L10599           DO K=kts,kte
L10600              QV1D(K)=0.
L10601              QC1D(K)=0.
L10602              QR1D(K)=0.
L10603              QI1D(K)=0.
L10604              QS1D(K)=0.
L10605              CLDFRA1D(k)=0.
L10606              QNDROP1D(k)=0.
L10607           ENDDO
L10608
L10609           DO K=kts,kte
L10610              QV1D(K)=QV3D(I,K,J)
L10611              QV1D(K)=max(0.,QV1D(K))
L10612           ENDDO
L10613
L10614           IF (o3input.eq.2) THEN
L10615              DO K=kts,kte
L10616                 O31D(K)=O33D(I,K,J)
L10617              ENDDO
L10618           ELSE
L10619              DO K=kts,kte
L10620                 O31D(K)=0.0
L10621              ENDDO
```

讲解：①**ghg_input 分支**：=1 时 read_CAMgases 读时变气体；否则 **CO2 年增长函数 co2=(280+90 · exp(0.02 · (yr−2000))) · 1e-6 ppm**（WRF v4.2 加入——280 起点与 2%/年指数；ch4=1774 ppb、n2o=319 ppb 是 2005 值）——**SW 侧 CO2 有年变化、LW 侧是固定值**，两文件口径不对称（历史遗留，改气体实验要注意两侧同步）；② **昼夜门控 dorrsw**：coszrs=xcoszen（radiation_driver 传来的均时差修正天顶角），≤0 跳过短波（coszen 在核心内钳 zepzen=1e-10 防除零）；③O31D 只在 o3input=2 时吃外场、否则零——SW 臭氧默认零。

```
!file: phys/module_ra_rrtmg_sw.F    L11220-L11231
L11220           iplon = 1
L11221           irng = 0
L11222           permuteseed = 1
L11223
L11224  ! Sub-column generator for McICA
```

```
L11225           lat = XLAT(i,j) !retrieve scalar latitude for column calculation
L11226           call mcica_subcol_sw(iplon, ncol, nlay, icld, permuteseed, irng, play, &
L11227                        cldfrac, ciwpth, clwpth, cswpth, rei, rel, res, taucld, ssacld, asmcld, fsfcld, &
L11228                        hgt, idcor, juldat, lat, &
L11229                        cldfmcl, ciwpmcl, clwpmcl, cswpmcl, reicmcl, relqmcl, resnmcl, &
L11230                        taucmcl, ssacmcl, asmcmcl, fsfcmcl)
L11231
```

讲解：**McICA 次列生成**：permuteseed=1（LW 是 150——两方案种子不同，随机噪声不可比）、irng=0（KISSVEC）；与 LW 版的差异是云光学带散射四件（taucld/ssacld/asmcld/fsfcld）一起进次列。

```
!file: phys/module_ra_rrtmg_sw.F    L11353-L11432
L11353  #if ( WRF_CHEM == 1 )
L11354     IF ( AER_RA_FEEDBACK == 1) then
L11355        do nb = 1, nbndsw
L11356           wavemid(nb)=0.5*(wavemin(nb)+wavemax(nb))  ! um
L11357        do k = kts,kte      !wig
L11358
L11359  ! convert optical properties at 300,400,600, and 999 to conform to the band wavelengths
L11360  ! tauaer - use angstrom exponent
L11361           if(tauaer300(i,k,j).gt.thresh .and. tauaer999(i,k,j).gt.thresh) then
L11362              ang=alog(tauaer300(i,k,j)/tauaer999(i,k,j))/alog(999./300.)
L11363              tauaer(ncol,k,nb)=tauaer400(i,k,j)*(0.4/wavemid(nb))**ang
L11364              !tauaer(ncol,k,nb)=tauaer600(i,k,j)*(0.6/wavemid(nb))**ang
L11365              if (i==30.and.j==49.and.k==2.and.nb==12) then
L11366               write(0,*) 'TAU from 600 vs 400 in RRTMG',tauaer600(i,k,j),tauaer400(i,k,j)
L11367               print*, 'TAU from 600 vs 400 in RRTMG',tauaer600(i,k,j),tauaer400(i,k,j)
L11368               write(0,*) tauaer600(i,k,j)*(0.6/wavemid(nb))**ang,tauaer400(i,k,j)*(0.4/wavemid(nb))**ang
L11369               print*, tauaer600(i,k,j)*(0.6/wavemid(nb))**ang,tauaer400(i,k,j)*(0.4/wavemid(nb))**ang
L11370              endif
L11371  ! ssa - linear interpolation; extrapolation
L11372              slope=(waer600(i,k,j)-waer400(i,k,j))/.2
L11373              ssaaer(ncol,k,nb) = slope*(wavemid(nb)-.6)+waer600(i,k,j)
L11374              if(ssaaer(ncol,k,nb).lt.0.4) ssaaer(ncol,k,nb)=0.4
L11375              if(ssaaer(ncol,k,nb).ge.1.0) ssaaer(ncol,k,nb)=1.0
L11376  ! g - linear interpolation;extrapolation
L11377              slope=(gaer600(i,k,j)-gaer400(i,k,j))/.2
L11378              asmaer(ncol,k,nb) = slope*(wavemid(nb)-.6)+gaer600(i,k,j) ! notice reversed varaibles
L11379              if(asmaer(ncol,k,nb).lt.0.5) asmaer(ncol,k,nb)=0.5
L11380              if(asmaer(ncol,k,nb).ge.1.0) asmaer(ncol,k,nb)=1.0
L11381           endif
L11382        end do ! k
L11383        end do ! nb
L11384
L11385  !wig beg
L11386        do nb = 1, nbndsw
L11387           slope = 0.  !use slope as a sum holder
L11388           do k = kts,kte
L11389              slope = slope + tauaer(ncol,k,nb)
L11390           end do
L11391           if( slope < 0. ) then
L11392              write(msg,'("ERROR: Negative total optical depth of ",f8.2," at point i,j,nb=",3i5)') slope,i,j,nb
L11393              call wrf_error_fatal(msg)
L11394           else if( slope > 6. ) then
L11395              call wrf_message("--------------------------")
L11396              write(msg,'("WARNING: Large total sw optical depth of ",f8.2," at point i,j,nb=",3i5)') slope,i,j,nb
L11397              call wrf_message(msg)
L11398
L11399              call wrf_message("Diagnostics 1: k, tauaer300, tauaer400, tauaer600, tauaer999, tauaer")
L11400              do k=kts,kte
L11401                 write(msg,'(i4,5f8.2)') k, tauaer300(i,k,j), tauaer400(i,k,j), &
L11402                      tauaer600(i,k,j), tauaer999(i,k,j),tauaer(ncol,k,nb)
L11403                 call wrf_message(msg)
L11404                 !czhao set an up-limit here to avoid segmentation fault
L11405                 !from extreme AOD
L11406                 tauaer(ncol,k,nb)=tauaer(ncol,k,nb)*6.0/slope
L11407              end do
L11408
L11409              call wrf_message("Diagnostics 2: k, gaer300, gaer400, gaer600, gaer999")
L11410              do k=kts,kte
L11411                 write(msg,'(i4,4f8.2)') k, gaer300(i,k,j), gaer400(i,k,j), &
L11412                      gaer600(i,k,j), gaer999(i,k,j)
L11413                 call wrf_message(msg)
L11414              end do
L11415
L11416              call wrf_message("Diagnostics 3: k, waer300, waer400, waer600, waer999")
L11417              do k=kts,kte
L11418                 write(msg,'(i4,4f8.2)') k, waer300(i,k,j), waer400(i,k,j), &
L11419                      waer600(i,k,j), waer999(i,k,j)
L11420                 call wrf_message(msg)
L11421              end do
L11422
L11423              call wrf_message("Diagnostics 4: k, ssaal, asyal, taual")
L11424              do k=kts-1,kte
L11425                 write(msg,'(i4,3f8.2)') k, ssaaer(i,k,nb), asmaer(i,k,nb), tauaer(i,k,nb)
L11426                 call wrf_message(msg)
L11427              end do
L11428              call wrf_message("--------------------------")
L11429           endif
L11430        enddo  ! nb
L11431        endif  ! aer_ra_feedback
L11432  #endif
```

讲解：**WRF_CHEM 气溶胶 4 波长 →14 波段展开**：AOD 用 Angstrom 指数律 ang=ln(τ300/τ999)/ln(999/300)，各波段 τ(nb)=τ400 • (0.4/λmid)^ang（λmid 为波段中点波长）；SSA 与不对称因子在 400–600 nm 线性内插/外推并钳位（SSA [0.4,1]、g [0.5,1]）；**守卫**：柱总 AOD<0 FATAL、>6 时打印全套诊断并**按比例钳到 6**（tauaer • 6/slope——防极端 AOD 段错误）。非化学编译时 tauaer 清零（ssaaer=1、asmaer=0），走 aer_opt=1/2/3 的 tauaer3d_sw 通道（associated 检查——radiation_driver 分配后才非空）。

```
!file: phys/module_ra_rrtmg_sw.F    L11443-L11540
L11443        IF ( PRESENT( aerod ) ) THEN
L11444        if ( aer_opt .eq. 0 ) then
L11445           do na = 1, naerec
L11446              do k = kts, kte+1
L11447                 ecaer(ncol,k,na) = 0.
L11448              enddo
L11449           enddo
L11450        else if ( aer_opt .eq. 1 ) then
L11451           do na = 1, naerec
L11452              do k = kts, kte
L11453                 ecaer(ncol,k,na) = aerod(i,k,j,na)
L11454              enddo
L11455  ! assuming 0 or same value at the top?
L11456  !           ecaer(ncol,kte+1,na) = ecaer(ncol,kte,na)
L11457              ecaer(ncol,kte+1,na) = 0.
L11458           enddo
L11459        endif
L11460        ENDIF
L11461
L11462  ! Call RRTMG shortwave radiation model
L11463
L11464           call rrtmg_sw &
L11465              (ncol    ,nlay    ,icld    , &
L11466               play    ,plev    ,tlay    ,tlev    ,tsfc   , &
L11467               h2ovmr , o3vmr   ,co2vmr  ,ch4vmr  ,n2ovmr ,o2vmr , &
L11468               asdir   ,asdif   ,aldir   ,aldif   , &
L11469               coszen  ,adjes   ,dyofyr  ,scon    , &
L11470               inflgsw ,iceflgsw,liqflgsw,cldfmcl , &
L11471               taucmcl ,ssacmcl ,asmcmcl ,fsfcmcl , &
L11472               ciwpmcl ,clwpmcl ,cswpmcl, reicmcl ,relqmcl ,resnmcl, &
L11473               tauaer  ,ssaaer  ,asmaer  ,ecaer   , &
L11474               swuflx  ,swdflx  ,swhr    ,swuflxc ,swdflxc ,swhrc, swuflxcln, swdflxcln, aer_opt, &
L11475  ! -----      Zhenxin added for ssib coupiling 2011-06-20 --------!
L11476               sibvisdir, sibvisdif, sibnirdir, sibnirdif,         &
L11477  ! --------------------   End of addiation by Zhenxin 2011-06-20 ------!
L11478               swdkdir, swdkdif,                      &  ! jararias, 2012/08/10
L11479               swdkdirc                               &  ! PAJ
L11480               ,calc_clean_atm_diag                  &
L11481               ,loc_sw_zbbcddir                       &  ! WRF-CMAQ twoway coupled model
L11482               ,loc_sw_dirdflux                       &  ! WRF-CMAQ twoway coupled model
L11483               ,loc_sw_difdflux                       &  ! WRF-CMAQ twoway coupled model
L11484                                                      )
L11485
L11486           ! WRF-CMAQ twoway coupled model
L11487           if (present(sw_zbbcddir)) then
L11488              sw_zbbcddir(i,j) = loc_sw_zbbcddir
L11489              sw_dirdflux(i,j) = loc_sw_dirdflux
L11490              sw_difdflux(i,j) = loc_sw_difdflux
L11491           end if
L11492
L11493  ! Output net absorbed shortwave surface flux and shortwave cloud forcing
L11494  ! at the top of atmosphere (W/m2)
L11495           gsw(i,j) = swdflx(1,1) - swuflx(1,1)
L11496           swcf(i,j) = (swdflx(1,kte+2) - swuflx(1,kte+2)) - (swdflxc(1,kte+2) - swuflxc(1,kte+2))
L11497
L11498           if (present(swupt)) then
L11499  ! Output up and down toa fluxes for total and clear sky
L11500              swupt(i,j)     = swuflx(1,kte+2)
L11501              swuptc(i,j)    = swuflxc(1,kte+2)
L11502              swdnt(i,j)     = swdflx(1,kte+2)
L11503              swdntc(i,j)    = swdflxc(1,kte+2)
L11504  ! Output up and down surface fluxes for total and clear sky
L11505              swupb(i,j)     = swuflx(1,1)
L11506              swupbc(i,j)    = swuflxc(1,1)
L11507              swdnb(i,j)     = swdflx(1,1)
L11508  ! Added by Zhenxin for 4 compenants of swdown radiation
L11509              swvisdir(i,j)  = sibvisdir(1,1)
L11510              swvisdif(i,j)  = sibvisdif(1,1)
L11511              swnirdir(i,j)  = sibnirdir(1,1)
L11512              swnirdif(i,j)  = sibnirdif(1,1)
L11513  !  Ended, Zhenxin (2011/06/20)
L11514              swdnbc(i,j)    = swdflxc(1,1)
L11515              if(calc_clean_atm_diag .gt. 0)then
L11516                  swuptcln(i,j)  = swuflxcln(1,kte+2)
L11517                  swdntcln(i,j)  = swdflxcln(1,kte+2)
L11518                  swupbcln(i,j)  = swuflxcln(1,1)
L11519                  swdnbcln(i,j)  = swdflxcln(1,1)
L11520              end if
L11521           endif
L11522              swddir(i,j)    = swdkdir(1,1)          ! jararias 2013/08/10
L11523              swddni(i,j)    = swddir(i,j) / coszrs  ! jararias 2013/08/10
L11524              swddif(i,j)    = swdkdif(1,1)          ! jararias 2013/08/10
L11525              swdownc(i, j)  = swdflxc(1,1)          ! PAJ: clear-sky GHI
L11526              swddirc(i,j)   = swdkdirc(1,1)         ! PAJ: clear-sky direct normal irradiance
L11527              swddnic(i,j)   = swddirc(i,j) / coszrs ! PAJ: clear-sky direct normal irradiance
L11528
L11529  ! Output up and down layer fluxes for total and clear sky.
L11530  ! Vertical ordering is from bottom to top in units of W m-2.
L11531           if ( present (swupflx) ) then
L11532           do k=kts,kte+2
L11533              swupflx(i,k,j)  = swuflx(1,k)
L11534              swupflxc(i,k,j) = swuflxc(1,k)
L11535              swdnflx(i,k,j)  = swdflx(1,k)
L11536              swdnflxc(i,k,j) = swdflxc(1,k)
L11537           enddo
L11538           endif
L11539
L11540  ! Output heating rate tendency; convert heating rate from K/d to K/s
```

讲解：①aer_opt=1（Tegen 气候场）把 aerod 六类 0.55 μm AOD 抄进 ecaer（顶层清零）；②**rrtmg_sw 调用与回写**：gsw=swdflx(1,1)−swuflx(1,1)——**地面净吸收短波**（即 GSW 口径：下行 − 上行）；SWCF（云辐射强迫）在 kte+2 界面取（模型顶外垫层），total−clear 差；四分量通量 swvisdir/dif、swnirdir/dif 直通 SSiB；**swddni=swddir/coszrs**——法向直射由水平直射除以 cos 天顶角（太阳能应

用的 DNI 口径）；swdownc/swddirc/swddnic 是晴空版（PAJ）。

六、云光学 cldprmc_sw（尾段：delta-M 标定）

```
!file: phys/module_ra_rrtmg_sw.F    L2399-L2498
L02399                                  (ssaliq1(index+1,ib) - ssaliq1(index,ib))
L02400                        if (fint .lt. 0._rb .and. ssacoliq(ig) .gt. 1._rb) &
L02401                             ssacoliq(ig) = ssaliq1(index,ib)
L02402                        gliq(ig) = asyliq1(index,ib) + fint * &
L02403                                   (asyliq1(index+1,ib) - asyliq1(index,ib))
L02404                        forwliq(ig) = gliq(ig)*gliq(ig)
L02405  ! Check to ensure all calculated quantities are within physical limits.
L02406                        if (extcoliq(ig) .lt. 0.0_rb) stop 'LIQUID EXTINCTION LESS THAN 0.0'
L02407                        if (ssacoliq(ig) .gt. 1.0_rb) stop 'LIQUID SSA GRTR THAN 1.0'
L02408                        if (ssacoliq(ig) .lt. 0.0_rb) stop 'LIQUID SSA LESS THAN 0.0'
L02409                        if (gliq(ig) .gt. 1.0_rb) stop 'LIQUID ASYM GRTR THAN 1.0'
L02410                        if (gliq(ig) .lt. 0.0_rb) stop 'LIQUID ASYM LESS THAN 0.0'
L02411                     endif
L02412
L02413
L02414                     if (iceflag .lt. 5) then
L02415                         tauliqorig = clwpmc(ig,lay) * extcoliq(ig)
L02416                         tauiceorig = ciwpmc(ig,lay) * extcoice(ig)
L02417                         taormc(ig,lay) = tauliqorig + tauiceorig
L02418
L02419                         ssaliq = ssacoliq(ig) * (1._rb - forwliq(ig)) / &
L02420                                  (1._rb - forwliq(ig) * ssacoliq(ig))
L02421                         tauliq = (1._rb - forwliq(ig) * ssacoliq(ig)) * tauliqorig
L02422                         ssaice = ssacoice(ig) * (1._rb - forwice(ig)) / &
L02423                                  (1._rb - forwice(ig) * ssacoice(ig))
L02424                         tauice = (1._rb - forwice(ig) * ssacoice(ig)) * tauiceorig
L02425                         scatliq = ssaliq * tauliq
L02426                         scatice = ssaice * tauice
L02427                         scatsno = 0.0_rb
L02428                         taucmc(ig,lay) = tauliq + tauice
L02429                     else
L02430                         tauliqorig = clwpmc(ig,lay) * extcoliq(ig)
L02431                         tauiceorig = ciwpmc(ig,lay) * extcoice(ig)
L02432                         tausnoorig = cswpmc(ig,lay) * extcosno(ig)
L02433                         taormc(ig,lay) = tauliqorig + tauiceorig + tausnoorig
L02434
L02435                         ssaliq = ssacoliq(ig) * (1._rb - forwliq(ig)) / &
L02436                                  (1._rb - forwliq(ig) * ssacoliq(ig))
L02437                         tauliq = (1._rb - forwliq(ig) * ssacoliq(ig)) * tauliqorig
L02438                         ssaice = ssacoice(ig) * (1._rb - forwice(ig)) / &
L02439                                  (1._rb - forwice(ig) * ssacoice(ig))
L02440                         tauice = (1._rb - forwice(ig) * ssacoice(ig)) * tauiceorig
L02441                         ssasno = ssacosno(ig) * (1._rb - forwsno(ig)) / &
L02442                                  (1._rb - forwsno(ig) * ssacosno(ig))
L02443                         tausno = (1._rb - forwsno(ig) * ssacosno(ig)) * tausnoorig
L02444                         scatliq = ssaliq * tauliq
L02445                         scatice = ssaice * tauice
L02446                         scatsno = ssasno * tausno
L02447                         taucmc(ig,lay) = tauliq + tauice + tausno
L02448                     endif
L02449
L02450  ! Ensure non-zero taucmc and scatice
L02451                     if(taucmc(ig,lay).eq.0.) taucmc(ig,lay) = cldmin
L02452                     if(scatice.eq.0.) scatice = cldmin
L02453                     if(scatsno.eq.0.) scatsno = cldmin
L02454
L02455                     if (iceflag .lt. 5) then
L02456                         ssacmc(ig,lay) = (scatliq + scatice) / taucmc(ig,lay)
L02457                     else
L02458                         ssacmc(ig,lay) = (scatliq + scatice + scatsno) / taucmc(ig,lay)
L02459                     endif
L02460
L02461                     if (iceflag .eq. 3 .or. iceflag.eq.4) then
L02462  ! In accordance with the 1996 Fu paper, equation A.3,
L02463  ! the moments for ice were calculated depending on whether using spheres
L02464  ! or hexagonal ice crystals.
L02465  ! Set asymetry parameter to first moment (istr=1)
L02466                        istr = 1
L02467                        asmcmc(ig,lay) = (1.0_rb/(scatliq+scatice))* &
L02468                           (scatliq*(gliq(ig)**istr - forwliq(ig)) / &
L02469                           (1.0_rb - forwliq(ig)) + scatice * ((gice(ig)-forwice(ig))/ &
L02470                           (1.0_rb - forwice(ig)))**istr)
L02471                     elseif (iceflag .eq. 5) then
L02472                        istr = 1
L02473                        asmcmc(ig,lay) = (1.0_rb/(scatliq+scatice+scatsno))                                  &
L02474                                       *  (scatliq*(gliq(ig)**istr - forwliq(ig))/(1.0_rb - forwliq(ig)))  &
L02475                                       + scatice * ((gice(ig)-forwice(ig))/(1.0_rb - forwice(ig)))       &
L02476                                       + scatsno * ((gsno(ig)-forwsno(ig))/(1.0_rb - forwsno(ig)))**istr)
L02477
L02478                     else
L02479  ! This code is the standard method for delta-m scaling.
L02480  ! Set asymetry parameter to first moment (istr=1)
L02481                        istr = 1
L02482                        asmcmc(ig,lay) = (scatliq *  &
L02483                           (gliq(ig)**istr - forwliq(ig)) / &
L02484                           (1.0_rb - forwliq(ig)) + scatice * (gice(ig)**istr - forwice(ig)) / &
L02485                           (1.0_rb - forwice(ig)))/(scatliq + scatice)
L02486                     endif
L02487
L02488                  endif
L02489
L02490               endif
L02491
L02492  ! End g-point interval loop
L02493            enddo
L02494
L02495  ! End layer loop
L02496         enddo
```

```
L02497
L02498        end subroutine cldprmc_sw
```

讲解：短波云光学与 LW（code_05 § 六）的本质差异全在这段：①**delta-Eddington（delta-M）标定**——原光学量先按真参数组装 taormc=clwp • extliq+ciwp • extice[+cswp • extsno]（未标定版，spcvmc 用于直射通量），再用前向散射分数 forw=g² 折算：ssaliq=ω(1−g²)/(1−ωg²)、tauliq=(1−ωg²) • τ_orig——**前向峰从相函数里剥掉**，两流方程才近似成立；② 标定后 taucmc=τliq+τice[+τsno]、**ssacmc=(散射份额和)/taucmc**、零值钳 cldmin 防病态；③**asmcmc 的三种矩混合**：iceflag=3/4 按 Fu 1996 式 (A.3)（球/六角形冰晶矩不同）、=5 加雪的三源混合、否则标准 delta-M 首矩法；④extcoice/extcoliq 等消光系数表与 LW 的 absice 表同源（rrsw_cld），越界 stop 同款老风格。头段（L2077–L2398，未录）是 inflag/iceflag/liqflag 的五选查表——与 LW cldprmc 的四选结构平行。

七、气体吸收 taumol_sw 与波段样本 taumol16

```
!file: phys/module_ra_rrtmg_sw.F    L3190-L3301
L03190        subroutine taumol_sw(nlayers, &
L03191                             colh2o, colco2, colch4, colo2, colo3, colmol, &
L03192                             laytrop, jp, jt, jt1, &
L03193                             fac00, fac01, fac10, fac11, &
L03194                             selffac, selffrac, indself, forfac, forfrac, indfor, &
L03195                             sfluxzen, taug, taur)
L03196  !----------------------------------------------------------------------------
L03197
L03198  ! ****************************************************************************
L03199  ! *                                                                          *
L03200  ! *                  Optical depths developed for the                        *
L03201  ! *                                                                          *
L03202  ! *                RAPID RADIATIVE TRANSFER MODEL (RRTM)                     *
L03203  ! *                                                                          *
L03204  ! *                                                                          *
L03205  ! *           ATMOSPHERIC AND ENVIRONMENTAL RESEARCH, INC.                   *
L03206  ! *                       131 HARTWELL AVENUE                                *
L03207  ! *                       LEXINGTON, MA 02421                                *
L03208  ! *                                                                          *
L03209  ! *                                                                          *
L03210  ! *                          ELI J. MLAWER                                   *
L03211  ! *                        JENNIFER DELAMERE                                 *
L03212  ! *                        STEVEN J. TAUBMAN                                 *
L03213  ! *                        SHEPARD A. CLOUGH                                 *
L03214  ! *                                                                          *
L03215  ! *                                                                          *
L03216  ! *                                                                          *
L03217  ! *                                                                          *
L03218  ! *                      email:  mlawer@aer.com                              *
L03219  ! *                      email:  jdelamer@aer.com                            *
L03220  ! *                                                                          *
L03221  ! *       The authors wish to acknowledge the contributions of the           *
L03222  ! *       following people:  Patrick D. Brown, Michael J. Iacono,            *
L03223  ! *       Ronald E. Farren, Luke Chen, Robert Bergstrom.                     *
L03224  ! *                                                                          *
L03225  ! ****************************************************************************
L03226  ! *    TAUMOL                                                                *
L03227  ! *                                                                          *
L03228  ! *    This file contains the subroutines TAUGBn (where n goes from          *
L03229  ! *    1 to 28).  TAUGBn calculates the optical depths and Planck fractions  *
L03230  ! *    per g-value and layer for band n.                                     *
L03231  ! *                                                                          *
L03232  ! * Output:  optical depths (unitless)                                      *
L03233  ! *          fractions needed to compute Planck functions at every layer     *
L03234  ! *              and g-value                                                 *
L03235  ! *                                                                          *
L03236  ! *    COMMON /TAUGCOM/  TAUG(MXLAY,MG)                                      *
L03237  ! *    COMMON /PLANKG/   FRACS(MXLAY,MG)                                     *
L03238  ! *                                                                          *
L03239  ! * Input                                                                   *
L03240  ! *                                                                          *
L03241  ! *    PARAMETER (MG=16, MXLAY=203, NBANDS=14)                               *
L03242  ! *                                                                          *
L03243  ! *    COMMON /FEATURES/ NG(NBANDS),NSPA(NBANDS),NSPB(NBANDS)                *
L03244  ! *    COMMON /PRECISE/  ONEMINUS                                            *
L03245  ! *    COMMON /PROFILE/  NLAYERS,PAVEL(MXLAY),TAVEL(MXLAY),                  *
L03246  ! *   &                  PZ(0:MXLAY),TZ(0:MXLAY),TBOUND                     *
L03247  ! *    COMMON /PROFDATA/ LAYTROP,LAYSWTCH,LAYLOW,                            *
L03248  ! *   &                  COLH2O(MXLAY),COLCO2(MXLAY),                       *
L03249  ! *   &                  COLO3(MXLAY),COLN2O(MXLAY),COLCH4(MXLAY),          *
L03250  ! *   &                  COLO2(MXLAY),CO2MULT(MXLAY)                        *
L03251  ! *    COMMON /INTFAC/   FAC00(MXLAY),FAC01(MXLAY),                          *
L03252  ! *   &                  FAC10(MXLAY),FAC11(MXLAY)                          *
L03253  ! *    COMMON /INTIND/   JP(MXLAY),JT(MXLAY),JT1(MXLAY)                      *
L03254  ! *    COMMON /SELF/     SELFFAC(MXLAY), SELFFRAC(MXLAY), INDSELF(MXLAY)     *
L03255  ! *                                                                          *
L03256  ! *    Description:                                                          *
L03257  ! *    NG(IBAND) - number of g-values in band IBAND                          *
L03258  ! *    NSPA(IBAND) - for the lower atmosphere, the number of reference       *
L03259  ! *                  atmospheres that are stored for band IBAND per          *
L03260  ! *                  pressure level and temperature.  Each of these          *
L03261  ! *                  atmospheres has different relative amounts of the       *
L03262  ! *                  key species for the band (i.e. different binary         *
L03263  ! *                  species parameters).                                    *
L03264  ! *    NSPB(IBAND) - same for upper atmosphere                               *
L03265  ! *    ONEMINUS - since problems are caused in some cases by interpolation   *
L03266  ! *               parameters equal to or greater than 1, for these cases     *
L03267  ! *               these parameters are set to this value, slightly < 1.      *
L03268  ! *    PAVEL - layer pressures (mb)                                          *
L03269  ! *    TAVEL - layer temperatures (degrees K)                                *
L03270  ! *    PZ - level pressures (mb)                                             *
```

```
L03271  ! *    TZ - Level temperatures (degrees K)                            *
L03272  ! *    LAYTROP - Layer at which switch is made from one combination of *
L03273  ! *              key species to another                                *
L03274  ! *    COLH2O, COLCO2, COLO3, COLN2O, COLCH4 - column amounts of water *
L03275  ! *              vapor,carbon dioxide, ozone, nitrous ozide, methane,  *
L03276  ! *              respectively (molecules/cm**2)                        *
L03277  ! *    CO2MULT - for bands in which carbon dioxide is implemented as a *
L03278  ! *              trace species, this is the factor used to multiply the *
L03279  ! *              band's average CO2 absorption coefficient to get the added *
L03280  ! *              contribution to the optical depth relative to 355 ppm. *
L03281  ! *    FACij(LAY) - for layer LAY, these are factors that are needed to *
L03282  ! *                 compute the interpolation factors that multiply the *
L03283  ! *                 appropriate reference k-values.  A value of 0 (1) for *
L03284  ! *                 i,j indicates that the corresponding factor multiplies *
L03285  ! *                 reference k-value for the lower (higher) of the two *
L03286  ! *                 appropriate temperatures, and altitudes, respectively. *
L03287  ! *    JP - the index of the lower (in altitude) of the two appropriate *
L03288  ! *         reference pressure levels needed for interpolation         *
L03289  ! *    JT, JT1 - the indices of the lower of the two appropriate reference *
L03290  ! *              temperatures needed for interpolation (for pressure   *
L03291  ! *              levels JP and JP+1, respectively)                     *
L03292  ! *    SELFFAC - scale factor needed to water vapor self-continuum, equals *
L03293  ! *              (water vapor density)/(atmospheric density at 296K and *
L03294  ! *              1013 mb)                                              *
L03295  ! *    SELFFRAC - factor needed for temperature interpolation of reference *
L03296  ! *               water vapor self-continuum data                      *
L03297  ! *    INDSELF - index of the lower of the two appropriate reference   *
L03298  ! *              temperatures needed for the self-continuum interpolation *
L03299  ! *                                                                    *
L03300  ! * Data input                                                         *
L03301  ! *    COMMON /Kn/ KA(NSPA(n),5,13,MG), KB(NSPB(n),5,13:59,MG), SELFREF(10,MG) *
```

讲解：k 分布头与 LW 版平行，三点不同：① 注释说 TAUGBn 1–28（历史口径）——实际短波实现为 taumol16..29（14 个内部子程序，全局波段号）；②**LAYSWTCH/LAYLOW 三层结构**——短波在平流层还要再分两层（低平流层与中间层的大气主吸收体组合不同，setcoef_sw 输出三个界标）；③**CO2MULT——CO2 作次要气体处理**：部分短波波段 CO2 不是主吸收体，用「相对 355 ppm 基准的增量系数」叠加（co2mult 在 setcoef_sw 里按 CO2 浓度比值算）——这就是 § 五 CO2 年增长函数能进 SW 计算的机制。

```
!file: phys/module_ra_rrtmg_sw.F    L3406-L3498    (subroutine taumol16)
L03406        subroutine taumol16
L03407  !----------------------------------------------------------------------------
L03408  !
L03409  !     band 16:  2600-3250 cm-1 (low - h2o,ch4; high - ch4)
L03410  !
L03411  !----------------------------------------------------------------------------
L03412
L03413  ! ------- Modules -------
L03414
L03415        use parrrsw, only : ng16
L03416        use rrsw_kg16, only : absa, ka, absb, kb, forref, selfref, &
L03417                              sfluxref, rayl, layreffr, strrat1
L03418
L03419  ! ------- Declarations -------
L03420
L03421  ! Local
L03422
L03423        integer(kind=im) :: ig, ind0, ind1, inds, indf, js, lay, laysolfr
L03424        real(kind=rb) :: fac000, fac001, fac010, fac011, fac100, fac101, &
L03425                         fac110, fac111, fs, speccomb, specmult, specparm, &
L03426                         tauray
L03427
L03428  ! Compute the optical depth by interpolating in ln(pressure),
L03429  ! temperature, and appropriate species.  Below LAYTROP, the water
L03430  ! vapor self-continuum is interpolated (in temperature) separately.
L03431
L03432  ! Lower atmosphere loop
L03433        do lay = 1, laytrop
L03434           speccomb = colh2o(lay) + strrat1*colch4(lay)
L03435           specparm = colh2o(lay)/speccomb
L03436           if (specparm .ge. oneminus) specparm = oneminus
L03437           specmult = 8._rb*(specparm)
L03438           js = 1 + int(specmult)
L03439           fs = mod(specmult, 1._rb )
L03440           fac000 = (1._rb - fs) * fac00(lay)
L03441           fac010 = (1._rb - fs) * fac10(lay)
L03442           fac100 = fs * fac00(lay)
L03443           fac110 = fs * fac10(lay)
L03444           fac001 = (1._rb - fs) * fac01(lay)
L03445           fac011 = (1._rb - fs) * fac11(lay)
L03446           fac101 = fs * fac01(lay)
L03447           fac111 = fs * fac11(lay)
L03448           ind0 = ((jp(lay)-1)*5+(jt(lay)-1))*nspa(16) + js
L03449           ind1 = (jp(lay)*5+(jt1(lay)-1))*nspa(16) + js
L03450           inds = indself(lay)
L03451           indf = indfor(lay)
L03452           tauray = colmol(lay) * rayl
L03453
L03454           do ig = 1, ng16
L03455              taug(lay,ig) = speccomb * &
L03456                  (fac000 * absa(ind0   ,ig) + &
L03457                   fac100 * absa(ind0 +1,ig) + &
L03458                   fac010 * absa(ind0 +9,ig) + &
L03459                   fac110 * absa(ind0+10,ig) + &
L03460                   fac001 * absa(ind1   ,ig) + &
L03461                   fac101 * absa(ind1 +1,ig) + &
L03462                   fac011 * absa(ind1 +9,ig) + &
L03463                   fac111 * absa(ind1+10,ig)) + &
L03464                   colh2o(lay) * &
L03465                   (selffac(lay) * (selfref(inds,ig) + &
L03466                   selffrac(lay) * &
```

```
L03467                   (selfref(inds+1,ig) - selfref(inds,ig))) + &
L03468                   forfac(lay) * (forref(indf,ig) + &
L03469                   forfrac(lay) * &
L03470                   (forref(indf+1,ig) - forref(indf,ig))))
L03471  !             ssa(lay,ig) = tauray/taug(lay,ig)
L03472                taur(lay,ig) = tauray
L03473             enddo
L03474          enddo
L03475
L03476          laysolfr = nlayers
L03477
L03478  ! Upper atmosphere loop
L03479          do lay = laytrop+1, nlayers
L03480             if (jp(lay-1) .lt. layreffr .and. jp(lay) .ge. layreffr) &
L03481                laysolfr = lay
L03482             ind0 = ((jp(lay)-13)*5+(jt(lay)-1))*nspb(16) + 1
L03483             ind1 = ((jp(lay)-12)*5+(jt1(lay)-1))*nspb(16) + 1
L03484             tauray = colmol(lay) * rayl
L03485
L03486             do ig = 1, ng16
L03487                taug(lay,ig) = colch4(lay) * &
L03488                    (fac00(lay) * absb(ind0  ,ig) + &
L03489                     fac10(lay) * absb(ind0+1,ig) + &
L03490                     fac01(lay) * absb(ind1  ,ig) + &
L03491                     fac11(lay) * absb(ind1+1,ig))
L03492  !             ssa(lay,ig) = tauray/taug(lay,ig)
L03493                if (lay .eq. laysolfr) sfluxzen(ig) = sfluxref(ig)
L03494                taur(lay,ig) = tauray
L03495             enddo
L03496          enddo
L03497
L03498          end subroutine taumol16
```

讲解：短波波段 16（全局号，820–2600 cm^{-1} 近红外）样本：结构与 taugb1 同款四角插值，差异是**主吸收体 h2o+co2 双组合与 sfluxzen（普朗克分数的 SW 对应物 =太阳源函数分数）**；短波 k 分布没有普朗克函数，fracs 换成 sfluxzen（各 g 点的入射太阳通量分配）。taumol17..29 同构——读一知十四。

八、两流传输 spcvmc_sw（头段）

```
!file: phys/module_ra_rrtmg_sw.F    L8196-L8236
L08196          subroutine spcvmc_sw &
L08197               (nlayers, istart, iend, icpr, iout, &
L08198                pavel, tavel, pz, tz, tbound, palbd, palbp, &
L08199                pcldfmc, ptaucmc, pasycmc, pomgcmc, ptaormc, &
L08200                ptaua, pasya, pomga, prmu0, coldry, wkl, adjflux, &
L08201                laytrop, layswtch, laylow, jp, jt, jt1, &
L08202                co2mult, colch4, colco2, colh2o, colmol, coln2o, colo2, colo3, &
L08203                fac00, fac01, fac10, fac11, &
L08204                selffac, selffrac, indself, forfac, forfrac, indfor, &
L08205                pbbfd, pbbfu, pbbcd, pbbcu, puvfd, puvcd, pnifd, pnicd, &
L08206                pbbfddir, pbbcddir, puvfddir, puvcddir, pnifddir, pnicddir)
L08207  ! ------------------------------------------------------------------------
L08208  !
L08209  ! Purpose: Contains spectral loop to compute the shortwave radiative fluxes,
L08210  !          using the two-stream method of H. Barker and McICA, the Monte-Carlo
L08211  !          Independent Column Approximation, for the representation of
L08212  !          sub-grid cloud variability (i.e. cloud overlap).
L08213  !
L08214  ! Interface:  *spcvmc_sw* is called from *rrtmg_sw.F90* or rrtmg_sw.1col.F90*
L08215  !
L08216  ! Method:
L08217  !    Adapted from two-stream model of H. Barker;
L08218  !    Two-stream model options (selected with kmodts in rrtmg_sw_reftra.F90):
L08219  !        1: Eddington, 2: PIFM, Zdunkowski et al., 3: discret ordinates
L08220  !
L08221  ! Modifications:
L08222  !
L08223  ! Original: H. Barker
L08224  ! Revision: Merge with RRTMG_SW: J.-J.Morcrette, ECMWF, Feb 2003
L08225  ! Revision: Add adjustment for Earth/Sun distance : MJIacono, AER, Oct 2003
L08226  ! Revision: Bug fix for use of PALBP and PALBD: MJIacono, AER, Nov 2003
L08227  ! Revision: Bug fix to apply delta scaling to clear sky: AER, Dec 2004
L08228  ! Revision: Code modified so that delta scaling is not done in cloudy profiles
L08229  !           if routine cldprop is used; delta scaling can be applied by swithcing
L08230  !           code below if cldprop is not used to get cloud properties.
L08231  !           AER, Jan 2005
L08232  ! Revision: Modified to use McICA: MJIacono, AER, Nov 2005
L08233  ! Revision: Uniform formatting for RRTMG: MJIacono, AER, Jul 2006
L08234  ! Revision: Use exponential lookup table for transmittance: MJIacono, AER,
L08235  !           Aug 2007
L08236  !
```

讲解：**Barker 两流 + McICA** 的光谱循环总控（H. Barker 原作、Morcrette 并入 RRTMG）：① 两流方案三选——kmodts 在 reftra_sw 里定（1 Eddington、2 PIFM/Zdunkowski、3 离散纵标）；②delta 标定的使用史注释链（2004-12 修晴空标定 bug、2005-01 cldprop/cldprmc 路径下云侧不再重复标定——即 § 六尾段只存未标定 taormc 的原因）；③2007-08 起透射率用**指数查表**加速；④ 出参 12 组：总/晴空的上下通量 + UV/NIR 分解 + 各自直射版——rrtmg_sw 的四分量输出全部由此产生。vrtqdr_sw（L8035–L8159，未录）是垂直求积（通量的层界恢复）。

九、ECMWF 六类气溶胶 swaerpr（全文）

```
!file: phys/module_ra_rrtmg_sw.F    L5032-L5132    (subroutine swaerpr)
L05032        subroutine swaerpr
L05033  !**************************************************************************
L05034
L05035  ! Purpose: Define spectral aerosol properties for six ECMWF aerosol types
L05036  ! as used in the ECMWF IFS model (see module rrsw_aer.F90 for details)
L05037  !
L05038  ! Original: Defined for rrtmg_sw 14 spectral bands, JJMorcrette, ECMWF Feb 2003
L05039  ! Revision: Reformatted for consistency with rrtmg_lw, MJIacono, AER, Jul 2006
L05040
L05041        use rrsw_aer, only : rsrtaua, rsrpiza, rsrasya
L05042
L05043        save
L05044
L05045        rsrtaua( 1, :) = (/ &
L05046          0.10849_rb, 0.66699_rb, 0.65255_rb, 0.11600_rb, 0.06529_rb, 0.04468_rb/)
L05047        rsrtaua( 2, :) = (/ &
L05048          0.10849_rb, 0.66699_rb, 0.65255_rb, 0.11600_rb, 0.06529_rb, 0.04468_rb/)
L05049        rsrtaua( 3, :) = (/ &
L05050          0.20543_rb, 0.84642_rb, 0.84958_rb, 0.21673_rb, 0.28270_rb, 0.10915_rb/)
L05051        rsrtaua( 4, :) = (/ &
L05052          0.20543_rb, 0.84642_rb, 0.84958_rb, 0.21673_rb, 0.28270_rb, 0.10915_rb/)
L05053        rsrtaua( 5, :) = (/ &
L05054          0.20543_rb, 0.84642_rb, 0.84958_rb, 0.21673_rb, 0.28270_rb, 0.10915_rb/)
L05055        rsrtaua( 6, :) = (/ &
L05056          0.20543_rb, 0.84642_rb, 0.84958_rb, 0.21673_rb, 0.28270_rb, 0.10915_rb/)
L05057        rsrtaua( 7, :) = (/ &
L05058          0.20543_rb, 0.84642_rb, 0.84958_rb, 0.21673_rb, 0.28270_rb, 0.10915_rb/)
L05059        rsrtaua( 8, :) = (/ &
L05060          0.52838_rb, 0.93285_rb, 0.93449_rb, 0.53078_rb, 0.67148_rb, 0.46608_rb/)
L05061        rsrtaua( 9, :) = (/ &
L05062          0.52838_rb, 0.93285_rb, 0.93449_rb, 0.53078_rb, 0.67148_rb, 0.46608_rb/)
L05063        rsrtaua(10, :) = (/ &
L05064          1.69446_rb, 1.11855_rb, 1.09212_rb, 1.72145_rb, 1.03858_rb, 1.12044_rb/)
L05065        rsrtaua(11, :) = (/ &
L05066          1.69446_rb, 1.11855_rb, 1.09212_rb, 1.72145_rb, 1.03858_rb, 1.12044_rb/)
L05067        rsrtaua(12, :) = (/ &
L05068          1.69446_rb, 1.11855_rb, 1.09212_rb, 1.72145_rb, 1.03858_rb, 1.12044_rb/)
L05069        rsrtaua(13, :) = (/ &
L05070          1.69446_rb, 1.11855_rb, 1.09212_rb, 1.72145_rb, 1.03858_rb, 1.12044_rb/)
L05071        rsrtaua(14, :) = (/ &
L05072          0.10849_rb, 0.66699_rb, 0.65255_rb, 0.11600_rb, 0.06529_rb, 0.04468_rb/)
L05073
L05074        rsrpiza( 1, :) = (/ &
L05075          .5230504_rb, .7868518_rb, .8531531_rb, .4048149_rb, .8748231_rb, .2355667_rb/)
L05076        rsrpiza( 2, :) = (/ &
L05077          .5230504_rb, .7868518_rb, .8531531_rb, .4048149_rb, .8748231_rb, .2355667_rb/)
L05078        rsrpiza( 3, :) = (/ &
L05079          .8287144_rb, .9949396_rb, .9279543_rb, .6765051_rb, .9467578_rb, .9955938_rb/)
L05080        rsrpiza( 4, :) = (/ &
L05081          .8287144_rb, .9949396_rb, .9279543_rb, .6765051_rb, .9467578_rb, .9955938_rb/)
L05082        rsrpiza( 5, :) = (/ &
L05083          .8287144_rb, .9949396_rb, .9279543_rb, .6765051_rb, .9467578_rb, .9955938_rb/)
L05084        rsrpiza( 6, :) = (/ &
L05085          .8287144_rb, .9949396_rb, .9279543_rb, .6765051_rb, .9467578_rb, .9955938_rb/)
L05086        rsrpiza( 7, :) = (/ &
L05087          .8287144_rb, .9949396_rb, .9279543_rb, .6765051_rb, .9467578_rb, .9955938_rb/)
L05088        rsrpiza( 8, :) = (/ &
L05089          .8970131_rb, .9984940_rb, .9245594_rb, .7768385_rb, .9532763_rb, .9999999_rb/)
L05090        rsrpiza( 9, :) = (/ &
L05091          .8970131_rb, .9984940_rb, .9245594_rb, .7768385_rb, .9532763_rb, .9999999_rb/)
L05092        rsrpiza(10, :) = (/ &
L05093          .9148907_rb, .9956173_rb, .7504584_rb, .8131335_rb, .9401905_rb, .9999999_rb/)
L05094        rsrpiza(11, :) = (/ &
L05095          .9148907_rb, .9956173_rb, .7504584_rb, .8131335_rb, .9401905_rb, .9999999_rb/)
L05096        rsrpiza(12, :) = (/ &
L05097          .9148907_rb, .9956173_rb, .7504584_rb, .8131335_rb, .9401905_rb, .9999999_rb/)
L05098        rsrpiza(13, :) = (/ &
L05099          .9148907_rb, .9956173_rb, .7504584_rb, .8131335_rb, .9401905_rb, .9999999_rb/)
L05100        rsrpiza(14, :) = (/ &
L05101          .5230504_rb, .7868518_rb, .8531531_rb, .4048149_rb, .8748231_rb, .2355667_rb/)
L05102
L05103        rsrasya( 1, :) = (/ &
L05104          0.700610_rb, 0.818871_rb, 0.702399_rb, 0.689886_rb, .4629866_rb, .1907639_rb/)
L05105        rsrasya( 2, :) = (/ &
L05106          0.700610_rb, 0.818871_rb, 0.702399_rb, 0.689886_rb, .4629866_rb, .1907639_rb/)
L05107        rsrasya( 3, :) = (/ &
L05108          0.636342_rb, 0.802467_rb, 0.691305_rb, 0.627497_rb, .6105750_rb, .4760794_rb/)
L05109        rsrasya( 4, :) = (/ &
L05110          0.636342_rb, 0.802467_rb, 0.691305_rb, 0.627497_rb, .6105750_rb, .4760794_rb/)
L05111        rsrasya( 5, :) = (/ &
L05112          0.636342_rb, 0.802467_rb, 0.691305_rb, 0.627497_rb, .6105750_rb, .4760794_rb/)
L05113        rsrasya( 6, :) = (/ &
L05114          0.636342_rb, 0.802467_rb, 0.691305_rb, 0.627497_rb, .6105750_rb, .4760794_rb/)
L05115        rsrasya( 7, :) = (/ &
L05116          0.636342_rb, 0.802467_rb, 0.691305_rb, 0.627497_rb, .6105750_rb, .4760794_rb/)
L05117        rsrasya( 8, :) = (/ &
L05118          0.668431_rb, 0.788530_rb, 0.698682_rb, 0.657422_rb, .6735182_rb, .6519706_rb/)
L05119        rsrasya( 9, :) = (/ &
L05120          0.668431_rb, 0.788530_rb, 0.698682_rb, 0.657422_rb, .6735182_rb, .6519706_rb/)
L05121        rsrasya(10, :) = (/ &
L05122          0.729019_rb, 0.803129_rb, 0.784592_rb, 0.712208_rb, .7008249_rb, .7270548_rb/)
L05123        rsrasya(11, :) = (/ &
L05124          0.729019_rb, 0.803129_rb, 0.784592_rb, 0.712208_rb, .7008249_rb, .7270548_rb/)
L05125        rsrasya(12, :) = (/ &
L05126          0.729019_rb, 0.803129_rb, 0.784592_rb, 0.712208_rb, .7008249_rb, .7270548_rb/)
L05127        rsrasya(13, :) = (/ &
L05128          0.729019_rb, 0.803129_rb, 0.784592_rb, 0.712208_rb, .7008249_rb, .7270548_rb/)
L05129        rsrasya(14, :) = (/ &
L05130          0.700610_rb, 0.818871_rb, 0.702399_rb, 0.689886_rb, .4629866_rb, .1907639_rb/)
L05131
L05132        end subroutine swaerpr
```

讲解：iaer=6（aer_opt=1，Tegen/ECMWF 路线）的数据底座：**rsrtaua/rsrpiza/rsrasya 三张 14 波段 ×6 类气溶胶表**（Morcrette 2003 定义）——输入只是六类各自的 0.55 μm AOD（ecaer），谱依赖全靠这三张表展开（rrtmg_sw 核心的 iaer=6 分支做加权混合）。六类是 ECMWF 标准划分（无能见度的海洋型/大陆型/城市型等），WRF 主线 aer_opt=1 用它；对照 aer_opt=2/3 的直接三参数输入（code_60）——**两条气溶胶路线的数据结构差异就在这两张表 vs 三维数组**。

十、改造入口

1. **CO2 年变化同步**：SW 侧年增长函数（§ 五 ①）与 LW 侧固定值的不对称——温室气体实验必查两侧。
2. **McICA 种子**：SW permuteseed=1 vs LW 150——集合试验想共享随机实现要改种子。
3. **云散射参数化**：iceflgsw/liqflgsw 四/二选 + delta-M 标定链（§ 六）——云光学敏感性实验入口。
4. **气溶胶路线**：aer_opt=1（六类表）/2（2D AOD 展开，code_60）/3（3D AOD）——swaerpr 表是 iaer=6 的数据底座。
5. **AOD 守卫**：>6 钳位（§ 五 ③）——沙尘暴模拟超阈值的隐式行为。
6. **两流方案**：kmodts 三选（reftra_sw 内）——两流格式敏感性实验。

十一、上下游链接

- 驱动层：`code_20_module_radiation_driver.md`（CASE RRTMG_SWSCHEME、coszen 传入、swint 插值、FARMs）
- 同族长波：`code_05_module_ra_rrtmg_lw.md`（parkind 共享；k 分布/查表结构平行）
- 气溶胶光学：`code_60_module_ra_aerosol.md`
- 教材正文：ch12_radiation_family.md

文库 code_07a：module_mp_thompson.F 上半——模块声明、初始化与列驱动

07a.0 版本指纹与阅读说明

- **源文件**：`phys/module_mp_thompson.F`（WRF 4.8.0，与教材同版本）
- **文件规模**：6192 行；本篇覆盖 L1–L4082（模块声明、`thompson_init`、`mp_gt_driver`、主实体 `mp_thompson`）；L4091–L6192 的辅助子程序族（碰撞冻结建表、数学函数、冰核化物理函数、有效半径与反射率、热力学函数）见 code_07b。
- **保真承诺**：本篇所有代码块由 `code_extract.py` 按字节抽取，块前指纹注释给出源 md5 与行段 md5，可用 `verify` 子命令逐块复核；讲解与代码分离，代码一字未改。
- **符号口径**：大气物理符号以《大气物理学》（盛裴轩，第 2 版）为准；源码变量名（如 `qv`、`qvs`、`pii`）按 Fortran 原样出现，物理对应在讲解中给出。
- **教材锚点**：本篇与《WRF 源码导论》第 13 章配套——13.6（方案分发）、13.7（Thompson 精读：饱和调整 13.7.4、守恒约束 13.7.5、冰核化三来源 13.7.2、有效半径接口 13.7.6）。教材讲“为什么这样设计”，本篇讲“每一行在做什么”。书公式只给指针，不重复推导。

07a.1 一句话定位与调用链

一句话：`module_mp_thompson.F` 是 Thompson 批量微物理方案的完整实现——在一个模块里闭合“云滴/雨/冰晶/雪/霰（+雹感知霰）”五类水凝物的成核、增长、碰并、相变、沉降全链条，并给辐射方案供有效半径、给雷达模拟器供反射率。

调用链（谁在调它）：



- `thompson_init`：仅在模拟初始化阶段被 `module_microphysics_driver.F` 的 `MP_INIT_THOMPSON` 分支调用一次，负责查找表分配与预计算。
- `mp_gt_driver`：每时间步被 `microphysics_driver` 调用一次，是“三维场 ↔ 单列”的适配层。
- `mp_thompson`：被 `mp_gt_driver` 逐格点列调用，是全部微物理过程的本体（2570 行）。
- 教材侧：调用节拍与 `dt` 的传递见 ch13 §13.6.1；方案号 `mp_physics=8/28` 的 Registry 口径见 ch13 §13.6.4。

07a.2 文件头注释原文与精译

Thompson 的文件头本身就是一份“用户改造指南”，NCAR 把哪些常数能改、哪些不能改全写在这里了。先读原文。

```
!file: phys/module_mp_thompson.F    L1-L45
L00001  !+---+-----------------------------------------------------------------+
L00002  !.. This subroutine computes the moisture tendencies of water vapor,
L00003  !.. cloud droplets, rain, cloud ice (pristine), snow, and graupel.
L00004  !.. Prior to WRFv2.2 this code was based on Reisner et al (1998), but
L00005  !.. few of those pieces remain.  A complete description is now found in
L00006  !.. Thompson, G., P. R. Field, R. M. Rasmussen, and W. D. Hall, 2008:
L00007  !.. Explicit Forecasts of winter precipitation using an improved bulk
L00008  !.. microphysics scheme. Part II: Implementation of a new snow
L00009  !.. parameterization.  Mon. Wea. Rev., 136, 5095-5115.
```

```
L00010  !.. Prior to WRFv3.1, this code was single-moment rain prediction as
L00011  !.. described in the reference above, but in v3.1 and higher, the
L00012  !.. scheme is two-moment rain (predicted rain number concentration).
L00013  !..
L00014  !.. Beginning with WRFv3.6, this is also the "aerosol-aware" scheme as
L00015  !.. described in Thompson, G. and T. Eidhammer, 2014:  A study of
L00016  !.. aerosol impacts on clouds and precipitation development in a large
L00017  !.. winter cyclone.  J. Atmos. Sci., 71, 3636-3658.  Setting WRF
L00018  !.. namelist option mp_physics=8 utilizes the older one-moment cloud
L00019  !.. water with constant droplet concentration set as Nt_c (found below)
L00020  !.. while mp_physics=28 uses double-moment cloud droplet number
L00021  !.. concentration, which is not permitted to exceed Nt_c_max below.
L00022  !..
L00023  !.. Most importantly, users may wish to modify the prescribed number of
L00024  !.. cloud droplets (Nt_c; see guidelines mentioned below).  Otherwise,
L00025  !.. users may alter the rain and graupel size distribution parameters
L00026  !.. to use exponential (Marshal-Palmer) or generalized gamma shape.
L00027  !.. The snow field assumes a combination of two gamma functions (from
L00028  !.. Field et al. 2005) and would require significant modifications
L00029  !.. throughout the entire code to alter its shape as well as accretion
L00030  !.. rates.  Users may also alter the constants used for density of rain,
L00031  !.. graupel, ice, and snow, but the latter is not constant when using
L00032  !.. Paul Field's snow distribution and moments methods.  Other values
L00033  !.. users can modify include the constants for mass and/or velocity
L00034  !.. power law relations and assumed capacitances used in deposition/
L00035  !.. sublimation/evaporation/melting.
L00036  !.. Remaining values should probably be left alone.
L00037  !..
L00038  !..Author: Greg Thompson, NCAR-RAL, gthompsn@ucar.edu, 303-497-2805
L00039  !..Last modified: 24 Jan 2018   Aerosol additions to v3.5.1 code 9/2013
L00040  !..                 Cloud fraction additions 11/2014 part of pre-v3.7
L00041  !..                 3-moment graupel/hail additions Aug. 2018 (released in v4.4)
L00042  !+---+-----------------------------------------------------------------+
L00043  !wrft:model_layer:physics
L00044  !+---+-----------------------------------------------------------------+
L00045  !
```

逐句精译与解读：

- **血统**（L2–L12 原文行号以指纹注释为准，下同）：WRFv2.2 之前基于 Reisner et al. (1998)，但“其中很少部分保留至今”；现行描述文献是 Thompson, Field, Rasmussen & Hall (2008, MWR 136, 5095–5115)——即 “Part II，新雪参数化” 那篇。v3.1 起雨改为**双矩**（预报雨滴数浓度 `nr`），这是 Thompson 与早期 Reisner 系方案的本质分界。
- **气溶胶感知**（v3.6 起）：`mp_physics=8` 是老版——云水单矩、云滴数取常数 `Nt_c`；`mp_physics=28` 是 aerosol-aware 版——云滴数双矩预报（上限 `Nt_c_max`）。文献 Thompson & Eidhammer (2014, JAS 71, 3636–3658)。气溶胶场 `nwfa`（水亲和性气溶胶数浓度）、`nifa`（冰核气溶胶数浓度）由 WRF-CHEM 或 `wif_input_opt` 供入——这条线在教材 ch11（气溶胶与水汽）与 ch13 §13.7.2 都有伏笔。
- **用户可改清单**（原文 “users may wish to modify…” 一段）：①`Nt_c`——海运约 100 个/cm³、大陆约 300 个/cm³（这是全方案里最常被调的参数）；② 雨、霰的谱形——可改用指数（Marshall–Palmer）或广义 gamma；③ 雪谱是 Field et al. (2005) **双伽马叠加**，“改它会牵动全代码的碰并率”，劝退；④ 雨/霰/冰/雪密度常数（雪的密度在用 Field 谱时不是常数）；⑤ 质量、落速幂律系数与凝华/蒸发/融化用到的**电容**常数。其余 “should probably be left alone”。
- **作者与版本史**：Greg Thompson（NCAR-RAL）；注释记录了 2013-09 气溶胶并入 v3.5.1、2014-11 云量（cloud fraction）接口、2018-08 三矩霰/雹（v4.4 发布，即 `is_hail_aware/ng/qb` 这条 3-moment graupel 线）。

07a.3 USE 依赖与模块级开关

```
!file: phys/module_mp_thompson.F    L48-L57
L00048        MODULE module_mp_thompson
L00049
L00050        USE module_wrf_error
L00051        USE module_mp_radar
L00052  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L00053        USE module_dm, ONLY : wrf_dm_max_real
L00054  #endif
L00055        USE module_model_constants, only : RE_QC_BG, RE_QI_BG, RE_QS_BG
L00056
L00057        IMPLICIT NONE
```

USE 逐项解读（模块头 4 项；另有 2 项藏在子程序内部——`qr_acr_qg` 内 `USE module_timing`、`table_ccnAct` 内 `USE module_domain`（+`module_dm`），见 code_07b，普查表所记 6 项无误）：

USE	拿来什么	用在哪
`module_wrf_error`	`wrf_debug`、`wrf_err_message`	全模块的分级日志（`wrft:model_layer:physics` 标签行 L43 即日志路由标记）

USE	拿来什么	用在哪
`module_mp_radar`	`radar_init` 及 `xam_r/xbm_r/xmu_r/xam_s/…`共享量	`thompson_init` 末尾把本方案的谱参数拷给雷达模拟器模块，`calc_refl10cm`（code_07b）据此算 10 cm 反射率
`module_dm`	`wrf_dm_max_real`（仅 DM_PARALLEL 且非 STUBMPI）	`thompson_init` 检查气溶胶场是否全零时做**全局**归约——单进程裸 MAXVAL 会漏判 “本 tile 恰好全零、别处有值” 的假阴性
`module_model_constants`	`RE_QC_BG/RE_QI_BG/RE_QS_BG`	有效半径背景值（`calc_effectRad` 的输出下限），常数定义在 share 层（文库 code_01），教材 ch13 §13.8.4 讲过这组闭合
`module_timing`（qr_acr_qg 内部） `module_domain`+`module_dm`（table_ccnAct 内部）	`start_timing/end_timing` `wrf_dm_on_monitor` 等	建表耗时统计（只在首次建表时走） 只在 monitor 进程读 CCN 活化表再广播

四个模块级开关（L59–L63）：

- `iiwarm = .false.`（PARAMETER）：暖云测试开关，编译期写死。置真时跳过全部冰相建表与冰相过程块——只用于开发调试，产品运行保持 false。
- `is_aerosol_aware`：**运行时**变量，由 `thompson_init` 里 `PRESENT(nwfa/nifa/nbca)` 判定。它决定主实体里走哪套冰核化与云滴活化分支。
- `dustyIce = .true.` / `homogIce = .true.`（PARAMETER）：冰核化物理的编译期选择——是否用尘埃依赖的 DeMott (2010) 冻结谱、是否允许 Koop (2001) 均质冻结。三来源冰核化的完整对照见教材 ch13 §13.7.2。
- `is_hail_aware`：运行时变量，由 `PRESENT(ng)` 判定，控制三矩霰/雹分支（`ng` 霰数浓度、`qb` 霰**体积**混合比、`idx_bg` 密度档位）。

07a.4 模块级常数逐组解读

Thompson 的模块声明段实质是一张 “方案参数总表”——批量方案的全部谱假设、质量-尺寸-落速幂律、收集效率、数值下限都在这里一次性定义，主实体里只引用不再改。以下按物理作用分六组走读，代码块保真原文。

07a.4.1 密度、云滴数与气溶胶缺省剖面

```
!file: phys/module_mp_thompson.F    L59-L99
L00059        LOGICAL, PARAMETER, PRIVATE:: iiwarm = .false.
L00060        LOGICAL, PRIVATE:: is_aerosol_aware = .false.
L00061        LOGICAL, PARAMETER, PRIVATE:: dustyIce = .true.
L00062        LOGICAL, PARAMETER, PRIVATE:: homogIce = .true.
L00063        LOGICAL, PRIVATE:: is_hail_aware = .false.
L00064
L00065        INTEGER, PARAMETER, PRIVATE:: IFDRY = 0
L00066        REAL, PARAMETER, PRIVATE:: T_0 = 273.15
L00067        REAL, PARAMETER, PRIVATE:: PI = 3.1415926536
L00068
L00069  !..Densities of rain, snow, graupel, and cloud ice.
L00070        REAL, PARAMETER, PRIVATE:: rho_w = 1000.0
L00071        REAL, PRIVATE:: rho_s = 100.0
L00072        REAL, PARAMETER, PRIVATE:: rho_i = 890.0
L00073        INTEGER, PARAMETER, PRIVATE:: NRHG = 9
L00074        INTEGER, PARAMETER, PRIVATE:: NRHG1 = 1
L00075        INTEGER :: dimNRHG
L00076
L00077        REAL, DIMENSION(NRHG), PARAMETER, PRIVATE:: &
L00078             rho_g = (/50., 100., 200., 300., 400., 500., 600., 700., 800./)
L00079        INTEGER, PARAMETER :: idx_bg1 = 5 ! index for rhog when mp=8 or 28
L00080
L00081  !..Prescribed number of cloud droplets.  Set according to known data or
L00082  !.. roughly 100 per cc (100.E6 m^-3) for Maritime cases and
L00083  !.. 300 per cc (300.E6 m^-3) for Continental.  Gamma shape parameter,
L00084  !.. mu_c, calculated based on Nt_c is important in autoconversion
L00085  !.. scheme.  In 2-moment cloud water, Nt_c represents a maximum of
L00086  !.. droplet concentration and nu_c is also variable depending on local
L00087  !.. droplet number concentration.
L00088        REAL, PARAMETER, PRIVATE:: Nt_c = 100.E6
L00089        REAL, PARAMETER, PRIVATE:: Nt_c_max = 1999.E6
L00090
L00091  !..Declaration of constants for assumed CCN/IN aerosols when none in
L00092  !.. the input data.  Look inside the init routine for modifications
L00093  !.. due to surface land-sea points or vegetation characteristics.
L00094        REAL, PARAMETER, PRIVATE:: naIN0 = 1.5E6
L00095        REAL, PARAMETER, PRIVATE:: naIN1 = 0.5E6
L00096        REAL, PARAMETER, PRIVATE:: naCCN0 = 300.0E6
L00097        REAL, PARAMETER, PRIVATE:: naCCN1 = 50.0E6
L00098        REAL, PARAMETER, PRIVATE:: naBC0 = 150.0E6
```

```
L00099        REAL, PARAMETER, PRIVATE:: naBC1 = 25.0E6
```

- **密度组**：`rho_w=1000`（液水）、`rho_i=890`（冰晶）；**`rho_s=100` 声明为变量而非参数**——用 Field 谱时雪密度随含水量与温度变化（主实体 L2248 处 `rho_s = MAX(rho_g(1), MIN(0.13/xDs, rho_i-100.))` 动态更新，见 code_07a §07a.5 主实体“冰相准备”段讲解）。
- **霰密度九档**：`rho_g = (50,100,200,⋯,800)` kg/m³，对应“飑线低密度霰 → 干雹”连续谱；`NRHG=9` 是档位数。`dimNRHG` 运行时为 9（hail-aware）或 1（老模式），控制查找表第三维大小——**同一张表，两种内存占用**。`idx_bg1=5` 即 500 kg/m³ 档，是 `mp=8/28`（无雹感知）时的固定档位。
- **`Nt_c=100.E6`**：云滴数缺省 100 个/cm³（注意单位是 m⁻³，100.E6 m⁻³ = 100 cm⁻³）。文件头说的“海运 100/大陆 300”就是改这里；`Nt_c_max=1999.E6` 是双矩模式下云滴数的物理上限（防止活化计算爆表）。
- **气溶胶缺省剖面常数**（L94–L99）：`naIN0/naIN1`（冰核：地面 1.5e6、高空本底 0.5e6 m⁻³）、`naCCN0/naCCN1`（CCN：300e6/50e6）、`naBC0/naBC1`（黑碳：150e6/25e6）。当输入场缺省时，`thompson_init` 用它们构造随高度指数衰减的剖面（见 §07a.5 init 第二段）。

07a.4.2 谱形参数：N(D) 的“形状”由谁定

```
!file: phys/module_mp_thompson.F    L101-L124
L00101  !..Generalized gamma distributions for rain, graupel and cloud ice.
L00102  !.. N(D) = N_0 * D**mu * exp(-lamda*D);  mu=0 is exponential.
L00103        REAL, PARAMETER, PRIVATE:: mu_r = 0.0
L00104        REAL, PARAMETER, PRIVATE:: mu_g = 0.0
L00105        REAL, PARAMETER, PRIVATE:: mu_i = 0.0
L00106        REAL, PRIVATE:: mu_c
L00107
L00108  !..Sum of two gamma distrib for snow (Field et al. 2005).
L00109  !.. N(D) = M2**4/M3**3 * [Kap0*exp(-M2*Lam0*D/M3)
L00110  !..    + Kap1*(M2/M3)**mu_s * D**mu_s * exp(-M2*Lam1*D/M3)]
L00111  !.. M2 and M3 are the (bm_s)th and (bm_s+1)th moments respectively
L00112  !.. calculated as function of ice water content and temperature.
L00113        REAL, PARAMETER, PRIVATE:: mu_s = 0.6357
L00114        REAL, PARAMETER, PRIVATE:: Kap0 = 490.6
L00115        REAL, PARAMETER, PRIVATE:: Kap1 = 17.46
L00116        REAL, PARAMETER, PRIVATE:: Lam0 = 20.78
L00117        REAL, PARAMETER, PRIVATE:: Lam1 = 3.29
L00118
L00119  !..Y-intercept parameter for graupel is not constant and depends on
L00120  !.. mixing ratio.  Also, when mu_g is non-zero, these become equiv
L00121  !.. y-intercept for an exponential distrib and proper values are
L00122  !.. computed based on same mixing ratio and total number concentration.
L00123        REAL, PARAMETER, PRIVATE:: gonv_min = 1.E2
L00124        REAL, PARAMETER, PRIVATE:: gonv_max = 1.E6
```

批量方案的核心假设是粒径谱形。Thompson 的五物种谱形选择一览：

- **雨、霰、冰**：`N(D) = N_0 • D^mu • exp(-λD)`，`mu_r = mu_g = mu_i = 0` → 退化为 Marshall–Palmer 指数谱。双矩模式通过 `(N, q)` 两个预报量反解 `(N_0, λ)`，谱形其实由物理量闭环决定（见主实体“谱参数化”段）。
- **云滴**：`mu_c` 是**变量**，按 Martin et al. (1994) 的离散度关系 `mu_c = MIN(15, 1000E6/Nt_c + 2)` 由云滴数定——海运洁净（N 小）谱窄单峰，大陆污染（N 大）谱宽。
- **雪**：Field et al. (2005) **双伽马叠加** `N(D) = M2⁴/M3³ • [Kap0•exp(-M2•Lam0•D/M3) + Kap1•(M2/M3)^mu_s • D^mu_s • exp(-M2•Lam1•D/M3)]`，五个常数（`mu_s=0.6357, Kap0=490.6, Kap1=17.46, Lam0=20.78, Lam1=3.29`）是 Field 谱的拟合结果；`M2、M3` 是由冰水含量和温度决定的矩量（主实体用 `sa/sb` 十项多项式换算，见 §07a.6）。
- **霰截距**：`gonv_min=1e2、gonv_max=1e6`——指数等效截距 `N_0` 的上下限（老模式由 qg 经 `zans1 = 3 + 2/7•(log10(qgρ)+8)` 经验式插值，见 driver 段讲解）。

07a.4.3 质量与落速幂律：把谱“接上”物理

```
!file: phys/module_mp_thompson.F    L126-L168
L00126  !..Mass power law relations:  mass = am*D**bm
L00127  !.. Snow from Field et al. (2005), others assume spherical form.
L00128        REAL, PARAMETER, PRIVATE:: am_r = PI*rho_w/6.0
L00129        REAL, PARAMETER, PRIVATE:: bm_r = 3.0
L00130        REAL, PARAMETER, PRIVATE:: am_s = 0.069
L00131        REAL, PARAMETER, PRIVATE:: bm_s = 2.0
L00132        REAL, DIMENSION(NRHG), PARAMETER,PRIVATE:: am_g = (/             &
L00133       &            PI*rho_g(1)/6.0, PI*rho_g(2)/6.0, PI*rho_g(3)/6.0,    &
L00134       &            PI*rho_g(4)/6.0, PI*rho_g(5)/6.0, PI*rho_g(6)/6.0,    &
L00135       &            PI*rho_g(7)/6.0, PI*rho_g(8)/6.0, PI*rho_g(9)/6.0 /)
L00136        REAL, PARAMETER, PRIVATE:: bm_g = 3.0
L00137        REAL, PARAMETER, PRIVATE:: am_i = PI*rho_i/6.0
L00138        REAL, PARAMETER, PRIVATE:: bm_i = 3.0
L00139
L00140  !..Fallspeed power laws relations:  v = (av*D**bv)*exp(-fv*D)
L00141  !.. Rain from Ferrier (1994), ice, snow, and graupel from
```

```
L00142  !.. Thompson et al (2008). Coefficient fv is zero for graupel/ice.
L00143        REAL, PARAMETER, PRIVATE:: av_r = 4854.0
L00144        REAL, PARAMETER, PRIVATE:: bv_r = 1.0
L00145        REAL, PARAMETER, PRIVATE:: fv_r = 195.0
L00146        REAL, PARAMETER, PRIVATE:: av_s = 40.0
L00147        REAL, PARAMETER, PRIVATE:: bv_s = 0.55
L00148        REAL, PARAMETER, PRIVATE:: fv_s = 100.0
L00149        REAL, PARAMETER, PRIVATE:: av_g_old = 442.0
L00150        REAL, PARAMETER, PRIVATE:: bv_g_old = 0.89
L00151        REAL, DIMENSION(NRHG), PRIVATE:: & ! Computed from A. Heymsfield: Best - Reynolds relation
L00152       &    av_g = (/ 45.9173813, 67.0867386, 98.0158463, 122.353378,     &
L00153       &              143.204224, 161.794724, 178.762115, 194.488785,     &
L00154       &              209.225876/)
L00155        REAL, DIMENSION(NRHG), PRIVATE:: & ! Computed from A. Heymsfield: Best - Reynolds relation
L00156       &    bv_g = (/0.640961647, 0.640961647, 0.640961647, 0.640961647,  &
L00157       &             0.640961647, 0.640961647, 0.640961647, 0.640961647,  &
L00158       &             0.640961647/)
L00159        REAL, PARAMETER, PRIVATE:: a_coeff = 0.47244157
L00160        REAL, PARAMETER, PRIVATE:: b_coeff = 0.54698726
L00161        REAL, PARAMETER, PRIVATE:: av_i = 1493.9
L00162        REAL, PARAMETER, PRIVATE:: bv_i = 1.0
L00163        REAL, PARAMETER, PRIVATE:: av_c = 0.316946E8
L00164        REAL, PARAMETER, PRIVATE:: bv_c = 2.0
L00165
L00166  !..Capacitance of sphere and plates/aggregates: D**3, D**2
L00167        REAL, PARAMETER, PRIVATE:: C_cube = 0.5
L00168        REAL, PARAMETER, PRIVATE:: C_sqrd = 0.15
```

- **质量幂律** `m = am * D^bm`：雨、冰、霰按球形 `am = π*ρ/6, bm = 3`（霰的 `am_g` 是随密度档变化的九元素数组）；雪用 Field 拟合 `am_s=0.069, bm_s=2.0`——**非球形**，这是雪谱与其它物种的本质区别，也解释了为什么雪的矩量要走多项式换算。
- **落速幂律** `v = av * D^bv * exp(-fv * D)`：末速度不是简单的幂律，末尾的 `exp(-fv * D)` 把大粒子端压平（雨 `fv_r=195`、雪 `fv_s=100`，霰/冰 `fv=0`）。这使 $\int v(D)\cdot m(D)\cdot N(D)dD$ 型积分出现 `Γ(n) * (λ+fv)^(-n)` 结构——主实体里到处可见的 `((lamr+fv_r)**(-cre(9)))` 就是它。雨落速系数取 Ferrier (1994)，雪取 Thompson et al. (2008)，霰九档系数由 Heymsfield 的 Best–Reynolds 关系预计算（`av_g/bv_g` 数组，`bv_g` 九档同值 0.64096…）。
- **电容**：`C_cube=0.5`（球，D^3 律）、`C_sqrd=0.15`（板状/聚合体，D^2 律）。凝华/融化率公式里雪的电容在两值间随温度插值（主实体 `C_snow` 一行），电容的物理来源（静电类比）见书 §6.3 与教材 ch13 §13.3 的指针。

07a.4.4 收集效率、数值下限与冰核化常数

```
!file: phys/module_mp_thompson.F    L170-L199
L00170  !..Collection efficiencies.  Rain/snow/graupel collection of cloud
L00171  !.. droplets use variables (Ef_rw, Ef_sw, Ef_gw respectively) and
L00172  !.. get computed elsewhere because they are dependent on stokes
L00173  !.. number.
L00174        REAL, PARAMETER, PRIVATE:: Ef_si = 0.05
L00175        REAL, PARAMETER, PRIVATE:: Ef_rs = 0.95
L00176        REAL, PARAMETER, PRIVATE:: Ef_rg = 0.75
L00177        REAL, PARAMETER, PRIVATE:: Ef_ri = 0.95
L00178
L00179  !..Minimum microphys values
L00180  !.. R1 value, 1.E-12, cannot be set lower because of numerical
L00181  !.. problems with Paul Field's moments and should not be set larger
L00182  !.. because of truncation problems in snow/ice growth.
L00183        REAL, PARAMETER, PRIVATE:: R1 = 1.E-12
L00184        REAL, PARAMETER, PRIVATE:: R2 = 1.E-6
L00185        REAL, PARAMETER, PRIVATE:: eps = 1.E-15
L00186
L00187  !..Constants in Cooper curve relation for cloud ice number.
L00188        REAL, PARAMETER, PRIVATE:: TNO = 5.0
L00189        REAL, PARAMETER, PRIVATE:: ATO = 0.304
L00190
L00191  !..Rho_not used in fallspeed relations (rho_not/rho)**.5 adjustment.
L00192        REAL, PARAMETER, PRIVATE:: rho_not = 101325.0/(287.05*298.0)
L00193
L00194  !..Schmidt number
L00195        REAL, PARAMETER, PRIVATE:: Sc = 0.632
L00196        REAL, PRIVATE:: Sc3
L00197
L00198  !..Homogeneous freezing temperature
L00199        REAL, PARAMETER, PRIVATE:: HGFR = 235.16
```

- **收集效率**：雨/雪/霰收集**云水**的效率 `Ef_rw/Ef_sw/Ef_gw` 依赖 Stokes 数，运行时查表（`t_Efrw/t_Efsw`，建表见 code_07b `table_Efrw/table_Efsw`）；而雪收集冰晶 `Ef_si=0.05`、雨收集雪 `Ef_rs=0.95`、雨收集霰 `Ef_rg=0.75`、雨收集冰晶 `Ef_ri=0.95` 是**常数**——冰-冰相碰并效率低（冰晶不易粘附）而液-固相碰并效率接近 1，这组数字本身就是微物理常识的编码。
- **数值下限**：`R1=1e-12`（kg/kg）是**不许再小**的下限——注释明说 Field 矩量法在更小值处数值出问题、更大值又会让雪/冰增长被截断；`R2=1e-6` 是数浓度下限尺度，`eps=1e-15` 是判零容差。主实体所有“守恒/钳位”都以这三个量为标尺。
- **Cooper 冰核曲线**：`TNO=5.0, ATO=0.304`——`N_IN = TNO * exp(ATO * (T_0-T))`（L^{-1}），气溶胶不可用时的冰核缺省谱（Cooper 1986），主实体 L2624 处现写 `MIN(250.E3, TNO*EXP(ATO*(T_0-temp(k))))`。

- **`rho_not`**：标准密度（p=101325 Pa、T=298 K），落速的密度订正因子 `rhof = sqrt(rho_not/rho)` 用它——粒子在低密度高空落得更快。
- **`Sc=0.632`**（Schmidt 数）、`HGFR=235.16 K`（均质冻结温度，−38 °C）。

07a.4.5 气体常数与潜热

```
!file: phys/module_mp_thompson.F    L201-L229
L00201  !..Water vapor and air gas constants at constant pressure
L00202        REAL, PARAMETER, PRIVATE:: Rv = 461.5
L00203        REAL, PARAMETER, PRIVATE:: oRv = 1./Rv
L00204        REAL, PARAMETER, PRIVATE:: R = 287.04
L00205        REAL, PARAMETER, PRIVATE:: Cp = 1004.0
L00206        REAL, PARAMETER, PRIVATE:: R_uni = 8.314                          ! J (mol K)-1
L00207
L00208        DOUBLE PRECISION, PARAMETER, PRIVATE:: k_b = 1.38065E-23          ! Boltzmann constant [J/K]
L00209        DOUBLE PRECISION, PARAMETER, PRIVATE:: M_w = 18.01528E-3          ! molecular mass of water [kg/mol]
L00210        DOUBLE PRECISION, PARAMETER, PRIVATE:: M_a = 28.96E-3             ! molecular mass of air [kg/mol]
L00211        DOUBLE PRECISION, PARAMETER, PRIVATE:: N_avo = 6.022E23           ! Avogadro number [1/mol]
L00212        DOUBLE PRECISION, PARAMETER, PRIVATE:: ma_w = M_w / N_avo         ! mass of water molecule [kg]
L00213        REAL, PARAMETER, PRIVATE:: ar_volume = 4./3.*PI*(2.5e-6)**3       ! assume radius of 0.025 micrometer, 2.5e-6 cm
L00214
L00215  !..Enthalpy of sublimation, vaporization, and fusion at 0C.
L00216        REAL, PARAMETER, PRIVATE:: lsub = 2.834E6
L00217        REAL, PARAMETER, PRIVATE:: lvap0 = 2.5E6
L00218        REAL, PARAMETER, PRIVATE:: lfus = lsub - lvap0
L00219        REAL, PARAMETER, PRIVATE:: olfus = 1./lfus
L00220
L00221  !..Ice initiates with this mass (kg), corresponding diameter calc.
L00222  !..Min diameters and mass of cloud, rain, snow, and graupel (m, kg).
L00223        REAL, PARAMETER, PRIVATE:: xm0i = 1.E-12
L00224        REAL, PARAMETER, PRIVATE:: D0c = 1.E-6
L00225        REAL, PARAMETER, PRIVATE:: D0r = 50.E-6
L00226        REAL, PARAMETER, PRIVATE:: D0s = 300.E-6
L00227        REAL, PARAMETER, PRIVATE:: D0g = 350.E-6
L00228        REAL, PRIVATE:: D0i, xm0s, xm0g
L00229
```

- **气体常数组**：`Rv=461.5`，`R=287.04`，`Cp=1004`——注意这三者在模块内**自定义**而非引用 `module_model_constants`，方案自成体系；`R_uni/k_b/M_w/M_a/N_avo/ma_w` 是分子尺度常数（单位标注在行尾注释），只被 Koop 均质冻结函数（`iceKoop`，code_07b）使用。
- **潜热**：`lsub=2.834e6`（0 °C 升华）、`lvap0=2.5e6`（0 °C 汽化）、`lfus = lsub − lvap0`（融化）。温度依赖的 `lvap(T) = lvap0 + (2106 − 4218)·Tc` 在主实体现场计算（2106 是水汽 Cp、4218 是液水 Cp，差值即克希霍夫律的斜率——书 (6.1.13) 对应的实践版）。三套 Lv(T) 口径的全书对照见教材 ch13 §13.9.2。
- **初始冰质量**：`xm0i=1e-12` kg——每个新冰晶出生时的质量（直径约 20 μm）；四物种最小直径 `D0c/D0r/D0s/D0g` 是谱积分下限。

07a.4.6 查找表体系与预计算矩阵

```
!file: phys/module_mp_thompson.F    L230-L413
L00230  !..Lookup table dimensions
L00231        INTEGER, PARAMETER, PRIVATE:: nbins = 100
L00232        INTEGER, PARAMETER, PRIVATE:: nbc = nbins
L00233        INTEGER, PARAMETER, PRIVATE:: nbi = nbins
L00234        INTEGER, PARAMETER, PRIVATE:: nbr = nbins
L00235        INTEGER, PARAMETER, PRIVATE:: nbs = nbins
L00236        INTEGER, PARAMETER, PRIVATE:: nbg = nbins
L00237        INTEGER, PARAMETER, PRIVATE:: ntb_c = 37
L00238        INTEGER, PARAMETER, PRIVATE:: ntb_i = 64
L00239        INTEGER, PARAMETER, PRIVATE:: ntb_r = 37
L00240        INTEGER, PARAMETER, PRIVATE:: ntb_s = 37
L00241        INTEGER, PARAMETER, PRIVATE:: ntb_g = 37
L00242        INTEGER, PARAMETER, PRIVATE:: ntb_g1 = 37
L00243        INTEGER, PARAMETER, PRIVATE:: ntb_r1 = 37
L00244        INTEGER, PARAMETER, PRIVATE:: ntb_i1 = 55
L00245        INTEGER, PARAMETER, PRIVATE:: ntb_t = 9
L00246        INTEGER, PRIVATE:: nic1, nic2, nii2, nii3, nir2, nir3, nis2, nig2, nig3
L00247        INTEGER, PARAMETER, PRIVATE:: ntb_arc = 7
L00248        INTEGER, PARAMETER, PRIVATE:: ntb_arw = 9
L00249        INTEGER, PARAMETER, PRIVATE:: ntb_art = 7
L00250        INTEGER, PARAMETER, PRIVATE:: ntb_arr = 5
L00251        INTEGER, PARAMETER, PRIVATE:: ntb_ark = 4
L00252        INTEGER, PARAMETER, PRIVATE:: ntb_IN = 55
L00253        INTEGER, PRIVATE:: niIN2
L00254
L00255        DOUBLE PRECISION, DIMENSION(nbins+1):: xDx
L00256        DOUBLE PRECISION, DIMENSION(nbc):: Dc, dtc
L00257        DOUBLE PRECISION, DIMENSION(nbi):: Di, dti
L00258        DOUBLE PRECISION, DIMENSION(nbr):: Dr, dtr
L00259        DOUBLE PRECISION, DIMENSION(nbs):: Ds, dts
L00260        DOUBLE PRECISION, DIMENSION(nbg):: Dg, dtg
L00261        DOUBLE PRECISION, DIMENSION(nbc):: t_Nc
L00262
L00263  !..Lookup tables for cloud water content (kg/m**3).
L00264        REAL, DIMENSION(ntb_c), PARAMETER, PRIVATE:: &
L00265        r_c = (/1.e-6,2.e-6,3.e-6,4.e-6,5.e-6,6.e-6,7.e-6,8.e-6,9.e-6, &
L00266                1.e-5,2.e-5,3.e-5,4.e-5,5.e-5,6.e-5,7.e-5,8.e-5,9.e-5, &
L00267                1.e-4,2.e-4,3.e-4,4.e-4,5.e-4,6.e-4,7.e-4,8.e-4,9.e-4, &
L00268                1.e-3,2.e-3,3.e-3,4.e-3,5.e-3,6.e-3,7.e-3,8.e-3,9.e-3, &
L00269                1.e-2/)
L00270
L00271  !..Lookup tables for cloud ice content (kg/m**3).
```

```
L00272        REAL, DIMENSION(ntb_i), PARAMETER, PRIVATE:: &
L00273        r_i = (/1.e-10,2.e-10,3.e-10,4.e-10, &
L00274                5.e-10,6.e-10,7.e-10,8.e-10,9.e-10, &
L00275                1.e-9,2.e-9,3.e-9,4.e-9,5.e-9,6.e-9,7.e-9,8.e-9,9.e-9, &
L00276                1.e-8,2.e-8,3.e-8,4.e-8,5.e-8,6.e-8,7.e-8,8.e-8,9.e-8, &
L00277                1.e-7,2.e-7,3.e-7,4.e-7,5.e-7,6.e-7,7.e-7,8.e-7,9.e-7, &
L00278                1.e-6,2.e-6,3.e-6,4.e-6,5.e-6,6.e-6,7.e-6,8.e-6,9.e-6, &
L00279                1.e-5,2.e-5,3.e-5,4.e-5,5.e-5,6.e-5,7.e-5,8.e-5,9.e-5, &
L00280                1.e-4,2.e-4,3.e-4,4.e-4,5.e-4,6.e-4,7.e-4,8.e-4,9.e-4, &
L00281                1.e-3/)
L00282
L00283   !..Lookup tables for rain content (kg/m**3).
L00284        REAL, DIMENSION(ntb_r), PARAMETER, PRIVATE:: &
L00285        r_r = (/1.e-6,2.e-6,3.e-6,4.e-6,5.e-6,6.e-6,7.e-6,8.e-6,9.e-6, &
L00286                1.e-5,2.e-5,3.e-5,4.e-5,5.e-5,6.e-5,7.e-5,8.e-5,9.e-5, &
L00287                1.e-4,2.e-4,3.e-4,4.e-4,5.e-4,6.e-4,7.e-4,8.e-4,9.e-4, &
L00288                1.e-3,2.e-3,3.e-3,4.e-3,5.e-3,6.e-3,7.e-3,8.e-3,9.e-3, &
L00289                1.e-2/)
L00290
L00291   !..Lookup tables for graupel content (kg/m**3).
L00292        REAL, DIMENSION(ntb_g), PARAMETER, PRIVATE:: &
L00293        r_g = (/1.e-6,2.e-6,3.e-6,4.e-6,5.e-6,6.e-6,7.e-6,8.e-6,9.e-6, &
L00294                1.e-5,2.e-5,3.e-5,4.e-5,5.e-5,6.e-5,7.e-5,8.e-5,9.e-5, &
L00295                1.e-4,2.e-4,3.e-4,4.e-4,5.e-4,6.e-4,7.e-4,8.e-4,9.e-4, &
L00296                1.e-3,2.e-3,3.e-3,4.e-3,5.e-3,6.e-3,7.e-3,8.e-3,9.e-3, &
L00297                1.e-2/)
L00298
L00299   !..Lookup tables for snow content (kg/m**3).
L00300        REAL, DIMENSION(ntb_s), PARAMETER, PRIVATE:: &
L00301        r_s = (/1.e-6,2.e-6,3.e-6,4.e-6,5.e-6,6.e-6,7.e-6,8.e-6,9.e-6, &
L00302                1.e-5,2.e-5,3.e-5,4.e-5,5.e-5,6.e-5,7.e-5,8.e-5,9.e-5, &
L00303                1.e-4,2.e-4,3.e-4,4.e-4,5.e-4,6.e-4,7.e-4,8.e-4,9.e-4, &
L00304                1.e-3,2.e-3,3.e-3,4.e-3,5.e-3,6.e-3,7.e-3,8.e-3,9.e-3, &
L00305                1.e-2/)
L00306
L00307   !..Lookup tables for rain y-intercept parameter (/m**4).
L00308        REAL, DIMENSION(ntb_r1), PARAMETER, PRIVATE:: &
L00309        N0r_exp = (/1.e6,2.e6,3.e6,4.e6,5.e6,6.e6,7.e6,8.e6,9.e6, &
L00310                    1.e7,2.e7,3.e7,4.e7,5.e7,6.e7,7.e7,8.e7,9.e7, &
L00311                    1.e8,2.e8,3.e8,4.e8,5.e8,6.e8,7.e8,8.e8,9.e8, &
L00312                    1.e9,2.e9,3.e9,4.e9,5.e9,6.e9,7.e9,8.e9,9.e9, &
L00313                    1.e10/)
L00314
L00315   !..Lookup tables for graupel y-intercept parameter (/m**4).
L00316        REAL, DIMENSION(ntb_g1), PARAMETER, PRIVATE:: &
L00317        N0g_exp = (/1.e2,2.e2,3.e2,4.e2,5.e2,6.e2,7.e2,8.e2,9.e2, &
L00318                    1.e3,2.e3,3.e3,4.e3,5.e3,6.e3,7.e3,8.e3,9.e3, &
L00319                    1.e4,2.e4,3.e4,4.e4,5.e4,6.e4,7.e4,8.e4,9.e4, &
L00320                    1.e5,2.e5,3.e5,4.e5,5.e5,6.e5,7.e5,8.e5,9.e5, &
L00321                    1.e6/)
L00322
L00323   !..Lookup tables for ice number concentration (/m**3).
L00324        REAL, DIMENSION(ntb_i1), PARAMETER, PRIVATE:: &
L00325        Nt_i = (/1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0,9.0, &
L00326                 1.e1,2.e1,3.e1,4.e1,5.e1,6.e1,7.e1,8.e1,9.e1, &
L00327                 1.e2,2.e2,3.e2,4.e2,5.e2,6.e2,7.e2,8.e2,9.e2, &
L00328                 1.e3,2.e3,3.e3,4.e3,5.e3,6.e3,7.e3,8.e3,9.e3, &
L00329                 1.e4,2.e4,3.e4,4.e4,5.e4,6.e4,7.e4,8.e4,9.e4, &
L00330                 1.e5,2.e5,3.e5,4.e5,5.e5,6.e5,7.e5,8.e5,9.e5, &
L00331                 1.e6/)
L00332
L00333   !..Aerosol table parameter: Number of available aerosols, vertical
L00334   !.. velocity, temperature, aerosol mean radius, and hygroscopicity.
L00335        REAL, DIMENSION(ntb_arc), PARAMETER, PRIVATE:: &
L00336        ta_Na = (/10.0, 31.6, 100.0, 316.0, 1000.0, 3160.0, 10000.0/)
L00337        REAL, DIMENSION(ntb_arw), PARAMETER, PRIVATE:: &
L00338        ta_Ww = (/0.01, 0.0316, 0.1, 0.316, 1.0, 3.16, 10.0, 31.6, 100.0/)
L00339        REAL, DIMENSION(ntb_art), PARAMETER, PRIVATE:: &
L00340        ta_Tk = (/243.15, 253.15, 263.15, 273.15, 283.15, 293.15, 303.15/)
L00341        REAL, DIMENSION(ntb_arr), PARAMETER, PRIVATE:: &
L00342        ta_Ra = (/0.01, 0.02, 0.04, 0.08, 0.16/)
L00343        REAL, DIMENSION(ntb_ark), PARAMETER, PRIVATE:: &
L00344        ta_Ka = (/0.2, 0.4, 0.6, 0.8/)
L00345
L00346   !..Lookup tables for IN concentration (/m**3) from 0.001 to 1000/Liter.
L00347        REAL, DIMENSION(ntb_IN), PARAMETER, PRIVATE:: &
L00348        Nt_IN = (/1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0,9.0, &
L00349                  1.e1,2.e1,3.e1,4.e1,5.e1,6.e1,7.e1,8.e1,9.e1, &
L00350                  1.e2,2.e2,3.e2,4.e2,5.e2,6.e2,7.e2,8.e2,9.e2, &
L00351                  1.e3,2.e3,3.e3,4.e3,5.e3,6.e3,7.e3,8.e3,9.e3, &
L00352                  1.e4,2.e4,3.e4,4.e4,5.e4,6.e4,7.e4,8.e4,9.e4, &
L00353                  1.e5,2.e5,3.e5,4.e5,5.e5,6.e5,7.e5,8.e5,9.e5, &
L00354                  1.e6/)
L00355
L00356   !..For snow moments conversions (from Field et al. 2005)
L00357        REAL, DIMENSION(10), PARAMETER, PRIVATE:: &
L00358        sa = (/ 5.065339, -0.062659, -3.032362, 0.029469, -0.000285, &
L00359                0.31255,   0.000204,  0.003199, 0.0,      -0.015952/)
L00360        REAL, DIMENSION(10), PARAMETER, PRIVATE:: &
L00361        sb = (/ 0.476221, -0.015896,  0.165977, 0.007468, -0.000141, &
L00362                0.060366,  0.000079,  0.000594, 0.0,      -0.003577/)
L00363
L00364   !..Temperatures (5 C interval 0 to -40) used in lookup tables.
L00365        REAL, DIMENSION(ntb_t), PARAMETER, PRIVATE:: &
L00366        Tc = (/-0.01, -5., -10., -15., -20., -25., -30., -35., -40./)
L00367
L00368   !..Lookup tables for various accretion/collection terms.
L00369   !.. ntb_x refers to the number of elements for rain, snow, graupel,
L00370   !.. and temperature array indices.  Variables beginning with t-p/c/m/n
L00371   !.. represent lookup tables.  Save compile-time memory by making
L00372   !.. allocatable (2009Jun12, J. Michalakes).
L00373        INTEGER, PARAMETER, PRIVATE:: R8SIZE = 8
L00374        INTEGER, PARAMETER, PRIVATE:: R4SIZE = 4
L00375        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:,:,:):: &
L00376                  tcg_racg, tmr_racg, tcr_gacr,                  & ! tmg_gacr
L00377                  tnr_racg, tnr_gacr
L00378        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:,:):: &
L00379                  tcs_racs1, tmr_racs1, tcs_racs2, tmr_racs2, &
L00380                  tcr_sacr1, tms_sacr1, tcr_sacr2, tms_sacr2, &
```

```
L00381                  tnr_racs1, tnr_racs2, tnr_sacr1, tnr_sacr2
L00382        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:,:):: &
L00383                  tpi_qcfz, tni_qcfz
L00384        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:,:):: &
L00385                  tpi_qrfz, tpg_qrfz, tni_qrfz, tnr_qrfz
L00386        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:):: &
L00387                  tps_iaus, tni_iaus, tpi_ide
L00388        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:):: t_Efrw
L00389        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:):: t_Efsw
L00390        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:):: tnr_rev
L00391        REAL (KIND=R8SIZE), ALLOCATABLE, DIMENSION(:,:,:):: &
L00392                  tpc_wev, tnc_wev
L00393        REAL (KIND=R4SIZE), ALLOCATABLE, DIMENSION(:,:,:,:,:):: tnccn_act
L00394
L00395  !..Variables holding a bunch of exponents and gamma values (cloud water,
L00396  !.. cloud ice, rain, snow, then graupel).
L00397        REAL, DIMENSION(5,15), PRIVATE:: cce, ccg
L00398        REAL, DIMENSION(15), PRIVATE::  ocg1, ocg2
L00399        REAL, DIMENSION(7), PRIVATE:: cie, cig
L00400        REAL, PRIVATE:: oig1, oig2, obmi
L00401        REAL, DIMENSION(13), PRIVATE:: cre, crg
L00402        REAL, PRIVATE:: ore1, org1, org2, org3, obmr
L00403        REAL, DIMENSION(17), PRIVATE:: cse, csg
L00404        REAL, PRIVATE:: oams, obms, ocms
L00405        REAL, DIMENSION(12,NRHG), PRIVATE:: cge, cgg
L00406        REAL, PRIVATE:: oge1, ogg1, ogg2, ogg3, obmg
L00407        REAL, DIMENSION(NRHG), PRIVATE:: oamg, ocmg
L00408
L00409  !..Declaration of precomputed constants in various rate eqns.
L00410        REAL:: t1_qr_qc, t1_qr_qi, t2_qr_qi, t1_qg_qc, t1_qs_qc, t1_qs_qi
L00411        REAL:: t1_qr_ev, t2_qr_ev
L00412        REAL:: t1_qs_sd, t2_qs_sd, t1_qg_sd, t2_qg_sd
L00413        REAL:: t1_qs_me, t2_qs_me, t1_qg_me, t2_qg_me
```

Thompson 快的秘诀一半在这里：**凡是可以离线算的积分，绝不在线算**。

- **bin 体系**（L231–L261）：六套 100 格粒径 bin（云、雨、雪、霰、冰、云滴数），bin 边界在 `thompson_init` 里按对数等距构建；`nic2…niIN2` 是各表的“10 的幂次”索引基准，用于把物理量直接折算成表内下标（主实体里 `idx = INT(q/10.**n) + 10*(n-nic2) - (n-nic2)` 一族）。
- **内容表**（L264–L354）：含水量表 `r_c/r_i/r_r/r_g/r_s`（每表首元素 1e-6 kg/m³ 量级、对数等距）、截距表 `N0r_exp/N0g_exp`、冰晶数表 `Nt_i`、冰核表 `Nt_IN`；气溶胶活化表五维轴 `ta_Na/ta_Ww/ta_Tk/ta_Ra/ta_Ka`（气溶胶数、垂直速度、温度、气溶胶半径、吸湿性）。
- **运行时大表**（L373–L393，ALLOCATABLE）：`tcg_racg` 等**雨-霰互相碰并表是五维**（霰截距 × 霰含量 × 霰密度档 × 雨截距 × 雨含量）——因为雨-霰碰撞-并合-破碎过程“不能像 Wisner (1972) 或 Mizuno (1990) 那样近似”（主实体原注释），只能显式二项展开后离线造表；`tnccn_act` 是气溶胶活化概率表；`t_Efrw/t_Efsw` 是碰并效率表。全部在 `thompson_init` 分配。
- **预计算矩阵**（L395–L413）：`cce/ccg`（云）、`cie/cig`（冰）、`cre/crg`（雨）、`cse/csg`（雪 17 组）、`cge/cgg`（霰 12×9 档）存的是各谱矩积分需要的 Γ 函数值与指数——例如 `cce(2,n) = bm_r + n + 1`，`ccg(2,n) = WGAMMA(cce(2,n))`。主实体里 `crg(9)`、`cgg(6,idx)` 这些下标取值就是预积分的矩；`t1_qr_qc` 等 14 个标量（L410–L413）是把“π/4 • av • Γ(•)”型的速率常数一次性折叠。这套记号初读晦涩，但一旦明白“**c**?e 是指数、**c**?g 是 Γ 值、o? 是其倒数”，全篇速率方程就都可读了。

07a.5 子程序总表

#	子程序/函数	行数	功能	教材 §
1	`thompson_init`	641	方案初始化：气溶胶缺省剖面、查找表分配、常数预计算、全部建表调用	13.6.1
2	`mp_gt_driver`	431	三维场 ↔ 单列适配：提列、调主实体、降水/诊断回写	13.6.1
3	`mp_thompson`	2570	单列微物理本体：全部源汇项计算与沉降	13.7 全节
4	`qr_acr_qg` / `qr_acr_qs`	216/279	雨-霰 / 雨-雪碰撞并合查找表构建（含破碎）	—
5	`freezeH2O`	192	Bigg (1953) 过冷水滴冻结表（含尘埃订正）	13.7.2
6	`qi_aut_qs`	44	冰晶自动转化雪表	13.7.3
7	`table_Efrw/Efsw/dropEvap`	57/37/89	雨/雪-云滴碰并效率表、雨蒸发表	—
8	`Eff_aero`	37	气溶胶清除效率	11 章指针
9	`table_ccnAct` / `activ_ncloud`	57/76	CCN 活化表构建 / 查表活化	13.7.2
10	`GCF/GSER/GAMMLN/GAMMP/WGAMMA`	34/30/23/19/8	不完全 Γ 函数族（Numerical Recipes 系）	—

#	子程序/函数	行数	功能	教材 §
11	`RSLF/RSIF`	31/27	液面/冰面饱和混合比（Flatau 1992 多项式）	11 章指针
12	`iceDeMott/iceKoop`	68/26	DeMott (2010) 尘埃冰核 / Koop (2001) 均质冻结	13.7.2
13	`delta_p`	30	雪分布矩量换算辅助	—
14	`calc_effectRad`	106	云/冰/雪有效半径 → 辐射接口	13.7.6
15	`calc_refl10cm`	319	10 cm 雷达反射率	—
16	`theta_e/t_lcl/t_dew/theta_wetb/compT_fr_The`	31/20/19/35/39	Bolton (1980) 热力学函数族（湿球温度反演用）	13.5.3

表中 4–16 号全部在 code_07b 逐段讲解；本篇（07a）只精讲前三个。

07a.6 `thompson_init` 逐段精讲

07a.6.1 签名与 “PRESENT 判型”

```
!file: phys/module_mp_thompson.F    L424-L466
L00424        SUBROUTINE thompson_init(hgt, orho, nwfa2d, nbca2d, ng,           &
L00425                                 nwfa, nifa, nbca, wif_input_opt, frc_urb2d,   &
L00426                                 dx, dy, is_start,                             &
L00427                                 ids, ide, jds, jde, kds, kde,                 &
L00428                                 ims, ime, jms, jme, kms, kme,                 &
L00429                                 its, ite, jts, jte, kts, kte)
L00430
L00431        IMPLICIT NONE
L00432
L00433        INTEGER, INTENT(IN):: ids,ide, jds,jde, kds,kde, &
L00434                              ims,ime, jms,jme, kms,kme, &
L00435                              its,ite, jts,jte, kts,kte
L00436        REAL, DIMENSION(ims:ime,kms:kme,jms:jme), INTENT(IN):: hgt
L00437
L00438  !..OPTIONAL variables that control application of hail-aware scheme
L00439        REAL, DIMENSION(ims:ime,kms:kme,jms:jme), OPTIONAL, INTENT(INOUT) :: ng
L00440
L00441  !..OPTIONAL variables that control application of aerosol-aware scheme
L00442
L00443        REAL, DIMENSION(ims:ime,kms:kme,jms:jme), OPTIONAL, INTENT(INOUT) :: nwfa, nifa, nbca
L00444        REAL, DIMENSION(ims:ime,jms:jme), OPTIONAL, INTENT(INOUT) :: nwfa2d, nbca2d
L00445        REAL, DIMENSION(ims:ime,kms:kme,jms:jme), OPTIONAL, INTENT(IN) :: orho
L00446        REAL, DIMENSION(ims:ime,jms:jme), OPTIONAL, INTENT(IN) :: frc_urb2d
L00447        REAL, OPTIONAL, INTENT(IN) :: DX, DY
L00448        LOGICAL, OPTIONAL, INTENT(IN) :: is_start
L00449        INTEGER, OPTIONAL, INTENT(IN) :: wif_input_opt
L00450        CHARACTER*256:: mp_debug
L00451
L00452
L00453        INTEGER:: i, j, k, l, m, n
L00454        REAL:: h_01, niIN3, niCCN3, niBC3, max_test, z1
L00455        LOGICAL:: micro_init, has_CCN, has_IN
L00456
L00457  !+---+
L00458
L00459        if (PRESENT(ng)) then
L00460           is_hail_aware = .TRUE.
L00461           dimNRHG = NRHG
L00462        else
L00463           av_g(idx_bg1) = av_g_old
L00464           bv_g(idx_bg1) = bv_g_old
L00465           dimNRHG = NRHG1
L00466        endif
```

- **参数即开关**：这个初始化子程序没有任何 namelist 输入——它的 “配置” 全靠 `OPTIONAL` 实参是否在场：传了 `ng` → hail-aware（三矩霰）；传了 `nwfa/nifa` → aerosol-aware。这与 `microphysics_driver` 从 Registry 包（`package thompsonaero`）取实参的方式相扣：**方案号 → Registry 包 → 实参在/不在 → 方案行为分叉**，一条完整的编译-运行期配置链（教材 ch13 §13.6.4 讲 Registry 包，ch04 讲机制）。
- **`dimNRHG` 的妙用**：hail-aware 时查找表霰密度维开满 9 档；否则只用 1 档（第 5 档 500 kg/m³ 的参数副本 `av_g_old/bv_g_old` 写回 `idx_bg1` 位）——同一套代码跑两代方案，内存差 9 倍。
- `hgt` 是地形高度，`orho` 是参考密度；`frc_urb2d` 是**城市覆盖比**——黑碳人为源在城市加权，见 init 第二段 `nbca2d` 一行。

07a.6.2 气溶胶缺省剖面

```
!file: phys/module_mp_thompson.F    L468-L602
L00468        is_aerosol_aware = .FALSE.
L00469        micro_init = .FALSE.
L00470        has_CCN    = .FALSE.
L00471        has_IN     = .FALSE.
L00472
L00473        write(mp_debug,*) ' DEBUG  checking column of hgt ', its+1,jts+1
L00474        CALL wrf_debug(250, mp_debug)
```

```
L00475         do k = kts, kte
L00476            write(mp_debug,*) ' DEBUGT  k, hgt = ', k, hgt(its+1,k,jts+1)
L00477            CALL wrf_debug(250, mp_debug)
L00478         enddo
L00479
L00480         if (PRESENT(nwfa2d) .AND. PRESENT(nwfa) .AND. PRESENT(nifa)) is_aerosol_aware = .TRUE.
L00481
L00482         if (is_aerosol_aware) then
L00483
L00484  !..Check for existing aerosol data, both CCN and IN aerosols.  If missing
L00485  !.. fill in just a basic vertical profile, somewhat boundary-layer following.
L00486
L00487  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L00488         max_test = wrf_dm_max_real ( MAXVAL(nwfa(its:ite-1,:,jts:jte-1)) )
L00489  #else
L00490         max_test = MAXVAL ( nwfa(its:ite-1,:,jts:jte-1) )
L00491  #endif
L00492
L00493         if (max_test .lt. eps) then
L00494            write(mp_debug,*) ' Apparently there are no initial CCN aerosols.'
L00495            CALL wrf_debug(100, mp_debug)
L00496            write(mp_debug,*) '   checked column at point (i,j) = ', its,jts
L00497            CALL wrf_debug(100, mp_debug)
L00498            do j = jts, min(jde-1,jte)
L00499            do i = its, min(ide-1,ite)
L00500               if (hgt(i,1,j).le.1000.0) then
L00501                  h_01 = 0.8
L00502               elseif (hgt(i,1,j).ge.2500.0) then
L00503                  h_01 = 0.01
L00504               else
L00505                  h_01 = 0.8*cos(hgt(i,1,j)*0.001 - 1.0)
L00506               endif
L00507               niCCN3 = -1.0*ALOG(naCCN1/naCCN0)/h_01
L00508               nwfa(i,1,j) = naCCN1+naCCN0*exp(-((hgt(i,2,j)-hgt(i,1,j))/1000.)*niCCN3)
L00509               z1=hgt(i,2,j)-hgt(i,1,j)
L00510               nwfa2d(i,j) = nwfa(i,1,j) * 0.000196 * (50./z1)
L00511               do k = 2, kte
L00512                  nwfa(i,k,j) = naCCN1+naCCN0*exp(-((hgt(i,k,j)-hgt(i,1,j))/1000.)*niCCN3)
L00513               enddo
L00514            enddo
L00515            enddo
L00516         else
L00517            has_CCN    = .TRUE.
L00518            write(mp_debug,*) ' Apparently initial CCN aerosols are present.'
L00519            CALL wrf_debug(100, mp_debug)
L00520            write(mp_debug,*) '   column sum at point (i,j) = ', its,jts, SUM(nwfa(its,:,jts))
L00521            CALL wrf_debug(100, mp_debug)
L00522         endif
L00523
L00524
L00525  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L00526         max_test = wrf_dm_max_real ( MAXVAL(nifa(its:ite-1,:,jts:jte-1)) )
L00527  #else
L00528         max_test = MAXVAL ( nifa(its:ite-1,:,jts:jte-1) )
L00529  #endif
L00530
L00531         if (max_test .lt. eps) then
L00532            write(mp_debug,*) ' Apparently there are no initial IN aerosols.'
L00533            CALL wrf_debug(100, mp_debug)
L00534            write(mp_debug,*) '   checked column at point (i,j) = ', its,jts
L00535            CALL wrf_debug(100, mp_debug)
L00536            do j = jts, min(jde-1,jte)
L00537            do i = its, min(ide-1,ite)
L00538               if (hgt(i,1,j).le.1000.0) then
L00539                  h_01 = 0.8
L00540               elseif (hgt(i,1,j).ge.2500.0) then
L00541                  h_01 = 0.01
L00542               else
L00543                  h_01 = 0.8*cos(hgt(i,1,j)*0.001 - 1.0)
L00544               endif
L00545               niIN3 = -1.0*ALOG(naIN1/naIN0)/h_01
L00546               nifa(i,1,j) = naIN1+naIN0*exp(-((hgt(i,2,j)-hgt(i,1,j))/1000.)*niIN3)
L00547               do k = 2, kte
L00548                  nifa(i,k,j) = naIN1+naIN0*exp(-((hgt(i,k,j)-hgt(i,1,j))/1000.)*niIN3)
L00549               enddo
L00550            enddo
L00551            enddo
L00552         else
L00553            has_IN     = .TRUE.
L00554            write(mp_debug,*) ' Apparently initial IN aerosols are present.'
L00555            CALL wrf_debug(100, mp_debug)
L00556            write(mp_debug,*) '   column sum at point (i,j) = ', its,jts, SUM(nifa(its,:,jts))
L00557            CALL wrf_debug(100, mp_debug)
L00558         endif
L00559
L00560
L00561         if ( wif_input_opt .eq. 2 ) then
L00562
L00563  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L00564            max_test = wrf_dm_max_real ( MAXVAL(nbca(its:ite-1,:,jts:jte-1)) )
L00565  #else
L00566            max_test = MAXVAL ( nbca(its:ite-1,:,jts:jte-1) )
L00567  #endif
L00568
L00569            if (max_test .lt. eps) then
L00570               write(mp_debug,*) ' Apparently there are no initial BC aerosols.'
L00571               CALL wrf_debug(100, mp_debug)
L00572               write(mp_debug,*) '   checked column at point (i,j) = ', its,jts
L00573               CALL wrf_debug(100, mp_debug)
L00574               do j = jts, min(jde-1,jte)
L00575               do i = its, min(ide-1,ite)
L00576                  if (hgt(i,1,j).le.1000.0) then
L00577                     h_01 = 0.8
L00578                  elseif (hgt(i,1,j).ge.2500.0) then
L00579                     h_01 = 0.01
L00580                  else
L00581                     h_01 = 0.8*cos(hgt(i,1,j)*0.001 - 1.0)
L00582                  endif
```

```
L00583                  niBC3 = -1.0*ALOG(naBC1/naBC0)/h_01
L00584                  nbca(i,1,j) = naBC1+naBC0*exp(-((hgt(i,2,j)-hgt(i,1,j))/1000.)*niBC3)
L00585                  z1=hgt(i,2,j)-hgt(i,1,j)
L00586                  nbca2d(i,j) = nbca(i,1,j) * 0.000098 * (50./z1) * (1. + frc_urb2d(i,j))
L00587                  do k = 2, kte
L00588                     nbca(i,k,j) = naBC1+naBC0*exp(-((hgt(i,k,j)-hgt(i,1,j))/1000.)*niBC3)
L00589                  enddo
L00590               enddo
L00591               enddo
L00592            else
L00593               write(mp_debug,*) ' Apparently initial BC aerosols are present.'
L00594               CALL wrf_debug(100, mp_debug)
L00595               write(mp_debug,*) '   column sum at point (i,j) = ', its,jts, SUM(nbca(its,:,jts))
L00596               CALL wrf_debug(100, mp_debug)
L00597            endif
L00598
L00599         endif
L00600
L00601         endif
L00602
```

三段结构完全平行（CCN/IN/黑碳），以 CCN 段为例：

- **全局判空**：`wrf_dm_max_real(MAXVAL(nwfa(its:ite-1,:,jts:jte-1)))`——并行归约取全场最大值，`< eps` 即认为输入没提供气溶胶（`wrfinput` 里 `nwfa` 全零）。注意下标 `its:ite-1`：最右侧/最上层一格是 staggered 冗余位，不计。
- **缺省指数剖面**：`h_01` 是标高系数——海平面以下 0.8、2500 m 以上 0.01、中间余弦过渡（山地气溶胶少、边界层多）；`niCCN3 = -ln(naCCN1/naCCN0)/h_01` 是使剖面在标高处从 `naCCN0` 衰减到 `naCCN1` 的指数；随后 `nwfa(i,k,j) = naCCN1 + naCCN0•exp(-(z_k - z_1)/1000•niCCN3)` 逐层填充。物理上这就是 “边界层富集、自由大气本底” 的一维气候剖面。
- **2D 通量场**：`nwfa2d = nwfa(i,1,j)•0.000196•(50/z1)`——底层气溶胶换算成 “数/kg/s” 的排放通量尺度（0.000196 是质量混合比换算系数，z1 是最低层厚度，50 m 归一）。这个 2D 场每步被 `mp_gt_driver` 当 “伪排放” 加回最底层（见 §07a.7 driver 第三段）。
- **黑碳段的差别**：只在 `wif_input_opt=2` 时启用，且 `nbca2d` 乘 `(1 + frc_urb2d)`——城市格点黑碳排放加倍以上。
- `wif_input_opt` 全称 water-ice-friendly input option：1=只有水亲和性气溶胶、2=再加黑碳（冰亲和）。它与 `aer_init_opt`（气溶胶初值来源：气候态/GEOS-5 逐级）共同构成 Thompson 4.8 的气溶胶配置面，Registry 关联见 ch13 §13.6.4。

07a.6.3 查找表分配与常数预计算

```
!file: phys/module_mp_thompson.F    L604-L779
L00604  !..Allocate space for lookup tables (J. Michalakes 2009Jun08).
L00605
L00606         if (.NOT. ALLOCATED(tcg_racg) ) then
L00607            ALLOCATE(tcg_racg(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00608            micro_init = .TRUE.
L00609         endif
L00610
L00611         if (.NOT. ALLOCATED(tmr_racg)) ALLOCATE(tmr_racg(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00612         if (.NOT. ALLOCATED(tcr_gacr)) ALLOCATE(tcr_gacr(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00613         ! if (.NOT. ALLOCATED(tmg_gacr)) ALLOCATE(tmg_gacr(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00614         if (.NOT. ALLOCATED(tnr_racg)) ALLOCATE(tnr_racg(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00615         if (.NOT. ALLOCATED(tnr_gacr)) ALLOCATE(tnr_gacr(ntb_g1,ntb_g,dimNRHG,ntb_r1,ntb_r))
L00616
L00617         if (.NOT. ALLOCATED(tcs_racs1)) ALLOCATE(tcs_racs1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00618         if (.NOT. ALLOCATED(tmr_racs1)) ALLOCATE(tmr_racs1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00619         if (.NOT. ALLOCATED(tcs_racs2)) ALLOCATE(tcs_racs2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00620         if (.NOT. ALLOCATED(tmr_racs2)) ALLOCATE(tmr_racs2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00621         if (.NOT. ALLOCATED(tcr_sacr1)) ALLOCATE(tcr_sacr1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00622         if (.NOT. ALLOCATED(tms_sacr1)) ALLOCATE(tms_sacr1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00623         if (.NOT. ALLOCATED(tcr_sacr2)) ALLOCATE(tcr_sacr2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00624         if (.NOT. ALLOCATED(tms_sacr2)) ALLOCATE(tms_sacr2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00625         if (.NOT. ALLOCATED(tnr_racs1)) ALLOCATE(tnr_racs1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00626         if (.NOT. ALLOCATED(tnr_racs2)) ALLOCATE(tnr_racs2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00627         if (.NOT. ALLOCATED(tnr_sacr1)) ALLOCATE(tnr_sacr1(ntb_s,ntb_t,ntb_r1,ntb_r))
L00628         if (.NOT. ALLOCATED(tnr_sacr2)) ALLOCATE(tnr_sacr2(ntb_s,ntb_t,ntb_r1,ntb_r))
L00629
L00630         if (.NOT. ALLOCATED(tpi_qcfz)) ALLOCATE(tpi_qcfz(ntb_c,nbc,45,ntb_IN))
L00631         if (.NOT. ALLOCATED(tni_qcfz)) ALLOCATE(tni_qcfz(ntb_c,nbc,45,ntb_IN))
L00632
L00633         if (.NOT. ALLOCATED(tpi_qrfz)) ALLOCATE(tpi_qrfz(ntb_r,ntb_r1,45,ntb_IN))
L00634         if (.NOT. ALLOCATED(tpg_qrfz)) ALLOCATE(tpg_qrfz(ntb_r,ntb_r1,45,ntb_IN))
L00635         if (.NOT. ALLOCATED(tni_qrfz)) ALLOCATE(tni_qrfz(ntb_r,ntb_r1,45,ntb_IN))
L00636         if (.NOT. ALLOCATED(tnr_qrfz)) ALLOCATE(tnr_qrfz(ntb_r,ntb_r1,45,ntb_IN))
L00637
L00638         if (.NOT. ALLOCATED(tps_iaus)) ALLOCATE(tps_iaus(ntb_i,ntb_i1))
L00639         if (.NOT. ALLOCATED(tni_iaus)) ALLOCATE(tni_iaus(ntb_i,ntb_i1))
L00640         if (.NOT. ALLOCATED(tpi_ide)) ALLOCATE(tpi_ide(ntb_i,ntb_i1))
L00641
L00642         if (.NOT. ALLOCATED(t_Efrw)) ALLOCATE(t_Efrw(nbr,nbc))
L00643         if (.NOT. ALLOCATED(t_Efsw)) ALLOCATE(t_Efsw(nbs,nbc))
L00644
L00645         if (.NOT. ALLOCATED(tnr_rev)) ALLOCATE(tnr_rev(nbr, ntb_r1, ntb_r))
L00646         if (.NOT. ALLOCATED(tpc_wev)) ALLOCATE(tpc_wev(nbc,ntb_c,nbc))
L00647         if (.NOT. ALLOCATED(tnc_wev)) ALLOCATE(tnc_wev(nbc,ntb_c,nbc))
L00648
L00649         if (.NOT. ALLOCATED(tnccn_act))                                    &
L00650               ALLOCATE(tnccn_act(ntb_arc,ntb_arw,ntb_art,ntb_arr,ntb_ark))
L00651
```

```
L00652        if (micro_init) then
L00653
L00654  !..From Martin et al. (1994), assign gamma shape parameter mu for cloud
L00655  !.. drops according to general dispersion characteristics (disp=~0.25
L00656  !.. for Maritime and 0.45 for Continental).
L00657  !.. disp=SQRT((mu+2)/(mu+1) - 1) so mu varies from 15 for Maritime
L00658  !.. to 2 for really dirty air.  This not used in 2-moment cloud water
L00659  !.. scheme and nu_c used instead and varies from 2 to 15 (integer-only).
L00660        mu_c = MIN(15., (1000.E6/Nt_c + 2.))
L00661
L00662  !..Schmidt number to one-third used numerous times.
L00663        Sc3 = Sc**(1./3.)
L00664
L00665  !..Compute min ice diam from mass, min snow/graupel mass from diam.
L00666        D0i = (xm0i/am_i)**(1./bm_i)
L00667        xm0s = am_s * D0s**bm_s
L00668        xm0g = am_g(NRHG) * D0g**bm_g
L00669
L00670  !..These constants various exponents and gamma() assoc with cloud,
L00671  !.. rain, snow, and graupel.
L00672        do n = 1, 15
L00673           cce(1,n) = n + 1.
L00674           cce(2,n) = bm_r + n + 1.
L00675           cce(3,n) = bm_r + n + 4.
L00676           cce(4,n) = n + bv_c + 1.
L00677           cce(5,n) = bm_r + n + bv_c + 1.
L00678           ccg(1,n) = WGAMMA(cce(1,n))
L00679           ccg(2,n) = WGAMMA(cce(2,n))
L00680           ccg(3,n) = WGAMMA(cce(3,n))
L00681           ccg(4,n) = WGAMMA(cce(4,n))
L00682           ccg(5,n) = WGAMMA(cce(5,n))
L00683           ocg1(n) = 1./ccg(1,n)
L00684           ocg2(n) = 1./ccg(2,n)
L00685        enddo
L00686
L00687        cie(1) = mu_i + 1.
L00688        cie(2) = bm_i + mu_i + 1.
L00689        cie(3) = bm_i + mu_i + bv_i + 1.
L00690        cie(4) = mu_i + bv_i + 1.
L00691        cie(5) = mu_i + 2.
L00692        cie(6) = bm_i*0.5 + mu_i + bv_i + 1.
L00693        cie(7) = bm_i*0.5 + mu_i + 1.
L00694        cig(1) = WGAMMA(cie(1))
L00695        cig(2) = WGAMMA(cie(2))
L00696        cig(3) = WGAMMA(cie(3))
L00697        cig(4) = WGAMMA(cie(4))
L00698        cig(5) = WGAMMA(cie(5))
L00699        cig(6) = WGAMMA(cie(6))
L00700        cig(7) = WGAMMA(cie(7))
L00701        oig1 = 1./cig(1)
L00702        oig2 = 1./cig(2)
L00703        obmi = 1./bm_i
L00704
L00705        cre(1) = bm_r + 1.
L00706        cre(2) = mu_r + 1.
L00707        cre(3) = bm_r + mu_r + 1.
L00708        cre(4) = bm_r*2. + mu_r + 1.
L00709        cre(5) = mu_r + bv_r + 1.
L00710        cre(6) = bm_r + mu_r + bv_r + 1.
L00711        cre(7) = bm_r*0.5 + mu_r + bv_r + 1.
L00712        cre(8) = bm_r + mu_r + bv_r + 3.
L00713        cre(9) = mu_r + bv_r + 3.
L00714        cre(10) = mu_r + 2.
L00715        cre(11) = 0.5*(bv_r + 5. + 2.*mu_r)
L00716        cre(12) = bm_r*0.5 + mu_r + 1.
L00717        cre(13) = bm_r*2. + mu_r + bv_r + 1.
L00718        do n = 1, 13
L00719           crg(n) = WGAMMA(cre(n))
L00720        enddo
L00721        obmr = 1./bm_r
L00722        ore1 = 1./cre(1)
L00723        org1 = 1./crg(1)
L00724        org2 = 1./crg(2)
L00725        org3 = 1./crg(3)
L00726
L00727        cse(1) = bm_s + 1.
L00728        cse(2) = bm_s + 2.
L00729        cse(3) = bm_s*2.
L00730        cse(4) = bm_s + bv_s + 1.
L00731        cse(5) = bm_s*2. + bv_s + 1.
L00732        cse(6) = bm_s*2. + 1.
L00733        cse(7) = bm_s + mu_s + 1.
L00734        cse(8) = bm_s + mu_s + 2.
L00735        cse(9) = bm_s + mu_s + 3.
L00736        cse(10) = bm_s + mu_s + bv_s + 1.
L00737        cse(11) = bm_s*2. + mu_s + bv_s + 1.
L00738        cse(12) = bm_s*2. + mu_s + 1.
L00739        cse(13) = bv_s + 2.
L00740        cse(14) = bm_s + bv_s
L00741        cse(15) = mu_s + 1.
L00742        cse(16) = 1.0 + (1.0 + bv_s)/2.
L00743        cse(17) = bm_s + bv_s + 2.
L00744        do n = 1, 17
L00745           csg(n) = WGAMMA(cse(n))
L00746        enddo
L00747        oams = 1./am_s
L00748        obms = 1./bm_s
L00749        ocms = oams**obms
L00750
L00751        cge(1,:) = bm_g + 1.
L00752        cge(2,:) = mu_g + 1.
L00753        cge(3,:) = bm_g + mu_g + 1.
L00754        cge(4,:) = bm_g*2. + mu_g + 1.
L00755        cge(10,:) = mu_g + 2.
L00756        cge(12,:) = bm_g*0.5 + mu_g + 1.
L00757        do m = 1, NRHG
L00758           cge(5,m) = bm_g*2. + mu_g + bv_g(m) + 1.
```

```
L00759           cge(6,m) = bm_g + mu_g + bv_g(m) + 1.
L00760           cge(7,m) = bm_g*0.5 + mu_g + bv_g(m) + 1.
L00761           cge(8,m) = mu_g + bv_g(m) + 1.                                  ! not used
L00762           cge(9,m) = mu_g + bv_g(m) + 3.
L00763           cge(11,m) = 0.5*(bv_g(m) + 5. + 2.*mu_g)
L00764        enddo
L00765        do m = 1, NRHG
L00766        do n = 1, 12
L00767           cgg(n,m) = WGAMMA(cge(n,m))
L00768        enddo
L00769        enddo
L00770        oamg = 1./am_g
L00771        obmg = 1./bm_g
L00772        do m = 1, NRHG
L00773           oamg(m) = 1./am_g(m)
L00774           ocmg(m) = oamg(m)**obmg
L00775        enddo
L00776        oge1 = 1./cge(1,1)
L00777        ogg1 = 1./cgg(1,1)
L00778        ogg2 = 1./cgg(2,1)
L00779        ogg3 = 1./cgg(3,1)
```

- **分配纪律**：所有 `if (.NOT. ALLOCATED(...)) ALLOCATE(...)`——模块表是**进程级单例**，多域/多次调用不重复分配；`micro_init` 只在首轮置真，后面的建表与预计算整个跳过（嵌套运行、restart 都安全）。Michalakes 2009 的改造注释（模块声明段 L371）说明了把静态大表改 ALLOCATABLE 是为了省编译期内存。
- **预计算四步**：①`mu_c`（Martin 1994 离散度 →gamma 形参数）；②`Sc3`；③ 最小冰晶直径 `D0i = (xm0i/am_i)^(1/bm_i)`——由出生质量反解直径，还有雪/霰最小质量 `xm0s/xm0g`（由最小直径正解质量）；④ 五行 `cce/ccg` 循环——云滴谱所有积分矩的 Γ 值（`cce(2,n) = bm_r + n + 1` 等），随后是冰 7 个、雨 13 个、雪 17 个、霰 12×9 个。**这一段是全方案数值工作量的“一次性预付”**。
- `ccg(1,n)·ocg1(n)` 这样的组合（Γ 值乘其倒数）在主实体反复出现，形态是 `Γ(x+1)/Γ(x) = x` 的推广——批量方案把所有 `∫D^n·N(D)dD` 归约为 `N_0·Γ(·)/λ^(·)` 后，就只剩这些预存系数。

07a.6.4 粒径 bin 与建表调用链

```
!file: phys/module_mp_thompson.F    L781-L1064
L00781  !+---+-----------------------------------------------------------------+
L00782  !..Simplify various rate eqns the best we can now.
L00783  !+---+-----------------------------------------------------------------+
L00784
L00785  !..Rain collecting cloud water and cloud ice
L00786        t1_qr_qc = PI*.25*av_r * crg(9)
L00787        t1_qr_qi = PI*.25*av_r * crg(9)
L00788        t2_qr_qi = PI*.25*am_r*av_r * crg(8)
L00789
L00790  !..Graupel collecting cloud water
L00791  !     t1_qg_qc = PI*.25*av_g * cgg(9)
L00792
L00793  !..Snow collecting cloud water
L00794        t1_qs_qc = PI*.25*av_s
L00795
L00796  !..Snow collecting cloud ice
L00797        t1_qs_qi = PI*.25*av_s
L00798
L00799  !..Evaporation of rain; ignore depositional growth of rain.
L00800        t1_qr_ev = 0.78 * crg(10)
L00801        t2_qr_ev = 0.308*Sc3*SQRT(av_r) * crg(11)
L00802
L00803  !..Sublimation/depositional growth of snow
L00804        t1_qs_sd = 0.86
L00805        t2_qs_sd = 0.28*Sc3*SQRT(av_s)
L00806
L00807  !..Melting of snow
L00808        t1_qs_me = PI*4.*C_sqrd*olfus * 0.86
L00809        t2_qs_me = PI*4.*C_sqrd*olfus * 0.28*Sc3*SQRT(av_s)
L00810
L00811  !..Sublimation/depositional growth of graupel
L00812        t1_qg_sd = 0.86 * cgg(10,1)
L00813  !     t2_qg_sd = 0.28*Sc3*SQRT(av_g) * cgg(11)
L00814
L00815  !..Melting of graupel
L00816        t1_qg_me = PI*4.*C_cube*olfus * 0.86 * cgg(10,1)
L00817  !     t2_qg_me = PI*4.*C_cube*olfus * 0.28*Sc3*SQRT(av_g) * cgg(11)
L00818
L00819  !..Constants for helping find lookup table indexes.
L00820        nic2 = NINT(ALOG10(r_c(1)))
L00821        nii2 = NINT(ALOG10(r_i(1)))
L00822        nii3 = NINT(ALOG10(Nt_i(1)))
L00823        nir2 = NINT(ALOG10(r_r(1)))
L00824        nir3 = NINT(ALOG10(N0r_exp(1)))
L00825        nis2 = NINT(ALOG10(r_s(1)))
L00826        nig2 = NINT(ALOG10(r_g(1)))
L00827        nig3 = NINT(ALOG10(N0g_exp(1)))
L00828        niIN2 = NINT(ALOG10(Nt_IN(1)))
L00829
L00830  !..Create bins of cloud water (from min diameter up to 100 microns).
L00831        Dc(1) = D0c*1.0d0
L00832        dtc(1) = D0c*1.0d0
L00833        do n = 2, nbc
L00834           Dc(n) = Dc(n-1) + 1.0D-6
L00835           dtc(n) = (Dc(n) - Dc(n-1))
L00836        enddo
```

```
L00837
L00838  !..Create bins of cloud ice (from min diameter up to 5x min snow size).
L00839         xDx(1) = D0i*1.0d0
L00840         xDx(nbi+1) = 2.0d0*D0s
L00841         do n = 2, nbi
L00842            xDx(n) = DEXP(DFLOAT(n-1)/DFLOAT(nbi) &
L00843                     *DLOG(xDx(nbi+1)/xDx(1)) +DLOG(xDx(1)))
L00844         enddo
L00845         do n = 1, nbi
L00846            Di(n) = DSQRT(xDx(n)*xDx(n+1))
L00847            dti(n) = xDx(n+1) - xDx(n)
L00848         enddo
L00849
L00850  !..Create bins of rain (from min diameter up to 5 mm).
L00851         xDx(1) = D0r*1.0d0
L00852         xDx(nbr+1) = 0.005d0
L00853         do n = 2, nbr
L00854            xDx(n) = DEXP(DFLOAT(n-1)/DFLOAT(nbr) &
L00855                     *DLOG(xDx(nbr+1)/xDx(1)) +DLOG(xDx(1)))
L00856         enddo
L00857         do n = 1, nbr
L00858            Dr(n) = DSQRT(xDx(n)*xDx(n+1))
L00859            dtr(n) = xDx(n+1) - xDx(n)
L00860         enddo
L00861
L00862  !..Create bins of snow (from min diameter up to 2 cm).
L00863         xDx(1) = D0s*1.0d0
L00864         xDx(nbs+1) = 0.02d0
L00865         do n = 2, nbs
L00866            xDx(n) = DEXP(DFLOAT(n-1)/DFLOAT(nbs) &
L00867                     *DLOG(xDx(nbs+1)/xDx(1)) +DLOG(xDx(1)))
L00868         enddo
L00869         do n = 1, nbs
L00870            Ds(n) = DSQRT(xDx(n)*xDx(n+1))
L00871            dts(n) = xDx(n+1) - xDx(n)
L00872         enddo
L00873
L00874  !..Create bins of graupel (from min diameter up to 5 cm).
L00875         xDx(1) = D0g*1.0d0
L00876         xDx(nbg+1) = 0.05d0
L00877         do n = 2, nbg
L00878            xDx(n) = DEXP(DFLOAT(n-1)/DFLOAT(nbg) &
L00879                     *DLOG(xDx(nbg+1)/xDx(1)) +DLOG(xDx(1)))
L00880         enddo
L00881         do n = 1, nbg
L00882            Dg(n) = DSQRT(xDx(n)*xDx(n+1))
L00883            dtg(n) = xDx(n+1) - xDx(n)
L00884         enddo
L00885
L00886  !..Create bins of cloud droplet number concentration (1 to 3000 per cc).
L00887         xDx(1) = 1.0d0
L00888         xDx(nbc+1) = 3000.0d0
L00889         do n = 2, nbc
L00890            xDx(n) = DEXP(DFLOAT(n-1)/DFLOAT(nbc)                          &
L00891                     *DLOG(xDx(nbc+1)/xDx(1)) +DLOG(xDx(1)))
L00892         enddo
L00893         do n = 1, nbc
L00894            t_Nc(n) = DSQRT(xDx(n)*xDx(n+1)) * 1.D6
L00895         enddo
L00896         nic1 = DLOG(t_Nc(nbc)/t_Nc(1))
L00897
L00898  !+---+-----------------------------------------------------------------+
L00899  !..Create lookup tables for most costly calculations.
L00900  !+---+-----------------------------------------------------------------+
L00901
L00902         do m = 1, ntb_r
L00903            do k = 1, ntb_r1
L00904               do n = 1, dimNRHG
L00905               do j = 1, ntb_g
L00906                  do i = 1, ntb_g1
L00907                        tcg_racg(i,j,n,k,m) = 0.0d0
L00908                        tmr_racg(i,j,n,k,m) = 0.0d0
L00909                        tcr_gacr(i,j,n,k,m) = 0.0d0
L00910                        !tmg_gacr(i,j,n,k,m) = 0.0d0
L00911                        tnr_racg(i,j,n,k,m) = 0.0d0
L00912                        tnr_gacr(i,j,n,k,m) = 0.0d0
L00913                     enddo
L00914                  enddo
L00915               enddo
L00916            enddo
L00917         enddo
L00918
L00919         do m = 1, ntb_r
L00920            do k = 1, ntb_r1
L00921               do j = 1, ntb_t
L00922                  do i = 1, ntb_s
L00923                     tcs_racs1(i,j,k,m) = 0.0d0
L00924                     tmr_racs1(i,j,k,m) = 0.0d0
L00925                     tcs_racs2(i,j,k,m) = 0.0d0
L00926                     tmr_racs2(i,j,k,m) = 0.0d0
L00927                     tcr_sacr1(i,j,k,m) = 0.0d0
L00928                     tms_sacr1(i,j,k,m) = 0.0d0
L00929                     tcr_sacr2(i,j,k,m) = 0.0d0
L00930                     tms_sacr2(i,j,k,m) = 0.0d0
L00931                     tnr_racs1(i,j,k,m) = 0.0d0
L00932                     tnr_racs2(i,j,k,m) = 0.0d0
L00933                     tnr_sacr1(i,j,k,m) = 0.0d0
L00934                     tnr_sacr2(i,j,k,m) = 0.0d0
L00935                  enddo
L00936               enddo
L00937            enddo
L00938         enddo
L00939
L00940         do m = 1, ntb_IN
L00941            do k = 1, 45
L00942               do j = 1, ntb_r1
L00943                  do i = 1, ntb_r
L00944                     tpi_qrfz(i,j,k,m) = 0.0d0
```

```
L00945                     tni_qrfz(i,j,k,m) = 0.0d0
L00946                     tpg_qrfz(i,j,k,m) = 0.0d0
L00947                     tnr_qrfz(i,j,k,m) = 0.0d0
L00948                  enddo
L00949               enddo
L00950               do j = 1, nbc
L00951                  do i = 1, ntb_c
L00952                     tpi_qcfz(i,j,k,m) = 0.0d0
L00953                     tni_qcfz(i,j,k,m) = 0.0d0
L00954                  enddo
L00955               enddo
L00956            enddo
L00957         enddo
L00958
L00959         do j = 1, ntb_i1
L00960            do i = 1, ntb_i
L00961               tps_iaus(i,j) = 0.0d0
L00962               tni_iaus(i,j) = 0.0d0
L00963               tpi_ide(i,j) = 0.0d0
L00964            enddo
L00965         enddo
L00966
L00967         do j = 1, nbc
L00968            do i = 1, nbr
L00969               t_Efrw(i,j) = 0.0
L00970            enddo
L00971            do i = 1, nbs
L00972               t_Efsw(i,j) = 0.0
L00973            enddo
L00974         enddo
L00975
L00976         do k = 1, ntb_r
L00977            do j = 1, ntb_r1
L00978               do i = 1, nbr
L00979                  tnr_rev(i,j,k) = 0.0d0
L00980               enddo
L00981            enddo
L00982         enddo
L00983
L00984         do k = 1, nbc
L00985            do j = 1, ntb_c
L00986               do i = 1, nbc
L00987                  tpc_wev(i,j,k) = 0.0d0
L00988                  tnc_wev(i,j,k) = 0.0d0
L00989               enddo
L00990            enddo
L00991         enddo
L00992
L00993         do m = 1, ntb_ark
L00994            do l = 1, ntb_arr
L00995               do k = 1, ntb_art
L00996                  do j = 1, ntb_arw
L00997                     do i = 1, ntb_arc
L00998                        tnccn_act(i,j,k,l,m) = 1.0
L00999                     enddo
L01000                  enddo
L01001               enddo
L01002            enddo
L01003         enddo
L01004
L01005         CALL wrf_debug(150, 'CREATING MICROPHYSICS LOOKUP TABLES ... ')
L01006         WRITE (wrf_err_message, '(a, f5.2, a, f5.2, a, f5.2, a, f5.2)') &
L01007             ' using: mu_c=',mu_c,' mu_i=',mu_i,' mu_r=',mu_r,' mu_g=',mu_g
L01008         CALL wrf_debug(150, wrf_err_message)
L01009
L01010   !..Read a static file containing CCN activation of aerosols. The
L01011   !.. data were created from a parcel model by Feingold & Heymsfield with
L01012   !.. further changes by Eidhammer and Kriedenweis.
L01013         if (is_aerosol_aware) then
L01014            CALL wrf_debug(200, '  calling table_ccnAct routine')
L01015            call table_ccnAct
L01016         endif
L01017
L01018   !..Collision efficiency between rain/snow and cloud water.
L01019         CALL wrf_debug(200, '  creating qc collision eff tables')
L01020         call table_Efrw
L01021         call table_Efsw
L01022
L01023   !..Drop evaporation.
L01024         CALL wrf_debug(200, '  creating rain evap table')
L01025         call table_dropEvap
L01026
L01027   !..Initialize various constants for computing radar reflectivity.
L01028         xam_r = am_r
L01029         xbm_r = bm_r
L01030         xmu_r = mu_r
L01031         xam_s = am_s
L01032         xbm_s = bm_s
L01033         xmu_s = mu_s
L01034         xam_g = am_g(idx_bg1)
L01035         xbm_g = bm_g
L01036         xmu_g = mu_g
L01037         call radar_init
L01038
L01039         if (.not. iiwarm) then
L01040
L01041   !..Rain collecting graupel & graupel collecting rain.
L01042         CALL wrf_debug(200, '  creating rain collecting graupel table')
L01043
L01044            call qr_acr_qg(dimNRHG)
L01045
L01046   !..Rain collecting snow & snow collecting rain.
L01047         CALL wrf_debug(200, '  creating rain collecting snow table')
L01048         call qr_acr_qs
L01049
L01050   !..Cloud water and rain freezing (Bigg, 1953).
L01051         CALL wrf_debug(200, '  creating freezing of water drops table')
L01052         call freezeH2O
L01053
```

```
L01054  !..Conversion of some ice mass into snow category.
L01055        CALL wrf_debug(200, '  creating ice converting to snow table')
L01056        call qi_aut_qs
L01057
L01058        endif
L01059
L01060        CALL wrf_debug(150, ' ... DONE microphysical lookup tables')
L01061
L01062        endif
L01063
L01064        END SUBROUTINE thompson_init
```

- **速率常数折叠**（L785–L818 原文）：`t1_qr_qc = π/4 • av_r • crg(9)` 等 14 个标量——把“碰撞截面 × 速度差 × 谱矩”里的常数部分先乘好。注释里被 ! 掉的 `t1_qg_qc` 说明霰的对应项因密度分档只能运行时现算（主实体里它每次按 `idx_bg` 重算，见 code_07a §07a.8.4）。
- **bin 构建**（L831–L896）：云滴线性 1 μm 步进（1–100 μm）；冰/雨/雪/霰**对数等距**（统一用 `xDx` 中间变量：端点已知 → 对数内插 → `Di = sqrt(xDx(n) • xDx(n+1))` 取几何平均、`dti` 是 bin 宽）；`t_Nc` 是云滴数浓度 bin（1–3000 cm^{-3}）。**几何平均是查表法的标准做法**——被插值量在对数坐标里线性变化时，bin 中心取几何平均误差最小。
- **建表调用链**（L1005–L1058）：`table_ccnAct`（气溶胶活化，仅 aware 模式）→ `table_Efrw/table_Efsw`（Stokes 数碰并效率）→ `table_dropEvap`（雨蒸发）→ `radar_init`（把谱参数传给雷达模块）→ `qr_acr_qg(dimNRHG)`（雨-霰五维表）→ `qr_acr_qs`（雨-雪四维表）→ `freezeH2O`（Bigg 冻结，四维表含 `idx_IN` 尘埃维）→ `qi_aut_qs`（冰 → 雪自动转化表）。`iiwarm` 为真时跳过冰相五件。各建表子程序本体都在 code_07b。

07a.7 `mp_gt_driver` 逐段精讲

07a.7.1 签名与实参契约

```
!file: phys/module_mp_thompson.F    L1070-L1197
L01070        SUBROUTINE mp_gt_driver(qv, qc, qr, qi, qs, qg, qb, ni, nr, nc, ng,&
L01071                                nwfa, nifa, nbca, nwfa2d, nifa2d, nbca2d,  &
L01072                                aer_init_opt, wif_input_opt,               &
L01073                                th, pii, p, w, dz,                         &
L01074                                dt_in, itimestep,                          &
L01075                                RAINNC, RAINNCV, &
L01076                                SNOWNC, SNOWNCV, &
L01077                                GRAUPELNC, GRAUPELNCV, SR, &
L01078  #if ( WRF_CHEM == 1 )
L01079                                wetscav_on, rainprod, evapprod, &
L01080  #endif
L01081                                refl_10cm, diagflag, ke_diag, do_radar_ref,      &
L01082                                re_cloud, re_ice, re_snow,                       &
L01083                                has_reqc, has_reqi, has_reqs,                    &
L01084                                ids,ide, jds,jde, kds,kde, &             ! domain dims
L01085                                ims,ime, jms,jme, kms,kme, &             ! memory dims
L01086                                its,ite, jts,jte, kts,kte)               ! tile dims
L01087
L01088        implicit none
L01089
L01090  !..Subroutine arguments
L01091        INTEGER, INTENT(IN):: ids,ide, jds,jde, kds,kde, &
L01092                              ims,ime, jms,jme, kms,kme, &
L01093                              its,ite, jts,jte, kts,kte
L01094        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), INTENT(INOUT):: &
L01095                              qv, qc, qr, qi, qs, qg, ni, nr, th
L01096        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), OPTIONAL, INTENT(INOUT):: &
L01097                              nc, nwfa, nifa, nbca, qb, ng
L01098        REAL, DIMENSION(ims:ime, jms:jme), OPTIONAL, INTENT(IN):: nwfa2d, nifa2d, &
L01099                                                                  nbca2d
L01100        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), INTENT(INOUT):: &
L01101                              re_cloud, re_ice, re_snow
L01102        INTEGER, INTENT(IN):: has_reqc, has_reqi, has_reqs
L01103  #if ( WRF_CHEM == 1 )
L01104        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), INTENT(INOUT):: &
L01105                              rainprod, evapprod
L01106  #endif
L01107        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), INTENT(IN):: &
L01108                              pii, p, w, dz
L01109        REAL, DIMENSION(ims:ime, jms:jme), INTENT(INOUT):: &
L01110                              RAINNC, RAINNCV, SR
L01111        REAL, DIMENSION(ims:ime, jms:jme), OPTIONAL, INTENT(INOUT)::      &
L01112                              SNOWNC, SNOWNCV, GRAUPELNC, GRAUPELNCV
L01113        REAL, DIMENSION(ims:ime, kms:kme, jms:jme), INTENT(INOUT)::       &
L01114                              refl_10cm
L01115        REAL, INTENT(IN):: dt_in
L01116        INTEGER, INTENT(IN):: itimestep
L01117        INTEGER, OPTIONAL, INTENT(IN):: aer_init_opt, wif_input_opt
L01118  #if ( WRF_CHEM == 1 )
L01119        LOGICAL, INTENT(in) :: wetscav_on
L01120  #endif
L01121
L01122  !..Local variables
L01123        REAL, DIMENSION(kts:kte):: &
L01124                              qv1d, qc1d, qi1d, qr1d, qs1d, qg1d, qb1d,     &
L01125                              ni1d, nr1d, nc1d, ng1d, nwfa1d, nifa1d, nbca1d,&
L01126                              t1d, p1d, w1d, dz1d, rho, dBZ
L01127        REAL, DIMENSION(kts:kte):: re_qc1d, re_qi1d, re_qs1d
L01128  #if ( WRF_CHEM == 1 )
L01129        REAL, DIMENSION(kts:kte):: &
L01130                              rainprod1d, evapprod1d
```

```
L01131  #endif
L01132        REAL, DIMENSION(its:ite, jts:jte):: pcp_ra, pcp_sn, pcp_gr, pcp_ic
L01133        REAL:: dt, pptrain, pptsnow, pptgraul, pptice
L01134        REAL:: qc_max, qr_max, qs_max, qi_max, qg_max, ni_max, nr_max
L01135        REAL:: nwfa1
L01136        REAL:: ygra1, zans1
L01137        DOUBLE PRECISION:: lamg, lam_exp, lamr, N0_min, N0_exp
L01138        INTEGER:: i, j, k, k_0, k_inj
L01139        INTEGER:: imax_qc,imax_qr,imax_qi,imax_qs,imax_qg,imax_ni,imax_nr
L01140        INTEGER:: jmax_qc,jmax_qr,jmax_qi,jmax_qs,jmax_qg,jmax_ni,jmax_nr
L01141        INTEGER:: kmax_qc,kmax_qr,kmax_qi,kmax_qs,kmax_qg,kmax_ni,kmax_nr
L01142        INTEGER:: i_start, j_start, i_end, j_end
L01143        LOGICAL, OPTIONAL, INTENT(IN) :: diagflag
L01144        INTEGER, OPTIONAL, INTENT(IN) :: do_radar_ref, ke_diag
L01145        CHARACTER*256:: mp_debug
L01146
L01147        integer :: kediagloc
L01148
L01149  !+---+
L01150
L01151        i_start = its
L01152        j_start = jts
L01153        i_end   = MIN(ite, ide-1)
L01154        j_end   = MIN(jte, jde-1)
L01155
L01156  !..For idealized testing by developer.
L01157  !     if ( (ide-ids+1).gt.4 .and. (jde-jds+1).lt.4 .and.                &
L01158  !          ids.eq.its.and.ide.eq.ite.and.jds.eq.jts.and.jde.eq.jte) then
L01159  !        i_start = its + 2
L01160  !        i_end   = ite
L01161  !        j_start = jts
L01162  !        j_end   = jte
L01163  !     endif
L01164
L01165        dt = dt_in
L01166
L01167        qc_max = 0.
L01168        qr_max = 0.
L01169        qs_max = 0.
L01170        qi_max = 0.
L01171        qg_max = 0
L01172        ni_max = 0.
L01173        nr_max = 0.
L01174        imax_qc = 0
L01175        imax_qr = 0
L01176        imax_qi = 0
L01177        imax_qs = 0
L01178        imax_qg = 0
L01179        imax_ni = 0
L01180        imax_nr = 0
L01181        jmax_qc = 0
L01182        jmax_qr = 0
L01183        jmax_qi = 0
L01184        jmax_qs = 0
L01185        jmax_qg = 0
L01186        jmax_ni = 0
L01187        jmax_nr = 0
L01188        kmax_qc = 0
L01189        kmax_qr = 0
L01190        kmax_qi = 0
L01191        kmax_qs = 0
L01192        kmax_qg = 0
L01193        kmax_ni = 0
L01194        kmax_nr = 0
L01195        do i = 1, 256
L01196           mp_debug(i:i) = char(0)
L01197        enddo
```

- **实参契约**（对照教材 ch13 §13.6.1 的调用链图）：`qv/qc/qr/qi/qs/qg/ni/nr` 八个 INOUT 水凝物场 + `th/pii/p/w/dz` 五个 IN 动力场 + 降水诊断（`RAINNC/RAINNCV/SNOWNC/…/SR`）+ 诊断接口（`refl_10cm`、`re_cloud/re_ice/re_snow` 与 `has_reqc/has_reqi/has_reqs` 标志）。**OPTIONAL 字段（nc/nwfa/nifa/nbca/qb/ng）在不在场 = 方案是 8 号还是 28 号**——与 init 同一把钥匙。
- **局部变量骨架**：14 个 `*1d` 单列数组 + `rho`（现场算的干空气密度近似 `ρ = 0.622 • p/(R • T • (qv+0.622))`——即 ρ = p/(R_d • T_v) 的展开式，书 (2.2.10) 对应的源码形态）。
- `qc_max…nr_max` 与 `imax_qc…kmax_nr` 三组变量是“全场极值追踪器”：tile 循环结束后输出一条 debug 日志（L1485 起的 `MP-GT:` 行），定位最大含水量格点——排数值爆表时的第一入口。

07a.7.2 提列与霰参数反推

```
!file: phys/module_mp_thompson.F    L1199-L1283
L01199        if (.NOT. is_aerosol_aware .AND. PRESENT(nc) .AND. PRESENT(nwfa)  &
L01200                  .AND. PRESENT(nifa) .AND. PRESENT(nwfa2d)) then
L01201           write(mp_debug,*) 'WARNING, nc-nwfa-nifa-nwfa2d present but is_aerosol_aware is FALSE'
L01202           CALL wrf_debug(0, mp_debug)
L01203        endif
L01204
L01205        j_loop:  do j = j_start, j_end
L01206        i_loop:  do i = i_start, i_end
L01207
L01208           pptrain = 0.
L01209           pptsnow = 0.
L01210           pptgraul = 0.
L01211           pptice = 0.
L01212           RAINNCV(i,j) = 0.
L01213           IF ( PRESENT (snowncv) ) THEN
L01214              SNOWNCV(i,j) = 0.
L01215           ENDIF
L01216           IF ( PRESENT (graupelncv) ) THEN
```

```
L01217            GRAUPELNCV(i,j) = 0.
L01218         ENDIF
L01219         SR(i,j) = 0.
L01220
L01221         do k = kts, kte
L01222            t1d(k) = th(i,k,j)*pii(i,k,j)
L01223            p1d(k) = p(i,k,j)
L01224            w1d(k) = w(i,k,j)
L01225            dz1d(k) = dz(i,k,j)
L01226            qv1d(k) = qv(i,k,j)
L01227            qc1d(k) = qc(i,k,j)
L01228            qi1d(k) = qi(i,k,j)
L01229            qr1d(k) = qr(i,k,j)
L01230            qs1d(k) = qs(i,k,j)
L01231            qg1d(k) = qg(i,k,j)
L01232            ni1d(k) = ni(i,k,j)
L01233            nr1d(k) = nr(i,k,j)
L01234            rho(k) = 0.622*p1d(k)/(R*t1d(k)*(qv1d(k)+0.622))
L01235         enddo
L01236         if (is_aerosol_aware) then
L01237            do k = kts, kte
L01238               nc1d(k) = nc(i,k,j)
L01239               nwfa1d(k) = nwfa(i,k,j)
L01240               nifa1d(k) = nifa(i,k,j)
L01241               if (wif_input_opt .eq. 2) then
L01242                  nbca1d(k) = nbca(i,k,j)
L01243               else
L01244                  nbca1d(k) = 0.0
L01245               endif
L01246            enddo
L01247            nwfa1 = nwfa2d(i,j)
L01248         else
L01249            do k = kts, kte
L01250               nc1d(k) = Nt_c/rho(k)
L01251               nwfa1d(k) = 11.1E6/rho(k)
L01252               nifa1d(k) = naIN1*0.01/rho(k)
L01253               nbca1d(k) = 5.55E6/rho(k)
L01254            enddo
L01255         endif
L01256
L01257  !..If not the variable-density graupel-hail hybrid, then set the vol mixing
L01258  !.. ratio to mass mixing ratio divided by constant density (500kg/m3) value.
L01259
L01260         if (is_hail_aware) then
L01261            do k = kts, kte
L01262               ng1d(k) = ng(i,k,j)
L01263               qb1d(k) = qb(i,k,j)
L01264            enddo
L01265         else
L01266
L01267            do k = kte, kts, -1
L01268               if (qg1d(k).gt.R1) then
L01269                  ygra1 = alog10(max(1.E-9, qg1d(k)*rho(k)))
L01270                  zans1 = 3.0 + 2./7.*(ygra1+8.)
L01271                  zans1 = MAX(2., MIN(zans1, 6.))
L01272                  N0_exp = 10.**(zans1)
L01273                  lam_exp = (N0_exp*am_g(idx_bg1)*cgg(1,1)/(rho(k)*qg1d(k)))**oge1
L01274                  lamg = lam_exp * (cgg(3,1)*ogg2*ogg1)**obmg
L01275                  ng1d(k) = cgg(2,1)*ogg3*rho(k)*qg1d(k)*lamg**bm_g / am_g(idx_bg1)
L01276                  ng1d(k) = MAX(R2, ng1d(k)/rho(k))
L01277                  qb1d(k) = qg1d(k)/rho_g(idx_bg1)
L01278               else
L01279                  ng1d(k) = 0
L01280                  qb1d(k) = 0
L01281               endif
L01282            enddo
L01283         endif
```

- **提列**（L1221–L1235）：`t1d(k) = th(i,k,j)*pii(i,k,j)`——位温 ×Exner 函数 =真实温度（书 (6.2.8) 与 Exner 函数定义见教材 ch10）。**WRF 模式层变量进出物理方案的标准换算**：动力内核存位温，物理过程用温度，`pii` 就是那座桥。
- **缺省气溶胶**（非 aware 分支，L1249–L1254）：`nc1d = Nt_c/rho`——把数浓度 m^{-3} 换成数浓度混合比（个数/kg）——**WRF 的 moist/scalar 场一律存 “每千克干空气”**，这是初学者最常见的单位坑（教材 ch11 §11.4 讲过 moisture 字段口径）。
- **霰参数反推**（L1260–L1283，非 hail-aware 分支）：只有 `qg` 预报量时，霰数浓度从经验关系反推——`zans1 = 3 + 2/7*(log10(qg*ρ)+8)` 得指数谱等效截距 `N_0_exp`（clamp 在 `gonv_min~gonv_max` 即 1e2–1e6 m^{-4}，正是模块常数段那对值）；再由 `lam_exp → lamg = lam_exp*(cgg(3,1)*ogg2*ogg1)^obmg` 换算 gamma 谱斜率；`ng = cgg(2,1)*ogg3*ρ*qg*λ^bm_g / am_g` 闭环。这 20 行是 “单矩 → 双矩” 的技术补偿：物理信息不够，经验关系来凑。
- `qb1d = qg1d/rho_g(idx_bg1)`：霰的**体积**混合比 = 质量混合比/密度——三矩方案的第三矩的退化形态。

07a.7.3 主实体调用与降水回写

```
!file: phys/module_mp_thompson.F    L1285-L1347
L01285         call mp_thompson(aer_init_opt, wif_input_opt, &
L01286                          qv1d, qc1d, qi1d, qr1d, qs1d, qg1d, qb1d, ni1d,     &
L01287                          nr1d, nc1d, ng1d, nwfa1d, nifa1d, nbca1d, t1d, p1d, w1d, dz1d,  &
L01288                       pptrain, pptsnow, pptgraul, pptice, &
L01289  #if ( WRF_CHEM == 1 )
L01290                       wetscav_on, rainprod1d, evapprod1d, &
L01291  #endif
```

```
L01292                       kts, kte, dt, i, j)
L01293
L01294           pcp_ra(i,j) = pptrain
L01295           pcp_sn(i,j) = pptsnow
L01296           pcp_gr(i,j) = pptgraul
L01297           pcp_ic(i,j) = pptice
L01298           RAINNCV(i,j) = pptrain + pptsnow + pptgraul + pptice
L01299           RAINNC(i,j) = RAINNC(i,j) + pptrain + pptsnow + pptgraul + pptice
L01300           IF ( PRESENT(snowncv) .AND. PRESENT(snownc) ) THEN
L01301              SNOWNCV(i,j) = pptsnow + pptice
L01302              SNOWNC(i,j) = SNOWNC(i,j) + pptsnow + pptice
L01303           ENDIF
L01304           IF ( PRESENT(graupelncv) .AND. PRESENT(graupelnc) ) THEN
L01305              GRAUPELNCV(i,j) = pptgraul
L01306              GRAUPELNC(i,j) = GRAUPELNC(i,j) + pptgraul
L01307           ENDIF
L01308           SR(i,j) = (pptsnow + pptgraul + pptice)/(RAINNCV(i,j)+1.e-12)
L01309
L01310
L01311   !..
L01312   !..BEGIN AEROSOL EMISSIONS
L01313   !..
L01314   !..Reset lowest model level to initial state aerosols (fake sfc source).
L01315   !.. Changed 13 May 2013 to fake emissions in which nwfa2d is aerosol
L01316   !.. number tendency (number per kg per second).
L01317            if (is_aerosol_aware) then
L01318   !..Add anthropogenic emissions
L01319   !-GT           nwfa1d(kts) = nwfa1
L01320              nwfa1d(kts) = nwfa1d(kts) + nwfa2d(i,j)*dt_in
L01321              nifa1d(kts) = nifa1d(kts) + nifa2d(i,j)*dt_in
L01322              if (wif_input_opt .eq. 2) then
L01323                 nbca1d(kts) = nbca1d(kts) + nbca2d(i,j)*dt_in
L01324              else
L01325                 nbca1d(kts) = 0.0
L01326              endif
L01327   !..
L01328   !..END AEROSOL EMISSIONS
L01329   !..
L01330
L01331              do k = kts, kte
L01332                 nc(i,k,j) = nc1d(k)
L01333                 nwfa(i,k,j) = nwfa1d(k)
L01334                 nifa(i,k,j) = nifa1d(k)
L01335                 if (wif_input_opt .eq. 2) then
L01336                    nbca(i,k,j) = nbca1d(k)
L01337                 else
L01338                    nbca(i,k,j) = 0.0
L01339                 endif
L01340              enddo
L01341           endif
L01342           if (is_hail_aware) then
L01343              do k = kts, kte
L01344                 ng(i,k,j) = ng1d(k)
L01345                 qb(i,k,j) = qb1d(k)
L01346              enddo
L01347           endif
```

- **单列调用**（L1285–L1292）：把 1d 数组交给 `mp_thompson`，取回四个地面降水率累计 `pptrain/pptsnow/pptgraul/pptice`。
- **降水记账**（L1294–L1308）：`RAINNCV`（本步网格尺度降水）= 四项之和；`RAINNC`（累计）+= 同值；雪 = 雪相 +冰相（**冰晶沉降也算雪**）；`SR`（雪比）= 固相/(总 +1e-12)——分母防零。注意 `RAINNC` 与 `RAINSH`（积云贡献）的分工在 `microphysics_driver` 层面就分好了，此处只管网格尺度显式降水。
- **气溶胶伪排放**（L1317–L1326）：最底层 `nwfa1d(kts) += nwfa2d(i,j)*dt_in`——2D 通量场按 dt 加回；这是 2013-05-13 改造（原注释）把 “边界条件供气溶胶” 改成 “通量供气溶胶” 的痕迹。黑碳同法。

07a.7.4 回写、负值治理与诊断接口

```
!file: phys/module_mp_thompson.F    L1349-L1500
L01349           do k = kts, kte
L01350              qv(i,k,j) = qv1d(k)
L01351              qc(i,k,j) = qc1d(k)
L01352              qi(i,k,j) = qi1d(k)
L01353              qr(i,k,j) = qr1d(k)
L01354              qs(i,k,j) = qs1d(k)
L01355              qg(i,k,j) = qg1d(k)
L01356              ni(i,k,j) = ni1d(k)
L01357              nr(i,k,j) = nr1d(k)
L01358              th(i,k,j) = t1d(k)/pii(i,k,j)
L01359   #if ( WRF_CHEM == 1 )
L01360            IF ( wetscav_on ) THEN
L01361              rainprod(i,k,j) = rainprod1d(k)
L01362              evapprod(i,k,j) = evapprod1d(k)
L01363            ENDIF
L01364   #endif
L01365              if (qc1d(k) .gt. qc_max) then
L01366               imax_qc = i
L01367               jmax_qc = j
L01368               kmax_qc = k
L01369               qc_max = qc1d(k)
L01370              elseif (qc1d(k) .lt. 0.0) then
L01371               write(mp_debug,*) 'WARNING, negative qc ', qc1d(k),        &
L01372                                 ' at i,j,k=', i,j,k
L01373               CALL wrf_debug(150, mp_debug)
L01374              endif
```

```
L01375              if (qr1d(k) .gt. qr_max) then
L01376               imax_qr = i
L01377               jmax_qr = j
L01378               kmax_qr = k
L01379               qr_max = qr1d(k)
L01380              elseif (qr1d(k) .lt. 0.0) then
L01381               write(mp_debug,*) 'WARNING, negative qr ', qr1d(k),        &
L01382                         ' at i,j,k=', i,j,k
L01383               CALL wrf_debug(150, mp_debug)
L01384              endif
L01385              if (nr1d(k) .gt. nr_max) then
L01386               imax_nr = i
L01387               jmax_nr = j
L01388               kmax_nr = k
L01389               nr_max = nr1d(k)
L01390              elseif (nr1d(k) .lt. 0.0) then
L01391               write(mp_debug,*) 'WARNING, negative nr ', nr1d(k),        &
L01392                         ' at i,j,k=', i,j,k
L01393               CALL wrf_debug(150, mp_debug)
L01394              endif
L01395              if (qs1d(k) .gt. qs_max) then
L01396               imax_qs = i
L01397               jmax_qs = j
L01398               kmax_qs = k
L01399               qs_max = qs1d(k)
L01400              elseif (qs1d(k) .lt. 0.0) then
L01401               write(mp_debug,*) 'WARNING, negative qs ', qs1d(k),        &
L01402                         ' at i,j,k=', i,j,k
L01403               CALL wrf_debug(150, mp_debug)
L01404              endif
L01405              if (qi1d(k) .gt. qi_max) then
L01406               imax_qi = i
L01407               jmax_qi = j
L01408               kmax_qi = k
L01409               qi_max = qi1d(k)
L01410              elseif (qi1d(k) .lt. 0.0) then
L01411               write(mp_debug,*) 'WARNING, negative qi ', qi1d(k),        &
L01412                         ' at i,j,k=', i,j,k
L01413               CALL wrf_debug(150, mp_debug)
L01414              endif
L01415              if (qg1d(k) .gt. qg_max) then
L01416               imax_qg = i
L01417               jmax_qg = j
L01418               kmax_qg = k
L01419               qg_max = qg1d(k)
L01420              elseif (qg1d(k) .lt. 0.0) then
L01421               write(mp_debug,*) 'WARNING, negative qg ', qg1d(k),        &
L01422                         ' at i,j,k=', i,j,k
L01423               CALL wrf_debug(150, mp_debug)
L01424              endif
L01425              if (ni1d(k) .gt. ni_max) then
L01426               imax_ni = i
L01427               jmax_ni = j
L01428               kmax_ni = k
L01429               ni_max = ni1d(k)
L01430              elseif (ni1d(k) .lt. 0.0) then
L01431               write(mp_debug,*) 'WARNING, negative ni ', ni1d(k),        &
L01432                         ' at i,j,k=', i,j,k
L01433               CALL wrf_debug(150, mp_debug)
L01434              endif
L01435              if (qv1d(k) .lt. 0.0) then
L01436               write(mp_debug,*) 'WARNING, negative qv ', qv1d(k),        &
L01437                         ' at i,j,k=', i,j,k
L01438               CALL wrf_debug(150, mp_debug)
L01439               if (k.lt.kte-2 .and. k.gt.kts+1) then
L01440                  write(mp_debug,*) '   below and above are: ', qv(i,k-1,j), qv(i,k+1,j)
L01441                  CALL wrf_debug(150, mp_debug)
L01442                  qv(i,k,j) = MAX(1.E-7, 0.5*(qv(i,k-1,j) + qv(i,k+1,j)))
L01443               else
L01444                  qv(i,k,j) = 1.E-7
L01445               endif
L01446              endif
L01447            enddo
L01448
L01449            IF ( PRESENT (diagflag) ) THEN
L01450            if (diagflag .and. do_radar_ref == 1) then
L01451
L01452             IF ( present(ke_diag) ) THEN
L01453               kediagloc = ke_diag
L01454             ELSE
L01455               kediagloc = kte
L01456             ENDIF
L01457
L01458             call calc_refl10cm (qv1d, qc1d, qr1d, nr1d, qs1d, qg1d,       &
L01459                       ng1d, qb1d, t1d, p1d, dBZ, kts, kte, i, j, kediagloc)
L01460             do k = kts, kte
L01461                refl_10cm(i,k,j) = MAX(-35., dBZ(k))
L01462             enddo
L01463            endif
L01464            ENDIF
L01465
L01466            IF (has_reqc.ne.0 .and. has_reqi.ne.0 .and. has_reqs.ne.0) THEN
L01467             do k = kts, kte
L01468                re_qc1d(k) = RE_QC_BG
L01469                re_qi1d(k) = RE_QI_BG
L01470                re_qs1d(k) = RE_QS_BG
L01471             enddo
L01472             call calc_effectRad (t1d, p1d, qv1d, qc1d, nc1d, qi1d, ni1d, qs1d,  &
L01473                       re_qc1d, re_qi1d, re_qs1d, kts, kte)
L01474             do k = kts, kte
L01475                re_cloud(i,k,j) = MAX(RE_QC_BG, MIN(re_qc1d(k), 50.E-6))
L01476                re_ice(i,k,j)   = MAX(RE_QI_BG, MIN(re_qi1d(k), 125.E-6))
L01477                re_snow(i,k,j)  = MAX(RE_QS_BG, MIN(re_qs1d(k), 999.E-6))
L01478             enddo
L01479            ENDIF
L01480
L01481          enddo i_loop
L01482          enddo j_loop
```

```
L01483
L01484  ! DEBUG - GT
L01485        write(mp_debug,'(a,7(a,e13.6,1x,a,i3,a,i3,a,i3,a,1x))') 'MP-GT:', &
L01486           'qc: ', qc_max, '(', imax_qc, ',', jmax_qc, ',', kmax_qc, ')', &
L01487           'qr: ', qr_max, '(', imax_qr, ',', jmax_qr, ',', kmax_qr, ')', &
L01488           'qi: ', qi_max, '(', imax_qi, ',', jmax_qi, ',', kmax_qi, ')', &
L01489           'qs: ', qs_max, '(', imax_qs, ',', jmax_qs, ',', kmax_qs, ')', &
L01490           'qg: ', qg_max, '(', imax_qg, ',', jmax_qg, ',', kmax_qg, ')', &
L01491           'ni: ', ni_max, '(', imax_ni, ',', jmax_ni, ',', kmax_ni, ')', &
L01492           'nr: ', nr_max, '(', imax_nr, ',', jmax_nr, ',', kmax_nr, ')'
L01493        CALL wrf_debug(150, mp_debug)
L01494  ! END DEBUG - GT
L01495
L01496        do i = 1, 256
L01497           mp_debug(i:i) = char(0)
L01498        enddo
L01499
L01500        END SUBROUTINE mp_gt_driver
```

- **回写与极值追踪**（L1349–L1447）：`th(i,k,j) = t1d(k)/pii(i,k,j)`——温度除回 Exner 还原成位温；随后 7 个物种逐一“更新极值 / 负值告警”。**唯独 qv 的负值被当场修复**：非边界层用邻层平均、边界层钳 1e-7——水汽是全方案中最敏感的量（负水汽会让饱和计算崩溃），其它物种只告警不修（留给守恒段处理）。
- **雷达反射率**（L1449–L1464）：`diagflag .and. do_radar_ref == 1` 才算——`calc_refl10cm`（code_07b）按列算 dBZ，钳位 $[-35, +\infty)$。
- **有效半径接口**（L1466–L1479）：`has_reqc/has_reqi/has_reqs` 三标志全非零才进——背景值取 `RE_QC_BG` 族（share 常数层），`calc_effectRad`（code_07b）算出后**钳位**：云 50 μm、冰 125 μm、雪 999 μm 上限。教材 ch13 §13.7.6 讲了这个接口如何闭合第 12 章辐射的伏笔；`calc_effectRad` 的物理判据（云滴活化阈值、冰晶增长模式）见 code_07b。

07a.8 `mp_thompson` 主实体逐段精讲

07a.8.0 签名与局部变量

```
!file: phys/module_mp_thompson.F    L1513-L1636
L01513        subroutine mp_thompson (aer_init_opt, wif_input_opt, &
L01514                                qv1d, qc1d, qi1d, qr1d, qs1d, qg1d, qb1d,       &
L01515                                ni1d, nr1d, nc1d, ng1d, nwfa1d, nifa1d, nbca1d, &
L01516                                t1d, p1d, w1d, dzq,                             &
L01517                                pptrain, pptsnow, pptgraul, pptice, &
L01518  #if ( WRF_CHEM == 1 )
L01519                                wetscav_on, rainprod, evapprod, &
L01520  #endif
L01521                                kts, kte, dt, ii, jj)
L01522
L01523        implicit none
L01524
L01525  !..Sub arguments
L01526        INTEGER, OPTIONAL, INTENT(IN):: aer_init_opt, wif_input_opt
L01527        INTEGER, INTENT(IN):: kts, kte, ii, jj
L01528        REAL, DIMENSION(kts:kte), INTENT(INOUT):: &
L01529                                qv1d, qc1d, qi1d, qr1d, qs1d, qg1d, qb1d,     &
L01530                                ni1d, nr1d, nc1d, ng1d, nwfa1d, nifa1d, nbca1d, t1d
L01531        REAL, DIMENSION(kts:kte), INTENT(IN):: p1d, w1d, dzq
L01532        REAL, INTENT(INOUT):: pptrain, pptsnow, pptgraul, pptice
L01533        REAL, INTENT(IN):: dt
L01534  #if ( WRF_CHEM == 1 )
L01535        REAL, DIMENSION(kts:kte), INTENT(INOUT):: &
L01536                                rainprod, evapprod
L01537        LOGICAL, INTENT(IN) :: wetscav_on
L01538  #endif
L01539
L01540  !..Local variables
L01541        REAL, DIMENSION(kts:kte):: tten, qvten, qcten, qiten, qrten,            &
L01542             qsten, qgten, qbten, niten, nrten, ncten, ngten, nwfaten, nifaten, &
L01543                                                                  nbcaten
L01544
L01545        DOUBLE PRECISION, DIMENSION(kts:kte):: prw_vcd
L01546
L01547        DOUBLE PRECISION, DIMENSION(kts:kte):: pnc_wcd, pnc_wau, pnc_rcw, &
L01548             pnc_scw, pnc_gcw
L01549
L01550        DOUBLE PRECISION, DIMENSION(kts:kte):: pna_rca, pna_sca, pna_gca, &
L01551             pnd_rcd, pnd_scd, pnd_gcd, pnb_rcb, pnb_scb, pnb_gcb
L01552
L01553        DOUBLE PRECISION, DIMENSION(kts:kte):: prr_wau, prr_rcw, prr_rcs, &
L01554             prr_rcg, prr_sml, prr_gml, &
L01555             prr_rci, prv_rev,          &
L01556             pnr_wau, pnr_rcs, pnr_rcg, &
L01557             pnr_rci, pnr_sml, pnr_gml, &
L01558             pnr_rev, pnr_rcr, pnr_rfz
L01559
L01560        DOUBLE PRECISION, DIMENSION(kts:kte):: pri_inu, pni_inu, pri_ihm, &
L01561             pni_ihm, pri_wfz, pni_wfz, &
L01562             pri_rfz, pni_rfz, pri_ide, &
L01563             pni_ide, pri_rci, pni_rci, &
L01564             pni_sci, pni_iau, pri_iha, pni_iha
L01565
L01566        DOUBLE PRECISION, DIMENSION(kts:kte):: prs_iau, prs_sci, prs_rcs, &
L01567             prs_scw, prs_sde, prs_ihm, &
L01568             prs_ide
L01569
L01570        DOUBLE PRECISION, DIMENSION(kts:kte):: prg_scw, prg_rfz, prg_gde, &
```

```
L01571             prg_gcw, prg_rci, prg_rcs, prg_rcg, prg_ihm,                &
L01572             png_rcs, png_rcg, png_scw, png_gde,                         &
L01573             pbg_scw, pbg_rfz, pbg_gcw, pbg_rci, pbg_rcs, pbg_rcg,       &
L01574             pbg_sml, pbg_gml
L01575
L01576        DOUBLE PRECISION, PARAMETER:: zeroD0 = 0.0d0
L01577
L01578        REAL, DIMENSION(kts:kte):: temp, twet, pres, qv
L01579        REAL, DIMENSION(kts:kte):: rc, ri, rr, rs, rg, rb
L01580        REAL, DIMENSION(kts:kte):: ni, nr, nc, ns, ng, nwfa, nifa, nbca
L01581        REAL, DIMENSION(kts:kte):: rho, rhof, rhof2
L01582        REAL, DIMENSION(kts:kte):: qvs, qvsi, delQvs
L01583        REAL, DIMENSION(kts:kte):: satw, sati, ssatw, ssati
L01584        REAL, DIMENSION(kts:kte):: diffu, visco, vsc2, &
L01585             tcond, lvap, ocp, lvt2
L01586
L01587        DOUBLE PRECISION, DIMENSION(kts:kte):: ilamr, ilamg, N0_r, N0_g
L01588        DOUBLE PRECISION:: N0_melt
L01589        REAL, DIMENSION(kts:kte):: mvd_r, mvd_c, mvd_g
L01590        REAL, DIMENSION(kts:kte):: smob, smo2, smo1, smo0, &
L01591             smoc, smod, smoe, smof, smog
L01592
L01593        REAL, DIMENSION(kts:kte):: sed_r,sed_s,sed_g,sed_i,sed_n,sed_c,sed_b
L01594
L01595        REAL:: rgvm, delta_tp, orho, lfus2
L01596        REAL, DIMENSION(5):: onstep
L01597        DOUBLE PRECISION:: N0_exp, N0_min, lam_exp, lamc, lamr, lamg
L01598        DOUBLE PRECISION:: lami, ilami, ilamc
L01599        REAL:: xDc, Dc_b, Dc_g, xDi, xDr, xDs, xDg, Ds_m, Dg_m
L01600        DOUBLE PRECISION:: Dr_star, Dc_star
L01601        REAL:: zeta1, zeta, taud, tau
L01602        REAL:: stoke_r, stoke_s, stoke_g, stoke_i
L01603        REAL:: vti, vtr, vts, vtg, vtc
L01604        REAL:: xrho_g, afall, vtg1, vtg2
L01605        REAL:: bfall = 3*b_coeff - 1
L01606        REAL, DIMENSION(kts:kte+1):: vtik, vtnik, vtrk, vtnrk, vtsk, vtgk,  &
L01607                  vtngk, vtck, vtnck
L01608        REAL, DIMENSION(kts:kte):: vts_boost
L01609        REAL:: M0, slam1, slam2
L01610        REAL:: Mrat, ils1, ils2, t1_vts, t2_vts, t3_vts, t4_vts, C_snow
L01611        REAL:: a_, b_, loga_, A1, A2, tf
L01612        REAL:: tempc, tc0, r_mvd1, r_mvd2, xkrat
L01613        REAL:: dew_t, Tlcl, The
L01614        REAL:: xnc, xri, xni, xmi, oxmi, xrc, xrr, xnr, xrg, xng, xrb
L01615        REAL:: xsat, rate_max, sump, ratio
L01616        REAL:: clap, fcd, dfcd
L01617        REAL:: otemp, rvs, rvs_p, rvs_pp, gamsc, alphsc, t1_evap, t1_subl
L01618        REAL:: r_frac, g_frac, const_Ri, rime_dens
L01619        REAL:: Ef_rw, Ef_sw, Ef_gw, Ef_rr
L01620        REAL:: Ef_ra, Ef_sa, Ef_ga
L01621        REAL:: dtsave, odts, odt, odzq, hgt_agl
L01622        REAL:: xslw1, ygra1, zans1, eva_factor
L01623        REAL:: SR, melt_f
L01624        INTEGER:: i, k, k2, n, nn, nstep, k_0, kbot, IT, iexfrq, k_melting
L01625        INTEGER, DIMENSION(5):: ksed1
L01626        INTEGER:: nir, nis, nig, nii, nic, niin
L01627        INTEGER:: idx_tc, idx_t, idx_s, idx_g1, idx_g, idx_r1, idx_r,     &
L01628                  idx_i1, idx_i, idx_c, idx, idx_d, idx_n, idx_in
L01629        INTEGER, DIMENSION(kts:kte):: idx_bg
L01630
L01631        LOGICAL:: melti, no_micro
L01632        LOGICAL, DIMENSION(kts:kte):: L_qc, L_qi, L_qr, L_qs, L_qg
L01633        LOGICAL:: debug_flag
L01634        CHARACTER*256:: mp_debug
L01635        INTEGER:: nu_c
L01636
```

- **实参极简**：14 个单列数组 + 4 个降水输出 + `kts/kte/dt/ii/jj`——没有维度三元组（纯 1d 列）、没有 namelist、没有诊断开关。**方案的全部“智能”都通过模块级开关（is_aerosol_aware/is_hail_aware）与 OPTIONAL 实参传入**。
- **局部变量的两组账本**：`p*` 源汇项（约 60 个，DOUBLE PRECISION——注意源汇用双精度而场量单精度，防止长链代数和的舍入漂移）与 `*ten` 倾向项（REAL）。另有谱参数（`lamr/lamg/ilamr/N0_r/…`，DOUBLE）、查找表索引（`idx_*` 一族）、五物种沉降通量（`sed_*`）、子步长（`nstep/onstep/ksed1`）。
- `onstep(5)` 数组只有 4 个被用（雨/冰/雪/霰），第 5 位是云水占位（云水不分子步，`onstep(5)` 恒 1 未用）——历史遗留的数组尺寸。
- `dzq`（列层厚）、`tten/qvten/...` 命名里的 `t` 是 tendency 的缩写，与温度 `temp` 区分——初读最易混淆的一对。

先读“变量命名法典”——主实体开头有一段注释定义了全部源汇项的命名规则，它是读懂 2570 行主实体的钥匙，先原样呈现：

```
!file: phys/module_mp_thompson.F    L1652-L1665
L01652  !+---+-----------------------------------------------------------------+
L01653  !.. Source/sink terms.  First 2 chars: "pr" represents source/sink of
L01654  !.. mass while "pn" represents source/sink of number.  Next char is one
L01655  !.. of "v" for water vapor, "r" for rain, "i" for cloud ice, "w" for
L01656  !.. cloud water, "s" for snow, and "g" for graupel.  Next chars
L01657  !.. represent processes: "de" for sublimation/deposition, "ev" for
L01658  !.. evaporation, "fz" for freezing, "ml" for melting, "au" for
L01659  !.. autoconversion, "nu" for ice nucleation, "hm" for Hallet/Mossop
L01660  !.. secondary ice production, and "c" for collection followed by the
L01661  !.. character for the species being collected.  ALL of these terms are
L01662  !.. positive (except for deposition/sublimation terms which can switch
L01663  !.. signs based on super/subsaturation) and are treated as negatives
L01664  !.. where necessary in the tendency equations.
```

```
L01665  !+---+-----------------------------------------------------------------+
```

命名法典解读：源汇项变量一律 p 开头 + 三段命名：

- 第 1–2 字符：pr = **质量**源汇，pn = **数浓度**源汇（另有 pna/pnd/pnb = CCN/IN/黑碳数，pbg = 霰**体积**）；
- 第 3 字符：**受方物种**——v 水汽、r 雨、i 冰晶、w 云水、s 雪、g 霰、b 霰体积、a/d/b 气溶胶族；
- 第 4 字符起：**过程**——de 凝华/升华、ev 蒸发、fz 冻结、ml 融化、au 自动转化、nu 冰核化、hm Hallett–Mossop 次生冰晶、cd 云滴凝结蒸发、iha Koop 均质冻结、c+物种碰并收集某物种；
- **全项恒正**（凝华/升华按过饱和度可正可负），需要 “损失” 时在倾向方程里**取负号累加**——因此每条倾向方程都是 “收入项正、支出项负” 的显式账本。

例如 prr_rcw = p-r-r-cw = **r**ain **r**ate collecting cloud **w**ater（雨的质量增长率，来自收集云水）；pnr_rcw = 雨的数浓度增长率（同过程）。

07a.8.1 倾向数组清零

```
!file: phys/module_mp_thompson.F    L1667-L1793
L01667         do k = kts, kte
L01668            tten(k) = 0.
L01669            qvten(k) = 0.
L01670            qcten(k) = 0.
L01671            qiten(k) = 0.
L01672            qrten(k) = 0.
L01673            qsten(k) = 0.
L01674            qgten(k) = 0.
L01675            ngten(k) = 0.
L01676            qbten(k) = 0.
L01677            niten(k) = 0.
L01678            nrten(k) = 0.
L01679            ncten(k) = 0.
L01680            nwfaten(k) = 0.
L01681            nifaten(k) = 0.
L01682            nbcaten(k) = 0.
L01683
L01684            prw_vcd(k) = 0.
L01685
L01686            pnc_wcd(k) = 0.
L01687            pnc_wau(k) = 0.
L01688            pnc_rcw(k) = 0.
L01689            pnc_scw(k) = 0.
L01690            pnc_gcw(k) = 0.
L01691
L01692            prv_rev(k) = 0.
L01693            prr_wau(k) = 0.
L01694            prr_rcw(k) = 0.
L01695            prr_rcs(k) = 0.
L01696            prr_rcg(k) = 0.
L01697            prr_sml(k) = 0.
L01698            prr_gml(k) = 0.
L01699            prr_rci(k) = 0.
L01700            pnr_wau(k) = 0.
L01701            pnr_rcs(k) = 0.
L01702            pnr_rcg(k) = 0.
L01703            pnr_rci(k) = 0.
L01704            pnr_sml(k) = 0.
L01705            pnr_gml(k) = 0.
L01706            pnr_rev(k) = 0.
L01707            pnr_rcr(k) = 0.
L01708            pnr_rfz(k) = 0.
L01709
L01710            pri_inu(k) = 0.
L01711            pni_inu(k) = 0.
L01712            pri_ihm(k) = 0.
L01713            pni_ihm(k) = 0.
L01714            pri_wfz(k) = 0.
L01715            pni_wfz(k) = 0.
L01716            pri_rfz(k) = 0.
L01717            pni_rfz(k) = 0.
L01718            pri_ide(k) = 0.
L01719            pni_ide(k) = 0.
L01720            pri_rci(k) = 0.
L01721            pni_rci(k) = 0.
L01722            pni_sci(k) = 0.
L01723            pni_iau(k) = 0.
L01724            pri_iha(k) = 0.
L01725            pni_iha(k) = 0.
L01726
L01727            prs_iau(k) = 0.
L01728            prs_sci(k) = 0.
L01729            prs_rcs(k) = 0.
L01730            prs_scw(k) = 0.
L01731            prs_sde(k) = 0.
L01732            prs_ihm(k) = 0.
L01733            prs_ide(k) = 0.
L01734
L01735            prg_scw(k) = 0.
L01736            prg_rfz(k) = 0.
L01737            prg_gde(k) = 0.
L01738            prg_gcw(k) = 0.
L01739            prg_rci(k) = 0.
L01740            prg_rcs(k) = 0.
```

```
L01741           prg_rcg(k) = 0.
L01742           prg_ihm(k) = 0.
L01743           !   new source/sink terms for 3-moment graupel
L01744           png_scw(k) = 0.
L01745           png_rcs(k) = 0.
L01746           png_rcg(k) = 0.
L01747           png_gde(k) = 0.
L01748
L01749           pbg_scw(k) = 0.
L01750           pbg_rfz(k) = 0.
L01751           pbg_gcw(k) = 0.
L01752           pbg_rci(k) = 0.
L01753           pbg_rcs(k) = 0.
L01754           pbg_rcg(k) = 0.
L01755           pbg_sml(k) = 0.
L01756           pbg_gml(k) = 0.
L01757
L01758           pna_rca(k) = 0.
L01759           pna_sca(k) = 0.
L01760           pna_gca(k) = 0.
L01761
L01762           pnd_rcd(k) = 0.
L01763           pnd_scd(k) = 0.
L01764           pnd_gcd(k) = 0.
L01765
L01766           pnb_rcb(k) = 0.
L01767           pnb_scb(k) = 0.
L01768           pnb_gcb(k) = 0.
L01769         enddo
L01770   #if ( WRF_CHEM == 1 )
L01771       if ( wetscav_on ) then
L01772         do k = kts, kte
L01773           rainprod(k) = 0.
L01774           evapprod(k) = 0.
L01775         enddo
L01776       endif
L01777   #endif
L01778
L01779   !..Bug fix (2016Jun15), prevent use of uninitialized value(s) of snow moments.
L01780         do k = kts, kte
L01781           smo0(k) = 0.
L01782           smo1(k) = 0.
L01783           smo2(k) = 0.
L01784           smob(k) = 0.
L01785           smoc(k) = 0.
L01786           smod(k) = 0.
L01787           smoe(k) = 0.
L01788           smof(k) = 0.
L01789           smog(k) = 0.
L01790           ns(k)   = 0.
L01791           mvd_r(k) = 0.
L01792           mvd_c(k) = 0.
L01793         enddo
```

- **清零循环**：约 60 个源汇项 + 16 个倾向项全部清零——**这个方案没有任何“隐式记忆”**：每步每列从零开始算账。WRF-CHEM 挂钩的 `rainprod/evapprod` 同样清零（湿清除化学用）。
- **2016-06-15 的 bug fix**（L1779 原注释）：雪矩量数组 `smo0…smog` 与 `ns` 未初始化就被使用过——现在显式清零。这段小历史说明：**矩量数组跨 k 循环复用，任何提前 `CYCLE` 的路径都会留下未定义值**。

07a.8.2 列装入与谱参数化：从 (q, N) 到 (N_0, λ)

```
!file: phys/module_mp_thompson.F    L1795-L1951
L01795   !+---+-----------------------------------------------------------------+
L01796   !..Put column of data into local arrays.
L01797   !+---+-----------------------------------------------------------------+
L01798         do k = kts, kte
L01799            temp(k) = t1d(k)
L01800            qv(k) = MAX(1.E-10, qv1d(k))
L01801            pres(k) = p1d(k)
L01802            rho(k) = 0.622*pres(k)/(R*temp(k)*(qv(k)+0.622))
L01803            if (is_aerosol_aware) then
L01804               if (aer_init_opt .lt. 2) then  ! Constant or climatology (e.g., GOCART)
L01805                  nwfa(k) = MAX(11.1E6, MIN(9999.E6, nwfa1d(k)*rho(k)))
L01806                  nifa(k) = MAX(naIN1*0.01, MIN(9999.E6, nifa1d(k)*rho(k)))
L01807                  if (wif_input_opt .eq. 2) then  ! Considering BC aerosol
L01808                     nbca(k) = MAX(5.55E6, MIN(9999.E6, nbca1d(k)*rho(k)))
L01809                  else
L01810                     nbca(k) = 0.0
L01811                  endif
L01812               else   ! First guess (e.g., GEOS-5)
L01813                  nwfa(k) = MAX(0.0, nwfa1d(k)*rho(k))
L01814                  nifa(k) = MAX(0.0, nifa1d(k)*rho(k))
L01815                  if (wif_input_opt .eq. 2) then  ! Considering BC aerosol
L01816                     nbca(k) = MAX(0.0, nbca1d(k)*rho(k))
L01817                  else
L01818                     nbca(k) = 0.0
L01819                  endif
L01820               endif
L01821            else
L01822               nwfa(k) = MAX(11.1E6, MIN(9999.E6, nwfa1d(k)*rho(k)))
L01823               nifa(k) = MAX(naIN1*0.01, MIN(9999.E6, nifa1d(k)*rho(k)))
L01824               nbca(k) = MAX(5.55E6, MIN(9999.E6, nbca1d(k)*rho(k)))
L01825            endif
L01826
L01827            if (qc1d(k) .gt. R1) then
L01828               no_micro = .false.
L01829               rc(k) = qc1d(k)*rho(k)
L01830               nc(k) = MAX(2., MIN(nc1d(k)*rho(k), Nt_c_max))
```

```
L01831              L_qc(k) = .true.
L01832              nu_c = MIN(15, NINT(1000.E6/nc(k)) + 2)
L01833              lamc = (nc(k)*am_r*ccg(2,nu_c)*ocg1(nu_c)/rc(k))**obmr
L01834              xDc = (bm_r + nu_c + 1.) / lamc
L01835              if (xDc.lt. D0c) then
L01836               lamc = cce(2,nu_c)/D0c
L01837              elseif (xDc.gt. D0r*2.) then
L01838               lamc = cce(2,nu_c)/(D0r*2.)
L01839              endif
L01840              nc(k) = MIN( DBLE(Nt_c_max), ccg(1,nu_c)*ocg2(nu_c)*rc(k)   &
L01841                   / am_r*lamc**bm_r)
L01842              if (.NOT. is_aerosol_aware) nc(k) = Nt_c
L01843           else
L01844              qc1d(k) = 0.0
L01845              nc1d(k) = 0.0
L01846              rc(k) = R1
L01847              nc(k) = 2.
L01848              L_qc(k) = .false.
L01849           endif
L01850
L01851           if (qi1d(k) .gt. R1) then
L01852              no_micro = .false.
L01853              ri(k) = qi1d(k)*rho(k)
L01854              ni(k) = MAX(R2, ni1d(k)*rho(k))
L01855              if (ni(k).le. R2) then
L01856                 lami = cie(2)/5.E-6
L01857                 ni(k) = MIN(999.D3, cig(1)*oig2*ri(k)/am_i*lami**bm_i)
L01858              endif
L01859              L_qi(k) = .true.
L01860              lami = (am_i*cig(2)*oig1*ni(k)/ri(k))**obmi
L01861              ilami = 1./lami
L01862              xDi = (bm_i + mu_i + 1.) * ilami
L01863              if (xDi.lt. 5.E-6) then
L01864               lami = cie(2)/5.E-6
L01865               ni(k) = MIN(999.D3, cig(1)*oig2*ri(k)/am_i*lami**bm_i)
L01866              elseif (xDi.gt. 300.E-6) then
L01867               lami = cie(2)/300.E-6
L01868               ni(k) = cig(1)*oig2*ri(k)/am_i*lami**bm_i
L01869              endif
L01870           else
L01871              qi1d(k) = 0.0
L01872              ni1d(k) = 0.0
L01873              ri(k) = R1
L01874              ni(k) = R2
L01875              L_qi(k) = .false.
L01876           endif
L01877
L01878           if (qr1d(k) .gt. R1) then
L01879              no_micro = .false.
L01880              rr(k) = qr1d(k)*rho(k)
L01881              nr(k) = MAX(R2, nr1d(k)*rho(k))
L01882              if (nr(k).le. R2) then
L01883                 mvd_r(k) = 1.0E-3
L01884                 lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L01885                 nr(k) = crg(2)*org3*rr(k)*lamr**bm_r / am_r
L01886              endif
L01887              L_qr(k) = .true.
L01888              lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L01889              mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
L01890              if (mvd_r(k) .gt. 2.5E-3) then
L01891                 mvd_r(k) = 2.5E-3
L01892                 lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L01893                 nr(k) = crg(2)*org3*rr(k)*lamr**bm_r / am_r
L01894              elseif (mvd_r(k) .lt. D0r*0.75) then
L01895                 mvd_r(k) = D0r*0.75
L01896                 lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L01897                 nr(k) = crg(2)*org3*rr(k)*lamr**bm_r / am_r
L01898              endif
L01899           else
L01900              qr1d(k) = 0.0
L01901              nr1d(k) = 0.0
L01902              rr(k) = R1
L01903              nr(k) = R2
L01904              L_qr(k) = .false.
L01905           endif
L01906           if (qs1d(k) .gt. R1) then
L01907              no_micro = .false.
L01908              rs(k) = qs1d(k)*rho(k)
L01909              L_qs(k) = .true.
L01910           else
L01911              qs1d(k) = 0.0
L01912              rs(k) = R1
L01913              L_qs(k) = .false.
L01914           endif
L01915           if (qg1d(k) .gt. R1) then
L01916              no_micro = .false.
L01917              L_qg(k) = .true.
L01918              rg(k) = qg1d(k)*rho(k)
L01919              ng(k) = MAX(R2, ng1d(k)*rho(k))
L01920              rb(k) = MAX(qg1d(k)/rho_g(NRHG), qb1d(k))
L01921              rb(k) = MIN(qg1d(k)/rho_g(1), rb(k))
L01922              qb1d(k) = rb(k)
L01923              idx_bg(k) = MAX(1,MIN(NINT(qg1d(k)/rb(k) *0.01)+1,NRHG))
L01924              if (ng(k).le. R2) then
L01925                 mvd_g(k) = 1.5E-3
L01926                 lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L01927                 ng(k) = cgg(2,1)*ogg3*rg(k)*lamg**bm_g / am_g(idx_bg(k))
L01928              endif
L01929              lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*ng(k)/rg(k))**obmg
L01930              mvd_g(k) = (3.0 + mu_g + 0.672) / lamg
L01931              if (mvd_g(k) .gt. 25.4E-3) then
L01932                 mvd_g(k) = 25.4E-3
L01933                 lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L01934                 ng(k) = cgg(2,1)*ogg3*rg(k)*lamg**bm_g / am_g(idx_bg(k))
L01935              elseif (mvd_g(k) .lt. D0r) then
L01936                 mvd_g(k) = D0r
```

```
L01937                lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L01938                ng(k) = cgg(2,1)*ogg3*rg(k)*lamg**bm_g / am_g(idx_bg(k))
L01939             endif
L01940          else
L01941             qg1d(k) = 0.0
L01942             ng1d(k) = 0.0
L01943             qb1d(k) = 0.0
L01944             idx_bg(k) = idx_bg1
L01945             rg(k) = R1
L01946             ng(k) = R2
L01947             rb(k) = R1/rho(k)/rho_g(NRHG)
L01948             L_qg(k) = .false.
L01949          endif
L01950          if (.not. is_hail_aware) idx_bg(k) = idx_bg1
L01951       enddo
```

- **列装入**（L1798 起）：`temp = t1d`、`qv = MAX(1e-10, qv1d)`（防零）、`rho` 现场重算；气溶胶两个入口分支（`aer_init_opt<2` 用气候态且钳下限 11.1e6；否则用 GEOS-5 逐值只钳非负）。
- **`no_micro` 判定**（结合上段）：任何一层有水凝物超 `R1` 就置假——全柱干净且冰未饱和时直接 return（L2020），**这是无云格点的零开销通道**。
- 以下才是 Thompson 最核心的“技术底座”代码——**五物种逐一执行同一套三步法**，以云水为例：

1. **反解斜率**：`lamc = (nc • am_r • ccg(2,nu_c) • ocg1(nu_c)/rc)^obmr`——由双矩 (`rc, nc`) 反解 λ。这个式子来自积分 `q = N_0 • Γ(bm+mu+1)/(am • λ^(bm+mu+1))` 与 `N = N_0 • Γ(mu+1)/λ^(mu+1)` 两式相除（Γ 值已预存在 `ccg` 里，`nu_c` 是运行时 gamma 形参数——注意云滴的 `nu_c` 每层随 `nc` 重算，与 `mu_c` 的关系同 Martin 1994）；
2. **算特征直径并钳位**：`xDc = (bm_r + nu_c + 1)/lamc`（质量加权平均径），小于 `D0c`（1 μm）或大于 `2 • D0r`（100 μm）时把 λ 钳到边界值；
3. **由钳位后的 λ 重算 N**：`nc = MIN(Nt_c_max, ccg(1,nu_c) • ocg2(nu_c) • rc/am_r • lamc^bm_r)`——**N 是诊断量不是预报量**，钳位后闭环回算，保证谱形、质量、数浓度三者自洽。

冰晶、雨两段同构，差别只在谱参数（`cie/cig`、`cre/crg`）与钳位阈值（冰 5–300 μm、雨 37 μm–2.5 mm，即中位径）。雨的钳位用 `mvd_r = (3.0+mu_r+0.672)/lamr`（中位体积径）而非平均径——碰并计算用 MVD 更稳。

雪没有数浓度预报量（单矩），直接 `L_qs` 置位就走；霰在 hail-aware 下多一个 `rb`（体积混合比）与 `idx_bg`（密度档 = `qg/rb/0.01` 四舍五入 clamp 到 1–9）——**密度档是三矩霰的“第三张面孔”**：同一个 qg，rb 小则密度高（湿霰）、rb 大则密度低（霰），查表、落速、融化全按档切换。

07a.8.3 热力学环境场

```
!file: phys/module_mp_thompson.F    L1965-L2020
L01965  !+---+-----------------------------------------------------------------+
L01966  !..Derive various thermodynamic variables frequently used.
L01967  !.. Saturation vapor pressure (mixing ratio) over liquid/ice comes from
L01968  !.. Flatau et al. 1992; enthalpy (latent heat) of vaporization from
L01969  !.. Bohren & Albrecht 1998; others from Pruppacher & Klett 1978.
L01970  !+---+-----------------------------------------------------------------+
L01971        k_melting = 0
L01972        do k = kts, kte
L01973           tempc = temp(k) - 273.15
L01974           rhof(k) = SQRT(RHO_NOT/rho(k))
L01975           rhof2(k) = SQRT(rhof(k))
L01976           qvs(k) = rslf(pres(k), temp(k))
L01977           delQvs(k) = MAX(0.0, rslf(pres(k), 273.15)-qv(k))
L01978           if (tempc .le. 0.0) then
L01979            qvsi(k) = rsif(pres(k), temp(k))
L01980           else
L01981            qvsi(k) = qvs(k)
L01982            k_melting = MAX(k, k_melting)
L01983           endif
L01984           satw(k) = qv(k)/qvs(k)
L01985           sati(k) = qv(k)/qvsi(k)
L01986           ssatw(k) = satw(k) - 1.
L01987           ssati(k) = sati(k) - 1.
L01988           if (abs(ssatw(k)).lt. eps) ssatw(k) = 0.0
L01989           if (abs(ssati(k)).lt. eps) ssati(k) = 0.0
L01990           if (no_micro .and. ssati(k).gt. 0.0) no_micro = .false.
L01991           diffu(k) = 2.11E-5*(temp(k)/273.15)**1.94 * (101325./pres(k))
L01992           if (tempc .ge. 0.0) then
L01993              visco(k) = (1.718+0.0049*tempc)*1.0E-5
L01994           else
L01995              visco(k) = (1.718+0.0049*tempc-1.2E-5*tempc*tempc)*1.0E-5
L01996           endif
L01997           ocp(k) = 1./(Cp*(1.+0.887*qv(k)))
L01998           vsc2(k) = SQRT(rho(k)/visco(k))
L01999           lvap(k) = lvap0 + (2106.0 - 4218.0)*tempc
L02000           tcond(k) = (5.69 + 0.0168*tempc)*1.0E-5 * 418.936
L02001           twet(k) = temp(k)
L02002        enddo
L02003
L02004        if (k_melting .gt. 0) then
L02005           do k = kts, k_melting
L02006              if (satw(k) .lt. 0.999) then
```

```
L02007                dew_t = MIN(temp(k)-0.001, t_dew(pres(k), qv(k)))
L02008                Tlcl = t_lcl(temp(k), dew_t)
L02009                The = theta_e(pres(k), temp(k), qv(k), Tlcl)
L02010                twet(k) = MIN(temp(k), compT_fr_The(The, pres(k)))
L02011             endif
L02012          enddo
L02013          endif
L02014
L02015  !+---+-----------------------------------------------------------------+
L02016  !..If no existing hydrometeor species and no chance to initiate ice or
L02017  !.. condense cloud water, just exit quickly!
L02018  !+---+-----------------------------------------------------------------+
L02019
L02020          if (no_micro) return
```

- **饱和与过饱和**：`qvs = rslf(pres, temp)`（液面，Flatau 1992 多项式——code_07b），`qvsi`（T<0 用冰面 `rsif`，T≥0 用液面——**0 °C 分界**）；`satw/sati` 是相对饱和比，`ssatw/ssati` 是过饱和度（|·|<eps 归零防噪声）；`delQvs` 是“降到冰点饱和还缺多少水汽”（融化吸湿用）。
- **输运系数**（逐温度参数化）：`diffu`（水汽扩散率，2.11e-5·(T/273.15)^1.94·(101325/p)——Pruppacher & Klett）、`visco`（空气粘滞，0 °C 上下两式）、`tcond`（导热率）、`lvap`（**克希霍夫线性式** `lvap0 + (2106-4218)·Tc`）、`ocp`（湿空气热容倒数 `1/(Cp(1+0.887qv))`——0.887 即书里 q·Cp_v/Cp_d 的系数化）。
- **Srivastava & Coen (1992) 修正**在凝华率里用到（主实体稍后），此处先备好 `vsc2 = sqrt(rho/visco)`（通风因子的一半）。
- **湿球温度**（L2004–L2013）：只在暖层（`k_melting` 以下）且未饱和时算——`t_dew`（露点）→ `t_lcl`（Bolton (1980) 式 15）→ `theta_e`（Bolton 式 43）→ `compT_fr_The`（由 θe 沿湿绝热线迭代回 T）。**为什么需要 twet**：霰碰并云水的效率在湿球温度 >0 °C 时被乘 0.1 压制（湿霰表面有水膜、效率骤降）、融化增强项也用 twet——这条“湿球温度线”是 Thompson 处理湿生长的关键技巧。Bolton 函数族原文在 code_07b。

07a.8.4 雪矩量与谱参数（雨/霰）

```
!file: phys/module_mp_thompson.F    L2023-L2151
L02023  !..Calculate y-intercept, slope, and useful moments for snow.
L02024  !+---+-----------------------------------------------------------------+
L02025          if (.not. iiwarm) then
L02026          do k = kts, kte
L02027             if (.not. L_qs(k)) CYCLE
L02028             tc0 = MIN(-0.1, temp(k)-273.15)
L02029             smob(k) = rs(k)*oams
L02030
L02031  !..All other moments based on reference, 2nd moment.  If bm_s.ne.2,
L02032  !.. then we must compute actual 2nd moment and use as reference.
L02033             if (bm_s.gt.(2.0-1.e-3) .and. bm_s.lt.(2.0+1.e-3)) then
L02034                smo2(k) = smob(k)
L02035             else
L02036                loga_ = sa(1) + sa(2)*tc0 + sa(3)*bm_s &
L02037                   + sa(4)*tc0*bm_s + sa(5)*tc0*tc0 &
L02038                   + sa(6)*bm_s*bm_s + sa(7)*tc0*tc0*bm_s &
L02039                   + sa(8)*tc0*bm_s*bm_s + sa(9)*tc0*tc0*tc0 &
L02040                   + sa(10)*bm_s*bm_s*bm_s
L02041                a_ = 10.0**loga_
L02042                b_ = sb(1) + sb(2)*tc0 + sb(3)*bm_s &
L02043                   + sb(4)*tc0*bm_s + sb(5)*tc0*tc0 &
L02044                   + sb(6)*bm_s*bm_s + sb(7)*tc0*tc0*bm_s &
L02045                   + sb(8)*tc0*bm_s*bm_s + sb(9)*tc0*tc0*tc0 &
L02046                   + sb(10)*bm_s*bm_s*bm_s
L02047                smo2(k) = (smob(k)/a_)**(1./b_)
L02048             endif
L02049
L02050  !..Calculate 0th moment.  Represents snow number concentration.
L02051             loga_ = sa(1) + sa(2)*tc0 + sa(5)*tc0*tc0 + sa(9)*tc0*tc0*tc0
L02052             a_ = 10.0**loga_
L02053             b_ = sb(1) + sb(2)*tc0 + sb(5)*tc0*tc0 + sb(9)*tc0*tc0*tc0
L02054             smo0(k) = a_ * smo2(k)**b_
L02055
L02056  !..Calculate 1st moment.  Useful for depositional growth and melting.
L02057             loga_ = sa(1) + sa(2)*tc0 + sa(3) &
L02058                   + sa(4)*tc0 + sa(5)*tc0*tc0 &
L02059                   + sa(6) + sa(7)*tc0*tc0 &
L02060                   + sa(8)*tc0 + sa(9)*tc0*tc0*tc0 &
L02061                   + sa(10)
L02062             a_ = 10.0**loga_
L02063             b_ = sb(1)+ sb(2)*tc0 + sb(3) + sb(4)*tc0 &
L02064                + sb(5)*tc0*tc0 + sb(6) &
L02065                + sb(7)*tc0*tc0 + sb(8)*tc0 &
L02066                + sb(9)*tc0*tc0*tc0 + sb(10)
L02067             smo1(k) = a_ * smo2(k)**b_
L02068
L02069  !..Calculate bm_s+1 (th) moment.  Useful for diameter calcs.
L02070             loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(1) &
L02071                   + sa(4)*tc0*cse(1) + sa(5)*tc0*tc0 &
L02072                   + sa(6)*cse(1)*cse(1) + sa(7)*tc0*tc0*cse(1) &
L02073                   + sa(8)*tc0*cse(1)*cse(1) + sa(9)*tc0*tc0*tc0 &
L02074                   + sa(10)*cse(1)*cse(1)*cse(1)
L02075             a_ = 10.0**loga_
L02076             b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(1) + sb(4)*tc0*cse(1) &
L02077                + sb(5)*tc0*tc0 + sb(6)*cse(1)*cse(1) &
L02078                + sb(7)*tc0*tc0*cse(1) + sb(8)*tc0*cse(1)*cse(1) &
L02079                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(1)*cse(1)*cse(1)
L02080             smoc(k) = a_ * smo2(k)**b_
```

```
L02081
L02082  !..Calculate snow number concentration (explicit integral, not smo0)
L02083           M0 = smob(k)/smoc(k)
L02084           Mrat = smob(k)*M0*M0*M0
L02085           slam1 = M0 * Lam0
L02086           slam2 = M0 * Lam1
L02087           ns(k) = Mrat*Kap0/slam1                                        &
L02088                 + Mrat*Kap1*M0**mu_s*csg(15)/slam2**cse(15)
L02089
L02090  !..Calculate bv_s+2 (th) moment.  Useful for riming.
L02091           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(13) &
L02092                 + sa(4)*tc0*cse(13) + sa(5)*tc0*tc0 &
L02093                 + sa(6)*cse(13)*cse(13) + sa(7)*tc0*tc0*cse(13) &
L02094                 + sa(8)*tc0*cse(13)*cse(13) + sa(9)*tc0*tc0*tc0 &
L02095                 + sa(10)*cse(13)*cse(13)*cse(13)
L02096           a_ = 10.0**loga_
L02097           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(13) + sb(4)*tc0*cse(13) &
L02098                + sb(5)*tc0*tc0 + sb(6)*cse(13)*cse(13) &
L02099                + sb(7)*tc0*tc0*cse(13) + sb(8)*tc0*cse(13)*cse(13) &
L02100                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(13)*cse(13)*cse(13)
L02101           smoe(k) = a_ * smo2(k)**b_
L02102
L02103  !..Calculate 1+(bv_s+1)/2 (th) moment.  Useful for depositional growth.
L02104           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(16) &
L02105                 + sa(4)*tc0*cse(16) + sa(5)*tc0*tc0 &
L02106                 + sa(6)*cse(16)*cse(16) + sa(7)*tc0*tc0*cse(16) &
L02107                 + sa(8)*tc0*cse(16)*cse(16) + sa(9)*tc0*tc0*tc0 &
L02108                 + sa(10)*cse(16)*cse(16)*cse(16)
L02109           a_ = 10.0**loga_
L02110           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(16) + sb(4)*tc0*cse(16) &
L02111                + sb(5)*tc0*tc0 + sb(6)*cse(16)*cse(16) &
L02112                + sb(7)*tc0*tc0*cse(16) + sb(8)*tc0*cse(16)*cse(16) &
L02113                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(16)*cse(16)*cse(16)
L02114           smof(k) = a_ * smo2(k)**b_
L02115
L02116  !..Calculate bm_s + bv_s+2 (th) moment.  Useful for riming into graupel.
L02117           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(17) &
L02118                 + sa(4)*tc0*cse(17) + sa(5)*tc0*tc0 &
L02119                 + sa(6)*cse(17)*cse(17) + sa(7)*tc0*tc0*cse(17) &
L02120                 + sa(8)*tc0*cse(17)*cse(17) + sa(9)*tc0*tc0*tc0 &
L02121                 + sa(10)*cse(17)*cse(17)*cse(17)
L02122           a_ = 10.0**loga_
L02123           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(17) + sb(4)*tc0*cse(17) &
L02124                + sb(5)*tc0*tc0 + sb(6)*cse(17)*cse(17) &
L02125                + sb(7)*tc0*tc0*cse(17) + sb(8)*tc0*cse(17)*cse(17) &
L02126                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(17)*cse(17)*cse(17)
L02127           smog(k) = a_ * smo2(k)**b_
L02128
L02129        enddo
L02130
L02131  !+---+-----------------------------------------------------------------+
L02132  !..Calculate y-intercept, slope values for graupel.
L02133  !+---+-----------------------------------------------------------------+
L02134
L02135        do k = kte, kts, -1
L02136           lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*ng(k)/rg(k))**obmg
L02137           ilamg(k) = 1./lamg
L02138           N0_g(k) = ng(k)*ogg2*lamg**cge(2,1)
L02139        enddo
L02140
L02141        endif
L02142
L02143  !+---+-----------------------------------------------------------------+
L02144  !..Calculate y-intercept, slope values for rain.
L02145  !+---+-----------------------------------------------------------------+
L02146        do k = kte, kts, -1
L02147           lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L02148           ilamr(k) = 1./lamr
L02149           mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
L02150           N0_r(k) = nr(k)*org2*lamr**cre(2)
L02151        enddo
```

- **雪的矩量法**（Field 2005）：单矩雪只有含水量 → `smob = rs•oams`（零阶矩的替代：`m = am•D²` 时第 2 矩 ∝ 质量流量）。以第 2 矩为参照矩，其余矩 `smo0`（数浓度）、`smo1`（凝华/融化用 1 矩）、`smoc`（直径用 bm_s+1 矩）、`smoe`（淞附用 bv_s+2 矩）、`smof`（凝华用）、`smog`（淞附成霰用）全部由 `sa/sb` 十项多项式按 `(温度 tc0, bm_s)` 换算——多项式系数就是模块常数段那两组 `sa/sb` 数组。**矩量法的意义**：不用谱形假设就能从 “质量 +温度” 推出沉降、凝华、碰并各自需要的谱矩。

- **雪数浓度显式积分**（L2083–L2088）：`ns` 不是 `smo0`（那是矩量近似），而是把 Field 双伽马谱显式积分的结果：`ns = Mrat•Kap0/slam1 + Mrat•Kap1•M0^mu_s•csg(15)/slam2^cse(15)`——双伽马两项各积一次。

- **霰/雨谱参数**：`lamg/lamr` 斜率、`N0_g/N0_r` 截距、`ilamg/ilamr` 存倒数——后面每条速率方程都要用。注意霰的 `N0_g = ng•ogg2•lamg^cge(2,1)` 是数浓度积分的逆运算（`cge(2,1)=mu_g+1`）。

07a.8.5 暖雨过程

```
!file: phys/module_mp_thompson.F    L2154-L2234
L02154  !..Compute warm-rain process terms (except evap done later).
L02155  !+---+-----------------------------------------------------------------+
L02156
L02157         do k = kts, kte
L02158
L02159  !..Rain self-collection follows Seifert, 1994 and drop break-up
L02160  !.. follows Verlinde and Cotton, 1993.                                  RAIN2M
L02161           if (L_qr(k) .and. mvd_r(k).gt. D0r) then
L02162  !-GT      Ef_rr = 1.0
L02163  !-GT      if (mvd_r(k) .gt. 1500.0E-6) then
L02164              Ef_rr = 1.0 - EXP(2300.0*(mvd_r(k)-1950.0E-6))
L02165  !-GT      endif
L02166           pnr_rcr(k) = Ef_rr * 2.0*nr(k)*rr(k)
L02167           endif
L02168
L02169           mvd_c(k) = D0c
L02170           if (L_qc(k)) then
L02171            nu_c = MIN(15, NINT(1000.E6/nc(k)) + 2)
L02172            xDc = MAX(D0c*1.E6, ((rc(k)/(am_r*nc(k)))**obmr) * 1.E6)
L02173            lamc = (nc(k)*am_r* ccg(2,nu_c) * ocg1(nu_c) / rc(k))**obmr
L02174            mvd_c(k) = (3.0+nu_c+0.672) / lamc
L02175            mvd_c(k) = MAX(D0c, MIN(mvd_c(k), D0r))
L02176           endif
L02177
L02178  !..Autoconversion follows Berry & Reinhardt (1974) with characteristic
L02179  !.. diameters correctly computed from gamma distrib of cloud droplets.
L02180           if (rc(k).gt. 0.01e-3) then
L02181            Dc_g = ((ccg(3,nu_c)*ocg2(nu_c))**obmr / lamc) * 1.E6
L02182            Dc_b = (xDc*xDc*xDc*Dc_g*Dc_g*Dc_g - xDc*xDc*xDc*xDc*xDc*xDc) &
L02183                 **(1./6.)
L02184            zeta1 = 0.5*((6.25E-6*xDc*Dc_b*Dc_b*Dc_b - 0.4) &
L02185                      + abs(6.25E-6*xDc*Dc_b*Dc_b*Dc_b - 0.4))
L02186            zeta = 0.027*rc(k)*zeta1
L02187            taud = 0.5*((0.5*Dc_b - 7.5) + abs(0.5*Dc_b - 7.5)) + R1
L02188            tau  = 3.72/(rc(k)*taud)
L02189            prr_wau(k) = zeta/tau
L02190            prr_wau(k) = MIN(DBLE(rc(k)*odts), prr_wau(k))
L02191            pnr_wau(k) = prr_wau(k) / (am_r*nu_c*10.*D0r*D0r)           ! RAIN2M
L02192            pnc_wau(k) = MIN(DBLE(nc(k)*odts), prr_wau(k)                &
L02193                       / (am_r*mvd_c(k)*mvd_c(k)*mvd_c(k)))              ! Qc2M
L02194           endif
L02195
L02196  !..Rain collecting cloud water.  In CE, assume Dc<<Dr and vtc=~0.
L02197           if (L_qr(k) .and. mvd_r(k).gt. D0r .and. mvd_c(k).gt. D0c) then
L02198            lamr = 1./ilamr(k)
L02199            idx = 1 + INT(nbr*DLOG(mvd_r(k)/Dr(1))/DLOG(Dr(nbr)/Dr(1)))
L02200            idx = MIN(idx, nbr)
L02201            Ef_rw = t_Efrw(idx, INT(mvd_c(k)*1.E6))
L02202            prr_rcw(k) = rhof(k)*t1_qr_qc*Ef_rw*rc(k)*N0_r(k) &
L02203                         *((lamr+fv_r)**(-cre(9)))
L02204            prr_rcw(k) = MIN(DBLE(rc(k)*odts), prr_rcw(k))
L02205            pnc_rcw(k) = rhof(k)*t1_qr_qc*Ef_rw*nc(k)*N0_r(k)           &
L02206                         *((lamr+fv_r)**(-cre(9)))                       ! Qc2M
L02207            pnc_rcw(k) = MIN(DBLE(nc(k)*odts), pnc_rcw(k))
L02208           endif
L02209
L02210  !..Rain collecting aerosols, wet scavenging.
L02211           if (L_qr(k) .and. mvd_r(k).gt. D0r) then
L02212            Ef_ra = Eff_aero(mvd_r(k),0.04E-6,visco(k),rho(k),temp(k),'r')
L02213            lamr = 1./ilamr(k)
L02214            pna_rca(k) = rhof(k)*t1_qr_qc*Ef_ra*nwfa(k)*N0_r(k)          &
L02215                         *((lamr+fv_r)**(-cre(9)))
L02216            pna_rca(k) = MIN(DBLE(nwfa(k)*odts), pna_rca(k))
L02217
L02218            Ef_ra = Eff_aero(mvd_r(k),0.8E-6,visco(k),rho(k),temp(k),'r')
L02219            pnd_rcd(k) = rhof(k)*t1_qr_qc*Ef_ra*nifa(k)*N0_r(k)          &
L02220                         *((lamr+fv_r)**(-cre(9)))
L02221            pnd_rcd(k) = MIN(DBLE(nifa(k)*odts), pnd_rcd(k))
L02222
L02223            if (present(wif_input_opt)) then
L02224              if (wif_input_opt .eq. 2) then
L02225                Ef_ra = Eff_aero(mvd_r(k),0.0236E-6, &
L02226                                 visco(k),rho(k),temp(k),'r')
L02227                pnb_rcb(k) = rhof(k)*t1_qr_qc*Ef_ra*nbca(k)*N0_r(k)      &
L02228                             *((lamr+fv_r)**(-cre(9)))
L02229                pnb_rcb(k) = MIN(DBLE(nbca(k)*odts), pnb_rcb(k))
L02230              endif
L02231            endif
L02232           endif
L02233
L02234         enddo
```

- **雨自碰并 + 破碎**（L2161–L2167）：`pnr_rcr` 只减不增（雨数因并合减少）——`Ef_rr = 1 − exp(2300 • (mvd_r−1950μm))`，即雨滴大于约 2 mm 后破碎概率指数上升（Verlinde & Cotton 1993 的破碎判据，Seifert 1994 自碰并）。**注意 `pnr_rcr` 是数浓度损失项**，质量守恒不受影响。
- **云滴特征径**（L2170–L2176）：`nu_c` 每层重算 + `mvd_c` 钳位 [D0c, D0r]。
- **自动转化**（Berry & Reinhardt 1974，L2180–L2194）：门槛 `rc > 0.01 g/kg`；特征直径 `Dc_b` 由 `xDc` 与 `Dc_g`（质量加权径）的六次方组合构成；`zeta = 0.027 • rc • zeta1`、`tau = 3.72/(rc • taud)`、`prr_wau = zeta/tau`——B&R 参数化的标准形态；钳位 “每步最多转走全部云水”。**数浓度账**（L2191–L2193）：雨数增量 = 质量转化率/(am_r • nu_c • 10 • D0r³)（转化出的雨滴按 10× 最小雨径计）；云水数减量按 `mvd_c` 径计——**同一份质量，两本数浓度账各按各自粒径记账**，这是双矩方案保证 N–q 自洽的通用手法。
- **雨碰并云水**（L2197–L2208）：碰并效率 `Ef_rw` 从 `t_Efrw` 表按 `(mvd_r bin, mvd_c μm)` 查——

Stokes 数的表化；速率 `prr_rcw = rhof • t1_qr_qc • Ef_rw • rc • N0_r • (λ+fv_r)^(-cre(9))`（`t1_qr_qc` 就是 init 里预折的 π/4 • av_r • Γ 值；`rhof` 是密度订正；`(λ+fv_r)` 是落速 exp 压制项的解析积分结果）。

- **气溶胶湿清除**（L2210–L2232）：雨碰并 CCN（0.04 μm 代表半径）、IN（0.8 μm）、黑碳（0.0236 μm）——`Eff_aero`（code_07b）按粒子类型查效率；注意三个半径就是三种气溶胶的 “代表性粒径”。

07a.8.6 冰相过程准备：查表索引与淞附效率

```
!file: phys/module_mp_thompson.F    L2239-L2400
L02239         if (.not. iiwarm) then
L02240         !..vts_boost is the factor applied to snow terminal
L02241         !..fallspeed due to riming of snow
L02242         do k = kts, kte
L02243            vts_boost(k) = 1.0
L02244            orho = 1./rho(k)
L02245
L02246            if (L_qs(k)) then
L02247               xDs = smoc(k) / smob(k)
L02248               rho_s = MAX(rho_g(1), MIN(0.13/xDs, rho_i-100.))
L02249            else
L02250               xDs = 0.
L02251               rho_s = 100.
L02252            endif
L02253
L02254   !..Temperature lookup table indexes.
L02255            tempc = temp(k) - 273.15
L02256            idx_tc = MAX(1, MIN(NINT(-tempc), 45) )
L02257            idx_t = INT( (tempc-2.5)/5. ) - 1
L02258            idx_t = MAX(1, -idx_t)
L02259            idx_t = MIN(idx_t, ntb_t)
L02260            IT = MAX(1, MIN(NINT(-tempc), 31) )
L02261
L02262   !..Cloud water lookup table index.
L02263            if (rc(k).gt. r_c(1)) then
L02264             nic = NINT(ALOG10(rc(k)))
L02265             do nn = nic-1, nic+1
L02266                n = nn
L02267                if ( (rc(k)/10.**nn).ge.1.0 .and. &
L02268                     (rc(k)/10.**nn).lt.10.0) goto 141
L02269             enddo
L02270    141      continue
L02271             idx_c = INT(rc(k)/10.**n) + 10*(n-nic2) - (n-nic2)
L02272             idx_c = MAX(1, MIN(idx_c, ntb_c))
L02273            else
L02274             idx_c = 1
L02275            endif
L02276
L02277   !..Cloud droplet number lookup table index.
L02278            idx_n = NINT(1.0 + FLOAT(nbc) * DLOG(nc(k)/t_Nc(1)) / nic1)
L02279            idx_n = MAX(1, MIN(idx_n, nbc))
L02280
L02281   !..Cloud ice lookup table indexes.
L02282            if (ri(k).gt. r_i(1)) then
L02283             nii = NINT(ALOG10(ri(k)))
L02284             do nn = nii-1, nii+1
L02285                n = nn
L02286                if ( (ri(k)/10.**nn).ge.1.0 .and. &
L02287                     (ri(k)/10.**nn).lt.10.0) goto 142
L02288             enddo
L02289    142      continue
L02290             idx_i = INT(ri(k)/10.**n) + 10*(n-nii2) - (n-nii2)
L02291             idx_i = MAX(1, MIN(idx_i, ntb_i))
L02292            else
L02293             idx_i = 1
L02294            endif
L02295
L02296            if (ni(k).gt. Nt_i(1)) then
L02297             nii = NINT(ALOG10(ni(k)))
L02298             do nn = nii-1, nii+1
L02299                n = nn
L02300                if ( (ni(k)/10.**nn).ge.1.0 .and. &
L02301                     (ni(k)/10.**nn).lt.10.0) goto 143
L02302             enddo
L02303    143      continue
L02304             idx_i1 = INT(ni(k)/10.**n) + 10*(n-nii3) - (n-nii3)
L02305             idx_i1 = MAX(1, MIN(idx_i1, ntb_i1))
L02306            else
L02307             idx_i1 = 1
L02308            endif
L02309
L02310   !..Rain lookup table indexes.
L02311            if (rr(k).gt. r_r(1)) then
L02312             nir = NINT(ALOG10(rr(k)))
L02313             do nn = nir-1, nir+1
L02314                n = nn
L02315                if ( (rr(k)/10.**nn).ge.1.0 .and. &
L02316                     (rr(k)/10.**nn).lt.10.0) goto 144
L02317             enddo
L02318    144      continue
L02319             idx_r = INT(rr(k)/10.**n) + 10*(n-nir2) - (n-nir2)
L02320             idx_r = MAX(1, MIN(idx_r, ntb_r))
L02321
L02322             lamr = 1./ilamr(k)
L02323             lam_exp = lamr * (crg(3)*org2*org1)**bm_r
L02324             N0_exp = org1*rr(k)/am_r * lam_exp**cre(1)
L02325             nir = NINT(DLOG10(N0_exp))
L02326             do nn = nir-1, nir+1
L02327                n = nn
L02328                if ( (N0_exp/10.**nn).ge.1.0 .and. &
```

```
L02329                   (N0_exp/10.**nn).lt.10.0) goto 145
L02330            enddo
L02331   145      continue
L02332            idx_r1 = INT(N0_exp/10.**n) + 10*(n-nir3) - (n-nir3)
L02333            idx_r1 = MAX(1, MIN(idx_r1, ntb_r1))
L02334           else
L02335            idx_r = 1
L02336            idx_r1 = ntb_r1
L02337           endif
L02338
L02339  !..Snow lookup table index.
L02340           if (rs(k).gt. r_s(1)) then
L02341            nis = NINT(ALOG10(rs(k)))
L02342            do nn = nis-1, nis+1
L02343               n = nn
L02344               if ( (rs(k)/10.**nn).ge.1.0 .and. &
L02345                    (rs(k)/10.**nn).lt.10.0) goto 146
L02346            enddo
L02347   146      continue
L02348            idx_s = INT(rs(k)/10.**n) + 10*(n-nis2) - (n-nis2)
L02349            idx_s = MAX(1, MIN(idx_s, ntb_s))
L02350           else
L02351            idx_s = 1
L02352           endif
L02353
L02354  !..Graupel lookup table index.
L02355           if (rg(k).gt. r_g(1)) then
L02356            nig = NINT(ALOG10(rg(k)))
L02357            do nn = nig-1, nig+1
L02358               n = nn
L02359               if ( (rg(k)/10.**nn).ge.1.0 .and. &
L02360                    (rg(k)/10.**nn).lt.10.0) goto 147
L02361            enddo
L02362   147      continue
L02363            idx_g = INT(rg(k)/10.**n) + 10*(n-nig2) - (n-nig2)
L02364            idx_g = MAX(1, MIN(idx_g, ntb_g))
L02365
L02366            lamg = 1./ilamg(k)
L02367            lam_exp = lamg * (cgg(3,1)*ogg2*ogg1)**bm_g
L02368            N0_exp = ogg1*rg(k)/am_g(idx_bg(k)) * lam_exp**cge(1,1)
L02369            nig = NINT(DLOG10(N0_exp))
L02370            do nn = nig-1, nig+1
L02371               n = nn
L02372               if ( (N0_exp/10.**nn).ge.1.0 .and. &
L02373                    (N0_exp/10.**nn).lt.10.0) goto 148
L02374            enddo
L02375   148      continue
L02376            idx_g1 = INT(N0_exp/10.**n) + 10*(n-nig3) - (n-nig3)
L02377            idx_g1 = MAX(1, MIN(idx_g1, ntb_g1))
L02378           else
L02379            idx_g = 1
L02380            idx_g1 = ntb_g1
L02381           endif
L02382
L02383  !..Deposition/sublimation prefactor (from Srivastava & Coen 1992).
L02384           otemp = 1./temp(k)
L02385           rvs = rho(k)*qvsi(k)
L02386           rvs_p = rvs*otemp*(lsub*otemp*oRv - 1.)
L02387           rvs_pp = rvs * ( otemp*(lsub*otemp*oRv - 1.) &
L02388                           *otemp*(lsub*otemp*oRv - 1.) &
L02389                           + (-2.*lsub*otemp*otemp*otemp*oRv) &
L02390                           + otemp*otemp)
L02391           gamsc = lsub*diffu(k)/tcond(k) * rvs_p
L02392           alphsc = 0.5*(gamsc/(1.+gamsc))*(gamsc/(1.+gamsc)) &
L02393                    * rvs_pp/rvs_p * rvs/rvs_p
L02394           alphsc = MAX(1.E-9, alphsc)
L02395           xsat = ssati(k)
L02396           if (abs(xsat).lt. 1.E-9) xsat=0.
L02397           t1_subl = 4.*PI*( 1.0 - alphsc*xsat &
L02398                 + 2.*alphsc*alphsc*xsat*xsat &
L02399                 - 5.*alphsc*alphsc*alphsc*xsat*xsat*xsat ) &
L02400                 / (1.+gamsc)
```

- **雪密度动态更新**（L2246–L2252）：`rho_s = MAX(rho_g(1), MIN(0.13/xDs, rho_i-100))`——由特征直径反推淞附密度：霰粒越大越密（0.13/xDs 是经验式），钳位 [50, 790]。
- **温度索引三兄弟**（L2255–L2260）：`idx_tc`（1–45，按 −Tc 四舍五入——Bigg 冻结表的温度维）、`idx_t`（1–9，5 °C 步进——雨雪碰并表的温度维）、`IT`（1–31，1 °C 步进——未被本段使用，留给反射率）。三个索引粗细不同，因为三张表的温度分辨率设计不同。
- **对数查表索引**（L2262–L2381）：六物种逐一执行同一套 `n = 取量级 → idx = INT(q/10^n) + 10·(n−nic2) − (n−nic2)` 折算——把 “对数等距表” 的下标算术化；雨/霰还有第二套截距索引 `idx_r1/idx_g1`（先把谱换算成指数等效 `N0_exp` 再索引）。
- **Srivastava & Coen (1992) 凝华预因子**（L2383–L2400）：`gamsc = lsub·diffu/tcond·rvs_p`（潜热反馈强度 = 扩散导热比 × 饱和梯度），`alphsc` 是其二阶泰勒系数，`t1_subl = 4π·(1 − α·s + 2α²s² − 5α³s³)/(1+gamsc)`——**纯凝华率的 Ventzke 修正**：过饱和越强、潜热反馈越强，增长越偏离经典 $4\pi \cdot C \cdot D \cdot \Delta s$ 定律。教科书公式（书 §6.3 的凝华增长）到源码要多出这一层订正，教材 ch13 §13.7.4 有对应讨论。

07a.8.7 雪/霰淞附与雨-雪/雨-霰碰并

```
!file: phys/module_mp_thompson.F    L2401-L2547
L02401
L02402  !..Snow collecting cloud water.  In CE, assume Dc<<Ds and vtc=~0.
L02403           if (L_qc(k) .and. mvd_c(k).gt. D0c) then
L02404            if (xDs .gt. D0s) then
L02405             idx = 1 + INT(nbs*DLOG(xDs/Ds(1))/DLOG(Ds(nbs)/Ds(1)))
L02406             idx = MIN(idx, nbs)
L02407             Ef_sw = t_Efsw(idx, INT(mvd_c(k)*1.E6))
L02408             prs_scw(k) = rhof(k)*t1_qs_qc*Ef_sw*rc(k)*smoe(k)
L02409             prs_scw(k) = MIN(DBLE(rc(k)*odts), prs_scw(k))
L02410             pnc_scw(k) = rhof(k)*t1_qs_qc*Ef_sw*nc(k)*smoe(k)                ! Qc2M
L02411             pnc_scw(k) = MIN(DBLE(nc(k)*odts), pnc_scw(k))
L02412            endif
L02413
L02414  !..Graupel collecting cloud water.  In CE, assume Dc<<Dg and vtc=~0.
L02415            if (rg(k).ge. r_g(1) .and. mvd_c(k).gt. D0c) then
L02416             xDg = (bm_g + mu_g + 1.) * ilamg(k)
L02417             vtg = rhof(k)*av_g(idx_bg(k))*cgg(6,idx_bg(k))*ogg3 * ilamg(k)**bv_g(idx_bg(k))
L02418             stoke_g = mvd_c(k)*mvd_c(k)*vtg*rho_w/(9.*visco(k)*xDg)
L02419  !..Rime density formula of Cober and List (1993) also used by Milbrandt and Morrison (2014).
L02420             const_Ri = -1.*(mvd_c(k)*0.5E6)*vtg/MIN(-0.1,tempc)
L02421             const_Ri = MAX(0.1, MIN(const_Ri, 10.))
L02422             rime_dens = (0.051 + 0.114*const_Ri - 0.0055*const_Ri*const_Ri)*1000.
L02423              if (stoke_g.ge.0.4 .and. stoke_g.le.10.) then
L02424               Ef_gw = 0.55*ALOG10(2.51*stoke_g)
L02425              elseif (stoke_g.lt.0.4) then
L02426               Ef_gw = 0.0
L02427              elseif (stoke_g.gt.10) then
L02428               Ef_gw = 0.77
L02429              endif
L02430  !!! Not sure what to do here - hail increases size rapidly here below melting level.
L02431              if (twet(k).gt.T_0) Ef_gw = Ef_gw*0.1
L02432              t1_qg_qc = PI*.25*av_g(idx_bg(k)) * cgg(9,idx_bg(k))
L02433              prg_gcw(k) = rhof(k)*t1_qg_qc*Ef_gw*rc(k)*N0_g(k) &
L02434                           *ilamg(k)**cge(9,idx_bg(k))
L02435              pnc_gcw(k) = rhof(k)*t1_qg_qc*Ef_gw*nc(k)*N0_g(k)           &
L02436                           *ilamg(k)**cge(9,idx_bg(k))                      ! Qc2M
L02437              pnc_gcw(k) = MIN(DBLE(nc(k)*odts), pnc_gcw(k))
L02438              if (twet(k).lt.T_0) pbg_gcw(k) = prg_gcw(k)/rime_dens
L02439            endif
L02440           endif
L02441
L02442  !..Snow and graupel collecting aerosols, wet scavenging.
L02443           if (rs(k) .gt. r_s(1)) then
L02444            Ef_sa = Eff_aero(xDs,0.04E-6,visco(k),rho(k),temp(k),'s')
L02445            pna_sca(k) = rhof(k)*t1_qs_qc*Ef_sa*nwfa(k)*smoe(k)
L02446            pna_sca(k) = MIN(DBLE(nwfa(k)*odts), pna_sca(k))
L02447
L02448            Ef_sa = Eff_aero(xDs,0.8E-6,visco(k),rho(k),temp(k),'s')
L02449            pnd_scd(k) = rhof(k)*t1_qs_qc*Ef_sa*nifa(k)*smoe(k)
L02450            pnd_scd(k) = MIN(DBLE(nifa(k)*odts), pnd_scd(k))
L02451
L02452            if (present(wif_input_opt)) then
L02453              if (wif_input_opt .eq. 2) then
L02454                Ef_sa = Eff_aero(xDs,0.0236E-6,visco(k),rho(k),temp(k),'s')
L02455                pnb_scb(k) = rhof(k)*t1_qs_qc*Ef_sa*nbca(k)*smoe(k)
L02456                pnb_scb(k) = MIN(DBLE(nbca(k)*odts), pnb_scb(k))
L02457              endif
L02458            endif
L02459           endif
L02460           if (rg(k) .gt. r_g(1)) then
L02461            xDg = (bm_g + mu_g + 1.) * ilamg(k)
L02462            Ef_ga = Eff_aero(xDg,0.04E-6,visco(k),rho(k),temp(k),'g')
L02463            t1_qg_qc = PI*.25*av_g(idx_bg(k)) * cgg(9,idx_bg(k))
L02464            pna_gca(k) = rhof(k)*t1_qg_qc*Ef_ga*nwfa(k)*N0_g(k)           &
L02465                         *ilamg(k)**cge(9,idx_bg(k))
L02466            pna_gca(k) = MIN(DBLE(nwfa(k)*odts), pna_gca(k))
L02467
L02468            Ef_ga = Eff_aero(xDg,0.8E-6,visco(k),rho(k),temp(k),'g')
L02469            pnd_gcd(k) = rhof(k)*t1_qg_qc*Ef_ga*nifa(k)*N0_g(k)           &
L02470                         *ilamg(k)**cge(9,idx_bg(k))
L02471            pnd_gcd(k) = MIN(DBLE(nifa(k)*odts), pnd_gcd(k))
L02472
L02473            if (present(wif_input_opt)) then
L02474              if (wif_input_opt .eq. 2) then
L02475                Ef_ga = Eff_aero(xDg,0.0236E-6,visco(k),rho(k),temp(k),'g')
L02476                pnb_gcb(k) = rhof(k)*t1_qg_qc*Ef_ga*nbca(k)*N0_g(k)       &
L02477                             *ilamg(k)**cge(9,idx_bg(k))
L02478                pnb_gcb(k) = MIN(DBLE(nbca(k)*odts), pnb_gcb(k))
L02479              endif
L02480            endif
L02481           endif
L02482
L02483  !..Rain collecting snow.  Cannot assume Wisner (1972) approximation
L02484  !.. or Mizuno (1990) approach so we solve the CE explicitly and store
L02485  !.. results in lookup table.
L02486           if (rr(k).ge. r_r(1)) then
L02487            if (rs(k).ge. r_s(1)) then
L02488             if (twet(k).lt.T_0) then
L02489              prr_rcs(k) = -(tmr_racs2(idx_s,idx_t,idx_r1,idx_r) &
L02490                           + tcr_sacr2(idx_s,idx_t,idx_r1,idx_r) &
L02491                           + tmr_racs1(idx_s,idx_t,idx_r1,idx_r) &
L02492                           + tcr_sacr1(idx_s,idx_t,idx_r1,idx_r))
L02493              prs_rcs(k) = tmr_racs2(idx_s,idx_t,idx_r1,idx_r) &
L02494                           + tcr_sacr2(idx_s,idx_t,idx_r1,idx_r) &
L02495                           - tcs_racs1(idx_s,idx_t,idx_r1,idx_r) &
L02496                           - tms_sacr1(idx_s,idx_t,idx_r1,idx_r)
L02497              prg_rcs(k) = tmr_racs1(idx_s,idx_t,idx_r1,idx_r) &
L02498                           + tcr_sacr1(idx_s,idx_t,idx_r1,idx_r) &
L02499                           + tcs_racs1(idx_s,idx_t,idx_r1,idx_r) &
L02500                           + tms_sacr1(idx_s,idx_t,idx_r1,idx_r)
L02501              prr_rcs(k) = MAX(DBLE(-rr(k)*odts), prr_rcs(k))
L02502              prs_rcs(k) = MAX(DBLE(-rs(k)*odts), prs_rcs(k))
L02503              prg_rcs(k) = MIN(DBLE((rr(k)+rs(k))*odts), prg_rcs(k))
```

```
L02504             pnr_rcs(k) = tnr_racs1(idx_s,idx_t,idx_r1,idx_r)            &   ! RAIN2M
L02505                        + tnr_racs2(idx_s,idx_t,idx_r1,idx_r)            &
L02506                        + tnr_sacr1(idx_s,idx_t,idx_r1,idx_r)            &
L02507                        + tnr_sacr2(idx_s,idx_t,idx_r1,idx_r)
L02508             pnr_rcs(k) = MIN(DBLE(nr(k)*odts), pnr_rcs(k))
L02509             png_rcs(k) = pnr_rcs(k)
L02510  !-GT       pbg_rcs(k) = prg_rcs(k)/(0.5*(rho_i+rho_s))
L02511             pbg_rcs(k) = prg_rcs(k)/rho_i
L02512            else
L02513             prs_rcs(k) = -tcs_racs1(idx_s,idx_t,idx_r1,idx_r)           &
L02514                        - tms_sacr1(idx_s,idx_t,idx_r1,idx_r)            &
L02515                        + tmr_racs2(idx_s,idx_t,idx_r1,idx_r)            &
L02516                        + tcr_sacr2(idx_s,idx_t,idx_r1,idx_r)
L02517             prs_rcs(k) = MAX(DBLE(-rs(k)*odts), prs_rcs(k))
L02518             prr_rcs(k) = -prs_rcs(k)
L02519            endif
L02520           endif
L02521
L02522  !..Rain collecting graupel.  Cannot assume Wisner (1972) approximation
L02523  !.. or Mizuno (1990) approach so we solve the CE explicitly and store
L02524  !.. results in lookup table.
L02525           if (rg(k).ge. r_g(1)) then
L02526            if (twet(k).lt.T_0) then
L02527             prg_rcg(k) = tmr_racg(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)  &
L02528                        + tcr_gacr(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)
L02529             prg_rcg(k) = MIN(DBLE(rr(k)*odts), prg_rcg(k))
L02530             prr_rcg(k) = -prg_rcg(k)
L02531             pnr_rcg(k) = tnr_racg(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)  &   ! RAIN2M
L02532                        + tnr_gacr(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)
L02533             pnr_rcg(k) = MIN(DBLE(nr(k)*odts), pnr_rcg(k))
L02534  !-GT       pbg_rcg(k) = prg_rcg(k)/(0.5*(rho_i+rho_g(idx_bg(k))))
L02535             pbg_rcg(k) = prg_rcg(k)/rho_i
L02536            else
L02537             prr_rcg(k) = tcg_racg(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)
L02538             prr_rcg(k) = MIN(DBLE(rg(k)*odts), prr_rcg(k))
L02539             prg_rcg(k) = -prr_rcg(k)
L02540             png_rcg(k) = tnr_racg(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)
L02541  !!!                   + tnr_gacr(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)
L02542             png_rcg(k) = MIN(DBLE(ng(k)*odts), png_rcg(k))
L02543             pbg_rcg(k) = prg_rcg(k)/rho_g(idx_bg(k))
L02544  !..Put in explicit drop break-up due to collisions.
L02545             pnr_rcg(k) = -1.5*tnr_gacr(idx_g1,idx_g,idx_bg(k),idx_r1,idx_r)  ! RAIN2M
L02546            endif
L02547           endif
```

- **雪淞附云水**（L2401–L2411）：条件 `xDs > D0s`（雪得足够大）；效率 `Ef_sw` 从 `t_Efsw` 表按雪径 bin × 云滴径查。
- **霰淞附云水**（L2413–L2438）：这是**淞附增长的核心段**——①Stokes 数 `stoke_g = mvd_c² • vtg • rho_w/(9 • visco • xDg)`（云滴惯性参数：小滴能否绕过流线撞上霰粒）；② 效率分段：`stoke<0.4` 不碰、`0.4–10` 对数段 `Ef = 0.55 • log10(2.51 • stoke)`、`>10` 饱和 0.77；③ **湿生长压制**：`twet > T_0` 时 `Ef *= 0.1`（水膜效应）；④ **淞密度**（Cober & List 1993）：`const_Ri` 由云滴径 × 霰落速/过冷温度构成，`rime_dens = (0.051 + 0.114 • Ri − 0.0055 • Ri²) • 1000`——撞得快（大滴高速）淞得密、撞得慢淞得疏，这个密度直接进 `pbg_gcw`（体积账）。
- **雨-雪 / 雨-霰碰并**（L2482–L2547）：查 `tmr_racs1/2`、`tcr_sacr1/2`…多表并组合——表名拆读：`t`=table、`c/m/n`=charge/mass/number（电荷/质量/数）、第一物种 =收集者、第二物种 =被收集者、`1/2`=两项展开。**湿球温度分叉**：`twet < T_0` 时雨雪碰并同时产生雨损失、雪收支、霰生成（部分并合物转霰）；`twet > T_0` 时走融化侧分支。雨-霰在暖层走 “霰化雨” 反分支（`pnr_rcg` 负值 = 破碎减数）。**这一段是 Thompson 区别于 WSM6 的标志性工程**：Wisner 近似在雨-雪区间误差大，故用五维显式表。

07a.8.8 冰核化三来源

```
!file: phys/module_mp_thompson.F    L2550-L2641
L02550  !+---+-----------------------------------------------------------------+
L02551  !..Next IF block handles only those processes below 0C.
L02552  !+---+-----------------------------------------------------------------+
L02553
L02554           if (temp(k).lt.T_0) then
L02555
L02556            vts_boost(k) = 1.0
L02557            rate_max = (qv(k)-qvsi(k))*rho(k)*odts*0.999
L02558
L02559  !+---+---------------- BEGIN NEW ICE NUCLEATION ----------------------+
L02560  !..Freezing of supercooled water (rain or cloud) is influenced by dust
L02561  !.. but still using Bigg 1953 with a temperature adjustment of a few
L02562  !.. degrees depending on dust concentration.  A default value by way
L02563  !.. of idx_IN is 1.0 per liter of air is used when dustyIce flag is
L02564  !.. false.  Next, a combination of deposition/condensation freezing
L02565  !.. using DeMott et al (2010) dust nucleation when water saturated or
L02566  !.. Phillips et al (2008) when below water saturation; else, without
L02567  !.. dustyIce flag, use the previous Cooper (1986) temperature-dependent
L02568  !.. value.  Lastly, allow homogeneous freezing of deliquesced aerosols
L02569  !.. following Koop et al. (2001, Nature).
L02570  !.. Implemented by T. Eidhammer and G. Thompson 2012Dec18
L02571  !+---+-----------------------------------------------------------------+
L02572
L02573            if (dustyIce .AND. is_aerosol_aware) then
L02574             xni = iceDeMott(tempc,qvs(k),qvs(k),qvsi(k),rho(k),nifa(k))
L02575            else
L02576             xni = 1.0 *1000.                                          ! Default is 1.0 per Liter
L02577            endif
```

```
L02578
L02579  !..Ice nuclei lookup table index.
L02580           if (xni.gt. Nt_IN(1)) then
L02581            niin = NINT(ALOG10(xni))
L02582            do nn = niin-1, niin+1
L02583               n = nn
L02584               if ( (xni/10.**nn).ge.1.0 .and. &
L02585                    (xni/10.**nn).lt.10.0) goto 149
L02586            enddo
L02587   149      continue
L02588            idx_IN = INT(xni/10.**n) + 10*(n-niin2) - (n-niin2)
L02589            idx_IN = MAX(1, MIN(idx_IN, ntb_IN))
L02590           else
L02591            idx_IN = 1
L02592           endif
L02593
L02594  !..Freezing of water drops into graupel/cloud ice (Bigg 1953).
L02595           if (rr(k).gt. r_r(1)) then
L02596            prg_rfz(k) = tpg_qrfz(idx_r,idx_r1,idx_tc,idx_IN)*odts
L02597            pri_rfz(k) = tpi_qrfz(idx_r,idx_r1,idx_tc,idx_IN)*odts
L02598            pni_rfz(k) = tni_qrfz(idx_r,idx_r1,idx_tc,idx_IN)*odts
L02599            pnr_rfz(k) = tnr_qrfz(idx_r,idx_r1,idx_tc,idx_IN)*odts          ! RAIN2M
L02600            pnr_rfz(k) = MIN(DBLE(nr(k)*odts), pnr_rfz(k))
L02601           elseif (rr(k).gt. R1 .and. temp(k).lt.HGFR) then
L02602            pri_rfz(k) = rr(k)*odts
L02603            pni_rfz(k) = nr(k)*odts                                         ! RAIN2M
L02604           endif
L02605           pbg_rfz(k) = prg_rfz(k)/rho_i
L02606
L02607           if (rc(k).gt. r_c(1)) then
L02608            pri_wfz(k) = tpi_qcfz(idx_c,idx_n,idx_tc,idx_IN)*odts
L02609            pri_wfz(k) = MIN(DBLE(rc(k)*odts), pri_wfz(k))
L02610            pni_wfz(k) = tni_qcfz(idx_c,idx_n,idx_tc,idx_IN)*odts
L02611            pni_wfz(k) = MIN(DBLE(nc(k)*odts), pri_wfz(k)/(2.*xm0i),     &
L02612                             pni_wfz(k))
L02613           elseif (rc(k).gt. R1 .and. temp(k).lt.HGFR) then
L02614            pri_wfz(k) = rc(k)*odts
L02615            pni_wfz(k) = nc(k)*odts
L02616           endif
L02617
L02618  !..Deposition nucleation of dust/mineral from DeMott et al (2010)
L02619  !.. we may need to relax the temperature and ssati constraints.
L02620           if ( (ssati(k).ge. 0.25) .or. (ssatw(k).gt. eps &
L02621                                .and. temp(k).lt.253.15) ) then
L02622            if (dustyIce .AND. is_aerosol_aware) then
L02623             xnc = iceDeMott(tempc,qv(k),qvs(k),qvsi(k),rho(k),nifa(k))
L02624            else
L02625             xnc = MIN(250.E3, TNO*EXP(ATO*(T_0-temp(k))))
L02626            endif
L02627            xni = ni(k) + (pni_rfz(k)+pni_wfz(k))*dtsave
L02628            pni_inu(k) = 0.5*(xnc-xni + abs(xnc-xni))*odts
L02629            pri_inu(k) = MIN(DBLE(rate_max), xm0i*pni_inu(k))
L02630            pni_inu(k) = pri_inu(k)/xm0i
L02631           endif
L02632
L02633  !..Freezing of aqueous aerosols based on Koop et al (2001, Nature)
L02634           xni = ns(k)+ni(k) + (pni_rfz(k)+pni_wfz(k)+pni_inu(k))*dtsave
L02635           if (is_aerosol_aware .AND. homogIce .AND. (xni.le.999.E3)     &
L02636      &               .AND.(temp(k).lt.238).AND.(ssati(k).ge.0.4) ) then
L02637             xnc = iceKoop(temp(k),qv(k),qvs(k),nwfa(k), dtsave)
L02638             pni_iha(k) = xnc*odts
L02639             pri_iha(k) = MIN(DBLE(rate_max), xm0i*0.1*pni_iha(k))
L02640             pni_iha(k) = pri_iha(k)/(xm0i*0.1)
L02641           endif
```

条件外壳 `temp < T_0`，然后按三个物理来源依次入账（对应教材 ch13 §13.7.2 的 “三来源” 框架）：

1. **Bigg (1953) 过冷水冻结**（L2593–L2615）：雨滴冻结成霰 +冰（`tpg/tpi/tni/tnr_qrfz` 四表）、云滴冻结成冰晶（`tpi/tni_qcfz` 两表）——表是四维 (含量，截距，温度，`IN` 浓度)：**尘埃越多冻结越快**（`dustyIce` 的温度订正体现在建表时，见 code_07b freezeH2O）；`odts` 把表值换成 “每秒”。极冷（< HGFR = 235.16 K）时冻结概率 ≈1，直接全量冻结（`pri_rfz = rr • odts`）。
2. **异质核化**（L2617–L2630）：触发条件 `ssati ≥ 0.25`（强冰过饱和）或 “水面过饱和且 T < 253.15 K”；来源二选一——aware 模式用 `iceDeMott`（DeMott 2010 尘埃谱，code_07b），否则用 Cooper 曲线 `MIN(250e3, TNO • exp(ATO • (T0−T)))`；`pni_inu = 0.5 • (xnc−xni+|xnc−xni|) • odts` 是**标准的 “只增不减” 半绝对值式**（现有冰核 xni 与目标 xnc 取 max 的整数等价）；质量按出生质量 `xm0i` 配平。
3. **Koop (2001) 均质冻结**（L2632–L2640）：`T < 238 K` 且 `ssati ≥ 0.4` 且现有冰晶数 < 999/L 时，气溶胶液滴直接均质成冰——`iceKoop`（code_07b）按温度-水汽-气溶胶算冻结数，出生质量取 0.1 • xm0i（更小）。

三来源的 “守门条件” 互不重叠（冻结 → 核化 → 均质按温度递冷排布），合起来覆盖 0 到 −40 °C 的全部冰晶生成路径。

07a.8.9 冰晶/雪/霰的凝华与转化

```
!file: phys/module_mp_thompson.F    L2644-L2776
L02644
L02645  !..Deposition/sublimation of cloud ice (Srivastava & Coen 1992).
L02646            if (L_qi(k)) then
L02647             lami = (am_i*cig(2)*oig1*ni(k)/ri(k))**obmi
L02648             ilami = 1./lami
L02649             xDi = MAX(DBLE(D0i), (bm_i + mu_i + 1.) * ilami)
L02650             xmi = am_i*xDi**bm_i
L02651             oxmi = 1./xmi
L02652             pri_ide(k) = C_cube*t1_subl*diffu(k)*ssati(k)*rvs &
L02653                   *oig1*cig(5)*ni(k)*ilami
L02654
L02655             if (pri_ide(k) .lt. 0.0) then
L02656              pri_ide(k) = MAX(DBLE(-ri(k)*odts), pri_ide(k), DBLE(rate_max))
L02657              pni_ide(k) = pri_ide(k)*oxmi
L02658              pni_ide(k) = MAX(DBLE(-ni(k)*odts), pni_ide(k))
L02659             else
L02660              pri_ide(k) = MIN(pri_ide(k), DBLE(rate_max))
L02661              prs_ide(k) = (1.0D0-tpi_ide(idx_i,idx_i1))*pri_ide(k)
L02662              pri_ide(k) = tpi_ide(idx_i,idx_i1)*pri_ide(k)
L02663             endif
L02664
L02665  !..Some cloud ice needs to move into the snow category.  Use lookup
L02666  !.. table that resulted from explicit bin representation of distrib.
L02667             if ( (idx_i.eq. ntb_i) .or. (xDi.gt. 5.0*D0s) ) then
L02668              prs_iau(k) = ri(k)*.99*odts
L02669              pni_iau(k) = ni(k)*.95*odts
L02670             elseif (xDi.lt. 0.1*D0s) then
L02671              prs_iau(k) = 0.
L02672              pni_iau(k) = 0.
L02673             else
L02674              prs_iau(k) = tps_iaus(idx_i,idx_i1)*odts
L02675              prs_iau(k) = MIN(DBLE(ri(k)*.99*odts), prs_iau(k))
L02676              pni_iau(k) = tni_iaus(idx_i,idx_i1)*odts
L02677              pni_iau(k) = MIN(DBLE(ni(k)*.95*odts), pni_iau(k))
L02678             endif
L02679            endif
L02680
L02681  !..Deposition/sublimation of snow/graupel follows Srivastava & Coen
L02682  !.. (1992).
L02683            if (L_qs(k)) then
L02684             C_snow = C_sqrd + (tempc+1.5)*(C_cube-C_sqrd)/(-30.+1.5)
L02685             C_snow = MAX(C_sqrd, MIN(C_snow, C_cube))
L02686             prs_sde(k) = C_snow*t1_subl*diffu(k)*ssati(k)*rvs &
L02687                        * (t1_qs_sd*smo1(k) &
L02688                         + t2_qs_sd*rhof2(k)*vsc2(k)*smof(k))
L02689             if (prs_sde(k).lt. 0.) then
L02690              prs_sde(k) = MAX(DBLE(-rs(k)*odts), prs_sde(k), DBLE(rate_max))
L02691             else
L02692              prs_sde(k) = MIN(prs_sde(k), DBLE(rate_max))
L02693             endif
L02694            endif
L02695
L02696            if (L_qg(k) .and. ssati(k).lt. -eps) then
L02697             t2_qg_sd = 0.28*Sc3*SQRT(av_g(idx_bg(k))) * cgg(11,idx_bg(k))
L02698             prg_gde(k) = C_cube*t1_subl*diffu(k)*ssati(k)*rvs &
L02699                 * N0_g(k) * (t1_qg_sd*ilamg(k)**cge(10,1) &
L02700                 + t2_qg_sd*vsc2(k)*rhof2(k)*ilamg(k)**cge(11,idx_bg(k)))
L02701             if (prg_gde(k).lt. 0.) then
L02702              prg_gde(k) = MAX(DBLE(-rg(k)*odts), prg_gde(k), DBLE(rate_max))
L02703              png_gde(k) = prg_gde(k) * ng(k)/rg(k)
L02704             else
L02705              prg_gde(k) = MIN(prg_gde(k), DBLE(rate_max))
L02706             endif
L02707            endif
L02708
L02709  !..Snow collecting cloud ice.  In CE, assume Di<<Ds and vti=~0.
L02710            if (L_qi(k)) then
L02711             lami = (am_i*cig(2)*oig1*ni(k)/ri(k))**obmi
L02712             ilami = 1./lami
L02713             xDi = MAX(DBLE(D0i), (bm_i + mu_i + 1.) * ilami)
L02714             xmi = am_i*xDi**bm_i
L02715             oxmi = 1./xmi
L02716             if (rs(k).ge. r_s(1)) then
L02717              prs_sci(k) = t1_qs_qi*rhof(k)*Ef_si*ri(k)*smoe(k)
L02718              pni_sci(k) = prs_sci(k) * oxmi
L02719             endif
L02720
L02721  !..Rain collecting cloud ice.  In CE, assume Di<<Dr and vti=~0.
L02722             if (rr(k).ge. r_r(1) .and. mvd_r(k).gt. 4.*xDi) then
L02723              lamr = 1./ilamr(k)
L02724              pri_rci(k) = rhof(k)*t1_qr_qi*Ef_ri*ri(k)*N0_r(k) &
L02725                         *((lamr+fv_r)**(-cre(9)))
L02726              pnr_rci(k) = rhof(k)*t1_qr_qi*Ef_ri*ni(k)*N0_r(k)           &   ! RAIN2M
L02727                         *((lamr+fv_r)**(-cre(9)))
L02728              pnr_rci(k) = MIN(DBLE(nr(k)*odts), pnr_rci(k))
L02729              pni_rci(k) = pri_rci(k) * oxmi
L02730              prr_rci(k) = rhof(k)*t2_qr_qi*Ef_ri*ni(k)*N0_r(k) &
L02731                         *((lamr+fv_r)**(-cre(8)))
L02732              prr_rci(k) = MIN(DBLE(rr(k)*odts), prr_rci(k))
L02733              prg_rci(k) = pri_rci(k) + prr_rci(k)
L02734              pbg_rci(k) = prg_rci(k)/rho_i
L02735             endif
L02736            endif
L02737
L02738  !..Ice multiplication from rime-splinters (Hallet & Mossop 1974).
L02739            if (prg_gcw(k).gt. eps .and. tempc.gt.-8.0) then
L02740             tf = 0.
L02741             if (tempc.ge.-5.0 .and. tempc.lt.-3.0) then
L02742              tf = 0.5*(-3.0 - tempc)
L02743             elseif (tempc.gt.-8.0 .and. tempc.lt.-5.0) then
L02744              tf = 0.33333333*(8.0 + tempc)
L02745             endif
L02746             pni_ihm(k) = 3.5E8*tf*prg_gcw(k)
```

```
L02747             pri_ihm(k) = xm0i*pni_ihm(k)
L02748             prs_ihm(k) = prs_scw(k)/(prs_scw(k)+prg_gcw(k)) &
L02749                        * pri_ihm(k)
L02750             prg_ihm(k) = prg_gcw(k)/(prs_scw(k)+prg_gcw(k)) &
L02751                        * pri_ihm(k)
L02752           endif
L02753
L02754  !..A portion of rimed snow converts to graupel but some remains snow.
L02755  !.. Interp from 15 to 95% as riming factor increases from 2.0 to 30.0
L02756  !.. 0.028 came from (.95-.15)/(30.-2.).  This remains ad-hoc and should
L02757  !.. be revisited.
L02758           if (prs_scw(k).gt.2.0*prs_sde(k) .and. &
L02759                       prs_sde(k).gt.eps) then
L02760            r_frac = MIN(30.0D0, prs_scw(k)/prs_sde(k))
L02761            g_frac = MIN(0.95, 0.15 + (r_frac-2.)*.028)
L02762            vts_boost(k) = MIN(1.5, 1.1 + (r_frac-2.)*.014)
L02763            prg_scw(k) = g_frac*prs_scw(k)
L02764            png_scw(k) = prg_scw(k)*smo0(k)/rs(k)
L02765  !..GT       png_scw(k) = prg_scw(k)*ns(k)/rs(k)
L02766            vts = av_s*xDs**bv_s * EXP(-fv_s*xDs)
L02767            const_Ri = -1.*(mvd_c(k)*0.5E6)*vts/MIN(-0.1,tempc)
L02768            const_Ri = MAX(0.1, MIN(const_Ri, 10.))
L02769            rime_dens = (0.051 + 0.114*const_Ri - 0.0055*const_Ri*const_Ri)*1000.
L02770            if(rime_dens .lt. 150.) then                                  ! Idea of A. Jensen
L02771               g_frac = 0.
L02772               prg_scw(k)=0.
L02773               png_scw(k)=0.
L02774            endif
L02775            pbg_scw(k) = prg_scw(k)/(0.5*(rime_dens+rho_s))
L02776            prs_scw(k) = (1. - g_frac)*prs_scw(k)
```

- **冰晶凝华/升华**（L2644–L2662）：`pri_ide = C_cube • t1_subl • diffu • ssati • rvs • oig1 • cig(5) • ni • ilami`——球电容 ×Ventzke 修正 × 谱积分；升华（负值）时钳位“不许升华过全部冰、不许让过饱和转正”；**凝华时**按 `tpi_ide` 表分账：一部分留冰晶、其余转雪（`prs_ide = (1−tpi_ide) • pri_ide`）——**冰晶长大后自动归类雪**，这就是“冰-雪分界”的表驱动实现。
- **冰 → 雪自动转化**（L2664–L2677）：查 `tps_iaus/tni_iaus` 表（显式 bin 模拟的结果，见 code_07b qi_aut_qs）；冰径超 5× 雪最小径或表顶格时直接 99% 质量/95% 数量转化。
- **雪/霰凝华**（L2680–L2706）：雪的电容 `C_snow` 在 `C_sqrd`–`C_cube` 间按温度线性插值（冷 → 板状 → 小电容）；霰只在**冰面欠饱和**时升华（凝华增长归入淞附主导 regime，不单独算）。
- **雪碰冰 / 雨碰冰**（L2708–L2734）：`prs_sci = t1_qs_qi • rhof • Ef_si • ri • smoe`（常数效率 0.05）；雨碰冰分两笔——冰晶并进雨里（`pri_rci`）与雨滴碰撞损失（`prr_rci`），合流进霰（`prg_rci = pri_rci + prr_rci`）——**雨-冰相撞的产物归霰**（冻结-并合）。
- **Hallett–Mossop 次生冰晶**（L2737–L2751）：只在 −3～−8 °C 窗口、且有霰淞附增长（`prg_gcw > eps`）时触发——三角窗（−3～−5 线性降、−5～−8 线性升），`pni_ihm = 3.5e8 • tf • prg_gcw`（每 kg 淞附质量产 3.5e8 个冰屑），按淞附比例分给冰/雪/霰三个受体。这是次生冰晶的“冰 multiplication”通道。
- **雪 → 霰转化**（L2753–L2776）：淞附越强转霰越多——`r_frac = MIN(30, prs_scw/prs_sde)`（淞附率/凝华率比），`g_frac = 0.15 + (r_frac−2) • 0.028` 钳到 0.95（原注释自承 ad-hoc）；同时 `vts_boost = 1.1 + (r_frac−2) • 0.014`（**淞附雪落速加成**，最多 1.5 倍——后面沉降段用它）；`rime_dens < 150` 时全部取消（A. Jensen 想法：淞得太疏还是雪）。

07a.8.10 融化分支

```
!file: phys/module_mp_thompson.F    L2778-L2845
L02778
L02779           else
L02780
L02781  !..Melt snow and graupel and enhance from collisions with liquid.
L02782  !.. We also need to sublimate snow and graupel if subsaturated.
L02783           if (L_qs(k)) then
L02784            prr_sml(k) = (tempc*tcond(k)-lvap0*diffu(k)*delQvs(k))       &
L02785                       * (t1_qs_me*smo1(k) + t2_qs_me*rhof2(k)*vsc2(k)*smof(k))
L02786            if (prr_sml(k) .gt. 0.) then
L02787               prr_sml(k) = prr_sml(k) + 4218.*olfus*(twet(k)-T_0)       &
L02788                                       * (prr_rcs(k)+prs_scw(k))
L02789            endif
L02790            prr_sml(k) = MIN(DBLE(rs(k)*odts), MAX(0.D0, prr_sml(k)))
L02791
L02792            if (prr_sml(k).gt.0.0) then
L02793               pnr_sml(k) = smo0(k)/rs(k)*prr_sml(k) * 10.0**(-0.25*(twet(k)-T_0))
L02794            elseif (ssati(k).lt. 0.) then
L02795               prs_sde(k) = C_sqrd*t1_subl*diffu(k)*ssati(k)*rvs &
L02796                          * (t1_qs_sd*smo1(k) &
L02797                           + t2_qs_sd*rhof2(k)*vsc2(k)*smof(k))
L02798             prs_sde(k) = MAX(DBLE(-rs(k)*odts), prs_sde(k))
L02799            endif
L02800           endif
L02801
L02802           if (L_qg(k)) then
L02803            N0_melt = N0_g(k)
L02804            if ((rg(k)*ng(k)) .lt. 1.E-4) then
L02805               lamg = 1./ilamg(k)
L02806               N0_melt = (1.E-4/rg(k))*ogg2*lamg**cge(2,1)
L02807            endif
L02808            t2_qg_me = PI*4.*C_cube*olfus * 0.28*Sc3*SQRT(av_g(idx_bg(k))) * cgg(11,idx_bg(k))
L02809            prr_gml(k) = (tempc*tcond(k)-lvap0*diffu(k)*delQvs(k))       &
L02810                       * N0_melt*(t1_qg_me*ilamg(k)**cge(10,1)           &
```

```
L02811                        + t2_qg_me*rhof2(k)*vsc2(k)*ilamg(k)**cge(11,idx_bg(k)))
L02812  !          if (prr_gml(k) .gt. 0.) then
L02813  !             prr_gml(k) = prr_gml(k) + 4218.*olfus*(twet(k)-T_0)       &
L02814  !                                     * (prr_rcg(k)+prg_gcw(k))
L02815  !          endif
L02816             prr_gml(k) = MIN(DBLE(rg(k)*odts), MAX(0.D0, prr_gml(k)))
L02817  !          pnr_gml(k) = N0_g(k)*cgg(2)*ilamg(k)**cge(2) / rg(k)         &   ! RAIN2M
L02818  !                     * prr_gml(k) * 10.0**(-0.5*tempc)
L02819
L02820             if (prr_gml(k) .gt. 0.0) then
L02821              melt_f = MAX(0.05, MIN(prr_gml(k)*dt/rg(k),1.0))
L02822              !..1000 is density water, 50 is lower limit (max ice density is 800)
L02823              pbg_gml(k) = prr_gml(k) / MAX(MIN(melt_f*rho_g(idx_bg(k)),1000.),50.)
L02824  !-GT        pnr_gml(k) = prr_gml(k)*ng(k)/rg(k)
L02825              pnr_gml(k) = prr_gml(k)*ng(k)/rg(k) * 10.0**(-0.33*(twet(k)-T_0))
L02826             elseif (ssati(k).lt. 0.) then
L02827              t2_qg_sd = 0.28*Sc3*SQRT(av_g(idx_bg(k))) * cgg(11,idx_bg(k))
L02828              prg_gde(k) = C_cube*t1_subl*diffu(k)*ssati(k)*rvs &
L02829                  * N0_g(k) * (t1_qg_sd*ilamg(k)**cge(10,1) &
L02830                  + t2_qg_sd*vsc2(k)*rhof2(k)*ilamg(k)**cge(11,idx_bg(k)))
L02831              prg_gde(k) = MAX(DBLE(-rg(k)*odts), prg_gde(k))
L02832              png_gde(k) = prg_gde(k) * ng(k)/rg(k)
L02833             endif
L02834            endif
L02835
L02836  !.. This change will be required if users run adaptive time step that
L02837  !.. results in delta-t that is generally too long to allow cloud water
L02838  !.. collection by snow/graupel above melting temperature.
L02839  !.. Credit to Bjorn-Egil Nygaard for discovering.
L02840            if (dt .gt. 120.) then
L02841               prr_rcw(k)=prr_rcw(k)+prs_scw(k)+prg_gcw(k)
L02842               prs_scw(k)=0.
L02843               prg_gcw(k)=0.
L02844            endif
L02845
```

- **雪融化**（L2782–L2798）：驱动力 = `tempc • tcond − lvap0 • diffu • delQvs`（导热进热 − 蒸发耗潜）；融化增强：湿球温度 > 0 时加“碰并液水带进的热”（`4218 • olfus • (twet−T0) • (prr_rcs+prs_scw)`——4218 是液水比热，碰并的暖水把融化加速）；数浓度衰减因子 `10^(−0.25 • (twet−T0))`（融得越久数越少）。
- **霰融化**（L2801–L2832）：同构但用霰谱参数；`melt_f`（融化份额）控制体积账 `pbg_gml` 的密度分母——**融得越彻底体积账越按水密度记**；数浓度衰减因子换成 0.33 幂（霰比雪大、衰减慢）。
- **自适应时步修正**（L2835–L2843，Nygaard 发现的 bug）：`dt > 120 s` 时，融化层之上雪/霰碰并云水的项**转记为雨碰并云水**——因为长时步里雪/霰在暖层停留期间“碰并-融化-蒸发”的链条已经失真，干脆把碰并直接记给雨。**这是“物理过程对时步敏感”的活教材**（教材 ch21 讲自适应时步时的旁证）。

07a.8.11 守恒五保险

```
!file: phys/module_mp_thompson.F    L2852-L2951
L02852
L02853  !+---+-----------------------------------------------------------------+
L02854  !..Ensure we do not deplete more hydrometeor species than exists.
L02855  !+---+-----------------------------------------------------------------+
L02856        do k = kts, kte
L02857
L02858  !..If ice supersaturated, ensure sum of depos growth terms does not
L02859  !.. deplete more vapor than possibly exists.  If subsaturated, limit
L02860  !.. sum of sublimation terms such that vapor does not reproduce ice
L02861  !.. supersat again.
L02862           sump = pri_inu(k) + pri_ide(k) + prs_ide(k) &
L02863                + prs_sde(k) + prg_gde(k) + pri_iha(k)
L02864           rate_max = (qv(k)-qvsi(k))*rho(k)*odts*0.999
L02865           if ( (sump.gt. eps .and. sump.gt. rate_max) .or. &
L02866                (sump.lt. -eps .and. sump.lt. rate_max) ) then
L02867            ratio = rate_max/sump
L02868            pri_inu(k) = pri_inu(k) * ratio
L02869            pri_ide(k) = pri_ide(k) * ratio
L02870            pni_ide(k) = pni_ide(k) * ratio
L02871            prs_ide(k) = prs_ide(k) * ratio
L02872            prs_sde(k) = prs_sde(k) * ratio
L02873            prg_gde(k) = prg_gde(k) * ratio
L02874            pri_iha(k) = pri_iha(k) * ratio
L02875           endif
L02876
L02877  !..Cloud water conservation.
L02878           sump = -prr_wau(k) - pri_wfz(k) - prr_rcw(k) &
L02879                  - prs_scw(k) - prg_scw(k) - prg_gcw(k)
L02880           rate_max = -rc(k)*odts
L02881           if (sump.lt. rate_max .and. L_qc(k)) then
L02882            ratio = rate_max/sump
L02883            prr_wau(k) = prr_wau(k) * ratio
L02884            pri_wfz(k) = pri_wfz(k) * ratio
L02885            prr_rcw(k) = prr_rcw(k) * ratio
L02886            prs_scw(k) = prs_scw(k) * ratio
L02887            prg_scw(k) = prg_scw(k) * ratio
L02888            prg_gcw(k) = prg_gcw(k) * ratio
L02889           endif
L02890
L02891  !..Cloud ice conservation.
L02892           sump = pri_ide(k) - prs_iau(k) - prs_sci(k) &
L02893                  - pri_rci(k)
L02894           rate_max = -ri(k)*odts
```

```
L02895           if (sump.lt. rate_max .and. L_qi(k)) then
L02896            ratio = rate_max/sump
L02897            pri_ide(k) = pri_ide(k) * ratio
L02898            prs_iau(k) = prs_iau(k) * ratio
L02899            prs_sci(k) = prs_sci(k) * ratio
L02900            pri_rci(k) = pri_rci(k) * ratio
L02901           endif
L02902
L02903  !..Rain conservation.
L02904           sump = -prg_rfz(k) - pri_rfz(k) - prr_rci(k) &
L02905                  + prr_rcs(k) + prr_rcg(k)
L02906           rate_max = -rr(k)*odts
L02907           if (sump.lt. rate_max .and. L_qr(k)) then
L02908            ratio = rate_max/sump
L02909            prg_rfz(k) = prg_rfz(k) * ratio
L02910            pri_rfz(k) = pri_rfz(k) * ratio
L02911            prr_rci(k) = prr_rci(k) * ratio
L02912            prr_rcs(k) = prr_rcs(k) * ratio
L02913            prr_rcg(k) = prr_rcg(k) * ratio
L02914           endif
L02915
L02916  !..Snow conservation.
L02917           sump = prs_sde(k) - prs_ihm(k) - prr_sml(k) &
L02918                  + prs_rcs(k)
L02919           rate_max = -rs(k)*odts
L02920           if (sump.lt. rate_max .and. L_qs(k)) then
L02921            ratio = rate_max/sump
L02922            prs_sde(k) = prs_sde(k) * ratio
L02923            prs_ihm(k) = prs_ihm(k) * ratio
L02924            prr_sml(k) = prr_sml(k) * ratio
L02925            prs_rcs(k) = prs_rcs(k) * ratio
L02926           endif
L02927
L02928  !..Graupel conservation.
L02929           sump = prg_gde(k) - prg_ihm(k) - prr_gml(k) &
L02930                + prg_rcg(k)
L02931           rate_max = -rg(k)*odts
L02932           if (sump.lt. rate_max .and. L_qg(k)) then
L02933            ratio = rate_max/sump
L02934            prg_gde(k) = prg_gde(k) * ratio
L02935            prg_ihm(k) = prg_ihm(k) * ratio
L02936            prr_gml(k) = prr_gml(k) * ratio
L02937            prg_rcg(k) = prg_rcg(k) * ratio
L02938           endif
L02939
L02940  !..Re-enforce proper mass conservation for subsequent elements in case
L02941  !.. any of the above terms were altered.  Thanks P. Blossey. 2009Sep28
L02942           pri_ihm(k) = prs_ihm(k) + prg_ihm(k)
L02943           ratio = MIN( ABS(prr_rcg(k)), ABS(prg_rcg(k)) )
L02944           prr_rcg(k) = ratio * SIGN(1.0, SNGL(prr_rcg(k)))
L02945           prg_rcg(k) = -prr_rcg(k)
L02946           if (twet(k).gt.T_0) then
L02947              ratio = MIN( ABS(prr_rcs(k)), ABS(prs_rcs(k)) )
L02948              prr_rcs(k) = ratio * SIGN(1.0, SNGL(prr_rcs(k)))
L02949              prs_rcs(k) = -prr_rcs(k)
L02950           endif
L02951
```

所有源汇项算完后、装配倾向前，**六个物种逐一做“破产清算”**（对应教材 ch13 §13.7.5）：

1. **水汽账**（L2861–L2874）：六项凝华类源汇 `sump` 与 `rate_max = (qv−qvsi)·ρ·odts·0.999` 比例缩放——**过饱和时**（sump>0）最多耗掉 99.9% 过饱和量；**欠饱和时**（sump<0）最多把水汽降到冰面饱和、不许反向制造过饱和。`0.999` 是留 0.1% 余量防浮点越界。
2. **云水/冰晶/雨/雪/霰账**（L2876–L2937）：五物种同构——各物种的支出项之和 `sump` 不得低于 `−q·odts`（不许花超过存量），超了就按 `ratio = rate_max/sump` **等比例缩放所有支出项**。等比例（而非逐项钳位）是关键：保持各过程相对强度，只整体减速。
3. **收尾对账**（L2939–L2949）：`pri_ihm = prs_ihm + prg_ihm`（次生冰晶总账重算，防两受体分账后被缩放弄破）；雨-霰、雨-雪的碰并对账**取两者绝对值的最小者并强制符号相反**（P. Blossey 2009-09-28 的贡献）——质量不能凭空生灭，收方拿多少、付方就付多少。

这一整套是“守恒约束写在过程计算之后”的范本：**先让物理自由计算，再统一破产清算**——比在每条过程公式里嵌限制更干净，也是绝大多数批量方案的通用架构。

07a.8.12 倾向装配与潜热

```
!file: phys/module_mp_thompson.F    L2953-L3182
L02953
L02954  !+---+-----------------------------------------------------------------+
L02955  !..Calculate tendencies of all species but constrain the number of ice
L02956  !.. to reasonable values.
L02957  !+---+-----------------------------------------------------------------+
L02958        do k = kts, kte
L02959           orho = 1./rho(k)
L02960           lfus2 = lsub - lvap(k)
L02961
L02962  !..Aerosol number tendency
L02963           if (is_aerosol_aware) then
L02964              nwfaten(k) = nwfaten(k) - (pna_rca(k) + pna_sca(k)          &
L02965                         + pna_gca(k) + pni_iha(k)) * orho
L02966              nifaten(k) = nifaten(k) - (pnd_rcd(k) + pnd_scd(k)          &
L02967                         + pnd_gcd(k)) * orho
L02968              if (wif_input_opt .eq. 2) then
L02969                 nbcaten(k) = nbcaten(k) - (pnb_rcb(k) + pnb_scb(k)       &
```

```
                              + pnb_gcb(k)) * orho
            else
               nbcaten(k) = 0.0
            endif
            if (dustyIce) then
               nifaten(k) = nifaten(k) - pni_inu(k)*orho
            else
               nifaten(k) = 0.
            endif
         endif

!..Water vapor tendency
         qvten(k) = qvten(k) + (-pri_inu(k) - pri_iha(k) - pri_ide(k)   &
                     - prs_ide(k) - prs_sde(k) - prg_gde(k)) &
                     * orho

!..Cloud water tendency
         qcten(k) = qcten(k) + (-prr_wau(k) - pri_wfz(k) &
                     - prr_rcw(k) - prs_scw(k) - prg_scw(k) &
                     - prg_gcw(k)) &
                     * orho

!..Cloud water number tendency
         ncten(k) = ncten(k) + (-pnc_wau(k) - pnc_rcw(k) &
                     - pni_wfz(k) - pnc_scw(k) - pnc_gcw(k)) &
                     * orho

!..Cloud water mass/number balance; keep mass-wt mean size between
!.. 1 and 50 microns.  Also no more than Nt_c_max drops total.
         xrc=MAX(R1, (qc1d(k) + qcten(k)*dtsave)*rho(k))
         xnc=MAX(2., (nc1d(k) + ncten(k)*dtsave)*rho(k))
         if (xrc .gt. R1) then
          nu_c = MIN(15, NINT(1000.E6/xnc) + 2)
          lamc = (xnc*am_r*ccg(2,nu_c)*ocg1(nu_c)/rc(k))**obmr
          xDc = (bm_r + nu_c + 1.) / lamc
          if (xDc.lt. D0c) then
           lamc = cce(2,nu_c)/D0c
           xnc = ccg(1,nu_c)*ocg2(nu_c)*xrc/am_r*lamc**bm_r
           ncten(k) = (xnc-nc1d(k)*rho(k))*odts*orho
          elseif (xDc.gt. D0r*2.) then
           lamc = cce(2,nu_c)/(D0r*2.)
           xnc = ccg(1,nu_c)*ocg2(nu_c)*xrc/am_r*lamc**bm_r
           ncten(k) = (xnc-nc1d(k)*rho(k))*odts*orho
          endif
         else
          ncten(k) = -nc1d(k)*odts
         endif
         xnc=MAX(0.,(nc1d(k) + ncten(k)*dtsave)*rho(k))
         if (xnc.gt.Nt_c_max) &
               ncten(k) = (Nt_c_max-nc1d(k)*rho(k))*odts*orho

!..Cloud ice mixing ratio tendency
         qiten(k) = qiten(k) + (pri_inu(k) + pri_iha(k) + pri_ihm(k)    &
                     + pri_wfz(k) + pri_rfz(k) + pri_ide(k) &
                     - prs_iau(k) - prs_sci(k) - pri_rci(k)) &
                     * orho

!..Cloud ice number tendency.
         niten(k) = niten(k) + (pni_inu(k) + pni_iha(k) + pni_ihm(k)    &
                     + pni_wfz(k) + pni_rfz(k) + pni_ide(k) &
                     - pni_iau(k) - pni_sci(k) - pni_rci(k)) &
                     * orho

!..Cloud ice mass/number balance; keep mass-wt mean size between
!.. 5 and 300 microns.  Also no more than 500 xtals per liter.
         xri=MAX(R1,(qi1d(k) + qiten(k)*dtsave)*rho(k))
         xni=MAX(R2,(ni1d(k) + niten(k)*dtsave)*rho(k))
         if (xri.gt. R1) then
           lami = (am_i*cig(2)*oig1*xni/xri)**obmi
           ilami = 1./lami
           xDi = (bm_i + mu_i + 1.) * ilami
           if (xDi.lt. 5.E-6) then
            lami = cie(2)/5.E-6
            xni = MIN(999.D3, cig(1)*oig2*xri/am_i*lami**bm_i)
            niten(k) = (xni-ni1d(k)*rho(k))*odts*orho
           elseif (xDi.gt. 300.E-6) then
            lami = cie(2)/300.E-6
            xni = cig(1)*oig2*xri/am_i*lami**bm_i
            niten(k) = (xni-ni1d(k)*rho(k))*odts*orho
           endif
         else
          niten(k) = -ni1d(k)*odts
         endif
         xni=MAX(0.,(ni1d(k) + niten(k)*dtsave)*rho(k))
         if (xni.gt.999.E3) &
               niten(k) = (999.E3-ni1d(k)*rho(k))*odts*orho

!..Rain tendency
         qrten(k) = qrten(k) + (prr_wau(k) + prr_rcw(k) &
                     + prr_sml(k) + prr_gml(k) + prr_rcs(k) &
                     + prr_rcg(k) - prg_rfz(k) &
                     - pri_rfz(k) - prr_rci(k)) &
                     * orho

!..Rain number tendency
         nrten(k) = nrten(k) + (pnr_wau(k) + pnr_sml(k) + pnr_gml(k)    &
                     - (pnr_rfz(k) + pnr_rcr(k) + pnr_rcg(k))           &
                     + pnr_rcs(k) + pnr_rci(k) + pni_rfz(k)) )          &
                     * orho

!..Rain mass/number balance; keep median volume diameter between
!.. 37 microns (D0r*0.75) and 2.5 mm.
         xrr=MAX(R1,(qr1d(k) + qrten(k)*dtsave)*rho(k))
         xnr=MAX(R2,(nr1d(k) + nrten(k)*dtsave)*rho(k))
         if (xrr.gt. R1) then
           lamr = (am_r*crg(3)*org2*xnr/xrr)**obmr
           mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
```

```
L03077             if (mvd_r(k) .gt. 2.5E-3) then
L03078                mvd_r(k) = 2.5E-3
L03079                lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L03080                xnr = crg(2)*org3*xrr*lamr**bm_r / am_r
L03081                nrten(k) = (xnr-nr1d(k)*rho(k))*odts*orho
L03082             elseif (mvd_r(k) .lt. D0r*0.75) then
L03083                mvd_r(k) = D0r*0.75
L03084                lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L03085                xnr = crg(2)*org3*xrr*lamr**bm_r / am_r
L03086                nrten(k) = (xnr-nr1d(k)*rho(k))*odts*orho
L03087             endif
L03088           else
L03089             qrten(k) = -qr1d(k)*odts
L03090             nrten(k) = -nr1d(k)*odts
L03091           endif
L03092
L03093  !..Snow tendency
L03094           qsten(k) = qsten(k) + (prs_iau(k) + prs_sde(k) + prs_sci(k)    &
L03095                       + prs_scw(k) + prs_rcs(k) + prs_ide(k)             &
L03096                       - prs_ihm(k) - prr_sml(k))                         &
L03097                       * orho
L03098
L03099  !..Graupel tendency
L03100           qgten(k) = qgten(k) + (prg_scw(k) + prg_rfz(k) &
L03101                       + prg_gde(k) + prg_rcg(k) + prg_gcw(k) &
L03102                       + prg_rci(k) + prg_rcs(k) - prg_ihm(k) &
L03103                       - prr_gml(k))                          &
L03104                       * orho
L03105
L03106  !..Graupel number tendency
L03107           ngten(k) = ngten(k) + (png_scw(k) + pnr_rfz(k) - png_rcg(k)    &
L03108                       + pnr_rci(k) + png_rcs(k) + png_gde(k)             &
L03109                       - pnr_gml(k)) * orho
L03110
L03111  !..Graupel volume mixing ratio tendency
L03112           qbten(k) = qbten(k) + (pbg_scw(k) + pbg_rfz(k)                 &
L03113                       + pbg_gcw(k) + pbg_rci(k) + pbg_rcs(k)             &
L03114                       + pbg_rcg(k) + pbg_sml(k) - pbg_gml(k)             &
L03115                       + (prg_gde(k) - prg_ihm(k)) /rho_g(idx_bg(k)) )    &
L03116                       * orho
L03117
L03118  !..Graupel mass/number balance; keep its median volume diameter between
L03119  !.. 3.0 times minimum size (D0g) and 25 mm.
L03120           xrg=MAX(R1,(qg1d(k) + qgten(k)*dtsave)*rho(k))
L03121           xng=MAX(R2,(ng1d(k) + ngten(k)*dtsave)*rho(k))
L03122           xrb=MAX(xrg/rho(k)/rho_g(NRHG),(qb1d(k) + qbten(k)*dtsave))
L03123           xrb=MIN(xrg/rho(k)/rho_g(1), xrb)
L03124
L03125           if (xrg .gt. R1) then
L03126              lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*xng/xrg)**obmg
L03127              mvd_g(k) = (3.0 + mu_g + 0.672) / lamg
L03128
L03129         if(debug_flag) then
L03130              xrho_g = MAX(rho_g(1),MIN(rg(k)/rho(k)/rb(k),rho_g(NRHG)))
L03131              afall = a_coeff*((4.0*xrho_g*9.8)/(3.0*rho(k)))**b_coeff
L03132              afall = afall * visco(k)**(1.0-2.0*b_coeff)
L03133              vtg1 = rhof(k)*afall*cgg(6,idx_bg(k))*ogg3 / lamg**bfall
L03134              vtg2 = rhof(k)*afall*cgg(7,idx_bg(k))/cgg(12,idx_bg(k)) / lamg**bfall
L03135           write(mp_debug, *) 'DEBUG: xrg, xng, xrb, ns, idx_bg, rho_g, mvd, vtg: ', k, xrg, xng, xrb, ns(k), idx_bg(k),
↪  xrg/rho(k)/xrb, mvd_g(k)*1000., vtg1, vtg2
L03136           CALL wrf_debug(0, mp_debug)
L03137           write(mp_debug, *) 'DEBUG: qgten: ', qgten(k), prg_scw(k), prg_rfz(k), prg_gde(k), prg_rcg(k), prg_gcw(k),
↪  prg_rci(k), prg_rcs(k), prg_ihm(k), prr_gml(k)
L03138           CALL wrf_debug(0, mp_debug)
L03139           write(mp_debug, *) 'DEBUG: ngten: ', ngten(k), png_scw(k), pnr_rfz(k), png_rcg(k), pnr_rci(k), png_rcs(k),
↪  pnr_gml(k)
L03140           CALL wrf_debug(0, mp_debug)
L03141           write(mp_debug, *) 'DEBUG: qbten: ', qbten(k), pbg_scw(k), pbg_rfz(k), pbg_gcw(k), pbg_rci(k), pbg_rcs(k),
↪  pbg_rcg(k), pbg_sml(k), pbg_gml(k)
L03142           CALL wrf_debug(0, mp_debug)
L03143         endif
L03144
L03145              if (mvd_g(k) .gt. 25.4E-3) then
L03146                 mvd_g(k) = 25.4E-3
L03147                 lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L03148                 xng = cgg(2,1)*ogg3*xrg*lamg**bm_g / am_g(idx_bg(k))
L03149                 ngten(k) = (xng-ng1d(k)*rho(k))*odts*orho
L03150              elseif (mvd_g(k) .lt. D0r) then
L03151                 mvd_g(k) = D0r
L03152                 lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L03153                 xng = cgg(2,1)*ogg3*xrg*lamg**bm_g / am_g(idx_bg(k))
L03154                 ngten(k) = (xng-ng1d(k)*rho(k))*odts*orho
L03155              endif
L03156
L03157           else
L03158             qgten(k) = -qg1d(k)*odts
L03159             ngten(k) = -ng1d(k)*odts
L03160             qbten(k) = -qb1d(k)*odts
L03161           endif
L03162
L03163  !..Temperature tendency
L03164           if (temp(k).lt.T_0) then
L03165            tten(k) = tten(k) &
L03166                      + ( lsub*ocp(k)*(pri_inu(k) + pri_ide(k) &
L03167                                     + prs_ide(k) + prs_sde(k) &
L03168                                     + prg_gde(k) + pri_iha(k)) &
L03169                       + lfus2*ocp(k)*(pri_wfz(k) + pri_rfz(k) &
L03170                                     + prg_rfz(k) + prs_scw(k) &
L03171                                     + prg_scw(k) + prg_gcw(k) &
L03172                                     + prg_rcs(k) + prs_rcs(k) &
L03173                                     + prr_rci(k) + prg_rcg(k)) &
L03174                         )*orho * (1-IFDRY)
L03175           else
L03176            tten(k) = tten(k) &
L03177                      + ( lfus*ocp(k)*(-prr_sml(k) - prr_gml(k) &
L03178                                     - prr_rcg(k) - prr_rcs(k)) &
```

```
L03179                       + lsub*ocp(k)*(prs_sde(k) + prg_gde(k)) &
L03180                        )*orho * (1-IFDRY)
L03181          endif
L03182
```

六个物种 + 霰体积 + 气溶胶 + 温度的倾向方程逐一装配（全部按命名法典取正负号）：

- **气溶胶倾向**（L2962–L2978）：CCN 被雨/雪/霰清除 + Koop 均质冻结消耗；IN 被 DeMott 核化消耗（`dustyIce` 才记）；黑碳只在 `wif_input_opt=2` 记账。
- **水汽/云水/冰晶/雨/雪/霰倾向**（L2980–L3115）：每条都是 5–8 个源汇项的代数和乘 `orho`（换回混合比单位）。**注意霰体积倾向**（L3111）单独一条——`pbg_*` 项之和 + 凝华/次生项按霰密度折算。
- **质量-数浓度平衡钳位**（L2996–L3018 等）：每个双矩物种在倾向装配后立即做一次“谱自洽检查”——用倾向后的 (q, N) 反解特征直径，超界则重设 λ 并**回算 N 的倾向**（`ncten = (xnc − nc1d • ρ) • odts • orho`）。冰晶上限 999/L、云滴上限 Nt_c_max 在此兑现。**这套钳位在主实体里出现三次**（列装入、TAU+1 更新、最终回写），三处阈值必须一致——改谱参数时最容易漏改的就是这里。
- **温度倾向**（L3162–L3180）：分相记账——冷层（T<0）：`lsub • (凝华类六项) + lfus2 • (冻结类十项)`，其中 `lfus2 = lsub − lvap(T)` 是**温度相关的融化潜热**；暖层：`lfus • (融化四项) + lsub • (雪/霰升华)`。**全部乘 (1−IFDRY)**——`IFDRY=0` 时潜热全量释放，置 1 则“干跑”（不加热只转变物种，用于敏感性实验的调试开关）。

07a.8.13 TAU+1 预更新

```
!file: phys/module_mp_thompson.F    L3184-L3389
L03184
L03185  !+---+-----------------------------------------------------------------+
L03186  !..Update variables for TAU+1 before condensation & sedimention.
L03187  !+---+-----------------------------------------------------------------+
L03188        do k = kts, kte
L03189           temp(k) = t1d(k) + DT*tten(k)
L03190           otemp = 1./temp(k)
L03191           tempc = temp(k) - 273.15
L03192           qv(k) = MAX(1.E-10, qv1d(k) + DT*qvten(k))
L03193           rho(k) = 0.622*pres(k)/(R*temp(k)*(qv(k)+0.622))
L03194           rhof(k) = SQRT(RHO_NOT/rho(k))
L03195           rhof2(k) = SQRT(rhof(k))
L03196           qvs(k) = rslf(pres(k), temp(k))
L03197           ssatw(k) = qv(k)/qvs(k) - 1.
L03198           if (abs(ssatw(k)).lt. eps) ssatw(k) = 0.0
L03199           diffu(k) = 2.11E-5*(temp(k)/273.15)**1.94 * (101325./pres(k))
L03200           if (tempc .ge. 0.0) then
L03201              visco(k) = (1.718+0.0049*tempc)*1.0E-5
L03202           else
L03203              visco(k) = (1.718+0.0049*tempc-1.2E-5*tempc*tempc)*1.0E-5
L03204           endif
L03205           vsc2(k) = SQRT(rho(k)/visco(k))
L03206           lvap(k) = lvap0 + (2106.0 - 4218.0)*tempc
L03207           tcond(k) = (5.69 + 0.0168*tempc)*1.0E-5 * 418.936
L03208           ocp(k) = 1./(Cp*(1.+0.887*qv(k)))
L03209           lvt2(k)=lvap(k)*lvap(k)*ocp(k)*oRv*otemp*otemp
L03210
L03211           nwfa(k) = MAX(11.1E6, (nwfa1d(k) + nwfaten(k)*DT)*rho(k))
L03212        enddo
L03213
L03214        do k = kts, kte
L03215           if ((qc1d(k) + qcten(k)*DT) .gt. R1) then
L03216              rc(k) = (qc1d(k) + qcten(k)*DT)*rho(k)
L03217              nc(k) = MAX(2., MIN((nc1d(k)+ncten(k)*DT)*rho(k), Nt_c_max))
L03218              if (.NOT. is_aerosol_aware) nc(k) = Nt_c
L03219              L_qc(k) = .true.
L03220           else
L03221              rc(k) = R1
L03222              nc(k) = 2.
L03223              L_qc(k) = .false.
L03224           endif
L03225
L03226           if ((qi1d(k) + qiten(k)*DT) .gt. R1) then
L03227              ri(k) = (qi1d(k) + qiten(k)*DT)*rho(k)
L03228              ni(k) = MAX(R2, (ni1d(k) + niten(k)*DT)*rho(k))
L03229              L_qi(k) = .true.
L03230           else
L03231              ri(k) = R1
L03232              ni(k) = R2
L03233              L_qi(k) = .false.
L03234           endif
L03235
L03236           if ((qr1d(k) + qrten(k)*DT) .gt. R1) then
L03237              rr(k) = (qr1d(k) + qrten(k)*DT)*rho(k)
L03238              nr(k) = MAX(R2, (nr1d(k) + nrten(k)*DT)*rho(k))
L03239              L_qr(k) = .true.
L03240              lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L03241              mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
L03242              if (mvd_r(k) .gt. 2.5E-3) then
L03243                 mvd_r(k) = 2.5E-3
L03244                 lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L03245                 nr(k) = crg(2)*org3*rr(k)*lamr**bm_r / am_r
L03246              elseif (mvd_r(k) .lt. D0r*0.75) then
L03247                 mvd_r(k) = D0r*0.75
L03248                 lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
```

```
L03249                nr(k) = crg(2)*org3*rr(k)*lamr**bm_r / am_r
L03250             endif
L03251          else
L03252             rr(k) = R1
L03253             nr(k) = R2
L03254             L_qr(k) = .false.
L03255          endif
L03256
L03257          if ((qs1d(k) + qsten(k)*DT) .gt. R1) then
L03258             rs(k) = (qs1d(k) + qsten(k)*DT)*rho(k)
L03259             L_qs(k) = .true.
L03260          else
L03261             rs(k) = R1
L03262             L_qs(k) = .false.
L03263          endif
L03264       enddo
L03265
L03266       if (is_hail_aware) then
L03267          do k = kts, kte
L03268             if ((qg1d(k) + qgten(k)*DT) .gt. R1) then
L03269                L_qg(k) = .true.
L03270                rg(k) = (qg1d(k) + qgten(k)*DT)*rho(k)
L03271                ng(k) = MAX(R2, (ng1d(k) + ngten(k)*DT)*rho(k))
L03272                rb(k) = MAX(rg(k)/rho(k)/rho_g(NRHG), qb1d(k) + qbten(k)*DT)
L03273                rb(k) = MIN(rg(k)/rho(k)/rho_g(1), rb(k))
L03274                idx_bg(k) = MAX(1,MIN(NINT(rg(k)/rho(k)/rb(k) *0.01)+1,NRHG))
L03275             else
L03276                rg(k) = R1
L03277                ng(k) = R2
L03278                rb(k) = R1/rho(k)/rho_g(NRHG)
L03279                idx_bg(k) = idx_bg1
L03280                L_qg(k) = .false.
L03281             endif
L03282          enddo
L03283       else
L03284          do k = kte, kts, -1
L03285             idx_bg(k) = idx_bg1
L03286          enddo
L03287          do k = kte, kts, -1
L03288          if ((qg1d(k) + qgten(k)*DT) .gt. R1) then
L03289             rg(k) = (qg1d(k) + qgten(k)*DT)*rho(k)
L03290                ygra1 = alog10(max(1.E-9, rg(k)))
L03291                zans1 = 3.0 + 2./7.*(ygra1+8.)
L03292                zans1 = MAX(2., MIN(zans1, 6.))
L03293                N0_exp = 10.**(zans1)
L03294                lam_exp = (N0_exp*am_g(idx_bg(k))*cgg(1,1)/rg(k))**oge1
L03295                lamg = lam_exp * (cgg(3,1)*ogg2*ogg1)**obmg
L03296                ng(k) = cgg(2,1)*ogg3*rg(k)*lamg**bm_g / am_g(idx_bg(k))
L03297                rb(k) = rg(k)/rho(k)/rho_g(idx_bg(k))
L03298          else
L03299             rg(k) = R1
L03300                ng(k) = R2
L03301                rb(k) = R1/rho(k)/rho_g(NRHG)
L03302             L_qg(k) = .false.
L03303          endif
L03304       enddo
L03305       endif
L03306
L03307    !+---+-----------------------------------------------------------------+
L03308    !..With tendency-updated mixing ratios, recalculate snow moments and
L03309    !.. intercepts/slopes of graupel and rain.
L03310    !+---+-----------------------------------------------------------------+
L03311          if (.not. iiwarm) then
L03312          do k = kts, kte
L03313             smo2(k) = 0.
L03314             smob(k) = 0.
L03315             smoc(k) = 0.
L03316             smod(k) = 0.
L03317          enddo
L03318          do k = kts, kte
L03319             if (.not. L_qs(k)) CYCLE
L03320             tc0 = MIN(-0.1, temp(k)-273.15)
L03321             smob(k) = rs(k)*oams
L03322
L03323    !..All other moments based on reference, 2nd moment.  If bm_s.ne.2,
L03324    !.. then we must compute actual 2nd moment and use as reference.
L03325             if (bm_s.gt.(2.0-1.e-3) .and. bm_s.lt.(2.0+1.e-3)) then
L03326                smo2(k) = smob(k)
L03327             else
L03328                loga_ = sa(1) + sa(2)*tc0 + sa(3)*bm_s &
L03329                   + sa(4)*tc0*bm_s + sa(5)*tc0*tc0 &
L03330                   + sa(6)*bm_s*bm_s + sa(7)*tc0*tc0*bm_s &
L03331                   + sa(8)*tc0*bm_s*bm_s + sa(9)*tc0*tc0*tc0 &
L03332                   + sa(10)*bm_s*bm_s*bm_s
L03333                a_ = 10.0**loga_
L03334                b_ = sb(1) + sb(2)*tc0 + sb(3)*bm_s &
L03335                   + sb(4)*tc0*bm_s + sb(5)*tc0*tc0 &
L03336                   + sb(6)*bm_s*bm_s + sb(7)*tc0*tc0*bm_s &
L03337                   + sb(8)*tc0*bm_s*bm_s + sb(9)*tc0*tc0*tc0 &
L03338                   + sb(10)*bm_s*bm_s*bm_s
L03339                smo2(k) = (smob(k)/a_)**(1./b_)
L03340             endif
L03341
L03342    !..Calculate bm_s+1 (th) moment.  Useful for diameter calcs.
L03343             loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(1) &
L03344                   + sa(4)*tc0*cse(1) + sa(5)*tc0*tc0 &
L03345                   + sa(6)*cse(1)*cse(1) + sa(7)*tc0*tc0*cse(1) &
L03346                   + sa(8)*tc0*cse(1)*cse(1) + sa(9)*tc0*tc0*tc0 &
L03347                   + sa(10)*cse(1)*cse(1)*cse(1)
L03348             a_ = 10.0**loga_
L03349             b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(1) + sb(4)*tc0*cse(1) &
L03350                + sb(5)*tc0*tc0 + sb(6)*cse(1)*cse(1) &
L03351                + sb(7)*tc0*tc0*cse(1) + sb(8)*tc0*cse(1)*cse(1) &
L03352                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(1)*cse(1)*cse(1)
L03353             smoc(k) = a_ * smo2(k)**b_
L03354
L03355    !..Calculate bm_s+bv_s (th) moment.  Useful for sedimentation.
```

```
L03356           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(14) &
L03357                 + sa(4)*tc0*cse(14) + sa(5)*tc0*tc0 &
L03358                 + sa(6)*cse(14)*cse(14) + sa(7)*tc0*tc0*cse(14) &
L03359                 + sa(8)*tc0*cse(14)*cse(14) + sa(9)*tc0*tc0*tc0 &
L03360                 + sa(10)*cse(14)*cse(14)*cse(14)
L03361           a_ = 10.0**loga_
L03362           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(14) + sb(4)*tc0*cse(14) &
L03363                + sb(5)*tc0*tc0 + sb(6)*cse(14)*cse(14) &
L03364                + sb(7)*tc0*tc0*cse(14) + sb(8)*tc0*cse(14)*cse(14) &
L03365                + sb(9)*tc0*tc0*tc0 + sb(10)*cse(14)*cse(14)*cse(14)
L03366           smod(k) = a_ * smo2(k)**b_
L03367         enddo
L03368
L03369  !+---+-----------------------------------------------------------------+
L03370  !..Calculate y-intercept, slope values for graupel.
L03371  !+---+-----------------------------------------------------------------+
L03372
L03373         do k = kte, kts, -1
L03374            lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*ng(k)/rg(k))**obmg
L03375            ilamg(k) = 1./lamg
L03376            N0_g(k) = ng(k)*ogg2*lamg**cge(2,1)
L03377         enddo
L03378
L03379         endif
L03380
L03381  !+---+-----------------------------------------------------------------+
L03382  !..Calculate y-intercept, slope values for rain.
L03383  !+---+-----------------------------------------------------------------+
L03384         do k = kte, kts, -1
L03385            lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L03386            ilamr(k) = 1./lamr
L03387            mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
L03388            N0_r(k) = nr(k)*org2*lamr**cre(2)
L03389         enddo
```

- **为什么要有这一段**：饱和调整和沉降要用**本步结束后的状态**（q+dt • tendency）而不是步初状态——否则凝结量与沉降量会系统性偏差半步。源码注释 “Update variables for TAU+1 before condensation & sedimentation” 即此意。
- **热力学量重算**（L3187–L3211）：`lvt2 = lvap² • ocp • oRv • otemp²` 出现——这是**饱和调整牛顿迭代的分母系数**（见下一段）。
- **谱参数重算**（L3213–L3389）：按新 (q, N) 重走一遍 “反解 λ→ 钳位 → 重算 N→ 雪矩量 → 霰/雨谱参数”——**与 §07a.8.2 完全同构的代码复制**（Thompson 代码可读性的主要代价：没有函数化，同构逻辑平铺两遍）。雪矩量重算只算沉降需要的 `smod`（bm_s+bv_s 矩）。

07a.8.14 饱和调整（凝结/蒸发）

```
!file: phys/module_mp_thompson.F    L3391-L3494
L03391  !+---+-----------------------------------------------------------------+
L03392  !..Cloud water condensation and evaporation.  Nucleate cloud droplets
L03393  !.. using explicit CCN aerosols with hygroscopicity like sulfates using
L03394  !.. parcel model lookup table results provided by T. Eidhammer.  Evap
L03395  !.. drops using calculation of max drop size capable of evaporating in
L03396  !.. single timestep and explicit number of drops smaller than Dc_star
L03397  !.. from lookup table.
L03398  !+---+-----------------------------------------------------------------+
L03399         do k = kts, kte
L03400            orho = 1./rho(k)
L03401            if ( (ssatw(k).gt. eps) .or. (ssatw(k).lt. -eps .and. &
L03402                  L_qc(k)) ) then
L03403             clap = (qv(k)-qvs(k))/(1. + lvt2(k)*qvs(k))
L03404             do n = 1, 3
L03405                fcd = qvs(k)* EXP(lvt2(k)*clap) - qv(k) + clap
L03406                dfcd = qvs(k)*lvt2(k)* EXP(lvt2(k)*clap) + 1.
L03407                clap = clap - fcd/dfcd
L03408             enddo
L03409             xrc = rc(k) + clap*rho(k)
L03410             xnc = 0.
L03411             if (xrc.gt. R1) then
L03412              prw_vcd(k) = clap*odt
L03413  !+---+-----------------------------------------------------------------+ !  DROPLET NUCLEATION
L03414              if (clap .gt. eps) then
L03415               if (is_aerosol_aware) then
L03416                  xnc = MAX(2., activ_ncloud(temp(k), w1d(k), nwfa(k)))
L03417               else
L03418                  xnc = Nt_c
L03419               endif
L03420               pnc_wcd(k) = 0.5*(xnc-nc(k) + abs(xnc-nc(k)))*odts*orho
L03421
L03422  !+---+-----------------------------------------------------------------+ !  EVAPORATION
L03423             elseif (clap .lt. -eps .AND. ssatw(k).lt.-1.E-6 .AND. is_aerosol_aware) then
L03424              tempc = temp(k) - 273.15
L03425              otemp = 1./temp(k)
L03426              rvs = rho(k)*qvs(k)
L03427              rvs_p = rvs*otemp*(lvap(k)*otemp*oRv - 1.)
L03428              rvs_pp = rvs * ( otemp*(lvap(k)*otemp*oRv - 1.) &
L03429                             *otemp*(lvap(k)*otemp*oRv - 1.) &
L03430                             + (-2.*lvap(k)*otemp*otemp*otemp*oRv) &
L03431                             + otemp*otemp)
L03432              gamsc = lvap(k)*diffu(k)/tcond(k) * rvs_p
L03433              alphsc = 0.5*(gamsc/(1.+gamsc))*(gamsc/(1.+gamsc)) &
L03434                     * rvs_pp/rvs_p * rvs/rvs_p
L03435              alphsc = MAX(1.E-9, alphsc)
L03436              xsat = ssatw(k)
L03437              if (abs(xsat).lt. 1.E-9) xsat=0.
L03438              t1_evap = 2.*PI*( 1.0 - alphsc*xsat  &
L03439                     + 2.*alphsc*alphsc*xsat*xsat  &
L03440                     - 5.*alphsc*alphsc*alphsc*xsat*xsat*xsat ) &
```

```
L03441                    / (1.+gamsc)
L03442
L03443             Dc_star = DSQRT(-2.D0*DT * t1_evap/(2.*PI) &
L03444                     * 4.*diffu(k)*ssatw(k)*rvs/rho_w)
L03445             idx_d = MAX(1, MIN(INT(1.E6*Dc_star), nbc))
L03446
L03447             idx_n = NINT(1.0 + FLOAT(nbc) * DLOG(nc(k)/t_Nc(1)) / nic1)
L03448             idx_n = MAX(1, MIN(idx_n, nbc))
L03449
L03450  !..Cloud water lookup table index.
L03451             if (rc(k).gt. r_c(1)) then
L03452              nic = NINT(ALOG10(rc(k)))
L03453              do nn = nic-1, nic+1
L03454                 n = nn
L03455                 if ( (rc(k)/10.**nn).ge.1.0 .and. &
L03456                      (rc(k)/10.**nn).lt.10.0) goto 159
L03457              enddo
L03458   159        continue
L03459              idx_c = INT(rc(k)/10.**n) + 10*(n-nic2) - (n-nic2)
L03460              idx_c = MAX(1, MIN(idx_c, ntb_c))
L03461             else
L03462              idx_c = 1
L03463             endif
L03464
L03465            !prw_vcd(k) = MAX(DBLE(-rc(k)*orho*odt),                    &
L03466            !          -tpc_wev(idx_d, idx_c, idx_n)*orho*odt)
L03467             prw_vcd(k) = MAX(DBLE(-rc(k)*0.99*orho*odt), prw_vcd(k))
L03468             pnc_wcd(k) = MAX(DBLE(-nc(k)*0.99*orho*odt),               &
L03469                              DBLE(-tnc_wev(idx_d, idx_c, idx_n)*orho*odt))
L03470
L03471            endif
L03472           else
L03473            prw_vcd(k) = -rc(k)*orho*odt
L03474            pnc_wcd(k) = -nc(k)*orho*odt
L03475           endif
L03476
L03477  !+---+-----------------------------------------------------------------+
L03478
L03479           qvten(k) = qvten(k) - prw_vcd(k)
L03480           qcten(k) = qcten(k) + prw_vcd(k)
L03481           ncten(k) = ncten(k) + pnc_wcd(k)
L03482           nwfaten(k) = nwfaten(k) - pnc_wcd(k)
L03483           tten(k) = tten(k) + lvap(k)*ocp(k)*prw_vcd(k)*(1-IFDRY)
L03484           rc(k) = MAX(R1, (qc1d(k) + DT*qcten(k))*rho(k))
L03485           if (rc(k).eq.R1) L_qc(k) = .false.
L03486           nc(k) = MAX(2., MIN((nc1d(k)+ncten(k)*DT)*rho(k), Nt_c_max))
L03487           if (.NOT. is_aerosol_aware) nc(k) = Nt_c
L03488           qv(k) = MAX(1.E-10, qv1d(k) + DT*qvten(k))
L03489           temp(k) = t1d(k) + DT*tten(k)
L03490           rho(k) = 0.622*pres(k)/(R*temp(k)*(qv(k)+0.622))
L03491           qvs(k) = rslf(pres(k), temp(k))
L03492           ssatw(k) = qv(k)/qvs(k) - 1.
L03493          endif
L03494       enddo
```

这是与书 (6.6.3) 直接对应的一段（数学原型与三种解法的对照见教材 ch13 §13.7.4 / 13.9.3），流程：

1. **触发条件**（L3401）：水面过饱和（`ssatw > eps`）或“欠饱和且有云水”（可蒸发）。
2. **牛顿迭代解凝结量**（L3403–L3408）：解 `fcd(clap) = qvs·exp(lvt2·clap) − qv + clap = 0`——`clap` 是凝结出的液水混合比，凝结后水汽变成 `qv − clap`、温度升 `lvap·clap/cp`（含在 `lvt2` 里），饱和比沿 Clausius–Clapeyron 指数上移；**恰好迭代 3 次**（`do n = 1, 3`）——3 次牛顿对这光滑函数收敛到毫开尔文级（教材 13.9.3 的三法对照实验）。
3. **凝结分支**（L3414–L3420）：`clap > eps` 时活化云滴——aware 模式 `activ_ncloud(temp, w, nwfa)` 查活化表（气溶胶数、垂直速度、温度三要素 → 新云滴数，表由 code_07b table_ccnAct 构建），否则用常数 `Nt_c`；`pnc_wcd` 半绝对值式只增不减。
4. **蒸发分支**（L3422–L3471）：`Dc_star = sqrt(−2·DT·t1_evap/(2π)·4·diffu·ssatw·rvs/rho_w)`——**单步内能完全蒸发掉的最大云滴径**（蒸发位移的解析反解）；比它小的滴全部蒸发（查 `tpc_wev/tnc_wev` 表按质量/数分账）。
5. **入账与状态同步**（L3479–L3493）：`qvten −= prw_vcd`、`qcten += prw_vcd`、`tten += lvap·ocp·prw_vcd`；**立即更新局部状态**（`qv/temp/rho/qvs/ssatw` 同步重算）——下一个格点层（雨蒸发）用的是“凝结后”的过饱和度。

07a.8.15 雨的蒸发

```
!file: phys/module_mp_thompson.F    L3496-L3586
L03496  !+---+-----------------------------------------------------------------+
L03497  !.. If still subsaturated, allow rain to evaporate, following
L03498  !.. Srivastava & Coen (1992).
L03499  !+---+-----------------------------------------------------------------+
L03500        do k = kts, kte
L03501           if ( (ssatw(k).lt. -eps) .and. L_qr(k) &
L03502                     .and. (.not.(prw_vcd(k).gt. 0.)) ) then
L03503            tempc = temp(k) - 273.15
L03504            otemp = 1./temp(k)
L03505            orho = 1./rho(k)
L03506            rhof(k) = SQRT(RHO_NOT*orho)
L03507            rhof2(k) = SQRT(rhof(k))
L03508            diffu(k) = 2.11E-5*(temp(k)/273.15)**1.94 * (101325./pres(k))
L03509            if (tempc .ge. 0.0) then
L03510               visco(k) = (1.718+0.0049*tempc)*1.0E-5
L03511            else
```

```
L03512              visco(k) = (1.718+0.0049*tempc-1.2E-5*tempc*tempc)*1.0E-5
L03513            endif
L03514            vsc2(k) = SQRT(rho(k)/visco(k))
L03515            lvap(k) = lvap0 + (2106.0 - 4218.0)*tempc
L03516            tcond(k) = (5.69 + 0.0168*tempc)*1.0E-5 * 418.936
L03517            ocp(k) = 1./(Cp*(1.+0.887*qv(k)))
L03518
L03519            rvs = rho(k)*qvs(k)
L03520            rvs_p = rvs*otemp*(lvap(k)*otemp*oRv - 1.)
L03521            rvs_pp = rvs * ( otemp*(lvap(k)*otemp*oRv - 1.) &
L03522                           *otemp*(lvap(k)*otemp*oRv - 1.) &
L03523                           + (-2.*lvap(k)*otemp*otemp*otemp*oRv) &
L03524                           + otemp*otemp)
L03525            gamsc = lvap(k)*diffu(k)/tcond(k) * rvs_p
L03526            alphsc = 0.5*(gamsc/(1.+gamsc))*(gamsc/(1.+gamsc)) &
L03527                   * rvs_pp/rvs_p * rvs/rvs_p
L03528            alphsc = MAX(1.E-9, alphsc)
L03529            xsat   = MIN(-1.E-9, ssatw(k))
L03530            t1_evap = 2.*PI*( 1.0 - alphsc*xsat  &
L03531                  + 2.*alphsc*alphsc*xsat*xsat  &
L03532                  - 5.*alphsc*alphsc*alphsc*xsat*xsat*xsat ) &
L03533                  / (1.+gamsc)
L03534
L03535            lamr = 1./ilamr(k)
L03536  !..Rapidly eliminate near zero values when low humidity (<95%)
L03537            if (qv(k)/qvs(k) .lt. 0.95 .AND. rr(k)*orho.le.1.E-8) then
L03538            prv_rev(k) = rr(k)*orho*odts
L03539            else
L03540            prv_rev(k) = t1_evap*diffu(k)*(-ssatw(k))*N0_r(k)*rvs &
L03541                * (t1_qr_ev*ilamr(k)**cre(10) &
L03542                + t2_qr_ev*vsc2(k)*rhof2(k)*((lamr+0.5*fv_r)**(-cre(11))))
L03543            rate_max = MIN((rr(k)*orho*odts), (qvs(k)-qv(k))*odts)
L03544            prv_rev(k) = MIN(DBLE(rate_max), prv_rev(k)*orho)
L03545
L03546  !..TEST: G. Thompson  10 May 2013
L03547  !..Reduce the rain evaporation in same places as melting graupel occurs.
L03548  !..Rationale: falling and simultaneous melting graupel in subsaturated
L03549  !..regions will not melt as fast because particle temperature stays
L03550  !..at 0C.  Also not much shedding of the water from the graupel so
L03551  !..likely that the water-coated graupel evaporating much slower than
L03552  !..if the water was immediately shed off.
L03553            IF (prr_gml(k).gt.0.0) THEN
L03554               eva_factor = MIN(1.0, 0.01+(0.99-0.01)*(tempc/20.0))
L03555               prv_rev(k) = prv_rev(k)*eva_factor
L03556            ENDIF
L03557            endif
L03558
L03559            pnr_rev(k) = MIN(DBLE(nr(k)*0.99*orho*odts),                &   ! RAIN2M
L03560                         prv_rev(k) * nr(k)/rr(k))
L03561
L03562            qrten(k) = qrten(k) - prv_rev(k)
L03563            qvten(k) = qvten(k) + prv_rev(k)
L03564            nrten(k) = nrten(k) - pnr_rev(k)
L03565            nwfaten(k) = nwfaten(k) + pnr_rev(k)
L03566            tten(k) = tten(k) - lvap(k)*ocp(k)*prv_rev(k)*(1-IFDRY)
L03567
L03568            rr(k) = MAX(R1, (qr1d(k) + DT*qrten(k))*rho(k))
L03569            qv(k) = MAX(1.E-10, qv1d(k) + DT*qvten(k))
L03570            nr(k) = MAX(R2, (nr1d(k) + DT*nrten(k))*rho(k))
L03571            temp(k) = t1d(k) + DT*tten(k)
L03572            rho(k) = 0.622*pres(k)/(R*temp(k)*(qv(k)+0.622))
L03573           endif
L03574         enddo
L03575  #if ( WRF_CHEM == 1 )
L03576         if( wetscav_on ) then
L03577           do k = kts, kte
L03578             evapprod(k) = prv_rev(k) - (min(zeroD0,prs_sde(k)) + &
L03579                                         min(zeroD0,prg_gde(k)))
L03580             rainprod(k) = prr_wau(k) + prr_rcw(k) + prs_scw(k) + &
L03581                                        prg_scw(k) + prs_iau(k) + &
L03582                                        prg_gcw(k) + prs_sci(k) + &
L03583                                        pri_rci(k)
L03584           enddo
L03585         endif
L03586  #endif
```

- **触发**：仍欠饱和、有雨、且云水凝结段没有产生凝结（`.not.(prw_vcd>0)`）。
- **Srivastava & Coen (1992) 蒸发率**：与凝华同构的 Ventzke 修正，但用**液面饱和**与雨谱矩（`t1_qr_ev = 0.78 • crg(10)`、`t2_qr_ev` 含通风因子 `vsc2 • rhof2`）。
- **小雨速灭**（L3537–L3538）：RH < 95% 且雨径混合比 < 1e-8 时直接全蒸发——数值卫生。
- **融霰蒸发抑制**（L3546–L3556，Thompson 2013-05-10 试验注释）：融霰区（`prr_gml > 0`）的雨蒸发乘 `eva_factor = 0.01 + 0.98 • (tempc/20)`——**理由写在注释里**：水膜霰融化时粒子温度钉在 0 °C、水不立即脱落，包水霰的蒸发远慢于 “立即脱落” 假设。这是 “模式物理 = 实验室物理 + 田间修正” 的好例子。
- **WRF-CHEM 挂钩**（L3575–L3586）：`rainprod/evapprod` 给化学湿清除用——`evapprod = 蒸发 − (雪/霰升华负值)`、`rainprod = 八项液态增长之和`。

07a.8.16 末速度与子步长判定

```
!file: phys/module_mp_thompson.F    L3588-L3782
!+---+-----------------------------------------------------------------+
!..Find max terminal fallspeed (distribution mass-weighted mean
!.. velocity) and use it to determine if we need to split the timestep
!.. (var nstep>1).  Either way, only bother to do sedimentation below
!.. 1st level that contains any sedimenting particles (k=ksed1 on down).
!.. New in v3.0+ is computing separate for rain, ice, snow, and
!.. graupel species thus making code faster with credit to J. Schmidt.
!+---+-----------------------------------------------------------------+
      nstep = 0
      onstep(:) = 1.0
      ksed1(:) = 1
      do k = kte+1, kts, -1
         vtrk(k) = 0.
         vtnrk(k) = 0.
         vtik(k) = 0.
         vtnik(k) = 0.
         vtsk(k) = 0.
         vtgk(k) = 0.
         vtngk(k) = 0.
         vtck(k) = 0.
         vtnck(k) = 0.
      enddo

      if (ANY(L_qr .eqv. .true.)) then
      do k = kte, kts, -1
         vtr = 0.
         rhof(k) = SQRT(RHO_NOT/rho(k))

         if (rr(k).gt. R1) then
          lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
          vtr = rhof(k)*av_r*crg(6)*org3 * lamr**cre(3)                &
                       *((lamr+fv_r)**(-cre(6)))
          vtrk(k) = vtr
! First below is technically correct:
!         vtr = rhof(k)*av_r*crg(5)*org2 * lamr**cre(2)                &
!                      *((lamr+fv_r)**(-cre(5)))
! Test: make number fall faster (but still slower than mass)
! Goal: less prominent size sorting
          vtr = rhof(k)*av_r*crg(7)/crg(12) * lamr**cre(12)            &
                       *((lamr+fv_r)**(-cre(7)))
          vtnrk(k) = vtr
         else
          vtrk(k) = vtrk(k+1)
          vtnrk(k) = vtnrk(k+1)
         endif

         if (MAX(vtrk(k),vtnrk(k)) .gt. 1.E-3) then
            ksed1(1) = MAX(ksed1(1), k)
            delta_tp = dzq(k)/(MAX(vtrk(k),vtnrk(k)))
            nstep = MAX(nstep, INT(DT/delta_tp + 1.))
         endif
      enddo
      if (ksed1(1) .eq. kte) ksed1(1) = kte-1
      if (nstep .gt. 0) onstep(1) = 1./REAL(nstep)
      endif

!+---+-----------------------------------------------------------------+

      if (ANY(L_qc .eqv. .true.)) then
      hgt_agl = 0.
      do k = kts, kte-1
         if (rc(k) .gt. R2) ksed1(5) = k
         hgt_agl = hgt_agl + dzq(k)
         if (hgt_agl .gt. 500.0) goto 151
      enddo
 151  continue

      do k = ksed1(5), kts, -1
         vtc = 0.
         if (rc(k) .gt. R1 .and. w1d(k) .lt. 1.E-1) then
          nu_c = MIN(15, NINT(1000.E6/nc(k)) + 2)
          lamc = (nc(k)*am_r*ccg(2,nu_c)*ocg1(nu_c)/rc(k))**obmr
          ilamc = 1./lamc
          vtc = rhof(k)*av_c*ccg(5,nu_c)*ocg2(nu_c) * ilamc**bv_c
          vtck(k) = vtc
          vtc = rhof(k)*av_c*ccg(4,nu_c)*ocg1(nu_c) * ilamc**bv_c
          vtnck(k) = vtc
         endif
      enddo
      endif

!+---+-----------------------------------------------------------------+

      if (.not. iiwarm) then

       if (ANY(L_qi .eqv. .true.)) then
       nstep = 0
       do k = kte, kts, -1
          vti = 0.

          if (ri(k).gt. R1) then
           lami = (am_i*cig(2)*oig1*ni(k)/ri(k))**obmi
           ilami = 1./lami
           vti = rhof(k)*av_i*cig(3)*oig2 * ilami**bv_i
           vtik(k) = vti
! First below is technically correct:
!          vti = rhof(k)*av_i*cig(4)*oig1 * ilami**bv_i
! Goal: less prominent size sorting
           vti = rhof(k)*av_i*cig(6)/cig(7) * ilami**bv_i
           vtnik(k) = vti
          else
           vtik(k) = vtik(k+1)
           vtnik(k) = vtnik(k+1)
          endif
```

```
L03692
L03693           if (vtik(k) .gt. 1.E-3) then
L03694              ksed1(2) = MAX(ksed1(2), k)
L03695              delta_tp = dzq(k)/vtik(k)
L03696              nstep = MAX(nstep, INT(DT/delta_tp + 1.))
L03697           endif
L03698         enddo
L03699         if (ksed1(2) .eq. kte) ksed1(2) = kte-1
L03700         if (nstep .gt. 0) onstep(2) = 1./REAL(nstep)
L03701         endif
L03702
L03703   !+---+-----------------------------------------------------------------+
L03704
L03705         if (ANY(L_qs .eqv. .true.)) then
L03706         nstep = 0
L03707         do k = kte, kts, -1
L03708            vts = 0.
L03709
L03710            if (rs(k).gt. R1) then
L03711             xDs = smoc(k) / smob(k)
L03712             Mrat = 1./xDs
L03713             ils1 = 1./(Mrat*Lam0 + fv_s)
L03714             ils2 = 1./(Mrat*Lam1 + fv_s)
L03715             t1_vts = Kap0*csg(4)*ils1**cse(4)
L03716             t2_vts = Kap1*Mrat**mu_s*csg(10)*ils2**cse(10)
L03717             ils1 = 1./(Mrat*Lam0)
L03718             ils2 = 1./(Mrat*Lam1)
L03719             t3_vts = Kap0*csg(1)*ils1**cse(1)
L03720             t4_vts = Kap1*Mrat**mu_s*csg(7)*ils2**cse(7)
L03721             vts = rhof(k)*av_s * (t1_vts+t2_vts)/(t3_vts+t4_vts)
L03722             if (prr_sml(k) .gt. 0.0) then
L03723              SR = rs(k)/(rs(k)+rr(k))
L03724              vtsk(k) = vts*SR + (1.-SR)*vtrk(k)
L03725             else
L03726              vtsk(k) = vts*vts_boost(k)
L03727             endif
L03728            else
L03729              vtsk(k) = vtsk(k+1)
L03730            endif
L03731
L03732            if (vtsk(k) .gt. 1.E-3) then
L03733               ksed1(3) = MAX(ksed1(3), k)
L03734               delta_tp = dzq(k)/vtsk(k)
L03735               nstep = MAX(nstep, INT(DT/delta_tp + 1.))
L03736            endif
L03737         enddo
L03738         if (ksed1(3) .eq. kte) ksed1(3) = kte-1
L03739         if (nstep .gt. 0) onstep(3) = 1./REAL(nstep)
L03740         endif
L03741
L03742   !+---+-----------------------------------------------------------------+
L03743
L03744         if (ANY(L_qg .eqv. .true.)) then
L03745         nstep = 0
L03746         do k = kte, kts, -1
L03747            vtg = 0.
L03748
L03749            if (rg(k).gt. R1) then
L03750             if (is_hail_aware) then
L03751                xrho_g = MAX(rho_g(1),MIN(rg(k)/rho(k)/rb(k),rho_g(NRHG)))
L03752                afall = a_coeff*((4.0*xrho_g*9.8)/(3.0*rho(k)))**b_coeff
L03753                afall = afall * visco(k)**(1.0-2.0*b_coeff)
L03754             else
L03755                afall = av_g_old
L03756                bfall = bv_g_old
L03757             endif
L03758             vtg = rhof(k)*afall*cgg(6,idx_bg(k))*ogg3 * ilamg(k)**bfall
L03759              vtgk(k) = vtg
L03760   ! Goal: less prominent size sorting
L03761   !    the ELSE section below is technically (mathematically) correct:
L03762             if (mu_g .eq. 0) then
L03763                vtg = rhof(k)*afall*cgg(7,idx_bg(k))/cgg(12,idx_bg(k)) * ilamg(k)**bfall
L03764             else
L03765                vtg = rhof(k)*afall*cgg(8,idx_bg(k))*ogg2 * ilamg(k)**bfall
L03766             endif
L03767             vtngk(k) = vtg
L03768            else
L03769              vtgk(k) = vtgk(k+1)
L03770             vtngk(k) = vtngk(k+1)
L03771            endif
L03772
L03773            if (vtgk(k) .gt. 1.E-3) then
L03774               ksed1(4) = MAX(ksed1(4), k)
L03775               delta_tp = dzq(k)/vtgk(k)
L03776               nstep = MAX(nstep, INT(DT/delta_tp + 1.))
L03777            endif
L03778         enddo
L03779         if (ksed1(4) .eq. kte) ksed1(4) = kte-1
L03780         if (nstep .gt. 0) onstep(4) = 1./REAL(nstep)
L03781         endif
L03782        endif
```

- **质量加权末速度**：`vtr = rhof·av_r·crg(6)·org3·λ^cre(3)·(λ+fv_r)^(−cre(6))`——即 $\int v \cdot m \cdot N\ dD / \int m \cdot N\ dD$ 的解析结果。**数浓度落速的“作弊”**（L3621–L3627 原注释：First below is technically correct / Test: make number fall faster）：数浓度本该用 `crg(5)` 矩（数学正确），实测却用 `crg(7)/crg(12)`——**让数比质量落得快一点，削弱“大雨滴跑得快”造成的尺寸分选伪影**。这类“数学让位于物理效果”的调参在源码注释里明确标记，是读源码比读论文多出来的信息。
- **子步长判定**（L3634–L3641）：`delta_tp = dzq/vt`，`nstep = INT(DT/delta_tp + 1)`——**一步里粒子能穿过几个层距**，超过 1 就分 nstep 个子步做沉降（CFL 类约束的沉降版）。五物种各自独立判定（`onstep(1..4)`）+ 云水不分子步）。`ksed1` 是各物种沉降的起始层（最上有水凝物

的层），其下不用白算。

- **云滴沉降**（L3646–L3667）：只算**地面 500 m 以内**（`hgt_agl > 500` 即止）且垂直速度 < 0.1 m/s 时——云滴沉降只在边界层云底附近有意义（雾滴沉降），这是显著的计算量裁剪。
- **雪落速**（L3705–L3740）：Field 双伽马谱的落速积分是两项之比 `(t1_vts+t2_vts)/(t3_vts+t4_vts)`（两个伽马项各按 mass/mom 积分）；`vts_boost`（淞附加成）在此乘入；**融化中的雪混叠**（L3722–L3724）：`SR = rs/(rs+rr)` 加权雪速与雨速——融雪的落速在雪雨之间连续过渡。
- **霰落速**（L3744–L3781）：hail-aware 用 Best–Reynolds 密度分档（`afall = a_coeff • (4 • ρg • 9.8/(3ρ))^b_coeff • visco^(1−2b)`），否则用老系数 `av_g_old/bv_g_old`；数浓度落速同样“数学正确 vs 物理调参”二选一（`mu_g=0` 时用 cgg(7)/cgg(12)）。

07a.8.17 沉降积分与瞬时相变

```
!file: phys/module_mp_thompson.F    L3784-L3967
L03784  !+---+-----------------------------------------------------------------+
L03785  !..Sedimentation of mixing ratio is the integral of v(D)*m(D)*N(D)*dD,
L03786  !.. whereas neglect m(D) term for number concentration.  Therefore,
L03787  !.. cloud ice has proper differential sedimentation.
L03788  !+---+-----------------------------------------------------------------+
L03789
L03790        if (ANY(L_qr .eqv. .true.)) then
L03791        nstep = NINT(1./onstep(1))
L03792        do n = 1, nstep
L03793           do k = kte, kts, -1
L03794              sed_r(k) = vtrk(k)*rr(k)
L03795              sed_n(k) = vtnrk(k)*nr(k)
L03796           enddo
L03797           k = kte
L03798           odzq = 1./dzq(k)
L03799           orho = 1./rho(k)
L03800           qrten(k) = qrten(k) - sed_r(k)*odzq*onstep(1)*orho
L03801           nrten(k) = nrten(k) - sed_n(k)*odzq*onstep(1)*orho
L03802           rr(k) = MAX(R1, rr(k) - sed_r(k)*odzq*DT*onstep(1))
L03803           nr(k) = MAX(R2, nr(k) - sed_n(k)*odzq*DT*onstep(1))
L03804           do k = ksed1(1), kts, -1
L03805              odzq = 1./dzq(k)
L03806              orho = 1./rho(k)
L03807              qrten(k) = qrten(k) + (sed_r(k+1)-sed_r(k))                &
L03808                                          *odzq*onstep(1)*orho
L03809              nrten(k) = nrten(k) + (sed_n(k+1)-sed_n(k))                &
L03810                                          *odzq*onstep(1)*orho
L03811              rr(k) = MAX(R1, rr(k) + (sed_r(k+1)-sed_r(k)) &
L03812                                          *odzq*DT*onstep(1))
L03813              nr(k) = MAX(R2, nr(k) + (sed_n(k+1)-sed_n(k)) &
L03814                                          *odzq*DT*onstep(1))
L03815           enddo
L03816
L03817           if (rr(kts).gt.R1*1000.) &
L03818           pptrain = pptrain + sed_r(kts)*DT*onstep(1)
L03819        enddo
L03820        endif
L03821
L03822  !+---+-----------------------------------------------------------------+
L03823
L03824        if (ANY(L_qc .eqv. .true.)) then
L03825        do k = kte, kts, -1
L03826           sed_c(k) = vtck(k)*rc(k)
L03827           sed_n(k) = vtnck(k)*nc(k)
L03828        enddo
L03829        do k = ksed1(5), kts, -1
L03830           odzq = 1./dzq(k)
L03831           orho = 1./rho(k)
L03832           qcten(k) = qcten(k) + (sed_c(k+1)-sed_c(k)) *odzq*orho
L03833           ncten(k) = ncten(k) + (sed_n(k+1)-sed_n(k)) *odzq*orho
L03834           rc(k) = MAX(R1, rc(k) + (sed_c(k+1)-sed_c(k)) *odzq*DT)
L03835           nc(k) = MAX(10., nc(k) + (sed_n(k+1)-sed_n(k)) *odzq*DT)
L03836        enddo
L03837        endif
L03838
L03839  !+---+-----------------------------------------------------------------+
L03840
L03841        if (ANY(L_qi .eqv. .true.)) then
L03842        nstep = NINT(1./onstep(2))
L03843        do n = 1, nstep
L03844           do k = kte, kts, -1
L03845              sed_i(k) = vtik(k)*ri(k)
L03846              sed_n(k) = vtnik(k)*ni(k)
L03847           enddo
L03848           k = kte
L03849           odzq = 1./dzq(k)
L03850           orho = 1./rho(k)
L03851           qiten(k) = qiten(k) - sed_i(k)*odzq*onstep(2)*orho
L03852           niten(k) = niten(k) - sed_n(k)*odzq*onstep(2)*orho
L03853           ri(k) = MAX(R1, ri(k) - sed_i(k)*odzq*DT*onstep(2))
L03854           ni(k) = MAX(R2, ni(k) - sed_n(k)*odzq*DT*onstep(2))
L03855           do k = ksed1(2), kts, -1
L03856              odzq = 1./dzq(k)
L03857              orho = 1./rho(k)
L03858              qiten(k) = qiten(k) + (sed_i(k+1)-sed_i(k))                &
L03859                                          *odzq*onstep(2)*orho
L03860              niten(k) = niten(k) + (sed_n(k+1)-sed_n(k))                &
L03861                                          *odzq*onstep(2)*orho
L03862              ri(k) = MAX(R1, ri(k) + (sed_i(k+1)-sed_i(k)) &
L03863                                          *odzq*DT*onstep(2))
L03864              ni(k) = MAX(R2, ni(k) + (sed_n(k+1)-sed_n(k)) &
L03865                                          *odzq*DT*onstep(2))
```

```
L03866           enddo
L03867
L03868           if (ri(kts).gt.R1*1000.) &
L03869           pptice = pptice + sed_i(kts)*DT*onstep(2)
L03870        enddo
L03871        endif
L03872
L03873  !+---+-----------------------------------------------------------------+
L03874
L03875        if (ANY(L_qs .eqv. .true.)) then
L03876        nstep = NINT(1./onstep(3))
L03877        do n = 1, nstep
L03878           do k = kte, kts, -1
L03879              sed_s(k) = vtsk(k)*rs(k)
L03880           enddo
L03881           k = kte
L03882           odzq = 1./dzq(k)
L03883           orho = 1./rho(k)
L03884           qsten(k) = qsten(k) - sed_s(k)*odzq*onstep(3)*orho
L03885           rs(k) = MAX(R1, rs(k) - sed_s(k)*odzq*DT*onstep(3))
L03886           do k = ksed1(3), kts, -1
L03887              odzq = 1./dzq(k)
L03888              orho = 1./rho(k)
L03889              qsten(k) = qsten(k) + (sed_s(k+1)-sed_s(k))                &
L03890                                          *odzq*onstep(3)*orho
L03891              rs(k) = MAX(R1, rs(k) + (sed_s(k+1)-sed_s(k)) &
L03892                                       *odzq*DT*onstep(3))
L03893           enddo
L03894
L03895           if (rs(kts).gt.R1*1000.) &
L03896           pptsnow = pptsnow + sed_s(kts)*DT*onstep(3)
L03897        enddo
L03898        endif
L03899
L03900  !+---+-----------------------------------------------------------------+
L03901
L03902        if (ANY(L_qg .eqv. .true.)) then
L03903        nstep = NINT(1./onstep(4))
L03904        do n = 1, nstep
L03905           do k = kte, kts, -1
L03906              sed_g(k) = vtgk(k)*rg(k)
L03907              sed_n(k) = vtngk(k)*ng(k)
L03908              sed_b(k) = vtgk(k)*rb(k)
L03909           enddo
L03910           k = kte
L03911           odzq = 1./dzq(k)
L03912           orho = 1./rho(k)
L03913           qgten(k) = qgten(k) - sed_g(k)*odzq*onstep(4)*orho
L03914           ngten(k) = ngten(k) - sed_n(k)*odzq*onstep(4)*orho
L03915           qbten(k) = qbten(k) - sed_b(k)*odzq*onstep(4)
L03916           rg(k) = MAX(R1, rg(k) - sed_g(k)*odzq*DT*onstep(4))
L03917           ng(k) = MAX(R2, ng(k) - sed_n(k)*odzq*DT*onstep(4))
L03918           rb(k) = MAX(R1/rho(k)/rho_g(NRHG), rb(k) - sed_b(k)*odzq*DT*onstep(4))
L03919           do k = ksed1(4), kts, -1
L03920              odzq = 1./dzq(k)
L03921              orho = 1./rho(k)
L03922              qgten(k) = qgten(k) + (sed_g(k+1)-sed_g(k))                &
L03923                                          *odzq*onstep(4)*orho
L03924              ngten(k) = ngten(k) + (sed_n(k+1)-sed_n(k))                &
L03925                                          *odzq*onstep(4)*orho
L03926              qbten(k) = qbten(k) + (sed_b(k+1)-sed_b(k))                &
L03927                                          *odzq*onstep(4)
L03928              rg(k) = MAX(R1, rg(k) + (sed_g(k+1)-sed_g(k)) &
L03929                                       *odzq*DT*onstep(4))
L03930              ng(k) = MAX(R2, ng(k) + (sed_n(k+1)-sed_n(k)) &
L03931                                       *odzq*DT*onstep(4))
L03932              rb(k) = MAX(rg(k)/rho(k)/rho_g(NRHG), rb(k) + (sed_b(k+1)-sed_b(k))  &
L03933                                       *odzq*DT*onstep(4))
L03934           enddo
L03935
L03936           if (rg(kts).gt.R1*1000.) &
L03937           pptgraul = pptgraul + sed_g(kts)*DT*onstep(4)
L03938        enddo
L03939        endif
L03940
L03941  !+---+-----------------------------------------------------------------+
L03942  !.. Instantly melt any cloud ice into cloud water if above 0C and
L03943  !.. instantly freeze any cloud water found below HGFR.
L03944  !+---+-----------------------------------------------------------------+
L03945        if (.not. iiwarm) then
L03946        do k = kts, kte
L03947           xri = MAX(0.0, qi1d(k) + qiten(k)*DT)
L03948           if ( (temp(k).gt. T_0) .and. (xri.gt. 0.0) ) then
L03949            qcten(k) = qcten(k) + xri*odt
L03950            ncten(k) = ncten(k) + ni1d(k)*odt
L03951            qiten(k) = qiten(k) - xri*odt
L03952            niten(k) = -ni1d(k)*odt
L03953            tten(k) = tten(k) - lfus*ocp(k)*xri*odt*(1-IFDRY)
L03954           endif
L03955
L03956           xrc = MAX(0.0, qc1d(k) + qcten(k)*DT)
L03957           if ( (temp(k).lt. HGFR) .and. (xrc.gt. 0.0) ) then
L03958            lfus2 = lsub - lvap(k)
L03959            xnc = nc1d(k) + ncten(k)*DT
L03960            qiten(k) = qiten(k) + xrc*odt
L03961            niten(k) = niten(k) + xnc*odt
L03962            qcten(k) = qcten(k) - xrc*odt
L03963            ncten(k) = ncten(k) - xnc*odt
L03964            tten(k) = tten(k) + lfus2*ocp(k)*xrc*odt*(1-IFDRY)
L03965           endif
L03966        enddo
L03967        endif
```

- **迎风格式沉降**（雨为例，L3790–L3820）：`sed_r(k) = vtrk(k)·rr(k)`（通量）→ 顶层特殊（只有出无进）→ 内层 `qten += (sed(k+1)−sed(k))·odzq·onstep·orho`——**通量形式一阶迎风**，物质守恒但扩散（教材 ch21 讲平流格式时的对照点）；nstep 子步内局部状态 `rr/nr` 同

步更新。**地面出流**（L3817–L3818）：最底层通量 × DT × onstep 累加成 `pptrain`——**这就是网格尺度降水的最终来源**（门槛 `R1 • 1000` 防微雨记账）。

- **云水沉降不分子步**（L3824–L3837）：云滴太慢，单步迎风即可。
- **霰沉降三通量**（L3902–L3939）：质量、数、**体积**（`sed_b = vtgk • rb`）三条通量并行——体积通量使密度档随沉降演化（大密度的先落地）。
- **瞬时相变**（L3941–L3967）：沉降后兜底——云冰在暖层瞬时变云水（吸热 `lfus`）、云水在 HGFR 以下瞬时变冰（放热 `lfus2`）。**为什么放在沉降后：**沉降可能把冰晶带进暖层/把云水带进极冷层，倾向段的相变判断已失效，此处按 “最终温度” 强制归位。

07a.8.18 最终回写

```
!file: phys/module_mp_thompson.F    L3969-L4082
L03969  !+---+-----------------------------------------------------------------+
L03970  !.. All tendencies computed, apply and pass back final values to parent.
L03971  !+---+-----------------------------------------------------------------+
L03972        do k = kts, kte
L03973           t1d(k)  = t1d(k) + tten(k)*DT
L03974           qv1d(k) = MAX(1.E-10, qv1d(k) + qvten(k)*DT)
L03975           qc1d(k) = qc1d(k) + qcten(k)*DT
L03976           nc1d(k) = MAX(2./rho(k), MIN(nc1d(k) + ncten(k)*DT, Nt_c_max))
L03977           if (is_aerosol_aware) then
L03978              if (aer_init_opt .lt. 2) then
L03979                 nwfa1d(k) = MAX(11.1E6, MIN(9999.E6,                    &
L03980                           (nwfa1d(k)+nwfaten(k)*DT)))
L03981                 nifa1d(k) = MAX(naIN1*0.01, MIN(9999.E6,                &
L03982                           (nifa1d(k)+nifaten(k)*DT)))
L03983                 if (wif_input_opt .eq. 2) then
L03984                    nbca1d(k) = MAX(5.55E6, MIN(9999.E6,                 &
L03985                              (nbca1d(k)+nbcaten(k)*DT)))
L03986                 else
L03987                    nbca1d(k) = 0.0
L03988                 endif
L03989              else
L03990                 nwfa1d(k) = MAX(0.0, (nwfa1d(k)+nwfaten(k)*DT))
L03991                 nifa1d(k) = MAX(0.0, (nifa1d(k)+nifaten(k)*DT))
L03992                 if (wif_input_opt .eq. 2) then
L03993                    nbca1d(k) = MAX(0.0, (nbca1d(k)+nbcaten(k)*DT))
L03994                 else
L03995                    nbca1d(k) = 0.0
L03996                 endif
L03997              endif
L03998           else
L03999              nwfa1d(k) = MAX(11.1E6, MIN(9999.E6,                       &
L04000                        (nwfa1d(k)+nwfaten(k)*DT)))
L04001              nifa1d(k) = MAX(naIN1*0.01, MIN(9999.E6,                   &
L04002                        (nifa1d(k)+nifaten(k)*DT)))
L04003              nbca1d(k) = MAX(5.55E6, MIN(9999.E6,                       &
L04004                        (nbca1d(k)+nbcaten(k)*DT)))
L04005           endif
L04006
L04007           if (qc1d(k) .le. R1) then
L04008             qc1d(k) = 0.0
L04009             nc1d(k) = 0.0
L04010           else
L04011             nu_c = MIN(15, NINT(1000.E6/(nc1d(k)*rho(k))) + 2)
L04012             lamc = (am_r*ccg(2,nu_c)*ocg1(nu_c)*nc1d(k)/qc1d(k))**obmr
L04013             xDc = (bm_r + nu_c + 1.) / lamc
L04014             if (xDc.lt. D0c) then
L04015              lamc = cce(2,nu_c)/D0c
L04016             elseif (xDc.gt. D0r*2.) then
L04017              lamc = cce(2,nu_c)/(D0r*2.)
L04018             endif
L04019             nc1d(k) = MIN(ccg(1,nu_c)*ocg2(nu_c)*qc1d(k)/am_r*lamc**bm_r,&
L04020                       DBLE(Nt_c_max)/rho(k))
L04021           endif
L04022
L04023           qi1d(k) = qi1d(k) + qiten(k)*DT
L04024           ni1d(k) = MAX(R2/rho(k), ni1d(k) + niten(k)*DT)
L04025           if (qi1d(k) .le. R1) then
L04026             qi1d(k) = 0.0
L04027             ni1d(k) = 0.0
L04028           else
L04029             lami = (am_i*cig(2)*oig1*ni1d(k)/qi1d(k))**obmi
L04030             ilami = 1./lami
L04031             xDi = (bm_i + mu_i + 1.) * ilami
L04032             if (xDi.lt. 5.E-6) then
L04033              lami = cie(2)/5.E-6
L04034             elseif (xDi.gt. 300.E-6) then
L04035              lami = cie(2)/300.E-6
L04036             endif
L04037             ni1d(k) = MIN(cig(1)*oig2*qi1d(k)/am_i*lami**bm_i,          &
L04038                       999.D3/rho(k))
L04039           endif
L04040           qr1d(k) = qr1d(k) + qrten(k)*DT
L04041           nr1d(k) = MAX(R2/rho(k), nr1d(k) + nrten(k)*DT)
L04042           if (qr1d(k) .le. R1) then
L04043             qr1d(k) = 0.0
L04044             nr1d(k) = 0.0
L04045           else
L04046             lamr = (am_r*crg(3)*org2*nr1d(k)/qr1d(k))**obmr
L04047             mvd_r(k) = (3.0 + mu_r + 0.672) / lamr
L04048             if (mvd_r(k) .gt. 2.5E-3) then
L04049                mvd_r(k) = 2.5E-3
L04050             elseif (mvd_r(k) .lt. D0r*0.75) then
L04051                mvd_r(k) = D0r*0.75
L04052             endif
L04053             lamr = (3.0 + mu_r + 0.672) / mvd_r(k)
L04054             nr1d(k) = crg(2)*org3*qr1d(k)*lamr**bm_r / am_r
```

```
L04055            endif
L04056            qs1d(k) = qs1d(k) + qsten(k)*DT
L04057            if (qs1d(k) .le. R1) qs1d(k) = 0.0
L04058            qg1d(k) = qg1d(k) + qgten(k)*DT
L04059            ng1d(k) = MAX(R2/rho(k), ng1d(k) + ngten(k)*DT)
L04060            if (qg1d(k) .le. R1) then
L04061              qg1d(k) = 0.0
L04062              ng1d(k) = 0.0
L04063              qb1d(k) = 0.0
L04064            else
L04065              qb1d(k) = MAX(qg1d(k)/rho_g(NRHG), qb1d(k) + qbten(k)*DT)
L04066              qb1d(k) = MIN(qg1d(k)/rho_g(1), qb1d(k))
L04067              idx_bg(k) = MAX(1,MIN(NINT(qg1d(k)/qb1d(k) *0.01)+1,NRHG))
L04068              if (.not. is_hail_aware) idx_bg(k) = idx_bg1
L04069              lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*ng1d(k)/qg1d(k))**obmg
L04070              mvd_g(k) = (3.0 + mu_g + 0.672) / lamg
L04071              if (mvd_g(k) .gt. 25.4E-3) then
L04072                 mvd_g(k) = 25.4E-3
L04073              elseif (mvd_g(k) .lt. D0r) then
L04074                 mvd_g(k) = D0r
L04075              endif
L04076              lamg = (3.0 + mu_g + 0.672) / mvd_g(k)
L04077              ng1d(k) = cgg(2,1)*ogg3*qg1d(k)*lamg**bm_g / am_g(idx_bg(k))
L04078            endif
L04079
L04080          enddo
L04081
L04082          end subroutine mp_thompson
```

- **倾向应用**（L3972–L4005）：`t1d += tten•DT`、`qv1d = MAX(1e-10, qv1d + qvten•DT)`……气溶胶按两套口径钳位（气候态下限 11.1e6 / GEOS-5 只钳非负）——**与列装入段完全对称的收尾**。
- **谱参数最终闭环**（L4007–L4078）：五物种按最终 (q, N) 重解 λ、钳位、**回写 N**（`nc1d = MIN(Γ积分, Nt_c_max/ρ)`）——注意这里 N 是**诊断重算**而不是用 `ncten` 累加的结果：保证“回出去的场”谱形自洽（driver 层不再改）。霰的 `idx_bg` 密度档也在此重定。
- **没有温度钳位**：温度倾向直接应用——微物理的潜热不设上限（饱和调整已保证凝结量有限），与辐射/边界层的温度处理一致。

07a.9 改造入口（→ 教材 ch20/ch21 案索引）

想改什么	动哪里	教材案
云滴数缺省（污染情景）	模块常数 `Nt_c`/`Nt_c_max`（本篇 §07a.4.1）	ch13 §13.8.4 闭合
谱形假设	`mu_r`/`mu_g`/`mu_i`、`am`/`bm` 族	—
冰核化灵敏度	`dustyIce`/`homogIce` 开关 + `iceDeMott`/`iceKoop`（code_07b）	ch13 §13.7.2
淞附/淞密度参数	主实体 `rime_dens` 两处（§07a.8.7）	—
潜热干跑实验	`IFDRY`（§07a.8.12）	—
新增水凝物物种	Registry 包 + `mp_gt_driver` 实参 + 主实体倾向组	ch20 案（T2 峰值修正、虚拟温度阈值）
输出诊断（qv 极值追踪等）	`mp_gt_driver` 极值追踪块（§07a.7.1）	ch20 案

07a.10 上下游链接

- **下游（本模块内）**：code_07b——辅助子程序族（建表、Γ 函数、饱和比、冰核化物理、有效半径、反射率、Bolton 热力学函数）。
- **上游**：code_21——`module_microphysics_driver.F`（`micro_select` 分发、THOMPSON CASE 实参装配、倾向回写主环）。
- **底层**：code_01——`share/module_model_constants.F`（`RE_QC_BG` 族常数）。
- **教材**：ch11（饱和水汽压公式族、气溶胶）、ch13（Thompson 精读与热力学对应）、ch12（有效半径接口的辐射侧）、ch20/ch21（改造案例与 solve_em 调用位）。

文库 code_07b：module_mp_thompson.F 下半——建表族、物理函数族与诊断接口

07b.0 版本指纹与阅读说明

- **源文件**：`phys/module_mp_thompson.F`（WRF 4.8.0）；本篇覆盖 L4091–L6192——碰撞/冻结/活化**建表族**（10 个）、不完全 Γ 函数族（5 个）、饱和比函数（2 个）、冰核化物理函数（3 个）、辐射与雷达接口（2 个）、Bolton 热力学函数族（5 个），共 27 个子程序/函数。模块声明、`thompson_init`、`mp_gt_driver` 与主实体 `mp_thompson` 见 code_07a。
- **保真承诺**：同 code_07a——所有代码块由 `code_extract.py` 按字节抽取并附 md5 指纹，`verify` 可复核。
- **符号口径**：同 code_07a。
- **阅读方式**：本篇是 “工具箱” 性质，按需查阅；但 §07b.2 建表族建议顺序读——它们的表结构在 code_07a 主实体的用法里已反复出现。

07b.1 建表族：离线算什么、在线省什么

主实体的速率方程处处是 ∫∫(碰撞截面)·|v 差 |·m(D)·N(D)dD 型二重积分。对雨-雪、雨-霰这类**双方都有复杂谱形且相对速度可正可负**的组合，解析积分不存在或误差大——Thompson 的对策是把二重积分在初始化时按谱参数网格**离线积一遍存表**，运行时只查表。本节逐个走读 10 个建表子程序。

07b.2.1 `qr_acr_qg`：雨-霰碰撞并合五维表

```
!file: phys/module_mp_thompson.F    L4091-L4184
L04091        subroutine qr_acr_qg(NRHGtable)
L04092
L04093        USE module_timing
L04094        implicit none
L04095
L04096        INTEGER, INTENT(IN) ::NRHGtable
L04097
L04098  !..Local variables
L04099        INTEGER:: i, j, k, m, n, n2, n3, idx_bg
L04100        INTEGER:: km, km_s, km_e
L04101        DOUBLE PRECISION, DIMENSION(nbg):: N_g
L04102        DOUBLE PRECISION, DIMENSION(nbg,NRHGtable):: vg
L04103        DOUBLE PRECISION, DIMENSION(nbr):: vr, N_r
L04104        DOUBLE PRECISION:: N0_r, N0_g, lam_exp, lamg, lamr
L04105        DOUBLE PRECISION:: massg, massr, dvg, dvr, t1, t2, z1, z2, y1, y2
L04106        LOGICAL force_read_thompson, write_thompson_tables, write_thompson_mp38table
L04107        LOGICAL lexist,lopen
L04108        INTEGER good
L04109        LOGICAL, EXTERNAL :: wrf_dm_on_monitor
L04110        CHARACTER (LEN=20):: qr_acr_qg_name
L04111  !+---+
L04112
L04113        CALL nl_get_force_read_thompson(1,force_read_thompson)
L04114        CALL nl_get_write_thompson_tables(1,write_thompson_tables)
L04115        CALL nl_get_write_thompson_mp38table(1,write_thompson_mp38table)
L04116
L04117        if (is_hail_aware) then
L04118           ! Table for mp=38
L04119           ! V1: new table dimension (NRHG), computes all, rho_g(1:NRHG)
L04120           qr_acr_qg_name="qr_acr_qg_mp38V1.dat"
L04121        else
L04122           ! Table for mp=8 or mp=28
L04123           ! V4: new table dimension (NRHG), computes only rho_g(idx_bg1)
L04124           qr_acr_qg_name="qr_acr_qg_V4.dat"
L04125     endif
L04126
L04127        good = 0
L04128        IF ( wrf_dm_on_monitor() ) THEN
L04129          INQUIRE(FILE=qr_acr_qg_name,EXIST=lexist)
L04130          IF ( lexist ) THEN
L04131            CALL wrf_message("ThompMP: read "//qr_acr_qg_name//" instead of computing")
L04132            OPEN(63,file=qr_acr_qg_name,form="unformatted",err=1234)
L04133            READ(63,err=1234) tcg_racg
L04134            READ(63,err=1234) tmr_racg
```

```
L04135            READ(63,err=1234) tcr_gacr
L04136            !READ(63,err=1234) tmg_gacr
L04137            READ(63,err=1234) tnr_racg
L04138            READ(63,err=1234) tnr_gacr
L04139            good = 1
L04140     1234   CONTINUE
L04141            IF ( good .NE. 1 ) THEN
L04142              INQUIRE(63,opened=lopen)
L04143              IF (lopen) THEN
L04144                IF( force_read_thompson ) THEN
L04145                  CALL wrf_error_fatal("Error reading "//qr_acr_qg_name//". Aborting because force_read_thompson is
↪   .true.")
L04146                ENDIF
L04147                CLOSE(63)
L04148              ELSE
L04149                IF( force_read_thompson ) THEN
L04150                  CALL wrf_error_fatal("Error opening "//qr_acr_qg_name//". Aborting because force_read_thompson is
↪   .true.")
L04151                ENDIF
L04152              ENDIF
L04153            ELSE
L04154              INQUIRE(63,OPENED=lopen)
L04155              IF (lopen) THEN
L04156                CLOSE(63)
L04157              ENDIF
L04158            ENDIF
L04159          ELSE
L04160            IF( force_read_thompson ) THEN
L04161              CALL wrf_error_fatal("Non-existent "//qr_acr_qg_name//". Aborting because force_read_thompson is .true.")
L04162            ENDIF
L04163            IF( (is_hail_aware) .AND. (.NOT. write_thompson_mp38table)) THEN
L04164              CALL wrf_error_fatal("Non-existent "//qr_acr_qg_name//". Aborting because write_thompson_mp38table is
↪   .false.")
L04165            ENDIF
L04166          ENDIF
L04167        ENDIF
L04168  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04169        CALL wrf_dm_bcast_integer(good,1)
L04170  #endif
L04171
L04172        IF ( good .EQ. 1 ) THEN
L04173  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04174          CALL wrf_dm_bcast_double(tcg_racg,SIZE(tcg_racg))
L04175          CALL wrf_dm_bcast_double(tmr_racg,SIZE(tmr_racg))
L04176          CALL wrf_dm_bcast_double(tcr_gacr,SIZE(tcr_gacr))
L04177          !CALL wrf_dm_bcast_double(tmg_gacr,SIZE(tmg_gacr))
L04178          CALL wrf_dm_bcast_double(tnr_racg,SIZE(tnr_racg))
L04179          CALL wrf_dm_bcast_double(tnr_gacr,SIZE(tnr_gacr))
L04180  #endif
L04181        ELSE
L04182          CALL wrf_message("ThompMP: computing qr_acr_qg table: "//qr_acr_qg_name)
L04183  !+---+-----------------------------------------------------------------+
L04184          CALL start_timing
```

- **namelist 三开关**（`nl_get_*` 是 Registry 生成的 namelist 读取例程，对应 `&physics` 里的 `force_read_thompson`、`write_thompson_tables`、`write_thompson_mp38table`——namelist 全表见教材 ch13 §13.6.4）：`force_read_thompson=.true.` 时表文件缺失/损坏**直接 fatal**（业务运行锁定表版本用）；`write_thompson_tables=.true.` 时算完写盘供下次读。
- **表文件缓存**：hail-aware 用 `qr_acr_qg_mp38V1.dat`（mp_physics=38，九档密度全算），否则 `qr_acr_qg_V4.dat`（只算 500 kg/m³ 档）。**表名带版本号是刻意的防呆**——谱参数改了而旧表还在，读旧表就是静默错误。
- **并行建表纪律**：`wrf_dm_on_monitor()` 判定只有 monitor 进程读文件/算表，结果用 `wrf_dm_bcast_*` 广播全体——27 张大表若每进程各算一遍既慢又可能因浮点求和顺序不同而**进程间不一致**。
- `USE module_timing` 在这里（code_07a §07a.3 说的 “藏在子程序内部” 的依赖）。

```
!file: phys/module_mp_thompson.F    L4185-L4305
L04185
L04186          do n2 = 1, nbr
L04187  !        vr(n2) = av_r*Dr(n2)**bv_r * DEXP(-fv_r*Dr(n2))
L04188           vr(n2) = -0.1021 + 4.932E3*Dr(n2) - 0.9551E6*Dr(n2)*Dr(n2)     &
L04189                + 0.07934E9*Dr(n2)*Dr(n2)*Dr(n2)                          &
L04190                - 0.002362E12*Dr(n2)*Dr(n2)*Dr(n2)*Dr(n2)
L04191          enddo
L04192          do n3 = 1, NRHGtable
L04193          do n = 1, nbg
L04194           ! idx_bg indexes module coefficients, not vg
L04195           if (.not. is_hail_aware) idx_bg = idx_bg1
L04196           if (is_hail_aware) idx_bg = n3
L04197           vg(n,n3) = av_g(idx_bg)*Dg(n)**bv_g(idx_bg)
L04198          enddo
L04199          enddo
L04200
L04201  !..Note values returned from wrf_dm_decomp1d are zero-based, add 1 for
L04202  !.. fortran indices.  J. Michalakes, 2009Oct30.
L04203
L04204  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04205          CALL wrf_dm_decomp1d ( ntb_r*ntb_r1, km_s, km_e )
L04206  #else
L04207          km_s = 0
L04208          km_e = ntb_r*ntb_r1 - 1
L04209  #endif
L04210
L04211          do km = km_s, km_e
L04212           m = km / ntb_r1 + 1
L04213           k = mod( km , ntb_r1 ) + 1
L04214
```

```
L04215           lam_exp = (N0r_exp(k)*am_r*crg(1)/r_r(m))**ore1
L04216           lamr = lam_exp * (crg(3)*org2*org1)**obmr
L04217           N0_r = N0r_exp(k)/(crg(2)*lam_exp) * lamr**cre(2)
L04218           do n2 = 1, nbr
L04219              N_r(n2) = N0_r*Dr(n2)**mu_r *DEXP(-lamr*Dr(n2))*dtr(n2)
L04220           enddo
L04221
L04222           do n3 = 1, NRHGtable
L04223            if (.not. is_hail_aware) idx_bg = idx_bg1
L04224            if (is_hail_aware) idx_bg = n3
L04225
L04226           do j = 1, ntb_g
L04227           do i = 1, ntb_g1
L04228              lam_exp = (N0g_exp(i)*am_g(idx_bg)*cgg(1,1)/r_g(j))**oge1
L04229              lamg = lam_exp * (cgg(3,1)*ogg2*ogg1)**obmg
L04230              N0_g = N0g_exp(i)/(cgg(2,1)*lam_exp) * lamg**cge(2,1)
L04231              do n = 1, nbg
L04232                 N_g(n) = N0_g*Dg(n)**mu_g * DEXP(-lamg*Dg(n))*dtg(n)
L04233              enddo
L04234
L04235              t1 = 0.0d0
L04236              t2 = 0.0d0
L04237              z1 = 0.0d0
L04238              z2 = 0.0d0
L04239              y1 = 0.0d0
L04240              y2 = 0.0d0
L04241              do n2 = 1, nbr
L04242                 massr = am_r * Dr(n2)**bm_r
L04243                 do n = 1, nbg
L04244                    massg = am_g(idx_bg) * Dg(n)**bm_g
L04245
L04246                    dvg = 0.5d0*((vr(n2) - vg(n,n3)) + DABS(vr(n2)-vg(n,n3)))
L04247                    dvr = 0.5d0*((vg(n,n3) - vr(n2)) + DABS(vg(n,n3)-vr(n2)))
L04248
L04249                    t1 = t1+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04250                        *dvg*massg * N_g(n)* N_r(n2)
L04251                    z1 = z1+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04252                        *dvg*massr * N_g(n)* N_r(n2)
L04253                    y1 = y1+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04254                        *dvg       * N_g(n)* N_r(n2)
L04255
L04256                    t2 = t2+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04257                        *dvr*massr * N_g(n)* N_r(n2)
L04258                    y2 = y2+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04259                        *dvr       * N_g(n)* N_r(n2)
L04260                    z2 = z2+ PI*.25*Ef_rg*(Dg(n)+Dr(n2))*(Dg(n)+Dr(n2)) &
L04261                        *dvr*massg * N_g(n)* N_r(n2)
L04262                 enddo
L04263  97             continue
L04264              enddo
L04265              tcg_racg(i,j,n3,k,m) = t1
L04266              tmr_racg(i,j,n3,k,m) = DMIN1(z1, r_r(m)*1.0d0)
L04267              tcr_gacr(i,j,n3,k,m) = t2
L04268              !tmg_gacr(i,j,n3,k,m) = DMIN1(z2, r_g(j)*1.0d0)
L04269              tnr_racg(i,j,n3,k,m) = y1
L04270              tnr_gacr(i,j,n3,k,m) = y2
L04271           enddo
L04272           enddo
L04273           enddo
L04274          enddo
L04275
L04276  !..Note wrf_dm_gatherv expects zero-based km_s, km_e (J. Michalakes, 2009Oct30).
L04277
L04278  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04279          CALL wrf_dm_gatherv(tcg_racg, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04280          CALL wrf_dm_gatherv(tmr_racg, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04281          CALL wrf_dm_gatherv(tcr_gacr, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04282         !CALL wrf_dm_gatherv(tmg_gacr, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04283          CALL wrf_dm_gatherv(tnr_racg, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04284          CALL wrf_dm_gatherv(tnr_gacr, ntb_g*ntb_g1*NRHGtable, km_s, km_e, R8SIZE)
L04285  #endif
L04286          CALL end_timing ( TRIM("table computation "//qr_acr_qg_name//"") )
L04287  !+---+-----------------------------------------------------------------+
L04288
L04289
L04290          IF ( write_thompson_tables .AND. wrf_dm_on_monitor() ) THEN
L04291            CALL wrf_message("Writing "//qr_acr_qg_name//" in Thompson MP init")
L04292            OPEN(63,file=qr_acr_qg_name,form="unformatted",err=9234)
L04293            WRITE(63,err=9234) tcg_racg
L04294            WRITE(63,err=9234) tmr_racg
L04295            WRITE(63,err=9234) tcr_gacr
L04296            !WRITE(63,err=9234) tmg_gacr
L04297            WRITE(63,err=9234) tnr_racg
L04298            WRITE(63,err=9234) tnr_gacr
L04299            CLOSE(63)
L04300            RETURN    ! ----- RETURN
L04301  9234      CONTINUE
L04302            CALL wrf_error_fatal("Error writing "//qr_acr_qg_name//"")
L04303          ENDIF
L04304        ENDIF
L04305
```

- **雨落速是四阶多项式**（L4187 起）：`vr = -0.1021 + 4.932e3*D - 0.9551e6*D² + 0.07934e9*D³ - 0.002362e12*D⁴`——**不是**模块常数段的 `av_r * D^bv * exp(-fv * D)` 幂律！建表用更高精度的多项式落速（幂律是大滴端压制近似），被 ! 掉的幂律行是历史痕迹。同一个文件里两套落速并存，改落速参数时**必须两处同步**——改造陷阱。
- **方向判定用半绝对值**：`dvg = 0.5 * (Δv+|Δv|)`（Δv=vr−vg）取正部、`dvr` 取反部——雨比霰快则雨收霰（dvg>0 计入 t1/z1/y1 组），霰比雨快则霰收雨（dvr>0 计入 t2/z2/y2 组）。**一张表同时存双向碰撞**。
- **五张表的账目**（L4264–L4269）：`tcg_racg`=霰收雨的霰质量增；`tmr_racg`=雨质量的转移量（**DMIN1 钳位不超雨含量 r_r(m)**——质量账封顶）；`tcr_gacr`=霰收雨时雨的损失；`tnr_racg/`

tnr_gacr=数浓度转移（不带质量权重）。主实体 L2526–L2531 把它们组合成 prg_rcg/prr_rcg/pnr_rcg。

- **积分核**：π/4 • (Dg+Dr)² • Δv • m(D) • N(D)——圆柱碰并截面（直径和为碰撞直径）× 相对速度 × 质量 × 双方谱密度；N_g/N_r 是各 bin 的谱密度（N0 • D^mu • exp(-λD) • dtD）。
- wrf_dm_decomp1d/gatherv 把 (ntb_r×ntb_r1) 个 (m,k) 组合均分给各进程（Michalakes 2009 注释：**返回值零基**，Fortran 侧加 1——并行改造里最容易踩的索引坑，注释连年份都写了）。

07b.2.2 qr_acr_qs：雨-雪碰撞四维表（12 张小表）

```
!file: phys/module_mp_thompson.F    L4312-L4400
L04312
L04313        subroutine qr_acr_qs
L04314
L04315        implicit none
L04316
L04317  !..Local variables
L04318        INTEGER:: i, j, k, m, n, n2
L04319        INTEGER:: km, km_s, km_e
L04320        DOUBLE PRECISION, DIMENSION(nbr):: vr, D1, N_r
L04321        DOUBLE PRECISION, DIMENSION(nbs):: vs, N_s
L04322        DOUBLE PRECISION:: loga_, a_, b_, second, M0, M2, M3, Mrat, oM3
L04323        DOUBLE PRECISION:: N0_r, lam_exp, lamr, slam1, slam2
L04324        DOUBLE PRECISION:: dvs, dvr, masss, massr
L04325        DOUBLE PRECISION:: t1, t2, t3, t4, z1, z2, z3, z4
L04326        DOUBLE PRECISION:: y1, y2, y3, y4
L04327        LOGICAL force_read_thompson, write_thompson_tables
L04328        LOGICAL lexist,lopen
L04329        INTEGER good
L04330        LOGICAL, EXTERNAL :: wrf_dm_on_monitor
L04331
L04332  !+---+
L04333
L04334        CALL nl_get_force_read_thompson(1,force_read_thompson)
L04335        CALL nl_get_write_thompson_tables(1,write_thompson_tables)
L04336
L04337        good = 0
L04338        IF ( wrf_dm_on_monitor() ) THEN
L04339          INQUIRE(FILE="qr_acr_qsV2.dat",EXIST=lexist)
L04340          IF ( lexist ) THEN
L04341            CALL wrf_message("ThompMP: read qr_acr_qsV2.dat instead of computing")
L04342            OPEN(63,file="qr_acr_qsV2.dat",form="unformatted",err=1234)
L04343            READ(63,err=1234)tcs_racs1
L04344            READ(63,err=1234)tmr_racs1
L04345            READ(63,err=1234)tcs_racs2
L04346            READ(63,err=1234)tmr_racs2
L04347            READ(63,err=1234)tcr_sacr1
L04348            READ(63,err=1234)tms_sacr1
L04349            READ(63,err=1234)tcr_sacr2
L04350            READ(63,err=1234)tms_sacr2
L04351            READ(63,err=1234)tnr_racs1
L04352            READ(63,err=1234)tnr_racs2
L04353            READ(63,err=1234)tnr_sacr1
L04354            READ(63,err=1234)tnr_sacr2
L04355            good = 1
L04356   1234     CONTINUE
L04357            IF ( good .NE. 1 ) THEN
L04358              INQUIRE(63,opened=lopen)
L04359              IF (lopen) THEN
L04360                IF( force_read_thompson ) THEN
L04361                  CALL wrf_error_fatal("Error reading qr_acr_qsV2.dat. Aborting because force_read_thompson is .true.")
L04362                ENDIF
L04363                CLOSE(63)
L04364              ELSE
L04365                IF( force_read_thompson ) THEN
L04366                  CALL wrf_error_fatal("Error opening qr_acr_qsV2.dat. Aborting because force_read_thompson is .true.")
L04367                ENDIF
L04368              ENDIF
L04369            ELSE
L04370              INQUIRE(63,opened=lopen)
L04371              IF (lopen) THEN
L04372                CLOSE(63)
L04373              ENDIF
L04374            ENDIF
L04375          ELSE
L04376            IF( force_read_thompson ) THEN
L04377              CALL wrf_error_fatal("Non-existent qr_acr_qsV2.dat. Aborting because force_read_thompson is .true.")
L04378            ENDIF
L04379          ENDIF
L04380        ENDIF
L04381  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04382        CALL wrf_dm_bcast_integer(good,1)
L04383  #endif
L04384
L04385        IF ( good .EQ. 1 ) THEN
L04386  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04387          CALL wrf_dm_bcast_double(tcs_racs1,SIZE(tcs_racs1))
L04388          CALL wrf_dm_bcast_double(tmr_racs1,SIZE(tmr_racs1))
L04389          CALL wrf_dm_bcast_double(tcs_racs2,SIZE(tcs_racs2))
L04390          CALL wrf_dm_bcast_double(tmr_racs2,SIZE(tmr_racs2))
L04391          CALL wrf_dm_bcast_double(tcr_sacr1,SIZE(tcr_sacr1))
L04392          CALL wrf_dm_bcast_double(tms_sacr1,SIZE(tms_sacr1))
L04393          CALL wrf_dm_bcast_double(tcr_sacr2,SIZE(tcr_sacr2))
L04394          CALL wrf_dm_bcast_double(tms_sacr2,SIZE(tms_sacr2))
L04395          CALL wrf_dm_bcast_double(tnr_racs1,SIZE(tnr_racs1))
L04396          CALL wrf_dm_bcast_double(tnr_racs2,SIZE(tnr_racs2))
L04397          CALL wrf_dm_bcast_double(tnr_sacr1,SIZE(tnr_sacr1))
L04398          CALL wrf_dm_bcast_double(tnr_sacr2,SIZE(tnr_sacr2))
L04399  #endif
L04400        ELSE
```

与 qr_acr_qg 完全同构的 “读缓存 → 广播 → 否则计算” 框架；表名 qr_acr_qsV2.dat。四维 (雪含量，温度，雨截距，雨含量)——雪没有数浓度预报量，温度维（ntb_t=9，0 到 −40 °C 每 5 度）替代了霰表的截距维。**12 张表 = 12 个 “来源 × 去向” 账本**，主实体 L2488–L2508 按湿球温度分叉组合。

```
!file: phys/module_mp_thompson.F    L4401-L4590
L04401          CALL wrf_message("ThompMP: computing qr_acr_qs")
L04402          do n2 = 1, nbr
L04403  !        vr(n2) = av_r*Dr(n2)**bv_r * DEXP(-fv_r*Dr(n2))
L04404           vr(n2) = -0.1021 + 4.932E3*Dr(n2) - 0.9551E6*Dr(n2)*Dr(n2)     &
L04405                  + 0.07934E9*Dr(n2)*Dr(n2)*Dr(n2)                        &
L04406                  - 0.002362E12*Dr(n2)*Dr(n2)*Dr(n2)*Dr(n2)
L04407           D1(n2) = (vr(n2)/av_s)**(1./bv_s)
L04408          enddo
L04409          do n = 1, nbs
L04410           vs(n) = 1.5*av_s*Ds(n)**bv_s * DEXP(-fv_s*Ds(n))
L04411          enddo
L04412
L04413  !..Note values returned from wrf_dm_decomp1d are zero-based, add 1 for
L04414  !.. fortran indices.  J. Michalakes, 2009Oct30.
L04415
L04416  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04417          CALL wrf_dm_decomp1d ( ntb_r*ntb_r1, km_s, km_e )
L04418  #else
L04419          km_s = 0
L04420          km_e = ntb_r*ntb_r1 - 1
L04421  #endif
L04422
L04423          do km = km_s, km_e
L04424           m = km / ntb_r1 + 1
L04425           k = mod( km , ntb_r1 ) + 1
L04426
L04427           lam_exp = (N0r_exp(k)*am_r*crg(1)/r_r(m))**ore1
L04428           lamr = lam_exp * (crg(3)*org2*org1)**obmr
L04429           N0_r = N0r_exp(k)/(crg(2)*lam_exp) * lamr**cre(2)
L04430           do n2 = 1, nbr
L04431              N_r(n2) = N0_r*Dr(n2)**mu_r * DEXP(-lamr*Dr(n2))*dtr(n2)
L04432           enddo
L04433
L04434           do j = 1, ntb_t
L04435              do i = 1, ntb_s
L04436
L04437  !..From the bm_s moment, compute plus one moment.  If we are not
L04438  !.. using bm_s=2, then we must transform to the pure 2nd moment
L04439  !.. (variable called "second") and then to the bm_s+1 moment.
L04440
L04441                 M2 = r_s(i)*oams *1.0d0
L04442                 if (bm_s.gt.2.0-1.E-3 .and. bm_s.lt.2.0+1.E-3) then
L04443                    loga_ = sa(1) + sa(2)*Tc(j) + sa(3)*bm_s &
L04444                       + sa(4)*Tc(j)*bm_s + sa(5)*Tc(j)*Tc(j) &
L04445                       + sa(6)*bm_s*bm_s + sa(7)*Tc(j)*Tc(j)*bm_s &
L04446                       + sa(8)*Tc(j)*bm_s*bm_s + sa(9)*Tc(j)*Tc(j)*Tc(j) &
L04447                       + sa(10)*bm_s*bm_s*bm_s
L04448                    a_ = 10.0**loga_
L04449                    b_ = sb(1) + sb(2)*Tc(j) + sb(3)*bm_s &
L04450                       + sb(4)*Tc(j)*bm_s + sb(5)*Tc(j)*Tc(j) &
L04451                       + sb(6)*bm_s*bm_s + sb(7)*Tc(j)*Tc(j)*bm_s &
L04452                       + sb(8)*Tc(j)*bm_s*bm_s + sb(9)*Tc(j)*Tc(j)*Tc(j) &
L04453                       + sb(10)*bm_s*bm_s*bm_s
L04454                    second = (M2/a_)**(1./b_)
L04455                 else
L04456                    second = M2
L04457                 endif
L04458
L04459                 loga_ = sa(1) + sa(2)*Tc(j) + sa(3)*cse(1) &
L04460                    + sa(4)*Tc(j)*cse(1) + sa(5)*Tc(j)*Tc(j) &
L04461                    + sa(6)*cse(1)*cse(1) + sa(7)*Tc(j)*Tc(j)*cse(1) &
L04462                    + sa(8)*Tc(j)*cse(1)*cse(1) + sa(9)*Tc(j)*Tc(j)*Tc(j) &
L04463                    + sa(10)*cse(1)*cse(1)*cse(1)
L04464                 a_ = 10.0**loga_
L04465                 b_ = sb(1)+sb(2)*Tc(j)+sb(3)*cse(1) + sb(4)*Tc(j)*cse(1) &
L04466                    + sb(5)*Tc(j)*Tc(j) + sb(6)*cse(1)*cse(1) &
L04467                    + sb(7)*Tc(j)*Tc(j)*cse(1) + sb(8)*Tc(j)*cse(1)*cse(1) &
L04468                    + sb(9)*Tc(j)*Tc(j)*Tc(j)+sb(10)*cse(1)*cse(1)*cse(1)
L04469                 M3 = a_ * second**b_
L04470
L04471                 oM3 = 1./M3
L04472                 Mrat = M2*(M2*oM3)*(M2*oM3)*(M2*oM3)
L04473                 M0   = (M2*oM3)**mu_s
L04474                 slam1 = M2 * oM3 * Lam0
L04475                 slam2 = M2 * oM3 * Lam1
L04476
L04477                 do n = 1, nbs
L04478                    N_s(n) = Mrat*(Kap0*DEXP(-slam1*Ds(n)) &
L04479                       + Kap1*M0*Ds(n)**mu_s * DEXP(-slam2*Ds(n)))*dts(n)
L04480                 enddo
L04481
L04482                 t1 = 0.0d0
L04483                 t2 = 0.0d0
L04484                 t3 = 0.0d0
L04485                 t4 = 0.0d0
L04486                 z1 = 0.0d0
L04487                 z2 = 0.0d0
L04488                 z3 = 0.0d0
L04489                 z4 = 0.0d0
L04490                 y1 = 0.0d0
L04491                 y2 = 0.0d0
L04492                 y3 = 0.0d0
L04493                 y4 = 0.0d0
L04494                 do n2 = 1, nbr
L04495                    massr = am_r * Dr(n2)**bm_r
L04496                    do n = 1, nbs
L04497                       masss = am_s * Ds(n)**bm_s
L04498
L04499                       dvs = 0.5d0*((vr(n2) - vs(n)) + DABS(vr(n2)-vs(n)))
```

```
L04500                      dvr = 0.5d0*((vs(n) - vr(n2)) + DABS(vs(n)-vr(n2)))
L04501
L04502                      if (massr .gt. 1.5*masss) then
L04503                      t1 = t1+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04504                          *dvs*masss * N_s(n)* N_r(n2)
L04505                      z1 = z1+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04506                          *dvs*massr * N_s(n)* N_r(n2)
L04507                      y1 = y1+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04508                          *dvs       * N_s(n)* N_r(n2)
L04509                      else
L04510                      t3 = t3+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04511                          *dvs*masss * N_s(n)* N_r(n2)
L04512                      z3 = z3+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04513                          *dvs*massr * N_s(n)* N_r(n2)
L04514                      y3 = y3+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04515                          *dvs       * N_s(n)* N_r(n2)
L04516                      endif
L04517
L04518                      if (massr .gt. 1.5*masss) then
L04519                      t2 = t2+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04520                          *dvr*massr * N_s(n)* N_r(n2)
L04521                      y2 = y2+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04522                          *dvr       * N_s(n)* N_r(n2)
L04523                      z2 = z2+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04524                          *dvr*masss * N_s(n)* N_r(n2)
L04525                      else
L04526                      t4 = t4+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04527                          *dvr*massr * N_s(n)* N_r(n2)
L04528                      y4 = y4+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04529                          *dvr       * N_s(n)* N_r(n2)
L04530                      z4 = z4+ PI*.25*Ef_rs*(Ds(n)+Dr(n2))*(Ds(n)+Dr(n2)) &
L04531                          *dvr*masss * N_s(n)* N_r(n2)
L04532                      endif
L04533
L04534                    enddo
L04535                  enddo
L04536                  tcs_racs1(i,j,k,m) = t1
L04537                  tmr_racs1(i,j,k,m) = DMIN1(z1, r_r(m)*1.0d0)
L04538                  tcs_racs2(i,j,k,m) = t3
L04539                  tmr_racs2(i,j,k,m) = z3
L04540                  tcr_sacr1(i,j,k,m) = t2
L04541                  tms_sacr1(i,j,k,m) = z2
L04542                  tcr_sacr2(i,j,k,m) = t4
L04543                  tms_sacr2(i,j,k,m) = z4
L04544                  tnr_racs1(i,j,k,m) = y1
L04545                  tnr_racs2(i,j,k,m) = y3
L04546                  tnr_sacr1(i,j,k,m) = y2
L04547                  tnr_sacr2(i,j,k,m) = y4
L04548                enddo
L04549              enddo
L04550            enddo
L04551
L04552  !..Note wrf_dm_gatherv expects zero-based km_s, km_e (J. Michalakes, 2009Oct30).
L04553
L04554  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04555            CALL wrf_dm_gatherv(tcs_racs1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04556            CALL wrf_dm_gatherv(tmr_racs1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04557            CALL wrf_dm_gatherv(tcs_racs2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04558            CALL wrf_dm_gatherv(tmr_racs2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04559            CALL wrf_dm_gatherv(tcr_sacr1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04560            CALL wrf_dm_gatherv(tms_sacr1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04561            CALL wrf_dm_gatherv(tcr_sacr2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04562            CALL wrf_dm_gatherv(tms_sacr2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04563            CALL wrf_dm_gatherv(tnr_racs1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04564            CALL wrf_dm_gatherv(tnr_racs2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04565            CALL wrf_dm_gatherv(tnr_sacr1, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04566            CALL wrf_dm_gatherv(tnr_sacr2, ntb_s*ntb_t, km_s, km_e, R8SIZE)
L04567  #endif
L04568
L04569            IF ( write_thompson_tables .AND. wrf_dm_on_monitor() ) THEN
L04570              CALL wrf_message("Writing qr_acr_qsV2.dat in Thompson MP init")
L04571              OPEN(63,file="qr_acr_qsV2.dat",form="unformatted",err=9234)
L04572              WRITE(63,err=9234)tcs_racs1
L04573              WRITE(63,err=9234)tmr_racs1
L04574              WRITE(63,err=9234)tcs_racs2
L04575              WRITE(63,err=9234)tmr_racs2
L04576              WRITE(63,err=9234)tcr_sacr1
L04577              WRITE(63,err=9234)tms_sacr1
L04578              WRITE(63,err=9234)tcr_sacr2
L04579              WRITE(63,err=9234)tms_sacr2
L04580              WRITE(63,err=9234)tnr_racs1
L04581              WRITE(63,err=9234)tnr_racs2
L04582              WRITE(63,err=9234)tnr_sacr1
L04583              WRITE(63,err=9234)tnr_sacr2
L04584              CLOSE(63)
L04585              RETURN    ! ----- RETURN
L04586   9234       CONTINUE
L04587              CALL wrf_error_fatal("Error writing qr_acr_qsV2.dat")
L04588            ENDIF
L04589          ENDIF
L04590
```

- **质量分界 1.5**（L4501/L4517）：`massr > 1.5 • masss` 时雨是收集者（t1/z1/y1 组存 `*racs1`），否则雪是收集者（t3/z3/y3 组存 `*racs2`）——**谁大谁收**的物理直觉直接编码为表的分列存储。
- **雪落速乘 1.5**（L4409）：`vs = 1.5 • av_s • D^bv • exp(−fv • D)`——**源码无注释解释**。可对照的线索：主实体沉降用的雪落速是矩量积分（无此系数），此处 1.5 只影响碰撞方向判定。读代码读到这种 “裸经验系数” 时，文库的态度是标注出来而非硬解释——改造者若动雪落速，此处是否跟随是必须自查的问题。
- **雪谱再入**（L4440–L4478）：由雪含量先换算第 2 矩 `M2 = rs • oams`，经 `sa/sb` 多项式得到 `M3`（bm_s+1 矩），再出 `Mrat/M0/slam1/slam2`——与 Field 双伽马谱的完整重建；bin 密度 `N_s(n)`

按双伽马两项直接求和。**这是 Field 谱“离线积分”的样板**：运行时不可承受的双谱二重积分，在此以四维网格预积。

- 小瑕疵：`D1` 数组（L4319/L4406）赋值后从未使用——死代码，读时不必纠结。

07b.2.3 `freezeH2O`：Bigg 冻结表与尘埃订正

```
!file: phys/module_mp_thompson.F    L4599-L4676
L04599
L04600        subroutine freezeH2O
L04601
L04602        implicit none
L04603
L04604  !..Local variables
L04605        INTEGER:: i, j, k, m, n, n2
L04606        INTEGER:: km, km_s, km_e
L04607        DOUBLE PRECISION:: N_r, N_c
L04608        DOUBLE PRECISION, DIMENSION(nbr):: massr
L04609        DOUBLE PRECISION, DIMENSION(nbc):: massc
L04610        DOUBLE PRECISION:: sum1, sum2, sumn1, sumn2, &
L04611                           prob, vol, Texp, orho_w, &
L04612                           lam_exp, lamr, N0_r, lamc, N0_c, y
L04613        INTEGER:: nu_c
L04614        REAL:: T_adjust
L04615        LOGICAL force_read_thompson, write_thompson_tables
L04616        LOGICAL lexist,lopen
L04617        INTEGER good
L04618        LOGICAL, EXTERNAL :: wrf_dm_on_monitor
L04619
L04620  !+---+
L04621        CALL nl_get_force_read_thompson(1,force_read_thompson)
L04622        CALL nl_get_write_thompson_tables(1,write_thompson_tables)
L04623
L04624        good = 0
L04625        IF ( wrf_dm_on_monitor() ) THEN
L04626          INQUIRE(FILE="freezeH2O.dat",EXIST=lexist)
L04627          IF ( lexist ) THEN
L04628            CALL wrf_message("ThompMP: read freezeH2O.dat instead of computing")
L04629            OPEN(63,file="freezeH2O.dat",form="unformatted",err=1234)
L04630            READ(63,err=1234)tpi_qrfz
L04631            READ(63,err=1234)tni_qrfz
L04632            READ(63,err=1234)tpg_qrfz
L04633            READ(63,err=1234)tnr_qrfz
L04634            READ(63,err=1234)tpi_qcfz
L04635            READ(63,err=1234)tni_qcfz
L04636            good = 1
L04637   1234     CONTINUE
L04638            IF ( good .NE. 1 ) THEN
L04639              INQUIRE(63,opened=lopen)
L04640              IF (lopen) THEN
L04641                IF( force_read_thompson ) THEN
L04642                  CALL wrf_error_fatal("Error reading freezeH2O.dat. Aborting because force_read_thompson is .true.")
L04643                ENDIF
L04644                CLOSE(63)
L04645              ELSE
L04646                IF( force_read_thompson ) THEN
L04647                  CALL wrf_error_fatal("Error opening freezeH2O.dat. Aborting because force_read_thompson is .true.")
L04648                ENDIF
L04649              ENDIF
L04650            ELSE
L04651              INQUIRE(63,opened=lopen)
L04652              IF (lopen) THEN
L04653                CLOSE(63)
L04654              ENDIF
L04655            ENDIF
L04656          ELSE
L04657            IF( force_read_thompson ) THEN
L04658              CALL wrf_error_fatal("Non-existent freezeH2O.dat. Aborting because force_read_thompson is .true.")
L04659            ENDIF
L04660          ENDIF
L04661        ENDIF
L04662
L04663  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04664        CALL wrf_dm_bcast_integer(good,1)
L04665  #endif
L04666
L04667        IF ( good .EQ. 1 ) THEN
L04668  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L04669          CALL wrf_dm_bcast_double(tpi_qrfz,SIZE(tpi_qrfz))
L04670          CALL wrf_dm_bcast_double(tni_qrfz,SIZE(tni_qrfz))
L04671          CALL wrf_dm_bcast_double(tpg_qrfz,SIZE(tpg_qrfz))
L04672          CALL wrf_dm_bcast_double(tnr_qrfz,SIZE(tnr_qrfz))
L04673          CALL wrf_dm_bcast_double(tpi_qcfz,SIZE(tpi_qcfz))
L04674          CALL wrf_dm_bcast_double(tni_qcfz,SIZE(tni_qcfz))
L04675  #endif
L04676        ELSE
```

缓存文件 `freezeH2O.dat`，六张表：雨冻结的 `tpi/tni_qrfz`（→ 冰晶）、`tpg/tnr_qrfz`（→ 霰）与云滴冻结的 `tpi/tni_qcfz`。四维 (含量，截距，温度，IN 浓度)——**第四维就是尘埃**。

```
!file: phys/module_mp_thompson.F    L4677-L4791
L04677          CALL wrf_message("ThompMP: computing freezeH2O")
L04678
L04679          orho_w = 1./rho_w
L04680
L04681          do n2 = 1, nbr
L04682           massr(n2) = am_r*Dr(n2)**bm_r
L04683          enddo
L04684          do n = 1, nbc
L04685           massc(n) = am_r*Dc(n)**bm_r
L04686          enddo
L04687
```

```
L04688  ! Need to split loops between MPI processes to speedup
L04689  ! (2017Jul26, Jason Do)
L04690  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04691          CALL wrf_dm_decomp1d ( ntb_IN*45, km_s, km_e )
L04692  #else
L04693          km_s = 0
L04694          km_e = ntb_IN*45 - 1
L04695  #endif
L04696
L04697  !..Freeze water (smallest drops become cloud ice, otherwise graupel).
L04698          do km = km_s, km_e
L04699           m = km / 45 + 1
L04700           k = mod( km , 45 ) + 1
L04701           T_adjust = MAX(-3.0, MIN(3.0 - ALOG10(Nt_IN(m)), 3.0))
L04702  !         print*, ' Freezing water for temp = ', -k
L04703           Texp = DEXP( DFLOAT(k) - T_adjust*1.0D0 ) - 1.0D0
L04704  #ifdef THOMPSON_OMP
L04705  !$OMP PARALLEL DO SCHEDULE(dynamic) &
L04706  !$OMP PRIVATE(j,i,lam_exp,lamr,N0_r,sum1,sum2,sumn1,sumn2,n2,N_r,vol,prob)
L04707  #endif
L04708           do j = 1, ntb_r1
L04709              do i = 1, ntb_r
L04710                 lam_exp = (N0r_exp(j)*am_r*crg(1)/r_r(i))**ore1
L04711                 lamr = lam_exp * (crg(3)*org2*org1)**obmr
L04712                 N0_r = N0r_exp(j)/(crg(2)*lam_exp) * lamr**cre(2)
L04713                 sum1 = 0.0d0
L04714                 sum2 = 0.0d0
L04715                 sumn1 = 0.0d0
L04716                 sumn2 = 0.0d0
L04717                 do n2 = nbr, 1, -1
L04718                    N_r = N0_r*Dr(n2)**mu_r*DEXP(-lamr*Dr(n2))*dtr(n2)
L04719                    vol = massr(n2)*orho_w
L04720                    prob = MAX(0.0D0, 1.0D0 - DEXP(-120.0D0*vol*5.2D-4 * Texp))
L04721                    if (massr(n2) .lt. xm0g) then
L04722                       sumn1 = sumn1 + prob*N_r
L04723                       sum1 = sum1 + prob*N_r*massr(n2)
L04724                    else
L04725                       sumn2 = sumn2 + prob*N_r
L04726                       sum2 = sum2 + prob*N_r*massr(n2)
L04727                    endif
L04728                    if ((sum1+sum2).ge.r_r(i)) EXIT
L04729                 enddo
L04730                 tpi_qrfz(i,j,k,m) = sum1
L04731                 tni_qrfz(i,j,k,m) = sumn1
L04732                 tpg_qrfz(i,j,k,m) = sum2
L04733                 tnr_qrfz(i,j,k,m) = sumn2
L04734              enddo
L04735           enddo
L04736  #ifdef THOMPSON_OMP
L04737  !$OMP END PARALLEL DO
L04738
L04739  !$OMP PARALLEL DO SCHEDULE(dynamic) &
L04740  !$OMP PRIVATE(j,i,nu_c,lamc,N0_c,sum1,sumn2,vol,prob,N_c)
L04741  #endif
L04742           do j = 1, nbc
L04743              nu_c = MIN(15, NINT(1000.E6/t_Nc(j)) + 2)
L04744              do i = 1, ntb_c
L04745                 lamc = (t_Nc(j)*am_r* ccg(2,nu_c) * ocg1(nu_c) / r_c(i))**obmr
L04746                 N0_c = t_Nc(j)*ocg1(nu_c) * lamc**cce(1,nu_c)
L04747                 sum1 = 0.0d0
L04748                 sumn2 = 0.0d0
L04749                 do n = nbc, 1, -1
L04750                    vol = massc(n)*orho_w
L04751                    prob = MAX(0.0D0, 1.0D0 - DEXP(-120.0D0*vol*5.2D-4 * Texp))
L04752                    N_c = N0_c*Dc(n)**nu_c*EXP(-lamc*Dc(n))*dtc(n)
L04753                    sumn2 = MIN(t_Nc(j), sumn2 + prob*N_c)
L04754                    sum1 = sum1 + prob*N_c*massc(n)
L04755                    if (sum1 .ge. r_c(i)) EXIT
L04756                 enddo
L04757                 tpi_qcfz(i,j,k,m) = sum1
L04758                 tni_qcfz(i,j,k,m) = sumn2
L04759              enddo
L04760           enddo
L04761  #ifdef THOMPSON_OMP
L04762  !$OMP END PARALLEL DO
L04763  #endif
L04764          enddo
L04765
L04766  #if ( defined( DM_PARALLEL ) && ( ! defined( STUBMPI ) ) )
L04767          CALL wrf_dm_gatherv(tpi_qrfz, ntb_r*ntb_r1, km_s, km_e, R8SIZE)
L04768          CALL wrf_dm_gatherv(tni_qrfz, ntb_r*ntb_r1, km_s, km_e, R8SIZE)
L04769          CALL wrf_dm_gatherv(tpg_qrfz, ntb_r*ntb_r1, km_s, km_e, R8SIZE)
L04770          CALL wrf_dm_gatherv(tnr_qrfz, ntb_r*ntb_r1, km_s, km_e, R8SIZE)
L04771          CALL wrf_dm_gatherv(tpi_qcfz, ntb_c*nbc, km_s, km_e, R8SIZE)
L04772          CALL wrf_dm_gatherv(tni_qcfz, ntb_c*nbc, km_s, km_e, R8SIZE)
L04773  #endif
L04774
L04775          IF ( write_thompson_tables .AND. wrf_dm_on_monitor() ) THEN
L04776            CALL wrf_message("Writing freezeH2O.dat in Thompson MP init")
L04777            OPEN(63,file="freezeH2O.dat",form="unformatted",err=9234)
L04778            WRITE(63,err=9234)tpi_qrfz
L04779            WRITE(63,err=9234)tni_qrfz
L04780            WRITE(63,err=9234)tpg_qrfz
L04781            WRITE(63,err=9234)tnr_qrfz
L04782            WRITE(63,err=9234)tpi_qcfz
L04783            WRITE(63,err=9234)tni_qcfz
L04784            CLOSE(63)
L04785            RETURN    ! ----- RETURN
L04786   9234     CONTINUE
L04787            CALL wrf_error_fatal("Error writing freezeH2O.dat")
L04788          ENDIF
L04789        ENDIF
L04790
L04791        end subroutine freezeH2O
```

- **Bigg (1954) 冻结概率**（L4720）：`prob = 1 − exp(−120 • vol • 5.2e-4 • Texp)`——`vol` 是水滴体积（m³），`5.2e-4` 与 `120` 合起来是 Bigg 单位体积冻结率 J(T) 的离散化（J 随过冷度指数

上升）；`Texp = exp(k - T_adjust) - 1`，k 是过冷度（1 度步进，1–45 K）。**从大滴往小滴积分**（`do n2 = nbr, 1, -1`）并在累计冻结量达到含量时 `EXIT`——概率虽小但含量账封顶。

- **尘埃订正 `T_adjust`**（L4701）：`T_adjust = clamp(3 - log10(Nt_IN), -3, 3)`——IN 浓度 1/L 时 T_adjust=3（冻结温度等效上移 3 K），IN 越少订正越负（冻结推迟，最多 3 K）。**`dustyIce` 的全部机制就在这一行**：冰核浓度通过 “等效过冷度” 调节冻结概率，主实体按 `(含量,截距,温度,IN)` 四维索引查表。
- **冻结产物分流**（L4721–L4727）：雨滴质量 < `xm0g`（霰出生质量）的冻结成**冰晶**（sum1/sumn1），≥ 的直接成**霰**（sum2/sumn2）——冻结的产物归属由粒子尺寸决定，与主实体 `pri_rfz/prg_rfz` 的分账对应。
- **OpenMP**（`THOMPSON_OMP` 宏，Jason Do 2017 注释）：两个内层 j 循环并行——建表是初始化期的大计算量，能并行则并行。

07b.2.4 `qi_aut_qs`：冰晶 → 雪自动转化表

```
!file: phys/module_mp_thompson.F    L4806-L4849
L04806        subroutine qi_aut_qs
L04807
L04808        implicit none
L04809
L04810  !..Local variables
L04811        INTEGER:: i, j, n2
L04812        DOUBLE PRECISION, DIMENSION(nbi):: N_i
L04813        DOUBLE PRECISION:: N0_i, lami, Di_mean, t1, t2
L04814        REAL:: xlimit_intg
L04815
L04816  !+---+
L04817
L04818        do j = 1, ntb_i1
L04819           do i = 1, ntb_i
L04820              lami = (am_i*cig(2)*oig1*Nt_i(j)/r_i(i))**obmi
L04821              Di_mean = (bm_i + mu_i + 1.) / lami
L04822              N0_i = Nt_i(j)*oig1 * lami**cie(1)
L04823              t1 = 0.0d0
L04824              t2 = 0.0d0
L04825              if (SNGL(Di_mean) .gt. 5.*D0s) then
L04826               t1 = r_i(i)
L04827               t2 = Nt_i(j)
L04828               tpi_ide(i,j) = 0.0D0
L04829              elseif (SNGL(Di_mean) .lt. D0i) then
L04830               t1 = 0.0D0
L04831               t2 = 0.0D0
L04832               tpi_ide(i,j) = 1.0D0
L04833              else
L04834               xlimit_intg = lami*D0s
L04835               tpi_ide(i,j) = GAMMP(mu_i+2.0, xlimit_intg) * 1.0D0
L04836               do n2 = 1, nbi
L04837                 N_i(n2) = N0_i*Di(n2)**mu_i * DEXP(-lami*Di(n2))*dti(n2)
L04838                 if (Di(n2).ge.D0s) then
L04839                    t1 = t1 + N_i(n2) * am_i*Di(n2)**bm_i
L04840                    t2 = t2 + N_i(n2)
L04841                 endif
L04842               enddo
L04843              endif
L04844              tps_iaus(i,j) = t1
L04845              tni_iaus(i,j) = t2
L04846           enddo
L04847        enddo
L04848
L04849        end subroutine qi_aut_qs
```

- **判定逻辑**：把冰晶谱按 bin 离散，直径 ≥ `D0s`（300 μm）的部分质量/数量计入转雪表 `tps_iaus/tni_iaus`——**自动转化在这里不是经验阈值公式，而是 “谱尾积分”**。
- **不完全 Γ 函数出马**（L4835）：`tpi_ide = GAMMP(mu_i+2, lami * D0s)`——直径小于 D0s 的冰晶占谱的份额（对谱密度 N0 • D^mu • exp(−λD) 从 0 积到 D0s，换元后恰是不完全 Γ 函数 P(a,x)）。主实体用它把冰晶凝华增长**分账**：小粒径端留冰晶、大粒径端归雪（Harrington et al. 1995 思想，模块头注释写明）。Γ 函数族在 §07b.3。
- 两端特判：平均径 > 5×D0s 全转雪（`tpi_ide=0`），< D0i 全不转（`tpi_ide=1`）。

07b.2.5 `table_Efrw` / `table_Efsw`：碰并效率表

```
!file: phys/module_mp_thompson.F    L4857-L4913
L04857        subroutine table_Efrw
L04858
L04859        implicit none
L04860
L04861  !..Local variables
L04862        DOUBLE PRECISION:: vtr, stokes, reynolds, Ef_rw
L04863        DOUBLE PRECISION:: p, yc0, F, G, H, z, K0, X
L04864        INTEGER:: i, j
L04865
L04866        do j = 1, nbc
L04867        do i = 1, nbr
L04868           Ef_rw = 0.0
L04869           p = Dc(j)/Dr(i)
L04870           if (Dr(i).lt.50.E-6 .or. Dc(j).lt.3.E-6) then
L04871            t_Efrw(i,j) = 0.0
```

```
L04872           elseif (p.gt.0.25) then
L04873            X = Dc(j)*1.D6
L04874            if (Dr(i) .lt. 75.e-6) then
L04875               Ef_rw = 0.026794*X - 0.20604
L04876            elseif (Dr(i) .lt. 125.e-6) then
L04877               Ef_rw = -0.00066842*X*X + 0.061542*X - 0.37089
L04878            elseif (Dr(i) .lt. 175.e-6) then
L04879               Ef_rw = 4.091e-06*X*X*X*X - 0.00030908*X*X*X              &
L04880                     + 0.0066237*X*X - 0.0013687*X - 0.073022
L04881            elseif (Dr(i) .lt. 250.e-6) then
L04882               Ef_rw = 9.6719e-5*X*X*X - 0.0068901*X*X + 0.17305*X       &
L04883                     - 0.65988
L04884            elseif (Dr(i) .lt. 350.e-6) then
L04885               Ef_rw = 9.0488e-5*X*X*X - 0.006585*X*X + 0.16606*X        &
L04886                     - 0.56125
L04887            else
L04888               Ef_rw = 0.00010721*X*X*X - 0.0072962*X*X + 0.1704*X       &
L04889                     - 0.46929
L04890            endif
L04891           else
L04892            vtr = -0.1021 + 4.932E3*Dr(i) - 0.9551E6*Dr(i)*Dr(i) &
L04893                + 0.07934E9*Dr(i)*Dr(i)*Dr(i) &
L04894                - 0.002362E12*Dr(i)*Dr(i)*Dr(i)*Dr(i)
L04895            stokes = Dc(j)*Dc(j)*vtr*rho_w/(9.*1.718E-5*Dr(i))
L04896            reynolds = 9.*stokes/(p*p*rho_w)
L04897
L04898            F = DLOG(reynolds)
L04899            G = -0.1007D0 - 0.358D0*F + 0.0261D0*F*F
L04900            K0 = DEXP(G)
L04901            z = DLOG(stokes/(K0+1.D-15))
L04902            H = 0.1465D0 + 1.302D0*z - 0.607D0*z*z + 0.293D0*z*z*z
L04903            yc0 = 2.0D0/PI * ATAN(H)
L04904            Ef_rw = (yc0+p)*(yc0+p) / ((1.+p)*(1.+p))
L04905
L04906           endif
L04907
L04908           t_Efrw(i,j) = MAX(0.0, MIN(SNGL(Ef_rw), 0.95))
L04909
L04910        enddo
L04911        enddo
L04912
L04913        end subroutine table_Efrw
```

- **两段式物理**：① 小滴比 `p = Dc/Dr < 0.25` 走 **Beard & Grover (1974) 半解析式**——Stokes 数与 Reynolds 数算出后经 `G/K0/z/H/yc0` 四步拟合成 `yc0 = 2/π • atan(H)`，效率 `(yc0+p)²/(1+p)²`（Langmuir 流场小滴轨迹积分的标准拟合）；② 大滴比走 **Pruppacher & Klett 图 14-9 的六段多项式**（按雨径分 50–75–125–175–250–350 μm+ 六段，每段是云滴径 X 的二/三/四次式）——**理论式与实验拟合拼接**是碰撞效率建模的常态。
- 端点零效率：雨径 < 50 μm 或云滴 < 3 μm；上限钳 0.95。

```
!file: phys/module_mp_thompson.F    L4921-L4957
L04921        subroutine table_Efsw
L04922
L04923        implicit none
L04924
L04925  !..Local variables
L04926        DOUBLE PRECISION:: Ds_m, vts, vtc, stokes, reynolds, Ef_sw
L04927        DOUBLE PRECISION:: p, yc0, F, G, H, z, K0
L04928        INTEGER:: i, j
L04929
L04930        do j = 1, nbc
L04931        vtc = 1.19D4 * (1.0D4*Dc(j)*Dc(j)*0.25D0)
L04932        do i = 1, nbs
L04933           vts = av_s*Ds(i)**bv_s * DEXP(-fv_s*Ds(i)) - vtc
L04934           Ds_m = (am_s*Ds(i)**bm_s / am_r)**obmr
L04935           p = Dc(j)/Ds_m
L04936           if (p.gt.0.25 .or. Ds(i).lt.D0s .or. Dc(j).lt.6.E-6 &
L04937                .or. vts.lt.1.E-3) then
L04938            t_Efsw(i,j) = 0.0
L04939           else
L04940            stokes = Dc(j)*Dc(j)*vts*rho_w/(9.*1.718E-5*Ds_m)
L04941            reynolds = 9.*stokes/(p*p*rho_w)
L04942
L04943            F = DLOG(reynolds)
L04944            G = -0.1007D0 - 0.358D0*F + 0.0261D0*F*F
L04945            K0 = DEXP(G)
L04946            z = DLOG(stokes/(K0+1.D-15))
L04947            H = 0.1465D0 + 1.302D0*z - 0.607D0*z*z + 0.293D0*z*z*z
L04948            yc0 = 2.0D0/PI * ATAN(H)
L04949            Ef_sw = (yc0+p)*(yc0+p) / ((1.+p)*(1.+p))
L04950
L04951            t_Efsw(i,j) = MAX(0.0, MIN(SNGL(Ef_sw), 0.95))
L04952           endif
L04953
L04954        enddo
L04955        enddo
L04956
L04957        end subroutine table_Efsw
```

- **雪的等效水球径**（L4934）：`Ds_m = (am_s • D^bm_s/am_r)^(1/bm_r)`——把非球形的雪按质量换算成同质量水球，才能进 Stokes 判据；`vtc` 是云滴 Stokes 流落速、`vts` 是雪落速减去云滴落速（相对速度）。
- 其余与 `table_Efrw` 同构（Wang & Ji 2000 方法，模块头注释）。

07b.2.6 `Eff_aero`：气溶胶清除效率（运行时函数）

```
!file: phys/module_mp_thompson.F    L4965-L5001
L04965        real function Eff_aero(D, Da, visc,rhoa,Temp,species)
L04966
L04967        implicit none
L04968        real:: D, Da, visc, rhoa, Temp
L04969        character(LEN=1):: species
L04970        real:: aval, Cc, diff, Re, Sc, St, St2, vt, Eff
L04971        real, parameter:: boltzman = 1.3806503E-23
L04972        real, parameter:: meanPath = 0.0256E-6
L04973
L04974        vt = 1.
L04975        if (species .eq. 'r') then
L04976           vt = -0.1021 + 4.932E3*D - 0.9551E6*D*D                    &
L04977                + 0.07934E9*D*D*D - 0.002362E12*D*D*D*D
L04978        elseif (species .eq. 's') then
L04979           vt = av_s*D**bv_s
L04980        elseif (species .eq. 'g') then
L04981           vt = av_g(idx_bg1)*D**bv_g(idx_bg1)
L04982        endif
L04983
L04984        Cc    = 1. + 2.*meanPath/Da *(1.257+0.4*exp(-0.55*Da/meanPath))
L04985        diff  = boltzman*Temp*Cc/(3.*PI*visc*Da)
L04986
L04987        Re    = 0.5*rhoa*D*vt/visc
L04988        Sc    = visc/(rhoa*diff)
L04989
L04990        St    = Da*Da*vt*1000./(9.*visc*D)
L04991        aval  = 1.+LOG(1.+Re)
L04992        St2   = (1.2 + 1./12.*aval)/(1.+aval)
L04993
L04994        Eff = 4./(Re*Sc) * (1. + 0.4*SQRT(Re)*Sc**0.3333               &
L04995                          + 0.16*SQRT(Re)*SQRT(Sc))                    &
L04996                 + 4.*Da/D * (0.02 + Da/D*(1.+2.*SQRT(Re)))
L04997
L04998        if (St.gt.St2) Eff = Eff  + ( (St-St2)/(St-St2+0.666667))**1.5
L04999        Eff_aero = MAX(1.E-5, MIN(Eff, 1.0))
L05000
L05001        end function Eff_aero
```

这是建表族里唯一的**运行时**函数（主实体每个收集过程都调）。Slinn (1983) 框架（Wang et al. 2010, ACP 实现）：三项之和——- **布朗扩散** `4/(Re • Sc) • (1 + 0.4 • √Re • Sc^{1/3} + 0.16 • √Re • √Sc)`（小粒子主导）；- **截留** `4 • Da/D • (0.02 + Da/D • (1+2√Re))`（粒径比主导）；- **惯性撞击** `((St−St2)/(St−St2+2/3))^{3/2}`，仅当 `St > St2`（大粒子主导），临界 Stokes 数 `St2 = (1.2 + ln(1+Re)/12)/(1+ln(1+Re))`。- Cunningham 滑移修正 `Cc`（气溶胶小到接近分子平均自由程时扩散增强）；`vt` 按收集者物种选落速式（雨多项式/雪霰幂律）。钳位 [1e-5, 1]。- 主实体用法（code_07a §07a.8.5）：对 CCN 用 0.04 μm、IN 用 0.8 μm、黑碳用 0.0236 μm 的代表性粒径——**一个函数服务三类气溶胶，差异全在调用参数。**

07b.2.7 `table_dropEvap`：蒸发的累积分布表

```
!file: phys/module_mp_thompson.F    L5011-L5099
L05011        subroutine table_dropEvap
L05012
L05013        implicit none
L05014
L05015  !..Local variables
L05016        INTEGER:: i, j, k, n
L05017        DOUBLE PRECISION, DIMENSION(nbc):: N_c, massc
L05018        DOUBLE PRECISION:: summ, summ2, lamc, N0_c
L05019        INTEGER:: nu_c
L05020  !      DOUBLE PRECISION:: Nt_r, N0, lam_exp, lam
L05021  !      REAL:: xlimit_intg
L05022
L05023        do n = 1, nbc
L05024           massc(n) = am_r*Dc(n)**bm_r
L05025        enddo
L05026
L05027        do k = 1, nbc
L05028           nu_c = MIN(15, NINT(1000.E6/t_Nc(k)) + 2)
L05029           do j = 1, ntb_c
L05030              lamc = (t_Nc(k)*am_r* ccg(2,nu_c)*ocg1(nu_c) / r_c(j))**obmr
L05031              N0_c = t_Nc(k)*ocg1(nu_c) * lamc**cce(1,nu_c)
L05032              do i = 1, nbc
L05033  !-GT             tnc_wev(i,j,k) = GAMMP(nu_c+1., SNGL(Dc(i)*Lamc))*t_Nc(k)
L05034                 N_c(i) = N0_c* Dc(i)**nu_c*EXP(-lamc*Dc(i))*dtc(i)
L05035  !     if(j.eq.18 .and. k.eq.50) print*, ' N_c = ', N_c(i)
L05036                 summ = 0.
L05037                 summ2 = 0.
L05038                 do n = 1, i
L05039                    summ = summ + massc(n)*N_c(n)
L05040                    summ2 = summ2 + N_c(n)
L05041                 enddo
L05042  !      if(j.eq.18 .and. k.eq.50) print*, '  DEBUG-TABLE: ', r_c(j), t_Nc(k), summ2, summ
L05043                 tpc_wev(i,j,k) = summ
L05044                 tnc_wev(i,j,k) = summ2
L05045              enddo
L05046           enddo
L05047        enddo
L05048
L05049  !
L05050  !..To do the same thing for rain.
L05051  !
L05052  !     do k = 1, ntb_r
L05053  !     do j = 1, ntb_r1
```

```
L05054  !        lam_exp = (N0r_exp(j)*am_r*crg(1)/r_r(k))**ore1
L05055  !        lam = lam_exp * (crg(3)*org2*org1)**obmr
L05056  !        N0 = N0r_exp(j)/(crg(2)*lam_exp) * lam**cre(2)
L05057  !        Nt_r = N0 * crg(2) / lam**cre(2)
L05058  !        do i = 1, nbr
L05059  !           xlimit_intg = lam*Dr(i)
L05060  !           tnr_rev(i,j,k) = GAMMP(mu_r+1.0, xlimit_intg) * Nt_r
L05061  !        enddo
L05062  !     enddo
L05063  !     enddo
L05064
L05065  ! TO APPLY TABLE ABOVE
L05066  !..Rain lookup table indexes.
L05067  !         Dr_star = DSQRT(-2.D0*DT * t1_evap/(2.*PI) &
L05068  !                   * 0.78*4.*diffu(k)*xsat*rvs/rho_w)
L05069  !         idx_d = NINT(1.0 + FLOAT(nbr) * DLOG(Dr_star/D0r)            &
L05070  !              / DLOG(Dr(nbr)/D0r))
L05071  !         idx_d = MAX(1, MIN(idx_d, nbr))
L05072  !
L05073  !         nir = NINT(ALOG10(rr(k)))
L05074  !         do nn = nir-1, nir+1
L05075  !            n = nn
L05076  !            if ( (rr(k)/10.**nn).ge.1.0 .and. &
L05077  !                 (rr(k)/10.**nn).lt.10.0) goto 154
L05078  !         enddo
L05079  !154      continue
L05080  !         idx_r = INT(rr(k)/10.**n) + 10*(n-nir2) - (n-nir2)
L05081  !         idx_r = MAX(1, MIN(idx_r, ntb_r))
L05082  !
L05083  !         lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L05084  !         lam_exp = lamr * (crg(3)*org2*org1)**bm_r
L05085  !         N0_exp = org1*rr(k)/am_r * lam_exp**cre(1)
L05086  !         nir = NINT(DLOG10(N0_exp))
L05087  !         do nn = nir-1, nir+1
L05088  !            n = nn
L05089  !            if ( (N0_exp/10.**nn).ge.1.0 .and. &
L05090  !                 (N0_exp/10.**nn).lt.10.0) goto 155
L05091  !         enddo
L05092  !155      continue
L05093  !         idx_r1 = INT(N0_exp/10.**n) + 10*(n-nir3) - (n-nir3)
L05094  !         idx_r1 = MAX(1, MIN(idx_r1, ntb_r1))
L05095  !
L05096  !         pnr_rev(k) = MIN(nr(k)*odts, SNGL(tnr_rev(idx_d,idx_r1,idx_r) &   ! RAIN2M
L05097  !                    * odts))
L05098
L05099        end subroutine table_dropEvap
```

- **表内容：** `tpc_wev/tnx_wev(i,j,k)` = 直径 ≤ `Dc(i)` 的云滴**累积**质量/数量——三维索引是 (直径上界, 云水含量, 云滴数)。主实体蒸发分支算出 “单步内全蒸发最大径 `Dc_star`” 后按它索引，直接取 “这批全蒸发的小滴” 的质量与数（code_07a §07a.8.14 蒸发段）。
- **开发史痕迹**（L5033 起的大段注释）：被 ！ 掉的 `GAMMP` 版本（解析累积分布）与完整的雨侧应用代码——**表实现与解析实现的对照现场**；最终雨蒸发用了解析积分（主实体 `t1_qr_ev` 系数），云滴蒸发用本表。读 WRF 源码要习惯这种 “注释里埋着另一条实现路径” 的写法。

07b.2.8 `table_ccnAct` 与 `activ_ncloud`：气溶胶活化

```
!file: phys/module_mp_thompson.F    L5110-L5166
L05110        subroutine table_ccnAct
L05111
L05112        USE module_domain
L05113        USE module_dm
L05114        implicit none
L05115
L05116        LOGICAL, EXTERNAL:: wrf_dm_on_monitor
L05117
L05118  !..Local variables
L05119        INTEGER:: iunit_mp_th1, i
L05120        LOGICAL:: opened
L05121        CHARACTER*64 errmess
L05122
L05123        iunit_mp_th1 = -1
L05124        IF ( wrf_dm_on_monitor() ) THEN
L05125          DO i = 20,99
L05126            INQUIRE ( i , OPENED = opened )
L05127            IF ( .NOT. opened ) THEN
L05128              iunit_mp_th1 = i
L05129              GOTO 2010
L05130            ENDIF
L05131          ENDDO
L05132   2010   CONTINUE
L05133        ENDIF
L05134  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L05135        CALL wrf_dm_bcast_bytes ( iunit_mp_th1 , IWORDSIZE )
L05136  #endif
L05137        IF ( iunit_mp_th1 < 0 ) THEN
L05138          CALL wrf_error_fatal ( 'module_mp_thompson: table_ccnAct: '//   &
L05139             'Can not find unused fortran unit to read in lookup table.')
L05140        ENDIF
L05141
L05142        IF ( wrf_dm_on_monitor() ) THEN
L05143          WRITE(errmess, '(A,I2)') 'module_mp_thompson: opening CCN_ACTIVATE.BIN on unit ',iunit_mp_th1
L05144          CALL wrf_debug(150, errmess)
L05145          OPEN(iunit_mp_th1,FILE='CCN_ACTIVATE.BIN',                    &
L05146               FORM='UNFORMATTED',STATUS='OLD',ERR=9009)
L05147        ENDIF
L05148
L05149  #define DM_BCAST_MACRO(A) CALL wrf_dm_bcast_bytes(A, size(A)*R4SIZE)
L05150
L05151        IF ( wrf_dm_on_monitor() ) READ(iunit_mp_th1,ERR=9010) tnccn_act
```

```
L05152  #if defined(DM_PARALLEL) && !defined(STUBMPI)
L05153        DM_BCAST_MACRO(tnccn_act)
L05154  #endif
L05155
L05156
L05157        RETURN
L05158   9009 CONTINUE
L05159        WRITE( errmess , '(A,I2)' ) 'module_mp_thompson: error opening CCN_ACTIVATE.BIN on unit ',iunit_mp_th1
L05160        CALL wrf_error_fatal(errmess)
L05161        RETURN
L05162   9010 CONTINUE
L05163        WRITE( errmess , '(A,I2)' ) 'module_mp_thompson: error reading CCN_ACTIVATE.BIN on unit ',iunit_mp_th1
L05164        CALL wrf_error_fatal(errmess)
L05165
L05166        end subroutine table_ccnAct
```

- **数据来源：**`CCN_ACTIVATE.BIN` 是 Eidhammer 气团模式（parcel model）预计算的**五维活化概率表**（轴定义见模块常数段 `ta_Na/ta_Ww/ta_Tk/ta_Ra/ta_Ka`）。建表不是“算”而是“读”——**物理复杂度最高的一张表反而是外部数据**。
- **单元号扫描**（20–99 找空闲 Fortran unit）+ `wrf_dm_bcast_bytes` 广播；`DM_BCAST_MACRO` 用 `#define` 宏简化重复的广播调用——C 预处理器在 Fortran 代码里的罕见用例（`.F` 大写后缀的含义正在于此）。

```
!file: phys/module_mp_thompson.F    L5178-L5252
L05178        real function activ_ncloud(Tt, Ww, NCCN)
L05179
L05180        implicit none
L05181        REAL, INTENT(IN):: Tt, Ww, NCCN
L05182        REAL:: n_local, w_local
L05183        INTEGER:: i, j, k, l, m, n
L05184        REAL:: A, B, C, D, t, u, x1, x2, y1, y2, nx, wy, fraction
L05185
L05186
L05187  !     ta_Na = (/10.0, 31.6, 100.0, 316.0, 1000.0, 3160.0, 10000.0/)  ntb_arc
L05188  !     ta_Ww = (/0.01, 0.0316, 0.1, 0.316, 1.0, 3.16, 10.0, 31.6, 100.0/)  ntb_arw
L05189  !     ta_Tk = (/243.15, 253.15, 263.15, 273.15, 283.15, 293.15, 303.15/)  ntb_art
L05190  !     ta_Ra = (/0.01, 0.02, 0.04, 0.08, 0.16/)  ntb_arr
L05191  !     ta_Ka = (/0.2, 0.4, 0.6, 0.8/)  ntb_ark
L05192
L05193        n_local = NCCN * 1.E-6
L05194        w_local = Ww
L05195
L05196        if (n_local .ge. ta_Na(ntb_arc)) then
L05197           n_local = ta_Na(ntb_arc) - 1.0
L05198        elseif (n_local .le. ta_Na(1)) then
L05199           n_local = ta_Na(1) + 1.0
L05200        endif
L05201        do n = 2, ntb_arc
L05202           if (n_local.ge.ta_Na(n-1) .and. n_local.lt.ta_Na(n)) goto 8003
L05203        enddo
L05204   8003 continue
L05205        i = n
L05206        x1 = LOG(ta_Na(i-1))
L05207        x2 = LOG(ta_Na(i))
L05208
L05209        if (w_local .ge. ta_Ww(ntb_arw)) then
L05210           w_local = ta_Ww(ntb_arw) - 1.0
L05211        elseif (w_local .le. ta_Ww(1)) then
L05212           w_local = ta_Ww(1) + 0.001
L05213        endif
L05214        do n = 2, ntb_arw
L05215           if (w_local.ge.ta_Ww(n-1) .and. w_local.lt.ta_Ww(n)) goto 8005
L05216        enddo
L05217   8005 continue
L05218        j = n
L05219        y1 = LOG(ta_Ww(j-1))
L05220        y2 = LOG(ta_Ww(j))
L05221
L05222        k = MAX(1, MIN( NINT( (Tt - ta_Tk(1))*0.1) + 1, ntb_art))
L05223
L05224  !..The next two values are indexes of mean aerosol radius and
L05225  !.. hygroscopicity.  Currently these are constant but a future version
L05226  !.. should implement other variables to allow more freedom such as
L05227  !.. at least simple separation of tiny size sulfates from larger
L05228  !.. sea salts.
L05229        l = 3
L05230        m = 2
L05231
L05232        A = tnccn_act(i-1,j-1,k,l,m)
L05233        B = tnccn_act(i,j-1,k,l,m)
L05234        C = tnccn_act(i,j,k,l,m)
L05235        D = tnccn_act(i-1,j,k,l,m)
L05236        nx = LOG(n_local)
L05237        wy = LOG(w_local)
L05238
L05239        t = (nx-x1)/(x2-x1)
L05240        u = (wy-y1)/(y2-y1)
L05241
L05242  !     t = (n_local-ta(Na(i-1))/(ta_Na(i)-ta_Na(i-1))
L05243  !     u = (w_local-ta_Ww(j-1))/(ta_Ww(j)-ta_Ww(j-1))
L05244
L05245        fraction = (1.0-t)*(1.0-u)*A + t*(1.0-u)*B + t*u*C + (1.0-t)*u*D
L05246
L05247  !     if (NCCN*fraction .gt. 0.75*Nt_c_max) then
L05248  !        write(*,*) ' DEBUG-GT ', n_local, w_local, Tt, i, j, k
L05249  !     endif
L05250
L05251        activ_ncloud = NCCN*fraction
L05252
```

- **五维查表：**气溶胶数与垂直速度**对数双线性**（物理量跨四个量级、表在对数轴上等距）、温度

线性索引、平均半径 `l=3`（0.04 μm）与吸湿性 `m=2`（kappa=0.4）**写死**——原注释自承 “未来版本应放开，至少把硫酸盐与海盐分开”。改造气溶胶敏感性实验时，`l/m` 这两个死值是第一个要动的。

- **双线性插值**（A/B/C/D 四角 + t/u 权重）后 `activ_ncloud = NCCN • fraction`——返回 “实际活化出的云滴数”。

07b.3 不完全 Γ 函数族（Numerical Recipes）

```
!file: phys/module_mp_thompson.F    L5256-L5290
L05256  !+---+-----------------------------------------------------------------+
L05257        SUBROUTINE GCF(GAMMCF,A,X,GLN)
L05258  !     --- RETURNS THE INCOMPLETE GAMMA FUNCTION Q(A,X) EVALUATED BY ITS
L05259  !     --- CONTINUED FRACTION REPRESENTATION AS GAMMCF.  ALSO RETURNS
L05260  !     --- LN(GAMMA(A)) AS GLN.  THE CONTINUED FRACTION IS EVALUATED BY
L05261  !     --- A MODIFIED LENTZ METHOD.
L05262  !     --- USES GAMMLN
L05263        IMPLICIT NONE
L05264        INTEGER, PARAMETER:: ITMAX=100
L05265        REAL, PARAMETER:: gEPS=3.E-7
L05266        REAL, PARAMETER:: FPMIN=1.E-30
L05267        REAL, INTENT(IN):: A, X
L05268        REAL:: GAMMCF,GLN
L05269        INTEGER:: I
L05270        REAL:: AN,B,C,D,DEL,H
L05271        GLN=GAMMLN(A)
L05272        B=X+1.-A
L05273        C=1./FPMIN
L05274        D=1./B
L05275        H=D
L05276        DO 11 I=1,ITMAX
L05277          AN=-I*(I-A)
L05278          B=B+2.
L05279          D=AN*D+B
L05280          IF(ABS(D).LT.FPMIN)D=FPMIN
L05281          C=B+AN/C
L05282          IF(ABS(C).LT.FPMIN)C=FPMIN
L05283          D=1./D
L05284          DEL=D*C
L05285          H=H*DEL
L05286          IF(ABS(DEL-1.).LT.gEPS)GOTO 1
L05287   11   CONTINUE
L05288        PRINT *, 'A TOO LARGE, ITMAX TOO SMALL IN GCF'
L05289   1    GAMMCF=EXP(-X+A*LOG(X)-GLN)*H
L05290        END SUBROUTINE GCF
```

版权提示：`GCF/GSER/GAMMLN/GAMMP` 四件是 Numerical Recipes Software 1986–92 的例程（每段尾部保留版权行）。WRF 主体是 public domain，但这四个函数是受版权的历史遗留——本篇按 “学习性引用” 收录并明确标注；改造复用时注意其许可状态。

`GCF` 用连分式（modified Lentz 法）算 Q(A,X)=Γ(A,X)/Γ(A)，迭代上限 ITMAX=100、收敛 3e-7、`FPMIN=1e-30` 防下溢。

```
!file: phys/module_mp_thompson.F    L5292-L5322
L05292  !+---+-----------------------------------------------------------------+
L05293        SUBROUTINE GSER(GAMSER,A,X,GLN)
L05294  !     --- RETURNS THE INCOMPLETE GAMMA FUNCTION P(A,X) EVALUATED BY ITS
L05295  !     --- ITS SERIES REPRESENTATION AS GAMSER.  ALSO RETURNS LN(GAMMA(A))
L05296  !     --- AS GLN.
L05297  !     --- USES GAMMLN
L05298        IMPLICIT NONE
L05299        INTEGER, PARAMETER:: ITMAX=100
L05300        REAL, PARAMETER:: gEPS=3.E-7
L05301        REAL, INTENT(IN):: A, X
L05302        REAL:: GAMSER,GLN
L05303        INTEGER:: N
L05304        REAL:: AP,DEL,SUM
L05305        GLN=GAMMLN(A)
L05306        IF(X.LE.0.)THEN
L05307          IF(X.LT.0.) PRINT *, 'X < 0 IN GSER'
L05308          GAMSER=0.
L05309          RETURN
L05310        ENDIF
L05311        AP=A
L05312        SUM=1./A
L05313        DEL=SUM
L05314        DO 11 N=1,ITMAX
L05315          AP=AP+1.
L05316          DEL=DEL*X/AP
L05317          SUM=SUM+DEL
L05318          IF(ABS(DEL).LT.ABS(SUM)*gEPS)GOTO 1
L05319   11   CONTINUE
L05320        PRINT *,'A TOO LARGE, ITMAX TOO SMALL IN GSER'
L05321   1    GAMSER=SUM*EXP(-X+A*LOG(X)-GLN)
L05322        END SUBROUTINE GSER
```

`GSER` 用级数算 P(A,X)；`x≤0` 特判（负 x 报错、零返回 0）。

```
!file: phys/module_mp_thompson.F    L5324-L5347
L05324  !+---+-----------------------------------------------------------------+
L05325        REAL FUNCTION GAMMLN(XX)
L05326  !     --- RETURNS THE VALUE LN(GAMMA(XX)) FOR XX > 0.
L05327        IMPLICIT NONE
```

```
L05328        REAL, INTENT(IN):: XX
L05329        DOUBLE PRECISION, PARAMETER:: STP = 2.506628274631000 5D0
L05330        DOUBLE PRECISION, DIMENSION(6), PARAMETER:: &
L05331                 COF = (/76.18009172947146D0, -86.50532032941677D0, &
L05332                         24.01409824083091D0, -1.231739572450155D0, &
L05333                        .1208650973866179D-2, -.5395239384953D-5/)
L05334        DOUBLE PRECISION:: SER,TMP,X,Y
L05335        INTEGER:: J
L05336
L05337        X=XX
L05338        Y=X
L05339        TMP=X+5.5D0
L05340        TMP=(X+0.5D0)*LOG(TMP)-TMP
L05341        SER=1.000000000190015D0
L05342        DO 11 J=1,6
L05343          Y=Y+1.D0
L05344          SER=SER+COF(J)/Y
L05345  11    CONTINUE
L05346        GAMMLN=TMP+LOG(STP*SER/X)
L05347        END FUNCTION GAMMLN
```

GAMMLN 用 Lanczos 逼近算 ln Γ(x)（六系数 COF + 常数 STP=√(2π)）——**先取对数再指数**是 Γ 函数数值稳定的标准手法（Γ(15)≈4.4e10 已接近单精度上限）。

```
!file: phys/module_mp_thompson.F    L5349-L5377
L05349  !+---+-----------------------------------------------------------------+
L05350        REAL FUNCTION GAMMP(A,X)
L05351  !     --- COMPUTES THE INCOMPLETE GAMMA FUNCTION P(A,X)
L05352  !     --- SEE ABRAMOWITZ AND STEGUN 6.5.1
L05353  !     --- USES GCF,GSER
L05354        IMPLICIT NONE
L05355        REAL, INTENT(IN):: A,X
L05356        REAL:: GAMMCF,GAMSER,GLN
L05357        GAMMP = 0.
L05358        IF((X.LT.0.) .OR. (A.LE.0.)) THEN
L05359          PRINT *, 'BAD ARGUMENTS IN GAMMP'
L05360          RETURN
L05361        ELSEIF(X.LT.A+1.)THEN
L05362          CALL GSER(GAMSER,A,X,GLN)
L05363          GAMMP=GAMSER
L05364        ELSE
L05365          CALL GCF(GAMMCF,A,X,GLN)
L05366          GAMMP=1.-GAMMCF
L05367        ENDIF
L05368        END FUNCTION GAMMP
L05369  !  (C) Copr. 1986-92 Numerical Recipes Software 2.02
L05370  !+---+-----------------------------------------------------------------+
L05371        REAL FUNCTION WGAMMA(y)
L05372
L05373        IMPLICIT NONE
L05374        REAL, INTENT(IN):: y
L05375
L05376        WGAMMA = EXP(GAMMLN(y))
L05377
```

GAMMP 按 `x < a+1` 分派：小 x 用级数（GSER）、大 x 用 1− 连分式（GCF）——两式收敛域的标准分工。`WGAMMA = exp(GAMMLN(y))` 是 Γ(y) 的安全包装，模块常数段预计算全部经它。

07b.4 饱和混合比函数：RSLF / RSIF

```
!file: phys/module_mp_thompson.F    L5379-L5412
L05379  !+---+-----------------------------------------------------------------+
L05380  ! THIS FUNCTION CALCULATES THE LIQUID SATURATION VAPOR MIXING RATIO AS
L05381  ! A FUNCTION OF TEMPERATURE AND PRESSURE
L05382  !
L05383        REAL FUNCTION RSLF(P,T)
L05384
L05385        IMPLICIT NONE
L05386        REAL, INTENT(IN):: P, T
L05387        REAL:: ESL,X
L05388        REAL, PARAMETER:: C0= .611583699E03
L05389        REAL, PARAMETER:: C1= .444606896E02
L05390        REAL, PARAMETER:: C2= .143177157E01
L05391        REAL, PARAMETER:: C3= .264224321E-1
L05392        REAL, PARAMETER:: C4= .299291081E-3
L05393        REAL, PARAMETER:: C5= .203154182E-5
L05394        REAL, PARAMETER:: C6= .702620698E-8
L05395        REAL, PARAMETER:: C7= .379534310E-11
L05396        REAL, PARAMETER:: C8=-.321582393E-13
L05397
L05398        X=MAX(-80.,T-273.16)
L05399
L05400  !      ESL=612.2*EXP(17.67*X/(T-29.65))
L05401        ESL=C0+X*(C1+X*(C2+X*(C3+X*(C4+X*(C5+X*(C6+X*(C7+X*C8)))))))
L05402        ESL=MIN(ESL, P*0.15)        ! Even with P=1050mb and T=55C, the sat. vap. pres only contributes to ~15% of total
↪  pres.
L05403        RSLF=.622*ESL/(P-ESL)
L05404
L05405  !    ALTERNATIVE
L05406  !  ; Source: Murphy and Koop, Review of the vapour pressure of ice and
L05407  !             supercooled water for atmospheric applications, Q. J. R.
L05408  !             Meteorol. Soc (2005), 131, pp. 1539-1565.
L05409  !    ESL = EXP(54.842763 - 6763.22 / T - 4.210 * ALOG(T) + 0.000367 * T
L05410  !        + TANH(0.0415 * (T - 218.8)) * (53.878 - 1331.22
L05411  !        / T - 9.44523 * ALOG(T) + 0.014025 * T))
L05412
```

- **Flatau et al. (1992) 九阶多项式**：`es(T)` 直接拟合（单位 Pa，`C0=611.2` 是 0 °C 值）——相比积分式（书 (2.2.6)/(2.2.7) 的 Clausius–Clapeyron 积分）多项式在过冷水区（−80~0 °C）更准；`X = MAX(-80, T-273.16)` 设下限。
- **物理钳位** `ESL = MIN(ESL, 0.15*P)`：注释给出理由——P=1050 hPa、T=55 °C 时 es 也只占总压 ~15%，超出即非物理（防数值爆表）。
- **被注释的两个替代口径**：Bolton 指数式（`612.2*exp(17.67X/(T-29.65))`）与 Murphy & Koop (2005)——加上现行多项式，**同一个饱和比三套公式共存于 20 行内**。教材 ch11 的 “三口径” 结论在源码里就是这么写的；三口径的数值差异对照实验见教材 ch13 §13.9.2。
- `RSLF = 0.622*es/(p-es)`——比湿分母的口式（书 (2.2.11) 的混合比形态）。

```
!file: phys/module_mp_thompson.F    L5414-L5443
L05414  !+---+-----------------------------------------------------------------+
L05415  ! THIS FUNCTION CALCULATES THE ICE SATURATION VAPOR MIXING RATIO AS A
L05416  ! FUNCTION OF TEMPERATURE AND PRESSURE
L05417  !
L05418        REAL FUNCTION RSIF(P,T)
L05419
L05420        IMPLICIT NONE
L05421        REAL, INTENT(IN):: P, T
L05422        REAL:: ESI,X
L05423        REAL, PARAMETER:: C0= .609868993E03
L05424        REAL, PARAMETER:: C1= .499320233E02
L05425        REAL, PARAMETER:: C2= .184672631E01
L05426        REAL, PARAMETER:: C3= .402737184E-1
L05427        REAL, PARAMETER:: C4= .565392987E-3
L05428        REAL, PARAMETER:: C5= .521693933E-5
L05429        REAL, PARAMETER:: C6= .307839583E-7
L05430        REAL, PARAMETER:: C7= .105785160E-9
L05431        REAL, PARAMETER:: C8= .161444444E-12
L05432
L05433        X=MAX(-80.,T-273.16)
L05434        ESI=C0+X*(C1+X*(C2+X*(C3+X*(C4+X*(C5+X*(C6+X*(C7+X*C8))))))
L05435        ESI=MIN(ESI, P*0.15)
L05436        RSIF=.622*ESI/max(1.e-4,(P-ESI))
L05437
L05438  !    ALTERNATIVE
L05439  !  ; Source: Murphy and Koop, Review of the vapour pressure of ice and
L05440  !              supercooled water for atmospheric applications, Q. J. R.
L05441  !              Meteorol. Soc (2005), 131, pp. 1539-1565.
L05442  !     ESI = EXP(9.550426 - 5723.265/T + 3.53068*ALOG(T) - 0.00728332*T)
L05443
```

冰面版 `RSIF` 同构（系数不同）；**多了 `max(1.e-4, P-ESI)` 防除零**——极低温下 es→0 会使分母 →p（无危险），此保护防的是输入 p 本身异常的情况。Murphy & Koop 冰面替代式同样以注释形式在场。

07b.5 冰核化物理函数：iceDeMott / iceKoop / delta_p

```
!file: phys/module_mp_thompson.F    L5446-L5513
L05446  !+---+-----------------------------------------------------------------+
L05447        real function iceDeMott(tempc, qv, qvs, qvsi, rho, nifa)
L05448        implicit none
L05449
L05450        REAL, INTENT(IN):: tempc, qv, qvs, qvsi, rho, nifa
L05451
L05452  !..Local vars
L05453        REAL:: satw, sati, siw, p_x, si0x, dtt, dsi, dsw, dab, fc, hx
L05454        REAL:: ntilde, n_in, nmax, nhat, mux, xni, nifa_cc
L05455        REAL, PARAMETER:: p_c1    = 1000.
L05456        REAL, PARAMETER:: p_rho_c = 0.76
L05457        REAL, PARAMETER:: p_alpha = 1.0
L05458        REAL, PARAMETER:: p_gam   = 2.
L05459        REAL, PARAMETER:: delT    = 5.
L05460        REAL, PARAMETER:: T0x     = -40.
L05461        REAL, PARAMETER:: Sw0x    = 0.97
L05462        REAL, PARAMETER:: delSi   = 0.1
L05463        REAL, PARAMETER:: hdm     = 0.15
L05464        REAL, PARAMETER:: p_psi   = 0.058707*p_gam/p_rho_c
L05465        REAL, PARAMETER:: aap     = 1.
L05466        REAL, PARAMETER:: bbp     = 0.
L05467        REAL, PARAMETER:: y1p     = -35.
L05468        REAL, PARAMETER:: y2p     = -25.
L05469        REAL, PARAMETER:: rho_not0 = 101325./(287.05*273.15)
L05470
L05471  !+---+
L05472
L05473        xni = 0.0
L05474  !     satw = qv/qvs
L05475  !     sati = qv/qvsi
L05476  !     siw = qvs/qvsi
L05477  !     p_x = -1.0261+(3.1656e-3*tempc)+(5.3938e-4*(tempc*tempc))        &
L05478  !               +  (8.2584e-6*(tempc*tempc*tempc))
L05479  !     si0x = 1.+(10.**p_x)
L05480  !     if (sati.ge.si0x .and. satw.lt.0.985) then
L05481  !        dtt = delta_p (tempc, T0x, T0x+delT, 1., hdm)
L05482  !        dsi = delta_p (sati, Si0x, Si0x+delSi, 0., 1.)
L05483  !        dsw = delta_p (satw, Sw0x, 1., 0., 1.)
L05484  !        fc = dtt*dsi*0.5
L05485  !        hx = min(fc+((1.-fc)*dsw), 1.)
L05486  !        ntilde = p_c1*p_gam*((exp(12.96*(sati-1.1)))**0.3) / p_rho_c
L05487  !        if (tempc .le. y1p) then
L05488  !           n_in = ntilde
L05489  !        elseif (tempc .ge. y2p) then
```

```
L05490  !           n_in = p_psi*p_c1*exp(12.96*(sati-1.)-0.639)
L05491  !        else
L05492  !           if (tempc .le. -30.) then
L05493  !              nmax = p_c1*p_gam*(exp(12.96*(siw-1.)))**0.3/p_rho_c
L05494  !           else
L05495  !              nmax = p_psi*p_c1*exp(12.96*(siw-1.)-0.639)
L05496  !           endif
L05497  !           ntilde = MIN(ntilde, nmax)
L05498  !           nhat = MIN(p_psi*p_c1*exp(12.96*(sati-1.)-0.639), nmax)
L05499  !           dab = delta_p (tempc, y1p, y2p, aap, bbp)
L05500  !           n_in = MIN(nhat*(ntilde/nhat)**dab, nmax)
L05501  !        endif
L05502  !        mux = hx*p_alpha*n_in*rho
L05503  !        xni = mux*((6700.*nifa)-200.)/((6700.*5.E5)-200.)
L05504  !     elseif (satw.ge.0.985 .and. tempc.gt.HGFR-273.15) then
L05505           nifa_cc = MAX(0.5, nifa*RHO_NOT0*1.E-6/rho)
L05506  !        xni  = 3.*nifa_cc**(1.25)*exp((0.46*(-tempc))-11.6)            !  [DeMott, 2015]
L05507           xni = (5.94e-5*(-tempc)**3.33)                              & !  [DeMott, 2010]
L05508                    * (nifa_cc**((-0.0264*(tempc))+0.0033))
L05509           xni = xni*rho/RHO_NOT0 * 1000.
L05510  !     endif
L05511
L05512        iceDeMott = MAX(0., xni)
L05513
```

- **大段注释代码 = Phillips et al. (2008) 完整实现**（`delta_p` 辅助函数就是为它写的）——现行代码只保留 DeMott (2010)。**两个时代的冰核化参数化叠在同一函数里**，读时注意！的范围。
- **现行式**（L5506–L5508）：`xni = 5.94e-5 • (−Tc)^3.33 • nifa_cc^(−0.0264 • Tc+0.0033)`——温度三次方幂律 × 气溶胶数的**温度依赖指数**（越冷对气溶胶越敏感，这是 DeMott 2010 对 Cooper 纯温度式的本质升级——把 “尘埃越多冰核越多” 定量化）；`nifa_cc` 是换算到标准态的气溶胶数（cm^{-3}）。
- 被注释的 `[DeMott, 2015]` 更新式（`3 • nifa^1.25 • exp(0.46 • (−T)−11.6)`）——**论文谱系 2010→2015→(被弃的 2008) 三代同堂**。
- 返回前单位还原：`xni • ρ/ρ_not0 • 1000`（cm^{-3}→m^{-3} 的密度订正）。

```
!file: phys/module_mp_thompson.F    L5516-L5545
L05516  !+---+-----------------------------------------------------------------+
L05517  !..Newer research since Koop et al (2001) suggests that the freezing
L05518  !.. rate should be lower than original paper, so J_rate is reduced
L05519  !.. by two orders of magnitude.
L05520
L05521        real function iceKoop(temp, qv, qvs, naero, dt)
L05522        implicit none
L05523
L05524        REAL, INTENT(IN):: temp, qv, qvs, naero, DT
L05525        REAL:: mu_diff, a_w_i, delta_aw, log_J_rate, J_rate, prob_h, satw
L05526        REAL:: xni
L05527
L05528        xni = 0.0
L05529        satw = qv/qvs
L05530        mu_diff    = 210368.0 + (131.438*temp) - (3.32373E6/temp)        &
L05531       &           - (41729.1*alog(temp))
L05532        a_w_i      = exp(mu_diff/(R_uni*temp))
L05533        delta_aw   = satw - a_w_i
L05534        log_J_rate = -906.7 + (8502.0*delta_aw)                          &
L05535       &           - (26924.0*delta_aw*delta_aw)                        &
L05536       &           + (29180.0*delta_aw*delta_aw*delta_aw)
L05537        log_J_rate = MIN(20.0, log_J_rate)
L05538        J_rate     = 10.**log_J_rate                                      ! cm-3 s-1
L05539        prob_h     = MIN(1.-exp(-J_rate*ar_volume*DT), 1.)
L05540        if (prob_h .gt. 0.) then
L05541           xni     = MIN(prob_h*naero, 1000.E3)
L05542        endif
L05543
L05544        iceKoop = MAX(0.0, xni)
L05545
```

- **Koop (2001, Nature) 均质冻结**：核心变量是**水活度差** `delta_aw = satw − a_w_i`——过冷水滴相对冰的水活度超出量（`a_w_i` 由冰的化学势差多项式 `mu_diff` 算）。冻结率 `log_J = −906.7 + 8502Δaw − 26924Δaw² + 29180Δaw³`（$cm^{-3}s^{-1}$）——**J 对 Δaw 的立方多项式在 Δaw≈0.25–0.35 区间跨越十几个量级**，这就是 “极冷 +强过饱和才均质成冰” 的定量来源。
- **两处调参如实读**：① 模块头注释声明 “新研究（2001 后）认为原论文冻结率应低两个量级”——现行多项式系数即已调低后的版本（源码未再乘 0.01，调低藏在系数里）；②`MIN(20, log_J_rate)` 纯数值防溢出。
- **泊松冻结**（L5538）：`prob = 1 − exp(−J • ar_volume • dt)`——单滴体积 `ar_volume` 用模块常数段的 2.5 μm 半径假设；`prob • naero` 即本步冻结的气溶胶数，上限 1000/L。

```
!file: phys/module_mp_thompson.F    L5548-L5579
L05548  !+---+-----------------------------------------------------------------+
L05549  !.. Helper routine for Phillips et al (2008) ice nucleation.  Trude
L05550
L05551        REAL FUNCTION delta_p (yy, y1, y2, aa, bb)
L05552        IMPLICIT NONE
L05553
L05554        REAL, INTENT(IN):: yy, y1, y2, aa, bb
L05555        REAL:: dab, A, B, a0, a1, a2, a3
L05556
L05557        A   = 6.*(aa-bb)/((y2-y1)*(y2-y1)*(y2-y1))
```

```
L05558         B   = aa+(A*y1*y1*y1/6.)-(A*y1*y1*y2*0.5)
L05559         a0  = B
L05560         a1  = A*y1*y2
L05561         a2  = -A*(y1+y2)*0.5
L05562         a3  = A/3.
L05563
L05564         if (yy.le.y1) then
L05565            dab = aa
L05566         else if (yy.ge.y2) then
L05567            dab = bb
L05568         else
L05569            dab = a0+(a1*yy)+(a2*yy*yy)+(a3*yy*yy*yy)
L05570         endif
L05571
L05572         if (dab.lt.aa) then
L05573            dab = aa
L05574         endif
L05575         if (dab.gt.bb) then
L05576            dab = bb
L05577         endif
L05578         delta_p = dab
L05579
```

`delta_p` 是**平滑阶跃函数**（y1→y2 区间内从 aa 到 bb 的三次多项式插值、区间外取端值、钳位单调）——Phillips (2008) 方案的开关平滑器。现行活跃代码不再调用它（调用点全在被注释的 Phillips 块里），但函数本体保留——**孤儿函数**，读代码时知道其用途即可。

07b.6 辐射与雷达接口

```
!file: phys/module_mp_thompson.F    L5585-L5698
L05585  !+---+-----------------------------------------------------------------+
L05586  !..Compute _radiation_ effective radii of cloud water, ice, and snow.
L05587  !.. These are entirely consistent with microphysics assumptions, not
L05588  !.. constant or otherwise ad hoc as is internal to most radiation
L05589  !.. schemes.  Since only the smallest snowflakes should impact
L05590  !.. radiation, compute from first portion of complicated Field number
L05591  !.. distribution, not the second part, which is the larger sizes.
L05592  !+---+-----------------------------------------------------------------+
L05593
L05594        subroutine calc_effectRad (t1d, p1d, qv1d, qc1d, nc1d, qi1d, ni1d, qs1d,   &
L05595       &                re_qc1d, re_qi1d, re_qs1d, kts, kte)
L05596
L05597        IMPLICIT NONE
L05598
L05599  !..Sub arguments
L05600        INTEGER, INTENT(IN):: kts, kte
L05601        REAL, DIMENSION(kts:kte), INTENT(IN)::                              &
L05602       &                    t1d, p1d, qv1d, qc1d, nc1d, qi1d, ni1d, qs1d
L05603        REAL, DIMENSION(kts:kte), INTENT(INOUT):: re_qc1d, re_qi1d, re_qs1d
L05604  !..Local variables
L05605        INTEGER:: k
L05606        REAL, DIMENSION(kts:kte):: rho, rc, nc, ri, ni, rs
L05607        REAL:: smo2, smob, smoc
L05608        REAL:: tc0, loga_, a_, b_
L05609        DOUBLE PRECISION:: lamc, lami
L05610        LOGICAL:: has_qc, has_qi, has_qs
L05611        INTEGER:: inu_c
L05612        real, dimension(15), parameter:: g_ratio = (/24,60,120,210,336,   &
L05613       &                504,720,990,1320,1716,2184,2730,3360,4080,4896/)
L05614
L05615        has_qc = .false.
L05616        has_qi = .false.
L05617        has_qs = .false.
L05618
L05619        re_qc1d(:) = RE_QC_BG
L05620        re_qi1d(:) = RE_QI_BG
L05621        re_qs1d(:) = RE_QS_BG
L05622
L05623        do k = kts, kte
L05624           rho(k) = 0.622*p1d(k)/(R*t1d(k)*(qv1d(k)+0.622))
L05625           rc(k) = MAX(R1, qc1d(k)*rho(k))
L05626           nc(k) = MAX(2., MIN(nc1d(k)*rho(k), Nt_c_max))
L05627           if (.NOT. is_aerosol_aware) nc(k) = Nt_c
L05628           if (rc(k).gt.R1 .and. nc(k).gt.R2) has_qc = .true.
L05629           ri(k) = MAX(R1, qi1d(k)*rho(k))
L05630           ni(k) = MAX(R2, ni1d(k)*rho(k))
L05631           if (ri(k).gt.R1 .and. ni(k).gt.R2) has_qi = .true.
L05632           rs(k) = MAX(R1, qs1d(k)*rho(k))
L05633           if (rs(k).gt.R1) has_qs = .true.
L05634        enddo
L05635
L05636        if (has_qc) then
L05637        do k = kts, kte
L05638           if (rc(k).le.R1 .or. nc(k).le.R2) CYCLE
L05639           if (nc(k).lt.100) then
L05640              inu_c = 15
L05641           elseif (nc(k).gt.1.E10) then
L05642              inu_c = 2
L05643           else
L05644              inu_c = MIN(15, NINT(1000.E6/nc(k)) + 2)
L05645           endif
L05646           lamc = (nc(k)*am_r*g_ratio(inu_c)/rc(k))**obmr
L05647           re_qc1d(k) = MAX(2.51E-6, MIN(SNGL(0.5D0 * DBLE(3.+inu_c)/lamc), 50.E-6))
L05648        enddo
L05649        endif
L05650
L05651        if (has_qi) then
L05652        do k = kts, kte
L05653           if (ri(k).le.R1 .or. ni(k).le.R2) CYCLE
L05654           lami = (am_i*cig(2)*oig1*ni(k)/ri(k))**obmi
L05655           re_qi1d(k) = MAX(2.51E-6, MIN(SNGL(0.5D0 * DBLE(3.+mu_i)/lami), 125.E-6))
```

```
L05656        enddo
L05657        endif
L05658
L05659        if (has_qs) then
L05660        do k = kts, kte
L05661           if (rs(k).le.R1) CYCLE
L05662           tc0 = MIN(-0.1, t1d(k)-273.15)
L05663           smob = rs(k)*oams
L05664
L05665  !..All other moments based on reference, 2nd moment.  If bm_s.ne.2,
L05666  !.. then we must compute actual 2nd moment and use as reference.
L05667           if (bm_s.gt.(2.0-1.e-3) .and. bm_s.lt.(2.0+1.e-3)) then
L05668              smo2 = smob
L05669           else
L05670              loga_ = sa(1) + sa(2)*tc0 + sa(3)*bm_s &
L05671       &         + sa(4)*tc0*bm_s + sa(5)*tc0*tc0 &
L05672       &         + sa(6)*bm_s*bm_s + sa(7)*tc0*tc0*bm_s &
L05673       &         + sa(8)*tc0*bm_s*bm_s + sa(9)*tc0*tc0*tc0 &
L05674       &         + sa(10)*bm_s*bm_s*bm_s
L05675              a_ = 10.0**loga_
L05676              b_ = sb(1) + sb(2)*tc0 + sb(3)*bm_s &
L05677       &         + sb(4)*tc0*bm_s + sb(5)*tc0*tc0 &
L05678       &         + sb(6)*bm_s*bm_s + sb(7)*tc0*tc0*bm_s &
L05679       &         + sb(8)*tc0*bm_s*bm_s + sb(9)*tc0*tc0*tc0 &
L05680       &         + sb(10)*bm_s*bm_s*bm_s
L05681              smo2 = (smob/a_)**(1./b_)
L05682           endif
L05683  !..Calculate bm_s+1 (th) moment.  Useful for diameter calcs.
L05684           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(1) &
L05685       &         + sa(4)*tc0*cse(1) + sa(5)*tc0*tc0 &
L05686       &         + sa(6)*cse(1)*cse(1) + sa(7)*tc0*tc0*cse(1) &
L05687       &         + sa(8)*tc0*cse(1)*cse(1) + sa(9)*tc0*tc0*tc0 &
L05688       &         + sa(10)*cse(1)*cse(1)*cse(1)
L05689           a_ = 10.0**loga_
L05690           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(1) + sb(4)*tc0*cse(1) &
L05691       &         + sb(5)*tc0*tc0 + sb(6)*cse(1)*cse(1) &
L05692       &         + sb(7)*tc0*tc0*cse(1) + sb(8)*tc0*cse(1)*cse(1) &
L05693       &         + sb(9)*tc0*tc0*tc0 + sb(10)*cse(1)*cse(1)*cse(1)
L05694           smoc = a_ * smo2**b_
L05695           re_qs1d(k) = MAX(5.01E-6, MIN(0.5*(smoc/smob), 999.E-6))
L05696        enddo
L05697        endif
L05698
```

- **设计原则**（模块注释原文）：“entirely consistent with microphysics assumptions, not constant or otherwise ad hoc”——多数辐射方案的有效半径是内部 ad hoc 常数，Thompson 从谱假设直接算，闭合教材 ch12 埋下的接口伏笔（对应讨论见教材 ch13 §13.7.6）。
- **云水有效半径**（L5645–L5646）：`re = 0.5•(3+nu_c)/lamc`——gamma 谱的有效半径解析式 `r_eff = ∫r³N dr / ∫r²N dr = Γ(4+mu)/(Γ(3+mu)•λ)•(1/2)`（直径 → 半径乘 0.5）；`g_ratio` 是预存的 Γ 比值整数列。下限 2.51 μm、上限 50 μm。
- **冰晶**：`mu_i=0` 退化为 `0.5•3/lami`，上限 125 μm；**雪**：`0.5•smoc/smob`（bm_s+1 矩/2 矩即质量加权有效径，Field 谱矩量法再现），下限 5.01 μm、上限 999 μm。**三物种同一公式思想、各自谱形实现**——对照主实体谱参数化段（code_07a §07a.8.2）阅读。
- `has_qc/has_qi/has_qs` 三标志先扫描再分支——整柱无该物种时整段跳过（与主实体 `no_micro` 同思路）。

```
!file: phys/module_mp_thompson.F    L5701-L6027
L05701  !+---+-----------------------------------------------------------------+
L05702  !..Compute radar reflectivity assuming 10 cm wavelength radar and using
L05703  !.. Rayleigh approximation.  Only complication is melted snow/graupel
L05704  !.. which we treat as water-coated ice spheres and use Uli Blahak's
L05705  !.. library of routines.  The meltwater fraction is simply the amount
L05706  !.. of frozen species remaining from what initially existed at the
L05707  !.. melting level interface.
L05708  !+---+-----------------------------------------------------------------+
L05709
L05710        subroutine calc_refl10cm (qv1d, qc1d, qr1d, nr1d, qs1d, qg1d,     &
L05711                                  ng1d, qb1d, t1d, p1d, dBZ, kts, kte, ii, jj, ke_diag)
L05712
L05713        IMPLICIT NONE
L05714
L05715  !..Sub arguments
L05716        INTEGER, INTENT(IN):: kts, kte, ii, jj, ke_diag
L05717        REAL, DIMENSION(kts:kte), INTENT(IN)::                            &
L05718             qv1d, qc1d, qr1d, nr1d, qs1d, qg1d, ng1d, qb1d, t1d, p1d
L05719        REAL, DIMENSION(kts:kte), INTENT(INOUT):: dBZ
L05720  !     REAL, DIMENSION(kts:kte), INTENT(INOUT):: vt_dBZ
L05721
L05722  !..Local variables
L05723        REAL, DIMENSION(kts:kte):: temp, pres, qv, rho, rhof
L05724        REAL, DIMENSION(kts:kte):: rc, rr, nr, rs, rg, ng, rb
L05725
L05726        DOUBLE PRECISION, DIMENSION(kts:kte):: ilamr, ilamg, N0_r, N0_g
L05727        REAL, DIMENSION(kts:kte):: mvd_r
L05728        REAL, DIMENSION(kts:kte):: smob, smo2, smoc, smoz
L05729        REAL:: oM3, M0, Mrat, slam1, slam2, xDs
L05730        REAL:: ils1, ils2, t1_vts, t2_vts, t3_vts, t4_vts
L05731        REAL:: vtr_dbz_wt, vts_dbz_wt, vtg_dbz_wt
L05732
L05733        REAL, DIMENSION(kts:kte):: ze_rain, ze_snow, ze_graupel
L05734        INTEGER, DIMENSION(kts:kte):: idx_bg
L05735
L05736        DOUBLE PRECISION:: N0_exp, N0_min, lam_exp, lamr, lamg
L05737        REAL:: a_, b_, loga_, tc0
L05738        DOUBLE PRECISION:: fmelt_s, fmelt_g
L05739
L05740        INTEGER:: i, k, k_0, kbot, n, ktop
L05741        LOGICAL:: melti
```

```
L05742        LOGICAL, DIMENSION(kts:kte):: L_qr, L_qs, L_qg
L05743
L05744        DOUBLE PRECISION:: cback, x, eta, f_d
L05745        REAL:: xslw1, ygra1, zans1
L05746        INTEGER :: k_0loop
L05747
L05748  !+---+
L05749
L05750        do k = kts, kte
L05751           dBZ(k) = -35.0
L05752        enddo
L05753
L05754  !+---+-----------------------------------------------------------------+
L05755  !..Put column of data into local arrays.
L05756  !+---+-----------------------------------------------------------------+
L05757
L05758        do k = kts, kte
L05759           temp(k) = t1d(k)
L05760           qv(k) = MAX(1.E-10, qv1d(k))
L05761           pres(k) = p1d(k)
L05762           rho(k) = 0.622*pres(k)/(R*temp(k)*(qv(k)+0.622))
L05763           rhof(k) = SQRT(RHO_NOT/rho(k))
L05764           rc(k) = MAX(R1, qc1d(k)*rho(k))
L05765           if (qr1d(k) .gt. R1) then
L05766              rr(k) = qr1d(k)*rho(k)
L05767              nr(k) = MAX(R2, nr1d(k)*rho(k))
L05768              lamr = (am_r*crg(3)*org2*nr(k)/rr(k))**obmr
L05769              ilamr(k) = 1./lamr
L05770              N0_r(k) = nr(k)*org2*lamr**cre(2)
L05771              mvd_r(k) = (3.0 + mu_r + 0.672) * ilamr(k)
L05772              L_qr(k) = .true.
L05773           else
L05774              rr(k) = R1
L05775              nr(k) = R1
L05776              mvd_r(k) = 50.E-6
L05777              L_qr(k) = .false.
L05778           endif
L05779           if (qs1d(k) .gt. R2) then
L05780              rs(k) = qs1d(k)*rho(k)
L05781              L_qs(k) = .true.
L05782           else
L05783              rs(k) = R1
L05784              L_qs(k) = .false.
L05785           endif
L05786           if (qg1d(k) .gt. R2) then
L05787              rg(k) = qg1d(k)*rho(k)
L05788              ng(k) = MAX(R2, ng1d(k)*rho(k))
L05789              rb(k) = MAX(qg1d(k)/rho_g(NRHG), qb1d(k))
L05790              rb(k) = MIN(qg1d(k)/rho_g(1), rb(k))
L05791              idx_bg(k) = MAX(1,MIN(NINT(qg1d(k)/rb(k) *0.01)+1,NRHG))
L05792              if (.not. is_hail_aware) idx_bg(k) = idx_bg1
L05793              L_qg(k) = .true.
L05794           else
L05795              rg(k) = R1
L05796              ng(k) = R2
L05797              idx_bg(k) = idx_bg1
L05798              L_qg(k) = .false.
L05799           endif
L05800        enddo
L05801
L05802  !+---+-----------------------------------------------------------------+
L05803  !..Calculate y-intercept, slope, and useful moments for snow.
L05804  !+---+-----------------------------------------------------------------+
L05805        do k = kts, kte
L05806           smo2(k) = 0.
L05807           smob(k) = 0.
L05808           smoc(k) = 0.
L05809           smoz(k) = 0.
L05810        enddo
L05811        if ( ( ke_diag > kts .and. ANY(L_qs .eqv. .true.) ) .or.  &
L05812             (ke_diag == kts .and. L_qs(kts) .eqv. .true. ) ) then
L05813        do k = kts, ke_diag ! kte
L05814           if (.not. L_qs(k)) CYCLE
L05815           tc0 = MIN(-0.1, temp(k)-273.15)
L05816           smob(k) = rs(k)*oams
L05817
L05818  !..All other moments based on reference, 2nd moment.  If bm_s.ne.2,
L05819  !.. then we must compute actual 2nd moment and use as reference.
L05820           if (bm_s.gt.(2.0-1.e-3) .and. bm_s.lt.(2.0+1.e-3)) then
L05821              smo2(k) = smob(k)
L05822           else
L05823              loga_ = sa(1) + sa(2)*tc0 + sa(3)*bm_s &
L05824       &         + sa(4)*tc0*bm_s + sa(5)*tc0*tc0 &
L05825       &         + sa(6)*bm_s*bm_s + sa(7)*tc0*tc0*bm_s &
L05826       &         + sa(8)*tc0*bm_s*bm_s + sa(9)*tc0*tc0*tc0 &
L05827       &         + sa(10)*bm_s*bm_s*bm_s
L05828              a_ = 10.0**loga_
L05829              b_ = sb(1) + sb(2)*tc0 + sb(3)*bm_s &
L05830       &         + sb(4)*tc0*bm_s + sb(5)*tc0*tc0 &
L05831       &         + sb(6)*bm_s*bm_s + sb(7)*tc0*tc0*bm_s &
L05832       &         + sb(8)*tc0*bm_s*bm_s + sb(9)*tc0*tc0*tc0 &
L05833       &         + sb(10)*bm_s*bm_s*bm_s
L05834              smo2(k) = (smob(k)/a_)**(1./b_)
L05835           endif
L05836
L05837  !..Calculate bm_s+1 (th) moment.  Useful for diameter calcs.
L05838           loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(1) &
L05839       &         + sa(4)*tc0*cse(1) + sa(5)*tc0*tc0 &
L05840       &         + sa(6)*cse(1)*cse(1) + sa(7)*tc0*tc0*cse(1) &
L05841       &         + sa(8)*tc0*cse(1)*cse(1) + sa(9)*tc0*tc0*tc0 &
L05842       &         + sa(10)*cse(1)*cse(1)*cse(1)
L05843           a_ = 10.0**loga_
L05844           b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(1) + sb(4)*tc0*cse(1) &
L05845       &        + sb(5)*tc0*tc0 + sb(6)*cse(1)*cse(1) &
L05846       &        + sb(7)*tc0*tc0*cse(1) + sb(8)*tc0*cse(1)*cse(1) &
L05847       &        + sb(9)*tc0*tc0*tc0 + sb(10)*cse(1)*cse(1)*cse(1)
L05848           smoc(k) = a_ * smo2(k)**b_
```

```
L05849
L05850  !..Calculate bm_s*2 (th) moment.  Useful for reflectivity.
L05851          loga_ = sa(1) + sa(2)*tc0 + sa(3)*cse(3) &
L05852       &        + sa(4)*tc0*cse(3) + sa(5)*tc0*tc0 &
L05853       &        + sa(6)*cse(3)*cse(3) + sa(7)*tc0*tc0*cse(3) &
L05854       &        + sa(8)*tc0*cse(3)*cse(3) + sa(9)*tc0*tc0*tc0 &
L05855       &        + sa(10)*cse(3)*cse(3)*cse(3)
L05856          a_ = 10.0**loga_
L05857          b_ = sb(1)+ sb(2)*tc0 + sb(3)*cse(3) + sb(4)*tc0*cse(3) &
L05858       &       + sb(5)*tc0*tc0 + sb(6)*cse(3)*cse(3) &
L05859       &       + sb(7)*tc0*tc0*cse(3) + sb(8)*tc0*cse(3)*cse(3) &
L05860       &       + sb(9)*tc0*tc0*tc0 + sb(10)*cse(3)*cse(3)*cse(3)
L05861          smoz(k) = a_ * smo2(k)**b_
L05862        enddo
L05863        endif
L05864
L05865  !+---+-----------------------------------------------------------------+
L05866  !..Calculate y-intercept, slope values for graupel.
L05867  !+---+-----------------------------------------------------------------+
L05868
L05869        if (ANY(L_qg .eqv. .true.)) then
L05870        do k = kte, kts, -1
L05871           lamg = (am_g(idx_bg(k))*cgg(3,1)*ogg2*ng(k)/rg(k))**obmg
L05872           ilamg(k) = 1./lamg
L05873           N0_g(k) = ng(k)*ogg2*lamg**cge(2,1)
L05874        enddo
L05875        else
L05876           ilamg(:) = 0.
L05877           N0_g(:) = 0.
L05878        endif
L05879
L05880  !+---+-----------------------------------------------------------------+
L05881  !..Locate K-Level of start of melting (k_0 is level above).
L05882  !+---+-----------------------------------------------------------------+
L05883        melti = .false.
L05884        k_0 = kts
L05885        do k = kte-1, kts, -1
L05886           if ( (temp(k).gt.273.15) .and. L_qr(k)                      &
L05887        &                           .and. (L_qs(k+1).or.L_qg(k+1)) ) then
L05888              k_0 = MAX(k+1, k_0)
L05889              melti=.true.
L05890              goto 195
L05891           endif
L05892        enddo
L05893   195  continue
L05894
L05895  ! Set loop limit for wet ice according to whether the full 3D field is needed or just k=1
L05896        k_0loop = Min(k_0, ke_diag+1)
L05897
L05898  !+---+-----------------------------------------------------------------+
L05899  !..Do not do the so-called bright band if using the variable density
L05900  !.. graupel-hail category since the density increases during melting and
L05901  !.. will account for bright band behavior explicitly.
L05902  !+---+-----------------------------------------------------------------+
L05903  !     if (is_hail_aware) melti = .false.
L05904
L05905  !+---+-----------------------------------------------------------------+
L05906  !..Assume Rayleigh approximation at 10 cm wavelength. Rain (all temps)
L05907  !.. and non-water-coated snow and graupel when below freezing are
L05908  !.. simple. Integrations of m(D)*m(D)*N(D)*dD.
L05909  !+---+-----------------------------------------------------------------+
L05910
L05911        do k = kts, ke_diag ! kte
L05912           ze_rain(k) = 1.e-22
L05913           ze_snow(k) = 1.e-22
L05914           ze_graupel(k) = 1.e-22
L05915           if (L_qr(k)) ze_rain(k) = N0_r(k)*crg(4)*ilamr(k)**cre(4)
L05916           if (L_qs(k)) ze_snow(k) = (0.176/0.93) * (6.0/PI)*(6.0/PI)     &
L05917        &                          * (am_s/900.0)*(am_s/900.0)*smoz(k)
L05918           if (L_qg(k)) ze_graupel(k) = (0.176/0.93) * (6.0/PI)*(6.0/PI)  &
L05919        &               * (am_g(idx_bg(k))/900.0)*(am_g(idx_bg(k))/900.0)  &
L05920        &               * N0_g(k)*cgg(4,1)*ilamg(k)**cge(4,1)
L05921        enddo
L05922
L05923  !+---+-----------------------------------------------------------------+
L05924  !..Special case of melting ice (snow/graupel) particles.  Assume the
L05925  !.. ice is surrounded by the liquid water.  Fraction of meltwater is
L05926  !.. extremely simple based on amount found above the melting level.
L05927  !.. Uses code from Uli Blahak (rayleigh_soak_wetgraupel and supporting
L05928  !.. routines).
L05929  !+---+-----------------------------------------------------------------+
L05930
L05931        if (.not. iiwarm .and. melti .and. k_0.ge.2) then
L05932         do k = k_0loop-1, kts, -1
L05933
L05934  !..Reflectivity contributed by melting snow
L05935           if (L_qs(k) .and. L_qs(k_0) ) then
L05936            fmelt_s = MAX(0.05d0, MIN(1.0d0-rs(k)/rs(k_0), 0.99d0))
L05937            eta = 0.d0
L05938            oM3 = 1./smoc(k)
L05939            M0 = (smob(k)*oM3)
L05940            Mrat = smob(k)*M0*M0*M0
L05941            slam1 = M0 * Lam0
L05942            slam2 = M0 * Lam1
L05943            do n = 1, nrbins
L05944               x = am_s * xxDs(n)**bm_s
L05945               call rayleigh_soak_wetgraupel (x, DBLE(ocms), DBLE(obms), &
L05946        &              fmelt_s, melt_outside_s, m_w_0, m_i_0, lamda_radar, &
L05947        &              CBACK, mixingrulestring_s, matrixstring_s,          &
L05948        &              inclusionstring_s, hoststring_s,                    &
L05949        &              hostmatrixstring_s, hostinclusionstring_s)
L05950               f_d = Mrat*(Kap0*DEXP(-slam1*xxDs(n))                       &
L05951        &              + Kap1*(M0*xxDs(n))**mu_s * DEXP(-slam2*xxDs(n)))
L05952               eta = eta + f_d * CBACK * simpson(n) * xdts(n)
L05953            enddo
L05954            ze_snow(k) = SNGL(lamda4 / (pi5 * K_w) * eta)
L05955           endif
L05956
```

```
L05957  !..Reflectivity contributed by melting graupel
L05958
L05959  !         if (L_qg(k) .and. L_qg(k_0) ) then
L05960  !          fmelt_g = MAX(0.05d0, MIN(1.0d0-rg(k)/rg(k_0), 0.99d0))
L05961  !          eta = 0.d0
L05962  !          lamg = 1./ilamg(k)
L05963  !          do n = 1, nrbins
L05964  !             x = am_g(idx_bg(k)) * xxDg(n)**bm_g
L05965  !             call rayleigh_soak_wetgraupel (x, DBLE(ocmg(idx_bg(k))), DBLE(obmg), &
L05966  !    &             fmelt_g, melt_outside_g, m_w_0, m_i_0, lamda_radar, &
L05967  !    &             CBACK, mixingrulestring_g, matrixstring_g,          &
L05968  !    &             inclusionstring_g, hoststring_g,                    &
L05969  !    &             hostmatrixstring_g, hostinclusionstring_g)
L05970  !             f_d = N0_g(k)*xxDg(n)**mu_g * DEXP(-lamg*xxDg(n))
L05971  !             eta = eta + f_d * CBACK * simpson(n) * xdtg(n)
L05972  !          enddo
L05973  !          ze_graupel(k) = SNGL(lamda4 / (pi5 * K_w) * eta)
L05974  !         endif
L05975
L05976          enddo
L05977        endif
L05978
L05979        do k = ke_diag, kts, -1
L05980           dBZ(k) = 10.*log10((ze_rain(k)+ze_snow(k)+ze_graupel(k))*1.d18)
L05981        enddo
L05982
L05983
L05984  !..Reflectivity-weighted terminal velocity (snow, rain, graupel, mix).
L05985  !     do k = kte, kts, -1
L05986  !        vt_dBZ(k) = 1.E-3
L05987  !        if (rs(k).gt.R2) then
L05988  !         Mrat = smob(k) / smoc(k)
L05989  !         ils1 = 1./(Mrat*Lam0 + fv_s)
L05990  !         ils2 = 1./(Mrat*Lam1 + fv_s)
L05991  !         t1_vts = Kap0*csg(5)*ils1**cse(5)
L05992  !         t2_vts = Kap1*Mrat**mu_s*csg(11)*ils2**cse(11)
L05993  !         ils1 = 1./(Mrat*Lam0)
L05994  !         ils2 = 1./(Mrat*Lam1)
L05995  !         t3_vts = Kap0*csg(6)*ils1**cse(6)
L05996  !         t4_vts = Kap1*Mrat**mu_s*csg(12)*ils2**cse(12)
L05997  !         vts_dbz_wt = rhof(k)*av_s * (t1_vts+t2_vts)/(t3_vts+t4_vts)
L05998  !         if (temp(k).ge.273.15 .and. temp(k).lt.275.15) then
L05999  !            vts_dbz_wt = vts_dbz_wt*1.5
L06000  !         elseif (temp(k).ge.275.15) then
L06001  !            vts_dbz_wt = vts_dbz_wt*2.0
L06002  !         endif
L06003  !        else
L06004  !         vts_dbz_wt = 1.E-3
L06005  !        endif
L06006
L06007  !        if (rr(k).gt.R1) then
L06008  !         lamr = 1./ilamr(k)
L06009  !         vtr_dbz_wt = rhof(k)*av_r*crg(13)*(lamr+fv_r)**(-cre(13))      &
L06010  !    &               / (crg(4)*lamr**(-cre(4)))
L06011  !        else
L06012  !         vtr_dbz_wt = 1.E-3
L06013  !        endif
L06014
L06015  !        if (rg(k).gt.R2) then
L06016  !         lamg = 1./ilamg(k)
L06017  !         vtg_dbz_wt = rhof(k)*av_g(idx_bg(k))*cgg(5,idx_bg(k))*lamg**(-cge(5,idx_bg(k)))              &
L06018  !    &               / (cgg(4,1)*lamg**(-cge(4,1)))
L06019  !        else
L06020  !         vtg_dbz_wt = 1.E-3
L06021  !        endif
L06022
L06023  !        vt_dBZ(k) = (vts_dbz_wt*ze_snow(k) + vtr_dbz_wt*ze_rain(k)      &
L06024  !    &                + vtg_dbz_wt*ze_graupel(k))                       &
L06025  !    &               / (ze_rain(k)+ze_snow(k)+ze_graupel(k))
L06026  !     enddo
L06027
```

- **Rayleigh 近似**（L5910–L5920）：ze ∝ ∫D⁶·N(D)dD——雨直接积分（cre(4)=7，D⁶ 的谱矩）；雪/霰乘三组修正：0.176/0.93（冰与水的介电因子比 |K_i|²/|K_w|²）、(6/π)²（质量 → 直径的球化换算）、am/900（密度归一）。**Rayleigh 10 cm 近似对小滴精确、对大雹欠准**——注释明说这是工程折中。
- **融化亮带**（L5882–L5892 定位 + L5930–L5976 计算）：融化层判定 k_0 =“暖层有雨且其上一层有雪/霰”的最高层；融水份额 fmelt_s = 1 − rs(k)/rs(k_0)（**用雪含量沿程损耗定义融化进度**——含量守恒的巧妙用法，初值钳 0.05 下限）；然后按 Field 双伽马谱逐 bin 调 rayleigh_soak_wetgraupel（Uli Blahak 库，module_mp_radar 提供——水包冰核的 Maxwell-Garnett 混合介质后向散射），Simpson 权重积分成 η，ze = λ⁴/(π⁵·|K_w|²)·η。**亮带（雷达回波融化增强）是雷达气象的经典现象**，此处是它的模式内模拟。
- 被注释的霰亮带块（L5958 起）与 is_hail_aware 时关闭亮带的开关（L5902）——三矩方案密度随融化演变，亮带效应“已显式包含”，无需参数化。
- **ke_diag**：诊断层上限（只算需要输出的层）——calc_refl10cm 是诊断量，裁剪计算量全靠它。
- 大段注释的 vt_dBZ（反射率加权末速度）：完整实现已在注释里，含雪落速的融化层增强（1.5×/2×）——**又一个“注释里埋着完整功能”的例子**。

07b.7 Bolton (1980) 热力学函数族

湿球温度反演链 `t_dew → t_lcl → theta_e → compT_fr_The`（主实体融化分支用，code_07a §07a.8.3）的五个函数——**教材 ch13 §13.5.3 温湿参量的直接源码对应**。

```
!file: phys/module_mp_thompson.F    L6031-L6061
L06031  !
L06032        real function theta_e(pres_Pa,temp_K,w_non,tlcl_K)
L06033  !..
L06034  !..         The following code was based on Bolton (1980) eqn #43
L06035  !..         and claims to have 0.3 K maximum error within -35 < T < 35 C
L06036  !..            pres_Pa = Pressure in Pascals
L06037  !..            temp_K  = Temperature in Kelvin
L06038  !..            w_non   = mixing ratio (non-dimensional = kg/kg)
L06039  !..            tlcl_K  = Temperature at Lifting Condensation Level (K)
L06040  !..
L06041        IMPLICIT NONE
L06042
L06043        real:: pres_Pa, temp_K, w_non, tlcl_K
L06044        real:: pp, tt, rr, tlc, power, xx, p1, p2
L06045
L06046  !+---+
L06047
L06048        pp = pres_Pa
L06049        tt = temp_K
L06050        rr = w_non + 1.e-8
L06051        tlc = tlcl_K
L06052
L06053        power=(0.2854*(1.0 - (0.28*rr) ))
L06054        xx = tt * (100000.0/pp)**power
L06055
L06056        p1 = (3.376/tlc) - 0.00254
L06057        p2 = (rr*1000.0) * (1.0 + 0.81*rr)
L06058
L06059        theta_e = xx * exp(p1*p2)
L06060
L06061        return
```

`theta_e`：Bolton (1980) **式 (43)**，假相当位温的 0.3 K 精度近似——`θe = T·(1e5/p)^(0.2854(1−0.28r))·exp((3.376/T_LCL − 0.00254)·r·1000·(1+0.81r))`。对照书 (6.3.8) 的定义式（指针见教材 ch13 §13.3）：Bolton 式是“含过饱和修正项”的工作公式，系数 0.2854 = R_d/c_pd、0.28r 修正湿空气、0.81r 修正水汽对热容的贡献。

```
!file: phys/module_mp_thompson.F    L6065-L6085
L06065  !
L06066        real function t_lcl(temp_K,tdew_K)
L06067  !..
L06068  !..         The following code was based on Bolton (1980) eqn #15
L06069  !..         and claims to have 0.1 K maximum error within -35 < T < 35 C
L06070  !..            temp_K  = Temperature in Kelvin
L06071  !..            tdew_K  = Dewpoint T at Lifting Condensation Level (K)
L06072  !..
L06073        IMPLICIT NONE
L06074
L06075        real:: temp_K, tdew_K
L06076        real:: tt, tttd, denom
L06077
L06078  !+---+
L06079
L06080        tt = temp_K
L06081        tttd= tdew_K
L06082        denom= ( 1.0/(tttd-56.0) ) + (log(tt/tttd)/800.)
L06083        t_lcl = ( 1.0 / denom ) + 56.0
L06084        return
L06085        end
```

`t_lcl`：Bolton **式 (15)**——由 (T, Td) 解析反解 LCL 温度（0.1 K 精度），`denom = 1/(Td−56) + ln(T/Td)/800` 的倒数结构来自 Clausius–Clapeyron 在 LCL 邻域的线性化。

```
!file: phys/module_mp_thompson.F    L6089-L6106
L06089        real function t_dew(pres_Pa,w_non)
L06090  !..
L06091  !..            pres_Pa = Pressure in Pascals
L06092  !..            w_non   = mixing ratio (non-dimensional = kg/kg)
L06093  !..
L06094        IMPLICIT NONE
L06095
L06096        real:: pres_Pa, w_non
L06097        real:: p, RR, ES, ESLN
L06098
L06099  !+---+
L06100
L06101        p = pres_Pa
L06102        RR=w_non+1e-8
L06103        ES=P*RR/(.622+RR)
L06104        ESLN=LOG(ES)
L06105        T_Dew=(35.86*ESLN-4947.2325)/(ESLN-23.6837)
L06106        return
```

`t_dew`：由混合比反解露点——先 `es = p·r/(0.622+r)`（水汽分压，书 (2.2.10) 的形态），再 Magnus 型反演 `(35.86·ln es − 4947.2325)/(ln es − 23.6837)`。

```
!file: phys/module_mp_thompson.F    L6110-L6145
L06110  !
L06111        real function theta_wetb(thetae_K)
L06112  !..
L06113  !..             Eqn below was gotten from polynomial fit to data in
L06114  !..             Smithsonian Meteorological Tables showing Theta-e
L06115  !..             and Theta-w
L06116  !..
L06117        IMPLICIT NONE
L06118
L06119        real:: thetae_K
L06120        real:: x, answer
L06121
L06122  !+---+
L06123
L06124        real*8 c(0:6), d(0:6)
L06125        data c/-1.00922292e-10, -1.47945344e-8, -1.7303757e-6             &
L06126       &      ,-0.00012709,      1.15849867e-6, -3.518296861e-9            &
L06127       &      ,3.5741522e-12/
L06128        data d/0.00000000,   -3.5223513e-10, -5.7250807e-8                &
L06129       &     ,-5.83975422e-6, 4.72445163e-8, -1.13402845e-10              &
L06130       &     ,8.729580402e-14/
L06131
L06132        x=min(475.0,thetae_K)
L06133
L06134        if( x .le. 335.5 ) then
L06135           answer = c(0)+x*(c(1)+x*(c(2)+x*(c(3)+x*(c(4)+x*(c(5)+         &
L06136       &            x*c(6) )))))
L06137        else
L06138           answer = d(0)+x*(d(1)+x*(d(2)+x*(d(3)+x*(d(4)+x*(d(5)+         &
L06139       &            x*d(6) )))))
L06140        endif
L06141
L06142        theta_wetb = answer + 273.15
L06143
L06144        return
L06145        end
```

`theta_wetb`：θe→θw（湿球位温）的**两段七次多项式**（Smithsonian 气象表拟合；335.5 K 分段、上限 475 K 截断）。主实体用它做 `compT_fr_The` 不收敛时的回退估算。

```
!file: phys/module_mp_thompson.F    L6148-L6187
L06148  !
L06149        real function compT_fr_The(thelcl_K,pres_Pa)
L06150  !..
L06151  !..             pres_Pa = Pressure in Pascals
L06152  !..             thelcl  = Theta-e at LCL (units in Kelvin)
L06153  !..
L06154  !..             Temperature (K) is returned given Theta-e at LCL
L06155  !..             and a pressure.  This describes a moist-adiabat.
L06156  !..             This temperature is the parcel temp at level Pres
L06157  !..             along moist adiabat described by theta-e.
L06158  !..
L06159        IMPLICIT NONE
L06160
L06161        real:: thelcl_K, pres_Pa
L06162        real:: guess, epsilon, w1, w2, tenu, tenup, cor, thwlcl_K
L06163        integer:: iter
L06164
L06165  !+---+
L06166
L06167        guess= (thelcl_K - 0.5 * ( max(thelcl_K-270., 0.))**1.05)         &
L06168       &     * (pres_Pa/100000.0)**.2
L06169        epsilon=0.01
L06170        do iter=1,100
L06171           w1 = rslf(pres_Pa,guess)
L06172           w2 = rslf(pres_Pa,guess+1.)
L06173           tenu = theta_e(pres_Pa,guess,w1,guess)
L06174           tenup = theta_e(pres_Pa,guess+1,w2,guess+1.)
L06175           cor = (thelcl_K - tenu) / (tenup - tenu)
L06176           guess = guess + cor
L06177           if( (cor.lt.epsilon) .and. (-cor.lt.epsilon) ) then
L06178              compT_fr_The=guess
L06179              return
L06180           endif
L06181        enddo
L06182  !     print*, '  convergence not reached '
L06183        thwlcl_K=theta_wetb(thelcl_K)
L06184        compT_fr_The = thwlcl_K*((pres_Pa/100000.0)**0.286)
L06185
L06186        return
L06187        end
```

`compT_fr_The`：由 θe 与气压反解气块温度——“沿湿绝热线迭代”。**读这段必须知道一个源码事实**：L6172–L6173 调用 `theta_e(pres, guess, w1, guess)` 时**第四参数（应是 TLCL）传的是 guess 本身**——这不是笔误：饱和气块的 TLCL 就是自身温度，迭代过程把当前 guess 当饱和气块处理，“θe 沿湿绝热线不变”的隐式方程 `theta_e(p, T, w_s(T), T) = θe_given` 用割线法解（`cor = (目标 − 当前)/相邻差`）。100 次不收敛则退回 `θw • (p/p0)^0.286` 的干绝热外推——**低气压端（厚云顶）的兜底**。收敛阈值 `epsilon=0.01`（0.01 K）；初值 `guess = (θe − 0.5 • max(θe−270,0)^1.05) • (p/1e5)^0.2` 是经验初估。

07b.8 改造入口（→ 教材 ch20/ch21 案索引）

想改什么	动哪里	注意
碰并效率参数	`table_Efrw/table_Efsw` 多项式段	改后必须删 `.dat` 缓存或开 `write_thompson_tables` 重新生成
Bigg 冻结温度灵敏度	`freezeH2O` 的 `T_adjust` 行	表四维含 IN，动订正即动全表
冰核化参数化换代	`iceDeMott`（2010→2015 注释已留）	注意返回单位换算
气溶胶活化敏感性	`activ_ncloud` 的 `1/m` 死值	换 κ/半径需重做 CCN_ACTIVATE.BIN
有效半径口径	`calc_effectRad` 的三段	影响辐射方案（ch12 接口）
雷达模拟	`calc_refl10cm`	亮带块依赖 `module_mp_radar` 的 Blahak 库
谱形改造	回 code_07a §07a.4.2 + 本篇各建表	**表与谱参数必须成对改**——最高风险项

07b.9 上下游链接

- **上游（本模块内）**：code_07a——模块常数（各表维度与预存系数的出处）、主实体（各表运行时的用法）。
- **外部依赖**：`module_mp_radar`（`radar_init`、`rayleigh_soak_wetgraupel`、`nrbins/xxDs/xdts/simpson` 等反射率积分设施）；`CCN_ACTIVATE.BIN`（run 目录随附数据文件）。
- **教材**：ch11（饱和水汽压口径、气溶胶）、ch13（Thompson 精读、三口径对照、有效半径接口）、ch15（湿球温度与温湿参量的热力学基础）。

code_08 — 尘排放简易方案（module_dust_emis.F + module_data_gocart_dust.F：bulk_dust_emis 与 GOCART 五 _bins 数据）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_dust_emis.F	189	07741bb134983bc2e445f0132bc6bbda
phys/module_data_gocart_dust.F	39	936c5881e15ea3b0e99f677aec11f984

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（C）：dust_emis 头段 + gocart 数据模块全文；bulk_dust_emis 主体（L56–L187，GOCART 排放公式的实现）注记不录。教材锚点：ch13_microphysics_family.md（Thompson-aero 的前置件）、ch20_modifying_wrf.md。

二、定位

一句话：**化学开关之外的简易起尘方案**——bulk_dust_emis（Mei Xu 2017-01 从 mosaic_dust_gocartemis 改写 + G. Thompson 2017-09 增补）从风速/土壤湿度/可蚀性直接算尘排放**数浓度**（#/kg/s），经 nifa2d 更新 QNIFA（NIFA 预报量），再被辐射的 aer_opt=3 链路消费（code_20 § 五 gt_aod）——**不用 chem_opt 也能做沙尘辐射效应**的轻量通道。

谁在调它：module_surface_driver（thompson 方案配套的 dustrt 选项分支）；数据底座 module_data_gocart_dust 与 WRF_CHEM 的 gocart 共用（radiation_driver 的 USE 清单里就有它，code_13 § 三）。

三、头部与数据底座

```
!file: phys/module_dust_emis.F    L1-L55
L00001  !WRF:MEDIATION_LAYER:PHYSICS
L00002  ! *** add new modules of schemes here
L00003  !
L00004  MODULE module_dust_emis
L00005  CONTAINS
L00006
L00007  !=================================================================================
L00008  ! simple dust emission scheme outside of WRF_CHEM
L00009  !       revised from s/r mosaic_dust_gocartemis
L00010  !         Mei Xu 2017Jan31 and additions by G. Thompson 2017Sep19
L00011  !=================================================================================
L00012    subroutine bulk_dust_emis (ktau,dt,num_soil_layers,u_phy,v_phy,          &
L00013           rho_phy,alt,u10,v10,p8w,dz8w,smois,erod,                          &
L00014           ivgtyp,isltyp,vegfra,albbck,xland,dx,g,                           &
L00015           nifa2d,                                                           &
L00016           ids,ide, jds,jde, kds,kde,                                        &
L00017           ims,ime, jms,jme, kms,kme,                                        &
L00018           its,ite, jts,jte, kts,kte                                         )
L00019  !=================================================================================
L00020  ! INPUT:
L00021  ! ktau          - simulation time
L00022  ! dt            - size of timestep
L00023  ! u_phy,v_phy    - state variables of wind
L00024  ! rho_phy,alt    - state variable of density and inverse of density
L00025  ! dz8w,p8w       - model layer depth
L00026  ! ivgtyp,isltyp - dominant vegetation, soil type
L00027  ! vegfra        - vegetation fraction (0-100)
L00028  ! albbck        - background sfc albedo used to increase erod where desert-like
L00029  ! xland         - land mask (1: land, 2: water)
L00030  ! dx, g         - model grid increment, gravity constant
L00031  ! smois         - soil moisture
```

```
L00032  ! erod          - fraction of erodible grid cell, for 1: Sand, 2: Silt, 3: Clay
L00033  !               - read in from wrfinput
L00034  ! OUTPUT:
L00035  ! nifa2d        - dust generation in #/kg/s - to be used to update QNIFA (# kg-1)
L00036  !-------------------------------------------------------------------------------
L00037  ! Local variables:
L00038  ! nmx           - number of dust size bins
L00039  ! ilwi          - land/water flag
L00040  ! bems          - binned dust emission in kg/timestep/cell
L00041  ! totalemis     - dust emission from 0.1-10 um in radius, in ug/m2/s
L00042  ! erodin        - fraction of erodible grid cel,
L00043  ! w10m          - total wind speed at 10 m
L00044  ! gwet          - volumetric soil moisture over porosity
L00045  ! dxy           - model grid box area
L00046  ! airmas        - grid box air mass
L00047  ! airden        - air density
L00048  ! DSRC          - source of each dust type          (kg/timestep/cell)
L00049  !
L00050  ! USED from module_data_gocart_dust:
L00051  ! porosity      -
L00052  ! ch_dust       - Constant to fudge the total emission of dust  (s2/m2)
L00053  ! frac_s        - mass fraction of dust
L00054  !===============================================================================
L00055
```

讲解：① **入参即物理**：u10/v10（10 m 风）、smois/erod（土壤湿度与可蚀性分数——**erod 从 wrfinput 读入**，按沙/粉/黏三类）、vegfra/albbck（植被遮蔽与背景反照率——反照率用于识别沙漠样格点放大可蚀性）、dx/g（格距与重力）；② 输出只有一个标量场 **nifa2d——尘排放以数浓度为单位（#/kg/s）直接进 QNIFA**（Thompson 的凝结核预报量），这是「沙尘 →CCN→ 微物理 → 辐射」全链条的入口；③ 局部量注释给出完整账本（bems 逐 bin 排放、totalemis 0.1–10 μm 半径段 μg/m²/s、gwet=土壤湿度/孔隙度）；④ 依赖 module_data_gocart_dust 的 porosity/ch_dust/frac_s。

```
!file: phys/module_data_gocart_dust.F    L1-L39
L00001  Module module_data_gocart_dust
L00002    INTEGER, PARAMETER :: ndust=5,ndcls=3,ndsrc=1,maxstypes=100
L00003    INTEGER, PARAMETER :: ngsalt=9
L00004  ! GAC-->
L00005  ! 20130219 - Drypoint no longer needed.  Use NOAA porosity for all schemes to
L00006  ! allow use of AFWA dust scheme by all LSMs, not just NOAA and RUC.
L00007  !  real, dimension (maxstypes) :: drypoint
L00008  !  real, dimension (maxstypes) :: porosity
L00009    real, dimension (19), PARAMETER :: porosity=(/0.339, 0.421, 0.434, 0.476, 0.476, 0.439, &
L00010                                                  0.404, 0.464, 0.465, 0.406, 0.468, 0.468, &
L00011                                                  0.439, 1.000, 0.200, 0.421, 0.468, 0.200, &
L00012                                                  0.339/)
L00013  ! <--GAC
L00014
L00015    REAL :: ch_dust(ndust,12)
L00016    REAL,    PARAMETER :: dyn_visc = 1.5E-5
L00017  !  real*8, DIMENSION (5), PARAMETER :: den_dust(5)=(/2500.,2650.,2650.,2650.,2650./)
L00018    real*8, DIMENSION (5)            :: den_dust(5)=(/2500.,2650.,2650.,2650.,2650./)
L00019    real*8, DIMENSION (5), PARAMETER :: reff_dust(5)=(/0.73D-6,1.4D-6,2.4D-6,4.5D-6,8.0D-6/)
L00020    INTEGER, DIMENSION (5), PARAMETER :: ipoint(5)=(/3,2,2,2,2/)
L00021    REAL, DIMENSION (5), PARAMETER :: frac_s(5)=(/0.1,0.25,0.25,0.25,0.25/)
L00022
L00023    real*8, DIMENSION (ngsalt), PARAMETER ::
↪   reff_salt=(/0.71D-6,1.37D-6,2.63D-6,5.00D-6,9.50D-6,18.1D-6,34.5D-6,65.5D-6,125.D-6/)
L00024    real*8, DIMENSION (ngsalt), PARAMETER :: den_salt=(/2500.,2650.,2650.,2650.,2650.,2650.,2650.,2650.,2650./)
L00025    INTEGER, DIMENSION (ngsalt), PARAMETER :: spoint=(/1,2,2,2,2,2,3,3,3/)  ! 1 Clay, 2 Silt, 3 Sand
L00026    real*8, DIMENSION (ngsalt), PARAMETER :: frac_salt=(/1.,0.2,0.2,0.2,0.2,0.2,0.333,0.333,0.333/)
L00027
L00028    real*8, DIMENSION (ndust), PARAMETER :: distr_dust=(/1.074D-1,1.012D-1,2.078D-1,4.817D-1,1.019D-1/)
L00029  ! real*8, DIMENSION (ndust), PARAMETER :: lo_dust=(/0.1D-6,1.0D-6,1.8D-6,3.0D-6,6.0D-6/)
L00030  ! real*8, DIMENSION (ndust), PARAMETER :: up_dust=(/1.0D-6,1.8D-6,3.0D-6,6.0D-6,10.0D-6/)
L00031
L00032    !GOCART bins radii ranges
L00033    real*8, DIMENSION (5), PARAMETER :: ra_dust(5)=(/1.d-1,1.0d0,1.8d0,3.0d0,6.0d0/)
L00034    real*8, DIMENSION (5), PARAMETER :: rb_dust(5)=(/1.0d0,1.8d0,3.0d0,6.0d0,10.0d0/)
L00035
L00036
L00037  END Module module_data_gocart_dust
L00038
L00039
```

讲解：GOCART 数据底座全文（39 行；**本文件是文库 32 个源文件中唯一的 CRLF 行尾**——归一化后抽取，字节 md5 936c5881e15ea3b0e99f677aec11f984 为 fp_files 按原始字节计，块指纹为归一化口径）：① **五 bins 尘**（ndust=5）：有效半径 reff 0.73–8.0 μm、密度 2500/2650 kg/m³、质量分数 frac_s 0.1/0.25×4、半径区间 ra/rb（0.1–10 μm）、谱分布 distr_dust；② **孔隙度 19 类表**（porosity，对应 STAS 土壤类；2013-02-19 注：drypoint 废弃、统一用 NOAA 孔隙度让 AFWA 尘方案适配所有 LSM——**与 noahlsm 的 MAXSMC 表同值不同源**，改土壤参数时两处同步）；③ch_dust(ndust,12) 逐月风蚀系数（可调经验常数）、dyn_visc 空气动力粘滞；④ **海盐九 bins（ngsalt=9）**：reff 0.71–125 μm、按 Clay/Silt/Sand 三类源指针——尘模块顺带携带海盐参数（AFWA 起尘方案家族）；⑤ndcls=3（沙/粉/黏三类源类型）、maxstypes=100。

四、主体与改造入口

1. **bulk_dust_emis 主体（L56–L187，未录）**：GOCART 型排放公式——w10m=|V10|+风阵性订正、阈值摩擦速度、gw 湿度抑制、逐 bin 谱分配（distr_dust）、逐格质量守恒换算 #/kg/s；改起尘物理（如 MB97 替

代 GOCART）就在这一段。

2. **erod 场**：wrfinput 输入——换可蚀性数据集（如高分辨率土壤格点）走 geogrid→real 链（ch07）。
3. **与化学路线分流**：本方案输出 QNIFA（数浓度），WRF_CHEM 的 gocart 输出质量混合比（chem 数组）——两条沙尘线不能混读。

五、上下游链接

- 下游：`code_21_module_microphysics_driver.md`（QNIFA 演变）、`code_20_module_radiation_driver.md`（aer_opt=3 的 gt_aod）
- 教材正文：ch13_microphysics_family.md

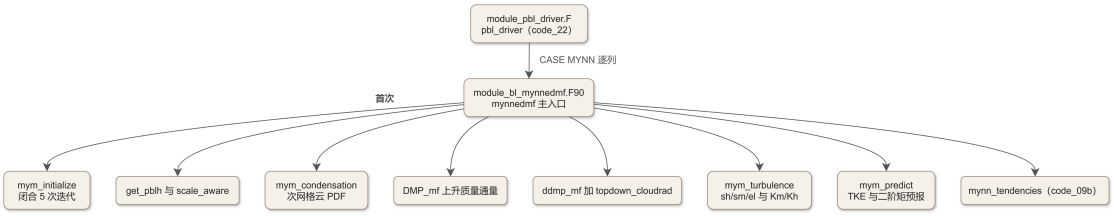
文库 code_09a：module_bl_mynnedmf.F90 上半——主入口、闭合初始化、混合长与湍流闭合

09a.0 版本指纹与阅读说明

- **源文件**：`phys/MYNN-EDMF/module_bl_mynnedmf.F90`（WRF 4.8.0，自由格式.F90），全文 8776 行。本篇覆盖 L1–L3917（文件头、主入口 `mynnedmf`、闭合初始化、Level 2、混合长族、湍流闭合、二阶矩预测、次网格云 PDF）；L3923–L8776（`mynn_tendencies` 倾向驱动、`DMP_mf/ddmp_mf` 质量通量、`GET_PBLH`、三对角求解器族、`topdown_cloudrad`、blend 函数族）见 code_09b。
- **编译实体勘正**：MYNN 方案没有同名单文件 `module_bl_mynn.F`——编译实体就是本文件（教材 ch14 已勘正的口径）。
- **保真承诺**：所有代码块由 `code_extract.py` v1.1 按字节抽取（剥离行号前缀后与源逐位一致，md5 指纹可复核）。
- **符号口径**：闭合常数（a1/a2/b1/b2/c1/c5/g1/g2）与括号约定遵循 Nakanishi & Niino 系列；大气物理符号按《大气物理学》（盛裴轩）——位温/虚位温对应见教材 ch13 §13.2。
- **教材锚点**：ch14（边界层方案族：MYNN-EDMF 精读、PBLH 三型判据、BouLac 亲缘）、ch21（调用节拍）。

09a.1 一句话定位与调用链

一句话：`module_bl_mynnedmf.F90` 是 MYNN-EDMF 边界层方案的完整实现——1.5 阶（Level 2.5）到 3 阶湍流闭合（TKE + 温湿二阶矩）叠加上升/下沉质量通量（EDMF），一次列调用内完成 “PBL 高度诊断 → 次网格云 → 湍流扩散系数 → TKE 预报 → 九变量倾向” 全链条。



- 调用位：`module_pbl_driver.F` 的 `CASE (MYNN)` 分支每时间步逐列调用（code_22）；文件头注释里的流程图（原文 L45–L59）与本图一致。
- 与教材 ch14 的对应：`qke = 2 • TKE` 的口径、PBLH 混合判据、BouLac 混合长亲缘（`boulac_length` 系从 BouLac 方案移植）都在本篇落地。

09a.2 文件头注释原文与精译

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L1-L134
L00001  !>\file module_bl_mynnedmf.F90
L00002  !! This file contains the entity of MYNN-EDMF PBL scheme.
L00003  ! ==================================================================
L00004  ! *   An improved Mellor-Yamada turbulence closure model            *
L00005  ! *                                                                  *
L00006  ! *      Original author: M. Nakanishi (N.D.A), naka@nda.ac.jp       *
L00007  ! *      Translated into F90 and implemented in WRF-ARW by:          *
L00008  ! *                        Mariusz Pagowski (NOAA-GSL)               *
```

```
L00009  ! *      Subsequently developed by:                                      *
L00010  ! *                Joseph Olson, Jaymes Kenyon (NOAA/GSL),              *
L00011  ! *                Wayne Angevine (NOAA/CSL), Kay Suselj (NASA/JPL),    *
L00012  ! *                Franciano Puhales (UFSM), Laura Fowler (NCAR),       *
L00013  ! *                Elynn Wu (UCSD), and Jordan Schnell (NOAA/GSL),      *
L00014  ! *                Tiziani Cherubin (MKWC, Univ. Hawaii),               *
L00015  ! *                Xia Sun (NOAA/GSL/CIRES)                             *
L00016  ! *                                                                     *
L00017  ! *   Contents:                                                         *
L00018  ! *                                                                     *
L00019  ! *   mynnedmf - the main subroutine which calls all other routines     *
L00020  ! *   --------------                                                    *
L00021  ! *     1. mym_initialize  (to be called once initially)                *
L00022  ! *        gives the closure constants and initializes the turbulent    *
L00023  ! *        quantities.                                                  *
L00024  ! *     2. get_pblh                                                     *
L00025  ! *        Calculates the boundary layer height                         *
L00026  ! *     3. scale_aware                                                  *
L00027  ! *        Calculates scale-adaptive tapering functions                 *
L00028  ! *     4. mym_condensation                                             *
L00029  ! *        determines the liquid water content and the cloud fraction   *
L00030  ! *        diagnostically.                                              *
L00031  ! *     5. dmp_mf                                                       *
L00032  ! *        Calls the (nonlocal) mass-flux component                     *
L00033  ! *     6. ddmp_mf                                                      *
L00034  ! *        Calls the downdraft mass-flux component                      *
L00035  ! *    (-) mym_level2      (called in the other subroutines)            *
L00036  ! *        calculates the stability functions at level 2.               *
L00037  ! *    (-) mym_length      (called in the other subroutines)            *
L00038  ! *        calculates the master length scale.                          *
L00039  ! *     7. mym_turbulence                                               *
L00040  ! *        calculates the vertical diffusivity coefficients and the     *
L00041  ! *        production terms for the turbulent quantities.               *
L00042  ! *     8. mym_predict                                                  *
L00043  ! *        predicts the turbulent quantities at the next step.          *
L00044  ! *                                                                     *
L00045  ! *             call mym_initialize                                     *
L00046  ! *                  |                                                  *
L00047  ! *                  |<-----------------+                               *
L00048  ! *                  |                  |                               *
L00049  ! *             call get_pblh           |                               *
L00050  ! *             call scale_aware        |                               *
L00051  ! *             call mym_condensation   |                               *
L00052  ! *             call dmp_mf             |                               *
L00053  ! *             call ddmp_mf            |                               *
L00054  ! *             call mym_turbulence     |                               *
L00055  ! *             call mym_predict        |                               *
L00056  ! *                  |                  |                               *
L00057  ! *                  |------------------+                               *
L00058  ! *                  |                                                  *
L00059  ! *                 end                                                 *
L00060  ! *                                                                     *
L00061  ! *   Variables worthy of special mention:                               *
L00062  ! *     tref   : Reference temperature                                   *
L00063  ! *     thl    : Liquid water potential temperature                     *
L00064  ! *     qw     : Total water (water vapor+liquid water) content         *
L00065  ! *     ql     : Liquid water content                                   *
L00066  ! *     vt, vq : Functions for computing the buoyancy flux              *
L00067  ! *     qke    : 2 * TKE                                                *
L00068  ! *     el     : mixing length                                          *
L00069  ! *                                                                     *
L00070  ! *     If the water contents are unnecessary, e.g., in the case of     *
L00071  ! *     ocean models, thl is the potential temperature and qw, ql, vt   *
L00072  ! *     and vq are all zero.                                            *
L00073  ! *                                                                     *
L00074  ! *   Grid arrangement:                                                 *
L00075  ! *             k+1 +---------+                                         *
L00076  ! *                 |         |     i = 1 - nx                          *
L00077  ! *             (k) |    *    |     k = 1 - nz                          *
L00078  ! *                 |         |                                         *
L00079  ! *              k  +---------+                                         *
L00080  ! *                 i   (i)  i+1                                        *
L00081  ! *                                                                     *
L00082  ! *     All the predicted variables are defined at the center (*) of     *
L00083  ! *     the grid boxes. The diffusivity coefficients and two of their   *
L00084  ! *     components (el and stability functions sh & sm) are, however,   *
L00085  ! *     defined on the walls of the grid boxes.                         *
L00086  ! *     # Upper boundary values are given at k=nz.                      *
L00087  ! *                                                                     *
L00088  ! *   References:                                                       *
L00089  ! *     1. Nakanishi, M., 2001:                                         *
L00090  ! *        Boundary-Layer Meteor., 99, 349-378.                         *
L00091  ! *     2. Nakanishi, M. and H. Niino, 2004:                            *
L00092  ! *        Boundary-Layer Meteor., 112, 1-31.                           *
L00093  ! *     3. Nakanishi, M. and H. Niino, 2006:                            *
L00094  ! *        Boundary-Layer Meteor., 119, 397-407.                        *
L00095  ! *     4. Nakanishi, M. and H. Niino, 2009:                            *
L00096  ! *        Jour. Meteor. Soc. Japan, 87, 895-912.                       *
L00097  ! *     5. Olson J. and coauthors, 2019: A description of the           *
L00098  ! *        MYNN-EDMF scheme and coupling to other components in         *
L00099  ! *        WRF-ARW. NOAA Tech. Memo. OAR GSD, 61, 37 pp.,               *
L00100  ! *        https://doi.org/10.25923/n9wm-be49.                          *
L00101  ! *     6. Puhales, Franciano S. and coauthors, 2020: Turbulent         *
L00102  ! *        Kinetic Energy Budget for MYNN-EDMF PBL Scheme in WRF model.*
L00103  ! *        Universidade Federal de Santa Maria Technical Note. 9 pp.    *
L00104  ! *        https://repositorio.ufsm.br/handle/1/28327                   *
L00105  ! *     7. Olson, J. B., W. M. Angevine, D. D. Turner, X. Sun,          *
L00106  ! *        J. M. Simonson, C. Evans, J. S. Kenyon, H. Li, J. Schnell,   *
L00107  ! *        F. S. Puhales, T. Cherubini, W. Li, and M. Zhang, 2026:      *
L00108  ! *        A Description of the MYNN-EDMF Turbulence Scheme. NOAA Tech.*
L00109  ! *        Memo. OAR GSL-77. 60 pp. https://doi.org/10.25923/rahr-sj70 *
L00110  !***********************************************************************
L00111  ! notes on the versioning of the myhnn-edmf:
L00112  !
L00113  ! Version 1.0.0: Described by references 1-4 (above). Implemented into WRF
L00114  !    WRFv3.0-v3.4.1 for HRRRv1/RAPv2.
L00115  !
```

```
L00116  ! Version 2.0.0: Still best described by references 1-4 (above). Approximately pplies to
L00117  !    WRFv3.5-v3.6.1 and used in HRRRv2/RAPv3.
L00118  !
L00119  ! Version 3.0.0:  Still best described by references 1-4 (above), despite significant
L00120  !    code departures from the original scheme. This version pproximately pplies to
L00121  !    WRFv3.7-3.9 and used for HRRRv3/RAPv4.
L00122  !
L00123  ! Version 4.0.0: Best described by reference 5 (above). Approximately applies to
L00124  !    WRFv4.0-v4.4 (but not well synced during this period) and used for HRRRv4/RAPv5.
L00125  !
L00126  ! Version 5.0.0: Mods stemming from both RRFSv1 development and community development in WRF.
L00127  !    Approximately applies to WRFv4.5-v4.7 / CCPP
L00128  !
L00129  ! Version 6.0.0: Mods stemming from both RRFSv2+ development and community development in WRF.
L00130  !     This version marks the begining of the MYNN-EDMF submodule repository, where all
L00131  !     development is now tracked in a public-facing github repository (https://github.com/NCAR/MYNN-EDMF.git).
L00132  !     WRFv4.8 / CCPP / MPAS. Many updates captured in the Olson et al. (2026) MYNN-EDMF
L00133  !     tech note (see listed references above).
L00134  !==================================================================
```

逐句精译与解读：

- **血统**：原始作者 Nakanishi（日本防卫大学）；F90 化与 WRF 移植 Mariusz Pagowski（NOAA/GSL）；此后 NOAA/GSL 的 Olson、Kenyon、Angevine 等人持续开发——**这是业务驱动的演进线**（HRRR 快速更新循环是它的主战场）。
- **版本史**（L111–L133 原注释）：V1.0（WRFv3.0–3.4，HRRRv1）→ V2.0（HRRRv2）→ V3.0（HRRRv3，代码已大幅偏离原方案）→ V4.0（WRFv4.0–4.4，Olson et al. 2019 技术备忘录）→ V5.0（RRFSv1，WRFv4.5–4.7/CCPP）→ **V6.0（WRFv4.8/CCPP/MPAS）**——开发转入独立 GitHub 仓库 NCAR/MYNN-EDMF，配套 Olson et al. (2026) 技术备忘录 GSL-77。**版本 6 的代码已经是 "独立项目定期同步进 WRF" 的形态**，读 4.8 源码时要意识到它比 WRF 主干更"新"。
- **变量特记**（L61–L72）：`thl` 液水位温（θ_l = θ − L_v/c_p • q_c − L_s/c_p • q_i）、`qw` 总水、`qke = 2 • TKE`（Mellor-Yamada 系的 q² 记法）、`el` 主混合长、`vt/vq` 浮力通量函数——"若不需水物质（如海洋模式），thl 即位温、vt/vq 为零" 的说明点明了这套闭合对干/湿配置的兼容设计。
- **网格约定**（L74–L87）：预报量在格心，扩散系数与 el/sh/sm 在格面（interface）——**stagger 纪律**贯穿全部子程序（参数表里 `kts:kte+1` 即界面维度）。
- **参考文献七篇**：NN2001/2004/2006/2009 四篇理论基础 + Olson 2019 技术备忘录（整体描述）+ Puhales 2020（TKE 收支）+ Olson 2026（最新描述）。**读本文件时手边放 NN2009 与 Olson 2026 两篇**。

09a.3 模块头与调试开关

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L136-L147
L00136  module module_bl_mynnedmf
L00137
L00138   implicit none
L00139
L00140  !---------------------------------------------------------------------
L00141  ! global debugging configuration options:
L00142  !---------------------------------------------------------------------
L00143   logical,parameter:: debug_code = .false.
L00144   integer,parameter:: idbg       = 452 !specific i-point to write out
L00145   integer,parameter:: jdbg       = 272 !specific j-point to write out
L00146
L00147  contains
```

模块级只有三个调试常数（`debug_code=false`、`idbg/jdbg` 指定诊断格点）——**与 Thompson 的 "无状态模块" 同一哲学**：全部状态在实参里，模块零共享数据。`contains` 之后 30 个子程序全部平铺。

09a.4 主入口 `mynnedmf` 逐段精讲

09a.4.1 签名与实参契约

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L157-L218
L00157    subroutine mynnedmf(           i               , j               , &
L00158                initflag          , restart         , cycling         , &
L00159                delt              , dz1             , dx              , &
L00160                !3d state variables
L00161                u1                , v1              , w1              , &
L00162                th1               , sqv1            , sqc1            , &
L00163                sqi1              , sqs1            , qnc1            , &
L00164                qni1              , qnwfa1          , qnifa1          , &
L00165                qnbca1            , ozone1          , pres1           , &
L00166                ex1               , rho1            , tk1             , &
L00167                !2d surface fields
L00168                xland             , ts              , qsfc            , &
```

```
L00169              ps                  , ust               , ch                , &
L00170              hfx                 , qfx               , znt               , &
L00171              wspd                , uoce              , voce              , &
L00172              !eddy-diffusivity
L00173              qke1                , qke_adv1          , el1               , &
L00174              sh1                 , sm1               , kh1               , &
L00175              km1                 ,                                       &
L00176              !smoke variables
L00177              nchem               , ndvel             ,                     &
L00178              chem1               , vdep              , frp               , &
L00179              emis_ant_no         , mix_chem,enh_mix  , settle1           , &
L00180              !generic scalar array
L00181              scalars             , nscalars          ,                     &
L00182              !higher-order moments
L00183              tsq1                , qsq1              , cov1              , &
L00184              !tendencies
L00185              du1                 , dv1               , dth1              , &
L00186              dqv1                , dqc1              , dqi1              , &
L00187              dqnc1               , dqni1             , dqs1              , &
L00188              dqnwfa1             , dqnifa1           , dqnbca1           , &
L00189              dozone1             , rthraten1         ,                     &
L00190              !2d output
L00191              pblh                , kpbl              , maxwidth_dd       , &
L00192              maxwidth            , maxmf             , ztop_plume        , &
L00193              excess_h            , excess_q          , maxmf_dd          , &
L00194              maxtkeprod          , cldtop_cooling    , ent_eff           , &
L00195              !tke budget arrays
L00196              dqke1               , qwt1              , qshear1           , &
L00197              qbuoy1              , qdiss1            ,                     &
L00198              !subgrid clouds
L00199              qc_bl1              , qi_bl1            , cldfra_bl1        , &
L00200              !namelist configurations option
L00201              bl_mynn_tkeadvect   , tke_budget        , bl_mynn_cloudpdf  , &
L00202              bl_mynn_mixlength   , bl_mynn_closure   , bl_mynn_ess       , &
L00203              bl_mynn_edmf        , bl_mynn_edmf_mom  , bl_mynn_edmf_tke  , &
L00204              bl_mynn_mixscalars , bl_mynn_mixaerosols,bl_mynn_mixnumcon , &
L00205              bl_mynn_output      , bl_mynn_cloudmix  , bl_mynn_mixqt     , &
L00206              bl_mynn_edmf_dd     ,                                       &
L00207              !3d emdf output
L00208              edmf_a1             , edmf_w1           , edmf_qt1          , &
L00209              edmf_thl1           , edmf_ent1         , edmf_qc1          , &
L00210              sub_thl1            , sub_sqv1          , det_thl1          , &
L00211              det_sqv1            ,                                       &
L00212              !spp
L00213              spp_pbl             , pattern_spp_pbl1  ,                     &
L00214              !check flags
L00215              FLAG_QC             , FLAG_QI           , FLAG_QNC          , &
L00216              FLAG_QNI            , FLAG_QS           , FLAG_QNWFA        , &
L00217              FLAG_QNIFA          , FLAG_QNBCA        , FLAG_OZONE        , &
L00218              KTS                 , KTE               , errmsg, errflg     )
```

- **实参即接口清单**（对照教材 ch14 §14.7 的 namelist 全表）：17 个 `bl_mynn_*` namelist 开关直接作为形参传入（由 `pbl_driver` 从 `config_flags` 解包）——**开关不经全局模块、逐列透传**，CCPP 化的痕迹（物理与框架解耦）。
- **字段三组**：环境/表面（`u1…tk1` 状态列 + `ts/qsfc/ps/ust/ch/hfx/qfx` 表面量）、湍流量（`qke1/el1/sh1/sm1/kh1/km1` + 二阶矩 `tsq1/qsq1/cov1`）、输出（倾向 `du1…`与 EDMF 诊断 `edmf_a1/edmf_w1/…`）。
- **`FLAG_*` 九个逻辑量**：与 driver 的 `f_*` 同源（Registry 包判定字段在场性），方案内据此跳过缺失物种——与 microphysics_driver 的 OPTIONAL 实参同一套 “在场即配置” 哲学。
- **`spp_pbl/pattern_spp_pbl1`**：随机参数扰动（Stochastic Parameter Perturbation）入口——湍流系数的集合扰动从这一对参数进（用法见 `mym_turbulence` 末尾）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L220-L341
L00220  !-------------------------------------------------------------------
L00221   use module_bl_mynnedmf_common, only: cp,xlvcp,xlscp,p608,p1000mb,rcp,&
L00222        karman,gtr,cphh_st,cphm_unst,cphh_unst,b1,zero,one,p01,p5,      &
L00223        five,kind_phys
L00224
L00225   integer, intent(in) :: initflag
L00226   logical, intent(in) :: restart,cycling
L00227   integer, intent(in) :: tke_budget
L00228   integer, intent(in) :: bl_mynn_cloudpdf
L00229   integer, intent(in) :: bl_mynn_mixlength
L00230   integer, intent(in) :: bl_mynn_edmf
L00231   integer, intent(in) :: bl_mynn_edmf_dd
L00232   logical, intent(in) :: bl_mynn_tkeadvect
L00233   integer, intent(in) :: bl_mynn_edmf_mom
L00234   integer, intent(in) :: bl_mynn_edmf_tke
L00235   integer, intent(in) :: bl_mynn_mixscalars
L00236   integer, intent(in) :: bl_mynn_mixaerosols
L00237   integer, intent(in) :: bl_mynn_mixnumcon
L00238   integer, intent(in) :: bl_mynn_output
L00239   integer, intent(in) :: bl_mynn_cloudmix
L00240   integer, intent(in) :: bl_mynn_mixqt
L00241   integer, intent(in) :: bl_mynn_ess
L00242   real(kind_phys), intent(in) :: bl_mynn_closure
L00243
L00244   logical, intent(in) :: FLAG_QI,FLAG_QNI,FLAG_QC,FLAG_QNC,&
L00245                          FLAG_QNWFA,FLAG_QNIFA,FLAG_QNBCA, &
L00246                          FLAG_OZONE,FLAG_QS
L00247
L00248   logical, intent(in) :: mix_chem,enh_mix
L00249
L00250   integer, intent(in) :: KTS,KTE
L00251
L00252   character(len=*),intent(out):: &
L00253      errmsg       ! output error message (-).
L00254   integer,intent(out):: &
L00255      errflg       ! output error flag (-).
```

```
L00256
L00257  #ifdef HARDCODE_VERTICAL
L00258  # define kts 1
L00259  # define kte HARDCODE_VERTICAL
L00260  #endif
L00261
L00262  !scalar variables
L00263   real(kind_phys), intent(in)    :: delt
L00264   real(kind_phys), intent(in)    :: dx
L00265   real(kind_phys), intent(in)    :: ust,ch,qsfc,ps,wspd,xland
L00266   real(kind_phys), intent(in)    :: ts,znt,hfx,qfx,uoce,voce
L00267   real(kind_phys), intent(inout) :: pblh,excess_h,excess_q
L00268   real(kind_phys), intent(inout) :: maxmf,maxwidth,ztop_plume
L00269   real(kind_phys), intent(inout) :: maxmf_dd,maxwidth_dd,maxtkeprod,cldtop_cooling,ent_eff
L00270   integer,         intent(in)    :: i,j
L00271   integer,         intent(inout) :: kpbl
L00272  !local
L00273   real(kind_phys) :: psig_bl,psig_shcu,rmol,rmolh
L00274   integer :: imd,jmd
L00275   integer :: k,kproblem
L00276
L00277  !column variables (all end with a "1")
L00278   real(kind_phys), dimension(kts:kte), intent(in)      ::            &
L00279         dz1,u1,v1,w1,th1,pres1,ex1,rho1,tk1,rthraten1
L00280   real(kind_phys), dimension(kts:kte), intent(inout)   ::            &
L00281         sqv1,sqc1,sqi1,sqs1,qni1,qnc1,qnwfa1,qnifa1,qnbca1,ozone1,   &
L00282         qke1,tsq1,qsq1,cov1,qke_adv1,                               &
L00283         sh1,sm1,el1,                                                & !interface, but kte+1 not included
L00284         du1,dv1,dth1,dqv1,dqc1,dqi1,dqs1,                           &
L00285         dqni1,dqnc1,dqnwfa1,dqnifa1,dqnbca1,dozone1,                &
L00286         qc_bl1,qi_bl1,cldfra_bl1,edmf_a1,edmf_w1,                   &
L00287         edmf_qt1,edmf_thl1,edmf_ent1,edmf_qc1,                      &
L00288         sub_thl1,sub_sqv1,det_thl1,det_sqv1
L00289   real(kind_phys), dimension(kts:kte), intent(inout), optional ::    &
L00290         qwt1,qshear1,qbuoy1,qdiss1,dqke1
L00291   real(kind_phys), dimension(kts:kte), intent(out)     ::            & !interface
L00292         kh1,km1
L00293  !local
L00294   real(kind_phys), dimension(kts:kte)                  ::            &
L00295         qc_bl1_old,qi_bl1_old,cldfra_bl1_old,dummy1,dummy2,         &
L00296         diss_heat1,                                                 &
L00297         thl1,thv1,thlv1,qv1,qc1,qi1,qs1,sqw1,                       &
L00298         thl_tot1,qc_tot1,qi_tot1,                                   &
L00299         dfm1, dfh1, dfq1, tcd1, qcd1,                               &
L00300         pdk1, pdt1, pdq1, pdc1,                                     &
L00301         vt1, vq1, sgm1, kzero1
L00302   real(kind_phys), dimension(kts:kte)                  ::            &
L00303         edmf_u1,edmf_v1,edmf_qv1
L00304
L00305  !smoke/chemical arrays
L00306   integer, intent(in) ::   nchem, ndvel
L00307   real(kind_phys), dimension(kts:kte,nchem), intent(inout) :: chem1
L00308   real(kind_phys), dimension(kts:kte,nchem), intent(in   ) :: settle1
L00309   real(kind_phys), dimension(ndvel), intent(in)    :: vdep
L00310   real(kind_phys),                   intent(in)    :: frp,emis_ant_no
L00311   real(kind_phys), dimension(kts:kte+1,nchem)      :: s_awchem1
L00312   integer :: ic
L00313  !scalar array
L00314   integer, intent(in) ::   nscalars
L00315   real(kind_phys), dimension(kts:kte,nscalars), intent(inout) :: scalars
L00316   real(kind_phys), dimension(kts:kte+1,nscalars)      :: s_awscalars1
L00317
L00318  !local vars
L00319  !mass-flux variables
L00320   real(kind_phys), dimension(kts:kte) ::                            &
L00321         dth1mf,dqv1mf,dqc1mf,du1mf,dv1mf
L00322   real(kind_phys), dimension(kts:kte) ::                            &
L00323         edmf_a_dd1,edmf_w_dd1,edmf_qt_dd1,edmf_thl_dd1,             &
L00324         edmf_ent_dd1,edmf_qc_dd1
L00325   real(kind_phys), dimension(kts:kte) ::                            &
L00326         sub_u1,sub_v1,det_sqc1,det_u1,det_v1
L00327   real(kind_phys), dimension(kts:kte+1) ::                          & !interface
L00328         s_aw1,s_awthl1,s_awqt1,                                     &
L00329         s_awqv1,s_awqc1,s_awu1,s_awv1,s_awqke1,s_awqsq1,            &
L00330         s_awqnc1,s_awqni1,s_awqnwfa1,s_awqnifa1,                    &
L00331         s_awqnbca1
L00332   real(kind_phys), dimension(kts:kte+1) ::                          & !interface
L00333         sd_aw1,sd_awthl1,sd_awqt1,                                  &
L00334         sd_awqv1,sd_awqc1,sd_awu1,sd_awv1,sd_awqke1,                &
L00335         sd_awqi1,sd_awqnc1,sd_awqni1,                               &
L00336         sd_awqnwfa1,sd_awqnifa1
L00337   integer:: ktop_plume
L00338   real(kind_phys), dimension(kts:kte+1) :: zw1              !interface
L00339  !mass flux (or analytical) tke production
L00340   real(kind_phys), dimension(kts:kte) :: TKEprod_dn,TKEprod_up
L00341
```

- **依赖极简**：唯一的模块依赖是 `module_bl_mynnedmf_common`（常数包：cp/xlvcp/karman/b1/zero…与 kind_phys 精度参数）——**方案自带常数命名空间**，与 model_constants 隔离（教材 ch14 §14.3 讲过 MYNN 常数体系）。
- **列变量全部 *1 后缀**（单列版本）：`sqv1/sqc1…`是比湿（specific q，分母含干空气+水汽）——注意 L665–L668 处转换成混合比 `qv1 = sqv1/(1−sqv1)`；`sh1/sm1/el1` 注释明确 “interface 维度但不含 kte+1”。
- **局部工作数组**：`thl1/thv1/thlv1` 热力学三兄弟、`dfm1/dfh1/dfq1` 扩散系数、`pdk1/pdt1/pdq1/pdc1` 生产项、`s_aw*1/sd_aw*1` 两组界面量（aw = area-weighted，上升/下沉质量通量的界面通量）、`edmf_*` 上升流属性族。
- `errflg/errmsg` 输出对：CCPP 标准错误协议。

09a.4.2 编译期配置选项

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L342-L384
L00342  !-------------------------------------------------
L00343  !-------- configuration options:
L00344  !-------------------------------------------------
L00345  !Option to switch flux-profile relationship for surface (from Puhales et al. 2020)
L00346  !0: use original Dyer-Hicks, 1: use Cheng-Brustaert and Blended COARE
L00347   integer,parameter:: bl_mynn_stfunc = 0
L00348
L00349  !>Use Canuto/Kitamura mod (remove Ric and negative TKE) (real; 1.0:yes, 0.0:no)
L00350  !!For more info, see Canuto et al. (2008 JAS) and Kitamura (Journal of the
L00351  !!Meteorological Society of Japan, Vol. 88, No. 5, pp. 857-864, 2010).
L00352  !!Note that this change required further modification of other parameters
L00353  !!above (c2, c3). If you want to remove this option, set c2 and c3 constants
L00354  !!(above) back to NN2009 values (see commented out lines next to the
L00355  !!parameters above). This only removes the negative TKE problem
L00356  !!but does not necessarily improve performance - neutral impact. Also,
L00357  !!it is a real variable because it is used within an equation to
L00358  !!activate/deactive this option
L00359   real(kind_phys),parameter:: CKmod   = 1.0
L00360
L00361  !>Parameter used for the conversion of tsq to qpe when using the TTE closure
L00362   real(kind_phys),parameter:: taue    = 80.
L00363
L00364  !>Option to control the upwind/centerd finite differencing of explicit mass-flux solver when
L00365  !>bl_mynn_edmf = 2  (0: mass flux inactive, 1: implicit, 2: explicit)
L00366   real(kind_phys),parameter:: upwind  = 1.0 ! upwind=1.0: use upwind approximation for mass-flux calculation
L00367                                             ! upwind=0.5: use centered difference for mass-flux calculation
L00368                                             ! explicit mass-flux can use either upwind or centered-difference
L00369                                             ! implicit mass-flux only uses the centered differencing method.
L00370  !-------------------------------------------------
L00371
L00372   real(kind_phys) :: cpm,sqcg,flt,fltv,flq,flqv,flqc,               &
L00373         pmz,phh,exnerg,zet,phi_m,                                   &
L00374         afk,abk,th_sfc,wsp
L00375
L00376   logical :: INITIALIZE_QKE,problem
L00377
L00378  !stochastic fields
L00379   integer,  intent(in)                              :: spp_pbl
L00380   real(kind_phys), dimension(kts:kte), intent(in)   :: pattern_spp_pbl1
L00381
L00382  !substepping TKE
L00383   integer :: nsub
L00384   real(kind_phys) :: delt2
```

- **`bl_mynn_stfunc=0`**（PARAMETER，编译期写死）：0 = 原始 Kansas 型通量廓线（Dyer–Hicks），1 = Puhales et al. (2020) 的 Cheng–Brutsaert 与 COARE 混合——**切换开关但默认关闭**，2020 新物理以参数形式待命。
- **`CKmod=1.0`**：Canuto/Kitamura 修正（去除临界 Ric 与负 TKE 问题）——注释明说“启用它需要连带改 c2/c3 常数，若要退回 NN2009 原值须把上方的常数改回”。**一个开关牵动常数族**是这套闭合改造的第一陷阱。
- **`taue=80`**（s）：温度方差到潜在能量（qpe）的转换时间尺度——TTE（total turbulent energy）闭合的核心参数（Machulskaya & Mironov 2020）。
- **`upwind=1.0`**：显式质量通量求解器（`bl_mynn_edmf=2`）的上风/中心差分选择——1.0 上风（保守）、0.5 中心（精确）。

09a.4.3 调试、清零与首次初始化

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L386-L449
L00386      errmsg = " "
L00387      errflg = 0
L00388
L00389      if (debug_code) then !check incoming values
L00390         problem = .false.
L00391         do k=kts,kte
L00392            wsp  = sqrt(u1(k)**2 + v1(k)**2)
L00393            if (abs(hfx) > 1200. .or. abs(qfx) > 0.001 .or.          &
L00394                wsp > 270. .or. tk1(k) > 380. .or. tk1(k) < 160. .or. &
L00395                sqv1(k) < zero .or. sqc1(k) < zero .or.              &
L00396                (ps-pres1(kts)) < zero) then
L00397               kproblem = k
L00398               problem = .true.
L00399               print*,"Incoming problem at: i=",i," j=",j," k=",k
L00400               print*," QFX=",qfx," HFX=",hfx
L00401               print*," wsp=",wsp," T=",tk1(k)
L00402               print*," qv=",sqv1(k)," qc=",sqc1(k)
L00403               print*," u*=",ust," wspd=",wspd
L00404               print*," xland=",xland," ts=",ts
L00405               print*," ps=",ps,"delp1=",ps-pres1(kts)
L00406               print*," znt=",znt," dx=",dx," dz(1)=",dz1(1)
L00407            endif
L00408         enddo
L00409         if (problem) then
L00410            print*,"===tk:",tk1(max(kproblem-3,1):min(kproblem+3,kte))
L00411            print*,"===qv:",sqv1(max(kproblem-3,1):min(kproblem+3,kte))
L00412            print*,"===qc:",sqc1(max(kproblem-3,1):min(kproblem+3,kte))
L00413            print*,"===qi:",sqi1(max(kproblem-3,1):min(kproblem+3,kte))
L00414            print*,"====u:",u1(max(kproblem-3,1):min(kproblem+3,kte))
L00415            print*,"====v:",v1(max(kproblem-3,1):min(kproblem+3,kte))
L00416            print*,"====p:",pres1(max(kproblem-3,1):min(kproblem+3,kte))
L00417         endif
L00418      endif
L00419
```

```
L00420  !***  Begin debugging
L00421      IMD=10
L00422      JMD=10
L00423  !***  End debugging
L00424
L00425      !initialize 3d output arrays
L00426      edmf_a1       =zero
L00427      edmf_w1       =zero
L00428      edmf_qt1      =zero
L00429      edmf_thl1     =zero
L00430      edmf_ent1     =zero
L00431      edmf_qc1      =zero
L00432      edmf_u1       =zero
L00433      edmf_v1       =zero
L00434      edmf_qv1      =zero
L00435      sub_thl1      =zero
L00436      sub_sqv1      =zero
L00437      det_thl1      =zero
L00438      det_sqv1      =zero
L00439      edmf_a_dd1    =zero
L00440      edmf_w_dd1    =zero
L00441      edmf_qc_dd1   =zero
L00442      !initialize 2d fields
L00443      ktop_plume    =0    !int
L00444      ztop_plume    =zero
L00445      maxwidth      =zero
L00446      maxmf         =zero
L00447      excess_h      =zero
L00448      excess_q      =zero
L00449      kzero1        =zero
```

- **入口体检**（debug_code 开启时）：hfx>1200 W/m²、风速 >270 m/s、温度出 [160,380] K、负比湿——**输入越界打印格点现场**（排查物理-动力接口异常的第一入口）。
- **EDMF 输出数组清零**：`edmf_*1`（上升流面积/速度/属性）、`sub_*/det_*`（次网格通量的上升/下沉分解）——每列每步从零算起。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L451-L584
L00451      ! DH* CHECK HOW MUCH OF THIS INIT IF-BLOCK IS ACTUALLY NEEDED FOR RESTARTS
L00452  !> - Within the MYNN-EDMF, there is a dependecy check for the first time step,
L00453  !! If true, a three-dimensional initialization loop is entered. Within this loop,
L00454  !! several arrays are initialized and k-oriented (vertical) subroutines are called
L00455  !! at every i and j point, corresponding to the x- and y- directions, respectively.
L00456      if (initflag > 0 .and. .not.restart) then
L00457
L00458         !test to see if we want to initialize qke
L00459         if ( (restart .or. cycling)) then
L00460            if (maxval(qke1(:)) < 0.0002) then
L00461               initialize_qke = .true.
L00462               !print*,"qke is too small, must initialize"
L00463            else
L00464               initialize_qke = .false.
L00465               !print*,"using background qke, will not initialize"
L00466            endif
L00467         else ! not cycling or restarting:
L00468            initialize_qke = .true.
L00469            !print*,"not restart nor cycling, must initialize qke"
L00470         endif
L00471
L00472         if (.not.restart .or. .not.cycling) THEN
L00473            sh1          =zero
L00474            sm1          =zero
L00475            el1          =zero
L00476            tsq1         =zero
L00477            qsq1         =zero
L00478            cov1         =zero
L00479            cldfra_bl1   =zero
L00480            qc_bl1       =zero
L00481            qi_bl1       =zero
L00482            qke1         =zero
L00483            qke_adv1     =zero
L00484         end if
L00485         dqc1            =zero
L00486         dqi1            =zero
L00487         dqni1           =zero
L00488         dqnc1           =zero
L00489         dqnwfa1         =zero
L00490         dqnifa1         =zero
L00491         dqnbca1         =zero
L00492         dozone1         =zero
L00493         qc_bl1_old      =zero
L00494         cldfra_bl1_old  =zero
L00495         sgm1            =zero
L00496         vt1             =zero
L00497         vq1             =zero
L00498         km1             =zero
L00499         kh1             =zero
L00500
L00501         if (tke_budget .eq. 1) then
L00502            qwt1         =zero
L00503            qshear1      =zero
L00504            qbuoy1       =zero
L00505            qdiss1       =zero
L00506            dqke1        =zero
L00507         endif
L00508
L00509         zw1(kts)=zero
L00510         do k=kts,kte
L00511            !keep snow out for now - increases ceiling bias
L00512            sqw1(k)=sqv1(k)+sqc1(k)+sqi1(k)!+sqs1(k)
L00513            thl1(k)=th1(k) - xlvcp/ex1(k)*sqc1(k) &
L00514                 &         - xlscp/ex1(k)*(sqi1(k))!+sqs1(k))
L00515            !Use form from Tripoli and Cotton (1981) with their
L00516            !suggested min temperature to improve accuracy.
L00517            !thl1(k)=th1(k)*(1.- xlvcp/MAX(tk1(k),TKmin)*sqc1(k) &
L00518            !    &             - xlscp/MAX(tk1(k),TKmin)*sqi1(k))
```

```
L00519           thlv1(k)=thl1(k)*(one+p608*sqv1(k))
L00520           thv1(k)=th1(k)*(one+p608*sqv1(k) - (sqc1(k)+sqi1(k)))
L00521           zw1(k+1)=zw1(k)+dz1(k)
L00522        enddo
L00523
L00524        if (INITIALIZE_QKE) then
L00525           !Initialize tke for initial PBLH calc only - using
L00526           !simple PBLH form of Koracin and Berkowicz (1988, BLM)
L00527           !to linearly taper off tke towards top of PBL.
L00528           do k=kts,kte
L00529              qke1(k)=five * ust * MAX((ust*700._kind_phys - zw1(k))/(MAX(ust,p01)*700.), p01)
L00530           enddo
L00531        endif
L00532
L00533
L00534  !>  - Call get_pblh() to calculate hybrid (\f$\theta_{v}-TKE\f$) PBL height.
L00535        call get_pblh(kts,kte,pblh,thv1,qke1,ust,zw1,dz1,xland,kpbl)
L00536
L00537  !>  - Call scale_aware() to calculate similarity functions for scale-adaptive control
L00538  !! (\f$P_{\sigma-PBL}\f$ and \f$P_{\sigma-shcu}\f$).
L00539        call scale_aware(dx,pblh,psig_bl,psig_shcu)
L00540
L00541        ! DH* CHECK IF WE CAN DO WITHOUT CALLING THIS ROUTINE FOR RESTARTS
L00542  !>  - Call mym_initialize() to initializes the mixing length, TKE, \f$\theta^{'2}\f$,
L00543  !! \f$q^{'2}\f$, and \f$\theta^{'}q^{'}\f$. These variables are calculated after
L00544  !! obtaining prerequisite variables by calling the following subroutines from
L00545  !! within mym_initialize(): mym_level2() and mym_length().
L00546        rmol = zero
L00547        rmolh= zero
L00548        call mym_initialize (                &
L00549             &kts,kte,ckmod,taue,xland,pblh, &
L00550             &dz1, dx, zw1, pres1, ex1,      &
L00551             &u1, v1, thl1, sqv1,            &
L00552             &th1, thv1, thlv1,              &
L00553             &sh1, sm1,                      &
L00554             &ust, rmol, rmolh,              &
L00555             &el1, qke1,                     &
L00556             &tsq1, qsq1, cov1,              &
L00557             &psig_bl, cldfra_bl1,           &
L00558             &bl_mynn_mixlength,             &
L00559             &bl_mynn_ess,                   &
L00560             &edmf_w1,edmf_a1,               &
L00561             &edmf_w_dd1,edmf_a_dd1,         &
L00562             &initialize_qke,                &
L00563             &spp_pbl,pattern_spp_pbl1       )
L00564
L00565        if (.not.restart) then
L00566           !initialize qke_adv array if using advection
L00567           IF (bl_mynn_tkeadvect) THEN
L00568              DO k=KTS,KTE
L00569                 qke_adv1(k)=qke1(k)
L00570              ENDDO
L00571           ENDIF
L00572        ENDIF
L00573
L00574  !***  Begin debugging
L00575  !       IF (I==IMD .AND. J==JMD) THEN
L00576  !          PRINT*,"MYNN DRIVER INIT: k=",1," sh=",sh1(k)
L00577  !          PRINT*," sqw=",sqw1(k)," thl=",thl1(k)," k_m=",km1(k)
L00578  !          PRINT*," xland=",xland," rmol=",rmol," ust=",ust
L00579  !          PRINT*," qke=",qke1(k)," el=",el1(k)," tsq=",tsq1(k)
L00580  !          PRINT*," PBLH=",PBLH," u=",u1(k)," v=",v1(k)
L00581  !       ENDIF
L00582  !***  End debugging
L00583
L00584     ENDIF ! end initflag
```

- **initflag 首步分支**（`initflag > 0 .and. .not.restart`）：三件事——① 清零湍流量（`sh1/sm1/el1/tsq1/…`）；② 构造热力学列：`sqw1 = sqv+sqc+sqi`（注释：雪暂不纳入——提高云底偏差）、`thl1 = th1 − xlvcp/ex • sqc − xlscp/ex • sqi`（液水位温的定义式，书 (6.3) 对应的湿系统推广）、`thv1 = th1 • (1+0.608 • sqv − (sqc+sqi))`（虚位温，书 (2.2.10) 的位温形态）；③`INITIALIZE_QKE` 判定——restart/cycling 时仅当 `maxval(qke1) < 0.0002` 才重建，否则沿用背景场。
- **QKE 初始化廓线**（Koracin & Berkowicz 1988）：`qke1(k) = 5•u*²•MAX((u*•700−z)/(max(u*, 0.01)•700), 0.01)`——以 700 m 为尺度的线性衰减，只用于首步 PBLH 估计。
- **初始化三调用**：`get_pblh`（混合 θ_v–TKE 判据的高度诊断）→ `scale_aware`（网格自适应函数 P_σ-PBL/P_σ-shcu——**灰区模拟（4–16 km）的平滑退化为局地闭合**）→ `mym_initialize`（闭合常数初始化，内部 5 次迭代，见 §09a.5）。
- `tke_budget=1` 时 TKE 收支五项（`qwt/qshear/qbuoy/qdiss/dqke`）同步清零。

09a.4.4 每步主流程：列构造与 Monin–Obukhov 参数

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L586-L678
L00586  !> - After initializing all required variables, the regular procedures
L00587  !! performed at every time step are ready for execution.
L00588     IF (bl_mynn_tkeadvect) THEN
L00589        qke1(kts:kte)=qke_adv1(kts:kte)
L00590     ENDIF
L00591
L00592     !Initialize some arrays
L00593     if (tke_budget .eq. 1) then
L00594        dqke1(kts:kte)=qke1(kts:kte)
L00595     endif
L00596     cldfra_bl1_old(kts:kte)=cldfra_bl1(kts:kte)
```

```
L00597      qc_bl1_old(kts:kte)=qc_bl1(kts:kte)
L00598      qi_bl1_old(kts:kte)=qi_bl1(kts:kte)
L00599      cldfra_bl1 =zero
L00600      qc_bl1     =zero
L00601      qi_bl1     =zero
L00602      dqc1       =zero
L00603      dqi1       =zero
L00604      dqs1       =zero
L00605      dqni1      =zero
L00606      dqnc1      =zero
L00607      dqnwfa1    =zero
L00608      dqnifa1    =zero
L00609      dqnbca1    =zero
L00610      dozone1    =zero
L00611      !edmf
L00612      edmf_a1    =zero
L00613      edmf_w1    =zero
L00614      edmf_qc1   =zero
L00615      edmf_qv1   =zero
L00616      edmf_thl1  =zero
L00617      edmf_u1    =zero
L00618      edmf_v1    =zero
L00619      s_aw1      =zero
L00620      s_awthl1   =zero
L00621      s_awqt1    =zero
L00622      s_awqv1    =zero
L00623      s_awqc1    =zero
L00624      s_awu1     =zero
L00625      s_awv1     =zero
L00626      s_awqke1   =zero
L00627      s_awqsq1   =zero
L00628      s_awqnc1   =zero
L00629      s_awqni1   =zero
L00630      s_awqnwfa1 =zero
L00631      s_awqnifa1 =zero
L00632      s_awqnbca1 =zero
L00633      s_awchem1(kts:kte+1,1:nchem) = zero
L00634      s_awscalars1(kts:kte+1,1:nscalars) = zero
L00635      !edmf-downdraft
L00636      edmf_a_dd1 =zero
L00637      edmf_w_dd1 =zero
L00638      edmf_qc_dd1=zero
L00639      sd_aw1     =zero
L00640      sd_awthl1  =zero
L00641      sd_awqt1   =zero
L00642      sd_awqv1   =zero
L00643      sd_awqc1   =zero
L00644      sd_awqi1   =zero
L00645      sd_awqnc1  =zero
L00646      sd_awqni1  =zero
L00647      sd_awqnwfa1=zero
L00648      sd_awqnifa1=zero
L00649      sd_awu1    =zero
L00650      sd_awv1    =zero
L00651      sd_awqke1  =zero
L00652      sub_thl1   =zero
L00653      sub_sqv1   =zero
L00654      sub_u1     =zero
L00655      sub_v1     =zero
L00656      det_thl1   =zero
L00657      det_sqv1   =zero
L00658      det_sqc1   =zero
L00659      det_u1     =zero
L00660      det_v1     =zero
L00661
L00662      zw1(kts)=zero
L00663      do k = kts,kte
L00664         zw1(k+1)=zw1(k)+dz1(k)
L00665         qv1(k) = sqv1(k)/(one-sqv1(k))
L00666         qc1(k) = sqc1(k)/(one-sqv1(k))
L00667         qi1(k) = sqi1(k)/(one-sqv1(k))
L00668         qs1(k) = sqs1(k)/(one-sqv1(k))
L00669         !keep snow out for now - increases ceiling bias
L00670         sqw1(k)= sqv1(k)+sqc1(k)+sqi1(k)!+sqs1(k)
L00671         thl1(k)= th1(k) - xlvcp/ex1(k)*sqc1(k) &
L00672               &         - xlscp/ex1(k)*(sqi1(k))!+sqs1(k))
L00673         !Use form from Tripoli and Cotton (1981) with their
L00674         !suggested min temperature to improve accuracy.
L00675         !thl1(k)=th1(i,k)*(one- xlvcp/MAX(tk1(k),TKmin)*sqc1(k) &
L00676         !    &                 - xlscp/MAX(tk1(k),TKmin)*sqi1(k))
L00677         thv1(k)=th1(k)*(one+p608*sqv1(k) - (sqc1(k)+sqi1(k)))
L00678      enddo ! end k
```

- **TKE 平流开关的记账法**（L588–L590）：`bl_mynn_tkeadvect` 时 `qke1 ← qke_adv1`——qke 的水平平流交给动力内核（通过 `qke_adv` 字段），物理只算垂直过程；步末再 `qke_adv1 ← qke1`（L1064）交接。**物理-动力往返交接的另一例**（与 qke 同构的做法见 T 的时间分裂）。
- **混合比转换**（L665–L668）：`qv1 = sqv1/(1-sqv1)`——比湿 → 混合比，方案内部全程用混合比。
- `zw1(k+1) = zw1(k)+dz1(k)` 构造界面高度（L662–L664 与 L521–L522 两处同构——init 与每步各一次）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L680-L716
L00680  !>  - Call get_pblh() to calculate the hybrid \f$\theta_{v}-TKE\f$
L00681  !! PBL height diagnostic.
L00682      call get_pblh(kts,kte,pblh,thv1,qke1,ust,zw1,dz1,xland,kpbl)
L00683
L00684  !>  - Call scale_aware() to calculate the similarity functions,
L00685  !! \f$P_{\sigma-PBL}\f$ and \f$P_{\sigma-shcu}\f$, to control
L00686  !! the scale-adaptive behaviour for the local and nonlocal
L00687  !! components, respectively.
L00688      call scale_aware(dx,pblh,psig_bl,psig_shcu)
L00689
```

```
L00690      sqcg= zero   !ill-defined variable; qcg has been removed
L00691      cpm=cp*(one + 0.84_kind_phys*max(sqv1(kts),1e-8_kind_phys))
L00692      exnerg=(ps/p1000mb)**rcp
L00693
L00694      !-------------------------------------------------
L00695      !ORIGINAL CODE
L00696      !flt = hfx/( rho(i,kts)*cpm ) &
L00697      ! +xlvcp*ch(i)*(sqc(kts)/exner(i,kts) -sqcg/exnerg)
L00698      !flq = qfx/  rho(i,kts)       &
L00699      !    -ch(i)*(sqc(kts)   -sqcg )
L00700      !-------------------------------------------------
L00701      flqv = qfx/rho1(kts)
L00702      flqc = zero !currently no sea-spray fluxes, fog settling handled elsewhere
L00703      th_sfc = ts/ex1(kts)
L00704
L00705      ! TURBULENT FLUX FOR TKE BOUNDARY CONDITIONS
L00706      flq =flqv+flqc                  !! LATENT
L00707      flt =hfx/(rho1(kts)*cpm )-xlvcp*flqc/ex1(kts)  !! Temperature flux
L00708      fltv=flt + flqv*p608*th_sfc      !! Virtual temperature flux
L00709
L00710      ! Update 1/L using updated sfc heat flux and friction velocity
L00711      rmol= -karman*gtr*fltv/max(ust**3,1.0e-6_kind_phys)
L00712      rmolh=-karman*gtr*flt /max(ust**3,1.0e-6_kind_phys)
L00713      zet = p5*dz1(kts)*rmol
L00714      zet = max(zet, -10._kind_phys)
L00715      zet = min(zet,  10._kind_phys)
L00716      !if(i.eq.idbg)print*,"updated z/L=",zet
```

- **每步再诊断：`get_pblh/scale_aware` 每步调用（不只是 init）**——PBLH 随热力场演化。
- **表面通量（L694–L708）：`flqv = qfx/ρ`（水汽通量）、`flqc = 0`（注释：当前无海盐飞沫通量）、`flt = hfx/(ρ·cpm) − xlvcp·flqc/ex`（温度通量，感热 +潜热耦合项）、`fltv = flt + flqv·0.608·th_sfc`（虚位温通量**——浮力的正确度量，书 (2.2.10) 虚温订正在通量层的应用）。原始代码（被注释的 ORIGINAL CODE）显示曾用 ch 系数形态，现为显式通量形态。
- **Monin–Obukhov 参数（L711–L715）：`rmol = −κ·(g/θ)·fltv/max(u*³,1e-6)`** 即 1/L（Obukhov 长度倒数，书 (6.8) 稳定度参数的源码形态）；`zet = 0.5·dz1(kts)·rmol` 取最底层中点 z/L 并钳 ±10。

09a.4.5 表面相似函数分支

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L717-L745
L00717      if (bl_mynn_stfunc == 0) then
L00718         !Original Kansas-type stability functions
L00719         if ((xland-1.5) .ge. zero) then       ! WATER
L00720            if ( zet >= zero ) then
L00721               !pmz = one + (cphm_st-one) * zet
L00722               pmz = one + 3.0_kind_phys * zet
L00723               phh = one +  cphh_st * zet
L00724            else
L00725               pmz = one/    (one-cphm_unst*zet)**0.25 - zet
L00726               phh = one/sqrt(one-cphh_unst*zet)
L00727            end if
L00728         else                                  ! LAND
L00729            if ( zet >= zero ) then
L00730               !pmz = one + (cphm_st-one) * zet
L00731               pmz = 0.95_kind_phys + 3.5_kind_phys * zet
L00732               phh = one + cphh_st       * zet
L00733            else
L00734               !pmz = one/(one-cphm_unst*zet)**0.25 - zet
L00735               pmz = 0.95_kind_phys/(one-cphm_unst*zet)**0.25 - zet
L00736               phh = one       /sqrt(one-cphh_unst*zet)
L00737            end if
L00738         endif
L00739         phi_m = phim(zet)
L00740      else
L00741         !Updated stability functions (Puhales, 2020)
L00742         phi_m = phim(zet)
L00743         pmz   = phi_m - zet
L00744         phh   = phih(zet)
L00745      end if
```

- **Kansas 型稳定函数**（stfunc=0）：水面/陆地两套——水面稳定侧 `pmz = 1+3.0·zet`，不稳定侧 `pmz = (1−cphm_unst·zet)^0.25 − zet`；**陆地不稳定侧加了 0.95 系数**（陆面粗糙非均匀的经验削弱）。`phi_m = phim(zet)` 由尾部函数（code_09b phim/phih）提供。
- **Puhales (2020) 替代**（stfunc=1）：`pmz = phi_m − zet`（积分关系）、`phh = phih(zet)`——同一组函数的统一表达，消除水陆非对称。
- `pmz/phh` 是表面层（kts）的动量/热量稳定函数，供 `mym_predict` 的底边界生产项用（§09a.10）。

09a.4.6 次网格云与质量通量调用

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L747-L762
L00747  !>  - Call mym_condensation() to calculate the nonconvective component
L00748  !! of the subgrid cloud fraction and mixing ratio as well as the functions
L00749  !! used to calculate the buoyancy flux. Different cloud PDFs can be
L00750  !! selected by use of the namelist parameter \p bl_mynn_cloudpdf.
L00751
L00752      call mym_condensation (kts,kte,                 &
L00753           &dx,dz1,zw1,xland,                         &
```

```
L00754           &thl1,sqw1,sqv1,sqc1,sqi1,sqs1,             &
L00755           &pres1,ex1,tsq1,qsq1,cov1,                  &
L00756           &sh1,el1,bl_mynn_cloudpdf,                  &
L00757           &qc_bl1,qi_bl1,cldfra_bl1,                  &
L00758           &pblh,hfx,                                  &
L00759           &vt1, vq1, th1, sgm1,                       &
L00760           &bl_mynn_closure,                           &
L00761           &spp_pbl, pattern_spp_pbl1                  )
L00762
```

`mym_condensation`（§09a.11）：输出次网格云量 `qc_bl1/qi_bl1/cldfra_bl1` 与**浮力通量函数 vt/vq**——后者把 “云水凝结对浮力的贡献” 折算进湍流闭合（NN2009 附录 B 的 β_θ/β_q）。`bl_mynn_cloudpdf` 三选（0/1/2）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L764-L812
L00764      TKEprod_up = zero
L00765      if (bl_mynn_edmf > 0) then
L00766         !PRINT*,"Calling DMP Mass-Flux"
L00767         call DMP_mf(i,j,                                 &
L00768              &kts,kte,delt,zw1,dz1,pres1,rho1,           &
L00769              &bl_mynn_edmf_mom,                          &
L00770              &bl_mynn_edmf_tke,                          &
L00771              &bl_mynn_mixscalars,                        &
L00772              &bl_mynn_mixaerosols,                       &
L00773              &bl_mynn_mixnumcon,                         &
L00774              &bl_mynn_closure,                           &
L00775              &u1,v1,w1,th1,thl1,thv1,tk1,                &
L00776              &sqw1,sqv1,sqc1,qke1,qsq1,                  &
L00777              &qnc1,qni1,qnwfa1,qnifa1,qnbca1,            &
L00778              &ex1,vt1,vq1,sgm1,                          &
L00779              &ust,flt,fltv,flq,flqv,                     &
L00780              &pblh,kpbl,dx,                              &
L00781              &xland,th_sfc,                              &
L00782              ! now outputs - tendencies
L00783              ! &,dth1mf,dqv1mf,dqc1mf,du1mf,dv1mf        &
L00784              ! outputs - updraft properties
L00785              &edmf_a1,edmf_w1,edmf_qt1,                  &
L00786              &edmf_thl1,edmf_ent1,edmf_qc1,              &
L00787              &edmf_qv1,edmf_u1,edmf_v1,                  &
L00788              ! for the solver
L00789              &s_aw1,s_awthl1,s_awqt1,                    &
L00790              &s_awqv1,s_awqc1,                           &
L00791              &s_awu1,s_awv1,s_awqke1,s_awqsq1,           &
L00792              &s_awqnc1,s_awqni1,                         &
L00793              &s_awqnwfa1,s_awqnifa1,s_awqnbca1,          &
L00794              &sub_thl1,sub_sqv1,                         &
L00795              &sub_u1,sub_v1,                             &
L00796              &det_thl1,det_sqv1,det_sqc1,                &
L00797              &det_u1,det_v1,                             &
L00798              ! chem/smoke mixing
L00799              &nchem,chem1,s_awchem1,                     &
L00800              &mix_chem,                                  &
L00801              &nscalars,scalars,s_awscalars1,             &
L00802              &qc_bl1,cldfra_bl1,                         &
L00803              &qc_bl1_old,cldfra_bl1_old,                 &
L00804              &FLAG_QC,FLAG_QI,                           &
L00805              &FLAG_QNC,FLAG_QNI,                         &
L00806              &FLAG_QNWFA,FLAG_QNIFA,FLAG_QNBCA,          &
L00807              &Psig_shcu,                                 &
L00808              &maxwidth,ktop_plume,                       &
L00809              &maxmf,ztop_plume,excess_h,excess_q,        &
L00810              &spp_pbl,pattern_spp_pbl1,                  &
L00811              &TKEprod_up,el1                             )
L00812      endif
```

- **`DMP_mf`（上升质量通量）**：`bl_mynn_edmf > 0` 时调用——实参约 60 个（上升流求解器是全方案最重的子程序，1345 行，code_09b）。输出 `edmf_a1/w1/qt1/thl1/ent1/qc1`（面积、速度、总水、液水位温、卷入率、云水）与界面通量 `s_aw*1` 族。
- **`TKEprod_up` 输出**：上升流对 TKE 的非局地生产——进 `mym_turbulence` 的生产项（L2848 起各占一半）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L814-L861
L00814      !> Options for downdrafts or analytical top-down method:
L00815      !!  - Add TKE source driven by cloud top cooling and
L00816      !!  calculate the buoyancy production of TKE from cloud-top cooling
L00817      !!  for downdraft or analytic options.
L00818      tkeprod_dn     = zero
L00819      maxtkeprod     = zero
L00820      cldtop_cooling = zero
L00821      ent_eff        = zero
L00822      maxmf_dd       = zero
L00823      maxwidth_dd    = zero
L00824      if (bl_mynn_edmf_dd > 0) then
L00825         call ddmp_mf(kts,kte,delt,dx,zw1,dz1,pres1,    &
L00826              &u1,v1,th1,thl1,thv1,tk1,                 &
L00827              &sqw1,sqv1,sqc1,sqi1,qnc1,qni1,           &
L00828              &qnwfa1,qnifa1,                           &
L00829              &qke1,rho1,ex1,                           &
L00830              &qc_bl1,qi_bl1,cldfra_bl1,                &
L00831              &ust,flt,flq,fltv,                        &
L00832              &pblh,kpbl,                               &
L00833              &edmf_a_dd1,edmf_w_dd1,edmf_qt_dd1,       &
L00834              &edmf_thl_dd1,edmf_ent_dd1,               &
L00835              &edmf_qc_dd1,                             &
L00836              &sd_aw1,sd_awthl1,sd_awqt1,               &
L00837              &sd_awqv1,sd_awqc1,sd_awqi1,              &
L00838              &sd_awqnc1,sd_awqni1,                     &
L00839              &sd_awqnwfa1,sd_awqnifa1,                 &
L00840              &sd_awu1,sd_awv1,                         &
```

```
L00841              &sd_awqke1,                                 &
L00842              &tkeprod_dn,el1,                            &
L00843              &rthraten1,psig_bl,                         &
L00844              &maxmf_dd,maxwidth_dd                       )
L00845
L00846         !make sure there is some the production for shallow fog,
L00847         !when the nonlocal (mf) approach is no longer appropriate.
L00848         call topdown_cloudrad(kts,kte,                   &
L00849              &dz1,zw1,fltv,u1(kts),v1(kts),              &
L00850              &xland,kpbl,pblh,                           &
L00851              &sqc1,sqi1,sqw1,thl1,th1,                   &
L00852              &ex1,pres1,rho1,thv1,                       &
L00853              &cldfra_bl1,qc_bl1,qi_bl1,rthraten1,        &
L00854              &tkeprod_dn,psig_bl,                        &
L00855              maxtkeprod,cldtop_cooling,ent_eff           )
L00856      endif
L00857
L00858      !Capability to substep the eddy-diffusivity portion
L00859      !do nsub = 1,2
L00860      delt2 = delt !*0.5    !only works if topdown=0
L00861
```

- 下沉流（`bl_mynn_edmf_dd > 0`）：`ddmp_mf`（546 行，code_09b）+ 云顶辐射冷却的 **TKE 源** `topdown_cloudrad`——层云顶辐射冷却驱动下沉流的 “解析自上而下” 通道（Shall 雾/层云的关键）。注释明说 “浅雾在非局地（mf）方法失效时也要保证有 TKE 生产”。
- `cldtop_cooling/ent_eff/maxtkeprod/maxmf_dd` 是研究输出（诊断下沉流效率）。

09a.4.7 湍流闭合与二阶矩预测

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L862-L915
L00862      !thlv with updated subgrid clouds
L00863      do k=kts,kte
L00864         qc_tot1(k) =max(qc_bl1(k),sqc1(k))
L00865         qi_tot1(k) =max(qi_bl1(k),sqi1(k)+sqs1(k))
L00866         thlv1(k)   =(th1(k) - xlvcp/ex1(k)*qc_tot1(k)  &
L00867              &              - xlscp/ex1(k)*qi_tot1(k)) &
L00868              &             * (one+p608*sqv1(k))
L00869         thl_tot1(k)=th1(k)  - xlvcp/ex1(k)*qc_tot1(k)  &
L00870              &              - xlscp/ex1(k)*qi_tot1(k)
L00871         thv1(k)    =th1(k)*(one+p608*sqv1(k) - (qc_tot1(k)+qi_tot1(k)))
L00872      enddo
L00873
L00874      call mym_turbulence(                                &
L00875             &kts,kte,xland,bl_mynn_closure,ckmod,taue,   &
L00876             &dz1, dx, zw1, pres1, ex1,                   &
L00877             &u1, v1, thl1, thv1, thlv1,                  &
L00878             &sqc1, sqw1,                                 &
L00879             &qke1, tsq1, qsq1, cov1,                     &
L00880             &vt1, vq1,                                   &
L00881             &rmol, rmolh, flt, fltv, flq,                &
L00882             &pblh, th1,                                  &
L00883             &sh1,sm1,el1,                                &
L00884             &Dfm1,Dfh1,Dfq1,                             &
L00885             &Tcd1,Qcd1,Pdk1,                             &
L00886             &Pdt1,Pdq1,Pdc1,                             &
L00887             &qWT1,qSHEAR1,qBUOY1,qDISS1,                 &
L00888             &tke_budget,                                 &
L00889             &Psig_bl,Psig_shcu,                          &
L00890             &cldfra_bl1,bl_mynn_mixlength,               &
L00891             &bl_mynn_ess,                                &
L00892             &edmf_w1,edmf_a1,                            &
L00893             &edmf_w_dd1,edmf_a_dd1,                      &
L00894             &TKEprod_dn,TKEprod_up,                      &
L00895             &spp_pbl,pattern_spp_pbl1                    )
L00896
L00897  !>  - Call mym_predict() to solve TKE and
L00898  !! \f$\theta^{'2}, q^{'2}, and \theta^{'}q^{'}\f$
L00899  !! for the following time step.
L00900      call mym_predict(kts,kte,bl_mynn_closure,           &
L00901             &delt2, dz1,                                 &
L00902             &ust, flt, flq, pmz, phh,                    &
L00903             &el1, dfq1, rho1, pdk1, pdt1, pdq1, pdc1,    &
L00904             &qke1, tsq1, qsq1, cov1,                     &
L00905             &s_aw1, s_awqke1, s_awqsq1,                  &
L00906             &bl_mynn_edmf, bl_mynn_edmf_tke,             &
L00907             &qWT1, qDISS1, tke_budget, upwind            )
L00908
L00909  !>  - Calculate the heating due to dissipation of TKE
L00910  !! Set max dissipative heating rate to 7.2 K per hour and limit heating above 100 mb.
L00911      do k=kts,kte-1
L00912         diss_heat1(k) = min(max((qke1(k)**1.5_kind_phys)/(b1*max(p5*(el1(k)+el1(k+1)),one))/cp, zero),0.002_kind_phys)
L00913         diss_heat1(k) = diss_heat1(k) * exp(-10000._kind_phys/max(pres1(k),one))
L00914      enddo
L00915      diss_heat1(kte) = zero
```

- **总凝结量的热力学重建**（L863–L872）：`qc_tot1 = max(qc_bl1, sqc1)`（次网格与网格尺度取大）、`qi_tot1 = max(qi_bl1, sqi1+sqs1)`——**雪算进冰相浮力**（与 sqw1 的 “不含雪” 形成对照：云量不含雪、浮力含雪）；`thl_tot1/thv1/thlv1` 按总凝结量重建。
- **`mym_turbulence`**（§09a.9）：算混合长、稳定函数、生产项、扩散系数。
- **`mym_predict`**（§09a.10）：Crank–Nicolson 隐式推进 qke/tsq/qsq/cov。
- **TKE 耗散加热**（L909–L915）：`diss_heat = min(qke^1.5/(b1•el)/cp, 0.002)`——**上限 0.002 K/s = 7.2 K/h**；再乘 `exp(−10000/p)`（100 hPa 以上指数衰减归零）——湍流耗散加热到平流层要关掉，物理直觉直接编码。

09a.4.8 倾向求解与收尾

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L917-L1035
L00917  !>  - Call mynn_tendencies() to solve for tendencies of
L00918  !! \f$U, V, \theta, q_{v}, q_{c}, and q_{i}\f$.
L00919      call mynn_tendencies(kts,kte,i,                                 &
L00920               &delt, dz1, zw1, xland, pblh, rho1,                    &
L00921               &u1, v1, th1, tk1, qv1,                                &
L00922               &qc1, qi1, kzero1, qnc1, qni1,                         & !kzero replaces qs1 - not mixing snow
L00923               &ps, pres1, ex1, thl1,                                 &
L00924               &sqv1, sqc1, sqi1, kzero1, sqw1,                       & !kzero replaces sqs - not mixing snow
L00925               &thl_tot1, qc_tot1, qi_tot1,                           &
L00926               &qnwfa1, qnifa1, qnbca1, ozone1,                       &
L00927               &ust,flt,flq,flqv,flqc,                                &
L00928               &wspd,uoce,voce,                                       &
L00929               &tsq1, qsq1, cov1,                                     &
L00930               &tcd1, qcd1,                                           &
L00931               &dfm1, dfh1, dfq1,                                     &
L00932               &Du1, Dv1, Dth1, Dqv1,                                 &
L00933               &Dqc1, Dqi1, Dqs1, Dqnc1, Dqni1,                       &
L00934               &Dqnwfa1, Dqnifa1, Dqnbca1,                            &
L00935               &Dozone1,                                              &
L00936               &diss_heat1,                                           &
L00937               ! mass flux components
L00938               &s_aw1,s_awthl1,s_awqt1,                               &
L00939               &s_awqv1,s_awqc1,s_awu1,s_awv1,                        &
L00940               &s_awqnc1,s_awqni1,                                    &
L00941               &s_awqnwfa1,s_awqnifa1,s_awqnbca1,                     &
L00942               &sd_aw1,sd_awthl1,sd_awqt1,                            &
L00943               &sd_awqv1,sd_awqc1,sd_awqi1,                           &
L00944               &sd_awqnc1,sd_awqni1,                                  &
L00945               &sd_awqnwfa1,sd_awqnifa1,                              &
L00946               &sd_awu1,sd_awv1,                                      &
L00947               &sub_thl1,sub_sqv1,                                    &
L00948               &sub_u1,sub_v1,                                        &
L00949               &det_thl1,det_sqv1,det_sqc1,                           &
L00950               &det_u1,det_v1,                                        &
L00951               &FLAG_QC,FLAG_QI,FLAG_QNC,                             &
L00952               &FLAG_QNI,FLAG_QS,                                     &
L00953               &FLAG_QNWFA,FLAG_QNIFA,                                &
L00954               &FLAG_QNBCA,FLAG_OZONE,                                &
L00955               &cldfra_bl1,                                           &
L00956               &bl_mynn_cloudmix,                                     &
L00957               &bl_mynn_mixqt,                                        &
L00958               &bl_mynn_edmf,                                         &
L00959               &bl_mynn_edmf_mom,                                     &
L00960               &bl_mynn_mixscalars,                                   &
L00961               &bl_mynn_mixaerosols,                                  &
L00962               &bl_mynn_mixnumcon,                                    &
L00963               &upwind                                                )
L00964
L00965      if ( mix_chem ) then
L00966           call mynn_mix_chem(kts,kte,i,                              &
L00967                &delt, dz1, pblh,                                     &
L00968                &nchem, ndvel,                                        &
L00969                &chem1, vdep, settle1,                                &
L00970                &rho1, flt,                                           &
L00971                &tcd1, qcd1,                                          &
L00972                &dfh1,                                                &
L00973                &s_aw1,s_awchem1,                                     &
L00974                &emis_ant_no,                                         &
L00975                &frp,                                                 &
L00976                &enh_mix,                                             &
L00977                &bl_mynn_edmf, upwind                                 )
L00978         do ic = 1,nchem
L00979            do k = kts,kte
L00980               chem1(k,ic) = max(1.e-12, chem1(k,ic))
L00981            enddo
L00982         enddo
L00983      endif
L00984
L00985      if ( bl_mynn_mixscalars .eq. 1) then
L00986         call mynn_mix_scalars(kts,kte,i,                           &
L00987              &delt, dz1,                                           &
L00988              &nscalars, scalars,                                   &
L00989              &rho1, flt,                                           &
L00990              &tcd1, qcd1,                                          &
L00991              &dfh1,                                                &
L00992              &s_aw1,s_awscalars1,                                  &
L00993              &bl_mynn_edmf, upwind                                 )
L00994      endif
L00995
L00996
L00997      call retrieve_exchange_coeffs(kts,kte,                         &
L00998           dfm1, dfh1, dz1, km1, kh1                                 )
L00999
L01000      !update the TKE budget
L01001      if (tke_budget .eq. 1) then
L01002         !! TKE budget is now given in m**2/s**-3 (Puhales, 2020)
L01003         !! Lower boundary condtions (using similarity relationships such as the prognostic equation for Qke)
L01004         k=kts
L01005         qSHEAR1(k)    = 4.*(ust**3*phi_m/(karman*dz1(k)))-qSHEAR1(k+1) !! staggered
L01006         qBUOY1(k)     = 4.*(-ust**3*zet/(karman*dz1(k)))-qBUOY1(k+1) !! staggered
L01007         !! unstaggering SHEAR and BUOY and trasfering all TKE budget to 3D array
L01008         do k = kts,kte-1
L01009            dummy1(k) = p5*(qSHEAR1(k)+qSHEAR1(k+1)) !!! unstaggering in z
L01010            dummy2(k) = p5*(qBUOY1(k)+qBUOY1(k+1)) !!! unstaggering in z
L01011            dqke1(k)  = p5*(qke1(k)-dqke1(k))/delt
L01012         enddo
L01013         qSHEAR1       = dummy1
L01014         qBUOY1        = dummy2
L01015         !! Upper boundary conditions
L01016         k=kte
L01017         qSHEAR1(k)    = zero
L01018         qBUOY1(k)     = zero
L01019         qWT1(k)       = zero
L01020         qDISS1(k)     = zero
L01021         dqke1(k)      = zero
```

```
L01022      endif
L01023
L01024      !update updraft/downdraft properties
L01025      if (bl_mynn_output > 0) then !research mode == 1 or 2
L01026         !if mode 2, then overwrite updrafts with downdrafts
L01027         if (bl_mynn_output == 2 .and. bl_mynn_edmf_dd == 1) then
L01028            edmf_a1(kts:kte)   =edmf_a_dd1(kts:kte)
L01029            edmf_w1(kts:kte)   =edmf_w_dd1(kts:kte)
L01030            edmf_qt1(kts:kte)  =edmf_qt_dd1(kts:kte)
L01031            edmf_thl1(kts:kte) =edmf_thl_dd1(kts:kte)
L01032            edmf_ent1(kts:kte) =edmf_ent_dd1(kts:kte)
L01033            edmf_qc1(kts:kte)  =edmf_qc_dd1(kts:kte)
L01034         endif
L01035      endif
```

- **mynn_tendencies**（1574 行，code_09b）：消耗 dfm/dfh/tcd/qcd（局地扩散 +逆梯度）与 s_aw/sd_aw/sub/det（质量通量通量），产出九变量倾向（u/v/θ/qv/qc/qi/qnc/qni/qs + 气溶胶/臭氧）。
- **化学/标量混合**（L965–L994）：mynn_mix_chem（烟羽混合、沉降 vdep、火点 frp 增强 enh_mix）与 mynn_mix_scalars（通用标量）——**边界层化学耦合的内嵌通道**。
- **retrieve_exchange_coeffs**：由 dfm/dfh/dz 反解 km1/kh1 输出（给 driver/历史用）。
- **TKE 收支**（L1000–L1022）：底边界用相似律补齐（qSHEAR(kts) = 4u*³ϕm/(κ·dz) − qSHEAR(kts+1)——staggered 差分），逐层 unstagger（0.5 相邻平均），顶边界清零；dqke1 = 0.5·(qke−dqke1)/delt 还原成趋势。
- **bl_mynn_output=2 时 EDMF 诊断输出换成下沉流**（L1027–L1034）——同一组输出字段承载升/降两种视角。

09a.5 mym_initialize：闭合的迭代初始化

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L1074-L1305
L01074  !=================================================================
L01075  !     SUBROUTINE  mym_initialize:
L01076  !
L01077  !     Input variables:
L01078  !       iniflag         : <>0; turbulent quantities will be initialized
L01079  !                         = 0; turbulent quantities have been already
L01080  !                              given, i.e., they will not be initialized
L01081  !       nx, nz          : Dimension sizes of the
L01082  !                         x and z directions, respectively
L01083  !       tref            : Reference temperature                    (K)
L01084  !       dz(nz)          : Vertical grid spacings                   (m)
L01085  !                         # dz(nz)=dz(nz-1)
L01086  !       zw(nz+1)        : Heights of the walls of the grid boxes   (m)
L01087  !                         # zw(1)=0.0 and zw(k)=zw(k-1)+dz(k-1)
L01088  !       exner(nx,nz)    : Exner function at zw*h+zg             (J/kg K)
L01089  !                         defined by c_p*( p_basic/1000hPa )^kappa
L01090  !                         This is usually computed by integrating
L01091  !                         d(pi0)/dz = -h*g/tref.
L01092  !       rmol(nx)        : Inverse of the Obukhov length         (m^(-1))
L01093  !       flt, flq(nx)    : Turbulent fluxes of potential temperature and
L01094  !                         total water, respectively:
L01095  !                                   flt=-u_*Theta_*            (K m/s)
L01096  !                                   flq=-u_*qw_*            (kg/kg m/s)
L01097  !       ust(nx)         : Friction velocity                        (m/s)
L01098  !       pmz(nx)         : phi_m-zeta at z1*h+z0, where z1 (=0.5*dz(1))
L01099  !                         is the first grid point above the surafce, z0
L01100  !                         the roughness length and zeta=(z1*h+z0)*rmo
L01101  !       phh(nx)         : phi_h at z1*h+z0
L01102  !       u, v(nx,nz)     : Components of the horizontal wind        (m/s)
L01103  !       thl(nx,nz)      : Liquid water potential temperature
L01104  !                                                                  (K)
L01105  !       qw(nx,nz)       : Total water content Q_w                (kg/kg)
L01106  !
L01107  !     Output variables:
L01108  !       ql(nx,nz)       : Liquid water content                   (kg/kg)
L01109  !       vt, vq(nx,nz)   : Functions for computing the buoyancy flux
L01110  !       qke(nx,nz)      : Twice the turbulent kinetic energy q^2
L01111  !                                                           (m^2/s^2)
L01112  !       tsq(nx,nz)      : Variance of Theta_l                  (K^2)
L01113  !       qsq(nx,nz)      : Variance of Q_w
L01114  !       cov(nx,nz)      : Covariance of Theta_l and Q_w            (K)
L01115  !       el(nx,nz)       : Master length scale L                    (m)
L01116  !                         defined on the walls of the grid boxes
L01117  !
L01118  !     Work arrays:        see subroutine mym_level2
L01119  !       pd?(nx,nz,ny) : Half of the production terms at Level 2
L01120  !                         defined on the walls of the grid boxes
L01121  !       qkw(nx,nz,ny) : q on the walls of the grid boxes        (m/s)
L01122  !
L01123  !     # As to dtl, ...gh, see subroutine mym_turbulence.
L01124  !
L01125  !-------------------------------------------------------------------
L01126
L01127  !>\ingroup gsd_mynn_edmf
L01128  !! This subroutine initializes the mixing length, TKE, \f$\theta^{'2}\f$,
L01129  !! \f$q^{'2}\f$, and \f$\theta^{'}q^{'}\f$.
L01130  !!\section gen_mym_ini GSD MYNN-EDMF mym_initialize General Algorithm
L01131  !> @{
L01132    SUBROUTINE  mym_initialize (                                &
L01133         &            kts,kte,ckmod,taue,xland,pblh,           &
L01134         &            dz, dx, zw, p, exner,                    &
L01135         &            u, v, thl, qw,                           &
L01136  !       &             ust, rmo, pmz, phh, flt, flq,            &
```

```
L01137         &            theta, thv, thlv, sh, sm,                &
L01138         &            ust, rmol, rmolh, el,                    &
L01139         &            Qke, Tsq, Qsq, Cov,                      &
L01140         &            Psig_bl, cldfra_bl1,                     &
L01141         &            bl_mynn_mixlength,                       &
L01142         &            bl_mynn_ess,                             &
L01143         &            edmf_w1,edmf_a1,                         &
L01144         &            edmf_w_dd1,edmf_a_dd1,                   &
L01145         &            INITIALIZE_QKE,                          &
L01146         &            spp_pbl,pattern_spp_pbl1                 )
L01147  !
L01148  !-------------------------------------------------------------------
L01149      use module_bl_mynnedmf_common, only: b1,b2,gtr,karman,    &
L01150           qkemin,zero,one,two,three,hundred,p01,p333,p5,p666,  &
L01151           kind_phys
L01152
L01153      integer, intent(in)           :: kts,kte
L01154      integer, intent(in)           :: bl_mynn_mixlength
L01155      integer, intent(in)           :: bl_mynn_ess
L01156      logical, intent(in)           :: INITIALIZE_QKE
L01157  !    real(kind_phys), intent(in)   :: ust, rmol, pmz, phh, flt, flq
L01158      real(kind_phys), intent(in)   :: rmol, rmolh, Psig_bl, xland, pblh
L01159      real(kind_phys), intent(in)   :: dx, ust, ckmod, taue
L01160      real(kind_phys), dimension(kts:kte),   intent(in) :: dz
L01161      real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L01162      real(kind_phys), dimension(kts:kte),   intent(in) :: u,v,thl,&
L01163           &thlv,qw,cldfra_bl1,edmf_w1,edmf_a1,edmf_w_dd1,edmf_a_dd1,&
L01164           &p,exner
L01165      real(kind_phys), dimension(kts:kte),   intent(inout) :: tsq,qsq,cov
L01166      real(kind_phys), dimension(kts:kte),   intent(inout) :: el,qke
L01167      real(kind_phys), dimension(kts:kte) ::                       &
L01168           &ql,pdk,pdt,pdq,pdc,dtl,dqw,dtv,                        &
L01169           &gm,gh,sm,sh,qkw,qtw,vt,vq,qpe
L01170      integer :: k,l,lmax
L01171      real(kind_phys):: phm,vkz,elq,elv,b1l,b2l,pmz=1.,phh=1.,     &
L01172           &flt=0.,fltv=0.,flq=0.,tmpq
L01173      real(kind_phys):: ustm
L01174      real(kind_phys), dimension(kts:kte) :: theta,thv
L01175      real(kind_phys), dimension(kts:kte) :: pattern_spp_pbl1
L01176      integer ::spp_pbl
L01177
L01178  !> - At first ql, vt and vq are set to zero.
L01179      DO k = kts,kte
L01180         ql(k) = zero
L01181         vt(k) = zero
L01182         vq(k) = zero
L01183         qkw(k)= zero
L01184         qtw(k)= zero
L01185      END DO
L01186      ustm = max(p01, ust)
L01187  !
L01188  !> - Call mym_level2() to calculate the stability functions at level 2.
L01189      CALL mym_level2 ( kts,kte,                                 &
L01190           &            ckmod, bl_mynn_ess,                     &
L01191           &            zw, dz, xland, pblh,                    &
L01192           &            u, v, thl, thv, thlv,                   &
L01193           &            theta, p, exner,                        &
L01194           &            qke, cldfra_bl1,                        &
L01195           &            edmf_a1, edmf_w1,                       &
L01196           &            qw, ql, vt, vq,                         &
L01197           &            dtl, dqw, dtv, gm, gh, sm, sh )
L01198  !
L01199  !   **  Preliminary setting  **
L01200
L01201      el(kts) = zero
L01202      IF (INITIALIZE_QKE) THEN
L01203         !qke(kts) = ust**2 * ( b1*pmz )**(2.0/3.0)
L01204         qke(kts) = 1.5_kind_phys * ustm**2 * ( b1*pmz )**p666
L01205         DO k = kts+1,kte
L01206            !qke(k) = 0.0
L01207            !linearly taper off towards top of pbl
L01208            qke(k)=qke(kts)*MAX((ust*700._kind_phys - zw(k))/(ustm*700._kind_phys), p01)
L01209         ENDDO
L01210      ENDIF
L01211  !
L01212      phm      = phh*b2 / ( b1*pmz )**p333
L01213      tsq(kts) = min(three,          phm*( max(zero,flt)/ustm )**2)
L01214      qsq(kts) = min(5e-6_kind_phys, phm*( max(zero,flq)/ustm )**2)
L01215      cov(kts) = phm*( flt/ustm )*( flq/ustm )
L01216      qpe(kts) = min(three, max(zero, two * tsq(kts) * gtr**2 * taue**2))
L01217  !
L01218      DO k = kts+1,kte
L01219         vkz = karman*zw(k)
L01220         el(k) = vkz/( one + vkz/hundred )
L01221  !       qke(k) = 0.0
L01222  !
L01223         tsq(k) = zero
L01224         qsq(k) = zero
L01225         cov(k) = zero
L01226         qpe(k) = zero
L01227      END DO
L01228  !
L01229  !   **  Initialization with an iterative manner          **
L01230  !   **  lmax is the iteration count. This is arbitrary.  **
L01231      lmax = 5
L01232  !
L01233      DO l = 1,lmax
L01234  !
L01235  !> - call mym_length() to calculate the master length scale.
L01236         CALL mym_length (                                       &
L01237              &            kts,kte,xland,                       &
L01238              &            dz, dx, zw,                          &
L01239              &            rmol, rmolh,                         &
L01240              &            flt, fltv, flq,                      &
L01241              &            vt, vq,                              &
L01242              &            u, v, qke, qpe,                      &
L01243              &            dtv,                                 &
L01244              &            el,                                  &
```

```
L01245              &            pblh,theta,              &
L01246              &            qkw, qtw,                &
L01247              &            Psig_bl,cldfra_bl1,      &
L01248              &            bl_mynn_mixlength,       &
L01249              &            edmf_w1,edmf_a1,         &
L01250              &            edmf_w_dd1,edmf_a_dd1    )
L01251  !
L01252         DO k = kts+1,kte
L01253            elq    = el(k)*qkw(k)
L01254            pdk(k) = elq*( sm(k)*gm(k) + &
L01255                 &         sh(k)*gh(k) )
L01256            pdt(k) = elq*  sh(k)*dtl(k)**2
L01257            pdq(k) = elq*  sh(k)*dqw(k)**2
L01258            pdc(k) = elq*  sh(k)*dtl(k)*dqw(k)
L01259         END DO
L01260  !
L01261  !   **  Strictly, vkz*h(i,j) -> karman*( 0.5*dz(1)*h(i,j)+z0 )  **
L01262         vkz = karman*p5*dz(kts)
L01263         elv = p5*( el(kts+1)+el(kts) ) /  vkz
L01264         IF (INITIALIZE_QKE)THEN
L01265            !qke(kts) = ust**2 * ( b1*pmz*elv    )**(2.0/3.0)
L01266            qke(kts) = one * ustm**2 * ( b1*pmz*elv    )**p666
L01267         ENDIF
L01268
L01269         phm      = phh*b2 / ( b1*pmz/elv**2 )**p333
L01270         tsq(kts) = min(three,          phm*( max(zero,flt)/ustm )**2)
L01271         qsq(kts) = min(5e-6_kind_phys, phm*( max(zero,flq)/ustm )**2)
L01272         cov(kts) = phm*( flt/ustm )*( flq/ustm )
L01273
L01274         DO k = kts+1,kte-1
L01275            b1l = b1*0.25*( el(k+1)+el(k) )
L01276            !tmpq=MAX(b1l*( pdk(k+1)+pdk(k) ),qkemin)
L01277            !add MIN to limit unreasonable QKE
L01278            tmpq=MIN(MAX(b1l*( pdk(k+1)+pdk(k) ),qkemin),125._kind_phys)
L01279            !print*,'tmpq=',tmpq,pdk(k+1),pdk(k)
L01280            IF (INITIALIZE_QKE)THEN
L01281               qke(k) = tmpq**p666
L01282            ENDIF
L01283
L01284            IF ( qke(k) .LE. zero ) THEN
L01285               b2l = zero
L01286            ELSE
L01287               b2l = b2*( b1l/b1 ) / SQRT( qke(k) )
L01288            END IF
L01289
L01290            tsq(k) = min(three,          max(zero, b2l*( pdt(k+1)+pdt(k) )))
L01291            qsq(k) = min(5e-6_kind_phys, max(zero, b2l*( pdq(k+1)+pdq(k) )))
L01292            cov(k) = b2l*( pdc(k+1)+pdc(k) )
L01293         END DO
L01294
L01295      END DO
L01296
L01297      IF (INITIALIZE_QKE)THEN
L01298         qke(kts)=p5*(qke(kts)+qke(kts+1))
L01299         qke(kte)=qke(kte-1)
L01300      ENDIF
L01301      tsq(kte)=tsq(kte-1)
L01302      qsq(kte)=qsq(kte-1)
L01303      cov(kte)=cov(kte-1)
L01304
L01305    END SUBROUTINE mym_initialize
```

- **任务**：给定初值场，算出 el/qke/tsq/qsq/cov 的“自洽初值”。核心是 **5 次迭代**（L1231 lmax = 5）：每轮 mym_level2（稳定函数）→ mym_length（混合长）→ 由生产项半积分诊断 pdk/pdt/pdq/pdc（L1252–L1259）→ 更新 qke/二阶矩（L1264–L1293）。**闭合方程组的初值靠不动点迭代**，这是 1.5 阶闭合初始化的标准做法。
- **底边界方差**（L1212–L1216）：phm = phh·b2/(b1·pmz)^(1/3)，tsq(kts) = min(3, phm·(max(0,flt)/u*)²)——表面温度方差由 Monin–Obukhov 相似律给出；qpe = 2·tsq·(g/θ)²·taue²（TTE 闭合的潜在能量初值）。
- **钳位**：tmpq = MIN(MAX(b1l·(pdk(k+1)+pdk(k)), qkemin), 125)——**qke 上限 125**（注释：防止不合理 QKE）；tsq ≤ 3、qsq ≤ 5e-6。
- INITIALIZE_QKE 为假时只算 el/二阶矩、qke 保持输入值（restart 场景）。

09a.6 mym_level2：Level 2 稳定函数与有效静力稳定度

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L1309-L1551
L01309  ! ==================================================================
L01310  !     SUBROUTINE  mym_level2:
L01311  !
L01312  !     Input variables:    see subroutine mym_initialize
L01313  !
L01314  !     Output variables:
L01315  !       dtl(nx,nz,ny) : Vertical gradient of Theta_l             (K/m)
L01316  !       dqw(nx,nz,ny) : Vertical gradient of Q_w
L01317  !       dtv(nx,nz,ny) : Vertical gradient of Theta_V             (K/m)
L01318  !       gm (nx,nz,ny) : G_M divided by L^2/q^2                (s^(-2))
L01319  !       gh (nx,nz,ny) : G_H divided by L^2/q^2                (s^(-2))
L01320  !       sm (nx,nz,ny) : Stability function for momentum, at Level 2
L01321  !       sh (nx,nz,ny) : Stability function for heat, at Level 2
L01322  !
L01323  !       These are defined on the walls of the grid boxes.
L01324  !
L01325
L01326  !>\ingroup gsd_mynn_edmf
```

```
L01327  !! This subroutine calculates the level 2, non-dimensional wind shear
L01328  !! \f$G_M\f$ and vertical temperature gradient \f$G_H\f$ as well as
L01329  !! the level 2 stability funcitons \f$S_h\f$ and \f$S_m\f$.
L01330  !!\param kts    horizontal dimension
L01331  !!\param kte    vertical dimension
L01332  !!\param dz     vertical grid spacings (\f$m\f$)
L01333  !!\param u      west-east component of the horizontal wind (\f$m s^{-1}\f$)
L01334  !!\param v      south-north component of the horizontal wind (\f$m s^{-1}\f$)
L01335  !!\param thl    liquid water potential temperature
L01336  !!\param qw     total water content \f$Q_w\f$
L01337  !!\param ql     liquid water content (\f$kg kg^{-1}\f$)
L01338  !!\param vt
L01339  !!\param vq
L01340  !!\param dtl     vertical gradient of \f$\theta_l\f$ (\f$K m^{-1}\f$)
L01341  !!\param dqw     vertical gradient of \f$Q_w\f$
L01342  !!\param dtv     vertical gradient of \f$\theta_V\f$ (\f$K m^{-1}\f$)
L01343  !!\param gm      \f$G_M\f$ divided by \f$L^{2}/q^{2}\f$ (\f$s^{-2}\f$)
L01344  !!\param gh      \f$G_H\f$ divided by \f$L^{2}/q^{2}\f$ (\f$s^{-2}\f$)
L01345  !!\param sm      stability function for momentum, at Level 2
L01346  !!\param sh      stability function for heat, at Level 2
L01347  !!\section gen_mym_level2 GSD MYNN-EDMF mym_level2 General Algorithm
L01348  !! @ {
L01349    SUBROUTINE  mym_level2 (kts,kte,                &
L01350         &            ckmod, bl_mynn_ess,           &
L01351         &            zw, dz, xland, pblh,          &
L01352         &            u, v, thl, thv, thlv,         &
L01353         &            th, p, exner,                 &
L01354         &            qke, cldfra,                  &
L01355         &            edmf_a, edmf_w,               &
L01356         &            qw, ql, vt, vq,               &
L01357         &            dtl, dqw, dtv, gm, gh, sm, sh )
L01358  !
L01359  !-------------------------------------------------------------------
L01360   use module_bl_mynnedmf_common, only: tv0,gtr,cp, &
L01361        qkemin,a1,a2,b1,c1,c2,c5,g1,g2,zero,one,two,&
L01362        three,four,ten,twenty,hundred,p1,p2,p25,p3, &
L01363        p5,p95,kind_phys
L01364
L01365   integer, intent(in)   :: kts,kte
L01366
L01367  #ifdef HARDCODE_VERTICAL
L01368  # define kts 1
L01369  # define kte HARDCODE_VERTICAL
L01370  #endif
L01371
L01372   real(kind_phys), intent(in)::xland,pblh,ckmod
L01373   integer, intent(in)        ::bl_mynn_ess
L01374   real(kind_phys), dimension(kts:kte), intent(in)  :: dz
L01375   real(kind_phys), dimension(kts:kte), intent(in)  :: u,v, &
L01376        &thl,qw,ql,vt,vq,thv,thlv,th,p,exner,edmf_a,edmf_w, &
L01377        &qke,cldfra
L01378   real(kind_phys), dimension(kts:kte+1), intent(in):: zw
L01379   real(kind_phys), dimension(kts:kte), intent(out) ::      &
L01380        &dtl,dqw,dtv,gm,gh,sm,sh
L01381   real(kind_phys), dimension(kts:kte):: ri
L01382   integer :: k
L01383   real(kind_phys):: rfc,f1,f2,rf1,rf2,smc,shc,             &
L01384        &ri1,ri2,ri3,ri4,duz,dtz,dqz,vtt,vqq,dtq,dzk,       &
L01385        &afk,abk,rf
L01386   real(kind_phys):: a2fac,thv1,thv0,eth0,eth1,lambda,qsat, &
L01387        &xl,tabs,clam0,clam,wt,mfi,qkei,cldfrai,ugrid,      &
L01388        &cfac,uonset,taper
L01389   real(kind_phys), parameter:: thvp = 0.0 !0.25 !percentage of thv in blend
L01390
L01391  !    ev  = 2.5e6
L01392  !    tv0 = 0.61*tref
L01393  !    tv1 = 1.61*tref
L01394  !    gtr = 9.81/tref
L01395  !
L01396  !intialize output
L01397   dtl(kts)=zero
L01398   dqw(kts)=zero
L01399   dtv(kts)=zero
L01400   gm(kts)=zero
L01401   gh(kts)=zero
L01402   sm(kts)=zero
L01403   sh(kts)=zero
L01404   ri(kts)=zero
L01405
L01406   select case (bl_mynn_ess)  !moist static stability
L01407
L01408      case (0) !use buoyancy flux functions to calculate effective static stability (ess)
L01409
L01410         do k = kts+1,kte
L01411            dzk = p5 *( dz(k)+dz(k-1) )
L01412            afk = dz(k)/( dz(k)+dz(k-1) )
L01413            abk = one -afk
L01414            duz = ( u(k)-u(k-1) )**2 +( v(k)-v(k-1) )**2
L01415            duz =   duz                 /dzk**2
L01416            dtz = ( thl(k)-thl(k-1) )/( dzk )
L01417            dqz = ( qw(k)-qw(k-1) )/( dzk )
L01418
L01419            vtt =  one +vt(k)*abk +vt(k-1)*afk  ! Beta-theta in NN09, Eq. 39
L01420            vqq =  tv0 +vq(k)*abk +vq(k-1)*afk  ! Beta-q
L01421            dtq =  vtt*dtz +vqq*dqz
L01422
L01423            dtl(k) =  dtz
L01424            dqw(k) =  dqz
L01425            dtv(k) =  dtq
L01426
L01427            gm(k)  =  duz
L01428            gh(k)  = -dtq*gtr
L01429
L01430            !   **  Gradient Richardson number  **
L01431            ri(k) = -gh(k)/MAX( duz, 1.0e-10_kind_phys )
L01432         enddo
L01433
L01434      case (1) !form of effective static stability (ess) similar to O'Gorman (2011, JAS)
```

```
L01435          !but modified to better fit within the MYNN-EDMF. This approach
L01436          !destabilizes the lapse rates in grid cells with tke/mf/clouds by adding
L01437          !a negative eqiv potential temperature gradient proportional to the
L01438          !turbulence and cloud fractions. The coefficient for lambda (clam) has
L01439          !units of s/m (i.e., eddy timescale / pbl depth ~ 1.8), but needs to be
L01440          !limited over land as noted below.
L01441          !-------------------------------
L01442          !taper ess for hurricane conditions
L01443          ugrid  = sqrt(u(kts)**2 + v(kts)**2)
L01444          uonset = 15._kind_phys
L01445          taper  = one - p95*min(one, max(zero, ugrid - uonset)/twenty) !reduce to 0.1
L01446          if ((xland-1.5).GE.zero) then !water
L01447             clam0 = 1.8_kind_phys * taper
L01448          else                          !land
L01449             !since the MF and TKE is much large over land and pblhs are much deeper,
L01450             !we need a smaller tuning constant.
L01451             clam0 = 0.08_kind_phys
L01452          endif
L01453
L01454          do k = kts+1,kte
L01455             dzk   = p5 *  ( dz(k)+dz(k-1) )
L01456             afk   = dz(k)/( dz(k)+dz(k-1) )
L01457             abk   = one -afk
L01458             duz   = ( u(k)-u(k-1) )**2 + ( v(k)-v(k-1) )**2
L01459             duz   =   duz              / dzk**2
L01460             dtz   = ( thl(k)-thl(k-1) ) / dzk
L01461             dqz   = ( qw(k) -qw(k-1)  ) / dzk
L01462
L01463             !blend clam to free tropospheric value (0.2) above the pblh
L01464             wt    = min(one, max(zero, zw(k)-(pblh+hundred))/max(400._kind_phys, p3*pblh)) !0 below pblh, 1 above pblh
L01465             clam  = clam0*(one-wt) + p2*wt
L01466             !choke of clam near the surface
L01467             clam  = clam * min(one, max(zero, zw(k)-twenty)/200.0_kind_phys)
L01468
L01469             !Use a blended subgrid-cloud-included theta-v and theta-l-v for background
L01470             !thermodynamic profile:
L01471             thv1  = thvp*thv(k)   + (one-thvp)*thlv(k)
L01472             thv0  = thvp*thv(k-1) + (one-thvp)*thlv(k-1)
L01473             !Then use the equivalent potential pemerature profile to adjust the sbability in partially
L01474             !saturated conditions with an upwards vertical component.
L01475             !equivalent potential temperature at k
L01476             tabs  = th(k)*exner(k)
L01477             xl    = xl_blend(tabs)            ! blended latent heat
L01478             qsat  = qsat_blend(tabs,p(k))     ! saturation water vapor mixing ratio
L01479             eth1  = th(k)*exp(xl*qsat/(cp*tabs))
L01480             !equivalent potential temperature at k-1
L01481             tabs  = th(k-1)*exner(k-1)
L01482             xl    = xl_blend(tabs)            ! blended latent heat
L01483             qsat  = qsat_blend(tabs,p(k-1))   ! saturation water vapor mixing ratio
L01484             eth0  = th(k-1)*exp(xl*qsat/(cp*tabs))
L01485
L01486             !mass flux at interface (0 to 0.2 m/s):
L01487             mfi   = max(zero, min(p2, (p5*(edmf_a(k)+edmf_a(k-1))) * (p5*(edmf_w(k)+edmf_w(k-1)))))
L01488
L01489             !normalize tke to be comparable in magnitude to marine mass flux (0 to 0.1 m/s).
L01490             !Roughly half the tke is ascending, so 0.5*tke = 0.25*qke.
L01491             qkei  = max(qkemin, p5*(qke(k)+qke(k-1)))  !avg qke at interface
L01492             qkei  = min(p5, sqrt(p25*qkei))*p2
L01493
L01494             !control factor for cloudy grid cells and/or have nonzero mass flux
L01495             cldfrai =  p5*(cldfra(k)+cldfra(k-1)) !avg cloud fraction at interface
L01496             !introduce factor for limit reduction of ess in fully resolved clouds where latent heating is explicit
L01497             cfac    = max( 0.01_kind_phys, min(2.7_kind_phys * (cldfrai - one)**2, one))
L01498             !cfac    = max( 0.01_kind_phys, min(4.5_kind_phys * (min(0.85_kind_phys,cldfrai) - 0.85_kind_phys)**2, one))
L01499             cldfrai = min(p1, cldfrai*cfac)*ten
L01500             !cldfrai  = max(ncld, min(p1,mfi)) !TEST: always allow some destabilization in grid cells with plumes.
L01501
L01502             !lambda significantly departs from OGorman (2011) by using MYNN-EDMF-specific information:
L01503             lambda= clam * max(mfi, qkei) * cldfrai
L01504             !lambda= clam * (mfi + qkei) * cldfrai
L01505             dtq   = ( thv1-thv0 )/dzk + lambda*min(zero, ( eth1-eth0 )/dzk )
L01506             !dtq = max(zero, dtq)
L01507
L01508             dtl(k)=  dtz
L01509             dqw(k)=  dqz
L01510             dtv(k)=  dtq
L01511
L01512             gm(k) =  duz
L01513             gh(k) = -dtq*gtr
L01514
L01515             !   **  Gradient Richardson number  **
L01516             ri(k) = -gh(k)/MAX( duz, 1.0e-10_kind_phys )
L01517          enddo
L01518
L01519       end select
L01520
L01521       do k = kts+1,kte
L01522          !a2fac is needed for the Canuto/Kitamura mod
L01523          a2fac = one/(one + ckmod*MAX(ri(k),zero))
L01524
L01525          rfc = g1/( g1+g2 )
L01526          f1  = b1*( g1-c1 ) +three*a2*a2fac *( one    -c2 )*( one-c5 ) &
L01527              &              +two  *a1*( three-two*c2 )
L01528          f2  = b1*( g1+g2 ) -three*a1*( one      -c2 )
L01529          rf1 = b1*( g1-c1 )/f1
L01530          rf2 = b1*  g1     /f2
L01531          smc = a1   /(a2*a2fac)*  f1/f2
L01532          shc = three*(a2*a2fac)*( g1+g2 )
L01533
L01534          ri1 = p5/smc
L01535          ri2 = rf1*smc
L01536          ri3 = four*rf2*smc -two*ri2
L01537          ri4 = ri2**2
L01538
L01539          !   **  Flux Richardson number  **
L01540          rf = MIN( ri1*( ri(k) + ri2-SQRT(ri(k)**2 - ri3*ri(k) + ri4) ), rfc )
L01541
L01542          sh(k) = shc*( rfc-rf )/( one-rf )
```

```
L01543         sm(k) = smc*( rf1-rf )/( rf2-rf ) * sh(k)
L01544      enddo
L01545
L01546  #ifdef HARDCODE_VERTICAL
L01547  # undef kts
L01548  # undef kte
L01549  #endif
L01550
L01551    END SUBROUTINE mym_level2
```

- **输出**：剪切 `gm = duz`（风切变平方/dz²）、浮力 `gh = −dtq·g/θ`（dtq 是含水汽凝结订正的 θ_v 梯度：`dtq = vtt·dtz + vqq·dqz`，vtt/vq 即 β_θ/β_q——NN2009 式 39）、梯度 Richardson 数 `ri = −gh/max(duz,1e-10)`（书 (6.8.10) 判据的源码形态，教材 ch14 §14.2）。
- **ess 两种口径**（`bl_mynn_ess` select）：
  - **case 0**（浮力通量函数法）：直接用 `vt/vq` 加权出 dtq——NN2009 原始方案。
  - **case 1**（O'Gorman 2011 修正）：在 dtq 上加 `lambda·min(0, dθe/dz)`——**用等效位温梯度在部分饱和区破稳**（湿静力稳定度的非局地订正）；`lambda = clam·max(mfi, qkei)·cldfrai`，`clam` 水面 1.8、陆地 0.08（注释明说陆地湍流太强需更小调参常数），飓风风速 >15 m/s 线性衰减（taper），层结向自由大气值 0.2 混合（wt 权重）。**这是 MYNN6 对湿边界层最重要的新物理**。
- **Level 2 稳定函数闭式**（L1521–L1544）：`a2fac = 1/(1+CKmod·max(ri,0))`（Canuto/Kitamura 因子）→ `rfc/f1/f2/rf1/rf2/smc/shc` 五个中间量（全由闭合常数 a1/a2/b1/c1/c2/c5/g1/g2 组成）→ 通量 Richardson 数 `rf` 的二次方程取小根并封顶 rfc → `sh = shc·(rfc−rf)/(1−rf)`、`sm = smc·(rf1−rf)/(rf2−rf)·sh`。**这就是 Mellor–Yamada Level 2 的解析解**——L2.5/L3 的稳定函数都从它出发（§09a.9）。

09a.7 `mym_length`：主混合长的三套配方

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L1571-L2011
L01571    SUBROUTINE  mym_length (                          &
L01572      &            kts,kte,xland,                     &
L01573      &            dz, dx, zw,                        &
L01574      &            rmol, rmolh, flt, fltv, flq,       &
L01575      &            vt, vq,                            &
L01576      &            u1, v1, qke, qpe,                  &
L01577      &            dtv,                               &
L01578      &            el,                                &
L01579      &            pblh, theta, qkw, qtw,             &
L01580      &            Psig_bl, cldfra_bl1,               &
L01581      &            bl_mynn_mixlength,                 &
L01582      &            edmf_w1,edmf_a1,                   &
L01583      &            edmf_w_dd1,edmf_a_dd1              )
L01584
L01585  !-------------------------------------------------------------------
L01586      use module_bl_mynnedmf_common, only:       &
L01587           qkemin,qmin,tv0,gtr,karman,grav,zmax, &
L01588           zero,one,two,four,five,ten,twenty,    &
L01589           thirty,forty,fifty,hundred,           &
L01590           p1,p2,p25,p3,p333,p4,p5,kind_phys
L01591
L01592      integer, intent(in)   :: kts,kte
L01593
L01594  #ifdef HARDCODE_VERTICAL
L01595  # define kts 1
L01596  # define kte HARDCODE_VERTICAL
L01597  #endif
L01598
L01599      integer, intent(in)   :: bl_mynn_mixlength
L01600      real(kind_phys), dimension(kts:kte),   intent(in) :: dz
L01601      real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L01602      real(kind_phys), intent(in) :: rmol,rmolh,flt,fltv,flq,Psig_bl,xland
L01603      real(kind_phys), intent(in) :: dx,pblh
L01604      real(kind_phys), dimension(kts:kte), intent(in)   :: u1,v1,  &
L01605           &qke,qpe,vt,vq,cldfra_bl1,edmf_w1,edmf_a1,edmf_w_dd1,edmf_a_dd1
L01606      real(kind_phys), dimension(kts:kte), intent(out)  :: el
L01607      real(kind_phys), dimension(kts:kte), intent(inout):: qkw,qtw
L01608      real(kind_phys), dimension(kts:kte), intent(in)   :: dtv
L01609      real(kind_phys):: elt,vsc
L01610      real(kind_phys), dimension(kts:kte), intent(in) :: theta
L01611      real(kind_phys), dimension(kts:kte) :: qtke,elBLmin,elBLavg,thetaw
L01612      real(kind_phys):: wt,wt2,pblh2,h1,h2,hs,elBLmin0,elBLavg0,cldavg
L01613
L01614      ! THE FOLLOWING CONSTANTS ARE IMPORTANT FOR REGULATING THE
L01615      ! MIXING LENGTHS:
L01616      real(kind_phys):: cns,   &   !< for surface layer (els) in stable conditions
L01617             alp1,             &   !< for turbulent length scale (elt)
L01618             alp2,             &   !< for buoyancy length scale (elb)
L01619             alp3,             &   !< for buoyancy enhancement factor of elb
L01620             alp4,             &   !< for surface layer (els) in unstable conditions
L01621             alp5,             &   !< for BouLac mixing length or above PBLH
L01622             alp6                  !< for mass-flux/
L01623
L01624      !THE FOLLOWING LIMITS DO NOT DIRECTLY AFFECT THE ACTUAL PBLH.
L01625      !THEY ONLY IMPOSE LIMITS ON THE CALCULATION OF THE MIXING LENGTH
L01626      !SCALES SO THAT THE BOULAC MIXING LENGTH (IN FREE ATMOS) DOES
L01627      !NOT ENCROACH UPON THE BOUNDARY LAYER MIXING LENGTH (els, elb & elt).
L01628      real(kind_phys), parameter :: minpblh     = 250.  !< min mixed-layer height
L01629      real(kind_phys), parameter :: maxdz       = 750.  !< max (half) transition layer depth
L01630                                       !! =0.3*2500 m PBLH, so the transition
L01631                                       !! layer stops growing for PBLHs > 2.5 km.
```

```
L01632      real(kind_phys), parameter :: mindz       = 250.  !< min (half) transition layer depth
L01633
L01634      !SURFACE LAYER LENGTH SCALE MODS TO REDUCE IMPACT IN UPPER BOUNDARY LAYER
L01635      real(kind_phys), parameter :: ZSLH        = 100.  !< Max height correlated to surface conditions (m)
L01636      real(kind_phys), parameter :: CSL         = 2.    !< CSL = constant of proportionality to L O(1)
L01637      real(kind_phys), parameter :: qkw_elb_min = 0.05  !this assumes some minumum TKE/TPE is present
L01638
L01639      integer :: i,j,k
L01640      real(kind_phys):: afk,abk,zwk,zwk1,dzk,qdz,vflx,bv,tau_cloud,      &
L01641            & wstar,elb,els,elf,el_stab,el_mf,el_stab_mf,elb_mf,elt_max,&
L01642            & pblh_plus_ent,uonset,ugrid,wt_u1,wt_u2,el_les,qkw_mf,     &
L01643            & z_m,el_unstab,els1,alp3z,cpblh,wt_dx
L01644      real(kind_phys), parameter :: ctau = 1000. !constant for tau_cloud
L01645
L01646  !    tv0 = 0.61*tref
L01647  !    gtr = 9.81/tref
L01648
L01649      SELECT CASE(bl_mynn_mixlength)
L01650
L01651        CASE (0) ! ORIGINAL MYNN MIXING LENGTH + BouLac
L01652
L01653          cns  = 2.7_kind_phys
L01654          alp1 = 0.23_kind_phys
L01655          alp2 = one
L01656          alp3 = five
L01657          alp4 = hundred
L01658          alp5 = p3
L01659
L01660          ! Impose limits on the height integration for elt and the transition layer depth
L01661          pblh2= pblh+dz(kts)          !originally integrated to model top, not just pblh.
L01662          h1   = max(p3*pblh2,mindz)
L01663          h1   = min(h1,maxdz)         ! 1/2 transition layer depth
L01664          h2   = h1*p5                 ! 1/4 transition layer depth
L01665
L01666          qkw(kts) = SQRT(MAX(qke(kts), qkemin))
L01667          DO k = kts+1,kte
L01668             afk = dz(k)/( dz(k)+dz(k-1) )
L01669             abk = one -afk
L01670             qkw(k) = SQRT(MAX(qke(k)*abk+qke(k-1)*afk, qkemin))
L01671          END DO
L01672          qtw = qkw
L01673
L01674          elt = 1.0e-5_kind_phys
L01675          vsc = 1.0e-5_kind_phys
L01676
L01677          !   **  Strictly, zwk*h(i,j) -> ( zwk*h(i,j)+z0 )  **
L01678          k   = kts+1
L01679          zwk = zw(k)
L01680          DO WHILE (zwk .LE. pblh2+h1)
L01681             dzk = p5*( dz(k)+dz(k-1) )
L01682             qdz = MAX( qkw(k)-qmin, 0.03 )*dzk
L01683             elt = elt +qdz*zwk
L01684             vsc = vsc +qdz
L01685             k   = k+1
L01686             zwk = zw(k)
L01687          END DO
L01688
L01689          elt  = alp1*elt/vsc
L01690          vflx = ( vt(kts)+one )*flt +( vq(kts)+tv0 )*flq
L01691          vsc  = ( gtr*elt*MAX( vflx, zero ) )**p333
L01692
L01693          !   **  Strictly, el(i,k=1) is not zero.  **
L01694          el(kts) = zero
L01695          zwk1    = zw(kts+1)
L01696
L01697          DO k = kts+1,kte
L01698             zwk = zw(k)              !full-sigma levels
L01699
L01700             !   **  Length scale limited by the buoyancy effect  **
L01701             IF ( dtv(k) .GT. zero ) THEN
L01702                bv  = SQRT( gtr*dtv(k) )
L01703                elb = alp2*qkw(k) / bv &
L01704                    &       *( one + alp3/alp2*&
L01705                    &SQRT( vsc/( bv*elt ) ) )
L01706                elf = alp2 * qkw(k)/bv
L01707
L01708             ELSE
L01709                elb = 1.0e10_kind_phys
L01710                elf = elb
L01711             ENDIF
L01712
L01713             !   **  Length scale in the surface layer  **
L01714             IF ( rmol .GT. 0.0 ) THEN
L01715                els  = karman*zwk/(one+cns*MIN( zwk*rmol, zmax ))
L01716             ELSE
L01717                els  = karman*zwk*( one - alp4* zwk*rmol )**p2
L01718             END IF
L01719
L01720             !   ** HARMONC AVERGING OF MIXING LENGTH SCALES:
L01721             !       el(k) =    MIN(elb/( elb/elt+elb/els+one ),elf)
L01722             !       el(k) =    elb/( elb/elt+elb/els+one )
L01723
L01724             wt=p5*TANH((zwk - (pblh2+h1))/h2) + p5
L01725
L01726             el(k) = MIN(elb/( elb/elt+elb/els+one ),elf)
L01727
L01728          END DO
L01729
L01730        CASE (1) !NONLOCAL (using BouLac) FORM OF MIXING LENGTH
L01731
L01732          !wt_u* are for hurricane tuning, meant to reduce diffusion in hurricanes
L01733          ugrid = sqrt(u1(kts)**2 + v1(kts)**2)
L01734          uonset= 15._kind_phys
L01735          wt_u1 = one - p2*min(one, max(zero, ugrid - uonset)/forty) !reduce to 0.8
L01736          wt_u2 = one - p5*min(one, max(zero, ugrid - uonset)/forty) !reduce to 0.5
L01737          !scale-awareness for the mesoscale greyzone (4-16 km)
L01738          wt_dx = one - min(one, (max(zero, dx-4000._kind_phys)/12000._kind_phys))
L01739
L01740          cns   = 3.5_kind_phys
```

```
L01741          alp1  = 0.23_kind_phys*wt_dx + (one-wt_dx)*0.24_kind_phys
L01742          alp2  = p3*wt_dx          + (one-wt_dx)*0.40_kind_phys
L01743          alp3  = five * wt_u2 !taper off bouyancy enhancement in shear-driven pbls
L01744          if ((xland-1.5).GE.zero) then !hurricane tuning, over water only
L01745             alp4  = twenty * wt_u2
L01746          else
L01747             alp4  = 15.0_kind_phys
L01748          endif
L01749          alp5  = p3
L01750          alp6  = fifty
L01751
L01752          ! Impose limits on the height integration for elt and the transition layer depth
L01753          pblh2 = max(pblh,200._kind_phys)
L01754          h1    = max(p3*pblh2,300._kind_phys)
L01755          h1    = min(h1,600._kind_phys)   ! 1/2 transition layer depth
L01756          h2    = h1*p5                    ! 1/4 transition layer depth
L01757
L01758          qtke(kts)  =max(p5*qke(kts), p5*qkemin) !tke at full sigma levels
L01759          thetaw(kts)=theta(kts)            !theta at full-sigma levels
L01760          qkw(kts)   =sqrt(max(qke(kts)         , qkemin))
L01761          qtw(kts)   =sqrt(max(qke(kts)+qpe(kts), qkemin))
L01762          DO k = kts+1,kte
L01763             afk      = dz(k)/( dz(k)+dz(k-1) )
L01764             abk      = one -afk
L01765             qkw(k)   = sqrt(max(qke(k)*abk+qke(k-1)*afk, qkemin))
L01766             qtw(k)   = sqrt(max((qke(k)+qpe(k))*abk+(qke(k-1)+qpe(k-1))*afk, qkemin))
L01767             qtke(k)  = max(p5*(qkw(k)**2), 0.005_kind_phys) ! q -> TKE
L01768             thetaw(k)= theta(k)*abk + theta(k-1)*afk
L01769          END DO
L01770
L01771          elt = 1.0e-5_kind_phys
L01772          vsc = 1.0e-5_kind_phys
L01773
L01774          !   **  Strictly, zwk*h(i,j) -> ( zwk*h(i,j)+z0 )  **
L01775          k   = kts+1
L01776          zwk = zw(k)
L01777          DO WHILE (zwk .LE. pblh2+max(200.,min(p3*pblh2,600._kind_phys)))
L01778             dzk = p5*( dz(k)+dz(k-1) )
L01779             qdz = min(max( qkw(k), 0.01_kind_phys ), 30.0_kind_phys)*dzk
L01780             elt = elt +qdz*zwk
L01781             vsc = vsc +qdz
L01782             k   = k+1
L01783             zwk = zw(k)
L01784          END DO
L01785
L01786          if ((xland-1.5).GE.zero) then !hurricane tuning, over water only
L01787             elt_max=350.+100.*min(one, max(zero, ugrid - twenty)/thirty)
L01788          else
L01789             elt_max=400._kind_phys
L01790          endif
L01791          elt = MIN( MAX( alp1*elt/vsc, 8._kind_phys), elt_max)
L01792          !avoid use of buoyancy flux functions which are ill-defined at the surface
L01793          !vflx = ( vt(kts)+one )*flt + ( vq(kts)+tv0 )*flq
L01794          vflx= fltv-0.005_kind_phys
L01795          vsc = ( gtr*elt*MAX( vflx, zero ) )**p333
L01796
L01797          !   **  Strictly, el(i,j,1) is not zero.  **
L01798          el(kts) = zero
L01799          zwk1    = zw(kts+1)         !full-sigma levels
L01800
L01801          ! COMPUTE BouLac mixing length
L01802          CALL boulac_length(kts,kte,zw,dz,qtke,thetaw,elBLmin,elBLavg)
L01803
L01804          DO k = kts+1,kte
L01805             zwk    = zw(k)          !full-sigma levels
L01806             qkw_mf = max((p5*((edmf_a1(k)+edmf_a1(k-1))))*(p5*((edmf_w1(k)+edmf_w1(k-1)))), &
L01807                  & abs(edmf_a_dd1(k-1)*edmf_w_dd1(k-1)))
L01808             !pblh-dependent modifier
L01809             cpblh  = min((zwk + p25*pblh2)/pblh2, one)
L01810             qkw_mf = cpblh * qkw_mf
L01811             alp3z  = cpblh * alp3
L01812             !   **  Length scale limited by the buoyancy effect  **
L01813             IF ( dtv(k) .GT. 0.0 ) THEN
L01814                bv     = max( sqrt( gtr*dtv(k) ), 0.001_kind_phys)
L01815                elb_mf = alp6*qkw_mf/bv
L01816                elb_mf = elb_mf / (one + (elb_mf/100._kind_phys))
L01817                elb    = alp2*(max(qkw(k),qkw_elb_min))/bv    &
L01818                     &  *( one + alp3z*SQRT( vsc/(bv*elt) ) )
L01819                elb    = max(elb, elb_mf)
L01820                elb    = MIN(elb, zwk)
L01821                elf    = one * max(qkw(k), qkw_elb_min)/bv
L01822                elf    = max(elf, elb_mf)
L01823                elBLavg(k) = MAX(alp5*elBLavg(k), elb_mf)
L01824             ELSE
L01825                elb        = 1.0e10_kind_phys
L01826                elf        = elb
L01827                elBLavg(k) = p4*elBLavg(k)
L01828             ENDIF
L01829
L01830             z_m = MAX(karman, zwk - four)
L01831             !   **  Length scale in the surface layer  **
L01832             IF ( rmol .GT. 0.0 ) THEN
L01833                els  = karman*zwk/(one+cns*MIN( zwk*rmol, zmax ))
L01834                els1 = karman*z_m/(one+cns*MIN( zwk*rmol, zmax ))
L01835             ELSE
L01836                els  = karman*zwk*( one - alp4* zwk*rmol)**p2
L01837                els1 = karman*z_m*( one - alp4* zwk*rmol)**p2
L01838             END IF
L01839
L01840             !   ** NOW BLEND THE MIXING LENGTH SCALES:
L01841             wt =p5*TANH((zwk - (pblh2+h1))/h2) + p5
L01842             !add blending to use BouLac mixing length in free atmos;
L01843             !defined relative to the PBLH (pblh) + transition layer (h1)
L01844             !el(k) = MIN(elb/( elb/elt+elb/els+one ),elf)
L01845             !squared blending - but take out elb (makes it underdiffusive)
L01846             !el_unstab = SQRT( els**2/(one + (els**2/elt**2) +(els**2/elb**2)))
L01847             !el_unstab = sqrt( els**2/(one + (els**2/elt**2)))
L01848             !non-squared blending:
```

```
L01849            el_unstab = els/(one + (els1/elt))
L01850            el(k)     = min(el_unstab, elb)
L01851            el(k)     = min(el(k), elf)  !elf can be smaller than elb in upper pbl
L01852            if ((xland-1.5).GE.zero) then !hurricane tuning, over water only
L01853               el(k)  = el(k)*wt_u1
L01854            endif
L01855            el(k)     = el(k)*(one-wt) + elBLavg(k)*wt
L01856
L01857            !if (el(k) > 1000.) then
L01858            !   print*,"big ML at k=",k," el=",el(k),"elb=",elb," elBL=",elBLavg(k), &
L01859            !   " elt=",elt," els=",els," N=",bv," qtke=",qtke(k), &
L01860            !   " pblh=",pblh2," zwk=",zwk," wt=",wt
L01861            !endif
L01862
L01863         END DO
L01864
L01865       CASE (2) !Local (mostly) mixing length formulation
L01866
L01867         Uonset = 3.5_kind_phys + dz(kts)*p1
L01868         Ugrid  = sqrt(u1(kts)**2 + v1(kts)**2)
L01869         cns    = 3.5_kind_phys !  * (one - MIN(MAX(Ugrid - Uonset, 0.0)/10.0, 1.0))
L01870         alp1   = 0.22_kind_phys
L01871         alp2   = p3
L01872         alp3   = 2.5_kind_phys
L01873         alp4   = five
L01874         alp5   = alp2  !like alp2, but for free atmosphere
L01875         alp6   = fifty !used for MF mixing length
L01876
L01877         ! Impose limits on the height integration for elt and the transition layer depth
L01878         !pblh2=MAX(pblh,minpblh)
L01879         pblh2=MAX(pblh, 200._kind_phys)
L01880         !h1=MAX(0.3*pblh2,mindz)
L01881         !h1=MIN(h1,maxdz)         ! 1/2 transition layer depth
L01882         h1=MAX(p3*pblh2,300._kind_phys)
L01883         h1=MIN(h1,600._kind_phys)
L01884         h2=h1*p5                 ! 1/4 transition layer depth
L01885
L01886         qtke(kts)=MAX(p5*qke(kts), p5*qkemin) !tke at full sigma levels
L01887         qkw(kts) = SQRT(MAX(qke(kts)          , qkemin))
L01888         qtw(kts) = SQRT(MAX(qke(kts)+qpe(kts), qkemin))
L01889         DO k = kts+1,kte
L01890            afk    = dz(k)/( dz(k)+dz(k-1) )
L01891            abk    = one -afk
L01892            qkw(k) = SQRT(MAX(qke(k)*abk+qke(k-1)*afk                    , qkemin))
L01893            qtw(k) = SQRT(MAX((qke(k)+qpe(k))*abk+(qke(k-1)+qpe(k-1))*afk, qkemin))
L01894            qtke(k)= p5*qkw(k)**2  ! qkw -> TKE
L01895         END DO
L01896
L01897         elt = 1.0e-5
L01898         vsc = 1.0e-5
L01899
L01900         !   **  Strictly, zwk*h(i,j) -> ( zwk*h(i,j)+z0 )  **
L01901         PBLH_PLUS_ENT = MAX(pblh+h1, 100.)
L01902         k = kts+1
L01903         zwk = zw(k)
L01904         DO WHILE (zwk .LE. PBLH_PLUS_ENT)
L01905            dzk = p5*( dz(k)+dz(k-1) )
L01906            qdz = min(max( qkw(k)-qmin, 0.03 ), 30.0)*dzk
L01907            elt = elt +qdz*zwk
L01908            vsc = vsc +qdz
L01909            k   = k+1
L01910            zwk = zw(k)
L01911         END DO
L01912
L01913         elt = MIN( MAX(alp1*elt/vsc, ten), 400.)
L01914         !avoid use of buoyancy flux functions which are ill-defined at the surface
L01915         !vflx = ( vt(kts)+one )*flt +( vq(kts)+tv0 )*flq
L01916         vflx = fltv
L01917         vsc  = ( gtr*elt*MAX( vflx, zero ) )**p333
L01918
L01919         !   **  Strictly, el(i,j,1) is not zero.  **
L01920         el(kts) = zero
L01921         zwk1    = zw(kts+1)
L01922
L01923         DO k = kts+1,kte
L01924            zwk = zw(k)              !full-sigma levels
L01925            dzk = p5*( dz(k)+dz(k-1) )
L01926            cldavg = p5*(cldfra_bl1(k-1)+cldfra_bl1(k))
L01927            qkw_mf = max((p5*(edmf_a1(k)+edmf_a1(k-1)))*(p5*(edmf_w1(k)+edmf_w1(k-1))), &
L01928                  & abs(edmf_a_dd1(k-1)*edmf_w_dd1(k-1)))
L01929
L01930            !   **  Length scale limited by the buoyancy effect  **
L01931            IF ( dtv(k) .GT. 0.0 ) THEN
L01932               !impose min value on bv
L01933               bv  = MAX( SQRT( gtr*dtv(k) ), 0.001)
L01934               !elb_mf = alp2*qkw(k) / bv  &
L01935               elb_mf = MAX(alp2*qkw(k),                    &
L01936                   &        alp6*qkw_mf) / bv               &
L01937                   &  *( one + alp3*SQRT( vsc/( bv*elt ) ) )
L01938               elb = MIN(MAX(alp5*qkw(k), alp6*qkw_mf)/bv, zwk)
L01939
L01940               !tau_cloud = MIN(MAX(0.5*pblh/((gtr*pblh*MAX(vflx,1.0e-4))**p333),30.),150.)
L01941               wstar = 1.25*(gtr*pblh*MAX(vflx,1.0e-4))**p333
L01942               tau_cloud = MIN(MAX(ctau * wstar/grav, 30.), 150.)
L01943               !minimize influence of surface heat flux on tau far away from the PBLH.
L01944               wt=p5*TANH((zwk - (pblh2+h1))/h2) + p5
L01945               tau_cloud = tau_cloud*(1.-wt) + fifty*wt
L01946               elf = MIN(MAX(tau_cloud*SQRT(MIN(qtke(k),forty)), &
L01947                   &         alp6*qkw_mf/bv), zwk)
L01948
L01949               !IF (zwk > pblh .AND. elf > 400.) THEN
L01950               !   ! COMPUTE BouLac mixing length
L01951               !   !CALL boulac_length0(k,kts,kte,zw,dz,qtke,thetaw,elBLmin0,elBLavg0)
L01952               !   !elf = alp5*elBLavg0
L01953               !   elf = MIN(MAX(50.*SQRT(qtke(k)), 400.), zwk)
L01954               !ENDIF
L01955
L01956            ELSE
```

```
L01957               ! use version in development for RAP/HRRR 2016
L01958               ! JAYMES-
L01959               ! tau_cloud is an eddy turnover timescale;
L01960               ! see Teixeira and Cheinet (2004), Eq. 1, and
L01961               ! Cheinet and Teixeira (2003), Eq. 7.  The
L01962               ! coefficient 0.5 is tuneable. Expression in
L01963               ! denominator is identical to vsc (a convective
L01964               ! velocity scale), except that elt is relpaced
L01965               ! by pblh, and zero is replaced by 1.0e-4 to
L01966               ! prevent division by zero.
L01967               !tau_cloud = MIN(MAX(0.5*pblh/((gtr*pblh*MAX(vflx,1.0e-4))**p333),50.),150.)
L01968               wstar     = 1.25*(gtr*pblh*MAX(vflx,1.0e-4))**p333
L01969               tau_cloud = MIN(MAX(ctau * wstar/grav, fifty), 200.)
L01970               !minimize influence of surface heat flux on tau far away from the PBLH.
L01971               wt        = p5*TANH((zwk - (pblh2+h1))/h2) + p5
L01972               !tau_cloud = tau_cloud*(1.-wt) + 50.*wt
L01973               tau_cloud = tau_cloud*(one-wt) + MAX(hundred,dzk*p25)*wt
L01974
L01975               elb       = MIN(tau_cloud*SQRT(MIN(qtke(k),40.)), zwk)
L01976               !elf = elb
L01977               elf       = elb !/(1. + (elb/800.))  !bound free-atmos mixing length to < 800 m.
L01978               elb_mf    = elb
L01979            END IF
L01980            elf    = elf/(one + (elf/800.))  !bound free-atmos mixing length to < 800 m.
L01981            elb_mf = MAX(elb_mf, 0.01) !to avoid divide-by-zero below
L01982
L01983            !   **  Length scale in the surface layer  **
L01984            IF ( rmol .GT. 0.0 ) THEN
L01985               els  = karman*zwk/(one+cns*MIN( zwk*rmol, zmax ))
L01986            ELSE
L01987               els  =  karman*zwk*( one - alp4* zwk*rmol)**p2
L01988            END IF
L01989
L01990            !   ** NOW BLEND THE MIXING LENGTH SCALES:
L01991            wt=p5*TANH((zwk - (pblh2+h1))/h2) + p5
L01992
L01993            !try squared-blending
L01994            el(k) = SQRT( els**2/(one + (els**2/elt**2) +(els**2/elb_mf**2)))
L01995            el(k) = el(k)*(one-wt) + elf*wt
L01996         END DO
L01997
L01998      END SELECT
L01999
L02000      ! include scale-awareness. limit el to be < 0.25*dz)
L02001      DO k = kts+1,kte
L02002         el_les = p25*p5*( dz(k)+dz(k-1) )
L02003         el(k)  = el(k)*Psig_bl + (one-Psig_bl)*min(el_les,el(k))
L02004      ENDDO
L02005
L02006   #ifdef HARDCODE_VERTICAL
L02007   # undef kts
L02008   # undef kte
L02009   #endif
L02010
L02011     END SUBROUTINE mym_length
```

公共骨架：① 界面速度 `qkw = sqrt(qke` 界面插值`)`；② **湍流长度 elt** = 从底到 PBL 顶 +过渡层的加权积分 `∫q·z·dz/∫q·dz × alp1`（长度尺度 = 积分尺度 × 系数）；③ **浮力限制长度 elb**（稳定层结 `dtv>0` 时 `elb ∝ q/bv·(1+alp3·sqrt(vsc/(bv·elt)))`，bv=Brunt–Väisälä 频率）；④ **表面层长度 els**（`κz/(1+cns·z/L)` 稳定侧 / `κz·(1−alp4·z/L)^(1/2)` 不稳定侧——**MO 相似律的混合长形态**）；⑤ 按高度 tanh 混合到自由大气。三套 CASE 的差异全在系数与混合方式：

- **CASE 0（原始 MYNN）**：`el = MIN(elb/(elb/elt+elb/els+1), elf)`——**谐波平均**（elt/els/elb 三者取调和权重，哪个小听哪个），系数 cns=2.7、alp1=0.23、alp3=5。
- **CASE 1（非局地 BouLac 混合，默认 `bl_mynn_mixlength=1`）**：最复杂的一套——① **飓风调参** `wt_u1/wt_u2`（风速 >15 m/s 时线性削弱 alp3/alp4/el）；② **灰区感知** `wt_dx`（dx>4 km 向 0.24/0.40 系数过渡）；③ **质量通量混合长** `elb_mf = alp6·qkw_mf/bv`（qkw_mf = 上升流 aw×w 或下沉流 aw×w）——**非局地热泡的混合长直接由通量强度定**，且 `cpblh` 令其只在 PBL 上部起作用；④ 先非平方混合 `el_unstab = els/(1+els1/elt)` 再与 elb/elf 取小；⑤ **自由大气用 BouLac 长度**（`CALL boulac_length` → `elBLavg`，tanh 权重过渡）——`elBLavg(k) = MAX(alp5·elBLavg, elb_mf)` 让 MF 长度在云层也可生效。
- **CASE 2（局地为主）**：自由大气改用**涡翻转时间尺度** `tau_cloud = clamp(1000·w*/g, 30, 150)` 秒，`elf = MIN(tau_cloud·sqrt(qtke), alp6·qkw_mf/bv, zwk)`——Teixeira & Cheinet (2004) 的思想（注释里带推导指引）；平方混合 `el = sqrt(els²/(1+els²/elt²+els²/elb_mf²))`。
- **共同收尾**（L2000–L2004）：`el = el·Psig_bl + (1−Psig_bl)·min(0.25·dz, el)`——**网格自适应：粗网格用连续长度、LES 分辨率限到 1/4 层厚**。
- 常数注释自承 “THE FOLLOWING CONSTANTS ARE IMPORTANT FOR REGULATING THE MIXING LENGTHS”——**改边界层扩散行为的第一动刀点就是这组 alp/cns**。

09a.8 `boulac_length0` / `boulac_length`：气块抬升距离法

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L2026-L2175
L02026    SUBROUTINE boulac_length0(k,kts,kte,zw,dz,qtke,theta,lb1,lb2)
L02027  !
L02028  !    NOTE: This subroutine was taken from the BouLac scheme in WRF-ARW
L02029  !          and modified for integration into the MYNN PBL scheme.
L02030  !          WHILE loops were added to reduce the computational expense.
L02031  !          This subroutine computes the length scales up and down
L02032  !          and then computes the min, average of the up/down
L02033  !          length scales, and also considers the distance to the
L02034  !          surface.
L02035  !
L02036  !      dlu = the distance a parcel can be lifted upwards give a finite
L02037  !            amount of TKE.
L02038  !      dld = the distance a parcel can be displaced downwards given a
L02039  !            finite amount of TKE.
L02040  !      lb1 = the minimum of the length up and length down
L02041  !      lb2 = the average of the length up and length down
L02042  !-----------------------------------------------------------------
L02043      use module_bl_mynnedmf_common, only: gtr,zero,two,p1,p5,kind_phys
L02044
L02045       integer, intent(in) :: k,kts,kte
L02046       real(kind_phys), dimension(kts:kte), intent(in) :: qtke,dz,theta
L02047       real(kind_phys), intent(out) :: lb1,lb2
L02048       real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L02049
L02050       !LOCAL VARS
L02051       integer :: izz, found
L02052       real(kind_phys):: dlu,dld
L02053       real(kind_phys):: dzt, zup, beta, zup_inf, bbb, tl, zdo, zdo_sup, zzz
L02054
L02055
L02056       !---------------------------------
L02057       ! FIND DISTANCE UPWARD
L02058       !---------------------------------
L02059       zup=zero
L02060       dlu=zw(kte+1)-zw(k)-dz(k)*p5
L02061       zzz=zero
L02062       zup_inf=zero
L02063       beta=gtr           !Buoyancy coefficient (g/tref)
L02064
L02065       !print*,"FINDING Dup, k=",k," zw=",zw(k)
L02066
L02067       if (k .lt. kte) then      !cant integrate upwards from highest level
L02068          found = 0
L02069          izz=k
L02070          DO WHILE (found .EQ. 0)
L02071
L02072             if (izz .lt. kte) then
L02073                dzt=dz(izz)                    ! layer depth above
L02074                zup=zup-beta*theta(k)*dzt      ! initial PE the parcel has at k
L02075                !print*,"  ",k,izz,theta(izz),dz(izz)
L02076                zup=zup+beta*(theta(izz+1)+theta(izz))*dzt*p5 ! PE gained by lifting a parcel to izz+1
L02077                zzz=zzz+dzt                    ! depth of layer k to izz+1
L02078                !print*,"  PE=",zup," TKE=",qtke(k)," z=",zw(izz)
L02079                if (qtke(k).lt.zup .and. qtke(k).ge.zup_inf) then
L02080                   bbb=(theta(izz+1)-theta(izz))/dzt
L02081                   if (bbb .ne. zero) then
L02082                      !fractional distance up into the layer where TKE becomes < PE
L02083                      tl=(-beta*(theta(izz)-theta(k)) + &
L02084                        & sqrt( max(zero,(beta*(theta(izz)-theta(k)))**2 + &
L02085                        &       two*bbb*beta*(qtke(k)-zup_inf))))/bbb/beta
L02086                   else
L02087                      if (theta(izz) .ne. theta(k))then
L02088                         tl=(qtke(k)-zup_inf)/(beta*(theta(izz)-theta(k)))
L02089                      else
L02090                         tl=zero
L02091                      endif
L02092                   endif
L02093                   dlu=zzz-dzt+tl
L02094                   !print*,"  FOUND Dup:",dlu," z=",zw(izz)," tl=",tl
L02095                   found =1
L02096                endif
L02097                zup_inf=zup
L02098                izz=izz+1
L02099             ELSE
L02100                found = 1
L02101             ENDIF
L02102
L02103          ENDDO
L02104
L02105       endif
L02106
L02107       !---------------------------------
L02108       ! FIND DISTANCE DOWN
L02109       !---------------------------------
L02110       zdo=zero
L02111       zdo_sup=zero
L02112       dld=zw(k)
L02113       zzz=zero
L02114
L02115       !print*,"FINDING Ddown, k=",k," zwk=",zw(k)
L02116       if (k .gt. kts) then  !cant integrate downwards from lowest level
L02117
L02118          found = 0
L02119          izz=k
L02120          DO WHILE (found .EQ. 0)
L02121
L02122             if (izz .gt. kts) then
L02123                dzt=dz(izz-1)
L02124                zdo=zdo+beta*theta(k)*dzt
L02125                !print*,"  ",k,izz,theta(izz),dz(izz-1)
L02126                zdo=zdo-beta*(theta(izz-1)+theta(izz))*dzt*p5
L02127                zzz=zzz+dzt
L02128                !print*,"  PE=",zdo," TKE=",qtke(k)," z=",zw(izz)
L02129                if (qtke(k).lt.zdo .and. qtke(k).ge.zdo_sup) then
```

```
L02130                  bbb=(theta(izz)-theta(izz-1))/dzt
L02131                  if (bbb .ne. zero) then
L02132                     tl=(beta*(theta(izz)-theta(k))+ &
L02133                       & sqrt( max(zero,(beta*(theta(izz)-theta(k)))**2 + &
L02134                       &       two*bbb*beta*(qtke(k)-zdo_sup))))/bbb/beta
L02135                  else
L02136                     if (theta(izz) .ne. theta(k)) then
L02137                        tl=(qtke(k)-zdo_sup)/(beta*(theta(izz)-theta(k)))
L02138                     else
L02139                        tl=zero
L02140                     endif
L02141                  endif
L02142                  dld=zzz-dzt+tl
L02143                  !print*,"  FOUND Ddown:",dld," z=",zw(izz)," tl=",tl
L02144                  found = 1
L02145               endif
L02146               zdo_sup=zdo
L02147               izz=izz-1
L02148            ELSE
L02149               found = 1
L02150            ENDIF
L02151         ENDDO
L02152
L02153      endif
L02154
L02155      !---------------------------------
L02156      ! GET MINIMUM (OR AVERAGE)
L02157      !---------------------------------
L02158      !The surface layer length scale can exceed z for large z/L,
L02159      !so keep maximum distance down > z.
L02160      dld = min(dld,zw(k+1))!not used in PBL anyway, only free atmos
L02161      lb1 = min(dlu,dld)     !minimum
L02162      !JOE-fight floating point errors
L02163      dlu=MAX(p1,MIN(dlu,1000._kind_phys))
L02164      dld=MAX(p1,MIN(dld,1000._kind_phys))
L02165      lb2 = sqrt(dlu*dld)    !average - biased towards smallest
L02166      !lb2 = 0.5*(dlu+dld)   !average
L02167
L02168      if (k .eq. kte) then
L02169         lb1 = zero
L02170         lb2 = zero
L02171      endif
L02172      !print*,"IN MYNN-BouLac",k,lb1
L02173      !print*,"IN MYNN-BouLac",k,dld,dlu
L02174
L02175    END SUBROUTINE boulac_length0
```

BouLac 方案的混合长（教材 ch14 §14.5 的“双长度 dlu/dld”在此落地）：**把气块动能换算成抬升位能**——从 k 层出发向上/向下逐层积分位能变化 `zup = −β•θ(k)•dzt + β•θ̄•dzt`（β=g/θ₀），直到 TKE < 累积位能；层内精确落点用**求根公式**（`tl = (−β•Δθ + sqrt(β²Δθ²+2•bbb•β•ΔTKE))/(bbb•β)`，bbb 是层内位温梯度）。`lb1 = min(dlu,dld)`（限制长度）、`lb2 = sqrt(dlu•dld)`（几何平均——**偏向较小者**，注释自承）。`length0` 是单层版（返回标量）、`length` 是整列版（WHILE 循环降开销）。

物理对照：这与书 (6.8) 的“气块法判据”同源——TKE 能把气块抬多高，混合长就多大；教材 ch14 §14.5 讲 BouLac 时用“双长度”术语，这里能看到它的积分实现。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L2186-L2333
L02186    SUBROUTINE boulac_length(kts,kte,zw,dz,qtke,theta,lb1,lb2)
L02187  !      dlu = the distance a parcel can be lifted upwards give a finite
L02188  !            amount of TKE.
L02189  !      dld = the distance a parcel can be displaced downwards given a
L02190  !            finite amount of TKE.
L02191  !      lb1 = the minimum of the length up and length down
L02192  !      lb2 = the average of the length up and length down
L02193  !-----------------------------------------------------------------
L02194      use module_bl_mynnedmf_common, only: gtr,zero,one,two,p1,p5,kind_phys
L02195
L02196      integer, intent(in) :: kts,kte
L02197      real(kind_phys), dimension(kts:kte),   intent(in) :: qtke,dz,theta
L02198      real(kind_phys), dimension(kts:kte),   intent(out):: lb1,lb2
L02199      real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L02200
L02201      !LOCAL VARS
L02202      integer :: iz, izz, found
L02203      real(kind_phys), dimension(kts:kte) :: dlu,dld
L02204      real(kind_phys), parameter :: Lmax=1500.  !soft limit
L02205      real(kind_phys):: dzt, zup, beta, zup_inf, bbb, tl, zdo, zdo_sup, zzz
L02206
L02207      !print*,"IN MYNN-BouLac",kts, kte
L02208
L02209      do iz=kts,kte
L02210
L02211         !---------------------------------
L02212         ! FIND DISTANCE UPWARD
L02213         !---------------------------------
L02214         zup    =zero
L02215         dlu(iz)=zw(kte+1)-zw(iz)-dz(iz)*p5
L02216         zzz    =zero
L02217         zup_inf=zero
L02218         beta   =gtr           !Buoyancy coefficient (g/tref)
L02219
L02220         !print*,"FINDING Dup, k=",iz," zw=",zw(iz)
L02221
L02222         if (iz .lt. kte) then      !cant integrate upwards from highest level
L02223
L02224           found = 0
L02225           izz=iz
L02226           DO WHILE (found .EQ. 0)
L02227
L02228             if (izz .lt. kte) then
```

```
L02229                dzt=dz(izz)                     ! Layer depth above
L02230                zup=zup-beta*theta(iz)*dzt      ! initial PE the parcel has at iz
L02231                !print*,"  ",iz,izz,theta(izz),dz(izz)
L02232                zup=zup+beta*(theta(izz+1)+theta(izz))*dzt*p5 ! PE gained by lifting a parcel to izz+1
L02233                zzz=zzz+dzt                     ! depth of layer iz to izz+1
L02234                !print*,"  PE=",zup," TKE=",qtke(iz)," z=",zw(izz)
L02235                if (qtke(iz).lt.zup .and. qtke(iz).ge.zup_inf) then
L02236                   bbb=(theta(izz+1)-theta(izz))/dzt
L02237                   if (bbb .ne. 0.) then
L02238                      !fractional distance up into the layer where TKE becomes < PE
L02239                      tl=(-beta*(theta(izz)-theta(iz)) + &
L02240                        & sqrt( max(zero,(beta*(theta(izz)-theta(iz)))**2 + &
L02241                        &       two*bbb*beta*(qtke(iz)-zup_inf))))/bbb/beta
L02242                   else
L02243                      if (theta(izz) .ne. theta(iz))then
L02244                         tl=(qtke(iz)-zup_inf)/(beta*(theta(izz)-theta(iz)))
L02245                      else
L02246                         tl=zero
L02247                      endif
L02248                   endif
L02249                   dlu(iz)=zzz-dzt+tl
L02250                   !print*,"  FOUND Dup:",dlu(iz)," z=",zw(izz)," tl=",tl
L02251                   found =1
L02252                endif
L02253                zup_inf=zup
L02254                izz=izz+1
L02255               ELSE
L02256                 found = 1
L02257               ENDIF
L02258
L02259             ENDDO
L02260
L02261          endif
L02262
L02263          !---------------------------------
L02264          ! FIND DISTANCE DOWN
L02265          !---------------------------------
L02266          zdo    =zero
L02267          zdo_sup=zero
L02268          dld(iz)=zw(iz)
L02269          zzz    =zero
L02270
L02271          !print*,"FINDING Ddown, k=",iz," zwk=",zw(iz)
L02272          if (iz .gt. kts) then  !cant integrate downwards from lowest level
L02273
L02274            found = 0
L02275            izz=iz
L02276            DO WHILE (found .EQ. 0)
L02277
L02278              if (izz .gt. kts) then
L02279                dzt=dz(izz-1)
L02280                zdo=zdo+beta*theta(iz)*dzt
L02281                !print*,"  ",iz,izz,theta(izz),dz(izz-1)
L02282                zdo=zdo-beta*(theta(izz-1)+theta(izz))*dzt*p5
L02283                zzz=zzz+dzt
L02284                !print*,"  PE=",zdo," TKE=",qtke(iz)," z=",zw(izz)
L02285                if (qtke(iz).lt.zdo .and. qtke(iz).ge.zdo_sup) then
L02286                   bbb=(theta(izz)-theta(izz-1))/dzt
L02287                   if (bbb .ne. 0.) then
L02288                      tl=(beta*(theta(izz)-theta(iz))+ &
L02289                        & sqrt( max(zero,(beta*(theta(izz)-theta(iz)))**2 + &
L02290                        &       two*bbb*beta*(qtke(iz)-zdo_sup))))/bbb/beta
L02291                   else
L02292                      if (theta(izz) .ne. theta(iz)) then
L02293                         tl=(qtke(iz)-zdo_sup)/(beta*(theta(izz)-theta(iz)))
L02294                      else
L02295                         tl=zero
L02296                      endif
L02297                   endif
L02298                   dld(iz)=zzz-dzt+tl
L02299                   !print*,"  FOUND Ddown:",dld(iz)," z=",zw(izz)," tl=",tl
L02300                   found = 1
L02301                endif
L02302                zdo_sup=zdo
L02303                izz=izz-1
L02304              ELSE
L02305                found = 1
L02306              ENDIF
L02307            ENDDO
L02308
L02309          endif
L02310
L02311          !---------------------------------
L02312          ! GET MINIMUM (OR AVERAGE)
L02313          !---------------------------------
L02314          !The length scale down should not exceed z.
L02315          dld(iz) = min(dld(iz),zw(iz+1))
L02316          !apply soft upper limit (only impacts very large lb; lb=100 by 5%, lb=500 by 20%)
L02317          dlu(iz)=MAX(p1, dlu(iz)/(one + (dlu(iz)/Lmax)) )
L02318          dld(iz)=MAX(p1, dld(iz)/(one + (dld(iz)/Lmax)) )
L02319
L02320          !minimum of up/down length scales
L02321          lb1(iz) = min(dlu(iz),dld(iz))
L02322          !average - biased towards smallest
L02323          lb2(iz) = sqrt(dlu(iz)*dld(iz))
L02324
L02325          !print*,"IN MYNN-BouLac",kts, kte,lb1(iz)
L02326          !print*,"IN MYNN-BouLac",iz,dld(iz),dlu(iz)
L02327
L02328        ENDDO
L02329
L02330        lb1(kte) = lb1(kte-1)
L02331        lb2(kte) = lb2(kte-1)
L02332
L02333    END SUBROUTINE boulac_length
```

整列版的两个工程细节：**软上限 `dlu/(1+dlu/1500)`**（1500 m 处削 5%、5000 m 削 25%——比硬

clamp 平滑）；顶/底边界取邻层值（`lb1(kte) = lb1(kte-1)`）。

09a.9 `mym_turbulence`：从 L2 到 L3 的扩散系数

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L2384-L2579
L02384    SUBROUTINE  mym_turbulence (                                       &
L02385      &            kts,kte,                                            &
L02386      &            xland,closure,ckmod,taue,                           &
L02387      &            dz, dx, zw, p, exner,                               &
L02388      &            u, v, thl, thv, thlv, ql, qw,                       &
L02389      &            qke, tsq, qsq, cov,                                 &
L02390      &            vt, vq,                                             &
L02391      &            rmol, rmolh, flt, fltv, flq,                        &
L02392      &            pblh,theta,                                         &
L02393      &            sh, sm,                                             &
L02394      &            El,                                                 &
L02395      &            Dfm, Dfh, Dfq, Tcd, Qcd, Pdk, Pdt, Pdq, Pdc, &
L02396      &         qWT1,qSHEAR1,qBUOY1,qDISS1,                   &
L02397      &            tke_budget,                                         &
L02398      &            Psig_bl,Psig_shcu,cldfra_bl1,                       &
L02399      &            bl_mynn_mixlength,                                  &
L02400      &            bl_mynn_ess,                                        &
L02401      &            edmf_w1,edmf_a1,                                    &
L02402      &            edmf_w_dd1,edmf_a_dd1,                              &
L02403      &            TKEprod_dn,TKEprod_up,                              &
L02404      &            spp_pbl,pattern_spp_pbl1                            )
L02405
L02406    !-------------------------------------------------------------------
L02407        use module_bl_mynnedmf_common, only: gtr,tv0,                   &
L02408             a1,a2,b1,b2,c1,e1c,e2c,e3c,e4c,e5c,cc3,                    &
L02409             zero,one,two,three,four,seven,nine,p1,p2,p5,               &
L02410             kind_phys
L02411
L02412        integer, intent(in)   :: kts,kte
L02413
L02414    #ifdef HARDCODE_VERTICAL
L02415    # define kts 1
L02416    # define kte HARDCODE_VERTICAL
L02417    #endif
L02418
L02419        integer, intent(in)              :: bl_mynn_mixlength
L02420        integer, intent(in)              :: tke_budget
L02421        integer, intent(in)              :: bl_mynn_ess
L02422        real(kind_phys), intent(in)      :: closure,ckmod,taue
L02423        real(kind_phys), dimension(kts:kte),   intent(in) :: dz
L02424        real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L02425        real(kind_phys), intent(in)      :: rmol,rmolh,flt,fltv,flq,          &
L02426             &Psig_bl,Psig_shcu,xland,dx,pblh
L02427        real(kind_phys), dimension(kts:kte), intent(in) :: u,v,thl,thv,       &
L02428             &thlv,qw,ql,vt,vq,qke,tsq,qsq,cov,cldfra_bl1,edmf_w1,edmf_a1,     &
L02429             &edmf_w_dd1,edmf_a_dd1,TKEprod_dn,TKEprod_up,p,exner
L02430
L02431        real(kind_phys), dimension(kts:kte), intent(out) :: dfm,dfh,dfq,      &
L02432             &pdk,pdt,pdq,pdc,tcd,qcd,el
L02433
L02434        real(kind_phys), dimension(kts:kte), intent(inout), optional ::       &
L02435             qWT1,qSHEAR1,qBUOY1,qDISS1
L02436        real(kind_phys):: q3sq_old,qWTP_old,qWTP_new
L02437        real(kind_phys):: dudz,dvdz,dTdz,upwp,vpwp,Tpwp
L02438
L02439        real(kind_phys), dimension(kts:kte) :: qkw,qtw,dtl,dqw,dtv,gm,gh,sm,sh,qpe
L02440
L02441        integer :: k
L02442    !    real(kind_phys):: cc2,cc3,e1c,e2c,e3c,e4c,e5c
L02443        real(kind_phys):: e6c,dzk,afk,abk,vtt,vqq,                          &
L02444             &cw25,clow,cupp,gamt,gamq,smd,gamv,elq,elh,elqt
L02445
L02446        real(kind_phys):: cldavg
L02447        real(kind_phys), dimension(kts:kte), intent(in) :: theta
L02448
L02449        real(kind_phys)::  a2fac, duz, ri !JOE-Canuto/Kitamura mod
L02450
L02451        real(kind_phys):: auh,aum,adh,adm,aeh,aem,Req,Rsl,Rsl2,              &
L02452              gmelq,sm20,sh20,sm25max,sh25max,sm25min,sh25min,               &
L02453              sm_pbl,sh_pbl,pblh2,wt,slht,mfmax,cpblh
L02454
L02455        DOUBLE PRECISION  q2sq, t2sq, r2sq, c2sq, elsq, gmel, ghel
L02456        DOUBLE PRECISION  q3sq, t3sq, r3sq, c3sq, dlsq, qdiv
L02457        DOUBLE PRECISION  e1, e2, e3, e4, enum, eden, wden
L02458
L02459    !   Stochastic
L02460        integer,         intent(in)                  :: spp_pbl
L02461        real(kind_phys), dimension(kts:kte)          :: pattern_spp_pbl1
L02462        real(kind_phys):: shb, Prlim
L02463        real(kind_phys), parameter :: Prlimit_fre = 6.0 !Pr limit in free troposphere
L02464        real(kind_phys), parameter :: Prlimit_sfc = 4.0 !Pr limit at the surface
L02465
L02466    !
L02467    !    tv0 = 0.61*tref
L02468    !    gtr = 9.81/tref
L02469    !
L02470    !    cc2 =  1.0-c2
L02471    !    cc3 =  1.0-c3
L02472    !    e1c =  3.0*a2*b2*cc3
L02473    !    e2c =  9.0*a1*a2*cc2
L02474    !    e3c =  9.0*a2*a2*cc2*( 1.0-c5 )
L02475    !    e4c = 12.0*a1*a2*cc2
L02476    !    e5c =  6.0*a1*a1
L02477
L02478        qkw  = zero
L02479        qtw  = zero
L02480        pblh2= max(pblh, 200._kind_phys)
L02481        if (closure .lt. 2.7) then
L02482           qpe = zero
```

```
L02483      else
L02484         do k = kts,kte
L02485            !Calculate qpe (twice tpe) as in Machulskaya and Mironov (2020, BLM), but with magnitudes
L02486            !tapered at low-levels to limit high 10-m wind speed biases:
L02487            cpblh  = min((zw(k) + p1*pblh2)/(0.8_kind_phys*pblh2), one)
L02488            !qpe(k) = two * tsq(k) * gtr**2 * (p5*(el(k+1)+el(k))/10.)**2/(p5*max(0.01,qke(k)))
L02489            qpe(k) = min(three, max(zero, cpblh * two * tsq(k) * gtr**2 * taue**2))
L02490         enddo
L02491      endif
L02492
L02493      CALL mym_level2 (kts,kte,                  &
L02494      &            ckmod, bl_mynn_ess,           &
L02495      &            zw, dz, xland, pblh,          &
L02496      &            u, v, thl, thv, thlv,         &
L02497      &            theta, p, exner,              &
L02498      &            qke, cldfra_bl1,              &
L02499      &            edmf_a1, edmf_w1,             &
L02500      &            qw, ql, vt, vq,               &
L02501      &            dtl, dqw, dtv, gm, gh, sm, sh  )
L02502   !
L02503      CALL mym_length (                          &
L02504      &            kts,kte,xland,                &
L02505      &            dz, dx, zw,                   &
L02506      &            rmol, rmolh, flt, fltv, flq,  &
L02507      &            vt, vq,                       &
L02508      &            u, v, qke, qpe,               &
L02509      &            dtv,                          &
L02510      &            el,                           &
L02511      &            pblh, theta,                  &
L02512      &            qkw, qtw,                     &
L02513      &            Psig_bl, cldfra_bl1,          &
L02514      &            bl_mynn_mixlength,            &
L02515      &            edmf_w1,edmf_a1,              &
L02516      &            edmf_w_dd1,edmf_a_dd1          )
L02517   !
L02518
L02519      DO k = kts+1,kte
L02520         dzk  = p5 *( dz(k)+dz(k-1) )
L02521         afk  = dz(k)/( dz(k)+dz(k-1) )
L02522         abk  = one -afk
L02523         elsq = el(k)**2
L02524         q3sq = qkw(k)**2
L02525         q2sq = b1*elsq*( sm(k)*gm(k)+sh(k)*gh(k) )
L02526
L02527         sh20 = MAX(sh(k), 1e-5_kind_phys)
L02528         sm20 = MAX(sm(k), 1e-5_kind_phys)
L02529         sh(k)= MAX(sh(k), 1e-5_kind_phys)
L02530
L02531         !Canuto/Kitamura mod
L02532         duz = ( u(k)-u(k-1) )**2 +( v(k)-v(k-1) )**2
L02533         duz =   duz                      /dzk**2
L02534         !   **  Gradient Richardson number  **
L02535         ri = -gh(k)/MAX( duz, 1.0e-10_kind_phys )
L02536         !a2fac is needed for the Canuto-Kitamura modification (CKmod)
L02537         a2fac = one/(one + ckmod*max(ri,zero))
L02538
L02539         !The form of Zilitinkevich et al. (2006) but modified
L02540         !half-way towards Esau and Grachev (2007, Wind Eng)
L02541         !Prlim = MIN(0.76_kind_phys + 3.0_kind_phys*MAX(ri,zero), Prlimit_fre)
L02542         !Prlim = MIN(0.76_kind_phys + 4.0_kind_phys*MAX(ri,zero), Prlimit_fre)
L02543
L02544         !TZC - Kondo Correction
L02545         if (ri >= one) then
L02546            ! Kh/Km = 1/(7*Ri)
L02547            Prlim = min(seven*ri, Prlimit_fre)
L02548         elseif (ri >= 0.01 .and. ri <= one) then
L02549            ! Kh/Km(i,k) = 1/(6.873*Ri + 1/(6.873*Ri))
L02550            Prlim = min(6.873_kind_phys*ri + one/(6.873_kind_phys*ri), Prlimit_fre)
L02551         else
L02552            ! no Pr limit required?
L02553            Prlim = Prlimit_fre
L02554         end if
L02555         wt    = min(one, zw(k)/max(pblh, 200._kind_phys))  !0 at sfc, 1 at/above pblh
L02556         Prlim = wt*Prlim + (one-wt)*min(Prlimit_sfc, Prlim)
L02557   !
L02558   !  Modified: Dec/22/2005, from here, (dlsq -> elsq)
L02559         gmel = gm(k)*elsq
L02560         ghel = gh(k)*elsq
L02561   !  Modified: Dec/22/2005, up to here
L02562
L02563         ! Level 2.0 debug prints
L02564         IF ( debug_code ) THEN
L02565           IF (sh(k)<0.0 .OR. sm(k)<0.0) THEN
L02566             print*,"MYNN; mym_turbulence 2.0; sh=",sh(k)," k=",k
L02567             print*," gm=",gm(k)," gh=",gh(k)," sm=",sm(k)
L02568             print*," q2sq=",q2sq," q3sq=",q3sq," q3/q2=",q3sq/q2sq
L02569             print*," qke=",qke(k)," el=",el(k)," ri=",ri
L02570             print*," PBLH=",pblh," u=",u(k)," v=",v(k)
L02571           ENDIF
L02572         ENDIF
L02573
L02574   !     **  Since qkw is set to more than 0.0, q3sq > 0.0.  **
L02575
L02576   !     new stability criteria in level 2.5 (as well as level 3) - little/no impact
L02577   !     **  Limitation on q, instead of L/q  **
L02578         dlsq =  elsq
L02579         IF ( q3sq/dlsq .LT. -gh(k) ) q3sq = -dlsq*gh(k)
```

- **qpe**（closure≥2.7，TTE 闭合）：`qpe = min(3, max(0, cpblh • 2 • tsq • (g/θ)² • taue²))`——温度方差 × 浮力 ²× 时间尺度 ²=潜在能量（Machulskaya & Mironov 2020），`cpblh` 低层衰减控制 10 m 风速偏差。
- **重算 L2**：`mym_level2`/`mym_length` 每步重调（qke 已被上一步更新）。
- **Pr 数限制**（Kondo 修正分段）：Ri≥1 时 `Prlim = min(7Ri, 6)`、0.01≤Ri<1 时 `Prlim = min(6.873Ri + 1/(6.873Ri), 6)`（**先增后减的中段凸形**）、再按高度把表面层限制为 4——

Kh/Km 比的物理上限（稳定层结热量扩散比动量衰减更快）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L2581-L2938
L02581          IF ( q3sq .LT. q2sq ) THEN
L02582             !Apply Helfand & Labraga mod
L02583             qdiv = SQRT( q3sq/q2sq )   !HL89: (1-alfa)
L02584  !
L02585             !Use level 2.5 stability functions
L02586             !e1   = q3sq - e1c*ghel*a2fac
L02587             !e2   = q3sq - e2c*ghel*a2fac
L02588             !e3   = e1   + e3c*ghel*a2fac**2
L02589             !e4   = e1   - e4c*ghel*a2fac
L02590             !eden = e2*e4 + e3*e5c*gmel
L02591             !eden = MAX( eden, 1.0d-20 )
L02592             !sm(k) = q3sq*a1*( e3-3.0*c1*e4       )/eden
L02593             !!JOE-Canuto/Kitamura mod
L02594             !!sh(k) = q3sq*a2*( e2+3.0*c1*e5c*gmel )/eden
L02595             !sh(k) = q3sq*(a2*a2fac)*( e2+3.0*c1*e5c*gmel )/eden
L02596             !sm(k) = sm(k) * qdiv
L02597
L02598             !Use level 2.0 functions as in original MYNN
L02599             sh(k) = sh(k) * qdiv
L02600             sm(k) = sm(k) * qdiv
L02601
L02602             !Recalculate terms for later use
L02603             !JOE-Canuto/Kitamura mod
L02604             !e1   = q3sq - e1c*ghel * qdiv**2
L02605             !e2   = q3sq - e2c*ghel * qdiv**2
L02606             !e3   = e1   + e3c*ghel * qdiv**2
L02607             !e4   = e1   - e4c*ghel * qdiv**2
L02608             e1   = q3sq - e1c*ghel*a2fac * qdiv**2
L02609             e2   = q3sq - e2c*ghel*a2fac * qdiv**2
L02610             e3   = e1   + e3c*ghel*a2fac**2 * qdiv**2
L02611             e4   = e1   - e4c*ghel*a2fac * qdiv**2
L02612             eden = e2*e4 + e3*e5c*gmel * qdiv**2
L02613             eden = MAX( eden, 1.0d-20 )
L02614             !!JOE-Canuto/Kitamura mod
L02615             !!sh(k) = q3sq*a2*( e2+3.0*c1*e5c*gmel )/eden
L02616             !sh(k) = q3sq*(a2*a2fac)*( e2+3.0*c1*e5c*gmel )/eden
L02617          ELSE
L02618             !JOE-Canuto/Kitamura mod
L02619             !e1   = q3sq - e1c*ghel
L02620             !e2   = q3sq - e2c*ghel
L02621             !e3   = e1   + e3c*ghel
L02622             !e4   = e1   - e4c*ghel
L02623             e1   = q3sq - e1c*ghel*a2fac
L02624             e2   = q3sq - e2c*ghel*a2fac
L02625             e3   = e1   + e3c*ghel*a2fac**2
L02626             e4   = e1   - e4c*ghel*a2fac
L02627             eden = e2*e4 + e3*e5c*gmel
L02628             eden = MAX( eden, 1.0d-20 )
L02629
L02630             qdiv = one
L02631             !Use level 2.5 stability functions
L02632             sm(k) = q3sq*a1*( e3-three*c1*e4       )/eden
L02633             !  sm_pbl = q3sq*a1*( e3-3.0*c1*e4       )/eden
L02634             !!JOE-Canuto/Kitamura mod
L02635             !!sh(k) = q3sq*a2*( e2+3.0*c1*e5c*gmel )/eden
L02636             sh(k) = q3sq*(a2*a2fac)*( e2+three*c1*e5c*gmel )/eden
L02637          END IF !end Helfand & Labraga check
L02638
L02639          !Impose broad limits on Sh and Sm:
L02640          gmelq    = max(real(gmel/q3sq, kind_phys), 1e-8_kind_phys)
L02641          sm25max  = four  !MIN(sm20*3.0, SQRT(.1936/gmelq))
L02642          sh25max  = four  !MIN(sh20*3.0, 0.76*b2)
L02643          sm25min  = zero !MAX(sm20*0.1, 1e-6)
L02644          sh25min  = zero !MAX(sh20*0.1, 1e-6)
L02645
L02646          !JOE: Level 2.5 debug prints
L02647          ! HL88 , lev2.5 criteria from eqs. 3.17, 3.19, & 3.20
L02648          IF ( debug_code ) THEN
L02649            IF ((sh(k)<sh25min .OR. sm(k)<sm25min .OR. &
L02650                 sh(k)>sh25max .OR. sm(k)>sm25max) ) THEN
L02651              print*,"In mym_turbulence 2.5: k=",k
L02652              print*," sm=",sm(k)," sh=",sh(k)
L02653              print*," ri=",ri," Pr=",sm(k)/max(sh(k),1e-8_kind_phys)
L02654              print*," gm=",gm(k)," gh=",gh(k)
L02655              print*," q2sq=",q2sq," q3sq=",q3sq, q3sq/q2sq
L02656              print*," qke=",qke(k)," el=",el(k)
L02657              print*," PBLH=",pblh," u=",u(k)," v=",v(k)
L02658              print*," SMnum=",q3sq*a1*( e3-three*c1*e4)," SMdenom=",eden
L02659              print*," SHnum=",q3sq*(a2*a2fac)*( e2+three*c1*e5c*gmel ),&
L02660                   " SHdenom=",eden
L02661            ENDIF
L02662          ENDIF
L02663
L02664          !Enforce constraints for level 2.5 functions
L02665          IF ( sh(k) > sh25max ) sh(k) = sh25max
L02666          IF ( sh(k) < sh25min ) sh(k) = sh25min
L02667          !IF ( sm(k) > sm25max ) sm(k) = sm25max
L02668          !IF ( sm(k) < sm25min ) sm(k) = sm25min
L02669
L02670          shb   = max(sh(k), 0.004_kind_phys)
L02671          sm(k) = min(sm(k), Prlim*shb)
L02672
L02673  !   **  Level 3 : start  **
L02674          IF ( closure .GE. 3.0 ) THEN
L02675             t2sq = qdiv*b2*elsq*sh(k)*dtl(k)**2
L02676             r2sq = qdiv*b2*elsq*sh(k)*dqw(k)**2
L02677             c2sq = qdiv*b2*elsq*sh(k)*dtl(k)*dqw(k)
L02678             t3sq = MAX( tsq(k)*abk+tsq(k-1)*afk, zero )
L02679             r3sq = MAX( qsq(k)*abk+qsq(k-1)*afk, zero )
L02680             c3sq =      cov(k)*abk+cov(k-1)*afk
L02681
L02682  !  Modified: Dec/22/2005, from here
L02683             c3sq = SIGN( MIN( ABS(c3sq), SQRT(t3sq*r3sq) ), c3sq )
L02684  !
L02685             vtt  = one +vt(k)*abk +vt(k-1)*afk
```

```
L02686            vqq  = tv0 +vq(k)*abk +vq(k-1)*afk
L02687
L02688            t2sq = vtt*t2sq +vqq*c2sq
L02689            r2sq = vtt*c2sq +vqq*r2sq
L02690            c2sq = MAX( vtt*t2sq+vqq*r2sq, 0.0d0 )
L02691            t3sq = vtt*t3sq +vqq*c3sq
L02692            r3sq = vtt*c3sq +vqq*r3sq
L02693            c3sq = MAX( vtt*t3sq+vqq*r3sq, 0.0d0 )
L02694  !
L02695            cw25 = e1*( e2 + three*c1*e5c*gmel*qdiv**2 )/( three*eden )
L02696  !
L02697  !     **  Limitation on q, instead of L/q  **
L02698            dlsq =  elsq
L02699            IF ( q3sq/dlsq .LT. -gh(k) ) q3sq = -dlsq*gh(k)
L02700  !
L02701  !     **  Limitation on c3sq (0.12 =< cw =< 0.76) **
L02702            ! Use Janjic's (2001; p 13-17) methodology (eqs 4.11-414 and 5.7-5.10)
L02703            ! to calculate an exact limit for c3sq:
L02704            auh = 27._kind_phys*a1*((a2*a2fac)**2)*b2*(gtr)**2
L02705            aum = 54._kind_phys*(a1**2)*(a2*a2fac)*b2*c1*(gtr)
L02706            adh = nine*a1*((a2*a2fac)**2)*(12._kind_phys*a1 + three*b2)*(gtr)**2
L02707            adm = 18._kind_phys*(a1**2)*(a2*a2fac)*(b2 - three*(a2*a2fac))*(gtr)
L02708
L02709            aeh = (nine*a1*((a2*a2fac)**2)*b1 +nine*a1*((a2*a2fac)**2)*            &
L02710                  (12._kind_phys*a1 + three*b2))*(gtr)
L02711            aem = three*a1*(a2*a2fac)*b1*(three*(a2*a2fac) + three*b2*c1 +         &
L02712                  (18._kind_phys*a1*c1 - b2)) +                                    &
L02713                  (18._kind_phys)*(a1**2)*(a2*a2fac)*(b2 - three*(a2*a2fac))
L02714
L02715            Req = -aeh/aem
L02716            Rsl = (auh + aum*Req)/(three*adh + three*adm*Req)
L02717            !For now, use default values, since tests showed little/no sensitivity
L02718            Rsl = 0.12_kind_phys   !lower limit
L02719            Rsl2= one - two*Rsl    !upper limit
L02720            !IF (k==2)print*,"Dynamic limit RSL=",Rsl
L02721            !IF (Rsl < 0.10 .OR. Rsl > 0.18) THEN
L02722            !   print*,'--- ERROR: MYNN: Dynamic Cw '// &
L02723            !        'limit exceeds reasonable limits'
L02724            !   print*," MYNN: Dynamic Cw limit needs attention=",Rsl
L02725            !ENDIF
L02726
L02727            !JOE-Canuto/Kitamura mod
L02728            !e2   = q3sq - e2c*ghel * qdiv**2
L02729            !e3   = q3sq + e3c*ghel * qdiv**2
L02730            !e4   = q3sq - e4c*ghel * qdiv**2
L02731            e2   = q3sq - e2c*ghel*a2fac * qdiv**2
L02732            e3   = q3sq + e3c*ghel*a2fac**2 * qdiv**2
L02733            e4   = q3sq - e4c*ghel*a2fac * qdiv**2
L02734            eden = e2*e4  + e3 *e5c*gmel * qdiv**2
L02735
L02736            !JOE-Canuto/Kitamura mod
L02737            !wden = cc3*gtr**2 * dlsq**2/elsq * qdiv**2 &
L02738            !     &        *( e2*e4c - e3c*e5c*gmel * qdiv**2 )
L02739            wden = cc3*gtr**2 * dlsq**2/elsq * qdiv**2 &
L02740                 &        *( e2*e4c*a2fac - e3c*e5c*gmel*a2fac**2 * qdiv**2 )
L02741
L02742            IF ( wden .NE. zero ) THEN
L02743               !set dynamic limits
L02744               clow = q3sq*( 0.12-cw25 )*eden/wden
L02745               cupp = q3sq*( 0.76-cw25 )*eden/wden
L02746               !clow = q3sq*( Rsl -cw25 )*eden/wden
L02747               !cupp = q3sq*( Rsl2-cw25 )*eden/wden
L02748  !
L02749               IF ( wden .GT. zero ) THEN
L02750                  c3sq  = MIN( MAX( c3sq, c2sq+clow ), c2sq+cupp )
L02751               ELSE
L02752                  c3sq  = MAX( MIN( c3sq, c2sq+clow ), c2sq+cupp )
L02753               END IF
L02754            END IF
L02755  !
L02756            e1   = e2 + e5c*gmel * qdiv**2
L02757            eden = MAX( eden, 1.0d-20 )
L02758  !  Modified: Dec/22/2005, up to here
L02759
L02760            !JOE-Canuto/Kitamura mod
L02761            !e6c  = 3.0*a2*cc3*gtr * dlsq/elsq
L02762            e6c  = 3.0*(a2*a2fac)*cc3*gtr * dlsq/elsq
L02763
L02764            !============================
L02765            !     **  for Gamma_theta  **
L02766            !!          enum = qdiv*e6c*( t3sq-t2sq )
L02767            IF ( t2sq .GE. zero ) THEN
L02768               enum = MAX( qdiv*e6c*( t3sq-t2sq ), 0.0d0 )
L02769            ELSE
L02770               enum = MIN( qdiv*e6c*( t3sq-t2sq ), 0.0d0 )
L02771            ENDIF
L02772            gamt =-e1  *enum    /eden
L02773
L02774            !============================
L02775            !     **  for Gamma_q  **
L02776            !!          enum = qdiv*e6c*( r3sq-r2sq )
L02777            IF ( r2sq .GE. zero ) THEN
L02778               enum = MAX( qdiv*e6c*( r3sq-r2sq ), 0.0d0 )
L02779            ELSE
L02780               enum = MIN( qdiv*e6c*( r3sq-r2sq ), 0.0d0 )
L02781            ENDIF
L02782            gamq =-e1  *enum    /eden
L02783
L02784            !============================
L02785            !     **  for Sm' and Sh'd(Theta_V)/dz  **
L02786            !!          enum = qdiv*e6c*( c3sq-c2sq )
L02787            enum = MAX( qdiv*e6c*( c3sq-c2sq ), 0.0d0)
L02788
L02789            !JOE-Canuto/Kitamura mod
L02790            !smd  = dlsq*enum*gtr/eden * qdiv**2 * (e3c+e4c)*a1/a2
L02791            smd  = dlsq*enum*gtr/eden * qdiv**2 * (e3c*a2fac**2 + &
L02792                 & e4c*a2fac)*a1/(a2*a2fac)
L02793
L02794            gamv = e1  *enum*gtr/eden
```

```
L02795            sm(k) = sm(k) +smd
L02796
L02797            !============================
L02798            !     **  For elh (see below), qdiv at Level 3 is reset to 1.0.  **
L02799            qdiv = one
L02800
L02801            ! Level 3 debug prints
L02802            IF ( debug_code ) THEN
L02803              IF (sh(k)<-0.3 .OR. sm(k)<-0.3 .OR. &
L02804                qke(k) < -0.1 .or. ABS(smd) .gt. 2.0) THEN
L02805                print*," MYNN; mym_turbulence3.0; sh=",sh(k)," k=",k
L02806                print*," gm=",gm(k)," gh=",gh(k)," sm=",sm(k)
L02807                print*," q2sq=",q2sq," q3sq=",q3sq," q3/q2=",q3sq/q2sq
L02808                print*," qke=",qke(k)," el=",el(k)," ri=",ri
L02809                print*," PBLH=",pblh," u=",u(k)," v=",v(k)
L02810              ENDIF
L02811            ENDIF
L02812
L02813  !   **  Level 3 : end  **
L02814
L02815          ELSE
L02816  !     **  At Level 2.5, qdiv is not reset.  **
L02817            gamt = zero
L02818            gamq = zero
L02819            gamv = zero
L02820          END IF
L02821  !
L02822  !       Add min background stability function (diffusivity) within model levels
L02823  !       with active plumes and clouds.
L02824          cldavg = p5*(cldfra_bl1(k-1) + cldfra_bl1(k))
L02825          mfmax  = max(p5*(edmf_a1(k-1)+edmf_a1(k))*p5*(edmf_w1(k-1)+edmf_w1(k)), abs(edmf_a_dd1(k)*edmf_w_dd1(k)))
L02826          ! impose minimum for mass-flux columns
L02827          sm(k) = max(sm(k), 0.04_kind_phys*min(10._kind_phys*mfmax,one) )
L02828          sh(k) = max(sh(k), 0.04_kind_phys*min(10._kind_phys*mfmax,one) )
L02829          ! impose minimum for clouds
L02830          sm(k) = max(sm(k), 0.04_kind_phys*min(cldavg, p5) )
L02831          sh(k) = max(sh(k), 0.04_kind_phys*min(cldavg, p5) )
L02832          ! impose minimum for tkeprod_down
L02833          if (tkeprod_dn(k)>1e-6) sm(k) = max(sm(k), 0.01_kind_phys)
L02834          if (tkeprod_dn(k)>1e-6) sh(k) = max(sh(k), 0.01_kind_phys)
L02835          ! impose minimum sm for tte configurations. this may overide Pr limitations above.
L02836          if (closure .ge. 2.7 .and. closure .lt. 3.0) then
L02837            sm(k) = max(sm(k),min(p2, max(zero,three*qpe(k))))
L02838          endif
L02839          !
L02840          elq   = el(k)*qkw(k)
L02841          elqt  = el(k)*qtw(k)  !for km with total turbulent energy; elq = elqt when closure < 2.7
L02842          elh   = elq*qdiv
L02843
L02844          ! Production of TKE (pdk), T-variance (pdt),
L02845          ! q-variance (pdq), and covariance (pdc)
L02846          pdk(k) = elq *(sm(k)*gm(k)                    &
L02847               & +       sh(k)*gh(k)+gamv ) +           &
L02848               &   p5*TKEprod_dn(k)         +           & ! xmchen
L02849               &   p5*TKEprod_up(k)
L02850          pdt(k) = elh *( sh(k)*dtl(k)+gamt )*dtl(k)
L02851          pdq(k) = elh *( sh(k)*dqw(k)+gamq )*dqw(k)
L02852          pdc(k) = elh *( sh(k)*dtl(k)+gamt )*dqw(k)*p5 &
L02853               & + elh *( sh(k)*dqw(k)+gamq )*dtl(k)*p5
L02854
L02855          ! Contergradient terms
L02856          tcd(k) = elq*gamt
L02857          qcd(k) = elq*gamq
L02858
L02859          ! Eddy Diffusivity/Viscosity divided by dz
L02860          dfm(k) = elqt*sm(k) / dzk
L02861          dfh(k) = elq *sh(k) / dzk
L02862  !  Modified: Dec/22/2005, from here
L02863  !   **  In sub.mym_predict, dfq for the TKE and scalar variance **
L02864  !   **  are set to 3.0*dfm and 1.0*dfm, respectively. (Sqfac)   **
L02865          dfq(k) =     dfm(k)
L02866  !  Modified: Dec/22/2005, up to here
L02867
L02868      IF (tke_budget .eq. 1) THEN
L02869          !TKE BUDGET
L02870  !       dudz = ( u(k)-u(k-1) )/dzk
L02871  !       dvdz = ( v(k)-v(k-1) )/dzk
L02872  !       dTdz = ( thl(k)-thl(k-1) )/dzk
L02873
L02874  !       upwp = -elq*sm(k)*dudz
L02875  !       vpwp = -elq*sm(k)*dvdz
L02876  !       Tpwp = -elq*sh(k)*dTdz
L02877  !       Tpwp = SIGN(MAX(ABS(Tpwp),1.E-6),Tpwp)
L02878
L02879
L02880  !!  TKE budget  (Puhales, 2020, WRF 4.2.1)  << EOB
L02881
L02882          !!!Shear Term
L02883          !!!qSHEAR1(k)=-(upwp*dudz + vpwp*dvdz)
L02884          qSHEAR1(k) = elq*sm(k)*gm(k) !staggered
L02885
L02886          !!!Buoyancy Term
L02887          !!!qBUOY1(k)=grav*Tpwp/thl(k)
L02888          !qBUOY1(k)= elq*(sh(k)*gh(k) + gamv)
L02889          !qBUOY1(k) = elq*(sh(k)*(-dTdz*grav/thl(k)) + gamv) !! ORIGINAL CODE
L02890
L02891          !! Buoyncy term takes the TKEprodTD(k) production now
L02892          qBUOY1(k) = elq*(sh(k)*gh(k)+gamv)  +         &
L02893          &           p5*TKEprod_dn(k)       +          & ! xmchen
L02894          &           p5*TKEprod_up(k)
L02895
L02896          !!!Dissipation Term (now it evaluated in mym_predict)
L02897          !qDISS1(k) = (q3sq**(3./2.))/(b1*MAX(el(k),1.)) !! ORIGINAL CODE
L02898
L02899          !! >> EOB
L02900       ENDIF
L02901
L02902       END DO
```

```
L02903  !
L02904      !make sure all inent(out) vars are initialized
L02905      dfm(kts) = zero
L02906      dfh(kts) = zero
L02907      dfq(kts) = zero
L02908      pdk(kts) = zero
L02909      pdt(kts) = zero
L02910      pdq(kts) = zero
L02911      pdc(kts) = zero
L02912      tcd(kts) = zero
L02913      qcd(kts) = zero
L02914
L02915      tcd(kte) = zero
L02916      qcd(kte) = zero
L02917
L02918  !
L02919      DO k = kts,kte-1
L02920         dzk = dz(k)
L02921         tcd(k) = ( tcd(k+1)-tcd(k) )/( dzk )
L02922         qcd(k) = ( qcd(k+1)-qcd(k) )/( dzk )
L02923      END DO
L02924  !
L02925      if (spp_pbl==1) then
L02926         DO k = kts,kte
L02927            dfm(k)= dfm(k) + dfm(k)* pattern_spp_pbl1(k) * 1.5 * MAX(exp(-MAX(zw(k)-8000.,zero)/2000.),0.001)
L02928            dfh(k)= dfh(k) + dfh(k)* pattern_spp_pbl1(k) * 1.5 * MAX(exp(-MAX(zw(k)-8000.,zero)/2000.),0.001)
L02929         END DO
L02930      endif
L02931
L02932  !    RETURN
L02933  #ifdef HARDCODE_VERTICAL
L02934  # undef kts
L02935  # undef kte
L02936  #endif
L02937
L02938    END SUBROUTINE mym_turbulence
```

- **Helfand & Labraga (1989) 判据**（L2581）：`q3sq < q2sq`（湍动能不足）时 `qdiv = sqrt(q3sq/q2sq)` = (1−α)，L2.5 稳定函数乘 qdiv 衰减；q3sq≥q2sq 时用完整 L2.5 闭式 `sm = q3sq • a1 • (e3−3c1 • e4)/eden`、`sh = q3sq • (a2 • a2fac) • (e2+3c1 • e5c • gmel)/eden`（e1–e4/eden 是含 GM/GH 的多项式组）。**L2 与 L2.5 的切换由湍流强度判据动态决定**——MYNN 与标准 MY 的关键差异点（教材 ch14 §14.4）。
- **Level 3 对比度项**（closure≥3，L2673 起）：由预报的二阶矩（t3sq/r3sq/c3sq 与 L2.5 诊断值 t2sq/r2sq/c2sq 之差）算非高斯修正——`gamt/gamq` 进逆梯度项 `tcd/qcd`，`gamv` 进 TKE 浮力生产；`c3sq` 的动态限（Janjic 2001 方法算 Rsl，**实际写死 0.12**，注释保留了动态限代码）保证 cw ∈ [0.12, 0.76] 物理区间。**L3 = L2.5 + 逆梯度扩散**。
- **云/羽流的最小扩散**（L2822–L2838）：有质量通量（`mfmax>0`）、有云（`cldavg>0`）、有下沉 TKE 生产时，`sm/sh` 强制 ≥0.04——**湍流闭合为非局地过程保底**；TTE 配置另有 qpe 下限。
- **生产项与扩散系数**（L2840–L2866）：`pdk = el • q • (sm • gm+sh • gh+gamv) + 0.5 • TKEprod_dn + 0.5 • TKEprod_up`（**MF 生产各占一半进入剪切层平均**）、`dfm = el • qtw • sm/dzk`、`dfh = el • q • sh/dzk`——注意 `elq` 用 qkw（TKE 速度）而 `elqt` 用 qtw（含 qpe 的总湍流速度，closure<2.7 时两者相等）。
- **TKE 收支**（tke_budget=1）：剪切项 `qSHEAR = el • q • sm • gm`、浮力项 `qBUOY = el • q • (sh • gh+gamv) + MF` 生产（Puhales 2020 口径，单位 m²/s³）。
- **SPP 随机扰动**（L2925–L2930）：`dfm/dfh ×= (1 + 1.5 • pattern • exp(−max(zw−8000,0)/2000))`——扰动只在 8 km 以下生效、随高度指数衰减。**集合预报的湍流不确定性通道**。

09a.10 `mym_predict`：Crank–Nicolson 三对角预测

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L2986-L3155
L02986  SUBROUTINE  mym_predict (kts,kte,                                  &
L02987       &            closure,                                         &
L02988       &            delt, dz,                                        &
L02989       &            ust, flt, flq, pmz, phh,                         &
L02990       &            el,  dfq, rho,                                   &
L02991       &            pdk, pdt, pdq, pdc,                              &
L02992       &            qke, tsq, qsq, cov,                              &
L02993       &            s_aw1, s_awqke1, s_awqsq1,                       &
L02994       &            bl_mynn_edmf, bl_mynn_edmf_tke,                  &
L02995       &            qWT1, qDISS1,tke_budget, upwind                  )
L02996
L02997  !-------------------------------------------------------------------
L02998    use module_bl_mynnedmf_common, only: qkemin,karman,sqfac,b1,b2,     &
L02999         zero,one,two,three,p5,kind_phys
L03000
L03001    integer, intent(in) :: kts,kte
L03002
L03003  #ifdef HARDCODE_VERTICAL
L03004  # define kts 1
L03005  # define kte HARDCODE_VERTICAL
L03006  #endif
L03007
L03008  real(kind_phys), intent(in)    :: closure,upwind
L03009  integer, intent(in) :: bl_mynn_edmf,bl_mynn_edmf_tke,tke_budget
L03010  real(kind_phys), dimension(kts:kte), intent(in) :: dz, dfq, el, rho
L03011  real(kind_phys), dimension(kts:kte), intent(inout) :: pdk, pdt, pdq, pdc
```

```
L03012  real(kind_phys), intent(in)    :: flt, flq, pmz, phh
L03013  real(kind_phys), intent(in)    :: ust, delt
L03014  real(kind_phys), dimension(kts:kte), intent(inout) :: qke,tsq, qsq, cov
L03015  real(kind_phys), dimension(kts:kte+1), intent(inout) :: s_awqke1,s_awqsq1,s_aw1
L03016
L03017  !!  TKE budget  (Puhales, 2020, WRF 4.2.1)  << EOB
L03018  real(kind_phys), dimension(kts:kte), intent(out) :: qWT1, qDISS1
L03019  real(kind_phys), dimension(kts:kte) :: tke_up,dzinv
L03020  !! >> EOB
L03021
L03022  integer :: k
L03023  real(kind_phys), dimension(kts:kte) :: qkw, bp, rp, df3q
L03024  real(kind_phys):: vkz,pdk1,phm,pdt1,pdq1,pdc1,b1l,b2l,onoff
L03025  real(kind_phys), dimension(kts:kte) :: dtz,upcont,dncont
L03026  real(kind_phys), dimension(kts:kte) :: a,b,c,d,x
L03027
L03028  real(kind_phys), dimension(kts:kte) :: rhoinv
L03029  real(kind_phys), dimension(kts:kte+1) :: rhoz,kqdz,kmdz
L03030  !-------------------------------------------------------------------
L03031  ! REGULATE THE MOMENTUM MIXING FROM THE MASS-FLUX SCHEME (on or off)
L03032  IF (bl_mynn_edmf_tke == 0) THEN
L03033     onoff=zero
L03034  ELSE
L03035     onoff=one
L03036  ENDIF
L03037
L03038  !   **  Strictly, vkz*h(i,j) -> karman*( 0.5*dz(1)*h(i,j)+z0 )  **
L03039  vkz = karman*p5*dz(kts)
L03040  !
L03041  !   **  dfq for the TKE is 3.0*dfm.  **
L03042  !
L03043  DO k = kts,kte
L03044  !   qke(k) = MAX(qke(k), zero)
L03045     qkw(k) = SQRT( MAX( qke(k), qkemin ) )
L03046     df3q(k)=Sqfac*dfq(k)
L03047     dtz(k)=delt/dz(k)
L03048  END DO
L03049
L03050  !add stability criteria
L03051  !Prepare "constants" for diffusion equation.
L03052  !khdz = rho*Kh/dz = rho*dfh
L03053  rhoz(kts)  =rho(kts)
L03054  rhoinv(kts)=one/rho(kts)
L03055  kqdz(kts)  =rhoz(kts)*df3q(kts)
L03056  kmdz(kts)  =rhoz(kts)*dfq(kts)
L03057  DO k=kts+1,kte
L03058     rhoz(k)  =(rho(k)*dz(k-1) + rho(k-1)*dz(k))/(dz(k-1)+dz(k))
L03059     rhoz(k)  =    MAX(rhoz(k),1E-4_kind_phys)
L03060     rhoinv(k)=one/MAX(rho(k) ,1E-4_kind_phys)
L03061     kqdz(k)  = rhoz(k)*df3q(k) ! for TKE
L03062     kmdz(k)  = rhoz(k)*dfq(k)  ! for T'2, q'2, and T'q'
L03063  ENDDO
L03064  rhoz(kte+1)=rhoz(kte)
L03065  kqdz(kte+1)=rhoz(kte+1)*df3q(kte)
L03066  kmdz(kte+1)=rhoz(kte+1)*dfq(kte)
L03067
L03068  if (bl_mynn_edmf == 1) then
L03069     !stability criteria for mf
L03070     DO k=kts+1,kte-1
L03071        kqdz(k) = MAX(kqdz(k),  p5* s_aw1(k))
L03072        kqdz(k) = MAX(kqdz(k), -p5*(s_aw1(k)-s_aw1(k+1)))
L03073        kmdz(k) = MAX(kmdz(k),  p5* s_aw1(k))
L03074        kmdz(k) = MAX(kmdz(k), -p5*(s_aw1(k)-s_aw1(k+1)))
L03075  !      kqdz(k) = max(kqdz(k),  0.5*(s_aw1(k)+sd_aw1(k)))
L03076  !      kqdz(k) = max(kqdz(k), -0.5*(s_aw1(k)-s_aw1(k+1)) -0.5*(sd_aw1(k)-sd_aw1(k+1)) )
L03077  !      kmdz(k) = max(kmdz(k),  0.5*(s_aw1(k)+sd_aw1(k)))
L03078  !      kmdz(k) = max(kmdz(k), -0.5*(s_aw1(k)-s_aw1(k+1)) -0.5*(sd_aw1(k)-sd_aw1(k+1)) )
L03079     ENDDO
L03080  endif
L03081
L03082  !-------------------------------------------------------------------------
L03083  !   **  Compute production terms at the surface  **
L03084  !-------------------------------------------------------------------------
L03085  pdk1 = two*ust**3*pmz/( vkz )
L03086  phm  = two/ust   *phh/( vkz )
L03087  pdt1 = phm*flt**2
L03088  pdq1 = phm*flq**2
L03089  pdc1 = phm*flt*flq
L03090  !
L03091  ! **  pdk(1)+pdk(2) corresponds to pdk1.  **
L03092  pdk(kts) = pdk1 - pdk(kts+1)
L03093
L03094  !!  pdt(kts) = pdt1 -pdt(kts+1)
L03095  !!  pdq(kts) = pdq1 -pdq(kts+1)
L03096  !!  pdc(kts) = pdc1 -pdc(kts+1)
L03097  pdt(kts) = pdt(kts+1)
L03098  pdq(kts) = pdq(kts+1)
L03099  pdc(kts) = pdc(kts+1)
L03100
L03101  !-------------------------------------------------------------------------
L03102  !   **  Prediction of twice the turbulent kinetic energy  **
L03103  !-------------------------------------------------------------------------
L03104  !! DO k = kts+1,kte-1
L03105  DO k = kts,kte-1
L03106     b1l = b1*p5*( el(k+1)+el(k) )
L03107     bp(k) = two*qkw(k) / b1l
L03108     rp(k) = pdk(k+1) + pdk(k)
L03109  ENDDO
L03110  bp(kte) = zero
L03111
L03112  ! Since df3q(kts)=0.0, a(1)=0.0 and b(1)=1.+dtz(k)*df3q(k+1)+bp(k)*delt.
L03113  if (bl_mynn_edmf > 1) then
L03114
L03115      DO k=kts+1,kte-1
L03116         upcont(k)= s_awqke1(k)- s_aw1(k)*(qke(k)*upwind+qke(k-1)*(one-upwind))
L03117         dncont(k)=zero !sd_awqke1(k)-sd_aw1(k)*(qke(k)*upwind+qke(k-1)*(one-upwind))
↪
L03118      ENDDO
```

```
L03119      ! no flux at the top of the atmosphere
L03120      upcont(kte)=zero
L03121      dncont(kte)=zero
L03122
L03123      k=kts
L03124      a(1)=zero
L03125      b(1)=one + dtz(k)*kqdz(k+1)*rhoinv(k) + bp(k)*delt
L03126      c(1)=    - dtz(k)*kqdz(k+1)*rhoinv(k)
L03127      d(1)=max(qkemin, qke(k)) + rp(k)*delt                          &
L03128          &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L03129
L03130      DO k=kts+1,kte-1
L03131         a(k)=   - dtz(k)*kqdz(k)*rhoinv(k)
L03132         b(k)=one+ dtz(k)*(kqdz(k)+kqdz(k+1))*rhoinv(k) + bp(k)*delt
L03133         c(k)=   - dtz(k)*kqdz(k+1)*rhoinv(k)
L03134         d(k)=max(qkemin, qke(k)) + rp(k)*delt                       &
L03135           &    - dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L03136      ENDDO
L03137
L03138   else !implicit
L03139      DO k=kts,kte-1
L03140   !       a(k-kts+1)=-dtz(k)*df3q(k)
L03141   !       b(k-kts+1)=1.+dtz(k)*(df3q(k)+df3q(k+1))+bp(k)*delt
L03142   !       c(k-kts+1)=-dtz(k)*df3q(k+1)
L03143   !       d(k-kts+1)=rp(k)*delt + qke(k)
L03144   !JOE 8/22/20 improve conservation
L03145         a(k)=   - dtz(k)*kqdz(k)*rhoinv(k)                          &
L03146             &   + 0.5*dtz(k)*rhoinv(k)*s_aw1(k)*onoff
L03147         b(k)=1. + dtz(k)*(kqdz(k)+kqdz(k+1))*rhoinv(k)              &
L03148             &   + 0.5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))*onoff &
L03149             &   + bp(k)*delt
L03150         c(k)=   - dtz(k)*kqdz(k+1)*rhoinv(k)                        &
L03151             &   - 0.5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff
L03152         d(k)=rp(k)*delt + qke(k)                                    &
L03153             &   + dtz(k)*rhoinv(k)*(s_awqke1(k)-s_awqke1(k+1))*onoff
L03154      ENDDO
L03155   endif
```

- **头注释的方程推导**（L2956–L2981 原文）：把湍流量方程 `dQ/dt = Dh − Ah + P − F`，`Q` 的垂直扩散与耗散用 Crank–Nicolson 半隐式离散成三对角方程组 `−a·Q(k+1) + [1+b+dt·F/2]·Q(k) − c·Q(k−1) =` 右端——**这是全方案数值方法的核心**（Nakanishi 2001 的原始离散）。
- **kqdz/kmdz**：界面上的扩散通量系数 `ρ·K/dz`（TKE 用 `Sqfac·dfq`=3·Km、二阶矩用 dfq=Km）；**JOE 8/22/20 的守恒性改进**：用 kqdz（通量形式）替代旧的 df3q（差分形式），注释自承 “improve conservation”。
- **MF 隐式耦合**（closure 分支）：`bl_mynn_edmf=1`（隐式 MF）时 `a/b/c` 里加 `±0.5·dtz·s_aw·onoff` 项、右端加 `(s_awqke(k)−s_awqke(k+1))`——**质量通量的垂直输运与局地扩散同矩阵求解**；`onoff = bl_mynn_edmf_tke`（MF 对 TKE 的输运开关）。
- **显式 MF**（`bl_mynn_edmf>1`）：MF 输运改到右端 `upcont`（上风差分 `qke(k)·upwind+qke(k−1)·(1−upwind)` 的通量形式）。
- **底边界生产**（L3083–L3099）：`pdk1 = 2u*³·pmz/(κ·dz/2)`（相似律剪切生产）、`pdt1 = (2/u*)·phh/(κ·dz/2)·flt²`（温度方差生产）——表面层用 §09a.4.5 的 pmz/phh。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L3157-L3417
L03157  !!"no flux at top"
L03158  !  a(kte)=-1. !0.
L03159  !  b(kte)=1.
L03160  !  c(kte)=0.
L03161  !  d(kte)=0.
L03162  !! "prescribed value"
L03163  a(kte)=0.
L03164  b(kte)=1.
L03165  c(kte)=0.
L03166  d(kte)=qke(kte)
L03167
L03168  !CALL tridiag(kte,a,b,c,d)
L03169  CALL tridiag2(kte,a,b,c,d,x)
L03170
L03171  DO k=kts,kte
L03172  !   qke(k)=max(d(k-kts+1), qkemin)
L03173     qke(k)=max(x(k), qkemin)
L03174     qke(k)=min(qke(k), 150.)
L03175  ENDDO
L03176
L03177
L03178  !! TKE budget  (Puhales, 2020, WRF 4.2.1)  << EOB
L03179  IF (tke_budget .eq. 1) THEN
L03180     !! TKE Vertical transport << EOBvt
L03181     tke_up=p5*qke
L03182     dzinv=one/dz
L03183     k=kts
L03184     qWT1(k)=dzinv(k)*(                                                &
L03185          &  (kqdz(k+1)*(tke_up(k+1)-tke_up(k))-kqdz(k)*tke_up(k))     &
L03186          &  + p5*rhoinv(k)*(s_aw1(k+1)*tke_up(k+1)                    &
L03187          &  +      (s_aw1(k+1)-s_aw1(k))*tke_up(k)                    &
L03188          &  +      (s_awqke1(k)-s_awqke1(k+1)))*onoff) !unstaggered
L03189     DO k=kts+1,kte-1
L03190        qWT1(k)=dzinv(k)*(                                             &
L03191             & (kqdz(k+1)*(tke_up(k+1)-tke_up(k))-kqdz(k)*(tke_up(k)-tke_up(k-1))) &
L03192             &  + p5*rhoinv(k)*(s_aw1(k+1)*tke_up(k+1)                 &
L03193             &  +      (s_aw1(k+1)-s_aw1(k))*tke_up(k)                 &
L03194             &  -                 s_aw1(k)*tke_up(k-1)                &
L03195             &  +      (s_awqke1(k)-s_awqke1(k+1)))*onoff) !unstaggered
L03196     ENDDO
L03197     k=kte
L03198     qWT1(k)=dzinv(k)*(-kqdz(k)*(tke_up(k)-tke_up(k-1))                &
```

```
L03199         &  + p5*rhoinv(k)*(-s_aw1(k)*tke_up(k)-s_aw1(k)*tke_up(k-1)+s_awqke1(k))*onoff) !unstaggered
L03200     !!  >> EOBvt
L03201     qDISS1=bp*tke_up !! TKE dissipation rate !unstaggered
L03202   END IF
L03203   !! >> EOB
L03204
L03205   if ( closure > 2.5 ) then
L03206     !-----------------------------------------------------------------
L03207     !   **  Prediction of the moisture variance  **
L03208     !-----------------------------------------------------------------
L03209     do k = kts,kte-1
L03210        b2l   = b2*p5*( el(k+1)+el(k) )
L03211        bp(k) = two*qkw(k) / b2l
L03212        rp(k) = pdq(k+1) + pdq(k)
L03213     enddo
L03214     bp(kte) = zero
L03215
L03216
L03217     if (bl_mynn_edmf > 1) then
L03218        do k=kts+1,kte-1
L03219           upcont(k)= s_awqsq1(k)- s_aw1(k)*(qsq(k)*upwind+qsq(k-1)*(one-upwind))
L03220           dncont(k)=zero !sd_awqsq1(k)-sd_aw1(k)*(qsq(k)*upwind+qsq(k-1)*(one-upwind))
L03221        enddo
L03222        ! no flux at the top of the atmosphere
L03223        upcont(kte)=zero
L03224        dncont(kte)=zero
L03225
L03226        k=kts
L03227        a(1)=zero
L03228        b(1)=one + dtz(k)*kmdz(k+1)*rhoinv(k) + bp(k)*delt
L03229        c(1)=    - dtz(k)*kmdz(k+1)*rhoinv(k)
L03230        d(1)=max(zero, qsq(k)) + rp(k)*delt                        &
L03231            &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L03232
L03233        do k=kts+1,kte-1
L03234           a(k)=   - dtz(k)*kmdz(k)*rhoinv(k)
L03235           b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k) + bp(k)*delt
L03236           c(k)=   - dtz(k)*kmdz(k+1)*rhoinv(k)
L03237           d(k)=max(zero, qsq(k)) + rp(k)*delt                     &
L03238               &    - dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L03239        enddo
L03240
L03241     else !implicit
L03242
L03243        do k=kts,kte-1
L03244           a(k)=   - dtz(k)*kmdz(k)*rhoinv(k)                        &
L03245               &   + 0.5*dtz(k)*rhoinv(k)*s_aw1(k)*onoff
L03246           b(k)=1. + dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k)            &
L03247               &   + 0.5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))*onoff &
L03248               &   + bp(k)*delt
L03249           c(k)=   - dtz(k)*kmdz(k+1)*rhoinv(k)                      &
L03250               &   - 0.5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff
L03251           d(k)=rp(k)*delt + qsq(k)                                  &
L03252               &   + dtz(k)*rhoinv(k)*(s_awqsq1(k)-s_awqsq1(k+1))*onoff
L03253        enddo
L03254     endif
L03255
L03256     a(kte)=-one
L03257     b(kte)=one
L03258     c(kte)=zero
L03259     d(kte)=zero !qsq(kte)
L03260
L03261     !call tridiag(kte,a,b,c,d)
L03262     call tridiag2(kte,a,b,c,d,x)
L03263
L03264     do k=kts,kte
L03265        !qke(k)=max(d(k-kts+1), qkemin)
L03266        qsq(k)=max(x(k), 1e-17_kind_phys)
L03267        qsq(k)=min(qsq(k),6e-6_kind_phys)
L03268     enddo
L03269
L03270   else
L03271     !Level 2.5 - use level 2 diagnostic
L03272     do k = kts,kte-1
L03273        if ( qkw(k) .le. zero ) then
L03274           b2l = zero
L03275        else
L03276           b2l = b2*0.25_kind_phys*( el(k+1)+el(k) )/qkw(k)
L03277        end if
L03278        qsq(k) = min(6e-6_kind_phys, max(1e-17_kind_phys, b2l*( pdq(k+1)+pdq(k) )))
L03279     enddo
L03280     qsq(kte)=qsq(kte-1)
L03281   endif
L03282
L03283   !!!!!!!!!!!!!!!!!!!!!!end level 2.6
L03284
L03285       IF ( closure .GE. 2.7 ) THEN
L03286   !
L03287   !   **  dfq for the scalar variance is 1.0*dfm.  **
L03288   !
L03289   !   **  Prediction of the temperature variance  **
L03290   !!       DO k = kts+1,kte-1
L03291          DO k = kts,kte-1
L03292             b2l   = b2*p5*( el(k+1)+el(k) )
L03293             bp(k) = two*qkw(k) / b2l
L03294             rp(k) = pdt(k+1) + pdt(k)
L03295          END DO
L03296
L03297   !zero gradient for tsq at bottom and top
L03298
L03299   !!       a(1)=0.
L03300   !!       b(1)=1.
L03301   !!       c(1)=-1.
L03302   !!       d(1)=0.
L03303
L03304   ! Since dfq(kts)=0.0, a(1)=0.0 and b(1)=1.+dtz(k)*dfq(k+1)+bp(k)*delt.
L03305          DO k=kts,kte-1
L03306             !a(k-kts+1)=-dtz(k)*dfq(k)
```

```
L03307            !b(k-kts+1)=1.+dtz(k)*(dfq(k)+dfq(k+1))+bp(k)*delt
L03308            !c(k-kts+1)=-dtz(k)*dfq(k+1)
L03309            !d(k-kts+1)=rp(k)*delt + tsq(k)
L03310  !JOE 8/22/20 improve conservation
L03311            a(k)=   - dtz(k)*kmdz(k)*rhoinv(k)
L03312            b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k) + bp(k)*delt
L03313            c(k)=   - dtz(k)*kmdz(k+1)*rhoinv(k)
L03314            d(k)=rp(k)*delt + max(zero, tsq(k))
L03315         ENDDO
L03316
L03317  !!       DO k=kts+1,kte-1
L03318  !!          a(k-kts+1)=-dtz(k)*dfq(k)
L03319  !!          b(k-kts+1)=1.+dtz(k)*(dfq(k)+dfq(k+1))
L03320  !!          c(k-kts+1)=-dtz(k)*dfq(k+1)
L03321  !!          d(k-kts+1)=rp(k)*delt + tsq(k) - tsq(k)*bp(k)*delt
L03322  !!       ENDDO
L03323
L03324         a(kte)=-1. !0.
L03325         b(kte)=one
L03326         c(kte)=zero
L03327         d(kte)=zero
L03328
L03329  !       CALL tridiag(kte,a,b,c,d)
L03330         CALL tridiag2(kte,a,b,c,d,x)
L03331
L03332         DO k=kts,kte
L03333            !tsq(k)=d(k-kts+1)
L03334            tsq(k)=min(three, max(zero,x(k)))
L03335         ENDDO
L03336      ELSE
L03337         !less than level 2.7 - default to level 2 diagnostic
L03338         DO k = kts,kte-1
L03339            IF ( qkw(k) .LE. zero ) THEN
L03340               b2l = zero
L03341            ELSE
L03342               b2l = b2*0.25*( el(k+1)+el(k) )/qkw(k)
L03343            END IF
L03344            tsq(k) = min(three, max(zero, b2l*( pdt(k+1)+pdt(k) )))
L03345         END DO
L03346         tsq(kte)=tsq(kte-1)
L03347      ENDIF
L03348
L03349      !Full level 3 closure
L03350      IF ( closure .GE. 3.0 ) THEN
L03351  !   ** Prediction of the temperature-moisture covariance  **
L03352         DO k = kts,kte-1
L03353            b2l = b2*p5*( el(k+1)+el(k) )
L03354            bp(k) = two*qkw(k) / b2l
L03355            rp(k) = pdc(k+1) + pdc(k)
L03356         END DO
L03357
L03358  !zero gradient for tqcov at bottom and top
L03359  !!       a(1)=0.
L03360  !!       b(1)=1.
L03361  !!       c(1)=-1.
L03362  !!       d(1)=0.
L03363
L03364  ! Since dfq(kts)=0.0, a(1)=0.0 and b(1)=1.+dtz(k)*dfq(k+1)+bp(k)*delt.
L03365         DO k=kts,kte-1
L03366            !a(k-kts+1)=-dtz(k)*dfq(k)
L03367            !b(k-kts+1)=1.+dtz(k)*(dfq(k)+dfq(k+1))+bp(k)*delt
L03368            !c(k-kts+1)=-dtz(k)*dfq(k+1)
L03369            !d(k-kts+1)=rp(k)*delt + cov(k)
L03370  !JOE 8/22/20 improve conservation
L03371            a(k)=   - dtz(k)*kmdz(k)*rhoinv(k)
L03372            b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k) + bp(k)*delt
L03373            c(k)=   - dtz(k)*kmdz(k+1)*rhoinv(k)
L03374            d(k)=rp(k)*delt + cov(k)
L03375         ENDDO
L03376
L03377  !!       DO k=kts+1,kte-1
L03378  !!          a(k-kts+1)=-dtz(k)*dfq(k)
L03379  !!          b(k-kts+1)=1.+dtz(k)*(dfq(k)+dfq(k+1))
L03380  !!          c(k-kts+1)=-dtz(k)*dfq(k+1)
L03381  !!          d(k-kts+1)=rp(k)*delt + cov(k) - cov(k)*bp(k)*delt
L03382  !!       ENDDO
L03383
L03384         a(kte)=-1. !0.
L03385         b(kte)=one
L03386         c(kte)=zero
L03387         d(kte)=zero
L03388
L03389  !       CALL tridiag(kte,a,b,c,d)
L03390         CALL tridiag2(kte,a,b,c,d,x)
L03391
L03392         DO k=kts,kte
L03393  !          cov(k)=d(k-kts+1)
L03394            cov(k)=x(k)
L03395         ENDDO
L03396
L03397      ELSE
L03398
L03399         !Not level 3 - default to level 2 diagnostic
L03400         DO k = kts,kte-1
L03401            IF ( qkw(k) .LE. zero ) THEN
L03402               b2l = zero
L03403            ELSE
L03404               b2l = b2*0.25*( el(k+1)+el(k) )/qkw(k)
L03405            END IF
L03406            cov(k) = b2l*( pdc(k+1)+pdc(k) )
L03407         END DO
L03408         cov(kte)=cov(kte-1)
L03409
L03410      END IF
L03411
L03412  #ifdef HARDCODE_VERTICAL
L03413  # undef kts
L03414  # undef kte
L03415  #endif
```

```
L03416
L03417    END SUBROUTINE mym_predict
```

- **三对角求解**：`CALL tridiag2(kte,a,b,c,d,x)`（求解器三兄弟 tridiag/tridiag2/tridiag3 在 code_09b；这里统一用 tridiag2）。
- **四量预测的分支结构**：qke（`bl_mynn_edmf` 两种耦合，解出后钳 [qkemin, 150]）；qsq（closure>2.5 走三对角、否则 L2 诊断 `b2l • (pdq+pdq)`，钳 [1e-17, 6e-6]）；tsq（closure≥2.7 三对角、钳 [0,3]）；cov（closure≥3.0 三对角、否则诊断）——**closure 参数（2.5/2.6/2.7/3.0）逐量决定 “预报还是诊断”**，这就是 1.5 阶到 3 阶闭合的软件开关面。
- TKE 收支的垂直输运项 `qWT1` 与耗散项 `qDISS1 = bp • tke_up`（qke 的一半——`qke=2TKE`，耗散率 = qke^1.5/(b1 • el)）在此一并输出。

09a.11 `mym_condensation`：次网格云 PDF 三选

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L3453-L3617
L03453    SUBROUTINE  mym_condensation (kts,kte,   &
L03454      &           dx, dz, zw, xland,         &
L03455      &           thl, qw, qv, qc, qi, qs,   &
L03456      &           p,exner,                   &
L03457      &           tsq, qsq, cov,             &
L03458      &           Sh, el, bl_mynn_cloudpdf, &
L03459      &           qc_bl1, qi_bl1,            &
L03460      &           cldfra_bl1,                &
L03461      &           PBLH1,HFX1,                &
L03462      &           Vt, Vq, th, sgm,           &
L03463      &           closure,                   &
L03464      &           spp_pbl, pattern_spp_pbl1 )
L03465
L03466   !-----------------------------------------------------------------
L03467      use module_bl_mynnedmf_common, only: cp,xlv,r_d,r_v,xlvcp,ep_2,   &
L03468           ep_3,p608,cpv,rr2,rrp,tv0,tice,tliq,b2,                      &
L03469           zero,one,two,three,four,five,ten,hundred,p1,p2,p5,           &
L03470           kind_phys
L03471
L03472      integer, intent(in)   :: kts,kte, bl_mynn_cloudpdf
L03473
L03474   #ifdef HARDCODE_VERTICAL
L03475   # define kts 1
L03476   # define kte HARDCODE_VERTICAL
L03477   #endif
L03478
L03479      real(kind_phys), intent(in)      :: HFX1,xland
L03480      real(kind_phys), intent(in)      :: dx,pblh1,closure
L03481      real(kind_phys), dimension(kts:kte), intent(in) :: dz
L03482      real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L03483      real(kind_phys), dimension(kts:kte), intent(in) :: p,exner,thl,qw,   &
L03484           &qv,qc,qi,qs,tsq,qsq,cov,th
L03485
L03486      real(kind_phys), dimension(kts:kte), intent(inout) :: vt,vq,sgm
L03487
L03488      real(kind_phys), dimension(kts:kte) :: alp,a,bet,b,ql,q1,RH
L03489      real(kind_phys), dimension(kts:kte), intent(out) :: qc_bl1,qi_bl1,   &
L03490           &cldfra_bl1
L03491      DOUBLE PRECISION :: t3sq, r3sq, c3sq
L03492
L03493      real(kind_phys):: qsl,esat,qsat,dqsl,cld0,q1k,qlk,eq1,ql1,           &
L03494           &q2p,pt,rac,qt,t,xl,rsl,cpm,Fng,qww,alpha,beta,bb,sgmq,sgmc,    &
L03495           &ls,wt,wt2,cld_factor,fac_damp,liq_frac,ql_ice,ql_water,        &
L03496           &qmq,qsat_tk,q1_rh,rh_adj,zsl,maxqc,cldfra_rh,cldfra_qsq,       &
L03497           &cldfra_rh0,cldfra_rh1,cldfra_qsq0,cldfra_qsq1,clim,qlim
L03498      !lower limits for sgm (for mixing ratio estimates) in case sgm falls out (% of qw)
L03499      real(kind_phys), parameter :: qlim_sfc =0.008
L03500      real(kind_phys), parameter :: qlim_pbl =0.020
L03501      real(kind_phys), parameter :: qlim_trp =0.025
L03502      !lower limits for sqm (for cloud fraction) in case sgm falls out (% of qw)
L03503      real(kind_phys), parameter :: clim_sfc =0.010
L03504      real(kind_phys), parameter :: clim_pbl =0.020
L03505      real(kind_phys), parameter :: clim_trp =0.030
L03506      real(kind_phys), parameter :: rhcrit   =0.83 !for cloudpdf = 2
L03507      real(kind_phys), parameter :: rhmax    =1.10 !for cloudpdf = 2
L03508      integer :: i,j,k
L03509
L03510      real(kind_phys):: erf
L03511
L03512      !VARIABLES FOR ALTERNATIVE SIGMA
L03513      real:: dth,dtl,dqw,dzk,els
L03514      real(kind_phys), dimension(kts:kte), intent(in) :: Sh,el
L03515
L03516      !variables for SGS BL clouds
L03517      real(kind_phys)           :: zagl,damp,PBLH2
L03518      real(kind_phys)           :: cfmax
L03519
L03520      !JAYMES:  variables for tropopause-height estimation
L03521      real(kind_phys)           :: theta1, theta2, ht1, ht2
L03522      integer                   :: k_tropo
L03523
L03524   !   Stochastic
L03525      integer,  intent(in)      :: spp_pbl
L03526      real(kind_phys), dimension(kts:kte) :: pattern_spp_pbl1
L03527      real(kind_phys)           :: qw_pert
L03528
L03529   ! First, obtain an estimate for the tropopause height (k), using the method employed in the
L03530   ! Thompson subgrid-cloud scheme.  This height will be a consideration later when determining
L03531   ! the "final" subgrid-cloud properties.
L03532   ! JAYMES:  added 3 Nov 2016, adapted from G. Thompson
L03533
```

```
L03534      DO k = kte-3, kts, -1
L03535         theta1 = th(k)
L03536         theta2 = th(k+2)
L03537         ht1 = 44307.692 * (one - (p(k)/101325.)**0.190)
L03538         ht2 = 44307.692 * (one - (p(k+2)/101325.)**0.190)
L03539         if ( (((theta2-theta1)/(ht2-ht1)) .lt. 10./1500. ) .AND.       &
L03540       &                     (ht1.lt.19000.) .and. (ht1.gt.4000.) ) then
L03541            goto 86
L03542         endif
L03543      ENDDO
L03544   86   continue
L03545      k_tropo = MAX(kts+2, k+2)
L03546
L03547      zagl = 0.
L03548
L03549      SELECT CASE(bl_mynn_cloudpdf)
L03550
L03551        CASE (0) ! ORIGINAL MYNN PARTIAL-CONDENSATION SCHEME
L03552
L03553          DO k = kts,kte-1
L03554             t  = th(k)*exner(k)
L03555
L03556  !x      if ( ct .gt. zero ) then
L03557  !       a  =  17.27
L03558  !       b  = 237.3
L03559  !x      else
L03560  !x        a  =  21.87
L03561  !x        b  = 265.5
L03562  !x      end if
L03563  !
L03564  !   **  3.8 = 0.622*6.11 (hPa)  **
L03565
L03566             !SATURATED VAPOR PRESSURE
L03567             esat = esat_blend(t)
L03568             !SATURATED SPECIFIC HUMIDITY
L03569             !qsl=ep_2*esat/(p(k)-ep_3*esat)
L03570             qsl=ep_2*esat/max(1.e-4,(p(k)-ep_3*esat))
L03571             !dqw/dT: Clausius-Clapeyron
L03572             dqsl = qsl*ep_2*xlv/( r_d*t**2 )
L03573
L03574             alp(k) = one/( one+dqsl*xlvcp )
L03575             bet(k) = dqsl*exner(k)
L03576
L03577             !Sommeria and Deardorff (1977) scheme, as implemented
L03578             !in Nakanishi and Niino (2009), Appendix B
L03579             t3sq = MAX( tsq(k), zero )
L03580             r3sq = MAX( qsq(k), zero )
L03581             c3sq =      cov(k)
L03582             c3sq = SIGN( MIN( ABS(c3sq), SQRT(t3sq*r3sq) ), c3sq )
L03583             r3sq = r3sq +bet(k)**2*t3sq -2.0*bet(k)*c3sq
L03584             !DEFICIT/EXCESS WATER CONTENT
L03585             qmq  = qw(k) -qsl
L03586             !ORIGINAL STANDARD DEVIATION
L03587             sgm(k) = SQRT( MAX( r3sq, 1.0d-10 ))
L03588             !NORMALIZED DEPARTURE FROM SATURATION
L03589             q1(k)   = qmq / sgm(k)
L03590             !CLOUD FRACTION. rr2 = 1/SQRT(2) = 0.707
L03591             cldfra_bl1(k) = 0.5*( one+erf( q1(k)*rr2 ) )
L03592
L03593             q1k  = q1(k)
L03594             eq1  = rrp*EXP( -0.5*q1k*q1k )
L03595             qll  = MAX( cldfra_bl1(k)*q1k + eq1, zero )
L03596             !ESTIMATED LIQUID WATER CONTENT (UNNORMALIZED)
L03597             ql(k) = alp(k)*sgm(k)*qll
L03598             !LIMIT SPECIES TO TEMPERATURE RANGES
L03599             liq_frac = min(one, max(zero,(t-240.0)/29.0))
L03600             qc_bl1(k) = liq_frac*ql(k)
L03601             qi_bl1(k) = (one - liq_frac)*ql(k)
L03602
L03603             !Now estimate the buoyancy flux functions
L03604             q2p = xlvcp/exner(k)
L03605             pt = thl(k) +q2p*ql(k) ! potential temp
L03606
L03607             !qt is a THETA-V CONVERSION FOR TOTAL WATER (i.e., THETA-V = qt*THETA)
L03608             qt   = one +p608*qw(k) -(1.+p608)*(qc_bl1(k)+qi_bl1(k))*cldfra_bl1(k)
L03609             rac  = alp(k)*( cldfra_bl1(K)-qll*eq1 )*( q2p*qt-(1.+p608)*pt )
L03610
L03611             !BUOYANCY FACTORS: wherever vt and vq are used, there is a
L03612             !"+1" and "+tv0", respectively, so these are subtracted out here.
L03613             !vt is unitless and vq has units of K.
L03614             vt(k) =      qt-one -rac*bet(k)
L03615             vq(k) = p608*pt-tv0 +rac
L03616
L03617          END DO
```

- **对流层顶估计**（L3529–L3545，Jaymes 2016 改自 Thompson 方案）：找位温垂直变化率 < 10 K/1500 m 且 4–19 km 的层——**云 PDF 在平流层关闭的判据**（L3842：k≥k_tropo 清零）。
- **CASE 0（原始 MYNN，NN2009 附录 B）**：Sommeria & Deardorff (1977) 正态 PDF——总水方差 `r3sq = qsq + bet²·tsq − 2·bet·cov`（**温湿协方差换算到饱和扰动方差**）、云量 `cldfra = 0.5·(1+erf(q1/√2))`（q1 = 饱和差/σ）、归一化液水 `qll = cldfra·q1 + rrp·exp(−q1²/2)`（SD77 式 29a——**erf 的解析伴随项**）、`ql = alp·sgm·qll`；`liq_frac = (T−240)/29` 温度混合决定 qc/qi 分相。
- **vt/vq 浮力因子**（L3603–L3615）：`rac` 项把 “云量内的凝结加热” 折进浮力通量——NN2009 式 B8 的实现；注释明说 vt/vq 在使用处会 +1/+tv0，这里预先扣除。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L3619-L3674
L03619        CASE (1, -1) !ALTERNATIVE FORM (Nakanishi & Niino 2004 BLM, eq. B6, and
L03620                     !Kuwano-Yoshida et al. 2010 QJRMS, eq. 7):
L03621          DO k = kts,kte-1
```

```
L03622             t  = th(k)*exner(k)
L03623             !SATURATED VAPOR PRESSURE
L03624             esat = esat_blend(t)
L03625             !SATURATED SPECIFIC HUMIDITY
L03626             !qsl=ep_2*esat/(p(k)-ep_3*esat)
L03627             qsl=ep_2*esat/max(1.e-4,(p(k)-ep_3*esat))
L03628             !dqw/dT: Clausius-Clapeyron
L03629             dqsl = qsl*ep_2*xlv/( r_d*t**2 )
L03630
L03631             alp(k) = one/( one+dqsl*xlvcp )
L03632             bet(k) = dqsl*exner(k)
L03633
L03634             if (k .eq. kts) then
L03635               dzk = 0.5*dz(k)
L03636             else
L03637               dzk = dz(k)
L03638             end if
L03639             dth = 0.5*(thl(k+1)+thl(k)) - 0.5*(thl(k)+thl(MAX(k-1,kts)))
L03640             dqw = 0.5*(qw(k+1) + qw(k)) - 0.5*(qw(k) + qw(MAX(k-1,kts)))
L03641             sgm(k) = SQRT( MAX( (alp(k)**2 * MAX(el(k)**2, p1) * &
L03642                              b2 * MAX(Sh(k),0.03))/four * &
L03643                       (dqw/dzk - bet(k)*(dth/dzk ))**2 , 1.0e-10) )
L03644             qmq   = qw(k) -qsl
L03645             q1(k) = qmq / sgm(k)
L03646             cldfra_bl1(K) = p5*( one+erf( q1(k)*rr2 ) )
L03647
L03648             !now compute estimated lwc for PBL scheme's use
L03649             !qll IS THE NORMALIZED LIQUID WATER CONTENT (Sommeria and
L03650             !Deardorff (1977, eq 29a). rrp = 1/(sqrt(2*pi)) = 0.3989
L03651             q1k  = q1(k)
L03652             eq1  = rrp*EXP( -0.5*q1k*q1k )
L03653             qll  = MAX( cldfra_bl1(K)*q1k + eq1, zero )
L03654             !ESTIMATED LIQUID WATER CONTENT (UNNORMALIZED)
L03655             ql (k) = alp(k)*sgm(k)*qll
L03656             liq_frac = min(one, max(zero,(t-240.0)/29.0))
L03657             qc_bl1(k) = liq_frac*ql(k)
L03658             qi_bl1(k) = (one - liq_frac)*ql(k)
L03659
L03660             !Now estimate the buoyancy flux functions
L03661             q2p = xlvcp/exner(k)
L03662             pt = thl(k) +q2p*ql(k) ! potential temp
L03663
L03664             !qt is a THETA-V CONVERSION FOR TOTAL WATER (i.e., THETA-V = qt*THETA)
L03665             qt   = one +p608*qw(k) -(1.+p608)*(qc_bl1(k)+qi_bl1(k))*cldfra_bl1(k)
L03666             rac  = alp(k)*( cldfra_bl1(K)-q1k*eq1 )*( q2p*qt-(1.+p608)*pt )
L03667
L03668             !BUOYANCY FACTORS: wherever vt and vq are used, there is a
L03669             !"+1" and "+tv0", respectively, so these are subtracted out here.
L03670             !vt is unitless and vq has units of K.
L03671             vt(k) =      qt-one -rac*bet(k)
L03672             vq(k) = p608*pt-tv0 +rac
L03673
L03674           END DO
```

CASE 1（−1）：Nakanishi & Niino (2004) 式 B6 与 Kuwano-Yoshida (2010) 式 7——σ 不用二阶矩而是用混合长闭合诊断：`sgm² = (alp²•el²•b2•max(sh,0.03)/4)•(dqw/dz − bet•dth/dz)²`——湍流尺度的水汽起伏直接由梯度 × 混合长给出（无 tsq/qsq 时的退化方案）。云量/液水/浮力因子与 CASE 0 同构。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L3676-L3917
L03676         CASE (2, -2)
L03677
L03678           !Diagnostic statistical scheme of Chaboureau and Bechtold (2002), JAS
L03679           !but with use of higher-order moments to estimate sigma
L03680           pblh2=MAX(ten,pblh1)
L03681           DO k = kts,kte-1
L03682              zagl   = zw(k) + p5*dz(k)
L03683
L03684              t      = th(k)*exner(k)
L03685              xl     = xl_blend(t)              ! obtain latent heat
L03686              qsat_tk= qsat_blend(t,  p(k))     ! saturation water vapor mixing ratio at tk and p
L03687              rh(k)  = MAX(MIN(rhmax, qw(k)/MAX(1E-10_kind_phys,qsat_tk)),0.001_kind_phys)
L03688
L03689              !dqw/dT: Clausius-Clapeyron
L03690              dqsl   = qsat_tk*ep_2*xlv/( r_d*t**2 )
L03691              alp(k) = one/(one + dqsl*xlvcp )
L03692              bet(k) = dqsl*exner(k)
L03693
L03694              rsl    = xl*qsat_tk / (r_v*t**2)  ! slope of C-C curve at t (=abs temperature)
L03695                                                ! CB02, Eqn. 4
L03696              cpm    = cp + qw(k)*cpv           ! CB02, sec. 2, para. 1
L03697              a(k)   = one/(one + xl*rsl/cpm)   ! CB02 variable "a"
L03698              b(k)   = a(k)*rsl                 ! CB02 variable "b"
L03699
L03700              !SPP
L03701              qw_pert= qw(k) + qw(k)*p5*pattern_spp_pbl1(k)*real(spp_pbl,kind=kind_phys)
L03702
L03703              !This form of qmq (the numerator of Q1) no longer uses the a(k) factor
L03704              qmq    = qw_pert - qsat_tk          ! saturation deficit/excess;
L03705
L03706              !Use the form of Eq. (6) in Chaboureau and Bechtold (2002)
L03707              !except neglect all but the first term for sig_r
L03708              r3sq   = max( qsq(k), zero )
L03709              t3sq   = max( tsq(k), zero )
L03710              c3sq   =      cov(k)
L03711              c3sq   = SIGN( MIN( ABS(c3sq), SQRT(t3sq*r3sq) ), c3sq )
L03712
L03713              if (closure .lt. 2.7) then
L03714                 r3sq = r3sq
L03715              elseif ((closure .ge. 2.7) .and. (closure .lt. 3.0)) then
L03716                 r3sq = r3sq +bet(k)**2*t3sq
L03717              else
L03718                 r3sq = r3sq +bet(k)**2*t3sq -two*bet(k)*c3sq
```

```
L03719              endif
L03720
L03721              !Calculate sigma using higher-order moments:
L03722              sgm(k) = max(1e-13, sqrt( real(r3sq,kind_phys) ))
L03723              !Set constraints on sigma relative to total water
L03724              sgm(k) = min( sgm(k), qw(k)*p5 )
L03725
L03726              !introduce vertical grid spacing dependence on min sgm
L03727              wt     = min(one, max(zero, dz(k)-100.)/500.) !=0 for dz < 100 m, =1 for dz > 600 m
L03728              sgm(k) = sgm(k) + sgm(k)*p2*wt !inflate sgm for coarse dz
L03729              !save sgm for mixing ratio and cloud fraction estimates
L03730              sgmq   = sgm(k)
L03731              sgmc   = sgm(k)
L03732
L03733              !allow minimum sgm to vary with z.
L03734              wt     = min(one, max(zero, (zagl - (pblh2+10.)))/250.) !0 in pbl, 1 aloft
L03735              clim   = clim_pbl*(one-wt) + clim_trp*wt
L03736              zsl    = min(150., max(50., p1*pblh2))        !crude ekman layer
L03737              wt     = min(one, max(zero, zagl - zsl)/150.)  !0 near sfc, 1 above
L03738              clim   = clim_sfc*(one-wt) + clim*wt
L03739              sgmc   = max( sgmc, qw(k)*clim )
L03740              !apply absolute lower limit in case qw = 0.
L03741              sgmc   = max(1e-13, sgmc)
L03742              !For cloud fractions in saturated conditions, apply lower limit on sgmc
L03743              if (qmq .ge. zero) sgmc = max(0.02*qw(k), sgmc)
L03744
L03745              q1(k)  = qmq  / sgmc  ! Q1, the normalized saturation
L03746
L03747              !Add condition for falling/settling into low-RH layers, so at least
L03748              !some cloud fraction is applied for all qc, qs, and qi.
L03749              wt2    = min(one, max(zero, zagl - pblh2)/300.) !0 in pbl, 1 aloft
L03750              !ensure adequate RH & q1 when qi is at least 1e-9 (above the PBLH)
L03751              if ((qi(k)+qs(k))>1.e-10 .and. (zagl .gt. pblh2)) then
L03752                 rh_adj  =min(rhmax, rhcrit + wt2*0.037_kind_phys*(max(zero, ten + log10(qi(k)+qs(k))) ))
L03753                 rh_adj  =max(rh(k), rh_adj)
L03754                 !add rh-based q1
L03755                 q1_rh   =-3. + three*(rh_adj-rhcrit)/(one-rhcrit)
L03756                 q1(k)   =max(q1_rh, q1(k) )
L03757              endif
L03758              !ensure adequate rh & q1 when qc is at least 1e-5 (above the PBLH)
L03759              if (qc(k)>1.e-5 .and. (zagl .gt. pblh2)) then
L03760                 rh_adj  =min(rhmax, rhcrit + wt2*0.07_kind_phys*(max(zero, five + log10(qc(k))) ))
L03761                 rh_adj  =max(rh(k), rh_adj)
L03762                 !add rh-based q1
L03763                 q1_rh   =-3. + three*(rh_adj-rhcrit)/(one-rhcrit)
L03764                 q1(k)   =max(q1_rh, q1(k) )
L03765              endif
L03766
L03767              q1k    = q1(k)          ! backup Q1 for later modification
L03768
L03769              ! Specify cloud fraction
L03770              !Original C-B cloud fraction, allows cloud fractions out to q1 = -3.5
L03771              !cldfra_bl1(K) = max(0., min(1., 0.5+0.36*atan(1.55*q1(k)))) ! Eq. 7 in CB02
L03772              !Waynes LES fit  - over-diffuse, when limits removed from vt & vq & fng
L03773              !cldfra_bl1(K) = max(0., min(1., 0.5+0.36*atan(1.2*(q1(k)+0.4))))
L03774              !Best compromise: Improves marine stratus without adding much cold bias.
L03775              !RRFSv1
L03776              !cldfra_bl1(k) = max(zero, min(one, 0.5+0.36*atan(1.8*(q1k+0.2))))
L03777              !exp1
L03778              !cldfra_bl1(k) = max(zero, min(one, p5+0.36*atan(1.65*q1k)))
L03779
L03780              !Use specialized forms within and outside the pbl.
L03781              wt2           = min(one, max(zero, (zagl - (pblh1-100.))/200.)) !0 in pbl, 1 aloft
L03782
L03783              !cldfra_qsq0   = max(zero, min(one, p5+0.35*atan(4.1*(q1k))))
L03784              cldfra_qsq0   = max(zero, min(one, p5+0.35*atan(3.6*(q1k+0.05))))
L03785              !cldfra_qsq1   = max(zero, min(one, p5+0.37*atan(2.1*(q1k+0.4))))
L03786              cldfra_qsq1   = max(zero, min(one, p5+0.39*atan(1.6*(q1k+0.55))))
L03787              cldfra_qsq    = cldfra_qsq0*(one-wt2) + cldfra_qsq1*wt2
L03788
L03789              !For ceiling detection, apply minimum rh-based cloud fraction
L03790              cldfra_rh0    = min(one, max(zero, 0.56*tanh((rh(k)-0.976)/0.030)+p5))
L03791              cldfra_rh1    = min(one, max(zero, 0.55*tanh((rh(k)-0.955)/0.055)+p5))
L03792              cldfra_rh     = zero !cldfra_rh0*(one-wt2) + cldfra_rh1*wt2
L03793
L03794              cldfra_bl1(k) = max(cldfra_qsq, cldfra_rh)
L03795
L03796              !Specify hydrometeors (grid mean = in-cloud * cloud fraction)
L03797              !allow minimum sgmq (lower limit of mixing ratios) to vary with z.
L03798              wt     = min(one, max(zero, (zagl - (pblh2+ten)))/300.) !0 in pbl, 1 aloft
L03799              qlim   = qlim_pbl*(one-wt) + qlim_trp*wt
L03800              zsl    = min(150., max(50., p1*pblh2)) !height of surface layer
L03801              wt     = min(one, max(zero, zagl - zsl)/200.)  !0 near sfc, 1 above
L03802              qlim   = qlim_sfc*(one-wt) + qlim*wt
L03803              sgmq   = max(sgmq, qw(k)*qlim)
L03804
L03805              ql_water = min(sgmq, 0.025*qw(k))*cldfra_bl1(k)
L03806              ql_ice   = min(sgmq, 0.025*qw(k))*cldfra_bl1(k)
L03807
L03808              ! The cloud water formulations are taken from CB02, Eq. 8.
L03809  !           maxqc = max(qw(k) - qsat_tk, zero)
L03810  !           if (q1k < zero) then       !unsaturated
L03811  !              !orig: ql_water = sgm(k)*exp(1.2*q1k-one)
L03812  !              !orig: ql_ice   = sgm(k)*exp(1.2*q1k-one)
L03813  !              ql_water = min(sgm(k),0.03*qw(k))*cldfra_bl1(k)
L03814  !              ql_ice   = min(sgm(k),0.03*qw(k))*cldfra_bl1(k)
L03815  !           elseif (q1k > 2.) then !supersaturated
L03816  !              ql_water = min(sgm(k)*q1k, maxqc)
L03817  !              ql_ice   =     sgm(k)*q1k
L03818  !           else                      !slightly saturated (0 > q1 < 2)
L03819  !              !orig: ql_water = min(sgm(k)*(exp(-1.) + 0.66*q1k + 0.086*q1k**2), maxqc)
L03820  !              !orig: ql_ice   =     sgm(k)*(exp(-1.) + 0.66*q1k + 0.086*q1k**2)
L03821  !              ql_water = min(sgm(k),0.02*qw(k))*(exp(-1.) + 0.66*q1k + 0.086*q1k**2)
L03822  !              ql_ice   = min(sgm(k),0.02*qw(k))*(exp(-1.) + 0.66*q1k + 0.086*q1k**2)
L03823  !           endif
L03824
L03825              !In saturated grid cells, use average of SGS and resolved values
L03826              !if ( qc(k) > 1.e-6 ) ql_water = 0.5 * ( ql_water + qc(k) )
```

```
L03827            !ql_ice is actually the total frozen condensate (snow+ice),
L03828            !if ( (qi(k)+qs(k)) > 1.e-9 ) ql_ice = 0.5 * ( ql_ice + (qi(k)+qs(k)) )
L03829
L03830            if (cldfra_bl1(k) < 0.001) then
L03831               ql_ice       = zero
L03832               ql_water     = zero
L03833               cldfra_bl1(k) = zero
L03834            endif
L03835
L03836            liq_frac = MIN(one, MAX(zero, (t-tice)/(tliq-tice)))
L03837            qc_bl1(k) = liq_frac*ql_water       ! apply liq_frac to ql_water and ql_ice
L03838            qi_bl1(k) = (one-liq_frac)*ql_ice
L03839
L03840            !Above tropopause:  eliminate subgrid clouds from CB scheme. Note that this was
L03841            !"k_tropo - 1" as of 20 Feb 2023. Changed to allow more high-level clouds.
L03842            if (k .ge. k_tropo) then
L03843               cldfra_bl1(K) = zero
L03844               qc_bl1(k)     = zero
L03845               qi_bl1(k)     = zero
L03846            endif
L03847
L03848            !Buoyancy-flux-related calculations follow...
L03849            !limiting Q1 to avoid too much diffusion in cloud layers
L03850            !when using the buoyancy-flux functions.
L03851            !q1k=max(Q1(k),-2.0)
L03852            if ((xland-1.5).GE.zero) then  ! water
L03853               q1k=max(Q1(k),-2.5)
L03854            else                           ! land
L03855               q1k=max(Q1(k),-2.0)
L03856            endif
L03857            ! "Fng" represents the non-Gaussian transport factor
L03858            ! (non-dimensional) from Bechtold and Siebesma (1998, JAS)
L03859            if (q1k .ge. one) then
L03860               Fng = one
L03861            elseif (q1k .ge. -1.7 .and. q1k .lt. one) then
L03862               Fng = exp(-0.4*(q1k-one))
L03863            elseif (q1k .ge. -2.5 .and. q1k .lt. -1.7) then
L03864               Fng = three + exp(-3.8*(q1k+1.7))
L03865            else
L03866               Fng = min(23.9 + exp(-1.6*(q1k+2.5)), 60._kind_phys)
L03867            endif
L03868
L03869            cfmax = min(cldfra_bl1(k), 0.6_kind_phys)
L03870            !Further limit the cf going into vt & vq near the surface
L03871            zsl   = min(max(25., p1*pblh2), hundred)
L03872            wt    = min(zagl/zsl, one) !=0 at z=0 m, =1 above ekman layer
L03873            cfmax = cfmax*wt
L03874
L03875            bb = b(k)*t/th(k) ! bb is "b" in BCMT95.  Their "b" differs from
L03876                              ! "b" in CB02 (i.e., b(k) above) by a factor
L03877                              ! of T/theta.  Strictly, b(k) above is formulated in
L03878                              ! terms of sat. mixing ratio, but bb in BCMT95 is
L03879                              ! cast in terms of sat. specific humidity.  The
L03880                              ! conversion is neglected here.
L03881            qww   = one + p608*qw(k)
L03882            alpha = p608*th(k)
L03883            beta  = (th(k)/t)*(xl/cp) - 1.61*th(k)
L03884            vt(k) = qww   - cfmax*beta*bb*Fng   - one
L03885            vq(k) = alpha + cfmax*beta*a(k)*Fng - tv0
L03886            ! vt and vq correspond to beta-theta and beta-q, respectively,
L03887            ! in NN09, Eq. B8.  They also correspond to the bracketed
L03888            ! expressions in BCMT95, Eq. 15, since (s*ql/sigma^2) = cldfra*Fng
L03889            ! The "-1" and "-tv0" terms are included for consistency with
L03890            ! the legacy vt and vq formulations (above).
L03891          enddo
L03892
L03893        END SELECT !end cloudPDF option
L03894
L03895        !For testing purposes only, option for isolating on the mass-flux clouds.
L03896        IF (bl_mynn_cloudpdf .LT. 0) THEN
L03897           DO k = kts,kte-1
L03898              cldfra_bl1(k) = zero
L03899              qc_bl1(k)     = zero
L03900              qi_bl1(k)     = zero
L03901           END DO
L03902        ENDIF
L03903
L03904        ql(kte)        = ql(kte-1)
L03905        vt(kte)        = vt(kte-1)
L03906        vq(kte)        = vq(kte-1)
L03907        qc_bl1(kte)    = zero
L03908        qi_bl1(kte)    = zero
L03909        cldfra_bl1(kte)= zero
L03910      RETURN
L03911
L03912  #ifdef HARDCODE_VERTICAL
L03913  # undef kts
L03914  # undef kte
L03915  #endif
L03916
L03917    END SUBROUTINE mym_condensation
```

CASE 2（−2，默认）：Chaboureau & Bechtold (2002) 诊断统计方案 + 高阶矩 σ—— - **σ 三段口径**（closure<2.7/2.7–3.0/≥3.0 递进加 tsq/cov 贡献）、粗网格膨胀（dz>100 m 时 +20%）；- **分层下限**：`clim`（表面层 1%→PBL 2%→ 自由大气 3% 的 qw 份额）保证云量不至于消失；- **冰云保护**（L3747–L3765）：PBL 之上冰/雪 > 1e-10 或云水 > 1e-5 时，用 `rh_adj = rhcrit + wt • 系数 • log10(q)` 反推 q1 下限——**有冰晶在就给云量**（卷云/砧云的次网格表达）；- **云量公式**：PBL 内 `0.5+0.35 • atan(3.6 • (q1+0.05))`、之上 `0.5+0.39 • atan(1.6 • (q1+0.55))`（**源码注释里保留了四代试错版本**——RRFSv1/exp1/Wayne LES fit，参数化调优的化石层）；- **非高斯因子 Fng**（Bechtold & Siebesma 1998，L3857–L3867）：q1 < 1 时 `Fng = exp(−0.4(q1−1))` 起爆、q1 < −2.5 时到 60——**强欠饱和时云内凝结的非高斯增强**（nc 云量小但 ql 不为零的机制）；- `cfmax ≤ 0.6` 且表面层线性衰减（雾

滴沉积在别处处理的口径）；- `bl_mynn_cloudpdf < 0` 的负值选项：**只留质量通量云、清零次网格云**——分离两套云机制的实验开关。

09a.12 改造入口（→ 教材 ch20/ch21 案索引）

想改什么	动哪里	注意
混合长/扩散强度	`mym_length` 三 CASE 的 alp/cns 系数	常数注释自承“最重要”
稳定度函数修正	`mym_level2` 闭式 + CKmod	牵动 c2/c3 常数族
湿静力稳定度	`mym_level2` case 1 的 clam/lambda	水陆/飓风分支
次网格云	`mym_condensation` CASE 2 的 atan 系数	注释保留四代版本可考古
逆梯度扩散	`mym_turbulence` L3 段 gamt/gamq	closure≥3.0 才生效
闭合阶数实验	`bl_mynn_closure`（2.5/2.6/2.7/3.0）	`mym_predict` 分支矩阵
集合扰动	SPP 段（`mym_turbulence` 末尾）	pattern 归一化
教材案	T2 峰值修正/虚拟温度阈值/BEM 反照率的 ch20 案均与此链无关；边界层改造案见 ch21	—

09a.13 上下游链接

- **下游（本模块内）**：code_09b——`mynn_tendencies`（倾向驱动）、`DMP_mf/ddmp_mf`（质量通量）、`GET_PBLH`、`tridiag*` 求解器、`topdown_cloudrad`、blend 函数族。
- **上游**：code_22——`module_pbl_driver.F`（CASE 分发与逐列装配）；伴生件 `phys/MYNN-EDMF/module_bl_mynnedmf_common.F`（常数包）不在文库单列（C 级，头注释见 code_09a 引用处）。
- **教材**：ch14（MYNN-EDMF 精读、PBLH 三型判据、BouLac 亲缘、qke=2TKE 口径）、ch13 §13.2–13.3（位温/液水位温/等效位温的热力学基础）、ch21（solve_em 调用位）。

文库 code_09b：module_bl_mynnedmf.F90 下半——倾向驱动、质量通量与辅助函数族

09b.0 版本指纹与阅读说明

- **源文件**：`phys/MYNN-EDMF/module_bl_mynnedmf.F90`（WRF 4.8.0），本篇覆盖 L3923–L8776——`mynn_tendencies` 倾向驱动（1574 行）、负值修复、化学/标量混合、三对角求解器族、`GET_PBLH`、上升流谱积分 `DMP_mf`（1345 行）、下沉流 `ddmp_mf`（546 行）、灰区函数 `SCALE_AWARE`、饱和混合函数族、新相似函数 `phim/phih`、云顶辐射冷却解析源 `topdown_cloudrad`。模块声明与闭合主体见 code_09a。
- **保真承诺**：同 code_09a（v1.1 行号显示层，剥离后与源逐位一致）。
- **教材锚点**：ch14（MYNN-EDMF 精读、dmp/ddmp_mf、get_pblh 全文、BouLac 亲缘）、ch21（倾向回写主环）。

09b.1 `mynn_tendencies`：九变量倾向的三对角装配

主入口（code_09a §09a.4.8）把扩散系数（dfm/dfh）、逆梯度项（tcd/qcd）、质量通量通量（s_aw/*sd_aw*）与环境修正（sub_*/det_*）备齐后，`mynn_tendencies` 逐变量解三对角方程——**每个变量一段完全同构的 a/b/c/d 装配**，读完 u 的方程其余就是复制。分段走读：

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L3923-L4190
L03923    SUBROUTINE mynn_tendencies(kts,kte,i,       &
L03924         &delt,dz,zw,xland,pblh,rho,            &
L03925         &u,v,th,tk,qv,qc,qi,qs,qnc,qni,        &
L03926         &psfc,p,exner,                         &
L03927         &thl,sqv,sqc,sqi,sqs,sqw,              &
L03928         &thl_tot1,qc_tot1,qi_tot1,             &
L03929         &qnwfa,qnifa,qnbca,ozone,              &
L03930         &ust,flt,flq,flqv,flqc,wspd,           &
L03931         &uoce,voce,                            &
L03932         &tsq,qsq,cov,                          &
L03933         &tcd,qcd,                              &
L03934         &dfm,dfh,dfq,                          &
L03935         &Du,Dv,Dth,Dqv,Dqc,Dqi,Dqs,Dqnc,Dqni,  &
L03936         &Dqnwfa,Dqnifa,Dqnbca,Dozone,          &
L03937         &diss_heat,                            &
L03938         &s_aw1,s_awthl1,                       &
L03939         &s_awqt1,s_awqv1,s_awqc1,              &
L03940         &s_awu1,s_awv1,                        &
L03941         &s_awqnc1,s_awqni1,                    &
L03942         &s_awqnwfa1,s_awqnifa1,s_awqnbca1,     &
L03943         &sd_aw1,sd_awthl1,sd_awqt1,sd_awqv1,   &
L03944         &sd_awqc1,sd_awqi1,                    &
L03945         &sd_awqnc1,sd_awqni1,                  &
L03946         &sd_awqnwfa1,sd_awqnifa1,              &
L03947         &sd_awu1,sd_awv1,                      &
L03948         &sub_thl,sub_sqv,                      &
L03949         &sub_u,sub_v,                          &
L03950         &det_thl,det_sqv,det_sqc,              &
L03951         &det_u,det_v,                          &
L03952         &FLAG_QC,FLAG_QI,FLAG_QNC,FLAG_QNI,    &
L03953         &FLAG_QS,                              &
L03954         &FLAG_QNWFA,FLAG_QNIFA,FLAG_QNBCA,     &
L03955         &FLAG_OZONE,                           &
L03956         &cldfra_bl1,                           &
L03957         &bl_mynn_cloudmix,                     &
L03958         &bl_mynn_mixqt,                        &
L03959         &bl_mynn_edmf,                         &
L03960         &bl_mynn_edmf_mom,                     &
L03961         &bl_mynn_mixscalars,                   &
L03962         &bl_mynn_mixaerosols,                  &
L03963         &bl_mynn_mixnumcon,                    &
L03964         &upwind                                )
L03965
L03966  !-------------------------------------------------------------------
L03967      use module_bl_mynnedmf_common, only: r_d,p608,grav,xlvcp,xlscp,       &
L03968           wfa_max,wfa_min,ifa_max,ifa_min,wfa_ht,ifa_ht,zero,one,          &
L03969           p333,p5,kind_phys
L03970
L03971      integer, intent(in) :: kts,kte,i
L03972
L03973  #ifdef HARDCODE_VERTICAL
```

```
L03974  # define kts 1
L03975  # define kte HARDCODE_VERTICAL
L03976  #endif
L03977
L03978       integer, intent(in) :: bl_mynn_cloudmix,bl_mynn_mixqt,                &
L03979                              bl_mynn_edmf,bl_mynn_edmf_mom,                 &
L03980                              bl_mynn_mixscalars,bl_mynn_mixaerosols,        &
L03981                              bl_mynn_mixnumcon
L03982       logical, intent(in) :: FLAG_QI,FLAG_QNI,FLAG_QC,FLAG_QS,              &
L03983            &FLAG_QNC,FLAG_QNWFA,FLAG_QNIFA,FLAG_QNBCA,FLAG_OZONE
L03984
L03985  ! thl - liquid water potential temperature
L03986  ! qw - total water
L03987  ! dfm,dfh,dfq - diffusivities i.e., dfh(k) = elq*sh(k) / dzk
L03988  ! flt - surface flux of thl
L03989  ! flq - surface flux of qw
L03990
L03991  ! mass-flux plumes
L03992       real(kind_phys), dimension(kts:kte+1), intent(in) :: s_aw1,            &
L03993            &s_awthl1,s_awqt1,s_awqnc1,s_awqni1,s_awqv1,s_awqc1,s_awu1,s_awv1,&
L03994            &s_awqnwfa1,s_awqnifa1,s_awqnbca1,                             &
L03995            &sd_aw1,sd_awthl1,sd_awqt1,sd_awqv1,sd_awqc1,sd_awqi1,         &
L03996            &sd_awqnc1,sd_awqni1,sd_awqnwfa1,sd_awqnifa1,sd_awu1,sd_awv1
L03997  ! tendencies from mass-flux environmental subsidence and detrainment
L03998       real(kind_phys), dimension(kts:kte), intent(in) :: sub_thl,sub_sqv,   &
L03999            &sub_u,sub_v,det_thl,det_sqv,det_sqc,det_u,det_v
L04000       real(kind_phys), dimension(kts:kte), intent(in) :: u,v,th,tk,qv,qc,qi,&
L04001            &qs,qni,qnc,rho,p,exner,dfq,dz,zw,tsq,qsq,cov,tcd,qcd,          &
L04002            &cldfra_bl1,diss_heat,thl_tot1,qc_tot1,qi_tot1
L04003       real(kind_phys), dimension(kts:kte), intent(inout) :: thl,sqw,sqv,sqc,&
L04004            &sqi,sqs,qnwfa,qnifa,qnbca,ozone,dfm,dfh
L04005       real(kind_phys), dimension(kts:kte), intent(inout) :: du,dv,dth,dqv,  &
L04006            &dqc,dqi,dqs,dqni,dqnc,dqnwfa,dqnifa,dqnbca,dozone
L04007       real(kind_phys), intent(in) :: flt,flq,flqv,flqc,uoce,voce
L04008       real(kind_phys), intent(in) :: ust,delt,psfc,wspd,xland,pblh,upwind
L04009       !debugging
L04010       real(kind_phys):: wsp,wsp2,tk2,th2
L04011       logical :: problem
L04012       integer :: kproblem
L04013
L04014  !    real(kind_phys), intent(in) :: gradu_top,gradv_top,gradth_top,gradqv_top
L04015
L04016  !local vars
L04017
L04018       real(kind_phys), dimension(kts:kte) :: dtz,dfhc,dfmc,delp
L04019       real(kind_phys), dimension(kts:kte) :: sqv2,sqc2,sqi2,sqs2,sqw2,      &
L04020            &qni2,qnc2,qnwfa2,qnifa2,qnbca2,ozone2
L04021       real(kind_phys), dimension(kts:kte) :: zfac,plumeKh,rhoinv
L04022       real(kind_phys), dimension(kts:kte) :: upcont,dncont ! updraft/downdraft contribution to fluxes for explicit
↪  calculation
L04023       real(kind_phys), dimension(kts:kte) :: a,b,c,d,x
L04024       real(kind_phys), dimension(kts:kte+1) :: rhoz,                        & !rho on model interface
L04025            &khdz,kmdz
L04026       real(kind_phys):: rhs,gfluxm,gfluxp,dztop,maxdfh,mindfh,maxcf,maxKh
L04027       real(kind_phys):: t,esat,qsl,onoff,kh,km,dzk,rhosfc,ustovwsp
L04028       real(kind_phys):: qvflux,aero_min,aero_max
L04029       real(kind_phys):: th_new,portion_qc,portion_qi,condensate,qsat
L04030       integer :: k,kk
L04031
L04032       !Activate nonlocal mixing from the mass-flux scheme for
L04033       !number concentrations and aerosols (0.0 = no; 1.0 = yes)
L04034       real(kind_phys), parameter :: nonloc  = 1.0
L04035       real(kind_phys), parameter :: nc_min  = 100.0
L04036       real(kind_phys), parameter :: ni_min  = 1e-6
L04037       real(kind_phys) :: wfa_max2, wt
L04038
L04039       dztop=p5*(dz(kte)+dz(kte-1))
L04040
L04041       ! REGULATE THE MOMENTUM MIXING FROM THE MASS-FLUX SCHEME (on or off)
L04042       ! Note that s_awu and s_awv already come in as 0.0 if bl_mynn_edmf_mom == 0, so
L04043       ! we only need to zero-out the MF term
L04044       IF (bl_mynn_edmf_mom == 0) THEN
L04045          onoff=zero
L04046       ELSE
L04047          onoff=one
L04048       ENDIF
L04049
L04050       ! USTAR/WSPD make sure it does not blow up when WSPD = 0.
L04051       IF (wspd .le. 1e-6) THEN
L04052          ustovwsp=zero
L04053       ELSE
L04054          ustovwsp=ust/wspd
L04055       ENDIF
L04056
L04057       !Prepare "constants" for diffusion equation.
L04058       !khdz = rho*Kh/dz = rho*dfh
L04059       rhosfc     = psfc/(R_d*(tk(kts)+p608*qv(kts)))
L04060       dtz(kts)   = delt/dz(kts)
L04061       rhoz(kts)  = rho(kts)
L04062       rhoinv(kts)= one/rho(kts)
L04063       khdz(kts)  = rhoz(kts)*dfh(kts)
L04064       kmdz(kts)  = rhoz(kts)*dfm(kts)
L04065       DO k=kts+1,kte
L04066          dtz(k)   = delt/dz(k)
L04067          rhoz(k)  = (rho(k)*dz(k-1) + rho(k-1)*dz(k))/(dz(k-1)+dz(k))
L04068          rhoz(k)  =  MAX(rhoz(k),1E-4_kind_phys)
L04069          rhoinv(k)= one/MAX(rho(k),1E-4_kind_phys)
L04070          dzk      = p5 *( dz(k)+dz(k-1) )
L04071          khdz(k)  = rhoz(k)*dfh(k)
L04072          kmdz(k)  = rhoz(k)*dfm(k)
L04073       ENDDO
L04074       rhoz(kte+1)= rhoz(kte)
L04075       khdz(kte+1)= rhoz(kte+1)*dfh(kte)
L04076       kmdz(kte+1)= rhoz(kte+1)*dfm(kte)
L04077
L04078       !delta-p for the moisture check
L04079       !delp(kts)  = psfc - (p(kts+1)*dz(kts) + p(kts)*dz(kts+1))/(dz(kts)+dz(kts+1))
L04080       DO k=kts,kte !kts+1,kte-1
```

```
L04081         !delp(k)  = (p(k)*dz(k-1) + p(k-1)*dz(k))/(dz(k)+dz(k-1)) - &
L04082         !           (p(k+1)*dz(k) + p(k)*dz(k+1))/(dz(k)+dz(k+1))
L04083         delp(k) = rho(k)*grav*dz(k)
L04084      ENDDO
L04085      !delp(kte)  =delp(kte-1)
L04086      if ( delp(kts) < p333*delp(kts+1) )delp(kts)=p333*delp(kts+1)
L04087
L04088      !stability criteria for implicit mf
L04089      if (bl_mynn_edmf == 1) then
L04090         DO k=kts+1,kte-1
L04091            khdz(k) = max(khdz(k),  p5*(s_aw1(k) +sd_aw1(k)))
L04092            khdz(k) = max(khdz(k), -p5*(s_aw1(k) -s_aw1(k+1))  &
L04093                                   -p5*(sd_aw1(k)-sd_aw1(k+1)) )
L04094            kmdz(k) = max(kmdz(k),  p5*(s_aw1(k) +sd_aw1(k)))
L04095            kmdz(k) = max(kmdz(k), -p5*(s_aw1(k) -s_aw1(k+1))  &
L04096                                   -p5*(sd_aw1(k)-sd_aw1(k+1)) )
L04097         ENDDO
L04098      endif
L04099
L04100      dth(kts:kte) = zero  ! must initialize for moisture_check routine
L04101
L04102   !!=========================================
L04103   !! u
L04104   !!=========================================
L04105    if (bl_mynn_edmf > 1) then
L04106
L04107       do k=kts+1,kte-1
L04108          upcont(k)=onoff*(s_awu1(k) - s_aw1(k)*(u(k)*upwind+u(k-1)*(one-upwind)))
L04109          dncont(k)=onoff*(sd_awu1(k)-sd_aw1(k)*(u(k)*upwind+u(k-1)*(one-upwind)))
L04110       enddo
L04111       ! no flux at the top of the atmosphere
L04112       upcont(kte)=zero
L04113       dncont(kte)=zero
L04114       ! upcont(1) and dncont(1) are not used so they don't need to be set
L04115
L04116       k=kts
L04117       a(1)=zero
L04118       b(1)=one + dtz(k)*(kmdz(k+1)+rhosfc*ust*ustovwsp)*rhoinv(k)
L04119       c(1)=    - dtz(k)*kmdz(k+1)*rhoinv(k)
L04120       d(1)=u(k)+ dtz(k)*uoce*ust**2/wspd*rho(k)                      &
L04121                - dtz(k)*(upcont(k+1)+dncont(k+1))
L04122
L04123       DO k=kts+1,kte-1
L04124          a(k)=   - dtz(k)*kmdz(k)*rhoinv(k)
L04125          b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k)
L04126          c(k)=   - dtz(k)*kmdz(k+1)*rhoinv(k)
L04127          d(k)=u(k) -dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
L04128             &  + sub_u(k)*delt + det_u(k)*delt
L04129       ENDDO
L04130
L04131    else !implicit
L04132
L04133       k=kts
L04134       !rho-weighted (drag in b-vector):
L04135       a(k)=  -dtz(k)*kmdz(k)*rhoinv(k)
L04136       b(k)=one+dtz(k)*(kmdz(k+1)+rhosfc*ust*ustovwsp)*rhoinv(k)      &
L04137              & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff                &
L04138              & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04139       c(k)=  -dtz(k)*kmdz(k+1)*rhoinv(k)                             &
L04140              & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff                &
L04141              & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04142       d(k)=u(k)  + dtz(k)*uoce*ust**2/wspd                           &
L04143              & - dtz(k)*rhoinv(k)*s_awu1(k+1)*onoff                  &
L04144              & + dtz(k)*rhoinv(k)*sd_awu1(k+1)*onoff                 &
L04145              & + sub_u(k)*delt + det_u(k)*delt
L04146
L04147       do k=kts+1,kte-1
L04148          a(k)=  -dtz(k)*kmdz(k)*rhoinv(k)                            &
L04149              &  + p5*dtz(k)*rhoinv(k)*s_aw1(k)*onoff                 &
L04150              &  + p5*dtz(k)*rhoinv(k)*sd_aw1(k)*onoff
L04151          b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k)              &
L04152              &  + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))*onoff    &
L04153              &  + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))*onoff
L04154          c(k)=  - dtz(k)*kmdz(k+1)*rhoinv(k)                         &
L04155              &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff               &
L04156              &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04157          d(k)=u(k) + dtz(k)*rhoinv(k)*(s_awu1(k)-s_awu1(k+1))*onoff  &
L04158              &  - dtz(k)*rhoinv(k)*(sd_awu1(k)-sd_awu1(k+1))*onoff   &
L04159              &  + sub_u(k)*delt + det_u(k)*delt
L04160       enddo
L04161
L04162    endif
L04163
L04164   !! no flux at the top
L04165   !    a(kte)=-1.
L04166   !    b(kte)=1.
L04167   !    c(kte)=0.
L04168   !    d(kte)=0.
L04169
L04170   !! specified gradient at the top
L04171   !    a(kte)=-1.
L04172   !    b(kte)=1.
L04173   !    c(kte)=0.
L04174   !    d(kte)=gradu_top*dztop
L04175
L04176   !! prescribed value
L04177       a(kte)=zero
L04178       b(kte)=one
L04179       c(kte)=zero
L04180       d(kte)=u(kte)
L04181
L04182   !    CALL tridiag(kte,a,b,c,d)
L04183       CALL tridiag2(kte,a,b,c,d,x)
L04184   !    CALL tridiag3(kte,a,b,c,d,x)
L04185
L04186       DO k=kts,kte
L04187   !       du(k)=(d(k-kts+1)-u(k))/delt
L04188          du(k)=(x(k)-u(k))/delt
```

```
L04189      ENDDO
L04190
```

- **准备段**：`khdz/kmdz = ρ·K/dz` 界面通量系数（与 `mym_predict` 同构）；`delp = ρ·g·dz`（层气压厚度，负值修复用）；底层 `delp(kts) < (1/3)·delp(kts+1)` 时抬高——**表层过薄时限制修复强度**。
- **u 方程的两种耦合**（`bl_mynn_edmf`）：
  - **显式 MF**（>1）：MF 输运进右端 `upcont/dncont = s_awu − s_aw·(u 上风插值)`；
  - **隐式 MF**（=1）：MF 进左端 a/b/c（`±0.5·dtz·s_aw`）与右端（`(s_awu(k)−s_awu(k+1))`）——**与 TKE 方程同一数学形态**。
- **表面边界**：`b(kts)` 含 `rhosfc·u*²/wspd`——**摩擦拖曳直接进对角元**（bulk 公式的隐式化）；右端 `uoce·u*²/wspd` 是**洋流修正的海面应力**（海流不抵消风应力）。`sub_u/det_u·delt` 是环境下沉/夹带倾向。
- **顶边界**：prescribed value（`d(kte)=u(kte)`——不混合顶层）；被注释的 no-flux/指定梯度两种历史方案。
- **`onoff = bl_mynn_edmf_mom`**：MF 动量输运总开关（L4041 注释：s_awu 已在 DMP_mf 里按开关清零，这里只是双保险）。
- 解出 `x(k)` 后 `du = (x−u)/delt`——**WRF 的物理层只交倾向，不交新状态**。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L4191-L4462
L04191  !!==========================================
L04192  !! v
L04193  !!==========================================
L04194
L04195   if (bl_mynn_edmf > 1) then
L04196
L04197      DO k=kts+1,kte-1
L04198         upcont(k)=onoff*(s_awv1(k)-s_aw1(k)*(v(k)*upwind+v(k-1)*(one-upwind)))
L04199         dncont(k)=onoff*(sd_awv1(k)-sd_aw1(k)*(v(k)*upwind+v(k-1)*(one-upwind)))
L04200      ENDDO
L04201      ! no flux at the top of the atmosphere
L04202      upcont(kte)=0.
L04203      dncont(kte)=0.
L04204      ! upcont(1) and dncont(1) are not used so they don't need to be set
L04205
L04206      k=kts
L04207      a(1)=zero
L04208      b(1)=one + dtz(k)*(kmdz(k+1)+rhosfc*ust*ustovwsp)*rhoinv(k)
L04209      c(1)=    - dtz(k)*kmdz(k+1)*rhoinv(k)
L04210      d(1)=v(k)+ dtz(k)*voce*ust**2/wspd*rho(k)                    &
L04211               - dtz(k)*(upcont(k+1)+dncont(k+1))
L04212
L04213      DO k=kts+1,kte-1
L04214         a(k)=    - dtz(k)*kmdz(k)*rhoinv(k)
L04215         b(k)=one+ dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k)
L04216         c(k)=    - dtz(k)*kmdz(k+1)*rhoinv(k)
L04217         d(k)=v(k) -dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
L04218            &  + sub_v(k)*delt + det_v(k)*delt
L04219      ENDDO
L04220
L04221   else !implicit
L04222
L04223      k=kts
L04224      !rho-weighted (drag in b-vector):
L04225      a(k)=  -dtz(k)*kmdz(k)*rhoinv(k)
L04226      b(k)=one+dtz(k)*(kmdz(k+1) + rhosfc*ust**2/wspd)*rhoinv(k)    &
L04227          &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff                 &
L04228          &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04229      c(k)=  -dtz(k)*kmdz(k+1)*rhoinv(k)                            &
L04230          &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff                 &
L04231          &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04232      d(k)=v(k)  + dtz(k)*voce*ust**2/wspd                          &
L04233          &  - dtz(k)*rhoinv(k)*s_awv1(k+1)*onoff                   &
L04234          &  + dtz(k)*rhoinv(k)*sd_awv1(k+1)*onoff                  &
L04235          &  + sub_v(k)*delt + det_v(k)*delt
L04236
L04237      do k=kts+1,kte-1
L04238         a(k)=  -dtz(k)*kmdz(k)*rhoinv(k)                           &
L04239           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)*onoff                  &
L04240           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)*onoff
L04241         b(k)=one+dtz(k)*(kmdz(k)+kmdz(k+1))*rhoinv(k)              &
L04242           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))*onoff      &
L04243           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))*onoff
L04244         c(k)=  -dtz(k)*kmdz(k+1)*rhoinv(k)                         &
L04245           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*onoff                 &
L04246           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)*onoff
L04247         d(k)=v(k) + dtz(k)*rhoinv(k)*(s_awv1(k)-s_awv1(k+1))*onoff &
L04248           & - dtz(k)*rhoinv(k)*(sd_awv1(k)-sd_awv1(k+1))*onoff     &
L04249           & + sub_v(k)*delt + det_v(k)*delt
L04250      enddo
L04251   endif
L04252  !! no flux at the top
L04253  !    a(kte)=-1.
L04254  !    b(kte)=one
L04255  !    c(kte)=zero
L04256  !    d(kte)=zero
L04257
L04258  !! specified gradient at the top
L04259  !    a(kte)=-1.
L04260  !    b(kte)=one
L04261  !    c(kte)=zero
L04262  !    d(kte)=gradv_top*dztop
L04263
```

```
!! prescribed value
    a(kte)=zero
    b(kte)=one
    c(kte)=zero
    d(kte)=v(kte)

!    CALL tridiag(kte,a,b,c,d)
    CALL tridiag2(kte,a,b,c,d,x)
!    CALL tridiag3(kte,a,b,c,d,x)

    DO k=kts,kte
!       dv(k)=(d(k-kts+1)-v(k))/delt
       dv(k)=(x(k)-v(k))/delt
    ENDDO

!!=========================================
!! thl tendency
!!=========================================
 if (bl_mynn_edmf > 1) then

    DO k=kts+1,kte-1
       upcont(k)= s_awthl1(k)- s_aw1(k)*(thl(k)*upwind+thl(k-1)*(one-upwind))
       dncont(k)=sd_awthl1(k)-sd_aw1(k)*(thl(k)*upwind+thl(k-1)*(one-upwind))
    ENDDO
    ! no flux at the top of the atmosphere
    upcont(kte)=zero
    dncont(kte)=zero
    ! upcont(1) and dncont(1) are not used so they don't need to be set

    k=kts
    a(1)=zero
    b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
    c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
    d(1)=thl(k)+dtz(k)*rhosfc*flt*rhoinv(k) + tcd(k)*delt             &
         - dtz(k)*(upcont(k+1)+dncont(k+1))                           &
         + diss_heat(k)*delt + sub_thl(k)*delt + det_thl(k)*delt

    DO k=kts+1,kte-1
       a(k)=    - dtz(k)*khdz(k)*rhoinv(k)
       b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
       c(k)=    - dtz(k)*khdz(k+1)*rhoinv(k)
       d(k)=thl(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
          &  + diss_heat(k)*delt + sub_thl(k)*delt + det_thl(k)*delt
    ENDDO

 else !implicit

    k=kts
!rho-weighted: rhosfc*x*rhoinv(k)
    a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
    b(k)=1.+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)            &
       &    - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
       &    - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
    c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                      &
       &    - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
       &    - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
    d(k)=thl(k) + dtz(k)*rhosfc*flt*rhoinv(k) + tcd(k)*delt  &
!    d(k)=thl_tot1(k) + dtz(k)*rhosfc*flt*rhoinv(k) + tcd(k)*delt  &
       &    - dtz(k)*rhoinv(k)*s_awthl1(k+1)                &
       &    + dtz(k)*rhoinv(k)*sd_awthl1(k+1)               &
       &    + diss_heat(k)*delt + sub_thl(k)*delt + det_thl(k)*delt

    do k=kts+1,kte-1
       a(k)= -dtz(k)*khdz(k)*rhoinv(k)                      &
       &    + p5*dtz(k)*rhoinv(k)*s_aw1(k)                  &
       &    + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
       b(k)=1.+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)         &
       &    + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))     &
       &    + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
       c(k)= -dtz(k)*khdz(k+1)*rhoinv(k)                    &
       &    - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
       &    - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
       d(k)=thl(k) + tcd(k)*delt                            &
!       d(k)=thl_tot1(k) + tcd(k)*delt                              &
       &    + dtz(k)*rhoinv(k)*(s_awthl1(k)-s_awthl1(k+1))  &
       &    - dtz(k)*rhoinv(k)*(sd_awthl1(k)-sd_awthl1(k+1))  &
       &    +     diss_heat(k)*delt                         &
       &    +     sub_thl(k)*delt + det_thl(k)*delt
    enddo
 endif
!! no flux at the top
!    a(kte)=-1.
!    b(kte)=1.
!    c(kte)=0.
!    d(kte)=0.

!! specified gradient at the top
!assume gradthl_top=gradth_top
!    a(kte)=-1.
!    b(kte)=1.
!    c(kte)=0.
!    d(kte)=gradth_top*dztop

!! prescribed value
    a(kte)=zero
    b(kte)=one
    c(kte)=zero
    d(kte)=thl(kte)

!    CALL tridiag(kte,a,b,c,d)
    CALL tridiag2(kte,a,b,c,d,x)
!    CALL tridiag3(kte,a,b,c,d,x)

    DO k=kts,kte
       !thl(k)=d(k-kts+1)
       thl(k)=x(k)
    ENDDO

```

```
L04372  IF (bl_mynn_mixqt > 0) THEN
L04373   !=========================================
L04374   ! MIX total water (sqw = sqc + sqv + sqi)
L04375   ! NOTE: no total water tendency is output; instead, we must calculate
L04376   !       the saturation specific humidity and then
L04377   !       subtract out the moisture excess (sqc & sqi)
L04378   !=========================================
L04379   if (bl_mynn_edmf > 1) then
L04380
L04381      DO k=kts+1,kte-1
L04382         upcont(k)= s_awqt1(k)- s_aw1(k)*(sqw(k)*upwind+sqw(k-1)*(one-upwind))
L04383         dncont(k)=sd_awqt1(k)-sd_aw1(k)*(sqw(k)*upwind+sqw(k-1)*(one-upwind))
L04384      ENDDO
L04385      ! no flux at the top of the atmosphere
L04386      upcont(kte)=zero
L04387      dncont(kte)=zero
L04388      ! upcont(1) and dncont(1) are not used so they don't need to be set
L04389
L04390      k=kts
L04391      a(1)=zero
L04392      b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04393      c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04394      d(1)=sqw(k)+dtz(k)*rhosfc*flq*rhoinv(k) + qcd(k)*delt              &
L04395           - dtz(k)*(upcont(k+1)+dncont(k+1))                            &
L04396           + sub_sqv(k)*delt + det_sqv(k)*delt
L04397
L04398      DO k=kts+1,kte-1
L04399         a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04400         b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04401         c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04402         d(k)=sqw(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
L04403            &  + sub_sqv(k)*delt + det_sqv(k)*delt
L04404      ENDDO
L04405
L04406    else !implicit
L04407
L04408      k=kts
L04409      !rho-weighted:
L04410      a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04411      b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)         &
L04412         & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                 &
L04413         & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04414      c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                    &
L04415         & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                 &
L04416         & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04417      d(k)=sqw(k)  + dtz(k)*rhosfc*flq*rhoinv(k) + qcd(k)*delt &
L04418         &  - dtz(k)*rhoinv(k)*s_awqt1(k+1)                  &
L04419         &  + dtz(k)*rhoinv(k)*sd_awqt1(k+1)
L04420
L04421      do k=kts+1,kte-1
L04422         a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                   &
L04423         & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                   &
L04424         & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04425         b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)      &
L04426         & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))      &
L04427         & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04428         c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                 &
L04429         & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                 &
L04430         & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04431         d(k)=sqw(k) + qcd(k)*delt                          &
L04432         & + dtz(k)*rhoinv(k)*(s_awqt1(k)-s_awqt1(k+1))     &
L04433         & - dtz(k)*rhoinv(k)*(sd_awqt1(k)-sd_awqt1(k+1))
L04434      enddo
L04435   endif
L04436   !! no flux at the top
L04437   !    a(kte)=-1.
L04438   !    b(kte)=1.
L04439   !    c(kte)=0.
L04440   !    d(kte)=0.
L04441   !! specified gradient at the top
L04442   !assume gradqw_top=gradqv_top
L04443   !    a(kte)=-1.
L04444   !    b(kte)=1.
L04445   !    c(kte)=0.
L04446   !    d(kte)=gradqv_top*dztop
L04447   !! prescribed value
L04448      a(kte)=zero
L04449      b(kte)=one
L04450      c(kte)=zero
L04451      d(kte)=sqw(kte)
L04452
L04453   !    CALL tridiag(kte,a,b,c,d)
L04454      CALL tridiag2(kte,a,b,c,d,sqw2)
L04455   !    CALL tridiag3(kte,a,b,c,d,sqw2)
L04456
L04457   !    DO k=kts,kte
L04458   !       sqw2(k)=d(k-kts+1)
L04459   !    ENDDO
L04460   ELSE
L04461      sqw2=sqw
L04462   ENDIF
```

- **v 方程**：与 u 完全同构（`voce` 洋流 v 分量）。
- **θ_l 方程**：右端多四项——表面通量 `rhosfc * flt`、逆梯度 `tcd * delt`、**TKE 耗散加热 `diss_heat * delt`**（code_09a §09a.4.7 算出的 7.2 K/h 上限量）、环境项 `sub_thl/det_thl`。**θ_l 是被推进的变量**（液水位温），位温趋势稍后由凝结量重建。
- **总水 sqw 方程**（`bl_mynn_mixqt > 0`）：同构三对角解出 `sqw2`——注释明说“不输出总水倾向；稍后从饱和检查里拆出 sqv/sqc/sqi”。**混合总水 vs 分相混合**（mixqt=0 时分别推 sqc/sqv）是该方案水分处理的两条路线：总水路线在饱和拆分处统一凝结构造，物理上更自洽（凝结不改变总水）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L4464-L4780
L04464  IF (bl_mynn_mixqt == 0) THEN
L04465  !=========================================
L04466  ! cloud water ( sqc ). If mixing total water (bl_mynn_mixqt > 0),
L04467  ! then sqc will be backed out of saturation check (below).
L04468  !=========================================
L04469   IF (bl_mynn_cloudmix > 0 .AND. FLAG_QC) THEN
L04470
L04471     if (bl_mynn_edmf > 1) then
L04472
L04473        DO k=kts+1,kte-1
L04474           upcont(k)= s_awqc1(k)- s_aw1(k)*(sqc(k)*upwind+sqc(k-1)*(one-upwind))
L04475           dncont(k)=sd_awqc1(k)-sd_aw1(k)*(sqc(k)*upwind+sqc(k-1)*(one-upwind))
L04476        ENDDO
L04477        ! no flux at the top of the atmosphere
L04478        upcont(kte)=zero
L04479        dncont(kte)=zero
L04480        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04481
L04482        k=kts
L04483        a(1)=zero
L04484        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04485        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04486        d(1)=sqc(k)+dtz(k)*rhosfc*flqc*rhoinv(k) + qcd(k)*delt             &
L04487           &    - dtz(k)*(upcont(k+1)+dncont(k+1))                         &
L04488           &    + det_sqc(k)*delt
L04489
L04490        DO k=kts+1,kte-1
L04491           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04492           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04493           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04494           d(k)=sqc(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
L04495              &  + det_sqc(k)*delt
L04496        ENDDO
L04497
L04498      else !implicit
L04499
L04500         k=kts
L04501         !rho-weighted:
L04502         a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04503         b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)        &
L04504         &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04505         &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04506         c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                   &
L04507         &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04508         &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04509         d(k)=sqc(k)  + dtz(k)*rhosfc*flqc*rhoinv(k) + qcd(k)*delt &
L04510         &  - dtz(k)*rhoinv(k)*s_awqc1(k+1)                    &
L04511         &  + dtz(k)*rhoinv(k)*sd_awqc1(k+1)                   &
L04512         &  + det_sqc(k)*delt
L04513
L04514         do k=kts+1,kte-1
L04515            a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                  &
L04516            & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                  &
L04517            & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04518            b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)     &
L04519            & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))     &
L04520            & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04521            c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                &
L04522            & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
L04523            & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04524            d(k)=sqc(k) + qcd(k)*delt                         &
L04525            & + dtz(k)*rhoinv(k)*(s_awqc1(k)-s_awqc1(k+1))    &
L04526            & - dtz(k)*rhoinv(k)*(sd_awqc1(k)-sd_awqc1(k+1))  &
L04527            & + det_sqc(k)*delt
L04528         enddo
L04529      endif
L04530
L04531  ! prescribed value
L04532     a(kte)=zero
L04533     b(kte)=one
L04534     c(kte)=zero
L04535     d(kte)=sqc(kte)
L04536
L04537  !    CALL tridiag(kte,a,b,c,d)
L04538     CALL tridiag2(kte,a,b,c,d,sqc2)
L04539  !    CALL tridiag3(kte,a,b,c,d,sqc2)
L04540
L04541  !    DO k=kts,kte
L04542  !       sqc2(k)=d(k-kts+1)
L04543  !    ENDDO
L04544    ELSE
L04545      !If not mixing clouds, set "updated" array equal to original array
L04546     sqc2=sqc
L04547    ENDIF
L04548  ENDIF
L04549
L04550  IF (bl_mynn_mixqt == 0) THEN
L04551    !=========================================
L04552    ! MIX WATER VAPOR ONLY ( sqv ). If mixing total water (bl_mynn_mixqt > 0),
L04553    ! then sqv will be backed out of saturation check (below).
L04554    !=========================================
L04555
L04556     k=kts
L04557     !limit unreasonably large negative fluxes:
L04558     qvflux = flqv
L04559     if (qvflux < zero) then
L04560        !do not allow specified surface flux to reduce qv below 1e-8 kg/kg
L04561        qvflux = max(qvflux, (min(0.9*sqv(kts) - 1e-8, zero)/dtz(kts)))
L04562     endif
L04563
L04564     if (bl_mynn_edmf > 1) then
L04565
L04566        DO k=kts+1,kte-1
L04567           upcont(k)= s_awqv1(k)- s_aw1(k)*(sqv(k)*upwind+sqv(k-1)*(one-upwind))
L04568           dncont(k)=sd_awqv1(k)-sd_aw1(k)*(sqv(k)*upwind+sqv(k-1)*(one-upwind))
L04569        ENDDO
L04570        ! no flux at the top of the atmosphere
```

```
L04571        upcont(kte)=zero
L04572        dncont(kte)=zero
L04573        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04574
L04575        k=kts
L04576        a(1)=zero
L04577        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04578        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04579        d(1)=sqv(k)+dtz(k)*rhosfc*qvflux*rhoinv(k) + qcd(k)*delt             &
L04580            &    - dtz(k)*(upcont(k+1)+dncont(k+1))              &
L04581            &    + sub_sqv(k)*delt + det_sqv(k)*delt
L04582
L04583        DO k=kts+1,kte-1
L04584           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04585           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04586           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04587           d(k)=sqv(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k)) &
L04588               &   + sub_sqv(k)*delt + det_sqv(k)*delt
L04589        ENDDO
L04590
L04591     else !implicit
L04592
L04593        k=kts
L04594        !rho-weighted:
L04595        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04596        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)        &
L04597        & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                   &
L04598        & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04599        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                   &
L04600        & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                   &
L04601        & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04602        d(k)=sqv(k)  + dtz(k)*rhosfc*qvflux*rhoinv(k) + qcd(k)*delt &
L04603        & - dtz(k)*rhoinv(k)*s_awqv1(k+1)                    &
L04604        & + dtz(k)*rhoinv(k)*sd_awqv1(k+1)                   &
L04605        & + sub_sqv(k)*delt + det_sqv(k)*delt
L04606
L04607        do k=kts+1,kte-1
L04608           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                  &
L04609           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                  &
L04610           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04611           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)     &
L04612           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))     &
L04613           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04614           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                &
L04615           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
L04616           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04617           d(k)=sqv(k) + qcd(k)*delt                         &
L04618           & + dtz(k)*rhoinv(k)*(s_awqv1(k)-s_awqv1(k+1))    &
L04619           & - dtz(k)*rhoinv(k)*(sd_awqv1(k)-sd_awqv1(k+1))  &
L04620           & + sub_sqv(k)*delt + det_sqv(k)*delt
L04621        enddo
L04622     endif
L04623
L04624  ! no flux at the top
L04625  !    a(kte)=-1.
L04626  !    b(kte)=1.
L04627  !    c(kte)=0.
L04628  !    d(kte)=0.
L04629
L04630  ! specified gradient at the top
L04631  ! assume gradqw_top=gradqv_top
L04632  !    a(kte)=-1.
L04633  !    b(kte)=1.
L04634  !    c(kte)=0.
L04635  !    d(kte)=gradqv_top*dztop
L04636
L04637  ! prescribed value
L04638      a(kte)=zero
L04639      b(kte)=one
L04640      c(kte)=zero
L04641      d(kte)=sqv(kte)
L04642
L04643  !    CALL tridiag(kte,a,b,c,d)
L04644      CALL tridiag2(kte,a,b,c,d,sqv2)
L04645  !    CALL tridiag3(kte,a,b,c,d,sqv2)
L04646
L04647  !    DO k=kts,kte
L04648  !       sqv2(k)=d(k-kts+1)
L04649  !    ENDDO
L04650  ELSE
L04651      sqv2=sqv
L04652  ENDIF
L04653
L04654  !=========================================
L04655  ! MIX CLOUD ICE ( sqi )
L04656  !=========================================
L04657  IF (bl_mynn_cloudmix > 0 .AND. FLAG_QI) THEN
L04658
L04659     if (bl_mynn_edmf > 1) then
L04660
L04661        DO k=kts+1,kte-1
L04662           upcont(k)= zero !s_awqi1(k)- s_aw1(k)*(sqi(k)*upwind+sqi(k-1)*(one-upwind))
L04663           dncont(k)=sd_awqi1(k)-sd_aw1(k)*(sqi(k)*upwind+sqi(k-1)*(one-upwind))
L04664        ENDDO
L04665        ! no flux at the top of the atmosphere
L04666        upcont(kte)=zero
L04667        dncont(kte)=zero
L04668        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04669
L04670        k=kts
L04671        a(1)=zero
L04672        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04673        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04674        d(1)=sqi(k)                                          &
L04675            &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L04676
L04677        DO k=kts+1,kte-1
L04678           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
```

```
L04679           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04680           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04681           d(k)=sqi(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L04682        ENDDO
L04683
L04684     else !implicit
L04685
L04686        k=kts
L04687        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04688        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)      &
L04689  !      &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)               &
L04690        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04691        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                 &
L04692  !      &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)               &
L04693        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04694        d(k)=sqi(k)                                        &
L04695  !      &  - dtz(k)*rhoinv(k)*s_awqi1(k+1)                &
L04696        &  + dtz(k)*rhoinv(k)*sd_awqi1(k+1)
L04697
L04698        do k=kts+1,kte-1
L04699           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                &
L04700  !         & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                &
L04701           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04702           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)   &
L04703  !         & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))  &
L04704           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04705           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)              &
L04706  !         & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)              &
L04707           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04708           d(k)=sqi(k)                                     &
L04709  !         & + dtz(k)*rhoinv(k)*(s_awqi1(k)-s_awqi1(k+1)) &
L04710           & - dtz(k)*rhoinv(k)*(sd_awqi1(k)-sd_awqi1(k+1))
L04711        enddo
L04712     endif
L04713
L04714  !! no flux at the top
L04715  !    a(kte)=-1.
L04716  !    b(kte)=1.
L04717  !    c(kte)=0.
L04718  !    d(kte)=0.
L04719
L04720  !! specified gradient at the top
L04721  !assume gradqw_top=gradqv_top
L04722  !    a(kte)=-1.
L04723  !    b(kte)=1.
L04724  !    c(kte)=0.
L04725  !    d(kte)=gradqv_top*dztop
L04726
L04727  !! prescribed value
L04728      a(kte)=zero
L04729      b(kte)=one
L04730      c(kte)=zero
L04731      d(kte)=sqi(kte)
L04732
L04733  !    CALL tridiag(kte,a,b,c,d)
L04734      CALL tridiag2(kte,a,b,c,d,sqi2)
L04735  !    CALL tridiag3(kte,a,b,c,d,sqi2)
L04736
L04737  !    DO k=kts,kte
L04738  !       sqi2(k)=d(k-kts+1)
L04739  !    ENDDO
L04740  ELSE
L04741     sqi2=sqi
L04742  ENDIF
L04743
L04744  !===========================================
L04745  ! MIX SNOW ( sqs )
L04746  !===========================================
L04747  !hard-code to not mix snow
L04748  IF (bl_mynn_cloudmix > 0 .AND. .false.) THEN
L04749
L04750      k=kts
L04751  !rho-weighted:
L04752      a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04753      b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)
L04754      c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)
L04755      d(k)=sqs(k)
L04756
L04757      DO k=kts+1,kte-1
L04758         a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04759         b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04760         c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)
L04761         d(k)=sqs(k)
L04762      ENDDO
L04763
L04764  !! prescribed value
L04765      a(kte)=zero
L04766      b(kte)=one
L04767      c(kte)=zero
L04768      d(kte)=sqs(kte)
L04769
L04770  !    CALL tridiag(kte,a,b,c,d)
L04771      CALL tridiag2(kte,a,b,c,d,sqs2)
L04772  !    CALL tridiag3(kte,a,b,c,d,sqs2)
L04773
L04774  !    DO k=kts,kte
L04775  !       sqs2(k)=d(k-kts+1)
L04776  !    ENDDO
L04777  ELSE
L04778     sqs2=sqs
L04779  ENDIF
L04780
```

- **sqc 方程**（`bl_mynn_cloudmix > 0 .AND. FLAG_QC` 且 mixqt=0）：右端含 `flqc`（海盐/雾沉降通量，当前恒零）与 `det_sqc`。不混合云时 **`sqc2=sqc` 原样传过**。
- **sqv 方程**：表面通量加了**负通量保护**（L4557–L4562）：`qvflux = max(flqv, (min(0.9`

sqv(kts)−1e-8, 0))/dtz)——**蒸发不许把最底层比湿抽到 1e-8 以下**（每步最多抽走 90%）。

- **sqi 方程**：MF 侧只有下沉流贡献（upcont ≡ 0 被注释清零——**上升流不携带冰**，冰相在上升流中由 condensation_edmf 归零）；implicit 分支里 s_aw 项全被注释、只留 sd_aw——**上升流的冰通量从倾向方程中显式剔除**。
- **sqs（雪）**：IF (bl_mynn_cloudmix > 0 .AND. .false.)——**硬编码关闭**（注释 “hard-code to not mix snow”）。边界层不混合雪：与 sqw1 定义（不含雪）一致的方案级决策。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L4782-L5250
L04782  !! cloud ice number concentration (qni)
L04783  !!=========================================
L04784  IF (bl_mynn_cloudmix > 0 .AND. FLAG_QNI .AND. &
L04785         bl_mynn_mixnumcon > 0) THEN
L04786
L04787     DO k=kts,kte
L04788        qni2(k)=max(qni(k), zero)
L04789        !enforce minimum number concentration
L04790        if (sqi(k) .gt. 1e-12)qni2(k)=max(qni2(k), ni_min)
L04791     ENDDO
L04792
L04793     if (bl_mynn_edmf > 1) then
L04794
L04795        DO k=kts+1,kte-1
L04796           upcont(k)= s_awqni1(k)- s_aw1(k)*(qni2(k)*upwind+qni2(k-1)*(one-upwind))
L04797           dncont(k)=sd_awqni1(k)-sd_aw1(k)*(qni2(k)*upwind+qni2(k-1)*(one-upwind))
L04798        ENDDO
L04799        ! no flux at the top of the atmosphere
L04800        upcont(kte)=zero
L04801        dncont(kte)=zero
L04802        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04803
L04804        k=kts
L04805        a(1)=zero
L04806        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04807        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04808        d(1)=qni2(k)                                    &
L04809            &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L04810
L04811        DO k=kts+1,kte-1
L04812           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04813           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04814           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04815           d(k)=qni2(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L04816        ENDDO
L04817
L04818     else !implicit
L04819
L04820        k=kts
L04821        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04822        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)        &
L04823        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04824        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04825        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                   &
L04826        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04827        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04828        d(k)=qni2(k)                                         &
L04829        &  - dtz(k)*rhoinv(k)*s_awqni1(k+1)                  &
L04830        &  + dtz(k)*rhoinv(k)*sd_awqni1(k+1)
L04831
L04832        do k=kts+1,kte-1
L04833           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                  &
L04834           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                  &
L04835           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04836           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)     &
L04837           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))     &
L04838           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04839           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                &
L04840           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
L04841           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04842           d(k)=qni2(k)                                      &
L04843           & + dtz(k)*rhoinv(k)*(s_awqni1(k)-s_awqni1(k+1))  &
L04844           & - dtz(k)*rhoinv(k)*(sd_awqni1(k)-sd_awqni1(k+1))
L04845        enddo
L04846     endif
L04847
L04848  !! prescribed value
L04849      a(kte)=zero
L04850      b(kte)=one
L04851      c(kte)=zero
L04852      d(kte)=qni2(kte)
L04853
L04854  !    CALL tridiag(kte,a,b,c,d)
L04855      CALL tridiag2(kte,a,b,c,d,x)
L04856  !    CALL tridiag3(kte,a,b,c,d,x)
L04857
L04858      DO k=kts,kte
L04859         !qni2(k)=d(k-kts+1)
L04860         qni2(k)=x(k)
L04861         !enforce minimum number concentration
L04862         if (sqi2(k) .gt. 1e-12)qni2(k)=max(qni2(k), ni_min)
L04863      ENDDO
L04864
L04865  ELSE
L04866      qni2=qni
L04867  ENDIF
L04868
L04869  !!=========================================
L04870  !! cloud water number concentration (qnc)
L04871  !! include non-local transport
L04872  !!=========================================
L04873  IF (bl_mynn_cloudmix > 0 .AND. FLAG_QNC .AND. &
L04874      bl_mynn_mixnumcon > 0) THEN
L04875
L04876     DO k=kts,kte
```

```
L04877        qnc2(k)=max(qnc(k),zero)
L04878        !enforce minimum number concentration
L04879        if (sqc(k) .gt. 1e-12)qnc2(k)=max(qnc2(k), nc_min)
L04880     ENDDO
L04881
L04882     if (bl_mynn_edmf > 1) then
L04883
L04884        DO k=kts+1,kte-1
L04885           upcont(k)= s_awqnc1(k)- s_aw1(k)*(qnc2(k)*upwind+qnc2(k-1)*(one-upwind))
L04886           dncont(k)=sd_awqnc1(k)-sd_aw1(k)*(qnc2(k)*upwind+qnc2(k-1)*(one-upwind))
L04887        ENDDO
L04888        ! no flux at the top of the atmosphere
L04889        upcont(kte)=zero
L04890        dncont(kte)=zero
L04891        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04892
L04893        k=kts
L04894        a(1)=zero
L04895        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04896        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04897        d(1)=qnc2(k)                                        &
L04898           &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L04899
L04900        DO k=kts+1,kte-1
L04901           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04902           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04903           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04904           d(k)=qnc2(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L04905        ENDDO
L04906
L04907     else !implicit
L04908
L04909        k=kts
L04910        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04911        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)        &
L04912        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04913        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04914        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                   &
L04915        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                  &
L04916        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04917        d(k)=qnc2(k)                                         &
L04918        &  - dtz(k)*rhoinv(k)*s_awqnc1(k+1)                  &
L04919        &  + dtz(k)*rhoinv(k)*sd_awqnc1(k+1)
L04920
L04921        do k=kts+1,kte-1
L04922           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                  &
L04923           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                  &
L04924           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L04925           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)     &
L04926           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))     &
L04927           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L04928           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                &
L04929           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                &
L04930           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04931           d(k)=qnc2(k)                                      &
L04932           & + dtz(k)*rhoinv(k)*(s_awqnc1(k)-s_awqnc1(k+1))  &
L04933           & - dtz(k)*rhoinv(k)*(sd_awqnc1(k)-sd_awqnc1(k+1))
L04934        enddo
L04935     endif
L04936
L04937  !! prescribed value
L04938      a(kte)=zero
L04939      b(kte)=one
L04940      c(kte)=zero
L04941      d(kte)=qnc2(kte)
L04942
L04943  !    CALL tridiag(kte,a,b,c,d)
L04944      CALL tridiag2(kte,a,b,c,d,x)
L04945  !    CALL tridiag3(kte,a,b,c,d,x)
L04946
L04947      DO k=kts,kte
L04948         !qnc2(k)=d(k-kts+1)
L04949         qnc2(k)=x(k)
L04950         !enforce a minimum number concentration
L04951         if (sqc2(k) .gt. 1e-12)qnc2(k)=max(qnc2(k), nc_min)
L04952      ENDDO
L04953
L04954  ELSE
L04955      qnc2=qnc
L04956  ENDIF
L04957
L04958  !=========================================
L04959  ! Water-friendly aerosols ( qnwfa ).
L04960  !=========================================
L04961  IF (FLAG_QNWFA .AND. bl_mynn_mixaerosols > 0) THEN
L04962
L04963     do k=kts,kte
L04964        qnwfa2(k)=max(qnwfa(k),zero)
L04965     enddo
L04966
L04967     if (bl_mynn_edmf > 1) then
L04968
L04969        DO k=kts+1,kte-1
L04970           upcont(k)= s_awqnwfa1(k)- s_aw1(k)*(qnwfa2(k)*upwind+qnwfa2(k-1)*(one-upwind))
L04971           dncont(k)=sd_awqnwfa1(k)-sd_aw1(k)*(qnwfa2(k)*upwind+qnwfa2(k-1)*(one-upwind))
L04972        ENDDO
L04973        ! no flux at the top of the atmosphere
L04974        upcont(kte)=zero
L04975        dncont(kte)=zero
L04976        ! upcont(1) and dncont(1) are not used so they don't need to be set
L04977
L04978        k=kts
L04979        a(1)=zero
L04980        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L04981        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L04982        d(1)=qnwfa2(k)                                      &
L04983           &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L04984
```

```
L04985        DO k=kts+1,kte-1
L04986           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L04987           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L04988           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L04989           d(k)=qnwfa2(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L04990        ENDDO
L04991
L04992     else !implicit
L04993
L04994        k=kts
L04995        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L04996        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)            &
L04997        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                      &
L04998        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L04999        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                       &
L05000        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                      &
L05001        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L05002        d(k)=qnwfa2(k)                                           &
L05003        &  - dtz(k)*rhoinv(k)*s_awqnwfa1(k+1)                    &
L05004        &  + dtz(k)*rhoinv(k)*sd_awqnwfa1(k+1)
L05005
L05006        do k=kts+1,kte-1
L05007           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                      &
L05008           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                      &
L05009           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L05010           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)         &
L05011           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))         &
L05012           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L05013           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                    &
L05014           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                    &
L05015           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L05016           d(k)=qnwfa2(k)                                        &
L05017           & + dtz(k)*rhoinv(k)*(s_awqnwfa1(k)-s_awqnwfa1(k+1))  &
L05018           & - dtz(k)*rhoinv(k)*(sd_awqnwfa1(k)-sd_awqnwfa1(k+1))
L05019        enddo
L05020     endif
L05021
L05022  ! prescribed value
L05023     a(kte)=zero
L05024     b(kte)=one
L05025     c(kte)=zero
L05026     d(kte)=qnwfa2(kte)
L05027
L05028  !    CALL tridiag(kte,a,b,c,d)
L05029     call tridiag2(kte,a,b,c,d,x)
L05030  !    call tridiag3(kte,a,b,c,d,x)
L05031
L05032     do k=kts,kte
L05033        !qnwfa2(k)=d(k)
L05034        qnwfa2(k)=max(x(k),zero)
L05035     enddo
L05036
L05037     !apply liberal bounds
L05038     do k=kts,kte
L05039        !patch for high wfa over water (useful for cold starts)
L05040        if ((xland-1.5).ge.0)then   ! water
L05041           wt       = min(one, max(zero, zw(k)-100._kind_phys)/max(200._kind_phys, pblh)) !0 below 100 m, 1 above 300 m
L05042           wfa_max2 = (one - wt)*6.e8_kind_phys + wt*wfa_max
L05043        else                       ! land
L05044           wfa_max2 = wfa_max
L05045        endif
L05046        aero_min = wfa_min * exp(-zw(k)/wfa_ht)
L05047        aero_max = wfa_max2* exp(-zw(k)/wfa_ht)
L05048        qnwfa2(k)= min(max(aero_min, qnwfa2(k)), aero_max)
L05049     enddo
L05050  else
L05051     !if not mixing aerosols, set "updated" array equal to original array
L05052     qnwfa2=qnwfa
L05053  endif
L05054
L05055  !=========================================
L05056  ! Ice-friendly aerosols ( qnifa ).
L05057  !=========================================
L05058  IF (FLAG_QNIFA .AND. bl_mynn_mixaerosols > 0) THEN
L05059
L05060     do k=kts,kte
L05061        qnifa2(k)=max(qnifa(k),zero)
L05062     enddo
L05063
L05064     if (bl_mynn_edmf > 1) then
L05065
L05066        DO k=kts+1,kte-1
L05067           upcont(k)= s_awqnifa1(k)- s_aw1(k)*(qnifa2(k)*upwind+qnifa2(k-1)*(one-upwind))
L05068           dncont(k)=sd_awqnifa1(k)-sd_aw1(k)*(qnifa2(k)*upwind+qnifa2(k-1)*(one-upwind))
L05069        ENDDO
L05070        ! no flux at the top of the atmosphere
L05071        upcont(kte)=zero
L05072        dncont(kte)=zero
L05073
L05074        k=kts
L05075        a(1)=zero
L05076        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L05077        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L05078        d(1)=qnifa2(k)                                           &
L05079           &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L05080
L05081        DO k=kts+1,kte-1
L05082           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L05083           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L05084           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L05085           d(k)=qnifa2(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L05086        ENDDO
L05087
L05088     else !implicit
L05089
L05090        k=kts
L05091        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L05092        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)            &
```

```
L05093        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                      &
L05094        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L05095        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                       &
L05096        &  - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                      &
L05097        &  - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L05098        d(k)=qnifa2(k)                                           &
L05099        &  - dtz(k)*rhoinv(k)*s_awqnifa1(k+1)                    &
L05100        &  + dtz(k)*rhoinv(k)*sd_awqnifa1(k+1)
L05101
L05102        do k=kts+1,kte-1
L05103           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                      &
L05104           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)                      &
L05105           & + p5*dtz(k)*rhoinv(k)*sd_aw1(k)
L05106           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)         &
L05107           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))         &
L05108           & + p5*dtz(k)*rhoinv(k)*(sd_aw1(k)-sd_aw1(k+1))
L05109           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                    &
L05110           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)                    &
L05111           & - p5*dtz(k)*rhoinv(k)*sd_aw1(k+1)
L05112           d(k)=qnifa2(k)                                        &
L05113           & + dtz(k)*rhoinv(k)*(s_awqnifa1(k)-s_awqnifa1(k+1))  &
L05114           & - dtz(k)*rhoinv(k)*(sd_awqnifa1(k)-sd_awqnifa1(k+1))
L05115        enddo
L05116     endif
L05117
L05118  ! prescribed value
L05119     a(kte)=zero
L05120     b(kte)=one
L05121     c(kte)=zero
L05122     d(kte)=qnifa2(kte)
L05123
L05124  !    CALL tridiag(kte,a,b,c,d)
L05125     CALL tridiag2(kte,a,b,c,d,x)
L05126  !    CALL tridiag3(kte,a,b,c,d,x)
L05127
L05128     DO k=kts,kte
L05129        !qnifa2(k)=d(k-kts+1)
L05130        qnifa2(k)=max(x(k),zero)
L05131     ENDDO
L05132
L05133     !apply bounds
L05134     DO k=kts,kte
L05135        aero_min = ifa_min * exp(-zw(k)/ifa_ht)
L05136        aero_max = ifa_max * exp(-zw(k)/ifa_ht)
L05137        qnifa2(k)= min(max(aero_min, qnifa2(k)), aero_max)
L05138     ENDDO
L05139
L05140  ELSE
L05141     !If not mixing aerosols, set "updated" array equal to original array
L05142     qnifa2=qnifa
L05143  ENDIF
L05144
L05145  !=========================================
L05146  ! Black-carbon aerosols ( qnbca ).
L05147  !=========================================
L05148  IF (FLAG_QNBCA .AND. bl_mynn_mixaerosols > 0) THEN
L05149
L05150     if (bl_mynn_edmf > 1) then
L05151
L05152        DO k=kts+1,kte-1
L05153           upcont(k)= s_awqnbca1(k)- s_aw1(k)*(qnbca(k)*upwind+qnbca(k-1)*(one-upwind))
L05154           dncont(k)=zero !sd_awqnbca1(k)-sd_aw1(k)*(qnbca(k)*upwind+qnbca(k-1)*(one-upwind))
L05155        ENDDO
L05156        ! no flux at the top of the atmosphere
L05157        upcont(kte)=zero
L05158        dncont(kte)=zero
L05159
L05160        k=kts
L05161        a(1)=zero
L05162        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L05163        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L05164        d(1)=qnbca(k)                                    &
L05165            &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L05166
L05167        DO k=kts+1,kte-1
L05168           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L05169           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L05170           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L05171           d(k)=qnbca(k)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L05172        ENDDO
L05173
L05174     else !implicit
L05175
L05176        k=kts
L05177        a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L05178        b(k)=one+dtz(k)*(khdz(k) + khdz(k+1))*rhoinv(k)          &
L05179        & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*nonloc
L05180        c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                       &
L05181        & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*nonloc
L05182        d(k)=qnbca(k)  - dtz(k)*rhoinv(k)*s_awqnbca1(k+1)*nonloc
L05183
L05184        do k=kts+1,kte-1
L05185           a(k)=  -dtz(k)*khdz(k)*rhoinv(k)                      &
L05186           & + p5*dtz(k)*rhoinv(k)*s_aw1(k)*nonloc
L05187           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)         &
L05188           & + p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))*nonloc
L05189           c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)                    &
L05190           & - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)*nonloc
L05191           d(k)=qnbca(k) + dtz(k)*rhoinv(k)*(s_awqnbca1(k)-s_awqnbca1(k+1))*nonloc
L05192        enddo
L05193     endif
L05194
L05195  ! prescribed value
L05196     a(kte)=zero
L05197     b(kte)=one
L05198     c(kte)=zero
L05199     d(kte)=qnbca(kte)
L05200
```

```
L05201  !    CALL tridiag(kte,a,b,c,d)
L05202     CALL tridiag2(kte,a,b,c,d,x)
L05203  !    CALL tridiag3(kte,a,b,c,d,x)
L05204
L05205       DO k=kts,kte
L05206          !qnbca2(k)=d(k-kts+1)
L05207          qnbca2(k)=max(zero, x(k))
L05208       ENDDO
L05209
L05210    ELSE
L05211       !If not mixing aerosols, set "updated" array equal to original array
L05212       qnbca2=qnbca
L05213    ENDIF
L05214
L05215    !=========================================
L05216    ! Ozone - local mixing only
L05217    !=========================================
L05218    IF (FLAG_OZONE) THEN
L05219       k=kts
L05220
L05221    !rho-weighted:
L05222       a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L05223       b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k)
L05224       c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)
L05225       d(k)=ozone(k)
L05226
L05227       DO k=kts+1,kte-1
L05228          a(k)=  -dtz(k)*khdz(k)*rhoinv(k)
L05229          b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L05230          c(k)=  -dtz(k)*khdz(k+1)*rhoinv(k)
L05231          d(k)=ozone(k)
L05232       ENDDO
L05233
L05234    ! prescribed value
L05235       a(kte)=zero
L05236       b(kte)=one
L05237       c(kte)=zero
L05238       d(kte)=ozone(kte)
L05239
L05240    !    CALL tridiag(kte,a,b,c,d)
L05241       CALL tridiag2(kte,a,b,c,d,x)
L05242    !    CALL tridiag3(kte,a,b,c,d,x)
L05243
L05244       DO k=kts,kte
L05245          !ozone2(k)=d(k-kts+1)
L05246          dozone(k)=(max(x(k),zero) - ozone(k))/delt
L05247       ENDDO
L05248    ELSE
L05249       dozone(:)=zero
L05250    ENDIF
```

- **数浓度与气溶胶**（结构再次同构）：qni/qnc（`bl_mynn_mixnumcon > 0` 时）用 `nc_min=100/ni_min=1e-6` 保底（`sqi>1e-12` 时强制最小数浓度）；qnwfa/qnifa/qnbca（`bl_mynn_mixaerosols > 0`）混合后做**指数廓线钳位**：`aero_min/max = wfa_min/max • exp(−z/wfa_ht)`——气溶胶被约束在气候廓线附近（水面另加 100–300 m 高度过渡的 6e8 冷启动补丁）。`nonloc=1.0` 参数控制黑碳的非局地输运（L5179 起 implicit 分支用 `nonloc` 加权）。
- **臭氧**：`FLAG_OZONE` 时**只做局地扩散**（无 MF 项），输出 `dozone` 倾向并做非负修复（`Dozone • delt+ozone<0` 时取 `-0.99 • ozone/delt`）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5252-L5496
L05252  !!=============================================
L05253  !! Compute tendencies and convert to mixing ratios for WRF.
L05254  !! Note that the momentum tendencies are calculated above.
L05255  !!=============================================
L05256
L05257     IF (bl_mynn_mixqt > 0) THEN
L05258        DO k=kts,kte
L05259           !compute updated theta using updated thl and old condensate
L05260           th_new = thl(k) + xlvcp/exner(k)*sqc(k) &
L05261             &             + xlscp/exner(k)*sqi(k)
L05262
L05263           t  = th_new*exner(k)
L05264           qsat = qsat_blend(t,p(k))
L05265           !SATURATED VAPOR PRESSURE
L05266           !esat=esat_blend(t)
L05267           !SATURATED SPECIFIC HUMIDITY
L05268           !qsl=ep_2*esat/(p(k)-ep_3*esat)
L05269           !qsl=ep_2*esat/max(1.e-4,(p(k)-ep_3*esat))
L05270
L05271           IF (sqc(k) > zero .or. sqi(k) > zero) THEN !initially saturated
L05272              sqv2(k) = MIN(sqw2(k),qsat)
L05273              portion_qc = sqc(k)/(sqc(k) + sqi(k))
L05274              portion_qi = sqi(k)/(sqc(k) + sqi(k))
L05275              condensate = MAX(sqw2(k) - qsat, zero)
L05276              sqc2(k) = condensate*portion_qc
L05277              sqi2(k) = condensate*portion_qi
L05278           ELSE                        ! initially unsaturated -----
L05279              sqv2(k) = sqw2(k)        ! let microphys decide what to do
L05280              sqi2(k) = zero            ! if sqw2 > qsat
L05281              sqc2(k) = zero
L05282           ENDIF
L05283        ENDDO
L05284     ENDIF
L05285
L05286
L05287      !====================
L05288      ! WATER VAPOR TENDENCY
L05289      !====================
L05290      DO k=kts,kte
L05291         Dqv(k)=(sqv2(k) - sqv(k))/delt
L05292         !if (sqv2(k) < zero)print*,"neg qv:",sqv2(k),k
```

```
      ENDDO

      IF (bl_mynn_cloudmix > 0) THEN
        !=====================
        ! CLOUD WATER TENDENCY
        !=====================
        !print*,"FLAG_QC:",FLAG_QC
        IF (FLAG_QC) THEN
           DO k=kts,kte
              Dqc(k)=(sqc2(k) - sqc(k))/delt
              !if (sqc2(k) < zero)print*,"neg qc:",sqc2(k),k
           ENDDO
        ELSE
           DO k=kts,kte
             Dqc(k) = 0.
           ENDDO
        ENDIF

        !====================
        ! CLOUD WATER NUM CONC TENDENCY
        !====================
        IF (FLAG_QNC .AND. bl_mynn_mixnumcon > 0) THEN
           DO k=kts,kte
             Dqnc(k) = (qnc2(k)-qnc(k))/delt
             !IF(Dqnc(k)*delt + qnc(k) < 0.)Dqnc(k)=-qnc(k)/delt
           ENDDO
        ELSE
           DO k=kts,kte
             Dqnc(k) = 0.
           ENDDO
        ENDIF

        !====================
        ! CLOUD ICE TENDENCY
        !====================
        IF (FLAG_QI) THEN
           DO k=kts,kte
             Dqi(k)=(sqi2(k) - sqi(k))/delt
             !if (sqi2(k) < zero)print*,"neg qi:",sqi2(k),k
           ENDDO
        ELSE
           DO k=kts,kte
             Dqi(k) = 0.
           ENDDO
        ENDIF

        !====================
        ! CLOUD SNOW TENDENCY
        !====================
        IF (.false.) THEN !disabled
           DO k=kts,kte
             Dqs(k)=(sqs2(k) - sqs(k))/delt
           ENDDO
        ELSE
           DO k=kts,kte
             Dqs(k) = 0.
           ENDDO
        ENDIF

        !====================
        ! CLOUD ICE NUM CONC TENDENCY
        !====================
        IF (FLAG_QNI .AND. bl_mynn_mixnumcon > 0) THEN
           DO k=kts,kte
             Dqni(k)=(qni2(k)-qni(k))/delt
             !IF(Dqni(k)*delt + qni(k) < 0.)Dqni(k)=-qni(k)/delt
           ENDDO
        ELSE
           DO k=kts,kte
             Dqni(k)=0.
           ENDDO
        ENDIF
      ELSE !-MIX CLOUD SPECIES?
        !CLOUDS ARE NOT NIXED (when bl_mynn_cloudmix == 0)
        DO k=kts,kte
           Dqc(k) =0.
           Dqnc(k)=0.
           Dqi(k) =0.
           Dqni(k)=0.
           Dqs(k) =0.
        ENDDO
      ENDIF

      !ensure non-negative moist species
      CALL moisture_check(kte, delt, delp, exner,        &
                          sqv2, sqc2, sqi2, sqs2, thl,   &
                          dqv, dqc, dqi, dqs, dth        )

      !=====================
      ! OZONE TENDENCY CHECK
      !=====================
      DO k=kts,kte
         IF(Dozone(k)*delt + ozone(k) < 0.) THEN
           Dozone(k)=-ozone(k)*0.99/delt
         ENDIF
      ENDDO

      !====================
      ! THETA TENDENCY
      !====================
      IF (FLAG_QI) THEN
        DO k=kts,kte
           Dth(k)=(thl(k) + xlvcp/exner(k)*sqc(k)          &
             &            + xlscp/exner(k)*(sqi(k))        & !+sqs2(k)) &
             &            - th(k))/delt
           !Dth(k)=(thl(k) + xlvcp/exner(k)*qc_tot1(k)       &
           !  &            + xlscp/exner(k)*qi_tot1(k)       &
           !  &            - th(k))/delt
           !Use form from Tripoli and Cotton (1981) with their
```

```
L05402           !suggested min temperature to improve accuracy:
L05403           !Dth(k)=(thl(k)*(one+xlvcp/MAX(tk(k),TKmin)*sqc(k)  &
L05404           !  &               + xlscp/MAX(tk(k),TKmin)*sqi(k)) &
L05405           !  &               - th(k))/delt
L05406         ENDDO
L05407       ELSE
L05408         DO k=kts,kte
L05409            Dth(k)=(thl(k)+xlvcp/exner(k)*sqc2(k) - th(k))/delt
L05410           !Use form from Tripoli and Cotton (1981) with their
L05411           !suggested min temperature to improve accuracy.
L05412           !Dth(k)=(thl(k)*(one+xlvcp/MAX(tk(k),TKmin)*sqc(k))  &
L05413           !&               - th(k))/delt
L05414         ENDDO
L05415       ENDIF
L05416
L05417       !===================
L05418       ! AEROSOL TENDENCIES
L05419       !===================
L05420       IF (FLAG_QNWFA .AND. FLAG_QNIFA .AND. &
L05421           bl_mynn_mixaerosols > 0) THEN
L05422          DO k=kts,kte
L05423             !=====================
L05424             ! WATER-friendly aerosols
L05425             !=====================
L05426             Dqnwfa(k)=(qnwfa2(k) - qnwfa(k))/delt
L05427             !=====================
L05428             ! Ice-friendly aerosols
L05429             !=====================
L05430             Dqnifa(k)=(qnifa2(k) - qnifa(k))/delt
L05431          ENDDO
L05432       ELSE
L05433          DO k=kts,kte
L05434             Dqnwfa(k)=0.
L05435             Dqnifa(k)=0.
L05436          ENDDO
L05437       ENDIF
L05438
L05439       !========================
L05440       ! BLACK-CARBON TENDENCIES
L05441       !========================
L05442       IF (FLAG_QNBCA .AND. bl_mynn_mixaerosols > 0) THEN
L05443          DO k=kts,kte
L05444             Dqnbca(k)=(qnbca2(k) - qnbca(k))/delt
L05445          ENDDO
L05446       ELSE
L05447          DO k=kts,kte
L05448             Dqnbca(k)=0.
L05449          ENDDO
L05450       ENDIF
L05451
L05452       !ensure non-negative moist species
L05453       !note: if called down here, dth needs to be updated, but
L05454       !      if called before the theta-tendency calculation, do not compute dth
L05455       !CALL moisture_check(kte, delt, delp, exner,     &
L05456       !                   sqv, sqc, sqi, thl,         &
L05457       !                   dqv, dqc, dqi, dth )
L05458
L05459       if (debug_code) then
L05460          problem = .false.
L05461          do k=kts,kte
L05462             wsp  = sqrt(u(k)**2 + v(k)**2)
L05463             wsp2 = sqrt((u(k)+du(k)*delt)**2 + (v(k)+du(k)*delt)**2)
L05464             th2  = th(k) + Dth(k)*delt
L05465             tk2  = th2*exner(k)
L05466             if (wsp2 > 200. .or. tk2 > 360. .or. tk2 < 160.) then
L05467                problem = .true.
L05468                print*,"After tendencies problem at: i=",i," k=",k
L05469                print*," wsp=",wsp," updated wsp=",wsp2
L05470                print*," T=",th(k)*exner(k)," updated T:",tk2
L05471                print*," du=",du(k)*delt," dv=",dv(k)*delt," dth=",dth(k)*delt
L05472                print*," km=",kmdz(k)*dz(k)," kh=",khdz(k)*dz(k)
L05473                print*," u*=",ust," wspd=",wspd
L05474                print*," rhosfc=",rhosfc," delp=",delp(k)
L05475                print*," LH=",flq*rhosfc*1004.," HFX=",flt*rhosfc*1004.
L05476                print*," flq=",flq," flt=",flt," exner=",exner(k)
L05477                print*," drag term=",ust**2/wspd*dtz(k)*rhosfc/rho(kts)
L05478                kproblem = k
L05479             endif
L05480          enddo
L05481          if (problem) then
L05482             print*,"==thl:",thl(max(kproblem-3,1):min(kproblem+3,kte))
L05483             print*,"===qv:",sqv2(max(kproblem-3,1):min(kproblem+3,kte))
L05484             print*,"===qc:",sqc2(max(kproblem-3,1):min(kproblem+3,kte))
L05485             print*,"===qi:",sqi2(max(kproblem-3,1):min(kproblem+3,kte))
L05486             print*,"====u:",u(max(kproblem-3,1):min(kproblem+3,kte))
L05487             print*,"====v:",v(max(kproblem-3,1):min(kproblem+3,kte))
L05488          endif
L05489       endif
L05490
L05491    #ifdef HARDCODE_VERTICAL
L05492    # undef kts
L05493    # undef kte
L05494    #endif
L05495
L05496      END SUBROUTINE mynn_tendencies
```

- **总水拆分**（mixqt>0，L5257–L5284）：先用旧凝结量重建 `th_new` 与 `qsat`；**初始饱和**的层：`sqv2 = min(sqw2, qsat)`，凝结增量 `condensate = max(sqw2−qsat, 0)` 按旧 qc/qi 份额分给 sqc2/sqi2——**凝结产物按相态比例继承**；**初始未饱和**：全部给 sqv2（注释 “let microphys decide”——微物理层处理饱和）。
- **倾向输出**：全部 `(新 − 旧)/delt`；`moisture_check` 统一修复负值（下节）；θ 趋势由 `thl + xlvcp/ex • sqc + xlscp/ex • sqi − th` 重建（**液水位温还原成位温**，FLAG_QI 时用旧凝结量、否则用新 sqc2——两个分支对应 “冰是否参与” 的口径差）。

- **出口体检**（debug_code）：更新后风速 >200 m/s 或温度出 [160,360] K 打印完整诊断（含拖曳项、通量、km/kh）。

09b.2 `moisture_check`：负值修复与柱水量守恒

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5499-L5588
L05499    SUBROUTINE moisture_check(kte, delt, dp, exner,   &
L05500                              qv, qc, qi, qs, th,     &
L05501                              dqv, dqc, dqi, dqs, dth )
L05502
L05503    ! This subroutine was adopted from the CAM-UW ShCu scheme and
L05504    ! adapted for use here.
L05505    !
L05506    ! If qc < qcmin, qi < qimin, or qv < qvmin happens in any layer,
L05507    ! force them to be larger than minimum value by (1) condensating
L05508    ! water vapor into liquid or ice, and (2) by transporting water vapor
L05509    ! from the very lower layer.
L05510    !
L05511    ! We then update the final state variables and tendencies associated
L05512    ! with this correction. If any condensation happens, update theta too.
L05513    ! Note that (qv,qc,qi,th) are the final state variables after
L05514    ! applying corresponding input tendencies and corrective tendencies.
L05515      use module_bl_mynnedmf_common, only: xlvcp,xlscp,zero,two,p5,kind_phys
L05516
L05517      implicit none
L05518      integer,         intent(in)     :: kte
L05519      real(kind_phys), intent(in)     :: delt
L05520      real(kind_phys), dimension(kte), intent(in)     :: dp, exner
L05521      real(kind_phys), dimension(kte), intent(inout)  :: qv, qc, qi, qs, th
L05522      real(kind_phys), dimension(kte), intent(inout)  :: dqv, dqc, dqi, dqs, dth
L05523      integer:: k
L05524      real(kind_phys)::  dqc2, dqi2, dqs2, dqv2, sum, aa, dum
L05525      real(kind_phys), parameter :: qvmin = 1e-20,       &
L05526                                    qcmin = zero,         &
L05527                                    qimin = zero
L05528
L05529      do k = kte, 1, -1  ! From the top to the surface
L05530         dqc2 = max(zero, qcmin-qc(k)) !qc deficit (>=0)
L05531         dqi2 = max(zero, qimin-qi(k)) !qi deficit (>=0)
L05532         dqs2 = max(zero, qimin-qs(k)) !qs deficit (>=0)
L05533
L05534         !fix tendencies
L05535         dqc(k) = dqc(k) +  dqc2/delt
L05536         dqi(k) = dqi(k) +  dqi2/delt
L05537         dqs(k) = dqs(k) +  dqs2/delt
L05538         dqv(k) = dqv(k) - (dqc2+dqi2+dqs2)/delt
L05539         dth(k) = dth(k) + xlvcp/exner(k)*(dqc2/delt) + &
L05540                           xlscp/exner(k)*((dqi2+dqs2)/delt)
L05541         !update species
L05542         qc(k)  = qc(k)  +  dqc2
L05543         qi(k)  = qi(k)  +  dqi2
L05544         qs(k)  = qs(k)  +  dqs2
L05545         qv(k)  = qv(k)  -  dqc2 - dqi2 - dqs2
L05546         th(k)  = th(k)  +  xlvcp/exner(k)*dqc2 + &
L05547                            xlscp/exner(k)*(dqi2+dqs2)
L05548
L05549         !then fix qv
L05550         dqv2   = max(zero, qvmin-qv(k)) !qv deficit (>=0)
L05551         dqv(k) = dqv(k) + dqv2/delt
L05552         qv(k)  = qv(k)  + dqv2
L05553         if ( k .ne. 1 ) then
L05554            !print*,k,"dqv2=",dqv2," dp(k)/dp(k-1)=",dp(k)/dp(k-1)
L05555            qv(k-1)   = qv(k-1)  - dqv2*dp(k)/dp(k-1)
L05556            dqv(k-1)  = dqv(k-1) - dqv2*dp(k)/dp(k-1)/delt
L05557         endif
L05558         qv(k) = max(qv(k),qvmin)
L05559         qc(k) = max(qc(k),qcmin)
L05560         qi(k) = max(qi(k),qimin)
L05561         qs(k) = max(qs(k),qimin)
L05562      end do
L05563      ! Extra moisture used to satisfy 'qv(1)>=qvmin' is proportionally
L05564      ! extracted from all the layers that has 'qv > 2*qvmin'. This fully
L05565      ! preserves column moisture.
L05566      if( dqv2 .gt. 1.e-20 ) then
L05567          sum = zero
L05568          do k = 1, kte
L05569             if( qv(k) .gt. two*qvmin ) sum = sum + qv(k)*dp(k)
L05570          enddo
L05571          aa = dqv2*dp(1)/max(1.e-20_kind_phys,sum)
L05572          if( aa .lt. p5 ) then
L05573              do k = 1, kte
L05574                 if( qv(k) .gt. two*qvmin ) then
L05575                     dum    = aa*qv(k)
L05576                     qv(k)  = qv(k) - dum
L05577                     dqv(k) = dqv(k) - dum/delt
L05578                 endif
L05579              enddo
L05580          else
L05581          ! For testing purposes only (not yet found in any output):
L05582          !    write(*,*) 'Full moisture conservation is impossible'
L05583          endif
L05584      endif
L05585
L05586      return
L05587
L05588    END SUBROUTINE moisture_check
```

- **来源**：CAM-UW ShCu 方案移植（头注释）。**自顶向下**扫描（`do k = kte, 1, -1`）：凝结物亏空（qc/qi/qs < 0）先由同层水汽**凝结**补足（`dqv -= 缺量`、`dth += 潜热`），水汽亏空再从**下一层抽取**（`qv(k-1) -= dqv2 * dp(k)/dp(k-1)`——按气压厚度加权）。

- **柱守恒兜底**（L5563–L5584）：最底层为满足 `qv≥1e-20` 额外用的水分，按 `qv • dp` 比例从全柱 `qv > 2 • qvmin` 的层**等比回收**——注释自承 “完全保持柱水量”。`aa < 0.5` 的保护（回收比例不许超 50%）。
- **与 Thompson 的守恒段对照**：微物理用 “等比例缩放源汇”，边界层用 “逐层借贷 +柱回收”——两个方案面对同一问题（负水汽）的不同解法，教材 ch13 §13.7.5 与本节可对读。

09b.3 `mynn_mix_chem` / `mynn_mix_scalars`：化学与标量的边界层混合

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5592-L5751
L05592  SUBROUTINE mynn_mix_chem(kts,kte,i,     &
L05593       delt,dz,pblh,                      &
L05594       nchem, ndvel,                      &
L05595       chem1, vd1, settle1,               &
L05596       rho,                               &
L05597       flt, tcd, qcd,                     &
L05598       dfh,                               &
L05599       s_aw1, s_awchem1,                  &
L05600       emis_ant_no, frp,                  &
L05601       enh_mix,                           &
L05602       bl_mynn_edmf, upwind               )
L05603
L05604  !-------------------------------------------------------------
L05605    use module_bl_mynnedmf_common, only: zero,one,two,p5,kind_phys
L05606
L05607  integer, intent(in) :: kts,kte,i,bl_mynn_edmf
L05608  real(kind_phys), dimension(kts:kte), intent(in) :: dfh,dz,tcd,qcd
L05609  real(kind_phys), dimension(kts:kte), intent(in) :: rho
L05610  real(kind_phys), intent(in)    :: flt
L05611  real(kind_phys), intent(in)    :: delt,pblh,upwind
L05612  integer, intent(in) :: nchem, ndvel
L05613  real(kind_phys), dimension( kts:kte+1), intent(in) :: s_aw1
L05614  real(kind_phys), dimension( kts:kte, nchem ), intent(inout) :: chem1
L05615  real(kind_phys), dimension( kts:kte, nchem ), intent(in) :: settle1
L05616  real(kind_phys), dimension( kts:kte+1,nchem), intent(in) :: s_awchem1
L05617  real(kind_phys), dimension( ndvel ), intent(in) :: vd1
L05618  real(kind_phys), intent(in) :: emis_ant_no,frp
L05619  logical, intent(in) :: enh_mix
L05620  !local vars
L05621  real(kind_phys), dimension(kts:kte) :: dtz,upcont,dncont
L05622  real(kind_phys), dimension(kts:kte) :: a,b,c,d,x
L05623  real(kind_phys):: rhs,dztop
L05624  real(kind_phys):: t,dzk
L05625  real(kind_phys):: hght
L05626  real(kind_phys):: khdz_old, khdz_back
L05627  integer :: k,kk,kmaxfire                    ! JLS 12/21/21
L05628  integer :: ic  ! Chemical array loop index
L05629
L05630  integer, SAVE :: icall
L05631
L05632  real(kind_phys), dimension(kts:kte) :: rhoinv
L05633  real(kind_phys), dimension(kts:kte+1) :: rhoz,khdz
L05634  real(kind_phys), parameter :: NO_threshold    = 0.1      ! For anthropogenic sources
L05635  real(kind_phys), parameter :: frp_threshold   = 10.0     ! RAR 02/11/22: I increased the frp threshold to enhance
↪   mixing over big fires
L05636  real(kind_phys), parameter :: pblh_threshold  = 100.0
L05637
L05638  dztop=.5*(dz(kte)+dz(kte-1))
L05639
L05640  DO k=kts,kte
L05641     dtz(k)=delt/dz(k)
L05642  ENDDO
L05643
L05644  !Prepare "constants" for diffusion equation.
L05645  !khdz = rho*Kh/dz = rho*dfh
L05646  rhoz(kts)  =rho(kts)
L05647  rhoinv(kts)=one/rho(kts)
L05648  khdz(kts)  =rhoz(kts)*dfh(kts)
L05649
L05650  DO k=kts+1,kte
L05651     rhoz(k)  =(rho(k)*dz(k-1) + rho(k-1)*dz(k))/(dz(k-1)+dz(k))
L05652     rhoz(k)  =  MAX(rhoz(k),1E-4_kind_phys)
L05653     rhoinv(k)=one/MAX(rho(k),1E-4_kind_phys)
L05654     dzk      = p5  *( dz(k)+dz(k-1) )
L05655     khdz(k)  = rhoz(k)*dfh(k)
L05656  ENDDO
L05657  rhoz(kte+1)=rhoz(kte)
L05658  khdz(kte+1)=rhoz(kte+1)*dfh(kte)
L05659
L05660  if (bl_mynn_edmf == 1) then
L05661     !stability criteria for mf
L05662     DO k=kts+1,kte-1
L05663        khdz(k) = MAX(khdz(k),  p5*s_aw1(k))
L05664        khdz(k) = MAX(khdz(k), -p5*(s_aw1(k)-s_aw1(k+1)))
L05665     ENDDO
L05666  endif
L05667
L05668  !Enhanced mixing over fires
L05669  IF ( enh_mix ) THEN
L05670     DO k=kts+1,kte-1
L05671        khdz_old  = khdz(k)
L05672        khdz_back = pblh * 0.15_kind_phys / dz(k)
L05673        !Modify based on anthropogenic emissions of NO and FRP
L05674        IF ( pblh < pblh_threshold ) THEN
L05675           IF ( emis_ant_no > NO_threshold ) THEN
L05676              khdz(k) = MAX(1.1*khdz(k),sqrt((emis_ant_no / NO_threshold)) / dz(k) * rhoz(k)) ! JLS 12/21/21
L05677  !            khdz(k) = MAX(khdz(k),khdz_back)
L05678           ENDIF
L05679           IF ( frp > frp_threshold ) THEN
L05680              kmaxfire = ceiling(log(frp))
```

```
L05681              khdz(k) = MAX(1.1*khdz(k), (one - k/(kmaxfire*two)) * ((log(frp))**2 - two*log(frp)) / dz(k)*rhoz(k)) ! JLS
↪  12/21/21
L05682  !            khdz(k) = MAX(khdz(k),khdz_back)
L05683           ENDIF
L05684        ENDIF
L05685     ENDDO
L05686  ENDIF
L05687
L05688  !=========================================
L05689  ! Patterned after mixing of water vapor in mynn_tendencies.
L05690  !=========================================
L05691
L05692  DO ic = 1,nchem
L05693
L05694     if (bl_mynn_edmf > 1) then
L05695
L05696        DO k=kts+1,kte-1
L05697           upcont(k)= s_awchem1(k,ic)- s_aw1(k)*(chem1(k,ic)*upwind+chem1(k-1,ic)*(one-upwind))
L05698           dncont(k)= zero !sd_awchem1(k,ic)-sd_aw1(k)*(chem1(k,ic)*upwind+chem1(k-1,ic)*(one-upwind))
L05699        ENDDO
L05700        ! no flux at the top of the atmosphere
L05701        upcont(kte)=zero
L05702        dncont(kte)=zero
L05703
L05704        k=kts
L05705        a(1)=zero
L05706        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L05707        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L05708        d(1)=chem1(k,ic)                                    &
L05709           &     - dtz(k)*(upcont(k+1)+dncont(k+1))
L05710
L05711        DO k=kts+1,kte-1
L05712           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L05713           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L05714           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L05715           d(k)=chem1(k,ic)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L05716        ENDDO
L05717
L05718     else !implicit
L05719
L05720        k=kts
L05721        a(k)=   -dtz(k)*khdz(k)*rhoinv(k)
L05722        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k) - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05723        c(k)=   -dtz(k)*khdz(k+1)*rhoinv(k)           - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05724        d(k)=chem1(k,ic) & !dtz(k)*flt  !neglecting surface sources
L05725             & - dtz(k)*vd1(ic)*chem1(k,ic) &
L05726             & - dtz(k)*rhoinv(k)*s_awchem1(k+1,ic)
L05727
L05728        DO k=kts+1,kte-1
L05729           a(k)=   -dtz(k)*khdz(k)*rhoinv(k)     + p5*dtz(k)*rhoinv(k)*s_aw1(k)
L05730           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k) + &
L05731              &    p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))
L05732           c(k)=   -dtz(k)*khdz(k+1)*rhoinv(k) - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05733           d(k)=chem1(k,ic) + dtz(k)*rhoinv(k)*(s_awchem1(k,ic)-s_awchem1(k+1,ic))
L05734        ENDDO
L05735     endif
L05736
L05737     ! prescribed value at top
L05738     a(kte)=zero
L05739     b(kte)=one
L05740     c(kte)=zero
L05741     d(kte)=chem1(kte,ic)
L05742
L05743     CALL tridiag3(kte,a,b,c,d,x)
L05744
L05745     DO k=kts,kte
L05746        chem1(k,ic)=x(k) + settle1(k,ic)
L05747     ENDDO
L05748
L05749  ENDDO
L05750
L05751  END SUBROUTINE mynn_mix_chem
```

- **火点增强混合**（`enh_mix`，L5668–L5686）：PBLH<100 m 且（人为 NO 排放 >0.1 或火辐射功率 FRP>10）时，`khdz` 强制放大——NO 按根号排放比放大、FRP 按**对数剖面**（`kmaxfire = ceil(log(frp))`，高度衰减 `(1−k/(2·kmaxfire))·((ln FRP)²−2·ln FRP)`）——**烟羽抬升的参数化替代**（JLS 12/21/21 注释）。
- **沉降**：隐式分支右端 `−dtz·vd1(ic)·chem1`（干沉降速度一阶衰减）；解出后加 `settle1(k,ic)`（重力沉降倾向）。
- 求解用 **tridiag3**（区别于倾向主环的 tridiag2——见 §09b.6 的语义差异）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5755-L5877
L05755  SUBROUTINE mynn_mix_scalars(            &
L05756       kts,kte,i,                         &
L05757       delt,dz,                           &
L05758       nscalars, scalars,                 &
L05759       rho,                               &
L05760       flt, tcd, qcd,                     &
L05761       dfh,                               &
L05762       s_aw1, s_awscalars1,               &
L05763       bl_mynn_edmf, upwind               )
L05764
L05765  !-------------------------------------------------------------------
L05766    use module_bl_mynnedmf_common, only: zero,one,p5,kind_phys
L05767
L05768  integer, intent(in) :: kts,kte,i,bl_mynn_edmf
L05769  real(kind_phys), dimension(kts:kte), intent(in) :: dfh,dz,tcd,qcd
L05770  real(kind_phys), dimension(kts:kte), intent(in) :: rho
L05771  real(kind_phys), intent(in)    :: flt
L05772  real(kind_phys), intent(in)    :: delt,upwind
L05773  integer, intent(in) :: nscalars
```

```
L05774  real(kind_phys), dimension( kts:kte+1), intent(in) :: s_aw1
L05775  real(kind_phys), dimension( kts:kte, nscalars ), intent(inout) :: scalars
L05776  real(kind_phys), dimension( kts:kte+1,nscalars), intent(in) :: s_awscalars1
L05777  !local vars
L05778  real(kind_phys), dimension(kts:kte) :: dtz,upcont,dncont
L05779  real(kind_phys), dimension(kts:kte) :: a,b,c,d,x
L05780  real(kind_phys):: rhs,dztop
L05781  real(kind_phys):: dzk
L05782  integer :: k,ns  !loop indecies
L05783
L05784  real(kind_phys), dimension(kts:kte) :: rhoinv
L05785  real(kind_phys), dimension(kts:kte+1) :: rhoz,khdz
L05786
L05787  dztop=p5*(dz(kte)+dz(kte-1))
L05788
L05789  DO k=kts,kte
L05790     dtz(k)=delt/dz(k)
L05791  ENDDO
L05792
L05793  !Prepare "constants" for diffusion equation.
L05794  !khdz = rho*Kh/dz = rho*dfh
L05795  rhoz(kts)  =rho(kts)
L05796  rhoinv(kts)=one/rho(kts)
L05797  khdz(kts)  =rhoz(kts)*dfh(kts)
L05798
L05799  DO k=kts+1,kte
L05800     rhoz(k)  =(rho(k)*dz(k-1) + rho(k-1)*dz(k))/(dz(k-1)+dz(k))
L05801     rhoz(k)  =   MAX(rhoz(k),1E-4_kind_phys)
L05802     rhoinv(k)=one/MAX(rho(k),1E-4_kind_phys)
L05803     dzk      = p5*( dz(k)+dz(k-1) )
L05804     khdz(k)  = rhoz(k)*dfh(k)
L05805  ENDDO
L05806  rhoz(kte+1)=rhoz(kte)
L05807  khdz(kte+1)=rhoz(kte+1)*dfh(kte)
L05808
L05809  if (bl_mynn_edmf == 1) then
L05810     !stability criteria for mf
L05811     DO k=kts+1,kte-1
L05812        khdz(k) = MAX(khdz(k),  p5*s_aw1(k))
L05813        khdz(k) = MAX(khdz(k), -p5*(s_aw1(k)-s_aw1(k+1)))
L05814     ENDDO
L05815  endif
L05816
L05817  !=========================================
L05818  ! Patterned after mixing of water vapor in mynn_tendencies.
L05819  !=========================================
L05820
L05821  DO ns = 1,nscalars
L05822
L05823     if (bl_mynn_edmf > 1) then
L05824
L05825        DO k=kts+1,kte-1
L05826           upcont(k)= s_awscalars1(k,ns)- s_aw1(k)*(scalars(k,ns)*upwind+scalars(k-1,ns)*(one-upwind))
L05827           dncont(k)=zero !sd_awscalars1(k,ns)-sd_aw1(k)*(scalars(k,ns)*upwind+scalars(k-1,ns)*(one-upwind))
L05828        ENDDO
L05829        ! no flux at the top of the atmosphere
L05830        upcont(kte)=zero
L05831        dncont(kte)=zero
L05832
L05833        k=kts
L05834        a(1)=zero
L05835        b(1)=one + dtz(k)*khdz(k+1)*rhoinv(k)
L05836        c(1)=    - dtz(k)*khdz(k+1)*rhoinv(k)
L05837        d(1)=scalars(k,ns)                                          &
L05838             &    - dtz(k)*(upcont(k+1)+dncont(k+1))
L05839
L05840        DO k=kts+1,kte-1
L05841           a(k)=   - dtz(k)*khdz(k)*rhoinv(k)
L05842           b(k)=one+ dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k)
L05843           c(k)=   - dtz(k)*khdz(k+1)*rhoinv(k)
L05844           d(k)=scalars(k,ns)-dtz(k)*(upcont(k+1)-upcont(k)+dncont(k+1)-dncont(k))
L05845        ENDDO
L05846
L05847     else !implicit
L05848
L05849        k=kts
L05850        a(k)=   -dtz(k)*khdz(k)*rhoinv(k)
L05851        b(k)=one+dtz(k)*(khdz(k+1)+khdz(k))*rhoinv(k) - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05852        c(k)=   -dtz(k)*khdz(k+1)*rhoinv(k)           - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05853        d(k)=scalars(k,ns) - dtz(k)*rhoinv(k)*s_awscalars1(k+1,ns)
L05854
L05855        DO k=kts+1,kte-1
L05856           a(k)=   -dtz(k)*khdz(k)*rhoinv(k)     + p5*dtz(k)*rhoinv(k)*s_aw1(k)
L05857           b(k)=one+dtz(k)*(khdz(k)+khdz(k+1))*rhoinv(k) + &
L05858                &    p5*dtz(k)*rhoinv(k)*(s_aw1(k)-s_aw1(k+1))
L05859           c(k)=   -dtz(k)*khdz(k+1)*rhoinv(k) - p5*dtz(k)*rhoinv(k)*s_aw1(k+1)
L05860           d(k)=scalars(k,ns) + dtz(k)*rhoinv(k)*(s_awscalars1(k,ns)-s_awscalars1(k+1,ns))
L05861        ENDDO
L05862     endif
L05863
L05864     ! prescribed value at top
L05865     a(kte)=zero
L05866     b(kte)=one
L05867     c(kte)=zero
L05868     d(kte)=scalars(kte,ns)
L05869
L05870     CALL tridiag3(kte,a,b,c,d,x)
L05871
L05872     DO k=kts,kte
L05873        scalars(k,ns)=max(1e-12_kind_phys, x(k))
L05874     ENDDO
L05875  ENDDO
L05876
L05877  END SUBROUTINE mynn_mix_scalars
```

`mynn_mix_scalars` 与 mix_chem 完全同构（无沉降/火点项），`scalars` 数组按 `nscalars` 维循环——通用标量的接入通道。

09b.4 `retrieve_exchange_coeffs` 与三对角求解器三兄弟

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5881-L5907
L05881    SUBROUTINE retrieve_exchange_coeffs(kts,kte,dfm,dfh,dz,km1,kh1)
L05882
L05883  !-------------------------------------------------------------------
L05884      use module_bl_mynnedmf_common, only: zero,p5,kind_phys
L05885
L05886      integer , intent(in) :: kts,kte
L05887
L05888      real(kind_phys), dimension(kts:kte), intent(in)  :: dz,dfm,dfh
L05889      real(kind_phys), dimension(kts:kte), intent(out) :: km1, kh1
L05890
L05891
L05892      integer :: k
L05893      real(kind_phys):: dzk
L05894
L05895      km1(kts)=zero
L05896      kh1(kts)=zero
L05897
L05898      do k=kts+1,kte
L05899         dzk   = p5 *( dz(k)+dz(k-1) )
L05900         km1(k)=dfm(k)*dzk
L05901         kh1(k)=dfh(k)*dzk
L05902      enddo
L05903
L05904  !    km1(kte+1)=0.
L05905  !    kh1(kte+1)=0.
L05906
L05907    END SUBROUTINE retrieve_exchange_coeffs
```

`km1 = dfm • dzk`、`kh1 = dfh • dzk`——把 “除以 dz” 的扩散系数还原成真实量纲（m²/s）输出给 driver/历史。底层零（表面层用相似律另行处理）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5911-L5943
L05911    SUBROUTINE tridiag(n,a,b,c,d)
L05912
L05913  !! to solve system of linear eqs on tridiagonal matrix n times n
L05914  !! after Peaceman and Rachford, 1955
L05915  !! a,b,c,d - are vectors of order n
L05916  !! a,b,c - are coefficients on the LHS
L05917  !! d - is initially RHS on the output becomes a solution vector
L05918  !-------------------------------------------------------------------
L05919      use module_bl_mynnedmf_common, only: zero,one,kind_phys
L05920
L05921      integer, intent(in):: n
L05922      real(kind_phys), dimension(n), intent(in) :: a,b
L05923      real(kind_phys), dimension(n), intent(inout) :: c,d
L05924
L05925      integer :: i
L05926      real(kind_phys):: p
L05927      real(kind_phys), dimension(n) :: q
L05928
L05929      c(n)=zero
L05930      q(1)=-c(1)/b(1)
L05931      d(1)=d(1)/b(1)
L05932
L05933      DO i=2,n
L05934         p=one/(b(i)+a(i)*q(i-1))
L05935         q(i)=-c(i)*p
L05936         d(i)=(d(i)-a(i)*d(i-1))*p
L05937      ENDDO
L05938
L05939      DO i=n-1,1,-1
L05940         d(i)=d(i)+q(i)*d(i+1)
L05941      ENDDO
L05942
L05943    END SUBROUTINE tridiag
```

tridiag：Peaceman & Rachford (1955) 经典追赶法——**就地覆盖 d**（输入右端、输出解向量，c/d 声明 inout）。q 数组存消元系数。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5947-L5980
L05947        subroutine tridiag2(kte,a,b,c,d,x)
L05948
L05949  !      a - sub-diagonal (means it is the diagonal below the main diagonal)
L05950  !      b - the main diagonal
L05951  !      c - sup-diagonal (means it is the diagonal above the main diagonal)
L05952  !      d - right part
L05953  !      x - the answer
L05954  !      kte - number of unknowns (Levels)
L05955          use module_bl_mynnedmf_common, only: zero,one,kind_phys
L05956
L05957          integer,intent(in) :: kte
L05958          real(kind_phys), dimension(kte), intent(in) :: a,b,c,d
L05959          real(kind_phys), dimension(kte), intent(out):: x
L05960          real(kind_phys), dimension(kte)  :: cp1,dp1
L05961          real(kind_phys):: m
L05962          integer :: k
L05963
L05964          ! initialize c-prime and d-prime
L05965          cp1(1) = c(1)/b(1)
L05966          dp1(1) = d(1)/b(1)
L05967          ! solve for vectors c-prime and d-prime
L05968          do k = 2,kte
L05969             m = b(k)-cp1(k-1)*a(k)
L05970             cp1(k) = c(k)/m
L05971             dp1(k) = (d(k)-dp1(k-1)*a(k))/m
L05972          enddo
L05973          ! initialize x
```

```
L05974          x(kte) = dp1(kte)
L05975          ! solve for x from the vectors c-prime and d-prime
L05976          do k = kte-1, 1, -1
L05977             x(k) = dp1(k)-cp1(k)*x(k+1)
L05978          end do
L05979
L05980       end subroutine tridiag2
```

tridiag2：Thomas 算法的“教科书形态”——前向消元构造 c'/d'（cp1/dp1），回代得 x。**输入 a/b/c/d 全部保留（intent(in)）**、解写进独立输出 x——这是倾向主环统一采用的版本（安全：同组 a/b/c/d 可复用）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L5983-L6018
L05983          subroutine tridiag3(kte,a,b,c,d,x)
L05984
L05985    !ccccccccccccccccccccccccccccc
L05986    ! Aim: Inversion and resolution of a tridiagonal matrix
L05987    !          A X = D
L05988    ! Input:
L05989    !  a(*) lower diagonal (Ai,i-1)
L05990    !  b(*) principal diagonal (Ai,i)
L05991    !  c(*) upper diagonal (Ai,i+1)
L05992    !  d
L05993    ! Output
L05994    !  x     results
L05995    !ccccccccccccccccccccccccccccc
L05996          use module_bl_mynnedmf_common, only: zero,one,kind_phys
L05997
L05998          integer,intent(in)   :: kte
L05999          integer, parameter   :: kts=1
L06000          real(kind_phys), dimension(kte) :: a,b,c,d
L06001          real(kind_phys), dimension(kte), intent(out) :: x
L06002          integer :: k
L06003
L06004          do k=kte-1,kts,-1
L06005             d(k)=d(k)-c(k)*d(k+1)/b(k+1)
L06006             b(k)=b(k)-c(k)*a(k+1)/b(k+1)
L06007          enddo
L06008
L06009          do k=kts+1,kte
L06010             d(k)=d(k)-a(k)*d(k-1)/b(k-1)
L06011          enddo
L06012
L06013          do k=kts,kte
L06014             x(k)=d(k)/b(k)
L06015          enddo
L06016
L06017          return
L06018          end subroutine tridiag3
```

tridiag3：另一种就地消元（先上行消 c、再下行消 a、最后除主对角）——**会修改输入的 b/c/d**（无 intent 声明按默认 inout）。被 mix_chem/mix_scalars 采用。**三个求解器数学等价、接口语义不同**——改造时换求解器要检查输入是否被复用。

09b.5 GET_PBLH：混合型边界层高度诊断

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L6022-L6175
L06022    !! This subroutine calculates hybrid diagnotic boundary-layer height (PBLH).
L06023    !!
L06024    !! NOTES ON THE PBLH FORMULATION:
L06025    !!Two different diagnostics are calculated, each of them specialized for a
L06026    !!distinct regime (stbale and unstable). The two diagnostics are then blended
L06027    !!according to low-level stability, where the magnitude of the of the unstable
L06028    !!pbl height is used as a proxy for the low-level stability. The unstable pblh uses
L06029    !!the delta-theta-increase method, which defines the pbl height as the level at
L06030    !!which the potential temperature first exceeds the minimum potential
L06031    !!!temperature within the boundary layer by some specified delta (about 1.5 K).
L06032    !!When applied to observed temperatures, this method has been shown to produce PBL-
L06033    !!height estimates that are unbiased relative to profiler-based
L06034    !!estimates (Nielsen-Gammon et al. 2008\cite Nielsen_Gammon_2008),
L06035    !!but it can be biased in stable conditions. Two options
L06036    !!are now available for the stable pblh diagnostic, selectable by the
L06037    !!stable_method internal parameter. Option 0 uses a TKE-based PBL height,
L06038    !!which has been shown by Banta and Pichugina (2008) \cite Pichugina_2008 to be a good estimate of
L06039    !!the PBL height in stable LLJ conditions. Option 1 selects a friction-velocity method of
L06040    !!Koracin and Berkowicz (1988\cite Koracin_Berkowicz_1988), which is a very simple one-line
L06041    !!diagnostic that has been shown to be very reliable strictly within the stable regime
L06042    !!(Steeneveld et al. 2007\cite Steeneveld_et_al_2007). Therefore, a blending (hybrid)
L06043    !!method is implemented that uses both methods, weighting each higher in their
L06044    !!respective specialized regime.
L06045    !>\section gen_get_pblh  GSD get_pblh General Algorithm
L06046    !> @{
L06047    SUBROUTINE GET_PBLH(KTS,KTE,pblh,thv1,qke1,ust,zw1,dz1,landsea,kpbl)
L06048
L06049    !-----------------------------------------------------------------
L06050      use module_bl_mynnedmf_common, only: zero,one,ten,hundred,p5,kind_phys
L06051
L06052    integer,intent(in) :: KTS,KTE
L06053
L06054    #ifdef HARDCODE_VERTICAL
L06055    # define kts 1
L06056    # define kte HARDCODE_VERTICAL
L06057    #endif
L06058
L06059    real(kind_phys), intent(out) :: pblh
L06060    real(kind_phys), intent(in) :: landsea,ust
L06061    real(kind_phys), dimension(kts:kte), intent(in) :: thv1, qke1, dz1
```

```
L06062  real(kind_phys), dimension(kts:kte+1), intent(in) :: zw1
L06063  !local variables
L06064  real(kind_phys):: pblh_stable,qtke,qtkem1,wt,maxqke,TKEeps,minthv,cpblh
L06065  real(kind_phys):: delt_thv   !delta theta-v; dependent on land/sea point
L06066  real(kind_phys), parameter :: sbl_lim  = 200. !below this height (m), the stable PBLH method will dominate the
↪  blending.
L06067  real(kind_phys), parameter :: sbl_damp = 170. !transition length for blending (m).
L06068  integer :: i,j,k,kthv,ktke,kpbl
L06069  integer, parameter :: stable_method=1 !0: TKE-based PBLH, 1: ust*700
L06070
L06071  !Initialize kpbl
L06072  kpbl = 2
L06073
L06074  !> - FIND MIN THETAV IN THE LOWEST 200 M AGL
L06075  k = kts+1
L06076  kthv = 1
L06077  minthv = 9.E9_kind_phys
L06078  do while (zw1(k) .le. 200.)
L06079     if (minthv > thv1(k)) then
L06080        minthv = thv1(k)
L06081        kthv = k
L06082     endif
L06083     k = k+1
L06084  enddo
L06085
L06086  !> - FIND THETAV-BASED PBLH (BEST FOR DAYTIME).
L06087  pblh=zero
L06088  k = kthv+1
L06089  if ((landsea-1.5) .ge. zero) then
L06090     ! WATER
L06091     delt_thv = 0.75_kind_phys
L06092  else
L06093     ! LAND
L06094     delt_thv = 1.25_kind_phys
L06095  endif
L06096
L06097  pblh=zero
L06098  k = kthv+1
L06099  do k=kts+1,kte-1
L06100     if (thv1(k) .ge. (minthv + delt_thv)) then
L06101        pblh = zw1(k) - dz1(k-1)* &
L06102        & min((thv1(k)-(minthv + delt_thv))/ &
L06103        & max(thv1(k)-thv1(k-1),1E-6_kind_phys),one)
L06104     endif
L06105     if (k .EQ. kte-1) pblh = zw1(kts+1) !EXIT SAFEGUARD
L06106     if (pblh .gt. zero) exit
L06107  enddo
L06108  !print*,"IN GET_PBLH:",thsfc,pblh
L06109
L06110  if (stable_method == 0) then
L06111     !> - for stable boundary layers, use tke method to complement the
L06112     !! thetav-based definition (when the theta-v based pblh is below ~0.5 km).
L06113     !!the tanh weighting function will make the tke-based definition negligible
L06114     !!when the theta-v-based definition is above ~1 km.
L06115     ktke   = 1
L06116     maxqke = max(qke1(kts),zero)
L06117     !use 5% of tke max (kosovic and curry, 2000; jas)
L06118     !tkeeps = maxtke/20. = maxqke/40.
L06119     tkeeps = maxqke/40._kind_phys
L06120     tkeeps = max(tkeeps,0.01_kind_phys) !0.025)
L06121     pblh_stable=zero
L06122
L06123     k = ktke+1
L06124     do k=kts+1,kte-1
L06125        qtke  =max(p5*qke1(k)  , zero)
L06126        qtkem1=max(p5*qke1(k-1), zero)
L06127        if (qtke .le. tkeeps) then
L06128           pblh_stable = zw1(k) - dz1(k-1)* &
L06129           & min((tkeeps-qtke)/max(qtkem1-qtke, 1e-6_kind_phys), one)
L06130           !in case of near zero tke, set pblh = lowest level.
L06131           pblh_stable = max(pblh_stable,zw1(kts+1))
L06132           !print *,"pblh_stable:",i,pblh_stable, qke1(k)/2., zw1(kts+1)
L06133        endif
L06134        !k = k+1
L06135        if (k .eq. kte-1) pblh_stable = zw1(kts+1) !exit safeguard
L06136        if (pblh_stable .ne. zero) exit
L06137     enddo
L06138     !> - the tke-based pblh can (rarely) become very large
L06139     !! in grid points with deep convection (> 8 km!),
L06140     !! so a limit is imposed to not let pblh_stable exceed the
L06141     !! theta_v-based pbl height +/- 350 m.
L06142     !! this has no impact on 99% of the domain.
L06143     pblh_stable = min(pblh_stable,pblh+350._kind_phys)
L06144     pblh_stable = max(pblh_stable,max(pblh-350._kind_phys, ten))
L06145     !if in old pool situation, default to theta_v-based def
L06146     if (maxqke <= tkeeps) pblh_stable = pblh
L06147  else
L06148     !> - Method based on Koracin and Berkowicz (1988):
L06149     wt          = p5*TANH((pblh - hundred)/hundred) + p5
L06150     cpblh       = 400._kind_phys*(one-wt) + 700._kind_phys*wt
L06151     pblh_stable = max(ust*cpblh, ten)
L06152     !Even though this estimate will dominate in stable conditions, it should
L06153     !be liberally bounded:
L06154     pblh_stable = max(pblh_stable, pblh-sbl_lim)
L06155     pblh_stable = min(pblh_stable, pblh+sbl_lim)
L06156  endif
L06157
L06158  !blend the stable and unstable pblh heights
L06159  wt=p5*TANH((pblh - sbl_lim)/sbl_damp) + p5
L06160  pblh=pblh_stable*(one-wt) + pblh*wt
L06161
L06162  !Compute kpbl
L06163  do k=kts+1,kte-1
L06164     if ( zw1(k) >= pblh) then
L06165        kpbl = k-1
L06166        exit
L06167     endif
L06168  enddo
```

```
L06169
L06170  #ifdef HARDCODE_VERTICAL
L06171  # undef kts
L06172  # undef kte
L06173  #endif
L06174
L06175  END SUBROUTINE GET_PBLH
```

- **设计哲学**（头注释 L6022–L6046 原文）：两个专用诊断按稳定度混合——**不稳定侧用 θ_v 增量法**（Nielsen-Gammon 2008：θ_v 首次超过 PBL 内最小值 + Δ 的层；观测对比无偏）、**稳定侧两选一**（`stable_method=0`：TKE 阈值法，Banta & Pichugina 2008——低空急流稳定层的可靠估计；`=1`（现行默认）：Koracin & Berkowicz (1988) 摩擦速度法 `u*×400~700`，一行诊断、稳定段极可靠（Steeneveld 2007））。**tanh 权重按不稳定 PBLH 混合**（200 m 以下稳定法主导、170 m 过渡带）。
- **θ_v 增量法细节**：最小 θ_v 只在**最低 200 m** 内找（`minthv`）；Δθ_v **水面 0.75 K / 陆地 1.25 K**（海面湍流弱、阈值低）；层内线性插值落点；`kte-1` 兜底（PBL 顶到模式顶时回退最低层）。
- **TKE 法细节**（stable_method=0）：阈值 `tkeps = maxqke/40`（Kosovic & Curry 2000 的 5% TKE：qke=2TKE，5%=1/20 TKE=1/40 qke）；诊断出的稳定 PBLH **钳在 θ_v 法 ±350 m 内**（防深对流格点 PBLH 冲到 8 km——注释自承 “99% 的域不受影响”）。
- **kpbl**：`zw(k) ≥ pblh` 的首层前一索引——staggered 的 PBL 顶索引，供 DMP_mf/ddmp_mf 定位。

09b.6 `DMP_mf`：八羽谱积分上升流（EDMF 核心）

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L6197-L6404
L06197    SUBROUTINE DMP_mf(i,j,                                  &
L06198                   & kts,kte,dt,zw1,dz1,pres1,rho1,  &
L06199                   & momentum_opt,                  &
L06200                   & tke_opt,                       &
L06201                   & scalar_opt,                    &
L06202                   & aerosol_opt, numcon_opt,       &
L06203                   & bl_mynn_closure,               &
L06204                   & u1,v1,w1,th1,thl1,thv1,tk1,    &
L06205                   & qt1,qv1,qc1,qke1,qsq1,         &
L06206                   & qnc1,qni1,qnwfa1,qnifa1,qnbca1, &
L06207                   & ex1,vt1,vq1,sgm1,              &
L06208                   & ust,flt,fltv,flq,flqv,         &
L06209                   & pblh,kpbl,dx,landsea,ts,       &
L06210              ! outputs - updraft properties
L06211                   & edmf_a1,edmf_w1,               &
L06212                   & edmf_qt1,edmf_thl1,            &
L06213                   & edmf_ent1,edmf_qc1,            &
L06214                   & edmf_qv1,edmf_u1,edmf_v1,      &
L06215              ! outputs - variables needed for solver
L06216                   & s_aw1,s_awthl1,s_awqt1,        &
L06217                   & s_awqv1,s_awqc1,               &
L06218                   & s_awu1,s_awv1,s_awqke1,s_awqsq1,&
L06219                   & s_awqnc1,s_awqni1,             &
L06220                   & s_awqnwfa1,s_awqnifa1,         &
L06221                   & s_awqnbca1,                    &
L06222                   & sub_thl1,sub_sqv1,             &
L06223                   & sub_u1,sub_v1,                 &
L06224                   & det_thl1,det_sqv1,det_sqc1,    &
L06225                   & det_u1,det_v1,                 &
L06226              ! chem/smoke
L06227                   & nchem,chem1,s_awchem1,         &
L06228                   & mix_chem,                      &
L06229              ! generic scalar array
L06230                   & nscalars,scalars,s_awscalars1,  &
L06231              ! in/outputs - subgrid scale clouds
L06232                   & qc_bl1,cldfra_bl1,             &
L06233                   & qc_bl1_old,cldfra_bl1_old,     &
L06234              ! inputs - flags for moist arrays
L06235                   & F_QC,F_QI,                     &
L06236                   & F_QNC,F_QNI,                   &
L06237                   & F_QNWFA,F_QNIFA,F_QNBCA,       &
L06238                   & Psig_shcu,                     &
L06239              ! output info
L06240                   & maxwidth,ktop,maxmf,ztop,      &
L06241                   & excess_h,excess_q,             &
L06242              ! inputs for stochastic perturbations
L06243                   & spp_pbl,pattern_spp_pbl1,      &
L06244                   & tkeprod_up,el1                 )
L06245
L06246      use module_bl_mynnedmf_common, only: gtr,p608,grav,xlvcp,ep_2,  &
L06247           ep_3,r_v,cp,cpv,tv0,b1,                                    &
L06248           zero,one,two,three,four,five,thirty,hundred,               &
L06249           p2,p3,p333,p4,p5,kind_phys
L06250
L06251  #ifdef HARDCODE_VERTICAL
L06252  # define kts 1
L06253  # define kte HARDCODE_VERTICAL
L06254  #endif
L06255
L06256  ! configuration inputs:
L06257   integer, intent(in) ::                                        &
L06258        kts,                kte,                kpbl,        &
L06259        momentum_opt,       tke_opt,            scalar_opt,  &
L06260        aerosol_opt,        numcon_opt,                      &
L06261        spp_pbl,            i,                  j
L06262   real(kind_phys), intent(in):: bl_mynn_closure
L06263   real(kind_phys), dimension(kts:kte), intent(in)  ::           &
```

```
L06264        pattern_spp_pbl1
L06265   ! state variables
L06266    real(kind_phys), dimension(kts:kte), intent(in)  ::               &
L06267         &u1,v1,w1,th1,thl1,tk1,qt1,qv1,qc1,                          &
L06268         &ex1,dz1,thv1,pres1,rho1,qke1,qsq1,qnc1,qni1,                &
L06269         &qnwfa1,qnifa1,qnbca1,el1
L06270    real(kind_phys),dimension(kts:kte+1), intent(in) ::               &
L06271         &zw1
L06272    real(kind_phys), intent(in)                      ::               &
L06273         &flt,fltv,flq,flqv,Psig_shcu,                                &
L06274         &landsea,ts,dx,dt,ust,pblh
L06275    logical, optional :: F_QC,F_QI,F_QNC,F_QNI,F_QNWFA,F_QNIFA,F_QNBCA
L06276
L06277    ! outputs - updraft properties
L06278    real(kind_phys),dimension(kts:kte), intent(inout) ::              &
L06279         &edmf_a1,edmf_w1,edmf_qt1,edmf_thl1,edmf_ent1,edmf_qc1,      &
L06280         &edmf_u1,edmf_v1,edmf_qv1
L06281    ! add one local edmf variable:
L06282     real(kind_phys),dimension(kts:kte) :: edmf_th1
L06283    ! output
L06284    integer,         intent(inout) :: ktop
L06285    real(kind_phys), intent(inout) :: maxmf,ztop,maxwidth,            &
L06286         &excess_h,excess_q
L06287    ! outputs - variables needed for solver: sum ai*rho*wis_awphi
L06288    real(kind_phys),dimension(kts:kte+1), intent(inout) ::            &
L06289         &s_aw1,s_awthl1,s_awqt1,s_awqv1,s_awqc1,s_awqnc1,s_awqni1,   &
L06290         &s_awqnwfa1,s_awqnifa1,s_awqnbca1,s_awu1,s_awv1,             &
L06291         &s_awqke1,s_awqsq1
L06292    real(kind_phys),dimension(kts:kte+1) :: s_aw2
L06293
L06294    real(kind_phys),dimension(kts:kte), intent(inout) ::              &
L06295         &qc_bl1,cldfra_bl1,qc_bl1_old,cldfra_bl1_old,tkeprod_up
L06296    integer, parameter :: nup=8, debug_mf=0
L06297    real(kind_phys)    :: nup2
L06298
L06299    !------------- local variables ------------------
L06300    ! updraft properties defined on interfaces (k=1 is the top of the
L06301    ! first model layer
L06302    real(kind_phys),dimension(kts:kte+1,1:NUP) ::                     &
L06303         &UPW,UPTHL,UPQT,UPQC,UPQV,                                   &
L06304         &UPA,UPU,UPV,UPTHV,UPQKE,UPQSQ,UPQNC,                        &
L06305         &UPQNI,UPQNWFA,UPQNIFA,UPQNBCA
L06306    ! entrainment defined as the mass-layer mean
L06307    real(kind_phys),dimension(kts:kte,1:NUP) :: ENT
L06308    ! internal variables
L06309    integer :: k,ip,k50
L06310    real(kind_phys):: fltv2,wstar,qstar,thstar,sigmaW,sigmaQT,        &
L06311         &sigmaTH,z0,pwmin,pwmax,wmin,wmax,wlv,Psig_w,maxw,maxqc,wpbl
L06312    real(kind_phys):: B,QTn,THLn,THVn,QCn,Un,Vn,QKEn,QSQn,QNCn,QNIn,  &
L06313         &  QNWFAn,QNIFAn,QNBCAn,upak,                                &
L06314         &  Wn2,Wn,EntEXP,EntEXM,EntW,BCOEFF,THVkm1,THVk,Pk,rho_int
L06315
L06316    ! w parameters-used in the original entrainment form
L06317    real(kind_phys), parameter ::                                     &
L06318         &Wa=2./3.,      Wb=0.002,      Wc=1.5
L06319
L06320    ! Parameters/variables for regulating plumes:
L06321    real(kind_phys), parameter :: Atot = 0.10 ! Maximum total fractional area of all updrafts
L06322    real(kind_phys), parameter :: lmax = 1000.! diameter of largest plume (absolute maximum, can be smaller)
L06323    real(kind_phys), parameter :: lmin = 300. ! diameter of smallest plume (absolute minimum, can be larger)
L06324    real(kind_phys), parameter :: dlmin = 0.  ! delta increase in the diameter of smallest plume (large fltv)
L06325    real(kind_phys)            :: minwidth    ! actual width of smallest plume
L06326    real(kind_phys)            :: dl          ! variable increment of plume size
L06327    real(kind_phys), parameter :: dcut = 1.2  ! max diameter of plume to parameterize relative to dx (km)
L06328    real(kind_phys)::  d     != -2.3 to -1.7  ;=-1.9 in Neggers paper; power law exponent for number density (N=Cl^d).
L06329         ! Note that changing d to -2.0 makes each size plume equally contribute to the total coverage of all plumes.
L06330         ! Note that changing d to -1.7 doubles the area coverage of the largest plumes relative to the smallest plumes.
L06331    real(kind_phys):: cn,c,l,n,an2,hux,wspd_pbl,cloud_base,           &
L06332         maxwidth_dx,maxwidth_pbl,maxwidth_cld,maxwidth_flx
L06333
L06334    ! chem/smoke
L06335    integer, intent(in) :: nchem
L06336    real(kind_phys),dimension(kts:kte,   nchem)       :: chem1
L06337    real(kind_phys),dimension(kts:kte+1, nchem)       :: s_awchem1
L06338    real(kind_phys),dimension(nchem)                  :: chemn
L06339    real(kind_phys),dimension(kts:kte+1,1:NUP, nchem) :: UPCHEM
L06340    integer :: ic,cb_check
L06341    real(kind_phys),dimension(kts:kte,   nchem)       :: edmf_chem
L06342    logical, intent(in) :: mix_chem
L06343    !generic scalars
L06344    integer, intent(in) :: nscalars
L06345    real(kind_phys),dimension(kts:kte,   nscalars)    :: scalars
L06346    real(kind_phys),dimension(kts:kte+1, nscalars)    :: s_awscalars1
L06347    real(kind_phys),dimension(nscalars)               :: scalarsn
L06348    real(kind_phys),dimension(kts:kte+1,1:NUP,nscalars) :: upscalars
L06349    !local ktop for each plume
L06350    integer,dimension(1:NUP) :: ktop_plume
L06351    logical :: superadiabatic
L06352
L06353    ! Varaibles for mass flux cloud fraction
L06354    real(kind_phys),dimension(kts:kte), intent(inout) :: vt1, vq1, sgm1
L06355    real(kind_phys):: sigq,xl,rsl,cpm,a,qmq,Aup,Q1,diffqt,qsat_tk,    &
L06356           Fng,qww,alpha,beta,bb,f,pt,t,q2p,b9,satvp,rhgrid,entfac,   &
L06357           cf_strat,qc_strat,cf_mf,qc_mf,qc_mf_min,pct_mf,wt2,        &
L06358           dqwdz,tauc,mf_at_cb,qcfac,sigqfac,cffac
L06359    real(kind_phys), parameter :: cf_thresh = 0.5 ! only overwrite stratus CF less than this value
L06360
L06361    ! Variables interpolated to interface levels
L06362    real(kind_phys),dimension(kts:kte) :: exneri,dzi,rhoi,qti
L06363    ! Variables for plume interpolation/saturation check
L06364    real(kind_phys):: THp, QTp, QCp, QCs, esat, qsl
L06365    real(kind_phys):: csigma,acfac,ac_wsp
L06366
L06367    !plume overshoot
L06368    integer :: overshoot
L06369    real(kind_phys):: bvf, Frz, dzp
L06370
L06371    !Flux limiter: not let mass-flux of heat between k=1&2 exceed (fluxportion)*(surface heat flux).
```

```
L06372   !This limiter makes adjustments to the entire column.
L06373   real(kind_phys):: adjustment, flx1, flt2
L06374   real(kind_phys), parameter :: fluxportion=0.75 ! set liberally, so has minimal impact. Note that
L06375                                 ! 0.5 starts to have a noticeable impact
L06376                                 ! over land (decrease maxMF by 10-20%), but no impact over water.
L06377   !----- environmental subsidence-------------------------------------
L06378   !Option to activate environmental subsidence in mass-flux scheme
L06379   logical,parameter:: env_subs   = .false.
L06380   real(kind_phys),dimension(kts:kte) :: sub_thl1,sub_sqv1,         &
L06381           sub_u1,sub_v1,det_thl1,det_sqv1,det_sqc1,det_u1,det_v1,  &  !tendencied due to detrainment
L06382           envm_a,envm_w,envm_thl,envm_sqv,envm_sqc,                &
L06383           envm_u,envm_v                  !environmental variables defined at middle of layer
L06384   real(kind_phys),dimension(kts:kte+1) ::  envi_a,envi_w !environmental variables defined at model interface
L06385   real(kind_phys):: temp,sublim,qc_ent,qv_ent,qt_ent,thl_ent,detrate, &
L06386           detrateUV,oow,exc_fac,aratio,detturb,qc_grid,qc_sgs,        &
L06387           qc_plume,exc_heat,exc_moist,tk_int,tvs,dthvdz,zagl
L06388   real(kind_phys), parameter :: Cdet    = 1./45.
L06389   real(kind_phys), parameter :: dzpmax = 300. !limit dz used in detrainment - can be excessing in thick layers
L06390   !parameter "Csub" determines the propotion of upward vertical velocity that contributes to
L06391   !environmenatal subsidence. Some portion is expected to be compensated by downdrafts instead of
L06392   !gentle environmental subsidence. 1.0 assumes all upward vertical velocity in the mass-flux scheme
L06393   !is compensated by "gentle" environmental subsidence.
L06394   real(kind_phys), parameter :: Csub=0.25
L06395
L06396   !Factor for the pressure gradient effects on momentum transport
L06397   real(kind_phys), parameter :: pgfac = 0.00  ! Zhang and Wu showed 0.4 is more appropriate for lower troposphere
L06398   real(kind_phys):: Uk,Ukm1,Vk,Vkm1,dxsa
L06399
L06400   ! WA TEST 12/23/24 Vars for PBL-average QKE
L06401   real(kind_phys):: qkebl
L06402
L06403   !set for debugging at specific point
L06404   !integer, parameter::idbg = 452, jdbg = 272
```

- **签名**：60+ 实参（主入口 §09a.4.6 的装配表是它的镜像）；四个 “opt” 开关（momentum/tke/scalar/aerosol/numcon）逐项控制 MF 对各变量的输运。
- **关键常数**（全部 PARAMETER，注释自带出处）：
  - `nup = 8`——**八个不同直径的羽流**（谱积分，区别于单羽 EDMF）；
  - `Atot = 0.10`——全部上升流面积分数上限；
  - `lmin=300/lmax=1000` m——羽流直径上下限；`dcut=1.2` km——超过它参数化失效（dx 相关）；
  - `d = −1.9`——数密度幂律 `N = C•l^d` 的指数（Neggers 2015, JAMES 建议；注释：改 −2.0 则每档羽流等贡献面积、改 −1.7 则大羽流面积加倍）；
  - `fluxportion = 0.75`——通量限制器（首层 MF 热通量不得超过地表通量的 75%；0.5 会在陆地减 10–20% maxMF）；
  - `env_subs = .false.`——环境下沉补偿**关闭**（Csub=0.25、Cdet=1/45 是参数）；
  - `pgfac = 0.00`——动量的气压梯度项系数（注释：Zhang & Wu 认为 0.4 更合适）——**待启用的物理**。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L6406-L6610
L06406   ! Intitialize 2d ouput
L06407   ktop      =0    !integer
L06408   ztop      =zero
L06409   maxmf     =zero
L06410   maxwidth  =zero
L06411   excess_h  =zero
L06412   excess_q  =zero
L06413   ! Initialize individual updraft properties
L06414   upw       =zero
L06415   upthl     =zero
L06416   upthv     =zero
L06417   upqt      =zero
L06418   upa       =zero
L06419   upu       =zero
L06420   upv       =zero
L06421   upqc      =zero
L06422   upqv      =zero
L06423   upqke     =zero
L06424   upqsq     =zero
L06425   upqnc     =zero
L06426   upqni     =zero
L06427   upqnwfa   =zero
L06428   upqnifa   =zero
L06429   upqnbca   =zero
L06430   if ( mix_chem ) then
L06431      upchem(kts:kte+1,1:nup,1:nchem)=zero
L06432   endif
L06433   if ( scalar_opt > 0 ) then
L06434      upscalars(kts:kte+1,1:nup,1:nscalars)=zero
L06435   endif
L06436   ent       =0.001_kind_phys
L06437   ! Initialize mean updraft properties
L06438   edmf_a1   =zero
L06439   edmf_w1   =zero
L06440   edmf_qt1  =zero
L06441   edmf_thl1 =zero
L06442   edmf_ent1 =zero
L06443   edmf_qc1  =zero
L06444   edmf_qv1  =zero
L06445   edmf_u1   =zero
L06446   edmf_v1   =zero
L06447   if ( mix_chem ) then
L06448      edmf_chem(kts:kte,1:nchem) = zero
L06449   endif
L06450   ! Initialize the variables needed for implicit solver
L06451   s_aw1     =zero
L06452   s_awthl1  =zero
```

```
   s_awqt1   =zero
   s_awqv1   =zero
   s_awqc1   =zero
   s_awu1    =zero
   s_awv1    =zero
   s_awqke1  =zero
   s_awqsq1  =zero
   s_awqnc1  =zero
   s_awqni1  =zero
   s_awqnwfa1=zero
   s_awqnifa1=zero
   s_awqnbca1=zero
   if ( mix_chem ) then
      s_awchem1(kts:kte+1,1:nchem) = zero
   endif
   if ( scalar_opt > 0 ) then
      s_awscalars1(kts:kte+1,1:nscalars) = zero
   endif
   ! Initialize explicit tendencies for subsidence & detrainment
   sub_thl1 = zero
   sub_sqv1 = zero
   sub_u1   = zero
   sub_v1   = zero
   det_thl1 = zero
   det_sqv1 = zero
   det_sqc1 = zero
   det_u1   = zero
   det_v1   = zero
   nup2     = nup !start with nup, but set to zero if activation criteria fails
   tkeprod_up = zero

   if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
      print*,"===incoming forcing in mf component:"
      print*,"  flt=",flt," fltv=",fltv
      print*,"  Psig_shcu=",Psig_shcu," wspd=",sqrt(max(u1(kts)**2 + v1(kts)**2, 0.01_kind_phys))
   endif

   ! Taper off MF scheme when significant resolved-scale motions
   ! are present This function needs to be asymetric...
   maxw        = zero
   cloud_base  = 9000.0_kind_phys
   qkebl       = zero
   do k=1,kte-1
      zagl = zw1(k) + p5*dz1(k)
      if (zagl > (pblh + 500.)) exit

      wpbl = w1(k)
      if (w1(k) < zero)wpbl = two*w1(k)
      maxw = max(maxw,abs(wpbl))

      !Find highest k-level below 50m AGL
      if (zagl <= 50.)k50=k

      !Establish mean tke in pbl for entrainment
      if (k <= kpbl) then
         qkebl = qkebl + qke1(k)
      endif

      !Search for cloud base
      qc_sgs = max(qc1(k), qc_bl1(k))
      if ((qc_sgs > 1E-5) .and. (cldfra_bl1(k) .ge. p5) .and. cloud_base == 9000.0) then
         cloud_base = zw1(k) !height at interface below cloud mass level
      endif
   enddo
   qkebl = qkebl / real(kpbl, kind=kind_phys)

   !do nothing for small w (< 1 m/s), but linearly taper off for w > 1.0 m/s
   maxw = max(zero, maxw - one)
   Psig_w = max(zero, one - maxw)
   Psig_w = min(Psig_w, Psig_shcu)

   !Completely shut off MF scheme for strong resolved-scale vertical velocities.
   fltv2 = fltv
   if (Psig_w < 1e-2_kind_phys .and. fltv > zero) fltv2 = -1.*fltv

   if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
      print*,"===criteria for small w in pbl:"
      print*,"maxw=",maxw," Psig_w=",Psig_w," k50=",k50
   endif

   ! If surface buoyancy is positive we do integration, otherwise no.
   ! Also, ensure that it is at least slightly superadiabatic up through 50 m
   superadiabatic = .false.
   if ((landsea-1.5).ge.zero) then
      hux = -0.0024  ! WATER   ! dTHv/dz must be < -0.24 K per 100 m.
   else
      hux = -0.0030  ! LAND    ! dTHv/dz must be < -0.30 K per 100 m.
   endif
   tvs    = ts*(one+p608*qv1(kts))
   if ((thv1(kts)-tvs)/(p5*dz1(kts)) < hux) then
      !if superadiabatic at the surface, continue checking all layers below 50 m:
      superadiabatic = .true.
      do k=2,max(2,k50)
         hux = -0.0018  !allow for smaller superadiabatic layers above the surface
         dthvdz = (thv1(k)-thv1(k-1))/(p5*(dz1(k)+dz1(k-1)))
         if (dthvdz < hux) then
            superadiabatic = .true.
         else
            superadiabatic = .false.
            exit
         endif
      enddo
   endif

   if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
      print*," superadiatic=",superadiabatic," hux=",hux
   endif

   ! Determine the numer of updrafts/plumes in the grid column:
```

```
L06562   ! Some of these criteria may be a little redundant but useful for bullet-proofing.
L06563   !   (1) largest plume = 1.2 * dx.
L06564   !   (2) Apply a scale-break, assuming no plumes with diameter larger than 1.1*PBLH can exist.
L06565   !   (3) max plume size beneath clouds deck approx = 0.5 * cloud_base.
L06566   !   (4) add wspd-dependent limit, when plume model breaks down. (hurricanes)
L06567   !   (5) limit to reduce max plume sizes in weakly forced conditions. This is only
L06568   !       meant to "soften" the activation of the mass-flux scheme.
L06569   ! Criteria (1)
L06570   maxwidth_dx = min(dx*dcut, lmax)
L06571   !Criteria (2)
L06572   maxwidth_pbl = min(1.1_kind_phys*pblh, lmax)
L06573   ! Criteria (3)
L06574   if ((landsea-1.5) .lt. zero) then  !land
L06575      maxwidth_cld = min(lmax, max(0.5_kind_phys*cloud_base, 300._kind_phys))
L06576   else                               !water
L06577      maxwidth_cld = min(lmax, max(0.8_kind_phys*cloud_base, 300._kind_phys))
L06578   endif
L06579   ! Criteria (4)
L06580   wspd_pbl=sqrt(max(u1(kts)**2 + v1(kts)**2, 0.01_kind_phys))
L06581   !Note: area fraction (acfac) is modified below
L06582   ! Criteria (5) - function of fltv
L06583   if ((landsea-1.5) .lt. zero) then  !land
L06584      maxwidth_flx = MAX(MIN(1000.*(0.6*tanh((fltv - 0.040)/0.04) + p5),1000._kind_phys), zero)
L06585   else                               !water
L06586      maxwidth_flx = MAX(MIN(1000.*(0.6*tanh((fltv - 0.007)/0.02) + p5),1000._kind_phys), zero)
L06587      !width_flx = MAX(MIN(1000.*(0.6*tanh((fltv - 0.010)/0.025) + .5),1000._kind_phys), zero)
L06588   endif
L06589   maxwidth = MIN(maxwidth_dx, maxwidth_pbl)
L06590   maxwidth = MIN(maxwidth,    maxwidth_cld)
L06591   maxwidth = MIN(maxwidth,    maxwidth_flx)
L06592   minwidth = lmin
L06593
L06594   if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then  !land
L06595      print*,"===limiting factors on plume width:"
L06596      print*,"maxwidth_dx= ",maxwidth_dx," dx=",dx
L06597      print*,"maxwidth_pbl=",maxwidth_pbl," pblh=",pblh
L06598      print*,"maxwidth_cld=",maxwidth_cld," cldbase=",cloud_base
L06599      print*,"maxwidth_flx=",maxwidth_flx," fltv=",fltv
L06600      print*,"===final check for activation criteria:"
L06601      print*,"fltv2=",fltv2," superadiatic=",superadiabatic
L06602      print*,"maxwidth=",maxwidth," minwidth=",minwidth
L06603   endif
L06604
L06605   if (maxwidth .le. minwidth) then ! deactivate MF component
L06606      nup2 = 0
L06607      maxwidth = zero
L06608   endif
L06609
L06610   !Begin plume processing if passes criteria
```

- **清零族**：所有 UP/*edmf*_/s_aw*/sub/det 输出清零——无状态。
- **触发条件之 ①（解析运动衰减）**：PBL+500 m 以下网格尺度 w 的最大值 `maxw`，>1 m/s 线性衰减 `Psig_w = 1−(maxw−1)`、再与 `Psig_shcu`（灰区函数）取小——**可分辨运动强时 MF 退出**；`Psig_w < 1e-2` 且浮力通量为正时干脆把 `fltv2 = −fltv`（负浮力 → 羽流必死）。
- **触发条件之 ②（超绝热判据）**：表面层 `dθ_v/dz < −0.0024`（水）/`−0.0030`（陆） K/m（即 −0.24/−0.30 K/100 m）且 50 m 内持续超绝热（`hux = −0.0018` 上层放宽）——**热泡生成的大气条件**。
- **触发条件之 ③（羽流尺寸五判据）**：`maxwidth = min(dx•1.2, 1.1•PBLH, 0.5(陆)/0.8(水)•云底, fltv 的 tanh 函数, lmax)`——网格可容纳、PBL 深度可容纳、云底以下可容纳、浮力通量足够；`maxwidth ≤ 300 m`（`minwidth`）时 `nup2=0` 关闭 MF。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L6611-L6790
L06611   if ( flt > zero .AND. fltv2 > 0.008 .AND. (maxwidth > minwidth) .AND. superadiabatic) then
L06612
L06613      ! Find coef C for number size density N
L06614      cn = zero
L06615      d  =-1.9_kind_phys  !set d to value suggested by Neggers 2015 (JAMES).
L06616      dl = (maxwidth - minwidth)/real(nup-1,kind=kind_phys)
L06617      do ip=1,NUP
L06618         ! diameter of plume
L06619         l = minwidth + dl*real(ip-1,kind=kind_phys)
L06620         cn = cn + l**d * (l*l)/(dx*dx) * dl  ! sum fractional area of each plume
L06621      enddo
L06622      C = Atot/cn   !Normalize C according to the defined total fraction (Atot)
L06623
L06624      ! Make updraft area (UPA) a function of the buoyancy flux
L06625      if ((landsea-1.5) .lt. zero) then  !land
L06626         acfac = p5*tanh((fltv2 - 0.02_kind_phys)/0.05_kind_phys) + p5
L06627      else
L06628         acfac = p5*tanh((fltv2 - 0.012_kind_phys)/0.03_kind_phys) + p5
L06629      endif
L06630
L06631      !For hurricane tuning, add a windspeed-dependent adjustment to acfac that tapers off
L06632      !the mass-flux scheme linearly above sfc wind speeds of 13 m/s.
L06633      ac_wsp = one - min((max(wspd_pbl - 13.0_kind_phys, zero))/12._kind_phys, one)
L06634      acfac  = min(acfac, ac_wsp)
L06635
L06636      ! Find the portion of the total fraction (Atot) of each plume size:
L06637      An2 = zero
L06638      do ip=1,nup
L06639         ! diameter of plume
L06640         l  = minwidth + dl*real(ip-1,kind=kind_phys)
L06641         N  = C*l**d                          ! number density of plume n
L06642         UPA(1,ip) = N*l*l/(dx*dx) * dl       ! fractional area of plume n
L06643
L06644         UPA(1,ip) = UPA(1,ip)*acfac
L06645         An2 = An2 + UPA(1,ip)                ! total fractional area of all plumes
L06646         !if (upa(i,i)<zero)print*,"Neg plume area: plume size=",l," dl=",dl,&
```

```
L06647         !    " area=",UPA(1,I),"; total=",An2," acfac=",acfac," N=",N
L06648       end do
L06649
L06650       if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L06651          print*,"===limiting factors on fractional area coverage:"
L06652          print*,"total araa frac=",An2," atot=",atot
L06653          print*,"upa=",UPA(1,1:nup)
L06654          print*,"acfac fluxes=",acfac," fltv=",fltv
L06655          print*,"acfac wspd=",ac_wsp," wspd_pbl=",wspd_pbl
L06656       endif
L06657
L06658       ! set initial conditions for updrafts
L06659       z0    =50._kind_phys
L06660       pwmin =0.1_kind_phys    ! was 0.5
L06661       pwmax =0.4_kind_phys    ! was 3.0
L06662
L06663       wstar =max(1.E-2_kind_phys,(gtr*fltv2*pblh)**(p333))
L06664       qstar =max(flq,1e-5_kind_phys)/wstar
L06665       thstar=flt/wstar
L06666
L06667       csigma=1.34_kind_phys
L06668
L06669       if (env_subs) then
L06670          exc_fac = zero
L06671       else
L06672          if ((landsea-1.5).GE.zero) then
L06673             !water: increase factor to compensate for decreased pwmin/pwmax
L06674             !0.58*25 = 14.5
L06675             !0.58*20 = 11.6
L06676             !0.58*16 = 9.28
L06677             !0.58*10 = 5.8
L06678             !0.58*5  = 2.9
L06679             exc_fac = 9.28_kind_phys
L06680          else
L06681             !land: no need to increase factor - already sufficiently large superadiabatic layers
L06682             exc_fac = 0.58_kind_phys
L06683          endif
L06684       endif
L06685       !decrease excess for large wind speeds
L06686       exc_fac = exc_fac * ac_wsp
L06687
L06688       !Note: sigmaW is typically about 0.5*wstar
L06689       sigmaW =csigma*wstar*(z0/pblh)**(p333)*(one - 0.8_kind_phys*z0/pblh)
L06690       sigmaQT=csigma*qstar*(z0/pblh)**(p333)
L06691       sigmaTH=csigma*thstar*(z0/pblh)**(p333)
L06692
L06693       !Note: Given the pwmin & pwmax set above, these max/mins are
L06694       !      rarely exceeded.
L06695       wmin=MIN(sigmaW*pwmin,0.1_kind_phys)
L06696       wmax=MIN(sigmaW*pwmax,0.5_kind_phys)
L06697
L06698       if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L06699          print*,"===excess components:"
L06700          print*,"wstar=",wstar,"qstar=",qstar,"thstar=",thstar
L06701          print*,"csigma=",csigma,"exc_fac=",exc_fac,"wmin=",wmin
L06702          print*,"sigmaw=",sigmaw,"sigmaqt=",sigmaqt,"sigmath=",sigmath
L06703       endif
L06704
L06705       !SPECIFY UPDRAFT PROPERTIES AT MODEL INTERFACE BETWEEN K = 1 & 2
L06706       do ip=1,NUP
L06707          wlv=wmin+(wmax-wmin)/real(NUP2,kind=kind_phys)*real(ip-1,kind=kind_phys)
L06708          UPA(2,ip)    =UPA(1,ip)
L06709          !SURFACE UPDRAFT VERTICAL VELOCITY
L06710          UPW(2,ip)    =wmin + real(ip,kind=kind_phys)/real(NUP)*(wmax-wmin)
L06711          UPU(2,ip)    =(u1(kts)    *dz1(kts+1)+u1(kts+1)    *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06712          UPV(2,ip)    =(v1(kts)    *dz1(kts+1)+v1(kts+1)    *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06713          UPQC(2,ip)   =zero
L06714          !UPQC(1,ip)  =(qc1(kts)*dz1(kts+1)+qc1(kts+1)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06715
L06716          exc_heat     =exc_fac*UPW(2,ip)*sigmaTH/sigmaW
L06717          UPTHV(2,ip)  =(thv1(kts)  *dz1(kts+1)+thv1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1)) &
L06718              &         + exc_heat
L06719          UPTHL(2,ip)  =(thl1(kts)  *dz1(kts+1)+thl1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1)) &
L06720              &         + exc_heat
L06721          !calculate exc_moist by use of surface fluxes
L06722          exc_moist    =exc_fac*UPW(2,ip)*sigmaQT/sigmaW
L06723          UPQT(2,ip)   =(qt1(kts)   *dz1(kts+1)+qt1(kts+1)   *dz1(kts))/(dz1(kts)+dz1(kts+1))&
L06724              &         + exc_moist
L06725          UPQKE(2,ip)  =(qke1(kts)  *dz1(kts+1)+qke1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06726          UPQSQ(2,ip)  =(qsq1(kts)  *dz1(kts+1)+qsq1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06727          UPQNC(2,ip)  =(qnc1(kts)  *dz1(kts+1)+qnc1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06728          UPQNI(2,ip)  =(qni1(kts)  *dz1(kts+1)+qni1(kts+1)  *dz1(kts))/(dz1(kts)+dz1(kts+1))
L06729          UPQNWFA(2,ip)=(qnwfa1(kts)*dz1(kts+1)+qnwfa1(kts+1)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06730          UPQNIFA(2,ip)=(qnifa1(kts)*dz1(kts+1)+qnifa1(kts+1)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06731          UPQNBCA(2,ip)=(qnbca1(kts)*dz1(kts+1)+qnbca1(kts+1)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06732       enddo
L06733
L06734       if ( mix_chem ) then
L06735         do ip=1,NUP
L06736           do ic = 1,nchem
L06737             UPCHEM(2,ip,ic)=(chem1(kts,ic)*dz1(kts+1)+chem1(kts+1,ic)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06738           enddo
L06739         enddo
L06740       endif
L06741       if ( scalar_opt > 0 ) then
L06742         do ip=1,NUP
L06743           do ic = 1,nscalars
L06744             upscalars(2,ip,ic)=(scalars(kts,ic)*dz1(kts+1)+scalars(kts+1,ic)*dz1(kts))/(dz1(kts)+dz1(kts+1))
L06745           enddo
L06746         enddo
L06747      endif
L06748
L06749       !Initialize environmental variables which can be modified by detrainment
L06750       envm_thl(kts:kte)=thl1(kts:kte)
L06751       envm_sqv(kts:kte)=qv1(kts:kte)
L06752       envm_sqc(kts:kte)=qc1(kts:kte)
L06753       envm_u(kts:kte)  =u1(kts:kte)
L06754       envm_v(kts:kte)  =v1(kts:kte)
```

```
L06755      !Interpolate some mass-layer variables to interface variables
L06756      rhoi(kts)   = rho1(kts)
L06757      dzi(kts)    = p5*dz1(kts)
L06758      exneri(kts) = ex1(kts)
L06759      do k=kts+1,kte
L06760         rhoi(k)  = (rho1(k-1)*dz1(k)+rho1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06761         dzi(k)   = (dz1(k-1)*dz1(k)+dz1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06762         exneri(k)= (ex1(k-1)*dz1(k)+ex1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06763      enddo
L06764
L06765      !dxsa is scale-adaptive factor governing the pressure-gradient term of the momentum transport
L06766      dxsa = one - MIN(MAX((12000.0_kind_phys-dx)/(12000.0_kind_phys-3000.0_kind_phys), zero), one)
L06767
L06768      ktop_plume = 0
L06769      ! do integration  updraft
L06770      do ip=1,NUP
L06771         QCn = zero
L06772         overshoot = 0 !int
L06773         l  = minwidth + dl*real(ip-1,kind=kind_phys)    ! diameter of plume
L06774         do k=kts+1,kte-2
L06775            zagl = zw1(k) + p5*dz1(k)
L06776            !Entrainment from Tian and Kuang (2016)
L06777            !ENT(k,ip) = 0.35/(MIN(MAX(UPW(K-1,ip),0.75),1.9)*l)
L06778            wmin   = p3 + l*0.0005_kind_phys
L06779            !ENT(k,ip) = 0.33_kind_phys/(MIN(MAX(UPW(k,ip),wmin),one)*l)
L06780            !ENT(k,ip) = (0.20*sqrt(qkebl))/(MIN(MAX(UPW(K-1,ip),wmin),one)*l)
L06781            ! 0.34 ~ 1.62
L06782            ! 0.33 ~ 1.57
L06783            ! 0.30 ~ 1.43
L06784            ! 0.28 ~ 1.33
L06785            ! 0.26 ~ 1.24
L06786            entfac = 0.21_kind_phys * min(1.57_kind_phys, max(1.30_kind_phys, sqrt(qkebl)))
L06787            !entfac = 0.33_kind_phys
L06788            !make entfac tend to original value (0.33) above the pblh:
L06789            wt2    = min(one, max(zero, zagl - pblh)/500._kind_phys) !0 in pbl, 1 aloft
L06790            entfac = entfac*(one-wt2) + wt2*0.33_kind_phys
```

- **谱分配**：直径从 minwidth 到 maxwidth 均分 8 档（`dl`）；数密度 `N = C • l^d`，C 由总面积 = Atot 归一（L6614–L6622）；**面积分数再乘 `acfac`**——浮力通量的 tanh 函数（陆地阈值 0.02、水面 0.012 K • m/s），并叠加飓风衰减 `ac_wsp`（表面风 >13 m/s 线性关断，12 m/s 跨度）。
- **羽流初值**（50 m 参考高度 `z0`）：`wstar = (g/θ • fltv • PBLH)^(1/3)`（对流速度尺度）、`qstar/thstar = 通量/wstar`（Deardorff 尺度）；σ 因子 `csigma=1.34、(z0/PBLH)^(1/3) • (1−0.8 • z0/PBLH)` 剖面；**超绝热过量 `exc_fac`**——水面 9.28 / 陆地 0.58（注释列出 0.58×{25,20,16,10,5} 的演化史）——**羽流在 50 m 处的 θ/Qt 超量** = 环境值 + exc_fac • w • σθ/σw。
- **八个羽流的 w 分布**：`wmin~wmax`（σw 的 0.1~0.4 倍，钳 [0.1, 0.5] m/s）均分——**每羽流初始 w 不同，速度快的大羽流升得高**。
- **卷入率**（Tian & Kuang 2016）：`ENT = entfac/(max(w, wmin) • l)`，`entfac = 0.21 • f(sqrt(qkebl))∈[1.30,1.57]`（PBL 内）向 0.33 过渡（PBL 上 500 m 内）——**卷入反比于速度 × 直径**（大而快的羽流卷入少）；另有高层增强（>PBL+1500 m 线性加 5e-6 • dz）、SPP 扰动、上限 0.9/dz。被注释的 Neggers (2015) 幂律形式（`0.02 • l^−0.35`）保留考古。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L6791-L7030
L06791            ENT(k,ip) = entfac/(MIN(MAX(UPW(K-1,ip),wmin),one)*l)
L06792
L06793            !Entrainment from Negggers (2015, JAMES)
L06794            !ENT(k,ip) = 0.02*l**-0.35 - 0.0009
L06795            !ENT(k,ip) = 0.04*l**-0.50 - 0.0009   !more plume diversity
L06796            !ENT(k,ip) = 0.04*l**-0.495 - 0.0009  !"neg1+"
L06797
L06798            !Minimum background entrainment
L06799            ENT(k,ip) = max(ENT(k,ip),0.0003_kind_phys)
L06800            !ENT(k,ip) = max(ENT(k,ip),0.05/zw1(k))  !not needed for Tian and Kuang
L06801
L06802            !increase entrainment for plumes extending very high.
L06803            IF(zw1(k) >= MIN(pblh+1500., 4000.))THEN
L06804              ENT(k,ip)=ENT(k,ip) + (zw1(k)-MIN(pblh+1500.,4000.))*5.0E-6
L06805            ENDIF
L06806
L06807            !SPP
L06808            ENT(k,ip) = ENT(k,ip) * (one - pattern_spp_pbl1(k))
L06809
L06810            ENT(k,ip) = min(ENT(k,ip), 0.9_kind_phys/dz1(k))
L06811
L06812            ! Define environment U & V at the model interface levels
L06813            Uk     =(u1(k)*dz1(k+1)+u1(k+1)*dz1(k))/(dz1(k+1)+dz1(k))
L06814            Ukm1   =(u1(k-1)*dz1(k)+u1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06815            Vk     =(v1(k)*dz1(k+1)+v1(k+1)*dz1(k))/(dz1(k+1)+dz1(k))
L06816            Vkm1   =(v1(k-1)*dz1(k)+v1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06817
L06818            ! Linear entrainment:
L06819            EntExp =ENT(k,ip)*dz1(k)
L06820            EntExm =EntExp*p333    !reduce entrainment for momentum
L06821            QTn    =UPQT(k,IP)   *(one-EntExp) + qt1(k)*EntExp
L06822            THLn   =UPTHL(k,IP)  *(one-EntExp) + thl1(k)*EntExp
L06823            Un     =UPU(k,IP)    *(one-EntExm) + u1(k)*EntExm + dxsa*pgfac*(Uk - Ukm1)
L06824            Vn     =UPV(k,IP)    *(one-EntExm) + v1(k)*EntExm + dxsa*pgfac*(Vk - Vkm1)
L06825            QKEn   =UPQKE(k,IP)  *(one-EntExp) + qke1(k)*EntExp
L06826            QSQn   =UPQSQ(k,IP)  *(one-EntExp) + qsq1(k)*EntExp
L06827            QNCn   =UPQNC(k,IP)  *(one-EntExp) + qnc1(k)*EntExp
L06828            QNIn   =UPQNI(k,IP)  *(one-EntExp) + qni1(k)*EntExp
L06829            QNWFAn =UPQNWFA(k,IP)*(one-EntExp) + qnwfa1(k)*EntExp
L06830            QNIFAn =UPQNIFA(k,IP)*(one-EntExp) + qnifa1(k)*EntExp
L06831            QNBCAn =UPQNBCA(k,IP)*(one-EntExp) + qnbca1(k)*EntExp
L06832
L06833            !capture the updated qc, qt & thl modified by entranment alone,
```

```
L06834            !since they will be modified later if condensation occurs.
L06835            qc_ent =QCn
L06836            qt_ent =QTn
L06837            thl_ent=THLn
L06838
L06839            ! Exponential Entrainment:
L06840            !EntExp= exp(-ENT(K,IP)*(zw1(k)-zw1(k-1)))
L06841            !QTn =qt1(K) *(1-EntExp)+UPQT(K-1,IP)*EntExp
L06842            !THLn=thl1(K)*(1-EntExp)+UPTHL(K-1,IP)*EntExp
L06843            !Un  =u1(K)  *(1-EntExp)+UPU(K-1,IP)*EntExp
L06844            !Vn  =v1(K)  *(1-EntExp)+UPV(K-1,IP)*EntExp
L06845            !QKEn=qke1(k)*(1-EntExp)+UPQKE(K-1,ip)*EntExp
L06846
L06847            if ( mix_chem ) then
L06848              do ic = 1,nchem
L06849                ! Exponential Entrainment:
L06850                !chemn(ic) = chem(k,ic)*(1-EntExp)+UPCHEM(K-1,ip,ic)*EntExp
L06851                ! Linear entrainment:
L06852                chemn(ic)=UPCHEM(k,ip,ic)*(one-EntExp) + chem1(k,ic)*EntExp
L06853              enddo
L06854            endif
L06855            if ( scalar_opt > 0 ) then
L06856              do ic = 1,nscalars
L06857                scalarsn(ic)=upscalars(k,ip,ic)*(one-EntExp) + scalars(k,ic)*EntExp
L06858              enddo
L06859            endif
L06860
L06861            ! Define pressure at model interface we just integrated to (k+1).
L06862            Pk    =(pres1(k)*dz1(k+1)+pres1(k+1)*dz1(k))/(dz1(k)+dz1(k+1))
L06863            ! Compute plume properties thvn and qcn
L06864            call condensation_edmf(QTn,THLn,Pk,zw1(k+1),THVn,QCn)
L06865
L06866            ! Define environment THV at the model interface levels k & k+1
L06867            THVk  =(thv1(k)*dz1(k+1)+thv1(k+1)*dz1(k))/(dz1(k+1)+dz1(k))
L06868            THVkm1=(thv1(k-1)*dz1(k)+thv1(k)*dz1(k-1))/(dz1(k-1)+dz1(k))
L06869
L06870  !          B=g*(0.5*(THVn+UPTHV(k-1,ip))/thv1(k-1) - one)
L06871            B=grav*(THVn/THVk - one)
L06872            IF(B>0.)THEN
L06873              BCOEFF = 0.15        !w typically stays < 2.5, so doesnt hit the limits nearly as much
L06874            ELSE
L06875              BCOEFF = 0.2 !0.33
L06876            ENDIF
L06877
L06878            ! Original StEM with exponential entrainment
L06879            !EntW=exp(-2.*(Wb+Wc*ENT(K,ip))*(zw1(k)-zw1(k-1)))
L06880            !Wn2=UPW(K-1,ip)**2*EntW + (1.-EntW)*0.5*Wa*B/(Wb+Wc*ENT(K,ip))
L06881            ! Original StEM with linear entrainment
L06882            !Wn2=UPW(K-1,ip)**2*(1.-EntExp) + EntExp*0.5*Wa*B/(Wb+Wc*ENT(K,ip))
L06883            !Wn2=MAX(Wn2,zero)
L06884            !WA: TEMF form
L06885            !IF (B>zero .AND. UPW(K-1,ip) < 0.2 ) THEN
L06886            IF (UPW(K-1,ip) < 0.2 ) THEN
L06887               Wn = UPW(k,ip) + (-2. * ENT(k,ip) * UPW(k,ip) + BCOEFF*B / MAX(UPW(k,ip),0.2)) * MIN(dz1(k), 250.)
L06888            ELSE
L06889               Wn = UPW(k,ip) + (-2. * ENT(k,ip) * UPW(k,ip) + BCOEFF*B / UPW(k,ip)) * MIN(dz1(k), 250.)
L06890            ENDIF
L06891            !Do not allow a parcel to accelerate more than 1.25 m/s over 200 m.
L06892            !Add max increase of 2.0 m/s for coarse vertical resolution.
L06893            IF(Wn > UPW(k,ip) + MIN(1.25*dz1(k)/200., two) ) THEN
L06894               Wn = UPW(k,ip) + MIN(1.25*dz1(k)/200., two)
L06895            ENDIF
L06896            !Add symmetrical max decrease in w
L06897            IF(Wn < UPW(k,ip) - MIN(1.25*dz1(k)/200., two) ) THEN
L06898               Wn = UPW(k,ip) - MIN(1.25*dz1(k)/200., two)
L06899            ENDIF
L06900            Wn = MIN(MAX(Wn, zero), 3.0_kind_phys)
L06901
L06902            !Check to make sure that the plume made it up at least one level.
L06903            !if it failed, then set nup2=0 and exit the mass-flux portion.
L06904            if (k==kts+1 .and. wn < 1e-8) then
L06905               nup2=0
L06906               exit
L06907            endif
L06908
L06909            if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L06910              if (wn .ge. 3.0) then
L06911                ! surface values
L06912                print *," **** SUSPICIOUSLY LARGE W:"
L06913                print *,' QCn:',QCn,' ENT=',ENT(k,ip),' Nup2=',Nup2
L06914                print *,'pblh:',pblh,' Wn:',Wn,' UPW(k)=',UPW(k,ip)
L06915                print *,'K=',k,' B=',B,' dz=',dz1(k)
L06916              ENDIF
L06917            ENDIF
L06918
L06919            !Allow strongly forced plumes to overshoot if KE is sufficient
L06920            IF (Wn <= zero .AND. overshoot == 0) THEN
L06921               overshoot = 1
L06922               IF ( THVk-THVkm1 .GT. zero ) THEN
L06923                  bvf = SQRT( gtr*(THVk-THVkm1)/dz1(k) )
L06924                  !vertical Froude number
L06925                  Frz = UPW(k,ip)/(bvf*dz1(k))
L06926                  !IF ( Frz >= 0.5 ) Wn =  MIN(Frz,one)*UPW(K-1,I)
L06927                  dzp = dz1(k)*MAX(MIN(Frz,one),zero) ! portion of highest layer the plume penetrates
L06928               ENDIF
L06929            ELSE
L06930               dzp = dz1(k)
L06931            ENDIF
L06932
L06933            !minimize the plume penetratration in stratocu-topped PBL
L06934            !IF (fltv2 < 0.06) THEN
L06935            !   IF(ZW(k+1) >= pblh-200. .AND. qc1(k) > 1e-5 .AND. ip > 4) Wn=zero
L06936            !ENDIF
L06937
L06938            !Modify environment variables (representative of the model layer - envm*)
L06939            !following the updraft dynamical detrainment of Asai and Kasahara (1967, JAS).
L06940            !Reminder: w is limited to be non-negative (above)
L06941            aratio   = MIN(UPA(k,IP)/(one-UPA(k,IP)), p5) !limit should never get hit
```

```
L06942            detturb  = 0.00008
L06943            oow      = -0.060/MAX(one,(0.5*(Wn+UPW(k,IP))))   !coef for dynamical detrainment rate
L06944            detrate  = MIN(MAX(oow*(Wn-UPW(k,IP))/dz1(k), detturb), .0002) ! dynamical detrainment rate (m^-1)
L06945            detrateUV= MIN(MAX(oow*(Wn-UPW(k,IP))/dz1(k), detturb), .0001) ! dynamical detrainment rate (m^-1)
L06946            envm_thl(k)=envm_thl(k) + (p5*(thl_ent + UPTHL(k,IP)) - thl1(k))*detrate*aratio*MIN(dzp,dzpmax)
L06947            qv_ent = p5*(MAX(qt_ent-qc_ent,zero) + MAX(UPQT(k,IP)-UPQC(k,IP),zero))
L06948            envm_sqv(k)=envm_sqv(k) + (qv_ent-qv1(K))*detrate*aratio*MIN(dzp,dzpmax)
L06949            IF (UPQC(k,IP) > 1E-8) THEN
L06950               IF (qc1(k) > 1E-6) THEN
L06951                  qc_grid = qc1(k)
L06952               ELSE
L06953                  qc_grid = qc_bl1(K)
L06954               ENDIF
L06955               envm_sqc(k)=envm_sqc(k) + MAX(UPA(k,IP)*p5*(QCn + UPQC(k,IP)) - qc_grid,
↪  zero)*detrate*aratio*MIN(dzp,dzpmax)
L06956            ENDIF
L06957            envm_u(k)  =envm_u(k)   + (p5*(Un + UPU(k,IP)) - u1(K))*detrateUV*aratio*MIN(dzp,dzpmax)
L06958            envm_v(k)  =envm_v(k)   + (p5*(Vn + UPV(k,IP)) - v1(K))*detrateUV*aratio*MIN(dzp,dzpmax)
L06959
L06960            IF (Wn > 0.) THEN
L06961               !Update plume variables at the k+1 index we just integrated to.
L06962               UPW(k+1,IP)=Wn  !sqrt(Wn2)
L06963               UPTHV(k+1,IP)=THVn
L06964               UPTHL(k+1,IP)=THLn
L06965               UPQT(k+1,IP)=QTn
L06966               UPQC(k+1,IP)=QCn
L06967               UPU(k+1,IP)=Un
L06968               UPV(k+1,IP)=Vn
L06969               UPQKE(k+1,IP)=QKEn
L06970               UPQSQ(k+1,IP)=QSQn
L06971               UPQNC(k+1,IP)=QNCn
L06972               UPQNI(k+1,IP)=QNIn
L06973               UPQNWFA(k+1,IP)=QNWFAn
L06974               UPQNIFA(K+1,IP)=QNIFAn
L06975               UPQNBCA(k+1,IP)=QNBCAn
L06976               UPA(k+1,IP)=UPA(K,IP)
L06977               if ( mix_chem ) then
L06978                 do ic = 1,nchem
L06979                   UPCHEM(k+1,ip,ic) = chemn(ic)
L06980                 enddo
L06981               endif
L06982               if ( scalar_opt > 0 ) then
L06983                 do ic = 1,nscalars
L06984                   upscalars(k+1,ip,ic) = scalarsn(ic)
L06985                 enddo
L06986               endif
L06987               ktop = MAX(ktop,k)
L06988            ELSE
L06989               if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L06990                  print*,"plume #:",ip," ktop=",ktop
L06991                  print*,"area=",UPA(1:ktop,ip)
L06992                  print*,"w=",UPW(1:ktop,ip)
L06993               endif
L06994               exit  !exit k-loop
L06995            END IF
L06996         ENDDO !end k-loop
L06997
L06998         IF (debug_mf == 1 .and. i==idbg .and. j==jdbg) THEN
L06999            IF (MAXVAL(UPW(:,IP)) > 10.0 .OR. MINVAL(UPA(:,IP)) < zero .OR. &
L07000                MAXVAL(UPA(:,IP)) > Atot .OR. NUP2 > 10) THEN
L07001               ! surface values
L07002               print *,'flq:',flq,' fltv:',fltv2,' Nup2=',Nup2
L07003               print *,'pblh:',pblh,' wstar:',wstar,' ktop=',ktop
L07004               print *,'sigmaW=',sigmaW,' sigmaTH=',sigmaTH,' sigmaQT=',sigmaQT
L07005               ! means
L07006               print *,'u:',u1
L07007               print *,'v:',v1
L07008               print *,'thl:',thl1
L07009               print *,'UPA:',UPA(:,IP)
L07010               print *,'UPW:',UPW(:,IP)
L07011               print *,'UPTHL:',UPTHL(:,IP)
L07012               print *,'UPQT:',UPQT(:,IP)
L07013               print *,'ENT:',ENT(:,ip)
L07014            ENDIF
L07015         ENDIF
L07016         ktop_plume(ip)=k !index where each individual plume stopped rising.
L07017         excess_h =max(excess_h,exc_heat)
L07018         excess_q =max(excess_q,exc_moist)
L07019      ENDDO !end plume # loop
L07020   ELSE
L07021      !At least one of the conditions was not met for activating the MF scheme.
L07022      NUP2=0.
L07023   END IF !end criteria check for mass-flux scheme
L07024
L07025   ktop=MIN(ktop,KTE-1)
L07026   IF (ktop == 0) THEN
L07027      ztop = zero
L07028   ELSE
L07029      ztop = zw1(ktop)
L07030   ENDIF
```

- **逐层积分**（每羽流独立）：线性卷入 `EntExp = ENT•dz`，`QTn/THLn/QKEn/… = UP•(1−EntExp) + 环境•EntExp`；**动量卷入只取 1/3**（`EntExm = EntExp^(1/3)`——气压梯度与扰动压强使动量混合弱于标量）；动量另有 `dxsa•pgfac•du` 气压梯度项（pgfac=0 关闭）。
- **干湿判定**：`condensation_edmf(QTn, THLn, P, z, →THVn, QCn)`——羽流的饱和调整（§09b.7）。
- **垂直速度方程**（TEMF 形式）：`Wn = w + (−2•ENT•w + BCOEFF•B/w)•min(dz,250)`——卷入减速 + 浮力加速；低速羽流（w<0.2）改用 `BCOEFF•B/max(w,0.2)` 防除零（BCOEFF 浮力 >0 取 0.15、<0 取 0.2——**负浮力羽流减速更快**）；单层加速限 ±1.25 m/s per 200 m（粗网格放宽到 2.0）；`Wn` 钳 [0, 3]。

- **羽流穿越（overshoot）**：w≤0 且首次出现时算 **Froude 数** `Frz = w/(bv·dz)`，羽流以 `dzp = dz·min(Frz,1)` 的份额穿入稳定层——**动能换位能的层内穿透**。
- **动力夹带/环境修正**（Asai & Kasahara 1967）：`detrate = clamp(−0.06/w̄·(Wn−w_prev)/dz, 8e-5, 2e-4)`——羽流减速时把 “羽流-环境差” 按夹带率注入环境状态 `envm_*`（供 det 倾向用）；动量夹带率减半（detrateUV ≤ 1e-4）。
- **羽流终止**：`Wn ≤ 0` 后退出 k 循环（记录 `ktop_plume(ip)`）；首层即死（w<1e-8）则整个 MF 关闭（nup2=0）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7032-L7494
L07032   IF (nup2 > 0) THEN
L07033      !Calculate the fluxes for each variable
L07034      !All s_aw* variable are == 0 at k=1
L07035      DO ip=1,NUP
L07036         DO k=kts,kte-1
L07037            s_aw1(k)   = s_aw1(k)    + rhoi(k)*UPA(K,ip)*UPW(K,ip)*Psig_w
L07038            s_awthl1(k)= s_awthl1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPTHL(K,ip)*Psig_w
L07039            s_awqt1(k) = s_awqt1(k)  + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQT(K,ip)*Psig_w
L07040            !to conform to grid mean properties, move qc to qv in grid mean
L07041            !saturated layers, so total water fluxes are preserved but
L07042            !negative qc fluxes in unsaturated layers is reduced.
L07043   !        if (qc1(k) > 1e-12 .or. qc1(k+1) > 1e-12) then
L07044               qc_plume = UPQC(K,ip)
L07045   !        else
L07046   !           qc_plume = zero
L07047   !        endif
L07048            s_awqc1(k) = s_awqc1(k)  + rhoi(k)*UPA(K,ip)*UPW(K,ip)*qc_plume*Psig_w
L07049            s_awqv1(k) = s_awqt1(k)  - s_awqc1(k)
L07050         ENDDO
L07051      ENDDO
L07052      !momentum
L07053      if (momentum_opt > 0) then
L07054         do ip=1,nup
L07055            do k=kts,kte-1
L07056               s_awu1(k) = s_awu1(k)   + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPU(K,ip)*Psig_w
L07057               s_awv1(k) = s_awv1(k)   + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPV(K,ip)*Psig_w
L07058            enddo
L07059         enddo
L07060      endif
L07061      !tke
L07062      if (tke_opt > 0) then
L07063         do ip=1,nup
L07064            do k=kts,kte-1
L07065               s_awqke1(k)= s_awqke1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQKE(K,ip)*Psig_w
L07066            enddo
L07067         enddo
L07068         !qsq
L07069         if (bl_mynn_closure > 2.5) then
L07070            do ip=1,nup
L07071               do k=kts,kte-1
L07072                  s_awqsq1(k)= s_awqsq1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQSQ(K,ip)*Psig_w
L07073               enddo
L07074            enddo
L07075         endif
L07076      endif
L07077      !chem
L07078      if ( mix_chem ) then
L07079         do k=kts,kte-1
L07080            do ip=1,nup
L07081               do ic = 1,nchem
L07082                  s_awchem1(k,ic) = s_awchem1(k,ic) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPCHEM(K,ip,ic)*Psig_w
L07083               enddo
L07084            enddo
L07085         enddo
L07086      endif
L07087      !scalars
L07088      if ( scalar_opt > 0 ) then
L07089         do k=kts,kte-1
L07090            do ip=1,nup
L07091               do ic = 1,nscalars
L07092                  s_awscalars1(k,ic) = s_awscalars1(k,ic) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*upscalars(k,ip,ic)*Psig_w
L07093               enddo
L07094            enddo
L07095         enddo
L07096      endif
L07097      if (numcon_opt > 0) then
L07098         do ip=1,nup
L07099            do k=kts,kte-1
L07100               s_awqnc1(k)  = s_awqnc1(k)   + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQNC(K,ip)*Psig_w
L07101               s_awqni1(k)  = s_awqni1(k)   + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQNI(K,ip)*Psig_w
L07102            enddo
L07103         enddo
L07104      endif
L07105      if (aerosol_opt > 0) then
L07106         do ip=1,nup
L07107            do k=kts,kte-1
L07108               s_awqnwfa1(k)= s_awqnwfa1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQNWFA(K,ip)*Psig_w
L07109               s_awqnifa1(k)= s_awqnifa1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQNIFA(K,ip)*Psig_w
L07110               s_awqnbca1(k)= s_awqnbca1(k) + rhoi(k)*UPA(K,ip)*UPW(K,ip)*UPQNBCA(K,ip)*Psig_w
L07111            enddo
L07112         enddo
L07113      endif
L07114
L07115      !Flux limiter: Check ratio of heat flux at top of first model layer
L07116      !and at the surface. Make sure estimated flux out of the top of the
L07117      !layer is < fluxportion*surface_heat_flux
L07118      IF (abs(s_aw1(kts+1)) > 1e-8_kind_phys) THEN
L07119         flx1 = max(s_aw1(kts+1)*(thv1(kts)-thv1(kts+1))/dzi(kts+1),1.0e-6_kind_phys)
L07120      ELSE
L07121         flx1 = zero
L07122         !print*,"ERROR: s_aw(kts+1) == 0, NUP=",NUP," NUP2=",NUP2,&
L07123         !       " superadiabatic=",superadiabatic," KTOP=",KTOP
L07124      ENDIF
L07125      adjustment=one
```

```
L07126     flt2=max(fltv,zero)
L07127     !Print*,"Flux limiter in MYNN-EDMF, adjustment=",fluxportion*flt/dz(kts)/flx1
L07128     !Print*,"flt/dz=",flt/dz(kts),"  flx1=",flx1," s_aw(kts+1)=",s_aw(kts+1)
L07129     IF (flx1 > fluxportion*flt2/dz1(kts) .AND. flx1>zero) THEN
L07130        adjustment = max(0.01, fluxportion*flt2/dz1(kts)/flx1)
L07131        s_aw1      = s_aw1*adjustment
L07132        s_awthl1   = s_awthl1*adjustment
L07133        s_awqt1    = s_awqt1*adjustment
L07134        s_awqc1    = s_awqc1*adjustment
L07135        s_awqv1    = s_awqv1*adjustment
L07136        if (numcon_opt > 0) then
L07137           s_awqnc1   = s_awqnc1*adjustment
L07138           s_awqni1   = s_awqni1*adjustment
L07139        endif
L07140        if (aerosol_opt > 0) then
L07141           s_awqnwfa1 = s_awqnwfa1*adjustment
L07142           s_awqnifa1 = s_awqnifa1*adjustment
L07143           s_awqnbca1 = s_awqnbca1*adjustment
L07144        endif
L07145        IF (momentum_opt > 0) THEN
L07146           s_awu1  = s_awu1*adjustment
L07147           s_awv1  = s_awv1*adjustment
L07148        ENDIF
L07149        IF (tke_opt > 0) THEN
L07150           s_awqke1= s_awqke1*adjustment
L07151           if (bl_mynn_closure>2.5) s_awqsq1= s_awqsq1*adjustment
L07152        ENDIF
L07153        IF ( mix_chem ) THEN
L07154           s_awchem1 = s_awchem1*adjustment
L07155        ENDIF
L07156        IF ( scalar_opt > 0 ) THEN
L07157           s_awscalars1 = s_awscalars1*adjustment
L07158        ENDIF
L07159        UPA = UPA*adjustment
L07160     ENDIF
L07161     if (debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L07162        print*,"Flux adjustment:"
L07163        print*,"adjustment=",adjustment," fluxportion=",fluxportion
L07164        print*,"flt2=",flt2," flx1=",flx1
L07165     endif
L07166
L07167     !Calculate mean updraft properties for output:
L07168     !all edmf_* variables are interpolated plume quantities to mass levels
L07169     k=kts !at first mass level, can not interpolate--use first interface values
L07170     do ip=1,nup
L07171        edmf_a1(k)  =edmf_a1(k)  +upa(k+1,ip)
L07172        edmf_w1(k)  =edmf_w1(k)  +upa(k+1,ip)*p5*UPW(k+1,ip)
L07173        edmf_qt1(k) =edmf_qt1(k) +upa(k+1,ip)*UPQT(k+1,ip)
L07174        edmf_thl1(k)=edmf_thl1(k)+upa(k+1,ip)*UPTHL(k+1,ip)
L07175        edmf_ent1(k)=edmf_ent1(k)+upa(k+1,ip)*ENT(k+1,ip)
L07176        edmf_qc1(k) =edmf_qc1(k) +upa(k+1,ip)*UPQC(k+1,ip)
L07177     enddo
L07178     do ip=1,nup
L07179        do k=kts+1,ktop_plume(ip)-1  !within the plume, we can interpolate:
L07180           upak        =(upa(k+1,ip)*dzi(k) + upa(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07181           edmf_a1(k)  =edmf_a1(k)  +upak
L07182           edmf_w1(k)  =edmf_w1(k)  +upak*(upw(k+1,ip)*dzi(k)  + upw(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07183           edmf_qt1(k) =edmf_qt1(k) +upak*(upqt(k+1,ip)*dzi(k) + upqt(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07184           edmf_thl1(k)=edmf_thl1(k)+upak*(upthl(k+1,ip)*dzi(k)+ upthl(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07185           edmf_ent1(k)=edmf_ent1(k)+upak*(ent(k+1,ip)*dzi(k)  + ent(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07186           if (abs(upqc(k,ip)) < 1e-10_kind_phys .and. upqc(k+1,ip) > zero) then
L07187              !mass at cloud base is kept equal to the interface value
L07188              edmf_qc1(k) =edmf_qc1(k) +upak*upqc(k+1,ip)
L07189           else
L07190              edmf_qc1(k) =edmf_qc1(k) +upak*(upqc(k+1,ip)*dzi(k) + upqc(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07191           endif
L07192        enddo
L07193     enddo
L07194     !Now use a single level at the top of the plume so there is no averaging with zeros above:
L07195     do ip=1,nup
L07196        k=ktop_plume(ip)
L07197        edmf_a1(k)  =edmf_a1(k)  +upa(k,ip)
L07198        edmf_w1(k)  =edmf_w1(k)  +upa(k,ip)*upw(k,ip)
L07199        edmf_qt1(k) =edmf_qt1(k) +upa(k,ip)*upqt(k,ip)
L07200        edmf_thl1(k)=edmf_thl1(k)+upa(k,ip)*upthl(k,ip)
L07201        edmf_ent1(k)=edmf_ent1(k)+upa(k,ip)*ent(k,ip)
L07202        edmf_qc1(k) =edmf_qc1(k) +upa(k,ip)*upqc(k,ip)
L07203     enddo
L07204     do k=kts,kte-1
L07205        !Note that only edmf_a1 is multiplied by Psig_w. This helps with the scale/regime-awareness.
L07206        if (edmf_a1(k)>0.) then
L07207           edmf_w1(k)  =edmf_w1(k)/edmf_a1(k)
L07208           edmf_qt1(k) =edmf_qt1(k)/edmf_a1(k)
L07209           edmf_thl1(k)=edmf_thl1(k)/edmf_a1(k)
L07210           edmf_ent1(k)=edmf_ent1(k)/edmf_a1(k)
L07211           edmf_qc1(k) =edmf_qc1(k)/edmf_a1(k)
L07212           edmf_a1(k)  =edmf_a1(k)*Psig_w
L07213        endif
L07214     enddo ! end k
↪
L07215     do k=kts,kte-1
L07216        !FIND MAXIMUM MASS-FLUX IN THE COLUMN:
L07217        if(edmf_a1(k)*edmf_w1(k) > maxmf) maxmf = edmf_a1(k)*edmf_w1(k)
L07218        !tke production by udrafts
L07219        !instead of dTKE/dt = 1/2 w^3, multiply by 2 for QKE.
L07220        tkeprod_up(k)=(abs(edmf_w1(k))**3)*edmf_a1(k)/(b1*max(p5*(el1(k)+el1(k+1)),0.1))
L07221     enddo
L07222
L07223     !smoke/chem
L07224     if ( mix_chem ) then
L07225        do k=kts,kte-1
L07226          do ip=1,nup
L07227            do ic = 1,nchem
L07228              upak        =(upa(k+1,ip)*dzi(k) + upa(k,ip)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07229              edmf_chem(k,ic) = edmf_chem(k,ic) + upak*(upchem(k+1,ip,ic)*dzi(k) + upchem(k,ip,
↪  ic)*dzi(k+1))/(dzi(k+1)+dzi(k))
L07230            enddo
L07231          enddo
```

```
L07232        enddo
L07233        do k=kts,kte-1
L07234           if (edmf_a1(k)>0.) then
L07235              do ic = 1,nchem
L07236                 edmf_chem(k,ic) = edmf_chem(k,ic)/edmf_a1(k)
L07237              enddo
L07238           endif
L07239        enddo
L07240     endif
L07241
L07242     !Calculate the effects environmental subsidence.
L07243     !All envi_*variables are valid at the interfaces, like the edmf_* variables
L07244     IF (env_subs) THEN
L07245        DO k=kts+1,kte-1
L07246           !First, smooth the profiles of w & a, since sharp vertical gradients
L07247           !in plume variables are not likely extended to env variables
L07248           !Note1: w is treated as negative further below
L07249           !Note2: both w & a will be transformed into env variables further below
L07250           envi_w(k) = p333*(edmf_w1(k-1)+edmf_w1(k)+edmf_w1(k+1))
L07251           envi_a(k) = p333*(edmf_a1(k-1)+edmf_a1(k)+edmf_a1(k+1))*adjustment
L07252        ENDDO
L07253        !define env variables at k=1 (top of first model layer)
L07254        envi_w(kts) = edmf_w1(kts)
L07255        envi_a(kts) = edmf_a1(kts)
L07256        !define env variables at k=kte
L07257        envi_w(kte) = zero
L07258        envi_a(kte) = edmf_a1(kte)
L07259        !define env variables at k=kte+1
L07260        envi_w(kte+1) = zero
L07261        envi_a(kte+1) = edmf_a1(kte)
L07262        !Add limiter for very long time steps (i.e. dt > 300 s)
L07263        !Note that this is not a robust check - only for violations in
L07264        !   the first model level.
L07265        IF (envi_w(kts) > 0.9*dz1(kts)/dt) THEN
L07266           sublim = 0.9*dz1(kts)/dt/envi_w(kts)
L07267        ELSE
L07268           sublim = one
L07269        ENDIF
L07270        !Transform w & a into env variables
L07271        DO k=kts,kte
L07272           temp=envi_a(k)
L07273           envi_a(k)=one-temp
L07274           envi_w(k)=csub*sublim*envi_w(k)*temp/(1.-temp)
L07275        ENDDO
L07276        !calculate tendencies from subsidence and detrainment valid at the middle of
L07277        !each model layer. The lowest model layer uses an assumes w=0 at the surface.
L07278        sub_thl1(kts)= 0.5*envi_w(kts)*envi_a(kts)*                              &
L07279                       (rho1(kts+1)*thl1(kts+1)-rho1(kts)*thl1(kts))/dzi(kts)/rhoi(kts)
L07280        sub_sqv1(kts)= 0.5*envi_w(kts)*envi_a(kts)*                              &
L07281                       (rho1(kts+1)*qv1(kts+1)-rho1(kts)*qv1(kts))/dzi(kts)/rhoi(kts)
L07282        DO k=kts+1,kte-1
L07283           dzi(k)     = 0.5*(dz1(k)+dz1(k+1))
L07284           sub_thl1(k)= 0.5*(envi_w(k)+envi_w(k-1))*0.5*(envi_a(k)+envi_a(k-1)) * &
L07285                        (rho1(k+1)*thl1(k+1)-rho1(k)*thl1(k))/dzi(k)/rhoi(k)
L07286           sub_sqv1(k)= 0.5*(envi_w(k)+envi_w(k-1))*0.5*(envi_a(k)+envi_a(k-1)) * &
L07287                        (rho1(k+1)*qv1(k+1)-rho1(k)*qv1(k))/dzi(k)/rhoi(k)
L07288        ENDDO
L07289
L07290        DO k=kts,KTE-1
L07291           det_thl1(k)=Cdet*(envm_thl(k)-thl1(k))*envi_a(k)*Psig_w
L07292           det_sqv1(k)=Cdet*(envm_sqv(k)-qv1(k))*envi_a(k)*Psig_w
L07293           det_sqc1(k)=Cdet*(envm_sqc(k)-qc1(k))*envi_a(k)*Psig_w
L07294        ENDDO
L07295
L07296        IF (momentum_opt > 0) THEN
L07297           sub_u1(kts)=0.5*envi_w(kts)*envi_a(kts)*                              &
L07298                       (rho1(kts+1)*u1(kts+1)-rho1(kts)*u1(kts))/dzi(kts)/rhoi(kts)
L07299           sub_v1(kts)=0.5*envi_w(kts)*envi_a(kts)*                              &
L07300                       (rho1(kts+1)*v1(kts+1)-rho1(kts)*v1(kts))/dzi(kts)/rhoi(kts)
L07301           DO k=kts+1,kte-1
L07302              sub_u1(k)=0.5*(envi_w(k)+envi_w(k-1))*0.5*(envi_a(k)+envi_a(k-1)) * &
L07303                        (rho1(k+1)*u1(k+1)-rho1(k)*u1(k))/dzi(k)/rhoi(k)
L07304              sub_v1(k)=0.5*(envi_w(k)+envi_w(k-1))*0.5*(envi_a(k)+envi_a(k-1)) * &
L07305                        (rho1(k+1)*v1(k+1)-rho1(k)*v1(k))/dzi(k)/rhoi(k)
L07306           ENDDO
L07307
L07308           DO k=kts,KTE-1
L07309             det_u1(k) = Cdet*(envm_u(k)-u1(k))*envi_a(k)*Psig_w
L07310             det_v1(k) = Cdet*(envm_v(k)-v1(k))*envi_a(k)*Psig_w
L07311           ENDDO
L07312         ENDIF
L07313     ENDIF !end subsidence/env detranment
L07314
L07315     !First, compute plume theta & total water at interfaces (qti)
L07316     !These values do not need to be defined at k=kte (unused level).
L07317     DO k=kts,kte-1
L07318        edmf_th1(k)= edmf_thl1(k) + xlvcp/ex1(k)*edmf_qc1(K)
L07319        qti(k+1)   =(qt1(k)*dz1(k+1)+qt1(k+1)*dz1(k))/(dz1(k+1)+dz1(k))
L07320     ENDDO
L07321     qti(kts)=qt1(kts)
L07322
L07323  !update CLDFRA_bl1, qc_bl1. They have already been defined in
L07324  !     mym_condensation. Here, a shallow-cu component is added, but no cumulus
L07325  !     clouds can be added at k=1 (start loop at k=2).
L07326     cb_check = 0
L07327     do k=kts+1,kte-2
L07328        if (k > KTOP) exit
L07329        if (edmf_qc1(k) > zero ) then !.and. (cldfra_bl1(k) < cf_thresh))THEN
L07330           if (cb_check == 0) then
L07331              mf_at_cb = edmf_a1(k)*edmf_w1(k)
L07332              cb_check = 1
L07333            endif
L07334            !plume properties within mass layers.
L07335            Aup = edmf_a1(k)
L07336            THp = edmf_thl(k)
L07337            QTp = edmf_qt1(k)
L07338            QCp = edmf_qc1(k)
L07339            dqwdz = (qti(k)-qti(k+1))/dz1(k) !pos = decreasing upward
```

```
L07340            !convert TH to T
L07341            !t = THp*exner(k)
L07342            !SATURATED VAPOR PRESSURE
L07343            esat = esat_blend(tk1(k))
L07344            !SATURATED SPECIFIC HUMIDITY
L07345            qsl=ep_2*esat/max(1.e-7_kind_phys,(pres1(k)-ep_3*esat))
L07346
L07347            !COMPUTE CLDFRA & QC_BL FROM MASS-FLUX SCHEME and recompute vt & vq
L07348            xl = xl_blend(tk1(k))               ! obtain blended heat capacity
L07349            qsat_tk = qsat_blend(tk1(k),pres1(k))! get saturation water vapor mixing ratio
L07350                                                !   at t and p
L07351            rsl = xl*qsat_tk / (r_v*tk1(k)**2)  ! slope of C-C curve at t (abs temp)
L07352                                                ! CB02, Eqn. 4
L07353            cpm = cp + qt1(k)*cpv               ! CB02, sec. 2, para. 1
L07354            a   = one/(one + xl*rsl/cpm)        ! CB02 variable "a"
L07355            b9  = a*rsl                         ! CB02 variable "b"
L07356
L07357            q2p = xlvcp/ex1(k)
L07358            pt  = thl1(k) +q2p*QCp*Aup ! potential temp (env + plume)
L07359            bb  = b9*tk1(k)/pt ! bb is "b9" in BCMT95.  Their "b9" differs from
L07360                               ! "b9" in CB02 by a factor
L07361                               ! of T/theta.  Strictly, b9 above is formulated in
L07362                               ! terms of sat. mixing ratio, but bb in BCMT95 is
L07363                               ! cast in terms of sat. specific humidity.  The
L07364                               ! conversion is neglected here.
L07365            qww   = one+p608*qt1(k)
L07366            alpha = p608*pt
L07367            beta  = pt*xl/(tk1(k)*cp) - 1.61_kind_phys*pt
L07368            !Buoyancy flux terms have been moved to the end of this section...
L07369
L07370            !Now calculate convective component of the cloud fraction:
L07371            if (a > zero) then
L07372               f = MIN(one/a, four)      ! f is vertical profile scaling function (CB2005)
L07373            else
L07374               f = one
L07375            endif
L07376
L07377            !---CB form:
L07378            !sigq = 3.5E-3 * Aup * edmf_w1(k) * f  ! convective component of sigma (CB2005)
L07379            !sigq = SQRT(sigq**2 + sgm1(k)**2)     ! combined conv + stratus components
L07380
L07381            !---Per S.DeRoode 2009?
L07382            !sigq = nine * Aup * (QTp - qt1(k))
L07383
L07384            !---Extended CB form, tauc is timescale similar to the eddy turnover timescale.
L07385            tauc = 1800._kind_phys
L07386            sigq = five * Aup * edmf_w1(k) * max(4e-3_kind_phys, tauc*max(zero, dqwdz))
L07387
L07388            !constrain sigq wrt saturation:
L07389            !sigq = max(sigq, qsat_tk*0.03_kind_phys)
L07390            !constrain sigq wrt moisture excess in the updraft (deRoode)
L07391            sigq = max(sigq, three * Aup * (QTp - qt1(k)))
L07392            !sigq = SQRT(sigq**2 + sgm1(k)**2)     ! combined conv + stratus components
L07393            !sigq = max(sigq, sgm1(k))             ! use max of conv + stratus components
L07394
L07395            !qmq = a * (qt1(k) - qsat_tk)          ! saturation deficit/excess;
L07396            qmq = qt1(k) - qsat_tk                ! saturation deficit/excess;
L07397            if (qmq > zero) sigq = min(qsat_tk*0.015_kind_phys, sigq)
L07398            Q1  = qmq/sigq                        !   the numerator of Q1
L07399
L07400            if ((landsea-1.5).GE.zero) then   ! WATER
L07401               !modified form from LES
L07402               !cf_mf = min(max(0.5 + 0.36 * atan(1.20*(Q1+0.2)),0.01),0.6)
L07403               !Original CB
L07404               !cf_mf = min(max(p5 + 0.36_kind_phys * atan(1.55*Q1),0.01_kind_phys),0.8_kind_phys)
L07405               !increase cf in cases of large mf at cloudbase
L07406               cffac = min(max(zero, mf_at_cb - 0.05_kind_phys)/0.05_kind_phys, one)
L07407               cf_mf = min(max(p5 + 0.36_kind_phys * atan(1.8_kind_phys*(Q1+p2+cffac)),0.01_kind_phys), one)
L07408               cf_mf = max(cf_mf, 1.2_kind_phys * Aup)
L07409               !cf_mf = max(cf_mf, 1.8_kind_phys * Aup)
L07410               !cf_mf = min(cf_mf, 5.0 * Aup)
L07411            else                                  ! LAND
L07412               !LES form
L07413               !cf_mf = min(max(0.5 + 0.36 * atan(1.20*(Q1+0.4)),0.01),0.6)
L07414               !Original CB
L07415               !cf_mf = min(max(p5 + 0.36_kind_phys * atan(1.55*Q1),0.01_kind_phys),0.8_kind_phys
L07416               cf_mf = min(max(p5 + 0.36_kind_phys * atan(1.8_kind_phys*(Q1+p2)),0.01_kind_phys), one)
L07417               cf_mf = max(cf_mf, 1.8_kind_phys * Aup)
L07418               !cf_mf = min(cf_mf, 5.0 * Aup)
L07419            endif
L07420
L07421            !if ( debug_mf == 1 .and. i==idbg .and. j==jdbg) then
L07422            !   print*,"In MYNN-MF, macrophysics component"
L07423            !   print*," env qt=",qt1(k)," qsat=",qsat_tk
L07424            !   print*," k=",k," satdef=",QTp - qsat_tk," sgm=",sgm1(k)
L07425            !   print*," sigq=",sigq," qmq=",qmq," tk=",tk1(k)
L07426            !   print*," cf_mf=",cf_mf," cldfra_bl=",cldfra_bl1(k)," edmf_a1=",edmf_a1(k)
L07427            !endif
L07428
L07429            !Update cloud fractions and specific humidities in grid cells
L07430            !where the mass-flux scheme is active. The specific humidities
L07431            !are converted to grid means (not in-cloud quantities).
L07432            qcfac = 1.20_kind_phys + 1.0_kind_phys*min(max(zero, mf_at_cb - 0.04_kind_phys)/0.05_kind_phys, one)
L07433            if ((landsea-1.5).GE.zero) then  ! water
L07434               !if (QCp * Aup > 5e-5) then
L07435               !   qc_mf     = 1.86_kind_phys * (QCp * Aup) - 2.2e-5_kind_phys
L07436               !else
L07437               !   qc_mf     = 1.20_kind_phys * (QCp * Aup)
L07438               !endif
L07439               qc_mf     = qcfac * (QCp * Aup)
L07440            else                                 ! land
L07441               if (QCp * Aup > 5e-5) then
L07442                  qc_mf     = 1.86_kind_phys * (QCp * Aup) - 2.2e-5_kind_phys
L07443               else
L07444                  qc_mf     = 1.20_kind_phys * (QCp * Aup)
L07445               endif
L07446            endif
L07447              !In the condition of very large cloud fractions, where the instantaneous
```

```
L07448              !condensed water in the plume is not a reasonable estimate of the
L07449              !mixing ratio in a larger cloud, we must rely on a background estimate based
L07450              !off of environmental qsat.
L07451              wt2          = one - min(one, max(zero, (cf_mf - thirty))/(hundred-thirty))  !=1 for low cf_mf, =0 for
↪  cf_mf = 1.0
L07452              qc_mf_min    = wt2*qsat_tk*0.01_kind_phys*cf_mf + (one-wt2)*qsat_tk*0.025_kind_phys*cf_mf
L07453              qc_mf        = max(qc_mf, qc_mf_min)
L07454              !Then blend with the stratus component:
L07455              cf_strat     = cldfra_bl1(k)
L07456              qc_strat     = qc_bl1(k)
L07457              cldfra_bl1(k)= max(cf_mf, cf_strat)
L07458              pct_mf       = cf_mf/cldfra_bl1(k)
L07459              qc_bl1(k)    = qc_mf*pct_mf + (one-pct_mf)*qc_strat
L07460
L07461              !Now recalculate the terms for the buoyancy flux for mass-flux clouds:
L07462              !See mym_condensation for details on these formulations.
L07463              !Use Bechtold and Siebesma (1998) piecewise estimation of Fng with
L07464              !limits ,since they really should be recalculated after all the other changes...:
L07465              !Only overwrite vt & vq in non-stratus condition
L07466              !if ((landsea-1.5).GE.zero) then      ! WATER
L07467                 Q1=max(Q1,-2.25)
L07468              !else
L07469              !   Q1=max(Q1,-2.0)
L07470              !endif
L07471
L07472              if (Q1 .ge. one) then
L07473                 Fng = one
L07474              elseif (Q1 .ge. -1.7 .and. Q1 .lt. one) then
L07475                 Fng = EXP(-p4*(Q1-one))
L07476              elseif (Q1 .ge. -2.5 .and. Q1 .lt. -1.7) then
L07477                 Fng = three + EXP(-3.8*(Q1+1.7))
L07478              else
L07479                 Fng = min(23.9 + EXP(-1.6*(Q1+2.5)), 60.)
L07480              endif
L07481
L07482              !link the buoyancy flux function to active clouds only (c*Aup):
L07483              vt1(k) = qww   - (1.5*Aup)*beta*bb*Fng - one
L07484              vq1(k) = alpha + (1.5*Aup)*beta*a*Fng  - tv0
L07485           endif !check for (qc in plume) .and. (cldfra_bl < threshold)
L07486        enddo !k-loop
L07487
L07488  ENDIF  !end nup2 > 0
L07489
L07490  !modify output (negative: dry plume, positive: moist plume)
L07491  if (ktop > 0) then
L07492     maxqc = maxval(edmf_qc1(1:ktop))
L07493     if ( maxqc < 1.E-8) maxmf = -1.*maxmf
L07494  endif
```

- **通量装配**：`s_awφ(k) = Σ_ip ρ • a • w • φ • Psig_w`——八个羽流的加权和就是隐式求解器需要的“非局地通量系数”（回到 code_09a §09a.10 的 a/b/c 项）；qc/qv 通量的拆法保证总水守恒（`s_awqv = s_awqt − s_awqc`）。
- **通量限制器**（L7115–L7160）：首层 MF 热通量 `flx1 = s_aw(kts+1) • Δθ_v/dzi` 超过 `0.75 • fltv/dz` 时 ** 整柱所有 s_aw*（含面积 UPA）乘同一 adjustment**——等比缩放保结构（与 Thompson 守恒段同一思想）。
- **平均上升流属性**（输出诊断 edmf_a1/w1/qt1/thl1/ent1/qc1）：羽流内插值到质量层（面密度加权）、云底特殊处理（`UPQC(k)<1e-10` 时用界面值防零平均）、顶用单层防与零平均；`edmf_a1` 单独乘 Psig_w（注释：只有面积乘，属性不乘——尺度感知只作用在权重上）。
- **TKE 生产**：`tkeprod_up = w³ • a/(b1 • el)`——注释：dTKE/dt = ½w³（对流 TKE 通量的经典标度）乘 2 换成 QKE 口径。
- **MF 云量宏物理**（L7323–L7459，`nup2>0` 时）：在羽流有 qc 的层——①σ 的 MF 分量 `sigq = 5 • a • w • max(4e-3, 1800 • dqwdz)`（扩展 CB05：1800 s 涡翻转时间 × 总水梯度），下限 `3 • a • (QTp−qt)`（S. de Roode 建议的羽流过饱和度）；② 云量 `cf_mf = 0.5+0.36 • atan(1.8 • (Q1+0.5+cffac))`（水面 cffac 由云底 MF 增强）、下限 `1.2(水)/1.8(陆) • Aup`；③ 云水 `qc_mf = qcfac • (QCp • Aup)`（陆面大值区 1.86 倍线性式）、下限 `qsat • (1~2.5%) • cf_mf`（大云量时靠环境饱和比的背景估计）；④ 与层云分量**取大合并云量、按份额加权云水**；⑤**vt/vq 重算**（Fng 分段函数再现，系数改 1.5 • Aup 权重）。
- **干羽流标记**：`maxqc < 1e-8` 时 `maxmf = −maxmf`（输出负号 =干羽流，诊断用）。

09b.7 `condensation_edmf` 三件：羽流的饱和调整

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7545-L7601
L07545  subroutine condensation_edmf(QT,THL,P,zagl,THV,QC)
L07546  !
L07547  ! zero or one condensation for edmf: calculates THV and QC
L07548  !
L07549    use module_bl_mynnedmf_common, only: p1000mb,xlvcp,rcp,rvovrd,&
L07550         zero,one,p5,kind_phys
L07551
L07552  real(kind_phys),intent(in)   :: QT,THL,P,zagl
L07553  real(kind_phys),intent(inout):: THV
L07554  real(kind_phys),intent(inout):: QC
L07555
L07556  integer :: niter,ni
L07557  real(kind_phys):: diff,exn,t,th,qs,qcold
L07558
```

```
L07559  ! constants used from module_model_constants.F
L07560  ! p1000mb
L07561  ! rcp ... Rd/cp
L07562  ! xlv ... latent heat for water (2.5e6)
L07563  ! cp
L07564  ! rvord .. r_v/r_d  (1.6)
L07565
L07566  ! number of iterations
L07567    niter=50
L07568  ! minimum difference (usually converges in < 8 iterations with diff = 2e-5)
L07569    diff=1.e-6_kind_phys
L07570
L07571    EXN=(P/p1000mb)**rcp
L07572    QC=zero  !better first guess QC is incoming from lower level, do not set to zero
L07573    do ni=1,NITER
L07574       T=EXN*THL + xlvcp/EXN*QC
L07575       QS=qsat_blend(T,P)
L07576       QCOLD=QC
L07577       QC=p5*QC + p5*MAX((QT-QS),zero)
L07578       if (abs(QC-QCOLD)<Diff) exit
L07579    enddo
L07580
L07581    T=EXN*THL + xlvcp/EXN*QC
L07582    QS=qsat_blend(T,P)
L07583    QC=max(QT-QS,zero)
L07584
L07585    !Do not allow saturation below 100 m
L07586    if(zagl < 100.)QC=zero
L07587
L07588    !THV=(THL+xlv/cp*QC).*(1+(1-rvovrd)*(QT-QC)-QC);
L07589    THV=(THL+xlvcp/EXN*QC)*(one+QT*(rvovrd-one)-rvovrd*QC)
L07590
L07591  !  IF (QC > zero) THEN
L07592  !    PRINT*,"EDMF SAT, p:",p," iterations:",ni
L07593  !    PRINT*," T=",T," THL=",THL," THV=",THV
L07594  !    PRINT*," QS=",QS," QT=",QT," QC=",QC,"ratio=",qc/qs
L07595  !  ENDIF
L07596
L07597    !THIS BASICALLY GIVES THE SAME RESULT AS THE PREVIOUS LINE
L07598    !TH = THL + xlvcp/EXN*QC
L07599    !THV= TH*(1. + p608*QT - QC)
L07600
L07601  end subroutine condensation_edmf
```

- **不动点迭代**（非牛顿）：`T = exn • THL + xlvcp/exn • QC → QS = qsat_blend(T,P) → QC = ½ • QC + ½ • max(QT−QS, 0)`——**半隐式松弛**（比直接 `QC = QT−QS` 稳定），50 次上限、收敛 1e-6（注释：<8 次收敛）；迭代后再做一次显式步定稿。
- **100 m 以下禁饱和**（`zagl<100 → QC=0`）——贴地层云的数值卫生。
- **虚位温重建**：`THV = (THL + xlvcp/exn • QC) • (1 + QT • (rvovrd−1) − rvovrd • QC)`——**rvovrd = 1.6**（Rv/Rd），书 (2.2.10) 虚温订正的位温形态（对照：`1+0.608qv−qc` 的比湿版本）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7604-L7678
L07604  subroutine condensation_ddmf(qt,thl,p,zagl,thv,qc,qi,debug_dd)
L07605  !
L07606  ! zero or one condensation for edmf: calculates thv, qc, and qi
L07607  !
L07608    use module_bl_mynnedmf_common, only: p1000mb,xlvcp,xlscp,rcp, &
L07609         rvovrd,p608,zero,one,p5,kind_phys
L07610
L07611  real(kind_phys),intent(in)   :: qt,thl,p,zagl
L07612  real(kind_phys),intent(inout):: thv,qc,qi
L07613
L07614  integer :: niter,ni,debug_dd
L07615  real(kind_phys):: diff,exn,t,th,qs,qx,qxold,frac_ice,frac_liq,thvin
L07616
L07617  ! number of iterations
L07618    niter=50
L07619  ! minimum difference (usually converges in < 8 iterations with diff = 2e-5)
L07620    diff=1.e-6
L07621
L07622    if ((qi+qc) .gt. zero) then
L07623       frac_ice = qi/(qi+qc)
L07624       frac_liq = one - frac_ice
L07625    else
L07626       frac_ice = zero
L07627       frac_liq = zero
L07628    endif
L07629
L07630    if (debug_dd == 1) then
L07631       thvin = thv
L07632       print*,"------- in consensation_ddmf-------------"
L07633       print*,"input qc=",qc," qi=",qi," frac_liq=",frac_liq
L07634    endif
L07635
L07636    exn=(p/p1000mb)**rcp
L07637    do ni=1,niter
L07638       t=exn*thl + xlvcp/exn*qc + xlscp/exn*qi
L07639       qs=qsat_blend(t,p)
L07640       qxold=qc+qi
L07641       qx=qc+qi
L07642       qx=p5*qx + p5*max((qt-qs),zero)
L07643       qc=frac_liq*qx
L07644       qi=frac_ice*qx
L07645       if (abs(qx-qxold)<diff) exit
L07646    enddo
L07647
L07648    t=exn*thl + xlvcp/exn*qc + xlscp/exn*qi
L07649    qs=qsat_blend(t,p)
L07650    qx=max(qt-qs,zero)
L07651    qc=frac_liq*qx
L07652    qi=frac_ice*qx
L07653
L07654    !thv=(thl+xlv/cp*qc).*(1+(1-rvovrd)*(qt-qc)-qc)
```

```
L07655  !was this: thv=(thl+xlvcp*qc)*(1.+qt*(rvovrd-1.)-rvovrd*qc)
L07656    !thv=(thl + xlvcp*qc + xlscp*qi)*(1. + qt*(rvovrd-1.)-rvovrd*qc)
L07657    thv=(thl + xlvcp/exn*qc + xlscp/exn*qi)*(one + p608*(qt-qx) - qx)
L07658
L07659  !  if (qc > zero) then
L07660  !    print*,"edmf sat, p:",p," iterations:",ni
L07661  !    print*," t=",t," thl=",thl," thv=",thv
L07662  !    print*," qs=",qs," qt=",qt," qc=",qc,"ratio=",qc/qs
L07663  !  endif
L07664
L07665    !this basically give the same result as the previous line
L07666    !th = thl + xlv/cp/exn*qc
L07667    !thv= th*(1. + p608*qt)
L07668
L07669    !print *,'t,p,qt,qs,qc'
L07670    !print *,t,p,qt,qs,qc
L07671
L07672    if (debug_dd == 1) then
L07673       print*,"output qc=",qc," qi=",qi," frac_liq=",frac_liq
L07674       print*,"input thv=",thvin," out thv=",thv," diff=",thv-thvin
L07675       print*,"------- exiting consensation_ddmf----------"
L07676    endif
L07677
L07678  end subroutine condensation_ddmf
```

_ddmf（下沉流版）：混合相——冰/液份额由入场 qc/qi 比例冻结（`frac_ice`），总凝结 `qx` 按 50/50 松弛迭代后按份额分拆；虚位温式含冰（`xlscp/exn * qi` 项）与降水物订正（`(qt-qx)`）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7682-L7718
L07682  subroutine condensation_edmf_r(QT,THL,P,zagl,THV,QC)
L07683  !
L07684  ! zero or one condensation for edmf: calculates THL and QC
L07685  ! similar to condensation_edmf but with different inputs
L07686  !
L07687    use module_bl_mynnedmf_common, only: p1000mb,rcp,rvovrd,xlvcp,&
L07688         one,zero,kind_phys
L07689
L07690  real(kind_phys),intent(in)   :: QT,THV,P,zagl
L07691  real(kind_phys),intent(inout):: THL, QC
L07692
L07693  integer :: niter,ni
L07694  real(kind_phys):: diff,exn,t,th,qs,qcold
L07695
L07696  ! number of iterations
L07697    niter=50
L07698  ! minimum difference
L07699    diff=2.e-5_kind_phys
L07700
L07701    EXN=(P/p1000mb)**rcp
L07702    ! assume first that th = thv
L07703    T = THV*EXN
L07704    !QS = qsat_blend(T,P)
L07705    !QC = QS - QT
L07706
L07707    QC=zero
L07708
L07709    do ni=1,NITER
L07710       QCOLD = QC
L07711       T = EXN*THV/(one+QT*(rvovrd-one)-rvovrd*QC)
L07712       QS=qsat_blend(T,P)
L07713       QC= MAX((QT-QS), zero)
L07714       if (abs(QC-QCOLD)<Diff) exit
L07715    enddo
L07716    THL = (T - xlvcp*QC)/EXN
L07717
L07718  end subroutine condensation_edmf_r
```

_r（反解版）：输入 THV/QT 反解 THL/QC——被注释掉的调用可见于 ddmp_mf 初始化（现行用显式 def_th 扰动替代）。同样 50 次不动点。

09b.8 `ddmp_mf`：辐射冷却驱动的下沉流

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7720-L7838
L07720  !=================================================================
L07721  ! ddmp_mf is a downdraft mass flux scheme - analogous to the updrafts but
L07722  ! flipped upsidedown and revised to be more specific for turbulence
L07723  ! driven by radiative cooling at cloud tops. this scheme has been primarily
L07724  ! designed for stratocumulus cloud conditions. This is based off of a
L07725  ! combination of the original scheme [Wu et al. (2020, MWR)] but several
L07726  ! modifications were made to better fit it in the MYNN-EDMF and to
L07727  ! help the coupling to the Thompson microphysics scheme. Some of the
L07728  ! primary design changes include:
L07729  !
↪
L07730  ! 1) the spectral plume design, similar to the MYNNs updraft scheme
L07731  ! 2) generalized to any cloud, not just PBL-topped clouds
L07732  ! 3) always initialized at the cloud top
L07733  ! 4) can penetrate all the way to the surface
L07734  ! 5) generalized for mixed-phase clouds
L07735  ! 6) mixes number concentration for double moment schemes
L07736  ! 7) mixes aerosols
L07737  ! 8) revised the conection to the solver
L07738
L07739  subroutine ddmp_mf(kts,kte,dt,dx,zw,dz,p,           &
L07740                &u,v,th,thl,thv,tk,qt,qv,qc,qi,       &
L07741                &qnc,qni,qnwfa,qnifa,                 &
L07742                &qke,rho,exner,                       &
L07743                &qc_bl1,qi_bl1,cldfra_bl1,            &
```

```
L07744               &ust,flt,flq,fltv,pblh,kpbl,           &
L07745               &edmf_a_dd,edmf_w_dd, edmf_qt_dd,      &
L07746               &edmf_thl_dd,edmf_ent_dd,edmf_qc_dd,   &
L07747               &sd_aw,sd_awthl,sd_awqt,               &
L07748               &sd_awqv,sd_awqc,sd_awqi,              &
L07749               &sd_awqnc,sd_awqni,                    &
L07750               &sd_awqnwfa,sd_awqnifa,                &
L07751               &sd_awu,sd_awv,sd_awqke,               &
L07752               &tkeprod_dn,el,                        &
L07753               &rthraten,psig,                        &
L07754               &maxmf_dd,maxwidth_dd                  )
L07755
L07756    use module_bl_mynnedmf_common, only: cp,grav,p608, &
L07757         xlvcp,b1,one,two,zero,eight,p1,p2,p333,p5,    &
L07758         p666,kind_phys
L07759
L07760          integer, intent(in) :: kts,kte,kpbl
L07761          real(kind_phys), dimension(kts:kte), intent(in) ::           &
L07762              u,v,th,thl,tk,qt,qv,qc,qi,thv,p,qke,rho,exner,           &
L07763              qnc,qni,qnwfa,qnifa,dz,qc_bl1,qi_bl1,cldfra_bl1,el
L07764          real(kind_phys), dimension(kts:kte), intent(in) :: rthraten
L07765          ! zw .. heights of the downdraft levels (edges of boxes)
L07766          real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L07767          real(kind_phys), intent(in)  :: flt,flq,fltv
L07768          real(kind_phys), intent(in)  :: dt,dx,ust,pblh,psig
L07769     !outputs - downdraft properties
L07770          real(kind_phys), dimension(kts:kte), intent(inout) ::        &
L07771          edmf_a_dd,   edmf_w_dd,   edmf_qt_dd,  edmf_thl_dd,          &
L07772          edmf_ent_dd, edmf_qc_dd,  tkeprod_dn
L07773     !outputs - variables needed for solver (sd_aw - sum ai*wi, sd_awphi - sum ai*wi*phii)
L07774          real(kind_phys), dimension(kts:kte+1) ::                     &
L07775              sd_aw, sd_awthl, sd_awu, sd_awv,                         &
L07776              sd_awqt, sd_awqc, sd_awqv, sd_awqi, sd_awqnc, sd_awqni,  &
L07777              sd_awqnwfa, sd_awqnifa, sd_awqke, sd_aw2
L07778     !outputs - diagnostics
L07779          real(kind_phys), intent(inout) :: maxmf_dd,maxwidth_dd
L07780
L07781     !downdraft properties
L07782          integer, parameter::                 &
L07783              & ndd            = 5,          &  !number of downdrafts
L07784              & kmin           = 3              !lowest k-level where downdrafts can start
L07785          real(kind_phys),parameter ::         &
L07786              & minddd         = 100.,       &  !min downdraft diameter (m)
L07787              & maxddd         = 500.,       &  !max downdraft diameter (m)
L07788              & zmin           = 50.,        &  !lowest height where downdrafts can start
L07789              & dz200          = 200.,       &  !depth over which other parameters are normalized to
L07790              & cooling_thresh = -0.00010       !activation threshold of downdrafts
L07791                               !-0.000116 is ~ -10 C cooling at cloud top per day
L07792                               !-0.000093 is ~  -8 C cooling at cloud top per day
L07793                               !-0.000069 is ~  -6 C cooling at cloud top per day
L07794          real(kind_phys)::                    &
L07795              & maxdd2,                        &  !variable max downdraft diameter (m)
L07796              &    ddd,                        &  !downdraft diameter
L07797              &     dl,                        &  !diameter increment
L07798              &    add,                        &  !total area of downdrafts
L07799              & minexc,                        &  !minimum init downdraft temp pert
L07800              & maxexc                            !maximum init downdraft temp pert
L07801     ! k-index of downdraft starting height
L07802          integer,         dimension(1:ndd) :: dd_initk
L07803     ! downdraft column properties
L07804          real(kind_phys), dimension(kts:kte+1,1:ndd) ::               &
L07805              downw,downthl,downqt,downqc,downa,downu,downv,downthv,   &
L07806              downqi,downqnc,downqni,downqnwfa,downqnifa
L07807     ! entrainment variables
L07808          real(kind_phys), dimension(kts:kte+1,1:ndd) :: ent
L07809     ! internal variables
L07810          real(kind_phys), dimension(kts:kte):: massflux
L07811          integer :: k,i,ki,qltop,qlbase
L07812          real(kind_phys):: qstar,thstar,sigmaw,sigmaqt,               &
L07813              sigmath,z0,pwmin,pwmax,wmin,wmax,went,mindownw
L07814          real(kind_phys):: qtn,thln,thvn,qcn,un,vn,qken,wn2,wn,       &
L07815              qin,qncn,qnin,qnwfan,qnifan,                             &
L07816              thvk,pk,entexp,entw,beta_dm,entexm,rho_int
L07817          real(kind_phys):: jump_thv,jump_qt,jump_thetal,              &
L07818              refthl,refthv,refthlv,refqt,refqc,refqi,refqnc,refqni,   &
L07819              refqnwfa,refqnifa,refu,refv,refqke,refqt2,reftk,refp,    &
L07820              qx_k,qx_km1,refthvm1,cftop,crate,ac_wsp,wspd_pbl,        &
L07821              maxcooling
L07822          real(kind_phys):: dthvx,tmp1,rcldb,ent_eff
L07823
L07824     ! dd specific internal variables
L07825          real(kind_phys):: radflux, f0, wstar_rad, dz_ent
L07826          logical :: cloudflg
L07827          logical :: singlelayer            !check for single or multi-layer clouds
L07828          real(kind_phys):: sigq,xl,rsl,cpm,a,diffqt,dp,               &
L07829              fng,qww,alpha,beta,bb,f,pt,t,q2p,b9,satvp,rhgrid,        &
L07830              def_th,def_qt,frac_liq,frac_ice,qs1,qs2
L07831
L07832     ! w parameters
L07833          real(kind_phys),parameter ::                                 &
L07834              &wa=1., wb=1.5, z00=100.
L07835          real(kind_phys) :: buoy,bcoeff,ecoeff
L07836
L07837     ! additional printouts for debugging
L07838          integer, parameter :: debug_dd=0   !0:no debugging, 1:detailed output, 2:output for strange values only
```

头注释的八条设计声明（原文 L7721–L7737）——下沉流相对 Wu et al. (2020, MWR) 原始方案的改造：① 谱羽流设计（同上升流）；② 推广到任意云（不限于 PBL 顶盖）；③ 永远从云顶初始化；④ 可直达地面；⑤ 混合相推广；⑥ 双矩方案的数浓度；⑦ 气溶胶混合；⑧ 求解器接口重设计。**`ndd = 5` 个下沉羽流**、**`cooling_thresh = -1e-4` K/s**（注释折算：约 −10 °C/天的云顶冷却——三档历史阈值 6/8/10 °C 并列注释）。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L7840-L8087
L07840    ! ================================================================
L07841    ! initialize downdraft properties
L07842     downw      =zero
L07843     downthl    =zero
L07844     downthv    =zero
L07845     downqt     =zero
L07846     downa      =zero
L07847     downu      =zero
L07848     downv      =zero
L07849     downqc     =zero
L07850     downqi     =zero
L07851     downqnc    =zero
L07852     downqni    =zero
L07853     downqnwfa  =zero
L07854     downqnifa  =zero
L07855     ent        =zero
L07856     dd_initk   =0
L07857
L07858     edmf_a_dd  =zero
L07859     edmf_w_dd  =zero
L07860     edmf_qt_dd =zero
L07861     edmf_thl_dd=zero
L07862     edmf_ent_dd=zero
L07863     edmf_qc_dd =zero
L07864
L07865     sd_aw      =zero
L07866     sd_awthl   =zero
L07867     sd_awqt    =zero
L07868     sd_awqv    =zero
L07869     sd_awqc    =zero
L07870     sd_awqi    =zero
L07871     sd_awqnc   =zero
L07872     sd_awqni   =zero
L07873     sd_awqnwfa =zero
L07874     sd_awqnifa =zero
L07875     sd_awu     =zero
L07876     sd_awv     =zero
L07877     sd_awqke   =zero
L07878
L07879     ! first, check for cloud tops. if more than one, find the one with maximum cooling at the cloud top.
L07880     cloudflg   =.false.
L07881     singlelayer=.false.
L07882     qltop      =1     !initialize
L07883     qlbase     =1     !initialize
L07884     crate      =zero
L07885     do k = max(kmin,kpbl-5),min(kpbl+20,kte-1)
L07886        maxcooling = minval(rthraten(k-1:k+1))
L07887        !---------------------------------criteria for downdraft activation:
L07888        if ((cldfra_bl1(k).gt.p5 .and. cldfra_bl1(k+1).lt.p5) .and. &  !1) stratocu cloud top exists
L07889            (maxcooling  .lt. cooling_thresh)) then                    !2) significant radiative cooling
L07890           ! found sc cloud with significant radiative cooling
L07891           cloudflg =.true.
L07892           if (maxcooling < crate) then
L07893              qltop    =k              !index for sc cloud top
L07894              cftop    =cldfra_bl1(k)
L07895              crate    =maxcooling     !maximum cooling rate at cloud top
L07896              if (cldfra_bl1(k-1).lt.p2)singlelayer=.true.
L07897           endif
L07898        endif
L07899     enddo
L07900
L07901     ! determine maximum downdraft width and increments
L07902     maxdd2  = min(maxddd, 1.2_kind_phys*dx)                                    !1) grid-scale dependence
L07903     maxdd2  = min(maxdd2, -100._kind_phys*crate*3600._kind_phys + 200._kind_phys) !2) function of cloud top cooling rate
L07904     maxdd2  = min(maxdd2, 80._kind_phys + p666*zw(qltop+1))                    !3) limited for low cloud-top heights
L07905     maxwidth_dd = maxdd2
L07906     dl      = max(maxdd2-minddd, zero)/real(ndd-1)
L07907
L07908     !found sc cloud with conditions for downdrafts; compute downdrafts
L07909     if (cloudflg .and. ( dl .gt. zero)) then
L07910        do k = qltop, kts, -1
L07911           qx_k =max(qt(k), qc_bl1(k) +qi_bl1(k))     !total cloud water and ice at k
L07912           if (qx_k .gt. 1e-6_kind_phys) then
L07913              qlbase = k ! index for sc cloud base
L07914           endif
L07915        enddo
L07916
L07917        do i=1,ndd
L07918           ! downdraft starts at the first layer interface below the cloud top
L07919           dd_initk(i) = qltop
L07920        enddo
L07921
L07922        ! loop radflux
L07923        f0 = zero
L07924        do k = max(kmin,qltop-2), qltop+1
L07925           radflux = rthraten(k) * exner(k)     ! converts theta/s to temperature/s
L07926           dp      = p5 * (( p(k) - p(k+1) ) + ( p(k-1) - p(k) ))
L07927           radflux = radflux * cp / grav * dp   ! converts k/s to w/m^2
L07928           if ( radflux < zero ) f0 = abs(radflux) + f0
L07929        enddo
L07930        f0 = min(max(f0, one), 200._kind_phys)  !total radiative cooling (w/m2)
L07931
L07932        !allow the total fractional area of the downdrafts (add) to be proportional
L07933        !to the radiative forcing:
L07934        !for  50 w/m2, add = 0.10
L07935        !for 100 w/m2, add = 0.20
L07936        !for 150 w/m2, add = 0.30
L07937        add      = min( f0*0.003, 0.3_kind_phys)
L07938        !taper off area for high wind speeds
L07939        wspd_pbl = sqrt(max(u(kts)**2 + v(kts)**2, 0.01_kind_phys))
L07940        ac_wsp   = one - min(max(zero, wspd_pbl - 10._kind_phys)/15._kind_phys, one)
L07941        add      = add * min(ac_wsp, psig)
L07942
L07943        !find inversion strength across cloud top entrainment zone--normalized to 200 m vertical grid spacing
L07944        dz_ent      = p5 * (dz(qltop+1) + dz(qltop))
L07945        jump_thv    = (thv(qltop+1) - thv(qltop)) / dz_ent * dz200
L07946        jump_qt     = (qt(qltop+1)  - qt(qltop))  / dz_ent * dz200
L07947        jump_thetal = (thl(qltop+1) - thl(qltop)) / dz_ent * dz200
```

```
L07948
L07949          ki = qltop  !index of cloud top
L07950          if (singlelayer) then !initialize dd with cloud properties (not using info from above)
L07951             refthl  = thl(ki)
L07952             refthlv = refthl*(1.+p608*qv(ki))
L07953             refthv  = thv(ki)
L07954             refthvm1= thv(ki-1)
L07955             reftk   = tk(ki)
L07956             refqt   = qt(ki)
L07957             refqc   = qc(ki)
L07958             refqi   = qi(ki)
L07959             refqnc  = qnc(ki)
L07960             refqni  = qni(ki)
L07961             refqnwfa= qnwfa(ki)
L07962             refqnifa= qnifa(ki)
L07963             refu    = u(ki)
L07964             refv    = v(ki)
L07965             refqke  = qke(ki)
L07966             refp    = p(ki)
L07967          else                  !initialize dd with avg of in-cloud properties (at & below) cloud top
L07968             refthl  = (thl(ki-1)*dz(ki)   + thl(ki)*dz(ki-1))  /(dz(ki)+dz(ki-1))
L07969             refthlv = refthl*(1.+p608*(qv(ki-1)*dz(ki) + qv(ki)*dz(ki-1))/(dz(ki)+dz(ki-1)))
L07970             refthv  = (thv(ki-1)*dz(ki)   + thv(ki)*dz(ki-1))  /(dz(ki)+dz(ki-1))
L07971             refthvm1= (thv(ki-2)*dz(ki-1) + thv(ki-1)*dz(ki-2))/(dz(ki-1)+dz(ki-2))
L07972             reftk   = (tk(ki-1)*dz(ki)    + tk(ki)*dz(ki-1))   /(dz(ki)+dz(ki-1))
L07973             refqt   = (qt(ki-1)*dz(ki)    + qt(ki)*dz(ki-1))   /(dz(ki)+dz(ki-1))
L07974             refqc   = (qc(ki-1)*dz(ki)    + qc(ki)*dz(ki-1))   /(dz(ki)+dz(ki-1))
L07975             refqi   = (qi(ki-1)*dz(ki)    + qi(ki)*dz(ki-1))   /(dz(ki)+dz(ki-1))
L07976             refqnc  = (qnc(ki-1)*dz(ki)   + qnc(ki)*dz(ki-1))  /(dz(ki)+dz(ki-1))
L07977             refqni  = (qni(ki-1)*dz(ki)   + qni(ki)*dz(ki-1))  /(dz(ki)+dz(ki-1))
L07978             refqnwfa= (qnwfa(ki-1)*dz(ki) + qnwfa(ki)*dz(ki-1))/(dz(ki)+dz(ki-1))
L07979             refqnifa= (qnifa(ki-1)*dz(ki) + qnifa(ki)*dz(ki-1))/(dz(ki)+dz(ki-1))
L07980             refu    = (u(ki-1)*dz(ki)     + u(ki)*dz(ki-1))    /(dz(ki)+dz(ki-1))
L07981             refv    = (v(ki-1)*dz(ki)     + v(ki)*dz(ki-1))    /(dz(ki)+dz(ki-1))
L07982             refqke  = (qke(ki-1)*dz(ki)   + qke(ki)*dz(ki-1))  /(dz(ki)+dz(ki-1))
L07983             refp    = (p(ki-1)*dz(ki)     + p(ki)*dz(ki-1))    /(dz(ki)+dz(ki-1))
L07984          endif
L07985
L07986          ! w* from radiative forcing (m/s)
L07987          !wstar_rad =    ( grav * dz_ent * f0 / (refthl * rho(ki) * cp) ) **p333
L07988          wstar_rad = 1.25_kind_phys * ( grav*dz200 * f0 / (refthl * rho(ki) * cp) ) **p333
L07989          wstar_rad = min(max(wstar_rad, p1), 1.5_kind_phys)      ! (m/s)
L07990
L07991          !entrainment efficiency
L07992          dthvx     = (thl(ki+2) + th(ki+2)*p608*qt(ki+2)) - &
L07993                      (thl(ki)   + th(ki)  *p608*qt(ki))
L07994          dthvx     = dthvx * dz200/dz_ent  !normalized gradient
L07995          dthvx     = max(dthvx, p1)
L07996          rcldb     = max(refqc+refqi, qc_bl1(ki)+qi_bl1(ki))  !any type of cloud water (sgs or resolved)
L07997          tmp1      = xlvcp * rcldb/(exner(ki)*dthvx)
L07998          !entrainment efficiency: originally from Nichols and Turton (1986), where a2 = 60, but lowered
L07999          !here to 8, as in Grenier and Bretherton (2001).
L08000          ent_eff   = max(zero, min(one, p2 + p2*eight*tmp1))
L08001
L08002          went      = p333 * ent_eff * wstar_rad
L08003          qstar     = went * jump_qt / wstar_rad
L08004          thstar    = -f0/rho(ki)/cp/wstar_rad - went*jump_thv/wstar_rad
L08005
L08006          sigmaw    = 0.2 * wstar_rad   ! 0.8*wstar_rad ! tuning parameter ! 0.5 was good
L08007          sigmaqt   = 50. * abs(qstar)  ! 50 was good
L08008          sigmath   = 1.5 * abs(thstar) ! 0.5 was good
L08009
L08010          pwmin     = -1. ! drawing from the negative tail -3sigma to -1sigma
L08011          pwmax     = -3.
L08012          wmin      = max(min(sigmaw*pwmin, -0.1), -0.2)
L08013          wmax      = max(min(sigmaw*pwmax, -0.2), -0.5)
L08014
L08015          !initialize downdraft temperature deficit def_th
L08016          minexc = min(-0.06     , 0.5*(refthvm1 - refthv)) !increase prob to go down 1 level
L08017          maxexc = min(minexc-0.1, -0.6          )
L08018          maxexc = max(maxexc    , thv(1) - refthlv)        !init parcel temp >= thv_sfc
L08019          def_th = max(min(0.05*(-0.3)*sigmath/sigmaw, minexc), maxexc)
L08020
L08021          !initialize downdraft moisture deficit def_qt (consistent with def_th)
L08022          qs1    = qsat_blend(reftk       ,refp)
L08023          qs2    = qsat_blend(reftk+def_th,refp)
L08024          refqt2 = refqt*qs2/qs1
L08025          !def_qt = min(refqt2 - refqt, zero) !small negative moisture perturbation
L08026          def_qt = min(qs2 - qs1,  -0.33*(refqc+refqi)) !small negative moisture perturbation
L08027          !def_qt = 0.5*(-0.3)*sigmaqt/sigmaw
L08028
L08029          if (refqc+refqi > zero) then
L08030             frac_liq = refqc/(refqc+refqi)
L08031             frac_ice = one - frac_liq
L08032          else
L08033             frac_liq = zero
L08034             frac_ice = zero
L08035          endif
L08036
L08037          if (debug_dd .eq. 1) then
L08038             print*,"-------------------------------------"
L08039             print*,"found conditions for downdraft mixing"
L08040             print*,"qltop=",qltop," qlbase=",qlbase
L08041             print*,"q*=",qstar," theta*=",thstar," went=",went
L08042             print*,"f0=",f0," jump_thv=",jump_thv,"w*rad=",wstar_rad
L08043             print*,"u*=",ust," jump_qt=",jump_qt," fltv=",fltv
L08044             print*,"grav=",grav," pblh=",pblh," thv(1)=",thv(1)
L08045             print*,"p(1)=",p(1)," jump_thetal=",jump_thetal
L08046             print*,"sigmaw=",sigmaw," wmin=",wmin," wmax=",wmax
L08047          endif
L08048
L08049          !intialize downdrafts
L08050          !as of now, all downdrafts start at the same height
L08051          do i=1,ndd
L08052             ki = dd_initk(i)
L08053
L08054             !initialize a negative (downward) vertical velocity
```

```
L08055           downw(ki,i)=wmin-(abs(wmax)-abs(wmin))/real(ndd-1)*(i-1)
L08056
L08057           !perhaps multiply downa by cloud fraction, so it's impact will diminish if
L08058           !clouds are mixed away over the course of the longer radiation time step(?)
L08059           downa(ki,i)     = add/real(ndd)
L08060           downu(ki,i)     = refu
L08061           downv(ki,i)     = refv
L08062           downqc(ki,i)    = max(refqc + def_qt*frac_liq, zero)
L08063           downqi(ki,i)    = max(refqi + def_qt*frac_ice, zero)
L08064           downqnc(ki,i)   = refqnc
L08065           downqni(ki,i)   = refqni
L08066           downqnwfa(ki,i) = refqnwfa
L08067           downqnifa(ki,i) = refqnifa
L08068           downqt(ki,i)    = refqt  + def_qt
L08069           downthv(ki,i)   = refthv + def_th*real(i)/real(ndd)
L08070           downthl(ki,i)   = refthl + def_th*real(i)/real(ndd)
L08071
L08072           !input :: qt,thv,p,zagl,  output :: thl, qc
L08073  !         pk  =(p(ki-1)*dz(ki)+p(ki)*dz(ki-1))/(dz(ki)+dz(ki-1))
L08074  !         call condensation_edmf_r(downqt(ki,i),   &
L08075  !              &        downthl(ki,i),pk,zw(ki),   &
L08076  !              &     downthv(ki,i),downqc(ki,i)    )
L08077        enddo
L08078
L08079        if (debug_dd .eq. 1) then
L08080           print*,"=====initialized downdraft properties===="
L08081           print*,"downw=",downw(ki,:)
L08082           print*,"downa=",downa(ki,:)
L08083           print*,"downthv=",downthv(ki,:)
L08084           print*,"def_qt=",def_qt," def_th=",def_th," tot area=",add
L08085           print*,"begin integration of downdrafts (smallest to largest):"
L08086        endif
L08087
```

- **云顶搜索**：PBL 顶 −5～+20 层内找“层云顶”（`cldfra(k)>0.5` 且 `cldfra(k+1)<0.5`）且最大冷却率＜阈值；多顶取冷却最强者记录 `qltop/crate/cftop`；上方云量＜0.2 时 `singlelayer=.true.`（决定初始化用云顶值还是云内平均）。
- **宽度三判据**：`maxdd2 = min(500, 1.2dx, −100•crate•3600+200, 80+(2/3)•zw(qltop))`——网格尺度、冷却率越强羽流越宽、云顶越低羽流越窄。
- **辐射强迫积分**：云顶附近三层 `radflux = rthraten • exn • cp/g • Δp`（K/s → W/m²），负值累加 `f0`，钳 [1, 200] W/m²。
- **面积**：`add = min(0.003 • f0, 0.3)`——**50 W/m² → 15%、100 → 30%**（注释给了换算表）；风衰减（>10 m/s 线性）与 `psig` 相乘。
- **云顶逆温强度**：`jump_thv/qt/thetal` 按 200 m 归一化（分辨率无关）。
- **参考态与过冷扰动**：`ref*` 用云顶（singlelayer）或云内平均；`wstar_rad = 1.25 • (g • dz200 • f0/(θ • ρ • cp))^(1/3)` 钳 [0.1, 1.5] m/s；**卷入效率 `ent_eff`**（Nichols & Turton 1986 系数 a2=60 → 8，Grenier & Bretherton 2001）：`ent_eff = min(1, 1+8 • xlvcp • qc/(exn • dθv))`——**液水越多、云顶混合越强**（云顶凝结-辐射耦合）；`went = (1/3) • ent_eff • wstar_rad`。
- **下沉扰动初始化**：w 从负尾采样（pwmin=−1/pwmax=−3 σw）；`def_th`（温度亏损）与 `def_qt`（水汽亏损 = min(饱和差, $-\frac{1}{3}$ • 凝结物)）——**下沉羽流 =冷的干羽流**。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8088-L8283
L08088        do i=1,ndd
L08089           ddd  = minddd + dl*real(i-1) !downdraft diameter
L08090           !initialize input into condensation routine
L08091           thvn = downthv(ki,i)
L08092           qcn  = downqc(ki,i)
L08093           qin  = downqi(ki,i)
L08094           do k=dd_initk(i)-1,kts+1,-1
L08095              !entrainment from tian and kuang (2016), with constraints
L08096              wmin = p1 + ddd*0.0005
L08097              !ent(k,i) = 0.33/(min(max(abs(downw(k+1,i)),wmin),0.9)*ddd)
L08098              !ent(k,i) = 0.53/(min(max(abs(downw(k+1,i)),wmin),0.6)*ddd)
L08099              !was       0.26  before reducing downdraft sizes
L08100              ent(k,i) = 0.18/(min(max(abs(downw(k+1,i)),wmin),0.6)*ddd)
L08101
L08102              !minimum background entrainment and 1/z enhancement near surface
L08103              ent(k,i) = max(ent(k,i),0.0003)
L08104              ent(k,i) = max(ent(k,i),0.06/zw(k))
L08105              ent(k,i) = min(ent(k,i),0.9/dz(k)) ! liberal check for numerical stability
L08106
L08107              !starting at the first interface level below cloud top
L08108              !entexp=exp(-ent(k,i)*dz(k))
L08109              !entexp_m=exp(-ent(k,i)/3.*dz(k))
L08110              entexp  =ent(k,i)*dz(k)          !for all scalars
L08111              entexm  =ent(k,i)*dz(k)*p5       !for momentum
L08112
L08113              qtn     =downqt(k+1,i)   *(one-entexp) + qt(k)*entexp
L08114              thln    =downthl(k+1,i)  *(one-entexp) + thl(k)*entexp
L08115              qncn    =downqnc(k+1,i)  *(one-entexp) + qnc(k)*entexp
L08116              qnin    =downqni(k+1,i)  *(one-entexp) + qni(k)*entexp
L08117              qnwfan  =downqnwfa(k+1,i)*(one-entexp) + qnwfa(k)*entexp
L08118              qnifan  =downqnifa(k+1,i)*(one-entexp) + qnifa(k)*entexp
L08119              un      =downu(k+1,i)    *(one-entexm) + u(k)*entexm
L08120              vn      =downv(k+1,i)    *(one-entexm) + v(k)*entexm
L08121              !qken=downqke(k-1,i)*(1.-entexp) + qke(k)*entexp
L08122  !            qtn =downqt(k+1,i) +(qt(k) -downqt(k+1,i)) *(1.-entexp)
L08123  !            thln=downthl(k+1,i)+(thl(k)-downthl(k+1,i))*(1.-entexp)
L08124  !            un  =downu(k+1,i)  +(u(k)  -downu(k+1,i))*(1.-entexp_m)
L08125  !            vn  =downv(k+1,i)  +(v(k)  -downv(k+1,i))*(1.-entexp_m)
L08126
L08127              ! given new p & z, solve for thvn, qcn, and qin
```

```
L08128              pk  =(p(k-1)*dz(k)+p(k)*dz(k-1))/(dz(k)+dz(k-1))
L08129              call condensation_ddmf(qtn,thln,pk,zw(k),thvn,qcn,qin,debug_dd)
L08130
L08131              !calculate buoyancy
L08132              thvk =(thv(k-1)*dz(k)+thv(k)*dz(k-1))/(dz(k)+dz(k-1))
L08133              buoy =grav*(thvn/thvk - one)
L08134              !downdraft decelleration to limit large tendencies at the surface
L08135              if (buoy > zero) then
L08136                 bcoeff =  0.2  !0.15 = same as in updrafts
L08137                 ecoeff = -2.0
L08138              else
L08139                 !for numerical stability reasons, apply brakes near the surface.
L08140                 bcoeff = 0.2*(one - exp(-zw(k)/(0.15_kind_phys*pblh)))
L08141                 ecoeff = -2.0_kind_phys - exp(-zw(k)/(0.15_kind_phys*pblh))
L08142                 buoy   = buoy*(one - exp(-zw(k)/(0.15_kind_phys*pblh)))
L08143              endif
L08144              if (k==kts+2) buoy=max(buoy,0.01)
L08145              if (k==kts+1) buoy=max(buoy,0.05)
L08146
L08147  !            beta_dm = 2*wb*ent(k,i) + 0.5/(zw(k)-dz(k)) * &
L08148  !                 &    max(1. - exp((zw(k) -dz(k))/z00 - 1. ) , 0.)
L08149  !            entw=exp(-beta_dm * dz(k))
L08150  !            entw=entexp
L08151  !            if (beta_dm >0) then
L08152  !               wn2=downw(k+1,i)**2*entw - wa*buoy/beta_dm * (1. - entw)
L08153  !            else
L08154  !               wn2=downw(k+1,i)**2      - 2.*wa*buoy*dz(k)
L08155  !            end if
L08156
L08157              mindownw = min(downw(k+1,i),-0.2_kind_phys)
L08158              wn = downw(k+1,i) + (ecoeff*ent(k,i)*downw(k+1,i)        &
L08159                                - bcoeff*buoy/mindownw)*min(dz(k), 250.)
L08160
L08161              !do not allow a parcel to accelerate more than 1.25 m/s over 200 m.
L08162              !add max acceleration of -2.0 m/s for coarse vertical resolution.
L08163              if (wn < downw(k+1,i) - min(1.25*dz(k)/200., 2.0))then
L08164                  wn = downw(k+1,i) - min(1.25*dz(k)/200., 2.0)
L08165              endif
L08166              !add symmetrical max decrease in velocity (less negative)
L08167              if (wn > downw(k+1,i) + min(1.25*dz(k)/200., 2.0))then
L08168                  wn = downw(k+1,i) + min(1.25*dz(k)/200., 2.0)
L08169              endif
L08170              wn = max(min(wn,zero), -2.0_kind_phys)
L08171
L08172              if (debug_dd .eq.1) then
L08173                 print *, "downdraft #:", i," diameter:",ddd,"==========="
L08174                 print *, "  k       =",      k,      " z    =", zw(k)
L08175                 print *, "  ent     =",ent(k,i),     " bouy =", buoy
L08176                 print *, "  downthv =",   thvn,      " thvk =", thvk
L08177                 print *, "  downthl =",   thln,      " thl  =", thl(k)
L08178                 print *, "  downqt  =",   qtn ,      " qt   =", qt(k)
L08179                 print *, "  downqc  =",   qcn ,      " qc   =", qc(k)
L08180                 print *, "  downw+1 =",downw(k+1,i), " wn2  =", wn
L08181              endif
L08182
L08183              if (wn .lt. 0.) then !terminate when velocity is too small
L08184                 downw(k,i)     = wn !-sqrt(wn2)
L08185                 downthv(k,i)   = thvn
L08186                 downthl(k,i)   = thln
L08187                 downqt(k,i)    = qtn
L08188                 downqc(k,i)    = qcn
L08189                 downqi(k,i)    = qin
L08190                 downqnc(k,i)   = qncn
L08191                 downqni(k,i)   = qnin
L08192                 downqnwfa(k,i) = qnwfan
L08193                 downqnifa(k,i) = qnifan
L08194                 downu(k,i)     = un
L08195                 downv(k,i)     = vn
L08196                 downa(k,i)     = downa(k+1,i)
L08197              else
L08198                 !downdrafts must go at least 2 levels
L08199  !               if (dd_initk(i) - k .lt. 2) then
L08200  !                  downw(:,i)  = zero
L08201  !                  downthv(:,i)= zero
L08202  !                  downthl(:,i)= zero
L08203  !                  downqt(:,i) = zero
L08204  !                  downqc(:,i) = zero
L08205  !                  downu(:,i)  = zero
L08206  !                  downv(:,i)  = zero
L08207  !               endif
L08208                 exit
L08209              endif
L08210           enddo
L08211        enddo
L08212
L08213        downw(1,:) = zero !make sure downdrafts do not penetrate the surface
L08214        downa(1,:) = zero
L08215
L08216        !
L08217        ! combine all downdrafts into one averaged downdraft
L08218        !
L08219        do k=qltop,kts,-1
L08220           do i=1,ndd
L08221              edmf_a_dd(k)  =edmf_a_dd(k)  +downa(k,i)
L08222              edmf_w_dd(k)  =edmf_w_dd(k)  +downa(k,i)*downw(k,i)
L08223              edmf_qt_dd(k) =edmf_qt_dd(k) +downa(k,i)*downqt(k,i)
L08224              edmf_thl_dd(k)=edmf_thl_dd(k)+downa(k,i)*downthl(k,i)
L08225              edmf_ent_dd(k)=edmf_ent_dd(k)+downa(k,i)*ent(k,i)
L08226              edmf_qc_dd(k) =edmf_qc_dd(k) +downa(k,i)*downqc(k,i)
L08227           enddo
L08228
L08229           if (edmf_a_dd(k) > 0.) then
L08230              edmf_w_dd(k)  =edmf_w_dd(k)  /edmf_a_dd(k)
L08231              edmf_qt_dd(k) =edmf_qt_dd(k) /edmf_a_dd(k)
L08232              edmf_thl_dd(k)=edmf_thl_dd(k)/edmf_a_dd(k)
L08233              edmf_ent_dd(k)=edmf_ent_dd(k)/edmf_a_dd(k)
L08234              edmf_qc_dd(k) =edmf_qc_dd(k) /edmf_a_dd(k)
```

```
L08235           endif
L08236           !instead of dTKE/dt = 1/2 w^3, multiply by 1 for QKE.
L08237           tkeprod_dn(k)=(abs(edmf_w_dd(k))**3)*edmf_a_dd(k)/(b1*max(el(k),p5))
L08238           tkeprod_dn(k)=min(0.0004_kind_phys, tkeprod_dn(k))
L08239        enddo
L08240        ! add tke source for entrainment at layer above cloud. use same area
L08241        ! above cloud as used in the initialized downdraft area.
L08242        tkeprod_dn(qltop+1)=went**3*edmf_a_dd(qltop)/(b1*max(el(qltop+1),p5))
L08243        massflux = edmf_a_dd * edmf_w_dd
L08244        maxmf_dd = abs(minval(massflux))
L08245
L08246        if (debug_dd .eq. 2) then
L08247           do k=kts,kte
L08248              if (tkeprod_dn(k) > 0.0004_kind_phys .or. edmf_w_dd(k) < -two) then
L08249                 print*,"-------------------------------------"
L08250                 print*,"found strange behavior in downdrafts at",k
L08251                 print*,"qltop=",qltop," qlbase=",qlbase," a(top)=",edmf_a_dd(qltop)
L08252                 print*,"q*=",qstar," theta*=",thstar," went=",went
L08253                 print*,"f0=",f0," jump_thv=",jump_thv,"w*rad=",wstar_rad
L08254                 print*,"u*=",ust," jump_qt=",jump_qt," fltv=",fltv
L08255                 print*,"grav=",grav," pblh=",pblh," thv(1)=",thv(1)
L08256                 print*,"p(1)=",p(1)," jump_thl=",jump_thetal," ent_eff=",ent_eff
L08257                 print*,"sigmaw=",sigmaw," wmin=",wmin," wmax=",wmax
L08258                 print*,"tke prod=",tkeprod_dn(k)," w=",edmf_w_dd(k)," a(k)=",edmf_a_dd(k)
L08259              endif
L08260           enddo
L08261        endif
L08262        ! compute variables needed for solver
L08263        !
L08264        do k=kts,qltop
L08265           rho_int = (rho(k)*dz(k+1)+rho(k+1)*dz(k))/(dz(k+1)+dz(k))
L08266           do i=1,ndd
L08267              sd_aw(k)   =sd_aw(k)   +rho_int*downa(k,i)*abs(downw(k,i))
L08268              sd_awthl(k)=sd_awthl(k)+rho_int*downa(k,i)*downw(k,i)*downthl(k,i)
L08269              sd_awqt(k) =sd_awqt(k) +rho_int*downa(k,i)*downw(k,i)*downqt(k,i)
L08270              sd_awqc(k) =sd_awqc(k) +rho_int*downa(k,i)*downw(k,i)*downqc(k,i)
L08271              sd_awqi(k) =sd_awqi(k) +rho_int*downa(k,i)*downw(k,i)*downqi(k,i)
L08272              sd_awqnc(k)=sd_awqnc(k)+rho_int*downa(k,i)*downw(k,i)*downqnc(k,i)
L08273              sd_awqni(k)=sd_awqni(k)+rho_int*downa(k,i)*downw(k,i)*downqni(k,i)
L08274              sd_awqnwfa(k)=sd_awqnwfa(k)+rho_int*downa(k,i)*downw(k,i)*downqnwfa(k,i)
L08275              sd_awqnifa(k)=sd_awqnifa(k)+rho_int*downa(k,i)*downw(k,i)*downqnifa(k,i)
L08276              sd_awu(k)  =sd_awu(k)  +rho_int*downa(k,i)*downw(k,i)*downu(k,i)
L08277              sd_awv(k)  =sd_awv(k)  +rho_int*downa(k,i)*downw(k,i)*downv(k,i)
L08278           enddo
L08279           sd_awqv(k) = sd_awqt(k)  - sd_awqc(k) - sd_awqi(k)
L08280        enddo
L08281
L08282     endif ! end cloud flag
L08283
```

- **逐层下沉积分**：卷入率 `ent = 0.18/(max(|w|, wmin)·ddd)`（系数 0.18——上升流 0.21 的下沉版）；近地面 **1/z 增强**（`max(ent, 0.06/zw)`）与稳定性钳位。
- **近地面刹车**（L8135–L8145）：负浮力（下沉遇暖层）时 `bcoeff/ecoeff/buoy` 都乘 `(1−exp(−zw/(0.15·PBLH)))`——**越近地面越温和**，防止地表强迫过大；`kts+1/kts+2` 两层强制最小正浮力（0.05/0.01）。
- **速度更新与终止**：`wn = w + (ecoeff·ent·w − bcoeff·buoy/min(w,−0.2))·min(dz,250)`（注意负 w 的符号处理 `mindownw = min(w,−0.2)`）；加速度限对称 ±1.25 m/s per 200 m；`wn ≥ 0` 时终止（至少下沉 2 层的检查被注释）。
- **合并与输出**：五羽流加权平均成 `edmf_*_dd`；`tkeprod_dn = w³·a/(b1·el)` ≤ **4e-4**（与 topdown 同一上限）；云顶上一层附加 `went³·a` 的 TKE 源（**云顶卷入区的湍流生成**）；`sd_aw*` 界面通量族（隐式求解器用，`|w|` 取绝对值——**通量方向由符号约定定**）。

09b.9 `SCALE_AWARE`：灰区网格自适应函数

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8286-L8361
L08286
L08287  SUBROUTINE SCALE_AWARE(dx,pblh,Psig_bl,Psig_shcu)
L08288
L08289     !-------------------------------------------------------------
L08290     !             NOTES ON SCALE-AWARE FORMULATION
L08291     !
L08292     !     scale-aware factor (Psig) here, taken from Honnert et al. (2011,
L08293     !     JAS) and/or from Hyeyum Hailey Shin and Song-You Hong (2013, JAS)
L08294     !
L08295     ! Psig_bl tapers local mixing
L08296     ! Psig_shcu tapers nonlocal mixing
L08297
L08298    use module_bl_mynnedmf_common, only: zero,one,ten,kind_phys
L08299
L08300     real(kind_phys), intent(in)  :: dx,pblh
L08301     real(kind_phys), intent(out) :: Psig_bl,Psig_shcu
L08302     real(kind_phys)              :: dxdh
L08303
L08304     Psig_bl=one
L08305     Psig_shcu=one
L08306     dxdh=MAX(2.5_kind_phys*dx,ten)/MIN(PBLH,3000._kind_phys)
L08307     ! Honnert et al. 2011, TKE in PBL  *** original form used until 201605
L08308     !Psig_bl= ((dxdh**2) + 0.07*(dxdh**0.667))/((dxdh**2) + &
L08309     !         (3./21.)*(dxdh**0.67) + (3./42.))
L08310     ! Honnert et al. 2011, TKE in entrainment layer
L08311     !Psig_bl= ((dxdh**2) + (4./21.)*(dxdh**0.667))/((dxdh**2) + &
L08312      !        (3./20.)*(dxdh**0.67) + (7./21.))
L08313     ! New form to preseve parameterized mixing - only down 5% at dx = 750 m
```

```
L08314      Psig_bl= ((dxdh**2) + 0.106_kind_phys*(dxdh**0.667_kind_phys))/((dxdh**2) + &
L08315           0.066_kind_phys*(dxdh**0.667_kind_phys) + 0.071_kind_phys)
L08316
L08317      !assume a 500 m cloud depth for shallow-cu clods
L08318      dxdh=MAX(2.5_kind_phys*dx,ten)/MIN(PBLH+500._kind_phys,3500._kind_phys)
L08319      ! Honnert et al. 2011, TKE in entrainment layer *** original form used until 201605
L08320      !Psig_shcu= ((dxdh**2) + (4./21.)*(dxdh**0.667))/((dxdh**2) + &
L08321      !         (3./20.)*(dxdh**0.67) + (7./21.))
L08322
L08323      ! Honnert et al. 2011, TKE in cumulus
L08324      !Psig(i)= ((dxdh**2) + 1.67*(dxdh**1.4))/((dxdh**2) +1.66*(dxdh**1.4) +
L08325      !0.2)
L08326
L08327      ! Honnert et al. 2011, w'q' in PBL
L08328      !Psig(i)= 0.5 + 0.5*((dxdh**2) + 0.03*(dxdh**1.4) -
L08329      !(4./13.))/((dxdh**2) + 0.03*(dxdh**1.4) + (4./13.))
L08330      ! Honnert et al. 2011, w'q' in cumulus
L08331      !Psig(i)= ((dxdh**2) - 0.07*(dxdh**1.4))/((dxdh**2) -0.07*(dxdh**1.4) +
L08332      !0.02)
L08333
L08334      ! Honnert et al. 2011, q'q' in PBL
L08335      !Psig(i)= 0.5 + 0.5*((dxdh**2) + 0.25*(dxdh**0.667) -0.73)/((dxdh**2)
L08336      !-0.03*(dxdh**0.667) + 0.73)
L08337      ! Honnert et al. 2011, q'q' in cumulus
L08338      !Psig(i)= ((dxdh**2) - 0.34*(dxdh**1.4))/((dxdh**2) - 0.35*(dxdh**1.4)
L08339      !+ 0.37)
L08340
L08341      ! Hyeyum Hailey Shin and Song-You Hong 2013, TKE in PBL (same as Honnert's above)
L08342      !Psig_shcu= ((dxdh**2) + 0.070*(dxdh**0.667))/((dxdh**2)
L08343      !+0.142*(dxdh**0.667) + 0.071)
L08344      ! Hyeyum Hailey Shin and Song-You Hong 2013, TKE in entrainment zone  *** switch to this form 201605
L08345      Psig_shcu= ((dxdh**2) + 0.145_kind_phys*(dxdh**0.667_kind_phys))/((dxdh**2) + &
L08346           0.172_kind_phys*(dxdh**0.667_kind_phys) + 0.170_kind_phys)
L08347
L08348      ! Hyeyum Hailey Shin and Song-You Hong 2013, w'theta' in PBL
L08349      !Psig(i)= 0.5 + 0.5*((dxdh**2) -0.098)/((dxdh**2) + 0.106)
L08350      ! Hyeyum Hailey Shin and Song-You Hong 2013, w'theta' in entrainment zone
L08351      !Psig(i)= 0.5 + 0.5*((dxdh**2) - 0.112*(dxdh**0.25) -0.071)/((dxdh**2)
L08352      !+ 0.054*(dxdh**0.25) + 0.10)
L08353
L08354      !print*,"in scale_aware; dx, dxdh, Psig(i)=",dx,dxdh,Psig(i)
L08355      !If(Psig_bl(i) < 0.0 .OR. Psig(i) > 1.)print*,"dx, dxdh, Psig(i)=",dx,dxdh,Psig_bl(i)
L08356      If(Psig_bl > one) Psig_bl=one
L08357      If(Psig_bl < zero) Psig_bl=zero
L08358
L08359      If(Psig_shcu > one) Psig_shcu=one
L08360      If(Psig_shcu < zero) Psig_shcu=zero
L08361
```

- **两个因子：**`Psig_bl`（衰减局地混合）与 `Psig_shcu`（衰减非局地混合）——出处 Honnert et al. (2011, JAS) 与 Shin & Hong (2013, JAS)，自变量 `dxdh = 2.5 • dx/min(PBLH, 3000)`（网格距与边界层厚度之比）。
- **函数考古**（注释保留六代形式）：Honnert TKE-PBL/TKE-夹带层/w'q'/q'q'、Shin-Hong TKE/w'theta'——现行选择：Psig_bl 用 “保参数化混合” 版（dx=750 m 只降 5%）、Psig_shcu 用 Shin-Hong 夹带层版（201605 切换）。**这是 WRF 灰区模拟（convection-permitting 到 LES 之间）的教科书实现**——教材 ch14 §14.4 的延伸阅读指向。

09b.10 blend 饱和函数族：与 Thompson 同源的 “相位混合”

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8371-L8414
L08371  !! the module (replacing "svp").
L08372    FUNCTION esat_blend(t)
L08373
L08374        use module_bl_mynnedmf_common, only: t0c,tice,one,kind_phys
L08375
L08376        real(kind_phys), intent(in):: t
L08377        real(kind_phys):: esat_blend,XC,ESL,ESI,chi
L08378        !liquid
L08379        real(kind_phys), parameter:: J0= .611583699E03
L08380        real(kind_phys), parameter:: J1= .444606896E02
L08381        real(kind_phys), parameter:: J2= .143177157E01
L08382        real(kind_phys), parameter:: J3= .264224321E-1
L08383        real(kind_phys), parameter:: J4= .299291081E-3
L08384        real(kind_phys), parameter:: J5= .203154182E-5
L08385        real(kind_phys), parameter:: J6= .702620698E-8
L08386        real(kind_phys), parameter:: J7= .379534310E-11
L08387        real(kind_phys), parameter:: J8=-.321582393E-13
L08388        !ice
L08389        real(kind_phys), parameter:: K0= .609868993E03
L08390        real(kind_phys), parameter:: K1= .499320233E02
L08391        real(kind_phys), parameter:: K2= .184672631E01
L08392        real(kind_phys), parameter:: K3= .402737184E-1
L08393        real(kind_phys), parameter:: K4= .565392987E-3
L08394        real(kind_phys), parameter:: K5= .521693933E-5
L08395        real(kind_phys), parameter:: K6= .307839583E-7
L08396        real(kind_phys), parameter:: K7= .105785160E-9
L08397        real(kind_phys), parameter:: K8= .161444444E-12
L08398
L08399        XC=MAX(-80._kind_phys, t - t0c) !note t0c = 273.15, tice is set in module mynn_common to 240
L08400
L08401  ! For 240 < t < 268.16 K, the vapor pressures are "blended" as a function of temperature,
L08402  ! using the approach similar to Chaboureau and Bechtold (2002), JAS, p. 2363.  The resulting
L08403  ! values are returned from the function.
L08404        IF (t .GE. (t0c-6._kind_phys)) THEN
L08405            esat_blend = J0+XC*(J1+XC*(J2+XC*(J3+XC*(J4+XC*(J5+XC*(J6+XC*(J7+XC*J8)))))))
L08406        ELSE IF (t .LE. tice) THEN
```

```
L08407            esat_blend = K0+XC*(K1+XC*(K2+XC*(K3+XC*(K4+XC*(K5+XC*(K6+XC*(K7+XC*K8)))))))
L08408         ELSE
L08409            ESL = J0+XC*(J1+XC*(J2+XC*(J3+XC*(J4+XC*(J5+XC*(J6+XC*(J7+XC*J8)))))))
L08410            ESI = K0+XC*(K1+XC*(K2+XC*(K3+XC*(K4+XC*(K5+XC*(K6+XC*(K7+XC*K8)))))))
L08411            chi = ((t0c-6._kind_phys) - t)/((t0c-6._kind_phys) - tice)
L08412            esat_blend = (one-chi)*ESL  + chi*ESI
L08413         END IF
L08414
```

`esat_blend`：**JAYMES 2015-04-22 加入**，头注释明说 “Separate ice and liquid functions are used (**identical to those in module_mp_thompson.F, v3.6**)”——**与 Thompson 的 RSLF/RSIF 同一 Flatau 多项式**（J0–J8/K0–K8 系数逐位一致，文库 code_07b §07b.4）；温度混合区间 **240 K（tice）~267.15 K（t0c−6）**：`chi = (267.15−T)/(267.15−240)` 线性插值液/冰面饱和压——**Chaboureau & Bechtold (2002) 的相位混合思想**。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8423-L8471
L08423    FUNCTION qsat_blend(t, P)
L08424
L08425        use module_bl_mynnedmf_common, only: t0c,tice,one,kind_phys
L08426
L08427        real(kind_phys), intent(in):: t, P
L08428        real(kind_phys):: qsat_blend,XC,ESL,ESI,RSLF,RSIF,chi
L08429        !liquid
L08430        real(kind_phys), parameter:: J0= .611583699E03
L08431        real(kind_phys), parameter:: J1= .444606896E02
L08432        real(kind_phys), parameter:: J2= .143177157E01
L08433        real(kind_phys), parameter:: J3= .264224321E-1
L08434        real(kind_phys), parameter:: J4= .299291081E-3
L08435        real(kind_phys), parameter:: J5= .203154182E-5
L08436        real(kind_phys), parameter:: J6= .702620698E-8
L08437        real(kind_phys), parameter:: J7= .379534310E-11
L08438        real(kind_phys), parameter:: J8=-.321582393E-13
L08439        !ice
L08440        real(kind_phys), parameter:: K0= .609868993E03
L08441        real(kind_phys), parameter:: K1= .499320233E02
L08442        real(kind_phys), parameter:: K2= .184672631E01
L08443        real(kind_phys), parameter:: K3= .402737184E-1
L08444        real(kind_phys), parameter:: K4= .565392987E-3
L08445        real(kind_phys), parameter:: K5= .521693933E-5
L08446        real(kind_phys), parameter:: K6= .307839583E-7
L08447        real(kind_phys), parameter:: K7= .105785160E-9
L08448        real(kind_phys), parameter:: K8= .161444444E-12
L08449
L08450        XC=MAX(-80._kind_phys,t - t0c)
L08451
L08452        IF (t .GE. (t0c-6._kind_phys)) THEN
L08453            ESL  = J0+XC*(J1+XC*(J2+XC*(J3+XC*(J4+XC*(J5+XC*(J6+XC*(J7+XC*J8)))))))
L08454            ESL  = min(ESL, P*0.15_kind_phys) ! Even with P=1050mb and T=55C, the sat. vap. pres only contributes to ~15%
↪   of total pres.
L08455            qsat_blend = 0.622_kind_phys*ESL/max(P-ESL, 1e-5_kind_phys)
L08456        ELSE IF (t .LE. tice) THEN
L08457            ESI  = K0+XC*(K1+XC*(K2+XC*(K3+XC*(K4+XC*(K5+XC*(K6+XC*(K7+XC*K8)))))))
L08458            ESI  = min(ESI, P*0.15_kind_phys)
L08459            qsat_blend = 0.622*ESI/max(P-ESI, 1e-5_kind_phys)
L08460        ELSE
L08461            ESL  = J0+XC*(J1+XC*(J2+XC*(J3+XC*(J4+XC*(J5+XC*(J6+XC*(J7+XC*J8)))))))
L08462            ESL  = min(ESL, P*0.15_kind_phys)
L08463            ESI  = K0+XC*(K1+XC*(K2+XC*(K3+XC*(K4+XC*(K5+XC*(K6+XC*(K7+XC*K8)))))))
L08464            ESI  = min(ESI, P*0.15_kind_phys)
L08465            RSLF = 0.622_kind_phys*ESL/max(P-ESL, 1e-5_kind_phys)
L08466            RSIF = 0.622_kind_phys*ESI/max(P-ESI, 1e-5_kind_phys)
L08467  !          chi  = (268.16-t)/(268.16-240.)
L08468            chi  = ((t0c-6._kind_phys) - t)/((t0c-6._kind_phys) - tice)
L08469            qsat_blend = (one-chi)*RSLF + chi*RSIF
L08470        END IF
L08471
```

`qsat_blend`：同一混合逻辑应用到饱和混合比（es→q 换算含 `min(es, 0.15·P)` 钳位与 `max(P−es, 1e-5)` 防零——**Thompson 的两处保护在这里重现**）。**跨方案一致性**：边界层的饱和计算与微物理层用同一口径，消除了 “边界层云量与微物理云水判据打架” 的隐患。

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8481-L8501
L08481    FUNCTION xl_blend(t)
L08482
L08483       use module_bl_mynnedmf_common, only: xlv,xls,cpv,cliq,cice,t0c,tice,&
L08484            one,kind_phys
L08485
L08486        real(kind_phys), intent(in):: t
L08487        real(kind_phys):: xl_blend,xlvt,xlst,chi
L08488        !note: t0c = 273.15, tice is set in mynn_common
L08489
L08490        IF (t .GE. t0c) THEN
L08491            xl_blend = xlv + (cpv-cliq)*(t-t0c)  !vaporization/condensation
L08492        ELSE IF (t .LE. tice) THEN
L08493            xl_blend = xls + (cpv-cice)*(t-t0c)  !sublimation/deposition
L08494        ELSE
L08495            xlvt = xlv + (cpv-cliq)*(t-t0c)  !vaporization/condensation
L08496            xlst = xls + (cpv-cice)*(t-t0c)  !sublimation/deposition
L08497  !          chi  = (273.16-t)/(273.16-240.)
L08498            chi  = (t0c - t)/(t0c - tice)
L08499            xl_blend = (one-chi)*xlvt + chi*xlst     !blended
L08500        END IF
L08501
```

`xl_blend`：潜热的温度混合（Kircchoff 线性式 `xlv + (cpv−cliq)·(T−T0)` / `xls + (cpv−cice)·(T−T0)` 按 chi 插值）——与 Thompson 主实体的 `lvap(T)`（code_07a §07a.8.3）同思想但区间不同（这里 240–273.15 K 全混）。

09b.11 `phim` / `phih`：Puhales (2020) 新相似函数

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8506-L8603
L08506    FUNCTION phim(zet)
L08507       ! New stability function parameters for momentum (Puhales, 2020, WRF 4.2.1)
L08508       ! The forms in unstable conditions (z/L < 0) use Grachev et al. (2000), which are a blend of
L08509       ! the classical (Kansas) forms (i.e., Paulson 1970, Dyer and Hicks 1970), valid for weakly
L08510       ! unstable conditions (-1 < z/L < 0). The stability functions for stable conditions use an
L08511       ! updated form taken from Cheng and Brutsaert (2005), which extends the validity into very
L08512       ! stable conditions [z/L ~ O(10)].
L08513        use module_bl_mynnedmf_common, only: cphm_unst,zero,one,two,p333,p5,p666,kind_phys
L08514
L08515        real(kind_phys), intent(in):: zet
L08516        real(kind_phys):: dummy_0,dummy_1,dummy_11,dummy_2,dummy_22,dummy_3,dummy_33,dummy_4,dummy_44,dummy_psi
L08517        real(kind_phys), parameter :: am_st=6.1, bm_st=2.5, rbm_st=1./bm_st
L08518        real(kind_phys), parameter :: ah_st=5.3, bh_st=1.1, rbh_st=1./bh_st
L08519        real(kind_phys), parameter :: am_unst=10., ah_unst=34.
L08520        real(kind_phys):: phi_m,phim
L08521
L08522        if ( zet >= zero ) then
L08523           dummy_0 = one+zet**bm_st
L08524           dummy_1 = zet+dummy_0**(rbm_st)
L08525           dummy_11= one+dummy_0**(rbm_st-1)*zet**(bm_st-one)
L08526           dummy_2 = (-am_st/dummy_1)*dummy_11
L08527           phi_m   = one-zet*dummy_2
L08528        else
L08529           dummy_0 = (one-cphm_unst*zet)**0.25
L08530           phi_m   = one/dummy_0
L08531           dummy_psi = two*log(p5*(one+dummy_0))+log(p5*(one+dummy_0**2))-two*atan(dummy_0)+1.570796
L08532
L08533           dummy_0 = (one-am_unst*zet)          ! parentesis arg
L08534           dummy_1 = dummy_0**p333              ! y
L08535           dummy_11=-p333*am_unst*dummy_0**(-p666) ! dy/dzet
L08536           dummy_2 = p333*(dummy_1**2+dummy_1+one)      ! f
L08537           dummy_22= p333*dummy_11*(two*dummy_1+one)    ! df/dzet
L08538           dummy_3 = 0.57735*(two*dummy_1+one)  ! g
L08539           dummy_33= 1.1547*dummy_11            ! dg/dzet
L08540           dummy_4 = 1.5*log(dummy_2)-1.73205*atan(dummy_3)+1.813799364 !psic
L08541           dummy_44= (1.5/dummy_2)*dummy_22-1.73205*dummy_33/(one+dummy_3**2)! dpsic/dzet
L08542
L08543           dummy_0 = zet**2
L08544           dummy_1 = one/(one+dummy_0) ! denon
L08545           dummy_11= two*zet           ! denon/dzet
L08546           dummy_2 = ((one-phi_m)/zet+dummy_11*dummy_4+dummy_0*dummy_44)*dummy_1
L08547           dummy_22= -dummy_11*(dummy_psi+dummy_0*dummy_4)*dummy_1**2
L08548
L08549           phi_m = one-zet*(dummy_2+dummy_22)
L08550        end if
L08551
L08552        !phim = phi_m - zet
L08553        phim = phi_m
L08554
L08555    END FUNCTION phim
L08556  ! ==================================================================
L08557
L08558    FUNCTION phih(zet)
L08559      ! New stability function parameters for heat (Puhales, 2020, WRF 4.2.1)
L08560      ! The forms in unstable conditions (z/L < 0) use Grachev et al. (2000), which are a blend of
L08561      ! the classical (Kansas) forms (i.e., Paulson 1970, Dyer and Hicks 1970), valid for weakly
L08562      ! unstable conditions (-1 < z/L < 0). The stability functions for stable conditions use an
L08563      ! updated form taken from Cheng and Brutsaert (2005), which extends the validity into very
L08564      ! stable conditions [z/L ~ O(10)].
L08565        use module_bl_mynnedmf_common, only: cphh_unst,zero,one,two,p333,p5,p666,kind_phys
L08566
L08567        real(kind_phys), intent(in):: zet
L08568        real(kind_phys):: dummy_0,dummy_1,dummy_11,dummy_2,dummy_22,dummy_3,dummy_33,dummy_4,dummy_44,dummy_psi
L08569        real(kind_phys), parameter :: am_st=6.1, bm_st=2.5, rbm_st=1./bm_st
L08570        real(kind_phys), parameter :: ah_st=5.3, bh_st=1.1, rbh_st=1./bh_st
L08571        real(kind_phys), parameter :: am_unst=10., ah_unst=34.
L08572        real(kind_phys):: phh,phih
L08573
L08574        if ( zet >= zero ) then
L08575           dummy_0 = one+zet**bh_st
L08576           dummy_1 = zet+dummy_0**(rbh_st)
L08577           dummy_11= one+dummy_0**(rbh_st-one)*zet**(bh_st-one)
L08578           dummy_2 = (-ah_st/dummy_1)*dummy_11
L08579           phih    = one-zet*dummy_2
L08580        else
L08581           dummy_0 = (one-cphh_unst*zet)**p5
L08582           phh     = one/dummy_0
L08583           dummy_psi = two*log(p5*(one+dummy_0))
L08584
L08585           dummy_0 = (one-ah_unst*zet)          ! parenthesis arg
L08586           dummy_1 = dummy_0**p333              ! y
L08587           dummy_11=-p333*ah_unst*dummy_0**(-p666) ! dy/dzet
L08588           dummy_2 = p333*(dummy_1**2+dummy_1+one)      ! f
L08589           dummy_22= p333*dummy_11*(two*dummy_1+one)    ! df/dzet
L08590           dummy_3 = 0.57735*(two*dummy_1+one) ! g
L08591           dummy_33= 1.1547*dummy_11           ! dg/dzet
L08592           dummy_4 = 1.5*log(dummy_2)-1.73205*atan(dummy_3)+1.813799364 !psic
L08593           dummy_44= (1.5/dummy_2)*dummy_22-1.73205*dummy_33/(one+dummy_3**2)! dpsic/dzet
L08594
L08595           dummy_0 = zet**2
L08596           dummy_1 = one/(one+dummy_0)          ! denon
L08597           dummy_11= two*zet                    ! ddenon/dzet
L08598           dummy_2 = ((one-phh)/zet+dummy_11*dummy_4+dummy_0*dummy_44)*dummy_1
L08599           dummy_22= -dummy_11*(dummy_psi+dummy_0*dummy_4)*dummy_1**2
```

```
L08600
L08601           phih = one-zet*(dummy_2+dummy_22)
L08602         end if
L08603
```

- **设计**（头注释）：稳定侧用 **Cheng & Brutsaert (2005)**（`am_st=6.1`，`bm_st=2.5` 幂律有理式——把稳定函数的有效性延到 z/L ~ O(10) 强稳定）；不稳定侧用 **Grachev et al. (2000)**（Kansas（Paulson/Dyer-Hicks）与 Businger-Dyer 形式的混合，适用 −1<z/L<0），实现是一套 `dummy_0`…`dummy_44` 的链式求导（ψ 函数及其导数）——**复杂度高但光滑**。
- **调用位**：主入口 `bl_mynn_stfunc=1` 时（§09a.4.5）以及 `phi_m = phim(zet)`（TKE 收支底边界）——**默认路径仍走 Kansas 分支**，新函数待命。

09b.12 `topdown_cloudrad`：云顶辐射冷却的解析 TKE 源

```
!file: phys/MYNN-EDMF/module_bl_mynnedmf.F90    L8605-L8770
L08605  ! ==================================================================
L08606   subroutine topdown_cloudrad(kts,kte,                              &
L08607                 &dz1,zw,fltv,u1,v1,xland,kpbl,pblh,                 &
L08608                 &sqc,sqi,sqw,thl,th1,ex1,pres1,rho1,thv,            &
L08609                 &cldfra_bl1,qc_bl,qi_bl,rthraten,                   &
L08610                 &tkeprod_dn,psig,                                   &
L08611                 &maxtkeprod,cldtop_cooling,ent_eff)
L08612
L08613    use module_bl_mynnedmf_common, only: cp,ep_2,xlv,r_d,p608, &
L08614         xlvcp,grav,zero,one,two,three,eight,p1,p2,p25,p3,     &
L08615         p333,p5,kind_phys
L08616
L08617     !input
L08618     integer,         intent(in) :: kte,kts
L08619     real(kind_phys), dimension(kts:kte), intent(in) :: dz1,sqc,sqi,sqw,&
L08620          thl,th1,ex1,pres1,rho1,thv,cldfra_bl1,qc_bl,qi_bl
L08621     real(kind_phys), dimension(kts:kte), intent(in) :: rthraten
L08622     real(kind_phys), dimension(kts:kte+1), intent(in) :: zw
L08623     real(kind_phys), intent(in) :: pblh,fltv,u1,v1,psig
L08624     real(kind_phys), intent(in) :: xland
L08625     integer        , intent(in) :: kpbl
L08626     !output - production of tke from cloud-top radiative cooling
L08627     real(kind_phys), dimension(kts:kte), intent(inout) :: tkeprod_dn
L08628     real(kind_phys), intent(inout) :: maxtkeprod
L08629     real(kind_phys), intent(inout) :: cldtop_cooling
L08630     real(kind_phys), intent(inout) :: ent_eff
L08631     !local
L08632     real(kind_phys),parameter:: dz500   = 500.  !scale height for pbl
L08633     real(kind_phys),parameter:: dz200   = 200.  !scale height for entrainment layer
L08634     real(kind_phys),parameter:: cooling_thresh = -0.000069  !activation threshold of turbulence
L08635                                                !-0.000116 is ~ -10 C cooling at cloud top per day
L08636                                                !-0.000093 is ~  -8 C cooling at cloud top per day
L08637                                                !-0.000069 is ~  -6 C cooling at cloud top per day
L08638     real(kind_phys), dimension(kts:kte) :: zfac,zfacent,tkeprodorig
L08639     real(kind_phys) :: dthvx,tmp1,zfacent_max,zagl
L08640     real(kind_phys) :: temps,templ,zl1,zb1,dz_ent,wstar_rad,maxcooling
L08641     real(kind_phys) :: f0,radflux,went,rcldb,rvls,minrad,zminrad,dp
L08642     real(kind_phys) :: wspd_pbl,ac_wsp
L08643     real(kind_phys), parameter :: pfac =2.0, zfmin = 0.01, phifac=8.0
L08644     integer :: k,kk,kminrad
L08645     logical :: cloudflg
L08646     logical,parameter:: debug=.false.
L08647
L08648     cloudflg       = .false.
L08649     minrad         = zero
L08650     kminrad        = kpbl
L08651     zminrad        = pblh
L08652     !save the tke production from the downdraft scheme for comparison
L08653     tkeprodorig(kts:kte)=tkeprod_dn(kts:kte)
L08654
L08655     !check for stratocumulus- or fog-tops with cooling at top
L08656     do k = 1,min(kpbl+10,kte-1)
L08657        maxcooling = minval(rthraten(max(kts,k-1):k+1))
L08658        !---------------------------------criteria for downdraft activa tion:
L08659        if ((cldfra_bl1(k).gt.p5 .and. cldfra_bl1(k+1).lt.p5) .and. &  !1) stratocu cloud top exists
L08660            (maxcooling  .lt. cooling_thresh)) then                    !2) significant radiative cooling
L08661           ! found sc cloud with significant radiative cooling
L08662           cloudflg =.true.
L08663           if (maxcooling < minrad) then
L08664              minrad    =maxcooling      !maximum cooling rate at cloud top
L08665              kminrad   = k
L08666              zminrad   = zw(k) + p5*dz1(k) !Best estimate of height of TKE source (top of downdrafts)
L08667           endif
L08668        endif
L08669     enddo
L08670
L08671     if (cloudflg) then
L08672        zl1     = p25*dz1(kts)
L08673        k       = kminrad
L08674
L08675        templ   = thl(k)*ex1(k)
L08676        !rvls is ws at full level
L08677        rvls    = 100.*6.112*EXP(17.67*(templ-273.16)/(templ-29.65))*(ep_2/pres1(k+1))
L08678        temps   = templ + (sqw(k)-rvls)/(cp/xlv  +  ep_2*xlv*rvls/(r_d*templ**2))
L08679        rvls    = 100.*6.112*EXP(17.67*(temps-273.15)/(temps-29.65))*(ep_2/pres1(k+1))
L08680        rcldb   = max(sqw(k)-rvls,zero)         !resolved cloud water
L08681        rcldb   = max(rcldb, qc_bl(k)+qi_bl(k)) !any type of cloud water (sgs or resolved)
L08682
L08683        !entrainment efficiency
L08684        dthvx   = (thl(k+2) + th1(k+2)*p608*sqw(k+2)) - &
L08685                  (thl(k)   + th1(k)  *p608*sqw(k))
L08686        !reduce vertical resolution sensitivity; normalize to 200 m depth
```

```
L08687         dz_ent  = (dz1(k  )*dz1(k+1)+dz1(k+1)*dz1(k  ))/(dz1(k+1)+dz1(k)) + &
L08688                   (dz1(k+1)*dz1(k+2)+dz1(k+2)*dz1(k+1))/(dz1(k+1)+dz1(k+2))
L08689         dthvx   = dthvx * dz200/dz_ent  !normalized gradient
L08690         dthvx   = max(dthvx, p1)
L08691         tmp1    = xlvcp * rcldb/(ex1(k)*dthvx)
L08692         !entrainment efficiency: originally from Nichols and Turton (1986), where a2 = 60, but lowered
L08693         !here to 8, as in Grenier and Bretherton (2001).
L08694         ent_eff = min(two, p2 + p2*eight*tmp1)
L08695
L08696         f0      = zero
L08697         do kk = max(kts,kminrad-1),kminrad+1
L08698            radflux = rthraten(kk)*ex1(kk)             !converts theta/s to temp/s
L08699            dp      = rho1(k)*grav*dz1(k)
L08700            radflux = radflux * cp / grav * dp         !converts temp/s to w/m^2 over layer
L08701            if (radflux < zero ) then
L08702               f0   = abs(radflux)+f0                  !sum of cooling (w/m^2) over relevant layers
L08703            endif
L08704         enddo
L08705
L08706         !more strict limits over land to reduce stable-layer mixouts
L08707         if ((xland-1.5).ge.zero) then     ! water
L08708            f0     = min(f0, 150.0_kind_phys)
L08709         else                              ! land - limited considerably
L08710            f0     = min(p25*f0, 30._kind_phys)
L08711         endif
L08712         cldtop_cooling = abs(minrad) * 3600.   ! (K/hr) max cooling
L08713
L08714         !entrainment from pbl top thermals
L08715         wstar_rad = 1.25_kind_phys * ( grav/thv(k) * dz200 * f0 / (rho1(k) * cp) ) **p333
L08716         wstar_rad = min(max(wstar_rad, p1), three)
L08717         dthvx     = max(thv(k+1)-thv(k), p1)
L08718         went      = p333*ent_eff*wstar_rad     !entrainment velocity (m/s)
L08719
L08720         !compute normalized analytical profiles (max=1). constrain the profile of the production to not
L08721         !always go to the surface if forcing isnt strong enough, i.e., depth ~ wstar * Timescale of descent
L08722         zb1       = max(z11, zminrad - wstar_rad*800._kind_phys)
L08723         do k = kts,kminrad+3
L08724            !for fog at k=1, make min height above ground (zagl) larger than first model deptk
L08725            zagl       = zw(k) + p5*dz1(k)
L08726            if (k==kts .and. kminrad==kts) zagl = 0.4_kind_phys * dz1(kts)
L08727            !analytic vertical profile
L08728            zfac(k)    = min(max((one - max(zero,zagl-zb1)/(zminrad-z11)), zfmin), one)
L08729            zfacent(k) = max((zminrad-zagl)/zminrad, zero)*(one-zfac(k))**3
L08730         enddo
L08731         zfacent_max = maxval(zfacent(kts:kminrad+3))
L08732         zfacent     = zfacent/max(zfacent_max, 1e-5_kind_phys)  !normalize zfacent
L08733
L08734         !calculate tke production
L08735         do k = kts,kminrad+3
L08736            tkeprod_dn(k) = max(tkeprod_dn(k), p3*ent_eff*wstar_rad**3/dz500*zfacent(k))
L08737            tkeprod_dn(k) = max(tkeprod_dn(k), zero)
L08738            tkeprod_dn(k) = min(0.0004_kind_phys, tkeprod_dn(k))
L08739         enddo
L08740         !make sure there is some TKE source at the max cooling level
L08741         tkeprod_dn(kminrad)=max(tkeprod_dn(kminrad), p333*tkeprod_dn(max(kts,kminrad-1)))
L08742         !make sure there is a small TKE source above the max cooling level
L08743         tkeprod_dn(kminrad+1)=max(tkeprod_dn(kminrad+1), min(p5,ent_eff)*tkeprod_dn(max(kts,kminrad)))
L08744
L08745         !taper off tke production in high-wind conditions and at high resolutions
L08746         wspd_pbl   = sqrt(max(u1**2 + v1**2, 0.01_kind_phys))
L08747         ac_wsp     = one - min(max(zero, wspd_pbl - 10._kind_phys)/15._kind_phys, one)
L08748         tkeprod_dn = tkeprod_dn * min(ac_wsp, psig)
L08749      endif !end cloud check
L08750
L08751      !calculate the maximum tke production in the profile and convert to an hourly rate.
L08752      maxtkeprod = maxval(tkeprod_dn)*3600.
L08753
L08754      if (cloudflg .and. debug) then
L08755         !debug printouts
L08756         print *, '-----------------------------------------------------------------'
L08757         if ((xland-1.5).ge.zero) then
L08758            print *, ' over water, kminrad=',kminrad
L08759         else
L08760            print *, ' over land, kminrad=',kminrad
L08761         endif
L08762         print *, ' ctop cooling (K/hr)=',minrad*3600.,' rad cooling (W/m^2)=',f0
L08763         print *, ' ent_eff=',ent_eff,' went=',went,' w*rad=',wstar_rad
L08764         print *, '  k    cldfra   dT/dt(hr)  dTKE/dt_orig dTKE/dt(hr)  zfacent'
L08765         print *, '-----------------------------------------------------------------'
L08766         do k = kminrad+3,kts,-1
L08767            print '(I4,5F11.5)', k, cldfra_bl1(k), rthraten(k)*3600., tkeprodorig(k)*3600., tkeprod_dn(k)*3600.,
↪  zfacent(k)
L08768         enddo
L08769      endif
L08770
```

- **触发**：与 ddmp_mf 相同的云顶判据（层云顶＋冷却＜−6 °C/天）——**它是 ddmp_mf 的伴生通道**（主入口里先后调用，本函数保证 “MF 不适用时仍有 TKE 源”）。返回三个诊断：`maxtkeprod` / `cldtop_cooling` / `ent_eff`。
- **强迫强度 f0**：云顶附近辐射冷却的垂直积分（W/m²），**水面限 150、陆地限 0.25 · f0 且 ≤30 W/m²**（注释：“more strict limits over land to reduce stable-layer mixouts”——陆地层云混合过强是历史偏差）。
- **解析剖面**：`zfac` 线性衰减（从云顶到 `zb1 = max(0.25dz, zminrad − w* · 800)`——** 下沉穿透深度 = w*× 时间尺度 800 s**）、`zfacent` 三次方卷入剖面（归一化）。
- **TKE 生产**：`tkeprod_dn = max(orig, 0.3 · ent_eff · w*³/dz500 · zfacent)` ≤ 4e-4——**w*³ 标度（对流 TKE 通量）× ent_eff（云顶效率）× 解析形状 **；云顶处保底 1/3 邻层值、其上保底 min(0.5, ent_eff) 倍。

- **衰减**：风衰减 `ac_wsp`（>10 m/s 线性）× `psig`（灰区）——**三重一致性**（与 ddmp_mf 面积、DMP_mf 触发共用同一对衰减函数）。

09b.13 改造入口（→ 教材 ch20/ch21 案索引）

想改什么	动哪里	注意
羽流数/尺寸谱	`DMP_mf` 的 nup/lmin/lmax/d	d 的幂律敏感性注释在侧
卷入率	`DMP_mf` entfac（Tian & Kuang）	注释保留 Neggers/TEMF 备选
下沉流触发	`ddmp_mf` cooling_thresh	注释三档阈值折算 K/天
灰区行为	`SCALE_AWARE` 两因子	六代函数考古可对比
负值修复策略	`moisture_check`	与 Thompson 守恒段口径对读
饱和口径	blend 三函数	与 Thompson RSLF/RSIF 系数同源
PBLH 判据	`GET_PBLH` stable_method/delt_thv	水陆阈值分开
教材案	ch21 案（BouLac 对比/虚拟温度）	—

09b.14 上下游链接

- **上游（本模块内）**：code_09a——主入口调用序列、闭合常数（module_bl_mynnedmf_common 的 b1/b2/a1/a2）、`mym_turbulence/mym_predict` 的求解器对接。
- **外部对接**：`module_mp_thompson.F`（blend 函数族同源、`rthraten` 来自辐射方案的耦合——ddmp_mf/topdown 的输入）；`module_pbl_driver.F`（code_22，CASE 分发）。
- **教材**：ch14（MYNN-EDMF 精读：qke=2TKE、dmp/ddmp_mf、get_pblh 全文、BouLac 亲缘）、ch15（对流速度尺度 w* 与夹卷的积云对应）、ch21（solve_em 的 PBL 调用位）。

code_10 — MYNN 表面层方案（MYNN-SFC 四件套 + 双运行线）

一、版本指纹

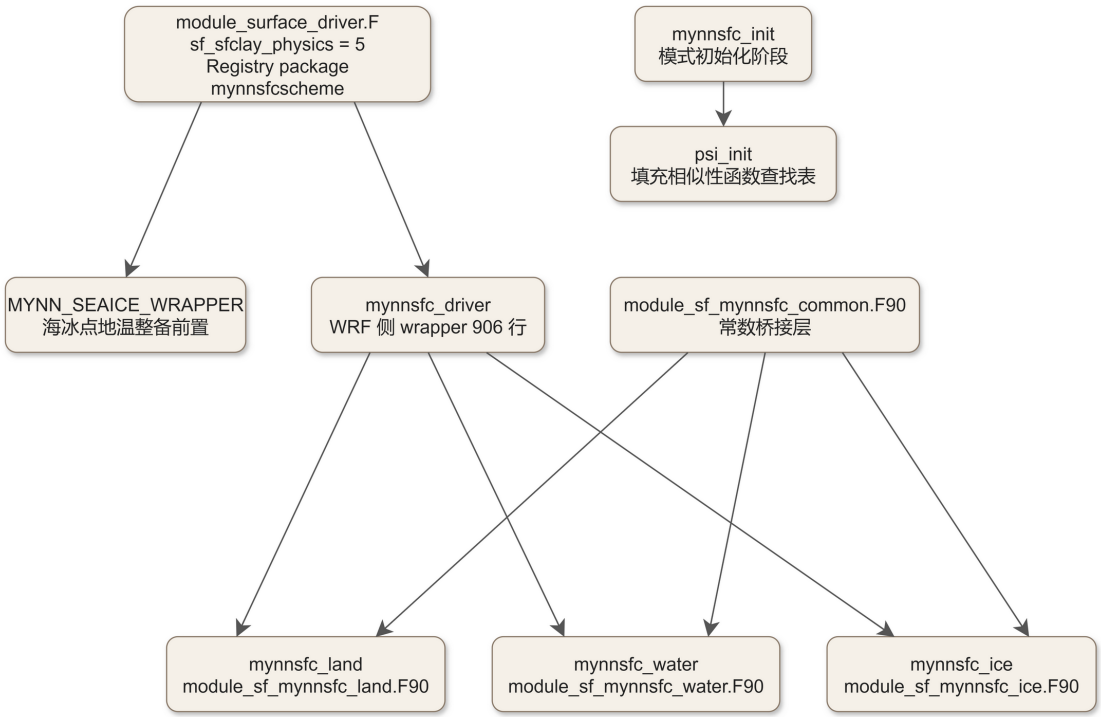
源文件	行数	md5（整文件，按字节）
phys/MYNN-SFC/module_sf_mynnsfc.F90	4087	3d4f0bba16ed9af55bc77ca2d804b661
phys/MYNN-SFC/WRF/module_sf_mynnsfc_driver.F90	906	3552d31bcd3f7f9a36087ca9dc4c6b99
phys/MYNN-SFC/WRF/module_sf_mynnsfc_common.F90	77	a159e6535dd733beadde213795809b7d
phys/MYNN-SFC/module_sf_mynnsfc_land.F90	1749	d523ca4a2811fe481582f7a55f2719ff
phys/MYNN-SFC/module_sf_mynnsfc_water.F90	2104	09a709192675a2fac471033a04c3bdb5
phys/MYNN-SFC/module_sf_mynnsfc_ice.F90	1552	82ecb47710aa19b163a5ebdf68f6421c

- 抽取工具：workspace/scripts/code_extract.py v1.2（多源 assemble），2026-10-12；每个代码块首部的指纹注释自带该块的 range_md5，可逐块复核。
- 本篇 B 级节选原则：主讲底本取注释最全的 module_sf_mynnsfc.F90（CCPP 单柱线）；WRF 运行线（wrapper + 三伴生件）以关键段 + C 级接口索引呈现。跳过内容：OpenACC 数据搬运指令（enter/exit data copyin/copyout 长清单，属 FV3 移植管线而非物理）、debug_code 调试打印块。

二、定位

一句话：MYNN 表面层方案把「最低模式层气象状态 + 地表状态」闭合为 Monin-Obukhov 相似性意义上的湍流尺度（u、*t*、q*、z/L）与交换系数，供 LSM 计算通量、供模式输出 2 m/10 m 诊断量。

谁在调它（WRF 4.8.0 实测调用链）：



双运行线警示（源码普查实证）：

- WRF-ARW 运行线：`surface_driver.CASE(MYNNSFCSCHEME)` → `mynnsfc_driver`（phys/MYNN-SFC/WRF/）→ 按 xland、雪深 5 cm 阈值、xice 阈值把每个格点分派给 **_land / _water / _ice 三件。**这条线**不经过 ** module_sf_mynnsfc.F90。
- CCPP-FV3 单柱线：phys/MYNN-SFC/CCPP/ 下的 driver 才 `USE module_sf_mynnsfc`，调用 SFCLAY_mynn → SFCLAY1D_mynn。MPAS 线同理在 phys/MYNN-SFC/MPAS/。
- 因此：网络资料常把「MYNN 面层」直接等同于 module_sf_mynnsfc.F——在 4.8.0 源码树中这对 WRF 用户不成立。**但物理内核完全同构**：三件套里每个 mynnsfc_ 就是 SFCLAY1D_mynn 剥掉三维包装、按单点展开后的版本，且各文件自带一套几乎相同的辅助函数库。本篇以 module_sf_mynnsfc.F90 为主讲底本（注释最全、局部变量文档化最好），WRF 线讲 wrapper 分派逻辑，三件生件给 C 级注记与差异清单。
- 权威文献：Olson et al. 2021, A description of the MYNN Surface-Layer Scheme, NOAA Tech. Memo. OAR GSL-67, doi:10.25923/f6a8-bc75（三件生件文件头自引）。
- 教材锚点：ch14_pbl_family.md（边界层方案族——面层与 PBL 方案的耦合；MYNN 2.5 精读同章）、ch16_land_surface_family.md（surface driver 与 LSM 的 CHS/CQS2 接口）、ch19_lanzhou_heatwave.md（namelist 实配）、ch20_modifying_wrf.md（改造案例落点）。
- 文库内关联：PBL 侧的 MYNN-EDMF 实体在 `code_09a_module_bl_mynnedmf.md` 与 `code_09b_module_bl_mynnedmf.md`；本篇与它通过 u、*w*、pblh 等量交换。同批后续的 PBL driver 走读篇（编号 22）是本篇的直接上游。

三、WRF 运行线：常数桥接层与 wrapper

3.1 常数桥接层 module_sf_mynnsfc_common（全文）

这一层是 MYNN 物理库「框架无关化」改造的关键：所有 MYNN 家族文件（_land/_water/_ice 与 MPAS/CCPP 各 driver）从这里拿常数，而不是各自硬编码。

```
!file: phys/MYNN-SFC/WRF/module_sf_mynnsfc_common.F90    L1-L77
L00001  !=================================================================
L00002
L00003   module module_sf_mynnsfc_common
L00004
L00005  !----------------------------------------
L00006  !Define Model-specific constants/parameters.
L00007  !This module will be used at the initialization stage
L00008  !where all model-specific constants are read and saved into
L00009  !memory. This module is then used again in the MYNNSFC_* scheme. All
L00010  !MYNN-specific constants are declared globally in the main
L00011  !module (module_sf_mynnsfc) further below:
L00012  !----------------------------------------
L00013  !
L00014  ! The following 5-6 lines are the only lines in this file that are not
L00015  ! universal for all dycores... Any ideas how to universalize it?
L00016  ! For MPAS:
L00017  ! use mpas_kind_types,only: kind_phys => RKIND
L00018  ! For WRF:
L00019    use ccpp_kind_types,  only : kind_phys
L00020
L00021  !WRF CONSTANTS
L00022    use module_model_constants, only:         &
L00023      & karman, g, p1000mb,                   &
L00024      & cp, r_d, r_v, rcp, xlv, xlf, xls,     &
L00025      & svp1, svp2, svp3, p608, ep_2, rvovrd, &
L00026      & cpv, cliq, cice, svpt0
L00027  !WRF CONFIGURATIONS
L00028    use module_state_description, only:       &
L00029      & ruclsmscheme
L00030
L00031   implicit none
L00032   save
L00033
L00034  ! To be specified from dycore
L00035  ! real:: cp           != 7.*r_d/2. (J/kg/K)
L00036  ! real:: cpv          != 4.*r_v    (J/kg/K) Spec heat H2O gas
L00037  ! real:: cice         != 2106.     (J/kg/K) Spec heat H2O ice
L00038  ! real:: cliq         != 4190.     (J/kg/K) Spec heat H2O liq
L00039  ! real:: p608         != R_v/R_d-1.
L00040  ! real:: ep_2          != R_d/R_v
L00041  ! real:: grav         != accel due to gravity
L00042  ! real:: karman       != von Karman constant
L00043  ! real:: t0c          != temperature of water at freezing, 273.15 K
L00044  ! real:: rcp          != r_d/cp
L00045  ! real:: r_d          != 287.  (J/kg/K) gas const dry air
L00046  ! real:: r_v          != 461.6 (J/kg/K) gas const water
L00047  ! real:: xlf          != 0.35E6 (J/kg) fusion at 0 C
L00048  ! real:: xlv          != 2.50E6 (J/kg) vaporization at 0 C
L00049  ! real:: xls          != 2.85E6 (J/kg) sublimation
L00050  ! real:: rvovrd       != r_v/r_d != 1.608
L00051
```

```
L00052  ! Specified locally
L00053   real(kind_phys),parameter:: tref  = 300.0   ! reference temperature (K)
L00054  ! real(kind_phys),parameter:: p1000mb=100000.0
L00055  ! real(kind_phys),parameter:: svp1  = 0.6112 !(kPa)
L00056  ! real(kind_phys),parameter:: svp2  = 17.67  !(dimensionless)
L00057  ! real(kind_phys),parameter:: svp3  = 29.65  !(K)
L00058   real(kind_phys),parameter:: grav  = g
L00059   real(kind_phys),parameter:: t0c   = svpt0     != 273.15
L00060   real(kind_phys),parameter:: wfa_max = 800e6   !kg-1
L00061   real(kind_phys),parameter:: wfa_min = 1e6     !kg-1
L00062   real(kind_phys),parameter:: ifa_max = 300e6   !kg-1
L00063   real(kind_phys),parameter:: ifa_min = 0.0     !kg-1
L00064   real(kind_phys),parameter:: wfa_ht  = 2000.   !meters
L00065   real(kind_phys),parameter:: ifa_ht  = 10000.  !meters
L00066
L00067  ! To be derived in the init routine
L00068   real(kind_phys),parameter:: ep_3   = 1.-ep_2 != 0.378
L00069   real(kind_phys),parameter:: gtr    = grav/tref
L00070   real(kind_phys),parameter:: rk     = cp/r_d
L00071   real(kind_phys),parameter:: tv0    =  p608*tref
L00072   real(kind_phys),parameter:: tv1    = (1.+p608)*tref
L00073   real(kind_phys),parameter:: xlscp  = (xlv+xlf)/cp
L00074   real(kind_phys),parameter:: xlvcp  = xlv/cp
L00075   real(kind_phys),parameter:: g_inv  = 1./grav
L00076
L00077   end module module_sf_mynnsfc_common
```

讲解：①kind_phys 来自 ccpp_kind_types——精度由宿主框架决定（WRF 下实为 8 字节 double）；②全部常数通过 `use module_model_constants` 引入后**重命名**（r_d→rd、p608→ep1 风格在伴生件里是反向的：伴生件从 common 拿 ep1、ep2、ep3），保证同一份物理代码在 WRF/MPAS/FV3 下常数口径一致；③tref=300 K 等局部常数是后续物化推导（gtr、xlscp、xlvcp）的种子。改造提示：想动 Von Karman 常数或热容口径，动的是 module_model_constants（共享全模式），不要在 MYNN 文件里另立数字。

3.2 wrapper 文件头：新选项命名与文献

```
!file: phys/MYNN-SFC/WRF/module_sf_mynnsfc_driver.F90    L1-L68
L00001  !> \file module_sf_mynnsfc_driver.F90
L00002  !!  This serves as the interface between the model surface driver and the MYNN
L00003  !!  surface-layer scheme(s) in module_sfc_mynnsfc_land.F90, module_sfc_mynnsfc_ice.F90, and
L00004  !!  module_sfc_mynnsfc_water.F90.
L00005
L00006  !>\ingroup gsd_mynnsfc
L00007  !> The following references best describe the code within
L00008  !!  Olson, J. B., T. Smirnova, J. S. Kenyon, D. Turner, J. M. Brown, W. Zheng,
L00009  !!  and B. W. Green, 2021: A description of the MYNN Surface-Layer Scheme.
L00010  !!  NOAA Tech. Memo. OAR GSL-67, 26 pp., doi:10.25923/f6a8-bc75.
L00011  !!
L00012  !! Note that the old tech note (Olson et al. 2021, doi:10.25923/f6a8-bc75) is
L00013  !! still the best reference for the older version of the scheme that is used in
L00014  !! HRRRv4 and RAPv5. An updated tech note is underway... The physics in this
L00015  !! universalized and modularized version is still very similar to that in the
L00016  !! 2021 tech note, but different model-dependent configuration options have been
L00017  !! change to MYNN-specific options, such as:
L00018  !!
L00019  !!   old option "iz0tlnd" is now "sf_mynn_sfcflux_land":
L00020  !!   (default) =0: Zilitinkevich (1995); Czil now set to 0.095
L00021  !!            =1: Czil_new (modified according to Chen & Zhang 2008)
L00022  !!            =2: Modified Yang et al (2002, 2008) - generalized for all landuse
L00023  !!            =3: constant zt = z0/7.4 (original form; Garratt 1992)
L00024  !!
L00025  !!   old option "isftcflx" is now "sf_mynn_sfcflux_water":
L00026  !!            =0: z0, zt, and zq from COARE 3.0 (Fairall et al. 2003)
L00027  !!   (default) =1: z0, zt, and zq from COARE 3.5 (Edson et al 2013)
L00028  !!            =2: z0 from Davis et al (2008), zt & zq from COARE 3.5
L00029  !!            =3: z0 from Davis et al (2008), zt & zq from Garratt (1992)
L00030  !!            =4: z0 from Taylor and Yelland (2004), zt and zq from COARE 3.5
L00031  !!
L00032  !! This way, the new sf_mynn* options exist in all model frameworks (WRF/MPAS/CCPP)
L00033  !! and they carry the same meaning, wheras before, they carried a different meaning for
L00034  !! different schemes.
L00035  !=================================================================================================================
L00036   module module_sf_mynnsfc_driver
L00037  !=================================================================================================================
L00038   use module_sf_mynnsfc_common,only: kind_phys,cp,lsm_ruc => ruclsmscheme
L00039   use module_sf_mynnsfc_land,  only: mynnsfc_land
L00040   use module_sf_mynnsfc_water, only: mynnsfc_water
L00041   use module_sf_mynnsfc_ice,   only: mynnsfc_ice
L00042
L00043   implicit none
L00044
L00045   !Global configuration options, to be moved to namelist variables:
L00046   integer, parameter :: psi_opt         = 0       !0: mynn, 1: gfs
L00047   logical, parameter :: compute_flux    = .true.  !whether or not to compute the surface fluxes
L00048   logical, parameter :: compute_diag    = .true.  !whether or not to compute the 2- and 10-m diagnostics
L00049
L00050  !define constant parameters for precision-control:
L00051   real(kind_phys), parameter :: sqrt3   = 1.7320508075688773
L00052   real(kind_phys), parameter :: atan1   = 0.785398163397     !in radians
L00053   real(kind_phys), parameter :: p333    = 1.0/3.0
L00054   real(kind_phys), parameter :: zero    = 0.0
L00055   real(kind_phys), parameter :: one     = 1.0
L00056   real(kind_phys), parameter :: two     = 2.0
L00057   real(kind_phys), parameter :: four    = 4.0
L00058   real(kind_phys), parameter :: ten     = 10.0
L00059
L00060   real(kind_phys), dimension(0:1000 ),save :: psim_stab,psim_unstab, &
L00061                                               psih_stab,psih_unstab
```

```
L00062
L00063   private
L00064   public:: mynnsfc_driver, &
L00065            mynnsfc_init,   &
L00066            mynnsfclay_finalize
L00067
L00068   contains
```

讲解：文件头给出本篇最重要的**选项名映射**——旧 WRF 选项 iz0tlnd（陆面 zt/zq）与 isftcflx（水面通量）被更名为 sf_mynn_sfcflux_land 与 sf_mynn_sfcflux_water，且**编号含义变了**（见 6.1 警示框）。模块级还硬编码了 psi_opt=0（用 MYNN 混合相似性函数表而非 GFS 表）、compute_flux 与 compute_diag 恒为真。psim_stab 等四张表存放在本模块的 SAVE 数组（0:1000），**以实参形式传给三个伴生件**——表只有一份、方案文件多份，这是三件套没有各自重建表的原因。

3.3 mynnsfc_init：查表初始化

```
!file: phys/MYNN-SFC/WRF/module_sf_mynnsfc_driver.F90    L73-L87    (subroutine mynnsfc_init)
L00073   subroutine mynnsfc_init(allowed_to_read,errmsg,errflg)
L00074
L00075      logical, intent(in)           :: allowed_to_read
L00076      character(len=*), intent(out) :: errmsg
L00077      integer, intent(out)          :: errflg
L00078
L00079      !Fill the PSIM and PSIH tables. This code was leveraged from
L00080      !module_sf_sfclayrev.F (from Pedro Jimenez and Jimy Dudhia).
L00081      !This subroutine returns a blended form from Dyer and Hicks (1974)
L00082      !and Grachev et al (2000) for unstable conditions and the form
L00083      !from Cheng and Brutsaert (2005) for stable conditions.
L00084
L00085       call psi_init(psi_opt,errmsg,errflg)
L00086
L00087   end subroutine mynnsfc_init
```

讲解：模式初始化阶段（surface driver 的 init 钩子）调用一次，把 z/L ∈ [0,10]（不稳定取负）以 0.01 步长预生成 1001 点的 ψ_m、ψ_h 表。运行期查表 + 线性插值，超越 ±10 才回退到闭式函数重算——GPU/性能考量，物理不变。

3.4 mynnsfc_driver：逐点分派

先看冷启动初值与逐点状态转换（3D→ 单点标量）：

```
!file: phys/MYNN-SFC/WRF/module_sf_mynnsfc_driver.F90    L325-L474
L00325   !--- local variables and arrays:
L00326    integer:: i,j,k,vegtype_1,iter,loc_z0_type,ncalls
L00327    logical:: loc_redrag,loc_iter,loc_cycle
L00328    real(kind_phys),dimension(ims:ime,jms:jme):: wstar
L00329    real(kind_phys),dimension(ims:ime,jms:jme):: qstar
L00330
L00331   !intermediate single-point variables will be *_1
L00332    real(kind_phys) :: mavail_1,pblh_1,xland_1,tsk_1,psfcpa_1,          &
L00333                       snowh_1,dx_1,lakemask_1,wat_depth_1,xice_1
L00334    real(kind_phys) :: u_1,v_1,u_2,v_2,qv_1,p_1,t_1,rho_1,dz8w_1,       &
L00335                       dz8w_2,rstoch_1,qvmr
L00336    real(kind_phys) :: hfx_1,qfx_1,lh_1,mol_1,rmol_1,                   &
L00337                       qsfc_1,qgh_1,znt_1,zol_1,ust_1,cpm_1,chs2_1,     &
L00338                       cqs_1,cqs2_1,chs_1,ch_1,flhc_1,flqc_1,gz1oz0_1,  &
L00339                       wspd_1,br_1,psim_1,psih_1
L00340    real(kind_phys) :: u10_1,v10_1,th2_1,t2_1,q2_1
L00341    real(kind_phys) :: cd_1,cda_1,ck_1,cka_1,ustm_1
L00342    !in WRF and MPAS, these variables do not propogate outside of this driver. Use as placeholders for CCPP driver:
L00343    real(kind_phys) :: wstar_1,qstar_1,tsurf_1,cm_1,stress_1,fm_1,fh_1,  &
L00344                       fm10_1,fh2_1,hflx_1,qflx_1
L00345    real(kind_phys) :: sigmaf_1,shdmax_1,z0pert_1,ztpert_1
L00346
L00347    integer, parameter:: debug_driver = 0  !0: no output, 1: write output
L00348   !-------------------------------------------------------------------------------------------------------------
L00349    errmsg = ' '
L00350    errflg = 0
L00351    iter   = 1 !ccpp variable
L00352
L00353    !cycling flag is not in WRF, use local variable:
L00354    if (present(cycling)) then
L00355       loc_cycle = cycling
L00356    else
L00357       loc_cycle = .false.
L00358    endif
L00359
L00360    if (debug_driver > 0) then
L00361       print*,"=======in beginning of mynn sfc driver=============="
L00362       print*,"flagc_lsm=",trim('does not exist')," flag_lsm=",flag_lsm
L00363       print*,"cycling=","does not exit"," loc_cycle=",loc_cycle
L00364       print*,"itimestep=",itimestep," restart=",restart
L00365    endif
L00366
L00367    if (itimestep==1 .and. (.not.(restart) .or. .not.(loc_cycle))) then
L00368       !cold starts
L00369       do j = jts,jte
L00370          do i = its,ite
L00371             if (ust(i,j) .le. 1e-4 .or. ust(i,j) .gt. 4.0) then
L00372                u_1      = u3d(i,kts,j)
L00373                v_1      = v3d(i,kts,j)
L00374                ust(i,j) = max(0.04_kind_phys*sqrt(u_1*u_1 + v_1*v_1),0.001_kind_phys)
```

```
            endif
            pblh(i,j) = max(ten, 700._kind_phys*ust(i,j)) !this estimate is much better than zero
            qfx(i,j)  = zero
            hfx(i,j)  = zero
            mol(i,j)  = zero
         enddo
      enddo
   endif
   if (itimestep==1) then
      qstar = zero !Local variable will not be read in for restarts or cycling
   endif

   do j = jts,jte
      do i = its,ite
         qvmr     = qv3d(i,kts,j)   !mixing ratio
         !convert 3D vars to single point - input only
         dz8w_1   = dz8w(i,kts,j)
         qv_1     = max(1e-8_kind_phys,qvmr/(one+qvmr)) !convert to specific humidity
         p_1      = p3d(i,kts,j)
         t_1      = t3d(i,kts,j)
         rho_1    = rho3d(i,kts,j)
         !2nd model level winds - for diags with high-resolution grids:
         dz8w_2   = dz8w(i,kts+1,j)
         u_2      = u3d(i,kts+1,j)
         v_2      = v3d(i,kts+1,j)

         u_1      = u3d(i,kts,j)
         v_1      = v3d(i,kts,j)
         ust_1    = ust(i,j)
         stress_1 = ust_1**2
         mol_1    = mol(i,j)
         qsfc_1   = qsfc(i,j)/(one+qvmr)  !convert to specific humidity
         qgh_1    = qgh(i,j)/(one+qvmr)   !convert to specific humidity
         qstar_1  = qstar(i,j)

         !spp - input only
         if(spp_pbl==1) then
            rstoch_1 = pattern_spp_pbl(i,kts,j)
         else
            rstoch_1 = zero
         endif

         !2d input only variables:
         mavail_1 = mavail(i,j)
         pblh_1   = pblh(i,j)
         xland_1  = xland(i,j)
         xice_1   = xice(i,j)
         tsk_1    = tsk(i,j)
         psfcpa_1 = psfcpa(i,j)
         snowh_1  = snowh(i,j)
         dx_1     = dx(i,j)
         if(present(lakemask)) then
            lakemask_1  = lakemask(i,j)
         else
            lakemask_1  = zero !default to ocean, since oceans >> lakes
         endif
         if(present(water_depth) .and. present(shalwater_z0)) then
            wat_depth_1  = water_depth(i,j)
         else
            wat_depth_1  = 1000._kind_phys !default to deep ocean, since open oceans >> lakes
         endif

         !inout arguments:
         hfx_1    = hfx(i,j)
         qfx_1    = qfx(i,j)/(one+qvmr)  !specific humidity flux
         hflx_1   = hfx_1/(cp*rho_1) !ccpp
         qflx_1   = qfx_1/rho_1      !ccpp
         lh_1     = lh(i,j)
         rmol_1   = rmol(i,j)
         znt_1    = znt(i,j)
         zol_1    = zol(i,j)
         cpm_1    = cpm(i,j)
         chs2_1   = chs2(i,j)
         cqs2_1   = cqs2(i,j)
         cqs_1    = cqs(i,j)
         chs_1    = chs(i,j)
         ch_1     = ch(i,j)
         flhc_1   = flhc(i,j)
         flqc_1   = flqc(i,j)
         gz1oz0_1 = gz1oz0(i,j)
         wspd_1   = wspd(i,j)
         br_1     = br(i,j)
         psim_1   = psim(i,j)
         psih_1   = psih(i,j)

         !output arguments:
         u10_1    = zero
         v10_1    = zero
         th2_1    = zero
         t2_1     = zero
         q2_1     = zero
         wstar_1  = zero

         !optional configuration inputs:
         loc_redrag = .false.
         if(present(redrag)) loc_redrag = redrag  !CCPP option from GFS
         loc_iter   = .true.
         if(present(flag_iter)) loc_iter = flag_iter !CCPP option. In WRF or MPAS, just set = True
         loc_z0_type= 0
         if(present(sfc_z0_type)) loc_z0_type = sfc_z0_type !CCPP option from GFS
```

讲解：① 冷启动（itimestep==1 且非 restart）给 u* 一个 0.04 • |V| 量级的初值、pblh 用 700 • u* 与 10 m 取大做粗估——比零初值好得多，这是迭代方案的启动技巧；② 逐点把三维场抽成标量并做**比湿换算**（qv_1 = qv/(1+qv)），返回时再换回混合比；③lakemask 缺省按海洋（0）、wat_depth 缺

省按深海 1000 m——湖泊点必须确认上游真传了这两个量，否则湖面按海水盐度修正饱和比压；④z0pert/ztpert/vegtype 等 GFS 移植参数在 WRF 下缺省为零。

再看核心的三路分派与回写：

```
!file: phys/MYNN-SFC/WRF/module_sf_mynnsfc_driver.F90    L534-L724
L00534         if (((xland_1-1.5) .lt. zero) .and. (snowh_1 .lt. snow_thresh)) then !5 cm threshold for binary snow/no-snow
L00535         !if ((xland_1-1.5) .lt. zero) then !alternative - best for fractional landuse/snow, but unnecessarily
↪  duplicative for dominate types
L00536            if (debug_driver > 0) then
L00537               print*,"--> calling mynnsfc_land..."
L00538               ncalls=ncalls+1
L00539            endif
L00540            call mynnsfc_land( &
L00541                  !model info
L00542                  i        = i        , j        = j        , itimestep= itimestep, flag_iter = loc_iter  , &
L00543                  dx       = dx_1     , xland    = xland_1  , iter     = iter     ,                         &
L00544                  !3d input - transformed to single point
L00545                  u_1      = u_1      , v_1      = v_1      , t_1      = t_1      , qv_1      = qv_1      , &
L00546                  p_1      = p_1      , dz8w_1   = dz8w_1   , rho_1    = rho_1    , u_2       = u_2       , &
L00547                  v_2      = v_2      , dz8w_2   = dz8w_2   ,                                               &
L00548                  !GFS input
L00549                  sigmaf   = sigmaf_1  , vegtype  = vegtype_1 , shdmax   = shdmax_1 , ivegsrc   = ivegsrc   , &
↪  !optional intent(in)
L00550                  z0pert   = z0pert_1  , ztpert   = ztpert_1  , redrag   =loc_redrag,sfc_z0_type=loc_z0_type, &
↪  !optional intent(in)
L00551                  !2d variables - transformed to single point
L00552                  chs      = chs_1     , chs2     = chs2_1    , cqs2     = cqs2_1    , cqs       = cqs_1     , &
L00553                  pblh     = pblh_1    , rmol     = rmol_1    , znt      = znt_1     , psfcpa    = psfcpa_1  , &
L00554                  ust      = ust_1     , ustm     = ustm_1    , stress   = stress_1 ,                        &
L00555                  mavail   = mavail_1  , zol      = zol_1     , mol      = mol_1     , tsurf     = tsurf_1   , &
L00556                  psim     = psim_1    , psih     = psih_1    , hfx      = hfx_1     , qfx       = qfx_1     , &
L00557                  tskin    = tsk_1     , u10      = u10_1     , v10      = v10_1     , th2       = th2_1     , &
L00558                  t2       = t2_1      , q2       = q2_1      , flhc     = flhc_1    , flqc      = flqc_1    , &
L00559                  snowh    = snowh_1   , qsfc     = qsfc_1    , qgh      = qgh_1     ,                        &
L00560                  lh       = lh_1      , gz1oz0   = gz1oz0_1  , wspd     = wspd_1    , rb        = br_1      , &
L00561                  cpm      = cpm_1     , ch       = ch_1      , cm       = cm_1      , rstoch_1  = rstoch_1  , &
L00562                  wstar    = wstar_1   , qstar    = qstar_1   ,                                              &
L00563                  ck       = ck_1      , cka      = cka_1     , cd       = cd_1      , cda       = cda_1     , &
L00564                  psix     = fm_1      , psit     = fh_1      , psix10   = fm10_1    , psit2     = fh2_1     , &
L00565                  !configuration options
L00566                  spp_sfc  = spp_pbl   , isfflx   = isfflx    , sf_mynn_sfcflux_land= sf_mynn_sfcflux_land  , &
L00567                  flag_restart= restart, flag_cycle= cycling  , compute_flux        = compute_flux          , &
L00568                  psi_opt  = psi_opt   , lsm      = flag_lsm  , compute_diag        = compute_diag          , &
L00569                  lsm_ruc  = lsm_ruc   ,                                                                     &
L00570                  !stability function tables
L00571                  psim_stab= psim_stab ,psim_unstab=psim_unstab,psih_stab=psih_stab ,psih_unstab=psih_unstab, &
L00572                  !error management
L00573                  errmsg   = errmsg    , errflg   = errflg                                                   &
L00574                     )
L00575         endif
L00576
L00577         !if (((xland_1-1.5) .ge. zero) .or. ((xice_1 .gt. zero) .and. (xice_1 .lt. one))) then
L00578         if ((xland_1-1.5) .ge. zero) then !best for fractional landuse/snow
L00579            if (debug_driver > 0) then
L00580               print*,"--> calling mynnsfc_water..."
L00581               ncalls=ncalls+1
L00582            endif
L00583            call mynnsfc_water( &
L00584                  !model info
L00585                  i        = i        , j        = j        , itimestep= itimestep, flag_iter = loc_iter  , &
L00586                  dx       = dx_1     , xland    = xland_1  , iter     = iter     , lakemask  = lakemask_1, &
L00587                  wat_depth=wat_depth_1,                                                                     &
L00588                  !3d input - transformed to single point
L00589                  u_1      = u_1      , v_1      = v_1      , t_1      = t_1      , qv_1      = qv_1      , &
L00590                  p_1      = p_1      , dz8w_1   = dz8w_1   , rho_1    = rho_1    , u_2       = u_2       , &
L00591                  v_2      = v_2      , dz8w_2   = dz8w_2   ,                                               &
L00592                  !GFS input
L00593                  sigmaf   = sigmaf_1  , vegtype  = vegtype_1 , shdmax   = shdmax_1 , ivegsrc   = ivegsrc   , &
↪  !optional intent(in)
L00594                  z0pert   = z0pert_1  , ztpert   = ztpert_1  , redrag   =loc_redrag, sfc_z0_type=loc_z0_type,&
↪  !optional intent(in)
L00595                  !2d variables - transformed to single point
L00596                  chs      = chs_1     , chs2     = chs2_1    , cqs2     = cqs2_1    , cqs       = cqs_1     , &
L00597                  pblh     = pblh_1    , rmol     = rmol_1    , znt      = znt_1     , psfcpa    = psfcpa_1  , &
L00598                  ust      = ust_1     , ustm     = ustm_1    , stress   = stress_1 ,                        &
L00599                  mavail   = mavail_1  , zol      = zol_1     , mol      = mol_1     , tsurf     = tsurf_1   , &
L00600                  psim     = psim_1    , psih     = psih_1    , hfx      = hfx_1     , qfx       = qfx_1     , &
L00601                  tskin    = tsk_1     , u10      = u10_1     , v10      = v10_1     , th2       = th2_1     , &
L00602                  t2       = t2_1      , q2       = q2_1      , flhc     = flhc_1    , flqc      = flqc_1    , &
L00603                  snowh    = snowh_1   , qsfc     = qsfc_1    , qgh      = qgh_1     ,                        &
L00604                  lh       = lh_1      , gz1oz0   = gz1oz0_1  , wspd     = wspd_1    , rb        = br_1      , &
L00605                  cpm      = cpm_1     , ch       = ch_1      , cm       = cm_1      , rstoch_1  = rstoch_1  , &
L00606                  wstar    = wstar_1   , qstar    = qstar_1   ,                                              &
L00607                  ck       = ck_1      , cka      = cka_1     , cd       = cd_1      , cda       = cda_1     , &
L00608                  psix     = fm_1      , psit     = fh_1      , psix10   = fm10_1    , psit2     = fh2_1     , &
L00609                  !configuration options
L00610                  spp_sfc  = spp_pbl   , isfflx   = isfflx    ,sf_mynn_sfcflux_water= sf_mynn_sfcflux_water , &
L00611                  flag_restart= restart, flag_cycle= cycling  , compute_flux        = compute_flux          , &
L00612                  psi_opt  = psi_opt   ,                        compute_diag        = compute_diag          , &
L00613                  lsm_ruc  = lsm_ruc   , lsm      = flag_lsm  , shalwater_z0        = shalwater_z0          , &
L00614                  !stability function tables
L00615                  psim_stab= psim_stab ,psim_unstab=psim_unstab,psih_stab=psih_stab ,psih_unstab=psih_unstab, &
L00616                  !error management
L00617                  errmsg   = errmsg    , errflg   = errflg                                                   &
L00618                     )
L00619         endif
L00620
L00621         if ((((xland_1-1.5) .lt. zero) .and. (snowh_1 .ge. snow_thresh)) .or. &  !land snow/ice
L00622              (xice_1.ge.xice_threshold .and. xice_1.lt.one)) then                 !partial seaice
L00623            if (debug_driver > 0) then
L00624               print*,"--> calling mynnsfc_ice..."
L00625               ncalls=ncalls+1
L00626            endif
L00627            call mynnsfc_ice( &
L00628                  !model info
```

```
                  i       = i       , j       = j       , itimestep= itimestep, flag_iter = loc_iter  , &
                  dx      = dx_1    , xland   = xland_1 , iter     = iter     ,                       &
                  !3d input - transformed to single point
                  u_1     = u_1     , v_1     = v_1     , t_1      = t_1      , qv_1      = qv_1      , &
                  p_1     = p_1     , dz8w_1  = dz8w_1  , rho_1    = rho_1    , u_2       = u_2       , &
                  v_2     = v_2     , dz8w_2  = dz8w_2  ,                                             &
                  !2d variables - transformed to single point
                  chs     = chs_1   , chs2    = chs2_1  , cqs2     = cqs2_1   , cqs       = cqs_1     , &
                  pblh    = pblh_1  , rmol    = rmol_1  , znt      = znt_1    , psfcpa    = psfcpa_1  , &
                  ust     = ust_1   , ustm    = ustm_1  , stress   = stress_1 ,                       &
                  mavail  = mavail_1, zol     = zol_1   , mol      = mol_1    , tsurf     = tsurf_1   , &
                  psim    = psim_1  , psih    = psih_1  , hfx      = hfx_1    , qfx       = qfx_1     , &
                  tskin   = tsk_1   , u10     = u10_1   , v10      = v10_1    , th2       = th2_1     , &
                  t2      = t2_1    , q2      = q2_1    , flhc     = flhc_1   , flqc      = flqc_1    , &
                  snowh   = snowh_1 , qsfc    = qsfc_1  , qgh      = qgh_1    ,                       &
                  lh      = lh_1    , gz1oz0  = gz1oz0_1, wspd     = wspd_1   , rb        = br_1      , &
                  cpm     = cpm_1   , ch      = ch_1    , cm       = cm_1     , rstoch    = rstoch_1  , &
                  wstar   = wstar_1 , qstar   = qstar_1 ,                                             &
                  ck      = ck_1    , cka     = cka_1   , cd       = cd_1     , cda       = cda_1     , &
                  psix    = fm_1    , psit    = fh_1    , psix10   = fm10_1   , psit2     = fh2_1     , &
                  !configuration options
                  spp_sfc  = spp_pbl  , isfflx   = isfflx  ,                                          &
                  flag_restart= restart, flag_cycle= cycling , compute_flux      = compute_flux       , &
                  psi_opt  = psi_opt  , lsm     = flag_lsm , compute_diag      = compute_diag       , &
                  lsm_ruc  = lsm_ruc  ,                                                               &
                  !stability function tables
                  psim_stab= psim_stab ,psim_unstab=psim_unstab,psih_stab=psih_stab ,psih_unstab=psih_unstab, &
                  !error management
                  errmsg   = errmsg   , errflg   = errflg                                             &
                     )
      endif

      !inout arguments:
      hfx(i,j)    = hfx_1
      qfx(i,j)    = qfx_1/(one-qv_1)  !mixing ratio flux
      lh(i,j)     = lh_1/(one-qv_1)   !mixing ratio flux
      mol(i,j)    = mol_1
      rmol(i,j)   = rmol_1
      qsfc(i,j)   = qsfc_1/(one-qv_1) !mixing ratio
      qgh(i,j)    = qgh_1/(one-qv_1)  !mixing ratio
      znt(i,j)    = znt_1
      zol(i,j)    = zol_1
      ust(i,j)    = ust_1
      cpm(i,j)    = cpm_1
      chs2(i,j)   = chs2_1
      cqs2(i,j)   = cqs2_1
      cqs(i,j)    = cqs_1
      chs(i,j)    = chs_1
      ch(i,j)     = ch_1
      flhc(i,j)   = flhc_1
      flqc(i,j)   = flqc_1
      gz1oz0(i,j) = gz1oz0_1
      wspd(i,j)   = wspd_1
      br(i,j)     = br_1
      psim(i,j)   = psim_1
      psih(i,j)   = psih_1

      !output arguments:
      u10(i,j)    = u10_1
      v10(i,j)    = v10_1
      th2(i,j)    = th2_1
      t2(i,j)     = t2_1
      q2(i,j)     = q2_1/(one-qv_1)  !mixing ratio
      wstar(i,j)  = wstar_1
      qstar(i,j)  = qstar_1

      !optional output arguments, mostly for ccpp:
      if(present(hflx))hflx(i,j)= hfx_1/(cp*rho_1)
      if(present(qflx))qflx(i,j)= qfx_1/rho_1
      if(present(ck) .and. present(cka) .and. present(cd) .and. present(cda)) then
         ck(i,j)  = ck_1
         cka(i,j) = cka_1
         cd(i,j)  = cd_1
         cda(i,j) = cda_1
      endif
      if(present(fm) .and. present(fh) .and. present(fm10) .and. present(fh2)) then
         fm(i,j)  = fm_1
         fh(i,j)  = fh_1
         fm10(i,j)= fm10_1
         fh2(i,j) = fh2_1
      endif
      if(present(ustm))  ustm(i,j)  = ustm_1
      if(present(stress))stress(i,j)= stress_1
      if(present(cm))    cm(i,j)    = cm_1

      if (debug_driver == 1) then
         write(*,*)"=== end of driver, ncalls=", ncalls
         write(*,*)"br=",br_1,"zol=",zol_1," wspd=",wspd_1
         write(*,*)"hfx=",hfx_1," qfx=",qfx_1," lh=",lh_1
         write(*,*)"flqc=",flqc_1," flhc=",flhc_1," u*=",ust_1
         !write(*,*)"psim_stab=",psim_stab(1)," psim_unstab=",psim_unstab(1)
         !write(*,*)"psih_stab=",psih_stab(1)," psih_unstab=",psih_unstab(1)
         write(*,*)"================================================="
      endif
   enddo !i-loop
enddo !j-loop
```

讲解：分派判据（每点最多命中两类）：陆 = xland<1.5 且 snowh<5 cm → mynnsfc_land；水 = xland≥1.5 → mynnsfc_water；雪/冰 =（陆地雪 ≥5 cm）或（0<xice<xice_threshold 之外的部分海冰）→ mynnsfc_ice。注意三件被调时**四张 ψ 表作为实参传入**（psim_stab=psim_stab…），伴生件自身不持有表。回写段有两个易踩的坑：qfx 与 lh 从「比湿通量」换回「混合比通量」用 /(1−qv_1)——与入口的 /(1+qv) 不对称，这是因为潜热通量符号约定在伴生件内部按混合比差计算；q2 同样换算。改造提示：想在 MYNN 面层加自定义通量项，最稳的位置是伴生件内部（单点、无 ACC 包裹），回写段

只要保持换算方向不变即可。

四、CCPP 单柱线主讲：module_sf_mynnsfc

4.1 模块头与选项文档

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1-L113
!>\file module_sf_mynnsfc.F90
!! This file contains
!WRF:MODEL_LAYER:PHYSICS
!
!>\ingroup mynn_sfc
!> This module contain routines to calculate stability parameters, kinematic viscosity
!! in MYNN surface layer scheme
MODULE module_sf_mynnsfc

!-------------------------------------------------------------------
!Modifications implemented by Joseph Olson NOAA/GSL
!The following overviews the current state of this scheme::
!
!   BOTH LAND AND WATER:
!1) Calculation of stability parameter (z/L) taken from Li et al. (2010 BLM)
!   for first iteration of first time step; afterwards, exact calculation
!   using basically the same iterative technique in the module_sf_sfclayrev.F,
!   which leverages Pedro Jimenez's code, and is adapted for MYNN.
!2) Fixed isflux=0 option to turn off scalar fluxes, but keep momentum
!   fluxes for idealized studies (credit: Anna Fitch).
!3) Kinematic viscosity varies with temperature according to Andreas (1989).
!4) Uses the blended Monin-Obukhov flux-profile relationships COARE (Fairall
!   et al 2003) for the unstable regime (a blended mix of Dyer-Hicks 1974 and
!   Grachev et al (2000). Uses Cheng and Brutsaert (2005) for stable conditions.
!5) The following overviews the namelist variables that control the
!   aerodynamic roughness lengths (over water) and the thermal and moisture
!   roughness lengths (defaults are recommended):
!
!   LAND only:
!   "iz0tlnd" namelist option is used to select the following momentum options:
!   (default) =0: Zilitinkevich (1995); Czil now set to 0.095
!             =1: Czil_new (modified according to Chen & Zhang 2008)
!             =2: Modified Yang et al (2002, 2008) - generalized for all landuse
!             =3: constant zt = z0/7.4 (original form; Garratt 1992)
!             =4: GFS - taken from sfc_diff.f, for comparison/testing
!
!   WATER only:
!   "isftcflx" namelist option is used to select the following scalar options:
!   (default) =0: z0, zt, and zq from the COARE algorithm. Set COARE_OPT (below) to
!                 3.0 (Fairall et al. 2003, default)
!                 3.5 (Edson et al 2013)
!             =1: z0 from Davis et al (2008), zt & zq from COARE 3.0/3.5
!             =2: z0 from Davis et al (2008), zt & zq from Garratt (1992)
!             =3: z0 from Taylor and Yelland (2004), zt and zq from COARE 3.0/3.5
!             =4: GFS - taken from sfc_diff.f, for comparison/testing
!
!   SNOW/ICE only:
!   Andreas (2002) snow/ice parameterization for thermal and
!   moisture roughness is used over all gridpoints with snow deeper than
!   0.1 m. This algorithm calculates a z0 for snow (Andreas et al. 2005, BLM),
!   which is only used as part of the thermal and moisture roughness
!   length calculation, not to directly impact the surface winds.
!
! Misc:
!1) Added a more elaborate diagnostic for u10 & V10 for high vertical resolution
!   model configurations but for most model configurations with depth of
!   the lowest half-model level near 10 m, a neutral-log diagnostic is used.
!
!2) Option to activate stochastic parameter perturbations (SPP), which
!   perturb z0, zt, and zq, along with many other parameters in the MYNN-
!   EDMF scheme.
!
!NOTE: This code was primarily tested in combination with the RUC LSM.
!      Performance with the Noah (or other) LSM is relatively unknown.
!-------------------------------------------------------------------
!Include host model constants
      use physcons, only : cp     => con_cp,     & !=7*Rd/2
     &                     grav   => con_g,      & !=9.81
     &                     Rd     => con_rd,     & !=287.
     &                     Rv     => con_rv,     & !=461.6
!    &                      cpv    => con_cvap,   & !=4*Rv
     &                     rovcp  => con_rocp,   & !=Rd/cp
     &                     xlv    => con_hvap,   & !2.5e6
     &                     xlf    => con_hfus,   & !3.5e5
     &                     ep1    => con_fvirt,  & !Rv/Rd - 1
     &                     ep2    => con_eps       !Rd/Rv

!use kind_phys for real-types
    use machine , only : kind_phys

!-------------------------------------------------------------------
  IMPLICIT NONE
!-------------------------------------------------------------------
!Drive and/or define more constant:
  real(kind_phys), parameter :: ep3           = 1.-ep2
  real(kind_phys), parameter :: g_inv         = 1.0/grav
  real(kind_phys), parameter :: rvovrd        = Rv/Rd
  real(kind_phys), parameter :: wmin          = 0.1    ! Minimum wind speed
  real(kind_phys), parameter :: karman        = 0.4
  real(kind_phys), parameter :: SVP1          = 0.6112
  real(kind_phys), parameter :: SVP2          = 17.67
  real(kind_phys), parameter :: SVP3          = 29.65
  real(kind_phys), parameter :: SVPT0         = 273.15
  real(kind_phys), parameter :: VCONVC        = 1.25
  real(kind_phys), parameter :: onethird      = 1./3.
```

```
L00096    real(kind_phys), parameter :: sqrt3         = 1.7320508075688773
L00097    real(kind_phys), parameter :: atan1         = 0.785398163397     !in radians
L00098    real(kind_phys), parameter :: log01         = log(0.01)
L00099    real(kind_phys), parameter :: log05         = log(0.05)
L00100    real(kind_phys), parameter :: log07         = log(0.07)
L00101    real(kind_phys), parameter :: SNOWZ0        = 0.011
L00102    real(kind_phys), parameter :: COARE_OPT     = 3.0  ! 3.0 or 3.5
L00103
L00104    !For debugging purposes:
L00105    INTEGER, PARAMETER :: debug_code = 0  !0: no extra ouput
L00106                                          !1: check input
L00107                                          !2: everything - heavy I/O
L00108
L00109    REAL(kind_phys), DIMENSION(0:1000 ),SAVE :: psim_stab,psim_unstab, &
L00110                                        psih_stab,psih_unstab
L00111  !$acc declare create(psim_stab, psim_unstab, psih_stab, psih_unstab)
L00112
L00113  CONTAINS
```

讲解：① 头部注释完整记录了方案现状（Li 2010 初猜 + Jimenez 式迭代、Andreas 1989 动理黏性、COARE 混合不稳定相似性函数、Cheng–Brutsaert 2005 稳定相似性函数）与两套 namelist 选项（iz0tlnd 陆面 0–4、isftcflx 水面 0–4——**这是 CCPP 线的编号**，与 WRF 线的 sf_mynn_sfcflux_* 编号不同，见 6.1）；②SPP 随机参数扰动选项（spp_sfc）在 z0/zt/zq 三处生效；③psim_stab 等表在本模块为 SAVE 数组并有 `!$acc declare create`——GPU 侧常驻。

4.2 SFCLAY_mynn：三维包装（全文）

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L119-L511    (subroutine SFCLAY_mynn)
L00119      SUBROUTINE SFCLAY_mynn(                                 &
L00120                 U3D,V3D,T3D,QV3D,P3D,dz8w,                   & !in
L00121                 th3d,pi3d,qc3d,                              & !in
L00122                 PSFCPA,PBLH,MAVAIL,XLAND,DX,                 & !in
L00123                 ISFFLX,isftcflx,lsm,lsm_ruc,                 & !in
L00124                 compute_flux,compute_diag,                   & !in
L00125                 iz0tlnd,psi_opt,                             & !in
L00126                 sigmaf,vegtype,shdmax,ivegsrc,               & !intent(in)
L00127                 z0pert,ztpert,                               & !intent(in)
L00128                 redrag,sfc_z0_type,                          & !intent(in)
L00129                 itimestep,iter,flag_iter,                    & !in
L00130                 flag_restart,                                & !in
L00131                      wet,       dry,       icy,             & !intent(in)
L00132                 tskin_wat, tskin_lnd, tskin_ice,             & !intent(in)
L00133                 tsurf_wat, tsurf_lnd, tsurf_ice,             & !intent(in)
L00134                  qsfc_wat,  qsfc_lnd,  qsfc_ice,             & !intent(in)
L00135                 snowh_wat, snowh_lnd, snowh_ice,             & !intent(in)
L00136                   znt_wat,   znt_lnd,   znt_ice,             & !intent(inout)
L00137                   ust_wat,   ust_lnd,   ust_ice,             & !intent(inout)
L00138                    cm_wat,    cm_lnd,    cm_ice,             & !intent(inout)
L00139                    ch_wat,    ch_lnd,    ch_ice,             & !intent(inout)
L00140                    rb_wat,    rb_lnd,    rb_ice,             & !intent(inout)
L00141                stress_wat,stress_lnd,stress_ice,             & !intent(inout)
L00142                    fm_wat,    fm_lnd,    fm_ice,             & !intent(inout)
L00143                    fh_wat,    fh_lnd,    fh_ice,             & !intent(inout)
L00144                  fm10_wat,  fm10_lnd,  fm10_ice,             & !intent(inout)
L00145                   fh2_wat,   fh2_lnd,   fh2_ice,             & !intent(inout)
L00146                  HFLX_wat,  HFLX_lnd,  HFLX_ice,             &
L00147                  QFLX_wat,  QFLX_lnd,  QFLX_ice,             &
L00148                 CH,CHS,CHS2,CQS2,CPM,                        &
L00149                 ZNT,USTM,ZOL,MOL,RMOL,                       &
L00150                 PSIM,PSIH,                                   &
L00151                 HFLX,HFX,QFLX,QFX,LH,FLHC,FLQC,              &
L00152                 QGH,QSFC,                                    &
L00153                 U10,V10,TH2,T2,Q2,                           &
L00154                 GZ1OZ0,WSPD,WSTAR,                           &
L00155                 spp_sfc,pattern_spp_sfc,                     &
L00156                 ids,ide, jds,jde, kds,kde,                   &
L00157                 ims,ime, jms,jme, kms,kme,                   &
L00158                 its,ite, jts,jte, kts,kte,                   &
L00159                 errmsg, errflg                               )
L00160  !-----------------------------------------------------------------
L00161        IMPLICIT NONE
L00162  !-----------------------------------------------------------------
L00163  !-- U3D         3D u-velocity interpolated to theta points (m/s)
L00164  !-- V3D         3D v-velocity interpolated to theta points (m/s)
L00165  !-- T3D         3D temperature (K)
L00166  !-- QV3D        3D water vapor mixing ratio (Kg/Kg)
L00167  !-- P3D         3D pressure (Pa)
L00168  !-- dz8w        3D dz between full levels (m)
L00169  !-- CP          heat capacity at constant pressure for dry air (J/kg/K)
L00170  !-- grav        acceleration due to gravity (m/s^2)
L00171  !-- ROVCP       R/CP
L00172  !-- Rd          gas constant for dry air (J/kg/K)
L00173  !-- XLV         latent heat of vaporization for water (J/kg)
L00174  !-- PSFCPA      surface pressure (Pa)
L00175  !-- ZNT         roughness length (m)
L00176  !-- UST         u* in similarity theory (m/s)
L00177  !-- USTM        u* in similarity theory (m/s) w* added to WSPD. This is
L00178  !               used to couple with TKE scheme but not in MYNN.
L00179  !               (as of now, USTM = UST in this version)
L00180  !-- PBLH        PBL height from previous time (m)
L00181  !-- MAVAIL      surface moisture availability (between 0 and 1)
L00182  !-- ZOL         z/L height over Monin-Obukhov length
L00183  !-- MOL         T* (similarity theory) (K)
L00184  !-- RMOL        Reciprocal of M-O length (/m)
L00185  !-- REGIME      flag indicating PBL regime (stable, unstable, etc.)
L00186  !-- PSIM        similarity stability function for momentum
L00187  !-- PSIH        similarity stability function for heat
L00188  !-- XLAND       land mask (1 for land, 2 for water)
L00189  !-- HFX         upward heat flux at the surface (W/m^2)
```

```
L00190  !                  HFX = HFLX * rho * cp
L00191  !-- HFLX         upward temperature flux at the surface (K m s^-1)
L00192  !-- QFX          upward moisture flux at the surface (kg/m^2/s)
L00193  !                  QFX = QFLX * rho
L00194  !-- QFLX         upward moisture flux at the surface (kg kg-1 m s-1)
L00195  !-- LH           net upward latent heat flux at surface (W/m^2)
L00196  !-- TSK          surface temperature (K)
L00197  !-- FLHC         exchange coefficient for heat (W/m^2/K)
L00198  !-- FLQC         exchange coefficient for moisture (kg/m^2/s)
L00199  !-- CHS          heat/moisture exchange coefficient for LSM (m/s)
L00200  !-- QGH          lowest-level saturated mixing ratio
L00201  !-- QSFC         qv (specific humidity) at the surface
L00202  !-- QSFCMR       qv (mixing ratio) at the surface
L00203  !-- U10          diagnostic 10m u wind
L00204  !-- V10          diagnostic 10m v wind
L00205  !-- TH2          diagnostic 2m theta (K)
L00206  !-- T2           diagnostic 2m temperature (K)
L00207  !-- Q2           diagnostic 2m mixing ratio (kg/kg)
L00208  !-- SNOWH        Snow height (m)
L00209  !-- GZ1OZ0       log((z1+ZNT)/ZNT) where ZNT is roughness length
L00210  !-- WSPD         wind speed at lowest model level (m/s)
L00211  !-- BR           bulk Richardson number in surface layer
L00212  !-- ISFFLX       isfflx=1 for surface heat and moisture fluxes
L00213  !-- DX           horizontal grid size (m)
L00214  !-- SVP1         constant for saturation vapor pressure (=0.6112 kPa)
L00215  !-- SVP2         constant for saturation vapor pressure (=17.67 dimensionless)
L00216  !-- SVP3         constant for saturation vapor pressure (=29.65 K)
L00217  !-- SVPT0        constant for saturation vapor pressure (=273.15 K)
L00218  !-- EP1          constant for virtual temperature (Rv/Rd - 1) (dimensionless)
L00219  !-- EP2          constant for spec. hum. calc (Rd/Rv = 0.622) (dimensionless)
L00220  !-- EP3          constant for spec. hum. calc (1 - Rd/Rv = 0.378 ) (dimensionless)
L00221  !-- KARMAN       Von Karman constant
L00222  !-- ck           enthalpy exchange coeff at 10 meters
L00223  !-- cd           momentum exchange coeff at 10 meters
L00224  !-- cka          enthalpy exchange coeff at the lowest model level
L00225  !-- cda          momentum exchange coeff at the lowest model level
L00226  !-- isftcflx     =0: z0, zt, and zq from COARE3.0/3.5 (Fairall et al 2003/Edson et al 2013)
L00227  !   (water       =1: z0 from Davis et al (2008), zt & zq from COARE3.0/3.5
L00228  !    only)       =2: z0 from Davis et al (2008), zt & zq from Garratt (1992)
L00229  !                =3: z0 from Taylor and Yelland (2004), zt and zq from COARE 3.0/3.5
L00230  !-- iz0tlnd      =0: Zilitinkevich (1995) with Czil=0.095,
L00231  !   (land        =1: Czil_new (modified according to Chen & Zhang 2008)
L00232  !    only)       =2: Modified Yang et al (2002, 2008) - generalized for all landuse
L00233  !                =3: constant zt = z0/7.4 (Garratt 1992)
L00234  !
L00235  !-- ids          start index for i in domain
L00236  !-- ide          end index for i in domain
L00237  !-- jds          start index for j in domain
L00238  !-- jde          end index for j in domain
L00239  !-- kds          start index for k in domain
L00240  !-- kde          end index for k in domain
L00241  !-- ims          start index for i in memory
L00242  !-- ime          end index for i in memory
L00243  !-- jms          start index for j in memory
L00244  !-- jme          end index for j in memory
L00245  !-- kms          start index for k in memory
L00246  !-- kme          end index for k in memory
L00247  !-- its          start index for i in tile
L00248  !-- ite          end index for i in tile
L00249  !-- jts          start index for j in tile
L00250  !-- jte          end index for j in tile
L00251  !-- kts          start index for k in tile
L00252  !-- kte          end index for k in tile
L00253  !-- errmsg       CCPP error message
L00254  !-- errflg       CCPP error code
L00255  !=================================================================
L00256  ! SCALARS
L00257  !=================================
L00258        INTEGER,  INTENT(IN)   ::        ids,ide, jds,jde, kds,kde, &
L00259                                         ims,ime, jms,jme, kms,kme, &
L00260                                         its,ite, jts,jte, kts,kte
L00261        INTEGER,  INTENT(IN)   ::        itimestep,iter
L00262  !NAMELIST/CONFIGURATION OPTIONS:
L00263        integer, intent(in)           :: ISFFLX, LSM, LSM_RUC
L00264        INTEGER, OPTIONAL, INTENT(IN) :: ISFTCFLX, IZ0TLND
L00265        INTEGER, OPTIONAL, INTENT(IN) :: spp_sfc, psi_opt
L00266        logical, intent(in) :: compute_flux,compute_diag
L00267        integer, intent(in) :: ivegsrc
L00268        integer, intent(in) :: sfc_z0_type ! option for calculating surface roughness length over ocean
L00269        logical, intent(in) :: redrag ! reduced drag coeff. flag for high wind over sea (j.han)
L00270        logical, intent(in) :: flag_restart
L00271
L00272  !Input data
L00273        integer, dimension(ims:ime), intent(in) :: vegtype
L00274        real(kind_phys), dimension(ims:ime), intent(in) ::           &
L00275       &                     sigmaf,shdmax,z0pert,ztpert
L00276  !=================================
L00277  ! 3D VARIABLES
L00278  !=================================
L00279        REAL(kind_phys), DIMENSION( ims:ime, kms:kme )             , &
L00280                                 INTENT(IN   )   ::               dz8w, &
L00281                                                                  QV3D, &
L00282                                                                   P3D, &
L00283                                                                   T3D, &
L00284                                                                  QC3D, &
L00285                                                                U3D,V3D, &
L00286                                                               th3d,pi3d
L00287
L00288        !GJF: This array must be assumed-shape since it is conditionally-allocated
L00289        REAL(kind_phys), DIMENSION( :,: ), OPTIONAL,                 &
L00290                                 INTENT(IN) ::         pattern_spp_sfc
L00291  !=================================
L00292  ! 2D VARIABLES
L00293  !=================================
L00294        REAL(kind_phys), DIMENSION( ims:ime )                      , &
L00295                                 INTENT(IN   )          ::      MAVAIL, &
L00296                                                                  PBLH, &
L00297                                                                 XLAND, &
```

```
                                                          PSFCPA, &
                                                             DX
      REAL(kind_phys), DIMENSION( ims:ime )                       , &
                              INTENT(OUT  )          ::     U10,V10, &
                                                          TH2,T2,Q2

      REAL(kind_phys), DIMENSION( ims:ime )                       , &
                              INTENT(INOUT)          ::    HFLX,HFX, &
                                                           QFLX,QFX, &
                                                               RMOL, &
                                                              QSFC, &
                                                               QGH, &
                                                               ZNT, &
                                                               CPM, &
                                                               CHS, &
                                                                CH, &
                                                        FLHC,FLQC, &
                                                      GZ1OZ0,WSPD, &
                                                        PSIM,PSIH, &
                                                        USTM,CHS2, &
                                                        CQS2, WSTAR
      REAL(kind_phys), DIMENSION( ims:ime ),                      &
                              INTENT(INOUT)          ::          LH, &
                                                              ZOL, &
                                                              MOL

      LOGICAL, DIMENSION( ims:ime ), INTENT(IN)    ::              &
&                               wet,  dry,  icy,  flag_iter

      REAL(kind_phys), DIMENSION( ims:ime ), INTENT(IN) ::         &
     &                    tskin_wat, tskin_lnd, tskin_ice,        &
     &                    tsurf_wat, tsurf_lnd, tsurf_ice,        &
     &                    snowh_wat, snowh_lnd, snowh_ice

      REAL(kind_phys), DIMENSION( ims:ime), INTENT(INOUT) ::       &
     &                      ZNT_wat,   ZNT_lnd,   ZNT_ice,        &
     &                      UST_wat,   UST_lnd,   UST_ice,        &
     &                       cm_wat,    cm_lnd,    cm_ice,        &
     &                       ch_wat,    ch_lnd,    ch_ice,        &
     &                       rb_wat,    rb_lnd,    rb_ice,        &
     &                   stress_wat,stress_lnd,stress_ice,        &
     &                       fm_wat,    fm_lnd,    fm_ice,        &
     &                       fh_wat,    fh_lnd,    fh_ice,        &
     &                     fm10_wat,  fm10_lnd,  fm10_ice,        &
     &                      fh2_wat,   fh2_lnd,   fh2_ice,        &
     &                     HFLX_wat,  HFLX_lnd,  HFLX_ice,        &
     &                     QFLX_wat,  QFLX_lnd,  QFLX_ice,        &
     &                     qsfc_wat,  qsfc_lnd,  qsfc_ice

! CCPP error handling
      character(len=*), intent(inout) :: errmsg
      integer,          intent(inout) :: errflg

!ADDITIONAL OUTPUT
!JOE-begin
      REAL(kind_phys), DIMENSION( ims:ime ) :: qstar
!JOE-end
!==================================
! 1D LOCAL ARRAYS
!==================================
      REAL(kind_phys), DIMENSION( its:ite ) ::           U1D,V1D, & !level1 winds
                                                      U1D2,V1D2, & !level2 winds
                                                           QV1D, &
                                                             P1D, &
                                                       T1D,QC1D, &
                                                         dz8w1d, & !level 1 height
                                                         dz2w1d   !level 2 height

      REAL(kind_phys), DIMENSION( its:ite ) ::      rstoch1D

      INTEGER ::  I,J,K,itf,ktf
!-----------------------------------------------------------------

      ! Initialize error-handling
      errflg = 0
      errmsg = ''

!$acc enter data copyin( dz8w,U3D,V3D,QV3D,QC3D,P3D,T3D,       &
!$acc                    pattern_spp_sfc)

!$acc enter data copyin( UST_WAT(:), UST_LND(:),  UST_ICE(:),     &
!$acc                    MOL(:),     QFLX(:),     HFLX(:),        &
!$acc                    QSFC(:),    QSFC_WAT(:), QSFC_LND(:),    &
!$acc                    QSFC_ICE(:))

!$acc enter data create( dz8w1d(:),  dz2w1d(:),   U1D(:),         &
!$acc                    V1D(:),     U1D2(:),     V1D2(:),        &
!$acc                    QV1D(:),    QC1D(:),     P1D(:),         &
!$acc                    T1D(:),     rstoch1D(:), qstar(:))


      IF (debug_code >= 1) THEN
        write(*,*)"======= printing of constants:"
        write(*,*)"cp=",    cp," g=",     grav
        write(*,*)"Rd=",    Rd," ep1=",    ep1
        write(*,*)"xlv=",  XLV," xlf=",   XLF
        write(*,*)"ep2=", ep2
      ENDIF

      itf=ite !MIN0(ite,ide-1)
      ktf=kte !MIN0(kte,kde-1)

!$acc parallel loop present(dz8w,U3D,V3D,QV3D,QC3D,P3D,T3D,       &
!$acc                       pattern_spp_sfc,dz8w1d,dz2w1d,U1D,    &
!$acc                       V1D,U1D2,V1D2,QV1D,QC1D,P1D,T1D,      &
!$acc                       rstoch1D,qstar)
      DO i=its,ite
```

```
L00407           dz8w1d(I) = dz8w(i,kts)
L00408           dz2w1d(I) = dz8w(i,kts+1)
L00409           U1D(i) =U3D(i,kts)
L00410           V1D(i) =V3D(i,kts)
L00411           !2nd model level winds - for diags with high-res grids
L00412           U1D2(i) =U3D(i,kts+1)
L00413           V1D2(i) =V3D(i,kts+1)
L00414           QV1D(i)=QV3D(i,kts)
L00415           QC1D(i)=QC3D(i,kts)
L00416           P1D(i) =P3D(i,kts)
L00417           T1D(i) =T3D(i,kts)
L00418           if (spp_sfc==1) then
L00419              rstoch1D(i)=pattern_spp_sfc(i,kts)
L00420           else
L00421              rstoch1D(i)=0.0
L00422           endif
L00423           qstar(i)=0.0
L00424        ENDDO
L00425
L00426        IF (itimestep==1 .AND. iter==1) THEN
L00427  !$acc parallel loop present(U1D,V1D,UST_WAT,UST_LND,UST_ICE,MOL,  &
L00428  !$acc                       QFLX,HFLX,QV3D,QSFC,QSFC_WAT,         &
L00429  !$acc                       QSFC_LND,QSFC_ICE)
L00430           DO i=its,ite
L00431              IF (.not. flag_restart) THEN
L00432                 !Everything here is used before calculated
L00433                 if (ust_wat(i) .lt. 1e-4 .or. ust_wat(i) .gt. 3.0) then
L00434                    UST_WAT(i)=MAX(0.04*SQRT(U1D(i)*U1D(i) + V1D(i)*V1D(i)),0.001_kind_phys)
L00435                 endif
L00436                 if (ust_lnd(i) .lt. 1e-4 .or. ust_lnd(i) .gt. 3.0) then
L00437                    UST_LND(i)=MAX(0.04*SQRT(U1D(i)*U1D(i) + V1D(i)*V1D(i)),0.001_kind_phys)
L00438                 endif
L00439                 if (ust_ice(i) .lt. 1e-4 .or. ust_ice(i) .gt. 3.0) then
L00440                    UST_ICE(i)=MAX(0.04*SQRT(U1D(i)*U1D(i) + V1D(i)*V1D(i)),0.001_kind_phys)
L00441                 endif
L00442                 MOL(i)=0.0
L00443              ENDIF ! restart
L00444              QFLX(i)=0.
L00445              HFLX(i)=0.
L00446              if ( LSM == LSM_RUC ) then
L00447              !- qsfc_lnd and qsfc_ice are already available
L00448                 QSFC(i)=QV3D(i,kts)/(1.+QV3D(i,kts))
L00449                 QSFC_WAT(i)=QSFC(i)
L00450              else
L00451                 QSFC(i)=QV3D(i,kts)/(1.+QV3D(i,kts))
L00452                 QSFC_WAT(i)=QSFC(i)
L00453                 QSFC_LND(i)=QSFC(i)
L00454                 QSFC_ICE(i)=QSFC(i)
L00455              endif ! lsm==lsm_ruc
L00456           ENDDO
L00457        ENDIF
L00458
L00459  !$acc exit data delete( dz8w,U3D,V3D,QV3D,QC3D,P3D,T3D,         &
L00460  !$acc                   pattern_spp_sfc, QC1D)
L00461
L00462        CALL SFCLAY1D_mynn(flag_iter,                             &
L00463             J,U1D,V1D,T1D,QV1D,P1D,dz8w1d,                       &
L00464             U1D2,V1D2,dz2w1d,                                    &
L00465             PSFCPA,PBLH,MAVAIL,XLAND,DX,                         &
L00466             ISFFLX,isftcflx,iz0tlnd,psi_opt,                     &
L00467             compute_flux,compute_diag,                           &
L00468             sigmaf,vegtype,shdmax,ivegsrc,                       &  !intent(in)
L00469             z0pert,ztpert,                                       &  !intent(in)
L00470             redrag,sfc_z0_type,                                  &  !intent(in)
L00471             itimestep,iter,flag_restart,lsm,lsm_ruc,             &
L00472                   wet,         dry,         icy,                 &  !intent(in)
L00473              tskin_wat,   tskin_lnd,   tskin_ice,                &  !intent(in)
L00474              tsurf_wat,   tsurf_lnd,   tsurf_ice,                &  !intent(in)
L00475               qsfc_wat,    qsfc_lnd,    qsfc_ice,                &  !intent(in)
L00476              snowh_wat,   snowh_lnd,   snowh_ice,                &  !intent(in)
L00477                ZNT_wat,     ZNT_lnd,     ZNT_ice,                &  !intent(inout)
L00478                UST_wat,     UST_lnd,     UST_ice,                &  !intent(inout)
L00479                 cm_wat,      cm_lnd,      cm_ice,                &  !intent(inout)
L00480                 ch_wat,      ch_lnd,      ch_ice,                &  !intent(inout)
L00481                 rb_wat,      rb_lnd,      rb_ice,                &  !intent(inout)
L00482             stress_wat,  stress_lnd,  stress_ice,                &  !intent(inout)
L00483                 fm_wat,      fm_lnd,      fm_ice,                &  !intent(inout)
L00484                 fh_wat,      fh_lnd,      fh_ice,                &  !intent(inout)
L00485               fm10_wat,    fm10_lnd,    fm10_ice,                &  !intent(inout)
L00486                fh2_wat,     fh2_lnd,     fh2_ice,                &
L00487               HFLX_wat,    HFLX_lnd,    HFLX_ice,               &
L00488               QFLX_wat,    QFLX_lnd,    QFLX_ice,               &
L00489             ch,CHS,CHS2,CQS2,CPM,                                &
L00490             ZNT,USTM,ZOL,MOL,RMOL,                               &
L00491             PSIM,PSIH,                                           &
L00492             HFLX,HFX,QFLX,QFX,LH,FLHC,FLQC,                      &
L00493             QGH,QSFC,U10,V10,TH2,T2,Q2,                          &
L00494             GZ1OZ0,WSPD,wstar,qstar,                             &
L00495             spp_sfc,rstoch1D,                                    &
L00496             ids,ide, jds,jde, kds,kde,                           &
L00497             ims,ime, jms,jme, kms,kme,                           &
L00498             its,ite, jts,jte, kts,kte,                           &
L00499             errmsg, errflg                                       )
L00500
L00501  !$acc exit data copyout( UST_WAT(:), UST_LND(:),  UST_ICE(:),   &
L00502  !$acc                    MOL(:),     QFLX(:),     HFLX(:),      &
L00503  !$acc                    QSFC(:),    QSFC_WAT(:), QSFC_LND(:),  &
L00504  !$acc                    QSFC_ICE(:))
L00505
L00506  !$acc exit data delete( dz8w1d(:),   dz2w1d(:),   U1D(:),       &
L00507  !$acc                   V1D(:),      U1D2(:),     V1D2(:),      &
L00508  !$acc                   QV1D(:),     T1D(:),      P1D(:),       &
L00509  !$acc                   rstoch1D(:), qstar(:))
L00510
L00511     END SUBROUTINE SFCLAY_MYNN
```

讲解：这是 CCPP 线的入口签名（WRF 侧对应物是 3.4 的 wrapper）。三件事：① 把最低层（kts）

与第二层（kts+1）的量抽成 1D 列向量，第二层风是高分辨率 10 m 诊断用的；②itimestep==1 且 iter==1 时给三分量 u* 初值（0.04 • |V|，下限 0.001）并清零通量、初始化 QSFC——RUC LSM 时 QSFC 直接取 LSM 回传值，否则按表面饱和计算；③ 调用 SFCLAY1D_mynn。OpenACC enter/exit data 指令成对出现：GPU 数据域管理，物理逻辑全在 SFCLAY1D_mynn。

4.3 SFCLAY1D_mynn：物理核心（十段走读）

S1 局部变量全景——读后续代码前先看这张「变量地图」：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L688-L732
L00688  !---------------------------------------------------------------
L00689  ! LOCAL VARS
L00690  !---------------------------------------------------------------
L00691        REAL(kind_phys), DIMENSION(its:ite) ::                    &
L00692                    ZA, &    !Height of lowest 1/2 sigma level(m)
L00693                   ZA2, &    !Height of 2nd lowest 1/2 sigma level(m)
L00694                 THV1D, &    !Theta-v at lowest 1/2 sigma (K)
L00695                  TH1D, &    !Theta at lowest 1/2 sigma (K)
L00696                  TC1D, &    !T at lowest 1/2 sigma (Celsius)
L00697                  TV1D, &    !Tv at lowest 1/2 sigma (K)
L00698                 RHO1D, &    !density at lowest 1/2 sigma level
L00699                  QVSH, &    !qv at lowest 1/2 sigma (spec humidity)
L00700                 PSIH2, &    !M-O stability functions at z=2 m
L00701                PSIM10, &    !M-O stability functions at z=10 m
L00702                PSIH10, &    !M-O stability functions at z=10 m
L00703                 WSPDI, &
L00704                GOVRTH, &    !grav/theta
L00705                  PSFC, &    !press at surface (Pa/1000)
L00706                QSFCMR, &    !qv at surface (mixing ratio, kg/kg)
L00707                 THCON, &    !conversion from temp to theta
L00708             zratio_lnd,   zratio_ice,   zratio_wat, & !z0/zt
L00709                TSK_lnd,      TSK_ice,      TSK_wat, & !absolute temperature
L00710              THSK_lnd,     THSK_ice,     THSK_wat, & !theta
L00711             THVSK_lnd,    THVSK_ice,    THVSK_wat, & !theta-v
L00712            GZ1OZ0_lnd,   GZ1OZ0_ice,   GZ1OZ0_wat, & !LOG((ZA(I)+ZNT(i))/ZNT(i))
L00713            GZ1OZt_lnd,   GZ1OZt_ice,   GZ1OZt_wat, & !LOG((ZA(I)+ZT(i))/ZT(i))
L00714            GZ2OZ0_lnd,   GZ2OZ0_ice,   GZ2OZ0_wat, & !LOG((2.0+ZNT(I))/ZNT(I))
L00715            GZ2OZt_lnd,   GZ2OZt_ice,   GZ2OZt_wat, & !LOG((2.0+ZT(I))/ZT(I))
L00716           GZ10OZ0_lnd,  GZ10OZ0_ice,  GZ10OZ0_wat, & !LOG((10.+ZNT(I))/ZNT(I))
L00717           GZ10OZt_lnd,  GZ10OZt_ice,  GZ10OZt_wat, & !LOG((10.+ZT(I))/ZT(I))
L00718         ZNTstoch_lnd, ZNTstoch_ice, ZNTstoch_wat, &
L00719               ZT_lnd,       ZT_ice,       ZT_wat, &
L00720               ZQ_lnd,       ZQ_ice,       ZQ_wat, &
L00721             PSIQ_lnd,     PSIQ_ice,     PSIQ_wat, &
L00722            PSIQ2_lnd,    PSIQ2_ice,    PSIQ2_wat, &
L00723           QSFCMR_lnd,   QSFCMR_ice,   QSFCMR_wat
L00724
L00725        INTEGER ::  N,I,K,L,yesno
L00726
L00727        REAL(kind_phys) :: PL,E1,TABS
L00728        REAL(kind_phys) :: WSPD_lnd, WSPD_ice, WSPD_wat
L00729        REAL(kind_phys) :: DTHVDZ,DTHVM,VCONV,ZOL2,ZOL10,ZOLZA,ZOLZ0,ZOLZT
L00730        REAL(kind_phys) :: DTG,DTTHX,PSIQ,PSIQ2,PSIQ10,PSIT10
L00731        REAL(kind_phys) :: FLUXC,VSGD
L00732        REAL(kind_phys) :: restar,VISC,DQG,OLDUST,OLDTST
```

讲解：核心设计是**三下标并行变量组**：ZNT_lnd/ZNT_ice/ZNT_wat、GZ1OZ0_lnd/GZ1OZ0_ice/GZ1OZ0_wat……每个物理量按陆/冰/水三套并行存放，i 循环内用 dry/icy/wet 掩码各算各的，最后取最大（WSPD）或按掩码选一（ZNT、QSFC）。这是「部分海冰/混合下垫面」时代的代码形态——CCPP 线用这个三套并行结构，而 WRF 线把三套拆进三个独立文件。GZ1OZ0 族即 log((z+z0)/z0) 型对数——相似性理论的「中性骨架」，后面所有阻抗都是它减 ψ。

S2 表面比湿初始化：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L792-L905
L00792  !$acc parallel loop present(PSFCPA, PSFC, QSFC, T1D, flag_iter, tsurf_lnd,    &
L00793  !$acc                       QSFC_wat, QSFCMR_wat, wet, TSK_wat, tskin_wat,    &
L00794  !$acc                       QSFC_lnd, QSFCMR_lnd, dry, TSK_lnd, tskin_lnd,    &
L00795  !$acc                       QSFC_ice, QSFCMR_ice, icy, TSK_ice, tskin_ice)
L00796        DO I=its,ite
L00797
L00798           ! PSFC ( in cmb) is used later in saturation checks
L00799           PSFC(I)=PSFCPA(I)/1000.
L00800           !tgs - do computations if flag_iter(i) = .true.
L00801           if ( flag_iter(i) ) then
L00802
L00803           IF (ITIMESTEP == 1) THEN
L00804           !initialize surface specific humidity and mixing ratios for land, ice and water
L00805              IF (wet(i)) THEN
L00806                 TSK_wat(I) = tskin_wat(i)
L00807                 IF (TSK_wat(I) .LT. 273.15) THEN
L00808                    !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00809                    E1=SVP1*EXP(4648*(1./273.15 - 1./TSK_wat(I)) - &
L00810                      & 11.64*LOG(273.15/TSK_wat(I)) + 0.02265*(273.15 - TSK_wat(I)))
L00811                 ELSE
L00812                    !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00813                    E1=SVP1*EXP(SVP2*(TSK_wat(I)-SVPT0)/(TSK_wat(i)-SVP3))
L00814                 ENDIF
L00815                 QSFC_wat(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00816                 QSFCMR_wat(I)=EP2*E1/(PSFC(I)-E1)                !mixing ratio
L00817                 IF(QSFC_wat(I)>1..or.QSFC_wat(I)<0.) print *,' QSFC_wat(I)',itimestep,i,QSFC_wat(I),TSK_wat(i)
L00818              ENDIF
L00819              IF (dry(i)) THEN
L00820                TSK_lnd(I) = tskin_lnd(i)
L00821                if( lsm == lsm_ruc) then
```

```
L00822                 QSFCMR_lnd(I)=QSFC_lnd(I)/(1.-QSFC_lnd(I))       !mixing ratio
L00823               else
L00824                TABS = 0.5*(TSK_lnd(I) + T1D(I))
L00825                IF (TABS .LT. 273.15) THEN
L00826                   !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00827                   E1=SVP1*EXP(4648*(1./273.15 - 1./TABS) - &
L00828                     & 11.64*LOG(273.15/TABS) + 0.02265*(273.15 - TABS))
L00829                ELSE
L00830                   !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00831                   E1=SVP1*EXP(SVP2*(TABS-SVPT0)/(TABS-SVP3))
L00832                ENDIF
L00833                  QSFC_lnd(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00834                  QSFC_lnd(I)=0.5*(QSFC_lnd(I) + QSFC(I))
L00835                  QSFCMR_lnd(I)=QSFC_lnd(I)/(1.-QSFC_lnd(I))       !mixing ratio
L00836               endif ! lsm
L00837               IF(QSFC_lnd(I)>1..or.QSFC_lnd(I)<0.) print *,' QSFC_lnd(I)',itimestep,i,QSFC_lnd(I),Tskin_lnd(i),
↪  tsurf_lnd(i),qsfc(i)
L00838             ENDIF
L00839             IF (icy(i)) THEN
L00840               TSK_ice(I) = tskin_ice(i)
L00841               if( lsm == lsm_ruc) then
L00842                 QSFCMR_ice(I)=QSFC_ice(I)/(1.-QSFC_ice(I))        !mixing ratio
L00843               else
L00844                IF (TSK_ice(I) .LT. 273.15) THEN
L00845                   !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00846                   E1=SVP1*EXP(4648*(1./273.15 - 1./TSK_ice(I)) - &
L00847                     & 11.64*LOG(273.15/TSK_ice(I)) + 0.02265*(273.15 - TSK_ice(I)))
L00848                ELSE
L00849                   !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00850                   E1=SVP1*EXP(SVP2*(TSK_ice(I)-SVPT0)/(TSK_ice(i)-SVP3))
L00851                ENDIF
L00852                  QSFC_ice(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00853                  QSFCMR_ice(I)=EP2*E1/(PSFC(I)-E1)                !mixing ratio
L00854               endif ! lsm
L00855               IF(QSFC_ice(I)>1..or.QSFC_ice(I)<0.) print *,' QSFC_ice(I)',itimestep,i,QSFC_ice(I),TSK_ice(i)
L00856             ENDIF
L00857
L00858           ELSE
L00859
L00860              ! Use what comes out of the NST, LSM, SICE after check
L00861              IF (wet(i)) then
L00862                 TSK_wat(I) = tskin_wat(i)
L00863                 IF (TSK_wat(I) .LT. 273.15) THEN
L00864                    !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00865                    E1=SVP1*EXP(4648*(1./273.15 - 1./TSK_wat(I)) - &
L00866                      & 11.64*LOG(273.15/TSK_wat(I)) + 0.02265*(273.15 - TSK_wat(I)))
L00867                 ELSE
L00868                    !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00869                    E1=SVP1*EXP(SVP2*(TSK_wat(I)-SVPT0)/(TSK_wat(i)-SVP3))
L00870                 ENDIF
L00871                 QSFC_wat(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00872              ENDIF
L00873              IF (dry(i).and.(QSFC_lnd(I)>1..or.QSFC_lnd(I)<0.)) then
L00874                 !print *,'bad QSFC_lnd(I)',itimestep,iter,i,QSFC_lnd(I),TSKin_lnd(I)
L00875                 TABS = 0.5*(TSKin_lnd(I) + T1D(I))
L00876                 IF (TABS .LT. 273.15) THEN
L00877                    !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00878                    E1=SVP1*EXP(4648*(1./273.15 - 1./TABS) - &
L00879                      & 11.64*LOG(273.15/TABS) + 0.02265*(273.15 - TABS))
L00880                 ELSE
L00881                    !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00882                    E1=SVP1*EXP(SVP2*(TABS-SVPT0)/(TABS-SVP3))
L00883                 ENDIF
L00884                   QSFC_lnd(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00885                   QSFC_lnd(I)=0.5*(QSFC_lnd(I) + QSFC(I))
L00886              ENDIF
L00887              IF (icy(i).and.(QSFC_ice(I)>1..or.QSFC_ice(I)<0.)) then
L00888                 !print *,'bad QSFC_ice(I)',itimestep,iter,i,QSFC_ice(I),TSKin_ice(I)
L00889                 IF (TSKin_ice(I) .LT. 273.15) THEN
L00890                    !SATURATION VAPOR PRESSURE WRT ICE (SVP1=.6112; 10*mb)
L00891                    E1=SVP1*EXP(4648*(1./273.15 - 1./TSKin_ice(I)) - &
L00892                      & 11.64*LOG(273.15/TSKin_ice(I)) + 0.02265*(273.15 - TSKin_ice(I)))
L00893                 ELSE
L00894                    !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L00895                    E1=SVP1*EXP(SVP2*(TSKin_ice(I)-SVPT0)/(TSKin_ice(i)-SVP3))
L00896                 ENDIF
L00897                   QSFC_ice(I)=EP2*E1/(PSFC(I)-ep3*E1)             !specific humidity
L00898              ENDIF
L00899
L00900              IF (wet(i)) QSFCMR_wat(I)=QSFC_wat(I)/(1.-QSFC_wat(I))
L00901              IF (dry(i)) QSFCMR_lnd(I)=QSFC_lnd(I)/(1.-QSFC_lnd(I))
L00902              IF (icy(i)) QSFCMR_ice(I)=QSFC_ice(I)/(1.-QSFC_ice(I))
L00903
L00904           ENDIF
L00905         endif ! flag_iter
```

讲解：饱和比压双公式——T<273.15 K 用冰面（Magnus 型，系数 4648/11.64/0.02265），否则用 Bolton 1980 水面（SVP1/SVP2/SVP3=0.6112 kPa/17.67/29.65 K）。陆地点在非 RUC LSM 时取 0.5×(饱和值+模式值) 折中——因为陆面真实蒸发受土壤湿度限制，这个折中是给后续 MAVAIL 机制的初值垫底。itimestep>1 走 else 分支：只对越界（QSFC>1 或 <0）的点做修复，正常点信任上一时刻值。

S3 皮肤温度、位温与半层高：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L937-L1005
L00937
L00938  !$acc parallel loop present(PSFC, PSFCPA, QVSH, QV1D, THCON, flag_iter,        &
L00939  !$acc       dry, tskin_lnd, TSK_lnd, tsurf_lnd, THSK_lnd, THVSK_lnd, qsfc_lnd, &
L00940  !$acc       icy, tskin_ice, TSK_ice, tsurf_ice, THSK_ice, THVSK_ice, qsfc_ice, &
L00941  !$acc       wet, tskin_wat, TSK_wat, tsurf_wat, THSK_wat, THVSK_wat, qsfc_wat)
L00942        DO I=its,ite
L00943           ! PSFC ( in cmb) is used later in saturation checks
L00944           PSFC(I)=PSFCPA(I)/1000.
```

```
L00945           QVSH(I)=QV1D(I)/(1.+QV1D(I))        !CONVERT TO SPEC HUM (kg/kg)
L00946           THCON(I)=(100000./PSFCPA(I))**ROVCP
L00947          if( flag_iter(i) ) then
L00948           ! DEFINE SKIN TEMPERATURES FOR LAND/WATER/ICE
L00949           if(dry(i)) then
L00950             TSK_lnd(I) = tskin_lnd(i)
L00951             !TSK_lnd(I) = 0.5 * (tsurf_lnd(i)+tskin_lnd(i))
L00952             ! CONVERT SKIN TEMPERATURES TO POTENTIAL TEMPERATURE:
L00953             THSK_lnd(I) = TSK_lnd(I)*THCON(I)   !(K)
L00954             THVSK_lnd(I) = THSK_lnd(I)*(1.+EP1*qsfc_lnd(I))
L00955             if(THVSK_lnd(I) < 160. .or. THVSK_lnd(I) > 390.) &
L00956             print *,'THVSK_lnd(I)',itimestep,i,THVSK_lnd(I),THSK_lnd(i),tsurf_lnd(i),tskin_lnd(i),qsfc_lnd(i)
L00957           endif
L00958           if(icy(i)) then
L00959             TSK_ice(I) = tskin_ice(i)
L00960             !TSK_ice(I) = 0.5 * (tsurf_ice(i)+tskin_ice(i))
L00961             ! CONVERT SKIN TEMPERATURES TO POTENTIAL TEMPERATURE:
L00962             THSK_ice(I) = TSK_ice(I)*THCON(I)   !(K)
L00963             THVSK_ice(I) = THSK_ice(I)*(1.+EP1*qsfc_ice(I))   !(K)
L00964             if(THVSK_ice(I) < 160. .or. THVSK_ice(I) > 390.) &
L00965             print *,'THVSK_ice(I)',itimestep,i,THVSK_ice(I),THSK_ice(i),tsurf_ice(i),tskin_ice(i),qsfc_ice(i)
L00966           endif
L00967           if(wet(i)) then
L00968             TSK_wat(I) = tskin_wat(i)
L00969             !TSK_wat(I) = 0.5 * (tsurf_wat(i)+tskin_wat(i))
L00970             ! CONVERT SKIN TEMPERATURES TO POTENTIAL TEMPERATURE:
L00971             THSK_wat(I) = TSK_wat(I)*THCON(I)   !(K)
L00972             THVSK_wat(I) = THSK_wat(I)*(1.+EP1*QVSH(I))   !(K)
L00973             if(THVSK_wat(I) < 160. .or. THVSK_wat(I) > 390.) &
L00974             print *,'THVSK_wat(I)',i,THVSK_wat(I),THSK_wat(i),tsurf_wat(i),tskin_wat(i),qsfc_wat(i)
L00975           endif
L00976          endif ! flag_iter
L00977         ENDDO
L00978
L00979  !$acc parallel loop present(TH1D, T1D, P1D, TC1D)
L00980         DO I=its,ite
L00981           ! CONVERT LOWEST LAYER TEMPERATURE TO POTENTIAL TEMPERATURE:
L00982           TH1D(I)=T1D(I)*(100000./P1D(I))**ROVCP  !(Theta, K)
L00983           TC1D(I)=T1D(I)-273.15                   !(T, Celsius)
L00984         ENDDO
L00985
L00986  !$acc parallel loop present(THV1D, TH1D, QVSH, TV1D, T1D)
L00987         DO I=its,ite
L00988           ! CONVERT TO VIRTUAL TEMPERATURE
L00989           THV1D(I)=TH1D(I)*(1.+EP1*QVSH(I))             !(K)
L00990           TV1D(I)=T1D(I)*(1.+EP1*QVSH(I))               !(K)
L00991         ENDDO
L00992
L00993  !$acc parallel loop present(RHO1D, P1D, TV1D, TH1D, ZA, ZA2, dz2w1d, dz8w1d, GOVRTH)
L00994         DO I=its,ite
L00995           RHO1D(I)=P1D(I)/(Rd*TV1D(I))     !now using value calculated in sfc driver
L00996           ZA(I)=0.5*dz8w1d(I)              !height of first half-sigma level
L00997           ZA2(I)=dz8w1d(I) + 0.5*dz2w1d(I) !height of 2nd half-sigma level
L00998           GOVRTH(I)=grav/TH1D(I)
L00999         ENDDO
L01000
L01001         !tgs - should QFX and HFX be separate for land, ice and water?
L01002  !$acc parallel loop present(QFX, QFLX, RHO1D, HFX, HFLX)
L01003         DO I=its,ite
L01004           QFX(i)=QFLX(i)*RHO1D(I)
L01005           HFX(i)=HFLX(i)*RHO1D(I)*cp
```

讲解：①THSK/THVSK——地表虚位温，是 rb 与 z/L 的驱动温差一端；②ZA=0.5 • dz8w(1)（第一半层高）、ZA2=dz8w(1)+0.5 • dz2w(1)（第二半层高）——10 m 诊断的分辨率分支要用；③RHO1D 由最低层气压与虚温得，注释强调「现在用 sfc driver 算的值」；④QFX=QFLX • ρ、HFX=HFLX • ρ • cp——动理通量（m/s、K • m/s）与能量通量（W/m²）在此互相换算，后续每段用哪套要分清。

S4 最低层饱和混合比 QGH 与湿静力容重 CPM：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1025-L1041
L01025
L01026  !$acc parallel loop present(T1D,P1D,QGH,QV1D,CPM)
L01027         DO I=its,ite
L01028           ! QGH CHANGED TO USE LOWEST-LEVEL AIR TEMP
L01029           ! Q2SAT = QGH IN LSM
L01030           IF (T1D(I) .LT. 273.15) THEN
L01031              !SATURATION VAPOR PRESSURE WRT ICE
L01032              E1=SVP1*EXP(4648.*(1./273.15 - 1./T1D(I)) - &
L01033              &  11.64*LOG(273.15/T1D(I)) + 0.02265*(273.15 - T1D(I)))
L01034           ELSE
L01035              !SATURATION VAPOR PRESSURE WRT WATER (Bolton 1980)
L01036              E1=SVP1*EXP(SVP2*(T1D(I)-SVPT0)/(T1D(I)-SVP3))
L01037           ENDIF
L01038           PL=P1D(I)/1000.
L01039           !QGH(I)=EP2*E1/(PL-ep3*E1)    !specific humidity
L01040           QGH(I)=EP2*E1/(PL-E1)          !mixing ratio
L01041           CPM(I)=CP*(1.+0.84*QV1D(I))
```

讲解：QGH 是最低层气温（非表面温度）对应的饱和混合比，供 LSM 的 β 机制（蒸发阻抗）用；CPM=cp(1+0.84q) 是湿空气比热——感热通量公式里 ρ • cp 实为 ρ • CPM。

S5 风速、对流速度尺度与总体 Richardson 数：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1066-L1205
L01066
L01067  !$acc parallel loop present(flag_iter,U1D,V1D,WSPD,wet,dry,icy,    &
L01068  !$acc                    THV1D,THVSK_wat,THVSK_lnd,THVSK_ice,   &
L01069  !$acc                    hfx,RHO1D,qfx,WSTAR,pblh,dx,GOVRTH,ZA, &
```

```
!$acc                   TSK_wat,TSK_lnd,TSK_ice,              &
!$acc                   rb_wat,rb_lnd,rb_ice)
      DO I=its,ite
        if( flag_iter(i) ) then
         ! DH* 20200401 - note. A weird bug in Intel 18 on hera prevents using the
         ! normal -O2 optimization in Release mode for this file. Not reproducible
         ! by every user, the bug manifests itself in the resulting wind speed WSPD(I)
         ! being -99.0 despite the assignments in lines 932 and 933. *DH
         WSPD(I)=SQRT(U1D(I)*U1D(I)+V1D(I)*V1D(I))
         WSPD_wat = -99.
         WSPD_ice = -99.
         WSPD_lnd = -99.

         IF (wet(i)) THEN
            DTHVDZ=(THV1D(I)-THVSK_wat(I))
            !-------------------------------------------------------
            ! Calculate the convective velocity scale (WSTAR) and
            ! subgrid-scale velocity (VSGD) following Beljaars (1995, QJRMS)
            ! and Mahrt and Sun (1995, MWR), respectively
            !-------------------------------------------------------
            !tgs - the line below could be used when hflx_wat,qflx_wat are moved from
            !      Interstitial to Sfcprop
            !fluxc = max(hflx_wat(i) + ep1*THVSK_wat(I)*qflx_wat(i),0.)
            fluxc = max(hfx(i)/RHO1D(i)/cp                        &
            &    + ep1*THVSK_wat(I)*qfx(i)/RHO1D(i),0._kind_phys)
            !WSTAR(I) = vconvc*(grav/TSK(i)*pblh(i)*fluxc)**onethird
            WSTAR(I) = vconvc*(grav/TSK_wat(i)*pblh(i)*fluxc)**onethird
            !-------------------------------------------------------
            ! Mahrt and Sun low-res correction - modified for water points (halved)
            ! (for 13 km ~ 0.18 m/s; for 3 km == 0 m/s)
            !-------------------------------------------------------
            VSGD = MIN( 0.25 * (max(dx(i)/5000.-1.,0._kind_phys))**onethird , 0.5_kind_phys)
            WSPD_wat=SQRT(WSPD(I)*WSPD(I)+WSTAR(I)*WSTAR(I)+vsgd*vsgd)
            WSPD_wat=MAX(WSPD_wat,wmin)
            !-------------------------------------------------------
            ! CALCULATE THE BULK RICHARDSON NUMBER OF SURFACE LAYER,
            ! ACCORDING TO AKB(1976), EQ(12).
            !-------------------------------------------------------
            rb_wat(I)=GOVRTH(I)*ZA(I)*DTHVDZ/(WSPD_wat*WSPD_wat)
            rb_wat(I)=MAX(rb_wat(I),-2.0_kind_phys)
            rb_wat(I)=MIN(rb_wat(I), 2.0_kind_phys)
         ENDIF ! end water point

         IF (dry(i)) THEN
            DTHVDZ=(THV1D(I)-THVSK_lnd(I))
            !-------------------------------------------------------
            ! Calculate the convective velocity scale (WSTAR) and
            ! subgrid-scale velocity (VSGD) following Beljaars (1995, QJRMS)
            ! and Mahrt and Sun (1995, MWR), respectively
            !-------------------------------------------------------
            !tgs - the line below could be used when hflx_lnd,qflx_wat are moved from
            !      Interstitial to Sfcprop
            !fluxc = max(hflx_lnd(i) + ep1*THVSK_lnd(I)*qflx_lnd(i),0.)
            fluxc = max(hfx(i)/RHO1D(i)/cp                        &
            &    + ep1*THVSK_lnd(I)*qfx(i)/RHO1D(i),0._kind_phys)
            ! WSTAR(I) = vconvc*(g/TSK(i)*pblh(i)*fluxc)**onethird
            ! increase height scale, assuming that the non-local transoport
            ! from the mass-flux (plume) mixing exceedsd the PBLH.
            WSTAR(I) = vconvc*(grav/TSK_lnd(i)*MIN(1.5*pblh(i),4000._kind_phys)*fluxc)**onethird
            !-------------------------------------------------------
            ! Mahrt and Sun low-res correction
            ! (for 13 km ~ 0.37 m/s; for 3 km == 0 m/s)
            !-------------------------------------------------------
            VSGD = MIN( 0.32 * (max(dx(i)/5000.-1.,0._kind_phys))**onethird , 0.5_kind_phys)
            WSPD_lnd=SQRT(WSPD(I)*WSPD(I)+WSTAR(I)*WSTAR(I)+vsgd*vsgd)
            WSPD_lnd=MAX(WSPD_lnd,wmin)
            !-------------------------------------------------------
            ! CALCULATE THE BULK RICHARDSON NUMBER OF SURFACE LAYER,
            ! ACCORDING TO AKB(1976), EQ(12).
            !-------------------------------------------------------
            rb_lnd(I)=GOVRTH(I)*ZA(I)*DTHVDZ/(WSPD_lnd*WSPD_lnd)
            !From Tilden Meyers:
            !IF (rb_lnd(I) .GE 0.0) THEN
            !   ust_lnd(i)=WSPD_lnd*0.1/(1.0 + 10.0*rb_lnd(I))
            !ELSE
            !   ust_lnd(i)=WSPD_lnd*0.1*(1.0 - 10.0*rb_lnd(I))**onethird
            !ENDIF
            rb_lnd(I)=MAX(rb_lnd(I),-2.0_kind_phys)
            rb_lnd(I)=MIN(rb_lnd(I), 2.0_kind_phys)
         ENDIF ! end land point

         IF (icy(i)) THEN
            DTHVDZ=(THV1D(I)-THVSK_ice(I))
            !-------------------------------------------------------
            ! Calculate the convective velocity scale (WSTAR) and
            ! subgrid-scale velocity (VSGD) following Beljaars (1995, QJRMS)
            ! and Mahrt and Sun (1995, MWR), respectively
            !-------------------------------------------------------
            !tgs - the line below could be used when hflx_ice,qflx_ice are moved from
            !      Interstitial to Sfcprop
            !fluxc = max(hflx_ice(i) + ep1*THVSK_ice(I)*qflx_ice(i)/RHO1D(i),0.)
            fluxc = max(hfx(i)/RHO1D(i)/cp                        &
            &    + ep1*THVSK_ice(I)*qfx(i)/RHO1D(i),0._kind_phys)
            ! WSTAR(I) = vconvc*(g/TSK(i)*pblh(i)*fluxc)**onethird
            ! increase height scale, assuming that the non-local transport
            ! from the mass-flux (plume) mixing exceedsd the PBLH.
            WSTAR(I) = vconvc*(grav/TSK_ice(i)*MIN(1.5*pblh(i),4000._kind_phys)*fluxc)**onethird
            !-------------------------------------------------------
            ! Mahrt and Sun low-res correction
            ! (for 13 km ~ 0.37 m/s; for 3 km == 0 m/s)
            !-------------------------------------------------------
            VSGD = MIN( 0.32 * (max(dx(i)/5000.-1.,0._kind_phys))**onethird , 0.5_kind_phys)
            WSPD_ice=SQRT(WSPD(I)*WSPD(I)+WSTAR(I)*WSTAR(I)+vsgd*vsgd)
            WSPD_ice=MAX(WSPD_ice,wmin)
            !-------------------------------------------------------
            ! CALCULATE THE BULK RICHARDSON NUMBER OF SURFACE LAYER,
            ! ACCORDING TO AKB(1976), EQ(12).
            !-------------------------------------------------------
```

```
L01178             rb_ice(I)=GOVRTH(I)*ZA(I)*DTHVDZ/(WSPD_ice*WSPD_ice)
L01179             rb_ice(I)=MAX(rb_ice(I),-2.0_kind_phys)
L01180             rb_ice(I)=MIN(rb_ice(I), 2.0_kind_phys)
L01181           ENDIF ! end ice point
L01182
L01183           !NOW CONDENSE THE POSSIBLE WSPD VALUES BY TAKING THE MAXIMUM
L01184           WSPD(I) = MAX(WSPD_ice,WSPD_wat)
L01185           WSPD(I) = MAX(WSPD_lnd,WSPD(I))
L01186
L01187           IF (debug_code == 2) THEN
L01188              write(*,*)"===== After rb calc in mynn sfc layer:"
L01189              write(*,*)"ITIMESTEP=",ITIMESTEP
L01190              write(*,*)"WSPD=", WSPD(I)," WSTAR=", WSTAR(I)," vsgd=",vsgd
L01191              IF (icy(i))write(*,*)"rb_ice=", rb_ice(I)," DTHVDZ=",DTHVDZ
L01192              IF (wet(i))write(*,*)"rb_wat=", rb_wat(I)," DTHVDZ=",DTHVDZ
L01193              IF (dry(i))write(*,*)"rb_lnd=", rb_lnd(I)," DTHVDZ=",DTHVDZ
L01194           ENDIF
L01195
L01196           ! IF PREVIOUSLY UNSTABLE, DO NOT LET INTO REGIMES 1 AND 2 (STABLE)
L01197           !if (itimestep .GT. 1) THEN
L01198           !    IF(MOL(I).LT.0.)BR(I)=MIN(BR(I),0.0)
L01199           !ENDIF
L01200
L01201         endif ! flag_iter
L01202       ENDDO
L01203
L01204   1006   format(A,F7.3,A,f9.4,A,f9.5,A,f9.4)
L01205   1007   format(A,F2.0,A,f6.2,A,f7.3,A,f7.2)
```

讲解：三下标各算一遍，结构相同、系数不同：①fluxc=max(HFX/ρ/cp + ep1 • THVSK • QFX/ρ, 0)——感热 +潜热的浮力贡献合成；②W=*VCONVC • (g/TSK • h_eff • fluxc)^(1/3)*，*Beljaars 1995* 对流速度尺度；*h_eff* 水面用 *PBLH*，陆面/冰面用 *min(1.5 • PBLH, 4000 m)*——「质量流羽流输运可超 *PBL* 顶」的高度尺度放大；③*VSGD* 是 *Mahrt–Sun* 低分辨率修正，*(dx/5000−1)^(1/3)* 的形式使 *dx≤5 km* 时恒为零——*3 km* 教学域下 *W* 修正不生效，13 km 下约 0.2–0.4 m/s；WSPD=√(V²+W*²+VSGD²) 与 wmin=0.1 m/s 下限；④rb=GOVRTH • ZA • Δθv/WSPD²（AKB 1976 式 12），钳位 ±2；⑤ 水面 VSGD 系数 0.25、陆/冰 0.32。最后三路 WSPD 取最大者收纳。

S6 粗糙度 z0/zt/zq 分派——本方案全部「选项」的汇集地：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1213-L1495
L01213  !$acc parallel loop present(flag_iter, PSFCPA, dz8w1d, pblh, &
L01214  !$acc                       device_errmsg, device_errflg, &
L01215  !$acc                       device_special_errmsg, device_special_errflg, &
L01216  !$acc                       wet, dry, icy, &
L01217  !$acc                       ZT_wat, ZT_lnd, ZT_ice, &
L01218  !$acc                       ZNT_wat, ZNT_lnd, ZNT_ice, &
L01219  !$acc                       ZNTstoch_wat, ZNTstoch_lnd, ZNTstoch_ice, &
L01220  !$acc                       UST_wat, UST_lnd, UST_ice, &
L01221  !$acc                       ZQ_wat, ZQ_lnd, ZQ_ice, &
L01222  !$acc                       snowh_wat, snowh_lnd, snowh_ice, &
L01223  !$acc                       THVSK_wat, THVSK_lnd, THVSK_ice, &
L01224  !$acc                       tskin_wat, tskin_lnd, tskin_ice, &
L01225  !$acc                       tsurf_wat, tsurf_lnd, tsurf_ice, &
L01226  !$acc                       qsfc_wat, qsfc_lnd, qsfc_ice, &
L01227  !$acc                       GZ1OZ0_wat, GZ1OZt_wat, GZ2OZ0_wat, GZ2OZt_wat, GZ10OZ0_wat, GZ10OZt_wat, &
L01228  !$acc                       GZ1OZ0_lnd, GZ1OZt_lnd, GZ2OZ0_lnd, GZ2OZt_lnd, GZ10OZ0_lnd, GZ10OZt_lnd, &
L01229  !$acc                       GZ1OZ0_ice, GZ1OZt_ice, GZ2OZ0_ice, GZ2OZt_ice, GZ10OZ0_ice, GZ10OZt_ice, &
L01230  !$acc                       zratio_wat, zratio_lnd, zratio_ice, &
L01231  !$acc                       stress_wat, stress_lnd, stress_ice, &
L01232  !$acc                       rb_wat, rb_lnd, rb_ice, &
L01233  !$acc                       qflx, qflx_lnd, &
L01234  !$acc                       hflx, hflx_lnd, &
L01235  !$acc                       psim, psih, psim10, psih10, psih2, &
L01236  !$acc                       psix_wat, psix10_wat, psit_wat, psit2_wat, psiq_wat, psiq2_wat, &
L01237  !$acc                       psix_lnd, psix10_lnd, psit_lnd, psit2_lnd, psiq_lnd, psiq2_lnd, &
L01238  !$acc                       psix_ice, psix10_ice, psit_ice, psit2_ice, psiq_ice, psiq2_ice, &
L01239  !$acc                       WSPD, WSPDI, U1D, V1D, TC1D, THV1D, rstoch1D, USTM, ZA, ZOL, QVSH, &
L01240  !$acc                       shdmax, vegtype, z0pert, ztpert, mol, rmol, wstar, qstar, sigmaf)
L01241
L01242   DO I=its,ite
L01243     if( flag_iter(i) ) then
L01244
L01245      !COMPUTE KINEMATIC VISCOSITY (m2/s) Andreas (1989) CRREL Rep. 89-11
L01246      !valid between -173 and 277 degrees C.
L01247      VISC=1.326e-5*(1. + 6.542e-3*TC1D(I) + 8.301e-6*TC1D(I)*TC1D(I) &
L01248                        - 4.84e-9*TC1D(I)*TC1D(I)*TC1D(I))
L01249
L01250      IF (wet(i)) THEN
L01251         !---------------------------------------
L01252         ! WATER
L01253         !---------------------------------------
L01254         if (sfc_z0_type >= 0) then ! Avoid calculation is using wave model
L01255            ! CALCULATE z0 (znt)
L01256            !---------------------------------------
L01257
L01258            IF (debug_code == 2) THEN
L01259              write(*,*)"=============Input to ZNT over water:"
L01260              write(*,*)"u*:",UST_wat(i)," wspd=",WSPD(i)," visc=",visc," za=",ZA(I)
L01261            ENDIF
L01262
L01263            IF ( PRESENT(ISFTCFLX) ) THEN
L01264               IF ( ISFTCFLX .EQ. 0 ) THEN
L01265                  IF (COARE_OPT .EQ. 3.0) THEN
L01266                     !COARE 3.0 (MISLEADING SUBROUTINE NAME)
L01267                     CALL charnock_1955(ZNT_wat(i),UST_wat(i),WSPD(i),visc,ZA(I))
L01268                  ELSE
L01269                     !COARE 3.5
L01270                     CALL edson_etal_2013(ZNT_wat(i),UST_wat(i),WSPD(i),visc,ZA(I))
L01271                  ENDIF
```

```
L01272               ELSEIF ( ISFTCFLX .EQ. 1 .OR. ISFTCFLX .EQ. 2 ) THEN
L01273                  CALL davis_etal_2008(ZNT_wat(i),UST_wat(i))
L01274               ELSEIF ( ISFTCFLX .EQ. 3 ) THEN
L01275                  CALL Taylor_Yelland_2001(ZNT_wat(i),UST_wat(i),WSPD(i))
L01276               ELSEIF ( ISFTCFLX .EQ. 4 ) THEN
L01277                  !GFS surface layer scheme
L01278                  CALL GFS_z0_wat(ZNT_wat(i),UST_wat(i),WSPD(i),ZA(I),sfc_z0_type,redrag)
L01279               ENDIF
L01280            ELSE
L01281               !DEFAULT TO COARE 3.0/3.5
L01282               IF (COARE_OPT .EQ. 3.0) THEN
L01283                  !COARE 3.0
L01284                  CALL charnock_1955(ZNT_wat(i),UST_wat(i),WSPD(i),visc,ZA(I))
L01285               ELSE
L01286                  !COARE 3.5
L01287                  CALL edson_etal_2013(ZNT_wat(i),UST_wat(i),WSPD(i),visc,ZA(I))
L01288               ENDIF
L01289            ENDIF
L01290         endif !-end wave model check
L01291
L01292         ! add stochastic perturbation of ZNT
L01293         if (spp_sfc==1) then
L01294            ZNTstoch_wat(I)  = MAX(ZNT_wat(I) + ZNT_wat(I)*1.0*rstoch1D(i), 1e-6_kind_phys)
L01295         else
L01296            ZNTstoch_wat(I)  = ZNT_wat(I)
L01297         endif
L01298
L01299         IF (debug_code > 1) THEN
L01300            write(*,*)"==========Output ZNT over water:"
L01301            write(*,*)"ZNT:",ZNTstoch_wat(i)
L01302         ENDIF
L01303
L01304         !COMPUTE ROUGHNESS REYNOLDS NUMBER (restar) USING NEW ZNT
L01305         ! AHW: Garrattt formula: Calculate roughness Reynolds number
L01306         !      Kinematic viscosity of air (linear approx to
L01307         !      temp dependence at sea level)
L01308         restar=MAX(ust_wat(i)*ZNTstoch_wat(i)/visc, 0.1_kind_phys)
L01309
L01310         !---------------------------------------
L01311         !CALCULATE z_t and z_q
L01312         !---------------------------------------
L01313         IF (debug_code > 1) THEN
L01314            write(*,*)"==============Input to ZT over water:"
L01315            write(*,*)"u*:",UST_wat(i)," restar=",restar," visc=",visc
L01316         ENDIF
L01317
L01318         IF ( PRESENT(ISFTCFLX) ) THEN
L01319            IF ( ISFTCFLX .EQ. 0 ) THEN
L01320               IF (COARE_OPT .EQ. 3.0) THEN
L01321                  CALL fairall_etal_2003(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01322                                         rstoch1D(i),spp_sfc)
L01323               ELSE
L01324                  CALL fairall_etal_2014(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01325                                         rstoch1D(i),spp_sfc)
L01326               ENDIF
L01327            ELSEIF ( ISFTCFLX .EQ. 1 ) THEN
L01328               IF (COARE_OPT .EQ. 3.0) THEN
L01329                  CALL fairall_etal_2003(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01330                                         rstoch1D(i),spp_sfc)
L01331               ELSE
L01332                  CALL fairall_etal_2014(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01333                                         rstoch1D(i),spp_sfc)
L01334               ENDIF
L01335            ELSEIF ( ISFTCFLX .EQ. 2 ) THEN
L01336               CALL garratt_1992(ZT_wat(i),ZQ_wat(i),ZNTstoch_wat(i),restar,2.0_kind_phys)
L01337            ELSEIF ( ISFTCFLX .EQ. 3 ) THEN
L01338               IF (COARE_OPT .EQ. 3.0) THEN
L01339                  CALL fairall_etal_2003(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01340                                         rstoch1D(i),spp_sfc)
L01341               ELSE
L01342                  CALL fairall_etal_2014(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01343                                         rstoch1D(i),spp_sfc)
L01344               ENDIF
L01345            ELSEIF ( ISFTCFLX .EQ. 4 ) THEN
L01346               !GFS zt formulation
L01347               CALL GFS_zt_wat(ZT_wat(i),ZNTstoch_wat(i),restar,WSPD(i),ZA(i),sfc_z0_type,device_errmsg,device_errflg)
L01348               if(errflg/=0) return
L01349
L01350               ZQ_wat(i)=ZT_wat(i)
L01351            ENDIF
L01352         ELSE
L01353            !DEFAULT TO COARE 3.0/3.5
L01354            IF (COARE_OPT .EQ. 3.0) THEN
L01355               CALL fairall_etal_2003(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01356                                      rstoch1D(i),spp_sfc)
L01357            ELSE
L01358               CALL fairall_etal_2014(ZT_wat(i),ZQ_wat(i),restar,UST_wat(i),visc,&
L01359                                      rstoch1D(i),spp_sfc)
L01360            ENDIF
L01361         ENDIF
L01362
L01363         IF (debug_code > 1) THEN
L01364           write(*,*)"==============Output ZT & ZQ over water:"
L01365           write(*,*)"ZT:",ZT_wat(i)," ZQ:",ZQ_wat(i)
L01366         ENDIF
L01367
L01368         GZ1OZ0_wat(I)= LOG((ZA(I)+ZNTstoch_wat(I))/ZNTstoch_wat(I))
L01369         GZ1OZt_wat(I)= LOG((ZA(I)+ZNTstoch_wat(i))/ZT_wat(i))
L01370         GZ2OZ0_wat(I)= LOG((2.0+ZNTstoch_wat(I))/ZNTstoch_wat(I))
L01371         GZ2OZt_wat(I)= LOG((2.0+ZNTstoch_wat(i))/ZT_wat(i))
L01372         GZ10OZ0_wat(I)=LOG((10.+ZNTstoch_wat(I))/ZNTstoch_wat(I))
L01373         GZ10OZt_wat(I)=LOG((10.+ZNTstoch_wat(i))/ZT_wat(i))
L01374         zratio_wat(i)=ZNTstoch_wat(I)/ZT_wat(I)   !need estimate for Li et al.
L01375
L01376      ENDIF !end water point
L01377
L01378      IF (dry(I)) THEN
L01379
```

```
L01380         if ( IZ0TLND .EQ. 4 ) then
L01381            CALL GFS_z0_lnd(ZNT_lnd(i),shdmax(i),ZA(i),vegtype(i),ivegsrc,z0pert(i))
L01382         endif
L01383
L01384         ! add stochastic perturbaction of ZNT
L01385         if (spp_sfc==1) then
L01386            ZNTstoch_lnd(I)  = MAX(ZNT_lnd(I) + ZNT_lnd(I)*1.0*rstoch1D(i), 1e-6_kind_phys)
L01387         else
L01388            ZNTstoch_lnd(I)  = ZNT_lnd(I)
L01389         endif
L01390         !add limit to prevent ridiculous values of z0 (more than dz/15)
L01391         ZNTstoch_lnd(I) = min(ZNTstoch_lnd(I), dz8w1d(i)*0.0666_kind_phys)
L01392
L01393         !---------------------------------------
L01394         ! LAND
L01395         !---------------------------------------
L01396         !COMPUTE ROUGHNESS REYNOLDS NUMBER (restar) USING DEFAULT ZNT
L01397         restar=MAX(ust_lnd(i)*ZNTstoch_lnd(i)/visc, 0.1_kind_phys)
L01398
L01399         !---------------------------------------
L01400         !GET z_t and z_q
L01401         !---------------------------------------
L01402         IF (snowh_lnd(i) > 50.) THEN  ! (mm) Treat as snow cover - use Andreas
L01403            CALL Andreas_2002(ZNTstoch_lnd(i),visc,ust_lnd(i),ZT_lnd(i),ZQ_lnd(i))
L01404         ELSE
L01405            IF ( PRESENT(IZ0TLND) ) THEN
L01406               IF ( IZ0TLND .LE. 1 ) THEN
L01407                  CALL zilitinkevich_1995(ZNTstoch_lnd(i),ZT_lnd(i),ZQ_lnd(i),restar,&
L01408                        UST_lnd(I),KARMAN,1.0_kind_phys,IZ0TLND,spp_sfc,rstoch1D(i))
L01409               ELSEIF ( IZ0TLND .EQ. 2 ) THEN
L01410                  ! DH note - at this point, qstar is either not initialized
L01411                  ! or initialized to zero, but certainly not set correctly
L01412                  device_special_errmsg = 'Logic error: qstar is not set correctly when calling Yang_2008'
L01413                  device_special_errflg = 1
L01414  #ifndef _OPENACC
L01415  ! Necessary since OpenACC does not support branching in parallel code
L01416  ! Must sync errmsg and errflg with device_errmsg and device_errflg, respectively
L01417  ! so that proper error message and error flag codes are returned.
L01418                  errmsg = device_special_errmsg
L01419                  errflg = device_special_errflg
L01420                  return
L01421  #endif
L01422                  CALL Yang_2008(ZNTSTOCH_lnd(i),ZT_lnd(i),ZQ_lnd(i),UST_lnd(i),MOL(I),&
L01423                                 qstar(I),restar,visc)
L01424               ELSEIF ( IZ0TLND .EQ. 3 ) THEN
L01425                  !Original MYNN in WRF-ARW used this form:
L01426                  CALL garratt_1992(ZT_lnd(i),ZQ_lnd(i),ZNTSTOCH_lnd(i),restar,1.0_kind_phys)
L01427               ELSEIF ( IZ0TLND .EQ. 4 ) THEN
L01428                  !GFS:
L01429                  CALL GFS_zt_lnd(ZT_lnd(i),ZNTSTOCH_lnd(i),sigmaf(i),ztpert(i),UST_lnd(i))
L01430                  ZQ_lnd(i)=ZT_lnd(i)
L01431               ENDIF
L01432            ELSE
L01433               !DEFAULT TO ZILITINKEVICH
L01434               CALL zilitinkevich_1995(ZNTSTOCH_lnd(i),ZT_lnd(i),ZQ_lnd(i),restar,&
L01435                           UST_lnd(I),KARMAN,1.0_kind_phys,0,spp_sfc,rstoch1D(i))
L01436            ENDIF
L01437         ENDIF
L01438
L01439         IF (ZNTstoch_lnd(i) < 1E-8 .OR. Zt_lnd(i) < 1E-10) THEN
L01440           write(0,*)"===(land) capture bad input in mynn sfc layer, i=:",i
L01441           write(0,*)" ZNT=", ZNTstoch_lnd(i)," ZT=",Zt_lnd(i)
L01442           write(0,*)" tsk=", tskin_lnd(i)," restar=",restar,&
L01443           " tsurf=", tsurf_lnd(i)," qsfc=", qsfc_lnd(i)," znt=", znt_lnd(i),&
L01444           " ust=", ust_lnd(i)," snowh=", snowh_lnd(i),"psfcpa=",PSFCPA(i), &
L01445           " dz=",dz8w1d(i)," qflx=",qflx_lnd(i)," hflx=",hflx_lnd(i)," hpbl=",pblh(i)
L01446         ENDIF
L01447
L01448         GZ1OZ0_lnd(I)= LOG((ZA(I)+ZNTstoch_lnd(I))/ZNTstoch_lnd(I))
L01449         GZ1OZt_lnd(I)= LOG((ZA(I)+ZNTstoch_lnd(i))/ZT_lnd(i))
L01450         GZ2OZ0_lnd(I)= LOG((2.0+ZNTstoch_lnd(I))/ZNTstoch_lnd(I))
L01451         GZ2OZt_lnd(I)= LOG((2.0+ZNTstoch_lnd(i))/ZT_lnd(i))
L01452         GZ10OZ0_lnd(I)=LOG((10.+ZNTstoch_lnd(I))/ZNTstoch_lnd(I))
L01453         GZ10OZt_lnd(I)=LOG((10.+ZNTstoch_lnd(i))/ZT_lnd(i))
L01454         zratio_lnd(i)=ZNTstoch_lnd(I)/ZT_lnd(I)   !need estimate for Li et al.
L01455
L01456      ENDIF !end land point
L01457
L01458      IF (icy(I)) THEN
L01459
L01460         ! add stochastic perturbaction of ZNT
L01461         if (spp_sfc==1) then
L01462            ZNTstoch_ice(I)  = MAX(ZNT_ice(I) + ZNT_ice(I)*1.0*rstoch1D(i), 1e-6_kind_phys)
L01463         else
L01464            ZNTstoch_ice(I)  = ZNT_ice(I)
L01465         endif
L01466
L01467         !---------------------------------------
L01468         ! ICE
L01469         !---------------------------------------
L01470         !COMPUTE ROUGHNESS REYNOLDS NUMBER (restar) USING DEFAULT ZNT
L01471         restar=MAX(ust_ice(i)*ZNTstoch_ice(i)/visc, 0.1_kind_phys)
L01472         !---------------------------------------
L01473         !GET z_t and z_q
L01474         !---------------------------------------
L01475         CALL Andreas_2002(ZNTstoch_ice(i),visc,ust_ice(i),ZT_ice(i),ZQ_ice(i))
L01476
L01477         GZ1OZ0_ice(I)= LOG((ZA(I)+ZNTstoch_ice(I))/ZNTstoch_ice(I))
L01478         GZ1OZt_ice(I)= LOG((ZA(I)+ZNTstoch_ice(i))/ZT_ice(i))
L01479         GZ2OZ0_ice(I)= LOG((2.0+ZNTstoch_ice(I))/ZNTstoch_ice(I))
L01480         GZ2OZt_ice(I)= LOG((2.0+ZNTstoch_ice(i))/ZT_ice(i))
L01481         GZ10OZ0_ice(I)=LOG((10.+ZNTstoch_ice(I))/ZNTstoch_ice(I))
L01482         GZ10OZt_ice(I)=LOG((10.+ZNTstoch_ice(i))/ZT_ice(i))
L01483         zratio_ice(i)=ZNTstoch_ice(I)/ZT_ice(I)   !need estimate for Li et al.
L01484
L01485      ENDIF  !end ice point
L01486
L01487      !Capture a representative ZNT
```

```
L01488      !tgs - should this be changed for fractional grid or fractional sea ice?
L01489      IF (dry(i)) THEN
L01490         ZNT(i)=ZNTstoch_lnd(I)
L01491      ELSEIF (wet(i)) THEN
L01492         ZNT(i)=ZNTstoch_wat(I)
L01493      ELSEIF (icy(i)) THEN
L01494         ZNT(i)=ZNTstoch_ice(I)
L01495      ENDIF
```

讲解：先算动理黏性 VISC（Andreas 1989 三次多项式，−173~277 °C 有效）——后面 roughness Reynolds 数 restar=u* • z0/VISC 的分母。**水面**：sfc_z0_type≥0 才自算 z0（负数留给波浪模型耦合），isftcflx 分派五式：0=Charnock 1955（COARE 3.0 名义默认）、1/2=Davis 2008、3=Taylor–Yelland 2001、4=GFS_z0_wat；zt/zq 由 Fairall 2003/2014（restar 参数化）或 Garratt 1992（factor=2.0）；**陆面**：iz0tlnd 分派五式（详见 4.5 组一），雪深 >50 mm 直接走 Andreas 2002；SPP 扰动以乘性形式 z0 • (1+rstoch) 加在 ZNTstoch 上；znt 钳位上限 dz/15——防止粗糙度超过格距可比量；**冰面**：无选项，Andreas 2002 一条路。最后每个下垫面组六个 GZ 对数 + zratio=z0/zt（Li 2010 初猜要用）。

S7 稳定度分类 → ψ 函数 → 阻抗：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1497-L1906
L01497      !-------------------------------------------------------------------
L01498      !--- DIAGNOSE STABILITY FUNCTIONS FOR THE APPROPRIATE STABILITY CLASS:
L01499      !    THE STABILITY CLASSES ARE DETERMINED BY THE BULK RICHARDSON NUMBER.
L01500      !-------------------------------------------------------------------
L01501
L01502      IF (wet(i)) THEN
L01503         IF (rb_wat(I) .GT. 0.0) THEN
L01504
L01505            IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01506               !COMPUTE z/L first guess:
L01507               CALL Li_etal_2010(ZOL(I),rb_wat(I),ZA(I)/ZNTstoch_wat(I),zratio_wat(I))
L01508               !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_wat(I)*UST_wat(I),0.0001))
L01509               ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01510               ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01511
L01512               IF (debug_code >= 1) THEN
L01513                 IF (ZNTstoch_wat(i) < 1E-8 .OR. Zt_wat(i) < 1E-10) THEN
L01514                   write(0,*)"===(wet) capture bad input in mynn sfc layer, i=:",i
L01515                   write(0,*)"rb=", rb_wat(I)," ZNT=", ZNTstoch_wat(i)," ZT=",Zt_wat(i)
L01516                   write(0,*)" tsk=", tskin_wat(i)," prev z/L=",ZOL(I),&
L01517                   " tsurf=", tsurf_wat(i)," qsfc=", qsfc_wat(i)," znt=", znt_wat(i),&
L01518                   " ust=", ust_wat(i)," snowh=", snowh_wat(i),"psfcpa=",PSFCPA(i),  &
L01519                   " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
L01520                 ENDIF
L01521               ENDIF
L01522
L01523               !Use Pedros iterative function to find z/L
L01524               !zol(I)=zolri(rb_wat(I),ZA(I),ZNTstoch_wat(I),ZT_wat(I),ZOL(I),psi_opt)
L01525               !Use brute-force method
L01526               zol(I)=zolrib(rb_wat(I),ZA(I),ZNTstoch_wat(I),zt_wat(I),GZ1OZ0_wat(I),GZ1OZt_wat(I),ZOL(I),psi_opt)
L01527            ENDIF ! restart
L01528            ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01529            ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01530
L01531            zolzt = zol(I)*zt_wat(I)/ZA(I)               ! zt/L
L01532            zolz0 = zol(I)*ZNTstoch_wat(I)/ZA(I)         ! z0/L
L01533            zolza = zol(I)*(za(I)+ZNTstoch_wat(I))/za(I)  ! (z+z0/L
L01534            zol10 = zol(I)*(10.+ZNTstoch_wat(I))/za(I)    ! (10+z0)/L
L01535            zol2  = zol(I)*(2.+ZNTstoch_wat(I))/za(I)     ! (2+z0)/L
L01536
L01537            !COMPUTE PSIM and PSIH
L01538            !CALL PSI_Suselj_Sood_2010(PSIM(I),PSIH(I),ZOL(I))
L01539            !CALL PSI_Beljaars_Holtslag_1991(PSIM(I),PSIH(I),ZOL(I))
L01540            !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01541            !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_wat(I),ZNTstoch_wat(I),ZA(I))
L01542            !CALL PSI_CB2005(PSIM(I),PSIH(I),zolza,zolz0)
L01543            ! or use tables
L01544            psim(I)=psim_stable(zolza,psi_opt)-psim_stable(zolz0,psi_opt)
L01545            psih(I)=psih_stable(zolza,psi_opt)-psih_stable(zolzt,psi_opt)
L01546            psim10(I)=psim_stable(zol10,psi_opt)-psim_stable(zolz0,psi_opt)
L01547            psih10(I)=psih_stable(zol10,psi_opt)-psih_stable(zolz0,psi_opt)
L01548            psih2(I)=psih_stable(zol2,psi_opt)-psih_stable(zolzt,psi_opt)
L01549
L01550            ! 1.0 over Monin-Obukhov length
L01551            RMOL(I)= ZOL(I)/ZA(I)
L01552
L01553         ELSEIF(rb_wat(I) .EQ. 0.) THEN
L01554            !=========================================================
L01555            !-----CLASS 3; FORCED CONVECTION/NEUTRAL:
L01556            !=========================================================
L01557
L01558            PSIM(I)=0.0
L01559            PSIH(I)=PSIM(I)
L01560            PSIM10(I)=0.
L01561            PSIH10(I)=0.
L01562            PSIH2(I)=0.
L01563
L01564            ZOL(I)  =0.
L01565            RMOL(I) =0.
L01566
L01567         ELSEIF(rb_wat(I) .LT. 0.)THEN
L01568            !=========================================================
L01569            !-----CLASS 4; FREE CONVECTION:
L01570            !=========================================================
L01571
L01572            !COMPUTE z/L first guess:
L01573            IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01574               CALL Li_etal_2010(ZOL(I),rb_wat(I),ZA(I)/ZNTstoch_wat(I),zratio_wat(I))
L01575               !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_wat(I)*UST_wat(I),0.001))
```

```
L01576               ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01577               ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01578
L01579               IF (debug_code >= 1) THEN
L01580                 IF (ZNTstoch_wat(i) < 1E-8 .OR. Zt_wat(i) < 1E-10) THEN
L01581                   write(0,*)"===(wet) capture bad input in mynn sfc layer, i=:",i
L01582                   write(0,*)"rb=", rb_wat(I)," ZNT=", ZNTstoch_wat(i)," ZT=",Zt_wat(i)
L01583                   write(0,*)" tsk=", tskin_wat(i)," wstar=",wstar(i)," prev z/L=",ZOL(I),&
L01584                   " tsurf=", tsurf_wat(i)," qsfc=", qsfc_wat(i)," znt=", znt_wat(i),&
L01585                   " ust=", ust_wat(i)," snowh=", snowh_wat(i),"psfcpa=",PSFCPA(i),  &
L01586                   " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
L01587                 ENDIF
L01588               ENDIF
L01589
L01590               !Use Pedros iterative function to find z/L
L01591               !zol(I)=zolri(rb_wat(I),ZA(I),ZNTstoch_wat(I),ZT_wat(I),ZOL(I),psi_opt)
L01592               !Use brute-force method
L01593               zol(I)=zolrib(rb_wat(I),ZA(I),ZNTstoch_wat(I),zt_wat(I),GZ1OZ0_wat(I),GZ1OZt_wat(I),ZOL(I),psi_opt)
L01594             ENDIF ! restart
L01595             ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01596             ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01597
L01598             zolzt = zol(I)*zt_wat(I)/ZA(I)                ! zt/L
L01599             zolz0 = zol(I)*ZNTstoch_wat(I)/ZA(I)          ! z0/L
L01600             zolza = zol(I)*(za(I)+ZNTstoch_wat(I))/za(I)  ! (z+z0/L
L01601             zol10 = zol(I)*(10.+ZNTstoch_wat(I))/za(I)    ! (10+z0)/L
L01602             zol2  = zol(I)*(2.+ZNTstoch_wat(I))/za(I)     ! (2+z0)/L
L01603
L01604             !COMPUTE PSIM and PSIH
L01605             !CALL PSI_SuseLj_Sood_2010(PSIM(I),PSIH(I),ZOL(I))
L01606             !CALL PSI_Hogstrom_1996(PSIM(I),PSIH(I),ZOL(I), ZT_wat(I), ZNTstoch_wat(I), ZA(I))
L01607             !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01608             !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_wat(I),ZNTstoch_wat(I),ZA(I))
L01609             ! use tables
L01610             psim(I)=psim_unstable(zolza,psi_opt)-psim_unstable(zolz0,psi_opt)
L01611             psih(I)=psih_unstable(zolza,psi_opt)-psih_unstable(zolzt,psi_opt)
L01612             psim10(I)=psim_unstable(zol10,psi_opt)-psim_unstable(zolz0,psi_opt)
L01613             psih10(I)=psih_unstable(zol10,psi_opt)-psih_unstable(zolz0,psi_opt)
L01614             psih2(I)=psih_unstable(zol2,psi_opt)-psih_unstable(zolzt,psi_opt)
L01615
L01616             !---LIMIT PSIH AND PSIM IN THE CASE OF THIN LAYERS AND
L01617             !---HIGH ROUGHNESS.  THIS PREVENTS DENOMINATOR IN FLUXES
L01618             !---FROM GETTING TOO SMALL
L01619             PSIH(I)=MIN(PSIH(I),0.9*GZ1OZt_wat(I))
L01620             PSIM(I)=MIN(PSIM(I),0.9*GZ1OZ0_wat(I))
L01621             PSIH2(I)=MIN(PSIH2(I),0.9*GZ2OZt_wat(I))
L01622             PSIM10(I)=MIN(PSIM10(I),0.9*GZ10OZ0_wat(I))
L01623             PSIH10(I)=MIN(PSIH10(I),0.9*GZ10OZt_wat(I))
L01624
L01625             RMOL(I) = ZOL(I)/ZA(I)
L01626
L01627           ENDIF
L01628
L01629           ! CALCULATE THE RESISTANCE:
L01630           PSIX_wat(I)  =MAX(GZ1OZ0_wat(I)-PSIM(I)   , 1.0_kind_phys)    ! = fm
L01631           PSIX10_wat(I)=MAX(GZ10OZ0_wat(I)-PSIM10(I), 1.0_kind_phys)    ! = fm10
L01632           PSIT_wat(I)  =MAX(GZ1OZt_wat(I)-PSIH(I)   , 1.0_kind_phys)    ! = fh
L01633           PSIT2_wat(I) =MAX(GZ2OZt_wat(I)-PSIH2(I)  , 1.0_kind_phys)    ! = fh2
L01634           PSIQ_wat(I)  =MAX(LOG((ZA(I)+ZQ_wat(i))/ZQ_wat(I))-PSIH(I) ,1.0_kind_phys)
L01635           PSIQ2_wat(I) =MAX(LOG((2.0+ZQ_wat(i))/ZQ_wat(I))-PSIH2(I) ,1.0_kind_phys)
L01636
L01637         ENDIF ! end water points
L01638
L01639         IF (dry(i)) THEN
L01640           IF (rb_lnd(I) .GT. 0.0) THEN
L01641
L01642             IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01643               !COMPUTE z/L first guess:
L01644               CALL Li_etal_2010(ZOL(I),rb_lnd(I),ZA(I)/ZNTstoch_lnd(I),zratio_lnd(I))
L01645               !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_lnd(I)*UST_lnd(I),0.0001))
L01646               ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01647               ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01648
L01649               IF (debug_code >= 1) THEN
L01650                 IF (ZNTstoch_lnd(i) < 1E-8 .OR. Zt_lnd(i) < 1E-10) THEN
L01651                   write(0,*)"===(land) capture bad input in mynn sfc layer, i=:",i
L01652                   write(0,*)"rb=", rb_lnd(I)," ZNT=", ZNTstoch_lnd(i)," ZT=",Zt_lnd(i)
L01653                   write(0,*)" tsk=", tskin_lnd(i)," prev z/L=",ZOL(I),&
L01654                   " tsurf=", tsurf_lnd(i)," qsfc=", qsfc_lnd(i)," znt=", znt_lnd(i),&
L01655                   " ust=", ust_lnd(i)," snowh=", snowh_lnd(i),"psfcpa=",PSFCPA(i),  &
L01656                   " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
L01657                 ENDIF
L01658               ENDIF
L01659
L01660               !Use Pedros iterative function to find z/L
L01661               !zol(I)=zolri(rb_lnd(I),ZA(I),ZNTstoch_lnd(I),ZT_lnd(I),ZOL(I),psi_opt)
L01662               !Use brute-force method
L01663               zol(I)=zolrib(rb_lnd(I),ZA(I),ZNTstoch_lnd(I),zt_lnd(I),GZ1OZ0_lnd(I),GZ1OZt_lnd(I),ZOL(I),psi_opt)
L01664             ENDIF ! restart
L01665             ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01666             ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01667
L01668             zolzt = zol(I)*zt_lnd(I)/ZA(I)                ! zt/L
L01669             zolz0 = zol(I)*ZNTstoch_lnd(I)/ZA(I)          ! z0/L
L01670             zolza = zol(I)*(za(I)+ZNTstoch_lnd(I))/za(I)  ! (z+z0/L
L01671             zol10 = zol(I)*(10.+ZNTstoch_lnd(I))/za(I)    ! (10+z0)/L
L01672             zol2  = zol(I)*(2.+ZNTstoch_lnd(I))/za(I)     ! (2+z0)/L
L01673
L01674             !COMPUTE PSIM and PSIH
L01675             !CALL PSI_Beljaars_Holtslag_1991(PSIM(I),PSIH(I),ZOL(I))
L01676             !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01677             !CALL PSI_Zilitinkevich_Esau_2007(PSIM(I),PSIH(I),ZOL(I))
L01678             !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_lnd(I),ZNTstoch_lnd(I),ZA(I))
L01679             !CALL PSI_CB2005(PSIM(I),PSIH(I),zolza,zolz0)
L01680             psim(I)=psim_stable(zolza,psi_opt)-psim_stable(zolz0,psi_opt)
L01681             psih(I)=psih_stable(zolza,psi_opt)-psih_stable(zolzt,psi_opt)
L01682             psim10(I)=psim_stable(zol10,psi_opt)-psim_stable(zolz0,psi_opt)
L01683             psih10(I)=psih_stable(zol10,psi_opt)-psih_stable(zolz0,psi_opt)
```

```
L01684            psih2(I)=psih_stable(zol2,psi_opt)-psih_stable(zolzt,psi_opt)
L01685
L01686            ! 1.0 over Monin-Obukhov length
L01687            RMOL(I)= ZOL(I)/ZA(I)
L01688
L01689         ELSEIF(rb_lnd(I) .EQ. 0.) THEN
L01690            !=========================================================
L01691            !-----CLASS 3; FORCED CONVECTION/NEUTRAL:
L01692            !=========================================================
L01693
L01694            PSIM(I)=0.0
L01695            PSIH(I)=PSIM(I)
L01696            PSIM10(I)=0.
L01697            PSIH10(I)=0.
L01698            PSIH2(I)=0.
L01699
L01700            ZOL(I)  =0.
L01701            RMOL(I) =0.
L01702
L01703         ELSEIF(rb_lnd(I) .LT. 0.)THEN
L01704            !=========================================================
L01705            !-----CLASS 4; FREE CONVECTION:
L01706            !=========================================================
L01707
L01708            IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01709               !COMPUTE z/L first guess:
L01710               CALL Li_etal_2010(ZOL(I),rb_lnd(I),ZA(I)/ZNTstoch_lnd(I),zratio_lnd(I))
L01711               !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_lnd(I)*UST_lnd(I),0.001))
L01712               ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01713               ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01714
L01715               IF (debug_code >= 1) THEN
L01716                 IF (ZNTstoch_lnd(i) < 1E-8 .OR. Zt_lnd(i) < 1E-10) THEN
L01717                   write(0,*)"===(land) capture bad input in mynn sfc layer, i=:",i
L01718                   write(0,*)"rb=", rb_lnd(I)," ZNT=", ZNTstoch_lnd(i)," ZT=",Zt_lnd(i)
L01719                   write(0,*)" tsk=", tskin_lnd(i)," wstar=",wstar(i)," prev z/L=",ZOL(I),&
L01720                   " tsurf=", tsurf_lnd(i)," qsfc=", qsfc_lnd(i)," znt=", znt_lnd(i),&
L01721                   " ust=", ust_lnd(i)," snowh=", snowh_lnd(i),"psfcpa=",PSFCPA(i),  &
L01722                   " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
L01723                 ENDIF
L01724               ENDIF
L01725
L01726               !Use Pedros iterative function to find z/L
L01727               !zol(I)=zolri(rb_lnd(I),ZA(I),ZNTstoch_lnd(I),ZT_lnd(I),ZOL(I),psi_opt)
L01728               !Use brute-force method
L01729               zol(I)=zolrib(rb_lnd(I),ZA(I),ZNTstoch_lnd(I),zt_lnd(I),GZ1OZ0_lnd(I),GZ1OZt_lnd(I),ZOL(I),psi_opt)
L01730            ENDIF ! restart
L01731            ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01732            ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01733
L01734            zolzt = zol(I)*zt_lnd(I)/ZA(I)                ! zt/L
L01735            zolz0 = zol(I)*ZNTstoch_lnd(I)/ZA(I)          ! z0/L
L01736            zolza = zol(I)*(za(I)+ZNTstoch_lnd(I))/za(I)  ! (z+z0/L
L01737            zol10 = zol(I)*(10.+ZNTstoch_lnd(I))/za(I)    ! (10+z0)/L
L01738            zol2  = zol(I)*(2.+ZNTstoch_lnd(I))/za(I)     ! (2+z0)/L
L01739
L01740            !COMPUTE PSIM and PSIH
L01741            !CALL PSI_Hogstrom_1996(PSIM(I),PSIH(I),ZOL(I), ZT_lnd(I), ZNTstoch_lnd(I), ZA(I))
L01742            !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01743            !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_lnd(I),ZNTstoch_lnd(I),ZA(I))
L01744            ! use tables
L01745            psim(I)=psim_unstable(zolza,psi_opt)-psim_unstable(zolz0,psi_opt)
L01746            psih(I)=psih_unstable(zolza,psi_opt)-psih_unstable(zolzt,psi_opt)
L01747            psim10(I)=psim_unstable(zol10,psi_opt)-psim_unstable(zolz0,psi_opt)
L01748            psih10(I)=psih_unstable(zol10,psi_opt)-psih_unstable(zolz0,psi_opt)
L01749            psih2(I)=psih_unstable(zol2,psi_opt)-psih_unstable(zolzt,psi_opt)
L01750
L01751            !---LIMIT PSIH AND PSIM IN THE CASE OF THIN LAYERS AND
L01752            !---HIGH ROUGHNESS.  THIS PREVENTS DENOMINATOR IN FLUXES
L01753            !---FROM GETTING TOO SMALL
L01754            PSIH(I)=MIN(PSIH(I),0.9*GZ1OZt_lnd(I))
L01755            PSIM(I)=MIN(PSIM(I),0.9*GZ1OZ0_lnd(I))
L01756            PSIH2(I)=MIN(PSIH2(I),0.9*GZ2OZt_lnd(I))
L01757            PSIM10(I)=MIN(PSIM10(I),0.9*GZ10OZ0_lnd(I))
L01758            PSIH10(I)=MIN(PSIH10(I),0.9*GZ10OZt_lnd(I))
L01759
L01760            RMOL(I) = ZOL(I)/ZA(I)
L01761
L01762         ENDIF
L01763
L01764         ! CALCULATE THE RESISTANCE:
L01765         PSIX_lnd(I)  =MAX(GZ1OZ0_lnd(I)-PSIM(I), 1.0_kind_phys)
L01766         PSIX10_lnd(I)=MAX(GZ10OZ0_lnd(I)-PSIM10(I), 1.0_kind_phys)
L01767         PSIT_lnd(I)  =MAX(GZ1OZt_lnd(I)-PSIH(I) , 1.0_kind_phys)
L01768         PSIT2_lnd(I) =MAX(GZ2OZt_lnd(I)-PSIH2(I), 1.0_kind_phys)
L01769         PSIQ_lnd(I)  =MAX(LOG((ZA(I)+ZQ_lnd(i))/ZQ_lnd(I))-PSIH(I) ,1.0_kind_phys)
L01770         PSIQ2_lnd(I) =MAX(LOG((2.0+ZQ_lnd(i))/ZQ_lnd(I))-PSIH2(I) ,1.0_kind_phys)
L01771
L01772      ENDIF ! end land points
L01773
L01774      IF (icy(i)) THEN
L01775         IF (rb_ice(I) .GT. 0.0) THEN
L01776
L01777            IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01778               !COMPUTE z/L first guess:
L01779               CALL Li_etal_2010(ZOL(I),rb_ice(I),ZA(I)/ZNTstoch_ice(I),zratio_ice(I))
L01780               !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_ice(I)*UST_ice(I),0.0001))
L01781               ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01782               ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01783
L01784               IF (debug_code >= 1) THEN
L01785                 IF (ZNTstoch_ice(i) < 1E-8 .OR. Zt_ice(i) < 1E-10) THEN
L01786                   write(0,*)"===(ice) capture bad input in mynn sfc layer, i=:",i
L01787                   write(0,*)"rb=", rb_ice(I)," ZNT=", ZNTstoch_ice(i)," ZT=",Zt_ice(i)
L01788                   write(0,*)" tsk=", tskin_ice(i)," prev z/L=",ZOL(I),&
L01789                   " tsurf=", tsurf_ice(i)," qsfc=", qsfc_ice(i)," znt=", znt_ice(i),&
L01790                   " ust=", ust_ice(i)," snowh=", snowh_ice(i),"psfcpa=",PSFCPA(i),  &
L01791                   " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
```

```
L01792                ENDIF
L01793             ENDIF
L01794
L01795             !Use Pedros iterative function to find z/L
L01796             !zol(I)=zolri(rb_ice(I),ZA(I),ZNTstoch_ice(I),ZT_ice(I),ZOL(I),psi_opt)
L01797             !Use brute-force method
L01798             zol(I)=zolrib(rb_ice(I),ZA(I),ZNTstoch_ice(I),zt_ice(I),GZ1OZ0_ice(I),GZ1OZt_ice(I),ZOL(I),psi_opt)
L01799          ENDIF ! restart
L01800          ZOL(I)=MAX(ZOL(I),0.0_kind_phys)
L01801          ZOL(I)=MIN(ZOL(I),20._kind_phys)
L01802
L01803          zolzt = zol(I)*zt_ice(I)/ZA(I)               ! zt/L
L01804          zolz0 = zol(I)*ZNTstoch_ice(I)/ZA(I)         ! z0/L
L01805          zolza = zol(I)*(za(I)+ZNTstoch_ice(I))/za(I)  ! (z+z0/L
L01806          zol10 = zol(I)*(10.+ZNTstoch_ice(I))/za(I)    ! (10+z0)/L
L01807          zol2  = zol(I)*(2.+ZNTstoch_ice(I))/za(I)     ! (2+z0)/L
L01808
L01809          !COMPUTE PSIM and PSIH
L01810          !CALL PSI_Beljaars_Holtslag_1991(PSIM(I),PSIH(I),ZOL(I))
L01811          !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01812          !CALL PSI_Zilitinkevich_Esau_2007(PSIM(I),PSIH(I),ZOL(I))
L01813          !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_ice(I),ZNTstoch_ice(I),ZA(I))
L01814          !CALL PSI_CB2005(PSIM(I),PSIH(I),zolza,zolz0)
L01815          psim(I)=psim_stable(zolza,psi_opt)-psim_stable(zolz0,psi_opt)
L01816          psih(I)=psih_stable(zolza,psi_opt)-psih_stable(zolzt,psi_opt)
L01817          psim10(I)=psim_stable(zol10,psi_opt)-psim_stable(zolz0,psi_opt)
L01818          psih10(I)=psih_stable(zol10,psi_opt)-psih_stable(zolz0,psi_opt)
L01819          psih2(I)=psih_stable(zol2,psi_opt)-psih_stable(zolzt,psi_opt)
L01820
L01821          ! 1.0 over Monin-Obukhov Length
L01822          RMOL(I)= ZOL(I)/ZA(I)
L01823
L01824       ELSEIF(rb_ice(I) .EQ. 0.) THEN
L01825          !=========================================================
L01826          !-----CLASS 3; FORCED CONVECTION/NEUTRAL:
L01827          !=========================================================
L01828
L01829          PSIM(I)=0.0
L01830          PSIH(I)=PSIM(I)
L01831          PSIM10(I)=0.
L01832          PSIH10(I)=0.
L01833          PSIH2(I)=0.
L01834
L01835          ZOL(I)  =0.
L01836          RMOL(I) =0.
L01837
L01838       ELSEIF(rb_ice(I) .LT. 0.)THEN
L01839          !=========================================================
L01840          !-----CLASS 4; FREE CONVECTION:
L01841          !=========================================================
L01842
L01843          IF (.not. flag_restart .or. (flag_restart .and. itimestep > 1) ) THEN
L01844             !COMPUTE z/L first guess:
L01845             CALL Li_etal_2010(ZOL(I),rb_ice(I),ZA(I)/ZNTstoch_ice(I),zratio_ice(I))
L01846             !ZOL(I)=ZA(I)*KARMAN*grav*MOL(I)/(TH1D(I)*MAX(UST_ice(I)*UST_ice(I),0.001))
L01847             ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01848             ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01849
L01850             IF (debug_code >= 1) THEN
L01851               IF (ZNTstoch_ice(i) < 1E-8 .OR. Zt_ice(i) < 1E-10) THEN
L01852                 write(0,*)"===(ice) capture bad input in mynn sfc layer, i=:",i
L01853                 write(0,*)"rb=", rb_ice(I)," ZNT=", ZNTstoch_ice(i)," ZT=",Zt_ice(i)
L01854                 write(0,*)" tsk=", tskin_ice(i)," wstar=",wstar(i)," prev z/L=",ZOL(I),&
L01855                 " tsurf=", tsurf_ice(i)," qsfc=", qsfc_ice(i)," znt=", znt_ice(i),&
L01856                 " ust=", ust_ice(i)," snowh=", snowh_ice(i),"psfcpa=",PSFCPA(i),  &
L01857                 " dz=",dz8w1d(i)," qflx=",qflx(i)," hflx=",hflx(i)," hpbl=",pblh(i)
L01858               ENDIF
L01859             ENDIF
L01860
L01861             !Use Pedros iterative function to find z/L
L01862             !zol(I)=zolri(rb_ice(I),ZA(I),ZNTstoch_ice(I),ZT_ice(I),ZOL(I),psi_opt)
L01863             !Use brute-force method
L01864             zol(I)=zolrib(rb_ice(I),ZA(I),ZNTstoch_ice(I),zt_ice(I),GZ1OZ0_ice(I),GZ1OZt_ice(I),ZOL(I),psi_opt)
L01865          ENDIF ! restart
L01866          ZOL(I)=MAX(ZOL(I),-20.0_kind_phys)
L01867          ZOL(I)=MIN(ZOL(I),0.0_kind_phys)
L01868
L01869          zolzt = zol(I)*zt_ice(I)/ZA(I)               ! zt/L
L01870          zolz0 = zol(I)*ZNTstoch_ice(I)/ZA(I)         ! z0/L
L01871          zolza = zol(I)*(za(I)+ZNTstoch_ice(I))/za(I)  ! (z+z0/L
L01872          zol10 = zol(I)*(10.+ZNTstoch_ice(I))/za(I)    ! (10+z0)/L
L01873          zol2  = zol(I)*(2.+ZNTstoch_ice(I))/za(I)     ! (2+z0)/L
L01874
L01875          !COMPUTE PSIM and PSIH
L01876          !CALL PSI_Hogstrom_1996(PSIM(I),PSIH(I),ZOL(I), ZT_ice(I), ZNTstoch_ice(I), ZA(I))
L01877          !CALL PSI_Businger_1971(PSIM(I),PSIH(I),ZOL(I))
L01878          !CALL PSI_DyerHicks(PSIM(I),PSIH(I),ZOL(I),ZT_ice(I),ZNTstoch_ice(I),ZA(I))
L01879          ! use tables
L01880          psim(I)=psim_unstable(zolza,psi_opt)-psim_unstable(zolz0,psi_opt)
L01881          psih(I)=psih_unstable(zolza,psi_opt)-psih_unstable(zolzt,psi_opt)
L01882          psim10(I)=psim_unstable(zol10,psi_opt)-psim_unstable(zolz0,psi_opt)
L01883          psih10(I)=psih_unstable(zol10,psi_opt)-psih_unstable(zolz0,psi_opt)
L01884          psih2(I)=psih_unstable(zol2,psi_opt)-psih_unstable(zolzt,psi_opt)
L01885
L01886          !---LIMIT PSIH AND PSIM IN THE CASE OF THIN LAYERS AND
L01887          !---HIGH ROUGHNESS.  THIS PREVENTS DENOMINATOR IN FLUXES
L01888          !---FROM GETTING TOO SMALL
L01889          PSIH(I)=MIN(PSIH(I),0.9*GZ1OZt_ice(I))
L01890          PSIM(I)=MIN(PSIM(I),0.9*GZ1OZ0_ice(I))
L01891          PSIH2(I)=MIN(PSIH2(I),0.9*GZ2OZt_ice(I))
L01892          PSIM10(I)=MIN(PSIM10(I),0.9*GZ10OZ0_ice(I))
L01893          PSIH10(I)=MIN(PSIH10(I),0.9*GZ10OZt_ice(I))
L01894
L01895          RMOL(I) = ZOL(I)/ZA(I)
L01896
L01897       ENDIF
L01898
L01899       ! CALCULATE THE RESISTANCE:
```

```
L01900         PSIX_ice(I)  =MAX(GZ1OZ0_ice(I)-PSIM(I)   , 1.0_kind_phys)
L01901         PSIX10_ice(I)=MAX(GZ10OZ0_ice(I)-PSIM10(I), 1.0_kind_phys)
L01902         PSIT_ice(I)  =MAX(GZ1OZt_ice(I)-PSIH(I)   , 1.0_kind_phys)
L01903         PSIT2_ice(I) =MAX(GZ2OZt_ice(I)-PSIH2(I)  , 1.0_kind_phys)
L01904         PSIQ_ice(I)  =MAX(LOG((ZA(I)+ZQ_ice(i))/ZQ_ice(I))-PSIH(I) ,1.0_kind_phys)
L01905         PSIQ2_ice(I) =MAX(LOG((2.0+ZQ_ice(i))/ZQ_ice(I))-PSIH2(I)  ,1.0_kind_phys)
L01906
```

讲解：三下标三段几乎逐行同构，以水面段为例读懂一遍即可平移：①rb>0 稳定侧：Li_etal_2010 显式初猜 ZOL，再 zolrib 不动点迭代（20 步上限、收敛阈 0.01，失败回退 Li 2010）；ZOL 钳位 [0,20]；五个 z/L 变体（z0/L、zt/L、(z+z0)/L、(10+z0)/L、(2+z0)/L）分别进 ψ_m、ψ_h 查表做**差分** ψ(z)−ψ(z0)——「积分相似性函数差」；②rb=0 中性：ψ 全零；③rb<0 不稳定侧：同迭代流程但 ZOL 钳位 [−20,0]，且多一步 ψ 钳位 min(ψ, 0.9 • GZ)——薄层高粗糙时防分母过小通量爆掉；④ 每个下垫面段末组装六个阻抗：PSIX=GZ1OZ0−ψ_m（动量）、PSIT=GZ1OZt−ψ_h（热）、PSIQ=log((z+zq)/zq)−ψ_h（水汽，注意 zq 骨架）等，一律下限 1.0。psi_opt 参数决定查哪张表（0=MYNN 混合、1=GFS），调用点全部走表函数 psim_stable(zolf,psi_opt)。

S8 摩擦速度与温度/湿度尺度：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L1908-L2007
L01908
L01909       !-------------------------------------------------------------
L01910       !-----COMPUTE THE FRICTIONAL VELOCITY:
L01911       !-------------------------------------------------------------
L01912
L01913       IF (wet(I)) THEN
L01914          ! TO PREVENT OSCILLATIONS AVERAGE WITH OLD VALUE
L01915          OLDUST = UST_wat(I)
L01916          UST_wat(I)=0.5*UST_wat(I)+0.5*KARMAN*WSPD(I)/PSIX_wat(I)
L01917          !NON-AVERAGED:
L01918          !UST_wat(I)=KARMAN*WSPD(I)/PSIX_wat(I)
L01919          stress_wat(i)=ust_wat(i)**2
L01920
L01921          ! Compute u* without vconv for use in HFX calc when isftcflx > 0
L01922          WSPDI(I)=MAX(SQRT(U1D(I)*U1D(I)+V1D(I)*V1D(I)), wmin)
L01923          !tgs - should USTM be separater for dry, icy, wet?
L01924          USTM(I)=0.5*USTM(I)+0.5*KARMAN*WSPDI(I)/PSIX_wat(I)
L01925
L01926          ! for possible future changes in sea-ice fraction from 0 to >0:
L01927          if (.not. icy(i)) ust_ice(i)=ust_wat(i)
L01928       ENDIF ! end water points
L01929
L01930       IF (dry(I)) THEN
L01931          ! TO PREVENT OSCILLATIONS AVERAGE WITH OLD VALUE
L01932          OLDUST = UST_lnd(I)
L01933          UST_lnd(I)=0.5*UST_lnd(I)+0.5*KARMAN*WSPD(I)/PSIX_lnd(I)
L01934          !NON-AVERAGED:
L01935          !UST_lnd(I)=KARMAN*WSPD(I)/PSIX_lnd(I)
L01936          !From Tilden Meyers:
L01937          !IF (rb_lnd(I) .GE. 0.0) THEN
L01938          !   ust_lnd(i)=WSPD_lnd*0.1/(1.0 + 10.0*rb_lnd(I))
L01939          !ELSE
L01940          !   ust_lnd(i)=WSPD_lnd*0.1*(1.0 - 10.0*rb_lnd(I))**onethird
L01941          !ENDIF
L01942          UST_lnd(I)=MAX(UST_lnd(I),0.005_kind_phys)
L01943          stress_lnd(i)=ust_lnd(i)**2
L01944
L01945          !set ustm = ust over land.
L01946          !tgs - should USTM be separater for dry, icy, wet?
L01947          USTM(I)=UST_lnd(I)
L01948       ENDIF ! end water points
L01949
L01950       IF (icy(I)) THEN
L01951          ! TO PREVENT OSCILLATIONS AVERAGE WITH OLD VALUE
L01952          OLDUST = UST_ice(I)
L01953          UST_ice(I)=0.5*UST_ice(I)+0.5*KARMAN*WSPD(I)/PSIX_ice(I)
L01954          !NON-AVERAGED:
L01955          !UST_ice(I)=KARMAN*WSPD(I)/PSIX_ice(I)
L01956          UST_ice(I)=MAX(UST_ice(I),0.005_kind_phys)
L01957          stress_ice(i)=ust_ice(i)**2
L01958
L01959          !Set ustm = ust over ice.
L01960          !tgs - should USTM be separate for for dry, icy, wet?
L01961          USTM(I)=UST_ice(I)
L01962
L01963          ! for possible future changes in sea-ice fraction from 1 to <1:
L01964          !tgs - sea ice can be <1 now
L01965          if (.not. wet(i)) ust_wat(i)=ust_ice(i)
L01966       ENDIF ! end ice points
L01967
L01968       !-----------------------------------------------------
L01969       !----COMPUTE THE TEMPERATURE SCALE (a.k.a. FRICTION TEMPERATURE, T*, or MOL)
L01970       !----AND COMPUTE THE MOISTURE SCALE (or q*)
L01971       !-----------------------------------------------------
L01972
L01973       !tgs - should we have MOL and qstar separate for dry, icy and wet?
L01974       IF (wet(I)) THEN
L01975          DTG=THV1D(I)-THVSK_wat(I)
L01976          OLDTST=MOL(I)
L01977          MOL(I)=KARMAN*DTG/PSIT_wat(I)/PRT
L01978          !t_star(I) = -HFX(I)/(UST(I)*CPM(I)*RHO1D(I))
L01979          !t_star(I) = MOL(I)
L01980          !-----------------------------------------------------
L01981          DQG=(QVSH(i)-qsfc_wat(i))*1000.   !(kg/kg -> g/kg)
L01982          qstar(I)=KARMAN*DQG/PSIQ_wat(I)/PRT
L01983       ENDIF
L01984
L01985       IF (dry(I)) THEN
L01986          DTG=THV1D(I)-THVSK_lnd(I)
```

```
L01987         OLDTST=MOL(I)
L01988         MOL(I)=KARMAN*DTG/PSIT_lnd(I)/PRT
L01989         !t_star(I) = -HFX(I)/(UST(I)*CPM(I)*RHO1D(I))
L01990         !t_star(I) = MOL(I)
L01991         !-------------------------------------------------
L01992         DQG=(QVSH(i)-qsfc_lnd(i))*1000.   !(kg/kg -> g/kg)
L01993         qstar(I)=KARMAN*DQG/PSIQ_lnd(I)/PRT
L01994       ENDIF
L01995
L01996       IF (icy(I)) THEN
L01997         DTG=THV1D(I)-THVSK_ice(I)
L01998         OLDTST=MOL(I)
L01999         MOL(I)=KARMAN*DTG/PSIT_ice(I)/PRT
L02000         !t_star(I) = -HFX(I)/(UST(I)*CPM(I)*RHO1D(I))
L02001         !t_star(I) = MOL(I)
L02002         !-------------------------------------------------
L02003         DQG=(QVSH(i)-qsfc_ice(i))*1000.   !(kg/kg -> g/kg)
L02004         qstar(I)=KARMAN*DQG/PSIQ_ice(I)/PRT
L02005       ENDIF
L02006
L02007      endif ! flag_iter
```

讲解：①u* = 0.5 • u*_old + 0.5 • κ • WSPD/PSIX——与上时刻值对半平滑防振荡（注释保留了非平滑版）；陆/冰侧下限 0.005 m/s；水面 USTM 用无 W* 的 WSPDI 另算（isftcflx>0 的耗散加热项要用纯动力 u），并做 *wet/icy* 交叉同步（全冰点把冰的 *u* 抄给水——为将来部分海冰铺路）；②t（变量名 *MOL*，注意与「*Monin-Obukhov* 长度 *L*」的命名陷阱：本文件里 *ZOL=z/L*、*RMOL=1/L*、*MOL=t*）=κ • Δθv/PSIT/PRT，q*=κ • Δq[g/kg]/PSIQ/PRT，PRT=1 湍流 Prandtl 数。

S9 通量组装：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2051-L2265
L02051
L02052      !-------------------------------------------------------------
L02053      !  COMPUTE SURFACE HEAT AND MOISTURE FLUXES
L02054      !-------------------------------------------------------------
L02055   !$acc parallel loop present(flag_iter, dry, wet, icy, &
L02056   !$acc                       QFX, HFX, FLHC, FLQC, LH, CHS, CH, CHS2, CQS2, &
L02057   !$acc                       RHO1D, MAVAIL, USTM, &
L02058   !$acc                       UST_lnd, UST_wat, UST_ice, &
L02059   !$acc                       PSIQ_lnd, PSIT_lnd, PSIX_lnd, &
L02060   !$acc                       PSIQ_wat, PSIT_wat, PSIX_wat, &
L02061   !$acc                       PSIQ_ice, PSIT_ice, PSIX_ice, &
L02062   !$acc                       PSIQ2_lnd, PSIT2_lnd, &
L02063   !$acc                       PSIQ2_wat, PSIT2_wat, &
L02064   !$acc                       PSIQ2_ice, PSIT2_ice, &
L02065   !$acc                       QSFC, QSFC_lnd, QSFC_wat, QSFC_ice, &
L02066   !$acc                       QFLX, QFLX_lnd, QFLX_wat, QFLX_ice, &
L02067   !$acc                       HFLX, HFLX_lnd, HFLX_wat, HFLX_ice, &
L02068   !$acc                       QSFCMR_lnd, QSFCMR_wat, QSFCMR_ice, &
L02069   !$acc                       QV1D, WSPD, WSPDI, CPM, TH1D, &
L02070   !$acc                       THSK_lnd, THSK_wat, THSK_ice, &
L02071   !$acc                       ch_lnd, ch_wat, ch_ice, &
L02072   !$acc                       cm_lnd, cm_wat, cm_ice)
L02073    DO I=its,ite
L02074     if( flag_iter(i) ) then
L02075
L02076       IF (ISFFLX .LT. 1) THEN
L02077
L02078          QFX(i)  = 0.
L02079          HFX(i)  = 0.
L02080          HFLX(i) = 0.
L02081          FLHC(I) = 0.
L02082          FLQC(I) = 0.
L02083          LH(I)   = 0.
L02084          CHS(I)  = 0.
L02085          CH(I)   = 0.
L02086          CHS2(i) = 0.
L02087          CQS2(i) = 0.
L02088          ch_wat(I)= 0.
L02089          cm_wat(I)= 0.
L02090          ch_lnd(I)= 0.
L02091          cm_lnd(I)= 0.
L02092          ch_ice(I)= 0.
L02093          cm_ice(I)= 0.
L02094
L02095       ELSE
L02096
L02097         IF (dry(i)) THEN
L02098
L02099            !-----------------------------------------
L02100            ! CALCULATE THE EXCHANGE COEFFICIENTS FOR HEAT (FLHC)
L02101            ! AND MOISTURE (FLQC)
L02102            !-----------------------------------------
L02103            !tgs - should FLQC, FLHC be separate for dry, icy and wet?
L02104            FLQC(I)=RHO1D(I)*MAVAIL(I)*UST_lnd(I)*KARMAN/PSIQ_lnd(i)
L02105            FLHC(I)=RHO1D(I)*CPM(I)*UST_lnd(I)*KARMAN/PSIT_lnd(I)
L02106
L02107            IF (compute_flux) THEN
L02108               !---------------------------------
L02109               ! COMPUTE SURFACE MOISTURE FLUX:
L02110               !---------------------------------
L02111               QFX(I)=FLQC(I)*(QSFCMR_lnd(I)-QV1D(I))
L02112               !QFX(I)=FLQC(I)*(QSFC_lnd(I)-QV1D(I))
L02113               QFX(I)=MAX(QFX(I),-0.02_kind_phys)      !allows small neg QFX
L02114               LH(i)=XLV*QFX(i)
L02115               ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02116               QFLX_lnd(i)=QFX(i)/RHO1D(i)
L02117               QFLX(i)=QFLX_lnd(i)
L02118
L02119               !---------------------------------
L02120               ! COMPUTE SURFACE HEAT FLUX:
```

```
L02121              !-----------------------------------
L02122              !HFX(I)=FLHC(I)*(THSK_lnd(I)-TH1D(I))
L02123              HFX(I)=RHO1D(I)*CPM(I)*KARMAN*WSPD(i)/PSIX_lnd(I)*KARMAN/PSIT_lnd(I)*(THSK_lnd(I)-TH1D(i))
L02124              HFX(I)=MAX(HFX(I),-250._kind_phys)
L02125              ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02126              HFLX_lnd(I)=HFX(I)/(RHO1D(I)*cpm(I))
L02127              HFLX(I)=HFLX_lnd(I)
L02128           ENDIF
L02129
L02130           !TRANSFER COEFF FOR SOME LSMs:
L02131           !CHS(I)=UST(I)*KARMAN/(ALOG(KARMAN*UST(I)*ZA(I) &
L02132           !       /XKA+ZA(I)/ZL)-PSIH(I))
L02133
L02134           !tgs - should QSFC, CHS, CHS2 and CQS2 be separate for dry, icy and wet?
L02135           CHS(I)=UST_lnd(I)*KARMAN/PSIT_lnd(I)
L02136
L02137           !THESE ARE USED FOR 2-M DIAGNOSTICS ONLY
L02138           CQS2(I)=UST_lnd(I)*KARMAN/PSIQ2_lnd(i)
L02139           CHS2(I)=UST_lnd(I)*KARMAN/PSIT2_lnd(I)
L02140
L02141           QSFC(I)=QSFC_lnd(I)
L02142
L02143        ELSEIF (wet(i)) THEN
L02144
L02145           !-----------------------------------------
L02146           ! CALCULATE THE EXCHANGE COEFFICIENTS FOR HEAT (FLHC)
L02147           ! AND MOISTURE (FLQC)
L02148           !-----------------------------------------
L02149           FLQC(I)=RHO1D(I)*MAVAIL(I)*UST_wat(I)*KARMAN/PSIQ_wat(i)
L02150           FLHC(I)=RHO1D(I)*CPM(I)*UST_wat(I)*KARMAN/PSIT_wat(I)
L02151
L02152           IF (compute_flux) THEN
L02153              !-----------------------------------
L02154              ! COMPUTE SURFACE MOISTURE FLUX:
L02155              !-----------------------------------
L02156              QFX(I)=FLQC(I)*(QSFCMR_wat(I)-QV1D(I))
L02157              !QFX(I)=FLQC(I)*(QSFC_wat(I)-QV1D(I))
L02158              QFX(I)=MAX(QFX(I),-0.02_kind_phys)      !allows small neg QFX
L02159              LH(I)=XLV*QFX(I)
L02160              ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02161              QFLX_wat(i)=QFX(i)/RHO1D(i)
L02162              QFLX(i)=QFLX_wat(i)
L02163
L02164              !-----------------------------------
L02165              ! COMPUTE SURFACE HEAT FLUX:
L02166              !-----------------------------------
L02167              !HFX(I)=FLHC(I)*(THSK_wat(I)-TH1D(I))
L02168              HFX(I)=RHO1D(I)*CPM(I)*KARMAN*WSPD(i)/PSIX_wat(I)*KARMAN/PSIT_wat(I)*(THSK_wat(I)-TH1D(i))
L02169              IF ( PRESENT(ISFTCFLX) ) THEN
L02170                 IF ( ISFTCFLX.NE.0 ) THEN
L02171                    ! AHW: add dissipative heating term
L02172                    HFX(I)=HFX(I)+RHO1D(I)*USTM(I)*USTM(I)*WSPDI(I)
L02173                 ENDIF
L02174              ENDIF
L02175              ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02176              HFLX_wat(I)=HFX(I)/(RHO1D(I)*cpm(I))
L02177              HFLX(I)=HFLX_wat(I)
L02178           ENDIF
L02179
L02180           !TRANSFER COEFF FOR SOME LSMs:
L02181           !CHS(I)=UST(I)*KARMAN/(ALOG(KARMAN*UST(I)*ZA(I) &
L02182           !       /XKA+ZA(I)/ZL)-PSIH(I))
L02183           CHS(I)=UST_wat(I)*KARMAN/PSIT_wat(I)
L02184
L02185           !THESE ARE USED FOR 2-M DIAGNOSTICS ONLY
L02186           CQS2(I)=UST_wat(I)*KARMAN/PSIQ2_wat(i)
L02187           CHS2(I)=UST_wat(I)*KARMAN/PSIT2_wat(I)
L02188
L02189           QSFC(I)=QSFC_wat(I)
L02190
L02191        ELSEIF (icy(i)) THEN
L02192
L02193           !-----------------------------------------
L02194           ! CALCULATE THE EXCHANGE COEFFICIENTS FOR HEAT (FLHC)
L02195           ! AND MOISTURE (FLQC)
L02196           !-----------------------------------------
L02197           FLQC(I)=RHO1D(I)*MAVAIL(I)*UST_ice(I)*KARMAN/PSIQ_ice(i)
L02198           FLHC(I)=RHO1D(I)*CPM(I)*UST_ice(I)*KARMAN/PSIT_ice(I)
L02199
L02200           IF (compute_flux) THEN
L02201              !-----------------------------------
L02202              ! COMPUTE SURFACE MOISTURE FLUX:
L02203              !-----------------------------------
L02204              QFX(I)=FLQC(I)*(QSFCMR_ice(I)-QV1D(I))
L02205              !QFX(I)=FLQC(I)*(QSFC_ice(I)-QV1D(I))
L02206              QFX(I)=MAX(QFX(I),-0.02_kind_phys)      !allows small neg QFX
L02207              LH(I)=XLF*QFX(I)
L02208              ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02209              QFLX_ice(i)=QFX(i)/RHO1D(i)
L02210              QFLX(i)=QFLX_ice(i)
L02211
L02212              !-----------------------------------
L02213              ! COMPUTE SURFACE HEAT FLUX:
L02214              !-----------------------------------
L02215              !HFX(I)=FLHC(I)*(THSK_ice(I)-TH1D(I))
L02216              HFX(I)=RHO1D(I)*CPM(I)*KARMAN*WSPD(i)/PSIX_ice(I)*KARMAN/PSIT_ice(I)*(THSK_ice(I)-TH1D(i))
L02217              HFX(I)=MAX(HFX(I),-250._kind_phys)
L02218              ! BWG, 2020-06-17: Mod next 2 lines for fractional
L02219              HFLX_ice(I)=HFX(I)/(RHO1D(I)*cpm(I))
L02220              HFLX(I)=HFLX_ice(I)
L02221           ENDIF
L02222
L02223           !TRANSFER COEFF FOR SOME LSMs:
L02224           !CHS(I)=UST(I)*KARMAN/(ALOG(KARMAN*UST(I)*ZA(I) &
L02225           !       /XKA+ZA(I)/ZL)-PSIH(I))
L02226           CHS(I)=UST_ice(I)*KARMAN/PSIT_ice(I)
L02227
L02228           !THESE ARE USED FOR 2-M DIAGNOSTICS ONLY
```

```
L02229           CQS2(I)=UST_ice(I)*KARMAN/PSIQ2_ice(i)
L02230           CHS2(I)=UST_ice(I)*KARMAN/PSIT2_ice(I)
L02231
L02232           QSFC(I)=QSFC_ice(I)
L02233
L02234        ENDIF
L02235
L02236        IF (debug_code > 1) THEN
L02237           write(*,*)"QFX=",QFX(I),"FLQC=",FLQC(I)
L02238           if(icy(i))write(*,*)"ice, MAVAIL:",MAVAIL(I)," u*=",UST_ice(I)," psiq=",PSIQ_ice(i)
L02239           if(dry(i))write(*,*)"lnd, MAVAIL:",MAVAIL(I)," u*=",UST_lnd(I)," psiq=",PSIQ_lnd(i)
L02240           if(wet(i))write(*,*)"ocn, MAVAIL:",MAVAIL(I)," u*=",UST_wat(I)," psiq=",PSIQ_wat(i)
L02241        ENDIF
L02242
L02243        ! The exchange coefficient for cloud water is assumed to be the
L02244        ! same as that for heat. CH is multiplied by WSPD.
L02245        ch(i)=flhc(i)/( cpm(i)*RHO1D(i) )
L02246
L02247        !-----------------------------------------
L02248        !--- COMPUTE EXCHANGE COEFFICIENTS FOR FV3
L02249        !-----------------------------------------
L02250        IF (wet(i)) THEN
L02251           ch_wat(I)=(karman/psix_wat(I))*(karman/psit_wat(i))
L02252           cm_wat(I)=(karman/psix_wat(I))*(karman/psix_wat(I))
L02253        ENDIF
L02254        IF (dry(i)) THEN
L02255           ch_lnd(I)=(karman/psix_lnd(I))*(karman/psit_lnd(i))
L02256           cm_lnd(I)=(karman/psix_lnd(I))*(karman/psix_lnd(I))
L02257        ENDIF
L02258        IF (icy(i)) THEN
L02259           ch_ice(I)=(karman/psix_ice(I))*(karman/psit_ice(i))
L02260           cm_ice(I)=(karman/psix_ice(I))*(karman/psix_ice(I))
L02261        ENDIF
L02262
L02263     ENDIF !end ISFFLX option
L02264    endif ! flag_iter
L02265  ENDDO ! end i-loop
```

讲解：ISFFLX<1 时全零（理想化实验关标量通量保动量——头注提到的 isflux=0 选项）。三分支：①FLQC=ρ • MAVAIL • u • *κ/PSIQ*、*FLHC=ρ • CPM • u* • κ/PSIT——**MAVAIL 只乘在潜热侧**（土壤湿度可用度是陆面概念，水面恒 1）；②QFX=FLQC • (QSFCMR−QV1D)，下限 −0.02 允许微量负凝结；HFX 的展开式是 ρ • CPM • (κWSPD/PSIX) • (κ/PSIT) • Δθ——即 CD • CH • WSPD • Δθ 的完整形式；水面 isftcflx≠0 加**耗散加热** ρ • USTM² • WSPDI（高速风下浪破碎耗散加热海面，台风试验开关）；冰面潜热用 XLF（升华/冻结）而水面用 XLV；③CHS=u* • κ/PSIT 给 LSM 作热交换系数（Noah 在此接手真实 2 m 诊断），CHS2/CQS2 是 2 m 版；④FV3 系数 ch=(κ/PSIX)(κ/PSIT)、cm=(κ/PSIX)²——无量纲交换系数（即 CD/CH 的 κ 归一形式），乘 WSPD 才是 WRF 意义的 CH。

S10 近地面诊断：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2267-L2388
L02267  IF (compute_diag) then
L02268     !$acc parallel loop present(flag_iter, dry, wet, icy, &
L02269     !$acc                       ZA, ZA2, T2, TH2, TH1D, Q2, QV1D, PSFCPA, &
L02270     !$acc                       THSK_lnd, THSK_wat, THSK_ice, &
L02271     !$acc                       QSFC_lnd, QSFC_wat, QSFC_ice, &
L02272     !$acc                       U10, V10, U1D, V1D, U1D2, V1D2, &
L02273     !$acc                       ZNTstoch_lnd, ZNTstoch_lnd, ZNTstoch_ice, &
L02274     !$acc                       PSIX_lnd, PSIX_wat, PSIX_ice, &
L02275     !$acc                       PSIX10_lnd, PSIX10_wat, PSIX10_ice, &
L02276     !$acc                       PSIT2_lnd, PSIT2_wat, PSIT2_ice, &
L02277     !$acc                       PSIT_lnd, PSIT_wat, PSIT_ice, &
L02278     !$acc                       PSIQ2_lnd, PSIQ2_wat, PSIQ2_ice, &
L02279     !$acc                       PSIQ_lnd, PSIQ_wat, PSIQ_ice)
L02280     DO I=its,ite
L02281       if( flag_iter(i) ) then
L02282        !-----------------------------------------------------
L02283        !COMPUTE DIAGNOSTICS
L02284        !-----------------------------------------------------
L02285        !COMPUTE 10 M WNDS
L02286        !-----------------------------------------------------
L02287        ! If the lowest model level is close to 10-m, use it
L02288        ! instead of the flux-based diagnostic formula.
L02289        if (ZA(i) .le. 7.0) then
L02290           ! high vertical resolution
L02291           if(ZA2(i) .gt. 7.0 .and. ZA2(i) .lt. 13.0) then
L02292              !use 2nd model level
L02293              U10(I)=U1D2(I)
L02294              V10(I)=V1D2(I)
L02295           else
L02296              IF (dry(i)) THEN
L02297                !U10(I)=U1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02298                !V10(I)=V1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02299                !use neutral-log:
L02300                U10(I)=U1D(I)*log(10./ZNTstoch_lnd(I))/log(ZA(I)/ZNTstoch_lnd(I))
L02301                V10(I)=V1D(I)*log(10./ZNTstoch_lnd(I))/log(ZA(I)/ZNTstoch_lnd(I))
L02302              ELSEIF (wet(i)) THEN
L02303                U10(I)=U1D(I)*log(10./ZNTstoch_wat(I))/log(ZA(I)/ZNTstoch_wat(I))
L02304                V10(I)=V1D(I)*log(10./ZNTstoch_wat(I))/log(ZA(I)/ZNTstoch_wat(I))
L02305              ELSEIF (icy(i)) THEN
L02306                U10(I)=U1D(I)*log(10./ZNTstoch_ice(I))/log(ZA(I)/ZNTstoch_ice(I))
L02307                V10(I)=V1D(I)*log(10./ZNTstoch_ice(I))/log(ZA(I)/ZNTstoch_ice(I))
L02308              ENDIF
L02309           endif
L02310        elseif (ZA(i) .gt. 7.0 .and. ZA(i) .lt. 13.0) then
L02311           !moderate vertical resolution
L02312           IF (dry(i)) THEN
L02313              !U10(I)=U1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02314              !V10(I)=V1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02315              !use neutral-log:
```

```
L02316              U10(I)=U1D(I)*log(10./ZNTstoch_lnd(I))/log(ZA(I)/ZNTstoch_lnd(I))
L02317              V10(I)=V1D(I)*log(10./ZNTstoch_lnd(I))/log(ZA(I)/ZNTstoch_lnd(I))
L02318           ELSEIF (wet(i)) THEN
L02319              U10(I)=U1D(I)*log(10./ZNTstoch_wat(I))/log(ZA(I)/ZNTstoch_wat(I))
L02320              V10(I)=V1D(I)*log(10./ZNTstoch_wat(I))/log(ZA(I)/ZNTstoch_wat(I))
L02321           ELSEIF (icy(i)) THEN
L02322              U10(I)=U1D(I)*log(10./ZNTstoch_ice(I))/log(ZA(I)/ZNTstoch_ice(I))
L02323              V10(I)=V1D(I)*log(10./ZNTstoch_ice(I))/log(ZA(I)/ZNTstoch_ice(I))
L02324           ENDIF
L02325        else
L02326           ! very coarse vertical resolution
L02327           IF (dry(i)) THEN
L02328              U10(I)=U1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02329              V10(I)=V1D(I)*PSIX10_lnd(I)/PSIX_lnd(I)
L02330           ELSEIF (wet(i)) THEN
L02331              U10(I)=U1D(I)*PSIX10_wat(I)/PSIX_wat(I)
L02332              V10(I)=V1D(I)*PSIX10_wat(I)/PSIX_wat(I)
L02333           ELSEIF (icy(i)) THEN
L02334              U10(I)=U1D(I)*PSIX10_ice(I)/PSIX_ice(I)
L02335              V10(I)=V1D(I)*PSIX10_ice(I)/PSIX_ice(I)
L02336           ENDIF
L02337        endif
L02338
L02339        !-----------------------------------------------------
L02340        !COMPUTE 2m T, TH, AND Q
L02341        !THESE WILL BE OVERWRITTEN FOR LAND POINTS IN THE LSM
L02342        !-----------------------------------------------------
L02343        IF (dry(i)) THEN
L02344           DTG=TH1D(I)-THSK_lnd(I)
L02345           TH2(I)=THSK_lnd(I)+DTG*PSIT2_lnd(I)/PSIT_lnd(I)
L02346           !***  BE CERTAIN THAT THE 2-M THETA IS BRACKETED BY
L02347           !***  THE VALUES AT THE SURFACE AND LOWEST MODEL LEVEL.
L02348           IF ((TH1D(I)>THSK_lnd(I) .AND. (TH2(I)<THSK_lnd(I) .OR. TH2(I)>TH1D(I))) .OR. &
L02349              (TH1D(I)<THSK_lnd(I) .AND. (TH2(I)>THSK_lnd(I) .OR. TH2(I)<TH1D(I)))) THEN
L02350              TH2(I)=THSK_lnd(I) + 2.*(TH1D(I)-THSK_lnd(I))/ZA(I)
L02351           ENDIF
L02352           T2(I)=TH2(I)*(PSFCPA(I)/100000.)**ROVCP
L02353
L02354           Q2(I)=QSFC_lnd(I)+(QV1D(I)-QSFC_lnd(I))*PSIQ2_lnd(i)/PSIQ_lnd(i)
L02355           Q2(I)= MAX(Q2(I), MIN(QSFC_lnd(I), QV1D(I)))
L02356           Q2(I)= MIN(Q2(I), 1.05*QV1D(I))
L02357        ELSEIF (wet(i)) THEN
L02358           DTG=TH1D(I)-THSK_wat(I)
L02359           TH2(I)=THSK_wat(I)+DTG*PSIT2_wat(I)/PSIT_wat(I)
L02360           !***  BE CERTAIN THAT THE 2-M THETA IS BRACKETED BY
L02361           !***  THE VALUES AT THE SURFACE AND LOWEST MODEL LEVEL.
L02362           IF ((TH1D(I)>THSK_wat(I) .AND. (TH2(I)<THSK_wat(I) .OR. TH2(I)>TH1D(I))) .OR. &
L02363              (TH1D(I)<THSK_wat(I) .AND. (TH2(I)>THSK_wat(I) .OR. TH2(I)<TH1D(I)))) THEN
L02364              TH2(I)=THSK_wat(I) + 2.*(TH1D(I)-THSK_wat(I))/ZA(I)
L02365           ENDIF
L02366           T2(I)=TH2(I)*(PSFCPA(I)/100000.)**ROVCP
L02367
L02368           Q2(I)=QSFC_wat(I)+(QV1D(I)-QSFC_wat(I))*PSIQ2_wat(i)/PSIQ_wat(i)
L02369           Q2(I)= MAX(Q2(I), MIN(QSFC_wat(I), QV1D(I)))
L02370           Q2(I)= MIN(Q2(I), 1.05*QV1D(I))
L02371        ELSEIF (icy(i)) THEN
L02372           DTG=TH1D(I)-THSK_ice(I)
L02373           TH2(I)=THSK_ice(I)+DTG*PSIT2_ice(I)/PSIT_ice(I)
L02374           !***  BE CERTAIN THAT THE 2-M THETA IS BRACKETED BY
L02375           !***  THE VALUES AT THE SURFACE AND LOWEST MODEL LEVEL.
L02376           IF ((TH1D(I)>THSK_ice(I) .AND. (TH2(I)<THSK_ice(I) .OR. TH2(I)>TH1D(I))) .OR. &
L02377              (TH1D(I)<THSK_ice(I) .AND. (TH2(I)>THSK_ice(I) .OR. TH2(I)<TH1D(I)))) THEN
L02378              TH2(I)=THSK_ice(I) + 2.*(TH1D(I)-THSK_ice(I))/ZA(I)
L02379           ENDIF
L02380           T2(I)=TH2(I)*(PSFCPA(I)/100000.)**ROVCP
L02381
L02382           Q2(I)=QSFC_ice(I)+(QV1D(I)-QSFC_ice(I))*PSIQ2_ice(i)/PSIQ_ice(i)
L02383           Q2(I)= MAX(Q2(I), MIN(QSFC_ice(I), QV1D(I)))
L02384           Q2(I)= MIN(Q2(I), 1.05*QV1D(I))
L02385        ENDIF
L02386       endif ! flag_iter
L02387     ENDDO
L02388  ENDIF ! end compute_diag
```

讲解：10 m 风三分支按最低层高 ZA 与层厚自适应：高分辨率（ZA≤7 m 且 ZA2∈(7,13) m）直接用第二模式层风——比诊断公式更真；中等分辨率用中性对数外插 log(10/z0)/log(ZA/z0)；粗分辨率（ZA>13 m）才用通量比 PSIX10/PSIX（含稳定度修正）。2 m 位温 = THSK + Δθ • PSIT2/PSIT，带**括位保护**：结果必须夹在表面值与最低层值之间，越界退化为线性插值 THSK+2Δθ/ZA；Q2 同结构并钳位 [min(QSFC,QV1D), 1.05 • QV1D]。陆地点的 T2/Q2 注释明言将被 LSM 覆写（Noah 有自己的 2 m 迭代）。

4.4 z/L 迭代器与初猜

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3661-L3831
L03661  !>\ingroup mynn_sfc
L03662        REAL(kind_phys) function zolri(ri,za,z0,zt,zol1,psi_opt)
L03663
L03664        !> This iterative algorithm was taken from the revised surface layer
L03665        !! scheme in WRF-ARW, written by Pedro Jimenez and Jimy Dudhia and
L03666        !! summarized in Jimenez et al. (2012, MWR). This function was adapted
L03667        !! to input the thermal roughness length, zt, (as well as z0) and use initial
L03668        !! estimate of z/L.
L03669
L03670        IMPLICIT NONE
L03671        REAL(kind_phys), INTENT(IN) :: ri,za,z0,zt,zol1
L03672        INTEGER, INTENT(IN) :: psi_opt
L03673        REAL(kind_phys) :: x1,x2,fx1,fx2
L03674        INTEGER :: n
L03675        INTEGER, PARAMETER :: nmax = 20
```

```
L03676        !REAL(kind_phys), DIMENSION(nmax):: zLhux
L03677
L03678        if (ri.lt.0.)then
L03679           x1=zol1 - 0.02  !-5.
L03680           x2=0.
L03681        else
L03682           x1=0.
L03683           x2=zol1 + 0.02 !5.
L03684        endif
L03685
L03686        n=1
L03687        fx1=zolri2(x1,ri,za,z0,zt,psi_opt)
L03688        fx2=zolri2(x2,ri,za,z0,zt,psi_opt)
L03689
L03690        Do While (abs(x1 - x2) > 0.01 .and. n < nmax)
L03691          if(abs(fx2).lt.abs(fx1))then
L03692            x1=x1-fx1/(fx2-fx1)*(x2-x1)
L03693            fx1=zolri2(x1,ri,za,z0,zt,psi_opt)
L03694            zolri=x1
L03695          else
L03696            x2=x2-fx2/(fx2-fx1)*(x2-x1)
L03697            fx2=zolri2(x2,ri,za,z0,zt,psi_opt)
L03698            zolri=x2
L03699          endif
L03700          n=n+1
L03701          !print*," n=",n," x1=",x1," x2=",x2
L03702          !zLhux(n)=zolri
L03703        enddo
L03704
L03705        if (n==nmax .and. abs(x1 - x2) >= 0.01) then
L03706           !if convergence fails, use approximate values:
L03707           CALL Li_etal_2010(zolri, ri, za/z0, z0/zt)
L03708           !zLhux(n)=zolri
L03709           !print*,"iter FAIL, n=",n," Ri=",ri," z0=",z0
L03710        else
L03711           !print*,"SUCCESS,n=",n," Ri=",ri," z0=",z0
L03712        endif
L03713
L03714        end function
L03715  !---------------------------------------------------------------------
L03716        REAL(kind_phys) function zolri2(zol2,ri2,za,z0,zt,psi_opt)
L03717
L03718        ! INPUT: ==================================
L03719        ! zol2 - estimated z/L
L03720        ! ri2  - calculated bulk Richardson number
L03721        ! za   - 1/2 depth of first model layer
L03722        ! z0   - aerodynamic roughness length
L03723        ! zt   - thermal roughness length
L03724        ! OUTPUT: =================================
L03725        ! zolri2 - delta Ri
L03726
L03727        IMPLICIT NONE
L03728        INTEGER, INTENT(IN)            :: psi_opt
L03729        REAL(kind_phys), INTENT(IN)    :: ri2,za,z0,zt
L03730        REAL(kind_phys), INTENT(INOUT) :: zol2
L03731        REAL(kind_phys) :: zol20,zol3,psim1,psih1,psix2,psit2,zolt
L03732
L03733        if(zol2*ri2 .lt. 0.)zol2=0.  ! limit zol2 - must be same sign as ri2
L03734
L03735        zol20=zol2*z0/za ! z0/L
L03736        zol3=zol2+zol20  ! (z+z0)/L
L03737        zolt=zol2*zt/za  ! zt/L
L03738
L03739        if (ri2.lt.0) then
L03740           !psix2=log((za+z0)/z0)-(psim_unstable(zol3)-psim_unstable(zol20))
L03741           !psit2=log((za+zt)/zt)-(psih_unstable(zol3)-psih_unstable(zol20))
L03742           psit2=MAX(log((za+z0)/zt)-(psih_unstable(zol3,psi_opt)-psih_unstable(zolt,psi_opt)), 1.0_kind_phys)
L03743           psix2=MAX(log((za+z0)/z0)-(psim_unstable(zol3,psi_opt)-psim_unstable(zol20,psi_opt)), 1.0_kind_phys)
L03744        else
L03745           !psix2=log((za+z0)/z0)-(psim_stable(zol3)-psim_stable(zol20))
L03746           !psit2=log((za+zt)/zt)-(psih_stable(zol3)-psih_stable(zol20))
L03747           psit2=MAX(log((za+z0)/zt)-(psih_stable(zol3,psi_opt)-psih_stable(zolt,psi_opt)), 1.0_kind_phys)
L03748           psix2=MAX(log((za+z0)/z0)-(psim_stable(zol3,psi_opt)-psim_stable(zol20,psi_opt)),1.0_kind_phys)
L03749        endif
L03750
L03751        zolri2=zol2*psit2/psix2**2 - ri2
L03752        !print*,"  target ri=",ri2," est ri=",zol2*psit2/psix2**2
L03753
L03754        end function
L03755  !=====================================================================
L03756
L03757        REAL(kind_phys) function zolrib(ri,za,z0,zt,logz0,logzt,zol1,psi_opt)
L03758
L03759        !$acc routine seq
L03760        ! This iterative algorithm to compute z/L from bulk-Ri
L03761
L03762        IMPLICIT NONE
L03763        REAL(kind_phys), INTENT(IN) :: ri,za,z0,zt,logz0,logzt
L03764        INTEGER, INTENT(IN)         :: psi_opt
L03765        REAL(kind_phys), INTENT(INOUT) :: zol1
L03766        REAL(kind_phys) :: zol20,zol3,zolt,zolold
L03767        INTEGER :: n
L03768        INTEGER, PARAMETER :: nmax = 20
L03769        !REAL(kind_phys), DIMENSION(nmax):: zLhux
L03770        REAL(kind_phys) :: psit2,psix2
L03771
L03772        !print*,"+++++++INCOMING: z/L=",zol1," ri=",ri
L03773        if (zol1*ri .lt. 0.) THEN
L03774           !print*,"begin: WRONG QUADRANTS: z/L=",zol1," ri=",ri
L03775           zol1=0.
L03776        endif
L03777
L03778        if (ri .lt. 0.) then
L03779          zolold=-99999.
L03780          zolrib=-66666.
L03781        else
L03782          zolold=99999.
L03783          zolrib=66666.
L03784        endif
```

```
L03785        n=1
L03786
L03787        DO While (abs(zolold - zolrib) > 0.01 .and. n < nmax)
L03788
L03789          if(n==1)then
L03790            zolold=zol1
L03791          else
L03792            zolold=zolrib
L03793          endif
L03794          zol20=zolold*z0/za ! z0/L
L03795          zol3=zolold+zol20  ! (z+z0)/L
L03796          zolt=zolold*zt/za  ! zt/L
L03797          !print*,"z0/L=",zol20," (z+z0)/L=",zol3," zt/L=",zolt
L03798          if (ri.lt.0) then
L03799             !psit2=log((za+zt)/zt)-(psih_unstable(zol3)-psih_unstable(zol20))
L03800             !psit2=log((za+z0)/zt)-(psih_unstable(zol3)-psih_unstable(zol20))
L03801             psit2=MAX(logzt-(psih_unstable(zol3,psi_opt)-psih_unstable(zolt,psi_opt)), 1.0_kind_phys)
L03802             psix2=MAX(logz0-(psim_unstable(zol3,psi_opt)-psim_unstable(zol20,psi_opt)), 1.0_kind_phys)
L03803          else
L03804             !psit2=log((za+zt)/zt)-(psih_stable(zol3)-psih_stable(zol20))
L03805             !psit2=log((za+z0)/zt)-(psih_stable(zol3)-psih_stable(zol20))
L03806             psit2=MAX(logzt-(psih_stable(zol3,psi_opt)-psih_stable(zolt,psi_opt)), 1.0_kind_phys)
L03807             psix2=MAX(logz0-(psim_stable(zol3,psi_opt)-psim_stable(zol20,psi_opt)), 1.0_kind_phys)
L03808          endif
L03809          !print*,"n=",n," psit2=",psit2," psix2=",psix2
L03810          zolrib=ri*psix2**2/psit2
L03811          !zLhux(n)=zolrib
L03812          n=n+1
L03813        enddo
L03814
L03815        if (n==nmax .and. abs(zolold - zolrib) > 0.01 ) then
L03816           !print*,"iter FAIL, n=",n," Ri=",ri," z/L=",zolri
L03817           !if convergence fails, use approximate values:
L03818           CALL Li_etal_2010(zolrib, ri, za/z0, z0/zt)
L03819           !zLhux(n)=zolrib
L03820           !print*,"FAILED, n=",n," Ri=",ri," z0=",z0
L03821           !print*,"z/L=",zLhux(1:nmax)
L03822        else
L03823           !if(zolrib*ri .lt. 0.) THEN
L03824           !   !print*,"end: WRONG QUADRANTS: z/L=",zolrib," ri=",ri
L03825           !   !CALL Li_etal_2010(zolrib, ri, za/z0, z0/zt)
L03826           !endif
L03827           !print*,"SUCCESS,n=",n," Ri=",ri," z0=",z0
L03828        endif
L03829
L03830        end function
L03831  !=================================================================
```

讲解：三个函数，两套算法：①zolri+zolri2——Pedro Jimenez（sfclayrev）的**割线/二分混合**：对 f(z/L)=z/L • PSIT/PSIX²−Ri 求根，失败回退 Li 2010；当前主流程注释掉了；②zolrib——实际在用的**不动点迭代**：z/L ← Ri • PSIX²/PSIT，20 步上限，初猜同号钳位，失败回退 Li 2010。初猜与回退共用 Li_etal_2010（见 4.5 组四）——一个把 rb、z/z0、z0/zt 作为自变量的显式有理拟合，保证任何稳定度下都有合法 z/L 起点。改造提示：改收敛判据（0.01）或步数（nmax=20）只影响精度/速度不影响物理；把 zolrib 换成 zolri 是纯数值试验，两者物理等价。

4.5 相似性函数与粗糙度参数化库

组一：陆面 zt/zq 参数化（iz0tlnd/sf_mynn_sfcflux_land 分派目标）：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2575-L2635    (subroutine zilitinkevich_1995)
L02575    SUBROUTINE zilitinkevich_1995(Z_0,Zt,Zq,restar,ustar,KARMAN,&
L02576          & landsea,IZ0TLND2,spp_sfc,rstoch)
L02577
L02578         !$acc routine seq
L02579         IMPLICIT NONE
L02580         REAL(kind_phys), INTENT(IN)       :: Z_0,restar,ustar,KARMAN,landsea
L02581         INTEGER,  OPTIONAL,   INTENT(IN)  :: IZ0TLND2
L02582         REAL(kind_phys), INTENT(OUT)      :: Zt,Zq
L02583         REAL(kind_phys) :: CZIL       !=0.100 in Chen et al. (1997)
L02584                                       !=0.075 in Zilitinkevich (1995)
L02585                                       !=0.500 in Lemone et al. (2008)
L02586         INTEGER,  INTENT(IN)  ::    spp_sfc
L02587         REAL(kind_phys), INTENT(IN)  :: rstoch
L02588
L02589
L02590         IF (landsea-1.5 .GT. 0) THEN    !WATER
L02591
L02592            !THIS IS BASED ON Zilitinkevich, Grachev, and Fairall (2001;
L02593            !Their equations 15 and 16).
L02594            IF (restar .LT. 0.1) THEN
L02595               Zt = Z_0*EXP(KARMAN*2.0)
L02596               Zt = MIN( Zt, 6.0e-5_kind_phys)
L02597               Zt = MAX( Zt, 2.0e-9_kind_phys)
L02598               Zq = Z_0*EXP(KARMAN*3.0)
L02599               Zq = MIN( Zq, 6.0e-5_kind_phys)
L02600               Zq = MAX( Zq, 2.0e-9_kind_phys)
L02601            ELSE
L02602               Zt = Z_0*EXP(-KARMAN*(4.0*SQRT(restar)-3.2))
L02603               Zt = MIN( Zt, 6.0e-5_kind_phys)
L02604               Zt = MAX( Zt, 2.0e-9_kind_phys)
L02605               Zq = Z_0*EXP(-KARMAN*(4.0*SQRT(restar)-4.2))
L02606               Zq = MIN( Zt, 6.0e-5_kind_phys)
L02607               Zq = MAX( Zt, 2.0e-9_kind_phys)
L02608            ENDIF
L02609
L02610         ELSE                             !LAND
L02611
L02612            !Option to modify CZIL according to Chen & Zhang, 2009
```

```
L02613            IF ( IZ0TLND2 .EQ. 1 ) THEN
L02614               CZIL = 10.0 ** ( -0.40 * ( Z_0 / 0.07 ) )
L02615            ELSE
L02616               CZIL = 0.095 !0.075 !0.10
L02617            END IF
L02618
L02619            Zt = Z_0*EXP(-KARMAN*CZIL*SQRT(restar))
L02620            Zt = MIN( Zt, 0.75*Z_0)
L02621
L02622            Zq = Z_0*EXP(-KARMAN*CZIL*SQRT(restar))
L02623            Zq = MIN( Zq, 0.75*Z_0)
L02624
L02625   ! stochastically perturb thermal and moisture roughness length.
L02626   ! currently set to half the amplitude:
L02627            if (spp_sfc==1) then
L02628               Zt = Zt + Zt * 0.5 * rstoch
L02629               Zt = MAX(Zt, 0.0001_kind_phys)
L02630               Zq = Zt
L02631            endif
L02632
L02633         ENDIF
L02634
L02635      END SUBROUTINE zilitinkevich_1995
```

讲解：默认式（选项 0/1）。Zilitinkevich 1995：zt=z0 • exp(−κ • Cz • Re_r^0.45)，Cz 选项 0 取常数 0.095、选项 1 按 Chen–Zhang 2008 随植被/粗糙度变——物理上 zt 远小于 z0（标量输运比动量更受分子层控制）。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2868-L2898    (subroutine Yang_2008)
L02868         SUBROUTINE Yang_2008(Z_0,Zt,Zq,ustar,tstar,qst,Ren,visc)
L02869
L02870         !$acc routine seq
L02871         IMPLICIT NONE
L02872         REAL(kind_phys), INTENT(IN)  :: Z_0, Ren, ustar, tstar, qst, visc
L02873         REAL(kind_phys) ::      ht,     &! roughness height at critical Reynolds number
L02874                                 tstar2, &! bounded T*, forced to be non-positive
L02875                                 qstar2, &! bounded q*, forced to be non-positive
L02876                                 Z_02,   &! bounded Z_0 for variable Renc2 calc
L02877                                 Renc2    ! variable Renc, function of Z_0
L02878         REAL(kind_phys), INTENT(OUT) :: Zt,Zq
L02879         REAL(kind_phys), PARAMETER   :: Renc=300., & !old constant Renc
L02880                                         beta=1.5,  & !important for diurnal variation
L02881                                         m=170.,    & !slope for Renc2 function
L02882                                         b=691.       !y-intercept for Renc2 function
L02883
L02884         Z_02 = MIN(Z_0,0.5_kind_phys)
L02885         Z_02 = MAX(Z_02,0.04_kind_phys)
L02886         Renc2= b + m*log(Z_02)
L02887         ht     = Renc2*visc/MAX(ustar,0.01_kind_phys)
L02888         tstar2 = MIN(tstar, 0.0_kind_phys)
L02889         qstar2 = MIN(qst,0.0_kind_phys)
L02890
L02891         Zt     = ht * EXP(-beta*(ustar**0.5)*(ABS(tstar2)**1.0))
L02892         Zq     = ht * EXP(-beta*(ustar**0.5)*(ABS(qstar2)**1.0))
L02893         !Zq     = Zt
L02894
L02895         Zt = MIN(Zt, Z_0/2.0)
L02896         Zq = MIN(Zq, Z_0/2.0)
L02897
L02898      END SUBROUTINE Yang_2008
```

讲解：Yang 2002/2008 修正式（选项 2），把 zt 与 u* • z0/VISC（restar）显式挂钩并考虑空气密度效应；注意 S6 分派处的警示——走此选项时 qstar 尚未正确赋值，代码直接报 Logic error（CCPP 化后加的保护）。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2747-L2769    (subroutine garratt_1992)
L02747      SUBROUTINE garratt_1992(Zt,Zq,Z_0,Ren,landsea)
L02748         !$acc routine seq
L02749         IMPLICIT NONE
L02750         REAL(kind_phys), INTENT(IN)  :: Ren, Z_0,landsea
L02751         REAL(kind_phys), INTENT(OUT) :: Zt,Zq
L02752         REAL(kind_phys)              :: Rq
L02753         REAL(kind_phys), PARAMETER   :: e=2.71828183
L02754
L02755         IF (landsea-1.5 .GT. 0) THEN    !WATER
L02756
L02757            Zt = Z_0*EXP(2.0 - (2.48*(Ren**0.25)))
L02758            Zq = Z_0*EXP(2.0 - (2.28*(Ren**0.25)))
L02759
L02760            Zq = MIN( Zq, 5.5e-5_kind_phys)
L02761            Zq = MAX( Zq, 2.0e-9_kind_phys)
L02762            Zt = MIN( Zt, 5.5e-5_kind_phys)
L02763            Zt = MAX( Zt, 2.0e-9_kind_phys) !same lower limit as ECMWF
L02764         ELSE                               !LAND
L02765            Zq = Z_0/(e**2.)      !taken from Garratt (1980,1992)
L02766            Zt = Zq
L02767         ENDIF
L02768
L02769      END SUBROUTINE garratt_1992
```

讲解：最简式（选项 3）：zt=z0/7.4、zq=z0/factor。同一段代码被陆面（factor=1.0）与水面（isftcflx=2，factor=2.0）共用——factor 实参区分两处调用。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2902-L2959    (subroutine GFS_z0_lnd)
L02902      SUBROUTINE GFS_z0_lnd(z0max,shdmax,z1,vegtype,ivegsrc,z0pert)
L02903
L02904          !$acc routine seq
L02905          REAL(kind_phys), INTENT(OUT)  :: z0max
L02906          REAL(kind_phys), INTENT(IN)   :: shdmax,z1,z0pert
L02907          INTEGER, INTENT(IN)           :: vegtype,ivegsrc
L02908          REAL(kind_phys)               :: tem1, tem2
L02909
L02910  !            z0max = max(1.0e-6, min(0.01 * z0max, z1))
L02911  !already converted into meters in the wrapper
L02912              z0max = max(1.0e-6_kind_phys, min(z0max, z1))
L02913  !** xubin's new z0  over land
L02914              tem1  = 1.0 - shdmax
L02915              tem2  = tem1 * tem1
L02916              tem1  = 1.0  - tem2
L02917
L02918              if( ivegsrc == 1 ) then
L02919
L02920                if (vegtype == 10) then
L02921                  z0max = exp( tem2*log01 + tem1*log07 )
L02922                elseif (vegtype == 6) then
L02923                  z0max = exp( tem2*log01 + tem1*log05 )
L02924                elseif (vegtype == 7) then
L02925  !               z0max = exp( tem2*log01 + tem1*log01 )
L02926                  z0max = 0.01
L02927                elseif (vegtype == 16) then
L02928  !               z0max = exp( tem2*log01 + tem1*log01 )
L02929                  z0max = 0.01
L02930                else
L02931                  z0max = exp( tem2*log01 + tem1*log(z0max) )
L02932                endif
L02933
L02934              elseif (ivegsrc == 2 ) then
L02935
L02936                if (vegtype == 7) then
L02937                  z0max = exp( tem2*log01 + tem1*log07 )
L02938                elseif (vegtype == 8) then
L02939                  z0max = exp( tem2*log01 + tem1*log05 )
L02940                elseif (vegtype == 9) then
L02941  !               z0max = exp( tem2*log01 + tem1*log01 )
L02942                  z0max = 0.01
L02943                elseif (vegtype == 11) then
L02944  !               z0max = exp( tem2*log01 + tem1*log01 )
L02945                  z0max = 0.01
L02946                else
L02947                  z0max = exp( tem2*log01 + tem1*log(z0max) )
L02948                endif
L02949
L02950              endif
L02951
L02952  ! mg, sfc-perts: add surface perturbations to z0max over land
L02953              if (z0pert /= 0.0 ) then
L02954                z0max = z0max * (10.**z0pert)
L02955              endif
L02956
L02957              z0max = max(z0max, 1.0e-6_kind_phys)
L02958
L02959      END SUBROUTINE GFS_z0_lnd
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2963-L2989    (subroutine GFS_zt_lnd)
L02963      SUBROUTINE GFS_zt_lnd(ztmax,z0max,sigmaf,ztpert,ustar_lnd)
L02964
L02965          !$acc routine seq
L02966          REAL(kind_phys), INTENT(OUT)  :: ztmax
L02967          REAL(kind_phys), INTENT(IN)   :: z0max,sigmaf,ztpert,ustar_lnd
L02968          REAL(kind_phys)               :: czilc, tem1, tem2
L02969          REAL(kind_phys), PARAMETER    :: ca = 0.4
L02970
L02971  !           czilc = 10.0 ** (- (0.40/0.07) * z0) ! fei's canopy height dependance of czil
L02972             czilc = 0.8
L02973
L02974             tem1  = 1.0 - sigmaf
L02975             ztmax = z0max*exp( - tem1*tem1                &
L02976      &                     * czilc*ca*sqrt(ustar_lnd*(0.01/1.5e-05)))
L02977  !
L02978  !           czilc = 10.0 ** (- 4. * z0max) ! Trier et al. (2011, WAF)
L02979  !           ztmax = z0max * exp( - czilc * ca      &
L02980  !     &           * 258.2 * sqrt(ustar_lnd*z0max) )
L02981
L02982
L02983  ! mg, sfc-perts: add surface perturbations to ztmax/z0max ratio over land
L02984              if (ztpert /= 0.0) then
L02985                ztmax = ztmax * (10.**ztpert)
L02986              endif
L02987              ztmax = max(ztmax, 1.0e-6_kind_phys)
L02988
L02989      END SUBROUTINE GFS_zt_lnd
```

讲解：GFS 对照组（选项 4）：z0 从植被类型查表（ivegsrc 两套分类）并按 shdmax（最大植被覆盖度）与 z0pert（随机扰动）调整；zt 带 sigmaf（植被占比）权重与 ztpert。这是 NCEP sfc_diff.f 的移植，物理与 MYNN 主线无关，供对比试验。

组二：水面 z0 参数化（isftcflx/sf_mynn_sfcflux_water 分派目标）：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2697-L2713    (subroutine charnock_1955)
L02697      SUBROUTINE charnock_1955(Z_0,ustar,wsp10,visc,zu)
L02698          !$acc routine seq
L02699          IMPLICIT NONE
L02700          REAL(kind_phys), INTENT(IN)  :: ustar, visc, wsp10, zu
L02701          REAL(kind_phys), INTENT(OUT) :: Z_0
L02702          REAL(kind_phys), PARAMETER   :: CZO2=0.011
```

```
L02703         REAL(kind_phys)              :: CZC    ! variable charnock "constant"
L02704         REAL(kind_phys)              :: wsp10m ! logarithmically calculated 10 m
L02705
L02706         wsp10m = wsp10*log(10./1e-4)/log(zu/1e-4)
L02707         CZC = CZO2 + 0.007*MIN(MAX((wsp10m-10.)/8., 0._kind_phys), 1.0_kind_phys)
L02708
L02709         Z_0 = CZC*ustar*ustar*g_inv + (0.11*visc/MAX(ustar,0.05_kind_phys))
L02710         Z_0 = MAX( Z_0, 1.27e-7_kind_phys)  !These max/mins were suggested by
L02711         Z_0 = MIN( Z_0, 2.85e-3_kind_phys)  !Davis et al. (2008)
L02712
L02713     END SUBROUTINE charnock_1955
```

讲解：COARE 3.0 的 z0——Charnock 关系 z0=α • u²/g 加黏性光滑流修正项 *0.11 • VISC/u*；COARE_OPT 编译期常数选 3.0/3.5 两套系数。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2720-L2738    (subroutine edson_etal_2013)
L02720     SUBROUTINE edson_etal_2013(Z_0,ustar,wsp10,visc,zu)
L02721         !$acc routine seq
L02722         IMPLICIT NONE
L02723         REAL(kind_phys), INTENT(IN)  :: ustar, visc, wsp10, zu
L02724         REAL(kind_phys), INTENT(OUT) :: Z_0
L02725         REAL(kind_phys), PARAMETER   :: m=0.0017, b=-0.005
L02726         REAL(kind_phys)              :: CZC    ! variable charnock "constant"
L02727         REAL(kind_phys)              :: wsp10m ! logarithmically calculated 10 m
L02728
L02729         wsp10m = wsp10*log(10/1e-4)/log(zu/1e-4)
L02730         wsp10m = MIN(19._kind_phys, wsp10m)
L02731         CZC    = m*wsp10m + b
L02732         CZC    = MAX(CZC, 0.0_kind_phys)
L02733
L02734         Z_0 = CZC*ustar*ustar*g_inv + (0.11*visc/MAX(ustar,0.07_kind_phys))
L02735         Z_0 = MAX( Z_0, 1.27e-7_kind_phys)  !These max/mins were suggested by
L02736         Z_0 = MIN( Z_0, 2.85e-3_kind_phys)  !Davis et al. (2008)
L02737
L02738     END SUBROUTINE edson_etal_2013
```

讲解：COARE 3.5（Edson 2013）——重标定的 Charnock α 随 u* 弱变化；WRF 线的 sf_mynn_sfcflux_water=1（默认）即此式。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2638-L2666    (subroutine davis_etal_2008)
L02638     SUBROUTINE davis_etal_2008(Z_0,ustar)
L02639
L02640      !a.k.a. : Donelan et al. (2004)
L02641      !This formulation for roughness length was designed to match
L02642      !the labratory experiments of Donelan et al. (2004).
L02643      !This is an update version from Davis et al. 2008, which
L02644      !corrects a small-bias in Z_0 (AHW real-time 2012).
L02645
L02646         !$acc routine seq
L02647         IMPLICIT NONE
L02648         REAL(kind_phys), INTENT(IN)  :: ustar
L02649         REAL(kind_phys), INTENT(OUT) :: Z_0
L02650         REAL(kind_phys) :: ZW, ZN1, ZN2
L02651         REAL(kind_phys), PARAMETER :: OZO=1.59E-5
L02652
L02653         !OLD FORM: Z_0 = 10.*EXP(-10./(ustar**onethird))
L02654         !NEW FORM:
L02655
L02656         ZW  = MIN((ustar/1.06)**(0.3),1.0_kind_phys)
L02657         ZN1 = 0.011*ustar*ustar*g_inv + OZO
L02658         ZN2 = 10.*exp(-9.5*ustar**(-onethird)) + &
L02659               0.11*1.5E-5/MAX(ustar,0.01_kind_phys)
L02660               !0.11*1.5E-5/AMAX1(ustar,0.01)
L02661         Z_0 = (1.0-ZW) * ZN1 + ZW * ZN2
L02662
L02663         Z_0 = MAX( Z_0, 1.27e-7_kind_phys)  !These max/mins were suggested by
L02664         Z_0 = MIN( Z_0, 2.85e-3_kind_phys)  !Davis et al. (2008)
L02665
L02666     END SUBROUTINE davis_etal_2008
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2671-L2690    (subroutine Taylor_Yelland_2001)
L02671     SUBROUTINE Taylor_Yelland_2001(Z_0,ustar,wsp10)
L02672         !$acc routine seq
L02673         IMPLICIT NONE
L02674         REAL(kind_phys), INTENT(IN)  :: ustar,wsp10
L02675         REAL(kind_phys), INTENT(OUT) :: Z_0
L02676         REAL(kind_phys), parameter   :: pi=3.14159265
L02677         REAL(kind_phys) :: hs, Tp, Lp
L02678
L02679         !hs is the significant wave height
L02680          hs = 0.0248*(wsp10**2.)
L02681         !Tp dominant wave period
L02682          Tp = 0.729*MAX(wsp10,0.1_kind_phys)
L02683         !Lp is the wavelength of the dominant wave
L02684          Lp = grav*Tp**2/(2*pi)
L02685
L02686         Z_0 = 1200.*hs*(hs/Lp)**4.5
L02687         Z_0 = MAX( Z_0, 1.27e-7_kind_phys)  !These max/mins were suggested by
L02688         Z_0 = MIN( Z_0, 2.85e-3_kind_phys)  !Davis et al. (2008)
L02689
L02690     END SUBROUTINE Taylor_Yelland_2001
```

讲解：Davis 2008 是「有限拖曳系数」型（高速风下 CD 饱和，台风模拟常用）；Taylor–Yelland 2001 则用**波陡**参数化——依赖浪场信息，单柱模式里用有效波高/波长近似，脱离波浪模型时物理意义打折。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2992-L3043    (subroutine GFS_z0_wat)
L02992      SUBROUTINE GFS_z0_wat(z0rl_wat,ustar_wat,WSPD,z1,sfc_z0_type,redrag)
L02993
L02994          !$acc routine seq
L02995          REAL(kind_phys), INTENT(OUT)  :: z0rl_wat
L02996          REAL(kind_phys), INTENT(INOUT):: ustar_wat
L02997          REAL(kind_phys), INTENT(IN)   :: wspd,z1
L02998          LOGICAL, INTENT(IN)           :: redrag
L02999          INTEGER, INTENT(IN)           :: sfc_z0_type
L03000          REAL(kind_phys)               :: z0,z0max,wind10m
L03001          REAL(kind_phys), PARAMETER    :: charnock = 0.014, z0s_max=.317e-2
L03002
L03003  !            z0          = 0.01 * z0rl_wat
L03004  !Already converted to meters in the wrapper
L03005              z0          = z0rl_wat
L03006              z0max       = max(1.0e-6_kind_phys, min(z0,z1))
L03007              ustar_wat   = sqrt(grav * z0 / charnock)
L03008              wind10m     = wspd*log(10./1e-4)/log(z1/1e-4)
L03009              !wind10m     = sqrt(u10m(i)*u10m(i)+v10m(i)*v10m(i))
L03010  !
L03011              if (sfc_z0_type >= 0) then
L03012                if (sfc_z0_type == 0) then
L03013                  z0 = (charnock / grav) * ustar_wat * ustar_wat
L03014
L03015  ! mbek -- toga-coare flux algorithm
L03016  !               z0 = (charnock / g) * ustar(i)*ustar(i) +  arnu/ustar(i)
L03017  !  new implementation of z0
L03018  !               cc = ustar(i) * z0 / rnu
L03019  !               pp = cc / (1. + cc)
L03020  !               ff = g * arnu / (charnock * ustar(i) ** 3)
L03021  !               z0 = arnu / (ustar(i) * ff ** pp)
L03022
L03023                  if (redrag) then
L03024                    !z0rl_wat = 100.0 * max(min(z0, z0s_max), 1.e-7)
L03025                    z0rl_wat = max(min(z0, z0s_max), 1.e-7_kind_phys)
L03026                  else
L03027                    !z0rl_wat = 100.0 * max(min(z0,.1), 1.e-7)
L03028                    z0rl_wat = max(min(z0,.1_kind_phys), 1.e-7_kind_phys)
L03029                  endif
L03030
L03031                elseif (sfc_z0_type == 6) then   ! wang
L03032                   call znot_m_v6(wind10m, z0)  ! wind, m/s, z0, m
L03033                   !z0rl_wat = 100.0 * z0          ! cm
L03034                elseif (sfc_z0_type == 7) then   ! wang
L03035                   call znot_m_v7(wind10m, z0)  ! wind, m/s, z0, m
L03036                   !z0rl_wat = 100.0 * z0          ! cm
L03037                else
L03038                   z0rl_wat = 1.0e-6
L03039                endif
L03040
L03041              endif
L03042
L03043      END SUBROUTINE GFS_z0_wat
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3046-L3105    (subroutine GFS_zt_wat)
L03046      SUBROUTINE GFS_zt_wat(ztmax,z0rl_wat,restar,WSPD,z1,sfc_z0_type,device_errmsg,device_errflg)
L03047          !$acc routine seq
L03048          real(kind_phys), INTENT(OUT)  :: ztmax
L03049          real(kind_phys), INTENT(IN)   :: wspd,z1,z0rl_wat,restar
L03050          INTEGER, INTENT(IN)           :: sfc_z0_type
L03051
L03052  ! Using device_errmsg and device_errflg rather than the CCPP errmsg and errflg
L03053  ! so that this subroutine can be run on an accelerator device with OpenACC.
L03054  !        character(len=*), intent(out) :: errmsg
L03055  !        integer,          intent(out) :: errflg
L03056          character(len=512), intent(out) :: device_errmsg
L03057          integer,            intent(out) :: device_errflg
L03058
L03059          real(kind_phys)               :: z0,z0max,wind10m,rat,ustar_wat
L03060          real(kind_phys), PARAMETER    :: charnock = 0.014, z0s_max=.317e-2
L03061
L03062          ! Initialize error-handling
L03063  !        errflg = 0
L03064  !        errmsg = ''
L03065          device_errflg = 0
L03066          device_errmsg = ''
L03067
L03068  !            z0          = 0.01 * z0rl_wat
L03069  !Already converted to meters in the wrapper
L03070              z0          = z0rl_wat
L03071              z0max       = max(1.0e-6_kind_phys, min(z0,z1))
L03072              ustar_wat   = sqrt(grav * z0 / charnock)
L03073              wind10m     = wspd*log(10./1e-4)/log(z1/1e-4)
L03074
L03075  !**  test xubin's new z0
L03076
L03077  !           ztmax  = z0max
L03078
L03079  !input            restar = max(ustar_wat(i)*z0max*visi, 0.000001)
L03080
L03081  !           restar = log(restar)
L03082  !           restar = min(restar,5.)
L03083  !           restar = max(restar,-5.)
L03084  !           rat    = aa1 + (bb1 + cc1*restar) * restar
L03085  !           rat    = rat    / (1. + (bb2 + cc2*restar) * restar))
L03086  !  rat taken from zeng, zhao and dickinson 1997
L03087
L03088              rat   = min(7.0_kind_phys, 2.67 * sqrt(sqrt(restar)) - 2.57)
L03089              ztmax = max(z0max * exp(-rat), 1.0e-6_kind_phys)
L03090  !
L03091              if (sfc_z0_type == 6) then
L03092                call znot_t_v6(wind10m, ztmax)   ! 10-m wind,m/s, ztmax(m)
L03093              else if (sfc_z0_type == 7) then
L03094                call znot_t_v7(wind10m, ztmax)   ! 10-m wind,m/s, ztmax(m)
L03095              else if (sfc_z0_type > 0) then
L03096                write(0,*)'not a valid option for sfc_z0_type=',sfc_z0_type
```

```
L03097  !              errflg = 1
L03098  !              errmsg = 'ERROR(GFS_zt_wat): sfc_z0_type not valid.'
L03099                device_errflg = 1
L03100                device_errmsg = 'ERROR(GFS_zt_wat): sfc_z0_type not valid.'
L03101                return
L03102
L03103              endif
L03104
L03105      END SUBROUTINE GFS_zt_wat
```

讲解：GFS 水面对照（选项 4/5）：z0 三种（常数 1e-5、u* 型、Liu 型 restar 幂律）由 sfc_z0_type 细分，sfc_z0_type=6/7 时转波浪模型式 znot_m_v6/v7；redrag 是 NCEP 的高速风降拖曳开关。zt_wat 同样按 restar 幂律并支持波浪模型选项。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3111-L3156    (subroutine znot_m_v6)
L03111        SUBROUTINE znot_m_v6(uref, znotm)
L03112        !$acc routine seq
L03113        use machine , only : kind_phys
L03114        IMPLICIT NONE
L03115  ! Calculate areodynamical roughness over water with input 10-m wind
L03116  ! For low-to-moderate winds, try to match the Cd-U10 relationship from COARE V3.5 (Edson et al. 2013)
L03117  ! For high winds, try to fit available observational data
L03118  !
L03119  ! Bin Liu, NOAA/NCEP/EMC 2017
L03120  !
L03121  ! uref(m/s)   :   wind speed at 10-m height
L03122  ! znotm(meter):   areodynamical roughness scale over water
L03123  !
L03124
L03125        REAL(kind_phys), INTENT(IN) :: uref
L03126        REAL(kind_phys), INTENT(OUT):: znotm
L03127        REAL(kind_phys), PARAMETER  :: p13 = -1.296521881682694e-02,     &
L03128       &      p12 =  2.855780863283819e-01, p11 = -1.597898515251717e+00,&
L03129       &      p10 = -8.396975715683501e+00,                              &
L03130
L03131       &      p25 =  3.790846746036765e-10, p24 =  3.281964357650687e-09,&
L03132       &      p23 =  1.962282433562894e-07, p22 = -1.240239171056262e-06,&
L03133       &      p21 =  1.739759082358234e-07, p20 =  2.147264020369413e-05,&
L03134
L03135       &      p35 =  1.840430200185075e-07, p34 = -2.793849676757154e-05,&
L03136       &      p33 =  1.735308193700643e-03, p32 = -6.139315534216305e-02,&
L03137       &      p31 =  1.255457892775006e+00, p30 = -1.663993561652530e+01,&
L03138
L03139       &      p40 =  4.579369142033410e-04
L03140
L03141
L03142         if (uref >= 0.0 .and.  uref <= 6.5 ) then
L03143          znotm = exp(p10 + uref * (p11 + uref * (p12 + uref*p13)))
L03144         elseif (uref > 6.5 .and. uref <= 15.7) then
L03145          znotm = p20 + uref * (p21 + uref * (p22 + uref * (p23       &
L03146       &             + uref * (p24 + uref * p25))))
L03147         elseif (uref > 15.7 .and. uref <= 53.0) then
L03148          znotm = exp( p30 + uref * (p31 + uref * (p32 + uref * (p33  &
L03149       &                  + uref * (p34 + uref * p35)))))
L03150         elseif ( uref > 53.0) then
L03151           znotm = p40
L03152         else
L03153           print*, 'Wrong input uref value:',uref
L03154         endif
L03155
L03156        END SUBROUTINE znot_m_v6
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3167-L3221    (subroutine znot_t_v6)
L03167        SUBROUTINE znot_t_v6(uref, znott)
L03168
L03169        !$acc routine seq
L03170        IMPLICIT NONE
L03171  !
L03172        REAL(kind_phys), INTENT(IN) :: uref
L03173        REAL(kind_phys), INTENT(OUT):: znott
L03174        REAL(kind_phys), PARAMETER  ::       p00 =  1.100000000000000e-04,&
L03175       &      p15 = -9.144581627678278e-10, p14 =  7.020346616456421e-08,&
L03176       &      p13 = -2.155602086883837e-06, p12 =  3.333848806567684e-05,&
L03177       &      p11 = -2.628501274963990e-04, p10 =  8.634221567969181e-04,&
L03178
L03179       &      p25 = -8.654513012535990e-12, p24 =  1.232380050058077e-09,&
L03180       &      p23 = -6.837922749505057e-08, p22 =  1.871407733439947e-06,&
L03181       &      p21 = -2.552246987137160e-05, p20 =  1.428968311457630e-04,&
L03182
L03183       &      p35 =  3.207515102100162e-12, p34 = -2.945761895342535e-10,&
L03184       &      p33 =  8.788972147364181e-09, p32 = -3.814457439412957e-08,&
L03185       &      p31 = -2.448983648874671e-06, p30 =  3.436721779020359e-05,&
L03186
L03187       &      p45 = -3.530687797132211e-11, p44 =  3.939867958963747e-09,&
L03188       &      p43 = -1.227668406985956e-08, p42 = -1.367469811838390e-05,&
L03189       &      p41 =  5.988240863928883e-04, p40 = -7.746288511324971e-03,&
L03190
L03191       &      p56 = -1.187982453329086e-13, p55 =  4.801984186231693e-11,&
L03192       &      p54 = -8.049200462388188e-09, p53 =  7.169872601310186e-07,&
L03193       &      p52 = -3.581694433758150e-05, p51 =  9.503919224192534e-04,&
L03194       &      p50 = -1.036679430885215e-02,                              &
L03195
L03196       &      p60 =  4.751256171799112e-05
L03197
L03198        if (uref >= 0.0 .and. uref < 5.9 ) then
L03199           znott = p00
L03200        elseif (uref >= 5.9 .and. uref <= 15.4) then
L03201           znott = p10 + uref * (p11 + uref * (p12 + uref * (p13  &
L03202       &                + uref * (p14 + uref * p15))))
L03203        elseif (uref > 15.4 .and. uref <= 21.6) then
L03204           znott = p20 + uref * (p21 + uref * (p22 + uref * (p23  &
L03205       &                + uref * (p24 + uref * p25))))
L03206        elseif (uref > 21.6 .and. uref <= 42.2) then
L03207           znott = p30 + uref * (p31 + uref * (p32 + uref * (p33  &
```

```
L03208       &               + uref * (p34 + uref * p35))))
L03209        elseif ( uref > 42.2 .and. uref <= 53.3) then
L03210           znott = p40 + uref * (p41 + uref * (p42 + uref * (p43  &
L03211       &               + uref * (p44 + uref * p45))))
L03212        elseif ( uref > 53.3 .and. uref <= 80.0) then
L03213           znott = p50 + uref * (p51 + uref * (p52 + uref * (p53  &
L03214       &               + uref * (p54 + uref * (p55 + uref * p56)))))
L03215        elseif ( uref > 80.0) then
L03216           znott = p60
L03217        else
L03218           print*, 'Wrong input uref value:',uref
L03219        endif
L03220
L03221        END SUBROUTINE znot_t_v6
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3231-L3271    (subroutine znot_m_v7)
L03231        SUBROUTINE znot_m_v7(uref, znotm)
L03232
L03233        !$acc routine seq
L03234        IMPLICIT NONE
L03235  !
L03236  ! uref(m/s)   :   wind speed at 10-m height
L03237  ! znotm(meter):   areodynamical roughness scale over water
L03238  !
L03239
L03240        REAL(kind_phys), INTENT(IN) :: uref
L03241        REAL(kind_phys), INTENT(OUT):: znotm
L03242
L03243        REAL(kind_phys), PARAMETER  ::      p13 = -1.296521881682694e-02,&
L03244       &      p12 =  2.855780863283819e-01, p11 = -1.597898515251717e+00,&
L03245       &      p10 = -8.396975715683501e+00,                              &
L03246
L03247       &      p25 =  3.790846746036765e-10, p24 =  3.281964357650687e-09,&
L03248       &      p23 =  1.962282433562894e-07, p22 = -1.240239171056262e-06,&
L03249       &      p21 =  1.739759082358234e-07, p20 =  2.147264020369413e-05,&
L03250
L03251       &      p35 =  1.897534489606422e-07, p34 = -3.019495980684978e-05,&
L03252       &      p33 =  1.931392924987349e-03, p32 = -6.797293095862357e-02,&
L03253       &      p31 =  1.346757797103756e+00, p30 = -1.707846930193362e+01,&
L03254
L03255       &      p40 =  3.371427455376717e-04
L03256
L03257        if (uref >= 0.0 .and.  uref <= 6.5 ) then
L03258          znotm = exp( p10 + uref * (p11 + uref * (p12 + uref * p13)))
L03259        elseif (uref > 6.5 .and. uref <= 15.7) then
L03260          znotm = p20 + uref * (p21 + uref * (p22 + uref * (p23        &
L03261       &              + uref * (p24 + uref * p25))))
L03262        elseif (uref > 15.7 .and. uref <= 53.0) then
L03263          znotm = exp( p30 + uref * (p31 + uref * (p32 + uref * (p33   &
L03264       &                   + uref * (p34 + uref * p35)))))
L03265        elseif ( uref > 53.0) then
L03266          znotm = p40
L03267        else
L03268          print*, 'Wrong input uref value:',uref
L03269        endif
L03270
L03271        END SUBROUTINE znot_m_v7
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3280-L3336    (subroutine znot_t_v7)
L03280        SUBROUTINE znot_t_v7(uref, znott)
L03281
L03282        !$acc routine seq
L03283        IMPLICIT NONE
L03284  !
L03285  ! uref(m/s)   :   wind speed at 10-m height
L03286  ! znott(meter):   scalar roughness scale over water
L03287  !
L03288
L03289        REAL(kind_phys), INTENT(IN) :: uref
L03290        REAL(kind_phys), INTENT(OUT):: znott
L03291        REAL(kind_phys), PARAMETER  ::      p00 =  1.100000000000000e-04,&
L03292       &      p15 = -9.193764479895316e-10, p14 =  7.052217518653943e-08,&
L03293       &      p13 = -2.163419217747114e-06, p12 =  3.342963077911962e-05,&
L03294       &      p11 = -2.633566691328004e-04, p10 =  8.644979973037803e-04,&
L03295
L03296       &      p25 = -9.402722450219142e-12, p24 =  1.325396583616614e-09,&
L03297       &      p23 = -7.299148051141852e-08, p22 =  1.982901461144764e-06,&
L03298       &      p21 = -2.680293455916390e-05, p20 =  1.484341646128200e-04,&
L03299
L03300       &      p35 =  7.921446674311864e-12, p34 = -1.019028029546602e-09,&
L03301       &      p33 =  5.251986927351103e-08, p32 = -1.337841892062716e-06,&
L03302       &      p31 =  1.659454106237737e-05, p30 = -7.558911792344770e-05,&
L03303
L03304       &      p45 = -2.694370426850801e-10, p44 =  5.817362913967911e-08,&
L03305       &      p43 = -5.000813324746342e-06, p42 =  2.143803523428029e-04,&
L03306       &      p41 = -4.588070983722060e-03, p40 =  3.924356617245624e-02,&
L03307
L03308       &      p56 = -1.663918773476178e-13, p55 =  6.724854483077447e-11,&
L03309       &      p54 = -1.127030176632823e-08, p53 =  1.003683177025925e-06,&
L03310       &      p52 = -5.012618091180904e-05, p51 =  1.329762020689302e-03,&
L03311       &      p50 = -1.450062148367566e-02, p60 =  6.840803042788488e-05
L03312
L03313          if (uref >= 0.0 .and. uref < 5.9 ) then
L03314             znott = p00
L03315          elseif (uref >= 5.9 .and. uref <= 15.4) then
L03316             znott = p10 + uref * (p11 + uref * (p12 + uref * (p13     &
L03317       &                + uref * (p14 + uref * p15))))
L03318          elseif (uref > 15.4 .and. uref <= 21.6) then
L03319             znott = p20 + uref * (p21 + uref * (p22 + uref * (p23     &
L03320       &                + uref * (p24 + uref * p25))))
L03321          elseif (uref > 21.6 .and. uref <= 42.6) then
L03322             znott = p30 + uref * (p31 + uref * (p32 + uref * (p33     &
L03323       &                + uref * (p34 + uref * p35))))
L03324          elseif ( uref > 42.6 .and. uref <= 53.0) then
L03325             znott = p40 + uref * (p41 + uref * (p42 + uref * (p43     &
L03326       &                + uref * (p44 + uref * p45))))
```

```
L03327          elseif ( uref > 53.0 .and. uref <= 80.0) then
L03328             znott = p50 + uref * (p51 + uref * (p52 + uref * (p53     &
L03329       &                + uref * (p54 + uref * (p55 + uref * p56))))
L03330          elseif ( uref > 80.0) then
L03331             znott = p60
L03332          else
L03333             print*, 'Wrong input uref value:',uref
L03334          endif
L03335
L03336          END SUBROUTINE znot_t_v7
```

讲解：四个波浪模型耦合用 z0 参数化（Wang 系列 v6/v7，按 10 m 风的经验拟合）。注意 phys/module_sf_exchcoef.F 里还有一份同名拷贝（WRF 旧波浪耦合路径用的），两处实现同源——改这里时搜一下另一处是否也要同步。v6 与 v7 的差别是 v7 在高风速下略降 CD。

组三：水面/雪冰 zt/zq：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2781-L2815    (subroutine fairall_etal_2003)
L02781      SUBROUTINE fairall_etal_2003(Zt,Zq,Ren,ustar,visc,rstoch,spp_sfc)
L02782         !$acc routine seq
L02783         IMPLICIT NONE
L02784         REAL(kind_phys), INTENT(IN)   :: Ren,ustar,visc,rstoch
L02785         INTEGER, INTENT(IN)           :: spp_sfc
L02786         REAL(kind_phys), INTENT(OUT)  :: Zt,Zq
L02787
L02788         IF (Ren .le. 2.) then
L02789
L02790            Zt = (5.5e-5)*(Ren**(-0.60))
L02791            Zq = Zt
L02792            !FOR SMOOTH SEAS, CAN USE GARRATT
L02793            !Zq = 0.2*visc/MAX(ustar,0.1)
L02794            !Zq = 0.3*visc/MAX(ustar,0.1)
L02795
L02796         ELSE
L02797
L02798            !FOR ROUGH SEAS, USE COARE
L02799            Zt = (5.5e-5)*(Ren**(-0.60))
L02800            Zq = Zt
L02801
L02802         ENDIF
L02803
L02804         if (spp_sfc==1) then
L02805            Zt = Zt + Zt * 0.5 * rstoch
L02806            Zq = Zt
L02807         endif
L02808
L02809         Zt = MIN(Zt,1.0e-4_kind_phys)
L02810         Zt = MAX(Zt,2.0e-9_kind_phys)
L02811
L02812         Zq = MIN(Zt,1.0e-4_kind_phys)
L02813         Zq = MAX(Zt,2.0e-9_kind_phys)
L02814
L02815      END SUBROUTINE fairall_etal_2003
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L2823-L2842    (subroutine fairall_etal_2014)
L02823      SUBROUTINE fairall_etal_2014(Zt,Zq,Ren,ustar,visc,rstoch,spp_sfc)
L02824         !$acc routine seq
L02825         IMPLICIT NONE
L02826         REAL(kind_phys), INTENT(IN)  :: Ren,ustar,visc,rstoch
L02827         INTEGER, INTENT(IN)          :: spp_sfc
L02828         REAL(kind_phys), INTENT(OUT) :: Zt,Zq
L02829
L02830         !Zt = (5.5e-5)*(Ren**(-0.60))
L02831         Zt = MIN(1.6E-4_kind_phys, 5.8E-5/(Ren**0.72))
L02832         Zq = Zt
L02833
L02834         IF (spp_sfc ==1) THEN
L02835            Zt = MAX(Zt + Zt*0.5*rstoch,2.0e-9_kind_phys)
L02836            Zq = MAX(Zt + Zt*0.5*rstoch,2.0e-9_kind_phys)
L02837         ELSE
L02838            Zt = MAX(Zt,2.0e-9_kind_phys)
L02839            Zq = MAX(Zt,2.0e-9_kind_phys)
L02840         ENDIF
L02841
L02842      END SUBROUTINE fairall_etal_2014
```

讲解：COARE 3.0/3.5 的标量粗糙度：zt、zq 由 restar 的指数式给光滑流极限 + SPP 扰动挂点；3.5 版系数重标定。这是水面 zt/zq 的默认（isftcflx=0/1/3）。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3344-L3389    (subroutine Andreas_2002)
L03344      SUBROUTINE Andreas_2002(Z_0,bvisc,ustar,Zt,Zq)
L03345
L03346         !$acc routine seq
L03347         IMPLICIT NONE
L03348         REAL(kind_phys), INTENT(IN)  :: Z_0, bvisc, ustar
L03349         REAL(kind_phys), INTENT(OUT) :: Zt, Zq
L03350         REAL(kind_phys) :: Ren2, zntsno
L03351
L03352         REAL(kind_phys), PARAMETER  ::                                  &
L03353                             bt0_s=1.25,  bt0_t=0.149,  bt0_r=0.317,  &
L03354                             bt1_s=0.0,   bt1_t=-0.55,  bt1_r=-0.565, &
L03355                             bt2_s=0.0,   bt2_t=0.0,    bt2_r=-0.183
L03356
L03357         REAL(kind_phys), PARAMETER  ::                                  &
L03358                             bq0_s=1.61,  bq0_t=0.351,  bq0_r=0.396,  &
L03359                             bq1_s=0.0,   bq1_t=-0.628, bq1_r=-0.512, &
L03360                             bq2_s=0.0,   bq2_t=0.0,    bq2_r=-0.180
L03361
```

```
L03362        !Calculate zo for snow (Andreas et al. 2005, BLM)
L03363         zntsno = 0.135*bvisc/ustar +                              &
L03364                 (0.035*(ustar*ustar)*g_inv) *                     &
L03365                 (5.*exp(-1.*(((ustar - 0.18)/0.1)*((ustar - 0.18)/0.1))) + 1.)
L03366         Ren2 = ustar*zntsno/bvisc
L03367
L03368         ! Make sure that Re is not outside of the range of validity
L03369         ! for using their equations
L03370         IF (Ren2 .gt. 1000.) Ren2 = 1000.
L03371
L03372         IF (Ren2 .le. 0.135) then
L03373
L03374            Zt = zntsno*EXP(bt0_s + bt1_s*LOG(Ren2) + bt2_s*LOG(Ren2)**2)
L03375            Zq = zntsno*EXP(bq0_s + bq1_s*LOG(Ren2) + bq2_s*LOG(Ren2)**2)
L03376
L03377         ELSE IF (Ren2 .gt. 0.135 .AND. Ren2 .lt. 2.5) then
L03378
L03379            Zt = zntsno*EXP(bt0_t + bt1_t*LOG(Ren2) + bt2_t*LOG(Ren2)**2)
L03380            Zq = zntsno*EXP(bq0_t + bq1_t*LOG(Ren2) + bq2_t*LOG(Ren2)**2)
L03381
L03382         ELSE
L03383
L03384            Zt = zntsno*EXP(bt0_r + bt1_r*LOG(Ren2) + bt2_r*LOG(Ren2)**2)
L03385            Zq = zntsno*EXP(bq0_r + bq1_r*LOG(Ren2) + bq2_r*LOG(Ren2)**2)
L03386
L03387         ENDIF
L03388
L03389      END SUBROUTINE Andreas_2002
```

讲解：雪/冰专用：由 restar 分三个流区（光滑/过渡/粗糙）给 zt、zq——SMT（surface renewal theory）参数化。冰面无选项直接用；陆面雪深 >50 mm（源码写 50.，参数 snowh_thresh 注释为 mm）时覆盖 Zilitinkevich。头注强调：雪 z0 只影响标量粗糙度，不直接改变动量 z0。

组四：经典 ψ 库与 Li 2010 初猜：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3523-L3548    (subroutine PSI_Businger_1971)
L03523      SUBROUTINE PSI_Businger_1971(psi_m, psi_h, zL)
L03524
L03525         IMPLICIT NONE
L03526         REAL(kind_phys), INTENT(IN)  :: zL
L03527         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03528         REAL(kind_phys)  :: x, y
L03529         REAL(kind_phys), PARAMETER  ::  Pi180 = 3.14159265/180.
L03530
L03531         IF (zL .lt. 0.) THEN  !UNSTABLE
L03532
L03533            x = (1. - 15.0*zL)**0.25
L03534            y = (1. - 9.0*zL)**0.5
L03535
L03536            psi_m = LOG(((1.+x)/2.)**2.) + &
L03537                  &LOG((1.+x**2.)/2.) - &
L03538                  &2.0*ATAN(x) + Pi180*90.
L03539            psi_h = 2.*LOG((1.+y)/2.)
L03540
L03541         ELSE                 !STABLE
L03542
L03543            psi_m = -4.7*zL
L03544            psi_h = -(4.7/0.74)*zL
L03545
L03546         ENDIF
L03547
L03548      END SUBROUTINE PSI_Businger_1971
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3430-L3460    (subroutine PSI_DyerHicks)
L03430      SUBROUTINE PSI_DyerHicks(psi_m, psi_h, zL, Zt, Z_0, Za)
L03431
L03432         IMPLICIT NONE
L03433         REAL(kind_phys), INTENT(IN)  :: zL, Zt, Z_0, Za
L03434         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03435         REAL(kind_phys)  :: x, x0, y, y0, zmL, zhL
L03436
L03437         zmL = Z_0*zL/Za  !Zo/L
L03438         zhL = Zt*zL/Za   !Zt/L
L03439
L03440         IF (zL .gt. 0.) THEN  !STABLE
L03441
L03442            psi_m = -5.0*(zL - zmL)
L03443            psi_h = -5.0*(zL - zhL)
L03444
L03445         ELSE                 !UNSTABLE
L03446
L03447            x = (1.-16.*zL)**0.25
L03448            x0= (1.-16.*zmL)**0.25
L03449
L03450            y = (1.-16.*zL)**0.5
L03451            y0= (1.-16.*zhL)**0.5
L03452
L03453            psi_m = 2.*LOG((1.+x)/(1.+x0)) +         &
L03454                    &LOG((1.+x**2.)/(1.+x0**2.)) - &
L03455                    &2.0*ATAN(x) + 2.0*ATAN(x0)
L03456            psi_h = 2.*LOG((1.+y)/(1.+y0))
L03457
L03458         ENDIF
L03459
L03460      END SUBROUTINE PSI_DyerHicks
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3394-L3423    (subroutine PSI_Hogstrom_1996)
L03394      SUBROUTINE PSI_Hogstrom_1996(psi_m, psi_h, zL, Zt, Z_0, Za)
L03395
L03396         IMPLICIT NONE
L03397         REAL(kind_phys), INTENT(IN)  :: zL, Zt, Z_0, Za
```

```
L03398         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03399         REAL(kind_phys)  :: x, x0, y, y0, zmL, zhL
L03400
L03401         zmL = Z_0*zL/Za
L03402         zhL = Zt*zL/Za
L03403
L03404         IF (zL .gt. 0.) THEN  !STABLE (not well tested - seem large)
L03405
L03406            psi_m = -5.3*(zL - zmL)
L03407            psi_h = -8.0*(zL - zhL)
L03408
L03409         ELSE                  !UNSTABLE
L03410
L03411            x = (1.-19.0*zL)**0.25
L03412            x0= (1.-19.0*zmL)**0.25
L03413            y = (1.-11.6*zL)**0.5
L03414            y0= (1.-11.6*zhL)**0.5
L03415
L03416            psi_m = 2.*LOG((1.+x)/(1.+x0)) +         &
L03417                      &LOG((1.+x**2.)/(1.+x0**2.)) - &
L03418                      &2.0*ATAN(x) + 2.0*ATAN(x0)
L03419            psi_h = 2.*LOG((1.+y)/(1.+y0))
L03420
L03421         ENDIF
L03422
L03423      END SUBROUTINE PSI_Hogstrom_1996
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3466-L3489    (subroutine PSI_Beljaars_Holtslag_1991)
L03466      SUBROUTINE PSI_Beljaars_Holtslag_1991(psi_m, psi_h, zL)
L03467
L03468         IMPLICIT NONE
L03469         REAL(kind_phys), INTENT(IN)  :: zL
L03470         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03471         REAL(kind_phys), PARAMETER   :: a=1., b=0.666, c=5., d=0.35
L03472
L03473         IF (zL .lt. 0.) THEN  !UNSTABLE
L03474
L03475            WRITE(*,*)"WARNING: Universal stability functions from"
L03476            WRITE(*,*)"         Beljaars and Holtslag (1991) should only"
L03477            WRITE(*,*)"         be used in the stable regime!"
L03478            psi_m = 0.
L03479            psi_h = 0.
L03480
L03481         ELSE                  !STABLE
L03482
L03483            psi_m = -(a*zL + b*(zL -(c/d))*exp(-d*zL) + (b*c/d))
L03484            psi_h = -((1.+.666*a*zL)**1.5 + &
L03485                    b*(zL - (c/d))*exp(-d*zL) + (b*c/d) -1.)
L03486
L03487         ENDIF
L03488
L03489      END SUBROUTINE PSI_Beljaars_Holtslag_1991
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3496-L3518    (subroutine PSI_Zilitinkevich_Esau_2007)
L03496      SUBROUTINE PSI_Zilitinkevich_Esau_2007(psi_m, psi_h, zL)
L03497
L03498         IMPLICIT NONE
L03499         REAL(kind_phys), INTENT(IN)  :: zL
L03500         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03501         REAL(kind_phys), PARAMETER   :: Cm=3.0, Ct=2.5
L03502
L03503         IF (zL .lt. 0.) THEN  !UNSTABLE
L03504
L03505            WRITE(*,*)"WARNING: Universal stability function from"
L03506            WRITE(*,*)"         Zilitinkevich and Esau (2007) should only"
L03507            WRITE(*,*)"         be used in the stable regime!"
L03508            psi_m = 0.
L03509            psi_h = 0.
L03510
L03511         ELSE                  !STABLE
L03512
L03513            psi_m = -Cm*(zL**(5./6.))
L03514            psi_h = -Ct*(zL**(4./5.))
L03515
L03516         ENDIF
L03517
L03518      END SUBROUTINE PSI_Zilitinkevich_Esau_2007
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3557-L3579    (subroutine PSI_Suselj_Sood_2010)
L03557      SUBROUTINE PSI_Suselj_Sood_2010(psi_m, psi_h, zL)
L03558
L03559         IMPLICIT NONE
L03560         REAL(kind_phys), INTENT(IN)  :: zL
L03561         REAL(kind_phys), INTENT(OUT) :: psi_m, psi_h
L03562         REAL(kind_phys), PARAMETER   :: Rfc=0.19, Ric=0.183, PHIT=0.8
L03563
L03564         IF (zL .gt. 0.) THEN  !STABLE
L03565
L03566            psi_m = -(zL/Rfc + 1.1223*EXP(1.-1.6666/zL))
L03567            !psi_h = -zL*Ric/((Rfc**2.)*PHIT) + 8.209*(zL**1.1091)
L03568            !THEIR EQ FOR PSI_H CRASHES THE MODEL AND DOES NOT MATCH
L03569            !THEIR FIG 1. THIS EQ (BELOW) MATCHES THEIR FIG 1 BETTER:
L03570            psi_h = -(zL*Ric/((Rfc**2.)*5.) + 7.09*(zL**1.1091))
L03571
L03572         ELSE                  !UNSTABLE
L03573
L03574            psi_m = 0.9904*LOG(1. - 14.264*zL)
L03575            psi_h = 1.0103*LOG(1. - 16.3066*zL)
L03576
L03577         ENDIF
L03578
L03579      END SUBROUTINE PSI_Suselj_Sood_2010
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3585-L3596    (subroutine PSI_CB2005)
L03585      SUBROUTINE PSI_CB2005(psim1,psih1,zL,z0L)
L03586
L03587         IMPLICIT NONE
L03588         REAL(kind_phys), INTENT(IN)  :: zL,z0L
L03589         REAL(kind_phys), INTENT(OUT) :: psim1,psih1
L03590
L03591         psim1 = -6.1*LOG(zL + (1.+ zL**2.5)**0.4)           &
L03592                 -6.1*LOG(z0L + (1.+ z0L**2.5)**0.4)
L03593         psih1 = -5.5*log(zL + (1.+ zL**1.1)**0.90909090909)  &
L03594                 -5.5*log(z0L + (1.+ z0L**1.1)**0.90909090909)
L03595
L03596      END SUBROUTINE PSI_CB2005
```

讲解：七套**备选**相似性函数——主流程默认不走它们（S7 里的调用点全部被注释掉），实际生效的是 4.6 的查表版 MYNN 混合式。保留这些子程序的价值：① 教学对照——Businger 1971 是教科书标准式、Dyer–Hicks 是最常见的简化式、Høgström 1996 是欧洲系常用重标定、Beljaars–Holtslag 是稳定侧带凸起修正的经典、Cheng–Brutsaert 2005 是当前稳定侧正式方案；② 试验挂点——把任一 CALL 反注释即可切函数（记得同步查表版的 *_full 函数，两处必须一致）。这套库在三个伴生件里各有一份小写命名拷贝。

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3602-L3659    (subroutine Li_etal_2010)
L03602      SUBROUTINE Li_etal_2010(zL, Rib, zaz0, z0zt)
L03603
L03604         !$acc routine seq
L03605         IMPLICIT NONE
L03606         REAL(kind_phys), INTENT(OUT)  :: zL
L03607         REAL(kind_phys), INTENT(IN) :: Rib, zaz0, z0zt
L03608         REAL(kind_phys) :: alfa, beta, zaz02, z0zt2
L03609         REAL(kind_phys), PARAMETER  ::                              &
L03610                          & au11=0.045,   bu11=0.003,   bu12=0.0059, &
L03611                          & bu21=-0.0828, bu22=0.8845,  bu31=0.1739, &
L03612                          & bu32=-0.9213, bu33=-0.1057
L03613         REAL(kind_phys), PARAMETER  ::                              &
L03614                          & aw11=0.5738,  aw12=-0.4399, aw21=-4.901, &
L03615                          & aw22=52.50,   bw11=-0.0539, bw12=1.540,  &
L03616                          & bw21=-0.669,  bw22=-3.282
L03617         REAL(kind_phys), PARAMETER  ::                              &
L03618                          & as11=0.7529,  as21=14.94,   bs11=0.1569, &
L03619                          & bs21=-0.3091, bs22=-1.303
L03620
L03621         !set limits according to Li et al (2010), p 157.
L03622         zaz02=zaz0
L03623         IF (zaz0 .lt. 100.0) zaz02=100.
L03624         IF (zaz0 .gt. 100000.0) zaz02=100000.
L03625
L03626         !set more limits according to Li et al (2010)
L03627         z0zt2=z0zt
L03628         IF (z0zt .lt. 0.5) z0zt2=0.5
L03629         IF (z0zt .gt. 100.0) z0zt2=100.
L03630
L03631         alfa = LOG(zaz02)
L03632         beta = LOG(z0zt2)
L03633
L03634         IF (Rib .le. 0.0) THEN
L03635            zL = au11*alfa*Rib**2 + (                  &
L03636                 &  (bu11*beta + bu12)*alfa**2 +       &
L03637                 &  (bu21*beta + bu22)*alfa    +       &
L03638                 &  (bu31*beta**2 + bu32*beta + bu33))*Rib
L03639            !if(zL .LT. -15 .OR. zl .GT. 0.)print*,"VIOLATION Rib<0:",zL
L03640            zL = MAX(zL,-15._kind_phys) !LIMITS SET ACCORDING TO Li et al (2010)
L03641            zL = MIN(zL,0._kind_phys)   !Figure 1.
L03642         ELSEIF (Rib .gt. 0.0 .AND. Rib .le. 0.2) THEN
L03643            zL = ((aw11*beta + aw12)*alfa +            &
L03644               &  (aw21*beta + aw22))*Rib**2 +         &
L03645               & ((bw11*beta + bw12)*alfa +            &
L03646               &  (bw21*beta + bw22))*Rib
L03647            !if(zL .LT. 0 .OR. zl .GT. 4)print*,"VIOLATION 0<Rib<0.2:",zL
L03648            zL = MIN(zL,4._kind_phys) !LIMITS APPROX SET ACCORDING TO Li et al (2010)
L03649            zL = MAX(zL,0._kind_phys) !THEIR FIGURE 1B.
L03650         ELSE
L03651            zL = (as11*alfa + as21)*Rib + bs11*alfa +   &
L03652               &  bs21*beta + bs22
L03653            !if(zL .LE. 1 .OR. zl .GT. 23)print*,"VIOLATION Rib>0.2:",zL
L03654            zL = MIN(zL,20._kind_phys) !LIMITS ACCORDING TO Li et al (2010), THIER
L03655                                       !FIGUE 1C.
L03656            zL = MAX(zL,1._kind_phys)
L03657         ENDIF
L03658
L03659      END SUBROUTINE Li_etal_2010
```

讲解：Li 2010 BLM 的显式初猜：由 rb、z/z0、z0/zt（zratio）三个无量纲数直接拟合 z/L，不稳定侧是带理函数项的复合式、稳定侧是幂律式。它是迭代启动的第一步，也是迭代失败的回退值——物理上不精确但永远有定义。

4.6 查表系统：psi_init 与两套表函数

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3834-L3877    (subroutine psi_init)
L03834      SUBROUTINE psi_init(psi_opt,errmsg,errflg)
L03835
L03836      integer                       :: N,psi_opt
L03837      real(kind_phys)               :: zolf
L03838      character(len=*), intent(out) :: errmsg
L03839      integer, intent(out)          :: errflg
```

```
L03840
L03841      if (psi_opt == 0) then
L03842         DO N=0,1000
L03843            ! stable function tables
L03844            zolf = float(n)*0.01
L03845            psim_stab(n)=psim_stable_full(zolf)
L03846            psih_stab(n)=psih_stable_full(zolf)
L03847
L03848            ! unstable function tables
L03849            zolf = -float(n)*0.01
L03850            psim_unstab(n)=psim_unstable_full(zolf)
L03851            psih_unstab(n)=psih_unstable_full(zolf)
L03852         ENDDO
L03853      else
L03854         DO N=0,1000
L03855            ! stable function tables
L03856            zolf = float(n)*0.01
L03857            psim_stab(n)=psim_stable_full_gfs(zolf)
L03858            psih_stab(n)=psih_stable_full_gfs(zolf)
L03859
L03860            ! unstable function tables
L03861            zolf = -float(n)*0.01
L03862            psim_unstab(n)=psim_unstable_full_gfs(zolf)
L03863            psih_unstab(n)=psih_unstable_full_gfs(zolf)
L03864         ENDDO
L03865      endif
L03866
L03867      !Simple test to see if initialization worked:
L03868      if (psim_stab(1) < 0. .AND. psih_stab(1) < 0. .AND. &
L03869          psim_unstab(1) > 0. .AND. psih_unstab(1) > 0.) then
L03870         errmsg = 'In MYNN SFC, Psi tables have been initialized'
L03871         errflg = 0
L03872      else
L03873         errmsg = 'Error in MYNN SFC: Problem initializing psi tables'
L03874         errflg = 1
L03875      endif
L03876
L03877     END SUBROUTINE psi_init
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc.F90    L3878-L4085
L03878  ! ==================================================================
L03879  ! ... integrated similarity functions from MYNN...
L03880  !
L03881  !>\ingroup mynn_sfc
L03882     real(kind_phys) function psim_stable_full(zolf)
L03883          !$acc routine seq
L03884          real(kind_phys) :: zolf
L03885
L03886          !psim_stable_full=-6.1*log(zolf+(1+zolf**2.5)**(1./2.5))
L03887          psim_stable_full=-6.1*log(zolf+(1+zolf**2.5)**0.4)
L03888
L03889     end function
L03890
L03891  !>\ingroup mynn_sfc
L03892     real(kind_phys) function psih_stable_full(zolf)
L03893          !$acc routine seq
L03894          real(kind_phys) :: zolf
L03895
L03896          !psih_stable_full=-5.3*log(zolf+(1+zolf**1.1)**(1./1.1))
L03897          psih_stable_full=-5.3*log(zolf+(1+zolf**1.1)**0.9090909090909090909)
L03898
L03899     end function
L03900
L03901  !>\ingroup mynn_sfc
L03902     real(kind_phys) function psim_unstable_full(zolf)
L03903          !$acc routine seq
L03904          real(kind_phys) :: zolf,x,ym,psimc,psimk
L03905
L03906          x=(1.-16.*zolf)**.25
L03907          !psimk=2*ALOG(0.5*(1+X))+ALOG(0.5*(1+X*X))-2.*ATAN(X)+2.*ATAN(1.)
L03908          !psimk=2.*ALOG(0.5*(1+X))+ALOG(0.5*(1+X*X))-2.*ATAN(X)+2.*atan1
L03909          psimk=2.*LOG(0.5*(1+X))+LOG(0.5*(1+X*X))-2.*ATAN(X)+2.*atan1
L03910
L03911          ym=(1.-10.*zolf)**onethird
L03912          !psimc=(3./2.)*log((ym**2.+ym+1.)/3.)-sqrt(3.)*ATAN((2.*ym+1)/sqrt(3.))+4.*ATAN(1.)/sqrt(3.)
L03913          psimc=1.5*log((ym**2 + ym+1.)*onethird)-sqrt3*ATAN((2.*ym+1)/sqrt3)+4.*atan1/sqrt3
L03914
L03915          psim_unstable_full=(psimk+zolf**2*(psimc))/(1+zolf**2.)
L03916
L03917     end function
L03918
L03919  !>\ingroup mynn_sfc
L03920     real(kind_phys) function psih_unstable_full(zolf)
L03921          !$acc routine seq
L03922          real(kind_phys) :: zolf,y,yh,psihc,psihk
L03923
L03924          y=(1.-16.*zolf)**.5
L03925          !psihk=2.*log((1+y)/2.)
L03926          psihk=2.*log((1+y)*0.5)
L03927
L03928          yh=(1.-34.*zolf)**onethird
L03929          !psihc=(3./2.)*log((yh**2.+yh+1.)/3.)-sqrt(3.)*ATAN((2.*yh+1)/sqrt(3.))+4.*ATAN(1.)/sqrt(3.)
L03930          psihc=1.5*log((yh**2.+yh+1.)*onethird)-sqrt3*ATAN((2.*yh+1)/sqrt3)+4.*atan1/sqrt3
L03931
L03932          psih_unstable_full=(psihk+zolf**2*(psihc))/(1+zolf**2)
L03933
L03934     end function
L03935
L03936  ! ==================================================================
L03937  ! ... integrated similarity functions from GFS...
L03938  !
L03939  !>\ingroup mynn_sfc
L03940  !!
L03941     REAL(kind_phys) function psim_stable_full_gfs(zolf)
L03942          !$acc routine seq
L03943          REAL(kind_phys) :: zolf
L03944          REAL(kind_phys), PARAMETER :: alpha4 = 20.
```

```
L03945          REAL(kind_phys) :: aa
L03946
L03947          aa     = sqrt(1. + alpha4 * zolf)
L03948          psim_stable_full_gfs  = -1.*aa + log(aa + 1.)
L03949
L03950     end function
L03951
L03952  !>\ingroup mynn_sfc
L03953  !!
L03954     real(kind_phys) function psih_stable_full_gfs(zolf)
L03955          !$acc routine seq
L03956          real(kind_phys) :: zolf
L03957          real(kind_phys), PARAMETER :: alpha4 = 20.
L03958          real(kind_phys) :: bb
L03959
L03960          bb     = sqrt(1. + alpha4 * zolf)
L03961          psih_stable_full_gfs  = -1.*bb + log(bb + 1.)
L03962
L03963     end function
L03964
L03965  !>\ingroup mynn_sfc
L03966  !!
L03967     real(kind_phys) function psim_unstable_full_gfs(zolf)
L03968          !$acc routine seq
L03969          real(kind_phys) :: zolf
L03970          real(kind_phys) :: hl1,tem1
L03971          real(kind_phys), PARAMETER :: a0=-3.975,  a1=12.32,  &
L03972                                b1=-7.755,  b2=6.041
L03973
L03974          if (zolf .ge. -0.5) then
L03975             hl1   = zolf
L03976             psim_unstable_full_gfs  = (a0  + a1*hl1)  * hl1   / (1.+ (b1+b2*hl1)  *hl1)
L03977          else
L03978             hl1   = -zolf
L03979             tem1  = 1.0 / sqrt(hl1)
L03980             psim_unstable_full_gfs  = log(hl1) + 2. * sqrt(tem1) - .8776
L03981          end if
L03982
L03983     end function
L03984
L03985  !>\ingroup mynn_sfc
L03986  !!
L03987     real(kind_phys) function psih_unstable_full_gfs(zolf)
L03988          !$acc routine seq
L03989          real(kind_phys) :: zolf
L03990          real(kind_phys) :: hl1,tem1
L03991          real(kind_phys), PARAMETER :: a0p=-7.941, a1p=24.75, &
L03992                                b1p=-8.705, b2p=7.899
L03993
L03994          if (zolf .ge. -0.5) then
L03995             hl1   = zolf
L03996             psih_unstable_full_gfs  = (a0p + a1p*hl1) * hl1   / (1.+ (b1p+b2p*hl1)*hl1)
L03997          else
L03998             hl1   = -zolf
L03999             tem1  = 1.0 / sqrt(hl1)
L04000             psih_unstable_full_gfs  = log(hl1) + .5 * tem1 + 1.386
L04001          end if
L04002
L04003     end function
L04004
L04005  !>\ingroup mynn_sfc
L04006  !! look-up table functions - or, if beyond -10 < z/L < 10, recalculate
L04007     real(kind_phys) function psim_stable(zolf,psi_opt)
L04008          !$acc routine seq
L04009          integer :: nzol,psi_opt
L04010          real(kind_phys) :: rzol,zolf
L04011
L04012          nzol = int(zolf*100.)
L04013          rzol = zolf*100. - nzol
L04014          if(nzol+1 .lt. 1000)then
L04015             psim_stable = psim_stab(nzol) + rzol*(psim_stab(nzol+1)-psim_stab(nzol))
L04016          else
L04017             if (psi_opt == 0) then
L04018                psim_stable = psim_stable_full(zolf)
L04019             else
L04020                psim_stable = psim_stable_full_gfs(zolf)
L04021             endif
L04022          endif
L04023
L04024     end function
L04025
L04026  !>\ingroup mynn_sfc
L04027     real(kind_phys) function psih_stable(zolf,psi_opt)
L04028          !$acc routine seq
L04029          integer :: nzol,psi_opt
L04030          real(kind_phys) :: rzol,zolf
L04031
L04032          nzol = int(zolf*100.)
L04033          rzol = zolf*100. - nzol
L04034          if(nzol+1 .lt. 1000)then
L04035             psih_stable = psih_stab(nzol) + rzol*(psih_stab(nzol+1)-psih_stab(nzol))
L04036          else
L04037             if (psi_opt == 0) then
L04038                psih_stable = psih_stable_full(zolf)
L04039             else
L04040                psih_stable = psih_stable_full_gfs(zolf)
L04041             endif
L04042          endif
L04043
L04044     end function
L04045
L04046  !>\ingroup mynn_sfc
L04047     real(kind_phys) function psim_unstable(zolf,psi_opt)
L04048          !$acc routine seq
L04049          integer :: nzol,psi_opt
L04050          real(kind_phys) :: rzol,zolf
L04051
L04052          nzol = int(-zolf*100.)
```

```
L04053          rzol = -zolf*100. - nzol
L04054          if(nzol+1 .lt. 1000)then
L04055             psim_unstable = psim_unstab(nzol) + rzol*(psim_unstab(nzol+1)-psim_unstab(nzol))
L04056          else
L04057             if (psi_opt == 0) then
L04058                psim_unstable = psim_unstable_full(zolf)
L04059             else
L04060                psim_unstable = psim_unstable_full_gfs(zolf)
L04061             endif
L04062          endif
L04063
L04064       end function
L04065
L04066    !>\ingroup mynn_sfc
L04067       real(kind_phys) function psih_unstable(zolf,psi_opt)
L04068          !$acc routine seq
L04069          integer :: nzol,psi_opt
L04070          real(kind_phys) :: rzol,zolf
L04071
L04072          nzol = int(-zolf*100.)
L04073          rzol = -zolf*100. - nzol
L04074          if(nzol+1 .lt. 1000)then
L04075             psih_unstable = psih_unstab(nzol) + rzol*(psih_unstab(nzol+1)-psih_unstab(nzol))
L04076          else
L04077             if (psi_opt == 0) then
L04078                psih_unstable = psih_unstable_full(zolf)
L04079             else
L04080                psih_unstable = psih_unstable_full_gfs(zolf)
L04081             endif
L04082          endif
L04083
L04084       end function
L04085    !=================================================================
```

讲解：两套四组共八个闭式函数 + 四个查表包装：**MYNN 组**（psi_opt=0，默认）——稳定侧 Cheng–Brutsaert 2005（ψ=−6.1 • ln(ζ+(1+ζ^2.5)0.4) 与 −5.3 • ln(ζ+(1+ζ^1.1)0.909…)），不稳定侧是**混合式**：ψ=(ψ_K+ζ² • ψ_C)/(1+ζ²)，ψ_K 是 Dyer–Hicks 1974（x=(1−16ζ)^(1/4) 的 Businger–Dyer 形），ψ_C 是 Grachev 2000 自由对流形（ym=(1−10ζ)^(1/3)），权重 ζ² 使近中性趋 Dyer–Hicks、强不稳定趋 Grachev——这就是头注说的「COARE 混合」；**GFS 组**（psi_opt=1）——稳定侧 α=20 的指数式、不稳定侧 a0/a1/b1/b2 有理式，ζ<−0.5 切自由对流幂律。查表包装：nzol=int(ζ • 100) 线性插值，|ζ|>10 出表回闭式。psi_init 末尾的自检（四表符号校验）是运行期 canary——表初始化失败立即报错。

五、三伴生件 C 级注记

三件 = WRF 运行线的实体。每件 = 单点版 SFCLAY1D（无三维包装、无 OpenACC、无 debug 大段）+ 自带辅助函数库拷贝。**任何物理修改都要问一句：主文件与三件、乃至 module_sf_exchcoef.F 里的拷贝，一共几处要同步？**

5.1 module_sf_mynnsfc_land.F90（1749 行，B 级不展开）

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_land.F90    L1-L46
L00001  !>\file module_sf_mynnsfc_land.F90
L00002  !! This file contains
L00003  !WRF:MODEL_LAYER:PHYSICS
L00004  !
L00005  !>\ingroup mynn_sfc
L00006  !> This module contains routines to calculate the surface exchange coefficients, u*,
L00007  !! scalar fluxes, and near-surface diagnostics as described in the latest
L00008  !! MYNN surface layer scheme tech note:
L00009  !! Olson, J. B., T. Smirnova, J. S. Kenyon, D. Turner, J. M. Brown, W. Zheng, and
L00010  !! B. W. Green, 2021: A description of the MYNN Surface-Layer Scheme. NOAA Tech. Memo.
L00011  !! OAR GSL-67, 26 pp., doi:10.25923/f6a8-bc75.
L00012  !!
L00013  MODULE module_sf_mynnsfc_land
L00014
L00015  !-------------------------------------------------------------------
L00016  !Modifications implemented by Joseph Olson NOAA/GSL
L00017  !This module contains the land-only portion of the MYNN surface-layer scheme:
L00018  !
L00019  !1) Calculation of stability parameter (z/L) taken from Li et al. (2010 BLM)
L00020  !   for first iteration of first time step; afterwards, exact calculation
L00021  !   using basically the same iterative technique in the module_sf_sfclayrev.F,
L00022  !   which leverages Pedro Jimenez's code, and is adapted for MYNN.
L00023  !2) Fixed isflux=0 option to turn off scalar fluxes, but keep momentum
L00024  !   fluxes for idealized studies (credit: Anna Fitch).
L00025  !3) Kinematic viscosity varies with temperature according to Andreas (1989).
L00026  !4) Uses the blended Monin-Obukhov flux-profile relationships of Dyer-Hicks 1974 and
L00027  !   Grachev et al (2000) in unstable conditions and uses Cheng and Brutsaert (2005) for stable conditions.
L00028  !5) The following overviews the namelist variables that control the
L00029  !   aerodynamic roughness lengths (over water) and the thermal and moisture
L00030  !   roughness lengths (defaults are recommended):
L00031  !
L00032  !   LAND only:
L00033  !   "sf_mynn_sfcfulx_land" namelist option is used to select the following momentum options:
L00034  !   (default) =0: Zilitinkevich (1995); Czil now set to 0.095
L00035  !            =1: Same as 0, but with z0-dependent Czil (according to Chen & Zhang 2008)
L00036  !            =2: Modified Yang et al (2002, 2008) - generalized for all landuse
L00037  !            =3: constant zt = z0/7.4 (original form; Garratt 1992)
L00038  !            =4: GFS - taken from sfc_diff.f, for comparison/testing
L00039  !
L00040  !6) Option to activate stochastic parameter perturbations (SPP), which
```

```
L00041  !   perturb z0, zt, and zq, along with many other parameters in the MYNN-
L00042  !   EDMF scheme.
L00043  !
L00044  !NOTE: This code was primarily tested in combination with the RUC LSM.
L00045  !      Performance with the Noah (or other) LSM is relatively unknown.
L00046  !-------------------------------------------------------------------
```

子程序清单（13 个）：mynnsfc_land（784 行主体）+ zilitinkevich_1995、garratt_1992、yang_2008、gfs_z0_land、gfs_zt_land + ψ 库七件（psi_hogstrom_1996/psi_dyerhicks/psi_beljaars_holtslag_1991/psi_zilitinkevich_esau_2007/psi_businger_1971/psi_suselj_sood_2010/psi_cb2005）+ li_etal_2010。

与主讲底本的差异：①zt/zq 分派按 **sf_mynn_sfcflux_land**（编号 0–4，语义见下节警示框）；②MAVAIL 在此真正参与 FLQC（水面件无此量）；③CHS 直连 LSM 接口；④ 无 GFS 水面族、无波浪模型族。

5.2 module_sf_mynnsfc_water.F90（2104 行，B 级不展开）

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_water.F90    L1-L54
L00001  !>\file module_sf_mynnsfc_water.F90
L00002  !! This file contains
L00003  !WRF:MODEL_LAYER:PHYSICS
L00004  !
L00005  !>\ingroup mynn_sfc
L00006  !> This module contains routines to calculate the surface exchange coefficients, u*,
L00007  !! scalar fluxes, and near-surface diagnostics as described in the latest
L00008  !! MYNN surface layer scheme tech note:
L00009  !! Olson, J. B., T. Smirnova, J. S. Kenyon, D. Turner, J. M. Brown, W. Zheng, and
L00010  !! B. W. Green, 2021: A description of the MYNN Surface-Layer Scheme. NOAA Tech. Memo.
L00011  !! OAR GSL-67, 26 pp., doi:10.25923/f6a8-bc75.
L00012  !!
L00013  MODULE module_sf_mynnsfc_water
L00014
L00015  !-------------------------------------------------------------------
L00016  !Modifications implemented by Joseph Olson NOAA/GSL
L00017  !The following overviews the current state of this scheme::
L00018  !
L00019  !   WATER ONLY:
L00020  !1) Calculation of stability parameter (z/L) taken from Li et al. (2010 BLM)
L00021  !   for first iteration of first time step; afterwards, exact calculation
L00022  !   using basically the same iterative technique in the module_sf_sfclayrev.F,
L00023  !   which leverages Pedro Jimenez's code, and is adapted for MYNN.
L00024  !2) Fixed isflux=0 option to turn off scalar fluxes, but keep momentum
L00025  !   fluxes for idealized studies (credit: Anna Fitch).
L00026  !3) Kinematic viscosity varies with temperature according to Andreas (1989).
L00027  !4) Uses the blended Monin-Obukhov flux-profile relationships COARE (Fairall
L00028  !   et al 2003) for the unstable regime (a blended mix of Dyer-Hicks 1974 and
L00029  !   Grachev et al (2000).
L00030  !5) The following overviews the namelist variables that control the
L00031  !   aerodynamic roughness lengths (over water) and the thermal and moisture
L00032  !   roughness lengths (defaults are recommended):
L00033  !
L00034  !   WATER only:
L00035  !   "sf_mynn_sfcflux_water" namelist option is used to select the following scalar options:
L00036  !             =0: z0, zt, and zq from the COARE 3.0 (Fairall et al. 2003)
L00037  !   (default) =1: z0, zt, and zq from the COARE 3.5 (Edson et al 2013)
L00038  !             =2: z0 from Davis et al (2008), zt & zq from COARE 3.5
L00039  !             =3: z0 from Davis et al (2008), zt & zq from Garratt (1992)
L00040  !             =4: z0 from Taylor and Yelland (2004), zt and zq from COARE 3.5
L00041  !             =5: GFS - taken from sfc_diff.f, for comparison/testing.
L00042  !
L00043  ! Misc:
L00044  !1) Added a more elaborate diagnostic for u10 & V10 for high vertical resolution
L00045  !   model configurations but for most model configurations with depth of
L00046  !   the lowest half-model level near 10 m, a neutral-log diagnostic is used.
L00047  !
L00048  !2) Option to activate stochastic parameter perturbations (SPP), which
L00049  !   perturb z0, zt, and zq, along with many other parameters in the MYNN-
L00050  !   EDMF scheme.
L00051  !
L00052  !NOTE: This code was primarily tested in static SST mode, so some modifications
L00053  !      are anticipated when moving to a fully-coupled air-sea interaction.
L00054  !-------------------------------------------------------------------
```

子程序清单（21 个）：mynnsfc_water（798 行主体）+ davis_etal_2008、taylor_yelland_2001、coare30_znt、edson_etal_2013、garratt_1992、fairall_etal_2003、fairall_etal_2014、gfs_z0_wat、gfs_zt_wat、znot_m_v6、znot_t_v6、znot_m_v7、znot_t_v7 + ψ 库六件 + li_etal_2010 + depth_dependent_z0。

湖面盐度校正（lakemask<0.5 视为海水，饱和比压降 1.88%）：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_water.F90    L293-L307
L00293  ! psfc ( in cmb) is used later in saturation checks
L00294  psfc=psfcpa/1000._kind_phys
L00295
L00296  !tgs - do computations if flag_iter = .true.
L00297  if ( flag_iter ) then
L00298     tsk = tskin
L00299     !saturation vapor pressure wrt water (bolton 1980)
L00300     e1=svp1*exp(svp2*(tsk-svpt0)/(tsk-svp3))
L00301     !For seawater with a salinity of 35 psu (practical salinity units), the saturation
L00302     !vapor pressure is reduced by approximately 1.88% compared to pure water.
L00303     if (lakemask .lt. p5) e1=e1*0.9812_kind_phys
L00304     qsfc=ep2*e1/(psfc-ep3*e1)             !specific humidity
L00305     qsfcmr=ep2*e1/(psfc-e1)               !mixing ratio
L00306     !if(qsfc>one.or.qsfc<0.) print *,' qsfc=',qsfc," tsk=",tsk," itimestep=",itimestep,i,j
L00307  endif ! flag_iter
```

z0 分派与浅水修正：

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_water.F90    L428-L453
L00428         if ( present(sf_mynn_sfcflux_water) ) then
L00429            if ( sf_mynn_sfcflux_water .eq. 0 ) then
L00430               !coare 3.0
L00431               call coare30_znt(znt,ust,wspd,visc,za)
L00432            elseif ( sf_mynn_sfcflux_water .eq. 1) then
L00433               !coare 3.5
L00434               call edson_etal_2013(znt,ust,wspd,visc,za)
L00435            elseif ( sf_mynn_sfcflux_water .eq. 2 .or. sf_mynn_sfcflux_water .eq. 3 ) then
L00436               call davis_etal_2008(znt,ust)
L00437            elseif ( sf_mynn_sfcflux_water .eq. 4 ) then
L00438               call taylor_yelland_2001(znt,ust,wspd)
L00439            elseif ( sf_mynn_sfcflux_water .eq. 5 ) then
L00440               !gfs surface layer physics
L00441               call gfs_z0_wat(znt,ust,wspd,za,sfc_z0_type,redrag)
L00442            endif
L00443         else
L00444            !default to coare 3.5
L00445            call edson_etal_2013(znt,ust,wspd,visc,za)
L00446         endif
L00447
L00448         !shallow water z0 blending to the open-ocean z0 calculated above (from Pedro & Jimy)
L00449         if (present(shalwater_z0)) then
L00450            if (shalwater_z0 .eq. 1) then
L00451               znt = depth_dependent_z0(wat_depth,znt,ust)
L00452            endif
L00453         endif
```

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_water.F90    L1027-L1051
L01027   real(kind=kind_phys) function depth_dependent_z0(water_depth,z0,ust)
L01028
L01029   implicit none
L01030   real(kind=kind_phys),intent(in):: water_depth,z0,ust
L01031   real(kind=kind_phys):: depth_b, wt
L01032   real(kind=kind_phys):: effective_depth
L01033
L01034   if (water_depth .lt. 10.0) then
L01035      effective_depth = 10.0_kind_phys
L01036   elseif (water_depth .gt. 100.0) then
L01037      effective_depth = 100.0_kind_phys
L01038   else
L01039      effective_depth = water_depth
L01040   endif
L01041
L01042   depth_b = one / 30.0_kind_phys * log (1260.0_kind_phys / effective_depth)
L01043   depth_dependent_z0 = exp((2.7_kind_phys * ust - 1.8_kind_phys / depth_b) / (ust + 0.17_kind_phys / depth_b) )
L01044   depth_dependent_z0 = MIN(depth_dependent_z0, p1)
L01045
L01046   !blend to open ocean between 75 and 150 m
L01047   wt = min(one, max(zero, (water_depth-75._kind_phys)/75._kind_phys))
L01048   depth_dependent_z0 = (one-wt)*depth_dependent_z0 + wt*z0
L01049
L01050   return
L01051   end function depth_dependent_z0
```

讲解：三处水域特有物理：① **湖/海**：35 psu 海水饱和比压比纯水低 1.88%，lakemask<0.5（海）才乘 0.9812——湖面按淡水处理；②**z0 分派**：sf_mynn_sfcflux_water 编号 0–5（见 6.1），缺省（实参缺省）走 COARE 3.5；③ **浅水**：shalwater_z0=1 时 depth_dependent_z0 把开阔海 z0 向深水值混合——浅水波陡大、粗糙度上修，blend 系数随水深与 u* 变化；此外 sf_mynn_sfcflux_water>1 时感热加耗散加热项。头注自述「此件在静态 SST 下测试，全耦合海气相互作用尚需改造」——做海气耦合改造时的已知风险点。

5.3 module_sf_mynnsfc_ice.F90（1552 行，B 级不展开）

```
!file: phys/MYNN-SFC/module_sf_mynnsfc_ice.F90    L1-L47
L00001  !>\file module_sf_mynnsfc_ice.F90
L00002  !! This file contains
L00003  !WRF:MODEL_LAYER:PHYSICS
L00004  !
L00005  !>\ingroup mynn_sfc
L00006  !> This module contains routines to calculate the surface exchange coefficients, u*,
L00007  !! scalar fluxes, and near-surface diagnostics as described in the latest
L00008  !! MYNN surface layer scheme tech note:
L00009  !! Olson, J. B., T. Smirnova, J. S. Kenyon, D. Turner, J. M. Brown, W. Zheng, and
L00010  !! B. W. Green, 2021: A description of the MYNN Surface-Layer Scheme. NOAA Tech. Memo.
L00011  !! OAR GSL-67, 26 pp., doi:10.25923/f6a8-bc75.
L00012  !!
L00013  MODULE module_sf_mynnsfc_ice
L00014
L00015  !-------------------------------------------------------------------
L00016  !Modifications implemented by Joseph Olson NOAA/GSL
L00017  !The following overviews the current state of this ice-only component of the
L00018  !MYNN surface-layer scheme::
L00019  !
L00020  !1) Calculation of stability parameter (z/L) taken from Li et al. (2010 BLM)
L00021  !   for first iteration of first time step; afterwards, exact calculation
L00022  !   using basically the same iterative technique in the module_sf_sfclayrev.F,
L00023  !   which leverages Pedro Jimenez's code, and is adapted for MYNN.
L00024  !2) Fixed isflux=0 option to turn off scalar fluxes, but keep momentum
L00025  !   fluxes for idealized studies (credit: Anna Fitch).
L00026  !3) Kinematic viscosity varies with temperature according to Andreas (1989).
L00027  !4) Uses the blended Monin-Obukhov flux-profile relationships COARE (Fairall
L00028  !   et al 2003) for the unstable regime (a blended mix of Dyer-Hicks 1974 and
L00029  !   Grachev et al (2000). Uses Cheng and Brutsaert (2005) for stable conditions.
L00030  !5) This module is only used over SNOW/ICE:
L00031  !   Andreas (2002) snow/ice parameterization for thermal and
L00032  !   moisture roughness is used over all gridpoints with snow deeper than
```

```
L00033  !   0.1 m. This algorithm calculates a z0 for snow (Andreas et al. 2005, BLM),
L00034  !   which is only used as part of the thermal and moisture roughness
L00035  !   length calculation, not to directly impact the surface winds.
L00036  !
L00037  ! Misc:
L00038  !1) Added a more elaborate diagnostic for u10 & V10 for high vertical resolution
L00039  !   model configurations but for most model configurations with depth of
L00040  !   the lowest half-model level near 10 m, a neutral-log diagnostic is used.
L00041  !
L00042  !2) Option to activate stochastic parameter perturbations (SPP), which
L00043  !   perturb z0, zt, and zq, along with many other parameters in the MYNN-
L00044  !   EDMF scheme.
L00045  !
L00046  !NOTE: This code was primarily tested in combination with the RUC LSM.
L00047  !      Performance with the Noah (or other) LSM is relatively unknown.
```

子程序清单（10 个）：mynnsfc_ice（721 行主体）+ Andreas_2002 + ψ 库六件 + li_etal_2010。

与主讲底本的差异：①zt/zq 只有 Andreas 2002 一条路，无选项；② 无 MAVAIL（恒饱和）；③ 潜热用 XLF；④ 准入条件由 wrapper 控制（雪 ≥5 cm 的陆点或部分海冰点）。

六、改造入口与验收

6.1 选项编号对照（改造前必查）

选项（WRF namelist）	Registry 默认	编号语义（WRF 4.8.0 实测）
sf_sfclay_physics=5	—	启用 MYNN 面层（package mynnsfcscheme）
sf_mynn_sfcflux_land	0	0=Zilitinkevich1995+Czil0.095；1=Czil 按 Chen–Zhang2008；2=Yang2008；3=Garratt1992（z0/7.4）；4=GFS 对照
sf_mynn_sfcflux_water	1	0=COARE3.0；1=COARE3.5；2=Davis2008+COARE3.5；3=Davis2008+Garratt；4=Taylor–Yelland+COARE3.5；5=GFS 对照

> **警示**：CCPP 主文件头注释里的 isftcflx（0–4）与 iz0tlnd（0–4）是**另一套编号**（isftcflx: 0=COARE3.0/3.5、1=Davis+COARE、2=Davis+Garratt、3=TY+COARE、4=GFS）。阅读代码时先确认走的是哪条线，抄 namelist 时以本表（WRF 线）为准。

6.2 高频改造点

1. **改 z0/zt/zq 参数化**（论文 2 类）：水面改 sf_mynn_sfcflux_water 分派处 + 对应子程序；陆面同。三件套各有一份拷贝——先 grep 子程序名确认全部副本。回归验证不必跑全模式：phys/MYNN-SFC/tests/ 有独立回归测试（tests/data/ 下 input_lnd/wat/icy.txt 与 output_*.txt 黄金文件，driver.F90 单独编译运行），改前留档、改后比对。
2. **改相似性函数**：S7 走查表，改函数要同时改 *_full 闭式（超界回退用）与 psi_init 表生成——两处不一致会在 |z/L|>10 段露馅。psi_opt=1 可整组切 GFS 表做对照。
3. **改稳定度迭代**：zolrib 收敛阈/步数/回退策略，见 4.4。
4. **加诊断量**：S10 的 compute_diag 块是唯一合法挂点；新增输出需走 Registry（ch04）。
5. **重新编译**：本篇所有文件是纯物理层（不涉 Registry.EM_COMMON 变更时），增量编译即可（ch05_build_and_dev_loop.md）；若动 Registry 增加 namelist 项，记得 touch Registry/Registry.EM 触发 registry 工具重建（ch04 教训）。

七、上下游链接

- 上游（PBL 侧耦合与调用者）：`code_09a_module_bl_mynnedmf.md`、`code_09b_module_bl_mynnedmf.md`（MYNN-EDMF：u、*w*、pblh、ustm 交换）；PBL driver 走读（编号 22，同批后续交付）是本篇在物理驱动链上的直接上游。
- 下游消费者：LSM（Noah/RUC，经 CHS/CHS2/CQS2/QSFC/QGH/MAVAIL；Noah 全文精读为编号 12、13 篇，后续批次交付）与动力通量累加（first_rk_step_part1，编号 31 待交付）。
- 教材正文：ch14_pbl_family.md、ch16_land_surface_family.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md、ch21_solve_em_dynamics.md。

code_11 — New Tiedtke 积云参数化的 WRF 外壳（module_cu_ntiedtke.F：CCPP 式三段桥接）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_cu_ntiedtke.F	533	c1e122b43d49f24a48868f942fd04092
phys/physics_mmm/cu_ntiedtke.F90（实体，本篇不转录）	3594	104dcb4bf2cee44771b8567c5d8056dc

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级全文（L1–L533 共 4 单元全收）+ physics_mmm 实体对照注记（单元地图与接口契约，见 § 八）。

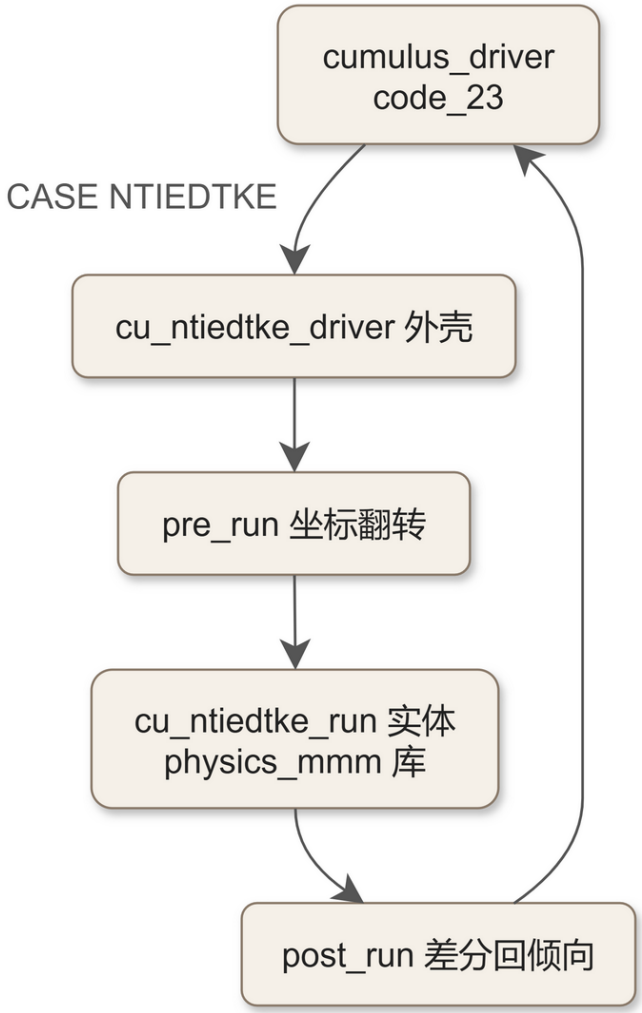
二、定位

一句话：New Tiedtke 积云方案（Tiedtke 1989 的质量通量闭合改造版，NCAR MMM 维护）在 WRF 4.8.0 的**接口外壳**——方案实体 cu_ntiedtke_run 在独立的 physics_mmm 物理库中，本文件只做三件事：WRF↔MMM 的垂直坐标翻转、积云步长与常数注入、状态差分式倾向回写。它是 WRF 4.8 物理库「CCPP 化」的活标本。

谁在调它：module_cumulus_driver.F 的 CASE('NTIEDTKE')（`code_23_module_cumulus_driver.md` §CASE 全走读）；教材锚点：ch15_cumulus_family.md。

架构事实（本树核验，2026-10-12 订正）：方案实体在 `phys/physics_mmm/cu_ntiedtke.F90`（3594 行 15 单元，见 § 八）——physics_mmm 是 NCAR MMM 维护的共享物理库目录，随本源码树携带；文库方案的取舍是「physics_mmm 目录不单独成篇」（外围 C 级排除），实体以对照注记形式收录。

三段桥接结构：



三、文件头与模块声明

```
!file: phys/module_cu_ntiedtke.F    L1-L18
L00001  !=================================================================================================================
L00002   module module_cu_ntiedtke
L00003   use ccpp_kind_types,only: kind_phys
L00004
L00005   use cu_ntiedtke,only: cu_ntiedtke_run, &
L00006                          cu_ntiedtke_init
L00007   use cu_ntiedtke_common
L00008
L00009   implicit none
L00010   private
L00011   public:: cu_ntiedtke_driver, &
L00012            ntiedtkeinit
L00013
L00014
L00015   contains
L00016
L00017
L00018  !=================================================================================================================
```

讲解：①use ccpp_kind_types 的 kind_phys——CCPP（Common Community Physics Package）物理容器的类型遗迹：New Tiedtke 从 WRF 4.4 起以 CCPP 兼容形态入库，WRF 侧保留这个外壳；②use cu_ntiedtke（实体：cu_ntiedtke_run/cu_ntiedtke_init）与 cu_ntiedtke_common（公共常数块）——两个模块都在 phys/physics_mmm/ 目录（§ 八）；③private + public 白名单只导出 cu_ntiedtke_driver 与 ntiedtkeinit——WRF 侧能看到的接口面被刻意收窄。

四、主入口 cu_ntiedtke_driver（全文）

```
!file: phys/module_cu_ntiedtke.F    L19-L275    (subroutine cu_ntiedtke_driver)
L00019   subroutine cu_ntiedtke_driver(                          &
L00020                    dt,itimestep,stepcu                    &
L00021                  ,raincv,pratec,qfx,hfx                   &
L00022                  ,u3d,v3d,w,t3d,qv3d,qc3d,qi3d,pi3d,rho3d &
L00023                  ,qvften,thften                           &
L00024                  ,dz8w,pcps,p8w,xland,cu_act_flag,dx      &
L00025                  ,f_qv,f_qc,f_qr,f_qi,f_qs                &
L00026                  ,grav,xlf,xls,xlv,rd,rv,cp               &
L00027                  ,rthcuten,rqvcuten,rqccuten,rqicuten     &
L00028                  ,rucuten,rvcuten                         &
L00029                  ,ids,ide,jds,jde,kds,kde                 &
L00030                  ,ims,ime,jms,jme,kms,kme                 &
L00031                  ,its,ite,jts,jte,kts,kte                 &
L00032                  ,errmsg,errflg)
L00033  !=================================================================================================================
L00034  !-- u3d         3d u-velocity interpolated to theta points (m/s)
L00035  !-- v3d         3d v-velocity interpolated to theta points (m/s)
L00036  !-- th3d        3d potential temperature (k)
L00037  !-- t3d         temperature (k)
L00038  !-- qv3d        3d water vapor mixing ratio (kg/kg)
L00039  !-- qc3d        3d cloud mixing ratio (kg/kg)
L00040  !-- qi3d        3d ice mixing ratio (kg/kg)
L00041  !-- rho3d       3d air density (kg/m^3)
L00042  !-- p8w         3d hydrostatic pressure at full levels (pa)
L00043  !-- pcps        3d hydrostatic pressure at half levels (pa)
L00044  !-- pi3d        3d exner function (dimensionless)
L00045  !-- qvften      3d total advective + PBL moisture tendency (kg kg-1 s-1)
L00046  !-- thften      3d total advective + PBL + radiative temperature tendency (k s-1)
L00047  !-- rthcuten      theta tendency due to
L00048  !                 cumulus scheme precipitation (k/s)
L00049  !-- rucuten       u wind tendency due to
L00050  !                 cumulus scheme precipitation (k/s)
L00051  !-- rvcuten       v wind tendency due to
L00052  !                 cumulus scheme precipitation (k/s)
L00053  !-- rqvcuten      qv tendency due to
L00054  !                 cumulus scheme precipitation (kg/kg/s)
L00055  !-- rqccuten      qc tendency due to
L00056  !                 cumulus scheme precipitation (kg/kg/s)
L00057  !-- rqicuten      qi tendency due to
L00058  !                 cumulus scheme precipitation (kg/kg/s)
L00059  !-- rainc         accumulated total cumulus scheme precipitation (mm)
L00060  !-- raincv        cumulus scheme precipitation (mm)
L00061  !-- pratec        precipitiation rate from cumulus scheme (mm/s)
L00062  !-- dz8w        dz between full levels (m)
L00063  !-- qfx         upward moisture flux at the surface (kg/m^2/s)
L00064  !-- hfx         upward heat flux at the surface (w/m^2)
L00065  !-- dt          time step (s)
L00066  !-- ids         start index for i in domain
L00067  !-- ide         end index for i in domain
L00068  !-- jds         start index for j in domain
L00069  !-- jde         end index for j in domain
L00070  !-- kds         start index for k in domain
L00071  !-- kde         end index for k in domain
L00072  !-- ims         start index for i in memory
L00073  !-- ime         end index for i in memory
L00074  !-- jms         start index for j in memory
L00075  !-- jme         end index for j in memory
L00076  !-- kms         start index for k in memory
L00077  !-- kme         end index for k in memory
L00078  !-- its         start index for i in tile
L00079  !-- ite         end index for i in tile
L00080  !-- jts         start index for j in tile
L00081  !-- jte         end index for j in tile
L00082  !-- kts         start index for k in tile
L00083  !-- kte         end index for k in tile
L00084  !-----------------------------------------------------------------------------------------------------------------
L00085
L00086  !--- input arguments:
L00087   logical,intent(in),optional:: f_qv,f_qc,f_qr,f_qi,f_qs
L00088
L00089   integer,intent(in):: ids,ide,jds,jde,kds,kde, &
L00090                        ims,ime,jms,jme,kms,kme, &
L00091                        its,ite,jts,jte,kts,kte
L00092
L00093   integer,intent(in):: itimestep,stepcu
L00094
L00095   real(kind=kind_phys),intent(in):: cp,grav,rd,rv,xlf,xls,xlv
L00096
L00097   real(kind=kind_phys),intent(in):: dt
L00098
L00099   real(kind=kind_phys),intent(in),dimension(ims:ime,jms:jme):: dx,hfx,qfx,xland
L00100
L00101   real(kind=kind_phys),intent(in),dimension(ims:ime,kms:kme,jms:jme):: &
L00102      dz8w,   &
L00103      pcps,   &
L00104      p8w,    &
L00105      pi3d,   &
L00106      qc3d,   &
L00107      qvften, &
L00108      thften, &
L00109      qi3d,   &
L00110      qv3d,   &
L00111      rho3d,  &
L00112      t3d,    &
L00113      u3d,    &
L00114      v3d,    &
L00115      w
L00116
L00117  !--- inout arguments:
L00118   logical,intent(inout),dimension(ims:ime,jms:jme):: cu_act_flag
L00119
L00120   real(kind=kind_phys),intent(inout),dimension(ims:ime,jms:jme):: raincv, pratec
L00121
```

```
L00122   real(kind=kind_phys),intent(inout),dimension(ims:ime,kms:kme,jms:jme),optional:: &
L00123   rqccuten,  &
L00124   rqicuten,  &
L00125   rqvcuten,  &
L00126   rthcuten,  &
L00127   rucuten,   &
L00128   rvcuten
L00129
L00130  !--- output arguments:
L00131   character(len=*),intent(out):: errmsg
L00132   integer,intent(out):: errflg
L00133
L00134  !--- local variables and arrays:
L00135   integer:: i,im,j,k,kx,kx1
L00136   integer,dimension(its:ite)::  slimsk
L00137
L00138   real(kind=kind_phys):: delt
L00139   real(kind=kind_phys),dimension(its:ite):: rn
L00140   real(kind=kind_phys),dimension(its:ite,kts:kte):: prsl,omg,ghtl
L00141   real(kind=kind_phys),dimension(its:ite,kts:kte):: uf,vf,tf,qvf,qcf,qif
L00142   real(kind=kind_phys),dimension(its:ite,kts:kte):: qvftenz,thftenz
L00143   real(kind=kind_phys),dimension(its:ite,kts:kte+1):: prsi,ghti,zi
L00144
L00145   real(kind=kind_phys),dimension(its:ite):: dx_hv,hfx_hv,qfx_hv,xland_hv
L00146   real(kind=kind_phys),dimension(its:ite,kts:kte):: dz_hv,pi_hv,prsl_hv
L00147   real(kind=kind_phys),dimension(its:ite,kts:kte):: qv_hv,qc_hv,qi_hv,rho_hv,t_hv,u_hv,v_hv
L00148   real(kind=kind_phys),dimension(its:ite,kts:kte):: qvften_hv,thften_hv
L00149   real(kind=kind_phys),dimension(its:ite,kts:kte+1):: prsi_hv,w_hv
L00150
L00151   real(kind=kind_phys),dimension(its:ite):: raincv_hv,pratec_hv
L00152   real(kind=kind_phys),dimension(its:ite,kts:kte):: rthcuten_hv,rqvcuten_hv,rqccuten_hv,rqicuten_hv, &
L00153                                                     rucuten_hv,rvcuten_hv
L00154
L00155  !-----------------------------------------------------------------------------------------------------------------
L00156
L00157   errmsg = ' '
L00158   errflg = 0
L00159
L00160   call cu_ntiedtke_init( &
L00161                        con_cp  = cp  , con_rd = rd   , con_rv   = rv   , con_xlv = xlv    , &
L00162                        con_xls = xls , con_xlf = xlf , con_grav = grav , errmsg  = errmsg , &
L00163                        errflg  = errflg                                                    &
L00164                       )
L00165
L00166   do j = jts,jte
L00167      do i = its,ite
L00168         cu_act_flag(i,j)=.true.
L00169      enddo
L00170   enddo
L00171
L00172   do j = jts,jte
L00173
L00174      do i = its,ite
L00175         dx_hv(i)    = dx(i,j)
L00176         hfx_hv(i)   = hfx(i,j)
L00177         qfx_hv(i)   = qfx(i,j)
L00178         xland_hv(i) = xland(i,j)
L00179      enddo
L00180
L00181      do k = kts,kte
L00182         do i = its,ite
L00183            dz_hv(i,k)   = dz8w(i,k,j)
L00184            pi_hv(i,k)   = pi3d(i,k,j)
L00185            prsl_hv(i,k) = pcps(i,k,j)
L00186            qv_hv(i,k)   = qv3d(i,k,j)
L00187            qc_hv(i,k)   = qc3d(i,k,j)
L00188            qi_hv(i,k)   = qi3d(i,k,j)
L00189            rho_hv(i,k)  = rho3d(i,k,j)
L00190            t_hv(i,k)    = t3d(i,k,j)
L00191            u_hv(i,k)    = u3d(i,k,j)
L00192            v_hv(i,k)    = v3d(i,k,j)
L00193
L00194            qvften_hv(i,k) = qvften(i,k,j)
L00195            thften_hv(i,k) = thften(i,k,j)
L00196         enddo
L00197      enddo
L00198      do k = kts,kte+1
L00199         do i = its,ite
L00200            prsi_hv(i,k) = p8w(i,k,j)
L00201            w_hv(i,k)    = w(i,k,j)
L00202         enddo
L00203      enddo
L00204
L00205      call cu_ntiedtke_pre_run( &
L00206                        its    = its       , ite     = ite     , kts     = kts     , kte       = kte       , &
L00207                        im     = im        , kx      = kx      , kx1     = kx1     , itimestep = itimestep , &
L00208                        stepcu = stepcu    , dt      = dt      , grav    = grav    , xland     = xland_hv  , &
L00209                        dz     = dz_hv     , pres    = prsl_hv , presi   = prsi_hv , t         = t_hv      , &
L00210                        rho    = rho_hv    , qv      = qv_hv   , qc      = qc_hv   , qi        = qi_hv     , &
L00211                        u      = u_hv      , v       = v_hv    , w       = w_hv    , qvften    = qvften_hv , &
L00212                        thften = thften_hv , qvftenz = qvftenz , thftenz = thftenz , slimsk    = slimsk    , &
L00213                        delt   = delt      , prsl    = prsl    , ghtl    = ghtl    , tf        = tf        , &
L00214                        qvf    = qvf       , qcf     = qcf     , qif     = qif     , uf        = uf        , &
L00215                        vf     = vf        , prsi    = prsi    , ghti    = ghti    , omg       = omg       , &
L00216                        errmsg = errmsg    , errflg  = errflg                                              &
L00217                               )
L00218
L00219      call cu_ntiedtke_run( &
L00220                        pu   = uf     , pv   = vf     , pt     = tf     , pqv    = qvf    , &
L00221                        pqc  = qcf    , pqi  = qif    , pqvf   = qvftenz , ptf   = thftenz , &
L00222                        poz  = ghtl   , pzz  = ghti   , pomg   = omg    , pap   = prsl    , &
L00223                        paph = prsi   , evap = qfx_hv , hfx    = hfx_hv , zprecc = rn      , &
L00224                        lndj = slimsk , lq   = im     , km     = kx     , km1   = kx1     , &
L00225                        dt   = delt   , dx   = dx_hv  , errmsg = errmsg , errflg = errflg   &
L00226                           )
L00227
L00228      call cu_ntiedtke_post_run( &
```

```
L00229                        its      = its         , ite      = ite         , kts      = kts         , kte      = kte
↪   , &
L00230                        stepcu   = stepcu      , dt       = dt          , exner    = pi_hv       , qv       = qv_hv
↪   , &
L00231                        qc       = qc_hv       , qi       = qi_hv       , t        = t_hv        , u        = u_hv
↪   , &
L00232                        v        = v_hv        , qvf      = qvf         , qcf      = qcf         , qif      = qif
↪   , &
L00233                        tf       = tf          , uf       = uf          , vf       = vf          , rn       = rn
↪   , &
L00234                        raincv   = raincv_hv   , pratec   = pratec_hv   , rthcuten = rthcuten_hv , rqvcuten = rqvcuten_hv
↪   , &
L00235                        rqccuten = rqccuten_hv , rqicuten = rqicuten_hv , rucuten  = rucuten_hv  , rvcuten  = rvcuten_hv
↪   , &
L00236                        errmsg   = errmsg      , errflg   = errflg
↪   &
L00237                                )
L00238
L00239      do i = its,ite
L00240         raincv(i,j) = raincv_hv(i)
L00241         pratec(i,j) = pratec_hv(i)
L00242      enddo
L00243
L00244      do k = kts,kte
L00245         do i = its,ite
L00246            rucuten(i,k,j)  = rucuten_hv(i,k)
L00247            rvcuten(i,k,j)  = rvcuten_hv(i,k)
L00248            rthcuten(i,k,j) = rthcuten_hv(i,k)
L00249            rqvcuten(i,k,j) = rqvcuten_hv(i,k)
L00250         enddo
L00251      enddo
L00252
L00253      if(present(rqccuten))then
L00254         if(f_qc) then
L00255            do k = kts,kte
L00256               do i = its,ite
L00257                  rqccuten(i,k,j) = rqccuten_hv(i,k)
L00258               enddo
L00259            enddo
L00260         endif
L00261      endif
L00262
L00263      if(present(rqicuten))then
L00264         if(f_qi) then
L00265            do k = kts,kte
L00266               do i = its,ite
L00267                  rqicuten(i,k,j) = rqicuten_hv(i,k)
L00268               enddo
L00269            enddo
L00270         endif
L00271      endif
L00272
L00273   enddo
L00274
L00275   end subroutine cu_ntiedtke_driver
```

讲解：七站走读。① **参数注释块**（L33–L84）：注意 qvften/thften 的口径——「平流 +PBL 的水汽倾向」与「平流 +PBL+辐射的温度倾向」，这是 New Tiedtke 闭合式的外部强迫输入（区别于 KF 系只吃状态量）；optional 的 f_qv/f_qc/f_qr/f_qi/f_qs 是积分器压缩数组开关。②**CCPP 错误协议**（L157–158）：errmsg/errflg 出参、errflg=0 正常——与旧式 wrf_error_fatal 风格并列的物理库新约定。③ **常数注入**（L160–164）：cu_ntiedtke_init 在**每个时间步**都被调用，把 cp/rd/rv/xlv/xls/xlf/grav 以 con_* 关键字实参注入——物理库自身不硬编码常数，双精度口径统一走 kind_phys。④cu_act_flag 全部置 true（L166–170）——WRF 侧不做触发门控，触发判据完全在实体闭合式内部；对比 KF 路径在 cumulus_driver 里的 CYCLE 门控（code_23）。⑤ **列组织**（L172–203）：J 循环内把 3D 场抄进 (i,k) 的 _hv 转置数组——MMM 库按「列向量 × 列数」组织，与 WRF 的 (i,k,j) 惯例互异；prsi/w 在整层（kte+1），其余在半层。⑥ **三段流水**（L205–237）：pre_run（翻转 +整形）→ run（实体物理）→ post_run（差分回倾向），全程关键字实参——CCPP 风格的自文档化调用。⑦ **回写门控**（L239–271）：rucuten/rvcuten/rthcuten/rqvcuten 无条件写回，rqccuten/rqicuten 要过 present + f_qc/f_qi 双门控——qc/qi 变量不存在于 Registry 时（低配微物理）跳过，避免写未分配数组。

五、ntiedtkeinit（全文）

```
!file: phys/module_cu_ntiedtke.F    L278-L350    (subroutine ntiedtkeinit)
L00278   subroutine ntiedtkeinit(rthcuten,rqvcuten,rqccuten,rqicuten, &
L00279                            rucuten,rvcuten,rthften,rqvften,     &
L00280                            restart,p_qc,p_qi,p_first_scalar,    &
L00281                            allowed_to_read,                     &
L00282                            ids, ide, jds, jde, kds, kde,        &
L00283                            ims, ime, jms, jme, kms, kme,        &
L00284                            its, ite, jts, jte, kts, kte)
L00285   !=================================================================================================================
L00286
L00287   !--- input arguments:
L00288    logical,intent(in):: allowed_to_read,restart
L00289
L00290    integer,intent(in):: ids, ide, jds, jde, kds, kde, &
L00291                         ims, ime, jms, jme, kms, kme, &
L00292                         its, ite, jts, jte, kts, kte
L00293    integer,intent(in):: p_first_scalar,p_qi,p_qc
L00294
```

```
L00295  !--- output arguments:
L00296   real(kind=kind_phys),intent(out),dimension(ims:ime,kms:kme,jms:jme ):: &
L00297      rthcuten,rqvcuten,rqccuten,rqicuten,rucuten,rvcuten,rthften,rqvften
L00298
L00299  !--- local variables and arrays:
L00300   integer:: i,j,k,itf,jtf,ktf
L00301
L00302  !-------------------------------------------------------------------------------------------------------------
L00303
L00304   jtf = min0(jte,jde-1)
L00305   ktf = min0(kte,kde-1)
L00306   itf = min0(ite,ide-1)
L00307
L00308   if(.not.restart)then
L00309      do j = jts,jtf
L00310         do k = kts,ktf
L00311            do i = its,itf
L00312               rthcuten(i,k,j) = 0.
L00313               rqvcuten(i,k,j) = 0.
L00314               rucuten(i,k,j)  = 0.
L00315               rvcuten(i,k,j)  = 0.
L00316            enddo
L00317         enddo
L00318      enddo
L00319
L00320      do j = jts,jtf
L00321         do k = kts,ktf
L00322            do i = its,itf
L00323               rthften(i,k,j)=0.
L00324               rqvften(i,k,j)=0.
L00325            enddo
L00326         enddo
L00327      enddo
L00328
L00329      if(p_qc .ge. p_first_scalar) then
L00330         do j = jts,jtf
L00331            do k = kts,ktf
L00332               do i = its,itf
L00333                  rqccuten(i,k,j)=0.
L00334               enddo
L00335            enddo
L00336         enddo
L00337      endif
L00338
L00339      if(p_qi .ge. p_first_scalar) then
L00340         do j = jts,jtf
L00341            do k = kts,ktf
L00342               do i = its,itf
L00343                  rqicuten(i,k,j)=0.
L00344               enddo
L00345            enddo
L00346         enddo
L00347      endif
L00348   endif
L00349
L00350   end subroutine ntiedtkeinit
```

讲解：倾向数组清零（restart 跳过——重启时倾向态从重启动文件来）；rthften/rqvften 是本方案的「外部倾向」暂存数组，同样清零；p_qc/p_qi≥p_first_scalar 才清 rqccuten/rqicuten——与 driver 的 present 门控同构，判断「该变量是否在 Registry 包内」（q_first_scalar 之前的变量是 2D/诊断量，不必清）。

六、cu_ntiedtke_pre_run（全文）

```
!file: phys/module_cu_ntiedtke.F    L353-L473    (subroutine cu_ntiedtke_pre_run)
L00353   subroutine cu_ntiedtke_pre_run(its,ite,kts,kte,im,kx,kx1,itimestep,stepcu,dt,grav,xland,dz,pres,presi,   &
L00354                                  t,rho,qv,qc,qi,u,v,w,qvften,thften,qvftenz,thftenz,slimsk,delt,prsl,ghtl, &
L00355                                  tf,qvf,qcf,qif,uf,vf,prsi,ghti,omg,errmsg,errflg)
L00356  !=============================================================================================================
L00357
L00358  !--- input arguments:
L00359   integer,intent(in):: its,ite,kts,kte
L00360   integer,intent(in):: itimestep
L00361   integer,intent(in):: stepcu
L00362
L00363   real(kind=kind_phys),intent(in):: dt,grav
L00364   real(kind=kind_phys),intent(in),dimension(its:ite):: xland
L00365   real(kind=kind_phys),intent(in),dimension(its:ite,kts:kte):: dz,pres,t,rho,qv,qc,qi,u,v
L00366   real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: qvften,thften
L00367   real(kind=kind_phys),intent(in),dimension(its:ite,kts:kte+1):: presi,w
L00368
L00369  !--- inout arguments:
L00370   integer,intent(inout):: im,kx,kx1
L00371   integer,intent(inout),dimension(its:ite):: slimsk
L00372
L00373   real(kind=kind_phys),intent(inout):: delt
L00374   real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: tf,qvf,qcf,qif,uf,vf
L00375   real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: ghtl,omg,prsl
L00376   real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: qvftenz,thftenz
L00377   real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte+1):: ghti,prsi
L00378
L00379  !--- output arguments:
L00380   character(len=*),intent(out):: errmsg
L00381   integer,intent(out):: errflg
L00382
L00383  !--- local variables and arrays:
L00384   integer:: i,k,pp,zz
L00385
L00386   real(kind=kind_phys),dimension(its:ite,kts:kte):: zl,dot
```

```
L00387   real(kind=kind_phys),dimension(its:ite,kts:kte+1):: zi
L00388
L00389  !-----------------------------------------------------------------------------------------------------------------
L00390
L00391   im  = ite-its+1
L00392   kx  = kte-kts+1
L00393   kx1 = kx+1
L00394
L00395   delt  = dt*stepcu
L00396
L00397   do i = its,ite
L00398      slimsk(i) = (abs(xland(i)-2.))
L00399   enddo
L00400
L00401   k = kts
L00402   do i = its,ite
L00403      zi(i,k) = 0.
L00404   enddo
L00405   do k = kts,kte
L00406      do i = its,ite
L00407         zi(i,k+1) = zi(i,k)+dz(i,k)
L00408      enddo
L00409   enddo
L00410   do k = kts,kte
L00411      do i = its,ite
L00412         zl(i,k)  = 0.5*(zi(i,k)+zi(i,k+1))
L00413         dot(i,k) = -0.5*grav*rho(i,k)*(w(i,k)+w(i,k+1))
L00414      enddo
L00415   enddo
L00416
L00417   pp = 0
L00418   do k = kts,kte+1
L00419      zz = kte + 1 - pp
L00420      do i = its,ite
L00421         ghti(i,zz) = zi(i,k)
L00422         prsi(i,zz) = presi(i,k)
L00423      enddo
L00424      pp = pp + 1
L00425   enddo
L00426   pp = 0
L00427   do k = kts,kte
L00428      zz = kte-pp
L00429      do i = its,ite
L00430         ghtl(i,zz) = zl(i,k)
L00431         omg(i,zz)  = dot(i,k)
L00432         prsl(i,zz) = pres(i,k)
L00433      enddo
L00434      pp = pp + 1
L00435   enddo
L00436
L00437   pp = 0
L00438   do k = kts,kte
L00439      zz = kte-pp
L00440      do i = its,ite
L00441         tf(i,zz)  = t(i,k)
L00442         qvf(i,zz) = qv(i,k)
L00443         qcf(i,zz) = qc(i,k)
L00444         qif(i,zz) = qi(i,k)
L00445         uf(i,zz)  = u(i,k)
L00446         vf(i,zz)  = v(i,k)
L00447      enddo
L00448      pp = pp + 1
L00449   enddo
L00450
L00451   if(itimestep == 1) then
L00452      do k = kts,kte
L00453         do i = its,ite
L00454            qvftenz(i,k) = 0.
L00455            thftenz(i,k) = 0.
L00456         enddo
L00457      enddo
L00458   else
L00459      pp = 0
L00460      do k = kts,kte
L00461         zz = kte-pp
L00462         do i = its,ite
L00463            qvftenz(i,zz) = qvften(i,k)
L00464            thftenz(i,zz) = thften(i,k)
L00465         enddo
L00466         pp = pp + 1
L00467      enddo
L00468   endif
L00469
L00470   errmsg = 'cu_ntiedtke_pre_run OK'
L00471   errflg = 0
L00472
L00473   end subroutine cu_ntiedtke_pre_run
```

讲解：本篇核心教学点——**垂直坐标翻转**。①im/kx/kx1：列数与层数（kx1=kte+1 整层）；delt=dt • stepcu——积云步长是模式步的 stepcu 倍（namelist cu_used_step 语义在 driver 侧），实体内部按 delt 做一个「聚合步」的物理。②slimsk=|xland−2|：陆地 1、水面 0（MMM 的陆海标记与 WRF 的 XLAND=1 陆/2 水方向相反的绝对值 trick）。③ **翻转**（pp/zz 对偶循环）：WRF 的 k 自上向下（kts 在顶），MMM 索引自下向上——ghti/prsi（整层高度/气压）、ghtl/omg/prsl（半层）、tf/qvf/qcf/qif/uf/vf（状态量）全部以 zz=kte+1−pp 或 kte−pp 倒序拷贝；zi 自地面向上累加 dz 得界面高度、zl 取层中。④**ω 型垂直速度**：omg=−0.5 • grav • ρ • (w(i,k)+w(i,k+1))——把 WRF 的 w（m/s）换成质量加权的气压坐标 ω 型，实体 pomg 输入口径（半层平均 +密度权重 +负号约定一次到位）。⑤qvftenz/thftenz：itimestep=1 清零、其后翻转传入——前期倾向（平流 +PBL[+辐射]）是闭合式的强迫项，「第一天不做积云修正」的初始化细节就在这。

七、cu_ntiedtke_post_run（全文）

```
!file: phys/module_cu_ntiedtke.F    L476-L529    (subroutine cu_ntiedtke_post_run)
L00476   subroutine cu_ntiedtke_post_run(its,ite,kts,kte,stepcu,dt,exner,qv,qc,qi,t,u,v,qvf,qcf,qif,tf,uf,vf,rn,raincv, &
L00477                                   pratec,rthcuten,rqvcuten,rqccuten,rqicuten,rucuten,rvcuten,errmsg,errflg)
L00478   !=================================================================================================================
L00479
L00480   !--- input arguments:
L00481    integer,intent(in):: its,ite,kts,kte
L00482    integer,intent(in):: stepcu
L00483
L00484    real(kind=kind_phys),intent(in):: dt
L00485    real(kind=kind_phys),intent(in),dimension(its:ite):: rn
L00486    real(kind=kind_phys),intent(in),dimension(its:ite,kts:kte):: exner,qv,qc,qi,t,u,v,qvf,qcf,qif,tf,uf,vf
L00487
L00488   !--- inout arguments:
L00489    real(kind=kind_phys),intent(inout),dimension(its:ite):: raincv,pratec
L00490    real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: rqvcuten,rqccuten,rqicuten
L00491    real(kind=kind_phys),intent(inout),dimension(its:ite,kts:kte):: rthcuten,rucuten,rvcuten
L00492
L00493   !--- output arguments:
L00494    character(len=*),intent(out):: errmsg
L00495    integer,intent(out):: errflg
L00496
L00497   !--- local variables and arrays:
L00498    integer:: i,k,pp,zz
L00499
L00500    real(kind=kind_phys):: delt,rdelt
L00501
L00502   !-----------------------------------------------------------------------------------------------------------------
L00503
L00504    delt  = dt*stepcu
L00505    rdelt = 1./delt
L00506
L00507    do i = its,ite
L00508       raincv(i) = rn(i)/stepcu
L00509       pratec(i) = rn(i)/(stepcu*dt)
L00510    enddo
L00511
L00512    pp = 0
L00513    do k = kts,kte
L00514       zz = kte - pp
L00515       do i = its,ite
L00516          rthcuten(i,k) = (tf(i,zz)-t(i,k))/exner(i,k)*rdelt
L00517          rqvcuten(i,k) = (qvf(i,zz)-qv(i,k))*rdelt
L00518          rqccuten(i,k) = (qcf(i,zz)-qc(i,k))*rdelt
L00519          rqicuten(i,k) = (qif(i,zz)-qi(i,k))*rdelt
L00520          rucuten(i,k)  = (uf(i,zz)-u(i,k))*rdelt
L00521          rvcuten(i,k)  = (vf(i,zz)-v(i,k))*rdelt
L00522       enddo
L00523       pp = pp + 1
L00524    enddo
L00525
L00526    errmsg = 'cu_ntiedtke_timestep_final OK'
L00527    errflg = 0
L00528
L00529    end subroutine cu_ntiedtke_post_run
```

讲解：①delt=dt • stepcu、rdelt=1/delt。② **降水均摊**：raincv=rn/stepcu（实体在聚合步内算的总降水 rn，均摊回每个模式步，mm/step 口径）、pratec=rn/(stepcu • dt)（mm/s）——cumulus_driver 据此累加 RAINC。③ **状态差分式倾向**：六个倾向全部按（翻转回 k 序后的终态 − 进入时的初态）/delt 给出，rthcuten 额外除 exner 把温度差转回位温倾向——「输出倾向」而非「直接改状态」，与 WRF 的非时间分裂物理串联约定（辐射 → 陆面 →PBL→ 积云……倾向统一进动力核）严丝合缝。对比 KF 系在实体内部直接修改 rqv/rth 数组再由 driver 收差分的做法（code_23 §CASE 对照）。

八、physics_mmm 实体对照（注记）

实体身份（2026-10-12 核验）：`phys/physics_mmm/cu_ntiedtke.F90`——3594 行、15 单元、md5 104dcb4bf2cee44771b8567c5d8056dc。physics_mmm 是 NCAR MMM 维护的共享物理库目录（内含 cu_ntiedtke、bl_mynn 系、mp_wsm6、bl_shinhong、bl_ysu、bl_gwdo、sf_sfclayrev 等），随本源码树携带；文库方案的取舍是「physics_mmm 目录不单独成篇」（外围 C 级排除，plan § 三写明），本篇按对照注记收录。

实体单元地图（扫描器输出，行段为引用不转录）：cu_ntiedtke_init（L81–123，常数注入落地）、cu_ntiedtke_finalize（L129–141）、**cu_ntiedtke_run（L148–332，实体侧总控——外壳三段调用的 run 落点）**、**cumastrn（L342–1085，744 行主实体：质量通量闭环主求解；教材 ch15「cumastrn 十步」精读即此）**、cuinin（L1091–1216）、cutypen（L1221–1750，两类测试气块）、cuascn（L1755–2258）、cudlfsn（L2262–2487）、cuddrafn（L2495–2721）、cuflxn（L2725–3060）、cudtdqn（L3064–3148）、cududvn（L3152–3252）、cuadjtqn（L3256–3402）、cubasmcn（L3406–3485）、cuentrn（L3489–3538）。各单元职能的逐段精读见教材 ch15_cumulus_family.md（已覆盖 cumastrn 十步、cutypen 两类气块、三类闭合、卷入卷出、Sundqvist 降水与 scale_fac）。

外壳 ↔ 实体接口契约（外壳调用点 L219–226 实参逐项对应，保真于本地源码）：pu/pv/pt/pqv/pqc/pqi=翻转后的 u/v/T/qv/qc/qi；pqvf/ptf=翻转后的前期倾向（闭合式强迫）；poz/pzz=半层/整层高度；

pomg=ω 型垂直速度；pap/paph=半层/整层气压；evap=qfx_hv（地表蒸发通量）、hfx=hfx_hv（地表感热）；zprecc=rn（对流降水输出，mm/聚合步）；lndj=slimsk（陆海标记）；lq=列数 im；km/kx 与 km1/kx1=层数；dt=delt；dx=格距（对流调整的水平尺度依赖）。

订正记录：批 3 成稿时曾误判「physics_mmm 子模块缺席本源码树」（当时只查了 WRF 根目录、漏了 phys/ 下的同名子目录）；用户复核指出后已核验订正本节并同步 notes/progress。教训：目录核查要下探一层。

九、改造入口

1. **常数与时间尺度**：cu_ntiedtke_init 的 con_* 注入与 delt=dt • stepcu——做积云步长敏感性实验（stepcu=1 逐步）只动 driver/namelist。
2. **触发门控**：cu_act_flag 恒 true——想加湿度/CAPE 触发阈值（如中科院风格 trigger），在 driver 的 J 循环置 false 即可跳过该列实体调用（实体完全不被触碰）。
3. **坐标翻转层通用化**：pre_run/post_run 的 pp/zz 对偶循环是「自下向上物理库」接入 WRF 的标准模板——移植 ICON/GFS 生态方案照抄此结构。
4. **闭合式强迫**：qvften/thften 的口径（平流 +PBL[+辐射]）——改 cumulus_driver 里 rthften/rqvften 的组装（code_23）即可改变闭合式看到的外部倾向。
5. **输出形态**：状态差分式倾向——若需「逐云过程分项诊断」（如分离凝结/蒸发/夹卷项），必须进实体，外壳只能给净差。

十、上下游链接

- 调用方：`code_23_module_cumulus_driver.md`（CASE NTIEDTKE 与 stepcu/nlifecycle 语义）
- 同族对照：KF/旧 Tiedtke 等 CASE 分支均在 code_23；微物理驱动链：`code_21_module_microphysics_driver.md`
- 教材正文：ch15_cumulus_family.md、ch20_modifying_wrf.md、ch21_solve_em_dynamics.md（倾向进入动力核的位置）

code_12a — Noah LSM 本体·上半（module_sf_noahlsm.F：主入口 SFLX 与蒸发/热力线）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_noahlsm.F	4759	9bc3fbaae919bd37ca4d05e44599c582

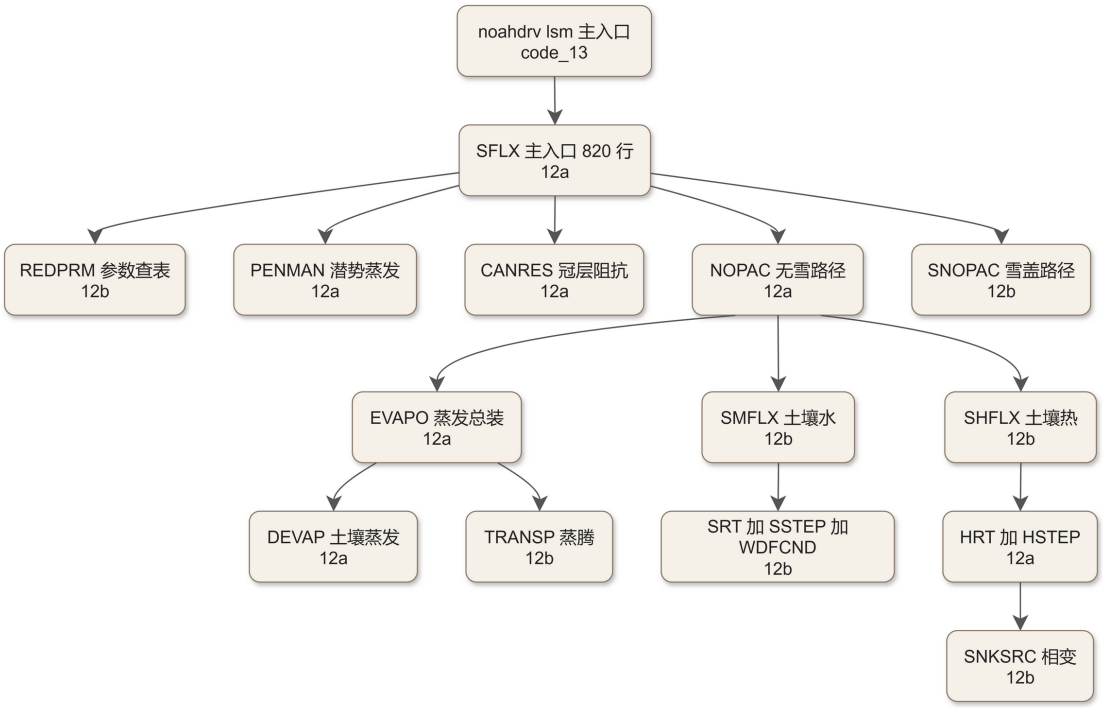
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级全文拆两：12a＝文件头＋SFLX（主入口）+ 蒸发/冠层/热力线共 13 单元（L1–L2312）；12b＝RED-PRM 起的参数查表/隐式求解器/雪线共 18 单元（L2315–L4756），见 `code_12b_module_sf_noahlsm.md`。

二、定位

一句话：Noah LSM（Chen–Dudhia 2001 谱系，Unified Noah v1.0 2007 统一化）的**方案本体**——土壤水、土壤温度、雪盖三套预报量的物理核心；一个模块 31 个子程序把潜热交换、径流、冻土相变、雪盖演化全部做完，noahdrv（code_13）只是它的逐格点驱动外壳。

谁在调它：module_sf_noahdrv.F 的 lsm 主入口按格点调 SFLX（`code_13_module_sf_noahdrv.md`）；再上游是 lsm_driver 的方案分派。教材锚点：ch16_land_surface_family.md 全章；urban 硬参数与灌溉通道是 ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md 的常客；与 BEP/BEM（code_14/15/16）的关系见 ch17_urban_canopy.md。

31 单元地图（实线 =调用，本篇/12b 标注）：



三、文件头：MPAS/WRF 双框架与三张参数表

```
!file: phys/module_sf_noahlsm.F    L1-L68
L00001  MODULE module_sf_noahlsm
L00002  #if defined(mpas)
L00003  use mpas_atmphys_constants,only: CP=>cp,R_D=>R_d,XLF=>xlf,XLV=>xlv,RHOWATER=>rho_w,STBOLT=>stbolt,KARMAN=>karman
L00004  use mpas_atmphys_utilities, only: physics_error_fatal
L00005  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00006  #else
L00007  USE module_model_constants, only : CP, R_D, XLF, XLV, RHOWATER, STBOLT, KARMAN
L00008  use module_wrf_error
L00009  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00010  #endif
L00011
L00012  !ckay=KIRAN ALAPATY @ US EPA -- November 01, 2015
L00013  !
L00014  ! Tim Glotfelty@CNSU; AJ Deng@PSU
L00015  !modified for use with FASDAS
L00016  !Flux Adjusting Surface Data Assimilation System to assimilate
L00017  !surface layer and soil layers temperature and moisture using
L00018  ! surfance reanalsys
L00019  !Reference: Alapaty et al., 2008: Development of the flux-adjusting surface
L00020  ! data assimilation system for mesoscale models. JAMC, 47, 2331-2350
L00021  !
L00022
L00023  !   REAL, PARAMETER    :: CP = 1004.5
L00024    REAL, PARAMETER      :: RD = 287.04, SIGMA = 5.67E-8,                &
L00025                            CPH2O = 4.218E+3,CPICE = 2.106E+3,           &
L00026                            LSUBF = 3.335E+5,                            &
L00027                            EMISSI_S = 0.95
L00028
L00029  ! VEGETATION PARAMETERS
L00030          INTEGER :: LUCATS , BARE
L00031          INTEGER :: NATURAL
L00032          INTEGER :: LCZ_1,LCZ_2,LCZ_3,LCZ_4,LCZ_5,LCZ_6,LCZ_7,LCZ_8,LCZ_9,LCZ_10,LCZ_11
L00033          integer, PARAMETER :: NLUS=50
L00034          CHARACTER(LEN=256) LUTYPE
L00035          INTEGER, DIMENSION(1:NLUS) :: NROTBL
L00036          real, dimension(1:NLUS) ::  SNUPTBL, RSTBL, RGLTBL, HSTBL,               &
L00037                                      SHDTBL, MAXALB,                              &
L00038                                      EMISSMINTBL, EMISSMAXTBL,                    &
L00039                                      LAIMINTBL, LAIMAXTBL,                        &
L00040                                      Z0MINTBL, Z0MAXTBL,                          &
L00041                                      ALBEDOMINTBL, ALBEDOMAXTBL,                  &
L00042                                      ZTOPVTBL,ZBOTVTBL
L00043          REAL ::    TOPT_DATA,CMCMAX_DATA,CFACTR_DATA,RSMAX_DATA
L00044
L00045  ! SOIL PARAMETERS
L00046          INTEGER :: SLCATS
L00047          INTEGER, PARAMETER :: NSLTYPE=30
L00048          CHARACTER(LEN=256) SLTYPE
L00049          REAL, DIMENSION (1:NSLTYPE) :: BB,DRYSMC,F11,                          &
L00050          MAXSMC, REFSMC,SATPSI,SATDK,SATDW, WLTSMC,QTZ
L00051
L00052  ! LSM GENERAL PARAMETERS
L00053          INTEGER :: SLPCATS
L00054          INTEGER, PARAMETER :: NSLOPE=30
L00055          REAL, DIMENSION (1:NSLOPE) :: SLOPE_DATA
L00056          REAL ::  SBETA_DATA,FXEXP_DATA,CSOIL_DATA,SALP_DATA,REFDK_DATA,          &
L00057                   REFKDT_DATA,FRZK_DATA,ZBOT_DATA,  SMLOW_DATA,SMHIGH_DATA,       &
L00058                              CZIL_DATA
L00059          REAL ::  LVCOEF_DATA
L00060
```

```
L00061          CHARACTER*256  :: err_message
L00062
L00063          integer, private :: iloc, jloc
L00064  !$omp threadprivate(iloc, jloc)
L00065  !
L00066  CONTAINS
L00067  !
L00068
```

讲解：① 前 10 行是 MPAS/WRF 双框架编译开关——MPAS 分支把六个常数换名自 mpas_atmphys_constants，错误出口换 physics_error_fatal；Noah 自 v3.8 起是 WRF/MPAS 共享物理，改代码时两条线都要能过编译。②WRF 分支只从 module_model_constants 拿 CP/R_D/XLF/XLV/RHOWATER/STBOLT/KARMAN，其余常数自带：RD=287.04、SIGMA=5.67E-8、CPH2O=4.218E3（液态水比热）、CPICE=2.106E3（冰比热）、LSUBF=3.335E5（融解潜热 J/kg）、EMISSI_S=0.95（雪面发射率）。③ 三张参数表的存储声明全在模块级：植被表 NLUS=50 类上限（NROTBL 根层数、SNUPTBL 雪深阈值、RSTBL 最小气孔阻抗、RGLTBL/HS 光照与湿度差应力参数、SHDTBL/MAXALB，以及 v3.8+ 的 EMISS/LAI/Z0/ALBEDO 各 min-max 对与 ZTOPV/ZBOTV 冠层顶底高——UA 积雪物理用），并声明了 LCZ_1 至 LCZ_11 城市气候区占位；土壤表 NSLTYPE=30 类十个参数（BB、DRYSMC、F11、MAXSMC、REFSMC、SATPSI、SATDK、SATDW、WLTSMC、QTZ）；通用参数 SLOPE/SBETA/FXEXP/CSOIL/SALP/REFDK/REFKDT/FRZK/ZBOT/CZIL/LVCOEF。这些数组由 noahdrv 的 read_mp_* 系列从 VEGPARM.TBL/SOILPARM.TBL/GENPARM.TBL 读入，REDPRM（12b）只做查表。④iloc/jloc 声明为 private + threadprivate——OpenMP 并行下单点诊断打印的安全坐标。

四、主入口 SFLX（全文）

```
!file: phys/module_sf_noahlsm.F    L69-L888    (subroutine SFLX)
L00069        SUBROUTINE SFLX (IILOC,JJLOC,FFROZP,ISURBAN,DT,ZLVL,NSOIL,SLDPTH, &    !C
L00070                         LOCAL,                                             &    !L
L00071                         LLANDUSE, LSOIL,                                   &    !CL
L00072                         LWDN,SOLDN,SOLNET,SFCPRS,PRCP,SFCTMP,Q2,SFCSPD,    &    !F
L00073                         COSZ,PRCPRAIN, SOLARDIRECT,                        &    !F
L00074                         TH2,Q2SAT,DQSDT2,                                  &    !I
L00075                         VEGTYP,SOILTYP,SLOPETYP,SHDFAC,SHDMIN,SHDMAX,      &    !I
L00076                         ALB, SNOALB,TBOT, Z0BRD, Z0, EMISSI, EMBRD,        &    !S
L00077                         CMC,T1,STC,SMC,SH2O,SNOWH,SNEQV,ALBEDO,CH,CM,      &    !H
L00078  ! ----------------------------------------------------------------------
L00079  ! OUTPUTS, DIAGNOSTICS, PARAMETERS BELOW GENERALLY NOT NECESSARY WHEN
L00080  ! COUPLED WITH E.G. A NWP MODEL (SUCH AS THE NOAA/NWS/NCEP MESOSCALE ETA
L00081  ! MODEL).  OTHER APPLICATIONS MAY REQUIRE DIFFERENT OUTPUT VARIABLES.
L00082  ! ----------------------------------------------------------------------
L00083                         ETA,SHEAT, ETA_KINEMATIC,FDOWN,                    &    !O
L00084                         EC,EDIR,ET,ETT,ESNOW,DRIP,DEW,                     &    !O
L00085                         BETA,ETP,SSOIL,                                    &    !O
L00086                         FLX1,FLX2,FLX3,                                    &    !O
L00087                 FLX4,FVB,FBUR,FGSN,UA_PHYS,                        &    !UA
L00088                         SNOMLT,SNCOVR,                                     &    !O
L00089                         RUNOFF1,RUNOFF2,RUNOFF3,                           &    !O
L00090                         RC,PC,RSMIN,XLAI,RCS,RCT,RCQ,RCSOIL,               &    !O
L00091                         SOILW,SOILM,Q1,SMAV,                               &    !D
L00092                         RDLAI2D,USEMONALB,                                 &
L00093                         SNOTIME1,                                          &
L00094                         RIBB,                                              &
L00095                         SMCWLT,SMCDRY,SMCREF,SMCMAX,NROOT,   &
L00096                         SFHEAD1RT,                                         &    !I
L00097                         INFXS1RT,ETPND1,OPT_THCND,AOASIS                   &    !P
L00098                        ,XSDA_QFX,HFX_PHY,QFX_PHY,XQNORM                    &    !fasdas
L00099                        ,fasdas,HCPCT_FASDAS,IRRIGATION_CHANNEL             )    !fasdas
L00100
L00101  ! ----------------------------------------------------------------------
L00102  ! SUBROUTINE SFLX - UNIFIED NOAHLSM VERSION 1.0 JULY 2007
L00103  ! ----------------------------------------------------------------------
L00104  ! SUB-DRIVER FOR "Noah LSM" FAMILY OF PHYSICS SUBROUTINES FOR A
L00105  ! SOIL/VEG/SNOWPACK LAND-SURFACE MODEL TO UPDATE SOIL MOISTURE, SOIL
L00106  ! ICE, SOIL TEMPERATURE, SKIN TEMPERATURE, SNOWPACK WATER CONTENT,
L00107  ! SNOWDEPTH, AND ALL TERMS OF THE SURFACE ENERGY BALANCE AND SURFACE
L00108  ! WATER BALANCE (EXCLUDING INPUT ATMOSPHERIC FORCINGS OF DOWNWARD
L00109  ! RADIATION AND PRECIP)
L00110  ! ----------------------------------------------------------------------
L00111  ! SFLX ARGUMENT LIST KEY:
L00112  ! ----------------------------------------------------------------------
L00113  !  C  CONFIGURATION INFORMATION
L00114  !  L  LOGICAL
L00115  ! CL  4-string character bearing logical meaning
L00116  !  F  FORCING DATA
L00117  !  I  OTHER (INPUT) FORCING DATA
L00118  !  S  SURFACE CHARACTERISTICS
L00119  !  H  HISTORY (STATE) VARIABLES
L00120  !  O  OUTPUT VARIABLES
L00121  !  D  DIAGNOSTIC OUTPUT
L00122  !  P  Parameters
L00123  !  Msic Miscellaneous terms passed from gridded driver
L00124  ! ----------------------------------------------------------------------
L00125  ! 1. CONFIGURATION INFORMATION (C):
L00126  ! ----------------------------------------------------------------------
L00127  !   DT         TIMESTEP (SEC) (DT SHOULD NOT EXCEED 3600 SECS, RECOMMEND
L00128  !                1800 SECS OR LESS)
L00129  !   ZLVL       HEIGHT (M) ABOVE GROUND OF ATMOSPHERIC FORCING VARIABLES
L00130  !   NSOIL      NUMBER OF SOIL LAYERS (AT LEAST 2, AND NOT GREATER THAN
L00131  !                PARAMETER NSOLD SET BELOW)
L00132  !   SLDPTH     THE THICKNESS OF EACH SOIL LAYER (M)
L00133  ! ----------------------------------------------------------------------
L00134  ! 2. LOGICAL:
```

```
L00135  ! ----------------------------------------------------------------------
L00136  !   LCH       Exchange coefficient (Ch) calculation flag (false: using
L00137  !               ch-routine SFCDIF; true: Ch is brought in)
L00138  !   LOCAL      Flag for local-site simulation (where there is no
L00139  !              maps for albedo, veg fraction, and roughness
L00140  !             true:  all LSM parameters (inluding albedo, veg fraction and
L00141  !                    roughness length) will be defined by three tables
L00142  !   LLANDUSE  (=USGS, using USGS landuse classification)
L00143  !   LSOIL     (=STAS, using FAO/STATSGO soil texture classification)
L00144  !   OPT_THCND option for how to treat thermal conductivity
L00145  ! ----------------------------------------------------------------------
L00146  ! 3. FORCING DATA (F):
L00147  ! ----------------------------------------------------------------------
L00148  !   LWDN       LW DOWNWARD RADIATION (W M-2; POSITIVE, NOT NET LONGWAVE)
L00149  !   SOLDN      SOLAR DOWNWARD RADIATION (W M-2; POSITIVE, NOT NET SOLAR)
L00150  !   SOLNET     NET DOWNWARD SOLAR RADIATION ((W M-2; POSITIVE)
L00151  !   SFCPRS     PRESSURE AT HEIGHT ZLVL ABOVE GROUND (PASCALS)
L00152  !   PRCP       PRECIP RATE (KG M-2 S-1) (NOTE, THIS IS A RATE)
L00153  !   SFCTMP     AIR TEMPERATURE (K) AT HEIGHT ZLVL ABOVE GROUND
L00154  !   TH2        AIR POTENTIAL TEMPERATURE (K) AT HEIGHT ZLVL ABOVE GROUND
L00155  !   Q2         MIXING RATIO AT HEIGHT ZLVL ABOVE GROUND (KG KG-1)
L00156  !   COSZ       Solar zenith angle (not used for now)
L00157  !   PRCPRAIN   Liquid-precipitation rate (KG M-2 S-1) (not used)
L00158  ! SOLARDIRECT  Direct component of downward solar radiation (W M-2) (not used)
L00159  !   FFROZP     FRACTION OF FROZEN PRECIPITATION
L00160  ! ----------------------------------------------------------------------
L00161  ! 4. OTHER FORCING (INPUT) DATA (I):
L00162  ! ----------------------------------------------------------------------
L00163  !   SFCSPD     WIND SPEED (M S-1) AT HEIGHT ZLVL ABOVE GROUND
L00164  !   Q2SAT      SAT SPECIFIC HUMIDITY AT HEIGHT ZLVL ABOVE GROUND (KG KG-1)
L00165  !   DQSDT2     SLOPE OF SAT SPECIFIC HUMIDITY CURVE AT T=SFCTMP
L00166  !                (KG KG-1 K-1)
L00167  ! ----------------------------------------------------------------------
L00168  ! 5. CANOPY/SOIL CHARACTERISTICS (S):
L00169  ! ----------------------------------------------------------------------
L00170  !   VEGTYP     VEGETATION TYPE (INTEGER INDEX)
L00171  !   SOILTYP    SOIL TYPE (INTEGER INDEX)
L00172  !   SLOPETYP   CLASS OF SFC SLOPE (INTEGER INDEX)
L00173  !   SHDFAC     AREAL FRACTIONAL COVERAGE OF GREEN VEGETATION
L00174  !                (FRACTION= 0.0-1.0)
L00175  !   SHDMIN     MINIMUM AREAL FRACTIONAL COVERAGE OF GREEN VEGETATION
L00176  !                (FRACTION= 0.0-1.0) <= SHDFAC
L00177  !   PTU        PHOTO THERMAL UNIT (PLANT PHENOLOGY FOR ANNUALS/CROPS)
L00178  !                (NOT YET USED, BUT PASSED TO REDPRM FOR FUTURE USE IN
L00179  !                VEG PARMS)
L00180  !   ALB        BACKROUND SNOW-FREE SURFACE ALBEDO (FRACTION), FOR JULIAN
L00181  !                DAY OF YEAR (USUALLY FROM TEMPORAL INTERPOLATION OF
L00182  !                MONTHLY MEAN VALUES' CALLING PROG MAY OR MAY NOT
L00183  !                INCLUDE DIURNAL SUN ANGLE EFFECT)
L00184  !   SNOALB     UPPER BOUND ON MAXIMUM ALBEDO OVER DEEP SNOW (E.G. FROM
L00185  !                ROBINSON AND KUKLA, 1985, J. CLIM. & APPL. METEOR.)
L00186  !   TBOT       BOTTOM SOIL TEMPERATURE (LOCAL YEARLY-MEAN SFC AIR
L00187  !                TEMPERATURE)
L00188  !   Z0BRD      Background fixed roughness length (M)
L00189  !   Z0         Time varying roughness length (M) as function of snow depth
L00190  !
L00191  !   EMBRD      Background surface emissivity (between 0 and 1)
L00192  !   EMISSI     Surface emissivity (between 0 and 1)
L00193  ! ----------------------------------------------------------------------
L00194  ! 6. HISTORY (STATE) VARIABLES (H):
L00195  ! ----------------------------------------------------------------------
L00196  !  CMC         CANOPY MOISTURE CONTENT (M)
L00197  !  T1          GROUND/CANOPY/SNOWPACK) EFFECTIVE SKIN TEMPERATURE (K)
L00198  !  STC(NSOIL)  SOIL TEMP (K)
L00199  !  SMC(NSOIL)  TOTAL SOIL MOISTURE CONTENT (VOLUMETRIC FRACTION)
L00200  !  SH2O(NSOIL) UNFROZEN SOIL MOISTURE CONTENT (VOLUMETRIC FRACTION)
L00201  !                NOTE: FROZEN SOIL MOISTURE = SMC - SH2O
L00202  !  SNOWH       ACTUAL SNOW DEPTH (M)
L00203  !  SNEQV       LIQUID WATER-EQUIVALENT SNOW DEPTH (M)
L00204  !                NOTE: SNOW DENSITY = SNEQV/SNOWH
L00205  !  ALBEDO      SURFACE ALBEDO INCLUDING SNOW EFFECT (UNITLESS FRACTION)
L00206  !                =SNOW-FREE ALBEDO (ALB) WHEN SNEQV=0, OR
L00207  !                =FCT(MSNOALB,ALB,VEGTYP,SHDFAC,SHDMIN) WHEN SNEQV>0
L00208  !  CH          SURFACE EXCHANGE COEFFICIENT FOR HEAT AND MOISTURE
L00209  !                (M S-1); NOTE: CH IS TECHNICALLY A CONDUCTANCE SINCE
L00210  !                IT HAS BEEN MULTIPLIED BY WIND SPEED.
L00211  !  CM          SURFACE EXCHANGE COEFFICIENT FOR MOMENTUM (M S-1); NOTE:
L00212  !                CM IS TECHNICALLY A CONDUCTANCE SINCE IT HAS BEEN
L00213  !                MULTIPLIED BY WIND SPEED.
L00214  ! ----------------------------------------------------------------------
L00215  ! 7. OUTPUT (O):
L00216  ! ----------------------------------------------------------------------
L00217  ! OUTPUT VARIABLES NECESSARY FOR A COUPLED NUMERICAL WEATHER PREDICTION
L00218  ! MODEL, E.G. NOAA/NWS/NCEP MESOSCALE ETA MODEL.  FOR THIS APPLICATION,
L00219  ! THE REMAINING OUTPUT/DIAGNOSTIC/PARAMETER BLOCKS BELOW ARE NOT
L00220  ! NECESSARY.  OTHER APPLICATIONS MAY REQUIRE DIFFERENT OUTPUT VARIABLES.
L00221  !   ETA        ACTUAL LATENT HEAT FLUX (W m-2: NEGATIVE, IF UP FROM
L00222  !              SURFACE)
L00223  !  ETA_KINEMATIC atctual latent heat flux in Kg m-2 s-1
L00224  !   SHEAT      SENSIBLE HEAT FLUX (W M-2: POSITIVE, IF UPWARD FROM
L00225  !              SURFACE)
L00226  !   FDOWN      Radiation forcing at the surface (W m-2) = SOLDN*(1-alb)+LWDN
L00227  ! ----------------------------------------------------------------------
L00228  !   EC         CANOPY WATER EVAPORATION (W m-2)
L00229  !   EDIR       DIRECT SOIL EVAPORATION (W m-2)
L00230  !   ET(NSOIL)  PLANT TRANSPIRATION FROM A PARTICULAR ROOT (SOIL) LAYER
L00231  !                (W m-2)
L00232  !   ETT        TOTAL PLANT TRANSPIRATION (W m-2)
L00233  !   ESNOW      SUBLIMATION FROM (OR DEPOSITION TO IF <0) SNOWPACK
L00234  !                (W m-2)
L00235  !   DRIP       THROUGH-FALL OF PRECIP AND/OR DEW IN EXCESS OF CANOPY
L00236  !                WATER-HOLDING CAPACITY (M)
L00237  !   DEW        DEWFALL (OR FROSTFALL FOR T<273.15) (M)
L00238  ! ----------------------------------------------------------------------
L00239  !   BETA       RATIO OF ACTUAL/POTENTIAL EVAP (DIMENSIONLESS)
L00240  !   ETP        POTENTIAL EVAPORATION (W m-2)
L00241  !   SSOIL      SOIL HEAT FLUX (W M-2: NEGATIVE IF DOWNWARD FROM SURFACE)
```

```
L00242  ! ----------------------------------------------------------------------
L00243  !   FLX1       PRECIP-SNOW SFC (W M-2)
L00244  !   FLX2       FREEZING RAIN LATENT HEAT FLUX (W M-2)
L00245  !   FLX3       PHASE-CHANGE HEAT FLUX FROM SNOWMELT (W M-2)
L00246  ! ----------------------------------------------------------------------
L00247  !   SNOMLT     SNOW MELT (M) (WATER EQUIVALENT)
L00248  !   SNCOVR     FRACTIONAL SNOW COVER (UNITLESS FRACTION, 0-1)
L00249  ! ----------------------------------------------------------------------
L00250  !   RUNOFF1    SURFACE RUNOFF (M S-1), NOT INFILTRATING THE SURFACE
L00251  !   RUNOFF2    SUBSURFACE RUNOFF (M S-1), DRAINAGE OUT BOTTOM OF LAST
L00252  !                SOIL LAYER (BASEFLOW)
L00253  !   RUNOFF3    NUMERICAL TRUNCTATION IN EXCESS OF POROSITY (SMCMAX)
L00254  !                FOR A GIVEN SOIL LAYER AT THE END OF A TIME STEP (M S-1).
L00255  ! Note: the above RUNOFF2 is actually the sum of RUNOFF2 and RUNOFF3
L00256  ! ----------------------------------------------------------------------
L00257  !   RC         CANOPY RESISTANCE (S M-1)
L00258  !   PC         PLANT COEFFICIENT (UNITLESS FRACTION, 0-1) WHERE PC*ETP
L00259  !                = ACTUAL TRANSP
L00260  !   XLAI       LEAF AREA INDEX (DIMENSIONLESS)
L00261  !   RSMIN      MINIMUM CANOPY RESISTANCE (S M-1)
L00262  !   RCS        INCOMING SOLAR RC FACTOR (DIMENSIONLESS)
L00263  !   RCT        AIR TEMPERATURE RC FACTOR (DIMENSIONLESS)
L00264  !   RCQ        ATMOS VAPOR PRESSURE DEFICIT RC FACTOR (DIMENSIONLESS)
L00265  !   RCSOIL     SOIL MOISTURE RC FACTOR (DIMENSIONLESS)
L00266  ! ----------------------------------------------------------------------
L00267  ! 8. DIAGNOSTIC OUTPUT (D):
L00268  ! ----------------------------------------------------------------------
L00269  !   SOILW      AVAILABLE SOIL MOISTURE IN ROOT ZONE (UNITLESS FRACTION
L00270  !              BETWEEN SMCWLT AND SMCMAX)
L00271  !   SOILM      TOTAL SOIL COLUMN MOISTURE CONTENT (FROZEN+UNFROZEN) (M)
L00272  !   Q1         Effective mixing ratio at surface (kg kg-1), used for
L00273  !              diagnosing the mixing ratio at 2 meter for coupled model
L00274  !   SMAV       Soil Moisture Availability for each layer, as a fraction
L00275  !              between SMCWLT and SMCMAX.
L00276  !  Documentation for SNOTIME1 and SNOABL2 ?????
L00277  !  What categories of arguments do these variables fall into ????
L00278  !  Documentation for RIBB ?????
L00279  !  What category of argument does RIBB fall into ?????
L00280  ! ----------------------------------------------------------------------
L00281  ! 9. PARAMETERS (P):
L00282  ! ----------------------------------------------------------------------
L00283  !   SMCWLT     WILTING POINT (VOLUMETRIC)
L00284  !   SMCDRY     DRY SOIL MOISTURE THRESHOLD WHERE DIRECT EVAP FRM TOP
L00285  !                LAYER ENDS (VOLUMETRIC)
L00286  !   SMCREF     SOIL MOISTURE THRESHOLD WHERE TRANSPIRATION BEGINS TO
L00287  !                STRESS (VOLUMETRIC)
L00288  !   SMCMAX     POROSITY, I.E. SATURATED VALUE OF SOIL MOISTURE
L00289  !                (VOLUMETRIC)
L00290  !   NROOT      NUMBER OF ROOT LAYERS, A FUNCTION OF VEG TYPE, DETERMINED
L00291  !              IN SUBROUTINE REDPRM.
L00292  ! ----------------------------------------------------------------------
L00293
L00294
L00295        IMPLICIT NONE
L00296  ! ----------------------------------------------------------------------
L00297
L00298  ! DECLARATIONS - LOGICAL AND CHARACTERS
L00299  ! ----------------------------------------------------------------------
L00300
L00301        INTEGER, INTENT(IN) :: IILOC, JJLOC
L00302        LOGICAL, INTENT(IN)::  LOCAL
L00303        LOGICAL             ::  FRZGRA, SNOWNG
L00304        CHARACTER (LEN=256), INTENT(IN)::  LLANDUSE, LSOIL
L00305
L00306  ! ----------------------------------------------------------------------
L00307  ! 1. CONFIGURATION INFORMATION (C):
L00308  ! ----------------------------------------------------------------------
L00309        INTEGER,INTENT(IN) ::  NSOIL,SLOPETYP,SOILTYP,VEGTYP
L00310        INTEGER, INTENT(IN) :: ISURBAN
L00311        INTEGER,INTENT(OUT)::  NROOT
L00312        INTEGER  KZ, K, iout
L00313
L00314  ! ----------------------------------------------------------------------
L00315  ! 2. LOGICAL:
L00316  ! ----------------------------------------------------------------------
L00317        LOGICAL, INTENT(IN) :: RDLAI2D
L00318        LOGICAL, INTENT(IN) :: USEMONALB
L00319        INTEGER, INTENT(IN) :: OPT_THCND
L00320
L00321        REAL, INTENT(INOUT):: SFHEAD1RT,INFXS1RT, ETPND1
L00322
L00323        REAL, INTENT(IN)   :: SHDMIN,SHDMAX,DT,DQSDT2,LWDN,PRCP,PRCPRAIN,     &
L00324                              Q2,Q2SAT,SFCPRS,SFCSPD,SFCTMP, SNOALB,          &
L00325                              SOLDN,SOLNET,TBOT,TH2,ZLVL,                           &
L00326                              FFROZP,AOASIS
L00327        REAL, INTENT(OUT)  :: EMBRD
L00328        REAL, INTENT(OUT)  :: ALBEDO
L00329        REAL, INTENT(INOUT):: COSZ, SOLARDIRECT,CH,CM,                        &
L00330                              CMC,SNEQV,SNCOVR,SNOWH,T1,XLAI,SHDFAC,Z0BRD,    &
L00331                              EMISSI, ALB
L00332        REAL, INTENT(INOUT):: SNOTIME1
L00333        REAL, INTENT(INOUT):: RIBB
L00334        REAL, DIMENSION(1:NSOIL), INTENT(IN) :: SLDPTH
L00335        REAL, DIMENSION(1:NSOIL), INTENT(OUT):: ET
L00336        REAL, DIMENSION(1:NSOIL), INTENT(OUT):: SMAV
L00337        REAL, DIMENSION(1:NSOIL), INTENT(INOUT) ::  SH2O, SMC, STC
L00338        REAL,DIMENSION(1:NSOIL)::   RTDIS, ZSOIL
L00339
L00340        REAL,INTENT(OUT)   :: ETA_KINEMATIC,BETA,DEW,DRIP,EC,EDIR,ESNOW,ETA,  &
L00341                              ETP,FLX1,FLX2,FLX3,SHEAT,PC,RUNOFF1,RUNOFF2,    &
L00342                              RUNOFF3,RC,RSMIN,RCQ,RCS,RCSOIL,RCT,SSOIL,      &
L00343                              SMCDRY,SMCMAX,SMCREF,SMCWLT,SNOMLT, SOILM,      &
L00344                              SOILW,FDOWN,Q1
L00345        LOGICAL, INTENT(IN) :: UA_PHYS   ! UA: flag for UA option
L00346        REAL,INTENT(OUT)    :: FLX4      ! UA: energy added to sensible heat
L00347        REAL,INTENT(OUT)    :: FVB       ! UA: frac. veg. w/snow beneath
L00348        REAL,INTENT(OUT)    :: FBUR      ! UA: fraction of canopy buried
L00349        REAL,INTENT(OUT)    :: FGSN      ! UA: ground snow cover fraction
```

```
L00350        REAL                :: ZTOPV     ! UA: height of canopy top
L00351        REAL                :: ZBOTV     ! UA: height of canopy bottom
L00352        REAL                :: GAMA      ! UA: = EXP(-1.* XLAI)
L00353        REAL                :: FNET      ! UA:
L00354        REAL                :: ETPN      ! UA:
L00355        REAL                :: RU        ! UA:
L00356
L00357        REAL :: BEXP,CFACTR,CMCMAX,CSOIL,CZIL,DF1,DF1H,DF1A,DKSAT,DWSAT,      &
L00358                DSOIL,DTOT,ETT,FRCSNO,FRCSOI,EPSCA,F1,FXEXP,FRZX,HS,          &
L00359                KDT,LVH2O,PRCP1,PSISAT,QUARTZ,R,RCH,REFKDT,RR,RGL,            &
L00360                RSMAX,                                                        &
L00361                RSNOW,SNDENS,SNCOND,SBETA,SN_NEW,SLOPE,SNUP,SALP,SOILWM,      &
L00362                SOILWW,T1V,T24,T2V,TH2V,TOPT,TFREEZ,TSNOW,ZBOT,Z0,PRCPF,      &
L00363                ETNS,PTU,LSUBS
L00364          REAL ::  LVCOEF
L00365        REAL :: INTERP_FRACTION
L00366        REAL :: LAIMIN,    LAIMAX
L00367        REAL :: ALBEDOMIN, ALBEDOMAX
L00368        REAL :: EMISSMIN,  EMISSMAX
L00369        REAL :: Z0MIN,     Z0MAX
L00370
L00371  ! ----------------------------------------------------------------------
L00372  ! DECLARATIONS - PARAMETERS
L00373  ! ----------------------------------------------------------------------
L00374        PARAMETER (TFREEZ = 273.15)
L00375        PARAMETER (LVH2O = 2.501E+6)
L00376        PARAMETER (LSUBS = 2.83E+6)
L00377        PARAMETER (R = 287.04)
L00378  !
L00379  !  FASDAS
L00380  !
L00381     INTEGER, INTENT(IN   )  ::  fasdas
L00382     REAL,    INTENT(INOUT)  ::  XSDA_QFX, XQNORM
L00383     REAL,    INTENT(INOUT)  ::  HFX_PHY, QFX_PHY
L00384     REAL,    INTENT(  OUT)  ::  HCPCT_FASDAS
L00385  !
L00386  !  END FASDAS
L00387  !
L00388  ! IRRIGATION
L00389    REAL, OPTIONAL,   INTENT(INOUT)  :: IRRIGATION_CHANNEL
L00390
L00391  ! ----------------------------------------------------------------------
L00392  !   INITIALIZATION
L00393  ! ----------------------------------------------------------------------
L00394        ILOC = IILOC
L00395        JLOC = JJLOC
L00396
L00397        RUNOFF1 = 0.0
L00398        RUNOFF2 = 0.0
L00399        RUNOFF3 = 0.0
L00400        SNOMLT = 0.0
L00401
L00402        IF ( .NOT. UA_PHYS ) THEN
L00403            FLX4 = 0.0
L00404            FVB  = 0.0
L00405            FBUR = 0.0
L00406            FGSN = 0.0
L00407        ENDIF
L00408
L00409  ! ----------------------------------------------------------------------
L00410  ! CALCULATE DEPTH (NEGATIVE) BELOW GROUND FROM TOP SKIN SFC TO BOTTOM OF
L00411  !   EACH SOIL LAYER.  NOTE:  SIGN OF ZSOIL IS NEGATIVE (DENOTING BELOW
L00412  !   GROUND)
L00413  ! ----------------------------------------------------------------------
L00414        ZSOIL (1) = - SLDPTH (1)
L00415        DO KZ = 2,NSOIL
L00416           ZSOIL (KZ) = - SLDPTH (KZ) + ZSOIL (KZ -1)
L00417        END DO
L00418
L00419  ! ----------------------------------------------------------------------
L00420  ! NEXT IS CRUCIAL CALL TO SET THE LAND-SURFACE PARAMETERS, INCLUDING
L00421  ! SOIL-TYPE AND VEG-TYPE DEPENDENT PARAMETERS.
L00422  ! ----------------------------------------------------------------------
L00423          CALL REDPRM (VEGTYP,SOILTYP,SLOPETYP,CFACTR,CMCMAX,RSMAX,TOPT,   &
L00424                       REFKDT,KDT,SBETA, SHDFAC,RSMIN,RGL,HS,ZBOT,FRZX,    &
L00425                         PSISAT,SLOPE,SNUP,SALP,BEXP,DKSAT,DWSAT,          &
L00426                         SMCMAX,SMCWLT,SMCREF,SMCDRY,F1,QUARTZ,FXEXP,      &
L00427                         RTDIS,SLDPTH,ZSOIL,NROOT,NSOIL,CZIL,              &
L00428                         LAIMIN, LAIMAX, EMISSMIN, EMISSMAX, ALBEDOMIN,    &
L00429                         ALBEDOMAX, Z0MIN, Z0MAX, CSOIL, PTU, LLANDUSE,    &
L00430                         LSOIL,LOCAL,LVCOEF,ZTOPV,ZBOTV)
L00431
L00432  !urban
L00433          IF(VEGTYP==ISURBAN)THEN
L00434               SHDFAC=0.05
L00435               RSMIN=400.0
L00436               SMCMAX = 0.45
L00437               SMCREF = 0.42
L00438               SMCWLT = 0.40
L00439               SMCDRY = 0.40
L00440          ENDIF
L00441
L00442          IF ( SHDFAC >= SHDMAX ) THEN
L00443             EMBRD = EMISSMAX
L00444             IF (.NOT. RDLAI2D) THEN
L00445                XLAI  = LAIMAX
L00446             ENDIF
L00447             IF (.NOT. USEMONALB) THEN
L00448                ALB   = ALBEDOMIN
L00449             ENDIF
L00450             Z0BRD = Z0MAX
L00451          ELSE IF ( SHDFAC <= SHDMIN ) THEN
L00452             EMBRD = EMISSMIN
L00453             IF(.NOT. RDLAI2D) THEN
L00454                XLAI  = LAIMIN
L00455             ENDIF
L00456             IF(.NOT. USEMONALB) then
L00457                ALB   = ALBEDOMAX
L00458             ENDIF
L00459             Z0BRD = Z0MIN
```

```
L00460           ELSE
L00461
L00462              IF ( SHDMAX > SHDMIN ) THEN
L00463
L00464                 INTERP_FRACTION = ( SHDFAC - SHDMIN ) / ( SHDMAX - SHDMIN )
L00465                 ! Bound INTERP_FRACTION between 0 and 1
L00466                 INTERP_FRACTION = MIN ( INTERP_FRACTION, 1.0 )
L00467                 INTERP_FRACTION = MAX ( INTERP_FRACTION, 0.0 )
L00468                 ! Scale Emissivity and LAI between EMISSMIN and EMISSMAX by INTERP_FRACTION
L00469                 EMBRD = ( ( 1.0 - INTERP_FRACTION ) * EMISSMIN  ) + ( INTERP_FRACTION * EMISSMAX  )
L00470                 IF (.NOT. RDLAI2D) THEN
L00471                    XLAI  = ( ( 1.0 - INTERP_FRACTION ) * LAIMIN    ) + ( INTERP_FRACTION * LAIMAX    )
L00472                 ENDIF
L00473                 if (.not. USEMONALB) then
L00474                    ALB   = ( ( 1.0 - INTERP_FRACTION ) * ALBEDOMAX ) + ( INTERP_FRACTION * ALBEDOMIN )
L00475                 endif
L00476                 Z0BRD = ( ( 1.0 - INTERP_FRACTION ) * Z0MIN     ) + ( INTERP_FRACTION * Z0MAX     )
L00477
L00478              ELSE
L00479
L00480                 EMBRD = 0.5 * EMISSMIN  + 0.5 * EMISSMAX
L00481                 IF (.NOT. RDLAI2D) THEN
L00482                    XLAI  = 0.5 * LAIMIN    + 0.5 * LAIMAX
L00483                 ENDIF
L00484                 if (.not. USEMONALB) then
L00485                    ALB   = 0.5 * ALBEDOMIN + 0.5 * ALBEDOMAX
L00486                 endif
L00487                 Z0BRD = 0.5 * Z0MIN     + 0.5 * Z0MAX
L00488
L00489              ENDIF
L00490
L00491           ENDIF
L00492  ! ----------------------------------------------------------------------
L00493  !  INITIALIZE PRECIPITATION LOGICALS.
L00494  ! ----------------------------------------------------------------------
L00495           SNOWNG = .FALSE.
L00496           FRZGRA = .FALSE.
L00497
L00498  ! ----------------------------------------------------------------------
L00499  ! IF INPUT SNOWPACK IS NONZERO, THEN COMPUTE SNOW DENSITY "SNDENS" AND
L00500  !   SNOW THERMAL CONDUCTIVITY "SNCOND" (NOTE THAT CSNOW IS A FUNCTION
L00501  !   SUBROUTINE)
L00502  ! ----------------------------------------------------------------------
L00503           IF ( SNEQV <= 1.E-7 ) THEN ! safer IF  kmh (2008/03/25)
L00504              SNEQV = 0.0
L00505              SNDENS = 0.0
L00506              SNOWH = 0.0
L00507              SNCOND = 1.0
L00508           ELSE
L00509              SNDENS = SNEQV / SNOWH
L00510              IF(SNDENS > 1.0) THEN
L00511                FATAL_ERROR( 'Physical snow depth is less than snow water equiv.' )
L00512              ENDIF
L00513              CALL CSNOW (SNCOND,SNDENS)
L00514           END IF
L00515  ! ----------------------------------------------------------------------
L00516  ! DETERMINE IF IT'S PRECIPITATING AND WHAT KIND OF PRECIP IT IS.
L00517  ! IF IT'S PRCPING AND THE AIR TEMP IS COLDER THAN 0 C, IT'S SNOWING!
L00518  ! IF IT'S PRCPING AND THE AIR TEMP IS WARMER THAN 0 C, BUT THE GRND
L00519  ! TEMP IS COLDER THAN 0 C, FREEZING RAIN IS PRESUMED TO BE FALLING.
L00520  ! ----------------------------------------------------------------------
L00521           IF (PRCP > 0.0) THEN
L00522  ! snow defined when fraction of frozen precip (FFROZP) > 0.5,
L00523  ! passed in from model microphysics.
L00524              IF (FFROZP .GT. 0.5) THEN
L00525                 SNOWNG = .TRUE.
L00526              ELSE
L00527                 IF (T1 <= TFREEZ) FRZGRA = .TRUE.
L00528              END IF
L00529           END IF
L00530  ! ----------------------------------------------------------------------
L00531  ! IF EITHER PRCP FLAG IS SET, DETERMINE NEW SNOWFALL (CONVERTING PRCP
L00532  ! RATE FROM KG M-2 S-1 TO A LIQUID EQUIV SNOW DEPTH IN METERS) AND ADD
L00533  ! IT TO THE EXISTING SNOWPACK.
L00534  ! NOTE THAT SINCE ALL PRECIP IS ADDED TO SNOWPACK, NO PRECIP INFILTRATES
L00535  ! INTO THE SOIL SO THAT PRCP1 IS SET TO ZERO.
L00536  ! ----------------------------------------------------------------------
L00537           IF ( (SNOWNG) .OR. (FRZGRA) ) THEN
L00538              SN_NEW = PRCP * DT * 0.001
L00539              SNEQV = SNEQV + SN_NEW
L00540              PRCPF = 0.0
L00541
L00542  ! ----------------------------------------------------------------------
L00543  ! UPDATE SNOW DENSITY BASED ON NEW SNOWFALL, USING OLD AND NEW SNOW.
L00544  ! UPDATE SNOW THERMAL CONDUCTIVITY
L00545  ! ----------------------------------------------------------------------
L00546              CALL SNOW_NEW (SFCTMP,SN_NEW,SNOWH,SNDENS)
L00547              CALL CSNOW (SNCOND,SNDENS)
L00548
L00549  ! ----------------------------------------------------------------------
L00550  ! PRECIP IS LIQUID (RAIN), HENCE SAVE IN THE PRECIP VARIABLE THAT
L00551  ! LATER CAN WHOLELY OR PARTIALLY INFILTRATE THE SOIL (ALONG WITH
L00552  ! ANY CANOPY "DRIP" ADDED TO THIS LATER)
L00553  ! ----------------------------------------------------------------------
L00554           ELSE
L00555              PRCPF = PRCP
L00556           ENDIF
L00557  ! ----------------------------------------------------------------------
L00558  ! DETERMINE SNOWCOVER AND ALBEDO OVER LAND.
L00559  ! ----------------------------------------------------------------------
L00560  ! ----------------------------------------------------------------------
L00561  ! IF SNOW DEPTH=0, SET SNOW FRACTION=0, ALBEDO=SNOW FREE ALBEDO.
L00562  ! ----------------------------------------------------------------------
L00563           IF (SNEQV  == 0.0) THEN
L00564              SNCOVR = 0.0
L00565              ALBEDO = ALB
L00566              EMISSI = EMBRD
L00567          IF(UA_PHYS) FGSN = 0.0
L00568          IF(UA_PHYS) FVB = 0.0
L00569          IF(UA_PHYS) FBUR = 0.0
```

```
L00570           ELSE
L00571  ! ----------------------------------------------------------------------
L00572  ! DETERMINE SNOW FRACTIONAL COVERAGE.
L00573  ! DETERMINE SURFACE ALBEDO MODIFICATION DUE TO SNOWDEPTH STATE.
L00574  ! ----------------------------------------------------------------------
L00575              CALL SNFRAC (SNEQV,SNUP,SALP,SNOWH,SNCOVR, &
L00576                           XLAI,SHDFAC,FVB,GAMA,FBUR,    &
L00577                           FGSN,ZTOPV,ZBOTV,UA_PHYS)
L00578
L00579              IF ( UA_PHYS ) then
L00580                IF(SFCTMP <= T1) THEN
L00581                  RU = 0.
L00582                ELSE
L00583                  RU = 100.*SHDFAC*FGSN*MIN((SFCTMP-T1)/5., 1.)*(1.-EXP(-XLAI))
L00584                ENDIF
L00585                CH = CH/(1.+RU*CH)
L00586              ENDIF
L00587
L00588              SNCOVR = MIN(SNCOVR,0.98)
L00589
L00590              CALL ALCALC (ALB,SNOALB,EMBRD,SHDFAC,SHDMIN,SNCOVR,T1, &
L00591                   ALBEDO,EMISSI,DT,SNOWNG,SNOTIME1,LVCOEF)
L00592           ENDIF
L00593  ! ----------------------------------------------------------------------
L00594  ! NEXT CALCULATE THE SUBSURFACE HEAT FLUX, WHICH FIRST REQUIRES
L00595  ! CALCULATION OF THE THERMAL DIFFUSIVITY.  TREATMENT OF THE
L00596  ! LATTER FOLLOWS THAT ON PAGES 148-149 FROM "HEAT TRANSFER IN
L00597  ! COLD CLIMATES", BY V. J. LUNARDINI (PUBLISHED IN 1981
L00598  ! BY VAN NOSTRAND REINHOLD CO.) I.E. TREATMENT OF TWO CONTIGUOUS
L00599  ! "PLANE PARALLEL" MEDIUMS (NAMELY HERE THE FIRST SOIL LAYER
L00600  ! AND THE SNOWPACK LAYER, IF ANY). THIS DIFFUSIVITY TREATMENT
L00601  ! BEHAVES WELL FOR BOTH ZERO AND NONZERO SNOWPACK, INCLUDING THE
L00602  ! LIMIT OF VERY THIN SNOWPACK.  THIS TREATMENT ALSO ELIMINATES
L00603  ! THE NEED TO IMPOSE AN ARBITRARY UPPER BOUND ON SUBSURFACE
L00604  ! HEAT FLUX WHEN THE SNOWPACK BECOMES EXTREMELY THIN.
L00605  ! ----------------------------------------------------------------------
L00606  ! FIRST CALCULATE THERMAL DIFFUSIVITY OF TOP SOIL LAYER, USING
L00607  ! BOTH THE FROZEN AND LIQUID SOIL MOISTURE, FOLLOWING THE
L00608  ! SOIL THERMAL DIFFUSIVITY FUNCTION OF PETERS-LIDARD ET AL.
L00609  ! (1998,JAS, VOL 55, 1209-1224), WHICH REQUIRES THE SPECIFYING
L00610  ! THE QUARTZ CONTENT OF THE GIVEN SOIL CLASS (SEE ROUTINE REDPRM)
L00611  ! ----------------------------------------------------------------------
L00612  ! ----------------------------------------------------------------------
L00613  ! NEXT ADD SUBSURFACE HEAT FLUX REDUCTION EFFECT FROM THE
L00614  ! OVERLYING GREEN CANOPY, ADAPTED FROM SECTION 2.1.2 OF
L00615  ! PETERS-LIDARD ET AL. (1997, JGR, VOL 102(D4))
L00616  ! ----------------------------------------------------------------------
L00617              CALL TDFCND (DF1,SMC (1),QUARTZ,SMCMAX,SH2O (1),BEXP, PSISAT, SOILTYP, OPT_THCND)
L00618
L00619  !urban
L00620              IF ( VEGTYP == ISURBAN ) DF1=3.24
L00621
L00622              DF1 = DF1 * EXP (SBETA * SHDFAC)
L00623  !
L00624  ! kmh 09/03/2006
L00625  ! kmh 03/25/2008  change SNCOVR threshold to 0.97
L00626  !
L00627              IF (  SNCOVR .GT. 0.97 ) THEN
L00628                 DF1 = SNCOND
L00629              ENDIF
L00630  !
L00631  ! ----------------------------------------------------------------------
L00632  ! FINALLY "PLANE PARALLEL" SNOWPACK EFFECT FOLLOWING
L00633  ! V.J. LINARDINI REFERENCE CITED ABOVE. NOTE THAT DTOT IS
L00634  ! COMBINED DEPTH OF SNOWDEPTH AND THICKNESS OF FIRST SOIL LAYER
L00635  ! ----------------------------------------------------------------------
L00636
L00637           DSOIL = - (0.5 * ZSOIL (1))
L00638           IF (SNEQV == 0.) THEN
L00639              SSOIL = DF1 * (T1- STC (1) ) / DSOIL
L00640           ELSE
L00641              DTOT = SNOWH + DSOIL
L00642              FRCSNO = SNOWH / DTOT
L00643
L00644  ! 1. HARMONIC MEAN (SERIES FLOW)
L00645  !           DF1 = (SNCOND*DF1)/(FRCSOI*SNCOND+FRCSNO*DF1)
L00646              FRCSOI = DSOIL / DTOT
L00647  ! 2. ARITHMETIC MEAN (PARALLEL FLOW)
L00648  !           DF1 = FRCSNO*SNCOND + FRCSOI*DF1
L00649              DF1H = (SNCOND * DF1)/ (FRCSOI * SNCOND+ FRCSNO * DF1)
L00650
L00651  ! 3. GEOMETRIC MEAN (INTERMEDIATE BETWEEN HARMONIC AND ARITHMETIC MEAN)
L00652  !           DF1 = (SNCOND**FRCSNO)*(DF1**FRCSOI)
L00653  ! weigh DF by snow fraction
L00654  !           DF1 = DF1H*SNCOVR + DF1A*(1.0-SNCOVR)
L00655  !           DF1 = DF1H*SNCOVR + DF1*(1.0-SNCOVR)
L00656              DF1A = FRCSNO * SNCOND+ FRCSOI * DF1
L00657
L00658  ! ----------------------------------------------------------------------
L00659  ! CALCULATE SUBSURFACE HEAT FLUX, SSOIL, FROM FINAL THERMAL DIFFUSIVITY
L00660  ! OF SURFACE MEDIUMS, DF1 ABOVE, AND SKIN TEMPERATURE AND TOP
L00661  ! MID-LAYER SOIL TEMPERATURE
L00662  ! ----------------------------------------------------------------------
L00663              DF1 = DF1A * SNCOVR + DF1* (1.0- SNCOVR)
L00664              SSOIL = DF1 * (T1- STC (1) ) / DTOT
L00665           END IF
L00666  ! ----------------------------------------------------------------------
L00667  ! DETERMINE SURFACE ROUGHNESS OVER SNOWPACK USING SNOW CONDITION FROM
L00668  ! THE PREVIOUS TIMESTEP.
L00669  ! ----------------------------------------------------------------------
L00670           IF (SNCOVR  > 0. ) THEN
L00671              CALL SNOWZ0 (SNCOVR,Z0,Z0BRD,SNOWH,FBUR,FGSN,SHDMAX,UA_PHYS)
L00672           ELSE
L00673              Z0=Z0BRD
L00674              IF(UA_PHYS) CALL SNOWZ0 (SNCOVR,Z0,Z0BRD,SNOWH,FBUR,FGSN, &
L00675                                       SHDMAX,UA_PHYS)
L00676           END IF
L00677  ! ----------------------------------------------------------------------
L00678  ! NEXT CALL ROUTINE SFCDIF TO CALCULATE THE SFC EXCHANGE COEF (CH) FOR
```

```
L00679  ! HEAT AND MOISTURE.
L00680
L00681  ! NOTE !!!
L00682  ! DO NOT CALL SFCDIF UNTIL AFTER ABOVE CALL TO REDPRM, IN CASE
L00683  ! ALTERNATIVE VALUES OF ROUGHNESS LENGTH (Z0) AND ZILINTINKEVICH COEF
L00684  ! (CZIL) ARE SET THERE VIA NAMELIST I/O.
L00685
L00686  ! NOTE !!!
L00687  ! ROUTINE SFCDIF RETURNS A CH THAT REPRESENTS THE WIND SPD TIMES THE
L00688  ! "ORIGINAL" NONDIMENSIONAL "Ch" TYPICAL IN LITERATURE.  HENCE THE CH
L00689  ! RETURNED FROM SFCDIF HAS UNITS OF M/S.  THE IMPORTANT COMPANION
L00690  ! COEFFICIENT OF CH, CARRIED HERE AS "RCH", IS THE CH FROM SFCDIF TIMES
L00691  ! AIR DENSITY AND PARAMETER "CP".  "RCH" IS COMPUTED IN "CALL PENMAN".
L00692  ! RCH RATHER THAN CH IS THE COEFF USUALLY INVOKED LATER IN EQNS.
L00693
L00694  ! NOTE !!!
L00695  ! ---------------------------------------------------------------------
L00696  ! SFCDIF ALSO RETURNS THE SURFACE EXCHANGE COEFFICIENT FOR MOMENTUM, CM,
L00697  ! ALSO KNOWN AS THE SURFACE DRAGE COEFFICIENT. Needed as a state variable
L00698  ! for iterative/implicit solution of CH in SFCDIF
L00699  ! ---------------------------------------------------------------------
L00700  !        IF(.NOT.LCH) THEN
L00701  !          T1V = T1 * (1.0+ 0.61 * Q2)
L00702  !          TH2V = TH2 * (1.0+ 0.61 * Q2)
L00703  !          CALL SFCDIF_off (ZLVL,Z0,T1V,TH2V,SFCSPD,CZIL,CM,CH)
L00704  !        ENDIF
L00705
L00706  ! ---------------------------------------------------------------------
L00707  ! CALL PENMAN SUBROUTINE TO CALCULATE POTENTIAL EVAPORATION (ETP), AND
L00708  ! OTHER PARTIAL PRODUCTS AND SUMS SAVE IN COMMON/RITE FOR LATER
L00709  ! CALCULATIONS.
L00710  ! ---------------------------------------------------------------------
L00711  ! ---------------------------------------------------------------------
L00712  ! CALCULATE TOTAL DOWNWARD RADIATION (SOLAR PLUS LONGWAVE) NEEDED IN
L00713  ! PENMAN EP SUBROUTINE THAT FOLLOWS
L00714  ! ---------------------------------------------------------------------
L00715  !         FDOWN = SOLDN * (1.0- ALBEDO) + LWDN
L00716          FDOWN =  SOLNET + LWDN
L00717  ! ---------------------------------------------------------------------
L00718  ! CALC VIRTUAL TEMPS AND VIRTUAL POTENTIAL TEMPS NEEDED BY SUBROUTINES
L00719  ! PENMAN.
L00720          T2V = SFCTMP * (1.0+ 0.61 * Q2 )
L00721
L00722          iout=0
L00723          if(iout.eq.1) then
L00724          print*,'before penman'
L00725          print*,' SFCTMP',SFCTMP,'SFCPRS',SFCPRS,'CH',CH,'T2V',T2V,      &
L00726        'TH2',TH2,'PRCP',PRCP,'FDOWN',FDOWN,'T24',T24,'SSOIL',SSOIL,      &
L00727         'Q2',Q2,'Q2SAT',Q2SAT,'ETP',ETP,'RCH',RCH,                       &
L00728         'EPSCA',EPSCA,'RR',RR  ,'SNOWNG',SNOWNG,'FRZGRA',FRZGRA,           &
L00729         'DQSDT2',DQSDT2,'FLX2',FLX2,'SNOWH',SNOWH,'SNEQV',SNEQV,         &
L00730         ' DSOIL',DSOIL,' FRCSNO',FRCSNO,' SNCOVR',SNCOVR,' DTOT',DTOT,   &
L00731        ' ZSOIL (1)',ZSOIL(1),' DF1',DF1,'T1',T1,' STC1',STC(1),          &
L00732         'ALBEDO',ALBEDO,'SMC',SMC,'STC',STC,'SH2O',SH2O
L00733          endif
L00734
L00735          CALL PENMAN (SFCTMP,SFCPRS,CH,T2V,TH2,PRCP,FDOWN,T24,SSOIL,      &
L00736                       Q2,Q2SAT,ETP,RCH,EPSCA,RR,SNOWNG,FRZGRA,             &
L00737                       DQSDT2,FLX2,EMISSI,SNEQV,T1,SNCOVR,AOASIS,           &
L00738               ALBEDO,SOLDN,FVB,GAMA,STC(1),ETPN,FLX4,UA_PHYS)
L00739  !
L00740  ! ---------------------------------------------------------------------
L00741  ! CALL CANRES TO CALCULATE THE CANOPY RESISTANCE AND CONVERT IT INTO PC
L00742  ! IF NONZERO GREENNESS FRACTION
L00743  ! ---------------------------------------------------------------------
L00744
L00745  ! ---------------------------------------------------------------------
L00746  !  FROZEN GROUND EXTENSION: TOTAL SOIL WATER "SMC" WAS REPLACED
L00747  !  BY UNFROZEN SOIL WATER "SH2O" IN CALL TO CANRES BELOW
L00748  ! ---------------------------------------------------------------------
L00749          IF ( (SHDFAC > 0.) .AND. (XLAI > 0.) ) THEN
L00750             CALL CANRES (SOLDN,CH,SFCTMP,Q2,SFCPRS,SH2O,ZSOIL,NSOIL,      &
L00751                          SMCWLT,SMCREF,RSMIN,RC,PC,NROOT,Q2SAT,DQSDT2,    &
L00752                          TOPT,RSMAX,RGL,HS,XLAI,                          &
L00753                          RCS,RCT,RCQ,RCSOIL,EMISSI)
L00754          ELSE
L00755             RC = 0.0
L00756          END IF
L00757  ! ---------------------------------------------------------------------
L00758  ! NOW DECIDE MAJOR PATHWAY BRANCH TO TAKE DEPENDING ON WHETHER SNOWPACK
L00759  ! EXISTS OR NOT:
L00760  ! ---------------------------------------------------------------------
L00761          ESNOW = 0.0
L00762          IF (SNEQV  == 0.0) THEN
L00763             CALL NOPAC (ETP,ETA,PRCP,SMC,SMCMAX,SMCWLT,                   &
L00764                         SMCREF,SMCDRY,CMC,CMCMAX,NSOIL,DT,                &
L00765                         SHDFAC,                                           &
L00766                         SBETA,Q2,T1,SFCTMP,T24,TH2,FDOWN,F1,EMISSI,       &
L00767                         SSOIL,                                            &
L00768                         STC,EPSCA,BEXP,PC,RCH,RR,CFACTR,                  &
L00769                         SH2O,SLOPE,KDT,FRZX,PSISAT,ZSOIL,                 &
L00770                         DKSAT,DWSAT,TBOT,ZBOT,RUNOFF1,RUNOFF2,            &
L00771                         RUNOFF3,EDIR,EC,ET,ETT,NROOT,RTDIS,               &
L00772                         QUARTZ,FXEXP,CSOIL,                               &
L00773                         BETA,DRIP,DEW,FLX1,FLX3,VEGTYP,ISURBAN,           &
L00774                         SFHEAD1RT,INFXS1RT,ETPND1,SOILTYP,OPT_THCND       &
L00775                        ,XSDA_QFX,QFX_PHY,XQNORM,fasdas,HCPCT_FASDAS,IRRIGATION_CHANNEL  ) !fasdas
L00776
L00777             ETA_KINEMATIC = ETA
L00778          ELSE
L00779             CALL SNOPAC (ETP,ETA,PRCP,PRCPF,SNOWNG,SMC,SMCMAX,SMCWLT,     &
L00780                          SMCREF,SMCDRY,CMC,CMCMAX,NSOIL,DT,                &
L00781                          SBETA,DF1,                                        &
L00782                          Q2,T1,SFCTMP,T24,TH2,FDOWN,F1,SSOIL,STC,EPSCA,    &
L00783                          SFCPRS,BEXP,PC,RCH,RR,CFACTR,SNCOVR,SNEQV,SNDENS,&
L00784                          SNOWH,SH2O,SLOPE,KDT,FRZX,PSISAT,                 &
L00785                          ZSOIL,DWSAT,DKSAT,TBOT,ZBOT,SHDFAC,RUNOFF1,       &
L00786                          RUNOFF2,RUNOFF3,EDIR,EC,ET,ETT,NROOT,SNOMLT,      &
L00787                          RTDIS,QUARTZ,FXEXP,CSOIL,                         &
L00788                          BETA,DRIP,DEW,FLX1,FLX2,FLX3,ESNOW,ETNS,EMISSI,   &
```

```
L00789                          RIBB,SOLDN,                                     &
L00790                          ISURBAN,                                        &
L00791                          VEGTYP,                                         &
L00792                          ETPN,FLX4,UA_PHYS,                              &
L00793                          SFHEAD1RT,INFXS1RT,ETPND1,SOILTYP,OPT_THCND     &
L00794                         ,QFX_PHY,fasdas,HCPCT_FASDAS                     ) !fasdas
L00795             ETA_KINEMATIC =  ESNOW + ETNS - 1000.0*DEW
L00796           END IF
L00797
L00798  !     Calculate effective mixing ratio at grnd level (skin)
L00799  !
L00800  !     Q1=Q2+ETA*CP/RCH
L00801       Q1=Q2+ETA_KINEMATIC*CP/RCH
L00802  !
L00803  ! ---------------------------------------------------------------------
L00804  ! DETERMINE SENSIBLE HEAT (H) IN ENERGY UNITS (W M-2)
L00805  ! ---------------------------------------------------------------------
L00806
L00807           SHEAT = - (CH * CP * SFCPRS)/ (R * T2V) * ( TH2- T1 )
L00808           IF(UA_PHYS) SHEAT = SHEAT + FLX4
L00809  !
L00810  ! FASDAS
L00811  !
L00812        IF ( fasdas == 1 ) THEN
L00813          HFX_PHY = SHEAT
L00814        ENDIF
L00815  !
L00816  ! END FASDAS
L00817  !
L00818  ! ---------------------------------------------------------------------
L00819  ! CONVERT EVAP TERMS FROM KINEMATIC (KG M-2 S-1) TO ENERGY UNITS (W M-2)
L00820  ! ---------------------------------------------------------------------
L00821        EDIR = EDIR * LVH2O
L00822        EC = EC * LVH2O
L00823        DO K=1,4
L00824        ET(K) = ET(K) * LVH2O
L00825        ENDDO
L00826        ETT = ETT * LVH2O
L00827
L00828        ETPND1=ETPND1 * LVH2O
L00829
L00830        ESNOW = ESNOW * LSUBS
L00831        ETP = ETP*((1.-SNCOVR)*LVH2O + SNCOVR*LSUBS)
L00832        IF(UA_PHYS) ETPN = ETPN*((1.-SNCOVR)*LVH2O + SNCOVR*LSUBS)
L00833        IF (ETP .GT. 0.) THEN
L00834           ETA = EDIR + EC + ETT + ESNOW
L00835        ELSE
L00836          ETA = ETP
L00837        ENDIF
L00838  ! ---------------------------------------------------------------------
L00839  ! DETERMINE BETA (RATIO OF ACTUAL TO POTENTIAL EVAP)
L00840  ! ---------------------------------------------------------------------
L00841        IF (ETP == 0.0) THEN
L00842          BETA = 0.0
L00843        ELSE
L00844          BETA = ETA/ETP
L00845        ENDIF
L00846
L00847  ! ---------------------------------------------------------------------
L00848  ! CONVERT THE SIGN OF SOIL HEAT FLUX SO THAT:
L00849  !   SSOIL>0: WARM THE SURFACE  (NIGHT TIME)
L00850  !   SSOIL<0: COOL THE SURFACE  (DAY TIME)
L00851  ! ---------------------------------------------------------------------
L00852           SSOIL = -1.0* SSOIL
L00853
L00854  ! ---------------------------------------------------------------------
L00855  !  FOR THE CASE OF LAND:
L00856  !  CONVERT RUNOFF3 (INTERNAL LAYER RUNOFF FROM SUPERSAT) FROM M TO M S-1
L00857  !  AND ADD TO SUBSURFACE RUNOFF/DRAINAGE/BASEFLOW.  RUNOFF2 IS ALREADY
L00858  !  A RATE AT THIS POINT
L00859  ! ---------------------------------------------------------------------
L00860           RUNOFF3 = RUNOFF3/ DT
L00861           RUNOFF2 = RUNOFF2+ RUNOFF3
L00862           SOILM = -1.0* SMC (1)* ZSOIL (1)
L00863           DO K = 2,NSOIL
L00864              SOILM = SOILM + SMC (K)* (ZSOIL (K -1) - ZSOIL (K))
L00865           END DO
L00866           SOILWM = -1.0* (SMCMAX - SMCWLT)* ZSOIL (1)
L00867           SOILWW = -1.0* (SMC (1) - SMCWLT)* ZSOIL (1)
L00868
L00869           DO K = 1,NSOIL
L00870              SMAV(K)=(SMC(K) - SMCWLT)/(SMCMAX - SMCWLT)
L00871           END DO
L00872
L00873           IF (NROOT >= 2) THEN
L00874              DO K = 2,NROOT
L00875                   SOILWM = SOILWM + (SMCMAX - SMCWLT)* (ZSOIL (K -1) - ZSOIL (K))
L00876                   SOILWW = SOILWW + (SMC(K) - SMCWLT)* (ZSOIL (K -1) - ZSOIL (K))
L00877              END DO
L00878           END IF
L00879           IF (SOILWM .LT. 1.E-6) THEN
L00880             SOILWM = 0.0
L00881             SOILW  = 0.0
L00882             SOILM  = 0.0
L00883           ELSE
L00884             SOILW = SOILWW / SOILWM
L00885           END IF
L00886
L00887  ! ---------------------------------------------------------------------
L00888    END SUBROUTINE SFLX
```

讲解：Noah 的一次积分从参数准备到通量输出共十二站。**接口合同**（L101–293）：参数按 C/L/F/I/S/H/O/D/P 九类注释——Noah 统一版 2007 的完整文档，DT 建议 1800 s 及以下；IRRIGATION_CHANNEL 是 OPTIONAL 参数（灌溉通道，透传给 SMFLX，见 § 六）。

① **ZSOIL 负深**（L414–417）：ZSOIL(1)=−SLDPTH(1) 向下累加为负值——全模块「负深」约定的源头，一切层厚公式（如 −0.5 • ZSOIL(2) 作半层距）都依赖这个符号约定。

② **REDPRM 查表 + urban 硬参数**（L423–440）：全部土壤/植被/通用参数查表（12b § 三）；紧接着城市硬覆盖：VEGTYP==ISURBAN 时 SHDFAC=0.05、RSMIN=400、SMC 四值 0.45/0.42/0.40/0.40——城市格点不信任地表数据集给的植被与土壤参数，直接硬编码。这是**改城市下垫面水热性质的第一入口**（更精细的城市方案走 BEP/BEM，ch17）。

③ **绿度插值**（L442–491）：EMBRD（发射率）、XLAI、ALB、Z0BRD（粗糙度）四个背景量由 SHDFAC 在 SHDMAX/SHDMIN 区间的位置决定——封顶取 EMISSMAX/LAIMAX/ALBEDOMIN/Z0MAX，裸土取反，中间按 INTERP_FRACTION 线性插值（钳位 0–1）；SHDMAX≤SHDMIN 的异常格点退化为 0.5 混合。RDLAI2D/USEMONALB 为真时 LAI/ALB 保留外场输入不覆盖。

④ **降水分型与雪增加**（L495–556）：SNEQV≤1e-7 清雪态（kmh 2008/03/25 加固，防残留小量）；有雪时 SNDENS=SNEQV/SNOWH（大于 1 报物理错误）并调 CSNOW 算雪导热；降水分型看 FFROZP——微物理方案给的冻水比例大于 0.5 判雪，否则地面温度低于冰点判冻雨；雪/冻雨时新雪 SN_NEW=PRCP • DT • 0.001 全部进雪库、PRCPF=0（雪天雨水不入土），SNOW_NEW（12b）+CSNOW 更新密度与导热；雨天 PRCPF=PRCP 留给入渗。

⑤ **雪盖与反照率**（L563–592）：无雪直通 SNCOVR=0/ALBEDO=ALB/EMISSI=EMBRD；有雪调 SNFRAC（12b，SALP 曲线；UA 还输出 FVB/GAMA/FBUR/FGSN 并按雪埋拖曳 RU 修正 CH=CH/(1+RU • CH)）；SNCOVR 上限 0.98；ALCALC（§ 五）算雪龄反照率。

⑥ **热扩散与雪的平面平行串联**（L617–665）：TDFCND（12b）算表层导热 DF1（OPT_THCND=1 走 Peters-Lidard 1998，=2 时砂壤/粉砂壤退回 McCumber-Pielke 老公式）；urban 格点 DF1=3.24 硬编码；DF1 • exp(SBETA • SHDFAC) 是冠层遮蔽对地热通量的衰减（Peters-Lidard 1997 §2.1.2）；SNCOVR>0.97 时 DF1 直接取雪导热。有雪时把「半层土壤 +雪层」当两块平面平行介质：串联用调和平均 DF1H、并联用算术平均 DF1A、几何平均式仅注释，实际取 DF1=DF1A • SNCOVR+DF1 • (1−SNCOVR) 按雪盖率线性混合；SSOIL=DF1 • (T1−STC1)/DTOT。Lunardini 1981 的这套处理让薄雪极限不再病态——老 Noah 需对 SSOIL 设上限的问题就此消除。

⑦ **SNOWZ0**（L670–676）：SNCOVR>0 调 SNOWZ0（12b）算雪面粗糙度；否则 Z0=Z0BRD。

⑧ **PENMAN**（L716–738）：FDOWN=SOLNET+LWDN——净短波由驱动层直接给（注释保留 SOLDN • (1−ALBEDO)+LWDN 老式）；T2V 虚温；PENMAN（§ 十）输出潜势蒸发 ETP 与 RCH/EPSCA/RR/T24/FLX2 全套中间量。

⑨ **CANRES**（L749–756）：SHDFAC>0 且 XLAI>0 才算冠层阻抗 RC 与植物系数 PC（§ 五）；冻土版把 SMC 换成未冻水 SH2O。

⑩ **主分支**（L761–796）：SNEQV==0 走 NOPAC（§ 九），否则 SNOPAC（12b § 九）；ETA_KINEMATIC 相应取 ETA 或 ESNOW+ETNS−1000 • DEW。

⑪ **收尾能量换算**（L801–837）：Q1=Q2+ETA_KINEMATIC • CP/RCH——皮肤面有效混合比，驱动层据此诊断 2 m 比湿；SHEAT=−CH • CP • SFCPRS/(R • T2V) • (TH2−T1)，UA 时加 FLX4；fasdas=1 时 HFX_PHY=SHEAT（同化挂钩）。蒸发项乘 LVH2O 转 W/m²；**注意 DO K=1,4——ET 只转前四层**，Noah 标准四层假设的硬编码，改层数的实验必须检查这里；ETP 按 (1−SNCOVR) • LVH2O+SNCOVR • LSUBS 混合潜热；ETP>0 时 ETA=EDIR+EC+ETT+ESNOW，否则 ETA=ETP（露/凝华整体计为潜热通量）。

⑫ **收尾水量诊断**（L839–885）：BETA=ETA/ETP；SSOIL 变号为「向下加热地面为正」；RUNOFF3/DT（层间超孔隙水）并入 RUNOFF2；SOILM/SOILWM/SOILWW/SOILW/SMAV 是根区含水量诊断量。

五、雪反照率 ALCALC 与冠层阻抗 CANRES

```
!file: phys/module_sf_noahlsm.F    L891-L1006    (subroutine ALCALC)
L00891        SUBROUTINE ALCALC (ALB,SNOALB,EMBRD,SHDFAC,SHDMIN,SNCOVR,TSNOW,ALBEDO,EMISSI,   &
L00892                           DT,SNOWNG,SNOTIME1,LVCOEF)
L00893
L00894  ! ----------------------------------------------------------------------
L00895  ! CALCULATE ALBEDO INCLUDING SNOW EFFECT (0 -> 1)
L00896  !   ALB     SNOWFREE ALBEDO
L00897  !   SNOALB  MAXIMUM (DEEP) SNOW ALBEDO
L00898  !   SHDFAC    AREAL FRACTIONAL COVERAGE OF GREEN VEGETATION
L00899  !   SHDMIN    MINIMUM AREAL FRACTIONAL COVERAGE OF GREEN VEGETATION
L00900  !   SNCOVR  FRACTIONAL SNOW COVER
L00901  !   ALBEDO  SURFACE ALBEDO INCLUDING SNOW EFFECT
L00902  !   TSNOW   SNOW SURFACE TEMPERATURE (K)
L00903  ! ----------------------------------------------------------------------
L00904        IMPLICIT NONE
L00905
L00906  ! ----------------------------------------------------------------------
L00907  ! SNOALB IS ARGUMENT REPRESENTING MAXIMUM ALBEDO OVER DEEP SNOW,
L00908  ! AS PASSED INTO SFLX, AND ADAPTED FROM THE SATELLITE-BASED MAXIMUM
L00909  ! SNOW ALBEDO FIELDS PROVIDED BY D. ROBINSON AND G. KUKLA
```

```
L00910  ! (1985, JCAM, VOL 24, 402-411)
L00911  ! ----------------------------------------------------------------------
L00912        REAL, INTENT(IN)  ::  ALB, SNOALB, EMBRD, SHDFAC, SHDMIN, SNCOVR, TSNOW
L00913        REAL, INTENT(IN)  :: DT
L00914        LOGICAL, INTENT(IN) :: SNOWNG
L00915        REAL, INTENT(INOUT):: SNOTIME1
L00916        REAL, INTENT(OUT) ::  ALBEDO, EMISSI
L00917        REAL              :: SNOALB2
L00918        REAL              :: TM,SNOALB1
L00919        REAL, INTENT(IN)  :: LVCOEF
L00920        REAL, PARAMETER   :: SNACCA=0.94,SNACCB=0.58,SNTHWA=0.82,SNTHWB=0.46
L00921  ! turn of vegetation effect
L00922  !      ALBEDO = ALB + (1.0- (SHDFAC - SHDMIN))* SNCOVR * (SNOALB - ALB)
L00923  !      ALBEDO = (1.0-SNCOVR)*ALB + SNCOVR*SNOALB !this is equivalent to below
L00924        ALBEDO = ALB + SNCOVR*(SNOALB-ALB)
L00925        EMISSI = EMBRD + SNCOVR*(EMISSI_S - EMBRD)
L00926
L00927  !     BASE FORMULATION (DICKINSON ET AL., 1986, COGLEY ET AL., 1990)
L00928  !          IF (TSNOW.LE.263.16) THEN
L00929  !            ALBEDO=SNOALB
L00930  !          ELSE
L00931  !            IF (TSNOW.LT.273.16) THEN
L00932  !              TM=0.1*(TSNOW-263.16)
L00933  !              SNOALB1=0.5*((0.9-0.2*(TM**3))+(0.8-0.16*(TM**3)))
L00934  !            ELSE
L00935  !              SNOALB1=0.67
L00936  !             IF(SNCOVR.GT.0.95) SNOALB1= 0.6
L00937  !              SNOALB1 = ALB + SNCOVR*(SNOALB-ALB)
L00938  !            ENDIF
L00939  !          ENDIF
L00940  !            ALBEDO = ALB + SNCOVR*(SNOALB1-ALB)
L00941
L00942  !     ISBA FORMULATION (VERSEGHY, 1991; BAKER ET AL., 1990)
L00943  !          SNOALB1 = SNOALB+COEF*(0.85-SNOALB)
L00944  !          SNOALB2=SNOALB1
L00945  !!m          LSTSNW=LSTSNW+1
L00946  !          SNOTIME1 = SNOTIME1 + DT
L00947  !          IF (SNOWNG) THEN
L00948  !             SNOALB2=SNOALB
L00949  !!m            LSTSNW=0
L00950  !            SNOTIME1 = 0.0
L00951  !          ELSE
L00952  !            IF (TSNOW.LT.273.16) THEN
L00953  !!             SNOALB2=SNOALB-0.008*LSTSNW*DT/86400
L00954  !!m              SNOALB2=SNOALB-0.008*SNOTIME1/86400
L00955  !              SNOALB2=(SNOALB2-0.65)*EXP(-0.05*DT/3600)+0.65
L00956  !!             SNOALB2=(ALBEDO-0.65)*EXP(-0.01*DT/3600)+0.65
L00957  !            ELSE
L00958  !              SNOALB2=(SNOALB2-0.5)*EXP(-0.0005*DT/3600)+0.5
L00959  !!             SNOALB2=(SNOALB-0.5)*EXP(-0.24*LSTSNW*DT/86400)+0.5
L00960  !!m              SNOALB2=(SNOALB-0.5)*EXP(-0.24*SNOTIME1/86400)+0.5
L00961  !            ENDIF
L00962  !          ENDIF
L00963  !
L00964  !!              print*,'SNOALB2',SNOALB2,'ALBEDO',ALBEDO,'DT',DT
L00965  !          ALBEDO = ALB + SNCOVR*(SNOALB2-ALB)
L00966  !          IF (ALBEDO .GT. SNOALB2) ALBEDO=SNOALB2
L00967  !!m          LSTSNW1=LSTSNW
L00968  !!         SNOTIME = SNOTIME1
L00969
L00970  ! formulation by Livneh
L00971  ! ----------------------------------------------------------------------
L00972  ! SNOALB IS CONSIDERED AS THE MAXIMUM SNOW ALBEDO FOR NEW SNOW, AT
L00973  ! A VALUE OF 85%. SNOW ALBEDO CURVE DEFAULTS ARE FROM BRAS P.263. SHOULD
L00974  ! NOT BE CHANGED EXCEPT FOR SERIOUS PROBLEMS WITH SNOW MELT.
L00975  ! TO IMPLEMENT ACCUMULATIN PARAMETERS, SNACCA AND SNACCB, ASSERT THAT IT
L00976  ! IS INDEED ACCUMULATION SEASON. I.E. THAT SNOW SURFACE TEMP IS BELOW
L00977  ! ZERO AND THE DATE FALLS BETWEEN OCTOBER AND FEBRUARY
L00978  ! ----------------------------------------------------------------------
L00979          SNOALB1 = SNOALB+LVCOEF*(0.85-SNOALB)
L00980          SNOALB2=SNOALB1
L00981  ! ---------------- Initial LSTSNW -------------------------------------
L00982           IF (SNOWNG) THEN
L00983              SNOTIME1 = 0.
L00984           ELSE
L00985            SNOTIME1=SNOTIME1+DT
L00986  !              IF (TSNOW.LT.273.16) THEN
L00987                 SNOALB2=SNOALB1*(SNACCA**((SNOTIME1/86400.0)**SNACCB))
L00988  !              ELSE
L00989  !                 SNOALB2 =SNOALB1*(SNTHWA**((SNOTIME1/86400.0)**SNTHWB))
L00990  !              ENDIF
L00991           ENDIF
L00992  !
L00993            SNOALB2 = MAX ( SNOALB2, ALB )
L00994            ALBEDO = ALB + SNCOVR*(SNOALB2-ALB)
L00995            IF (ALBEDO .GT. SNOALB2) ALBEDO=SNOALB2
L00996
L00997  !         IF (TSNOW.LT.273.16) THEN
L00998  !           ALBEDO=SNOALB-0.008*DT/86400
L00999  !         ELSE
L01000  !           ALBEDO=(SNOALB-0.5)*EXP(-0.24*DT/86400)+0.5
L01001  !         ENDIF
L01002
L01003  !      IF (ALBEDO > SNOALB) ALBEDO = SNOALB
L01004
L01005  ! ----------------------------------------------------------------------
L01006    END SUBROUTINE ALCALC
```

讲解：雪反照率三套公式并存——Dickinson 1986/Cogley 1990 与 ISBA（Verseghy 1991）两套完整注释保留，活跃代码是**线性混合 + Livneh 老化曲线**：① 混合式 ALBEDO=ALB+SNCOVR • (SNOALB−ALB)、EMISSI=EMBRD+SNCOVR • (EMISSI_S−EMBRD)（雪面发射率 0.95）；②Livneh 老化：SNOALB1=SNOALB+LVCOEF • (0.85−SNOALB)——卫星最大雪反照率场（Robinson–Kukla 1985）被 LVCOEF（GENPARM.TBL）拉向 0.85 的「新雪」值；降雪时雪龄 SNOTIME1 清零，否则按秒累计；SNOALB2=SNOALB1 • SNACCA$^{((SNOTIME1/86400)}$SNACCB)（SNACCA=0.94、SNACCB=

0.58，Bras P.263 曲线；融雪期参数 SNTHWA/SNTHWB 被注释——当前版本不区分积/融季）；③SNOALB2 下限取 ALB（不能比裸土还暗反着改），最终 ALBEDO 不超过 SNOALB2。改雪反照率方案就在这个函数，注释里的两套老方案是现成参考。

```
!file: phys/module_sf_noahlsm.F    L1009-L1145    (subroutine CANRES)
L01009        SUBROUTINE CANRES (SOLAR,CH,SFCTMP,Q2,SFCPRS,SMC,ZSOIL,NSOIL,        &
L01010                           SMCWLT,SMCREF,RSMIN,RC,PC,NROOT,Q2SAT,DQSDT2,     &
L01011                           TOPT,RSMAX,RGL,HS,XLAI,                           &
L01012                           RCS,RCT,RCQ,RCSOIL,EMISSI)
L01013
L01014  ! ----------------------------------------------------------------------
L01015  ! SUBROUTINE CANRES
L01016  ! ----------------------------------------------------------------------
L01017  ! CALCULATE CANOPY RESISTANCE WHICH DEPENDS ON INCOMING SOLAR RADIATION,
L01018  ! AIR TEMPERATURE, ATMOSPHERIC WATER VAPOR PRESSURE DEFICIT AT THE
L01019  ! LOWEST MODEL LEVEL, AND SOIL MOISTURE (PREFERABLY UNFROZEN SOIL
L01020  ! MOISTURE RATHER THAN TOTAL)
L01021  ! ----------------------------------------------------------------------
L01022  ! SOURCE:  JARVIS (1976), NOILHAN AND PLANTON (1989, MWR), JACQUEMIN AND
L01023  ! NOILHAN (1990, BLM)
L01024  ! SEE ALSO:  CHEN ET AL (1996, JGR, VOL 101(D3), 7251-7268), EQNS 12-14
L01025  ! AND TABLE 2 OF SEC. 3.1.2
L01026  ! ----------------------------------------------------------------------
L01027  ! INPUT:
L01028  !   SOLAR   INCOMING SOLAR RADIATION
L01029  !   CH      SURFACE EXCHANGE COEFFICIENT FOR HEAT AND MOISTURE
L01030  !   SFCTMP  AIR TEMPERATURE AT 1ST LEVEL ABOVE GROUND
L01031  !   Q2      AIR HUMIDITY AT 1ST LEVEL ABOVE GROUND
L01032  !   Q2SAT   SATURATION AIR HUMIDITY AT 1ST LEVEL ABOVE GROUND
L01033  !   DQSDT2  SLOPE OF SATURATION HUMIDITY FUNCTION WRT TEMP
L01034  !   SFCPRS  SURFACE PRESSURE
L01035  !   SMC     VOLUMETRIC SOIL MOISTURE
L01036  !   ZSOIL   SOIL DEPTH (NEGATIVE SIGN, AS IT IS BELOW GROUND)
L01037  !   NSOIL   NO. OF SOIL LAYERS
L01038  !   NROOT   NO. OF SOIL LAYERS IN ROOT ZONE (1.LE.NROOT.LE.NSOIL)
L01039  !   XLAI    LEAF AREA INDEX
L01040  !   SMCWLT  WILTING POINT
L01041  !   SMCREF  REFERENCE SOIL MOISTURE (WHERE SOIL WATER DEFICIT STRESS
L01042  !             SETS IN)
L01043  ! RSMIN, RSMAX, TOPT, RGL, HS ARE CANOPY STRESS PARAMETERS SET IN
L01044  !   SURBOUTINE REDPRM
L01045  ! OUTPUT:
L01046  !   PC  PLANT COEFFICIENT
L01047  !   RC  CANOPY RESISTANCE
L01048  ! ----------------------------------------------------------------------
L01049
L01050        IMPLICIT NONE
L01051        INTEGER, INTENT(IN) :: NROOT,NSOIL
L01052        INTEGER  K
L01053        REAL,    INTENT(IN) :: CH,DQSDT2,HS,Q2,Q2SAT,RSMIN,RGL,RSMAX,        &
L01054                               SFCPRS,SFCTMP,SMCREF,SMCWLT, SOLAR,TOPT,XLAI, &
L01055                               EMISSI
L01056        REAL,DIMENSION(1:NSOIL), INTENT(IN) :: SMC,ZSOIL
L01057        REAL,    INTENT(OUT):: PC,RC,RCQ,RCS,RCSOIL,RCT
L01058        REAL                :: DELTA,FF,GX,P,RR
L01059        REAL, DIMENSION(1:NSOIL) ::  PART
L01060        REAL, PARAMETER     :: SLV = 2.501000E6
L01061
L01062
L01063  ! ----------------------------------------------------------------------
L01064  ! INITIALIZE CANOPY RESISTANCE MULTIPLIER TERMS.
L01065  ! ----------------------------------------------------------------------
L01066        RCS = 0.0
L01067        RCT = 0.0
L01068        RCQ = 0.0
L01069        RCSOIL = 0.0
L01070
L01071  ! ----------------------------------------------------------------------
L01072  ! CONTRIBUTION DUE TO INCOMING SOLAR RADIATION
L01073  ! ----------------------------------------------------------------------
L01074        RC = 0.0
L01075        FF = 0.55*2.0* SOLAR / (RGL * XLAI)
L01076        RCS = (FF + RSMIN / RSMAX) / (1.0+ FF)
L01077
L01078  ! ----------------------------------------------------------------------
L01079  ! CONTRIBUTION DUE TO AIR TEMPERATURE AT FIRST MODEL LEVEL ABOVE GROUND
L01080  ! RCT EXPRESSION FROM NOILHAN AND PLANTON (1989, MWR).
L01081  ! ----------------------------------------------------------------------
L01082        RCS = MAX (RCS,0.0001)
L01083        RCT = 1.0- 0.0016* ( (TOPT - SFCTMP)**2.0)
L01084
L01085  ! ----------------------------------------------------------------------
L01086  ! CONTRIBUTION DUE TO VAPOR PRESSURE DEFICIT AT FIRST MODEL LEVEL.
L01087  ! RCQ EXPRESSION FROM SSIB
L01088  ! ----------------------------------------------------------------------
L01089        RCT = MAX (RCT,0.0001)
L01090        RCQ = 1.0/ (1.0+ HS * (Q2SAT - Q2))
L01091
L01092  ! ----------------------------------------------------------------------
L01093  ! CONTRIBUTION DUE TO SOIL MOISTURE AVAILABILITY.
L01094  ! DETERMINE CONTRIBUTION FROM EACH SOIL LAYER, THEN ADD THEM UP.
L01095  ! ----------------------------------------------------------------------
L01096        RCQ = MAX (RCQ,0.01)
L01097        GX = (SMC (1) - SMCWLT) / (SMCREF - SMCWLT)
L01098        IF (GX  >  1.) GX = 1.
L01099        IF (GX  <  0.) GX = 0.
L01100
L01101  ! ----------------------------------------------------------------------
L01102  ! USE SOIL DEPTH AS WEIGHTING FACTOR
L01103  ! ----------------------------------------------------------------------
L01104  ! ----------------------------------------------------------------------
L01105  ! USE ROOT DISTRIBUTION AS WEIGHTING FACTOR
L01106  !      PART(1) = RTDIS(1) * GX
L01107  ! ----------------------------------------------------------------------
L01108        PART (1) = (ZSOIL (1)/ ZSOIL (NROOT)) * GX
L01109        DO K = 2,NROOT
L01110           GX = (SMC (K) - SMCWLT) / (SMCREF - SMCWLT)
```

```
L01111           IF (GX >  1.) GX = 1.
L01112           IF (GX <  0.) GX = 0.
L01113   ! ----------------------------------------------------------------------
L01114   ! USE SOIL DEPTH AS WEIGHTING FACTOR
L01115   ! ----------------------------------------------------------------------
L01116   ! ----------------------------------------------------------------------
L01117   ! USE ROOT DISTRIBUTION AS WEIGHTING FACTOR
L01118   !        PART(K) = RTDIS(K) * GX
L01119   ! ----------------------------------------------------------------------
L01120           PART (K) = ( (ZSOIL (K) - ZSOIL (K -1))/ ZSOIL (NROOT)) * GX
L01121         END DO
L01122         DO K = 1,NROOT
L01123            RCSOIL = RCSOIL + PART (K)
L01124         END DO
L01125
L01126   ! ----------------------------------------------------------------------
L01127   ! DETERMINE CANOPY RESISTANCE DUE TO ALL FACTORS.  CONVERT CANOPY
L01128   ! RESISTANCE (RC) TO PLANT COEFFICIENT (PC) TO BE USED WITH POTENTIAL
L01129   ! EVAP IN DETERMINING ACTUAL EVAP.  PC IS DETERMINED BY:
L01130   !   PC * LINERIZED PENMAN POTENTIAL EVAP =
L01131   !   PENMAN-MONTEITH ACTUAL EVAPORATION (CONTAINING RC TERM).
L01132   ! ----------------------------------------------------------------------
L01133         RCSOIL = MAX (RCSOIL,0.0001)
L01134
L01135         RC = RSMIN / (XLAI * RCS * RCT * RCQ * RCSOIL)
L01136   !      RR = (4.* SIGMA * RD / CP)* (SFCTMP **4.)/ (SFCPRS * CH) + 1.0
L01137         RR = (4.* EMISSI *SIGMA * RD / CP)* (SFCTMP **4.)/ (SFCPRS * CH) &
L01138                + 1.0
L01139
L01140         DELTA = (SLV / CP)* DQSDT2
L01141
L01142         PC = (RR + DELTA)/ (RR * (1. + RC * CH) + DELTA)
L01143
L01144   ! ----------------------------------------------------------------------
L01145     END SUBROUTINE CANRES
```

讲解：Jarvis (1976) 型气孔阻抗四因子连乘：RCS 辐射因子 =(FF+RSMIN/RSMAX)/(1+FF)、FF=0.55 • 2 • SOLAR/(RGL • XLAI)；RCT 温度因子 =1−0.0016 • (TOPT−SFCTMP)²（Noilhan–Planton 1989）；RCQ 湿度差因子 =1/(1+HS • (Q2SAT−Q2))（SSiB 口径）；RCSOIL 土壤水因子——逐根层 GX=(SMC−SMCWLT)/(SMCREF−SMCWLT) 钳位 0–1 后按「层厚/根区总深」加权（PART 数组；注释保留 RTDIS 加权备选——与 TRANSP 的 RTDIS+GX 联合权重是两套不同源的分法）。四因子各设下限（1e-4/1e-4/0.01/1e-4）防切线性病态。RC=RSMIN/(XLAI • RCS • RCT • RCQ • RCSOIL)。**PC 植物系数**是 Penman–Monteith 的线性化：RR=4 • EMISSI • σ • RD/CP • SFCTMP⁴/(SFCPRS • CH)+1（长波线性化项，含发射率——与 PENMAN 的 RR 同式，两处必须一致地改）；DELTA=(SLV/CP) • DQSDT2；PC=(RR+DELTA)/(RR • (1+RC • CH)+DELTA)，含义是「PC • 潜势蒸发 = 含气孔阻抗的实际蒸腾」。

六、雪导热 CSNOW 与蒸发线 DEVAP、DEVAP_hydro、EVAPO、FAC2MIT

```
!file: phys/module_sf_noahlsm.F    L1148-L1187    (subroutine CSNOW)
L01148         SUBROUTINE CSNOW (SNCOND,DSNOW)
L01149
L01150   ! ----------------------------------------------------------------------
L01151   ! SUBROUTINE CSNOW
L01152   ! FUNCTION CSNOW
L01153   ! ----------------------------------------------------------------------
L01154   ! CALCULATE SNOW TERMAL CONDUCTIVITY
L01155   ! ----------------------------------------------------------------------
L01156         IMPLICIT NONE
L01157         REAL, INTENT(IN) :: DSNOW
L01158         REAL, INTENT(OUT):: SNCOND
L01159         REAL             :: C
L01160         REAL, PARAMETER  :: UNIT = 0.11631
L01161
L01162   ! ----------------------------------------------------------------------
L01163   ! SNCOND IN UNITS OF CAL/(CM*HR*C), RETURNED IN W/(M*C)
L01164   ! CSNOW IN UNITS OF CAL/(CM*HR*C), RETURNED IN W/(M*C)
L01165   ! BASIC VERSION IS DYACHKOVA EQUATION (1960), FOR RANGE 0.1-0.4
L01166   ! ----------------------------------------------------------------------
L01167         C = 0.328*10** (2.25* DSNOW)
L01168   !      CSNOW=UNIT*C
L01169
L01170   ! ----------------------------------------------------------------------
L01171   ! DE VAUX EQUATION (1933), IN RANGE 0.1-0.6
L01172   ! ----------------------------------------------------------------------
L01173   !      SNCOND=0.0293*(1.+100.*DSNOW**2)
L01174   !      CSNOW=0.0293*(1.+100.*DSNOW**2)
L01175
L01176   ! ----------------------------------------------------------------------
L01177   ! E. ANDERSEN FROM FLERCHINGER
L01178   ! ----------------------------------------------------------------------
L01179   !      SNCOND=0.021+2.51*DSNOW**2
L01180   !      CSNOW=0.021+2.51*DSNOW**2
L01181
L01182   !      SNCOND = UNIT * C
L01183   ! double snow thermal conductivity
L01184         SNCOND = 2.0 * UNIT * C
L01185
L01186   ! ----------------------------------------------------------------------
L01187     END SUBROUTINE CSNOW
```

讲解：雪导热三选一：活跃式 Dyachkova 1960 指数律 C=0.328 • 10^(2.25 • DSNOW)（0.1–0.4 g/cm³ 适用），乘 UNIT=0.11631 完成 cal/(cm • hr • K) 到 W/(m • K) 的换算，**再 ×2.0 经验翻倍**（注释明说

double snow thermal conductivity——WRF 团队的经验增强，改雪物理时的第一个嫌疑参数）；De Vaux 1933 与 Andersen（Flerchinger）公式注释保留。

```
!file: phys/module_sf_noahlsm.F    L1189-L1228    (subroutine DEVAP)
L01189        SUBROUTINE DEVAP (EDIR,ETP1,SMC,ZSOIL,SHDFAC,SMCMAX,BEXP,         &
L01190                          DKSAT,DWSAT,SMCDRY,SMCREF,SMCWLT,FXEXP)
L01191
L01192  ! ----------------------------------------------------------------------
L01193  ! SUBROUTINE DEVAP
L01194  ! FUNCTION DEVAP
L01195  ! ----------------------------------------------------------------------
L01196  ! CALCULATE DIRECT SOIL EVAPORATION
L01197  ! ----------------------------------------------------------------------
L01198        IMPLICIT NONE
L01199        REAL, INTENT(IN) :: ETP1,SMC,BEXP,DKSAT,DWSAT,FXEXP,              &
L01200                            SHDFAC,SMCDRY,SMCMAX,ZSOIL,SMCREF,SMCWLT
L01201        REAL, INTENT(OUT):: EDIR
L01202        REAL             :: FX, SRATIO
L01203
L01204
L01205  ! ----------------------------------------------------------------------
L01206  ! DIRECT EVAP A FUNCTION OF RELATIVE SOIL MOISTURE AVAILABILITY, LINEAR
L01207  ! WHEN FXEXP=1.
L01208  ! ----------------------------------------------------------------------
L01209  ! ----------------------------------------------------------------------
L01210  ! FX > 1 REPRESENTS DEMAND CONTROL
L01211  ! FX < 1 REPRESENTS FLUX CONTROL
L01212  ! ----------------------------------------------------------------------
L01213
L01214        SRATIO = (SMC - SMCDRY) / (SMCMAX - SMCDRY)
L01215        IF (SRATIO > 0.) THEN
L01216          FX = SRATIO**FXEXP
L01217          FX = MAX ( MIN ( FX, 1. ) ,0. )
L01218        ELSE
L01219          FX = 0.
L01220        ENDIF
L01221
L01222  ! ----------------------------------------------------------------------
L01223  ! ALLOW FOR THE DIRECT-EVAP-REDUCING EFFECT OF SHADE
L01224  ! ----------------------------------------------------------------------
L01225        EDIR = FX * ( 1.0- SHDFAC ) * ETP1
L01226
L01227  ! ----------------------------------------------------------------------
L01228    END SUBROUTINE DEVAP
```

讲解：直接土壤蒸发：SRATIO=(SMC−SMCDRY)/(SMCMAX−SMCDRY)，FX=SRATIO^FXEXP 钳位 0–1；FXEXP 来自 GENPARM.TBL（=1 线性，>1 偏需求控制、<1 偏通量控制——注释给了物理解释）；EDIR=FX • (1−SHDFAC) • ETP1，植被遮蔽按 (1−SHDFAC) 削减。

```
!file: phys/module_sf_noahlsm.F    L1230-L1320    (subroutine DEVAP_hydro)
L01230        SUBROUTINE DEVAP_hydro (EDIR,ETP1,SMC,ZSOIL,SHDFAC,SMCMAX,BEXP,         &
L01231                          DKSAT,DWSAT,SMCDRY,SMCREF,SMCWLT,FXEXP,         &
L01232                          SFHEAD1RT,ETPND1,DT)
L01233
L01234  ! ----------------------------------------------------------------------
L01235  ! SUBROUTINE DEVAP
L01236  ! FUNCTION DEVAP
L01237  ! ----------------------------------------------------------------------
L01238  ! CALCULATE DIRECT SOIL EVAPORATION
L01239  ! ----------------------------------------------------------------------
L01240        IMPLICIT NONE
L01241        REAL, INTENT(IN) :: ETP1,SMC,BEXP,DKSAT,DWSAT,FXEXP,              &
L01242                            SHDFAC,SMCDRY,SMCMAX,ZSOIL,SMCREF,SMCWLT
L01243        REAL, INTENT(OUT):: EDIR
L01244        REAL             :: FX, SRATIO
L01245
L01246        REAL, INTENT(INOUT) :: SFHEAD1RT,ETPND1
L01247        REAL, INTENT(IN   ) :: DT
L01248         REAL               :: EDIRTMP
L01249
L01250
L01251
L01252  ! ----------------------------------------------------------------------
L01253  ! DIRECT EVAP A FUNCTION OF RELATIVE SOIL MOISTURE AVAILABILITY, LINEAR
L01254  ! WHEN FXEXP=1.
L01255  ! ----------------------------------------------------------------------
L01256  ! ----------------------------------------------------------------------
L01257  ! FX > 1 REPRESENTS DEMAND CONTROL
L01258  ! FX < 1 REPRESENTS FLUX CONTROL
L01259  ! ----------------------------------------------------------------------
L01260
L01261        SRATIO = (SMC - SMCDRY) / (SMCMAX - SMCDRY)
L01262        IF (SRATIO > 0.) THEN
L01263          FX = SRATIO**FXEXP
L01264          FX = MAX ( MIN ( FX, 1. ) ,0. )
L01265        ELSE
L01266          FX = 0.
L01267        ENDIF
L01268
L01269  !DJG NDHMS/WRF-Hydro edits... Adjustment for ponded surface water : Reduce ETP1
L01270        EDIRTMP = 0.
L01271        ETPND1 = 0.
L01272
L01273  !DJG NDHMS/WRF-Hydro edits...  Calc Max Potential Dir Evap. (ETP1 units: }=m/s)
L01274
L01275  !DJG NDHMS/WRF-Hydro...currently set ponded water evap to 0.0 until further notice...11/5/2012
L01276  !EDIRTMP = ( 1.0- SHDFAC ) * ETP1
L01277
L01278  ! Convert all units to (m)
L01279  ! Convert EDIRTMP  from (kg m{-2} s{-1}=m/s) to (m) ...
L01280        EDIRTMP = EDIRTMP * DT
L01281
L01282  !DJG NDHMS/WRF-Hydro edits... Convert SFHEAD from (mm) to (m) ...
L01283        SFHEAD1RT=SFHEAD1RT * 0.001
```

```
L01284
L01285
L01286
L01287  !DJG NDHMS/WRF-Hydro edits... Calculate ETPND as reduction in EDIR(TMP)...
L01288        IF (EDIRTMP > 0.) THEN
L01289         IF ( EDIRTMP > SFHEAD1RT ) THEN
L01290           ETPND1 = SFHEAD1RT
L01291           SFHEAD1RT=0.
L01292           EDIRTMP = EDIRTMP - ETPND1
L01293         ELSE
L01294           ETPND1 = EDIRTMP
L01295           EDIRTMP = 0.
L01296           SFHEAD1RT = SFHEAD1RT - ETPND1
L01297         END IF
L01298        END IF
L01299
L01300  !DJG NDHMS/WRF-Hydro edits... Convert SFHEAD units back to (mm)
L01301        IF ( SFHEAD1RT /= 0.) SFHEAD1RT=SFHEAD1RT * 1000.
L01302
L01303  !DJG NDHMS/WRF-Hydro edits...Convert ETPND and EDIRTMP back to (mm/s=kg m{-2} s{-1})
L01304        ETPND1 = ETPND1 / DT
L01305        EDIRTMP = EDIRTMP / DT
L01306  !DEBUG    print *, "After DEVAP...SFCHEAD+ETPND1",SFHEAD1RT+ETPND1*DT
L01307
L01308
L01309  ! ----------------------------------------------------------------------
L01310  ! ALLOW FOR THE DIRECT-EVAP-REDUCING EFFECT OF SHADE
L01311  ! ----------------------------------------------------------------------
L01312  !DJG NDHMS/WRF-Hydro edits...
L01313  !       EDIR = FX * ( 1.0- SHDFAC ) * ETP1
L01314         EDIR = FX * EDIRTMP
L01315
L01316
L01317
L01318
L01319  ! ----------------------------------------------------------------------
L01320    END SUBROUTINE DEVAP_hydro
```

讲解：WRF-Hydro 积水蒸发版——设计是把地表积水头 SFHEAD1RT（mm 与 m 反复换算）先耗于积水蒸发 ETPND1、剩余再给土壤蒸发（EDIR=FX • EDIRTMP）。但 EDIRTMP 当前硬编码 0（2012/11/5 注释「currently set ponded water evap to 0.0 until further notice」），且 EVAPO 里对它的调用连 WRF_HYDRO 分支内都被注释——**WRF 4.8.0 中是死代码**。收录价值：看 WRF-Hydro 挂点长什么样（#ifdef WRF_HYDRO + INOUT 标量透传），为 ch20 的水文耦合改造提供模板。

```
!file: phys/module_sf_noahlsm.F    L1323-L1421    (subroutine EVAPO)
L01323        SUBROUTINE EVAPO (ETA1,SMC,NSOIL,CMC,ETP1,DT,ZSOIL,              &
L01324                          SH2O,                                          &
L01325                          SMCMAX,BEXP,PC,SMCWLT,DKSAT,DWSAT,             &
L01326                          SMCREF,SHDFAC,CMCMAX,                          &
L01327                          SMCDRY,CFACTR,                                 &
L01328                          EDIR,EC,ET,ETT,SFCTMP,Q2,NROOT,RTDIS,FXEXP,    &
L01329                          SFHEAD1RT,ETPND1)
L01330
L01331  ! ----------------------------------------------------------------------
L01332  ! SUBROUTINE EVAPO
L01333  ! ----------------------------------------------------------------------
L01334  ! CALCULATE SOIL MOISTURE FLUX.  THE SOIL MOISTURE CONTENT (SMC - A PER
L01335  ! UNIT VOLUME MEASUREMENT) IS A DEPENDENT VARIABLE THAT IS UPDATED WITH
L01336  ! PROGNOSTIC EQNS. THE CANOPY MOISTURE CONTENT (CMC) IS ALSO UPDATED.
L01337  ! FROZEN GROUND VERSION:  NEW STATES ADDED: SH2O, AND FROZEN GROUND
L01338  ! CORRECTION FACTOR, FRZFACT AND PARAMETER SLOPE.
L01339  ! ----------------------------------------------------------------------
L01340        IMPLICIT NONE
L01341        INTEGER, INTENT(IN)   :: NSOIL, NROOT
L01342        INTEGER               :: I,K
L01343        REAL,    INTENT(IN)   :: BEXP, CFACTR,CMC,CMCMAX,DKSAT,           &
L01344                                    DT,DWSAT,ETP1,FXEXP,PC,Q2,SFCTMP,          &
L01345                                    SHDFAC,SMCDRY,SMCMAX,SMCREF,SMCWLT
L01346        REAL,    INTENT(OUT)  :: EC,EDIR,ETA1,ETT
L01347        REAL                  :: CMC2MS
L01348        REAL,DIMENSION(1:NSOIL), INTENT(IN) :: RTDIS, SMC, SH2O, ZSOIL
L01349        REAL,DIMENSION(1:NSOIL), INTENT(OUT) :: ET
L01350
L01351        REAL,   INTENT(INOUT) :: SFHEAD1RT,ETPND1
L01352
L01353  ! ----------------------------------------------------------------------
L01354  ! EXECUTABLE CODE BEGINS HERE IF THE POTENTIAL EVAPOTRANSPIRATION IS
L01355  ! GREATER THAN ZERO.
L01356  ! ----------------------------------------------------------------------
L01357        EDIR = 0.
L01358        EC = 0.
L01359        ETT = 0.
L01360        DO K = 1,NSOIL
L01361           ET (K) = 0.
L01362        END DO
L01363
L01364  ! ----------------------------------------------------------------------
L01365  ! RETRIEVE DIRECT EVAPORATION FROM SOIL SURFACE.  CALL THIS FUNCTION
L01366  ! ONLY IF VEG COVER NOT COMPLETE.
L01367  ! FROZEN GROUND VERSION:  SH2O STATES REPLACE SMC STATES.
L01368  ! ----------------------------------------------------------------------
L01369        IF (ETP1 > 0.0) THEN
L01370           IF (SHDFAC <  1.) THEN
L01371  #ifdef WRF_HYDRO
L01372  !            CALL DEVAP_hydro (EDIR,ETP1,SMC (1),ZSOIL (1),SHDFAC,SMCMAX,      &
L01373  !                        BEXP,DKSAT,DWSAT,SMCDRY,SMCREF,SMCWLT,FXEXP,    &
L01374  !                         SFHEAD1RT,ETPND1,DT)
L01375  !DJG Reduce ETP1 by EDIR & ETPND1...
L01376  !               ETP1=ETP1-EDIR-ETPND1
L01377
L01378  ! following is the temparay setting ...
L01379               CALL DEVAP (EDIR,ETP1,SMC (1),ZSOIL (1),SHDFAC,SMCMAX,      &
L01380                           BEXP,DKSAT,DWSAT,SMCDRY,SMCREF,SMCWLT,FXEXP)
L01381  !            ETP1=ETP1-EDIR
```

```
L01382  #else
L01383               CALL DEVAP (EDIR,ETP1,SMC (1),ZSOIL (1),SHDFAC,SMCMAX,      &
L01384                           BEXP,DKSAT,DWSAT,SMCDRY,SMCREF,SMCWLT,FXEXP)
L01385  #endif
L01386            END IF
L01387  ! ---------------------------------------------------------------------
L01388  ! INITIALIZE PLANT TOTAL TRANSPIRATION, RETRIEVE PLANT TRANSPIRATION,
L01389  ! AND ACCUMULATE IT FOR ALL SOIL LAYERS.
L01390  ! ---------------------------------------------------------------------
L01391
L01392            IF (SHDFAC > 0.0) THEN
L01393               CALL TRANSP (ET,NSOIL,ETP1,SH2O,CMC,ZSOIL,SHDFAC,SMCWLT,     &
L01394                            CMCMAX,PC,CFACTR,SMCREF,SFCTMP,Q2,NROOT,RTDIS)
L01395               DO K = 1,NSOIL
L01396                  ETT = ETT + ET ( K )
L01397               END DO
L01398  ! ---------------------------------------------------------------------
L01399  ! CALCULATE CANOPY EVAPORATION.
L01400  ! IF STATEMENTS TO AVOID TANGENT LINEAR PROBLEMS NEAR CMC=0.0.
L01401  ! ---------------------------------------------------------------------
L01402               IF (CMC > 0.0) THEN
L01403                  EC = SHDFAC * ( ( CMC / CMCMAX ) ** CFACTR ) * ETP1
L01404               ELSE
L01405                  EC = 0.0
L01406               END IF
L01407  ! ---------------------------------------------------------------------
L01408  ! EC SHOULD BE LIMITED BY THE TOTAL AMOUNT OF AVAILABLE WATER ON THE
L01409  ! CANOPY.  -F.CHEN, 18-OCT-1994
L01410  ! ---------------------------------------------------------------------
L01411               CMC2MS = CMC / DT
L01412               EC = MIN ( CMC2MS, EC )
L01413            END IF
L01414         END IF
L01415  ! ---------------------------------------------------------------------
L01416  ! TOTAL UP EVAP AND TRANSP TYPES TO OBTAIN ACTUAL EVAPOTRANSP
L01417  ! ---------------------------------------------------------------------
L01418         ETA1 = EDIR + ETT + EC
L01419
L01420  ! ---------------------------------------------------------------------
L01421    END SUBROUTINE EVAPO
```

讲解：蒸发总装：清零后 ETP1>0 才工作；SHDFAC<1 调 DEVAP 得 EDIR；SHDFAC>0 调 TRANSP（12b）得逐层蒸腾 ET/ETT；冠层截留蒸发 EC=SHDFAC • (CMC/CMCMAX)^CFACTR • ETP1，且被 CMC/DT 钳位（F. Chen 1994-10-18：一个步长内不能蒸干超过冠层持水量）；ETA1=EDIR+ETT+EC。WRF_HYDRO 分支里同样只调 DEVAP——印证 DEVAP_hydro 是死代码。冻土版传给 TRANSP 的是未冻水 SH2O。

```
!file: phys/module_sf_noahlsm.F    L1424-L1444    (subroutine FAC2MIT)
L01424    SUBROUTINE FAC2MIT(SMCMAX,FLIMIT)
L01425      IMPLICIT NONE
L01426      REAL, INTENT(IN)  :: SMCMAX
L01427      REAL, INTENT(OUT) :: FLIMIT
L01428
L01429      FLIMIT = 0.90
L01430
L01431      IF ( SMCMAX == 0.395 ) THEN
L01432         FLIMIT = 0.59
L01433      ELSE IF ( ( SMCMAX == 0.434 ) .OR. ( SMCMAX == 0.404 ) ) THEN
L01434         FLIMIT = 0.85
L01435      ELSE IF ( ( SMCMAX == 0.465 ) .OR. ( SMCMAX == 0.406 ) ) THEN
L01436         FLIMIT = 0.86
L01437      ELSE IF ( ( SMCMAX == 0.476 ) .OR. ( SMCMAX == 0.439 ) ) THEN
L01438         FLIMIT = 0.74
L01439      ELSE IF ( ( SMCMAX == 0.200 ) .OR. ( SMCMAX == 0.464 ) ) THEN
L01440         FLIMIT = 0.80
L01441      ENDIF
L01442
L01443  ! ---------------------------------------------------------------------
L01444    END SUBROUTINE FAC2MIT
```

讲解：按孔隙度 SMCMAX 查入渗限制系数表：默认 FLIMIT=0.90，特定土壤（0.395→0.59 等）单独给值——Mitchell 的建议：FAC2（各层 SH2O/SMCMAX 最大值）超过 FLIMIT 时土壤已近饱和，SMFLX 里强制走双步长时间方案抑制径流与土壤湿度的数值不稳定（见 12b § 六）。

七、冻土液态水 FRH2O

```
!file: phys/module_sf_noahlsm.F    L1447-L1585    (subroutine FRH2O)
L01447          SUBROUTINE FRH2O (FREE,TKELV,SMC,SH2O,SMCMAX,BEXP,PSIS)
L01448
L01449  ! ---------------------------------------------------------------------
L01450  ! SUBROUTINE FRH2O
L01451  ! ---------------------------------------------------------------------
L01452  ! CALCULATE AMOUNT OF SUPERCOOLED LIQUID SOIL WATER CONTENT IF
L01453  ! TEMPERATURE IS BELOW 273.15K (T0).  REQUIRES NEWTON-TYPE ITERATION TO
L01454  ! SOLVE THE NONLINEAR IMPLICIT EQUATION GIVEN IN EQN 17 OF KOREN ET AL
L01455  ! (1999, JGR, VOL 104(D16), 19569-19585).
L01456  ! ---------------------------------------------------------------------
L01457  ! NEW VERSION (JUNE 2001): MUCH FASTER AND MORE ACCURATE NEWTON
L01458  ! ITERATION ACHIEVED BY FIRST TAKING LOG OF EQN CITED ABOVE -- LESS THAN
L01459  ! 4 (TYPICALLY 1 OR 2) ITERATIONS ACHIEVES CONVERGENCE.  ALSO, EXPLICIT
L01460  ! 1-STEP SOLUTION OPTION FOR SPECIAL CASE OF PARAMETER CK=0, WHICH
L01461  ! REDUCES THE ORIGINAL IMPLICIT EQUATION TO A SIMPLER EXPLICIT FORM,
L01462  ! KNOWN AS THE "FLERCHINGER EQN". IMPROVED HANDLING OF SOLUTION IN THE
L01463  ! LIMIT OF FREEZING POINT TEMPERATURE T0.
L01464  ! ---------------------------------------------------------------------
```

```
L01465  ! INPUT:
L01466
L01467  !   TKELV.........TEMPERATURE (Kelvin)
L01468  !   SMC...........TOTAL SOIL MOISTURE CONTENT (VOLUMETRIC)
L01469  !   SH2O..........LIQUID SOIL MOISTURE CONTENT (VOLUMETRIC)
L01470  !   SMCMAX........SATURATION SOIL MOISTURE CONTENT (FROM REDPRM)
L01471  !   B.............SOIL TYPE "B" PARAMETER (FROM REDPRM)
L01472  !   PSIS..........SATURATED SOIL MATRIC POTENTIAL (FROM REDPRM)
L01473
L01474  ! OUTPUT:
L01475  !   FRH2O.........SUPERCOOLED LIQUID WATER CONTENT
L01476  !   FREE..........SUPERCOOLED LIQUID WATER CONTENT
L01477  ! ----------------------------------------------------------------------
L01478        IMPLICIT NONE
L01479        REAL, INTENT(IN)      :: BEXP,PSIS,SH2O,SMC,SMCMAX,TKELV
L01480        REAL, INTENT(OUT)     :: FREE
L01481        REAL                  :: BX,DENOM,DF,DSWL,FK,SWL,SWLK
L01482        INTEGER               :: NLOG,KCOUNT
L01483  !      PARAMETER(CK = 0.0)
L01484        REAL, PARAMETER       :: CK = 8.0, BLIM = 5.5, ERROR = 0.005,       &
L01485                                 HLICE = 3.335E5, GS = 9.81,DICE = 920.0,   &
L01486                                 DH2O = 1000.0, T0 = 273.15
L01487
L01488  ! ----------------------------------------------------------------------
L01489  ! LIMITS ON PARAMETER B: B < 5.5  (use parameter BLIM)
L01490  ! SIMULATIONS SHOWED IF B > 5.5 UNFROZEN WATER CONTENT IS
L01491  ! NON-REALISTICALLY HIGH AT VERY LOW TEMPERATURES.
L01492  ! ----------------------------------------------------------------------
L01493        BX = BEXP
L01494
L01495  ! ----------------------------------------------------------------------
L01496  ! INITIALIZING ITERATIONS COUNTER AND ITERATIVE SOLUTION FLAG.
L01497  ! ----------------------------------------------------------------------
L01498        IF (BEXP >  BLIM) BX = BLIM
L01499        NLOG = 0
L01500
L01501  ! ----------------------------------------------------------------------
L01502  !  IF TEMPERATURE NOT SIGNIFICANTLY BELOW FREEZING (T0), SH2O = SMC
L01503  ! ----------------------------------------------------------------------
L01504        KCOUNT = 0
L01505  !      FRH2O = SMC
L01506        IF (TKELV > (T0- 1.E-3)) THEN
L01507          FREE = SMC
L01508        ELSE
L01509
L01510  ! ----------------------------------------------------------------------
L01511  ! OPTION 1: ITERATED SOLUTION FOR NONZERO CK
L01512  ! IN KOREN ET AL, JGR, 1999, EQN 17
L01513  ! ----------------------------------------------------------------------
L01514  ! INITIAL GUESS FOR SWL (frozen content)
L01515  ! ----------------------------------------------------------------------
L01516           IF (CK /= 0.0) THEN
L01517              SWL = SMC - SH2O
L01518  ! ----------------------------------------------------------------------
L01519  ! KEEP WITHIN BOUNDS.
L01520  ! ----------------------------------------------------------------------
L01521              IF (SWL > (SMC -0.02)) SWL = SMC -0.02
L01522
L01523  ! ----------------------------------------------------------------------
L01524  !  START OF ITERATIONS
L01525  ! ----------------------------------------------------------------------
L01526              IF (SWL < 0.) SWL = 0.
L01527   1001       Continue
L01528                IF (.NOT.( (NLOG < 10) .AND. (KCOUNT == 0)))   goto 1002
L01529                NLOG = NLOG +1
L01530                DF = ALOG ( ( PSIS * GS / HLICE ) * ( ( 1. + CK * SWL )**2.) * &
L01531                     ( SMCMAX / (SMC - SWL) )** BX) - ALOG ( - (               &
L01532                     TKELV - T0)/ TKELV)
L01533                DENOM = 2. * CK / ( 1. + CK * SWL ) + BX / ( SMC - SWL )
L01534                SWLK = SWL - DF / DENOM
L01535  ! ----------------------------------------------------------------------
L01536  ! BOUNDS USEFUL FOR MATHEMATICAL SOLUTION.
L01537  ! ----------------------------------------------------------------------
L01538                IF (SWLK > (SMC -0.02)) SWLK = SMC - 0.02
L01539                IF (SWLK < 0.) SWLK = 0.
L01540
L01541  ! ----------------------------------------------------------------------
L01542  ! MATHEMATICAL SOLUTION BOUNDS APPLIED.
L01543  ! ----------------------------------------------------------------------
L01544                DSWL = ABS (SWLK - SWL)
L01545
L01546  ! ----------------------------------------------------------------------
L01547  ! IF MORE THAN 10 ITERATIONS, USE EXPLICIT METHOD (CK=0 APPROX.)
L01548  ! WHEN DSWL LESS OR EQ. ERROR, NO MORE ITERATIONS REQUIRED.
L01549  ! ----------------------------------------------------------------------
L01550                SWL = SWLK
L01551                IF ( DSWL <= ERROR ) THEN
L01552                      KCOUNT = KCOUNT +1
L01553                END IF
L01554  ! ----------------------------------------------------------------------
L01555  !  END OF ITERATIONS
L01556  ! ----------------------------------------------------------------------
L01557  ! BOUNDS APPLIED WITHIN DO-BLOCK ARE VALID FOR PHYSICAL SOLUTION.
L01558  ! ----------------------------------------------------------------------
L01559  !          FRH2O = SMC - SWL
L01560              goto 1001
L01561   1002    continue
L01562              FREE = SMC - SWL
L01563           END IF
L01564  ! ----------------------------------------------------------------------
L01565  ! END OPTION 1
L01566  ! ----------------------------------------------------------------------
L01567  ! ----------------------------------------------------------------------
L01568  ! OPTION 2: EXPLICIT SOLUTION FOR FLERCHINGER EQ. i.e. CK=0
L01569  ! IN KOREN ET AL., JGR, 1999, EQN 17
L01570  ! APPLY PHYSICAL BOUNDS TO FLERCHINGER SOLUTION
L01571  ! ----------------------------------------------------------------------
L01572           IF (KCOUNT == 0) THEN
L01573               PRINT *,'Flerchinger USEd in NEW version. Iterations=',NLOG
L01574                    FK = ( ( (HLICE / (GS * ( - PSIS)))*                  &
```

```
L01575                        ( (TKELV - T0)/ TKELV))** ( -1/ BX))* SMCMAX
L01576  !            FRH2O = MIN (FK, SMC)
L01577               IF (FK < 0.02) FK = 0.02
L01578               FREE = MIN (FK, SMC)
L01579  ! ----------------------------------------------------------------------
L01580  ! END OPTION 2
L01581  ! ----------------------------------------------------------------------
L01582            END IF
L01583         END IF
L01584  ! ----------------------------------------------------------------------
L01585    END SUBROUTINE FRH2O
```

讲解：冻土液态水含量（超冷液态水）：T>T0−1e-3 时 FREE=SMC（未冻）；否则解 Koren et al. 1999（JGR 104, 19569）**式 17 的隐式方程**——2001-06 新版先取对数再牛顿迭代（典型 1–2 次收敛，旧版直接迭代慢且不准）；CK=8（CK=0 时方程退化为 Flerchinger 显式式，作为迭代 10 次不收敛的回退并打印提示）；BEXP>5.5 钳位到 BLIM（B 太大时极低温下未冻水虚高——模拟经验）；迭代变量 SWL（含冰量）初值 SMC−SH2O、钳位 [0, SMC−0.02]；收敛容限 ERROR=0.005。函数本体被 HRT 的 SNKSRC 调用（12b § 八），是冻土实验三件套之一。

八、土壤热传导右端 HRT 与推进 HSTEP

```
!file: phys/module_sf_noahlsm.F    L1588-L1850    (subroutine HRT)
L01588         SUBROUTINE HRT (RHSTS,STC,SMC,SMCMAX,NSOIL,ZSOIL,YY,ZZ1,          &
L01589                         TBOT,ZBOT,PSISAT,SH2O,DT,BEXP,SOILTYP,OPT_THCND, &
L01590                         F1,DF1,QUARTZ,CSOIL,AI,BI,CI,VEGTYP,ISURBAN      &
L01591                        ,HCPCT_FASDAS                                     ) !fasdas
L01592
L01593  ! ----------------------------------------------------------------------
L01594  ! SUBROUTINE HRT
L01595  ! ----------------------------------------------------------------------
L01596  ! CALCULATE THE RIGHT HAND SIDE OF THE TIME TENDENCY TERM OF THE SOIL
L01597  ! THERMAL DIFFUSION EQUATION.  ALSO TO COMPUTE ( PREPARE ) THE MATRIX
L01598  ! COEFFICIENTS FOR THE TRI-DIAGONAL MATRIX OF THE IMPLICIT TIME SCHEME.
L01599  ! ----------------------------------------------------------------------
L01600         IMPLICIT NONE
L01601         LOGICAL               :: ITAVG
L01602         INTEGER, INTENT(IN)   :: OPT_THCND
L01603         INTEGER, INTENT(IN)   :: NSOIL, VEGTYP, SOILTYP
L01604         INTEGER, INTENT(IN)   :: ISURBAN
L01605         INTEGER               :: I, K
L01606
L01607         REAL, INTENT(IN)      :: BEXP, CSOIL, DF1, DT,F1,PSISAT,QUARTZ,     &
L01608                                  SMCMAX ,TBOT,YY,ZZ1, ZBOT
L01609         REAL, DIMENSION(1:NSOIL), INTENT(IN)   :: SMC,STC,ZSOIL
L01610         REAL, DIMENSION(1:NSOIL), INTENT(INOUT):: SH2O
L01611         REAL, DIMENSION(1:NSOIL), INTENT(OUT)  :: RHSTS
L01612         REAL, DIMENSION(1:NSOIL), INTENT(OUT)  :: AI, BI,CI
L01613         REAL                  :: DDZ, DDZ2, DENOM, DF1N, DF1K, DTSDZ,       &
L01614                                  DTSDZ2,HCPCT,QTOT,SSOIL,SICE,TAVG,TBK,     &
L01615                                  TBK1,TSNSR,TSURF,CSOIL_LOC
L01616         REAL, PARAMETER       :: T0 = 273.15, CAIR = 1004.0, CICE = 2.106E6,&
L01617                                  CH2O = 4.2E6
L01618
L01619  !
L01620  ! FASDAS
L01621  !
L01622         REAL, INTENT(  OUT)       :: HCPCT_FASDAS
L01623  !
L01624  ! END FASDAS
L01625  !
L01626
L01627  !urban
L01628          IF( VEGTYP == ISURBAN ) then
L01629              CSOIL_LOC=3.0E6
L01630          ELSE
L01631              CSOIL_LOC=CSOIL
L01632          ENDIF
L01633
L01634  ! ----------------------------------------------------------------------
L01635  ! INITIALIZE LOGICAL FOR SOIL LAYER TEMPERATURE AVERAGING.
L01636  ! ----------------------------------------------------------------------
L01637          ITAVG = .TRUE.
L01638  ! ----------------------------------------------------------------------
L01639  ! BEGIN SECTION FOR TOP SOIL LAYER
L01640  ! ----------------------------------------------------------------------
L01641  ! CALC THE HEAT CAPACITY OF THE TOP SOIL LAYER
L01642  ! ----------------------------------------------------------------------
L01643         HCPCT = SH2O (1)* CH2O + (1.0- SMCMAX)* CSOIL_LOC + (SMCMAX - SMC (1))&
L01644          * CAIR                                                            &
L01645                + ( SMC (1) - SH2O (1) )* CICE
L01646  !
L01647  ! FASDAS
L01648  !
L01649         HCPCT_FASDAS = HCPCT
L01650  !
L01651  ! END FASDAS
L01652  !
L01653  ! ----------------------------------------------------------------------
L01654  ! CALC THE MATRIX COEFFICIENTS AI, BI, AND CI FOR THE TOP LAYER
L01655  ! ----------------------------------------------------------------------
L01656         DDZ = 1.0 / ( -0.5 * ZSOIL (2) )
L01657         AI (1) = 0.0
L01658         CI (1) = (DF1 * DDZ) / (ZSOIL (1) * HCPCT)
L01659
L01660  ! ----------------------------------------------------------------------
L01661  ! CALCULATE THE VERTICAL SOIL TEMP GRADIENT BTWN THE 1ST AND 2ND SOIL
L01662  ! LAYERS.  THEN CALCULATE THE SUBSURFACE HEAT FLUX. USE THE TEMP
L01663  ! GRADIENT AND SUBSFC HEAT FLUX TO CALC "RIGHT-HAND SIDE TENDENCY
L01664  ! TERMS", OR "RHSTS", FOR TOP SOIL LAYER.
```

```
! ------------------------------------------------------------------
      BI (1) = - CI (1) + DF1 / (0.5 * ZSOIL (1) * ZSOIL (1)* HCPCT *    &
       ZZ1)
      DTSDZ = (STC (1) - STC (2)) / ( -0.5 * ZSOIL (2))
      SSOIL = DF1 * (STC (1) - YY) / (0.5 * ZSOIL (1) * ZZ1)
!      RHSTS(1) = (DF1 * DTSDZ - SSOIL) / (ZSOIL(1) * HCPCT)
      DENOM = (ZSOIL (1) * HCPCT)

! ------------------------------------------------------------------
! NEXT CAPTURE THE VERTICAL DIFFERENCE OF THE HEAT FLUX AT TOP AND
! BOTTOM OF FIRST SOIL LAYER FOR USE IN HEAT FLUX CONSTRAINT APPLIED TO
! POTENTIAL SOIL FREEZING/THAWING IN ROUTINE SNKSRC.
! ------------------------------------------------------------------
!      QTOT = SSOIL - DF1*DTSDZ
      RHSTS (1) = (DF1 * DTSDZ - SSOIL) / DENOM

! ------------------------------------------------------------------
! CALCULATE FROZEN WATER CONTENT IN 1ST SOIL LAYER.
! ------------------------------------------------------------------
      QTOT = -1.0* RHSTS (1)* DENOM

! ------------------------------------------------------------------
! IF TEMPERATURE AVERAGING INVOKED (ITAVG=TRUE; ELSE SKIP):
! SET TEMP "TSURF" AT TOP OF SOIL COLUMN (FOR USE IN FREEZING SOIL
! PHYSICS LATER IN FUNCTION SUBROUTINE SNKSRC).  IF SNOWPACK CONTENT IS
! ZERO, THEN TSURF EXPRESSION BELOW GIVES TSURF = SKIN TEMP.  IF
! SNOWPACK IS NONZERO (HENCE ARGUMENT ZZ1=1), THEN TSURF EXPRESSION
! BELOW YIELDS SOIL COLUMN TOP TEMPERATURE UNDER SNOWPACK.  THEN
! CALCULATE TEMPERATURE AT BOTTOM INTERFACE OF 1ST SOIL LAYER FOR USE
! LATER IN FUNCTION SUBROUTINE SNKSRC
! ------------------------------------------------------------------
      SICE = SMC (1) - SH2O (1)
      IF (ITAVG) THEN
         TSURF = (YY + (ZZ1-1) * STC (1)) / ZZ1
! ------------------------------------------------------------------
! IF FROZEN WATER PRESENT OR ANY OF LAYER-1 MID-POINT OR BOUNDING
! INTERFACE TEMPERATURES BELOW FREEZING, THEN CALL SNKSRC TO
! COMPUTE HEAT SOURCE/SINK (AND CHANGE IN FROZEN WATER CONTENT)
! DUE TO POSSIBLE SOIL WATER PHASE CHANGE
! ------------------------------------------------------------------
        CALL TBND (STC (1),STC (2),ZSOIL,ZBOT,1,NSOIL,TBK)
        IF ( (SICE > 0.) .OR. (STC (1) < T0) .OR.                        &
           (TSURF < T0) .OR. (TBK < T0) ) THEN
!         TSNSR = SNKSRC (TAVG,SMC(1),SH2O(1),
          CALL TMPAVG (TAVG,TSURF,STC (1),TBK,ZSOIL,NSOIL,1)
          CALL SNKSRC (TSNSR,TAVG,SMC (1),SH2O (1),                      &
                       ZSOIL,NSOIL,SMCMAX,PSISAT,BEXP,DT,1,QTOT)
!         RHSTS(1) = RHSTS(1) - TSNSR / ( ZSOIL(1) * HCPCT )
          RHSTS (1) = RHSTS (1) - TSNSR / DENOM
        END IF
      ELSE
!         TSNSR = SNKSRC (STC(1),SMC(1),SH2O(1),
        IF ( (SICE > 0.) .OR. (STC (1) < T0) ) THEN
          CALL SNKSRC (TSNSR,STC (1),SMC (1),SH2O (1),                   &
                       ZSOIL,NSOIL,SMCMAX,PSISAT,BEXP,DT,1,QTOT)
!         RHSTS(1) = RHSTS(1) - TSNSR / ( ZSOIL(1) * HCPCT )
          RHSTS (1) = RHSTS (1) - TSNSR / DENOM
        END IF
! ------------------------------------------------------------------
! THIS ENDS SECTION FOR TOP SOIL LAYER.
! ------------------------------------------------------------------
      END IF

! INITIALIZE DDZ2
! ------------------------------------------------------------------

      DDZ2 = 0.0
      DF1K = DF1

! ------------------------------------------------------------------
! LOOP THRU THE REMAINING SOIL LAYERS, REPEATING THE ABOVE PROCESS
! (EXCEPT SUBSFC OR "GROUND" HEAT FLUX NOT REPEATED IN LOWER LAYERS)
! ------------------------------------------------------------------
! CALCULATE HEAT CAPACITY FOR THIS SOIL LAYER.
! ------------------------------------------------------------------
      DO K = 2,NSOIL
         HCPCT = SH2O (K)* CH2O + (1.0- SMCMAX)* CSOIL_LOC + (SMCMAX - SMC (  &
                 K))* CAIR + ( SMC (K) - SH2O (K) )* CICE
! ------------------------------------------------------------------
! THIS SECTION FOR LAYER 2 OR GREATER, BUT NOT LAST LAYER.
! ------------------------------------------------------------------
! CALCULATE THERMAL DIFFUSIVITY FOR THIS LAYER.
! ------------------------------------------------------------------
        IF (K /= NSOIL) THEN

! ------------------------------------------------------------------
! CALC THE VERTICAL SOIL TEMP GRADIENT THRU THIS LAYER
! ------------------------------------------------------------------
          CALL TDFCND (DF1N,SMC (K),QUARTZ,SMCMAX,SH2O (K),BEXP, PSISAT, SOILTYP, OPT_THCND)

!urban
       IF ( VEGTYP == ISURBAN ) DF1N = 3.24

          DENOM = 0.5 * ( ZSOIL (K -1) - ZSOIL (K +1) )

! ------------------------------------------------------------------
! CALC THE MATRIX COEF, CI, AFTER CALC'NG ITS PARTIAL PRODUCT
! ------------------------------------------------------------------
          DTSDZ2 = ( STC (K) - STC (K +1) ) / DENOM
          DDZ2 = 2. / (ZSOIL (K -1) - ZSOIL (K +1))

! ------------------------------------------------------------------
! IF TEMPERATURE AVERAGING INVOKED (ITAVG=TRUE; ELSE SKIP):  CALCULATE
! TEMP AT BOTTOM OF LAYER.
! ------------------------------------------------------------------
          CI (K) = - DF1N * DDZ2 / ( (ZSOIL (K -1) - ZSOIL (K)) *      &
       HCPCT)
          IF (ITAVG) THEN
            CALL TBND (STC (K),STC (K +1),ZSOIL,ZBOT,K,NSOIL,TBK1)
```

```
L01774             END IF
L01775
L01776           ELSE
L01777  ! ----------------------------------------------------------------------
L01778  ! SPECIAL CASE OF BOTTOM SOIL LAYER:  CALCULATE THERMAL DIFFUSIVITY FOR
L01779  ! BOTTOM LAYER.
L01780  ! ----------------------------------------------------------------------
L01781
L01782  ! ----------------------------------------------------------------------
L01783  ! CALC THE VERTICAL SOIL TEMP GRADIENT THRU BOTTOM LAYER.
L01784  ! ----------------------------------------------------------------------
L01785             CALL TDFCND (DF1N,SMC (K),QUARTZ,SMCMAX,SH2O (K),BEXP, PSISAT, SOILTYP, OPT_THCND)
L01786
L01787
L01788  !urban
L01789           IF ( VEGTYP == ISURBAN ) DF1N = 3.24
L01790
L01791             DENOM = .5 * (ZSOIL (K -1) + ZSOIL (K)) - ZBOT
L01792
L01793  ! ----------------------------------------------------------------------
L01794  ! SET MATRIX COEF, CI TO ZERO IF BOTTOM LAYER.
L01795  ! ----------------------------------------------------------------------
L01796             DTSDZ2 = (STC (K) - TBOT) / DENOM
L01797
L01798  ! ----------------------------------------------------------------------
L01799  ! IF TEMPERATURE AVERAGING INVOKED (ITAVG=TRUE; ELSE SKIP):  CALCULATE
L01800  ! TEMP AT BOTTOM OF LAST LAYER.
L01801  ! ----------------------------------------------------------------------
L01802             CI (K) = 0.
L01803             IF (ITAVG) THEN
L01804                CALL TBND (STC (K),TBOT,ZSOIL,ZBOT,K,NSOIL,TBK1)
L01805             END IF
L01806  ! ----------------------------------------------------------------------
L01807  ! THIS ENDS SPECIAL LOOP FOR BOTTOM LAYER.
L01808           END IF
L01809  ! ----------------------------------------------------------------------
L01810  ! CALCULATE RHSTS FOR THIS LAYER AFTER CALC'NG A PARTIAL PRODUCT.
L01811  ! ----------------------------------------------------------------------
L01812           DENOM = ( ZSOIL (K) - ZSOIL (K -1) ) * HCPCT
L01813           RHSTS (K) = ( DF1N * DTSDZ2- DF1K * DTSDZ ) / DENOM
L01814           QTOT = -1.0* DENOM * RHSTS (K)
L01815
L01816           SICE = SMC (K) - SH2O (K)
L01817           IF (ITAVG) THEN
L01818              CALL TMPAVG (TAVG,TBK,STC (K),TBK1,ZSOIL,NSOIL,K)
L01819  !            TSNSR = SNKSRC(TAVG,SMC(K),SH2O(K),ZSOIL,NSOIL,
L01820              IF ( (SICE > 0.) .OR. (STC (K) < T0) .OR.                    &
L01821                 (TBK .lt. T0) .OR. (TBK1 .lt. T0) ) THEN
L01822                 CALL SNKSRC (TSNSR,TAVG,SMC (K),SH2O (K),ZSOIL,NSOIL,    &
L01823                              SMCMAX,PSISAT,BEXP,DT,K,QTOT)
L01824                 RHSTS (K) = RHSTS (K) - TSNSR / DENOM
L01825              END IF
L01826           ELSE
L01827  !            TSNSR = SNKSRC(STC(K),SMC(K),SH2O(K),ZSOIL,NSOIL,
L01828              IF ( (SICE > 0.) .OR. (STC (K) < T0) ) THEN
L01829                 CALL SNKSRC (TSNSR,STC (K),SMC (K),SH2O (K),ZSOIL,NSOIL, &
L01830                              SMCMAX,PSISAT,BEXP,DT,K,QTOT)
L01831                 RHSTS (K) = RHSTS (K) - TSNSR / DENOM
L01832              END IF
L01833           END IF
L01834
L01835  ! ----------------------------------------------------------------------
L01836  ! CALC MATRIX COEFS, AI, AND BI FOR THIS LAYER.
L01837  ! ----------------------------------------------------------------------
L01838           AI (K) = - DF1K * DDZ / ( (ZSOIL (K -1) - ZSOIL (K)) * HCPCT)
L01839
L01840  ! ----------------------------------------------------------------------
L01841  ! RESET VALUES OF DF1, DTSDZ, DDZ, AND TBK FOR LOOP TO NEXT SOIL LAYER.
L01842  ! ----------------------------------------------------------------------
L01843           BI (K) = - (AI (K) + CI (K))
L01844           TBK = TBK1
L01845           DF1K = DF1N
L01846           DTSDZ = DTSDZ2
L01847           DDZ = DDZ2
L01848         END DO
L01849  ! ----------------------------------------------------------------------
L01850     END SUBROUTINE HRT
```

讲解：土壤热传导方程的空间离散：① 逐层体积热容四组分 HCPCT=SH2O • CH2O+(1−SMCMAX) • CSOIL_LOC+(SMCMAX−SMC) • CAIR+(SMC−SH2O) • CICE（水/土壤骨架/空气/冰）；urban 格点 CSOIL_LOC=3.0E6 硬覆盖 GENPARM 给的 CSOIL——城市土壤热容改造点。② 顶层 AI=0，CI/BI 里出现 ZZ1——皮肤温度耦合系数（无雪时由 NOPAC 给出、雪下取 1），SSOIL=DF1 • (STC1−YY)/(0.5 • ZSOIL1 • ZZ1)；RHSTS(1)=(DF1 • DTSDZ−SSOIL)/DENOM 是「下层通量 − 地表通量」的散度。③ 中间层每层重调 TDFCND 算 DF1N（urban 同样硬 3.24）；底层与 TBOT（年平均地温）定梯度。④ITAVG=.TRUE. 的冻土路径：TBND（12b）算层界面温度、TMPAVG（12b）算冻融层平均温度（0°C 等温面分区加权），SNKSRC（12b）以该层净热通量 QTOT=−DENOM • RHSTS(K) 为约束做相变钳位，潜热源项从 RHSTS 中扣除——「热通量先冻结水、剩余才降温」的能量守恒细节全在这条链上。

```
!file: phys/module_sf_noahlsm.F    L1853-L1901    (subroutine HSTEP)
L01853        SUBROUTINE HSTEP (STCOUT,STCIN,RHSTS,DT,NSOIL,AI,BI,CI)
L01854
L01855  ! ----------------------------------------------------------------------
L01856  ! SUBROUTINE HSTEP
L01857  ! ----------------------------------------------------------------------
L01858  ! CALCULATE/UPDATE THE SOIL TEMPERATURE FIELD.
L01859  ! ----------------------------------------------------------------------
L01860        IMPLICIT NONE
L01861        INTEGER, INTENT(IN)  :: NSOIL
```

```
L01862        INTEGER             :: K
L01863
L01864        REAL, DIMENSION(1:NSOIL), INTENT(IN):: STCIN
L01865        REAL, DIMENSION(1:NSOIL), INTENT(OUT):: STCOUT
L01866        REAL, DIMENSION(1:NSOIL), INTENT(INOUT):: RHSTS
L01867        REAL, DIMENSION(1:NSOIL), INTENT(INOUT):: AI,BI,CI
L01868        REAL, DIMENSION(1:NSOIL) :: RHSTSin
L01869        REAL, DIMENSION(1:NSOIL) :: CIin
L01870        REAL                  :: DT
L01871
L01872  ! ----------------------------------------------------------------------
L01873  ! CREATE FINITE DIFFERENCE VALUES FOR USE IN ROSR12 ROUTINE
L01874  ! ----------------------------------------------------------------------
L01875        DO K = 1,NSOIL
L01876           RHSTS (K) = RHSTS (K) * DT
L01877           AI (K) = AI (K) * DT
L01878           BI (K) = 1. + BI (K) * DT
L01879           CI (K) = CI (K) * DT
L01880        END DO
L01881  ! ----------------------------------------------------------------------
L01882  ! COPY VALUES FOR INPUT VARIABLES BEFORE CALL TO ROSR12
L01883  ! ----------------------------------------------------------------------
L01884        DO K = 1,NSOIL
L01885           RHSTSin (K) = RHSTS (K)
L01886        END DO
L01887        DO K = 1,NSOIL
L01888           CIin (K) = CI (K)
L01889        END DO
L01890  ! ----------------------------------------------------------------------
L01891  ! SOLVE THE TRI-DIAGONAL MATRIX EQUATION
L01892  ! ----------------------------------------------------------------------
L01893        CALL ROSR12 (CI,AI,BI,CIin,RHSTSin,RHSTS,NSOIL)
L01894  ! ----------------------------------------------------------------------
L01895  ! CALC/UPDATE THE SOIL TEMPS USING MATRIX SOLUTION
L01896  ! ----------------------------------------------------------------------
L01897        DO K = 1,NSOIL
L01898           STCOUT (K) = STCIN (K) + CI (K)
L01899        END DO
L01900  ! ----------------------------------------------------------------------
L01901    END SUBROUTINE HSTEP
```

讲解：热推进：RHSTS • DT、AI • DT、BI 改 1+BI • DT、CI • DT 把右端化成「量」，备份 RHSTTin/CIin 后调 ROSR12（12b 三对角追赶法），STCOUT=STCIN+解。与 SSTEP（12b）是同构姐妹函数——水与热各一份隐式推进，读其一可通其二。

九、无雪路径 NOPAC

```
!file: phys/module_sf_noahlsm.F    L1904-L2192    (subroutine NOPAC)
L01904        SUBROUTINE NOPAC (ETP,ETA,PRCP,SMC,SMCMAX,SMCWLT,                &
L01905                          SMCREF,SMCDRY,CMC,CMCMAX,NSOIL,DT,SHDFAC,      &
L01906                          SBETA,Q2,T1,SFCTMP,T24,TH2,FDOWN,F1,EMISSI,    &
L01907                          SSOIL,                                         &
L01908                          STC,EPSCA,BEXP,PC,RCH,RR,CFACTR,               &
L01909                          SH2O,SLOPE,KDT,FRZFACT,PSISAT,ZSOIL,           &
L01910                          DKSAT,DWSAT,TBOT,ZBOT,RUNOFF1,RUNOFF2,         &
L01911                          RUNOFF3,EDIR,EC,ET,ETT,NROOT,RTDIS,            &
L01912                          QUARTZ,FXEXP,CSOIL,                            &
L01913                          BETA,DRIP,DEW,FLX1,FLX3,VEGTYP,ISURBAN,        &
L01914                          SFHEAD1RT,INFXS1RT,ETPND1,SOILTYP,OPT_THCND    &
L01915                         ,XSDA_QFX,QFX_PHY,XQNORM,fasdas,HCPCT_FASDAS    &
L01916                         ,IRRIGATION_CHANNEL    ) !fasdas
L01917
L01918  ! ----------------------------------------------------------------------
L01919  ! SUBROUTINE NOPAC
L01920  ! ----------------------------------------------------------------------
L01921  ! CALCULATE SOIL MOISTURE AND HEAT FLUX VALUES AND UPDATE SOIL MOISTURE
L01922  ! CONTENT AND SOIL HEAT CONTENT VALUES FOR THE CASE WHEN NO SNOW PACK IS
L01923  ! PRESENT.
L01924  ! ----------------------------------------------------------------------
L01925        IMPLICIT NONE
L01926
L01927        INTEGER, INTENT(IN)  :: OPT_THCND
L01928        INTEGER, INTENT(IN)  :: NROOT,NSOIL,VEGTYP,SOILTYP
L01929        INTEGER, INTENT(IN)  :: ISURBAN
L01930        INTEGER              :: K
L01931
L01932        REAL, INTENT(IN)     :: BEXP,CFACTR, CMCMAX,CSOIL,DKSAT,DT,DWSAT, &
L01933                                EPSCA,ETP,FDOWN,F1,FXEXP,FRZFACT,KDT,PC,  &
L01934                                PRCP,PSISAT,Q2,QUARTZ,RCH,RR,SBETA,SFCTMP,&
L01935                                SHDFAC,SLOPE,SMCDRY,SMCMAX,SMCREF,SMCWLT, &
L01936                                T24,TBOT,TH2,ZBOT,EMISSI
L01937        REAL, INTENT(INOUT)  :: CMC,BETA,T1
L01938        REAL, INTENT(OUT)    :: DEW,DRIP,EC,EDIR,ETA,ETT,FLX1,FLX3,       &
L01939                                RUNOFF1,RUNOFF2,RUNOFF3,SSOIL
L01940  !DJG NDHMS/WRF-Hydro edit...
L01941        REAL, INTENT(INOUT)  :: SFHEAD1RT,INFXS1RT,ETPND1
L01942
L01943        REAL, DIMENSION(1:NSOIL),INTENT(IN)     :: RTDIS,ZSOIL
L01944        REAL, DIMENSION(1:NSOIL),INTENT(OUT)    :: ET
L01945        REAL, DIMENSION(1:NSOIL), INTENT(INOUT) :: SMC,SH2O,STC
L01946        REAL, DIMENSION(1:NSOIL) :: ET1
L01947        REAL                  :: EC1,EDIR1,ETT1,DF1,ETA1,ETP1,PRCP1,YY,    &
L01948                                 YYNUM,ZZ1
L01949  !
L01950  ! FASDAS
L01951  !
L01952        REAL                       ::  XSDA_QFX, QFX_PHY, XQNORM
L01953        INTEGER                    ::  fasdas
L01954        REAL , DIMENSION(1:NSOIL)  ::  EFT(NSOIL), wetty(1:NSOIL)
L01955        REAL                       ::  EFDIR, EFC, EALL_now
L01956        REAL, INTENT(  OUT)        ::  HCPCT_FASDAS
```

```
L01957        REAL, INTENT(IN),OPTIONAL  ::  IRRIGATION_CHANNEL
L01958  !
L01959  ! END FASDAS
L01960  !
L01961  ! ----------------------------------------------------------------------
L01962  ! EXECUTABLE CODE BEGINS HERE:
L01963  ! CONVERT ETP Fnd PRCP FROM KG M-2 S-1 TO M S-1 AND INITIALIZE DEW.
L01964  ! ----------------------------------------------------------------------
L01965        PRCP1 = PRCP * 0.001
L01966        ETP1 = ETP * 0.001
L01967        DEW = 0.0
L01968  ! ----------------------------------------------------------------------
L01969  ! INITIALIZE EVAP TERMS.
L01970  ! ----------------------------------------------------------------------
L01971  !
L01972  ! FASDAS
L01973  !
L01974        QFX_PHY = 0.0
L01975  !
L01976  ! END FASDAS
L01977  !
L01978        EDIR = 0.
L01979        EDIR1 = 0.
L01980        EC1 = 0.
L01981        EC = 0.
L01982        DO K = 1,NSOIL
L01983          ET(K) = 0.
L01984          ET1(K) = 0.
L01985  !
L01986  ! FASDAS
L01987  !
L01988          wetty(K) = 1.0
L01989  !
L01990  ! END FASDAS
L01991  !
L01992        END DO
L01993        ETT = 0.
L01994        ETT1 = 0.
L01995
L01996  !DJG NDHMS/WRF-Hydro edit...
L01997        ETPND1 = 0.
L01998
L01999
L02000        IF (ETP > 0.0) THEN
L02001           CALL EVAPO (ETA1,SMC,NSOIL,CMC,ETP1,DT,ZSOIL,                  &
L02002                       SH2O,                                              &
L02003                       SMCMAX,BEXP,PC,SMCWLT,DKSAT,DWSAT,                 &
L02004                       SMCREF,SHDFAC,CMCMAX,                              &
L02005                       SMCDRY,CFACTR,                                     &
L02006                        EDIR1,EC1,ET1,ETT1,SFCTMP,Q2,NROOT,RTDIS,FXEXP,   &
L02007                       SFHEAD1RT,ETPND1 )
L02008  !
L02009  ! FASDAS
L02010  !
L02011        IF( fasdas == 1 ) THEN
L02012          DO K=1,NSOIL
L02013          QFX_PHY = QFX_PHY + ET1(K)    ! m/s
L02014  ! dont add moisture fluxes if soil moisture is = or > smcref
L02015          IF(SMC(K).GE.SMCREF.and.XSDA_QFX.gt.0.0) wetty(K)=0.0
L02016          END DO
L02017         QFX_PHY = EDIR1+EC1+QFX_PHY    ! m/s
L02018         EALL_now = QFX_PHY             ! m/s
L02019         QFX_PHY = QFX_PHY*1000.0       ! Kg/m2/s
L02020
L02021         if(EALL_now.ne.0.0) then
L02022         EFDIR = (EDIR1/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02023         EFDIR = EFDIR * wetty(1)
L02024            !TWG2015 Bugfix Flip Sign to conform to Net upward Flux
L02025            EDIR1 = EDIR1 + EFDIR    ! new value
L02026
L02027         EFC = (EC1/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02028         !TWG2015 Bugfix Flip Sign to conform to Net upward Flux
L02029         EC1 = EC1 + EFC         ! new value
L02030
L02031
L02032         DO K=1,NSOIL
L02033          EFT(K) = (ET1(K)/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02034          EFT(K) =  EFT(K) * wetty(K)
L02035          !TWG2015 Bugfix Flip Sign to conform to Net upward Flux
L02036          ET1(K) = ET1(K) + EFT(K) ! new value
L02037         END DO
L02038
L02039
L02040         END IF ! for non-zero eall_now
L02041        ELSE
L02042          QFX_PHY = 0.0
L02043        ENDIF
L02044  !
L02045  ! END FASDAS
L02046  !
L02047           CALL SMFLX (SMC,NSOIL,CMC,DT,PRCP1,ZSOIL,                      &
L02048                       SH2O,SLOPE,KDT,FRZFACT,                            &
L02049                       SMCMAX,BEXP,SMCWLT,DKSAT,DWSAT,                    &
L02050                       SHDFAC,CMCMAX,                                     &
L02051                       RUNOFF1,RUNOFF2,RUNOFF3,                           &
L02052                       EDIR1,EC1,ET1,                                     &
L02053                       DRIP, SFHEAD1RT,INFXS1RT,IRRIGATION_CHANNEL)
L02054
L02055  ! ----------------------------------------------------------------------
L02056  ! CONVERT MODELED EVAPOTRANSPIRATION FROM  M S-1  TO  KG M-2 S-1.
L02057  ! ----------------------------------------------------------------------
L02058
L02059           ETA = ETA1 * 1000.0
L02060
L02061  ! ----------------------------------------------------------------------
L02062  ! IF ETP < 0, ASSUME DEW FORMS (TRANSFORM ETP1 INTO DEW AND REINITIALIZE
L02063  ! ETP1 TO ZERO).
L02064  ! ----------------------------------------------------------------------
L02065        ELSE
L02066           DEW = - ETP1
```

```
L02067
L02068  ! ------------------------------------------------------------------
L02069  ! CONVERT PRCP FROM 'KG M-2 S-1' TO 'M S-1' AND ADD DEW AMOUNT.
L02070  ! ------------------------------------------------------------------
L02071
L02072           PRCP1 = PRCP1+ DEW
L02073  !
L02074  ! FASDAS
L02075  !
L02076       IF( fasdas == 1 ) THEN
L02077         DO K=1,NSOIL
L02078          QFX_PHY = QFX_PHY + ET1(K)   ! m/s
L02079  ! dont add moisture fluxes if soil moisture is = or > smcref
L02080          IF(SMC(K).GE.SMCREF.and.XSDA_QFX.gt.0.0) wetty(K)=0.0
L02081         END DO
L02082         QFX_PHY = EDIR1+EC1+QFX_PHY    ! m/s
L02083          EALL_now = QFX_PHY      ! m/s
L02084         QFX_PHY = QFX_PHY*1000.0    ! Kg/m2/s
L02085
L02086         IF(EALL_now.ne.0.0) then
L02087         EFDIR = (EDIR1/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02088          EFDIR =  EFDIR * wetty(1)
L02089           !TWG2015 Bugfix Flip Sign to conform to Net Upward Flux
L02090            EDIR1 = EDIR1 + EFDIR    ! new value
L02091
L02092          EFC = (EC1/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02093          !TWG2015 Bugfix Flip Sign to conform to Net Upward Flux
L02094          EC1 = EC1+ EFC         ! new value
L02095
L02096         DO K=1,NSOIL
L02097          EFT(K) = (ET1(K)/EALL_now)*XSDA_QFX*1.0E-03*XQNORM
L02098          EFT(K) = EFT(K) * wetty(K)
L02099          !TWG2015 Bugfix Flip Sign to conform to Net Upward Flux
L02100          ET1(K) = ET1(K) + EFT(K)   ! new value
L02101         END DO
L02102
L02103          END IF ! for non-zero eall_now
L02104        ELSE
L02105          QFX_PHY = 0.0
L02106        ENDIF
L02107  !
L02108  ! END FASDAS
L02109  !
L02110           CALL SMFLX (SMC,NSOIL,CMC,DT,PRCP1,ZSOIL,                      &
L02111                       SH2O,SLOPE,KDT,FRZFACT,                            &
L02112                       SMCMAX,BEXP,SMCWLT,DKSAT,DWSAT,                    &
L02113                       SHDFAC,CMCMAX,                                     &
L02114                       RUNOFF1,RUNOFF2,RUNOFF3,                           &
L02115                       EDIR1,EC1,ET1,                                     &
L02116                       DRIP, SFHEAD1RT,INFXS1RT,IRRIGATION_CHANNEL)
L02117
L02118  ! ------------------------------------------------------------------
L02119  ! CONVERT MODELED EVAPOTRANSPIRATION FROM 'M S-1' TO 'KG M-2 S-1'.
L02120  ! ------------------------------------------------------------------
L02121  !         ETA = ETA1 * 1000.0
L02122        END IF
L02123
L02124  ! ------------------------------------------------------------------
L02125  ! BASED ON ETP AND E VALUES, DETERMINE BETA
L02126  ! ------------------------------------------------------------------
L02127
L02128        IF ( ETP <= 0.0 ) THEN
L02129           BETA = 0.0
L02130           ETA = ETP
L02131           IF ( ETP < 0.0 ) THEN
L02132              BETA = 1.0
L02133           END IF
L02134        ELSE
L02135           BETA = ETA / ETP
L02136        END IF
L02137
L02138  ! ------------------------------------------------------------------
L02139  ! CONVERT MODELED EVAPOTRANSPIRATION COMPONENTS 'M S-1' TO 'KG M-2 S-1'.
L02140  ! ------------------------------------------------------------------
L02141        EDIR = EDIR1*1000.
L02142        EC = EC1*1000.
L02143        DO K = 1,NSOIL
L02144          ET(K) = ET1(K)*1000.
L02145        END DO
L02146        ETT = ETT1*1000.
L02147
L02148  ! ------------------------------------------------------------------
L02149  ! GET SOIL THERMAL DIFFUXIVITY/CONDUCTIVITY FOR TOP SOIL LYR,
L02150  ! CALC. ADJUSTED TOP LYR SOIL TEMP AND ADJUSTED SOIL FLUX, THEN
L02151  ! CALL SHFLX TO COMPUTE/UPDATE SOIL HEAT FLUX AND SOIL TEMPS.
L02152  ! ------------------------------------------------------------------
L02153
L02154        CALL TDFCND (DF1,SMC (1),QUARTZ,SMCMAX,SH2O (1),BEXP, PSISAT, SOILTYP, OPT_THCND)
L02155
L02156  !urban
L02157        IF ( VEGTYP == ISURBAN ) DF1=3.24
L02158  !
L02159
L02160  ! ------------------------------------------------------------------
L02161  ! VEGETATION GREENNESS FRACTION REDUCTION IN SUBSURFACE HEAT FLUX
L02162  ! VIA REDUCTION FACTOR, WHICH IS CONVENIENT TO APPLY HERE TO THERMAL
L02163  ! DIFFUSIVITY THAT IS LATER USED IN HRT TO COMPUTE SUB SFC HEAT FLUX
L02164  ! (SEE ADDITIONAL COMMENTS ON VEG EFFECT SUB-SFC HEAT FLX IN
L02165  ! ROUTINE SFLX)
L02166  ! ------------------------------------------------------------------
L02167        DF1 = DF1 * EXP (SBETA * SHDFAC)
L02168  ! ------------------------------------------------------------------
L02169  ! COMPUTE INTERMEDIATE TERMS PASSED TO ROUTINE HRT (VIA ROUTINE
L02170  ! SHFLX BELOW) FOR USE IN COMPUTING SUBSURFACE HEAT FLUX IN HRT
L02171  ! ------------------------------------------------------------------
L02172        YYNUM = FDOWN - EMISSI*SIGMA * T24
L02173        YY = SFCTMP + (YYNUM / RCH + TH2- SFCTMP - BETA * EPSCA) / RR
L02174
L02175        ZZ1 = DF1 / ( -0.5 * ZSOIL (1) * RCH * RR ) + 1.0
L02176
```

```
L02177  !urban
L02178        CALL SHFLX (SSOIL,STC,SMC,SMCMAX,NSOIL,T1,DT,YY,ZZ1,ZSOIL,       &
L02179                    TBOT,ZBOT,SMCWLT,PSISAT,SH2O,BEXP,F1,DF1,            &
L02180                    QUARTZ,CSOIL,VEGTYP,ISURBAN,SOILTYP,OPT_THCND        &
L02181                   ,HCPCT_FASDAS                                         ) !fasdas
L02182
L02183  ! ----------------------------------------------------------------------
L02184  ! SET FLX1 AND FLX3 (SNOPACK PHASE CHANGE HEAT FLUXES) TO ZERO SINCE
L02185  ! THEY ARE NOT USED HERE IN SNOPAC.  FLX2 (FREEZING RAIN HEAT FLUX) WAS
L02186  ! SIMILARLY INITIALIZED IN THE PENMAN ROUTINE.
L02187  ! ----------------------------------------------------------------------
L02188        FLX1 = CPH2O * PRCP * (T1- SFCTMP)
L02189        FLX3 = 0.0
L02190
L02191  ! ----------------------------------------------------------------------
L02192    END SUBROUTINE NOPAC
```

讲解：无雪路径总控，七步：① 清零与单位换算。②ETP>0 调 EVAPO 三通道（fasdas=1 时按 EDIR/EC/ET 的通量占比分配同化增量 XSDA_QFX • XQNORM，wetty 屏蔽已达田间持水量的层——Flux-Adjusting 同化，Alapaty et al. 2008）；ETP≤0 则 DEW=−ETP1 并入 PRCP1（露/霜水直接当降水参与下渗）。③ 两分支都调 SMFLX（12b）推进土壤水，IRRIGATION_CHANNEL 一并透传。④BETA 收口：ETP=0→0、ETP<0→1、否则 ETA/ETP。⑤TDFCND+urban 3.24+exp(SBETA • SHDFAC) 重算 DF1——与 SFLX 第 ⑥ 站同一套，这里再算一遍供 HRT 用（代码冗余但保持两路径独立可读）。⑥ **皮肤温度耦合**：YYNUM=FDOWN−EMISSI • σ • T24；YY=SFCTMP+(YYNUM/RCH+TH2−SFCTMP−BETA • EPSCA)/RR——Penman 线性化能量平衡的反解；ZZ1=DF1/(−0.5 • ZSOIL1 • RCH • RR)+1——把皮肤温度与表层土温的耦合强度写进这两个量，交给 HRT 的 BI(1)。⑦SHFLX（12b）推进土温；FLX1=CPH2O • PRCP • (T1−SFCTMP)（雨滴与地面的温差热交换）、FLX3=0（雪相变项无雪恒零）。

十、潜势蒸发 PENMAN

```
!file: phys/module_sf_noahlsm.F    L2195-L2312    (subroutine PENMAN)
L02195        SUBROUTINE PENMAN (SFCTMP,SFCPRS,CH,T2V,TH2,PRCP,FDOWN,T24,SSOIL, &
L02196       &                   Q2,Q2SAT,ETP,RCH,EPSCA,RR,SNOWNG,FRZGRA,      &
L02197       &                   DQSDT2,FLX2,EMISSI_IN,SNEQV,T1,SNCOVR,AOASIS,  &
L02198                    ALBEDO,SOLDN,FVB,GAMA,STC1,ETPN,FLX4,UA_PHYS)
L02199
L02200  ! ----------------------------------------------------------------------
L02201  ! SUBROUTINE PENMAN
L02202  ! ----------------------------------------------------------------------
L02203  ! CALCULATE POTENTIAL EVAPORATION FOR THE CURRENT POINT.  VARIOUS
L02204  ! PARTIAL SUMS/PRODUCTS ARE ALSO CALCULATED AND PASSED BACK TO THE
L02205  ! CALLING ROUTINE FOR LATER USE.
L02206  ! ----------------------------------------------------------------------
L02207        IMPLICIT NONE
L02208        LOGICAL, INTENT(IN)      :: SNOWNG, FRZGRA
L02209        REAL, INTENT(IN)         :: CH, DQSDT2,FDOWN,PRCP,                 &
L02210                                    Q2, Q2SAT,SSOIL, SFCPRS, SFCTMP,       &
L02211                                    T2V, TH2,EMISSI_IN,SNEQV,AOASIS
L02212        REAL, INTENT(IN)         :: T1 , SNCOVR
L02213        REAL, INTENT(IN)         :: ALBEDO,SOLDN,FVB,GAMA,STC1
L02214        LOGICAL, INTENT(IN)      :: UA_PHYS
L02215  !
L02216        REAL, INTENT(OUT)        :: EPSCA,ETP,FLX2,RCH,RR,T24
L02217        REAL, INTENT(OUT)        :: FLX4,ETPN
L02218        REAL                     :: A, DELTA, FNET,RAD,RHO,EMISSI,ELCP1,LVS
L02219        REAL                     :: TOTABS,UCABS,SIGNCK,FNETN,RADN,EPSCAN
L02220
L02221        REAL, PARAMETER       :: ELCP = 2.4888E+3, LSUBC = 2.501000E+6,CP = 1004.6
L02222        REAL, PARAMETER       :: LSUBS = 2.83E+6
L02223        REAL, PARAMETER       :: ALGDSN = 0.5, ALVGSN = 0.13
L02224
L02225  ! ----------------------------------------------------------------------
L02226  ! EXECUTABLE CODE BEGINS HERE:
L02227  ! ----------------------------------------------------------------------
L02228  ! ----------------------------------------------------------------------
L02229  ! PREPARE PARTIAL QUANTITIES FOR PENMAN EQUATION.
L02230  ! ----------------------------------------------------------------------
L02231         EMISSI=EMISSI_IN
L02232         ELCP1  = (1.0-SNCOVR)*ELCP  + SNCOVR*ELCP*LSUBS/LSUBC
L02233         LVS    = (1.0-SNCOVR)*LSUBC + SNCOVR*LSUBS
L02234
L02235        FLX2 = 0.0
L02236  !      DELTA = ELCP * DQSDT2
L02237        DELTA = ELCP1 * DQSDT2
L02238        T24 = SFCTMP * SFCTMP * SFCTMP * SFCTMP
L02239  !      RR = T24 * 6.48E-8 / (SFCPRS * CH) + 1.0
L02240        RR = EMISSI*T24 * 6.48E-8 / (SFCPRS * CH) + 1.0
L02241        RHO = SFCPRS / (RD * T2V)
L02242
L02243  ! ----------------------------------------------------------------------
L02244  ! ADJUST THE PARTIAL SUMS / PRODUCTS WITH THE LATENT HEAT
L02245  ! EFFECTS CAUSED BY FALLING PRECIPITATION.
L02246  ! ----------------------------------------------------------------------
L02247        RCH = RHO * CP * CH
L02248        IF (.NOT. SNOWNG) THEN
L02249           IF (PRCP >  0.0) RR = RR + CPH2O * PRCP / RCH
L02250        ELSE
L02251           RR = RR + CPICE * PRCP / RCH
L02252        END IF
L02253
L02254  ! ----------------------------------------------------------------------
L02255  ! INCLUDE THE LATENT HEAT EFFECTS OF FRZNG RAIN CONVERTING TO ICE ON
L02256  ! IMPACT IN THE CALCULATION OF FLX2 AND FNET.
```

```
L02257  ! ----------------------------------------------------------------------
L02258  !      FNET = FDOWN - SIGMA * T24- SSOIL
L02259         FNET = FDOWN -  EMISSI*SIGMA * T24- SSOIL
L02260
L02261         FLX4 = 0.0
L02262         IF(UA_PHYS) THEN
L02263           IF(SNEQV > 0. .AND. FNET > 0. .AND. SOLDN > 0. ) THEN
L02264            TOTABS = (1.-ALBEDO)*SOLDN*FVB           ! solar radiation absorbed
L02265                                                     ! by vegetated fraction
L02266            UCABS = MIN(TOTABS,((1.0-ALGDSN)*(1.0-ALVGSN)*SOLDN*GAMA)*FVB)
L02267  !          print*,'penman',UCABS,TOTABS,SOLDN,GAMA,FVB
L02268  !          UCABS = MIN(TOTABS,(0.44*SOLDN*GAMA)*FVB)
L02269                                                     ! UCABS -> solar radiation
L02270                              ! absorbed under canopy
L02271            FLX4 = MIN(TOTABS - UCABS, MIN(250., 0.5*(1.-ALBEDO)*SOLDN))
L02272           ENDIF
L02273
L02274           SIGNCK = (STC1-273.15)*(SFCTMP-273.15)
L02275
L02276           IF(FLX4 > 0. .AND. (SIGNCK <= 0. .OR. STC1 < 273.15)) THEN
L02277             IF(FNET >= FLX4) THEN
L02278              FNETN = FNET - FLX4
L02279             ELSE
L02280              FLX4 = FNET
L02281              FNETN = 0.
L02282             ENDIF
L02283           ELSE
L02284             FLX4 = 0.0
L02285             FNETN = 0.
L02286           ENDIF
L02287         ENDIF
L02288
L02289         IF (FRZGRA) THEN
L02290            FLX2 = - LSUBF * PRCP
L02291            FNET = FNET - FLX2
L02292            IF(UA_PHYS) FNETN = FNETN - FLX2
L02293  ! ----------------------------------------------------------------------
L02294  ! FINISH PENMAN EQUATION CALCULATIONS.
L02295  ! ----------------------------------------------------------------------
L02296         END IF
L02297         RAD = FNET / RCH + TH2- SFCTMP
L02298  !      A = ELCP * (Q2SAT - Q2)
L02299         A = ELCP1 * (Q2SAT - Q2)
L02300         EPSCA = (A * RR + RAD * DELTA) / (DELTA + RR)
L02301  ! Fei-Mike
L02302         IF (EPSCA>0.) EPSCA = EPSCA * AOASIS
L02303  !      ETP = EPSCA * RCH / LSUBC
L02304         ETP = EPSCA * RCH / LVS
L02305
L02306         IF(UA_PHYS) THEN
L02307           RADN = FNETN / RCH + TH2- SFCTMP
L02308           EPSCAN = (A * RR + RADN * DELTA) / (DELTA + RR)
L02309           ETPN = EPSCAN * RCH / LVS
L02310         END IF
L02311  ! ----------------------------------------------------------------------
L02312    END SUBROUTINE PENMAN
```

讲解：Penman 潜势蒸发 + 中间量工厂：①T24=SFCTMP⁴；RR=EMISSI • T24 • 6.48E-8/(SFCPRS • CH)+1——长波线性化辐射项（6.48E-8 是 4σ/(RD • CP) 口径），含发射率是 v3.6+ 的修正；RHO=SFCPRS/(RD • T2V)、RCH=RHO • CP • CH——CH 本身是 conductance（m/s），乘 ρ • CP 才是热导 W/(m²K)。② 降水的 RR 增项：雨 CPH2O • PRCP/RCH、雪 CPICE • PRCP/RCH（降水热容加入能量平衡）。③FLX2 冻雨潜热 =−LSUBF • PRCP，进 FNET=FDOWN−EMISSI • σ • T24−SSOIL。④ELCP1/LVS 按雪盖率混合潜热（雪面用 LSUBS=2.83E6 升华潜热）：DELTA=ELCP1 • DQSDT2、A=ELCP1 • (Q2SAT−Q2)。⑤EPSCA=(A • RR+RAD • DELTA)/(DELTA+RR)、RAD=FNET/RCH+TH2−SFCTMP；**Fei–Mike 补丁：EPSCA>0 时乘 AOASIS**（绿洲系数，由驱动层传入——干旱区绿洲效应实验的现成挂点）；ETP=EPSCA • RCH/LVS。⑥UA 分支：FLX4 是冠层对雪面短波的截留再分配（TOTABS 植被吸收与 UCABS 冠下吸收之差，钳位 min(250, 0.5 • (1−ALBEDO) • SOLDN)），只在「有雪+地表土温与气温异号或地表冻结」时从 FNET 扣除；ETPN 是扣除 FLX4 后的 UA 版潜势蒸发（SNOPAC 里雪面升华用它）。

十一、改造入口

1. **城市下垫面**：SFLX 的 ISURBAN 硬参数块（SHDFAC/RSMIN/SMC 四值）+ HRT 的 CSOIL_LOC=3.0E6 + 三处 DF1=3.24——改城市土壤热性质的全部位置；与 BEP/BEM（code_14/15/16）互斥使用。
2. **灌溉**：IRRIGATION_CHANNEL 的 OPTIONAL 透传链 SFLX→NOPAC→SMFLX（noahdrv 侧的驱动与格点源项见 code_13）；SMFLX 内 PCPDRP 加 0.001 • IRRIGATION_CHANNEL。
3. **绿洲/干旱实验**：PENMAN 的 AOASIS 乘子（Fei–Mike 补丁）。
4. **雪反照率与雪导热**：ALCALC 的 SNACCA/SNACCB/LVCOEF 与 CSNOW 的 ×2.0 翻倍——城市雪面辐射偏差排查的第一站。
5. **fasdas 同化**：XSDA_QFX/XQNORM/QFX_PHY/HFX_PHY/HCPCT_FASDAS 贯穿 NOPAC/SFLX——地表数据同化二次开发的现成模板。
6. **四层假设陷阱**：SFLX 收尾 DO K=1,4 的 ET 能量换算与 SOILWM 根区累积——NSOIL≠4 的分层实验必须检查这些硬编码循环。

十二、上下游链接

- 下半（参数查表、三对角求解器、雪线、土壤水 SRT/SSTEP、SFCDIF_off）：`code_12b_module_sf_noahlsm.md`
- 驱动层（逐格点循环、TBL 读取、海冰/冰川分支）：`code_13_module_sf_noahdrv.md`
- 面层对照（WRF 主线中 CH 的真实来源）：`code_10_module_sf_mynnsfc.md`；城市方案：`code_14a_module_sf_bep_bem.md`
- 教材正文：ch16_land_surface_family.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_12b — Noah LSM 本体 · 下半（module_sf_noahlsm.F：参数查表、隐式求解器与雪线）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_noahlsm.F	4759	9bc3fbaae919bd37ca4d05e44599c582

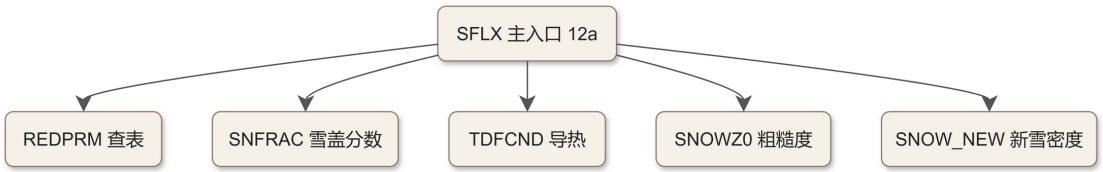
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。

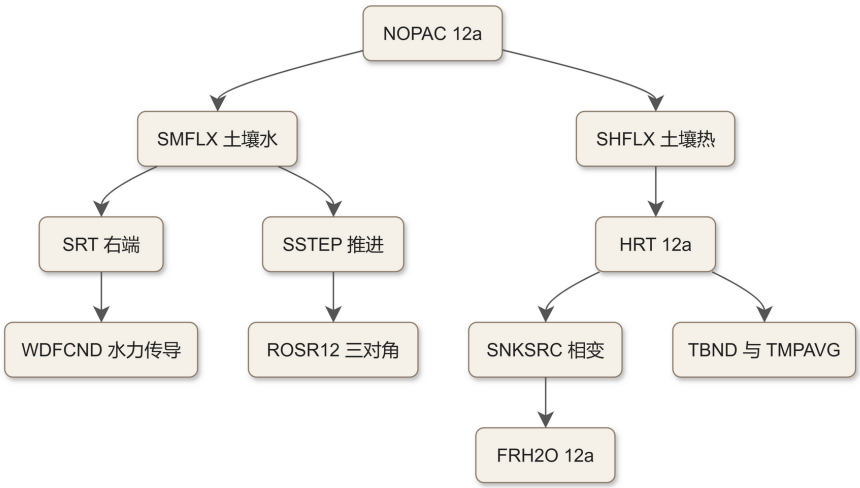
- 本篇是 A 级全文拆分的下半：REDPRM 起共 18 单元（L2315–L4756）；上半（文件头 +SFLX 主入口 + 蒸发/热力线 13 单元，L1–L2312）见 `code_12a_module_sf_noahlsm.md`。

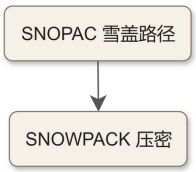
二、定位

一句话：Noah 本体的「底层」——三张 TBL 的查表出口、水热两套三对角隐式求解器、土壤水与雪盖的全部状态推进，以及 WRF 主线中已休眠的自带面层方案 SFCDIF_off；被 12a 的 SFLX/NOPAC 与 12b 的 SNOPAC 调用。

单元地图（实线 =调用）：







三、参数查表 REDPRM

```
!file: phys/module_sf_noahlsm.F    L2315-L2535    (subroutine REDPRM)
L02315        SUBROUTINE REDPRM (VEGTYP,SOILTYP,SLOPETYP,CFACTR,CMCMAX,RSMAX,      &
L02316                           TOPT,                                              &
L02317                           REFKDT,KDT,SBETA, SHDFAC,RSMIN,RGL,HS,ZBOT,FRZX,   &
L02318                           PSISAT,SLOPE,SNUP,SALP,BEXP,DKSAT,DWSAT,           &
L02319                           SMCMAX,SMCWLT,SMCREF,SMCDRY,F1,QUARTZ,FXEXP,       &
L02320                           RTDIS,SLDPTH,ZSOIL, NROOT,NSOIL,CZIL,              &
L02321                           LAIMIN, LAIMAX, EMISSMIN, EMISSMAX, ALBEDOMIN,     &
L02322                           ALBEDOMAX, Z0MIN, Z0MAX, CSOIL, PTU, LLANDUSE,     &
L02323                           LSOIL, LOCAL,LVCOEF,ZTOPV,ZBOTV)
L02324
L02325        IMPLICIT NONE
L02326  ! ---------------------------------------------------------------------
L02327  ! Internally set (default valuess)
L02328  ! all soil and vegetation parameters required for the execusion oF
L02329  ! the Noah lsm are defined in VEGPARM.TBL, SOILPARM.TB, and GENPARM.TBL.
L02330  ! ---------------------------------------------------------------------
L02331  !     Vegetation parameters:
L02332  !             ALBBRD: SFC background snow-free albedo
L02333  !             CMXTBL: MAX CNPY Capacity
L02334  !              Z0BRD: Background roughness length
L02335  !             SHDFAC: Green vegetation fraction
L02336  !              NROOT: Rooting depth
L02337  !              RSMIN: Mimimum stomatal resistance
L02338  !              RSMAX: Max. stomatal resistance
L02339  !                RGL: Parameters used in radiation stress function
L02340  !                 HS: Parameter used in vapor pressure deficit functio
L02341  !               TOPT: Optimum transpiration air temperature.
L02342  !             CMCMAX: Maximum canopy water capacity
L02343  !             CFACTR: Parameter used in the canopy inteception calculation
L02344  !               SNUP: Threshold snow depth (in water equivalent m) that
L02345  !                     implies 100 percent snow cover
L02346  !                LAI: Leaf area index
L02347  !
L02348  ! ---------------------------------------------------------------------
L02349  !      Soil parameters:
L02350  !        SMCMAX: MAX soil moisture content (porosity)
L02351  !        SMCREF: Reference soil moisture  (field capacity)
L02352  !        SMCWLT: Wilting point soil moisture
L02353  !        SMCWLT: Air dry soil moist content limits
L02354  !       SSATPSI: SAT (saturation) soil potential
```

```
L02355  !         DKSAT: SAT soil conductivity
L02356  !          BEXP: B parameter
L02357  !        SSATDW: SAT soil diffusivity
L02358  !           F1: Soil thermal diffusivity/conductivity coef.
L02359  !        QUARTZ: Soil quartz content
L02360  !  Modified by F. Chen (12/22/97)  to use the STATSGO soil map
L02361  !  Modified By F. Chen (01/22/00)  to include PLaya, Lava, and White San
L02362  !  Modified By F. Chen (08/05/02)  to include additional parameters for the Noah
L02363  ! NOTE: SATDW = BB*SATDK*(SATPSI/MAXSMC)
L02364  !         F11 = ALOG10(SATPSI) + BB*ALOG10(MAXSMC) + 2.0
L02365  !       REFSMC1=MAXSMC*(5.79E-9/SATDK)**(1/(2*BB+3)) 5.79E-9 m/s= 0.5 mm
L02366  !       REFSMC=REFSMC1+1./3.(MAXSMC-REFSMC1)
L02367  !       WLTSMC1=MAXSMC*(200./SATPSI)**(-1./BB)    (Wetzel and Chang, 198
L02368  !       WLTSMC=WLTSMC1-0.5*WLTSMC1
L02369  ! Note: the values for playa is set for it to have a thermal conductivit
L02370  ! as sand and to have a hydrulic conductivity as clay
L02371  !
L02372  ! ------------------------------------------------------------------
L02373  ! Class parameter 'SLOPETYP' was included to estimate linear reservoir
L02374  ! coefficient 'SLOPE' to the baseflow runoff out of the bottom layer.
L02375  ! lowest class (slopetyp=0) means highest slope parameter = 1.
L02376  ! definition of slopetyp from 'zobler' slope type:
L02377  ! slope class  percent slope
L02378  ! 1            0-8
L02379  ! 2            8-30
L02380  ! 3            > 30
L02381  ! 4            0-30
L02382  ! 5            0-8 & > 30
L02383  ! 6            8-30 & > 30
L02384  ! 7            0-8, 8-30, > 30
L02385  ! 9            GLACIAL ICE
L02386  ! BLANK        OCEAN/SEA
L02387  !       SLOPE_DATA: linear reservoir coefficient
L02388  !       SBETA_DATA: parameter used to caluculate vegetation effect on soil heat
L02389  !       FXEXP_DAT:  soil evaporation exponent used in DEVAP
L02390  !       CSOIL_DATA: soil heat capacity [J M-3 K-1]
L02391  !       SALP_DATA: shape parameter of  distribution function of snow cover
L02392  !       REFDK_DATA and REFKDT_DATA: parameters in the surface runoff parameteriz
L02393  !       FRZK_DATA: frozen ground parameter
L02394  !       ZBOT_DATA: depth[M] of lower boundary soil temperature
L02395  !       CZIL_DATA: calculate roughness length of heat
L02396  !       SMLOW_DATA and MHIGH_DATA: two soil moisture wilt, soil moisture referen
L02397  !                 parameters
L02398  ! Set maximum number of soil-, veg-, and slopetyp in data statement.
L02399  ! ------------------------------------------------------------------
L02400        INTEGER, PARAMETER     :: MAX_SLOPETYP=30,MAX_SOILTYP=30,MAX_VEGTYP=30
L02401        LOGICAL                :: LOCAL
L02402        CHARACTER (LEN=256), INTENT(IN)::  LLANDUSE, LSOIL
L02403
L02404  ! Veg parameters
L02405        INTEGER, INTENT(IN)    :: VEGTYP
L02406        INTEGER, INTENT(OUT)   :: NROOT
L02407        REAL, INTENT(INOUT)    :: SHDFAC
L02408        REAL, INTENT(OUT)      :: HS,RSMIN,RGL,SNUP,                          &
L02409                                  CMCMAX,RSMAX,TOPT,                          &
L02410                                  EMISSMIN,  EMISSMAX,                        &
L02411                                  LAIMIN,    LAIMAX,                          &
L02412                                  Z0MIN,     Z0MAX,                           &
L02413                                  ALBEDOMIN, ALBEDOMAX, ZTOPV, ZBOTV
L02414  ! Soil parameters
L02415        INTEGER, INTENT(IN)    :: SOILTYP
L02416        REAL, INTENT(OUT)      :: BEXP,DKSAT,DWSAT,F1,QUARTZ,SMCDRY,          &
L02417                                  SMCMAX,SMCREF,SMCWLT,PSISAT
L02418  ! General parameters
L02419        INTEGER, INTENT(IN)    :: SLOPETYP,NSOIL
L02420        INTEGER                :: I
L02421
L02422        REAL,    INTENT(OUT)   :: SLOPE,CZIL,SBETA,FXEXP,                     &
L02423                                  CSOIL,SALP,FRZX,KDT,CFACTR,      &
L02424                                  ZBOT,REFKDT,PTU
L02425        REAL,    INTENT(OUT)   :: LVCOEF
L02426        REAL,DIMENSION(1:NSOIL),INTENT(IN) :: SLDPTH,ZSOIL
L02427        REAL,DIMENSION(1:NSOIL),INTENT(OUT):: RTDIS
L02428        REAL                   :: FRZFACT,FRZK,REFDK
L02429
L02430  !      SAVE
L02431  ! ------------------------------------------------------------------
L02432  !
L02433                IF (SOILTYP .gt. SLCATS) THEN
L02434                      FATAL_ERROR( 'Warning: too many input soil types' )
L02435                END IF
L02436                IF (VEGTYP .gt. LUCATS) THEN
L02437                      FATAL_ERROR( 'Warning: too many input landuse types' )
L02438                END IF
L02439                IF (SLOPETYP .gt. SLPCATS) THEN
L02440                      FATAL_ERROR( 'Warning: too many input slope types' )
L02441                END IF
L02442
L02443  ! ------------------------------------------------------------------
L02444  !  SET-UP SOIL PARAMETERS
L02445  ! ------------------------------------------------------------------
L02446        CSOIL = CSOIL_DATA
L02447        BEXP = BB (SOILTYP)
L02448        DKSAT = SATDK (SOILTYP)
L02449        DWSAT = SATDW (SOILTYP)
L02450        F1 = F11 (SOILTYP)
L02451        PSISAT = SATPSI (SOILTYP)
L02452        QUARTZ = QTZ (SOILTYP)
L02453        SMCDRY = DRYSMC (SOILTYP)
L02454        SMCMAX = MAXSMC (SOILTYP)
L02455        SMCREF = REFSMC (SOILTYP)
L02456        SMCWLT = WLTSMC (SOILTYP)
L02457  ! ------------------------------------------------------------------
L02458  ! Set-up universal parameters (not dependent on SOILTYP, VEGTYP or
L02459  ! SLOPETYP)
L02460  ! ------------------------------------------------------------------
L02461        ZBOT = ZBOT_DATA
L02462        SALP = SALP_DATA
L02463        SBETA = SBETA_DATA
```

```
L02464         REFDK = REFDK_DATA
L02465         FRZK = FRZK_DATA
L02466         FXEXP = FXEXP_DATA
L02467         REFKDT = REFKDT_DATA
L02468         PTU = 0.    ! (not used yet) to satisify intent(out)
L02469         KDT = REFKDT * DKSAT / REFDK
L02470         CZIL = CZIL_DATA
L02471         SLOPE = SLOPE_DATA (SLOPETYP)
L02472         LVCOEF = LVCOEF_DATA
L02473
L02474    ! ----------------------------------------------------------------------
L02475    ! TO ADJUST FRZK PARAMETER TO ACTUAL SOIL TYPE: FRZK * FRZFACT
L02476    ! ----------------------------------------------------------------------
L02477         FRZFACT = (SMCMAX / SMCREF) * (0.412 / 0.468)
L02478         FRZX = FRZK * FRZFACT
L02479
L02480    ! ----------------------------------------------------------------------
L02481    ! SET-UP VEGETATION PARAMETERS
L02482    ! ----------------------------------------------------------------------
L02483         TOPT = TOPT_DATA
L02484         CMCMAX = CMCMAX_DATA
L02485         CFACTR = CFACTR_DATA
L02486         RSMAX = RSMAX_DATA
L02487         NROOT = NROTBL (VEGTYP)
L02488         SNUP = SNUPTBL (VEGTYP)
L02489         RSMIN = RSTBL (VEGTYP)
L02490         RGL = RGLTBL (VEGTYP)
L02491         HS = HSTBL (VEGTYP)
L02492         EMISSMIN  = EMISSMINTBL  (VEGTYP)
L02493         EMISSMAX  = EMISSMAXTBL  (VEGTYP)
L02494         LAIMIN    = LAIMINTBL    (VEGTYP)
L02495         LAIMAX    = LAIMAXTBL    (VEGTYP)
L02496         Z0MIN     = Z0MINTBL     (VEGTYP)
L02497         Z0MAX     = Z0MAXTBL     (VEGTYP)
L02498         ALBEDOMIN = ALBEDOMINTBL (VEGTYP)
L02499         ALBEDOMAX = ALBEDOMAXTBL (VEGTYP)
L02500         ZTOPV     = ZTOPVTBL     (VEGTYP)
L02501         ZBOTV     = ZBOTVTBL     (VEGTYP)
L02502
L02503                 IF (VEGTYP .eq. BARE) SHDFAC = 0.0
L02504                 IF (NROOT .gt. NSOIL) THEN
L02505                    WRITE (err_message,*) 'Error: too many root layers ',  &
L02506                                                 NSOIL,NROOT
L02507                    FATAL_ERROR( err_message )
L02508    ! ----------------------------------------------------------------------
L02509    ! CALCULATE ROOT DISTRIBUTION.  PRESENT VERSION ASSUMES UNIFORM
L02510    ! DISTRIBUTION BASED ON SOIL LAYER DEPTHS.
L02511    ! ----------------------------------------------------------------------
L02512                 END IF
L02513                 DO I = 1,NROOT
L02514                    RTDIS (I) = - SLDPTH (I)/ ZSOIL (NROOT)
L02515    ! ----------------------------------------------------------------------
L02516    !  SET-UP SLOPE PARAMETER
L02517    ! ----------------------------------------------------------------------
L02518                 END DO
L02519
L02520    !        print*,'end of PRMRED'
L02521    !       print*,'VEGTYP',VEGTYP,'SOILTYP',SOILTYP,'SLOPETYP',SLOPETYP,    &
L02522    !    & 'CFACTR',CFACTR,'CMCMAX',CMCMAX,'RSMAX',RSMAX,'TOPT',TOPT,        &
L02523    !    & 'REFKDT',REFKDT,'KDT',KDT,'SBETA',SBETA,  'SHDFAC',SHDFAC,        &
L02524    !    &  'RSMIN',RSMIN,'RGL',RGL,'HS',HS,'ZBOT',ZBOT,'FRZX',FRZX,         &
L02525    !    &  'PSISAT',PSISAT,'SLOPE',SLOPE,'SNUP',SNUP,'SALP',SALP,'BEXP',    &
L02526    !    &   BEXP,                                                           &
L02527    !    &  'DKSAT',DKSAT,'DWSAT',DWSAT,                                     &
L02528    !    &  'SMCMAX',SMCMAX,'SMCWLT',SMCWLT,'SMCREF',SMCREF,'SMCDRY',SMCDRY, &
L02529    !    &  'F1',F1,'QUARTZ',QUARTZ,'FXEXP',FXEXP,                           &
L02530    !    &  'RTDIS',RTDIS,'SLDPTH',SLDPTH,'ZSOIL',ZSOIL,  'NROOT',NROOT,     &
L02531    !    &  'NSOIL',NSOIL,'Z0',Z0,'CZIL',CZIL,'LAI',LAI,                     &
L02532    !    &  'CSOIL',CSOIL,'PTU',PTU,                                         &
L02533    !    &  'LOCAL', LOCAL
L02534
L02535         END  SUBROUTINE REDPRM
```

讲解：三张 TBL 的查表出口，Noah 所有可调参数的「出厂设置」：① 土壤参数按 SOILTYP 查 SOILPARM.TBL（BB、SATDK、SATDW、F11、SATPSI、QTZ、DRYSMC、MAXSMC、REFSMC、WLTSMC）；② 通用参数查 GENPARM.TBL（ZBOT、SALP、SBETA、REFDK、FRZK、FXEXP、REFKDT、CZIL、LVCOEF，SLOPE 按 SLOPETYP 查 Zobler 坡度类 1–7+9 冰川）；其中 **KDT=REFKDT·DKSAT/REFDK**——Schaake 入渗参数 KDT 由参考值标定；**FRZFACT=(SMCMAX/SMCREF)·(0.412/0.468)**、FRZX=FRZK•FRZFACT——冻土参数按土壤持水特性折算。③ 植被参数按 VEGTYP 查 VEGPARM.TBL（全局 TOPT/CMCMAX/CFACTR/RSMAX + 每类 NROTBL/SNUPTBL/RSTBL/RGLTBL/HS 与 EMISS/LAI/Z0/ALBEDO 的 min-max 对、ZTOPV/ZBOTV）；BARE 类强制 SHDFAC=0；NROOT>NSOIL 报错。④**RTDIS(I)=−SLDPTH(I)/ZSOIL(NROOT)**——均匀根区分布（层厚/根区总深），注释区保留了「按层深比例」等备选。换土壤/植被数据集（如高分辨率土壤库、论文 2 的下垫面参数改造）改 TBL 或在此加分支即可。

四、三对角求解器 ROSR12

```
!file: phys/module_sf_noahlsm.F    L2537-L2596    (subroutine ROSR12)
L02537         SUBROUTINE ROSR12 (P,A,B,C,D,DELTA,NSOIL)
L02538
L02539    ! ----------------------------------------------------------------------
L02540    ! SUBROUTINE ROSR12
L02541    ! ----------------------------------------------------------------------
L02542    ! INVERT (SOLVE) THE TRI-DIAGONAL MATRIX PROBLEM SHOWN BELOW:
```

```
L02543  ! ###                                             ### ###  ###   ###  ###
L02544  ! #B(1), C(1),  0  ,  0  ,  0  ,   . . .  ,    0   # #      #   #      #
L02545  ! #A(2), B(2), C(2),  0  ,  0  ,   . . .  ,    0   # #      #   #      #
L02546  ! # 0  , A(3), B(3), C(3),  0  ,   . . .  ,    0   # #      #   # D(3) #
L02547  ! # 0  ,  0  , A(4), B(4), C(4),   . . .  ,    0   # # P(4) #   # D(4) #
L02548  ! # 0  ,  0  ,  0  , A(5), B(5),   . . .  ,    0   # # P(5) #   # D(5) #
L02549  ! # .                                             .   # #  .   # = #   .  #
L02550  ! # .                                             .   # #  .   #   #   .  #
L02551  ! # .                                             .   # #  .   #   #   .  #
L02552  ! # 0  , . . . , 0 , A(M-2), B(M-2), C(M-2),    0   # #P(M-2)#   #D(M-2)#
L02553  ! # 0  , . . . , 0 ,   0   , A(M-1), B(M-1), C(M-1)# #P(M-1)#   #D(M-1)#
L02554  ! # 0  , . . . , 0 ,   0   ,   0   ,  A(M) ,  B(M) # # P(M) #   # D(M) #
L02555  ! ###                                           ### ###  ###   ###  ###
L02556  ! ------------------------------------------------------------------
L02557        IMPLICIT NONE
L02558
L02559        INTEGER, INTENT(IN)   :: NSOIL
L02560        INTEGER               :: K, KK
L02561
L02562        REAL, DIMENSION(1:NSOIL), INTENT(IN):: A, B, D
L02563        REAL, DIMENSION(1:NSOIL),INTENT(INOUT):: C,P,DELTA
L02564
L02565  ! ------------------------------------------------------------------
L02566  ! INITIALIZE EQN COEF C FOR THE LOWEST SOIL LAYER
L02567  ! ------------------------------------------------------------------
L02568        C (NSOIL) = 0.0
L02569        P (1) = - C (1) / B (1)
L02570  ! ------------------------------------------------------------------
L02571  ! SOLVE THE COEFS FOR THE 1ST SOIL LAYER
L02572  ! ------------------------------------------------------------------
L02573
L02574  ! ------------------------------------------------------------------
L02575  ! SOLVE THE COEFS FOR SOIL LAYERS 2 THRU NSOIL
L02576  ! ------------------------------------------------------------------
L02577        DELTA (1) = D (1) / B (1)
L02578        DO K = 2,NSOIL
L02579           P (K) = - C (K) * ( 1.0 / (B (K) + A (K) * P (K -1)) )
L02580           DELTA (K) = (D (K) - A (K)* DELTA (K -1))* (1.0/ (B (K) + A (K)&
L02581                     * P (K -1)))
L02582        END DO
L02583  ! ------------------------------------------------------------------
L02584  ! SET P TO DELTA FOR LOWEST SOIL LAYER
L02585  ! ------------------------------------------------------------------
L02586        P (NSOIL) = DELTA (NSOIL)
L02587
L02588  ! ------------------------------------------------------------------
L02589  ! ADJUST P FOR SOIL LAYERS 2 THRU NSOIL
L02590  ! ------------------------------------------------------------------
L02591        DO K = 2,NSOIL
L02592           KK = NSOIL - K + 1
L02593           P (KK) = P (KK) * P (KK +1) + DELTA (KK)
L02594        END DO
L02595  ! ------------------------------------------------------------------
L02596    END SUBROUTINE ROSR12
```

讲解：Thomas 算法（追赶法）：前向消元算 P(K)=−C(K)/(B(K)+A(K)·P(K−1)) 与 DELTA(K)=(D(K)−A(K)·DELTA(K−1))/(B(K)+A(K)·P(K−1))，回代 P(KK)=P(KK)·P(KK+1)+DELTA(KK)；C(NSOIL)=0 底边界无下邻。Noah 水热两套三对角方程共用这一个求解器——ROSR12 是全模块唯一的线性代数设施。

五、土壤热推进外壳 SHFLX

```
!file: phys/module_sf_noahlsm.F    L2600-L2666    (subroutine SHFLX)
L02600        SUBROUTINE SHFLX (SSOIL,STC,SMC,SMCMAX,NSOIL,T1,DT,YY,ZZ1,ZSOIL, &
L02601                          TBOT,ZBOT,SMCWLT,PSISAT,SH2O,BEXP,F1,DF1,     &
L02602                          QUARTZ,CSOIL,VEGTYP,ISURBAN,SOILTYP,OPT_THCND &
L02603                         ,HCPCT_FASDAS                                  ) ! fasdas
L02604
L02605  ! ------------------------------------------------------------------
L02606  ! SUBROUTINE SHFLX
L02607  ! ------------------------------------------------------------------
L02608  ! UPDATE THE TEMPERATURE STATE OF THE SOIL COLUMN BASED ON THE THERMAL
L02609  ! DIFFUSION EQUATION AND UPDATE THE FROZEN SOIL MOISTURE CONTENT BASED
L02610  ! ON THE TEMPERATURE.
L02611  ! ------------------------------------------------------------------
L02612        IMPLICIT NONE
L02613
L02614        INTEGER, INTENT(IN)   :: OPT_THCND
L02615        INTEGER, INTENT(IN)   :: NSOIL, VEGTYP, ISURBAN, SOILTYP
L02616        INTEGER               :: I
L02617
L02618        REAL, INTENT(IN)      :: BEXP,CSOIL,DF1,DT,F1,PSISAT,QUARTZ,     &
L02619                                 SMCMAX, SMCWLT, TBOT,YY, ZBOT,ZZ1
L02620        REAL, INTENT(INOUT)   :: T1
L02621        REAL, INTENT(OUT)     :: SSOIL
L02622        REAL, DIMENSION(1:NSOIL), INTENT(IN)    :: SMC,ZSOIL
L02623        REAL, DIMENSION(1:NSOIL), INTENT(INOUT) :: SH2O
L02624        REAL, DIMENSION(1:NSOIL), INTENT(INOUT) :: STC
L02625        REAL, DIMENSION(1:NSOIL)                :: AI, BI, CI, STCF,RHSTS
L02626        REAL, PARAMETER       :: T0 = 273.15
L02627
L02628  !
L02629  ! FASDAS
L02630  !
L02631        REAL, INTENT(  OUT)     :: HCPCT_FASDAS
L02632  !
L02633  ! END FASDAS
L02634  !
L02635  ! ------------------------------------------------------------------
L02636  ! HRT ROUTINE CALCS THE RIGHT HAND SIDE OF THE SOIL TEMP DIF EQN
```

```
L02637  ! ----------------------------------------------------------------------
L02638
L02639         ! Land case
L02640
L02641         CALL HRT (RHSTS,STC,SMC,SMCMAX,NSOIL,ZSOIL,YY,ZZ1,TBOT,     &
L02642                   ZBOT,PSISAT,SH2O,DT,BEXP,SOILTYP,OPT_THCND,       &
L02643                   F1,DF1,QUARTZ,CSOIL,AI,BI,CI,VEGTYP,ISURBAN       &
L02644                  ,HCPCT_FASDAS                                      ) !fasdas
L02645
L02646         CALL HSTEP (STCF,STC,RHSTS,DT,NSOIL,AI,BI,CI)
L02647
L02648         DO I = 1,NSOIL
L02649            STC (I) = STCF (I)
L02650         ENDDO
L02651
L02652  ! ----------------------------------------------------------------------
L02653  ! IN THE NO SNOWPACK CASE (VIA ROUTINE NOPAC BRANCH,) UPDATE THE GRND
L02654  ! (SKIN) TEMPERATURE HERE IN RESPONSE TO THE UPDATED SOIL TEMPERATURE
L02655  ! PROFILE ABOVE.  (NOTE: INSPECTION OF ROUTINE SNOPAC SHOWS THAT T1
L02656  ! BELOW IS A DUMMY VARIABLE ONLY, AS SKIN TEMPERATURE IS UPDATED
L02657  ! DIFFERENTLY IN ROUTINE SNOPAC)
L02658  ! ----------------------------------------------------------------------
L02659  ! ----------------------------------------------------------------------
L02660  ! CALCULATE SURFACE SOIL HEAT FLUX
L02661  ! ----------------------------------------------------------------------
L02662         T1 = (YY + (ZZ1- 1.0) * STC (1)) / ZZ1
L02663         SSOIL = DF1 * (STC (1) - T1) / (0.5 * ZSOIL (1))
L02664
L02665  ! ----------------------------------------------------------------------
L02666    END SUBROUTINE SHFLX
```

讲解：HRT（右端 +矩阵系数）→ HSTEP（ROSR12 推进）→ STC 原地更新的三步外壳。结尾的 T1=(YY+(ZZ1−1) • STC1)/ZZ1 与 SSOIL=DF1 • (STC1−T1)/(0.5 • ZSOIL1) 是皮肤温度反算：NOPAC 路径里 T1 被它真正更新（无雪时的皮肤温度由表层土温外推）；SNOPAC 路径里传入的 T1 是 dummy（雪面温度已在 SNOPAC 开头定死，注释明确说明）。

六、土壤水推进 SMFLX

```
!file: phys/module_sf_noahlsm.F    L2669-L2813    (subroutine SMFLX)
L02669         SUBROUTINE SMFLX (SMC,NSOIL,CMC,DT,PRCP1,ZSOIL,                  &
L02670        &                  SH2O,SLOPE,KDT,FRZFACT,                       &
L02671        &                  SMCMAX,BEXP,SMCWLT,DKSAT,DWSAT,               &
L02672        &                  SHDFAC,CMCMAX,                                &
L02673        &                  RUNOFF1,RUNOFF2,RUNOFF3,                      &
L02674        &                  EDIR,EC,ET,                                   &
L02675        &                  DRIP, SFHEAD1RT,INFXS1RT,                     &
L02676                           IRRIGATION_CHANNEL )
L02677
L02678  ! ----------------------------------------------------------------------
L02679  ! SUBROUTINE SMFLX
L02680  ! ----------------------------------------------------------------------
L02681  ! CALCULATE SOIL MOISTURE FLUX.  THE SOIL MOISTURE CONTENT (SMC - A PER
L02682  ! UNIT VOLUME MEASUREMENT) IS A DEPENDENT VARIABLE THAT IS UPDATED WITH
L02683  ! PROGNOSTIC EQNS. THE CANOPY MOISTURE CONTENT (CMC) IS ALSO UPDATED.
L02684  ! FROZEN GROUND VERSION:  NEW STATES ADDED: SH2O, AND FROZEN GROUND
L02685  ! CORRECTION FACTOR, FRZFACT AND PARAMETER SLOPE.
L02686  ! ----------------------------------------------------------------------
L02687         IMPLICIT NONE
L02688
L02689         INTEGER, INTENT(IN)   :: NSOIL
L02690         INTEGER               :: I,K
L02691
L02692         REAL, INTENT(IN)      :: BEXP, CMCMAX, DKSAT,DWSAT, DT, EC, EDIR,  &
L02693                                  KDT, PRCP1, SHDFAC, SLOPE, SMCMAX, SMCWLT
L02694         REAL, INTENT(OUT)                     :: DRIP, RUNOFF1, RUNOFF2, RUNOFF3
L02695         REAL, INTENT(INOUT)   :: CMC
L02696         REAL, DIMENSION(1:NSOIL), INTENT(IN)   :: ET,ZSOIL
L02697         REAL, DIMENSION(1:NSOIL), INTENT(INOUT):: SMC, SH2O
L02698         REAL, DIMENSION(1:NSOIL)             :: AI, BI, CI, STCF,RHSTS, RHSTT, &
L02699                                                 SICE, SH2OA, SH2OFG, SH2OIN
L02700         REAL                  :: DUMMY, EXCESS,FRZFACT,PCPDRP,RHSCT,TRHSCT
L02701         REAL :: FAC2
L02702         REAL :: FLIMIT
L02703         REAL, INTENT(IN),OPTIONAL      :: IRRIGATION_CHANNEL
L02704         REAL,    INTENT(INOUT)                :: SFHEAD1RT,INFXS1RT
L02705
L02706  ! ----------------------------------------------------------------------
L02707  ! EXECUTABLE CODE BEGINS HERE.
L02708  ! ----------------------------------------------------------------------
L02709  ! ----------------------------------------------------------------------
L02710  ! COMPUTE THE RIGHT HAND SIDE OF THE CANOPY EQN TERM ( RHSCT )
L02711  ! ----------------------------------------------------------------------
L02712         DUMMY = 0.
L02713
L02714  ! ----------------------------------------------------------------------
L02715  ! CONVERT RHSCT (A RATE) TO TRHSCT (AN AMOUNT) AND ADD IT TO EXISTING
L02716  ! CMC.  IF RESULTING AMT EXCEEDS MAX CAPACITY, IT BECOMES DRIP AND WILL
L02717  ! FALL TO THE GRND.
L02718  ! ----------------------------------------------------------------------
L02719         RHSCT = SHDFAC * PRCP1- EC
L02720         DRIP = 0.
L02721         TRHSCT = DT * RHSCT
L02722         EXCESS = CMC + TRHSCT
L02723
L02724  ! ----------------------------------------------------------------------
L02725  ! PCPDRP IS THE COMBINED PRCP1 AND DRIP (FROM CMC) THAT GOES INTO THE
L02726  ! SOIL
L02727  ! ----------------------------------------------------------------------
L02728         IF (EXCESS > CMCMAX) DRIP = EXCESS - CMCMAX
L02729         PCPDRP = (1. - SHDFAC) * PRCP1+ DRIP / DT
```

```
L02730        IF(PRESENT(IRRIGATION_CHANNEL)) THEN
L02731           IF (IRRIGATION_CHANNEL.NE.0.)PCPDRP =PCPDRP+ 0.001*IRRIGATION_CHANNEL !conversion of units
L02732        END IF
L02733  ! ---------------------------------------------------------------------
L02734  ! STORE ICE CONTENT AT EACH SOIL LAYER BEFORE CALLING SRT and SSTEP
L02735  !
L02736        DO I = 1,NSOIL
L02737           SICE (I) = SMC (I) - SH2O (I)
L02738        END DO
L02739  ! ---------------------------------------------------------------------
L02740  ! CALL SUBROUTINES SRT AND SSTEP TO SOLVE THE SOIL MOISTURE
L02741  ! TENDENCY EQUATIONS.
L02742  ! IF THE INFILTRATING PRECIP RATE IS NONTRIVIAL,
L02743  !   (WE CONSIDER NONTRIVIAL TO BE A PRECIP TOTAL OVER THE TIME STEP
L02744  !    EXCEEDING ONE ONE-THOUSANDTH OF THE WATER HOLDING CAPACITY OF
L02745  !    THE FIRST SOIL LAYER)
L02746  ! THEN CALL THE SRT/SSTEP SUBROUTINE PAIR TWICE IN THE MANNER OF
L02747  !   TIME SCHEME "F" (IMPLICIT STATE, AVERAGED COEFFICIENT)
L02748  !   OF SECTION 2 OF KALNAY AND KANAMITSU (1988, MWR, VOL 116,
L02749  !   PAGES 1945-1958)TO MINIMIZE 2-DELTA-T OSCILLATIONS IN THE
L02750  !   SOIL MOISTURE VALUE OF THE TOP SOIL LAYER THAT CAN ARISE BECAUSE
L02751  !   OF THE EXTREME NONLINEAR DEPENDENCE OF THE SOIL HYDRAULIC
L02752  !   DIFFUSIVITY COEFFICIENT AND THE HYDRAULIC CONDUCTIVITY ON THE
L02753  !   SOIL MOISTURE STATE
L02754  ! OTHERWISE CALL THE SRT/SSTEP SUBROUTINE PAIR ONCE IN THE MANNER OF
L02755  !   TIME SCHEME "D" (IMPLICIT STATE, EXPLICIT COEFFICIENT)
L02756  !   OF SECTION 2 OF KALNAY AND KANAMITSU
L02757  ! PCPDRP IS UNITS OF KG/M**2/S OR MM/S, ZSOIL IS NEGATIVE DEPTH IN M
L02758  ! ---------------------------------------------------------------------
L02759  !  According to Dr. Ken Mitchell's suggestion, add the second contraint
L02760  !  to remove numerical instability of runoff and soil moisture
L02761  !  FLIMIT is a limit value for FAC2
L02762        FAC2=0.0
L02763        DO I=1,NSOIL
L02764           FAC2=MAX(FAC2,SH2O(I)/SMCMAX)
L02765        ENDDO
L02766        CALL FAC2MIT(SMCMAX,FLIMIT)
L02767
L02768  ! ---------------------------------------------------------------------
L02769  ! FROZEN GROUND VERSION:
L02770  ! SMC STATES REPLACED BY SH2O STATES IN SRT SUBR.  SH2O & SICE STATES
L02771  ! INC&UDED IN SSTEP SUBR.  FROZEN GROUND CORRECTION FACTOR, FRZFACT
L02772  ! ADDED.  ALL WATER BALANCE CALCULATIONS USING UNFROZEN WATER
L02773  ! ---------------------------------------------------------------------
L02774
L02775  #ifdef WRF_HYDRO
L02776  !DJG NDHMS/WRF-Hydro edit... Add previous ponded water to new precip drip...
L02777      PCPDRP = PCPDRP + SFHEAD1RT/1000./DT   ! convert SFHEAD1RT to (m/s)
L02778  #endif
L02779
L02780
L02781        IF ( ( (PCPDRP * DT) > (0.0001*1000.0* (- ZSOIL (1))* SMCMAX) )   &
L02782             .OR. (FAC2 > FLIMIT) ) THEN
L02783           CALL SRT (RHSTT,EDIR,ET,SH2O,SH2O,NSOIL,PCPDRP,ZSOIL,           &
L02784                     DWSAT,DKSAT,SMCMAX,BEXP,RUNOFF1,                      &
L02785                     RUNOFF2,DT,SMCWLT,SLOPE,KDT,FRZFACT,SICE,AI,BI,CI,  &
L02786                     SFHEAD1RT,INFXS1RT)
L02787           CALL SSTEP (SH2OFG,SH2O,DUMMY,RHSTT,RHSCT,DT,NSOIL,SMCMAX,     &
L02788                       CMCMAX,RUNOFF3,ZSOIL,SMC,SICE,AI,BI,CI,INFXS1RT)
L02789           DO K = 1,NSOIL
L02790              SH2OA (K) = (SH2O (K) + SH2OFG (K)) * 0.5
L02791           END DO
L02792           CALL SRT (RHSTT,EDIR,ET,SH2O,SH2OA,NSOIL,PCPDRP,ZSOIL,          &
L02793                     DWSAT,DKSAT,SMCMAX,BEXP,RUNOFF1,                      &
L02794                     RUNOFF2,DT,SMCWLT,SLOPE,KDT,FRZFACT,SICE,AI,BI,CI,  &
L02795                     SFHEAD1RT,INFXS1RT)
L02796           SH2OIN=SH2O
L02797           CALL SSTEP (SH2O,SH2OIN,CMC,RHSTT,RHSCT,DT,NSOIL,SMCMAX,        &
L02798                       CMCMAX,RUNOFF3,ZSOIL,SMC,SICE,AI,BI,CI,INFXS1RT)
L02799
L02800        ELSE
L02801           CALL SRT (RHSTT,EDIR,ET,SH2O,SH2O,NSOIL,PCPDRP,ZSOIL,           &
L02802                     DWSAT,DKSAT,SMCMAX,BEXP,RUNOFF1,                      &
L02803                     RUNOFF2,DT,SMCWLT,SLOPE,KDT,FRZFACT,SICE,AI,BI,CI,  &
L02804                    SFHEAD1RT,INFXS1RT)
L02805           SH2OIN=SH2O
L02806           CALL SSTEP (SH2O,SH2OIN,CMC,RHSTT,RHSCT,DT,NSOIL,SMCMAX,        &
L02807                       CMCMAX,RUNOFF3,ZSOIL,SMC,SICE,AI,BI,CI,INFXS1RT)
L02808  !      RUNOF = RUNOFF
L02809
L02810        END IF
L02811
L02812  ! ---------------------------------------------------------------------
L02813    END SUBROUTINE SMFLX
```

讲解：① **冠层水收支**：RHSCT=SHDFAC • PRCP1−EC（植被截留降雨减冠层蒸发）；超 CMCMAX 的部分变 DRIP 落地；**PCPDRP=(1−SHDFAC) · PRCP1+DRIP/DT**——裸地直通雨加冠层滴下，才是真正进土壤的降水。② **灌溉通道**：PRESENT(IRRIGATION_CHANNEL) 时 PCPDRP 加 0.001 • IRRIGATION_CHANNEL（mm 量到 m/s 的换算）——论文 2 类灌溉实验的物理挂点在 SMFLX、驱动挂点在 noahdrv。③WRF_HYDRO 下积水头并入 PCPDRP。④ **双约束双步长**（Kalnay–Kanamitsu 1988 MWR 116, 1945）：FAC2=各层 SH2O/SMCMAX 最大值；当 PCPDRP • DT 超过第一层持水量的万分之一或 FAC2>FLIMIT（FAC2MIT，12a § 六）时走「F 方案」——先算一遍 SRT/SSTEP 得试探态 SH2OFG、平均出 SH2OA、用 SH2OA 重算 SRT 再真正推进（隐式态 +平均系数）；否则「D 方案」单步（隐式态 +显式系数）。目的是抑制水力扩散率/导水率对土壤湿度的极端非线性在表层引起的 2Δt 振荡。⑤ 冻土版全程用未冻水 SH2O，SICE 冻结前快照。

七、雪盖分数 SNFRAC 与相变源 SNKSRC

```
!file: phys/module_sf_noahlsm.F    L2817-L2919    (subroutine SNFRAC)
L02817        SUBROUTINE SNFRAC (SNEQV,SNUP,SALP,SNOWH,SNCOVR, &
L02818                           XLAI,SHDFAC,FVB,GAMA,FBUR,    &
L02819                           FGSN,ZTOPV,ZBOTV,UA_PHYS)
L02820
L02821  ! ----------------------------------------------------------------------
L02822  ! SUBROUTINE SNFRAC
L02823  ! ----------------------------------------------------------------------
L02824  ! CALCULATE SNOW FRACTION (0 -> 1)
L02825  ! SNEQV   SNOW WATER EQUIVALENT (M)
L02826  ! SNUP    THRESHOLD SNEQV DEPTH ABOVE WHICH SNCOVR=1
L02827  ! SALP    TUNING PARAMETER
L02828  ! SNCOVR  FRACTIONAL SNOW COVER
L02829  ! ----------------------------------------------------------------------
L02830        IMPLICIT NONE
L02831
L02832        REAL, INTENT(IN)     :: SNEQV,SNUP,SALP,SNOWH
L02833        REAL, INTENT(OUT)    :: SNCOVR
L02834        REAL                 :: RSNOW, Z0N
L02835        LOGICAL, INTENT(IN)  :: UA_PHYS  ! UA: flag for UA option
L02836        REAL, INTENT(IN)     :: ZTOPV    ! UA: height of canopy top
L02837        REAL, INTENT(IN)     :: ZBOTV    ! UA: height of canopy bottom
L02838        REAL, INTENT(IN)     :: SHDFAC   ! UA: vegetation fraction
L02839        REAL, INTENT(INOUT)  :: XLAI     ! UA: LAI modified by snow
L02840        REAL, INTENT(OUT)    :: FVB      ! UA: frac. veg. w/snow beneath
L02841        REAL, INTENT(OUT)    :: GAMA     ! UA: = EXP(-1.* XLAI)
L02842        REAL, INTENT(OUT)    :: FBUR     ! UA: fraction of canopy buried
L02843        REAL, INTENT(OUT)    :: FGSN     ! UA: ground snow cover fraction
L02844
L02845        REAL ::  SNUPGRD = 0.02          ! UA: SWE limit for ground cover
L02846
L02847  ! ----------------------------------------------------------------------
L02848  ! SNUP IS VEG-CLASS DEPENDENT SNOWDEPTH THRESHHOLD (SET IN ROUTINE
L02849  ! REDPRM) ABOVE WHICH SNOCVR=1.
L02850  ! ----------------------------------------------------------------------
L02851        IF (SNEQV < SNUP) THEN
L02852           RSNOW = SNEQV / SNUP
L02853           SNCOVR = 1. - ( EXP ( - SALP * RSNOW) - RSNOW * EXP ( - SALP))
L02854        ELSE
L02855           SNCOVR = 1.0
L02856        END IF
L02857
L02858  !     FORMULATION OF DICKINSON ET AL. 1986
L02859  !     Z0N = 0.035
L02860
L02861  !        SNCOVR=SNOWH/(SNOWH + 5*Z0N)
L02862
L02863  !     FORMULATION OF MARSHALL ET AL. 1994
L02864  !        SNCOVR=SNEQV/(SNEQV + 2*Z0N)
L02865
L02866        IF(UA_PHYS) THEN
L02867
L02868  !-----------------------------------------------------------------
L02869  ! FGSN: FRACTION OF SOIL COVERED WITH SNOW
L02870  !-----------------------------------------------------------------
L02871         IF (SNEQV < SNUPGRD) THEN
L02872          FGSN = SNEQV / SNUPGRD
L02873         ELSE
L02874          FGSN = 1.0
L02875         END IF
L02876  !-----------------------------------------------------------------
L02877  ! FBUR: VERTICAL FRACTION OF VEGETATION COVERED BY SNOW
L02878  ! GRASS, CROP, AND SHRUB: MULTIPLY 0.4 BY ZTOPV AND ZBOTV BECAUSE
L02879  ! THEY WILL BE PRESSED DOWN BY THE SNOW.
L02880  ! FOREST: DON'T NEED TO CHANGE ZTOPV AND ZBOTV.
L02881
L02882         IF(ZBOTV > 0. .AND. SNOWH > ZBOTV) THEN
L02883           IF(ZBOTV <= 0.5) THEN
L02884             FBUR = (SNOWH - 0.4*ZBOTV) / (0.4*(ZTOPV-ZBOTV)) ! short veg.
L02885           ELSE
L02886             FBUR = (SNOWH - ZBOTV) / (ZTOPV-ZBOTV)           !  tall veg.
L02887           ENDIF
L02888         ELSE
L02889           FBUR = 0.
L02890         ENDIF
L02891
L02892         FBUR = MIN(MAX(FBUR,0.0),1.0)
L02893
L02894  ! XLAI IS ADJUSTED FOR VERTICAL BURYING BY SNOW
L02895         XLAI = XLAI * (1.0 - FBUR)
L02896  ! ----------------------------------------------------------------------
L02897  ! SNOW-COVERED SOIL: (1-SHDFAC)*FGSN
L02898  ! VEGETATION WITH SNOW ABOVE DUE TO BURIAL FVEG_SN_AB = SHDFAC*FBUR
L02899  ! SNOW ON THE GROUND THAT CAN BE "SEEN" BY SATELLITE
L02900  ! (IF XLAI GOES TO ZERO): GAMA*FVB
L02901  ! Where GAMA = exp(-XLAI)
L02902  ! ----------------------------------------------------------------------
L02903
L02904  ! VEGETATION WITH SNOW BELOW
L02905         FVB = SHDFAC * FGSN * (1.0 - FBUR)
L02906
L02907  ! GAMA IS USED TO DIVIDE FVB INTO TWO PARTS:
L02908  ! GAMA=1 FOR XLAI=0 AND GAMA=0 FOR XLAI=6
L02909         GAMA = EXP(-1.* XLAI)
L02910        ELSE
L02911          ! Define intent(out) terms for .NOT. UA_PHYS case
L02912          FVB  = 0.0
L02913          GAMA = 0.0
L02914          FBUR = 0.0
L02915          FGSN = 0.0
L02916        END IF    ! UA_PHYS
L02917
L02918  ! ----------------------------------------------------------------------
L02919    END SUBROUTINE SNFRAC
```

讲解：雪盖分数的 SALP 曲线：RSNOW=SNEQV/SNUP（SNUP 是植被类雪深阈值），SNCOVR=1−(exp(−SALP • RSNOW)−RSNOW • exp(−SALP))，SNEQV≥SNUP 时全盖（SALP 来自 GENPARM.TBL）；Dickinson 1986 与 Marshall 1994 备选式注释保留。**UA 分支**输出四件套：FGSN 地面积雪分数（0.02 m 水当量阈值）；FBUR 植被被雪掩埋的垂直分数（矮植被把冠层顶底高乘 0.4 模拟压弯，高植被不变）；XLAI 乘 (1−FBUR)（被埋的叶子不蒸腾）；FVB=SHDFAC • FGSN • (1−FBUR)、GAMA=exp(−XLAI)——「卫星视角能看到的雪」的分解式。这套量只在 UA 物理链（SNOWZ0/PENMAN/SFLX 的 RU 修正）里消费。

```
!file: phys/module_sf_noahlsm.F    L2922-L3007    (subroutine SNKSRC)
L02922        SUBROUTINE SNKSRC (TSNSR,TAVG,SMC,SH2O,ZSOIL,NSOIL,              &
L02923       &                         SMCMAX,PSISAT,BEXP,DT,K,QTOT)
L02924    ! ----------------------------------------------------------------------
L02925    ! SUBROUTINE SNKSRC
L02926    ! ----------------------------------------------------------------------
L02927    ! CALCULATE SINK/SOURCE TERM OF THE TERMAL DIFFUSION EQUATION. (SH2O) IS
L02928    ! AVAILABLE LIQUED WATER.
L02929    ! ----------------------------------------------------------------------
L02930        IMPLICIT NONE
L02931
L02932        INTEGER, INTENT(IN)   :: K,NSOIL
L02933        REAL, INTENT(IN)      :: BEXP, DT, PSISAT, QTOT, SMC, SMCMAX,    &
L02934                                 TAVG
L02935        REAL, INTENT(INOUT)   :: SH2O
L02936
L02937        REAL, DIMENSION(1:NSOIL), INTENT(IN):: ZSOIL
L02938
L02939        REAL                  :: DF, DZ, DZH, FREE, TSNSR,               &
L02940                                 TDN, TM, TUP, TZ, X0, XDN, XH2O, XUP
L02941
L02942        REAL, PARAMETER       :: DH2O = 1.0000E3, HLICE = 3.3350E5,      &
L02943                                 T0 = 2.7315E2
L02944
L02945        IF (K == 1) THEN
L02946           DZ = - ZSOIL (1)
L02947        ELSE
L02948           DZ = ZSOIL (K -1) - ZSOIL (K)
L02949        END IF
L02950    ! ----------------------------------------------------------------------
L02951    ! VIA FUNCTION FRH2O, COMPUTE POTENTIAL OR 'EQUILIBRIUM' UNFROZEN
L02952    ! SUPERCOOLED FREE WATER FOR GIVEN SOIL TYPE AND SOIL LAYER TEMPERATURE.
L02953    ! FUNCTION FRH20 INVOKES EQN (17) FROM V. KOREN ET AL (1999, JGR, VOL.
L02954    ! 104, PG 19573).  (ASIDE:  LATTER EQN IN JOURNAL IN CENTIGRADE UNITS.
L02955    ! ROUTINE FRH2O USE FORM OF EQN IN KELVIN UNITS.)
L02956    ! ----------------------------------------------------------------------
L02957    !      FREE = FRH2O(TAVG,SMC,SH2O,SMCMAX,BEXP,PSISAT)
L02958
L02959    ! ----------------------------------------------------------------------
L02960    ! IN NEXT BLOCK OF CODE, INVOKE EQN 18 OF V. KOREN ET AL (1999, JGR,
L02961    ! VOL. 104, PG 19573.)  THAT IS, FIRST ESTIMATE THE NEW AMOUNTOF LIQUID
L02962    ! WATER, 'XH2O', IMPLIED BY THE SUM OF (1) THE LIQUID WATER AT THE BEGIN
L02963    ! OF CURRENT TIME STEP, AND (2) THE FREEZE OF THAW CHANGE IN LIQUID
L02964    ! WATER IMPLIED BY THE HEAT FLUX 'QTOT' PASSED IN FROM ROUTINE HRT.
L02965    ! SECOND, DETERMINE IF XH2O NEEDS TO BE BOUNDED BY 'FREE' (EQUIL AMT) OR
L02966    ! IF 'FREE' NEEDS TO BE BOUNDED BY XH2O.
L02967    ! ----------------------------------------------------------------------
L02968        CALL FRH2O (FREE,TAVG,SMC,SH2O,SMCMAX,BEXP,PSISAT)
L02969
L02970    ! ----------------------------------------------------------------------
L02971    ! FIRST, IF FREEZING AND REMAINING LIQUID LESS THAN LOWER BOUND, THEN
L02972    ! REDUCE EXTENT OF FREEZING, THEREBY LETTING SOME OR ALL OF HEAT FLUX
L02973    ! QTOT COOL THE SOIL TEMP LATER IN ROUTINE HRT.
L02974    ! ----------------------------------------------------------------------
L02975        XH2O = SH2O + QTOT * DT / (DH2O * HLICE * DZ)
L02976        IF ( XH2O < SH2O .AND. XH2O < FREE) THEN
L02977           IF ( FREE > SH2O ) THEN
L02978              XH2O = SH2O
L02979           ELSE
L02980              XH2O = FREE
L02981           END IF
L02982        END IF
L02983    ! ----------------------------------------------------------------------
L02984    ! SECOND, IF THAWING AND THE INCREASE IN LIQUID WATER GREATER THAN UPPER
L02985    ! BOUND, THEN REDUCE EXTENT OF THAW, THEREBY LETTING SOME OR ALL OF HEAT
L02986    ! FLUX QTOT WARM THE SOIL TEMP LATER IN ROUTINE HRT.
L02987    ! ----------------------------------------------------------------------
L02988        IF ( XH2O > SH2O .AND. XH2O > FREE ) THEN
L02989           IF ( FREE < SH2O ) THEN
L02990              XH2O = SH2O
L02991           ELSE
L02992              XH2O = FREE
L02993           END IF
L02994        END IF
L02995
L02996    ! ----------------------------------------------------------------------
L02997    ! CALCULATE PHASE-CHANGE HEAT SOURCE/SINK TERM FOR USE IN ROUTINE HRT
L02998    ! AND UPDATE LIQUID WATER TO REFLCET FINAL FREEZE/THAW INCREMENT.
L02999    ! ----------------------------------------------------------------------
L03000    !      SNKSRC = -DH2O*HLICE*DZ*(XH2O-SH2O)/DT
L03001        IF (XH2O < 0.) XH2O = 0.
L03002        IF (XH2O > SMC) XH2O = SMC
L03003        TSNSR = - DH2O * HLICE * DZ * (XH2O - SH2O)/ DT
L03004        SH2O = XH2O
L03005
L03006    ! ----------------------------------------------------------------------
L03007      END SUBROUTINE SNKSRC
```

讲解：冻土相变源项，Koren 1999 **式 18** 的钳位实现：①FREE=FRH2O(TAVG,…)——当前层温下的平衡液态水（12a § 七）；②XH2O=SH2O+QTOT • DT/(DH2O • HLICE • DZ)——假设热通量 QTOT 全部用于相变时的液态水增量；③ 双向钳位：冻结方向 XH2O<SH2O 且 XH2O<FREE 时取 SH2O/FREE 中较大者（不能冻过头）、融化方向对称（不能融过头）；④**TSNSR=−DH2O · HLICE · DZ ·**

(XH2O−SH2O)/DT——实际发生的相变潜热作为源项顶回能量方程（HRT 里从 RHSTS 扣除），SH2O 原地更新并钳位 [0,SMC]。能量守恒的「先相变、剩余降温」就由这一对钳位 +源项回加完成。

八、雪盖路径 SNOPAC（全文）

```
!file: phys/module_sf_noahlsm.F    L3010-L3413    (subroutine SNOPAC)
L03010        SUBROUTINE SNOPAC (ETP,ETA,PRCP,PRCPF,SNOWNG,SMC,SMCMAX,SMCWLT,   &
L03011                           SMCREF,SMCDRY,CMC,CMCMAX,NSOIL,DT,             &
L03012                           SBETA,DF1,                                     &
L03013                           Q2,T1,SFCTMP,T24,TH2,FDOWN,F1,SSOIL,STC,EPSCA,&
L03014                          SFCPRS,BEXP,PC,RCH,RR,CFACTR,SNCOVR,ESD,SNDENS,&
L03015                           SNOWH,SH2O,SLOPE,KDT,FRZFACT,PSISAT,           &
L03016                           ZSOIL,DWSAT,DKSAT,TBOT,ZBOT,SHDFAC,RUNOFF1,    &
L03017                           RUNOFF2,RUNOFF3,EDIR,EC,ET,ETT,NROOT,SNOMLT,   &
L03018                           RTDIS,QUARTZ,FXEXP,CSOIL,                      &
L03019                           BETA,DRIP,DEW,FLX1,FLX2,FLX3,ESNOW,ETNS,EMISSI,&
L03020                           RIBB,SOLDN,                                    &
L03021                           ISURBAN,                                       &
L03022                           VEGTYP,                                        &
L03023                           ETPN,FLX4,UA_PHYS,                             &
L03024                           SFHEAD1RT,INFXS1RT,ETPND1,SOILTYP,OPT_THCND    &
L03025                          ,QFX_PHY,fasdas,HCPCT_FASDAS                    ) !fasdas
L03026  ! ----------------------------------------------------------------------
L03027  ! SUBROUTINE SNOPAC
L03028  ! ----------------------------------------------------------------------
L03029  ! CALCULATE SOIL MOISTURE AND HEAT FLUX VALUES & UPDATE SOIL MOISTURE
L03030  ! CONTENT AND SOIL HEAT CONTENT VALUES FOR THE CASE WHEN A SNOW PACK IS
L03031  ! PRESENT.
L03032  ! ----------------------------------------------------------------------
L03033        IMPLICIT NONE
L03034
L03035        INTEGER, INTENT(IN)   :: OPT_THCND
L03036        INTEGER, INTENT(IN)   :: NROOT, NSOIL,VEGTYP,SOILTYP
L03037        INTEGER, INTENT(IN)   :: ISURBAN
L03038        INTEGER               :: K
L03039  !
L03040  ! kmh 09/03/2006 add IT16 for surface temperature iteration
L03041  !
L03042        INTEGER               :: IT16
L03043        LOGICAL, INTENT(IN)   :: SNOWNG
L03044
L03045  !DJG NDHMS/WRF-Hydro edit...
L03046         REAL, INTENT(INOUT)    ::  SFHEAD1RT,INFXS1RT,ETPND1
L03047
L03048        REAL, INTENT(IN)      :: BEXP,CFACTR, CMCMAX,CSOIL,DF1,DKSAT,     &
L03049                                 DT,DWSAT, EPSCA,FDOWN,F1,FXEXP,          &
L03050                                 FRZFACT,KDT,PC, PRCP,PSISAT,Q2,QUARTZ,   &
L03051                                 RCH,RR,SBETA,SFCPRS, SFCTMP, SHDFAC,     &
L03052                                 SLOPE,SMCDRY,SMCMAX,SMCREF,SMCWLT, T24,  &
L03053                                 TBOT,TH2,ZBOT,EMISSI,SOLDN
L03054        REAL, INTENT(INOUT)   :: CMC, BETA, ESD,FLX2,PRCPF,SNOWH,SNCOVR,  &
L03055                                 SNDENS, T1, RIBB, ETP
L03056        REAL, INTENT(OUT)     :: DEW,DRIP,EC,EDIR, ETNS, ESNOW,ETT,       &
L03057                                 FLX1,FLX3, RUNOFF1,RUNOFF2,RUNOFF3,      &
L03058                                 SSOIL,SNOMLT
L03059        REAL, DIMENSION(1:NSOIL),INTENT(IN)     :: RTDIS,ZSOIL
L03060        REAL, DIMENSION(1:NSOIL),INTENT(OUT)    :: ET
L03061        REAL, DIMENSION(1:NSOIL), INTENT(INOUT) :: SMC,SH2O,STC
L03062        REAL, DIMENSION(1:NSOIL) :: ET1
L03063        REAL                  :: DENOM,DSOIL,DTOT,EC1,EDIR1,ESDFLX,ETA,   &
L03064                                 ETT1, ESNOW1, ESNOW2, ETA1,ETP1,ETP2,    &
L03065                                 ETP3, ETNS1, ETANRG, ETAX, EX, FLX3X,    &
L03066                                 FRCSNO,FRCSOI, PRCP1, QSAT,RSNOW, SEH,   &
L03067                                 SNCOND,SSOIL1, T11,T12, T12A, T12AX,     &
L03068                                 T12B, T14, YY, ZZ1
L03069  !                               T12B, T14, YY, ZZ1,EMISSI_S
L03070  !
L03071  ! kmh 01/11/2007 add T15, T16, and DTOT2 for SFC T iteration and snow heat flux
L03072  !
L03073        REAL                  :: T15, T16, DTOT2
L03074        REAL, PARAMETER       :: ESDMIN = 1.E-6, LSUBC = 2.501000E+6,     &
L03075                                 LSUBS = 2.83E+6, TFREEZ = 273.15,        &
L03076                                 SNOEXP = 2.0
L03077        LOGICAL, INTENT(IN)   :: UA_PHYS  ! UA: flag for UA option
L03078        REAL, INTENT(INOUT)   :: FLX4     ! UA: energy removed by canopy
L03079        REAL, INTENT(IN)      :: ETPN     ! UA: adjusted pot. evap. [mm/s]
L03080        REAL                  :: ETP1N    ! UA: adjusted pot. evap. [m/s]
L03081
L03082  !
L03083  !  FASDAS
L03084  !
L03085        REAL                  :: QFX_PHY
L03086        INTEGER               :: fasdas
L03087        REAL, INTENT(  OUT)   :: HCPCT_FASDAS
L03088  !
L03089  !  END FASDAS
L03090  !
L03091  ! ----------------------------------------------------------------------
L03092  ! EXECUTABLE CODE BEGINS HERE:
L03093  ! ----------------------------------------------------------------------
L03094  ! ----------------------------------------------------------------------
L03095  ! INITIALIZE EVAP TERMS.
L03096  ! ----------------------------------------------------------------------
L03097  ! conversions:
L03098  ! ESNOW [KG M-2 S-1]
L03099  ! ESDFLX [KG M-2 S-1] .le. ESNOW
L03100  ! ESNOW1 [M S-1]
L03101  ! ESNOW2 [M]
L03102  ! ETP [KG M-2 S-1]
L03103  ! ETP1 [M S-1]
L03104  ! ETP2 [M]
L03105  ! ----------------------------------------------------------------------
```

```
L03106        DEW = 0.
L03107        EDIR = 0.
L03108        EDIR1 = 0.
L03109        EC1 = 0.
L03110        EC = 0.
L03111  !      EMISSI_S=0.95    ! For snow
L03112
L03113        DO K = 1,NSOIL
L03114           ET (K) = 0.
L03115           ET1 (K) = 0.
L03116        END DO
L03117        ETT = 0.
L03118        ETT1 = 0.
L03119
L03120  !DJG NDHMS/WRF-Hydro edit...
L03121        ETPND1 = 0.
L03122
L03123
L03124        ETNS = 0.
L03125        ETNS1 = 0.
L03126        ESNOW = 0.
L03127        ESNOW1 = 0.
L03128        ESNOW2 = 0.
L03129
L03130  ! ---------------------------------------------------------------------
L03131  ! CONVERT POTENTIAL EVAP (ETP) FROM KG M-2 S-1 TO ETP1 IN M S-1
L03132  ! ---------------------------------------------------------------------
L03133        PRCP1 = PRCPF *0.001
L03134  ! ---------------------------------------------------------------------
L03135  ! IF ETP<0 (DOWNWARD) THEN DEWFALL (=FROSTFALL IN THIS CASE).
L03136  ! ---------------------------------------------------------------------
L03137        BETA = 1.0
L03138        IF (ETP <= 0.0) THEN
L03139           IF ( ( RIBB >= 0.1 ) .AND. ( FDOWN > 150.0 ) ) THEN
L03140              ETP=(MIN(ETP*(1.0-RIBB),0.)*SNCOVR/0.980 + ETP*(0.980-SNCOVR))/0.980
L03141           ENDIF
L03142           IF(ETP == 0.) BETA = 0.0
L03143           ETP1 = ETP * 0.001
L03144           IF(UA_PHYS) ETP1N = ETPN * 0.001
L03145           DEW = -ETP1
L03146           ESNOW2 = ETP1*DT
L03147           ETANRG = ETP*((1.-SNCOVR)*LSUBC + SNCOVR*LSUBS)
L03148        ELSE
L03149           ETP1 = ETP * 0.001
L03150           IF(UA_PHYS) ETP1N = ETPN * 0.001
L03151           !  LAND CASE
L03152           IF (SNCOVR <  1.) THEN
L03153              CALL EVAPO (ETNS1,SMC,NSOIL,CMC,ETP1,DT,ZSOIL,           &
L03154                          SH2O,                                        &
L03155                          SMCMAX,BEXP,PC,SMCWLT,DKSAT,DWSAT,           &
L03156                          SMCREF,SHDFAC,CMCMAX,                        &
L03157                          SMCDRY,CFACTR,                               &
L03158                          EDIR1,EC1,ET1,ETT1,SFCTMP,Q2,NROOT,RTDIS,    &
L03159                          FXEXP, SFHEAD1RT,ETPND1)
L03160  ! ---------------------------------------------------------------------
L03161              EDIR1 = EDIR1* (1. - SNCOVR)
L03162              EC1 = EC1* (1. - SNCOVR)
L03163              DO K = 1,NSOIL
L03164                 ET1 (K) = ET1 (K)* (1. - SNCOVR)
L03165              END DO
L03166              ETT1 = ETT1*(1.-SNCOVR)
L03167  !            ETNS1 = EDIR1+ EC1+ ETT1
L03168              ETNS1 = ETNS1*(1.-SNCOVR)
L03169  ! ---------------------------------------------------------------------
L03170              EDIR = EDIR1*1000.
L03171              EC = EC1*1000.
L03172              DO K = 1,NSOIL
L03173                 ET (K) = ET1 (K)*1000.
L03174              END DO
L03175  !
L03176  ! FASDAS
L03177  !
L03178              if( fasdas ==  1 ) then
L03179                QFX_PHY = EDIR + EC
L03180                DO K=1,NSOIL
L03181                   QFX_PHY = QFX_PHY + ET(K)
L03182                END DO
L03183              endif
L03184  !
L03185  ! END FASDAS
L03186  !
L03187              ETT = ETT1*1000.
L03188              ETNS = ETNS1*1000.
L03189
L03190
L03191  !DJG NDHMS/WRF-Hydro edit...
L03192                 ETPND1 = ETPND1*1000.
L03193
L03194
L03195  ! ---------------------------------------------------------------------
L03196
L03197           ENDIF
L03198           ESNOW = ETP*SNCOVR
L03199           IF(UA_PHYS) ESNOW = ETPN*SNCOVR   ! USE ADJUSTED ETP
L03200           ESNOW1 = ESNOW*0.001
L03201           ESNOW2 = ESNOW1*DT
L03202           ETANRG = ESNOW*LSUBS + ETNS*LSUBC
L03203        ENDIF
L03204
L03205  ! ---------------------------------------------------------------------
L03206  ! IF PRECIP IS FALLING, CALCULATE HEAT FLUX FROM SNOW SFC TO NEWLY
L03207  ! ACCUMULATING PRECIP.  NOTE THAT THIS REFLECTS THE FLUX APPROPRIATE FOR
L03208  ! THE NOT-YET-UPDATED SKIN TEMPERATURE (T1).  ASSUMES TEMPERATURE OF THE
L03209  ! SNOWFALL STRIKING THE GROUND IS =SFCTMP (LOWEST MODEL LEVEL AIR TEMP).
L03210  ! ---------------------------------------------------------------------
L03211        FLX1 = 0.0
L03212        IF (SNOWNG) THEN
L03213           FLX1 = CPICE * PRCP * (T1- SFCTMP)
L03214        ELSE
L03215           IF (PRCP >  0.0) FLX1 = CPH2O * PRCP * (T1- SFCTMP)
```

```
L03216  ! ------------------------------------------------------------------
L03217  ! CALCULATE AN 'EFFECTIVE SNOW-GRND SFC TEMP' (T12) BASED ON HEAT FLUXES
L03218  ! BETWEEN THE SNOW PACK AND THE SOIL AND ON NET RADIATION.
L03219  ! INCLUDE FLX1 (PRECIP-SNOW SFC) AND FLX2 (FREEZING RAIN LATENT HEAT)
L03220  ! FLUXES.  FLX1 FROM ABOVE, FLX2 BROUGHT IN VIA COMMOM BLOCK RITE.
L03221  ! FLX2 REFLECTS FREEZING RAIN LATENT HEAT FLUX USING T1 CALCULATED IN
L03222  ! PENMAN.
L03223  ! ------------------------------------------------------------------
L03224        END IF
L03225        DSOIL = - (0.5 * ZSOIL (1))
L03226        DTOT = SNOWH + DSOIL
L03227        DENOM = 1.0+ DF1 / (DTOT * RR * RCH)
L03228  ! surface emissivity weighted by snow cover fraction
L03229  !       T12A = ( (FDOWN - FLX1 - FLX2 -                                 &
L03230  !     &       ((SNCOVR*EMISSI_S)+EMISSI*(1.0-SNCOVR))*SIGMA *T24)/RCH    &
L03231  !     &       + TH2 - SFCTMP - ETANRG/RCH ) / RR
L03232        T12A = ( (FDOWN - FLX1- FLX2- EMISSI * SIGMA * T24)/ RCH                  &
L03233                   + TH2- SFCTMP - ETANRG / RCH ) / RR
L03234
L03235        T12B = DF1 * STC (1) / (DTOT * RR * RCH)
L03236
L03237  ! ------------------------------------------------------------------
L03238  ! IF THE 'EFFECTIVE SNOW-GRND SFC TEMP' IS AT OR BELOW FREEZING, NO SNOW
L03239  ! MELT WILL OCCUR.  SET THE SKIN TEMP TO THIS EFFECTIVE TEMP.  REDUCE
L03240  ! (BY SUBLIMINATION ) OR INCREASE (BY FROST) THE DEPTH OF THE SNOWPACK,
L03241  ! DEPENDING ON SIGN OF ETP.
L03242  ! UPDATE SOIL HEAT FLUX (SSOIL) USING NEW SKIN TEMPERATURE (T1)
L03243  ! SINCE NO SNOWMELT, SET ACCUMULATED SNOWMELT TO ZERO, SET 'EFFECTIVE'
L03244  ! PRECIP FROM SNOWMELT TO ZERO, SET PHASE-CHANGE HEAT FLUX FROM SNOWMELT
L03245  ! TO ZERO.
L03246  ! ------------------------------------------------------------------
L03247  ! SUB-FREEZING BLOCK
L03248  ! ------------------------------------------------------------------
L03249        T12 = (SFCTMP + T12A + T12B) / DENOM
L03250        IF (T12 <=  TFREEZ) THEN
L03251           T1 = T12
L03252           SSOIL = DF1 * (T1- STC (1)) / DTOT
L03253  !        ESD = MAX (0.0, ESD- ETP2)
L03254           ESD = MAX(0.0, ESD-ESNOW2)
L03255           FLX3 = 0.0
L03256           EX = 0.0
L03257
L03258           SNOMLT = 0.0
L03259           IF(UA_PHYS) FLX4 = 0.0
L03260  ! ------------------------------------------------------------------
L03261  ! IF THE 'EFFECTIVE SNOW-GRND SFC TEMP' IS ABOVE FREEZING, SNOW MELT
L03262  ! WILL OCCUR.  CALL THE SNOW MELT RATE,EX AND AMT, SNOMLT.  REVISE THE
L03263  ! EFFECTIVE SNOW DEPTH.  REVISE THE SKIN TEMP BECAUSE IT WOULD HAVE CHGD
L03264  ! DUE TO THE LATENT HEAT RELEASED BY THE MELTING. CALC THE LATENT HEAT
L03265  ! RELEASED, FLX3. SET THE EFFECTIVE PRECIP, PRCP1 TO THE SNOW MELT RATE,
L03266  ! EX FOR USE IN SMFLX.  ADJUSTMENT TO T1 TO ACCOUNT FOR SNOW PATCHES.
L03267  ! CALCULATE QSAT VALID AT FREEZING POINT.  NOTE THAT ESAT (SATURATION
L03268  ! VAPOR PRESSURE) VALUE OF 6.11E+2 USED HERE IS THAT VALID AT FRZZING
L03269  ! POINT.  NOTE THAT ETP FROM CALL PENMAN IN SFLX IS IGNORED HERE IN
L03270  ! FAVOR OF BULK ETP OVER 'OPEN WATER' AT FREEZING TEMP.
L03271  ! UPDATE SOIL HEAT FLUX (S) USING NEW SKIN TEMPERATURE (T1)
L03272  ! ------------------------------------------------------------------
L03273  ! ABOVE FREEZING BLOCK
L03274  ! ------------------------------------------------------------------
L03275        ELSE
L03276  !     From V3.9 original code (commented) replaced to allow complete melting of small snow amounts
L03277  !        T1 = TFREEZ * SNCOVR ** SNOEXP + T12 * (1.0- SNCOVR ** SNOEXP)
L03278           T1 = TFREEZ * max(0.01,SNCOVR ** SNOEXP) + T12 * (1.0- max(0.01,SNCOVR ** SNOEXP))
L03279           BETA = 1.0
L03280
L03281  ! ------------------------------------------------------------------
L03282  ! IF POTENTIAL EVAP (SUBLIMATION) GREATER THAN DEPTH OF SNOWPACK.
L03283  ! BETA<1
L03284  ! SNOWPACK HAS SUBLIMATED AWAY, SET DEPTH TO ZERO.
L03285  ! ------------------------------------------------------------------
L03286           SSOIL = DF1 * (T1- STC (1)) / DTOT
L03287           IF (ESD-ESNOW2 <= ESDMIN) THEN
L03288              ESD = 0.0
L03289              EX = 0.0
L03290              SNOMLT = 0.0
L03291              FLX3 = 0.0
L03292              IF(UA_PHYS) FLX4 = 0.0
L03293  ! ------------------------------------------------------------------
L03294  ! SUBLIMATION LESS THAN DEPTH OF SNOWPACK
L03295  ! SNOWPACK (ESD) REDUCED BY ESNOW2 (DEPTH OF SUBLIMATED SNOW)
L03296  ! ------------------------------------------------------------------
L03297           ELSE
L03298              ESD = ESD-ESNOW2
L03299              ETP3 = ETP * LSUBC
L03300              SEH = RCH * (T1- TH2)
L03301              T14 = T1* T1
L03302              T14 = T14* T14
L03303  !           FLX3 = FDOWN - FLX1 - FLX2 -                                &
L03304  !                  ((SNCOVR*EMISSI_S)+EMISSI*(1-SNCOVR))*SIGMA*T14 -    &
L03305  !                    SSOIL - SEH - ETANRG
L03306              FLX3 = FDOWN - FLX1- FLX2- EMISSI*SIGMA * T14- SSOIL - SEH - ETANRG
L03307              IF (FLX3 <= 0.0) FLX3 = 0.0
L03308
L03309              IF(UA_PHYS .AND. FLX4 > 0. .AND. FLX3 > 0.) THEN
L03310                IF(FLX3 >= FLX4) THEN
L03311                  FLX3 = FLX3 - FLX4
L03312                ELSE
L03313                  FLX4 = FLX3
L03314                  FLX3 = 0.
L03315                ENDIF
L03316              ELSE
L03317                FLX4 = 0.0
L03318              ENDIF
L03319
L03320  ! ------------------------------------------------------------------
L03321  ! SNOWMELT REDUCTION DEPENDING ON SNOW COVER
L03322  ! ------------------------------------------------------------------
L03323              EX = FLX3*0.001/ LSUBF
L03324
```

```
L03325  ! ----------------------------------------------------------------------
L03326  ! ESDMIN REPRESENTS A SNOWPACK DEPTH THRESHOLD VALUE BELOW WHICH WE
L03327  ! CHOOSE NOT TO RETAIN ANY SNOWPACK, AND INSTEAD INCLUDE IT IN SNOWMELT.
L03328  ! ----------------------------------------------------------------------
L03329              SNOMLT = EX * DT
L03330              IF (ESD- SNOMLT >=  ESDMIN) THEN
L03331                 ESD = ESD- SNOMLT
L03332  ! ----------------------------------------------------------------------
L03333  ! SNOWMELT EXCEEDS SNOW DEPTH
L03334  ! ----------------------------------------------------------------------
L03335              ELSE
L03336                 EX = ESD / DT
L03337                 FLX3 = EX *1000.0* LSUBF
L03338                 SNOMLT = ESD
L03339
L03340                 ESD = 0.0
L03341  ! ----------------------------------------------------------------------
L03342  ! END OF 'ESD .LE. ETP2' IF-BLOCK
L03343  ! ----------------------------------------------------------------------
L03344              END IF
L03345           END IF
L03346
L03347  ! ----------------------------------------------------------------------
L03348  ! END OF 'T12 .LE. TFREEZ' IF-BLOCK
L03349  ! ----------------------------------------------------------------------
L03350  ! ----------------------------------------------------------------------
L03351  ! IF NON-GLACIAL LAND, ADD SNOWMELT RATE (EX) TO PRECIP RATE TO BE USED
L03352  ! IN SUBROUTINE SMFLX (SOIL MOISTURE EVOLUTION) VIA INFILTRATION.
L03353  !
L03354  ! RUNOFF/BASEFLOW LATER NEAR THE END OF SFLX (AFTER RETURN FROM CALL TO
L03355  ! SUBROUTINE SNOPAC)
L03356  ! ----------------------------------------------------------------------
L03357           PRCP1 = PRCP1+ EX
L03358
L03359  ! ----------------------------------------------------------------------
L03360  ! SET THE EFFECTIVE POTNL EVAPOTRANSP (ETP1) TO ZERO SINCE THIS IS SNOW
L03361  ! CASE, SO SURFACE EVAP NOT CALCULATED FROM EDIR, EC, OR ETT IN SMFLX
L03362  ! (BELOW).
L03363  ! SMFLX RETURNS UPDATED SOIL MOISTURE VALUES FOR NON-GLACIAL LAND.
L03364  ! ----------------------------------------------------------------------
L03365        END IF
L03366        CALL SMFLX (SMC,NSOIL,CMC,DT,PRCP1,ZSOIL,                       &
L03367                    SH2O,SLOPE,KDT,FRZFACT,                             &
L03368                    SMCMAX,BEXP,SMCWLT,DKSAT,DWSAT,                     &
L03369                    SHDFAC,CMCMAX,                                      &
L03370                    RUNOFF1,RUNOFF2,RUNOFF3,                            &
L03371                    EDIR1,EC1,ET1,                                      &
L03372                    DRIP, SFHEAD1RT,INFXS1RT)
L03373  ! ----------------------------------------------------------------------
L03374  ! BEFORE CALL SHFLX IN THIS SNOWPACK CASE, SET ZZ1 AND YY ARGUMENTS TO
L03375  ! SPECIAL VALUES THAT ENSURE THAT GROUND HEAT FLUX CALCULATED IN SHFLX
L03376  ! MATCHES THAT ALREADY COMPUTER FOR BELOW THE SNOWPACK, THUS THE SFC
L03377  ! HEAT FLUX TO BE COMPUTED IN SHFLX WILL EFFECTIVELY BE THE FLUX AT THE
L03378  ! SNOW TOP SURFACE.  T11 IS A DUMMY ARGUEMENT SO WE WILL NOT USE THE
L03379  ! SKIN TEMP VALUE AS REVISED BY SHFLX.
L03380  ! ----------------------------------------------------------------------
L03381        ZZ1 = 1.0
L03382        YY = STC (1) -0.5* SSOIL * ZSOIL (1)* ZZ1/ DF1
L03383
L03384  ! ----------------------------------------------------------------------
L03385  ! SHFLX WILL CALC/UPDATE THE SOIL TEMPS.  NOTE:  THE SUB-SFC HEAT FLUX
L03386  ! (SSOIL1) AND THE SKIN TEMP (T11) OUTPUT FROM THIS SHFLX CALL ARE NOT
L03387  ! USED  IN ANY SUBSEQUENT CALCULATIONS. RATHER, THEY ARE DUMMY VARIABLES
L03388  ! HERE IN THE SNOPAC CASE, SINCE THE SKIN TEMP AND SUB-SFC HEAT FLUX ARE
L03389  ! UPDATED INSTEAD NEAR THE BEGINNING OF THE CALL TO SNOPAC.
L03390  ! ----------------------------------------------------------------------
L03391        T11 = T1
L03392        CALL SHFLX (SSOIL1,STC,SMC,SMCMAX,NSOIL,T11,DT,YY,ZZ1,ZSOIL,     &
L03393                    TBOT,ZBOT,SMCWLT,PSISAT,SH2O,BEXP,F1,DF1,           &
L03394                    QUARTZ,CSOIL,VEGTYP,ISURBAN,SOILTYP,OPT_THCND       &
L03395                   ,HCPCT_FASDAS                                         ) !fasdas
L03396
L03397  ! ----------------------------------------------------------------------
L03398  ! SNOW DEPTH AND DENSITY ADJUSTMENT BASED ON SNOW COMPACTION.  YY IS
L03399  ! ASSUMED TO BE THE SOIL TEMPERTURE AT THE TOP OF THE SOIL COLUMN.
L03400  ! ----------------------------------------------------------------------
L03401        ! LAND
L03402        IF (ESD >  0.) THEN
L03403           CALL SNOWPACK (ESD,DT,SNOWH,SNDENS,T1,YY,SNOMLT,UA_PHYS)
L03404        ELSE
L03405           ESD = 0.
L03406           SNOWH = 0.
L03407           SNDENS = 0.
L03408           SNCOND = 1.
L03409           SNCOVR = 0.
L03410        END IF
L03411
L03412  ! ----------------------------------------------------------------------
L03413    END SUBROUTINE SNOPAC
```

讲解：有雪时 SFLX 的主分支，九步：① 清零与单位换算（ESNOW 四口径并存：ESNOW kg/(m²s)、ESDFLX 上限、ESNOW1 m/s、ESNOW2 m）。②**ETP≤0 露/凝华：**BETA 逻辑；RIBB≥0.1 且 FDOWN>150 时的稳定边界层修正（把 ETP 按雪盖/无雪份额重新加权）；ETANRG 按 (1−SNCOVR) • LSUBC+SNCOVR • LSUBS 混合潜热；DEW=−ETP1、ESNOW2=ETP1 • DT（雪面凝华增厚）。③**ETP>0：**SNCOVR<1 时调 EVAPO 三通道并把结果全部乘 (1−SNCOVR)——无雪份额照常蒸发；ESNOW=ETP • SNCOVR（UA 用 ETPN）为雪面升华、乘 LSUBS。④FLX1 降水-雪面热交换（雪 CPICE、雨 CPH2O 乘温差）。⑤ **有效雪面温度 T12 的隐式解：**T12A=（辐射 −FLX1−FLX2 项的皮肤能量平衡反解）/RR、T12B=DF1 • STC1/(DTOT • RR • RCH)（雪层 +半层土的导热项）、T12=(SFCTMP+T12A+T12B)/DENOM、DENOM=1+DF1/(DTOT • RR • RCH)——与 NOPAC 的 YY/ZZ1 同构但多了一层雪。⑥**T12≤ 冰点：**不融，T1=T12、SSOIL=DF1 • (T1−STC1)/DTOT、雪库削去升华量 ESNOW2、

FLX3=0。⑦**T12> 冰点融雪**：T1=TFREEZ • max(0.01,SNCOVR$^{SNOEXP)+T12}$ • (1−max(0.01,SNCOVR$^{SNOEXP))$（V3.9 改动——SNCOVR 极小时不再把 T1 完全压回 T12）；升华先扣（雪库见底则全清）；融雪热 FLX3=FDOWN−FLX1−FLX2−EMISSI • σ • T14−SSOIL−SEH−ETANRG（T14=T1^4、SEH=RCH • (T1−TH2)），负值归零，UA 的 FLX4 再扣；**EX=FLX3 · 0.001/LSUBF** 是融雪水当量率（m/s）、SNOMLT=EX • DT；雪库扛不住就融完、EX=ESD/DT 回填 FLX3（能量不白给）；**PRCP1 加 EX**——融雪水当降水入渗。⑧SMFLX 推进土壤水（入渗的是雨 +融雪）。⑨**SHFLX 的雪下特化**：ZZ1=1、YY=STC1−0.5 • SSOIL • ZSOIL1 • ZZ1/DF1——强制 SHFLX 算出的地热通量与雪下通量一致（皮肤温度已定，T11 是 dummy）；ESD>0 调 SNOWPACK 压密，否则清雪态。

九、雪物理三件套 SNOWPACK、SNOWZ0、SNOW_NEW

```
!file: phys/module_sf_noahlsm.F    L3417-L3549    (subroutine SNOWPACK)
L03417        SUBROUTINE SNOWPACK (ESD,DTSEC,SNOWH,SNDENS,TSNOW,TSOIL,SNOMLT,UA_PHYS)
L03418
L03419  ! ----------------------------------------------------------------------
L03420  ! SUBROUTINE SNOWPACK
L03421  ! ----------------------------------------------------------------------
L03422  ! CALCULATE COMPACTION OF SNOWPACK UNDER CONDITIONS OF INCREASING SNOW
L03423  ! DENSITY, AS OBTAINED FROM AN APPROXIMATE SOLUTION OF E. ANDERSON'S
L03424  ! DIFFERENTIAL EQUATION (3.29), NOAA TECHNICAL REPORT NWS 19, BY VICTOR
L03425  ! KOREN, 03/25/95.
L03426  ! ----------------------------------------------------------------------
L03427  ! ESD     WATER EQUIVALENT OF SNOW (M)
L03428  ! DTSEC   TIME STEP (SEC)
L03429  ! SNOWH   SNOW DEPTH (M)
L03430  ! SNDENS  SNOW DENSITY (G/CM3=DIMENSIONLESS FRACTION OF H2O DENSITY)
L03431  ! TSNOW   SNOW SURFACE TEMPERATURE (K)
L03432  ! TSOIL   SOIL SURFACE TEMPERATURE (K)
L03433
L03434  ! SUBROUTINE WILL RETURN NEW VALUES OF SNOWH AND SNDENS
L03435  ! ----------------------------------------------------------------------
L03436        IMPLICIT NONE
L03437
L03438        INTEGER                 :: IPOL, J
L03439        REAL, INTENT(IN)        :: ESD, DTSEC,TSNOW,TSOIL
L03440        REAL, INTENT(INOUT)     :: SNOWH, SNDENS
L03441        REAL                    :: BFAC,DSX,DTHR,DW,SNOWHC,PEXP,          &
L03442                                   TAVGC,TSNOWC,TSOILC,ESDC,ESDCX
L03443        REAL, PARAMETER         :: C1 = 0.01, C2 = 21.0, G = 9.81,        &
L03444                                   KN = 4000.0
L03445        LOGICAL, INTENT(IN)     :: UA_PHYS  ! UA: flag for UA option
L03446        REAL, INTENT(IN)        :: SNOMLT   ! UA: snow melt [m]
L03447        REAL                    :: SNOMLTC  ! UA: snow melt [cm]
L03448  ! ----------------------------------------------------------------------
L03449  ! CONVERSION INTO SIMULATION UNITS
L03450  ! ----------------------------------------------------------------------
L03451        SNOWHC = SNOWH *100.
L03452        ESDC = ESD *100.
L03453        IF(UA_PHYS) SNOMLTC = SNOMLT *100.
L03454        DTHR = DTSEC /3600.
L03455        TSNOWC = TSNOW -273.15
L03456        TSOILC = TSOIL -273.15
L03457
L03458  ! ----------------------------------------------------------------------
L03459  ! CALCULATING OF AVERAGE TEMPERATURE OF SNOW PACK
L03460  ! ----------------------------------------------------------------------
L03461  ! ----------------------------------------------------------------------
L03462  ! CALCULATING OF SNOW DEPTH AND DENSITY AS A RESULT OF COMPACTION
L03463  !  SNDENS=DS0*(EXP(BFAC*ESD)-1.)/(BFAC*ESD)
L03464  !  BFAC=DTHR*C1*EXP(0.08*TAVGC-C2*DS0)
L03465  ! NOTE: BFAC*ESD IN SNDENS EQN ABOVE HAS TO BE CAREFULLY TREATED
L03466  ! NUMERICALLY BELOW:
L03467  !   C1 IS THE FRACTIONAL INCREASE IN DENSITY (1/(CM*HR))
L03468  !   C2 IS A CONSTANT (CM3/G) KOJIMA ESTIMATED AS 21 CMS/G
L03469  ! ----------------------------------------------------------------------
L03470        TAVGC = 0.5* (TSNOWC + TSOILC)
L03471        IF (ESDC >  1.E-2) THEN
L03472           ESDCX = ESDC
L03473        ELSE
L03474           ESDCX = 1.E-2
L03475        END IF
L03476
L03477  !      DSX = SNDENS*((DEXP(BFAC*ESDC)-1.)/(BFAC*ESDC))
L03478  ! ----------------------------------------------------------------------
L03479  ! THE FUNCTION OF THE FORM (e**x-1)/x EMBEDDED IN ABOVE EXPRESSION
L03480  ! FOR DSX WAS CAUSING NUMERICAL DIFFICULTIES WHEN THE DENOMINATOR "x"
L03481  ! (I.E. BFAC*ESDC) BECAME ZERO OR APPROACHED ZERO (DESPITE THE FACT THAT
L03482  ! THE ANALYTICAL FUNCTION (e**x-1)/x HAS A WELL DEFINED LIMIT AS
L03483  ! "x" APPROACHES ZERO), HENCE BELOW WE REPLACE THE (e**x-1)/x
L03484  ! EXPRESSION WITH AN EQUIVALENT, NUMERICALLY WELL-BEHAVED
L03485  ! POLYNOMIAL EXPANSION.
L03486
L03487  ! NUMBER OF TERMS OF POLYNOMIAL EXPANSION, AND HENCE ITS ACCURACY,
L03488  ! IS GOVERNED BY ITERATION LIMIT "IPOL".
L03489  !      IPOL GREATER THAN 9 ONLY MAKES A DIFFERENCE ON DOUBLE
L03490  !           PRECISION (RELATIVE ERRORS GIVEN IN PERCENT %).
L03491  !       IPOL=9, FOR REL.ERROR <~ 1.6 E-6 % (8 SIGNIFICANT DIGITS)
L03492  !       IPOL=8, FOR REL.ERROR <~ 1.8 E-5 % (7 SIGNIFICANT DIGITS)
L03493  !       IPOL=7, FOR REL.ERROR <~ 1.8 E-4 % ...
L03494  ! ----------------------------------------------------------------------
L03495        BFAC = DTHR * C1* EXP (0.08* TAVGC - C2* SNDENS)
L03496        IPOL = 4
L03497        PEXP = 0.
L03498  !        PEXP = (1. + PEXP)*BFAC*ESDC/REAL(J+1)
L03499        DO J = IPOL,1, -1
L03500           PEXP = (1. + PEXP)* BFAC * ESDCX / REAL (J +1)
L03501        END DO
L03502
```

```
L03503          PEXP = PEXP + 1.
L03504    ! ------------------------------------------------------------------
L03505    ! ABOVE LINE ENDS POLYNOMIAL SUBSTITUTION
L03506    ! ------------------------------------------------------------------
L03507    !     END OF KOREAN FORMULATION
L03508
L03509    !     BASE FORMULATION (COGLEY ET AL., 1990)
L03510    !     CONVERT DENSITY FROM G/CM3 TO KG/M3
L03511    !       DSM=SNDENS*1000.0
L03512
L03513    !       DSX=DSM+DTSEC*0.5*DSM*G*ESD/
L03514    !    &      (1E7*EXP(-0.02*DSM+KN/(TAVGC+273.16)-14.643))
L03515
L03516    !  &   CONVERT DENSITY FROM KG/M3 TO G/CM3
L03517    !       DSX=DSX/1000.0
L03518
L03519    !     END OF COGLEY ET AL. FORMULATION
L03520
L03521    ! ------------------------------------------------------------------
L03522    ! SET UPPER/LOWER LIMIT ON SNOW DENSITY
L03523    ! ------------------------------------------------------------------
L03524          DSX = SNDENS * (PEXP)
L03525          IF (DSX > 0.40) DSX = 0.40
L03526          IF (DSX < 0.05) DSX = 0.05
L03527    ! ------------------------------------------------------------------
L03528    ! UPDATE OF SNOW DEPTH AND DENSITY DEPENDING ON LIQUID WATER DURING
L03529    ! SNOWMELT.  ASSUMED THAT 13% OF LIQUID WATER CAN BE STORED IN SNOW PER
L03530    ! DAY DURING SNOWMELT TILL SNOW DENSITY 0.40.
L03531    ! ------------------------------------------------------------------
L03532          SNDENS = DSX
L03533          IF (TSNOWC >=  0.) THEN
L03534             DW = 0.13* DTHR /24.
L03535             IF ( UA_PHYS .AND. TSOILC >= 0.) THEN
L03536                DW = MIN (DW, 0.13*SNOMLTC/(ESDCX+0.13*SNOMLTC))
L03537             ENDIF
L03538             SNDENS = SNDENS * (1. - DW) + DW
L03539             IF (SNDENS >=  0.40) SNDENS = 0.40
L03540    ! ------------------------------------------------------------------
L03541    ! CALCULATE SNOW DEPTH (CM) FROM SNOW WATER EQUIVALENT AND SNOW DENSITY.
L03542    ! CHANGE SNOW DEPTH UNITS TO METERS
L03543    ! ------------------------------------------------------------------
L03544          END IF
L03545          SNOWHC = ESDC / SNDENS
L03546          SNOWH = SNOWHC *0.01
L03547
L03548    ! ------------------------------------------------------------------
L03549      END SUBROUTINE SNOWPACK
```

讲解：雪压密（Koren 1995，Anderson 差分方程 3.29 的近似解）：①BFAC=DTHR • C1 • exp(0.08 • TAVGC−C2 • SNDENS)——C1=0.01 每厘米每小时、C2=21 cm³/g 是 Kojima 系数、TAVGC 取雪面/土面均温；② 解析解含 (e^x−1)/x，x→0 时数值病态——**IPOL=4 项多项式展开替代**（注释给出精度表：IPOL=9 可达 8 位有效数字，代码取 4 项够用）；③DSX=SNDENS • PEXP 钳位 [0.05, 0.40] g/cm³；④ 融雪期液水增密：TSNOWC≥0 时每小时密度向 1 收缩 13%/24 的份额（UA 版受 SNOMLT 限制）；⑤SNOWH=ESDC/SNDENS——深度由水当量/密度回算。Cogley 1990 备选公式完整注释。

```
!file: phys/module_sf_noahlsm.F    L3552-L3597    (subroutine SNOWZ0)
L03552          SUBROUTINE SNOWZ0 (SNCOVR,Z0, Z0BRD, SNOWH,FBUR,FGSN,SHDMAX,UA_PHYS)
L03553
L03554    ! ------------------------------------------------------------------
L03555    ! SUBROUTINE SNOWZ0
L03556    ! ------------------------------------------------------------------
L03557    ! CALCULATE TOTAL ROUGHNESS LENGTH OVER SNOW
L03558    ! SNCOVR  FRACTIONAL SNOW COVER
L03559    ! Z0      ROUGHNESS LENGTH (m)
L03560    ! Z0S     SNOW ROUGHNESS LENGTH:=0.001 (m)
L03561    ! ------------------------------------------------------------------
L03562          IMPLICIT NONE
L03563          REAL, INTENT(IN)         :: SNCOVR, Z0BRD
L03564          REAL, INTENT(OUT)        :: Z0
L03565          REAL, PARAMETER          :: Z0S=0.001
L03566          REAL, INTENT(IN)         :: SNOWH
L03567          REAL                     :: BURIAL
L03568          REAL                     :: Z0EFF
L03569          LOGICAL, INTENT(IN)      :: UA_PHYS   ! UA: flag for UA option
L03570          REAL, INTENT(IN)         :: FBUR      ! UA: fraction of canopy buried
L03571          REAL, INTENT(IN)         :: FGSN      ! UA: ground snow cover fraction
L03572          REAL, INTENT(IN)         :: SHDMAX    ! UA: maximum vegetation fraction
L03573          REAL, PARAMETER          :: Z0G=0.01  ! UA: soil roughness
L03574          REAL                     :: FV,A1,A2
L03575
L03576          IF(UA_PHYS) THEN
L03577
L03578              FV = SHDMAX * (1.-FBUR)
L03579              A1 = (1.-FV)**2*((1.-FGSN**2)*LOG(Z0G) + (FGSN**2)*LOG(Z0S))
L03580              A2 = (1.-(1.-FV)**2)*LOG(Z0BRD)
L03581              Z0 = EXP(A1+A2)
L03582
L03583          ELSE
L03584
L03585    !         Z0 = (1.- SNCOVR)* Z0BRD + SNCOVR * Z0S
L03586              BURIAL = 7.0*Z0BRD - SNOWH
L03587              IF(BURIAL.LE.0.0007) THEN
L03588                  Z0EFF = Z0S
L03589              ELSE
L03590                  Z0EFF = BURIAL/7.0
L03591              ENDIF
L03592
L03593              Z0 = (1.- SNCOVR)* Z0BRD + SNCOVR * Z0EFF
L03594
L03595          ENDIF
L03596    ! ------------------------------------------------------------------
L03597      END SUBROUTINE SNOWZ0
```

讲解：雪面粗糙度。无 UA：掩埋法——BURIAL=7 • Z0BRD−SNOWH，埋没殆尽（BURIAL≤0.0007）时 Z0EFF=Z0S=0.001 m，否则 Z0EFF=BURIAL/7（积雪把地形粗糙度逐渐「填平」的线性近似）；Z0=(1−SNCOVR) • Z0BRD+SNCOVR • Z0EFF 按雪盖率插值。UA：FV=SHDMAX • (1−FBUR) 植被暴露分数，对数域四分量混合 A1=(1−FV)² • ((1−FGSN²) • ln Z0G+FGSN² • ln Z0S)、A2=(1−(1−FV)²) • ln Z0BRD、Z0=exp(A1+A2)。

```
!file: phys/module_sf_noahlsm.F    L3601-L3650    (subroutine SNOW_NEW)
L03601        SUBROUTINE SNOW_NEW (TEMP,NEWSN,SNOWH,SNDENS)
L03602
L03603  ! ----------------------------------------------------------------------
L03604  ! SUBROUTINE SNOW_NEW
L03605  ! ----------------------------------------------------------------------
L03606  ! CALCULATE SNOW DEPTH AND DENSITY TO ACCOUNT FOR THE NEW SNOWFALL.
L03607  ! NEW VALUES OF SNOW DEPTH & DENSITY RETURNED.
L03608
L03609  ! TEMP    AIR TEMPERATURE (K)
L03610  ! NEWSN   NEW SNOWFALL (M)
L03611  ! SNOWH   SNOW DEPTH (M)
L03612  ! SNDENS  SNOW DENSITY (G/CM3=DIMENSIONLESS FRACTION OF H2O DENSITY)
L03613  ! ----------------------------------------------------------------------
L03614        IMPLICIT NONE
L03615        REAL, INTENT(IN)         :: NEWSN, TEMP
L03616        REAL, INTENT(INOUT)      :: SNDENS, SNOWH
L03617        REAL                     :: DSNEW, HNEWC, SNOWHC,NEWSNC,TEMPC
L03618
L03619  ! ----------------------------------------------------------------------
L03620  ! CONVERSION INTO SIMULATION UNITS
L03621  ! ----------------------------------------------------------------------
L03622        SNOWHC = SNOWH *100.
L03623        NEWSNC = NEWSN *100.
L03624
L03625  ! ----------------------------------------------------------------------
L03626  ! CALCULATING NEW SNOWFALL DENSITY DEPENDING ON TEMPERATURE
L03627  ! EQUATION FROM GOTTLIB L. 'A GENERAL RUNOFF MODEL FOR SNOWCOVERED
L03628  ! AND GLACIERIZED BASIN', 6TH NORDIC HYDROLOGICAL CONFERENCE,
L03629  ! VEMADOLEN, SWEDEN, 1980, 172-177PP.
L03630  !----------------------------------------------------------------------
L03631        TEMPC = TEMP -273.15
L03632        IF (TEMPC <=  -15.) THEN
L03633           DSNEW = 0.05
L03634        ELSE
L03635           DSNEW = 0.05+0.0017* (TEMPC +15.)**1.5
L03636        END IF
L03637  ! ----------------------------------------------------------------------
L03638  ! ADJUSTMENT OF SNOW DENSITY DEPENDING ON NEW SNOWFALL
L03639  ! ----------------------------------------------------------------------
L03640        HNEWC = NEWSNC / DSNEW
L03641        IF (SNOWHC + HNEWC .LT. 1.0E-3) THEN
L03642           SNDENS = MAX(DSNEW,SNDENS)
L03643        ELSE
L03644           SNDENS = (SNOWHC * SNDENS + HNEWC * DSNEW)/ (SNOWHC + HNEWC)
L03645        ENDIF
L03646        SNOWHC = SNOWHC + HNEWC
L03647        SNOWH = SNOWHC *0.01
L03648
L03649  ! ----------------------------------------------------------------------
L03650    END SUBROUTINE SNOW_NEW
```

讲解：新雪密度按气温分档（Gottlib 1980）：TEMPC≤−15°C 时 0.05 g/cm³，否则 0.05+0.0017 • (TEMPC+15)^1.5——冷干雪轻、湿雪重；新雪按自身密度换深度 HNEWC 与旧雪库做深度加权平均密度，总深累加。

十、土壤水求解器 SRT

```
!file: phys/module_sf_noahlsm.F    L3653-L3947    (subroutine SRT)
L03653        SUBROUTINE SRT (RHSTT,EDIR,ET,SH2O,SH2OA,NSOIL,PCPDRP,            &
L03654                        ZSOIL,DWSAT,DKSAT,SMCMAX,BEXP,RUNOFF1,            &
L03655                       RUNOFF2,DT,SMCWLT,SLOPE,KDT,FRZX,SICE,AI,BI,CI,  &
L03656                       SFHEAD1RT,INFXS1RT )
L03657
L03658  ! ----------------------------------------------------------------------
L03659  ! SUBROUTINE SRT
L03660  ! ----------------------------------------------------------------------
L03661  ! CALCULATE THE RIGHT HAND SIDE OF THE TIME TENDENCY TERM OF THE SOIL
L03662  ! WATER DIFFUSION EQUATION.  ALSO TO COMPUTE ( PREPARE ) THE MATRIX
L03663  ! COEFFICIENTS FOR THE TRI-DIAGONAL MATRIX OF THE IMPLICIT TIME SCHEME.
L03664  ! ----------------------------------------------------------------------
L03665        IMPLICIT NONE
L03666        INTEGER, INTENT(IN)       :: NSOIL
L03667        INTEGER                   :: IALP1, IOHINF, J, JJ,  K, KS
L03668
L03669  !DJG NDHMS/WRF-Hydro edit... Variables used in OV routing infiltration calcs
L03670         REAL, INTENT(INOUT)     :: SFHEAD1RT, INFXS1RT
L03671         REAL                    :: SFCWATR,chcksm
L03672
L03673
L03674
L03675        REAL, INTENT(IN)          :: BEXP, DKSAT, DT, DWSAT, EDIR, FRZX,  &
L03676                                     KDT, PCPDRP, SLOPE, SMCMAX, SMCWLT
L03677        REAL, INTENT(OUT)         :: RUNOFF1, RUNOFF2
L03678        REAL, DIMENSION(1:NSOIL), INTENT(IN)   :: ET, SH2O, SH2OA, SICE,  &
L03679                                                  ZSOIL
L03680        REAL, DIMENSION(1:NSOIL), INTENT(OUT)  :: RHSTT
L03681        REAL, DIMENSION(1:NSOIL), INTENT(OUT)  :: AI, BI, CI
L03682        REAL, DIMENSION(1:NSOIL)  :: DMAX
L03683        REAL                      :: ACRT, DD, DDT, DDZ, DDZ2, DENOM,     &
L03684                                     DENOM2,DICE, DSMDZ, DSMDZ2, DT1,     &
```

```
L03685                                    FCR,INFMAX,MXSMC,MXSMC2,NUMER,PDDUM, &
L03686                                    PX, SICEMAX,SLOPX, SMCAV, SSTT,      &
L03687                                    SUM, VAL, WCND, WCND2, WDF, WDF2
L03688       INTEGER, PARAMETER        :: CVFRZ = 3
L03689
L03690  ! ----------------------------------------------------------------------
L03691  ! FROZEN GROUND VERSION:
L03692  ! REFERENCE FROZEN GROUND PARAMETER, CVFRZ, IS A SHAPE PARAMETER OF
L03693  ! AREAL DISTRIBUTION FUNCTION OF SOIL ICE CONTENT WHICH EQUALS 1/CV.
L03694  ! CV IS A COEFFICIENT OF SPATIAL VARIATION OF SOIL ICE CONTENT.  BASED
L03695  ! ON FIELD DATA CV DEPENDS ON AREAL MEAN OF FROZEN DEPTH, AND IT CLOSE
L03696  ! TO CONSTANT = 0.6 IF AREAL MEAN FROZEN DEPTH IS ABOVE 20 CM.  THAT IS
L03697  ! WHY PARAMETER CVFRZ = 3 (INT{1/0.6*0.6}).
L03698  ! CURRENT LOGIC DOESN'T ALLOW CVFRZ BE BIGGER THAN 3
L03699  ! ----------------------------------------------------------------------
L03700
L03701  ! ----------------------------------------------------------------------
L03702  ! DETERMINE RAINFALL INFILTRATION RATE AND RUNOFF.  INCLUDE THE
L03703  ! INFILTRATION FORMULE FROM SCHAAKE AND KOREN MODEL.
L03704  ! MODIFIED BY Q DUAN
L03705  ! ----------------------------------------------------------------------
L03706  ! ----------------------------------------------------------------------
L03707  ! LET SICEMAX BE THE GREATEST, IF ANY, FROZEN WATER CONTENT WITHIN SOIL
L03708  ! LAYERS.
L03709  ! ----------------------------------------------------------------------
L03710       IOHINF = 1
L03711       SICEMAX = 0.0
L03712       DO KS = 1,NSOIL
L03713          IF (SICE (KS) >  SICEMAX) SICEMAX = SICE (KS)
L03714  ! ----------------------------------------------------------------------
L03715  ! DETERMINE RAINFALL INFILTRATION RATE AND RUNOFF
L03716  ! ----------------------------------------------------------------------
L03717       END DO
L03718
L03719  #ifdef WRF_HYDRO
L03720  !DJG NDHMS/WRF-Hydro edit...
L03721  !DJG Use previously merged Precip and Sfchead for infil. cap. calc.
L03722       SFCWATR = PCPDRP
L03723       PDDUM = SFCWATR
L03724  !DJG original   PDDUM = PCPDRP
L03725       RUNOFF1 = 0.0
L03726       INFXS1RT = 0.0
L03727  #else
L03728       PDDUM = PCPDRP
L03729       RUNOFF1 = 0.0
L03730  #endif
L03731
L03732
L03733
L03734  ! ----------------------------------------------------------------------
L03735  ! MODIFIED BY Q. DUAN, 5/16/94
L03736  ! ----------------------------------------------------------------------
L03737  !        IF (IOHINF == 1) THEN
L03738
L03739  #ifdef WRF_HYDRO
L03740  !DJG NDHMS/WRF-Hydro edit...
L03741  !DJG IF (PCPDRP /=  0.0) THEN
L03742       IF (SFCWATR /=  0.0) THEN
L03743  #else
L03744       IF (PCPDRP /=  0.0) THEN
L03745  #endif
L03746          DT1 = DT /86400.
L03747          SMCAV = SMCMAX - SMCWLT
L03748
L03749  ! ----------------------------------------------------------------------
L03750  ! FROZEN GROUND VERSION:
L03751  ! ----------------------------------------------------------------------
L03752          DMAX (1)= - ZSOIL (1)* SMCAV
L03753
L03754          DICE = - ZSOIL (1) * SICE (1)
L03755          DMAX (1)= DMAX (1)* (1.0- (SH2OA (1) + SICE (1) - SMCWLT)/      &
L03756                     SMCAV)
L03757
L03758          DD = DMAX (1)
L03759
L03760  ! ----------------------------------------------------------------------
L03761  ! FROZEN GROUND VERSION:
L03762  ! ----------------------------------------------------------------------
L03763          DO KS = 2,NSOIL
L03764
L03765             DICE = DICE+ ( ZSOIL (KS -1) - ZSOIL (KS) ) * SICE (KS)
L03766             DMAX (KS) = (ZSOIL (KS -1) - ZSOIL (KS))* SMCAV
L03767             DMAX (KS) = DMAX (KS)* (1.0- (SH2OA (KS) + SICE (KS)        &
L03768                        - SMCWLT)/ SMCAV)
L03769             DD = DD+ DMAX (KS)
L03770  ! ----------------------------------------------------------------------
L03771  ! VAL = (1.-EXP(-KDT*SQRT(DT1)))
L03772  ! IN BELOW, REMOVE THE SQRT IN ABOVE
L03773  ! ----------------------------------------------------------------------
L03774          END DO
L03775          VAL = (1. - EXP ( - KDT * DT1))
L03776          DDT = DD * VAL
L03777  #ifdef WRF_HYDRO
L03778  !DJG NDHMS/WRF-Hydro edit...
L03779  !DJG        PX = PCPDRP * DT
L03780          PX = SFCWATR * DT
L03781  #else
L03782          PX = PCPDRP * DT
L03783  #endif
L03784          IF (PX <  0.0) PX = 0.0
L03785
L03786
L03787
L03788  ! ----------------------------------------------------------------------
L03789  ! FROZEN GROUND VERSION:
L03790  ! REDUCTION OF INFILTRATION BASED ON FROZEN GROUND PARAMETERS
L03791  ! ----------------------------------------------------------------------
L03792          INFMAX = (PX * (DDT / (PX + DDT)))/ DT
L03793          FCR = 1.
L03794          IF (DICE >  1.E-2) THEN
```

```
L03795              ACRT = CVFRZ * FRZX / DICE
L03796              SUM = 1.
L03797              IALP1 = CVFRZ - 1
L03798              DO J = 1,IALP1
L03799                 K = 1
L03800                 DO JJ = J +1,IALP1
L03801                    K = K * JJ
L03802                 END DO
L03803                 SUM = SUM + (ACRT ** ( CVFRZ - J)) / FLOAT (K)
L03804              END DO
L03805              FCR = 1. - EXP ( - ACRT) * SUM
L03806           END IF
L03807
L03808  ! ------------------------------------------------------------------
L03809  ! CORRECTION OF INFILTRATION LIMITATION:
L03810  ! IF INFMAX .LE. HYDROLIC CONDUCTIVITY ASSIGN INFMAX THE VALUE OF
L03811  ! HYDROLIC CONDUCTIVITY
L03812  ! ------------------------------------------------------------------
L03813  !         MXSMC = MAX ( SH2OA(1), SH2OA(2) )
L03814           INFMAX = INFMAX * FCR
L03815
L03816           MXSMC = SH2OA (1)
L03817           CALL WDFCND (WDF,WCND,MXSMC,SMCMAX,BEXP,DKSAT,DWSAT,           &
L03818                        SICEMAX)
L03819           INFMAX = MAX (INFMAX,WCND)
L03820
L03821           INFMAX = MIN (INFMAX,PX/DT)
L03822  #ifdef WRF_HYDRO
L03823  !DJG NDHMS/WRF-Hydro edit...
L03824  !DJG       IF (PCPDRP >  INFMAX) THEN
L03825           IF (SFCWATR > INFMAX) THEN
L03826  !DJG          RUNOFF1 = PCPDRP - INFMAX
L03827            RUNOFF1 = SFCWATR - INFMAX
L03828  #else
L03829           IF (PCPDRP >  INFMAX) THEN
L03830              RUNOFF1 = PCPDRP - INFMAX
L03831  #endif
L03832            INFXS1RT = RUNOFF1*DT*1000.
L03833            PDDUM = INFMAX
L03834           END IF
L03835
L03836  ! ------------------------------------------------------------------
L03837  ! TO AVOID SPURIOUS DRAINAGE BEHAVIOR, 'UPSTREAM DIFFERENCING' IN LINE
L03838  ! BELOW REPLACED WITH NEW APPROACH IN 2ND LINE:
L03839  ! 'MXSMC = MAX(SH2OA(1), SH2OA(2))'
L03840  ! ------------------------------------------------------------------
L03841        END IF
L03842
L03843        MXSMC = SH2OA (1)
L03844        CALL WDFCND (WDF,WCND,MXSMC,SMCMAX,BEXP,DKSAT,DWSAT,              &
L03845                     SICEMAX)
L03846  ! ------------------------------------------------------------------
L03847  ! CALC THE MATRIX COEFFICIENTS AI, BI, AND CI FOR THE TOP LAYER
L03848  ! ------------------------------------------------------------------
L03849        DDZ = 1. / ( - .5 * ZSOIL (2) )
L03850        AI (1) = 0.0
L03851        BI (1) = WDF * DDZ / ( - ZSOIL (1) )
L03852
L03853  ! ------------------------------------------------------------------
L03854  ! CALC RHSTT FOR THE TOP LAYER AFTER CALC'NG THE VERTICAL SOIL MOISTURE
L03855  ! GRADIENT BTWN THE TOP AND NEXT TO TOP LAYERS.
L03856  ! ------------------------------------------------------------------
L03857        CI (1) = - BI (1)
L03858        DSMDZ = ( SH2O (1) - SH2O (2) ) / ( - .5 * ZSOIL (2) )
L03859        RHSTT (1) = (WDF * DSMDZ + WCND- PDDUM + EDIR + ET (1))/ ZSOIL (1)
L03860
L03861  ! ------------------------------------------------------------------
L03862  ! INITIALIZE DDZ2
L03863  ! ------------------------------------------------------------------
L03864        SSTT = WDF * DSMDZ + WCND+ EDIR + ET (1)
L03865
L03866  ! ------------------------------------------------------------------
L03867  ! LOOP THRU THE REMAINING SOIL LAYERS, REPEATING THE ABV PROCESS
L03868  ! ------------------------------------------------------------------
L03869        DDZ2 = 0.0
L03870        DO K = 2,NSOIL
L03871           DENOM2 = (ZSOIL (K -1) - ZSOIL (K))
L03872           IF (K /= NSOIL) THEN
L03873
L03874  ! ------------------------------------------------------------------
L03875  ! AGAIN, TO AVOID SPURIOUS DRAINAGE BEHAVIOR, 'UPSTREAM DIFFERENCING' IN
L03876  ! LINE BELOW REPLACED WITH NEW APPROACH IN 2ND LINE:
L03877  ! 'MXSMC2 = MAX (SH2OA(K), SH2OA(K+1))'
L03878  ! ------------------------------------------------------------------
L03879              SLOPX = 1.
L03880
L03881              MXSMC2 = SH2OA (K)
L03882              CALL WDFCND (WDF2,WCND2,MXSMC2,SMCMAX,BEXP,DKSAT,DWSAT,     &
L03883                           SICEMAX)
L03884  ! ------------------------------------------------------------------
L03885  ! CALC SOME PARTIAL PRODUCTS FOR LATER USE IN CALC'NG RHSTT
L03886  ! ------------------------------------------------------------------
L03887              DENOM = (ZSOIL (K -1) - ZSOIL (K +1))
L03888
L03889  ! ------------------------------------------------------------------
L03890  ! CALC THE MATRIX COEF, CI, AFTER CALC'NG ITS PARTIAL PRODUCT
L03891  ! ------------------------------------------------------------------
L03892              DSMDZ2 = (SH2O (K) - SH2O (K +1)) / (DENOM * 0.5)
L03893              DDZ2 = 2.0 / DENOM
L03894              CI (K) = - WDF2 * DDZ2 / DENOM2
L03895
L03896           ELSE
L03897  ! ------------------------------------------------------------------
L03898  ! SLOPE OF BOTTOM LAYER IS INTRODUCED
L03899  ! ------------------------------------------------------------------
L03900
L03901  ! ------------------------------------------------------------------
L03902  ! RETRIEVE THE SOIL WATER DIFFUSIVITY AND HYDRAULIC CONDUCTIVITY FOR
L03903  ! THIS LAYER
L03904  ! ------------------------------------------------------------------
```

```
L03905             SLOPX = SLOPE
L03906           CALL WDFCND (WDF2,WCND2,SH2OA (NSOIL),SMCMAX,BEXP,DKSAT,DWSAT,     &
L03907                        SICEMAX)
L03908
L03909  ! ----------------------------------------------------------------------
L03910  ! CALC A PARTIAL PRODUCT FOR LATER USE IN CALC'NG RHSTT
L03911  ! ----------------------------------------------------------------------
L03912
L03913  ! ----------------------------------------------------------------------
L03914  ! SET MATRIX COEF CI TO ZERO
L03915  ! ----------------------------------------------------------------------
L03916             DSMDZ2 = 0.0
L03917             CI (K) = 0.0
L03918  ! ----------------------------------------------------------------------
L03919  ! CALC RHSTT FOR THIS LAYER AFTER CALC'NG ITS NUMERATOR
L03920  ! ----------------------------------------------------------------------
L03921           END IF
L03922           NUMER = (WDF2 * DSMDZ2) + SLOPX * WCND2- (WDF * DSMDZ)         &
L03923                 - WCND+ ET (K)
L03924
L03925  ! ----------------------------------------------------------------------
L03926  ! CALC MATRIX COEFS, AI, AND BI FOR THIS LAYER
L03927  ! ----------------------------------------------------------------------
L03928           RHSTT (K) = NUMER / ( - DENOM2)
L03929           AI (K) = - WDF * DDZ / DENOM2
L03930
L03931  ! ----------------------------------------------------------------------
L03932  ! RESET VALUES OF WDF, WCND, DSMDZ, AND DDZ FOR LOOP TO NEXT LYR
L03933  ! RUNOFF2:  SUB-SURFACE OR BASEFLOW RUNOFF
L03934  ! ----------------------------------------------------------------------
L03935           BI (K) = - ( AI (K) + CI (K) )
L03936           IF (K .eq. NSOIL) THEN
L03937              RUNOFF2 = SLOPX * WCND2
L03938           END IF
L03939           IF (K .ne. NSOIL) THEN
L03940              WDF = WDF2
L03941              WCND = WCND2
L03942              DSMDZ = DSMDZ2
L03943              DDZ = DDZ2
L03944           END IF
L03945         END DO
L03946  ! ----------------------------------------------------------------------
L03947    END SUBROUTINE SRT
```

讲解：土壤水扩散方程的右端与矩阵系数（与 HRT 同构的水版）：①**Schaake–Koren 入渗分配**：DDT=DD • (1−exp(−KDT • DT1))（DD 是各层可入渗空隙总量、DT1=DT/86400）、INFMAX=PX • DDT/(PX+DDT)/DT——降水 PX 在「入渗能力」与「实际降水」之间的平滑 min；冻土修正 FCR=1−exp(−ACRT) • Σ——CVFRZ=3 的分布函数形状参数、DICE 冻水深度（注：CVFRZ 逻辑不允许大于 3），冻土降低入渗；INFMAX 与表层导水率取大、与 PX/DT 取小；超入渗变 RUNOFF1（WRF-Hydro 下同步写 INFXS1RT 供河道汇流）。② **底边界**：SLOPX=SLOPE（Zobler 坡度类的线性水库系数）、RUNOFF2=SLOPE • WCND(NSOIL)——底层重力排水即基流。③ 逐层 WDFCND 算扩散率 WDF 与导水率 WCND；RHSTT(K)=通量差/层厚；AI/BI/CI 三对角组装（BI=−(AI+CI)）；表层湿度梯度用 SH2OA(1) 单点而非 max(SH2OA(1),SH2OA(2))——注释解释：上游差分造成伪排水，已改。

十一、土壤水推进 SSTEP 与水力传导 WDFCND

```
!file: phys/module_sf_noahlsm.F    L3950-L4077    (subroutine SSTEP)
L03950         SUBROUTINE SSTEP (SH2OOUT,SH2OIN,CMC,RHSTT,RHSCT,DT,             &
L03951                           NSOIL,SMCMAX,CMCMAX,RUNOFF3,ZSOIL,SMC,SICE,     &
L03952                           AI,BI,CI, INFXS1RT)
L03953
L03954  ! ----------------------------------------------------------------------
L03955  ! SUBROUTINE SSTEP
L03956  ! ----------------------------------------------------------------------
L03957  ! CALCULATE/UPDATE SOIL MOISTURE CONTENT VALUES AND CANOPY MOISTURE
L03958  ! CONTENT VALUES.
L03959  ! ----------------------------------------------------------------------
L03960         IMPLICIT NONE
L03961         INTEGER, INTENT(IN)       :: NSOIL
L03962         INTEGER                   :: I, K, KK11
L03963
L03964  !!DJG NDHMS/WRF-Hydro edit...
L03965         REAL, INTENT(INOUT)       :: INFXS1RT
L03966         REAL                      :: AVAIL
L03967
L03968         REAL, INTENT(IN)          :: CMCMAX, DT, SMCMAX
L03969         REAL, INTENT(OUT)         :: RUNOFF3
L03970         REAL, INTENT(INOUT)       :: CMC
L03971         REAL, DIMENSION(1:NSOIL), INTENT(IN)     :: SH2OIN, SICE, ZSOIL
L03972         REAL, DIMENSION(1:NSOIL), INTENT(OUT)    :: SH2OOUT
L03973         REAL, DIMENSION(1:NSOIL), INTENT(INOUT)  :: RHSTT, SMC
L03974         REAL, DIMENSION(1:NSOIL), INTENT(INOUT)  :: AI, BI, CI
L03975         REAL, DIMENSION(1:NSOIL)  :: RHSTTin
L03976         REAL, DIMENSION(1:NSOIL)  :: CIin
L03977         REAL                      :: DDZ, RHSCT, STOT, WPLUS
L03978
L03979  ! ----------------------------------------------------------------------
L03980  ! CREATE 'AMOUNT' VALUES OF VARIABLES TO BE INPUT TO THE
L03981  ! TRI-DIAGONAL MATRIX ROUTINE.
L03982  ! ----------------------------------------------------------------------
L03983         DO K = 1,NSOIL
L03984            RHSTT (K) = RHSTT (K) * DT
L03985            AI (K) = AI (K) * DT
L03986            BI (K) = 1. + BI (K) * DT
```

```
L03987           CI (K) = CI (K) * DT
L03988        END DO
L03989  ! ---------------------------------------------------------------------
L03990  ! COPY VALUES FOR INPUT VARIABLES BEFORE CALL TO ROSR12
L03991  ! ---------------------------------------------------------------------
L03992        DO K = 1,NSOIL
L03993           RHSTTin (K) = RHSTT (K)
L03994        END DO
L03995        DO K = 1,NSOIL
L03996           CIin (K) = CI (K)
L03997        END DO
L03998  ! ---------------------------------------------------------------------
L03999  ! CALL ROSR12 TO SOLVE THE TRI-DIAGONAL MATRIX
L04000  ! ---------------------------------------------------------------------
L04001        CALL ROSR12 (CI,AI,BI,CIin,RHSTTin,RHSTT,NSOIL)
L04002  ! ---------------------------------------------------------------------
L04003  ! SUM THE PREVIOUS SMC VALUE AND THE MATRIX SOLUTION TO GET A
L04004  ! NEW VALUE.  MIN ALLOWABLE VALUE OF SMC WILL BE 0.02.
L04005  ! RUNOFF3: RUNOFF WITHIN SOIL LAYERS
L04006  ! ---------------------------------------------------------------------
L04007        WPLUS = 0.0
L04008        RUNOFF3 = 0.
L04009
L04010        DDZ = - ZSOIL (1)
L04011        DO K = 1,NSOIL
L04012           IF (K /= 1) DDZ = ZSOIL (K - 1) - ZSOIL (K)
L04013           SH2OOUT (K) = SH2OIN (K) + CI (K) + WPLUS / DDZ
L04014           STOT = SH2OOUT (K) + SICE (K)
L04015           IF (STOT > SMCMAX) THEN
L04016              IF (K .eq. 1) THEN
L04017                 DDZ = - ZSOIL (1)
L04018              ELSE
L04019                 KK11 = K - 1
L04020                 DDZ = - ZSOIL (K) + ZSOIL (KK11)
L04021              END IF
L04022              WPLUS = (STOT - SMCMAX) * DDZ
L04023           ELSE
L04024              WPLUS = 0.
L04025           END IF
L04026           SMC (K) = MAX ( MIN (STOT,SMCMAX),0.02 )
L04027           SH2OOUT (K) = MAX ( (SMC (K) - SICE (K)),0.0)
L04028        END DO
L04029  #ifdef WRF_HYDRO
L04030  !DJG NDHMS/WRF-Hydro edit...
L04031  !DJG Modifications to redstribute WPLUS/RUNOFF3 (soil moisture closure error) to soil profile
L04032  !DJG beginning at bottom layer (NSOIL)
L04033           IF (WPLUS > 0.) THEN
L04034             DO K=NSOIL,2,-1
L04035
L04036                IF (K .eq. 2) THEN     !Assign soil depths
L04037                  DDZ = -ZSOIL(1)
L04038                ELSE
L04039                  DDZ = ZSOIL(K-2)-ZSOIL(K-1)
L04040                END IF
L04041
L04042                AVAIL = (SMCMAX - SMC(K-1)) * DDZ    !Det. Avail. Stor.
L04043
L04044  !              print *, "ZZZZZ", K,DDZ,AVAIL,WPLUS,SMC(K),SMC(K-1),SMCMAX
L04045
L04046                IF (WPLUS <= AVAIL) THEN
L04047                  SMC(K-1) = SMC(K-1) + WPLUS/DDZ
L04048                  WPLUS = 0.
L04049                ELSE
L04050                  SMC(K-1) = SMCMAX
L04051                  WPLUS = WPLUS - AVAIL
L04052                  IF (K-1 .eq. 1) THEN
L04053                    INFXS1RT = INFXS1RT + WPLUS*1000
L04054                    WPLUS = 0.
L04055                  END IF
L04056                END IF
L04057
L04058  !             SMC (K) = MAX ( MIN (STOT,SMCMAX),0.02 )
L04059                SH2OOUT (K) = MAX ( (SMC (K) - SICE (K)),0.0)
L04060
L04061             END DO
L04062           END IF
L04063  !DJG NDHMS/WRF-Hydro edit...End of modification
L04064  #endif
L04065
L04066
L04067  ! ---------------------------------------------------------------------
L04068  ! UPDATE CANOPY WATER CONTENT/INTERCEPTION (CMC).  CONVERT RHSCT TO
L04069  ! AN 'AMOUNT' VALUE AND ADD TO PREVIOUS CMC VALUE TO GET NEW CMC.
L04070  ! ---------------------------------------------------------------------
L04071        RUNOFF3 = WPLUS
L04072        CMC = CMC + DT * RHSCT
L04073        IF (CMC < 1.E-20) CMC = 0.0
L04074        CMC = MIN (CMC,CMCMAX)
L04075
L04076  ! ---------------------------------------------------------------------
L04077    END SUBROUTINE SSTEP
```

讲解：与 HSTEP 同构的水版推进：化量 →ROSR12→SH2OOUT=SH2OIN+解 +WPLUS/DDZ；STOT=SH2OOUT+SICE 超孔隙度的部分变 WPLUS 逐层上抛、SMC=MAX(MIN(STOT,SMCMAX),0.02)（干下限 0.02）、SH2OOUT=SMC−SICE；WRF-Hydro 下 WPLUS 从底层向下重新分配（按 AVAIL 容量逐层塞满），仍超则进 INFXS1RT；**RUNOFF3=WPLUS**（层间超孔隙水，SFLX 收尾时并入 RUNOFF2）；CMC=CMC+DT • RHSCT 钳位 [0, CMCMAX]（与 SMFLX 的 DRIP 逻辑衔接——DRIP 是溢出量、这里钳位是硬上限）。

```
!file: phys/module_sf_noahlsm.F    L4461-L4519    (subroutine WDFCND)
L04461        SUBROUTINE WDFCND (WDF,WCND,SMC,SMCMAX,BEXP,DKSAT,DWSAT,          &
L04462       &                   SICEMAX)
L04463
```

```
L04464  ! ----------------------------------------------------------------------
L04465  ! SUBROUTINE WDFCND
L04466  ! ----------------------------------------------------------------------
L04467  ! CALCULATE SOIL WATER DIFFUSIVITY AND SOIL HYDRAULIC CONDUCTIVITY.
L04468  ! ----------------------------------------------------------------------
L04469        IMPLICIT NONE
L04470        REAL     BEXP
L04471        REAL     DKSAT
L04472        REAL     DWSAT
L04473        REAL     EXPON
L04474        REAL     FACTR1
L04475        REAL     FACTR2
L04476        REAL     SICEMAX
L04477        REAL     SMC
L04478        REAL     SMCMAX
L04479        REAL     VKwgt
L04480        REAL     WCND
L04481
L04482  ! ----------------------------------------------------------------------
L04483  !     CALC THE RATIO OF THE ACTUAL TO THE MAX PSBL SOIL H2O CONTENT
L04484  ! ----------------------------------------------------------------------
L04485        REAL     WDF
L04486        FACTR1 = 0.05 / SMCMAX
L04487
L04488  ! ----------------------------------------------------------------------
L04489  ! PREP AN EXPNTL COEF AND CALC THE SOIL WATER DIFFUSIVITY
L04490  ! ----------------------------------------------------------------------
L04491        FACTR2 = SMC / SMCMAX
L04492        FACTR1 = MIN(FACTR1,FACTR2)
L04493        EXPON = BEXP + 2.0
L04494
L04495  ! ----------------------------------------------------------------------
L04496  ! FROZEN SOIL HYDRAULIC DIFFUSIVITY.  VERY SENSITIVE TO THE VERTICAL
L04497  ! GRADIENT OF UNFROZEN WATER. THE LATTER GRADIENT CAN BECOME VERY
L04498  ! EXTREME IN FREEZING/THAWING SITUATIONS, AND GIVEN THE RELATIVELY
L04499  ! FEW AND THICK SOIL LAYERS, THIS GRADIENT SUFFERES SERIOUS
L04500  ! TRUNCTION ERRORS YIELDING ERRONEOUSLY HIGH VERTICAL TRANSPORTS OF
L04501  ! UNFROZEN WATER IN BOTH DIRECTIONS FROM HUGE HYDRAULIC DIFFUSIVITY.
L04502  ! THEREFORE, WE FOUND WE HAD TO ARBITRARILY CONSTRAIN WDF
L04503  ! --
L04504  ! VERSION D_10CM: ........  FACTR1 = 0.2/SMCMAX
L04505  ! WEIGHTED APPROACH...................... PABLO GRUNMANN, 28_SEP_1999.
L04506  ! ----------------------------------------------------------------------
L04507        WDF = DWSAT * FACTR2 ** EXPON
L04508        IF (SICEMAX .gt. 0.0) THEN
L04509           VKWGT = 1./ (1. + (500.* SICEMAX)**3.)
L04510           WDF = VKWGT * WDF + (1. - VKWGT)* DWSAT * FACTR1** EXPON
L04511  ! ----------------------------------------------------------------------
L04512  ! RESET THE EXPNTL COEF AND CALC THE HYDRAULIC CONDUCTIVITY
L04513  ! ----------------------------------------------------------------------
L04514        END IF
L04515        EXPON = (2.0 * BEXP) + 3.0
L04516        WCND = DKSAT * FACTR2 ** EXPON
L04517
L04518  ! ----------------------------------------------------------------------
L04519    END SUBROUTINE WDFCND
```

讲解：Clapp–Hornberger 型水力参数：WDF=DWSAT • FACTR2$^{(BEXP+2)}$（扩散率）、WCND=DKSAT • FACTR2$^{(2 • BEXP+3)}$（导水率）、FACTR2=SMC/SMCMAX；**冻土修正**：SICEMAX>0 时按 VKWGT=1/(1+(500 • SICEMAX)³) 把 WDF 向 FACTR1=0.05/SMCMAX 的极小值混合——注释长文解释：冻融锋面附近未冻水垂直梯度极大、厚土层截断误差会给出虚假的巨大水汽迁移，只能经验约束（D_10cm 老版本用 0.2/SMCMAX）。冻土实验三件套（FRH2O/SNKSRC/WDFCND）至此齐了。

十二、温度插值三件 TBND、TDFCND、TMPAVG

```
!file: phys/module_sf_noahlsm.F    L4080-L4120    (subroutine TBND)
L04080        SUBROUTINE TBND (TU,TB,ZSOIL,ZBOT,K,NSOIL,TBND1)
L04081
L04082  ! ----------------------------------------------------------------------
L04083  ! SUBROUTINE TBND
L04084  ! ----------------------------------------------------------------------
L04085  ! CALCULATE TEMPERATURE ON THE BOUNDARY OF THE LAYER BY INTERPOLATION OF
L04086  ! THE MIDDLE LAYER TEMPERATURES
L04087  ! ----------------------------------------------------------------------
L04088        IMPLICIT NONE
L04089        INTEGER, INTENT(IN)       :: NSOIL
L04090        INTEGER                   :: K
L04091        REAL, INTENT(IN)          :: TB, TU, ZBOT
L04092        REAL, INTENT(OUT)         :: TBND1
L04093        REAL, DIMENSION(1:NSOIL), INTENT(IN)   :: ZSOIL
L04094        REAL                      :: ZB, ZUP
L04095        REAL, PARAMETER           :: T0 = 273.15
L04096
L04097  ! ----------------------------------------------------------------------
L04098  ! USE SURFACE TEMPERATURE ON THE TOP OF THE FIRST LAYER
L04099  ! ----------------------------------------------------------------------
L04100       IF (K == 1) THEN
L04101           ZUP = 0.
L04102        ELSE
L04103           ZUP = ZSOIL (K -1)
L04104        END IF
L04105  ! ----------------------------------------------------------------------
L04106  ! USE DEPTH OF THE CONSTANT BOTTOM TEMPERATURE WHEN INTERPOLATE
L04107  ! TEMPERATURE INTO THE LAST LAYER BOUNDARY
L04108  ! ----------------------------------------------------------------------
L04109        IF (K ==  NSOIL) THEN
L04110           ZB = 2.* ZBOT - ZSOIL (K)
L04111        ELSE
L04112           ZB = ZSOIL (K +1)
L04113        END IF
```

```
L04114  ! ----------------------------------------------------------------------
L04115  ! LINEAR INTERPOLATION BETWEEN THE AVERAGE LAYER TEMPERATURES
L04116  ! ----------------------------------------------------------------------
L04117
L04118        TBND1 = TU + (TB - TU)* (ZUP - ZSOIL (K))/ (ZUP - ZB)
L04119  ! ----------------------------------------------------------------------
L04120    END SUBROUTINE TBND
```

讲解:层界面温度线性插值:顶层上界面取地表(ZUP=0)、底层下界面外推 ZB=2•ZBOT−ZSOIL(NSOIL)(用恒定底边界温度补出对称点)；TBND1=TU+(TB−TU)•(ZUP−ZSOIL(K))/(ZUP−ZB)。供 HRT/TMPAVG 用。

```
!file: phys/module_sf_noahlsm.F    L4124-L4247    (subroutine TDFCND)
L04124        SUBROUTINE TDFCND ( DF, SMC, QZ, SMCMAX, SH2O, BEXP, PSISAT, SOILTYP, OPT_THCND)
L04125
L04126  ! ----------------------------------------------------------------------
L04127  ! SUBROUTINE TDFCND
L04128  ! ----------------------------------------------------------------------
L04129  ! CALCULATE THERMAL DIFFUSIVITY AND CONDUCTIVITY OF THE SOIL FOR A GIVEN
L04130  ! POINT AND TIME.
L04131  ! ----------------------------------------------------------------------
L04132  ! PETERS-LIDARD APPROACH (PETERS-LIDARD et al., 1998)
L04133  ! June 2001 CHANGES: FROZEN SOIL CONDITION.
L04134  ! ----------------------------------------------------------------------
L04135        IMPLICIT NONE
L04136        INTEGER, INTENT(IN)       :: SOILTYP, OPT_THCND
L04137        REAL, INTENT(IN)          :: QZ,  SMC, SMCMAX, SH2O, BEXP, PSISAT
L04138        REAL, INTENT(OUT)         :: DF
L04139        REAL                      :: AKE, GAMMD, THKDRY, THKICE, THKO,    &
L04140                                     THKQTZ,THKSAT,THKS,THKW,SATRATIO,XU, &
L04141                                     XUNFROZ,AKEI,AKEL,PSIF,PF
L04142
L04143  ! ----------------------------------------------------------------------
L04144  ! WE NOW GET QUARTZ AS AN INPUT ARGUMENT (SET IN ROUTINE REDPRM):
L04145  !      DATA QUARTZ /0.82, 0.10, 0.25, 0.60, 0.52,
L04146  !     &             0.35, 0.60, 0.40, 0.82/
L04147  ! ----------------------------------------------------------------------
L04148  ! IF THE SOIL HAS ANY MOISTURE CONTENT COMPUTE A PARTIAL SUM/PRODUCT
L04149  ! OTHERWISE USE A CONSTANT VALUE WHICH WORKS WELL WITH MOST SOILS
L04150  ! ----------------------------------------------------------------------
L04151  !  THKW ......WATER THERMAL CONDUCTIVITY
L04152  !  THKQTZ ....THERMAL CONDUCTIVITY FOR QUARTZ
L04153  !  THKO ......THERMAL CONDUCTIVITY FOR OTHER SOIL COMPONENTS
L04154  !  THKS ......THERMAL CONDUCTIVITY FOR THE SOLIDS COMBINED(QUARTZ+OTHER)
L04155  !  THKICE ....ICE THERMAL CONDUCTIVITY
L04156  !  SMCMAX ....POROSITY (= SMCMAX)
L04157  !  QZ .........QUARTZ CONTENT (SOIL TYPE DEPENDENT)
L04158  ! ----------------------------------------------------------------------
L04159  ! USE AS IN PETERS-LIDARD, 1998 (MODIF. FROM JOHANSEN, 1975).
L04160
L04161  !                                 PABLO GRUNMANN, 08/17/98
L04162  ! REFS.:
L04163  !      FAROUKI, O.T.,1986: THERMAL PROPERTIES OF SOILS. SERIES ON ROCK
L04164  !             AND SOIL MECHANICS, VOL. 11, TRANS TECH, 136 PP.
L04165  !      JOHANSEN, O., 1975: THERMAL CONDUCTIVITY OF SOILS. PH.D. THESIS,
L04166  !             UNIVERSITY OF TRONDHEIM,
L04167  !      PETERS-LIDARD, C. D., ET AL., 1998: THE EFFECT OF SOIL THERMAL
L04168  !             CONDUCTIVITY PARAMETERIZATION ON SURFACE ENERGY FLUXES
L04169  !             AND TEMPERATURES. JOURNAL OF THE ATMOSPHERIC SCIENCES,
L04170  !             VOL. 55, PP. 1209-1224.
L04171  ! ----------------------------------------------------------------------
L04172
L04173  IF ( OPT_THCND == 1 .OR. ( OPT_THCND == 2 .AND. (SOILTYP /= 4 .AND. SOILTYP /= 3)) )THEN
L04174
L04175  ! NEEDS PARAMETERS
L04176  ! POROSITY(SOIL TYPE):
L04177  !      POROS = SMCMAX
L04178  ! SATURATION RATIO:
L04179  ! PARAMETERS  W/(M.K)
L04180        SATRATIO = SMC / SMCMAX
L04181  ! ICE CONDUCTIVITY:
L04182        THKICE = 2.2
L04183  ! WATER CONDUCTIVITY:
L04184        THKW = 0.57
L04185  ! THERMAL CONDUCTIVITY OF "OTHER" SOIL COMPONENTS
L04186  !      IF (QZ .LE. 0.2) THKO = 3.0
L04187        THKO = 2.0
L04188  ! QUARTZ' CONDUCTIVITY
L04189        THKQTZ = 7.7
L04190  ! SOLIDS' CONDUCTIVITY
L04191        THKS = (THKQTZ ** QZ)* (THKO ** (1. - QZ))
L04192
L04193  ! UNFROZEN FRACTION (FROM 1., i.e., 100%LIQUID, TO 0. (100% FROZEN))
L04194        XUNFROZ = SH2O / SMC
L04195  ! UNFROZEN VOLUME FOR SATURATION (POROSITY*XUNFROZ)
L04196        XU = XUNFROZ * SMCMAX
L04197
L04198  ! SATURATED THERMAL CONDUCTIVITY
L04199        THKSAT = THKS ** (1. - SMCMAX)* THKICE ** (SMCMAX - XU)* THKW **   &
L04200           (XU)
L04201
L04202  ! DRY DENSITY IN KG/M3
L04203        GAMMD = (1. - SMCMAX)*2700.
L04204
L04205  ! DRY THERMAL CONDUCTIVITY IN W.M-1.K-1
L04206        THKDRY = (0.135* GAMMD+ 64.7)/ (2700. - 0.947* GAMMD)
L04207  ! FROZEN
L04208           AKEI = SATRATIO
L04209  ! UNFROZEN
L04210  ! RANGE OF VALIDITY FOR THE KERSTEN NUMBER (AKE)
L04211
L04212  ! KERSTEN NUMBER (USING "FINE" FORMULA, VALID FOR SOILS CONTAINING AT
L04213  ! LEAST 5% OF PARTICLES WITH DIAMETER LESS THAN 2.E-6 METERS.)
L04214  ! (FOR "COARSE" FORMULA, SEE PETERS-LIDARD ET AL., 1998).
L04215
L04216           IF ( SATRATIO >  0.1 ) THEN
```

```
L04217
L04218              AKEL = LOG10 (SATRATIO) + 1.0
L04219
L04220  ! USE K = KDRY
L04221            ELSE
L04222
L04223               AKEL = 0.0
L04224            END IF
L04225         AKE = ((SMC-SH2O)*AKEI + SH2O*AKEL)/SMC
L04226  !  THERMAL CONDUCTIVITY
L04227
L04228
L04229         DF = AKE * (THKSAT - THKDRY) + THKDRY
L04230
L04231       ELSE
L04232
L04233  ! use the Mccumber and Pielke approach for silt loam (4), sandy loam (3)
L04234
L04235            PSIF = PSISAT*100.*(SMCMAX/(SMC))**BEXP
L04236  !--- PSIF should be in [CM] to compute PF
L04237             PF=log10(abs(PSIF))
L04238  !--- HK is for McCumber thermal conductivity
L04239            IF(PF.LE.5.1) THEN
L04240              DF=420.*EXP(-(PF+2.7))
L04241            ELSE
L04242              DF=.1744
L04243            END IF
L04244
L04245       ENDIF  ! for OPT_THCND OPTIONS
L04246  ! ----------------------------------------------------------------------
L04247    END SUBROUTINE TDFCND
```

讲解：土壤导热两方案（OPT_THCND namelist 选择）：**=1（默认）Peters-Lidard 1998**（JAS 55, 1209，Johansen 1975 改）：固体导热 THKS=THKQTZ^{QZ} • THKO$^{(1-QZ)}$（石英 7.7、其他 2.0 W/(m • K) 的对数混合）、饱和导热 THKSAT=THKS$^{(1-SMCMAX)}$ • 冰$^{(SMCMAX-XU)}$ • 水 ^XU、干土 THKDRY=(0.135 • GAMMD+64.7)/(2700−0.947 • GAMMD)（GAMMD=(1−SMCMAX) • 2700 为干密度）、KERSTEN 数冻结相 AKEI=SATRATIO、未冻相 AKEL=log10(SATRATIO)+1（SATRATIO≤0.1 时 0），按液态水份额加权 AKE 后 DF=AKE • (THKSAT−THKDRY)+THKDRY；**=2 且土壤类 3/4（砂壤/粉砂壤）走 McCumber-Pielke 老式**：PSIF=PSISAT • 100 • (SMCMAX/SMC)^BEXP、PF=log10|PSIF|、PF≤5.1 时 DF=420 • exp(−(PF+2.7)) 否则 0.1744。文件头注释保留了老版按 QTZ 分 THKO=3.0 的写法——导热方案改造（如加第三套）就在这个 IF 分支上动刀。

```
!file: phys/module_sf_noahlsm.F    L4250-L4352    (subroutine TMPAVG)
L04250        SUBROUTINE TMPAVG (TAVG,TUP,TM,TDN,ZSOIL,NSOIL,K)
L04251
L04252  ! ----------------------------------------------------------------------
L04253  ! SUBROUTINE TMPAVG
L04254  ! ----------------------------------------------------------------------
L04255  ! CALCULATE SOIL LAYER AVERAGE TEMPERATURE (TAVG) IN FREEZING/THAWING
L04256  ! LAYER USING UP, DOWN, AND MIDDLE LAYER TEMPERATURES (TUP, TDN, TM),
L04257  ! WHERE TUP IS AT TOP BOUNDARY OF LAYER, TDN IS AT BOTTOM BOUNDARY OF
L04258  ! LAYER.  TM IS LAYER PROGNOSTIC STATE TEMPERATURE.
L04259  ! ----------------------------------------------------------------------
L04260        IMPLICIT NONE
L04261        INTEGER  K
L04262
L04263        INTEGER  NSOIL
L04264        REAL     DZ
L04265        REAL     DZH
L04266        REAL     T0
L04267        REAL     TAVG
L04268        REAL     TDN
L04269        REAL     TM
L04270        REAL     TUP
L04271        REAL     X0
L04272        REAL     XDN
L04273        REAL     XUP
L04274
L04275        REAL     ZSOIL (NSOIL)
L04276
L04277  ! ----------------------------------------------------------------------
L04278        PARAMETER (T0 = 2.7315E2)
L04279        IF (K .eq. 1) THEN
L04280           DZ = - ZSOIL (1)
L04281        ELSE
L04282           DZ = ZSOIL (K -1) - ZSOIL (K)
L04283        END IF
L04284
L04285        DZH = DZ *0.5
L04286        IF (TUP .lt. T0) THEN
L04287           IF (TM .lt. T0) THEN
L04288  ! ----------------------------------------------------------------------
L04289  ! TUP, TM, TDN < T0
L04290  ! ----------------------------------------------------------------------
L04291              IF (TDN .lt. T0) THEN
L04292                 TAVG = (TUP + 2.0* TM + TDN)/ 4.0
L04293  ! ----------------------------------------------------------------------
L04294  ! TUP & TM < T0,  TDN .ge. T0
L04295  ! ----------------------------------------------------------------------
L04296              ELSE
L04297                 X0 = (T0- TM) * DZH / (TDN - TM)
L04298                 TAVG = 0.5 * (TUP * DZH + TM * (DZH + X0) + T0* (        &
L04299       &                2.* DZH - X0)) / DZ
L04300              END IF
L04301           ELSE
L04302  ! ----------------------------------------------------------------------
L04303  ! TUP < T0, TM .ge. T0, TDN < T0
L04304  ! ----------------------------------------------------------------------
L04305              IF (TDN .lt. T0) THEN
L04306                 XUP = (T0- TUP) * DZH / (TM - TUP)
```

```
L04307                XDN = DZH - (T0- TM) * DZH / (TDN - TM)
L04308                TAVG = 0.5 * (TUP * XUP + T0* (2.* DZ - XUP - XDN)       &
L04309       &                 + TDN * XDN) / DZ
L04310    ! ----------------------------------------------------------------------
L04311    ! TUP < T0, TM .ge. T0, TDN .ge. T0
L04312    ! ----------------------------------------------------------------------
L04313              ELSE
L04314                XUP = (T0- TUP) * DZH / (TM - TUP)
L04315                TAVG = 0.5 * (TUP * XUP + T0* (2.* DZ - XUP)) / DZ
L04316              END IF
L04317            END IF
L04318         ELSE
L04319            IF (TM .lt. T0) THEN
L04320    ! ----------------------------------------------------------------------
L04321    ! TUP .ge. T0, TM < T0, TDN < T0
L04322    ! ----------------------------------------------------------------------
L04323              IF (TDN .lt. T0) THEN
L04324                XUP = DZH - (T0- TUP) * DZH / (TM - TUP)
L04325                TAVG = 0.5 * (T0* (DZ - XUP) + TM * (DZH + XUP)          &
L04326       &                 + TDN * DZH) / DZ
L04327    ! ----------------------------------------------------------------------
L04328    ! TUP .ge. T0, TM < T0, TDN .ge. T0
L04329    ! ----------------------------------------------------------------------
L04330              ELSE
L04331                XUP = DZH - (T0- TUP) * DZH / (TM - TUP)
L04332                XDN = (T0- TM) * DZH / (TDN - TM)
L04333                TAVG = 0.5 * (T0* (2.* DZ - XUP - XDN) + TM *            &
L04334       & (XUP + XDN)) / DZ
L04335              END IF
L04336            ELSE
L04337    ! ----------------------------------------------------------------------
L04338    ! TUP .ge. T0, TM .ge. T0, TDN < T0
L04339    ! ----------------------------------------------------------------------
L04340              IF (TDN .lt. T0) THEN
L04341                XDN = DZH - (T0- TM) * DZH / (TDN - TM)
L04342                TAVG = (T0* (DZ - XDN) +0.5* (T0+ TDN)* XDN) / DZ
L04343    ! ----------------------------------------------------------------------
L04344    ! TUP .ge. T0, TM .ge. T0, TDN .ge. T0
L04345    ! ----------------------------------------------------------------------
L04346              ELSE
L04347                TAVG = (TUP + 2.0* TM + TDN) / 4.0
L04348              END IF
L04349            END IF
L04350         END IF
L04351    ! ----------------------------------------------------------------------
L04352      END SUBROUTINE TMPAVG
```

讲解：冻融层平均温度：TUP/TM/TDN 相对 T0=273.15K 的八种冷热组合分区——先线性定位 0°C 等温面（X0/XUP/XDN），层内分段线性积分求平均；全冻或全融时退化为 (TUP+2 • TM+TDN)/4（辛普森式）。给 SNKSRC 提供相变用的代表温度。

十三、蒸腾 TRANSP

```
!file: phys/module_sf_noahlsm.F    L4355-L4458    (subroutine TRANSP)
L04355         SUBROUTINE TRANSP (ET,NSOIL,ETP1,SMC,CMC,ZSOIL,SHDFAC,SMCWLT,     &
L04356       &                       CMCMAX,PC,CFACTR,SMCREF,SFCTMP,Q2,NROOT,    &
L04357       &                       RTDIS)
L04358
L04359    ! ----------------------------------------------------------------------
L04360    ! SUBROUTINE TRANSP
L04361    ! ----------------------------------------------------------------------
L04362    ! CALCULATE TRANSPIRATION FOR THE VEG CLASS.
L04363    ! ----------------------------------------------------------------------
L04364         IMPLICIT NONE
L04365         INTEGER  I
L04366         INTEGER  K
L04367         INTEGER  NSOIL
L04368
L04369         INTEGER  NROOT
L04370         REAL     CFACTR
L04371         REAL     CMC
L04372         REAL     CMCMAX
L04373         REAL     DENOM
L04374         REAL     ET (NSOIL)
L04375         REAL     ETP1
L04376         REAL     ETP1A
L04377    !.....REAL PART(NSOIL)
L04378         REAL     GX (NROOT)
L04379         REAL     PC
L04380         REAL     Q2
L04381         REAL     RTDIS (NSOIL)
L04382         REAL     RTX
L04383         REAL     SFCTMP
L04384         REAL     SGX
L04385         REAL     SHDFAC
L04386         REAL     SMC (NSOIL)
L04387         REAL     SMCREF
L04388         REAL     SMCWLT
L04389
L04390    ! ----------------------------------------------------------------------
L04391    ! INITIALIZE PLANT TRANSP TO ZERO FOR ALL SOIL LAYERS.
L04392    ! ----------------------------------------------------------------------
L04393         REAL     ZSOIL (NSOIL)
L04394         DO K = 1,NSOIL
L04395            ET (K) = 0.
L04396    ! ----------------------------------------------------------------------
L04397    ! CALCULATE AN 'ADJUSTED' POTENTIAL TRANSPIRATION
L04398    ! IF STATEMENT BELOW TO AVOID TANGENT LINEAR PROBLEMS NEAR ZERO
L04399    ! NOTE: GX AND OTHER TERMS BELOW REDISTRIBUTE TRANSPIRATION BY LAYER,
L04400    ! ET(K), AS A FUNCTION OF SOIL MOISTURE AVAILABILITY, WHILE PRESERVING
L04401    ! TOTAL ETP1A.
L04402    ! ----------------------------------------------------------------------
```

```
L04403        END DO
L04404        IF (CMC .ne. 0.0) THEN
L04405           ETP1A = SHDFAC * PC * ETP1 * (1.0- (CMC / CMCMAX) ** CFACTR)
L04406        ELSE
L04407           ETP1A = SHDFAC * PC * ETP1
L04408        END IF
L04409        SGX = 0.0
L04410        DO I = 1,NROOT
L04411           GX (I) = ( SMC (I) - SMCWLT ) / ( SMCREF - SMCWLT )
L04412           GX (I) = MAX ( MIN ( GX (I), 1. ), 0. )
L04413           SGX = SGX + GX (I)
L04414        END DO
L04415
L04416        SGX = SGX / NROOT
L04417        DENOM = 0.
L04418        DO I = 1,NROOT
L04419           RTX = RTDIS (I) + GX (I) - SGX
L04420           GX (I) = GX (I) * MAX ( RTX, 0. )
L04421           DENOM = DENOM + GX (I)
L04422        END DO
L04423
L04424        IF (DENOM .le. 0.0) DENOM = 1.
L04425        DO I = 1,NROOT
L04426           ET (I) = ETP1A * GX (I) / DENOM
L04427  ! ----------------------------------------------------------------------
L04428  ! ABOVE CODE ASSUMES A VERTICALLY UNIFORM ROOT DISTRIBUTION
L04429  ! CODE BELOW TESTS A VARIABLE ROOT DISTRIBUTION
L04430  ! ----------------------------------------------------------------------
L04431  !      ET(1) = ( ZSOIL(1) / ZSOIL(NROOT) ) * GX * ETP1A
L04432  !      ET(1) = ( ZSOIL(1) / ZSOIL(NROOT) ) * ETP1A
L04433  ! ----------------------------------------------------------------------
L04434  ! USING ROOT DISTRIBUTION AS WEIGHTING FACTOR
L04435  ! ----------------------------------------------------------------------
L04436  !      ET(1) = RTDIS(1) * ETP1A
L04437  !      ET(1) = ETP1A * PART(1)
L04438  ! ----------------------------------------------------------------------
L04439  ! LOOP DOWN THRU THE SOIL LAYERS REPEATING THE OPERATION ABOVE,
L04440  ! BUT USING THE THICKNESS OF THE SOIL LAYER (RATHER THAN THE
L04441  ! ABSOLUTE DEPTH OF EACH LAYER) IN THE FINAL CALCULATION.
L04442  ! ----------------------------------------------------------------------
L04443  !      DO K = 2,NROOT
L04444  !        GX = ( SMC(K) - SMCWLT ) / ( SMCREF - SMCWLT )
L04445  !        GX = MAX ( MIN ( GX, 1. ), 0. )
L04446  ! TEST CANOPY RESISTANCE
L04447  !        GX = 1.0
L04448  !        ET(K) = ((ZSOIL(K)-ZSOIL(K-1))/ZSOIL(NROOT))*GX*ETP1A
L04449  !        ET(K) = ((ZSOIL(K)-ZSOIL(K-1))/ZSOIL(NROOT))*ETP1A
L04450  ! ----------------------------------------------------------------------
L04451  ! USING ROOT DISTRIBUTION AS WEIGHTING FACTOR
L04452  ! ----------------------------------------------------------------------
L04453  !        ET(K) = RTDIS(K) * ETP1A
L04454  !        ET(K) = ETP1A*PART(K)
L04455  !      END DO
L04456        END DO
L04457  ! ----------------------------------------------------------------------
L04458    END SUBROUTINE TRANSP
```

讲解：逐层蒸腾分配：ETP1A=SHDFAC • PC • ETP1 • (1−(CMC/CMCMAX)^CFACTR)——冠层湿润时气孔受抑（与 EVAPO 里 EC 用同一 (CMC/CMCMAX)^CFACTR 函数，两处共享参数 CFACTR）；GX=(SMC−SMCWLT)/(SMCREF−SMCWLT) 钳位 0–1；**RTX=RTDIS(I)+GX(I)−SGX**——根密度权重与水分可用性偏离均值的联合权重，负则砍零；GX 再乘 RTX、按 DENOM 归一后 ET(I)=ETP1A • GX/DENOM——保持总蒸腾 =ETP1A 的同时向「湿根层」倾斜；DENOM≤0（全枯）时 DENOM=1、ET 全零。注释保留按层厚、按 RTDIS 等多种备选分配。

十四、老版地表交换系数 SFCDIF_off

```
!file: phys/module_sf_noahlsm.F    L4522-L4756    (subroutine SFCDIF_off)
L04522        SUBROUTINE SFCDIF_off (ZLM,Z0,THZ0,THLM,SFCSPD,CZIL,AKMS,AKHS)
L04523
L04524  ! ----------------------------------------------------------------------
L04525  ! SUBROUTINE SFCDIF (renamed SFCDIF_off to avoid clash with Eta PBL)
L04526  ! ----------------------------------------------------------------------
L04527  ! CALCULATE SURFACE LAYER EXCHANGE COEFFICIENTS VIA ITERATIVE PROCESS.
L04528  ! SEE CHEN ET AL (1997, BLM)
L04529  ! ----------------------------------------------------------------------
L04530
L04531        IMPLICIT NONE
L04532        REAL     WWST, WWST2, G, VKRM, EXCM, BETA, BTG, ELFC, WOLD, WNEW
L04533        REAL     PIHF, EPSU2, EPSUST, EPSIT, EPSA, ZTMIN, ZTMAX, HPBL,     &
L04534       & SQVISC
L04535        REAL     RIC, RRIC, FHNEU, RFC, RFAC, ZZ, PSLMU, PSLMS, PSLHU,     &
L04536       & PSLHS
L04537        REAL     XX, PSPMU, YY, PSPMS, PSPHU, PSPHS, ZLM, Z0, THZ0, THLM
L04538        REAL     SFCSPD, CZIL, AKMS, AKHS, ZILFC, ZU, ZT, RDZ, CXCH
L04539        REAL     DTHV, DU2, BTGH, WSTAR2, USTAR, ZSLU, ZSLT, RLOGU, RLOGT
L04540        REAL     RLMO, ZETALT, ZETALU, ZETAU, ZETAT, XLU4, XLT4, XU4, XT4
L04541  !CC    ......REAL ZTFC
L04542
L04543        REAL     XLU, XLT, XU, XT, PSMZ, SIMM, PSHZ, SIMH, USTARK, RLMN,  &
L04544       &         RLMA
L04545
L04546        INTEGER  ITRMX, ILECH, ITR
L04547        PARAMETER                                                          &
L04548       &        (WWST = 1.2,WWST2 = WWST * WWST,G = 9.8,VKRM = 0.40,       &
L04549       &         EXCM = 0.001                                              &
L04550       &        ,BETA = 1./270.,BTG = BETA * G,ELFC = VKRM * BTG           &
L04551       &                  ,WOLD =.15,WNEW = 1. - WOLD,ITRMX = 05,          &
L04552       &                   PIHF = 3.14159265/2.)
```

```
L04553        PARAMETER                                                       &
L04554       &         (EPSU2 = 1.E-4,EPSUST = 0.07,EPSIT = 1.E-4,EPSA = 1.E-8  &
L04555       &         ,ZTMIN = -5.,ZTMAX = 1.,HPBL = 1000.0                    &
L04556       &          ,SQVISC = 258.2)
L04557        PARAMETER                                                       &
L04558       &       (RIC = 0.183,RRIC = 1.0/ RIC,FHNEU = 0.8,RFC = 0.191      &
L04559       &        ,RFAC = RIC / (FHNEU * RFC * RFC))
L04560
L04561  ! ----------------------------------------------------------------------
L04562  ! NOTE: THE TWO CODE BLOCKS BELOW DEFINE FUNCTIONS
L04563  ! ----------------------------------------------------------------------
L04564  ! LECH'S SURFACE FUNCTIONS
L04565  ! ----------------------------------------------------------------------
L04566        PSLMU (ZZ)= -0.96* log (1.0-4.5* ZZ)
L04567        PSLMS (ZZ)= ZZ * RRIC -2.076* (1. -1./ (ZZ +1.))
L04568        PSLHU (ZZ)= -0.96* log (1.0-4.5* ZZ)
L04569
L04570  ! ----------------------------------------------------------------------
L04571  ! PAULSON'S SURFACE FUNCTIONS
L04572  ! ----------------------------------------------------------------------
L04573        PSLHS (ZZ)= ZZ * RFAC -2.076* (1. -1./ (ZZ +1.))
L04574        PSPMU (XX)= -2.* log ( (XX +1.)*0.5) - log ( (XX * XX +1.)*0.5)   &
L04575       &       +2.* ATAN (XX)                                             &
L04576       &- PIHF
L04577        PSPMS (YY)= 5.* YY
L04578        PSPHU (XX)= -2.* log ( (XX * XX +1.)*0.5)
L04579
L04580  ! ----------------------------------------------------------------------
L04581  ! THIS ROUTINE SFCDIF CAN HANDLE BOTH OVER OPEN WATER (SEA, OCEAN) AND
L04582  ! OVER SOLID SURFACE (LAND, SEA-ICE).
L04583  ! ----------------------------------------------------------------------
L04584        PSPHS (YY)= 5.* YY
L04585
L04586  ! ----------------------------------------------------------------------
L04587  !     ZTFC: RATIO OF ZOH/ZOM  LESS OR EQUAL THAN 1
L04588  !     C......ZTFC=0.1
L04589  !     CZIL: CONSTANT C IN Zilitinkevich, S. S.1995,:NOTE ABOUT ZT
L04590  ! ----------------------------------------------------------------------
L04591        ILECH = 0
L04592
L04593  ! ----------------------------------------------------------------------
L04594        ZILFC = - CZIL * VKRM * SQVISC
L04595  !     C.......ZT=Z0*ZTFC
L04596        ZU = Z0
L04597        RDZ = 1./ ZLM
L04598        CXCH = EXCM * RDZ
L04599        DTHV = THLM - THZ0
L04600
L04601  ! ----------------------------------------------------------------------
L04602  ! BELJARS CORRECTION OF USTAR
L04603  ! ----------------------------------------------------------------------
L04604        DU2 = MAX (SFCSPD * SFCSPD,EPSU2)
L04605  !cc   If statements to avoid TANGENT LINEAR problems near zero
L04606        BTGH = BTG * HPBL
L04607        IF (BTGH * AKHS * DTHV .ne. 0.0) THEN
L04608           WSTAR2 = WWST2* ABS (BTGH * AKHS * DTHV)** (2./3.)
L04609        ELSE
L04610           WSTAR2 = 0.0
L04611        END IF
L04612
L04613  ! ----------------------------------------------------------------------
L04614  ! ZILITINKEVITCH APPROACH FOR ZT
L04615  ! ----------------------------------------------------------------------
L04616        USTAR = MAX (SQRT (AKMS * SQRT (DU2+ WSTAR2)),EPSUST)
L04617
L04618  ! ----------------------------------------------------------------------
L04619        ZT = EXP (ZILFC * SQRT (USTAR * Z0))* Z0
L04620        ZSLU = ZLM + ZU
L04621  !     PRINT*,'ZSLT=',ZSLT
L04622  !     PRINT*,'ZLM=',ZLM
L04623  !     PRINT*,'ZT=',ZT
L04624
L04625        ZSLT = ZLM + ZT
L04626        RLOGU = log (ZSLU / ZU)
L04627
L04628        RLOGT = log (ZSLT / ZT)
L04629  !     PRINT*,'RLMO=',RLMO
L04630  !     PRINT*,'ELFC=',ELFC
L04631  !     PRINT*,'AKHS=',AKHS
L04632  !     PRINT*,'DTHV=',DTHV
L04633  !     PRINT*,'USTAR=',USTAR
L04634
L04635        RLMO = ELFC * AKHS * DTHV / USTAR **3
L04636  ! ----------------------------------------------------------------------
L04637  ! 1./MONIN-OBUKKHOV LENGTH-SCALE
L04638  ! ----------------------------------------------------------------------
L04639        DO ITR = 1,ITRMX
L04640           ZETALT = MAX (ZSLT * RLMO,ZTMIN)
L04641           RLMO = ZETALT / ZSLT
L04642           ZETALU = ZSLU * RLMO
L04643           ZETAU = ZU * RLMO
L04644
L04645           ZETAT = ZT * RLMO
L04646           IF (ILECH .eq. 0) THEN
L04647              IF (RLMO .lt. 0.)THEN
L04648                 XLU4 = 1. -16.* ZETALU
L04649                 XLT4 = 1. -16.* ZETALT
L04650                 XU4 = 1. -16.* ZETAU
L04651
L04652                 XT4 = 1. -16.* ZETAT
L04653                 XLU = SQRT (SQRT (XLU4))
L04654                 XLT = SQRT (SQRT (XLT4))
L04655                 XU = SQRT (SQRT (XU4))
L04656
L04657                 XT = SQRT (SQRT (XT4))
L04658  !     PRINT*,'-----------1------------'
L04659  !     PRINT*,'PSMZ=',PSMZ
L04660  !     PRINT*,'PSPMU(ZETAU)=',PSPMU(ZETAU)
L04661  !     PRINT*,'XU=',XU
L04662  !     PRINT*,'------------------------'
```

```
L04663                PSMZ = PSPMU (XU)
L04664                SIMM = PSPMU (XLU) - PSMZ + RLOGU
L04665                PSHZ = PSPHU (XT)
L04666                SIMH = PSPHU (XLT) - PSHZ + RLOGT
L04667             ELSE
L04668                ZETALU = MIN (ZETALU,ZTMAX)
L04669                ZETALT = MIN (ZETALT,ZTMAX)
L04670  !     PRINT*,'-----------2------------'
L04671  !     PRINT*,'PSMZ=',PSMZ
L04672  !     PRINT*,'PSPMS(ZETAU)=',PSPMS(ZETAU)
L04673  !     PRINT*,'ZETAU=',ZETAU
L04674  !     PRINT*,'------------------------'
L04675                PSMZ = PSPMS (ZETAU)
L04676                SIMM = PSPMS (ZETALU) - PSMZ + RLOGU
L04677                PSHZ = PSPHS (ZETAT)
L04678                SIMH = PSPHS (ZETALT) - PSHZ + RLOGT
L04679             END IF
L04680  ! ---------------------------------------------------------------------
L04681  ! LECH'S FUNCTIONS
L04682  ! ---------------------------------------------------------------------
L04683           ELSE
L04684              IF (RLMO .lt. 0.)THEN
L04685  !     PRINT*,'-----------3------------'
L04686  !     PRINT*,'PSMZ=',PSMZ
L04687  !     PRINT*,'PSLMU(ZETAU)=',PSLMU(ZETAU)
L04688  !     PRINT*,'ZETAU=',ZETAU
L04689  !     PRINT*,'------------------------'
L04690                PSMZ = PSLMU (ZETAU)
L04691                SIMM = PSLMU (ZETALU) - PSMZ + RLOGU
L04692                PSHZ = PSLHU (ZETAT)
L04693                SIMH = PSLHU (ZETALT) - PSHZ + RLOGT
L04694             ELSE
L04695                ZETALU = MIN (ZETALU,ZTMAX)
L04696
L04697                ZETALT = MIN (ZETALT,ZTMAX)
L04698  !     PRINT*,'-----------4------------'
L04699  !     PRINT*,'PSMZ=',PSMZ
L04700  !     PRINT*,'PSLMS(ZETAU)=',PSLMS(ZETAU)
L04701  !     PRINT*,'ZETAU=',ZETAU
L04702  !     PRINT*,'------------------------'
L04703                PSMZ = PSLMS (ZETAU)
L04704                SIMM = PSLMS (ZETALU) - PSMZ + RLOGU
L04705                PSHZ = PSLHS (ZETAT)
L04706                SIMH = PSLHS (ZETALT) - PSHZ + RLOGT
L04707             END IF
L04708  ! ---------------------------------------------------------------------
L04709  ! BELJAARS CORRECTION FOR USTAR
L04710  ! ---------------------------------------------------------------------
L04711           END IF
L04712
L04713  ! ---------------------------------------------------------------------
L04714  ! ZILITINKEVITCH FIX FOR ZT
L04715  ! ---------------------------------------------------------------------
L04716           USTAR = MAX (SQRT (AKMS * SQRT (DU2+ WSTAR2)),EPSUST)
L04717
L04718           ZT = EXP (ZILFC * SQRT (USTAR * Z0))* Z0
L04719           ZSLT = ZLM + ZT
L04720  !---------------------------------------------------------------------
L04721           RLOGT = log (ZSLT / ZT)
L04722           USTARK = USTAR * VKRM
L04723           AKMS = MAX (USTARK / SIMM,CXCH)
L04724  !---------------------------------------------------------------------
L04725  ! IF STATEMENTS TO AVOID TANGENT LINEAR PROBLEMS NEAR ZERO
L04726  !---------------------------------------------------------------------
L04727           AKHS = MAX (USTARK / SIMH,CXCH)
L04728           IF (BTGH * AKHS * DTHV .ne. 0.0) THEN
L04729              WSTAR2 = WWST2* ABS (BTGH * AKHS * DTHV)** (2./3.)
L04730           ELSE
L04731              WSTAR2 = 0.0
L04732           END IF
L04733  !---------------------------------------------------------------------
L04734           RLMN = ELFC * AKHS * DTHV / USTAR **3
L04735  !---------------------------------------------------------------------
L04736  !     IF(ABS((RLMN-RLMO)/RLMA).LT.EPSIT)    GO TO 110
L04737  !---------------------------------------------------------------------
L04738           RLMA = RLMO * WOLD+ RLMN * WNEW
L04739  !---------------------------------------------------------------------
L04740           RLMO = RLMA
L04741  !     PRINT*,'----------------------------'
L04742  !     PRINT*,'SFCDIF OUTPUT !  ! ! ! ! ! ! ! !  !   !    !'
L04743
L04744  !     PRINT*,'ZLM=',ZLM
L04745  !     PRINT*,'Z0=',Z0
L04746  !     PRINT*,'THZ0=',THZ0
L04747  !     PRINT*,'THLM=',THLM
L04748  !     PRINT*,'SFCSPD=',SFCSPD
L04749  !     PRINT*,'CZIL=',CZIL
L04750  !     PRINT*,'AKMS=',AKMS
L04751  !     PRINT*,'AKHS=',AKHS
L04752  !     PRINT*,'----------------------------'
L04753
L04754        END DO
L04755  ! ---------------------------------------------------------------------
L04756    END SUBROUTINE SFCDIF_off
```

讲解：Chen et al. 1997（BLM）地表交换系数的 Noah 自带版（与 Eta PBL 的 sfcdif 同源，为避免重名加 _off 后缀）：①Paulson 1970 不稳定函数 + 稳定区 Webb 修正（PSMU/PSMS/PHU/PHS 四个语句函数），Lech 函数组保留但 ILECH=0 关闭；②**Zilitinkevich 1995 热粗糙度**：ZT=exp(ZILFC • sqrt(USTAR • Z0)) • Z0、ZILFC=−CZIL • VKRM • SQVISC（SQVISC=258.2）——CZIL 是 namelist 可调的标定参数；③Beljaars 对流速度修正 WSTAR2=WWST2 • |BTGH • AKHS • DTHV|^(2/3)——弱风强层结时湍流不塌缩；④ITRMX=5 次迭代解 Monin–Obukhov 长度（RLMO 松弛 WOLD=0.15），AKMS/AKHS 取 MAX(USTARK/SIMM, CXCH)（EXCM=0.001 的下限防病态）。**WRF 主线中它**

是休眠代码：SFLX 里 LCH 分支整段注释（12a § 四），CH 一律由 PBL 面层方案传入（code_10 MYNN 面层等）；SFCDIF_off 活跃于离线 HRLDAS 与旧耦合。收录价值：①Noah 面层的「原貌」，与 ch14 各面层方案对照；② 论文 2 做 CH 敏感性实验时的直接改造对象。

十五、改造入口

1. **参数表改造**：REDPRM 是 TBL 到物理量的唯一出口——换土壤/植被数据集改 TBL 或在此加分支；KDT、FRZFACT 两个标定式也在此。
2. **径流方案**：SRT 的 Schaake 入渗（KDT/REFKDT/FRZX）+ SLOPE 底流——对照 Noah-MP 换 Green-Ampt 的教学点。
3. **冻土实验三件套**：FRH2O（12a）+ SNKSRC + WDFCND——相变平衡、潜热钳位、水力冻结修正各司其职。
4. **雪物理全家**：SNOWPACK/SNOWZ0/SNOW_NEW/SNFRAC/CSNOW 全在本文件——改进雪盖模拟不需要出模块。
5. **导热方案**：TDFCND 的 OPT_THCND 分支——加第三套方案（如考虑有机质/冻融循环修正）的位置。
6. **面层交换**：SFCDIF_off 是 Noah 自带 CH 的后门——LCH 通道打开（恢复 SFLX 注释段）即可启用，做 CH 敏感性实验用。

十六、上下游链接

- 上半（文件头、SFLX 主入口、PENMAN/CANRES/EVAPO/DEVAP/FRH2O/HRT/HSTEP/NOPAC）：`code_12a_module_sf_noahlsm.md`
- 驱动层（逐格点循环、TBL 读取、海冰/冰川分支）：`code_13_module_sf_noahdrv.md`
- 面层对照：`code_10_module_sf_mynnsfc.md`；同族陆面方案对照见教材 ch16_land_surface_family.md
- 教材正文：ch16_land_surface_family.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_13 — Noah 驱动外壳（module_sf_noahdrv.F：格点循环、海冰/冰川分派与 TBL 读取）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_noahdrv.F	5447	e0b6ac00686774a44b7fd04d1d28d249

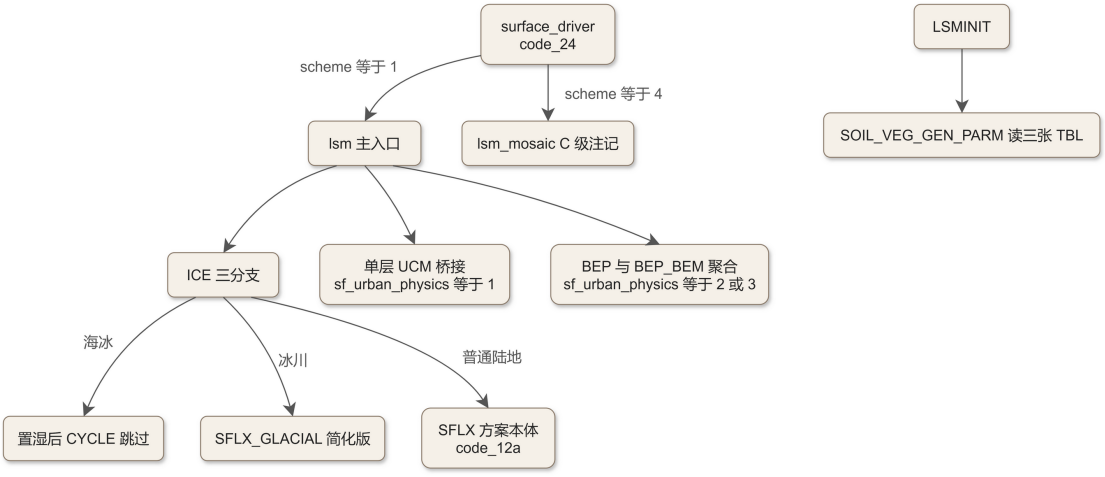
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B+C）：B 级节选——lsm 主入口九段、LSMINIT 两段、SOIL_VEG_GEN_PARM 一段；C 级注记——lsm_mosaic 接口索引、lsm_mosaic_init 全文不录。方案本体见 `code_12a_module_sf_noahlsm.md` 与 `code_12b_module_sf_noahlsm.md`。

二、定位

一句话：surface_driver 与 Noah 方案本体之间的**逐格点驱动外壳**——把 Registry 三维场切成格点标量、完成能量/水分单位换算、按海冰/冰川/普通陆地三分支分派、桥接三类城市方案（单层 UCM、BEP、BEP+BEM）、把结果回写并聚合。

谁在调它：module_surface_driver.F 按 sf_surface_scheme 分派（Noah=1 调 lsm、mosaic=4 调 lsm_mosaic），见 `code_24_module_surface_driver.md`。教材锚点：ch16_land_surface_family.md（驱动链与 TBL 机制）、ch17_urban_canopy.md（城市桥接）、ch20_modifying_wrf.md（灌溉与海冰改造）。

5 单元地图（lsm 1780 行 / LSMINIT 206 行 / SOIL_VEG_GEN_PARM 342 行 / lsm_mosaic 2647 行 / lsm_mosaic_init 403 行）：



三、文件头：依赖清单

```
!file: phys/module_sf_noahdrv.F    L1-L33
L00001  MODULE module_sf_noahdrv
L00002
L00003  !------------------------------
L00004    USE module_sf_noahlsm,  only: SFLX, XLF, XLV, CP, R_D, RHOWATER, NATURAL, SHDTBL, LUTYPE, SLTYPE, STBOLT, &
L00005        &                         KARMAN, LUCATS, NROTBL, RSTBL, RGLTBL, HSTBL, SNUPTBL, MAXALB, LAIMINTBL,   &
L00006        &                         LAIMAXTBL, Z0MINTBL, Z0MAXTBL, ALBEDOMINTBL, ALBEDOMAXTBL, EMISSMINTBL,     &
L00007        &                         EMISSMAXTBL, TOPT_DATA, CMCMAX_DATA, CFACTR_DATA, RSMAX_DATA, BARE, NLUS,   &
L00008        &                         SLCATS, BB, DRYSMC, F11, MAXSMC, REFSMC, SATPSI, SATDK, SATDW, WLTSMC, QTZ, &
L00009        &                         NSLTYPE, SLPCATS, SLOPE_DATA, SBETA_DATA, FXEXP_DATA, CSOIL_DATA,           &
L00010        &                         SALP_DATA, REFDK_DATA, REFKDT_DATA, FRZK_DATA, ZBOT_DATA, CZIL_DATA,        &
L00011        &                         SMLOW_DATA, SMHIGH_DATA, LVCOEF_DATA, NSLOPE, &
L00012        &                         FRH2O,ZTOPVTBL,ZBOTVTBL, &
L00013        &                         LCZ_1,LCZ_2,LCZ_3,LCZ_4,LCZ_5,LCZ_6,LCZ_7,LCZ_8,LCZ_9,LCZ_10,LCZ_11
L00014    USE module_sf_urban,    only: urban, oasis, IRI_SCHEME
L00015    USE module_sf_noahlsm_glacial_only, only: sflx_glacial
L00016    USE module_sf_bep,      only: bep
L00017    USE module_sf_bep_bem,  only: bep_bem
L00018  #if defined(mpas)
L00019  use mpas_atmphys_date_time, only: cal_mon_day
L00020  use mpas_atmphys_utilities, only: physics_error_fatal
L00021  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00022  #else
L00023   use module_ra_gfdleta,  only: cal_mon_day
L00024   use module_wrf_error
L00025  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00026  #endif
L00027  #if ( WRF_CHEM == 1 )
L00028    USE module_data_gocart_dust
L00029  #endif
L00030  !------------------------------
L00031
L00032  !
L00033  CONTAINS
```

讲解：USE 全家福交代了本模块的三重身份。① 从 noahlsm 进来的不只是 SFLX，还有**三张参数表的全部存储数组**（BB/MAXSMC/SATDK 等 + SHDTBL/LUTYPE/BARE/NATURAL/LCZ_1 至 LCZ_11）——noahdrv 负责把 TBL 读进这些模块级数组（§ 六），noahlsm 只查表；② 城市侧：module_sf_urban 的 urban（单层 UCM 主入口）、oasis（绿洲系数）、IRI_SCHEME（灌溉方案开关），以及 bep/bep_bem 多层城市方案；③ 冰川：SFLX_GLACIAL 来自独立文件 module_sf_noahlsm_glacial_only.F——冰川不需要土壤水，走精简方案。另有 MPAS/WRF 双框架宏（cal_mon_day 的来源切换）与 WRF_CHEM 的 gocart 沙尘依赖。lsm 主入口的完整签名（L38–L129）带约 150 个城市专用 OPTIONAL 参数——单层 UCM 的全部状态量都在这里过手。

四、lsm 主入口（B 级节选）

4.1 分层设置与首步初始化

```
!file: phys/module_sf_noahdrv.F    L757-L808
L00757         SLOPETYP=1
L00758  !      SHDMIN=0.00
L00759
L00760
L00761         NSOIL=num_soil_layers
L00762
L00763        DO NS=1,NSOIL
L00764        SLDPTH(NS)=DZS(NS)
L00765        ENDDO
L00766
L00767        JLOOP : DO J=jts,jte
L00768
L00769         IF(ITIMESTEP.EQ.1)THEN
L00770           DO 50 I=its,ite
L00771  !*** initialize soil conditions for IHOP 31 May case
L00772  !          IF((XLAND(I,J)-1.5) < 0.)THEN
L00773  !             if (I==108.and.j==85) then
L00774  !                  DO NS=1,NSOIL
L00775  !                      SMOIS(I,NS,J)=0.10
L00776  !                      SH2O(I,NS,J)=0.10
L00777  !                  enddo
L00778  !              endif
L00779  !          ENDIF
L00780
L00781  !*** SET ZERO-VALUE FOR SOME OUTPUT DIAGNOSTIC ARRAYS
L00782            IF((XLAND(I,J)-1.5).GE.0.)THEN
L00783  ! check sea-ice point
L00784  #if 0
L00785              IF( XICE(I,J).GE. XICE_THRESHOLD .and. IPRINT ) PRINT*, ' sea-ice at water point, I=',I,'J=',J
L00786  #endif
L00787  !***   Open Water Case
L00788              SMSTAV(I,J)=1.0
L00789              SMSTOT(I,J)=1.0
L00790              DO NS=1,NSOIL
L00791                SMOIS(I,NS,J)=1.0
L00792                TSLB(I,NS,J)=273.16                                    !STEMP
L00793                SMCREL(I,NS,J)=1.0
L00794              ENDDO
L00795            ELSE
L00796              IF ( XICE(I,J) .GE. XICE_THRESHOLD ) THEN
L00797  !***        SEA-ICE CASE
L00798                SMSTAV(I,J)=1.0
```

```
L00799                SMSTOT(I,J)=1.0
L00800                DO NS=1,NSOIL
L00801                  SMOIS(I,NS,J)=1.0
L00802                  SMCREL(I,NS,J)=1.0
L00803                ENDDO
L00804              ENDIF
L00805            ENDIF
L00806  !
L00807     50   CONTINUE
L00808        ENDIF                                                    ! end of initialization over ocean
```

讲解：①SLOPETYP=1 硬编码（坡度类固定，老版曾为 2）——Noah 在 WRF 中不做地形坡度分异；②SLDPTH 从 DZS 逐层拷贝——DZS 是 Registry 态、SLDPTH 是 Noah 的本地语言；③ITIMESTEP=1 时对**水面点**（XLAND−1.5≥0）初始化 SMOIS=1/TSLB=273.16/SMCREL=1；**海冰点**（XICE≥XICE_THRESHOLD）同样 SMOIS=1 但不动 TSLB——土壤水满是海冰点跳过 LSM 的配合条件（§4.4）。

4.2 格点预处理：单位换算与强迫量

```
!file: phys/module_sf_noahdrv.F    L811-L948
L00811         ILOOP : DO I=its,ite
L00812  ! surface pressure
L00813          PSFC=P8w3D(i,1,j)
L00814  ! pressure in middle of lowest layer
L00815          SFCPRS=(P8W3D(I,KTS+1,j)+P8W3D(i,KTS,j))*0.5
L00816  ! convert from mixing ratio to specific humidity
L00817           Q2K=QV3D(i,1,j)/(1.0+QV3D(i,1,j))
L00818  ! initializing local variables
L00819          SOILW=0.
L00820          FLX4=0.
L00821          FVB=0.
L00822          FBUR=0.
L00823          FGSN=0.
L00824  !
L00825  !         Q2SAT=QGH(I,j)
L00826           Q2SAT=QGH(I,J)/(1.0+QGH(I,J))        ! Q2SAT is sp humidity
L00827  ! add check on myj=.true.
L00828  !        IF((Q2K.GE.Q2SAT*TRESH).AND.Q2K.LT.QZ0(I,J))THEN
L00829          IF((myj).AND.(Q2K.GE.Q2SAT*TRESH).AND.Q2K.LT.QZ0(I,J))THEN
L00830            SATFLG=0.
L00831            CHKLOWQ(I,J)=0.
L00832          ELSE
L00833            SATFLG=1.0
L00834            CHKLOWQ(I,J)=1.
L00835          ENDIF
L00836
L00837          SFCTMP=T3D(i,1,j)
L00838          ZLVL=0.5*DZ8W(i,1,j)
L00839
L00840  !        TH2=SFCTMP+(0.0097545*ZLVL)
L00841  ! calculate SFCTH2 via Exner function vs lapse-rate (above)
L00842           APES=(1.E5/PSFC)**CAPA
L00843           APELM=(1.E5/SFCPRS)**CAPA
L00844           SFCTH2=SFCTMP*APELM
L00845           TH2=SFCTH2/APES
L00846  !
L00847           EMISSI = EMISS(I,J)
L00848           LWDN=GLW(I,J)*EMISSI
L00849  ! SOLDN is total incoming solar
L00850          SOLDN=SWDOWN(I,J)
L00851  ! GSW is net downward solar
L00852  !        SOLNET=GSW(I,J)
L00853  ! use mid-day albedo to determine net downward solar (no solar zenith angle correction)
L00854          SOLNET=SOLDN*(1.-ALBEDO(I,J))
L00855          PRCP=RAINBL(i,j)/DT
L00856          IF(PRESENT(IRR_CHAN)) THEN
L00857           IF(IRR_CHAN(i,j).NE.0) THEN
L00858             IRRIGATION_CHANNEL=IRR_CHAN(i,j)/DT
L00859           ELSE
L00860             IRRIGATION_CHANNEL=0.
L00861           END IF
L00862          ENDIF
L00863          VEGTYP=IVGTYP(I,J)
L00864          SOILTYP=ISLTYP(I,J)
L00865          SHDFAC=VEGFRA(I,J)/100.
L00866          T1=TSK(I,J)
L00867          CHK=CHS(I,J)
L00868          SHMIN=SHDMIN(I,J)/100. !NEW
L00869          SHMAX=SHDMAX(I,J)/100. !NEW
L00870  ! convert snow water equivalent from mm to meter
L00871          SNEQV=SNOW(I,J)*0.001
L00872  ! snow depth in meters
L00873          SNOWHK=SNOWH(I,J)
L00874          SNCOVR=SNOWC(I,J)
L00875
L00876  ! if "SR" present, set frac of frozen precip ("FFROZP") = snow-ratio ("SR", range:0-1)
L00877  ! SR from e.g. Ferrier microphysics
L00878  ! otherwise define from 1st atmos level temperature
L00879         IF(FRPCPN) THEN
L00880            FFROZP=SR(I,J)
L00881          ELSE
L00882            IF (SFCTMP <=  273.15) THEN
L00883              FFROZP = 1.0
L00884        ELSE
L00885          FFROZP = 0.0
L00886        ENDIF
L00887          ENDIF
L00888  !***
L00889          IF((XLAND(I,J)-1.5).GE.0.)THEN                                  ! begining of land/sea if block
L00890  ! Open water points
L00891            TSK_RURAL(I,J)=TSK(I,J)
L00892            HFX_RURAL(I,J)=HFX(I,J)
```

```
L00893            QFX_RURAL(I,J)=QFX(I,J)
L00894            LH_RURAL(I,J)=LH(I,J)
L00895            EMISS_RURAL(I,J)=EMISS(I,J)
L00896            GRDFLX_RURAL(I,J)=GRDFLX(I,J)
L00897
L00898          ELSE
L00899  ! Land or sea-ice case
L00900
L00901            IF (XICE(I,J) >= XICE_THRESHOLD) THEN
L00902               ! Sea-ice point
L00903               ICE = 1
L00904            ELSE IF ( VEGTYP == ISICE ) THEN
L00905               ! Land-ice point
L00906               ICE = -1
L00907            ELSE
L00908               ! Neither sea ice or land ice.
L00909               ICE=0
L00910            ENDIF
L00911            DQSDT2=Q2SAT*A23M4/(SFCTMP-A4)**2
L00912
L00913            IF(SNOW(I,J).GT.0.0)THEN
L00914  ! snow on surface (use ice saturation properties)
L00915              SFCTSNO=SFCTMP
L00916              E2SAT=611.2*EXP(6174.*(1./273.15 - 1./SFCTSNO))
L00917              Q2SATI=0.622*E2SAT/(SFCPRS-E2SAT)
L00918              Q2SATI=Q2SATI/(1.0+Q2SATI)     ! spec. hum.
L00919              IF (T1 .GT. 273.14) THEN
L00920  ! warm ground temps, weight the saturation between ice and water according to SNOWC
L00921                Q2SAT=Q2SAT*(1.-SNOWC(I,J)) + Q2SATI*SNOWC(I,J)
L00922                DQSDT2=DQSDT2*(1.-SNOWC(I,J)) + Q2SATI*6174./(SFCTSNO**2)*SNOWC(I,J)
L00923              ELSE
L00924  ! cold ground temps, use ice saturation only
L00925                Q2SAT=Q2SATI
L00926                DQSDT2=Q2SATI*6174./(SFCTSNO**2)
L00927              ENDIF
L00928  ! for snow cover fraction at 0 C, ground temp will not change, so DQSDT2 effectively zero
L00929  ! V3.8 add condition for SWDOWN to restrict condition to positive forcing (JD)
L00930              IF(T1 .GT. 273. .AND. SNOWC(I,J) .GT. 0. .AND. SWDOWN(I,J) .GT. 10.)DQSDT2=DQSDT2*(1.-SNOWC(I,J))
L00931            ENDIF
L00932
L00933            ! Land-ice or land points use the usual deep-soil temperature.
L00934            TBOT=TMN(I,J)
L00935
L00936            IF(ISURBAN.EQ.1) THEN
L00937  ! assumes these only need to be set for USGS land data
L00938            IF(VEGTYP.EQ.25) SHDFAC=0.0000
L00939            IF(VEGTYP.EQ.26) SHDFAC=0.0000
L00940            IF(VEGTYP.EQ.27) SHDFAC=0.0000
L00941            ENDIF
L00942            IF(SOILTYP.EQ.14.AND.XICE(I,J).EQ.0.)THEN
L00943  #if 0
L00944           IF(IPRINT)PRINT*,' SOIL TYPE FOUND TO BE WATER AT A LAND-POINT'
L00945           IF(IPRINT)PRINT*,i,j,'RESET SOIL in surfce.F'
L00946  #endif
L00947              SOILTYP=7
L00948            ENDIF
```

讲解：逐格点八件事。①PSFC 取最底层气压、SFCPRS 取最底两层平均——Noah 的「第一层大气」是半层高；Q2K 由混合比换比湿。②SATFLG/CHKLOWQ 近饱和标记只在 myj（MYJ 面层）路径生效（TRESH=0.95、QZ0 上限）——其他面层方案恒为 1。③ZLVL=0.5 • DZ8W(1)；TH2 用两层 Exner 函数换算（旧式 0.0097545 • ZLVL 气温直减率近似被注释）。④LWDN=GLW • EMISSI（长波按发射率折算）；**SOLNET=SOLDN · (1−ALBEDO)**——用当前时变反照率算净短波（GSW 老口径注释掉）。⑤PRCP=RAINBL/DT（mm/step 到 kg/(m²s)）；灌溉通道 IRR_CHAN/DT 透传。⑥SHDFAC=VEGFRA/100、SNEQV=SNOW • 0.001（mm 到 m）。⑦**FFROZP 冻水比例**：frpcpn（Ferrier 微物理开关）时直接用 SR 场（0–1 连续值），否则按第一层气温 0°C 二值判——SFLX 里以 >0.5 判雪。⑧ **雪面强迫换冰面饱和**：SNOW>0 时由冰面饱和蒸汽压式算 Q2SATI，T1>273.14（暖地面）按 SNOWC 加权水/冰饱和，DQSDT2 同步换冰温斜率（V3.8 加 SWDOWN>10 条件限制在白昼有辐射强迫时生效）。杂项：USGS 口径（ISURBAN=1）下三类城市（25 低密/26 高密/27 工业）SHDFAC 强制 0；土壤类 14（水体）误落在陆地改 7；SNOWHK 兜底——雪深状态缺失或小于水当量时按密度 0.2 反推 SNOWHK=5 • SNEQV。

4.3 城市前处理与单层 UCM 的灌溉/绿洲挂点

```
!file: phys/module_sf_noahdrv.F    L982-L1054
L00982  !Fei: urban. for urban surface, if calling UCM, redefine the natural surface in cities as
L00983  ! the "NATURAL" category in the VEGPARM.TBL
L00984                    IF(SF_URBAN_PHYSICS == 1.OR. SF_URBAN_PHYSICS==2.OR.SF_URBAN_PHYSICS==3 ) THEN
L00985                    IF( IVGTYP(I,J) == ISURBAN    .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L00986                        IVGTYP(I,J) == LCZ_3      .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L00987                        IVGTYP(I,J) == LCZ_6      .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L00988                        IVGTYP(I,J) == LCZ_9      .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 ) THEN
L00989
L00990           VEGTYP = NATURAL
L00991                   SHDFAC = SHDTBL(NATURAL)
L00992                   ALBEDOK =0.2          !  0.2
L00993                   ALBBRD  =0.2          !0.2
L00994                   EMISSI = 0.98                                  !for VEGTYP=5
L00995                   LWDN   = GLW(I,J) * EMISSI
L00996                   SOLNET = SOLDN * (1.0 - ALBEDOK)
L00997
L00998           IF ( FRC_URB2D(I,J) < 0.99 ) THEN
L00999                     if(sf_urban_physics.eq.1)then
L01000                       T1= ( TSK(I,J) -FRC_URB2D(I,J) * TS_URB2D (I,J) )/ (1-FRC_URB2D(I,J))
```

```
L01001                    elseif((sf_urban_physics.eq.2).OR.(sf_urban_physics.eq.3))then
L01002                      T1=tsk_rural_bep(i,j)
L01003                    endif
L01004                  ELSE
L01005            T1 = TSK(I,J)
L01006                  ENDIF
L01007                ENDIF
L01008              ELSE
L01009               IF( IVGTYP(I,J) == ISURBAN    .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L01010                   IVGTYP(I,J) == LCZ_3      .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L01011                   IVGTYP(I,J) == LCZ_6      .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L01012                   IVGTYP(I,J) == LCZ_9      .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 ) THEN
L01013                     VEGTYP = ISURBAN
L01014                    ENDIF
L01015
L01016              ENDIF
L01017
L01018  !===Yang, 2014/10/08, hydrological processes for urban vegetation in single layer UCM===
L01019              AOASIS = 1.0
L01020              USOIL = 1
L01021              DSOIL = 2
L01022              IRIOPTION=IRI_SCHEME
L01023         IF(SF_URBAN_PHYSICS == 1) THEN
L01024              OMG= OMG_URB2D(I,J)
L01025              tloc=mod(int(OMG/3.14159*180./15.+12.+0.5 ),24)
L01026              if (tloc.lt.0) tloc=tloc+24
L01027              if (tloc==0) tloc=24
L01028              CALL cal_mon_day(julian,julyr,jmonth,jday)
L01029                IF( IVGTYP(I,J) == ISURBAN    .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L01030                    IVGTYP(I,J) == LCZ_3      .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L01031                    IVGTYP(I,J) == LCZ_6      .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L01032                    IVGTYP(I,J) == LCZ_9      .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 ) THEN
L01033                  AOASIS = oasis  ! urban oasis effect
L01034                    IF (IRIOPTION ==1) THEN
L01035                    IF (tloc==21 .or. tloc==22) THEN  !irrigation on vegetaion in urban area, MAY-SEP, 9-10pm
L01036                    IF (jmonth==5 .or. jmonth==6 .or. jmonth==7 .or. jmonth==8 .or. jmonth==9) THEN
L01037  !                     IF (SMC(USOIL) .LT. SMCREF) SMC(USOIL)= SMCREF
L01038  !                     IF (SMC(DSOIL) .LT. SMCREF) SMC(DSOIL)= SMCREF
L01039                       IF (SMC(USOIL) .LT. SMCREF) SMC(USOIL)= REFSMC(ISLTYP(I,J))
L01040                       IF (SMC(DSOIL) .LT. SMCREF) SMC(DSOIL)= REFSMC(ISLTYP(I,J))
L01041                    ENDIF
L01042                    ENDIF
L01043                    ENDIF
L01044                  ENDIF
L01045              ENDIF
L01046
L01047         IF(SF_URBAN_PHYSICS == 2 .or. SF_URBAN_PHYSICS == 3) THEN
L01048             IF(AOASIS > 1.0) THEN
L01049             FATAL_ERROR('Urban oasis option is for SF_URBAN_PHYSICS == 1 only')
L01050             ENDIF
L01051             IF(IRIOPTION == 1) THEN
L01052             FATAL_ERROR('Urban irrigation option is for SF_URBAN_PHYSICS == 1 only')
L01053             ENDIF
L01054             ENDIF
```

讲解：①SF_URBAN_PHYSICS=1/2/3 时城市类格点（ISURBAN 或 LCZ_1 至 LCZ_11）的「自然下垫面」被重定义：**UCM 路径** VEGTYP=NATURAL、SHDFAC=SHDTBL(NATURAL)、ALBBRD=ALBEDOK=0.2、EMISSI=0.98——城市里的「自然部分」按植被表 NATURAL 类取参数；**BEP/BEM 路径** VEGTYP=ISURBAN（进 SFLX 后触发 12a § 四的 urban 硬参数）。②FRC_URB2D<0.99（非纯城市）时 T1 做 rural 重建：UCM 从 (TSK−FRC • TS_URB2D)/(1−FRC) 反推、BEP/BEM 直接用 tsk_rural_bep——皮肤温度的城市/乡村解耦。③**Yang 2014 水文挂点**：AOASIS=1 默认（城市格点换 oasis 系数进 PENMAN，12a § 十）；IRI_SCHEME=1 时城市格点 5–9 月每日 21–22 时把上两层 SMC 补到该土壤类的田间持水量 REFSMC——「定时灌溉」的最粗颗粒实现，直接改 SMC 数组。④oasis/IRI 与 BEP/BEM 组合直接 FATAL_ERROR——挂点只接单层 UCM，想移植到 BEP 路径就得自己接 IRRIGATION_CHANNEL 通道。

4.4 ICE 三分支：海冰、普通陆地、冰川

```
!file: phys/module_sf_noahdrv.F    L1082-L1205
L01082            IF (rdlai2d) THEN
L01083               IF (SHDFAC > 0.0 .AND. LAI(I,J) <= 0.0) LAI(I,J) = 0.01
L01084               xlai = lai(i,j)
L01085            endif
L01086
L01087      IF ( ICE == 1 ) THEN
L01088
L01089         ! Sea-ice case
L01090
L01091         DO NS = 1, NSOIL
L01092            SH2O(I,NS,J) = 1.0
L01093         ENDDO
L01094         LAI(I,J) = 0.01
L01095
L01096         CYCLE ILOOP
L01097
L01098      ELSEIF (ICE == 0) THEN
L01099
L01100         ! Non-glacial Land
L01101  !
L01102  !  FASDAS
L01103  !
L01104         IF( fasdas == 1 ) THEN
L01105
L01106           DZQ = DZ8W(I,1,J)
L01107           XSDA_HFX= SDA_HFX(I,J)*RHO(I,1,J)*CPM(I,J)*DZQ  ! W/m^2
L01108           ! TWG2015 Bugfix remove factor of 1000.0 for correct units
```

```
L01109            XSDA_QFX= SDA_QFX(I,J)*RHO(I,1,J)*DZQ          ! Kg/m2/s of water
L01110            XQNORM = QNORM(I,J)
L01111
L01112          ENDIF
L01113  !
L01114  !  END FASDAS
L01115  !
L01116  #ifdef WRF_HYDRO
L01117          etpnd1_hydro = 0.
L01118          sfcheadrt_hydro = sfcheadrt(i,j)
L01119          infxsrt_hydro = infxsrt(i,j)
L01120  #else
L01121          etpnd1_hydro = 0.
L01122          sfcheadrt_hydro = 0.
L01123          infxsrt_hydro = 0.
L01124  #endif
L01125          CALL SFLX (I,J,FFROZP, ISURBAN, DT,ZLVL,NSOIL,SLDPTH,       &    !C
L01126                     LOCAL,                                            &    !L
L01127                     LUTYPE, SLTYPE,                                   &    !CL
L01128                     LWDN,SOLDN,SOLNET,SFCPRS,PRCP,SFCTMP,Q2K,DUMMY,   &    !F
L01129                     DUMMY,DUMMY, DUMMY,                               &    !F PRCPRAIN not used
L01130                     TH2,Q2SAT,DQSDT2,                                 &    !I
L01131                     VEGTYP,SOILTYP,SLOPETYP,SHDFAC,SHMIN,SHMAX,       &    !I
L01132                     ALBBRD, SNOALB1,TBOT, Z0BRD, Z0K, EMISSI, EMBRD,  &    !S
L01133                     CMC,T1,STC,SMC,SWC,SNOWHK,SNEQV,ALBEDOK,CHK,dummy,&    !H
L01134                     ETA,SHEAT, ETA_KINEMATIC,FDOWN,                   &    !O
L01135                     EC,EDIR,ET,ETT,ESNOW,DRIP,DEW,                    &    !O
L01136                     BETA,ETP,SSOIL,                                   &    !O
L01137                     FLX1,FLX2,FLX3,                                   &    !O
L01138            FLX4,FVB,FBUR,FGSN,UA_PHYS,                        &    !UA
L01139                     SNOMLT,SNCOVR,                                    &    !O
L01140                     RUNOFF1,RUNOFF2,RUNOFF3,                          &    !O
L01141                     RC,PC,RSMIN,XLAI,RCS,RCT,RCQ,RCSOIL,              &    !O
L01142                     SOILW,SOILM,Q1,SMAV,                              &    !D
L01143                     RDLAI2D,USEMONALB,                                &
L01144                     SNOTIME1,                                         &
L01145                     RIBB,                                             &
L01146                     SMCWLT,SMCDRY,SMCREF,SMCMAX,NROOT,                &
L01147  ! WRF_HYDRO vars
L01148                     sfcheadrt_hydro,                                  &    !I
L01149                     INFXSRT_hydro,ETPND1_hydro                        &    !O
L01150                    ,OPT_THCND,AOASIS                                  &    !O
L01151                    ,XSDA_QFX, HFX_PHY, QFX_PHY, XQNORM, fasdas, HCPCT_FASDAS    &   ! fasdas
L01152                    ,IRRIGATION_CHANNEL)
L01153
L01154  #ifdef WRF_HYDRO
L01155                     soldrain(i,j) = RUNOFF2*DT*1000.0
L01156                     sfcheadrt(i,j) = sfcheadrt_hydro
L01157                     infxsrt(i,j) = INFXSRT_hydro
L01158                     etpnd1 = etpnd1_hydro
L01159  #endif
L01160      ELSEIF (ICE == -1) THEN
L01161
L01162          !
L01163          ! Set values that the LSM is expected to update,
L01164          ! but don't get updated for glacial points.
L01165          !
L01166          SOILM = 0.0 !BSINGH(PNNL)- SOILM is undefined for this case, it is used for diagnostics so setting it to zero
L01167          XLAI = 0.01 ! KWM Should this be Zero over land ice?  Does this value matter?
L01168          RUNOFF2 = 0.0
L01169          RUNOFF3 = 0.0
L01170          DO NS = 1, NSOIL
L01171             SWC(NS) = 1.0
L01172             SMC(NS) = 1.0
L01173             SMAV(NS) = 1.0
L01174          ENDDO
L01175  !
L01176  !  FASDAS
L01177  !
L01178          IF( fasdas == 1 ) THEN
L01179
L01180            DZQ = DZ8W(I,1,J)
L01181            XSDA_HFX= SDA_HFX(I,J)*RHO(I,1,J)*CPM(I,J)*DZQ  ! W/m^2
L01182            XSDA_QFX= 0.0          ! Kg/m2/s of water
L01183            XQNORM = 0.0
L01184
L01185          ENDIF
L01186  !
L01187  !  END FASDAS
L01188  !
L01189
L01190          CALL SFLX_GLACIAL(I,J,ISICE,FFROZP,DT,ZLVL,NSOIL,SLDPTH,   &    !C
L01191               &    LWDN,SOLNET,SFCPRS,PRCP,SFCTMP,Q2K,              &    !F
L01192               &    TH2,Q2SAT,DQSDT2,                                &    !I
L01193               &    ALBBRD, SNOALB1,TBOT, Z0BRD, Z0K, EMISSI, EMBRD, &    !S
L01194               &    T1,STC(1:NSOIL),SNOWHK,SNEQV,ALBEDOK,CHK,        &    !H
L01195               &    ETA,SHEAT,ETA_KINEMATIC,FDOWN,                   &    !O
L01196               &    ESNOW,DEW,                                       &    !O
L01197               &    ETP,SSOIL,                                       &    !O
L01198               &    FLX1,FLX2,FLX3,                                  &    !O
L01199               &    SNOMLT,SNCOVR,                                   &    !O
L01200               &    RUNOFF1,                                         &    !O
L01201               &    Q1,                                              &    !D
L01202               &    SNOTIME1,                                        &
L01203               &    RIBB)
L01204
L01205      ENDIF
```

讲解：rdlai2d 路径把外场 LAI 传给 xlai（下限 0.01）。随后按 ICE 三分派：**ICE=1（海冰）**——SH2O 全层置 1、LAI=0.01 后 CYCLE ILOOP **直接跳过 LSM**：海冰点的感热/潜热全部由面层方案（PBL 的 sfclay 系）按 TSK 计算，Noah 只负责把土壤湿置满（§4.1 配合）；**ICE=−1（陆地冰川，VEGTYP==ISICE）**——SWC/SMC/SMAV 全置 1、RUNOFF2/3 清零后调 SFLX_GLACIAL（冰川简化版：无土壤水、无蒸腾，只算雪/冰面能量平衡、升华与 RUNOFF1，签名见调用处实参）；**ICE=0（普通陆地）**——fasdas=1 时把同化增量换算成格点通量（XSDA_HFX=SDA_HFX·ρ·CPM·DZQ，

XSDA_QFX 同理），WRF_HYDRO 的 sfcheadrt/infxsrt 经 _hydro 本地变量桥接，然后 **SFLX 全参调用**——实参顺序即 12a § 四的九类接口合同；注意 CH 位传的是 CHK=CHS(I,J)——面层方案给的交换系数，CM 位传 DUMMY，印证 SFCDIF_off 在 WRF 主线的休眠（12b § 十四）。

4.5 状态回写、fasdas 皮肤温度与能量残差

```
!file: phys/module_sf_noahdrv.F    L1239-L1334
L01239  !***  UPDATE STATE VARIABLES
L01240            CANWAT(I,J)=CMC*1000.
L01241            SNOW(I,J)=SNEQV*1000.
L01242  !          SNOWH(I,J)=SNOWHK*1000.
L01243            SNOWH(I,J)=SNOWHK                    ! SNOWHK in meters
L01244            ALBEDO(I,J)=ALBEDOK
L01245            ALB_RURAL(I,J)=ALBEDOK
L01246            ALBBCK(I,J)=ALBBRD
L01247            Z0(I,J)=Z0BRD
L01248            EMISS(I,J) = EMISSI
L01249            EMISS_RURAL(I,J) = EMISSI
L01250  ! Noah: activate time-varying roughness length (V3.3 Feb 2011)
L01251            ZNT(I,J)=Z0K
L01252  !
L01253  ! FASDAS
L01254  !
L01255  ! Update Skin Temperature
L01256         IF( fasdas == 1 ) THEN
L01257             XSDA_QFX= XSDA_QFX*ELWV*XQNORM
L01258
L01259          !TWG2015 Bugfix to multiply Heat Capacity by Soil Depth for correct
L01260          !units
L01261
L01262             T1 = T1 + (XSDA_HFX-XSDA_QFX)*DT/(HCPCT_FASDAS*DZS(1))
L01263
L01264         END IF
L01265  !
L01266  ! END FASDAS
L01267  !
L01268            TSK(I,J)=T1
L01269            TSK_RURAL(I,J)=T1
L01270            if (present(tsk_rural_bep)) then
L01271               IF(SF_URBAN_PHYSICS == 2 .or. SF_URBAN_PHYSICS == 3) THEN
L01272                 TSK_RURAL_BEP(I,J)=T1
L01273               END IF
L01274            endif
L01275            HFX(I,J)=SHEAT
L01276            HFX_RURAL(I,J)=SHEAT
L01277  ! MEk Jul07 add potential evap accum
L01278          POTEVP(I,J)=POTEVP(I,J)+ETP*FDTW
L01279            QFX(I,J)=ETA_KINEMATIC
L01280            QFX_RURAL(I,J)=ETA_KINEMATIC
L01281
L01282  #ifdef WRF_HYDRO
L01283  !added by Wei Yu
L01284  !          QFX(I,J) = QFX(I,J) + ETPND1
L01285  !          ETA = ETA + ETPND1/2.501E6*dt
L01286  !end added by Wei Yu
L01287  #endif
L01288
L01289
L01290            LH(I,J)=ETA
L01291            LH_RURAL(I,J)=ETA
L01292            GRDFLX(I,J)=SSOIL
L01293            GRDFLX_RURAL(I,J)=SSOIL
L01294            SNOWC(I,J)=SNCOVR
L01295            CHS2(I,J)=CQS2(I,J)
L01296            SNOTIME(I,J) = SNOTIME1
L01297  !       prevent diagnostic ground q (q1) from being greater than qsat(tsk)
L01298  !       as happens over snow cover where the cqs2 value also becomes irrelevant
L01299  !       by setting cqs2=chs in this situation the 2m q should become just qv(k=1)
L01300  ! ww: comment out this change to avoid Q2 drop due to change of radiative flux
L01301  !          IF (Q1 .GT. QSFC(I,J)) THEN
L01302  !            CQS2(I,J) = CHS(I,J)
L01303  !          ENDIF
L01304  !          QSFC(I,J)=Q1
L01305  ! Convert QSFC back to mixing ratio
L01306             QSFC(I,J)= Q1/(1.0-Q1)
L01307  !
L01308             ! QSFC_RURAL(I,J)= Q1/(1.0-Q1)
L01309  ! Calculate momentum flux from rural surface for use with multi-layer UCM (Martilli et al. 2002)
L01310
L01311            DO 80 NS=1,NSOIL
L01312             SMOIS(I,NS,J)=SMC(NS)
L01313             TSLB(I,NS,J)=STC(NS)                                    !  STEMP
L01314             SH2O(I,NS,J)=SWC(NS)
L01315     80     CONTINUE
L01316  !       ENDIF
L01317
L01318          FLX4_2D(I,J)  = FLX4
L01319       FVB_2D(I,J)   = FVB
L01320       FBUR_2D(I,J)  = FBUR
L01321       FGSN_2D(I,J)  = FGSN
L01322
L01323        !
L01324        ! Residual of surface energy balance equation terms
L01325        !
L01326
L01327        IF ( UA_PHYS ) THEN
L01328            noahres(i,j) = ( solnet + lwdn ) - sheat + ssoil - eta &
L01329                 - ( emissi * STBOLT * (t1**4) ) - flx1 - flx2 - flx3 - flx4
L01330
L01331        ELSE
L01332            noahres(i,j) = ( solnet + lwdn ) - sheat + ssoil - eta &
L01333                 - ( emissi * STBOLT * (t1**4) ) - flx1 - flx2 - flx3
L01334        ENDIF
```

讲解：①CANWAT/SNOW/SNOWH/ALBEDO/ALBBCK/Z0/EMISS 逐项回写；ZNT=Z0K 激活时变粗糙度（V3.3 起——PBL 面层的粗糙度从这里来）。②**fasdas=1 的皮肤温度增量**：XSDA_QFX • ELWV • XQNORM 换成能量单位，与 XSDA_HFX 一起按 HCPCT • DZS(1)（表层热容 × 层厚）直接加热 T1——同化增量的「温度形式」实现。③TSK=T1/HFX=SHEAT/QFX=ETA_KINEMATIC/LH=ETA/GRDFLX=SSOIL 回写，rural 副本（*_RURAL 数组）同步维护供 BEP 聚合；POTEVP 累计 ETP • FDTW（FDTW=DT/(XLV • ρwater)）；QSFC=Q1/(1−Q1) 由比湿换回混合比。④**NOAHRES 能量残差**：(SOLNET+LWDN)−SHEAT+SSOIL−ETA−EMISSI • σ • $T1^4$−FLX1−FLX2−FLX3（UA 再减 FLX4）——注意 +SSOIL 的符号约定（SFLX 已把 SSOIL 变号为向下正），残差理论为零，是 Noah 能量不闭合排错的第一现场。

4.6 单层 UCM 桥接与 FRC 加权

```
!file: phys/module_sf_noahdrv.F    L1337-L1348
L01337            IF (SF_URBAN_PHYSICS == 1 ) THEN                                                  ! Beginning of UCM CALL if block
L01338   !---------------------------------------
L01339   ! URBAN CANOPY MODEL START - urban
L01340   !---------------------------------------
L01341   ! Input variables lsm --> urban
L01342
L01343               IF( IVGTYP(I,J) == ISURBAN    .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L01344                   IVGTYP(I,J) == LCZ_3      .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L01345                   IVGTYP(I,J) == LCZ_6      .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L01346                   IVGTYP(I,J) == LCZ_9      .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 ) THEN
L01347
L01348
```

```
!file: phys/module_sf_noahdrv.F    L1513-L1525
L01513               TS_URB2D(I,J) = TS_URB
L01514
L01515               ALBEDO(I,J) = FRC_URB2D(I,J)*ALB_URB+(1-FRC_URB2D(I,J))*ALBEDOK    ![-]
L01516               HFX(I,J) = FRC_URB2D(I,J)*SH_URB+(1-FRC_URB2D(I,J))*SHEAT          ![W/m/m]
L01517               QFX(I,J) = FRC_URB2D(I,J)*LH_KINEMATIC_URB &
L01518                        + (1-FRC_URB2D(I,J))*ETA_KINEMATIC                  ![kg/m/m/s]
L01519               LH(I,J) = FRC_URB2D(I,J)*LH_URB+(1-FRC_URB2D(I,J))*ETA             ![W/m/m]
L01520               GRDFLX(I,J) = FRC_URB2D(I,J)*G_URB+(1-FRC_URB2D(I,J))*SSOIL        ![W/m/m]
L01521               TSK(I,J) = FRC_URB2D(I,J)*TS_URB+(1-FRC_URB2D(I,J))*T1             ![K]
L01522               Q1 = FRC_URB2D(I,J)*QS_URB+(1-FRC_URB2D(I,J))*Q1             ![-]
L01523   ! Convert QSFC back to mixing ratio
L01524               QSFC(I,J)= Q1/(1.0-Q1)
L01525               UST(I,J)= FRC_URB2D(I,J)*UST_URB+(1-FRC_URB2D(I,J))*UST(I,J)       ![m/s]
```

讲解：UCM 调用块（L1337–L1610，约 270 行不整录）：入口条件 L1343 的城市类判断；实参装配把 20 余个局地大气量（TA/QA/UA/SSG/SSGD/LLG/RAIN/RHOO/ZA…）与 30 余个状态量（屋顶/墙体/路面温度层、XXX 系列倾向、水文 DREL/FLXHUM 系列）打包给 urban，返回后逐项写回 2D/3D 状态数组。**出口混合公式**（第二块）：ALBEDO/HFX/QFX/LH/GRDFLX/TSK/Q1/UST = 城市值 ×FRC_URB2D + 自然值 ×(1−FRC_URB2D)——单层 UCM「城市—乡村两份通量按城市面积比线性混合」的核心思想全在这几行；QSFC 随 Q1 重新换算。

4.7 诊断累积

```
!file: phys/module_sf_noahdrv.F    L1615-L1634
L01615   !***  DIAGNOSTICS
L01616               SMSTAV(I,J)=SOILW
L01617               SMSTOT(I,J)=SOILM*1000.
L01618               DO NS=1,NSOIL
L01619               SMCREL(I,NS,J)=SMAV(NS)
L01620               ENDDO
L01621
L01622
L01623   !           Convert the water unit into mm
L01624               SFCRUNOFF(I,J)=SFCRUNOFF(I,J)+RUNOFF1*DT*1000.0
L01625               UDRUNOFF(I,J)=UDRUNOFF(I,J)+RUNOFF2*DT*1000.0
L01626   ! snow defined when fraction of frozen precip (FFROZP) > 0.5,
L01627               IF(FFROZP.GT.0.5)THEN
L01628                 ACSNOW(I,J)=ACSNOW(I,J)+PRCP*DT
L01629               ENDIF
L01630               IF(SNOW(I,J).GT.0.)THEN
L01631                 ACSNOM(I,J)=ACSNOM(I,J)+SNOMLT*1000.
L01632   ! accumulated snow-melt energy
L01633                 SNOPCX(I,J)=SNOPCX(I,J)-SNOMLT/FDTLIW
L01634               ENDIF
```

讲解：SMSTAV=SOILW（根区有效水）、SMSTOT=SOILM • 1000（m 到 mm）、SMCREL=SMAV（逐层有效性）；SFCRUNOFF/UDRUNOFF 按 RUNOFF1/RUNOFF2 • DT • 1000 累积（速率到 mm 累计量）；ACSNOW 按 FFROZP>0.5 累计降雪；ACSNOM 累计 SNOMLT • 1000；**SNOPCX 累计融雪能耗 −SNOMLT/FDTLIW**（FDTLIW=DT/(ρ • Lf)——融雪水当量换 W/m² 的负号口径）。

4.8 BEP/BEM 聚合

```
!file: phys/module_sf_noahdrv.F    L1717-L1814
L01717      if((sf_urban_physics.eq.2).OR.(sf_urban_physics.eq.3))then         !Bep begin
L01718           do j=jts,jte
L01719           do i=its,ite
L01720              UMOM_URB(I,J)=0.
L01721              VMOM_URB(I,J)=0.
L01722              HFX_URB(I,J)=0.
L01723              QFX_URB(I,J)=0.
L01724           do k=kts,kte
L01725              a_u_bep(i,k,j)=a_u_bep(i,k,j)*frc_urb2d(i,j)
L01726              a_v_bep(i,k,j)=a_v_bep(i,k,j)*frc_urb2d(i,j)
L01727              a_t_bep(i,k,j)=a_t_bep(i,k,j)*frc_urb2d(i,j)
L01728              a_q_bep(i,k,j)=0.
L01729              a_e_bep(i,k,j)=0.
L01730              b_u_bep(i,k,j)=b_u_bep(i,k,j)*frc_urb2d(i,j)
L01731              b_v_bep(i,k,j)=b_v_bep(i,k,j)*frc_urb2d(i,j)
L01732              b_t_bep(i,k,j)=b_t_bep(i,k,j)*frc_urb2d(i,j)
L01733              b_q_bep(i,k,j)=b_q_bep(i,k,j)*frc_urb2d(i,j)
L01734              b_e_bep(i,k,j)=b_e_bep(i,k,j)*frc_urb2d(i,j)
L01735              HFX_URB(I,J)=HFX_URB(I,J)+B_T_BEP(I,K,J)*RHO(I,K,J)*CP*               &
L01736                           DZ8W(I,K,J)*VL_BEP(I,K,J)
L01737              QFX_URB(I,J)=QFX_URB(I,J)+B_Q_BEP(I,K,J)*                &
L01738                           DZ8W(I,K,J)*VL_BEP(I,K,J)
L01739              UMOM_URB(I,J)=UMOM_URB(I,J)+ (A_U_BEP(I,K,J)*U_PHY(I,K,J)+            &
L01740                           B_U_BEP(I,K,J))*DZ8W(I,K,J)*VL_BEP(I,K,J)
L01741              VMOM_URB(I,J)=VMOM_URB(I,J)+ (A_V_BEP(I,K,J)*V_PHY(I,K,J)+            &
L01742                           B_V_BEP(I,K,J))*DZ8W(I,K,J)*VL_BEP(I,K,J)
L01743              vl_bep(i,k,j)=(1.-frc_urb2d(i,j))+vl_bep(i,k,j)*frc_urb2d(i,j)
L01744              sf_bep(i,k,j)=(1.-frc_urb2d(i,j))+sf_bep(i,k,j)*frc_urb2d(i,j)
L01745           end do
L01746              a_u_bep(i,1,j)=(1.-frc_urb2d(i,j))*(-ust(I,J)*ust(I,J))/dz8w(i,1,j)/   &
L01747                           ((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)+a_u_bep(i,1,j)
L01748              a_v_bep(i,1,j)=(1.-frc_urb2d(i,j))*(-ust(I,J)*ust(I,J))/dz8w(i,1,j)/   &
L01749                           ((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)+a_v_bep(i,1,j)
L01750              b_t_bep(i,1,j)=(1.-frc_urb2d(i,j))*hfx_rural(i,j)/dz8w(i,1,j)/rho(i,1,j)/CP+ &
L01751                           b_t_bep(i,1,j)
L01752              b_q_bep(i,1,j)=(1.-frc_urb2d(i,j))*qfx_rural(i,j)/dz8w(i,1,j)/rho(i,1,j)+b_q_bep(i,1,j)
L01753              umom=(1.-frc_urb2d(i,j))*ust(i,j)*ust(i,j)*u_phy(i,1,j)/               &
L01754                           ((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)+umom_urb(i,j)
L01755              vmom=(1.-frc_urb2d(i,j))*ust(i,j)*ust(i,j)*v_phy(i,1,j)/               &
L01756                           ((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)+vmom_urb(i,j)
L01757              sf_bep(i,1,j)=1.
L01758
L01759  ! compute upward longwave radiation from the rural part and total
L01760  !           rl_up_rural=-emiss_rural(i,j)*sigma_sb*(tsk_rural(i,j)**4.)-(1.-emiss_rural(i,j))*glw(i,j)
L01761  !           rl_up_tot=(1.-frc_urb2d(i,j))*rl_up_rural+frc_urb2d(i,j)*rl_up_urb(i,j)
L01762  !           emiss(i,j)=(1.-frc_urb2d(i,j))*emiss_rural(i,j)+frc_urb2d(i,j)*emiss_urb(i,j)
L01763  ! using the emissivity and the total longwave upward radiation estimate the averaged skin temperature
L01764            IF (FRC_URB2D(I,J).GT.0.) THEN
L01765              rl_up_rural=-emiss_rural(i,j)*sigma_sb*(tsk_rural(i,j)**4.)-(1.-emiss_rural(i,j))*glw(i,j)
L01766              rl_up_tot=(1.-frc_urb2d(i,j))*rl_up_rural+frc_urb2d(i,j)*rl_up_urb(i,j)
L01767              emiss(i,j)=(1.-frc_urb2d(i,j))*emiss_rural(i,j)+frc_urb2d(i,j)*emiss_urb(i,j)
L01768              ts_urb2d(i,j)=(max(0.,(-rl_up_urb(i,j)-(1.-emiss_urb(i,j))*glw(i,j))/emiss_urb(i,j)/sigma_sb))**0.25
L01769              tsk(i,j)=(max(0., (-1.*rl_up_tot-(1.-emiss(i,j))*glw(i,j) )/emiss(i,j)/sigma_sb))**.25
L01770            rs_abs_tot=(1.-frc_urb2d(i,j))*swdown(i,j)*(1.-albedo(i,j))+frc_urb2d(i,j)*rs_abs_urb(i,j)
L01771           if(swdown(i,j).gt.0.)then
L01772            albedo(i,j)=1.-rs_abs_tot/swdown(i,j)
L01773           else
L01774            albedo(i,j)=alb_rural(i,j)
L01775           endif
L01776  ! rename *_urb to sh_urb2d,lh_urb2d,g_urb2d,rn_urb2d
L01777           grdflx(i,j)= (1.-frc_urb2d(i,j))*grdflx_rural(i,j)+frc_urb2d(i,j)*grdflx_urb(i,j)
L01778           qfx(i,j)=(1.-frc_urb2d(i,j))*qfx_rural(i,j)+qfx_urb(i,j)
L01779  !        lh(i,j)=(1.-frc_urb2d(i,j))*qfx_rural(i,j)*xlv
L01780           lh(i,j)=qfx(i,j)*xlv
L01781           HFX(I,J) = HFX_URB(I,J)+(1-FRC_URB2D(I,J))*HFX_RURAL(I,J)         ![W/m/m]
L01782              SH_URB2D(I,J)    = HFX_URB(I,J)/FRC_URB2D(I,J)
L01783              LH_URB2D(I,J)    = qfx_urb(i,j)*xlv
L01784              G_URB2D(I,J)     = grdflx_urb(i,j)
L01785              RN_URB2D(I,J)    = rs_abs_urb(i,j)+emiss_urb(i,j)*glw(i,j)-rl_up_urb(i,j)
L01786              ust(i,j)=(umom**2.+vmom**2.)**.25
L01787  !          if(tsk(i,j).gt.350)write(*,*)'tsk too big!',i,j,tsk(i,j)
L01788  !          if(tsk(i,j).lt.260)write(*,*)'tsk too small!',i,j,tsk(i,j),rl_up_tot,rl_up_urb(i,j),rl_up_rural
L01789  !                   print*,'ivgtyp,i,j,sigma_sb',ivgtyp(i,j),i,j,sigma_sb
L01790  !                   print*,'hfx,lh,qfx,grdflx,ts_urb2d',hfx(i,j),lh(i,j),qfx(i,j),grdflx(i,j),ts_urb2d(i,j)
L01791  !                   print*,'tsk,albedo,emiss',tsk(i,j),albedo(i,j),emiss(i,j)
L01792  !                if(i.eq.56.and.j.eq.29)then
L01793  !                   print*,'ivgtyp, qfx, hfx',ivgtyp(i,j),hfx_rural(i,j),qfx_rural(i,j)
L01794  !                   print*,'emiss_rural,emiss_urb',emiss_rural(i,j),emiss_urb(i,j)
L01795  !                   print*,'rl_up_rural,rl_up_urb(i,j)',rl_up_rural,rl_up_urb(i,j)
L01796  !                   print*,'tsk_rural,ts_urb2d(i,j),tsk',tsk_rural(i,j),ts_urb2d(i,j),tsk(i,j)
L01797  !                   print*,'reconstruction fei',((emiss(i,j)*tsk(i,j)**4.-frc_urb2d(i,j)*emiss_urb(i,j)*ts_urb2d(i,
↪   j)**4.)/(emiss_rural(i,j)*(1.-frc_urb2d(i,j))))**.25
L01798  !                   print*,'ivgtyp,hfx,hfx_urb,hfx_rural',hfx(i,j),hfx_urb(i,j),hfx_rural(i,j)
L01799  !                   print*,'lh,lh_rural',lh(i,j),lh_rural(i,j)
L01800  !                   print*,'qfx',qfx(i,j)
L01801  !                   print*,'ts_urb2d',ts_urb2d(i,j)
L01802  !                   print*,'ust',ust(i,j)
L01803  !                   print*,'swdown,glw',swdown(i,j),glw(i,j)
L01804  !                endif
L01805              else
L01806                SH_URB2D(I,J)    = 0.
L01807                LH_URB2D(I,J)    = 0.
L01808                G_URB2D(I,J)     = 0.
L01809                RN_URB2D(I,J)    = 0.
L01810              endif
L01811           enddo
L01812           enddo
L01813
L01814         endif                                                              !Bep end
```

讲解：SF_URBAN_PHYSICS=2/3 时 J/I 循环结束后**整片 tile 调一次** BEP/BEM（三维城市网格上算

出 A/B 源项），再逐点聚合四步：①a_*b*_ 全乘 FRC_URB2D（城市份额缩放，a_q/a_e 置零——BEP 不给水汽/TKE 的隐式分量）；② 第一层补回乡村份额 (1−FRC) 的 ust²/hfx/qfx 源项——非城市部分由乡村面层补全，保证净源项守恒；③HFX_URB=Σ B_T • ρ • cp • DZ • VL（显式源项按「层厚 × 空气体积分数」聚合）、UMOM/VMOM 同法，UST=(umom²+vmom²)^0.25 反演；④ 长波侧由 RL_UP_RURAL/RL_UP_URB 按面积比加权后**四次方根反演**总 TSK 与城市冠层 TS，反照率由吸收短波反解 rs_abs_tot/swdown。这是 ch17 与论文 2 的关键先例：**多层城市方案的输出不是直接写状态量，而是以源项 +面积比聚合进动力倾向**。

五、LSMINIT：状态初始化与 TBL 读取入口

```
!file: phys/module_sf_noahdrv.F    L1892-L1953
L01892  ! initialize three Noah LSM related tables
L01893     IF ( allowed_to_read ) THEN
L01894       CALL wrf_message( 'INITIALIZE THREE Noah LSM RELATED TABLES' )
L01895       CALL  SOIL_VEG_GEN_PARM( MMINLU, MMINSL )
L01896     ENDIF
L01897
L01898  ! GAC-->
L01899  ! 20130219 - No longer need these - see module_data_gocart_dust
L01900  !#if ( WRF_CHEM == 1 )
L01901  !!
L01902  !! need this parameter for dust parameterization in wrf/chem
L01903  !!
L01904  !   do I=1,NSLTYPE
L01905  !      porosity(i)=maxsmc(i)
L01906  !      drypoint(i)=drysmc(i)
L01907  !   enddo
L01908  !#endif
L01909  ! <--GAC
L01910
L01911     IF(.not.restart)THEN
L01912
L01913  #if ( EM_CORE==1 )
L01914     IF (irr_ph.NE.0)THEN
L01915     DO i = its,ite
L01916      DO j=jts,jte
L01917        my_seeds(j) =sqrt(ide*(real(j-1)**2))+sqrt(real(jde*i))
L01918  !      PRINT*,'myseed', my_seeds(j),j,jts,jds
L01919      END DO
L01920        CALL RANDOM_SEED ( PUT = my_seeds )
L01921        CALL RANDOM_NUMBER ( rand_tmp(i,:) )
L01922        CALL RANDOM_SEED ( GET = my_seeds )
L01923        CALL RANDOM_NUMBER ( rand_tmp(i,:) )
L01924        irr_rand_field(i,:)=int(modulo(rand_tmp(i,:)*100,real(irr_freq)))
L01925     END DO
L01926     END IF
L01927  #endif
L01928
L01929     itf=min0(ite,ide-1)
L01930     jtf=min0(jte,jde-1)
L01931
L01932     errflag = 0
L01933     DO j = jts,jtf
L01934       DO i = its,itf
L01935         IF ( ISLTYP( i,j ) .LT. 1 ) THEN
L01936           errflag = 1
L01937           WRITE(err_message,*)"module_sf_noahdrv.F: lsminit: out of range ISLTYP ",i,j,ISLTYP( i,j )
L01938           CALL wrf_message(err_message)
L01939         ENDIF
L01940         IF(.not.RDMAXALB) THEN
L01941           SNOALB(i,j)=MAXALB(IVGTYP(i,j))*0.01
L01942         ENDIF
L01943       ENDDO
L01944     ENDDO
L01945     IF ( errflag .EQ. 1 ) THEN
L01946  #if ( HWRF == 1 )
L01947        CALL wrf_message( "WARNING: message only; was fatal. module_sf_noahdrv.F: lsminit: out of range value "// &
L01948                          "of ISLTYP. Is this field in the input?" )
L01949  #else
L01950        CALL wrf_error_fatal( "module_sf_noahdrv.F: lsminit: out of range value "// &
L01951                          "of ISLTYP. Is this field in the input?" )
L01952  #endif
L01953     ENDIF
```

```
!file: phys/module_sf_noahdrv.F    L1955-L2016
L01955  ! initialize soil liquid water content SH2O
L01956
L01957  !  IF(.NOT.FNDSOILW) THEN
L01958
L01959  ! If no SWC, do the following
L01960  !        PRINT *,'SOIL WATER NOT FOUND - VALUE SET IN LSMINIT'
L01961         DO J = jts,jtf
L01962         DO I = its,itf
L01963           BX = BB(ISLTYP(I,J))
L01964           SMCMAX = MAXSMC(ISLTYP(I,J))
L01965           PSISAT = SATPSI(ISLTYP(I,J))
L01966          if ((bx > 0.0).and.(smcmax > 0.0).and.(psisat > 0.0)) then
L01967           DO NS=1, num_soil_layers
L01968  ! ----------------------------------------------------------------------
L01969  !SH2O  <= SMOIS for T < 273.149K (-0.001C)
L01970              IF (TSLB(I,NS,J) < 273.149) THEN
L01971  ! ----------------------------------------------------------------------
L01972  ! first guess following explicit solution for Flerchinger Eqn from Koren
L01973  ! et al, JGR, 1999, Eqn 17 (KCOUNT=0 in FUNCTION FRH2O).
L01974  ! ISLTPK is soil type
L01975                BX = BB(ISLTYP(I,J))
```

```
L01976               SMCMAX = MAXSMC(ISLTYP(I,J))
L01977               PSISAT = SATPSI(ISLTYP(I,J))
L01978               IF ( BX >  BLIM ) BX = BLIM
L01979               FK=(( (HLICE/(GRAV*(-PSISAT))) *                          &
L01980                  ((TSLB(I,NS,J)-T0)/TSLB(I,NS,J)) )**(-1/BX) )*SMCMAX
L01981               IF (FK < 0.02) FK = 0.02
L01982               SH2O(I,NS,J) = MIN( FK, SMOIS(I,NS,J) )
L01983  ! ---------------------------------------------------------------------
L01984  ! now use iterative solution for liquid soil water content using
L01985  ! FUNCTION FRH2O with the initial guess for SH2O from above explicit
L01986  ! first guess.
L01987               CALL FRH2O (FREE,TSLB(I,NS,J),SMOIS(I,NS,J),SH2O(I,NS,J),    &
L01988                  SMCMAX,BX,PSISAT)
L01989               SH2O(I,NS,J) = FREE
L01990              ELSE             ! of IF (TSLB(I,NS,J)
L01991  ! ---------------------------------------------------------------------
L01992  ! SH2O = SMOIS ( for T => 273.149K (-0.001C)
L01993               SH2O(I,NS,J)=SMOIS(I,NS,J)
L01994  ! ---------------------------------------------------------------------
L01995               ENDIF            ! of IF (TSLB(I,NS,J)
L01996             END DO             ! of DO NS=1, num_soil_layers
L01997            else                ! of if ((bx > 0.0)
L01998             DO NS=1, num_soil_layers
L01999              SH2O(I,NS,J)=SMOIS(I,NS,J)
L02000             END DO
L02001            endif               ! of if ((bx > 0.0)
L02002           ENDDO                ! DO I = its,itf
L02003           ENDDO                ! DO J = jts,jtf
L02004  !  ENDIF                       ! of IF(.NOT.FNDSOILW)THEN
L02005
L02006  ! initialize physical snow height SNOWH
L02007
L02008           IF(.NOT.FNDSNOWH)THEN
L02009  ! If no SNOWH do the following
L02010             CALL wrf_message( 'SNOW HEIGHT NOT FOUND - VALUE DEFINED IN LSMINIT' )
L02011             DO J = jts,jtf
L02012             DO I = its,itf
L02013               SNOWH(I,J)=SNOW(I,J)*0.005              ! SNOW in mm and SNOWH in m
L02014             ENDDO
L02015             ENDDO
L02016           ENDIF
```

讲解：allowed_to_read 时才调 SOIL_VEG_GEN_PARM（restart 不重读 TBL）；非 restart 时：ISLTYP<1 报错（HWRF 编译只警告——分值差异点）、RDMAXALB=false 时 SNOALB 按 VEGTYP 查 MAXALB 表。**SH2O 初值化**是本单元精华：T<273.149K 的层先用 Flerchinger 显式式给初猜 FK（KCOUNT=0 路径，12a § 七），再调 **FRH2O**（从 noahlsm USE 进来）牛顿迭代精化——冻土液态水的初始化与运行时用同一套物理，restart 重启后 SH2O 态直接可用；SNOWH 缺失时按 SNOW • 0.005（密度 0.2 g/cm³ 假设）反推；CANWAT 清零。irr_rand_field 是随机灌溉实验的种子场（仅 EM_CORE）。

六、SOIL_VEG_GEN_PARM：三张 TBL 的读取

```
!file: phys/module_sf_noahdrv.F    L2079-L2144
L02079           IF ( wrf_dm_on_monitor() ) THEN
L02080
L02081            OPEN(19, FILE='VEGPARM.TBL',FORM='FORMATTED',STATUS='OLD',IOSTAT=ierr)
L02082            IF(ierr .NE. OPEN_OK ) THEN
L02083              WRITE(message,FMT='(A)') &
L02084              'module_sf_noahlsm.F: soil_veg_gen_parm: failure opening VEGPARM.TBL'
L02085              CALL wrf_error_fatal ( message )
L02086            END IF
L02087
L02088
L02089            LUMATCH=0
L02090
L02091            loop_count = 0
L02092            READ (19,FMT='(A)',END=2002) a_string
L02093            FIND_LUTYPE : DO WHILE (LUMATCH == 0)
L02094               READ (19,*,END=2002)LUTYPE
L02095               READ (19,*)LUCATS,IINDEX
L02096
L02097               IF(LUTYPE.EQ.MMINLU)THEN
L02098                  WRITE( mess , * ) 'LANDUSE TYPE = ' // TRIM ( LUTYPE ) // ' FOUND', LUCATS,' CATEGORIES'
L02099                  CALL wrf_message( mess )
L02100                  LUMATCH=1
L02101               ELSE
L02102                  loop_count = loop_count+1
L02103                  call wrf_message ( "Skipping over LUTYPE = " // TRIM ( LUTYPE ) )
L02104                  FIND_VEGETATION_PARAMETER_FLAG : DO
L02105                     READ (19,FMT='(A)', END=2002) a_string
L02106                     IF ( a_string(1:21) .EQ. 'Vegetation Parameters' ) THEN
L02107                        EXIT FIND_VEGETATION_PARAMETER_FLAG
L02108                     ELSE IF ( loop_count .GE. loop_max ) THEN
L02109                        CALL wrf_error_fatal ( 'Too many loops in VEGPARM.TBL')
L02110                     ENDIF
L02111                  ENDDO FIND_VEGETATION_PARAMETER_FLAG
L02112               ENDIF
L02113            ENDDO FIND_LUTYPE
L02114  ! prevent possible array overwrite, Bill Bovermann, IBM, May 6, 2008
L02115           IF ( SIZE(SHDTBL)       < LUCATS .OR. &
L02116                SIZE(NROTBL)       < LUCATS .OR. &
L02117                SIZE(RSTBL)        < LUCATS .OR. &
L02118                SIZE(RGLTBL)       < LUCATS .OR. &
L02119                SIZE(HSTBL)        < LUCATS .OR. &
L02120                SIZE(SNUPTBL)      < LUCATS .OR. &
L02121                SIZE(MAXALB)       < LUCATS .OR. &
```

```
L02122              SIZE(LAIMINTBL)    < LUCATS .OR. &
L02123              SIZE(LAIMAXTBL)    < LUCATS .OR. &
L02124              SIZE(Z0MINTBL)     < LUCATS .OR. &
L02125              SIZE(Z0MAXTBL)     < LUCATS .OR. &
L02126              SIZE(ALBEDOMINTBL) < LUCATS .OR. &
L02127              SIZE(ALBEDOMAXTBL) < LUCATS .OR. &
L02128              SIZE(ZTOPVTBL) < LUCATS .OR. &
L02129              SIZE(ZBOTVTBL) < LUCATS .OR. &
L02130              SIZE(EMISSMINTBL ) < LUCATS .OR. &
L02131              SIZE(EMISSMAXTBL ) < LUCATS ) THEN
L02132            CALL wrf_error_fatal('Table sizes too small for value of LUCATS in module_sf_noahdrv.F')
L02133          ENDIF
L02134
L02135          IF(LUTYPE.EQ.MMINLU)THEN
L02136            DO LC=1,LUCATS
L02137               READ (19,*)IINDEX,SHDTBL(LC),                        &
L02138                          NROTBL(LC),RSTBL(LC),RGLTBL(LC),HSTBL(LC), &
L02139                          SNUPTBL(LC),MAXALB(LC), LAIMINTBL(LC),     &
L02140                          LAIMAXTBL(LC),EMISSMINTBL(LC),             &
L02141                          EMISSMAXTBL(LC), ALBEDOMINTBL(LC),         &
L02142                          ALBEDOMAXTBL(LC), Z0MINTBL(LC), Z0MAXTBL(LC),&
L02143             ZTOPVTBL(LC), ZBOTVTBL(LC)
L02144            ENDDO
```

讲解：VEGPARM.TBL 读取的标准双段式：wrf_dm_on_monitor 上单进程读、wrf_dm_bcast_* 向全组广播。按 MMINLU（namelist 地表数据类型串，如 USGS/MODIFIED_IGBP_MODIS_NOAH）逐块扫 LUTYPE，匹配后一次读入 LUCATS 行 ×17 列（SHD/NROOT/RS/RGL/HS/SNUP/MAXALB/LAI 与 EMISS/Z0/ALBEDO 的 min-max 对/ZTOPV/ZBOTV）；防越界检查（SIZE 小于 LUCATS 即 FATAL，Bill Bovermann 2008 补丁）；全局段读 TOPT/CMCMAX/CFACTR/RSMAX/BARE/NATURAL，末尾**强制校验 LCZ_1 至 LCZ_11 字段存在**——老版 TBL 缺这 11 行会明确报「老 TBL 配新代码」（WUDAPT LCZ 支持的门槛）。

差异注记（SOILPARM.TBL/GENPARM.TBL 段，L2237–L2306 与 L2311–L2357，回指原文不重录）：SOILPARM 按 MMINSL='STAS' 硬编码匹配 SLTYPE、读 SLCATS 行 ×10 列；GENPARM 无类匹配，直接读 NUM_SLOPE 条 SLOPE_DATA 与 SBETA/FXEXP/CSOIL/SALP/REFDK/REFKDT/FRZK/ZBOT/CZIL/SMLOW/SMHIGH/LVCOEF 十二个标量。**换土壤/植被数据集的实际操作 = 改 TBL 内容或加 LUTYPE 块，代码不动**；加新列才需要同步改 READ 行与 noahlsm 的数组声明（12a § 三）。

七、mosaic 双件（C 级注记）

7.1 lsm_mosaic 接口索引

```
!file: phys/module_sf_noahdrv.F    L2387-L2499
L02387  SUBROUTINE lsm_mosaic(DZ8W,QV3D,P8W3D,T3D,TSK,                    &
L02388                    HFX,QFX,LH,GRDFLX, QGH,GSW,SWDOWN,GLW,SMSTAV,SMSTOT, &
L02389                    SFCRUNOFF, UDRUNOFF,IVGTYP,ISLTYP,ISURBAN,ISICE,VEGFRA,    &
L02390                    ALBEDO,ALBBCK,ZNT,Z0,TMN,XLAND,XICE,EMISS,EMBCK,   &
L02391                    SNOWC,QSFC,RAINBL,MMINLU,                          &
L02392                    num_soil_layers,DT,DZS,ITIMESTEP,                  &
L02393                    SMOIS,TSLB,SNOW,CANWAT,                            &
L02394                    CHS,CHS2,CQS2,CPM,ROVCP,SR,chklowq,lai,qz0,   & !H
L02395                    myj,frpcpn,                                        &
L02396                    SH2O,SNOWH,                                        & !H
L02397                    U_PHY,V_PHY,                                       & !I
L02398                    SNOALB,SHDMIN,SHDMAX,SHDAVG,                       & !I
L02399                    SNOTIME,                                           & !?
L02400                    ACSNOM,ACSNOW,                                     & !O
L02401                    SNOPCX,                                            & !O
L02402                    POTEVP,                                            & !O
L02403                    SMCREL,                                            & !O
L02404                    XICE_THRESHOLD,                                    &
L02405                    RDLAI2D,USEMONALB,                                 &
L02406                    RIB,                                               & !?
L02407                    NOAHRES,OPT_THCND,                                 &
L02408                   NLCAT,landusef,landusef2,                            & ! danli mosaic
L02409                   sf_surface_mosaic,mosaic_cat,mosaic_cat_index,       & ! danli mosaic
L02410                   TSK_mosaic,QSFC_mosaic,                              & ! danli mosaic
L02411                   TSLB_mosaic,SMOIS_mosaic,SH2O_mosaic,                & ! danli mosaic
L02412                   CANWAT_mosaic,SNOW_mosaic,                           & ! danli mosaic
L02413                   SNOWH_mosaic,SNOWC_mosaic,                           & ! danli mosaic
L02414                   ALBEDO_mosaic,ALBBCK_mosaic,                         & ! danli mosaic
L02415                   EMISS_mosaic, EMBCK_mosaic,                          & ! danli mosaic
L02416                   ZNT_mosaic, Z0_mosaic,                               & ! danli mosaic
L02417                   HFX_mosaic,QFX_mosaic,                               & ! danli mosaic
L02418                   LH_mosaic, GRDFLX_mosaic, SNOTIME_mosaic,            & ! danli mosaic
L02419                   RC_mosaic, LAI_mosaic,                               &
L02420  ! Noah UA changes
L02421                    ua_phys,flx4_2d,fvb_2d,fbur_2d,fgsn_2d,       &
L02422                    ids,ide, jds,jde, kds,kde,                    &
L02423                    ims,ime, jms,jme, kms,kme,                    &
L02424                    its,ite, jts,jte, kts,kte,                    &
L02425                    sf_urban_physics,                             &
L02426                    CMR_SFCDIF,CHR_SFCDIF,CMC_SFCDIF,CHC_SFCDIF,  &
L02427                    CMGR_SFCDIF,CHGR_SFCDIF,                      &
L02428  !Optional Urban
L02429                    TR_URB2D,TB_URB2D,TG_URB2D,TC_URB2D,QC_URB2D, & !H urban
L02430                    UC_URB2D,                                     & !H urban
L02431                    XXXR_URB2D,XXXB_URB2D,XXXG_URB2D,XXXC_URB2D,  & !H urban
L02432                    TRL_URB3D,TBL_URB3D,TGL_URB3D,                & !H urban
L02433                    SH_URB2D,LH_URB2D,G_URB2D,RN_URB2D,TS_URB2D,  & !H urban
```

```
L02434                    TR_URB2D_mosaic,TB_URB2D_mosaic,              & !H urban  danli mosaic
L02435                    TG_URB2D_mosaic,TC_URB2D_mosaic,              & !H urban  danli mosaic
L02436                    QC_URB2D_mosaic,UC_URB2D_mosaic,              & !H urban  danli mosaic
L02437                    TRL_URB3D_mosaic,TBL_URB3D_mosaic,            & !H urban  danli mosaic
L02438                    TGL_URB3D_mosaic,                             & !H urban  danli mosaic
L02439                    SH_URB2D_mosaic,LH_URB2D_mosaic,              & !H urban  danli mosaic
L02440                    G_URB2D_mosaic,RN_URB2D_mosaic,               & !H urban  danli mosaic
L02441                    TS_URB2D_mosaic,                              & !H urban  danli mosaic
L02442                    TS_RUL2D_mosaic,                              & !H urban  danli mosaic
L02443                    PSIM_URB2D,PSIH_URB2D,U10_URB2D,V10_URB2D,    & !O urban
L02444                    GZ1OZ0_URB2D,  AKMS_URB2D,                    & !O urban
L02445                    TH2_URB2D,Q2_URB2D, UST_URB2D,                & !O urban
L02446                    DECLIN_URB,COSZ_URB2D,OMG_URB2D,              & !I urban
L02447                    XLAT_URB2D,                                   & !I urban
L02448                    num_roof_layers, num_wall_layers,             & !I urban
L02449                    num_road_layers, DZR, DZB, DZG,               & !I urban
L02450                    CMCR_URB2D,TGR_URB2D,TGRL_URB3D,SMR_URB3D,    & !H urban
L02451                    julian,julyr,                                 & !H urban
L02452                    DRELR_URB2D,DRELB_URB2D,DRELG_URB2D,          & !H urban
L02453                    FLXHUMR_URB2D,FLXHUMB_URB2D,FLXHUMG_URB2D,    & !H urban
L02454                    TVG_URB2D,XXXVG_URB2D,TVGL_URB3D,SMG_URB3D,TT_URB2D,CMCG_URB2D, & !H urban
L02455                    FLXHUMVG_URB2D,FLXHUMT_URB2D,                 & !H urban
L02456                    FRC_URB2D,UTYPE_URB2D,                        & !O
L02457                    num_urban_ndm,                                & !I multi-layer urban
L02458                    urban_map_zrd,                                & !I multi-layer urban
L02459                    urban_map_zwd,                                & !I multi-layer urban
L02460                    urban_map_gd,                                 & !I multi-layer urban
L02461                    urban_map_zd,                                 & !I multi-layer urban
L02462                    urban_map_zdf,                                & !I multi-layer urban
L02463                    urban_map_bd,                                 & !I multi-layer urban
L02464                    urban_map_wd,                                 & !I multi-layer urban
L02465                    urban_map_gbd,                                & !I multi-layer urban
L02466                    urban_map_fbd,                                & !I multi-layer urban
L02467                    urban_map_zgrd,                               & !I multi-layer urban
L02468                    num_urban_hi,                                 & !I multi-layer urban
L02469                    use_wudapt_lcz,                               & !I wudapt
L02470                    slucm_distributed_drag,                & !I slucm
L02471                    tsk_rural_bep,                                & !H multi-layer urban
L02472                    trb_urb4d,tw1_urb4d,tw2_urb4d,tgb_urb4d,      & !H multi-layer urban
L02473                    tlev_urb3d,qlev_urb3d,                        & !H multi-layer urban
L02474                    tw1lev_urb3d,tw2lev_urb3d,                    & !H multi-layer urban
L02475                    tglev_urb3d,tflev_urb3d,                      & !H multi-layer urban
L02476                    sf_ac_urb3d,lf_ac_urb3d,cm_ac_urb3d,          & !H multi-layer urban
L02477                    sfvent_urb3d,lfvent_urb3d,                    & !H multi-layer urban
L02478                    sfwin1_urb3d,sfwin2_urb3d,                    & !H multi-layer urban
L02479                    sfw1_urb3d,sfw2_urb3d,sfr_urb3d,sfg_urb3d,    & !H multi-layer urban
L02480                    ep_pv_urb3d,t_pv_urb3d,                         & !RMS
L02481                    trv_urb4d,qr_urb4d,qgr_urb3d,tgr_urb3d,         & !RMS
L02482                    drain_urb4d,draingr_urb3d,sfrv_urb3d,           & !RMS
L02483                    lfrv_urb3d,dgr_urb3d,dg_urb3d,lfr_urb3d,lfg_urb3d,& !RMS
L02484                    lp_urb2d,hi_urb2d,lb_urb2d,hgt_urb2d,         & !H multi-layer urban
L02485                    mh_urb2d,stdh_urb2d,lf_urb2d,                 & !SLUCM
L02486                    lf_urb2d_s, z0_urb2d,                         & !SLUCM
L02487                    th_phy,rho,p_phy,ust,                         & !I multi-layer urban
L02488                    gmt,julday,xlong,xlat,                        & !I multi-layer urban
L02489                    a_u_bep,a_v_bep,a_t_bep,a_q_bep,              & !O multi-layer urban
L02490                    a_e_bep,b_u_bep,b_v_bep,                      & !O multi-layer urban
L02491                    b_t_bep,b_q_bep,b_e_bep,dlg_bep,              & !O multi-layer urban
L02492                    dl_u_bep,sf_bep,vl_bep                        & !O multi-layer urban
L02493  #ifdef WRF_HYDRO
L02494                   ,sfcheadrt,INFXSRT, soldrain                   & !hydro
L02495  #endif
L02496                   ,SDA_HFX, SDA_QFX, HFX_BOTH, QFX_BOTH, QNORM, fasdas     &   !fasdas
L02497                   ,RC2,XLAI2                                     & !O
L02498                   ,IRR_CHAN                                      &
L02499                    )
```

讲解：sf_surface_mosaic=1 时的多斑块版（danli mosaic）：每个模式格点拆 mosaic_cat 个土地斑块——NLCAT 类土地利用比例 landusef 归一为 landusef2、mosaic_cat_index 记录斑块类别映射；全部状态与通量带 *mosaic* 后缀做并行副本，其中土壤三维量 *TSLB_mosaic/SMOIS_mosaic/SH2O_mosaic* 维度是 *num_soil_layers* • *mosaic_cat* 的**折叠存储**（与 *BEP* 的 *ind** 索引折叠同一思路，code_14a § 四 ①）；UCM/BEP 的 mosaic 副本也在参列表里。逐斑块调 SFLX 后按斑块面积加权聚合回格点（聚合段体例与 §4.5 相同，不重收）。教学价值：子网格地表异质性（ch16）的最简实现样本。

7.2 lsm_mosaic_init

C 注（L5041–L5443，403 行，全文不录）：非 restart 时选斑块——把 landusef 各类按占比排序（Temp*/NumPairs 的交换逻辑）取前 mosaic_cat 类并归一占比；海冰阈值 xice_threshold 由 fractional_seaice 决定（0 时 0.5、1 时 0.02——分值海冰开关）；把格点状态（TSK/SMOIS/TSLB/SNOW/CANWAT 与 ALBEDO/EMISS/Z0 及城市状态组）复制到 _mosaic 副本并按类别初值化。namelist 对应 sf_surface_mosaic/mosaic_cat。

八、改造入口

1. **海冰能量平衡改造**（如引入冰内传导/厚冰分类）：ICE=1 分支的 CYCLE 是替换点——现状海冰完全依赖面层方案。
2. **灌溉实验两级挂点**：粗——本模块 IRI_SCHEME 定时补湿（改 SMC 数组）；细——SMFLX 的 IRRIGATION_CHANNEL 通道（12b § 六，逐时间步加水当量）；论文 2/城市灌溉实验用后者。

3. **UCM 城市背景参数**：L982 起的 NATURAL 重定义、ALBEDOK=0.2、EMISSI=0.98。
4. **TBL 数据集更换**：§六的双段式与 LCZ 校验。
5. **能量闭合排错**：NOAHRES 定义式（§4.5④）——残差非零先查 FLX1/2/3 与 SSOIL 符号。
6. **fasdas 同化扩展**：XSDA_HFX/XSDA_QFX 的格点换算与 T1 直加热模式（§4.5②）。

九、上下游链接

- 方案本体：`code_12a_module_sf_noahlsm.md`、`code_12b_module_sf_noahlsm.md`
- 调用链上游：`code_24_module_surface_driver.md`；城市方案：`code_14a_module_sf_bep_bem.md`、`code_16_module_sf_bem.md`
- 冰川简化版 module_sf_noahlsm_glacial_only.F（SFLX_GLACIAL，暂不单列成篇，C 级备选）
- 教材正文：ch16_land_surface_family.md、ch17_urban_canopy.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_14a — BEP/BEM 共享底座 · 上半（module_sf_bep_bem.F：主入口与柱求解器）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_bep_bem.F	5706	8d13861f088e806d2558dd37e31126f2

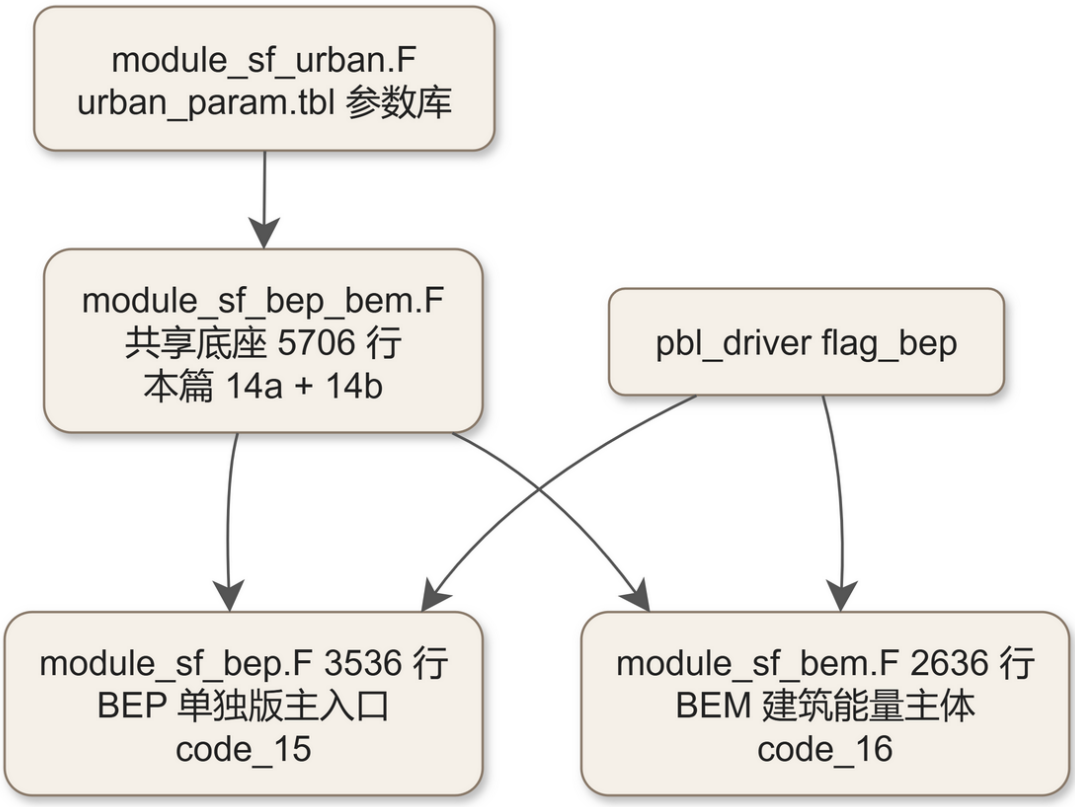
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级全文拆两：14a = 文件头 + BEP_BEM（主入口）+ BEP1D（柱求解器）+ param…buildings（L1–L2891）；14b = urban_meso 起的全部辐射/导热/通量/初始化单元（L2892–L5705），见 `code_14b_module_sf_bep_bem.md`。

二、定位

一句话：BEP（城市冠层参数化，Martilli–Clappier–Rotach 2002 BLM）与 BEM（建筑能量模型，Salamanca–Martilli 2009/2010）的**共享底座模块**——双网格（模式网格 ↔ 5 m 城市网格）、概率化建筑高度分布、多方向街道峡谷辐射、以及「方案源项 A/B 分解」的完整实现；module_sf_bep.F（BEP 单独版）与 module_sf_bem.F（BEM 主体）都建立在本模块之上。

谁在调它：module_sf_bep.F 与 module_sf_bem.F 在其主入口内调用本模块的 param/view_factors/flux_flat 等单元；直接驱动来自 pbl_driver 的 BEP/BEM 感知路径（flag_bep，见 `code_22_module_pbl_driver.md` §3.2/3.4——a_u_bep/a_t_bep 等源项数组经 diff3d 或各 PBL 方案的多城市分支进入动力倾向）。教材锚点：ch17_urban_canopy.md 全章；论文 2（BEM 标量 → 逐格场改造）的改造对象与先例机制都在本文件。

三文件分工（源码普查）：



三、文件头：维度常数与物理常数

```
!file: phys/module_sf_bep_bem.F    L1-L94
L00001  MODULE module_sf_bep_bem
L00002  #if defined(mpas)
L00003  use mpas_atmphys_utilities, only: physics_error_fatal
L00004  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00005  #else
L00006  use module_wrf_error
L00007  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00008  !USE module_model_constants
L00009  #endif
L00010   USE module_sf_urban
L00011   USE module_sf_bem
L00012   USE module_bep_bem_helper, ONLY: nurbm
L00013
L00014  ! SGClarke 09/11/2008
L00015  ! Access urban_param.tbl values through calling urban_param_init in module_physics_init
L00016  ! for CASE (BEPSCHEME) select sf_urban_physics
L00017  !
L00018  ! ------------------------------------------------------------------
L00019  !  Dimension for the array used in the BEP module
L00020  ! ------------------------------------------------------------------
L00021
L00022        integer nurbmax         ! Maximum number of urban classes
L00023        parameter (nurbmax=11)
L00024
L00025        integer ndm             ! Maximum number of street directions
L00026        parameter (ndm=2)
L00027
L00028        integer nz_um           ! Maximum number of vertical levels in the urban grid
L00029        parameter(nz_um=18)
L00030
L00031        integer ng_u            ! Number of grid levels in the ground
L00032        parameter (ng_u=10)
L00033
L00034        integer ngr_u            ! Number of grid levels in green roof
L00035        parameter (ngr_u=10)
L00036
L00037        integer nwr_u            ! Number of grid levels in the walls or roofs
L00038        parameter (nwr_u=10)
L00039
L00040        integer nf_u             !Number of grid levels in the floors (BEM)
L00041        parameter (nf_u=10)
L00042
L00043        integer ngb_u            !Number of grid levels in the ground below building (BEM)
L00044        parameter (ngb_u=10)
L00045
L00046        real dz_u                ! Urban grid resolution
L00047        parameter (dz_u=5.)
L00048
L00049        integer nbui_max         !maximum number of types of buildings in an urban class
L00050        parameter (nbui_max=15)   !must be less or equal than nz_um
L00051
L00052
```

```
L00053        real h_water
L00054        parameter(h_water=0.0009722) !mm of irrigation per hour
L00055
L00056  !-------------------------------------------------------------------------------
L00057  !Parameters of the windows. The glasses of windows are considered without films  -
L00058  !Read the paper of J.Karlsson and A.Roos(2000):"modelling the angular behaviour  -
L00059  !of the total solar energy transmittance of windows".Solar Energy Vol.69,No.4,   -
L00060  !pp. 321-329, for more details.                                                 -
L00061  !-------------------------------------------------------------------------------
L00062        integer p_num            !number of panes in the windows (1,2 or 3)
L00063        parameter (p_num=2)
L00064        integer q_num            !category number for the windows (q_num= 4, standard glasses)
L00065        parameter(q_num=4)       !Possible values 1,2,...,10
L00066
L00067  ! The change of ng_u, nwr_u should be done in agreement with the block data
L00068  !  in the routine "surf_temp"
L00069  ! ------------------------------------------------------------------------
L00070  !  Constant used in the BEP module
L00071  ! ------------------------------------------------------------------------
L00072
L00073        real vk                  ! von Karman constant
L00074        real g_u                 ! Gravity acceleration
L00075        real pi                  !
L00076        real r                   ! Perfect gas constant
L00077        real cp_u                ! Specific heat at constant pressure
L00078        real rcp_u               !
L00079        real sigma               !
L00080        real p0                  ! Reference pressure at the sea level
L00081        real latent              ! Latent heat of vaporization [J/kg] (used in BEM)
L00082        real dgmax               ! Maximum ground water holding capacity (mm)
L00083        real drmax               ! Maximum ground roof holding capacity (mm)
L00084
L00085        parameter(vk=0.40,g_u=9.81,pi=3.141592653,r=287.,cp_u=1004.)
L00086        parameter(rcp_u=r/cp_u,sigma=5.67e-08,p0=1.e+5,latent=2.45e+06,dgmax=1.,drmax=1.)
L00087
L00088  ! ------------------------------------------------------------------------
L00089
L00090
L00091
L00092
L00093     CONTAINS
L00094
```

讲解：① 本模块不用 module_model_constants——自带 vk/g_u/cp_u=1004/sigma/p0/latent=2.45e6 一套常数（MPAS/WRF 双框架编译，头两行宏切换错误出口）；② 城市网格维度全在此定死：nurbmax=11 城市类、ndm=2 街道方向、nz_um=18 城市层、dz_u=5 m（城市网格顶 85 m）、ng_u/ngr_u/nwr_u/nf_u/ngb_u=10（地面/绿屋顶/墙或屋顶/楼板/楼下地基各 10 层）、nbui_max=15 每类最多建筑型；③nurbm（实际城市类数）不在本文件——来自 module_bep_bem_helper，是「编译上限 vs 运行时类数」的解耦；④ 窗口参数 p_num=2 层玻璃、q_num=4 标准类别，对应 Karlsson–Roos 2000 角向透射模型（albwindow 用，14b）；⑤dgmax/drmax=1 mm 是路面/屋顶积水容量；⑥h_water=0.0009722 mm/h 灌溉率（绿屋顶灌溉调度用，BEP1D）。

四、主入口 BEP_BEM（全文）

```
!file: phys/module_sf_bep_bem.F    L95-L1162    (subroutine BEP_BEM)
L00095        subroutine BEP_BEM(FRC_URB2D,UTYPE_URB2D,itimestep,dz8w,dt,u_phy,v_phy,      &
L00096                           th_phy,rho,p_phy,swdown,glw,                          &
L00097                           gmt,julday,xlong,xlat,                                &
L00098                           declin_urb,cosz_urb2d,omg_urb2d,                      &
L00099                           num_urban_ndm,  urban_map_zrd,  urban_map_zwd, urban_map_gd, &
L00100                           urban_map_zd,  urban_map_zdf,   urban_map_bd, urban_map_wd,  &
L00101                           urban_map_gbd,  urban_map_fbd,                        &
L00102                           urban_map_zgrd,  num_urban_hi,                        &
L00103                           trb_urb4d,tw1_urb4d,tw2_urb4d,tgb_urb4d,              &
L00104                           tlev_urb3d,qlev_urb3d,tw1lev_urb3d,tw2lev_urb3d,      &
L00105                           tglev_urb3d,tflev_urb3d,sf_ac_urb3d,lf_ac_urb3d,      &
L00106                           cm_ac_urb3d,                                          &
L00107                           sfvent_urb3d,lfvent_urb3d,                            &
L00108                           sfwin1_urb3d,sfwin2_urb3d,                            &
L00109                           sfw1_urb3d,sfw2_urb3d,sfr_urb3d,sfg_urb3d,            &
L00110                ep_pv_urb3d,t_pv_urb3d,                                  &
L00111                trv_urb4d,qr_urb4d,qgr_urb3d,tgr_urb3d,                  &
L00112                          drain_urb4d,draingr_urb3d,                     &
L00113                sfrv_urb3d,lfrv_urb3d,                                   &
L00114                          dgr_urb3d,dg_urb3d,                            &
L00115                          lfr_urb3d,lfg_urb3d,rainbl,swddir,swddif,      &
L00116                          lp_urb2d,hi_urb2d,lb_urb2d,hgt_urb2d,          &
L00117                          a_u,a_v,a_t,a_e,b_u,b_v,                       &
L00118                          b_t,b_e,b_q,d1g,d1_u,sf,vl,                    &
L00119                          rl_up,rs_abs,emiss,grdflx_urb,qv_phy,          &
L00120                          ids,ide, jds,jde, kds,kde,                     &
L00121                          ims,ime, jms,jme, kms,kme,                     &
L00122                          its,ite, jts,jte, kts,kte)
L00123
L00124        implicit none
L00125
L00126  !-------------------------------------------------------------------------
L00127  !     Input
L00128  !-------------------------------------------------------------------------
L00129     INTEGER ::                       ids,ide, jds,jde, kds,kde,  &
L00130                                      ims,ime, jms,jme, kms,kme,  &
L00131                                      its,ite, jts,jte, kts,kte,  &
L00132                                      itimestep
L00133
L00134
L00135     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   DZ8W
```

```
L00136     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   P_PHY
L00137     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   RHO
L00138     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   TH_PHY
L00139     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   T_PHY
L00140     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   U_PHY
L00141     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   V_PHY
L00142     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   U
L00143     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   V
L00144     REAL, DIMENSION( ims:ime , jms:jme )        ::   GLW
L00145     REAL, DIMENSION( ims:ime , jms:jme )        ::   swdown
L00146     REAL, DIMENSION( ims:ime , jms:jme )        ::   swddir
L00147     REAL, DIMENSION( ims:ime , jms:jme )        ::   swddif
L00148      REAL, DIMENSION( ims:ime, jms:jme )         ::   UST
L00149     INTEGER, DIMENSION( ims:ime , jms:jme ), INTENT(IN )::   UTYPE_URB2D
L00150     REAL, DIMENSION( ims:ime , jms:jme ), INTENT(IN )::   FRC_URB2D
L00151     REAL, INTENT(IN   )   ::                                GMT
L00152     INTEGER, INTENT(IN  ) ::                               JULDAY
L00153     REAL, DIMENSION( ims:ime, jms:jme ),                          &
L00154           INTENT(IN   )  ::                             XLAT, XLONG
L00155     REAL, INTENT(IN) :: DECLIN_URB
L00156     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN) :: COSZ_URB2D
L00157     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN) :: OMG_URB2D
L00158     INTEGER, INTENT(IN  ) :: urban_map_zrd
L00159     INTEGER, INTENT(IN  ) :: urban_map_zwd
L00160     INTEGER, INTENT(IN  ) :: urban_map_gd
L00161     INTEGER, INTENT(IN  ) :: urban_map_zd
L00162     INTEGER, INTENT(IN  ) :: urban_map_zdf
L00163     INTEGER, INTENT(IN  ) :: urban_map_bd
L00164     INTEGER, INTENT(IN  ) :: urban_map_wd
L00165     INTEGER, INTENT(IN  ) :: urban_map_gbd
L00166     INTEGER, INTENT(IN  ) :: urban_map_fbd
L00167     INTEGER, INTENT(IN  ) :: num_urban_ndm
L00168     INTEGER, INTENT(IN) :: num_urban_hi
L00169     INTEGER , INTENT(IN)        ::      urban_map_zgrd
L00170     REAL, DIMENSION( ims:ime, 1:urban_map_zrd, jms:jme ), INTENT(INOUT) :: trb_urb4d
L00171     REAL, DIMENSION( ims:ime, 1:urban_map_zwd, jms:jme ), INTENT(INOUT) :: tw1_urb4d
L00172     REAL, DIMENSION( ims:ime, 1:urban_map_zwd, jms:jme ), INTENT(INOUT) :: tw2_urb4d
L00173     REAL, DIMENSION( ims:ime, 1:urban_map_gd , jms:jme ), INTENT(INOUT) :: tgb_urb4d
L00174     REAL, DIMENSION( ims:ime, 1:urban_map_zgrd, jms:jme ), INTENT(INOUT) :: trv_urb4d
L00175     REAL, DIMENSION( ims:ime, 1:urban_map_zgrd, jms:jme ), INTENT(INOUT) :: qr_urb4d
L00176     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: qgr_urb3d
L00177     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: tgr_urb3d
L00178     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: drain_urb4d
L00179     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN) :: rainbl
L00180     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: draingr_urb3d
L00181   !New variables used for BEM
L00182     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ):: qv_phy
L00183     REAL, DIMENSION( ims:ime, 1:urban_map_bd, jms:jme ), INTENT(INOUT) :: tlev_urb3d
L00184     REAL, DIMENSION( ims:ime, 1:urban_map_bd , jms:jme ), INTENT(INOUT) :: qlev_urb3d
L00185     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: tw1lev_urb3d
L00186     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: tw2lev_urb3d
L00187     REAL, DIMENSION( ims:ime, 1:urban_map_gbd, jms:jme ), INTENT(INOUT) :: tglev_urb3d
L00188     REAL, DIMENSION( ims:ime, 1:urban_map_fbd, jms:jme ), INTENT(INOUT) :: tflev_urb3d
L00189     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: lf_ac_urb3d
L00190     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: sf_ac_urb3d
L00191     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: cm_ac_urb3d
L00192     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: ep_pv_urb3d
L00193     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: t_pv_urb3d
L00194     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: sfvent_urb3d
L00195     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) :: lfvent_urb3d
L00196     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: sfwin1_urb3d
L00197     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: sfwin2_urb3d
L00198
L00199   !End variables
L00200     REAL, DIMENSION( ims:ime, 1:urban_map_zd , jms:jme ), INTENT(INOUT) :: sfw1_urb3d
L00201     REAL, DIMENSION( ims:ime, 1:urban_map_zd , jms:jme ), INTENT(INOUT) :: sfw2_urb3d
L00202     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: sfr_urb3d
L00203     REAL, DIMENSION( ims:ime, 1:num_urban_ndm, jms:jme ), INTENT(INOUT) :: sfg_urb3d
L00204     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: sfrv_urb3d
L00205     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: lfrv_urb3d
L00206     REAL, OPTIONAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ),INTENT(INOUT) :: dgr_urb3d !GRZ
L00207     REAL, OPTIONAL, DIMENSION( ims:ime, 1:num_urban_ndm, jms:jme ),INTENT(INOUT) :: dg_urb3d !GRZ
L00208     REAL, OPTIONAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ),INTENT(INOUT) :: lfr_urb3d !GRZ
L00209     REAL, OPTIONAL, DIMENSION( ims:ime, 1:num_urban_ndm, jms:jme ),INTENT(INOUT) :: lfg_urb3d !G
L00210
L00211     REAL, DIMENSION( ims:ime, 1:num_urban_hi, jms:jme ), INTENT(IN) :: hi_urb2d
L00212     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: lp_urb2d
L00213     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: lb_urb2d
L00214     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: hgt_urb2d
L00215
L00216     real z(ims:ime,kms:kme,jms:jme)            ! Vertical coordinates
L00217     REAL, INTENT(IN )::   DT      ! Time step
L00218
L00219   !-------------------------------------------------------------------------
L00220   !     Output
L00221   !-------------------------------------------------------------------------
L00222   !
L00223   !    Implicit and explicit components of the source and sink terms at each levels,
L00224   !     the fluxes can be computed as follow: FX = A*X + B   example: t_fluxes = a_t * pt + b_t
L00225         real a_u(ims:ime,kms:kme,jms:jme)         ! Implicit component for the momemtum in X-direction (center)
L00226         real a_v(ims:ime,kms:kme,jms:jme)         ! Implicit component for the momemtum in Y-direction (center)
L00227         real a_t(ims:ime,kms:kme,jms:jme)         ! Implicit component for the temperature
L00228         real a_e(ims:ime,kms:kme,jms:jme)         ! Implicit component for the TKE
L00229         real b_u(ims:ime,kms:kme,jms:jme)         ! Explicit component for the momemtum in X-direction (center)
L00230         real b_v(ims:ime,kms:kme,jms:jme)         ! Explicit component for the momemtum in Y-direction (center)
L00231         real b_t(ims:ime,kms:kme,jms:jme)         ! Explicit component for the temperature
L00232         real b_e(ims:ime,kms:kme,jms:jme)         ! Explicit component for the TKE
L00233         real b_q(ims:ime,kms:kme,jms:jme)         ! Explicit component for the Humidity
L00234         real dlg(ims:ime,kms:kme,jms:jme)         ! Height above ground (L_ground in formula (24) of the BLM paper).
L00235         real dl_u(ims:ime,kms:kme,jms:jme)        ! Length scale (lb in formula (22) ofthe BLM paper).
L00236   ! urban surface and volumes
L00237         real sf(ims:ime,kms:kme,jms:jme)           ! surface of the urban grid cells
L00238         real vl(ims:ime,kms:kme,jms:jme)             ! volume of the urban grid cells
L00239   ! urban fluxes
L00240         real rl_up(its:ite,jts:jte) ! upward long wave radiation
L00241         real rs_abs(its:ite,jts:jte) ! absorbed short wave radiation
```

```
L00242        real emiss(its:ite,jts:jte)  ! emissivity averaged for urban surfaces
L00243        real grdflx_urb(its:ite,jts:jte)  ! ground heat flux for urban areas
L00244  !---------------------------------------------------------------------
L00245  !     Local
L00246  !---------------------------------------------------------------------
L00247        real hi_urb(its:ite,1:nz_um,jts:jte) ! Height histograms of buildings
L00248        real hi_urb1D(nz_um)                 ! Height histograms of buildings
L00249        real ss_urb(nz_um,nurbmax)           ! Probability that a building has an height equal to z
L00250        real pb_urb(nz_um)                   ! Probability that a building has an height greater or equal to z
L00251        real hb_u(nz_um)                     ! Bulding's heights
L00252        integer nz_urb(nurbmax)              ! Number of layer in the urban grid
L00253        integer nzurban(nurbmax)
L00254
L00255  !    Building parameters
L00256        real alag_u(nurbmax)                    ! Ground thermal diffusivity [m^2 s^-1]
L00257        real alaw_u(nurbmax)                    ! Wall thermal diffusivity [m^2 s^-1]
L00258        real alar_u(nurbmax)                    ! Roof thermal diffusivity [m^2 s^-1]
L00259        real csg_u(nurbmax)                     ! Specific heat of the ground material [J m^3 K^-1]
L00260        real csw_u(nurbmax)                     ! Specific heat of the wall material [J m^3 K^-1]
L00261        real csr_u(nurbmax)                     ! Specific heat of the roof material [J m^3 K^-1]
L00262        real twini_u(nurbmax)                   ! Initial temperature inside the building's wall [K]
L00263        real trini_u(nurbmax)                   ! Initial temperature inside the building's roof [K]
L00264        real tgini_u(nurbmax)                   ! Initial road temperature
L00265
L00266  !
L00267  !   Building materials
L00268  !
L00269
L00270        real csg(ng_u)           ! Specific heat of the ground material [J m^3 K^-1]
L00271        real csw(nwr_u)          ! Specific heat of the wall material for the current urban class [J m^3 K^-1]
L00272        real csr(nwr_u)          ! Specific heat of the roof material for the current urban class [J m^3 K^-1]
L00273        real csgb(ngb_u)         ! Specific heat of the ground material below the buildings at each ground levels[J m^3
↪  K^-1]
L00274        real csf(nf_u)           ! Specific heat of the floors materials in the buildings at each levels[J m^3 K^-1]
L00275        real alar(nwr_u+1)       ! Roof thermal diffusivity for the current urban class [W/m K]
L00276        real alaw(nwr_u+1)       ! Walls thermal diffusivity for the current urban class [W/m K]
L00277        real alag(ng_u)          ! Ground thermal diffusivity for the current urban class [m^2 s^-1]
L00278        real alagb(ngb_u+1)      ! Ground thermal diffusivity below the building at each wall layer [W/m K]
L00279        real alaf(nf_u+1)        ! Floor thermal diffusivity at each wall layers [W/m K]
L00280        real dzr(nwr_u)          ! Layer sizes in the roofs [m]
L00281        real dzf(nf_u)           ! Layer sizes in the floors[m]
L00282        real dzw(nwr_u)          ! Layer sizes in the walls [m]
L00283        real dzgb(ngb_u)         ! Layer sizes in the ground below the buildings [m]
L00284
L00285  !
L00286  !New street and radiation parameters
L00287
L00288
L00289        real bs(ndm)              ! Building width for the current urban class
L00290        real ws(ndm)              ! Street widths of the current urban class
L00291        real strd(ndm)            ! Street lengths for the current urban class
L00292        real drst(ndm)            ! street directions for the current urban class
L00293        real ss(nz_um)            ! Probability to have a building with height h
L00294        real pb(nz_um)            ! Probability to have a building with an height equal
L00295        real HFGR_D(nz_um)
L00296  !New roughness and buildings parameters
L00297  !
L00298        real z0(ndm,nz_um)        ! Roughness lengths "profiles"
L00299        real bs_urb(ndm,nurbmax)  ! Building width
L00300        real ws_urb(ndm,nurbmax)  ! Street width
L00301
L00302  !
L00303  ! for twini_u, and trini_u the initial value at the deepest level is kept constant during the simulation
L00304  !
L00305  !    Radiation paramters
L00306        real albg_u(nurbmax)                    ! Albedo of the ground
L00307        real albw_u(nurbmax)                    ! Albedo of the wall
L00308        real albr_u(nurbmax)                    ! Albedo of the roof
L00309        real albwin_u(nurbmax)                  ! Albedo of the windows
L00310        real emwind_u(nurbmax)                  ! Emissivity of windows
L00311        real emg_u(nurbmax)                     ! Emissivity of ground
L00312        real emw_u(nurbmax)                     ! Emissivity of wall
L00313        real emr_u(nurbmax)                     ! Emissivity of roof
L00314        real gr_frac_roof_u(nurbmax)
L00315        real pv_frac_roof_u(nurbmax)
L00316        integer gr_flag_u
L00317        integer gr_type_u
L00318
L00319  !   fww_u,fwg_u,fgw_u,fsw_u,fsg_u are the view factors used to compute the long wave
L00320  !   and the short wave radiation.
L00321        real fww_u(nz_um,nz_um,ndm,nurbmax)       !  from wall to wall
L00322        real fwg_u(nz_um,ndm,nurbmax)             !  from wall to ground
L00323        real fgw_u(nz_um,ndm,nurbmax)             !  from ground to wall
L00324        real fsw_u(nz_um,ndm,nurbmax)             !  from sky to wall
L00325        real fws_u(nz_um,ndm,nurbmax)             !  from sky to wall
L00326        real fsg_u(ndm,nurbmax)                   !  from sky to ground
L00327
L00328  !    Roughness parameters
L00329        real z0g_u(nurbmax)       ! The ground's roughness length
L00330        real z0r_u(nurbmax)       ! The roof's roughness length
L00331
L00332  !    Street parameters
L00333        integer nd_u(nurbmax)     ! Number of street direction for each urban class
L00334        real strd_u(ndm,nurbmax)  ! Street length (fix to greater value to the horizontal length of the cells)
L00335        real drst_u(ndm,nurbmax)  ! Street direction
L00336        real ws_u(ndm,nurbmax)    ! Street width
L00337        real bs_u(ndm,nurbmax)    ! Building width
L00338        real h_b(nz_um,nurbmax)   ! Bulding's heights
L00339        real d_b(nz_um,nurbmax)   ! Probability that a building has an height h_b
L00340        real ss_u(nz_um,nurbmax)! Probability that a building has an height equal to z
L00341        real pb_u(nz_um,nurbmax)! Probability that a building has an height greater or equal to z
L00342
L00343
L00344  !    Grid parameters
L00345        integer nz_u(nurbmax)     ! Number of layer in the urban grid
L00346
L00347        real z_u(nz_um)         ! Height of the urban grid levels
```

```
L00348  !FS
L00349        real cop_u(nurbmax)
L00350        real bldac_frc_u(nurbmax)
L00351        real cooled_frc_u(nurbmax)
L00352        real pwin_u(nurbmax)
L00353        real beta_u(nurbmax)
L00354        integer sw_cond_u(nurbmax)
L00355        real time_on_u(nurbmax)
L00356        real time_off_u(nurbmax)
L00357        real targtemp_u(nurbmax)
L00358        real gaptemp_u(nurbmax)
L00359        real targhum_u(nurbmax)
L00360        real gaphum_u(nurbmax)
L00361        real perflo_u(nurbmax)
L00362        real hsesf_u(nurbmax)
L00363        real hsequip(24)
L00364        real irho(24)
L00365  ! 1D array used for the input and output of the routine "urban"
L00366
L00367        real z1D(kms:kme)               ! vertical coordinates
L00368        real ua1D(kms:kme)                ! wind speed in the x directions
L00369        real va1D(kms:kme)                ! wind speed in the y directions
L00370        real pt1D(kms:kme)                ! potential temperature
L00371        real da1D(kms:kme)                ! air density
L00372        real pr1D(kms:kme)                ! air pressure
L00373        real pt01D(kms:kme)               ! reference potential temperature
L00374        real zr1D                      ! zenith angle
L00375        real deltar1D                  ! declination of the sun
L00376        real ah1D                      ! hour angle (it should come from the radiation routine)
L00377        real rs1D                      ! solar radiation
L00378        real rld1D                     ! downward flux of the longwave radiation
L00379        real swddir1D
L00380        real swddif1D                  ! short wave diffuse solar radiation _gl
L00381
L00382
L00383
L00384        real tw1D(2*ndm,nz_um,nwr_u,nbui_max) ! temperature in each layer of the wall
L00385        real tg1D(ndm,ng_u)                   ! temperature in each layer of the ground
L00386        real tr1D(ndm,nz_um,nwr_u)   ! temperature in each layer of the roof
L00387        real trv1D(ndm,nz_um,ngr_u)   ! temperature in each layer of the GREEN roof
L00388        real qr1D(ndm,nz_um,ngr_u)   ! humidity in each layer of the GREEN roof
L00389
L00390  !
L00391  !New variable for BEM
L00392  !
L00393        real tlev1D(nz_um,nbui_max)            ! temperature in each floor and in each different type of building
L00394        real qlev1D(nz_um,nbui_max)            ! specific humidity in each floor and in each different type of building
L00395        real twlev1D(2*ndm,nz_um,nbui_max)     ! temperature in each window in each floor in each different type of
↪  building
L00396        real tglev1D(ndm,ngb_u,nbui_max)       ! temperature in each layer of the ground below of a type of building
L00397        real tflev1D(ndm,nf_u,nz_um-1,nbui_max)! temperature in each layer of the floors in each building
L00398        real lflev1D(nz_um,nz_um)           ! latent heat flux due to the air conditioning systems
L00399        real sflev1D(nz_um,nz_um)           ! sensible heat flux due to the air conditioning systems
L00400        real lfvlev1D(nz_um,nz_um)          ! latent heat flux due to ventilation
L00401        real sfvlev1D(nz_um,nz_um)          ! sensible heat flux due to ventilation
L00402        real sfwin1D(2*ndm,nz_um,nbui_max)     ! sensible heat flux from windows
L00403        real consumlev1D(nz_um,nz_um)       ! consumption due to the air conditioning systems
L00404        real eppvlev1D(nz_um)               ! electricity production of PV panels
L00405        real tair1D(nz_um)
L00406        real tpvlev1D(ndm,nz_um)
L00407        real qv1D(kms:kme)                  ! specific humidity
L00408        real meso_urb                       ! constant to link meso and urban scales [m-2]
L00409        real meso_urb_ac
L00410        real roof_frac                       ! Surface fraction occupied by roof
L00411        real d_urb(nz_um)
L00412        real sf_ac
L00413        integer ibui,nbui
L00414        integer nlev(nz_um)
L00415
L00416  !
L00417  !End new variables
L00418  !
L00419
L00420        real sfw1D(2*ndm,nz_um,nbui_max)      ! sensible heat flux from walls
L00421        real sfg1D(ndm)              ! sensible heat flux from ground (road)
L00422        real sfr1D(ndm,nz_um)      ! sensible heat flux from roofs
L00423        real sfrpv1D(ndm,nz_um)
L00424
L00425        real tpv1D(nbui_max)
L00426        real sfr_indoor1D(nbui_max)
L00427        real sfrv1D(ndm,nz_um)      ! sensible heat flux from roofs
L00428        real lfrv1D(ndm,nz_um)      ! latent heat flux from roofs
L00429        real dg1D(ndm)              ! water depth from ground
L00430        real dgr1D(ndm,nz_um)      ! water depth from roofs
L00431        real lfg1D(ndm)              ! latent heat flux from ground (road)
L00432        real lfr1D(ndm,nz_um)      ! latent heat flux from roofs
L00433        real drain1D(ndm,nz_um)      ! sensible heat flux from roofs
L00434        real sf1D(kms:kme)              ! surface of the urban grid cells
L00435        real vl1D(kms:kme)              ! volume of the urban grid cells
L00436        real a_u1D(kms:kme)             ! Implicit component of the momentum sources or sinks in the X-direction
L00437        real a_v1D(kms:kme)             ! Implicit component of the momentum sources or sinks in the Y-direction
L00438        real a_t1D(kms:kme)             ! Implicit component of the heat sources or sinks
L00439        real a_e1D(kms:kme)             ! Implicit component of the TKE sources or sinks
L00440        real b_u1D(kms:kme)             ! Explicit component of the momentum sources or sinks in the X-direction
L00441        real b_v1D(kms:kme)             ! Explicit component of the momentum sources or sinks in the Y-direction
L00442        real b_t1D(kms:kme)             ! Explicit component of the heat sources or sinks
L00443        real b_ac1D(kms:kme)
L00444        real b_e1D(kms:kme)             ! Explicit component of the TKE sources or sinks
L00445        real b_q1D(kms:kme)             ! Explicit component of the Humidity sources or sinks
L00446        real dlg1D(kms:kme)             ! Height above ground (L_ground in formula (24) of the BLM paper).
L00447        real dl_u1D(kms:kme)            ! Length scale (lb in formula (22) ofthe BLM paper)
L00448        real gfr1D(ndm,nz_um)
L00449        real time_bep
L00450  ! arrays used to collapse indexes
L00451        integer ind_zwd(nbui_max,nz_um,nwr_u,ndm)
L00452        integer ind_gd(ng_u,ndm)
L00453        integer ind_zd(nbui_max,nz_um,ndm)
```

```
L00454        integer ind_zdf(nz_um,ndm)
L00455        integer ind_zrd(nz_um,nwr_u,ndm)
L00456        integer ind_grd(nz_um,ngr_u,ndm)
L00457  !
L00458        integer ind_bd(nbui_max,nz_um)
L00459        integer ind_wd(nbui_max,nz_um,ndm)
L00460        integer ind_gbd(nbui_max,ngb_u,ndm)
L00461        integer ind_fbd(nbui_max,nf_u,nz_um-1,ndm)
L00462
L00463        integer ix,iy,iz,iurb,id,iz_u,iw,ig,ir,ix1,iy1,k
L00464        integer it, nint
L00465        integer iii
L00466        logical first
L00467        character(len=80) :: text
L00468        data first/.true./
L00469        save first,time_bep
L00470
L00471        save alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,                       &
L00472             albg_u,albw_u,albr_u,emg_u,emw_u,emr_u,                       &
L00473             z0g_u,z0r_u, nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b,ss_u,pb_u,  &
L00474             nz_u,z_u,albwin_u,emwind_u,cop_u,pwin_u,beta_u,sw_cond_u,     &
L00475             bldac_frc_u,cooled_frc_u,                                     &
L00476             time_on_u,time_off_u,targtemp_u,gaptemp_u,targhum_u,gaphum_u, &
L00477             perflo_u,gr_frac_roof_u,                    &
L00478             pv_frac_roof_u,hsesf_u,hsequip,irho,gr_flag_u,gr_type_u
L00479
L00480  !---------------------------------------------------------------------
L00481  !    Calculation of the momentum, heat and turbulent kinetic fluxes
L00482  !     produced by buildings
L00483  !
L00484  ! References:
L00485  ! Martilli, A., Clappier, A., Rotach, M.W.:2002, 'AN URBAN SURFACE EXCHANGE
L00486  ! PARAMETERISATION FOR MESOSCALE MODELS', Boundary-Layer Meteorolgy 104:
L00487  ! 261-304
L00488  !
L00489  ! F. Salamanca and A. Martilli, 2009: 'A new Building Energy Model coupled
L00490  ! with an Urban Canopy Parameterization for urban climate simulations - part II.
L00491  ! Validation with one dimension off-line simulations'. Theor Appl Climatol
L00492  ! DOI 10.1007/s00704-009-0143-8
L00493  !---------------------------------------------------------------------
L00494  !
L00495  !prepare the arrays to collapse indexes
L00496
L00497
L00498  !
L00499       if(urban_map_zwd.lt.nbui_max*nz_um*ndm*max(nwr_u,ng_u))then
L00500          write(*,*)'urban_map_zwd too small, please increase to at least ', nbui_max*nz_um*ndm*max(nwr_u,ng_u)
L00501          stop
L00502        endif
L00503  !
L00504  !New conditions for BEM
L00505  !
L00506        if(urban_map_bd.lt.nbui_max*nz_um)then !limit for indoor temperature and indoor humidity
L00507          write(*,*)'urban_map_bd too small, please increase to at least ', nbui_max*nz_um
L00508          stop
L00509        endif
L00510
L00511        if(urban_map_wd.lt.nbui_max*nz_um*ndm)then !limit for window temperature
L00512          write(*,*)'urban_map_wd too small, please increase to at least ', nbui_max*nz_um*ndm
L00513          stop
L00514        endif
L00515
L00516        if(urban_map_gbd.lt.nbui_max*ndm*ngb_u)then !limit for ground temperature below a building
L00517          write(*,*)'urban_map_gbd too small, please increase to at least ', nbui_max*ndm*ngb_u
L00518          stop
L00519        endif
L00520
L00521        if(urban_map_fbd.lt.(nz_um-1)*nbui_max*ndm*nf_u)then !limit for floor temperature
L00522          write(*,*)'urban_map_fbd too small, please increase to at least ', nbui_max*ndm*nf_u*(nz_um-1)
L00523          stop
L00524        endif
L00525
L00526        if (ndm.ne.2)then
L00527           write(*,*) 'number of directions is not correct',ndm
L00528           stop
L00529        endif
L00530
L00531  !End of new conditions
L00532  !
L00533  !
L00534  !Initialize collapse indexes
L00535  !
L00536        ind_zwd=0
L00537        ind_gd=0
L00538        ind_zd=0
L00539        ind_zdf=0
L00540        ind_zrd=0
L00541        ind_grd=0
L00542        ind_bd=0
L00543        ind_wd=0
L00544        ind_gbd=0
L00545        ind_fbd=0
L00546  !
L00547  !End initialization indexes
L00548  !
L00549
L00550        iii=0
L00551        do ibui=1,nbui_max
L00552        do iz_u=1,nz_um
L00553        do iw=1,nwr_u
L00554        do id=1,ndm
L00555         iii=iii+1
L00556         ind_zwd(ibui,iz_u,iw,id)=iii
L00557        enddo
L00558        enddo
L00559        enddo
L00560        enddo
L00561
L00562        iii=0
```

```
L00563         do ig=1,ng_u
L00564         do id=1,ndm
L00565          iii=iii+1
L00566          ind_gd(ig,id)=iii
L00567         enddo
L00568         enddo
L00569
L00570         iii=0
L00571         do ibui=1,nbui_max
L00572         do iz_u=1,nz_um
L00573         do id=1,ndm
L00574          iii=iii+1
L00575          ind_zd(ibui,iz_u,id)=iii
L00576         enddo
L00577         enddo
L00578         enddo
L00579
L00580         iii=0
L00581         do iz_u=1,nz_um
L00582         do iw=1,nwr_u
L00583         do id=1,ndm
L00584          iii=iii+1
L00585          ind_zrd(iz_u,iw,id)=iii
L00586         enddo
L00587         enddo
L00588         enddo
L00589
L00590        iii=0
L00591         do iz_u=1,nz_um
L00592         do iw=1,ngr_u
L00593         do id=1,ndm
L00594          iii=iii+1
L00595          ind_grd(iz_u,iw,id)=iii
L00596         enddo
L00597         enddo
L00598         enddo
L00599
L00600  !
L00601  !New indexes for BEM
L00602
L00603         iii=0
L00604         do iz_u=1,nz_um
L00605         do id=1,ndm
L00606            iii=iii+1
L00607            ind_zdf(iz_u,id)=iii
L00608         enddo ! id
L00609         enddo ! iz_u
L00610
L00611         iii=0
L00612         do ibui=1,nbui_max  !Type of building
L00613         do iz_u=1,nz_um     !vertical levels
L00614            iii=iii+1
L00615            ind_bd(ibui,iz_u)=iii
L00616         enddo !iz_u
L00617         enddo !ibui
L00618
L00619
L00620
L00621         iii=0
L00622         do ibui=1,nbui_max !type of building
L00623         do iz_u=1,nz_um !vertical levels
L00624         do id=1,ndm !direction
L00625            iii=iii+1
L00626            ind_wd(ibui,iz_u,id)=iii
L00627         enddo !id
L00628         enddo !iz_u
L00629         enddo !ibui
L00630
L00631         iii=0
L00632         do ibui=1,nbui_max!type of building
L00633         do iw=1,ngb_u !layers in the wall (ground below a building)
L00634         do id=1,ndm !direction
L00635            iii=iii+1
L00636            ind_gbd(ibui,iw,id)=iii
L00637         enddo !id
L00638         enddo !iw
L00639         enddo !ibui
L00640
L00641         iii=0
L00642         do ibui=1,nbui_max !type of building
L00643         do iw=1,nf_u !layers in the wall (floor)
L00644         do iz_u=1,nz_um-1 !vertical levels
L00645         do id=1,ndm  !direction
L00646            iii=iii+1
L00647            ind_fbd(ibui,iw,iz_u,id)=iii
L00648         enddo !id
L00649         enddo !iz_u
L00650         enddo !iw
L00651         enddo !ibui
L00652
L00653
L00654         !End of new indexes
L00655
L00656         if (num_urban_hi.ge.nz_um)then
L00657             write(*,*)'nz_um too small, please increase to at least ', num_urban_hi+1
L00658             stop
L00659         endif
L00660
L00661         do ix=its,ite
L00662         do iy=jts,jte
L00663         do iz_u=1,nz_um
L00664             hi_urb(ix,iz_u,iy)=0.
L00665         enddo
L00666         enddo
L00667         enddo
L00668
L00669         do ix=its,ite
L00670         do iy=jts,jte
L00671          z(ix,kts,iy)=0.
L00672          do iz=kts+1,kte+1
```

```
L00673          z(ix,iz,iy)=z(ix,iz-1,iy)+dz8w(ix,iz-1,iy)
L00674         enddo
L00675         iii=0
L00676         do iz_u=1,num_urban_hi
L00677            hi_urb(ix,iz_u,iy)= hi_urb2d(ix,iz_u,iy)
L00678            if (hi_urb(ix,iz_u,iy)/=0.) then
L00679               iii=iii+1
L00680            endif
L00681         enddo !iz_u
L00682         if (iii.gt.nbui_max) then
L00683            write(*,*) 'nbui_max too small, please increase to at least ',iii
L00684            stop
L00685         endif
L00686       enddo
L00687       enddo
L00688
L00689
L00690       if (first) then                          ! True only on first call
L00691
L00692          call init_para(alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,&
L00693                twini_u,trini_u,tgini_u,albg_u,albw_u,albr_u,albwin_u,emg_u,emw_u,&
L00694                emr_u,emwind_u,z0g_u,z0r_u,nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b,  &
L00695                cop_u,pwin_u,beta_u,sw_cond_u,time_on_u,time_off_u,targtemp_u,    &
L00696                bldac_frc_u,cooled_frc_u,                                        &
L00697                gaptemp_u,targhum_u,gaphum_u,perflo_u,                           &
L00698                gr_frac_roof_u,pv_frac_roof_u,                  &
L00699                hsesf_u,hsequip,irho,gr_flag_u,gr_type_u)
L00700
L00701  !Initialisation of the urban parameters and calculation of the view factor
L00702         call icBEP(nd_u,h_b,d_b,ss_u,pb_u,nz_u,z_u)
L00703
L00704       first=.false.
L00705
L00706       endif ! first
L00707
L00708  do ix=its,ite
L00709       do iy=jts,jte
L00710         if (FRC_URB2D(ix,iy).gt.0.) then    ! Calling BEP only for existing urban classes.
L00711
L00712          iurb=UTYPE_URB2D(ix,iy)
L00713
L00714          hi_urb1D=0.
L00715          do iz_u=1,nz_um
L00716             hi_urb1D(iz_u)=hi_urb(ix,iz_u,iy)
L00717
L00718          enddo
L00719
L00720          call icBEPHI_XY(iurb,hb_u,hi_urb1D,ss_urb,pb_urb,    &
L00721                          nz_urb(iurb),z_u)
L00722
L00723          call param(iurb,nz_u(iurb),nz_urb(iurb),nzurban(iurb),      &
L00724                     nd_u(iurb),csg_u,csg,alag_u,alag,csr_u,csr,      &
L00725                     alar_u,alar,csw_u,csw,alaw_u,alaw,               &
L00726                     ws_u,ws_urb,ws,bs_u,bs_urb,bs,z0g_u,z0r_u,z0,    &
L00727                     strd_u,strd,drst_u,drst,ss_u,ss_urb,ss,pb_u,     &
L00728                     pb_urb,pb,dzw,dzr,dzf,csf,alaf,dzgb,csgb,alagb,  &
L00729                     lp_urb2d(ix,iy),lb_urb2d(ix,iy),                 &
L00730                     hgt_urb2d(ix,iy),FRC_URB2D(ix,iy))
L00731
L00732  !
L00733  !We compute the view factors in the icBEP_XY routine
L00734  !
L00735
L00736          call icBEP_XY(iurb,fww_u,fwg_u,fgw_u,fsw_u,fws_u,fsg_u,   &
L00737                        nd_u(iurb),strd,ws,nzurban(iurb),z_u)
L00738
L00739          ibui=0
L00740          nlev=0
L00741          nbui=0
L00742          d_urb=0.
L00743          do iz=1,nz_um
L00744          if(ss_urb(iz,iurb).gt.0) then
L00745            ibui=ibui+1
L00746            nlev(ibui)=iz-1
L00747            d_urb(ibui)=ss_urb(iz,iurb)
L00748            nbui=ibui
L00749       endif
L00750          end do  !iz
L00751
L00752          if (nbui.gt.nbui_max) then
L00753             write (*,*) 'nbui_max must be increased to',nbui
L00754             stop
L00755          endif
L00756
L00757
L00758
L00759  do iz= kts,kte
L00760            ua1D(iz)=u_phy(ix,iz,iy)
L00761            va1D(iz)=v_phy(ix,iz,iy)
L00762       pt1D(iz)=th_phy(ix,iz,iy)
L00763       da1D(iz)=rho(ix,iz,iy)
L00764       pr1D(iz)=p_phy(ix,iz,iy)
L00765       pt01D(iz)=300.
L00766       z1D(iz)=z(ix,iz,iy)
L00767           qv1D(iz)=qv_phy(ix,iz,iy)
L00768           a_u1D(iz)=0.
L00769           a_v1D(iz)=0.
L00770           a_t1D(iz)=0.
L00771           a_e1D(iz)=0.
L00772           b_u1D(iz)=0.
L00773           b_v1D(iz)=0.
L00774           b_t1D(iz)=0.
L00775           b_ac1D(iz)=0.
L00776           b_e1D(iz)=0.
L00777          enddo
L00778       z1D(kte+1)=z(ix,kte+1,iy)
L00779
L00780
L00781
```

```
L00782           do id=1,ndm
L00783           do iz_u=1,nz_um
L00784           do iw=1,nwr_u
L00785           do ibui=1,nbui_max
L00786            tw1D(2*id-1,iz_u,iw,ibui)=tw1_urb4d(ix,ind_zwd(ibui,iz_u,iw,id),iy)
L00787            tw1D(2*id,iz_u,iw,ibui)=tw2_urb4d(ix,ind_zwd(ibui,iz_u,iw,id),iy)
L00788           enddo
L00789           enddo
L00790           enddo
L00791           enddo
L00792
L00793           do id=1,ndm
L00794             do ig=1,ng_u
L00795              tg1D(id,ig)=tgb_urb4d(ix,ind_gd(ig,id),iy)
L00796             enddo
L00797
L00798             do iz_u=1,nz_um
L00799               do ir=1,nwr_u
L00800                 tr1D(id,iz_u,ir)=trb_urb4d(ix,ind_zrd(iz_u,ir,id),iy)
L00801               enddo
L00802               do ir=1,ngr_u
L00803                 if(gr_flag_u.eq.1)then
L00804                   trv1D(id,iz_u,ir)=trv_urb4d(ix,ind_grd(iz_u,ir,id),iy)
L00805                   qr1D(id,iz_u,ir)=qr_urb4d(ix,ind_grd(iz_u,ir,id),iy)
L00806                 else
L00807                   trv1D(id,iz_u,ir)=0.
L00808                   qr1D(id,iz_u,ir)=0.
L00809                 endif
L00810               enddo
L00811             enddo
L00812          enddo
L00813
L00814
L00815
L00816  !Initialize variables for BEM
L00817
L00818           tlev1D=0.  !Indoor temperature
L00819           qlev1D=0.  !Indoor humidity
L00820
L00821           twlev1D=0. !Window temperature
L00822           tglev1D=0. !Ground temperature
L00823           tflev1D=0. !Floor temperature
L00824
L00825           sflev1D=0.    !Sensible heat flux from the a.c.
L00826           lflev1D=0.    !Latent heat flux from the a.c.
L00827           consumlev1D=0.!consumption of the a.c.
L00828           eppvlev1D=0.  !electricity production of PV panels
L00829           tpvlev1D=0.
L00830           sfvlev1D=0.   !Sensible heat flux from natural ventilation
L00831           lfvlev1D=0.   !Latent heat flux from natural ventilation
L00832           sfwin1D=0.    !Sensible heat flux from windows
L00833           sfw1D=0.      !Sensible heat flux from walls
L00834
L00835           do iz_u=1,nz_um    !vertical levels
L00836           do ibui=1,nbui_max !Type of building
L00837              tlev1D(iz_u,ibui)= tlev_urb3d(ix,ind_bd(ibui,iz_u),iy)
L00838              qlev1D(iz_u,ibui)= qlev_urb3d(ix,ind_bd(ibui,iz_u),iy)
L00839           enddo !ibui
L00840           enddo !iz_u
L00841
L00842
L00843
L00844           do id=1,ndm  !direction
L00845              do iz_u=1,nz_um !vertical levels
L00846                 do ibui=1,nbui_max !type of building
L00847                    twlev1D(2*id-1,iz_u,ibui)=tw1lev_urb3d(ix,ind_wd(ibui,iz_u,id),iy)
L00848                    twlev1D(2*id,iz_u,ibui)=tw2lev_urb3d(ix,ind_wd(ibui,iz_u,id),iy)
L00849                    sfwin1D(2*id-1,iz_u,ibui)=sfwin1_urb3d(ix,ind_wd(ibui,iz_u,id),iy)
L00850                    sfwin1D(2*id,iz_u,ibui)=sfwin2_urb3d(ix,ind_wd(ibui,iz_u,id),iy)
L00851                 enddo !ibui
L00852              enddo !iz_u
L00853           enddo !id
L00854
L00855           do id=1,ndm !direction
L00856              do iw=1,ngb_u !layer in the wall
L00857                 do ibui=1,nbui_max !type of building
L00858                    tglev1D(id,iw,ibui)=tglev_urb3d(ix,ind_gbd(ibui,iw,id),iy)
L00859                 enddo !ibui
L00860              enddo !iw
L00861           enddo !id
L00862
L00863           do id=1,ndm !direction
L00864              do iw=1,nf_u !layer in the walls
L00865                 do iz_u=1,nz_um-1 !verticals levels
L00866                    do ibui=1,nbui_max !type of building
L00867                       tflev1D(id,iw,iz_u,ibui)=tflev_urb3d(ix,ind_fbd(ibui,iw,iz_u,id),iy)
L00868
L00869                    enddo !ibui
L00870                 enddo ! iz_u
L00871              enddo !iw
L00872           enddo !id
L00873
L00874  !
L00875  !End initialization for BEM
L00876  !
L00877
L00878          do id=1,ndm
L00879            do iz=1,nz_um
L00880              do ibui=1,nbui_max !type of building
L00881              !!  sfw1D(2*id-1,iz)=sfw1(ix,iy,ind_zd(iz,id))
L00882              !!  sfw1D(2*id,iz)=sfw2(ix,iy,ind_zd(iz,id))
L00883                  sfw1D(2*id-1,iz,ibui)=sfw1_urb3d(ix,ind_zd(ibui,iz,id),iy)
L00884                  sfw1D(2*id,iz,ibui)=sfw2_urb3d(ix,ind_zd(ibui,iz,id),iy)
L00885              enddo
L00886            enddo
L00887          enddo
L00888
L00889          do id=1,ndm
L00890            sfg1D(id)=sfg_urb3d(ix,id,iy)
```

```
L00891            lfg1D(id)=lfg_urb3d(ix,id,iy)
L00892            dg1D(id)=dg_urb3d(ix,id,iy)
L00893
L00894          enddo
L00895
L00896        do id=1,ndm
L00897        do iz=1,nz_um
L00898         tpvlev1D(id,iz)=t_pv_urb3d(ix,ind_zdf(iz,id),iy)
L00899         sfr1D(id,iz)=sfr_urb3d(ix,ind_zdf(iz,id),iy)
L00900            lfr1D(id,iz)=lfr_urb3d(ix,ind_zdf(iz,id),iy)
L00901            dgr1D(id,iz)=dgr_urb3d(ix,ind_zdf(iz,id),iy)
L00902            if(gr_flag_u.eq.1)then
L00903            sfrv1D(id,iz)=sfrv_urb3d(ix,ind_zdf(iz,id),iy)
L00904            lfrv1D(id,iz)=lfrv_urb3d(ix,ind_zdf(iz,id),iy)
L00905            drain1D(id,iz)=drain_urb4d(ix,ind_zdf(iz,id),iy)
L00906            else
L00907            sfrv1D(id,iz)=0.
L00908            lfrv1D(id,iz)=0.
L00909            drain1D(id,iz)=0.
L00910            endif
L00911        enddo
L00912        enddo
L00913
L00914
L00915
L00916           rs1D=swdown(ix,iy)
L00917           rld1D=glw(ix,iy)
L00918           swddir1D=swddir(ix,iy)          !_gl
L00919           swddif1D=swddif(ix,iy)          !_gl
L00920           zr1D=acos(COSZ_URB2D(ix,iy))
L00921           deltar1D=DECLIN_URB
L00922           ah1D=OMG_URB2D(ix,iy)
L00923
L00924
L00925           call BEP1D(itimestep,ix,iy,iurb,kms,kme,kts,kte,z1D,dt,ua1D,va1D,pt1D,da1D,pr1D,pt01D,  &
L00926                     zr1D,deltar1D,ah1D,rs1D,rld1D,alagb,                &
L00927                     alag,alaw,alar,alaf,csgb,csg,csw,csr,csf,           &
L00928                     dzr,dzf,dzw,dzgb,xlat(ix,iy),swddir1D,swddif1D, &
L00929                     albg_u(iurb),albw_u(iurb),albr_u(iurb),             &
L00930                     albwin_u(iurb),emg_u(iurb),emw_u(iurb),             &
L00931                     emr_u(iurb),emwind_u(iurb),fww_u,fwg_u,             &
L00932                     fgw_u,fsw_u,fws_u,fsg_u,z0,                         &
L00933                     nd_u(iurb),strd,drst,ws,bs_urb,bs,ss,pb,            &
L00934                     nzurban(iurb),z_u,cop_u,pwin_u,beta_u,              &
L00935                     sw_cond_u,time_on_u,time_off_u,targtemp_u,          &
L00936                     gaptemp_u,targhum_u,gaphum_u,perflo_u,              &
L00937                     gr_frac_roof_u(iurb),pv_frac_roof_u(iurb),  &
L00938                     hsesf_u,hsequip,irho,gr_flag_u,gr_type_u,           &
L00939                     tw1D,tg1D,tr1D,trv1D,sfw1D,sfg1D,sfr1D,    &
L00940                     sfrv1D,lfrv1D,    &
L00941                     dgr1D,dg1D,lfr1D,lfg1D,                        &
L00942                     drain1D,rainbl(ix,iy),qr1D,                    &
L00943                     a_u1D,a_v1D,a_t1D,a_e1D,                            &
L00944                     b_u1D,b_v1D,b_t1D,b_ac1D,b_e1D,b_q1D,               &
L00945                     dlg1D,dl_u1D,sf1D,vl1D,rl_up(ix,iy),                &
L00946                     rs_abs(ix,iy),emiss(ix,iy),grdflx_urb(ix,iy),    &
L00947                     qv1D,tlev1D,qlev1D,sflev1D,lflev1D,consumlev1D,  &
L00948                     eppvlev1D,tpvlev1D,sfvlev1D,lfvlev1D,twlev1D,tglev1D,tflev1D,sfwin1D,tair1D,sfr_indoor1D,sfrpv1D,
↪  gfr1D)
L00949
L00950            do ibui=1,nbui_max !type of building
L00951          do iz=1,nz_um   !vertical levels
L00952                do id=1,ndm ! direction
L00953               sfw1_urb3d(ix,ind_zd(ibui,iz,id),iy)=sfw1D(2*id-1,iz,ibui)
L00954               sfw2_urb3d(ix,ind_zd(ibui,iz,id),iy)=sfw1D(2*id,iz,ibui)
L00955             enddo
L00956          enddo
L00957           enddo
L00958
L00959        do id=1,ndm
L00960         sfg_urb3d(ix,id,iy)=sfg1D(id)
L00961            lfg_urb3d(ix,id,iy)=lfg1D(id)
L00962            dg_urb3d(ix,id,iy)=dg1D(id)
L00963        enddo
L00964
L00965        do id=1,ndm
L00966        do iz=1,nz_um
L00967            t_pv_urb3d(ix,ind_zdf(iz,id),iy)=tpvlev1D(id,iz)
L00968         sfr_urb3d(ix,ind_zdf(iz,id),iy)=sfr1D(id,iz)
L00969            dgr_urb3d(ix,ind_zdf(iz,id),iy)=dgr1D(id,iz)
L00970            lfr_urb3d(ix,ind_zdf(iz,id),iy)=lfr1D(id,iz)
L00971            if(gr_flag_u.eq.1)then
L00972            sfrv_urb3d(ix,ind_zdf(iz,id),iy)=sfrv1D(id,iz)
L00973            lfrv_urb3d(ix,ind_zdf(iz,id),iy)=lfrv1D(id,iz)
L00974            drain_urb4d(ix,ind_zdf(iz,id),iy)=drain1D(id,iz)
L00975            endif
L00976        enddo
L00977        enddo
L00978
L00979           do ibui=1,nbui_max
L00980            do iz_u=1,nz_um
L00981            do iw=1,nwr_u
L00982            do id=1,ndm
L00983             tw1_urb4d(ix,ind_zwd(ibui,iz_u,iw,id),iy)=tw1D(2*id-1,iz_u,iw,ibui)
L00984             tw2_urb4d(ix,ind_zwd(ibui,iz_u,iw,id),iy)=tw1D(2*id,iz_u,iw,ibui)
L00985            enddo
L00986            enddo
L00987            enddo
L00988            enddo
L00989
L00990
L00991             do id=1,ndm
L00992               do ig=1,ng_u
L00993
L00994                  tgb_urb4d(ix,ind_gd(ig,id),iy)=tg1D(id,ig)
L00995               enddo
L00996               do iz_u=1,nz_um
L00997                  do ir=1,nwr_u
```

```
L00998                    trb_urb4d(ix,ind_zrd(iz_u,ir,id),iy)=tr1D(id,iz_u,ir)
L00999                 enddo
L01000                  if(gr_flag_u.eq.1)then
L01001                 do ir=1,ngr_u
L01002                    trv_urb4d(ix,ind_grd(iz_u,ir,id),iy)=trv1D(id,iz_u,ir)
L01003                    qr_urb4d(ix,ind_grd(iz_u,ir,id),iy)=qr1D(id,iz_u,ir)
L01004                 enddo
L01005                  endif
L01006              enddo
L01007             enddo
L01008  !
L01009
L01010  !
L01011  !Outputs of BEM
L01012  !
L01013
L01014           do ibui=1,nbui_max !type of building
L01015           do iz_u=1,nz_um !vertical levels
L01016              tlev_urb3d(ix,ind_bd(ibui,iz_u),iy)=tlev1D(iz_u,ibui)
L01017              qlev_urb3d(ix,ind_bd(ibui,iz_u),iy)=qlev1D(iz_u,ibui)
L01018           enddo !iz_u
L01019           enddo !ibui
L01020
L01021           do ibui=1,nbui_max !type of building
L01022           do iz_u=1,nz_um !vertical levels
L01023              do id=1,ndm !direction
L01024                 tw1lev_urb3d(ix,ind_wd(ibui,iz_u,id),iy)=twlev1D(2*id-1,iz_u,ibui)
L01025                 tw2lev_urb3d(ix,ind_wd(ibui,iz_u,id),iy)=twlev1D(2*id,iz_u,ibui)
L01026                 sfwin1_urb3d(ix,ind_wd(ibui,iz_u,id),iy)=sfwin1D(2*id-1,iz_u,ibui)
L01027                 sfwin2_urb3d(ix,ind_wd(ibui,iz_u,id),iy)=sfwin1D(2*id,iz_u,ibui)
L01028              enddo !id
L01029           enddo !iz_u
L01030           enddo !ibui
L01031
L01032           do ibui=1,nbui_max  !type of building
L01033              do iw=1,ngb_u !layers in the walls
L01034                 do id=1,ndm !direction
L01035                    tglev_urb3d(ix,ind_gbd(ibui,iw,id),iy)=tglev1D(id,iw,ibui)
L01036                 enddo !id
L01037              enddo !iw
L01038           enddo !ibui
L01039
L01040          do ibui=1,nbui_max !type of building
L01041          do iw=1,nf_u !layer in the walls
L01042                 do iz_u=1,nz_um-1 !verticals levels
L01043                   do  id=1,ndm
L01044                      tflev_urb3d(ix,ind_fbd(ibui,iw,iz_u,id),iy)=tflev1D(id,iw,iz_u,ibui)
L01045                   enddo !ibui
L01046                 enddo ! iz_u
L01047              enddo !iw
L01048           enddo !id
L01049
L01050
L01051
L01052           sf_ac_urb3d(ix,iy)=0.
L01053           lf_ac_urb3d(ix,iy)=0.
L01054           cm_ac_urb3d(ix,iy)=0.
L01055           ep_pv_urb3d(ix,iy)=0.
L01056           sfvent_urb3d(ix,iy)=0.
L01057           lfvent_urb3d(ix,iy)=0.
L01058           draingr_urb3d(ix,iy)=0.
L01059           qgr_urb3d(ix,iy)=0.
L01060           tgr_urb3d(ix,iy)=0.
L01061           meso_urb=(1./4.)*FRC_URB2D(ix,iy)/((bs_urb(1,iurb)+ws_urb(1,iurb))*bs_urb(2,iurb))+ &
L01062                  (1./4.)*FRC_URB2D(ix,iy)/((bs_urb(2,iurb)+ws_urb(2,iurb))*bs_urb(1,iurb))
L01063            meso_urb_ac=meso_urb*bldac_frc_u(iurb)*cooled_frc_u(iurb)
L01064            roof_frac=FRC_URB2D(ix,iy)*bs_urb(1,iurb)/(bs_urb(1,iurb)+ws_urb(1,iurb))
L01065           ibui=0
L01066           nlev=0
L01067           nbui=0
L01068           d_urb=0.
L01069           do iz=1,nz_um
L01070           if(ss_urb(iz,iurb).gt.0) then
L01071             ibui=ibui+1
L01072             nlev(ibui)=iz-1
L01073             d_urb(ibui)=ss_urb(iz,iurb)
L01074             nbui=ibui
L01075       endif
L01076           end do  !iz
L01077
L01078
L01079
L01080
L01081          do ibui=1,nbui       !type of building
L01082              ep_pv_urb3d(ix,iy)=ep_pv_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*eppvlev1D(ibui)
L01083           do iz_u=1,nlev(ibui) !vertical levels
L01084                 sf_ac_urb3d(ix,iy)=sf_ac_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*sflev1D(iz_u,ibui)
L01085                 lf_ac_urb3d(ix,iy)=lf_ac_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*lflev1D(iz_u,ibui)
L01086                 cm_ac_urb3d(ix,iy)=cm_ac_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*consumlev1D(iz_u,ibui)
L01087           !if(consumlev1D(iz_u,ibui).gt.0.)then
L01088           !print*,'IX',ix,'IY',iy,'IZ_U',iz_u,'IBUI',ibui,'CONSUM',consumlev1D(iz_u,ibui),'D_URB',d_urb(ibui),'MESO_URB',
↪  meso_urb_ac
L01089
L01090           !endif
L01091                 sfvent_urb3d(ix,iy)=sfvent_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*sfvlev1D(iz_u,ibui)
L01092                 lfvent_urb3d(ix,iy)=lfvent_urb3d(ix,iy)+meso_urb_ac*d_urb(ibui)*lfvlev1D(iz_u,ibui)
L01093           enddo !iz_u
L01094           enddo !ibui
L01095
L01096
L01097
L01098          if(gr_flag_u.eq.1)then
L01099          do id=1,ndm
L01100          do iz=2,nz_um-1
L01101           draingr_urb3d(ix,iy)=draingr_urb3d(ix,iy)+d_urb(iz-1)*roof_frac*drain1D(id,iz)*1000
L01102          do ig=1,ngr_u
L01103           qgr_urb3d(ix,iy)=qgr_urb3d(ix,iy)+qr1D(id,iz,ig)/ndm/(nz_um-2)/ngr_u
L01104           tgr_urb3d(ix,iy)=tgr_urb3d(ix,iy)+trv1D(id,iz,ig)/ndm/(nz_um-2)/ngr_u
```

```
L01105
L01106          enddo
L01107          enddo
L01108          enddo
L01109          endif
L01110  !End outputs of bem
L01111  !
L01112
L01113           sf_ac=0.
L01114           sf(ix,kts:kte,iy)=0.
L01115           vl(ix,kts:kte,iy)=0.
L01116           a_u(ix,kts:kte,iy)=0.
L01117           a_v(ix,kts:kte,iy)=0.
L01118           a_t(ix,kts:kte,iy)=0.
L01119           a_e(ix,kts:kte,iy)=0.
L01120           b_u(ix,kts:kte,iy)=0.
L01121           b_v(ix,kts:kte,iy)=0.
L01122           b_t(ix,kts:kte,iy)=0.
L01123           b_e(ix,kts:kte,iy)=0.
L01124           b_q(ix,kts:kte,iy)=0.
L01125           dlg(ix,kts:kte,iy)=0.
L01126           dl_u(ix,kts:kte,iy)=0.
L01127
L01128           do iz= kts,kte
L01129             sf(ix,iz,iy)=sf1D(iz)
L01130             vl(ix,iz,iy)=vl1D(iz)
L01131             a_u(ix,iz,iy)=a_u1D(iz)
L01132             a_v(ix,iz,iy)=a_v1D(iz)
L01133             a_t(ix,iz,iy)=a_t1D(iz)
L01134             a_e(ix,iz,iy)=a_e1D(iz)
L01135             b_u(ix,iz,iy)=b_u1D(iz)
L01136             b_v(ix,iz,iy)=b_v1D(iz)
L01137             b_t(ix,iz,iy)=b_t1D(iz)
L01138             sf_ac=sf_ac+b_ac1D(iz)*da1D(iz)*cp_u*dz8w(ix,iz,iy)*vl1D(iz)*FRC_URB2D(ix,iy)
L01139             b_e(ix,iz,iy)=b_e1D(iz)
L01140             b_q(ix,iz,iy)=b_q1D(iz)
L01141             dlg(ix,iz,iy)=dlg1D(iz)
L01142             dl_u(ix,iz,iy)=dl_u1D(iz)
L01143           enddo
L01144           sf(ix,kte+1,iy)=sf1D(kte+1)
L01145
L01146            endif ! FRC_URB2D
L01147
L01148
L01149         enddo  ! iy
L01150         enddo  ! ix
L01151
L01152
L01153           time_bep=time_bep+dt
L01154
L01155  !       print*, 'ss_urb', ss_urb
L01156  !       print*, 'pb_urb', pb_urb
L01157  !       print*, 'nz_urb', nz_urb
L01158  !       print*, 'd_urb',  d_urb
L01159
L01160
L01161         return
L01162         end subroutine BEP_BEM
```

讲解：结构分五段。① **索引折叠表**（L499–654）：ind_zwd/ind_gd/ind_zd/ind_zrd/ind_grd/ind_bd/ind_wd/ind_gbd/ind_fbd 十张整数表把「建筑型 × 层 × 墙层 × 方向」的物理四维折叠进 urban_map_zwd 等一维 Registry 维度——这是 WRF 里**高维状态量过 Registry 的标准先例**（论文 2 把 BEM 标量改逐格场时用的 vbs/br 列表变量机制与此同源；ind_* 在每次调用时重建，内容固定，属可移出热循环的冗余）。② **首调用初始化**（L690–706）：first 旗标下 init_para（读 urban_param.tbl，14b）+ icBEP（类级高度谱初始化，14b）。③ **逐点循环**（L708 起）：仅 FRC_URB2D>0 的点进入；icBEPHI_XY 用**该格点自己的建筑高度直方图**（hi_urb2d）重算 ss/pb——BEF（建筑高度分布）是逐格的，不是类级的；param 用 lp/lb/hgt 三个形态参数反解街道几何；icBEP_XY 按点重算视野因子。④ **折叠→BEP1D→ 写回**：把 tw1_urb4d/trb_urb4d/tlev_urb3d 等状态按 ind_* 折进 1D 数组调 BEP1D，返回后原路写回。⑤**BEM 输出聚合**（L1052–1109）：meso_urb=¼ • FRC/(bs+ws) • bs 的几何换算把单建筑型 AC 冷凝热/耗电/PV 发电聚合到格点（meso_urb_ac 乘 bldac_frc • cooled_frc 两个开机率参数）；绿屋顶排水 draingr 与屋顶含水量/温度均值也在此聚合。最后 sf/vl/a_/b_ 十三个源项数组写回给 pbl_driver。

五、柱求解器 BEP1D（全文）

```
!file: phys/module_sf_bep_bem.F    L1167-L1870    (subroutine BEP1D)
L01167         subroutine BEP1D(itimestep,ix,iy,iurb,kms,kme,kts,kte,z,dt,ua,va,pt,da,pr,pt0,   &
L01168                          zr,deltar,ah,rs,rld,alagb,                          &
L01169                          alag,alaw,alar,alaf,csgb,csg,csw,csr,csf,           &
L01170                          dzr,dzf,dzw,dzgb,xlat,swddir,swddif,                &
L01171                          albg,albw,albr,albwin,emg,emw,emr,                  &
L01172                          emwind,fww,fwg,fgw,fsw,fws,fsg,z0,                  &
L01173                          ndu,strd,drst,ws,bs_u,bs,ss,pb,                     &
L01174                          nzu,z_u,cop_u,pwin_u,beta_u,sw_cond_u,              &
L01175                          time_on_u,time_off_u,targtemp_u,                    &
L01176                          gaptemp_u,targhum_u,gaphum_u,perflo_u,              &
L01177                          gr_frac_roof,pv_frac_roof,          &
L01178                          hsesf_u,hsequip,irho,gr_flag,gr_type,                       &
L01179                          tw,tg,tr,trv,sfw,sfg,sfr,             &
L01180                          sfrv,lfrv,dgr,dg,lfr,lfg,drain,rainbl,qr,       &
L01181                          a_u,a_v,a_t,a_e,                                    &
```

```
L01182                        b_u,b_v,b_t,b_ac,b_e,b_q,                         &
L01183                        dlg,dl_u,sf,vl,rl_up,rs_abs,emiss,grdflx_urb,     &
L01184                        qv,tlev,qlev,sflev,lflev,consumlev,               &
L01185                        eppvlev,tpvlev,sfvlev,lfvlev,twlev,tglev,tflev,sfwin,tmp_u,sfr_indoor,sfrpv,gfr)
L01186          ! print*,'SFR_AFT',sfr(id,iz)
L01187
L01188
L01189
L01190        implicit none
L01191
L01192  ! ---------------------------------------------------------------------
L01193  ! INPUT:
L01194  ! ---------------------------------------------------------------------
L01195
L01196  ! Data relative to the "mesoscale grid"
L01197
L01198  !!    integer nz                  ! Number of vertical levels
L01199        integer kms,kme,kts,kte,ix,iy,itimestep
L01200        real z(kms:kme)               ! Altitude above the ground of the cell interfaces.
L01201        real ua(kms:kme)                ! Wind speed in the x direction
L01202        real va(kms:kme)                ! Wind speed in the y direction
L01203        real pt(kms:kme)                ! Potential temperature
L01204        real da(kms:kme)                ! Air density
L01205        real pr(kms:kme)                ! Air pressure
L01206        real pt0(kms:kme)               ! Reference potential temperature (could be equal to "pt")
L01207        real qv(kms:kme)              ! Specific humidity
L01208        real dt                     ! Time step
L01209        real zr                     ! Zenith angle
L01210        real deltar                 ! Declination of the sun
L01211        real ah                     ! Hour angle
L01212        real rs                     ! Solar radiation
L01213        real rld                    ! Downward flux of the longwave radiation
L01214        real xlat                   ! Latitude
L01215        real swddir                 ! short wave direct solar radiation   !_gl
L01216        real swddif                 ! short wave diffuse solar radiation  !_gl
L01217
L01218  ! Data relative to the "urban grid"
L01219
L01220        integer iurb                ! Current urban class
L01221
L01222  !    Radiation parameters
L01223        real albg                   ! Albedo of the ground
L01224        real albw                   ! Albedo of the wall
L01225        real albr                   ! Albedo of the roof
L01226        real albwin                 ! Albedo of the windows
L01227        real emwind                 ! Emissivity of windows
L01228        real emg                    ! Emissivity of ground
L01229        real emw                    ! Emissivity of wall
L01230        real emr                    ! Emissivity of roof
L01231
L01232
L01233  !    fww,fwg,fgw,fsw,fsg are the view factors used to compute the long and
L01234  !    short wave radation.
L01235  !    The calculation of these factor is explained in the Appendix A of the BLM paper
L01236        real fww(nz_um,nz_um,ndm,nurbm)  !  from wall to wall
L01237        real fwg(nz_um,ndm,nurbm)        !  from wall to ground
L01238        real fgw(nz_um,ndm,nurbm)        !  from ground to wall
L01239        real fsw(nz_um,ndm,nurbm)        !  from sky to wall
L01240        real fws(nz_um,ndm,nurbm)        !  from wall to sky
L01241        real fsg(ndm,nurbm)              !  from sky to ground
L01242
L01243  !    Street parameters
L01244        integer ndu                  ! Number of street direction for each urban class
L01245        real bs_u(ndm,nurbm)         ! Building width
L01246
L01247  !    Grid parameters
L01248        integer nzu           ! Number of layer in the urban grid
L01249        real z_u(nz_um)       ! Height of the urban grid levels
L01250  !FS
L01251        real cop_u(nurbm)
L01252        real pwin_u(nurbm)
L01253        real beta_u(nurbm)
L01254        integer sw_cond_u(nurbm)
L01255        real time_on_u(nurbm)
L01256        real time_off_u(nurbm)
L01257        real targtemp_u(nurbm)
L01258        real gaptemp_u(nurbm)
L01259        real targhum_u(nurbm)
L01260        real gaphum_u(nurbm)
L01261        real perflo_u(nurbm)
L01262        real hsesf_u(nurbm)
L01263        real hsequip(24)
L01264        real irho(24)
L01265        real gr_frac_roof
L01266        real pv_frac_roof
L01267        integer gr_flag
L01268        integer gr_type
L01269        real tpv(nbui_max)
L01270       real  sfpv(nbui_max)
L01271       real sfr_indoor(nbui_max)
L01272  ! ---------------------------------------------------------------------
L01273  ! INPUT-OUTPUT
L01274  ! ---------------------------------------------------------------------
L01275
L01276  ! Data relative to the "urban grid" which should be stored from the current time step to the next one
L01277
L01278        real tw(2*ndm,nz_um,nwr_u,nbui_max)  ! Temperature in each layer of the wall [K]
L01279        real tr(ndm,nz_um,nwr_u)  ! Temperature in each layer of the roof [K]
L01280        real tg(ndm,ng_u)          ! Temperature in each layer of the ground [K]
L01281        real trv(ndm,nz_um,ngr_u)  ! Temperature in each layer of the green roof [K]
L01282        real sfw(2*ndm,nz_um,nbui_max)      ! Sensible heat flux from walls
L01283        real sfg(ndm)              ! Sensible heat flux from ground (road)
L01284        real sfr(ndm,nz_um)      ! Sensible heat flux from roofs
L01285        real sfrv(ndm,nz_um)      ! Sensible heat flux from green roofs
L01286        real lfrv(ndm,nz_um)      ! Latent heat flux from green roofs
L01287        real dg(ndm)              ! water depth ground (road)
L01288        real dgr(ndm,nz_um)      ! water depth roofs
L01289        real lfr(ndm,nz_um)      ! Latent heat flux from roofs
```

```
L01290        real lfg(ndm)              ! Latent heat flux from ground (road)
L01291        real drain(ndm,nz_um)        ! Green roof drainage
L01292        real rainbl              ! Rainfall
L01293        real gfg(ndm)              ! Heat flux transferred from the surface of the ground (road) towards the interior
L01294        real gfr(ndm,nz_um)     ! Heat flux transferred from the surface of the roof towards the interior
L01295        real gfw(2*ndm,nz_um,nbui_max)     ! Heat flux transfered from the surface of the walls towards the interior
L01296        real qr(ndm,nz_um,ngr_u)  ! Green Roof soil moisture
L01297
L01298  ! ----------------------------------------------------------------------
L01299  ! OUTPUT:
L01300  ! ----------------------------------------------------------------------
L01301
L01302
L01303  ! Data relative to the "mesoscale grid"
L01304
L01305        real sf(kms:kme)             ! Surface of the "mesoscale grid" cells taking into account the buildings
L01306        real vl(kms:kme)               ! Volume of the "mesoscale grid" cells taking into account the buildings
L01307
L01308  !    Implicit and explicit components of the source and sink terms at each levels,
L01309  !     the fluxes can be computed as follow: FX = A*X + B   example: Heat fluxes = a_t * pt + b_t
L01310        real a_u(kms:kme)              ! Implicit component of the momentum sources or sinks in the X-direction
L01311        real a_v(kms:kme)              ! Implicit component of the momentum sources or sinks in the Y-direction
L01312        real a_t(kms:kme)              ! Implicit component of the heat sources or sinks
L01313        real a_e(kms:kme)              ! Implicit component of the TKE sources or sinks
L01314        real b_u(kms:kme)              ! Explicit component of the momentum sources or sinks in the X-direction
L01315        real b_v(kms:kme)              ! Explicit component of the momentum sources or sinks in the Y-direction
L01316        real b_t(kms:kme)              ! Explicit component of the heat sources or sinks
L01317        real b_ac(kms:kme)
L01318        real b_e(kms:kme)              ! Explicit component of the TKE sources or sinks
L01319        real b_q(kms:kme)              ! Explicit component of the humidity sources or sinks
L01320        real dlg(kms:kme)              ! Height above ground (L_ground in formula (24) of the BLM paper).
L01321        real dl_u(kms:kme)             ! Length scale (lb in formula (22) ofthe BLM paper).
L01322  ! ----------------------------------------------------------------------
L01323  ! LOCAL:
L01324  ! ----------------------------------------------------------------------
L01325
L01326        real dz(kms:kme)               ! vertical space steps of the "mesoscale grid"
L01327  ! Data interpolated from the "mesoscale grid" to the "urban grid"
L01328
L01329        real ua_u(nz_um)          ! Wind speed in the x direction
L01330        real va_u(nz_um)          ! Wind speed in the y direction
L01331        real pt_u(nz_um)          ! Potential temperature
L01332        real da_u(nz_um)          ! Air density
L01333        real pt0_u(nz_um)         ! Reference potential temperature
L01334        real pr_u(nz_um)          ! Air pressure
L01335        real qv_u(nz_um)          !Specific humidity
L01336
L01337  ! Data defining the building and street charateristics
L01338
L01339        real alag(ng_u)           ! Ground thermal diffusivity for the current urban class [m^2 s^-1]
L01340
L01341        real csg(ng_u)            ! Specific heat of the ground material of the current urban class [J m^3 K^-1]
L01342        real csr(nwr_u)            ! Specific heat of the roof material for the current urban class [J m^3 K^-1]
L01343        real csw(nwr_u)            ! Specific heat of the wall material for the current urban class [J m^3 K^-1]
L01344
L01345        real z0(ndm,nz_um)      ! Roughness lengths "profiles"
L01346        real ws(ndm)              ! Street widths of the current urban class
L01347        real bs(ndm)              ! Building widths of the current urban class
L01348        real strd(ndm)            ! Street lengths for the current urban class
L01349        real drst(ndm)            ! Street directions for the current urban class
L01350        real ss(nz_um)          ! Probability to have a building with height h
L01351        real pb(nz_um)          ! Probability to have a building with an height equal
L01352        real cdrag(nz_um)
L01353        real alp
L01354
L01355  ! Solar radiation at each level of the "urban grid"
L01356
L01357       real rsg(ndm)             ! Short wave radiation from the ground
L01358        real rsw(2*ndm,nz_um)     ! Short wave radiation from the walls
L01359        real rsd(2*ndm,nz_um)     ! Direct Short wave radiation received by the walls
L01360        real rlg(ndm)             ! Long wave radiation from the ground
L01361        real rlw(2*ndm,nz_um)     ! Long wave radiation from the walls
L01362
L01363  ! Potential temperature of the surfaces at each level of the "urban grid"
L01364
L01365        real ptg(ndm)              ! Ground potential temperatures
L01366        real ptr(ndm,nz_um)     ! Roof potential temperatures
L01367        real ptrv(ndm,nz_um)     ! Roof potential temperatures
L01368        real ptw(2*ndm,nz_um,nbui_max)     ! Walls potential temperatures
L01369
L01370        real tg_av(ndm)
L01371  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L01372  ! vertical surfaces (walls) ans horizontal surfaces (roofs and street)
L01373  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L01374  ! Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L01375
L01376        real uhb_u(ndm,nz_um)   ! U (wind component) Horizontal surfaces, B (explicit) term
L01377        real uva_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, A (implicit) term
L01378        real uvb_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, B (explicit) term
L01379        real vhb_u(ndm,nz_um)   ! V (wind component) Horizontal surfaces, B (explicit) term
L01380        real vva_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, A (implicit) term
L01381        real vvb_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, B (explicit) term
L01382        real thb_u(ndm,nz_um)   ! Temperature        Horizontal surfaces, B (explicit) term
L01383        real tva_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, A (implicit) term
L01384        real tvb_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, B (explicit) term
L01385
L01386
L01387   real tvb_ac(2*ndm,nz_um)
L01388        real ehb_u(ndm,nz_um)   ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L01389        real evb_u(2*ndm,nz_um)   ! Energy (TKE)         Vertical surfaces, B (explicit) term
L01390        real qhb_u(ndm,nz_um)     ! Humidity      Horizontal surfaces, B (explicit) term
L01391        real qvb_u(2*ndm,nz_um)   ! Humidity      Vertical surfaces, B (explicit) term
L01392  !
L01393        real rs_abs ! solar radiation absorbed by urban surfaces
L01394        real rl_up ! longwave radiation emitted by urban surface to the atmosphere
L01395        real emiss ! mean emissivity of the urban surface
L01396        real grdflx_urb ! ground heat flux
L01397        real dt_int ! internal time step
```

```
L01398        integer nt_int ! number of internal time step
L01399        integer iz,id, it_int,it
L01400        integer iw
L01401
L01402  !---------------------------------------
L01403  !New variables uses in BEM
L01404  !---------------------------------------
L01405
L01406        real tmp_u(nz_um)     !Air Temperature [K]
L01407
L01408        real dzw(nwr_u)       !Layer sizes in the walls
L01409        real dzr(nwr_u)       !Layer sizes in the roofs
L01410        real dzf(nf_u)        !Layer sizes in the floors
L01411        real dzgb(ngb_u)      !Layer sizes in the ground below the buildings
L01412
L01413        real csgb(ngb_u)      !Specific heat of the ground material below the buildings
L01414
L01415        real csf(nf_u)        !Specific heat of the floors materials in the buildings
L01416                              !of the current urban class at each levels[J m^3 K^-1]
L01417        real alar(nwr_u+1)    ! Roof thermal diffusivity for the current urban class [W/m K]
L01418        real alaw(nwr_u+1)    ! Walls thermal diffusivity for the current urban class [W/m K]
L01419        real alaf(nf_u+1)     ! Floor thermal diffusivity at each wall layers [W/m K]
L01420        real alagb(ngb_u+1)   ! Ground thermal diffusivity below the building at each wall layer [W/m K]
L01421
L01422        real sfrb(ndm,nbui_max)        ! Sensible heat flux from roofs [W/m2]
L01423        real sfrbpv(ndm,nbui_max)      ! Sensible heat flux from PV panels [W/m2]
L01424        real sfrpv(ndm,nz_um)          ! Sensible heat flux from PV panels [W/m2]
L01425        real sfrvb(ndm,nbui_max)        ! Sensible heat flux from roofs [W/m2]
L01426        real lfrvb(ndm,nbui_max)        ! Sensible heat flux from roofs [W/m2]
L01427        real lfrb(ndm,nbui_max)        ! Sensible heat flux from roofs [W/m2]
L01428
L01429        real gfrb(ndm,nbui_max)        ! Heat flux flowing inside the roofs [W/m2]
L01430        real sfwb1D(2*ndm,nz_um)    !Sensible heat flux from the walls [W/m2]
L01431        real sfwin(2*ndm,nz_um,nbui_max)!Sensible heat flux from windows [W/m2]
L01432        real sfwinb1D(2*ndm,nz_um)  !Sensible heat flux from windows [W/m2]
L01433        real gfwb1D(2*ndm,nz_um)    !Heat flux flowing inside the walls [W/m2]
L01434
L01435        real qlev(nz_um,nbui_max)      !specific humidity [kg/kg]
L01436        real qlevb1D(nz_um)         !specific humidity [kg/kg]
L01437        real tlev(nz_um,nbui_max)      !Indoor temperature [K]
L01438        real tlevb1D(nz_um)         !Indoor temperature [K]
L01439        real twb1D(2*ndm,nwr_u,nz_um)     !Wall temperature in BEM [K]
L01440        real twlev(2*ndm,nz_um,nbui_max)     !Window temperature in BEM [K]
L01441        real twlevb1D(2*ndm,nz_um)        !Window temperature in BEM [K]
L01442        real tglev(ndm,ngb_u,nbui_max)        !Ground temperature below a building in BEM [K]
L01443        real tglevb1D(ngb_u)               !Ground temperature below a building in BEM [K]
L01444        real tflev(ndm,nf_u,nz_um-1,nbui_max)!Floor temperature in BEM[K]
L01445        real tflevb1D(nf_u,nz_um-1)       !Floor temperature in BEM[K]
L01446        real trb(ndm,nwr_u,nbui_max)         !Roof temperature in BEM [K]
L01447        real trvb(ndm,ngr_u,nbui_max)         !Roof temperature in BEM [K]
L01448        real trb1D(nwr_u)
L01449
L01450        real sflev(nz_um,nz_um)     ! sensible heat flux due to the air conditioning systems [W]
L01451        real lflev(nz_um,nz_um)     ! latent heat flux due to the air conditioning systems  [W]
L01452        real consumlev(nz_um,nz_um) ! consumption due to the air conditioning systems [W]
L01453        real sflev1D(nz_um)         ! sensible heat flux due to the air conditioning systems [W]
L01454        real lflev1D(nz_um)         ! latent heat flux due to the air conditioning systems  [W]
L01455        real consumlev1D(nz_um)     ! consumption due to the air conditioning systems [W]
L01456        real eppvlev(nz_um)         ! Electricity production of PV panels [W]
L01457         real tpvlev(ndm,nz_um)
L01458        real tpvlevb(ndm,nbui_max)        ! Sensible heat flux from roofs [W/m2]
L01459        real sfvlev(nz_um,nz_um)    ! sensible heat flux due to ventilation [W]
L01460        real lfvlev(nz_um,nz_um)    ! latent heat flux due to ventilation [W]
L01461        real sfvlev1D(nz_um)        ! sensible heat flux due to ventilation [W]
L01462        real lfvlev1D(nz_um)        ! Latent heat flux due to ventilation [W]
L01463
L01464        real ptwin(2*ndm,nz_um,nbui_max)  ! window potential temperature
L01465        real tw_av(2*ndm,nz_um)        ! Averaged temperature of the wall surfaces
L01466        real twlev_av(2*ndm,nz_um)     ! Averaged temperature of the windows
L01467        real sfw_av(2*ndm,nz_um)       ! Averaged sensible heat from walls
L01468        real sfwind_av(2*ndm,nz_um)    ! Averaged sensible heat from windows
L01469        integer flag_pvp
L01470        integer nbui                !Total number of different type of buildings in an urban class
L01471        integer nlev(nz_um)         !Number of levels in each different type of buildings in an urban class
L01472        integer ibui,ily
L01473        real :: nhourday   ! Number of hours from midnight, local time
L01474        real :: st4,gamma,fp,lmr,smr,prova
L01475        real hfgr(ndm,nz_um)!heat flux green roof
L01476        real hfgrb(ndm,nbui_max)
L01477        real irri_per_ts
L01478        real irri_now
L01479        real tr_av(ndm,nz_um)
L01480        real tr_avb(ndm,nbui_max)
L01481        real sfr_avb(ndm,nbui_max)
L01482  ! ------------------------------------------------------------------------
L01483  ! END VARIABLES DEFINITIONS
L01484  ! ------------------------------------------------------------------------
L01485
L01486  ! Fix some usefull parameters for the computation of the sources or sinks
L01487  !
L01488  !initialize the variables inside the param routine
L01489
L01490          nhourday=ah/PI*180./15.+12.
L01491          if (nhourday >= 24) nhourday = nhourday - 24
L01492          if (nhourday < 0)  nhourday = nhourday + 24
L01493
L01494
L01495         if(sum(irho).gt.0)then
L01496           irri_per_ts=h_water/sum(irho)
L01497          else
L01498           irri_per_ts=0.
L01499          endif
L01500
L01501        if(irho(int(nhourday)+1).ne.0)then
L01502          irri_now=irri_per_ts
L01503        else
L01504          irri_now=0.
L01505        endif
```

```
L01506
L01507        do iz=kts,kte
L01508           dz(iz)=z(iz+1)-z(iz)
L01509        end do
L01510  ! Interpolation on the "urban grid"
L01511        call interpol(kms,kme,kts,kte,nzu,z,z_u,ua,ua_u)
L01512        call interpol(kms,kme,kts,kte,nzu,z,z_u,va,va_u)
L01513        call interpol(kms,kme,kts,kte,nzu,z,z_u,pt,pt_u)
L01514        call interpol(kms,kme,kts,kte,nzu,z,z_u,pt0,pt0_u)
L01515        call interpol(kms,kme,kts,kte,nzu,z,z_u,pr,pr_u)
L01516        call interpol(kms,kme,kts,kte,nzu,z,z_u,da,da_u)
L01517        call interpol(kms,kme,kts,kte,nzu,z,z_u,qv,qv_u)
L01518  ! Compute the modification of the radiation due to the buildings
L01519
L01520
L01521        call averaging_temp(tw,twlev,ss,pb,tw_av,twlev_av, &
L01522                            sfw_av,sfwind_av,sfw,sfwin)
L01523
L01524       do id=1,ndu
L01525         tg_av(id)=tg(id,ng_u)
L01526       do iz=1,nz_um
L01527
L01528         tr_av(id,iz)=((1-gr_frac_roof)*tr(id,iz,nwr_u)**4.+   &
L01529         gr_frac_roof*trv(id,iz,ngr_u)**4.)**(1./4.)
L01530
L01531       enddo
L01532       enddo
L01533
L01534
L01535
L01536
L01537
L01538       call modif_rad(iurb,ndu,nzu,z_u,ws,           &
L01539                      drst,strd,ss,pb,                &
L01540                      tw_av,tg_av,twlev_av,albg,albw,    &
L01541                      emw,emg,pwin_u(iurb),albwin,    &
L01542                      emwind,fww,fwg,fgw,fsw,fsg,     &
L01543                      zr,deltar,ah,xlat,swddir,swddif,      &  !_gl
L01544                      rs,rld,rsw,rsd,rsg,rlw,rlg)
L01545
L01546
L01547
L01548
L01549  ! calculation of the urban albedo and the upward long wave radiation
L01550
L01551
L01552         call upward_rad(ndu,nzu,ws,bs,sigma,pb,ss,                &
L01553                         tg_av,emg,albg,rlg,rsg,sfg,lfg,                 &
L01554                         tw_av,emw,albw,rlw,rsw,sfw_av,             &
L01555                         tr_av,emr,albr,emwind,                        &
L01556                         albwin,twlev_av,pwin_u(iurb),sfwind_av,rld,rs,sfr,sfrv,lfr,lfrv, &
L01557                         rs_abs,rl_up,emiss,grdflx_urb,gr_frac_roof,tpvlev,pv_frac_roof)
L01558
L01559      do id=1,ndu
L01560      if(dg(id).le.dgmax) then
L01561        dg(id)=dg(id)+(rainbl+(lfg(id)*dt)/latent)
L01562       endif
L01563      if (dg(id).lt.0) then
L01564        dg(id)=0
L01565      endif
L01566      if (dg(id).gt.dgmax) then
L01567        dg(id)=dgmax
L01568      endif
L01569     do iz=2,nz_um
L01570      if(dgr(id,iz).le.drmax) then
L01571       dgr(id,iz)=dgr(id,iz)+(rainbl+(lfr(id,iz)*dt)/latent)
L01572      endif
L01573      if (dgr(id,iz).lt.0) then
L01574       dgr(id,iz)=0
L01575      endif
L01576      if (dgr(id,iz).gt.drmax) then
L01577       dgr(id,iz)=drmax
L01578      endif
L01579     enddo
L01580    enddo !id
L01581
L01582
L01583
L01584
L01585       call surf_temp(ndu,pr_u,dt,                   &
L01586                      rld,rsg,rlg,                    &
L01587                      tg,alag,csg,emg,albg,ptg,sfg,lfg,gfg)
L01588       if(gr_flag.eq.1)then
L01589       if(gr_frac_roof.gt.0.)then
L01590       hfgr=0.
L01591       call roof_temp_veg(ndu,pr_u,dt,                  &
L01592                      rld,rs,                 &
L01593                      trv,ptrv,sfrv,lfrv,gfr,qr,rainbl,drain,hfgr,tr,alar(5),dzr(5),csr(5),nzu,irri_now,gr_type,
↪  pv_frac_roof,tpvlev)
L01594
L01595       endif
L01596       endif
L01597
L01598
L01599
L01600         do iz=1,nzu !Compute the outdoor temperature
L01601       tmp_u(iz)=pt_u(iz)*(pr_u(iz)/p0)**(rcp_u)
L01602         end do
L01603
L01604         ibui=0
L01605         nlev=0
L01606         nbui=0
L01607         hfgrb=0.
L01608         sfrb=0.      !Sensible heat flux from roof
L01609         sfrbpv=0.    !Sensible heat flux from PV panels
L01610         sfrpv=0.     !Sensible heat flux from PV panels
L01611         lfrvb=0.
L01612         lfrb=0.
L01613         sfrvb=0.
```

```
L01614          gfrb=0.      !Heat flux flowing inside the roof
L01615          sfwb1D=0.    !Sensible heat flux from walls
L01616          sfwinb1D=0.  !Sensible heat flux from windows
L01617          gfwb1D=0.    !Heat flux flowing inside the walls[W/m2]
L01618
L01619
L01620          twb1D=0.     !Wall temperature
L01621          twlevb1D=0.  !Window temperature
L01622          tglevb1D=0.  !Ground temperature below a building
L01623          tflevb1D=0.  !Floor temperature
L01624          trvb=0.
L01625          trb=0.       !Roof temperature
L01626          trb1D=0.     !Roof temperature
L01627          tr_avb=0.
L01628          qlevb1D=0. !Indoor humidity
L01629          tlevb1D=0. !indoor temperature
L01630
L01631          sflev1D=0.     !Sensible heat flux from the a.c.
L01632          lflev1D=0.     !Latent heat flux from the a.c.
L01633          consumlev1D=0.!Consumption from the a.c.
L01634          tpvlevb=0.
L01635          eppvlev=0.
L01636          sfvlev1D=0.    !Sensible heat flux from the natural ventilation
L01637          lfvlev1D=0.    !Latent heat flux from natural ventilation
L01638          ptw=0.         !Wall potential temperature
L01639          ptwin=0.       !Window potential temperature
L01640          ptr=0.         !Roof potential temperature
L01641
L01642          do iz=1,nz_um
L01643            if(ss(iz).gt.0) then
L01644              ibui=ibui+1
L01645              nlev(ibui)=iz-1
L01646              nbui=ibui
L01647              do id=1,ndm
L01648                 tr_avb(id,ibui)=tr_av(id,iz)
L01649               tpvlevb(id,ibui)=tpvlev(id,iz)
L01650                 hfgrb(id,ibui)=hfgr(id,iz)
L01651                 sfrb(id,ibui)=sfr(id,iz)
L01652                  sfrvb(id,ibui)=sfrv(id,iz)
L01653                 lfrvb(id,ibui)=lfrv(id,iz)
L01654                 lfrb(id,ibui)=lfr(id,iz)
L01655                 sfr_avb(id,ibui)=(1-gr_frac_roof)*sfr(id,iz)+gr_frac_roof*(sfrv(id,iz))
L01656                 do ily=1,nwr_u
L01657                    trb(id,ily,ibui)=tr(id,iz,ily)
L01658                 enddo
L01659                 do ily=1,ngr_u
L01660                    trvb(id,ily,ibui)=trv(id,iz,ily)
L01661                 enddo
L01662
L01663               enddo
L01664          endif
L01665            end do  !iz
L01666
L01667  !-------------------------------------------------------------------------------
L01668  !Loop over BEM  ---------------------------------------------------------------
L01669  !-------------------------------------------------------------------------------
L01670  !-------------------------------------------------------------------------------
L01671
L01672          do ibui=1,nbui
L01673             do iz=1,nz_um
L01674                qlevb1D(iz)=qlev(iz,ibui)
L01675                tlevb1D(iz)=tlev(iz,ibui)
L01676             enddo
L01677
L01678             do id=1,ndm
L01679
L01680                do ily=1,nwr_u
L01681                   trb1D(ily)=trb(id,ily,ibui)
L01682                enddo
L01683                do ily=1,ngb_u
L01684                   tglevb1D(ily)=tglev(id,ily,ibui)
L01685                enddo
L01686
L01687                do ily=1,nf_u
L01688                   do iz=1,nz_um-1
L01689                     tflevb1D(ily,iz)=tflev(id,ily,iz,ibui)
L01690                   enddo
L01691                enddo
L01692
L01693                do iz=1,nz_um
L01694                   sfwinb1D(2*id-1,iz)=sfwin(2*id-1,iz,ibui)
L01695                   sfwinb1D(2*id,iz)=sfwin(2*id,iz,ibui)
L01696                enddo
L01697
L01698                do iz=1,nz_um
L01699                   do ily=1,nwr_u
L01700                      twb1D(2*id-1,ily,iz)=tw(2*id-1,iz,ily,ibui)
L01701                      twb1D(2*id,ily,iz)=tw(2*id,iz,ily,ibui)
L01702                   enddo
L01703                   sfwb1D(2*id-1,iz)=sfw(2*id-1,iz,ibui)
L01704                   sfwb1D(2*id,iz)=sfw(2*id,iz,ibui)
L01705                   twlevb1D(2*id-1,iz)=twlev(2*id-1,iz,ibui)
L01706                   twlevb1D(2*id,iz)=twlev(2*id,iz,ibui)
L01707                enddo
L01708             enddo
L01709
L01710       !print*,'HFGR_BEFORE_CALLING_BEM',hfgr(nlev(ibui))
L01711
L01712             call BEM(nz_um,nlev(ibui),nhourday,dt,bs_u(1,iurb),               &
L01713                      bs_u(2,iurb),dz_u,nwr_u,nf_u,nwr_u,ngb_u,sfwb1D,gfwb1D,   &
L01714                      sfwinb1D,sfr_avb(1,ibui),lfrb(1,ibui),gfrb(1,ibui),       &
L01715                      sfrbpv(1,ibui),                                           &
L01716                      latent,sigma,albw,albwin,albr,                            &
L01717                      emr,emw,emwind,rsw,rlw,r,cp_u,                            &
L01718                      da_u,tmp_u,qv_u,pr_u,rs,swddif,rld,dzw,csw,alaw,pwin_u(iurb),    &
L01719                      cop_u(iurb),beta_u(iurb),sw_cond_u(iurb),time_on_u(iurb), &
L01720                      time_off_u(iurb),targtemp_u(iurb),gaptemp_u(iurb),        &
L01721                      targhum_u(iurb),gaphum_u(iurb),perflo_u(iurb),            &
L01722                      gr_frac_roof,pv_frac_roof,gr_flag,                        &
```

```
                    ua_u,va_u,                                                  &
                    hsesf_u(iurb),hsequip,                                      &
                    dzf,csf,alaf,dzgb,csgb,alagb,dzr,csr,                       &
                    alar,tlevb1D,qlevb1D,twb1D,twlevb1D,tflevb1D,tglevb1D,       &
                    trb1D,sflev1D,lflev1D,consumlev1D,eppvlev(ibui),            &
                    tpvlevb(1,ibui),                                            &
                    sfvlev1D,lfvlev1D,hfgrb(1,ibui),tr_avb(1,ibui),             &
                    tpv(ibui),sfpv(ibui),sfr_indoor(ibui))


!
!Temporal modifications
!
         tpvlevb(2,ibui)=tpvlevb(1,ibui)
         sfrb(2,ibui)=sfrb(1,ibui)
         sfrvb(2,ibui)=sfrvb(1,ibui)
         lfrvb(2,ibui)=lfrvb(1,ibui)
         lfrb(2,ibui)=lfrb(1,ibui)
         sfrbpv(2,ibui)=sfrbpv(1,ibui)
         gfrb(2,ibui)=gfrb(1,ibui)
         hfgrb(2,ibui)=hfgrb(1,ibui)
!End temporal modifications
!
           do iz=1,nz_um
             qlev(iz,ibui)=qlevb1D(iz)
             tlev(iz,ibui)=tlevb1D(iz)
             sflev(iz,ibui)=sflev1D(iz)
             lflev(iz,ibui)=lflev1D(iz)
             consumlev(iz,ibui)=consumlev1D(iz)
             sfvlev(iz,ibui)=sfvlev1D(iz)
             lfvlev(iz,ibui)=lfvlev1D(iz)
           enddo

           do id=1,ndm
              do ily=1,nwr_u
                 trb(id,ily,ibui)=trb1D(ily)
              enddo
              do ily=1,ngb_u
                 tglev(id,ily,ibui)=tglevb1D(ily)
              enddo

              do ily=1,nf_u
              do iz=1,nz_um-1
                 tflev(id,ily,iz,ibui)=tflevb1D(ily,iz)
              enddo
              enddo


             do iz=1,nz_um
                do ily=1,nwr_u
                   tw(2*id-1,iz,ily,ibui)=twb1D(2*id-1,ily,iz)
                   tw(2*id,iz,ily,ibui)=twb1D(2*id,ily,iz)
                enddo
                gfw(2*id-1,iz,ibui)=gfwb1D(2*id-1,iz)
                gfw(2*id,iz,ibui)=gfwb1D(2*id,iz)
                twlev(2*id-1,iz,ibui)=twlevb1D(2*id-1,iz)
                twlev(2*id,iz,ibui)=twlevb1D(2*id,iz)
             enddo
           enddo

         enddo !ibui

!-------------------------------------------------------------------------------
!End Loop over BEM ------------------------------------------------------------
!-------------------------------------------------------------------------------
!-------------------------------------------------------------------------------

        ibui=0

         do iz=1,nzu!nz_um

          if(ss(iz).gt.0) then
            ibui=ibui+1
            do id=1,ndm
               gfr(id,iz)=gfrb(id,ibui)
             tpvlev(id,iz)=tpvlevb(id,ibui)
               sfr(id,iz)=sfrb(id,ibui)
               hfgr(id,iz)=hfgrb(id,ibui)
               sfrpv(id,iz)=-sfrbpv(id,ibui)
               lfr(id,iz)=lfrb(id,ibui)
               do ily=1,nwr_u
                  tr(id,iz,ily)=trb(id,ily,ibui)
               enddo
               ptr(id,iz)=tr(id,iz,nwr_u)*(pr_u(iz)/p0)**(-rcp_u)
            enddo
          endif
         enddo !iz

!Compute the potential temperature for the vertical surfaces of the buildings

        do id=1,ndm
           do iz=1,nzu!nz_um
              do ibui=1,nbui
                 ptw(2*id-1,iz,ibui)=tw(2*id-1,iz,nwr_u,ibui)*(pr_u(iz)/p0)**(-rcp_u)
                 ptw(2*id,iz,ibui)=tw(2*id,iz,nwr_u,ibui)*(pr_u(iz)/p0)**(-rcp_u)
                 ptwin(2*id-1,iz,ibui)=twlev(2*id-1,iz,ibui)*(pr_u(iz)/p0)**(-rcp_u)
                 ptwin(2*id,iz,ibui)=twlev(2*id,iz,ibui)*(pr_u(iz)/p0)**(-rcp_u)

              enddo
           enddo
        enddo
!NEW CDRAG!
     do iz=1,nz_um
       alp=0.
       do id=1,ndu
        alp=alp+bs(id)/(ws(id)+bs(id))*pb(iz)
       enddo
       alp=alp/ndu
       if(alp.lt.0.29)then
```

```
L01832          cdrag(iz)=3.32*alp**0.47
L01833         else
L01834          cdrag(iz)=1.85
L01835         endif
L01836       enddo
L01837
L01838
L01839
L01840  ! Compute the implicit and explicit components of the sources or sinks on the "urban grid"
L01841
L01842       call buildings(iurb,ndu,nzu,z0,cdrag,ua_u,va_u,                                  &
L01843                      pt_u,pt0_u,ptg,ptr,ptrv,da_u,qv_u,pr_u,tmp_u,ptw,ptwin,pwin_u(iurb),drst,     &
↪
L01844                      uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u,qvb_u,qhb_u,   &
L01845                      uhb_u,vhb_u,thb_u,ehb_u,ss,dt,sfw,sfg,sfr,sfrpv,sfrv,lfrv,   &
L01846                      dgr,dg,lfr,lfg,                                                  &
L01847                      sfwin,pb,bs_u,dz_u,sflev,lflev,sfvlev,lfvlev,tvb_ac,ix,iy,rsg,rs,qr,gr_frac_roof,  &
L01848                      pv_frac_roof,gr_flag,gr_type)
L01849
L01850
L01851
L01852
L01853  ! Calculation of the sensible heat fluxes for the ground, the wall and roof
L01854  ! Sensible Heat Flux = density * Cp_U * ( A* potential temperature + B )
L01855  ! where A and B are the implicit and explicit components of the heat sources or sinks.
L01856
L01857  ! Interpolation on the "mesoscale grid"
L01858
L01859       call urban_meso(ndu,kms,kme,kts,kte,nzu,z,dz,z_u,pb,ss,bs,ws,sf,  &
L01860                      vl,uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u,     &
L01861                      uhb_u,vhb_u,thb_u,ehb_u,qhb_u,qvb_u,              &
L01862                      a_u,a_v,a_t,a_e,b_u,b_v,b_t,b_e,b_q,tvb_ac,b_ac)
L01863
L01864
L01865  ! Calculation of the length scale taking into account the buildings effects
L01866
L01867       call interp_length(ndu,kms,kme,kts,kte,nzu,z_u,z,ss,ws,bs,dlg,dl_u)
L01868
L01869       return
L01870       end subroutine BEP1D
```

讲解：单点单类一柱完成「辐射 → 表面温度 →BEM→ 源项」全链，九步：① 小时角换算 nhourday 与灌溉窗口判断（irho 24 元素小时表，非零才灌 irri_per_ts=h_water/Σirho）；②interpol 七个量从模式网格到城市网格（质量守恒的重叠体积平均，BLM 论文附录 B.1）；③averaging_temp 按建筑型概率 ss/pb 求**四次幂加权**的墙面/窗面均温（辐射计算要 T^4，先平均 T^4 再开四次方）；④tr_av 绿屋顶混合：$((1-gr_frac)\cdot tr^4+gr_frac\cdot trv^4)^{(1/4)}$——绿屋顶与普通屋顶按面积比在辐射意义下混合；⑤modif_rad：阴影 +多次反射（14b）；⑥upward_rad：给辐射方案的城市有效反照率/发射率/上行长波/吸收短波/地面热通量（14b）；⑦ 水量收支：dg/dgr += 降雨 +蒸发凝结，钳位 [0,1 mm]；⑧surf_temp（路面导热）与 roof_temp_veg（绿屋顶土壤-植被，14a 本篇）；⑨**BEM 循环**（L1671–1782）：对每个建筑型 ibui 折叠出 1D 调 module_sf_bem.BEM——室内温度/湿度、空调冷热、通风、窗户热流全在那边算；返回后展开；「Temporal modifications」块把方向 2 的屋顶量复制方向 1（BEM 只算一个方向的屋顶）。⑩**NEW CDRAG**（L1823–1835）：街道拖曳系数 cdrag=3.32 • alp^0.47（alp<0.29）否则 1.85，alp=各方向 bs/(bs+ws) • pb 的平均——建筑密度越高拖曳越强，饱和值 1.85。⑪buildings（源项 A/B，14a）→ urban_meso（城市网格 → 模式网格，14b）→ interp_length（混合长，14b）。

六、几何与材料参数 param（全文）

```
!file: phys/module_sf_bep_bem.F    L1875-L2095    (subroutine param)
L01875        subroutine param(iurb,nzu,nzurb,nzurban,ndu,                    &
L01876                         csg_u,csg,alag_u,alag,csr_u,csr,               &
L01877                         alar_u,alar,csw_u,csw,alaw_u,alaw,             &
L01878                         ws_u,ws_urb,ws,bs_u,bs_urb,bs,z0g_u,z0r_u,z0,  &
L01879                         strd_u,strd,drst_u,drst,ss_u,ss_urb,ss,pb_u,   &
L01880                         pb_urb,pb,dzw,dzr,dzf,csf,alaf,dzgb,csgb,alagb,&
L01881                         lp_urb,lb_urb,hgt_urb,frc_urb)
L01882
L01883  ! ------------------------------------------------------------------
L01884  !    This routine prepare some usefull parameters
L01885  ! ------------------------------------------------------------------
L01886
L01887       implicit none
L01888
L01889
L01890  ! ------------------------------------------------------------------
L01891  ! INPUT:
L01892  ! ------------------------------------------------------------------
L01893       integer iurb                ! Current urban class
L01894       integer nzu                 ! Number of vertical urban levels in the current class
L01895       integer ndu                 ! Number of street direction for the current urban class
L01896       integer nzurb               ! Number of vertical urban levels in the current class
L01897       real alag_u(nurbm)          ! Ground thermal diffusivity [m^2 s^-1]
L01898       real alar_u(nurbm)          ! Roof thermal diffusivity [m^2 s^-1]
L01899       real alaw_u(nurbm)          ! Wall thermal diffusivity [m^2 s^-1]
L01900       real bs_u(ndm,nurbm)        ! Building width
L01901       real csg_u(nurbm)           ! Specific heat of the ground material [J m^3 K^-1]
L01902       real csr_u(nurbm)           ! Specific heat of the roof material [J m^3 K^-1]
L01903       real csw_u(nurbm)           ! Specific heat of the wall material [J m^3 K^-1]
L01904       real drst_u(ndm,nurbm)      ! Street direction
L01905       real strd_u(ndm,nurbm)      ! Street length
L01906       real ws_u(ndm,nurbm)        ! Street width
L01907       real z0g_u(nurbm)           ! The ground's roughness length
```

```
L01908        real z0r_u(nurbm)            ! The roof's roughness length
L01909        real ss_u(nz_um,nurbm)       ! The probability that a building has an height equal to "z"
L01910        real pb_u(nz_um,nurbm)       ! The probability that a building has an height greater or equal to "z"
L01911        real lp_urb                 ! Building plan area density
L01912        real lb_urb                 ! Building surface area to plan area ratio
L01913        real hgt_urb                ! Average building height weighted by building plan area [m]
L01914        real frc_urb                ! Urban fraction
L01915
L01916  ! ------------------------------------------------------------------------
L01917  ! OUTPUT:
L01918  ! ------------------------------------------------------------------------
L01919        real alag(ng_u)            ! Ground thermal diffusivity at each ground levels
L01920        real csg(ng_u)             ! Specific heat of the ground material at each ground levels
L01921        real bs(ndm)               ! Building width for the current urban class
L01922        real drst(ndm)             ! street directions for the current urban class
L01923        real strd(ndm)             ! Street lengths for the current urban class
L01924        real ws(ndm)               ! Street widths of the current urban class
L01925        real z0(ndm,nz_um)       ! Roughness lengths "profiles"
L01926        real ss(nz_um)           ! Probability to have a building with height h
L01927        real pb(nz_um)           ! Probability to have a building with an height greater or equal to "z"
L01928        integer nzurban
L01929
L01930  !-------------------------------------------------------------------------
L01931  !INPUT/OUTPUT
L01932  !-------------------------------------------------------------------------
L01933
L01934        real dzw(nwr_u)         !Layer sizes in the walls [m]
L01935        real dzr(nwr_u)         !Layer sizes in the roofs [m]
L01936        real dzf(nf_u)          !Layer sizes in the floors [m]
L01937        real dzgb(ngb_u)        !layer sizes in the ground below the buildings [m]
L01938
L01939        real csr(nwr_u)         ! Specific heat of the roof material at each roof levels
L01940        real csw(nwr_u)         ! Specific heat of the wall material at each wall levels
L01941
L01942        real csf(nf_u)          !Specific heat of the floors materials in the buildings
L01943                                !of the current urban class [J m^3 K^-1]
L01944        real csgb(ngb_u)        !Specific heat of the ground material below the buildings
L01945                                !of the current urban class [J m^3 K^-1]
L01946        real alar(nwr_u+1)      ! Roof thermal diffusivity at each roof levels [W/ m K]
L01947        real alaw(nwr_u+1)      ! Wall thermal diffusivity at each wall levels [W/ m K]
L01948        real alaf(nf_u+1)       ! Floor thermal diffusivity at each wall levels [W/m K]
L01949        real alagb(ngb_u+1)     ! Ground thermal diffusivity below the building at each wall levels [W/m K]
L01950        real bs_urb(ndm,nurbm)          ! Building width
L01951        real ws_urb(ndm,nurbm)          ! Street width
L01952        real ss_urb(nz_um,nurbm)        ! The probability that a building has an height equal to "z"
L01953        real pb_urb(nz_um)              ! Probability that a building has an height greater or equal to z
L01954  ! ------------------------------------------------------------------------
L01955  ! LOCAL:
L01956  ! ------------------------------------------------------------------------
L01957        integer id,ig,ir,iw,iz,iflo,ihu
L01958  ! ------------------------------------------------------------------------
L01959  ! END VARIABLES DEFINITIONS
L01960  ! ------------------------------------------------------------------------
L01961  !
L01962  !Initialize variables
L01963  !
L01964        ss=0.
L01965        pb=0.
L01966        csg=0.
L01967        alag=0.
L01968        csgb=0.
L01969        alagb=0.
L01970        csf=0.
L01971        alaf=0.
L01972        csr=0.
L01973        alar=0.
L01974        csw=0.
L01975        alaw=0.
L01976        z0=0.
L01977        ws=0.
L01978        bs=0.
L01979        bs_urb=0.
L01980        ws_urb=0.
L01981        strd=0.
L01982        drst=0.
L01983        nzurban=0
L01984
L01985  !Define the layer sizes in the walls
L01986
L01987        dzgb=(/0.2,0.12,0.08,0.05,0.03,0.02,0.02,0.01,0.005,0.0025/)
L01988        dzr=(/0.02,0.02,0.02,0.02,0.02,0.02,0.02,0.01,0.005,0.0025/)
L01989        dzw=(/0.02,0.02,0.02,0.02,0.02,0.02,0.02,0.01,0.005,0.0025/)
L01990        dzf=(/0.02,0.02,0.02,0.02,0.02,0.02,0.02,0.02,0.02,0.02/)
L01991
L01992         ihu=0
L01993
L01994         do iz=1,nz_um
L01995            if (ss_urb(iz,iurb)/=0.) then
L01996               ihu=1
L01997               exit
L01998            else
L01999               continue
L02000            endif
L02001         enddo
L02002
L02003         if (ihu==1) then
L02004            do iz=1,nzurb+1
L02005               ss(iz)=ss_urb(iz,iurb)
L02006               pb(iz)=pb_urb(iz)
L02007            enddo
L02008            nzurban=nzurb
L02009         else
L02010            do iz=1,nzu+1
L02011               ss(iz)=ss_u(iz,iurb)
L02012               pb(iz)=pb_u(iz,iurb)
L02013               ss_urb(iz,iurb)=ss_u(iz,iurb)
L02014               pb_urb(iz)=pb_u(iz,iurb)
L02015            end do
L02016            nzurban=nzu
```

```
L02017          endif
L02018
L02019         do ig=1,ngb_u
L02020           csgb(ig) = csg_u(iurb)
L02021           alagb(ig)= csg_u(iurb)*alag_u(iurb)
L02022         enddo
L02023         alagb(ngb_u+1)= csg_u(iurb)*alag_u(iurb)
L02024
L02025         do iflo=1,nf_u
L02026           csf(iflo) = csw_u(iurb)
L02027           alaf(iflo)= csw_u(iurb)*alaw_u(iurb)
L02028         enddo
L02029         alaf(nf_u+1)= csw_u(iurb)*alaw_u(iurb)
L02030
L02031         do ir=1,nwr_u
L02032           csr(ir) = csr_u(iurb)
L02033           alar(ir)= csr_u(iurb)*alar_u(iurb)
L02034         enddo
L02035         alar(nwr_u+1)= csr_u(iurb)*alar_u(iurb)
L02036
L02037         do iw=1,nwr_u
L02038           csw(iw) = csw_u(iurb)
L02039           alaw(iw)= csw_u(iurb)*alaw_u(iurb)
L02040         enddo
L02041         alaw(nwr_u+1)=csw_u(iurb)*alaw_u(iurb)
L02042
L02043   !-----------------------------------------------------------------------
L02044
L02045          do ig=1,ng_u
L02046           csg(ig)=csg_u(iurb)
L02047           alag(ig)=alag_u(iurb)
L02048          enddo
L02049
L02050          do id=1,ndu
L02051             z0(id,1)=z0g_u(iurb)
L02052           do iz=2,nzurban+1
L02053              z0(id,iz)=z0r_u(iurb)
L02054           enddo
L02055          enddo
L02056
L02057          do id=1,ndu
L02058             strd(id)=strd_u(id,iurb)
L02059             drst(id)=drst_u(id,iurb)
L02060          enddo
L02061
L02062          do id=1,ndu
L02063             if ((hgt_urb<=0.).OR.(lp_urb<=0.).OR.(lb_urb<=0.)) then
L02064                 ws(id)=ws_u(id,iurb)
L02065                 bs(id)=bs_u(id,iurb)
L02066                 bs_urb(id,iurb)=bs_u(id,iurb)
L02067                 ws_urb(id,iurb)=ws_u(id,iurb)
L02068             else if ((lp_urb/frc_urb<1.).and.(lp_urb<lb_urb)) then
L02069                     bs(id)=2.*hgt_urb*lp_urb/(lb_urb-lp_urb)
L02070                     ws(id)=2.*hgt_urb*lp_urb*((frc_urb/lp_urb)-1.)/(lb_urb-lp_urb)
L02071                     bs_urb(id,iurb)=bs(id)
L02072                     ws_urb(id,iurb)=ws(id)
L02073                  else
L02074                     ws(id)=ws_u(id,iurb)
L02075                     bs(id)=bs_u(id,iurb)
L02076                     bs_urb(id,iurb)=bs_u(id,iurb)
L02077                     ws_urb(id,iurb)=ws_u(id,iurb)
L02078             endif
L02079          enddo
L02080          do id=1,ndu
L02081             if ((bs(id)<=1.).OR.(bs(id)>=150.)) then
L02082                bs(id)=bs_u(id,iurb)
L02083                ws(id)=ws_u(id,iurb)
L02084                bs_urb(id,iurb)=bs_u(id,iurb)
L02085                ws_urb(id,iurb)=ws_u(id,iurb)
L02086             endif
L02087             if ((ws(id)<=1.).OR.(ws(id)>=150.)) then
L02088                ws(id)=ws_u(id,iurb)
L02089                bs(id)=bs_u(id,iurb)
L02090                bs_urb(id,iurb)=bs_u(id,iurb)
L02091                ws_urb(id,iurb)=ws_u(id,iurb)
L02092             endif
L02093          enddo
L02094          return
L02095          end subroutine param
```

讲解：① 墙/屋顶/楼板/楼下地基十层厚度数组在此定死：dzgb 从 0.2 m 递减到 0.0025 m（几何级数式浅表加密）、dzr/dzw=0.02 m 前七层 +0.01/0.005/0.0025、dzf 全 0.02——**想改墙体分层结构就改这四个数组**（与 14b surf_temp 的 dzg_u、roof_temp_veg 的 dzgr_u 同值不同源，三处必须一致）；②ss/pb 双来源：有格点直方图（ihu=1）用类谱 ss_urb，否则用 TBL 类谱 ss_u 并**反写**回 ss_urb；③ 材料热性质从类参数展开到各层（alagb=alag_u • csg_u 的热扩散率 =导热率/热容口径）；④ **街道几何反解**（L2062–2093）：给定 lp（规划面积密度）、lb（建筑表面积/规划面积）、hgt（平均高）、frc（城市面积比）时，bs=2 • hgt • lp/(lb−lp)、ws=2 • hgt • lp • (frc/lp−1)/(lb−lp)——论文 2 域设计里「FRC_URB2D+建筑高度场」自动生成峡谷几何走的就是这条；越界（<1 m 或 >150 m）回退 TBL 表值。

七、插值与平均单元

```
!file: phys/module_sf_bep_bem.F    L2100-L2148    (subroutine interpol)
L02100        subroutine interpol(kms,kme,kts,kte,nz_u,z,z_u,c,c_u)
L02101
L02102  ! ------------------------------------------------------------------
L02103  !  This routine interpolate para
L02104  !  meters from the "mesoscale grid" to
L02105  !  the "urban grid".
L02106  !  See p300 Appendix B.1 of the BLM paper.
L02107  ! ------------------------------------------------------------------
L02108
L02109        implicit none
L02110
L02111  ! ------------------------------------------------------------------
L02112  ! INPUT:
L02113  ! ------------------------------------------------------------------
L02114  ! Data relative to the "mesoscale grid"
L02115        integer kts,kte,kms,kme
L02116        real z(kms:kme)          ! Altitude of the cell interface
L02117        real c(kms:kme)           ! Parameter which has to be interpolated
L02118  ! Data relative to the "urban grid"
L02119        integer nz_u           ! Number of levels
L02120  !!    real z_u(nz_u+1)       ! Altitude of the cell interface
L02121        real z_u(nz_um)      ! Altitude of the cell interface
L02122
L02123  ! ------------------------------------------------------------------
L02124  ! OUTPUT:
L02125  ! ------------------------------------------------------------------
L02126  !!    real c_u(nz_u)         ! Interpolated paramters in the "urban grid"
L02127        real c_u(nz_um)         ! Interpolated paramters in the "urban grid"
L02128
L02129  ! LOCAL:
L02130  ! ------------------------------------------------------------------
L02131        integer iz_u,iz
L02132        real ctot,dz
L02133
L02134  ! ------------------------------------------------------------------
L02135  ! END VARIABLES DEFINITIONS
L02136  ! ------------------------------------------------------------------
L02137
L02138         do iz_u=1,nz_u
L02139          ctot=0.
L02140          do iz=kts,kte
L02141           dz=max(min(z(iz+1),z_u(iz_u+1))-max(z(iz),z_u(iz_u)),0.)
L02142           ctot=ctot+c(iz)*dz
L02143          enddo
L02144          c_u(iz_u)=ctot/(z_u(iz_u+1)-z_u(iz_u))
L02145         enddo
L02146
L02147         return
L02148         end subroutine interpol
```

讲解：模式网格 → 城市网格的质量守恒插值：对每个城市层，按两套网格的**重叠高度**加权平均（dz=max(min(z 上,z_u 上)−max(z 下,z_u 下),0)）——不是线性插值而是体积平均，保证薄模式层不会丢失城市层的物理。

```
!file: phys/module_sf_bep_bem.F    L2153-L2228    (subroutine averaging_temp)
L02153        subroutine  averaging_temp(tw,twlev,ss,pb,tw_av,twlev_av,       &
L02154                                   sfw_av,sfwind_av,sfw,sfwin)
L02155
L02156        implicit none
L02157  !
L02158  !INPUT VARIABLES
L02159  !
L02160        real tw(2*ndm,nz_um,nwr_u,nbui_max)        ! Temperature in each layer of the wall [K]
L02161        real twlev(2*ndm,nz_um,nbui_max)     ! Window temperature in BEM [K]
L02162        real pb(nz_um)                    ! Probability to have a building with an height equal or greater h
L02163        real ss(nz_um)                    ! Probability to have a building with height h
L02164        real sfw(2*ndm,nz_um,nbui_max)             ! Surface fluxes from the walls
L02165        real sfwin(2*ndm,nz_um,nbui_max)     ! Surface fluxes from the windows
L02166  !
L02167  !OUTPUT VARIABLES
L02168  !
L02169        real tw_av(2*ndm,nz_um)           ! Averaged temperature of the wall surfaces
L02170        real twlev_av(2*ndm,nz_um)        ! Averaged temperature of the windows
L02171        real sfw_av(2*ndm,nz_um)          ! Averaged sensible heat from walls
L02172        real sfwind_av(2*ndm,nz_um)       ! Averaged sensible heat from windows
L02173  !
L02174  !LOCAL VARIABLES
L02175  !
L02176        real d_urb(nz_um)
L02177        integer nlev(nz_um)
L02178        integer id,iz
L02179        integer nbui,ibui
L02180  !
L02181  !Initialize Variables
L02182  !
L02183        tw_av=0.
L02184        twlev_av=0.
L02185        sfw_av=0.
L02186        sfwind_av=0.
L02187        ibui=0
L02188        nbui=0
L02189        nlev=0
L02190        d_urb=0.
L02191
L02192        do iz=1,nz_um
L02193           if(ss(iz).gt.0) then
L02194             ibui=ibui+1
L02195             d_urb(ibui)=ss(iz)
L02196             nlev(ibui)=iz-1
L02197             nbui=ibui
```

```
L02198           endif
L02199        enddo
L02200
L02201        do id=1,ndm
L02202           do iz=1,nz_um-1
L02203              if (pb(iz+1).gt.0) then
L02204                 do ibui=1,nbui
L02205                    if (iz.le.nlev(ibui)) then
L02206                       tw_av(2*id-1,iz)=tw_av(2*id-1,iz)+(d_urb(ibui)/pb(iz+1))*&
L02207                                        tw(2*id-1,iz,nwr_u,ibui)**4
L02208                       tw_av(2*id,iz)=tw_av(2*id,iz)+(d_urb(ibui)/pb(iz+1))*&
L02209                                      tw(2*id,iz,nwr_u,ibui)**4
L02210                       twlev_av(2*id-1,iz)=twlev_av(2*id-1,iz)+(d_urb(ibui)/pb(iz+1))*&
L02211                                           twlev(2*id-1,iz,ibui)**4
L02212                       twlev_av(2*id,iz)=twlev_av(2*id,iz)+(d_urb(ibui)/pb(iz+1))*&
L02213                                         twlev(2*id,iz,ibui)**4
L02214                       sfw_av(2*id-1,iz)=sfw_av(2*id-1,iz)+(d_urb(ibui)/pb(iz+1))*sfw(2*id-1,iz,ibui)
L02215                       sfw_av(2*id,iz)=sfw_av(2*id,iz)+(d_urb(ibui)/pb(iz+1))*sfw(2*id,iz,ibui)
L02216                       sfwind_av(2*id-1,iz)=sfwind_av(2*id-1,iz)+(d_urb(ibui)/pb(iz+1))*sfwin(2*id-1,iz,ibui)
L02217                       sfwind_av(2*id,iz)=sfwind_av(2*id,iz)+(d_urb(ibui)/pb(iz+1))*sfwin(2*id,iz,ibui)
L02218                    endif
L02219                 enddo
L02220                 tw_av(2*id-1,iz)=tw_av(2*id-1,iz)**(1./4.)
L02221                 tw_av(2*id,iz)=tw_av(2*id,iz)**(1./4.)
L02222                 twlev_av(2*id-1,iz)=twlev_av(2*id-1,iz)**(1./4.)
L02223                 twlev_av(2*id,iz)=twlev_av(2*id,iz)**(1./4.)
L02224              endif
L02225           enddo !iz
L02226        enddo !id
L02227        return
L02228        end subroutine averaging_temp
```

讲解：多建筑型的墙面状态平均：每个高度层按 d_urb/pb 加权累加 T^4 与热流，最后开四次方还原温度——「先辐射量平均再还原」的物理正确做法；pb(iz+1)=0（该层无墙）时跳过，tw_av 保持 0（上游不使用）。

八、辐射调制与表面温度

```
!file: phys/module_sf_bep_bem.F    L2232-L2321    (subroutine modif_rad)
L02232        subroutine modif_rad(iurb,nd,nz_u,z,ws,drst,strd,ss,pb,    &
L02233                             tw,tg_av,twlev,albg,albw,emw,emg,pwin,albwin,   &
L02234                             emwin,fww,fwg,fgw,fsw,fsg,            &
L02235                             zr,deltar,ah,xlat,swddir,swddif,          &
L02236                             rs,rl,rsw,rsd,rsg,rlw,rlg)
L02237
L02238  ! ---------------------------------------------------------------------
L02239  ! This routine computes the modification of the short wave and
L02240  !  long wave radiation due to the buildings.
L02241  ! ---------------------------------------------------------------------
L02242
L02243        implicit none
L02244
L02245
L02246  ! ---------------------------------------------------------------------
L02247  ! INPUT:
L02248  ! ---------------------------------------------------------------------
L02249        integer iurb              ! current urban class
L02250        integer nd                ! Number of street direction for the current urban class
L02251        integer nz_u              ! Number of layer in the urban grid
L02252        real z(nz_um)           ! Height of the urban grid levels
L02253        real ws(ndm)              ! Street widths of the current urban class
L02254        real drst(ndm)            ! street directions for the current urban class
L02255        real strd(ndm)            ! Street lengths for the current urban class
L02256        real ss(nz_um)          ! probability to have a building with height h
L02257        real pb(nz_um)          ! probability to have a building with an height equal
L02258        real tw(2*ndm,nz_um)    ! Temperature in each layer of the wall [K]
L02259        real tg_av(ndm)         ! Temperature in each layer of the ground [K]
L02260        real albg                 ! Albedo of the ground for the current urban class
L02261        real albw                 ! Albedo of the wall for the current urban class
L02262        real emg                  ! Emissivity of ground for the current urban class
L02263        real emw                  ! Emissivity of wall for the current urban class
L02264        real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
L02265        real fsg(ndm,nurbm)             ! View factors from sky to ground
L02266        real fsw(nz_um,ndm,nurbm)       ! View factors from sky to wall
L02267        real fws(nz_um,ndm,nurbm)       ! View factors from wall to sky
L02268        real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
L02269        real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
L02270        real ah                   ! Hour angle (it should come from the radiation routine)
L02271        real zr                   ! zenith angle
L02272        real deltar               ! Declination of the sun
L02273        real rs                   ! solar radiation
L02274        real rl                   ! downward flux of the longwave radiation
L02275        real xlat                 ! latitudine
L02276        real swddir               ! short wave direct solar radiation  _gl
L02277        real swddif               ! short wave diffuse solar radiation _gl
L02278
L02279  !
L02280  !New variables BEM
L02281  !
L02282        real twlev(2*ndm,nz_um)         ! Window temperature in BEM [K]
L02283        real pwin                       ! Coverage area fraction of windows in the walls of the buildings
L02284        real albwin                     ! Albedo of the windows for the current urban class
L02285        real emwin                      ! Emissivity of the windows for the current urban class
L02286        real alb_av                     ! Averaged albedo (window and wall)
L02287  ! ---------------------------------------------------------------------
L02288  ! OUTPUT:
L02289  ! ---------------------------------------------------------------------
L02290        real rlg(ndm)             ! Long wave radiation at the ground
L02291        real rlw(2*ndm,nz_um)     ! Long wave radiation at the walls
L02292        real rsg(ndm)             ! Short wave radiation at the ground
```

```
L02293        real rsw(2*ndm,nz_um)     ! Short wave radiation at the walls
L02294        real rsd(2*ndm,nz_um)     ! Direct Short wave radiation at the walls
L02295
L02296  ! ----------------------------------------------------------------------
L02297  ! LOCAL:
L02298  ! ----------------------------------------------------------------------
L02299
L02300        integer id,iz
L02301
L02302  !  Calculation of the shadow effects
L02303
L02304        call shadow_mas(nd,nz_u,zr,deltar,ah,drst,ws,ss,pb,z,        &
L02305                        swddir,rsw,rsg,xlat)
L02306        rsd=rsw
L02307
L02308  ! Calculation of the reflection effects
L02309        do id=1,nd
L02310           call long_rad(iurb,nz_u,id,emw,emg,emwin,pwin,twlev,      &
L02311                         fwg,fww,fgw,fsw,fsg,tg_av,tw,rlg,rlw,rl,pb)
L02312
L02313           alb_av=pwin*albwin+(1.-pwin)*albw
L02314
L02315          call short_rad_dd(iurb,nz_u,id,alb_av,                      &
L02316                            albg,swddif,fwg,fww,fgw,fsw,fsg,rsg,rsw,pb)
L02317
L02318
L02319        enddo
L02320        return
L02321        end subroutine modif_rad
```

讲解：辐射调制总调度：先 shadow_mas（直射阴影，14b）得各方向墙面/地面直射；rsd=rsw 存副本；再逐方向调 long_rad（长波多次反射线性系统）与 short_rad_dd（短波多次反射），墙的反射率/发射率按窗户占比混合 alb_av=pwin • albwin+(1−pwin) • albw。

```
!file: phys/module_sf_bep_bem.F    L2328-L2412    (subroutine surf_temp)
L02328        subroutine surf_temp(nd,pr,dt,rl,rsg,rlg,                  &
L02329                             tg,alag,csg,emg,albg,ptg,sfg,lfg,gfg)
L02330
L02331  ! ----------------------------------------------------------------------
L02332  ! Computation of the surface temperatures for walls, ground and roofs
L02333  ! ----------------------------------------------------------------------
L02334
L02335        implicit none
L02336
L02337  ! ----------------------------------------------------------------------
L02338  ! INPUT:
L02339  ! ----------------------------------------------------------------------
L02340
L02341        integer nd                ! Number of street direction for the current urban class
L02342        real alag(ng_u)           ! Ground thermal diffusivity for the current urban class [m^2 s^-1]
L02343
L02344        real albg                 ! Albedo of the ground for the current urban class
L02345
L02346        real csg(ng_u)            ! Specific heat of the ground material of the current urban class [J m^3 K^-1]
L02347
L02348        real dt                   ! Time step
L02349        real emg                  ! Emissivity of ground for the current urban class
L02350
L02351        real pr(nz_um)            ! Air pressure
L02352
L02353        real rl                   ! Downward flux of the longwave radiation
L02354        real rlg(ndm)             ! Long wave radiation at the ground
L02355
L02356        real rsg(ndm)             ! Short wave radiation at the ground
L02357
L02358        real sfg(ndm)             ! Sensible heat flux from ground (road)
L02359
L02360        real lfg(ndm)             ! Latent heat flux from ground (road)
L02361
L02362        real gfg(ndm)             ! Heat flux transferred from the surface of the ground (road) toward the interior
L02363
L02364        real tg(ndm,ng_u)         ! Temperature in each layer of the ground [K]
L02365
L02366  ! ----------------------------------------------------------------------
L02367  ! OUTPUT:
L02368  ! ----------------------------------------------------------------------
L02369        real ptg(ndm)             ! Ground potential temperatures
L02370
L02371  ! ----------------------------------------------------------------------
L02372  ! LOCAL:
L02373  ! ----------------------------------------------------------------------
L02374        integer id,ig,ir,iw,iz
L02375
L02376        real rtg(ndm)             ! Total radiation at ground(road) surface (solar+incoming long+outgoing long)
L02377
L02378        real tg_tmp(ng_u)
L02379
L02380        real dzg_u(ng_u)          ! Layer sizes in the ground
L02381
L02382        data dzg_u /0.2,0.12,0.08,0.05,0.03,0.02,0.02,0.01,0.005,0.0025/
L02383
L02384
L02385  ! ----------------------------------------------------------------------
L02386  ! END VARIABLES DEFINITIONS
L02387  ! ----------------------------------------------------------------------
L02388
L02389
L02390
L02391        do id=1,nd
L02392
L02393  !      Calculation for the ground surfaces
L02394         do ig=1,ng_u
L02395          tg_tmp(ig)=tg(id,ig)
L02396         end do
L02397  !
L02398  !       print*,'alag','cs',alag(1),csg(1)
```

```
L02399
L02400         call soil_temp(ng_u,dzg_u,tg_tmp,ptg(id),alag,csg,      &
L02401                        rsg(id),rlg(id),pr(1),                   &
L02402                        dt,emg,albg,                             &
L02403                        rtg(id),sfg(id),lfg(id),gfg(id))
L02404
L02405         do ig=1,ng_u
L02406          tg(id,ig)=tg_tmp(ig)
L02407         end do
L02408
L02409        end do !id
L02410
L02411        return
L02412        end subroutine surf_temp
```

讲解：路面温度：逐方向调 soil_temp（14b 的通用一维导热求解器），dzg_u 十层几何分层（0.2→0.0025 m，与 param 的 dzgb 同值）；返回路面位温 ptg 与感热/潜热/向内热通量。

```
!file: phys/module_sf_bep_bem.F    L2419-L2592    (subroutine roof_temp_veg)
L02419        subroutine roof_temp_veg(nd,pr,dt,rl,rsr,                &
L02420                                 trv,ptrv,sfrv,lfrv,gfr,qr,rainbl,drain,hfgroof,tr,alar,dzr,csr,nzu,irri_now,gr_type,
↪  pv_frac_roof,tpvlev)
L02421
L02422  ! ----------------------------------------------------------------------
L02423  ! Computation of the surface temperatures for walls, ground and roofs
L02424  ! ----------------------------------------------------------------------
L02425
L02426        implicit none
L02427
L02428  ! ----------------------------------------------------------------------
L02429  ! INPUT:
L02430  ! ----------------------------------------------------------------------
L02431        real rainbl
L02432        integer nd                ! Number of street direction for the current urban class
L02433
L02434        integer nzu                ! Number of urban layers
L02435        real irho(24)               ! Which hour of irrigation\
L02436
L02437
L02438        real alar           ! Roof thermal diffusivity for the current urban class [m^2 s^-1]
L02439        real pv_frac_roof
L02440        real csr
L02441
L02442        real dzr          ! Layer sizes in the roofs [m]
L02443
L02444        real dt                   ! Time step
L02445
L02446        real pr(nz_um)            ! Air pressure
L02447
L02448        real rl                   ! Downward flux of the longwave radiation
L02449
L02450        real rsr                 ! Short wave radiation at the ground
L02451
L02452       real tpvlev(ndm,nz_um)
L02453
L02454        real sfrv(ndm,nz_um)             ! Sensible heat flux from ground (road)
L02455
L02456        real lfrv(ndm,nz_um)             ! Latent heat flux from ground (road)
L02457
L02458        real gfr(ndm,nz_um)             ! Heat flux transferred from the surface of the ground (road) toward the interior
L02459
L02460        real trv(ndm,nz_um,ngr_u)         ! Temperature in each layer of the green roof [K]
L02461
L02462        real qr(ndm,nz_um,ngr_u)         ! Humidity in each layer of the green roof
L02463
L02464        real tr(ndm,nz_um,nwr_u)         !Roof temperature in BEM [K]
L02465
L02466  ! ----------------------------------------------------------------------
L02467  ! OUTPUT:
L02468  ! ----------------------------------------------------------------------
L02469        real ptrv(ndm,nz_um)             ! Ground potential temperatures
L02470
L02471        real hfgroof(ndm,nz_um)
L02472  ! ----------------------------------------------------------------------
L02473  ! LOCAL:
L02474  ! ----------------------------------------------------------------------
L02475        integer id,ig,ir,iw,iz
L02476
L02477        real alagr(ngr_u)           ! Green Roof thermal diffusivity for the current urban class [m^2 s^-1]
L02478
L02479        real rtr(ndm,nz_um)              ! Total radiation at ground(road) surface (solar+incoming long+outgoing long)
L02480
L02481        real tr_tmp(ngr_u)
L02482
L02483        real qr_tmp(ngr_u)
L02484        real qr_tmp_old(ngr_u)
L02485        real dzgr_u(ngr_u)           ! Layer sizes in the green roof
L02486  !MODIFICA
L02487        data dzgr_u /0.1,0.003,0.06,0.003,0.05,0.04,0.02,0.0125,0.005,0.0025/
L02488        real cs(ngr_u)  ! Specific heat of the ground material
L02489        real cw
L02490        parameter(cw=4.295e6)
L02491        real s(ngr_u)
L02492        real d(ngr_u)
L02493        real k(ngr_u)
L02494        real qr_m     ! mean soil moisture between layers
L02495        real qrmax(ngr_u)
L02496        real smax(ngr_u)
L02497        real kmax(ngr_u)
L02498        real b(ngr_u)
L02499        real cd(ngr_u)
L02500        real csa(4)
L02501        real ka(4)
L02502        real qref
L02503        parameter(qref=0.37)
L02504        data qrmax /0.0,0.0,0.0,0.0,0.439,0.37,0.37,0.37,0.37,0.37/
```

```
L02505        data smax /0,0,0,0,-0.01,-0.1,-0.1,-0.1,-0.1,-0.1/
L02506        data kmax /0,0,0,0,3.32e-3,2.162e-3,2.162e-3,2.162e-3,2.162e-3,2.162e-3/
L02507        data b /0,0,0,0,2.7,3.9,3.9,3.9,3.9,3.9/
L02508        data cd /0,0,0,0,331500,1.342e6,1.342e6,1.342e6,1.342e6,1.342e6/
L02509        data csa /7.5e4,2.1e6,4.48e4,2.1e6/
L02510        data ka /0.035,0.7,0.024,0.7/
L02511        real em_gr(1)
L02512        real alb_gr(1)
L02513        real irri_now
L02514        integer gr_type
L02515        real drain(ndm,nz_um)
L02516   ! ------------------------------------------------------------------------
L02517   ! END VARIABLES DEFINITIONS
L02518
L02519        if(gr_type.eq.1)then
L02520        em_gr=0.95
L02521        alb_gr=0.3
L02522        elseif(gr_type.eq.2)then
L02523         em_gr=0.83
L02524         alb_gr=0.154
L02525        endif
L02526
L02527
L02528        do iz=2,nzu
L02529
L02530        do id=1,nd
L02531
L02532
L02533
L02534
L02535   !      Calculation for the ground surfaces
L02536
L02537         do ig=1,ngr_u
L02538          tr_tmp(ig)=trv(id,iz,ig)
L02539          qr(id,iz,ig) = max(qr(id,iz,ig),1e-6) !cenlin, 11/4/2020
L02540          qr_tmp(ig)=qr(id,iz,ig)
L02541          qr_tmp_old(ig)=qr(id,iz,ig)
L02542
L02543        if(ig.le.4) then
L02544
L02545         cs(ig)=csa(ig)
L02546         alagr(ig)=ka(ig)/csa(ig)
L02547
L02548        else
L02549
L02550
L02551          if (ig.gt.5) then
L02552          qr_m=(qr(id,iz,ig)*dzgr_u(ig-1)+qr(id,iz,ig-1)*dzgr_u(ig))/(dzgr_u(ig)+dzgr_u(ig-1))
L02553          else
L02554          qr_m=qr(id,iz,ig)
L02555          endif
L02556          cs(ig)=(1-qr_m)*cd(ig)+qr_m*cw
L02557          s(ig)=smax(ig)*(qrmax(ig)/qr_m)**b(ig)
L02558          k(ig)=kmax(ig)*(qr_m/qrmax(ig))**(2*b(ig)+3)
L02559          d(ig)=-b(ig)*kmax(ig)*smax(ig)*((qr_m/qrmax(ig))**(b(ig)+3))/qr_m
L02560          if (log10(abs(s(ig))).le.5.1) then
L02561            alagr(ig)=exp(-(log10(abs(s(ig)))+2.7))*4.186e2/cs(ig)
L02562          endif
L02563          if (log10(abs(s(ig))).gt.5.1) then
L02564            alagr(ig)=0.00041*4.186e2/cs(ig)
L02565          endif
L02566
L02567         endif
L02568
L02569          end do
L02570          hfgroof(id,iz)=(alar/csr+alagr(1))*(tr_tmp(1)-tr(id,iz,5))/(dzr+dzgr_u(1))
L02571
L02572         call soil_temp_veg(hfgroof(id,iz),ngr_u,dzgr_u,tr_tmp,ptrv(id,iz),alagr,cs,      &
L02573                      rsr,rl,pr(iz),                    &
L02574                      dt,em_gr(1),alb_gr(1),                        &
L02575                      rtr(id,iz),sfrv(id,iz),lfrv(id,iz),gfr(id,iz),pv_frac_roof,tpvlev(id,iz))
L02576         do ig=1,ngr_u
L02577          trv(id,iz,ig)=tr_tmp(ig)
L02578         end do
L02579          drain(id,iz)=kmax(5)*(qr(id,iz,5)/qrmax(5))**(2*b(5)+3)
L02580          call soil_moist(ngr_u,dzgr_u,qr_tmp,dt,lfrv(id,iz),d,k,rainbl,drain(id,iz),irri_now)
L02581
L02582
L02583          do ig=1,ngr_u
L02584            ! qr(id,iz,ig)=min(qr_tmp(ig),qrmax(ig))
L02585            qr(id,iz,ig)=max(min(qr_tmp(ig),qrmax(ig)),1e-6) !cenlin,11/4/2020
L02586           end do
L02587
L02588        end do !id
L02589        end do !iz
L02590
L02591        return
L02592        end subroutine roof_temp_veg
```

讲解：**绿屋顶土壤-植被耦合**（gr_flag=1 且 gr_frac_roof>0 才调用）：① 十层结构前 4 层是屋顶建材（csa/ka 四组值），5 层起是土壤（qrmax/smax/kmax/b/cd 五组 van Genucht 型水力参数）；② 土壤导热率含水量依赖：cs=(1−qr)•cd+qr•cw（cw=4.295e6 水的热容），热扩散率由吸力 s 的对数分区给出；③ 与下层普通屋顶的界面热交换 hfgroof=(alar/csr+alagr(1))•(tr_tmp(1)−tr(...,5))/(dzr+dzgr_u(1))——绿屋顶与结构屋顶的耦合项；④ 调 soil_temp_veg（含 PV 面积调整的辐射，14b）更新温度，soil_moist（Richards 型含水量扩散 +降雨 +灌溉 +排水，14b）更新含水量，drain=kmax(5)•(qr/qrmax)^(2b+3) 是底部排水；⑤qr 下限 1e-6（cenlin 2020-11-04 补丁，防对数发散）。gr_type=1/2 两套植被参数（发射率 0.95/0.83、反照率 0.3/0.154、rsmin 40/150、LAI 2/3）——**改绿屋顶参数改这里**。

九、源项组装 buildings（全文）

```
!file: phys/module_sf_bep_bem.F    L2597-L2891    (subroutine buildings)
L02597        subroutine buildings(iurb,nd,nz,z0,cdrag,ua_u,va_u,pt_u,pt0_u,       &
L02598                             ptg,ptr,ptrv,da_u,qv_u,pr_u,tmp_u,ptw,ptwin,pwin,                &
L02599                             drst,uva_u,vva_u,uvb_u,vvb_u,                    &
L02600                             tva_u,tvb_u,evb_u,qvb_u,qhb_u,                   &
L02601                             uhb_u,vhb_u,thb_u,ehb_u,ss,dt,sfw,sfg,sfr,sfrpv,sfrv,lfrv,   &
L02602                             dgr,dg,lfr,lfg,                                  &
L02603                             sfwin,pb,bs_u,dz_u,sflev,lflev,sfvlev,lfvlev,tvb_ac,ix,iy,rsg,rs,qr,gr_frac_roof,  &
L02604                             pv_frac_roof,gr_flag,gr_type)
L02605
L02606  ! ------------------------------------------------------------------------
L02607  ! This routine computes the sources or sinks of the different quantities
L02608  ! on the urban grid. The actual calculation is done in the subroutines
L02609  ! called flux_wall, and flux_flat.
L02610  ! ------------------------------------------------------------------------
L02611
L02612        implicit none
L02613
L02614
L02615  ! ------------------------------------------------------------------------
L02616  ! INPUT:
L02617  ! ------------------------------------------------------------------------
L02618        integer nd                ! Number of street direction for the current urban class
L02619        integer ix,iy
L02620        integer nz                ! number of vertical space steps
L02621        real ua_u(nz_um)          ! Wind speed in the x direction on the urban grid
L02622        real va_u(nz_um)          ! Wind speed in the y direction on the urban grid
L02623        real da_u(nz_um)          ! air density on the urban grid
L02624        real qv_u(nz_um)          ! specific humidity on the urban grid
L02625        real pr_u(nz_um)          ! pressure on the urban grid
L02626        real tmp_u(nz_um)         ! temperaure on the urban grid
L02627        real drst(ndm)            ! Street directions for the current urban class
L02628        real dz
L02629        real pt_u(nz_um)          ! Potential temperature on the urban grid
L02630        real pt0_u(nz_um)         ! reference potential temperature on the urban grid
L02631        real ptg(ndm)             ! Ground potential temperatures
L02632        real ptr(ndm,nz_um)       ! Roof potential temperatures
L02633        real ptrv(ndm,nz_um)      ! Green Roof potential temperatures
L02634        real ptw(2*ndm,nz_um,nbui_max)     ! Walls potential temperatures
L02635        real ss(nz_um)            ! probability to have a building with height h
L02636        real pb(nz_um)
L02637        real cdrag(nz_um)
L02638        real z0(ndm,nz_um)        ! Roughness lengths "profiles"
L02639        real dt ! time step
L02640        integer iurb              !Urban class
L02641        real rsg(ndm)             ! Solar Radiation
L02642        real rs                  ! Solar Radiation
L02643        real qr(ndm,nz_um,ngr_u)         ! Ground Soil Moisture
L02644        real trv(ndm,nz_um,ngr_u)         ! Ground Soil Moisture
L02645        real roof_frac
L02646        real road_frac
L02647  !
L02648  !New variables (BEM)
L02649  !
L02650        real bs_u(ndm,nurbm)    ! Building width [m]
L02651        real dz_u               ! Urban grid resolution
L02652        real sflev(nz_um,nz_um)     ! sensible heat flux due to the air conditioning systems  [W]
L02653        real lflev(nz_um,nz_um)     ! latent heat flux due to the air conditioning systems  [W]
L02654        real sfvlev(nz_um,nz_um)    ! sensible heat flux due to ventilation [W]
L02655        real lfvlev(nz_um,nz_um)    ! latent heat flux due to ventilation [W]
L02656        real qvb_u(2*ndm,nz_um)
L02657        real qhb_u(ndm,nz_um)
L02658        real ptwin(2*ndm,nz_um,nbui_max)  ! window potential temperature
L02659        real pwin
L02660        real tvb_ac(2*ndm,nz_um)
L02661        real gr_frac_roof
L02662        real pv_frac_roof
L02663        integer gr_flag,gr_type
L02664
L02665  ! ------------------------------------------------------------------------
L02666  ! OUTPUT:
L02667  ! ------------------------------------------------------------------------
L02668  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L02669  ! vertical surfaces (walls) and horizontal surfaces (roofs and street)
L02670  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L02671  !  Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L02672
L02673        real uhb_u(ndm,nz_um)   ! U (wind component) Horizontal surfaces, B (explicit) term
L02674        real uva_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, A (implicit) term
L02675        real uvb_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, B (explicit) term
L02676        real vhb_u(ndm,nz_um)   ! V (wind component) Horizontal surfaces, B (explicit) term
L02677        real vva_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, A (implicit) term
L02678        real vvb_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, B (explicit) term
L02679        real thb_u(ndm,nz_um)   ! Temperature        Horizontal surfaces, B (explicit) term
L02680        real tva_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, A (implicit) term
L02681        real tvb_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, B (explicit) term
L02682        real ehb_u(ndm,nz_um)   ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L02683        real evb_u(2*ndm,nz_um)   ! Energy (TKE)         Vertical surfaces, B (explicit) term
L02684        real uhb(2*ndm,nz_um)
L02685        real vhb(2*ndm,nz_um)
L02686        real ehb(2*ndm,nz_um)
L02687        real sfw(2*ndm,nz_um,nbui_max)   ! sensible heat flux from walls
L02688        real sfwin(2*ndm,nz_um,nbui_max) ! sensible heat flux form windows
L02689        real sfr(ndm,nz_um)           ! sensible heat flux from roof
L02690        real sfrv(ndm,nz_um)           ! sensible heat flux from roof
L02691        real lfrv(ndm,nz_um)           ! Latent heat flux from roof
L02692        real dgr(ndm,nz_um)           ! sensible heat flux from roof
L02693        real dg(ndm)
L02694        real lfr(ndm,nz_um)           ! Latent heat flux from roof
L02695        real lfg(ndm)                 ! Latent heat flux from street
L02696        real sfrpv(ndm,nz_um)         ! sensible heat flux from PV panels
L02697        real sfg(ndm)                 ! sensible heat flux from street
L02698
```

```
L02699
L02700  ! ------------------------------------------------------------------
L02701  ! LOCAL:
L02702  ! ------------------------------------------------------------------
L02703        real d_urb(nz_um)
L02704        real uva_tmp
L02705        real vva_tmp
L02706        real uvb_tmp
L02707        real vvb_tmp
L02708        real evb_tmp
L02709        integer nlev(nz_um)
L02710        integer id,iz,ibui,nbui,il
L02711        real wfg     !Ground water pool fraction
L02712        real wfr     !Roof water pool fraction
L02713        real uhbv(2*ndm,nz_um)
L02714        real vhbv(2*ndm,nz_um)
L02715        real ehbv(2*ndm,nz_um)
L02716        real z0v     !Vegetation roughness
L02717        parameter(z0v=0.01)
L02718        real resg
L02719        real rsveg
L02720        real f1,f2,f3,f4
L02721        integer rsv(2)
L02722        real qr_tmp(ngr_u)
L02723        data rsv /0,1/
L02724        real fh,ric,utot
L02725  !-----------------------------------------------------------------
L02726  ! END VARIABLES DEFINITIONS
L02727  ! ------------------------------------------------------------------------
L02728        dz=dz_u
L02729        ibui=0
L02730        nbui=0
L02731        nlev=0
L02732        d_urb=0.
L02733
L02734        uva_u=0.
L02735        uvb_u=0.
L02736        vhb_u=0.
L02737        vva_u=0.
L02738        vvb_u=0.
L02739        thb_u=0.
L02740        tva_u=0.
L02741        tvb_u=0.
L02742        tvb_ac=0.
L02743        ehb_u=0.
L02744        evb_u=0.
L02745        qvb_u=0.
L02746        qhb_u=0.
L02747
L02748        uhb=0.
L02749        vhb=0.
L02750        ehb=0.
L02751        uhbv=0.
L02752        vhbv=0.
L02753        ehbv=0.
L02754
L02755
L02756        do iz=1,nz_um
L02757           if(ss(iz).gt.0)then
L02758             ibui=ibui+1
L02759             d_urb(ibui)=ss(iz)
L02760             nlev(ibui)=iz-1
L02761             nbui=ibui
L02762           endif
L02763        enddo
L02764
L02765  !        Calculation at the ground surfaces
L02766        do id=1,nd
L02767
L02768            call flux_flat(dz,z0(id,1),ua_u(1),va_u(1),pt_u(1),pt0_u(1),  &
L02769                        ptg(id),qv_u(1),uhb(id,1),                        &
L02770                        vhb(id,1),sfg(id),lfg(id),ehb(id,1),da_u(1),pr_u(1))
L02771            if(dg(id).gt.0)then
L02772             wfg=dg(id)/dgmax
L02773             lfg(id)=-da_u(1)*latent*(-(wfg*lfg(id))/(da_u(1)*latent))
L02774            else
L02775             qhb_u(id,1)=0.
L02776             lfg(id)=0.
L02777            endif
L02778           thb_u(id,1)=-(sfg(id))/(da_u(1)*cp_u)
L02779           vhb_u(id,1)=vhb(id,1)
L02780           uhb_u(id,1)=uhb(id,1)
L02781           ehb_u(id,1)=ehb(id,1)
L02782           qhb_u(id,1)=-lfg(id)/(da_u(1)*latent)
L02783           do iz=2,nz
L02784              if(ss(iz).gt.0)then
L02785
L02786                  call flux_flat(dz,z0(id,iz),ua_u(iz),&
L02787                          va_u(iz),pt_u(iz),pt0_u(iz), &
L02788                          ptr(id,iz),qv_u(iz),uhb(id,iz),       &
L02789                          vhb(id,iz),sfr(id,iz),lfr(id,iz),ehb(id,iz),da_u(iz),pr_u(iz))
L02790           if(dgr(id,iz).gt.0)then
L02791            wfr=dgr(id,iz)/drmax
L02792            lfr(id,iz)=-da_u(iz)*latent*(-(wfr*lfr(id,iz))/(da_u(iz)*latent))
L02793           else
L02794            lfr(id,iz)=0.
L02795           endif
L02796           if(gr_flag.eq.1.and.gr_frac_roof.gt.0.)then
L02797           do il=1,ngr_u
L02798             qr_tmp(il)=qr(id,iz,il)
L02799           enddo
L02800                 call flux_flat_roof(dz,z0v,ua_u(iz),va_u(iz),pt_u(iz),pt0_u(iz),  &
L02801                         ptrv(id,iz),uhbv(id,iz),                          &
L02802                         vhbv(id,iz),sfrv(id,iz),lfrv(id,iz),ehbv(id,iz),da_u(iz),qv_u(iz),pr_u(iz),rs,qr_tmp,resg,rsveg,
↪  f1,f2,f3,f4,gr_type,pv_frac_roof)
L02803           sfr(id,iz)=sfr(id,iz)+pv_frac_roof*sfrpv(id,iz)
L02804           thb_u(id,iz)=-((1.-gr_frac_roof)*sfr(id,iz)+gr_frac_roof*sfrv(id,iz))/(da_u(iz)*cp_u)
L02805           vhb_u(id,iz)=(1.-gr_frac_roof)*vhb(id,iz)+gr_frac_roof*vhbv(id,iz)
```

```
L02806           uhb_u(id,iz)=(1.-gr_frac_roof)*uhb(id,iz)+gr_frac_roof*uhbv(id,iz)
L02807           ehb_u(id,iz)=(1.-gr_frac_roof)*ehb(id,iz)+gr_frac_roof*ehbv(id,iz)
L02808           qhb_u(id,iz)=-(gr_frac_roof*lfrv(id,iz)+(1.-gr_frac_roof)*lfr(id,iz))/(da_u(iz)*latent)
L02809           sfr(id,iz)=sfr(id,iz)-pv_frac_roof*sfrpv(id,iz)
L02810           else
L02811           sfr(id,iz)=sfr(id,iz)+pv_frac_roof*sfrpv(id,iz)
L02812           thb_u(id,iz)=-sfr(id,iz)/(da_u(iz)*cp_u)
L02813           vhb_u(id,iz)=vhb(id,iz)
L02814           uhb_u(id,iz)=uhb(id,iz)
L02815           ehb_u(id,iz)=ehb(id,iz)
L02816           qhb_u(id,iz)=-lfr(id,iz)/(da_u(iz)*latent)
L02817           sfr(id,iz)=sfr(id,iz)-pv_frac_roof*sfrpv(id,iz)
L02818           endif
L02819              else
L02820                 uhb_u(id,iz) = 0.0
L02821                 vhb_u(id,iz) = 0.0
L02822                 thb_u(id,iz) = 0.0
L02823                 ehb_u(id,iz) = 0.0
L02824                 qhb_u(id,iz) = 0.0
L02825              endif
L02826           enddo
L02827
L02828
L02829
L02830  !        Calculation at the wall surfaces
L02831
L02832           do ibui=1,nbui
L02833           do iz=1,nlev(ibui)
L02834
L02835              call flux_wall(ua_u(iz),va_u(iz),pt_u(iz),da_u(iz),            &
L02836                          ptw(2*id-1,iz,ibui),ptwin(2*id-1,iz,ibui),         &
L02837                          uva_tmp,vva_tmp,                                   &
L02838                          uvb_tmp,vvb_tmp,                                   &
L02839                          sfw(2*id-1,iz,ibui),sfwin(2*id-1,iz,ibui),         &
L02840                          evb_tmp,drst(id),dt,cdrag(iz))
L02841
L02842              if (pb(iz+1).gt.0.) then
L02843
L02844                      uva_u(2*id-1,iz)=uva_u(2*id-1,iz)+d_urb(ibui)/pb(iz+1)*uva_tmp
L02845                      vva_u(2*id-1,iz)=vva_u(2*id-1,iz)+d_urb(ibui)/pb(iz+1)*vva_tmp
L02846                      uvb_u(2*id-1,iz)=uvb_u(2*id-1,iz)+d_urb(ibui)/pb(iz+1)*uvb_tmp
L02847                      vvb_u(2*id-1,iz)=vvb_u(2*id-1,iz)+d_urb(ibui)/pb(iz+1)*vvb_tmp
L02848                      evb_u(2*id-1,iz)=evb_u(2*id-1,iz)+d_urb(ibui)/pb(iz+1)*evb_tmp
L02849                      tvb_u(2*id-1,iz)=tvb_u(2*id-1,iz)-(d_urb(ibui)/pb(iz+1))*                    &
L02850                                       (sfw(2*id-1,iz,ibui)*(1.-pwin)+sfwin(2*id-1,iz,ibui)*pwin)/     &
L02851                                      da_u(iz)/cp_u-(1./4.)*(d_urb(ibui)/pb(iz+1))*(sfvlev(iz,ibui)-sflev(iz,ibui))/&
L02852                                       (dz*bs_u(id,iurb))/da_u(iz)/cp_u
L02853                      tvb_ac(2*id-1,iz)=tvb_ac(2*id-1,iz)-(1./4.)*(d_urb(ibui)/pb(iz+1))*(-sflev(iz,ibui))/&
L02854                                       (dz*bs_u(id,iurb))/da_u(iz)/cp_u
L02855                      qvb_u(2*id-1,iz)=qvb_u(2*id-1,iz)-(1.74.)*(d_urb(ibui)/pb(iz+1))*(lfvlev(iz,ibui)-lflev(iz,ibui))/&
L02856                                       (dz*bs_u(id,iurb))/da_u(iz)/latent
L02857
L02858              endif
L02859
L02860              call flux_wall(ua_u(iz),va_u(iz),pt_u(iz),da_u(iz),    &
L02861                          ptw(2*id,iz,ibui),ptwin(2*id,iz,ibui),     &
L02862                          uva_tmp,vva_tmp,                           &
L02863                          uvb_tmp,vvb_tmp,                           &
L02864                          sfw(2*id,iz,ibui),sfwin(2*id,iz,ibui),     &
L02865                          evb_tmp,drst(id),dt,cdrag(iz))
L02866
L02867              if (pb(iz+1).gt.0.) then
L02868
L02869                      uva_u(2*id,iz)=uva_u(2*id,iz)+d_urb(ibui)/pb(iz+1)*uva_tmp
L02870                      vva_u(2*id,iz)=vva_u(2*id,iz)+d_urb(ibui)/pb(iz+1)*vva_tmp
L02871                      uvb_u(2*id,iz)=uvb_u(2*id,iz)+d_urb(ibui)/pb(iz+1)*uvb_tmp
L02872                      vvb_u(2*id,iz)=vvb_u(2*id,iz)+d_urb(ibui)/pb(iz+1)*vvb_tmp
L02873                      evb_u(2*id,iz)=evb_u(2*id,iz)+d_urb(ibui)/pb(iz+1)*evb_tmp
L02874                      tvb_u(2*id,iz)=tvb_u(2*id,iz)-(d_urb(ibui)/pb(iz+1))*                  &
L02875                                       (sfw(2*id,iz,ibui)*(1.-pwin)+sfwin(2*id,iz,ibui)*pwin)/  &
L02876                                        da_u(iz)/cp_u-(1./4.)*(d_urb(ibui)/pb(iz+1))*(sfvlev(iz,ibui)-sflev(iz,ibui))/&
L02877                                       (dz*bs_u(id,iurb))/da_u(iz)/cp_u
L02878                      tvb_ac(2*id,iz)=tvb_ac(2*id,iz)-(1./4.)*(d_urb(ibui)/pb(iz+1))*(-sflev(iz,ibui))/&
L02879                                       (dz*bs_u(id,iurb))/da_u(iz)/cp_u
L02880                      qvb_u(2*id,iz)=qvb_u(2*id,iz)-(1./4.)*(d_urb(ibui)/pb(iz+1))*(lfvlev(iz,ibui)-lflev(iz,ibui))/&
L02881                                       (dz*bs_u(id,iurb))/da_u(iz)/latent
L02882
L02883              endif
L02884  !
L02885            enddo !iz
L02886           enddo !ibui
L02887
L02888        end do !id
L02889
L02890        return
L02891        end subroutine buildings
```

讲解：把各表面的通量转成城市网格上的 A/B 源项。① **地面**（iz=1）：flux_flat（Louis 1979 局地相似性，14b）出动量/感热/潜热/TKE 的显式项；有积水时潜热按**水池占比** wfg=dg/dgmax 重缩放（lfg=−ρ • L • (−wfg • lfg)/(ρ • L) 的绕行写法等效于乘 wfg）；感热进 thb_u=−sfg/(ρ • cp)。② **屋顶**（iz≥2 且 ss>0）：普通屋顶 flux_flat；gr_flag=1 时再加 flux_flat_roof（绿屋顶，带气孔阻抗），两组按 gr_frac_roof 权重混合，PV 面板部分以 sfrpv 单列后并入 sfr 再减回（「先加后减」保持收支账目清晰——PV 的净效应只进 thb_u）。③ **墙面**（逐方向逐建筑型）：flux_wall（14b，动量拖曳 +对流换热 hc）产出 uva/uvb 等；按 d_urb/pb 加权累加；tvb_u 里除墙面感热外还加了 **AC 与自然通风的室内源项**（sfvlev−sflev 的 ¼ 因子除以 dz • bs 的体积归一）——tvb_ac 单独记录 AC 部分供 urban_meso 分账 b_ac。④ 所有源项输出格式统一为 Flux=A • X+B（A 隐式、B 显式），与 pbl_driver 的 diff3d/各方案隐式求解器对接。

十、改造入口

1. **改建筑分层/材料**：param 的四组厚度数组 + init_para 的 TBL 换算（14b）；三处厚度数组（param/surf_temp/roof_temp_veg）必须一致。
2. **改绿屋顶**：roof_temp_veg 的五组水力参数与 gr_type 分支；灌溉计划在 BEP1D 的 irho 窗口。
3. **改峡谷几何**：param 的 lp/lb/hgt 反解式与 1–150 m 钳位；FRC_URB2D 进场路径在 module_sf_urban（ch17）。
4. **逐格参数场改造**（论文 2 路线）：BEM 内部标量 → 场的先例机制与索引折叠表见 BEP_BEM § 四 ①；BEM 主体改造点在 code_16。
5. **拖曳系数**：BEP1D 的 NEW CDRAG 两式（3.32 • alp^0.47 / 1.85）。

十一、上下游链接

- 下半（辐射、导热、通量、初始化单元全解）：`code_14b_module_sf_bep_bem.md`
- 同族：`code_15_module_sf_bep.md`（BEP 单独版主入口）、`code_16_module_sf_bem.md`（BEM 主体——本篇 BEP1D § 五 ⑨ 的被调方）
- 驱动链：`code_22_module_pbl_driver.md`（flag_bep 与 a_*_bep 数组消费）；面层侧 `code_10_module_sf_mynnsfc.md`
- 教材正文：ch17_urban_canopy.md、ch14_pbl_family.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_14b — BEP/BEM 共享底座 · 下半（module_sf_bep_bem.F：辐射、导热、通量与初始化单元）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_bep_bem.F	5706	8d13861f088e806d2558dd37e31126f2

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇承接 `code_14a_module_sf_bep_bem.md`（主入口 BEP_BEM、柱求解器 BEP1D、param/interpol/averaging_temp/modif_rad/surf_temp/roof_temp_veg/buildings 九单元）；14b 收录 L2892–L5705 全部单元（A 级全文）。

二、城市网格 → 模式网格

```
!file: phys/module_sf_bep_bem.F    L2897-L3115    (subroutine urban_meso)
L02897          subroutine urban_meso(nd,kms,kme,kts,kte,nz_u,z,dz,z_u,pb,ss,bs,ws,sf,vl,    &
L02898                                uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u, &
L02899                                uhb_u,vhb_u,thb_u,ehb_u,qhb_u,qvb_u,       &
L02900                                a_u,a_v,a_t,a_e,b_u,b_v,b_t,b_e,b_q,tvb_ac,b_ac)
L02901
L02902  ! ------------------------------------------------------------------
L02903  !  This routine interpolates the parameters from the "urban grid" to the
L02904  !  "mesoscale grid".
L02905  !  See p300-301 Appendix B.2 of the BLM paper.
L02906  ! ------------------------------------------------------------------
L02907
L02908        implicit none
L02909
L02910  ! ------------------------------------------------------------------
L02911  ! INPUT:
L02912  ! ------------------------------------------------------------------
L02913  ! Data relative to the "mesoscale grid"
L02914        integer kms,kme,kts,kte
L02915        real z(kms:kme)              ! Altitude above the ground of the cell interface
L02916        real dz(kms:kme)               ! Vertical space steps
L02917
L02918  ! Data relative to the "uban grid"
L02919        integer nz_u                 ! Number of layer in the urban grid
L02920        integer nd                   ! Number of street direction for the current urban class
L02921        real bs(ndm)                 ! Building widths of the current urban class
L02922        real ws(ndm)                 ! Street widths of the current urban class
L02923        real z_u(nz_um)            ! Height of the urban grid levels
L02924        real pb(nz_um)             ! Probability to have a building with an height equal
L02925        real ss(nz_um)             ! Probability to have a building with height h
L02926        real uhb_u(ndm,nz_um)    ! U (x-wind component) Horizontal surfaces, B (explicit) term
L02927        real uva_u(2*ndm,nz_um)    ! U (x-wind component) Vertical surfaces, A (implicit) term
L02928        real uvb_u(2*ndm,nz_um)    ! U (x-wind component) Vertical surfaces, B (explicit) term
L02929        real vhb_u(ndm,nz_um)    ! V (y-wind component) Horizontal surfaces, B (explicit) term
L02930        real vva_u(2*ndm,nz_um)    ! V (y-wind component) Vertical surfaces, A (implicit) term
L02931        real vvb_u(2*ndm,nz_um)    ! V (y-wind component) Vertical surfaces, B (explicit) term
L02932        real thb_u(ndm,nz_um)    ! Temperature        Horizontal surfaces, B (explicit) term
L02933        real tva_u(2*ndm,nz_um)    ! Temperature          Vertical surfaces, A (implicit) term
L02934        real tvb_u(2*ndm,nz_um)    ! Temperature          Vertical surfaces, B (explicit) term
L02935        real tvb_ac(2*ndm,nz_um)
L02936        real ehb_u(ndm,nz_um)    ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L02937        real evb_u(2*ndm,nz_um)    ! Energy (TKE)         Vertical surfaces, B (explicit) term
L02938  !
L02939  !New variables for BEM
L02940  !
L02941        real qhb_u(ndm,nz_um)
L02942        real qvb_u(2*ndm,nz_um)
L02943
L02944
L02945  ! ------------------------------------------------------------------
L02946  ! OUTPUT:
```

```
! ------------------------------------------------------------------------
! Data relative to the "mesoscale grid"
      real sf(kms:kme)             ! Surface of the "mesoscale grid" cells taking into account the buildings
      real vl(kms:kme)               ! Volume of the "mesoscale grid" cells taking into account the buildings
      real a_u(kms:kme)              ! Implicit component of the momentum sources or sinks in the X-direction
      real a_v(kms:kme)              ! Implicit component of the momentum sources or sinks in the Y-direction
      real a_t(kms:kme)              ! Implicit component of the heat sources or sinks
      real a_e(kms:kme)              ! Implicit component of the TKE sources or sinks
      real b_u(kms:kme)              ! Explicit component of the momentum sources or sinks in the X-direction
      real b_v(kms:kme)              ! Explicit component of the momentum sources or sinks in the Y-direction
      real b_t(kms:kme)              ! Explicit component of the heat sources or sinks
      real b_ac(kms:kme)
      real b_e(kms:kme)              ! Explicit component of the TKE sources or sinks
      real b_q(kms:kme)               ! Explicit component of the humidity sources or sinks
! ------------------------------------------------------------------------
! LOCAL:
! ------------------------------------------------------------------------
      real dzz
      real fact
      integer id,iz,iz_u
      real se,sr,st,su,sv,sq
      real uet(kms:kme)                ! Contribution to TKE due to walls
      real veb,vta,vtb,vte,vtot,vua,vub,vva,vvb,vqb,vtb_ac


! ------------------------------------------------------------------------
! END VARIABLES DEFINITIONS
! ------------------------------------------------------------------------

! initialisation

      do iz=kts,kte
         a_u(iz)=0.
         a_v(iz)=0.
         a_t(iz)=0.
         a_e(iz)=0.
         b_u(iz)=0.
         b_v(iz)=0.
         b_e(iz)=0.
         b_t(iz)=0.
         b_ac(iz)=0.
         uet(iz)=0.
         b_q(iz)=0.
      end do

! horizontal surfaces
      do iz=kts,kte
         sf(iz)=0.
         vl(iz)=0.
      enddo
      sf(kte+1)=0.

      do id=1,nd
         do iz=kts+1,kte+1
            sr=0.
            do iz_u=2,nz_u
               if(z(iz).lt.z_u(iz_u).and.z(iz).ge.z_u(iz_u-1))then
                  sr=pb(iz_u)
               endif
            enddo
            sf(iz)=sf(iz)+((ws(id)+(1.-sr)*bs(id))/(ws(id)+bs(id)))/nd
         enddo
      enddo

! volume
      do iz=kts,kte
         do id=1,nd
            vtot=0.
            do iz_u=1,nz_u
               dzz=max(min(z_u(iz_u+1),z(iz+1))-max(z_u(iz_u),z(iz)),0.)
               vtot=vtot+pb(iz_u+1)*dzz
            enddo
            vtot=vtot/(z(iz+1)-z(iz))
            vl(iz)=vl(iz)+(1.-vtot*bs(id)/(ws(id)+bs(id)))/nd
         enddo
      enddo

! horizontal surface impact

      do id=1,nd

         fact=1./vl(kts)/dz(kts)*ws(id)/(ws(id)+bs(id))/nd
         b_t(kts)=b_t(kts)+thb_u(id,1)*fact
         b_u(kts)=b_u(kts)+uhb_u(id,1)*fact
         b_v(kts)=b_v(kts)+vhb_u(id,1)*fact
         b_e(kts)=b_e(kts)+ehb_u(id,1)*fact*(z_u(2)-z_u(1))
         b_q(kts)=b_q(kts)+qhb_u(id,1)*fact

         do iz=kts,kte
            st=0.
            su=0.
            sv=0.
            se=0.
            sq=0.
            do iz_u=2,nz_u
               if(z(iz).le.z_u(iz_u).and.z(iz+1).gt.z_u(iz_u))then
                  st=st+ss(iz_u)*thb_u(id,iz_u)
                  su=su+ss(iz_u)*uhb_u(id,iz_u)
                  sv=sv+ss(iz_u)*vhb_u(id,iz_u)
                  se=se+ss(iz_u)*ehb_u(id,iz_u)*(z_u(iz_u+1)-z_u(iz_u))
                  sq=sq+ss(iz_u)*qhb_u(id,iz_u)
               endif
            enddo

            fact=bs(id)/(ws(id)+bs(id))/vl(iz)/dz(iz)/nd
            b_t(iz)=b_t(iz)+st*fact
            b_u(iz)=b_u(iz)+su*fact
            b_v(iz)=b_v(iz)+sv*fact
```

```
L03055              b_e(iz)=b_e(iz)+se*fact
L03056              b_q(iz)=b_q(iz)+sq*fact
L03057           enddo
L03058        enddo
L03059
L03060  ! vertical surface impact
L03061
L03062        do iz=kts,kte
L03063           uet(iz)=0.
L03064           do id=1,nd
L03065              vtb=0.
L03066              vtb_ac=0.
L03067              vta=0.
L03068              vua=0.
L03069              vub=0.
L03070              vva=0.
L03071              vvb=0.
L03072              veb=0.
L03073            vte=0.
L03074              vqb=0.
L03075              do iz_u=1,nz_u
L03076                 dzz=max(min(z_u(iz_u+1),z(iz+1))-max(z_u(iz_u),z(iz)),0.)
L03077                 fact=dzz/(ws(id)+bs(id))
L03078                 vtb=vtb+pb(iz_u+1)*                                    &
L03079                      (tvb_u(2*id-1,iz_u)+tvb_u(2*id,iz_u))*fact
L03080                 vtb_ac=vtb_ac+pb(iz_u+1)*                              &
L03081                      (tvb_ac(2*id-1,iz_u)+tvb_ac(2*id,iz_u))*fact
L03082                 vta=vta+pb(iz_u+1)*                                    &
L03083                     (tva_u(2*id-1,iz_u)+tva_u(2*id,iz_u))*fact
L03084                 vua=vua+pb(iz_u+1)*                                    &
L03085                      (uva_u(2*id-1,iz_u)+uva_u(2*id,iz_u))*fact
L03086                 vva=vva+pb(iz_u+1)*                                    &
L03087                      (vva_u(2*id-1,iz_u)+vva_u(2*id,iz_u))*fact
L03088                 vub=vub+pb(iz_u+1)*                                    &
L03089                      (uvb_u(2*id-1,iz_u)+uvb_u(2*id,iz_u))*fact
L03090                 vvb=vvb+pb(iz_u+1)*                                    &
L03091                      (vvb_u(2*id-1,iz_u)+vvb_u(2*id,iz_u))*fact
L03092                 veb=veb+pb(iz_u+1)*                                    &
L03093                      (evb_u(2*id-1,iz_u)+evb_u(2*id,iz_u))*fact
L03094                 vqb=vqb+pb(iz_u+1)*                                    &
L03095                      (qvb_u(2*id-1,iz_u)+qvb_u(2*id,iz_u))*fact
L03096              enddo
L03097
L03098              fact=1./vl(iz)/dz(iz)/nd
L03099              b_t(iz)=b_t(iz)+vtb*fact
L03100              b_ac(iz)=b_ac(iz)+vtb_ac*fact
L03101              a_t(iz)=a_t(iz)+vta*fact
L03102              a_u(iz)=a_u(iz)+vua*fact
L03103              a_v(iz)=a_v(iz)+vva*fact
L03104              b_u(iz)=b_u(iz)+vub*fact
L03105              b_v(iz)=b_v(iz)+vvb*fact
L03106              b_e(iz)=b_e(iz)+veb*fact
L03107              uet(iz)=uet(iz)+vte*fact
L03108              b_q(iz)=b_q(iz)+vqb*fact
L03109           enddo
L03110        enddo
L03111
L03112
L03113
L03114        return
L03115        end subroutine urban_meso
```

讲解：BEP 的「升网格」出口（BLM 论文附录 B.2），三件事：① **有效表面积 sf** 与**有效体积 vl**：按 pb（建筑存在概率）与街道几何折算模式格元内的可用空气面积/体积——后续通量归一的几何基数；② **水平表面源**（屋顶/路面）：按 ss（该高度有屋顶的概率）与 bs/(bs+ws) 加权，除以 vl • dz 归一到模式网格单位；③ **垂直表面源**（墙面拖曳/加热/加湿）：按 pb 与重叠高度 dzz 累加墙两面的贡献，再除 vl • dz；tva_u（隐式温度项）与 tvb_ac（AC 源项）分账——b_ac 经 BEP_BEM 的 sf_ac 聚合最终成为空调废热诊断量。源项符号约定与 pbl_driver 的隐式求解器（diff3d 及各方案）直接兼容。

```
!file: phys/module_sf_bep_bem.F    L3120-L3200    (subroutine interp_length)
L03120        subroutine interp_length(nd,kms,kme,kts,kte,nz_u,z_u,z,ss,ws,bs,             &
L03121                                 dlg,dl_u)
L03122
L03123  ! ----------------------------------------------------------------------
L03124  !    Calculation of the length scales
L03125  !    See p272-274 formula (22) and (24) of the BLM paper
L03126  ! ----------------------------------------------------------------------
L03127
L03128        implicit none
L03129
L03130
L03131  ! ----------------------------------------------------------------------
L03132  ! INPUT:
L03133  ! ----------------------------------------------------------------------
L03134        integer kms,kme,kts,kte
L03135        real z(kms:kme)              ! Altitude above the ground of the cell interface
L03136        integer nd                 ! Number of street direction for the current urban class
L03137        integer nz_u               ! Number of levels in the "urban grid"
L03138        real z_u(nz_um)          ! Height of the urban grid levels
L03139        real bs(ndm)               ! Building widths of the current urban class
L03140        real ss(nz_um)           ! Probability to have a building with height h
L03141        real ws(ndm)               ! Street widths of the current urban class
L03142
L03143
L03144  ! ----------------------------------------------------------------------
L03145  ! OUTPUT:
L03146  ! ----------------------------------------------------------------------
L03147        real dlg(kms:kme)              ! Height above ground (L_ground in formula (24) of the BLM paper).
L03148        real dl_u(kms:kme)             ! Length scale (lb in formula (22) ofthe BLM paper).
L03149
```

```
L03150  ! ------------------------------------------------------------------
L03151  ! LOCAL:
L03152  ! ------------------------------------------------------------------
L03153        real dlgtmp
L03154        integer id,iz,iz_u
L03155        real sftot
L03156        real ulu,ssl
L03157
L03158  ! ------------------------------------------------------------------
L03159  ! END VARIABLES DEFINITIONS
L03160  ! ------------------------------------------------------------------
L03161
L03162          do iz=kts,kte
L03163           ulu=0.
L03164           ssl=0.
L03165           do id=1,nd
L03166            do iz_u=2,nz_u
L03167             if(z_u(iz_u).gt.z(iz))then
L03168              ulu=ulu+ss(iz_u)/z_u(iz_u)/nd
L03169              ssl=ssl+ss(iz_u)/nd
L03170             endif
L03171            enddo
L03172           enddo
L03173
L03174          if(ulu.ne.0)then
L03175            dl_u(iz)=ssl/ulu
L03176           else
L03177            dl_u(iz)=0.
L03178           endif
L03179          enddo
L03180
L03181
L03182          do iz=kts,kte
L03183           dlg(iz)=0.
L03184            do id=1,nd
L03185             sftot=ws(id)
L03186             dlgtmp=ws(id)/((z(iz)+z(iz+1))/2.)
L03187             do iz_u=1,nz_u
L03188              if((z(iz)+z(iz+1))/2..gt.z_u(iz_u))then
L03189               dlgtmp=dlgtmp+ss(iz_u)*bs(id)/                          &
L03190                     ((z(iz)+z(iz+1))/2.-z_u(iz_u))
L03191               sftot=sftot+ss(iz_u)*bs(id)
L03192              endif
L03193             enddo
L03194             dlg(iz)=dlg(iz)+dlgtmp/sftot/nd
L03195            enddo
L03196            dlg(iz)=1./dlg(iz)
L03197          enddo
L03198
L03199         return
L03200         end subroutine interp_length
```

讲解：BEP 的混合长输出（BLM 论文式 22/24——供 PBL 方案的 TKE 闭合用）：dl_u=Σ(ss/z_u)/Σ(ss/z_u⁻¹) 的加权倒数——高于模式层顶的建筑对湍流混合长的贡献；dlg 是「到最近墙面的平均距离」型长度（含路面项 ws/z 与墙面项 ss • bs/(z−z_u) 的调和平均）。MYJ/BouLac/KEPS 的多城市分支用这两个量替换局地混合长。

三、峡谷辐射

```
!file: phys/module_sf_bep_bem.F    L3205-L3330    (subroutine shadow_mas)
L03205        subroutine shadow_mas(nd,nz_u,zr,deltar,ah,drst,ws,ss,pb,z,    &
L03206                              swddir,rsw,rsg,xlat)
L03207
L03208  ! ------------------------------------------------------------------
L03209  !         Modification of short wave radiation to take into account
L03210  !         the shadow produced by the buildings
L03211  ! ------------------------------------------------------------------
L03212
L03213        implicit none
L03214
L03215  ! ------------------------------------------------------------------
L03216  ! INPUT:
L03217  ! ------------------------------------------------------------------
L03218        integer nd                ! Number of street direction for the current urban class
L03219        integer nz_u              ! number of vertical layers defined in the urban grid
L03220        real ah                   ! Hour angle (it should come from the radiation routine)
L03221        real deltar               ! Declination of the sun
L03222        real drst(ndm)            ! street directions for the current urban class
L03223        real swddir                  ! solar radiation
L03224        real ss(nz_um)          ! probability to have a building with height h
L03225        real pb(nz_um)          ! Probability that a building has an height greater or equal to h
L03226        real ws(ndm)              ! Street width of the current urban class
L03227        real z(nz_um)           ! Height of the urban grid levels
L03228        real zr                   ! zenith angle
L03229        real xlat
L03230        real xlat_r
L03231  ! ------------------------------------------------------------------
L03232  ! OUTPUT:
L03233  ! ------------------------------------------------------------------
L03234        real rsg(ndm)             ! Short wave radiation at the ground
L03235        real rsw(2*ndm,nz_um)     ! Short wave radiation at the walls
L03236
L03237  ! ------------------------------------------------------------------
L03238  ! LOCAL:
L03239  ! ------------------------------------------------------------------
L03240        integer id,iz,jz
L03241        real aae,aaw,bbb,phix,rd,rtot,wsd
L03242
L03243  ! ------------------------------------------------------------------
L03244  ! END VARIABLES DEFINITIONS
```

```
! ------------------------------------------------------------------------

          xlat_r=xlat*pi/180

         if(swddir.eq.0.or.sin(zr).eq.1)then
            do id=1,nd
              rsg(id)=0.
              do iz=1,nz_u
                rsw(2*id-1,iz)=0.
                rsw(2*id,iz)=0.
              enddo
          enddo
         else
!test

         if(abs(sin(zr)).gt.1.e-10)then
           if(cos(deltar)*sin(ah)/sin(zr).ge.1)then
            bbb=pi/2.
           elseif(cos(deltar)*sin(ah)/sin(zr).le.-1)then
            bbb=-pi/2.
           else
            bbb=asin(cos(deltar)*sin(ah)/sin(zr))                !
            if(sin(deltar).lt.(cos(zr)*sin(xlat_r)))then         !
            bbb=pi-bbb                                           !
           endif
           endif
          else
           if(cos(deltar)*sin(ah).ge.0)then
            bbb=pi/2.
           elseif(cos(deltar)*sin(ah).lt.0)then
            bbb=-pi/2.
           endif
          endif
         phix=zr

          do id=1,nd

             rsg(id)=0.

             aae=bbb-drst(id)
             aaw=bbb-drst(id)+pi

             do iz=1,nz_u
                rsw(2*id-1,iz)=0.
                rsw(2*id,iz)=0.
             if(pb(iz+1).gt.0.)then
                do jz=1,nz_u
                 if(abs(sin(aae)).gt.1.e-10)then
                   call shade_wall(z(iz),z(iz+1),z(jz+1),phix,aae,   &
                        ws(id),rd)
                   rsw(2*id-1,iz)=rsw(2*id-1,iz)+swddir*rd*ss(jz+1)/pb(iz+1)
                 endif

                 if(abs(sin(aaw)).gt.1.e-10)then
                   call shade_wall(z(iz),z(iz+1),z(jz+1),phix,aaw,   &
                        ws(id),rd)
                   rsw(2*id,iz)=rsw(2*id,iz)+swddir*rd*ss(jz+1)/pb(iz+1)
                 endif
                enddo
             endif
             enddo
         if(abs(sin(aae)).gt.1.e-10)then
             wsd=abs(ws(id)/sin(aae))

             do jz=1,nz_u
                rd=max(0.,wsd-z(jz+1)*tan(phix))
                rsg(id)=rsg(id)+swddir*rd*ss(jz+1)/wsd
             enddo
             rtot=0.

             do iz=1,nz_u
                rtot=rtot+(rsw(2*id,iz)+rsw(2*id-1,iz))*           &
                          (z(iz+1)-z(iz))
             enddo
             rtot=rtot+rsg(id)*ws(id)
         else
             rsg(id)=swddir
         endif


          enddo
       endif

       return
       end subroutine shadow_mas
```

讲解：直射阴影：由太阳几何（bbb=太阳方位的几何式，含象限修正）与街道方向 drst 求夹角 aa e/aa w；对每个高度层逐「遮挡层」调 shade_wall 算墙面受照比 rd，按 ss(jz+1)/pb(iz+1) 概率加权累加；地面受照用 wsd=ws/sin(aae) 的投影长度减去建筑遮挡 rd=max(0, wsd−z • tan(phix))。swddir=0（阴天/夜间）全零出口。

```
!file: phys/module_sf_bep_bem.F    L3339-L3389    (subroutine shade_wall)
       subroutine shade_wall(z1,z2,hu,phix,aa,ws,rd)

! ------------------------------------------------------------------------
! This routine computes the effects of a shadow induced by a building of
! height hu, on a portion of wall between z1 and z2. See equation A10,
! and correction described below formula A11, and figure A1. Basically rd
! is the ratio between the horizontal surface illuminated and the portion
! of wall. Referring to figure A1, multiplying radiation flux density on
! a horizontal surface (rs) by x1-x2 we have the radiation energy per
! unit time. Dividing this by z2-z1, we obtain the radiation flux
! density reaching the portion of the wall between z2 and z1
```

```
L03350  ! (everything is assumed in 2D)
L03351  ! ------------------------------------------------------------------
L03352
L03353        implicit none
L03354
L03355  ! ------------------------------------------------------------------
L03356  ! INPUT:
L03357  ! ------------------------------------------------------------------
L03358        real aa                   ! Angle between the sun direction and the face of the wall (A12)
L03359        real hu                   ! Height of the building that generates the shadow
L03360        real phix                 ! Solar zenith angle
L03361        real ws                   ! Width of the street
L03362        real z1                   ! Height of the level z(iz)
L03363        real z2                   ! Height of the level z(iz+1)
L03364
L03365  ! ------------------------------------------------------------------
L03366  ! OUTPUT:
L03367  ! ------------------------------------------------------------------
L03368        real rd                   ! Ratio between (x1-x2)/(z2-z1), see Fig. 1A.
L03369                                  ! Multiplying rd by rs (radiation flux
L03370                                  ! density on a horizontal surface) gives
L03371                                  ! the radiation flux density on the
L03372                                  ! portion of wall between z1 and z2.
L03373  ! ------------------------------------------------------------------
L03374  ! LOCAL:
L03375  ! ------------------------------------------------------------------
L03376        real x1,x2                ! x1,x2 see Fig. A1.
L03377
L03378  ! ------------------------------------------------------------------
L03379  ! END VARIABLES DEFINITIONS
L03380  ! ------------------------------------------------------------------
L03381
L03382        x1=min((hu-z1)*tan(phix),max(0.,ws/sin(aa)))
L03383
L03384        x2=max((hu-z2)*tan(phix),0.)
L03385
L03386        rd=max(0.,sin(aa)*(max(0.,x1-x2))/(z2-z1))
L03387
L03388        return
L03389        end subroutine shade_wall
```

讲解：BLM 论文式 A10–A11 的 2D 几何：x1=min((hu−z1)•tan(天顶角), ws/sin(aa))、x2=(hu−z2)•tan(天顶角)，rd=sin(aa)•max(0, x1−x2)/(z2−z1)——「水平受照面能量 → 墙面通量密度」的转换比，论文配图 A1。

```
!file: phys/module_sf_bep_bem.F    L3394-L3545    (subroutine long_rad)
L03394        subroutine long_rad(iurb,nz_u,id,emw,emg,emwin,pwin,twlev,&
L03395                            fwg,fww,fgw,fsw,fsg,tg_av,tw,rlg,rlw,rl,pb)
L03396
L03397  ! ------------------------------------------------------------------
L03398  ! This routine computes the effects of the reflections of long-wave
L03399  ! radiation in the street canyon by solving the system
L03400  ! of 2*nz_u+1 eqn. in 2*nz_u+1
L03401  ! unkonwn defined in A4, A5 and A6 of the paper (pages 295 and 296).
L03402  ! The system is solved by solving A X= B,
L03403  ! with A matrix B vector, and X solution.
L03404  ! ------------------------------------------------------------------
L03405
L03406        implicit none
L03407
L03408
L03409
L03410  ! ------------------------------------------------------------------
L03411  ! INPUT:
L03412  ! ------------------------------------------------------------------
L03413        real emg                        ! Emissivity of ground for the current urban class
L03414        real emw                        ! Emissivity of wall for the current urban class
L03415        real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
L03416        real fsg(ndm,nurbm)             ! View factors from sky to ground
L03417        real fsw(nz_um,ndm,nurbm)       ! View factors from sky to wall
L03418        real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
L03419        real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
L03420        integer id                      ! Current street direction
L03421        integer iurb                    ! Current urban class
L03422        integer nz_u                    ! Number of layer in the urban grid
L03423        real pb(nz_um)                  ! Probability to have a building with an height equal
L03424        real rl                         ! Downward flux of the longwave radiation
L03425        real tg_av(ndm)                ! Temperature in each layer of the ground [K]
L03426        real tw(2*ndm,nz_um)            ! Temperature in each layer of the wall [K]
L03427  !
L03428  !New Variables for BEM
L03429  !
L03430        real twlev(2*ndm,nz_um)         ! Window temperature in BEM [K]
L03431        real emwin                      ! Emissivity of windows
L03432        real pwin                       ! Coverage area fraction of windows in the walls of the buildings (BEM)
L03433
L03434
L03435  ! ------------------------------------------------------------------
L03436  ! OUTPUT:
L03437  ! ------------------------------------------------------------------
L03438        real rlg(ndm)                   ! Long wave radiation at the ground
L03439        real rlw(2*ndm,nz_um)           ! Long wave radiation at the walls
L03440
L03441  ! ------------------------------------------------------------------
L03442  ! LOCAL:
L03443  ! ------------------------------------------------------------------
L03444        integer i,j
L03445        real aaa(2*nz_um+1,2*nz_um+1)   ! terms of the matrix
L03446        real bbb(2*nz_um+1)             ! terms of the vector
L03447
L03448  ! ------------------------------------------------------------------
L03449  ! END VARIABLES DEFINITIONS
L03450  ! ------------------------------------------------------------------
L03451
L03452
```

```
L03453  ! west wall
L03454
L03455        do i=1,nz_u
L03456
L03457         do j=1,nz_u
L03458          aaa(i,j)=0.
L03459         enddo
L03460
L03461         aaa(i,i)=1.
L03462
L03463         do j=nz_u+1,2*nz_u
L03464          aaa(i,j)=-(1.-emw*(1.-pwin)-emwin*pwin)* &
L03465                fww(j-nz_u,i,id,iurb)*pb(j-nz_u+1)
L03466         enddo
L03467
L03468  !!     aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i,id,iurb)*pb(i+1)
L03469         aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i,id,iurb)
L03470
L03471         bbb(i)=fsw(i,id,iurb)*rl+emg*fgw(i,id,iurb)*sigma*tg_av(id)**4
L03472         do j=1,nz_u
L03473            bbb(i)=bbb(i)+pb(j+1)*sigma*fww(j,i,id,iurb)* &
L03474                  (emw*(1.-pwin)*tw(2*id,j)**4+emwin*pwin*twlev(2*id,j)**4)+ &
L03475                  fww(j,i,id,iurb)*rl*(1.-pb(j+1))
L03476         enddo
L03477
L03478        enddo
L03479
L03480  ! east wall
L03481
L03482        do i=1+nz_u,2*nz_u
L03483
L03484         do j=1,nz_u
L03485          aaa(i,j)=-(1.-emw*(1.-pwin)-emwin*pwin)*fww(j,i-nz_u,id,iurb)*pb(j+1)
L03486         enddo
L03487
L03488         do j=1+nz_u,2*nz_u
L03489          aaa(i,j)=0.
L03490         enddo
L03491
L03492         aaa(i,i)=1.
L03493
L03494  !!     aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i-nz_u,id,iurb)*pb(i-nz_u+1)
L03495         aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i-nz_u,id,iurb)
L03496
L03497         bbb(i)=fsw(i-nz_u,id,iurb)*rl+  &
L03498                emg*fgw(i-nz_u,id,iurb)*sigma*tg_av(id)**4
L03499
L03500         do j=1,nz_u
L03501          bbb(i)=bbb(i)+pb(j+1)*sigma*fww(j,i-nz_u,id,iurb)*  &
L03502                 (emw*(1.-pwin)*tw(2*id-1,j)**4+emwin*pwin*twlev(2*id-1,j)**4)+&
L03503                 fww(j,i-nz_u,id,iurb)*rl*(1.-pb(j+1))
L03504         enddo
L03505
L03506        enddo
L03507
L03508  ! ground
L03509        do j=1,nz_u
L03510         aaa(2*nz_u+1,j)=-(1.-emw*(1.-pwin)-emwin*pwin)* &
L03511                         fwg(j,id,iurb)*pb(j+1)
L03512        enddo
L03513
L03514        do j=nz_u+1,2*nz_u
L03515         aaa(2*nz_u+1,j)=-(1.-emw*(1.-pwin)-emwin*pwin)* &
L03516                         fwg(j-nz_u,id,iurb)*pb(j-nz_u+1)
L03517        enddo
L03518
L03519        aaa(2*nz_u+1,2*nz_u+1)=1.
L03520
L03521        bbb(2*nz_u+1)=fsg(id,iurb)*rl
L03522
L03523        do i=1,nz_u
L03524         bbb(2*nz_u+1)=bbb(2*nz_u+1)+sigma*fwg(i,id,iurb)*pb(i+1)*         &
L03525                       (emw*(1.-pwin)*(tw(2*id-1,i)**4+tw(2*id,i)**4)+     &
L03526                       emwin*pwin*(twlev(2*id-1,i)**4+twlev(2*id,i)**4))+  &
L03527                       2.*fwg(i,id,iurb)*(1.-pb(i+1))*rl
L03528        enddo
L03529
L03530
L03531
L03532        call gaussj(aaa,2*nz_u+1,bbb,2*nz_um+1)
L03533
L03534        do i=1,nz_u
L03535         rlw(2*id-1,i)=bbb(i)
L03536        enddo
L03537
L03538        do i=nz_u+1,2*nz_u
L03539         rlw(2*id,i-nz_u)=bbb(i)
L03540        enddo
L03541
L03542        rlg(id)=bbb(2*nz_u+1)
L03543
L03544        return
L03545        end subroutine long_rad
```

讲解：街道峡谷长波**多次反射**：2 • nz+1 元非线性方程组（墙面 i 的辐射平衡：自身发射 +反射入射）线性化后用 gaussj 解；窗户按 pwin 混进发射率（emw(1−pwin)+emwin • pwin）与 T^4 源项；注释行展示了一个已修正的 bug（fgw 项曾误乘 pb）。输出 rlg/rlw 各面接收的长波。

```
!file: phys/module_sf_bep_bem.F    L3551-L3663    (subroutine short_rad_dd)
L03551        subroutine short_rad_dd(iurb,nz_u,id,albw,                    &
L03552                                albg,rsdif,fwg,fww,fgw,fsw,fsg,rsg,rsw,pb)
L03553
L03554  ! ------------------------------------------------------------------------
L03555  ! This routine computes the effects of the reflections of short-wave
L03556  ! (solar) radiation in the street canyon by solving the system
```

```
! of 2*nz_u+1 eqn. in 2*nz_u+1
! unkonwn defined in A4, A5 and A6 of the paper (pages 295 and 296).
! The system is solved by solving A X= B,
! with A matrix B vector, and X solution.
! ------------------------------------------------------------------

      implicit none



! ------------------------------------------------------------------
! INPUT:
! ------------------------------------------------------------------
      real albg                ! Albedo of the ground for the current urban class
      real albw                ! Albedo of the wall for the current urban class
      real rsdif               ! diffused short wave radiation
      real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
      real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
      real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
      real fsg(ndm,nurbm)             ! View factors from sky to ground
      real fsw(nz_um,ndm,nurbm)       ! View factors from sky to wall
      integer id               ! current street direction
      integer iurb             ! current urban class
      integer nz_u             ! Number of layer in the urban grid
      real pb(nz_um)          ! probability to have a building with an height equal

! ------------------------------------------------------------------
! OUTPUT:
! ------------------------------------------------------------------
      real rsg(ndm)            ! Short wave radiation at the ground
      real rsw(2*ndm,nz_um)    ! Short wave radiation at the walls

! ------------------------------------------------------------------
! LOCAL:
! ------------------------------------------------------------------
      integer i,j
      real aaa(2*nz_um+1,2*nz_um+1)  ! terms of the matrix
      real bbb(2*nz_um+1)            ! terms of the vector

! ------------------------------------------------------------------
! END VARIABLES DEFINITIONS
! ------------------------------------------------------------------

! west wall


        do i=1,nz_u
         do j=1,nz_u
            aaa(i,j)=0.
         enddo

         aaa(i,i)=1.

         do j=nz_u+1,2*nz_u
            aaa(i,j)=-albw*fww(j-nz_u,i,id,iurb)*pb(j-nz_u+1)
         enddo

         aaa(i,2*nz_u+1)=-albg*fgw(i,id,iurb)
         bbb(i)=rsw(2*id-1,i)+fsw(i,id,iurb)*rsdif

      enddo

! east wall
       do i=1+nz_u,2*nz_u
         do j=1,nz_u
            aaa(i,j)=-albw*fww(j,i-nz_u,id,iurb)*pb(j+1)
         enddo

         do j=1+nz_u,2*nz_u
            aaa(i,j)=0.
         enddo

        aaa(i,i)=1.
        aaa(i,2*nz_u+1)=-albg*fgw(i-nz_u,id,iurb)
        bbb(i)=rsw(2*id,i-nz_u)+fsw(i-nz_u,id,iurb)*rsdif

      enddo


! ground
      do j=1,nz_u
         aaa(2*nz_u+1,j)=-albw*fwg(j,id,iurb)*pb(j+1)
      enddo

      do j=nz_u+1,2*nz_u
         aaa(2*nz_u+1,j)=-albw*fwg(j-nz_u,id,iurb)*pb(j-nz_u+1)
      enddo

      aaa(2*nz_u+1,2*nz_u+1)=1.
      bbb(2*nz_u+1)=rsg(id)+fsg(id,iurb)*rsdif

      call gaussj(aaa,2*nz_u+1,bbb,2*nz_um+1)

      do i=1,nz_u
         rsw(2*id-1,i)=bbb(i)
      enddo

      do i=nz_u+1,2*nz_u
         rsw(2*id,i-nz_u)=bbb(i)
      enddo

      rsg(id)=bbb(2*nz_u+1)


      return
      end subroutine short_rad_dd
```

讲解：短波漫射的同一套线性系统（反射率代替发射率）；直射部分不进此系统——shadow_mas

已按几何直接算。两系统共用 gaussj。

```
!file: phys/module_sf_bep_bem.F    L3670-L3772    (subroutine gaussj)
L03670        subroutine gaussj(a,n,b,np)
L03671
L03672  ! ------------------------------------------------------------------------
L03673  ! This routine solve a linear system of n equations of the form
L03674  !              A X = B
L03675  !  where  A is a matrix a(i,j)
L03676  !         B a vector and X the solution
L03677  ! In output b is replaced by the solution
L03678  ! ------------------------------------------------------------------------
L03679
L03680        implicit none
L03681
L03682  ! ------------------------------------------------------------------------
L03683  ! INPUT:
L03684  ! ------------------------------------------------------------------------
L03685        integer np
L03686        real a(np,np)
L03687
L03688  ! ------------------------------------------------------------------------
L03689  ! OUTPUT:
L03690  ! ------------------------------------------------------------------------
L03691        real b(np)
L03692
L03693  ! ------------------------------------------------------------------------
L03694  ! LOCAL:
L03695  ! ------------------------------------------------------------------------
L03696        integer nmax
L03697        parameter (nmax=150)
L03698
L03699        real big,dum
L03700        integer i,icol,irow
L03701        integer j,k,l,ll,n
L03702        integer ipiv(nmax)
L03703        real pivinv
L03704
L03705  ! ------------------------------------------------------------------------
L03706  ! END VARIABLES DEFINITIONS
L03707  ! ------------------------------------------------------------------------
L03708
L03709        do j=1,n
L03710           ipiv(j)=0.
L03711        enddo
L03712
L03713        do i=1,n
L03714           big=0.
L03715           do j=1,n
L03716              if(ipiv(j).ne.1)then
L03717                 do k=1,n
L03718                    if(ipiv(k).eq.0)then
L03719                       if(abs(a(j,k)).ge.big)then
L03720                          big=abs(a(j,k))
L03721                          irow=j
L03722                          icol=k
L03723                       endif
L03724                    elseif(ipiv(k).gt.1)then
L03725                       FATAL_ERROR('singular matrix in gaussj')
L03726                    endif
L03727                 enddo
L03728              endif
L03729           enddo
L03730
L03731           ipiv(icol)=ipiv(icol)+1
L03732
L03733           if(irow.ne.icol)then
L03734              do l=1,n
L03735                 dum=a(irow,l)
L03736                 a(irow,l)=a(icol,l)
L03737                 a(icol,l)=dum
L03738              enddo
L03739
L03740              dum=b(irow)
L03741              b(irow)=b(icol)
L03742              b(icol)=dum
L03743
L03744           endif
L03745
L03746           if(a(icol,icol).eq.0) FATAL_ERROR('singular matrix in gaussj')
L03747
L03748           pivinv=1./a(icol,icol)
L03749           a(icol,icol)=1
L03750
L03751           do l=1,n
L03752              a(icol,l)=a(icol,l)*pivinv
L03753           enddo
L03754
L03755           b(icol)=b(icol)*pivinv
L03756
L03757           do ll=1,n
L03758              if(ll.ne.icol)then
L03759                 dum=a(ll,icol)
L03760                 a(ll,icol)=0.
L03761                 do l=1,n
L03762                    a(ll,l)=a(ll,l)-a(icol,l)*dum
L03763                 enddo
L03764
L03765                 b(ll)=b(ll)-b(icol)*dum
L03766
L03767              endif
L03768           enddo
L03769        enddo
L03770
L03771        return
L03772        end subroutine gaussj
```

讲解：Numerical Recipes 的全主元 Gauss–Jordan 消元（nmax=150 上限对应 2 • nz_um+1=37），奇

异矩阵直接 FATAL_ERROR。BEP 每步每方向每点都要解两个 37×37 线性系统——城市冠层的计算量大头之一（论文 2 三域 1 km 时注意性能）。

四、导热与水分求解器

```
!file: phys/module_sf_bep_bem.F    L3779-L3857    (subroutine soil_moist)
L03779        subroutine soil_moist(nz,dz,qv,dt,lf,d,k,rainbl,drain,irri_now)
L03780
L03781  ! ----------------------------------------------------------------------
L03782  ! This routine solves the Fourier diffusion equation for heat in
L03783  ! the material (wall, roof, or ground). Resolution is done implicitely.
L03784  ! Boundary conditions are:
L03785  ! - fixed temperature at the interior
L03786  ! - energy budget at the surface
L03787  ! ----------------------------------------------------------------------
L03788
L03789        implicit none
L03790
L03791
L03792
L03793  ! ----------------------------------------------------------------------
L03794  ! INPUT:
L03795  ! ----------------------------------------------------------------------
L03796        integer nz                ! Number of layers
L03797        real dt                   ! Time step
L03798        real lf                   ! Latent heat flux at the surface
L03799        real qv(nz)               ! Moisture in each layer [K]
L03800        real dz(nz)               ! Layer sizes [m]
L03801        real rainbl               ! Rainfall [mm]
L03802        real d(nz)                ! Soil water diffusivity
L03803        real k(nz)                ! Hydraulic conductivity
L03804        real gr                   ! Dummy variable
L03805        real drain
L03806        real irri_now
L03807  ! ----------------------------------------------------------------------
L03808  ! OUTPUT:
L03809  ! ----------------------------------------------------------------------
L03810
L03811
L03812  ! ----------------------------------------------------------------------
L03813  ! LOCAL:
L03814  ! ----------------------------------------------------------------------
L03815        integer iz
L03816        real a(nz,3)
L03817        real alpha
L03818        real c(nz)
L03819        real cddz(nz+2)
L03820        real dw     !water density Kg/m3
L03821        parameter(dw=1000.)
L03822  !----------------------------------------------------------------------
L03823  ! END VARIABLES DEFINITIONS
L03824  ! ----------------------------------------------------------------------
L03825
L03826        alpha=rainbl/(dw*dt)+lf/latent/dw+irri_now/dw
L03827        cddz(1)=0.
L03828        do iz=2,nz
L03829           cddz(iz)=2.*d(iz)/(dz(iz)+dz(iz-1))
L03830        enddo
L03831        do iz=1,4
L03832           a(iz,1)=0.
L03833           a(iz,2)=1.
L03834           a(iz,3)=0.
L03835           c(iz)=qv(iz)
L03836        enddo
L03837        do iz=6,nz-1
L03838           a(iz,1)=-cddz(iz)*dt/dz(iz)
L03839           a(iz,2)=1.+dt*(cddz(iz)+cddz(iz+1))/dz(iz)
L03840           a(iz,3)=-cddz(iz+1)*dt/dz(iz)
L03841           c(iz)=qv(iz)+dt*(k(iz+1)-k(iz))/dz(iz)
L03842        enddo
L03843           a(5,1)=0.
L03844           a(5,2)=1.+dt*(cddz(5+1))/dz(5)
L03845           a(5,3)=-cddz(5+1)*dt/dz(5)
L03846           c(5)=qv(5)+dt*(k(5+1)-drain)/dz(5)
L03847
L03848
L03849        a(nz,1)=-dt*cddz(nz)/dz(nz)
L03850        a(nz,2)=1.+dt*cddz(nz)/dz(nz)
L03851        a(nz,3)=0.
L03852        c(nz)=qv(nz)+dt*alpha/dz(nz)-dt*k(nz-1)/dz(nz)
L03853
L03854        call invert(nz,a,c,qv)
L03855
L03856        return
L03857        end subroutine soil_moist
```

讲解：土壤含水量隐式扩散（绿屋顶用）：源项 alpha=降雨/(ρ_w • dt)+蒸发/(L • ρ_w)+灌溉/ρ_w；第 5 层为排水层（k 换 drain）；第 6 层起为 Richards 型扩散项 + 对流项 (k(iz+1)−k(iz))/dz；前 4 层恒等（屋顶建材不含水）；invert 解三对角。

```
!file: phys/module_sf_bep_bem.F    L3866-L3959    (subroutine soil_temp_veg)
L03866        subroutine soil_temp_veg(heflro,nz,dz,temp,pt,ala,cs,                  &
L03867                                 rs,rl,press,dt,em,alb,rt,sf,lf,gf,pv_frac_roof,tpv)
L03868
L03869  ! ----------------------------------------------------------------------
L03870  ! This routine solves the Fourier diffusion equation for heat in
L03871  ! the material (wall, roof, or ground). Resolution is done implicitely.
L03872  ! Boundary conditions are:
L03873  ! - fixed temperature at the interior
```

```
L03874  ! - energy budget at the surface
L03875  ! ------------------------------------------------------------------------
L03876
L03877        implicit none
L03878
L03879
L03880
L03881  ! ------------------------------------------------------------------------
L03882  ! INPUT:
L03883  ! ------------------------------------------------------------------------
L03884        integer nz                ! Number of layers
L03885        real ala(nz)              ! Thermal diffusivity in each layers [m^2 s^-1]
L03886        real alb                  ! Albedo of the surface
L03887        real cs(nz)               ! Specific heat of the material [J m^3 K^-1]
L03888        real dt                   ! Time step
L03889        real em                   ! Emissivity of the surface
L03890        real press                ! Pressure at ground level
L03891        real rl                   ! Downward flux of the longwave radiation
L03892        real rs                   ! Solar radiation
L03893        real sf                   ! Sensible heat flux at the surface
L03894        real lf                   ! Latent heat flux at the surface
L03895        real temp(nz)             ! Temperature in each layer [K]
L03896        real dz(nz)               ! Layer sizes [m]
L03897        real heflro                ! Heat flux between roof and green roof
L03898        real rs_eff
L03899        real rl_eff
L03900        real tpv
L03901        real pv_frac_roof
L03902  ! ------------------------------------------------------------------------
L03903  ! OUTPUT:
L03904  ! ------------------------------------------------------------------------
L03905        real gf                   ! Heat flux transferred from the surface toward the interior
L03906        real pt                   ! Potential temperature at the surface
L03907        real rt                   ! Total radiation at the surface (solar+incoming long+outgoing long)
L03908
L03909  ! ------------------------------------------------------------------------
L03910  ! LOCAL:
L03911  ! ------------------------------------------------------------------------
L03912        integer iz
L03913        real a(nz,3)
L03914        real alpha
L03915        real c(nz)
L03916        real cddz(nz+2)
L03917        real tsig
L03918
L03919  ! ------------------------------------------------------------------------
L03920  ! END VARIABLES DEFINITIONS
L03921  ! ------------------------------------------------------------------------
L03922       if(pv_frac_roof.gt.0)then
L03923       rl_eff=(1-pv_frac_roof)*em*rl+em*sigma*tpv**4*pv_frac_roof
L03924        rs_eff=(1.-pv_frac_roof)*rs
L03925       else
L03926        rl_eff=em*rl
L03927        rs_eff=rs
L03928       endif
L03929        tsig=temp(nz)
L03930        alpha=(1.-alb)*rs_eff+rl_eff-em*sigma*(tsig**4)+sf+lf
L03931        cddz(1)=ala(1)/dz(1)
L03932        do iz=2,nz
L03933           cddz(iz)=2.*ala(iz)/(dz(iz)+dz(iz-1))
L03934        enddo
L03935
L03936        a(1,1)=0.
L03937        a(1,2)=1.
L03938        a(1,3)=0.
L03939        c(1)=temp(1)-heflro*dt/dz(1)
L03940        do iz=2,nz-1
L03941           a(iz,1)=-cddz(iz)*dt/dz(iz)
L03942           a(iz,2)=1.+dt*(cddz(iz)+cddz(iz+1))/dz(iz)
L03943           a(iz,3)=-cddz(iz+1)*dt/dz(iz)
L03944           c(iz)=temp(iz)
L03945        enddo
L03946        a(nz,1)=-dt*cddz(nz)/dz(nz)
L03947        a(nz,2)=1.+dt*cddz(nz)/dz(nz)
L03948        a(nz,3)=0.
L03949        c(nz)=temp(nz)+dt*alpha/cs(nz)/dz(nz)
L03950
L03951        call invert(nz,a,c,temp)
L03952
L03953        pt=temp(nz)*(press/1.e+5)**(-rcp_u)
L03954
L03955        rt=(1.-alb)*rs_eff+rl_eff-em*sigma*(tsig**4.)
L03956
L03957        gf=(1.-alb)*rs_eff+rl_eff-em*sigma*(tsig**4.)+sf
L03958        return
L03959        end subroutine soil_temp_veg
```

讲解：绿屋顶导热（soil_temp 的植被/PV 扩展）：PV 面板占比 pv_frac_roof 调整有效辐射（rl_eff=(1−pv) • em • rl+em • σ • T_pv^4 • pv——面板当发射体；rs_eff=(1−pv) • rs）；第 1 层加与结构屋顶的耦合热流 heflro（来自 roof_temp_veg 的 hfgroof）；表面能量收支作上边界、深层固定温度作下边界。

```
!file: phys/module_sf_bep_bem.F    L3964-L4049    (subroutine soil_temp)
L03964        subroutine soil_temp(nz,dz,temp,pt,ala,cs,                    &
L03965                             rs,rl,press,dt,em,alb,rt,sf,lf,gf)
L03966
L03967  ! ------------------------------------------------------------------------
L03968  ! This routine solves the Fourier diffusion equation for heat in
L03969  ! the material (wall, roof, or ground). Resolution is done implicitely.
L03970  ! Boundary conditions are:
L03971  ! - fixed temperature at the interior
L03972  ! - energy budget at the surface
L03973  ! ------------------------------------------------------------------------
L03974
```

```
L03975        implicit none
L03976
L03977
L03978
L03979  ! ------------------------------------------------------------------
L03980  ! INPUT:
L03981  ! ------------------------------------------------------------------
L03982        integer nz                ! Number of layers
L03983        real ala(nz)              ! Thermal diffusivity in each layers [m^2 s^-1]
L03984        real alb                  ! Albedo of the surface
L03985        real cs(nz)               ! Specific heat of the material [J m^3 K^-1]
L03986        real dt                   ! Time step
L03987        real em                   ! Emissivity of the surface
L03988        real press                ! Pressure at ground level
L03989        real rl                   ! Downward flux of the longwave radiation
L03990        real rs                   ! Solar radiation
L03991        real sf                   ! Sensible heat flux at the surface
L03992        real lf                   ! Latent heat flux at the surface
L03993        real temp(nz)             ! Temperature in each layer [K]
L03994        real dz(nz)               ! Layer sizes [m]
L03995
L03996  ! ------------------------------------------------------------------
L03997  ! OUTPUT:
L03998  ! ------------------------------------------------------------------
L03999        real gf                   ! Heat flux transferred from the surface toward the interior
L04000        real pt                   ! Potential temperature at the surface
L04001        real rt                   ! Total radiation at the surface (solar+incoming long+outgoing long)
L04002
L04003  ! ------------------------------------------------------------------
L04004  ! LOCAL:
L04005  ! ------------------------------------------------------------------
L04006        integer iz
L04007        real a(nz,3)
L04008        real alpha
L04009        real c(nz)
L04010        real cddz(nz+2)
L04011        real tsig
L04012
L04013  ! ------------------------------------------------------------------
L04014  ! END VARIABLES DEFINITIONS
L04015  ! ------------------------------------------------------------------
L04016
L04017        tsig=temp(nz)
L04018        alpha=(1.-alb)*rs+em*rl-em*sigma*(tsig**4)+sf+lf
L04019  ! Compute cddz=2*cd/dz
L04020        cddz(1)=ala(1)/dz(1)
L04021        do iz=2,nz
L04022           cddz(iz)=2.*ala(iz)/(dz(iz)+dz(iz-1))
L04023        enddo
L04024
L04025        a(1,1)=0.
L04026        a(1,2)=1.
L04027        a(1,3)=0.
L04028        c(1)=temp(1)
L04029        do iz=2,nz-1
L04030           a(iz,1)=-cddz(iz)*dt/dz(iz)
L04031           a(iz,2)=1.+dt*(cddz(iz)+cddz(iz+1))/dz(iz)
L04032           a(iz,3)=-cddz(iz+1)*dt/dz(iz)
L04033           c(iz)=temp(iz)
L04034        enddo
L04035        a(nz,1)=-dt*cddz(nz)/dz(nz)
L04036        a(nz,2)=1.+dt*cddz(nz)/dz(nz)
L04037        a(nz,3)=0.
L04038        c(nz)=temp(nz)+dt*alpha/cs(nz)/dz(nz)
L04039
L04040
L04041        call invert(nz,a,c,temp)
L04042
L04043        pt=temp(nz)*(press/1.e+5)**(-rcp_u)
L04044
L04045        rt=(1.-alb)*rs+em*rl-em*sigma*(tsig**4.)
L04046
L04047         gf=(1.-alb)*rs+em*rl-em*sigma*(tsig**4.)+sf
L04048        return
L04049        end subroutine soil_temp
```

讲解：通用一维 Fourier 导热求解器（墙面/路面/普通屋顶共用）：cddz=2 • ala/(dz_iz+dz_iz−1) 的层间调和平均，表面层能量收支 alpha=(1−alb) • rs+em • rl−em • σ • T^4+sf+lf 作 Neumann 条件，底层固定温度；返回位温 pt、总辐射 rt、向内热流 gf。BEP 所有固体温度演化的核心。

```
!file: phys/module_sf_bep_bem.F    L4056-L4102    (subroutine invert)
L04056        subroutine invert(n,a,c,x)
L04057
L04058  ! ------------------------------------------------------------------
L04059  !        Inversion and resolution of a tridiagonal matrix
L04060  !                   A X = C
L04061  ! ------------------------------------------------------------------
L04062
L04063        implicit none
L04064
L04065  ! ------------------------------------------------------------------
L04066  ! INPUT:
L04067  ! ------------------------------------------------------------------
L04068         integer n
L04069         real a(n,3)              !  a(*,1) lower diagonal (Ai,i-1)
L04070                                  !  a(*,2) principal diagonal (Ai,i)
L04071                                  !  a(*,3) upper diagonal (Ai,i+1)
L04072         real c(n)
L04073
L04074  ! ------------------------------------------------------------------
L04075  ! OUTPUT:
L04076  ! ------------------------------------------------------------------
L04077         real x(n)
L04078
L04079  ! ------------------------------------------------------------------
```

```
L04080  ! LOCAL:
L04081  ! ------------------------------------------------------------------------
L04082        integer i
L04083
L04084  ! ------------------------------------------------------------------------
L04085  ! END VARIABLES DEFINITIONS
L04086  ! ------------------------------------------------------------------------
L04087
L04088        do i=n-1,1,-1
L04089           c(i)=c(i)-a(i,3)*c(i+1)/a(i+1,2)
L04090           a(i,2)=a(i,2)-a(i,3)*a(i+1,1)/a(i+1,2)
L04091        enddo
L04092
L04093        do i=2,n
L04094           c(i)=c(i)-a(i,1)*c(i-1)/a(i-1,2)
L04095        enddo
L04096
L04097        do i=1,n
L04098           x(i)=c(i)/a(i,2)
L04099        enddo
L04100
L04101        return
L04102        end subroutine invert
```

讲解：托马斯消元（a(n,3) 三对角存储，原地破坏 a/c——与 pbl_driver 的 invert 同源不同名，与 code_09b tridiag 家族的「是否保留输入」差异一致；复用前必查）。

五、通量参数化

```
!file: phys/module_sf_bep_bem.F    L4108-L4199    (subroutine flux_wall)
L04108        subroutine flux_wall(ua,va,pt,da,ptw,ptwin,uva,vva,uvb,vvb,  &
L04109                             sfw,sfwin,evb,drst,dt,cdrag)
L04110
L04111  ! ------------------------------------------------------------------------
L04112  ! This routine computes the surface sources or sinks of momentum, tke,
L04113  ! and heat from vertical surfaces (walls).
L04114  ! ------------------------------------------------------------------------
L04115        implicit none
L04116
L04117  ! INPUT:
L04118  ! -----
L04119        real drst                 ! street directions for the current urban class
L04120        real da                   ! air density
L04121        real pt                   ! potential temperature
L04122        real ptw                  ! Walls potential temperatures
L04123        real ptwin                ! Windows potential temperatures
L04124        real ua                   ! wind speed
L04125        real va                   ! wind speed
L04126        real dt                   !time step
L04127        real cdrag
L04128  ! OUTPUT:
L04129  ! ------
L04130  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L04131  ! vertical surfaces (walls).
L04132  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L04133  ! Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L04134
L04135        real uva                  ! U (wind component)   Vertical surfaces, A (implicit) term
L04136        real uvb                  ! U (wind component)   Vertical surfaces, B (explicit) term
L04137        real vva                  ! V (wind component)   Vertical surfaces, A (implicit) term
L04138        real vvb                  ! V (wind component)   Vertical surfaces, B (explicit) term
L04139        real tva                  ! Temperature          Vertical surfaces, A (implicit) term
L04140        real tvb                  ! Temperature          Vertical surfaces, B (explicit) term
L04141        real evb                  ! Energy (TKE)         Vertical surfaces, B (explicit) term
L04142        real sfw                  ! Surfaces fluxes from the walls
L04143        real sfwin                ! Surfaces fluxes from the windows
L04144
L04145  ! LOCAL:
L04146  ! -----
L04147        real hc
L04148        real hcwin
L04149        real u_ort
L04150        real vett
L04151
L04152
L04153  ! -------------------------
L04154  ! END VARIABLES DEFINITIONS
L04155  ! -------------------------
L04156
L04157        vett=(ua**2+va**2)**.5
L04158
L04159        u_ort=abs((cos(drst)*ua-sin(drst)*va))
L04160
L04161        uva=-cdrag*u_ort/2.*cos(drst)*cos(drst)
L04162        vva=-cdrag*u_ort/2.*sin(drst)*sin(drst)
L04163
L04164        uvb=cdrag*u_ort/2.*sin(drst)*cos(drst)*va
L04165        vvb=cdrag*u_ort/2.*sin(drst)*cos(drst)*ua
L04166
L04167        if (vett.lt.4.88) then
L04168           hc=5.678*(1.09+0.23*(vett/0.3048))
L04169        else
L04170           hc=5.678*0.53*((vett/0.3048)**0.78)
L04171        endif
L04172
L04173        if (hc.gt.da*cp_u/dt)then
L04174           hc=da*cp_u/dt
L04175        endif
L04176
L04177         if (vett.lt.4.88) then
L04178            hcwin=5.678*(0.99+0.21*(vett/0.3048))
L04179         else
```

```
L04180            hcwin=5.678*0.50*((vett/0.3048)**0.78)
L04181          endif
L04182
L04183          if (hcwin.gt.da*cp_u/dt) then
L04184              hcwin=da*cp_u/dt
L04185          endif
L04186
L04187  !          tvb=hc*ptw/da/cp_u
L04188  !          tva=-hc/da/cp_u
L04189  !!!!!!!!!!!!!!!!!!!
L04190  ! explicit
L04191
L04192          sfw=hc*(pt-ptw)
L04193          sfwin=hcwin*(pt-ptwin)
L04194
L04195
L04196          evb=cdrag*(abs(u_ort)**3.)/2.
L04197
L04198          return
L04199          end subroutine flux_wall
```

讲解：墙面通量：① 动量——法向风 u_ort=|cos(drst) • ua−sin(drst) • va|，隐式项 uva=−cdrag • u_ort/2 • cos²(drst)、uvb=cdrag • u_ort/2 • sin • cos • va（正交分解的墙面拖曳，方向投影几何）；② 对流换热 hc——**经验分段式**（vett<4.88 m/s 用 McAdams 型线性式 5.678 • (1.09+0.23 • V/0.3048)，否则湍流幂律 5.678 • 0.53 • (V/0.3048)^0.78），上限钳 da • cp/dt（显式稳定极限）；窗户另用 hcwin（系数 0.99/0.21/0.50）；③TKE 源 evb=cdrag • |u_ort|³/2——墙面湍流产生项。

```
!file: phys/module_sf_bep_bem.F    L4204-L4400    (subroutine flux_flat_ground)
L04204          subroutine flux_flat_ground(dz,z0,ua,va,pt,pt0,ptg,                    &
L04205                                      uhb,vhb,sf,ehb,da,qv,pr,rsg,qg,resg,rsveg,f1,f2,f3,f4,fh,ric,utot,gr_type)
L04206
L04207  ! ------------------------------------------------------------------------
L04208  !           Calculation of the flux at the ground
L04209  !           Formulation of Louis (Louis, 1979)
L04210  ! ------------------------------------------------------------------------
L04211
L04212          implicit none
L04213
L04214          real dz                   ! first vertical level
L04215          real pt                   ! potential temperature
L04216          real pt0                  ! reference potential temperature
L04217          real ptg                  ! ground potential temperature
L04218          real ua                   ! wind speed
L04219          real va                   ! wind speed
L04220          real z0                   ! Roughness length
L04221          real da                   ! air density
L04222          real qv                   ! specific humidity
L04223          real pr                   ! pressure
L04224          real rsg                  ! solar radiation
L04225          real qg(ng_u)         ! Ground Soil Moisture
L04226
L04227
L04228
L04229  ! ------------------------------------------------------------------------
L04230  ! OUTPUT:
L04231  ! ------------------------------------------------------------------------
L04232  ! Explicit component of the momentum, temperature and TKE sources or sinks on horizontal
L04233  !  surfaces (roofs and street)
L04234  ! The fluxes can be computed as follow: Fluxes of X = B
L04235  !  Example: Momentum fluxes on horizontal surfaces =  uhb_u
L04236          real uhb                  ! U (wind component) Horizontal surfaces, B (explicit) term
L04237          real vhb                  ! V (wind component) Horizontal surfaces, B (explicit) term
L04238  !       real thb                  ! Temperature        Horizontal surfaces, B (explicit) term
L04239          real tva                  ! Temperature          Vertical surfaces, A (implicit) term
L04240          real tvb                  ! Temperature          Vertical surfaces, B (explicit) term
L04241          real ehb                  ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L04242          real sf
L04243          real lf
L04244
L04245  ! ------------------------------------------------------------------------
L04246  ! LOCAL:
L04247  ! ------------------------------------------------------------------------
L04248          real aa,ah
L04249          real z0t
L04250          real al
L04251          real buu
L04252          real c
L04253          real fbuw
L04254          real fbpt
L04255          real fh
L04256          real fm
L04257          real ric
L04258          real tstar
L04259          real qstar
L04260          real ustar
L04261          real utot
L04262          real wstar
L04263          real zz
L04264          real qvsg,qvs,es,esa,fbqq
L04265          real b,cm,ch,rr,tol
L04266          parameter(b=9.4,cm=7.4,ch=5.3,rr=0.74,tol=.001)
L04267
L04268          real f
L04269          real f1
L04270          real f2
L04271          real f3
L04272          real f4
L04273          real ta           ! surface air temperature
L04274          real tmp                  ! ground temperature
L04275          real rsveg        ! Stomatal resistance
L04276          real resg
L04277          real lai          ! leaf area index
L04278          real sdlim        ! radiation limit at which photosyntesis start W/m2
```

```
L04279        parameter(sdlim=100.)
L04280        real rsmin ! Minimum stomatal resistance
L04281        real rsmax ! Maximun stomatal resistance
L04282        real qw
L04283        parameter(qw=0.06)
L04284        real qref
L04285        parameter(qref=0.37)
L04286        real hs
L04287        parameter(hs=36.35)
L04288
L04289        real dzg_u(ng_u)          ! Layer sizes in the ground
L04290
L04291        data dzg_u /0.2,0.12,0.08,0.05,0.03,0.02,0.02,0.01,0.005,0.0025/
L04292
L04293        real gx,dzg_tot
L04294        integer gr_type,iz
L04295  ! ---------------------------------------------------------------------
L04296  ! END VARIABLES DEFINITIONS
L04297  ! ---------------------------------------------------------------------
L04298        z0t=z0/10.
L04299        if(gr_type.eq.1)then
L04300        rsmin=40.
L04301        rsmax=5000.
L04302        lai=2.
L04303        elseif(gr_type.eq.2)then
L04304        rsmin=150.
L04305        rsmax=5000.
L04306        lai=3.
L04307        endif
L04308  ! computation of the ground temperature
L04309
L04310        utot=(ua**2.+va**2.)**.5
L04311
L04312
L04313  !!!! Louis formulation
L04314  !
L04315  ! compute the bulk Richardson Number
L04316
L04317        zz=dz/2.
L04318
L04319  !        if(tstar.lt.0.)then
L04320  !         wstar=(-ustar*tstar*g*hii/pt)**(1./3.)
L04321  !        else
L04322  !         wstar=0.
L04323  !        endif
L04324  !
L04325  !      if (utot.le.0.7*wstar) utot=max(0.7*wstar,0.00001)
L04326
L04327        utot=max(utot,0.01)
L04328
L04329        ric=2.*g_u*zz*(pt-ptg)/((pt+ptg)*(utot**2))
L04330
L04331        aa=vk/log(zz/z0)
L04332        ah=vk/log(zz/z0t)
L04333
L04334  ! determine the parameters fm and fh for stable, neutral and unstable conditions
L04335
L04336        if(ric.gt.0)then
L04337           fm=1/(1+0.5*b*ric)**2.
L04338           fh=fm
L04339        else
L04340           c=b*cm*aa*aa*(zz/z0)**.5
L04341           fm=1-b*ric/(1+c*(-ric)**.5)
L04342           c=b*cm*aa*ah*(zz/z0t)**.5
L04343           c=c*ch/cm
L04344           fh=1-b*ric/(1+c*(-ric)**.5)
L04345        endif
L04346
L04347        fbuw=-aa*aa*utot*utot*fm
L04348        fbpt=-aa*ah*utot*(pt-ptg)*fh/rr
L04349        tmp=ptg*(pr/p0)**(rcp_u)-273.15
L04350        es=6.11*(10.**(tmp*7.5/(237.7+tmp)))
L04351        qvsg=0.62197*es/(0.01*pr-0.378*es)
L04352
L04353
L04354        f=0.55*rsg/sdlim*2./lai
L04355
L04356        f1=(f+rsmin/rsmax)/(1.+f)
L04357
L04358        ta=pt*(pr/p0)**(rcp_u)-273.15
L04359        esa=6.11*(10**(ta*7.5/(237.7+ta)))
L04360        qvs=0.62197*esa/(0.01*pr-0.378*esa)
L04361
L04362        f2= 1./(1.+hs*(qvs-qv))
L04363        f3=1.-0.0016*(25.-ta)**2.
L04364        f4=0.
L04365        dzg_tot=0.
L04366        do iz=1,ng_u
L04367         gx=(qg(iz)-qw)/(qref-qw)
L04368         if (gx.gt.1)gx=1.
L04369         if (gx.lt.0)gx=0.
L04370         f4=f4+gx*dzg_u(iz)
L04371         dzg_tot=dzg_tot+dzg_u(iz)
L04372        enddo
L04373        f4=f4/dzg_tot
L04374
L04375        rsveg=min(rsmin/max(lai*f1*f2*f3*f4,1e-9),rsmax)
L04376        resg= rr/(aa*aa*utot*fh)
L04377
L04378
L04379        fbqq=-(qv-qvsg)/(resg+rsveg)
L04380
L04381
L04382        ustar=(-fbuw)**.5
L04383        tstar=-fbpt/ustar
L04384        qstar=-fbqq/ustar
L04385
L04386        al=(vk*g_u*tstar)/(pt*ustar*ustar)
L04387
```

```
L04388        buu=-g_u/pt0*ustar*tstar
L04389
L04390        uhb=-ustar*ustar*ua/utot
L04391        vhb=-ustar*ustar*va/utot
L04392        sf= ustar*tstar*da*cp_u
L04393        lf= ustar*qstar*da*latent
L04394
L04395  !     thb= 0.
L04396        ehb=buu
L04397  !!!!!!!!!!!!!!
L04398
L04399        return
L04400        end subroutine flux_flat_ground
```

讲解：带植被气孔阻抗的地面通量（Louis 1979 骨架 + Jarvis 型 rsveg=f1 • f2 • f3 • f4 四因子：辐射/湿度亏空/温度/土壤含水量），gr_type=1/2 两套 rsmin/LAI。**注意：4.8.0 中本单元无任何调用方**（buildings 只调 flux_flat 与 flux_flat_roof）——gr_type 的植被参数实际经 flux_flat_roof 生效，本单元是遗留入口，改造时勿白改。

```
!file: phys/module_sf_bep_bem.F    L4404-L4589    (subroutine flux_flat_roof)
L04404        subroutine flux_flat_roof(dz,z0,ua,va,pt,pt0,ptg,                    &
L04405                             uhb,vhb,sf,lf,ehb,da,qv,pr,rsg,qr,resg,rsveg,f1,f2,f3,f4,gr_type,pv_frac_roof)
L04406
L04407  ! ------------------------------------------------------------------
L04408  !           Calculation of the flux at the ground
L04409  !           Formulation of Louis (Louis, 1979)
L04410  ! ------------------------------------------------------------------
L04411
L04412        implicit none
L04413
L04414        real dz                   ! first vertical level
L04415        real pt                   ! potential temperature
L04416        real pt0                  ! reference potential temperature
L04417        real ptg                  ! ground potential temperature
L04418        real ua                   ! wind speed
L04419        real va                   ! wind speed
L04420        real z0                   ! Roughness length
L04421        real da                   ! air density
L04422        real qv                   ! specific humidity
L04423        real pr                   ! pressure
L04424        real rsg                  ! solar radiation
L04425        real qr(ngr_u)         ! Ground Soil Moisture
L04426        real pv_frac_roof
L04427        real rs_eff
L04428
L04429  ! ------------------------------------------------------------------
L04430  ! OUTPUT:
L04431  ! ------------------------------------------------------------------
L04432  ! Explicit component of the momentum, temperature and TKE sources or sinks on horizontal
L04433  !  surfaces (roofs and street)
L04434  ! The fluxes can be computed as follow: Fluxes of X = B
L04435  !  Example: Momentum fluxes on horizontal surfaces =  uhb_u
L04436        real uhb                  ! U (wind component) Horizontal surfaces, B (explicit) term
L04437        real vhb                  ! V (wind component) Horizontal surfaces, B (explicit) term
L04438  !     real thb                  ! Temperature        Horizontal surfaces, B (explicit) term
L04439        real tva                  ! Temperature          Vertical surfaces, A (implicit) term
L04440        real tvb                  ! Temperature          Vertical surfaces, B (explicit) term
L04441        real ehb                  ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L04442        real sf
L04443        real lf
L04444
L04445  ! ------------------------------------------------------------------
L04446  ! LOCAL:
L04447  ! ------------------------------------------------------------------
L04448        real aa,ah
L04449        real al
L04450        real buu
L04451        real c
L04452        real fbuw
L04453        real fbpt
L04454        real fh
L04455        real fm
L04456        real ric
L04457        real tstar
L04458        real qstar
L04459        real ustar
L04460        real utot
L04461        real wstar
L04462        real zz
L04463        real z0t
L04464        real qvsg,qvs,es,esa,fbqq
L04465        real b,cm,ch,rr,tol
L04466        parameter(b=9.4,cm=7.4,ch=5.3,rr=0.74,tol=.001)
L04467
L04468        real f
L04469        real f1
L04470        real f2
L04471        real f3
L04472        real f4
L04473        real ta          ! surface air temperature
L04474        real tmp                 ! ground temperature
L04475        real rsveg       ! Stomatal resistance
L04476        real resg
L04477        real lai         ! Leaft area index
L04478        real sdlim       ! radiation limit at which photosyntesis start W/m2
L04479        parameter(sdlim=100.)
L04480        real rsmin
L04481        real rsmax ! Maximun stomatal resistance
L04482        real qw    ! Wilting point
L04483        parameter(qw=0.06)
L04484        real qref  ! Field capacity
L04485        parameter(qref=0.37)
L04486        real hs
L04487        parameter(hs=36.35)
L04488
```

```
L04489        real dzgr_u(ngr_u)          ! Layer sizes in the ground
L04490
L04491        data dzgr_u /0.1,0.003,0.06,0.003,0.05,0.04,0.02,0.0125,0.005,0.0025/
L04492
L04493        real gx,dzgr_tot
L04494        integer gr_type,iz
L04495  ! ------------------------------------------------------------------------
L04496  ! END VARIABLES DEFINITIONS
L04497
L04498  ! ------------------------------------------------------------------------
L04499        z0t=z0/10.
L04500        if(gr_type.eq.1)then
L04501        rsmin=40.
L04502        rsmax=5000.
L04503        lai=2.
L04504        elseif(gr_type.eq.2)then
L04505        rsmin=150.
L04506        rsmax=5000.
L04507        lai=3.
L04508        endif
L04509       rs_eff=(1-pv_frac_roof)*rsg
L04510  ! computation of the ground temperature
L04511
L04512        utot=(ua**2.+va**2.)**.5
L04513
L04514  !!!! Louis formulation
L04515  !
L04516  ! compute the bulk Richardson Number
L04517
L04518        zz=dz/2.
L04519
L04520
L04521        utot=max(utot,0.01)
L04522
L04523        ric=2.*g_u*zz*(pt-ptg)/((pt+ptg)*(utot**2))
L04524
L04525        aa=vk/log(zz/z0)
L04526        ah=vk/log(zz/z0t)
L04527
L04528        if(ric.gt.0.)then
L04529           fm=1./(1.+0.5*b*ric)**2.
L04530           fh=fm
L04531        else
L04532           c=b*cm*aa*aa*(zz/z0)**.5
L04533           fm=1.-b*ric/(1.+c*(-ric)**.5)
L04534           c=b*cm*aa*ah*(zz/z0t)**.5
L04535           c=c*ch/cm
L04536           fh=1.-b*ric/(1+c*(-ric)**.5)
L04537        endif
L04538
L04539        fbuw=-aa*aa*utot*utot*fm
L04540        fbpt=-aa*ah*utot*(pt-ptg)*fh/rr
L04541        tmp=ptg*(pr/p0)**(rcp_u)-273.15
L04542        es=6.11*(10.**(tmp*7.5/(237.7+tmp)))
L04543        qvsg=0.62197*es/(0.01*pr-0.378*es)
L04544
L04545
L04546        f=0.55*rs_eff/sdlim*2./lai
L04547
L04548        f1=(f+rsmin/rsmax)/(1.+f)
L04549
L04550        ta=pt*(pr/p0)**(rcp_u)-273.15
L04551        esa=6.11*(10**(ta*7.5/(237.7+ta)))
L04552        qvs=0.62197*esa/(0.01*pr-0.378*esa)
L04553
L04554        f2= 1./(1.+hs*(qvs-qv))
L04555        f3=1.-0.0016*(25.-ta)**2.
L04556        f4=0.
L04557        dzgr_tot=0.
L04558        do iz=5,ngr_u
L04559         gx=(qr(iz)-qw)/(qref-qw)
L04560         if (gx.gt.1)gx=1.
L04561         if (gx.lt.0)gx=0.
L04562         f4=f4+gx*dzgr_u(iz)
L04563         dzgr_tot=dzgr_tot+dzgr_u(iz)
L04564        enddo
L04565        f4=f4/dzgr_tot
L04566
L04567        rsveg=min(rsmin/max(lai*f1*f2*f3*f4,1e-9),rsmax)
L04568
L04569
L04570        resg= rr/(aa*aa*utot*fh)
L04571
L04572
L04573        fbqq=-(qv-qvsg)/(resg+rsveg)
L04574
L04575        ustar=(-fbuw)**.5
L04576        tstar=-fbpt/ustar
L04577        qstar=-fbqq/ustar
L04578
L04579        al=(vk*g_u*tstar)/(pt*ustar*ustar)
L04580
L04581        buu=-g_u/pt0*ustar*tstar
L04582
L04583        uhb=-ustar*ustar*ua/utot
L04584        vhb=-ustar*ustar*va/utot
L04585        sf= ustar*tstar*da*cp_u
L04586        lf= ustar*qstar*da*latent
L04587
L04588        ehb=buu
L04589        end subroutine flux_flat_roof
```

讲解：绿屋顶通量：Louis 1979 动量/热量 + 气孔阻抗（f4 用第 5 层起的土壤含水量加权，dzgr_u 十层与 roof_temp_veg 一致）；PV 占比缩放有效短波 rs_eff=(1−pv) • rsg。

```
!file: phys/module_sf_bep_bem.F    L4596-L4716    (subroutine flux_flat)
L04596        subroutine flux_flat(dz,z0,ua,va,pt,pt0,ptg,qv,                  &
L04597                             uhb,vhb,sf,lf,ehb,da,pr)
L04598
L04599  ! ------------------------------------------------------------------
L04600  !           Calculation of the flux at the ground
L04601  !           Formulation of Louis (Louis, 1979)
L04602  ! ------------------------------------------------------------------
L04603
L04604        implicit none
L04605        real pr
L04606        real dz                  ! first vertical level
L04607        real pt                  ! potential temperature
L04608        real pt0                 ! reference potential temperature
L04609        real ptg                 ! ground potential temperature
L04610        real ua                  ! wind speed
L04611        real va                  ! wind speed
L04612        real z0                  ! Roughness length
L04613        real da                  ! air density
L04614        real qv
L04615  ! ------------------------------------------------------------------
L04616  ! OUTPUT:
L04617  ! ------------------------------------------------------------------
L04618  ! Explicit component of the momentum, temperature and TKE sources or sinks on horizontal
L04619  !  surfaces (roofs and street)
L04620  ! The fluxes can be computed as follow: Fluxes of X = B
L04621  !  Example: Momentum fluxes on horizontal surfaces =  uhb_u
L04622        real uhb                 ! U (wind component) Horizontal surfaces, B (explicit) term
L04623        real vhb                 ! V (wind component) Horizontal surfaces, B (explicit) term
L04624  !     real thb                 ! Temperature        Horizontal surfaces, B (explicit) term
L04625        real tva                 ! Temperature          Vertical surfaces, A (implicit) term
L04626        real tvb                 ! Temperature          Vertical surfaces, B (explicit) term
L04627        real ehb                 ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L04628        real sf
L04629        real lf
L04630
L04631  ! ------------------------------------------------------------------
L04632  ! LOCAL:
L04633  ! ------------------------------------------------------------------
L04634        real aa
L04635        real al
L04636        real buu
L04637        real c
L04638        real fbuw
L04639        real fbpt
L04640        real fh
L04641        real fm
L04642        real ric
L04643        real tstar
L04644        real ustar
L04645        real qstar
L04646        real utot
L04647        real wstar
L04648        real zz
L04649        real qvsg,qvs,es,esa,fbqq,tmp,resg
L04650        real b,cm,ch,rr,tol
L04651        parameter(b=9.4,cm=7.4,ch=5.3,rr=0.74,tol=.001)
L04652
L04653  ! ------------------------------------------------------------------
L04654  ! END VARIABLES DEFINITIONS
L04655  ! ------------------------------------------------------------------
L04656
L04657
L04658  ! computation of the ground temperature
L04659
L04660        utot=(ua**2+va**2)**.5
L04661
L04662
L04663  !!!! Louis formulation
L04664  !
L04665  ! compute the bulk Richardson Number
L04666
L04667        zz=dz/2.
L04668
L04669
L04670        utot=max(utot,0.01)
L04671
L04672        ric=2.*g_u*zz*(pt-ptg)/((pt+ptg)*(utot**2))
L04673
L04674        aa=vk/log(zz/z0)
L04675
L04676
L04677
L04678        tmp=ptg*(pr/(1.e+5))**(rcp_u)-273.15
L04679        es=6.11*(10**(tmp*7.5/(237.7+tmp)))
L04680        qvsg=0.62197*es/(0.01*pr-0.378*es)
L04681
L04682
L04683
L04684  ! determine the parameters fm and fh for stable, neutral and unstable conditions
L04685
L04686        if(ric.gt.0.)then
L04687           fm=1./(1.+0.5*b*ric)**2
L04688           fh=fm
L04689        else
L04690           c=b*cm*aa*aa*(zz/z0)**.5
L04691           fm=1.-b*ric/(1.+c*(-ric)**.5)
L04692           c=c*ch/cm
L04693           fh=1.-b*ric/(1.+c*(-ric)**.5)
L04694        endif
L04695
L04696        resg= rr/(aa*aa*utot*fh)
L04697        fbuw=-aa*aa*utot*utot*fm
L04698        fbpt=-aa*aa*utot*(pt-ptg)*fh/rr
L04699        fbqq=-(qv-qvsg)/(resg)
L04700
L04701        ustar=(-fbuw)**.5
L04702        tstar=-fbpt/ustar
L04703        qstar=-fbqq/ustar
```

```
L04704        al=(vk*g_u*tstar)/(pt*ustar*ustar)
L04705
L04706        buu=-g_u/pt0*ustar*tstar
L04707
L04708        uhb=-ustar*ustar*ua/utot
L04709        vhb=-ustar*ustar*va/utot
L04710        sf= ustar*tstar*da*cp_u
L04711        lf= ustar*qstar*da*latent
L04712        ehb=buu
L04713  !!!!!!!!!!!!!!
L04714
L04715        return
L04716        end subroutine flux_flat
```

讲解：普通路面/屋顶通量（Louis 1979 全套：bulk Ri 分区 fm/fh、稳定 1/(1+0.5 • b • Ri)²、不稳定 1−b • Ri/(1+c • (−Ri)^½)；b=9.4、cm=7.4、ch=5.3、rr=0.74 为 Louis 常数）；潜热用水面饱和 qvsg 无阻抗（积水处理在 buildings 的 wfg/wfr 缩放）。module_sf_bep.F 内有同名单独拷贝（改需同步）。

六、初始化与视野因子

```
!file: phys/module_sf_bep_bem.F    L4721-L4820    (subroutine icBEP)
L04721        subroutine icBEP (nd_u,h_b,d_b,ss_u,pb_u,nz_u,z_u)
L04722
L04723        implicit none
L04724
L04725  !    Street parameters
L04726        integer nd_u(nurbm)     ! Number of street direction for each urban class
L04727        real h_b(nz_um,nurbm)   ! Bulding's heights [m]
L04728        real d_b(nz_um,nurbm)   ! The probability that a building has an height h_b
L04729  ! ----------------------------------------------------------------------
L04730  !     Output
L04731  !----------------------------------------------------------------------
L04732
L04733        real ss_u(nz_um,nurbm)     ! The probability that a building has an height equal to z
L04734        real pb_u(nz_um,nurbm)     ! The probability that a building has an height greater or equal to z
L04735
L04736  !    Grid parameters
L04737        integer nz_u(nurbm)     ! Number of layer in the urban grid
L04738        real z_u(nz_um)       ! Height of the urban grid levels
L04739
L04740
L04741  ! ----------------------------------------------------------------------
L04742  !     Local
L04743  !----------------------------------------------------------------------
L04744
L04745        integer iz_u,id,ilu,iurb
L04746
L04747        real dtot
L04748        real hbmax
L04749
L04750  ! ----------------------------------------------------------------------
L04751  !     This routine initialise the urban paramters for the BEP module
L04752  !----------------------------------------------------------------------
L04753  !
L04754  !Initialize variables
L04755  !
L04756   !
L04757        nz_u=0
L04758        z_u=0.
L04759        ss_u=0.
L04760        pb_u=0.
L04761
L04762  ! Computation of the urban levels height
L04763
L04764        z_u(1)=0.
L04765
L04766        do iz_u=1,nz_um-1
L04767           z_u(iz_u+1)=z_u(iz_u)+dz_u
L04768        enddo
L04769
L04770  ! Normalisation of the building density
L04771
L04772        do iurb=1,nurbm
L04773           dtot=0.
L04774           do ilu=1,nz_um
L04775              dtot=dtot+d_b(ilu,iurb)
L04776           enddo
L04777           do ilu=1,nz_um
L04778              d_b(ilu,iurb)=d_b(ilu,iurb)/dtot
L04779           enddo
L04780        enddo
L04781
L04782  ! Compute the view factors, pb and ss
L04783
L04784        do iurb=1,nurbm
L04785           hbmax=0.
L04786           nz_u(iurb)=0
L04787           do ilu=1,nz_um
L04788              if(h_b(ilu,iurb).gt.hbmax)hbmax=h_b(ilu,iurb)
L04789           enddo
L04790
L04791           do iz_u=1,nz_um-1
L04792              if(z_u(iz_u+1).gt.hbmax)go to 10
L04793           enddo
L04794
L04795   10      continue
L04796            nz_u(iurb)=iz_u+1
L04797
L04798           do id=1,nd_u(iurb)
L04799
L04800              do iz_u=1,nz_u(iurb)
L04801                 ss_u(iz_u,iurb)=0.
```

```
L04802                do ilu=1,nz_um
L04803                   if(z_u(iz_u).le.h_b(ilu,iurb)                    &
L04804                     .and.z_u(iz_u+1).gt.h_b(ilu,iurb))then
L04805                        ss_u(iz_u,iurb)=ss_u(iz_u,iurb)+d_b(ilu,iurb)
L04806                   endif
L04807                enddo
L04808             enddo
L04809
L04810             pb_u(1,iurb)=1.
L04811             do iz_u=1,nz_u(iurb)
L04812                pb_u(iz_u+1,iurb)=max(0.,pb_u(iz_u,iurb)-ss_u(iz_u,iurb))
L04813             enddo
L04814
L04815          enddo
L04816       end do
L04817
L04818
L04819       return
L04820       end subroutine icBEP
```

讲解：类级初始化：城市网格高度 z_u=5 m 均匀 18 层；建筑高度谱 d_b 归一化；每类从最高楼定 nz_u；按「楼高落在哪层」折算该层有屋顶的概率 ss_u，pb_u=1− 累计 ss（pb=该高度以上有建筑的概率——「存在概率」的补集定义）。

```
!file: phys/module_sf_bep_bem.F    L4826-L4985    (subroutine view_factors)
L04826       subroutine view_factors(iurb,nz_u,id,dxy,z,ws,fww,fwg,fgw,fsg,fsw,fws)
L04827
L04828       implicit none
L04829
L04830
L04831
L04832  ! ------------------------------------------------------------------------
L04833  !     Input
L04834  !------------------------------------------------------------------------
L04835
L04836       integer iurb            ! Number of the urban class
L04837       integer nz_u            ! Number of levels in the urban grid
L04838       integer id              ! Street direction number
L04839       real ws                 ! Street width
L04840       real z(nz_um)         ! Height of the urban grid levels
L04841       real dxy                ! Street lenght
L04842
L04843
L04844  ! ------------------------------------------------------------------------
L04845  !     Output
L04846  !------------------------------------------------------------------------
L04847
L04848  !   fww,fwg,fgw,fsw,fsg are the view factors used to compute the long wave
L04849  !   and the short wave radation. They are the part of radiation from a surface
L04850  !   or from the sky to another surface.
L04851
L04852       real fww(nz_um,nz_um,ndm,nurbm)            !  from wall to wall
L04853       real fwg(nz_um,ndm,nurbm)                  !  from wall to ground
L04854       real fgw(nz_um,ndm,nurbm)                  !  from ground to wall
L04855       real fsw(nz_um,ndm,nurbm)                  !  from sky to wall
L04856       real fws(nz_um,ndm,nurbm)                  !  from wall to sky
L04857       real fsg(ndm,nurbm)                        !  from sky to ground
L04858
L04859
L04860  ! ------------------------------------------------------------------------
L04861  !     Local
L04862  !------------------------------------------------------------------------
L04863
L04864       integer jz,iz
L04865
L04866       real hut
L04867       real f1,f2,f12,f23,f123,ftot
L04868       real fprl,fnrm
L04869       real a1,a2,a3,a4,a12,a23,a123
L04870
L04871  ! ------------------------------------------------------------------------
L04872  !     This routine calculates the view factors
L04873  !------------------------------------------------------------------------
L04874
L04875       hut=z(nz_u+1)
L04876
L04877       do jz=1,nz_u
L04878
L04879  ! radiation from wall to wall
L04880
L04881          do iz=1,nz_u
L04882
L04883             call fprls (fprl,dxy,abs(z(jz+1)-z(iz  )),ws)
L04884             f123=fprl
L04885             call fprls (fprl,dxy,abs(z(jz+1)-z(iz+1)),ws)
L04886             f23=fprl
L04887             call fprls (fprl,dxy,abs(z(jz  )-z(iz  )),ws)
L04888             f12=fprl
L04889             call fprls (fprl,dxy,abs(z(jz  )-z(iz+1)),ws)
L04890             f2 = fprl
L04891
L04892             a123=dxy*(abs(z(jz+1)-z(iz  )))
L04893             a12 =dxy*(abs(z(jz  )-z(iz  )))
L04894             a23 =dxy*(abs(z(jz+1)-z(iz+1)))
L04895             a1  =dxy*(abs(z(iz+1)-z(iz  )))
L04896             a2  =dxy*(abs(z(jz  )-z(iz+1)))
L04897             a3  =dxy*(abs(z(jz+1)-z(jz  )))
L04898
L04899             ftot=0.5*(a123*f123-a23*f23-a12*f12+a2*f2)/a1
L04900
L04901             fww(iz,jz,id,iurb)=ftot*a1/a3
L04902
L04903          enddo
L04904
L04905  ! radiation from ground to wall
L04906
```

```
L04907           call fnrms (fnrm,z(jz+1),dxy,ws)
L04908           f12=fnrm
L04909           call fnrms (fnrm,z(jz)  ,dxy,ws)
L04910           f1=fnrm
L04911
L04912           a1 = ws*dxy
L04913
L04914           a12= ws*dxy
L04915
L04916           a4=(z(jz+1)-z(jz))*dxy
L04917
L04918           ftot=(a12*f12-a12*f1)/a1
L04919
L04920           fgw(jz,id,iurb)=ftot*a1/a4
L04921
L04922  !  radiation from sky to wall
L04923
L04924           call fnrms(fnrm,hut-z(jz)  ,dxy,ws)
L04925           f12 = fnrm
L04926           call fnrms (fnrm,hut-z(jz+1),dxy,ws)
L04927           f1 =fnrm
L04928
L04929           a1 = ws*dxy
L04930
L04931           a12= ws*dxy
L04932
L04933           a4 = (z(jz+1)-z(jz))*dxy
L04934
L04935           ftot=(a12*f12-a12*f1)/a1
L04936
L04937           fsw(jz,id,iurb)=ftot*a1/a4
L04938
L04939         enddo
L04940
L04941  ! radiation from wall to sky
L04942          do iz=1,nz_u
L04943          call fnrms(fnrm,ws,dxy,hut-z(iz))
L04944          f12=fnrm
L04945          call fnrms(fnrm,ws,dxy,hut-z(iz+1))
L04946          f1=fnrm
L04947          a1 = (z(iz+1)-z(iz))*dxy
L04948          a2 = (hut-z(iz+1))*dxy
L04949          a12= (hut-z(iz))*dxy
L04950          a4 = ws*dxy
L04951          ftot=(a12*f12-a2*f1)/a1
L04952          fws(iz,id,iurb)=ftot*a1/a4
L04953
L04954         enddo
L04955  !!!!!!!!!!!!!
L04956
L04957
L04958          do iz=1,nz_u
L04959
L04960  ! radiation from wall to ground
L04961
L04962           call fnrms (fnrm,ws,dxy,z(iz+1))
L04963           f12=fnrm
L04964           call fnrms (fnrm,ws,dxy,z(iz  ))
L04965           f1 =fnrm
L04966
L04967           a1= (z(iz+1)-z(iz) )*dxy
L04968
L04969           a2 = z(iz)*dxy
L04970           a12= z(iz+1)*dxy
L04971           a4 = ws*dxy
L04972
L04973           ftot=(a12*f12-a2*f1)/a1
L04974
L04975           fwg(iz,id,iurb)=ftot*a1/a4
L04976
L04977         enddo
L04978
L04979  ! radiation from sky to ground
L04980
L04981         call fprls (fprl,dxy,ws,hut)
L04982         fsg(id,iurb)=fprl
L04983
L04984         return
L04985         end subroutine view_factors
```

讲解：峡谷视野因子全解析计算（BLM 论文附录 A）：fww（墙-墙）用两平行矩形角系数 fprls 的**差分组合** ½(a123 • f123−a23 • f23−a12 • f12+a2 • f2)/a1（矩形代数）；fwg/fsw/fws/fwg 用正交面角系数 fnrms 的同样差分技术；fsg（天空-地面）直接 fprls(ws, dxy, hut)。全部几何量解析——不随时间变，icBEP_XY 每点每步重算是因为它们依赖逐点的 ws/strd。

```
!file: phys/module_sf_bep_bem.F    L4991-L5016    (subroutine fprls)
L04991         SUBROUTINE fprls (fprl,a,b,c)
L04992
L04993         implicit none
L04994
L04995
L04996
L04997         real a,b,c
L04998         real x,y
L04999         real fprl
L05000
L05001
L05002         x=a/c
L05003         y=b/c
L05004
L05005         if(a.eq.0.or.b.eq.0.)then
L05006          fprl=0.
L05007         else
L05008          fprl=log( ( (1.+x**2)*(1.+y**2)/(1.+x**2+y**2) )**.5)+  &
L05009              y*((1.+x**2)**.5)*atan(y/((1.+x**2)**.5))+          &
```

```
L05010            x*((1.+y**2)**.5)*atan(x/((1.+y**2)**.5))-          &
L05011            y*atan(y)-x*atan(x)
L05012         fprl=fprl*2./(pi*x*y)
L05013        endif
L05014
L05015        return
L05016        end subroutine fprls
```

```
!file: phys/module_sf_bep_bem.F    L5021-L5049    (subroutine fnrms)
L05021        SUBROUTINE fnrms (fnrm,a,b,c)
L05022
L05023        implicit none
L05024
L05025
L05026
L05027        real a,b,c
L05028        real x,y,z,a1,a2,a3,a4,a5,a6
L05029        real fnrm
L05030
L05031        x=a/b
L05032        y=c/b
L05033        z=x**2+y**2
L05034
L05035        if(y.eq.0.or.x.eq.0)then
L05036         fnrm=0.
L05037        else
L05038         a1=log( (1.+x*x)*(1.+y*y)/(1.+z) )
L05039         a2=y*y*log(y*y*(1.+z)/z/(1.+y*y) )
L05040         a3=x*x*log(x*x*(1.+z)/z/(1.+x*x) )
L05041         a4=y*atan(1./y)
L05042         a5=x*atan(1./x)
L05043         a6=sqrt(z)*atan(1./sqrt(z))
L05044         fnrm=0.25*(a1+a2+a3)+a4+a5-a6
L05045         fnrm=fnrm/(pi*y)
L05046        endif
L05047
L05048        return
L05049        end subroutine fnrms
```

讲解：两个解析角系数公式：fprls=两平行等长矩形（教科书标准式，atan 组合）；fnrms=垂直矩形对（含 6 项对数/atan）。零面积保护出口。

```
!file: phys/module_sf_bep_bem.F    L5052-L5212    (subroutine init_para)
L05052          SUBROUTINE init_para(alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,&
L05053          twini_u,trini_u,tgini_u,albg_u,albw_u,albr_u,albwin_u,emg_u,emw_u,&
L05054          emr_u,emwind_u,z0g_u,z0r_u,nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b,  &
L05055          cop_u,pwin_u,beta_u,sw_cond_u,time_on_u,time_off_u,targtemp_u,    &
L05056          bldac_frc_u,cooled_frc_u,                                        &
L05057          gaptemp_u, targhum_u,gaphum_u,perflo_u,                          &
L05058          gr_frac_roof_u,pv_frac_roof_u,                &
L05059          hsesf_u,hsequip,irho,gr_flag_u,gr_type_u)
L05060
L05061
L05062  ! initialization routine, where the variables from the table are read
L05063
L05064        implicit none
L05065        integer iurb             ! urban class number
L05066  !    Building parameters
L05067        real alag_u(nurbm)       ! Ground thermal diffusivity [m^2 s^-1]
L05068        real alaw_u(nurbm)       ! Wall thermal diffusivity [m^2 s^-1]
L05069        real alar_u(nurbm)       ! Roof thermal diffusivity [m^2 s^-1]
L05070        real csg_u(nurbm)        ! Specific heat of the ground material [J m^3 K^-1]
L05071        real csw_u(nurbm)        ! Specific heat of the wall material [J m^3 K^-1]
L05072        real csr_u(nurbm)        ! Specific heat of the roof material [J m^3 K^-1]
L05073        real twini_u(nurbm)      ! Temperature inside the buildings behind the wall [K]
L05074        real trini_u(nurbm)      ! Temperature inside the buildings behind the roof [K]
L05075        real tgini_u(nurbm)      ! Initial road temperature
L05076
L05077  !    Radiation parameters
L05078        real albg_u(nurbm)       ! Albedo of the ground
L05079        real albw_u(nurbm)       ! Albedo of the wall
L05080        real albr_u(nurbm)       ! Albedo of the roof
L05081        real albwin_u(nurbm)     ! Albedo of the window
L05082        real emg_u(nurbm)        ! Emissivity of ground
L05083        real emw_u(nurbm)        ! Emissivity of wall
L05084        real emr_u(nurbm)        ! Emissivity of roof
L05085        real emwind_u(nurbm)     ! Emissivity of windows
L05086
L05087  !    Roughness parameters
L05088        real z0g_u(nurbm)        ! The ground's roughness length
L05089        real z0r_u(nurbm)        ! The roof's roughness length
L05090
L05091  !    Street parameters
L05092        integer nd_u(nurbm)      ! Number of street direction for each urban class
L05093
L05094        real strd_u(ndm,nurbm)   ! Street length (fix to greater value to the horizontal length of the cells)
L05095        real drst_u(ndm,nurbm)   ! Street direction [degree]
L05096        real ws_u(ndm,nurbm)     ! Street width [m]
L05097        real bs_u(ndm,nurbm)     ! Building width [m]
L05098        real h_b(nz_um,nurbm)    ! Bulding's heights [m]
L05099        real d_b(nz_um,nurbm)    ! The probability that a building has an height h_b
L05100
L05101        integer i,iu
L05102        integer nurb ! number of urban classes used
L05103          real, intent(out) :: bldac_frc_u(nurbm)
L05104        real, intent(out) :: cooled_frc_u(nurbm)
L05105        real, intent(out) :: cop_u(nurbm)
L05106        real, intent(out) :: pwin_u(nurbm)
L05107        real, intent(out) :: beta_u(nurbm)
L05108        integer, intent(out) :: sw_cond_u(nurbm)
L05109        real, intent(out) :: time_on_u(nurbm)
L05110        real, intent(out) :: time_off_u(nurbm)
L05111        real, intent(out) :: targtemp_u(nurbm)
L05112        real, intent(out) :: gaptemp_u(nurbm)
```

```
L05113        real, intent(out) :: targhum_u(nurbm)
L05114        real, intent(out) :: gaphum_u(nurbm)
L05115        real, intent(out) :: perflo_u(nurbm)
L05116        real, intent(out) :: gr_frac_roof_u(nurbm)
L05117        real, intent(out) :: pv_frac_roof_u(nurbm)
L05118        real, intent(out) :: hsesf_u(nurbm)
L05119        real, intent(out) :: hsequip(24)
L05120        real, intent(out) :: irho(24)
L05121       integer, intent(out) :: gr_flag_u,gr_type_u
L05122  !
L05123  !Initialize some variables
L05124  !
L05125
L05126         h_b=0.
L05127         d_b=0.
L05128
L05129         nurb=ICATE
L05130         do iu=1,nurb
L05131            nd_u(iu)=0
L05132         enddo
L05133
L05134         csw_u=CAPB_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L05135         csr_u=CAPR_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L05136         csg_u=CAPG_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L05137         do i=1,icate
L05138           alaw_u(i)=AKSB_TBL(i) / csw_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L05139           alar_u(i)=AKSR_TBL(i) / csr_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L05140           alag_u(i)=AKSG_TBL(i) / csg_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L05141         enddo
L05142         twini_u=TBLEND_TBL
L05143         trini_u=TRLEND_TBL
L05144         tgini_u=TGLEND_TBL
L05145         albw_u=ALBB_TBL
L05146         albr_u=ALBR_TBL
L05147         albg_u=ALBG_TBL
L05148         emw_u=EPSB_TBL
L05149         emr_u=EPSR_TBL
L05150         emg_u=EPSG_TBL
L05151         z0r_u=Z0R_TBL
L05152         z0g_u=Z0G_TBL
L05153         nd_u=NUMDIR_TBL
L05154  !FS
L05155       !  print*, 'g alla call', gr_frac_roof_u(iurb)
L05156         bldac_frc_u = bldac_frc_tbl
L05157         cooled_frc_u = cooled_frc_tbl
L05158         cop_u = cop_tbl
L05159         pwin_u = pwin_tbl
L05160         beta_u = beta_tbl
L05161         sw_cond_u = sw_cond_tbl
L05162         time_on_u = time_on_tbl
L05163         time_off_u = time_off_tbl
L05164         targtemp_u = targtemp_tbl
L05165         gaptemp_u = gaptemp_tbl
L05166         targhum_u = targhum_tbl
L05167         gaphum_u = gaphum_tbl
L05168         perflo_u = perflo_tbl
L05169         gr_frac_roof_u =gr_frac_roof_tbl
L05170         gr_flag_u=gr_flag_tbl
L05171         pv_frac_roof_u = pv_frac_roof_tbl
L05172         hsesf_u = hsesf_tbl
L05173         hsequip = hsequip_tbl
L05174         irho=irho_tbl
L05175         gr_type_u=gr_type_tbl
L05176         do iu=1,icate
L05177                if(ndm.lt.nd_u(iu))then
L05178                  write(*,*)'ndm too small in module_sf_bep_bem, please increase to at least ', nd_u(iu)
L05179                  write(*,*)'remember also that num_urban_layers should be equal or greater than nz_um*ndm*nwr-u!'
L05180                  stop
L05181                endif
L05182           do i=1,nd_u(iu)
L05183             drst_u(i,iu)=STREET_DIRECTION_TBL(i,iu) * pi/180.
L05184             ws_u(i,iu)=STREET_WIDTH_TBL(i,iu)
L05185             bs_u(i,iu)=BUILDING_WIDTH_TBL(i,iu)
L05186           enddo
L05187         enddo
L05188         do iu=1,ICATE
L05189            if(nz_um.lt.numhgt_tbl(iu)+3)then
L05190                write(*,*)'nz_um too small in module_sf_bep, please increase to at least ',numhgt_tbl(iu)+3
L05191                write(*,*)'remember also that num_urban_layers should be equal or greater than nz_um*ndm*nwr-u!'
L05192                stop
L05193             endif
L05194           do i=1,NUMHGT_TBL(iu)
L05195             h_b(i,iu)=HEIGHT_BIN_TBL(i,iu)
L05196             d_b(i,iu)=HPERCENT_BIN_TBL(i,iu)
L05197           enddo
L05198         enddo
L05199
L05200         do i=1,ndm
L05201          do iu=1,nurbm
L05202           strd_u(i,iu)=100000.
L05203          enddo
L05204         enddo
L05205
L05206         do iu=1,nurb
L05207            emwind_u(iu)=0.9
L05208            call albwindow(albwin_u(iu))
L05209         enddo
L05210
L05211         return
L05212         end subroutine init_para
```

讲解：urban_param.tbl → SI 的换算清单：热容 CAP*_TBL/(4.1868⁻¹ • 1e−6)（cal • cm⁻³ • K⁻¹→J • m⁻³ • K⁻¹）、导热率 AKS*_TBL 再除热容得扩散率；BEM 全套开关（cop/pwin/beta/sw_cond/time_on/off/targtemp/gaptemp/targhum/gaphum/perflo/bldac_frc/cooled_frc/gr_frac_roof/pv_frac_roof/hsesf/hsequip 24h 设备热谱/irho 24h 灌溉窗/gr_type）从 TBL 整体搬运；街道方向角转弧度；**emwind 硬**

编码 **0.9**、窗户反照率由 albwindow 数值积分算出。ICATE（实际类数）来自 module_sf_urban。论文 **2** 的标定参数（**α/ε/λ/d**）很多能对应到这些 **TBL** 量——改造时先在这里建「逐格场」的进场路径。

七、辐射诊断与窗户光学

```
!file: phys/module_sf_bep_bem.F    L5218-L5354    (subroutine upward_rad)
L05218         subroutine upward_rad(ndu,nzu,ws,bs,sigma,pb,ss,                &
L05219                               tg_av,emg_u,albg_u,rlg,rsg,sfg,lfg,              &
L05220                               tw,emw_u,albw_u,rlw,rsw,sfw,                &
L05221                               tr_av,emr_u,albr_u,emwind,albwind,twlev,pwin,     &
L05222                               sfwind,rld,rs, sfr,sfrv,lfr,lfrv,                      &
L05223                               rs_abs,rl_up,emiss,grdflx_urb,gr_frac_roof,tpvlev,pv_frac_roof)
L05224  !
L05225  ! IN this surboutine we compute the upward longwave flux, and the albedo
L05226  ! needed for the radiation scheme
L05227  !
L05228        implicit none
L05229
L05230  !
L05231  !INPUT VARIABLES
L05232  !
L05233        real rsw(2*ndm,nz_um)        ! Short wave radiation at the wall for a given canyon direction [W/m2]
L05234        real rlw(2*ndm,nz_um)         ! Long wave radiation at the walls for a given canyon direction [W/m2]
L05235        real rsg(ndm)                   ! Short wave radiation at the canyon for a given canyon direction [W/m2]
L05236        real rlg(ndm)                   ! Long wave radiation at the ground for a given canyon direction [W/m2]
L05237        real rs                        ! Short wave radiation at the horizontal surface from the sun [W/m2]
L05238        real sfw(2*ndm,nz_um)      ! Sensible heat flux from walls [W/m2]
L05239        real sfg(ndm)               ! Sensible heat flux from ground (road) [W/m2]
L05240        real lfg(ndm)
L05241        real sfr(ndm,nz_um)      ! Sensible heat flux from roofs [W/m2]
L05242        real lfr(ndm,nz_um)
L05243        real lfrv(ndm,nz_um)
L05244        real sfrv(ndm,nz_um)
L05245        real gr_frac_roof
L05246        real rld                        ! Long wave radiation from the sky [W/m2]
L05247        real albg_u                    ! albedo of the ground/street
L05248        real albw_u                    ! albedo of the walls
L05249        real albr_u                    ! albedo of the roof
L05250        real ws(ndm)                       ! width of the street
L05251        real bs(ndm)
L05252                          ! building size
L05253        real pb(nz_um)                ! Probability to have a building with an height equal or higher
L05254        integer nzu
L05255        real ss(nz_um)                ! Probability to have a building of a given height
L05256        real sigma
L05257        real emg_u                      ! emissivity of the street
L05258        real emw_u                      ! emissivity of the wall
L05259        real emr_u                      ! emissivity of the roof
L05260        real tw(2*ndm,nz_um)  ! Temperature in each layer of the wall [K]
L05261        real tr_av(ndm,nz_um)  ! Temperature in each layer of the roof [K]
L05262        real tpvlev(ndm,nz_um)
L05263        real pv_frac_roof
L05264        real tg_av(ndm)          ! Temperature in each layer of the ground [K]
L05265        integer id ! street direction
L05266        integer ndu ! number of street directions
L05267  !
L05268  !New variables BEM
L05269  !
L05270        real emwind               !Emissivity of the windows
L05271        real albwind              !Albedo of the windows
L05272        real twlev(2*ndm,nz_um)   !Averaged Temperature of the windows
L05273        real pwin                 !Coverage area fraction of the windows
L05274        real gflwin               !Heat stored for the windows
L05275        real sfwind(2*ndm,nz_um)  !Sensible heat flux from windows [W/m2]
L05276
L05277  !OUTPUT/INPUT
L05278        real rs_abs  ! absrobed solar radiationfor this street direction
L05279        real rl_up   ! upward longwave radiation for this street direction
L05280        real emiss ! mean emissivity
L05281        real grdflx_urb ! ground heat flux
L05282  !LOCAL
L05283        integer iz,iw
L05284        real rl_inc,rl_emit
L05285        real gfl
L05286        integer ix,iy,iwrong
L05287
L05288           iwrong=1
L05289        do iz=1,nzu+1
L05290        do id=1,ndu
L05291        if(tr_av(id,iz).lt.100.)then
L05292                write(203,*) tr_av(id,iz)
L05293                write(*,*)'in upward_rad ',iz,id,iw,tr_av(id,iz)
L05294                iwrong=0
L05295        endif
L05296        enddo
L05297        enddo
L05298             if(iwrong.eq.0)stop
L05299
L05300        rl_up=0.
L05301
L05302        rs_abs=0.
L05303        rl_inc=0.
L05304        emiss=0.
L05305        rl_emit=0.
L05306        grdflx_urb=0.
L05307        do id=1,ndu
L05308         rl_emit=rl_emit-( emg_u*sigma*(tg_av(id)**4.)+(1-emg_u)*rlg(id))*ws(id)/(ws(id)+bs(id))/ndu
L05309         rl_inc=rl_inc+rlg(id)*ws(id)/(ws(id)+bs(id))/ndu
L05310         rs_abs=rs_abs+(1.-albg_u)*rsg(id)*ws(id)/(ws(id)+bs(id))/ndu
L05311           gfl=(1.-albg_u)*rsg(id)+emg_u*rlg(id)-emg_u*sigma*(tg_av(id)**4.)+sfg(id)+lfg(id)
L05312           grdflx_urb=grdflx_urb-gfl*ws(id)/(ws(id)+bs(id))/ndu
```

```
L05313
L05314           do iz=2,nzu
L05315               rl_emit=rl_emit-(emr_u*sigma*(1.-pv_frac_roof)*tr_av(id,iz)**4.+0.79*sigma*pv_frac_roof*tpvlev(id,iz)**4+
↪  &
L05316                       (1-emr_u)*rld*(1.-pv_frac_roof)+(1-0.79)*pv_frac_roof*rld)*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L05317               rl_inc=rl_inc+rld*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L05318
↪  rs_abs=rs_abs+((1.-albr_u)*rs*(1.-pv_frac_roof)+(1.-0.11)*rs*pv_frac_roof)*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L05319               gfl=(1.-albr_u)*rs*(1-pv_frac_roof)+emr_u*rld*(1-pv_frac_roof)+pv_frac_roof*emr_u*sigma*tpvlev(id,iz)**4 &
L05320                  -emr_u*sigma*(tr_av(id,iz)**4.)+(1-gr_frac_roof)*sfr(id,iz)+(sfrv(id,iz)+lfrv(id,
↪  iz))*gr_frac_roof+(1.-gr_frac_roof)*lfr(id,iz)
L05321               grdflx_urb=grdflx_urb-gfl*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L05322           enddo
L05323
L05324           do iz=1,nzu
L05325
L05326              rl_emit=rl_emit-(emw_u*(1.-pwin)*sigma*(tw(2*id-1,iz)**4.+tw(2*id,iz)**4.)+ &
L05327                           (emwind*pwin*sigma*(twlev(2*id-1,iz)**4.+twlev(2*id,iz)**4.))+ &
L05328                  ((1.-emw_u)*(1.-pwin)+pwin*(1.-emwind))*(rlw(2*id-1,iz)+rlw(2*id,iz)))* &
L05329                           dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L05330
L05331              rl_inc=rl_inc+((rlw(2*id-1,iz)+rlw(2*id,iz)))*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L05332
L05333              rs_abs=rs_abs+(((1.-albw_u)*(1.-pwin)+(1.-albwind)*pwin)*(rsw(2*id-1,iz)+rsw(2*id,iz)))*&
L05334                           dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L05335
L05336              gfl=(1.-albw_u)*(rsw(2*id-1,iz)+rsw(2*id,iz)) +emw_u*( rlw(2*id-1,iz)+rlw(2*id,iz) )   &
L05337               -emw_u*sigma*( tw(2*id-1,iz)**4.+tw(2*id,iz)**4. )+(sfw(2*id-1,iz)+sfw(2*id,iz))
L05338
L05339              gflwin=(1.-albwind)*(rsw(2*id-1,iz)+rsw(2*id,iz)) +emwind*(rlw(2*id-1,iz)+rlw(2*id,iz))   &
L05340               -emwind*sigma*( twlev(2*id-1,iz)**4.+twlev(2*id,iz)**4.)+(sfwind(2*id-1,iz)+sfwind(2*id,iz))
L05341
L05342
L05343              grdflx_urb=grdflx_urb-(gfl*(1.-pwin)+pwin*gflwin)*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L05344
L05345           enddo
L05346
L05347        enddo
L05348          emiss=(emg_u+emw_u+emr_u)/3.
L05349          rl_up=(rl_inc+rl_emit)-rld
L05350
L05351
L05352        return
L05353
L05354        END SUBROUTINE upward_rad
```

讲解：给辐射方案的城市等效量：按 ws/(ws+bs)（地面权重）与 ss • bs/(ws+bs)（屋顶权重）与 dz_u • pb/(ws+bs)（墙面权重）对三个方向聚合——rl_emit（发射 − 反射）、rl_inc（入射）、rs_abs（吸收短波）、grdflx_urb（每表面能量收支残差 =储热）；屋顶 PV 部分用常数发射率 0.79/反照率 0.11；emiss=(emg+emw+emr)/3 简单平均。rl_up=rl_inc+rl_emit−rld 为辐射方案所需的向上长波。**开头的 <100 K 哨兵**（write(203)+stop）是调试遗留——生产运行若触发会硬停，改造时建议换成 wrf_error_fatal。

```
!file: phys/module_sf_bep_bem.F    L5361-L5437    (subroutine albwindow)
L05361           subroutine albwindow(albwin)
L05362
L05363  !-----------------------------------------------------------------
L05364       implicit none
L05365
L05366
L05367  ! ----------------------------------------------------------------
L05368  !Based on the
L05369  !paper of J.Karlsson and A.Roos(2000):"modelling the angular behaviour
L05370  !of the total solar energy transmittance of windows"
L05371  !Solar Energy Vol.69,No.4,pp. 321-329.
L05372  ! ----------------------------------------------------------------
L05373  !Input
L05374  !-----
L05375
L05376  !Output
L05377  !------
L05378           real albwin            ! albedo of the window
L05379  !Local
L05380  !-----
L05381       real a,b,c     !Polynomial coefficients
L05382       real alfa,delta,gama   !Polynomial powers
L05383       real g0           !transmittance when the angle
L05384                              !of incidence is normal to the surface.
L05385           real asup,ainf
L05386       real fonc
L05387
L05388  !Constants
L05389  !--------------------
L05390
L05391           real epsilon              !accuracy of the integration
L05392           parameter (epsilon=1.e-07)
L05393           real n1,n2                !Index of refraction for glasses and air
L05394           parameter(n1=1.,n2=1.5)
L05395           integer intg,k
L05396  !-----------------------------------------------------------------
L05397           if (q_num.eq.0) then
L05398             write(*,*) 'Category parameter of the windows no valid'
L05399             stop
L05400           endif
L05401
L05402           g0=4.*n1*n2/((n1+n2)*(n1+n2))
L05403       a=8.
L05404       b=0.25/q_num
L05405           c=1.-a-b
L05406       alfa =5.2 + (0.7*q_num)
L05407       delta =2.
L05408       gama =(5.26+0.06*p_num)+(0.73+0.04*p_num)*q_num
```

```
L05409
L05410          intg=1
L05411  !-----------------------------------------------------------------------
L05412
L05413
L05414  100     asup=0.
L05415          ainf=0.
L05416
L05417          do k=1,intg
L05418           call foncs(fonc,(pi*k/intg),a,b,c,alfa,delta,gama)
L05419           asup=asup+(pi/intg)*fonc
L05420          enddo
L05421
L05422          intg=intg+1
L05423
L05424          do k=1,intg
L05425           call foncs(fonc,(pi*k/intg),a,b,c,alfa,delta,gama)
L05426           ainf=ainf+(pi/intg)*fonc
L05427          enddo
L05428
L05429          if(abs(asup-ainf).lt.epsilon) then
L05430            albwin=1-g0+(g0/2.)*asup
L05431          else
L05432            goto 100
L05433          endif
L05434
L05435  !-----------------------------------------------------------------------
L05436      return
L05437      end subroutine albwindow
```

```
!file: phys/module_sf_bep_bem.F    L5441-L5454    (subroutine foncs)
L05441          subroutine foncs(fonc,x,aa,bb,cc,alf,delt,gam)
L05442
L05443          implicit none
L05444  !
L05445          real x,aa,bb,cc
L05446          real alf,delt,gam
L05447          real fonc
L05448
L05449          fonc=(((aa*(x**alf))/(pi**alf))+   &
L05450               ((bb*(x**delt))/(pi**delt))+  &
L05451               ((cc*(x**gam))/(pi**gam)))*sin(x)
L05452
L05453          return
L05454      end subroutine foncs
```

讲解：Karlsson–Roos 2000 窗户角向反射模型：三段幂多项式（a=8、b=0.25/q_num、c=1−a−b，指数 alfa/delta/gama 由玻璃层数 p_num 与类别 q_num 定）的 sin 加权积分，自适应步长直到相邻两次积分差 <1e-7；$g0=4n_1n_2/(n_1+n_2)^2$ 是法向透射（n_1=1 空气、n_2=1.5 玻璃）。只在 init_para 调一次。

```
!file: phys/module_sf_bep_bem.F    L5458-L5514    (subroutine icBEP_XY)
L05458        subroutine icBEP_XY(iurb,fww_u,fwg_u,fgw_u,fsw_u,            &
L05459                            fws_u,fsg_u,ndu,strd,ws,nzu,z_u)
L05460
L05461        implicit none
L05462
L05463  !    Street parameters
L05464        integer ndu     ! Number of street direction for each urban class
L05465        integer iurb
L05466
L05467        real strd(ndm)        ! Street length (fix to greater value to the horizontal length of the cells)
L05468        real ws(ndm)          ! Street width [m]
L05469
L05470  !    Grid parameters
L05471        integer nzu          ! Number of layer in the urban grid
L05472        real z_u(nz_um)       ! Height of the urban grid levels
L05473  ! ----------------------------------------------------------------------
L05474  !     Output
L05475  !-----------------------------------------------------------------------
L05476
L05477  !   fww_u,fwg_u,fgw_u,fsw_u,fsg_u are the view factors used to compute the long wave
L05478  !   and the short wave radation. They are the part of radiation from a surface
L05479  !   or from the sky to another surface.
L05480
L05481        real fww_u(nz_um,nz_um,ndm,nurbm)         !  from wall to wall
L05482        real fwg_u(nz_um,ndm,nurbm)               !  from wall to ground
L05483        real fgw_u(nz_um,ndm,nurbm)               !  from ground to wall
L05484        real fsw_u(nz_um,ndm,nurbm)               !  from sky to wall
L05485        real fws_u(nz_um,ndm,nurbm)               !  from sky to wall
L05486        real fsg_u(ndm,nurbm)                     !  from sky to ground
L05487
L05488  ! ----------------------------------------------------------------------
L05489  !     Local
L05490  !-----------------------------------------------------------------------
L05491
L05492        integer id
L05493
L05494  ! ----------------------------------------------------------------------
L05495  !     This routine compute the view factors
L05496  !-----------------------------------------------------------------------
L05497  !
L05498  !Initialize
L05499  !
L05500        fww_u=0.
L05501        fwg_u=0.
L05502        fgw_u=0.
L05503        fsw_u=0.
L05504        fws_u=0.
L05505        fsg_u=0.
L05506
L05507        do id=1,ndu
L05508
L05509             call view_factors(iurb,nzu,id,strd(id),z_u,ws(id),  &
L05510                               fww_u,fwg_u,fgw_u,fsg_u,fsw_u,fws_u)
```

```
L05511
L05512        enddo
L05513        return
L05514        end subroutine icBEP_XY
```

```
!file: phys/module_sf_bep_bem.F    L5517-L5634    (subroutine icBEPHI_XY)
L05517        subroutine icBEPHI_XY(iurb,hb_u,hi_urb1D,ss_u,pb_u,nzu,z_u)
L05518
L05519        implicit none
L05520  !---------------------------------------------------------------------
L05521  !    Inputs
L05522  !---------------------------------------------------------------------
L05523  !    Street parameters
L05524  !
L05525        real hi_urb1D(nz_um)    ! The probability that a building has an height h_b
L05526        integer iurb            ! Number of the urban class
L05527  !
L05528  !     Grid parameters
L05529  !
L05530        real z_u(nz_um)         ! Height of the urban grid levels
L05531  ! ---------------------------------------------------------------------
L05532  !     Output
L05533  !---------------------------------------------------------------------
L05534
L05535        real ss_u(nz_um,nurbm)  ! The probability that a building has an height equal to z
L05536        real pb_u(nz_um)        ! The probability that a building has an height greater or equal to z
L05537  !
L05538  !    Grid parameters
L05539  !
L05540        integer nzu                ! Number of layer in the urban grid
L05541
L05542  ! ---------------------------------------------------------------------
L05543  !     Local
L05544  !---------------------------------------------------------------------
L05545        real hb_u(nz_um)        ! Bulding's heights [m]
L05546        integer iz_u,id,ilu
L05547
L05548        real dtot
L05549        real hbmax
L05550
L05551  !---------------------------------------------------------------------
L05552
L05553  !Initialize variables
L05554  !
L05555
L05556        nzu=0
L05557        ss_u=0.
L05558        pb_u=0.
L05559
L05560  ! Normalisation of the building density
L05561
L05562           dtot=0.
L05563           hb_u=0.
L05564
L05565           do ilu=1,nz_um
L05566              dtot=dtot+hi_urb1D(ilu)
L05567           enddo
L05568
L05569           do ilu=1,nz_um
L05570              if (hi_urb1D(ilu)<0.) then
L05571  !              write(*,*) 'WARNING, HI_URB1D(ilu) < 0 IN BEP_BEM'
L05572                 go to 20
L05573              endif
L05574           enddo
L05575
L05576           if (dtot.gt.0.) then
L05577              continue
L05578           else
L05579  !           write(*,*) 'WARNING, HI_URB1D <= 0 IN BEP_BEM'
L05580              go to 20
L05581           endif
L05582
L05583           do ilu=1,nz_um
L05584              hi_urb1D(ilu)=hi_urb1D(ilu)/dtot
L05585           enddo
L05586
L05587           hb_u(1)=dz_u
L05588           do ilu=2,nz_um
L05589              hb_u(ilu)=dz_u+hb_u(ilu-1)
L05590           enddo
L05591
L05592
L05593  ! Compute pb and ss
L05594
L05595
L05596           hbmax=0.
L05597
L05598           do ilu=1,nz_um
L05599              if (hi_urb1D(ilu)>0.and.hi_urb1D(ilu)<=1.) then
L05600                  hbmax=hb_u(ilu)
L05601              endif
L05602           enddo
L05603
L05604           do iz_u=1,nz_um-1
L05605              if(z_u(iz_u+1).gt.hbmax)go to 10
L05606           enddo
L05607
L05608  10       continue
L05609
L05610           nzu=iz_u+1
L05611
L05612           if ((nzu+1).gt.nz_um) then
L05613               write(*,*) 'error, nz_um has to be increased to at least',nzu+1
L05614               stop
L05615           endif
L05616
L05617              do iz_u=1,nzu
L05618                 ss_u(iz_u,iurb)=0.
```

```
L05619                do ilu=1,nz_um
L05620                   if(z_u(iz_u).le.hb_u(ilu)                    &
L05621                     .and.z_u(iz_u+1).gt.hb_u(ilu))then
L05622                        ss_u(iz_u,iurb)=ss_u(iz_u,iurb)+hi_urb1D(ilu)
L05623                   endif
L05624                enddo
L05625             enddo
L05626
L05627             pb_u(1)=1.
L05628             do iz_u=1,nzu
L05629                pb_u(iz_u+1)=max(0.,pb_u(iz_u)-ss_u(iz_u,iurb))
L05630             enddo
L05631
L05632  20     continue
L05633         return
L05634         end subroutine icBEPHI_XY
```

讲解：两个逐点初始化单元：icBEP_XY 逐方向调 view_factors 算该点的视野因子（strd 固定 1e5 m——街道长取超长，即「无限长峡谷」近似）；icBEPHI_XY 把**该格点**的建筑高度直方图 hi_urb2d 归一化（dtot>0 与负值检查，异常静默跳过 goto 20——不重算 ss/pb，沿用类谱），再按楼高折层得 ss/pb。BEF 逐格化（BEP⇄BEM 差异之一）就发生在这里。

```
!file: phys/module_sf_bep_bem.F    L5639-L5705
L05639  ! ===6=8================================================================72
L05640  ! ===6=8================================================================72
L05641
L05642         FUNCTION bep_bem_nurbm () RESULT (bep_bem_val_nurbm)
L05643            USE module_sf_bep_bem
L05644            IMPLICIT NONE
L05645            INTEGER :: bep_bem_val_nurbm
L05646            bep_bem_val_nurbm = nurbm
L05647         END FUNCTION bep_bem_nurbm
L05648
L05649         FUNCTION bep_bem_ndm () RESULT (bep_bem_val_ndm)
L05650            USE module_sf_bep_bem
L05651            IMPLICIT NONE
L05652            INTEGER :: bep_bem_val_ndm
L05653            bep_bem_val_ndm = ndm
L05654         END FUNCTION bep_bem_ndm
L05655
L05656         FUNCTION bep_bem_nz_um () RESULT (bep_bem_val_nz_um)
L05657            USE module_sf_bep_bem
L05658            IMPLICIT NONE
L05659            INTEGER :: bep_bem_val_nz_um
L05660            bep_bem_val_nz_um = nz_um
L05661         END FUNCTION bep_bem_nz_um
L05662
L05663         FUNCTION bep_bem_ng_u () RESULT (bep_bem_val_ng_u)
L05664            USE module_sf_bep_bem
L05665            IMPLICIT NONE
L05666            INTEGER :: bep_bem_val_ng_u
L05667            bep_bem_val_ng_u = ng_u
L05668         END FUNCTION bep_bem_ng_u
L05669
L05670         FUNCTION bep_bem_nwr_u () RESULT (bep_bem_val_nwr_u)
L05671            USE module_sf_bep_bem
L05672            IMPLICIT NONE
L05673            INTEGER :: bep_bem_val_nwr_u
L05674            bep_bem_val_nwr_u = nwr_u
L05675         END FUNCTION bep_bem_nwr_u
L05676
L05677         FUNCTION bep_bem_nf_u () RESULT (bep_bem_val_nf_u)
L05678            USE module_sf_bep_bem
L05679            IMPLICIT NONE
L05680            INTEGER :: bep_bem_val_nf_u
L05681            bep_bem_val_nf_u = nf_u
L05682         END FUNCTION bep_bem_nf_u
L05683
L05684
L05685         FUNCTION bep_bem_ngb_u () RESULT (bep_bem_val_ngb_u)
L05686            USE module_sf_bep_bem
L05687            IMPLICIT NONE
L05688            INTEGER :: bep_bem_val_ngb_u
L05689            bep_bem_val_ngb_u = ngb_u
L05690         END FUNCTION bep_bem_ngb_u
L05691
L05692         FUNCTION bep_bem_nbui_max () RESULT (bep_bem_val_nbui_max)
L05693            USE module_sf_bep_bem
L05694            IMPLICIT NONE
L05695            INTEGER :: bep_bem_val_nbui_max
L05696            bep_bem_val_nbui_max = nbui_max
L05697         END FUNCTION bep_bem_nbui_max
L05698
L05699
L05700      FUNCTION bep_bem_ngr_u () RESULT (bep_bem_val_ngr_u)
L05701            USE module_sf_bep_bem
L05702            IMPLICIT NONE
L05703            INTEGER :: bep_bem_val_ngr_u
L05704            bep_bem_val_ngr_u = ngr_u
L05705         END FUNCTION bep_bem_ngr_u
```

讲解：模块外的九个宿主关联查询函数（bep_bem_nurbm/ndm/nz_um/ng_u/nwr_u/nf_u/ngb_u/nbui_max/ngr_u）——把本模块的 parameter 常量以函数形式暴露给 Registry 生成代码与外部模块做数组维度声明（Fortran 模块变量的跨模块只读访问惯例）。改维度常数（如 nz_um=18）后这组函数自动跟随，但**城市态数组维度在 Registry 的 urban_map_* 尺寸必须同步**（namelist num_urban_layers ≥ nz_um，init_para 的守卫会拦）。

八、改造入口

1. **辐射量计算性能**：long_rad/short_rad_dd 的 37×37 线性系统每点每步两解——灰区降尺度或大域时先测这里；可行方向：视野因子按类缓存（ws/strd 逐点变化才被迫逐点算）。
2. **导热分层**：param/surf_temp/roof_temp_veg 三处厚度数组 + soil_temp/soil_temp_veg 的求解器——改墙体结构三处同步。
3. **绿屋顶/PV**：roof_temp_veg 水力参数 + flux_flat_roof 气孔阻抗 + upward_rad 的 PV 常数（0.79/0.11）+ init_para 的 TBL 项。
4. **遗留代码**：flux_flat_ground 无调用方（勿改）；upward_rad 的硬 stop 哨兵建议替换。
5. **论文 2 对接**：ind_* 索引折叠表与 urban_map_* 维度是「标量 → 逐格场」的先例机制（notes 已录 vbs/br 先例）；BEM 主体改造见 code_16。

九、上下游链接

- 上半（主入口与柱求解器）：`code_14a_module_sf_bep_bem.md`
- 同族：`code_15_module_sf_bep.md`、`code_16_module_sf_bem.md`；参数库 module_sf_urban（code_17 篇待交付）
- 驱动链：`code_22_module_pbl_driver.md`；面层 `code_10_module_sf_mynnsfc.md`
- 教材正文：ch17_urban_canopy.md、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_15 — BEP 单独版（module_sf_bep.F 全文精读）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_bep.F	3536	5f74b56a221ca4ec69320a09ad3f9c3e

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级全文（32 单元全收）。约四分之三单元与共享底座 module_sf_bep_bem.F（code_14a/code_14b）**逐行同源**——那些单元的讲解给差异注记并回指 14a/14b，正文不重复；BEP 独有部分（主入口、简化柱求解、angle）展开讲。

二、定位

一句话：sf_urban_physics=1（BEP 单开、无 BEM）时的城市冠层方案主体——比 bep_bem 少了建筑能量模型（室内温度/空调/通风/窗户/绿屋顶/PV/水分收支），保留双网格拖曳与峡谷辐射的全部骨架。

谁在调它：module_sf_urban.F 的 urban（BEPSCHEME 分支）调 BEP；pbl_driver 经 flag_bep 消费 a_*/b*_ 源项（`code_22_module_pbl_driver.md`）。教材锚点：ch17_urban_canopy.md。

与 bep_bem 的四大差异（改造前必读）：

项	module_sf_bep.F（本篇）	module_sf_bep_bem.F（14a/14b）
拖曳系数	模块常数 cdrag=0.4（flux_wall 直接用）	NEW CDRAG：按建筑密度 3.32 • alp^0.47 / 1.85 逐点算
固体温度	surf_temp 一件管地面 +屋顶 +墙三面	地面 surf_temp；屋顶/墙由 BEM 内部导热管
水分/室内	无（无积水、无 AC、无绿屋顶）	dg/dgr 水量收支 + BEM 全套
状态数组	tw1/tw2/trb/tgb 四件	另有 trv/qr/tlev/qlev/twlev/tflev/tglev 七件

三、文件头

```
!file: phys/module_sf_bep.F    L1-L61
L00001  MODULE module_sf_bep
L00002  #if defined(mpas)
L00003  use mpas_atmphys_utilities, only: physics_error_fatal
L00004  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00005  #else
L00006  use module_wrf_error
L00007  #define FATAL_ERROR(M) call wrf_error_fatal( M)
L00008  !USE module_model_constants
L00009  #endif
L00010   USE module_sf_urban
L00011   USE module_bep_bem_helper, ONLY: nurbm
L00012
L00013  ! SGClarke 09/11/2008
L00014  ! Access urban_param.tbl values through calling urban_param_init in module_physics_init
L00015  ! for CASE (BEPSCHEME) select sf_urban_physics
L00016  !
L00017  ! ------------------------------------------------------------------
L00018  !  Dimension for the array used in the BEP module
L00019  ! ------------------------------------------------------------------
L00020        integer nurbmax         ! Maximum number of urban classes
L00021        parameter (nurbmax=11)
L00022
L00023        integer ndm             ! Maximum number of street directions
L00024        parameter (ndm=2)
L00025
L00026        integer nz_um           ! Maximum number of vertical levels in the urban grid
L00027        parameter(nz_um=18)
```

```
L00028
L00029        integer ng_u            ! Number of grid levels in the ground
L00030        parameter (ng_u=10)
L00031        integer nwr_u            ! Number of grid levels in the walls or roofs
L00032        parameter (nwr_u=10)
L00033
L00034        real dz_u                ! Urban grid resolution
L00035        parameter (dz_u=5.)
L00036
L00037  ! The change of ng_u, nwr_u should be done in agreement with the block data
L00038  !  in the routine "surf_temp"
L00039  ! ------------------------------------------------------------------------
L00040  !  Constant used in the BEP module
L00041  ! ------------------------------------------------------------------------
L00042
L00043        real vk                 ! von Karman constant
L00044        real g_u                 ! Gravity acceleration
L00045        real pi                 !
L00046        real r                  ! Perfect gas constant
L00047        real cp_u                ! Specific heat at constant pressure
L00048        real rcp_u              !
L00049        real sigma              !
L00050        real p0                 ! Reference pressure at the sea level
L00051        real cdrag              ! Drag force constant
L00052        parameter(vk=0.40,g_u=9.81,pi=3.141592653,r=287.,cp_u=1004.)
L00053        parameter(rcp_u=r/cp_u,sigma=5.67e-08,p0=1.e+5,cdrag=0.4)
L00054
L00055  ! ------------------------------------------------------------------------
L00056
L00057
L00058
L00059
L00060     CONTAINS
L00061
```

讲解：USE 清单只有 module_sf_urban（TBL 参数库）与 bep_bem_helper（nurbm）——**不 USE module_sf_bem**，这是「BEP 单独版」的编译级证据；维度常数与 bep_bem 相同（nurbmax=11、ndm=2、nz_um=18、dz_u=5 m、ng_u=nwr_u=10）但**没有** BEM 的 nf_u/ngb_u/ngr_u/nbui_max（无楼层/楼下地基/绿屋顶层/建筑型维度）；常数区多一个 cdrag=0.4——街道拖曳系数在此是常数而非参数化。

四、主入口 BEP（全文）

```
!file: phys/module_sf_bep.F    L62-L585    (subroutine BEP)
L00062        subroutine BEP(FRC_URB2D,UTYPE_URB2D,itimestep,dz8w,dt,u_phy,v_phy,      &
L00063                       th_phy,rho,p_phy,swdown,glw,                      &
L00064                       gmt,julday,xlong,xlat,                            &
L00065                       declin_urb,cosz_urb2d,omg_urb2d,                  &
L00066                       num_urban_ndm,  urban_map_zrd,  urban_map_zwd, urban_map_gd, &
L00067                        urban_map_zd,  urban_map_zdf,   urban_map_bd, urban_map_wd, &
L00068                       urban_map_gbd,  urban_map_fbd,                    &
L00069                        num_urban_hi,                                    &
L00070                       trb_urb4d,tw1_urb4d,tw2_urb4d,tgb_urb4d,          &
L00071                       sfw1_urb3d,sfw2_urb3d,sfr_urb3d,sfg_urb3d,        &
L00072                       lp_urb2d,hi_urb2d,lb_urb2d,hgt_urb2d,             &
L00073                       a_u,a_v,a_t,a_e,b_u,b_v,                          &
L00074                       b_t,b_e,b_q,dlg,dl_u,sf,vl,                       &
L00075                       rl_up,rs_abs,emiss,grdflx_urb,                    &
L00076                       ids,ide, jds,jde, kds,kde,                        &
L00077                       ims,ime, jms,jme, kms,kme,                        &
L00078                       its,ite, jts,jte, kts,kte)
L00079
L00080        implicit none
L00081
L00082  !-----------------------------------------------------------------------
L00083  !     Input
L00084  !-----------------------------------------------------------------------
L00085     INTEGER ::                        ids,ide, jds,jde, kds,kde,  &
L00086                                       ims,ime, jms,jme, kms,kme,  &
L00087                                       its,ite, jts,jte, kts,kte,  &
L00088                                       itimestep
L00089
L00090
L00091     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   DZ8W
L00092     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   P_PHY
L00093     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   RHO
L00094     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   TH_PHY
L00095     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   T_PHY
L00096     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   U_PHY
L00097     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   V_PHY
L00098     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   U
L00099     REAL, DIMENSION( ims:ime, kms:kme, jms:jme )::   V
L00100     REAL, DIMENSION( ims:ime , jms:jme )        ::   GLW
L00101     REAL, DIMENSION( ims:ime , jms:jme )        ::   swdown
L00102     REAL, DIMENSION( ims:ime, jms:jme )         ::   UST
L00103     INTEGER, DIMENSION( ims:ime , jms:jme ), INTENT(IN )::   UTYPE_URB2D
L00104     REAL, DIMENSION( ims:ime , jms:jme ), INTENT(IN )::   FRC_URB2D
L00105     REAL, INTENT(IN   )   ::                                GMT
L00106     INTEGER, INTENT(IN  ) ::                               JULDAY
L00107     REAL, DIMENSION( ims:ime, jms:jme ),                          &
L00108           INTENT(IN   )  ::                          XLAT, XLONG
L00109     REAL, INTENT(IN) :: DECLIN_URB
L00110     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN) :: COSZ_URB2D
L00111     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN) :: OMG_URB2D
L00112     INTEGER, INTENT(IN  ) :: num_urban_ndm
L00113     INTEGER, INTENT(IN  ) :: urban_map_zrd
L00114     INTEGER, INTENT(IN  ) :: urban_map_zwd
L00115     INTEGER, INTENT(IN  ) :: urban_map_gd
L00116     INTEGER, INTENT(IN  ) :: urban_map_zd
```

```
L00117     INTEGER, INTENT(IN   ) :: urban_map_zdf
L00118     INTEGER, INTENT(IN   ) :: urban_map_bd
L00119     INTEGER, INTENT(IN   ) :: urban_map_wd
L00120     INTEGER, INTENT(IN   ) :: urban_map_gbd
L00121     INTEGER, INTENT(IN   ) :: urban_map_fbd
L00122     INTEGER, INTENT(IN   ) :: num_urban_hi
L00123     REAL, DIMENSION( ims:ime, 1:urban_map_zrd, jms:jme ), INTENT(INOUT) :: trb_urb4d
L00124     REAL, DIMENSION( ims:ime, 1:urban_map_zwd, jms:jme ), INTENT(INOUT) :: tw1_urb4d
L00125     REAL, DIMENSION( ims:ime, 1:urban_map_zwd, jms:jme ), INTENT(INOUT) :: tw2_urb4d
L00126     REAL, DIMENSION( ims:ime, 1:urban_map_gd , jms:jme ), INTENT(INOUT) :: tgb_urb4d
L00127     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: sfw1_urb3d
L00128     REAL, DIMENSION( ims:ime, 1:urban_map_wd , jms:jme ), INTENT(INOUT) :: sfw2_urb3d
L00129     REAL, DIMENSION( ims:ime, 1:urban_map_zdf, jms:jme ), INTENT(INOUT) :: sfr_urb3d
L00130     REAL, DIMENSION( ims:ime, 1:num_urban_ndm, jms:jme ), INTENT(INOUT) :: sfg_urb3d
L00131     REAL, DIMENSION( ims:ime, 1:num_urban_hi, jms:jme ), INTENT(IN) :: hi_urb2d
L00132     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: lp_urb2d
L00133     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: lb_urb2d
L00134     REAL, DIMENSION( ims:ime,jms:jme), INTENT(IN) :: hgt_urb2d
L00135
L00136  !      integer nx,ny,nz              ! Number of points in the mesocsale grid
L00137        real z(ims:ime,kms:kme,jms:jme)           ! Vertical coordinates
L00138        REAL, INTENT(IN ):::   DT      ! Time step
L00139  !      real zr(ims:ime,jms:jme)               ! Solar zenith angle
L00140  !      real deltar(ims:ime,jms:jme)           ! Declination of the sun
L00141  !      real ah(ims:ime,jms:jme)               ! Hour angle
L00142  !      real rs(ims:ime,jms:jme)               ! Solar radiation
L00143  !-------------------------------------------------------------------------
L00144  !     Output
L00145  !-------------------------------------------------------------------------
L00146  !      real tsk(ims:ime,jms:jme)           ! Average of surface temperatures (roads and roofs)
L00147  !
L00148  !    Implicit and explicit components of the source and sink terms at each levels,
L00149  !     the fluxes can be computed as follow: FX = A*X + B   example: t_fluxes = a_t * pt + b_t
L00150        real a_u(ims:ime,kms:kme,jms:jme)         ! Implicit component for the momemtum in X-direction (center)
L00151        real a_v(ims:ime,kms:kme,jms:jme)         ! Implicit component for the momemtum in Y-direction (center)
L00152        real a_t(ims:ime,kms:kme,jms:jme)         ! Implicit component for the temperature
L00153        real a_e(ims:ime,kms:kme,jms:jme)         ! Implicit component for the TKE
L00154        real b_u(ims:ime,kms:kme,jms:jme)         ! Explicit component for the momemtum in X-direction (center)
L00155        real b_v(ims:ime,kms:kme,jms:jme)         ! Explicit component for the momemtum in Y-direction (center)
L00156        real b_t(ims:ime,kms:kme,jms:jme)         ! Explicit component for the temperature
L00157        real b_e(ims:ime,kms:kme,jms:jme)         ! Explicit component for the TKE
L00158        real b_q(ims:ime,kms:kme,jms:jme)         ! Explicit component for the humidity
L00159        real dlg(ims:ime,kms:kme,jms:jme)         ! Height above ground (L_ground in formula (24) of the BLM paper).
L00160        real dl_u(ims:ime,kms:kme,jms:jme)        ! Length scale (lb in formula (22) ofthe BLM paper).
L00161  ! urban surface and volumes
L00162        real sf(ims:ime,kms:kme,jms:jme)           ! surface of the urban grid cells
L00163        real vl(ims:ime,kms:kme,jms:jme)             ! volume of the urban grid cells
L00164  ! urban fluxes
L00165        real rl_up(its:ite,jts:jte) ! upward long wave radiation
L00166        real rs_abs(its:ite,jts:jte) ! absorbed short wave radiation
L00167        real emiss(its:ite,jts:jte)  ! emissivity averaged for urban surfaces
L00168        real grdflx_urb(its:ite,jts:jte)  ! ground heat flux for urban areas
L00169  !-------------------------------------------------------------------------
L00170  !     Local
L00171  !-------------------------------------------------------------------------
L00172  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L00173
L00174        real hi_urb(its:ite,1:nz_um,jts:jte)    ! Height histograms of buildings
L00175        real hi_urb1D(nz_um)                    ! Height histograms of buildings
L00176        real hb_u(nz_um)                        ! Bulding's heights
L00177        real ss_urb(nz_um)  ! Probability that a building has an height equal to z
L00178        real pb_urb(nz_um)  ! Probability that a building has an height greater or equal to z
L00179        integer nz_urb(nurbmax)                 ! Number of layer in the urban grid
L00180        integer nzurban(nurbmax)
L00181
L00182  !    Building parameters
L00183        real alag_u(nurbmax)                    ! Ground thermal diffusivity [m^2 s^-1]
L00184        real alaw_u(nurbmax)                    ! Wall thermal diffusivity [m^2 s^-1]
L00185        real alar_u(nurbmax)                    ! Roof thermal diffusivity [m^2 s^-1]
L00186        real csg_u(nurbmax)                     ! Specific heat of the ground material [J m^3 K^-1]
L00187        real csw_u(nurbmax)                     ! Specific heat of the wall material [J m^3 K^-1]
L00188        real csr_u(nurbmax)                     ! Specific heat of the roof material [J m^3 K^-1]
L00189        real twini_u(nurbmax)                   ! Initial temperature inside the building's wall [K]
L00190        real trini_u(nurbmax)                   ! Initial temperature inside the building's roof [K]
L00191        real tgini_u(nurbmax)                   ! Initial road temperature
L00192  !
L00193  !   Building materials
L00194  !
L00195        real csg(ng_u)                          ! Specific heat of the ground material [J m^3 K^-1]
L00196        real csr(nwr_u)                         ! Specific heat of the roof material [J m^3 K^-1]
L00197        real csw(nwr_u)                         ! Specific heat of the wall material [J m^3 K^-1]
L00198        real alag(ng_u)                         ! Ground thermal diffusivity [m^2 s^-1]
L00199        real alaw(nwr_u)                        ! Wall thermal diffusivity [m^2 s^-1]
L00200        real alar(nwr_u)                        ! Roof thermal diffusivity [m^2 s^-1]
L00201  !
L00202  ! for twini_u, and trini_u the initial value at the deepest level is kept constant during the simulation
L00203  !
L00204  !    Radiation parameters
L00205        real albg_u(nurbmax)                    ! Albedo of the ground
L00206        real albw_u(nurbmax)                    ! Albedo of the wall
L00207        real albr_u(nurbmax)                    ! Albedo of the roof
L00208        real emg_u(nurbmax)                     ! Emissivity of ground
L00209        real emw_u(nurbmax)                     ! Emissivity of wall
L00210        real emr_u(nurbmax)                     ! Emissivity of roof
L00211
L00212  !   fww_u,fwg_u,fgw_u,fsw_u,fsg_u are the view factors used to compute the long wave
L00213  !   and the short wave radation.
L00214        real fww_u(nz_um,nz_um,ndm,nurbmax)       !  from wall to wall
L00215        real fwg_u(nz_um,ndm,nurbmax)             !  from wall to ground
L00216        real fgw_u(nz_um,ndm,nurbmax)             !  from ground to wall
L00217        real fsw_u(nz_um,ndm,nurbmax)             !  from sky to wall
L00218        real fws_u(nz_um,ndm,nurbmax)             !  from sky to wall
L00219        real fsg_u(ndm,nurbmax)                   !  from sky to ground
L00220
L00221  !    Roughness parameters
L00222        real z0g_u(nurbmax)       ! The ground's roughness length
L00223        real z0r_u(nurbmax)       ! The roof's roughness length
```

```
L00224
L00225  !    Roughness parameters
L00226       real z0(ndm,nz_um)           ! Roughness lengths "profiles"
L00227
L00228  !    Street parameters
L00229       integer nd_u(nurbmax)     ! Number of street direction for each urban class
L00230       real strd_u(ndm,nurbmax)  ! Street length (fix to greater value to the horizontal length of the cells)
L00231       real drst_u(ndm,nurbmax)  ! Street direction
L00232       real ws_u(ndm,nurbmax)    ! Street width
L00233       real bs_u(ndm,nurbmax)    ! Building width
L00234       real h_b(nz_um,nurbmax)   ! Bulding's heights
L00235       real d_b(nz_um,nurbmax)   ! Probability that a building has an height h_b
L00236       real ss_u(nz_um,nurbmax)  ! Probability that a building has an height equal to z
L00237       real pb_u(nz_um,nurbmax)  ! Probability that a building has an height greater or equal to z
L00238  !
L00239  !    Street parameters
L00240  !
L00241       real bs(ndm)                 ! Building width
L00242       real ws(ndm)                 ! Street width
L00243       real drst(ndm)               ! street directions
L00244       real strd(ndm)               ! Street lengths
L00245       real ss(nz_um)               ! Probability to have a building with height h
L00246       real pb(nz_um)               ! Probability to have a building with an height equal
L00247
L00248  !    Grid parameters
L00249
L00250       integer nz_u(nurbmax)     ! Number of layer in the urban grid
L00251
L00252       real z_u(nz_um)         ! Height of the urban grid levels
L00253
L00254  ! 1D array used for the input and output of the routine "urban"
L00255
L00256       real z1D(kms:kme)               ! vertical coordinates
L00257       real ua1D(kms:kme)                ! wind speed in the x directions
L00258       real va1D(kms:kme)                ! wind speed in the y directions
L00259       real pt1D(kms:kme)                ! potential temperature
L00260       real da1D(kms:kme)                ! air density
L00261       real pr1D(kms:kme)                ! air pressure
L00262       real pt01D(kms:kme)               ! reference potential temperature
L00263       real zr1D                      ! zenith angle
L00264       real deltar1D                  ! declination of the sun
L00265       real ah1D                      ! hour angle (it should come from the radiation routine)
L00266       real rs1D                      ! solar radiation
L00267       real rld1D                     ! downward flux of the longwave radiation
L00268
L00269
L00270       real tw1D(2*ndm,nz_um,nwr_u)  ! temperature in each layer of the wall
L00271       real tg1D(ndm,ng_u)           ! temperature in each layer of the ground
L00272       real tr1D(ndm,nz_um,nwr_u)  ! temperature in each layer of the roof
L00273       real sfw1D(2*ndm,nz_um)      ! sensible heat flux from walls
L00274       real sfg1D(ndm)              ! sensible heat flux from ground (road)
L00275       real sfr1D(ndm,nz_um)      ! sensible heat flux from roofs
L00276       real sf1D(kms:kme)              ! surface of the urban grid cells
L00277       real vl1D(kms:kme)                ! volume of the urban grid cells
L00278       real a_u1D(kms:kme)               ! Implicit component of the momentum sources or sinks in the X-direction
L00279       real a_v1D(kms:kme)               ! Implicit component of the momentum sources or sinks in the Y-direction
L00280       real a_t1D(kms:kme)               ! Implicit component of the heat sources or sinks
L00281       real a_e1D(kms:kme)               ! Implicit component of the TKE sources or sinks
L00282       real b_u1D(kms:kme)               ! Explicit component of the momentum sources or sinks in the X-direction
L00283       real b_v1D(kms:kme)               ! Explicit component of the momentum sources or sinks in the Y-direction
L00284       real b_t1D(kms:kme)               ! Explicit component of the heat sources or sinks
L00285       real b_e1D(kms:kme)               ! Explicit component of the TKE sources or sinks
L00286       real dlg1D(kms:kme)               ! Height above ground (L_ground in formula (24) of the BLM paper).
L00287       real dl_u1D(kms:kme)              ! Length scale (lb in formula (22) ofthe BLM paper)
L00288       real time_bep
L00289  ! arrays used to collapse indexes
L00290       integer ind_zwd(nz_um,nwr_u,ndm)
L00291       integer ind_gd(ng_u,ndm)
L00292       integer ind_zd(nz_um,ndm)
L00293  !
L00294       integer ix,iy,iz,iurb,id,iz_u,iw,ig,ir,ix1,iy1,k
L00295       integer it, nint
L00296       integer iii
L00297       real time_h,tempo
L00298       logical first
L00299       character(len=80) :: text
L00300       data first/.true./
L00301       save first,time_bep
L00302
L00303       save alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,                    &
L00304            albg_u,albw_u,albr_u,emg_u,emw_u,emr_u,                    &
L00305            z0g_u,z0r_u, nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b,ss_u,pb_u, &
L00306            nz_u,z_u
L00307
L00308  !---------------------------------------------------------------------
L00309  !    Calculation of the momentum, heat and turbulent kinetic fluxes
L00310  !     produced by builgings
L00311  !
L00312  ! Reference:
L00313  ! Martilli, A., Clappier, A., Rotach, M.W.:2002, 'AN URBAN SURFACE EXCHANGE
L00314  ! PARAMETERISATION FOR MESOSCALE MODELS', Boundary-Layer Meteorolgy 104:
L00315  ! 261-304
L00316  !---------------------------------------------------------------------
L00317  !prepare the arrays to collapse indexes
L00318
L00319       if(urban_map_zrd.lt.nz_um*ndm*nwr_u)then
L00320         write(*,*)'urban_map_zrd too small, please increase to at least ', nz_um*ndm*nwr_u
L00321         stop
L00322       endif
L00323       iii=0
L00324       do iz_u=1,nz_um
L00325       do iw=1,nwr_u
L00326       do id=1,ndm
L00327        iii=iii+1
L00328        ind_zwd(iz_u,iw,id)=iii
L00329       enddo
L00330       enddo
L00331       enddo
```

```
L00332
L00333        iii=0
L00334        do ig=1,ng_u
L00335        do id=1,ndm
L00336         iii=iii+1
L00337         ind_gd(ig,id)=iii
L00338        enddo
L00339        enddo
L00340
L00341        iii=0
L00342        do iz_u=1,nz_um
L00343        do id=1,ndm
L00344         iii=iii+1
L00345         ind_zd(iz_u,id)=iii
L00346        enddo
L00347        enddo
L00348
L00349        if (num_urban_hi.ge.nz_um)then
L00350            write(*,*)'nz_um too small, please increase to at least ', num_urban_hi+1
L00351            stop
L00352        endif
L00353
L00354        do ix=its,ite
L00355            do iy=jts,jte
L00356                do iz_u=1,nz_um
L00357                    hi_urb(ix,iz_u,iy)=0.
L00358                enddo
L00359            enddo
L00360        enddo
L00361
L00362        do ix=its,ite
L00363        do iy=jts,jte
L00364         z(ix,kts,iy)=0.
L00365         do iz=kts+1,kte+1
L00366          z(ix,iz,iy)=z(ix,iz-1,iy)+dz8w(ix,iz-1,iy)
L00367         enddo !iz
L00368         do iz_u=1,num_urban_hi
L00369            hi_urb(ix,iz_u,iy)= hi_urb2d(ix,iz_u,iy)
L00370         enddo !iz_u
L00371        enddo
L00372        enddo
L00373
L00374
L00375        if (first) then                          ! True only on first call
L00376
L00377           call init_para(alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,&
L00378                  twini_u,trini_u,tgini_u,albg_u,albw_u,albr_u,emg_u,emw_u,&
L00379                  emr_u,z0g_u,z0r_u,nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b)
L00380
L00381  !      Initialisation of the urban parameters and calculation of the view factors
L00382
L00383           call icBEP(nd_u,h_b,d_b,ss_u,pb_u,nz_u,z_u)
L00384
L00385           first=.false.
L00386
L00387        endif ! first
L00388
L00389        do ix=its,ite
L00390        do iy=jts,jte
L00391          if (FRC_URB2D(ix,iy).gt.0.) then    ! Calling BEP only for existing urban classes.
L00392             iurb=UTYPE_URB2D(ix,iy)
L00393
L00394             hi_urb1D=0.
L00395             do iz_u=1,nz_um
L00396                hi_urb1D(iz_u)=hi_urb(ix,iz_u,iy)
L00397             enddo
L00398
L00399             call icBEPHI_XY(hb_u,hi_urb1D,ss_urb,pb_urb,    &
L00400                             nz_urb(iurb),z_u)
L00401
L00402             call param(iurb,nz_u(iurb),nz_urb(iurb),nzurban(iurb),    &
L00403                        nd_u(iurb),csg_u,csg,alag_u,alag,csr_u,csr,    &
L00404                        alar_u,alar,csw_u,csw,alaw_u,alaw,             &
L00405                        ws_u,ws,bs_u,bs,z0g_u,z0r_u,z0,                &
L00406                        strd_u,strd,drst_u,drst,ss_u,ss_urb,ss,pb_u,   &
L00407                        pb_urb,pb,lp_urb2d(ix,iy),                     &
L00408                        lb_urb2d(ix,iy),hgt_urb2d(ix,iy),FRC_URB2D(ix,iy))
L00409  !
L00410  !We compute the view factors in the icBEP_XY routine
L00411  !
L00412
L00413             call icBEP_XY(iurb,fww_u,fwg_u,fgw_u,fsw_u,fws_u,fsg_u,   &
L00414                           nd_u(iurb),strd,ws,nzurban(iurb),z_u)
L00415
L00416         do iz= kts,kte
L00417            ua1D(iz)=u_phy(ix,iz,iy)
L00418            va1D(iz)=v_phy(ix,iz,iy)
L00419        pt1D(iz)=th_phy(ix,iz,iy)
L00420        da1D(iz)=rho(ix,iz,iy)
L00421        pr1D(iz)=p_phy(ix,iz,iy)
L00422  !     pt01D(iz)=th_phy(ix,iz,iy)
L00423        pt01D(iz)=300.
L00424        z1D(iz)=z(ix,iz,iy)
L00425            a_u1D(iz)=0.
L00426            a_v1D(iz)=0.
L00427            a_t1D(iz)=0.
L00428            a_e1D(iz)=0.
L00429            b_u1D(iz)=0.
L00430            b_v1D(iz)=0.
L00431            b_t1D(iz)=0.
L00432            b_e1D(iz)=0.
L00433           enddo
L00434       z1D(kte+1)=z(ix,kte+1,iy)
L00435
L00436           do id=1,ndm
L00437           do iz_u=1,nz_um
L00438           do iw=1,nwr_u
L00439  !          tw1D(2*id-1,iz_u,iw)=tw1_u(ix,iy,ind_zwd(iz_u,iw,id))
L00440  !          tw1D(2*id,iz_u,iw)=tw2_u(ix,iy,ind_zwd(iz_u,iw,id))
```

```
L00441              if(ind_zwd(iz_u,iw,id).gt.urban_map_zwd)write(*,*)'ind_zwd too big w',ind_zwd(iz_u,iw,id)
L00442             tw1D(2*id-1,iz_u,iw)=tw1_urb4d(ix,ind_zwd(iz_u,iw,id),iy)
L00443             tw1D(2*id,iz_u,iw)=tw2_urb4d(ix,ind_zwd(iz_u,iw,id),iy)
L00444            enddo
L00445            enddo
L00446            enddo
L00447
L00448            do id=1,ndm
L00449             do ig=1,ng_u
L00450  !            tg1D(id,ig)=tg_u(ix,iy,ind_gd(ig,id))
L00451               tg1D(id,ig)=tgb_urb4d(ix,ind_gd(ig,id),iy)
L00452             enddo
L00453             do iz_u=1,nz_um
L00454             do ir=1,nwr_u
L00455  !            tr1D(id,iz_u,ir)=tr_u(ix,iy,ind_zwd(iz_u,ir,id))
L00456               if(ind_zwd(iz_u,ir,id).gt.urban_map_zwd)write(*,*)'ind_zwd too big r',ind_zwd(iz_u,ir,id)
L00457               tr1D(id,iz_u,ir)=trb_urb4d(ix,ind_zwd(iz_u,ir,id),iy)
L00458             enddo
L00459             enddo
L00460            enddo
L00461
L00462            do id=1,ndm
L00463         do iz=1,nz_um
L00464  !       sfw1D(2*id-1,iz)=sfw1(ix,iy,ind_zd(iz,id))
L00465  !       sfw1D(2*id,iz)=sfw2(ix,iy,ind_zd(iz,id))
L00466          sfw1D(2*id-1,iz)=sfw1_urb3d(ix,ind_zd(iz,id),iy)
L00467          sfw1D(2*id,iz)=sfw2_urb3d(ix,ind_zd(iz,id),iy)
L00468         enddo
L00469         enddo
L00470
L00471         do id=1,ndm
L00472  !       sfg1D(id)=sfg(ix,iy,id)
L00473          sfg1D(id)=sfg_urb3d(ix,id,iy)
L00474         enddo
L00475
L00476         do id=1,ndm
L00477         do iz=1,nz_um
L00478  !       sfr1D(id,iz)=sfr(ix,iy,ind_zd(iz,id))
L00479          sfr1D(id,iz)=sfr_urb3d(ix,ind_zd(iz,id),iy)
L00480         enddo
L00481         enddo
L00482
L00483
L00484            rs1D=swdown(ix,iy)
L00485            rld1D=glw(ix,iy)
L00486            time_h=(itimestep*dt)/3600.+gmt
L00487
L00488            zr1D=acos(COSZ_URB2D(ix,iy))
L00489            deltar1D=DECLIN_URB
L00490            ah1D=OMG_URB2D(ix,iy)
L00491  !     call angle(xlong(ix,iy),xlat(ix,iy),julday,time_h,zr1D,deltar1D,ah1D)
L00492  !         write(*,*) 'entro en BEP1D'
L00493            call BEP1D(iurb,kms,kme,kts,kte,z1D,dt,ua1D,va1D,pt1D,da1D,pr1D,pt01D,  &
L00494                       zr1D,deltar1D,ah1D,rs1D,rld1D,                       &
L00495                       alag,alaw,alar,csg,csw,csr,                          &
L00496                       albg_u(iurb),albw_u(iurb),albr_u(iurb),              &
L00497                       emg_u(iurb),emw_u(iurb),emr_u(iurb),                 &
L00498                       fww_u,fwg_u,fgw_u,fsw_u,                             &
L00499                       fws_u,fsg_u,z0,                                      &
L00500                       nd_u(iurb),strd,drst,ws,bs,ss,pb,                    &
L00501                       nzurban(iurb),z_u,                                   &
L00502                       tw1D,tg1D,tr1D,sfw1D,sfg1D,sfr1D,                    &
L00503                       a_u1D,a_v1D,a_t1D,a_e1D,                             &
L00504                       b_u1D,b_v1D,b_t1D,b_e1D,                             &
L00505                       dlg1D,dl_u1D,sf1D,vl1D,rl_up(ix,iy),                 &
L00506                       rs_abs(ix,iy),emiss(ix,iy),grdflx_urb(ix,iy))
L00507  !         write(*,*) 'salgo de BEP1D'
L00508            do id=1,ndm
L00509         do iz=1,nz_um
L00510          sfw1_urb3d(ix,ind_zd(iz,id),iy)=sfw1D(2*id-1,iz)
L00511          sfw2_urb3d(ix,ind_zd(iz,id),iy)=sfw1D(2*id,iz)
L00512         enddo
L00513         enddo
L00514
L00515         do id=1,ndm
L00516          sfg_urb3d(ix,id,iy)=sfg1D(id)
L00517         enddo
L00518
L00519         do id=1,ndm
L00520         do iz=1,nz_um
L00521          sfr_urb3d(ix,ind_zd(iz,id),iy)=sfr1D(id,iz)
L00522         enddo
L00523         enddo
L00524  !
L00525            do id=1,ndm
L00526            do iz_u=1,nz_um
L00527            do iw=1,nwr_u
L00528             tw1_urb4d(ix,ind_zwd(iz_u,iw,id),iy)=tw1D(2*id-1,iz_u,iw)
L00529             tw2_urb4d(ix,ind_zwd(iz_u,iw,id),iy)=tw1D(2*id,iz_u,iw)
L00530            enddo
L00531            enddo
L00532            enddo
L00533
L00534            do id=1,ndm
L00535             do ig=1,ng_u
L00536              tgb_urb4d(ix,ind_gd(ig,id),iy)=tg1D(id,ig)
L00537             enddo
L00538             do iz_u=1,nz_um
L00539             do ir=1,nwr_u
L00540              trb_urb4d(ix,ind_zwd(iz_u,ir,id),iy)=tr1D(id,iz_u,ir)
L00541             enddo
L00542             enddo
L00543            enddo
L00544
L00545             sf(ix,kts:kte,iy)=0.
L00546             vl(ix,kts:kte,iy)=0.
L00547             a_u(ix,kts:kte,iy)=0.
L00548             a_v(ix,kts:kte,iy)=0.
L00549             a_t(ix,kts:kte,iy)=0.
```

```
L00550            a_e(ix,kts:kte,iy)=0.
L00551            b_u(ix,kts:kte,iy)=0.
L00552            b_v(ix,kts:kte,iy)=0.
L00553            b_t(ix,kts:kte,iy)=0.
L00554            b_e(ix,kts:kte,iy)=0.
L00555            b_q(ix,kts:kte,iy)=0.
L00556            dlg(ix,kts:kte,iy)=0.
L00557            dl_u(ix,kts:kte,iy)=0.
L00558
L00559          do iz= kts,kte
L00560            sf(ix,iz,iy)=sf1D(iz)
L00561            vl(ix,iz,iy)=vl1D(iz)
L00562            a_u(ix,iz,iy)=a_u1D(iz)
L00563            a_v(ix,iz,iy)=a_v1D(iz)
L00564            a_t(ix,iz,iy)=a_t1D(iz)
L00565            a_e(ix,iz,iy)=a_e1D(iz)
L00566            b_u(ix,iz,iy)=b_u1D(iz)
L00567            b_v(ix,iz,iy)=b_v1D(iz)
L00568            b_t(ix,iz,iy)=b_t1D(iz)
L00569            b_e(ix,iz,iy)=b_e1D(iz)
L00570            dlg(ix,iz,iy)=dlg1D(iz)
L00571            dl_u(ix,iz,iy)=dl_u1D(iz)
L00572           enddo
L00573           sf(ix,kte+1,iy)=sf1D(kte+1)
L00574  !
L00575           endif ! FRC_URB2D
L00576
L00577         enddo  ! iy
L00578         enddo  ! ix
L00579
L00580
L00581           time_bep=time_bep+dt
L00582
L00583
L00584         return
L00585         end subroutine BEP
```

讲解：与 BEP_BEM 同构但瘦身：① 折叠索引表只保 **4 张**（ind_zwd/ind_gd/ind_zd + 墙面通量用 ind_zd）——BEM 的 ind_bd/ind_wd/ind_gbd/ind_fbd 都不需要；② 逐点流程：param（含 lp/lb/hgt 反解街道几何，同 14a § 六）→ icBEP_XY（逐点视野因子）→ 折叠 tw1/tw2/trb/tgb 与 sfw/sfg/sfr → BEP1D → 写回。③ 折叠时的越界哨兵（ind_zwd>urban_map_zwd 时 write 提示但不中断——比 bep_bem 的 stop 温和）；④time_bep 累计但基本无用（调试遗留）；⑤ 注意屋顶温度折叠用 **ind_zwd**（L457/540）而非 bep_bem 的 ind_zrd——两文件的同名维度表编号不同源，改任何一边都不能想当然。

五、柱求解器 BEP1D（全文）

```
!file: phys/module_sf_bep.F    L589-L905    (subroutine BEP1D)
L00589         subroutine BEP1D(iurb,kms,kme,kts,kte,z,dt,ua,va,pt,da,pr,pt0,  &
L00590                          zr,deltar,ah,rs,rld,                           &
L00591                          alag,alaw,alar,csg,csw,csr,                    &
L00592                          albg,albw,albr,emg,emw,emr,                    &
L00593                          fww,fwg,fgw,fsw,fws,fsg,z0,                    &
L00594                          ndu,strd,drst,ws,bs,ss,pb,                     &
L00595                          nzu,z_u,                                       &
L00596                          tw,tg,tr,sfw,sfg,sfr,                          &
L00597                          a_u,a_v,a_t,a_e,                               &
L00598                          b_u,b_v,b_t,b_e,                               &
L00599                          dlg,dl_u,sf,vl,rl_up,rs_abs,emiss,grdflx_urb)
L00600
L00601  ! ----------------------------------------------------------------------
L00602  ! This routine computes the effects of buildings on momentum, heat and
L00603  !  TKE (turbulent kinetic energy) sources or sinks and on the mixing length.
L00604  ! It provides momentum, heat and TKE sources or sinks at different levels of a
L00605  !  mesoscale grid defined by the altitude of its cell interfaces "z" and
L00606  !  its number of levels "nz".
L00607  ! The meteorological input parameters (wind, temperature, solar radiation)
L00608  !  are specified on the "mesoscale grid".
L00609  ! The inputs concerning the building and street charateristics are defined
L00610  !  on a "urban grid". The "urban grid" is defined with its number of levels
L00611  !  "nz_u" and its space step "dz_u".
L00612  ! The input parameters are interpolated on the "urban grid". The sources or sinks
L00613  !  are calculated on the "urban grid". Finally the sources or sinks are
L00614  !  interpolated on the "mesoscale grid".
L00615
L00616
L00617  !  Mesoscale grid            Urban grid             Mesoscale grid
L00618  !
L00619  ! z(4)    ---                                            ---
L00620  !          |                                              |
L00621  !          |                                              |
L00622  !          |   Interpolation                 Interpolation  |
L00623  !          |            Sources or sinks calculation        |
L00624  ! z(3)    ---                                            ---
L00625  !          | ua              ua_u  ---  uv_a         a_u    |
L00626  !          | va              va_u   |   uv_b         b_u    |
L00627  !          | pt              pt_u  ---  uh_b         a_v    |
L00628  ! z(2)    ---                       |    etc...      etc...---
L00629  !          |                z_u(1) ---                      |
L00630  !          |                        |                       |
L00631  ! z(1) ----------------------------------------------------------
L00632
L00633  !
L00634  ! Reference:
L00635  ! Martilli, A., Clappier, A., Rotach, M.W.:2002, 'AN URBAN SURFACE EXCHANGE
L00636  ! PARAMETERISATION FOR MESOSCALE MODELS', Boundary-Layer Meteorolgy 104:
```

```
L00637  ! 261-304
L00638
L00639  ! ------------------------------------------------------------------------
L00640
L00641        implicit none
L00642
L00643
L00644
L00645  ! ------------------------------------------------------------------------
L00646  ! INPUT:
L00647  ! ------------------------------------------------------------------------
L00648
L00649  ! Data relative to the "mesoscale grid"
L00650
L00651  !      integer nz                ! Number of vertical levels
L00652        integer kms,kme,kts,kte
L00653        real z(kms:kme)             ! Altitude above the ground of the cell interfaces.
L00654        real ua(kms:kme)              ! Wind speed in the x direction
L00655        real va(kms:kme)              ! Wind speed in the y direction
L00656        real pt(kms:kme)              ! Potential temperature
L00657        real da(kms:kme)              ! Air density
L00658        real pr(kms:kme)              ! Air pressure
L00659        real pt0(kms:kme)             ! Reference potential temperature (could be equal to "pt")
L00660        real dt                     ! Time step
L00661        real zr                     ! Zenith angle
L00662        real deltar                 ! Declination of the sun
L00663        real ah                     ! Hour angle
L00664        real rs                     ! Solar radiation
L00665        real rld                    ! Downward flux of the longwave radiation
L00666
L00667  ! Data relative to the "urban grid"
L00668
L00669        integer iurb                ! Current urban class
L00670
L00671  !    Building parameters
L00672        real alag(ng_u)             ! Ground thermal diffusivity [m^2 s^-1]
L00673        real alaw(nwr_u)            ! Wall thermal diffusivity [m^2 s^-1]
L00674        real alar(nwr_u)            ! Roof thermal diffusivity [m^2 s^-1]
L00675        real csg(ng_u)              ! Specific heat of the ground material [J m^3 K^-1]
L00676        real csw(nwr_u)             ! Specific heat of the wall material [J m^3 K^-1]
L00677        real csr(nwr_u)             ! Specific heat of the roof material [J m^3 K^-1]
L00678
L00679  !    Radiation parameters
L00680        real albg                   ! Albedo of the ground
L00681        real albw                   ! Albedo of the wall
L00682        real albr                   ! Albedo of the roof
L00683        real emg                    ! Emissivity of ground
L00684        real emw                    ! Emissivity of wall
L00685        real emr                    ! Emissivity of roof
L00686
L00687  !    fww,fwg,fgw,fsw,fsg are the view factors used to compute the long and
L00688  !    short wave radation.
L00689  !    The calculation of these factor is explained in the Appendix A of the BLM paper
L00690        real fww(nz_um,nz_um,ndm,nurbm)  !  from wall to wall
L00691        real fwg(nz_um,ndm,nurbm)        !  from wall to ground
L00692        real fgw(nz_um,ndm,nurbm)        !  from ground to wall
L00693        real fsw(nz_um,ndm,nurbm)        !  from sky to wall
L00694        real fws(nz_um,ndm,nurbm)        !  from wall to sky
L00695        real fsg(ndm,nurbm)              !  from sky to ground
L00696
L00697  !    Roughness parameters
L00698        real z0(ndm,nz_um)            ! Roughness lengths "profiles"
L00699
L00700  !    Street parameters
L00701        integer ndu                   ! Number of street direction for each urban class
L00702        real strd(ndm)                ! Street length (set to a greater value then the horizontal length of the cells)
L00703        real drst(ndm)                ! Street direction
L00704        real ws(ndm)                  ! Street width
L00705        real bs(ndm)                  ! Building width
L00706        real ss(nz_um)                ! The probability that a building has an height equal to "z"
L00707        real pb(nz_um)                ! The probability that a building has an height greater or equal to "z"
L00708
L00709  !    Grid parameters
L00710        integer nzu                   ! Number of layer in the urban grid
L00711        real z_u(nz_um)               ! Height of the urban grid levels
L00712
L00713
L00714  ! ------------------------------------------------------------------------
L00715  ! INPUT-OUTPUT
L00716  ! ------------------------------------------------------------------------
L00717
L00718  ! Data relative to the "urban grid" which should be stored from the current time step to the next one
L00719
L00720        real tw(2*ndm,nz_um,nwr_u)  ! Temperature in each layer of the wall [K]
L00721        real tr(ndm,nz_um,nwr_u)  ! Temperature in each layer of the roof [K]
L00722        real tg(ndm,ng_u)           ! Temperature in each layer of the ground [K]
L00723        real sfw(2*ndm,nz_um)       ! Sensible heat flux from walls
L00724        real sfg(ndm)               ! Sensible heat flux from ground (road)
L00725        real sfr(ndm,nz_um)       ! Sensible heat flux from roofs
L00726        real gfg(ndm)               ! Heat flux transferred from the surface of the ground (road) towards the interior
L00727        real gfr(ndm,nz_um)     ! Heat flux transferred from the surface of the roof towards the interior
L00728        real gfw(2*ndm,nz_um)     ! Heat flux transfered from the surface of the walls towards the interior
L00729  ! ------------------------------------------------------------------------
L00730  ! OUTPUT:
L00731  ! ------------------------------------------------------------------------
L00732
L00733  ! Data relative to the "mesoscale grid"
L00734
L00735        real sf(kms:kme)              ! Surface of the "mesoscale grid" cells taking into account the buildings
L00736        real vl(kms:kme)              ! Volume of the "mesoscale grid" cells taking into account the buildings
L00737
L00738  !    Implicit and explicit components of the source and sink terms at each levels,
L00739  !     the fluxes can be computed as follow: FX = A*X + B    example: Heat fluxes = a_t * pt + b_t
L00740        real a_u(kms:kme)             ! Implicit component of the momentum sources or sinks in the X-direction
L00741        real a_v(kms:kme)             ! Implicit component of the momentum sources or sinks in the Y-direction
L00742        real a_t(kms:kme)             ! Implicit component of the heat sources or sinks
L00743        real a_e(kms:kme)             ! Implicit component of the TKE sources or sinks
L00744        real b_u(kms:kme)             ! Explicit component of the momentum sources or sinks in the X-direction
L00745        real b_v(kms:kme)             ! Explicit component of the momentum sources or sinks in the Y-direction
```

```
L00746        real b_t(kms:kme)              ! Explicit component of the heat sources or sinks
L00747        real b_e(kms:kme)              ! Explicit component of the TKE sources or sinks
L00748        real dlg(kms:kme)              ! Height above ground (L_ground in formula (24) of the BLM paper).
L00749        real dl_u(kms:kme)             ! Length scale (lb in formula (22) ofthe BLM paper).
L00750
L00751  ! ------------------------------------------------------------------------
L00752  ! LOCAL:
L00753  ! ------------------------------------------------------------------------
L00754
L00755        real dz(kms:kme)               ! vertical space steps of the "mesoscale grid"
L00756
L00757  ! Data interpolated from the "mesoscale grid" to the "urban grid"
L00758
L00759        real ua_u(nz_um)           ! Wind speed in the x direction
L00760        real va_u(nz_um)           ! Wind speed in the y direction
L00761        real pt_u(nz_um)           ! Potential temperature
L00762        real da_u(nz_um)           ! Air density
L00763        real pt0_u(nz_um)          ! Reference potential temperature
L00764        real pr_u(nz_um)           ! Air pressure
L00765
L00766  ! Solar radiation at each level of the "urban grid"
L00767
L00768        real rsg(ndm)              ! Short wave radiation from the ground
L00769        real rsw(2*ndm,nz_um)      ! Short wave radiation from the walls
L00770        real rlg(ndm)              ! Long wave radiation from the ground
L00771        real rlw(2*ndm,nz_um)      ! Long wave radiation from the walls
L00772
L00773  ! Potential temperature of the surfaces at each level of the "urban grid"
L00774
L00775        real ptg(ndm)              ! Ground potential temperatures
L00776        real ptr(ndm,nz_um)      ! Roof potential temperatures
L00777        real ptw(2*ndm,nz_um)      ! Walls potential temperatures
L00778
L00779
L00780  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L00781  !  vertical surfaces (walls) ans horizontal surfaces (roofs and street)
L00782  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L00783  !  Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L00784
L00785        real uhb_u(ndm,nz_um)    ! U (wind component) Horizontal surfaces, B (explicit) term
L00786        real uva_u(2*ndm,nz_um)    ! U (wind component)   Vertical surfaces, A (implicit) term
L00787        real uvb_u(2*ndm,nz_um)    ! U (wind component)   Vertical surfaces, B (explicit) term
L00788        real vhb_u(ndm,nz_um)    ! V (wind component) Horizontal surfaces, B (explicit) term
L00789        real vva_u(2*ndm,nz_um)    ! V (wind component)   Vertical surfaces, A (implicit) term
L00790        real vvb_u(2*ndm,nz_um)    ! V (wind component)   Vertical surfaces, B (explicit) term
L00791        real thb_u(ndm,nz_um)    ! Temperature        Horizontal surfaces, B (explicit) term
L00792        real tva_u(2*ndm,nz_um)    ! Temperature          Vertical surfaces, A (implicit) term
L00793        real tvb_u(2*ndm,nz_um)    ! Temperature          Vertical surfaces, B (explicit) term
L00794        real ehb_u(ndm,nz_um)    ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L00795        real evb_u(2*ndm,nz_um)    ! Energy (TKE)         Vertical surfaces, B (explicit) term
L00796
L00797  !
L00798        real rs_abs ! solar radiation absorbed by urban surfaces
L00799        real rl_up ! longwave radiation emitted by urban surface to the atmosphere
L00800        real emiss ! mean emissivity of the urban surface
L00801        real grdflx_urb ! ground heat flux
L00802        integer iz,id
L00803        integer iw,ix,iy
L00804
L00805  ! ------------------------------------------------------------------------
L00806  ! END VARIABLES DEFINITIONS
L00807  ! ------------------------------------------------------------------------
L00808
L00809  ! Fix some usefull parameters for the computation of the sources or sinks
L00810
L00811        do iz=kts,kte
L00812           dz(iz)=z(iz+1)-z(iz)
L00813        end do
L00814
L00815  ! Interpolation on the "urban grid"
L00816        call interpol(kms,kme,kts,kte,nzu,z,z_u,ua,ua_u)
L00817        call interpol(kms,kme,kts,kte,nzu,z,z_u,va,va_u)
L00818        call interpol(kms,kme,kts,kte,nzu,z,z_u,pt,pt_u)
L00819        call interpol(kms,kme,kts,kte,nzu,z,z_u,pt0,pt0_u)
L00820        call interpol(kms,kme,kts,kte,nzu,z,z_u,pr,pr_u)
L00821        call interpol(kms,kme,kts,kte,nzu,z,z_u,da,da_u)
L00822
L00823
L00824  ! Compute the modification of the radiation due to the buildings
L00825
L00826        call modif_rad(iurb,ndu,nzu,z_u,ws,               &
L00827                       drst,strd,ss,pb,                   &
L00828                       tw,tg,albg,albw,emw,emg,           &
L00829                       fww,fwg,fgw,fsw,fsg,               &
L00830                       zr,deltar,ah,                      &
L00831                       rs,rld,rsw,rsg,rlw,rlg)
L00832
L00833  ! calculation of the urban albedo and the upward long wave radiation
L00834
L00835         call upward_rad(ndu,nzu,ws,bs,                   &
L00836                         sigma,pb,ss,                     &
L00837                         tg,emg,albg,rlg,rsg,sfg,         &
L00838                         tw,emw,albw,rlw,rsw,sfw,         &
L00839                         tr,emr,albr,rld,rs,sfr,          &
L00840                         rs_abs,rl_up,emiss,grdflx_urb)
L00841
L00842  ! Compute the surface temperatures
L00843
L00844
L00845        call surf_temp(nzu,ndu,pr_u,dt,ss,                &
L00846                       rs,rld,rsg,rlg,rsw,rlw,            &
L00847                       tg,alag,csg,emg,albg,ptg,sfg,gfg,  &
L00848                       tr,alar,csr,emr,albr,ptr,sfr,gfr,  &
L00849                       tw,alaw,csw,emw,albw,ptw,sfw,gfw)
L00850
L00851
L00852  ! Compute the implicit and explicit components of the sources or sinks on the "urban grid"
L00853
L00854        call buildings(ndu,nzu,z0,ua_u,va_u,                      &
```

```
L00855                      pt_u,pt0_u,ptg,ptr,da_u,ptw,drst,           &
L00856                      uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u,  &
L00857                      uhb_u,vhb_u,thb_u,ehb_u,ss,dt)
L00858
L00859
L00860  ! Calculation of the sensible heat fluxes for the ground, the wall and roof
L00861  ! Sensible Heat Flux = density * Cp_U * ( A* potential temperature + B )
L00862  ! where A and B are the implicit and explicit components of the heat sources or sinks.
L00863  !
L00864  !
L00865
L00866        do id=1,ndu
L00867           sfg(id)=-da_u(1)*cp_u*thb_u(id,1)
L00868           do iz=2,nzu
L00869              sfr(id,iz)=-da_u(iz)*cp_u*thb_u(id,iz)
L00870           enddo
L00871
L00872           do iz=1,nzu
L00873              sfw(2*id-1,iz)=-da_u(iz)*cp_u*(tvb_u(2*id-1,iz)+     &
L00874                   tva_u(2*id-1,iz)*pt_u(iz))
L00875              sfw(2*id,iz)=-da_u(iz)*cp_u*(tvb_u(2*id,iz)+         &
L00876                   tva_u(2*id,iz)*pt_u(iz))
L00877           enddo
L00878        enddo
L00879
L00880  ! calculation of the urban albedo and the upward long wave radiation
L00881
L00882  !!       call upward_rad(ndu,nzu,ws,bs,                       &
L00883  !!                       sigma,pb,ss,                         &
L00884  !!                       tg,emg,albg,rlg,rsg,sfg,             &
L00885  !!                       tw,emw,albw,rlw,rsw,sfw,             &
L00886  !!                       tr,emr,albr,rld,rs,sfr,              &
L00887  !!                       rs_abs,rl_up,emiss,grdflx_urb)
L00888
L00889  ! Interpolation on the "mesoscale grid"
L00890
L00891        call urban_meso(ndu,kms,kme,kts,kte,nzu,z,dz,z_u,pb,ss,bs,ws,sf, &
L00892                        vl,uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u,     &
L00893                        uhb_u,vhb_u,thb_u,ehb_u,                          &
L00894                        a_u,a_v,a_t,a_e,b_u,b_v,b_t,b_e)
L00895
L00896
L00897  ! computation of the mean road temperature tsk (this value could be used
L00898  ! to replace the surface temperature in the radiation routines, if needed).
L00899
L00900  ! Calculation of the length scale taking into account the buildings effects
L00901
L00902        call interp_length(ndu,kms,kme,kts,kte,nzu,z_u,z,ss,ws,bs,dlg,dl_u)
L00903
L00904        return
L00905        end subroutine BEP1D
```

讲解：文件头的 ASCII 示意图（L589–632）值得一看：左侧模式网格 z(1..4)、右侧城市网格 z_u，中间「interpolation→sources→interpolation back」——整篇 BLM 论文的流程图。执行序六步：①interpol 六个量（比 bep_bem 少 qv——无湿度源项）；②modif_rad（阴影 +多次反射，无直射/漫射分拆）；③upward_rad（给辐射方案的等效量）；④**surf_temp 一件管三面**（地面 +屋顶 +墙的导热全在这一个调用里——对照 14a：BEP_BEM 版把屋顶/墙交给了 BEM）；⑤buildings 源项 + 由 A/B 反算表面感热（sfg=−ρ • cp • thb_u 等——墙面感热公式 tvB+tva • pt_u 就是 A • X+B 的字面实现）；⑥urban_meso + interp_length。没有水量收支、没有 BEM 循环、没有 NEW CDRAG（buildings 用常数 cdrag=0.4）。

六、与底座同源的单元（差异注记）

```
!file: phys/module_sf_bep.F    L910-L1075    (subroutine param)
L00910         subroutine param(iurb,nzu,nzurb,nzurban,ndu,                &
L00911                          csg_u,csg,alag_u,alag,csr_u,csr,           &
L00912                          alar_u,alar,csw_u,csw,alaw_u,alaw,         &
L00913                          ws_u,ws,bs_u,bs,z0g_u,z0r_u,z0,            &
L00914                          strd_u,strd,drst_u,drst,ss_u,ss_urb,ss,pb_u,   &
L00915                          pb_urb,pb,lp_urb,lb_urb,hgt_urb,frc_urb)
L00916
L00917  ! ----------------------------------------------------------------------
L00918  !    This routine prepare some usefull parameters
L00919  ! ----------------------------------------------------------------------
L00920
L00921        implicit none
L00922
L00923
L00924  ! ----------------------------------------------------------------------
L00925  ! INPUT:
L00926  ! ----------------------------------------------------------------------
L00927        integer iurb                ! Current urban class
L00928        integer nzu                 ! Number of vertical urban levels in the current class
L00929        integer nzurb               ! Number of vertical urban levels in the current class
L00930        integer ndu                 ! Number of street direction for the current urban class
L00931        real alag_u(nurbm)          ! Ground thermal diffusivity [m^2 s^-1]
L00932        real alar_u(nurbm)          ! Roof thermal diffusivity [m^2 s^-1]
L00933        real alaw_u(nurbm)          ! Wall thermal diffusivity [m^2 s^-1]
L00934        real bs_u(ndm,nurbm)        ! Building width
L00935        real csg_u(nurbm)           ! Specific heat of the ground material [J m^3 K^-1]
L00936        real csr_u(nurbm)           ! Specific heat of the roof material [J m^3 K^-1]
L00937        real csw_u(nurbm)           ! Specific heat of the wall material [J m^3 K^-1]
L00938        real drst_u(ndm,nurbm)      ! Street direction
L00939        real strd_u(ndm,nurbm)      ! Street length
L00940        real ws_u(ndm,nurbm)        ! Street width
L00941        real z0g_u(nurbm)           ! The ground's roughness length
```

```
L00942        real z0r_u(nurbm)             ! The roof's roughness length
L00943        real ss_u(nz_um,nurbm)     ! The probability that a building has an height equal to "z"
L00944        real pb_u(nz_um,nurbm)     ! The probability that a building has an height greater or equal to "z"
L00945        real ss_urb(nz_um)     ! The probability that a building has an height equal to "z"
L00946        real pb_urb(nz_um)     ! The probability that a building has an height greater or equal to "z"
L00947        real lp_urb                ! Building plan area density
L00948        real lb_urb                ! Building surface area to plan area ratio
L00949        real hgt_urb               ! Average building height weighted by building plan area [m]
L00950        real frc_urb               ! Urban fraction
L00951
L00952  ! ------------------------------------------------------------------
L00953  ! OUTPUT:
L00954  ! ------------------------------------------------------------------
L00955        real alag(ng_u)           ! Ground thermal diffusivity at each ground levels
L00956        real alar(nwr_u)           ! Roof thermal diffusivity at each roof levels
L00957        real alaw(nwr_u)           ! Wall thermal diffusivity at each wall levels
L00958        real csg(ng_u)            ! Specific heat of the ground material at each ground levels
L00959        real csr(nwr_u)            ! Specific heat of the roof material at each roof levels
L00960        real csw(nwr_u)            ! Specific heat of the wall material at each wall levels
L00961        real bs(ndm)              ! Building width for the current urban class
L00962        real drst(ndm)            ! street directions for the current urban class
L00963        real strd(ndm)            ! Street lengths for the current urban class
L00964        real ws(ndm)              ! Street widths of the current urban class
L00965        real z0(ndm,nz_um)      ! Roughness lengths "profiles"
L00966        real ss(nz_um)          ! Probability to have a building with height h
L00967        real pb(nz_um)          ! Probability to have a building with an height equal
L00968        integer nzurban
L00969
L00970  ! ------------------------------------------------------------------
L00971  ! LOCAL:
L00972  ! ------------------------------------------------------------------
L00973        integer id,ig,ir,iw,iz,ihu
L00974
L00975  ! ------------------------------------------------------------------
L00976  ! END VARIABLES DEFINITIONS
L00977  ! ------------------------------------------------------------------
L00978  !
L00979  !Initialize
L00980  !
L00981        ss=0.
L00982        pb=0.
L00983        csg=0.
L00984        alag=0.
L00985        csr=0.
L00986        alar=0.
L00987        csw=0.
L00988        alaw=0.
L00989        z0=0.
L00990        ws=0.
L00991        bs=0.
L00992        strd=0.
L00993        drst=0.
L00994        nzurban=0
L00995
L00996        ihu=0
L00997
L00998         do iz=1,nz_um
L00999            if (ss_urb(iz)/=0.) then
L01000               ihu=1
L01001               exit
L01002            else
L01003               continue
L01004            endif
L01005         enddo
L01006
L01007         if (ihu==1) then
L01008            do iz=1,nzurb+1
L01009               ss(iz)=ss_urb(iz)
L01010               pb(iz)=pb_urb(iz)
L01011            enddo
L01012            nzurban=nzurb
L01013         else
L01014            do iz=1,nzu+1
L01015               ss(iz)=ss_u(iz,iurb)
L01016               pb(iz)=pb_u(iz,iurb)
L01017            end do
L01018            nzurban=nzu
L01019         endif
L01020
L01021         do id=1,ndu
L01022          z0(id,1)=z0g_u(iurb)
L01023          do iz=2,nzurban+1
L01024             z0(id,iz)=z0r_u(iurb)
L01025          enddo
L01026         enddo
L01027
L01028         do ig=1,ng_u
L01029          csg(ig)=csg_u(iurb)
L01030          alag(ig)=alag_u(iurb)
L01031         enddo
L01032
L01033         do ir=1,nwr_u
L01034          csr(ir)=csr_u(iurb)
L01035          alar(ir)=alar_u(iurb)
L01036         enddo
L01037
L01038         do iw=1,nwr_u
L01039          csw(iw)=csw_u(iurb)
L01040          alaw(iw)=alaw_u(iurb)
L01041         enddo
L01042
L01043         do id=1,ndu
L01044          strd(id)=strd_u(id,iurb)
L01045          drst(id)=drst_u(id,iurb)
L01046         enddo
L01047
L01048         do id=1,ndu
L01049            if ((hgt_urb<=0.).OR.(lp_urb<=0.).OR.(lb_urb<=0.)) then
L01050               ws(id)=ws_u(id,iurb)
```

```
L01051              bs(id)=bs_u(id,iurb)
L01052           else if ((lp_urb/frc_urb<1.).and.(lp_urb<lb_urb)) then
L01053                   bs(id)=2.*hgt_urb*lp_urb/(lb_urb-lp_urb)
L01054                   ws(id)=2.*hgt_urb*lp_urb*((frc_urb/lp_urb)-1.)/(lb_urb-lp_urb)
L01055                else
L01056                   ws(id)=ws_u(id,iurb)
L01057                   bs(id)=bs_u(id,iurb)
L01058           endif
L01059        enddo
L01060        do id=1,ndu
L01061           if ((bs(id)<=1.).OR.(bs(id)>=150.)) then
L01062  !           write(*,*) 'WARNING, WIDTH OF THE BUILDING WRONG',id,bs(id)
L01063  !           write(*,*) 'WIDTH OF THE STREET',id,ws(id)
L01064              bs(id)=bs_u(id,iurb)
L01065              ws(id)=ws_u(id,iurb)
L01066           endif
L01067           if ((ws(id)<=1.).OR.(ws(id)>=150.)) then
L01068  !           write(*,*) 'WARNING, WIDTH OF THE STREET WRONG',id,ws(id)
L01069  !           write(*,*) 'WIDTH OF THE BUILDING',id,bs(id)
L01070              bs(id)=bs_u(id,iurb)
L01071              ws(id)=ws_u(id,iurb)
L01072           endif
L01073        enddo
L01074        return
L01075        end subroutine param
```

与 14a § 六差异：无 BEM 材料层（csf/alaf/dzgb/alagb 缺），只展开 csg/csw/csr/alag/alaw/alar 与 dzw/dzr；lp/lb/hgt 反解与 1–150 m 钳位完全相同。

```
!file: phys/module_sf_bep.F    L1080-L1128    (subroutine interpol)
L01080        subroutine interpol(kms,kme,kts,kte,nz_u,z,z_u,c,c_u)
L01081
L01082  ! ------------------------------------------------------------------------
L01083  !  This routine interpolate para
L01084  !  meters from the "mesoscale grid" to
L01085  !  the "urban grid".
L01086  !  See p300 Appendix B.1 of the BLM paper.
L01087  ! ------------------------------------------------------------------------
L01088
L01089        implicit none
L01090
L01091  ! ------------------------------------------------------------------------
L01092  ! INPUT:
L01093  ! ------------------------------------------------------------------------
L01094  ! Data relative to the "mesoscale grid"
L01095        integer kts,kte,kms,kme
L01096        real z(kms:kme)          ! Altitude of the cell interface
L01097        real c(kms:kme)            ! Parameter which has to be interpolated
L01098  ! Data relative to the "urban grid"
L01099        integer nz_u             ! Number of levels
L01100  !!    real z_u(nz_u+1)       ! Altitude of the cell interface
L01101        real z_u(nz_um)        ! Altitude of the cell interface
L01102  ! ------------------------------------------------------------------------
L01103  ! OUTPUT:
L01104  ! ------------------------------------------------------------------------
L01105  !!    real c_u(nz_u)         ! Interpolated paramters in the "urban grid"
L01106        real c_u(nz_um)        ! Interpolated paramters in the "urban grid"
L01107
L01108
L01109  ! LOCAL:
L01110  ! ------------------------------------------------------------------------
L01111        integer iz_u,iz
L01112        real ctot,dz
L01113
L01114  ! ------------------------------------------------------------------------
L01115  ! END VARIABLES DEFINITIONS
L01116  ! ------------------------------------------------------------------------
L01117
L01118         do iz_u=1,nz_u
L01119          ctot=0.
L01120          do iz=kts,kte
L01121           dz=max(min(z(iz+1),z_u(iz_u+1))-max(z(iz),z_u(iz_u)),0.)
L01122           ctot=ctot+c(iz)*dz
L01123          enddo
L01124          c_u(iz_u)=ctot/(z_u(iz_u+1)-z_u(iz_u))
L01125         enddo
L01126
L01127         return
L01128         end subroutine interpol
```

```
!file: phys/module_sf_bep.F    L1133-L1205    (subroutine modif_rad)
L01133        subroutine modif_rad(iurb,nd,nz_u,z,ws,drst,strd,ss,pb,    &
L01134                             tw,tg,albg,albw,emw,emg,               &
L01135                             fww,fwg,fgw,fsw,fsg,                   &
L01136                             zr,deltar,ah,                          &
L01137                             rs,rl,rsw,rsg,rlw,rlg)
L01138
L01139  ! ------------------------------------------------------------------------
L01140  ! This routine computes the modification of the short wave and
L01141  !  long wave radiation due to the buildings.
L01142  ! ------------------------------------------------------------------------
L01143
L01144        implicit none
L01145
L01146
L01147  ! ------------------------------------------------------------------------
L01148  ! INPUT:
L01149  ! ------------------------------------------------------------------------
L01150        integer iurb               ! current urban class
L01151        integer nd                 ! Number of street direction for the current urban class
L01152        integer nz_u               ! Number of layer in the urban grid
L01153        real z(nz_um)            ! Height of the urban grid levels
L01154        real ws(ndm)               ! Street widths of the current urban class
L01155        real drst(ndm)             ! street directions for the current urban class
```

```
L01156        real strd(ndm)            ! Street lengths for the current urban class
L01157        real ss(nz_um)          ! probability to have a building with height h
L01158        real pb(nz_um)          ! probability to have a building with an height equal
L01159        real tw(2*ndm,nz_um,nwr_u) ! Temperature in each layer of the wall [K]
L01160        real tg(ndm,ng_u)         ! Temperature in each layer of the ground [K]
L01161        real albg                 ! Albedo of the ground for the current urban class
L01162        real albw                 ! Albedo of the wall for the current urban class
L01163        real emg                  ! Emissivity of ground for the current urban class
L01164        real emw                  ! Emissivity of wall for the current urban class
L01165        real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
L01166        real fsg(ndm,nurbm)             ! View factors from sky to ground
L01167        real fsw(nz_um,ndm,nurbm)       ! View factors from sky to wall
L01168        real fws(nz_um,ndm,nurbm)       ! View factors from wall to sky
L01169        real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
L01170        real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
L01171        real ah                   ! Hour angle (it should come from the radiation routine)
L01172        real zr                   ! zenith angle
L01173        real deltar               ! Declination of the sun
L01174        real rs                   ! solar radiation
L01175        real rl                   ! downward flux of the longwave radiation
L01176  ! ---------------------------------------------------------------------
L01177  ! OUTPUT:
L01178  ! ---------------------------------------------------------------------
L01179        real rlg(ndm)             ! Long wave radiation at the ground
L01180        real rlw(2*ndm,nz_um)     ! Long wave radiation at the walls
L01181        real rsg(ndm)             ! Short wave radiation at the ground
L01182        real rsw(2*ndm,nz_um)     ! Short wave radiation at the walls
L01183
L01184  ! ---------------------------------------------------------------------
L01185  ! LOCAL:
L01186  ! ---------------------------------------------------------------------
L01187
L01188        integer id,iz
L01189
L01190  !  Calculation of the shadow effects
L01191
L01192        call shadow_mas(nd,nz_u,zr,deltar,ah,drst,ws,ss,pb,z,           &
L01193                        rs,rsw,rsg)
L01194
L01195  ! Calculation of the reflection effects
L01196        do id=1,nd
L01197           call long_rad(iurb,nz_u,id,emw,emg,                 &
L01198                         fwg,fww,fgw,fsw,fsg,tg,tw,rlg,rlw,rl,pb)
L01199
L01200           call short_rad(iurb,nz_u,id,albw,albg,fwg,fww,fgw,rsg,rsw,pb)
L01201
L01202        enddo
L01203
L01204        return
L01205        end subroutine modif_rad
```

modif_rad 差异：**无 swddir/swddif 分量**——BEP 单独版把总短波 rs 直接给 shadow_mas（不区分直射漫射），且无窗户混合（alb_av=pwin • albwin+… 一行不存在）。long_rad/short_rad 内部因此少窗户项。对照 14a § 八。

```
!file: phys/module_sf_bep.F    L1774-L1891    (subroutine shadow_mas)
L01774        subroutine shadow_mas(nd,nz_u,zr,deltar,ah,drst,ws,ss,pb,z,         &
L01775                              rs,rsw,rsg)
L01776
L01777  ! ---------------------------------------------------------------------
L01778  !         Modification of short wave radiation to take into account
L01779  !         the shadow produced by the buildings
L01780  ! ---------------------------------------------------------------------
L01781
L01782        implicit none
L01783
L01784  ! ---------------------------------------------------------------------
L01785  ! INPUT:
L01786  ! ---------------------------------------------------------------------
L01787        integer nd                ! Number of street direction for the current urban class
L01788        integer nz_u              ! number of vertical layers defined in the urban grid
L01789        real ah                   ! Hour angle (it should come from the radiation routine)
L01790        real deltar               ! Declination of the sun
L01791        real drst(ndm)            ! street directions for the current urban class
L01792        real rs                   ! solar radiation
L01793        real ss(nz_um)          ! probability to have a building with height h
L01794        real pb(nz_um)          ! Probability that a building has an height greater or equal to h
L01795        real ws(ndm)              ! Street width of the current urban class
L01796        real z(nz_um)           ! Height of the urban grid levels
L01797        real zr                   ! zenith angle
L01798
L01799  ! ---------------------------------------------------------------------
L01800  ! OUTPUT:
L01801  ! ---------------------------------------------------------------------
L01802        real rsg(ndm)             ! Short wave radiation at the ground
L01803        real rsw(2*ndm,nz_um)     ! Short wave radiation at the walls
L01804
L01805  ! ---------------------------------------------------------------------
L01806  ! LOCAL:
L01807  ! ---------------------------------------------------------------------
L01808        integer id,iz,jz
L01809        real aae,aaw,bbb,phix,rd,rtot,wsd
L01810
L01811  ! ---------------------------------------------------------------------
L01812  ! END VARIABLES DEFINITIONS
L01813  ! ---------------------------------------------------------------------
L01814
L01815        if(rs.eq.0.or.sin(zr).eq.1)then
L01816           do id=1,nd
L01817              rsg(id)=0.
L01818              do iz=1,nz_u
L01819                 rsw(2*id-1,iz)=0.
L01820                 rsw(2*id,iz)=0.
L01821              enddo
L01822           enddo
```

```
L01823        else
L01824  !test
L01825           if(abs(sin(zr)).gt.1.e-10)then
L01826            if(cos(deltar)*sin(ah)/sin(zr).ge.1)then
L01827             bbb=pi/2.
L01828            elseif(cos(deltar)*sin(ah)/sin(zr).le.-1)then
L01829             bbb=-pi/2.
L01830            else
L01831             bbb=asin(cos(deltar)*sin(ah)/sin(zr))
L01832            endif
L01833           else
L01834            if(cos(deltar)*sin(ah).ge.0)then
L01835             bbb=pi/2.
L01836            elseif(cos(deltar)*sin(ah).lt.0)then
L01837             bbb=-pi/2.
L01838            endif
L01839           endif
L01840
L01841           phix=zr
L01842
L01843           do id=1,nd
L01844
L01845              rsg(id)=0.
L01846
L01847              aae=bbb-drst(id)
L01848              aaw=bbb-drst(id)+pi
L01849
L01850              do iz=1,nz_u
L01851                 rsw(2*id-1,iz)=0.
L01852                 rsw(2*id,iz)=0.
L01853              if(pb(iz+1).gt.0.)then
L01854                 do jz=1,nz_u
L01855                  if(abs(sin(aae)).gt.1.e-10)then
L01856                    call shade_wall(z(iz),z(iz+1),z(jz+1),phix,aae,   &
L01857                        ws(id),rd)
L01858                    rsw(2*id-1,iz)=rsw(2*id-1,iz)+rs*rd*ss(jz+1)/pb(iz+1)
L01859                  endif
L01860
L01861                  if(abs(sin(aaw)).gt.1.e-10)then
L01862                    call shade_wall(z(iz),z(iz+1),z(jz+1),phix,aaw,   &
L01863                        ws(id),rd)
L01864                    rsw(2*id,iz)=rsw(2*id,iz)+rs*rd*ss(jz+1)/pb(iz+1)
L01865                  endif
L01866                 enddo
L01867              endif
L01868              enddo
L01869           if(abs(sin(aae)).gt.1.e-10)then
L01870              wsd=abs(ws(id)/sin(aae))
L01871
L01872              do jz=1,nz_u
L01873                 rd=max(0.,wsd-z(jz+1)*tan(phix))
L01874                 rsg(id)=rsg(id)+rs*rd*ss(jz+1)/wsd
L01875              enddo
L01876              rtot=0.
L01877
L01878              do iz=1,nz_u
L01879                 rtot=rtot+(rsw(2*id,iz)+rsw(2*id-1,iz))*            &
L01880                           (z(iz+1)-z(iz))
L01881              enddo
L01882              rtot=rtot+rsg(id)*ws(id)
L01883           else
L01884              rsg(id)=rs
L01885           endif
L01886
L01887            enddo
L01888        endif
L01889
L01890        return
L01891        end subroutine shadow_mas
```

```
!file: phys/module_sf_bep.F    L1896-L1946    (subroutine shade_wall)
L01896        subroutine shade_wall(z1,z2,hu,phix,aa,ws,rd)
L01897
L01898  ! ------------------------------------------------------------------
L01899  ! This routine computes the effects of a shadow induced by a building of
L01900  ! height hu, on a portion of wall between z1 and z2. See equation A10,
L01901  ! and correction described below formula A11, and figure A1. Basically rd
L01902  ! is the ratio between the horizontal surface illuminated and the portion
L01903  ! of wall. Referring to figure A1, multiplying radiation flux density on
L01904  ! a horizontal surface (rs) by x1-x2 we have the radiation energy per
L01905  ! unit time. Dividing this by z2-z1, we obtain the radiation flux
L01906  ! density reaching the portion of the wall between z2 and z1
L01907  ! (everything is assumed in 2D)
L01908  ! ------------------------------------------------------------------
L01909
L01910        implicit none
L01911
L01912  ! ------------------------------------------------------------------
L01913  ! INPUT:
L01914  ! ------------------------------------------------------------------
L01915        real aa                   ! Angle between the sun direction and the face of the wall (A12)
L01916        real hu                   ! Height of the building that generates the shadow
L01917        real phix                 ! Solar zenith angle
L01918        real ws                   ! Width of the street
L01919        real z1                   ! Height of the level z(iz)
L01920        real z2                   ! Height of the level z(iz+1)
L01921
L01922  ! ------------------------------------------------------------------
L01923  ! OUTPUT:
L01924  ! ------------------------------------------------------------------
L01925        real rd                   ! Ratio between (x1-x2)/(z2-z1), see Fig. 1A.
L01926                                  ! Multiplying rd by rs (radiation flux
L01927                                  ! density on a horizontal surface) gives
L01928                                  ! the radiation flux density on the
L01929                                  ! portion of wall between z1 and z2.
L01930  ! ------------------------------------------------------------------
L01931  ! LOCAL:
```

```
L01932  ! ------------------------------------------------------------------
L01933        real x1,x2               ! x1,x2 see Fig. A1.
L01934
L01935  ! ------------------------------------------------------------------
L01936  ! END VARIABLES DEFINITIONS
L01937  ! ------------------------------------------------------------------
L01938
L01939        x1=min((hu-z1)*tan(phix),max(0.,ws/sin(aa)))
L01940
L01941        x2=max((hu-z2)*tan(phix),0.)
L01942
L01943        rd=max(0.,sin(aa)*(max(0.,x1-x2))/(z2-z1))
L01944
L01945        return
L01946        end subroutine shade_wall
```

与 14b § 三逐行同源（几何推导见彼处）。

```
!file: phys/module_sf_bep.F    L1951-L2092    (subroutine long_rad)
L01951        subroutine long_rad(iurb,nz_u,id,emw,emg,                 &
L01952                            fwg,fww,fgw,fsw,fsg,tg,tw,rlg,rlw,rl,pb)
L01953
L01954  ! ------------------------------------------------------------------
L01955  ! This routine computes the effects of the reflections of long-wave
L01956  ! radiation in the street canyon by solving the system
L01957  ! of 2*nz_u+1 eqn. in 2*nz_u+1
L01958  ! unknown defined in A4, A5 and A6 of the paper (pages 295 and 296).
L01959  ! The system is solved by solving A X= B,
L01960  ! with A matrix B vector, and X solution.
L01961  ! ------------------------------------------------------------------
L01962
L01963        implicit none
L01964
L01965
L01966
L01967  ! ------------------------------------------------------------------
L01968  ! INPUT:
L01969  ! ------------------------------------------------------------------
L01970        real emg                        ! Emissivity of ground for the current urban class
L01971        real emw                        ! Emissivity of wall for the current urban class
L01972        real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
L01973        real fsg(ndm,nurbm)             ! View factors from sky to ground
L01974        real fsw(nz_um,ndm,nurbm)       ! View factors from sky to wall
L01975        real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
L01976        real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
L01977        integer id                      ! Current street direction
L01978        integer iurb                    ! Current urban class
L01979        integer nz_u                    ! Number of layer in the urban grid
L01980        real pb(nz_um)                 ! Probability to have a building with an height equal
L01981        real rl                         ! Downward flux of the longwave radiation
L01982        real tg(ndm,ng_u)               ! Temperature in each layer of the ground [K]
L01983        real tw(2*ndm,nz_um,nwr_u)       ! Temperature in each layer of the wall [K]
L01984
L01985
L01986  ! ------------------------------------------------------------------
L01987  ! OUTPUT:
L01988  ! ------------------------------------------------------------------
L01989        real rlg(ndm)                   ! Long wave radiation at the ground
L01990        real rlw(2*ndm,nz_um)           ! Long wave radiation at the walls
L01991
L01992  ! ------------------------------------------------------------------
L01993  ! LOCAL:
L01994  ! ------------------------------------------------------------------
L01995        integer i,j
L01996        real aaa(2*nz_um+1,2*nz_um+1)   ! terms of the matrix
L01997        real bbb(2*nz_um+1)             ! terms of the vector
L01998
L01999  ! ------------------------------------------------------------------
L02000  ! END VARIABLES DEFINITIONS
L02001  ! ------------------------------------------------------------------
L02002
L02003
L02004  ! west wall
L02005
L02006        do i=1,nz_u
L02007
L02008          do j=1,nz_u
L02009           aaa(i,j)=0.
L02010          enddo
L02011
L02012          aaa(i,i)=1.
L02013
L02014          do j=nz_u+1,2*nz_u
L02015           aaa(i,j)=-(1.-emw)*fww(j-nz_u,i,id,iurb)*pb(j-nz_u+1)
L02016          enddo
L02017
L02018  !!      aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i,id,iurb)*pb(i+1)
L02019          aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i,id,iurb)
L02020
L02021          bbb(i)=fsw(i,id,iurb)*rl+emg*fgw(i,id,iurb)*sigma*tg(id,ng_u)**4
L02022          do j=1,nz_u
L02023           bbb(i)=bbb(i)+pb(j+1)*emw*sigma*fww(j,i,id,iurb)*       &
L02024                 tw(2*id,j,nwr_u)**4+                              &
L02025                 fww(j,i,id,iurb)*rl*(1.-pb(j+1))
L02026          enddo
L02027
L02028         enddo
L02029
L02030  ! east wall
L02031
L02032         do i=1+nz_u,2*nz_u
L02033
L02034          do j=1,nz_u
L02035           aaa(i,j)=-(1.-emw)*fww(j,i-nz_u,id,iurb)*pb(j+1)
L02036          enddo
L02037
L02038          do j=1+nz_u,2*nz_u
```

```
L02039           aaa(i,j)=0.
L02040          enddo
L02041
L02042          aaa(i,i)=1.
L02043
L02044  !!      aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i-nz_u,id,iurb)*pb(i-nz_u+1)
L02045          aaa(i,2*nz_u+1)=-(1.-emg)*fgw(i-nz_u,id,iurb)
L02046
L02047          bbb(i)=fsw(i-nz_u,id,iurb)*rl+                              &
L02048                emg*fgw(i-nz_u,id,iurb)*sigma*tg(id,ng_u)**4
L02049
L02050          do j=1,nz_u
L02051           bbb(i)=bbb(i)+pb(j+1)*emw*sigma*fww(j,i-nz_u,id,iurb)*  &
L02052                  tw(2*id-1,j,nwr_u)**4+                           &
L02053                  fww(j,i-nz_u,id,iurb)*rl*(1.-pb(j+1))
L02054          enddo
L02055
L02056         enddo
L02057
L02058  ! ground
L02059         do j=1,nz_u
L02060          aaa(2*nz_u+1,j)=-(1.-emw)*fwg(j,id,iurb)*pb(j+1)
L02061         enddo
L02062
L02063         do j=nz_u+1,2*nz_u
L02064          aaa(2*nz_u+1,j)=-(1.-emw)*fwg(j-nz_u,id,iurb)*pb(j-nz_u+1)
L02065         enddo
L02066
L02067         aaa(2*nz_u+1,2*nz_u+1)=1.
L02068
L02069         bbb(2*nz_u+1)=fsg(id,iurb)*rl
L02070
L02071         do i=1,nz_u
L02072          bbb(2*nz_u+1)=bbb(2*nz_u+1)+emw*sigma*fwg(i,id,iurb)*pb(i+1)*    &
L02073                        (tw(2*id-1,i,nwr_u)**4+tw(2*id,i,nwr_u)**4)+       &
L02074                        2.*fwg(i,id,iurb)*(1.-pb(i+1))*rl
L02075         enddo
L02076
L02077
L02078
L02079         call gaussj(aaa,2*nz_u+1,bbb,2*nz_um+1)
L02080
L02081         do i=1,nz_u
L02082          rlw(2*id-1,i)=bbb(i)
L02083         enddo
L02084
L02085         do i=nz_u+1,2*nz_u
L02086          rlw(2*id,i-nz_u)=bbb(i)
L02087         enddo
L02088
L02089         rlg(id)=bbb(2*nz_u+1)
L02090
L02091         return
L02092         end subroutine long_rad
```

```
!file: phys/module_sf_bep.F    L2097-L2205    (subroutine short_rad)
L02097         subroutine short_rad(iurb,nz_u,id,albw,                     &
L02098                              albg,fwg,fww,fgw,rsg,rsw,pb)
L02099
L02100  ! ------------------------------------------------------------------------
L02101  ! This routine computes the effects of the reflections of short-wave
L02102  ! (solar) radiation in the street canyon by solving the system
L02103  ! of 2*nz_u+1 eqn. in 2*nz_u+1
L02104  ! unkonwn defined in A4, A5 and A6 of the paper (pages 295 and 296).
L02105  ! The system is solved by solving A X= B,
L02106  ! with A matrix B vector, and X solution.
L02107  ! ------------------------------------------------------------------------
L02108
L02109        implicit none
L02110
L02111
L02112
L02113  ! ------------------------------------------------------------------------
L02114  ! INPUT:
L02115  ! ------------------------------------------------------------------------
L02116        real albg                 ! Albedo of the ground for the current urban class
L02117        real albw                 ! Albedo of the wall for the current urban class
L02118        real fgw(nz_um,ndm,nurbm)       ! View factors from ground to wall
L02119        real fwg(nz_um,ndm,nurbm)       ! View factors from wall to ground
L02120        real fww(nz_um,nz_um,ndm,nurbm) ! View factors from wall to wall
L02121        integer id                ! current street direction
L02122        integer iurb              ! current urban class
L02123        integer nz_u              ! Number of layer in the urban grid
L02124        real pb(nz_um)          ! probability to have a building with an height equal
L02125
L02126  ! ------------------------------------------------------------------------
L02127  ! OUTPUT:
L02128  ! ------------------------------------------------------------------------
L02129        real rsg(ndm)             ! Short wave radiation at the ground
L02130        real rsw(2*ndm,nz_um)     ! Short wave radiation at the walls
L02131
L02132  ! ------------------------------------------------------------------------
L02133  ! LOCAL:
L02134  ! ------------------------------------------------------------------------
L02135        integer i,j
L02136        real aaa(2*nz_um+1,2*nz_um+1)  ! terms of the matrix
L02137        real bbb(2*nz_um+1)            ! terms of the vector
L02138
L02139  ! ------------------------------------------------------------------------
L02140  ! END VARIABLES DEFINITIONS
L02141  ! ------------------------------------------------------------------------
L02142
L02143
L02144  ! west wall
L02145
L02146        do i=1,nz_u
L02147           do j=1,nz_u
L02148              aaa(i,j)=0.
```

```
L02149           enddo
L02150
L02151           aaa(i,i)=1.
L02152
L02153           do j=nz_u+1,2*nz_u
L02154              aaa(i,j)=-albw*fww(j-nz_u,i,id,iurb)*pb(j-nz_u+1)
L02155           enddo
L02156
L02157           aaa(i,2*nz_u+1)=-albg*fgw(i,id,iurb)
L02158           bbb(i)=rsw(2*id-1,i)
L02159
L02160        enddo
L02161
L02162  ! east wall
L02163
L02164        do i=1+nz_u,2*nz_u
L02165           do j=1,nz_u
L02166              aaa(i,j)=-albw*fww(j,i-nz_u,id,iurb)*pb(j+1)
L02167           enddo
L02168
L02169           do j=1+nz_u,2*nz_u
L02170              aaa(i,j)=0.
L02171           enddo
L02172
L02173          aaa(i,i)=1.
L02174          aaa(i,2*nz_u+1)=-albg*fgw(i-nz_u,id,iurb)
L02175          bbb(i)=rsw(2*id,i-nz_u)
L02176
L02177        enddo
L02178
L02179  ! ground
L02180
L02181        do j=1,nz_u
L02182           aaa(2*nz_u+1,j)=-albw*fwg(j,id,iurb)*pb(j+1)
L02183        enddo
L02184
L02185        do j=nz_u+1,2*nz_u
L02186           aaa(2*nz_u+1,j)=-albw*fwg(j-nz_u,id,iurb)*pb(j-nz_u+1)
L02187        enddo
L02188
L02189        aaa(2*nz_u+1,2*nz_u+1)=1.
L02190        bbb(2*nz_u+1)=rsg(id)
L02191
L02192        call gaussj(aaa,2*nz_u+1,bbb,2*nz_um+1)
L02193
L02194        do i=1,nz_u
L02195           rsw(2*id-1,i)=bbb(i)
L02196        enddo
L02197
L02198        do i=nz_u+1,2*nz_u
L02199           rsw(2*id,i-nz_u)=bbb(i)
L02200        enddo
L02201
L02202        rsg(id)=bbb(2*nz_u+1)
L02203
L02204        return
L02205        end subroutine short_rad
```

short_rad（非 _dd）：同一线性系统但**无窗户项**（albw 直接用，无 pwin 混合）；long_rad 同理。gaussj 同 NR 实现。

```
!file: phys/module_sf_bep.F    L2211-L2313    (subroutine gaussj)
L02211        subroutine gaussj(a,n,b,np)
L02212
L02213  ! ------------------------------------------------------------------
L02214  ! This routine solve a linear system of n equations of the form
L02215  !              A X = B
L02216  !  where  A is a matrix a(i,j)
L02217  !         B a vector and X the solution
L02218  ! In output b is replaced by the solution
L02219  ! ------------------------------------------------------------------
L02220
L02221        implicit none
L02222
L02223  ! ------------------------------------------------------------------
L02224  ! INPUT:
L02225  ! ------------------------------------------------------------------
L02226        integer np
L02227        real a(np,np)
L02228
L02229  ! ------------------------------------------------------------------
L02230  ! OUTPUT:
L02231  ! ------------------------------------------------------------------
L02232        real b(np)
L02233
L02234  ! ------------------------------------------------------------------
L02235  ! LOCAL:
L02236  ! ------------------------------------------------------------------
L02237        integer nmax
L02238        parameter (nmax=150)
L02239
L02240        real big,dum
L02241        integer i,icol,irow
L02242        integer j,k,l,ll,n
L02243        integer ipiv(nmax)
L02244        real pivinv
L02245
L02246  ! ------------------------------------------------------------------
L02247  ! END VARIABLES DEFINITIONS
L02248  ! ------------------------------------------------------------------
L02249
L02250        do j=1,n
L02251           ipiv(j)=0.
L02252        enddo
L02253
L02254        do i=1,n
L02255           big=0.
```

```
L02256           do j=1,n
L02257              if(ipiv(j).ne.1)then
L02258                 do k=1,n
L02259                    if(ipiv(k).eq.0)then
L02260                       if(abs(a(j,k)).ge.big)then
L02261                          big=abs(a(j,k))
L02262                          irow=j
L02263                          icol=k
L02264                       endif
L02265                    elseif(ipiv(k).gt.1)then
L02266                       FATAL_ERROR('singular matrix in gaussj')
L02267                    endif
L02268                 enddo
L02269              endif
L02270           enddo
L02271
L02272           ipiv(icol)=ipiv(icol)+1
L02273
L02274           if(irow.ne.icol)then
L02275              do l=1,n
L02276                 dum=a(irow,l)
L02277                 a(irow,l)=a(icol,l)
L02278                 a(icol,l)=dum
L02279              enddo
L02280
L02281              dum=b(irow)
L02282              b(irow)=b(icol)
L02283              b(icol)=dum
L02284
L02285           endif
L02286
L02287           if(a(icol,icol).eq.0) FATAL_ERROR('singular matrix in gaussj')
L02288
L02289           pivinv=1./a(icol,icol)
L02290           a(icol,icol)=1
L02291
L02292           do l=1,n
L02293              a(icol,l)=a(icol,l)*pivinv
L02294           enddo
L02295
L02296           b(icol)=b(icol)*pivinv
L02297
L02298           do ll=1,n
L02299              if(ll.ne.icol)then
L02300                 dum=a(ll,icol)
L02301                 a(ll,icol)=0.
L02302                 do l=1,n
L02303                    a(ll,l)=a(ll,l)-a(icol,l)*dum
L02304                 enddo
L02305
L02306                 b(ll)=b(ll)-b(icol)*dum
L02307
L02308              endif
L02309           enddo
L02310        enddo
L02311
L02312        return
L02313        end subroutine gaussj
```

```
!file: phys/module_sf_bep.F    L2318-L2409    (subroutine soil_temp)
L02318        subroutine soil_temp(nz,dz,temp,pt,ala,cs,                    &
L02319                             rs,rl,press,dt,em,alb,rt,sf,gf)
L02320
L02321  ! ------------------------------------------------------------------------
L02322  ! This routine solves the Fourier diffusion equation for heat in
L02323  ! the material (wall, roof, or ground). Resolution is done implicitely.
L02324  ! Boundary conditions are:
L02325  ! - fixed temperature at the interior
L02326  ! - energy budget at the surface
L02327  ! ------------------------------------------------------------------------
L02328
L02329        implicit none
L02330
L02331
L02332
L02333  ! ------------------------------------------------------------------------
L02334  ! INPUT:
L02335  ! ------------------------------------------------------------------------
L02336        integer nz                ! Number of layers
L02337        real ala(nz)              ! Thermal diffusivity in each layers [m^2 s^-1]
L02338        real alb                  ! Albedo of the surface
L02339        real cs(nz)               ! Specific heat of the material [J m^3 K^-1]
L02340        real dt                   ! Time step
L02341        real em                   ! Emissivity of the surface
L02342        real press                ! Pressure at ground level
L02343        real rl                   ! Downward flux of the longwave radiation
L02344        real rs                   ! Solar radiation
L02345        real sf                   ! Sensible heat flux at the surface
L02346        real temp(nz)             ! Temperature in each layer [K]
L02347        real dz(nz)               ! Layer sizes [m]
L02348
L02349
L02350  ! ------------------------------------------------------------------------
L02351  ! OUTPUT:
L02352  ! ------------------------------------------------------------------------
L02353        real gf                   ! Heat flux transferred from the surface toward the interior
L02354        real pt                   ! Potential temperature at the surface
L02355        real rt                   ! Total radiation at the surface (solar+incoming long+outgoing long)
L02356
L02357  ! ------------------------------------------------------------------------
L02358  ! LOCAL:
L02359  ! ------------------------------------------------------------------------
L02360        integer iz
L02361        real a(nz,3)
L02362        real alpha
L02363        real c(nz)
L02364        real cddz(nz+2)
```

```
L02365        real tsig
L02366
L02367  ! ------------------------------------------------------------------
L02368  ! END VARIABLES DEFINITIONS
L02369  ! ------------------------------------------------------------------
L02370
L02371        tsig=temp(nz)
L02372        alpha=(1.-alb)*rs+em*rl-em*sigma*(tsig**4)+sf
L02373  ! Compute cddz=2*cd/dz
L02374
L02375        cddz(1)=ala(1)/dz(1)
L02376        do iz=2,nz
L02377           cddz(iz)=2.*ala(iz)/(dz(iz)+dz(iz-1))
L02378        enddo
L02379  !        cddz(nz+1)=ala(nz+1)/dz(nz)
L02380
L02381        a(1,1)=0.
L02382        a(1,2)=1.
L02383        a(1,3)=0.
L02384        c(1)=temp(1)
L02385
L02386        do iz=2,nz-1
L02387           a(iz,1)=-cddz(iz)*dt/dz(iz)
L02388           a(iz,2)=1+dt*(cddz(iz)+cddz(iz+1))/dz(iz)
L02389           a(iz,3)=-cddz(iz+1)*dt/dz(iz)
L02390           c(iz)=temp(iz)
L02391        enddo
L02392
L02393        a(nz,1)=-dt*cddz(nz)/dz(nz)
L02394        a(nz,2)=1.+dt*cddz(nz)/dz(nz)
L02395        a(nz,3)=0.
L02396        c(nz)=temp(nz)+dt*alpha/cs(nz)/dz(nz)
L02397
L02398
L02399        call invert(nz,a,c,temp)
L02400
L02401
L02402        pt=temp(nz)*(press/1.e+5)**(-rcp_u)
L02403
L02404        rt=(1.-alb)*rs+em*rl-em*sigma*(tsig**4)
L02405
L02406  !      gf=-cddz(nz)*(temp(nz)-temp(nz-1))*cs(nz)
L02407         gf=(1.-alb)*rs+em*rl-em*sigma*(tsig**4)+sf
L02408        return
L02409        end subroutine soil_temp
```

```
!file: phys/module_sf_bep.F    L2414-L2460    (subroutine invert)
L02414        subroutine invert(n,a,c,x)
L02415
L02416  ! ------------------------------------------------------------------
L02417  !        Inversion and resolution of a tridiagonal matrix
L02418  !                    A X = C
L02419  ! ------------------------------------------------------------------
L02420
L02421        implicit none
L02422
L02423  ! ------------------------------------------------------------------
L02424  ! INPUT:
L02425  ! ------------------------------------------------------------------
L02426         integer n
L02427         real a(n,3)              !  a(*,1) lower diagonal (Ai,i-1)
L02428                                  !  a(*,2) principal diagonal (Ai,i)
L02429                                  !  a(*,3) upper diagonal (Ai,i+1)
L02430         real c(n)
L02431
L02432  ! ------------------------------------------------------------------
L02433  ! OUTPUT:
L02434  ! ------------------------------------------------------------------
L02435         real x(n)
L02436
L02437  ! ------------------------------------------------------------------
L02438  ! LOCAL:
L02439  ! ------------------------------------------------------------------
L02440         integer i
L02441
L02442  ! ------------------------------------------------------------------
L02443  ! END VARIABLES DEFINITIONS
L02444  ! ------------------------------------------------------------------
L02445
L02446         do i=n-1,1,-1
L02447            c(i)=c(i)-a(i,3)*c(i+1)/a(i+1,2)
L02448            a(i,2)=a(i,2)-a(i,3)*a(i+1,1)/a(i+1,2)
L02449         enddo
L02450
L02451         do i=2,n
L02452            c(i)=c(i)-a(i,1)*c(i-1)/a(i-1,2)
L02453         enddo
L02454
L02455         do i=1,n
L02456            x(i)=c(i)/a(i,2)
L02457         enddo
L02458
L02459         return
L02460         end subroutine invert
```

soil_temp/invert 与 14b § 四逐行同源（三处厚度数组一致性警示同样适用：本文件 surf_temp 内的 dzg_u/dzr/dzw 与 param 展开值必须一致）。

```
!file: phys/module_sf_bep.F    L2466-L2538    (subroutine flux_wall)
L02466        subroutine flux_wall(ua,va,pt,da,ptw,uva,vva,uvb,vvb,            &
L02467                             tva,tvb,evb,drst,dt)
L02468
L02469  ! ------------------------------------------------------------------
L02470  ! This routine computes the surface sources or sinks of momentum, tke,
```

```
L02471  ! and heat from vertical surfaces (walls).
L02472  ! ----------------------------------------------------------------------
L02473
L02474        implicit none
L02475
L02476
L02477
L02478  ! INPUT:
L02479  ! -----
L02480        real drst                 ! street directions for the current urban class
L02481        real da                   ! air density
L02482        real pt                   ! potential temperature
L02483        real ptw                  ! Walls potential temperatures
L02484        real ua                   ! wind speed
L02485        real va                   ! wind speed
L02486
L02487        real dt                   !time step
L02488  ! OUTPUT:
L02489  ! ------
L02490  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L02491  ! vertical surfaces (walls).
L02492  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L02493  !  Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L02494        real uva                  ! U (wind component)   Vertical surfaces, A (implicit) term
L02495        real uvb                  ! U (wind component)   Vertical surfaces, B (explicit) term
L02496        real vva                  ! V (wind component)   Vertical surfaces, A (implicit) term
L02497        real vvb                  ! V (wind component)   Vertical surfaces, B (explicit) term
L02498        real tva                  ! Temperature          Vertical surfaces, A (implicit) term
L02499        real tvb                  ! Temperature          Vertical surfaces, B (explicit) term
L02500        real evb                  ! Energy (TKE)         Vertical surfaces, B (explicit) term
L02501
L02502  ! LOCAL:
L02503  ! -----
L02504        real hc
L02505        real u_ort
L02506        real vett
L02507
L02508  ! -------------------------
L02509  ! END VARIABLES DEFINITIONS
L02510  ! -------------------------
L02511
L02512        vett=(ua**2+va**2)**.5
L02513
L02514        u_ort=abs((cos(drst)*ua-sin(drst)*va))
L02515
L02516        uva=-cdrag*u_ort/2.*cos(drst)*cos(drst)
L02517        vva=-cdrag*u_ort/2.*sin(drst)*sin(drst)
L02518
L02519        uvb=cdrag*u_ort/2.*sin(drst)*cos(drst)*va
L02520        vvb=cdrag*u_ort/2.*sin(drst)*cos(drst)*ua
L02521
L02522        hc=5.678*(1.09+0.23*(vett/0.3048))
L02523
L02524        if(hc.gt.da*cp_u/dt)then
L02525          hc=da*cp_u/dt
L02526        endif
L02527
L02528  !         tvb=hc*ptw/da/cp_u
L02529  !         tva=-hc/da/cp_u
L02530  !!!!!!!!!!!!!!!!!!!
L02531  ! explicit
L02532        tvb=hc*ptw/da/cp_u-hc/da/cp_u*pt !c
L02533        tva = 0.                  !c
L02534
L02535        evb=cdrag*(abs(u_ort)**3.)/2.
L02536
L02537        return
L02538        end subroutine flux_wall
```

flux_wall 差异：cdrag 来自**模块常数 0.4**（无 NEW CDRAG 调用链）；无窗户 hcwin 分支（无 sfwin 输出）。其余 hc 分段式与 TKE 源 evb 同 14b § 五。

```
!file: phys/module_sf_bep.F    L2544-L2663    (subroutine flux_flat)
L02544        subroutine flux_flat(dz,z0,ua,va,pt,pt0,ptg,                    &
L02545                             uhb,vhb,thb,ehb)
L02546
L02547  ! ----------------------------------------------------------------------
L02548  !           Calculation of the flux at the ground
L02549  !           Formulation of Louis (Louis, 1979)
L02550  ! ----------------------------------------------------------------------
L02551
L02552        implicit none
L02553
L02554
L02555
L02556  ! ----------------------------------------------------------------------
L02557  ! INPUT:
L02558  ! ----------------------------------------------------------------------
L02559        real dz                   ! first vertical level
L02560        real pt                   ! potential temperature
L02561        real pt0                  ! reference potential temperature
L02562        real ptg                  ! ground potential temperature
L02563        real ua                   ! wind speed
L02564        real va                   ! wind speed
L02565        real z0                   ! Roughness length
L02566
L02567  ! ----------------------------------------------------------------------
L02568  ! OUTPUT:
L02569  ! ----------------------------------------------------------------------
L02570  ! Explicit component of the momentum, temperature and TKE sources or sinks on horizontal
L02571  !  surfaces (roofs and street)
L02572  ! The fluxes can be computed as follow: Fluxes of X = B
L02573  !  Example: Momentum fluxes on horizontal surfaces =  uhb_u
L02574        real uhb                  ! U (wind component) Horizontal surfaces, B (explicit) term
L02575        real vhb                  ! V (wind component) Horizontal surfaces, B (explicit) term
L02576        real thb                  ! Temperature        Horizontal surfaces, B (explicit) term
```

```
L02577        real tva                  ! Temperature          Vertical surfaces, A (implicit) term
L02578        real tvb                  ! Temperature          Vertical surfaces, B (explicit) term
L02579        real ehb                  ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L02580
L02581
L02582  ! ------------------------------------------------------------------
L02583  ! LOCAL:
L02584  ! ------------------------------------------------------------------
L02585        real aa
L02586        real al
L02587        real buu
L02588        real c
L02589        real fbuw
L02590        real fbpt
L02591        real fh
L02592        real fm
L02593        real ric
L02594        real tstar
L02595        real ustar
L02596        real utot
L02597        real wstar
L02598        real zz
L02599
L02600        real b,cm,ch,rr,tol
L02601        parameter(b=9.4,cm=7.4,ch=5.3,rr=0.74,tol=.001)
L02602
L02603  ! ------------------------------------------------------------------
L02604  ! END VARIABLES DEFINITIONS
L02605  ! ------------------------------------------------------------------
L02606
L02607
L02608  ! computation of the ground temperature
L02609
L02610        utot=(ua**2+va**2)**.5
L02611
L02612
L02613  !!!! Louis formulation
L02614  !
L02615  ! compute the bulk Richardson Number
L02616
L02617        zz=dz/2.
L02618
L02619  !        if(tstar.lt.0.)then
L02620  !         wstar=(-ustar*tstar*g*hii/pt)**(1./3.)
L02621  !        else
L02622  !         wstar=0.
L02623  !        endif
L02624  !
L02625  !      if (utot.le.0.7*wstar) utot=max(0.7*wstar,0.00001)
L02626
L02627        utot=max(utot,0.01)
L02628
L02629        ric=2.*g_u*zz*(pt-ptg)/((pt+ptg)*(utot**2))
L02630
L02631        aa=vk/log(zz/z0)
L02632
L02633  ! determine the parameters fm and fh for stable, neutral and unstable conditions
L02634
L02635        if(ric.gt.0)then
L02636           fm=1/(1+0.5*b*ric)**2
L02637           fh=fm
L02638        else
L02639           c=b*cm*aa*aa*(zz/z0)**.5
L02640           fm=1-b*ric/(1+c*(-ric)**.5)
L02641           c=c*ch/cm
L02642           fh=1-b*ric/(1+c*(-ric)**.5)
L02643        endif
L02644
L02645        fbuw=-aa*aa*utot*utot*fm
L02646        fbpt=-aa*aa*utot*(pt-ptg)*fh/rr
L02647
L02648        ustar=(-fbuw)**.5
L02649        tstar=-fbpt/ustar
L02650
L02651        al=(vk*g_u*tstar)/(pt*ustar*ustar)
L02652
L02653        buu=-g_u/pt0*ustar*tstar
L02654
L02655        uhb=-ustar*ustar*ua/utot
L02656        vhb=-ustar*ustar*va/utot
L02657        thb=-ustar*tstar
L02658  !      thb= 0.
L02659        ehb=buu
L02660  !!!!!!!!!!!!!!!
L02661
L02662        return
L02663        end subroutine flux_flat
```

与 14b 版同源（Louis 1979 全套常数 b=9.4/cm=7.4/ch=5.3/rr=0.74）。

```
!file: phys/module_sf_bep.F    L1372-L1481    (subroutine buildings)
L01372        subroutine buildings(nd,nz,z0,ua_u,va_u,pt_u,pt0_u,         &
L01373                             ptg,ptr,da_u,ptw,                        &
L01374                             drst,uva_u,vva_u,uvb_u,vvb_u,            &
L01375                             tva_u,tvb_u,evb_u,                       &
L01376                             uhb_u,vhb_u,thb_u,ehb_u,ss,dt)
L01377
L01378  ! ------------------------------------------------------------------
L01379  ! This routine computes the sources or sinks of the different quantities
L01380  ! on the urban grid. The actual calculation is done in the subroutines
L01381  ! called flux_wall, and flux_flat.
L01382  ! ------------------------------------------------------------------
L01383
L01384        implicit none
L01385
L01386
L01387  ! ------------------------------------------------------------------
L01388  ! INPUT:
```

```
L01389  ! ------------------------------------------------------------------
L01390        integer nd                ! Number of street direction for the current urban class
L01391        integer nz                ! number of vertical space steps
L01392        real ua_u(nz_um)          ! Wind speed in the x direction on the urban grid
L01393        real va_u(nz_um)          ! Wind speed in the y direction on the urban grid
L01394        real da_u(nz_um)          ! air density on the urban grid
L01395        real drst(ndm)            ! Street directions for the current urban class
L01396        real dz
L01397        real pt_u(nz_um)          ! Potential temperature on the urban grid
L01398        real pt0_u(nz_um)         ! reference potential temperature on the urban grid
L01399        real ptg(ndm)             ! Ground potential temperatures
L01400        real ptr(ndm,nz_um)     ! Roof potential temperatures
L01401        real ptw(2*ndm,nz_um)     ! Walls potential temperatures
L01402        real ss(nz_um)          ! probability to have a building with height h
L01403        real z0(ndm,nz_um)      ! Roughness lengths "profiles"
L01404        real dt ! time step
L01405
L01406
L01407  ! ------------------------------------------------------------------
L01408  ! OUTPUT:
L01409  ! ------------------------------------------------------------------
L01410  ! Explicit and implicit component of the momentum, temperature and TKE sources or sinks on
L01411  ! vertical surfaces (walls) and horizontal surfaces (roofs and street)
L01412  ! The fluxes can be computed as follow: Fluxes of X = A*X + B
L01413  !  Example: Momentum fluxes on vertical surfaces = uva_u * ua_u + uvb_u
L01414
L01415        real uhb_u(ndm,nz_um)   ! U (wind component) Horizontal surfaces, B (explicit) term
L01416        real uva_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, A (implicit) term
L01417        real uvb_u(2*ndm,nz_um)   ! U (wind component)   Vertical surfaces, B (explicit) term
L01418        real vhb_u(ndm,nz_um)   ! V (wind component) Horizontal surfaces, B (explicit) term
L01419        real vva_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, A (implicit) term
L01420        real vvb_u(2*ndm,nz_um)   ! V (wind component)   Vertical surfaces, B (explicit) term
L01421        real thb_u(ndm,nz_um)   ! Temperature        Horizontal surfaces, B (explicit) term
L01422        real tva_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, A (implicit) term
L01423        real tvb_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, B (explicit) term
L01424        real ehb_u(ndm,nz_um)   ! Energy (TKE)       Horizontal surfaces, B (explicit) term
L01425        real evb_u(2*ndm,nz_um)   ! Energy (TKE)         Vertical surfaces, B (explicit) term
L01426
L01427  ! ------------------------------------------------------------------
L01428  ! LOCAL:
L01429  ! ------------------------------------------------------------------
L01430        integer id,iz
L01431
L01432  ! ------------------------------------------------------------------
L01433  ! END VARIABLES DEFINITIONS
L01434  ! ------------------------------------------------------------------
L01435         dz=dz_u
L01436
L01437        do id=1,nd
L01438
L01439  !        Calculation at the ground surfaces
L01440           call flux_flat(dz,z0(id,1),ua_u(1),va_u(1),pt_u(1),pt0_u(1),  &
L01441                          ptg(id),uhb_u(id,1),                            &
L01442                          vhb_u(id,1),thb_u(id,1),ehb_u(id,1))
L01443
L01444  !        Calculation at the roof surfaces
L01445           do iz=2,nz
L01446              if(ss(iz).gt.0)then
L01447                 call flux_flat(dz,z0(id,iz),ua_u(iz),                  &
L01448                         va_u(iz),pt_u(iz),pt0_u(iz),                   &
L01449                         ptr(id,iz),uhb_u(id,iz),                       &
L01450                         vhb_u(id,iz),thb_u(id,iz),ehb_u(id,iz))
L01451              else
L01452                 uhb_u(id,iz) = 0.0
L01453                 vhb_u(id,iz) = 0.0
L01454                 thb_u(id,iz) = 0.0
L01455                 ehb_u(id,iz) = 0.0
L01456              endif
L01457           end do
L01458
L01459  !        Calculation at the wall surfaces
L01460           do iz=1,nz
L01461              call flux_wall(ua_u(iz),va_u(iz),pt_u(iz),da_u(iz),     &
L01462                         ptw(2*id-1,iz),                              &
L01463                         uva_u(2*id-1,iz),vva_u(2*id-1,iz),           &
L01464                         uvb_u(2*id-1,iz),vvb_u(2*id-1,iz),           &
L01465                         tva_u(2*id-1,iz),tvb_u(2*id-1,iz),           &
L01466                         evb_u(2*id-1,iz),drst(id),dt)
L01467
L01468              call flux_wall(ua_u(iz),va_u(iz),pt_u(iz),da_u(iz),    &
L01469                         ptw(2*id,iz),                                &
L01470                         uva_u(2*id,iz),vva_u(2*id,iz),               &
L01471                         uvb_u(2*id,iz),vvb_u(2*id,iz),               &
L01472                         tva_u(2*id,iz),tvb_u(2*id,iz),               &
L01473                         evb_u(2*id,iz),drst(id),dt)
L01474  !
L01475
L01476           end do
L01477
L01478        end do
L01479
L01480        return
L01481        end subroutine buildings
```

与 14a § 九差异：无积水缩放（wfg/wfr）、无绿屋顶/PV 混合、无 AC/通风源项（tvb_ac 不存在）、动量/温度/TKE 三类源项结构相同但输出少 qhb/qvb。

```
!file: phys/module_sf_bep.F    L1487-L1683    (subroutine urban_meso)
L01487          subroutine urban_meso(nd,kms,kme,kts,kte,nz_u,z,dz,z_u,pb,ss,bs,ws,sf,vl,    &
L01488                                uva_u,vva_u,uvb_u,vvb_u,tva_u,tvb_u,evb_u, &
L01489                                uhb_u,vhb_u,thb_u,ehb_u,                   &
L01490                                a_u,a_v,a_t,a_e,b_u,b_v,b_t,b_e)
L01491
L01492  ! ------------------------------------------------------------------
L01493  !  This routine interpolates the parameters from the "urban grid" to the
```

```
!  "mesoscale grid".
!  See p300-301 Appendix B.2 of the BLM paper.
! ------------------------------------------------------------------------

      implicit none

! ------------------------------------------------------------------------
! INPUT:
! ------------------------------------------------------------------------
! Data relative to the "mesoscale grid"
      integer kms,kme,kts,kte
      real z(kms:kme)              ! Altitude above the ground of the cell interface
      real dz(kms:kme)               ! Vertical space steps

! Data relative to the "uban grid"
      integer nz_u              ! Number of layer in the urban grid
      integer nd                ! Number of street direction for the current urban class
      real bs(ndm)              ! Building widths of the current urban class
      real ws(ndm)              ! Street widths of the current urban class
      real z_u(nz_um)         ! Height of the urban grid levels
      real pb(nz_um)          ! Probability to have a building with an height equal
      real ss(nz_um)          ! Probability to have a building with height h
      real uhb_u(ndm,nz_um)   ! U (x-wind component) Horizontal surfaces, B (explicit) term
      real uva_u(2*ndm,nz_um)   ! U (x-wind component) Vertical surfaces, A (implicit) term
      real uvb_u(2*ndm,nz_um)   ! U (x-wind component) Vertical surfaces, B (explicit) term
      real vhb_u(ndm,nz_um)   ! V (y-wind component) Horizontal surfaces, B (explicit) term
      real vva_u(2*ndm,nz_um)   ! V (y-wind component) Vertical surfaces, A (implicit) term
      real vvb_u(2*ndm,nz_um)   ! V (y-wind component) Vertical surfaces, B (explicit) term
      real thb_u(ndm,nz_um)   ! Temperature        Horizontal surfaces, B (explicit) term
      real tva_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, A (implicit) term
      real tvb_u(2*ndm,nz_um)   ! Temperature          Vertical surfaces, B (explicit) term
      real ehb_u(ndm,nz_um)   ! Energy (TKE)       Horizontal surfaces, B (explicit) term
      real evb_u(2*ndm,nz_um)   ! Energy (TKE)         Vertical surfaces, B (explicit) term

! ------------------------------------------------------------------------
! OUTPUT:
! ------------------------------------------------------------------------
! Data relative to the "mesoscale grid"
      real sf(kms:kme)             ! Surface of the "mesoscale grid" cells taking into account the buildings
      real vl(kms:kme)               ! Volume of the "mesoscale grid" cells taking into account the buildings
      real a_u(kms:kme)              ! Implicit component of the momentum sources or sinks in the X-direction
      real a_v(kms:kme)              ! Implicit component of the momentum sources or sinks in the Y-direction
      real a_t(kms:kme)              ! Implicit component of the heat sources or sinks
      real a_e(kms:kme)              ! Implicit component of the TKE sources or sinks
      real b_u(kms:kme)              ! Explicit component of the momentum sources or sinks in the X-direction
      real b_v(kms:kme)              ! Explicit component of the momentum sources or sinks in the Y-direction
      real b_t(kms:kme)              ! Explicit component of the heat sources or sinks
      real b_e(kms:kme)              ! Explicit component of the TKE sources or sinks

! ------------------------------------------------------------------------
! LOCAL:
! ------------------------------------------------------------------------
      real dzz
      real fact
      integer id,iz,iz_u
      real se,sr,st,su,sv
      real uet(kms:kme)                ! Contribution to TKE due to walls
      real veb,vta,vtb,vte,vtot,vua,vub,vva,vvb


! ------------------------------------------------------------------------
! END VARIABLES DEFINITIONS
! ------------------------------------------------------------------------

! initialisation

      do iz=kts,kte
         a_u(iz)=0.
         a_v(iz)=0.
         a_t(iz)=0.
         a_e(iz)=0.
         b_u(iz)=0.
         b_v(iz)=0.
         b_e(iz)=0.
         b_t(iz)=0.
         uet(iz)=0.
      end do

! horizontal surfaces
      do iz=kts,kte
         sf(iz)=0.
         vl(iz)=0.
      enddo
      sf(kte+1)=0.

      do id=1,nd
         do iz=kts+1,kte+1
            sr=0.
            do iz_u=2,nz_u
               if(z(iz).lt.z_u(iz_u).and.z(iz).ge.z_u(iz_u-1))then
                  sr=pb(iz_u)
               endif
            enddo
            sf(iz)=sf(iz)+((ws(id)+(1.-sr)*bs(id))/(ws(id)+bs(id)))/nd
         enddo
      enddo

! volume
      do iz=kts,kte
         do id=1,nd
            vtot=0.
            do iz_u=1,nz_u
               dzz=max(min(z_u(iz_u+1),z(iz+1))-max(z_u(iz_u),z(iz)),0.)
               vtot=vtot+pb(iz_u+1)*dzz
            enddo
            vtot=vtot/(z(iz+1)-z(iz))
            vl(iz)=vl(iz)+(1.-vtot*bs(id)/(ws(id)+bs(id)))/nd
         enddo
```

```
L01602        enddo
L01603
L01604  ! horizontal surface impact
L01605
L01606        do id=1,nd
L01607
L01608           fact=1./vl(kts)/dz(kts)*ws(id)/(ws(id)+bs(id))/nd
L01609           b_t(kts)=b_t(kts)+thb_u(id,1)*fact
L01610           b_u(kts)=b_u(kts)+uhb_u(id,1)*fact
L01611           b_v(kts)=b_v(kts)+vhb_u(id,1)*fact
L01612           b_e(kts)=b_e(kts)+ehb_u(id,1)*fact*(z_u(2)-z_u(1))
L01613
L01614           do iz=kts,kte
L01615              st=0.
L01616              su=0.
L01617              sv=0.
L01618              se=0.
L01619              do iz_u=2,nz_u
L01620                 if(z(iz).le.z_u(iz_u).and.z(iz+1).gt.z_u(iz_u))then
L01621                    st=st+ss(iz_u)*thb_u(id,iz_u)
L01622                    su=su+ss(iz_u)*uhb_u(id,iz_u)
L01623                    sv=sv+ss(iz_u)*vhb_u(id,iz_u)
L01624                    se=se+ss(iz_u)*ehb_u(id,iz_u)*(z_u(iz_u+1)-z_u(iz_u))
L01625                 endif
L01626              enddo
L01627
L01628              fact=bs(id)/(ws(id)+bs(id))/vl(iz)/dz(iz)/nd
L01629              b_t(iz)=b_t(iz)+st*fact
L01630              b_u(iz)=b_u(iz)+su*fact
L01631              b_v(iz)=b_v(iz)+sv*fact
L01632              b_e(iz)=b_e(iz)+se*fact
L01633           enddo
L01634        enddo
L01635
L01636  ! vertical surface impact
L01637
L01638        do iz=kts,kte
L01639           uet(iz)=0.
L01640           do id=1,nd
L01641              vtb=0.
L01642              vta=0.
L01643              vua=0.
L01644              vub=0.
L01645              vva=0.
L01646              vvb=0.
L01647              veb=0.
L01648            vte=0.
L01649              do iz_u=1,nz_u
L01650                 dzz=max(min(z_u(iz_u+1),z(iz+1))-max(z_u(iz_u),z(iz)),0.)
L01651                 fact=dzz/(ws(id)+bs(id))
L01652                 vtb=vtb+pb(iz_u+1)*                                    &
L01653                      (tvb_u(2*id-1,iz_u)+tvb_u(2*id,iz_u))*fact
L01654                 vta=vta+pb(iz_u+1)*                                    &
L01655                     (tva_u(2*id-1,iz_u)+tva_u(2*id,iz_u))*fact
L01656                 vua=vua+pb(iz_u+1)*                                    &
L01657                      (uva_u(2*id-1,iz_u)+uva_u(2*id,iz_u))*fact
L01658                 vva=vva+pb(iz_u+1)*                                    &
L01659                      (vva_u(2*id-1,iz_u)+vva_u(2*id,iz_u))*fact
L01660                 vub=vub+pb(iz_u+1)*                                    &
L01661                      (uvb_u(2*id-1,iz_u)+uvb_u(2*id,iz_u))*fact
L01662                 vvb=vvb+pb(iz_u+1)*                                    &
L01663                      (vvb_u(2*id-1,iz_u)+vvb_u(2*id,iz_u))*fact
L01664                 veb=veb+pb(iz_u+1)*                                    &
L01665                      (evb_u(2*id-1,iz_u)+evb_u(2*id,iz_u))*fact
L01666              enddo
L01667
L01668              fact=1./vl(iz)/dz(iz)/nd
L01669              b_t(iz)=b_t(iz)+vtb*fact
L01670              a_t(iz)=a_t(iz)+vta*fact
L01671              a_u(iz)=a_u(iz)+vua*fact
L01672              a_v(iz)=a_v(iz)+vva*fact
L01673              b_u(iz)=b_u(iz)+vub*fact
L01674              b_v(iz)=b_v(iz)+vvb*fact
L01675              b_e(iz)=b_e(iz)+veb*fact
L01676              uet(iz)=uet(iz)+vte*fact
L01677           enddo
L01678        enddo
L01679
L01680
L01681
L01682        return
L01683        end subroutine urban_meso
```

```
!file: phys/module_sf_bep.F    L1689-L1769    (subroutine interp_length)
L01689        subroutine interp_length(nd,kms,kme,kts,kte,nz_u,z_u,z,ss,ws,bs,              &
L01690                                 dlg,dl_u)
L01691
L01692  ! ------------------------------------------------------------------
L01693  !    Calculation of the length scales
L01694  !    See p272-274 formula (22) and (24) of the BLM paper
L01695  ! ------------------------------------------------------------------
L01696
L01697        implicit none
L01698
L01699
L01700  ! ------------------------------------------------------------------
L01701  ! INPUT:
L01702  ! ------------------------------------------------------------------
L01703        integer kms,kme,kts,kte
L01704        real z(kms:kme)              ! Altitude above the ground of the cell interface
L01705        integer nd                 ! Number of street direction for the current urban class
L01706        integer nz_u               ! Number of levels in the "urban grid"
L01707        real z_u(nz_um)          ! Height of the urban grid levels
L01708        real bs(ndm)               ! Building widths of the current urban class
L01709        real ss(nz_um)           ! Probability to have a building with height h
L01710        real ws(ndm)               ! Street widths of the current urban class
L01711
```

```
L01712
L01713  ! ------------------------------------------------------------------
L01714  ! OUTPUT:
L01715  ! ------------------------------------------------------------------
L01716        real dlg(kms:kme)             ! Height above ground (L_ground in formula (24) of the BLM paper).
L01717        real dl_u(kms:kme)            ! Length scale (lb in formula (22) ofthe BLM paper).
L01718
L01719  ! ------------------------------------------------------------------
L01720  ! LOCAL:
L01721  ! ------------------------------------------------------------------
L01722        real dlgtmp
L01723        integer id,iz,iz_u
L01724        real sftot
L01725        real ulu,ssl
L01726
L01727  ! ------------------------------------------------------------------
L01728  ! END VARIABLES DEFINITIONS
L01729  ! ------------------------------------------------------------------
L01730
L01731          do iz=kts,kte
L01732           ulu=0.
L01733           ssl=0.
L01734           do id=1,nd
L01735            do iz_u=2,nz_u
L01736             if(z_u(iz_u).gt.z(iz))then
L01737              ulu=ulu+ss(iz_u)/z_u(iz_u)/nd
L01738              ssl=ssl+ss(iz_u)/nd
L01739             endif
L01740            enddo
L01741           enddo
L01742
L01743          if(ulu.ne.0)then
L01744            dl_u(iz)=ssl/ulu
L01745           else
L01746            dl_u(iz)=0.
L01747           endif
L01748          enddo
L01749
L01750
L01751          do iz=kts,kte
L01752           dlg(iz)=0.
L01753            do id=1,nd
L01754             sftot=ws(id)
L01755             dlgtmp=ws(id)/((z(iz)+z(iz+1))/2.)
L01756             do iz_u=1,nz_u
L01757              if((z(iz)+z(iz+1))/2..gt.z_u(iz_u))then
L01758               dlgtmp=dlgtmp+ss(iz_u)*bs(id)/                          &
L01759                      ((z(iz)+z(iz+1))/2.-z_u(iz_u))
L01760              sftot=sftot+ss(iz_u)*bs(id)
L01761              endif
L01762             enddo
L01763             dlg(iz)=dlg(iz)+dlgtmp/sftot/nd
L01764            enddo
L01765            dlg(iz)=1./dlg(iz)
L01766           enddo
L01767
L01768          return
L01769          end subroutine interp_length
```

与 14b § 二同源；urban_meso 少 tvb_ac/b_ac 分账。

```
!file: phys/module_sf_bep.F    L2668-L2770    (subroutine icBEP)
L02668        subroutine icBEP (nd_u,h_b,d_b,ss_u,pb_u,nz_u,z_u)
L02669
L02670        implicit none
L02671
L02672
L02673  !    Street parameters
L02674        integer nd_u(nurbm)     ! Number of street direction for each urban class
L02675        real h_b(nz_um,nurbm)   ! Bulding's heights [m]
L02676        real d_b(nz_um,nurbm)   ! The probability that a building has an height h_b
L02677  ! ------------------------------------------------------------------
L02678  !     Output
L02679  !-------------------------------------------------------------------
L02680
L02681        real ss_u(nz_um,nurbm)     ! The probability that a building has an height equal to z
L02682        real pb_u(nz_um,nurbm)     ! The probability that a building has an height greater or equal to z
L02683
L02684  !    Grid parameters
L02685        integer nz_u(nurbm)     ! Number of layer in the urban grid
L02686        real z_u(nz_um)       ! Height of the urban grid levels
L02687
L02688
L02689  ! ------------------------------------------------------------------
L02690  !     Local
L02691  !-------------------------------------------------------------------
L02692
L02693        integer iz_u,id,ilu,iurb
L02694
L02695        real dtot
L02696        real hbmax
L02697
L02698  !-------------------------------------------------------------------
L02699
L02700
L02701  ! ------------------------------------------------------------------
L02702  !     This routine initialise the urban paramters for the BEP module
L02703  !-------------------------------------------------------------------
L02704  !
L02705  !Initialize variables
L02706  !
L02707        z_u=0.
L02708        nz_u=0
L02709        ss_u=0.
L02710        pb_u=0.
L02711
L02712  ! Computation of the urban levels height
L02713
```

```
L02714        z_u(1)=0.
L02715
L02716        do iz_u=1,nz_um-1
L02717           z_u(iz_u+1)=z_u(iz_u)+dz_u
L02718        enddo
L02719
L02720  ! Normalisation of the building density
L02721
L02722        do iurb=1,nurbm
L02723           dtot=0.
L02724           do ilu=1,nz_um
L02725              dtot=dtot+d_b(ilu,iurb)
L02726           enddo
L02727           do ilu=1,nz_um
L02728              d_b(ilu,iurb)=d_b(ilu,iurb)/dtot
L02729           enddo
L02730        enddo
L02731
L02732  ! Compute the view factors, pb and ss
L02733
L02734        do iurb=1,nurbm
L02735           hbmax=0.
L02736           nz_u(iurb)=0
L02737           do ilu=1,nz_um
L02738              if(h_b(ilu,iurb).gt.hbmax)hbmax=h_b(ilu,iurb)
L02739           enddo
L02740
L02741           do iz_u=1,nz_um-1
L02742              if(z_u(iz_u+1).gt.hbmax)go to 10
L02743           enddo
L02744
L02745   10      continue
L02746           nz_u(iurb)=iz_u+1
L02747
L02748           do id=1,nd_u(iurb)
L02749
L02750              do iz_u=1,nz_u(iurb)
L02751                 ss_u(iz_u,iurb)=0.
L02752                 do ilu=1,nz_um
L02753                    if(z_u(iz_u).le.h_b(ilu,iurb)                    &
L02754                      .and.z_u(iz_u+1).gt.h_b(ilu,iurb))then
L02755                         ss_u(iz_u,iurb)=ss_u(iz_u,iurb)+d_b(ilu,iurb)
L02756                    endif
L02757                 enddo
L02758              enddo
L02759
L02760              pb_u(1,iurb)=1.
L02761              do iz_u=1,nz_u(iurb)
L02762                 pb_u(iz_u+1,iurb)=max(0.,pb_u(iz_u,iurb)-ss_u(iz_u,iurb))
L02763              enddo
L02764
L02765           enddo
L02766        end do
L02767
L02768
L02769        return
L02770        end subroutine icBEP
```

```
!file: phys/module_sf_bep.F    L2775-L2934    (subroutine view_factors)
L02775        subroutine view_factors(iurb,nz_u,id,dxy,z,ws,fww,fwg,fgw,fsg,fsw,fws)
L02776
L02777        implicit none
L02778
L02779
L02780
L02781  ! ------------------------------------------------------------------------
L02782  !     Input
L02783  !------------------------------------------------------------------------
L02784
L02785        integer iurb            ! Number of the urban class
L02786        integer nz_u            ! Number of levels in the urban grid
L02787        integer id              ! Street direction number
L02788        real ws                 ! Street width
L02789        real z(nz_um)         ! Height of the urban grid levels
L02790        real dxy                ! Street lenght
L02791
L02792
L02793  ! ------------------------------------------------------------------------
L02794  !     Output
L02795  !------------------------------------------------------------------------
L02796
L02797  !   fww,fwg,fgw,fsw,fsg are the view factors used to compute the long wave
L02798  !   and the short wave radation. They are the part of radiation from a surface
L02799  !   or from the sky to another surface.
L02800
L02801        real fww(nz_um,nz_um,ndm,nurbm)           !  from wall to wall
L02802        real fwg(nz_um,ndm,nurbm)                 !  from wall to ground
L02803        real fgw(nz_um,ndm,nurbm)                 !  from ground to wall
L02804        real fsw(nz_um,ndm,nurbm)                 !  from sky to wall
L02805        real fws(nz_um,ndm,nurbm)                 !  from wall to sky
L02806        real fsg(ndm,nurbm)                       !  from sky to ground
L02807
L02808
L02809  ! ------------------------------------------------------------------------
L02810  !     Local
L02811  !------------------------------------------------------------------------
L02812
L02813        integer jz,iz
L02814
L02815        real hut
L02816        real f1,f2,f12,f23,f123,ftot
L02817        real fprl,fnrm
L02818        real a1,a2,a3,a4,a12,a23,a123
L02819
L02820  ! ------------------------------------------------------------------------
L02821  !     This routine calculates the view factors
L02822  !------------------------------------------------------------------------
L02823
```

```
L02824        hut=z(nz_u+1)
L02825
L02826        do jz=1,nz_u
L02827
L02828  ! radiation from wall to wall
L02829
L02830           do iz=1,nz_u
L02831
L02832              call fprls (fprl,dxy,abs(z(jz+1)-z(iz  )),ws)
L02833              f123=fprl
L02834              call fprls (fprl,dxy,abs(z(jz+1)-z(iz+1)),ws)
L02835              f23=fprl
L02836              call fprls (fprl,dxy,abs(z(jz  )-z(iz  )),ws)
L02837              f12=fprl
L02838              call fprls (fprl,dxy,abs(z(jz  )-z(iz+1)),ws)
L02839              f2 = fprl
L02840
L02841              a123=dxy*(abs(z(jz+1)-z(iz  )))
L02842              a12 =dxy*(abs(z(jz  )-z(iz  )))
L02843              a23 =dxy*(abs(z(jz+1)-z(iz+1)))
L02844              a1  =dxy*(abs(z(iz+1)-z(iz  )))
L02845              a2  =dxy*(abs(z(jz  )-z(iz+1)))
L02846              a3  =dxy*(abs(z(jz+1)-z(jz  )))
L02847
L02848              ftot=0.5*(a123*f123-a23*f23-a12*f12+a2*f2)/a1
L02849
L02850              fww(iz,jz,id,iurb)=ftot*a1/a3
L02851
L02852           enddo
L02853
L02854  ! radiation from ground to wall
L02855
L02856           call fnrms (fnrm,z(jz+1),dxy,ws)
L02857           f12=fnrm
L02858           call fnrms (fnrm,z(jz)  ,dxy,ws)
L02859           f1=fnrm
L02860
L02861           a1 = ws*dxy
L02862
L02863           a12= ws*dxy
L02864
L02865           a4=(z(jz+1)-z(jz))*dxy
L02866
L02867           ftot=(a12*f12-a12*f1)/a1
L02868
L02869           fgw(jz,id,iurb)=ftot*a1/a4
L02870
L02871  !  radiation from sky to wall
L02872
L02873           call fnrms(fnrm,hut-z(jz)  ,dxy,ws)
L02874           f12 = fnrm
L02875           call fnrms (fnrm,hut-z(jz+1),dxy,ws)
L02876           f1 =fnrm
L02877
L02878           a1 = ws*dxy
L02879
L02880           a12= ws*dxy
L02881
L02882           a4 = (z(jz+1)-z(jz))*dxy
L02883
L02884           ftot=(a12*f12-a12*f1)/a1
L02885
L02886           fsw(jz,id,iurb)=ftot*a1/a4
L02887
L02888        enddo
L02889
L02890  ! radiation from wall to sky
L02891        do iz=1,nz_u
L02892         call fnrms(fnrm,ws,dxy,hut-z(iz))
L02893         f12=fnrm
L02894         call fnrms(fnrm,ws,dxy,hut-z(iz+1))
L02895         f1=fnrm
L02896         a1 = (z(iz+1)-z(iz))*dxy
L02897         a2 = (hut-z(iz+1))*dxy
L02898         a12= (hut-z(iz))*dxy
L02899         a4 = ws*dxy
L02900         ftot=(a12*f12-a2*f1)/a1
L02901         fws(iz,id,iurb)=ftot*a1/a4
L02902
L02903        enddo
L02904  !!!!!!!!!!!!!
L02905
L02906
L02907         do iz=1,nz_u
L02908
L02909  ! radiation from wall to ground
L02910
L02911           call fnrms (fnrm,ws,dxy,z(iz+1))
L02912           f12=fnrm
L02913           call fnrms (fnrm,ws,dxy,z(iz  ))
L02914           f1 =fnrm
L02915
L02916           a1= (z(iz+1)-z(iz) )*dxy
L02917
L02918           a2 = z(iz)*dxy
L02919           a12= z(iz+1)*dxy
L02920           a4 = ws*dxy
L02921
L02922           ftot=(a12*f12-a2*f1)/a1
L02923
L02924           fwg(iz,id,iurb)=ftot*a1/a4
L02925
L02926        enddo
L02927
L02928  ! radiation from sky to ground
L02929
L02930        call fprls (fprl,dxy,ws,hut)
L02931        fsg(id,iurb)=fprl
L02932
```

```
L02933        return
L02934        end subroutine view_factors
```

```
!file: phys/module_sf_bep.F    L2939-L2964    (subroutine fprls)
L02939        SUBROUTINE fprls (fprl,a,b,c)
L02940
L02941        implicit none
L02942
L02943
L02944
L02945        real a,b,c
L02946        real x,y
L02947        real fprl
L02948
L02949
L02950        x=a/c
L02951        y=b/c
L02952
L02953        if(a.eq.0.or.b.eq.0.)then
L02954         fprl=0.
L02955        else
L02956         fprl=log( ( (1.+x**2)*(1.+y**2)/(1.+x**2+y**2) )**.5)+  &
L02957             y*((1.+x**2)**.5)*atan(y/((1.+x**2)**.5))+          &
L02958             x*((1.+y**2)**.5)*atan(x/((1.+y**2)**.5))-          &
L02959             y*atan(y)-x*atan(x)
L02960         fprl=fprl*2./(pi*x*y)
L02961        endif
L02962
L02963        return
L02964        end subroutine fprls
```

```
!file: phys/module_sf_bep.F    L2969-L2997    (subroutine fnrms)
L02969        SUBROUTINE fnrms (fnrm,a,b,c)
L02970
L02971        implicit none
L02972
L02973
L02974
L02975        real a,b,c
L02976        real x,y,z,a1,a2,a3,a4,a5,a6
L02977        real fnrm
L02978
L02979        x=a/b
L02980        y=c/b
L02981        z=x**2+y**2
L02982
L02983        if(y.eq.0.or.x.eq.0)then
L02984         fnrm=0.
L02985        else
L02986         a1=log( (1.+x*x)*(1.+y*y)/(1.+z) )
L02987         a2=y*y*log(y*y*(1.+z)/z/(1.+y*y) )
L02988         a3=x*x*log(x*x*(1.+z)/z/(1.+x*x) )
L02989         a4=y*atan(1./y)
L02990         a5=x*atan(1./x)
L02991         a6=sqrt(z)*atan(1./sqrt(z))
L02992         fnrm=0.25*(a1+a2+a3)+a4+a5-a6
L02993         fnrm=fnrm/(pi*y)
L02994        endif
L02995
L02996        return
L02997        end subroutine fnrms
```

```
!file: phys/module_sf_bep.F    L3321-L3377    (subroutine icBEP_XY)
L03321        subroutine icBEP_XY(iurb,fww_u,fwg_u,fgw_u,fsw_u,             &
L03322                            fws_u,fsg_u,ndu,strd,ws,nzu,z_u)
L03323
L03324        implicit none
L03325
L03326  !    Street parameters
L03327        integer ndu     ! Number of street direction for each urban class
L03328        integer iurb
L03329
L03330        real strd(ndm)        ! Street length (fix to greater value to the horizontal length of the cells)
L03331        real ws(ndm)          ! Street width [m]
L03332
L03333  !    Grid parameters
L03334        integer nzu          ! Number of layer in the urban grid
L03335        real z_u(nz_um)       ! Height of the urban grid levels
L03336  ! ------------------------------------------------------------------------
L03337  !     Output
L03338  !------------------------------------------------------------------------
L03339
L03340  !   fww_u,fwg_u,fgw_u,fsw_u,fsg_u are the view factors used to compute the long wave
L03341  !   and the short wave radation. They are the part of radiation from a surface
L03342  !   or from the sky to another surface.
L03343
L03344        real fww_u(nz_um,nz_um,ndm,nurbm)         !  from wall to wall
L03345        real fwg_u(nz_um,ndm,nurbm)               !  from wall to ground
L03346        real fgw_u(nz_um,ndm,nurbm)               !  from ground to wall
L03347        real fsw_u(nz_um,ndm,nurbm)               !  from sky to wall
L03348        real fws_u(nz_um,ndm,nurbm)               !  from sky to wall
L03349        real fsg_u(ndm,nurbm)                     !  from sky to ground
L03350
L03351  ! ------------------------------------------------------------------------
L03352  !     Local
L03353  !------------------------------------------------------------------------
L03354
L03355        integer id
L03356
L03357  ! ------------------------------------------------------------------------
L03358  !     This routine compute the view factors
L03359  !------------------------------------------------------------------------
L03360  !
L03361  !Initialize
```

```
L03362  !
L03363        fww_u=0.
L03364        fwg_u=0.
L03365        fgw_u=0.
L03366        fsw_u=0.
L03367        fws_u=0.
L03368        fsg_u=0.
L03369
L03370        do id=1,ndu
L03371
L03372              call view_factors(iurb,nzu,id,strd(id),z_u,ws(id),  &
L03373                                fww_u,fwg_u,fgw_u,fsg_u,fsw_u,fws_u)
L03374
L03375        enddo
L03376        return
L03377        end subroutine icBEP_XY
```

```
!file: phys/module_sf_bep.F    L3381-L3497    (subroutine icBEPHI_XY)
L03381        subroutine icBEPHI_XY(hb_u,hi_urb1D,ss_u,pb_u,nzu,z_u)
L03382
L03383        implicit none
L03384  !-------------------------------------------------------------------------
L03385  !    Inputs
L03386  !-------------------------------------------------------------------------
L03387  !    Street parameters
L03388  !
L03389        real hi_urb1D(nz_um)    ! The probability that a building has an height h_b
L03390  !
L03391  !     Grid parameters
L03392  !
L03393        real z_u(nz_um)         ! Height of the urban grid levels
L03394  ! ------------------------------------------------------------------------
L03395  !     Output
L03396  !-------------------------------------------------------------------------
L03397
L03398        real ss_u(nz_um)   ! The probability that a building has an height equal to z
L03399        real pb_u(nz_um)         ! The probability that a building has an height greater or equal to z
L03400  !
L03401  !    Grid parameters
L03402  !
L03403        integer nzu                ! Number of layer in the urban grid
L03404
L03405  ! ------------------------------------------------------------------------
L03406  !     Local
L03407  !-------------------------------------------------------------------------
L03408        real hb_u(nz_um)        ! Bulding's heights [m]
L03409        integer iz_u,id,ilu
L03410
L03411        real dtot
L03412        real hbmax
L03413
L03414  !-------------------------------------------------------------------------
L03415
L03416  !Initialize variables
L03417  !
L03418
L03419        nzu=0
L03420        ss_u=0.
L03421        pb_u=0.
L03422
L03423  ! Normalisation of the building density
L03424
L03425           dtot=0.
L03426           hb_u=0.
L03427
L03428           do ilu=1,nz_um
L03429              dtot=dtot+hi_urb1D(ilu)
L03430           enddo
L03431
L03432           do ilu=1,nz_um
L03433              if (hi_urb1D(ilu)<0.) then
L03434  !              write(*,*) 'WARNING, HI_URB1D(ilu) < 0 IN BEP'
L03435                 go to 20
L03436              endif
L03437           enddo
L03438
L03439           if (dtot.gt.0.) then
L03440              continue
L03441           else
L03442  !           write(*,*) 'WARNING, HI_URB1D <= 0 IN BEP'
L03443              go to 20
L03444           endif
L03445
L03446           do ilu=1,nz_um
L03447              hi_urb1D(ilu)=hi_urb1D(ilu)/dtot
L03448           enddo
L03449
L03450           hb_u(1)=dz_u
L03451           do ilu=2,nz_um
L03452              hb_u(ilu)=dz_u+hb_u(ilu-1)
L03453           enddo
L03454
L03455
L03456  ! Compute pb and ss
L03457
L03458
L03459           hbmax=0.
L03460
L03461           do ilu=1,nz_um
L03462              if (hi_urb1D(ilu)>0.and.hi_urb1D(ilu)<=1.) then
L03463                 hbmax=hb_u(ilu)
L03464              endif
L03465           enddo
L03466
L03467           do iz_u=1,nz_um-1
L03468              if(z_u(iz_u+1).gt.hbmax)go to 10
L03469           enddo
L03470
```

```
L03471  10       continue
L03472
L03473           nzu=iz_u+1
L03474
L03475           if ((nzu+1).gt.nz_um) then
L03476               write(*,*) 'error, nz_um has to be increased to at least',nzu+1
L03477               stop
L03478           endif
L03479
L03480              do iz_u=1,nzu
L03481                 ss_u(iz_u)=0.
L03482                 do ilu=1,nz_um
L03483                    if(z_u(iz_u).le.hb_u(ilu)                    &
L03484                      .and.z_u(iz_u+1).gt.hb_u(ilu))then
L03485                          ss_u(iz_u)=ss_u(iz_u)+hi_urb1D(ilu)
L03486                    endif
L03487                 enddo
L03488              enddo
L03489
L03490              pb_u(1)=1.
L03491              do iz_u=1,nzu
L03492                 pb_u(iz_u+1)=max(0.,pb_u(iz_u)-ss_u(iz_u))
L03493              enddo
L03494
L03495  20    continue
L03496        return
L03497        end subroutine icBEPHI_XY
```

与 14b § 六同源；差异同上（无绿屋顶/无 BEM 维度）。icBEP_XY 的 strd 同样固定 1e5 m（无限长峡谷）。

```
!file: phys/module_sf_bep.F    L3000-L3108    (subroutine init_para)
L03000        SUBROUTINE init_para(alag_u,alaw_u,alar_u,csg_u,csw_u,csr_u,&
L03001                   twini_u,trini_u,tgini_u,albg_u,albw_u,albr_u,emg_u,emw_u,&
L03002                   emr_u,z0g_u,z0r_u,nd_u,strd_u,drst_u,ws_u,bs_u,h_b,d_b)
L03003
L03004  ! initialization routine, where the variables from the table are read
L03005
L03006        implicit none
L03007
L03008        integer iurb            ! urban class number
L03009  !    Building parameters
L03010        real alag_u(nurbm)      ! Ground thermal diffusivity [m^2 s^-1]
L03011        real alaw_u(nurbm)      ! Wall thermal diffusivity [m^2 s^-1]
L03012        real alar_u(nurbm)      ! Roof thermal diffusivity [m^2 s^-1]
L03013        real csg_u(nurbm)       ! Specific heat of the ground material [J m^3 K^-1]
L03014        real csw_u(nurbm)       ! Specific heat of the wall material [J m^3 K^-1]
L03015        real csr_u(nurbm)       ! Specific heat of the roof material [J m^3 K^-1]
L03016        real twini_u(nurbm)     ! Temperature inside the buildings behind the wall [K]
L03017        real trini_u(nurbm)     ! Temperature inside the buildings behind the roof [K]
L03018        real tgini_u(nurbm)     ! Initial road temperature
L03019
L03020  !    Radiation parameters
L03021        real albg_u(nurbm)      ! Albedo of the ground
L03022        real albw_u(nurbm)      ! Albedo of the wall
L03023        real albr_u(nurbm)      ! Albedo of the roof
L03024        real emg_u(nurbm)       ! Emissivity of ground
L03025        real emw_u(nurbm)       ! Emissivity of wall
L03026        real emr_u(nurbm)       ! Emissivity of roof
L03027
L03028  !    Roughness parameters
L03029        real z0g_u(nurbm)       ! The ground's roughness length
L03030        real z0r_u(nurbm)       ! The roof's roughness length
L03031
L03032  !    Street parameters
L03033        integer nd_u(nurbm)     ! Number of street direction for each urban class
L03034
L03035        real strd_u(ndm,nurbm)  ! Street length (fix to greater value to the horizontal length of the cells)
L03036        real drst_u(ndm,nurbm)  ! Street direction [degree]
L03037        real ws_u(ndm,nurbm)    ! Street width [m]
L03038        real bs_u(ndm,nurbm)    ! Building width [m]
L03039        real h_b(nz_um,nurbm)   ! Bulding's heights [m]
L03040        real d_b(nz_um,nurbm)   ! The probability that a building has an height h_b
L03041
L03042        integer i,iu
L03043        integer nurb ! number of urban classes used
L03044
L03045  !
L03046  !Initialize some variables
L03047  !
L03048
L03049         h_b=0.
L03050         d_b=0.
L03051
L03052         nurb=ICATE
L03053         do iu=1,nurb
L03054            nd_u(iu)=0
L03055         enddo
L03056
L03057         csw_u=CAPB_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L03058         csr_u=CAPR_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L03059         csg_u=CAPG_TBL / (( 1.0 / 4.1868 ) * 1.E-6)
L03060         do i=1,icate
L03061           alaw_u(i)=AKSB_TBL(i) / csw_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L03062           alar_u(i)=AKSR_TBL(i) / csr_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L03063           alag_u(i)=AKSG_TBL(i) / csg_u(i) / (( 1.0 / 4.1868 ) * 1.E-2)
L03064         enddo
L03065         twini_u=TBLEND_TBL
L03066         trini_u=TRLEND_TBL
L03067         tgini_u=TGLEND_TBL
L03068         albw_u=ALBB_TBL
L03069         albr_u=ALBR_TBL
L03070         albg_u=ALBG_TBL
L03071         emw_u=EPSB_TBL
L03072         emr_u=EPSR_TBL
L03073         emg_u=EPSG_TBL
```

```
L03074          z0r_u=Z0R_TBL
L03075          z0g_u=Z0G_TBL
L03076          nd_u=NUMDIR_TBL
L03077          do iu=1,icate
L03078                if(ndm.lt.nd_u(iu))then
L03079                  write(*,*)'ndm too small in module_sf_bep, please increase to at least ', nd_u(iu)
L03080                  write(*,*)'remember also that urban_map_zrd should be equal or greater than nz_um*ndm*nwr-u!'
L03081                  stop
L03082                endif
L03083            do i=1,nd_u(iu)
L03084              drst_u(i,iu)=STREET_DIRECTION_TBL(i,iu) * pi/180.
L03085              ws_u(i,iu)=STREET_WIDTH_TBL(i,iu)
L03086              bs_u(i,iu)=BUILDING_WIDTH_TBL(i,iu)
L03087            enddo
L03088          enddo
L03089          do iu=1,ICATE
L03090             if(nz_um.lt.numhgt_tbl(iu)+3)then
L03091                 write(*,*)'nz_um too small in module_sf_bep, please increase to at least ',numhgt_tbl(iu)+3
L03092                 write(*,*)'remember also that urban_map_zrd should be equal or greater than nz_um*ndm*nwr-u!'
L03093                 stop
L03094             endif
L03095            do i=1,NUMHGT_TBL(iu)
L03096              h_b(i,iu)=HEIGHT_BIN_TBL(i,iu)
L03097              d_b(i,iu)=HPERCENT_BIN_TBL(i,iu)
L03098            enddo
L03099          enddo
L03100
L03101          do i=1,ndm
L03102           do iu=1,nurbm
L03103            strd_u(i,iu)=100000.
L03104           enddo
L03105          enddo
L03106
L03107          return
L03108        END SUBROUTINE init_para
```

与 14b 版差异：只读 TBL 的 BEP 所需子集——**无任何窗户/BEM 量**（无 albwin_u/emwind_u/cop_u 等；BEP 单独版根本没有窗户辐射成分）；albwindow/foncs 单元只存在于 bep_bem（14b § 七），两文件互不调用——「跨模块隐性依赖」的说法不成立，此前的疑点已核销。守卫信息串同源（ndm/nz_um 下限检查）。

```
!file: phys/module_sf_bep.F    L3112-L3189    (subroutine angle)
L03112        subroutine angle(along,alat,day,realt,zr,deltar,ah)
L03113  !     ----------------
L03114  !
L03115  !         Computation of the solar angles
L03116  !         schayes (1982,atm.  env. , p1407)
L03117  ! Inputs
L03118  !=======================
L03119  ! along=longitud
L03120  ! alat=latitude
L03121  ! day=julian day (from the beginning of the year)
L03122  ! realt= time GMT in hours
L03123  ! Outputs
L03124  !============================
L03125  ! zr=solar zenith angle
L03126  ! deltar=declination angle
L03127  ! ah=hour angle
L03128  !==============================
L03129
L03130        implicit none
L03131        real along,alat, realt, zr, deltar, ah, arg
L03132        real rad,om,radh,initt, pii, drad, alongt, cphi, sphi
L03133        real c1, c2, c3, s1, s2, s3, delta, rmsr2, cd, sid
L03134        real et, ahor, chor, coznt
L03135        integer day
L03136
L03137
L03138        data rad,om,radh,initt/0.0174533,0.0172142,0.26179939,0/
L03139
L03140         zr=0.
L03141         deltar=0.
L03142         ah=0.
L03143
L03144         pii = 3.14159265358979312
L03145         drad = pii/180.
L03146
L03147         alongt=along/15.
L03148         cphi=cos(alat*drad)
L03149         sphi=sin(alat*drad)
L03150  !
L03151  !     declination
L03152  !
L03153         arg=om*day
L03154         c1=cos(arg)
L03155         c2=cos(2.*arg)
L03156         c3=cos(3.*arg)
L03157         s1=sin(arg)
L03158         s2=sin(2.*arg)
L03159         s3=sin(3.*arg)
L03160         delta=0.33281-22.984*c1-0.3499*c2-0.1398*c3+3.7872*s1+0.03205*s2+0.07187*s3
L03161         rmsr2=(1./(1.-0.01673*c1))**2
L03162         deltar=delta*rad
L03163         cd=cos(deltar)
L03164         sid=sin(deltar)
L03165  !
L03166  !     time equation in hours
L03167  !
L03168         et=0.0072*c1-0.0528*c2-0.0012*c3-0.1229*s1-0.1565*s2-0.0041*s3
L03169  !
L03170  !
L03171  !   hour angle
L03172  !
L03173
```

```
L03174  !      ifh=0
L03175
L03176   !     ahor=realt-12.+ifh+et+alongt
L03177        ahor=realt-12.+et+alongt
L03178        ah=ahor*radh
L03179        chor=cos(ah)
L03180  !
L03181  !    zenith angle
L03182  !
L03183        coznt=sphi*sid+cphi*cd*chor
L03184
L03185         zr=acos(coznt)
L03186
L03187        return
L03188
L03189        END SUBROUTINE angle
```

讲解：Schayes 1982 太阳角计算（Spencer 型 Fourier 级数赤纬 + 时差方程 + 时角 + 天顶角）。**当前未被调用**（BEP 主入口 L491 的调用被注释——太阳几何改由辐射 driver 的 declin_urb/cosz_urb2d/omg_urb2d 统一提供，避免两套日历口径）。保留单元的价值：理想化/离线试验可用；用时注意 along 单位是度、realt 是 GMT 小时。

```
!file: phys/module_sf_bep.F    L3194-L3314    (subroutine upward_rad)
L03194        subroutine upward_rad(ndu,nzu,ws,bs,sigma,pb,ss,                    &
L03195                              tg,emg_u,albg_u,rlg,rsg,sfg,                  &
L03196                              tw,emw_u,albw_u,rlw,rsw,sfw,                  &
L03197                              tr,emr_u,albr_u,rld,rs, sfr,                  &
L03198                              rs_abs,rl_up,emiss,grdflx_urb)
L03199  !
L03200  ! IN this surboutine we compute the upward longwave flux, and the albedo
L03201  ! needed for the radiation scheme
L03202  !
L03203                              implicit none
L03204
L03205  !
L03206  !INPUT VARIABLES
L03207  !
L03208        real rsw(2*ndm,nz_um)        ! Short wave radiation at the wall for a given canyon direction [W/m2]
L03209        real rlw(2*ndm,nz_um)         ! Long wave radiation at the walls for a given canyon direction [W/m2]
L03210        real rsg(ndm)                   ! Short wave radiation at the canyon for a given canyon direction [W/m2]
L03211        real rlg(ndm)                   ! Long wave radiation at the ground for a given canyon direction [W/m2]
L03212        real rs                        ! Short wave radiation at the horizontal surface from the sun [W/m2]
L03213        real sfw(2*ndm,nz_um)      ! Sensible heat flux from walls [W/m2]
L03214        real sfg(ndm)               ! Sensible heat flux from ground (road) [W/m2]
L03215        real sfr(ndm,nz_um)      ! Sensible heat flux from roofs [W/m2]
L03216        real rld                          ! Long wave radiation from the sky [W/m2]
L03217        real albg_u                     ! albedo of the ground/street
L03218        real albw_u                     ! albedo of the walls
L03219        real albr_u                     ! albedo of the roof
L03220        real ws(ndm)                        ! width of the street
L03221        real bs(ndm)
L03222                             ! building size
L03223        real pb(nz_um)                ! Probability to have a building with an height equal or higher
L03224        integer nzu
L03225        real ss(nz_um)                ! Probability to have a building of a given height
L03226        real sigma
L03227        real emg_u                        ! emissivity of the street
L03228        real emw_u                        ! emissivity of the wall
L03229        real emr_u                        ! emissivity of the roof
L03230        real tw(2*ndm,nz_um,nwr_u)  ! Temperature in each layer of the wall [K]
L03231        real tr(ndm,nz_um,nwr_u)  ! Temperature in each layer of the roof [K]
L03232        real tg(ndm,ng_u)          ! Temperature in each layer of the ground [K]
L03233        integer id ! street direction
L03234        integer ndu ! number of street directions
L03235  !OUTPUT/INPUT
L03236        real rs_abs  ! absrobed solar radiationfor this street direction
L03237        real rl_up   ! upward longwave radiation for this street direction
L03238        real emiss ! mean emissivity
L03239        real grdflx_urb ! ground heat flux
L03240  !LOCAL
L03241        integer iz,iw
L03242        real rl_inc,rl_emit
L03243        real gfl
L03244        integer ix,iy,iwrong
L03245
L03246           iwrong=1
L03247        do iz=1,nzu+1
L03248        do id=1,ndu
L03249        do iw=1,nwr_u
L03250          if(tr(id,iz,iw).lt.100.)then
L03251                write(*,*)'in upward_rad ',iz,id,iw,tr(id,iz,iw)
L03252                iwrong=0
L03253          endif
L03254          if(tw(2*id-1,iz,iw).lt.100.) then
L03255             write(*,*)'in upward_rad ',iz,id,iw,tw(2*id-1,iz,iw)
L03256             iwrong=0
L03257          endif
L03258          if(tw(2*id,iz,iw).lt.100.) then
L03259             write(*,*)'in upward_rad ',iz,id,iw,tw(2*id,iz,iw)
L03260             iwrong=0
L03261          endif
L03262        enddo
L03263        enddo
L03264        enddo
L03265        do id=1,ndu
L03266        do iw=1,ng_u
L03267            if(tg(id,iw).lt.100.) then
L03268              write(*,*)'in upward_rad ',id,iw,tg(id,iw)
L03269              iwrong=0
L03270            endif
L03271        enddo
L03272        enddo
L03273             if(iwrong.eq.0)stop
L03274
```

```
L03275        rl_up=0.
L03276
L03277        rs_abs=0.
L03278        rl_inc=0.
L03279        emiss=0.
L03280        rl_emit=0.
L03281        grdflx_urb=0.
L03282        do id=1,ndu
L03283         rl_emit=rl_emit-( emg_u*sigma*(tg(id,ng_u)**4.)+(1-emg_u)*rlg(id))*ws(id)/(ws(id)+bs(id))/ndu
L03284         rl_inc=rl_inc+rlg(id)*ws(id)/(ws(id)+bs(id))/ndu
L03285         rs_abs=rs_abs+(1.-albg_u)*rsg(id)*ws(id)/(ws(id)+bs(id))/ndu
L03286           gfl=(1.-albg_u)*rsg(id)+emg_u*rlg(id)-emg_u*sigma*(tg(id,ng_u)**4.)+sfg(id)
L03287           grdflx_urb=grdflx_urb-gfl*ws(id)/(ws(id)+bs(id))/ndu
L03288
L03289            do iz=2,nzu
L03290              rl_emit=rl_emit-(emr_u*sigma*(tr(id,iz,nwr_u)**4.)+(1-emr_u)*rld)*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L03291              rl_inc=rl_inc+rld*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L03292              rs_abs=rs_abs+(1.-albr_u)*rs*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L03293              gfl=(1.-albr_u)*rs+emr_u*rld-emr_u*sigma*(tr(id,iz,nwr_u)**4.)+sfr(id,iz)
L03294              grdflx_urb=grdflx_urb-gfl*ss(iz)*bs(id)/(ws(id)+bs(id))/ndu
L03295           enddo
L03296
L03297           do iz=1,nzu
L03298              rl_emit=rl_emit-(emw_u*sigma*( tw(2*id-1,iz,nwr_u)**4.+tw(2*id,iz,nwr_u)**4. )+          &
L03299                 (1-emw_u)*( rlw(2*id-1,iz)+rlw(2*id,iz) ) )*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L03300              rl_inc=rl_inc+(( rlw(2*id-1,iz)+rlw(2*id,iz) ) )*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L03301              rs_abs=rs_abs+((1.-albw_u)*( rsw(2*id-1,iz)+rsw(2*id,iz) ) )*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L03302              gfl=(1.-albw_u)*(rsw(2*id-1,iz)+rsw(2*id,iz)) +emw_u*( rlw(2*id-1,iz)+rlw(2*id,iz) )   &
L03303               -emw_u*sigma*( tw(2*id-1,iz,nwr_u)**4.+tw(2*id,iz,nwr_u)**4. )+(sfw(2*id-1,iz)+sfw(2*id,iz))
L03304              grdflx_urb=grdflx_urb-gfl*dz_u*pb(iz+1)/(ws(id)+bs(id))/ndu
L03305           enddo
L03306
L03307        enddo
L03308          emiss=(emg_u+emw_u+emr_u)/3.
L03309          rl_up=(rl_inc+rl_emit)-rld
L03310
L03311
L03312        return
L03313
L03314        END SUBROUTINE upward_rad
```

与 14b § 七差异：无 PV/绿屋顶项；<100 K 哨兵扫全部固体温度（同样 write+stop 硬停，生产建议换 wrf_error_fatal）。

```
!file: phys/module_sf_bep.F    L3499-L3535
L03499  ! ===6=8================================================================72
L03500  END MODULE module_sf_bep
L03501
L03502        FUNCTION bep_nurbm () RESULT (bep_val_nurbm)
L03503           USE module_sf_bep
L03504           IMPLICIT NONE
L03505           INTEGER :: bep_val_nurbm
L03506           bep_val_nurbm = nurbm
L03507        END FUNCTION bep_nurbm
L03508
L03509        FUNCTION bep_ndm () RESULT (bep_val_ndm)
L03510           USE module_sf_bep
L03511           IMPLICIT NONE
L03512           INTEGER :: bep_val_ndm
L03513           bep_val_ndm = ndm
L03514        END FUNCTION bep_ndm
L03515
L03516        FUNCTION bep_nz_um () RESULT (bep_val_nz_um)
L03517           USE module_sf_bep
L03518           IMPLICIT NONE
L03519           INTEGER :: bep_val_nz_um
L03520           bep_val_nz_um = nz_um
L03521        END FUNCTION bep_nz_um
L03522
L03523        FUNCTION bep_ng_u () RESULT (bep_val_ng_u)
L03524           USE module_sf_bep
L03525           IMPLICIT NONE
L03526           INTEGER :: bep_val_ng_u
L03527           bep_val_ng_u = ng_u
L03528        END FUNCTION bep_ng_u
L03529
L03530        FUNCTION bep_nwr_u () RESULT (bep_val_nwr_u)
L03531           USE module_sf_bep
L03532           IMPLICIT NONE
L03533           INTEGER :: bep_val_nwr_u
L03534           bep_val_nwr_u = nwr_u
L03535        END FUNCTION bep_nwr_u
```

讲解：五个宿主关联查询函数（bep_nurbm/ndm/nz_um/ng_u/nwr_u），机制同 14b § 七。

七、改造入口

1. **BEP 单开实验**（论文 2 对照组候选）：sf_urban_physics=1 配 BouLac/MYJ；注意它没有室内废热——UHI 模拟偏冷，与 BEP+BEM（=3）对照时可量化 BEM 贡献。
2. **两文件同步红线**：本文件与 bep_bem 的同源单元（param/shadow_mas/shade_wall/long_rad/flux_wall/flux_flat/icBEP 族…）是历史分叉拷贝——改其一必须 grep 另一文件同名子程序决定是否同步；surf_temp/flux_flat 在 module_sf_bem.F 还有一份（16 篇确认）。
3. **拖曳系数**：常数 cdrag=0.4 改成 bep_bem 的 NEW CDRAG 是最小改造（把 BEP1D 加 alp 计算传入 buildings）。

4. **太阳几何**：angle 单元可用于 ideal 案例自算太阳角（替代 radiation driver 依赖）。

八、上下游链接

- 共享底座全文：`code_14a_module_sf_bep_bem.md`、`code_14b_module_sf_bep_bem.md`
- 同族：`code_16_module_sf_bem.md`（BEM 主体）；module_sf_urban（17 篇待交付）
- 驱动链：`code_22_module_pbl_driver.md`；面层 `code_10_module_sf_mynnsfc.md`
- 教材正文：ch17_urban_canopy.md、ch20_modifying_wrf.md、ch19_lanzhou_heatwave.md

code_16 — BEM 建筑能量模型（module_sf_bem.F 全文精读）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_bem.F	2636	e93033f4eff5816a89e6cf51c65d738a

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 A 级全文（26 单元全收）。被调方：module_sf_bep_bem.F 的 BEP1D（code_14a § 五 ⑨ 的 BEM 循环——每时间步每格点每建筑型调一次）。

二、定位

一句话：BEM（Salamanca–Martilli 建筑能量模型）——把每栋「建筑型」当一组多层墙/楼板/窗户围合的空调房间，解室内气温/比湿的演化方程，输出空调废热、耗电、通风热与 PV 发电；它是**论文 2 的直接改造对象**（cal-steel 标定参数 cop/beta/targtemp… 目前是类级标量，改造目标 = 逐格场）。

物理边界（BEM 管什么 / 不管什么）：管——室内空气能量与水分收支、围护结构多层导热、室内外辐射交换、通风与空调；不管——城市峡谷气象（BEP 管，14a/14b）、墙外侧温度（来自 BEP 的 wall/roof 状态）。

六面房间约定（文件头 ASCII 图）：面 1/2=东西墙（bw • dzlev）、3/4=南北墙（bl • dzlev）、5=地板（bw • bl）、6=天花板（bw • bl）；底层面 5 接楼下地基 tgrd、顶层面 6 接屋顶 trof；窗户按 pwin 占比嵌在四面竖墙上。

三、文件头：BEM 物理常数

```
!file: phys/module_sf_bem.F    L1-L36
L00001  MODULE module_sf_bem
L00002  #if defined(mpas)
L00003  use mpas_atmphys_utilities, only: physics_error_fatal
L00004  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00005  #else
L00006  use module_wrf_error
L00007  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00008  #endif
L00009  ! ------------------------------------------------------------------
L00010  !  Variables and constants used in the BEM module
L00011  ! ------------------------------------------------------------------
L00012
L00013          real emins      !emissivity of the internal walls
L00014          parameter (emins=0.9)
L00015          real albins         !albedo of the internal walls
L00016  !!      parameter (albins=0.5)
L00017          parameter (albins=0.3)
L00018
L00019          real thickwin           !thickness of the window [m]
L00020          parameter (thickwin=0.006)
L00021          real cswin      !Specific heat of the windows [J/(m3.K)]
L00022          parameter(cswin= 2.268e+06)
L00023
L00024          real temp_rat            !power of the A.C. heating/cooling the indoor air [K/s]
L00025          parameter(temp_rat=0.001)
L00026
L00027          real hum_rat            !power of the A.C. drying/moistening the indoor air [(Kg/kg)/s]
L00028          parameter(hum_rat=1.e-06)
L00029
L00030          real,parameter :: effpv=0.19  ! Efficiency of PV panels installed at the roofs, typical values [0.10,0.15]
L00031
L00032      CONTAINS
L00033
L00034  !====6=========================================================72
L00035  !====6=========================================================72
L00036
```

讲解：① 室内表面发射率 emins=0.9、反照率 albins=0.3（曾被 0.5，注释在案）；② 窗户厚 6 mm、热容 2.268e6 J/(m³ • K)——窗户按**单层集总热容**演化；③temp_rat=0.001 K/s 与 hum_rat=1e-6 (kg/kg)/s 是空调的**最大调温/调湿速率**（防室内状态突变）；④effpv=0.19 光伏效率（注释给出典型区间 0.10–0.15，实际用了偏高值——论文 2 做 PV 敏感性实验时的第一检查点）。**论文 2 改造提示：这五个 parameter 是模块级硬编码，逐格场化时最先要「参数场化」的就是它们与 cop/beta 族 TBL 标量。**

四、主入口 BEM（全文）

```
!file: phys/module_sf_bem.F    L37-L820    (subroutine BEM)
L00037      subroutine BEM(nzcanm,nlev,nhourday,dt,bw,bl,dzlev,            &
L00038                     nwal,nflo,nrof,ngrd,hswalout,gswal,              &
L00039                     hswinout,hsrof,lsrof,gsrof,hspv,                 &
L00040                     latent,sigma,albwal,albwin,albrof,               &
L00041                  emrof,emwal,emwin,rswal,rlwal,rair,cp,              &
L00042                  rhoout,tout,humout,press,                          &
L00043                  rs,swddif,rl,dzwal,cswal,kwal,pwin,cop,beta,sw_cond,   &
L00044                     timeon,timeoff,targtemp,gaptemp,targhum,gaphum, &
L00045                     perflo,gr_frac_roof,pv_frac_roof,gr_flag, &
L00046                     uout,vout,                                        &
L00047                     hsesf,hsequip,dzflo,                              &
L00048                  csflo,kflo,dzgrd,csgrd,kgrd,dzrof,csrof,           &
L00049                  krof,tlev,shumlev,twal,twin,tflo,tgrd,trof,        &
L00050                  hsout,hlout,consump,eppv,tpv,hsvent,hlvent,hfgr,&
L00051                     tr_av,tpv_print,sfpv,sfr_indoor)
L00052
L00053
L00054
L00055
L00056  ! ----------------------------------------------------------------------
L00057      implicit none
L00058
L00059  ! ----------------------------------------------------------------------
L00060  !              TOP
L00061  !          ---------------------
L00062  !          ! ----------------- !--->roof (-) : level number
L00063  !          ! !       ! !     rem: the windows are given
L00064  !          ! !---------------! !                  with respect to the
L00065  !          ! !---------------! !                  vertical walls-->win(2)
L00066  !      (n)! !(1)         (1)!-!(n)
L00067  !          ! !---------------! !     2D vision of the building
L00068  !   WEST       ! !-------4-------! ! EAST
L00069  !       I ! ! 1    ilev    2! ! II              ^
L00070  !          ! !-------3--------! !           !
L00071  !          ! !---------------! !--->floor 1       !
L00072  !          ! !       ! !                  !
L00073  !          ! !       ! !                  !
L00074  !          ! ----------------- !          <--------------(n)
L00075  !          ------------------------>ground   ------------(1)
L00076  !             BOTTOM
L00077  !                i(6)
L00078  !                     i
L00079  !                       +---------v-----+
L00080  !                      /|              /|    3D vision of a room
L00081  !                     / | 4           / |
L00082  !                    /  |            /  |
L00083  !                   /   |           /   |
L00084  !                  /    |          /    |
L00085  !                 +---------------+     |
L00086  !                 |  1   |        |  2  |
L00087  !                 |      +---------|-----+
L00088  !       dzlev     |     /          |    /
L00089  !                 |    /    3      |   /
L00090  !                 |   /bw          |  /
L00091  !                 |  /             | /
L00092  !                 | /              |/
L00093  !                 +---------------+
L00094  !                       ^ bl
L00095  !              i
L00096  !                       i
L00097  !             (5)
L00098  !-----------------------------------------------------------------------
L00099
L00100
L00101  ! Input:
L00102  ! -----
L00103
L00104      real dt             !time step [s]
L00105
L00106            integer nzcanm                !Maximum number of vertical levels in the urban grid
L00107      integer nlev            !number of floors in the building
L00108      integer nwal                    !number of levels inside the wall
L00109      integer nrof                    !number of levels inside the roof
L00110      integer nflo                    !number of levels inside the floor
L00111      integer ngrd                    !number of levels inside the ground
L00112      real dzlev          !vertical grid resolution [m]
L00113      real bl             !Building length [m]
L00114      real bw                         !Building width [m]
L00115
L00116      real albwal         !albedo of the walls
L00117      real albwin         !albedo of the windows
L00118      real albrof         !albedo of the roof
L00119
L00120      real emwal              !emissivity of the walls
L00121
L00122      real emrof          !emissivity of the roof
L00123          real emwin                      !emissivity of the windows
L00124
```

```
      real pwin                       !window proportion
      real,    intent(in) :: cop      !Coefficient of performance of the A/C systems
      real,    intent(in) :: beta     !Thermal efficiency of the heat exchanger
          integer, intent(in) :: sw_cond  ! Air Conditioning switch
          real,    intent(in) :: timeon   ! Initial local time of A/C systems
          real,    intent(in) :: timeoff  ! Ending local time of A/C systems
          real,    intent(in) :: targtemp ! Target temperature of A/C systems
          real,    intent(in) :: gaptemp  ! Comfort range of indoor temperature
          real,    intent(in) :: targhum  ! Target humidity of A/C systems
          real,    intent(in) :: gaphum   ! Comfort range of specific humidity
          real,    intent(in) :: perflo   ! Peak number of occupants per unit floor area
          real gr_frac_roof
          real pv_frac_roof
          integer gr_flag
          real   uout(nzcanm)
          real    vout(nzcanm)
          real,    intent(in) :: hsesf    !
          real,    intent(in) :: hsequip(24) !

      real cswal(nwal)        !Specific heat of the wall [J/(m3.K)]

      real csflo(nflo)        !Specific heat of the floor [J/(m3.K)]
      real csrof(nrof)        !Specific heat of the roof [J/(m3.K)]
      real csgrd(ngrd)        !Specific heat of the ground [J/(m3.K)]

      real kwal(nwal+1)       !Thermal conductivity in each layers of the walls (face) [W/(m.K)]
      real kflo(nflo+1)       !Thermal diffusivity in each layers of the floors (face) [W/(m.K)]
      real krof(nrof+1)       !Thermal diffusivity in each layers of the roof (face) [W/(m.K)]
      real kgrd(ngrd+1)       !Thermal diffusivity in each layers of the ground (face) [W/(m.K)]

      real dzwal(nwal)        !Layer sizes of walls [m]
      real dzflo(nflo)        !Layer sizes of floors [m]
      real dzrof(nrof)        !Layer sizes of roof [m]
      real dzgrd(ngrd)        !Layer sizes of ground [m]
      real tpv
          real tr_av
      real latent                      !latent heat of evaporation [J/Kg]

          real swddif
      real rs             !external short wave radiation [W/m2]
      real rl             !external long wave radiation [W/m2]
      real rswal(4,nzcanm)        !short wave radiation reaching the exterior walls [W/m2]
          real rlwal(4,nzcanm)        !long wave radiation reaching the walls [W/m2]
      real rhoout(nzcanm)     !exterior air density [kg/m3]
      real tout(nzcanm)       !external temperature [K]
      real humout(nzcanm)     !absolute humidity [Kgwater/Kgair]
      real press(nzcanm)      !external air pressure [Pa]

      real hswalout(4,nzcanm)         !outside walls sensible heat flux [W/m2]
      real hswinout(4,nzcanm)         !outside window sensible heat flux [W/m2]
      real hsrof          !Sensible heat flux at the roof [W/m2]
      real lsrof
          real rair           !ideal gas constant  [J.kg-1.K-1]
      real sigma          !parameter (wall is not black body) [W/m2.K4]
      real cp             !specific heat of air [J/kg.K]
          real hfgr       !Green roof heat flux

  !Input-Output
  !------------
      real tlev(nzcanm)       !temperature of the floors [K]
      real shumlev(nzcanm)        !specific humidity of the floor [kg/kg]
      real twal(4,nwal,nzcanm)    !walls temperatures [K]
      real twin(4,nzcanm)     !windows temperatures [K]
      real tflo(nflo,nzcanm-1)    !floor temperatures [K]
      real tgrd(ngrd)             !ground temperature [K]
      real trof(nrof)             !roof temperature [K]
      real hsout(nzcanm)      !sensible heat emitted outside the floor [W]
      real hlout(nzcanm)      !latent heat emitted outside the floor [W]
          real consump(nzcanm)            !Consumption for the a.c. in each floor [W]
      real hsvent(nzcanm)     !sensible heat generated by natural ventilation [W]
      real hlvent(nzcanm)     !Latent heat generated by natural ventilation [W]
          real gsrof                      !heat flux flowing inside the roof [W/m2]
          real hspv           !Sensible heat flux at the roof from the PV panels [W/m2]
          real gswal(4,nzcanm)             !heat flux flowing inside the floors [W/m2]
          real eppv                        !Electricity production of PV panels [W]
          real sfr_indoor,sfpv,tpv_print
  ! Local:
  ! -----
      integer swwal                   !swich for the physical coefficients calculation
      integer ilev            !index for rooms
      integer iwal            !index for walls
      integer iflo            !index for floors
      integer ivw         !index for vertical walls
          integer igrd                    !index for ground
          integer irof                    !index for roof
          real hseqocc(nzcanm)        !sensible heat generated by equipments and occupants [W]
      real hleqocc(nzcanm)        !Latent heat generated by occupants [W]
          real hscond(nzcanm)     !sensible heat generated by wall conduction [W]
          real hslev(nzcanm)      !sensible heat flux generated inside the room [W]
          real hllev(nzcanm)      !latent heat flux generatd inside the room [W]
      real surwal(6,nzcanm)           !Surface of the walls [m2]
      real surwal1D(6)            !wall surfaces of a generic room [m2]
      real rsint(6)               !short wave radiation reaching the indoor walls[W/m2]
      real rswalins(6,nzcanm)         !internal short wave radiation for the building [W/m2]
      real twin1D(4)              !temperature of windows for a particular room [K]
      real twal_int(6)        !temperature of the first internal layers of a room [K]
      real rlint(6)               !internal wall long wave radiation [w/m2]
      real rlwalins(6,nzcanm)         !internal long wave radiation for the building [W/m2]
      real hrwalout(4,nzcanm)         !external radiative flux to the walls [W/m2]
      real hrwalins(6,nzcanm)         !inside radiative flux to the walls [W/m2]
      real hrwinout(4,nzcanm)         !external radiative flux to the window [W/m2]
      real hrwinins(4,nzcanm)         !inside radiative flux to the window [W/m2]
      real hrrof          !external radiative flux to the roof [W/m2]
      real hs
          real hsneed(nzcanm)     !sensible heat needed by the room [W]
      real hlneed(nzcanm)     !Latent heat needed by the room [W]
          real hswalins(6,nzcanm)         !inside walls sensible heat flux [W/m2]
```

```
L00232      real hswalins1D(6)
L00233      real hswinins(4,nzcanm)         !inside window sensible heat flux [W/m2]
L00234      real hswinins1D(4)
L00235      real htot(2)            !total heat flux at the wall [W/m2]
L00236      real twal1D(nwal)
L00237      real tflo1D(nflo)
L00238          real tgrd1D(ngrd)
L00239          real trof1D(nrof)
L00240      real rswal1D(4)
L00241      real Qb             !Overall heat capacity of the indoor air [J/K]
L00242      real vollev         !volume of the room [m3]
L00243      real rhoint         !density of the internal air [Kg/m3]
L00244      real cpint          !specific heat of the internal air [J/kg.K]
L00245          real humdry                     !specific humidiy of dry air [kg water/kg dry air]
L00246      real radflux                    !Function to compute the total radiation budget
L00247      real consumpbuild               !Energetic consumption for the entire building [KWh/s]
L00248          real hsoutbuild                 !Total sensible heat ejected into the atmosphere[W]
L00249                                          !by the air conditioning system and per building
L00250          real nhourday                   !number of hours from midnight, local time
L00251          real hfgrd                      !Dummy variable to assign hfgr=0 to walls, windows and ground
L00252
L00253  !
L00254          parameter(hfgrd=0)
L00255  !-------------------------------------------
L00256  !Initialization
L00257  !-------------------------------------------
L00258         do ilev=1,nzcanm
L00259            hseqocc(ilev)=0.
L00260            hleqocc(ilev)=0.
L00261            hscond(ilev)=0.
L00262            hslev(ilev)=0.
L00263            hllev(ilev)=0.
L00264         enddo
L00265
L00266  !Calculation of the surfaces of the building
L00267  !-------------------------------------------
L00268
L00269
L00270      do ivw=1,6
L00271      do ilev=1,nzcanm
L00272       surwal(ivw,ilev)=1.   !initialisation
L00273      end do
L00274      end do
L00275
L00276      do ilev=1,nlev
L00277        do ivw=1,2
L00278         surwal(ivw,ilev)=dzlev*bw
L00279        end do
L00280        do ivw=3,4
L00281         surwal(ivw,ilev)=dzlev*bl
L00282        end do
L00283        do ivw=5,6
L00284         surwal(ivw,ilev)=bw*bl
L00285        end do
L00286      end do
L00287
L00288
L00289  ! Calculation of the short wave radiations at the internal walls
L00290  ! ------------------------------------------------------------------
L00291
L00292
L00293      do ilev=1,nlev
L00294
L00295        do ivw=1,4
L00296          rswal1D(ivw)=rswal(ivw,ilev)
L00297        end do
L00298
L00299        do ivw=1,6
L00300          surwal1D(ivw)=surwal(ivw,ilev)
L00301        end do
L00302
L00303        call int_rsrad(albwin,albins,pwin,rswal1D,&
L00304                       surwal1D,bw,bl,dzlev,rsint)
L00305
L00306        do ivw=1,6
L00307          rswalins(ivw,ilev)=rsint(ivw)
L00308        end do
L00309
L00310      end do !ilev
L00311
L00312
L00313
L00314  ! Calculation of the Long wave radiation at the internal walls
L00315  !-------------------------------------------------------------
L00316
L00317
L00318  !Intermediate rooms
L00319
L00320         if (nlev.gt.2) then
L00321      do ilev=2,nlev-1
L00322
L00323        do ivw=1,4
L00324          twin1D(ivw)=twin(ivw,ilev)
L00325          twal_int(ivw)=twal(ivw,1,ilev)
L00326        end do
L00327
L00328         twal_int(5)=tflo(nflo,ilev-1)
L00329         twal_int(6)=tflo(1,ilev)
L00330
L00331         call int_rlrad(emins,emwin,sigma,twal_int,twin1D,&
L00332                    pwin,bw,bl,dzlev,rlint)
L00333
L00334
L00335        do ivw=1,6
L00336          rlwalins(ivw,ilev)=rlint(ivw)
L00337        end do
L00338
L00339      end do  !ilev
L00340        end if
```

```
L00341
L00342
L00343        if (nlev.ne.1) then
L00344
L00345  !bottom room
L00346
L00347        do ivw=1,4
L00348          twin1D(ivw)=twin(ivw,1)
L00349          twal_int(ivw)=twal(ivw,1,1)
L00350        end do
L00351
L00352        twal_int(5)=tgrd(ngrd)
L00353        twal_int(6)=tflo(1,1)
L00354
L00355
L00356         call int_rlrad(emins,emwin,sigma,twal_int,twin1D,&
L00357                    pwin,bw,bl,dzlev,rlint)
L00358
L00359        do ivw=1,6
L00360          rlwalins(ivw,1)=rlint(ivw)
L00361        end do
L00362
L00363  !top room
L00364
L00365        do ivw=1,4
L00366          twin1D(ivw)=twin(ivw,nlev)
L00367          twal_int(ivw)=twal(ivw,1,nlev)
L00368        end do
L00369
L00370        twal_int(5)=tflo(nflo,nlev-1)
L00371        twal_int(6)=trof(1)
L00372
L00373
L00374         call int_rlrad(emins,emwin,sigma,twal_int,twin1D,&
L00375                    pwin,bw,bl,dzlev,rlint)
L00376
L00377        do ivw=1,6
L00378          rlwalins(ivw,nlev)=rlint(ivw)
L00379        end do
L00380
L00381        else   !Top <---> Bottom
L00382
L00383        do ivw=1,4
L00384          twin1D(ivw)=twin(ivw,1)
L00385          twal_int(ivw)=twal(ivw,1,1)
L00386        end do
L00387
L00388        twal_int(5)=tgrd(ngrd)
L00389            twal_int(6)=trof(1)
L00390
L00391        call int_rlrad(emins,emwin,sigma,twal_int,twin1D, &
L00392                   pwin,bw,bl,dzlev,rlint)
L00393
L00394        do ivw=1,6
L00395          rlwalins(ivw,1)=rlint(ivw)
L00396        end do
L00397
L00398        end if
L00399
L00400
L00401  ! Calculation of the radiative fluxes
L00402  ! ------------------------------------
L00403
L00404  !External vertical walls and windows
L00405
L00406          do ilev=1,nlev
L00407       do ivw=1,4
L00408       call radfluxs(radflux,albwal,rswal(ivw,ilev),     &
L00409                             emwal,rlwal(ivw,ilev),sigma,   &
L00410                             twal(ivw,nwal,ilev))
L00411
L00412           hrwalout(ivw,ilev)=radflux
L00413
L00414       hrwinout(ivw,ilev)=emwin*rlwal(ivw,ilev)- &
L00415                             emwin*sigma*(twin(ivw,ilev)**4)
L00416
L00417
L00418       end do ! ivw
L00419      end do  ! ilev
L00420
L00421  !Roof and PV panels
L00422
L00423          if (pv_frac_roof.eq.0.) then
L00424             call radfluxs(radflux,albrof,rs,emrof,rl,sigma,tr_av)
L00425            hrrof=radflux
L00426            else
L00427              call radfluxspv(nzcanm,nlev,albrof,rs,swddif,emrof,rl,tr_av,tout,sigma,radflux,pv_frac_roof,tpv)
L00428              hrrof=radflux
L00429          endif
L00430
L00431  !Internal walls for intermediate rooms
L00432
L00433        if(nlev.gt.2) then
L00434
L00435      do ilev=2,nlev-1
L00436       do ivw=1,4
L00437
L00438       call radfluxs(radflux,albins,rswalins(ivw,ilev),     &
L00439                             emins,rlwalins(ivw,ilev),sigma,   &
L00440                             twal(ivw,1,ilev))
L00441
L00442       hrwalins(ivw,ilev)=radflux
L00443
L00444       end do !ivw
L00445
L00446       call radfluxs(radflux,albins,rswalins(5,ilev), &
L00447                               emins,rlwalins(5,ilev),sigma,&
L00448                               tflo(nflo,ilev-1))
L00449
```

```
L00450           hrwalins(5,ilev)=radflux
L00451
L00452           call radfluxs(radflux,albins,rswalins(6,ilev), &
L00453                         emins,rlwalins(6,ilev),sigma,&
L00454                         tflo(1,ilev))
L00455           hrwalins(6,ilev)=radflux
L00456
L00457         end do !ilev
L00458
L00459        end if
L00460
L00461
L00462  !Internal walls for the bottom and the top room
L00463  !
L00464        if (nlev.ne.1) then
L00465
L00466  !bottom floor
L00467
L00468       do ivw=1,4
L00469
L00470          call radfluxs(radflux,albins,rswalins(ivw,1),  &
L00471                        emins,rlwalins(ivw,1),sigma,   &
L00472                        twal(ivw,1,1))
L00473
L00474             hrwalins(ivw,1)=radflux
L00475
L00476       end do
L00477
L00478
L00479        call radfluxs(radflux,albins,rswalins(5,1),&
L00480                      emins,rlwalins(5,1),sigma,&    !bottom
L00481                      tgrd(ngrd))
L00482
L00483           hrwalins(5,1)=radflux
L00484
L00485
L00486           call radfluxs(radflux,albins,rswalins(6,1),&
L00487                         emins,rlwalins(6,1),sigma,&
L00488                         tflo(1,1))
L00489
L00490           hrwalins(6,1)=radflux
L00491
L00492  !roof floor
L00493
L00494          do ivw=1,4
L00495
L00496           call radfluxs(radflux,albins,rswalins(ivw,nlev),     &
L00497                         emins,rlwalins(ivw,nlev),sigma,&
L00498                         twal(ivw,1,nlev))
L00499
L00500        hrwalins(ivw,nlev)=radflux
L00501
L00502       end do                                          !top
L00503
L00504
L00505          call radfluxs(radflux,albins,rswalins(5,nlev),    &
L00506                        emins,rlwalins(5,nlev),sigma,&
L00507                        tflo(nflo,nlev-1))
L00508
L00509          hrwalins(5,nlev)=radflux
L00510
L00511       call radfluxs(radflux,albins,rswalins(6,nlev), &
L00512                        emins,rlwalins(6,nlev),sigma,&
L00513                        trof(1))
L00514
L00515          hrwalins(6,nlev)=radflux
L00516
L00517        else       ! Top <---> Bottom room
L00518
L00519       do ivw=1,4
L00520
L00521          call radfluxs(radflux,albins,rswalins(ivw,1),&
L00522                        emins,rlwalins(ivw,1),sigma, &
L00523                        twal(ivw,1,1))
L00524
L00525             hrwalins(ivw,1)=radflux
L00526
L00527          end do
L00528
L00529             call radfluxs(radflux,albins,rswalins(5,1),&
L00530                           emins,rlwalins(5,1),sigma,  &
L00531                           tgrd(ngrd))
L00532
L00533             hrwalins(5,1)=radflux
L00534
L00535             call radfluxs(radflux,albins,rswalins(6,nlev),     &
L00536                           emins,rlwalins(6,nlev),sigma,&
L00537                           trof(1))
L00538             hrwalins(6,1)=radflux
L00539
L00540        end if
L00541
L00542
L00543  !Windows
L00544
L00545       do ilev=1,nlev
L00546        do ivw=1,4
L00547           hrwinins(ivw,ilev)=emwin*rlwalins(ivw,ilev)-    &
L00548                              emwin*sigma*(twin(ivw,ilev)**4)
L00549        end do
L00550       end do
L00551
L00552
L00553  ! Calculation of the sensible heat fluxes
L00554  ! ---------------------------------------
L00555
L00556  !Vertical fluxes for walls
L00557
L00558      do ilev=1,nlev
```

```
L00559           do ivw=1,4
L00560
L00561                 call hsinsflux (2,2,tlev(ilev),twal(ivw,1,ilev),hs)
L00562
L00563                 hswalins(ivw,ilev)=hs
L00564
L00565           end do ! ivw
L00566          end do ! ilev
L00567
L00568
L00569  !Vertical fluxes for windows
L00570
L00571      do ilev=1,nlev
L00572
↪
L00573           do ivw=1,4
L00574
L00575             call hsinsflux (2,1,tlev(ilev),twin(ivw,ilev),hs)
L00576
L00577                 hswinins(ivw,ilev)=hs
L00578
L00579           end do ! ivw
L00580
L00581      end do !ilev
L00582
L00583  !Horizontal fluxes
L00584
L00585        if (nlev.gt.2) then
L00586
L00587          do ilev=2,nlev-1
L00588
L00589             call hsinsflux (1,2,tlev(ilev),tflo(nflo,ilev-1),hs)
L00590
L00591             hswalins(5,ilev)=hs
L00592
L00593             call hsinsflux (1,2,tlev(ilev),tflo(1,ilev),hs)
L00594
L00595             hswalins(6,ilev)=hs
L00596
L00597          end do ! ilev
L00598
L00599        end if
L00600
L00601        if (nlev.ne.1) then
L00602
L00603                 call hsinsflux (1,2,tlev(1),tgrd(ngrd),hs)
L00604
L00605          hswalins(5,1)=hs                !Bottom room
L00606
L00607          call hsinsflux (1,2,tlev(1),tflo(1,1),hs)
L00608
L00609          hswalins(6,1)=hs
L00610
L00611                 call hsinsflux (1,2,tlev(nlev),tflo(nflo,nlev-1),hs)
L00612
L00613          hswalins(5,nlev)=hs                !Top room
L00614
L00615          call hsinsflux (1,2,tlev(nlev),trof(1),hs)
L00616
L00617          hswalins(6,nlev)=hs
L00618                 sfr_indoor= hswalins(6,nlev)
L00619        else  ! Bottom<--->Top
L00620
L00621                 call hsinsflux (1,2,tlev(1),tgrd(ngrd),hs)
L00622
L00623          hswalins(5,1)=hs
L00624
L00625          call hsinsflux (1,2,tlev(nlev),trof(1),hs)
L00626
L00627          hswalins(6,nlev)=hs
L00628
L00629        end if
L00630  !!
L00631  !! Calculation of the sensible heat fluxes from the PV panels & electricity producti
L00632
L00633
L00634        if(pv_frac_roof.gt.0)then
L00635        call hsfluxpv(nzcanm,nlev,bl,bw,albrof,rs,swddif,emrof,rl,tr_av,tout,sigma,hspv,eppv,pv_frac_roof,uout,vout,tpv,
↪  dt)
L00636        sfpv=hspv
L00637        tpv_print=tpv
L00638        endif
L00639
L00640  !Calculation of the temperature for the different surfaces
L00641  ! ------------------------------------------------------------
L00642
L00643  ! Vertical walls
L00644
L00645         swwal=1
L00646         do ilev=1,nlev
L00647          do ivw=1,4
L00648
L00649         htot(1)=hswalins(ivw,ilev)+hrwalins(ivw,ilev)
L00650             htot(2)=hswalout(ivw,ilev)+hrwalout(ivw,ilev)
L00651             gswal(ivw,ilev)=htot(2)
L00652
L00653         do iwal=1,nwal
L00654            twal1D(iwal)=twal(ivw,iwal,ilev)
L00655         end do
L00656
L00657         call wall(swwal,nwal,dt,dzwal,kwal,cswal,htot,twal1D)
L00658
L00659         do iwal=1,nwal
L00660            twal(ivw,iwal,ilev)=twal1D(iwal)
L00661         end do
L00662
L00663      end do ! ivw
L00664         end do ! ilev
L00665
L00666  ! Windows
```

```
L00667
L00668          do ilev=1,nlev
L00669           do ivw=1,4
L00670
L00671            htot(1)=hswinins(ivw,ilev)+hrwinins(ivw,ilev)
L00672            htot(2)=hswinout(ivw,ilev)+hrwinout(ivw,ilev)
L00673
L00674            twin(ivw,ilev)=twin(ivw,ilev)+(dt/(cswin*thickwin))* &
L00675                           (htot(1)+htot(2))
L00676
L00677       end do ! ivw
L00678          end do ! ilev
L00679
L00680    ! Horizontal floors
L00681
L00682
L00683          if (nlev.gt.1) then
L00684           swwal=1
L00685           do ilev=1,nlev-1
L00686
L00687              htot(1)=hrwalins(6,ilev)+hswalins(6,ilev)
L00688              htot(2)=hrwalins(5,ilev+1)+hswalins(5,ilev+1)
L00689
L00690          do iflo=1,nflo
L00691             tflo1D(iflo)=tflo(iflo,ilev)
L00692          end do
L00693
L00694          call wall(swwal,nflo,dt,dzflo,kflo,csflo,htot,tflo1D)
L00695
L00696         do iflo=1,nflo
L00697            tflo(iflo,ilev)=tflo1D(iflo)
L00698         end do
L00699
L00700           end do ! ilev
L00701          end if
L00702
L00703
L00704    ! Ground
L00705
L00706       swwal=1
L00707
L00708       htot(1)=0.  !Diriclet b.c. at the internal boundary
L00709       htot(2)=hswalins(5,1)+hrwalins(5,1)
L00710
L00711           do igrd=1,ngrd
L00712              tgrd1D(igrd)=tgrd(igrd)
L00713           end do
L00714
L00715            call wall(swwal,ngrd,dt,dzgrd,kgrd,csgrd,htot,tgrd1D)
L00716
L00717           do igrd=1,ngrd
L00718              tgrd(igrd)=tgrd1D(igrd)
L00719           end do
L00720
L00721
L00722    ! Roof
L00723
L00724          swwal=1
L00725
L00726          htot(1)=hswalins(6,nlev)+hrwalins(6,nlev)
L00727          htot(2)=hsrof+hrrof+lsrof
L00728          gsrof=htot(2)
L00729
L00730          do irof=1,nrof
L00731             trof1D(irof)=trof(irof)
L00732
L00733          end do
L00734
L00735          if(gr_flag.eq.1)then
L00736          call wall_gr(hfgr,gr_frac_roof,swwal,nrof,dt,dzrof,krof,csrof,htot,trof1D)
L00737          else
L00738           call wall(swwal,nrof,dt,dzrof,krof,csrof,htot,trof1D)
L00739          endif
L00740          do irof=1,nrof
L00741             trof(irof)=trof1D(irof)
L00742
L00743           end do
L00744
L00745
L00746    ! Calculation of the heat fluxes and of the temperature of the rooms
L00747    ! ---------------------------------------------------------------------
L00748
L00749       do ilev=1,nlev
L00750
L00751        !Calculation of the heat generated by equipments and occupants
L00752
L00753        call fluxeqocc(nhourday,bw,bl,perflo,hsesf,hsequip,hseqocc(ilev),hleqocc(ilev))
L00754
L00755            !Calculation of the heat generated by natural ventilation
L00756
L00757          vollev=bw*bl*dzlev
L00758              humdry=shumlev(ilev)/(1.-shumlev(ilev))
L00759          rhoint=(press(ilev))/(rair*(1.+0.61*humdry)*tlev(ilev))
L00760          cpint=cp*(1.+0.84*humdry)
L00761
L00762
L00763          call fluxvent(cpint,rhoint,vollev,tlev(ilev),tout(ilev),     &
L00764                        latent,humout(ilev),rhoout(ilev),shumlev(ilev),&
L00765                        beta,hsvent(ilev),hlvent(ilev))
L00766
L00767
L00768            !Calculation of the heat generated by conduction
L00769
L00770           do iwal=1,6
L00771             hswalins1D(iwal)=hswalins(iwal,ilev)
L00772             surwal1D(iwal)=surwal(iwal,ilev)
L00773          end do
L00774
L00775           do iwal=1,4
L00776             hswinins1D(iwal)=hswinins(iwal,ilev)
```

```
L00777         end do
L00778
L00779        call fluxcond(hswalins1D,hswinins1D,surwal1D,pwin,&
L00780                      hscond(ilev))
L00781
L00782      !Calculation of the heat generated inside the room
L00783
L00784        call fluxroo(hseqocc(ilev),hleqocc(ilev),hsvent(ilev), &
L00785                hlvent(ilev),hscond(ilev),hslev(ilev),hllev(ilev))
L00786
L00787
L00788      !Evolution of the temperature and of the specific humidity
L00789
L00790        Qb=rhoint*cpint*vollev
L00791
L00792          ! temperature regulation
L00793
L00794            call regtemp(sw_cond,nhourday,dt,Qb,hslev(ilev),       &
L00795                         tlev(ilev),timeon,timeoff,targtemp,gaptemp,hsneed(ilev))
L00796
L00797          ! humidity regulation
L00798
L00799        call reghum(sw_cond,nhourday,dt,vollev,rhoint,latent, &
L00800                    hllev(ilev),shumlev(ilev),timeon,timeoff,&
L00801                    targhum,gaphum,hlneed(ilev))
L00802   !
L00803   !performance of the air conditioning system for the test
L00804   !
L00805
L00806            call air_cond(hsneed(ilev),hlneed(ilev),dt, &
L00807                          hsout(ilev),hlout(ilev),consump(ilev), cop)
L00808
L00809        tlev(ilev)=tlev(ilev)+(dt/Qb)*(hslev(ilev)-hsneed(ilev))
L00810
L00811        shumlev(ilev)=shumlev(ilev)+(dt/(vollev*rhoint*latent))* &
L00812                      (hllev(ilev)-hlneed(ilev))
L00813
L00814      end do !ilev
L00815
L00816          call consump_total(nzcanm,nlev,consumpbuild,hsoutbuild, &
L00817                             hsout,consump)
L00818
L00819        return
L00820        end subroutine BEM
```

讲解：八步流水线（每房间循环外只做一次的段 + 每房间一段的段）：① **表面积矩阵** surwal(6,ilev)——六面几何；② **室内短波** int_rsrad：窗外辐射透射 transmit=1−albwin 按 pwin 加权进室内，各向同性撒到六面，再解 6×6 多次反射系统（algebra_short）；③ **室内长波** int_rlrad：六面 T^4 发射 +反射 6×6 系统（algebra_long）；底房间面 5 换成楼下地基、顶房间面 6 换成屋顶——室内辐射「看得见」建筑外几何；④ **外表面辐射收支** radfluxs（=(1−alb)rs+em • rl−em • σT^4），屋顶有 PV 时改用 radfluxspv（PV 面板与屋顶间的辐射交换 hrad 串联式）；⑤ **室内对流** hsinsflux：Clarke 建筑热工手册系数（竖墙 5.678 • 1.09、窗 5.678 • 0.99 W/(m² • K)，光滑/粗糙面区分）；⑥ **围护结构导热** wall/wall_gr：墙、楼板、楼下地基、屋顶四类多层材料各自隐式三对角推进（上边界 =内外两侧热流和 htot，热流进 temp 的边界行；swwal 开关使 wall_coeff 系数只算一次——save 缓存，材料性质不随时间变）；窗户按 dt/(cswin • thickwin) 集总热容更新；绿屋顶时屋顶用 wall_gr（第 5 层注入 hfgr 绿屋顶耦合热流并按 gr_frac_roof 缩放——与 14a roof_temp_veg 的 hfgr 对接）；⑦ **房间空气收支**：fluxeqocc（人员 160 W/人显热 + 1.96e6/86400 W/人潜热 ×perflo×rocc + 设备 hsequip 24h 谱 ×hsesf）→ fluxvent（自然通风：每小时一次全换气 vollev/3600，换热器效率 beta 打折）→ fluxcond（六面对流进风的合计，符号翻转对齐 Twall−Tindoor）→ fluxroo 求和 → **regtemp/reghum 恒温恒湿调节**（开关 sw_cond、时间窗 timeon/off、目标值 ± 死区 gaptemp/gaphum、越界后算 hsneed 并用 temp_rat/hum_rat 限速）→ **air_cond**（COP 换算：制冷时废热 hsout=(1+1/cop) • |hsneed|+…——注意注释里备选式被注释掉的历史；制热时 hsout=0）→ 室内状态显式推进 tlev+=(dt/Qb)(hslev−hsneed)、shumlev+=(dt/(vol • ρ • L))(hllev−hlneed)；⑧consump_total 全楼汇总（耗电 kW • h/s）。**Qb=ρ_int • cp_int • vol 是室内空气总热容——「室内空气」是零惯量近似之外的唯一惯性，房间响应快于此即失真。**

五、PV 单元

```
!file: phys/module_sf_bem.F    L823-L905    (subroutine hsfluxpv)
L00823        subroutine hsfluxpv(nz,n,bl,bw,albr,rs,swddif,emr,rl,tr,tair,sigma,hspv,eppv,pv_frac_roof,uout,vout,tpv,dt)
L00824   !
L00825        implicit none
L00826   !
L00827   ! Input variables
L00828   !
L00829        integer,intent(in) :: nz     !Maximum number of vertical levels in the urban grid
L00830        real,intent(in) :: bl     !Building length [m]
L00831        real,intent(in) :: bw           !Building width [m]
L00832        real,intent(in) :: albr   !albedo of the roof (ext.)
L00833        real,intent(in) :: emr   !emissivity of the roof (ext.)
L00834        real,intent(in) :: rs    !external short wave radiation [W/m2]
L00835        real,intent(in) :: rl !external long wave radiation [W/m2]
L00836        real,intent(in) :: tr !roof surface temperature [K]
L00837        real,intent(in) :: pv_frac_roof ! fraction of PV  []
L00838        real,intent(in) :: sigma !Stefan-Boltzmann constant [W/m2.K4]
```

```
L00839        real,intent(in),dimension(1:nz) :: tair  !external temperature [K]
L00840        integer,intent(in) :: n  !number of floors in the building
L00841        real,intent(in), dimension(1:nz) :: uout
L00842        real,intent(in), dimension(1:nz) :: vout
L00843        real,intent(in)  :: dt
L00844        real,intent(in)  :: swddif
L00845  ! Output variables
L00846  !
L00847        real,intent(inout) :: hspv ! Sensible heat flux from the PV panels to the atmosphere [W/m2]
L00848        real,intent(inout) :: eppv ! Electricity production from PV panels [W]
L00849        real,intent(inout) :: tpv       !Temperature of the PV panels [K]
L00850  !
L00851  ! Local variables
L00852  !
L00853        real,parameter :: albpv=0.11 ! albedo of the PV panels
L00854        real,parameter :: empv_down=0.95  ! emissivity of the PV panels
L00855        real,parameter :: empv_up=0.79
L00856        real, parameter :: T_amb=25
L00857        real, parameter :: tiltangle=0.
L00858        real, parameter :: a=3.8
L00859        real, parameter :: b=6.9
L00860
L00861        real, parameter :: r1=2330.
L00862        real, parameter :: r2=1200.
L00863        real, parameter :: r3=3000.
L00864        real, parameter :: c1=677.
L00865        real, parameter :: c2=1250.
L00866        real, parameter :: c3=500.
L00867        real, parameter :: d1=0.0003
L00868        real, parameter :: d2=0.0005
L00869        real, parameter :: d3=0.003
L00870        real, parameter :: F12=1.
L00871        real :: lwuppv     !Long-wave emitted by the PV panels to the sky [W/m2]
L00872        real :: lwdwr      !Long-wave incoming radiation on the roof [W/m2]
L00873        real :: lwupr      !Long-wave coming up from the roof intercepted by the PV panels [W/m2]
L00874        real :: enerpv     !Energy produced by PV panels [W/m2]
L00875        real :: hc
L00876        real :: sw_d
L00877        real :: lw_d
L00878        real :: lwpv_out
L00879        real :: tpv_new
L00880        real :: hdown
L00881        real :: hup
L00882        real :: deltat
L00883        real :: uroof
L00884        real :: hrad
L00885        real :: Cm
L00886        real :: hf
L00887         Cm=r1*c1*d1+r2*c2*d2+r3*c3*d3
L00888         hrad=sigma/((1-empv_down)/empv_down+1/F12+(1-emr)/emr)
L00889         uroof=(uout(n+1)**2+vout(n+1)**2)**0.5
L00890        deltat=tpv-tair(n+1)
L00891      hf=2.5*(40./100.*uroof)**(0.5)
L00892      hc=9.482*abs(deltat)**(1./3.)/(7.238-abs(cos(tiltangle)))
L00893      hup=sqrt(hc**2.+(hf)**2.)
L00894      hc=1.810*abs(deltat)**(1./3.)/(1.382+abs(cos(tiltangle)))
L00895      hdown=sqrt(hc**2.+(hf)**2.)
L00896      enerpv=effpv*rs*min(1.,1.-0.005*(tpv-(T_amb+273.15)))
L00897      sw_d=(1-albpv)*(rs)
L00898      lw_d=empv_up*rl
L00899      lwpv_out=empv_up*sigma*tpv**4.
L00900      lwupr=hrad*(tr**4-tpv**4.)
L00901      hspv=(hup+hdown)*(tpv-tair(n+1))
L00902      tpv=tpv+(sw_d+lw_d-lwpv_out+lwupr-hspv-enerpv)/Cm*dt
L00903      eppv=enerpv*pv_frac_roof*bl*bw
L00904       return
L00905        end subroutine hsfluxpv
```

讲解：屋顶光伏板能量平衡：三层材料集热容 Cm（r/c/d 三组参数 =背板/电池/玻璃）；对空对流 hf=2.5 • (0.4 • V)^0.5 与自然对流 hc 分上下两面（倾角 0 的平板公式）；与屋顶的辐射交换 hrad=σ/((1−ε_pv)/ε_pv+1/F12+(1−ε_r)/ε_r)（两面灰体 +视角因子的串联发射率）；发电 enerpv=effpv • rs • min(1, 1−0.005 • (T_pv−298.15))——**温度每升 1°C 效率降 0.5%**；板温显式推进。eppv=enerpv • pv_frac_roof • bl • bw。albpv=0.11/empv_up=0.79/empv_down=0.95 与 14b upward_rad 的常数对齐（两处必须一致）。

六、导热求解器族

```
!file: phys/module_sf_bem.F    L909-L1033    (subroutine wall_gr)
L00909      subroutine wall_gr(hfgr,gr_frac_roof,swwall,nz,dt,dz,k,cs,flux,temp)
L00910
L00911  !_____________________________________________________________________
L00912
L00913  !The aim of this subroutine is to solve the 1D heat fiffusion equation
L00914  !for roof, walls and streets:
L00915  !
L00916  !   dT/dt=d/dz[K*dT/dz] where:
L00917  !
L00918  !   -T is the surface temperature(wall, street, roof)
L00919  !       -Kz is the heat diffusivity inside the material.
L00920  !
L00921  !The resolution is done implicitly with a FV discretisation along the
L00922  !different layers of the material:
L00923
L00924  !   _____________________
L00925  !     n              *
L00926  !                    *
L00927  !                    *
L00928  !       ____________________________
```

```
!    i+2
!                   I+1
!   ____________________________
!    i+1
!                    I               ==>   [T(I,n+1)-T(I,n)]/DT=
!   ____________________________             [F(i+1)-F(i)]/DZI
!    i
!                   I-1              ==> A*T(n+1)=B where:
!   ____________________________
!   i-1              *                  * A is a TRIDIAGONAL matrix.
!                    *                  * B=T(n)+S takes into account the sources.
!                    *
!     1 _____________________________

!_____________________________________________________________

    implicit none

!Input:
!-----
            real hfgr        !Green roof heat flux
            real gr_frac_roof     !Green roof fraction

    integer nz      !Number of layers inside the material
    real dt         !Time step
    real dz(nz)     !Layer sizes [m]
    real cs(nz)     !Specific heat of the material [J/(m3.K)]
    real k(nz+1)        !Thermal conductivity in each layers (face) [W/(m.K)]
    real flux(2)        !Internal and external flux terms.


!Input-Output:
!-------------

    integer swwall          !swich for the physical coefficients calculation
    real temp(nz)       !Temperature at each layer

!Local:
!-----

       real a(-1:1,nz)          !  a(-1,*) lower diagonal      A(i,i-1)
                                !  a(0,*)  principal diagonal  A(i,i)
                                !  a(1,*)  upper diagonal      A(i,i+1).

       real b(nz)           !Coefficients of the second term.
       real k1(20)
       real k2(20)
       real kc(20)
       save k1,k2,kc
       integer iz

!_____________________________________________________________
!
!Calculation of the coefficients

    if (swwall.eq.1) then

          if (nz.gt.20) then
             write(*,*) 'number of layers in the walls/roofs too big ',nz
             write(*,*) 'please decrease under of',20
             stop
          endif

        call wall_coeff(nz,dt,dz,cs,k,k1,k2,kc)
        swwall=0

    end if

!Computation of the first value (iz=1) of A and B:

         a(-1,1)=0.
         a(0,1)=1+k2(1)
         a(1,1)=-k2(1)
                b(1)=temp(1)+flux(1)*kc(1)

    do iz=2,nz-1
                a(-1,iz)=-k1(iz)
                if(iz.eq.5)then
                a(-1,iz)=-k1(iz)
                a(0,iz)=1+k1(iz)+(1-gr_frac_roof)*k2(iz)
                b(iz)=temp(iz)+(gr_frac_roof*hfgr*dt)/dz(iz)
                a(1,iz)=-k2(iz)*(1-gr_frac_roof)
                else
                a(0,iz)=1.+k1(iz)+k2(iz)
                b(iz)=temp(iz)
                a(1,iz)=-k2(iz)
                endif


    end do

!Computation of the external value (iz=n) of A and B:

        a(-1,nz)=-k1(nz)
        a(0,nz)=1.+k1(nz)
        a(1,nz)=0.
                b(nz)=temp(nz)+kc(nz)*flux(2)

!Resolution of the system A*T(n+1)=B

    call tridia(nz,a,b,temp)


        return
    end  subroutine wall_gr
```

```
!file: phys/module_sf_bem.F    L1039-L1151    (subroutine wall)
L01039      subroutine wall(swwall,nz,dt,dz,k,cs,flux,temp)
L01040
L01041  !_____________________________________________________________________
L01042
L01043  !The aim of this subroutine is to solve the 1D heat fiffusion equation
L01044  !for roof, walls and streets:
L01045  !
L01046  !   dT/dt=d/dz[K*dT/dz] where:
L01047  !
L01048  !   -T is the surface temperature(wall, street, roof)
L01049  !       -Kz is the heat diffusivity inside the material.
L01050  !
L01051  !The resolution is done implicitly with a FV discretisation along the
L01052  !different layers of the material:
L01053
L01054  !   _____________________________
L01055  !     n             *
L01056  !                   *
L01057  !                   *
L01058  !    _______________________________
L01059  !    i+2
L01060  !                  I+1
L01061  !   _______________________________
L01062  !    i+1
L01063  !                   I               ==>   [T(I,n+1)-T(I,n)]/DT=
L01064  !   _______________________________        [F(i+1)-F(i)]/DZI
L01065  !    i
L01066  !                  I-1              ==> A*T(n+1)=B where:
L01067  !
L01068  !   i-1             *                   * A is a TRIDIAGONAL matrix.
L01069  !                   *                   * B=T(n)+S takes into account the sources.
L01070  !                   *
L01071  !     1 _____________________________
L01072
L01073  !_____________________________________________________________________
L01074
L01075      implicit none
L01076
L01077  !Input:
L01078  !-----
L01079      integer nz      !Number of layers inside the material
L01080      real dt         !Time step
L01081      real dz(nz)     !Layer sizes [m]
L01082      real cs(nz)     !Specific heat of the material [J/(m3.K)]
L01083      real k(nz+1)        !Thermal conductivity in each layers (face) [W/(m.K)]
L01084      real flux(2)        !Internal and external flux terms.
L01085
L01086
L01087  !Input-Output:
L01088  !-------------
L01089
L01090      integer swwall          !swich for the physical coefficients calculation
L01091      real temp(nz)       !Temperature at each layer
L01092
L01093  !Local:
L01094  !-----
L01095
L01096        real a(-1:1,nz)           !  a(-1,*) lower diagonal      A(i,i-1)
L01097                                  !  a(0,*)  principal diagonal  A(i,i)
L01098                                  !  a(1,*)  upper diagonal      A(i,i+1).
L01099
L01100        real b(nz)            !Coefficients of the second term.
L01101        real k1(20)
L01102        real k2(20)
L01103        real kc(20)
L01104        save k1,k2,kc
L01105        integer iz
L01106
L01107  !_____________________________________________________________________
L01108  !
L01109  !Calculation of the coefficients
L01110
L01111      if (swwall.eq.1) then
L01112
L01113            if (nz.gt.20) then
L01114               write(*,*) 'number of layers in the walls/roofs too big ',nz
L01115               write(*,*) 'please decrease under of',20
L01116               stop
L01117            endif
L01118
L01119         call wall_coeff(nz,dt,dz,cs,k,k1,k2,kc)
L01120         swwall=0
L01121
L01122      end if
L01123
L01124  !Computation of the first value (iz=1) of A and B:
L01125
L01126           a(-1,1)=0.
L01127           a(0,1)=1+k2(1)
L01128           a(1,1)=-k2(1)
L01129                    b(1)=temp(1)+flux(1)*kc(1)
L01130
L01131      do iz=2,nz-1
L01132                 a(-1,iz)=-k1(iz)
L01133                 a(0,iz)=1+k1(iz)+k2(iz)
L01134                 b(iz)=temp(iz)
L01135                 a(1,iz)=-k2(iz)
L01136      end do
L01137
L01138  !Computation of the external value (iz=n) of A and B:
L01139
L01140          a(-1,nz)=-k1(nz)
L01141          a(0,nz)=1+k1(nz)
L01142          a(1,nz)=0.
L01143          b(nz)=temp(nz)+flux(2)*kc(nz)
L01144
L01145  !Resolution of the system A*T(n+1)=B
L01146
```

```
L01147      call tridia(nz,a,b,temp)
L01148
L01149
L01150          return
L01151      end  subroutine wall
```

```
!file: phys/module_sf_bem.F    L1156-L1206    (subroutine wall_coeff)
L01156      subroutine wall_coeff(nz,dt,dz,cs,k,k1,k2,kc)
L01157
L01158      implicit none
L01159
L01160  !-----------------------------------------------------------------------
L01161  !Input
L01162  !-----
L01163      integer nz      !Number of layers inside the material
L01164      real dt         !Time step
L01165      real dz(nz)     !Layer sizes [m]
L01166      real cs(nz)     !Specific heat of the material [J/(m3.K)]
L01167      real k(nz+1)        !Thermal diffusivity in each layers (face) [W/(m.K)]
L01168
L01169
L01170  !Input-Output
L01171  !------------
L01172
L01173      real flux(2)        !Internal and external flux terms.
L01174
L01175
L01176  !Output
L01177  !------
L01178          real k1(20)
L01179          real k2(20)
L01180          real kc(20)
L01181
L01182  !Local
L01183  !-----
L01184      integer iz
L01185      real kf(nz)
L01186
L01187  !-----------------------------------------------------------------------
L01188
L01189      do iz=2,nz
L01190       kc(iz)=dt/(dz(iz)*cs(iz))
L01191       kf(iz)=2*k(iz)/(dz(iz)+dz(iz-1))
L01192      end do
L01193
L01194      kc(1)=dt/(dz(1)*cs(1))
L01195          kf(1)=2*k(1)/(dz(1))
L01196
L01197      do iz=1,nz
L01198       k1(iz)=kc(iz)*kf(iz)
L01199      end do
L01200
L01201      do iz=1,nz-1
L01202       k2(iz)=kc(iz)*kf(iz+1)*cs(iz)/cs(iz+1)
L01203      end do
L01204
L01205      return
L01206      end subroutine wall_coeff
```

讲解：同一套隐式有限体积导热（dT/dt=d/dz(K • dT/dz)）的两个变体 + 系数预计算：wall_coeff 把 dt/(dz • cs) 与层间调和导热 2k/(dz_iz+dz_iz−1) 预乘成 k1/k2/kc（**swwal 开关 +save 缓存只算一次**——若论文 2 把材料参数场化，此缓存必须作废或改为逐点缓存，这是标量 → 场改造的第一个性能陷阱）；wall 是标准版（内边界 flux(1)、外边界 flux(2) 的 Neumann 条件，底层行 b(nz)=temp+kc • flux(2)）；wall_gr 在第 5 层注入绿屋顶耦合热流 hfgr • gr_frac_roof • dt/dz 并把该层向外系数乘 (1−gr_frac_roof)。nz≤20 硬上限。

```
!file: phys/module_sf_bem.F    L2366-L2410    (subroutine tridia)
L02366          subroutine tridia(n,a,b,x)
L02367
L02368  !     ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
L02369  !     +    by A. Clappier,     EPFL, CH 1015 Lausanne                  +
L02370  !     +                        phone: ++41-(0)21-693-61-60             +
L02371  !     +                        email:alain.clappier@epfl.ch            +
L02372  !     ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
L02373
L02374  ! ----------------------------------------------------------------------
L02375  !        Resolution of a * x = b    where a is a tridiagonal matrix
L02376  !
L02377  ! ----------------------------------------------------------------------
L02378
L02379          implicit none
L02380
L02381  ! Input
L02382          integer n
L02383          real a(-1:1,n)          !  a(-1,*) lower diagonal      A(i,i-1)
L02384                                  !  a(0,*)  principal diagonal  A(i,i)
L02385                                  !  a(1,*)  upper diagonal      A(i,i+1)
L02386          real b(n)
L02387
L02388  ! Output
L02389          real x(n)
L02390
L02391  ! Local
L02392          integer i
L02393
L02394  ! ----------------------------------------------------------------------
L02395
L02396          do i=n-1,1,-1
L02397             b(i)=b(i)-a(1,i)*b(i+1)/a(0,i+1)
L02398             a(0,i)=a(0,i)-a(1,i)*a(-1,i+1)/a(0,i+1)
L02399          enddo
```

```
L02400
L02401          do i=2,n
L02402             b(i)=b(i)-a(-1,i)*b(i-1)/a(0,i-1)
L02403          enddo
L02404
L02405          do i=1,n
L02406             x(i)=b(i)/a(0,i)
L02407          enddo
L02408
L02409          return
L02410          end subroutine tridia
```

```
!file: phys/module_sf_bem.F    L2414-L2516    (subroutine gaussjbem)
L02414         subroutine gaussjbem(a,n,b,np)
L02415
L02416  ! ------------------------------------------------------------------------
L02417  ! This routine solve a linear system of n equations of the form
L02418  !              A X = B
L02419  !  where  A is a matrix a(i,j)
L02420  !         B a vector and X the solution
L02421  ! In output b is replaced by the solution
L02422  ! ------------------------------------------------------------------------
L02423
L02424         implicit none
L02425
L02426  ! ------------------------------------------------------------------------
L02427  ! INPUT:
L02428  ! ------------------------------------------------------------------------
L02429         integer np
L02430         real a(np,np)
L02431
L02432  ! ------------------------------------------------------------------------
L02433  ! OUTPUT:
L02434  ! ------------------------------------------------------------------------
L02435         real b(np)
L02436
L02437  ! ------------------------------------------------------------------------
L02438  ! LOCAL:
L02439  ! ------------------------------------------------------------------------
L02440        integer nmax
L02441        parameter (nmax=150)
L02442
L02443        real big,dum
L02444        integer i,icol,irow
L02445        integer j,k,l,ll,n
L02446        integer ipiv(nmax)
L02447        real pivinv
L02448
L02449  ! ------------------------------------------------------------------------
L02450  ! END VARIABLES DEFINITIONS
L02451  ! ------------------------------------------------------------------------
L02452
L02453         do j=1,n
L02454            ipiv(j)=0.
L02455         enddo
L02456
L02457        do i=1,n
L02458           big=0.
L02459           do j=1,n
L02460              if(ipiv(j).ne.1)then
L02461                 do k=1,n
L02462                    if(ipiv(k).eq.0)then
L02463                       if(abs(a(j,k)).ge.big)then
L02464                          big=abs(a(j,k))
L02465                          irow=j
L02466                          icol=k
L02467                       endif
L02468                    elseif(ipiv(k).gt.1)then
L02469                       FATAL_ERROR('singular matrix in gaussjbem')
L02470                    endif
L02471                 enddo
L02472              endif
L02473           enddo
L02474
L02475           ipiv(icol)=ipiv(icol)+1
L02476
L02477           if(irow.ne.icol)then
L02478              do l=1,n
L02479                 dum=a(irow,l)
L02480                 a(irow,l)=a(icol,l)
L02481                 a(icol,l)=dum
L02482              enddo
L02483
L02484              dum=b(irow)
L02485              b(irow)=b(icol)
L02486              b(icol)=dum
L02487
L02488           endif
L02489
L02490           if(a(icol,icol).eq.0) FATAL_ERROR('singular matrix in gaussjbem')
L02491
L02492           pivinv=1./a(icol,icol)
L02493           a(icol,icol)=1
L02494
L02495           do l=1,n
L02496              a(icol,l)=a(icol,l)*pivinv
L02497           enddo
L02498
L02499           b(icol)=b(icol)*pivinv
L02500
L02501           do ll=1,n
L02502              if(ll.ne.icol)then
L02503                 dum=a(ll,icol)
L02504                 a(ll,icol)=0.
L02505                 do l=1,n
L02506                    a(ll,l)=a(ll,l)-a(icol,l)*dum
L02507                 enddo
```

```
L02508
L02509                b(ll)=b(ll)-b(icol)*dum
L02510
L02511             endif
L02512          enddo
L02513       enddo
L02514
L02515       return
L02516       end subroutine gaussjbem
```

讲解：tridia=A. Clappier 的三对角解器（a(−1:1,n) 存储风格，与 14b invert 同族但签名不同）；gaussjbem=Numerical Recipes gaussj 的 6×6 室内版（解 int_rsrad/int_rlrad 两次反射系统）。

七、室内辐射与对流

```
!file: phys/module_sf_bem.F    L1261-L1324    (subroutine int_rsrad)
L01261      subroutine int_rsrad(albwin,albwal,pwin,rswal,&
L01262                           surwal,bw,bl,zw,rsint)
L01263
L01264  ! ------------------------------------------------------------------
L01265      implicit none
L01266  ! ------------------------------------------------------------------
L01267
L01268  !Input
L01269  !-----
L01270      real albwin     !albedo of the windows
L01271      real albwal     !albedo of the internal wall
L01272      real rswal(4)       !incoming short wave radiation [W/m2]
L01273           real surwal(6)      !surface of the indoor walls [m2]
L01274      real bw,bl      !width of the walls [m]
L01275      real zw         !height of the wall [m]
L01276      real pwin               !window proportion
L01277
L01278  !Output
L01279  !------
L01280      real rsint(6)       !internal walls short wave radiation [W/m2]
L01281
L01282  !Local
L01283  !-----
L01284      real transmit   !transmittance of the direct/diffused radiation
L01285           real rstr   !solar radiation transmitted through the windows
L01286           real surtotwal  !total indoor surface of the walls in the room
L01287      integer iw
L01288      real b(6)   !second member for the system
L01289      real a(6,6) !matrix for the system
L01290
L01291  !------------------------------------------------------------------
L01292
L01293
L01294  ! Calculation of the solar radiation transmitted through windows
L01295
L01296           rstr = 0.
L01297           do iw=1,4
L01298              transmit=1.-albwin
L01299              rstr = rstr+(surwal(iw)*pwin)*(transmit*rswal(iw))
L01300           enddo
L01301
L01302  !We suppose that the radiation is spread isotropically within the
L01303  !room when it passes through the windows, so the flux [W/m2] in every
L01304  !wall is:
L01305
L01306           surtotwal=0.
L01307           do iw=1,6
L01308              surtotwal=surtotwal+surwal(iw)
L01309           enddo
L01310
L01311           rstr=rstr/surtotwal
L01312
L01313  !Computation of the short wave radiation reaching the internal walls
L01314
L01315        call algebra_short(rstr,albwal,albwin,bw,bl,zw,pwin,a,b)
L01316
L01317        call gaussjbem(a,6,b,6)
L01318
L01319           do iw=1,6
L01320              rsint(iw)=b(iw)
L01321           enddo
L01322
L01323        return
L01324        end subroutine int_rsrad
```

```
!file: phys/module_sf_bem.F    L1329-L1374    (subroutine int_rlrad)
L01329      subroutine int_rlrad(emwal,emwin,sigma,twal_int,twin,&
L01330                           pwin,bw,bl,zw,rlint)
L01331
L01332  ! ------------------------------------------------------------------
L01333      implicit none
L01334  ! ------------------------------------------------------------------
L01335
L01336  !Input
L01337  !-----
L01338
L01339      real emwal  !emissivity of the internal walls
L01340      real emwin  !emissivity of the window
L01341      real sigma  !Stefan-Boltzmann constant [W/m2.K4]
L01342      real twal_int(6)!temperature of the first internal layers of a room [K]
L01343      real twin(4)    !temperature of the windows [K]
L01344      real bw     !width of the wall
L01345      real bl     !length of the wall
L01346      real zw     !height of the wall
L01347      real pwin       !window proportion
```

```
L01348
L01349  !Output
L01350  !------
L01351
L01352      real rlint(6)   !internal walls long wave radiation [W/m2]
L01353
L01354  !Local
L01355  !------
L01356
L01357      real b(6)   !second member vector for the system
L01358      real a(6,6) !matrix for the system
L01359          integer iw
L01360  !---------------------------------------------------------------------
L01361
L01362  !Compute the long wave radiation reachs the internal walls
L01363
L01364      call algebra_long(emwal,emwin,sigma,twal_int,twin,pwin,&
L01365                            bw,bl,zw,a,b)
L01366
L01367      call gaussjbem(a,6,b,6)
L01368
L01369          do iw=1,6
L01370             rlint(iw)=b(iw)
L01371          enddo
L01372
L01373      return
L01374      end subroutine int_rlrad
```

```
!file: phys/module_sf_bem.F    L1379-L1517    (subroutine algebra_short)
L01379      subroutine algebra_short(rstr,albwal,albwin,aw,bw,zw,pwin,a,b)
L01380
L01381  !---------------------------------------------------------------------
L01382  !This routine calculates the algebraic system that will be solved for
L01383  !the computation of the total shortwave radiation that reachs every
L01384  !indoor wall in a floor.
L01385  !Write the matrix system ax=b to solve
L01386  !
L01387  !     -Rs(1)+a(1,2)Rs(2)+.................+a(1,6)Rs(6)=-Rs=b(1)
L01388  !a(2,1)Rs(1)-      Rs(2)+.................+a(2,6)Rs(6)=-Rs=b(2)
L01389  !a(3,1)Rs(1)+a(3,2)Rs(3)-Rs(3)+...........+a(3,6)Rs(6)=-Rs=b(3)
L01390  !a(4,1)Rs(1)+.................-Rs(4)+.....+a(4,6)Rs(6)=-Rs=b(4)
L01391  !a(5,1)Rs(1)+.....................-Rs(5)+a(5,6)Rs(6)=-Rs=b(5)
L01392  !a(6,1)Rs(1)+................................-R(6)=-Rs=b(6)
L01393  !
L01394  !This version suppose the albedo of the indoor walls is the same.
L01395  !---------------------------------------------------------------------
L01396      implicit none
L01397  !---------------------------------------------------------------------
L01398
L01399  !Input
L01400  !-----
L01401      real rstr   !solar radiation transmitted through the windows
L01402      real albwal !albedo of the internal walls
L01403      real albwin !albedo of the windows.
L01404      real bw     !length of the wall
L01405      real aw     !width of the wall
L01406      real zw     !height of the wall
L01407      real fprl_int   !view factor
L01408      real fnrm_int   !view factor
L01409      real pwin       !window proportion
L01410  !Output
L01411  !------
L01412      real a(6,6)     !Matrix for the system
L01413      real b(6)       !Second member for the system
L01414  !Local
L01415  !-----
L01416      integer iw,jw
L01417      real albm               !averaged albedo
L01418  !---------------------------------------------------------------------
L01419
L01420  !Initialise the variables
L01421
L01422      do iw=1,6
L01423            b(iw)= 0.
L01424        do jw=1,6
L01425             a(iw,jw)= 0.
L01426           enddo
L01427         enddo
L01428
L01429  !Calculation of the second member b
L01430
L01431      do iw=1,6
L01432       b(iw)=-rstr
L01433      end do
L01434
L01435  !Calculation of the averaged albedo
L01436
L01437      albm=pwin*albwin+(1-pwin)*albwal
L01438
L01439  !Calculation of the matrix a
L01440
L01441              a(1,1)=-1.
L01442
L01443              call fprl_ints(fprl_int,aw/bw,zw/bw)
L01444
L01445              a(1,2)=albm*fprl_int
L01446
L01447              call fnrm_ints(fnrm_int,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01448
L01449              a(1,3)=albm*(bw/aw)*fnrm_int
L01450
L01451              a(1,4)=a(1,3)
L01452
L01453              call fnrm_ints(fnrm_int,zw/aw,bw/aw,(bw*bw+zw*zw)/(aw*aw))
L01454
L01455              a(1,5)=albwal*(bw/zw)*fnrm_int
L01456
L01457              a(1,6)=a(1,5)
```

```
L01458
L01459
L01460              a(2,1)=a(1,2)
L01461              a(2,2)=-1.
L01462              a(2,3)=a(1,3)
L01463              a(2,4)=a(1,4)
L01464              a(2,5)=a(1,5)
L01465              a(2,6)=a(1,6)
L01466
L01467
L01468              call fnrm_ints(fnrm_int,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01469
L01470              a(3,1)=albm*(aw/bw)*fnrm_int
L01471          a(3,2)=a(3,1)
L01472          a(3,3)=-1.
L01473
L01474              call fprl_ints(fprl_int,zw/aw,bw/aw)
L01475
L01476          a(3,4)=albm*fprl_int
L01477
L01478              call fnrm_ints(fnrm_int,zw/bw,aw/bw,(aw*aw+zw*zw)/(bw*bw))
L01479
L01480          a(3,5)=albwal*(aw/zw)*fnrm_int
L01481              a(3,6)=a(3,5)
L01482
L01483
L01484              a(4,1)=a(3,1)
L01485              a(4,2)=a(3,2)
L01486              a(4,3)=a(3,4)
L01487              a(4,4)=-1.
L01488              a(4,5)=a(3,5)
L01489              a(4,6)=a(3,6)
L01490
L01491              call fnrm_ints(fnrm_int,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01492
L01493              a(5,1)=albm*(zw/bw)*fnrm_int
L01494
L01495              a(5,2)=a(5,1)
L01496
L01497              call fnrm_ints(fnrm_int,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01498
L01499              a(5,3)=albm*(zw/aw)*fnrm_int
L01500
L01501              a(5,4)=a(5,3)
L01502              a(5,5)=-1.
L01503
L01504              call fprl_ints(fprl_int,aw/zw,bw/zw)
L01505
L01506              a(5,6)=albwal*fprl_int
L01507
L01508
L01509              a(6,1)=a(5,1)
L01510              a(6,2)=a(5,2)
L01511              a(6,3)=a(5,3)
L01512              a(6,4)=a(5,4)
L01513              a(6,5)=a(5,6)
L01514              a(6,6)=-1.
L01515
L01516      return
L01517      end subroutine algebra_short
```

```
!file: phys/module_sf_bem.F    L1522-L1823    (subroutine algebra_long)
L01522      subroutine algebra_long(emwal,emwin,sigma,twalint,twinint,&
L01523                        pwin,aw,bw,zw,a,b)
L01524
L01525  !-----------------------------------------------------------------------
L01526  !This routine computes the algebraic system that will be solved to
L01527  !compute the longwave radiation that reachs the indoor
L01528  !walls in a floor.
L01529  !Write the matrix system ax=b to solve
L01530  !
L01531  !a(1,1)Rl(1)+.............................+Rl(6)=b(1)
L01532  !a(2,1)Rl(1)+.................+Rl(5)+a(2,6)Rl(6)=b(2)
L01533  !a(3,1)Rl(1)+.....+Rl(3)+............+a(3,6)Rl(6)=b(3)
L01534  !a(4,1)Rl(1)+............+Rl(4)+.....+a(4,6)Rl(6)=b(4)
L01535  !      Rl(1)+.......................+a(5,6)Rl(6)=b(5)
L01536  !a(6,1)Rl(1)+Rl(2)+.................+a(6,6)Rl(6)=b(6)
L01537  !
L01538  !-----------------------------------------------------------------------
L01539          implicit none
L01540
L01541  !-----------------------------------------------------------------------
L01542
L01543  !Input
L01544  !-----
L01545
L01546      real pwin       !window proportion
L01547      real emwal  !emissivity of the internal walls
L01548      real emwin  !emissivity of the window
L01549      real sigma  !Stefan-Boltzmann constant [W/m2.K4]
L01550      real twalint(6) !temperature of the first internal layers of a room [K]
L01551      real twinint(4) !temperature of the windows [K]
L01552      real aw     !width of the wall
L01553      real bw     !length of the wall
L01554      real zw     !height of the wall
L01555      real fprl_int   !view factor
L01556      real fnrm_int   !view factor
L01557            real fnrm_intx   !view factor
L01558            real fnrm_inty   !view factor
L01559
L01560  !Output
L01561  !------
L01562      real b(6)   !second member vector for the system
L01563      real a(6,6) !matrix for the system
L01564  !Local
L01565  !-----
L01566      integer iw,jw
```

```
L01567      real b_wall(6)
L01568      real b_wind(6)
L01569      real emwal_av       !averadge emissivity of the wall
L01570      real emwin_av       !averadge emissivity of the window
L01571          real em_av      !averadge emissivity
L01572          real twal_int(6)        !twalint
L01573      real twin(4)        !twinint
L01574  !-------------------------------------------------------------------
L01575
L01576  !Initialise the variables
L01577  !-------------------------
L01578
L01579       do iw=1,6
L01580             b(iw)= 0.
L01581             b_wall(iw)=0.
L01582             b_wind(iw)=0.
L01583            do jw=1,6
L01584              a(iw,jw)= 0.
L01585            enddo
L01586           enddo
L01587
L01588           do iw=1,6
L01589              twal_int(iw)=twalint(iw)
L01590           enddo
L01591
L01592           do iw=1,4
L01593              twin(iw)=twinint(iw)
L01594           enddo
L01595
L01596  !Calculation of the averadge emissivities
L01597  !----------------------------------------
L01598
L01599      emwal_av=(1-pwin)*emwal
L01600      emwin_av=pwin*emwin
L01601      em_av=emwal_av+emwin_av
L01602
L01603  !Calculation of the second term for the walls
L01604  !-------------------------------------------
L01605
L01606              call fprl_ints(fprl_int,aw/zw,bw/zw)
L01607              call fnrm_ints(fnrm_intx,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01608              call fnrm_ints(fnrm_inty,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01609
L01610              b_wall(1)=(emwal*sigma*(twal_int(5)**4)*           &
L01611                   fprl_int)+                                    &
L01612                   (sigma*(emwal_av*(twal_int(3)**4)+            &
L01613                    emwal_av*(twal_int(4)**4))*                  &
L01614                   (zw/aw)*fnrm_intx)+                           &
L01615                   (sigma*(emwal_av*(twal_int(1)**4)+            &
L01616                    emwal_av*(twal_int(2)**4))*                  &
L01617                   (zw/bw)*fnrm_inty)
L01618
L01619              call fprl_ints(fprl_int,aw/zw,bw/zw)
L01620              call fnrm_ints(fnrm_intx,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01621              call fnrm_ints(fnrm_inty,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01622
L01623              b_wall(2)=(emwal*sigma*(twal_int(6)**4)*           &
L01624                     fprl_int)+                                  &
L01625                    (sigma*(emwal_av*(twal_int(3)**4)+           &
L01626                    emwal_av*(twal_int(4)**4))*                  &
L01627                   (zw/aw)*fnrm_intx)+                           &
L01628                   (sigma*(emwal_av*(twal_int(1)**4)+            &
L01629                   emwal_av*(twal_int(2)**4))*                   &
L01630                   (zw/bw)*fnrm_inty)
L01631
L01632              call fprl_ints(fprl_int,zw/aw,bw/aw)
L01633              call fnrm_ints(fnrm_intx,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01634              call fnrm_ints(fnrm_inty,zw/bw,aw/bw,(aw*aw+zw*zw)/(bw*bw))
L01635
L01636              b_wall(3)=(emwal_av*sigma*(twal_int(4)**4)*        &
L01637                fprl_int)+                                  &
L01638                   (sigma*(emwal_av*(twal_int(2)**4)+            &
L01639                    emwal_av*(twal_int(1)**4))*                  &
L01640                   (aw/bw)*fnrm_intx)+                           &
L01641                   (sigma*(emwal*(twal_int(5)**4)+               &
L01642                    emwal*(twal_int(6)**4))*                     &
L01643                   (aw/zw)*fnrm_inty)
L01644
L01645              call fprl_ints(fprl_int,zw/aw,bw/aw)
L01646              call fnrm_ints(fnrm_intx,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01647              call fnrm_ints(fnrm_inty,zw/bw,aw/bw,(aw*aw+zw*zw)/(bw*bw))
L01648
L01649              b_wall(4)=(emwal_av*sigma*(twal_int(3)**4)*        &
L01650                   fprl_int)+                                    &
L01651                   (sigma*(emwal_av*(twal_int(2)**4)+            &
L01652                    emwal_av*(twal_int(1)**4))*                  &
L01653                   (aw/bw)*fnrm_intx)+                           &
L01654                   (sigma*(emwal*(twal_int(5)**4)+               &
L01655                    emwal*(twal_int(6)**4))*                     &
L01656                   (aw/zw)*fnrm_inty)
L01657
L01658              call fprl_ints(fprl_int,aw/bw,zw/bw)
L01659              call fnrm_ints(fnrm_intx,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01660              call fnrm_ints(fnrm_inty,zw/aw,bw/aw,(bw*bw+zw*zw)/(aw*aw))
L01661
L01662              b_wall(5)=(emwal_av*sigma*(twal_int(2)**4)*        &
L01663                   fprl_int)+                                    &
L01664                   (sigma*(emwal_av*(twal_int(3)**4)+            &
L01665                    emwal_av*(twal_int(4)**4))*                  &
L01666                   (bw/aw)*fnrm_intx)+                           &
L01667                   (sigma*(emwal*(twal_int(5)**4)+               &
L01668                    emwal*(twal_int(6)**4))*                     &
L01669                   (bw/zw)*fnrm_inty)
L01670
L01671              call fprl_ints(fprl_int,aw/bw,zw/bw)
L01672              call fnrm_ints(fnrm_intx,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01673              call fnrm_ints(fnrm_inty,zw/aw,bw/aw,(bw*bw+zw*zw)/(aw*aw))
L01674
```

```
L01675              b_wall(6)=(emwal_av*sigma*(twal_int(1)**4)*        &
L01676                   fprl_int)+                                    &
L01677                   (sigma*(emwal_av*(twal_int(3)**4)+            &
L01678                    emwal_av*(twal_int(4)**4))*                  &
L01679                   (bw/aw)*fnrm_intx)+                           &
L01680                   (sigma*(emwal*(twal_int(5)**4)+               &
L01681                   emwal*(twal_int(6)**4))*                      &
L01682                   (bw/zw)*fnrm_inty)
L01683
L01684  !Calculation of the second term for the windows
L01685  !-------------------------------------------
L01686              call fnrm_ints(fnrm_intx,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01687              call fnrm_ints(fnrm_inty,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01688
L01689              b_wind(1)=(sigma*(emwin_av*(twin(3)**4)+          &
L01690                    emwin_av*(twin(4)**4))*                     &
L01691                   (zw/aw)*fnrm_intx)+                          &
L01692                   (sigma*(emwin_av*(twin(1)**4)+               &
L01693                    emwin_av*(twin(2)**4))*                     &
L01694                   (zw/bw)*fnrm_inty)
L01695
L01696              call fnrm_ints(fnrm_intx,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01697              call fnrm_ints(fnrm_inty,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01698
L01699              b_wind(2)=(sigma*(emwin_av*(twin(3)**4)+          &
L01700                    emwin_av*(twin(4)**4))*                     &
L01701                   (zw/aw)*fnrm_intx)+                          &
L01702                   (sigma*(emwin_av*(twin(1)**4)+               &
L01703                    emwin_av*(twin(2)**4))*                     &
L01704                   (zw/bw)*fnrm_inty)
L01705
L01706              call fprl_ints(fprl_int,zw/aw,bw/aw)
L01707              call fnrm_ints(fnrm_int,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01708
L01709              b_wind(3)=emwin_av*sigma*(twin(4)**4)*            &
L01710                   fprl_int+(sigma*(emwin_av*                   &
L01711                   (twin(2)**4)+emwin_av*(twin(1)**4))*         &
L01712                   (aw/bw)*fnrm_int)
L01713
L01714              call fprl_ints(fprl_int,zw/aw,bw/aw)
L01715              call fnrm_ints(fnrm_int,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01716
L01717              b_wind(4)=emwin_av*sigma*(twin(3)**4)*            &
L01718                   fprl_int+(sigma*(emwin_av*                   &
L01719                    (twin(2)**4)+emwin_av*(twin(1)**4))*        &
L01720                   (aw/bw)*fnrm_int)
L01721
L01722              call fprl_ints(fprl_int,aw/bw,zw/bw)
L01723              call fnrm_ints(fnrm_int,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01724
L01725              b_wind(5)=emwin_av*sigma*(twin(2)**4)*            &
L01726                   fprl_int+(sigma*(emwin_av*                   &
L01727                   (twin(3)**4)+emwin_av*(twin(4)**4))*         &
L01728                   (bw/aw)*fnrm_int)
L01729
L01730              call fprl_ints(fprl_int,aw/bw,zw/bw)
L01731              call fnrm_ints(fnrm_int,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01732
L01733              b_wind(6)=emwin_av*sigma*(twin(1)**4)*            &
L01734                   fprl_int+(sigma*(emwin_av*                   &
L01735                   (twin(3)**4)+emwin_av*(twin(4)**4))*         &
L01736                   (bw/aw)*fnrm_int)
L01737
L01738  !Calculation of the total b term
L01739  !-------------------------------
L01740
L01741      do iw=1,6
L01742       b(iw)=b_wall(iw)+b_wind(iw)
L01743      end do
L01744
L01745
L01746  !Calculation of the matrix of the system
L01747  !---------------------------------------
L01748
L01749           call fnrm_ints(fnrm_int,bw/aw,zw/aw,(bw*bw+zw*zw)/(aw*aw))
L01750
L01751           a(1,1)=(em_av-1.)*(zw/bw)*fnrm_int
L01752
L01753           a(1,2)=a(1,1)
L01754
L01755           call fnrm_ints(fnrm_int,aw/bw,zw/bw,(aw*aw+zw*zw)/(bw*bw))
L01756
L01757           a(1,3)=(em_av-1.)*(zw/aw)*fnrm_int
L01758
L01759           a(1,4)=a(1,3)
L01760
L01761           call fprl_ints(fprl_int,aw/zw,bw/zw)
L01762
L01763           a(1,5)=(emwal-1.)*fprl_int
L01764           a(1,6)=1.
L01765
L01766           a(2,1)=a(1,1)
L01767           a(2,2)=a(1,2)
L01768           a(2,3)=a(1,3)
L01769           a(2,4)=a(1,4)
L01770           a(2,5)=1.
L01771           a(2,6)=a(1,5)
L01772
L01773           call fnrm_ints(fnrm_int,bw/zw,aw/zw,(bw*bw+aw*aw)/(zw*zw))
L01774
L01775           a(3,1)=(em_av-1.)*(aw/bw)*fnrm_int
L01776
L01777           a(3,2)=a(3,1)
L01778           a(3,3)=1.
L01779
L01780           call fprl_ints(fprl_int,zw/aw,bw/aw)
L01781
L01782           a(3,4)=(em_av-1.)*fprl_int
```

```
L01783
L01784           call fnrm_ints(fnrm_int,zw/bw,aw/bw,(aw*aw+zw*zw)/(bw*bw))
L01785
L01786           a(3,5)=(emwal-1.)*(aw/zw)*fnrm_int
L01787
L01788           a(3,6)=a(3,5)
L01789
L01790           a(4,1)=a(3,1)
L01791           a(4,2)=a(3,2)
L01792           a(4,3)=a(3,4)
L01793           a(4,4)=1.
L01794           a(4,5)=a(3,5)
L01795           a(4,6)=a(3,6)
L01796
L01797           a(5,1)=1.
L01798
L01799           call fprl_ints(fprl_int,aw/bw,zw/bw)
L01800
L01801           a(5,2)=(em_av-1.)*fprl_int
L01802
L01803           call fnrm_ints(fnrm_int,aw/zw,bw/zw,(aw*aw+bw*bw)/(zw*zw))
L01804
L01805           a(5,3)=(em_av-1.)*(bw/aw)*fnrm_int
L01806
L01807           a(5,4)=a(5,3)
L01808
L01809           call fnrm_ints(fnrm_int,zw/aw,bw/aw,(bw*bw+zw*zw)/(aw*aw))
L01810
L01811           a(5,5)=(emwal-1.)*(bw/zw)*fnrm_int
L01812
L01813           a(5,6)=a(5,5)
L01814
L01815           a(6,1)=a(5,2)
L01816           a(6,2)=1.
L01817           a(6,3)=a(5,3)
L01818           a(6,4)=a(5,4)
L01819           a(6,5)=a(5,5)
L01820           a(6,6)=a(6,5)
L01821
L01822         return
L01823         end subroutine algebra_long
```

讲解：房间内两次反射的 6×6 线性系统：algebra_short 组装 A 矩阵（对角 −1、非对角 =视角因子 × 反照率，窗户面用混合反照率 albm=pwin • albwin+(1−pwin) • albwal），b=−rstr（透射进室内的短波各向同性撒入）；algebra_long 同构但源项是 em • σT⁴（六面第一层温度 twal_int：1..4 面取墙内层、5/6 面取楼板顶/底或地基/屋顶）。视角因子用 fprl_ints/fnrm_ints 的解析矩形公式（房间内两面平行/正交）。**论文 2 视角：这套 6×6 系统对「逐格异质建筑」依然成立——它只依赖每栋楼自己的几何（bw/bl/dzlev/pwin），参数场化不破坏数学结构。**

```
!file: phys/module_sf_bem.F    L1210-L1257    (subroutine hsinsflux)
L01210      subroutine hsinsflux(swsurf,swwin,tin,tw,hsins)
L01211
L01212      implicit none
L01213
L01214  ! ---------------------------------------------------------------------
L01215  ! This routine computes the internal sensible heat flux.
L01216  ! The swsurf, makes rhe difference between a vertical and a
L01217  ! horizontal surface.
L01218  ! The values of the heat conduction coefficients hc are obtained from the book
L01219  ! "Energy Simulation in Building Design". J.A. Clarke.
L01220  ! Adam Hilger, Bristol, 362 pp.
L01221  ! ---------------------------------------------------------------------
L01222  !Input
L01223  !----
L01224      integer swsurf  !swich for the type of surface (horizontal/vertical)
L01225              integer swwin   !swich for the type of surface (window/wall)
L01226      real tin    !Inside temperature [K]
L01227      real tw     !Internal wall temperature [K]
L01228
L01229
L01230  !Output
L01231  !------
L01232      real hsins  !internal sensible heat flux [W/m2]
L01233  !Local
L01234  !-----
L01235      real hc     !heat conduction coefficient [W/\B0C.m2]
L01236  !---------------------------------------------------------------------
L01237
L01238      if (swsurf.eq.2) then   !vertical surface
L01239           if (swwin.eq.1) then
L01240              hc=5.678*0.99        !window surface (smooth surface)
L01241           else
L01242              hc=5.678*1.09        !wall surface (rough surface)
L01243           endif
L01244       hsins=hc*(tin-tw)
L01245      endif
L01246
L01247      if (swsurf.eq.1)  then   !horizontal surface
L01248           if (swwin.eq.1) then
L01249             hc=5.678*0.99        !window surface (smooth surface)
L01250           else
L01251             hc=5.678*1.09        !wall surface (rough surface)
L01252           endif
L01253           hsins=hc*(tin-tw)
L01254          endif
L01255
L01256      return
L01257      end subroutine hsinsflux
```

讲解：室内对流系数（Clarke 教材值）：竖墙/水平面、粗糙/光滑两组，swsurf/swwin 两开关目前只区分竖墙 vs 窗（水平分支与竖墙同系数——天花板/地板对流按竖墙处理的简化）。

```
!file: phys/module_sf_bem.F    L2521-L2540    (subroutine radfluxs)
L02521        subroutine radfluxs(radflux,alb,rs,em,rl,sigma,twal)
L02522
L02523        implicit none
L02524  !------------------------------------------------------------------
L02525  !This function calculates the radiative fluxe at a surface
L02526  !------------------------------------------------------------------
L02527
L02528
L02529       real alb    !albedo of the surface
L02530       real rs     !shor wave radiation
L02531       real em     !emissivity of the surface
L02532       real rl     !lon wave radiation
L02533       real sigma  !parameter (wall is not black body) [W/m2.K4]
L02534       real twal   !wall temperature [K]
L02535       real radflux
L02536
L02537        radflux=(1.-alb)*rs+em*rl-em*sigma*twal**4
L02538
L02539        return
L02540        end subroutine radfluxs
```

```
!file: phys/module_sf_bem.F    L2543-L2586    (subroutine radfluxspv)
L02543        subroutine radfluxspv(nz,n,alb,rs,swddif,em,rl,twal,tair,sigma,radflux,pv_frac_roof,tpv)
L02544  !
L02545        implicit none
L02546  !
L02547  ! This routine calculates the radiative fluxes at the surfaces
L02548  !
L02549  !     Integer and real kinds
L02550  !
L02551  !     integer, parameter :: kind_im = selected_int_kind(6)   ! 4 byte integer
L02552  !     integer, parameter :: kind_rb = selected_real_kind(12) ! 8 byte real
L02553  !
L02554  ! Input Variables
L02555  !
L02556        integer,intent(in) :: nz !Maximum number of vertical levels in the urban grid
L02557        real,intent(in) :: alb    !albedo of the surface
L02558        real,intent(in) :: rs !shortwave radiation [W m-2]
L02559        real,intent(in) :: swddif
L02560        real,intent(in) :: em !emissivity of the surface
L02561        real,intent(in) :: rl     !Longwave radiation [W m-2]
L02562        real,intent(in) :: twal   !surface temperature [K]
L02563        real,intent(in) :: sigma !Stefan-Boltzmann constant [W/m2.K4]
L02564        real,intent(in) :: tpv !Stefan-Boltzmann constant [W/m2.K4]
L02565        real,intent(in),dimension(1:nz) :: tair  !external temperature [K]
L02566        integer,intent(in) :: n  !number of floors in the building
L02567        real, intent(in) :: pv_frac_roof !
L02568        real :: empv
L02569        real :: hrad
L02570        real :: F12
L02571  ! Output variables
L02572  !
L02573        real,intent(inout) :: radflux !radiative flux at the surface [W m-2]
L02574  !
L02575  ! Local variables
L02576        F12=1.
L02577        empv=0.95
L02578        hrad=sigma/((1-empv)/empv+1/F12+(1-em)/em)
L02579        if ((n+1).gt.nz) then
L02580           write(*,*) 'Increase maximum number of vertical levels in the urban grid'
L02581           stop
L02582        endif
L02583        radflux=(1.-alb)*(1.-pv_frac_roof)*rs+em*(1.-pv_frac_roof)*rl+pv_frac_roof*hrad*(tpv**4-twal**4)- &
L02584                em*sigma*(1.-pv_frac_roof)*twal**4
L02585        return
L02586        end subroutine radfluxspv
```

```
!file: phys/module_sf_bem.F    L2594-L2608    (subroutine fprl_ints)
L02594          subroutine fprl_ints(fprl_int,vx,vy)
L02595
L02596          implicit none
L02597
L02598       real vx,vy
L02599       real fprl_int
L02600
L02601       fprl_int=(2./(3.141592653*vx*vy))*                        &
L02602                (log(sqrt((1.+vx*vx)*(1.+vy*vy)/(1.+vx*vx+vy*vy)))+ &
L02603                 (vy*sqrt(1.+vx*vx)*atan(vy/sqrt(1.+vx*vx)))+       &
L02604                 (vx*sqrt(1.+vy*vy)*atan(vx/sqrt(1.+vy*vy)))-       &
L02605                 vy*atan(vy)-vx*atan(vx))
L02606
L02607          return
L02608          end subroutine fprl_ints
```

```
!file: phys/module_sf_bem.F    L2618-L2632    (subroutine fnrm_ints)
L02618          subroutine fnrm_ints(fnrm_int,wx,wy,wz)
L02619
L02620          implicit none
L02621
L02622       real wx,wy,wz
L02623       real fnrm_int
L02624
L02625           fnrm_int=(1./(3.141592653*wy))*(wy*atan(1./wy)+wx*atan(1./wx)- &
L02626                 (sqrt(wz)*atan(1./sqrt(wz)))+                           &
L02627                 (1./4.)*(log((1.+wx*wx)*(1.+wy*wy)/(1.+wz))+            &
L02628                 wy*wy*log(wy*wy*(1.+wz)/(wz*(1.+wy*wy)))+               &
L02629                 wx*wx*log(wx*wx*(1.+wz)/(wz*(1.+wx*wx)))))
L02630
L02631          return
L02632          end subroutine fnrm_ints
```

讲解：radfluxs 是单面辐射收支一行式；radfluxspv 在屋顶面加 PV 板与屋顶的辐射串联项 pv_frac_

roof • hrad • (T_pv⁴ − T_roof⁴)，两式与 14b upward_rad 的屋顶 PV 常数（0.79/0.11）同源——改 PV 光学参数三处同步。

八、室内源与调节

```
!file: phys/module_sf_bem.F    L1829-L1864    (subroutine fluxroo)
L01829      subroutine fluxroo(hseqocc,hleqocc,hsvent,hlvent, &
L01830                         hscond,hslev,hllev)
L01831
L01832  !--------------------------------------------------------------------
L01833  !This routine calculates the heat flux generated inside the room
L01834  !and the heat ejected to the atmosphere.
L01835  !--------------------------------------------------------------------
L01836
L01837      implicit none
L01838
L01839  !--------------------------------------------------------------------
L01840
L01841  !Input
L01842  !-----
L01843      real hseqocc        !sensible heat generated by equipments and occupants [W]
L01844      real hleqocc        !latent heat generated by occupants [W]
L01845      real hsvent     !sensible heat generated by natural ventilation [W]
L01846      real hlvent     !latent heat generated by natural ventilation [W]
L01847      real hscond     !sensible heat generated by wall conduction
L01848
L01849  !Output
L01850  !------
L01851      real hslev      !sensible heat flux generated inside the room [W]
L01852      real hllev      !latent heat flux generatd inside the room
L01853
L01854
L01855  !Calculation of the total sensible heat generated inside the room
L01856
L01857      hslev=hseqocc+hsvent+hscond
L01858
L01859  !Calculation of the total latent heat generated inside the room
L01860
L01861      hllev=hleqocc+hlvent
L01862
L01863      return
L01864      end subroutine fluxroo
```

```
!file: phys/module_sf_bem.F    L1869-L1891    (subroutine phirat)
L01869      subroutine phirat(nhourday,rocc)
L01870
L01871  !--------------------------------------------------------------------
L01872  !This routine calculates the occupation ratio of a floor
L01873  !By now we suppose a constant value
L01874  !--------------------------------------------------------------------
L01875
L01876          implicit none
L01877
L01878  !Input
L01879  !-----
L01880
L01881      real nhourday   ! number of hours from midnight (local time)
L01882
L01883  !Output
L01884  !------
L01885      real rocc       !value between 0 and 1
L01886
L01887  !!TEST
L01888          rocc=1.
L01889
L01890      return
L01891      end subroutine phirat
```

```
!file: phys/module_sf_bem.F    L1896-L1917    (subroutine phiequ)
L01896      subroutine phiequ(nhourday,hsesf,hsequip,hsequ)
L01897
L01898  !--------------------------------------------------------------------
L01899  !This routine calculates the sensible heat gain from equipments
L01900  !--------------------------------------------------------------------
L01901          implicit none
L01902  !Input
L01903  !-----
L01904
L01905      real nhourday ! number of hours from midnight, Local time
L01906          real, intent(in) :: hsesf
L01907          real, intent(in), dimension(24) :: hsequip
L01908
L01909  !Output
L01910  !------
L01911      real hsequ    !sensible heat gain from equipment [Wm\AF2]
L01912
L01913  !--------------------------------------------------------------------
L01914
L01915          hsequ = hsequip(int(nhourday)+1) * hsesf
L01916
L01917      end subroutine phiequ
```

```
!file: phys/module_sf_bem.F    L1921-L1977    (subroutine fluxeqocc)
L01921      subroutine fluxeqocc(nhourday,bw,bl,perflo,hsesf,hsequip,hseqocc,hleqocc)
L01922
L01923      implicit none
L01924
L01925  !--------------------------------------------------------------------
L01926  !This routine calculates the sensible and the latent heat flux
```

```
L01927  !generated by equipments and occupants
L01928  !---------------------------------------------------------------------
L01929
L01930  !Input
L01931  !-----
L01932      real bw         !Room width [m]
L01933      real bl         !Room lengzh [m]
L01934      real nhourday       !number of hours from the beginning of the day
L01935          real, intent(in) :: perflo ! Peak number of occupants per unit floor area
L01936          real, intent(in) :: hsesf
L01937          real, intent(in), dimension(24) :: hsequip
L01938
L01939  !Output
L01940  !------
L01941      real hseqocc        !sensible heat generated by equipments and occupants [W]
L01942      real hleqocc        !latent heat generated by occupants [W]
L01943  !Local
L01944  !-----
L01945      real Af         !Air conditioned floor area [m2]
L01946      real rocc       !Occupation ratio of the floor [0,1]
L01947          real hsequ      !Heat generated from equipments
L01948
L01949          real hsocc      !Sensible heat generated by a person [W/Person]
L01950                              !Source Boundary Layer Climates,page 195 (book)
L01951          parameter (hsocc=160.)
L01952
L01953          real hlocc      !Latent heat generated by a person [W/Person]
L01954                              !Source Boundary Layer Climates,page 225 (book)
L01955          parameter (hlocc=1.96e6/86400.)
L01956
L01957  !---------------------------------------------------------------------
L01958  !           Sensible heat flux
L01959  !           ------------------
L01960
L01961       Af=bw*bl
L01962
L01963       call phirat(nhourday,rocc)
L01964
L01965           call phiequ(nhourday,hsesf,hsequip,hsequ)
L01966
L01967           hseqocc=Af*rocc*perflo*hsocc+Af*hsequ
L01968
L01969  !
L01970  !           Latent heat
L01971  !           -----------
L01972  !
L01973
L01974           hleqocc=Af*rocc*perflo*hlocc
L01975
L01976      return
L01977      end subroutine fluxeqocc
```

讲解：室内热源汇总（hslev=人员设备 +通风 +围护导热；hllev=人员 +通风）；phirat 占用率**硬编码 1.0**（TODO 注释在案——办公昼夜占用曲线是天然扩展点）；phiequ 设备热 =24h 谱 hsequip(int(nhourday)+1)×hsesf（谱面积因子）；人员显热 160 W/人（Oke《Boundary Layer Climates》p195）、潜热 1.96e6/86400 W/人（每日 1.96 MJ 水汽折算）。

```
!file: phys/module_sf_bem.F    L1982-L2029    (subroutine fluxvent)
L01982      subroutine fluxvent(cpint,rhoint,vollev,tlev,tout,latent,&
L01983                              humout,rhoout,humlev,beta,hsvent,hlvent)
L01984
L01985      implicit none
L01986
L01987  !---------------------------------------------------------------------
L01988  !This routine calculates the sensible and the latent heat flux
L01989  !generated by natural ventilation
L01990  !---------------------------------------------------------------------
L01991
L01992  !Input
L01993  !-----
L01994      real cpint      !specific heat of the indoor air [J/kg.K]
L01995      real rhoint     !density of the indoor air [Kg/m3]
L01996      real vollev     !volume of the room [m3]
L01997      real tlev       !Room temperature [K]
L01998      real tout       !outside air temperature [K]
L01999      real latent     !latent heat of evaporation [J/Kg]
L02000      real humout     !outside absolute humidity [Kgwater/Kgair]
L02001      real rhoout     !air density [kg/m3]
L02002      real humlev     !Specific humidity of the indoor air [Kgwater/Kgair]
L02003          real, intent(in) :: beta!Thermal efficiency of the heat exchanger
L02004
L02005  !Output
L02006  !------
L02007      real hsvent     !sensible heat generated by natural ventilation [W]
L02008      real hlvent     !latent heat generated by natural ventilation [W]
L02009
L02010  !Local
L02011  !-----
L02012
L02013  !---------------------------------------------------------------------
L02014
L02015  !           Sensible heat flux
L02016  !           ------------------
L02017
L02018      hsvent=(1.-beta)*cpint*rhoint*(vollev/3600.)*  &
L02019                 (tout-tlev)
L02020
L02021  !           Latent heat flux
L02022  !           ----------------
L02023
L02024      hlvent=(1.-beta)*latent*rhoint*(vollev/3600.)* &
L02025                 (humout-humlev)
L02026
L02027
L02028      return
```

```
L02029      end subroutine fluxvent
```

讲解：自然通风：**每小时一次全换气**（vollev/3600 m³/s 的隐含假设——既非 ACH 可调也无风压/热压驱动），换热器效率 beta 只对温差折扣；(1−beta) 因子把「回收后」的净通量加给室内。改造点：把 1/3600 换成可调 ACH 或加风压项是最常见的 BEM 扩展。

```
!file: phys/module_sf_bem.F    L2034-L2080    (subroutine fluxcond)
L02034      subroutine fluxcond(hswalins,hswinins,surwal,pwin,hscond)
L02035
L02036      implicit none
L02037
L02038  !------------------------------------------------------------------------
L02039  !This routine calculates the sensible heat flux generated by
L02040  !wall conduction.
L02041  !------------------------------------------------------------------------
L02042
L02043  !Input
L02044  !-----
L02045      real hswalins(6)    !sensible heat at the internal layers of the wall [W/m2]
L02046      real hswinins(4)    !internal window sensible heat flux [W/m2]
L02047      real surwal(6)          !surfaces of the room walls [m2]
L02048      real pwin               !window proportion
L02049
L02050
L02051  !Output
L02052  !------
L02053
L02054      real hscond     !sensible heat generated by wall conduction [W]
L02055
L02056  !Local
L02057  !-----
L02058
L02059      integer ivw
L02060
L02061  !------------------------------------------------------------------------
L02062
L02063        hscond=0.
L02064
L02065      do ivw=1,4
L02066         hscond=hscond+surwal(ivw)*(1-pwin)*hswalins(ivw)+ &
L02067                     surwal(ivw)*pwin*hswinins(ivw)
L02068      end do
L02069
L02070      do ivw=5,6
L02071            hscond=hscond+surwal(ivw)*hswalins(ivw)
L02072      end do
L02073  !
L02074  !Finally we must change the sign in hscond to be proportional
L02075  !to the difference (Twall-Tindoor).
L02076  !
L02077          hscond=(-1)*hscond
L02078
L02079      return
L02080      end subroutine fluxcond
```

讲解：围护导热进入房间的合计：四竖墙按 (1−pwin)·墙流 +pwin·窗流、地板/天花板全额；末尾符号翻转使 hscond∝(T_wall−T_indoor) 方向一致——调试室内温度符号问题先看这行。

```
!file: phys/module_sf_bem.F    L2085-L2165    (subroutine regtemp)
L02085      subroutine regtemp(swcond,nhourday,dt,Qb,hsroo,          &
L02086                             tlev,timeon,timeoff,targtemp,gaptemp,hsneed)
L02087
L02088      implicit none
L02089
L02090  !------------------------------------------------------------------------
L02091  !This routine calculates the sensible heat fluxes,
L02092  !after anthropogenic regulation (air conditioning)
L02093  !------------------------------------------------------------------------
L02094
L02095  !Input:
L02096  !-----.
L02097          integer swcond       !swich air conditioning
L02098      real nhourday        !number of hours from the beginning of the day real
L02099      real dt              !time step [s]
L02100      real Qb          !overall heat capacity of the indoor air [J/K]
L02101          real hsroo           !sensible heat flux generated inside the room [W]
L02102          real tlev            !room air temperature [K]
L02103          real, intent(in) :: timeon  ! Initial local time of A/C systems
L02104          real, intent(in) :: timeoff ! Ending local time of A/C systems
L02105          real, intent(in) :: targtemp! Target temperature of A/C systems
L02106          real, intent(in) :: gaptemp ! Comfort range of indoor temperature
L02107
L02108
L02109  !Local:
L02110  !-----.
L02111
L02112          real templev         !hipotetical room air temperature [K]
L02113          real alpha           !variable to control the heating/cooling of
L02114                               !the air conditining system
L02115  !Output:
L02116  !-----.
L02117      real hsneed      !sensible heat extracted to the indoor air [W]
L02118  !------------------------------------------------------------------------
L02119  !initialize variables
L02120  !--------------------
L02121          templev = 0.
L02122          alpha   = 0.
L02123
L02124          if (swcond.eq.0) then ! there is not air conditioning in the floor
L02125              hsneed = 0.
L02126              goto 100
L02127          else
```

```
L02128              if ((nhourday.ge.timeon).and.(nhourday.le.timeoff)) then
L02129                 templev=tlev+(dt/Qb)*hsroo
L02130                 goto 200
L02131              else
L02132                 hsneed = 0.     ! air conditioning is switched off
L02133                 goto 100
L02134              endif
L02135           endif
L02136
L02137  200      continue
L02138
L02139           if (abs(templev-targtemp).le.gaptemp) then
L02140              hsneed = 0.
L02141           else
L02142              if (templev.gt.(targtemp+gaptemp)) then
L02143                 hsneed=hsroo-(Qb/dt)*(targtemp+gaptemp-tlev)
L02144                 alpha=(abs(hsneed-hsroo)/Qb)
L02145                 if (alpha.gt.temp_rat) then
L02146                     hsneed=hsroo+temp_rat*Qb
L02147                     goto 100
L02148                 else
L02149                     goto 100
L02150                 endif
L02151              else
L02152                 hsneed=hsroo-(Qb/dt)*(targtemp-gaptemp-tlev)
L02153                 alpha=(abs(hsneed-hsroo)/Qb)
L02154                 if (alpha.gt.temp_rat) then
L02155                     hsneed=hsroo-temp_rat*Qb
L02156                     goto 100
L02157                 else
L02158                     goto 100
L02159                 endif
L02160              endif
L02161           endif
L02162
L02163  100      continue
L02164       return
L02165       end subroutine regtemp
```

```
!file: phys/module_sf_bem.F    L2170-L2253    (subroutine reghum)
L02170        subroutine reghum(swcond,nhourday,dt,volroo,rhoint,latent, &
L02171                          hlroo,shumroo,timeon,timeoff,targhum,gaphum,hlneed)
L02172
L02173        implicit none
L02174
L02175  !-------------------------------------------------------------------
L02176  !This routine calculates the latent heat fluxes,
L02177  !after anthropogenic regulation (air conditioning)
L02178  !-------------------------------------------------------------------
L02179
L02180  !Input:
L02181  !-----.
L02182           integer swcond    !swich air conditioning
L02183       real nhourday     !number of hours from the beginning of the day real[h]
L02184       real dt           !time step [s]
L02185       real volroo       !volume of the room [m3]
L02186           real rhoint       !density of the internal air [Kg/m3]
L02187           real latent       !latent heat of evaporation [J/Kg]
L02188           real hlroo        !latent heat flux generated inside the room [W]
L02189           real shumroo      !specific humidity of the indoor air [kg/kg]
L02190           real, intent(in) :: timeon  ! Initial local time of A/C systems
L02191           real, intent(in) :: timeoff ! Ending local time of A/C systems
L02192           real, intent(in) :: targhum ! Target humidity of the A/C systems
L02193           real, intent(in) :: gaphum  ! comfort range of the specific humidity
L02194
L02195  !Local:
L02196  !-----.
L02197
L02198           real humlev       !hipotetical specific humidity of the indoor [kg/kg]
L02199           real betha        !variable to control the drying/moistening of
L02200                             !the air conditioning system
L02201  !Output:
L02202  !-----.
L02203       real hlneed   !latent heat extracted to the indoor air [W]
L02204  !-------------------------------------------------------------------------
L02205  !initialize variables
L02206  !---------------------
L02207           humlev = 0.
L02208           betha  = 0.
L02209
L02210           if (swcond.eq.0) then ! there is not air conditioning in the floor
L02211               hlneed = 0.
L02212               goto 100
L02213           else
L02214               if ((nhourday.ge.timeon).and.(nhourday.le.timeoff)) then
L02215                  humlev=shumroo+(dt/(latent*rhoint*volroo))*hlroo
L02216                  goto 200
L02217               else
L02218                  hlneed = 0.     ! air conditioning is switched off
L02219                  goto 100
L02220               endif
L02221           endif
L02222
L02223  200      continue
L02224
L02225           if (abs(humlev-targhum).le.gaphum) then
L02226              hlneed = 0.
L02227           else
L02228              if (humlev.gt.(targhum+gaphum)) then
L02229                 hlneed=hlroo-((latent*rhoint*volroo)/dt)* &
L02230                             (targhum+gaphum-shumroo)
L02231                 betha=abs(hlneed-hlroo)/(latent*rhoint*volroo)
L02232                 if (betha.gt.hum_rat) then
L02233                     hlneed=hlroo+hum_rat*(latent*rhoint*volroo)
L02234                     goto 100
L02235                 else
L02236                     goto 100
```

```
L02237                endif
L02238             else
L02239                hlneed=hlroo-((latent*rhoint*volroo)/dt)* &
L02240                             (targhum-gaphum-shumroo)
L02241                betha=abs(hlneed-hlroo)/(latent*rhoint*volroo)
L02242                if (betha.gt.hum_rat) then
L02243                    hlneed=hlroo-hum_rat*(latent*rhoint*volroo)
L02244                    goto 100
L02245                else
L02246                    goto 100
L02247                endif
L02248             endif
L02249          endif
L02250
L02251  100     continue
L02252      return
L02253      end subroutine reghum
```

讲解：恒温器/湿度调节器的完整状态机：①sw_cond=0 或不在时间窗（nhourday∈[timeon,timeoff]）→ need=0；② 先算「无空调时的假想温度」templev=tlev+(dt/Qb)·hsroo；③|templev−targ|≤gap（死区）→ need=0；④ 越上界：need=hsroo−(Qb/dt)(targ+gap−tlev)（一步拉回死区边的热量需求），若需求速率超 temp_rat 则限速 hsneed=hsroo+temp_rat·Qb；⑤ 越下界对称。reghum 同构（容积×ρ×latent 代替 Qb）。**这套「一步拉回 +限速」是空调废热 hsout 的唯一来源——标定 targtemp/gaptemp 就是标定空调废热的时空分布（论文 2 CAL-STEAM 的直接抓手）。**

```
!file: phys/module_sf_bem.F    L2258-L2312    (subroutine air_cond)
L02258           subroutine air_cond(hsneed,hlneed,dt,hsout,hlout,consump,cop)
L02259
L02260           implicit none
L02261
L02262  !
L02263  !Performance of the air conditioning system
L02264  !
L02265  !INPUT/OUTPUT VARIABLES
L02266           real, intent(in) :: cop
L02267  !
L02268  !INPUT/OUTPUT VARIABLES
L02269  !
L02270           real hsneed     !sensible heat that is necessary for cooling/heating
L02271                           !the indoor air temperature [W]
L02272           real hlneed     !latent heat that is necessary for controling
L02273                           !the humidity of the indoor air [W]
L02274           real dt         !time step [s]
L02275  !
L02276  !OUTPUT VARIABLES
L02277  !
L02278           real hsout      !sensible heat pumped out into the atmosphere [W]
L02279           real hlout      !Latent heat pumped out into the atmosphere [W]
L02280           real consump    !Electrical consumption of the air conditioning system [W]
L02281
L02282
L02283  !
L02284  !Performance of the air conditioning system
L02285  !
L02286           if (hsneed.gt.0) then         ! air conditioning is cooling
L02287                                         ! and the heat is pumped out into the atmosphere
L02288         hsout=(1/cop)*(abs(hsneed)+abs(hlneed))+hsneed
L02289            hlout=hlneed
L02290            consump=(1./cop)*(abs(hsneed)+abs(hlneed))
L02291  !!        hsout=0.
L02292  !!        hlout=0.
L02293
L02294           else if(hsneed.eq.0.) then !air conditioning is not working to regulate the indoor temperature
L02295                 hlneed=0.       !no humidity regulation is considered
L02296                 hsout=0.        !no output into the atmosphere (sensible heat)
L02297                 hlout=0.        !no output into the atmosphere (latent heat)
L02298                 consump=0.      !no electrical consumption
L02299
L02300                else  !! hsneed < 0. !air conditioning is heating
L02301                 hlneed=0.           !no humidity regulation is considered
L02302                 hlout=0.            !no output into the atmosphere (latent heat)
L02303                 consump=(1./cop)*(abs(hsneed)+abs(hlneed))
L02304  !
L02305  !!We have two possibilities
L02306  !
L02307  !!              hsout=(1./cop)*(abs(hsneed)+abs(hlneed)) !output into the atmosphere
L02308                  hsout=0.                                 !no output into the atmosphere
L02309           end if
L02310
L02311           return
L02312           end subroutine air_cond
```

讲解：COP 账本：制冷（hsneed>0）：废热 hsout=(1/cop)·(|hsneed|+|hlneed|)+hsneed——压缩机耗电 1/cop 变废热排大气 + 从室内搬出的 hsneed；除湿 hlneed 直接排（冷凝水离开系统）；耗电 consump=(1/cop)(|hsneed|+|hlneed|)。制热（hsneed<0）：耗电同式，废热 hsout=0（注释保留了「废热 =耗电」的备选式）。hsneed=0 时全零。

```
!file: phys/module_sf_bem.F    L2317-L2363    (subroutine consump_total)
L02317           subroutine consump_total(nzcanm,nlev,consumpbuild,hsoutbuild, &
L02318                                    hsout,consump)
L02319
L02320           implicit none
L02321
L02322  !-----------------------------------------------------------------------
L02323  !Compute the total consumption in kWh/s (1kWh=3.6e+6 J) and sensible heat
L02324  !ejected into the atmosphere per building
L02325  !-----------------------------------------------------------------------
```

```
L02326  !
L02327  !INPUT VARIABLES
L02328  !
L02329  !
L02330          integer nzcanm            !Maximum number of vertical levels in the urban grid
L02331          real hsout(nzcanm)        !sensible heat emitted outside the room [W]
L02332          real consump(nzcanm)      !Electricity consumption for the a.c. in each floor[W]
L02333  !
L02334  !OUTPUT VARIABLES
L02335  !
L02336      real consumpbuild         !Energetic consumption for the entire building[kWh/s]
L02337          real hsoutbuild           !Total sensible heat ejected into the atmosphere
L02338                                    !by the air conditioning systems per building [W]
L02339  !
L02340  !LOCAL  VARIABLES
L02341  !
L02342          integer ilev
L02343
L02344  !
L02345  !INPUT VARIABLES
L02346  !
L02347          integer nlev
L02348
L02349  !
L02350  !INITIALIZE VARIABLES
L02351  !
L02352          consumpbuild=0.
L02353          hsoutbuild=0.
L02354  !
L02355          do ilev=1,nlev
L02356             consumpbuild=consumpbuild+consump(ilev)
L02357             hsoutbuild=hsoutbuild+hsout(ilev)
L02358          enddo !ilev
L02359
L02360          consumpbuild=consumpbuild/(3.6e+06)
L02361
L02362          return
L02363          end subroutine consump_total
```

讲解：全楼汇总：consumpbuild=Σconsump/(3.6e6)（W→kWh/s）、hsoutbuild=Σhsout（W）。上游 BEP_BEM 再乘 meso_urb_ac 折算到格点（14a § 四 ⑤）。

九、改造入口（论文 2 对接）

1. **标量 → 逐格场**（论文 2 主线）：cop/beta/sw_cond/timeon/timeoff/targtemp/gaptemp/targhum/gaphum/perflo/hsesf/hsequip/irho 目前全是「每类一标量」（init_para 从 TBL 读，经 BEP1D 的 `*_u(iurb)` 传入）。改造路径：①Registry 增 urban_map_* 维度与 3D 场（先例：ind_* 折叠表 +urban_map_zwd，14a § 四 ①）；②module_sf_urban 的 TBL 读入处加 auxinput 通道；③BEP_BEM 主入口的聚合式（meso_urb_ac× 类参数）同步改为逐格。**性能陷阱：wall/wall_gr 的 swwal+save 系数缓存按类失效——逐格参数下场化后必须逐点缓存或重算。**
2. **扩展点**：phirat 占用率昼夜曲线；fluxvent 换 ACH/风压驱动；effpv 与 PV 三层材料参数；temp_rat/hum_rat 限速。
3. **验证锚点**：室内温度 tlev/空调耗电 consump/废热 hsout 三诊断量与观测 TG、室内温度（论文 2 W240）对标；单点 sanity 见 ch19/ch17 的对照跑设计（sf_urban_physics=0 vs 3）。

十、上下游链接

- 上游调用者：`code_14a_module_sf_bep_bem.md`（BEP1D 的 BEM 循环）——实参表就是本篇 § 四的接口契约。
- 同族：`code_15_module_sf_bep.md`（BEP 单独版，不调本文件）；共享底座 `code_14b_module_sf_bep_bem.md`（辐射/导热/视野因子）；参数库 module_sf_urban（17 篇待交付）。
- 驱动链：`code_22_module_pbl_driver.md`；面层 `code_10_module_sf_mynnsfc.md`。
- 教材正文：ch17_urban_canopy.md（BEM 专节）、ch19_lanzhou_heatwave.md（城市冠层对照双跑）、ch20_modifying_wrf.md（三级改造实录）。

code_17 — 单层城市冠层模式 SLUCM（module_sf_urban.F：主入口选段与参数表）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_urban.F	5620	ad16e13c8f24b30c9fa815fe10702db1

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B，节选）：文件头 + 主入口五段 + SFCDIF_URB 头段 + 树冠两件；**教材 ch17_urban_canopy.md 已对 SLUCM 做逐段精读，本篇给完整源码上下文与总表**。
- **版本警示**：本地副本含 **urban-nbs 扩展**——TREEOPTION 树冠方案、GROPTION 绿屋顶、vegetated ground、人为潜热 alhoption 等带 `urban-nbs` 标签，且有 2026-01 的活注释（Chenghao/Yuqi）。与官方 tarball 的差异未逐行对照，讲解按本地副本为准。

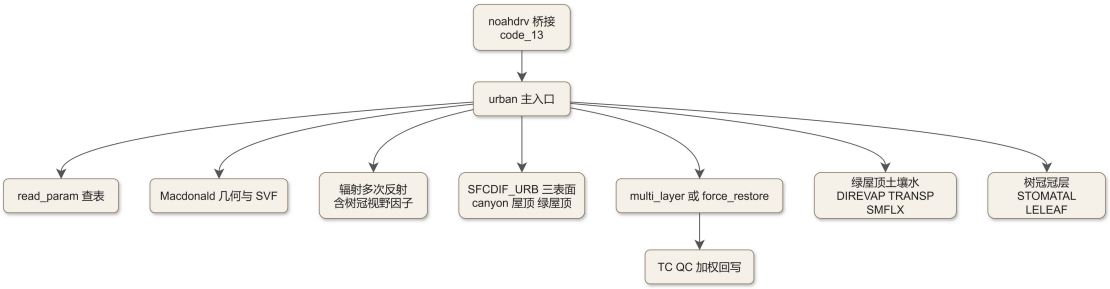
二、定位

一句话：Kusaka 2002 单层城市冠层模式（SLUCM）的 WRF 版（Fei Chen/Tewari 2006 维护）——一个峡谷单元（屋顶/墙/路面三表面 +峡谷空气）算城市能量收支，30 个子程序；本地副本叠加了 urban-nbs 的树冠与绿屋顶物理。

谁在调它：noahdrv 的 urban 前处理与 UCM 桥接（`code_13_module_sf_noahdrv.md` §4.3/§4.6——FRC 加权混合公式在那里）；参数表由 urban_param_init 从 urban_param.tbl 读入。**三方案分工**：单层 UCM（本篇）/ BEP（`code_15_module_sf_bep.md`）/BEP+BEM（`code_14a_module_sf_bep_bem.md`）。

30 单元分组总表：

组	单元	行段	说明
主链	urban	L323–L2539	SLUCM 主入口（2217 行单体）
交换	mos / louis79 / louis82 / SFCDIF_URB	L2545–L2816, L4108–L4312	峡谷内壁面交换三式 +城市面层
参数	read_param / urban_param_init / urban_var_init	L2822–L4044	urban_param.tbl 读取与 Registry 装载
复用 Noah 副本	DIREVAP/TRANSP/SMFLX/SRT/SSTEP/WDFCND/ROSR12/SHFLX/HRT/HSTEP/TBND/TDFCND	L4318–L5083	绿屋顶/植被地土壤水热的本地裁剪版
表面温度	force_restore / bisection / multi_layer	L4050–L4105, L2741–L2816	力恢复（含二分迭代）vs 多层热传导
树冠（urban-nbs）	SHADOW_TREE / STOMATAL / LELEAF / force_restore_tree / ROOT_UPTAKE / VF_TREE_ANALYTICAL / kanda_kawai_svf	L5165–L5617	阴影/气孔/叶面/树冠热储/根吸水/视野因子



三、文件头：参数表存储与三套扩展开关

```
!file: phys/module_sf_urban.F    L1-L129
L00001  MODULE module_sf_urban
L00002  #if defined(mpas)
L00003  use mpas_atmphys_utilities, only: physics_message,physics_error_fatal
L00004  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00005  #define WRITE_MESSAGE(M) call physics_message( M )
L00006  #else
L00007  use module_wrf_error
L00008  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00009  #define WRITE_MESSAGE(M) call wrf_message( M )
L00010  #endif
L00011  use module_model_constants, only :  piconst
L00012  !=================================================================================
L00013  !     Single-Layer Urban Canopy Model for WRF Noah-LSM
L00014  !     Original Version: 2002/11/06 by Hiroyuki Kusaka
L00015  !     Last Update:      2006/08/24 by Fei Chen and Mukul Tewari (NCAR/RAL)
L00016  !=================================================================================
L00017
L00018     CHARACTER(LEN=4)                :: LU_DATA_TYPE
L00019
L00020     INTEGER                         :: ICATE
L00021
L00022     REAL, ALLOCATABLE, DIMENSION(:) :: ZR_TBL
L00023     REAL, ALLOCATABLE, DIMENSION(:) :: Z0C_TBL
L00024     REAL, ALLOCATABLE, DIMENSION(:) :: Z0HC_TBL
L00025     REAL, ALLOCATABLE, DIMENSION(:) :: ZDC_TBL
L00026     REAL, ALLOCATABLE, DIMENSION(:) :: SVF_TBL
L00027     REAL, ALLOCATABLE, DIMENSION(:) :: R_TBL
L00028     REAL, ALLOCATABLE, DIMENSION(:) :: RW_TBL
L00029     REAL, ALLOCATABLE, DIMENSION(:) :: HGT_TBL
L00030     REAL, ALLOCATABLE, DIMENSION(:) :: AH_TBL
L00031     REAL, ALLOCATABLE, DIMENSION(:) :: ALH_TBL
L00032     REAL, ALLOCATABLE, DIMENSION(:) :: BETR_TBL
L00033     REAL, ALLOCATABLE, DIMENSION(:) :: BETB_TBL
L00034     REAL, ALLOCATABLE, DIMENSION(:) :: BETG_TBL
L00035     REAL, ALLOCATABLE, DIMENSION(:) :: FRC_URB_TBL
L00036
L00037     REAL, ALLOCATABLE, DIMENSION(:) :: COP_TBL
L00038     REAL, ALLOCATABLE, DIMENSION(:) :: BLDAC_FRC_TBL
L00039     REAL, ALLOCATABLE, DIMENSION(:) :: COOLED_FRC_TBL
L00040     REAL, ALLOCATABLE, DIMENSION(:) :: PWIN_TBL
L00041     REAL, ALLOCATABLE, DIMENSION(:) :: BETA_TBL
L00042     INTEGER, ALLOCATABLE, DIMENSION(:) :: SW_COND_TBL
L00043     REAL, ALLOCATABLE, DIMENSION(:) :: TIME_ON_TBL
L00044     REAL, ALLOCATABLE, DIMENSION(:) :: TIME_OFF_TBL
L00045     REAL, ALLOCATABLE, DIMENSION(:) :: TARGTEMP_TBL
L00046     REAL, ALLOCATABLE, DIMENSION(:) :: GAPTEMP_TBL
L00047     REAL, ALLOCATABLE, DIMENSION(:) :: TARGHUM_TBL
L00048     REAL, ALLOCATABLE, DIMENSION(:) :: GAPHUM_TBL
L00049     REAL, ALLOCATABLE, DIMENSION(:) :: PERFLO_TBL
L00050     REAL, ALLOCATABLE, DIMENSION(:) :: PV_FRAC_ROOF_TBL !GRZ
L00051     REAL, ALLOCATABLE, DIMENSION(:) :: GR_FRAC_ROOF_TBL !GRZ
L00052     INTEGER :: GR_FLAG_TBL !GRZ
L00053     INTEGER :: GR_TYPE_TBL !GRZ
L00054     REAL, DIMENSION(1:24)           :: IRHO_TBL
L00055     REAL, ALLOCATABLE, DIMENSION(:) :: HSESF_TBL
L00056
L00057     REAL, ALLOCATABLE, DIMENSION(:) :: CAPR_TBL, CAPB_TBL, CAPG_TBL
L00058     REAL, ALLOCATABLE, DIMENSION(:) :: AKSR_TBL, AKSB_TBL, AKSG_TBL
L00059     REAL, ALLOCATABLE, DIMENSION(:) :: ALBR_TBL, ALBB_TBL, ALBG_TBL
L00060     REAL, ALLOCATABLE, DIMENSION(:) :: EPSR_TBL, EPSB_TBL, EPSG_TBL
L00061     REAL, ALLOCATABLE, DIMENSION(:) :: Z0R_TBL,  Z0B_TBL,  Z0G_TBL
L00062     REAL, ALLOCATABLE, DIMENSION(:) :: SIGMA_ZED_TBL
L00063     REAL, ALLOCATABLE, DIMENSION(:) :: Z0HB_TBL, Z0HG_TBL
L00064     REAL, ALLOCATABLE, DIMENSION(:) :: TRLEND_TBL, TBLEND_TBL, TGLEND_TBL
L00065     REAL, ALLOCATABLE, DIMENSION(:) :: AKANDA_URBAN_TBL
L00066
L00067  !for BEP
L00068
L00069     ! MAXDIRS :: The maximum number of street directions we're allowed to define
L00070     INTEGER, PARAMETER :: MAXDIRS = 3
L00071     ! MAXHGTS :: The maximum number of building height bins we're allowed to define
L00072     INTEGER, PARAMETER :: MAXHGTS = 50
L00073
L00074     INTEGER, ALLOCATABLE, DIMENSION(:)   :: NUMDIR_TBL
L00075     REAL,    ALLOCATABLE, DIMENSION(:,:) :: STREET_DIRECTION_TBL
L00076     REAL,    ALLOCATABLE, DIMENSION(:,:) :: STREET_WIDTH_TBL
L00077     REAL,    ALLOCATABLE, DIMENSION(:,:) :: BUILDING_WIDTH_TBL
L00078     INTEGER, ALLOCATABLE, DIMENSION(:)   :: NUMHGT_TBL
L00079     REAL,    ALLOCATABLE, DIMENSION(:,:) :: HEIGHT_BIN_TBL
L00080     REAL,    ALLOCATABLE, DIMENSION(:,:) :: HPERCENT_BIN_TBL
L00081  !end BEP
L00082     INTEGER                         :: BOUNDR_DATA,BOUNDB_DATA,BOUNDG_DATA
```

```
L00083     INTEGER                          :: CH_SCHEME_DATA, TS_SCHEME_DATA
L00084     INTEGER                          :: ahoption        ! Miao, 2007/01/17, cal. ah
L00085     REAL, DIMENSION(1:24)            :: ahdiuprf        ! ah diurnal profile, tloc: 1-24
L00086     REAL, DIMENSION(1:24)            :: hsequip_tbl
L00087
L00088  !===Yang, 2014/10/08, urban hydrological processes for single layer UCM===
L00089     INTEGER                          :: IMP_SCHEME, IRI_SCHEME
L00090     INTEGER                          :: alhoption       ! anthropogenic latent heat option
L00091     INTEGER                          :: groption        ! anthropogenic latent heat option
L00092     LOGICAL                          :: distributed_aerodynamics_option
L00093     REAL                             :: fgr             ! green roof fraction
L00094     REAL                             :: oasis           ! urban oasis parameter
L00095     REAL, DIMENSION(1:4)             :: DZGR            ! Layer depth of green roof
L00096     REAL, DIMENSION(1:4)             :: alhseason       ! seasonal variation of alh
L00097     REAL, DIMENSION(1:48)            :: alhdiuprf       ! alh diurnal profile, tloc2: 1-48
L00098     REAL, DIMENSION(1:3)             :: porimp          ! porosity of pavement over impervious surface
L00099     REAL, DIMENSION(1:3)             :: dengimp         ! maximum water-holding depth of pavement
L00100
L00101  !===end hydrological processes===
L00102
L00103  !
L00104
L00105  !===Huang and Wang, 2025/12/28, urban nature-based solutions for single layer UCM (including trees)===
L00106     INTEGER                          :: TREEOPTION      ! Urban Tree&Vegetation Option ! urban-nbs
L00107     INTEGER                          :: TTscheme        ! Tree temperature solution   [0: diagnostic, 1: prognostic]
L00108     REAL, ALLOCATABLE, DIMENSION(:) :: FVG_TBL          ! vegetated ground fraction from URBPARM
L00109     REAL, ALLOCATABLE, DIMENSION(:) :: RTREE_TBL        ! Tree crown radius [normalized]
L00110     REAL, ALLOCATABLE, DIMENSION(:) :: RTREE_M_TBL      ! Tree crown radius in meter
L00111     REAL, ALLOCATABLE, DIMENSION(:) :: RW_M_TBL         ! Road width in meter
↪
L00112     REAL, ALLOCATABLE, DIMENSION(:) :: HTREE_TBL        ! Tree crown center height [normalized]
L00113     REAL, ALLOCATABLE, DIMENSION(:) :: DTREE_TBL        ! Tree crown center distance from wall (half canyon width
↪  currently) [normalized]
L00114     REAL, ALLOCATABLE, DIMENSION(:) :: LAI_TREE_TBL     ! Tree leaf area index
L00115     REAL, ALLOCATABLE, DIMENSION(:) :: LAI_VEG_TBL      ! Vegetated ground leaf area index
L00116     REAL, ALLOCATABLE, DIMENSION(:) :: Z0VG_TBL         ! Roughness length for momentum of vegetated ground
L00117     REAL, ALLOCATABLE, DIMENSION(:) :: Z0HVG_TBL        ! Roughness length for heat of vegetated ground
L00118     REAL, ALLOCATABLE, DIMENSION(:) :: CAPVG_TBL        ! Heat capacity of vegetated ground
L00119     REAL, ALLOCATABLE, DIMENSION(:) :: EPSVG_TBL        ! Emissivity of vegetated ground
L00120     REAL, ALLOCATABLE, DIMENSION(:) :: EPST_TBL         ! Emissivity of tree canopy
L00121     REAL, ALLOCATABLE, DIMENSION(:) :: ALBVG_TBL        ! Emissivity of vegetated ground
L00122     REAL, ALLOCATABLE, DIMENSION(:) :: ALBT_TBL         ! Emissivity of tree canopy
↪
L00123  !===end urban nature-based solutions processes===
L00124
L00125     CHARACTER (LEN=256) , PRIVATE    :: mesg
L00126
L00127     CONTAINS
L00128
L00129  !================================================================================
```

讲解：① **模块级 TBL 数组即 SLUCM 的全部可调参数**：峡谷形态表（ZR/Z0C/Z0HC/ZDC/SVF/R/RW/HGT——屋顶宽/路宽/楼高/峡谷高宽比/天空视野因子）、人为热（AH/ALH——W/m² 表值内部换 cgs）、反照率三表面（BETR/BETB/BETG）、空调三件（COP/BLDAC_FRC/COOLED_FRC）、窗户组（PWIN/BETA/SW_COND/TIME_ON/TIME_OFF/TARGTEMP/GAPTEMP/TARGHUM/GAPHUM/PERFLO——空调调度全表）、材料与热力（CAPR/AKSR/ALBR/EPSR/Z0R 及 B/G 对、SIGMA_ZED、TRLEND/TBLEND/TGLEND 终温、AKANDA_URBAN_TBL）；②**BEP 共享段**（L67–L81）：MAXDIRS=3/MAXHGTS=50 与街道方向/宽度、楼高 bin 表——BE 路线的形态参数也存这张表（三方案共用 urban_param.tbl 的入口面）；③**Yang 2014 水文开关**（IMP_SCHEME/IRI_SCHEME/alhoption/groption/oasis/fgr/DZGR）与 **urban-nbs 树冠段**（TREEOPTION、DTREE/HTREE/LAI_TREE/RTREE_M 等）——两代扩展在此叠加；④ 文件头自注版本史：Kusaka 2002 原版、Chen/Tewari 2006-08 最后更新——**2006 之后的一切（含 urban-nbs）都是社区叠加**。

四、主入口选段五处

4.1 Macdonald 峡谷几何与 NUDAPT SVF

```
!file: phys/module_sf_urban.F    L854-L905
L00854    !Calculate the following urban canyon geometry parameters following Macdonald's (1998) formulations
L00855        Cd         = 1.2
L00856        alpha_macd = 4.43
L00857        beta_macd  = 1.0
L00858
L00859
L00860        ZDC = ZR * ( 1.0 + ( alpha_macd ** ( -R ) )  * ( R - 1.0 ) )             ! e.q 23 in Macdonald's (1998)
L00861
L00862        Z0C = ZR * ( 1.0 - ZDC/ZR ) * &
L00863             exp (-(0.5 * beta_macd * Cd / (VonK**2) * ( 1.0-ZDC/ZR) * lambda_f )**(-0.5))         ! e.q 26 in
↪  Macdonald's (1998)
L00864
L00865        if(zo_check.eq.1)THEN
L00866          write(mesg,*) 'Roughness Length NUDAPT',Z0C
L00867          WRITE_MESSAGE(mesg)
L00868        end if
L00869
L00870        lambda_fr  = stdh_urb/(SW + BW)
L00871
L00872         Z0R = ZR * ( 1.0 - ZDC/ZR) &
L00873               * exp ( -(0.5 * beta_macd * Cd / (VonK**2) &
L00874                * ( 1.0-ZDC/ZR) * lambda_fr )**(-0.5))
L00875
```

```
L00876
L00877
L00878         Z0HC = 0.1 * Z0C
L00879
L00880     ! Calculate Sky View Factor
L00881
L00882        DHGT=HGT/100.
L00883        HGT2=0.
L00884        VFWS=0.
L00885        HGT2=HGT-DHGT/2.
L00886         do NUDAPT=1,99
L00887            HGT2=HGT2-DHGT
L00888            VFWS=VFWS+0.25*(1.-HGT2/SQRT(HGT2**2.+RW**2.))
L00889         end do
L00890
L00891        VFWS=VFWS/99.
L00892        VFWS=VFWS*2.
L00893
L00894        VFGS=1.-2.*VFWS*HGT/RW
L00895        SVF=VFGS
L00896
L00897       if(zo_check.eq.1)THEN
L00898         write(mesg,*) 'Roof Roughness Length NUDAPT',Z0R
L00899         WRITE_MESSAGE(mesg)
L00900         write(mesg,*) 'Sky View Factor NUDAPT',SVF
L00901         WRITE_MESSAGE(mesg)
L00902         write(mesg,*) 'normalized Height NUDAPT', HGT
L00903         WRITE_MESSAGE(mesg)
L00904       end if
L00905
```

讲解：①**Macdonald 1998 亲本构公式**：零平面位移 ZDC=ZR • (1+α^(−R) • (R−1))（式 23）、峡谷粗糙度 Z0C=ZR • (1−ZDC/ZR) • exp(−[0.5βCd • (1−ZDC/ZR) • λ_f/k²]^(1/2))（式 26）——λ_f 是**迎风面密度且按风向分档**（WDR 八向查 lf_urb，L841–852 的四象限二分）；屋顶粗糙度 Z0R 用 λ_fr=stdh/(SW+BW) 另算、Z0HC=0.1 • Z0C（热粗糙度十分之一律）；②**SVF 的 NUDAPT 数值积分**：把墙面按 DHGT=HGT/100 切 99 段，逐段 VF=0.25 • (1−H/√(H²+RW²)) 取平均乘 2 得墙面 SVF，路面 VFGS=1−2 • VFWS • HGT/RW——99 段足够收敛且每步重算（canyon 几何随 morph 查表固定，但这段在主循环里）；③zo_check 诊断打印是调试开关痕迹。

4.2 屋顶温度：二分迭代与双方案分派

```
!file: phys/module_sf_urban.F    L1296-L1325
L01296          DTR = F/DFDT
L01297
L01298          TR = TRP - DTR
L01299          TRP = TR
L01300
L01301          IF( ABS(F) < 0.000001 .AND. ABS(DTR) < 0.000001 ) EXIT
L01302
L01303        END DO
L01304
L01305   ! multi-layer heat equation model
L01306
L01307        CALL multi_layer(num_roof_layers,BOUNDR,G0R,CAPR,AKSR,TRL,DZR,DELT,TRLEND)
L01308
L01309      ELSE
L01310
L01311        ES=6.11*EXP( (2.5*10.**6./461.51)*(TRP-273.15)/(273.15*TRP) )
L01312        QS0R=0.622*ES/(PS-0.378*ES)
L01313
L01314        RR=EPSR*(RX-SIG*(TRP**4.)/60.)
L01315        HR=RHO*CP*CHR*UA*(TRP-TA)*100.
L01316        ELER=RHO*EL*CHR*UA*BETR*(QS0R-QA)*100.
L01317        G0R=SR+RR-HR-ELER
L01318
L01319        CALL force_restore(CAPR,AKSR,DELT,SR,RR,HR,ELER,TRLEND,TRP,TR)
L01320
L01321        TRP=TR
L01322
L01323      END IF
L01324
L01325      FLXTHR=HR/RHO/CP/100.
```

讲解：①TS_SCHEME 分派——**=1 走 multi_layer**（num_roof_layers 层热传导推进，CAPR/AKSR 热容与导热率来自 TBL）；**否则力恢复法**：先由表面饱和比湿 ES=6.11 • exp(Lv(T)/(Rv • T) • (T−273.15)/(273.15 • T))（Clausius–Clapeyron 积分式）、长波残差 RR=EPSR • (RX−σT⁴/60)、感热 HR 与潜热 ELER 组装净通量 G0R（**全 cgs 制**——HR=RHO • CP • CHR • UA • ΔT • 100 的 ×100 是 W 到 cal/(cm²s) 的换算），再调 force_restore；其上方的循环（L1290–L1302）是 TR 的**牛顿/二分混合迭代**（|F| 与 |DTR| 双容限 EXIT）；②FLXTHR/FLXHUMR 除回 RHO • CP • 100 把 cgs 通量转回模式语言——SLUCM 全程 cgs 与 SI 双制并行的换算点就在这些 ×100 与 /60（σT⁴ 的 /60 是 cal/(cm² • min) 老口径）。

4.3 绿屋顶段（urban-nbs）

```
!file: phys/module_sf_urban.F    L1327-L1355
L01327
L01328  !-----------------------------------------------------------------------------
L01329  ! Green Roof
L01330  ! Must use multiple layers scheme (TS_SCHEME=1)
L01331  !-----------------------------------------------------------------------------
L01332     IF (GROPTION == 1) THEN
L01333     T1VGR = TGRP* (1.0+ 0.61 * QA)
L01334     RLMO_URB=0.0
L01335     CALL SFCDIF_URB (ZA,Z0R,T1VGR,TH2V,UA,AKANDA_URBAN,CMGR_URB,CHGR_URB,RLMO_URB,CDGR)
L01336     ALPHAGR =  RHO*CP*CHGR_URB
L01337     CHGR=ALPHAGR/RHO/CP/UA
L01338     RUNOFF1 = 0.0
L01339     RUNOFF2 = 0.0
L01340     RUNOFF3 = 0.0
L01341     SROOT   = 0.0
L01342
L01343     KZ = 1
L01344     ZSOILR (KZ) = - DZGR (KZ)
L01345     DO KZ = 2,NGR
L01346        ZSOILR (KZ) = - DZGR(KZ) + ZSOILR (KZ -1)
L01347     END DO
L01348
L01349     DO ITERATION=1,100
L01350         KZ=1
L01351         ES=6.11*EXP( (2.5*10.**6./461.51)*(TGRP-273.15)/(273.15*TGRP) )
L01352         DESDT=(2.5*10.**6./461.51)*ES/(TGRP**2.)
L01353         QS0GR=0.622*ES/(PS-0.378*ES)
L01354         DQS0GRDTGR = DESDT*0.622*PS/((PS-0.378*ES)**2.)
L01355         EPGR=RHOO*CHGR*UA*(QS0GR-QA)   ! Potential evaporation [kg/m2/s]
```

讲解：GROPTION==1 的绿屋顶链：① 强制注释「Must use multiple layers scheme」——绿屋顶依赖多层热传导；②T1VGR 虚温 + SFCDIF_URB 独立算绿屋顶交换系数（CHGR）；③**ZSOILR 负深累计**——与 noahlsm 完全同款的层深约定（code_12a § 四 ①）；④ 后续（L1356–L1400）直接调 **DIREVAP/TRANSP/SMFLX**——注意这三个是**本文件 L4318 起的本地副本**（noahlsm 同名单元的裁剪版），绿屋顶土壤水走 Noah 型四层收支——**单层 UCM 的绿屋顶 =把 Noah 的土壤水模块塞进 SLUCM**，是 ch20「方案杂交」的现成案例。

4.4 峡谷交换与 CH_SCHEME

```
!file: phys/module_sf_urban.F    L1451-L1467
L01451     IF (.not. distributed_aerodynamics_option) THEN
L01452
L01453     T1VC = TCP* (1.0+ 0.61 * QA)
L01454     RLMO_URB=0.0
L01455     CALL SFCDIF_URB(ZA,Z0C,T1VC,TH2V,UA,AKANDA_URBAN,CMC_URB,CHC_URB,RLMO_URB,CDC)      ! CDC: momentum exchange
↪  coefficient for canopy
L01456     ALPHAC =  RHO*CP*CHC_URB
L01457
L01458     IF (CH_SCHEME == 1) THEN
L01459
L01460       Z=ZDC
L01461       BHB=LOG(Z0B/Z0HB)/0.4
L01462       BHG=LOG(Z0G/Z0HG)/0.4
L01463       RIBB=(9.8*2./(TCP+TBP))*(TCP-TBP)*(Z+Z0B)/(UC*UC)
L01464       RIBG=(9.8*2./(TCP+TGP))*(TCP-TGP)*(Z+Z0G)/(UC*UC)
L01465
L01466       CALL mos(XXXB,ALPHAB,CDB,BHB,RIBB,Z,Z0B,UC,TCP,TBP,RHO)
L01467       CALL mos(XXXG,ALPHAG,CDG,BHG,RIBG,Z,Z0G,UC,TCP,TGP,RHO)
```

讲解：①**distributed_aerodynamics_option 为假**才走 SLUCM 自身的峡谷面层（真值时由外部分布拖曳接管——与 BEP 的 a_u/b_u 源项思想衔接的实验开关）；②SFCDIF_URB 三连调的第一次（峡谷 CDC/CHC_URB）；③**CH_SCHEME=1 走 mos**（Monin–Obukhov 的壁面/路面内层交换：BHB=ln(Z0B/Z0HB)/0.4、RIBB 峡谷 Richardson 数、调 mos 解 XXXB/ALPHAB）——否则走 louis79/louis82 幂律方案；峡谷空气温度 TCP 是三表面交换的共同枢纽。

4.5 收尾：多次反射、树冠与 TC/QC 加权

```
!file: phys/module_sf_urban.F    L2182-L2290
L02182         CALL multi_layer(num_wall_layers,BOUNDB,G0B,CAPB,AKSB,TBL,DZB,DELT,TBLEND)
L02183         CALL multi_layer(num_road_layers,BOUNDG,G0G,CAPG,AKSG,TGL,DZG,DELT,TGLEND)
L02184         ! CALL multi_layer(num_road_layers,BOUNDG,G0VG,CAPVG,KVG/697.7/60,TVGL,DZVG*0.01,DELT,TGLEND) ! DZVG unit ([m] as
↪  input, needed to convert to[cm]) (Chenghao 2026/01/20)
L02185         CALL SHFLX (SSOILG,TVGL,SMG,SMCMAX,NVG,TVGP,DELT,YY,ZZ1,ZSOILG,TGLEND,ZBOT,SMCWLT,KVG,QUARTZ,CSOIL,CAPVG)   !
↪  Update temperature in vegetated ground soil layer (urban-nbs)
L02186
L02187
L02188      ELSE
L02189         !-----------------------------------------------------------------------
L02190         !  TB, TG  by Force-Restore Method
L02191         !  Note, Conflict With Vegetated Ground Scheme (urban-nbs)
L02192         !-----------------------------------------------------------------------
L02193
L02194         ES=6.11*EXP( (2.5*10**6/461.51)*(TBP-273.15)/(273.15*TBP) )
L02195         QS0B=0.622*ES/(PS-0.378*ES)
L02196
L02197         ES=6.11*EXP( (2.5*10**6/461.51)*(TGP-273.15)/(273.15*TGP) )
```

```
L02198          QS0G=0.622*ES/(PS-0.378*ES)
L02199
L02200          ES=6.11*EXP( (2.5*10**6/461.51)*(TTP-273.15)/(273.15*TTP) )
L02201          QS0T=0.622*ES/(PS-0.378*ES)
L02202
L02203          ES=6.11*EXP( (2.5*10**6/461.51)*(TVGP-273.15)/(273.15*TVGP) )   ! following green roof scheme (urban-nbs)
L02204          QS0VG=0.622*ES/(PS-0.378*ES)
L02205
L02206          EPSGE   = (1 - FVG) * EPSG + FVG * EPSVG           ! DEFINE NEW EPSVG HERE (urban-nbs)
L02207          TGEP    = (1 - FVG) * TGP  + FVG * TVGP
L02208          QS0GE   = (1 - FVG) * QS0G + FVG * QS0VG
L02209          ALPHAGE = (1 - FVG) * ALPHAG + FVG * ALPHAVG
L02210
L02211          EMITW = EPSB * SIG * TBP**4/60
L02212          EMITG = EPSGE * SIG * TGEP**4/60
L02213          EMITT = EPST * SIG * TTP**4/60
L02214
L02215          RG1 = EPSG * (RX * VFGS_TREE + 2 * EMITW * VFGW_TREE + &
L02216                EMITT * VFGT_TREE -  &
L02217                SIG * TGP**4/60)
L02218
L02219          RVG1= EPSVG * (RX * VFGS_TREE + 2 * EMITW * VFGW_TREE + &
L02220                EMITT * VFGT_TREE - &
L02221                SIG * TVGP**4/60)
L02222
L02223          RB1 = EPSB * (RX * VFWS_TREE + EMITW * VFWW_TREE + &
L02224                EMITG * VFWG_TREE + EMITT * VFWT_TREE - &
L02225                SIG * TBP**4/60)
L02226
L02227          RG2 = EPSG * ((2 * (RX * VFWS_TREE + EMITW * VFWW_TREE + &
L02228                EMITG * VFWG_TREE + EMITT * VFWT_TREE) * (1 - EPSB) * VFGW_TREE) + &
L02229                ((RX * VFTS_TREE + 2 * EMITW * VFTW_TREE + &
L02230                EMITG * VFTG_TREE) * (1 - EPST) * VFGT_TREE))
L02231
L02232          RVG2= EPSVG * ((2 * (RX * VFWS_TREE + EMITW * VFWW_TREE + &
L02233                EMITG * VFWG_TREE + EMITT * VFWT_TREE) * (1 - EPSB) * VFGW_TREE) + &
L02234                ((RX * VFTS_TREE + 2 * EMITW * VFTW_TREE + &
L02235                EMITG * VFTG_TREE) * (1 - EPST) * VFGT_TREE))
L02236
L02237          RB2 = EPSB * (((RX * VFGS_TREE + 2 * EMITW * VFGW_TREE + &
L02238                EMITT * VFGT_TREE) * (1 - EPSGE) * VFWG_TREE) + &
L02239                ((RX * VFWS_TREE + EMITW * VFWW_TREE + EMITG * VFWG_TREE + &
L02240                EMITT * VFWT_TREE) * (1 - EPSB) * VFWW_TREE) + &
L02241                ((RX * VFTS_TREE + 2 * EMITW * VFTW_TREE + &
L02242                EMITG * VFTG_TREE) * (1 - EPST) * VFWT_TREE))
L02243
L02244          RT1 = EPST * (RX * VFTS_TREE + 2 * EMITW * VFTW_TREE + &
L02245                EMITG * VFTG_TREE) - EMITT
L02246
L02247          RT2 = EPST * ((2 * (RX * VFWS_TREE + EMITW * VFWW_TREE + &
L02248                EMITG * VFWG_TREE + EMITT * VFWT_TREE) * (1 - EPSB) * VFTW_TREE) + &
L02249                ((RX * VFGS_TREE + 2 * EMITW * VFGW_TREE + &
L02250                EMITT * VFGT_TREE) * (1 - EPSGE) * VFTG_TREE))
L02251
L02252          RG = RG1 + RG2
L02253          RB = RB1 + RB2
L02254          RVG= RVG1+RVG2
L02255          RT = RT1 + RT2
L02256
L02257          HB=RHO*CP*CHB*UC*(TBP-TCP)*100
L02258          ELEB=RHO*EL*CHB*UC*BETB*(QS0B-QCP)*100
L02259          G0B=SB+RB-HB-ELEB     ! unit: cal/cm2/s
L02260
L02261          HG=RHO*CP*CHG*UC*(TGP-TCP)*100
L02262          ELEG=RHO*EL*CHG*UC*BETG*(QS0G-QCP)*100
L02263          G0G=SG+RG-HG-ELEG     ! unit: cal/cm2/s
L02264
L02265          RA_LEAF = 58 * (LAI_TREE**0.56) * SQRT(DLEAF/UC)
L02266          HT = (SLEAF + RLEAF - ELET - (TT - TTP)/DELT*640)/697.7/60. ! HT in unit of cal/cm2/s (unit consistent with HB
↪  and HG)
L02267          W2 = 0.5 * (SMGP(NVG/2-1) + SMGP(NVG/2))     ! POTENTIAL ERROR (Yuqi urban-nbs)
L02268          CALL STOMATAL(RS_LEAF, RSMIN, SLEAF, LAI_TREE, W2, SMCREF, SMCMAX, TC, TT, QC, QS0T)
L02269          CALL LELEAF(ELET, SLEAF, RLEAF, RA_LEAF, RS_LEAF, RHOO, TC, QC, QS0T)
L02270
L02271
L02272          CALL force_restore(CAPB,AKSB,DELT,SB,RB,HB,ELEB,TBLEND,TBP,TB)
L02273          CALL force_restore(CAPG,AKSG,DELT,SG,RG,HG,ELEG,TGLEND,TGP,TG)
L02274          CALL force_restore_tree(DELT,ST,RT,HT,ELET/(697.7*60),LAI_TREE,TTP,TT) ! to be fixed (Yuqi 01/21/2026)
L02275
L02276          TBP=TB
L02277          TGP=TG
L02278          TTP=TT
L02279          TC1 = RW * ALPHAC + RW * FVG * ALPHAVG + RW * (1 - FVG) * ALPHAG + W * ALPHAB
L02280          TC2 = RW * ALPHAC * TA + RW * FVG * ALPHAVG * TVGP + RW * (1 - FVG) * ALPHAG * TGP + W * ALPHAB * TBP + 2 *
↪  RTREE * LAI_TREE * (HT/100)
L02281          TC = TC2 / TC1
L02282           ! Chenghao: HT unit is cgs, keep it as is [RHO - g/cm3, CP - cal/g/K]
L02283
L02284          QC1 = W * ALPHAB * BETB + RW * (1 - FVG) * ALPHAG * BETG + RW * ALPHAC
L02285          QC2 = W * ALPHAB * BETB * QS0B + RW * (1 - FVG) * ALPHAG * BETG * QS0G + RW * FVG * ETAG *CP / 1000 + RW *
↪  ALPHAC * QA + (2 * RTREE * LAI_TREE * CP * (ELET/697.7/60.)/100/EL)
L02286          QC=QC2/QC1
L02287           ! Chenghao: ELET - [W/m2], convert it to cgs to be consistent with all other terms (e.g., ALPHAB)
L02288
L02289          TCP=TC
L02290          QCP=QC
```

讲解：force-restore 分支的收尾全景（urban-nbs 版）：①multi_layer 推进墙/路＋**SHFLX 更新植被地土温**（「Conflict With Vegetated Ground Scheme」注释承认与力恢复法不兼容——nbs 的杂交缝）；② **树冠多次反射六项** RG1/RG2/RB1/RB2/RT1/RT2——每个表面先收直射（RX • VF*_TREE）与一次反射（2 • EMITW • VFGW_TREE 的 2 是两面墙），二阶项 RG2 等乘 (1−EPS) 反射率与对面 VF——树冠视野因子把原三表面辐射网扩成四节点；③ **树冠热储 HT**=(SLEAF+RLEAF−ELET−(TT−TTP)/DELT • 640)/697.7/60——640 cal 是树冠比热 • 生物量假设、697.7 是 W→cal/(cm² • s) 系数，W2 取

中层两点均值并自带「**POTENTIAL ERROR (Yuqi urban-nbs)**」警告；④force_restore_tree 带「**to be fixed**」活注释——本地副本是实验进行时；⑤ **峡谷空气温湿的加权平均**：TC=(RW • αC • TA+RW • FVG • αVG • TVGP+RW • (1−FVG) • αG • TGP+W • αB • TBP+树冠项)/Σα——TC/QC 是各表面按交换系数与面积权重混合的峡谷状态，回写给 surface_driver 的 TS_URB2D 系。

五、特色单元三件

```
!file: phys/module_sf_urban.F    L5165-L5174    (function kanda_kawai_svf)
L05165  FUNCTION kanda_kawai_svf(lp, lf) RESULT (svf)
L05166      IMPLICIT NONE
L05167      real, intent(in) :: lp, lf
L05168      real :: hovl, vloc, vmod, svf
L05169
L05170      hovl = lf * lp ** (-0.5) / (1. - lp ** 0.5)
L05171      vloc = cos(atan(2. * hovl)) * (2. - 4. / piconst * atan(cos(atan(2. * hovl))))
L05172      vmod = 0.1120 * lp * vloc - 0.4817 * lp + 0.0246 * vloc + 0.9570
L05173      svf = vloc * vmod
L05174    END FUNCTION kanda_kawai_svf
```

讲解：Kanda–Kawai 解析天空视野因子（10 行 function）——与 §4.1 的 NUDAPT 数值积分并存的第二口径（urban-nbs 树冠方案的 SVF 需要逐点解析值）。

```
!file: phys/module_sf_urban.F    L5182-L5236    (subroutine SHADOW_TREE)
L05182      SUBROUTINE SHADOW_TREE (XI, H, W, HTREE, RTREE, DTREE, CHI_SHADED, ETA_SHADED, &
L05183                              CHI_SHADED_TREE, ETA_SHADED_TREE)
L05184
L05185        IMPLICIT NONE
L05186
L05187        REAL, INTENT(IN)    :: XI , H, W, HTREE, RTREE, DTREE
L05188        REAL, INTENT(OUT)   :: CHI_SHADED, ETA_SHADED, CHI_SHADED_TREE, ETA_SHADED_TREE
L05189        REAL                :: XI_A, XI_B, X0, Y0, X1, X2, Y1, Y2, CHI_TREE, ETA_TREE
L05190        ! CHI: SHADOW ON THE GROUND;        ETA: SHADOW ON THE WALL
L05191
L05192        XI_A = SQRT(1 + XI**2)
L05193        XI_B = SQRT(1 + XI**(-2))
L05194
L05195        X0 = MAX(0.0, W - H * XI)
L05196        Y0 = MAX(0.0, H - W / XI)
L05197
L05198        X1 = MAX(0.0, DTREE - HTREE * XI - RTREE * XI_A)
L05199        X2 = MAX(0.0, DTREE - HTREE * XI + RTREE * XI_A)
L05200
L05201        Y1 = MAX(0.0, HTREE - DTREE / XI - RTREE * XI_B)
L05202        Y2 = MAX(0.0, HTREE - DTREE / XI + RTREE * XI_B)
L05203
L05204        CHI_TREE = X2 - X1
L05205        ETA_TREE = Y2 - Y1
L05206
L05207        !-----------------------------------------------------------------
L05208        ! SHADOW LENGTH ON THE GROUND BY WALL2 AND TREE
L05209        !-----------------------------------------------------------------
L05210        IF (X2 .le. X0) THEN
L05211           CHI_SHADED = W - X0 + CHI_TREE
L05212           CHI_SHADED_TREE = CHI_TREE
L05213        ELSEIF (X1 .le. X0 .and. X0 .lt. X2) THEN
L05214           CHI_SHADED = W - X0 + CHI_TREE - (X2 - X0)
L05215           CHI_SHADED_TREE = CHI_TREE - (X2 - X0)
L05216        ELSE
L05217           CHI_SHADED = W - X0
L05218           CHI_SHADED_TREE = 0
L05219        END IF
L05220
L05221        !-----------------------------------------------------------------
L05222        ! SHADOW LENGTH ON THE WALL BY WALL2 AND TREE
L05223        !-----------------------------------------------------------------
L05224        IF (Y2 .le. Y0) THEN
L05225           ETA_SHADED = Y0
L05226           ETA_SHADED_TREE = 0
L05227        ELSEIF (Y1 .le. Y0 .and. Y0 .lt. Y2) THEN
L05228           ETA_SHADED = Y2
L05229           ETA_SHADED_TREE = Y2 - Y0
L05230        ELSE
L05231           ETA_SHADED = Y0 + ETA_TREE
L05232           ETA_SHADED_TREE = ETA_TREE
L05233        END IF
L05234
L05235  ! ------------------------------------------------------------------
L05236      END SUBROUTINE SHADOW_TREE
```

讲解：树冠阴影投影（urban-nbs）：给定太阳方位把树冠（高 HTREE、半径 RTREE、冠心距 DTREE）投影到墙面/路面，返回各表面的阴影占比（CHI/ETA_SHADED 及树冠版）——与 code_14a 的 shadow_mas（BEP 三向阴影）同问题不同口径：SLUCM 单峡谷两墙、BEP 多方向多高度 bin。

```
!file: phys/module_sf_urban.F    L4108-L4167
L04108  SUBROUTINE SFCDIF_URB (ZLM,Z0,THZ0,THLM,SFCSPD,AKANDA,AKMS,AKHS,RLMO,CD,ZT_OUT,VEGFRAC)
L04109
L04110  ! ----------------------------------------------------------------------
L04111  ! SUBROUTINE SFCDIF_URB (Urban version of SFCDIF_off)
L04112  ! ----------------------------------------------------------------------
L04113  ! CALCULATE SURFACE LAYER EXCHANGE COEFFICIENTS VIA ITERATIVE PROCESS.
L04114  ! SEE CHEN ET AL (1997, BLM)
L04115  ! ----------------------------------------------------------------------
L04116
L04117        IMPLICIT NONE
```

```
L04118        REAL     WWST, WWST2, G, VKRM, EXCM, BETA, BTG, ELFC, WOLD, WNEW
L04119        REAL     PIHF, EPSU2, EPSUST, EPSIT, EPSA, ZTMIN, ZTMAX, HPBL,     &
L04120       & SQVISC
L04121        REAL     RIC, RRIC, FHNEU, RFC,RLMO_THR, RFAC, ZZ, PSLMU, PSLMS, PSLHU,     &
L04122       & PSLHS
L04123        REAL     XX, PSPMU, YY, PSPMS, PSPHU, PSPHS, ZLM, Z0, THZ0, THLM
L04124        REAL     SFCSPD, AKANDA, AKMS, AKHS, ZU, ZT, RDZ, CXCH
L04125        REAL     DTHV, DU2, BTGH, WSTAR2, USTAR, ZSLU, ZSLT, RLOGU, RLOGT
L04126        REAL     RLMO, ZETALT, ZETALU, ZETAU, ZETAT, XLU4, XLT4, XU4, XT4
L04127        REAL, INTENT(OUT), OPTIONAL :: ZT_OUT
L04128        REAL, INTENT(IN), OPTIONAL :: VEGFRAC
L04129  !CC   ......REAL ZTFC
L04130
L04131        REAL     XLU, XLT, XU, XT, PSMZ, SIMM, PSHZ, SIMH, USTARK, RLMN,  &
L04132       &         RLMA
L04133
L04134        INTEGER  ITRMX, ILECH, ITR
L04135        REAL,    INTENT(OUT) :: CD
L04136        PARAMETER                                                        &
L04137       &        (WWST = 1.2,WWST2 = WWST * WWST,G = 9.8,VKRM = 0.40,      &
L04138       &         EXCM = 0.001                                             &
L04139       &        ,BETA = 1./270.,BTG = BETA * G,ELFC = VKRM * BTG          &
L04140       &                  ,WOLD =.15,WNEW = 1. - WOLD,ITRMX = 05,         &
L04141       &                   PIHF = 3.14159265/2.)
L04142        PARAMETER                                                        &
L04143       &         (EPSU2 = 1.E-4,EPSUST = 0.07,EPSIT = 1.E-4,EPSA = 1.E-8  &
L04144       &         ,ZTMIN = -5.,ZTMAX = 1.,HPBL = 1000.0                    &
L04145       &          ,SQVISC = 258.2)
L04146        PARAMETER                                                        &
L04147       &       (RIC = 0.183,RRIC = 1.0/ RIC,FHNEU = 0.8,RFC = 0.191      &
L04148       &        ,RLMO_THR = 0.001,RFAC = RIC / (FHNEU * RFC * RFC))
L04149
L04150  ! ------------------------------------------------------------------
L04151  ! NOTE: THE TWO CODE BLOCKS BELOW DEFINE FUNCTIONS
L04152  ! THIS ROUTINE SFCDIF CAN HANDLE BOTH OVER OPEN WATER (SEA, OCEAN) AND
L04153  ! OVER SOLID SURFACE (LAND, SEA-ICE).
L04154  ! ------------------------------------------------------------------
L04155  ! ------------------------------------------------------------------
L04156  ! LECH'S SURFACE FUNCTIONS              ! E.q A7&A8
L04157  ! ------------------------------------------------------------------
L04158        PSLMU (ZZ)= -0.96* log (1.0-4.5* ZZ)
L04159        PSLMS (ZZ)= (ZZ/RFC) -2.076* (1. -1./ (ZZ +1.))
L04160        PSLHU (ZZ)= -0.96* log (1.0-4.5* ZZ)
L04161        PSLHS (ZZ)= ZZ * RFAC -2.076* (1. - exp(-1.2 * ZZ))        ! bug fixed in 4.7.1
L04162  ! ------------------------------------------------------------------
L04163  ! PAULSON'S SURFACE FUNCTIONS             ! E.q A5&A6
L04164  ! ------------------------------------------------------------------
L04165        PSPMU (XX)= -2.* log ( (XX +1.)*0.5) - log ( (XX * XX +1.)*0.5) +2.* ATAN (XX) - PIHF
L04166        PSPMS (YY)= 5.* YY
L04167        PSPHU (XX)= -2.* log ( (XX * XX +1.)*0.5)
```

讲解：**SFCDIF_URB=SFCDIF_off 的城市版**（对照 code_12b § 十四）：① 同一套 Chen 1997 迭代骨架（ITRMX=5、WOLD=0.15、SQVISC=258.2、EXCM=0.001 下限、WWST=1.2 对流速度）；②Lech 与 Paulson 语句函数并存，**PSLHS 注明「bug fixed in 4.7.1」**（稳定函数的 RFAC 项修正——两版 sfcdif 的实际差异点之一）；③ 扩展两个 OPTIONAL：ZT_OUT（热粗糙度回传——BEP 链要）与 VEGFRAC（urban-nbs 植被修正）；④AKANDA_URBAN 是城市形态参数化的附加项（Kanda 2007 系），随 TBL 装载。

六、改造入口

1. **城市参数表**：urban_param.tbl 全部经 read_param/urban_param_init/urban_var_init 三件装载——换城市形态/材料/空调调度只改表；SIGMA_ZED（NUDAPT 形态标准差）是 BEP 共享。
2. **温度方案**：TS_SCHEME=1 多层 vs 力恢复——绿屋顶强制前者；CH_SCHEME=1 mos vs louis 系。
3. **urban-nbs 实验**：TREEOPTION/GROPTION/IRI_SCHEME/alhoption 全在文件头开关段——树冠「to be fixed」处是参与开发的入口。
4. **面层**：SFCDIF_URB 与 code_12b SFCDIF_off 双向对照——PSLHS 4.7.1 修正是两版唯一函数级差异。
5. **AH 人为热**：ahoption/ahdiuprf 24 小时 Profile——论文 2 的 AHE 敏感性实验入口（与 code_22 § 六的 distributed_ahe 分值场是两套口径）。

七、上下游链接

- 驱动与混合：`code_13_module_sf_noahdrv.md`（§4.3 前处理、§4.6 FRC 加权）；城市三方案：`code_14a_module_sf_bep_bem.md`、`code_15_module_sf_bep.md`、`code_16_module_sf_bem.md`
- 土壤水副本对照：`code_12a_module_sf_noahlsm.md`、`code_12b_module_sf_noahlsm.md`；面层对照：`code_12b_module_sf_noahlsm.md` § 十四
- 教材正文：ch17_urban_canopy.md（SLUCM 逐段精读）、ch19_lanzhou_heatwave.md、ch20_modifying_wrf.md

code_20 — 辐射驱动器（module_radiation_driver.F：节拍门控、云量三案与气溶胶管线）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_radiation_driver.F	5710	405c50cbd9c1841bda3436b86c9808ef

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。

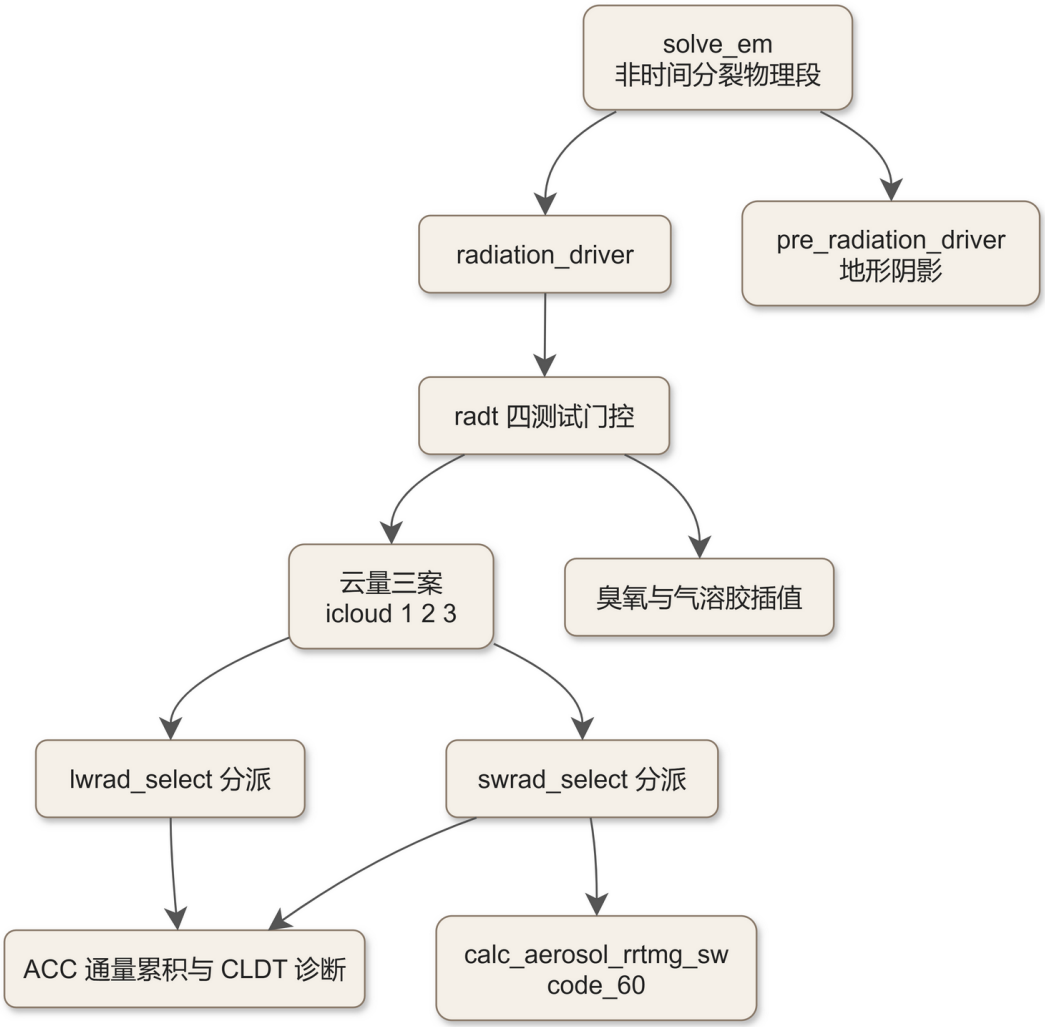
- 收录分级（B）：radiation_driver 主入口节选 15 段 + 伴生小单元 10 件全文/节选；主签名（L11–L986，含 WRF-Solar/FARMs/SPP 与 CuP 全部 OPTIONAL 参数）与声明区引用不录。教材 ch12 已精读节拍与 radconst/calc_coszen——本篇补全分发、云量、气溶胶与收尾上下文。

二、定位

一句话：**所有辐射方案的公共调度器**——radt 四测试门控、太阳几何（radconst/calc_coszen）、云量三案（icloud=1/2/3）、臭氧与气溶胶三条通道、按 lw_physics/sw_physics 双 SELECT CASE 分派、通量累积与云顶诊断收尾；另有 pre_radiation_driver 的地形阴影与 swint 短波插值两件独立小器。

谁在调它：dyn_em/solve_em.F 非时间分裂物理段之首（调用序辐射 → 陆面 →PBL→ 积云 → 浅对流 →FDDA，见 ch21_solve_em_dynamics.md）；pre_radiation_driver 由 first_rk_step_part1 调（阴影场提前算）。教材锚点：ch12_radiation_family.md（节拍/太阳几何已精读）；下游方案本体 `code_05_module_ra_rrtmg_lw.md`、`code_06_module_ra_rrtmg_sw.md`、气溶胶 `code_60_module_ra_aerosol.md`。

22 单元地图（radiation_driver 3272 行主入口 + 21 个伴生单元）：



三、节拍门控与太阳几何

```
!file: phys/module_radiation_driver.F    L1043-L1156
L01043     ! This allows radiation schemes to correctly
L01044     ! interface with the convection scheme when convection is only
L01045     ! enabled in some domains:
L01046     if(present(explicit_convection)) then
L01047        expl_conv=explicit_convection
L01048     else
L01049        expl_conv=.true. ! backward compatibility for ARW
L01050     endif
L01051
L01052     IF ( ICLOUD == 3 ) THEN
L01053        IF (PRESENT(dxkm)) then
L01054           gridkm = 1.414*SQRT(dxkm*dxkm + dy*0.001*dy*0.001)
L01055        ELSE IF (PRESENT(dx)) then
L01056           gridkm = SQRT(dx*0.001*dx*0.001 + dy*0.001*dy*0.001)
L01057        endif
L01058
L01059        if (itimestep .LE. 100) then
L01060          WRITE ( wrf_err_message , * ) 'Grid spacing in km ', dx, dy, gridkm
L01061          CALL wrf_debug (100, wrf_err_message)
L01062        endif
L01063     END IF
L01064
L01065     CALL wrf_debug (1, 'Top of Radiation Driver')
L01066  !  WRITE ( wrf_err_message , * ) 'itimestep = ',itimestep,', dt = ',dt,', lw and sw options = ',lw_physics,sw_physics
L01067  !  CALL wrf_debug (1, wrf_err_message )
L01068     if (lw_physics .eq. 0 .and. sw_physics .eq. 0)         return
L01069
L01070  ! amontornes-bcodina (2014-05-02) :: improving the namelist settings consistency for the FLG scheme
L01071  !  if (lw_physics .ne. FLGLWSCHEME .and. sw_physics .eq. FLGSWSCHEME)  then
L01072  !      call wrf_error_fatal('SW and LW schemes are in conflict. SW is FLG and LW is a different scheme!')
L01073  !  end if
L01074  !  if (lw_physics .eq. FLGLWSCHEME .and. sw_physics .ne. FLGSWSCHEME) then
L01075  !      call wrf_error_fatal('SW and LW schemes are in conflict. LW is FLG and SW is a different scheme!')
L01076  !  end if
L01077
L01078  ! ra_call_offset = -1 gives old method where radiation may be called just before output
L01079  ! ra_call_offset =  0 gives new method where radiation may be called just after output
L01080  !                    and is also consistent with removal of offset in new XTIME
L01081  ! also need to account for stepra=1 which always has zero modulo output
```

```
L01082
L01083     doing_adapt_dt = .FALSE.
L01084     IF ( PRESENT(adapt_step_flag) ) THEN
L01085        IF ( adapt_step_flag ) THEN
L01086           doing_adapt_dt = .TRUE.
L01087           IF ( radtacttime .eq. 0. ) THEN
L01088              radtacttime = CURR_SECS + radt*60.
L01089           END IF
L01090        END IF
L01091     END IF
L01092
L01093  !  Do we run through this scheme or not?
L01094
L01095  !    Test 1:  If this is the initial model time, then yes.
L01096  !                ITIMESTEP=1
L01097  !    Test 2:  If the user asked for the radiation to be run every time step, then yes.
L01098  !                RADT=0 or STEPRA=1
L01099  !    Test 3:  If not adaptive dt, and this is on the requested radiation frequency, then yes.
L01100  !                MOD(ITIMESTEP,STEPRA)=0 (or 1, depending on the offset setting)
L01101  !    Test 4:  If using adaptive dt and the current time is past the last requested activate radiation time, then yes.
L01102  !                CURR_SECS >= RADTACTTIME
L01103
L01104  !  If we do run through the scheme, we set the flag run_param to TRUE and we set the decided flag
L01105  !  to TRUE.  The decided flag says that one of these tests was able to say "yes", run the scheme.
L01106  !  We only proceed to other tests if the previous tests all have left decided as FALSE.
L01107
L01108  !  If we set run_param to TRUE and this is adaptive time stepping, we set the time to the next
L01109  !  radiation run.
L01110
L01111     run_param = .FALSE.
L01112     decided = .FALSE.
L01113     IF ( ( .NOT. decided ) .AND. &
L01114          ( itimestep .EQ. 1 ) ) THEN
L01115        run_param   = .TRUE.
L01116        decided     = .TRUE.
L01117     END IF
L01118
L01119     IF ( ( .NOT. decided ) .AND. &
L01120          ( ( radt .EQ. 0. ) .OR. ( stepra .EQ. 1 ) ) ) THEN
L01121        run_param   = .TRUE.
L01122        decided     = .TRUE.
L01123     END IF
L01124
L01125     IF ( ( .NOT. decided ) .AND. &
L01126          ( .NOT. doing_adapt_dt ) .AND. &
L01127          ( MOD(itimestep,stepra) .EQ. 1+ra_call_offset ) ) THEN
L01128        run_param   = .TRUE.
L01129        decided     = .TRUE.
L01130     END IF
L01131
L01132     IF ( ( .NOT. decided ) .AND. &
L01133          ( doing_adapt_dt ) .AND. &
L01134          ( curr_secs .GE. radtacttime ) ) THEN
L01135        run_param   = .TRUE.
L01136        decided     = .TRUE.
L01137        radtacttime = curr_secs + radt*60
L01138     END IF
L01139
L01140     IF ( mp_physics .EQ. nuwrf4icescheme ) THEN
L01141        DO ij = 1 , num_tiles
L01142           its = i_start(ij)
L01143           ite = i_end(ij)
L01144           jts = j_start(ij)
L01145           jte = j_end(ij)
L01146           DO j=jts,jte
L01147           DO k=kts,kte
L01148           DO i=its,ite
L01149              re_cloud(i,k,j) = re_cloud_gsfc(i,k,j) * 1.E-6
L01150              re_ice(i,k,j) = re_ice_gsfc(i,k,j) * 1.E-6
L01151              re_snow(i,k,j) = re_snow_gsfc(i,k,j) * 1.E-6
L01152           ENDDO
L01153           ENDDO
L01154           ENDDO
L01155        ENDDO
L01156     END IF
```

讲解：①lw_physics=0 且 sw_physics=0 直接 return；②**radt 四测试**与积云 driver（code_23 § 五）逐条同构——itimestep=1 必跑、radt=0 或 stepra=1 每步、MOD(itimestep,stepra)=1+ra_call_offset 按周期、自适应 dt 用 radtacttime 物理时刻推进；**ra_call_offset（±1）是把辐射调用对齐到输出时刻前后**的老/新两种约定；③icloud=3 时 gridkm=√(dx²+dy²)（1.414 因子用于旋转网格）——Thompson 2014 尺度感知云量要吃格距；④NUWRF 专属：微物理为 nuwrf4ice 时把 GSFC 有效半径从 μm 换成 m。

```
!file: phys/module_radiation_driver.F    L1158-L1256
L01158     if(swint_opt.eq.1 .or. swint_opt == 2) then
L01159        DO ij = 1 , num_tiles
L01160           its = i_start(ij)
L01161           ite = i_end(ij)
L01162           jts = j_start(ij)
L01163           jte = j_end(ij)
L01164           CALL radconst(XTIME,DECLIN,SOLCON,JULIAN,               &
L01165                         DEGRAD,DPD                                )
L01166           call calc_coszen(ims,ime,jms,jme,its,ite,jts,jte, &
L01167                            julian,xtime,gmt,declin,degrad,  &
L01168                            xlong,xlat,coszen_loc,hrang_loc)
L01169        end do
L01170     end if
L01171
L01172     solar_opt_local = 0
L01173     IF ( PRESENT(solar_opt) ) THEN
L01174        solar_opt_local = solar_opt
L01175     END IF
L01176     Solar_step: IF (run_param .or. solar_opt_local == 1 .or. swint_opt == 2) THEN
```

```
L01177
L01178       !---------------
L01179       ! Calculate constant for short wave radiation
L01180       ! moved up and out of OMP loop because it only needs to be computed once
L01181       ! and because it is not entirely thread-safe (XT24, TOLOCTM and XXLAT need
L01182       ! their thread-privacy)  JM 20100217
L01183         DO ij = 1 , num_tiles
L01184           its = i_start(ij)
L01185           ite = i_end(ij)
L01186           jts = j_start(ij)
L01187           jte = j_end(ij)
L01188           CALL radconst(XTIME,DECLIN,SOLCON,JULIAN,               &
L01189                         DEGRAD,DPD                                )
L01190
L01191  ! amontornes-bcodina 2015/09 solar eclipses
L01192  !    Solar eclipse prediction based on the Bessel's method
L01193  !    outputs: obscur, mask, elat_track, elon_track
L01194           CALL solar_eclipse(ims,ime,jms,jme,its,ite,jts,jte,        &
L01195                              julian,gmt,YR,xtime,radt,               &
L01196                              degrad,xlong,xlat,obscur_loc,mask_loc,  &
L01197                              elat_loc,elon_loc,sw_eclipse  )
L01198
L01199           IF(PRESENT(declinx).AND.PRESENT(solconx))THEN
L01200  ! saved to state arrays used in surface driver
L01201             declinx=declin
L01202             solconx=solcon
L01203           ENDIF
L01204  ! added coszen subroutine : jararias, 2013/08/10
L01205  !   outputs: coszen, hrang
L01206           call calc_coszen(ims,ime,jms,jme,its,ite,jts,jte,  &
L01207                            julian,xtime+radt*0.5,gmt, &
L01208                            declin,degrad,xlong,xlat,coszen,hrang)
L01209
L01210         ! backup the incoming hydrometeors
L01211
L01212           IF ( F_QC ) THEN
L01213             DO j=jts,jte
L01214             DO k=kts,kte
L01215             DO i=its,ite
L01216               qc_save(i,k,j) = qc(i,k,j)
L01217             ENDDO
L01218             ENDDO
L01219             ENDDO
L01220           ENDIF
L01221           IF ( F_QI ) THEN
L01222             DO j=jts,jte
L01223             DO k=kts,kte
L01224             DO i=its,ite
L01225               qi_save(i,k,j) = qi(i,k,j)
L01226             ENDDO
L01227             ENDDO
L01228             ENDDO
L01229           ENDIF
L01230
L01231           IF(aercu_opt.gt.0.0) THEN
L01232             IF ( F_QI ) THEN
L01233               DO j=jts,jte
L01234               DO k=kts,kte
L01235               DO i=its,ite
L01236                 qs_save(i,k,j) = qs(i,k,j)
L01237               ENDDO
L01238               ENDDO
L01239               ENDDO
L01240             ENDIF
L01241           END IF
L01242
L01243  ! Fill temporary water variable depending on micro package (tgs 25 Apr 2006)
L01244           if( F_QC ) then
L01245             DO j=jts,jte
L01246             DO k=kts,kte
L01247             DO i=its,ite
L01248               qc_temp(I,K,J)=qc(I,K,J)
L01249             ENDDO
L01250             ENDDO
L01251             ENDDO
L01252           else
L01253             DO j=jts,jte
L01254             DO k=kts,kte
L01255             DO i=its,ite
L01256               qc_temp(I,K,J)=0.
```

讲解：①swint_opt=1/2 时先算一套 coszen_loc（**不带 radt 半步**）供短波插值用；②Solar_step 块：radconst 算太阳赤纬 DECLIN 与太阳常数 SOLCON（提出 OMP 循环外——XT24/TOLOCTM/XXLAT 的线程私有性注释，JM 20100217）；solar_eclipse 用 Bessel 法算日食遮挡（obscur/mask/轨迹）；**calc_coszen 的调用点用 xtime+radt·0.5——辐射区间中点的太阳几何**（教材 ch12 的核心发现之一）；③DECLINX/SOLCONX 存回状态数组供 surface_driver 复用；④ **水物质备份** qc_save/qi_save——云量方案（尤其 icloud=3 的 modify_qvapor）会改水物质，跑完恢复（§ 八）。

```
!file: phys/module_radiation_driver.F    L3469-L3511    (subroutine radconst)
L03469      SUBROUTINE radconst(XTIME,DECLIN,SOLCON,JULIAN,                &
L03470                          DEGRAD,DPD                                 )
L03471  !-----------------------------------------------------------------------
L03472      USE module_wrf_error
L03473      IMPLICIT NONE
L03474  !-----------------------------------------------------------------------
L03475
L03476  ! !ARGUMENTS:
L03477      REAL, INTENT(IN   )     ::        DEGRAD,DPD,XTIME,JULIAN
L03478      REAL, INTENT(OUT  )     ::        DECLIN,SOLCON
L03479      REAL                    ::        OBECL,SINOB,SXLONG,ARG,  &
L03480                                        DECDEG,DJUL,RJUL,ECCFAC
L03481  !
L03482  ! !DESCRIPTION:
```

```
L03483  ! Compute terms used in radiation physics
L03484  !EOP
L03485
L03486  ! for short wave radiation
L03487
L03488     DECLIN=0.
L03489     SOLCON=0.
L03490
L03491  !-----OBECL : OBLIQUITY = 23.5 DEGREE.
L03492
L03493     OBECL=23.5*DEGRAD
L03494     SINOB=SIN(OBECL)
L03495
L03496  !-----CALCULATE LONGITUDE OF THE SUN FROM VERNAL EQUINOX:
L03497
L03498     IF(JULIAN.GE.80.)SXLONG=DPD*(JULIAN-80.)
L03499     IF(JULIAN.LT.80.)SXLONG=DPD*(JULIAN+285.)
L03500     SXLONG=SXLONG*DEGRAD
L03501     ARG=SINOB*SIN(SXLONG)
L03502     DECLIN=ASIN(ARG)
L03503     DECDEG=DECLIN/DEGRAD
L03504  !----SOLAR CONSTANT ECCENTRICITY FACTOR (PALTRIDGE AND PLATT 1976)
L03505     DJUL=JULIAN*360./365.
L03506     RJUL=DJUL*DEGRAD
L03507     ECCFAC=1.000110+0.034221*COS(RJUL)+0.001280*SIN(RJUL)+0.000719*  &
L03508            COS(2*RJUL)+0.000077*SIN(2*RJUL)
L03509     SOLCON=1370.*ECCFAC
L03510
L03511     END SUBROUTINE radconst
```

讲解：太阳赤纬与太阳常数：黄赤交角 OBECL=23.5°；太阳黄经从春分起算（JULIAN≥80 即 3 月 21 日后），SXLONG=DPD • (JULIAN−80)；DECLIN=asin(sinob • sinSXLONG)；**ECCFAC 是 Paltridge–Platt (1976) 的四项日地距离谐波**（1.000110+0.034221cos+0.001280sin+0.000719cos2+0.000077sin2），SOLCON=1370 • ECCFAC——教材 ch12 已逐符号核过。

```
!file: phys/module_radiation_driver.F    L3514-L3541    (subroutine calc_coszen)
L03514     SUBROUTINE calc_coszen(ims,ime,jms,jme,its,ite,jts,jte,  &
L03515                            julian,xtime,gmt, &
L03516                            declin,degrad,xlon,xlat,coszen,hrang)
L03517         ! Added Equation of Time correction : jararias, 2013/08/10
L03518         implicit none
L03519         integer, intent(in) :: ims,ime,jms,jme,its,ite,jts,jte
L03520         real, intent(in)    :: julian,declin,xtime,gmt,degrad
L03521         real, dimension(ims:ime,jms:jme), intent(in)    :: xlat,xlon
L03522         real, dimension(ims:ime,jms:jme), intent(inout) :: coszen,hrang
L03523
L03524         integer :: i,j
L03525         real    :: da,eot,xt24,tloctm,xxlat
L03526
L03527         da=6.2831853071795862*(julian-0.5)/365.
L03528         eot=(0.000075+0.001868*cos(da)-0.032077*sin(da) &
L03529              -0.014615*cos(2*da)-0.040849*sin(2*da))*(229.18)
L03530         xt24=mod(xtime,1440.)+eot
L03531         do j=jts,jte
L03532            do i=its,ite
L03533               tloctm=gmt+xt24/60.+xlon(i,j)/15.
L03534               hrang(i,j)=15.*(tloctm-12.)*degrad
L03535               xxlat=xlat(i,j)*degrad
L03536               coszen(i,j)=sin(xxlat)*sin(declin) &
L03537                           +cos(xxlat)*cos(declin) *cos(hrang(i,j))
L03538               coszen(i, j) = min (max (coszen(i, j), -1.0), 1.0)
L03539            enddo
L03540         enddo
L03541     END SUBROUTINE calc_coszen
```

讲解：太阳天顶角：**均时差修正** eot（Spencer 四项式 ×229.18 分钟换算）；xt24=mod(xtime,1440)+eot；地方平时 tloctm=GMT+xt24/60+经度/15；时角 hrang=15° • (tloctm−12)；coszen=sinφsinδ+cosφcosδcosH，钳位 [−1,1]——与书 (5.4.3a) 同式。

四、云量三案（namelist icloud）

```
!file: phys/module_radiation_driver.F    L1308-L1345
L01308  ! Choose how to compute cloud fraction (since 3.6)
L01309  ! Initialize to zero
L01310        DO j=jts,jte
L01311        DO k=kts,kte
L01312        DO i=its,ite
L01313           CLDFRA(i,k,j) = 0.
L01314        END DO
L01315        END DO
L01316        END DO
L01317  !---------------
L01318  ! Calculate constant for short wave radiation
L01319
L01320        IF ( ICLOUD == 1 ) THEN
L01321
L01322        IF ( F_QC .OR. F_QI ) THEN
L01323  ! Call to cloud fraction routine based on Randall 1994 (Hong Pan 1998)
L01324
L01325           CALL wrf_debug (1, 'CALL cldfra1')
L01326           CALL cal_cldfra1(CLDFRA,qv,qc,qi,qs,              &
L01327                     F_QV,F_QC,F_QI,F_QS,t,p,                &
L01328                     F_ICE_PHY,F_RAIN_PHY,                   &
L01329                     mp_physics,cldfra1_flag,                &
L01330                     ids,ide, jds,jde, kds,kde,              &
L01331                     ims,ime, jms,jme, kms,kme,              &
```

```
L01332                    its,ite, jts,jte, kts,kte              )
L01333          IF ( PRESENT ( CLDFRA_DP ) ) THEN
L01334  ! this is for Kain-Fritsch schemes
L01335            IF ( icloud_cu .EQ. 2 .OR. aercu_opt .GT. 0 ) THEN
L01336               CALL wrf_debug (1, 'use kf cldfra')
L01337               DO j = jts,jte
L01338               DO k = kts,kte
L01339               DO i = its,ite
L01340                  cldfra_cu(i,k,j)=cldfra_dp(i,k,j)+cldfra_sh(i,k,j) ! Cu cloud fraction
L01341                  CLDFRA(i,k,j)=(1.-cldfra_cu(i,k,j))*CLDFRA(i,k,j)  ! Update resolved cloud fraction for Cu
↪  punch-through
L01342                  CLDFRA(i,k,j)=CLDFRA(i,k,j)+cldfra_cu(i,k,j)       ! New total cloud fraction
L01343                  CLDFRA(i,k,j)=AMIN1(1.0,CLDFRA(i,k,j))
L01344                  qc(i,k,j) = qc(i,k,j)+qc_cu(i,k,j)*cldfra_cu(i,k,j)
L01345                  qi(i,k,j) = qi(i,k,j)+qi_cu(i,k,j)*cldfra_cu(i,k,j)
```

讲解：icloud=1（默认，Randall 1994/Hong-Pan 谱系）：调 cal_cldfra1；**KF 积云云量的 punch-through 混合**——icloud_cu=2 或 aercu_opt>0 时把 cldfra_dp+cldfra_sh（KF 深浅对流云量）按 CLDFRA=(1−cu)·格点云量 +cu 的方式并入，并把 KF 云水/云冰按云量比加进 qc/qi（MSKF 场景还要按权重混合有效半径 EFCS/EFIG 系）。

```
!file: phys/module_radiation_driver.F    L1449-L1505
L01449       ELSE IF ( ICLOUD == 2 ) THEN
L01450       IF ( F_QC .OR. F_QI ) THEN
L01451         CALL wrf_debug (1, 'CALL cldfra2')
L01452         CALL cal_cldfra2(CLDFRA,qc,qi,F_QC,F_QI,              &
L01453                          ids,ide, jds,jde, kds,kde,           &
L01454                          ims,ime, jms,jme, kms,kme,           &
L01455                          its,ite, jts,jte, kts,kte            )
L01456       ENDIF
L01457
L01458  !+---+-----------------------------------------------------------------+
L01459  !..New cloud fraction scheme added by G. Thompson (2014Oct31)
L01460  !+---+-----------------------------------------------------------------+
L01461
L01462       ELSEIF (ICLOUD == 3) THEN
L01463          IF ( F_QC .AND. F_QI ) THEN
L01464
L01465             CALL wrf_debug (150, 'DEBUG: using gthompsn cloud fraction scheme')
L01466
L01467             DO j = jts,jte
L01468             DO i = its,ite
L01469
L01470               DO k = kts,kte
L01471                  p_1d(k) = p(i,k,j)
L01472                  if (pert_cld3 .and. multi_perturb == 1) then
L01473                    t_1d(k) = (1.0 + perts_th(i, k, j) * pert_cld3_t) * t(i, k, j)
L01474                    qv_1d(k) = max (0.0, (1.0 + perts_qvapor(i, k, j) * pert_cld3_qv) * qv(i, k, j))
L01475                  else
L01476                    t_1d(k) = t(i,k,j)
L01477                    qv_1d(k) = qv(i,k,j)
L01478                  end if
L01479                  qc_1d(k) = qc(i,k,j)
L01480                  qi_1d(k) = qi(i,k,j)
L01481                  qs_1d(k) = qs(i,k,j)
L01482                  Dz_1d(k) = dz8w(i,k,j)
L01483                  cf_1d(k) = cldfra(i,k,j)
L01484               ENDDO
L01485
L01486               WRITE (wrf_err_message,*) 'DEBUG: calling cal_cldfra3 at (i,j): ', i,j, kms,kme,kts,kte
L01487               CALL wrf_debug (150, wrf_err_message)
L01488
L01489               CALL cal_cldfra3(cf_1d, qv_1d, qc_1d, qi_1d, qs_1d, Dz_1d, &
L01490       &                        p_1d, t_1d, XLAND(i,j), gridkm,          &
L01491       &                        .false., 1.5, kts, kte, .false.)
L01492
L01493               DO k = kts,kte
L01494                  qc(i,k,j) = qc_1d(k)
L01495                  qi(i,k,j) = qi_1d(k)
L01496                  qs(i,k,j) = qs_1d(k)
L01497                  cldfra(i,k,j) = cf_1d(k)
L01498               ENDDO
L01499
L01500             ENDDO
L01501             ENDDO
L01502
L01503          ELSE
L01504             CALL wrf_error_fatal('Can not use icloud = 3 option, missing QC or QI field.')
L01505          ENDIF
```

讲解：icloud=2：cal_cldfra2（qc/qi 阈值二值，见 § 四附）；icloud=3：**Thompson 2014 尺度感知云量**——逐列进 cal_cldfra3，SPP（multi_perturb=1）时对 t/qv 先乘扰动；**requirement：F_QC 且 F_QI 同时在场，否则 FATAL**。

```
!file: phys/module_radiation_driver.F    L3761-L3986    (subroutine cal_cldfra1)
L03761      SUBROUTINE cal_cldfra1(CLDFRA, QV, QC, QI, QS,                    &
L03762                             F_QV, F_QC, F_QI, F_QS, t_phy, p_phy,      &
L03763                             F_ICE_PHY,F_RAIN_PHY,                      &
L03764                             mp_physics, cldfra1_flag,                  &
L03765            ids,ide, jds,jde, kds,kde,                                  &
L03766            ims,ime, jms,jme, kms,kme,                                  &
L03767            its,ite, jts,jte, kts,kte                                   )
L03768       USE module_state_description, ONLY : KFCUPSCHEME, KFETASCHEME       !wig, CuP 4-Fb-2008 !BSINGH - For WRFCuP
↪  scheme
L03769
L03770       USE module_state_description, ONLY : FER_MP_HIRES, FER_MP_HIRES_ADVECT
L03771  !-----------------------------------------------------------------------
L03772       IMPLICIT NONE
L03773  !-----------------------------------------------------------------------
```

```
   INTEGER,  INTENT(IN   )   ::           ids,ide, jds,jde, kds,kde, &
                                          ims,ime, jms,jme, kms,kme, &
                                          its,ite, jts,jte, kts,kte
!
   INTEGER, DIMENSION( ims:ime, kms:kme, jms:jme ), INTENT(OUT  ) :: cldfra1_flag
   REAL, DIMENSION( ims:ime, kms:kme, jms:jme ), INTENT(OUT  ) ::    &
                                                         CLDFRA

   REAL, DIMENSION( ims:ime, kms:kme, jms:jme ), INTENT(IN   ) ::    &
                                                            QV, &
                                                            QI, &
                                                            QC, &
                                                            QS, &
                                                         t_phy, &
                                                         p_phy
!                                                          p_phy, &
!                                                       F_ICE_PHY, &
!                                                      F_RAIN_PHY
   REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                     &
         OPTIONAL,                                                   &
         INTENT(IN   ) ::                                            &
                                                       F_ICE_PHY, &
                                                      F_RAIN_PHY
   LOGICAL,OPTIONAL,INTENT(IN) :: F_QC,F_QI,F_QV,F_QS
   INTEGER :: mp_physics

!  REAL thresh
   INTEGER:: i,j,k
   REAL    :: RHUM, tc, esw, esi, weight, qvsw, qvsi, qvs_weight, QIMID, QWMID, QCLD, DENOM, ARG, SUBSAT

   REAL    ,PARAMETER :: ALPHA0=100., GAMMA=0.49, QCLDMIN=1.E-12,    &
                                      PEXP=0.25, RHGRID=1.0
   REAL    , PARAMETER ::  SVP1=0.61078
   REAL    , PARAMETER ::  SVP2=17.2693882
   REAL    , PARAMETER ::  SVPI2=21.8745584
   REAL    , PARAMETER ::  SVP3=35.86
   REAL    , PARAMETER ::  SVPI3=7.66
   REAL    , PARAMETER ::  SVPT0=273.15
   REAL    , PARAMETER ::  r_d = 287.
   REAL    , PARAMETER ::  r_v = 461.6
   REAL    , PARAMETER ::  ep_2=r_d/r_v
! !DESCRIPTION:
! Compute cloud fraction from input ice and cloud water fields
! if provided.
!
! Whether QI or QC is active or not is determined from the indices of
! the fields into the 4D scalar arrays in WRF. These indices are
! P_QI and P_QC, respectively, and they are passed in to the routine
! to enable testing to see if QI and QC represent active fields in
! the moisture 4D scalar array carried by WRF.
!
! If a field is active its index will have a value greater than or
! equal to PARAM_FIRST_SCALAR, which is also an input argument to
! this routine.
!EOP

!-----------------------------------------------------------------------
!---  COMPUTE GRID-SCALE CLOUD COVER FOR RADIATION
!     (modified by Ferrier, Feb '02)
!
!---  Cloud fraction parameterization follows Randall, 1994
!     (see Hong et al., 1998)
!-----------------------------------------------------------------------
! Note: ep_2=287./461.6 Rd/Rv
! Note: R_D=287.

! Alternative calculation for critical RH for grid saturation
!     RHGRID=0.90+.08*((100.-DX)/95.)**.5

! Calculate saturation mixing ratio weighted according to the fractions of
! water and ice.
! Following:
! Murray, F.W. 1966. ``On the computation of Saturation Vapor Pressure''  J. Appl. Meteor.  6 p.204
!    es (in mb) = 6.1078 . exp[ a . (T-273.16)/ (T-b) ]
!
!       over ice        over water
! a =   21.8745584      17.2693882
! b =   7.66            35.86

!-----------------------------------------------------------------------

     DO j = jts,jte
     DO k = kts,kte
     DO i = its,ite
       tc         = t_phy(i,k,j) - SVPT0
       esw     = 1000.0 * SVP1 * EXP( SVP2  * tc / ( t_phy(i,k,j) - SVP3  ) )
       esi     = 1000.0 * SVP1 * EXP( SVPI2 * tc / ( t_phy(i,k,j) - SVPI3 ) )
       QVSW = EP_2 * esw / ( p_phy(i,k,j) - esw )
       QVSI = EP_2 * esi / ( p_phy(i,k,j) - esi )

       ifouter: IF ( PRESENT(F_QI) .and. PRESENT(F_QC) .and. PRESENT(F_QS) ) THEN

! mji - For MP options 2, 4, 6, 7, 8, etc. (qc = liquid, qi = ice, qs = snow)
         IF ( F_QI .and. F_QC .and. F_QS) THEN
            QCLD = QI(i,k,j)+QC(i,k,j)+QS(i,k,j)
            IF (QCLD .LT. QCLDMIN) THEN
               weight = 0.
            ELSE
               weight = (QI(i,k,j)+QS(i,k,j)) / QCLD
            ENDIF
         ENDIF

! for P3, mp option 50 or 51
         IF ( F_QI .and. F_QC .and. .not. F_QS) THEN
            QCLD = QI(i,k,j)+QC(i,k,j)
            IF (QCLD .LT. QCLDMIN) THEN
               weight = 0.
```

```
L03884              ELSE
L03885                 weight = (QI(i,k,j)) / QCLD
L03886              ENDIF
L03887           ENDIF
L03888
L03889  ! mji - For MP options 1 and 3, (qc only)
L03890  !  For MP=1, qc = liquid, for MP=3, qc = liquid or ice depending on temperature
L03891           IF ( F_QC .and. .not. F_QI .and. .not. F_QS ) THEN
L03892              QCLD = QC(i,k,j)
L03893              IF (QCLD .LT. QCLDMIN) THEN
L03894                 weight = 0.
L03895              ELSE
L03896                 if (t_phy(i,k,j) .gt. 273.15) weight = 0.
L03897                 if (t_phy(i,k,j) .le. 273.15) weight = 1.
L03898              ENDIF
L03899           ENDIF
L03900
L03901  ! mji - For MP option 5; (qc = liquid, qs = ice)
L03902           IF ( F_QC .and. .not. F_QI .and. F_QS .and. PRESENT(F_ICE_PHY) ) THEN
L03903
L03904  ! Mixing ratios of cloud water & total ice (cloud ice + snow).
L03905  ! Mixing ratios of rain are not considered in this scheme.
L03906  ! F_ICE is fraction of ice
L03907  ! F_RAIN is fraction of rain
L03908
L03909             QIMID = QS(i,k,j)
L03910             QWMID = QC(i,k,j)
L03911  ! old method
L03912  !           QIMID = QC(i,k,j)*F_ICE_PHY(i,k,j)
L03913  !           QWMID = (QC(i,k,j)-QIMID)*(1.-F_RAIN_PHY(i,k,j))
L03914  !
L03915  !--- Total "cloud" mixing ratio, QCLD.  Rain is not part of cloud,
L03916  !    only cloud water + cloud ice + snow
L03917  !
L03918             QCLD=QWMID+QIMID
L03919             IF (QCLD .LT. QCLDMIN) THEN
L03920                weight = 0.
L03921             ELSE
L03922                weight = F_ICE_PHY(i,k,j)
L03923             ENDIF
L03924           ENDIF
L03925  !         IF ( F_QC .and. F_QI .and. .not. F_QS ) THEN
L03926           IF ( mp_physics .eq. FER_MP_HIRES .or. &
L03927                mp_physics==fer_mp_hires_advect) THEN
L03928             QIMID = QI(i,k,j)      !- total ice (cloud ice + snow)
L03929             QWMID = QC(i,k,j)      !- cloud water
L03930             QCLD=QWMID+QIMID       !- cloud water + total ice
L03931             IF (QCLD .LT. QCLDMIN) THEN
L03932                weight = 0.
L03933             ELSE
L03934                weight = QIMID/QCLD
L03935                if (tc<-40.) weight=1.
L03936             ENDIF
L03937           ENDIF
L03938
L03939        ELSE
L03940           CLDFRA(i,k,j)=0.
L03941
L03942        ENDIF ifouter !  IF ( F_QI .and. F_QC .and. F_QS)
L03943
L03944
L03945        QVS_WEIGHT = (1-weight)*QVSW + weight*QVSI
L03946        RHUM=QV(i,k,j)/QVS_WEIGHT   !--- Relative humidity
L03947  !
L03948  !--- Determine cloud fraction (modified from original algorithm)
L03949  !
L03950        cldfra1_flag(i,k,j) = 0
L03951        IF (QCLD .LT. QCLDMIN) THEN
L03952  !
L03953  !--- Assume zero cloud fraction if there is no cloud mixing ratio
L03954  !
L03955          CLDFRA(i,k,j)=0.
L03956          cldfra1_flag(i,k,j) = 1
L03957        ELSEIF(RHUM.GE.RHGRID)THEN
L03958  !
L03959  !--- Assume cloud fraction of unity if near saturation and the cloud
L03960  !    mixing ratio is at or above the minimum threshold
L03961  !
L03962          CLDFRA(i,k,j)=1.
L03963          cldfra1_flag(i,k,j) = 2
L03964        ELSE
L03965           cldfra1_flag(i,k,j) = 3
L03966  !
L03967  !--- Adaptation of original algorithm (Randall, 1994; Zhao, 1995)
L03968  !    modified based on assumed grid-scale saturation at RH=RHgrid.
L03969  !
L03970          SUBSAT=MAX(1.E-10,RHGRID*QVS_WEIGHT-QV(i,k,j))
L03971          DENOM=(SUBSAT)**GAMMA
L03972          ARG=MAX(-6.9, -ALPHA0*QCLD/DENOM)    ! <-- EXP(-6.9)=.001
L03973  ! prevent negative values  (new)
L03974          RHUM=MAX(1.E-10, RHUM)
L03975          CLDFRA(i,k,j)=(RHUM/RHGRID)**PEXP*(1.-EXP(ARG))
L03976  !!              ARG=-1000*QCLD/(RHUM-RHGRID)
L03977  !!              ARG=MAX(ARG, ARGMIN)
L03978  !!              CLDFRA(i,k,j)=(RHUM/RHGRID)*(1.-EXP(ARG))
L03979          IF (CLDFRA(i,k,j) .LT. .01) CLDFRA(i,k,j)=0.
L03980
L03981        ENDIF          !--- End IF (QCLD .LT. QCLDMIN) ...
L03982       ENDDO          !--- End DO i
L03983       ENDDO          !--- End DO k
L03984       ENDDO          !--- End DO j
L03985
L03986     END SUBROUTINE cal_cldfra1
```

讲解：Randall 型诊断云量全解：① 饱和水汽比 QVSW/QVSI 按 Murray 1966 系数（水 17.2693882/35.86、冰 21.8745584/7.66）分别算，**按冰水份额 weight 混合**——weight 的分派按微物理方案四路（qc+qi+qs 三组分 =P3 前主流；仅 qc=MP1/3；qc+qs=FICE 份额 =MP5；FER_MP_

HIRES 用 qi/总并 −40°C 全冰）；②RHUM=QV/QVS_WEIGHT；③ 三段式：无云水 →0（cldfra1_flag=1 诊断标记）、RH≥RHGRID(=1.0)→ 全盖、否则 **Xu–Randall 1996 式 CLDFRA=(RH/RHGRID)$^{0.25}$ · (1−exp(−100 · QCLD/SUBSAT$^{0.49}$))**（ARG 钳位 −6.9 防下溢、结果下限 0.01）；注释头写 Randall 1994、体内注明 Xu and Randall (JAS) 1996——两个引文并存的历史痕迹。

```
!file: phys/module_radiation_driver.F    L3676-L3749    (subroutine cal_cldfra2)
L03676     SUBROUTINE cal_cldfra2(CLDFRA,QC,QI,F_QC,F_QI,                    &
L03677                ids,ide, jds,jde, kds,kde,                             &
L03678                ims,ime, jms,jme, kms,kme,                             &
L03679                its,ite, jts,jte, kts,kte                              )
L03680       USE module_state_description, ONLY : KFCUPSCHEME, KFETASCHEME       !CuP, wig 5-Oct-2006  !BSINGH - For WRFCuP
↪   scheme
L03681   !-------------------------------------------------------------------
L03682      IMPLICIT NONE
L03683   !-------------------------------------------------------------------
L03684      INTEGER,  INTENT(IN   )   ::           ids,ide, jds,jde, kds,kde, &
L03685                                             ims,ime, jms,jme, kms,kme, &
L03686                                             its,ite, jts,jte, kts,kte
L03687
L03688   !
L03689      REAL, DIMENSION( ims:ime, kms:kme, jms:jme ), INTENT(OUT  ) ::    &
L03690                                                           CLDFRA
L03691
L03692      REAL, DIMENSION( ims:ime, kms:kme, jms:jme ), INTENT(IN   ) ::    &
L03693                                                               QI, &
L03694                                                               QC
L03695      LOGICAL,INTENT(IN) :: F_QC,F_QI
L03696
L03697      REAL thresh
L03698      INTEGER:: i,j,k
L03699   ! !DESCRIPTION:
L03700   ! Compute cloud fraction from input ice and cloud water fields
L03701   ! if provided.
L03702   !
L03703   ! Whether QI or QC is active or not is determined from the indices of
L03704   ! the fields into the 4D scalar arrays in WRF. These indices are
L03705   ! P_QI and P_QC, respectively, and they are passed in to the routine
L03706   ! to enable testing to see if QI and QC represent active fields in
L03707   ! the moisture 4D scalar array carried by WRF.
L03708   !
L03709   ! If a field is active its index will have a value greater than or
L03710   ! equal to PARAM_FIRST_SCALAR, which is also an input argument to
L03711   ! this routine.
L03712   !EOP
L03713   !-------------------------------------------------------------------
L03714        thresh=1.0e-6
L03715
L03716        IF ( f_qi .AND. f_qc ) THEN
L03717           DO j = jts,jte
L03718           DO k = kts,kte
L03719           DO i = its,ite
L03720              IF ( QC(i,k,j)+QI(I,k,j) .gt. thresh) THEN
L03721                 CLDFRA(i,k,j)=1.
L03722              ELSE
L03723                 CLDFRA(i,k,j)=0.
L03724              ENDIF
L03725           ENDDO
L03726           ENDDO
L03727           ENDDO
L03728        ELSE IF ( f_qc ) THEN
L03729           DO j = jts,jte
L03730           DO k = kts,kte
L03731           DO i = its,ite
L03732              IF ( QC(i,k,j) .gt. thresh) THEN
L03733                 CLDFRA(i,k,j)=1.
L03734              ELSE
L03735                 CLDFRA(i,k,j)=0.
L03736              ENDIF
L03737           ENDDO
L03738           ENDDO
L03739           ENDDO
L03740        ELSE
L03741           DO j = jts,jte
L03742           DO k = kts,kte
L03743           DO i = its,ite
L03744              CLDFRA(i,k,j)=0.
L03745           ENDDO
L03746           ENDDO
L03747           ENDDO
L03748        ENDIF
L03749     END SUBROUTINE cal_cldfra2
```

讲解：极简二值案：qc+qi>1e-6 即全盖（f_qc/f_qi 分别在场时的三路循环）——无部分云量，多为粗网格旧行为或对照实验用。

```
!file: phys/module_radiation_driver.F    L4003-L4110
L04003         SUBROUTINE cal_cldfra3(CLDFRA, qv, qc, qi, qs, dz,              &
L04004        &                 p, t, XLAND, gridkm,                          &
L04005        &                 modify_qvapor, max_relh,                      &
L04006        &                 kts,kte, debug_flag)
L04007   !
L04008         USE module_mp_thompson   , ONLY : rsif, rslf
L04009         IMPLICIT NONE
L04010   !
L04011         INTEGER, INTENT(IN):: kts, kte
L04012         LOGICAL, INTENT(IN):: modify_qvapor
L04013         REAL, DIMENSION(kts:kte), INTENT(INOUT):: qv, qc, qi, cldfra
L04014         REAL, DIMENSION(kts:kte), INTENT(IN):: p, t, dz, qs
L04015         REAL, INTENT(IN):: gridkm, XLAND, max_relh
L04016         LOGICAL, INTENT(IN):: debug_flag
L04017
L04018   !..Local vars.
L04019         REAL:: RH_00L, RH_00O, RH_00
```

```
L04020        REAL:: entrmnt=0.5
L04021        INTEGER:: k
L04022        REAL:: TC, qvsi, qvsw, RHUM, delz
L04023        REAL, DIMENSION(kts:kte):: qvs, rh, rhoa
L04024
L04025        character*512 dbg_msg
L04026
L04027  !+---+
L04028
L04029  !..Initialize cloud fraction, compute RH, and rho-air.
L04030
L04031           DO k = kts,kte
L04032              CLDFRA(K) = 0.0
L04033
L04034              qvsw = rslf(P(k), t(k))
L04035              qvsi = rsif(P(k), t(k))
L04036
L04037              tc = t(k) - 273.15
L04038              if (tc .ge. -12.0) then
L04039                 qvs(k) = qvsw
L04040              elseif (tc .lt. -35.0) then
L04041                 qvs(k) = qvsi
L04042              else
L04043                 qvs(k) = qvsw - (qvsw-qvsi)*(-12.0-tc)/(-12.0+35.)
L04044              endif
L04045
L04046              if (modify_qvapor) then
L04047                 if (qc(k).gt.1.E-8) then
L04048                    qv(k) = MAX(qv(k), qvsw)
L04049                    qvs(k) = qvsw
L04050                 endif
L04051                 if (qc(k).le.1.E-8 .and. qi(k).ge.1.E-9) then
L04052                    qv(k) = MAX(qv(k), qvsi*1.005)    !..To ensure a tiny bit ice supersaturation
L04053                    qvs(k) = qvsi
L04054                 endif
L04055              endif
L04056
L04057              rh(k) = MAX(0.01, qv(k)/qvs(k))
L04058              rhoa(k) = p(k)/(287.0*t(k))
L04059           ENDDO
L04060
L04061
L04062  !..First cut scale-aware. Higher resolution should require closer to
L04063  !.. saturated grid box for higher cloud fraction.  Simple functions
L04064  !.. chosen based on Mocko and Cotton (1995) starting point and desire
L04065  !.. to get near 100% RH as grid spacing moves toward 1.0km, but higher
L04066  !.. RH over ocean required as compared to over land.
L04067
L04068         DO k = kts,kte
L04069
L04070            delz = MAX(100., dz(k))
L04071            RH_00L = 0.77 + MIN(0.22,SQRT(1./(50.0+gridkm*gridkm*delz*0.01)))
L04072            RH_00O = 0.85 + MIN(0.14,SQRT(1./(50.0+gridkm*gridkm*delz*0.01)))
L04073            RHUM = rh(k)
L04074
L04075            if (qc(k).gt.1.E-6 .or. qi(k).ge.1.E-7                          &
L04076         &                  .or. (qs(k).gt.1.E-6 .and. t(k).lt.273.)) then
L04077               CLDFRA(K) = 1.0
L04078            elseif (((qc(k)+qi(k)).gt.1.E-10) .and.                         &
L04079         &                               ((qc(k)+qi(k)).lt.1.E-6)) then
L04080               CLDFRA(K) = MIN(0.99, 0.1*(11.0 + log10(qc(k)+qi(k))))
L04081            else
L04082
L04083               IF ((XLAND-1.5).GT.0.) THEN                                  !--- Ocean
L04084                  RH_00 = RH_00O
L04085               ELSE                                                         !--- Land
L04086                  RH_00 = RH_00L
L04087               ENDIF
L04088
L04089               tc = MAX(-80.0, t(k) - 273.15)
L04090               if (tc .lt. -12.0) RH_00 = RH_00L
L04091
L04092               if (tc .ge. 25.0) then
L04093                  CLDFRA(K) = 0.0
L04094               elseif (tc .ge. -12.0) then
L04095                  RHUM = MIN(rh(k), 1.0)
L04096                  CLDFRA(K) = MAX(0., 1.0-SQRT((1.001-RHUM)/(1.001-RH_00)))
L04097               else
L04098                  if (max_relh.gt.1.12 .or. (.NOT.(modify_qvapor)) ) then
L04099  !..For HRRR model, the following look OK.
L04100                     RHUM = MIN(rh(k), 1.45)
L04101                     RH_00 = RH_00 + (1.45-RH_00)*(-12.0-tc)/(-12.0+85.)
L04102                     CLDFRA(K) = MAX(0., 1.0-SQRT((1.46-RHUM)/(1.46-RH_00)))
L04103                  else
L04104  !..but for the GFS model, RH is way lower.
L04105                     RHUM = MIN(rh(k), 1.05)
L04106                     RH_00 = RH_00 + (1.05-RH_00)*(-12.0-tc)/(-12.0+85.)
L04107                     CLDFRA(K) = MAX(0., 1.0-SQRT((1.06-RHUM)/(1.06-RH_00)))
L04108                  endif
L04109               endif
L04110               if (CLDFRA(K).gt.0.) CLDFRA(K) = MAX(0.01, MIN(CLDFRA(K),0.99))
```

讲解：cal_cldfra3（Thompson 2014，配 icloud=3）核心段：① 饱和比按温度三段插值（≥−12°C 用水面 rslf、≤−35°C 用冰面 rsif、中间线性过渡——从 module_mp_thompson USE 进来，与微物理同一套饱和式）；②**modify_qvapor**：有云水时 qv 钳到 ≥qvsw、有云冰时钳到 ≥1.005 • qvsi（保证微过饱和——给冰面云量留活口）；③ **尺度感知临界 RH**：RH_00L=0.77+min(0.22, √(1/(50+gridkm² • dz • 0.01)))、海面 RH_00O=0.85+min(0.14,…)——格距趋于 1 km 时 RH_00 趋近 1（Mocko–Cotton 1995 起点）；④ 暖季 ≥25°C 直接无云；−12~25°C 用 1−√((1.001−RH)/(1.001−RH_00)) 的 Sundqvist 型；低温段按 max_relh>1.12 分 **HRRR 口径（RH 钳 1.45）与 GFS 口径（钳 1.05）**——namelist 挂 max_relh；⑤ 结果钳位 [0.01, 0.99]。

五、臭氧与气溶胶三通道

```
!file: phys/module_radiation_driver.F    L1599-L1617
L01599     Radiation_step: IF ( run_param ) then
L01600  ! CAM-specific additional radiation frequency - cam_abs_freq_s (=21600s by default)
L01601       STEPABS = nint(cam_abs_freq_s/(dt*STEPRA))*STEPRA
L01602       IF (itimestep .eq. 1 .or. mod(itimestep,STEPABS) .eq. 1 + ra_call_offset &
L01603                                        .or. STEPABS .eq. 1 ) THEN
L01604         doabsems = .true.
L01605       ELSE
L01606         doabsems = .false.
L01607       ENDIF
L01608     IF (PRESENT(adapt_step_flag)) THEN
L01609       IF ((adapt_step_flag)) THEN
L01610         IF ( (itimestep .EQ. 1) .OR. (cam_abs_freq_s .EQ. 0) .OR. &
L01611              ( CURR_SECS + dt >= ( INT( CURR_SECS / ( cam_abs_freq_s ) + 1 ) * cam_abs_freq_s) ) ) THEN
L01612           doabsems = .true.
L01613         ELSE
L01614           doabsems = .false.
L01615         ENDIF
L01616       ENDIF
L01617     ENDIF
```

讲解：Radiation_step 门的附加频率：CAM 的吸收率重算 doabsems 按 cam_abs_freq_s（默认 21600 s）比 radt 更稀疏——STEPABS=nint(cam_abs_freq_s/(dt • STEPRA)) • STEPRA，自适应 dt 用物理时刻上取整判定。

```
!file: phys/module_radiation_driver.F    L1624-L1695
L01624  ! Allocate aerosol arrays used by aer_opt = 2 option
L01625     IF ( PRESENT( AOD5502D ) ) THEN
L01626       ! jararias, 2013/11
L01627       IF ( aer_opt .EQ. 2 ) THEN
L01628          swrad_aerosol_select: select case(sw_physics)
L01629
L01630             case(GODDARDSWSCHEME)
L01631                allocate(tauaer_sw(ims:ime, kms:kme, jms:jme, 1:11))
L01632                allocate(ssaaer_sw(ims:ime, kms:kme, jms:jme, 1:11))
L01633                allocate(asyaer_sw(ims:ime, kms:kme, jms:jme, 1:11))
L01634
L01635             case(RRTMG_SWSCHEME &
L01636  #if( BUILD_RRTMG_FAST == 1)
L01637                  ,RRTMG_SWSCHEME_FAST &
L01638  #endif
L01639  #if( BUILD_RRTMK == 1)
L01640                  ,RRTMK_SWSCHEME &
L01641  #endif
L01642                  )
L01643                allocate(tauaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01644                allocate(ssaaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01645                allocate(asyaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01646
L01647          end select swrad_aerosol_select
L01648       ENDIF
L01649     ENDIF
L01650
L01651  ! Allocate aerosol arrays used by aer_opt = 3 option, and explicit AOD from QNWFA+QNIFA(+QNBCA)   (Trude)
L01652     IF (PRESENT(f_qnwfa) .AND. PRESENT(f_qnifa) .AND. PRESENT(taod5503d) .AND.  PRESENT(taod5502d)) THEN
L01653        IF (F_QNWFA .AND. aer_opt.eq.3 .AND.                                  &
L01654                  (sw_physics.eq.RRTMG_SWSCHEME                               &
L01655  #if( BUILD_RRTMG_FAST == 1)
L01656                   .OR. sw_physics.eq.RRTMG_SWSCHEME_FAST                     &
L01657  #endif
L01658  #if( BUILD_RRTMK == 1)
L01659                   .OR. sw_physics.eq.RRTMK_SWSCHEME                          &
L01660  #endif
L01661                   )) THEN
L01662           CALL wrf_debug (150, 'DEBUG-GT:  computing 3D AOD from QNWFA+QNIFA(+QNBCA)')
L01663
L01664           allocate(tauaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01665           allocate(ssaaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01666           allocate(asyaer_sw(ims:ime, kms:kme, jms:jme, 1:14))
L01667
L01668           !$OMP PARALLEL DO   &
L01669           !$OMP PRIVATE ( ij ,i,j,k,its,ite,jts,jte)
L01670           DO ij = 1 , num_tiles
L01671             its = i_start(ij)
L01672             ite = i_end(ij)
L01673             jts = j_start(ij)
L01674             jte = j_end(ij)
L01675
L01676             do j=jts,jte
L01677                do i=its,ite
L01678                   taod5502d(i,j) = 0.0
L01679                end do
L01680             end do
L01681
L01682             call gt_aod (p, DZ8W, t, qv, qnwfa, qnifa, qnbca, taod5503d,        &
L01683                   wif_input_opt, ims,ime, jms,jme, kms,kme,its,ite, jts,jte, kts,kte)
L01684
L01685             do j=jts,jte
L01686                do i=its,ite
L01687                   do k=kts,kte
L01688                      taod5502d(i,j) = taod5502d(i,j) + taod5503d(i,k,j)
L01689                   end do
L01690                end do
L01691             end do
L01692           ENDDO
L01693           !$OMP END PARALLEL DO
L01694        ENDIF
L01695     ENDIF
```

讲解：**aer_opt=2 通道**（AOD5502D 输入）：按短波方案分配 11 波段（Goddard）或 14 波段（RRTMG/RRTMK）的 tauaer/ssaaer/asyaer 数组——波段数差异是方案光谱结构的直接映射。**aer_opt=3 通道**（QNWFA+QNIFA+QNBCA 预报量）：先调 gt_aod 从数浓度逐层算 3D AOD（taod5503d），再柱积分出 taod5502d——**AOD 是模式自己的预报量而非外场**（Trude 的 HRRR-Smoke 路线）。

```
!file: phys/module_radiation_driver.F    L1799-L1837
L01799  !     Interpolating climatological ozone and aerosol to model time and levels
L01800  !     Adapted from camrad code
L01801  #if (EM_CORE==1)
L01802        IF ( o3input .EQ. 2 .AND. id .EQ. 1 ) THEN
L01803  #else
L01804        IF ( o3input .EQ. 2 ) THEN
L01805  #endif
L01806  !       ! Find the current month (adapted from Cavallo)
L01807  !       CALL cam_time_interp( ozmixm, pin, levsiz, date_str, &
L01808  !                             ids , ide , jds , jde , kds , kde , &
L01809  !                             ims , ime , jms , jme , kms , kme , &
L01810  !                             its , ite , jts , jte , kts , kte )
L01811  ! this is the CAM version
L01812  ! interpolate to model time-step
L01813          call ozn_time_int(julday,julian,ozmixm,ozmixt,levsiz,n_ozmixm,    &
L01814                            ids , ide , jds , jde , kds , kde ,     &
L01815                            ims , ime , jms , jme , kms , kme ,     &
L01816                            its , ite , jts , jte , kts , kte )
L01817
L01818  ! interpolate to model model levels
L01819          call ozn_p_int(p ,pin, levsiz, ozmixt, o3rad, &
L01820                            ids , ide , jds , jde , kds , kde ,     &
L01821                            ims , ime , jms , jme , kms , kme ,     &
L01822                            its , ite , jts , jte , kts , kte )
L01823        ENDIF
L01824
L01825        IF ( PRESENT( AEROD ) ) THEN
L01826        IF ( aer_opt .EQ. 1 .AND. id .EQ. 1 ) THEN
L01827           call aer_time_int(julday,julian,aerodm,aerodt,alevsiz,n_ozmixm-1,no_src_types,    &
L01828                            ids , ide , jds , jde , kds , kde ,     &
L01829                            ims , ime , jms , jme , kms , kme ,     &
L01830                            its , ite , jts , jte , kts , kte )
L01831
L01832           call aer_p_int(p ,pina, alevsiz, aerodt, aerod, no_src_types, p8w, AODTOT, &
L01833                            ids , ide , jds , jde , kds , kde ,     &
L01834                            ims , ime , jms , jme , kms , kme ,     &
L01835                            its , ite , jts , jte , kts , kte )
L01836        ENDIF
L01837        ENDIF
```

讲解：**o3input=2**（CAM 臭氧气候场）：ozn_time_int（月平均场时间插值到当前日）+ ozn_p_int（插到模式层）——这两件与积云 driver 的 aer_time_int/aer_p_int 是同源拷贝（差异注记见 code_23 §十二；本文件的 ozn 系多 pin 气压场参数）。aer_opt=1（Tegen 气候场）同理调 aer_time_int/aer_p_int，多 AODTOT 输出。

```
!file: phys/module_radiation_driver.F    L2281-L2337
L02281        ! jararias, 2013/11
L02282        IF ( aer_opt .EQ. 2 ) THEN
L02283        swrad_aerosol_select2: select case(sw_physics)
L02284
L02285           case(RRTMG_SWSCHEME &
L02286  #if( BUILD_RRTMG_FAST == 1)
L02287                ,RRTMG_SWSCHEME_FAST &
L02288  #endif
L02289  #if( BUILD_RRTMK == 1)
L02290                ,RRTMK_SWSCHEME &
L02291  #endif
L02292                )
L02293              call wrf_debug(100, 'call calc_aerosol_rrtmg_sw')
L02294              call calc_aerosol_rrtmg_sw(ht,dz8w,p,t,qv,aer_type,aer_aod550_opt,aer_angexp_opt,    &
L02295                                         aer_ssa_opt,aer_asy_opt,aer_aod550_val,aer_angexp_val,    &
L02296                                         aer_ssa_val,aer_asy_val,aod5502d,angexp2d,aerssa2d,       &
L02297                                         aerasy2d,ims,ime,jms,jme,kms,kme,its,ite,jts,jte,kts,kte, &
L02298                                         tauaer_sw,ssaaer_sw,asyaer_sw                            )
L02299              do j=jts,jte
L02300                 do i=its,ite
L02301                    do k=kts,kte
L02302                       aod5503d(i,k,j)=tauaer_sw(i,k,j,10) ! band at 550 nm
L02303                    end do
L02304                 end do
L02305              end do
L02306
L02307           case default
L02308        end select swrad_aerosol_select2
L02309        ENDIF
L02310        ENDIF
L02311
L02312        !..Different treatment for aer_opt=3 using QNWFA+QNIFA aerosol species (Trude)
L02313
L02314        IF (PRESENT(f_qnwfa) .AND. PRESENT(f_qnifa)) THEN
L02315          IF (F_QNWFA .AND. aer_opt.eq.3 .AND.                    &
L02316                           (sw_physics.eq.RRTMG_SWSCHEME          &
L02317  #if( BUILD_RRTMG_FAST == 1)
L02318                            .OR. sw_physics.eq.RRTMG_SWSCHEME_FAST    &
L02319  #endif
L02320  #if( BUILD_RRTMK == 1)
L02321                            .OR. sw_physics.eq.RRTMK_SWSCHEME &
L02322  #endif
L02323                           )) THEN
L02324            call wrf_debug(100, 'call calc_aerosol_rrtmg_sw with 3D AOD values')
L02325            call calc_aerosol_rrtmg_sw(ht,dz8w,p,t,qv,taer_type,taer_aod550_opt,taer_angexp_opt,  &
L02326                                       taer_ssa_opt,taer_asy_opt,aer_aod550_val,aer_angexp_val,   &
L02327                                       aer_ssa_val,aer_asy_val,taod5502d,angexp2d,aerssa2d,       &
L02328                                       aerasy2d,ims,ime,jms,jme,kms,kme,its,ite,jts,jte,kts,kte,  &
```

```
L02329                                    tauaer_sw,ssaaer_sw,asyaer_sw, taod5503d)
L02330
L02331            do j=jts,jte
L02332               do i=its,ite
L02333                     aod5502d(i,j)= taod5502d(i, j)
L02334               end do
L02335            end do
L02336         ENDIF
L02337       ENDIF
```

讲解：**短波气溶胶光学预处理器挂点**——calc_aerosol_rrtmg_sw（`code_60_module_ra_aerosol.md`）：aer_opt=2 时由 2D AOD550+Angstrom 指数/SSA/ASY 配置展开成 14 波段 3D 光学；aer_opt=3 时吃 3D AOD 版本；**aod5503d 直接取第 10 波段作 550 nm 代表**——两处调用都只在 RRTMG_SW 系下激活。

六、方案分派：长波

```
!file: phys/module_radiation_driver.F    L1839-L1869
L01839       lwrad_select: SELECT CASE(lw_physics)
L01840
L01841          CASE (RRTMSCHEME)
L01842               CALL wrf_debug (100, 'CALL rrtm')
L01843
L01844               IF ( PRESENT(p_top) ) THEN
L01845                  p_top_dummy = p_top
L01846               END IF
L01847
L01848               CALL RRTMLWRAD(                                        &
L01849                    P_TOP=p_top_dummy                                 &
L01850                   ,RTHRATEN=RTHRATEN,RTHRATENC=RTHRATENLWC,GLW=GLW   &
L01851                   ,OLR=RLWTOA,EMISS=EMISS                            &
L01852                   ,QV3D=QV                                           &
L01853                   ,QC3D=QC                                           &
L01854                   ,QR3D=QR                                           &
L01855                   ,QI3D=QI                                           &
L01856                   ,QS3D=QS                                           &
L01857                   ,QG3D=QG                                           &
L01858                   ,P8W=p8w,P3D=p,PI3D=pi,DZ8W=dz8w,TSK=tsk,T3D=t     &
L01859                   ,T8W=t8w,RHO3D=rho,CLDFRA3D=CLDFRA,R=R_d,G=G       &
L01860                   ,F_QV=F_QV,F_QC=F_QC,F_QR=F_QR                     &
L01861                   ,F_QI=F_QI,F_QS=F_QS,F_QG=F_QG                     &
L01862                   ,ICLOUD=icloud,WARM_RAIN=warm_rain                 &
L01863  !ccc Added for time-varying trace gases.
L01864                   ,YR=YR,JULIAN=JULIAN,GHG_INPUT=GHG_INPUT           &
L01865  !ccc
L01866                   ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L01867                   ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L01868                   ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte &
L01869                                                                      )
```

讲解：lwrad_select 首案 RRTM（rrtm 旧版，mp 方案号 1）：ghg_input 直通（CO2/CH4/N2O/CFC11/12 时间变化开关，radconst 之外的气体入口）。

```
!file: phys/module_radiation_driver.F    L2026-L2100
L02026          CASE (RRTMG_LWSCHEME)
L02027
L02028               CALL wrf_debug (100, 'CALL rrtmg_lw')
L02029
L02030               !Need to reset NLAYERS if vertical nesting is used.
L02031               !This code follows that for case RRTMSCHEME within
L02032               !subroutine RRTMLWRAD.
L02033               IF ( PRESENT(p_top) ) THEN
L02034                  p_top_dummy = p_top
L02035               END IF
L02036               IF ( p_top_dummy .GT. 0 ) THEN !
L02037                  !NLAYERS is recalculated
L02038                  !every time the radiation scheme is called. This is
L02039                  !necessary if e_vert parent .NE. e_vert nest since
L02040                  !NLAYERS could then be different for each domain.
L02041                  CALL RRTMG_LWINIT(                                  &
L02042                                p_top, .FALSE. ,                      &
L02043                                ids, ide, jds, jde, kds, kde,         &
L02044                                ims, ime, jms, jme, kms, kme,         &
L02045                                its, ite, jts, jte, kts, kte          )
L02046               ENDIF
L02047
L02048               CALL RRTMG_LWRAD(                                      &
L02049                    RTHRATENLW=RTHRATEN,                              &
L02050                    RTHRATENLWC=RTHRATENLWC,                          &
L02051                    LWUPT=LWUPT,LWUPTC=LWUPTC,LWUPTCLN=LWUPTCLN,      &
L02052                    LWDNT=LWDNT,LWDNTC=LWDNTC,LWDNTCLN=LWDNTCLN,      &
L02053                    LWUPB=LWUPB,LWUPBC=LWUPBC,LWUPBCLN=LWUPBCLN,      &
L02054                    LWDNB=LWDNB,LWDNBC=LWDNBC,LWDNBCLN=LWDNBCLN,      &
L02055                    GLW=GLW,OLR=RLWTOA,LWCF=LWCF,                     &
L02056                    EMISS=EMISS,                                      &
L02057                    P8W=p8w,P3D=p,PI3D=pi,DZ8W=dz8w,TSK=tsk,T3D=t,    &
L02058                    T8W=t8w,RHO3D=rho,R=R_d,G=G,                      &
L02059                    ICLOUD=icloud,WARM_RAIN=warm_rain,CLDFRA3D=CLDFRA,&
L02060                    cldovrlp=cldovrlp,idcor=idcor,XLAT=XLAT,          &
L02061  #if (EM_CORE == 1)
L02062                    LRADIUS=lradius, IRADIUS=iradius,                 &
L02063  #endif
L02064                    IS_CAMMGMP_USED=is_cammgmp_used,                  &
L02065
L02066  !ckay
L02067  !                 CLDFRA_KF3D=cldfra_KF,QC_KF3D=qc_KF,QI_KF3D=qi_KF,&
L02068                    F_ICE_PHY=F_ICE_PHY,F_RAIN_PHY=F_RAIN_PHY,        &
```

```
                    XLAND=XLAND,XICE=XICE,SNOW=SNOW,                  &
                    QV3D=QV,QC3D=QC,QR3D=QR,                          &
                    QI3D=QI,QS3D=QS,QG3D=QG,                          &
                    O3INPUT=O3INPUT,O33D=O3RAD,                       &
                    F_QV=F_QV,F_QC=F_QC,F_QR=F_QR,                    &
                    F_QI=F_QI,F_QS=F_QS,F_QG=F_QG,                    &
                    RE_CLOUD=re_cloud,RE_ICE=re_ice,RE_SNOW=re_snow,  & ! G. Thompson
                    has_reqc=has_reqc,has_reqi=has_reqi,has_reqs=has_reqs, & ! G. Thompson
#if ( WRF_CHEM == 1 )
                    TAUAERLW1=tauaerlw1,TAUAERLW2=tauaerlw2,          & ! jcb
                    TAUAERLW3=tauaerlw3,TAUAERLW4=tauaerlw4,          & ! jcb
                    TAUAERLW5=tauaerlw5,TAUAERLW6=tauaerlw6,          & ! jcb
                    TAUAERLW7=tauaerlw7,TAUAERLW8=tauaerlw8,          & ! jcb
                    TAUAERLW9=tauaerlw9,TAUAERLW10=tauaerlw10,        & ! jcb
                    TAUAERLW11=tauaerlw11,TAUAERLW12=tauaerlw12,      & ! jcb
                    TAUAERLW13=tauaerlw13,TAUAERLW14=tauaerlw14,      & ! jcb
                    TAUAERLW15=tauaerlw15,TAUAERLW16=tauaerlw16,      & ! jcb
                    aer_ra_feedback=aer_ra_feedback,                  &
!jdfcz              progn=progn,prescribe=prescribe,                   &
                    progn=progn,                                      &
#endif
                    calc_clean_atm_diag=calc_clean_atm_diag,          &
                    QNDROP3D=qndrop,F_QNDROP=f_qndrop,                &
!ccc Added for time-varying trace gases.
                    YR=YR,JULIAN=JULIAN,GHG_INPUT=GHG_INPUT,          &
!ccc
                    IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde,&
                    IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme,&
                    ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte,&
                    LWUPFLX=LWUPFLX,LWUPFLXC=LWUPFLXC,                &
                    LWDNFLX=LWDNFLX,LWDNFLXC=LWDNFLXC,                &
                    mp_physics=mp_physics                             )
```

讲解：RRTMG_LW 案（方案号 4）：①**NLAYERS 重置**——垂直嵌套父子域 e_vert 不同时每步重调 RRTMG_LWINIT(p_top)；② 实参全景：八组通量诊断（LWUPT/LWDNT 顶、LWUPB/LWDNB 底 + 各自 C 晴空版 + CLN 无气溶胶版）、cldovrlp/idcor 云重叠选项、F_ICE_PHY 等微物理冰份额、RE_CLOUD 系有效半径（Thompson 微物理直供）、O3INPUT/O33D、WRF_CHEM 的 16 波段 tauaerlw1..16；③ **注意 RTHRATENLW=RTHRATEN——长波倾向直接占用总倾向数组**（配合 § 七 SW 的叠加，见该节讲解）。

七、方案分派：短波

```
!file: phys/module_radiation_driver.F    L2339-L2369
      swrad_select: SELECT CASE(sw_physics)

         CASE (SWRADSCHEME)
              CALL wrf_debug(100, 'CALL swrad')
              CALL SWRAD(                                             &
                   DT=dt,RTHRATEN=rthraten,GSW=gsw                    &
                  ,XLAT=xlat,XLONG=xlong,ALBEDO=albedo                &
#if ( WRF_CHEM == 1 )
                  ,PM2_5_DRY=pm2_5_dry,PM2_5_WATER=pm2_5_water        &
                  ,PM2_5_DRY_EC=pm2_5_dry_ec                          &
#endif
                  ,RHO_PHY=rho,T3D=t                                  &
                  ,P3D=p,PI3D=pi,DZ8W=dz8w,GMT=gmt                    &
                  ,R=r_d,CP=cp,G=g,JULDAY=julday                      &
                  ,GHG_INPUT=ghg_input                                &
                  ,XTIME=xtime,DECLIN=declin,SOLCON=solcon            &
                  ,RADFRQ=radt,ICLOUD=icloud,DEGRAD=degrad            &
                  ,warm_rain=warm_rain                                &
                  ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
                  ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
                  ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte &
                  ,QV3D=qv                                            &
                  ,QC3D=qc                                            &
                  ,QR3D=qr                                            &
                  ,QI3D=qi                                            &
                  ,QS3D=qs                                            &
                  ,QG3D=qg                                            &
                  ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                      &
                  ,F_QI=f_qi,F_QS=f_qs,F_QG=f_qg                      &
                  ,coszen=coszen,julian=julian                        &
                  ,obscur=obscur_loc                                  )
```

讲解：swrad_select 首案 SW（Dudhia 短波，方案号 1）：单波段 +云散射参数化，吃 coszen 与 ghg_input。

```
!file: phys/module_radiation_driver.F    L2533-L2698
         CASE (RRTMG_SWSCHEME)

              CALL wrf_debug(100, 'CALL rrtmg_sw')

              if ( direct_sw_feedback .and. feedback_is_ready ) then
                 proceed_cmaq_sw = .true.

                 if (present(mass_ws_i)) then

                    mass_ws_i(:,kte+1,:) = mass_ws_i(:,kte,:)
                    mass_ws_j(:,kte+1,:) = mass_ws_j(:,kte,:)
                    mass_ws_k(:,kte+1,:) = mass_ws_k(:,kte,:)

                    mass_in_i(:,kte+1,:) = mass_in_i(:,kte,:)
                    mass_in_j(:,kte+1,:) = mass_in_j(:,kte,:)
                    mass_in_k(:,kte+1,:) = mass_in_k(:,kte,:)
```

```
L02549
L02550                     mass_ec_i(:,kte+1,:) = mass_ec_i(:,kte,:)
L02551                     mass_ec_j(:,kte+1,:) = mass_ec_j(:,kte,:)
L02552                     mass_ec_k(:,kte+1,:) = mass_ec_k(:,kte,:)
L02553
L02554                     mass_ss_i(:,kte+1,:) = mass_ss_i(:,kte,:)
L02555                     mass_ss_j(:,kte+1,:) = mass_ss_j(:,kte,:)
L02556                     mass_ss_k(:,kte+1,:) = mass_ss_k(:,kte,:)
L02557
L02558                     mass_h2o_i(:,kte+1,:) = mass_h2o_i(:,kte,:)
L02559                     mass_h2o_j(:,kte+1,:) = mass_h2o_j(:,kte,:)
L02560                     mass_h2o_k(:,kte+1,:) = mass_h2o_k(:,kte,:)
L02561
L02562                     dgn_i(:,kte+1,:) = dgn_i(:,kte,:)
L02563                     dgn_j(:,kte+1,:) = dgn_j(:,kte,:)
L02564                     dgn_k(:,kte+1,:) = dgn_k(:,kte,:)
L02565
L02566                     sig_i(:,kte+1,:) = sig_i(:,kte,:)
L02567                     sig_j(:,kte+1,:) = sig_j(:,kte,:)
L02568                     sig_k(:,kte+1,:) = sig_k(:,kte,:)
L02569                  end if
L02570               else
L02571                  proceed_cmaq_sw = .false.
L02572               end if
L02573
L02574               CALL RRTMG_SWRAD(                                          &
L02575                       RTHRATENSW=RTHRATENSW,                             &
L02576                       RTHRATENSWC=RTHRATENSWC,                           &
L02577                       SWUPT=SWUPT,SWUPTC=SWUPTC,SWUPTCLN=SWUPTCLN,       &
L02578                       SWDNT=SWDNT,SWDNTC=SWDNTC,SWDNTCLN=SWDNTCLN,       &
L02579                       SWUPB=SWUPB,SWUPBC=SWUPBC,SWUPBCLN=SWUPBCLN,       &
L02580                       SWDNB=SWDNB,SWDNBC=SWDNBC,SWDNBCLN=SWDNBCLN,       &
L02581                       SWCF=SWCF,GSW=GSW,                                 &
L02582                       XTIME=XTIME,GMT=GMT,XLAT=XLAT,XLONG=XLONG,         &
L02583                       RADT=RADT,DEGRAD=DEGRAD,DECLIN=DECLIN,             &
L02584                       COSZR=COSZR,JULDAY=JULDAY,SOLCON=SOLCON,           &
L02585                       ALBEDO=ALBEDO,t3d=t,t8w=t8w,TSK=TSK,               &
L02586                       p3d=p,p8w=p8w,pi3d=pi,rho3d=rho,                   &
L02587                       dz8w=dz8w,CLDFRA3D=CLDFRA,GHG_INPUT=GHG_INPUT,     &
L02588  #if (EM_CORE == 1)
L02589                       LRADIUS=lradius, IRADIUS=iradius,                  &
L02590  #endif
L02591                       IS_CAMMGMP_USED=is_cammgmp_used,                   &
L02592                       R=R_D,G=G,                &
L02593  !ckay
L02594  !                    CLDFRA_KF3D=cldfra_KF,QC_KF3D=qc_KF,QI_KF3D=qi_KF,&
L02595                       ICLOUD=icloud,WARM_RAIN=warm_rain,                 &
L02596                       cldovrlp=cldovrlp,idcor=idcor,                     &
L02597                       F_ICE_PHY=F_ICE_PHY,F_RAIN_PHY=F_RAIN_PHY,         &
L02598                       XLAND=XLAND,XICE=XICE,SNOW=SNOW,                   &
L02599                       QV3D=qv,QC3D=qc,QR3D=qr,                           &
L02600                       QI3D=qi,QS3D=qs,QG3D=qg,                           &
L02601                       O3INPUT=O3INPUT,O33D=O3RAD,                        &
L02602                       AER_OPT=AER_OPT,aerod=aerod,no_src=no_src_types,   &
L02603                       ALSWVISDIR=alswvisdir ,ALSWVISDIF=alswvisdif,      &  !Zhenxin ssib alb comp (06/2010)
L02604                       ALSWNIRDIR=alswnirdir ,ALSWNIRDIF=alswnirdif,      &  !Zhenxin ssib alb comp (06/2010)
L02605                       SWVISDIR=swvisdir ,SWVISDIF=swvisdif,              &  !Zhenxin ssib swr comp (06/2010)
L02606                       SWNIRDIR=swnirdir ,SWNIRDIF=swnirdif,              &  !Zhenxin ssib swr comp (06/2010)
L02607                       SF_SURFACE_PHYSICS=sf_surface_physics,             &  !Zhenxin ssib sw_phy   (06/2010)
L02608                       F_QV=f_qv,F_QC=f_qc,F_QR=f_qr,                     &
L02609                       F_QI=f_qi,F_QS=f_qs,F_QG=f_qg,                     &
L02610                       RE_CLOUD=re_cloud,RE_ICE=re_ice,RE_SNOW=re_snow,  & ! G. Thompson
L02611                       has_reqc=has_reqc,has_reqi=has_reqi,has_reqs=has_reqs, & ! G. Thompson
L02612  #if ( WRF_CHEM == 1 )
L02613                       TAUAER300=tauaer300,TAUAER400=tauaer400,           & ! jcb
L02614                       TAUAER600=tauaer600,TAUAER999=tauaer999,           & ! jcb
L02615                       GAER300=gaer300,GAER400=gaer400,                   & ! jcb
L02616                       GAER600=gaer600,GAER999=gaer999,                   & ! jcb
L02617                       WAER300=waer300,WAER400=waer400,                   & ! jcb
L02618                       WAER600=waer600,WAER999=waer999,                   & ! jcb
L02619                       aer_ra_feedback=aer_ra_feedback,                   &
L02620  !jdfcz               progn=progn,prescribe=prescribe,                   &
L02621                       progn=progn,                                       &
L02622  #endif
L02623                       calc_clean_atm_diag=calc_clean_atm_diag,           &
L02624                       QNDROP3D=qndrop,F_QNDROP=f_qndrop,                 &
L02625                       IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde,&
L02626                       IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme,&
L02627                       ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte,&
L02628                       SWUPFLX=SWUPFLX,SWUPFLXC=SWUPFLXC,                 &
L02629                       SWDNFLX=SWDNFLX,SWDNFLXC=SWDNFLXC,                 &
L02630                       tauaer3d_sw=tauaer_sw,                              & ! jararias 2013/11
L02631                       ssaaer3d_sw=ssaaer_sw,                              & ! jararias 2013/11
L02632                       asyaer3d_sw=asyaer_sw,                              & ! jararias 2013/11
L02633                       swddir=swddir,swddni=swddni,swddif=swddif,          & ! jararias 2013/08/10
L02634                       swdownc=swdownc, swddnic=swddnic, swddirc=swddirc, & ! PAJ
L02635                       obscur=obscur_loc,                                  &
L02636                       xcoszen=coszen,yr=yr,julian=julian,mp_physics=mp_physics, &   ! jararias 2013/08/14
L02637                       proceed_cmaq_sw=proceed_cmaq_sw,                    & ! WRF-CMAQ coupled model
L02638                       nmode=3,                                            & ! WRF-CMAQ coupled model
L02639                       mass_ws_i=mass_ws_i,                                & ! WRF-CMAQ coupled model
L02640                       mass_ws_j=mass_ws_j,                                & ! WRF-CMAQ coupled model
L02641                       mass_ws_k=mass_ws_k,                                & ! WRF-CMAQ coupled model
L02642                       mass_in_i=mass_in_i,                                & ! WRF-CMAQ coupled model
L02643                       mass_in_j=mass_in_j,                                & ! WRF-CMAQ coupled model
L02644                       mass_in_k=mass_in_k,                                & ! WRF-CMAQ coupled model
L02645                       mass_ec_i=mass_ec_i,                                & ! WRF-CMAQ coupled model
L02646                       mass_ec_j=mass_ec_j,                                & ! WRF-CMAQ coupled model
L02647                       mass_ec_k=mass_ec_k,                                & ! WRF-CMAQ coupled model
L02648                       mass_ss_i=mass_ss_i,                                & ! WRF-CMAQ coupled model
L02649                       mass_ss_j=mass_ss_j,                                & ! WRF-CMAQ coupled model
L02650                       mass_ss_k=mass_ss_k,                                & ! WRF-CMAQ coupled model
L02651                       mass_h2o_i=mass_h2o_i,                              & ! WRF-CMAQ coupled model
L02652                       mass_h2o_j=mass_h2o_j,                              & ! WRF-CMAQ coupled model
L02653                       mass_h2o_k=mass_h2o_k,                              & ! WRF-CMAQ coupled model
L02654                       dgn_i=dgn_i,                                        & ! WRF-CMAQ coupled model
L02655                       dgn_j=dgn_j,                                        & ! WRF-CMAQ coupled model
L02656                       dgn_k=dgn_k,                                        & ! WRF-CMAQ coupled model
L02657                       sig_i=sig_i,                                        & ! WRF-CMAQ coupled model
```

```
L02658                        sig_j=sig_j,                                          & ! WRF-CMAQ coupled model
L02659                        sig_k=sig_k,                                          & ! WRF-CMAQ coupled model
L02660                        gtauxar_01=sw_gtauxar_01,                             & ! WRF-CMAQ coupled model
L02661                        gtauxar_02=sw_gtauxar_02,                             & ! WRF-CMAQ coupled model
L02662                        gtauxar_03=sw_gtauxar_03,                             & ! WRF-CMAQ coupled model
L02663                        gtauxar_04=sw_gtauxar_04,                             & ! WRF-CMAQ coupled model
L02664                        gtauxar_05=sw_gtauxar_05,                             & ! WRF-CMAQ coupled model
L02665                        asy_fac_01=sw_asy_fac_01,                             & ! WRF-CMAQ coupled model
L02666                        asy_fac_02=sw_asy_fac_02,                             & ! WRF-CMAQ coupled model
L02667                        asy_fac_03=sw_asy_fac_03,                             & ! WRF-CMAQ coupled model
L02668                        asy_fac_04=sw_asy_fac_04,                             & ! WRF-CMAQ coupled model
L02669                        asy_fac_05=sw_asy_fac_05,                             & ! WRF-CMAQ coupled model
L02670                        ssa_01=sw_ssa_01,                                     & ! WRF-CMAQ coupled model
L02671                        ssa_02=sw_ssa_02,                                     & ! WRF-CMAQ coupled model
L02672                        ssa_03=sw_ssa_03,                                     & ! WRF-CMAQ coupled model
L02673                        ssa_04=sw_ssa_04,                                     & ! WRF-CMAQ coupled model
L02674                        ssa_05=sw_ssa_05,                                     & ! WRF-CMAQ coupled model
L02675                        sw_zbbcddir=sw_zbbcddir,                              & ! WRF-CMAQ coupled model
L02676                        sw_dirdflux=sw_dirdflux,                              & ! WRF-CMAQ coupled model
L02677                        sw_difdflux=sw_difdflux                               & ! WRF-CMAQ coupled model
L02678                                                                              )
L02679
L02680                ! = WRF-CMAQ twoway coupled model
L02681                if (proceed_cmaq_sw) then
L02682                   DO j=jts,jte
L02683                      DO i=its,ite
L02684                         sw_ttauxar_01(i,j) = sum(sw_gtauxar_01(i,:,j))
L02685                         sw_ttauxar_02(i,j) = sum(sw_gtauxar_02(i,:,j))
L02686                         sw_ttauxar_03(i,j) = sum(sw_gtauxar_03(i,:,j))
L02687                         sw_ttauxar_04(i,j) = sum(sw_gtauxar_04(i,:,j))
L02688                         sw_ttauxar_05(i,j) = sum(sw_gtauxar_05(i,:,j))
L02689                      END DO
L02690                   END DO
L02691                end if
L02692
L02693                DO j=jts,jte
L02694                DO k=kts,kte
L02695                DO i=its,ite
L02696                   RTHRATEN(I,K,J)=RTHRATEN(I,K,J)+RTHRATENSW(I,K,J)
L02697                ENDDO
L02698                ENDDO
```

讲解：RRTMG_SW 案（方案号 4）：①WRF-CMAQ 双向耦合前置——direct_sw_feedback 时把五模态气溶胶质量/谱参数在 kte+1 复制底层；② 实参全景与 LW 对称，另有 **SSiB 四分量反照率分解**（VIS/NIR× 直射/散射）与 tauaer_sw/ssaaer_sw/asyaer_sw 14 波段（aer_opt=1/2 的输入）、swddir/swddni/swddif 直散分解输出（太阳能应用）、obscur 日食；③ **关键口径：案尾 RTHRATEN=RTHRATEN+RTHRATENSW——短波倾向叠加进总倾向；而 § 六 LW 案是 RTHRATENLW=RTHRATEN 直接占用**。两者配套依赖调用序「先长波后短波」——改分发顺序会引入符号灾难，这是本文件最要紧的隐式约定。

八、收尾：FARMs、状态恢复与通量累积

```
!file: phys/module_radiation_driver.F    L3040-L3100
L03040
L03041          end select
L03042
L03043          !---------------
L03044          !$OMP PARALLEL DO   &
L03045          !$OMP PRIVATE ( ij ,i,j,k,its,ite,jts,jte)
L03046          do ij = 1,num_tiles
L03047            its = i_start(ij)
L03048            ite = i_end(ij)
L03049            jts = j_start(ij)
L03050            jte = j_end(ij)
L03051
L03052            if (pert_farms .and. multi_perturb == 1) then
L03053              allocate (qv_tmp(its:ite, kts:kte, jts:jte))
L03054              allocate (qc_tmp(its:ite, kts:kte, jts:jte))
L03055              allocate (qs_tmp(its:ite, kts:kte, jts:jte))
L03056
L03057              call Add_multi_perturb_swrad_perturbations (perts_albedo, perts_aod, perts_angstrom, &
L03058                  perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow, pert_farms_albedo, pert_farms_aod, &
L03059                  pert_farms_angexp, pert_farms_aerasy, pert_farms_qv, pert_farms_qc, pert_farms_qs, &
L03060                  albedo, aod5502d, angexp2d, aerasy2d, qv, qc, qs, qv_tmp, qc_tmp, qs_tmp, its, ite, jts, jte, &
L03061                  ims, ime, jms, jme, kms, kme, kts, kte)
L03062            end if
L03063
L03064            call farms_driver (ims, ime, jms, jme, its, ite, jts, jte, kms, kme, kts, kte, &
L03065                 p8w, rho, dz8w, albedo, aer_opt, aerssa2d, aerasy2d, aod5502d, angexp2d,  &
L03066                 coszen_loc, qv, qi, qs, qc, re_cloud, re_ice, re_snow,                   &
L03067                 julian, swdown2, swddir2, swddni2, swddif2, swdownc2, swddnic2,          &
L03068                 has_reqc, has_reqi, has_reqs, CLDFRA)
L03069
L03070            if (pert_farms .and. multi_perturb == 1) then
L03071              call Remove_multi_perturb_swrad_perturbations (perts_albedo, perts_aod, perts_angstrom, &
L03072                  perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow, pert_farms_albedo, pert_farms_aod, &
L03073                  pert_farms_angexp, pert_farms_aerasy, pert_farms_qv, pert_farms_qc, pert_farms_qs, &
L03074                  albedo, aod5502d, angexp2d, aerasy2d, qv, qc, qs, qv_tmp, qc_tmp, qs_tmp, its, ite, jts, jte, &
L03075                  ims, ime, jms, jme, kms, kme, kts, kte)
L03076
L03077              deallocate (qv_tmp)
L03078              deallocate (qc_tmp)
L03079              deallocate (qs_tmp)
L03080            end if
L03081          enddo
L03082          !$OMP END PARALLEL DO
L03083
```

```
L03084         if (couple_farms) then
L03085           !$OMP PARALLEL DO   &
L03086           !$OMP PRIVATE (ij, i, j, its, ite, jts, jte)
L03087           do ij = 1,num_tiles
L03088             its = i_start(ij)
L03089             ite = i_end(ij)
L03090             jts = j_start(ij)
L03091             jte = j_end(ij)
L03092             do j = jts, jte
L03093               do i = its, ite
L03094                 swdown(i, j) = swdown2(i, j)
L03095               end do
L03096             end do
L03097           end do
L03098           !$OMP END PARALLEL DO
L03099         end if
L03100       end if
```

讲解：**FARMs（Fast All-sky Radiation Model for Solar applications）**独立诊断通道：SPP 扰动包裹（multi_perturb=1 时 Add_multi_perturb 先改 albedo/aod/angexp/asy 与 qv/qc/qs，farms_driver 算 swdown2/swddir2/swddni2/swddif2，Remove 还原）；couple_farms 时用 FARMs 的 swdown2 覆盖模式 swdown（光伏应用流）。

```
!file: phys/module_radiation_driver.F    L3102-L3167
L03102
L03103   solar_opt_local = 0
L03104   IF ( PRESENT(solar_opt) ) THEN
L03105      solar_opt_local = solar_opt
L03106   END IF
L03107   IF (run_param .or. solar_opt_local == 1 .or. swint_opt == 2) THEN
L03108      do ij = 1,num_tiles
L03109        its = i_start(ij)
L03110        ite = i_end(ij)
L03111        jts = j_start(ij)
L03112        jte = j_end(ij)
L03113   ! Save resolved + unresolved hydrometeor mass mixing ratios for Solar diag
L03114        IF ( solar_opt_local == 1 ) THEN
L03115          IF ( PRESENT(qc_tot) .AND. PRESENT(qi_tot) ) THEN
L03116            IF ( F_QC ) THEN
L03117               DO j=jts,jte
L03118               DO k=kts,kte
L03119               DO i=its,ite
L03120                 qc_tot(i,k,j) = qc(i,k,j)
L03121               ENDDO
L03122               ENDDO
L03123               ENDDO
L03124            ENDIF
L03125            IF ( F_QI ) THEN
L03126               DO j=jts,jte
L03127               DO k=kts,kte
L03128               DO i=its,ite
L03129                 qi_tot(i,k,j) = qi(i,k,j)
L03130               ENDDO
L03131               ENDDO
L03132               ENDDO
L03133            ENDIF
L03134          END IF
L03135        ENDIF
L03136   ! Restore qc & qi for any model physics configuration
L03137        IF ( F_QC ) THEN
L03138           DO j=jts,jte
L03139           DO k=kts,kte
L03140           DO i=its,ite
L03141             qc(i,k,j) = qc_save(i,k,j)
L03142           ENDDO
L03143           ENDDO
L03144           ENDDO
L03145        ENDIF
L03146        IF ( F_QI ) THEN
L03147           DO j=jts,jte
L03148           DO k=kts,kte
L03149           DO i=its,ite
L03150             qi(i,k,j) = qi_save(i,k,j)
L03151           ENDDO
L03152           ENDDO
L03153           ENDDO
L03154        ENDIF
L03155
L03156        IF (aercu_opt.gt.0.0) THEN
L03157        IF ( F_QS ) THEN
L03158           DO j=jts,jte
L03159           DO k=kts,kte
L03160           DO i=its,ite
L03161             qs(i,k,j) = qs_save(i,k,j)
L03162           ENDDO
L03163           ENDDO
L03164           ENDDO
L03165        ENDIF
L03166        END IF
L03167      end do
```

讲解：**水物质恢复**——把 § 三备份的 qc_save/qi_save（与 aercu_opt 的 qs_save）写回；solar_opt=1 时先把「格点 +积云云水总量」存 qc_tot/qi_tot 供太阳能诊断，再恢复。

```
!file: phys/module_radiation_driver.F    L3169-L3250
L03169
L03170       accumulate_lw_select: SELECT CASE(lw_physics)
L03171
L03172       CASE (CAMLWSCHEME,&
L03173             RRTMG_LWSCHEME &
L03174  #if( BUILD_RRTMG_FAST == 1)
```

```
L03175             ,RRTMG_LWSCHEME_FAST &
L03176  #endif
L03177  #if( BUILD_RRTMK == 1)
L03178             ,RRTMK_LWSCHEME &
L03179  #endif
L03180            )
L03181     IF(PRESENT(LWUPTC))THEN
L03182  !  EM calls every DT
L03183     DTaccum = DT
L03184     !$OMP PARALLEL DO   &
L03185     !$OMP PRIVATE ( ij ,i,j,k,its,ite,jts,jte)
L03186
L03187     DO ij = 1 , num_tiles
L03188       its = i_start(ij)
L03189       ite = i_end(ij)
L03190       jts = j_start(ij)
L03191       jte = j_end(ij)
L03192
L03193          DO j=jts,jte
L03194          DO i=its,ite
L03195             ACLWUPT(I,J) = ACLWUPT(I,J) + LWUPT(I,J)*DTaccum
L03196             ACLWUPTC(I,J) = ACLWUPTC(I,J) + LWUPTC(I,J)*DTaccum
L03197             ACLWDNT(I,J) = ACLWDNT(I,J) + LWDNT(I,J)*DTaccum
L03198             ACLWDNTC(I,J) = ACLWDNTC(I,J) + LWDNTC(I,J)*DTaccum
L03199             ACLWUPB(I,J) = ACLWUPB(I,J) + LWUPB(I,J)*DTaccum
L03200             ACLWUPBC(I,J) = ACLWUPBC(I,J) + LWUPBC(I,J)*DTaccum
L03201             ACLWDNB(I,J) = ACLWDNB(I,J) + LWDNB(I,J)*DTaccum
L03202             ACLWDNBC(I,J) = ACLWDNBC(I,J) + LWDNBC(I,J)*DTaccum
L03203          ENDDO
L03204          ENDDO
L03205     ENDDO
L03206     !$OMP END PARALLEL DO
L03207     ENDIF
L03208       CASE DEFAULT
L03209       END SELECT accumulate_lw_select
L03210
L03211       accumulate_sw_select: SELECT CASE(sw_physics)
L03212
L03213       CASE (CAMSWSCHEME,&
L03214             RRTMG_SWSCHEME &
L03215  #if( BUILD_RRTMG_FAST == 1)
L03216             ,RRTMG_SWSCHEME_FAST &
L03217  #endif
L03218  #if( BUILD_RRTMK == 1)
L03219             ,RRTMK_SWSCHEME &
L03220  #endif
L03221            )
L03222     IF(PRESENT(SWUPTC))THEN
L03223  !  EM calls the driver every DT
L03224     DTaccum = DT
L03225     !$OMP PARALLEL DO   &
L03226     !$OMP PRIVATE ( ij ,i,j,k,its,ite,jts,jte)
L03227
L03228     DO ij = 1 , num_tiles
L03229       its = i_start(ij)
L03230       ite = i_end(ij)
L03231       jts = j_start(ij)
L03232       jte = j_end(ij)
L03233
L03234          DO j=jts,jte
L03235          DO i=its,ite
L03236             ACSWUPT(I,J) = ACSWUPT(I,J) + SWUPT(I,J)*DTaccum
L03237             ACSWUPTC(I,J) = ACSWUPTC(I,J) + SWUPTC(I,J)*DTaccum
L03238             ACSWDNT(I,J) = ACSWDNT(I,J) + SWDNT(I,J)*DTaccum
L03239             ACSWDNTC(I,J) = ACSWDNTC(I,J) + SWDNTC(I,J)*DTaccum
L03240             ACSWUPB(I,J) = ACSWUPB(I,J) + SWUPB(I,J)*DTaccum
L03241             ACSWUPBC(I,J) = ACSWUPBC(I,J) + SWUPBC(I,J)*DTaccum
L03242             ACSWDNB(I,J) = ACSWDNB(I,J) + SWDNB(I,J)*DTaccum
L03243             ACSWDNBC(I,J) = ACSWDNBC(I,J) + SWDNBC(I,J)*DTaccum
L03244          ENDDO
L03245          ENDDO
L03246     ENDDO
L03247     !$OMP END PARALLEL DO
L03248     ENDIF
L03249
L03250       CASE DEFAULT
```

讲解：**ACC 通量累积**：LWUPT 等瞬时通量 ×DT 累加进 ACLWUPT 系（CAM/RRTMG 系方案才有这些出参）——wrfout 里 ACLW* 是时间平均值的基础；DTaccum=DT（EM 每步都调 driver 的注释）。

```
!file: phys/module_radiation_driver.F    L3252-L3280
L03252
L03253  ! compute cloud diagnosis (random overlapping)
L03254  ! by ZCX
L03255   IF ( PRESENT ( CLDFRA ) .AND. PRESENT ( CLDT ) .AND.        &
L03256         PRESENT ( F_QC ) .AND. PRESENT ( F_QI ) ) THEN
L03257
L03258     DO ij = 1 , num_tiles
L03259       its = i_start(ij)
L03260       ite = i_end(ij)
L03261       jts = j_start(ij)
L03262       jte = j_end(ij)
L03263
L03264          DO j=jts,jte
L03265          DO i=its,ite
L03266            cldji=1.0
L03267            do k=kte-1,kts,-1
L03268              cldji=cldji*(1.0-cldfra(i,k,j))
L03269            enddo
L03270            cldt(i,j)=1.0-cldji
L03271  !         cldlji=1.0
L03272  !         do k=kte-1,kts,-1
L03273  !         if(znu(k).ge.0.69) then
L03274  !           cldlji=cldlji*(1.0-cldfra(i,k,j))
```

```
L03275  !         endif
L03276  !         enddo
L03277  !         cldl(i,j)=1.0-cldlji
L03278          END DO
L03279          END DO
L03280       END DO
```

讲解：云顶诊断（ZCX）：随机重叠假设下 CLDT=1−∏(1−cldfra) 自顶层累乘——与 RRTMG 内部的重叠处理（code_05 的 cldovrlp）口径呼应，注意这里从 kte−1 起（跳过最顶层）。

```
!file: phys/module_radiation_driver.F    L5623-L5666    (subroutine Add_multi_perturb_swrad_perturbations)
L05623      subroutine  Add_multi_perturb_swrad_perturbations (perts_albedo, perts_aod, perts_angstrom, &
L05624          perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow, pert_farms_albedo, pert_farms_aod, &
L05625          pert_farms_angexp, pert_farms_aerasy, pert_farms_qv, pert_farms_qc, pert_farms_qs, &
L05626          albedo, aod5502d, angexp2d, aerasy2d, qv, qc, qs, qv_tmp, qc_tmp, qs_tmp, its, ite, jts, jte, &
L05627          ims, ime, jms, jme, kms, kme, kts, kte)
L05628
L05629        implicit none
L05630
L05631        integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L05632        real, intent(in) :: pert_farms_albedo, pert_farms_aod, pert_farms_angexp, pert_farms_aerasy, &
L05633            pert_farms_qv, pert_farms_qc, pert_farms_qs
L05634        real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_albedo, perts_aod, perts_angstrom, &
L05635                perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow
L05636        real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: qv, qc, qs
L05637        real, dimension(ims:ime, jms:jme), intent (inout) :: albedo, aod5502d, angexp2d, aerasy2d
L05638        real, dimension (its:ite, kts:kte, jts:jte), intent(out) :: qv_tmp, qc_tmp, qs_tmp
L05639
L05640        integer :: i, j, k
L05641
L05642
L05643        k = 1
L05644        do j = jts, jte
L05645          do i = its, ite
L05646            albedo(i, j) = min (max (ALBEDO_MIN, (1.0 + perts_albedo(i, k, j) * pert_farms_albedo) * albedo(i, j)),
↪   AERASY_MAX)
L05647            angexp2d(i, j) = max (ANGEXP_MIN, (1.0 + perts_angstrom(i, k, j) * pert_farms_angexp) * angexp2d(i, j))
L05648            aod5502d(i, j) = max (AOD_MIN, (1.0 + perts_aod(i, k, j) * pert_farms_aod) * aod5502d(i, j))
L05649            aerasy2d(i, j) = min (max (AERASY_MIN, (1.0 + perts_assymfac(i, k, j) * pert_farms_aerasy) * aerasy2d(i, j)),
↪   AERASY_MAX)
L05650          end do
L05651        end do
L05652
L05653        do j = jts, jte
L05654          do k = kts, kte
L05655            do i = its, ite
L05656              qv_tmp(i, k, j) = qv(i, k, j)
L05657              qv(i, k, j) = max (QVAPOR_MIN, (1.0 + perts_qvapor(i, k, j) * pert_farms_qv) * qv(i, k, j))
L05658              qc_tmp(i, k, j) = qc(i, k, j)
L05659              qc(i, k, j) = max (QCLOUD_MIN, (1.0 + perts_qcloud(i, k, j) * pert_farms_qc) * qc(i, k, j))
L05660              qs_tmp(i, k, j) = qs(i, k, j)
L05661              qs(i, k, j) = max (QSNOW_MIN, (1.0 + perts_qsnow(i, k, j) * pert_farms_qs) * qs(i, k, j))
L05662            end do
L05663          end do
L05664        end do
L05665
L05666      end subroutine  Add_multi_perturb_swrad_perturbations
```

```
!file: phys/module_radiation_driver.F    L5668-L5708    (subroutine Remove_multi_perturb_swrad_perturbations)
L05668      subroutine  Remove_multi_perturb_swrad_perturbations (perts_albedo, perts_aod, perts_angstrom, &
L05669          perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow, pert_farms_albedo, pert_farms_aod, &
L05670          pert_farms_angexp, pert_farms_aerasy, pert_farms_qv, pert_farms_qc, pert_farms_qs, &
L05671          albedo, aod5502d, angexp2d, aerasy2d, qv, qc, qs, qv_tmp, qc_tmp, qs_tmp, its, ite, jts, jte, &
L05672          ims, ime, jms, jme, kms, kme, kts, kte)
L05673
L05674        implicit none
L05675
L05676        integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L05677        real, intent(in) :: pert_farms_albedo, pert_farms_aod, pert_farms_angexp, pert_farms_aerasy, &
L05678            pert_farms_qv, pert_farms_qc, pert_farms_qs
L05679        real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_albedo, perts_aod, perts_angstrom, &
L05680                perts_assymfac, perts_qvapor, perts_qcloud, perts_qsnow
L05681        real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: qv, qc, qs
L05682        real, dimension(ims:ime, jms:jme), intent (inout) :: albedo, aod5502d, angexp2d, aerasy2d
L05683        real, dimension (its:ite, kts:kte, jts:jte), intent(in) :: qv_tmp, qc_tmp, qs_tmp
L05684
L05685        integer :: i, j, k
L05686
L05687
L05688        k = 1
L05689        do j = jts, jte
L05690          do i = its, ite
L05691            albedo(i, j) = min (max (ALBEDO_MIN, albedo(i, j) / (1.0 + perts_albedo(i, k, j) * pert_farms_albedo)),
↪   ALBEDO_MAX)
L05692            angexp2d(i, j) = max (ANGEXP_MIN, angexp2d(i, j) / (1.0 + perts_angstrom(i, k, j) * pert_farms_angexp))
L05693            aod5502d(i, j) = max (AOD_MIN, aod5502d(i, j) / (1.0 + perts_aod(i, k, j) * pert_farms_aod))
L05694            aerasy2d(i, j) = min (max (AERASY_MIN, aerasy2d(i, j) / (1.0 + perts_assymfac(i, k, j) * pert_farms_aerasy)),
↪   AERASY_MAX)
L05695          end do
L05696        end do
L05697
L05698        do j = jts, jte
L05699          do k = kts, kte
L05700            do i = its, ite
L05701              qv(i, k, j) = max (QVAPOR_MIN, qv(i, k, j) - perts_qvapor(i, k, j) * pert_farms_qv * qv_tmp(i, k, j))
L05702              qc(i, k, j) = max (QCLOUD_MIN, qc(i, k, j) - perts_qcloud(i, k, j) * pert_farms_qc * qc_tmp(i, k, j))
L05703              qs(i, k, j) = max (QSNOW_MIN, qs(i, k, j) - perts_qsnow(i, k, j) * pert_farms_qs * qs_tmp(i, k, j))
L05704            end do
L05705          end do
L05706        end do
L05707
L05708      end subroutine  Remove_multi_perturb_swrad_perturbations
```

讲解：SPP 加/减对（stochastic perturbed physics）：四张 2D 地表量用**乘除对称**（albedo 等乘 (1+ε•幅度) 后除回去，钳位用模块头常量 ALBEDO_MIN 等），三个水物质用**备份-相减**（Remove 里是 qv−ε•qv_tmp 减法，与 PBL driver 的 SPP 包裹同款「q 用减法防负」约定，见 code_22）。本对的调用点在 § 八 FARMs 块与 § 四 icloud=3 段。

九、地形阴影 pre_radiation_driver

```
!file: phys/module_radiation_driver.F    L3285-L3463    (subroutine pre_radiation_driver)
L03285     SUBROUTINE pre_radiation_driver ( grid, config_flags                  &
L03286                 ,itimestep, ra_call_offset                               &
L03287                 ,XLAT, XLONG, GMT, julian, xtime, RADT, STEPRA           &
L03288                 ,ht,dx,dy,dx2d,area2d                                    &
L03289                 ,sina,cosa,shadowmask,slope_rad ,topo_shading            &
L03290                 ,shadlen,ht_shad,ht_loc                                  &
L03291                 ,ht_shad_bxs, ht_shad_bxe                                &
L03292                 ,ht_shad_bys, ht_shad_bye                                &
L03293                 ,nested, min_ptchsz                                      &
L03294                 ,spec_bdy_width                                          &
L03295                 ,ids, ide, jds, jde, kds, kde                            &
L03296                 ,ims, ime, jms, jme, kms, kme                            &
L03297                 ,ips, ipe, jps, jpe, kps, kpe                            &
L03298                 ,i_start, i_end                                          &
L03299                 ,j_start, j_end                                          &
L03300                 ,kts, kte                                                &
L03301                 ,num_tiles                                               )
L03302
L03303     USE module_domain  , ONLY : domain
L03304  #ifdef DM_PARALLEL
L03305     USE module_dm        , ONLY : ntasks_x,ntasks_y,local_communicator,mytask,ntasks,wrf_dm_minval_integer
L03306  # if (EM_CORE == 1)
L03307     USE module_comm_dm   , ONLY : halo_toposhad_sub
L03308  # endif
L03309  #endif
L03310     USE module_bc
L03311     USE module_model_constants
L03312
L03313     IMPLICIT NONE
L03314
L03315     INTEGER,      INTENT(IN   )    ::   ids,ide, jds,jde, kds,kde, &
L03316                                           ims,ime, jms,jme, kms,kme, &
L03317                                           ips,ipe, jps,jpe, kps,kpe, &
L03318                                                             kts,kte, &
L03319                                           num_tiles
L03320
L03321     TYPE(domain)                 , INTENT(INOUT)  :: grid
L03322     TYPE(grid_config_rec_type   ) ,   INTENT(IN   ) :: config_flags
L03323
L03324     INTEGER, INTENT(IN  ) :: itimestep, ra_call_offset, stepra,    &
L03325                              slope_rad, topo_shading,              &
L03326                              spec_bdy_width
L03327
L03328     INTEGER, INTENT(INOUT) :: min_ptchsz
L03329
L03330     INTEGER, DIMENSION(num_tiles), INTENT(IN) ::                   &
L03331                  i_start,i_end,j_start,j_end
L03332
L03333     REAL, INTENT(IN  )   :: GMT, radt, julian, xtime, dx, dy, shadlen
L03334
L03335     REAL, DIMENSION( ims:ime, jms:jme ),                           &
L03336           INTENT(IN   )  ::                                  XLAT, &
L03337                                                             XLONG, &
L03338                                                                HT, &
L03339                                                              SINA, &
L03340                                                              COSA
L03341     REAL, DIMENSION( ims:ime, jms:jme ),                           &
L03342           INTENT(IN   ), OPTIONAL ::                          DX2D, &
L03343                                                            AREA2D
L03344
L03345     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT)  ::  ht_shad,ht_loc
L03346
L03347     REAL,  DIMENSION( jms:jme , kds:kde , spec_bdy_width ),        &
L03348                      INTENT(IN   ) :: ht_shad_bxs, ht_shad_bxe
L03349     REAL,  DIMENSION( ims:ime , kds:kde , spec_bdy_width ),        &
L03350                      INTENT(IN   ) :: ht_shad_bys, ht_shad_bye
L03351
L03352     INTEGER, DIMENSION( ims:ime, jms:jme ),                        &
L03353           INTENT(INOUT)  ::                           shadowmask
L03354
L03355     LOGICAL,      INTENT(IN   )    :: nested
L03356
L03357  !Local
L03358  ! For orographic shading
L03359     INTEGER :: niter,ni,psx,psy,idum,jdum,i,j,ij
L03360     REAL :: DECLIN,SOLCON
L03361
L03362  ! Determine minimum patch size for slope-dependent radiation
L03363     if (itimestep .eq. 1) then
L03364       psx = ipe-ips+1
L03365       psy = jpe-jps+1
L03366       min_ptchsz = min(psx,psy)
L03367       idum = 0
L03368       jdum = 0
L03369     endif
L03370
L03371  # ifdef DM_PARALLEL
L03372     if (itimestep .eq. 1) then
L03373       call wrf_dm_minval_integer (psx,idum,jdum)
L03374       call wrf_dm_minval_integer (psy,idum,jdum)
L03375       min_ptchsz = min(psx,psy)
L03376     endif
L03377  # endif
```

```
L03378
L03379  ! Topographic shading
L03380
L03381     if ((topo_shading.eq.1).and.(itimestep .eq. 1 .or. &
L03382          mod(itimestep,STEPRA) .eq. 1 + ra_call_offset))  then
L03383
L03384  !---------------
L03385  ! Calculate constants for short wave radiation
L03386
L03387     CALL radconst(XTIME,DECLIN,SOLCON,JULIAN,DEGRAD,DPD)
L03388
L03389  ! Make a local copy of terrain height field
L03390       do j=jms,jme
L03391       do i=ims,ime
L03392         ht_loc(i,j) = ht(i,j)
L03393       enddo
L03394       enddo
L03395  ! Determine if iterations are necessary for shadows to propagate from one patch to another
L03396       if ((ids.eq.ips).and.(ide.eq.ipe).and.(jds.eq.jps).and.(jde.eq.jpe)) then
L03397         niter = 1
L03398       else
L03399         niter = int(shadlen/(dx*min_ptchsz)+3)
L03400       endif
L03401
L03402
L03403      IF( nested ) THEN
L03404
L03405        !$OMP PARALLEL DO   &
L03406        !$OMP PRIVATE ( ij )
L03407
L03408        DO ij = 1 , num_tiles
L03409
L03410             CALL spec_bdyfield(ht_shad,                          &
L03411                                ht_shad_bxs, ht_shad_bxe,         &
L03412                                ht_shad_bys, ht_shad_bye,         &
L03413                                'm', config_flags, spec_bdy_width, 2,&
L03414                                ids,ide, jds,jde, 1  ,1  ,  & ! domain dims
L03415                                ims,ime, jms,jme, 1  ,1  ,  & ! memory dims
L03416                                ips,ipe, jps,jpe, 1  ,1  ,  & ! patch  dims
L03417                                i_start(ij), i_end(ij),           &
L03418                                j_start(ij), j_end(ij),           &
L03419                                1    , 1              )
L03420        ENDDO
L03421      ENDIF
L03422
L03423       do ni = 1, niter
L03424
L03425     !$OMP PARALLEL DO   &
L03426     !$OMP PRIVATE ( ij,i,j )
L03427          do ij = 1 , num_tiles
L03428
L03429          call toposhad_init (ht_shad,ht_loc,                        &
L03430                        shadowmask,nested,ni,                        &
L03431                        ids,ide, jds,jde, kds,kde,                   &
L03432                        ims,ime, jms,jme, kms,kme,                   &
L03433                        ips,min(ipe,ide-1), jps,min(jpe,jde-1), kps,kpe,      &
L03434                        i_start(ij),min(i_end(ij), ide-1),j_start(ij),&
L03435                        min(j_end(ij), jde-1), kts, kte              )
L03436
L03437          enddo
L03438     !$OMP END PARALLEL DO
L03439
L03440
L03441     !$OMP PARALLEL DO   &
L03442     !$OMP PRIVATE ( ij,i,j )
L03443          do ij = 1 , num_tiles
L03444
L03445          call toposhad (xlat,xlong,sina,cosa,xtime,gmt,radt,declin,    &
L03446                        dx,dy,ht_shad,ht_loc,ni,                       &
L03447                        shadowmask,shadlen,                            &
L03448                        ids,ide, jds,jde, kds,kde,                     &
L03449                        ims,ime, jms,jme, kms,kme,                     &
L03450                        ips,min(ipe,ide-1), jps,min(jpe,jde-1), kps,kpe,        &
L03451                        i_start(ij),min(i_end(ij), ide-1),j_start(ij),&
L03452                        min(j_end(ij), jde-1), kts, kte              )
L03453
L03454          enddo
L03455     !$OMP END PARALLEL DO
L03456
L03457  #if defined( DM_PARALLEL ) && (EM_CORE == 1)
L03458  #     include "HALO_TOPOSHAD.inc"
L03459  #endif
L03460       enddo
L03461     endif
L03462
L03463     END SUBROUTINE pre_radiation_driver
```

讲解：topo_shading=1 时提前一子步算阴影场（由 first_rk_step_part1 调，主循环时阴影已就绪）：①radconst 算当日赤纬/太阳常数；②ht_loc 拷贝地形；③ **跨 patch 阴影传播**：niter=int(shadlen/(dx·min_ptchsz))+3——阴影长度超过本 patch 时需要多次迭代 +HALO_TOPOSHAD 交换（长阴影从邻居 patch 传入）；嵌套域用 spec_bdyfield 把阴影高度场写入边界缓冲；④ 每次迭代 toposhad_init（清 mask）+ toposhad（扫描）各一遍。toposhad（L4610–4862，头段见下）的几何：地方时/时角/coszen 同 calc_coszen，**太阳方位角 sol_azi 由 acos 反解并按网格旋转角 sina/cosa 转到 WRF 网格方向**，然后沿太阳来向逐格扫 gpshad=int(shadlen/dx+1) 格，插值地形仰角 atan(Δh/距离)，sin(仰角)≥coszen 即阴影：mask=1 且阴影高度 ht_shad 抬到 ht+距离 ×(tan 仰角 −tan 太阳仰角)——goto 120 的多向四分支（北/东/南/西）是手写太阳几何的老派实现。

十、短波插值器（swint_opt=1/2）

```
!file: phys/module_radiation_driver.F    L3543-L3624    (subroutine update_swinterp_parameters)
L03543     subroutine update_swinterp_parameters(ims,ime,jms,jme,its,ite,jts,jte, &
L03544                                           coszen,coszen_loc,swddir,swdown, &
L03545                                           swddir_ref,bb,Bx,                &
L03546                                           swdown_ref,gg,Gx,                &
L03547                                           coszen_ref                       )
L03548         ! Author: jararias 2013/11
L03549         implicit None
L03550         integer, intent(in) :: ims,ime,jms,jme,its,ite,jts,jte
L03551         real, dimension(ims:ime,jms:jme), intent(in)    :: coszen,coszen_loc,swddir,swdown
L03552         real, dimension(ims:ime,jms:jme), intent(inout) :: swddir_ref,bb,Bx, &
L03553                                                            swdown_ref,gg,Gx, &
L03554                                                            coszen_ref
L03555
L03556         integer :: i,j
L03557         real :: swddir_0,swdown_0,coszen_0
L03558         real, parameter :: coszen_min=1e-4
L03559
L03560         do j=jts,jte
L03561            do i=its,ite
L03562               if ((coszen(i,j).gt.coszen_min) .and. (coszen_loc(i,j).gt.coszen_min)) then
L03563                  ! parameters update for DIR
L03564                  if (Bx(i,j).le.0) then
L03565                     swddir_0 =(coszen_loc(i,j)/coszen(i,j))*swddir(i,j) ! linear first guess estimation
L03566                     coszen_0 =coszen_loc(i,j)
L03567                  else
L03568                     swddir_0 =swddir_ref(i,j)
L03569                     coszen_0 =coszen_ref(i,j)
L03570                  end if
L03571                  if ((coszen(i,j)/coszen_0).lt.1.) then
L03572                     bb(i,j) =log(max(1.,swddir(i,j))/max(1.,swddir_0)) / log(min(1.-1e-4,coszen(i,j)/coszen_0))
L03573                  elseif ((coszen(i,j)/coszen_0).gt.1) then
L03574                     bb(i,j) =log(max(1.,swddir(i,j))/max(1.,swddir_0)) / log(max(1.+1e-4,coszen(i,j)/coszen_0))
L03575                  else
L03576                     bb(i,j) =0.
L03577                  end if
L03578                  bb(i,j) =max(-.5,min(2.5,bb(i,j)))
L03579                  Bx(i,j) =swddir(i,j)/(coszen(i,j)**bb(i,j))
L03580
L03581                  !write(wrf_err_message,*) 'XXX I=',i,' J=',j,'  Bx=',Bx(i,j),'  bb=',bb(i,j),'  swddir=',swddir(i,j), &
L03582                  !                        '  swddir_0=',swddir_0,'  coszen=',coszen(i,j),'  coszen_0=',coszen_0
L03583                  !call wrf_debug(1,wrf_err_message)
L03584
L03585                  ! parameters update for GHI
L03586                  if (Gx(i,j).le.0) then
L03587                     swdown_0 =(coszen_loc(i,j)/coszen(i,j))*swdown(i,j) ! linear first guess estimation
L03588                     coszen_0 =coszen_loc(i,j)
L03589                  else
L03590                     swdown_0 =swdown_ref(i,j)
L03591                     coszen_0 =coszen_ref(i,j)
L03592                  end if
L03593                  if ((coszen(i,j)/coszen_0).lt.1.) then
L03594                     gg(i,j) =log(max(1.,swdown(i,j))/max(1.,swdown_0)) / log(min(1.-1e-4,coszen(i,j)/coszen_0))
L03595                  elseif ((coszen(i,j)/coszen_0).gt.1) then
L03596                     gg(i,j) =log(max(1.,swdown(i,j))/max(1.,swdown_0)) / log(max(1.+1e-4,coszen(i,j)/coszen_0))
L03597                  else
L03598                     gg(i,j) =0.
L03599                  end if
L03600                  gg(i,j) =max(-.5,min(2.5,gg(i,j)))
L03601                  Gx(i,j) =swdown(i,j)/(coszen(i,j)**gg(i,j))
L03602               else
L03603                  Bx(i,j) =0.
L03604                  bb(i,j) =0.
L03605                  Gx(i,j) =0.
L03606                  gg(i,j) =0.
L03607               end if
L03608
L03609               ! saving last SW run in state variables
L03610               coszen_ref(i,j) =coszen(i,j)
L03611               swdown_ref(i,j) =swdown(i,j)
L03612               swddir_ref(i,j) =swddir(i,j)
L03613
L03614               !if ((i.eq.20).and.(j.eq.20)) then
L03615               !   write(wrf_err_message,'("   RADSTEP : tn=",I4," csz_0=",F9.6," csz=",F9.6," csz_1=",F9.6," Gx=",F14.2,"
↪  gg=",F9.5,  &
L03616               !                          " Bx=",F14.2," bb=",F9.5)') itimestep,coszen_0,coszen_loc(i,j),coszen(i,j),
↪  Gx(i,j),gg(i,j), &
L03617               !                          Bx(i,j),bb(i,j)
L03618               !   call wrf_debug(1,wrf_err_message)
L03619               !end if
L03620
L03621            end do
L03622         end do
L03623
L03624     end subroutine update_swinterp_parameters
```

```
!file: phys/module_radiation_driver.F    L3626-L3670    (subroutine interp_sw_radiation)
L03626     subroutine interp_sw_radiation(ims,ime,jms,jme,its,ite,jts,jte,  &
L03627                                    coszen_ref,coszen_loc,swddir_ref, &
L03628                                    bb,Bx,swdown_ref,gg,Gx,albedo,    &
L03629                                    swdown,swddir,swddni,swddif,gsw   )
L03630         ! Author: jararias 2013/11
L03631         implicit None
L03632         integer, intent(in) :: ims,ime,jms,jme,its,ite,jts,jte
L03633         real, dimension(ims:ime,jms:jme), intent(in) :: coszen_ref,coszen_loc, &
L03634                                                         swddir_ref,Bx,bb,      &
L03635                                                         swdown_ref,Gx,gg,      &
L03636                                                         albedo
L03637
L03638         real, dimension(ims:ime,jms:jme), intent(inout) :: swddir,swdown, &
L03639                                                            swddif,swddni, gsw
L03640
```

```
L03641        integer :: i,j
L03642        real, parameter :: coszen_min=1e-4
L03643
L03644        do j=jts,jte
L03645           do i=its,ite
L03646              ! sza interpolation of surface fluxes
L03647              if ((coszen_ref(i,j).gt.coszen_min) .and. (coszen_loc(i,j).gt.coszen_min)) then
L03648                 if ((bb(i,j).eq.-0.5).or.(bb(i,j).eq.2.5).or.(bb(i,j).eq.0.0)) then
L03649                    swddir(i,j) =(coszen_loc(i,j)/coszen_ref(i,j))*swddir_ref(i,j)
L03650                 else
L03651                    swddir(i,j) =Bx(i,j)*(coszen_loc(i,j)**bb(i,j))
L03652                 end if
L03653                 if ((gg(i,j).eq.-0.5).or.(gg(i,j).eq.2.5).or.(gg(i,j).eq.0.0)) then
L03654                    swdown(i,j) =(coszen_loc(i,j)/coszen_ref(i,j))*swdown_ref(i,j)
L03655                 else
L03656                    swdown(i,j) =Gx(i,j)*(coszen_loc(i,j)**gg(i,j))
L03657                 end if
L03658                 swddif(i,j) =swdown(i,j)-swddir(i,j)
L03659                 swddni(i,j) =swddir(i,j)/coszen_loc(i,j)
L03660                 gsw(i,j)    =swdown(i,j)*(1.-albedo(i,j))
L03661              else
L03662                 swddir(i,j) =0.
L03663                 swdown(i,j) =0.
L03664                 swddif(i,j) =0.
L03665                 swddni(i,j) =0.
L03666                 gsw(i,j)    =0.
L03667              end if
L03668           end do
L03669        end do
L03670     end subroutine interp_sw_radiation
```

讲解：两次辐射调用之间的**太阳几何外插**（jararias 2013/11，太阳能诊断流）：①update 件以本次（swint 模式的 coszen_loc，非 radt 半步）与上次辐射调用为两锚点，拟合幂律 swddir=Bx • coszen^bb（bb 对数比钳位 [−0.5, 2.5]、边界值线性化），GHI 同法 gg/Gx，状态量存 ref；②interp 件按当前 coszen_loc 外插 swddir/swdown，导出散分量 swddif=swdown−swddir、法向直射 swddni=swddir/coszen，**gsw=swdown · (1−albedo)**；夜间（coszen≤1e-4）全零。这是「辐射调得稀、太阳能输出要连续」的补丁管线。

十一、单元总表（21 伴生单元）

单元	行段	功能	收录
pre_radiation_driver	L3285–L3463	地形阴影调度（topo_shading）	全文
radconst	L3469–L3511	太阳赤纬/太阳常数	全文
calc_coszen	L3514–L3541	天顶角（含均时差）	全文
update_swinterp_parameters / interp_sw_radiation	L3543–L3670	swint 幂律插值两件	全文
cal_cldfra1	L3761–L3986	Randall/Xu-Randall 云量	全文
cal_cldfra2	L3676–L3749	阈值二值云量	全文
cal_cldfra3	L4003–L4150	Thompson 尺度感知云量	核心段 L4003–L4110
find_cloudLayers/adjust_cloud*	L4157–L4469	云层识别与水物质量调整（GFDL 伴生）	注记不录
toposhad_init/toposhad	L4474–L4862	阴影扫描两件	toposhad 头段 L4610–L4700
ozn_time_int/ozn_p_int	L4864–L5105	CAM 臭氧月插值/气压插值	回指 code_23 同源件
aer_time_int/aer_p_int	L5107–L5377	Tegen 气溶胶插值（多 AODTOT）	回指 code_23 同源件
gt_aod	L5382–L5619	QNWFA/QNIFA 逐层 AOD	注记不录（aer_opt=3 通道）
Add/Remove_multi_perturb	L5623–L5708	SPP 扰动对	全文

十二、改造入口

1. **云量方案更换**：icloud=1/2/3 三案——加第四案（如云 PDF 型）在 § 四的 IF 链上动刀。
2. **气溶胶辐射效应**：aer_opt=1（Tegen 场）/2（2D AOD 输入）/3（QNWFA 预报 AOD）三通道；各方案的波段数组分配（11/14）见 § 五。
3. **SPP 模板**：Add/Remove 对——给任意辐射输入量加随机场扰动的现成包裹（乘除对称 +钳位常量）。
4. **调用序依赖**：LW 覆盖/SW 叠加 RTHRATEN 的隐式约定（§ 七 ③）——重排分发顺序前必查。
5. **太阳能应用**：swint 插值 + FARMs 通道 + swddir/dni/dif 输出——模式外的光伏后处理管线。

十三、上下游链接

- 下游方案本体：code_05_module_ra_rrtmg_lw.md、code_06_module_ra_rrtmg_sw.md；气溶胶光学：code_60_module_ra_aerosol.md
- 同级驱动器：code_22_module_pbl_driver.md、code_23_module_cumulus_driver.md（四测试门控同构、ozn/aer 插值同源）
- 教材正文：ch12_radiation_family.md、ch21_solve_em_dynamics.md

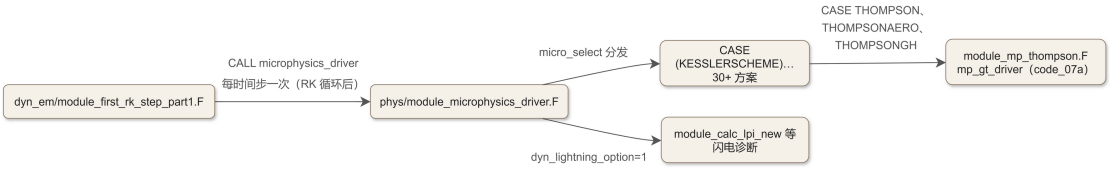
文库 code_21：module_microphysics_driver.F——微物理方案的总装配面

21.0 版本指纹与阅读说明

- **源文件**：`phys/module_microphysics_driver.F`（WRF 4.8.0），全文 3197 行。
- **收录级别**：B 级节选——本篇抽取十段（约 900 行源码），覆盖“模块头 → USE 普查 → 实参契约 → 前置处理 → 分发骨架 → Thompson 三 CASE → 收尾 → 扰动子程序”的完整主线；其余约 2300 行是**另外 30 余个方案的 CASE 调用段**，结构与 Thompson 三 CASE 完全同构，不逐一收录。
- **保真承诺**：同 code_07a——代码块由 `code_extract.py` 按字节抽取、附 md5 指纹、`verify` 可复核。
- **教材锚点**：ch13 §13.6（方案分发与调用链）、ch13 §13.6.4（Registry 方案号与包）、ch04（Registry 机制）、ch21（solve_em 调用位）。教材讲“分发为什么长这样”，本篇讲“每一行在做什么、加一个新方案要动哪里”。

21.1 一句话定位与调用链

一句话：`microphysics_driver` 是“一个 `mp_physics` 方案号 ↔ 三十多种微物理方案”的总装配面——它不做任何微物理计算，只做三件事：**核对实参在不在（PRESENT 检查墙）、按方案号分发（`micro_select`）、把 Registry 字段按各方案的形参名重新装配**。



- 上游调用位：`solve_em.F` 的 `moist_physics_prep_em → first_rk_step_part1 → moist_physics_finish` 链（教材 ch21 §21.4 有逐站走读；`dt` 如何传到这里见该节）。
- 下游：code_07a/07b（Thompson 本体）、以及其它 30+ 方案模块（不在文库收录范围）。
- **实参规模**：签名约 400 个实参（L9–L175）——这是全 WRF 物理层最大的单一实参表，原因是它要同时服务“单矩到五矩、无气溶胶到 CAM-MAM 模态气溶胶”的所有方案。

21.2 模块头

```
!file: phys/module_microphysics_driver.F    L1-L7
L00001  !WRF:MEDIATION_LAYER:PHYSICS
L00002  ! *** add new modules of schemes here
L00003  !
L00004  MODULE module_microphysics_driver
L00005    real, private, parameter :: QX_MIN = 1.E-12
L00006    real, private, parameter :: NI_MIN = 1.E-6
L00007  CONTAINS
```

- `!WRF:MEDIATION_LAYER:PHYSICS`（L1）是 WRF 的**文件角色标签**——mediation layer（中介层）即“只装配不计算”的层。全模块除两个扰动子程序外只有一个子程序。
- 模块级只有两个私有常数：`QX_MIN=1e-12`、`NI_MIN=1e-6`——专供扰动子程序钳位（§21.10），命名直白：质量最小值、数浓度最小值。
- `CONTAINS` 之后直接进主子程序——没有模块级数据。**driver 是无状态的**：所有状态都在 `*_curr` 字段里（属于 Registry/网格），这使它天然线程安全（OpenMP 并行的前提）。

21.3 USE 普查：44 项依赖的分组解读

```
!file: phys/module_microphysics_driver.F    L177-L275
L00177  ! Framework
L00178     USE module_state_description, ONLY :                                  &
L00179                         KESSLERSCHEME, LINSCHEME, SBU_YLINSCHEME, WSM3SCHEME, WSM5SCHEME    &
L00180                        ,WSM6SCHEME, ETAMPNEW, FER_MP_HIRES, THOMPSON, THOMPSONAERO, RCON_MP_SCHEME, THOMPSONGH,
↪   FAST_KHAIN_LYNN_SHPUND, MORR_TWO_MOMENT     &
L00181                        ,GSFCGCESCHEME, WDM5SCHEME, WDM6SCHEME, NSSL_2MOM, MADWRF_MP  &
L00182                        ,FER_MP_HIRES_ADVECT &
L00183                        ,WSM7SCHEME, WDM7SCHEME, UDMSCHEME &
L00184                        ,NUWRF4ICESCHEME &
L00185                        ,MILBRANDT2MOM , CAMMGMPSCHEME,FULL_KHAIN_LYNN, P3_1CATEGORY, P3_1CATEGORY_NC, P3_2CATEGORY,
↪   P3_1CAT_3MOM &
L00186                        ,MORR_TM_AERO, JENSEN_ISHMAEL, SPRINKLER, NTU, TEMPO !,MILBRANDT3MOM
L00187     USE module_state_description, ONLY : WSM6RSCHEME
L00188  #if ( WRFPLUS == 1 )
L00189     USE module_state_description, ONLY : LSCONDSCHEME, MKESSLERSCHEME
L00190  #endif
L00191  #if ( WRF_CHEM == 1 )
L00192  ! NUWRF
L00193     USE module_state_description, ONLY :                                  &
L00194                         num_chem                                          & !inline gocart
L00195                        ,p_bc1, p_bc2, p_oc1, p_oc2                        & !inline gocart
L00196                        ,p_dust_1, p_dust_2, p_dust_3                      & !inline gocart
L00197                        ,p_dust_4, p_dust_5                                & !inline gocart
L00198                        ,p_sulf, p_seas_1, p_seas_2                        & !inline gocart
L00199                        ,p_seas_3, p_seas_4
L00200
L00201  #endif
L00202
L00203  #ifdef DM_PARALLEL
L00204    USE module_dm, ONLY : &
L00205                    local_communicator, mytask,  wrf_dm_min_real, wrf_dm_max_real
L00206  #endif
L00207
L00208  ! Model Layer
L00209     USE module_model_constants
L00210     USE module_wrf_error
L00211     USE module_configure, only: grid_config_rec_type
L00212  #if ( WRF_CHEM == 1 )
L00213  !mchen   USE module_state_description, only: num_scalar              ! For CAMMGMP scheme Prognostic aerosols
L00214     USE module_state_description, only: num_chem              ! mchen
L00215     USE modal_aero_data, only:  ntot_amode_cam_mam => ntot_amode ! For CAMMGMP scheme Prognostic aerosols
L00216  #endif
L00217
L00218  ! *** add new modules of schemes here
L00219
L00220     USE module_mp_kessler
L00221  #if ( WRFPLUS == 1 )
L00222     USE module_mp_mkessler
L00223     USE module_mp_nconvp    !  added by Zhuxiao
L00224  #endif
L00225     USE module_mp_lin
L00226     USE module_mp_sbu_ylin
L00227     USE module_mp_wsm3
L00228     USE module_mp_wsm5
L00229     USE module_mp_wsm6
L00230     USE module_mp_wsm6r, ONLY:wsm6r
L00231     USE module_mp_wsm7
L00232     USE module_mp_etanew
L00233     USE module_mp_fer_hires
L00234     USE module_mp_rcon
L00235     USE module_mp_thompson
L00236     USE module_mp_tempo_cfgs, only : ty_tempo_cfgs
L00237     USE module_mp_tempo_driver, only : tempo_run, ty_tempo_driver_diags, &
L00238        tempo_aerosol_surface_emissions
L00239     USE module_mp_full_sbm
L00240  #if ( BUILD_SBM_FAST == 1 )
L00241     USE module_mp_fast_sbm
L00242  #endif
L00243     USE module_mp_gsfcgce
L00244     USE module_mp_gsfcgce_4ice_nuwrf, only: gsfcgce_4ice_nuwrf
L00245     USE module_mp_morr_two_moment
L00246     USE microphy_p3
L00247     USE module_mp_jensen_ishmael
L00248  # if (EM_CORE == 1)
L00249     USE module_mp_morr_two_moment_aero
L00250     USE module_mp_ntu
L00251  # endif
L00252     USE module_mp_wdm5
L00253     USE module_mp_wdm6
L00254     USE module_mp_wdm7
L00255     USE module_mp_udm
L00256     USE module_mp_milbrandt2mom
L00257  # if (EM_CORE == 1)
L00258     USE module_mp_cammgmp_driver, ONLY: CAMMGMP ! CAM5's microphysics driver
L00259  # endif
L00260  !  USE module_mp_milbrandt3mom
L00261  #if (WRFPLUS /= 1) & !defined( VAR4D )
L00262     USE module_mp_nssl_2mom
L00263  #endif
L00264     USE module_mixactivate, only: prescribe_aerosol_mixactivate
L00265
L00266  ! For checking model timestep is history time (for radar reflectivity)
L00267     USE module_utility, ONLY: WRFU_Clock, WRFU_Alarm
L00268     USE module_domain, ONLY : HISTORY_ALARM, Is_alarm_tstep
L00269     USE module_irrigation
L00270
L00271     USE module_fire_emis
L00272     USE module_calc_lpi_new
L00273     USE module_ltng_pe
L00274     USE module_ltng_strokes
L00275
```

44 项 USE 按功能分五组（方案普查底账在文库方案 § 三）：

1. **方案号字典**（L178–L199）：`module_state_description` 提供全部方案号常量——`KESSLERSCHEME=1` … `THOMPSON=8` … `THOMPSONAERO=28` … `THOMPSONGH`（完整号表见 `frame/module_state_description.F` 与教材 ch13 §13.6.4）。`SELECT CASE(mp_physics)` 里的每个 `CASE` 标签都来自这里。`WSM6RSCHEME`（L187）单独一条 USE 是历史遗留的并入库方式；WRFPLUS/CHEM 条件下的 LSCOND/化学物种索引按编译开关增减。
2. **并行与模型层**（L203–L216）：`module_dm`（`wrf_dm_min/max_real`——调试块与气溶胶归约用）、`module_model_constants`（`ep_2/svp1…xlv/xls/xlf/cp/g` 等常数——注意**常数经 driver 传给方案**而非方案自取，Kessler 的 `XLV=xlv, CP=cp` 实参就是这种 “常数也走实参” 的老派风格）、`module_configure`（`grid_config_rec_type`——namelist 配置结构体，`config_flags%aer_init_opt` 这类字段由此读）。
3. **方案实体模块**（L220–L263，25 个 USE）：每个被分发的方案各一条——“*** add new modules of schemes here” 注释出现两次（模块头 L2 与此处 L218），**加新方案的第一动刀点**。注意三个带条件的：`BUILD_SBM_FAST`（快速分档方案）、`EM_CORE==1`（Morrison 气溶胶版/NTU/CAMMGMP）、`WRFPLUS!=1`（NSSL）。`USE module_mp_wsm6` 是**整模块引用**（wsm6 子程序名不限定），而 `module_mp_wsm6r` 用 `ONLY:wsm6r` 限定——同一文件里两种风格并存。
4. **横向支撑**（L264–L275）：`module_mixactivate`（LIN/Morrison 方案的气溶胶活化前置）、`module_utility+module_domain` 的 `HISTORY_ALARM/Is_alarm_tstep`（**反射率只在历史输出时刻算**——`diagflag` 的时钟依据）、`module_irrigation`（喷灌方案）、`module_fire_emis`（生物质燃烧气溶胶注入）、闪电三件（`calc_lpi_new/ltng_pe/ltng_strokes`）。
5. **头注释**（L276–L296）列了方案清单——但它停在 WDM6 时代（2011 年口径），**注释维护落后于代码**（P3/NSSL/TEMPO 都没进清单）。读 WRF 要有 “注释是快照、代码是事实” 的觉悟。

21.4 网格结构与变量定义注释

```
!file: phys/module_microphysics_driver.F    L298-L351
L00298      IMPLICIT NONE
L00299   !=================================================================
L00300   ! Grid structure in physics part of WRF
L00301   !-----------------------------------------------------------------
L00302   ! The horizontal velocities used in the physics are unstaggered
L00303   ! relative to temperature/moisture variables. All predicted
L00304   ! variables are carried at half levels except w, which is at full
L00305   ! levels. Some arrays with names (*8w) are at w (full) levels.
L00306   !
L00307   !-----------------------------------------------------------------
L00308   ! In WRF, kms (smallest number) is the bottom level and kme (largest
L00309   ! number) is the top level.  In your scheme, if 1 is at the top level,
L00310   ! then you have to reverse the order in the k direction.
L00311   !
L00312   !         kme      -   half level (no data at this level)
L00313   !         kme    ----- full level
L00314   !         kme-1    -   half level
L00315   !         kme-1  ----- full level
L00316   !         .
L00317   !         .
L00318   !         .
L00319   !         kms+2    -   half level
L00320   !         kms+2  ----- full level
L00321   !         kms+1    -   half level
L00322   !         kms+1  ----- full level
L00323   !         kms      -   half level
L00324   !         kms    ----- full level
L00325   !
L00326   !=================================================================
L00327   ! Definitions
L00328   !-----------
L00329   ! Rho_d      dry density (kg/m^3)
L00330   ! Theta_m    moist potential temperature (K)
L00331   ! Qv         water vapor    mixing ratio (kg/kg)
L00332   ! Qc         cloud water    mixing ratio (kg/kg)
L00333   ! Qr         rain water     mixing ratio (kg/kg)
L00334   ! Qi         cloud ice      mixing ratio (kg/kg)
L00335   ! Qs         snow           mixing ratio (kg/kg)
L00336   ! Qg         graupel        mixing ratio (kg/kg)
L00337   ! Qh         hail           mixing ratio (kg/kg)
L00338   ! Qndrop     droplet number mixing ratio (#/kg)
L00339   ! Qni        cloud ice number concentration (#/kg)
L00340   ! Qns        snow      number concentration (#/kg)
L00341   ! Qnr        rain      number concentration (#/kg)
L00342   ! Qng        graupel   number concentration (#/kg)
L00343   ! Qnh        hail      number concentration (#/kg)
L00344
L00345   ! Qzr        rain             reflectivity (m6/kg)
L00346   ! Qzi        ice              reflectivity (m6/kg)
L00347   ! Qzs        snow             reflectivity (m6/kg)
L00348   ! Qzg        graupel          reflectivity (m6/kg)
L00349   ! Qzh        hail             reflectivity (m6/kg)
L00350
L00351   ! Qvolg        graupel   particle volume (m3/kg)
```

- **k 方向的层约定**（L300–L326 原注释）：物理层所有预报量在半层（mass 点）、w 在整层（w 点）、`*8w` 数组在整层；`kms` 是**底层**——方案若内部用 “1 在顶” 的老惯例，要自己倒 k。**这段注释是所有物理方案作者必读的网格契约**（教材 ch10 讲垂直坐标时的工程对应面）。

- 变量定义段（L329 起）是实参契约的 “字典前言”：Qv/Qc/Qr/Qi/Qs/Qg/Qh 七个混合比 + Qndrop/Qni/Qns/Qnr/Qng/Qnh 六个数浓度 + 五个反射量 Qz*——**这就是 Thompson 的 mp_gt_driver 实参在 driver 层的 “全集”**：Thompson 只取其中子集，全集里还有雪数浓度 qns、雹 qh 等它不需要的项。
- P_QV 系注释（L391 起）：P_* 是 Registry 生成的**物种下标**（P_QV=param_first_scalar 起），用于在 moist/scalar 数组里索引物种——本文件签名用 “独立数组” 风格（qv_curr 等），P_* 索引风格在 first_rk_step_part1 装配层（文库 code_31）。

21.5 实参契约：约 400 个实参的分组地图

```
!file: phys/module_microphysics_driver.F    L9-L175
L00009  SUBROUTINE microphysics_driver(                                    &
L00010                           th, rho, pi_phy, p                        &
L00011                          ,ht, dz8w, p8w, dt,dx,dy                   &
L00012                          ,mp_physics, spec_zone                     &
L00013                          ,specified, channel_switch                 &
L00014                          ,warm_rain                                 &
L00015                          ,t8w                                       &
L00016                          ,chem_opt, progn                           &
L00017                          ,cldfra, cldfra_old, exch_h, nsource       &
L00018                          ,qlsink, precr, preci, precs, precg        &
L00019                          ,xland,snowh,xice,itimestep                &
L00020                          ,f_ice_phy,f_rain_phy,f_rimef_phy          &
L00021                          ,lowlyr,sr, id                             &
L00022                          ,ids,ide, jds,jde, kds,kde                 &
L00023                          ,ims,ime, jms,jme, kms,kme                 &
L00024                          ,ips,ipe, jps,jpe, kps,kpe                 &
L00025                          ,i_start,i_end,j_start,j_end,kts,kte       &
L00026                          ,num_tiles, naer                           &
L00027                          ,irrigation,sf_surf_irr_scheme, irr_daily_amount   &
L00028                          ,irr_start_hour,irr_num_hours,julian_in    &
L00029                          ,irr_start_julianday,irr_end_julianday     &
L00030                          ,irr_freq,irr_ph,irr_rand_field            &
L00031                          ,gmt,xtime                                 &
L00032  !=====================
L00033                          !Variables required for CAMMGMP Scheme
L00034                          ,dlf,dlf2,t_phy,p_hyd,p8w_hyd,tke_pbl,pblh         &
L00035                          ,z_at_mass,z_at_w,qfx                      &
L00036                          ,rliq,turbtype3d,smaw3d,wsedl3d,cldfra_old_mp      &
L00037                          ,cldfra_mp,cldfra_mp_all,lradius,iradius   &
L00038                          ,cldfrai,cldfral,cldfra_conv               &
L00039                          ,alt                                       &
L00040                          ,accum_mode,aitken_mode,coarse_mode        &
L00041                          ,icwmrsh3d,icwmrdp3d,shfrc3d,cmfmc3d,cmfmc2_3d     &
L00042                          ,config_flags,fnm,fnp,rh_old_mp,lcd_old_mp         &
L00043  #if ( WRF_CHEM == 1 )
L00044                          ,chem                                      &! For CAMMGMP scheme Prognostic aerosols
L00045                          ,qme3d,prain3d,nevapr3d,rate1ord_cw2pr_st3d        &
L00046                          ,dgnum4D,dgnumwet4D                        &
L00047  #endif
L00048  !=====================
L00049                          ,qv_curr,qc_curr,qr_curr,qi_curr,qs_curr,qg_curr   &
L00050                          ,qic_curr,qip_curr,qid_curr &
L00051                          ,qnic_curr,qnip_curr,qnid_curr &
L00052                          ,qndrop_curr,qni_curr,qh_curr,qnh_curr             &
L00053                          ,qzr_curr,qzi_curr,qzs_curr,qzg_curr,qzh_curr      &
L00054                          ,qns_curr,qnr_curr,qng_curr,qnn_curr,qnc_curr      &
L00055                          ,qnwfa_curr,qnifa_curr,qnbca_curr                  & ! for water/ice-friendly/black carbon
↪  aerosols
L00056                          ,f_qnwfa,f_qnifa,f_qnbca                           & ! for water/ice-friendly/black carbon
↪  aerosols
L00057                          ,qvolg_curr,qvolh_curr                             &
L00058                          ,qdcn_curr,qtcn_curr,qccn_curr,qrcn_curr           & ! for ntu3m
L00059                          ,qnin_curr,fi_curr,fs_curr,vi_curr,vs_curr         & ! for ntu3m
L00060                          ,vg_curr,ai_curr,as_curr,ag_curr,ah_curr,i3m_curr  & ! for ntu3m
L00061                          ,f_qdcn,f_qtcn,f_qccn,f_qrcn,f_qnin,f_fi,f_fs      & ! for ntu3m
L00062                          ,f_vi,f_vs,f_vg,f_ai,f_as,f_ag,f_ah,f_i3m          & ! for ntu3m
L00063                          ,qir_curr,qib_curr                                 & ! for P3
L00064                          ,qi2_curr,qni2_curr,qir2_curr,qib2_curr            & ! for P3
L00065                          ,qvoli_curr,qaoli_curr                             & ! for Jensen ISHMAEL
L00066                          ,qvoli2_curr,qaoli2_curr                           & ! for Jensen ISHMAEL
L00067                          ,qi3_curr,qni3_curr,qvoli3_curr,qaoli3_curr        & ! for Jensen ISHMAEL
L00068                          ,effr_curr,ice_effr_curr,tot_effr_curr             &
L00069                          ,qic_effr_curr,qip_effr_curr,qid_effr_curr         &
L00070                          ,f_qv,f_qc,f_qr,f_qi,f_qs,f_qg,f_qndrop,f_qni      &
L00071                          ,f_qns,f_qnr,f_qng,f_qnc,f_qnn,f_qh,f_qnh          &
L00072                          ,           f_qzr,f_qzi,f_qzs,f_qzg,f_qzh          &
L00073                          ,f_qvolg,f_qvolh                                   &
L00074                          ,f_qic,f_qip,f_qid                                 &
L00075                          ,f_qnic,f_qnip,f_qnid                              &
L00076                          ,f_qir,f_qib                                       & ! for P3
L00077                          ,f_qi2,f_qni2,f_qir2,f_qib2                        & ! for P3
L00078                          ,f_qvoli,f_qaoli                                   & ! for Jensen ISHMAEL
L00079                          ,f_qvoli2,f_qaoli2                                 & ! for Jensen ISHMAEL
L00080                          ,f_qi3,f_qni3,f_qvoli3,f_qaoli3                    & ! for Jensen ISHMAEL
L00081                          ,f_effr,f_ice_effr,f_tot_effr                      &
L00082                          ,f_qic_effr,f_qip_effr,f_qid_effr                  &
L00083                          ,cu_used                                           &
L00084                          ,qrcuten, qscuten, qicuten, qccuten                &
L00085                          ,qt_curr,f_qt                                      &
L00086                          ,mp_restart_state,tbpvs_state,tbpvs0_state         & ! for etampnew or fer_mp_hires
L00087                          ,hail,ice2                                         & ! for mp_gsfcgce
L00088  !NUWRF JJS 20110525 vvvvv
L00089                          ,phys_tot, physc, physe, physd, physs, physm, physf& ! for gsfcgce
L00090                          ,acphys_tot, acphysc, acphyse, acphysd              & ! for gsfcgce
L00091                          ,acphyss, acphysm, acphysf                         & ! for gsfcgce
L00092                          ,re_cloud_gsfc, re_rain_gsfc, re_ice_gsfc          &
L00093                          ,re_snow_gsfc, re_graupel_gsfc, re_hail_gsfc       & ! cloud effective radius
L00094                          ,precr3d, preci3d, precs3d, precg3d, prech3d       &
```

```
L00095  #if ( WRF_CHEM == 1 )
L00096                        ,icn_diag, nc_diag                              &  ! inline gocart
L00097                        ,gsfcgce_gocart_coupling                           &
L00098  #endif
L00099  !NUWRF JJS 20110525 ^^^^^
L00100  !                     ,ccntype                                           & ! for mp_milbrandt2mom
L00101                        ,u,v,w,z                                          &
L00102                        ,rainnc,    rainncv                                &
L00103                        ,snownc,    snowncv                                &
L00104                        ,hailnc,    hailncv                                &
L00105                        ,graupelnc, graupelncv                             &
L00106                        ,hail_maxk1, hail_max2d                            &
L00107  #if ( WRF_CHEM == 1 )
L00108                        ,rainprod, evapprod                                &
L00109                        ,qv_b4mp, qc_b4mp, qi_b4mp, qs_b4mp                &
L00110  #endif
L00111                        ,qnwfa2d, qnifa2d, qnbca2d                         & ! for water/ice-friendly/black carbon
↪  aerosols
L00112                        ,qnocbb2d, qnbcbb2d                                & ! for biomass burning aerosols
L00113                        ,ssat,ssati                                        &
L00114                        ,refl_10cm                                         & ! HM, 9/22/09, add for refl
L00115                        ,vmi3d                                             & ! for P3
L00116                        ,di3d                                              & ! for P3
L00117                        ,rhopo3d                                           & ! for P3
L00118                        ,phii3d                                            & ! for Jensen ISHMAEL
L00119                        ,vmi3d_2                                           & ! for P3
L00120                        ,di3d_2                                            & ! for P3
L00121                        ,rhopo3d_2                                         & ! for P3
L00122                        ,phii3d_2                                          & ! for Jensen ISHMAEL
L00123                        ,vmi3d_3                                           & ! for Jensen ISHMAEL
L00124                        ,di3d_3                                            & ! for Jensen ISHMAEL
L00125                        ,rhopo3d_3                                         & ! for Jensen ISHMAEL
L00126                        ,phii3d_3                                          & ! for Jensen ISHMAEL
L00127                        ,itype                                             & ! for Jensen ISHMAEL
L00128                        ,itype_2                                           & ! for Jensen ISHMAEL
L00129                        ,itype_3                                           & ! for Jensen ISHMAEL
L00130
L00131  ! YLIN
L00132  ! Added the RI_CURR array to the call
L00133                        ,ri_curr                                           &
L00134                        ,diagflag,   do_radar_ref                          &
L00135                        ,ke_diag                                           &
L00136                        ,re_cloud, re_ice, re_snow                         & ! G. Thompson
L00137                        ,has_reqc, has_reqi, has_reqs                      & ! G. Thompson
L00138                        ,ccn_conc                                          & ! RAS
L00139                        ,scalar,num_scalar                                   &
L00140                        ,kext_ql,kext_qs,kext_qg            &
L00141                        ,kext_qh,kext_qa                          &
L00142                        ,kext_qic,kext_qid,kext_qip         &
L00143                        ,kext_ft_qic,kext_ft_qid,kext_ft_qip         &
L00144                        ,kext_ft_qs,kext_ft_qg            &
L00145                        ,height,tempc &
L00146                        ,TH_OLD                                            &
L00147                        ,QV_OLD                                            &
L00148                        ,xlat,xlong,ivgtyp                                 &
L00149                        ,qrimef_curr,f_qrimef                              &
L00150                        ,aercu_opt                                         &
L00151  # if( EM_CORE==1 )
L00152                        ,sbmradar,num_sbmradar                             &
L00153                        ,sbm_diagnostics                                   &
L00154                        ,aerocu,aercu_fct,no_src_types_cu                  &
L00155                        ,PBL,EFCG,EFIG,EFSG,WACT,CCN1_GS,CCN2_GS           &
L00156                        ,CCN3_GS,CCN4_GS,CCN5_GS,CCN6_GS,CCN7_GS           &
L00157                        ,NR_CU,QR_CU,NS_CU,QS_CU,CU_UAF,mskf_refl_10cm     &
L00158  # endif
L00159                        ,multi_perturb                                     &
L00160                        ,pert_thom, perts_qvapor, perts_qcloud, perts_qice &
L00161                        ,perts_qsnow, perts_ni                             &
L00162                        ,pert_thom_qv,pert_thom_qc,pert_thom_qi            &
L00163                        ,pert_thom_qs,pert_thom_ni                         &
L00164                        ,cloudnc                                           &
L00165  !Dynamic Lightning
L00166                        ,light_pe_curr                                     &
L00167                        ,light_ne_curr                                     &
L00168                        ,light_neu_curr                                    &
L00169                        ,lpi2d                                             &
L00170                        ,lpi3d                                             &
L00171                        ,f_light_pe                                        &
L00172                        ,f_light_ne                                        &
L00173                        ,f_light_neu                                       &
L00174                        ,lpos,lneg,lneu                                    &
L00175                                                                           )
```

签名太长不适合逐行讲，给**分组地图**（行段位置按指纹注释中的 ranges 对照）：

组	代表实参	说明
动力环境	`th, rho, pi_phy, p, ht, dz8w, p8w, dt, dx, dy, t8w, u, v, w, z`	全方案共用的环境场；`pi_phy` 是非 NCAR 命名的 Exner 函数（`pii` 的 driver 层叫法）
配置	`mp_physics, spec_zone, specified, warm_rain, config_flags, chem_opt, progn, hail, ice2`	方案号与运行开关；`config_flags` 透传 namelist
水凝物字段	`qv_curr … qg_curr, qnc/qni/qnr/qns/qng/qnn/qnh_curr, qz*_curr, qvol*_curr, qrimef_curr`	`*_curr` = 当前时步值（INOUT）；命名带方案注释（`! for P3`、`! for ntu3m`）——**字段的方案归属写在行尾**

组	代表实参	说明
在场标志	f_qv … f_qnwfa, f_qb…	与 OPTIONAL 实参配对的 LOGICAL——**双保险**：OPTIONAL 让编译器处理在场性，f_* 让方案内部也能判断（经实参透传）
降水诊断	rainnc, rainncv, snownc/v, graupelnc/v, hailnc/v, sr, cloudnc, hail_max*	网格尺度降水账本
垂直廓线诊断	qlsink, precr, preci, precs, precg, nsource	三维廓线（PBW 老接口，多数方案不填）
有效半径	re_cloud, re_ice, re_snow, has_reqc/i/s	辐射接口（Thompson 填写，见 code_07a §07a.7.4）
雷达/饱和诊断	refl_10cm, ssat, ssati, do_radar_ref, diagflag, ke_diag	diagflag=本步是否出诊断
气溶胶	qnwfa_curr, qnifa_curr, qnbca_curr, qnwfa2d, qnifa2d, qnbca2d, qnocbb2d, qnbcbb2d, naer, ccn_conc, aercu_opt, aerocu…	Thompson-aerosol 与化学耦合的家当；2D 项是排放通量
积云耦合	cu_used, qrcuten, qscuten, qicuten, qccuten	积云方案给的“对流性云水/雨倾向”（SCM 耦合用）
CAMMGMP/P3/ISHMAEL/ntu3m/SBM	dlf, dlf2, tke_pbl, pblh, …	大段方案专用字段（行尾注释标注归属），非专用方案不 PRESENT
灌溉	irrigation, sf_surf_irr_scheme, irr_*	喷灌参数（见 §21.6）
闪电	light_pe/ne/neu_curr, lpi2d/3d, lpos/lneg/lneu, f_light_*	动力闪电参数化字段
扰动	multi_perturb, pert_thom, perts_*, pert_thom_q*	WRF-Solar EPS 集合扰动（§21.10）
restart 状态	mp_restart_state, tbpvs_state, tbpvs0_state	eta/Ferrier 方案的查找表重启快照

为什么签名必须这么大：driver 是“一个接口服务所有方案”的代价面。Registry 里每个 state 变量都可能在某个方案的包里，包的并集就是这张表。**加新字段的标准流程：**Registry 加 state → module_state_description 加常量 → 本签名加实参 + 声明 → 方案 CASE 里装配——教材 ch20 的 T2 案例走的就是这条链。

```
!file: phys/module_microphysics_driver.F    L437-L456
L00437  !=================================================================
L00438    INTEGER,parameter :: iunit=6
L00439    INTEGER :: mpi_error_code=1
L00440
L00441     TYPE(grid_config_rec_type),  INTENT(IN   ) , OPTIONAL   :: config_flags
L00442     INTEGER,     INTENT(IN   )    :: mp_physics
L00443     LOGICAL,     INTENT(IN   )    :: specified
L00444     INTEGER, OPTIONAL, INTENT(IN   )    :: chem_opt, progn
L00445     INTEGER, OPTIONAL, INTENT(IN   )    :: hail, ice2 !, ccntype
L00446  !
L00447     INTEGER,       INTENT(IN   )    ::       ids,ide, jds,jde, kds,kde
L00448     INTEGER,       INTENT(IN   )    ::       ims,ime, jms,jme, kms,kme,num_scalar
L00449  #if (EM_CORE == 1)
L00450     INTEGER,       INTENT(IN   )    ::     num_sbmradar
L00451     INTEGER,       INTENT(IN   )    ::     sbm_diagnostics
L00452  #endif
L00453     INTEGER, OPTIONAL, INTENT(IN   )    ::       ips,ipe, jps,jpe, kps,kpe
L00454     INTEGER,       INTENT(IN   )    ::                        kts,kte
L00455     INTEGER,       INTENT(IN   )    ::     itimestep,num_tiles,spec_zone
L00456     INTEGER, DIMENSION(num_tiles), INTENT(IN) ::                        &
```

config_flags 是 OPTIONAL 结构体（唯一 PRESENT 检查在调用方做）；mp_physics 是**非 OPTIONAL 的 IN**——方案号是 driver 存在的理由。三套维度（domain/memory/tile）+ num_tiles tile 表——WRF 物理层标准的“三维度组”（教材 ch03 架构章讲过 dims 三元组语义）。

```
!file: phys/module_microphysics_driver.F    L638-L731
L00638     LOGICAL, OPTIONAL :: f_qdcn,f_qtcn,f_qccn,f_qrcn,f_qnin,f_fi,f_fs,   &  ! for ntu3m
L00639                          f_vi,f_vs,f_vg,f_ai,f_as,f_ag,f_ah,f_i3m          ! for ntu3m
L00640     REAL, OPTIONAL, DIMENSION( ims:ime , kms:kme, jms:jme ) , INTENT(OUT) :: vmi3d,di3d,rhopo3d,    & ! for P3
L00641                                                                             vmi3d_2,di3d_2,rhopo3d_2,  & ! for P3
L00642                                                                             phii3d,         & ! for Jensen ISHMAEL
L00643                                                                             phii3d_2,       & ! for Jensen ISHMAEL
L00644                                                                             vmi3d_3,di3d_3,rhopo3d_3,  & ! for Jensen
↪  ISHMAEL
L00645                                                                             phii3d_3,       & ! for Jensen ISHMAEL
L00646                                                                             itype,itype_2,itype_3            ! for Jensen
↪  ISHMAEL
L00647     LOGICAL,  OPTIONAL,   INTENT(IN   )    :: channel_switch
L00648     REAL, OPTIONAL,  INTENT(INOUT   ) :: naer  ! aerosol number concentration (/kg)
```

```
L00649     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(INOUT) , OPTIONAL :: qnwfa2d, qnifa2d, qnbca2d, &
L00650                                                                      qnocbb2d, qnbcbb2d
L00651     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00652           OPTIONAL,                                                 &
L00653           INTENT(INOUT ) ::                                         &
L00654                    u,v,w, z, t8w                                      &
L00655                   ,cldfra, cldfra_old, exch_h                       &
L00656                   ,qv_curr,qc_curr,qr_curr,qi_curr,qs_curr,qg_curr  &
L00657                   ,qt_curr,qndrop_curr,qni_curr,qh_curr,qnh_curr    &
L00658                   ,qns_curr,qnr_curr,qng_curr,qnn_curr,qnc_curr     &
L00659                   ,qic_curr,qip_curr,qid_curr &
L00660                   ,qnic_curr,qnip_curr,qnid_curr &
L00661                   ,qzr_curr,qzi_curr,qzs_curr,qzg_curr,qzh_curr     &
L00662                   ,qir_curr,qib_curr                                & ! for P3
L00663                   ,qi2_curr,qni2_curr,qir2_curr,qib2_curr           & ! for P3
L00664                   ,qvoli_curr,qaoli_curr                            & ! for Jensen ISHMAEL
L00665                   ,qvoli2_curr,qaoli2_curr                          & ! for Jensen ISHMAEL
L00666                   ,qi3_curr,qni3_curr,qvoli3_curr,qaoli3_curr       & ! for Jensen ISHMAEL
L00667                   ,effr_curr,ice_effr_curr,tot_effr_curr            &
L00668                   ,qic_effr_curr,qip_effr_curr,qid_effr_curr          &
L00669                   ,kext_ql,kext_qs,kext_qg          &
L00670                   ,kext_qh,kext_qa                      &
L00671                   ,kext_qic,kext_qip,kext_qid,tempc,height      &
L00672                   ,kext_ft_qic,kext_ft_qip,kext_ft_qid &
L00673                   ,kext_ft_qs,kext_ft_qg                            &
L00674                   ,qnwfa_curr,qnifa_curr,qnbca_curr                 & ! Added by G. Thompson
L00675                   ,qvolg_curr,qvolh_curr,qrimef_curr
L00676
L00677     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00678           OPTIONAL,                                                 &
L00679           INTENT(INOUT ) :: light_pe_curr,light_neu_curr,light_ne_curr        ! Dynamic Lightning
L00680
L00681     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00682           OPTIONAL,                                                 &
L00683           INTENT(IN) :: qrcuten, qscuten, qicuten, qccuten
L00684    INTEGER, INTENT(IN), optional ::     cu_used
L00685  #if ( WRF_CHEM == 1 )
L00686     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00687           INTENT(INOUT) :: rainprod, evapprod
L00688     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00689           INTENT(INOUT) :: qv_b4mp, qc_b4mp, qi_b4mp, qs_b4mp
L00690  #endif
L00691  ! YLIN
L00692  ! Added RI_CURR similar to microphysics fields above
L00693     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ),                   &
L00694           OPTIONAL,                                                 &
L00695           INTENT(INOUT) :: ri_curr
L00696
L00697
L00698     REAL, DIMENSION(ims:ime, kms:kme, jms:jme ),                    &
L00699           OPTIONAL,                                                 &
L00700           INTENT(OUT ) ::                                           &
L00701                    nsource
L00702
L00703  !
L00704     REAL, DIMENSION( ims:ime , jms:jme ),                           &
L00705           INTENT(INOUT),                                            &
L00706           OPTIONAL   ::                                             &
L00707                                                          RAINNC &
L00708                                                        ,RAINNCV &
L00709                                                         ,SNOWNC &
L00710                                                        ,SNOWNCV &
L00711                                                      ,GRAUPELNC &
L00712                                                     ,GRAUPELNCV &
L00713                                                         ,HAILNC &
L00714                                                        ,HAILNCV &
L00715                                                        ,CLOUDNC &
L00716                                          ,hail_maxk1, hail_max2d
L00717
L00718  #if ( WRF_CHEM == 1)
L00719  ! NUWRF JJS 20110525 vvvvv
L00720  ! for inline Gocart coupling
L00721   integer :: i24h
L00722   INTEGER, PARAMETER :: num_go = 14  ! number of the gocart aerosol species
L00723   REAL, DIMENSION( ims:ime, kms:kme, jms:jme, num_go) :: aero
L00724   REAL, DIMENSION( ims:ime, kms:kme, jms:jme), INTENT(OUT) :: icn_diag
L00725   REAL, DIMENSION( ims:ime, kms:kme, jms:jme), INTENT(OUT) :: nc_diag
L00726   integer, intent(in) :: gsfcgce_gocart_coupling ! EMK
L00727   REAL, PARAMETER :: frac(4)=(/ 0.01053,0.08421,0.25263,0.65263 /) !fraction for fine dust
L00728
L00729  ! NUWRF JJS 20110525 ^^^^^
L00730  #endif
L00731
```

- 声明段的**行尾注释密度极高**（`! for ntu3m`、`! for P3`、`! Added by G. Thompson`）——每个字段的“东家”直接可查。**这是读 driver 的关键技巧：先看注释归属，再决定这个字段与你关心的方案有没有关系。**
- 大段 OPTIONAL INOUT（L651–L675）把全部水凝物/诊断场声明在一起——`INTENT(INOUT)` 意味着**方案就地改场**（无本地副本、无回写段——Thompson 的 `mp_gt_driver` 直接改 `qv_curr` 等）。
- WRF_CHEM 块（L718–L730）：`aero(…,14)` 打包数组与 `num_go=14` 是 GOCART 内联耦合的局部工作区；`frac(4)` 尘埃细粒占比常数。
- 局部变量极少（L800–L825）：`i,j,k,its…` + 灌溉临时量 + TEMPO 配置结构——**driver 无状态**的另一佐证。

21.6 前置处理：从进入函数到 tile 循环

```
!file: phys/module_microphysics_driver.F    L800-L851
L00800     INTEGER :: i,j,k,its,ite,jts,jte,ij,sz,n
L00801     LOGICAL :: channel
L00802     LOGICAL :: nssl_progn = .false.
L00803     REAL    :: z0, z1, z2, w1, w2
L00804
L00805     integer, parameter :: ntot = 50
L00806     real :: wmin, wmax
L00807     integer :: ierr
L00808  ! IRRIGATION
L00809
L00810     REAL, DIMENSION( ims:ime , jms:jme ), OPTIONAL, INTENT(IN):: IRRIGATION !ARI
L00811     REAL,  OPTIONAL, INTENT(IN)::  irr_daily_amount, julian_in, xtime, gmt
L00812     INTEGER, OPTIONAL, INTENT(IN ):: sf_surf_irr_scheme, irr_start_hour, irr_num_hours, &
L00813                                      irr_start_julianday,irr_end_julianday,irr_freq,irr_ph
L00814     REAL, PARAMETER    :: PI_GRECO=3.14159
L00815     INTEGER  :: end_hour,a,b,xt24,irr_day,timing
L00816     REAL :: constants_irrigation,tloc,irr_start,phase
L00817     INTEGER, OPTIONAL, DIMENSION( ims:ime , jms:jme ), INTENT(INOUT) :: irr_rand_field
L00818
L00819  ! To accommodate shared physics
L00820     character*256 :: errmsg
L00821     integer :: errflg
L00822
L00823     type(ty_tempo_cfgs) :: tempo_cfgs
L00824     type(ty_tempo_driver_diags) :: tempo_driver_diags
L00825     logical :: config_tempo_aerosolaware, config_tempo_hailaware
L00826  !-------------------------------------------------------------------
L00827  !  check for microphysics type.  We need a clean way to
L00828  !  specify these things!
L00829  !-------------------------------------------------------------------
L00830
L00831     channel = .FALSE.
L00832     IF ( PRESENT ( channel_switch ) ) channel = channel_switch
L00833
L00834     if (mp_physics .eq. 0) return
L00835     IF( specified ) THEN
L00836       sz = spec_zone
L00837     ELSE
L00838       sz = 0
L00839     ENDIF
L00840
L00841  ! set this to true to print out the global max/min for W on each time step.
L00842     IF ( .false. ) THEN
L00843        wmax = maxval( w(ips:ipe,kps:kpe,jps:jpe) )
L00844        wmin = minval( w(ips:ipe,kps:kpe,jps:jpe) )
L00845  #if ( defined(DM_PARALLEL)  &&   ! defined(STUBMPI) )
L00846        wmax = wrf_dm_max_real ( wmax )
L00847        wmin = wrf_dm_min_real ( wmin )
L00848  #endif
L00849        WRITE( wrf_err_message , * ) 'microphysics_driver: GLOBAL w max/min = ', wmax, wmin
L00850        CALL wrf_message ( wrf_err_message )
L00851     ENDIF
```

- **三道早退/裁剪闸**：①`channel`（信道开关，L831–L832）——理想的 channel 模式不裁侧界；②`mp_physics==0` 直接 `return`（L834）——**“无微物理”是一等公民**（理想实验用）；③`specified → sz = spec_zone`（L835–L839）——侧边界松弛区内不跑微物理，`sz` 圈进 tile 循环的下标裁剪（L908–L912）。**物理不在 spec 区计算是 WRF 的通用纪律**，这里能看到它的实现位。
- 被关死的调试块（L842–L851，`IF (.false.)`）：全局 W 极值打印——需要时手开。注意 `wrf_dm_max/min_real` 的并行归约写法。

```
!file: phys/module_microphysics_driver.F    L853-L898
L00853  #ifdef XEON_OPTIMIZED_WSM5
L00854     ! the OpenMP loops are inside the scheme when running on MIC
L00855     IF ( mp_physics .EQ. WSM5SCHEME ) THEN
L00856         IF (channel) THEN
L00857           its = max(ips,ids)
L00858           ite = min(ipe,ide-1)
L00859         ELSE
L00860           its = max(ips,ids+sz)
L00861           ite = min(ipe,ide-1-sz)
L00862         ENDIF
L00863         jts = max(jps,jds+sz)
L00864         jte = min(jpe,jde-1-sz)
L00865
L00866         CALL wsm5(                                          &
L00867               TH=th                                         &
L00868              ,Q=qv_curr                                     &
L00869              ,QC=qc_curr                                    &
L00870              ,QR=qr_curr                                    &
L00871              ,QI=qi_curr                                    &
L00872              ,QS=qs_curr                                    &
L00873              ,DEN=rho,PII=pi_phy,P=p,DELZ=dz8w              &
L00874              ,DELT=dt,G=g,CPD=cp,CPV=cpv                    &
L00875              ,RD=r_d,RV=r_v,T0C=svpt0                       &
L00876              ,EP1=ep_1, EP2=ep_2, QMIN=epsilon              &
L00877              ,XLS=xls, XLV0=xlv, XLF0=xlf                   &
L00878              ,DEN0=rhoair0, DENR=rhowater                   &
L00879              ,CLIQ=cliq,CICE=cice,PSAT=psat                 &
L00880              ,RAIN=rainnc ,RAINNCV=rainncv                  &
L00881              ,SNOW=snownc ,SNOWNCV=snowncv                  &
L00882              ,SR=sr                                         &
L00883              ,REFL_10CM=refl_10cm                           &
L00884              ,diagflag=diagflag                             &
L00885              ,do_radar_ref=do_radar_ref                     &
L00886              ,has_reqc=has_reqc                             &  ! for radiation +
L00887              ,has_reqi=has_reqi                             &
L00888              ,has_reqs=has_reqs                             &
```

```
L00889             ,re_cloud=re_cloud                                 &
L00890             ,re_ice=re_ice                                     &
L00891             ,re_snow=re_snow                                   &  ! for radiation -
L00892             ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L00893             ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L00894             ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte &
L00895             )
L00896
L00897     ELSE
L00898  #endif
```

XEON 优化特例：`XEON_OPTIMIZED_WSM5` 编译宏下 WSM5 走独立路径——OpenMP 循环在方案内部（MIC/至强 _phi 时代遗产），tile 循环退化为单 tile。整段是“方案级并行 vs driver 级并行”的对照样本：本文件默认是 driver 级（下一段），WSM5 特例是方案级。

```
!file: phys/module_microphysics_driver.F    L900-L935
L00900     !$OMP PARALLEL DO   &
L00901     !$OMP PRIVATE ( ij, its, ite, jts, jte, i,j,k,n )
L00902
L00903     DO ij = 1 , num_tiles
L00904         IF (channel) THEN
L00905           its = max(i_start(ij),ids)
L00906           ite = min(i_end(ij),ide-1)
L00907         ELSE
L00908           its = max(i_start(ij),ids+sz)
L00909           ite = min(i_end(ij),ide-1-sz)
L00910         ENDIF
L00911         jts = max(j_start(ij),jds+sz)
L00912         jte = min(j_end(ij),jde-1-sz)
L00913
L00914  # if( EM_CORE==1 )
L00915         sf_surf_irr: SELECT CASE(sf_surf_irr_scheme)
L00916              CASE(SPRINKLER)
L00917               CALL sprinkler_irrigation(  julian_in                    &
L00918       &          ,irrigation, irr_daily_amount,rho,dz8w               &
L00919       &          ,irr_start_hour,irr_num_hours,irr_start_julianday    &
L00920       &          ,irr_end_julianday,irr_freq,irr_ph,qr_curr           &
L00921       &          ,gmt,xtime,dt,irr_rand_field                         &
L00922       &          ,ids,ide, jds,jde                                    & ! domain dims
L00923       &          ,ims,ime, jms,jme, kms,kme                           & ! memory dims
L00924       &          ,its,ite, jts,jte                 )
L00925         END SELECT sf_surf_irr
L00926  #endif
L00927
L00928
L00929
L00930  ! 2009-06009 rce - zero all these for safety
L00931         IF( PRESENT(qlsink) ) qlsink(its:ite,kts:kte,jts:jte) = 0.
L00932         IF( PRESENT(precr ) ) precr(its:ite,kts:kte,jts:jte)  = 0.
L00933         IF( PRESENT(preci ) ) preci(its:ite,kts:kte,jts:jte)  = 0.
L00934         IF( PRESENT(precs ) ) precs(its:ite,kts:kte,jts:jte)  = 0.
L00935         IF( PRESENT(precg ) ) precg(its:ite,kts:kte,jts:jte)  = 0.
```

- **`!$OMP PARALLEL DO` 包住整个 tile 循环**（L900–L903）——后续全部方案调用都在线程私有 tile 上执行。**这是物理层并行的实际发生地**：每个线程处理自己的 tile，方案内部不再并行（多数方案）。结合 ch21 的 solve_em 走读：动力内核的 MPI 分解 × 物理层的 OpenMP tile 分解，两层正交。
- **tile 循环第一件事是灌溉**（L914–L925，`sf_surf_irr: SELECT CASE`）：`SPRINKLER` 方案直接往 `qr_curr` 里加水——**喷灌被实现为“微物理前的雨量注入”**，位置在方案调用之前、tile 循环之内。
- **安全清零**（L930–L935，rce 2009-06-09 注释）：`qlsink/precr/preci/precs/precg` 每步清零——这些三维诊断多数方案不填，不清零就会带着上一步的旧值进历史输出。

```
!file: phys/module_microphysics_driver.F    L938-L974
L00938         IF( PRESENT(chem_opt) .AND. PRESENT(progn) ) THEN
L00939
L00940         ! ERM: check whether to use built-in droplet nucleation or use qndrop from CHEM
L00941         IF ( mp_physics==NSSL_2MOM .and. config_flags%nssl_2moment_on==1 ) THEN
L00942           IF ( progn > 0 ) THEN
L00943            IF ( .not. (chem_opt == 0 .or. chem_opt == 401) ) nssl_progn = .true.
L00944           ELSE
L00945             nssl_progn = .false. ! use NUCOND for droplet nucleation
L00946           ENDIF
L00947         ENDIF
L00948
L00949         !Add pass for dust-only wrf-chem option - RAS
L00950         IF( (chem_opt==0 .OR. chem_opt==401) .AND. progn==1 .AND. (mp_physics==LINSCHEME  .OR.
↪  mp_physics==MORR_TWO_MOMENT)) THEN
L00951           IF( PRESENT( QNDROP_CURR ) ) THEN
L00952             CALL wrf_debug ( 100 , 'microphysics_driver: calling prescribe_aerosol_mixactivate' )
L00953  ! 06-nov-2005 rce - id  & itimestep added to arg list
L00954               call prescribe_aerosol_mixactivate (                &
L00955                    id, itimestep, dt, naer,                       &
L00956                    ccn_conc, chem_opt,                            & !RAS13.1
L00957                    rho, th, pi_phy, w, cldfra, cldfra_old,        &
L00958                    z, dz8w, p8w, t8w, exch_h,                     &
L00959                    qv_curr, qc_curr, qi_curr, qndrop_curr,        &
L00960                    nsource,                                       &
L00961                    ids,ide, jds,jde, kds,kde,                     &
L00962                    ims,ime, jms,jme, kms,kme,                     &
L00963                    its,ite, jts,jte, kts,kte,                     &
L00964                    F_QC=f_qc, F_QI=f_qi                           )
L00965           END IF
L00966         ELSEIF ( (chem_opt==0 .OR. chem_opt==401) .AND. progn==1 .AND.    &
```

```
L00967                   (mp_physics==NSSL_2MOM .and. config_flags%nssl_2moment_on==1)) THEN
L00968  !          Do nothing here for the moment. Use activation of CCN within the NSSL_2MOM scheme instead, based on
↪  nssl_cccn namelist value.
L00969          ELSEIF ( progn==1 .AND. mp_physics/=LINSCHEME .AND. mp_physics/=MORR_TWO_MOMENT &
L00970                   .AND. .not. (mp_physics==NSSL_2MOM .and. config_flags%nssl_2moment_on==1) ) THEN
L00971                call wrf_error_fatal( &
L00972                "SETTINGS ERROR: Prognostic cloud droplet number can only be used with the mp_physics=LINSCHEME or
↪  MORRISON or NSSL_2MOM.")
L00973          END IF
L00974          END IF
```

化学-微物理联动（`chem_opt + progn`）：LIN/Morrison 方案 + `progn=1`（预报云滴数）时先调 `prescribe_aerosol_mixactivate`（化学供 CCN → 活化出 `qndrop`）；NSSL 双矩走自己的内生活化；**其它方案配 `progn=1` 直接 fatal**（L969–L972 的 "SETTINGS ERROR"）——**兼容矩阵在 driver 里逐条硬编码**，这是 WRF 防错误配置的主要手段之一（教材 ch13 §13.6.4 的兼容表源码实证）。

```
!file: phys/module_microphysics_driver.F    L976-L1006
L00976  #if ( WRF_CHEM == 1)
L00977  !NUWRF JJS 20110525 vvvvv
L00978  ! Pack gocart aerosol species
L00979  ! All aerosol species in chem are in "ug/kg-dryair"
L00980  !  and conerted to (g/m**3)
L00981
L00982     aero(:,:,:,:) = 0.
L00983     do k = kts, kte
L00984        do j = jts, jte
L00985           do i = its, ite
L00986           aero(i,k,j, 1) = max(0.0, chem(i,k,j,p_sulf)*1.0e-6*p(i,k,j)*    &
L00987                            96.0/(8.314*th(i,k,j)*pi_phy(i,k,j)))  ! 1 = SO4
L00988           aero(i,k,j, 2) = max(0.0, (chem(i,k,j,p_bc1)+chem(i,k,j,p_bc2))*1.0e-6*rho(i,k,j))  ! 2 = BC1+BC2
L00989           aero(i,k,j, 3) = max(0.0, chem(i,k,j,p_oc1)*1.0e-6*rho(i,k,j)*1.4e0)          !  3 = OC1
L00990           aero(i,k,j, 4) = max(0.0, chem(i,k,j,p_oc2)*1.0e-6*rho(i,k,j)*1.4e0)          !  4 = OC2
L00991           aero(i,k,j, 5) = max(0.0, chem(i,k,j,p_seas_1)*1.0e-6*rho(i,k,j))          !  5 = SS1
L00992           aero(i,k,j, 6) = max(0.0, (chem(i,k,j,p_seas_2)+chem(i,k,j,p_seas_3)+  &
L00993                                      chem(i,k,j,p_seas_4))*1.0e-6*rho(i,k,j))          !  6 = SS2+SS3+SS4
L00994           aero(i,k,j, 7) = max(0.0, chem(i,k,j,p_dust_1)*1.0e-6*rho(i,k,j)*frac(1))    !  7 = DU1 dust mode 1
L00995           aero(i,k,j, 8) = max(0.0, chem(i,k,j,p_dust_1)*1.0e-6*rho(i,k,j)*frac(2))    !  8 = DU1 dust mode 2
L00996           aero(i,k,j, 9) = max(0.0, chem(i,k,j,p_dust_1)*1.0e-6*rho(i,k,j)*frac(3))    !  9 = DU1 dust mode 3
L00997           aero(i,k,j,10) = max(0.0, chem(i,k,j,p_dust_1)*1.0e-6*rho(i,k,j)*frac(4))    ! 10 = DU1 dust mode 4
L00998           aero(i,k,j,11) = max(0.0, chem(i,k,j,p_dust_2)*1.0e-6*rho(i,k,j))          ! 11 = DU2 dust mode 5
L00999           aero(i,k,j,12) = max(0.0, chem(i,k,j,p_dust_3)*1.0e-6*rho(i,k,j))          ! 11 = DU3 dust mode 6
L01000           aero(i,k,j,13) = max(0.0, chem(i,k,j,p_dust_4)*1.0e-6*rho(i,k,j))          ! 11 = DU4 dust mode 7
L01001           aero(i,k,j,14) = max(0.0, chem(i,k,j,p_dust_5)*1.0e-6*rho(i,k,j))          ! 11 = DU5 dust mode 8
L01002           enddo ! i
L01003        enddo ! j
L01004     enddo ! k
L01005  !NUWRF JJS 20110525 ^^^^^
L01006  #endif
```

GOCART 气溶胶打包：chem 数组（μg/kg 干空气）逐物种换算成 μg/m³ 并拆尘埃模态（`frac(4)` 细粒占比）——**单位换算链写在算式里：`×1e-6×ρ` 或 `×1e-6×p×96/(R·T)`**（硫酸盐按摩尔质量 96 换算）——化学接口的量纲陷阱密集区。

21.7 分发骨架：`micro_select` 与 KESSLER 样板

```
!file: phys/module_microphysics_driver.F    L1008-L1032
L01008       micro_select: SELECT CASE(mp_physics)
L01009
L01010          CASE (KESSLERSCHEME)
L01011               CALL wrf_debug ( 100 , 'microphysics_driver: calling kessler' )
L01012               IF ( PRESENT( QV_CURR ) .AND. PRESENT( QC_CURR ) .AND.  &
L01013                                             PRESENT( QR_CURR ) .AND.  &
L01014                    PRESENT( RAINNC  ) .AND. PRESENT ( RAINNCV ) .AND.  &
L01015                                             PRESENT( Z       ))  THEN
L01016                 CALL kessler(                                          &
L01017                    T=th                                                &
L01018                   ,QV=qv_curr                                          &
L01019                   ,QC=qc_curr                                          &
L01020                   ,QR=qr_curr                                          &
L01021                   ,RHO=rho, PII=pi_phy,DT_IN=dt, Z=z, XLV=xlv, CP=cp &
L01022                   ,EP2=ep_2,SVP1=svp1,SVP2=svp2                        &
L01023                   ,SVP3=svp3,SVPT0=svpt0,RHOWATER=rhowater             &
L01024                   ,DZ8W=dz8w                                           &
L01025                   ,RAINNC=rainnc,RAINNCV=rainncv                       &
L01026                   ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L01027                   ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L01028                   ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte &
L01029                                                                        )
L01030               ELSE
L01031                  CALL wrf_error_fatal ( 'arguments not present for calling kessler' )
L01032               ENDIF
```

- **标签 SELECT**（`micro_select: SELECT CASE(mp_physics)`）：Fortran 命名块结构——`END SELECT micro_select`（L3035）配对，3000 行的分发体里随时能定位边界。
- **KESSLER 是最小样板**：`wrf_debug(100, …)` 报进入 → `IF (PRESENT(五件套))` 实参墙 → `CALL kessler(T=th, QV=qv_curr, …, XLV=xlv, CP=cp, …)`。三个可学习的细节：① **关键字实参**（`T=th`）——方案形参名与 driver 实参名不必相同，装配的 "翻译" 就发生在这些 `=` 号上；② **常数也走实参**（`XLV=xlv, CP=cp, SVP1=svp1…`）——Kessler 是老派风格，Thompson 则常数自

养（code_07a §07a.4）；③ 实参墙失败即 `wrf_error_fatal`——**配置错误在第一时步就炸，不带病运行**。

- 对照 RCON/THOMPSON 家族的实参墙（十几行 PRESENT 链）可见：**方案维度越高，墙越长**——实参墙长度就是方案的 “Registry 依赖面积”。

21.8 Thompson 三 CASE：8 号、28 号、38 号

```
!file: phys/module_microphysics_driver.F    L1146-L1249
L01146          CASE (THOMPSONAERO)
L01147           if (pert_thom .and. multi_perturb == 1) then
L01148             allocate (qv_tmp(its:ite, kts:kte, jts:jte))
L01149             allocate (qc_tmp(its:ite, kts:kte, jts:jte))
L01150             allocate (qi_tmp(its:ite, kts:kte, jts:jte))
L01151             allocate (qs_tmp(its:ite, kts:kte, jts:jte))
L01152             allocate (qni_tmp(its:ite, kts:kte, jts:jte))
L01153
L01154             call Add_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L01155               perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L01156               pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L01157               qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L01158           end if
L01159
L01160           CALL wrf_debug ( 100 , 'microphysics_driver: calling thompson' )
L01161           IF ( PRESENT( QV_CURR ) .AND. PRESENT ( QC_CURR )   .AND.  &
L01162                PRESENT( QR_CURR ) .AND. PRESENT ( QI_CURR )   .AND.  &
L01163                PRESENT( QS_CURR ) .AND. PRESENT ( QG_CURR )   .AND.  &
L01164                PRESENT( QNR_CURR) .AND. PRESENT ( QNI_CURR)   .AND.  &
L01165                PRESENT( QNC_CURR) .AND. PRESENT ( QNWFA_CURR) .AND.  &
L01166                PRESENT( QNIFA_CURR).AND.PRESENT ( QNWFA2D)    .AND.  &
L01167                PRESENT( QNIFA2D)                              .AND.  &
L01168                PRESENT( SNOWNC)   .AND. PRESENT ( SNOWNCV)    .AND.  &
L01169                PRESENT( GRAUPELNC).AND. PRESENT ( GRAUPELNCV) .AND.  &
L01170                PRESENT( RAINNC  ) .AND. PRESENT ( RAINNCV ) ) THEN
L01171  #if ( WRF_CHEM == 1 )
L01172                qv_b4mp(its:ite,kts:kte,jts:jte) = qv_curr(its:ite,kts:kte,jts:jte)
L01173                qc_b4mp(its:ite,kts:kte,jts:jte) = qc_curr(its:ite,kts:kte,jts:jte)
L01174                qi_b4mp(its:ite,kts:kte,jts:jte) = qi_curr(its:ite,kts:kte,jts:jte)
L01175                qs_b4mp(its:ite,kts:kte,jts:jte) = qs_curr(its:ite,kts:kte,jts:jte)
L01176  #endif
L01177           CALL mp_gt_driver(                          &
L01178                QV=qv_curr,                            &
L01179                QC=qc_curr,                            &
L01180                QR=qr_curr,                            &
L01181                QI=qi_curr,                            &
L01182                QS=qs_curr,                            &
L01183                QG=qg_curr,                            &
L01184                NI=qni_curr,                           &
L01185                NR=qnr_curr,                           &
L01186                NC=qnc_curr,                           &
L01187                NWFA=qnwfa_curr,                       &
L01188                NIFA=qnifa_curr,                       &
L01189                NBCA=qnbca_curr,                       &
L01190                NWFA2D=qnwfa2d,                        &
L01191                NIFA2D=qnifa2d,                        &
L01192                NBCA2D=qnbca2d,                        &
L01193                aer_init_opt=config_flags%aer_init_opt,   &
L01194                wif_input_opt=config_flags%wif_input_opt, &
L01195                TH=th,                                 &
L01196                PII=pi_phy,                            &
L01197                P=p,                                   &
L01198                W=w,                                   &
L01199                DZ=dz8w,                               &
L01200                DT_IN=dt,                              &
L01201                ITIMESTEP=itimestep,                   &
L01202                RAINNC=RAINNC,                         &
L01203                RAINNCV=RAINNCV,                       &
L01204                SNOWNC=SNOWNC,                         &
L01205                SNOWNCV=SNOWNCV,                       &
L01206                GRAUPELNC=GRAUPELNC,                   &
L01207                GRAUPELNCV=GRAUPELNCV,                 &
L01208                SR=SR,                                 &
L01209  #if ( WRF_CHEM == 1 )
L01210                WETSCAV_ON=config_flags%wetscav_onoff == 1, &
L01211                RAINPROD=rainprod,                     &
L01212                EVAPPROD=evapprod,                     &
L01213  #endif
L01214                REFL_10CM=refl_10cm,                   &
L01215                diagflag=diagflag,                     &
L01216                ke_diag = ke_diag,                     &
L01217                do_radar_ref=do_radar_ref,             &
L01218                re_cloud=re_cloud,                     &
L01219                re_ice=re_ice,                         &
L01220                re_snow=re_snow,                       &
L01221                has_reqc=has_reqc,                     & ! G. Thompson
L01222                has_reqi=has_reqi,                     & ! G. Thompson
L01223                has_reqs=has_reqs,                     & ! G. Thompson
L01224             IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde, &
L01225             IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme, &
L01226             ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L01227
L01228           if (pert_thom .and. multi_perturb == 1) then
L01229             call Remove_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L01230               perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L01231               pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L01232               qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L01233             deallocate (qv_tmp)
L01234             deallocate (qc_tmp)
L01235             deallocate (qi_tmp)
L01236             deallocate (qs_tmp)
L01237             deallocate (qni_tmp)
L01238           end if
```

```
L01239
L01240               IF (config_flags%aer_fire_emit_opt.gt.0)  then
L01241                 CALL wrf_debug ( 200 , ' call fire_emis_simple_plumerise' )
L01242                 CALL fire_emis_simple_plumerise (config_flags%wif_fire_inj, config_flags%aer_fire_emit_opt  &
L01243                     ,z_at_mass, pblh, qnwfa_curr, qnbca_curr   &
L01244                     ,qnocbb2d, qnbcbb2d, dt, ids, ide, jds, jde, kds, kde                    &
L01245                     ,ims, ime, jms, jme, kms, kme, its, ite, jts, jte, kts, kte                    )
L01246               ENDIF
L01247             ELSE
L01248                CALL wrf_error_fatal ( 'arguments not present for calling thompson_et_al' )
L01249             ENDIF
```

THOMPSONAERO（mp_physics=28，aerosol-aware）：- 集合扰动前置（L1147–L1158）：pert_thom .and. multi_perturb==1 时先 Add_multi_perturb_mp_perturbations 把扰动场乘进 qv/qc/qi/qs/qni——扰动在方案调用前加、调用后减（**L1228–L1238**），方案在 **“被扰动的世界”** 里跑一步，输出趋势再作用回原场——WRF-Solar EPS 的微物理参数化不确定性表征。- 实参墙检查 QNC/QNWFA/QNIFA/QNWFA2D/QNIFA2D 在场——**28 号比 8 号多五个气溶胶字段**。- CALL mp_gt_driver 的装配表：NC=qnc_curr, NWFA=qnwfa_curr, NIFA=qnifa_curr, NBCA=qnbca_curr, NWFA2D=qnwfa2d, NIFA2D=qnifa2d, NBCA2D=qnbca2d + aer_init_opt/wif_input_opt 两个 config 字段——**code_07a §07a.7 里 mp_gt_driver 的 OPTIONAL 实参全部在这里供上**。- WETSCAV_ON=config_flags%wetscav_onoff == 1（L1210）——**逻辑表达式直接作实参**：namelist 开关换算成 LOGICAL 传给方案。- 生物质燃烧注入（L1240–L1246，aer_fire_emit_opt>0）：方案跑完后把火排放注入 qnwfa/qnbca——**排放发生在微物理之后**，下一时步才被活化。

```
!file: phys/module_microphysics_driver.F    L1251-L1324
L01251          CASE (THOMPSONGH)
L01252               CALL wrf_debug ( 100 , 'microphysics_driver: calling thompsongh' )
L01253               IF ( PRESENT( QV_CURR ) .AND. PRESENT ( QC_CURR )    .AND.  &
L01254                    PRESENT( QR_CURR ) .AND. PRESENT ( QI_CURR )    .AND.  &
L01255                    PRESENT( QS_CURR ) .AND. PRESENT ( QG_CURR )    .AND.  &
L01256                    PRESENT( QNR_CURR) .AND. PRESENT ( QNI_CURR)    .AND.  &
L01257                    PRESENT( QNG_CURR) .AND. PRESENT ( QVOLG_CURR) .AND.  &
L01258                    PRESENT( QNC_CURR) .AND. PRESENT ( QNWFA_CURR) .AND.  &
L01259                    PRESENT( QNIFA_CURR).AND.PRESENT ( QNWFA2D)    .AND.  &
L01260                    PRESENT( QNIFA2D)                              .AND.  &
L01261                    PRESENT( SNOWNC)    .AND. PRESENT ( SNOWNCV)    .AND.  &
L01262                    PRESENT( GRAUPELNC).AND. PRESENT ( GRAUPELNCV) .AND.  &
L01263                    PRESENT( RAINNC  ) .AND. PRESENT ( RAINNCV ) ) THEN
L01264  #if ( WRF_CHEM == 1 )
L01265                   qv_b4mp(its:ite,kts:kte,jts:jte) = qv_curr(its:ite,kts:kte,jts:jte)
L01266                   qc_b4mp(its:ite,kts:kte,jts:jte) = qc_curr(its:ite,kts:kte,jts:jte)
L01267                   qi_b4mp(its:ite,kts:kte,jts:jte) = qi_curr(its:ite,kts:kte,jts:jte)
L01268                   qs_b4mp(its:ite,kts:kte,jts:jte) = qs_curr(its:ite,kts:kte,jts:jte)
L01269  #endif
L01270               CALL mp_gt_driver(                              &
L01271                       QV=qv_curr,                             &
L01272                       QC=qc_curr,                             &
L01273                       QR=qr_curr,                             &
L01274                       QI=qi_curr,                             &
L01275                       QS=qs_curr,                             &
L01276                       QG=qg_curr,                             &
L01277                       QB=qvolg_curr,                          &
L01278                       NI=qni_curr,                            &
L01279                       NR=qnr_curr,                            &
L01280                       NC=qnc_curr,                            &
L01281                       NG=qng_curr,                            &
L01282                       NWFA=qnwfa_curr,                        &
L01283                       NIFA=qnifa_curr,                        &
L01284                       NBCA=qnbca_curr,                        &
L01285                       NWFA2D=qnwfa2d,                         &
L01286                       NIFA2D=qnifa2d,                         &
L01287                       NBCA2D=qnbca2d,                         &
L01288                       aer_init_opt=config_flags%aer_init_opt,   &
L01289                       wif_input_opt=config_flags%wif_input_opt, &
L01290                       TH=th,                                  &
L01291                       PII=pi_phy,                             &
L01292                       P=p,                                    &
L01293                       W=w,                                    &
L01294                       DZ=dz8w,                                &
L01295                       DT_IN=dt,                               &
L01296                       ITIMESTEP=itimestep,                    &
L01297                       RAINNC=RAINNC,                          &
L01298                       RAINNCV=RAINNCV,                        &
L01299                       SNOWNC=SNOWNC,                          &
L01300                       SNOWNCV=SNOWNCV,                        &
L01301                       GRAUPELNC=GRAUPELNC,                    &
L01302                       GRAUPELNCV=GRAUPELNCV,                  &
L01303                       SR=SR,                                  &
L01304  #if ( WRF_CHEM == 1 )
L01305                       WETSCAV_ON=config_flags%wetscav_onoff == 1, &
L01306                       RAINPROD=rainprod,                      &
L01307                       EVAPPROD=evapprod,                      &
L01308  #endif
L01309                       REFL_10CM=refl_10cm,                    &
L01310                       diagflag=diagflag,                      &
L01311                       ke_diag = ke_diag,                      &
L01312                       do_radar_ref=do_radar_ref,              &
L01313                       re_cloud=re_cloud,                      &
L01314                       re_ice=re_ice,                          &
L01315                       re_snow=re_snow,                        &
L01316                       has_reqc=has_reqc,                      & ! G. Thompson
L01317                       has_reqi=has_reqi,                      & ! G. Thompson
L01318                       has_reqs=has_reqs,                      & ! G. Thompson
L01319                     IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde, &
L01320                     IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme, &
L01321                     ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L01322               ELSE
```

```
L01323                CALL wrf_error_fatal ( 'arguments not present for calling thompson_et_al' )
L01324             ENDIF
```

THOMPSONGH（三矩霰/雹版）：与 28 号几乎逐行相同，**唯二差异**在装配表——多传 `QB=qvolg_curr, NG=qng_curr`。回到 code_07a §07a.6.1 的结论：`mp_gt_driver` 靠 `PRESENT(ng)` 判 `is_hail_aware`——**driver 传不传这两个参数，就是三矩开关本身**。THOMPSONGH 的实参墙也多了 `QNG_CURR/QVOLG_CURR` 两项。方案的“家族梯度”（8 单矩云 →28 双矩云 →38 三矩霰）在 driver 层表现为**装配表的增量**。

```
!file: phys/module_microphysics_driver.F    L1326-L1410
L01326          CASE (THOMPSON)
L01327               CALL wrf_debug ( 100 , 'microphysics_driver: calling thompson' )
L01328               IF ( PRESENT( QV_CURR ) .AND. PRESENT ( QC_CURR ) .AND.  &
L01329                    PRESENT( QR_CURR ) .AND. PRESENT ( QI_CURR ) .AND.  &
L01330                    PRESENT( QS_CURR ) .AND. PRESENT ( QG_CURR ) .AND.  &
L01331                    PRESENT( QNR_CURR) .AND. PRESENT ( QNI_CURR) .AND.  &
L01332  !                  PRESENT( SNOWNC) .AND. PRESENT ( SNOWNCV) .AND.        &
L01333  !                  PRESENT( GRAUPELNC) .AND. PRESENT ( GRAUPELNCV) .AND.  &
L01334                    PRESENT( RAINNC  ) .AND. PRESENT ( RAINNCV ) ) THEN
L01335  #if ( WRF_CHEM == 1 )
L01336                   qv_b4mp(its:ite,kts:kte,jts:jte) = qv_curr(its:ite,kts:kte,jts:jte)
L01337                   qc_b4mp(its:ite,kts:kte,jts:jte) = qc_curr(its:ite,kts:kte,jts:jte)
L01338                   qi_b4mp(its:ite,kts:kte,jts:jte) = qi_curr(its:ite,kts:kte,jts:jte)
L01339                   qs_b4mp(its:ite,kts:kte,jts:jte) = qs_curr(its:ite,kts:kte,jts:jte)
L01340  #endif
L01341                if (pert_thom .and. multi_perturb == 1) then
L01342                  allocate (qv_tmp(its:ite, kts:kte, jts:jte))
L01343                  allocate (qc_tmp(its:ite, kts:kte, jts:jte))
L01344                  allocate (qi_tmp(its:ite, kts:kte, jts:jte))
L01345                  allocate (qs_tmp(its:ite, kts:kte, jts:jte))
L01346                  allocate (qni_tmp(its:ite, kts:kte, jts:jte))
L01347
L01348                  call Add_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L01349                    perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L01350                    pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L01351                    qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L01352                end if
L01353
L01354               CALL mp_gt_driver(                          &
L01355                       QV=qv_curr,                         &
L01356                       QC=qc_curr,                         &
L01357                       QR=qr_curr,                         &
L01358                       QI=qi_curr,                         &
L01359                       QS=qs_curr,                         &
L01360                       QG=qg_curr,                         &
L01361                       NI=qni_curr,                        &
L01362                       NR=qnr_curr,                        &
L01363                       TH=th,                              &
L01364                       PII=pi_phy,                         &
L01365                       P=p,                                &
L01366                       W=w,                                &
L01367                       DZ=dz8w,                            &
L01368                       DT_IN=dt,                           &
L01369                       ITIMESTEP=itimestep,                &
L01370                       RAINNC=RAINNC,                      &
L01371                       RAINNCV=RAINNCV,                    &
L01372                       SNOWNC=SNOWNC,                      &
L01373                       SNOWNCV=SNOWNCV,                    &
L01374                       GRAUPELNC=GRAUPELNC,                &
L01375                       GRAUPELNCV=GRAUPELNCV,              &
L01376                       SR=SR,                              &
L01377  #if ( WRF_CHEM == 1 )
L01378                       WETSCAV_ON=config_flags%wetscav_onoff == 1, &
L01379                       RAINPROD=rainprod,                  &
L01380                       EVAPPROD=evapprod,                  &
L01381  #endif
L01382                       REFL_10CM=refl_10cm,                &
L01383                       diagflag=diagflag,                  &
L01384                       ke_diag = ke_diag,                  &
L01385                       do_radar_ref=do_radar_ref,          &
L01386                       re_cloud=re_cloud,                  & ! G. Thompson
L01387                       re_ice=re_ice,                      & ! G. Thompson
L01388                       re_snow=re_snow,                    & ! G. Thompson
L01389                       has_reqc=has_reqc,                  & ! G. Thompson
L01390                       has_reqi=has_reqi,                  & ! G. Thompson
L01391                       has_reqs=has_reqs,                  & ! G. Thompson
L01392                     IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde, &
L01393                     IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme, &
L01394                     ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L01395
L01396                if (pert_thom .and. multi_perturb == 1) then
L01397                  call Remove_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L01398                    perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L01399                    pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L01400                    qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L01401                  deallocate (qv_tmp)
L01402                  deallocate (qc_tmp)
L01403                  deallocate (qi_tmp)
L01404                  deallocate (qs_tmp)
L01405                  deallocate (qni_tmp)
L01406                end if
L01407
L01408               ELSE
L01409                  CALL wrf_error_fatal ( 'arguments not present for calling thompson_et_al' )
L01410               ENDIF
```

THOMPSON（mp_physics=8）：- 实参墙**没有气溶胶字段**（SNOWNC/GRAUPELNC 的检查还被注释掉，L1332–L1333）——8 号可在最小 moist 包下运行。- `CALL mp_gt_driver` 不传 `NC/NWFA/NIFA/NBCA`——OPTIONAL 缺席 → `is_aerosol_aware=.FALSE.` → 方案内部走常数 `Nt_c` 分支。**“不传就**

是配置”——OPTIONAL 实参架构的全部意义。- 扰动对（Add/Remove）同样在场（L1341–L1406）——8 号也能做 EPS 扰动。

21.9 SELECT 收尾与闪电诊断

```
!file: phys/module_microphysics_driver.F    L3036-L3098
L03036
L03037        IF (config_flags%dyn_lightning_option.eq.1) THEN
L03038        IF (  PRESENT( QC_CURR ) .AND.  &
L03039              PRESENT( QR_CURR ) .AND. PRESENT ( QI_CURR )   .AND.  &
L03040              PRESENT( QS_CURR ) .AND. PRESENT ( QG_CURR )) THEN
L03041              CALL calc_lpi_new(                              &
L03042                       W=w,                                   &
L03043                       PI_PHY=pi_phy, RHO_PHY=rho,            &
L03044                       TH_PHY=TH,P_PHY=p,                     &
L03045                       DZ8w=dz8w,                             &
L03046                       QC=qc_curr,                            &
L03047                       QR=qr_curr,                            &
L03048                       QI=qi_curr,                            &
L03049                       QS=qs_curr,                            &
L03050                       QG=qg_curr,                            &
L03051                       QH=qh_curr,                            &
L03052                       lpi2d = lpi2d,                         &
L03053                       lpi3d = lpi3d                          &
L03054                      ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L03055                      ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L03056                      ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L03057              CALL   calc_dyn_pe(dx=dx,dy=dy, &
L03058                       W=w,                                   &
L03059                       PI_PHY=pi_phy, RHO_PHY=rho,            &
L03060                       TH_PHY=TH,P_PHY=p,                     &
L03061                       DZ8w=dz8w,                             &
L03062                       QC=qc_curr,                            &
L03063                       QR=qr_curr,                            &
L03064                       QI=qi_curr,                            &
L03065                       QS=qs_curr,                            &
L03066                       QG=qg_curr,                            &
L03067                       QH=qh_curr,                            &
L03068                       light_pe=light_pe_curr,                &
L03069                       light_ne=light_ne_curr,                &
L03070                       light_neu=light_neu_curr,              &
L03071                       dt=dt                                  &
L03072                      ,coul_pos=config_flags%coul_pos         &
L03073                      ,coul_neg=config_flags%coul_neg         &
L03074                      ,coul_neu=config_flags%coul_neu         &
L03075                      ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L03076                      ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L03077                      ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L03078               CALL   calcstroke(dx=dx,dy=dy,                 &
L03079                       W=w,                                   &
L03080                       PI_PHY=pi_phy, RHO_PHY=rho,            &
L03081                       TH_PHY=TH,P_PHY=p,                     &
L03082                       DZ8w=dz8w,                             &
L03083                       light_pe=light_pe_curr,                &
L03084                       light_ne=light_ne_curr,                &
L03085                       light_neu=light_neu_curr,              &
L03086                       lpos=lpos,lneg=lneg,lneu=lneu          &
L03087                      ,j_pos=config_flags%j_pos               &
L03088                      ,j_neg=config_flags%j_neg               &
L03089                      ,j_neu=config_flags%j_neu               &
L03090                      ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde &
L03091                      ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme &
L03092                      ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte)
L03093        ELSE
L03094             CALL wrf_error_fatal ( 'arguments not present for calling dynamic lightning scheme' )
L03095        ENDIF
L03096        END IF
L03097
L03098      ENDDO
```

- END SELECT micro_select 之后、tile 循环 ENDDO（L3097）之前，是**方案无关的收尾诊断**：dyn_lightning_option=1 时用微物理后的水凝物场算闪电——calc_lpi_new（LPI 指数）、calc_dyn_pe（正/负/总电荷产生率）、calcstroke（闪光定位）。**闪电紧贴微物理的原因**：它消费的是“本步刚算完”的水凝物与上升速度。
- !$OMP END PARALLEL DO（L3098）与 L900 配对——tile 循环到此结束，driver 的主体就这一个循环。

21.10 扰动子程序对

```
!file: phys/module_microphysics_driver.F    L3130-L3164    (subroutine Add_multi_perturb_mp_perturbations)
L03130      subroutine Add_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L03131          perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L03132          pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L03133          qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L03134
L03135        implicit none
L03136
L03137        integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L03138        real, intent(in) :: pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, pert_thom_ni
L03139        real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_qvapor, perts_qcloud, perts_qice, &
L03140            perts_qsnow, perts_ni
L03141        real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: qv_curr, qc_curr, qi_curr, qs_curr, qni_curr
L03142        real, dimension (its:ite, kts:kte, jts:jte), intent (out) :: qv_tmp, qc_tmp, qi_tmp, qs_tmp, qni_tmp
```

```
L03143
L03144       integer :: i, j, k
L03145
L03146
L03147       do j = jts, jte
L03148         do k = kts, kte
L03149           do i = its, ite
L03150             qv_tmp(i, k, j) = qv_curr(i, k, j)
L03151             qv_curr(i, k, j) = max (QX_MIN, (1.0 + perts_qvapor(i, k, j) * pert_thom_qv) * qv_curr(i, k, j))
L03152             qc_tmp(i, k, j) = qc_curr(i, k, j)
L03153             qc_curr(i, k, j) = max (QX_MIN, (1.0 + perts_qcloud(i, k, j) * pert_thom_qc) * qc_curr(i, k, j))
L03154             qi_tmp(i, k, j) = qi_curr(i, k, j)
L03155             qi_curr(i, k, j) = max (QX_MIN, (1.0 + perts_qice(i, k, j) * pert_thom_qi) * qi_curr(i, k, j))
L03156             qs_tmp(i, k, j) = qs_curr(i, k, j)
L03157             qs_curr(i, k, j) = max (QX_MIN, (1.0 + perts_qsnow(i, k, j) * pert_thom_qs) * qs_curr(i, k, j))
L03158             qni_tmp(i, k, j) = qni_curr(i, k, j)
L03159             qni_curr(i, k, j) = max (NI_MIN, (1.0 + perts_ni(i, k, j) * pert_thom_ni) * qni_curr(i, k, j))
L03160           end do
L03161         end do
L03162       end do
L03163
L03164     end subroutine Add_multi_perturb_mp_perturbations
```

```
!file: phys/module_microphysics_driver.F    L3166-L3195    (subroutine Remove_multi_perturb_mp_perturbations)
L03166     subroutine Remove_multi_perturb_mp_perturbations (perts_qvapor, perts_qcloud, perts_qice, &
L03167         perts_qsnow, perts_ni, pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, &
L03168         pert_thom_ni, qv_curr, qc_curr, qi_curr, qs_curr, qni_curr, qv_tmp, qc_tmp, qi_tmp, &
L03169         qs_tmp, qni_tmp, its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte)
L03170
L03171       implicit none
L03172
L03173       integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L03174       real, intent(in) :: pert_thom_qv, pert_thom_qc, pert_thom_qi, pert_thom_qs, pert_thom_ni
L03175       real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_qvapor, perts_qcloud, perts_qice, &
L03176           perts_qsnow, perts_ni
L03177       real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: qv_curr, qc_curr, qi_curr, qs_curr, qni_curr
L03178       real, dimension (its:ite, kts:kte, jts:jte), intent (in) :: qv_tmp, qc_tmp, qi_tmp, qs_tmp, qni_tmp
L03179
L03180       integer :: i, j, k
L03181
L03182
L03183       do j = jts, jte
L03184         do k = kts, kte
L03185           do i = its, ite
L03186             qv_curr(i, k, j) = max (QX_MIN, qv_curr(i, k, j) - perts_qvapor(i, k, j) * pert_thom_qv * qv_tmp(i, k, j))
L03187             qc_curr(i, k, j) = max (QX_MIN, qc_curr(i, k, j) - perts_qcloud(i, k, j) * pert_thom_qc * qc_tmp(i, k, j))
L03188             qi_curr(i, k, j) = max (QX_MIN, qi_curr(i, k, j) - perts_qice(i, k, j) * pert_thom_qi * qi_tmp(i, k, j))
L03189             qs_curr(i, k, j) = max (QX_MIN, qs_curr(i, k, j) - perts_qsnow(i, k, j) * pert_thom_qs * qs_tmp(i, k, j))
L03190             qni_curr(i, k, j) = max (NI_MIN, qni_curr(i, k, j) - perts_ni(i, k, j) * pert_thom_ni * qni_tmp(i, k, j))
L03191           end do
L03192         end do
L03193       end do
L03194
L03195     end subroutine Remove_multi_perturb_mp_perturbations
```

- **Add**：`q_curr ← max(QX_MIN, (1 + perts_field • pert_thom_qx) • q_curr)`——乘性扰动（`perts_*` 是标准化扰动场、`pert_thom_q*` 是幅度系数），备份到 `_tmp`；**Remove** 不是简单还原——`q_curr ← max(QX_MIN, q_curr − perts • pert • q_tmp)`：**减去的是“扰动场作用于备份值”的增量**，即把方案跑完后的场上“扰动贡献”剥离、保留方案的趋势。这是随机参数扰动（SPPT 家族）的正确数学形态——直接还原会把方案的物理趋势也一起扔掉。
- 两个子程序是本文件里唯二的“实际计算”——也不过是逐点乘法。**mediation layer 的“薄”是纪律而非偶然**。
- `QX_MIN=1e-12/NI_MIN=1e-6` 钳位（模块头那两个常数）保证扰动不产生负值/零下溢。

21.11 改造入口：加一个新微物理方案的完整清单

步骤	动哪里	对应本篇
1. 注册方案号	`frame/module_state_description.F` 加 `NEWSCHME=n` 常量	§21.3
2. Registry 包与字段	`Registry/Registry.EM_COMMON` 加 package/state（touch Registry/Registry.EM 触发重建）	教材 ch04/ch20
3. USE 方案模块	本文件 `add new modules of schemes here` 两处	§21.3
4. 签名扩容（若新字段）	L9–L175 签名 + 声明段 + `f_*` 标志	§21.5
5. 写 CASE	`micro_select` 内加 `CASE (NEWSCHEME)`：PRESENT 墙 + CALL 装配	§21.7/§21.8
6. namelist 口径	`run/README.namelist` 的 `mp_physics` 表	教材 ch13 §13.6.4
7. 初始化	`module_first_rk_step_part1.F` 的 `MP_INIT` 分发加 init 调用	文库 code_31
8. 兼容性检查	与 PBL/辐射/化学的接口矩阵（re_*/气溶胶）	§21.6/教材 ch12–13

教材 ch20 的五个实测案例（T2 峰值修正、虚拟温度阈值、BEM 反照率）走的就是这条链的子集——本表是它们的“通用化”。

21.12 上下游链接

- **上游**：code_31——`module_first_rk_step_part1.F`（本 driver 的调用方，`dt`/`config_flags` 的装配现场）；code_50——`solve_em.F` 导航页（`moist_physics_prep_em` 链条）。
- **下游**：code_07a/07b——Thompson 三 CASE 的实际目的地；其余方案模块不在文库收录范围（README 取舍说明）。
- **教材**：ch04（Registry）、ch13 §13.6（分发与方案号）、ch20（改造案例）、ch21（solve_em 调用位与 RK 节拍）。

code_22 — PBL 驱动器（module_pbl_driver.F 全 CASE 走读）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_pbl_driver.F	2887	1c4ca2a3bd7baa4c31b48527e60a4cad

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5 可逐块复核。
- 本篇 B 级「全 CASE 走读」：pbl_driver 主入口的参数清单（L11–L828，纯哑元声明）不收录——各方案的完整实参表就是每个 CASE 分支里的 CALL 语句本身；其余执行路径全收。

二、定位

一句话：五大 physics driver 之一——把 bl_pbl_physics 的整型开关翻译成对具体 PBL 方案子程序的调用，自己不含一行业务物理。

谁在调它：动力侧 `dyn_em/module_first_rk_step_part1.F` 在每个时间步的 RK 循环开头调用 pbl_driver（物理调用链：surface driver → pbl driver，见 ch21_solve_em_dynamics.md 的非时间分裂物理节拍）；本篇的直接下游是 14 个 PBL 方案入口——MYNN-EDMF 的方案实体已在 `code_09a_module_bl_mynnedmf.md`、`code_09b_module_bl_mynnedmf.md` 全文精读，面层侧对应 `code_10_module_sf_mynnsfc.md`。

Registry 方案号对照（bl_pbl_physics，实测自 Registry.EM_COMMON package 行）：

值	package	方案	Registry 挂载的 state
0	—	关闭	—
1	ysuscheme	YSU	—
2	myjpblscheme	MYJ（TKE 闭合 1.5 阶）	tke_pbl, el_pbl
3	gfsscheme	GFS（旧版局部扩散）	—
4	qnsepblscheme	QNSE	tke_pbl, el_pbl, massflux_EDKF 等 13 项
5	mynnpblscheme	MYNN-EDMF（2.5/3 共用入口）	qke, tsq, qsq, cov, cldfra_bl, el_pbl 等 + scalar qke_adv
7	acmpblscheme	ACM2	—
8	boulacscheme	BouLac（论文 2 边界层首选）	el_pbl, tke_pbl, wu/wv/wt/wq_tur
9	camuwpblscheme	CAMUW	tauresx2d 等 9 项
10	temfpblscheme	TEMF	te_temf 等 17 项
11	shinhongscheme	ShinHong（YSU 扩展）	el_pbl, tke_pbl
12	gbmpblscheme	GBM	exch_tke, el_pbl, tke_pbl
16	eepsscheme	EEPS（集合型 PBL）	pek_pbl, pep_pbl + scalar pek_adv/pep_adv
17	kepsscheme	k-ε（BEP 配套）	tpe/diss/tke_pbl, pr_pbl, w*_tur 等
99	mrfscheme	MRF（遗留）	—

警示：切换 bl_pbl_physics 到一个「挂载 state 不同」的方案（如 1→5），wrfout 输入历史里没有 qke/tsq/cov 等新 state——必须从 WPS 重跑 real 重新生成 wrfinput（ch08_running_wrf.md）；namelist 改了但 state 缺失会在初始化时报 Registry 缺变量。源码注释里的方案编号表（本文件 L223–240）把 TEMF 写成 10、keps 写成 17，与 Registry 一致；「mynnpbl2/3」是历史名称，4.8.0 里只有一个 MYNNPBLSCHEME（=5）。

三、主入口 pbl_driver

3.1 USE 清单与方案点名册

```
!file: phys/module_pbl_driver.F    L166-L242
L00166  !-------------------------------------------------------------------
L00167  #if (EM_CORE==1)
L00168     USE module_state_description, ONLY :                          &
L00169                     YSUSCHEME,MRFSCHEME,GFSSCHEME,MYJPBLSCHEME,ACMPBLSCHEME,&
L00170                     QNSEPBLSCHEME,MYNNPBLSCHEME,BOULACSCHEME,         &
L00171                     CAMUWPBLSCHEME,BEPSCHEME,BEP_BEMSCHEME,MYJSFCSCHEME, &
L00172                     FITCHSCHEME,SHINHONGSCHEME,                       &
L00173                     TEMFPBLSCHEME,GBMPBLSCHEME,EEPSSCHEME,KEPSSCHEME,                  &
L00174                     MAVSCHEME,  & ! Yulong add for WLM
L00175                     CAMMGMPSCHEME,p_qi,p_qni,p_qnc,param_first_scalar,& !CAMMGMPSCHEME, p_qni,p_qnc is used for camuwpbl
↪  scheme
L00176                     p_qnwfa,p_qnifa,p_qnbca
L00177  #if ( WRFPLUS == 1 )
L00178     USE module_state_description, ONLY : SURFDRAGSCHEME
L00179  #endif
L00180  #else
L00181     USE module_state_description, ONLY :                          &
L00182                     YSUSCHEME,MRFSCHEME,GFSSCHEME,MYJPBLSCHEME,ACMPBLSCHEME &
L00183                     , QNSEPBLSCHEME, p_qi,param_first_scalar &
L00184                     , TEMFPBLSCHEME, GFSEDMFSCHEME           &
L00185                     , CAMUWPBLSCHEME                                  &
L00186                     , FITCHSCHEME, SHINHONGSCHEME                     &
L00187                     , MAVSCHEME  ! Yulong add for WLM
L00188                     , GBMPBLSCHEME, MYJSFCSCHEME
L00189  #endif
L00190
L00191     USE module_model_constants
L00192
L00193  ! *** add new modules of schemes here
L00194
L00195     USE module_bl_myjpbl
L00196     USE module_bl_qnsepbl
L00197     USE module_bl_ysu
L00198     USE module_bl_shinhong
L00199     USE module_bl_mrf
L00200     USE module_bl_gfs
L00201     USE module_bl_acm
L00202     USE module_bl_gwdo
L00203     USE module_bl_gwdo_gsl
L00204     USE module_bl_myjurb
L00205     USE module_bl_boulac
L00206  #if ( WRFPLUS == 1 )
L00207     USE module_bl_surface_drag
L00208  #endif
L00209     USE module_bl_camuwpbl_driver, only:CAMUWPBL
L00210     USE module_bl_temf
L00211     USE module_bl_mfshconvpbl
L00212     USE module_bl_gbmpbl
L00213  #if (EM_CORE==1)
L00214     USE module_bl_mynnedmf_driver
L00215     USE module_bl_eeps
L00216     USE module_bl_keps
L00217     USE module_bl_fogdes
L00218     USE module_wind_fitch
L00219     USE module_wind_mav ! Yulong add for WLM
L00220  #endif
L00221     use module_ra_gfdleta, only: cal_mon_day
L00222
L00223     !  This driver calls subroutines for the PBL parameterizations.
L00224     !
L00225     !  pbl scheme:
L00226     !      1. ysupbl
L00227     !      2. myjpbl
L00228     !      4. qnsepbl
L00229     !      5. mynnpbl2
L00230     !      6. mynnpbl3
L00231     !      7. acmpbl
L00232     !      8. boulacpbl
L00233     !      9. camuwpbl
L00234     !      10. temfpbl
L00235     !      11. shinhongpbl
L00236     !      17. kepsstpe
L00237     !      98. surface_drag
L00238     !      99. mrfpbl
L00239     !      12. gbmpbl
L00240     !
L00241  !-------------------------------------------------------------------
L00242     IMPLICIT NONE
```

讲解：① 方案常量全部来自 module_state_description——它们是 Registry package 语句生成的整型符号，不是硬编码数字（ch04 的机制）；②EM 核下 USE 了一张扩展名单：mynnedmf_driver、eeps、keps、fogdes、wind_fitch（风机）、wind_mav（风机变体）；③ 紧随其后的网格结构注释（kms 在底、kme 在顶、半层/整层交错）是所有 physics driver 共用的约定声明——方案作者最容易踩的 k 方向坑。

3.2 局部变量与方案前置旗标

```
!file: phys/module_pbl_driver.F    L844-L907
L00844
L00845     REAL    :: DTMIN,DTBL
L00846   !
L00847     INTEGER :: initflag
L00848   !
L00849     INTEGER :: i,J,K,NK,jj,ij,its,ite,jts,jte
L00850     LOGICAL :: radiation
L00851     LOGICAL :: flag_bep
L00852     LOGICAL :: flag_myjsfc
L00853     LOGICAL :: flag_qv, flag_qc, flag_qr, flag_qi, flag_qs, flag_qg, &
L00854                flag_qnc,flag_qni, flag_qnwfa, flag_qnifa, flag_qnbca
L00855     CHARACTER*256 :: message
L00856     REAL    :: next_bl_time
L00857     LOGICAL :: run_param , doing_adapt_dt , decided
L00858     LOGICAL :: do_adapt
L00859     integer iu_bep,iurb,idiff
L00860     real seamask,thsk,zzz,unew,vnew,tnew,qnew,umom,vmom
L00861     REAL :: z0,z1,z2,w1,w2
L00862     REAL, DIMENSION( ims:ime, kms:kme, jms:jme ) :: TKE_windfarm ! Yulong add for WLM
L00863     INTEGER :: ihour, jmonth, jday
L00864   !
L00865   ! FASDAS
L00866   !
L00867     REAL, DIMENSION( ims:ime, jms:jme ) , INTENT(  OUT) , OPTIONAL ::   QNORM
L00868     INTEGER, INTENT(IN   )                                          ::   fasdas
L00869     REAL                                                            ::   scrp1
L00870   !
L00871   ! END FASDAS
L00872   !
L00873   ! To accommodate shared physics
L00874     character*256 :: errmsg
L00875     integer :: errflg
L00876   !
L00877   !-----------------------------------------------------------------
L00878   !
L00879   !!!!!!!if using BEP set flag_bep to true
L00880       SELECT CASE(sf_urban_physics)
L00881         CASE (BEPSCHEME)
L00882           flag_bep=.true.
L00883         CASE (BEP_BEMSCHEME)
L00884           flag_bep=.true.
L00885         CASE DEFAULT
L00886           flag_bep=.false.
L00887       END SELECT
L00888
L00889       SELECT CASE(sf_sfclay_physics)
L00890         CASE (MYJSFCSCHEME)
L00891            flag_myjsfc=.true.
L00892         CASE DEFAULT
L00893            flag_myjsfc=.false.
L00894       END SELECT
L00895   !
L00896     flag_qv = .FALSE. ; IF ( PRESENT( F_QV ) ) flag_qv = F_QV
L00897     flag_qc = .FALSE. ; IF ( PRESENT( F_QC ) ) flag_qc = F_QC
L00898     flag_qr = .FALSE. ; IF ( PRESENT( F_QR ) ) flag_qr = F_QR
L00899     flag_qi = .FALSE. ; IF ( PRESENT( F_QI ) ) flag_qi = F_QI
L00900     flag_qs = .FALSE. ; IF ( PRESENT( F_QS ) ) flag_qs = F_QS
L00901     flag_qg = .FALSE. ; IF ( PRESENT( F_QG ) ) flag_qg = F_QG
L00902     flag_qnc = .FALSE. ; IF ( PRESENT( F_QNC ) ) flag_qnc = F_QNC !Used in CAMUWPBL
L00903     flag_qni = .FALSE. ; IF ( PRESENT( F_QNI ) ) flag_qni = F_QNI !Used in CAMUWPBL
L00904     flag_qnwfa = .FALSE. ; IF ( PRESENT( F_QNWFA ) ) flag_qnwfa = F_QNWFA
L00905     flag_qnifa = .FALSE. ; IF ( PRESENT( F_QNIFA ) ) flag_qnifa = F_QNIFA
L00906     flag_qnbca = .FALSE. ; IF ( PRESENT( F_QNBCA ) ) flag_qnbca = F_QNBCA
L00907
```

讲解：①flag_bep 由 sf_urban_physics∈{BEPSCHEME, BEP_BEMSCHEME} 决定——BEP 感知是「城市冠层方案是否在场」的唯一判据，与 bl_pbl_physics 无关（MYJ、BouLac、ShinHong、KEPS 都会被传入 a_*_bep 族数组）；②flag_myjsfc 只影响 MYJ 配套面层的 ck/cd 诊断回传；③flag_qv…flag_qnbca 是水汽/云水/冰/气溶胶数浓度等示踪物的「在场」标志——WRF 变量按需实例化（param_first_scalar 机制），方案必须先探测再写入；④bl_pbl_physics=0 直接 return。

3.3 触发节拍与 PBL 时间步

```
!file: phys/module_pbl_driver.F    L909-L1035
L00909
L00910   ! RAINBL in mm (Accumulation between PBL calls)
L00911   !
L00912     doing_adapt_dt = .FALSE.
L00913     IF ( PRESENT(adapt_step_flag) ) THEN
L00914        IF ( adapt_step_flag ) THEN
L00915           doing_adapt_dt = .TRUE.
L00916           IF ( bldtacttime .eq. 0. ) THEN
L00917              bldtacttime = CURR_SECS + bldt*60.
L00918           END IF
L00919        END IF
L00920     END IF
L00921
L00922   !  Do we run through this scheme or not?
L00923
L00924   !    Test 1:  If this is the initial model time, then yes.
L00925   !             ITIMESTEP=1
L00926   !    Test 2:  If the user asked for the pbl to be run every time step, then yes.
L00927   !             BLDT=0 or STEPBL=1
L00928   !    Test 3:  If not adaptive dt, and this is on the requested pbl frequency, then yes.
L00929   !             MOD(ITIMESTEP,STEPBL)=0
L00930   !    Test 4:  If using adaptive dt and the current time is past the last requested activate pbl time, then yes.
```

```
L00931  !               CURR_SECS >= BLDTACTTIME
L00932
L00933  !  If we do run through the scheme, we set the flag run_param to TRUE and we set the decided flag
L00934  !  to TRUE.  The decided flag says that one of these tests was able to say "yes", run the scheme.
L00935  !  We only proceed to other tests if the previous tests all have left decided as FALSE.
L00936
L00937  !  If we set run_param to TRUE and this is adaptive time stepping, we set the time to the next
L00938  !  pbl run.
L00939
L00940     run_param = .FALSE.
L00941     decided = .FALSE.
L00942     IF ( ( .NOT. decided ) .AND. &
L00943          ( itimestep .EQ. 1 ) ) THEN
L00944        run_param   = .TRUE.
L00945        decided     = .TRUE.
L00946     END IF
L00947
L00948     IF ( PRESENT(bldt) )THEN
L00949        IF ( ( .NOT. decided ) .AND. &
L00950             ( ( bldt .EQ. 0. ) .OR. ( stepbl .EQ. 1 ) ) ) THEN
L00951           run_param   = .TRUE.
L00952           decided     = .TRUE.
L00953        END IF
L00954     ELSE
L00955        IF ( ( .NOT. decided ) .AND. &
L00956                                     ( stepbl .EQ. 1 )   ) THEN
L00957           run_param   = .TRUE.
L00958           decided     = .TRUE.
L00959        END IF
L00960     END IF
L00961
L00962     IF ( ( .NOT. decided ) .AND. &
L00963          ( .NOT. doing_adapt_dt ) .AND. &
L00964          ( MOD(itimestep,stepbl) .EQ. 0 ) ) THEN
L00965        run_param   = .TRUE.
L00966        decided     = .TRUE.
L00967     END IF
L00968
L00969     IF ( ( .NOT. decided ) .AND. &
L00970          ( doing_adapt_dt ) .AND. &
L00971          ( curr_secs .GE. bldtacttime ) ) THEN
L00972        run_param   = .TRUE.
L00973        decided     = .TRUE.
L00974        bldtacttime = curr_secs + bldt*60
L00975     END IF
L00976
L00977
L00978    IF (run_param) THEN
L00979     radiation = .false.
L00980     IF (ra_lw_physics .gt. 0) radiation = .true.
L00981
L00982  !----
L00983  ! CALCULATE CONSTANT
L00984
L00985     DTMIN=DT/60.
L00986  ! PBL schemes need PBL time step for updates
L00987
L00988      if (PRESENT(adapt_step_flag)) then
L00989         if (adapt_step_flag) then
L00990            do_adapt = .TRUE.
L00991         else
L00992            do_adapt = .FALSE.
L00993         endif
L00994      else
L00995         do_adapt = .FALSE.
L00996      endif
L00997
L00998     if (PRESENT(BLDT)) then
L00999        if (bldt .eq. 0) then
L01000           DTBL = dt
L01001        ELSE
L01002           if (do_adapt) then
L01003              IF ( curr_secs .LT. 2. * dt ) THEN
L01004                 call wrf_message("WARNING: When using an adaptive time-step the boundary layer"// &
L01005                                  " time-step should be 0 (i.e., equivalent to model time-step).  ")
L01006                 call wrf_message("In order to proceed, for boundary layer calculations, the "// &
L01007                                  "boundary layer time-step "// &
L01008                                  "will be rounded to the nearest minute," )
L01009                 call wrf_message("possibly resulting in innacurate results.")
L01010              END IF
L01011              DTBL=bldt*60
L01012           else
L01013              DTBL=DT*STEPBL
L01014           endif
L01015        endif
L01016     else
L01017        DTBL=DT*STEPBL
L01018     endif
L01019
L01020  !!Alberto. If idiff=1, then compute the tendencies at the end in the routine diff3d
L01021  !!Alberto. If idiff=0, then compute the tendencies in each PBL routine (standard version).
L01022         idiff=0
L01023
L01024     IF ( idiff .EQ. 1 ) THEN
L01025       ALLOCATE (a_u(ims:ime,kms:kme,jms:jme))       ! Implicit component for the momemtum in X-direction
L01026       ALLOCATE (a_v(ims:ime,kms:kme,jms:jme))       ! Implicit component for the momemtum in Y-direction
L01027       ALLOCATE (a_t(ims:ime,kms:kme,jms:jme))       ! Implicit component for the Pot. Temp.
L01028       ALLOCATE (a_q(ims:ime,kms:kme,jms:jme))       ! Implicit component for the water vapor
L01029       ALLOCATE (b_u(ims:ime,kms:kme,jms:jme))       ! Explicit component for the momemtum in X-direction
L01030       ALLOCATE (b_v(ims:ime,kms:kme,jms:jme))       ! Explicit component for the momemtum in Y-direction
L01031       ALLOCATE (b_t(ims:ime,kms:kme,jms:jme))       ! Explicit component for the Pot. Temp.
L01032       ALLOCATE (b_q(ims:ime,kms:kme,jms:jme))       ! Explicit component for the water vapor
L01033       ALLOCATE (sf(ims:ime,kms:kme,jms:jme) )       ! surfaces
L01034       ALLOCATE (vl(ims:ime,kms:kme,jms:jme) )       ! volumes
L01035     ENDIF
```

讲解：① 四个测试依序决定本步是否运行 PBL：首步必跑；bldt=0（随每步）或 stepbl=1；非自适应步长下 mod(itimestep, stepbl)=0；自适应步长下 curr_secs≥bldtacttime（到点后把下次触发时间顺延

bldt 分钟）。bldt 单位是分钟——PBL 允许隔步运行以省算力，与辐射的 bldt 机制同构；②DTBL：bldt=0 时取模式步长，自适应时直接取 bldt • 60（并警告精度损失），否则 DT • STEPBL——方案内部的积分时长与模式时间步解耦；③idiff 在 L1021 被无条件写 0（Alberto 注释保留）：idiff=1 的「driver 统一隐式扩散」路径目前是休眠代码，详见 3.6。

3.4 状态保存、倾向清零与 a/b/sf/vl 预备

```
!file: phys/module_pbl_driver.F    L1036-L1171
L01036
L01037  ! SAVE OLD VALUES
L01038
L01039     !$OMP PARALLEL DO   &
L01040     !$OMP PRIVATE ( ij,i,j,k )
L01041
L01042     DO ij = 1 , num_tiles
L01043        DO j=j_start(ij),j_end(ij)
L01044        DO i=i_start(ij),i_end(ij)
L01045           TSKOLD(i,j)=TSK(i,j)
L01046           USTOLD(i,j)=UST(i,j)
L01047           ZNTOLD(i,j)=ZNT(i,j)
L01048           HPBL_HOLD(i,j)=PBLH(i,j)
L01049
L01050           DO k=kts,kte
L01051              v_phytmp(i,k,j)=v_phy(i,k,j)+v_frame
L01052              u_phytmp(i,k,j)=u_phy(i,k,j)+u_frame
L01053           ENDDO
L01054
L01055  ! PSFC : in Pa
L01056
L01057           PSFC(I,J)=p8w(I,kms,J)
L01058
L01059           DO k=kts,min(kte+1,kde)
L01060              RTHBLTEN(I,K,J)=0.
L01061              RUBLTEN(I,K,J)=0.
L01062              RVBLTEN(I,K,J)=0.
L01063              IF ( PRESENT( RQCBLTEN )) RQCBLTEN(I,K,J)=0.
L01064              IF ( PRESENT( RQVBLTEN )) RQVBLTEN(I,K,J)=0.
L01065           ENDDO
L01066
L01067           IF (flag_QI .AND. PRESENT(RQIBLTEN) ) THEN
L01068              DO k=kts,min(kte+1,kde)
L01069                 RQIBLTEN(I,K,J)=0.
L01070              ENDDO
L01071           ENDIF
L01072           !Following if condition is added for CAMUWPBL scheme
L01073           IF (flag_QNI .AND. PRESENT(RQNIBLTEN) ) THEN
L01074              DO k=kts,min(kte+1,kde)
L01075                 RQNIBLTEN(I,K,J)=0.
L01076              ENDDO
L01077           ENDIF
L01078        ENDDO
L01079        ENDDO
L01080
L01081        IF (bl_pbl_physics .EQ. QNSEPBLSCHEME ) THEN
L01082  ! use u_phytmp instead of wthv_mf, and v_phytmp instead of lm_bl89
L01083             DO j=j_start(ij),j_end(ij)
L01084             DO i=i_start(ij),i_end(ij)
L01085             DO k=kms,kme
L01086                u_phytmp(i,k,j)=0.
L01087                v_phytmp(i,k,j)=0.
L01088             ENDDO
L01089             ENDDO
L01090             ENDDO
L01091        ENDIF
L01092
L01093        IF ( idiff.eq.1 ) THEN
L01094  !Alberto
L01095  ! Here we should put a switch:
L01096  ! if we do not use BEP, just initialize the values of the a, and b to zero, except for the lowest model level,
L01097  ! where the heat and momentum fluxes are introduced
L01098  ! if we use BEP, use the values computed by BEP weigthed by the urban fraction.
L01099  ! for the moment I put a simple "if" but it should be changed to a more elegant way after(using the CASE, maybe?)
L01100  !!!!!!!!!!!!!!
L01101  ! This is the more general way to go, however some of these variables (e. g. a_t, a_q) may be zero nearly all time.
L01102  ! if we need to be as tight as possible for the memory we can think how to reduce the storage.
L01103  !!!!!!!!!!!!!!!!!
L01104  ! This stuff is becoming large, probably better to put in a subroutine
L01105  !!!!!!!!!!!!!!!!!!!
L01106  ! preparing the a, b, sf, and vl terms
L01107
L01108           IF (flag_bep) THEN
L01109             do j=j_start(ij),j_end(ij)
L01110             do i=i_start(ij),i_end(ij)
L01111             do k=kts,kte
L01112               a_u(i,k,j)=a_u_bep(i,k,j)
L01113               a_v(i,k,j)=a_v_bep(i,k,j)
L01114               a_t(i,k,j)=a_t_bep(i,k,j)
L01115               a_q(i,k,j)=a_q_bep(i,k,j)
L01116               b_u(i,k,j)=b_u_bep(i,k,j)
L01117               b_v(i,k,j)=b_v_bep(i,k,j)
L01118               b_t(i,k,j)=b_t_bep(i,k,j)
L01119               b_q(i,k,j)=b_q_bep(i,k,j)
L01120               vl(i,k,j)=vl_bep(i,k,j)
L01121               sf(i,k,j)=sf_bep(i,k,j)
L01122             enddo
L01123             sf(i,kte+1,j)=1.
L01124             enddo
L01125             enddo
L01126           ELSE
L01127             do j=j_start(ij),j_end(ij)
L01128             do i=i_start(ij),i_end(ij)
L01129             do k=kts,kte
```

```
L01130              a_u(i,k,j)=0.
L01131              a_v(i,k,j)=0.
L01132              a_t(i,k,j)=0.
L01133              a_q(i,k,j)=0.
L01134              b_u(i,k,j)=0.
L01135              b_v(i,k,j)=0.
L01136              b_t(i,k,j)=0.
L01137              b_q(i,k,j)=0.
L01138              vl(i,k,j)=1.
L01139              sf(i,k,j)=1.
L01140            enddo
L01141            sf(i,kte+1,j)=1.
L01142  ! fix the values for the surface fluxes (source/sink terms)
L01143  ! here for momentum the resolution is done implicitely
L01144  ! while for heat and moisture is done explicitely
L01145              a_u(i,1,j)=(-ust(I,J)*ust(I,J))/dz8w(i,1,j)/((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)
L01146              a_v(i,1,j)=(-ust(I,J)*ust(I,J))/dz8w(i,1,j)/((u_phy(i,1,j)**2+v_phy(i,1,j)**2.)**.5)
L01147              b_t(i,1,j)=hfx(i,j)/rho(i,1,j)/CP/dz8w(i,1,j)
L01148              b_q(i,1,j)=qfx(i,j)/rho(i,1,j)/dz8w(i,1,j)
L01149  !! if you want to solve also the heat and moisture fluxes implicitely, uncomment the following lines.
L01150  !! (of course you will need the values of thz0,qz0,akhs).
L01151  !             b_t(i,1,j)=akhs(i,j)*(thz0(I,J))/dz8w(i,1,j)
L01152  !             b_q(i,1,j)=akhs(i,j)*(qz0(I,J))/dz8w(i,1,j)
L01153  !             a_t(i,1,j)=-1.*akhs(i,j)/dz8w(i,1,j)
L01154  !             a_q(i,1,j)=-1.*akhs(i,j)/dz8w(i,1,j)
L01155  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L01156            enddo
L01157            enddo
L01158
L01159          ENDIF
L01160
L01161         endif
L01162
L01163
L01164
L01165  !Alberto. From this point if some PBL scheme has a non local term
L01166  ! (not dependent on the local values of the variable)
L01167  ! this can be added to b_t, b_q, or b_u, b_v.
L01168  !!!!!!!!!!!!!!!!!!
L01169
L01170     ENDDO
L01171     !$OMP END PARALLEL DO
```

讲解：①TSKOLD/USTOLD/ZNTOLD/HPBL_HOLD 存旧值——面层方案在 PBL 之后用这些差分更新诊断量；②u_phytmp=u_phy+v_frame——**移动嵌套参考系**：所有方案拿到的是加上框架速度的风场（嵌套随大域风平移时物理风需要还原）；QNSE 方案把这两个 tmp 数组另用作 WTHV_MF/LM_BL89 传送带（L1081–1091 特意清零），是驱动层「借用」数组的罕见案例；③ 倾向数组清零到 kte+1；④idiff=1 时的 a/b/sf/vl 预备：BEP 在场时把 a_u_bep 等建筑拖曳系数搬进来，否则只有最低层放地表通量（a_u=−u*²/(dz • |V|) 动量隐式源、b_t=HFX/(ρ • cp • dz) 显式热源）——这套通用化设计（Martilli 2008）服务 BEP 的「建筑体积内源」需求。

3.5 方案分派 CASE 全走读

tile 循环开启，SELECT CASE(bl_pbl_physics)：

```
!file: phys/module_pbl_driver.F    L1173-L1231
L01173    !$OMP PARALLEL DO   &
L01174    !$OMP PRIVATE ( ij, i,j,k, its, ite, jts, jte, z0, z1, z2, w1, w2, message, initflag )
L01175    DO ij = 1 , num_tiles
L01176
L01177     its = i_start(ij)
L01178     ite = i_end(ij)
L01179     jts = j_start(ij)
L01180     jte = j_end(ij)
L01181
L01182     pbl_select: SELECT CASE(bl_pbl_physics)
L01183
L01184         CASE (TEMFPBLSCHEME)
L01185  ! WA Test
L01186          ! WRITE( message , * ) 'Calling TEMF PBL scheme, timestep = ', itimestep
L01187          ! CALL wrf_message ( message )
L01188          ! print *,'Calling TEMF PBL scheme'
L01189           CALL wrf_debug(100,'in TEMF PBL')
L01190              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01191                   PRESENT( qi_curr )                            .AND. &
L01192                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01193                   PRESENT( rqiblten )                           .AND. &
L01194                   PRESENT( te_temf ) .AND. PRESENT( cfm_temf ) .AND. &
L01195                   PRESENT( hol      ) ) THEN
L01196                CALL temfpbl(                                          &
L01197                  U3D=u_phytmp,V3D=v_phytmp,TH3D=th_phy,T3D=t_phy      &
L01198                 ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr               &
L01199                 ,P3D=p_phy,P3DI=p8w,PI3D=pi_phy,RHO=rho               &
L01200                 ,RUBLTEN=rublten,RVBLTEN=rvblten                      &
L01201                 ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                  &
L01202                 ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten                  &
L01203                 ,FLAG_QI=flag_qi                                      &
L01204                 ,g=g,cp=cp,rcp=rcp,r_d=r_d,r_v=r_v,cpv=cpv                    &
L01205                 ,Z=z,XLV=XLV,PSFC=PSFC                 &
L01206                 ,P_TOP=p_top                                          &
L01207                 ,ZNT=znt,HT=ht,UST=ust,ZOL=zol,HOL=hol,HPBL=pblh      &
L01208                 ,PSIM=psim,PSIH=psih,XLAND=xland                      &
L01209                 ,HFX=hfx,QFX=qfx,TSK=tskold,QSFC=qsfc,GZ1OZ0=gz1oz0   &
L01210                 ,U10=u10,V10=v10,T2=t2                                &
L01211                 ,WSPD=wspd,BR=br,DT=dtbl,DTMIN=dtmin,KPBL2D=kpbl      &
L01212                 ,SVP1=svp1,SVP2=svp2,SVP3=svp3,SVPT0=svpt0            &
L01213                 ,EP1=ep_1,EP2=ep_2,KARMAN=karman,EOMEG=eomeg          &
```

```
L01214                ,STBOLT=stbolt                                          &
L01215                ,te_temf=te_temf,kh_temf=kh_temf,km_temf=km_temf        &
L01216                ,shf_temf=shf_temf,qf_temf=qf_temf                      &
L01217                ,uw_temf=uw_temf,vw_temf=vw_temf                        &
L01218                ,hd_temf=hd_temf,lcl_temf=lcl_temf,hct_temf=hct_temf    &
L01219                ,wupd_temf=wupd_temf,mf_temf=mf_temf                    &
L01220                ,thup_temf=thup_temf,qtup_temf=qtup_temf,qlup_temf=qlup_temf  &
L01221                ,cf3d_temf=cf3d_temf,cfm_temf=cfm_temf                  &
L01222                ,flhc=flhc,flqc=flqc,exch_temf=exch_temf                &
L01223                ,fCor=f                                                 &
L01224                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L01225                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L01226                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte        &
L01227                                                                        )
L01228             ELSE
L01229                CALL wrf_error_fatal('Lack arguments to call TEMF pbl')
L01230             ENDIF
L01231
```

讲解：OMP 并行 tile 循环——物理 tile 是 OpenMP 线程的并行单元（ch03 架构）。第一个 CASE 是 TEMF（总能量守恒型），PRESENT 守卫的写法是全篇模板：缺任一必需实参就把 PRESENT 逻辑值打成 message 再 fatal——新增方案时建议照抄这套自报家门式报错。

```
!file: phys/module_pbl_driver.F    L1232-L1316
L01232         CASE (YSUSCHEME)
L01233           CALL wrf_debug(100,'in YSU PBL')
L01234              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01235                   PRESENT( qi_curr )                            .AND. &
L01236                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01237                   PRESENT( rqiblten )                           .AND. &
L01238                   PRESENT( hol      ) ) THEN
L01239  !
L01240                CALL ysu(                                               &
L01241                  U3D=u_phytmp,V3D=v_phytmp,T3D=t_phy                   &
L01242                 ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr                &
L01243                 ,P3D=p_phy,P3DI=p8w,PI3D=pi_phy                        &
L01244                 ,RUBLTEN=rublten,RVBLTEN=rvblten                       &
L01245                 ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                   &
L01246                 ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten                   &
L01247                 ,FLAG_QI=flag_qi,FLAG_QC=flag_qc                       &
L01248                 ,CP=cp,G=g,ROVCP=rcp,RD=r_d,ROVG=rovg                  &
L01249                 ,DZ8W=dz8w,XLV=XLV,RV=r_v,PSFC=PSFC                    &
L01250                 ,ZNT=znt,UST=ust,HPBL=pblh                             &
L01251                 ,PSIM=fm,PSIH=fhh,XLAND=xland                          &
L01252                 ,HFX=hfx,QFX=qfx                                       &
L01253                 ,U10=u10,V10=v10                                       &
L01254                 ,UOCE=uoce,VOCE=voce                                   &
L01255  ! paj
L01256                 ,CTOPO=ctopo,CTOPO2=ctopo2                             &
L01257                 ,YSU_TOPDOWN_PBLMIX=ysu_topdown_pblmix                 &
L01258                 ,WSPD=wspd,BR=br,DT=dtbl,KPBL2D=kpbl                   &
L01259                 ,EP1=ep_1,EP2=ep_2,KARMAN=karman                       &
L01260                 ,EXCH_H=exch_h,EXCH_M=exch_m                           &
L01261                 ,RTHRATEN=RTHRATEN                                     &
L01262  ! for multilayer UCM
L01263                 ,IDIFF=idiff,FLAG_BEP=flag_bep,FRC_URB2D=frc_urb2d     &
L01264                 ,A_U_BEP=a_u_bep,A_V_BEP=a_v_bep,A_T_BEP=a_t_bep       &
L01265                 ,A_Q_BEP=a_q_bep                                       &
L01266                 ,A_E_BEP=a_e_bep,B_U_BEP=b_u_bep,B_V_BEP=b_v_bep       &
L01267                 ,B_T_BEP=b_t_bep,B_Q_BEP=b_q_bep                       &
L01268                 ,B_E_BEP=b_e_bep,DLG_BEP=dlg_bep                       &
L01269                 ,DL_U_BEP=dl_u_bep,SF_BEP=sf_bep,VL_BEP=vl_bep         &
L01270  ! for grims shallow convection with ysupbl
L01271                 ,WSTAR=wstar,DELTA=delta                               &
L01272                 ,errmsg=errmsg,errflg=errflg                           &
L01273                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
L01274                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme       &
L01275                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte       &
L01276                                                                       )
L01277  !
L01278  ! FASDAS
L01279  !
L01280                 IF( fasdas == 1 ) THEN
L01281                     DO j=j_start(ij),j_end(ij)
L01282                     DO i=i_start(ij),i_end(ij)
L01283                        scrp1 = rqvblten(i,1,j)
L01284                        scrp1 = scrp1*(2.0*DT)/qv_curr(i,1,j)
L01285                        scrp1 = scrp1*100.0
L01286                        scrp1 = amax1(0.0,scrp1)
L01287                        scrp1 = amin1(5.0,scrp1)
L01288                        QNORM(I,J)= scrp1/100.0
L01289                     ENDDO
L01290                     ENDDO
L01291                 ENDIF
L01292  !
L01293  ! END FASDAS
L01294  !
L01295
L01296              ELSE
L01297                  WRITE ( message , FMT = '(A,7(L1,1X))' )               &
L01298                    'present: '//                                        &
L01299                    'qv_curr, '//                                        &
L01300                    'qc_curr, '//                                        &
L01301                    'qi_curr, '//                                        &
L01302                    'rqvblten, '//                                       &
L01303                    'rqcblten, '//                                       &
L01304                    'rqiblten, '//                                       &
L01305                    'hol = ' ,                                           &
L01306                     PRESENT( qv_curr ) ,                                &
L01307                     PRESENT( qc_curr ) ,                                &
L01308                     PRESENT( qi_curr ) ,                                &
L01309                     PRESENT( rqvblten ) ,                               &
L01310                     PRESENT( rqcblten ) ,                               &
L01311                     PRESENT( rqiblten ) ,                               &
L01312                     PRESENT( hol      )
```

```
L01313               CALL wrf_debug(0,message)
L01314               CALL wrf_error_fatal('Lack arguments to call YSU pbl')
L01315            ENDIF
L01316
```

讲解：YSU（1 阶非局地 K 扩散 +夹卷稳定度修正）：传参注意 PSIM=fm、PSIH=fhh——面层阻抗在 pbl driver 的实参表里换了名（fm/fhh 是 surface driver 传来的相似性阻抗，不是 ψ 函数值本身）；尾接 FASDAS 同化段：用最低层水汽倾向归一出 QNORM∈[0,5%]（土壤湿度同化的归一化因子，钳位防溢出）。

```
!file: phys/module_pbl_driver.F    L1317-L1385
L01317         CASE (SHINHONGSCHEME)
L01318           CALL wrf_debug(100,'in SHINHONG PBL')
L01319              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01320                   PRESENT( qi_curr )                            .AND. &
L01321                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01322                   PRESENT( rqiblten )                           .AND. &
L01323                   PRESENT( hol      ) ) THEN
L01324  !
L01325                CALL shinhong(                                          &
L01326                  U3D=u_phytmp,V3D=v_phytmp,T3D=t_phy                   &
L01327                 ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr                &
L01328                 ,P3D=p_phy,P3DI=p8w,PI3D=pi_phy                        &
L01329                 ,RUBLTEN=rublten,RVBLTEN=rvblten                       &
L01330                 ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                   &
L01331                 ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten                   &
L01332                 ,FLAG_QI=flag_qi,FLAG_QC=flag_qc                       &
L01333                 ,CP=cp,G=g,ROVCP=rcp,RD=r_D,ROVG=rovg                  &
L01334                 ,DZ8W=dz8w,XLV=XLV,RV=r_v,PSFC=PSFC                    &
L01335                 ,ZNT=znt,UST=ust,HPBL=pblh                             &
L01336                 ,PSIM=fm,PSIH=fhh,XLAND=xland                          &
L01337                 ,HFX=hfx,QFX=qfx                                       &
L01338                 ,U10=u10,V10=v10                                       &
L01339                 ,UOCE=uoce,VOCE=voce                                   &
L01340  ! paj
L01341                 ,shinhong_nonlocal_flux=shinhong_nonlocal_flux         &
L01342                 ,shinhong_scu_mixing=shinhong_scu_mixing               &
L01343                 ,shinhong_dissi_heating=shinhong_ke_dissipation        &
L01344                 ,tke=tke_pbl,el=el_pbl,corf=f                          &
L01345                 ,CTOPO=ctopo,CTOPO2=ctopo2,dx=dx2d                     &
L01346                 ,WSPD=wspd,BR=br,DT=dtbl,KPBL2D=kpbl                   &
L01347                 ,EP1=ep_1,EP2=ep_2,KARMAN=karman                       &
L01348                 ,EXCH_H=exch_h,EXCH_M=exch_m                           &
L01349                 ,RTHRATEN=RTHRATEN                                     &
L01350  ! for multilayer UCM
L01351                 ,IDIFF=idiff,FLAG_BEP=flag_bep,FRC_URB2D=frc_urb2d     &
L01352                 ,A_U_BEP=a_u_bep,A_V_BEP=a_v_bep,A_T_BEP=a_t_bep       &
L01353                 ,A_Q_BEP=a_q_bep                                       &
L01354                 ,A_E_BEP=a_e_bep,B_U_BEP=b_u_bep,B_V_BEP=b_v_bep       &
L01355                 ,B_T_BEP=b_t_bep,B_Q_BEP=b_q_bep                       &
L01356                 ,B_E_BEP=b_e_bep,DLG_BEP=dlg_bep                       &
L01357                 ,DL_U_BEP=dl_u_bep,SF_BEP=sf_bep,VL_BEP=vl_bep         &
L01358  ! for grims shallow convection
L01359                 ,WSTAR=wstar,DELTA=delta                               &
L01360                 ,errmsg=errmsg,errflg=errflg                           &
L01361                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
L01362                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme       &
L01363                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte       &
L01364                                                                        )
L01365              ELSE
L01366                  WRITE ( message , FMT = '(A,7(L1,1X))' )               &
L01367                    'present: '//                                        &
L01368                    'qv_curr, '//                                        &
L01369                    'qc_curr, '//                                        &
L01370                    'qi_curr, '//                                        &
L01371                    'rqvblten, '//                                       &
L01372                    'rqcblten, '//                                       &
L01373                    'rqiblten, '//                                       &
L01374                    'hol = ' ,                                           &
L01375                     PRESENT( qv_curr ) ,                                &
L01376                     PRESENT( qc_curr ) ,                                &
L01377                     PRESENT( qi_curr ) ,                                &
L01378                     PRESENT( rqvblten ) ,                               &
L01379                     PRESENT( rqcblten ) ,                               &
L01380                     PRESENT( rqiblten ) ,                               &
L01381                     PRESENT( hol      )
L01382                  CALL wrf_debug(0,message)
L01383                  CALL wrf_error_fatal('Lack arguments to call SHINHONG pbl')
L01384              ENDIF
L01385
```

讲解：ShinHong（YSU 的 TKE 扩展）：比 YSU 多传 shinhong_nonlocal_flux/scu_mixing/ke_dissipation 三个 namelist 开关与 tke_pbl/el_pbl——它复用 YSU 骨架、增加 TKE 诊断。BEP 数组同样传入（多城市实验可配 ShinHong）。

```
!file: phys/module_pbl_driver.F    L1386-L1473
L01386         CASE (MRFSCHEME)
L01387              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01388                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01389                   PRESENT( hol      )                           .AND. &
L01390                                                          .TRUE.  ) THEN
L01391
L01392                CALL wrf_debug(100,'in MRF')
L01393                CALL mrf(                                               &
L01394                  U3D=u_phytmp,V3D=v_phytmp,TH3D=th_phy,T3D=t_phy       &
L01395                 ,QV3D=qv_curr                                          &
L01396                 ,QC3D=qc_curr                                          &
L01397                 ,QI3D=qi_curr                                          &
L01398                 ,P3D=p_phy,PI3D=pi_phy                                 &
```

```
L01399                ,RUBLTEN=rublten,RVBLTEN=rvblten                        &
L01400                ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                    &
L01401                ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten                    &
L01402                ,CP=cp,G=g,ROVCP=rcp,R=r_d,ROVG=rovg                    &
L01403                ,DZ8W=dz8w,Z=z,XLV=xlv,RV=r_v,PSFC=psfc                 &
L01404                ,P1000MB=p1000mb                                        &
L01405                ,ZNT=znt,UST=ust,ZOL=zol,HOL=hol                        &
L01406                ,PBL=pblh,PSIM=psim,PSIH=psih                           &
L01407                ,XLAND=xland,HFX=hfx,QFX=qfx,TSK=tskold                 &
L01408                ,GZ1OZ0=gz1oz0,WSPD=wspd,BR=br                          &
L01409                ,DT=dtbl,DTMIN=dtmin,KPBL2D=kpbl                        &
L01410                ,SVP1=svp1,SVP2=svp2,SVP3=svp3,SVPT0=svpt0              &
L01411                ,EP1=ep_1,EP2=ep_2,KARMAN=karman,EOMEG=eomeg            &
L01412                ,STBOLT=stbolt,REGIME=regime                            &
L01413                ,FLAG_QI=flag_qi                                        &
L01414                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L01415                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L01416                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte        &
L01417                                                                        )
L01418              ELSE
L01419                  WRITE ( message , FMT = '(A,5(L1,1X))' )              &
L01420                    'present: '//                                       &
L01421                    'qv_curr, '//                                       &
L01422                    'qc_curr, '//                                       &
L01423                    'rqvblten, '//                                      &
L01424                    'rqcblten, '//                                      &
L01425                    'hol = ' ,                                          &
L01426                     PRESENT( qv_curr ) ,                               &
L01427                     PRESENT( qc_curr ) ,                               &
L01428                     PRESENT( rqvblten ) ,                              &
L01429                     PRESENT( rqcblten ) ,                              &
L01430                     PRESENT( hol      )
L01431                  CALL wrf_debug(0,message)
L01432                  CALL wrf_error_fatal('Lack arguments to call MRF pbl')
L01433              ENDIF
L01434
L01435          CASE (GFSSCHEME)
L01436              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01437                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01438                                                        .TRUE.  ) THEN
L01439                CALL wrf_debug(100,'in GFS')
L01440                CALL bl_gfs(                                            &
L01441                  U3D=u_phytmp,V3D=v_phytmp                             &
L01442                 ,TH3D=th_phy,T3D=t_phy                                 &
L01443                 ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr                &
L01444                 ,P3D=p_phy,PI3D=pi_phy                                 &
L01445                 ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten     &
L01446                 ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                   &
L01447                 ,RQIBLTEN=rqiblten                                     &
L01448                 ,CP=cp,G=g,ROVCP=rcp,R=r_d,ROVG=rovg                   &
L01449                 ,DZ8W=dz8w,z=z,PSFC=psfc                               &
L01450                 ,UST=ust,PBL=pblh,PSIM=psim,PSIH=psih                  &
L01451                 ,HFX=hfx,QFX=qfx,TSK=tskold,GZ1OZ0=gz1oz0              &
L01452                 ,WSPD=wspd,BR=br                                       &
L01453                 ,DT=dtbl,KPBL2D=kpbl,EP1=ep_1,KARMAN=karman            &
L01454                 ,P_QI=p_qi,P_FIRST_SCALAR=param_first_scalar           &
L01455                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
L01456                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme       &
L01457                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte       &
L01458                                                                        )
L01459              ELSE
L01460                  WRITE ( message , FMT = '(A,4(L1,1X))' )              &
L01461                    'present: '//                                       &
L01462                    'qv_curr, '//                                       &
L01463                    'qc_curr, '//                                       &
L01464                    'rqvblten, '//                                      &
L01465                    'rqcblten = ',                                      &
L01466                     PRESENT( qv_curr ) ,                               &
L01467                     PRESENT( qc_curr ) ,                               &
L01468                     PRESENT( rqvblten ) ,                              &
L01469                     PRESENT( rqcblten )
L01470                  CALL wrf_debug(0,message)
L01471                  CALL wrf_error_fatal('Lack arguments to call GFS pbl')
L01472              ENDIF
L01473
```

讲解：MRF（99，遗留）与 GFS（3）：两者是 1.5 阶闭合普及前的经典局地扩散方案，实参表明显短——无 tke/el、无 BEP、无城市。GFS 分支只要求 qv/qc 两个倾向在场（连 hol 都不要）。这两支是「最小依赖」参照组。

```
!file: phys/module_pbl_driver.F    L1474-L1548
L01474          CASE (MYJPBLSCHEME)
L01475              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01476                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01477                                                        .TRUE.  ) THEN
L01478
L01479                CALL wrf_debug(100,'in MYJPBL')
L01480               IF ( .not.flag_bep .and. idiff.ne.1) THEN
L01481                CALL myjpbl(DT=dt,STEPBL=stepbl,HT=ht,DZ=dz8w           &
L01482                 ,PMID=p_phy,PINT=p8w,TH=th_phy,T=t_phy,EXNER=pi_phy    &
L01483                 ,QV=qv_curr,QCW=qc_curr,QCI=qi_curr,QCS=qs_curr        & !BSF
L01484                 ,QCR=qr_curr,QCG=qg_curr                               & !BSF
L01485                 ,U=u_phy,V=v_phy,RHO=rho                               &
L01486                 ,TSK=tsk,QSFC=qsfc,CHKLOWQ=chklowq,THZ0=thz0           &
L01487                 ,QZ0=qz0,UZ0=uz0,VZ0=vz0                               &
L01488                 ,LOWLYR=lowlyr                                         &
L01489                 ,XLAND=xland,SICE=xice,SNOW=snow                       &
L01490                 ,TKE_MYJ=tke_pbl,EXCH_H=exch_h,USTAR=ust,ZNT=znt       &
L01491                 ,EL_MYJ=el_pbl,PBLH=pblh,KPBL=kpbl,CT=ct               &
L01492                 ,AKHS=akhs,AKMS=akms,ELFLX=lh,MIXHT=mixht              &
L01493                 ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten     &
L01494                 ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                   &
L01495                 ,RQIBLTEN=rqiblten,RQSBLTEN=rqsblten                   & !BSF
L01496                 ,RQRBLTEN=rqrblten,RQGBLTEN=rqgblten                   & !BSF
L01497                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
```

```
L01498                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01499                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      &
L01500                                                                      )
L01501              ELSE !(SF_URBAN_PHYSICS.EQ.2)
L01502  ! Bep changes begin
L01503
L01504               CALL myjurb(IDIFF=idiff,FLAG_BEP=flag_bep              &
L01505                ,DT=dtbl,STEPBL=stepbl,HT=ht,DZ=dz8w                  &
L01506                ,PMID=p_phy,PINT=p8w,TH=th_phy,T=t_phy,EXNER=pi_phy   &
L01507                ,QV=qv_curr, CWM=qc_curr                              &
L01508                ,U=u_phy,V=v_phy,RHO=rho                              &
L01509                ,TSK=tsk,QSFC=qsfc,CHKLOWQ=chklowq,THZ0=thz0          &
L01510                ,QZ0=qz0,UZ0=uz0,VZ0=vz0                              &
L01511                ,LOWLYR=lowlyr                                        &
L01512                ,XLAND=xland,SICE=xice,SNOW=snow                      &
L01513                ,TKE_MYJ=tke_pbl,EXCH_H=exch_h,EXCH_M=exch_m          &
L01514                ,USTAR=ust,ZNT=znt                                    &
L01515                ,EL_MYJ=el_pbl,PBLH=pblh,KPBL=kpbl,CT=ct              &
L01516                ,AKHS=akhs,AKMS=akms,ELFLX=lh                         &
L01517                ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten    &
L01518                ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                  &
L01519  ! Multi-layer UCM
L01520                ,FRC_URB2D=frc_urb2d                                  &
L01521                ,A_U_BEP=a_u_bep,A_V_BEP=a_v_bep,A_T_BEP=a_t_bep      &
L01522                ,A_Q_BEP=a_q_bep                                      &
L01523                ,A_E_BEP=a_e_bep,B_U_BEP=b_u_bep,B_V_BEP=b_v_bep      &
L01524                ,B_T_BEP=b_t_bep,B_Q_BEP=b_q_bep                      &
L01525                ,B_E_BEP=b_e_bep,DLG_BEP=dlg_bep                      &
L01526                ,DL_U_BEP=dl_u_bep,SF_BEP=sf_bep,VL_BEP=vl_bep        &
L01527  ! UCM end
L01528                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde      &
L01529                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01530                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      &
L01531                                                                      )
L01532               ENDIF
L01533
L01534              ELSE
L01535                  WRITE ( message , FMT = '(A,4(L1,1X))' )            &
L01536                    'present: '//                                     &
L01537                    'qv_curr, '//                                     &
L01538                    'qc_curr, '//                                     &
L01539                    'rqvblten, '//                                    &
L01540                    'rqcblten = ' ,                                   &
L01541                     PRESENT( qv_curr ) ,                             &
L01542                     PRESENT( qc_curr ) ,                             &
L01543                     PRESENT( rqvblten ) ,                            &
L01544                     PRESENT( rqcblten )
L01545                  CALL wrf_debug(0,message)
L01546                  CALL wrf_error_fatal('Lack arguments to call MYJ pbl')
L01547              ENDIF
L01548
```

讲解：MYJ（2）：if(.not.flag_bep .and. idiff.ne.1) 走原版 myjpbl（注意它要 DT=dt 而非 DTBL——MYJ 自己管节拍），否则走 myjurb（多城市版本，BEP 数组全量传入）。MYJ 的 akhs/akms 在场——它是「面层-PBL 一体化」方案，交换系数由 PBL 侧算好后回传 surface driver（与 MYNN 的 CHS 分工相反）。

```
!file: phys/module_pbl_driver.F    L1549-L1618
L01549         CASE (QNSEPBLSCHEME)
L01550              IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01551                   PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01552                                                          .TRUE.  ) THEN
L01553                  IF ( MFSHCONV.EQ.1 )THEN
L01554                  CALL wrf_debug(100,'in MFSHCONVPBL')
L01555                  CALL mfshconvpbl(DT=dt,STEPBL=stepbl,HT=ht,DZ=dz8w          &
L01556                       ,RHO=rho,PMID=p_phy,PINT=p8w,TH=th_phy,EXNER=pi_phy    &
L01557                       ,QV=qv_curr, QC=qc_curr                        &
L01558                       ,U=u_phy,V=v_phy                                       &
L01559                       ,HFX=hfx, QFX=qfx,TKE=tke_pbl                          &
L01560                       ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten     &
L01561                       ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                   &
L01562                       ,IDS=IDS,IDE=IDE,JDS=JDS,JDE=JDE,KDS=KDS,KDE=KDE       &
L01563                       ,IMS=IMS,IME=IME,JMS=JMS,JME=JME,KMS=KMS,KME=KME       &
L01564                       ,ITS=ITS,ITE=ITE,JTS=JTS,JTE=JTE,KTS=KTS,KTE=KTE       &
L01565                       ,KRR=2,MASSFLUX_EDKF=massflux_EDKF                     &
L01566                       ,ENTR_EDKF=entr_EDKF, DETR_EDKF=detr_EDKF              &
L01567                       ,THL_UP=thl_up, THV_UP=thv_up, RT_UP=rt_up             &
L01568                       ,RV_UP=rv_up,RC_UP=rc_up, U_UP=u_up, V_UP=v_up         &
L01569                       ,FRAC_UP=frac_up, RC_MF=rc_mf,WTHV=u_phytmp            &
L01570                       ,PLM_BL89=v_phytmp   )
L01571                  ENDIF
L01572
L01573                CALL wrf_debug(100,'in QNSEPBL')
L01574                CALL qnsepbl(                                          &
L01575                  DT=dt,STEPBL=stepbl,HT=ht,DZ=dz8w                    &
L01576                 ,PMID=p_phy,PINT=p8w,TH=th_phy,T=t_phy,EXNER=pi_phy   &
L01577                 ,QV=qv_curr, CWM=qc_curr                              &
L01578                 ,U=u_phy,V=v_phy,RHO=rho                              &
L01579                 ,TSK=tsk,QSFC=qsfc,CHKLOWQ=chklowq,THZ0=thz0          &
L01580                 ,QZ0=qz0,UZ0=uz0,VZ0=vz0,CORF=f                       &
L01581                 ,LOWLYR=lowlyr                                        &
L01582                 ,XLAND=xland,SICE=xice,SNOW=snow                      &
L01583                 ,TKE=tke_pbl,EXCH_H=exch_h,EXCH_M=exch_m,USTAR=ust,ZNT=znt      &
L01584                 ,EL_MYJ=el_pbl,PBLH=pblh,KPBL=kpbl,CT=ct              &
L01585                 ,AKHS=akhs,AKMS=akms,ELFLX=lh                         &
L01586                 ,HFX=hfx,WTHV_MF=u_phytmp,LM_BL89=v_phytmp            &
L01587                 ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten    &
L01588                 ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                  &
L01589                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde      &
L01590                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01591                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      &
L01592                                                                       )
L01593  ! note the tendencies have been added inside qnse
L01594  !            IF ( PRESENT (MFSHCONV) )THEN
```

```
L01595  !              IF (MFSHCONV.EQ.1)THEN
L01596  !              rublten(:,:,:)=rublten(:,:,:)+rumften(:,:,:)
L01597  !              rvblten(:,:,:)=rvblten(:,:,:)+rvmften(:,:,:)
L01598  !              rthblten(:,:,:)=rthblten(:,:,:)+rthmften(:,:,:)
L01599  !              rqvblten(:,:,:)=rqvblten(:,:,:)+rqvmften(:,:,:)
L01600  !              rqcblten(:,:,:)=rqcblten(:,:,:)+rqcmften(:,:,:)
L01601  !              ENDIF
L01602  !            ENDIF
L01603
L01604              ELSE
L01605                 WRITE ( message , FMT = '(A,4(L1,1X))' )             &
L01606                   'present: '//                                      &
L01607                   'qv_curr, '//                                      &
L01608                   'qc_curr, '//                                      &
L01609                   'rqvblten, '//                                     &
L01610                   'rqcblten = ' ,                                    &
L01611                    PRESENT( qv_curr ) ,                              &
L01612                    PRESENT( qc_curr ) ,                              &
L01613                    PRESENT( rqvblten ) ,                             &
L01614                    PRESENT( rqcblten )
L01615                 CALL wrf_debug(0,message)
L01616                 CALL wrf_error_fatal('Lack arguments to call QNSE pbl')
L01617              ENDIF
L01618
```

讲解：QNSE（4）：先有 MFSHCONV=1 时的浅对流质量流支路 mfshconvpbl（EDKF 夹卷/卷出谱），再进 qnsepbl 主体；注释掉的 tendency 累加段说明「方案内部已加」。QNSE 是驱动层里唯一「一个 CASE 可发两次调用」的方案。

```
!file: phys/module_pbl_driver.F    L1619-L1660
L01619        CASE (ACMPBLSCHEME)
L01620
L01621             !!  These are values that are not supplied to pbl driver, but are required by ACM
L01622             IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01623                  PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01624                                                        .TRUE.  ) THEN
L01625               CALL wrf_debug(100,'in ACM PBL')
L01626
L01627               CALL ACMPBL(                                                 &
L01628                 XTIME=itimestep, DTPBL=dtbl,U3D=u_phytmp, V3D=v_phytmp          &
L01629                ,PP3D=p_phy, DZ8W=dz8w, TH3D=th_phy, T3D=t_phy                   &
L01630                ,QV3D=qv_curr, QC3D=qc_curr, QI3D=qi_curr, RR3D=rho                &
L01631  #if (WRF_CHEM == 1)
L01632                ,CHEM3D=CHEM,   VD3D=VD,  NCHEM=nchem, KDVEL=kdvel              &
L01633                ,NDVEL=ndvel, NUM_VERT_MIX=num_vert_mix                         &
L01634  #endif
L01635                ,UST=UST, HFX=HFX, QFX=QFX, TSK=tsk                                &
L01636                ,PSFC=PSFC, EP1=EP_1, G=g, ROVCP=rcp,RD=r_D,CPD=cp                 &
L01637                ,PBLH=pblh, KPBL2D=kpbl, EXCH_H=exch_h, EXCH_M=exch_m              &
L01638                ,REGIME=regime                                                     &
L01639                ,GZ1OZ0=gz1oz0,WSPD=wspd,PSIM=psim, MUT=mut, RMOL=rmol             &
L01640                ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten                 &
L01641                ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten             &
L01642                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde                   &
L01643                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme                   &
L01644                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte                   &
L01645                                                                          )
L01646             ELSE
L01647                 WRITE ( message , FMT = '(A,4(L1,1X))' )             &
L01648                   'present: '//                                      &
L01649                   'qv_curr, '//                                      &
L01650                   'qc_curr, '//                                      &
L01651                   'rqvblten, '//                                     &
L01652                   'rqcblten = ' ,                                    &
L01653                    PRESENT( qv_curr ) ,                              &
L01654                    PRESENT( qc_curr ) ,                              &
L01655                    PRESENT( rqvblten ) ,                             &
L01656                    PRESENT( rqcblten )
L01657                 CALL wrf_debug(0,message)
L01658                 CALL wrf_error_fatal('Lack arguments to call ACM2 pbl')
L01659             ENDIF
L01660
```

讲解：ACM2（7）：非局地向上输送 +局地扩散混合方案，E-ε 闭合的简化替代；WRF-Chem 下额外传化学物种垂直扩散数组（#if WRF_CHEM==1 包裹——驱动层的条件编译样板）。

MYNN 分支（本文库 09 篇方案的入口）：

```
!file: phys/module_pbl_driver.F    L1661-L1868
L01661  #if (EM_CORE==1)
L01662
L01663          CASE (MYNNPBLSCHEME)
L01664
L01665             IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01666                  PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01667                  PRESENT( qi_curr ) .AND. PRESENT( rqiblten ) .AND.  &
L01668                  PRESENT( rqniblten ) .AND. PRESENT( qni_curr ) .AND.&
L01669                  PRESENT(qke) .AND. PRESENT(tsq) .AND.               &
L01670                  PRESENT(qsq) .AND. PRESENT(cov) .AND.               &
L01671                  PRESENT(ch) .AND.                                   &
L01672                  PRESENT(tke_budget) .AND. PRESENT(qke_adv) .AND.    &
L01673                  PRESENT(bl_mynn_tkeadvect)  ) THEN
L01674
L01675                CALL wrf_debug(100,'in MYNNPBL')
L01676
L01677                IF (itimestep==1) THEN
L01678                   initflag=1
L01679                ELSE
L01680                   initflag=0
L01681                ENDIF
```

```
L01682
L01683                if (pert_mynn .and. multi_perturb == 1) then
L01684                  allocate (qke_tmp(its:ite, kts:kte, jts:jte))
L01685
L01686                  call Add_multi_perturb_pbl_perturbations (               &
L01687                    perts_qvapor, perts_qcloud, perts_th, perts_tke,       &
L01688                    pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke,&
L01689                    th_phy, qke, qke_adv, qv_curr, qc_curr,                &
L01690                    bl_mynn_tkeadvect, qke_tmp, its, ite, jts, jte,        &
L01691                    ims, ime, jms, jme, kms, kme, kts, kte)
L01692                end if
L01693
L01694                CALL mynnedmf_driver(                                      &
L01695                     &initflag=initflag,restart=restart,cycling=cycling,   &
L01696                     &delt=dtbl,dz=dz8w,dxc=dx,znt=znt,                    &
L01697                     &u=u_phy,v=v_phy,w=w,th=th_phy,qv=qv_curr,            &
L01698                     &qc=qc_curr,qi=qi_curr,qs=qs_curr,                    &
L01699                     &qnc=qnc_curr,qni=qni_curr,                           &
L01700                     &QNWFA=qnwfa_curr,QNIFA=qnifa_curr,QNBCA=qnbca_curr,  &
L01701  !                   &ozone=ozone,                                         &
L01702                     &p=p_phy,exner=pi_phy,rho=rho,T3D=t_phy,              &
L01703                     &xland=xland,ts=tsk,qsfc=qsfc,ps=psfc,                &
L01704                     &ust=ust,ch=ch,hfx=hfx,qfx=qfx,wspd=wspd,             &
L01705                     &uoce=uoce,voce=voce,                                 & !Ocean currents
L01706                     &Qke=qke,qke_adv=qke_adv,Sh3d=Sh3d,Sm3d=Sm3d,         &
L01707  #if (WRF_CHEM == 1)
L01708                     &mix_chem=mynn_chem_vertmx,                           &
L01709                     &chem3d=chem,vd3d=vd,nchem=nchem,kdvel=kdvel,         &
L01710                     &ndvel=ndvel,num_vert_mix=num_vert_mix,               &
L01711  !                   &FRP_MEAN=FRP_MEAN,EMIS_ANT_NO=EMIS_ANT_NO,           & !to be included soon
L01712  !                   &fire_turb=enh_vermix,                                & !to be included soon
L01713  #endif
L01714                     &Tsq=tsq,Qsq=qsq,Cov=cov,                             &
L01715                     &RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten,   &
L01716                     &RQVBLTEN=rqvblten,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten,&
L01717                     &RQNCBLTEN=rqncblten,RQNIBLTEN=rqniblten,             &
L01718                     &RQSBLTEN=rqsblten,                                   &
L01719                     &RQNWFABLTEN=rqnwfablten,RQNIFABLTEN=rqnifablten,     &
L01720                     &RQNBCABLTEN=rqnbcablten,                             &
L01721  !                   &Ro3BLTEN=ro3blten,                                   &
L01722                     &EXCH_H=exch_h,EXCH_M=exch_m,                         &
L01723                     &pblh=pblh,KPBL=KPBL,                                 &
L01724                     &el_pbl=el_pbl,                                       &
L01725                     &dqke=dqke,                                           &
L01726                     &qWT=qWT,qSHEAR=qSHEAR,qBUOY=qBUOY,qDISS=qDISS,       &
L01727                     &qc_bl=qc_bl,qi_bl=qi_bl,cldfra_bl=cldfra_bl,         &
L01728                     &edmf_a=edmf_a,edmf_w=edmf_w,edmf_qt=edmf_qt,         &
L01729                     &edmf_thl=edmf_thl,edmf_ent=edmf_ent,edmf_qc=edmf_qc, &
L01730                     &sub_thl3D=sub_thl3D,sub_sqv3D=sub_sqv3D,             &
L01731                     &det_thl3D=det_thl3D,det_sqv3D=det_sqv3D,             &
L01732                     &maxwidth=maxwidth,maxMF=maxMF,                       &
L01733                     &ztop_plume=ztop_plume,maxmf_dd=maxmf_dd,             &
L01734                     &maxtkeprod=maxtkeprod,cldtop_cooling=cldtop_cooling, &
L01735                     &ent_eff=ent_eff,maxwidth_dd=maxwidth_dd,             &
L01736                     &excess_h=excess_h,excess_q=excess_q,                 &
L01737                     &RTHRATEN=RTHRATEN,                                   &
L01738                     &bl_mynn_tkeadvect=bl_mynn_tkeadvect,                 &
L01739                     &tke_budget=tke_budget,                               &
L01740                     &bl_mynn_cloudpdf=bl_mynn_cloudpdf,                   &
L01741                     &bl_mynn_mixlength=bl_mynn_mixlength,                 &
L01742                     &bl_mynn_edmf=bl_mynn_edmf,                           &
L01743                     &bl_mynn_edmf_dd=bl_mynn_edmf_dd,                     &
L01744                     &bl_mynn_edmf_mom=bl_mynn_edmf_mom,                   &
L01745                     &bl_mynn_edmf_tke=bl_mynn_edmf_tke,                   &
L01746                     &bl_mynn_mixscalars=bl_mynn_mixscalars,               &
L01747                     &bl_mynn_mixaerosols=bl_mynn_mixaerosols,             &
L01748                     &bl_mynn_mixnumcon=bl_mynn_mixnumcon,                 &
L01749                     &bl_mynn_output=bl_mynn_output,                       &
L01750                     &bl_mynn_cloudmix=bl_mynn_cloudmix,                   &
L01751                     &bl_mynn_mixqt=bl_mynn_mixqt,                         &
L01752                     &bl_mynn_closure=bl_mynn_closure,                     &
L01753                     &bl_mynn_ess=bl_mynn_ess,                             &
L01754                     &spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl,     &
L01755                     &FLAG_QC=flag_qc,FLAG_QI=flag_qi,FLAG_QS=flag_qs,     &
L01756                     &FLAG_QNC=flag_qnc,FLAG_QNI=flag_qni,                 &
L01757                     &FLAG_QNWFA=flag_qnwfa,FLAG_QNIFA=flag_qnifa,         &
L01758                     &FLAG_QNBCA=flag_qnbca,                               &
L01759                     &IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde,     &
L01760                     &IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme,     &
L01761                     &ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      )
L01762
L01763                if (pert_mynn .and. multi_perturb == 1) then
L01764                  call Remove_multi_perturb_pbl_perturbations (            &
L01765                    perts_qvapor, perts_qcloud, perts_th, perts_tke,       &
L01766                    pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke,&
L01767                    th_phy, qke, qke_adv, qv_curr, qc_curr,                &
L01768                    bl_mynn_tkeadvect, qke_tmp, its, ite, jts, jte,        &
L01769                    ims, ime, jms, jme, kms, kme, kts, kte)
L01770
L01771                    deallocate (qke_tmp)
L01772                end if
L01773             ELSE
L01774                 WRITE ( message , FMT = '(A,16(L1,1X))' )                 &
L01775                   'present: '//                                           &
L01776                   'qv_curr, '//                                           &
L01777                   'qc_curr, '//                                           &
L01778                   'qi_curr, '//                                           &
L01779                   'qni_curr, '//                                          &
L01780                   'rqvblten, '//                                          &
L01781                   'rqcblten, '//                                          &
L01782                   'rqiblten, '//                                          &
L01783                   'rqniblten, '//                                         &
L01784                   'qke, '//                                               &
L01785                   'tsq, '//                                               &
L01786                   'qsq, '//                                               &
L01787                   'cov, '//                                               &
L01788                   'ch, '//                                                &
L01789                   'tke_budget, '//                                        &
L01790                   'qke_adv, '//                                           &
```

```
L01791                   'bl_mynn_tkeadvect ' ,                          &
L01792                    PRESENT( qv_curr ) ,                           &
L01793                    PRESENT( qc_curr ) ,                           &
L01794                    PRESENT( qi_curr ) ,                           &
L01795                    PRESENT( qni_curr ) ,                          &
L01796                    PRESENT( rqvblten ) ,                          &
L01797                    PRESENT( rqcblten ) ,                          &
L01798                    PRESENT( rqiblten ) ,                          &
L01799                    PRESENT( rqniblten ) ,                         &
L01800                    PRESENT( qke      ) ,                          &
L01801                    PRESENT( tsq      ) ,                          &
L01802                    PRESENT( qsq      ) ,                          &
L01803                    PRESENT( cov      ) ,                          &
L01804                    PRESENT( ch       ) ,                          &
L01805                    PRESENT( tke_budget) ,                         &
L01806                    PRESENT( qke_adv  ) ,                          &
L01807                    PRESENT( bl_mynn_tkeadvect)
L01808                CALL wrf_debug(0,message)
L01809                CALL wrf_error_fatal('Lack arguments to call MYNN pbl')
L01810             ENDIF
L01811
L01812             !fill scalar_tend array. The RQN*BLTEN arrays are no longer
L01813             !passed to module_physics_addtendc.F (for MYNN)
L01814             IF (bl_mynn_mixnumcon .eq. 1) THEN
L01815                IF (PRESENT( qni_curr ) .AND. Flag_qni) THEN
L01816                  !print*,"Updating qni after mynn-edmf",P_QNI
L01817                  DO j=j_start(ij),j_end(ij)
L01818                  DO k=kts,kte
L01819                  DO i=i_start(ij),i_end(ij)
L01820                     scalar_tend(i,k,j,P_QNI)=RQNIBLTEN(i,k,j)
L01821                  enddo
L01822                  enddo
L01823                  enddo
L01824                ENDIF
L01825                IF (PRESENT( qnc_curr ) .AND. Flag_qnc) THEN
L01826                  !print*,"Updating qnc after mynn-edmf",P_QNC
L01827                  DO j=j_start(ij),j_end(ij)
L01828                  DO k=kts,kte
L01829                  DO i=i_start(ij),i_end(ij)
L01830                     scalar_tend(i,k,j,P_QNC)=RQNCBLTEN(i,k,j)
L01831                  enddo
L01832                  enddo
L01833                  enddo
L01834                ENDIF
L01835             endif
L01836             if (bl_mynn_mixaerosols .eq. 1) then
L01837                IF (PRESENT( qnwfa_curr ) .AND. Flag_qnwfa) THEN
L01838                  !print*,"Updating qnwfa after mynn-edmf",P_QNWFA
L01839                  DO j=j_start(ij),j_end(ij)
L01840                  DO k=kts,kte
L01841                  DO i=i_start(ij),i_end(ij)
L01842                     scalar_tend(i,k,j,P_QNWFA)=RQNWFABLTEN(i,k,j)
L01843                  enddo
L01844                  enddo
L01845                  enddo
L01846                ENDIF
L01847                IF (PRESENT( qnifa_curr ) .AND. Flag_qnifa) THEN
L01848                  !print*,"Updating qnifa after mynn-edmf",P_QNIFA
L01849                  DO j=j_start(ij),j_end(ij)
L01850                  DO k=kts,kte
L01851                  DO i=i_start(ij),i_end(ij)
L01852                     scalar_tend(i,k,j,P_QNIFA)=RQNIFABLTEN(i,k,j)
L01853                  enddo
L01854                  enddo
L01855                  enddo
L01856                ENDIF
L01857                IF (PRESENT( qnbca_curr ) .AND. Flag_qnbca) THEN
L01858                  !print*,"Updating qnbca after mynn-edmf",P_QNBCA
L01859                  DO j=j_start(ij),j_end(ij)
L01860                  DO k=kts,kte
L01861                  DO i=i_start(ij),i_end(ij)
L01862                     scalar_tend(i,k,j,P_QNBCA)=RQNBCABLTEN(i,k,j)
L01863                  enddo
L01864                  enddo
L01865                  enddo
L01866                ENDIF
L01867             ENDIF
L01868
```

讲解：①PRESENT 守卫升级为 16 项——qke/tsq/qsq/cov（TKE 及其通量-方差二阶矩）、ch（面层热交换系数，来自 code_10 的 surface driver）、tke_budget/qke_adv/bl_mynn_tkeadvect 缺一不可：这正是「切到 MYNN 必须 real 重跑」的根源；②initflag=首步标记传给 mynnedmf_driver；**③SPP 包裹**：pert_mynn 且 multi_perturb=1 时，先 Add_multi_perturb 把乘性扰动打在 qv/qc/th/qke 上再调方案、方案返回后 Remove 还原——扰动只在 PBL 方案作用域内生效（WRF-Solar EPS 集合技术）；④ 主体调用把 bl_mynn_* 十余个 namelist 开关原样透传（清单见 code_09a 篇首）；⑤ **收尾回填**：bl_mynn_mixnumcon/mixaerosols=1 时把 RQNIBLTEN/RQNCBLTEN/RQNWFA… 从专设倾向数组搬进 scalar_tend 数组的对应物种位——注释明言「RQN*BLTEN 不再传给 module_physics_addtendc.F」：数浓度与气溶胶的 PBL 倾向绕过了通用倾向累加器，直接在 driver 内完成（改 MYNN 混合行为时别在 addtendc 里白找）。

```
!file: phys/module_pbl_driver.F    L1869-L1916
L01869          CASE (EEPSSCHEME)
L01870
L01871               IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND. &
L01872                    PRESENT( qi_curr )                            .AND. &
L01873                    PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L01874                    PRESENT( rqiblten )                           .AND. &
L01875                    PRESENT( pek )      .AND. PRESENT( pep)       .AND. &
```

```
L01876                    PRESENT( pek_adv )  .AND. PRESENT( pep_adv ) ) THEN
L01877
L01878                  CALL wrf_debug(100,'in EEPSPBL')
L01879
L01880                  CALL  eeps(&
L01881                     U3D=u_phytmp,V3D=v_phytmp,T3D=t_phy                  &
L01882                    ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr               &
L01883                    ,QR3D=qr_curr,QS3D=qs_curr,QG3D=qg_curr               &
L01884                    ,P3D=p_phy,PI3D=pi_phy,PEK=pek,PEP=pep                &
L01885                    ,RHO3D=rho,RTHRATEN=rthraten                          &
L01886                    ,RUBLTEN=rublten,RVBLTEN=rvblten                      &
L01887                    ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                  &
L01888                    ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten,KPBL=kpbl        &
L01889                    ,DZ8W=dz8w,PSFC=psfc,TSK=tsk,QSFC=qsfc,WSPD=wspd      &
L01890                    ,UST=ust,XLAND=xland,HFX=hfx,QFX=qfx,RMOL=rmol        &
L01891                    ,DT=dtbl,DX=dx,EXCH_H=exch_h,EXCH_M=exch_m,PBLH=pblh  &
L01892                    ,ITIMESTEP=itimestep,PEK_ADV=pek_adv,PEP_ADV=pep_adv   &
L01893                     ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde      &
L01894                     ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01895                     ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      &
L01896                     )
L01897                ELSE
L01898                  WRITE ( message , FMT = '(A,6(L1,1X))' )             &
L01899                    'present: '//                                      &
L01900                    'qv_curr, '//                                      &
L01901                    'qc_curr, '//                                      &
L01902                    'qi_curr, '//                                      &
L01903                    'rqvblten, '//                                     &
L01904                    'rqcblten, '//                                     &
L01905                    'rqiblten, ' ,                                     &
L01906                     PRESENT( qv_curr ) ,                              &
L01907                     PRESENT( qc_curr ) ,                              &
L01908                     PRESENT( qi_curr ) ,                              &
L01909                     PRESENT( rqvblten ) ,                             &
L01910                     PRESENT( rqcblten ) ,                             &
L01911                     PRESENT( rqiblten )
L01912                  CALL wrf_debug(0,message)
L01913                  CALL wrf_error_fatal('Lack arguments to call EEPS pbl')
L01914               ENDIF
L01915
L01916
```

讲解：EEPS（16）：集合型 PBL——pek/pep（两组扰动集合的能量谱）作 state 携带，pek_adv/pep_adv 为 advect 标志；与 MYNN 的 SPP 同属「随机物理」家族但实现为显式集合。

```
!file: phys/module_pbl_driver.F    L1917-L1947
L01917                  CASE (KEPSSCHEME)
L01918
L01919                CALL wrf_debug(100,'in keps')
L01920                CALL KEPS(TSK=tsk,MOL=mol,XTIME=itimestep,FRC_URB2D=frc_urb2d           &
L01921                 , FLAG_BEP=flag_bep  &
L01922                 ,DZ8W=dz8w,DT=dt,U_PHY=u_phytmp                                  &
L01923                 ,V_PHY=v_phytmp,TH_PHY=th_phy                                    &
L01924                 ,PI_PHY=pi_phy,RTHRATEN=RTHRATEN,P8W=p8w                         &
L01925                 ,RHO=rho,QV_CURR=qv_curr,QC_CURR=qc_curr,HFX=hfx                 &
L01926                 ,QFX=qfx,USTAR=ust,CP=cp,G=g                                     &
L01927                 ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten               &
L01928                 ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                             &
L01929                 ,TKE_PBL=tke_pbl,DISS_PBL=diss_pbl,TPE_PBL=tpe_pbl               &
L01930                 ,TKE_ADV=tke_adv,DISS_ADV=diss_adv,TPE_ADV=tpe_adv               &
L01931                 ,PR_PBL=pr_pbl                                                   &
L01932                 ,WU=wu_tur,WV=wv_tur,WT=wt_tur,WQ=wq_tur                         &
L01933                 ,EXCH_H=exch_h,EXCH_M=exch_m,PBLH=pblh                           &
L01934                 ,A_U_BEP=a_u_bep,A_V_BEP=a_v_bep,A_T_BEP=a_t_bep                 &
L01935                 ,A_Q_BEP=a_q_bep                                                 &
L01936                 ,B_U_BEP=b_u_bep,B_V_BEP=b_v_bep                                 &
L01937                 ,B_T_BEP=b_t_bep                                                 &
L01938                 ,B_Q_BEP=b_q_bep                                                 &
L01939                 ,B_E_BEP=b_e_bep                                                 &
L01940                 ,SF_BEP=sf_bep,VL_BEP=vl_bep                                     &
L01941                 ,BR=br,ZNT=znt                                                   &
L01942                 ,PSIM=fm,PSIH=fhh                                              &
L01943                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde                 &
L01944                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme                 &
L01945                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte  )
L01946
L01947
```

讲解：KEPS（17）：k-ε 双方程闭合，专为 BEP/BEM 城市冠层设计（a_e_bep/b_e_bep 是建筑内 TKE 源项）——sf_urban_physics=3（论文 2）配 bl_pbl_physics=17 是官方推荐组合；BouLac=8 与 MYJ=2 是 BEP 兼容的另外两支。

```
!file: phys/module_pbl_driver.F    L1949-L1971
L01949          CASE (BOULACSCHEME)
L01950
L01951                CALL wrf_debug(100,'in boulac')
L01952
L01953                CALL BOULAC(FRC_URB2D=frc_urb2d,IDIFF=idiff,FLAG_BEP=flag_bep     &
L01954                 ,DZ8W=dz8w,DT=dt,U_PHY=u_phytmp                                  &
L01955                 ,V_PHY=v_phytmp,TH_PHY=th_phy                                    &
L01956                 ,RHO=rho,QV_CURR=qv_curr,QC_CURR=qc_curr,HFX=hfx                 &
L01957                 ,QFX=qfx,USTAR=ust,CP=cp,G=g                                     &
L01958                 ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten               &
L01959                 ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                             &
L01960                 ,TKE=tke_pbl,DLK=el_pbl,WU=wu_tur,WV=wv_tur,WT=wt_tur,WQ=wq_tur  &
L01961                 ,EXCH_H=exch_h,EXCH_M=exch_m,PBLH=pblh                           &
L01962                 ,A_U_BEP=a_u_bep,A_V_BEP=a_v_bep,A_T_BEP=a_t_bep                 &
L01963                 ,A_Q_BEP=a_q_bep                                                 &
L01964                 ,A_E_BEP=a_e_bep,B_U_BEP=b_u_bep,B_V_BEP=b_v_bep                 &
L01965                 ,B_T_BEP=b_t_bep,B_E_BEP=b_e_bep,DLG_BEP=dlg_bep                 &
L01966                 ,B_Q_BEP=b_q_bep                                                 &
L01967                 ,DL_U_BEP=dl_u_bep,SF_BEP=sf_bep,VL_BEP=vl_bep                   &
```

```
L01968               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde                &
L01969               ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme                &
L01970               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte  )
L01971
```

讲解：BouLac（8）：TKE-l 阶 1.5 闭合 + Bougeault–Lacarrère 混合长（方案实体精读见 code_09a 篇的 boulac 双长度对照节）；wu/wv/wt/wq_tur 湍流通量诊断全套传出。论文 2 的边界层首选。

```
!file: phys/module_pbl_driver.F    L1972-L2028
L01972             CASE (CAMUWPBLSCHEME)
L01973
L01974                CALL wrf_debug(100,'in camuwpbl')
L01975                IF ( PRESENT( qv_curr  )  .AND. PRESENT( qc_curr   )  .AND. &
L01976                     PRESENT( qi_curr  )  .AND. PRESENT( qnc_curr  )  .AND. &
L01977                     PRESENT( qni_curr )  .AND.                             &
L01978                     PRESENT( rqvblten )  .AND. PRESENT( rqcblten  )  .AND. &
L01979                     PRESENT( rqiblten )  .AND. PRESENT( rqniblten )        &
L01980                   )THEN
L01981                   CALL CAMUWPBL(DT=dt,U_PHY=u_phy,V_PHY=v_phy,TH_PHY=th_phy,RHO=rho   &
L01982                        ,QV_CURR=qv_curr,HFX=hfx,QFX=qfx,USTAR=ust,P8W=p8w,P_PHY=p_phy &
L01983                        ,Z=z,T_PHY=t_phy,QC_CURR=qc_curr,QI_CURR=qi_curr,Z_AT_W=z_at_w &
L01984                        ,CLDFRA_OLD_mp=cldfra_old_mp,CLDFRA=cldfra,HT=ht               &
L01985                        ,RTHRATENLW=rthratenlw,EXNER=pi_phy                            &
L01986                        ,is_CAMMGMP_used=is_CAMMGMP_used                               &
L01987                        ,ITIMESTEP=itimestep,QNC_CURR=qnc_curr,QNI_CURR=qni_curr       &
L01988                        ,WSEDL3D=wsedl3d                                               &
L01989
L01990                        ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde             &
L01991                        ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme             &
L01992                        ,ITS=its,ITE=ite, JTS=jts,JTE=jte, KTS=kts,KTE=kte             &
L01993  !Intent-inout
L01994                        ,TAURESX2D=tauresx2d,TAURESY2D=tauresy2d                       &
L01995                        ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten             &
L01996                        ,RQIBLTEN=rqiblten,RQNIBLTEN=rqniblten,RQVBLTEN=rqvblten       &
L01997                        ,RQCBLTEN=rqcblten,KVM3D=exch_m,KVH3D=exch_h                   &
L01998  !Intent-out
L01999                        ,TPERT2D=tpert2d,QPERT2D=qpert2d,WPERT2D=wpert2d,SMAW3D=smaw3d &
L02000                        ,TURBTYPE3D=turbtype3d                                         &
L02001                        ,TKE_pbl=tke_pbl,PBLH2D=pblh,KPBL2D=kpbl                       )
L02002                ELSE
L02003                   WRITE ( message , FMT = '(A,8(L1,1X))' )            &
L02004                        'present: '//                                      &
L02005                        'qv_curr, '//                                      &
L02006                        'qc_curr, '//                                      &
L02007                        'qi_curr, '//                                      &
L02008                        'qnc_curr, '//                                     &
L02009                        'qni_curr, '//                                     &
L02010                        'rqvblten, '//                                     &
L02011                        'rqcblten, '//                                     &
L02012                        'rqiblten, '//                                     &
L02013                        'rqniblten= ',                                     &
L02014                        PRESENT( qv_curr ) ,                               &
L02015                        PRESENT( qc_curr ) ,                               &
L02016                        PRESENT( qi_curr ) ,                               &
L02017                        PRESENT( qnc_curr ) ,                              &
L02018                        PRESENT( qni_curr ) ,                              &
L02019                        PRESENT( rqvblten ) ,                              &
L02020                        PRESENT( rqcblten ) ,                              &
L02021                        PRESENT( rqiblten ) ,                              &
L02022                        PRESENT( rqniblten )
L02023
L02024                   CALL wrf_debug(0,message)
L02025                   CALL wrf_error_fatal('Lack arguments to call CAMUWPBL pbl')
L02026                ENDIF
L02027
L02028  #endif
```

讲解：CAMUW（9）：CAM 垂直扩散型（社区大气模式移植），带湍流界面类型 turbtype3d、Galperin 稳定度函数 smaw3d 等精细诊断；要求 qnc/qni 数浓度在场。

```
!file: phys/module_pbl_driver.F    L2030-L2084
L02030             CASE (GBMPBLSCHEME)
L02031                CALL wrf_debug(100,'in gbmpbl')
L02032                IF ( PRESENT( qv_curr )  .AND. PRESENT( qc_curr )  .AND.&
L02033                PRESENT( qi_curr )                                 .AND. &
L02034                PRESENT( rqvblten ) .AND. PRESENT( rqcblten ) .AND. &
L02035                PRESENT( rqiblten )                                .AND. &
L02036                PRESENT( hol      )                                .AND. &
L02037                                        .TRUE.  ) THEN
L02038               CALL gbmpbl(                                              &
L02039                 U3D=u_phytmp,V3D=v_phytmp,TH3D=th_phy,T3D=t_phy         &
L02040                ,QV3D=qv_curr,QC3D=qc_curr,QI3D=qi_curr                  &
L02041                ,P3D=p_phy,PI3D=pi_phy                                   &
L02042                ,RUBLTEN=rublten,RVBLTEN=rvblten                         &
L02043                ,RTHBLTEN=rthblten,RQVBLTEN=rqvblten                     &
L02044                ,RQCBLTEN=rqcblten,RQIBLTEN=rqiblten                     &
L02045                ,KZM_GBM=exch_m,KTH_GBM=exch_h,KETHL_GBM=exch_tke        &
L02046                ,EL_GBM=el_pbl,DZ8W=dz8w,Z=z,PSFC=PSFC                   &
L02047                ,TKE_PBL=tke_pbl,RTHRATEN=rthraten                       &
L02048                ,ZNT=znt,UST=ust,ZOL=zol,HOL=hol,PBL=pblh                &
L02049                ,KPBL2D=kpbl,REGIME=regime                               &
L02050                ,PSIM=psim,PSIH=psih,XLAND=xland                         &
L02051                ,HFX=hfx,QFX=qfx,TSK=tskold,GZ1OZ0=gz1oz0                &
L02052                ,WSPD=wspd,BR=br,DT=dtbl,DTMIN=dtmin                     &
L02053                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde         &
L02054                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme         &
L02055                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte         &
L02056                                                                         )
L02057                ELSE
L02058                 CALL wrf_error_fatal('Lack arguments to call GBM pbl')
L02059                ENDIF
L02060
```

```
L02061  #if ( WRFPLUS == 1 )
L02062  ! this scheme is for WRFPlus only
L02063  !-----------------------------------
L02064          CASE (SURFDRAGSCHEME)
L02065
L02066             CALL wrf_debug(100,'in SURFDRAG scheme')
L02067
L02068             CALL surface_drag(                                        &
L02069               RUBLTEN=rublten,RVBLTEN=rvblten                         &  !
L02070              ,U_PHY=u_phy, V_PHY=v_phy, Z=z                                &  !
L02071              ,XLAND=xland, HT=ht, KPBL2D=kpbl                        &  !
L02072              ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L02073              ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L02074              ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte        &
L02075                                                                      )
L02076  #endif
L02077
L02078        CASE DEFAULT
L02079
L02080          WRITE( message , * ) 'The pbl option does not exist: bl_pbl_physics = ', bl_pbl_physics
L02081          CALL wrf_error_fatal ( message )
L02082
L02083      END SELECT pbl_select
L02084
```

讲解：GBM（12）与 SURFDRAG（98，仅 WRFPLUS）收尾；CASE DEFAULT 报「方案号不存在」。END SELECT 带命名标签 pbl_select——长 SELECT CASE 的可读性写法。

3.6 CASE 之外的四个附加模块

```
!file: phys/module_pbl_driver.F    L2085-L2174
L02085  #if (EM_CORE==1)
L02086  !
L02087  ! ... paj: wind farm ...
L02088  !
L02089      windfarm_select: SELECT CASE(windfarm_opt)
L02090
L02091        CASE (fitchscheme)
L02092
L02093                    IF (PRESENT(id) .AND.                                  &
L02094                       PRESENT(z_at_w) ) THEN
L02095  !
L02096                       CALL wrf_debug(100,'in phys/module_wind_fitch.F')
L02097                       CALL dragforce(                                 &
L02098                       & ID=id                                            &
L02099                       &,Z_AT_W=z_at_w,u=u_phy,v=v_phy                    &
L02100                       &,DX=dx,DZ=dz8w,DT=dt                              &
L02101                       &,QKE=qke                                          &
L02102                       &,DU=rublten,DV=rvblten                            &
L02103                       &,WINDFARM_OPT=windfarm_opt,POWER=power            &
L02104                       ! THE FOLLOWING LINES OF CODE ARE LICENSED UNDER doc/licenses/fraunhofer_license.txt
L02105                       &,THRUST=thrust                                    &
L02106                       ! END LICENSE
L02107                       &,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde  &
L02108                       &,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme  &
L02109                       &,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte  &
L02110                       &)
L02111
L02112                      IF (bl_mynn_tkeadvect) THEN
L02113                         qke_adv=qke
L02114                      ENDIF
L02115
L02116                    ELSE
L02117                       WRITE ( message , FMT = '(A,6(L1,1X))' )           &
L02118                       'present: '//                                      &
L02119                       'ID, '//                                           &
L02120                       'z_at_w, '//                                       &
L02121                       'xlat_u, '//                                       &
L02122                       'xlong_u, '//                                      &
L02123                       'xlat_v, '//                                       &
L02124                       'xlong_v = ' ,                                     &
L02125                        PRESENT( id ) ,                                   &
L02126                        PRESENT( z_at_w )
L02127                       CALL wrf_debug(0,message)
L02128                       CALL wrf_error_fatal('Lack arguments to call turbine_drag')
L02129                    ENDIF
L02130
L02131       ! Yulong add new wind farm schemes with wind turbine loss effect
L02132       CASE (mavscheme)
L02133                    IF (PRESENT(id) .AND.                                  &
L02134                       PRESENT(z_at_w) ) THEN
L02135                       CALL wrf_debug(100,'in phys/module_wind_mav.F')
L02136                       CALL dragforce_mav(itimestep                    &
L02137                       &,ID=id                                            &
L02138                       &,Z_AT_W=z_at_w,z_at_m=z,u=u_phy,v=v_phy           &
L02139                       &,DX=dx,DZ=dz8w,DT=dt                              &
L02140                       &,TKE=TKE_windfarm                                 &
L02141                       &,DU=rublten,DV=rvblten                            &
L02142                       &,WINDFARM_OPT=windfarm_opt,POWER=power            &
L02143                       &,windfarm_wake_model=windfarm_wake_model          &
L02144                       &,windfarm_overlap_method=windfarm_overlap_method  &
L02145                       &,xland=xland                                      &
L02146                       &,cosa=cosa,sina=sina                              &
L02147                       &,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde  &
L02148                       &,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme  &
L02149                       &,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte  &
L02150                       &)
L02151
L02152                       IF (bl_mynn_tkeadvect) THEN
L02153                          QKE = QKE + 2.*TKE_windfarm
L02154                          qke_adv=qke
L02155                       ENDIF
L02156
```

```
L02157                   ELSE
L02158                      WRITE ( message , FMT = '(A,6(L1,1X))' )           &
L02159                      'present: '//                                      &
L02160                      'ID, '//                                           &
L02161                      'z_at_w, '//                                       &
L02162                      'xlat_u, '//                                       &
L02163                      'xlong_u, '//                                      &
L02164                      'xlat_v, '//                                       &
L02165                      'xlong_v = ' ,                                     &
L02166                       PRESENT( id ) ,                                   &
L02167                       PRESENT( z_at_w )
L02168                      CALL wrf_debug(0,message)
L02169                      CALL wrf_error_fatal('Lack arguments to call dragforce_mav')
L02170                   ENDIF
L02171
L02172    END SELECT windfarm_select
L02173  #endif
L02174
```

讲解：**风机场参数化**独立成第二个 SELECT（windfarm_opt）：FITCH（16）与 MAV 两族风机拖曳方案直接把动量汇写进 rublten/rvblten——风机与 PBL 方案解耦、可任意组合；bl_mynn_tkeadvect 打开时把风机 TKE 源并回 qke（MAV 支路乘 2：QKE=2 • TKE 的约定，见 code_09a）。

```
!file: phys/module_pbl_driver.F    L2176-L2236
L02176     IF (PRESENT(GWD_opt)) THEN
L02177       IF (GWD_opt .EQ. 1) THEN
L02178         IF (PRESENT(dtaux3d)) THEN
L02179              CALL gwdo(                                                &
L02180                U3D=u_phy,V3D=v_phy,T3D=t_phy                           &
L02181               ,QV3D=qv_curr                                            &
L02182               ,P3D=p_phy,P3DI=p8w,PI3D=pi_phy,Z=z                      &
L02183               ,RUBLTEN=rublten,RVBLTEN=rvblten                         &
L02184               ,DTAUX3D=dtaux3d,DTAUY3D=dtauy3d                         &
L02185               ,DUSFCG=dusfcg,DVSFCG=dvsfcg                             &
L02186               ,VAR2D=var2d,OC12D=oc12d                                 &
L02187               ,OA2D1=oa1,OA2D2=oa2,OA2D3=oa3,OA2D4=oa4                 &
L02188               ,OL2D1=ol1,OL2D2=ol2,OL2D3=ol3,OL2D4=ol4                 &
L02189               ,elvmax=elvmax,SINA=sina,COSA=cosa                       &
L02190               ,ZNU=znu,ZNW=znw,P_TOP=p_top                             &
L02191               ,CP=cp,G=g,RD=r_d                                        &
L02192               ,RV=r_v,EP1=ep_1,PI=3.141592653                          &
L02193               ,DT=dtbl,DX=dx2d,KPBL2D=kpbl,ITIMESTEP=itimestep         &
L02194               ,errmsg=errmsg,errflg=errflg                             &
L02195               ,dx_factor = dx_factor, if_nonhyd = if_nonhyd            &
L02196               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde         &
L02197               ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme         &
L02198               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte         )
L02199         ENDIF
L02200       ELSEIF(gwd_opt .EQ. 3) THEN
L02201              CALL gwdo_gsl(                                            &
L02202                U3D=u_phy,V3D=v_phy,T3D=t_phy                           &
L02203               ,QV3D=qv_curr                                            &
L02204               ,P3D=p_phy,P3DI=p8w,PI3D=pi_phy,Z=z                      &
L02205               ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten       &
L02206               ,DTAUX3D_ls=dtaux3d_ls,DTAUY3D_ls=dtauy3d_ls             &
L02207               ,DTAUX3D_bl=dtaux3d_bl,DTAUY3D_bl=dtauy3d_bl             &
L02208               ,DTAUX3D_ss=dtaux3d_ss,DTAUY3D_ss=dtauy3d_ss             &
L02209               ,DTAUX3D_fd=dtaux3d_fd,DTAUY3D_fd=dtauy3d_fd             &
L02210               ,DUSFCG_ls=dusfcg_ls,DVSFCG_ls=dvsfcg_ls                 &
L02211               ,DUSFCG_bl=dusfcg_bl,DVSFCG_bl=dvsfcg_bl                 &
L02212               ,DUSFCG_ss=dusfcg_ss,DVSFCG_ss=dvsfcg_ss                 &
L02213               ,DUSFCG_fd=dusfcg_fd,DVSFCG_fd=dvsfcg_fd                 &
L02214               ,xland=xland,br=br                                       &
L02215               ,VAR2D=var2dls,OC12D=oc12dls                             &
L02216               ,OA2D1=oa1ls,OA2D2=oa2ls,OA2D3=oa3ls,OA2D4=oa4ls         &
L02217               ,OL2D1=ol1ls,OL2D2=ol2ls,OL2D3=ol3ls,OL2D4=ol4ls         &
L02218               ,VAR2Dss=var2dss,OC12Dss=oc12dss                         &
L02219               ,OA2D1ss=oa1ss,OA2D2ss=oa2ss,OA2D3ss=oa3ss               &
L02220               ,OA2D4ss=oa4ss,OL2D1ss=ol1ss,OL2D2ss=ol2ss               &
L02221               ,OL2D3ss=ol3ss,OL2D4ss=ol4ss                             &
L02222               ,SINA=sina,COSA=cosa                                     &
L02223               ,ZNU=znu,ZNW=znw,P_TOP=p_top                             &
L02224               ,DZ=dz8w,PBLH=PBLH                                       &
L02225               ,CP=cp,G=g,RD=r_d                                        &
L02226               ,RV=r_v,EP1=ep_1,PI=3.141592653                          &
L02227               ,DT=dtbl,DX=dx,KPBL2D=kpbl                               &
L02228               ,ITIMESTEP=itimestep,gwd_opt=gwd_opt                     &
L02229               ,gwd_diags=gwd_diags                                     &
L02230               ,spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl         &
L02231               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde         &
L02232               ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme         &
L02233               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte         )
L02234         ENDIF
L02235       ENDIF
L02236
```

讲解：**重力波拖曳**挂在本 driver 的末尾（GWD_opt=1 经典 gwdo / =3 GSL 扩展版）：次网格地形对动量的吸收/破碎项与 PBL 湍流应力同量纲、同出口（rublten/rvblten），故共用此驱动入口；GSL 版还接受 SPP 扰动场。

```
!file: phys/module_pbl_driver.F    L2237-L2255
L02237  #if (EM_CORE==1)
↪
L02238    !JOE-added - fog (cloud) water deposition calculation
L02239       IF (grav_settling .GE. 1) THEN
L02240          IF ( PRESENT(vdfg) .AND. PRESENT(qc_curr)) THEN
L02241             !NOTE 1: vdfg is calculated in the surface driver.
L02242             !NOTE 2: as of now, only qc_curr settles, not # conc.
L02243
L02244              CALL bl_fogdes(                                          &
L02245                   vdfg,qc_curr,dtbl,rho,dz8w,grav_settling,RQCBLTEN,&
```

```
L02246                ids,ide, jds,jde, kds,kde,                    &
L02247                ims,ime, jms,jme, kms,kme,                    &
L02248                i_start(ij),i_end(ij),                        &
L02249                j_start(ij),j_end(ij),kts,kte                 )
L02250        ELSE
L02251            CALL wrf_error_fatal('Missing args for bl_fogdes in pbl driver')
L02252        ENDIF
L02253     ENDIF
L02254  !JOE-END
L02255  #endif
```

讲解：**雾水沉降** bl_fogdes：grav_settling≥1（MYNN 面层 namelist 开关）时把云水重力沉降加进 RQCBLTEN；注释点明 vdfg（沉降速度）由 surface driver 算——跨 driver 的隐式数据流。

```
!file: phys/module_pbl_driver.F    L2257-L2332
L02257       IF (idiff.eq.1) THEN
L02258
L02259  !Alberto: here we call the general routine to solve the diffusion
L02260  ! + all the source/sink terms.
L02261  ! the only thing that should be passed from the PBL schemes is the value of the exch_h, and exch_m
L02262  ! (and the non local terms, if present, through the b).
L02263  ! So, this routine can be used by any of the previous schemes. We only need to pass the right variables
L02264  ! (and eliminate the computation of the tendencies from the previous routines, to avoid doing twice the job).
L02265  ! As I explain below, in the routine, here we could extract the vertical turbulent
L02266  ! fluxes (something that will be general for all the routines).
L02267  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02268
L02269
L02270            CALL diff3d  (DT=dtbl,CP=cp,DZ=dz8w,TH=th_phy,QV=qv_curr,QC=qc_curr,T=t_phy  &
L02271                ,U=u_phy,V=v_phy,RHO=rho,EXCH_H=exch_h                  &
L02272                ,EXCH_M=exch_m                                          &
L02273                ,RUBLTEN=rublten,RVBLTEN=rvblten,RTHBLTEN=rthblten      &
L02274                ,WU=wu_tur,WV=wv_tur,WT=wt_tur,WQ=wq_tur                &
L02275                ,RQVBLTEN=rqvblten,RQCBLTEN=rqcblten                    &
L02276                ,A_U=a_u,A_V=a_v,A_T=a_t,A_Q=a_q        &
L02277                ,B_U=b_u,B_V=b_v,B_T=b_t,B_Q=b_q        &
L02278                ,SF=sf,VL=vl            &
L02279                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L02280                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L02281                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte)
L02282
L02283            DEALLOCATE (a_u)       ! Implicit component for the momemtum in X-direction
L02284            DEALLOCATE (a_v)       ! Implicit component for the momemtum in Y-direction
L02285            DEALLOCATE (a_t)       ! Implicit component for the Pot. Temp.
L02286            DEALLOCATE (a_q)       ! Implicit component for the water vapor
L02287            DEALLOCATE (b_u)       ! Explicit component for the momemtum in X-direction
L02288            DEALLOCATE (b_v)       ! Explicit component for the momemtum in Y-direction
L02289            DEALLOCATE (b_t)       ! Explicit component for the Pot. Temp.
L02290            DEALLOCATE (b_q)       ! Explicit component for the water vapor
L02291            DEALLOCATE (sf )       ! surfaces
L02292            DEALLOCATE (vl )       ! volumes
L02293       ENDIF       !idiff
L02294
L02295            IF(scalar_pblmix .GT. 0)THEN
L02296              CALL diff4d  (DT=dtbl,DZ=dz8w, SCALAR=scalar, is_scalar=.true.  &
L02297                ,RHO=rho,EXCH_H=exch_h        &
L02298                ,EXCH_M=exch_m                &
L02299                ,SCALAR_TEND=scalar_tend      &
L02300                ,NUM_SCALAR=num_scalar, PARAM_FIRST_SCALAR=param_first_scalar &
L02301                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L02302                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L02303                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte)
L02304            ENDIF
L02305            IF(tracer_pblmix .GT. 0)THEN
L02306              CALL diff4d  (DT=dtbl,DZ=dz8w, SCALAR=tracer, is_scalar=.false.  &
L02307                ,RHO=rho,EXCH_H=exch_h        &
L02308                ,EXCH_M=exch_m                &
L02309                ,SCALAR_TEND=tracer_tend      &
L02310                ,NUM_SCALAR=num_tracer, PARAM_FIRST_SCALAR=param_first_scalar &
L02311                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde        &
L02312                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme        &
L02313                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte)
L02314            ENDIF
L02315
L02316       IF (distributed_ahe_opt == 1) THEN
L02317         call cal_mon_day(julday, julyr, jmonth, jday)
L02318         ihour = (jmonth - 1) * 24 + MOD(INT(gmt + xtime / 60.0), 24)
L02319         DO j = jts, jte
L02320         DO i = its, ite
L02321           ! Volumetric heat capacity of air = 1200 J/(K m3)
L02322           RTHBLTEN(i, 1, j) = RTHBLTEN(i, 1, j) + ahe(i, ihour, j) / 1200 / DZ8W(i, 1, j)
L02323         END DO
L02324         END DO
L02325       END IF
L02326
L02327     ENDDO
L02328     !$OMP END PARALLEL DO
L02329
L02330     ENDIF
L02331
L02332     END SUBROUTINE pbl_driver
```

讲解：①idiff=1 的统一隐式扩散出口 diff3d（当前休眠——L1021 硬编码 idiff=0）；②scalar_pblmix/tracer_pblmix>0 时对化学物种/示踪物调用 diff4d 做「用 PBL 交换系数的垂直混合」——WRF-Chem 与通用 tracer 的零额外物理复用；③ **分布式人为热 AHE**（distributed_ahe_opt=1）：cal_mon_day 把日期折算成 ahe 数组的月-时索引，把小时级人为热通量除以空气容积热容 1200 J/(K·m³) 与最低层厚度，直接加进最低层 RTHBLTEN——论文 2 的 ahe 数据集（geog y2050_ahe）正是走这个开关进场；④END SUBROUTINE 前的 ENDIF 闭合 run_param——本步未触发时全部逻辑被跳过。

四、SPP 与通用扩散辅助件

```
!file: phys/module_pbl_driver.F    L2334-L2384    (subroutine Add_multi_perturb_pbl_perturbations)
L02334     subroutine  Add_multi_perturb_pbl_perturbations (perts_qvapor, perts_qcloud, &
L02335                     perts_th, perts_tke, pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke, &
L02336                     th_phy, qke, qke_adv, qv_curr, qc_curr, bl_mynn_tkeadvect, qke_tmp, its, ite, jts, jte, &
L02337                     ims, ime, jms, jme, kms, kme, kts, kte)
L02338
L02339
L02340       implicit none
L02341
L02342       integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L02343       logical, optional, intent(in) :: bl_mynn_tkeadvect
L02344       real, intent(in) :: pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke
L02345       real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_qvapor, &
L02346         perts_qcloud, perts_th, perts_tke
L02347       real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: th_phy, qke, qv_curr, qc_curr
L02348       real, optional, dimension (ims:ime, kms:kme, jms:jme), intent(inout) :: qke_adv
L02349       real, dimension (its:ite, kts:kte, jts:jte), intent(out) :: qke_tmp
L02350
L02351       integer :: i, j, k
L02352
L02353
L02354       do j = jts, jte
L02355         do k = kts, kte
L02356           do i = its, ite
L02357             qc_curr(i, k, j) = max (QCLOUD_MIN, (1.0 + perts_qcloud(i, k, j) * pert_mynn_qc) * qc_curr(i, k, j))
L02358             qv_curr(i, k, j) = max (QVAPOR_MIN, (1.0 + perts_qvapor(i, k, j) * pert_mynn_qv) * qv_curr(i, k, j))
L02359             th_phy(i, k, j) = (1.0 + perts_th(i, k, j) * pert_mynn_t) * th_phy(i, k, j)
L02360           end do
L02361         end do
L02362       end do
L02363
L02364       if (bl_mynn_tkeadvect) then
L02365         do j = jts, jte
L02366           do k = kts, kte
L02367             do i = its, ite
L02368               qke_tmp(i, k, j) = qke_adv(i, k, j)
L02369               qke_adv(i, k, j) = max (QKE_MIN, (1.0 + perts_tke(i, k, j) * pert_mynn_qke) * qke_adv(i, k, j))
L02370             end do
L02371           end do
L02372         end do
L02373       else
L02374         do j = jts, jte
L02375           do k = kts, kte
L02376             do i = its, ite
L02377               qke_tmp(i, k, j) = qke(i, k, j)
L02378               qke(i, k, j) = max (QKE_MIN, (1.0 + perts_tke(i, k, j) * pert_mynn_qke) * qke(i, k, j))
L02379             end do
L02380           end do
L02381         end do
L02382       end if
L02383
L02384     end subroutine  Add_multi_perturb_pbl_perturbations
```

```
!file: phys/module_pbl_driver.F    L2386-L2435    (subroutine Remove_multi_perturb_pbl_perturbations)
L02386     subroutine  Remove_multi_perturb_pbl_perturbations (perts_qvapor, perts_qcloud, &
L02387                     perts_th, perts_tke, pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke, &
L02388                     th_phy, qke, qke_adv, qv_curr, qc_curr, bl_mynn_tkeadvect, qke_tmp, its, ite, jts, jte, &
L02389                     ims, ime, jms, jme, kms, kme, kts, kte)
L02390
L02391
L02392       implicit none
L02393
L02394       integer, intent(in) :: its, ite, jts, jte, ims, ime, jms, jme, kms, kme, kts, kte
L02395       logical, optional, intent(in) :: bl_mynn_tkeadvect
L02396       real, intent(in) :: pert_mynn_qv, pert_mynn_qc, pert_mynn_t, pert_mynn_qke
L02397       real, dimension(ims:ime, kms:kme, jms:jme), optional, intent (in) :: perts_qvapor, &
L02398         perts_qcloud, perts_th, perts_tke
L02399       real, dimension(ims:ime, kms:kme, jms:jme), intent (inout) :: th_phy, qke, qv_curr, qc_curr
L02400       real, optional, dimension (ims:ime, kms:kme, jms:jme), intent(inout) :: qke_adv
L02401       real, dimension (its:ite, kts:kte, jts:jte), intent(in) :: qke_tmp
L02402
L02403       integer :: i, j, k
L02404
L02405
L02406       do j = jts, jte
L02407         do k = kts, kte
L02408           do i = its, ite
L02409             qc_curr(i, k, j) = max (QCLOUD_MIN, qc_curr(i, k, j) / (1.0 + perts_qcloud(i, k, j) * pert_mynn_qc))
L02410             qv_curr(i, k, j) = max (QVAPOR_MIN, qv_curr(i, k, j) / (1.0 + perts_qvapor(i, k, j) * pert_mynn_qv))
L02411             th_phy(i, k, j) = th_phy(i, k, j) / (1.0 + perts_th(i, k, j) * pert_mynn_t)
L02412           end do
L02413         end do
L02414       end do
L02415
L02416       if (bl_mynn_tkeadvect) then
L02417         do j = jts, jte
L02418           do k = kts, kte
L02419             do i = its, ite
L02420               qke_adv(i, k, j) = max (QKE_MIN, qke_adv(i, k, j) - perts_tke(i, k, j) * pert_mynn_qke * qke_tmp(i, k, j))
L02421               qke(i, k, j) = qke_adv(i, k, j)
L02422             end do
L02423           end do
L02424         end do
L02425       else
L02426         do j = jts, jte
L02427           do k = kts, kte
L02428             do i = its, ite
L02429               qke(i, k, j) = max (QKE_MIN, qke(i, k, j) - perts_tke(i, k, j) * pert_mynn_qke * qke_tmp(i, k, j))
L02430             end do
L02431           end do
L02432         end do
L02433       end if
L02434
```

```
L02435     end subroutine  Remove_multi_perturb_pbl_perturbations
```

讲解：SPP 乘性扰动的成对实现：Add 把 (1+pattern • 幅度) 乘在 qv/qc/th 上（下限钳到 QCLOUD_MIN/QVAPOR_MIN），qke/qke_adv 同法但多存一份 qke_tmp；Remove 对 qv/qc/th 用**除法**还原、对 qke 用**减法**（qke−pattern • 幅度 • qke_tmp）——除法在扰动为负时可能把 qke 打到 0 以下，减法加 max(QKE_MIN, •) 保证非负。这是「同一物理量两套还原公式」的活教材：改 SPP 幅度时两处语义都要顾。

```
!file: phys/module_pbl_driver.F    L2438-L2639    (subroutine diff3d)
L02438            SUBROUTINE diff3d(DT,CP,DZ,TH ,QV,QC,T,U,V,RHO                          &
L02439                ,EXCH_H,EXCH_M                    &
L02440                ,RUBLTEN,RVBLTEN,RTHBLTEN    &
L02441                ,RQVBLTEN,RQCBLTEN                    &
L02442                ,WU,WV,WT,WQ                &
L02443                ,A_U,A_V,A_T,A_Q      &
L02444                ,B_U,B_V,B_T,B_Q      &
L02445                ,SF,VL       &
L02446                ,IDS,IDE,JDS,JDE,KDS,KDE      &
L02447                ,IMS,IME,JMS,JME,KMS,KME      &
L02448                ,ITS,ITE,JTS,JTE,KTS,KTE      &
L02449                                                                )
L02450  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02451  !    Subroutine written by Alberto Martilli, CIEMAT, Spain,
L02452  !    e-mail:alberto_martilli@ciemat.es
L02453  !    August 2008.
L02454  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02455  ! ALberto
L02456  ! This routine solves the vertical diffusion
L02457  ! + other source/sink terms (surface fluxes, building drag, non local terms, etc.)
L02458  ! for U,V, potential temperature and moisture. The equation should be written
L02459  ! as follow, for a generic variable M:
L02460  !
L02461  !      dM      1     d      dM
L02462  !     ---- =----  ---(rho*K----)+AM+B
L02463  !      dt     rho    dz     dz
L02464  ! where A and B are the implict and explcit coefficients of the source/sink terms
L02465  ! (at the lowest level the surface fluxes should go in these terms)
L02466  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02467  !-----------------------------------------------------------------------
L02468
L02469        IMPLICIT NONE
L02470  !
L02471  !-----------------------------------------------------------------------
L02472        INTEGER,INTENT(IN) :: IDS,IDE,JDS,JDE,KDS,KDE                    &
L02473       &                     ,IMS,IME,JMS,JME,KMS,KME                    &
L02474       &                     ,ITS,ITE,JTS,JTE,KTS,KTE
L02475
L02476  ! INputs
L02477        real DT,CP
L02478        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: DZ ! Depth of vertical levels
L02479        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: TH ! Potential Temperature
L02480        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: QV ! water vapor mixing ratio (kg/kg)
L02481        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: QC ! cloud water mixing ratio (kg/kg)
L02482        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: T  ! temperature
L02483        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: U ! X-compnent of the wind
L02484        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: V ! Y-compnent of the wind
L02485        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: RHO ! Air Density
L02486        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: EXCH_H ! Turbulent coefficient for heat
L02487        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: EXCH_M ! Turbulent coefficient for momentum
L02488
L02489        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: A_U ! Implicit coefficient for DRAG (x -component)
L02490        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: B_U ! Explicit coefficient for DRAG (x -component)
L02491        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: A_V ! Implicit coefficient for DRAG (y -component)
L02492        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: B_V ! Explicit coefficient for DRAG (y -component)
L02493        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: A_T ! Implicit coefficient for Heat flux from buildings
L02494        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: B_T ! Explicit coefficient for Heat flux from buildings
L02495        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: A_Q ! Implicit coefficient for moisture flux
L02496        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: B_Q ! Explicit coefficient for moisture flux
L02497
L02498        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: VL ! fraction of air volume in the grid cell
L02499        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: SF ! fraction of air at the face of the grid cell
L02500  !outputs
L02501        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: RUBLTEN ! tendency for x-wind component
L02502        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: RVBLTEN ! tendency for y-wind component
L02503        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: RTHBLTEN ! tendency for Potential Temperature
L02504        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: RQVBLTEN ! tendency for water vapor mixing ratio (kg/kg)
L02505        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: RQCBLTEN ! tendency for cloud water mixing ratio (kg/kg)
L02506        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: WU ! turbulent flux of momentum (x)
L02507        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: WV ! turbulent flux of momentum (y)
L02508        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: WT ! turbulent flux of potential temperature
L02509        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(OUT) :: WQ ! turbulent flux of water vapor
L02510  ! Parameters
L02511       REAL ELOCP
L02512  ! locals (same meaning as above, but 1D)
L02513
L02514        real u1d(kms:kme),v1d(kms:kme),exch_h1d(kms:kme)
L02515        real the1d(kms:kme) ! Equivalent potential temperature
L02516        real exch_m1d(kms:kme),qv1d(kms:kme),qc1d(kms:kme)
L02517        real dz1d(kms:kme),rho1d(kms:kme),rhoz1d(kms:kme)
L02518        real sf1d(kms:kme),vl1d(kms:kme)
L02519        real a_u1d(kms:kme),b_u1d(kms:kme)
L02520        real a_v1d(kms:kme),b_v1d(kms:kme)
L02521        real a_t1d(kms:kme),b_t1d(kms:kme)
L02522        real a_q1d(kms:kme),b_q1d(kms:kme)
L02523        real a_qc1d(kms:kme),b_qc1d(kms:kme)
L02524        real wu1d(kms:kme),wv1d(kms:kme),wt1d(kms:kme),wq1d(kms:kme),wqc1d(kms:kme)
L02525        real thnew
L02526  !
L02527        integer i,k,j
L02528  !-----------------------------------------------------------------------
L02529        ELOCP=2.72E6/CP
L02530        u1d=0.
L02531        v1d=0.
```

```
L02532        exch_h1d=0.
L02533        exch_m1d=0.
L02534        qv1d=0.
L02535        qc1d=0.
L02536        dz1d=0.
L02537        rho1d=0.
L02538        rhoz1d=0.
L02539        sf1d=0.
L02540        vl1d=0.
L02541        a_u1d=0.
L02542        a_v1d=0.
L02543        a_t1d=0.
L02544        a_q1d=0.
L02545        a_qc1d=0.
L02546        b_u1d=0.
L02547        b_v1d=0.
L02548        b_t1d=0.
L02549        b_q1d=0.
L02550        b_qc1d=0.
L02551
L02552        do j=jts,jte
L02553        do i=its,ite
L02554
L02555  ! put three D variables in temporary 1D variables
L02556
L02557         do k=kts,kte
L02558          u1d(k)=U(i,k,j)
L02559          v1d(k)=V(i,k,j)
L02560          the1d(k)=TH(i,k,j)*(QC(i,k,j)*(-ELOCP/T(i,k,j))+1)
L02561          qv1d(k)=qv(i,k,j)
L02562          dz1d(k)=dz(i,k,j)
L02563          rho1d(k)=rho(i,k,j)
L02564          a_u1d(k)=a_u(i,k,j)
L02565          b_u1d(k)=b_u(i,k,j)
L02566          a_v1d(k)=a_v(i,k,j)
L02567          b_v1d(k)=b_v(i,k,j)
L02568          a_t1d(k)=a_t(i,k,j)
L02569          b_t1d(k)=b_t(i,k,j)
L02570          a_q1d(k)=a_q(i,k,j)
L02571          b_q1d(k)=b_q(i,k,j)
L02572          a_qc1d(k)=0.
L02573          b_qc1d(k)=0.
L02574          vl1d(k)=vl(i,k,j)
L02575          sf1d(k)=sf(i,k,j)
L02576         enddo
L02577         sf1d(kte+1)=1.
L02578         do k=kts,kte
L02579          exch_h1d(k)=exch_h(i,k,j)
L02580          exch_m1d(k)=exch_m(i,k,j)
L02581         enddo
L02582         exch_h1d(kts)=0.
L02583  !       exch_h1d(kte+1)=0
L02584        exch_m1d(kts)=0.
L02585  !       exch_m1d(kte+1)=0
L02586         rhoz1d(kts)=rho1d(kts)
L02587         do k=kts+1,kte
L02588          rhoz1d(k)=(rho1d(k)*dz1d(k-1)+rho1d(k-1)*dz1d(k))/       &
L02589       &                       (dz1d(k-1)+dz1d(k))
L02590         enddo
L02591         rhoz1d(kte+1)=rho1d(kte)
L02592
L02593
L02594  ! solve the diffusion for x-component of the wind
L02595
L02596         call diff(kms,kme,kts,kte,dt,u1d,rho1d,rhoz1d,exch_m1d,a_u1d,b_u1d,sf1d, &
L02597       &            vl1d,dz1d,wu1d)
L02598
L02599  ! solve the diffusion for y-component of the wind
L02600
L02601         call diff(kms,kme,kts,kte,dt,v1d,rho1d,rhoz1d,exch_m1d,a_v1d,b_v1d,sf1d, &
L02602       &            vl1d,dz1d,wv1d)
L02603
L02604  ! solve the diffusion for equivalent potential temperature
L02605
L02606         call diff(kms,kme,kts,kte,dt,the1d,rho1d,rhoz1d,exch_h1d,a_t1d,b_t1d,sf1d, &
L02607       &            vl1d,dz1d,wt1d)
L02608
L02609  ! solve the diffusion for the water vapor mixing ratio
L02610
L02611         call diff(kms,kme,kts,kte,dt,qv1d,rho1d,rhoz1d,exch_h1d,a_q1d,b_q1d,sf1d, &
L02612       &            vl1d,dz1d,wq1d)
L02613
L02614  ! solve the diffusion for the cloud water mixing ratio
L02615
L02616         call diff(kms,kme,kts,kte,dt,qc1d,rho1d,rhoz1d,exch_h1d,a_qc1d,b_qc1d,sf1d, &
L02617       &            vl1d,dz1d,wqc1d)
L02618
L02619  !
L02620  ! compute the tendencies
L02621
L02622          do k=kts,kte
L02623            rublten(i,k,j)=(u1d(k)-u(i,k,j))/dt
L02624            rvblten(i,k,j)=(v1d(k)-v(i,k,j))/dt
L02625            thnew=the1d(k)/(QC(i,k,j)*(-ELOCP/T(i,k,j))+1)
L02626            rthblten(i,k,j)=(thnew-th(i,k,j))/dt
L02627            rqvblten(i,k,j)=(qv1d(k)-qv(i,k,j))/dt
L02628            rqcblten(i,k,j)=(qc1d(k)-qc(i,k,j))/dt
L02629            wu(i,k,j)=wu1d(k)
L02630            wv(i,k,j)=wv1d(k)
L02631            wt(i,k,j)=wt1d(k)
L02632            wq(i,k,j)=wq1d(k)
L02633          enddo
L02634        enddo
L02635        enddo
L02636  !!!!!!!!!!!!
L02637
L02638
L02639        END SUBROUTINE diff3d
```

讲解：Martilli 2008 写的通用隐式扩散求解器（为 BEP 设计，driver 层共用）：解 ∂M/∂t=(1/ρ)∂(ρK∂M/∂z)/∂z+AM+B——A/B 是源汇的隐式/显式系数（最低层放地表通量、中层放建筑拖曳），sf/vl 是城市建筑对格点面积/体积的占用比。逐列降维后调 diff 组三对角、invert 解算。

```
!file: phys/module_pbl_driver.F    L2642-L2770    (subroutine diff4d)
L02642            SUBROUTINE diff4d(DT,DZ,SCALAR,IS_SCALAR,RHO             &
L02643                ,EXCH_H,EXCH_M  &
L02644                ,SCALAR_TEND    &
L02645                ,NUM_SCALAR, PARAM_FIRST_SCALAR     &
L02646                ,IDS,IDE,JDS,JDE,KDS,KDE      &
L02647                ,IMS,IME,JMS,JME,KMS,KME      &
L02648                ,ITS,ITE,JTS,JTE,KTS,KTE      &
L02649                                                                )
L02650  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02651  !    Based on subroutine written by Alberto Martilli, CIEMAT, Spain,
L02652  !    e-mail:alberto_martilli@ciemat.es
L02653  !    August 2008.
L02654  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02655  ! Alberto
L02656  ! This routine solves the vertical diffusion
L02657  ! + other source/sink terms (surface fluxes, building drag, non local terms, etc.)
L02658  ! for U,V, potential temperature and moisture. The equation should be written
L02659  ! as follow, for a generic variable M:
L02660  !
L02661  !      dM      1     d      dM
L02662  !     ---- =----  ---(rho*K----)+AM+B
L02663  !      dt     rho    dz     dz
L02664  ! where A and B are the implict and explcit coefficients of the source/sink terms
L02665  ! (at the lowest level the surface fluxes should go in these terms)
L02666  !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
L02667  !-----------------------------------------------------------------------
L02668        USE module_state_description, ONLY: &
L02669                   P_QNS, P_QNR, P_QNG, P_QT, P_QNH, P_QVOLG, P_QKE_ADV
L02670
L02671
L02672        IMPLICIT NONE
L02673  !
L02674  !-----------------------------------------------------------------------
L02675        INTEGER,INTENT(IN) :: NUM_SCALAR, PARAM_FIRST_SCALAR
L02676        INTEGER,INTENT(IN) :: IDS,IDE,JDS,JDE,KDS,KDE                    &
L02677       &                     ,IMS,IME,JMS,JME,KMS,KME                    &
L02678       &                     ,ITS,ITE,JTS,JTE,KTS,KTE
L02679
L02680  ! inputs
L02681        REAL, INTENT(IN)   :: DT
L02682        LOGICAL,INTENT(IN) :: IS_SCALAR
L02683        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: DZ ! Depth of vertical levels
L02684        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME,NUM_SCALAR),INTENT(IN) :: SCALAR ! 4d scalar mixing ratio
L02685        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: RHO ! Air Density
L02686        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: EXCH_H ! Turbulent coefficient for heat
L02687        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME),INTENT(IN) :: EXCH_M ! Turbulent coefficient for momentum
L02688
L02689  ! outputs
L02690        REAL,DIMENSION(IMS:IME,KMS:KME,JMS:JME,NUM_SCALAR),INTENT(INOUT) :: SCALAR_TEND ! 4d scalar mixing ratio tendency
L02691  ! locals (same meaning as above, but 1D)
L02692
L02693        real s1d(kms:kme),exch_h1d(kms:kme)
L02694        real exch_m1d(kms:kme)
L02695        real dz1d(kms:kme),rho1d(kms:kme),rhoz1d(kms:kme)
L02696        real sf1d(kms:kme),vl1d(kms:kme)
L02697        real a_s1d(kms:kme),b_s1d(kms:kme)
L02698        real ws1d(kms:kme)
L02699  !
L02700        integer i,k,j,im
L02701  !-----------------------------------------------------------------------
L02702        s1d=0.
L02703        exch_h1d=0.
L02704        exch_m1d=0.
L02705        rho1d=0.
L02706        rhoz1d=0.
L02707        sf1d=0.
L02708        vl1d=0.
L02709        a_s1d=0.
L02710        b_s1d=0.
L02711
L02712        DO im=PARAM_FIRST_SCALAR,NUM_SCALAR
L02713  ! need to avoid mixing scalars associated with precipitating species (e.g. Nr)
L02714         IF((IS_SCALAR .AND. im.NE.P_QNS .AND. im.NE.P_QNR .AND. im.NE.P_QNG &
L02715           .AND. im.NE.P_QNH .AND. im.NE.P_QT .AND. im.NE.P_QVOLG .AND. im.NE.P_QKE_ADV) &
L02716           .OR. (.not. IS_SCALAR)) THEN
L02717         do j=jts,jte
L02718         do i=its,ite
L02719
L02720  ! put three D variables in temporary 1D variables
L02721
L02722         do k=kts,kte
L02723          s1d(k)=SCALAR(i,k,j,im)
L02724          dz1d(k)=dz(i,k,j)
L02725          rho1d(k)=rho(i,k,j)
L02726  !        a_s1d(k)=a_s(i,k,j)   ! implicit source
L02727  !        b_s1d(k)=b_s(i,k,j)   ! explicit source
L02728          vl1d(k)=1.
L02729          sf1d(k)=1.
L02730         enddo
L02731         sf1d(kte+1)=1.
L02732         do k=kts,kte
L02733          exch_h1d(k)=exch_h(i,k,j)
L02734          exch_m1d(k)=exch_m(i,k,j)
L02735         enddo
L02736         exch_h1d(kts)=0.
L02737  !       exch_h1d(kte+1)=0
L02738         exch_m1d(kts)=0.
L02739  !       exch_m1d(kte+1)=0
L02740         rhoz1d(kts)=rho1d(kts)
L02741         do k=kts+1,kte
L02742          rhoz1d(k)=(rho1d(k)*dz1d(k-1)+rho1d(k-1)*dz1d(k))/       &
```

```
L02743       &                      (dz1d(k-1)+dz1d(k))
L02744           enddo
L02745           rhoz1d(kte+1)=rho1d(kte)
L02746
L02747
L02748  ! solve the diffusion for scalar
L02749
L02750           call diff(kms,kme,kts,kte,dt,s1d,rho1d,rhoz1d,exch_h1d,a_s1d,b_s1d,sf1d, &
L02751       &            vl1d,dz1d,ws1d)
L02752
L02753  !
L02754  ! compute the tendencies
L02755
L02756           do k=kts,kte
L02757             scalar_tend(i,k,j,im)=(s1d(k)-scalar(i,k,j,im))/dt
L02758  !          ws(i,k,j)=ws1d(k)   ! output fluxes
L02759           enddo
L02760           enddo
L02761           enddo
L02762         ELSE
L02763  !       print *,'scalar skipped im=',im, is_scalar
L02764         ENDIF
L02765
L02766         ENDDO ! im loop
L02767  !!!!!!!!!!!!
L02768
L02769
L02770         END SUBROUTINE diff4d
```

```
!file: phys/module_pbl_driver.F    L2774-L2848    (subroutine diff)
L02774         subroutine diff(kms,kme,kts,kte,dt,co,da,daz,cd,aa,bb,sf,vl,dz,fc)
L02775
L02776  !-------------------------------------------------------------------------
L02777  !           Calculation of the diffusion in 1D
L02778  !-------------------------------------------------------------------------
L02779  !  - Input:
L02780  !       nz    : number of points
L02781  !       iz1   : first calculated point
L02782  !       co    : concentration of the variable of interest
L02783  !       dz    : vertical levels
L02784  !       cd    : diffusion coefficients
L02785  !       da    : density of the air at the center
L02786  !       daz   : density of the air at the face
L02787  !       dt    : diffusion time step
L02788  !  - Output:
L02789  !       co :concentration of the variable of interest
L02790
L02791  !  - Internal:
L02792  !       cddz  : constant terms in the equations
L02793  !-------------------------------------------------------------------------
L02794
L02795           implicit none
L02796           integer iz,iz1,izf
L02797           integer kms,kme,kts,kte
L02798           real dt,dzv
L02799           real co(kms:kme),cd(kms:kme),dz(kms:kme)
L02800           real da(kms:kme),daz(kms:kme)
L02801           real cddz(kms:kme),fc(kms:kme),df(kms:kme)
L02802           real a(kms:kme,3),c(kms:kme)
L02803           real sf(kms:kme),vl(kms:kme)
L02804           real aa(kms:kme),bb(kms:kme)
L02805
L02806
L02807  ! Compute cddz=2*cd/dz
L02808
L02809           cddz(kts)=sf(kts)*daz(kts)*cd(kts)/dz(kts)
L02810           do iz=kts+1,kte
L02811            cddz(iz)=2.*sf(iz)*daz(iz)*cd(iz)/(dz(iz)+dz(iz-1))
L02812           enddo
L02813           cddz(kte+1)=sf(kte+1)*daz(kte+1)*cd(kte+1)/dz(kte)
L02814
L02815             iz1=1
L02816             izf=1
L02817
L02818             do iz=iz1,kte-1
L02819
L02820              dzv=vl(iz)*dz(iz)
L02821              a(iz,1)=-cddz(iz)*dt/dzv/da(iz)
L02822              a(iz,2)=1+dt*(cddz(iz)+cddz(iz+1))/dzv/da(iz)-aa(iz)*dt
L02823              a(iz,3)=-cddz(iz+1)*dt/dzv/da(iz)
L02824              c(iz)=co(iz)+bb(iz)*dt
L02825             enddo
L02826
L02827             do iz=kte-(izf-1),kte
L02828              a(iz,1)=0.
L02829              a(iz,2)=1
L02830              a(iz,3)=0.
L02831              c(iz)=co(iz)
L02832             enddo
L02833             call invert (kms,kme,kts,kte,a,c,co)
L02834
L02835  !let compute the fluxes, as diagnostic variable
L02836
L02837             do iz=kts,iz1
L02838              fc(iz)=0.
L02839             enddo
L02840
L02841             do iz=iz1+1,kte
L02842              fc(iz)=-(cddz(iz)*(co(iz)-co(iz-1)))/da(iz)
L02843             enddo
L02844
L02845
L02846
L02847          return
L02848          end subroutine diff
```

```
!file: phys/module_pbl_driver.F    L2851-L2883    (subroutine invert)
L02851          subroutine invert(kms,kme,kts,kte,a,c,x)
L02852
L02853  !ccccccccccccccccccccccccccccc
L02854  ! Aim: Inversion and resolution of a tridiagonal matrix
L02855  !          A X = C
L02856  ! Input:
L02857  !  a(*,1) lower diagonal (Ai,i-1)
L02858  !  a(*,2) principal diagonal (Ai,i)
L02859  !  a(*,3) upper diagonal (Ai,i+1)
L02860  !  c
L02861  ! Output
L02862  !  x     results
L02863  !ccccccccccccccccccccccccccccc
L02864
L02865          implicit none
L02866          integer kms,kme,kts,kte,in
L02867          real a(kms:kme,3),c(kms:kme),x(kms:kme)
L02868
L02869           do in=kte-1,kts,-1
L02870            c(in)=c(in)-a(in,3)*c(in+1)/a(in+1,2)
L02871            a(in,2)=a(in,2)-a(in,3)*a(in+1,1)/a(in+1,2)
L02872           enddo
L02873
L02874           do in=kts+1,kte
L02875            c(in)=c(in)-a(in,1)*c(in-1)/a(in-1,2)
L02876           enddo
L02877
L02878           do in=kts,kte
L02879            x(in)=c(in)/a(in,2)
L02880           enddo
L02881
L02882           return
L02883           end subroutine invert
```

讲解：diff4d 是 diff3d 的标量/示踪物通用版（is_scalar 区分两类池，系数仍是 exch_h/exch_m）。diff 组装三对角矩阵：cddz=2 • sf • ρ_face • cd/(dz_iz+dz_iz−1)（面在层间，几何平均厚度），隐式对角占优保证无条件稳定；顶部两层把系数置为单位阵（零通量边界）。invert 是托马斯消元的紧凑实现：自上而下消上对角、自下而上回代。三对角家族在 WRF 物理层至少还有 code_09b 讲过的 mynn tridiag 三兄弟——语义差异（是否破坏输入）是复用时的第一检查点，本 invert 原地破坏 a/c。

五、改造入口

1. **新增一个 PBL 方案**（ch20 第一级改造的完整案例）：①Registry：package 行绑定新 bl_pbl_physics 值 + state 清单；②module_state_description 生成方案常量；③ 本 driver：USE 方案模块 + 新 CASE 分支（照抄 PRESENT 守卫模板）；④ 方案实体文件（ch20 案例走 MYNN 家族路径）。
2. **改 MYNN 行为**：先读 code_09a/09b——本 driver 的 MYNN 分支只是传参与 SPP 包裹；改 namelist 透传清单时 driver 与 mynnedmf_driver 实参表必须同步。
3. **城市实验组合速查**（论文 2）：BEP/BEM（sf_urban_physics=3）× 边界层 {BouLac=8 / MYJ=2 / KEPS=17（k-ε 全耦合）/ QNSE-ShinHong 部分}；风机实验叠加 windfarm_opt；人为热走 distributed_ahe_opt=1。
4. **增量编译**：只改本文件时直接编译（无需 touch Registry）；新增 namelist 变量则必须（ch04 教训）。

六、上下游链接

- 上游：dyn_em/module_first_rk_step_part1.F（编号 31 篇，待交付）在 RK 第一步调用本 driver。
- 下游（方案实体）：MYNN-EDMF → `code_09a_module_bl_mynnedmf.md` / `code_09b_module_bl_mynnedmf.md`；面层 MYNN → `code_10_module_sf_mynnsfc.md`；其余方案（YSU/MYJ/BouLac/…）编号 05–08 段待交付。
- 教材正文：ch14_pbl_family.md（方案族总览）、ch17_urban_canopy.md（BEP 旗标与城市组合）、ch19_lanzhou_heatwave.md（namelist 实配）、ch20_modifying_wrf.md（新增方案四级步骤）、ch21_solve_em_dynamics.md（调用时序）。

code_23 — 积云参数化公共调度器（module_cumulus_driver.F：时间门控与全 CASE 分派）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_cumulus_driver.F	1902	e91ad7c7472fece6ae891a2e6d897ca8

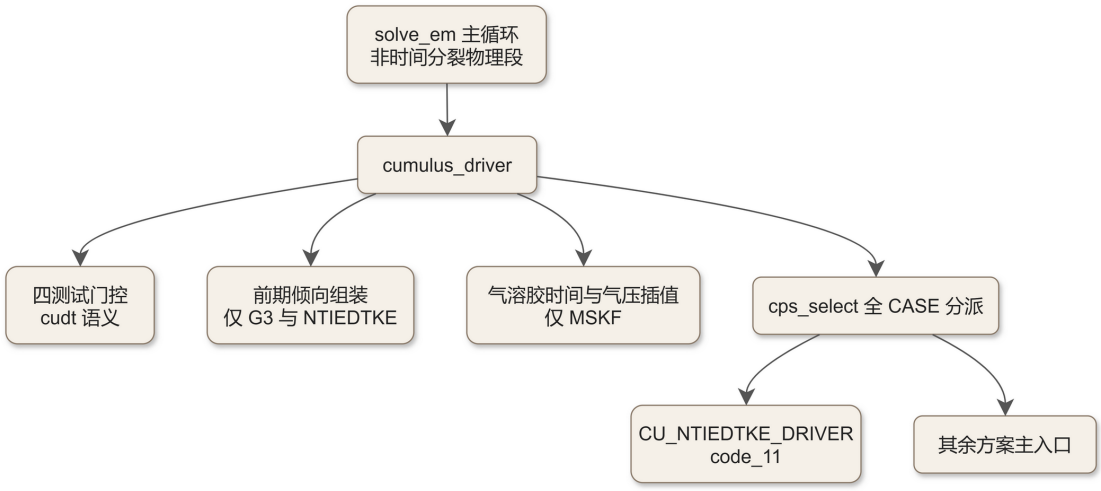
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：分派与门控全部选段走读 + 气溶胶插值两单元全文；主签名（L6–L142，约 20 组方案专用 OPTIONAL 参数军团）与变量文档注释块（L222–L380）引用不录。

二、定位

一句话：**十几个积云方案的公共调度器**——不管任何方案的物理，只管三件事：什么时候调（CUDT/STEPCU 四测试门控）、给方案什么（外部倾向组装、气溶胶插值）、按 cu_physics 的 CASE 把状态量字典式转发给对应方案主入口。

谁在调它：dyn_em/solve_em.F 的时间步主循环（RK 子步之外的非时间分裂物理段，调用序辐射 → 陆面 →PBL→ 积云 → 浅对流 →FDDA，见 ch21_solve_em_dynamics.md）；方案名单与编号的权威定义在 frame/module_state_description.F。教材锚点：ch15_cumulus_family.md 全章；ch20_modifying_wrf.md（加新 CASE 的挂点就在 cps_select）。

CASE 全家福（符号名以 module_state_description 为准）：KF、BMJ、KFETA、MSKF、GD、GFL、SAS/SCALESAS/OSAS、G3、CAMZM、TIEDTKE（旧）、NTIEDTKE（新，`code_11_module_cu_ntiedtke.md`）、KSAS、NSAS、KFCUP、DUCU（仅 WRFPLUS）。



三、USE 链与方案名单

```
!file: phys/module_cumulus_driver.F    L143-L220
L00143  !-------------------------------------------------------------------
L00144     USE module_model_constants
L00145     USE module_state_description, ONLY:     KFSCHEME,BMJSCHEME          &
L00146                                            ,KFETASCHEME,GDSCHEME        &
L00147                                            ,G3SCHEME,GFLSCHEME          &
L00148                                            ,P_QC,P_QI,Param_FIRST_SCALAR &
L00149                                            ,CAMZMSCHEME, SASSCHEME      &
L00150                                            ,OSASSCHEME                  &
L00151                                            ,SCALESASSCHEME              &  ! scale-sware sas
L00152                                            ,KSASSCHEME, NSASSCHEME      &
L00153  #if (EM_CORE == 1)
L00154                                            ,MSKFSCHEME                  &
L00155                                            ,CAMMGMPSCHEME               &
L00156                                            ,KFCUPSCHEME                 & !CuP, wig 3-Aug-2006  !BSINGH - For WRFCuP
↪  scheme
L00157                                            ,num_chem                    & !CuP, rce 10-may-2012 !BSINGH - For WRFCuP
↪  scheme
L00158  #endif
L00159  #if ( WRF_CHEM == 1)
L00160                                            ,num_tracer                  &
L00161  #endif
L00162                                            ,TIEDTKESCHEME               &
L00163                                            ,NTIEDTKESCHEME
L00164
L00165  #if ( WRF_CHEM == 1 )
L00166    USE module_chem_share, only:      get_last_gas
L00167  #endif
L00168  #if ( WRFPLUS == 1 )
L00169     USE module_state_description, ONLY:    DUCUSCHEME
L00170  #endif
L00171
L00172
L00173  ! *** add new modules of schemes here
L00174
L00175     USE module_cu_kf     ,  ONLY : kfcps
L00176     USE module_cu_bmj    ,  ONLY : bmjdrv
L00177  #ifdef DM_PARALLEL
L00178     USE module_dm        , ONLY : ntasks_x,ntasks_y,local_communicator,mytask,ntasks
L00179  #if (EM_CORE == 1)
L00180     USE module_comm_dm   ,  ONLY : halo_cup_g3_in_sub, halo_cup_g3_out_sub
L00181  #endif
L00182  #endif
L00183     USE module_domain    , ONLY: domain
L00184     USE module_cu_kfeta  , ONLY : kf_eta_cps
L00185  #if (EM_CORE==1)
L00186     USE module_cu_mskf   , ONLY : mskf_cps
L00187  #endif
L00188     USE module_cu_gd     , ONLY : grelldrv
L00189     USE module_cu_gfl    , ONLY : cu_grell_freitas_li
L00190     USE module_cu_g3     , ONLY : g3drv,conv_grell_spread3d
L00191  #if ( WRFPLUS == 1 )
L00192     USE module_cu_du     , ONLY : DUCU
L00193  #endif
L00194     USE module_cu_sas    , ONLY : cu_sas
L00195     USE module_cu_scalesas    , ONLY : cu_scalesas
L00196
L00197  #if (EM_CORE == 1)
L00198     USE module_cu_kfcup  , ONLY : KF_CUP_CPS !wig, 3-Aug-2006 !BSINGH - For WRFCuP scheme
L00199  #endif
L00200     USE module_cu_osas   , ONLY : cu_osas
L00201     USE module_cu_camzm_driver, ONLY : camzm_driver
L00202     USE module_cu_tiedtke, ONLY : cu_tiedtke
L00203     USE module_cu_ntiedtke,ONLY : cu_ntiedtke_driver
L00204     USE module_cu_ksas   , ONLY : cu_ksas
L00205     USE module_cu_nsas   , ONLY : cu_nsas
L00206     USE module_wrf_error , ONLY : wrf_err_message
L00207
L00208     !  This driver calls subroutines for the cumulus parameterizations.
L00209     !
L00210     !  1. Kain & Fritsch (1993)
L00211     !  2. Betts-Miller-Janjic (Janjic, 1994)
L00212     !  3. Grell-Devenyi (Grell and Devenyi, 2002)
L00213     !  4. Simplified Arakawa-Schubert scheme (NCEP)
L00214     !     (adapted by Zhang and Wang to work with ARW in V3.3)
L00215     !  5. Grell 3D ensemble scheme
L00216     !  6. Modified Tiedtke scheme (Zhang and Wang 2010)
L00217     ! 14. New simplified Arakawa-Schubert scheme (NCEP, YSU)
L00218     ! 16. New Tiedtke scheme (Bechtold et al. 2004, 2008, 2014)
L00219     !
L00220  !-------------------------------------------------------------------
```

讲解：①module_state_description 的符号常量导入是 CASE 判断的唯一依据——方案编号**不写在 driver 里**，加方案先改 state_description；②USE 链即方案名单：kfcps/bmjdrv/kf_eta_cps/mskf_cps/grelldrv/cu_grell_freitas_li/g3drv/cu_sas/cu_scalesas/cu_osas/camzm_driver/cu_tiedtke/cu_ntiedtke_driver/cu_ksas/cu_nsas/KF_CUP_CPS（DUCU 仅 WRFPLUS）——**每加一个方案 = 一行 USE + 一个 CASE**，这是 WRF 物理驱动器的标准扩展模式；③ 头部注释的方案清单（KF 1993、BMJ 1994、GD 2002、SAS、G3、旧/新 Tiedtke…）与实际 USE 链一致；④DM_PARALLEL 下 G3 还要用 halo_cup_g3 的通信子程序（集合成员的邻域交换）。

四、本地初始化与缺省值

```
!file: phys/module_cumulus_driver.F    L747-L798
L00747  ! To accommodate shared physics
L00748     character*256 :: errmsg
L00749     integer :: errflg
L00750
L00751  !-------------------------------------------------------------------
L00752      pattern_spp_conv=0.
L00753      field_conv=0.
L00754      spp_conv=0
L00755
L00756
L00757     l_flux=.FALSE.
L00758     if (present(f_flux)) l_flux=f_flux
L00759     if (.not. PRESENT(CURR_SECS)) then
L00760        curr_secs_pass = -1
L00761     else
L00762        curr_secs_pass = curr_secs
L00763     endif
L00764
L00765     if (.not. PRESENT(CUDT)) then
L00766        cudt_pass = -1
L00767        cudtacttime_pass = -1
L00768     else
L00769        cudt_pass = cudt
L00770        cudtacttime_pass = cudtacttime
L00771     endif
L00772
L00773     if (.not. PRESENT(adapt_step_flag)) then
L00774        adapt_step_flag_pass = .false.
L00775     else
L00776        adapt_step_flag_pass = adapt_step_flag
L00777     endif
L00778
L00779     ! Initialize tmppratec to pratec
L00780
L00781     if ( PRESENT ( pratec ) ) then
L00782        tmppratec(:,:) = pratec(:,:)
L00783     else
L00784        tmppratec(:,:) = 0.
L00785     end if
L00786
L00787     if (.not. PRESENT(kfeta_trigger)) then
L00788        trigger_kf = 1
L00789     else
L00790        trigger_kf = kfeta_trigger
L00791     endif
L00792
L00793     if (.not. PRESENT(nsas_dx_factor)) then
L00794        dx_factor_nsas = 0
L00795     else
L00796        dx_factor_nsas = nsas_dx_factor
L00797     endif
L00798
```

讲解：①spp_conv 的 pattern/field 数组本地清零——随机参数化（SPP）在 4.8.0 的积云侧是占位；②l_flux 从 optional 的 f_flux 取；③ **缺省值模板**：CURR_SECS/CUDT 缺席时传 −1（多数方案不关心，仅 CAMZM/自适应 dt 用）、adapt_step_flag 缺席为 false；④**tmppratec 暂存**——PRATEC 是 OPTIONAL，但绝大多数方案的实参表里都要它：先拷进 tmppratec，调用后按 PRESENT(PRATEC) 决定是否写回（§ 九）；⑤trigger_kf 缺省 1、dx_factor_nsas 缺省 0——两个 namelist 可调开关的兜底。

五、四测试门控：CUDT 的真实语义

```
!file: phys/module_cumulus_driver.F    L799-L864
L00799     IF (cu_physics .eq. 0) return
L00800
L00801  !  Initialization for adaptive time step.
L00802
L00803     IF ( adapt_step_flag_pass ) THEN
L00804        doing_adapt_dt = .TRUE.
L00805        IF ( cudtacttime_pass .EQ. 0. ) THEN
L00806           cudtacttime_pass = curr_secs_pass + cudt_pass*60.
L00807        END IF
L00808     ELSE
L00809        doing_adapt_dt = .FALSE.
L00810     END IF
L00811
L00812  !  Do we run through this scheme or not?
L00813
L00814  !    Test 1:  If this is the initial model time, then yes.
L00815  !             ITIMESTEP=1
L00816  !    Test 2:  If the user asked for the cumulus to be run every time step, then yes.
L00817  !             CUDT=0 or STEPCU=1
L00818  !    Test 3:  If not adaptive dt, and this is on the requested cumulus frequency, then yes.
L00819  !             MOD(ITIMESTEP,STEPCU)=0
L00820  !    Test 4:  If using adaptive dt and the current time is past the last requested activate cumulus time, then yes.
L00821  !             CURR_SECS >= CUDTACTTIME
L00822
L00823  !  If we do run through the scheme, we set the flag run_param to TRUE and we set the decided flag
L00824  !  to TRUE.  The decided flag says that one of these tests was able to say "yes", run the scheme.
L00825  !  We only proceed to other tests if the previous tests all have left decided as FALSE.
L00826
L00827  !  If we set run_param to TRUE and this is adaptive time stepping, we set the time to the next
L00828  !  cumulus run.
L00829
```

```
L00830     decided = .FALSE.
L00831     run_param = .FALSE.
L00832     IF ( ( .NOT. decided ) .AND. &
L00833          ( itimestep .EQ. 1 ) ) THEN
L00834        run_param   = .TRUE.
L00835        decided     = .TRUE.
L00836     END IF
L00837
L00838     IF ( ( .NOT. decided ) .AND. &
L00839          ( ( cudt_pass .EQ. 0. ) .OR. ( stepcu .EQ. 1 ) ) ) THEN
L00840        run_param   = .TRUE.
L00841        decided     = .TRUE.
L00842     END IF
L00843
L00844     IF ( ( .NOT. decided ) .AND. &
L00845          ( .NOT. doing_adapt_dt ) .AND. &
L00846          ( MOD(itimestep,stepcu) .EQ. 0 ) ) THEN
L00847        run_param   = .TRUE.
L00848        decided     = .TRUE.
L00849     END IF
L00850
L00851     IF ( ( .NOT. decided ) .AND. &
L00852          ( doing_adapt_dt ) .AND. &
L00853          ( curr_secs_pass .GE. cudtacttime_pass ) ) THEN
L00854        run_param   = .TRUE.
L00855        decided     = .TRUE.
L00856        cudtacttime_pass = curr_secs_pass + cudt_pass*60
L00857     END IF
L00858
L00859     IF ( run_param ) THEN
L00860  !print *,'calling CU scheme'
L00861     ELSE
L00862  !print *,'NOT calling CU scheme'
L00863        RETURN
L00864     END IF
```

讲解：cu_physics=0 直接 return（无积云）。然后是**「要不要调方案」的四测试**（注释原文即测试清单）：①itimestep=1 必调；②cudt_pass=0 或 stepcu=1 每步都调；③ 非自适应时间步下 MOD(itimestep,stepcu)=0 按周期调——**cudt（分钟）经换算给出 stepcu，门控本身只认整数周期**；④ 自适应 dt 下 curr_secs≥cudtacttime 时调，并把 cudtacttime 推进到下一激活时刻（curr_secs+cudt·60）——自适应步长下「次数」不可靠，改用「物理时刻」。decided/run_param 两旗标的短路写法是标准的「多条件或」展开。**这四测试解释了为什么 KF 系不能在这里做时间步判定**（§ 八注释：KF 需要跨步累积的 W 场，判步在方案内部自己做）。

六、前期倾向组装：G3 与 NTIEDTKE 专属

```
!file: phys/module_cumulus_driver.F    L866-L895
L00866  #if  ( EM_CORE == 1 )
L00867        if(cu_physics == G3SCHEME .or. cu_physics == NTIEDTKESCHEME) then
L00868        !$OMP PARALLEL DO   &
L00869        !$OMP PRIVATE ( ij,i,j,k,its,ite,jts,jte )
L00870
L00871        DO ij = 1 , num_tiles
L00872          its = i_start(ij)
L00873          ite = i_end(ij)
L00874          jts = j_start(ij)
L00875          jte = j_end(ij)
L00876          do j=jts,min(jte,jde-1)
L00877          do k=kts,kte
L00878          do i=its,min(ite,ide-1)
L00879             RTHFTEN(i,k,j)=(RTHFTEN(i,k,j)+RTHRATEN(i,k,j) &
L00880                                  +RTHBLTEN(i,k,j))*pi(i,k,j)
L00881             RQVFTEN(i,k,j)=RQVFTEN(i,k,j)+RQVBLTEN(i,k,j)
L00882          enddo
L00883          enddo
L00884          enddo
L00885        ENDDO
L00886        !$OMP END PARALLEL DO
L00887        endif
L00888
L00889        IF ( cu_physics == G3SCHEME .OR.  cu_physics == GFLSCHEME .OR.     &
L00890             cu_physics == KFETASCHEME .OR. cu_physics == MSKFSCHEME) THEN
L00891  #ifdef DM_PARALLEL
L00892  #include "HALO_CUP_G3_IN.inc"
L00893  #endif
L00894        ENDIF
L00895  #endif
```

讲解：cu_physics 为 G3SCHEME 或 NTIEDTKESCHEME 时，闭合式需要的「外部强迫倾向」在此组装：**RTHFTEN=(RTHFTEN+RTHRATEN+RTHBLTEN)·pi**——平流倾向（动力部分每步刷新）叠加辐射倾向与 PBL 温度倾向，再乘 Exner 函数 pi 把「温度倾向 K/s」转成「位温倾向 K/s」；**RQVFTEN 加 RQVBLTEN**。这与旧 Tiedtke 只吃 RQVBLTEN（§ 十 CASE 对照）形成鲜明对比——新 Tiedtke 的 moisture-convergence 闭合要看到全部非对流水汽强迫。注意这是 inout 数组：组装值写回 RTHFTEN/RQVFTEN，方案读取后 driver 不再还原——下个时间步动力部分会重新刷新，故无累积误差。G3/GFL/KFETA/MSKF 四案还需 HALO_CUP_G3_IN 交换（集合成员邻域）。

七、tile 循环与 MSKF 气溶胶插值

```
!file: phys/module_cumulus_driver.F    L902-L952
L00902         !$OMP PARALLEL DO   &
L00903         !$OMP PRIVATE ( ij ,its,ite,jts,jte, i,j,k)
L00904
L00905         DO ij = 1 , num_tiles
L00906           its = i_start(ij)
L00907           ite = i_end(ij)
L00908           jts = j_start(ij)
L00909           jte = j_end(ij)
L00910
L00911  #if (EM_CORE == 1)
L00912           !BSINGH - For WRFCuP scheme
L00913           !wig, beg: testing for kpbl to get output to wrfout since kpbl won't output as an integer
L00914           do j=jts,jte
L00915              do i=its,ite
L00916                 akpbl(i,j) = kpbl(i,j)
L00917              end do
L00918           end do
L00919           !wig, end.
L00920           !BSINGH - ENDS
L00921
L00922           !The following is for introducing aerosol data into the MSKF scheme !
L00923           !If the flag is set, the aerosol data will be interpolated in       !
L00924           !time and then pressure to the current state.
L00925           IF ( aercu_opt .GT. 0 ) THEN
L00926             IF ( aercu_opt .GT. 0 .AND. id .EQ. 1 ) THEN
L00927               call aer_time_int_cu(julday,julian,aeromcu,aerotcu,alevsiz_cu,num_months,no_src_types_cu,&
L00928                                    ids , ide , jds , jde , kds , kde ,     &
L00929                                    ims , ime , jms , jme , kms , kme ,     &
L00930                                    its , ite , jts , jte , kts , kte )
L00931
L00932               ! INTERPOLATE PRESSURE IN TIME
L00933               call aer_time_int_cu(julday,julian,aeropcu,aeroptcu,alevsiz_cu,num_months,1, &
L00934                                    ids , ide , jds , jde , kds , kde ,     &
L00935                                    ims , ime , jms , jme , kms , kme ,     &
L00936                                    its , ite , jts , jte , kts , kte )
L00937
L00938               call aer_p_int_cu(p ,aeroptcu, alevsiz_cu, aerotcu, aerocu, no_src_types_cu, p8w, &
L00939                                    ids , ide , jds , jde , kds , kde ,     &
L00940                                    ims , ime , jms , jme , kms , kme ,     &
L00941                                    its , ite , jts , jte , kts , kte )
L00942
L00943             do j=jts,jte
L00944                do k=kts,kte
L00945                   do i=its,ite
L00946                      aerovar(i,k,j)=aerocu(i,k,j,2)
L00947                   end do
L00948                end do
L00949             end do
L00950
L00951             ENDIF
L00952           ENDIF
```

讲解：OpenMP tile 循环从这里开始（CASE 分派整段都在 ij 循环内）；akpbl=kpbl 的整型转实型输出（kpbl 不入 wrfout 的 workaround）；**aercu_opt>0 且仅主域 id=1** 时为 MSKF 做气溶胶预处理：aer_time_int_cu 做月平均场的时间插值（调两次——分层量与气压量）、aer_p_int_cu 把数据层插到模式层、aerovar 取第 2 类源（硫酸盐类）作有效变量。气溶胶-对流交互（活化 CCN、降水效率修正）只喂给 MSKF 一个方案。

八、CASE 分派·上：KF 家族、BMJ、MSKF、GD

```
!file: phys/module_cumulus_driver.F    L956-L1128
L00956     cps_select: SELECT CASE(cu_physics)
L00957
L00958       CASE (KFSCHEME)
L00959            CALL wrf_debug(100,'in kfcps')
L00960
L00961            CALL KFCPS(                                          &
L00962                ! order independent arguments
L00963                  DT=dt ,KTAU=itimestep ,DX=dx , CUDT=cudt_pass   &
L00964                 ,ADAPT_STEP_FLAG=adapt_step_flag_pass           &
L00965                 ,RHO=rho                                        &
L00966                 ,U=u ,V=v ,TH=th ,T=t ,W=w                      &
L00967                 ,PCPS=p ,PI=pi                                  &
L00968                 ,XLV0=xlv0 ,XLV1=xlv1 ,XLS0=xls0 ,XLS1=xls1     &
L00969                 ,RAINCV=raincv, PRATEC=tmppratec, NCA=nca       &
L00970                 ,DZ8W=dz8w                                      &
L00971                 ,W0AVG=w0avg                                    &
L00972                 ,CP=cp ,R=r_d ,G=g ,EP1=ep_1 ,EP2=ep_2          &
L00973                 ,SVP1=svp1 ,SVP2=svp2 ,SVP3=svp3 ,SVPT0=svpt0   &
L00974                 ,STEPCU=stepcu                                  &
L00975                 ,CU_ACT_FLAG=cu_act_flag                        &
L00976                 ,WARM_RAIN=warm_rain                            &
L00977                 ,QV=qv_curr                                     &
L00978                 ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L00979                 ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L00980                 ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L00981                ! optionals
L00982                 ,RTHCUTEN=rthcuten ,RQVCUTEN=rqvcuten           &
L00983                 ,RQCCUTEN=rqccuten ,RQRCUTEN=rqrcuten           &
L00984                 ,RQICUTEN=rqicuten ,RQSCUTEN=rqscuten           &
L00985                 ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                  &
L00986                 ,F_QI=f_qi,F_QS=f_qs                            &
L00987                 ,UDR_KF=udr_kf,DDR_KF=ddr_kf                    & !kf_edrates
L00988                 ,UER_KF=uer_kf,DER_KF=der_kf                    &
```

```
L00989                ,TIMEC_KF=timec_kf,KF_EDRATES=kf_edrates         &
L00990                                                                 )
L00991
L00992       CASE (BMJSCHEME)
L00993            CALL wrf_debug(100,'in bmj_cps')
L00994            CALL BMJDRV(                                         &
L00995                  TH=th,T=T ,RAINCV=raincv, PRATEC=tmppratec      &
L00996                ,RHO=rho                                         &
L00997                ,DT=dt ,ITIMESTEP=itimestep ,STEPCU=stepcu       &
L00998                ,CUTOP=htop, CUBOT=hbot, KPBL=kpbl               &
L00999                ,DZ8W=dz8w ,PINT=p8w, PMID=p, PI=pi              &
L01000                ,CP=cp ,R=r_d ,ELWV=xlv ,ELIV=xls ,G=g           &
L01001                ,TFRZ=svpt0 ,D608=ep_1 ,CLDEFI=cldefi            &
L01002                ,LOWLYR=lowlyr ,XLAND=xland                      &
L01003                ,CU_ACT_FLAG=cu_act_flag                         &
L01004                ,QV=qv_curr                                      &
L01005                ,CCLDFRA=ccldfra, CONVCLD=convcld                &
L01006                ,QCCONV=qcconv, QICONV=qiconv                    &
L01007                ,BMJ_RAD_FEEDBACK=bmj_rad_feedback               &
L01008                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01009                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01010                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01011               ! optionals
L01012                ,RTHCUTEN=rthcuten ,RQVCUTEN=rqvcuten            &
L01013                                                                 )
L01014
L01015       CASE (KFETASCHEME)
L01016            CALL wrf_debug(100,'in kf_eta_cps')
L01017            CALL KF_ETA_CPS(                                     &
L01018                  U=u ,V=v ,TH=th ,T=t ,W=w ,RHO=rho              &
L01019                ,CUDT=cudt_pass                                  &
L01020                ,ADAPT_STEP_FLAG=adapt_step_flag_pass            &
L01021                ,RAINCV=raincv, PRATEC=tmppratec, NCA=nca        &
L01022                ,DZ8W=dz8w                                       &
L01023                ,PCPS=p, PI=pi ,W0AVG=W0AVG                      &
L01024                ,CUTOP=HTOP,CUBOT=HBOT                           &
L01025                ,XLV0=XLV0 ,XLV1=XLV1 ,XLS0=XLS0 ,XLS1=XLS1      &
L01026                ,CP=CP ,R=R_d ,G=G ,EP1=EP_1 ,EP2=EP_2           &
L01027                ,SVP1=SVP1 ,SVP2=SVP2 ,SVP3=SVP3 ,SVPT0=SVPT0    &
L01028                ,DT=dt ,KTAU=itimestep ,DX=dx                    &
L01029                ,STEPCU=stepcu                                   &
L01030                ,CU_ACT_FLAG=cu_act_flag ,WARM_RAIN=warm_rain    &
L01031                ,QV=qv_curr                                      &
L01032                ,SHALL=shall                                     & !BSINGH - For WRFCuP scheme added "shall"
L01033                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01034                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01035                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01036                ,trigger=trigger_kf                              &
L01037               ! optionals
L01038                ,RTHCUTEN=rthcuten                               &
L01039                ,RQVCUTEN=rqvcuten ,RQCCUTEN=rqccuten            &
L01040                ,RQRCUTEN=rqrcuten ,RQICUTEN=rqicuten            &
L01041                ,RQSCUTEN=rqscuten, RQVFTEN=RQVFTEN              &
L01042                ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                   &
L01043                ,F_QI=f_qi,F_QS=f_qs                             &
L01044                ,CLDFRA_DP_KF=cldfra_dp                          & ! ckay for sub-grid cloud
L01045                ,CLDFRA_SH_KF=cldfra_sh                          &
L01046                ,QC_KF=QC_CU,QI_KF=QI_CU                         &
L01047                ,UDR_KF=udr_kf,DDR_KF=ddr_kf                     & !kf_edrates
L01048                ,UER_KF=uer_kf,DER_KF=der_kf                     &
L01049                ,TIMEC_KF=timec_kf,KF_EDRATES=kf_edrates         )
L01050  #if (EM_CORE==1)
L01051       CASE (MSKFSCHEME)
L01052            CALL wrf_debug(100,'in mskf_cps_mp')
L01053
L01054            CALL MSKF_CPS(                                       &
L01055                  U=u ,V=v ,TH=th ,T=t ,W=w ,RHO=rho              &
L01056                ,CUDT=cudt_pass                                  &
L01057                ,ADAPT_STEP_FLAG=adapt_step_flag_pass            &
L01058                ,RAINCV=raincv, PRATEC=tmppratec, NCA=nca        &
L01059                ,DZ8W=dz8w                                       &
L01060                ,PCPS=p, PI=pi ,W0AVG=W0AVG                      &
L01061                ,CUTOP=HTOP,CUBOT=HBOT                           &
L01062                ,XLV0=XLV0 ,XLV1=XLV1 ,XLS0=XLS0 ,XLS1=XLS1      &
L01063                ,CP=CP ,R=R_d ,G=G ,EP1=EP_1 ,EP2=EP_2           &
L01064                ,SVP1=SVP1 ,SVP2=SVP2 ,SVP3=SVP3 ,SVPT0=SVPT0    &
L01065                ,DT=dt ,KTAU=itimestep ,DX=dx                    &
L01066                ,STEPCU=stepcu                                   &
L01067                ,CU_ACT_FLAG=cu_act_flag ,WARM_RAIN=warm_rain    &
L01068                ,QV=qv_curr                                      &
L01069                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01070                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01071                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01072                ,trigger=trigger_kf                              &
L01073               ! optionals
L01074                ,RTHCUTEN=rthcuten                               &
L01075                ,RQVCUTEN=rqvcuten ,RQCCUTEN=rqccuten            &
L01076                ,RQRCUTEN=rqrcuten ,RQICUTEN=rqicuten            &
L01077                ,RQSCUTEN=rqscuten, RQVFTEN=RQVFTEN              &
L01078                ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                   &
L01079                ,F_QI=f_qi,F_QS=f_qs                             &
L01080                ,CLDFRA_DP_KF=cldfra_dp                          & ! ckay for sub-grid cloud
L01081                ,CLDFRA_SH_KF=cldfra_sh                          &
L01082                ,QC_KF=QC_CU,QI_KF=QI_CU,QR_KF=QR_CU,QS_KF=QS_CU &
L01083                ,NC_KF=NC_CU,NI_KF=NI_CU,NR_KF=NR_CU,NS_KF=NS_CU &
L01084                ,CCN_KF=CCN_CU,AINC_FRAC=CU_UAF                  &
L01085                ,UDR_KF=udr_kf,DDR_KF=ddr_kf                     & !kf_edrates
L01086                ,UER_KF=uer_kf,DER_KF=der_kf                     &
L01087                ,TIMEC_KF=timec_kf,KF_EDRATES=kf_edrates         &
L01088                ,ZOL=zol,HFX=hfx,UST=ust,PBLH=pblh               & !ckay
L01089                ,AEROCU=aerocu,NO_SRC_TYPES_CU=no_src_types_cu   &
L01090                ,AERCU_FCT=aercu_fct,AERCU_OPT=aercu_opt        &
L01091                ,EFCS=EFCS,EFIS=EFIS,EFSS=EFSS                   &
L01092                ,RUCUTEN=RUCUTEN,RVCUTEN=RVCUTEN,XLAND=XLAND)       !JTR
L01093  #endif
L01094
L01095       CASE (GDSCHEME)
L01096            CALL wrf_debug(100,'in grelldrv')
L01097            CALL GRELLDRV(                                       &
```

```
L01098                DT=dt, ITIMESTEP=itimestep, DX=dx                &
L01099               ,U=u,V=v,T=t,W=w ,RHO=rho                         &
L01100               ,P=p,PI=pi ,Q=qv_curr ,RAINCV=raincv              &
L01101               ,DZ8W=dz8w,P8W=p8w,XLV=xlv,CP=cp,G=g,R_V=r_v      &
L01102               ,PRATEC=tmppratec                                 &
L01103               ,APR_GR=apr_gr,APR_W=apr_w,APR_MC=apr_mc          &
L01104               ,APR_ST=apr_st,APR_AS=apr_as                      &
L01105               ,APR_CAPMA=apr_capma,APR_CAPME=apr_capme          &
L01106               ,APR_CAPMI=apr_capmi,MASS_FLUX=mass_flux          &
L01107               ,XF_ENS=xf_ens,PR_ENS=pr_ens,HT=ht                &
L01108               ,xland=xland,gsw=gsw                              &
L01109               ,GDC=gd_cloud,GDC2=gd_cloud2 &
L01110               ,ENSDIM=ensdim,MAXIENS=maxiens,MAXENS=maxens      &
L01111               ,MAXENS2=maxens2,MAXENS3=maxens3                  &
L01112               ,htop=htop,hbot=hbot                              &
L01113               ,ktop_deep=ktop_deep                              &
L01114               ,CU_ACT_FLAG=CU_ACT_FLAG,warm_rain=warm_rain      &
L01115               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01116               ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01117               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01118               ,PERIODIC_X=periodic_x,PERIODIC_Y=periodic_y      &
L01119              ! optionals
L01120               ,RTHCUTEN=RTHCUTEN ,RTHFTEN=RTHFTEN               &
L01121               ,RQICUTEN=RQICUTEN ,RQVFTEN=RQVFTEN               &
L01122               ,RTHRATEN=RTHRATEN,RTHBLTEN=RTHBLTEN              &
L01123               ,RQVCUTEN=RQVCUTEN,RQCCUTEN=RQCCUTEN              &
L01124               ,RQVBLTEN=RQVBLTEN                                &
L01125               ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                    &
L01126               ,F_QI=f_qi,F_QS=f_qs                              &
L01127               ,CFU1=CFU1,CFD1=CFD1,DFU1=DFU1,EFU1=EFU1          &
L01128               ,DFD1=DFD1,EFD1=EFD1,f_flux=l_flux                )
```

讲解：全 CASE 公共模式先立此存照：关键字实参字典、主倾向对 RTHCUTEN/RQVCUTEN、PRATEC=tmppratec、CU_ACT_FLAG 透传、F_Q* 压缩开关、IDS…KTE 六套索引。分案差异：**KF**——W0AVG（跨步累积垂直速度，KF 判步在内部）、NCA 云弛豫计数器、kf_edrates 四组诊断收支（udr/ddr/uer/der）；**BMJ**——最简参表（只回 RTHCUTEN/RQVCUTEN），加 CONVCLD/QCCONV 对流云水与 CLDEFI 降水效率、bmj_rad_feedback 开关（BMJ 温度倾向回溃给辐射）；**KFETA**——trigger=trigger_kf（namelist kfeta_trigger 选触发函数）、cldfra_dp/cldfra_sh 亚格深/浅云量、QC_KF/QI_KF 方案内云水冰诊断；**MSKF**（仅 EM_CORE）——在 KFETA 基础上加气溶胶全套（AEROCU/EFCS 效率系数组）与动量倾向 RUCUTEN/RVCUTEN、ZOL/HFX/UST/PBLH 边界层交互量；**GD（Grell–Devenyi）**——集合参数五件套 ensdim/maxiens/maxens/maxens2/maxens3 与 XF_ENS/PR_ENS 成员场、APR 系逐过程降水收支（apr_gr/w/mc/st/as）、质量通量诊断 CFU1 系六数组（f_flux 开关）。

九、CASE 分派·中：SAS 三兄弟、G3、GFL

```
!file: phys/module_cumulus_driver.F    L1131-L1296
L01131       CASE (SASSCHEME)
L01132
↪
L01133            IF ( adapt_step_flag_pass ) THEN
L01134              WRITE( wrf_err_message , * ) 'The SAS cumulus option will not work properly with an adaptive time step'
L01135              CALL wrf_error_fatal ( wrf_err_message )
L01136            END IF
L01137            CALL wrf_debug(100,'in cu_sas')
L01138            CALL CU_SAS(                                        &
L01139                 DT=dt,ITIMESTEP=itimestep,STEPCU=STEPCU        &
L01140                ,RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN            &
L01141                ,RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN            &
L01142                ,RUCUTEN=RUCUTEN, RVCUTEN=RVCUTEN               &
L01143                ,RAINCV=RAINCV,PRATEC=tmpPRATEC,HTOP=HTOP,HBOT=HBOT &
L01144                ,U3D=u,V3D=v,W=w,T3D=t                          &
L01145                ,QV3D=QV_CURR,QC3D=QC_CURR,QI3D=QI_CURR         &
L01146                ,PI3D=pi,RHO3D=rho                              &
L01147                ,DZ8W=dz8w,PCPS=p,P8W=p8w,XLAND=XLAND           &
L01148                ,CU_ACT_FLAG=CU_ACT_FLAG                        &
L01149                ,P_QC=p_qc                                      &
L01150                ,MOMMIX=MOMMIX  &
L01151                ,pgcon=pgcon,sas_mass_flux=sas_mass_flux        &
L01152                ,pert_sas=pert_sas,ens_random_seed=ens_random_seed   &
L01153                ,ens_sasamp=ens_sasamp &
L01154                ,shalconv=shalconv,shal_pgcon=shal_pgcon        &
L01155                ,hpbl2d=hpbl2d,evap2d=evap2d,heat2d=heat2d      &
L01156                ,P_QI=p_qi,P_FIRST_SCALAR=param_first_scalar    &
L01157                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01158                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01159                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01160                                                                )
L01161       CASE (SCALESASSCHEME)   ! 2015-12-11 added to call scale-aware SAS
L01162
L01163            IF ( adapt_step_flag_pass ) THEN
L01164              WRITE( wrf_err_message , * ) 'The SCALE-AWARE SAS cumulus option will not work properly with an adaptive
↪  time step'
L01165              CALL wrf_error_fatal ( wrf_err_message )
L01166            END IF
L01167            CALL wrf_debug(100,'in cu_scalesas')
L01168            CALL CU_SCALESAS(                                   &
L01169                 DT=dt,ITIMESTEP=itimestep,STEPCU=STEPCU        &
L01170                ,RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN            &
L01171                ,RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN            &
L01172                ,RUCUTEN=RUCUTEN, RVCUTEN=RVCUTEN               &
L01173                ,RAINCV=RAINCV,PRATEC=tmpPRATEC,HTOP=HTOP,HBOT=HBOT &
L01174                ,U3D=u,V3D=v,W=w,T3D=t                          &
L01175                ,QV3D=QV_CURR,QC3D=QC_CURR,QI3D=QI_CURR         &
```

```
L01176                ,PI3D=pi,RHO3D=rho                                    &
L01177                ,DZ8W=dz8w,PCPS=p,P8W=p8w,XLAND=XLAND                 &
L01178                ,CU_ACT_FLAG=CU_ACT_FLAG                              &
L01179                ,P_QC=p_qc                                            &
L01180                ,MOMMIX=MOMMIX  &
L01181                ,pgcon=pgcon,sas_mass_flux=sas_mass_flux              &
L01182                ,shalconv=shalconv,shal_pgcon=shal_pgcon              &
L01183                ,hpbl2d=hpbl2d,evap2d=evap2d,heat2d=heat2d            &
L01184                ,P_QI=p_qi,P_FIRST_SCALAR=param_first_scalar          &
L01185                ,DX2D=dx2d,DY=dynmm                                   & ! 2 new
L01186                ,SCALEFUN=SCALEFUN,SCALEFUN1=SCALEFUN1                & ! CNV scale functions
L01187                ,SIGMU=SIGMU,SIGMU1=SIGMU1                            & ! CNV updraft fractions
L01188                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde      &
L01189                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01190                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte      &
L01191                                                                      )
L01192       CASE (OSASSCHEME)
L01193
↪
L01194            IF ( adapt_step_flag_pass ) THEN
L01195              WRITE( wrf_err_message , * ) 'The SAS cumulus option will not work properly with an adaptive time step'
L01196              CALL wrf_error_fatal ( wrf_err_message )
L01197            END IF
L01198            CALL wrf_debug(100,'in cu_osas')
L01199            CALL CU_OSAS(                                              &
L01200                 DT=dt,ITIMESTEP=itimestep,STEPCU=STEPCU               &
L01201                ,RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN                   &
L01202                ,RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN                   &
L01203                ,RUCUTEN=RUCUTEN, RVCUTEN=RVCUTEN                      &
L01204                ,RAINCV=RAINCV,PRATEC=tmpPRATEC,HTOP=HTOP,HBOT=HBOT &
L01205                ,U3D=u,V3D=v,W=w,T3D=t                                 &
L01206                ,QV3D=QV_CURR,QC3D=QC_CURR,QI3D=QI_CURR                &
L01207                ,PI3D=pi,RHO3D=rho                                     &
L01208                ,DZ8W=dz8w,PCPS=p,P8W=p8w,XLAND=XLAND                  &
L01209                ,CU_ACT_FLAG=CU_ACT_FLAG                               &
L01210                ,P_QC=p_qc                                             &
L01211                ,store_rand=store_rand  &
L01212                ,MOMMIX=MOMMIX  &
L01213                ,P_QI=p_qi,P_FIRST_SCALAR=param_first_scalar           &
L01214                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
L01215                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme       &
L01216                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte       &
L01217                                                                       )
L01218       CASE (G3SCHEME)
L01219            CALL wrf_debug(100,'in grelldrv')
L01220  #if ( WRF_DFI_RADAR == 1 )
L01221            if (do_capsuppress == 1) then
L01222              WRITE( wrf_err_message , * ) 'G3 do CAP suppress',its,jts,min( jte,jde-1 ),min( ite,ide-1 ),kte
L01223              CALL wrf_debug(200, wrf_err_message)
L01224              DO j = jts, min( jte,jde-1 )
L01225              DO i = its, min( ite,ide-1 )
L01226                 cap_suppress_loc(i,j) = grid%dfi_tten_rad(i,kte,j)
L01227              ENDDO
L01228              ENDDO
L01229            endif
L01230  #endif
L01231            CALL G3DRV(                                                &
L01232                 DT=dt, ITIMESTEP=itimestep, DX=dx                     &
L01233                ,U=u,V=v,T=t,W=w ,RHO=rho                              &
L01234                ,P=p,PI=pi,Q=qv_curr,RAINCV=raincv                     &
L01235                ,DZ8W=dz8w ,P8W=p8w,XLV=xlv,CP=cp,G=g,R_V=r_v          &
L01236                ,APR_GR=apr_gr,APR_W=apr_w,APR_MC=apr_mc               &
L01237                ,APR_ST=apr_st,APR_AS=apr_as,PRATEC=tmppratec          &
L01238                ,APR_CAPMA=apr_capma,APR_CAPME=apr_capme               &
L01239                ,APR_CAPMI=apr_capmi,MASS_FLUX=mass_flux               &
L01240                ,XF_ENS=xf_ens,PR_ENS=pr_ens,HT=ht                     &
L01241                ,xland=xland,gsw=gsw,edt_out=edt_out                   &
L01242                ,GDC=gd_cloud,GDC2=gd_cloud2,kpbl=kpbl                 &
L01243                ,k22_shallow=k22_shallow                               &
L01244                ,kbcon_shallow=kbcon_shallow                           &
L01245                ,ktop_shallow=ktop_shallow                             &
L01246                ,xmb_shallow=xmb_shallow                               &
L01247                ,ktop_deep=ktop_deep                                   &
L01248                ,cugd_tten=cugd_tten,cugd_qvten=cugd_qvten             &
L01249                ,cugd_ttens=cugd_ttens,cugd_qvtens=cugd_qvtens         &
L01250                ,cugd_qcten=cugd_qcten,cugd_avedx=cugd_avedx           &
L01251                ,imomentum=imomentum,ishallow_g3=ishallow              &
L01252                ,ENSDIM=ensdim,MAXIENS=maxiens,MAXENS=maxens           &
L01253                ,MAXENS2=maxens2,MAXENS3=maxens3,ichoice=clos_choice &
L01254                ,htop=htop,hbot=hbot                                   &
L01255                ,CU_ACT_FLAG=CU_ACT_FLAG,warm_rain=warm_rain           &
L01256                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde       &
L01257                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme       &
L01258                ,IPS=ips,IPE=ipe,JPS=jps,JPE=jpe,KPS=kps,KPE=kpe       &
L01259                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte       &
L01260                ,PERIODIC_X=periodic_x,PERIODIC_Y=periodic_y            &
L01261               ! optionals
L01262                ,RTHCUTEN=RTHCUTEN ,RTHFTEN=RTHFTEN                    &
L01263                ,RQICUTEN=RQICUTEN ,RQVFTEN=RQVFTEN                    &
L01264                ,rqvblten=rqvblten,rthblten=rthblten                    &
L01265                ,RQVCUTEN=RQVCUTEN,RQCCUTEN=RQCCUTEN                   &
L01266                ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                         &
L01267                ,F_QI=f_qi,F_QS=f_qs                                   &
L01268  #if ( WRF_DFI_RADAR == 1 )
L01269                  ! Optional CAP suppress option
L01270               ,do_capsuppress=do_capsuppress                          &
L01271               ,cap_suppress_loc=cap_suppress_loc                      &
L01272  #endif
L01273                                                                       )
L01274       CASE (GFLSCHEME)
L01275           CALL wrf_debug(100,'in grelldrv')
L01276
L01277           CALL CU_GRELL_FREITAS_LI(                                        &
L01278                itimestep=itimestep,dt=dt,dxcell=dx,u=u,v=v,w=w             &
L01279               ,t=t,q=qv_curr,rho=rho,p=p,pi=pi,p8w=p8w,dz8w=dz8w           &
L01280               ,ht=ht,xland=xland,hfx=hfx,qfx=qfx,gsw=gsw                   &
L01281               ,rqvften=rqvften,rthften=rthften,rthblten=rthblten           &
L01282               ,rqvblten=rqvblten,rthraten=rthraten,kpbl=kpbl               &
L01283               ,xlv=xlv,cp=cp,g=g,r_v=r_v,ichoice_deep=clos_choice          &
```

```
L01284               ,ishallow_g3=ishallow,htop=htop,hbot=hbot             &
L01285               ,k22_shallow=k22_shallow,kbcon_shallow=kbcon_shallow  &
L01286               ,ktop_shallow=ktop_shallow,xmb_shallow=xmb_shallow    &
L01287               ,raincv=raincv,pratec=tmppratec,gdc=gd_cloud          &
L01288               ,gdc2=gd_cloud2,rthcuten=rthcuten,rqvcuten=rqvcuten   &
L01289               ,rqccuten=rqccuten,rqicuten=rqicuten                  &
L01290               ,rucuten=rucuten,rvcuten=rvcuten                      &
L01291  #if ( WRF_CHEM == 1 )
L01292               ,num_chem=num_chem,chem3d_in=grid%chem                &
L01293  #endif
L01294               ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme      &
L01295               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde      &
L01296               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte)
```

讲解：**SAS/SCALESAS/OSAS 三兄弟**共同点：自适应时间步直接 FATAL_ERROR——SAS 内部有跨步状态（HPBL_HOLD 等），步长可变会破坏其收支（注释原话「will not work properly with an adaptive time step」）；差异：SCALESAS 吃 dx2d/DYNMM 并输出尺度函数 SCALEFUN/SIGMU 上升气流分数（scale-aware 改造点）；OSAS 吃 store_rand 随机场。**G3（Grell 3D 集合）**：浅对流五件套（k22/kbcon/ktop/xmb_shallow + ktop_deep）、cugd_tten/qvten 系倾向缓存（供 § 十一的 spread 平滑）、DFI 雷达抑制（WRF_DFI_RADAR 下 cap_suppress_loc 取 dfi_tten_rad 顶层的强加热作 CAP 屏障）、周期边界标记。**GFL（Grell–Freitas 尺度自适应）**：参表最「现代」——吃 hfx/qfx/gsw 地表通量与 rthraten 辐射倾向、kpbl，WRF_CHEM 下直接把 grid%chem 整段传入（化学-对流交互在线耦合的样板）。

十、CASE 分派·下：CAMZM、新旧 Tiedtke、KSAS/NSAS、KFCUP

```
!file: phys/module_cumulus_driver.F    L1298-L1552
L01298        CASE (CAMZMSCHEME)
L01299             IF (PRESENT(z_at_w) .AND. PRESENT(mavail)                &
L01300                    .AND. PRESENT(pblh) .AND. PRESENT(psfc).AND.PRESENT(RQCNCUTEN))THEN
L01301             CALL wrf_debug(100,'in camzm_cps')
L01302          IF(.not.f_qi)THEN
L01303             WRITE( wrf_err_message , * ) 'This cumulus option requires ice microphysics option: f_qi = ', f_qi
L01304             CALL wrf_error_fatal ( wrf_err_message )
L01305          ENDIF
L01306             CALL CAMZM_DRIVER(                                       &
L01307                   IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde    &
L01308                  ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme    &
L01309                  ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte    &
L01310                  ,ITIMESTEP=itimestep, BL_PBL_PHYSICS=bl_pbl_physics  &
L01311                  ,SF_SFCLAY_PHYSICS=sf_sfclay_physics                 &
L01312                  ,TH=th, T_PHY=t, TSK=tsk, TKE_PBL=tke_pbl            &
L01313                  ,UST=ust, QV=qv_curr, QC=qc_curr, QI=qi_curr         &
L01314                  ,MAVAIL=mavail,KPBL=kpbl, PBLH=pblh, XLAND=xland     &
L01315                  ,Z=z, Z_AT_W=z_at_w                                  &
L01316                  ,DZ8W=dz8w, HT=ht                                    &
L01317                  ,P=p, P8W=p8w, PI_PHY=pi, PSFC=psfc                  &
L01318                  ,U_PHY=u, V_PHY=v, HFX=hfx, QFX=qfx, CLDFRA=cldfra   &
L01319                  ,CLDFRA_MP_ALL=cldfra_mp_all                         &
L01320                  ,IS_CAMMGMP_USED=is_CAMMGMP_used                     &
L01321                  ,TPERT_CAMUWPBL=tpert2d                              &
L01322                  ,DX=dx, DT=dt, STEPCU=stepcu, CUDT=cudt              &
L01323                  ,CURR_SECS=curr_secs                                 &
L01324                  ,ADAPT_STEP_FLAG=adapt_step_flag                     &
L01325                  ,CUDTACTTIME=cudtacttime_pass                        &
L01326                  ,CAPE_OUT=cape                                       &
L01327                  ,MU_OUT=zmmu, MD_OUT=zmmd                            &
L01328                  ,ZMDT=zmdt, ZMDQ=zmdq, DLF_OUT=dlf, RLIQ_OUT=rliq    &
L01329                  ,PCONVT=pconvt, PCONVB=pconvb, CUBOT=hbot, CUTOP=htop&
L01330                  ,RAINCV=raincv, PRATEC=tmppratec                     &
L01331                  ,RUCUTEN=rucuten, RVCUTEN=rvcuten                    &
L01332                  ,RTHCUTEN=rthcuten, RQVCUTEN=rqvcuten                &
L01333                  ,RQCCUTEN=rqccuten, RQICUTEN=rqicuten                &
L01334                  ,RQCNCUTEN=rqcncuten, RQINCUTEN=rqincuten            &
L01335                  ,EVAPTZM=evaptzm, FZSNTZM=fzsntzm, EVSNTZM=evsntzm   &
L01336                  ,EVAPQZM=evapqzm, ZMFLXPRC=zmflxprc                  &
L01337                  ,ZMFLXSNW=zmflxsnw, ZMNTPRPD=zmntprpd                &
L01338                  ,ZMNTSNPD=zmntsnpd, ZMEIHEAT=zmeiheat                &
L01339                  ,CMFMC=cmfmc, CMFMCDZM=cmfmcdzm                      &
L01340                  ,PRECCDZM=preccdzm, PRECZ=precz                      &
L01341                  ,ZMMTU=zmmtu, ZMMTV=zmmtv, ZMUPGU=zmupgu             &
L01342                  ,ZMUPGD=zmupgd, ZMVPGU=zmvpgu, ZMVPGD=zmvpgd         &
L01343                  ,ZMICUU=zmicuu, ZMICUD=zmicud, ZMICVU=zmicvu         &
L01344                  ,ZMICVD=zmicvd, ZMDICE=zmdice, ZMDLIQ=zmdliq         &
L01345                  ,EVAPCDP3D=evapcdp3d, ICWMRDP3D=icwmrdp3d            &
L01346                  ,RPRDDP3D=rprddp3d,DP3D=dp3d, DU3D=du3d, ED3D=ed3d   &
L01347                  ,EU3D=eu3d, MD3D=md3d, MU3D=mu3d,DSUBCLD2D=dsubcld2d &
L01348                  ,IDEEP2D=ideep2d, JT2D=jt2d, MAXG2D=maxg2d           &
L01349                  ,LENGATH2D=lengath2d                                 )
L01350             ELSE
L01351                WRITE( wrf_err_message , * ) 'Insufficient arguments to call CAMZM cu scheme'
L01352                CALL wrf_error_fatal ( wrf_err_message )
L01353             ENDIF
L01354  ! TIEDTKE SCHEME - ZCX&YQW (U of Hawaii)
L01355        CASE (TIEDTKESCHEME)
L01356
L01357           IF ( PRESENT ( QFX ) .AND. PRESENT( ZNU ) ) THEN
L01358
L01359             CALL wrf_debug(100,'in cu_tiedtke')
L01360             CALL CU_TIEDTKE(                                         &
L01361                   DT=dt,ITIMESTEP=itimestep,STEPCU=STEPCU            &
L01362                  ,RAINCV=RAINCV,PRATEC=tmppratec,QFX=qfx,ZNU=znu     &
L01363                  ,U3D=u,V3D=v,W=w,T3D=t,PI3D=pi,RHO3D=rho            &
L01364                  ,QV3D=QV_CURR,QC3D=QC_CURR,QI3D=QI_CURR             &
L01365                  ,QVPBLTEN=RQVBLTEN                                  &
```

```
L01366                ,DZ8W=dz8w,PCPS=p,P8W=p8w,XLAND=XLAND            &
L01367                ,CU_ACT_FLAG=CU_ACT_FLAG                         &
L01368                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01369                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01370                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01371               ! optionals
L01372                ,QVFTEN=RQVFTEN                                  &
L01373                ,RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN             &
L01374                ,RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN             &
L01375                ,RUCUTEN = RUCUTEN,RVCUTEN = RVCUTEN             &
L01376                ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                   &
L01377                ,F_QI=f_qi,F_QS=f_qs                             &
L01378                                                               )
L01379          ELSE
L01380            CALL wrf_error_fatal('Lacking arguments for CU_TIEDTKE in cumulus driver')
L01381          ENDIF
L01382
L01383  ! NEW TIEDTKE SCHEME - ZCX&YQW (U of Hawaii)
L01384       CASE (NTIEDTKESCHEME)
L01385
L01386            CALL wrf_debug(100,'in cu_ntiedtke')
L01387            CALL CU_NTIEDTKE_DRIVER(                              &
L01388                 DT=dt,ITIMESTEP=itimestep,STEPCU=STEPCU,HFX=hfx  &
L01389                ,RAINCV=RAINCV,PRATEC=tmppratec,QFX=qfx           &
L01390                ,U3D=u,V3D=v,W=w,T3D=t,PI3D=pi,RHO3D=rho          &
L01391                ,QV3D=QV_CURR,QC3D=QC_CURR,QI3D=QI_CURR           &
L01392                ,DZ8W=dz8w,PCPS=p,P8W=p8w,XLAND=XLAND,DX=dx2d     &
L01393                ,CU_ACT_FLAG=CU_ACT_FLAG                          &
L01394                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01395                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01396                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01397               ! optionals
L01398                ,QVFTEN=RQVFTEN,THFTEN=RTHFTEN                    &
L01399                ,RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN              &
L01400                ,RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN              &
L01401                ,RUCUTEN = RUCUTEN,RVCUTEN = RVCUTEN              &
L01402                ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                    &
L01403                ,F_QI=f_qi,F_QS=f_qs                              &
L01404                ,GRAV=g,XLF=xlf,XLS=xls,XLV=xlv                   &
L01405                ,RD=r_d,RV=r_v,CP=cp                              &
L01406                ,errmsg=errmsg,errflg=errflg                      &
L01407                                                               )
L01408
L01409  ! New KIM SAS SCHEME - (KIAPS, South Korea)
L01410       CASE (KSASSCHEME)
L01411          IF ( PRESENT ( QFX ) .AND. PRESENT( HFX ) ) THEN
L01412            CALL wrf_debug(100,'in ksas_cps')
L01413            CALL CU_KSAS(                                         &
L01414                 DT=dt,DX=dx,P3DI=p8w,P3D=p,PI3D=pi,              &
L01415                 QC3D=QC_CURR,QI3D=QI_CURR,RHO3D=rho,             &
L01416                 ITIMESTEP=itimestep,STEPCU=STEPCU,               &
L01417                 HBOT=HBOT,HTOP=HTOP,                             &
L01418                 CU_ACT_FLAG=CU_ACT_FLAG,                         &
L01419                 RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN,             &
L01420                 RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN,             &
L01421                 RUCUTEN=RUCUTEN,RVCUTEN=RVCUTEN,                 &
L01422                 QV3D=QV_CURR,T3D=t,                              &
L01423                 RAINCV=RAINCV,PRATEC=tmpPRATEC,                  &
L01424                 XLAND=XLAND,DZ8W=dz8w,W=w,U3D=u,V3D=v,           &
L01425                 HPBL=pblh,HFX=hfx,QFX=qfx,                       &
L01426                 HPBL_HOLD=hpbl_hold,                             &
L01427                 ZNU=znu,                                         &
L01428                 MP_PHYSICS=mp_physics,                           &
L01429                 DX_FACTOR_NSAS=dx_factor_nsas,                   &
L01430                 pgcon=pgcon,                                     &
L01431                 P_QC=p_qc,P_QI=p_qi,                             &
L01432                 P_FIRST_SCALAR=param_first_scalar                &
L01433                ,CP=cp,CLIQ=cliq,CPV=cpv,G=g,XLV=xlv,R_D=r_d      &
L01434                ,R_V=r_v,EP_1=ep_1,EP_2=EP_2                     &
L01435                ,CICE=cice,XLS=xls,PSAT=psat                     &
L01436                ,F_QI=f_qi,F_QC=f_qc                             &
L01437                ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01438                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01439                ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01440                                                                 )
L01441          ELSE
L01442            CALL wrf_error_fatal('Lacking arguments for CU_KSAS in cumulus driver')
L01443          ENDIF
L01444
L01445  ! New GFS SAS SCHEME - (Yonsei Univ., South Korea)
L01446       CASE (NSASSCHEME)
L01447          IF ( PRESENT ( QFX ) .AND. PRESENT( HFX ) ) THEN
L01448            CALL wrf_debug(100,'in nsas_cps')
L01449            CALL CU_NSAS(                                         &
L01450                 DT=dt,DX=dx,P3DI=p8w,P3D=p,PI3D=pi,              &
L01451                 QC3D=QC_CURR,QI3D=QI_CURR,RHO3D=rho,             &
L01452                 ITIMESTEP=itimestep,STEPCU=STEPCU,               &
L01453                 HBOT=HBOT,HTOP=HTOP,                             &
L01454                 CU_ACT_FLAG=CU_ACT_FLAG,                         &
L01455                 RTHCUTEN=RTHCUTEN,RQVCUTEN=RQVCUTEN,             &
L01456                 RQCCUTEN=RQCCUTEN,RQICUTEN=RQICUTEN,             &
L01457                 RUCUTEN=RUCUTEN,RVCUTEN=RVCUTEN,                 &
L01458                 QV3D=QV_CURR,T3D=t,                              &
L01459                 RAINCV=RAINCV,PRATEC=tmpPRATEC,                  &
L01460                 XLAND=XLAND,DZ8W=dz8w,W=w,U3D=u,V3D=v,           &
L01461                 HPBL=pblh,HFX=hfx,QFX=qfx,                       &
L01462                 MP_PHYSICS=mp_physics,                           &
L01463                 DX_FACTOR_NSAS=dx_factor_nsas,                   &
L01464                 pgcon=pgcon,                                     &
L01465                P_QC=p_qc,P_QI=p_qi,                             &
L01466                 P_FIRST_SCALAR=param_first_scalar                &
L01467                ,CP=cp,CLIQ=cliq,CPV=cpv,G=g,XLV=xlv,R_D=r_d      &
L01468                ,R_V=r_v,EP_1=ep_1,EP_2=EP_2                     &
L01469                ,CICE=cice,XLS=xls,PSAT=psat                     &
L01470                ,F_QI=f_qi,F_QC=f_qc                             &
L01471               ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde &
L01472                ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme &
L01473               ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte &
L01474                                                                 )
L01475          ELSE
```

```
L01476           CALL wrf_error_fatal('Lacking arguments for CU_NSAS in cumulus driver')
L01477         ENDIF
L01478
L01479  #if (EM_CORE == 1)
L01480         !BSINGH -  For WRFCuP scheme
L01481       CASE (KFCUPSCHEME)
L01482
L01483         CALL wrf_debug(100,'in cu_kfcup')
L01484         CALL KF_CUP_CPS( GRID_ID=grid%grid_id                          & !rce 10-may-2012 add grid%id
L01485              ,U=u ,V=v ,TH=th ,T=t ,W=w ,RHO=rho                       &
L01486              ,RAINCV=raincv,NCA=nca ,DZ8W=dz8w,XLAND=xland             & !LD add PRATEC 21-Apr-2011
L01487              ,PCPS=p, PI=pi ,W0AVG=W0AVG                               &
L01488              ,CUTOP=HTOP,CUBOT=HBOT                                    &
L01489              ,XLV0=XLV0 ,XLV1=XLV1 ,XLS0=XLS0 ,XLS1=XLS1               &
L01490              ,CP=CP ,R=R_d ,G=G ,EP1=EP_1 ,EP2=EP_2                    &
L01491              ,SVP1=SVP1 ,SVP2=SVP2 ,SVP3=SVP3 ,SVPT0=SVPT0             &
L01492              ,DT=dt ,KTAU=itimestep ,DX=dx                             &
L01493              ,STEPCU=stepcu                                            &
L01494              ,CU_ACT_FLAG=cu_act_flag ,warm_rain=warm_rain             &
L01495              ,QV=qv_curr                                               &
L01496              ,IDS=ids,IDE=ide,JDS=jds,JDE=jde,KDS=kds,KDE=kde          &
L01497              ,IMS=ims,IME=ime,JMS=jms,JME=jme,KMS=kms,KME=kme          &
L01498              ,ITS=its,ITE=ite,JTS=jts,JTE=jte,KTS=kts,KTE=kte          &
L01499              ,PSFC=psfc,Z=z,Z_AT_W=z_at_w,HT=ht,TSK=tsk                & !CuP, wig 5-Oct-2006
L01500              ,HFX=hfx,QFX=qfx                                          & !CuP, wig, 24-Aug-2006
L01501              ,MAVAIL=mavail,SF_SFCLAY_PHYSICS=sf_sfclay_physics        & !CuP, wig, 24-Aug-2006
L01502              ,BR=br,REGIME=regime,PBLH=pblh,KPBL=kpbl                  & !CuP, wig, 24-Aug-2006
L01503              ,T2=t2,Q2=q2                                              & !CuP, wig, 24-Aug-2006
L01504              ,SLOPESFC=slopeSfc,SLOPEEZ=slopeEZ                        & !CuP, wig, 24-Aug-2006
L01505              ,SIGMASFC=sigmasfc,SIGMAEZ=sigmaEZ                        & !CuP, wig, 24-Aug-2006
L01506              ,CUPFLAG=cupflag,CLDFRA_CUP=cldfra_cup                    & !CuP, wig, 18-Sep-2006
L01507              ,CLDFRATEND_CUP=cldfratend_cup                            & !CuP, wig, 18-Sep-2006
L01508              ,SHALL=shall,TAUCLOUD=taucloud,TACTIVE=tactive            & !CuP, wig, 18-Sep-2006
L01509              ,ACTIVEFRAC=activeFrac                                    & !CuP, lkb, 5-May-2010
L01510              ,TSTAR=tstar, LNTERMS=lnterms                             & !CuP, wig 4-Oct-2006
L01511              ,LNINT=lnint                                              & !CuP, wig 4-Oct-2006
L01512              ,NUMBINS=numBins                                          & !CuP, lkb 4-Nov-2009
L01513              ,THBINSIZE=thBinSize, RBINSIZE=rBinSize                   & !CuP, lkb 4-Nov-2009
L01514              ,MINDEEPFREQ=minDeepFreq, MINSHALLOWFREQ=minShallowFreq  & !CuP, lkb 4-Nov-2009
L01515              ,WCLOUDBASE=wCloudBase                                    & !CuP, lkb 29-April-2010
L01516              ,WACT_CUP=wact_cup                                        & !CuP, rce 10-may-2012
L01517              ,WULCL_CUP=wulcl_cup                                      & !CuP, rce 10-may-2012
L01518              ,WUP_CUP=wup_cup                                          & !CuP, rce 15-mar-2013 !BSINGH (12/06/2013)
L01519              ,QC_IC_CUP=qc_ic_cup                                      & !CuP, rce 10-may-2012
L01520              ,QNDROP_IC_CUP=qndrop_ic_cup                              & !CuP, rce 10-may-2012
L01521              ,QC_IU_CUP=qc_iu_cup                                      & !CuP, rce 10-may-2012
L01522              ,FCVT_QC_TO_PR_CUP=fcvt_qc_to_pr_cup                      & !CuP, rce 10-may-2012
L01523              ,FCVT_QC_TO_QI_CUP=fcvt_qc_to_qi_cup                      & !CuP, rce 10-may-2012
L01524              ,FCVT_QI_TO_PR_CUP=fcvt_qi_to_pr_cup                      & !CuP, rce 10-may-2012
L01525              ,MFUP_CUP=mfup_cup                                        & !CuP, rce 10-may-2012
L01526              ,MFUP_ENT_CUP=mfup_ent_cup                                & !CuP, rce 10-may-2012
L01527              ,MFDN_CUP=mfdn_cup                                        & !CuP, rce 10-may-2012
L01528              ,MFDN_ENT_CUP=mfdn_ent_cup                                & !CuP, rce 10-may-2012
L01529              ,UPDFRA_CUP=updfra_cup                                    & !CuP, rce 10-may-2012
L01530              ,TCLOUD_CUP=tcloud_cup                                    & !CuP, rce 10-may-2012
L01531              ,SHCU_AEROSOLS_OPT=shcu_aerosols_opt                      & !CuP, rce 10-may-2012
L01532  #if ( WRF_CHEM == 1 )
L01533              ,CHEM_OPT=chem_opt                                        & !CuP, rce 10-may-2012
L01534              ,CHEM=grid%chem                                           & !CuP, rce 10-may-2012
L01535  #endif
L01536              ! optionals
L01537              ,RTHCUTEN=rthcuten                                        &
L01538              ,RQVCUTEN=rqvcuten ,RQCCUTEN=rqccuten                     &
L01539              ,RQRCUTEN=rqrcuten ,RQICUTEN=rqicuten                     &
L01540              ,RQSCUTEN=rqscuten                                        &
L01541              ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                            &
L01542              ,F_QI=f_qi,F_QS=f_qs                                      &
L01543              )
L01544
L01545
L01546         DO J = jts,jte
L01547            DO I=its,ite
L01548               tmppratec(i,j)=raincv(i,j)/DT      !!!!!byang, move the pratec after calling cup
L01549            end do
L01550         end do
L01551         !BSINGH -ENDS
L01552  #endif
```

讲解：**CAMZM**：入口自检五件 PRESENT（z_at_w/mavail/pblh/psfc/RQCNCUTEN）缺一 FATAL；f_qi=false 直接 FATAL（CAMZM 必须配冰相微物理）；输出 CAM 全套诊断（质量通量 zmmu/zmmd、云内加热 zmdt/zmdq、降水蒸发 evaptzm 系、与 CAMMGMP 气溶胶耦合的 dp3d/du3d 系）。**旧 Tiedtke**：PRESENT(QFX,ZNU) 检查；QVPBLTEN=RQVBLTEN——闭合只看 PBL 水汽倾向（§六对比）。**新 Tiedtke**：THFTEN=RTHFTEN（已含辐射 +PBL 的组装版）、常数注入 G/XLF/XLS/XLV/RD/RV/CP、errmsg/errflg CCPP 协议——外壳见 `code_11_module_cu_ntiedtke.md`。**KSAS/NSAS**：PRESENT(QFX,HFX) 检查；常数表注入最全（CP/CLIQ/CPV/CICE/PSAT/XLS/EP_1/EP_2——GFS 风格物理库不吃隐含常数）；NSAS 有 nsas_dx_factor 格距因子与 hpbl_hold（注意 NSAS 未传 hpbl_hold，KSAS 传——两方案的同源差异）。**KFCUP（WRF-CuP）**：气溶胶-云交互 20 余组诊断数组（mfup/mfdn 质量通量、fcvt_* 转换分数、qndrop_ic 活化数浓度）全在这一案；调用后手工 **tmppratec=raincv/DT**（byang 注释——KF_CUP 不自己回 PRATEC，与其他方案又一个差异点）。DUCU 案（L1553–L1582，仅 WRFPLUS 编译，切线性伴随方案）注记不录。**CASE DEFAULT**：未知方案 FATAL——加新方案漏写 CASE 时的兜底见 §十三块尾。

```
!file: phys/module_cumulus_driver.F    L1584-L1589
L01584       CASE DEFAULT
L01585
L01586          WRITE( wrf_err_message , * ) 'The cumulus option does not exist: cu_physics = ', cu_physics
```

```
L01587          CALL wrf_error_fatal ( wrf_err_message )
L01588
L01589     END SELECT cps_select
```

讲解：DEFAULT 出口与 END SELECT——cps_select 结构到此闭合。

十一、收尾：G3 平滑、PRATEC 与激活时刻回写

```
!file: phys/module_cumulus_driver.F    L1591-L1634
L01591        ENDDO
L01592        !$OMP END PARALLEL DO
L01593  #if  ( EM_CORE == 1 )
L01594     IF(cu_physics .eq. 5 )then
L01595  #ifdef DM_PARALLEL
L01596  #     include "HALO_CUP_G3_OUT.inc"
L01597  #endif
L01598        !$OMP PARALLEL DO   &
L01599        !$OMP PRIVATE ( ij ,its,ite,jts,jte, i,j,k)
L01600        DO ij = 1 , num_tiles
L01601          its = i_start(ij)
L01602          ite = i_end(ij)
L01603          jts = j_start(ij)
L01604          jte = j_end(ij)
L01605
L01606          call conv_grell_spread3d(rthcuten=rthcuten,rqvcuten=rqvcuten                &
L01607       &            ,rqccuten=rqccuten,raincv=raincv,cugd_avedx=cugd_avedx          &
L01608       &            ,cugd_tten=cugd_tten,cugd_qvten=cugd_qvten,rqicuten=rqicuten    &
L01609       &            ,cugd_ttens=cugd_ttens,cugd_qvtens=cugd_qvtens                 &
L01610       &            ,cugd_qcten=cugd_qcten,pi_phy=pi,moist_qv=qv_curr              &
L01611       &            ,PRATEC=tmppratec,dt=dt,num_tiles=num_tiles                    &
L01612       &            ,imomentum=imomentum                          &
L01613       &            ,F_QV=F_QV,F_QC=F_QC,F_QR=F_QR,F_QI=F_QI,F_QS=F_QS             &
L01614       &            ,ids=IDS,ide=IDE, jds=JDS,jde=JDE, kds=KDS,kde=KDE             &
L01615       &            ,ips=IPS,ipe=IPE, jps=JPS,jpe=JPE, kps=KPS,kpe=KPE             &
L01616       &            ,ims=IMS,ime=IME, jms=JMS,jme=JME, kms=KMS,kme=KME             &
L01617       &            ,its=its,ite=ite, jts=jts,jte=jte, kts=kts,kte=kte)
L01618        ENDDO
L01619        !$OMP END PARALLEL DO
L01620     endif
L01621  #endif
L01622
L01623
L01624     ! Copy pratec back to output array, if necessary.
L01625
L01626     if (PRESENT(PRATEC)) then
L01627        pratec(:,:) = tmppratec(:,:)
L01628     endif
L01629
L01630     ! Copy cudtacttime back if necessary
L01631
L01632     if ( PRESENT(CUDTACTTIME) ) then
L01633        cudtacttime = cudtacttime_pass
L01634     end if
```

讲解：①tile 循环收尾；②**cu_physics==5（G3 硬编码数字！）**时调 conv_grell_spread3d：把 G3 的 cugd_* 缓存倾向做水平平滑（cugd_avedx 平均格距）后并入 rthcuten/rqvcuten 系，浅对流倾向同时落账——G3 的「深对流平滑 +浅对流直加」两段式输出全在这；③tmppratec 按 PRESENT(PRATEC) 写回（§ 四的暂存闭环）；④cudtacttime_pass 回写（自适应 dt 的下一激活时刻）。

十二、气溶胶插值两单元（全文）

```
!file: phys/module_cumulus_driver.F    L1647-L1754    (subroutine aer_time_int_cu)
L01647  SUBROUTINE aer_time_int_cu(julday,julian,aerodm,aerodt,levsiz,num_months,no_src,&
L01648                             ids , ide , jds , jde , kds , kde ,     &
L01649                             ims , ime , jms , jme , kms , kme ,     &
L01650                             its , ite , jts , jte , kts , kte )
L01651
L01652  ! adapted from oznint from CAM module
L01653  !  input: aerodm - read from physics_init
L01654  ! output: aerodt - time interpolated
L01655
L01656  !  USE module_ra_cam_support, ONLY : getfactors
L01657
L01658     IMPLICIT NONE
L01659
L01660     INTEGER,    INTENT(IN) ::           ids,ide, jds,jde, kds,kde, &
L01661                                         ims,ime, jms,jme, kms,kme, &
L01662                                         its,ite, jts,jte, kts,kte
L01663
L01664     INTEGER,      INTENT(IN   )    ::   levsiz, num_months, no_src
L01665
L01666     REAL,  DIMENSION( ims:ime, levsiz, jms:jme, num_months, no_src ),      &
L01667            INTENT(IN   ) ::                                  aerodm
L01668
L01669     INTEGER, INTENT(IN )      ::        JULDAY
L01670     REAL,    INTENT(IN )      ::        JULIAN
L01671
L01672     REAL,  DIMENSION( ims:ime, levsiz, jms:jme, no_src ),      &
L01673            INTENT(OUT  ) ::                                  aerodt
L01674
L01675     !Local
L01676     REAL      :: intJULIAN
L01677     integer   :: np1,np,nm,m,k,i,j,s
L01678     integer   :: IJUL
```

```
L01679     integer, dimension(12) ::  date_oz
L01680     data date_oz/16, 45, 75, 105, 136, 166, 197, 228, 258, 289, 319, 350/
L01681     real, parameter :: daysperyear = 365.  ! number of days in a year
L01682     real      :: cdayozp, cdayozm
L01683     real      :: fact1, fact2, deltat
L01684     logical   :: finddate
L01685     logical   :: ozncyc
L01686     CHARACTER(LEN=256) :: msgstr
L01687
L01688     ozncyc = .true.
L01689     ! JULIAN starts from 0.0 at 0Z on 1 Jan.
L01690     intJULIAN = JULIAN + 1.0       ! offset by one day
L01691  ! jan 1st 00z is julian=1.0 here
L01692     IJUL=INT(intJULIAN)
L01693  !  Note that following will drift.
L01694  !    Need to use actual month/day info to compute julian.
L01695     intJULIAN=intJULIAN-FLOAT(IJUL)
L01696     IJUL=MOD(IJUL,365)
L01697     IF(IJUL.EQ.0)IJUL=365
L01698     intJULIAN=intJULIAN+IJUL
L01699     np1=1
L01700     finddate=.false.
L01701
L01702  !  do m=1,num_months
L01703     do m=1,12
L01704        if(date_oz(m).gt.intjulian.and..not.finddate) then
L01705          np1=m
L01706          finddate=.true.
L01707        endif
L01708     enddo
L01709     cdayozp=date_oz(np1)
L01710
L01711     if(np1.gt.1) then
L01712        cdayozm=date_oz(np1-1)
L01713        np=np1
L01714        nm=np-1
L01715     else
L01716        cdayozm=date_oz(12)
L01717        np=np1
L01718        nm=12
L01719     endif
L01720
L01721  !  call getfactors(ozncyc,np1, cdayozm, cdayozp,intjulian, &
L01722  !                  fact1, fact2)
L01723  !
L01724  ! Determine time interpolation factors.  Account for December-January
L01725  ! interpolation if dataset is being cycled yearly.
L01726  !
L01727     if (ozncyc .and. np1 == 1) then                    ! Dec-Jan interpolation
L01728        deltat = cdayozp + daysperyear - cdayozm
L01729        if (intjulian > cdayozp) then                   ! We are in December
L01730           fact1 = (cdayozp + daysperyear - intjulian)/deltat
L01731           fact2 = (intjulian - cdayozm)/deltat
L01732        else                                            ! We are in January
L01733           fact1 = (cdayozp - intjulian)/deltat
L01734           fact2 = (intjulian + daysperyear - cdayozm)/deltat
L01735        end if
L01736     else
L01737        deltat = cdayozp - cdayozm
L01738        fact1 = (cdayozp - intjulian)/deltat
L01739        fact2 = (intjulian - cdayozm)/deltat
L01740     end if
L01741  !
L01742  ! Time interpolation.
L01743  !
L01744        do s=1, no_src
L01745          do j=jts,jte
L01746            do k=1,levsiz
L01747              do i=its,ite
L01748              aerodt(i,k,j,s) = aerodm(i,k,j,nm,s)*fact1 + aerodm(i,k,j,np,s)*fact2
L01749              end do
L01750            end do
L01751          end do
L01752        end do
L01753
L01754  END SUBROUTINE aer_time_int_cu
```

讲解：从辐射驱动拷来的月平均场时间插值（注释自承 adapted from oznint of CAM）：① 锚点表 date_oz 取每月 16 日（16/45/75/…/350）——月平均值的代表时刻；②JULIAN 的 365 日循环处理（intJULIAN 修正链）；③**Dec–Jan 环绕插值**：np1=1 时分「在 12 月」与「在 1 月」两支分别算 fact1/fact2 权重——年度循环数据无断点；④ 插值 aerodt=aerodm(上月)·fact1+aerodm(当月)·fact2。

```
!file: phys/module_cumulus_driver.F    L1757-L1897    (subroutine aer_p_int_cu)
L01757  SUBROUTINE aer_p_int_cu(p ,pin_cu, levsiz_cu, aerotcu, aerocu, no_src_cu, pf, &
L01758                       ids , ide , jds , jde , kds , kde ,     &
L01759                       ims , ime , jms , jme , kms , kme ,     &
L01760                       its , ite , jts , jte , kts , kte )
L01761
L01762  !-------------------------------------------------------------------------
L01763  !
L01764  ! Purpose: Interpolate aerosol from current time-interpolated values to model
L01765  ! levels
L01766  !
L01767  ! Method: Use pressure values to determine interpolation levels
L01768  !
L01769  ! Author: Bruce Briegleb
L01770  ! WW: Adapted for general use
L01771  ! PSH: Modified to not calculate totaod
L01772  !
L01773  !   p:  model level pressure at half levels (Pa, bottom-up)
L01774  !   pf: model level pressure at full levles (Pa, bottom-up)
L01775  !
L01776  !-------------------------------------------------------------------------
L01777     implicit none
```

```
!-------------------------------------------------------------------------
!
! Arguments
!

   INTEGER,    INTENT(IN) ::           ids,ide, jds,jde, kds,kde, &
                                       ims,ime, jms,jme, kms,kme, &
                                       its,ite, jts,jte, kts,kte

   integer, intent(in) :: levsiz_cu              ! number of aerosol layers
   integer, intent(in) :: no_src_cu              ! types of aerosol

   real, intent(in) :: p(ims:ime,kms:kme,jms:jme)
   real, intent(in) :: pf(ims:ime,kms:kme,jms:jme)
   real, intent(in) :: pin_cu(ims:ime,levsiz_cu,jms:jme)        ! aerosol data level pressures (mks, top-down)
   real, intent(in) :: aerotcu(ims:ime,levsiz_cu,jms:jme,no_src_cu) ! aerosol optical depth
   real, intent(out) :: aerocu(ims:ime,kms:kme,jms:jme,no_src_cu+1) ! aerosol optical depth
!
! local storage
!
   real    pmid(its:ite,kts:kte)
   integer i,j                ! Longitude index
   integer k, kk, kkstart, kout! Level indices
   integer kupper(its:ite)    ! Level indices for interpolation
   integer kount              ! Counter
   integer ncol, pver, s

   real    dpu                ! upper level pressure difference
   real    dpl                ! lower level pressure difference

   ncol = ite - its + 1
   pver = kte - kts + 1

   do s=1,no_src_cu
   do j=jts,jte
!
! Initialize index array
!
   do i=its, ite
      kupper(i) = 1
   end do
!
! The pressure from incoming data is in hPa and top-down,
!     while model pressure is in Pa and bottom-up
!
      do k = kts,kte
         kk = kte - k + kts
      do i = its,ite
         pmid(i,kk) = p(i,k,j)!*0.01
      enddo
      enddo

   do k=1,pver

      kout = pver - k + 1

!
! Top level we need to start looking is the top level for the previous k
! for all longitude points
!
      kkstart = levsiz_cu
      do i=its,ite
         kkstart = min0(kkstart,kupper(i))
      end do
      kount = 0

!
! Store level indices for interpolation
!
      do kk=kkstart,levsiz_cu-1
         do i=its,ite
            if (pin_cu(i,kk,j).lt.pmid(i,k) .and. pmid(i,k).le.pin_cu(i,kk+1,j)) then
               kupper(i) = kk
               kount = kount + 1
            end if
         end do
!
! If all indices for this level have been found, do the interpolation and
! go to the next level
!
         if (kount.eq.ncol) then
            do i=its,ite
               dpu = pmid(i,k) - pin_cu(i,kupper(i),j)
               dpl = pin_cu(i,kupper(i)+1,j) - pmid(i,k)
               aerocu(i,kout,j,s) = (aerotcu(i,kupper(i),j,s)*dpl + &
                                     aerotcu(i,kupper(i)+1,j,s)*dpu)/(dpl + dpu)
            end do
            goto 35
         end if
      end do

!
! If we've fallen through the kk=1,levsiz_cu-1 loop, we cannot interpolate and
! must extrapolate from the bottom or top aerosol data level for at least some
! of the longitude points.
!
      do i=its,ite
         if (pmid(i,k) .lt. pin_cu(i,1,j)) then
            aerocu(i,kout,j,s) = aerotcu(i,1,j,s)*pmid(i,k)/pin_cu(i,1,j)
         else if (pmid(i,k) .gt. pin_cu(i,levsiz_cu,j)) then
            aerocu(i,kout,j,s) = aerotcu(i,levsiz_cu,j,s)
         else
            dpu = pmid(i,k) - pin_cu(i,kupper(i),j)
            dpl = pin_cu(i,kupper(i)+1,j) - pmid(i,k)
            aerocu(i,kout,j,s) = (aerotcu(i,kupper(i),j,s)*dpl + &
                                  aerotcu(i,kupper(i)+1,j,s)*dpu)/(dpl + dpu)
         end if
      end do

```

```
L01887         if (kount.gt.ncol) then
L01888            call wrf_error_fatal ('AER_P_INT: Bad aerosol data: non-monotonicity suspected')
L01889         end if
L01890  35     continue
L01891
L01892      end do
L01893      end do
L01894      end do
L01895
L01896      return
L01897   END SUBROUTINE aer_p_int_cu
```

讲解：数据层到模式层的气压插值（CAM 拷贝件）：① **坐标翻转**——气溶胶数据 top-down（pin_cu 注释 mks 制、自顶向下），模式气压 bottom-up，pmid 先翻转（kk=kte−k+kts）；② 逐模式层找夹逼层 kupper（kkstart 用全列最小值做扫描起点——逐点缓存加速），线性插值 aerocu=(上 • dpl+下 • dpu)/(dpl+dpu)；③ **越界外推**：模式顶高于数据顶按 pmid/pin 比例外推（气压递减 → 浓度递减的物理假设）、模式底低于数据底取常数；④kount>ncol 判非单调数据 FATAL。goto 35 的提前跳出是老式插值代码的活化石。

十三、改造入口

1. **加新积云方案**：state_description 加符号 → USE 一行 → CASE 一案（§ 三）；参列表学 NTIEDTKE 的显式常数注入（§ 十）而非依赖 module_model_constants。
2. **改调用频率**：四测试门控（§ 五）——cudt/stepcu 语义、自适应 dt 的时刻推进都在这；想按 CAPE/湿度自适应调频改 run_param 判定。
3. **闭合式强迫**：前期倾向组装段（§ 六）——给闭合式「看什么」的开关；加新方案时决定它吃 RTHFTEN 组装版还是裸平流。
4. **气溶胶-对流交互**：aer_* 两单元 + MSKF 案的 AEROCU 管线——推广到其他方案（如 KFCUP）照抄 § 七三步。
5. **亚格云诊断**：cldfra_dp/cldfra_sh、QC_CU/QI_CU 的 CASE 间差异——辐射驱动怎么用这些量见 code_18（辐射驱动，待批 4）。

十四、上下游链接

- 下游方案外壳：`code_11_module_cu_ntiedtke.md`（NTIEDTKE 全文精读）
- 上游调用：ch21_solve_em_dynamics.md（非时间分裂物理调用序）；倾向消费：动力核 RK 循环
- 同级驱动器：`code_22_module_pbl_driver.md`、`code_24_module_surface_driver.md`
- 教材正文：ch15_cumulus_family.md、ch20_modifying_wrf.md

code_24 — 表面驱动器（module_surface_driver.F 两族 CASE 选段）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_surface_driver.F	7428	e86f31fb163f96f7b7f958865de1085c

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 本篇 B 级两族选段：主入口 surface_driver（L7–4619）中只收「面层方案族 sfclay_select 全 CASE」与「陆面方案族 sfc_select 的 SLAB+Noah 代表段」，加前处理、收尾与 MYNN 海冰 wrapper；不收 L7–2066 的哑元声明区（各方案的完整实参表即 CASE 内的 CALL 本身）与其余六个 LSM CASE（NoahMP/RUC/PX/CLM/CTSM/SSIB，结构同 Noah，教材 ch16_land_surface_family.md 差异表覆盖）。

二、定位

一句话：五大 physics driver 之一——面层方案（sf_sfclay_physics）与陆面方案（sf_surface_physics）两族 CASE 的总调度，外加海冰/SST/湖泊/海洋混合层/坡地辐射/人为热等「近地面杂项」的枢纽。

谁在调它：dyn_em/module_first_rk_step_part1.F 在 pbl_driver **之前**调用（物理时序：surface → pbl，ch21_solve_em_dynamics.md）。下游两族：面层 → `code_10_module_sf_mynnsfc.md`（MYNN）等；陆面 → Noah（12/13 篇待交付）等；城市冠层经 pbl_driver 的 BEP 路径（`code_14a_module_sf_bep_bem.md`）。

两族 SELECT CASE 方案号对照（Registry 实测）：

sf_sfclay_physics	方案	sf_surface_physics	方案
1	SFCLAY（旧）	1	5-layer thermal diffusion（SLAB）
2	MYJ	2	Noah（LSMSCHEME）
3	GFS	3	NoahMP
4	QNSE	4	RUC
5	MYNN（code_10）	7	PX
7	PX	8	CLM4
10	TEMF	9	CTSM
11	ShinHong	6	SSIB
91	SFCLAY（rev）	0	off

三、前处理：耦合、灌溉、SST 与海冰

```
!file: phys/module_surface_driver.F    L1570-L1780
L01570    IF ( coupler_on .and. present(cplmask) .and. present(sst_input) ) THEN
L01571
L01572       CALL cpl_rcv( id, 'SST',            &
L01573          &              ids, ide, jds, jde, kds, kde, &
L01574          &              ims, ime, jms, jme, kms, kme, &
L01575          &              ips, ipe, jps, jpe, kps, kpe, &
L01576          &              max_edom, cplmask, SST, SST_INPUT )
L01577
L01578       CALL cpl_rcv( id, 'UOCE',           &
L01579          &              ids, ide, jds, jde, kds, kde, &
L01580          &              ims, ime, jms, jme, kms, kme, &
L01581          &              ips, ipe, jps, jpe, kps, kpe, &
L01582          &              max_edom, cplmask, UOCE )
L01583
L01584       CALL cpl_rcv( id, 'VOCE',           &
```

```
L01585          &              ids, ide, jds, jde, kds, kde, &
L01586          &              ims, ime, jms, jme, kms, kme, &
L01587          &              ips, ipe, jps, jpe, kps, kpe, &
L01588          &              max_edom, cplmask, VOCE )
L01589
L01590    END IF
L01591
L01592  #if (EM_CORE==1)
L01593       spp_lsm_loc = spp_lsm
L01594  #else
L01595       spp_lsm_loc = 0
L01596  #endif
L01597
L01598
L01599  !$OMP PARALLEL DO &
L01600  !$OMP PRIVATE (ij, i, j, k)
L01601    DO ij = 1,num_tiles
L01602      DO j = j_start(ij),j_end(ij)
L01603        DO k = kms,kme
L01604          DO i = i_start(ij),i_end(ij)
L01605            v_phytmp(i, k, j) = 0.
L01606            u_phytmp(i, k, j) = 0.
L01607          ENDDO
L01608        ENDDO
L01609        DO i = i_start(ij),i_end(ij)
L01610           QGH(i,j) = 0.
L01611           CHS(i,j) = 0.
L01612           CPM(i,j) = 0.
L01613           CHS2(i,j) = 0.
L01614        ENDDO
L01615      ENDDO
L01616    ENDDO
L01617    DTMIN = 0.
L01618    DTBL = 0.
L01619
L01620  ! RAINBL in mm (Accumulation between PBL calls)
L01621
L01622    IF ( PRESENT( rainncv ) .AND. PRESENT( rainbl ) ) THEN
L01623      !$OMP PARALLEL DO   &
L01624      !$OMP PRIVATE ( ij, i, j, k )
L01625      DO ij = 1 , num_tiles
L01626        DO j=j_start(ij),j_end(ij)
L01627        DO i=i_start(ij),i_end(ij)
L01628           RAINBL(i,j) = RAINBL(i,j) + RAINCV(i,j) + RAINNCV(i,j)
L01629           IF ( PRESENT( rainshv ))RAINBL(i,j) = RAINBL(i,j) + RAINSHV(i,j)
L01630           RAINBL(i,j) = MAX (RAINBL(i,j), 0.0)
L01631  #if (EM_CORE==1)
L01632           IRRIGATION_CHANNEL(i,j) = 0.
L01633           sf_surf_irr: SELECT CASE(sf_surf_irr_scheme)
L01634              CASE(DRIP)
L01635                 CALL drip_irrigation(                                  &
L01636                  &  julian_in,IRRIGATION(i,j),sf_surf_irr_scheme,      &
L01637                  &  irr_daily_amount,irr_start_hour,irr_num_hours,     &
L01638                  &  irr_start_julianday,irr_end_julianday,             &
L01639                  &  irr_freq,irr_ph,i,j,RAINBL(i,j),                   &
L01640                  &  IRRIGATION_CHANNEL(i,j),gmt,xtime,dt,              &
L01641                  &  irr_rand_field(i,j)                                &
L01642                  )
L01643              CASE(CHANNEL)
L01644                 CALL channel_irrigation(                               &
L01645                  &  julian_in,IRRIGATION(i,j),sf_surf_irr_scheme,      &
L01646                  &  irr_daily_amount,irr_start_hour,irr_num_hours,     &
L01647                  &  irr_start_julianday,irr_end_julianday,             &
L01648                  &  irr_freq,irr_ph,i,j,RAINBL(i,j),                   &
L01649                  &  IRRIGATION_CHANNEL(i,j),gmt,xtime,dt,              &
L01650                  &  irr_rand_field(i,j)                                &
L01651                  )
L01652            END SELECT sf_surf_irr
L01653  #endif
L01654        ENDDO
L01655        ENDDO
L01656      ENDDO
L01657      !$OMP END PARALLEL DO
L01658    ELSE IF ( PRESENT( rainbl ) ) THEN
L01659      !$OMP PARALLEL DO   &
L01660      !$OMP PRIVATE ( ij, i, j, k )
L01661      DO ij = 1 , num_tiles
L01662        DO j=j_start(ij),j_end(ij)
L01663        DO i=i_start(ij),i_end(ij)
L01664           RAINBL(i,j) = RAINBL(i,j) + RAINCV(i,j)
L01665           IF ( PRESENT( rainshv ))RAINBL(i,j) = RAINBL(i,j) + RAINSHV(i,j)
L01666           RAINBL(i,j) = MAX (RAINBL(i,j), 0.0)
L01667
L01668  #if (EM_CORE==1)
L01669           sf_surf_irr1: SELECT CASE(sf_surf_irr_scheme)
L01670              CASE(DRIP)
L01671                 CALL drip_irrigation(                                  &
L01672                  &  julian_in,IRRIGATION(i,j),sf_surf_irr_scheme,      &
L01673                  &  irr_daily_amount,irr_start_hour,irr_num_hours,     &
L01674                  &  irr_start_julianday,irr_end_julianday,             &
L01675                  &  irr_freq,irr_ph,i,j,RAINBL(i,j),                   &
L01676                  &  IRRIGATION_CHANNEL(i,j),gmt,xtime,dt,              &
L01677                  &  irr_rand_field(i,j)                                &
L01678                  )
L01679
L01680              CASE(CHANNEL)
L01681                 CALL channel_irrigation(                               &
L01682                  &  julian_in,IRRIGATION(i,j),sf_surf_irr_scheme,      &
L01683                  &  irr_daily_amount,irr_start_hour,irr_num_hours,     &
L01684                  &  irr_start_julianday,irr_end_julianday,             &
L01685                  &  irr_freq,irr_ph,i,j,RAINBL(i,j),                   &
L01686                  &  IRRIGATION_CHANNEL(i,j),gmt,xtime,dt,              &
L01687                  &  irr_rand_field(i,j)                                &
L01688                  )
L01689          END SELECT sf_surf_irr1
L01690  #endif
L01691
L01692        ENDDO
```

```
L01693        ENDDO
L01694      ENDDO
L01695      !$OMP END PARALLEL DO
L01696    ENDIF
L01697  ! Update SST
L01698    IF (sst_update .EQ. 1) THEN
L01699      CALL wrf_debug( 100, 'SST_UPDATE is on' )
L01700      !$OMP PARALLEL DO   &
L01701      !$OMP PRIVATE ( ij, i, j, k )
L01702      DO ij = 1 , num_tiles
L01703        DO j=j_start(ij),j_end(ij)
L01704        DO i=i_start(ij),i_end(ij)
L01705   ! check for lake model
L01706  #if (EM_CORE==1)
L01707            if ( lakemodel==1) then
L01708              if(lakemask(i,j).eq.1.) then
L01709                if ( xice(i,j).gt.xice_threshold) then   !mchen
L01710                   xice(i,j)=0.0
L01711                endif
L01712               endif
L01713            endif
L01714  #else
L01715            if ( lakemodel==1) then
L01716              if(ht(i,j)>=lake_min_elev) then
L01717                if ( xice(i,j).gt.xice_threshold) then   !mchen
L01718                   xice(i,j)=0.0
L01719                endif
L01720               endif
L01721            endif
L01722  #endif
L01723  ! end check lake model
L01724           XICE_save(I,J) = XICEM(I,J)
L01725
L01726           IF ( FRACTIONAL_SEAICE == 1 ) then
L01727              IF ( ( XICE(I,J) .NE. XICEM(I,J) ) .AND. ( XICEM(I,J) .GT. XICE_THRESHOLD ) ) THEN
L01728                 ! Fractional values of ALBEDO and EMISSIVITY are valid according to the
L01729                 ! earlier fractional seaice value, XICEM.  Recompute them for the new
L01730                 ! seaice value XICE.
L01731                  IF ( SEAICE_ALBEDO_OPT ==2 ) THEN
L01732                      IF ( ALBSI(I,J) < -1.E6 ) THEN
L01733                          call wrf_error_fatal("Field ALBSI not found in input.  Field ALBSI is required if
↪  SEAICE_ALBEDO_OPT=2")
L01734                      ENDIF
L01735                      ALBEDO(I,J) = 0.08 + XICE(I,J)/XICEM(I,J) * ( ALBSI(I,J) - 0.08 )
L01736                  ELSE
L01737                      ALBEDO(I,J) = 0.08 + XICE(I,J)/XICEM(I,J) * ( ALBEDO(I,J) - 0.08 )
L01738                  ENDIF
L01739                  EMISS (I,J) = 0.98 + XICE(I,J)/XICEM(I,J) * ( EMISS (I,J) - 0.98 )
L01740  ! use old tsk from seaice part
L01741                  TSK(I,J) = TSK_SAVE(I,J)*XICE(I,J) + (1.-XICE(I,J))*SST(I,J)
L01742              ENDIF
L01743           ENDIF
L01744
L01745          IF ( XLAND(i,j) .GT. 1.5 .AND. XICE(I,J) .GE. XICE_THRESHOLD .AND. XICEM(I,J) .LT. XICE_THRESHOLD ) THEN
L01746            ! water point turns to sea-ice point
L01747            XICEM(I,J) = XICE(I,J)
L01748            XLAND(I,J) = 1.
L01749            IVGTYP(I,J) = ISICE
L01750            ISLTYP(I,J) = 16
L01751            VEGFRA(I,J) = 0.
L01752            TMN(I,J) = 271.4
L01753
L01754            ! Over new ice, initial guesses of ALBEDO and EMISS are
L01755            ! based on default water and ice values for albedo and
L01756            ! emissivity.  The land-surface schemes can update these
L01757            ! values
L01758
L01759            SELECT CASE ( SEAICE_ALBEDO_OPT )
L01760            CASE ( 0, 1 )
L01761
L01762                ALBEDO(I,J) = SEAICE_ALBEDO_DEFAULT * XICE(I,J) + 0.08 * ( 1.0-XICE(I,J) )
L01763                ALBBCK(I,J) = SEAICE_ALBEDO_DEFAULT
L01764
L01765            CASE ( 2 )
L01766
L01767                IF ( ALBSI(I,J) < -1.E6 ) THEN
L01768                    call wrf_error_fatal("Field ALBSI not found in input.  Field ALBSI is required if
↪  SEAICE_ALBEDO_OPT=2")
L01769                ENDIF
L01770
L01771                ALBEDO(I,J) = ALBSI(I,J) * XICE(I,J) + 0.08 * ( 1.0-XICE(I,J) )
L01772                ALBBCK(I,J) = ALBSI(I,J)
L01773
L01774            END SELECT
L01775
L01776            EMISS(I,J)  = 0.98 * XICE(I,J) + 0.98 * ( 1.0-XICE(I,J) )
L01777            EMBCK(I,J)  = 0.98
L01778            DO nk = 1, num_soil_layers
L01779              TSLB(I,NK,J) = TSK(I,J)
L01780              SMOIS(I,NK,J) = 1.0
```

讲解：① **区域耦合器**（coupler_on）：cpl_rcv 收 OASIS 型外部 SST/海流——区域气候嵌套场景的进场口；② **灌溉**：sf_surf_irr 两级 SELECT（drip_irrigation/channel_irrigation），RAINBL 在此先累加对流 +大尺度降水（RAINBL=RAINNCV+RAINSHV——陆面方案的水量输入在此结账）；③**SST_UPDATE=1**：从 wrfbdy/wrfinput 读逐时 SST，分数海冰（FRACTIONAL_SEAICE=1）时 XICEM 新冰出现/消失触发 TSK 与反照率重算，SEAICE_ALBEDO_OPT 三选。这一段是「下边界外源」的全部进场通道——做 SST/海冰敏感性实验（论文 2 的 lake/river 处理同源）改这里。

四、面层方案族 sfclay_select（全 CASE）

```
!file: phys/module_surface_driver.F    L2067-L2555
L02067       CASE (SFCLAYSCHEME)
L02068  ! DX varies spatially in NMM, therefore, SFCLAY cannot be called
L02069  ! because it takes a scalar DX. NMM passes in a dummy value for this
L02070  ! scalar.  NEEDS FURTHER ATTENTION. JM 20050215
L02071         IF(PRESENT(SCM_FORCE_FLUX))THEN
L02072           IF (scm_force_flux .EQ. 1) THEN
L02073  ! surface forcing by observed fluxes
L02074           CALL scmflux(u_phytmp, v_phytmp, t_phy, qv_curr, p_phy, dz8w,   &
L02075                        cp, rcp, xlv, psfc, cpm, xland,                    &
L02076                        psim, psih, hfx, qfx, lh, tsk, flhc, flqc,         &
L02077                        znt, gz1oz0, wspd,                                 &
L02078                        julian_in, karman, p1000mb,                        &
L02079                        itimestep,chklowq,                                 &
L02080                        ids, ide, jds, jde, kds, kde,                      &
L02081                        ims, ime, jms, jme, kms, kme,                      &
L02082                        i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte   )
L02083           ENDIF
L02084         ENDIF
L02085         IF(PRESENT(SCM_FORCE_SKINTEMP))THEN
L02086           IF (scm_force_skintemp .EQ. 1) THEN
L02087  ! surface forcing by observed skin temperature
L02088           CALL scmskintemp(tsk, julian_in, itimestep,                    &
L02089                        ids, ide, jds, jde, kds, kde,                     &
L02090                        ims, ime, jms, jme, kms, kme,                     &
L02091                        i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte   )
L02092           ENDIF
L02093  !         IF (scm_force_skintemp .EQ. 2) THEN
L02094  ! surface forcing by gabls2 skin temperature
L02095  !         CALL scmgabls2(tsk, itimestep, dt,                             &
L02096  !                      ids, ide, jds, jde, kds, kde,                     &
L02097  !                      ims, ime, jms, jme, kms, kme,                     &
L02098  !                      i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte   )
L02099  !         ENDIF
L02100         ENDIF
L02101         IF (PRESENT(qv_curr)                              .AND.    &
L02102             PRESENT(mol)        .AND.  PRESENT(regime)  .AND.    &
L02103                                                        .TRUE. ) THEN
L02104           CALL wrf_debug( 100, 'in SFCLAY' )
L02105           IF ( FRACTIONAL_SEAICE == 1 ) THEN
L02106              CALL SFCLAY_SEAICE_WRAPPER(u_phytmp,v_phytmp,t_phy,qv_curr,&
L02107                   p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02108                   znt,ust,pblh,mavail,zol,mol,regime,psim,psih,fm,fhh, &
L02109                   xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02110                   u10,v10,th2,t2,q2,                                  &
L02111                   gz1oz0,wspd,br,isfflx,dx2d,                         &
L02112                   svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,eomeg,stbolt, &
L02113                   P1000mb,lakemask,                                   &
L02114                   XICE,SST,TSK_SEA,                                                &
L02115                   CHS2_SEA,CHS_SEA,CPM_SEA,CQS2_SEA,FLHC_SEA,FLQC_SEA,             &
L02116                   HFX_SEA,LH_SEA,QFX_SEA,QGH_SEA,QSFC_SEA,ZNT_SEA,                 &
L02117                   ITIMESTEP,TICE2TSK_IF2COLD,XICE_THRESHOLD,                       &
L02118                   ids,ide, jds,jde, kds,kde,                          &
L02119                   ims,ime, jms,jme, kms,kme,                          &
L02120                   i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
L02121                   ustm,ck,cka,cd,cda,isftcflx,iz0tlnd,scm_force_flux, &
L02122                   sf_surface_physics  )
L02123           ELSE
L02124           CALL SFCLAY(u_phytmp,v_phytmp,t_phy,qv_curr,              &
L02125                p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02126                znt,ust,pblh,mavail,zol,mol,regime,psim,psih,fm,fhh, &
L02127                xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02128                u10,v10,th2,t2,q2,                                  &
L02129                gz1oz0,wspd,br,isfflx,dx2d,                         &
L02130                svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,eomeg,stbolt, &
L02131                P1000mb,lakemask,                                   &
L02132                ids,ide, jds,jde, kds,kde,                          &
L02133                ims,ime, jms,jme, kms,kme,                          &
L02134                i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
L02135                ustm,ck,cka,cd,cda,isftcflx,iz0tlnd,scm_force_flux  )
L02136  #if ( EM_CORE==1)
L02137            DO j = j_start(ij),j_end(ij)
L02138            DO i = i_start(ij),i_end(ij)
L02139              ch(i,j) = chs (i,j)
L02140  !!            ch(i,j) = flhc(i,j)/( cpm(i,j)*rho(i,kts,j) )
L02141            end do
L02142            end do
L02143  #endif
L02144           ENDIF
L02145         ELSE
L02146           CALL wrf_error_fatal('Lacking arguments for SFCLAY in surface driver')
L02147         ENDIF
L02148
L02149      CASE (SFCLAYREVSCHEME)
L02150  ! DX varies spatially in NMM, therefore, SFCLAY cannot be called
L02151  ! because it takes a scalar DX. NMM passes in a dummy value for this
L02152  ! scalar.  NEEDS FURTHER ATTENTION. JM 20050215
L02153         IF (PRESENT(qv_curr)                              .AND.    &
L02154             PRESENT(mol)        .AND.  PRESENT(regime)  .AND.    &
L02155                                                        .TRUE. ) THEN
L02156           CALL wrf_debug( 100, 'in SFCLAY' )
L02157
L02158           IF ( FRACTIONAL_SEAICE == 1 ) THEN
L02159              CALL SFCLAYREV_SEAICE_WRAPPER(u_phytmp,v_phytmp,t_phy,qv_curr,&
L02160                   p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02161                   znt,ust,pblh,mavail,zol,mol,regime,psim,psih,fm,fhh, &
L02162                   xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02163                   u10,v10,th2,t2,q2,                                  &
L02164                   gz1oz0,wspd,br,isfflx,dx2d,                         &
L02165                   svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,              &
L02166                   P1000mb,lakemask,                                   &
L02167                   kim_tofd, tofd_factor, var2d,                       &
L02168                   XICE,SST,TSK_SEA,                                                &
L02169                   CHS2_SEA,CHS_SEA,CPM_SEA,CQS2_SEA,FLHC_SEA,FLQC_SEA,             &
```

```
L02170                  HFX_SEA,LH_SEA,QFX_SEA,QGH_SEA,QSFC_SEA,ZNT_SEA,                  &
L02171                  ITIMESTEP,TICE2TSK_IF2COLD,XICE_THRESHOLD,                        &
L02172                  ids,ide, jds,jde, kds,kde,                                &
L02173                  ims,ime, jms,jme, kms,kme,                                &
L02174                  i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
L02175                  ustm,ck,cka,cd,cda,isftcflx,iz0tlnd,                      &
L02176                  shalwater_z0,water_depth,                                 &
L02177                  scm_force_flux,sf_surface_physics,errmsg,errflg     )
L02178           ELSE
L02179           CALL SFCLAYREV(u_phytmp,v_phytmp,t_phy,qv_curr,&
L02180                p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02181                znt,ust,pblh,mavail,zol,mol,regime,psim,psih,fm,fhh, &
L02182                xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02183                u10,v10,th2,t2,q2,                                  &
L02184                gz1oz0,wspd,br,isfflx,dx2d,                         &
L02185                svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,              &
L02186                P1000mb,lakemask,                                   &
L02187                kim_tofd, tofd_factor, var2d,                       &
L02188                ids,ide, jds,jde, kds,kde,                          &
L02189                ims,ime, jms,jme, kms,kme,                          &
L02190                i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
L02191                ustm,ck,cka,cd,cda,isftcflx,iz0tlnd,                &
L02192                shalwater_z0,water_depth,                           &
L02193                scm_force_flux,errmsg,errflg                        )
L02194  #if ( EM_CORE==1)
L02195           DO j = j_start(ij),j_end(ij)
L02196           DO i = i_start(ij),i_end(ij)
L02197             ch(i,j) = chs (i,j)
L02198  !!           ch(i,j) = flhc(i,j)/( cpm(i,j)*rho(i,kts,j) )
L02199           end do
L02200           end do
L02201  #endif
L02202           ENDIF
L02203         ELSE
L02204           CALL wrf_error_fatal('Lacking arguments for SFCLAY in surface driver')
L02205         ENDIF
L02206
L02207       CASE (PXSFCSCHEME)
L02208         IF (PRESENT(qv_curr)                                .AND.    &
L02209             PRESENT(mol)        .AND.  PRESENT(regime)  .AND.    &
L02210                                                        .TRUE. ) THEN
L02211           CALL wrf_debug( 100, 'in PX Surface Layer scheme' )
L02212           IF ( FRACTIONAL_SEAICE == 1 ) THEN
L02213              CALL PXSFCLAY_SEAICE_WRAPPER(u_phytmp,v_phytmp,t_phy,th_phy,qv_curr,&
L02214                   p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02215                   znt,ust,pblh,mavail,zol,mol,regime,psim,psih,       &
L02216                   xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02217                   u10,v10,                                            &
L02218                   gz1oz0,wspd,br,isfflx,dx,                           &
L02219                   svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,              &
L02220                   XICE, SST, ITIMESTEP, TICE2TSK_IF2COLD,XICE_THRESHOLD, &
L02221                   CHS_SEA, CHS2_SEA, CPM_SEA, CQS2_SEA,FLHC_SEA,FLQC_SEA,&
L02222                   HFX_SEA, LH_SEA, QFX_SEA, QGH_SEA, QSFC_SEA, TSK_SEA,  &
L02223                   ids,ide, jds,jde, kds,kde,                          &
L02224                   ims,ime, jms,jme, kms,kme,                          &
L02225                   i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02226           ELSE
L02227           CALL PXSFCLAY(u_phytmp,v_phytmp,t_phy,th_phy,qv_curr,&
L02228                p_phy,dz8w,cp,g,rcp,r_d,xlv,psfc,chs,chs2,cqs2,cpm, &
L02229                znt,ust,pblh,mavail,zol,mol,regime,psim,psih,       &
L02230                xland,hfx,qfx,lh,tsk,flhc,flqc,qgh,qsfc,rmol,       &
L02231                u10,v10,                                            &
L02232                gz1oz0,wspd,br,isfflx,dx,                           &
L02233                svp1,svp2,svp3,svpt0,ep_1,ep_2,karman,itimestep,    &
L02234                ids,ide, jds,jde, kds,kde,                          &
L02235                ims,ime, jms,jme, kms,kme,                          &
L02236                i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02237           ENDIF
L02238         ELSE
L02239           CALL wrf_error_fatal('Lacking arguments for PX Surface Layer in surface driver')
L02240         ENDIF
L02241
L02242        CASE (MYJSFCSCHEME)
L02243         IF (PRESENT(qv_curr)    .AND.  PRESENT(qc_curr) .AND.    &
L02244                                                        .TRUE. ) THEN
L02245
L02246          CALL wrf_debug(100,'in MYJSFC')
L02247          IF ( FRACTIONAL_SEAICE == 1 ) THEN
L02248             CALL MYJSFC_SEAICE_WRAPPER(itimestep,ht,dz8w,             &
L02249                  p_phy,p8w,th_phy,t_phy,                              &
L02250                  qv_curr,qc_curr,                                     &
L02251                  u_phy,v_phy,tke_pbl,                                 &
L02252                  tsk,qsfc,thz0,qz0,uz0,vz0,                           &
L02253                  lowlyr,                                              &
L02254                  xland,ivgtyp,isurban,iz0tlnd,                        &
L02255                  TICE2TSK_IF2COLD,                                    & ! Extra for wrapper.
L02256                  XICE_THRESHOLD,                                      & ! Extra for wrapper.
L02257                  XICE, SST,                                           & ! Extra for wrapper.
L02258                  CHS_SEA, CHS2_SEA, CQS2_SEA, CPM_SEA,            &
L02259                  FLHC_SEA, FLQC_SEA, QSFC_SEA, &
L02260                  QGH_SEA, QZ0_SEA, HFX_SEA, QFX_SEA, LH_SEA,         &
L02261                  TSK_SEA,                                             &
L02262                  ust,znt,z0,pblh,mavail,rmol,                         &
L02263                  akhs,akms,                                           &
L02264                  br,                                                 &
L02265                  chs,chs2,cqs2,hfx,qfx,lh,flhc,flqc,qgh,cpm,ct,       &
L02266                  u10,v10,t2,th2,tshltr,th10,q2,qshltr,q10,pshltr,             &
L02267                  p1000mb,u10e,v10e,                                   &
L02268                  ids,ide, jds,jde, kds,kde,                           &
L02269                  ims,ime, jms,jme, kms,kme,                           &
L02270                  i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02271           ELSE
L02272              CALL MYJSFC(itimestep,ht,dz8w,                          &
L02273                p_phy,p8w,th_phy,t_phy,                               &
L02274                qv_curr,qc_curr,                                      &
L02275                u_phy,v_phy,tke_pbl,                                  &
L02276                tsk,qsfc,thz0,qz0,uz0,vz0,                            &
L02277                lowlyr,                                               &
L02278                xland,ivgtyp,isurban,iz0tlnd,                         &
```

```
L02279               ust,znt,z0,pblh,mavail,rmol,                          &
L02280               akhs,akms,                                            &
L02281               br,                                                  &
L02282               chs,chs2,cqs2,hfx,qfx,lh,flhc,flqc,qgh,cpm,ct,        &
L02283               u10,v10,t2,th2,tshltr,th10,q2,qshltr,q10,pshltr,              &
L02284               p1000mb,u10e,v10e,                                    &
L02285               ids,ide, jds,jde, kds,kde,                            &
L02286               ims,ime, jms,jme, kms,kme,                            &
L02287               i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02288         ENDIF
L02289  !       ustm is needed for LES tke calculation (ustm is ust used in friction)
L02290          DO j = j_start(ij),j_end(ij)
L02291             DO i = i_start(ij),i_end(ij)
L02292                ustm(i,j) = ust(i,j)
L02293                wspd(i,j) = MAX(SQRT(u_phy(i,kts,j)**2+v_phy(i,kts,j)**2),0.001)
L02294                ch(i,j) = chs (i,j)
L02295             END DO
L02296          END DO
L02297        ELSE
L02298          CALL wrf_error_fatal('Lacking arguments for MYJSFC in surface driver')
L02299        ENDIF
L02300
L02301       CASE (QNSESFCSCHEME)
L02302        IF(PRESENT(SCM_FORCE_FLUX))THEN
L02303          IF (scm_force_flux .EQ. 1) THEN
L02304  ! surface forcing by observed fluxes
L02305          CALL scmflux(u_phytmp, v_phytmp, t_phy, qv_curr, p_phy, dz8w,   &
L02306                       cp, rcp, xlv, psfc, cpm, xland,                    &
L02307                       psim, psih, hfx, qfx, lh, tsk, flhc, flqc,         &
L02308                       znt, gz1oz0, wspd,                                 &
L02309                       julian_in, karman, p1000mb,                        &
L02310                       itimestep,chklowq,                                 &
L02311                       ids, ide, jds, jde, kds, kde,                      &
L02312                       ims, ime, jms, jme, kms, kme,                      &
L02313                       i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte   )
L02314          ENDIF
L02315        ENDIF
L02316        IF(PRESENT(SCM_FORCE_SKINTEMP))THEN
L02317          IF (scm_force_skintemp .EQ. 1) THEN
L02318  ! surface forcing by observed skin temperature
L02319          CALL scmskintemp(tsk, julian_in, itimestep,                    &
L02320                       ids, ide, jds, jde, kds, kde,                     &
L02321                       ims, ime, jms, jme, kms, kme,                     &
L02322                       i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte   )
L02323          ENDIF
L02324        ENDIF
L02325
L02326        IF (PRESENT(qv_curr)    .AND.  PRESENT(qc_curr) .AND.    &
L02327                                                  .TRUE. ) THEN
L02328            CALL wrf_debug(100,'in QNSESFC')
L02329             IF ( FRACTIONAL_SEAICE == 1 ) THEN
L02330            CALL QNSESFC_SEAICE_WRAPPER(itimestep,ht,dz8w,             &
L02331                 p_phy,p8w,th_phy,t_phy,                               &
L02332                 qv_curr,qc_curr,                                      &
L02333                 u_phy,v_phy,tke_pbl,                                  &
L02334                 tsk,qsfc,thz0,qz0,uz0,vz0,                            &
L02335                 lowlyr,                                               &
L02336                 xland,                                                &
L02337                 TICE2TSK_IF2COLD,                                     & ! Extra for wrapper.
L02338                 XICE_THRESHOLD,                                       & ! Extra for wrapper.
L02339                 XICE, SST,                                            & ! Extra for wrapper.
L02340                 CHS_SEA, CHS2_SEA, CQS2_SEA, CPM_SEA,            &
L02341                 FLHC_SEA, FLQC_SEA, QSFC_SEA, &
L02342                 QGH_SEA, QZ0_SEA, HFX_SEA, QFX_SEA, LH_SEA,         &
L02343                 TSK_SEA,                                              &
L02344                 ust,znt,z0,pblh,mavail,rmol,                          &
L02345                 akhs,akms,                                            &
L02346                 br,                                                  &
L02347                 chs,chs2,cqs2,hfx,qfx,lh,flhc,flqc,qgh,cpm,ct,        &
L02348                 u10,v10,t2,th2,tshltr,th10,q2,qshltr,q10,pshltr,      &
L02349                 u10e,v10e,                                            &
L02350                 ids,ide, jds,jde, kds,kde,                            &
L02351                 ims,ime, jms,jme, kms,kme,                            &
L02352                 i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,SCM_FORCE_FLUX    )
L02353             ELSE
L02354              CALL QNSESFC(itimestep,ht,dz8w,                           &
L02355                p_phy,p8w,th_phy,t_phy,                                 &
L02356                qv_curr,qc_curr,                                        &
L02357                u_phy,v_phy,tke_pbl,                                    &
L02358                tsk,qsfc,thz0,qz0,uz0,vz0,                              &
L02359                lowlyr,                                                 &
L02360                xland,                                                  &
L02361                ust,znt,z0,pblh,mavail,rmol,                            &
L02362                akhs,akms,                                              &
L02363                br,                                                    &
L02364                chs,chs2,cqs2,hfx,qfx,lh,flhc,flqc,qgh,cpm,ct,          &
L02365                u10,v10,t2,th2,tshltr,th10,q2,qshltr,q10,pshltr,        &
L02366                u10e,v10e,                                              &
L02367                ids,ide, jds,jde, kds,kde,                              &
L02368                ims,ime, jms,jme, kms,kme,                              &
L02369                i_start(ij),i_end(ij), j_start(ij),j_end(ij),     &
L02370                kts,kte,scm_force_flux    )
L02371             ENDIF
L02372  #if ( EM_CORE==1)
L02373          DO j = j_start(ij),j_end(ij)
L02374             DO i = i_start(ij),i_end(ij)
L02375                wspd(i,j) = MAX(SQRT(u_phy(i,kts,j)**2+v_phy(i,kts,j)**2),0.001)
L02376                ch(i,j) = chs (i,j)
L02377  !!           ch(i,j) = flhc(i,j)/( cpm(i,j)*rho(i,kts,j) )
L02378             END DO
L02379          END DO
L02380  #endif
L02381        ELSE
L02382          CALL wrf_error_fatal('Lacking arguments for QNSESFC in surface driver')
L02383        ENDIF
L02384
L02385      CASE (GFSSFCSCHEME)
L02386        IF (PRESENT(qv_curr) .AND. .TRUE. ) THEN
L02387        CALL wrf_debug( 100, 'in GFSSFC' )
```

```
L02388         IF (FRACTIONAL_SEAICE == 1) THEN
L02389            CALL SF_GFS_SEAICE_WRAPPER(u_phytmp,v_phytmp,t_phy,qv_curr, &
L02390                 p_phy,CP,RCP,R_d,XLV,PSFC,CHS,CHS2,CQS2,CPM,        &
L02391                 ZNT,UST,PSIM,PSIH,                                  &
L02392                 XLAND,HFX,QFX,LH,TSK,FLHC,FLQC,                     &
L02393                 QGH,QSFC,U10,V10,                                   &
L02394                 GZ1OZ0,WSPD,BR,ISFFLX,                              &
L02395                 EP_1,EP_2,KARMAN,itimestep,                         &
L02396                 TICE2TSK_IF2COLD,                                &
L02397                 XICE_THRESHOLD,                                  &
L02398                 CHS_SEA, CHS2_SEA, CPM_SEA, CQS2_SEA,            &
L02399                 FLHC_SEA, FLQC_SEA,                              &
L02400                 HFX_SEA, LH_SEA, QFX_SEA, QGH_SEA, QSFC_SEA, &
L02401                 UST_SEA, ZNT_SEA, SST, XICE,                     &
L02402                 ids,ide, jds,jde, kds,kde,                          &
L02403                 ims,ime, jms,jme, kms,kme,                          &
L02404                 i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02405         ELSE
L02406            CALL SF_GFS(u_phytmp,v_phytmp,t_phy,qv_curr,             &
L02407                 p_phy,CP,RCP,R_d,XLV,PSFC,CHS,CHS2,CQS2,CPM,        &
L02408                 ZNT,UST,PSIM,PSIH,                                  &
L02409                 XLAND,HFX,QFX,LH,TSK,FLHC,FLQC,                     &
L02410                 QGH,QSFC,U10,V10,                                   &
L02411                 GZ1OZ0,WSPD,BR,ISFFLX,                              &
L02412                 EP_1,EP_2,KARMAN,itimestep,                         &
L02413                 ids,ide, jds,jde, kds,kde,                          &
L02414                 ims,ime, jms,jme, kms,kme,                          &
L02415                 i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02416         ENDIF
L02417           CALL wrf_debug(100,'in SFCDIAGS')
L02418          ELSE
L02419            CALL wrf_error_fatal('Lacking arguments for SF_GFS in surface driver')
L02420         ENDIF
L02421
L02422  #if ( EM_CORE==1)
L02423      CASE(MYNNSFCSCHEME)
L02424
L02425         IF (PRESENT(qv_curr)    .AND.  PRESENT(qc_curr)     &
L02426             & .AND.  PRESENT(qcg) ) THEN
L02427
L02428            CALL wrf_debug(100,'in MYNNSFC')
L02429
L02430           IF (FRACTIONAL_SEAICE == 1) THEN
L02431            CALL MYNN_SEAICE_WRAPPER(                                 &
L02432                 u3d=u_phytmp,v3d=v_phytmp,t3d=t_phy,qv3d=qv_curr,     &
L02433                 p3d=p_phy,dz8w=dz8w,th3d=th_phy,rho=rho,              &
L02434                 psfc=psfc,chs=chs,chs2=chs2,cqs=cqs,cqs2=cqs2,cpm=cpm,&
L02435                 znt=znt,ust=ust,pblh=pblh,mavail=mavail,zol=zol,      &
L02436                 mol=mol,psim=psim,psih=psih,qgh=qgh,                  &
L02437                 xland=xland,hfx=hfx,qfx=qfx,lh=lh,tsk=tsk,            &
L02438                 flhc=flhc,flqc=flqc,qsfc=qsfc,rmol=rmol,              &
L02439                 u10=u10,v10=v10,th2=th2,t2=t2,q2=q2,snowh=snowh,      &
L02440                 gz1oz0=gz1oz0,wspd=wspd,br=br,dx=dx2d,                &
L02441                 itimestep=itimestep,ch=ch,                            &
L02442                 spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl,      &
L02443                 xice=XICE,sst=SST,tsk_sea=TSK_SEA,qgh_sea=QGH_SEA,    &
L02444                 chs2_sea=CHS2_SEA,chs_sea=CHS_SEA,cpm_sea=CPM_SEA,    &
L02445                 cqs2_sea=CQS2_SEA,flhc_sea=FLHC_SEA,                  &
L02446                 flqc_sea=FLQC_SEA,hfx_sea=HFX_SEA,lh_sea=LH_SEA,      &
L02447                 qfx_sea=QFX_SEA,qsfc_sea=QSFC_SEA,                    &
L02448                 znt_sea=ZNT_SEA,tice2tsk_if2cold=TICE2TSK_IF2COLD,    &
L02449                 xice_threshold=XICE_THRESHOLD,                        &
L02450                 ids=ids,ide=ide, jds=jds,jde=jde, kds=kds,kde=kde,    &
L02451                 ims=ims,ime=ime, jms=jms,jme=jme, kms=kms,kme=kme,    &
L02452                 its=i_start(ij),ite=i_end(ij),                        &
L02453                 jts=j_start(ij),jte=j_end(ij),kts=kts,kte=kte,        &
L02454                 ustm=ustm,ck=ck,cka=cka,cd=cd,cda=cda,                &
L02455                 sf_mynn_sfcflux_land=sf_mynn_sfcflux_land,            &
L02456                 sf_mynn_sfcflux_water=sf_mynn_sfcflux_water,          &
L02457                 isfflx=isfflx,shalwater_z0=shalwater_z0,              &
L02458                 water_depth=water_depth,lakemask=lakemask,            &
L02459                 flag_lsm=sf_surface_physics,restart=restart_flag,     &
L02460                 errmsg=errmsg,errflg=errflg                           )
L02461           ELSE
L02462            CALL mynnsfc_driver(u3d=u_phytmp,v3d=v_phytmp,t3d=t_phy, &
L02463                 qv3d=qv_curr,p3d=p_phy,dz8w=dz8w,th3d=th_phy,         &
L02464                 rho3d=rho,psfcpa=psfc,                                &
L02465                 chs=chs,chs2=chs2,cqs=cqs,cqs2=cqs2,cpm=cpm,          &
L02466                 znt=znt,ust=ust,pblh=pblh,mavail=mavail,zol=zol,      &
L02467                 mol=mol,psim=psim,psih=psih,qgh=qgh,                  &
L02468                 xland=xland,hfx=hfx,qfx=qfx,lh=lh,tsk=tsk,            &
L02469                 flhc=flhc,flqc=flqc,qsfc=qsfc,rmol=rmol,              &
L02470                 u10=u10,v10=v10,th2=th2,t2=t2,q2=q2,snowh=snowh,      &
L02471                 gz1oz0=gz1oz0,wspd=wspd,br=br,dx=dx2d,                &
L02472                 itimestep=itimestep,ch=ch,                            &
L02473                 spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl,      &
L02474                 ids=ids,ide=ide, jds=jds,jde=jde, kds=kds,kde=kde,    &
L02475                 ims=ims,ime=ime, jms=jms,jme=jme, kms=kms,kme=kme,    &
L02476                 its=i_start(ij),ite=i_end(ij),jts=j_start(ij),        &
L02477                 jte=j_end(ij), kts=kts,kte=kte,                       &
L02478                 ustm=ustm,ck=ck,cka=cka,cd=cd,cda=cda,                &
L02479                 xice=xice,xice_threshold=xice_threshold,              &
L02480                 sf_mynn_sfcflux_land=sf_mynn_sfcflux_land,            &
L02481                 sf_mynn_sfcflux_water=sf_mynn_sfcflux_water,          &
L02482                 isfflx=isfflx,shalwater_z0=shalwater_z0,              &
L02483                 water_depth=water_depth,lakemask=lakemask,            &
L02484                 flag_lsm=sf_surface_physics,restart=restart_flag,     &
L02485                 errmsg=errmsg,errflg=errflg                           )
L02486           ENDIF
L02487         ELSE
L02488            CALL wrf_error_fatal('Lacking arguments for SFCLAY_mynn in surface driver')
L02489         ENDIF
L02490  #endif
L02491
L02492  #if ( EM_CORE==1)
L02493       CASE (TEMFSFCSCHEME)
L02494         IF (PRESENT(qv_curr).and.PRESENT(hd_temf)) THEN
L02495           CALL wrf_debug( 100, 'in TEMFSFCLAY' )
L02496  ! WA 9/7/09 must initialize Z0 and ZNT for TEMF in ideal cases
```

```
L02497         ! DO J=j_start(ij),j_end(ij)
L02498         ! DO I=i_start(ij),i_end(ij)
L02499         !    CHKLOWQ(i,j) = 1.0
L02500         !    Z0(i,j) = 0.03       ! For GABLS2
L02501         !    ZNT(i,j) = 0.03      ! For GABLS2
L02502         ! ENDDO
L02503         ! ENDDO
L02504           CALL TEMFSFCLAY(u3d=u_phytmp,v3d=v_phytmp,th3d=th_phy,    &
L02505                qv3d=qv_curr,p3d=p_phy,pi3d=pi_phy,rho=rho,z=z,ht=ht, &
L02506                CP=cp,G=g,ROVCP=rcp,R=r_d,XLV=xlv,psfc=psfc,chs=chs,&
L02507                chs2=chs2,cqs2=cqs2,CPM=cpm,znt=znt,ust=ust,        &
L02508                MAVAIL=mavail,XLAND=xland,HFX=hfx,QFX=qfx,LH=lh,   &
L02509                TSK=tsk,FLHC=flhc,FLQC=flqc,QGH=qgh,qsfc=qsfc,      &
L02510                U10=u10,V10=v10,TH2=th2,T2=t2,Q2=q2,                &
L02511                SVP1=svp1,SVP2=svp2,SVP3=svp3,SVPT0=svpt0,EP1=ep_1, &
L02512                EP2=ep_2,KARMAN=karman,fCor=fCor,te_temf=te_temf,   &
L02513                hd_temf=hd_temf,exch_temf=exch_temf,wm_temf=wm_temf,&
L02514                ids=ids,ide=ide, jds=jds,jde=jde, kds=kds,kde=kde,  &
L02515                ims=ims,ime=ime, jms=jms,jme=jme, kms=kms,kme=kme,  &
L02516                its=i_start(ij),ite=i_end(ij),                      &
L02517                jts=j_start(ij),jte=j_end(ij), kts=kts,kte=kte )
L02518         ELSE
L02519           CALL wrf_error_fatal('Lacking arguments for TEMFSFCLAY in surface driver')
L02520         ENDIF
L02521
L02522       CASE (IDEALSCMSFCSCHEME)
L02523         IF (PRESENT(qv_curr)) THEN
L02524           CALL wrf_debug( 100, 'in IDEALSCMSFCLAY' )
L02525           CALL IDEALSCMSFCLAY(u3d=u_phytmp,v3d=v_phytmp,th3d=th_phy,    &
L02526                qv3d=qv_curr,p3d=p_phy,pi3d=pi_phy,rho=rho,z=z,ht=ht, &
L02527                CP=cp,G=g,ROVCP=rcp,R=r_d,XLV=xlv,psfc=psfc,chs=chs,  &
L02528                chs2=chs2,cqs2=cqs2,CPM=cpm,znt=znt,ust=ust,        &
L02529                MAVAIL=mavail,XLAND=xland,HFX=hfx,QFX=qfx,LH=lh,   &
L02530                TSK=tsk,FLHC=flhc,FLQC=flqc,QGH=qgh,qsfc=qsfc,      &
L02531                U10=u10,V10=v10,TH2=th2,T2=t2,Q2=q2,                &
L02532                SVP1=svp1,SVP2=svp2,SVP3=svp3,SVPT0=svpt0,EP1=ep_1, &
L02533                EP2=ep_2,KARMAN=karman,fCor=fCor,   &
L02534                exch_temf=exch_temf,                &
L02535                hfx_force=hfx_force,lh_force=lh_force,tsk_force=tsk_force, &
L02536                hfx_force_tend=hfx_force_tend,                      &
L02537                lh_force_tend=lh_force_tend,                        &
L02538                tsk_force_tend=tsk_force_tend,                      &
L02539                dt=dt,itimestep=itimestep,                          &
L02540                ids=ids,ide=ide, jds=jds,jde=jde, kds=kds,kde=kde,  &
L02541                ims=ims,ime=ime, jms=jms,jme=jme, kms=kms,kme=kme,  &
L02542                its=i_start(ij),ite=i_end(ij),                      &
L02543                jts=j_start(ij),jte=j_end(ij), kts=kts,kte=kte )
L02544         ELSE
L02545           CALL wrf_error_fatal('Lacking arguments for IDEALSCMSFCLAY in surface driver')
L02546         ENDIF
L02547  #endif
L02548
L02549       CASE DEFAULT
L02550
L02551         WRITE( message , * )                                &
L02552     'The sfclay option does not exist: sf_sfclay_physics = ', sf_sfclay_physics
L02553         CALL wrf_error_fatal ( message )
L02554
L02555       END SELECT sfclay_select
```

讲解：九个面层方案的调用分派，逐案看三个维度：① **实参口径**——SFCLAY/MRF/GFS 族要 SVP1/SVP2/…/STBOLT 常数组、MYJ 要 CHKLOWQ/THZ0/QZ0/UZ0/VZ0（面层-PBL 一体化回传）、MYNN 走 mynnsfc_driver 新接口（errmsg/errflg，code_10 §3.4）；② **海冰前置**——MYJ/QNSE/GFS/MYNN 四案有 seaice_wrapper（本篇 § 六收 MYNN 版），在 CASE 内先调 wrapper 处理 xice>xice_threshold 的点再调方案主体；③**TSK 语义**——方案实参统一传 tskold（本步开始时的旧地温）：面层通量用旧地温、LSM 更新后地温，避免同一步内自馈。CASE DEFAULT 报错。每案 CALL 前的 PRESENT 守卫与 code_22 同款。

五、陆面方案族 sfc_select：SLAB 与 Noah 代表段

```
!file: phys/module_surface_driver.F    L2629-L3133
L02629       CASE (SLABSCHEME)
L02630
L02631         IF (PRESENT(qv_curr)                              .AND.    &
L02632             PRESENT(capg)        .AND.    &
L02633                                                          .TRUE. ) THEN
L02634             DO j=j_start(ij),j_end(ij)
L02635             DO i=i_start(ij),i_end(ij)
L02636  !          CQS2 ACCOUNTS FOR MAVAIL FOR SFCDIAGS 2M Q
L02637                CQS2(I,J)= CQS2(I,J)*MAVAIL(I,J)
L02638             ENDDO
L02639             ENDDO
L02640
L02641          CALL wrf_debug(100,'in SLAB')
L02642            CALL SLAB(t_phy,qv_curr,p_phy,flhc,flqc,  &
L02643               psfc,xland,tmn,hfx,qfx,lh,tsk,qsfc,chklowq,          &
L02644               gsw,glw,capg,thc,snowc,emiss,mavail,                 &
L02645               dtbl,rcp,xlv,dtmin,ifsnow,                           &
L02646               svp1,svp2,svp3,svpt0,ep_2,karman,eomeg,stbolt,       &
L02647               tslb,zs,dzs,num_soil_layers,radiation,               &
L02648               p1000mb,                                             &
L02649               ids,ide, jds,jde, kds,kde,                           &
L02650               ims,ime, jms,jme, kms,kme,                           &
L02651               i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte)
L02652
L02653             DO j=j_start(ij),j_end(ij)
L02654             DO i=i_start(ij),i_end(ij)
```

```
L02655                SFCEVP(I,J)= SFCEVP(I,J) + QFX(I,J)*DTBL
L02656                IF(PRESENT(ACHFX))ACHFX(I,J)=ACHFX(I,J) + HFX(I,J)*DT
L02657                IF(PRESENT(ACLHF))ACLHF(I,J)=ACLHF(I,J) + LH(I,J)*DT
L02658              ENDDO
L02659              ENDDO
L02660
L02661           CALL wrf_debug(100,'in SFCDIAGS')
L02662             CALL SFCDIAGS(hfx,qfx,tsk,qsfc,chs2,cqs2,t2,th2,q2,      &
L02663                           psfc,cp,r_d,rcp,CHS,t_phy,qv_curr,ua_phys,    &
L02664                           ids,ide, jds,jde, kds,kde,                    &
L02665                           ims,ime, jms,jme, kms,kme,                    &
L02666                i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
L02667
L02668           ENDIF
L02669
L02670         CASE (LSMSCHEME)
L02671
L02672           IF (PRESENT(qv_curr)    .AND.  PRESENT(rainbl)        .AND.    &
L02673  !          PRESENT(emiss)      .AND.  PRESENT(t2)            .AND.    &
L02674  !          PRESENT(declin) .AND.  PRESENT(coszen)    .AND.    &
L02675  !          PRESENT(hrang)  .AND. PRESENT( xlat_urb2d)    .AND.    &
L02676  !          PRESENT(dzr)        .AND.    &
L02677  !          PRESENT( dzb)           .AND. PRESENT(dzg)        .AND.    &
L02678  !          PRESENT(tr_urb2d) .AND. PRESENT(tb_urb2d)         .AND.    &
L02679  !          PRESENT(tg_urb2d) .AND. PRESENT(tc_urb2d) .AND.            &
L02680  !          PRESENT(qc_urb2d) .AND. PRESENT(uc_urb2d) .AND.            &
L02681  !          PRESENT(xxxr_urb2d) .AND. PRESENT(xxxb_urb2d) .AND.        &
L02682  !          PRESENT(xxxg_urb2d) .AND.                                  &
L02683  !          PRESENT(xxxc_urb2d) .AND. PRESENT(trl_urb3d) .AND.         &
L02684  !          PRESENT(tbl_urb3d)   .AND. PRESENT(tgl_urb3d)  .AND.       &
L02685  !          PRESENT(sh_urb2d) .AND. PRESENT(lh_urb2d)  .AND.           &
L02686  !          PRESENT(g_urb2d)   .AND. PRESENT(rn_urb2d) .AND.           &
L02687  !          PRESENT(ts_urb2d)                          .AND.           &
L02688  !          PRESENT(frc_urb2d) .AND. PRESENT(utype_urb2d)   .AND.      &
L02689                                                        .TRUE. ) THEN
L02690  !-----------------------------------------------------------------
L02691           IF( PRESENT(sr) ) THEN
L02692             frpcpn=.true.
L02693           ENDIF
L02694           IF ( FRACTIONAL_SEAICE == 1) THEN
L02695              ! The fields passed to LSM need to represent the full ice values, not
L02696              ! the fractional values.  Convert ALBEDO and EMISS from the blended value
L02697              ! to a value representing only the sea-ice portion.   Albedo over open
L02698              ! water is taken to be 0.08. Emissivity over open water is taken to be 0.98
L02699              DO j = j_start(ij) , j_end(ij)
L02700                 DO i = i_start(ij) , i_end(ij)
L02701                    IF ( ( XICE(I,J) .GE. XICE_THRESHOLD ) .AND. ( XICE(i,j) .LE. 1 ) ) THEN
L02702                       ALBEDO(I,J) = (ALBEDO(I,J)-(1.-XICE(I,J))*0.08)/XICE(I,J)
L02703                       EMISS(I,J) = (EMISS(I,J)-(1.-XICE(I,J))*0.98)/XICE(I,J)
L02704                    ENDIF
L02705                 ENDDO
L02706              ENDDO
L02707
L02708              IF ( isisfc ) THEN
L02709                 ! Use surface layer routine values from the ice portion of grid point
L02710              ELSE
L02711                 !
L02712                 ! We don't have surface layer routine values at this time, so
L02713                 ! just use what we have.  Use ice component of TSK
L02714                 !
L02715                 CALL get_local_ice_tsk( ims, ime, jms, jme,                 &
L02716                                         i_start(ij), i_end(ij),             &
L02717                                         j_start(ij), j_end(ij),             &
L02718                                         itimestep, .false., tice2tsk_if2cold, &
L02719                                         XICE, XICE_THRESHOLD,               &
L02720                                         SST, TSK, TSK_SEA, TSK_LOCAL )
L02721
L02722                 DO j = j_start(ij) , j_end(ij)
L02723                    DO i = i_start(ij) , i_end(ij)
L02724                       TSK(i,j) = TSK_LOCAL(i,j)
L02725                    ENDDO
L02726                 ENDDO
L02727              ENDIF
L02728           ENDIF
L02729
L02730  !added for WRF_HYDRO
L02731  #ifdef WRF_HYDRO
L02732           if(HYDRO_dt .ge. 0) HYDRO_dt = dtbl
L02733  #endif
L02734
L02735  ! added RA population for WRF/Noah-CMAQ RS Consistency
L02736  ! following Garland et al. (1977) and Nemitz et al., 2009
L02737           IF ( PRESENT(RA) ) THEN
L02738           DO j=j_start(ij),j_end(ij)
L02739           DO i=i_start(ij),i_end(ij)
L02740              RA(I,J) = WSPD(I,J)/UST(I,J)**2.0
L02741           ENDDO
L02742           ENDDO
L02743           END IF
L02744
L02745           CALL wrf_debug(100,'in NOAH DRV')
L02746
L02747           IF (sf_surface_mosaic == 1) THEN
L02748
L02749             IF ( PRESENT( TSK_mosaic ) .AND. PRESENT( HFX_mosaic ) ) THEN
L02750               CALL lsm_mosaic(dz8w,qv_curr,p8w,t_phy,tsk,                &
L02751                  hfx,qfx,lh,grdflx,qgh,gsw,swdown,glw,smstav,smstot,    &
L02752                  sfcrunoff,udrunoff,ivgtyp,isltyp,isurban,isice,vegfra,        &
L02753                  albedo,albbck,znt,z0, tmn,xland,xice,emiss, embck,    &
L02754                  snowc,qsfc,rainbl,                                    &
L02755                  mminlu,                                               &
L02756                  num_soil_layers,dtbl,dzs,itimestep,                   &
L02757                  smois,tslb,snow,canwat,                               &
L02758                  chs, chs2, cqs2, cpm,rcp,SR,chklowq,lai,qz0,    &
L02759                  myj,frpcpn,                                           &
L02760           sh2o,snowh,                                          & !h
L02761                  u_phy,v_phy,                                          & !I
L02762                  snoalb,shdmin,shdmax,shdavg,                          & !i
L02763                  snotime,                                              & !o
```

```
                  acsnom,acsnow,                                    & !o
                  snopcx,                                           & !o
                  potevp,                                           & !o
                  smcrel,                                           & !o
                  xice_threshold,                                   &
                  rdlai2d,usemonalb,                                &
                  br,                                               & !?
                  NOAHRES,opt_thcnd,                                &
                  NLCAT,landusef,landusef2,                         & ! danli mosaic
                  sf_surface_mosaic,mosaic_cat,mosaic_cat_index,    & ! danli mosaic
                  TSK_mosaic,QSFC_mosaic,                           & ! danli mosaic
                  TSLB_mosaic,SMOIS_mosaic,SH2O_mosaic,             & ! danli mosaic
                  CANWAT_mosaic,SNOW_mosaic,                        & ! danli mosaic
                  SNOWH_mosaic,SNOWC_mosaic,                        & ! danli mosaic
                  ALBEDO_mosaic,ALBBCK_mosaic,                      & ! danli mosaic
                  EMISS_mosaic, EMBCK_mosaic,                       & ! danli mosaic
                  ZNT_mosaic, Z0_mosaic,                            & ! danli mosaic
                  HFX_mosaic,QFX_mosaic,                            & ! danli mosaic
                  LH_mosaic, GRDFLX_mosaic, SNOTIME_mosaic,         & ! danli mosaic
                  RS_mosaic, LAI_mosaic,                            & ! mosaic
                  ua_phys,flx4,fvb,fbur,fgsn,                       &
                  ids,ide, jds,jde, kds,kde,                        &
                  ims,ime, jms,jme, kms,kme,                        &
                  i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
                  sf_urban_physics                                  &
!Optional urban
                  ,cmr_sfcdif,chr_sfcdif,cmc_sfcdif,chc_sfcdif      &
                  ,cmgr_sfcdif,chgr_sfcdif                          &
                  ,tr_urb2d,tb_urb2d,tg_urb2d,tc_urb2d,qc_urb2d,    & !H urban
                  uc_urb2d,                                         & !H urban
                  xxxr_urb2d,xxxb_urb2d,xxxg_urb2d,xxxc_urb2d,      & !H urban
                  trl_urb3d,tbl_urb3d,tgl_urb3d,                    & !H urban
                  sh_urb2d,lh_urb2d,g_urb2d,rn_urb2d,ts_urb2d,      & !H urban
                  TR_URB2D_mosaic,TB_URB2D_mosaic,                  & !H urban  danli mosaic
                  TG_URB2D_mosaic,TC_URB2D_mosaic,                  & !H urban  danli mosaic
                  QC_URB2D_mosaic,UC_URB2D_mosaic,                  & !H urban  danli mosaic
                  TRL_URB3D_mosaic,TBL_URB3D_mosaic,                & !H urban  danli mosaic
                  TGL_URB3D_mosaic,                                 & !H urban  danli mosaic
                  SH_URB2D_mosaic,LH_URB2D_mosaic,                  & !H urban  danli mosaic
                  G_URB2D_mosaic,RN_URB2D_mosaic,                   & !H urban  danli mosaic
                  TS_URB2D_mosaic,                                  & !H urban  danli mosaic
                  TS_RUL2D_mosaic,                                  & !H urban  danli mosaic
                  psim_urb2d,psih_urb2d,u10_urb2d,v10_urb2d,        & !O urban
                  GZ1OZ0_urb2d, AKMS_URB2D,                         & !O urban
                  th2_urb2d,q2_urb2d,ust_urb2d,                     & !O urban
                  declin,coszen,hrang,                              & !I solar
                  xlat_urb2d,                                       & !I urban
                  num_roof_layers, num_wall_layers,                 & !I urban
                  num_road_layers, DZR, DZB, DZG,                   & !I urban
                  CMCR_URB2D,TGR_URB2D,TGRL_URB3D,SMR_URB3D,        & !H urban
                  julian,julyr,                                     & !H urban
                  DRELR_URB2D,DRELB_URB2D,DRELG_URB2D,              & !H urban
                  FLXHUMR_URB2D,FLXHUMB_URB2D,FLXHUMG_URB2D,        & !H urban
                  ! added by Chenghao Wang
                  TVG_URB2D,XXXVG_URB2D,TVGL_URB3D,SMG_URB3D,TT_URB2D,CMCG_URB2D, & !H urban
                  FLXHUMVG_URB2D,FLXHUMT_URB2D,                     & !H urban
                  FRC_URB2D, UTYPE_URB2D,                           & !I urban
                  num_urban_ndm,                                    & !I multi-layer urban
                  urban_map_zrd,                                    & !I multi-layer urban
                  urban_map_zwd,                                    & !I multi-layer urban
                  urban_map_gd,                                     & !I multi-layer urban
                  urban_map_zd,                                     & !I multi-layer urban
                  urban_map_zdf,                                    & !I multi-layer urban
                  urban_map_bd,                                     & !I multi-layer urban
                  urban_map_wd,                                     & !I multi-layer urban
                  urban_map_gbd,                                    & !I multi-layer urban
                  urban_map_fbd,                                    & !I multi-layer urban
                  urban_map_zgrd,                                   & !I multi-layer urban
                  num_urban_hi,                                     & !I multi-layer urban
                  use_wudapt_lcz,                                   & !I wudapt
                  slucm_distributed_drag,                           & !I SLUCM
                  tsk_rural,                                        & !H multi-layer urban
                  trb_urb4d,tw1_urb4d,tw2_urb4d,tgb_urb4d,          & !H multi-layer urban
                  tlev_urb3d,qlev_urb3d,                            & !H multi-layer urban
                  tw1lev_urb3d,tw2lev_urb3d,                        & !H multi-layer urban
                  tglev_urb3d,tflev_urb3d,                          & !H multi-layer urban
                  sf_ac_urb3d,lf_ac_urb3d,cm_ac_urb3d,              & !H multi-layer urban
                  sfvent_urb3d,lfvent_urb3d,                        & !H multi-layer urban
                  sfwin1_urb3d,sfwin2_urb3d,                        & !H multi-layer urban
                  sfw1_urb3d,sfw2_urb3d,sfr_urb3d,sfg_urb3d,        & !H multi-layer urban
                   ep_pv_urb3d,t_pv_urb3d,                           & !GRZ
                  trv_urb4d,qr_urb4d,qgr_urb3d,tgr_urb3d,           & !GRZ
                  drain_urb4d,draingr_urb3d,sfrv_urb3d,             & !GRZ
                  lfrv_urb3d,dgr_urb3d,dg_urb3d,lfr_urb3d,lfg_urb3d,& !GRZ
                  lp_urb2d,hi_urb2d,lb_urb2d,hgt_urb2d,             & !H multi-layer urban
                  mh_urb2d,stdh_urb2d,lf_urb2d,                     & !SLUCM
                  lf_urb2d_s, z0_urb2d,                             & !SLUCM
                  th_phy,rho,p_phy,ust,                             & !I multi-layer urban
                  gmt,julday,xlong,xlat,                            & !I multi-layer urban
                  a_u_bep,a_v_bep,a_t_bep,a_q_bep,                  & !O multi-layer urban
                  a_e_bep,b_u_bep,b_v_bep,                          & !O multi-layer urban
                  b_t_bep,b_q_bep,b_e_bep,dlg_bep,                  & !O multi-layer urban
                  dl_u_bep,sf_bep,vl_bep                            & !O multi-layer urban
#ifdef WRF_HYDRO
                  ,sfcheadrt,INFXSRT, soldrain                      & !hydro
#endif
                  ,SDA_HFX, SDA_QFX, HFX_BOTH, QFX_BOTH, QNORM, fasdas    & ! fasdas
                  ,RS,XLAIDYN,IRRIGATION_CHANNEL)
             ELSE
                 CALL wrf_error_fatal('Lack arguments to call lsm_mosaic')
             ENDIF

       ELSEIF (sf_surface_mosaic == 0) THEN

!
!  FASDAS
!
               IF( fasdas == 1 ) THEN
                 DO j=j_start(ij),j_end(ij)
```

```
L02873                DO i=i_start(ij),i_end(ij)
L02874
L02875  !ckay2015 only do indirect nudging over land areas
L02876                     IF(XLAND(i,j) .GT. 1.5) then
L02877                      SDA_QFX(I,J) = 0.0
L02878                      SDA_HFX(I,J) = 0.0
L02879                     END IF
L02880
L02881  ! TWG2015 Removed lines that update fluxes to ensure this section only defines
L02882  ! the output
L02883                     QFXOLD(I,J)=QFX(I,J)
L02884                     QFX_KAY = SDA_QFX(I,J)*RHO(I,1,J)*DZ8W(I,1,J)
L02885                     QFX_KAY = QFX_KAY * QNORM(I,J)
L02886                     QFX_BOTH(I,J)=QFX(I,J)+QFX_KAY
L02887
L02888                     HFXOLD(I,J)=HFX(I,J)
L02889                     HFX_KAY = SDA_HFX(I,J)*RHO(I,1,J)*CPM(I,J)*DZ8W(I,1,J)
L02890                     HFX_BOTH(I,J)=HFX(I,J)+HFX_KAY
L02891
L02892                ENDDO
L02893                ENDDO
L02894              END IF
L02895  !
L02896  !  END FASDAS
L02897  !
L02898                if (pert_noah .and. multi_perturb == 1) then
L02899                  allocate (tslb_tmp(i_start(ij):i_end(ij), 1:num_soil_layers, j_start(ij):j_end(ij)))
L02900                  allocate (smois_tmp(i_start(ij):i_end(ij), 1:num_soil_layers, j_start(ij):j_end(ij)))
L02901
L02902                  call Add_multi_perturb_lsm_perturbations (perts_qvapor, perts_th, perts_smois, perts_tsoil, &
L02903                    pert_noah_qv, pert_noah_t, pert_noah_smois, pert_noah_tslb, t_phy, qv_curr, tslb, smois, &
L02904                    tslb_tmp, smois_tmp, num_soil_layers, i_start(ij), i_end(ij), j_start(ij), j_end(ij), ims, &
L02905                    ime, jms, jme, kms, kme, kts, kte)
L02906                end if
L02907
L02908                CALL lsm(dz8w,qv_curr,p8w,t_phy,tsk,                &
L02909                 hfx,qfx,lh,grdflx,qgh,gsw,swdown,swddir,swddif,     &
L02910                 glw,smstav,smstot,    &
L02911                 sfcrunoff,udrunoff,ivgtyp,isltyp,isurban,isice,vegfra,        &
L02912                 albedo,albbck,znt,z0, tmn,xland,xice,emiss, embck,    &
L02913                 snowc,qsfc,rainbl,                                  &
L02914                 mminlu,                                             &
L02915                 num_soil_layers,dtbl,dzs,itimestep,                 &
L02916                 smois,tslb,snow,canwat,                             &
L02917                 chs, chs2, cqs2, cpm,rcp,SR,chklowq,lai,qz0,    &
L02918                 myj,frpcpn,                                         &
L02919          sh2o,snowh,                                       & !h
L02920                 u_phy,v_phy,                                        & !I
L02921                 snoalb,shdmin,shdmax,                               & !i
L02922                 snotime,                                            & !o
L02923                 acsnom,acsnow,                                      & !o
L02924                 snopcx,                                             & !o
L02925                 potevp,                                             & !o
L02926                 smcrel,                                             & !o
L02927                 xice_threshold,                                     &
L02928                 rdla12d,usemonalb,                                  &
L02929                 br,                                                 & !?
L02930                 NOAHRES,opt_thcnd,                                  &
L02931                 ua_phys,flx4,fvb,fbur,fgsn,                         &
L02932                 ids,ide, jds,jde, kds,kde,                          &
L02933                 ims,ime, jms,jme, kms,kme,                          &
L02934                 i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte,    &
L02935                 sf_urban_physics                                    &
L02936  !Optional urban
L02937                 ,cmr_sfcdif,chr_sfcdif,cmc_sfcdif,chc_sfcdif    &
L02938                 ,cmgr_sfcdif,chgr_sfcdif                            &
L02939                 ,tr_urb2d,tb_urb2d,tg_urb2d,tc_urb2d,qc_urb2d,  & !H urban
L02940                 uc_urb2d,                                           & !H urban
L02941                 xxxr_urb2d,xxxb_urb2d,xxxg_urb2d,xxxc_urb2d,    & !H urban
L02942                 trl_urb3d,tbl_urb3d,tgl_urb3d,                  & !H urban
L02943                 sh_urb2d,lh_urb2d,g_urb2d,rn_urb2d,ts_urb2d,    & !H urban
L02944                 psim_urb2d,psih_urb2d,u10_urb2d,v10_urb2d,      & !O urban
L02945                 GZ1OZ0_urb2d, AKMS_URB2D,                       & !O urban
L02946                 th2_urb2d,q2_urb2d,ust_urb2d,                   & !O urban
L02947                 declin,coszen,hrang,                            & !I solar
L02948                 xlat_urb2d,                                     & !I urban
L02949                 num_roof_layers, num_wall_layers,               & !I urban
L02950                 num_road_layers, DZR, DZB, DZG,                 & !I urban
L02951                 CMCR_URB2D,TGR_URB2D,TGRL_URB3D,SMR_URB3D,      & !H urban
L02952                 DRELR_URB2D,DRELB_URB2D,DRELG_URB2D,            & !H urban
L02953                 FLXHUMR_URB2D,FLXHUMB_URB2D,FLXHUMG_URB2D,      & !H urban
L02954                 TVG_URB2D,XXXVG_URB2D,TVGL_URB3D,SMG_URB3D,TT_URB2D,CMCG_URB2D, & !H urban
L02955                 FLXHUMVG_URB2D,FLXHUMT_URB2D,                   & !H urban
L02956                 julian, julyr,                                  & !H urban
L02957                 FRC_URB2D, UTYPE_URB2D,                         & !I urban
L02958                 num_urban_ndm,                                  & !I multi-layer urban
L02959                 urban_map_zrd,                                  & !I multi-layer urban
L02960                 urban_map_zwd,                                  & !I multi-layer urban
L02961                 urban_map_gd,                                   & !I multi-layer urban
L02962                 urban_map_zd,                                   & !I multi-layer urban
L02963                 urban_map_zdf,                                  & !I multi-layer urban
L02964                 urban_map_bd,                                   & !I multi-layer urban
L02965                 urban_map_wd,                                   & !I multi-layer urban
L02966                 urban_map_gbd,                                  & !I multi-layer urban
L02967                 urban_map_fbd,                                  & !I multi-layer urban
L02968                 urban_map_zgrd,                                 & !I multi-layer urban
L02969                 num_urban_hi,                                   & !I multi-layer urban
L02970                 tsk_rural,                                      & !H multi-layer urban
L02971                 trb_urb4d,tw1_urb4d,tw2_urb4d,tgb_urb4d,        & !H multi-layer urban
L02972                 tlev_urb3d,qlev_urb3d,                          & !H multi-layer urban
L02973                 tw1lev_urb3d,tw2lev_urb3d,                      & !H multi-layer urban
L02974                 tglev_urb3d,tflev_urb3d,                        & !H multi-layer urban
L02975                 sf_ac_urb3d,lf_ac_urb3d,cm_ac_urb3d,            & !H multi-layer urban
L02976                 sfvent_urb3d,lfvent_urb3d,                      & !H multi-layer urban
L02977                 sfwin1_urb3d,sfwin2_urb3d,                      & !H multi-layer urban
L02978                 sfw1_urb3d,sfw2_urb3d,sfr_urb3d,sfg_urb3d,      & !H multi-layer urban
L02979                 ep_pv_urb3d,t_pv_urb3d,                         & !GRZ
L02980                 trv_urb4d,qr_urb4d,qgr_urb3d,tgr_urb3d,         & !GRZ
L02981                 drain_urb4d,draingr_urb3d,sfrv_urb3d,           & !GRZ
```

```
L02982                  lfrv_urb3d,dgr_urb3d,dg_urb3d,lfr_urb3d,lfg_urb3d,& !GRZ
L02983                  lp_urb2d,hi_urb2d,lb_urb2d,hgt_urb2d,           & !H multi-layer urban
L02984                  mh_urb2d,stdh_urb2D,lf_urb2d,                   & !SLUCM
L02985                  lf_urb2d_s, z0_urb2d,                           & !SLUCM
L02986                  th_phy,rho,p_phy,ust,                           & !I multi-layer urban
L02987                  gmt,julday,xlong,xlat,                          & !I multi-layer urban
L02988                  a_u_bep,a_v_bep,a_t_bep,a_q_bep,                & !O multi-layer urban
L02989                  a_e_bep,b_u_bep,b_v_bep,                        & !O multi-layer urban
L02990                  b_t_bep,b_q_bep,b_e_bep,dlg_bep,                & !O multi-layer urban
L02991                  dl_u_bep,sf_bep,vl_bep                          & !O multi-layer urban
L02992  #ifdef WRF_HYDRO
L02993                  ,sfcheadrt,INFXSRT, soldrain                    &
L02994  #endif
L02995                  ,SDA_HFX, SDA_QFX, HFX_BOTH, QFX_BOTH, QNORM, fasdas    &
L02996                  ,RS,XLAIDYN,IRRIGATION_CHANNEL)
L02997           ENDIF
L02998
L02999           if (pert_noah .and. multi_perturb == 1) then
L03000             call Remove_multi_perturb_lsm_perturbations (perts_qvapor, perts_th, perts_smois, perts_tsoil, &
L03001                 pert_noah_qv, pert_noah_t, pert_noah_smois, pert_noah_tslb, t_phy, qv_curr, tslb, smois, &
L03002                 tslb_tmp, smois_tmp, num_soil_layers, i_start(ij), i_end(ij), j_start(ij), j_end(ij), ims, &
L03003                 ime, jms, jme, kms, kme, kts, kte)
L03004             deallocate (tslb_tmp)
L03005             deallocate (smois_tmp)
L03006           end if
L03007
L03008           call seaice_noah( SEAICE_ALBEDO_OPT, SEAICE_ALBEDO_DEFAULT, SEAICE_THICKNESS_OPT, &
L03009                &            SEAICE_THICKNESS_DEFAULT, SEAICE_SNOWDEPTH_OPT,             &
L03010                &            SEAICE_SNOWDEPTH_MAX, SEAICE_SNOWDEPTH_MIN,                 &
L03011                &            t_phy, qv_curr, p8w, dz8w, num_soil_layers, dt, frpcpn, sr, &
L03012                &            glw, swdown, rainbl, snoalb, qgh, xice, xice_threshold,     &
L03013                &            albsi, icedepth, snowsi,                                    &
L03014                &            tslb, emiss, albedo, z0, tsk, snow, snowc, snowh,           &
L03015                &            chs, chs2, cqs2,                                            &
L03016                &            br, znt, lh, hfx, qfx, potevp, grdflx, qsfc, acsnow,        &
L03017                &            acsnom, snopcx, sfcrunoff, noahres,                         &
L03018                &            sf_urban_physics, b_t_bep, b_q_bep, rho,                    &
L03019                &            ids,ide, jds,jde, kds,kde,                                  &
L03020                &            ims,ime, jms,jme, kms,kme,                                  &
L03021                &            i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte )
L03022
L03023           IF ( FRACTIONAL_SEAICE == 1 ) THEN
L03024              ! LSM Returns full land/ice values, no fractional values.
L03025              ! We return to a fractional component here.  SFLX currently hard-wires
L03026              ! emissivity over sea ice to 0.98, the same value as over open water, so
L03027              ! the fractional consideration doesn't have any effect for emissivity.
L03028              DO j=j_start(ij),j_end(ij)
L03029                 DO i=i_start(ij),i_end(ij)
L03030                    IF ( ( XICE(I,J) .GE. XICE_THRESHOLD) .AND. ( XICE(i,j) .LE. 1.0 ) ) THEN
L03031                       albedo(i,j) = ( albedo(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * 0.08  )
L03032                       emiss(i,j) = ( emiss(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * 0.98  )
L03033                    ENDIF
L03034                 ENDDO
L03035              ENDDO
L03036
L03037              IF ( isisfc ) THEN
L03038                 DO j=j_start(ij),j_end(ij)
L03039                    DO i=i_start(ij),i_end(ij)
L03040                       IF ( ( XICE(I,J) .GE. XICE_THRESHOLD) .AND. ( XICE(i,j) .LE. 1.0 ) ) THEN
L03041                          !  Weighted average of fields between ice-cover values and open-water values.
L03042                          flhc(i,j) = ( flhc(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * flhc_sea(i,j) )
L03043                          flqc(i,j) = ( flqc(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * flqc_sea(i,j) )
L03044                          cpm(i,j)  = ( cpm(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * cpm_sea(i,j)  )
L03045                          cqs2(i,j) = ( cqs2(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * cqs2_sea(i,j) )
L03046                          chs2(i,j) = ( chs2(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * chs2_sea(i,j) )
L03047                          chs(i,j)  = ( chs(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * chs_sea(i,j)  )
L03048                          qsfc(i,j) = ( qsfc(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * qsfc_sea(i,j) )
L03049                          qgh(i,j)  = ( qgh(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * qgh_sea(i,j)  )
L03050                          qz0(i,j)  = ( qz0(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * qz0_sea(i,j)  )
L03051                                                                !  print *,'hfx =',hfx_sea(170,20)
L03052                                                                !   print *,'XICE =',XICE(170,20)
L03053                                                                !    print *,'QSFC =',QSFC(170,20)
L03054                          hfx(i,j)  = ( hfx(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * hfx_sea(i,j)  )
L03055                          qfx(i,j)  = ( qfx(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * qfx_sea(i,j)  )
L03056                          lh(i,j)   = ( lh(i,j)   * XICE(i,j) ) + ( (1.0-XICE(i,j)) * lh_sea(i,j)   )
L03057  !save old tsk_ice
L03058                          tsk_save(i,j)  = tsk(i,j)
L03059                          tsk(i,j)  = ( tsk(i,j)  * XICE(i,j) ) + ( (1.0-XICE(i,j)) * tsk_sea(i,j)  )
L03060                       ENDIF
L03061                    ENDDO
L03062                 ENDDO
L03063              ELSE
L03064                 DO j = j_start(ij) , j_end(ij)
L03065                    DO i = i_start(ij) , i_end(ij)
L03066                       IF ( ( XICE(I,J) .GE. XICE_THRESHOLD ) .AND. ( XICE(i,j) .LE. 1.0 ) ) THEN
L03067                          ! Compute TSK as the open-water and ice-cover average
L03068                          tsk(i,j) = ( tsk(i,j) * XICE(i,j) ) + ( (1.0-XICE(i,j)) * tsk_sea(i,j) )
L03069                       ENDIF
L03070                    ENDDO
L03071                 ENDDO
L03072              ENDIF
L03073           ENDIF
L03074             DO j=j_start(ij),j_end(ij)
L03075             DO i=i_start(ij),i_end(ij)
L03076  !             CHKLOWQ(I,J)= 1.0
L03077                SFCEVP(I,J)= SFCEVP(I,J) + QFX(I,J)*DTBL
L03078                SFCEXC(I,J)= CHS(I,J)
L03079                IF(PRESENT(ACHFX))ACHFX(I,J)=ACHFX(I,J) + HFX(I,J)*DT
L03080                IF(PRESENT(ACLHF))ACLHF(I,J)=ACLHF(I,J) + LH(I,J)*DT
L03081                IF(PRESENT(ACGRDFLX))ACGRDFLX(I,J)=ACGRDFLX(I,J) + GRDFLX(I,J)*DT
L03082             ENDDO
L03083             ENDDO
L03084
L03085            CALL SFCDIAGS(HFX,QFX,TSK,QSFC,CHS2,CQS2,T2,TH2,Q2,      &
L03086                          PSFC,CP,R_d,RCP,CHS,t_phy,qv_curr,ua_phys,    &
L03087                          ids,ide, jds,jde, kds,kde,                    &
L03088                          ims,ime, jms,jme, kms,kme,                    &
L03089               i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte    )
```

```
L03090  !urban
L03091        IF(SF_URBAN_PHYSICS.eq.1) THEN
L03092          DO j=j_start(ij),j_end(ij)                                !urban
L03093            DO i=i_start(ij),i_end(ij)                              !urban
L03094              IF(IVGTYP(I,J) == ISURBAN   .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L03095                 IVGTYP(I,J) == LCZ_3     .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L03096                 IVGTYP(I,J) == LCZ_6     .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L03097                 IVGTYP(I,J) == LCZ_9     .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 )THEN
L03098                 U10(I,J)  = U10_URB2D(I,J)                         !urban
L03099                 V10(I,J)  = V10_URB2D(I,J)                         !urban
L03100                 PSIM(I,J) = PSIM_URB2D(I,J)                        !urban
L03101                 PSIH(I,J) = PSIH_URB2D(I,J)                        !urban
L03102                 GZ1OZ0(I,J) = GZ1OZ0_URB2D(I,J)                    !urban
L03103  !m               AKHS(I,J) = AKHS_URB2D(I,J)                      !urban
L03104                 AKHS(I,J) = CHS(I,J)                               !urban
L03105                 AKMS(I,J) = AKMS_URB2D(I,J)                        !urban
L03106               END IF                                               !urban
L03107             ENDDO                                                  !urban
L03108           ENDDO                                                    !urban
L03109         ENDIF
L03110  ! urban BEP
L03111        IF((SF_URBAN_PHYSICS.eq.2).OR.(SF_URBAN_PHYSICS.eq.3)) THEN
L03112          DO j=j_start(ij),j_end(ij)                                !urban
L03113            DO i=i_start(ij),i_end(ij)                              !urban
L03114              IF( IVGTYP(I,J) == ISURBAN    .or. IVGTYP(I,J) == LCZ_1 .or. IVGTYP(I,J) == LCZ_2 .or. &
L03115                  IVGTYP(I,J) == LCZ_3     .or. IVGTYP(I,J) == LCZ_4 .or. IVGTYP(I,J) == LCZ_5 .or. &
L03116                  IVGTYP(I,J) == LCZ_6     .or. IVGTYP(I,J) == LCZ_7 .or. IVGTYP(I,J) == LCZ_8 .or. &
L03117                  IVGTYP(I,J) == LCZ_9     .or. IVGTYP(I,J) == LCZ_10 .or. IVGTYP(I,J) == LCZ_11 )THEN
L03118                T2(I,J)   = TH_PHY(i,1,j)/((1.E5/PSFC(I,J))**RCP) !urban
L03119                TH2(I,J) = TH_PHY(i,1,j) !urban
L03120                Q2(I,J)   = qv_curr(i,1,j)  !urban
L03121                U10(I,J)  = U_phy(I,1,J)                                !urban
L03122                V10(I,J)  = V_phy(I,1,J)                                !urban
L03123              END IF                                                    !urban
L03124            ENDDO                                                       !urban
L03125          ENDDO                                                         !urban
L03126        ENDIF
L03127
L03128  !-------------------------------------------------------------------
L03129
L03130          ELSE
L03131            CALL wrf_error_fatal('Lacking arguments for LSM in surface driver')
L03132          ENDIF
L03133
```

讲解：①**SLAB**（sf_surface_physics=1）：能量平衡单层地温 + 五层热扩散，实参最短——「最小陆面」参照；②**Noah**（=2，教材 16 章主角）：先 **lsm_mosaic**（mosaic_opt=1 时按植被类型分片多次调用——城市格点的自然片/人造片分离就在这里）再 CALL lsm（module_sf_noahdrv 的主入口，12/13 篇全文精读）；实参含 TSLB/ZS/DZS 土壤态、SFCEVP/FLXPOP 调试开关、海洋/湖面点跳过逻辑。其余六案（NoahMP=3/RUC=4/PX=7/CLM=8/CTSM=9/SSIB=6）结构同款：PRESENT 守卫→CALL→L4478 收口，不重复收录；CLM/Lake 的耦合变量（h2osoi_*3d 等）在收尾段的 Lake 调用可见。

六、收尾：海洋/湖泊/诊断限幅/人为热

```
!file: phys/module_surface_driver.F    L4478-L4619
L04478        END SELECT sfc_select
L04479
L04480        ENDDO
L04481        !$OMP END PARALLEL DO
L04482
L04483   430 CONTINUE
L04484
L04485  #if ( EM_CORE==1)
L04486     IF (sf_ocean_physics .EQ. OMLSCHEME .or. sf_ocean_physics .EQ. PWP3DSCHEME) THEN
L04487  ! simple ocean mixed layer model based Pollard, Rhines and Thompson (1973)
L04488        CALL wrf_debug( 100, 'Call OCEANML' )
L04489        !$OMP PARALLEL DO   &
L04490        !$OMP PRIVATE ( ij )
L04491        DO ij = 1 , num_tiles
L04492           CALL ocean_driver(tml,t0ml,hml,h0ml,huml,hvml,ust,u_phy,v_phy, &
L04493                             tmoml,f,g,oml_gamma,                         &
L04494                             xland,hfx,lh,tsk,gsw,glw,emiss,              &
L04495                             dtbl,STBOLT,oml_relaxation_time,             &
L04496                             ids,ide, jds,jde, kds,kde,                   &
L04497                             ims,ime, jms,jme, kms,kme,                   &
L04498                             i_start(ij),i_end(ij), j_start(ij),j_end(ij), kts,kte, &
L04499                             sf_ocean_physics,okms, okme,                 & !cyl
L04500                             om_tmp,om_s,om_u, om_v,  om_depth, om_ml,    & !cyl
L04501                             om_lat, om_lon,                              & !cyl
L04502                             QFX,                                         & !cyl
L04503                             rdx, rdy, msfu, msfv, msft,xtime,            & !cyl
L04504                             om_tini,om_sini,id,omdt,                     & !cyl
L04505                             itimestep )                                    !cyl
L04506        ENDDO
L04507        !$OMP END PARALLEL DO
L04508     ENDIF
L04509  #endif
L04510  ! adding a lake model -- 07/02/2010
L04511     IF ( LakeModel == 1 ) THEN
L04512
L04513        CALL wrf_debug( 100, 'Call LakeModel' )
L04514
L04515        DO ij = 1 , num_tiles
L04516
L04517           CALL Lake(  t_phy        ,p8w           ,dz8w         ,qv_curr         ,&  !i
L04518                       u_phy        ,v_phy         , glw         ,emiss           ,&
```

```
L04519                      rainbl       ,dtbl          ,swdown       ,albedo          ,&
L04520                      xlat_urb2d   ,z_lake3d      ,dz_lake3d    ,lakedepth2d     ,&
L04521                      watsat3d     ,csol3d        ,tkmg3d       ,tkdry3d         ,&
L04522                      tksatu3d     ,ivgtyp        ,ht           ,xland           ,&
L04523                      iswater      ,xice          ,xice_threshold, lake_min_elev    ,&
L04524                      ids          ,ide           ,jds          ,jde             ,&
L04525                      kds          ,kde           ,ims          ,ime             ,&
L04526                      jms          ,jme           ,kms          ,kme             ,&
L04527                      i_start(ij)  ,i_end(ij)     ,j_start(ij)  ,j_end(ij)       ,&
L04528                      kts          ,kte                                          ,&
L04529                      h2osno2d     ,snowdp2d      ,snl2d        ,z3d             ,&  !h
L04530                      dz3d         ,zi3d          ,h2osoi_vol3d ,h2osoi_liq3d    ,&
L04531                      h2osoi_ice3d ,t_grnd2d      ,t_soisno3d   ,t_lake3d        ,&
L04532                      savedtke12d  ,lake_icefrac3d                               ,&
L04533  #if ( EM_CORE==1)
L04534    !                  lakemask  ,lakeflag                                       ,&
L04535                      lakemask                                          ,&
L04536  #endif
L04537                      hfx          ,lh            ,grdflx       ,tsk             ,&  !o
L04538                      qfx          ,t2            ,th2          ,q2 )
L04539
L04540
L04541         ENDDO
L04542
L04543      ENDIF
L04544
L04545  ! Reset RAINBL in mm (Accumulation between PBL calls)
L04546
L04547        IF ( PRESENT( rainbl ) ) THEN
L04548          !$OMP PARALLEL DO   &
L04549          !$OMP PRIVATE ( ij, i, j, k )
L04550          DO ij = 1 , num_tiles
L04551            DO j=j_start(ij),j_end(ij)
L04552            DO i=i_start(ij),i_end(ij)
L04553               RAINBL(i,j) = 0.
L04554            ENDDO
L04555            ENDDO
L04556          ENDDO
L04557          !$OMP END PARALLEL DO
L04558        ENDIF
L04559
L04560  ! Limit Q2 diagnostic to no more than 5 per cent higher than lowest level value
L04561  !     This prevents unrealistic values when QFX is not mostly surface flux
L04562  !          because calculation is based on surface flux only
L04563  !     Problems occurred in transition periods and weak winds and vegetation source
L04564          !$OMP PARALLEL DO   &
L04565          !$OMP PRIVATE ( ij, i, j, k )
L04566          DO ij = 1 , num_tiles
L04567            DO j=j_start(ij),j_end(ij)
L04568            DO i=i_start(ij),i_end(ij)
L04569               IF (XLAND(I,J).LT.1.5) THEN
L04570               Q2(i,j) = MIN(Q2(i,j),1.05*QV_CURR(i,1,j))
L04571               END IF
L04572            ENDDO
L04573            ENDDO
L04574          ENDDO
L04575          !$OMP END PARALLEL DO
L04576
L04577        IF( PRESENT(slope_rad).AND. radiation )THEN
L04578  ! topographic slope effects removed from SWDOWN and GSW here for output
L04579          IF (slope_rad .EQ. 1) THEN
L04580
L04581          !$OMP PARALLEL DO   &
L04582          !$OMP PRIVATE ( ij, i, j, k )
L04583          DO ij = 1 , num_tiles
L04584            DO j=j_start(ij),j_end(ij)
L04585            DO i=i_start(ij),i_end(ij)
L04586            IF(SWNORM(I,J) .GT. 1.E-3)THEN  ! daytime
L04587               SWSAVE = SWDOWN(i,j)
L04588  ! SWDOWN contains unaffected SWDOWN in output
L04589               SWDOWN(i,j) = SWNORM(i,j)
L04590  ! SWNORM contains slope-affected SWDOWN in output
L04591               SWNORM(i,j) = SWSAVE
L04592               GSW(i,j) = GSWSAVE(i,j)
L04593            ENDIF
L04594            ENDDO
L04595            ENDDO
L04596          ENDDO
L04597          !$OMP END PARALLEL DO
L04598
L04599          ENDIF
L04600        ENDIF
L04601
L04602        IF (distributed_ahe_opt == 2) THEN
L04603          call cal_mon_day(julday, julyr, jmonth, jday)
L04604          ihour = (jmonth - 1) * 24 + MOD(INT(gmt + xtime / 60.0), 24)
L04605          !$OMP PARALLEL DO   &
L04606          !$OMP PRIVATE ( ij, i, j, k )
L04607          DO ij = 1, num_tiles
L04608            DO j = j_start(ij), j_end(ij)
L04609            DO i = i_start(ij), i_end(ij)
L04610              HFX(i, j) = HFX(i, j) + ahe(i, ihour, j)
L04611            END DO
L04612            END DO
L04613          END DO
L04614          !$OMP END PARALLEL DO
L04615        END IF
L04616
L04617      ENDIF run_param_if
L04618
L04619      END SUBROUTINE surface_driver
```

讲解：两族 SELECT 之后的五件事：① **海洋混合层** sf_ocean_physics∈{OML,PWP3D}：ocean_driver 用 Pollard–Rhines–Thompson 型夹卷更新 TML/HML——海温反馈的简单化替身；② **湖泊模型** Lake-Model=1：CLM-Lake（三维湖体分层的完整调用——水体温盐态 h2osoi_*/t_lake3d 直接覆盖 tsk/hfx）；③RAINBL 清零（本次陆面步的水账结清，下个 PBL 步重新累计）；④**Q2 限幅** min(Q2, 1.05 •

QV(1))——注释解释了原因：2 m 比湿由表面通量外插，弱风/植被蒸腾时段会失真，全 driver 统一兜底（与 code_10 §4.3 S10 的方案内限幅双保险）；⑤ **坡地辐射 SWDOWN/SWNORM 互换**（slope_rad=1 时输出用坡地修正值）与 **distributed_ahe_opt=2：AHE 直接加进 HFX**——对照 code_22 的 =1 路径（加最低层 RTHBLTEN）：两条人为热进场线，=1 走动力倾向（PBL 感受）、=2 走面层通量（LSM/PBL 同时感受），论文 2 选择时注意口径差异。

七、MYNN 海冰 wrapper（全文）

```
!file: phys/module_surface_driver.F    L5361-L5695    (subroutine mynn_seaice_wrapper)
L05361     SUBROUTINE mynn_seaice_wrapper(U3D,V3D,T3D,QV3D,                 &
L05362                P3D,dz8w,th3d,rho,                                    &
L05363                PSFC,CHS,CHS2,CQS,CQS2,CPM,                           &
L05364                ZNT,UST,PBLH,MAVAIL,ZOL,MOL,PSIM,PSIH,                &
L05365                XLAND,HFX,QFX,LH,TSK,FLHC,FLQC,QSFC,RMOL,QGH,         &
L05366                U10,V10,TH2,T2,Q2,SNOWH,                              &
L05367                GZ1OZ0,WSPD,BR,ISFFLX,DX,                             &
L05368                &itimestep,ch,                                        &
L05369                &spp_pbl,pattern_spp_pbl,                             &
L05370                XICE,SST,TSK_SEA,QGH_SEA,                             &
L05371                CHS2_SEA,CHS_SEA,CPM_SEA,CQS2_SEA,FLHC_SEA,FLQC_SEA, &
L05372                HFX_SEA,LH_SEA,QFX_SEA,QSFC_SEA,ZNT_SEA,              &
L05373                TICE2TSK_IF2COLD,XICE_THRESHOLD,                      &
L05374                ids,ide, jds,jde, kds,kde,                            &
L05375                ims,ime, jms,jme, kms,kme,                            &
L05376                its,ite, jts,jte, kts,kte,                            &
L05377                ustm,ck,cka,cd,cda,                                   &
L05378                sf_mynn_sfcflux_land,sf_mynn_sfcflux_water,           &
L05379                flag_lsm,restart,                                     &
L05380                shalwater_z0,water_depth,lakemask,                    &
L05381                errmsg,errflg                                         )
L05382
L05383       USE module_sf_mynnsfc_driver
L05384
L05385       implicit none
L05386
L05387       !--- ccpp arguments:
L05388       character(len=*), intent(inout) :: errmsg
L05389       integer,          intent(inout) :: errflg
L05390
L05391       INTEGER,  INTENT(IN )   ::       ids,ide, jds,jde, kds,kde,  &
L05392                                        ims,ime, jms,jme, kms,kme,  &
L05393                                        its,ite, jts,jte, kts,kte
L05394       INTEGER,  INTENT(IN )   ::       itimestep, ISFFLX
L05395       INTEGER,  INTENT(IN ), optional ::  sf_mynn_sfcflux_land,    &
L05396                                           sf_mynn_sfcflux_water,   &
L05397                                           flag_lsm,                &
L05398                                           shalwater_z0
L05399       LOGICAL,  INTENT(IN)    ::        restart
L05400       INTEGER,  INTENT(IN), optional      ::     spp_pbl
L05401       REAL,     DIMENSION( ims:ime, kms:kme, jms:jme ),            &
L05402                 INTENT(IN), OPTIONAL      ::   pattern_spp_pbl
L05403
L05404       REAL,     DIMENSION( ims:ime, kms:kme, jms:jme )           , &
L05405                 INTENT(IN   )   ::                            dz8w, &
L05406                                                               QV3D, &
L05407                                                                P3D, &
L05408                                                                T3D, &
L05409                                                                U3D, &
L05410                                                                V3D, &
L05411                                                            rho,th3d
L05412
L05413       REAL,     DIMENSION( ims:ime, jms:jme )                    , &
L05414                 INTENT(IN   )               ::              MAVAIL, &
L05415                                                              XLAND, &
L05416                                                         water_depth, &
L05417                                                            lakemask, &
L05418                                                                PSFC, &
L05419                                                               SNOWH, &
L05420                                                                 TSK, &
L05421                                                                  dx
L05422
L05423       REAL,     DIMENSION( ims:ime, jms:jme )                    , &
L05424                 INTENT(OUT  )               ::                U10, &
L05425                                                                V10, &
L05426                                                                TH2, &
L05427                                                                 T2, &
L05428                                                                 Q2
L05429
L05430       REAL,     DIMENSION( ims:ime, jms:jme )                    , &
L05431                 INTENT(INOUT)               ::                PBLH, &
L05432                                                                HFX, &
L05433                                                                QFX, &
L05434                                                                 LH, &
L05435                                                           MOL,RMOL, &
L05436                                                                QSFC, &
L05437                                                                 QGH, &
L05438                                                                 ZNT, &
L05439                                                                 ZOL, &
L05440                                                                 UST, &
L05441                                                                 CPM, &
L05442                                                                CHS2, &
L05443                                                                 CQS, &
L05444                                                                CQS2, &
L05445                                                                 CHS, &
L05446                                                                  CH, &
L05447                                                        FLHC,FLQC, &
L05448                                                     GZ1OZ0,WSPD,BR, &
L05449                                                         PSIM,PSIH
L05450
L05451       REAL,     DIMENSION( ims:ime, jms:jme )                    , &
L05452                 INTENT(OUT), OPTIONAL  ::      ck,cka,cd,cda,ustm
```

```
L05453
L05454  !-------------------------------------------------------------------
L05455  !    New for wrapper
L05456  !-------------------------------------------------------------------
L05457       LOGICAL,  INTENT(IN)               ::      TICE2TSK_IF2COLD
L05458       REAL,     INTENT(IN)               ::      XICE_THRESHOLD
L05459       REAL,     DIMENSION( ims:ime, jms:jme ),                     &
L05460                 INTENT(IN)               ::      XICE
L05461       REAL,     DIMENSION( ims:ime, jms:jme ),                     &
L05462                 INTENT(INOUT)            ::      SST
L05463       REAL,     DIMENSION( ims:ime, jms:jme ), OPTIONAL,           &
L05464                 INTENT(OUT)              ::      TSK_SEA,           &
L05465                                                  CHS2_SEA,          &
L05466                                                  CHS_SEA,           &
L05467                                                  CPM_SEA,           &
L05468                                                  CQS2_SEA,          &
L05469                                                  FLHC_SEA,          &
L05470                                                  FLQC_SEA,          &
L05471                                                  HFX_SEA,           &
L05472                                                  LH_SEA,            &
L05473                                                  QFX_SEA,           &
L05474                                                  QGH_SEA,           &
L05475                                                  QSFC_SEA,          &
L05476                                                  ZNT_SEA
L05477
L05478  !-------------------------------------------------------------------
L05479  !    Local
L05480  !-------------------------------------------------------------------
L05481       INTEGER :: I, J
L05482       REAL,     DIMENSION( ims:ime, jms:jme ) :: TSK_LOCAL,        &
L05483                   XLAND_SEA,                                       &
L05484                   MAVAIL_sea,                                      &
L05485                   BR_SEA,                        BR_HOLD,          &
L05486                                                  FLHC_HOLD,        &
L05487                                                  FLQC_HOLD,        &
L05488                   GZ1OZ0_SEA,                    GZ1OZ0_HOLD,      &
L05489                                                  HFX_HOLD,         &
L05490                                                  LH_HOLD,          &
L05491                   CH_SEA,                                          &
L05492                   MOL_SEA,                       MOL_HOLD,         &
L05493                   PSIH_SEA,                      PSIH_HOLD,        &
L05494                   PSIM_SEA,                      PSIM_HOLD,        &
L05495                                                  QFX_HOLD,         &
L05496                                                  QGH_HOLD,         &
L05497                                                  CPM_HOLD,         &
L05498                   RMOL_SEA,                      RMOL_HOLD,        &
L05499                   UST_SEA,                       UST_HOLD,         &
L05500                   WSPD_SEA,                      WSPD_HOLD,        &
L05501                                                  ZNT_HOLD,         &
L05502                   ZOL_SEA,                       ZOL_HOLD,         &
L05503                   Q2_SEA,                                          &
L05504                   T2_SEA,                                          &
L05505                   TH2_SEA,                                         &
L05506                   U10_SEA,                                         &
L05507                   V10_SEA,                                         &
L05508                   CD_SEA,                                          &
L05509                   CDA_SEA,                                         &
L05510                   CK_SEA,                                          &
L05511                   CKA_SEA,                                         &
L05512                   USTM_SEA,                      USTM_HOLD
L05513
L05514
L05515      CALL get_local_ice_tsk( ims, ime, jms, jme, its, ite, jts, jte,  &
L05516                              itimestep, .true., tice2tsk_if2cold,     &
L05517                              XICE, XICE_THRESHOLD,                    &
L05518                              SST, TSK, TSK_SEA, TSK_LOCAL )
L05519
L05520
L05521  ! DFS 8/25/10 Set TSK to ice value
L05522  !    DO j = JTS , JTE
L05523  !       DO i = ITS , ITE
L05524  !          TSK(i,j) = TSK_LOCAL(i,j)
L05525  !       ENDDO
L05526  !    ENDDO
L05527
L05528  ! Save the variables before the first call
L05529  ! (for land/frozen water) to SFCLAY_mynn.
L05530       BR_HOLD(its:ite,jts:jte)   = BR(its:ite,jts:jte)
L05531       CPM_HOLD(its:ite,jts:jte)  = CPM(its:ite,jts:jte)
L05532       FLHC_HOLD(its:ite,jts:jte) = FLHC(its:ite,jts:jte)
L05533       FLQC_HOLD(its:ite,jts:jte) = FLQC(its:ite,jts:jte)
L05534       GZ1OZ0_HOLD(its:ite,jts:jte) = GZ1OZ0(its:ite,jts:jte)
L05535       HFX_HOLD(its:ite,jts:jte)  = HFX(its:ite,jts:jte)
L05536       LH_HOLD(its:ite,jts:jte)   = LH(its:ite,jts:jte)
L05537       MOL_HOLD(its:ite,jts:jte)  = MOL(its:ite,jts:jte)
L05538       PSIH_HOLD(its:ite,jts:jte) = PSIH(its:ite,jts:jte)
L05539       PSIM_HOLD(its:ite,jts:jte) = PSIM(its:ite,jts:jte)
L05540       QFX_HOLD(its:ite,jts:jte)  = QFX(its:ite,jts:jte)
L05541       QGH_HOLD(its:ite,jts:jte)  = QGH(its:ite,jts:jte)
L05542       RMOL_HOLD(its:ite,jts:jte) = RMOL(its:ite,jts:jte)
L05543       UST_HOLD(its:ite,jts:jte)  = UST(its:ite,jts:jte)
L05544       USTM_HOLD(its:ite,jts:jte) = USTM(its:ite,jts:jte)
L05545       WSPD_HOLD(its:ite,jts:jte) = WSPD(its:ite,jts:jte)
L05546       ZNT_HOLD(its:ite,jts:jte)  = ZNT(its:ite,jts:jte)
L05547       ZOL_HOLD(its:ite,jts:jte)  = ZOL(its:ite,jts:jte)
L05548
L05549       ! We'll want to save the ouput
L05550       ! for weighting after the second call to SFCLAY.
L05551
L05552       ! land/frozen-water call
L05553            CALL mynnsfc_driver(u3d=u3d,v3d=v3d,t3d=t3d,         &
L05554                 qv3d=qv3d,p3d=p3d,dz8w=dz8w,th3d=th3d,          &
L05555                 rho3d=rho,psfcpa=psfc,                          &
L05556                 chs=chs,chs2=chs2,cqs=cqs,cqs2=cqs2,cpm=cpm,    &
L05557                 znt=znt,ust=ust,pblh=pblh,mavail=mavail,zol=zol,&
L05558                 mol=mol,psim=psim,psih=psih,qgh=qgh,            &
L05559                 xland=xland,hfx=hfx,qfx=qfx,lh=lh,tsk=tsk_local,&
L05560                 flhc=flhc,flqc=flqc,qsfc=qsfc,rmol=rmol,        &
L05561                 u10=u10,v10=v10,th2=th2,t2=t2,q2=q2,snowh=snowh,&
```

```
L05562                gz1oz0=gz1oz0,wspd=wspd,br=br,dx=dx,           &
L05563                itimestep=itimestep,ch=ch,                     &
L05564                spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl,&
L05565                ids=ids,ide=ide,jds=jds,jde=jde,kds=kds,kde=kde,&
L05566                ims=ims,ime=ime,jms=jms,jme=jme,kms=kms,kme=kme,&
L05567                its=its,ite=ite,jts=jts,jte=jte,kts=kts,kte=kte,&
L05568                ustm=ustm,ck=ck,cka=cka,cd=cd,cda=cda,         &
L05569                xice=xice,xice_threshold=xice_threshold,       &
L05570                sf_mynn_sfcflux_land=sf_mynn_sfcflux_land,     &
L05571                sf_mynn_sfcflux_water=sf_mynn_sfcflux_water,   &
L05572                isfflx=isfflx,shalwater_z0=shalwater_z0,       &
L05573                water_depth=water_depth,lakemask=lakemask,     &
L05574                flag_lsm=flag_lsm,restart=restart,             &
L05575                errmsg=errmsg,errflg=errflg                    )
L05576
L05577       ! Set up lower boundary conditions to force an open-water call
L05578        DO j = JTS , JTE
L05579           DO i = ITS , ITE
L05580              IF ( ( XICE(i,j) .GE. XICE_THRESHOLD ) .and. ( XICE(i,j) .LE. 1.0 ) ) THEN
L05581                 XLAND_SEA(i,j) = 2. !water
L05582                 MAVAIL_SEA(i,j)= 1.
L05583                 ZNT_SEA(i,j)   = 0.0001 !will be recalculated anyway
L05584                 TSK_SEA(i,j)   = SST(i,j)
L05585                 IF ( SST(i,j) .LT. 271.4 ) THEN
L05586                    SST(i,j)    = 271.4
L05587                    TSK_SEA(i,j)= SST(i,j)
L05588                 ENDIF
L05589                 QSFC_SEA(i,j)  = QSFC(i,j) !will be recalculated anyway
L05590              ELSE
L05591                 !keep original values
L05592                 XLAND_SEA(i,j) = XLAND(i,j)
L05593                 MAVAIL_SEA(i,j)= MAVAIL(i,j)
L05594                 ZNT_SEA(i,j)   = ZNT_HOLD(i,j)
L05595                 TSK_SEA(i,j)   = TSK_LOCAL(i,j)
L05596                 QSFC_SEA(i,j)  = QSFC(i,j)
L05597              ENDIF
L05598           ENDDO
L05599       ENDDO
L05600
L05601       ! Restore the values from before the land/frozen-water call
L05602       BR_SEA(its:ite,jts:jte)   = BR_HOLD(its:ite,jts:jte)
L05603       CPM_SEA(its:ite,jts:jte)  = CPM_HOLD(its:ite,jts:jte)
L05604       FLHC_SEA(its:ite,jts:jte) = FLHC_HOLD(its:ite,jts:jte)
L05605       FLQC_SEA(its:ite,jts:jte) = FLQC_HOLD(its:ite,jts:jte)
L05606       GZ1OZ0_SEA(its:ite,jts:jte) = GZ1OZ0_HOLD(its:ite,jts:jte)
L05607       HFX_SEA(its:ite,jts:jte)  = HFX_HOLD(its:ite,jts:jte)
L05608       LH_SEA(its:ite,jts:jte)   = LH_HOLD(its:ite,jts:jte)
L05609       MOL_SEA(its:ite,jts:jte)  = MOL_HOLD(its:ite,jts:jte)
L05610       PSIH_SEA(its:ite,jts:jte) = PSIH_HOLD(its:ite,jts:jte)
L05611       PSIM_SEA(its:ite,jts:jte) = PSIM_HOLD(its:ite,jts:jte)
L05612       QFX_SEA(its:ite,jts:jte)  = QFX_HOLD(its:ite,jts:jte)
L05613       QGH_SEA(its:ite,jts:jte)  = QGH_HOLD(its:ite,jts:jte)
L05614       RMOL_SEA(its:ite,jts:jte) = RMOL_HOLD(its:ite,jts:jte)
L05615       UST_SEA(its:ite,jts:jte)  = UST_HOLD(its:ite,jts:jte)
L05616       USTM_SEA(its:ite,jts:jte) = USTM_HOLD(its:ite,jts:jte)
L05617       WSPD_SEA(its:ite,jts:jte) = WSPD_HOLD(its:ite,jts:jte)
L05618       ZOL_SEA(its:ite,jts:jte)  = ZOL_HOLD(its:ite,jts:jte)
L05619
L05620
L05621       ! open-water call
L05622        CALL mynnsfc_driver(u3d=u3d,v3d=v3d,t3d=t3d,                                &
L05623             qv3d=qv3d,p3d=p3d,dz8w=dz8w,th3d=th3d,rho3d=rho,psfcpa=psfc,           &
L05624             chs=chs_sea,chs2=chs2_sea,cqs=cqs,cqs2=cqs2_sea,cpm=cpm_sea,           &
L05625             znt=znt_sea,ust=ust_sea,pblh=pblh,mavail=mavail_sea,zol=zol_sea,       &
L05626             mol=mol_sea,psim=psim_sea,psih=psih_sea,qgh=qgh_sea,                   &
L05627             xland=xland_sea,hfx=hfx_sea,qfx=qfx_sea,lh=lh_sea,tsk=tsk_sea,         &
L05628             flhc=flhc_sea,flqc=flqc_sea,qsfc=qsfc_sea,rmol=rmol_sea,               &
L05629             u10=u10_sea,v10=v10_sea,th2=th2_sea,t2=t2_sea,q2=q2_sea,snowh=snowh,&
L05630             gz1oz0=gz1oz0_sea,wspd=wspd_sea,br=br_sea,dx=dx,                       &
L05631             itimestep=itimestep,ch=ch,                                             &
L05632             spp_pbl=spp_pbl,pattern_spp_pbl=pattern_spp_pbl,                       &
L05633             ids=ids,ide=ide, jds=jds,jde=jde, kds=kds,kde=kde,                     &
L05634             ims=ims,ime=ime, jms=jms,jme=jme, kms=kms,kme=kme,                     &
L05635             its=its,ite=ite, jts=jts,jte=jte, kts=kts,kte=kte,                     &
L05636             ustm=ustm_sea,ck=ck_sea,cka=cka_sea,cd=cd_sea,cda=cda_sea,             &
L05637             xice=xice,xice_threshold=xice_threshold,                               &
L05638             sf_mynn_sfcflux_land=sf_mynn_sfcflux_land,                             &
L05639             sf_mynn_sfcflux_water=sf_mynn_sfcflux_water,isfflx=isfflx,             &
L05640             flag_lsm=flag_lsm,restart=restart,                                     &
L05641             errmsg=errmsg,errflg=errflg                                            )
L05642
L05643       DO j = JTS , JTE
L05644          DO i = ITS, ITE
L05645             IF ( ( XICE(I,J) .GE. XICE_THRESHOLD )  .and.( XICE(i,j) .LE. 1.0 ) ) THEN
L05646                ! weighted average for sea ice points
L05647                br(i,j)     = br(i,j)     * XICE(i,j) + (1.0-XICE(i,j)) * br_sea(i,j)
L05648                ! CHS2 -- wait
L05649                ! CHS  -- wait
L05650                ! CPM  -- wait
L05651                ! CQS2 -- wait
L05652                !FLHC(i,j)   = FLHC(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * FLHC_SEA (i,j)
L05653                !FLQC(i,j)   = FLQC(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * FLQC_SEA (i,j)
L05654                gz1oz0(i,j) = gz1oz0(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * gz1oz0_sea(i,j)
L05655                ! HFX  -- wait
L05656                ! LH   -- wait
L05657                mol(i,j)    = mol(i,j)    * XICE(i,j) + (1.0-XICE(i,j)) * mol_sea(i,j)
L05658                psih(i,j)   = psih(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * psih_sea(i,j)
L05659                psim(i,j)   = psim(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * psim_sea(i,j)
L05660                ! QFX  -- wait
L05661                ! QGH  -- wait
L05662                rmol(i,j)   = rmol(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * rmol_sea(i,j)
L05663                ust(i,j)    = ust(i,j)    * XICE(i,j) + (1.0-XICE(i,j)) * ust_sea(i,j)
L05664                wspd(i,j)   = wspd(i,j)   * XICE(i,j) + (1.0-XICE(i,j)) * wspd_sea(i,j)
L05665                zol(i,j)    = zol(i,j)    * XICE(i,j) + (1.0-XICE(i,j)) * zol_sea(i,j)
L05666                ch(i,j)     = ch(i,j)     * XICE(i,j) + (1.0-XICE(i,j)) * ch_sea(i,j)
L05667                ! INTENT(OUT)
L05668                ! ------------------------------------------------------------------
L05669                IF ( PRESENT ( CD ) ) THEN
```

```
L05670                   CD(i,j)  = CD(i,j)  * XICE(i,j) + (1.0-XICE(i,j)) * CD_sea(i,j)
L05671                ENDIF
L05672                IF ( PRESENT ( CDA ) ) THEN
L05673                   CDA(i,j) = CDA(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * CDA_sea(i,j)
L05674                ENDIF
L05675                IF ( PRESENT ( CK ) ) THEN
L05676                   CK(i,j)  = CK(i,j)  * XICE(i,j) + (1.0-XICE(i,j)) * CK_sea(i,j)
L05677                ENDIF
L05678                IF ( PRESENT ( CKA ) ) THEN
L05679                   CKA(i,j) = CKA(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * CKA_sea(i,j)
L05680                ENDIF
L05681                q2(i,j)     = q2(i,j)  * XICE(i,j) + (1.0-XICE(i,j)) * q2_sea(i,j)
L05682                ! QSFC -- wait
L05683                t2(i,j)     = t2(i,j)  * XICE(i,j) + (1.0-XICE(i,j)) * t2_sea(i,j)
L05684                th2(i,j)    = th2(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * th2_sea(i,j)
L05685                u10(i,j)    = u10(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * u10_sea(i,j)
L05686                IF ( PRESENT ( USTM ) ) THEN
L05687                   USTM(i,j)= USTM(i,j)* XICE(i,j) + (1.0-XICE(i,j)) * USTM_sea(i,j)
L05688                ENDIF
L05689                v10(i,j)    = v10(i,j) * XICE(i,j) + (1.0-XICE(i,j)) * v10_sea(i,j)
L05690             ENDIF
L05691          END DO
L05692        END DO
L05693
L05694
L05695     END SUBROUTINE mynn_seaice_wrapper
```

讲解：MYNNSFCSCHEME CASE 的前置件（主入口 L2431 调用）：对 xice≥xice_threshold 的点，把海冰点的水面状态冻结（TSK 置冰点附近、ZNT 用冰粗糙度、QSFC 按冰面饱和）后调 mynnsfc_driver——**海冰点不另走 ice 方案文件，而是在 wrapper 里先改下边界状态**。与 code_10 § 三的「mynnsfc_driver 三路分派」衔接：wrapper 改的是 driver 的输入而非分派逻辑。七个 *_seaice_wrapper（myj/qnse/gfs/sfclayrev/sfclay/px/mynn）是各面层方案对部分海冰的各自应对——改造海冰处理时七处都要排查。

八、改造入口

1. **新增面层方案**：sfclay_select 加 CASE + USE + Registry package（三步同 code_22 § 五）；海冰点需同步写 *_seaice_wrapper。
2. **新增陆面方案/耦合外源**：sfc_select 加 CASE；SST/海冰/湖泊通道见 § 三/§ 六。
3. **人为热**：distributed_ahe_opt=1（pbl driver）vs =2（本文件）的口径选择。
4. **诊断口径**：Q2 双重限幅（方案内 +driver 兜底）——改 MYNN 诊断时两处一致性要查。

九、上下游链接

- 上游：dyn_em/module_first_rk_step_part1.F（31 篇待交付，先于 pbl_driver 调用）。
- 下游：面层族 `code_10_module_sf_mynnsfc.md`（MYNN）；陆面族 Noah（12/13 篇待交付）；城市冠层经 pbl driver（code_14a/code_16）。
- 教材正文：ch16_land_surface_family.md、ch14_pbl_family.md、ch17_urban_canopy.md、ch19_lanzhou_heatwave.md、ch21_solve_em_dynamics.md。

code_30 — 大步长动力工具箱（dyn_em/module_big_step_utilities_em.F：状态方程、phy_prep 与 rhs_ph）

一、版本指纹

源文件	行数	md5（整文件，按字节）
dyn_em/module_big_step_utilities_em.F	6758	23a6e9f7a1d4b39a5243b36bee662e62

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：六件核心（calc_mu_uv/calc_alt/calc_p_rho_phi/calc_php/phy_prep 全文 + rhs_ph 头段）；43 单元的分组总表见 § 九。教材锚点：ch21_solve_em_dynamics.md（十四站）、ch10_vertical_coords_hydrostatics.md（垂直坐标与静力平衡的公式源头）。

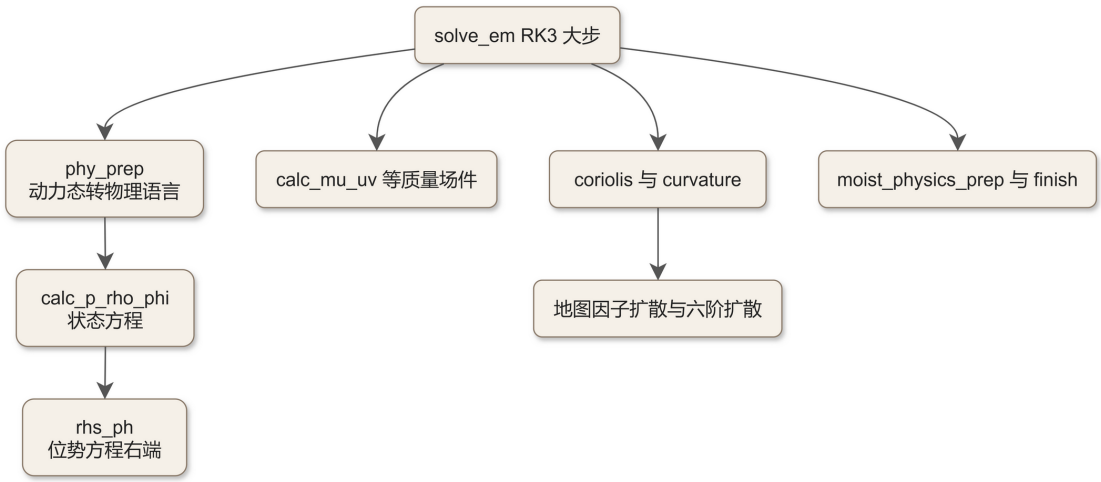
二、定位

一句话：**动力核的「大步长」工具箱**——RK3 大步里所有非平流的支持件：质量场换算（mu 系）、状态方程诊断（p/ρ/φ）、物理语言翻译（phy_prep）、位势方程右端（rhs_ph）、Coriolis/曲率项、地图因子扩散、六阶扩散、Rayleigh 阻尼、水物质的 RK 内准备等。

谁在调它：solve_em 的各站（part1 的 phy_prep、advance_w/advance_mu_t 等的 rhs_ph/coriolis/couple 系，见 `code_50_solve_em.md`）；part2 亦用 conv_t_tendf_to_moist（code_32 § 十）。仅 5 项 USE（L0 底座级依赖，plan § 三）。

43 单元分组总表：

组	单元	行段	说明
质量场	calc_mu_uv / calc_mu_uv_1 / calc_mu_staggered / couple / couple_momentum / decouple	L26–L949, L4475–L4568	mu 交错换算与 mu 耦合/解耦
状态诊断	calc_ww_cp / calc_cq / calc_alt / calc_p_rho_phi / calc_php / diagnose_w	L640–L1361	ω/cq/比重容/状态方程/全位势/w 诊断
动力右端	rhs_ph / horizontal_pressure_gradient / pg_buoy_w / w_damp	L1365–L2711	位势方程右端/水平气压梯度/浮力/w 阻尼
扩散	horizontal_diffusion 系 4 件 / vertical_diffusion 系 5 件 / sixth_order_diffusion	L2715–L3581, L6220–L6636	地图因子扩散族（与 module_diffusion_em 分工）
旋转项	coriolis / perturbation_coriolis / curvature	L3640–L4469	科氏力/扰动科氏/曲率项
工具	calculate_full / zero_tend 系 / pole_point_bc / zero_pole	L3587–L4724	极点与清零杂件
物理准备	phy_prep / phy_prep_part2 / moist_physics_prep_em / moist_physics_finish_em / initialize_moist_old / conv_t_tendf_to_moist / Cloud_tracer_nudge	L4730–L6756	动力 ↔ 物理翻译与水物质 RK 内管理
框架	init_module_big_step / set_tend / rk_rayleigh_damp / theta_relaxation	L5789–L6216	倾向设零/Rayleigh 阻尼/θ 松弛



三、质量场：calc_mu_uv

```
!file: dyn_em/module_big_step_utilities_em.F    L26-L180    (subroutine calc_mu_uv)
L00026  SUBROUTINE calc_mu_uv ( config_flags,                    &
L00027                          mu, mub, muu, muv,               &
L00028                          ids, ide, jds, jde, kds, kde, &
L00029                          ims, ime, jms, jme, kms, kme, &
L00030                          its, ite, jts, jte, kts, kte )
L00031
L00032     IMPLICIT NONE
L00033
L00034     ! Input data
L00035
L00036     TYPE(grid_config_rec_type   ) ,   INTENT(IN   ) :: config_flags
L00037
L00038     INTEGER ,          INTENT(IN   ) :: ids, ide, jds, jde, kds, kde, &
L00039                                         ims, ime, jms, jme, kms, kme, &
L00040                                         its, ite, jts, jte, kts, kte
L00041
L00042     REAL, DIMENSION( ims:ime , jms:jme ) , INTENT(  OUT) :: muu, muv
L00043     REAL, DIMENSION( ims:ime , jms:jme ) , INTENT(IN   ) :: mu, mub
L00044
L00045     !  local stuff
L00046
L00047     INTEGER :: i, j, itf, jtf, im, jm
L00048
L00049  !<DESCRIPTION>
L00050  !
L00051  !  calc_mu_uv calculates the full column dry-air mass at the staggered
L00052  !  horizontal velocity points (u,v) and places the results in muu and muv.
L00053  !  This routine uses the reference state (mub) and perturbation state (mu)
L00054  !
L00055  !</DESCRIPTION>
L00056
L00057
L00058        itf=ite
L00059        jtf=MIN(jte,jde-1)
L00060
L00061        IF      ( ( its .NE. ids ) .AND. ( ite .NE. ide ) ) THEN
L00062           DO j=jts,jtf
L00063           DO i=its,itf
L00064              MUU(i,j) = 0.5*(MU(i,j)+MU(i-1,j)+MUB(i,j)+MUB(i-1,j))
L00065           ENDDO
L00066           ENDDO
L00067        ELSE IF ( ( its .EQ. ids ) .AND. ( ite .NE. ide ) ) THEN
L00068           DO j=jts,jtf
L00069           DO i=its+1,itf
L00070              MUU(i,j) = 0.5*(MU(i,j)+MU(i-1,j)+MUB(i,j)+MUB(i-1,j))
L00071           ENDDO
L00072           ENDDO
L00073           i=its
L00074           im = its
L00075           if(config_flags%periodic_x) im = its-1
L00076           DO j=jts,jtf
L00077  !           MUU(i,j) =      MU(i,j)          +MUB(i,j)
L00078  !  fix for periodic b.c., 13 march 2004, wcs
L00079              MUU(i,j) = 0.5*(MU(i,j)+MU(im,j)+MUB(i,j)+MUB(im,j))
L00080           ENDDO
L00081        ELSE IF ( ( its .NE. ids ) .AND. ( ite .EQ. ide ) ) THEN
L00082           DO j=jts,jtf
L00083           DO i=its,itf-1
L00084              MUU(i,j) = 0.5*(MU(i,j)+MU(i-1,j)+MUB(i,j)+MUB(i-1,j))
L00085           ENDDO
L00086           ENDDO
L00087           i=ite
L00088           im = ite-1
L00089           if(config_flags%periodic_x) im = ite
L00090           DO j=jts,jtf
L00091  !           MUU(i,j) =      MU(i-1,j)        +MUB(i-1,j)
L00092  !  fix for periodic b.c., 13 march 2004, wcs
L00093              MUU(i,j) = 0.5*(MU(i-1,j)+MU(im,j)+MUB(i-1,j)+MUB(im,j))
L00094           ENDDO
L00095        ELSE IF ( ( its .EQ. ids ) .AND. ( ite .EQ. ide ) ) THEN
L00096           DO j=jts,jtf
L00097           DO i=its+1,itf-1
```

```
L00098             MUU(i,j) = 0.5*(MU(i,j)+MU(i-1,j)+MUB(i,j)+MUB(i-1,j))
L00099           ENDDO
L00100           ENDDO
L00101           i=its
L00102           im = its
L00103           if(config_flags%periodic_x) im = its-1
L00104           DO j=jts,jtf
L00105  !             MUU(i,j) =      MU(i,j)          +MUB(i,j)
L00106  !  fix for periodic b.c., 13 march 2004, wcs
L00107             MUU(i,j) = 0.5*(MU(i,j)+MU(im,j)+MUB(i,j)+MUB(im,j))
L00108           ENDDO
L00109           i=ite
L00110           im = ite-1
L00111           if(config_flags%periodic_x) im = ite
L00112           DO j=jts,jtf
L00113  !             MUU(i,j) =      MU(i-1,j)        +MUB(i-1,j)
L00114  !  fix for periodic b.c., 13 march 2004, wcs
L00115             MUU(i,j) = 0.5*(MU(i-1,j)+MU(im,j)+MUB(i-1,j)+MUB(im,j))
L00116           ENDDO
L00117        END IF
L00118
L00119        itf=MIN(ite,ide-1)
L00120        jtf=jte
L00121
L00122        IF      ( ( jts .NE. jds ) .AND. ( jte .NE. jde ) ) THEN
L00123           DO j=jts,jtf
L00124           DO i=its,itf
L00125              MUV(i,j) = 0.5*(MU(i,j)+MU(i,j-1)+MUB(i,j)+MUB(i,j-1))
L00126           ENDDO
L00127           ENDDO
L00128        ELSE IF ( ( jts .EQ. jds ) .AND. ( jte .NE. jde ) ) THEN
L00129           DO j=jts+1,jtf
L00130           DO i=its,itf
L00131              MUV(i,j) = 0.5*(MU(i,j)+MU(i,j-1)+MUB(i,j)+MUB(i,j-1))
L00132           ENDDO
L00133           ENDDO
L00134           j=jts
L00135           jm = jts
L00136           if(config_flags%periodic_y) jm = jts-1
L00137           DO i=its,itf
L00138  !             MUV(i,j) =      MU(i,j)          +MUB(i,j)
L00139  !  fix for periodic b.c., 13 march 2004, wcs
L00140             MUV(i,j) = 0.5*(MU(i,j)+MU(i,jm)+MUB(i,j)+MUB(i,jm))
L00141           ENDDO
L00142        ELSE IF ( ( jts .NE. jds ) .AND. ( jte .EQ. jde ) ) THEN
L00143           DO j=jts,jtf-1
L00144           DO i=its,itf
L00145              MUV(i,j) = 0.5*(MU(i,j)+MU(i,j-1)+MUB(i,j)+MUB(i,j-1))
L00146           ENDDO
L00147           ENDDO
L00148           j=jte
L00149           jm = jte-1
L00150           if(config_flags%periodic_y) jm = jte
L00151           DO i=its,itf
L00152  !             MUV(i,j) =      MU(i,j-1)        +MUB(i,j-1)
L00153  !  fix for periodic b.c., 13 march 2004, wcs
L00154             MUV(i,j) = 0.5*(MU(i,j-1)+MU(i,jm)+MUB(i,j-1)+MUB(i,jm))
L00155           ENDDO
L00156        ELSE IF ( ( jts .EQ. jds ) .AND. ( jte .EQ. jde ) ) THEN
L00157           DO j=jts+1,jtf-1
L00158           DO i=its,itf
L00159              MUV(i,j) = 0.5*(MU(i,j)+MU(i,j-1)+MUB(i,j)+MUB(i,j-1))
L00160           ENDDO
L00161           ENDDO
L00162           j=jts
L00163           jm = jts
L00164           if(config_flags%periodic_y) jm = jts-1
L00165           DO i=its,itf
L00166  !             MUV(i,j) =      MU(i,j)          +MUB(i,j)
L00167  !  fix for periodic b.c., 13 march 2004, wcs
L00168             MUV(i,j) = 0.5*(MU(i,j)+MU(i,jm)+MUB(i,j)+MUB(i,jm))
L00169           ENDDO
L00170           j=jte
L00171           jm = jte-1
L00172           if(config_flags%periodic_y) jm = jte
L00173           DO i=its,itf
L00174  !             MUV(i,j) =      MU(i,j-1)        +MUB(i,j-1)
L00175  !  fix for periodic b.c., 13 march 2004, wcs
L00176             MUV(i,j) = 0.5*(MU(i,j-1)+MU(i,jm)+MUB(i,j-1)+MUB(i,jm))
L00177           ENDDO
L00178        END IF
L00179
L00180  END SUBROUTINE calc_mu_uv
```

讲解：交错 u/v 点上的干空气柱质量：MUU(i,j)=0.5·(MU(i,j)+MU(i−1,j)+MUB(i,j)+MUB(i−1,j))——扰动 mu 与基态 mub 同式平均（**全量 muu=μ′+μ̄ 在交错点线性内插**）；四角 if 分类处理 patch 边界，**周期边界的越界点用 im=its−1/ite 取环回值**（wcs 2004-03-13 的 fix——旧式直接用单点会破坏周期守恒）；v 侧同构。calc_mu_uv_1/calc_mu_staggered 是单点与整层变体。

四、状态诊断：calc_alt、calc_p_rho_phi、calc_php

```
!file: dyn_em/module_big_step_utilities_em.F    L910-L949    (subroutine calc_alt)
L00910  SUBROUTINE calc_alt ( alt, al, alb,                    &
L00911                        ids, ide, jds, jde, kds, kde,    &
L00912                        ims, ime, jms, jme, kms, kme,    &
L00913                        its, ite, jts, jte, kts, kte  )
L00914
L00915     IMPLICIT NONE
L00916
L00917     ! Input data
```

```
L00918
L00919     INTEGER ,          INTENT(IN   ) :: ids, ide, jds, jde, kds, kde, &
L00920                                         ims, ime, jms, jme, kms, kme, &
L00921                                         its, ite, jts, jte, kts, kte
L00922
L00923     REAL, DIMENSION( ims:ime , kms:kme , jms:jme ), INTENT(IN   ) :: alb, al
L00924     REAL, DIMENSION( ims:ime , kms:kme , jms:jme ), INTENT(  OUT) :: alt
L00925
L00926     ! Local stuff
L00927
L00928     INTEGER :: i, j, k, itf, jtf, ktf
L00929
L00930  !<DESCRIPTION>
L00931  !
L00932  ! calc_alt computes the full inverse density
L00933  !
L00934  !</DESCRIPTION>
L00935
L00936        itf=MIN(ite,ide-1)
L00937        jtf=MIN(jte,jde-1)
L00938        ktf=MIN(kte,kde-1)
L00939
L00940        DO j=jts,jtf
L00941        DO k=kts,ktf
L00942        DO i=its,itf
L00943          alt(i,k,j) = al(i,k,j)+alb(i,k,j)
L00944        ENDDO
L00945        ENDDO
L00946        ENDDO
L00947
L00948
L00949  END SUBROUTINE calc_alt
```

讲解：全比重容 alt=al+alb（扰动 +基态）——一行件，但它是物理方案密度（ρ=1/alt • (1+qv)，见 phy_prep）的源头。

```
!file: dyn_em/module_big_step_utilities_em.F    L953-L1223    (subroutine calc_p_rho_phi)
L00953  SUBROUTINE calc_p_rho_phi ( moist, n_moist, hypsometric_opt,      &
L00954                              al, alb, mu, muts,                    &
L00955                              c1, c2, c3h, c4h, c3f, c4f,           &
L00956                              ph, phb, p, pb,                       &
L00957                              t, p0, t0, ptop, znu, znw, dnw, rdnw, &
L00958                              rdn, non_hydrostatic, use_theta_m,    &
L00959                              ids, ide, jds, jde, kds, kde,         &
L00960                              ims, ime, jms, jme, kms, kme,         &
L00961                              its, ite, jts, jte, kts, kte          )
L00962
L00963    IMPLICIT NONE
L00964
L00965     ! Input data
L00966
L00967    LOGICAL ,          INTENT(IN   ) :: non_hydrostatic
L00968    INTEGER ,          INTENT(IN   ) :: use_theta_m
L00969
L00970    INTEGER ,          INTENT(IN   ) :: ids, ide, jds, jde, kds, kde, &
L00971                                        ims, ime, jms, jme, kms, kme, &
L00972                                        its, ite, jts, jte, kts, kte
L00973
L00974    INTEGER ,          INTENT(IN   ) :: n_moist
L00975    INTEGER ,          INTENT(IN   ) :: hypsometric_opt
L00976
L00977    REAL, DIMENSION( ims:ime , kms:kme , jms:jme ), INTENT(IN   ) :: alb,  &
L00978                                                                     pb,   &
L00979                                                                     t
L00980
L00981    REAL, DIMENSION( ims:ime , kms:kme , jms:jme, n_moist ), INTENT(IN   ) :: moist
L00982
L00983    REAL, DIMENSION( ims:ime , kms:kme , jms:jme ), INTENT(  OUT) :: al, p
L00984
L00985    REAL, DIMENSION( ims:ime , kms:kme , jms:jme ), INTENT(INOUT) :: ph, phb
L00986
L00987    REAL, DIMENSION( ims:ime , jms:jme ), INTENT(IN   ) :: mu, muts
L00988
L00989    REAL, DIMENSION( kms:kme ), INTENT(IN   ) :: znu, znw, dnw, rdnw, rdn
L00990
L00991    REAL, DIMENSION( kms:kme ), INTENT(IN   ) :: c1, c2, c3h, c4h, c3f, c4f
L00992
L00993    REAL,   INTENT(IN   ) :: t0, p0, ptop
L00994
L00995    ! Local stuff
L00996
L00997    INTEGER :: i, j, k, kk, itf, jtf, ktf, ispe
L00998    REAL    :: qtot, qf1, qf2, qvf
L00999    REAL, DIMENSION( its:ite) :: temp,cpovcv_v
L01000    REAL    :: pfu, phm, pfd
L01001
L01002  !<DESCRIPTION>
L01003  !
L01004  ! For the nonhydrostatic option, calc_p_rho_phi calculates the
L01005  ! diagnostic quantities pressure and (inverse) density from the
L01006  ! prognostic variables using the equation of state.
L01007  !
L01008  ! For the hydrostatic option, calc_p_rho_phi calculates the
L01009  ! diagnostic quantities (inverse) density and geopotential from the
L01010  ! prognostic variables using the equation of state and the hydrostatic
L01011  ! equation.
L01012  !
L01013  !</DESCRIPTION>
L01014
L01015    itf=MIN(ite,ide-1)
L01016    jtf=MIN(jte,jde-1)
L01017    ktf=MIN(kte,kde-1)
L01018
L01019  #ifndef INTELMKL
L01020    cpovcv_v = cpovcv
L01021  #endif
L01022
```

```
L01023    nonhydro : IF (non_hydrostatic) THEN
L01024
L01025        hypso_nonhydro : IF (hypsometric_opt == 1) THEN
L01026          DO j=jts,jtf
L01027          DO k=kts,ktf
L01028          DO i=its,itf
L01029            al(i,k,j)=-1./(c1(k)*muts(i,j)+c2(k))*(alb(i,k,j)*(c1(k)*mu(i,j)) + rdnw(k)*(ph(i,k+1,j)-ph(i,k,j)))
L01030          END DO
L01031          END DO
L01032          END DO
L01033        ELSE IF (hypsometric_opt == 2) THEN hypso_nonhydro
L01034
L01035          ! The relation used to get specific volume, al, is: al = -dZ/dp,
L01036          ! where dp = mut * d(eta). The pressure depth, dp, is replaced with
L01037          ! p*(dp/p) ~ p*LOG((p+0.5dp)/(p-0.5dp)). Difference between dp and p*dLOG(p)
L01038          ! is as follows: p*dLOG(p) - dp = 1/12*(dp/p)**3 + 1/90*(dp/p)**5 + ...
L01039          ! Therefore, p*dLOG(p) is always larger than dp and the difference is
L01040          ! in proportion to dp/p. TKW, 02/16/2010
L01041
L01042          DO j=jts,jtf
L01043          DO k=kts,ktf
L01044          DO i=its,itf
L01045            pfu = c3f(k+1)*MUTS(i,j) + c4f(k+1) + ptop
L01046            pfd = c3f(k  )*MUTS(i,j) + c4f(k  ) + ptop
L01047            phm = c3h(k  )*MUTS(i,j) + c4h(k  ) + ptop
L01048            al(i,k,j) = (ph(i,k+1,j)-ph(i,k,j)+phb(i,k+1,j)-phb(i,k,j))/phm/LOG(pfd/pfu)-alb(i,k,j)
L01049          END DO
L01050          END DO
L01051          END DO
L01052        ELSE hypso_nonhydro
L01053          CALL wrf_error_fatal ( 'calc_p_rho_phi: hypsometric_opt should be 1 or 2' )
L01054        END IF hypso_nonhydro
L01055
L01056        moist_nonhydro : IF (n_moist >= PARAM_FIRST_SCALAR ) THEN
L01057
L01058          DO j=jts,jtf
L01059          DO k=kts,ktf
L01060          DO i=its,itf
L01061          IF ( use_theta_m .EQ. 1 ) THEN
L01062            temp(i)=(r_d*(t0+t(i,k,j)))/(p0*(al(i,k,j)+alb(i,k,j)))
L01063          ELSE
L01064            qvf = 1.+rvovrd*moist(i,k,j,P_QV)
L01065            temp(i)=(r_d*(t0+t(i,k,j))*qvf)/(p0*(al(i,k,j)+alb(i,k,j)))
L01066          ENDIF
L01067          ENDDO
L01068  #ifdef INTELMKL
L01069          CALL VPOWX ( itf-its+1, temp(its), cpovcv, p(its,k,j) )
L01070  #else
L01071  ! use vector version from libmassv or from compat lib in frame/libmassv.F
L01072          CALL VPOW  ( p(its,k,j), temp(its), cpovcv_v(its), itf-its+1 )
L01073  #endif
L01074          DO i=its,itf
L01075             p(i,k,j)= p(i,k,j)*p0-pb(i,k,j)
L01076          ENDDO
L01077          ENDDO
L01078          ENDDO
L01079
L01080        ELSE moist_nonhydro
L01081
L01082          DO j=jts,jtf
L01083          DO k=kts,ktf
L01084          DO i=its,itf
L01085            p(i,k,j)=p0*( (r_d*(t0+t(i,k,j)))/                    &
L01086                          (p0*(al(i,k,j)+alb(i,k,j))) )**cpovcv  &
L01087                             -pb(i,k,j)
L01088          ENDDO
L01089          ENDDO
L01090          ENDDO
L01091
L01092        END IF moist_nonhydro
L01093
L01094     ELSE nonhydro
L01095
L01096  !  hydrostatic pressure, al, and ph1 calc; WCS, 5 sept 2001
L01097
L01098
L01099        moist_hydro : IF (n_moist >= PARAM_FIRST_SCALAR ) THEN
L01100
L01101          DO j=jts,jtf
L01102
L01103            k=ktf          ! top layer
L01104            DO i=its,itf
L01105
L01106              qtot = 0.
L01107              DO ispe=PARAM_FIRST_SCALAR,n_moist
L01108                qtot = qtot + moist(i,k,j,ispe)
L01109              ENDDO
L01110              qf2 = 1.
L01111              qf1 = qtot*qf2
L01112
L01113              p(i,k,j) = - 0.5*((c1(k)*mu(i,j))+qf1*(c1(k)*muts(i,j)+c2(k)))/rdnw(k)/qf2
L01114              IF ( use_theta_m .EQ. 1 ) THEN
L01115              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)* &
L01116                  (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01117              ELSE
L01118              qvf = 1.+rvovrd*moist(i,k,j,P_QV)
L01119              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)*qvf* &
L01120                  (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01121              END IF
L01122
L01123            ENDDO
L01124
L01125            DO k=ktf-1,kts,-1  ! remaining layers, integrate down
L01126              DO i=its,itf
L01127
L01128              qtot = 0.
L01129              DO ispe=PARAM_FIRST_SCALAR,n_moist
L01130                qtot = qtot + 0.5*(  moist(i,k  ,j,ispe) + moist(i,k+1,j,ispe) )
L01131              ENDDO
L01132              qf2 = 1.
```

```
L01133              qf1 = qtot*qf2
L01134
L01135              p(i,k,j) = p(i,k+1,j) - ((c1(k)*mu(i,j)) + qf1*(c1(k)*muts(i,j)+c2(k)))/qf2/rdn(k+1)
L01136              IF ( use_theta_m .EQ. 1 ) THEN
L01137              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)* &
L01138                          (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01139              ELSE
L01140              qvf = 1.+rvovrd*moist(i,k,j,P_QV)
L01141              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)*qvf* &
L01142                          (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01143              END IF
L01144              ENDDO
L01145            ENDDO
L01146
L01147          ENDDO
L01148
L01149        ELSE moist_hydro
L01150
L01151          DO j=jts,jtf
L01152
L01153            k=ktf          ! top layer
L01154            DO i=its,itf
L01155
L01156              qtot = 0.
L01157              qf2 = 1.
L01158              qf1 = qtot*qf2
L01159
L01160              p(i,k,j) = - 0.5*((c1(k)*mu(i,j))+qf1*(c1(k)*muts(i,j)+c2(k)))/rdnw(k)/qf2
L01161              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)* &
L01162                  (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01163
L01164            ENDDO
L01165
L01166            DO k=ktf-1,kts,-1  ! remaining layers, integrate down
L01167              DO i=its,itf
L01168
L01169              qtot = 0.
L01170              qf2 = 1.
L01171              qf1 = qtot*qf2
L01172
L01173              p(i,k,j) = p(i,k+1,j) - ((c1(k)*mu(i,j)) + qf1*(c1(k)*muts(i,j)+c2(k)))/qf2/rdn(k+1)
L01174              al(i,k,j) = (r_d/p1000mb)*(t(i,k,j)+t0)* &
L01175                          (((p(i,k,j)+pb(i,k,j))/p1000mb)**cvpm) - alb(i,k,j)
L01176              ENDDO
L01177            ENDDO
L01178
L01179          ENDDO
L01180
L01181        END IF moist_hydro
L01182
L01183        hypso_hydro : IF (hypsometric_opt == 1) THEN
L01184           DO j=jts,jtf
L01185             DO kk=2,ktf+1  ! integrate hydrostatic equation for geopotential
L01186               k = kk-1
L01187               DO i=its,itf
L01188                 ph(i,k+1,j) = ph(i,k,j) - (dnw(k))*(           &
L01189                             ((c1(k)*muts(i,j)+c2(k)))*al(i,k,j)+           &
L01190                              (c1(k)*mu(i,j))*alb(i,k,j)  )
L01191               ENDDO
L01192             ENDDO
L01193           ENDDO
L01194        ELSE hypso_hydro
L01195
L01196        ! Revised hypsometric eq.: dZ=-al*p*dLOG(p), where p is dry pressure
L01197
L01198         DO j=jts,jtf
L01199           DO i=its,itf
L01200              ph(i,kts,j) = phb(i,kts,j)
L01201           END DO
L01202
L01203           DO k=kts+1,ktf+1
L01204             DO i=its,itf
L01205               pfu = c3f(k  )*MUTS(i,j) + c4f(k  ) + ptop
L01206               pfd = c3f(k-1)*MUTS(i,j) + c4f(k-1) + ptop
L01207               phm = c3h(k-1)*MUTS(i,j) + c4h(k-1) + ptop
L01208               ph(i,k,j) = ph(i,k-1,j) + (al(i,k-1,j)+alb(i,k-1,j))*phm*LOG(pfd/pfu)
L01209             ENDDO
L01210           ENDDO
L01211
L01212           DO k=kts,ktf+1
L01213             DO i=its,itf
L01214                ph(i,k,j) = ph(i,k,j) - phb(i,k,j)
L01215             END DO
L01216           END DO
L01217         END DO
L01218
L01219        END IF hypso_hydro
L01220
L01221      END IF nonhydro
L01222
L01223    END SUBROUTINE calc_p_rho_phi
```

讲解：**状态方程与静力平衡的核心件**，四路分支：① **非静力 hypsometric_opt=1**（默认）：al=−[alb • c1μ+rdnw • Δph]/(c1μ+c2)——由位势厚度反解比重容（c1/c2 是高度坐标系数、rdnw=1/dnw 层厚倒数）；②**opt=2（TKW 2010-02-16）**：al=Δφ/(φ_m • ln(pfd/pfu))−alb——注释给了误差展开 p • dlnp−dp=(1/12)(dp/p)³+…，**用对数气压厚度代替线性厚度的高阶修订**；③ 气压：先 temp=r_d(t0+t) • qvf/(p0 • al_full)（qvf=1+rvovrd • qv 的虚温订正；use_theta_m=1 时 t 已含水汽、不加 qvf），**p_full=p0 · temp^cpovcv**——用 libmassv 的向量幂函数 VPOW（INTELMKL 时 VPOWX），再减基态 pb 得扰动气压 p——注意 **cvpm=−cv/cp 出现在静力分支**而这里用正的 cpovcv，两条状态方程口径；④ **静力模式**：从顶层起算 p(i,ktf)=−0.5(c1μ+qf1(c1μ+c2))/rdnw（顶边界条件），逐层向下 p(k)=p(k+1)−(c1μ+qf1(c1μ+c2))/rdn(k+1)——**qf1=Σ 水物质**：气压由干空气柱 μ 与水物质重量共同

承担：al 用 (r_d/p1000mb)(t+t0)qvf • (p_full/p1000mb)^cvpm−alb；静力位势 ph 向下积分 ph(k+1)= ph(k)−dnw[(c1μ+c2) • al+c1μ • alb]，修订版走 dZ=−al • p • dlnp（TKW 同款高阶口径）。**这一件是教材 ch10 全部公式的源码本体。**

```
!file: dyn_em/module_big_step_utilities_em.F    L1227-L1266    (subroutine calc_php)
L01227  SUBROUTINE calc_php ( php, ph, phb,                    &
L01228                        ids, ide, jds, jde, kds, kde,  &
L01229                        ims, ime, jms, jme, kms, kme,  &
L01230                        its, ite, jts, jte, kts, kte  )
L01231
L01232     IMPLICIT NONE
L01233
L01234     ! Input data
L01235
L01236     INTEGER ,          INTENT(IN   ) :: ids, ide, jds, jde, kds, kde, &
L01237                                         ims, ime, jms, jme, kms, kme, &
L01238                                         its, ite, jts, jte, kts, kte
L01239
L01240     REAL, DIMENSION( ims:ime, kms:kme , jms:jme ), INTENT(IN   ) :: phb, ph
L01241     REAL, DIMENSION( ims:ime, kms:kme , jms:jme ), INTENT(  OUT) :: php
L01242
L01243     ! Local stuff
L01244
L01245     INTEGER :: i, j, k, itf, jtf, ktf
L01246
L01247  !<DESCRIPTION>
L01248  !
L01249  !  calc_php calculates the full geopotential from the reference state
L01250  !  geopotential and the perturbation geopotential (phb_ph).
L01251  !
L01252  !</DESCRIPTION>
L01253
L01254        itf=MIN(ite,ide-1)
L01255        jtf=MIN(jte,jde-1)
L01256        ktf=MIN(kte,kde-1)
L01257
L01258        DO j=jts,jtf
L01259        DO k=kts,ktf
L01260        DO i=its,itf
L01261          php(i,k,j) = 0.5*(phb(i,k,j)+phb(i,k+1,j)+ph(i,k,j)+ph(i,k+1,j))
L01262        ENDDO
L01263        ENDDO
L01264        ENDDO
L01265
L01266  END SUBROUTINE calc_php
```

讲解：全位惠（half-level 上的 full geopotential）php=0.5(phb+phb 上层 +ph+ph 上层)——w 层与质量层之间的插值件，w 方程的气压梯度项用它。

五、物理翻译官 phy_prep（全文）

```
!file: dyn_em/module_big_step_utilities_em.F    L4730-L4973    (subroutine phy_prep)
L04730     SUBROUTINE phy_prep ( config_flags,                                 &  ! input
L04731                           mut, muu, muv,                                 &
L04732                           c1h, c2h, c1f, c2f,                            &
L04733                           u, v, p, pb, alt, ph,                          &  ! input
L04734                           phb, t, moist, n_moist,                        &  ! input
L04735                           rho, th_phy, th_phy_m_t0,                      &  ! output
L04736                           p_phy , pi_phy ,                               &  ! output
L04737                           u_phy, v_phy, p8w, t_phy, t8w,                 &  ! output
L04738                           z, z_at_w, dz8w,                               &  ! output
L04739                           p_hyd, p_hyd_w, dnw,                           &  ! output
L04740                           fzm, fzp, znw, p_top,                          &  ! params
L04741                           ids, ide, jds, jde, kds, kde,                  &
L04742                           ims, ime, jms, jme, kms, kme,                  &
L04743                           its, ite, jts, jte, kts, kte                   )
L04744  !-------------------------------------------------------------------------
L04745     IMPLICIT NONE
L04746  !-------------------------------------------------------------------------
L04747
L04748     TYPE(grid_config_rec_type) ,     INTENT(IN   ) :: config_flags
L04749
L04750     INTEGER ,        INTENT(IN   ) ::   ids, ide, jds, jde, kds, kde, &
L04751                                         ims, ime, jms, jme, kms, kme, &
L04752                                         its, ite, jts, jte, kts, kte
L04753     INTEGER ,          INTENT(IN   ) :: n_moist
L04754
L04755     REAL, DIMENSION( ims:ime, kms:kme , jms:jme , n_moist ), INTENT(IN) :: moist
L04756
L04757
L04758     REAL , DIMENSION( ims:ime, jms:jme ), INTENT(IN   )   ::     mut, muu, muv
L04759
L04760     REAL , DIMENSION( ims:ime , kms:kme , jms:jme ) ,                   &
L04761            INTENT(  OUT)                                    ::   u_phy, &
L04762                                                                     v_phy, &
L04763                                                                  pi_phy, &
L04764                                                                   p_phy, &
L04765                                                                    p8w, &
L04766                                                                  t_phy, &
L04767                                                                 th_phy, &
L04768                                                           th_phy_m_t0, &
L04769                                                                    t8w, &
L04770                                                                    rho, &
L04771                                                                      z, &
L04772                                                                   dz8w, &
L04773                                                                  z_at_w
L04774
L04775     REAL , DIMENSION( ims:ime , kms:kme , jms:jme ) ,                   &
L04776            INTENT(  OUT)                                    ::   p_hyd, &
L04777                                                                   p_hyd_w
```

```
L04778
L04779     REAL , INTENT(IN   )                                    ::   p_top
L04780
L04781     REAL , DIMENSION( ims:ime , kms:kme , jms:jme ) ,                    &
L04782            INTENT(IN   )                                    ::      pb, &
L04783                                                                      p, &
L04784                                                                      u, &
L04785                                                                      v, &
L04786                                                                    alt, &
L04787                                                                     ph, &
L04788                                                                    phb, &
L04789                                                                      t
L04790
L04791
L04792     REAL , DIMENSION( kms:kme ) ,           INTENT(IN   ) ::     fzm,   &
L04793                                                                    fzp
L04794
L04795     REAL , DIMENSION( kms:kme ) ,           INTENT(IN   ) ::     znw, &
L04796                                                                    dnw
L04797
L04798     REAL, DIMENSION( kms:kme ) ,            INTENT(IN   ) ::     c1h, c2h, c1f, c2f
L04799
L04800     REAL, DIMENSION( kms:kme ) :: c1, c2
L04801     INTEGER :: i_start, i_end, j_start, j_end, k_start, k_end, i_startu, j_startv
L04802     INTEGER :: i, j, k
L04803     REAL    :: w1, w2, z0, z1, z2
L04804     REAL    :: qtot
L04805     INTEGER :: n
L04806
L04807  !-----------------------------------------------------------------------
L04808
L04809  !<DESCRIPTION>
L04810  !
L04811  !  phys_prep calculates a number of diagnostic quantities needed by
L04812  !  the physics routines.
L04813  !
L04814  !</DESCRIPTION>
L04815
L04816      c1 = c1h
L04817      c2 = c2h
L04818  !  set up loop bounds for this grid's boundary conditions
L04819
L04820      i_start = its
L04821      i_end   = min( ite,ide-1 )
L04822      j_start = jts
L04823      j_end   = min( jte,jde-1 )
L04824
L04825      k_start = kts
L04826      k_end = min( kte, kde-1 )
L04827
L04828  !  compute thermodynamics and velocities at pressure points (or half levels)
L04829
L04830      IF ( ( config_flags%use_theta_m .EQ. 1 ) .AND. (P_Qv .GE. PARAM_FIRST_SCALAR) ) THEN
L04831         do j = j_start,j_end
L04832         do k = k_start, k_end
L04833         do i = i_start, i_end
L04834           th_phy(i,k,j) = (t(i,k,j)+t0)/(1.+R_v/R_d*moist(i,k,j,P_QV))
L04835         enddo
L04836         enddo
L04837         enddo
L04838      ELSE
L04839         do j = j_start,j_end
L04840         do k = k_start, k_end
L04841         do i = i_start, i_end
L04842           th_phy(i,k,j) =  t(i,k,j)+t0
L04843         enddo
L04844         enddo
L04845         enddo
L04846      END IF
L04847
L04848      do j = j_start,j_end
L04849      do k = k_start, k_end
L04850      do i = i_start, i_end
L04851
L04852        th_phy_m_t0(i,k,j) = th_phy(i,k,j)-t0
L04853        p_phy(i,k,j) = p(i,k,j) + pb(i,k,j)
L04854        pi_phy(i,k,j) = (p_phy(i,k,j)/p1000mb)**rcp
L04855        t_phy(i,k,j) = th_phy(i,k,j)*pi_phy(i,k,j)
L04856        rho(i,k,j) = 1./alt(i,k,j)*(1.+moist(i,k,j,P_QV))
L04857        u_phy(i,k,j) = 0.5*(u(i,k,j)+u(i+1,k,j))
L04858        v_phy(i,k,j) = 0.5*(v(i,k,j)+v(i,k,j+1))
L04859
L04860      enddo
L04861      enddo
L04862      enddo
L04863
L04864  !  compute z at w points
L04865
L04866      do j = j_start,j_end
L04867      do k = k_start, kte
L04868      do i = i_start, i_end
L04869        z_at_w(i,k,j) = (phb(i,k,j)+ph(i,k,j))/g
L04870      enddo
L04871      enddo
L04872      enddo
L04873
L04874      do j = j_start,j_end
L04875      do k = k_start, kte-1
L04876      do i = i_start, i_end
L04877        dz8w(i,k,j) = z_at_w(i,k+1,j)-z_at_w(i,k,j)
L04878      enddo
L04879      enddo
L04880      enddo
L04881
L04882      do j = j_start,j_end
L04883      do i = i_start, i_end
L04884        dz8w(i,kte,j) = 0.
L04885      enddo
L04886      enddo
L04887
```

```
L04888  !  compute z at p points or half levels (average of z at full levels)
L04889
L04890       do j = j_start,j_end
L04891       do k = k_start, k_end
L04892       do i = i_start, i_end
L04893         z(i,k,j) = 0.5*(z_at_w(i,k,j) +z_at_w(i,k+1,j) )
L04894       enddo
L04895       enddo
L04896       enddo
L04897
L04898  !  interp t and p to full levels
L04899
L04900       do j = j_start,j_end
L04901       do k = 2, k_end
L04902       do i = i_start, i_end
L04903         p8w(i,k,j) = fzm(k)*p_phy(i,k,j)+fzp(k)*p_phy(i,k-1,j)
L04904         t8w(i,k,j) = fzm(k)*t_phy(i,k,j)+fzp(k)*t_phy(i,k-1,j)
L04905       enddo
L04906       enddo
L04907       enddo
L04908
L04909  !  extrapolate p and t to surface and top.
L04910  !  we'll use an extrapolation in z for now
L04911
L04912       do j = j_start,j_end
L04913       do i = i_start, i_end
L04914
L04915  ! bottom
L04916
L04917         z0 = z_at_w(i,1,j)
L04918         z1 = z(i,1,j)
L04919         z2 = z(i,2,j)
L04920         w1 = (z0 - z2)/(z1 - z2)
L04921         w2 = 1. - w1
L04922         p8w(i,1,j) = w1*p_phy(i,1,j)+w2*p_phy(i,2,j)
L04923         t8w(i,1,j) = w1*t_phy(i,1,j)+w2*t_phy(i,2,j)
L04924
L04925  ! top
L04926
L04927         z0 = z_at_w(i,kte,j)
L04928         z1 = z(i,k_end,j)
L04929         z2 = z(i,k_end-1,j)
L04930         w1 = (z0 - z2)/(z1 - z2)
L04931         w2 = 1. - w1
L04932
L04933  !      p8w(i,kde,j) = w1*p_phy(i,kde-1,j)+w2*p_phy(i,kde-2,j)
L04934  !!!  bug fix      extrapolate ln(p) so p is positive definite
L04935         p8w(i,kde,j) = exp(w1*log(p_phy(i,kde-1,j))+w2*log(p_phy(i,kde-2,j)))
L04936         t8w(i,kde,j) = w1*t_phy(i,kde-1,j)+w2*t_phy(i,kde-2,j)
L04937
L04938       enddo
L04939       enddo
L04940
L04941  ! calculate hydrostatic pressure at both full and half levels
L04942  ! first, full level p: assuming dry over model top
L04943
L04944       do j = j_start,j_end
L04945       do i = i_start, i_end
L04946          p_hyd_w(i,kte,j) = p_top
L04947       enddo
L04948       enddo
L04949
L04950       do j = j_start,j_end
L04951       do k = kte-1, k_start, -1
L04952       do i = i_start, i_end
L04953          qtot = 0.
L04954          do n = PARAM_FIRST_SCALAR,n_moist
L04955                qtot = qtot + moist(i,k,j,n)
L04956          enddo
L04957          p_hyd_w(i,k,j) = p_hyd_w(i,k+1,j) - (1.+qtot)*(c1(k)*MUT(i,j)+c2(k))*dnw(k)
L04958  !       p_hyd_w(i,k,j) = p_hyd_w(i,k+1,j)+1./alt(i,k,j)*(1.+moist(i,k,j,P_QV))*g*dz8w(i,k,j)
L04959       enddo
L04960       enddo
L04961       enddo
L04962
L04963  ! now calculate hydrostatic pressure at half levels
L04964
L04965       do j = j_start,j_end
L04966       do k = k_start, k_end
L04967       do i = i_start, i_end
L04968          p_hyd(i,k,j) = 0.5*(p_hyd_w(i,k,j)+p_hyd_w(i,k+1,j))
L04969       enddo
L04970       enddo
L04971       enddo
L04972
L04973  END SUBROUTINE phy_prep
```

讲解：code_31 § 四调用对象的本体，动力态 → 物理语言的全部换算：①th_phy：use_theta_m=1 时 (t+t0)/(1+rvovrd • qv)（湿位温还原干位温），否则 t+t0；**th_phy_m_t0=th_phy−t0**（physics 里常见减 t0 口径的来源）；②p_phy=p+pb（扰动 +基态 =全气压）、pi_phy=(p_phy/p1000mb)^rcp（Exner）、t_phy=θ • π；③**rho=1/alt • (1+qv)**——干空气比重容的倒数乘 (1+qv) 得湿空气密度（水物质质量计入分母的隐式处理）；④u_phy/v_phy 消交错；⑤z_at_w=φ/g、dz8w 层厚（**顶层 dz8w=0**——上层厚度的哨兵）、z 层中；⑥p8w/t8w 在 w 层的垂直插值（fzm/fzp 系数），**底边界 z 外插两点线性、顶边界 p 用 ln(p) 外插**（注释 bug fix——线性外插 p 可能出负值）；⑦**p_hyd_w 从顶层 p_top 向下静水积分**：p_hyd_w(k)=p_hyd_w(k+1)−(1+Σmoist)(c1 • mut+c2) • dnw——含水柱重的静水压（注释保留老式 alt • g • dz 口径），半层平均得 p_hyd——**物理方案吃的「静水压分量」在此成形**（radiation_driver 的 P=grid%p_hyd、noahdrv 的 P8W=P_hyd_w）。

六、位势方程右端 rhs_ph（头段）

```
!file: dyn_em/module_big_step_utilities_em.F    L1365-L1520
L01365  SUBROUTINE rhs_ph( ph_tend, u, v, ww,               &
L01366                     ph, ph_old, phb, w,              &
L01367                     mut, muuf, muvf,                 &
L01368                     c1f, c2f,                        &
L01369                     fnm, fnp,                        &
L01370                     rdnw, cfn, cfn1, rdx, rdy,       &
L01371                     msfux, msfuy, msfvx,             &
L01372                     msfvx_inv, msfvy,                &
L01373                     msftx, msfty,                    &
L01374                     non_hydrostatic,                 &
L01375                     config_flags,                    &
L01376                     ids, ide, jds, jde, kds, kde,    &
L01377                     ims, ime, jms, jme, kms, kme,    &
L01378                     its, ite, jts, jte, kts, kte    )
L01379     IMPLICIT NONE
L01380
L01381     TYPE(grid_config_rec_type), INTENT(IN   ) :: config_flags
L01382
L01383     INTEGER ,          INTENT(IN   ) :: ids, ide, jds, jde, kds, kde, &
L01384                                         ims, ime, jms, jme, kms, kme, &
L01385                                         its, ite, jts, jte, kts, kte
L01386
L01387     REAL, DIMENSION( ims:ime, kms:kme , jms:jme ), INTENT(IN   ) ::        &
L01388                                                                      u,   &
L01389                                                                      v,   &
L01390                                                                      ww,  &
L01391                                                                      ph,  &
L01392                                                                      ph_old, &
L01393                                                                      phb, &
L01394                                                                     w
L01395
L01396     REAL, DIMENSION( ims:ime, kms:kme , jms:jme ), INTENT(INOUT) :: ph_tend
L01397
L01398     REAL, DIMENSION( ims:ime, jms:jme ), INTENT(IN   ) :: muuf, muvf, mut,   &
L01399                                                           msfux, msfuy, &
L01400                                                           msfvx, msfvy, &
L01401                                                           msftx, msfty, &
L01402                                                           msfvx_inv
L01403
L01404     REAL, DIMENSION( kms:kme ), INTENT(IN   ) :: rdnw, fnm, fnp
L01405
L01406     REAL, DIMENSION( kms:kme ), INTENT(IN   ) :: c1f, c2f
L01407
L01408
L01409     REAL,  INTENT(IN   ) :: cfn, cfn1, rdx, rdy
L01410
L01411     LOGICAL,  INTENT(IN   )  ::  non_hydrostatic
L01412
L01413     ! Local stuff
L01414
L01415     INTEGER :: i, j, k, itf, jtf, ktf, kz, i_start, j_start
L01416     REAL    :: ur, ul, ub, vr, vl, vb
L01417     REAL, DIMENSION(its:ite,kts:kte) :: wdwn
L01418
L01419     INTEGER :: advective_order
L01420
L01421     LOGICAL :: specified
L01422
L01423  !<DESCRIPTION>
L01424  !
L01425  ! rhs_ph calculates the large-timestep tendency terms for the geopotential
L01426  ! equation.  These terms include the advection and "gw".  The geopotential
L01427  ! equation is cast in advective form, so we don't use the flux form advection
L01428  ! algorithms here.
L01429  !
L01430  !</DESCRIPTION>
L01431
L01432     specified = .false.
L01433     if(config_flags%specified .or. config_flags%nested) specified = .true.
L01434
L01435     advective_order = config_flags%h_sca_adv_order
L01436
L01437     itf=MIN(ite,ide-1)
L01438     jtf=MIN(jte,jde-1)
L01439     ktf=MIN(kte,kde-1)
L01440
L01441  !  Notes on map scale factors (WCS, 2 march 2008)
L01442  !  phi equation is:   mu/my d/dt(phi) = -(1/my) mx mu u  d/dx(phi)
L01443  !                                       -(1/my) my mu v d/dy(phi)
L01444  !                                       - omega d/d_eta(phi)
L01445  !                                              +mu g w/my
L01446  !
L01447  !  A little further explanation...
L01448  !  The tendency term we are computing here is for mu/my d/dt(phi).  It is advective form
L01449  !  but it is multiplied be mu/my.  It will be decoupled from (mu/my) when the implicit w-phi
L01450  !  solution is computed in subourine advance_w.  The formulation dates from the early
L01451  !  days of the mass coordinate model when we were testing both a flux and an advective formulation
L01452  !  for the geopotential equation and different forms of the vertical momentum equation and the
L01453  !  vertically implicit solver.
L01454
L01455  ! advective form for the geopotential equation
L01456
L01457     !  RHS term 3 is: - omega partial d/dnu(phi)
L01458
L01459     DO j = jts, jtf
L01460        IF (config_flags%phi_adv_z == 2) THEN
L01461           !  First get staggered partial d/dnu(phi) and then multiply with omega
L01462           !  to avoid double staggering of omega
L01463
L01464           DO k = 2, kte
L01465           DO i = its, itf
L01466              wdwn(i,k) = rdnw(k-1)*(ph(i,k,j)-ph(i,k-1,j)+phb(i,k,j)-phb(i,k-1,j))
L01467           ENDDO
L01468           ENDDO
```

```
L01469
L01470           DO k = 2, kte-1
L01471           DO i = its, itf
L01472                ph_tend(i,k,j) = ph_tend(i,k,j)                          &
L01473                                 - ww(i,k,j)*(fnm(k)*wdwn(i,k+1)+fnp(k)*wdwn(i,k))
L01474           ENDDO
L01475           ENDDO
L01476         ELSE
L01477           !  First destagger omega and multiply with partial d/dnu(phi), then stagger the product
L01478
L01479           DO k = 2, kte
L01480           DO i = its, itf
L01481                wdwn(i,k) = .5*(ww(i,k,j)+ww(i,k-1,j))*rdnw(k-1)              &
L01482                              *(ph(i,k,j)-ph(i,k-1,j)+phb(i,k,j)-phb(i,k-1,j))
L01483           ENDDO
L01484           ENDDO
L01485
L01486           DO k = 2, kte-1
L01487           DO i = its, itf
L01488                ph_tend(i,k,j) = ph_tend(i,k,j)                          &
L01489                                 - (fnm(k)*wdwn(i,k+1)+fnp(k)*wdwn(i,k))
L01490           ENDDO
L01491           ENDDO
L01492         ENDIF
L01493
L01494       ENDDO
L01495
L01496       IF (non_hydrostatic) THEN  ! add in "gw" term.
L01497       DO j = jts, jtf            ! in hydrostatic mode, "gw" will be diagnosed
L01498                                  ! after the timestep to give us "w"
L01499         DO i = its, itf
L01500            ph_tend(i,kde,j) = 0.
L01501         ENDDO
L01502
L01503         DO k = 2, kte
L01504         DO i = its, itf
L01505            ! phi equation RHS term 4
L01506            ph_tend(i,k,j) = ph_tend(i,k,j) + (c1f(k)*mut(i,j)+c2f(k))*g*w(i,k,j)/msfty(i,j)
L01507         ENDDO
L01508         ENDDO
L01509
L01510       ENDDO
L01511
L01512       END IF
L01513
L01514    !  Notes on map scale factors:
L01515    !  RHS terms 1 and 2 are: -(1/my) mx u mu partial d/dx(phi)
L01516    !                         -(1/my) my v mu partial d/dy(phi)
L01517
L01518       IF (advective_order <= 2) THEN
L01519
L01520    !  y (v) advection
```

讲解：φ 方程大步倾向的**对流形式**（注释：mu/my • dφ/dt=−(1/my)mx • μu • ∂xφ−(1/my)my • μv • ∂yφ−ω • ∂ηφ+μgw/my）——不用通量形式的历史原因（质量坐标早期通式/ advective 两版并测，「will be decoupled from (mu/my) in advance_w」——μ/my 因子在 advance_w 的隐式解里解耦）；①**ω • ∂ηφ 项**：phi_adv_z=2 先差分 φ 再乘 ω（避免 ω 双重交错）、否则先消 ω 交错再差分（phi_adv_z namelist 的两种差分序）；②**gw 项仅非静力模式**（静力模式的 w 在时间步后诊断），ph_tend(kde)=0 顶边界、μ • g • w/msfty——地图因子只除 y 向；③ 地图因子注记（WCS 2008-03-02）推导 mx/my 的位置；后续段（未录）是二阶/高阶水平平流（h_sca_adv_order）与 specified/nested 边界处理。rhs_ph 与 horizontal_pressure_gradient/pg_buoy_w/w_damp 同属大步右端族（2183–2711 行，未录）。

七、未录单元注记

1. **coriolis/perturbation_coriolis/curvature（L3640–L4469）**：科氏力（地图投影的 f/旋转项）、扰动流场科氏、曲率项（地图投影 β 平面修正）——q 向量与地图因子的配套，公式见 ch10。
2. **horizontal_pressure_gradient/pg_buoy_w/w_damp（L2183–L2711）**：非静力水平气压梯度（全位势 php 差分）、w 方程浮力项、w_damp（namelist w-damping，w_alpha/w_beta 常数在 code_01）。
3. **couple/decouple/couple_momentum（L524–L949, L4475–L4568）**：μ 耦合/解耦——通量形式变量（μu）与速度（u）的互相换算，RK 平流前 couple、倾向后 decouple。
4. **moist_physics_prep_em/finish（L5394–L5784）**：RK 子步内水物质的前后处理（饱和调整前的负值清理与守恒回填，与微物理的五保险衔接，见 ch13）。
5. **rk_rayleigh_damp/theta_relaxation（L5836–L6216）**：大步 Rayleigh 阻尼与 θ 松弛（damp_opt 分派）；sixth_order_diffusion（L6220–L6636）六阶数值扩散（diff_6th_opt）。
6. **horizontal/vertical_diffusion 系（L2715–L3581）**：地图因子版扩散族——与 module_diffusion_em（TKE/Km 系，part2 用）分工：这里是无 TKE 的经典 Km 扩散。

八、改造入口

1. **状态方程**：calc_p_rho_phi 的 hypsometric_opt 两式与静水积分——高阶静力平衡修订（如 CAPF 式）的挂点；use_theta_m 分支同步。
2. **物理翻译**：phy_prep 的新诊断量出口（如加等效位温）——下游 physics 全部自动获得。
3. **顶边界外插**：phy_prep 的 ln(p) 外插——模式顶气压异常的第一嫌疑处。
4. **地图因子**：rhs_ph/coriolis/diffusion 系的 msf 系——极区/兰勃特变形敏感实验的统一入口。

九、上下游链接

- 调用方：`code_50_solve_em.md`、`code_31_module_first_rk_step_part1.md`（phy_prep） 、`code_32_module_first_rk_step_part2.md`（conv_t_tendf_to_moist）
- 常数底座：`code_01_module_model_constants.md`（cpovcv/cvpm/rvovrd/p1000mb）
- 教材正文：ch10_vertical_coords_hydrostatics.md、ch21_solve_em_dynamics.md

code_31 — 非时间分裂物理总装·上（dyn_em/module_first_rk_step_part1.F：辐射 → 陆面 →PBL→ 积云 → 浅对流 →FDDA 调用序）

一、版本指纹

源文件	行数	md5（整文件，按字节）
dyn_em/module_first_rk_step_part1.F	2027	c5f5921d9fef20d6f1e9227dd472bdec

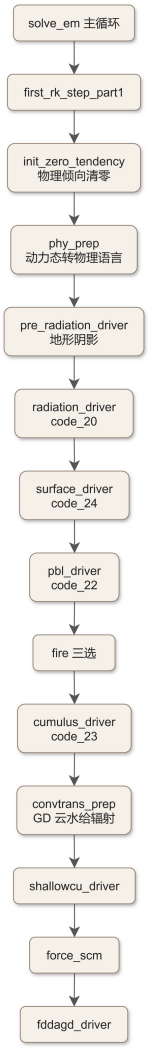
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：单子程序 first_rk_step_part1（L10–L2025）按调用段全收——正是本篇价值所在：每个物理驱动器的**完整实参表**（谁把哪个 Registry 态喂给谁）在别处看不到全景；讲解聚焦调用序与关键实参，方案内部见各方案篇。

二、定位

一句话：solve_em 时间步主循环里、RK3 循环**之前**调用的物理总装上半——把物理倾向数组清零、把动力态换算成物理语言（phy_prep），然后按**辐射 → 陆面 →PBL→ 积云 → 浅对流 →SCM 强迫 → 格点 nudging** 的顺序逐个驱动（教材 ch21 §21.2 的「非时间分裂物理」在源码层的本尊）。

谁在调它：dyn_em/solve_em.F 的主循环（`code_50_solve_em.md` 的调用点地图）；下半（微物理等）在 `code_32_first_rk_step_part2.md`。教材锚点：ch21_solve_em_dynamics.md。

调用序地图：



三、模块头与 USE 链

```
!file: dyn_em/module_first_rk_step_part1.F    L6-L72
L00006  MODULE module_first_rk_step_part1
L00007
L00008  CONTAINS
L00009
L00010    SUBROUTINE first_rk_step_part1 (   grid , config_flags              &
L00011                                , moist , moist_tend                &
L00012                                , chem  , chem_tend                 &
L00013                                , tracer, tracer_tend               &
L00014                                , scalar , scalar_tend              &
L00015                                , fdda3d, fdda2d                    &
L00016                                , aerod                             &
L00017                                , ru_tendf, rv_tendf                &
L00018                                , rw_tendf, t_tendf                 &
L00019                                , ph_tendf, mu_tendf                &
L00020                                , tke_tend                          &
L00021                                , adapt_step_flag , curr_secs       &
L00022                                , curr_mins2                        &
L00023                                , psim , psih , gz1oz0 , chklowq    &
L00024                                , cu_act_flag , hol , th_phy        &
L00025                                , pi_phy , p_phy , t_phy            &
L00026                                , dz8w , p8w , t8w                  &
L00027                                , ids, ide, jds, jde, kds, kde      &
L00028                                , ims, ime, jms, jme, kms, kme      &
L00029                                , ips, ipe, jps, jpe, kps, kpe      &
L00030                                , imsx,imex,jmsx,jmex,kmsx,kmex     &
L00031                                , ipsx,ipex,jpsx,jpex,kpsx,kpex     &
L00032                                , imsy,imey,jmsy,jmey,kmsy,kmey     &
L00033                                , ipsy,ipey,jpsy,jpey,kpsy,kpey     &
L00034                                , k_start , k_end                   &
L00035                                , f_flux                            &
L00036                                , aerocu                            &
L00037                                , restart_flag                      &
L00038                                , feedback_is_ready                 &
L00039                               )
L00040      USE module_state_description
L00041      USE module_model_constants
L00042      USE module_domain, ONLY : domain, domain_clock_get, get_ijk_from_subgrid
L00043      USE module_configure, ONLY : grid_config_rec_type, model_config_rec
L00044      USE module_radiation_driver, ONLY : pre_radiation_driver, radiation_driver
L00045      USE module_surface_driver, ONLY : surface_driver
```

```
L00046      USE module_cumulus_driver, ONLY : cumulus_driver
L00047      USE module_shallowcu_driver, ONLY : shallowcu_driver
L00048      USE module_pbl_driver, ONLY : pbl_driver
L00049      USE module_fr_fire_driver_wrf, ONLY : fire_driver_em_step
L00050      USE module_fddagd_driver, ONLY : fddagd_driver
L00051      USE module_em, ONLY : init_zero_tendency
L00052      USE module_force_scm
L00053      USE module_convtrans_prep
L00054      USE module_big_step_utilities_em, ONLY : phy_prep
L00055   !use module_scalar_tables
L00056   #ifdef DM_PARALLEL
L00057      USE module_dm, ONLY : local_communicator, mytask, ntasks, ntasks_x, ntasks_y, local_communicator_periodic,
↪   wrf_dm_maxval
L00058      USE module_comm_dm, ONLY : halo_em_phys_a_sub,halo_em_fdda_sfc_sub,halo_pwp_sub,halo_em_chem_e_3_sub, &
L00059      halo_em_chem_e_5_sub, halo_em_hydro_noahmp_sub, halo_em_tracer_e_5_sub, halo_em_cfbm_sub
L00060   #if ( WRFPLUS == 1 )
L00061      USE module_comm_dm, ONLY : halo_em_phys_a_bl_surf_sub
L00062   #endif
L00063   #endif
L00064      USE module_utility
L00065
L00066   #ifdef WRF_CFBM
L00067        ! CFBM modules
L00068      use advance_mod, only : Advance_state
L00069      use wrf_mod, only : Interp_wrfwinds_to_cfbm, Provide_atm_feedback
L00070      use interp_mod, only : VINTERP_WINDS_FROM_3D_WINDS, VINTERP_WINDS_FROM_10M_WINDS
L00071      use coupling_mod, only : Interp_horizontal
L00072   #endif
```

讲解：USE 链就是调用序的静态清单——五大驱动器（radiation/surface/cumulus/shallowcu/pbl）+ fire + fddagd + force_scm + convtrans_prep + big_step_utilities 的 phy_prep；DM_PARALLEL 下八种 HALO 交换子程序（phys_a/fdda_sfc/pwp/chem_e_3/chem_e_5/noahmp/tracer_e_5/cfbm）——**每次调用前后的 halo 刷新都是显式 include 的**（§四各段的 include 行）；WRF_CFBM（耦合火灾模式）单独一组模块。

四、倾向清零与状态换算

```
!file: dyn_em/module_first_rk_step_part1.F    L171-L233
L00171    CALL get_ijk_from_subgrid (  grid ,                   &
L00172                                 sids, side, sjds, sjde, skds, skde,    &
L00173                                 sims, sime, sjms, sjme, skms, skme,    &
L00174                                 sips, sipe, sjps, sjpe, skps, skpe    )
L00175
L00176    ! initialize all tendencies to zero in order to update physics
L00177    ! tendencies first (separate from dry dynamics).
L00178
L00179      l_flux=.FALSE.
L00180      if (present(f_flux)) l_flux=f_flux
L00181
L00182       rk_step = 1
L00183
L00184   BENCH_START(init_zero_tend_tim)
L00185          !$OMP PARALLEL DO   &
L00186          !$OMP PRIVATE ( ij )
L00187
L00188          DO ij = 1 , grid%num_tiles
L00189
L00190            CALL wrf_debug ( 200 , ' call init_zero_tendency' )
L00191            CALL init_zero_tendency ( ru_tendf, rv_tendf, rw_tendf,     &
L00192                                      ph_tendf, t_tendf, tke_tend,      &
L00193                                      mu_tendf,                         &
L00194                                      moist_tend,chem_tend,scalar_tend, &
L00195                                      tracer_tend,num_tracer,           &
L00196                                      num_moist,num_chem,num_scalar,    &
L00197                                                                        &
L00198                                      rk_step,                          &
L00199                                      ids, ide, jds, jde, kds, kde,     &
L00200                                      ims, ime, jms, jme, kms, kme,     &
L00201                                      grid%i_start(ij), grid%i_end(ij), &
L00202                                      grid%j_start(ij), grid%j_end(ij), &
L00203                                      k_start, k_end                   )
L00204          END DO
L00205          !$OMP END PARALLEL DO
L00206   BENCH_END(init_zero_tend_tim)
L00207
L00208   #ifdef DM_PARALLEL
L00209   #     include "HALO_EM_PHYS_A.inc"
L00210   #endif
L00211
L00212         !$OMP PARALLEL DO   &
L00213         !$OMP PRIVATE ( ij )
L00214         DO ij = 1 , grid%num_tiles
L00215
L00216           CALL wrf_debug ( 200 , ' call phy_prep' )
L00217           CALL phy_prep ( config_flags,                                  &
L00218                           grid%mut, grid%muu, grid%muv,                  &
L00219                           grid%c1h, grid%c2h, grid%c1f, grid%c2f,        &
L00220                           grid%u_2, grid%v_2, grid%p, grid%pb, grid%alt, &
L00221                           grid%ph_2, grid%phb, grid%t_2, moist, num_moist, &
L00222                           grid%rho,th_phy, grid%th_phy_m_t0,             &
L00223                           p_phy, pi_phy, grid%u_phy, grid%v_phy,         &
L00224                           p8w, t_phy, t8w, grid%z, grid%z_at_w, dz8w,    &
L00225                           grid%p_hyd, grid%p_hyd_w, grid%dnw,            &
L00226                           grid%fnm, grid%fnp, grid%znw, grid%p_top,      &
L00227                           ids, ide, jds, jde, kds, kde,                  &
L00228                           ims, ime, jms, jme, kms, kme,                  &
L00229                           grid%i_start(ij), grid%i_end(ij),              &
L00230                           grid%j_start(ij), grid%j_end(ij),              &
L00231                           k_start, k_end                                 )
```

```
L00232        ENDDO
L00233        !$OMP END PARALLEL DO
```

讲解：①get_ijk_from_subgrid 取子格索引（共享内存 OpenMP 的纵向分块 k_start/k_end 就是传给各物理调用的 kte）；②**rk_step=1 硬编码**——物理倾向数组（ru_tendf 系）独立于动力倾向，init_zero_tendency 只在每步开头清一次（微物理的 tendency 走另一条账，见 code_32）；③**phy_prep 是动力↔物理的「翻译官」**：从半拉格朗日动力态（u_2/v_2/p/pb/alt/ph_2/t_2/moist）换算出物理语言全套（th_phy/pi_phy/p_phy/t_phy/p8w/t8w/z/z_at_w/dz8w）——注意入参带 **grid%th_phy_m_t0**（湿位温减 t0，use_theta_m 选项的痕迹）与 **grid%p_hyd/p_hyd_w**（静力气压——物理方案吃的是静水压分量而非全气压）；前面 include 的 HALO_EM_PHYS_A 保证云水等标量 halo 新鲜。

五、辐射段：pre_radiation_driver + radiation_driver

```
!file: dyn_em/module_first_rk_step_part1.F    L237-L268
L00237  ! radiation
L00238        CALL domain_clock_get( grid, current_time=currentTime, &
L00239                               current_timestr=mesg )
L00240        CALL WRFU_TimeGet( currentTime, YY=yr, dayOfYear=day, H=hr, M=minute, S=sec, rc=rc)
L00241            IF( rc/= WRFU_SUCCESS)THEN
L00242            CALL wrf_error_fatal('WRFU_TimeGet failed')
L00243            ENDIF
L00244
L00245  ! this driver is only needed to handle non-local shadowing effects
L00246         CALL pre_radiation_driver ( grid, config_flags                    &
L00247        &         ,itimestep=grid%itimestep, ra_call_offset=grid%ra_call_offset    &
L00248        &         ,XLAT=grid%xlat, XLONG=grid%xlong, GMT=grid%gmt          &
L00249        &         ,julian=grid%julian, xtime=grid%xtime, RADT=grid%radt    &
L00250        &         ,STEPRA=grid%stepra                                      &
L00251        &         ,ht=grid%ht,dx=grid%dx,dy=grid%dy                        &
L00252        &         ,dx2d=grid%dx2d,area2d=grid%area2d                       &
L00253        &         ,sina=grid%sina,cosa=grid%cosa                           &
L00254        &         ,shadowmask=grid%shadowmask,slope_rad=config_flags%slope_rad     &
L00255        &         ,topo_shading=config_flags%topo_shading                  &
L00256        &         ,shadlen=config_flags%shadlen,ht_shad=grid%ht_shad,ht_loc=ht_loc &
L00257        &         ,ht_shad_bxs=grid%ht_shad_bxs, ht_shad_bxe=grid%ht_shad_bxe      &
L00258        &         ,ht_shad_bys=grid%ht_shad_bys, ht_shad_bye=grid%ht_shad_bye      &
L00259        &         ,nested=config_flags%nested, min_ptchsz=grid%min_ptchsz          &
L00260        &         ,spec_bdy_width=config_flags%spec_bdy_width              &
L00261              ! indexes
L00262        &         ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde       &
L00263        &         ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme       &
L00264        &         ,IPS=ips,IPE=ipe, JPS=jps,JPE=jpe, KPS=kps,KPE=kpe       &
L00265        &         ,i_start=grid%i_start,i_end=min(grid%i_end, ide-1)        &
L00266        &         ,j_start=grid%j_start,j_end=min(grid%j_end, jde-1)        &
L00267        &         ,kts=k_start, kte=min(k_end,kde-1)                        &
L00268        &         ,num_tiles=grid%num_tiles                                 )
```

讲解：① 先取当前时刻（WRFU_TimeGet 拆 yr/hr/min/s——ESMF 时钟，yr 直通 radiation_driver 的 ghg_input）；②pre_radiation_driver 的注释自承「only needed to handle non-local shadowing effects」——topo_shading=1 时的跨 patch 阴影提前算（阴影要越过处理器 patch 边界，放在物理 tile 循环外做全局迭代，见 code_20 § 九）；实参里 ht_shad_bxs/bxe/bys/bye 是**边界缓冲版本阴影场**（嵌套域边界交换用）。

```
!file: dyn_em/module_first_rk_step_part1.F    L270-L543
L00270        CALL wrf_debug ( 200 , ' call radiation_driver' )
L00271  BENCH_START(rad_driver_tim)
L00272
L00273         CALL radiation_driver(                                           &
L00274        &          p_top=grid%p_top & !DJW 140312 added p_top for vertical nesting
L00275        &         ,ACFRCV=grid%acfrcv      ,ACFRST=grid%acfrst      ,ALBEDO=grid%albedo  &
L00276        &         ,CFRACH=grid%cfrach      ,CFRACL=grid%cfracl      ,CFRACM=grid%cfracm  &
L00277        &         ,CUPPT=grid%cuppt        ,CZMEAN=grid%czmean      ,DT=grid%dt          &
L00278        &         ,DZ8W=dz8w               ,EMISS=grid%emiss        ,GLW=grid%glw        &
L00279        &         ,GMT=grid%gmt            ,GSW=grid%gsw            ,HBOT=grid%hbot      &
L00280        &         ,HTOP=grid%htop          ,HBOTR=grid%hbotr        ,HTOPR=grid%htopr    &
L00281        &         ,ICLOUD=config_flags%icloud                                            &
L00282        &         ,ITIMESTEP=grid%itimestep,JULDAY=grid%julday      , JULIAN=grid%julian &
L00283        &         ,JULYR=grid%julyr        ,LW_PHYSICS=config_flags%ra_lw_physics       &
L00284        &         ,NCFRCV=grid%ncfrcv      ,NCFRST=grid%ncfrst      ,NPHS=1              &
L00285        &         ,o3input=config_flags%o3input     ,O3rad=grid%o3rad                  &
L00286        &         ,aer_opt=config_flags%aer_opt ,aerod=aerod(:,:,:,P_ocarbon:P_upperaer) &
L00287        &         ,swint_opt=config_flags%swint_opt                                      &
L00288        &         ,solar_opt=config_flags%solar_diagnostics                              &
L00289        &         ,P8W=grid%p_hyd_w        ,P=grid%p_hyd            ,PI=pi_phy           &
L00290        &         ,RADT=grid%radt          ,RA_CALL_OFFSET=grid%ra_call_offset          &
L00291        &         ,RHO=grid%rho            ,RLWTOA=grid%rlwtoa                          &
L00292        &         ,RSWTOA=grid%rswtoa      ,RTHRATEN=grid%rthraten                      &
L00293        &         ,RTHRATENLW=grid%rthratenlw       ,RTHRATENSW=grid%rthratensw         &
L00294        &         ,RTHRATENLWC=grid%rthratenlwc     ,RTHRATENSWC=grid%rthratenswc       &
L00295        &         ,SNOW=grid%snow          ,STEPRA=grid%stepra      ,SWDOWN=grid%swdown  &
L00296        &         ,SWDOWNC=grid%swdownc    ,SW_PHYSICS=config_flags%ra_sw_physics       &
L00297        &         ,T8W=t8w                 ,T=grid%t_phy           ,TAUCLDC=grid%taucldc &
L00298        &         ,TAUCLDI=grid%taucldi    ,TSK=grid%tsk            ,VEGFRA=grid%vegfra  &
L00299        &         ,WARM_RAIN=grid%warm_rain ,XICE=grid%xice          ,XLAND=grid%xland    &
L00300        &         ,XLAT=grid%xlat          ,XLONG=grid%xlong        ,YR=yr               &
L00301              ! SSiB LSM radiation components (fds 06/2010)
L00302        &         ,ALSWVISDIR=grid%alswvisdir ,ALSWVISDIF=grid%alswvisdif      &  !ssib
L00303        &         ,ALSWNIRDIR=grid%alswnirdir ,ALSWNIRDIF=grid%alswnirdif      &  !ssib
L00304        &         ,SWVISDIR=grid%swvisdir ,SWVISDIF=grid%swvisdif              &  !ssib
L00305        &         ,SWNIRDIR=grid%swnirdir ,SWNIRDIF=grid%swnirdif              &  !ssib
```

```
L00306       &        ,SF_SURFACE_PHYSICS=config_flags%sf_surface_physics          &  !ssib
L00307  ! WRF-solar and aerosol variables from jararias 2013/8 and 2013/11
L00308       &       ,SWDDIR=grid%swddir,SWDDNI=grid%swddni,SWDDIF=grid%swddif                                  &
L00309       &       ,SWDDIRC=grid%swddirc, SWDDNIC=grid%swddnic                                                &
L00310       &       ,Gx=grid%Gx,Bx=grid%Bx,gg=grid%gg,bb=grid%bb                                               &
L00311       &       ,swdown_ref=grid%swdown_ref,swddir_ref=grid%swddir_ref                                     &
L00312       &       ,coszen_ref=grid%coszen_ref                                                                &
L00313       &       ,aer_type=config_flags%aer_type                                                            &
L00314       &       ,aer_aod550_opt=config_flags%aer_aod550_opt,aer_aod550_val=config_flags%aer_aod550_val &
L00315       &       ,aer_angexp_opt=config_flags%aer_angexp_opt,aer_angexp_val=config_flags%aer_angexp_val &
L00316       &       ,aer_ssa_opt=config_flags%aer_ssa_opt,aer_ssa_val=config_flags%aer_ssa_val                 &
L00317       &       ,aer_asy_opt=config_flags%aer_asy_opt,aer_asy_val=config_flags%aer_asy_val                 &
L00318       &       ,aod5502d=grid%aod5502d,angexp2d=grid%angexp2d,aerssa2d=grid%aerssa2d                       &
L00319       &       ,aerasy2d=grid%aerasy2d,aod5503d=grid%aod5503d                                             &
L00320       &       ,taod5502d=grid%taod5502d,taod5503d=grid%taod5503d                                         & ! Trude
L00321  !Optional solar variables
L00322       &        ,DECLINX=grid%declin ,SOLCONX=grid%solcon ,COSZEN=grid%coszen ,HRANG=grid%hrang    &
L00323       &        , CEN_LAT=grid%cen_lat                                                                    &
L00324       &        ,Z=grid%z                                                                                 &
L00325       &        ,ALEVSIZ=grid%alevsiz, no_src_types=grid%no_src_types                                     &
L00326       &        ,LEVSIZ=grid%levsiz, N_OZMIXM=num_ozmixm                                                  &
L00327       &        ,N_AEROSOLC=num_aerosolc                                                                  &
L00328       &        ,PAERLEV=grid%paerlev   ,ID=grid%id                                                       &
L00329       &        ,CAM_ABS_DIM1=grid%cam_abs_dim1, CAM_ABS_DIM2=grid%cam_abs_dim2 &
L00330       &        ,CAM_ABS_FREQ_S=grid%cam_abs_freq_s                                                       &
L00331       &        ,XTIME=grid%xtime                                                                         &
L00332                ,CURR_SECS=curr_secs, ADAPT_STEP_FLAG=adapt_step_flag                                     &
L00333  #if ( EM_CORE == 1)
L00334       &        ,swdown2=grid%swdown2, swddni2=grid%swddni2                                               & ! FARMS coupling
L00335       &        ,swddif2=grid%swddif2, swddir2=grid%swddir2                                               & ! FARMS coupling
L00336       &        ,swdownc2=grid%swdownc2, swddnic2=grid%swddnic2                                           & ! FARMS coupling
L00337       &        ,couple_farms=config_flags%couple_farms                                                   &
L00338  #endif
L00339  ! WRF-Solar EPS: FARMS
L00340       &       ,multi_perturb=config_flags% multi_perturb                                                 &
L00341       &       ,pert_farms=config_flags%pert_farms                                                        &
L00342       &       ,perts_albedo=grid%pert3d(:,:,:,P_PALBEDO)                                                 &
L00343       &       ,perts_aod=grid%pert3d(:,:,:,P_PAOD)                                                       &
L00344       &       ,perts_angstrom=grid%pert3d(:,:,:,P_PANGSTROM)                                             &
L00345       &       ,perts_assymfac=grid%pert3d(:,:,:,P_PASSYMFAC)                                             &
L00346       &       ,perts_qvapor=grid%pert3d(:,:,:,P_PQVAPOR)                                                 &
L00347       &       ,perts_qcloud=grid%pert3d(:,:,:,P_PQCLOUD)                                                 &
L00348       &       ,perts_qsnow=grid%pert3d(:,:,:,P_PQSNOW)                                                   &
L00349       &       ,pert_farms_albedo=config_flags%pert_farms_albedo                                          &
L00350       &       ,pert_farms_aod=config_flags%pert_farms_aod                                                &
L00351       &       ,pert_farms_angexp=config_flags%pert_farms_angexp                                          &
L00352       &       ,pert_farms_aerasy=config_flags%pert_farms_aerasy                                          &
L00353       &       ,pert_farms_qv=config_flags%pert_farms_qv                                                  &
L00354       &       ,pert_farms_qc=config_flags%pert_farms_qc                                                  &
L00355       &       ,pert_farms_qs=config_flags%pert_farms_qs                                                  &
L00356  ! WRF-Solar EPS: cld3
L00357       &       ,pert_cld3=config_flags%pert_cld3                                                          &
L00358       &       ,perts_th=grid%pert3d(:,:,:,P_PTH)                                                           &
L00359       &       ,pert_cld3_qv=config_flags%pert_cld3_qv                                                    &
L00360       &       ,pert_cld3_t=config_flags%pert_cld3_t                                                      &
L00361  !BSINGH - For WRFCuP scheme
L00362       &        ,CU_PHYSICS=config_flags%cu_physics                                                       & !CuP, wig 5-Oct-2006
L00363       &        ,SHALLOWCU_FORCED_RA=config_flags%shallowcu_forced_ra                                     & !CuP, wig
L00364       &        ,CUBOT=grid%cubot, CUTOP=grid%cutop                                                       & !CuP, wig 9-Oct-2006
L00365       &        ,CLDFRA_CUP=grid%cldfra_cup                                                               & !CuP, wig 1-Oct-2006
L00366       &        ,SHALL=grid%shall                                                                         & !CuP, wig 4-Feb-2008
L00367  !BSINGH - ENDS
L00368                ! indexes
L00369       &        ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde                                        &
L00370       &        ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme                                        &
L00371       &        ,i_start=grid%i_start,i_end=min(grid%i_end, ide-1)                                         &
L00372       &        ,j_start=grid%j_start,j_end=min(grid%j_end, jde-1)                                         &
L00373       &        ,kts=k_start, kte=min(k_end,kde-1)                                                         &
L00374       &        ,num_tiles=grid%num_tiles                                                                 &
L00375                ! Optional
L00376  !JJS 20101020 vvvvv
L00377  #if ( WRF_CHEM == 1)
L00378       &        , CHEM=chem                                                                               &
L00379       &        , AOD_OUT=grid%aod_out                                                                    &
L00380       &        , AOD2D_OUT=grid%aod2d_out                                                                &  !goddardrad
L00381       &        , ATOP2D_OUT=grid%atop2d_out                                                              &  !goddardrad
L00382       &        , CHEM_OPT=config_flags%chem_opt                                                          &
L00383       &        , GSFCRAD_GOCART_COUPLING=config_flags%gsfcrad_gocart_coupling &
L00384  #endif
L00385       &        , TLWDN=grid%tlwdn, TLWUP=grid%tlwup                                                      & ! goddard schemes
L00386       &        , SLWDN=grid%slwdn, SLWUP=grid%slwup                                                      & ! goddard schemes
L00387       &        , TSWDN=grid%tswdn, TSWUP=grid%tswup                                                      & ! goddard schemes
L00388       &        , SSWDN=grid%sswdn, SSWUP=grid%sswup                                                      & ! goddard schemes
L00389       &        , RE_CLOUD_GSFC=grid%re_cloud_gsfc                                                        & ! goddard schemes
L00390       &        , RE_RAIN_GSFC=grid%re_rain_gsfc                                                          & ! goddard schemes
L00391       &        , RE_ICE_GSFC=grid%re_ice_gsfc                                                            & ! goddard schemes
L00392       &        , RE_SNOW_GSFC=grid%re_snow_gsfc                                                          & ! goddard schemes
L00393       &        , RE_GRAUPEL_GSFC=grid%re_graupel_gsfc                                                    & ! goddard schemes
L00394       &        , RE_HAIL_GSFC=grid%re_hail_gsfc                                                          & ! goddard schemes
L00395       &        , COD2D_OUT=grid%cod2d_out                                                                & ! goddardrad
L00396       &        , CTOP2D_OUT=grid%ctop2d_out                                                              & ! goddardrad
L00397  !JJS 20101020 ^^^^^
L00398  !ZCX+ cloud fraction for CLWRF
L00399       &        , CLDT=grid%cldt, ZNU=grid%znu                                                            &
L00400  !ZCX-
L00401       &        , CLDFRA=grid%cldfra, CLDFRA_MP_ALL=grid%cldfra_mp_all                                    &
L00402       &        , CCLDFRA=grid%ccldfra                                                                    &
L00403       &        , QCCONV=grid%qcconv, QICONV=grid%qiconv                                                  &
L00404       &        , BMJ_RAD_FEEDBACK=config_flags%bmj_rad_feedback                                          &
L00405       &        , LRADIUS=grid%LRADIUS,IRADIUS=grid%IRADIUS                                               & !BSINGH(01/22/2014)
L00406       &        , CLDFRA_DP=grid%cldfra_dp                                                                & ! okay for subgrid cloud
L00407       &        , CLDFRA_SH=grid%cldfra_sh                                                                &
L00408       &        , icloud_bl=config_flags%icloud_bl                                                        &
L00409       &        , cldovrlp=config_flags%cldovrlp                                                          &
L00410       &        , idcor=config_flags%idcor                                                                &
L00411       &        , qc_bl=grid%qc_bl,qi_bl=grid%qi_bl,cldfra_bl=grid%cldfra_bl&
L00412       &        , re_cloud=grid%re_cloud, re_ice=grid%re_ice, re_snow=grid%re_snow & ! G. Thompson
L00413       &        , has_reqc=grid%has_reqc, has_reqi=grid%has_reqi, has_reqs=grid%has_reqs & ! G. Thompson
L00414       &        , PB=grid%pb                                                                              &
```

```
L00415       &        , F_ICE_PHY=grid%f_ice_phy,F_RAIN_PHY=grid%f_rain_phy       &
L00416       &        , F_QNC=F_QNC                                               &
L00417       &        , QNC_CURR=scalar(ims,kms,jms,P_QNC)                        &
L00418       &        , QV=moist(ims,kms,jms,P_QV), F_QV=F_QV                     &
L00419       &        , QC=moist(ims,kms,jms,P_QC), F_QC=F_QC                     &
L00420       &        , QR=moist(ims,kms,jms,P_QR), F_QR=F_QR                     &
L00421       &        , QI=moist(ims,kms,jms,P_QI), F_QI=F_QI                     &
L00422       &        , QI2=moist(ims,kms,jms,P_QI2), F_QI2=F_QI2                 & ! for P3
L00423       &        , QI3=moist(ims,kms,jms,P_QI3), F_QI3=F_QI3                 & ! for Jensen ISHMAEL
L00424       &        , QS=moist(ims,kms,jms,P_QS), F_QS=F_QS                     &
L00425       &        , QG=moist(ims,kms,jms,P_QG), F_QG=F_QG                     &
L00426       &        , QH=moist(ims,kms,jms,P_QH), F_QH=F_QH                     & ! goddardrad
L00427       &        , QNDROP=scalar(ims,kms,jms,P_QNDROP), F_QNDROP=F_QNDROP    &
L00428       &        ,QNIFA=scalar(ims,kms,jms,P_QNIFA),F_QNIFA=F_QNIFA          & !Trude
L00429       &        ,QNWFA=scalar(ims,kms,jms,P_QNWFA),F_QNWFA=F_QNWFA          & !Trude
L00430       &        ,QNBCA=scalar(ims,kms,jms,P_QNBCA)                          &
L00431       &        ,wif_input_opt=config_flags%wif_input_opt                   &
L00432       &        ,qc_tot=grid%qc_tot, qi_tot=grid%qi_tot                     & ! Solar diag
L00433       &        ,ACSWUPT=grid%acswupt     ,ACSWUPTC=grid%acswuptc           &
L00434       &        ,ACSWDNT=grid%acswdnt     ,ACSWDNTC=grid%acswdntc           &
L00435       &        ,ACSWUPB=grid%acswupb     ,ACSWUPBC=grid%acswupbc           &
L00436       &        ,ACSWDNB=grid%acswdnb     ,ACSWDNBC=grid%acswdnbc           &
L00437       &        ,ACLWUPT=grid%aclwupt     ,ACLWUPTC=grid%aclwuptc           &
L00438       &        ,ACLWDNT=grid%aclwdnt     ,ACLWDNTC=grid%aclwdntc           &
L00439       &        ,ACLWUPB=grid%aclwupb     ,ACLWUPBC=grid%aclwupbc           &
L00440       &        ,ACLWDNB=grid%aclwdnb     ,ACLWDNBC=grid%aclwdnbc           &
L00441       &        ,SWUPT=grid%swupt,SWUPTC=grid%swuptc,SWUPTCLN=grid%swuptcln &
L00442       &        ,SWDNT=grid%swdnt,SWDNTC=grid%swdntc,SWDNTCLN=grid%swdntcln &
L00443       &        ,SWUPB=grid%swupb,SWUPBC=grid%swupbc,SWUPBCLN=grid%swupbcln &
L00444       &        ,SWDNB=grid%swdnb,SWDNBC=grid%swdnbc,SWDNBCLN=grid%swdnbcln &
L00445       &        ,LWUPT=grid%lwupt,LWUPTC=grid%lwuptc,LWUPTCLN=grid%lwuptcln &
L00446       &        ,LWDNT=grid%lwdnt,LWDNTC=grid%lwdntc,LWDNTCLN=grid%lwdntcln &
L00447       &        ,LWUPB=grid%lwupb,LWUPBC=grid%lwupbc,LWUPBCLN=grid%lwupbcln &
L00448       &        ,LWDNB=grid%lwdnb,LWDNBC=grid%lwdnbc,LWDNBCLN=grid%lwdnbcln &
L00449       &        ,LWCF=grid%lwcf                                                 &
L00450       &        ,SWCF=grid%swcf                                                 &
L00451       &        ,OLR=grid%olr                                                   &
L00452       &        ,AERODM=grid%aerodm, PINA=grid%pina, AODTOT=grid%aodtot          &
L00453       &        ,OZMIXM=grid%ozmixm, PIN=grid%pin                               &
L00454       &        ,M_PS_1=grid%m_ps_1, M_PS_2=grid%m_ps_2, AEROSOLC_1=grid%aerosolc_1        &
L00455       &        ,AEROSOLC_2=grid%aerosolc_2, M_HYBI0=grid%m_hybi                  &
L00456       &        ,ABSTOT=grid%abstot, ABSNXT=grid%absnxt, EMSTOT=grid%emstot              &
L00457       &        ,RADTACTTIME=grid%radtacttime                                    &
L00458       &        ,ICLOUD_CU=config_flags%ICLOUD_CU                                &
L00459       &        ,QC_CU=grid%QC_CU , QI_CU=grid%QI_CU                             &
L00460       &        ,CALC_CLEAN_ATM_DIAG=config_flags%calc_clean_atm_diag             &
L00461  #if (WRF_CHEM == 1)
L00462       &        ,AER_RA_FEEDBACK=config_flags%aer_ra_feedback                    &
L00463       &        ,PM2_5_DRY=grid%pm2_5_dry, PM2_5_WATER=grid%pm2_5_water          &
L00464       &        ,PM2_5_DRY_EC=grid%pm2_5_dry_ec                                  &
L00465       &        ,TAUAER300=grid%tauaer1, TAUAER400=grid%tauaer2 & ! jcb
L00466       &        ,TAUAER600=grid%tauaer3, TAUAER999=grid%tauaer4 & ! jcb
L00467       &        ,GAER300=grid%gaer1, GAER400=grid%gaer2, GAER600=grid%gaer3, GAER999=grid%gaer4 & ! jcb
L00468       &        ,WAER300=grid%waer1, WAER400=grid%waer2, WAER600=grid%waer3, WAER999=grid%waer4 & ! jcb
L00469       &        ,TAUAERlw1  =grid%tauaerlw1,  TAUAERlw2=grid%tauaerlw2    &
L00470       &        ,TAUAERlw3  =grid%tauaerlw3,  TAUAERlw4=grid%tauaerlw4    &
L00471       &        ,TAUAERlw5  =grid%tauaerlw5,  TAUAERlw6=grid%tauaerlw6    &
L00472       &        ,TAUAERlw7  =grid%tauaerlw7,  TAUAERlw8=grid%tauaerlw8    &
L00473       &        ,TAUAERlw9  =grid%tauaerlw9,  TAUAERlw10=grid%tauaerlw10   &
L00474       &        ,TAUAERlw11 =grid%tauaerlw11, TAUAERlw12=grid%tauaerlw12   &
L00475       &        ,TAUAERlw13 =grid%tauaerlw13, TAUAERlw14=grid%tauaerlw14   &
L00476       &        ,TAUAERlw15 =grid%tauaerlw15, TAUAERlw16=grid%tauaerlw16   &
L00477       &        ,progn=config_flags%progn                                        &
L00478  #endif
L00479       &         ,slope_rad=config_flags%slope_rad,topo_shading=config_flags%topo_shading     &
L00480       &         ,shadowmask=grid%shadowmask,ghg_input=config_flags%ghg_input   &
L00481       &         ,ht=grid%ht,dx=grid%dx,dy=grid%dy,dx2d=grid%dx2d,area2d=grid%area2d &
L00482       &         ,diffuse_frac=grid%diffuse_frac &
L00483       &         ,obscur=grid%ECOBSC, mask=grid%ECMASK                          &
L00484       &         ,elat_track=grid%elat_track,elon_track=grid%elon_track         &
L00485       &         ,SW_ECLIPSE=config_flags%ra_sw_eclipse                         &
L00486       &         ,IS_CAMMGMP_USED = grid%is_CAMMGMP_used    &
L00487       &         ,cw_rad=grid%cw_rad                  &
L00488       &         ,shcu_physics=config_flags%shcu_physics                        &
L00489       &         ,MP_PHYSICS=CONFIG_FLAGS%MP_PHYSICS        &
L00490       &         ,EFCG=grid%EFCG,EFCS=grid%EFCS,EFIG=grid%EFIG &
L00491       &         ,EFIS=grid%EFIS,EFSG=grid%EFSG,aercu_opt=config_flags%aercu_opt &
L00492       &         ,EFSS=grid%EFSS,QS_CU=grid%QS_CU &
L00493       &         ,feedback_is_ready=feedback_is_ready   &
L00494  #if ( WRF_CMAQ == 1)
L00495       &         ,mass_ws_i=grid%mass_ws_i              &
L00496       &         ,mass_ws_j=grid%mass_ws_j              &
L00497       &         ,mass_ws_k=grid%mass_ws_k              &
L00498       &         ,mass_in_i=grid%mass_in_i              &
L00499       &         ,mass_in_j=grid%mass_in_j              &
L00500       &         ,mass_in_k=grid%mass_in_k              &
L00501       &         ,mass_ec_i=grid%mass_ec_i              &
L00502       &         ,mass_ec_j=grid%mass_ec_j              &
L00503       &         ,mass_ec_k=grid%mass_ec_k              &
L00504       &         ,mass_ss_i=grid%mass_ss_i              &
L00505       &         ,mass_ss_j=grid%mass_ss_j              &
L00506       &         ,mass_ss_k=grid%mass_ss_k              &
L00507       &         ,mass_h2o_i=grid%mass_h2o_i            &
L00508       &         ,mass_h2o_j=grid%mass_h2o_j            &
L00509       &         ,mass_h2o_k=grid%mass_h2o_k            &
L00510       &         ,dgn_i=grid%dgn_i                      &
L00511       &         ,dgn_j=grid%dgn_j                      &
L00512       &         ,dgn_k=grid%dgn_k                      &
L00513       &         ,sig_i=grid%sig_i                      &
L00514       &         ,sig_j=grid%sig_j                      &
L00515       &         ,sig_k=grid%sig_k                      &
L00516       &         ,sw_gtauxar_01=grid%sw_gtauxar_01      &
L00517       &         ,sw_gtauxar_02=grid%sw_gtauxar_02      &
L00518       &         ,sw_gtauxar_03=grid%sw_gtauxar_03      &
L00519       &         ,sw_gtauxar_04=grid%sw_gtauxar_04      &
L00520       &         ,sw_gtauxar_05=grid%sw_gtauxar_05      &
L00521       &         ,sw_ttauxar_01=grid%sw_ttauxar_01      &
L00522       &         ,sw_ttauxar_02=grid%sw_ttauxar_02      &
L00523       &         ,sw_ttauxar_03=grid%sw_ttauxar_03      &
```

```
L00524       &         ,sw_ttauxar_04=grid%sw_ttauxar_04      &
L00525       &         ,sw_ttauxar_05=grid%sw_ttauxar_05      &
L00526       &         ,sw_asy_fac_01=grid%sw_asy_fac_01      &
L00527       &         ,sw_asy_fac_02=grid%sw_asy_fac_02      &
L00528       &         ,sw_asy_fac_03=grid%sw_asy_fac_03      &
L00529       &         ,sw_asy_fac_04=grid%sw_asy_fac_04      &
L00530       &         ,sw_asy_fac_05=grid%sw_asy_fac_05      &
L00531       &         ,sw_ssa_01=grid%sw_ssa_01              &
L00532       &         ,sw_ssa_02=grid%sw_ssa_02              &
L00533       &         ,sw_ssa_03=grid%sw_ssa_03              &
L00534       &         ,sw_ssa_04=grid%sw_ssa_04              &
L00535       &         ,sw_ssa_05=grid%sw_ssa_05              &
L00536       &         ,ozone=grid%ozone                      &
L00537       &         ,sw_zbbcddir=grid%sw_zbbcddir          &
L00538       &         ,sw_dirdflux=grid%sw_dirdflux          &
L00539       &         ,sw_difdflux=grid%sw_difdflux          &
L00540  #endif
L00541                                                        )
L00542
L00543  BENCH_END(rad_driver_tim)
```

讲解：radiation_driver 实参全景（274 行），分拣要点：①**NPHS=1**——辐射的累积输出频率恒 1（辐射自带 radt 门控，这里不设频率）；②**P=grid%p_hyd**——物理气压取静水压分量（对照 phy_prep）；③ 水物质按 P_QV/P_QC…直接切片 moist(ims,kms,jms,P_XX) 传入并配 F_XX 在场标志——**P3 方案的 QI2/QI3、Ishmael 的 QI3 也在列**；④ 气溶胶三通道全接通：aer_opt/aerod(P_ocarbon:P_upperaer 切片——**aerod 是 Registry 数组的连续切片直接当入参**)、aod550 系（code_60）、QNWFA/QNIFA/QNBCA（aer_opt=3）；⑤WRF-Solar 全家（swddir/dni/dif、Gx/Bx/gg/bb 插值状态、FARMS 的 swdown2 系、SPP 的 pert3d 七切片）；⑥WRF_CHEM 的 300/400/600/999 四波长气溶胶与 tauaerlw1..16；⑦CuP 的 cubot/cutop/cldfra_cup（积云云量给辐射）、EFCG/EFCS 五套有效半径系数（MSKF 交互）；⑧CLDT/ZNU（CLWRF 云顶诊断）、obscur/mask（日食）、AC 系 16 组累积通量、RADTACTTIME（自适应 dt）。下游细节见 `code_20_module_radiation_driver.md`。

六、陆面段：surface_driver

```
!file: dyn_em/module_first_rk_step_part1.F    L545-L1150
L00545  !******** Surface driver
L00546  ! surface
L00547
L00548  BENCH_START(surf_driver_tim)
L00549
L00550  !gmm halo of wtd and riverflow for leafhydro
L00551  #ifdef DM_PARALLEL
L00552    IF ( ( config_flags%sf_surface_physics.eq.NOAHMPSCHEME ) .and. (config_flags%opt_runsub.eq.5 ) ) THEN
L00553         IF ( mod(grid%itimestep,grid%STEPWTD).eq.0 )  THEN
L00554  #     include "HALO_EM_HYDRO_NOAHMP.inc"
L00555         ENDIF
L00556    ENDIF
L00557  #endif
L00558
L00559  !-----------------------------------------------------------------
L00560  ! urban related variable are added to arguments of surface_driver
L00561  !-----------------------------------------------------------------
L00562        num_roof_layers = grid%num_soil_layers !urban
L00563        num_wall_layers = grid%num_soil_layers !urban
L00564        num_road_layers = grid%num_soil_layers !urban
L00565        CALL nl_get_iswater(grid%id, iswater)
L00566        CALL nl_get_isurban(grid%id, isurban)
L00567        call nl_get_mminlu(grid%id, mminlu)
L00568
L00569  #ifdef DM_PARALLEL
L00570  #     include "HALO_PWP.inc"
L00571  #endif
L00572
L00573        CALL wrf_debug ( 200 , ' call surface_driver' )
L00574
L00575        if( grid%num_nests .lt. 1 )then
L00576            HYDRO_dt = 0
L00577        else
L00578            HYDRO_dt = -1
L00579        endif
L00580
L00581  !  Ensure WRF_Chem is compiled if bio_emiss_opt is tested, and add flag to read MEGANv2.1
L00582  !  file if bio_emiss_opt is set properly (even if chem_opt == 0).
L00583  !  Also explicitly set do_bioe to .false. if chem_opt <= 0.   BJG 3/28/19
L00584  #if( WRF_USE_CLM == 1) && (WRF_CHEM == 1)
L00585        if( config_flags%bio_emiss_opt == MEGAN2_CLM ) then
L00586          do_meganfile = .true.
L00587          if(config_flags%chem_opt > 0) then !czhao +++++
L00588            do_bioe = grid%itimestep == 1 .or. mod( grid%itimestep,grid%stepbioe ) == 0
L00589          else
L00590            do_bioe = .false.
L00591          endif
L00592        else
L00593          do_bioe = .false.
L00594          do_meganfile = .false.
L00595        endif !czhao-----
L00596  #else
L00597        do_bioe = .false.
L00598        do_meganfile = .false.
L00599  #endif
L00600
L00601
L00602
L00603        CALL surface_driver(                                              &
```

```
L00604       &        HYDRO_dt=HYDRO_dt,  sfcheadrt=grid%sfcheadrt,              &
L00605       &        INFXSRT=grid%INFXSRT,  soldrain=grid%soldrain,             &
L00606       &        qtiledrain=grid%qtiledrain,ZWATBLE2D=grid%ZWATBLE2D,       & ! tile drainage for WRF-Hydro
L00607       &         ACGRDFLX=grid%acgrdflx  ,ACHFX=grid%achfx        ,ACLHF=grid%aclhf        &
L00608       &        ,ACSNOM=grid%acsnom      ,ACSNOW=grid%acsnow      ,AKHS=grid%akhs          &
L00609       &        ,AKMS=grid%akms          ,ALBBCK=grid%albbck      ,ALBEDO=grid%albedo      &
L00610       &        ,EMBCK=grid%embck                                                          &
L00611       &        ,BR=grid%br              ,CANWAT=grid%canwat      ,CHKLOWQ=chklowq    &
L00612       &        ,CT=grid%ct              ,DT=grid%dt         ,DX=grid%dx             &
L00613       &        ,DX2D=grid%dx2d          ,AREA2d=grid%area2d                         &
L00614       &        ,DZ8W=dz8w          ,DZS=grid%dzs            ,FLHC=grid%flhc          &
L00615       &        ,FM=grid%fm         ,FHH=grid%fh                                     &
L00616       &        ,FLQC=grid%flqc          ,GLW=grid%glw            ,GRDFLX=grid%grdflx      &
L00617       &        ,GSW=grid%gsw    ,SWDOWN=grid%swdown        ,GZ1OZ0=gz1oz0      ,HFX=grid%hfx              &
L00618       &        ,HT=grid%ht              ,IFSNOW=config_flags%ifsnow      ,ISFFLX=config_flags%isfflx      &
L00619       &        ,FRACTIONAL_SEAICE=config_flags%fractional_seaice          &
L00620       &        ,SEAICE_ALBEDO_OPT=config_flags%seaice_albedo_opt          &
L00621       &        ,SEAICE_ALBEDO_DEFAULT=config_flags%seaice_albedo_default   &
L00622       &        ,SEAICE_THICKNESS_OPT=config_flags%seaice_thickness_opt    &
L00623       &        ,SEAICE_THICKNESS_DEFAULT=config_flags%seaice_thickness_default     &
L00624       &        ,SEAICE_SNOWDEPTH_OPT=config_flags%seaice_snowdepth_opt    &
L00625       &        ,SEAICE_SNOWDEPTH_MAX=config_flags%seaice_snowdepth_max    &
L00626       &        ,SEAICE_SNOWDEPTH_MIN=config_flags%seaice_snowdepth_min    &
L00627       &        ,TICE2TSK_IF2COLD=config_flags%tice2tsk_if2cold            &
L00628       &        ,IFNDALBSI=grid%ifndalbsi, IFNDICEDEPTH=grid%ifndicedepth   &
L00629  #if ( WRF_CHEM == 1 )
L00630                ,NE_AREA=grid%ne_area,E_BIO=grid%e_bio &
L00631  #endif
L00632                ,IFNDSNOWSI=grid%ifndsnowsi, DO_BIOE=do_bioe, DO_MEGANFILE=do_meganfile          &
L00633       &        ,ISLTYP=grid%isltyp      ,ITIMESTEP=grid%itimestep, JULIAN_IN=grid%julian                &
L00634       &        ,IRRIGATION=grid%irrigation,SF_SURF_IRR_SCHEME=config_flags%sf_surf_irr_scheme  &
L00635       &        ,IRR_DAILY_AMOUNT=config_flags%irr_daily_amount                                                  &
L00636       &        ,IRR_START_HOUR=config_flags%irr_start_hour,IRR_NUM_HOURS=config_flags%irr_num_hours             &
L00637       &        ,IRR_START_JULIANDAY=config_flags%irr_start_julianday                                            &
L00638       &        ,IRR_END_JULIANDAY=config_flags%irr_end_julianday                                                &
L00639       &        ,IRR_FREQ=config_flags%irr_freq,IRR_PH=config_flags%irr_ph,IRR_RAND_FIELD=grid%irr_rand_field  &
L00640       &        ,IVGTYP=grid%ivgtyp      ,LH=grid%lh                ,LOWLYR=grid%lowlyr      &
L00641       &        ,MAVAIL=grid%mavail      ,NUM_SOIL_LAYERS=config_flags%num_soil_layers       &
L00642       &        ,P8W=grid%p_hyd_w            ,PBLH=grid%pblh          ,PI_PHY=pi_phy      &
L00643       &        ,PSFC=grid%psfc          ,PSHLTR=grid%pshltr      ,PSIH=psih          &
L00644       &        ,BLDT=grid%bldt     ,CURR_SECS=curr_secs, ADAPT_STEP_FLAG=adapt_step_flag  &
L00645       &        ,BLDTACTTIME=grid%bldtacttime                                    &
L00646       &        ,PSIM=psim          ,P_PHY=grid%p_hyd        ,Q10=grid%q10           &
L00647       &        ,Q2=grid%q2              ,QFX=grid%qfx            ,QSFC=grid%qsfc          &
L00648       &        ,QSHLTR=grid%qshltr      ,QZ0=grid%qz0            ,RAINCV=grid%raincv      &
L00649       &        ,RA_LW_PHYSICS=config_flags%ra_lw_physics         ,RHO=grid%rho            &
L00650       &        ,RMOL=grid%rmol          ,SFCEVP=grid%sfcevp      ,SFCEXC=grid%sfcexc      &
L00651       &        ,SFCRUNOFF=grid%sfcrunoff,ACRUNOFF=grid%ACRUNOFF                 &
L00652       &        ,opt_thcnd=config_flags%opt_thcnd                                &
L00653       &        ,SF_SFCLAY_PHYSICS=config_flags%sf_sfclay_physics                &
L00654       &        ,BL_PBL_PHYSICS=config_flags%bl_pbl_physics                      &
L00655       &        ,SF_SURFACE_PHYSICS=config_flags%sf_surface_physics  ,SH2O=grid%sh2o       &
L00656       &        ,SF_MYNN_SFCFLUX_LAND=config_flags%sf_mynn_sfcflux_land          &
L00657       &        ,SF_MYNN_SFCFLUX_WATER=config_flags%sf_mynn_sfcflux_water        &
L00658       &        ,SHDMAX=grid%shdmax      ,SHDMIN=grid%shdmin      ,SMOIS=grid%smois        &
L00659       &        ,SHDAVG=grid%shdavg                                              &
L00660       &        ,SMSTAV=grid%smstav      ,SMSTOT=grid%smstot      ,SNOALB=grid%snoalb      &
L00661       &        ,SNOW=grid%snow          ,SNOWC=grid%snowc        ,SNOWH=grid%snowh        &
L00662       &        ,SMCREL=grid%smcrel                                              &
L00663       &        ,SST=grid%sst            ,SST_INPUT=grid%sst_input,SST_UPDATE=grid%sst_update                  &
L00664       &        ,SSTSK=grid%sstsk        ,DTW=grid%dtw            ,SST_SKIN=grid%sst_skin  &
L00665       &        ,SCM_FORCE_SKINTEMP=grid%scm_force_skintemp                      &
L00666       &        ,SCM_FORCE_FLUX=grid%scm_force_flux                              &
L00667       &        ,STEPBL=grid%stepbl      ,TH10=grid%th10          ,TH2=grid%th2            &
L00668       &        ,THZ0=grid%thz0          ,TH_PHY=th_phy      ,TKE_PBL=grid%tke_pbl    &
L00669       &        ,TMN=grid%tmn            ,TSHLTR=grid%tshltr      ,TSK=grid%tsk            &
L00670       &        ,TYR=grid%tyr            ,TYRA=grid%tyra          ,TDLY=grid%tdly          &
L00671       &        ,TLAG=grid%tlag          ,LAGDAY=config_flags%lagday      ,NYEAR=grid%nyear       &
L00672       &        ,NDAY=grid%nday          ,TMN_UPDATE=grid%tmn_update      ,YR=yr           &
L00673       &        ,TSLB=grid%tslb          ,T_PHY=t_phy        ,U10=grid%u10            &
L00674       &        ,URATX=grid%uratx        ,VRATX=grid%vratx   ,TRATX=grid%tratx        &
L00675       &        ,UDRUNOFF=grid%udrunoff  ,UST=grid%ust       ,UZ0=grid%uz0            &
L00676       &        ,U_FRAME=grid%u_frame    ,U_PHY=grid%u_phy   ,V10=grid%v10            &
L00677       &        ,U10E=grid%u10e          ,V10E=grid%v10e                          &
L00678       &        ,UOCE=grid%uoce          ,VOCE=grid%voce                          &
L00679       &        ,VEGFRA=grid%vegfra      ,VZ0=grid%vz0       ,V_FRAME=grid%v_frame    &
L00680       &        ,V_PHY=grid%v_phy        ,WARM_RAIN=grid%warm_rain                &
L00681       &        ,WSPD=grid%wspd          ,XICE=grid%xice     ,XLAND=grid%xland        &
L00682       &        ,MAX_EDOM=grid%num_ext_model_couple_dom      ,CPLMASK=grid%cplmask    &
L00683       &        ,Z0=grid%z0              ,Z=grid%z       ,ZNT=grid%znt            &
L00684       &        ,ZS=grid%zs              ,ALBSI=grid%albsi , ICEDEPTH=grid%icedepth   &
L00685       &        ,SNOWSI=grid%snowsi                                              &
L00686       &        ,XICEM=grid%xicem   ,ISICE=grid%landuse_isice                    &
L00687       &        ,USTM=grid%ustm          ,CK=grid%ck         ,CKA=grid%cka            &
L00688       &                                 ,CD=grid%cd         ,CDA=grid%cda            &
L00689       &        ,ISFTCFLX=config_flags%isftcflx, IZ0TLND=config_flags%iz0tlnd    &
L00690       &        ,SF_OCEAN_PHYSICS=config_flags%sf_ocean_physics                  &
L00691       &        ,OML_HML0=config_flags%oml_hml0 ,OML_GAMMA=config_flags%oml_gamma     &
L00692       &        ,TML=grid%tml, T0ML=grid%t0ml, HML=grid%hml, H0ML=grid%h0ml      &
L00693       &        ,HUML=grid%huml, HVML=grid%hvml, F=grid%f                        &
L00694       &        ,TMOML=grid%TMOML,ISWATER=iswater                                &
L00695       &        ,OML_RELAXATION_TIME=grid%OML_RELAXATION_TIME                    &
L00696       &        ,shalwater_z0=config_flags%shalwater_z0                          &
L00697       &        ,water_depth=grid%water_depth                                    &
L00698       &        ,shalwater_depth=config_flags%shalwater_depth                    &
L00699       &        ,lakedepth2d=grid%lakedepth2d,    savedtke12d=grid%savedtke12d   &
L00700       &        ,snowdp2d=grid%snowdp2d,          h2osno2d=grid%h2osno2d         & !lake
L00701       &        ,snl2d=grid%snl2d,                t_grnd2d=grid%t_grnd2d         &
L00702       &        ,t_lake3d=grid%t_lake3d,          lake_icefrac3d=grid%lake_icefrac3d  & !lake
L00703       &        ,z_lake3d=grid%z_lake3d,          dz_lake3d=grid%dz_lake3d       &
L00704       &        ,t_soisno3d=grid%t_soisno3d,      h2osoi_ice3d=grid%h2osoi_ice3d      & !lake
L00705       &        ,h2osoi_liq3d=grid%h2osoi_liq3d,  h2osoi_vol3d=grid%h2osoi_vol3d      &
L00706       &        ,z3d=grid%z3d,                    dz3d=grid%dz3d                 & !lake
L00707       &        ,zi3d=grid%zi3d,                  watsat3d=grid%watsat3d         &
L00708       &        ,csol3d=grid%csol3d,              tkmg3d=grid%tkmg3d             & !lake
L00709       &        ,tkdry3d=grid%tkdry3d,            tksatu3d=grid%tksatu3d         &
L00710       &        ,LakeModel=grid%sf_lake_physics,  lake_min_elev=grid%lake_min_elev    & !lake
L00711  #if ( EM_CORE == 1)
L00712       &        ,LakeMask=grid%LakeMask                                          & !lake
```

```
L00713       &        ,restart_flag=restart_flag                                          & !flag showing if is a restart
↪  timestep
L00714  ! TOFD for KIM gwdo scheme
L00715       &        ,kim_tofd=config_flags%kim_tofd                                     &
L00716       &        ,tofd_factor=config_flags%tofd_factor                               &
L00717       &        ,VAR2D=grid%var2d                                                   &
L00718  #endif
L00719  ! CLM Varaibles
L00720       &        ,NUMC=grid%numc,NUMP=grid%nump,SABV=grid%sabv,SABG=grid%sabg,       &
L00721       &         LWUP=grid%lwup,SNL=grid%snl,                                       &
L00722       &         HISTORY_INTERVAL=config_flags%history_interval ,                   & !ylu add hist inverval for
↪  accumulation T max/min
L00723       &         SNOWDP=grid%snowdp, WTC=grid%wtc,WTP=grid%wtp, H2OSNO=grid%h2osno,  &
L00724       &         T_GRND=grid%t_grnd,T_VEG=grid%t_veg,                               &
L00725       &         H2OCAN=grid%h2ocan, H2OCAN_COL=grid%h2ocan_col,T2M_MAX=grid%t2m_max, &
L00726       &         T2M_MIN=grid%t2m_min,T2CLM=grid%t2clm,                             &
L00727       &         T_REF2M=grid%t_ref2m,H2OSOI_LIQ_S1=grid%h2osoi_liq_s1,             &
L00728       &         H2OSOI_LIQ_S2=grid%h2osoi_liq_s2,                                  &
L00729       &         H2OSOI_LIQ_S3=grid%h2osoi_liq_s3,H2OSOI_LIQ_S4=grid%h2osoi_liq_s4,  &
L00730       &         H2OSOI_LIQ_S5=grid%h2osoi_liq_s5,                                  &
L00731       &         H2OSOI_LIQ1=grid%h2osoi_liq1,H2OSOI_LIQ2=grid%h2osoi_liq2,         &
L00732       &         H2OSOI_LIQ3=grid%h2osoi_liq3,H2OSOI_LIQ4=grid%h2osoi_liq4,         &
L00733       &         H2OSOI_LIQ5=grid%h2osoi_liq5,H2OSOI_LIQ6=grid%h2osoi_liq6,         &
L00734       &         H2OSOI_LIQ7=grid%h2osoi_liq7,H2OSOI_LIQ8=grid%h2osoi_liq8,         &
L00735       &         H2OSOI_LIQ9=grid%h2osoi_liq9, H2OSOI_LIQ10=grid%h2osoi_liq10,      &
L00736       &         H2OSOI_ICE_S1=grid%h2osoi_ice_s1,H2OSOI_ICE_S2=grid%h2osoi_ice_s2,  &
L00737       &         H2OSOI_ICE_S3=grid%h2osoi_ice_s3, H2OSOI_ICE_S4=grid%h2osoi_ice_s4, &
L00738       &         H2OSOI_ICE_S5=grid%h2osoi_ice_s5,                                  &
L00739       &         H2OSOI_ICE1=grid%h2osoi_ice1, H2OSOI_ICE2=grid%h2osoi_ice2,        &
L00740       &         H2OSOI_ICE3=grid%h2osoi_ice3,H2OSOI_ICE4=grid%h2osoi_ice4,         &
L00741       &         H2OSOI_ICE5=grid%h2osoi_ice5, H2OSOI_ICE6=grid%h2osoi_ice6,        &
L00742       &         H2OSOI_ICE7=grid%h2osoi_ice7,H2OSOI_ICE8=grid%h2osoi_ice8,         &
L00743       &         H2OSOI_ICE9=grid%h2osoi_ice9,H2OSOI_ICE10=grid%h2osoi_ice10,       &
L00744       &         T_SOISNO_S1=grid%t_soisno_s1,T_SOISNO_S2=grid%t_soisno_s2,         &
L00745       &         T_SOISNO_S3=grid%t_soisno_s3,T_SOISNO_S4=grid%t_soisno_s4,         &
L00746       &         T_SOISNO_S5=grid%t_soisno_s5,T_SOISNO1=grid%t_soisno1,             &
L00747       &         T_SOISNO2=grid%t_soisno2,T_SOISNO3=grid%t_soisno3,                 &
L00748       &         T_SOISNO4=grid%t_soisno4,T_SOISNO5=grid%t_soisno5,                 &
L00749       &         T_SOISNO6=grid%t_soisno6,T_SOISNO7=grid%t_soisno7,                 &
L00750       &         T_SOISNO8=grid%t_soisno8,T_SOISNO9=grid%t_soisno9,                 &
L00751       &         T_SOISNO10=grid%t_soisno10,DZSNOW1=grid%dzsnow1,DZSNOW2=grid%dzsnow2,&
L00752       &         DZSNOW3=grid%dzsnow3,DZSNOW4=grid%dzsnow4, DZSNOW5=grid%dzsnow5,    &
L00753       &         SNOWRDS1=grid%snowrds1,SNOWRDS2=grid%snowrds2,                      &
L00754       &         SNOWRDS3=grid%snowrds3 ,SNOWRDS4=grid%snowrds4,                     &
L00755       &         SNOWRDS5=grid%snowrds5,                                            &
L00756       &         T_LAKE1=grid%t_lake1,T_LAKE2=grid%t_lake2,T_LAKE3=grid%t_lake3,    &
L00757       &         T_LAKE4=grid%t_lake4,                                              &
L00758       &         T_LAKE5=grid%t_lake5,T_LAKE6=grid%t_lake6, T_LAKE7=grid%t_lake7,   &
L00759       &         T_LAKE8=grid%t_lake8, T_LAKE9=grid%t_lake9,T_LAKE10=grid%t_lake10,  &
L00760       &         H2OSOI_VOL1=grid%h2osoi_vol1,H2OSOI_VOL2=grid%h2osoi_vol2,         &
L00761       &         H2OSOI_VOL3=grid%h2osoi_vol3,H2OSOI_VOL4=grid%h2osoi_vol4,         &
L00762       &         H2OSOI_VOL5=grid%h2osoi_vol5,                                      &
L00763       &         H2OSOI_VOL6=grid%h2osoi_vol6,H2OSOI_VOL7=grid%h2osoi_vol7,         &
L00764       &         H2OSOI_VOL8=grid%h2osoi_vol8,                                      &
L00765       &         H2OSOI_VOL9=grid%h2osoi_vol9,H2OSOI_VOL10=grid%h2osoi_vol10,       &
L00766       &         MAXPATCH=config_flags%maxpatch,                                    &
L00767       &         INEST=grid%id,ALBEDOsubgrid=grid%ALBEDOsubgrid,                    &
L00768       &         LHsubgrid=grid%LHsubgrid,                                          &
L00769       &         HFXsubgrid=grid%HFXsubgrid,LWUPsubgrid=grid%LWUPsubgrid,           &
L00770       &         Q2subgrid=grid%Q2subgrid,SABVsubgrid=grid%SABVsubgrid,             &
L00771       &         SABGsubgrid=grid%SABGsubgrid,NRAsubgrid=grid%NRAsubgrid,           &
L00772       &         SWUPsubgrid=grid%SWUPsubgrid,LHsoi=grid%LHsoi,                     &
L00773       &         LHveg=grid%LHveg, LHtran=grid%LHtran                               &
L00774                 ,t_veg24=grid%t_veg24,           t_veg240=grid%t_veg240,           & !voce accum variables
L00775                 fsun24=grid%fsun24,              fsun240=grid%fsun240,             & !voce accum variables
L00776                 fsd24=grid%fsd24,                fsd240=grid%fsd240,               & !voce accum variables
L00777                 fsi24=grid%fsi24,                fsi240=grid%fsi240,               & !voce accum variables
L00778                 laip=grid%laip,                                                    & !voce accum variables
L00779                 pct_pft_input=grid%pct_pft_input,num_pft_input=grid%num_pft_clm,  &
L00780                 input_pft_flag=config_flags%input_pft                              &
L00781  ! end of CLM variables
L00782       &        ,SLOPE_RAD=config_flags%slope_rad,TOPO_SHADING=config_flags%topo_shading & ! solar
L00783       &        ,SHADOWMASK=grid%shadowmask,DIFFUSE_FRAC=grid%diffuse_frac              & ! solar
L00784       &        ,SLOPE=grid%slope, SLP_AZI=grid%slp_azi, SWNORM=grid%swnorm             & ! solar
L00785       &        ,DECLIN=grid%declin ,SOLCON=grid%solcon ,COSZEN=grid%coszen ,HRANG=grid%hrang    &
L00786       &        ,xlat_urb2d=grid%XLAT                                          & !I urban
L00787       &        ,NUM_ROOF_LAYERS=num_roof_layers                               & !I urban
L00788       &        ,NUM_WALL_LAYERS=num_wall_layers                               & !I urban
L00789       &        ,NUM_ROAD_LAYERS=num_road_layers                               &
L00790       &        ,DZR=grid%dzr ,DZB=grid%dzb ,DZG=grid%dzg                      & !I urban
L00791       &        ,TR_URB2D=grid%tr_urb2d ,TB_URB2D=grid%tb_urb2d                &
L00792       &        ,TG_URB2D=grid%tg_urb2d                                        & !H urban
L00793       &        ,TC_URB2D=grid%tc_urb2d ,QC_URB2D=grid%qc_urb2d                & !H urban
L00794       &        ,UC_URB2D=grid%uc_urb2d                                        & !H urban
L00795       &        ,XXXR_URB2D=grid%xxxr_urb2d                                    &
L00796       &        ,XXXB_URB2D=grid%xxxb_urb2d                                    & !H urban
L00797       &        ,XXXG_URB2D=grid%xxxg_urb2d                                    &
L00798       &        ,XXXC_URB2D=grid%xxxc_urb2d                                    & !H urban
L00799       &        ,CMCR_URB2D=grid%cmcr_urb2d,TGR_URB2D=grid%tgr_urb2d           & !H urban
L00800       &        ,TGRL_URB3D=grid%tgrl_urb3d,SMR_URB3D=grid%smr_urb3d           & !H urban
L00801       &        ,JULIAN=grid%julday, JULYR=grid%julyr                          & !I urban
L00802       &        ,DRELR_URB2D=grid%drelr_urb2d,DRELB_URB2D=grid%drelb_urb2d     & !H urban
L00803       &        ,DRELG_URB2D=grid%drelg_urb2d                                  & !H urban
L00804       &        ,FLXHUMR_URB2D=grid%flxhumr_urb2d                              & !H urban
L00805       &        ,FLXHUMB_URB2D=grid%flxhumb_urb2d                              & !H urban
L00806       &        ,FLXHUMG_URB2D=grid%flxhumg_urb2d                              & !H urban
L00807                 ! added by Chenghao Wang
L00808       &        ,TVG_URB2D=grid%tvg_urb2d                                      & !H urban
L00809       &        ,XXXVG_URB2D=grid%xxxvg_urb2d                                  & !H urban
L00810       &        ,TVGL_URB3D=grid%tvgl_urb3d                                    & !H urban
L00811       &        ,SMG_URB3D=grid%smg_urb3d                                      & !H urban
L00812       &        ,TT_URB2D=grid%tt_urb2d                                        & !H urban
L00813       &        ,CMCG_URB2D=grid%cmcg_urb2d                                    & !H urban
L00814       &        ,FLXHUMVG_URB2D=grid%flxhumvg_urb2d                            & !H urban
L00815       &        ,FLXHUMT_URB2D=grid%flxhumt_urb2d                              & !H urban
L00816       &        ,TRL_URB3D=grid%trl_urb3d    ,TBL_URB3D=grid%tbl_urb3d         & !H urban
L00817       &        ,TGL_URB3D=grid%tgl_urb3d                                      & !H urban
L00818       &        ,SH_URB2D=grid%sh_urb2d     ,LH_URB2D=grid%lh_urb2d            &
L00819       &        ,G_URB2D=grid%g_urb2d                                          & !H urban
```

```
L00820       &        ,RN_URB2D=grid%rn_urb2d     , TS_URB2D=grid%ts_urb2d        & !H urban
L00821       &        ,FRC_URB2D=grid%frc_urb2d                                   & !H urban
L00822       &        ,UTYPE_URB2D=grid%utype_urb2d                               & !H urban
L00823       &        ,SWDDIR=grid%swddir,SWDDIF=grid%swddif                      & !multi-layer urban _gl
L00824            ! Optional urban for BEP scheme
L00825       &        ,SF_URBAN_PHYSICS=config_flags%sf_urban_physics             &
L00826       &        ,num_urban_ndm = config_flags%num_urban_ndm                 & !multi-layer urban
L00827       &        ,urban_map_zrd = config_flags%urban_map_zrd                 & !multi-layer urban
L00828       &        ,urban_map_zwd = config_flags%urban_map_zwd                 & !multi-layer urban
L00829       &        ,urban_map_gd  = config_flags%urban_map_gd                  & !multi-layer urban
L00830       &        ,urban_map_zd  = config_flags%urban_map_zd                  & !multi-layer urban
L00831       &        ,urban_map_zdf = config_flags%urban_map_zdf                 & !multi-layer urban
L00832       &        ,urban_map_bd  = config_flags%urban_map_bd                  & !multi-layer urban
L00833       &        ,urban_map_wd  = config_flags%urban_map_wd                  & !multi-layer urban
L00834       &        ,urban_map_gbd = config_flags%urban_map_gbd                 & !multi-layer urban
L00835       &        ,urban_map_fbd = config_flags%urban_map_fbd                 & !multi-layer urban
L00836       &        ,urban_map_zgrd = config_flags%urban_map_zgrd               & !multi-layer urban
L00837       &        ,NUM_URBAN_HI=config_flags%num_urban_hi                     & !multi-layer urban
L00838       &        ,use_wudapt_lcz=config_flags%use_wudapt_lcz                 & !wudapt
L00839       &        ,slucm_distributed_drag=config_flags%slucm_distributed_drag & !SLUCM
L00840       &        ,TSK_RURAL=grid%tsk_rural                                   & !multi-layer urban
L00841       &        ,TRB_URB4D=grid%trb_urb4d,TW1_URB4D=grid%tw1_urb4d          & !multi-layer urban
L00842       &        ,TW2_URB4D=grid%tw2_urb4d,TGB_URB4D=grid%tgb_urb4d          & !multi-layer urban
L00843       &        ,TLEV_URB3D=grid%tlev_urb3d                                 & !multi-layer urban
L00844       &        ,QLEV_URB3D=grid%qlev_urb3d                                 & !multi-layer urban
L00845       &        ,TW1LEV_URB3D=grid%tw1lev_urb3d                             & !multi-layer urban
L00846       &        ,TW2LEV_URB3D=grid%tw2lev_urb3d                             & !multi-layer urban
L00847       &        ,TGLEV_URB3D=grid%tglev_urb3d                               & !multi-layer urban
L00848       &        ,TFLEV_URB3D=grid%tflev_urb3d                               & !multi-layer urban
L00849       &        ,SF_AC_URB3D=grid%sf_ac_urb3d                               & !multi-layer urban
L00850       &        ,LF_AC_URB3D=grid%lf_ac_urb3d                               & !multi-layer urban
L00851       &        ,CM_AC_URB3D=grid%cm_ac_urb3d                               & !multi-layer urban
L00852       &        ,SFVENT_URB3D=grid%sfvent_urb3d                             & !multi-layer urban
L00853       &        ,LFVENT_URB3D=grid%lfvent_urb3d                             & !multi-layer urban
L00854       &        ,SFWIN1_URB3D=grid%sfwin1_urb3d                             & !multi-layer urban
L00855       &        ,SFWIN2_URB3D=grid%sfwin2_urb3d                             & !multi-layer urban
L00856       &        ,SFW1_URB3D=grid%sfw1_urb3d,SFW2_URB3D=grid%sfw2_urb3d      & !multi-layer urban
L00857       &        ,SFR_URB3D=grid%sfr_urb3d,SFG_URB3D=grid%sfg_urb3d          & !multi-layer urban
L00858       &        ,EP_PV_URB3D=grid%ep_pv_urb3d                               & !GRZ
L00859       &        ,T_PV_URB3D=grid%t_pv_urb3d                                 & !GRZ
L00860       &        ,TRV_URB4D=grid%trv_urb4d                                   & !GRZ
L00861       &        ,QR_URB4D=grid%qr_urb4d                                     & !GRZ
L00862       &        ,QGR_URB3D=grid%qgr_urb3d                                   & !GRZ
L00863       &        ,TGR_URB3D=grid%tgr_urb3d                                   & !GRZ
L00864       &        ,DRAIN_URB4D=grid%drain_urb4d                               & !GRZ
L00865       &        ,DRAINGR_URB3D=grid%draingr_urb3d                           & !GRZ
L00866       &        ,SFRV_URB3D=grid%sfrv_urb3d                                 & !GRZ
L00867       &        ,LFRV_URB3D=grid%lfrv_urb3d                                 & !GRZ
L00868       &        ,DGR_URB3D=grid%dgr_urb3d                                   & !GRZ
L00869       &        ,DG_URB3D=grid%dg_urb3d                                     & !GRZ
L00870       &        ,LFR_URB3D=grid%lfr_urb3d                                   & !GRZ
L00871       &        ,LFG_URB3D=grid%lfg_urb3d                                   & !GRZ
L00872       &        ,LP_URB2D=grid%lp_urb2d,HI_URB2D=grid%hi_urb2d              & !multi-layer urban
L00873       &        ,LB_URB2D=grid%lb_urb2d,HGT_URB2D=grid%hgt_urb2d            & !multi-layer urban
L00874       &        ,MH_URB2D=grid%mh_urb2d,STDH_URB2D=grid%stdh_urb2d          & !SLUCM
L00875       &        ,LF_URB2D=grid%lf_urb2d                                     &
L00876       &        ,lf_urb2d_s=grid%lf_urb2d_s, z0_urb2d=grid%z0_urb2d         &
L00877       &        ,GMT=grid%gmt,XLAT=grid%xlat,XLONG=grid%xlong,JULDAY=grid%julday  &
L00878       &        ,distributed_ahe_opt=grid%distributed_ahe_opt, ahe=grid%ahe  & !For anthropogenic heat
L00879       &        ,A_U_BEP=grid%a_u_bep,A_V_BEP=grid%a_v_bep,A_T_BEP=grid%a_t_bep   &
L00880       &        ,A_Q_BEP=grid%a_q_bep                                       &
L00881       &        ,B_U_BEP=grid%b_u_bep,B_V_BEP=grid%b_v_bep,B_T_BEP=grid%b_t_bep   &
L00882       &        ,B_Q_BEP=grid%b_q_bep                                       &
L00883       &        ,SF_BEP=grid%sf_bep,VL_BEP=grid%vl_bep                      &
L00884       &        ,A_E_BEP=grid%a_e_bep,B_E_BEP=grid%b_e_bep,DLG_BEP=grid%dlg_bep   &
L00885       &        ,DL_U_BEP=grid%dl_u_bep                                     &
L00886       &        ,CMR_SFCDIF=grid%cmr_sfcdif, CHR_SFCDIF=grid%chr_sfcdif     & !I/O urban
L00887       &        ,CMC_SFCDIF=grid%cmc_sfcdif, CHC_SFCDIF=grid%chc_sfcdif     & !I/O urban
L00888       &        ,CMGR_SFCDIF=grid%cmgr_sfcdif, CHGR_SFCDIF=grid%chgr_sfcdif & !I/O urban
L00889            ! P-X LSM Variables
L00890       &        ,LANDUSEF=grid%landusef, SOILCTOP=grid%soilctop             &   ! P-X LSM
L00891       &        ,SOILCBOT=grid%soilcbot                                     &   ! P-X LSM
L00892       &        ,RA=grid%ra, RS=grid%rs, LAI=grid%lai, IMPERV=grid%imperv   &   ! P-X LSM
L00893       &        ,CANFRA=grid%canfra, NLCAT=grid%num_land_cat                &   ! P-X LSM
L00894       &        ,NSCAT=grid%num_soil_cat                                    &   ! P-X LSM
L00895       &        ,VEGF_PX=grid%vegf_px, SNOWNCV=grid%snowncv                 &   ! P-X LSM
L00896       &        ,ANAL_INTERVAL=config_flags%auxinput9_interval_s+config_flags%auxinput9_interval_m*60  &   ! P-X LSM
L00897       &        ,PXLSM_SMOIS_INIT=config_flags%pxlsm_smois_init             &   ! P-X LSM
L00898       &        ,PXLSM_SOIL_NUDGE=config_flags%pxlsm_soil_nudge             &   ! P-X LSM
L00899            ! SSiB LSM variables (fds 06/2010)
L00900       &        ,alswvisdir=grid%alswvisdir, alswvisdif=grid%alswvisdif     & !ssib
L00901       &        ,alswnirdir=grid%alswnirdir, alswnirdif=grid%alswnirdif     & !ssib
L00902       &        ,swvisdir=grid%swvisdir, swvisdif=grid%swvisdif             & !ssib
L00903       &        ,swnirdir=grid%swnirdir, swnirdif=grid%swnirdif             & !ssib
L00904       &        ,ssib_br=grid%ssib_br,  ssib_fm=grid%ssib_fm                & !ssib
L00905       &        ,ssib_fh=grid%ssib_fh,  ssib_cm=grid%ssib_cm                & !ssib
L00906       &        ,ssibxdd=grid%ssibxdd,  ssib_lhf=grid%ssib_lhf              & !ssib
L00907       &        ,ssib_shf=grid%ssib_shf, ssib_ghf=grid%ssib_ghf             & !ssib
L00908       &        ,ssib_egs=grid%ssib_egs, ssib_eci=grid%ssib_eci             & !ssib
L00909       &        ,ssib_ect=grid%ssib_ect, ssib_egi=grid%ssib_egi             & !ssib
L00910       &        ,ssib_egt=grid%ssib_egt, ssib_sdn=grid%ssib_sdn             & !ssib
L00911       &        ,ssib_sup=grid%ssib_sup, ssib_ldn=grid%ssib_ldn             & !ssib
L00912       &        ,ssib_lup=grid%ssib_lup, ssib_wat=grid%ssib_wat             & !ssib
L00913       &        ,ssib_shc=grid%ssib_shc, ssib_shg=grid%ssib_shg             & !ssib
L00914       &        ,ssib_lai=grid%ssib_lai, ssib_vcf=grid%ssib_vcf             & !ssib
L00915       &        ,ssib_z00=grid%ssib_z00, ssib_veg=grid%ssib_veg             & !ssib
L00916       &        ,cldfra=grid%cldfra                                         & !ssib
L00917       &        ,ISNOW=grid%isnow, SWE=grid%swe, SNOWDEN=grid%snowden       & !ssib snow
L00918       &        ,SNOWDEPTH=grid%snowdepth, TKAIR=grid%tkair                 & !ssib snow
L00919       &        ,DZO1=grid%dzo1, WO1=grid%wo1, TSSN1=grid%tssn1, TSSNO1=grid%tssno1     & !ssib snow
L00920       &        ,BWO1=grid%bwo1, BTO1=grid%bto1, CTO1=grid%cto1, FIO1=grid%fio1         & !ssib snow
L00921       &        ,FLO1=grid%flo1, BIO1=grid%bio1, BLO1=grid%blo1, HO1=grid%ho1           & !ssib snow
L00922       &        ,DZO2=grid%dzo2, WO2=grid%wo2, TSSN2=grid%tssn2, TSSNO2=grid%tssno2     & !ssib snow
L00923       &        ,BWO2=grid%bwo2, BTO2=grid%bto2, CTO2=grid%cto2, FIO2=grid%fio2         & !ssib snow
L00924       &        ,FLO2=grid%flo2, BIO2=grid%bio2, BLO2=grid%blo2, HO2=grid%ho2           & !ssib snow
L00925       &        ,DZO3=grid%dzo3, WO3=grid%wo3, TSSN3=grid%tssn3, TSSNO3=grid%tssno3     & !ssib snow
L00926       &        ,BWO3=grid%bwo3, BTO3=grid%bto3, CTO3=grid%cto3, FIO3=grid%fio3         & !ssib snow
L00927       &        ,FLO3=grid%flo3, BIO3=grid%bio3, BLO3=grid%blo3, HO3=grid%ho3           & !ssib snow
L00928       &        ,DZO4=grid%dzo4, WO4=grid%wo4, TSSN4=grid%tssn4, TSSNO4=grid%tssno4     & !ssib snow
```

```
L00929       &        ,BWO4=grid%bwo4, BTO4=grid%bto4, CTO4=grid%cto4, FIO4=grid%fio4         & !ssib snow
L00930       &        ,FLO4=grid%flo4, BIO4=grid%bio4, BLO4=grid%blo4, HO4=grid%ho4           & !ssib snow
L00931       &        ,RA_SW_PHYSICS=config_flags%ra_sw_physics                   & !ssib
L00932    !-------------------------------------------------------------------------
L00933               ! Optional PX LSM nudging
L00934       &        ,t2_ndg_old=grid%t2_ndg_old                &
L00935       &        ,q2_ndg_old=grid%q2_ndg_old                &
L00936       &        ,t2_ndg_new=grid%t2_ndg_new                &
L00937       &        ,q2_ndg_new=grid%q2_ndg_new                &
L00938       &        ,sn_ndg_old=grid%sn_ndg_old                &
L00939       &        ,sn_ndg_new=grid%sn_ndg_new                &
L00940       &        ,pxlsm_modis_veg=config_flags%pxlsm_modis_veg                  &
L00941       &        ,LAI_PX=grid%lai_px                                            &
L00942       &        ,WWLT_PX=grid%wwlt_px, WFC_PX=grid%wfc_px                      &
L00943       &        ,WSAT_PX=grid%wsat_px, CLAY_PX=grid%clay_px                    &
L00944       &        ,CSAND_PX=grid%csand_px, FMSAND_PX=grid%fmsand_px              &
L00945    ! for Noah-MP LSM
L00946       &        ,idveg=config_flags%dveg,         iopt_crs=config_flags%opt_crs &
L00947       &        ,iopt_btr=config_flags%opt_btr,iopt_runsub=config_flags%opt_runsub &
L00948       &        ,iopt_sfc=config_flags%opt_sfc,   iopt_frz=config_flags%opt_frz &
L00949       &        ,iopt_inf=config_flags%opt_inf,   iopt_rad=config_flags%opt_rad &
L00950       &        ,iopt_alb=config_flags%opt_alb,   iopt_snf=config_flags%opt_snf &
L00951       &        ,iopt_tbot=config_flags%opt_tbot, iopt_stc=config_flags%opt_stc &
L00952       &        ,iopt_gla=config_flags%opt_gla,   iopt_rsf=config_flags%opt_rsf &
L00953       &        ,iopt_soil=config_flags%opt_soil, iopt_pedo=config_flags%opt_pedo  &
L00954       &        ,iopt_crop=config_flags%opt_crop, iopt_irr=config_flags%opt_irr    &
L00955       &        ,iopt_irrm=config_flags%opt_irrm                               &
L00956       &        ,iopt_infdv=config_flags%opt_infdv,iopt_tdrn=config_flags%opt_tdrn &
L00957       &        ,soiltstep=config_flags%soiltstep                              &
L00958       &        ,iopt_runsrf=config_flags%opt_runsrf,iopt_tksno=config_flags%opt_tksno &
L00959       &        ,iopt_compact=config_flags%opt_compact, iopt_scf=config_flags%opt_scf  &
L00960       &        ,iopt_wetland=config_flags%opt_wetland                         &
L00961       &        , isnowxy=grid%isnowxy  ,    tvxy=grid%tvxy    ,    tgxy=grid%tgxy     &
L00962       &        ,canicexy=grid%canicexy ,canliqxy=grid%canliqxy,   eahxy=grid%eahxy    &
L00963       &        ,   tahxy=grid%tahxy    ,    cmxy=grid%cmxy    ,    chxy=grid%chxy     &
L00964       &        ,  fwetxy=grid%fwetxy   ,sneqvoxy=grid%sneqvoxy,alboldxy=grid%alboldxy &
L00965       &        , qsnowxy=grid%qsnowxy  ,qrainxy=grid%qrainxy  ,wslakexy=grid%wslakexy &
L00966       &        , zwtxy=grid%zwtxy                                             &
L00967       &        ,    waxy=grid%waxy     ,    wtxy=grid%wtxy    ,  tsnoxy=grid%tsnoxy   &
L00968       &        , zsnsoxy=grid%zsnsoxy  , snicexy=grid%snicexy , snliqxy=grid%snliqxy  &
L00969       &        ,lfmassxy=grid%lfmassxy ,rtmassxy=grid%rtmassxy,stmassxy=grid%stmassxy &
L00970       &        ,  woodxy=grid%woodxy   ,stblcpxy=grid%stblcpxy,fastcpxy=grid%fastcpxy &
L00971       &        , grainxy=grid%grainxy  ,   gddxy=grid%gddxy   ,   pgsxy=grid%pgsxy    &
L00972       &        , cropcat=grid%cropcat                                         &
L00973       &        ,planting=grid%planting , harvest=grid%harvest ,season_gdd=grid%season_gdd  &
L00974       &        , soilcomp=grid%soilcomp                                       &
L00975       &        , soilcl1=grid%soilcl1  , soilcl2=grid%soilcl2                 &
L00976       &        , soilcl3=grid%soilcl3  , soilcl4=grid%soilcl4                 &
L00977       &        ,  xsaixy=grid%xsaixy   , taussxy=grid%taussxy                 &
L00978       &        ,  t2mvxy=grid%t2mvxy   ,  t2mbxy=grid%t2mbxy                  &
L00979       &        ,  q2mvxy=grid%q2mvxy   ,  q2mbxy=grid%q2mbxy                  &
L00980       &        ,  tradxy=grid%tradxy   ,   neexy=grid%neexy   ,   gppxy=grid%gppxy    &
L00981       &        ,   nppxy=grid%nppxy    ,  fvegxy=grid%fvegxy  , runsfxy=grid%runsfxy  &
L00982       &        , runsbxy=grid%runsbxy  ,  ecanxy=grid%ecanxy  ,  edirxy=grid%edirxy   &
L00983       &        , etranxy=grid%etranxy  ,   fsaxy=grid%fsaxy   ,  firaxy=grid%firaxy   &
L00984       &        ,  aparxy=grid%aparxy   ,   psnxy=grid%psnxy   ,   savxy=grid%savxy    &
L00985       &        ,   sagxy=grid%sagxy    , rssunxy=grid%rssunxy , rsshaxy=grid%rsshaxy  &
L00986       &        ,  bgapxy=grid%bgapxy   ,  wgapxy=grid%wgapxy  ,   tgvxy=grid%tgvxy    &
L00987       &        ,   tgbxy=grid%tgbxy    ,   chvxy=grid%chvxy   ,   chbxy=grid%chbxy    &
L00988       &        ,   shgxy=grid%shgxy    ,   shcxy=grid%shcxy   ,   shbxy=grid%shbxy    &
L00989       &        ,   evgxy=grid%evgxy    ,   evbxy=grid%evbxy   ,   ghvxy=grid%ghvxy    &
L00990       &        ,   ghbxy=grid%ghbxy    ,   irgxy=grid%irgxy   ,   ircxy=grid%ircxy    &
L00991       &        ,   irbxy=grid%irbxy    ,    trxy=grid%trxy    ,   evcxy=grid%evcxy    &
L00992       &        ,chleafxy=grid%chleafxy ,  chucxy=grid%chucxy                  &
L00993       &        ,  chv2xy=grid%chv2xy   ,  chb2xy=grid%chb2xy , chstarxy=grid%chstarxy &
L00994    ! Noah-MP extra output fields
L00995       &        ,    qintsxy=grid%qintsxy    ,   qintrxy=grid%qintrxy   ,   qdripsxy=grid%qdripsxy   &
L00996       &        ,   qdriprxy=grid%qdriprxy   ,  qthrosxy=grid%qthrosxy  ,   qthrorxy=grid%qthrorxy   &
L00997       &        ,   qsnsubxy=grid%qsnsubxy   ,  qsnfroxy=grid%qsnfroxy  ,    qsubcxy=grid%qsubcxy    &
L00998       &        ,    qfrocxy=grid%qfrocxy    ,   qfrzcxy=grid%qfrzcxy   ,   qmeltcxy=grid%qmeltcxy   &
L00999       &        ,   qsnbotxy=grid%qsnbotxy   , pondingxy=grid%pondingxy ,      PAHXY=grid%PAHXY      &
L01000       &        ,     PAHGXY=grid%PAHGXY     ,    PAHVXY=grid%PAHVXY    ,     PAHBXY=grid%PAHBXY     &
L01001       &        ,    qmeltxy=grid%qmeltxy    ,   FPICEXY=grid%FPICEXY   ,   qevacxy=grid%qevacxy    &
L01002       &        ,    qdewcxy=grid%qdewcxy    ,   RAINLSM=grid%RAINLSM   ,    SNOWLSM=grid%SNOWLSM    &
L01003       &        ,   forctlsm=grid%forctlsm   ,  forcqlsm=grid%forcqlsm  ,   forcplsm=grid%forcplsm   &
L01004       &        ,   forczlsm=grid%forczlsm   ,  forcwlsm=grid%forcwlsm                                &
L01005       &        ,  acc_ssoil=grid%acc_ssoil  ,acc_qinsur=grid%acc_qinsur,  acc_qseva=grid%acc_qseva  &
L01006       &        , acc_etrani=grid%acc_etrani ,   eflxbxy=grid%eflxbxy                                 &
L01007       &        , soilenergy=grid%soilenergy ,snowenergy=grid%snowenergy,    canhsxy=grid%canhsxy    &
L01008       &        , ACC_DWATERXY=grid%ACC_DWATERXY, ACC_PRCPXY=grid%ACC_PRCPXY,ACC_ECANXY=grid%ACC_ECANXY &
L01009       &        , ACC_ETRANXY=grid%ACC_ETRANXY, ACC_EDIRXY=grid%ACC_EDIRXY  &
L01010    ! Noah-MP wetland
L01011       &        , FSATXY=grid%FSATXY, WSURFXY=grid%WSURFXY  &
L01012    ! Noah-MP SNICAR snow albedo
L01013       &        ,SNICAR_BANDNUMBER_OPT=config_flags%SNICAR_BANDNUMBER_OPT   &
L01014       &        ,SNICAR_SOLARSPEC_OPT=config_flags%SNICAR_SOLARSPEC_OPT     &
L01015       &        ,SNICAR_SNOWOPTICS_OPT=config_flags%SNICAR_SNOWOPTICS_OPT   &
L01016       &        ,SNICAR_DUSTOPTICS_OPT=config_flags%SNICAR_DUSTOPTICS_OPT   &
L01017       &        ,SNICAR_RTSOLVER_OPT=config_flags%SNICAR_RTSOLVER_OPT       &
L01018       &        ,SNICAR_SNOWSHAPE_OPT=config_flags%SNICAR_SNOWSHAPE_OPT     &
L01019       &        ,SNICAR_USE_AEROSOL=config_flags%SNICAR_USE_AEROSOL         &
L01020       &        ,SNICAR_SNOWBC_INTMIX=config_flags%SNICAR_SNOWBC_INTMIX     &
L01021       &        ,SNICAR_SNOWDUST_INTMIX=config_flags%SNICAR_SNOWDUST_INTMIX &
L01022       &        ,SNICAR_USE_OC=config_flags%SNICAR_USE_OC                   &
L01023       &        ,SNICAR_AEROSOL_READTABLE=config_flags%SNICAR_AEROSOL_READTABLE   &
L01024       &        ,SNRDSXY=grid%SNRDSXY, SNFRXY=grid%SNFRXY, BCPHIXY=grid%BCPHIXY   &
L01025       &        ,BCPHOXY=grid%BCPHOXY, OCPHIXY=grid%OCPHIXY, OCPHOXY=grid%OCPHOXY &
L01026       &        ,DUST1XY=grid%DUST1XY, DUST2XY=grid%DUST2XY, DUST3XY=grid%DUST3XY &
L01027       &        ,DUST4XY=grid%DUST4XY, DUST5XY=grid%DUST5XY    &
L01028       &        ,MassConcBCPHIXY=grid%MassConcBCPHIXY          &
L01029       &        ,MassConcBCPHOXY=grid%MassConcBCPHOXY, MassConcOCPHIXY=grid%MassConcOCPHIXY  &
L01030       &        ,MassConcOCPHOXY=grid%MassConcOCPHOXY, MassConcDUST1XY=grid%MassConcDUST1XY  &
L01031       &        ,MassConcDUST2XY=grid%MassConcDUST2XY, MassConcDUST3XY=grid%MassConcDUST3XY  &
L01032       &        ,MassConcDUST4XY=grid%MassConcDUST4XY, MassConcDUST5XY=grid%MassConcDUST5XY  &
L01033       &        ,ALBSOILDIRXY=grid%ALBSOILDIRXY, ALBSOILDIFXY=grid%ALBSOILDIFXY        &
L01034    ! Noah-MP irrigation
L01035       &        , IRFRACT=grid%IRFRACT  , SIFRACT=grid%SIFRACT ,MIFRACT=grid%MIFRACT   &
L01036       &        , FIFRACT=grid%FIFRACT  , IRNUMSI=grid%IRNUMSI ,IRNUMMI=grid%IRNUMMI   &
L01037       &        , IRNUMFI=grid%IRNUMFI  , IRWATSI=grid%IRWATSI ,IRWATMI=grid%IRWATMI   &
```

```
L01038       &        , IRWATFI=grid%IRWATFI  , IRELOSS=grid%IRELOSS ,IRSIVOL=grid%IRSIVOL   &
L01039       &        , IRMIVOL=grid%IRMIVOL  , IRFIVOL=grid%IRFIVOL ,IRRSPLH=grid%IRRSPLH   &
L01040  ! Noah-MP tile drainage
L01041       &        , QTDRAIN=grid%QTDRAIN  , TD_FRACTION=grid%TD_FRACTION                 &
L01042  ! Optional hydro variables in NOAHMP
L01043       &        ,smcwtdxy=grid%smcwtdxy ,rechxy=grid%rechxy ,deeprechxy=grid%deeprechxy &
L01044       &        ,fdepthxy=grid%fdepthxy, areaxy=grid%areaxy, rivercondxy=grid%rivercondxy  &
L01045       &        ,riverbedxy=grid%riverbedxy, eqzwt=grid%eqzwt, pexpxy=grid%pexpxy &
L01046       &        ,qrfxy=grid%qrfxy, qspringxy=grid%qspringxy, qslatxy=grid%qslatxy, qrfsxy=grid%qrfsxy, qlatxy=grid%qlatxy
↪  &
L01047       &        ,qspringsxy=grid%qspringsxy, smoiseq=grid%smoiseq, wtddt=config_flags%wtddt, stepwtd=grid%stepwtd &
L01048       &        ,gecros_state=grid%gecros_state                                  &   ! Optional gecros crop
L01049            ! Noah UA changes
L01050       &        ,ua_phys=config_flags%ua_phys,flx4=grid%flx4,fvb=grid%fvb       &
L01051       &        ,fbur=grid%fbur,fgsn=grid%fgsn                                  &
L01052            ! Indexes
L01053       &        ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde          &
L01054       &        ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme          &
L01055       &        ,IPS=ips,IPE=ipe, JPS=jps,JPE=jpe, KPS=kps,KPE=kpe          &
L01056       &        , I_START=grid%i_start,I_END=min(grid%i_end, ide-1)          &
L01057       &        , J_START=grid%j_start,J_END=min(grid%j_end, jde-1)          &
L01058       &        , KTS=k_start, KTE=min(k_end,kde-1)                         &
L01059       &        , NUM_TILES=grid%num_tiles                                  &
L01060            ! Variables required by TEMF PBL - WA 1/7/09
L01061            ,te_temf=grid%te_temf,hd_temf=grid%hd_temf                     &
L01062            ,fCor=grid%f,exch_temf=exch_temf,wm_temf=grid%wm_temf          &
L01063            ! Variables required by IDEAL SCM sfc scheme - WA 1/6/10
L01064            ,hfx_force=grid%hfx_force,lh_force=grid%lh_force               &
L01065            ,tsk_force=grid%tsk_force                                      &
L01066            ,hfx_force_tend=grid%hfx_force_tend                            &
L01067            ,lh_force_tend=grid%lh_force_tend                              &
L01068            ,tsk_force_tend=grid%tsk_force_tend                            &
L01069            ! Optional
L01070       &        ,QV_CURR=moist(ims,kms,jms,P_QV), F_QV=F_QV                 &
L01071       &        ,QC_CURR=moist(ims,kms,jms,P_QC), F_QC=F_QC                 &
L01072       &        ,QR_CURR=moist(ims,kms,jms,P_QR), F_QR=F_QR                 &
L01073       &        ,QI_CURR=moist(ims,kms,jms,P_QI), F_QI=F_QI                 &
L01074       &        ,QS_CURR=moist(ims,kms,jms,P_QS), F_QS=F_QS                 &
L01075       &        ,QG_CURR=moist(ims,kms,jms,P_QG), F_QG=F_QG                 &
L01076       &        ,CAPG=grid%capg, EMISS=grid%emiss, HOL=hol,MOL=grid%mol     &
L01077       &        ,T2OBS=grid%t2obs, Q2OBS=grid%q2obs                         &
L01078       &        ,RAINBL=grid%rainbl,SR=grid%sr,RAINSHV=grid%rainshv         &
L01079       &        ,GRAUPELNCV=grid%graupelncv, HAILNCV=grid%hailncv           &
L01080       &        ,RAINNCV=grid%rainncv,REGIME=grid%regime,T2=grid%t2,THC=grid%thc           &
L01081       &        ,QSG=grid%qsg,QVG=grid%qvg,QCG=grid%qcg,SOILT1=grid%soilt1,TSNAV=grid%tsnav & ! ruc lsm
L01082       &        ,SMFR3D=grid%smfr3d,KEEPFR3DFLAG=grid%keepfr3dflag,DEW=grid%dew            & ! ruc lsm
L01083       &        ,POTEVP=grid%POTEVP, SNOPCX=grid%SNOPCX, SOILTB=grid%SOILTB               & ! ruc lsm
L01084       &        ,rhosnf=grid%rhosnf ,precipfr=grid%precipfr                 &   ! RUC LSM
L01085       &        ,snowfallac=grid%snowfallac                                 &   ! RUC LSM
L01086       &        ,MOSAIC_LU=config_flags%mosaic_lu                           & ! RUC LSM
L01087       &        ,MOSAIC_SOIL=config_flags%mosaic_soil                       &   ! RUC LSM
L01088       &        ,ISURBAN=isurban, MMINLU=TRIM(mminlu)                       &
L01089       &        ,SNOTIME = grid%SNOTIME                                     &
L01090       &        ,RDLAI2D=config_flags%rdlai2d                               &
L01091       &        ,usemonalb=config_flags%usemonalb                           &
L01092       &        ,NOAHRES=grid%noahres                                       &
L01093       &        ,TSK_SAVE=grid%tsk_save                                     &
L01094       &        ,ch=grid%ch                                                 &
L01095       &        ,fgdp=grid%fgdp,dfgdp=grid%dfgdp,vdfg=grid%vdfg             & !Katata - fogdes
L01096       &        ,grav_settling=config_flags%grav_settling                   & !Katata - fogdes
L01097       &        ,OM_TMP=grid%om_tmp, OM_S=grid%om_s, OM_U=grid%om_u, OM_V=grid%om_v   & !cyl:3DPWP
L01098       &        ,OM_DEPTH=grid%om_depth, OM_ML=grid%OM_ML, OM_LON=grid%om_lon         & !cyl:3DPWP
L01099       &        ,OM_LAT=grid%om_lat                                        & !cy:3DPWP
L01100       &        , okms = 1, okme=config_flags%ocean_levels                  & ! cyl:3DPWP
L01101       &        ,rdx=grid%rdx, rdy=grid%rdy,msfu=grid%msfu,msfv=grid%msfv,msft=grid%msft &!cyl: 3DPWP
L01102       &        ,XTIME=grid%xtime,OM_TINI=grid%om_tini,OM_SINI=grid%om_sini,id=grid%id,omdt=config_flags%omdt &!cyl:
↪  3DPWP
L01103       &        ,sf_surface_mosaic=config_flags%sf_surface_mosaic,  mosaic_cat=config_flags%mosaic_cat    &
L01104       &        ,mosaic_cat_index=grid%mosaic_cat_index                                                  &  !danli
↪  mosaic
L01105       &        ,landusef2=grid%landusef2,TSK_mosaic=grid%TSK_mosaic,QSFC_mosaic=grid%QSFC_mosaic        &
L01106       &        ,TSLB_mosaic=grid%TSLB_mosaic,SMOIS_mosaic=grid%SMOIS_mosaic,SH2O_mosaic=grid%SH2O_mosaic &  !danli
↪  mosaic
L01107       &        ,CANWAT_mosaic=grid%CANWAT_mosaic,SNOW_mosaic=grid%SNOW_mosaic                           &
L01108       &        ,SNOWH_mosaic=grid%SNOWH_mosaic,SNOWC_mosaic=grid%SNOWC_mosaic                           &  !danli
↪  mosaic
L01109       &        ,ALBEDO_mosaic=grid%ALBEDO_mosaic,ALBBCK_mosaic=grid%ALBBCK_mosaic                       &
L01110       &        ,EMISS_mosaic=grid%EMISS_mosaic, EMBCK_mosaic=grid%EMBCK_mosaic                          &
L01111       &        ,ZNT_mosaic=grid%ZNT_mosaic, Z0_mosaic=grid%Z0_mosaic                                    &  !danli
↪  mosaic
L01112       &        ,HFX_mosaic=grid%HFX_mosaic,QFX_mosaic=grid%QFX_mosaic, LH_mosaic=grid%LH_mosaic         &
L01113       &        ,GRDFLX_mosaic=grid%GRDFLX_mosaic,SNOTIME_mosaic=grid%SNOTIME_mosaic                     &  !danli
↪  mosaic
L01114       &        ,RS_mosaic=grid%RS_mosaic,LAI_mosaic=grid%LAI_mosaic                                     &
L01115       &        ,TR_URB2D_mosaic=grid%TR_URB2D_mosaic,TB_URB2D_mosaic=grid%TB_URB2D_mosaic               &  !danli
↪  mosaic
L01116       &        ,TG_URB2D_mosaic=grid%TG_URB2D_mosaic,TC_URB2D_mosaic=grid%TC_URB2D_mosaic               &  !danli
↪  mosaic
L01117       &        ,QC_URB2D_mosaic=grid%QC_URB2D_mosaic,UC_URB2D_mosaic=grid%UC_URB2D_mosaic               &  !danli
↪  mosaic
L01118       &        ,TRL_URB3D_mosaic=grid%TRL_URB3D_mosaic,TBL_URB3D_mosaic=grid%TBL_URB3D_mosaic           &  !danli
↪  mosaic
L01119       &        ,TGL_URB3D_mosaic=grid%TGL_URB3D_mosaic                                                  &  !danli
↪  mosaic
L01120       &        ,SH_URB2D_mosaic=grid%SH_URB2D_mosaic,LH_URB2D_mosaic=grid%LH_URB2D_mosaic               &  !danli
↪  mosaic
L01121       &        ,G_URB2D_mosaic=grid%G_URB2D_mosaic,RN_URB2D_mosaic=grid%RN_URB2D_mosaic                 &  !danli
↪  mosaic
L01122       &        ,TS_URB2D_mosaic=grid%TS_URB2D_mosaic                                                    &  !danli
↪  mosaic
L01123       &        ,TS_RUL2D_mosaic=grid%TS_RUL2D_mosaic                                                    &  !danli
↪  mosaic
L01124       &        ,ZOL=grid%ZOL                                                                            &
L01125       &        ,SDA_HFX=grid%SDA_HFX, SDA_QFX=grid%SDA_QFX,HFX_BOTH=grid%HFX_BOTH                       &  !fasdas
L01126       &        ,QFX_BOTH=grid%QFX_BOTH,QNORM=grid%QNORM,fasdas=config_flags%fasdas                      &  !fasdas
L01127       &        ,XLAIDYN=grid%XLAIDYN                                                                    &
```

```
L01128       &        ,spp_lsm=config_flags%spp_lsm,pattern_spp_lsm=grid%pattern_spp_lsm     & !SPP
L01129       &        ,field_sf=grid%field_sf                                                & !SPP
L01130       &        ,spp_pbl=config_flags%spp_pbl,pattern_spp_pbl=grid%pattern_spp_pbl     & !SPP
L01131     ! WRF-Solar EPS: Noah LSM
L01132       &        ,multi_perturb=config_flags% multi_perturb                             &
L01133       &        ,pert_noah=config_flags%pert_noah                                      &
L01134       &        ,perts_qvapor=grid%pert3d(:,:,:,P_PQVAPOR)                             &
L01135       &        ,perts_th=grid%pert3d(:,:,:,P_PTH)                                     &
L01136       &        ,perts_smois=grid%pert3d(:,:,:,P_PSMOIS)                               &
L01137       &        ,perts_tsoil=grid%pert3d(:,:,:,P_PTSOIL)                               &
L01138       &        ,pert_noah_qv=config_flags%pert_noah_qv                                &
L01139       &        ,pert_noah_t=config_flags%pert_noah_t                                  &
L01140       &        ,pert_noah_smois=config_flags%pert_noah_smois                          &
L01141       &        ,pert_noah_tslb=config_flags%pert_noah_tslb                            &
L01142       &                                                        )
L01143
L01144     #ifdef WRF_HYDRO
L01145            if(HYDRO_dt .gt. 1 ) call wrf_drv_HYDRO(HYDRO_dt, grid,  &
L01146       &       grid%i_start(1),min(grid%i_end(1), ide-1),      &
L01147       &       grid%j_start(1),min(grid%j_end(1), jde-1) )
L01148     #endif
L01149
L01150     BENCH_END(surf_driver_tim)
```

讲解：最长的一段（605 行实参），前置三件：①Noah-MP 的 opt_runsub=5 时按 STEPWTD 周期刷 HALO_EM_HYDRO_NOAHMP（径流汇流的 halo）；② **城市层数临时赋值 num_roof/wall/road_layers=num_soil_layers**（Urban 参数表的层数下限兜底——namelist 的 num_urban_* 只在多层方案下覆盖）；③nl_get 取 iswater/isurban/mminlu（土地利用字符串走 namelist 而非 Registry）。实参分拣：海冰五选项（fractional_seaice/albedo/thickness/snowdepth 系——code_13 的 ICE 分支数据源）、**灌溉全家桶**（IRRIGATION/IRR_DAILY_AMOUNT/START_HOUR/NUM_HOURS/START_END_JULIANDAY/FREQ/PH/RAND_FIELD——sf_surf_irr_scheme 的新灌溉体系，与 code_13 的 IRI_SCHEME 是两代挂点）、OML 海洋混合层、lake 三维态、CLM 全套（H2OSOI 系 10 层雪/土壤）、SSiB 全套（ssib_* 约 40 项）、PX LSM 与其 nudging、Noah-MP 的 opt_* 全表 22 项与输出 xy 系（含 PAH 系人为热分配）、urban 全家（2D/3D 状态、BEP 的 a_/b_ 源项、CMR/CHR 系面层系数）、distributed_ahe_opt/ahe（code_22 § 六的 AHE 数据）、slope_rad/solar 系（坡面辐射）。下游见 `code_24_module_surface_driver.md`。

七、边界层段：pbl_driver

```
!file: dyn_em/module_first_rk_step_part1.F    L1152-L1402
L01152   !*********
L01153   ! pbl
L01154   #if ( WRFPLUS == 1 )
L01155   #ifdef DM_PARALLEL
L01156         IF ( config_flags%bl_pbl_physics .EQ. 98 ) THEN
L01157   #     include "HALO_EM_PHYS_A_BL_SURF.inc"
L01158         ENDIF
L01159   #endif
L01160   #endif
L01161
L01162         CALL wrf_debug ( 200 , ' call pbl_driver' )
L01163   BENCH_START(pbl_driver_tim)
L01164         CALL pbl_driver(                                                        &
L01165       &        AKHS=grid%akhs          ,AKMS=grid%akms                          &
L01166       &        ,BL_PBL_PHYSICS=config_flags%bl_pbl_physics                      &
L01167       &        ,WINDFARM_OPT=config_flags%windfarm_opt,power=grid%power         &
L01168       ! THE FOLLOWING LINES OF CODE ARE LICENSED UNDER doc/licenses/fraunhofer_license.txt
L01169       &        ,thrust=grid%thrust                                              &
L01170       ! END LICENSE
L01171       &        ,windfarm_wake_model=config_flags%windfarm_wake_model          & ! Yulong add for WLM
L01172       &        ,windfarm_overlap_method=config_flags%windfarm_overlap_method  & ! Yulong add for WLM
L01173       &        ,BLDT=grid%bldt, CURR_SECS=curr_secs, ADAPT_STEP_FLAG=adapt_step_flag &
L01174       &        ,BLDTACTTIME=grid%bldtacttime                                  &
L01175       &        ,BR=grid%br          ,CHKLOWQ=chklowq    ,CT=grid%ct           &
L01176       &        ,DT=grid%dt          ,DX=grid%dx         ,DY=grid%dy           &
L01177       &        ,DX2D=grid%dx2d      ,AREA2D=grid%area2d                       &
L01178       &        ,DZ8W=dz8w           ,HPBL_HOLD=hpbl_hold                      &
L01179       &        ,EXCH_H=grid%exch_h      ,EXCH_M=grid%exch_m                   &
L01180       &        ,FM=grid%fm          ,FHH=grid%fh                              &
L01181       &        ,F=grid%f   ,GRDFLX=grid%grdflx      &
L01182       &        ,GZ1OZ0=gz1oz0       ,HFX=grid%hfx           ,HT=grid%ht       &
L01183       &        ,ID=grid%id          ,ITIMESTEP=grid%itimestep     ,KPBL=grid%kpbl          &
L01184       &        ,LH=grid%lh               ,LOWLYR=grid%lowlyr      ,P8W=grid%p_hyd_w   &
L01185       &        ,PBLH=grid%pblh           ,PI_PHY=pi_phy      ,PSIH=psih          &
L01186       &        ,PSIM=psim           ,P_PHY=grid%p_hyd        ,QFX=grid%qfx        &
L01187       &        ,QSFC=grid%qsfc          ,QZ0=grid%qz0   ,MIXHT=mixht           &
L01188       &        ,RA_LW_PHYSICS=config_flags%ra_lw_physics                &
L01189       &        ,RHO=grid%rho            ,RQCBLTEN=grid%rqcblten  ,RQIBLTEN=grid%rqiblten  &
L01190       &        ,RQVBLTEN=grid%rqvblten  ,RTHBLTEN=grid%rthblten  ,RUBLTEN=grid%rublten    &
L01191       &        ,RVBLTEN=grid%rvblten    ,SNOW=grid%snow          ,STEPBL=grid%stepbl      &
L01192       &        ,THZ0=grid%thz0          ,TH_PHY=th_phy                            &
L01193       &        ,TSK=grid%tsk            ,T_PHY=grid%t_phy        ,UST=grid%ust       &
L01194       &        ,U10=grid%u10 ,UZ0=grid%uz0      ,U_FRAME=grid%u_frame    ,U_PHY=grid%u_phy   &
L01195       &        ,V10=grid%v10 ,VZ0=grid%vz0      ,V_FRAME=grid%v_frame    ,V_PHY=grid%v_phy   &
L01196       &        ,W=grid%w_2   ,UOCE=grid%uoce    ,VOCE=grid%voce                      &
L01197                ,T2=grid%t2 &
L01198       &        ,WARM_RAIN=grid%warm_rain                ,WSPD=grid%wspd     &
L01199       &        ,XICE=grid%xice          ,XLAND=grid%xland        ,Z=grid%z               &
L01200       &        ,ZNT=grid%znt                                                  &
L01201       &        ,ysu_topdown_pblmix=config_flags%ysu_topdown_pblmix            &
L01202       &        ,shinhong_tke_diag=config_flags%shinhong_tke_diag              &
L01203       &        ,shinhong_scu_mixing=config_flags%shinhong_scu_mixing              &
```

```
L01204       &        ,shinhong_nonlocal_flux=config_flags%shinhong_nonlocal_flux              &
L01205       &        ,shinhong_ke_dissipation=config_flags%shinhong_ke_dissipation              &
L01206  ! paj: topo_wind
L01207       &        ,CTOPO=grid%ctopo,CTOPO2=grid%ctopo2                                    &
L01208  ! variables added for BEP
L01209       &          ,FRC_URB2D=grid%frc_urb2d                                               &
L01210       &          ,A_U_BEP=grid%a_u_bep,A_V_BEP=grid%a_v_bep,A_T_BEP=grid%a_t_bep         &
L01211       &          ,A_Q_BEP=grid%a_q_bep                                                   &
L01212       &          ,B_U_BEP=grid%b_u_bep,B_V_BEP=grid%b_v_bep,B_T_BEP=grid%b_t_bep         &
L01213       &          ,B_Q_BEP=grid%b_q_bep                                                   &
L01214       &          ,SF_BEP=grid%sf_bep,VL_BEP=grid%vl_bep                                  &
L01215       &          ,A_E_BEP=grid%a_e_bep,B_E_BEP=grid%b_e_bep,DLG_BEP=grid%dlg_bep         &
L01216       &          ,DL_U_BEP=grid%dl_u_bep                                                 &
L01217       &          ,SF_SFCLAY_PHYSICS=config_flags%sf_sfclay_physics                       &
L01218       &          ,SF_URBAN_PHYSICS=config_flags%sf_urban_physics                         &
L01219  ! Bep changes end
L01220  ! add tke_pbl, and turbulent fluxes
L01221       &          ,TKE_PBL=grid%tke_pbl,EL_PBL=grid%el_pbl,WU_TUR=grid%wu_tur             &
L01222       &          , gmt=grid%gmt, xtime=grid%xtime,julday=grid%julday,julyr=grid%julyr     &
L01223       &          , ahe=grid%ahe,distributed_ahe_opt=grid%distributed_ahe_opt              &
L01224       &          ,WV_tur=grid%wv_tur,WT_tur=grid%wt_tur,WQ_tur=grid%wq_tur               &
L01225       &          ,DISS_PBL=grid%diss_pbl,TPE_PBL=grid%tpe_pbl                            &
L01226       &          ,TKE_ADV=scalar(ims,kms,jms,P_tke_adv)                                  &
L01227       &          ,DISS_ADV=scalar(ims,kms,jms,P_diss_adv)                                &
L01228       &          ,TPE_ADV=scalar(ims,kms,jms,P_tpe_adv),PR_PBL=grid%pr_pbl               &
L01229  ! end add tke_pbl, and turbulent fluxes
L01230  ! GBMPBL change: add exch_tke, rthraten
L01231       &        ,EXCH_TKE=grid%exch_tke, RTHRATEN=grid%rthraten             &
L01232       &        ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde          &
L01233       &        ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme          &
L01234       &        ,I_START=grid%i_start,I_END=min(grid%i_end, ide-1)           &
L01235       &        ,J_START=grid%j_start,J_END=min(grid%j_end, jde-1)           &
L01236       &        ,KTS=k_start, KTE=min(k_end,kde-1)                          &
L01237       &        ,NUM_TILES=grid%num_tiles                                   &
L01238            ! Variables Required by ACM PBL  - jp
L01239       &        ,ZNU=grid%znu,ZNW=grid%znw,MUT=grid%mut,P_TOP=grid%p_top    &
L01240            ! Variables required by TEMF PBL - WA 9/9/08
L01241                ,te_temf=grid%te_temf                                      &
L01242                ,kh_temf=grid%kh_temf,km_temf=grid%km_temf                 &
L01243                ,shf_temf=grid%shf_temf,qf_temf=grid%qf_temf               &
L01244                ,uw_temf=grid%uw_temf,vw_temf=grid%vw_temf                 &
L01245                ,hd_temf=grid%hd_temf,lcl_temf=grid%lcl_temf               &
L01246                ,wupd_temf=grid%wupd_temf,mf_temf=grid%mf_temf             &
L01247                ,thup_temf=grid%thup_temf,qtup_temf=grid%qtup_temf         &
L01248                ,qlup_temf=grid%qlup_temf                                  &
L01249                ,cf3d_temf=grid%cf3d_temf,cfm_temf=grid%cfm_temf           &
L01250                ,hct_temf=grid%hct_temf                                    &
L01251                ,flhc=grid%flhc,flqc=grid%flqc                             &
L01252                ,exch_temf=exch_temf                                       &
L01253            ! optional
L01254       &        ,QV_CURR=moist(ims,kms,jms,P_QV), F_QV=F_QV                 &
L01255       &        ,QC_CURR=moist(ims,kms,jms,P_QC), F_QC=F_QC                 &
L01256       &        ,QR_CURR=moist(ims,kms,jms,P_QR), F_QR=F_QR                 &
L01257       &        ,QI_CURR=moist(ims,kms,jms,P_QI), F_QI=F_QI                 &
L01258       &        ,QS_CURR=moist(ims,kms,jms,P_QS), F_QS=F_QS                 &
L01259       &        ,QG_CURR=moist(ims,kms,jms,P_QG), F_QG=F_QG                 &
L01260       &        ,QNIFA_CURR=scalar(ims,kms,jms,P_QNIFA),F_QNIFA=F_QNIFA     &
L01261       &        ,QNWFA_CURR=scalar(ims,kms,jms,P_QNWFA),F_QNWFA=F_QNWFA     &
L01262       &        ,QNBCA_CURR=scalar(ims,kms,jms,P_QNBCA),F_QNBCA=F_QNBCA     &
L01263       &        ,HOL=HOL, MOL=grid%mol, REGIME=grid%REGIME                  &
L01264  !mynn mp@
L01265       &        ,QKE=grid%qke, Sh3d=grid%sh3d, Sm3d=grid%sm3d               &
L01266       &        ,QKE_ADV=scalar(ims,kms,jms,P_qke_adv)                      & !ACF-QKE advection
L01267       &        ,bl_mynn_tkeadvect=config_flags%bl_mynn_tkeadvect           & !ACF-QKE advection
L01268       &        ,tsq=grid%tsq, qsq=grid%qsq, cov=grid%cov                   &
L01269       &        ,DQKE=grid%dqke,QWT=grid%qWT                                &
L01270       &        ,QSHEAR=grid%qSHEAR,QBUOY=grid%qBUOY,QDISS=grid%qDISS       &
L01271       &        ,tke_budget=config_flags%tke_budget                         &
L01272       &        ,bl_mynn_cloudpdf=config_flags%bl_mynn_cloudpdf             &
L01273       &        ,bl_mynn_mixlength=config_flags%bl_mynn_mixlength           &
L01274       &        ,qc_bl=grid%qc_bl,qi_bl=grid%qi_bl,cldfra_bl=grid%cldfra_bl &
L01275       &        ,bl_mynn_edmf=config_flags%bl_mynn_edmf                     &
L01276                ,bl_mynn_edmf_dd=config_flags%bl_mynn_edmf_dd               &
L01277       &        ,bl_mynn_edmf_mom=config_flags%bl_mynn_edmf_mom             &
L01278       &        ,bl_mynn_edmf_tke=config_flags%bl_mynn_edmf_tke             &
L01279       &        ,bl_mynn_mixscalars=config_flags%bl_mynn_mixscalars         &
L01280       &        ,bl_mynn_mixaerosols=config_flags%bl_mynn_mixaerosols       &
L01281       &        ,bl_mynn_mixnumcon=config_flags%bl_mynn_mixnumcon           &
L01282       &        ,bl_mynn_output=config_flags%bl_mynn_output                 &
L01283       &        ,bl_mynn_cloudmix=config_flags%bl_mynn_cloudmix             &
L01284       &        ,bl_mynn_mixqt=config_flags%bl_mynn_mixqt                   &
L01285       &        ,bl_mynn_ess=config_flags%bl_mynn_ess                       &
L01286       &        ,bl_mynn_closure=config_flags%bl_mynn_closure               &
L01287       &        ,edmf_a=grid%edmf_a,edmf_w=grid%edmf_w                      &
L01288       &        ,edmf_thl=grid%edmf_thl,edmf_qt=grid%edmf_qt                &
L01289       &        ,edmf_ent=grid%edmf_ent,edmf_qc=grid%edmf_qc                &
L01290       &        ,sub_thl3D=grid%sub_thl3D,sub_sqv3D=grid%sub_sqv3D          &
L01291       &        ,det_thl3D=grid%det_thl3D,det_sqv3D=grid%det_sqv3D          &
L01292       &        ,rmol=grid%rmol, ch=grid%ch                                 &
L01293       &        ,qcg=grid%qcg, grav_settling=config_flags%grav_settling     &
L01294  !    &        ,K_m=grid%K_m, K_h=grid%K_h, K_q=grid%K_q                   &
L01295       &        ,vdfg=grid%vdfg,maxwidth=grid%maxwidth,maxMF=grid%maxmf     &
L01296       &        ,ztop_plume=grid%ztop_plume,maxmf_dd=grid%maxmf_dd          &
L01297       &        ,maxtkeprod=grid%maxtkeprod,maxwidth_dd=grid%maxwidth_dd    &
L01298       &        ,cldtop_cooling=grid%cldtop_cooling,ent_eff=grid%ent_eff    &
L01299       &        ,excess_h=grid%excess_h,excess_q=grid%excess_q              &
L01300       &        ,spp_pbl=config_flags%spp_pbl                               &
L01301       &        ,pattern_spp_pbl=grid%pattern_spp_pbl                       &
L01302       &        ,restart=config_flags%restart,cycling=config_flags%cycling  &
L01303  !EEPS for ARW
L01304       &        ,pek=grid%pek_pbl,pep=grid%pep_pbl                          &
L01305       &        ,PEK_ADV=scalar(ims,kms,jms,P_pek_adv)                      &!TKEadvection
L01306       &        ,PEP_ADV=scalar(ims,kms,jms,P_pep_adv)                      &!TKEadvection
L01307  !GWD for ARW
L01308       &        ,GWD_OPT=config_flags%gwd_opt                               &
L01309       &        ,gwd_diags=config_flags%gwd_diags                           &
L01310       &        ,dx_factor=config_flags%gwd_dx_factor                       &
L01311       &        ,if_nonhyd=config_flags%gwd_if_nonhyd                       &
L01312       &        ,DTAUX3D=grid%dtaux3d,DTAUY3D=grid%dtauy3d &
```

```
L01313       &        ,DUSFCG=grid%dusfcg,DVSFCG=grid%dvsfcg &
L01314       &        ,VAR2D=grid%var2d,OC12D=grid%oc12d     &
L01315       &        ,OA1=grid%oa1,OA2=grid%oa2,OA3=grid%oa3,OA4=grid%oa4        &
L01316       &        ,OL1=grid%ol1,OL2=grid%ol2,OL3=grid%ol3,OL4=grid%ol4        &
L01317       &        ,elvmax=grid%elvmax                                         &
L01318       &        ,SINA=grid%sina,COSA=grid%cosa                              &
L01319       &        ,dtaux3d_ls=grid%dtaux3d_ls,dtauy3d_ls=grid%dtauy3d_ls      &
L01320       &        ,dtaux3d_bl=grid%dtaux3d_bl,dtauy3d_bl=grid%dtauy3d_bl      &
L01321       &        ,dtaux3d_ss=grid%dtaux3d_ss,dtauy3d_ss=grid%dtauy3d_ss      &
L01322       &        ,dtaux3d_fd=grid%dtaux3d_fd,dtauy3d_fd=grid%dtauy3d_fd      &
L01323       &        ,DUSFCG_ls=grid%dusfcg_ls,DVSFCG_ls=grid%dvsfcg_ls &
L01324       &        ,DUSFCG_bl=grid%dusfcg_bl,DVSFCG_bl=grid%dvsfcg_bl &
L01325       &        ,DUSFCG_ss=grid%dusfcg_ss,DVSFCG_ss=grid%dvsfcg_ss &
L01326       &        ,DUSFCG_fd=grid%dusfcg_fd,DVSFCG_fd=grid%dvsfcg_fd &
L01327       &        ,VAR2DLS=grid%var2dls,OC12DLS=grid%oc12dls                  &
L01328       &        ,OA1LS=grid%oa1ls,OA2LS=grid%oa2ls,OA3LS=grid%oa3ls         &
L01329       &        ,OA4LS=grid%oa4ls,OL1LS=grid%ol1ls,OL2LS=grid%ol2ls         &
L01330       &        ,OL3LS=grid%ol3ls,OL4LS=grid%ol4ls                          &
L01331       &        ,VAR2DSS=grid%var2dss,OC12DSS=grid%oc12dss                  &
L01332       &        ,OA1SS=grid%oa1ss,OA2SS=grid%oa2ss,OA3SS=grid%oa3ss         &
L01333       &        ,OA4SS=grid%oa4ss,OL1SS=grid%ol1ss,OL2SS=grid%ol2ss         &
L01334       &        ,OL3SS=grid%ol3ss,OL4SS=grid%ol4ss                          &
L01335       &        ,MFSHCONV=config_flags%mfshconv                             &
L01336       &        ,MASSFLUX_EDKF=grid%massflux_EDKF                           &
L01337       &        ,ENTR_EDKF=grid%entr_EDKF, DETR_EDKF=grid%detr_EDKF         &
L01338       &        ,THL_UP=grid%thl_up                                         &
L01339       &        ,THV_UP=grid%thv_up, RT_UP=grid%rt_up ,RV_UP=grid%rv_up     &
L01340       &        ,RC_UP=grid%rc_up, U_UP=grid% u_up, V_UP=grid%v_up          &
L01341       &        ,FRAC_UP=grid%frac_up, RC_MF=grid%RC_MF                     &
L01342  ! For Wind Turbine Drag Parameterizations
L01343       &        ,phb=grid%phb                                               &
L01344       &        ,XLAT_U=grid%xlat_u,XLONG_U=grid%xlong_u                    &
L01345    ! WRF-Solar EPS: MYNN
L01346       &        ,multi_perturb=config_flags%multi_perturb                   &
L01347       &        ,pert_mynn=config_flags%pert_mynn                           &
L01348       &        ,perts_qvapor=grid%pert3d(:,:,:,P_PQVAPOR)                  &
L01349       &        ,perts_qcloud=grid%pert3d(:,:,:,P_PQCLOUD)                  &
L01350       &        ,perts_th=grid%pert3d(:,:,:,P_PTH)                          &
L01351       &        ,perts_tke=grid%pert3d(:,:,:,P_PTKE)                        &
L01352       &        ,pert_mynn_qv=config_flags%pert_mynn_qv                     &
L01353       &        ,pert_mynn_qc=config_flags%pert_mynn_qc                     &
L01354       &        ,pert_mynn_t=config_flags%pert_mynn_t                       &
L01355       &        ,pert_mynn_qke=config_flags%pert_mynn_qke                   &
L01356  !Variables required for camuwpbl scheme
L01357       &        ,Z_AT_W=grid%z_at_w,CLDFRA_OLD_MP=grid%cldfra_old_mp        &
L01358       &        ,CLDFRA=grid%cldfra                                         &
L01359       &        ,RTHRATENLW=grid%rthratenlw,TAURESX2D=grid%tauresx2d        &
L01360       &        ,TAURESY2D=grid%tauresy2d                                   &
L01361       &        ,TPERT2D=grid%tpert2d,QPERT2D=grid%qpert2d                  &
L01362       &        ,WPERT2D=grid%wpert2d,WSEDL3D=grid%wsedl3d                  &
L01363       &        ,TURBTYPE3D=grid%turbtype3d,SMAW3D=grid%smaw3d              &
L01364       &        ,QNC_CURR=scalar(ims,kms,jms,P_QNC), F_QNC=f_qnc            &
L01365       &        ,QNI_CURR=scalar(ims,kms,jms,P_QNI), F_QNI=f_qni            &
L01366       &        ,RQNIBLTEN=grid%rqniblten                                   &
L01367       &        ,XLAT_V=grid%xlat_v,XLONG_V=grid%xlong_v,FNM=grid%fnm       &
L01368       &        ,FNP=grid%fnp, IS_CAMMGMP_USED = grid%is_CAMMGMP_used       &
L01369  ! for grims shallow convection with ysupbl
L01370       &        ,WSTAR=grid%wstar_ysu,DELTA=grid%delta_ysu                  &
L01371  ! for pbl mixing of scalars and tracers
L01372       &        ,SCALAR=scalar,SCALAR_TEND=scalar_tend,NUM_SCALAR=num_scalar&
L01373       &        ,TRACER=tracer,TRACER_TEND=tracer_tend,NUM_TRACER=num_tracer&
L01374       &        ,SCALAR_PBLMIX=config_flags%scalar_pblmix                   &
L01375       &        ,TRACER_PBLMIX=config_flags%tracer_pblmix                   &
L01376  #if (WRF_CHEM == 1)
L01377       &        ,mynn_chem_vertmx=config_flags%mynn_chem_vertmx             &
L01378       &        ,CHEM=chem,VD=grid%dep_vel                                  &
L01379       &        ,NCHEM=num_chem,kdvel=config_flags%kdepvel                  &
L01380       &        ,ndvel=config_flags%ndepvel                                 &
L01381       &        ,NUM_VERT_MIX=grid%num_vert_mix                             &
L01382  #endif
L01383       &        ,QNORM=grid%QNORM, fasdas=config_flags%fasdas               & !fasdas
L01384       &        )
L01385
L01386  #if (WRF_CHEM == 1)
L01387  #ifdef DM_PARALLEL
L01388       IF ( num_chem >= PARAM_FIRST_SCALAR .AND. (config_flags%bl_pbl_physics == &
L01389       & mynnpblscheme)  ) then
L01390          CALL wrf_debug ( 200 , ' call HALO CHEM AFTER PBL' )
L01391          IF      ( config_flags%h_sca_adv_order <= 4 ) THEN
L01392  #      include "HALO_EM_CHEM_E_3.inc"
L01393          ELSE IF ( config_flags%h_sca_adv_order <= 6 ) THEN
L01394  #      include "HALO_EM_CHEM_E_5.inc"
L01395          ELSE
L01396               WRITE(message,*)'solve_em: invalid h_sca_adv_order = ',&
L01397               & config_flags%h_sca_adv_order
L01398          ENDIF
L01399       ENDIF
L01400  #endif
L01401  #endif
L01402
```

讲解：pbl_driver 实参段（251 行）——除常规态量外注意：exch_temf（MYJ 系交换系数回传给面层）、u_phytmp/v_phytmp（移动嵌套参考系，code_22 § 三）、rthraten/rthblten/rqvblten（辐射与 PBL 的前期倾向，供积云闭合的组合原料）、WRF_CHEM 的标量倾向 mix_chem 通道；段尾 HALO CHEM AFTER PBL（化学标量倾向的 halo）。下游见 `code_22_module_pbl_driver.md`。

八、火段：ifire 三选

```
!file: dyn_em/module_first_rk_step_part1.F    L1404-L1551
L01404
L01405  !*****
L01406  ! fire
L01407       Select_fire_model: select case (config_flags%ifire)
L01408         case (0)
L01409           continue
L01410  #ifdef WRF_CFBM
L01411         case (1)
L01412            ! Interpolate from Atm to fire grid
L01413  #ifdef DM_PARALLEL
L01414  #     include "HALO_EM_CFBM.inc"
L01415  #endif
L01416           call Interp_horizontal (grid%znt, grid%fire_state%proj, ims, ime, jms, jme, &
L01417               grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01418               grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01419               grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01420               grid%fire_state%fz0)
L01421
L01422           call Interp_horizontal (grid%t2, grid%fire_state%proj, ims, ime, jms, jme, &
L01423               grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01424               grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01425               grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01426               grid%fire_state%fire_t2)
L01427
L01428           call Interp_horizontal (grid%q2, grid%fire_state%proj, ims, ime, jms, jme, &
L01429               grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01430               grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01431               grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01432               grid%fire_state%fire_q2)
L01433
L01434           call Interp_horizontal (grid%psfc, grid%fire_state%proj, ims, ime, jms, jme, &
L01435               grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01436               grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01437               grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01438               grid%fire_state%fire_psfc)
L01439
L01440           call Interp_horizontal (grid%rainc + grid%rainnc, grid%fire_state%proj, ims, ime, jms, jme, &
L01441               grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01442               grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01443               grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01444               grid%fire_state%fire_rain)
L01445
L01446           Select_vinterp_opt: select case  (grid%cfbm_namelist%wind_vinterp_opt)
L01447             case (VINTERP_WINDS_FROM_3D_WINDS)
L01448               call Interp_wrfwinds_to_cfbm (grid%u_phy, grid%v_phy, grid%z_at_w, ims, ime, kms, kme, jms, jme, &
L01449                   grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01450                   grid%fire_state%ifps, grid%fire_state%ifpe, grid%fire_state%jfps, grid%fire_state%jfpe, &
L01451                   grid%fire_state%kfds, grid%fire_state%kfde, &
L01452                   grid%fire_state%lats, grid%fire_state%lons, grid%fire_state%proj, grid%fire_state%fz0,  &
L01453                   grid%cfbm_namelist%fire_lsm_zcoupling, grid%cfbm_namelist%fire_lsm_zcoupling_ref, &
L01454                   grid%cfbm_namelist%fire_wind_height, grid%fire_state%uf, grid%fire_state%vf)
L01455
L01456             case (VINTERP_WINDS_FROM_10M_WINDS)
L01457               call Interp_horizontal (grid%u10, grid%fire_state%proj, ims, ime, jms, jme, &
L01458                   grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01459                   grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01460                   grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01461                   grid%fire_state%uf)
L01462
L01463               call Interp_horizontal (grid%v10, grid%fire_state%proj, ims, ime, jms, jme, &
L01464                   grid%fire_state%ifms, grid%fire_state%ifme, grid%fire_state%jfms, grid%fire_state%jfme, &
L01465                   grid%fire_state%num_tiles, grid%fire_state%i_start, grid%fire_state%i_end, grid%fire_state%j_start, &
L01466                   grid%fire_state%j_end, grid%cfbm_namelist%hinterp_opt, grid%fire_state%lats, grid%fire_state%lons, &
L01467                   grid%fire_state%vf)
L01468
L01469               call grid%fire_state%Apply_wafs ()
L01470
L01471             case default
L01472               call wrf_error_fatal ('The vertical wind interpolation option does not exist in CFBM')
L01473
L01474           end select Select_vinterp_opt
L01475
L01476           call Advance_state (grid = grid%fire_state, config_flags = grid%cfbm_namelist)
L01477
L01478           if (grid%fire_state%num_tiles == grid%num_tiles .and. &
L01479              grid%cfbm_namelist%tile_strategy == config_flags%tile_strategy) then
L01480             !$OMP PARALLEL DO   &
L01481             !$OMP PRIVATE (ij)
L01482             do ij = 1, grid%num_tiles
L01483               call Provide_atm_feedback (grid%cfbm_namelist,            &
L01484                 grid%fire_state%ifms, grid%fire_state%ifme,             &
L01485                 grid%fire_state%jfms, grid%fire_state%jfme,             &
L01486                 grid%fire_state%i_start(ij), grid%fire_state%i_end(ij), &
L01487                 grid%fire_state%j_start(ij), grid%fire_state%j_end(ij), &
L01488                 grid%fire_state%ifps, grid%fire_state%ifpe,             &
L01489                 grid%fire_state%jfps, grid%fire_state%jfpe,             &
L01490                 ids, ide, kds, kde, jds, jde,                           &
L01491                 ims, ime, kms, kme, jms, jme,                           &
L01492                 grid%i_start(ij), grid%i_end(ij),                       &
L01493                 k_start, k_end,                                         &
L01494                 grid%j_start(ij), grid%j_end(ij),                       &
L01495                 grid%sr_x, grid%sr_y,                                   &
L01496                 grid%fire_state%emis_smoke, tracer(:,:,:,p_fire_smoke), &
L01497                 config_flags%tracer_opt,                                &
L01498                 p_phy, t_phy, moist(ims,kms,jms,P_QV),                  &
L01499                 grid%aod5502d_smoke,                                    &
L01500                 grid%fire_state%fgrnhfx, grid%fire_state%fgrnqfx,       &
L01501                 grid%grnhfx, grid%grnqfx, grid%canhfx, grid%canqfx,     &
L01502                 grid%grnsmk,                                            &
L01503                 ! obsolete WRF flags
L01504                 config_flags%fire_ext_grnd,config_flags%fire_ext_crwn,  &
L01505                 config_flags%fire_crwn_hgt, grid%rho, dz8w,             &
L01506                 grid%z_at_w, grid%mut, grid%c1h, grid%c2h,              &
L01507                 grid%rthfrten, grid%rqvfrten)          ! theta and Qv tendencies
```

```
L01508            end do
L01509            !$OMP END PARALLEL DO
L01510
L01511            if (config_flags%tracer_opt == 3) then
L01512  #ifdef DM_PARALLEL
L01513  #     include "HALO_EM_TRACER_E_5.inc"
L01514  #endif
L01515            end if
L01516          else
L01517            call wrf_error_fatal ('num_tiles in WRF and CFBM should match')
L01518          endif
L01519
L01520          call grid%fire_state%Handle_output (grid%cfbm_namelist)
L01521  #endif
L01522        case (2)
L01523  ! Jan Mandel's call to SFIRE
L01524
L01525        IF ((grid%sr_x > 0 .OR. grid%sr_y > 0) .AND. config_flags%ifire == 2) THEN
L01526
L01527  BENCH_START(fire_driver_tim)
L01528          if(config_flags%ifire.eq.2)then
L01529              ! initialization moved to start_em:start_domain_em
L01530  !            if(grid%initestep.eq.1) &
L01531  !            call fire_driver_em_init (  grid , config_flags    &
L01532  !            ,ids,ide, kds,kde, jds,jde                          &
L01533  !            ,ims,ime, kms,kme, jms,jme                          &
L01534  !            ,ips,ipe, kps,kpe, jps,jpe  )
L01535              ! one timestep of the fire model
L01536              call fire_driver_em_step (  grid , config_flags    &
L01537              ,ids,ide, kds,kde, jds,jde                          &
L01538              ,ims,ime, kms,kme, jms,jme                          &
L01539              ,ips,ipe, kps,kpe, jps,jpe                          &
L01540              ,grid%rho,grid%z_at_w,dz8w)
L01541          endif
L01542
L01543  BENCH_END(fire_driver_tim)
L01544          ENDIF
L01545
L01546          case default
L01547            CALL wrf_error_fatal('The selected fire model does not exist')
L01548
L01549        end select Select_fire_model
L01550
L01551  ! cumulus para.
```

讲解：Select_fire_model 按 namelist ifire 三分支：0 直通；1 是 **WRF-CFBM 耦合路径**——把 znt/t2/q2/psfc/降水水平插值到火网格（fire_state 的专用投影 proj）、风场按两种垂插方案（3D 风 or 10m 风）换算，然后 Advance_state 推进火模型、Provide_atm_feedback 回给大气；2 是标准 WRF-Fire：call fire_driver_em_step。火方案不在五大驱动器之列、但同在本总装层——ch20 案例库的耦合改造样板。

九、积云段：cumulus_driver + convtrans_prep

```
!file: dyn_em/module_first_rk_step_part1.F    L1553-L1750
L01553        CALL wrf_debug ( 200 , ' call cumulus_driver' )
L01554
L01555  #if ( WRF_DFI_RADAR == 1 )
L01556        do_capsupress=0
L01557           if(grid%dfi_stage == DFI_FWD ) do_capsupress=1
L01558           if(grid%itimestep <= 31 .and. grid%dfi_stage == DFI_FST ) do_capsupress=1
L01559  #endif
L01560
L01561  BENCH_START(cu_driver_tim)
L01562        CALL cumulus_driver(grid                                                &
L01563                  ! Prognostic variables
L01564        &           ,U=grid%u_phy    ,V=grid%v_phy    ,TH=th_phy  ,T=grid%t_phy                &
L01565        &           ,W=grid%w_2      ,P=grid%p_hyd    ,PI=pi_phy  ,RHO=grid%rho             &
L01566                  ! Other arguments
L01567        &           ,ITIMESTEP=grid%itimestep ,DT=grid%dt      ,DX=grid%dx              &
L01568        &           ,DX2D=grid%dx2d, AREA2D=grid%area2d                                 &
L01569        &           ,CUDT=grid%cudt,CURR_SECS=curr_secs,ADAPT_STEP_FLAG=adapt_step_flag &
L01570        &           ,CCLDFRA=grid%ccldfra     ,CONVCLD=grid%convcld                     &
L01571        &           ,QCCONV=grid%qcconv     ,QICONV=grid%qiconv                         &
L01572        &           ,CUDTACTTIME=grid%cudtacttime                                       &
L01573        &           ,RAINC=grid%rainc    ,RAINCV=grid%raincv    ,PRATEC=grid%pratec     &
L01574        &           ,NCA=grid%nca                                                       &
L01575        &           ,CLDFRA_DP=grid%cldfra_dp  ,CLDFRA_SH=grid%cldfra_sh            &
L01576        &           ,QC_CU=grid%QC_CU, QI_CU=grid%QI_CU, QR_CU=grid%QR_CU, QS_CU=grid%QS_CU &
L01577        &           ,NC_CU=grid%NC_CU, NI_CU=grid%NI_CU, NR_CU=grid%NR_CU, NS_CU=grid%NS_CU &
L01578        &           ,CCN_CU=grid%CCN_CU, CU_UAF=grid%CU_UAF                             &
L01579        &           ,UDR_KF=grid%udr_kf,DDR_KF=grid%ddr_kf                          & ! kf_edrates
L01580        &           ,UER_KF=grid%uer_kf,DER_KF=grid%der_kf,TIMEC_KF=grid%timec_kf       &
L01581        &           ,KF_EDRATES=config_flags%kf_edrates                             &
L01582        &           ,HTOP=grid%cutop     ,HBOT=grid%cubot       ,KPBL=grid%kpbl         &
L01583        &           ,Z=grid%z ,Z_AT_W=grid%z_at_w ,MAVAIL=grid%mavail ,PBLH=grid%pblh   &
L01584        &           ,HPBL_HOLD=hpbl_hold                                            &
L01585        &           ,DZ8W=dz8w     ,P8W=grid%p_hyd_w, PSFC=grid%psfc, TSK=grid%tsk      &
L01586        &           ,TKE_PBL=grid%tke_pbl, UST=grid%ust                             &
L01587        &           ,W0AVG=grid%w0avg   ,STEPCU=grid%stepcu                         &
L01588        &           ,CLDEFI=grid%cldefi ,LOWLYR=grid%lowlyr ,XLAND=grid%xland           &
L01589        &           ,APR_GR=grid%apr_gr ,APR_W=grid%apr_w   ,APR_MC=grid%apr_mc         &
L01590        &           ,APR_ST=grid%apr_st ,APR_AS=grid%apr_as ,APR_CAPMA=grid%apr_capma   &
L01591        &           ,APR_CAPME=grid%apr_capme          ,APR_CAPMI=grid%apr_capmi     &
L01592        &           ,MASS_FLUX=grid%mass_flux          ,XF_ENS=grid%xf_ens           &
L01593        &           ,PR_ENS=grid%pr_ens ,HT=grid%ht,EDT_OUT=grid%edt_out             &
L01594        &           ,imomentum=grid%imomentum,clos_choice=grid%clos_choice           &
L01595        &           ,ishallow=config_flags%ishallow                                 &
L01596        &           ,cugd_tten=grid%cugd_tten,cugd_qvten=grid%cugd_qvten,cugd_qcten=grid%cugd_qcten   &
L01597        &           ,cugd_ttens=grid%cugd_ttens,cugd_qvtens=grid%cugd_qvtens   &
```

```
L01598       &             ,ENSDIM=config_flags%ensdim ,MAXIENS=config_flags%maxiens ,MAXENS=config_flags%maxens         &
L01599       &             ,MAXENS2=config_flags%maxens2 ,MAXENS3=config_flags%maxens3       &
L01600       &             ,CU_ACT_FLAG=cu_act_flag   ,WARM_RAIN=grid%warm_rain   &
L01601       &             ,HFX=grid%hfx, QFX=grid%qfx                            &
L01602       &             ,CLDFRA=grid%cldfra,CLDFRA_MP_ALL=grid%cldfra_mp_all  &
L01603       &             ,TPERT2D=grid%tpert2d                                  &
L01604       &             ,GSW=grid%gsw,cugd_avedx=config_flags%cugd_avedx       &
L01605       !BSINGH - For WRFCuP scheme
L01606       &             ,AKPBL=grid%akpbl,BR=grid%br, REGIME=grid%regime, T2=grid%t2, Q2=grid%q2   & !CuP, wig 3-Aug-2006
L01607       &             ,SLOPESFC=grid%slopeSfc, SLOPEEZ=grid%slopeEZ                 & !CuP, wig 7-Aug-2006
L01608       &             ,SIGMASFC=grid%sigmaSfc, SIGMAEZ=grid%sigmaEZ                 & !CuP, wig 7-Aug-2006
L01609       &             ,CUPFLAG=grid%cupflag                                 & !CuP, wig 9-Oct-2006
L01610       &             ,CLDFRA_CUP=grid%cldfra_cup, CLDFRATEND_CUP=grid%cldfratend_cup  & !CuP, wig 18-Sep-2006
L01611       &             ,SHALL=grid%shall, TAUCLOUD=grid%taucloud, TACTIVE=grid%tactive       & !CuP, wig 18-Sep-2006
L01612       &             ,TSTAR=grid%tstar, LNTERMS=grid%lnterms, LNINT=grid%lnint             & !CuP, wig 4-Oct-2006
L01613       &             ,ACTIVEFRAC=grid%activeFrac                           & !CuP, lkb
L01614       &             ,NUMBINS=config_flags%numBins                         & !CuP, wig
L01615       &             ,THBINSIZE=config_flags%thBinSize                     & !CuP, wig
L01616       &             ,RBINSIZE=config_flags%rBinSize                       & !CuP, wig
L01617       &             ,MINDEEPFREQ=config_flags%minDeepFreq                 & !CuP, wig
L01618       &             ,MINSHALLOWFREQ=config_flags%minShallowFreq           & !CuP, wig
L01619       &             ,WCLOUDBASE=grid%wCloudBase                           & !CuP, lkb
L01620       &             ,WACT_CUP=grid%wact_cup                               & !CuP, rce 25-aug-2011
L01621       &             ,WULCL_CUP=grid%wulcl_cup                             & !CuP, rce 23-jan-2012
L01622       &             ,WUP_CUP=grid%wup_cup                                 & !CuP, rce 15-mar-2013
↪  !BSINGH(12/05/2013)
L01623       &             ,QC_IC_CUP=grid%qc_ic_cup                             & !CuP, rce 29-aug-2011
L01624       &             ,QNDROP_IC_CUP=grid%qndrop_ic_cup                     & !CuP, rce 29-aug-2011
L01625       &             ,QC_IU_CUP=grid%qc_iu_cup                             & !CuP, rce 08-feb-2012
L01626       &             ,FCVT_QC_TO_PR_CUP=grid%fcvt_qc_to_pr_cup             & !CuP, rce 12-apr-2012
L01627       &             ,FCVT_QC_TO_QI_CUP=grid%fcvt_qc_to_qi_cup             & !CuP, rce 12-apr-2012
L01628       &             ,FCVT_QI_TO_PR_CUP=grid%fcvt_qi_to_pr_cup             & !CuP, rce 12-apr-2012
L01629       &             ,MFUP_CUP=grid%mfup_cup                               & !CuP, rce 23-jan-2012
L01630       &             ,MFUP_ENT_CUP=grid%mfup_ent_cup                       & !CuP, rce 23-jan-2012
L01631       &             ,MFDN_CUP=grid%mfdn_cup                               & !CuP, rce 12-apr-2012
L01632       &             ,MFDN_ENT_CUP=grid%mfdn_ent_cup                       & !CuP, rce 12-apr-2012
L01633       &             ,UPDFRA_CUP=grid%updfra_cup                           & !CuP, rce 23-jan-2012
L01634       &             ,TCLOUD_CUP=grid%tcloud_cup                           & !CuP, rce 06-feb-2012
L01635       !BSINGH -ENDS
L01636       &             ,k22_shallow=grid%k22_shallow,kbcon_shallow=grid%kbcon_shallow &
L01637       &             ,ktop_shallow=grid%ktop_shallow,xmb_shallow=grid%xmb_shallow   &
L01638       &             ,ktop_deep=grid%ktop_deep                             &
L01639       &             ,PERIODIC_X=(config_flags%polar .OR. config_flags%periodic_x)  &
L01640       &             ,PERIODIC_Y=config_flags%periodic_y                   &
L01641       &             ,IS_CAMMGMP_USED = grid%is_CAMMGMP_used  & !BSINGH - TKE at the interfaces for Zhang-McFarlane
↪  Scheme
L01642                   ! Zhang-McFarlane outputs
L01643       &             ,EVAPCDP3D=grid%evapcdp3d, ICWMRDP3D=grid%icwmrdp3d    & !Balwinder.Singh@pnnl.gov: For CAM's
↪  wetscavenging
L01644       &             ,RPRDDP3D=grid%rprddp3d                               &
L01645       &             ,CAPE=grid%cape ,ZMMU=grid%zmmu ,ZMMD=grid%zmmd       &
L01646       &             ,ZMDT=grid%zmdt ,ZMDQ=grid%zmdq                       &
L01647       &             ,DLF=grid%dlf, RLIQ=grid%rliq                         &
L01648       &             ,PCONVB=grid%pconvb, PCONVT=grid%pconvt               &
L01649       &             ,EVAPTZM=grid%evaptzm, FZSNTZM=grid%fzsntzm, EVSNTZM=grid%evsntzm     &
L01650       &             ,EVAPQZM=grid%evapqzm, ZMFLXPRC=grid%zmflxprc         &
L01651       &             ,ZMFLXSNW=grid%zmflxsnw, ZMNTPRPD=grid%zmntprpd       &
L01652       &             ,ZMNTSNPD=grid%zmntsnpd, ZMEIHEAT=grid%zmeiheat       &
L01653       &             ,CMFMC=grid%cmfmc, CMFMCDZM=grid%cmfmcdzm             &
L01654       &             ,PRECCDZM=grid%preccdzm, PRECZ=grid%precz             &
L01655       &             ,ZMMTU=grid%zmmtu, ZMMTV=grid%zmmtv  &
L01656       &             ,ZMUPGU=grid%zmupgu, ZMUPGD=grid%zmupgd               &
L01657       &             ,ZMVPGU=grid%zmvpgu, ZMVPGD=grid%zmvpgd               &
L01658       &             ,ZMICUU=grid%zmicuu, ZMICUD=grid%zmicud               &
L01659       &             ,ZMICVU=grid%zmicvu, ZMICVD=grid%zmicvd               &
L01660       &             ,ZMDICE=grid%zmdice, ZMDLIQ=grid%zmdliq               &
L01661       &             ,dp3d=grid%dp3d, du3d=grid%du3d, ed3d=grid%ed3d       &
L01662       &             ,eu3d=grid%eu3d, md3d=grid%md3d, mu3d=grid%mu3d       &
L01663       &             ,dsubcld2d=grid%dsubcld2d,ideep2d=grid%ideep2d        &
L01664       &             ,jt2d=grid%jt2d,maxg2d=grid%maxg2d                    &
L01665       &             ,lengath2d=grid%lengath2d                             &
L01666                   ! Selection flag
L01667       &             ,pgcon=config_flags%sas_pgcon                         &
L01668       &             ,BMJ_RAD_FEEDBACK=config_flags%bmj_rad_feedback       &
L01669       &             ,CU_PHYSICS=config_flags%cu_physics                   &
L01670       &             ,BL_PBL_PHYSICS=config_flags%bl_pbl_physics           &
L01671       &             ,SF_SFCLAY_PHYSICS=config_flags%sf_sfclay_physics     &
L01672       !BSINGH - For WRFCuP scheme
L01673       &             ,SHCU_AERosOLS_OPT=config_flags%shcu_aerosols_opt           & !CuP, rce 22-aug-2011
L01674       !BSINGH -ENDS
L01675       &             ,KFETA_TRIGGER=config_flags%kfeta_trigger             &
L01676       &             ,NSAS_DX_FACTOR=config_flags%nsas_dx_factor           &
L01677                   ! Dimension arguments
L01678       &             ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde    &
L01679       &             ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme    &
L01680       &             ,IPS=ips,IPE=ipe, JPS=jps,JPE=jpe, KPS=kps,KPE=kpe    &
L01681       &             ,I_START=grid%i_start,I_END=min(grid%i_end, ide-1)     &
L01682       &             ,J_START=grid%j_start,J_END=min(grid%j_end, jde-1)     &
L01683       &             ,KTS=k_start, KTE=min(k_end,kde-1)                    &
L01684       &             ,NUM_TILES=grid%num_tiles                             &
L01685                   ! Moisture tendency arguments
L01686       &             ,RQVCUTEN=grid%rqvcuten , RQCCUTEN=grid%rqccuten      &
L01687       &             ,RQSCUTEN=grid%rqscuten , RQICUTEN=grid%rqicuten      &
L01688       &             ,RQRCUTEN=grid%rqrcuten , RQCNCUTEN=grid%rqcncuten    &
L01689       &             ,RQINCUTEN=grid%rqincuten                             &
L01690       &             ,RQVBLTEN=grid%rqvblten , RQVFTEN=grid%rqvften        &
L01691                   ! Other tendency arguments
L01692       &             ,RTHRATEN=grid%rthraten , RTHBLTEN=grid%rthblten      &
L01693       &             ,RUCUTEN=grid%rucuten   , RVCUTEN=grid%rvcuten        &
L01694       &             ,RTHCUTEN=grid%rthcuten , RTHFTEN=grid%rthften        &
L01695                   ! Moisture tracer arguments
L01696       &             ,QV_CURR=moist(ims,kms,jms,P_QV), F_QV=F_QV           &
L01697       &             ,QC_CURR=moist(ims,kms,jms,P_QC), F_QC=F_QC           &
L01698       &             ,QR_CURR=moist(ims,kms,jms,P_QR), F_QR=F_QR           &
L01699       &             ,QI_CURR=moist(ims,kms,jms,P_QI), F_QI=F_QI           &
L01700       &             ,QS_CURR=moist(ims,kms,jms,P_QS), F_QS=F_QS           &
L01701       &             ,QG_CURR=moist(ims,kms,jms,P_QG), F_QG=F_QG           &
L01702       &             ,ZOL=grid%ZOL                                         & !ckay
```

```
L01703  ! Variables for Tiedtke and NSAS schemes
L01704        &             ,ZNU=grid%znu                                     &
L01705        &             ,MP_PHYSICS=config_flags%mp_physics               &
L01706        &             ,GD_CLOUD=grid%GD_CLOUD,GD_CLOUD2=grid%GD_CLOUD2  &
L01707  #if (WRF_CHEM == 1)
L01708        &             ,CHEM_OPT=config_flags%chem_opt                        & !CuP, rce 22-aug-2011 !BSINGH - For
↪  WRFCuP scheme
L01709        &             ,CONV_TR_WETSCAV=config_flags%conv_tr_wetscav     &
L01710        &             ,CONV_TR_AQCHEM=config_flags%conv_tr_aqchem       &
L01711        &             ,CHEM_CONV_TR=config_flags%chem_conv_tr           &
L01712        &             ,TRACEROPT=config_flags%tracer_opt                &
L01713  #endif
L01714
L01715  #if ( WRF_DFI_RADAR == 1 )
L01716        &             ,DO_CAPSUPPRESS=do_capsupress                     &
L01717  #endif
L01718        &             ,cfu1=grid%cfu1,cfd1=grid%cfd1,dfu1=grid%dfu1,efu1=grid%efu1,dfd1=grid%dfd1,efd1=grid%efd1,
↪  f_flux=l_flux &
L01719                      ,alevsiz_cu=grid%alevsiz_cu,num_months=grid%num_months          &
L01720                      ,no_src_types_cu=grid%no_src_types_cu                           &
L01721                      ,aercu_opt=config_flags%aercu_opt                               &
L01722                      ,aercu_fct=config_flags%aercu_fct                               &
L01723                      ,aeromcu=grid%aeromcu,aerocu=aerocu(:,:,:,P_cu_sulfate:P_cu_phibcar) &
L01724                      ,aeropcu=grid%aeropcu,ID=grid%id                                &
L01725                      ,JULDAY=grid%julday, JULIAN=grid%julian                         &
L01726                      ,aerovar=grid%aerovar,EFCS=grid%EFCS,EFIS=grid%EFIS,EFSS=grid%EFSS)
L01727  BENCH_END(cu_driver_tim)
L01728  !
L01729  ! this for calculating (G3 scheme only)  time averaged variables for online (WRF-CHem) or offline (other models) chem
↪  runs
L01730  !
L01731       if(config_flags%cu_diag.eq.1)then
L01732        !$OMP PARALLEL DO   &
L01733        !$OMP PRIVATE ( ij )
L01734        DO ij = 1 , grid%num_tiles
L01735             call convtrans_prep(grid%gd_cloud,grid%gd_cloud2,grid%gd_cloud_a,&
L01736       &          grid%QC_CU,grid%raincv,grid%raincv_a,grid%raincv_b,     &
L01737       &          grid%gd_cldfr,moist,p_QV,p_QC,p_qi,T_PHY,P_PHY,num_moist,    &
L01738       &          grid%gd_cloud2_a,grid%QI_CU,grid%convtrans_avglen_m,&
L01739       &          adapt_step_flag,curr_secs,                  &
L01740       &          grid%itimestep,grid%dt,                                  &
L01741       &          config_flags%cu_physics,                                 &
L01742       &          ids,ide, jds,jde, kds,kde,                               &
L01743       &          ims,ime, jms,jme, kms,kme                                &
L01744       &          ,ITS=grid%i_start(ij),ITE=min(grid%i_end(ij), ide-1)           &
L01745       &          ,JTS=grid%j_start(ij),JTE=min(grid%j_end(ij), jde-1)           &
L01746       &          ,KTS=k_start, KTE=min(k_end,kde-1))
L01747        ENDDO
L01748        !$OMP END PARALLEL DO
L01749       endif
L01750
```

讲解：cumulus_driver 实参段——关键项：**RTHRATEN/RTHBLTEN/RQVBLTEN 与 RTHFTEN/RQVFTEN 同时传入**（新 Tiedtke/G3 的闭合式要吃组合倾向，code_23 § 六）、forcet/forceq（SCM 强迫同入口）、kfeta_trigger/NSAS_DX_FACTOR 等 namelist 开关、XENSLV 系集合成员数组、CFU1 系通量诊断；**段尾 convtrans_prep（L1735）**——把 GD/G3 的云水/云冰（gd_cloud 系）整理给辐射的 CuP 云量，注意它在 cumulus 之后：**先算对流再供辐射**，与主循环里辐射在前的顺序互补（辐射用的是上一步的积云云量，code_20 § 八的 CUTOP/CUBOT 入口）。下游见 `code_23_module_cumulus_driver.md`。

十、浅对流与 SCM 强迫

```
!file: dyn_em/module_first_rk_step_part1.F    L1752-L1866
L01752       CALL wrf_debug ( 200 , ' call shallow_cumulus_driver' )
L01753
L01754  BENCH_START(shcu_driver_tim)
L01755       CALL shallowcu_driver(                                           &
L01756       &             IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde     &
L01757       &            ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme     &
L01758       &            ,IPS=ips,IPE=ipe, JPS=jps,JPE=jpe, KPS=kps,KPE=kpe     &
L01759       &            ,I_START=grid%i_start, I_END=min(grid%i_end, ide-1)    &
L01760       &            ,J_START=grid%j_start, J_END=min(grid%j_end, jde-1)    &
L01761       &            ,KTS=k_start, KTE=min(k_end, kde-1)                    &
L01762       &            ,NUM_TILES=grid%num_tiles                              &
L01763       &            ,U=grid%u_phy, V=grid%v_phy, TH=th_phy, T=t_phy        &
L01764       &            ,P=grid%p_hyd, PI=pi_phy, RHO=grid%rho, MOIST=moist    &
L01765       &            ,NUM_MOIST=num_moist                                   &
L01766       &            ,ITIMESTEP=grid%itimestep, DT=grid%dt, DX=grid%dx      &
L01767       &            ,DX2D=grid%dx2d, AREA2D=grid%area2d                    &
L01768       &            ,CUDT=grid%cudt                                        &
L01769       &            ,CURR_SECS=curr_secs, ADAPT_STEP_FLAG=adapt_step_flag  &
L01770       &            ,RAINSH=grid%rainsh, PRATESH=grid%pratesh, NCA=grid%nca&
L01771       &            ,RAINSHV=grid%rainshv                                  &
L01772       &            ,Z=grid%z, Z_AT_W=grid%z_at_w, DZ8W=dz8w               &
L01773       &            ,MAVAIL=grid%mavail, PBLH=grid%pblh, P8W=grid%p_hyd_w  &
L01774       &            ,TKE_PBL=grid%tke_pbl                                  &
L01775       &            ,CLDFRA=grid%cldfra, CLDFRA_OLD=grid%cldfra_old        &
L01776       &            ,CLDFRA_OLD_MP=grid%cldfra_old_mp                      &
L01777       &            ,CLDFRA_CONV=grid%cldfra_conv                          &
L01778       &            ,CLDFRASH=grid%cldfrash, HTOP=grid%htop, HBOT=grid%hbot&
L01779       &            ,SHCU_PHYSICS=grid%shcu_physics                        &
L01780       &            ,QV_CURR=moist(ims,kms,jms,P_QV)                       &
L01781       &            ,QC_CURR=moist(ims,kms,jms,P_QC)                       &
L01782       &            ,QR_CURR=moist(ims,kms,jms,P_QR)                       &
L01783       &            ,QI_CURR=moist(ims,kms,jms,P_QI)                       &
L01784       &            ,QS_CURR=moist(ims,kms,jms,P_QS)                       &
```

```
L01785       &             ,QG_CURR=moist(ims,kms,jms,P_QG)                     &
L01786       &             ,QNC_CURR=scalar(ims,kms,jms,P_QNC)                  & !BSINGH - Neede for UWSHCU scheme
L01787       &             ,QNI_CURR=scalar(ims,kms,jms,P_QNI)                  & !BSINGH - Neede for UWSHCU schem
L01788  #if (WRF_CHEM == 1)
L01789       &             ,CHEM=chem,chem_opt=config_flags%chem_opt            &
L01790  #endif
L01791       &             ,DLF=grid%dlf, RLIQ=grid%rliq, RLIQ2=grid%rliq2      &
L01792       &             ,DLF2=grid%dlf2                                      & ! Required for CAMMGMP microphysics scheme
L01793       &             ,CMFMC=grid%cmfmc, CMFMC2=grid%cmfmc2                &
L01794       &             ,CUSH=grid%cush, SNOWSH=grid%snowsh                  &
L01795       &             ,ICWMRSH=grid%icwmrsh, RPRDSH=grid%rprdsh            &
L01796       &             ,CBMF=grid%cbmf_cu, CMFSL=grid%cmfsl, CMFLQ=grid%cmflq &
L01797       &             ,EVAPCSH=grid%evapcsh                                &
L01798       &             ,RQVSHTEN=grid%rqvshten, RQCSHTEN=grid%rqcshten      &
L01799       &             ,RQRSHTEN=grid%rqrshten, RQISHTEN=grid%rqishten      &
L01800       &             ,RQSSHTEN=grid%rqsshten, RQGSHTEN=grid%rqgshten      &
L01801       &             ,RQCNSHTEN=grid%rqcnshten, RQINSHTEN=grid%rqinshten  &
L01802       &             ,RQVBLTEN=grid%rqvblten, RQVFTEN=grid%rqvften        &
L01803       &             ,RUSHTEN=grid%rushten, RVSHTEN=grid%rvshten          &
L01804       &             ,RTHSHTEN=grid%rthshten, RTHRATEN=grid%rthraten      &
L01805       &             ,RTHBLTEN=grid%rthblten, RTHFTEN=grid%rthften        &
L01806       &             ,F_QV=f_qv,F_QC=f_qc,F_QR=f_qr                       &
L01807       &             ,F_QI=f_qi,F_QS=f_qs,F_QG=f_qg                       &
L01808       &             ,HT=grid%ht                                          &
L01809       &             ,SHFRC3D=grid%shfrc3d                                & !Balwinder.Singh@pnnl.gov: For CAM's
↪  wetscavenging
L01810       &             ,IS_CAMMGMP_USED = grid%is_CAMMGMP_used              &
L01811       &             ,WSTAR=grid%wstar_ysu,DELTA=grid%delta_ysu           &
L01812       &             ,KPBL=grid%kpbl,ZNU=grid%znu                         &
L01813       &             ,RAINCV=grid%raincv                                  &
L01814       &             ,W=grid%w_2 ,XLAND=grid%xland                        &
L01815       &             ,HFX=grid%hfx, QFX=grid%qfx                          &
L01816       &             ,MP_PHYSICS=config_flags%mp_physics                  &
L01817       &             ,pgcon=config_flags%sas_pgcon                        &
L01818       &             ,RDCASHTEN=grid%RDCASHTEN, RQCDCSHTEN=grid%RQCDCSHTEN  & ! Deng sh
L01819       &             ,W0AVG=grid%W0AVG                                    &
L01820       &             ,clddpthb=grid%clddpthb, cldtopb=grid%cldtopb        &
L01821       &             ,cldareaa=grid%cldareaa, cldareab=grid%cldareab      &
L01822       &             ,cldliqa=grid%cldliqa, cldliqb=grid%cldliqb          &
L01823       &             ,cldfra_sh=grid%cldfra_sh,ca_rad=grid%ca_rad,  cw_rad=grid%cw_rad               &
L01824       &             ,wub=grid%wub, pblmax=grid%pblmax, xlong=grid%xlong  &
L01825       &             ,rainshvb=grid%rainshvb, capesave=grid%capesave      &
L01826       &             ,radsave=grid%radsave, ainckfsa=grid%ainckfsa        &
L01827       &             ,ltopb=grid%ltopb, kdcldtop=grid%kdcldtop            &
L01828       &             ,kdcldbas=grid%kdcldbas                              &
L01829       &             ,el_pbl=grid%el_pbl                                  &
L01830       &             ,rthratenlw=grid%rthratenlw, rthratensw=grid%rthratensw &
L01831       &             ,exch_h=grid%exch_h                                  &
L01832       &             ,dnw=grid%dnw, XTIME=grid%XTIME, XTIME1=grid%XTIME1  &
L01833       &             ,GMT=grid%gmt                                        &
L01834       &             ,qke=grid%qke                                        &
L01835       &             ,PBLHAVG=grid%PBLHAVG, TKEAVG=grid%TKEAVG            &
L01836       &             ,BL_PBL_PHYSICS=config_flags%bl_pbl_physics          &
L01837     ! WRF-Solar EPS
L01838       &             ,multi_perturb=config_flags%multi_perturb            &
L01839       &             ,pert_deng=config_flags%pert_deng                    &
L01840       &             ,perts_qvapor=grid%pert3d(:,:,:,P_PQVAPOR)           &
L01841       &             ,perts_qcloud=grid%pert3d(:,:,:,P_PQCLOUD)           &
L01842       &             ,perts_th=grid%pert3d(:,:,:,P_PTH)                   &
L01843       &             ,perts_w=grid%pert3d(:,:,:,P_PW)                     &
L01844       &             ,pert_deng_qv=config_flags%pert_deng_qv              &
L01845       &             ,pert_deng_qc=config_flags%pert_deng_qc              &
L01846       &             ,pert_deng_t=config_flags%pert_deng_t                &
L01847       &             ,pert_deng_w=config_flags%pert_deng_w                &
L01848       &                                                                  )
L01849
L01850  #if (WRF_CHEM == 1)
L01851  #ifdef DM_PARALLEL
L01852        IF( config_flags%shcu_physics == CAMUWSHCUSCHEME ) THEN
L01853           CALL wrf_debug ( 200 , ' call HALO CHEM AFTER SHALLOW CUMULUS' )
L01854           IF      ( config_flags%h_sca_adv_order <= 4 ) THEN
L01855  #      include "HALO_EM_CHEM_E_3.inc"
L01856           ELSE IF ( config_flags%h_sca_adv_order <= 6 ) THEN
L01857  #      include "HALO_EM_CHEM_E_5.inc"
L01858           ELSE
L01859              WRITE(message,*)'module_first_rk_step_part1: invalid h_sca_adv_order = ',config_flags%h_sca_adv_order
L01860              CALL wrf_error_fatal(TRIM(message))
L01861           ENDIF
L01862        ENDIF
L01863  #endif
L01864  #endif
L01865  BENCH_END(shcu_driver_tim)
L01866
```

讲解：shallowcu_driver 是**独立于 cumulus_driver 的第二积云通道**（shcu_physics 方案号独立——与主积云可同时开，教材 ch15 的「独立浅对流」；G3 的浅对流则在 cumulus 内部，两套并存注意别叠加）。force_scm：单柱模式（SCM）的强迫倾向注入（itimestep/dt 键字实参）。

十一、格点 nudging 收尾

```
!file: dyn_em/module_first_rk_step_part1.F    L1955-L2025
L01955        CALL wrf_debug ( 200 , ' call fddagd_driver' )
L01956
L01957  BENCH_START(fdda_driver_tim)
L01958        CALL fddagd_driver(itimestep=grid%itimestep,dt=grid%dt,xtime=grid%XTIME,      &
L01959                     curr_mins2=curr_mins2, id=grid%id,                              &
L01960                     RUNDGDTEN=grid%rundgdten,RVNDGDTEN=grid%rvndgdten,              &
L01961                     RTHNDGDTEN=grid%rthndgdten,RPHNDGDTEN=grid%rphndgdten,          &
L01962                     RQVNDGDTEN=grid%rqvndgdten,RMUNDGDTEN=grid%rmundgdten,          &
L01963  !
```

```
L01964  ! FASDAS
L01965  !
L01966                    SDA_HFX=grid%SDA_HFX, SDA_QFX=grid%SDA_QFX,      &
L01967                    HFX_FDDA=grid%HFX_FDDA,  &
L01968  !
L01969  ! END FASDAS
L01970  !
L01971                    u_ndg_old=fdda3d(ims,kms,jms,P_u_ndg_old),                        &
L01972                    v_ndg_old=fdda3d(ims,kms,jms,P_v_ndg_old),                        &
L01973                    t_ndg_old=fdda3d(ims,kms,jms,P_t_ndg_old),                        &
L01974                    ph_ndg_old=fdda3d(ims,kms,jms,P_ph_ndg_old),                      &
L01975                    q_ndg_old=fdda3d(ims,kms,jms,P_q_ndg_old),                        &
L01976                    mu_ndg_old=fdda2d(ims,1,jms,P_mu_ndg_old),                        &
L01977                    u_ndg_new=fdda3d(ims,kms,jms,P_u_ndg_new),                        &
L01978                    v_ndg_new=fdda3d(ims,kms,jms,P_v_ndg_new),                        &
L01979                    t_ndg_new=fdda3d(ims,kms,jms,P_t_ndg_new),                        &
L01980                    ph_ndg_new=fdda3d(ims,kms,jms,P_ph_ndg_new),                      &
L01981                    q_ndg_new=fdda3d(ims,kms,jms,P_q_ndg_new),                        &
L01982                    mu_ndg_new=fdda2d(ims,1,jms,P_mu_ndg_new),                        &
L01983                    u3d=grid%u_2,v3d=grid%v_2,th_phy=th_phy,                          &
L01984                    ph=grid%ph_2,rho=grid%rho,moist=moist,                            &
L01985                    p_phy=p_phy,pi_phy=pi_phy,p8w=p8w,t_phy=grid%t_phy,               &
L01986                    dz8w=dz8w,z=grid%z,z_at_w=grid%z_at_w,                            &
L01987                    grid=grid,config_flags=config_flags,dx=grid%DX,n_moist=num_moist, &
L01988                    STEPFG=grid%STEPFG,                                               &
L01989                    pblh=grid%pblh,ht=grid%ht,REGIME=grid%regime,ZNT=grid%znt         &
L01990                     ,IDS=ids,IDE=ide, JDS=jds,JDE=jde, KDS=kds,KDE=kde               &
L01991                     ,IMS=ims,IME=ime, JMS=jms,JME=jme, KMS=kms,KME=kme               &
L01992                     ,I_START=grid%i_start,I_END=min(grid%i_end, ide-1)               &
L01993                     ,J_START=grid%j_start,J_END=min(grid%j_end, jde-1)               &
L01994                     ,KTS=k_start, KTE=min(k_end,kde-1)                              &
L01995                     , num_tiles=grid%num_tiles,                                     &
L01996                     u10=grid%u10, v10=grid%v10, th2=grid%th2, q2=grid%q2,           &
L01997                     u10_ndg_old=grid%u10_ndg_old,                    &
L01998                     v10_ndg_old=grid%v10_ndg_old,                    &
L01999                     t2_ndg_old=grid%t2_ndg_old,                      &
L02000                     th2_ndg_old=grid%th2_ndg_old,                    &
L02001                     q2_ndg_old=grid%q2_ndg_old,                      &
L02002                     rh_ndg_old=grid%rh_ndg_old,                      &
L02003                     psl_ndg_old=grid%psl_ndg_old,                    &
L02004                     ps_ndg_old=grid%ps_ndg_old,                      &
L02005                     tob_ndg_old=grid%tob_ndg_old,                                   &
L02006                     odis_ndg_old=grid%odis_ndg_old,                                 &
L02007                     u10_ndg_new=grid%u10_ndg_new,                    &
L02008                     v10_ndg_new=grid%v10_ndg_new,                    &
L02009                     t2_ndg_new=grid%t2_ndg_new,                      &
L02010                     th2_ndg_new=grid%th2_ndg_new,                    &
L02011                     q2_ndg_new=grid%q2_ndg_new,                      &
L02012                     rh_ndg_new=grid%rh_ndg_new,                      &
L02013                     psl_ndg_new=grid%psl_ndg_new,                    &
L02014                     ps_ndg_new=grid%ps_ndg_new,                      &
L02015                     tob_ndg_new=grid%tob_ndg_new,                                   &
L02016                     odis_ndg_new=grid%odis_ndg_new                                  &
L02017                     ,IPS=ips,IPE=ipe, JPS=jps,JPE=jpe, KPS=kps,KPE=kpe              &
L02018                     ,IMSX=imsx,IMEX=imex,JMSX=jmsx,JMEX=jmex,KMSX=kmsx,KMEX=kmex    &
L02019                     ,IPSX=ipsx,IPEX=ipex,JPSX=jpsx,JPEX=jpex,KPSX=kpsx,KPEX=kpex    &
L02020                     ,IMSY=imsy,IMEY=imey,JMSY=jmsy,JMEY=jmey,KMSY=kmsy,KMEY=kmey    &
L02021                     ,IPSY=ipsy,IPEY=ipey,JPSY=jpsy,JPEY=jpey,KPSY=kpsy,KPEY=kpey    )
L02022
L02023  BENCH_END(fdda_driver_tim)
L02024
L02025    END SUBROUTINE first_rk_step_part1
```

讲解：fddagd_driver——格点松弛逼近（grid nudging）的驱动，吃 fdda3d/fdda2d 态与 nudging 系数（XMASS/XMASS_V 等）；放总装末尾的意义：nudging 是「观测对模式的拉回」，加在所有物理倾向之后、动力倾向之前——物理不 nudging 自身，nudging 修正进动力方程。

十二、改造入口

1. **加第七个物理驱动器**（如大气化学在线耦合到主循环）：USE + 本文件插段 + solve_em 传参——本文件就是「非时间分裂物理」的定义处。
2. **调用序实验**：交换 radiation/surface 顺序等——注意积云 → 辐射的云量时序（§ 九）与 pbl→surface 的交换系数依赖（exch_temf）。
3. **倾向记账**：init_zero_tendency 的 rk_step=1——把某物理倾向改成跨步累积的挂点。
4. **子格并行**：k_start/k_end 的纵向分块——GPU/OpenMP 移植的现成结构。

十三、上下游链接

- 上游调用：`code_50_solve_em.md`（主循环位置）；下半：`code_32_first_rk_step_part2.md`
- 五大驱动器：`code_20_module_radiation_driver.md`、`code_24_module_surface_driver.md`、`code_22_module_pbl_driver.md`、`code_23_module_cumulus_driver.md`
- 教材正文：ch21_solve_em_dynamics.md、ch20_modifying_wrf.md

code_32 — 非时间分裂物理总装·下（dyn_em/module_first_rk_step_part2.F：倾向记账、TKE 混合与随机扰动）

一、版本指纹

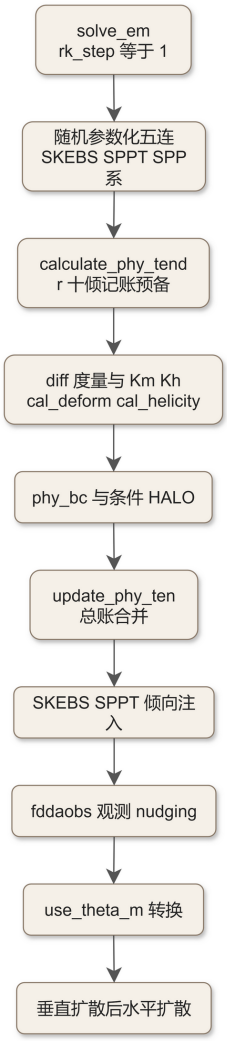
源文件	行数	md5（整文件，按字节）
dyn_em/module_first_rk_step_part2.F	1132	10f3fc73e862a80a7330437dd53d4783

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：十段精选全收（随机参数化五连、倾向记账、扩散度量与 Km/Kh、条件 HALO、倾向合并、观测 nudging、use_theta_m 转换、垂直 +水平扩散）；主签名与声明区引用不录。上游：`code_31_module_first_rk_step_part1.md`。

二、定位

一句话：总装下半——**没有微物理**（微物理由 solve_em 在 RK 循环后直接调，ch21_solve_em_dynamics.md），本文件做四件事：① 把各物理方案写出的 r*ten 数组**记账进** ru_tendf 系总账（calculate_phy_tend + update_phy_ten）；② 算次格距混合的全部原料（变形率/散度/TKE/Km/Kh）并在无 PBL 方案时补垂直扩散；③SKEBS/SPPT/SPP 随机扰动的生成与注入；④obs nudging 与 use_theta_m 的倾向口径转换。

谁在调它：solve_em 主循环、rk_step==1 时（与 part1 配对）。教材锚点：ch21_solve_em_dynamics.md。



三、USE 链

```
!file: dyn_em/module_first_rk_step_part2.F    L36-L64
L00036      USE module_state_description
L00037      USE module_model_constants
L00038      USE module_domain, ONLY : domain
L00039      USE module_configure, ONLY : grid_config_rec_type, model_config_rec
L00040      USE module_big_step_utilities_em, ONLY : conv_t_tendf_to_moist
L00041  #ifdef DM_PARALLEL
L00042      USE module_dm, ONLY : local_communicator, mytask, ntasks, ntasks_x, ntasks_y, local_communicator_periodic, &
L00043                            wrf_dm_maxval, wrf_err_message, local_communicator_x, local_communicator_y
L00044      USE module_comm_dm, ONLY : halo_em_tke_c_sub,halo_em_tke_d_sub,halo_em_tke_e_sub            &
L00045             ,halo_em_phys_pbl_sub,halo_em_phys_shcu_sub &
L00046             ,halo_em_fdda_sub,halo_em_phys_diffusion_sub,halo_em_tke_3_sub &
L00047             ,halo_em_tke_5_sub,halo_obs_nudge_sub,period_bdy_em_a1_sub,period_bdy_em_phy_bc_sub &
L00048             ,period_bdy_em_fdda_bc_sub,period_bdy_em_chem_sub,halo_em_phys_cu_sub,halo_em_helicity_sub
L00049  #endif
L00050
L00051      USE module_driver_constants
L00052      USE module_diffusion_em, ONLY : phy_bc, cal_deform_and_div, compute_diff_metrics, &
L00053                                      vertical_diffusion_2, horizontal_diffusion_2, calculate_km_kh, &
L00054                                      tke_rhs, cal_helicity, &
L00055                                      nonlocal_flux,meso_length_scale,free_atmos_length, &
L00056                                      vertical_diffusion_implicit !XZ
L00057      USE module_em, ONLY : calculate_phy_tend
L00058      USE module_fddaobs_driver, ONLY : fddaobs_driver
L00059      USE module_bc, ONLY : set_physical_bc3d, set_physical_bc2d
L00060      USE module_physics_addtendc, ONLY : update_phy_ten
L00061
L00062      USE module_sfs_driver !JDM
L00063      USE module_stoch, ONLY : update_stoch_ten, perturb_physics_tend, RAND_PERT_UPDATE, contiguize_2d
L00064
```

讲解：本文件的「工具箱」清单：module_diffusion_em 六件（phy_bc/cal_deform_and_div/compute_diff_metrics/vertical_diffusion_2/horizontal_diffusion_2/calculate_km_kh/tke_rhs/cal_helicity + km_opt=5 的 nonlocal_flux/meso_length_scale/vertical_diffusion_implicit）；module_em 的 calculate_phy_tend（倾向记账）；fddaobs_driver（观测 nudging）；physics_addtendc 的 update_phy_ten（总账合并）；**sfs_driver（JDM 的 SFS 湍流模式）与 module_stoch（SKEBS/SPPT 的随机引擎）**；halo 子程序 14 种。

四、随机参数化五连

```
!file: dyn_em/module_first_rk_step_part2.F    L153-L381
L00153     rk_step = 1
L00154
L00155        IF ((grid%skebs_on==1).and.(grid%id .EQ. 1 )) then
L00156            ! update and backtransform T
L00157            CALL RAND_PERT_UPDATE(grid,'T',                                  &
L00158                                  grid%SPTFORCS,grid%SPTFORCC,               &
L00159                                  grid%SPT_AMP,grid%ALPH_T,                  &
L00160                                  ips, ipe, jps, jpe, kps, kpe,              &
L00161                                  ids, ide, jds, jde, kds, kde,              &
L00162                                  ims, ime, jms, jme, kms, kme,              &
L00163                                  k_start, k_end,                            &
L00164                                  imsx,imex,jmsx,jmex,kmsx,kmex,             &
L00165                                  ipsx,ipex,jpsx,jpex,kpsx,kpex,             &
L00166                                  imsy,imey,jmsy,jmey,kmsy,kmey,             &
L00167                                  ipsy,ipey,jpsy,jpey,kpsy,kpey,             &
L00168                                  grid%num_stoch_levels,grid%num_stoch_levels,       &
L00169                                  grid%num_stoch_levels,grid%num_stoch_levels,       &
L00170                                  config_flags%restart, grid%iseedarr_skebs,        &
L00171                                  config_flags%seed_dim,                            &
L00172                                  grid%DX,grid%DY,grid%skebs_vertstruc,             &
L00173                                  grid%rt_tendf_stoch,                              &
L00174                                  grid%stddev_cutoff_sppt,grid%gridpt_stddev_sppt, &
L00175                                  grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPUV     )
L00176            ! Update streamfunction, backtransform U
L00177             CALL RAND_PERT_UPDATE(grid,'U',                                 &
L00178                                  grid%SPSTREAMFORCS,grid%SPSTREAMFORCC,     &
L00179                                  grid%SPSTREAM_AMP,grid%ALPH_PSI,           &
L00180                                  ips, ipe, jps, jpe, kps, kpe,              &
L00181                                  ids, ide, jds, jde, kds, kde,              &
L00182                                  ims, ime, jms, jme, kms, kme,              &
L00183                                  k_start, k_end,                            &
L00184                                  imsx,imex,jmsx,jmex,kmsx,kmex,             &
L00185                                  ipsx,ipex,jpsx,jpex,kpsx,kpex,             &
L00186                                  imsy,imey,jmsy,jmey,kmsy,kmey,             &
L00187                                  ipsy,ipey,jpsy,jpey,kpsy,kpey,             &
L00188                                  grid% num_stoch_levels,grid% num_stoch_levels,     &
L00189                                  grid% num_stoch_levels,grid% num_stoch_levels,     &
L00190                                  config_flags%restart, grid%iseedarr_skebs,        &
L00191                                  config_flags%seed_dim,                            &
L00192                                  grid%DX,grid%DY,grid%skebs_vertstruc,             &
L00193                                  grid%ru_tendf_stoch,                              &
L00194                                  grid%stddev_cutoff_sppt,grid%gridpt_stddev_sppt, &
L00195                                  grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPUV     )
L00196            ! Don't update streamfunction, backtransform V
L00197             CALL RAND_PERT_UPDATE(grid,'V',                                 &
L00198                                  grid%SPSTREAMFORCS,grid%SPSTREAMFORCC,     &
L00199                                  grid%SPSTREAM_AMP,grid%ALPH_PSI,           &
L00200                                  ips, ipe, jps, jpe, kps, kpe,              &
L00201                                  ids, ide, jds, jde, kds, kde,              &
L00202                                  ims, ime, jms, jme, kms, kme,              &
L00203                                  k_start, k_end,                            &
L00204                                  imsx,imex,jmsx,jmex,kmsx,kmex,             &
L00205                                  ipsx,ipex,jpsx,jpex,kpsx,kpex,             &
L00206                                  imsy,imey,jmsy,jmey,kmsy,kmey,             &
L00207                                  ipsy,ipey,jpsy,jpey,kpsy,kpey,             &
L00208                                  grid% num_stoch_levels,grid% num_stoch_levels,     &
L00209                                  grid% num_stoch_levels,grid% num_stoch_levels,     &
L00210                                  config_flags%restart, grid%iseedarr_skebs,        &
L00211                                  config_flags%seed_dim,                            &
L00212                                  grid%DX,grid%DY,grid%skebs_vertstruc,             &
L00213                                  grid%rv_tendf_stoch,                              &
L00214                                  grid%stddev_cutoff_sppt,grid%gridpt_stddev_sppt, &
L00215                                  grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT      )
L00216         ENDIF !skebs_on
L00217
L00218        if ((grid%sppt_on==1).and.(grid%id .EQ. 1 )) then
L00219            CALL RAND_PERT_UPDATE(grid,'T',                                  &
L00220                                  grid%SPPTFORCS,grid%SPPTFORCC,             &
L00221                                  grid%SPPT_AMP,grid%ALPH_SPPT,              &
L00222                                  ips, ipe, jps, jpe, kps, kpe,              &
L00223                                  ids, ide, jds, jde, kds, kde,              &
L00224                                  ims, ime, jms, jme, kms, kme,              &
L00225                                  k_start, k_end,                            &
L00226                                  imsx,imex,jmsx,jmex,kmsx,kmex,             &
L00227                                  ipsx,ipex,jpsx,jpex,kpsx,kpex,             &
L00228                                  imsy,imey,jmsy,jmey,kmsy,kmey,             &
L00229                                  ipsy,ipey,jpsy,jpey,kpsy,kpey,             &
L00230                                  grid%num_stoch_levels,grid%num_stoch_levels,      &
L00231                                  grid%num_stoch_levels,grid%num_stoch_levels,      &
L00232                                  config_flags%restart, grid%iseedarr_sppt,         &
L00233                                  config_flags%seed_dim,                            &
L00234                                  grid%DX,grid%DY,grid%sppt_vertstruc,              &
L00235                                  grid%rstoch,                                      &
L00236                                  grid%stddev_cutoff_sppt,grid%gridpt_stddev_sppt, &
L00237                                  grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT      )
L00238         ENDIF !sppt_on
L00239
L00240        if ((grid%rand_perturb_on==1).and.(grid%id .EQ. 1 )) then
L00241            CALL RAND_PERT_UPDATE(grid,'T',                                  &
L00242                                  grid%SPFORCS,grid%SPFORCC,                 &
L00243                                  grid%SP_AMP,grid%ALPH_RAND,                &
L00244                                  ips, ipe, jps, jpe, kps, kpe,              &
L00245                                  ids, ide, jds, jde, kds, kde,              &
L00246                                  ims, ime, jms, jme, kms, kme,              &
L00247                                  k_start, k_end,                            &
L00248                                  imsx,imex,jmsx,jmex,kmsx,kmex,             &
L00249                                  ipsx,ipex,jpsx,jpex,kpsx,kpex,             &
L00250                                  imsy,imey,jmsy,jmey,kmsy,kmey,             &
L00251                                  ipsy,ipey,jpsy,jpey,kpsy,kpey,             &
L00252                                  grid%num_stoch_levels,grid%num_stoch_levels,      &
L00253                                  grid%num_stoch_levels,grid%num_stoch_levels,      &
L00254                                  config_flags%restart, grid%iseedarr_rand_pert,    &
L00255                                  config_flags%seed_dim,                            &
L00256                                  grid%DX,grid%DY,grid%rand_pert_vertstruc,         &
```

```
                              grid%RAND_PERT,                                    &
                              grid%stddev_cutoff_rand_pert,grid%gridpt_stddev_rand_pert, &
                              grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT       )
      ENDIF !rand_perturb_on

      IF ( grid%stepsp > 0 ) THEN
         stepsp_test = ( MOD(grid%itimestep, grid%stepsp) == 0 )
      ELSE
         stepsp_test = .FALSE.
      END IF
      If_multi_perturb: if ((config_flags%multi_perturb==1).and.(grid%id == 1) .and. &
          (stepsp_test .or. grid%itimestep == 1)) then
        Loop_multi_perturb: DO n = 2, config_flags%num_pert_3d
          CALL contiguize_2d ( .TRUE. , grid%SPFORCS3d, SPFORCS3d_loc, n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
          CALL contiguize_2d ( .TRUE. , grid%SPFORCC3d, SPFORCC3d_loc, n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
          CALL contiguize_2d ( .TRUE. , grid%SP_AMP3d,  SP_AMP3d_loc,  n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
          CALL RAND_PERT_UPDATE(grid,'T',                                       &
                               SPFORCS3d_loc, SPFORCC3d_loc, SP_AMP3d_loc,      &
                               grid%ALPH_RAND3d(n),                             &
                               ips, ipe, jps, jpe, kps, kpe,                    &
                               ids, ide, jds, jde, kds, kde,                    &
                               ims, ime, jms, jme, kms, kme,                    &
                               k_start, k_end,                                  &
                               imsx,imex,jmsx,jmex,kmsx,kmex,                   &
                               ipsx,ipex,jpsx,jpex,kpsx,kpex,                   &
                               imsy,imey,jmsy,jmey,kmsy,kmey,                   &
                               ipsy,ipey,jpsy,jpey,kpsy,kpey,                   &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               config_flags%restart, grid%iseedarr_mult3d(:,n), &
                               config_flags%seed_dim,                           &
                               grid%DX,grid%DY,grid%mult3d_vertstruc(n),        &
                               grid%PERT3D(:,:,:,n),                            &
                               grid%stddev_cutoff_mult3d(n),                    &
                               grid%gridpt_stddev_mult3d(n),                    &
                               grid%VERTSTRUCC3d(:,:,:,n),                      &
                               grid%VERTSTRUCS3d(:,:,:,n),                      &
                               grid%VERTAMPT3d(:,n)                             )
          CALL contiguize_2d ( .FALSE., grid%SPFORCS3d, SPFORCS3d_loc, n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
          CALL contiguize_2d ( .FALSE., grid%SPFORCC3d, SPFORCC3d_loc, n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
          CALL contiguize_2d ( .FALSE., grid%SP_AMP3d,  SP_AMP3d_loc,  n,       &
                               ips, ipe, jps, jpe, 1, config_flags%num_pert_3d, &
                               ids, ide, jds, jde, 1, config_flags%num_pert_3d, &
                               ims, ime, jms, jme, 1, config_flags%num_pert_3d  )
        END DO Loop_multi_perturb
      ENDIF If_multi_perturb

      if ((grid%spp_conv==1).and.(grid%id .EQ. 1 )) then
          CALL RAND_PERT_UPDATE(grid,'T',                                       &
                               grid%SPFORCS2,grid%SPFORCC2,                     &
                               grid%SP_AMP2,grid%ALPH_RAND2,                    &
                               ips, ipe, jps, jpe, kps, kpe,                    &
                               ids, ide, jds, jde, kds, kde,                    &
                               ims, ime, jms, jme, kms, kme,                    &
                               k_start, k_end,                                  &
                               imsx,imex,jmsx,jmex,kmsx,kmex,                   &
                               ipsx,ipex,jpsx,jpex,kpsx,kpex,                   &
                               imsy,imey,jmsy,jmey,kmsy,kmey,                   &
                               ipsy,ipey,jpsy,jpey,kpsy,kpey,                   &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               config_flags%restart, grid%iseedarr_spp_conv,    &
                               config_flags%seed_dim,                           &
                               grid%DX,grid%DY,grid%vertstruc_spp_conv,         &
                               grid%pattern_spp_conv,                           &
                               grid%stddev_cutoff_spp_conv,grid%gridpt_stddev_spp_conv, &
                               grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT    )
      ENDIF !spp_conv
      if ((grid%spp_pbl==1).and.(grid%id .EQ. 1 )) then
          CALL RAND_PERT_UPDATE(grid,'T',                                       &
                               grid%SPFORCS3,grid%SPFORCC3,                     &
                               grid%SP_AMP3,grid%ALPH_RAND3,                    &
                               ips, ipe, jps, jpe, kps, kpe,                    &
                               ids, ide, jds, jde, kds, kde,                    &
                               ims, ime, jms, jme, kms, kme,                    &
                               k_start, k_end,                                  &
                               imsx,imex,jmsx,jmex,kmsx,kmex,                   &
                               ipsx,ipex,jpsx,jpex,kpsx,kpex,                   &
                               imsy,imey,jmsy,jmey,kmsy,kmey,                   &
                               ipsy,ipey,jpsy,jpey,kpsy,kpey,                   &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               grid%num_stoch_levels,grid%num_stoch_levels,     &
                               config_flags%restart, grid%iseedarr_spp_pbl,     &
                               config_flags%seed_dim,                           &
                               grid%DX,grid%DY,grid%vertstruc_spp_pbl,          &
                               grid%pattern_spp_pbl,                            &
                               grid%stddev_cutoff_spp_pbl,grid%gridpt_stddev_spp_pbl, &
                               grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT    )
      ENDIF !spp_pbl
      if ((grid%spp_lsm==1).and.(grid%id .EQ. 1 )) then
          CALL RAND_PERT_UPDATE(grid,'T',                                       &
                               grid%SPFORCS4,grid%SPFORCC4,                     &
                               grid%SP_AMP4,grid%ALPH_RAND4,                    &
```

```
L00365                            ips, ipe, jps, jpe, kps, kpe,                    &
L00366                            ids, ide, jds, jde, kds, kde,                    &
L00367                            ims, ime, jms, jme, kms, kme,                    &
L00368                            k_start, k_end,                                  &
L00369                            imsx,imex,jmsx,jmex,kmsx,kmex,                   &
L00370                            ipsx,ipex,jpsx,jpex,kpsx,kpex,                   &
L00371                            imsy,imey,jmsy,jmey,kmsy,kmey,                   &
L00372                            ipsy,ipey,jpsy,jpey,kpsy,kpey,                   &
L00373                            grid%num_stoch_levels,grid%num_stoch_levels,     &
L00374                            grid%num_stoch_levels,grid%num_stoch_levels,     &
L00375                            config_flags%restart, grid%iseedarr_spp_lsm,     &
L00376                            config_flags%seed_dim,                           &
L00377                            grid%DX,grid%DY,grid%vertstruc_spp_lsm,          &
L00378                            grid%pattern_spp_lsm,                            &
L00379                            grid%stddev_cutoff_spp_lsm,grid%gridpt_stddev_spp_lsm, &
L00380                            grid%VERTSTRUCC,grid%VERTSTRUCS,grid%VERTAMPT    )
L00381       ENDIF !spp_lsm
```

讲解：五组随机扰动全部走 module_stoch 的 **RAND_PERT_UPDATE**（谱空间生成 → 投影回格点 → 按 stddev_cutoff/gridpt_stddev 钳幅），仅主域 id=1：①**SKEBS**（随机动能背散射）三次调 T/U/V——T 用 SPTFORCS/CC 谱系数、U/V 共用流函数谱 SPSTREAMFORCS（注释点明 U 更新流函数、V 不更新）；②**SPPT**（随机物理倾向扰动）——输出 rstoch 场（后面 perturb_physics_tend 消费）；③rand_perturb（通用随机强迫）；④**multi_perturb**（WRF-Solar EPS）：contiguize_2d 把第 n 个成员的谱系数抽成连续临时数组、扰动、再写回——PERT3D(:,:,:,n) 第三维是成员维；⑤**spp_conv/spp_pbl/spp_lsm** 三件——给积云/PBL/陆面方案用的随机场（pattern_spp_，与 *code_23* 的 *pattern_spp_conv* 占位、*code_22* 的 *SPP* 包裹对应）。每件的垂直结构由 *vertstruc* 控制、种子 *iseedarr_* 独立。

五、倾向记账预备 calculate_phy_tend

```
!file: dyn_em/module_first_rk_step_part2.F    L383-L423
L00383  ! calculate_phy_tend
L00384
L00385  BENCH_START(cal_phy_tend)
L00386        !$OMP PARALLEL DO   &
L00387        !$OMP PRIVATE ( ij )
L00388
L00389        DO ij = 1 , grid%num_tiles
L00390
L00391          CALL wrf_debug ( 200 , ' call calculate_phy_tend' )
L00392          CALL calculate_phy_tend (config_flags,grid%c1h,grid%c2h,       &
L00393                       grid%mut,grid%muu,grid%muv,pi_phy,                &
L00394                       grid%rthraten,                                    &
L00395                       grid%rublten,grid%rvblten,grid%rthblten,          &
L00396                       grid%rqvblten,grid%rqcblten,grid%rqiblten,        &
L00397                       grid%rucuten,grid%rvcuten,grid%rthcuten,          &
L00398                       grid%rqvcuten,grid%rqccuten,grid%rqrcuten,        &
L00399                       grid%rqicuten,grid%rqscuten,                      &
L00400                       grid%rushten,grid%rvshten,grid%rthshten,          &
L00401                       grid%rqvshten,grid%rqcshten,grid%rqrshten,        &
L00402                       grid%rqishten,grid%rqsshten,grid%rqgshten,        &
L00403                       grid%RUNDGDTEN,grid%RVNDGDTEN,grid%RTHNDGDTEN,grid%RQVNDGDTEN, &
L00404                       grid%RMUNDGDTEN,                                  &
L00405                       grid%ruiauten,grid%rviauten,grid%rthiauten,       &
L00406                       grid%rqviauten,grid%rqciauten,grid%rqriauten,     &
L00407                       grid%rqiiauten,grid%rqsiauten,grid%rqgiauten,     &
L00408                       grid%rmuiauten,grid%rphiauten,                    &
L00409                       grid%u_iau,grid%v_iau,grid%t_iau,grid%ph_iau,     &
L00410                       grid%mu_iau,grid%qv_iau,grid%qc_iau,grid%qr_iau,  &
L00411                       grid%qs_iau,grid%qi_iau,grid%qg_iau,              &
L00412                       grid%itimestep,                                   &
L00413                       scalar, scalar_tend, num_scalar,                  &
L00414                       tracer, tracer_tend, num_tracer,                  &
L00415                       ids,ide, jds,jde, kds,kde,                        &
L00416                       ims,ime, jms,jme, kms,kme,                        &
L00417                       grid%i_start(ij), min(grid%i_end(ij),ide-1),      &
L00418                       grid%j_start(ij), min(grid%j_end(ij),jde-1),      &
L00419                       k_start    , min(k_end,kde-1)                     )
L00420
L00421        ENDDO
L00422        !$OMP END PARALLEL DO
L00423  BENCH_END(cal_phy_tend)
```

讲解：把 40 余个 *rten* 数组（辐射 *rthraten*、*PBL* 五件、积云六件、浅对流六件、格点 *nudging* 五件、*IAU* 十二件 *+u_iau* 等分析增量）一次性交给 *module_em* 的 *calculate_phy_tend*——它按 ***mu***（干空气柱质量）与 ***c1h/c2h*** 坐标系数加权把「方案倾向」换算成动力方程口径的倾向增量。*IAU*（*incremental analysis update*）的两组数组（*r*iauten 倾向与 *_iau 增量）也在此入口。

六、扩散度量与 Km/Kh

```
!file: dyn_em/module_first_rk_step_part2.F    L427-L647
L00427         IF(config_flags%diff_opt .eq. 2 .OR. config_flags%diff_opt .eq. 1) THEN
L00428
L00429   BENCH_START(comp_diff_metrics_tim)
L00430           !$OMP PARALLEL DO   &
L00431           !$OMP PRIVATE ( ij )
L00432           DO ij = 1 , grid%num_tiles
L00433             CALL wrf_debug ( 200 , ' call compute_diff_metrics ' )
L00434             CALL compute_diff_metrics ( config_flags, grid%ph_2, grid%phb, grid%z, grid%rdz, grid%rdzw, &
L00435                                         grid%zx, grid%zy, grid%rdx, grid%rdy,                           &
L00436                                         ids, ide, jds, jde, kds, kde,          &
L00437                                         ims, ime, jms, jme, kms, kme,          &
L00438                                         grid%i_start(ij), grid%i_end(ij),      &
L00439                                         grid%j_start(ij), grid%j_end(ij),      &
L00440                                         k_start    , k_end                    )
L00441           ENDDO
L00442           !$OMP END PARALLEL DO
L00443   BENCH_END(comp_diff_metrics_tim)
L00444
L00445   #ifdef DM_PARALLEL
L00446   #  include "HALO_EM_TKE_C.inc"
L00447   #  include "PERIOD_BDY_EM_A1.inc"
L00448   #endif
L00449
L00450   BENCH_START(tke_diff_bc_tim)
L00451           !$OMP PARALLEL DO   &
L00452           !$OMP PRIVATE ( ij )
L00453
L00454           DO ij = 1 , grid%num_tiles
L00455
L00456             CALL wrf_debug ( 200 , ' call bc for diffusion_metrics ' )
L00457             CALL set_physical_bc3d( grid%rdzw , 'w', config_flags,          &
L00458                                     ids, ide, jds, jde, kds, kde,       &
L00459                                     ims, ime, jms, jme, kms, kme,       &
L00460                                     ips, ipe, jps, jpe, kps, kpe,       &
L00461                                     grid%i_start(ij), grid%i_end(ij),   &
L00462                                     grid%j_start(ij), grid%j_end(ij),   &
L00463                                     k_start    , k_end                  )
L00464             CALL set_physical_bc3d( grid%rdz , 'w', config_flags,          &
L00465                                     ids, ide, jds, jde, kds, kde,       &
L00466                                     ims, ime, jms, jme, kms, kme,       &
L00467                                     ips, ipe, jps, jpe, kps, kpe,       &
L00468                                     grid%i_start(ij), grid%i_end(ij),   &
L00469                                     grid%j_start(ij), grid%j_end(ij),   &
L00470                                     k_start    , k_end                  )
L00471             CALL set_physical_bc3d( grid%z , 'w', config_flags,             &
L00472                                     ids, ide, jds, jde, kds, kde,       &
L00473                                     ims, ime, jms, jme, kms, kme,       &
L00474                                     ips, ipe, jps, jpe, kps, kpe,       &
L00475                                     grid%i_start(ij), grid%i_end(ij),   &
L00476                                     grid%j_start(ij), grid%j_end(ij),   &
L00477                                     k_start    , k_end                  )
L00478             CALL set_physical_bc3d( grid%zx , 'e', config_flags,            &
L00479                                     ids, ide, jds, jde, kds, kde,       &
L00480                                     ims, ime, jms, jme, kms, kme,       &
L00481                                     ips, ipe, jps, jpe, kps, kpe,       &
L00482                                     grid%i_start(ij), grid%i_end(ij),   &
L00483                                     grid%j_start(ij), grid%j_end(ij),   &
L00484                                     k_start    , k_end                  )
L00485             CALL set_physical_bc3d( grid%zy , 'f', config_flags,            &
L00486                                     ids, ide, jds, jde, kds, kde,       &
L00487                                     ims, ime, jms, jme, kms, kme,       &
L00488                                     ips, ipe, jps, jpe, kps, kpe,       &
L00489                                     grid%i_start(ij), grid%i_end(ij),   &
L00490                                     grid%j_start(ij), grid%j_end(ij),   &
L00491                                     k_start    , k_end                  )
L00492             CALL set_physical_bc2d( grid%ustm, 't', config_flags,           &
L00493                                     ids, ide, jds, jde,                 &
L00494                                     ims, ime, jms, jme,                 &
L00495                                     ips, ipe, jps, jpe,                 &
L00496                                     grid%i_start(ij), grid%i_end(ij),   &
L00497                                     grid%j_start(ij), grid%j_end(ij)   )
L00498              CALL set_physical_bc2d( grid%ust, 't', config_flags,           &
L00499                                     ids, ide, jds, jde,                 &
L00500                                     ims, ime, jms, jme,                 &
L00501                                     ips, ipe, jps, jpe,                 &
L00502                                     grid%i_start(ij), grid%i_end(ij),   &
L00503                                     grid%j_start(ij), grid%j_end(ij)   )
L00504
L00505           ENDDO
L00506           !$OMP END PARALLEL DO
L00507   BENCH_END(tke_diff_bc_tim)
L00508
L00509   BENCH_START(deform_div_tim)
L00510
L00511           !$OMP PARALLEL DO   &
L00512           !$OMP PRIVATE ( ij )
L00513
L00514           DO ij = 1 , grid%num_tiles
L00515
L00516             CALL wrf_debug ( 200 , ' call cal_deform_and_div' )
L00517             CALL cal_deform_and_div ( config_flags,grid%u_2,grid%v_2,grid%w_2,grid%div,  &
L00518                                       grid%defor11,grid%defor22,grid%defor33,            &
L00519                                       grid%defor12,grid%defor13,grid%defor23,            &
L00520                                       nba_rij, n_nba_rij,                                & !JDM
L00521                                       grid%u_base, grid%v_base,grid%msfux,grid%msfuy,    &
L00522                                       grid%msfvx,grid%msfvy,grid%msftx,grid%msfty,       &
L00523                                       grid%rdx, grid%rdy, grid%dn, grid%dnw, grid%rdz,   &
L00524                                       grid%rdzw,grid%fnm,grid%fnp,grid%cf1,grid%cf2,     &
L00525                                       grid%cf3,grid%zx,grid%zy,            &
L00526                                       ids, ide, jds, jde, kds, kde,        &
L00527                                       ims, ime, jms, jme, kms, kme,        &
L00528                                       grid%i_start(ij), grid%i_end(ij),    &
L00529                                       grid%j_start(ij), grid%j_end(ij),    &
```

```
                                    k_start    , k_end                 )
          ENDDO
          !$OMP END PARALLEL DO
  BENCH_END(deform_div_tim)

  ! Updraft helicity between output times

  #ifdef DM_PARALLEL
  #     include "HALO_EM_HELICITY.inc"
  #endif

        IF ( ( config_flags%nwp_diagnostics .eq. 1 ) .OR. &
             ( ( config_flags%afwa_diag_opt .eq. 1 ) .AND. ( config_flags%afwa_severe_opt .EQ. 1 ) ) ) THEN
  BENCH_START(helicity_tim)

        !$OMP PARALLEL DO   &
        !$OMP PRIVATE ( ij )

        DO ij = 1 , grid%num_tiles

           CALL wrf_debug ( 200 , ' call cal_helicity' )
           CALL cal_helicity ( config_flags,grid%u_2,grid%v_2,grid%w_2,  &
                               grid%uh,                                  &
                               grid%up_heli_max,                         &
                               grid%ph_2,grid%phb,                       &
                               grid%msfux,grid%msfuy,                    &
                               grid%msfvx,grid%msfvy,                    &
                               grid%ht,                                  &
                               grid%rdx, grid%rdy, grid%dn, grid%dnw, grid%rdz, grid%rdzw,   &
                               grid%fnm,grid%fnp,grid%cf1,grid%cf2,grid%cf3,grid%zx,grid%zy, &
                               ids, ide, jds, jde, kds, kde,             &
                               ims, ime, jms, jme, kms, kme,             &
                               grid%i_start(ij), grid%i_end(ij),         &
                               grid%j_start(ij), grid%j_end(ij),         &
                               k_start    , k_end                 )
        ENDDO
        !$OMP END PARALLEL DO
  BENCH_END(helicity_tim)
        ENDIF

  #ifdef DM_PARALLEL
  #     include "HALO_EM_TKE_D.inc"
  #endif

  ! calculate tke, kmh, and kmv

  BENCH_START(calc_tke_tim)
          !$OMP PARALLEL DO   &
          !$OMP PRIVATE ( ij )
          DO ij = 1 , grid%num_tiles

  !XZ
          IF( config_flags%diff_opt .eq. 2 .and. config_flags%km_opt .eq. 5 ) THEN
            CALL nonlocal_flux(config_flags,grid%nlflux,grid%gamu,grid%gamv,      &
                               grid%pblh,grid%kpbl,grid%dx,grid%dy,grid%dt,       &
                               grid%ust,grid%hfx,grid%qfx,grid%br,ep_1,ep_2,      &
                               karman,grid%u_phy,grid%v_phy,th_phy,grid%rho,      &
                               moist,num_moist,                                   &
                               grid%msftx,grid%msfty,grid%rdzw,                   &
                               grid%u10,grid%v10,grid%wspd,                       &
                               ids, ide, jds, jde, kds, kde,                      &
                               ims, ime, jms, jme, kms, kme,                      &
                               grid%i_start(ij), grid%i_end(ij),                  &
                               grid%j_start(ij), grid%j_end(ij),                  &
                               k_start, k_end  )

            CALL free_atmos_length(config_flags,grid%dx,grid%dy,grid%rdzw,        &
                                   grid%rdz,grid%tke_2,th_phy,grid%elmin,         &
                                   grid%hfx,grid%qfx,moist,num_moist,             &
                                   ids, ide, jds, jde, kds, kde,                  &
                                   ims, ime, jms, jme, kms, kme,                  &
                                   grid%i_start(ij),grid%i_end(ij),               &
                                   grid%j_start(ij), grid%j_end(ij),              &
                                   k_start, k_end  )

            CALL meso_length_scale(config_flags,grid%dx,grid%dy,grid%rdzw,        &
                                   grid%rdz,grid%tke_2,p8w,t8w,th_phy,            &
                                   grid%dlk,grid%pblh,grid%elmin,                 &
                                   grid%rmol,grid%rho,grid%hfx,grid%qfx,          &
                                   moist,num_moist,                               &
                                   ids, ide, jds, jde, kds, kde,                  &
                                   ims, ime, jms, jme, kms, kme,                  &
                                   grid%i_start(ij), grid%i_end(ij),              &
                                   grid%j_start(ij), grid%j_end(ij),              &
                                   k_start, k_end  )
          ENDIF
  !!

            CALL wrf_debug ( 200 , ' call calculate_km_kh' )
            CALL calculate_km_kh( config_flags,grid%dt,grid%dampcoef,grid%zdamp,         &
                                  config_flags%damp_opt,                                 &
                                  grid%xkmh,grid%xkmv,grid%xkhh,grid%xkhv,grid%bn2,      &
                                  grid%khdif,grid%kvdif,grid%div,                        &
                                  grid%defor11,grid%defor22,grid%defor33,grid%defor12,   &
                                  grid%defor13,grid%defor23,                             &
                                  grid%tke_2,p8w,t8w,th_phy,                             &
                                  t_phy,p_phy,moist,grid%dn,grid%dnw,                    &
                                  grid%dx,grid%dy,grid%rdz,grid%rdzw,                    &
                                  config_flags%mix_isotropic,num_moist,                  &
                                  grid%cf1, grid%cf2, grid%cf3, grid%warm_rain,          &
                                  grid%mix_upper_bound,                                  &
                                  grid%msftx, grid%msfty,                                &
                                  grid%zx, grid%zy,                                      &
                                  grid%pblh, grid%dlk, grid%xkmv_meso,                   &
                                  ids,ide, jds,jde, kds,kde,                             &
                                  ims,ime, jms,jme, kms,kme,                             &
                                  grid%i_start(ij), grid%i_end(ij),                      &
                                  grid%j_start(ij), grid%j_end(ij),                      &
                                  k_start    , k_end                         )
```

```
L00639           ENDDO
L00640         !$OMP END PARALLEL DO
L00641  BENCH_END(calc_tke_tim)
L00642
L00643  #ifdef DM_PARALLEL
L00644  #     include "HALO_EM_TKE_E.inc"
L00645  #endif
L00646
L00647         ENDIF
```

讲解：diff_opt≥1 时的混合原料流水线（每步重算）：①compute_diff_metrics（rdz/rdzw/zx/zy——**斜坡地形下的度量项**）；② 六组 set_physical_bc3d + 两组 2D（rdzw/rdz/z/zx/zy/ustm/ust 的物理边界——**扩散量也守 BC**）；③cal_deform_and_div（六个变形率分量 defor11..23 + 散度 div + 基本流扣除，nba_rij 是 SFS 模式的各向异性变形率）；④cal_helicity（nwp_diagnostics/afwa_severe_opt=1 时的上升气流螺旋度诊断）；⑤km_opt=5（TKE-ε 闭合）的三件套：nonlocal_flux（PBL 非局地通量 nlflux/gamu/gamv）、free_atmos_length（自由大气混合长 elmin）、meso_length_scale（中尺度混合长 dlk）；⑥**calculate_km_kh**——四张混合系数场（xkmh/xkmv/xkhh/xkhv）按 km_opt 分派（1/2/3/4/5 各式），bn2 浮力频率、mix_isotropic 各向同性开关、mix_upper_bound 上限。HALO_EM_TKE_C/D/E 三次刷新保证每步的 halo 新鲜。

七、条件 HALO 与点阵地图

```
!file: dyn_em/module_first_rk_step_part2.F    L691-L770
L00691  #ifdef DM_PARALLEL
L00692  !-----------------------------------------------------------------------
L00693  !
L00694  ! MPP for some physics tendency, km, kh, deformation, and divergence
L00695  !
L00696  !                                                    * * * * * * *
L00697  !                                          * * * * *   * * * * * * *
L00698  !               *                   *      * * * * *   * * * * * * *
L00699  !             * + * *     * + *     +      * * + * *   * * * + * * *
L00700  !               *                   *      * * * * *   * * * * * * *
L00701  !                                          * * * * *   * * * * * * *
L00702  !                                                    * * * * * * *
L00703  !
L00704  ! (for PBL)
L00705  ! rublten                  x
L00706  ! rvblten                           x
L00707  !
L00708  ! (for Cumulus)
L00709  ! rucuten                  x
L00710  ! rvcuten                           x
L00711  !
L00712  ! (for Shallow Cumulus)
L00713  ! rushten                  x
L00714  ! rvshten                           x
L00715  !
L00716  ! (for FDDA)
L00717  ! rundgdten     x
L00718  ! rvndgdten     x
L00719  !
L00720  ! (for TKE3)
L00721  ! tke_2                                      x
L00722  ! (for TKE5)
L00723  ! tke_2                                                  x
L00724  !
L00725  ! (for diff_opt >= 1)
L00726  ! defor11                  x
L00727  ! defor22                           x
L00728  ! defor12       x
L00729  ! defor13                  x
L00730  ! defor23                           x
L00731  ! div           x
L00732  ! xkmv          x
L00733  ! xkmh          x
L00734  ! xkhv          x
L00735  ! xkhh          x
L00736  ! tke           x
L00737  !
L00738  !-----------------------------------------------------------------------
L00739         IF ( config_flags%bl_pbl_physics .ge. 1 ) THEN
L00740  #      include "HALO_EM_PHYS_PBL.inc"
L00741         ENDIF
L00742         IF ( config_flags%shcu_physics .gt. 1 ) THEN
L00743  #      include "HALO_EM_PHYS_SHCU.inc"
L00744         ENDIF
L00745         IF ( config_flags%cu_physics == SASSCHEME      .or.   &
L00746              config_flags%cu_physics == TIEDTKESCHEME  .or.   &
L00747              config_flags%cu_physics == NTIEDTKESCHEME .or.   &
L00748              config_flags%cu_physics == MSKFSCHEME     .or.   &
L00749              config_flags%cu_physics == CAMZMSCHEME    .or.   &
L00750              config_flags%cu_physics == SCALESASSCHEME .or.   &
L00751              config_flags%cu_physics == NSASSCHEME     .or.   &
L00752              config_flags%cu_physics == KSASSCHEME ) THEN
L00753  #      include "HALO_EM_PHYS_CU.inc"
L00754         ENDIF
L00755         IF ( config_flags%grid_fdda .ge. 1) THEN
L00756  #      include "HALO_EM_FDDA.inc"
L00757         ENDIF
L00758         IF ( config_flags%diff_opt .ge. 1 ) THEN
L00759  #      include "HALO_EM_PHYS_DIFFUSION.inc"
L00760         ENDIF
L00761
L00762         IF      ( config_flags%h_sca_adv_order <= 4 ) THEN
```

```
L00763  #       include "HALO_EM_TKE_3.inc"
L00764          ELSE IF ( config_flags%h_sca_adv_order <= 6 ) THEN
L00765  #       include "HALO_EM_TKE_5.inc"
L00766          ELSE
L00767            WRITE(wrf_err_message,*)'solve_em: invalid h_sca_adv_order = ',config_flags%h_sca_adv_order
L00768            CALL wrf_error_fatal(TRIM(wrf_err_message))
L00769          ENDIF
L00770  #endif
```

讲解：① phy_bc（扩散量与倾向的物理边界）；②**sfs_driver**——sfs_opt>0 且 diff_opt=2 时的 SFS（subfilter-scale）湍流模式驱动（JDM；与 BEP 的 A/B 分解思想同族的另一套各向异性变形率账户）。③ 本文件的 ASCII 点阵图（MP for some physics tendency…）是 **halo 覆盖需求表**——哪个数组需要哪个方向哪些点的 halo 一目了然；④ 条件 HALO：PBL/SHCU 各一；**HALO_EM_PHYS_CU 的八方案名单（SAS/TIEDTKE/NTIEDTKE/MSKF/CAMZM/SCALESAS/NSAS/KSAS）**——恰好是会写动量倾向的积云方案（KF 系与 BMJ 不在名单，与 code_23 各案不传 RUCUTEN 交叉印证；G3 的动量倾向走 imomentum 选项与 G3 专用 halo）；FDDA/diffusion 各一；TKE_3/TKE_5 按 h_sca_adv_order 四阶/六阶标量平流精度选。

八、总账合并 update_phy_ten

```
!file: dyn_em/module_first_rk_step_part2.F    L772-L832
L00772  BENCH_START(update_phy_ten_tim)
L00773          !$OMP PARALLEL DO   &
L00774          !$OMP PRIVATE ( ij )
L00775
L00776          DO ij = 1 , grid%num_tiles
L00777
L00778            CALL wrf_debug ( 200 , ' call update_phy_ten' )
L00779  #if ( WRF_DFI_RADAR == 1 )
L00780            if (config_flags%cu_physics .gt. 0) then
L00781              i_start = grid%i_start(ij)
L00782              i_end   = min( grid%i_end(ij),ide-1 )
L00783              j_start = grid%j_start(ij)
L00784              j_end   = min( grid%j_end(ij),jde-1 )
L00785              if (grid%dfi_stage == DFI_FWD ) &
L00786                   CALL wrf_debug ( 200 , ' Zero out cu_physics' )
L00787              DO j = j_start, j_end
L00788              DO k = k_start, min( k_end,kde-1 ) - 1
L00789              DO i = i_start, i_end
L00790                if (grid%dfi_stage ==DFI_FWD  &
L00791                .and. grid%dfi_tten_rad(i,k,j) >= 1.0e-7 .and.  &
L00792                      grid%dfi_tten_rad(i,k,j) <= 10.) then
L00793  ! zero out cu-param temp tendency
L00794                   grid%rthcuten(i,k,j) = 0.0
L00795                endif
L00796              ENDDO
L00797              ENDDO
L00798              ENDDO
L00799            ENDIF
L00800  #endif
L00801            CALL update_phy_ten(ph_tendf,t_tendf, ru_tendf, rv_tendf,moist_tend ,&
L00802                                scalar_tend, mu_tendf,                          &
L00803                                grid%rthraten,grid%rthblten,grid%rthcuten,grid%rthshten, &
L00804                                grid%rublten,grid%rucuten,grid%rushten,         &
L00805                                grid%rvblten,grid%rvcuten,grid%rvshten,         &
L00806                                grid%rqvblten,grid%rqcblten,grid%rqiblten,      &
L00807                                grid%rqniblten,                                 & !CAMUWPBL scheme
L00808                                grid%rqvcuten,grid%rqccuten,grid%rqrcuten,      &
L00809                                grid%rqicuten,grid%rqscuten,                    &
L00810                                grid%rqcncuten,grid%rqincuten,                  & !BSINGH - Added two CU tends
L00811                                grid%rqvshten,grid%rqcshten,grid%rqrshten,      &
L00812                                grid%rqishten,grid%rqsshten,grid%rqgshten,      &
L00813                                grid%rqcnshten,grid%rqinshten,                  &!BSINGH - Added two SHCU tends
L00814                                grid%RUNDGDTEN,                                 &
L00815                                grid%RVNDGDTEN,grid%RTHNDGDTEN,grid%RPHNDGDTEN, &
L00816                                grid%RQVNDGDTEN,grid%RMUNDGDTEN,                &
L00817                                grid%rthfrten,grid%rqvfrten,                    &  ! fire
L00818                                grid%ruiauten,grid%rviauten,grid%rthiauten,     &
L00819                                grid%rqviauten,grid%rqciauten,grid%rqriauten,   &
L00820                                grid%rqiiauten,grid%rqsiauten,grid%rqgiauten,   &
L00821                                grid%rphiauten,grid%rmuiauten,                  &
L00822                                num_moist,num_scalar,config_flags,rk_step,      &
L00823                                grid%adv_moist_cond,                            &
L00824                                ids, ide, jds, jde, kds, kde,                   &
L00825                                ims, ime, jms, jme, kms, kme,                   &
L00826                                grid%i_start(ij), grid%i_end(ij),               &
L00827                                grid%j_start(ij), grid%j_end(ij),               &
L00828                                k_start, k_end                                  )
L00829
L00830          END DO
L00831          !$OMP END PARALLEL DO
L00832  BENCH_END(update_phy_ten_tim)
```

讲解：**物理倾向的最终合并**：update_phy_ten（module_physics_addtendc）把 rthraten+rthblten+rthcuten+rthshten+rthndgdt+rthfrten(火)+iau 系全部加进 t_tendf/ru_tendf/rv_tendf/moist_tend/scalar_tend/mu_tendf——**此后 t_tendf 等才是 RK3 动力步消费的总倾向**。DFI_RADAR 细节：dfi_fwd 阶段且 dfi_tten_rad∈[1e-7,10] 的点把 rthcuten 清零（雷达 DFI 的 CAP 屏蔽——与 code_23 G3 案的 cap_suppress 是同一机制的两端）。rk_step 参数随行（总账只在 rk_step=1 记，RK 其余子步复用）。

```
!file: dyn_em/module_first_rk_step_part2.F    L834-L871
L00834        IF (grid%skebs_on==1) then
L00835            !$OMP PARALLEL DO   &
L00836            !$OMP PRIVATE ( ij )
L00837            DO ij = 1 , grid%num_tiles
L00838                 CALL wrf_debug ( 200 , ' call update_stoch_ten' )
L00839                  CALL update_stoch_ten(ru_tendf, rv_tendf, t_tendf,&
L00840                                grid%ru_tendf_stoch,                          &
L00841                                grid%rv_tendf_stoch,                          &
L00842                                grid%rt_tendf_stoch,                          &
L00843                                grid%mu_2, grid%mub, grid%c1h, grid%c2h,      &
L00844                                ids, ide, jds, jde, kds, kde,                 &
L00845                                ims, ime, jms, jme, kms, kme,                 &
L00846                                grid%i_start(ij), grid%i_end(ij),             &
L00847                                grid%j_start(ij), grid%j_end(ij),             &
L00848                                k_start, k_end,                               &
L00849                                grid%num_stoch_levels,grid%num_stoch_levels   )
L00850
L00851             ENDDO
L00852             !$OMP END PARALLEL DO
L00853        ENDIF !skebs_on
L00854
L00855        IF (grid%sppt_on==1) then
L00856            !$OMP PARALLEL DO   &
L00857            !$OMP PRIVATE ( ij )
L00858            DO ij = 1 , grid%num_tiles
L00859                 call perturb_physics_tend(grid%gridpt_stddev_sppt,           &
L00860                       grid%stddev_cutoff_sppt,grid%rstoch,                   &
L00861                       ru_tendf,rv_tendf,t_tendf,                             &
L00862                       moist_tend(ims,kms,jms,p_qv),                          &
L00863                       ids, ide, jds, jde, kds, kde,                          &
L00864                       ims, ime, jms, jme, kms, kme,                          &
L00865                       grid%i_start(ij), grid%i_end(ij),                      &
L00866                       grid%j_start(ij), grid%j_end(ij),                      &
L00867                       k_start, k_end,                                        &
L00868                       grid%num_stoch_levels,grid%num_stoch_levels            )
L00869             ENDDO
L00870             !$OMP END PARALLEL DO
L00871    ENDIF
```

讲解：随机倾向注入的落点：update_stoch_ten（SKEBS 的 ru/rv/rt_tendf_stoch 按 mu 加权加进总账）；perturb_physics_tend（SPPT——把 rstoch 场乘到 ru/rv/t/qv 四类倾向上；**moist_tend 只切 P_QV 一片**）。

九、PLANET 彩蛋 damptop

```
!file: dyn_em/module_first_rk_step_part2.F    L873-L902
L00873  #ifdef PLANET
L00874         ! do rayleigh (and zonal-average newtonian) damping during
L00875         ! first iteration of RK loop only
L00876
L00877  =============================================================
L00878  INTENTIONALLY PLACED HERE TO BREAK COMPILE
L00879  1. The DAMPTOP routine needs to have 1d column arrays c1 and c2
L00880  to correctly use the column pressures mut, muu, and muv.
L00881  2. The DAMPTOP routine needs to have the grid%t_2 evaluated
L00882  to see if perturbation of moist potential temperature is OK.
L00883  =============================================================
L00884
L00885         IF ( (config_flags%damp_opt == 101) .OR. &
L00886              (config_flags%damp_opt == 103)      ) THEN
L00887           !$OMP PARALLEL DO   &
L00888           !$OMP PRIVATE ( ij )
L00889           DO ij = 1 , grid%num_tiles
L00890             CALL damptop( grid%u_2, grid%v_2, grid%t_2, &
L00891                           grid%mut, grid%muu, grid%muv, &
L00892                           pi_phy,                       &
L00893                           t_tendf, ru_tendf, rv_tendf, P2SI,     &
L00894                           ids, ide, jds, jde, kds, kde,          &
L00895                           ims, ime, jms, jme, kms, kme,          &
L00896                           grid%i_start(ij), grid%i_end(ij),      &
L00897                           grid%j_start(ij), grid%j_end(ij),      &
L00898                           k_start, k_end                         )
L00899           END DO
L00900           !$OMP END PARALLEL DO
L00901         END IF
L00902  #endif
```

讲解：`#ifdef PLANET` 段里**故意断编译的源码**——「INTENTIONALLY PLACED HERE TO BREAK COMPILE」+ 两条理由（damptop 需要 c1/c2 列压数组、需要 t_2 判断湿位温扰动）。行星 WRF（火星等）的 Rayleigh 顶层阻尼代码改到一半没人收尾，作者把断编译当 TODO 提醒——读 WRF 源码的幽默时刻，也是「非地球行星仍共用本模块」的证据。

十、观测 nudging 与 use_theta_m

```
!file: dyn_em/module_first_rk_step_part2.F    L941-L1003
L00941         IF ( grid%obs_nudge_opt .EQ. 1 .AND. grid%xtime <= grid%fdda_end ) THEN
L00942  # ifdef DM_PARALLEL
L00943  #       include "HALO_OBS_NUDGE.inc"
L00944  #endif
L00945  !*************************************************************************
L00946  ! This section for obs nudging
L00947           !$OMP PARALLEL DO   &
L00948           !$OMP PRIVATE ( ij )
L00949
L00950           DO ij = 1 , grid%num_tiles
L00951
L00952             CALL fddaobs_driver (grid%grid_id, model_config_rec%grid_id, &
L00953                    model_config_rec%parent_id, config_flags%restart,    &
L00954                    config_flags,                                        &
L00955                    grid%obs_nudge_opt,                                  &
L00956                    grid%obs_ipf_errob,                                  &
L00957                    grid%obs_ipf_nudob,                                  &
L00958                    grid%fdda_start,                                     &
L00959                    grid%fdda_end,                                       &
L00960                    grid%obs_nudge_wind,                                 &
L00961                    grid%obs_nudge_temp,                                 &
L00962                    grid%obs_nudge_mois,                                 &
L00963                    grid%obs_nudge_pstr,                                 &
L00964                    grid%obs_coef_wind,                                  &
L00965                    grid%obs_coef_temp,                                  &
L00966                    grid%obs_coef_mois,                                  &
L00967                    grid%obs_coef_pstr,                                  &
L00968                    grid%obs_rinxy,                                      &
L00969                    grid%obs_rinsig,                                     &
L00970                    grid%obs_npfi,                                       &
L00971                    grid%obs_ionf,                                       &
L00972                    grid%obs_prt_max,                                    &
L00973                    grid%obs_prt_freq,                                   &
L00974                    grid%obs_idynin,                                     &
L00975                    grid%obs_dtramp,                                     &
L00976                    grid%parent_grid_ratio,                              &
L00977                    grid%max_dom, grid%itimestep,                        &
L00978                    grid%xtime,                                          &
L00979                    grid%dt, grid%gmt, grid%julday, grid%fdob,           &
L00980                    grid%max_obs,                                        &
L00981                    model_config_rec%nobs_ndg_vars,                      &
L00982                    model_config_rec%nobs_err_flds,                      &
L00983                    grid%fdob%nstat, grid%fdob%varobs, grid%fdob%errf,   &
L00984                    grid%dx, grid%KPBL,grid%HT,                          &
L00985                    grid%mut, grid%muu, grid%muv, grid%c1h, grid%c2h,    &
L00986                    grid%msftx, grid%msfty, grid%msfux, grid%msfuy, grid%msfvx, grid%msfvy, &
L00987                    p_phy, t_tendf, t0,                                  &
L00988                    grid%u_2, grid%v_2, grid%th_phy_m_t0,                &
L00989                    moist(ims,kms,jms,P_QV),                             &
L00990                    grid%pb, grid%p_top, grid%p, grid%phb, grid%ph_2,    &
L00991                    grid%uratx, grid%vratx, grid%tratx,                  &
L00992                    ru_tendf, rv_tendf,                                  &
L00993                    moist_tend(ims,kms,jms,P_QV), grid%obs_savwt,        &
L00994                    grid%regime, grid%pblh, grid%z_at_w, grid%z,         &
L00995                    ids,ide, jds,jde, kds,kde,                           &
L00996                    ims,ime, jms,jme, kms,kme,                           &
L00997                    grid%i_start(ij), min(grid%i_end(ij),ide-1),         &
L00998                    grid%j_start(ij), min(grid%j_end(ij),jde-1),         &
L00999                    k_start    , min(k_end,kde-1)                     )
L01000
L01001           ENDDO
L01002           !$OMP END PARALLEL DO
L01003         ENDIF  ! obs_nudge_opt .eq. 1
```

讲解：obs_nudge_opt=1 且 xtime≤fdda_end 时调 fddaobs_driver——观测逼近（obs nudging）的整套控制参数（风/温/湿/地表气压四开关、系数、时间窗 obs_dtramp、隔离半径 rinxy、ipf 错误防护标志）与观测数组 fdob（nstat/varobs/errf）；注意它直接改 ru/rv/t/moist_qv 的**总账**而非 r*ten——obs nudging 是对状态的直接拉回。th_phy_m_t0 入参再次出现（use_theta_m 口径）。

```
!file: dyn_em/module_first_rk_step_part2.F    L1005-L1018
L01005         IF ( config_flags%use_theta_m .EQ. 1 ) THEN
L01006           !$OMP PARALLEL DO   &
L01007           !$OMP PRIVATE ( ij )
L01008           DO ij = 1 , grid%num_tiles
L01009             CALL conv_t_tendf_to_moist ( grid%t_1 , moist_old(ims,kms,jms,P_qv) ,  &
L01010                                          t_tendf  , moist_tend(ims,kms,jms,P_qv) , &
L01011                                          ids, ide, jds, jde, kds, kde ,            &
L01012                                          ims, ime, jms, jme, kms, kme ,            &
L01013                                          grid%i_start(ij), grid%i_end(ij),         &
L01014                                          grid%j_start(ij), grid%j_end(ij),         &
L01015                                          k_start    , k_end                        )
L01016           END DO
L01017           !$OMP END PARALLEL DO
L01018         END IF ! use moist theta
```

讲解：use_theta_m=1（湿位温为预报量）时，conv_t_tendf_to_moist 把干位温口径的温度倾向转成湿位温倾向（用 moist_old 的 qv 与 t_1）——两套热力学变量并存的核心换算点（教材 ch21 的 use_theta_m 注记在此落地）。

十一、垂直扩散后水平扩散

```
!file: dyn_em/module_first_rk_step_part2.F    L1020-L1124
L01020  ! calculate vertical diffusion first and then horizontal
L01021  ! (keep this order)
L01022
L01023         IF(config_flags%diff_opt .eq. 2) THEN
L01024
L01025           IF (config_flags%bl_pbl_physics .eq. 0) THEN
L01026
L01027  !XZ     implicit solver for vertical diffusion tendency
L01028           IF( config_flags%km_opt .eq. 5 ) THEN
L01029                !$OMP PARALLEL DO   &
L01030                !$OMP PRIVATE ( ij )
L01031                DO ij = 1 , grid%num_tiles
L01032                CALL vertical_diffusion_implicit(ru_tendf, rv_tendf, rw_tendf,                          &
L01033                                                 t_tendf, tke_tend,                                     &
L01034                                                 moist_tend, num_moist,                                 &
L01035                                                 chem_tend, num_chem,                                   &
L01036                                                 scalar_tend, num_scalar,                               &
L01037                                                 tracer_tend, num_tracer,                               &
L01038                                                 grid%u_2, grid%v_2,grid%w_2,grid%dt,                   &
L01039                                                 grid%t_2,grid%u_base,grid%v_base,grid%t_base,grid%qv_base, &
L01040                                                 grid%mut,grid%tke_2,th_phy,config_flags,               &
L01041                                                 moist, chem, scalar, tracer,                           &
L01042                                                 grid%xkmv, grid%xkhv, grid%xkmh, config_flags%km_opt,  &
L01043                                                 grid%fnm, grid%fnp, grid%dn, grid%dnw, grid%rdz, grid%rdzw,&
L01044                                                 grid%hfx, grid%qfx, grid%ustm, grid%rho,               &
L01045                                                 grid%nlflux, grid%gamu, grid%gamv,                     &
L01046                                                 grid%xkmv_meso, grid%l_diss,                           &
L01047                                                 grid%c1h, grid%c2h, grid%c1f, grid%c2f,                &
L01048                                                 ids, ide, jds, jde, kds, kde,                          &
L01049                                                 ims, ime, jms, jme, kms, kme,                          &
L01050                                                 grid%i_start(ij), grid%i_end(ij),                      &
L01051                                                 grid%j_start(ij), grid%j_end(ij),                      &
L01052                                                 k_start, k_end      )
L01053                ENDDO
L01054                !$OMP END PARALLEL DO
L01055
L01056           ELSE  ! original explicit
L01057
L01058  BENCH_START(vert_diff_tim)
L01059             !$OMP PARALLEL DO   &
L01060             !$OMP PRIVATE ( ij )
L01061             DO ij = 1 , grid%num_tiles
L01062
L01063               CALL wrf_debug ( 200 , ' call vertical_diffusion_2 ' )
L01064               CALL vertical_diffusion_2( ru_tendf, rv_tendf, rw_tendf,           &
L01065                                          t_tendf, tke_tend,                      &
L01066                                          moist_tend, num_moist,                   &
L01067                                          chem_tend, num_chem,                    &
L01068                                          scalar_tend, num_scalar,                  &
L01069                                          tracer_tend, num_tracer,                  &
L01070                                          grid%u_2, grid%v_2,                                &
L01071                                          grid%t_2,grid%u_base,grid%v_base,grid%t_base,grid%qv_base,          &
L01072                                          grid%tke_2, th_phy, config_flags,          &
L01073                                          grid%defor13,grid%defor23,grid%defor33,              &
L01074                                          nba_mij, num_nba_mij,          & !JDM
L01075                                          grid%div, moist, chem, scalar,tracer,        &
L01076                                          grid%xkmv, grid%xkhv, grid%xkmh, config_flags%km_opt,       & ! xkmh added
↪  Oct2013
L01077                                          grid%fnm, grid%fnp, grid%dn, grid%dnw, grid%rdz, grid%rdzw, &
L01078                                          grid%hfx, grid%qfx, grid%ustm, grid%rho,   &
L01079                                          ids, ide, jds, jde, kds, kde,              &
L01080                                          ims, ime, jms, jme, kms, kme,              &
L01081                                          grid%i_start(ij), grid%i_end(ij),          &
L01082                                          grid%j_start(ij), grid%j_end(ij),          &
L01083                                          k_start, k_end                             )
L01084
L01085             ENDDO
L01086             !$OMP END PARALLEL DO
L01087  BENCH_END(vert_diff_tim)
L01088
L01089           ENDIF
L01090         ENDIF
L01091  !
L01092  BENCH_START(hor_diff_tim)
L01093           !$OMP PARALLEL DO   &
L01094           !$OMP PRIVATE ( ij )
L01095           DO ij = 1 , grid%num_tiles
L01096
L01097             CALL wrf_debug ( 200 , ' call horizontal_diffusion_2' )
L01098             CALL horizontal_diffusion_2( t_tendf, ru_tendf, rv_tendf, rw_tendf, &
L01099                                          tke_tend,                             &
L01100                                          moist_tend, num_moist,                 &
L01101                                          chem_tend, num_chem,                  &
L01102                                          scalar_tend, num_scalar,                &
L01103                                          tracer_tend, num_tracer,                &
L01104                                          grid%t_2, th_phy,                          &
L01105                                          grid%tke_2, config_flags,             &
L01106                                          grid%defor11, grid%defor22, grid%defor12,           &
L01107                                          grid%defor13, grid%defor23,   &
L01108                                          nba_mij, num_nba_mij,         & !JDM
L01109                                          grid%div,                     &
L01110                                          moist, chem, scalar,tracer,              &
L01111                                          grid%msfux,grid%msfuy, grid%msfvx,grid%msfvy, grid%msftx,  &
L01112                                          grid%msfty, grid%xkmh, grid%xkmv, grid%xkhh, config_flags%km_opt,     &
L01113                                          grid%rdx, grid%rdy, grid%rdz, grid%rdzw,                  &
L01114                                          grid%fnm, grid%fnp, grid%cf1, grid%cf2, grid%cf3,         &
L01115                                          grid%zx, grid%zy, grid%dn, grid%dnw, grid%rho,            &
L01116                                          ids, ide, jds, jde, kds, kde,          &
L01117                                          ims, ime, jms, jme, kms, kme,          &
L01118                                          grid%i_start(ij), grid%i_end(ij),      &
L01119                                          grid%j_start(ij), grid%j_end(ij),      &
L01120                                          k_start    , k_end                   )
L01121           ENDDO
L01122           !$OMP END PARALLEL DO
```

```
L01123  BENCH_END(hor_diff_tim)
L01124         ENDIF
```

讲解：① **注释明令顺序「keep this order」**：先垂直后水平；② 垂直扩散**只在 bl_pbl_physics=0 时做**——PBL 方案自带垂直混合，开了 PBL 再做会双重混合；③km_opt=5 走 vertical_diffusion_implicit（**隐式求解**——非局地通量 nlflux/gamu/gamv 与中尺度混合长 xkmv_meso 入账），其余走显式 vertical_diffusion_2（吃 defor13/23/33 三分量与 nba_mij）；④ 水平扩散 horizontal_diffusion_2 吃 defor11/22/12/13/23 五分量与地图因子 msf 系——**真正的地方差分在 module_diffusion_em（code_30 同族）**，本文件只管编排。

十二、改造入口

1. **随机扰动体系**：RAND_PERT_UPDATE 的五连调用——加新 SPP 分量的模板（谱系数 Registry 态 + contiguize + 调用）。
2. **倾向记账**：update_phy_ten 的实参表——加新物理倾向数组的最后一站（Registry r*ten → 总账）。
3. **HALO 名单**：HALO_EM_PHYS_CU 的八方案——新积云方案若带动量倾向要加名单。
4. **扩散体系**：km_opt/diff_opt 组合的分派树——换湍流闭合（如 TKE-ε 改进）在 calculate_km_kh/tke_rhs。
5. **双重混合陷阱**：bl_pbl_physics>0 时垂直扩散自动关闭——自定义 PBL 若不闭合垂直混合会漏混合。

十三、上下游链接

- 上半：`code_31_module_first_rk_step_part1.md`；主循环：`code_50_solve_em.md`
- 扩散与混合工具：code_30（big_step_utilities，同族 module_diffusion_em 的伴随件）
- 教材正文：ch21_solve_em_dynamics.md、ch14_pbl_family.md（TKE 闭合）

code_40 — real 初始化的基态构造（dyn_em/module_initialize_real.F 选段：基态气压/温度/位势与 500 hPa 调整）

一、版本指纹

源文件	行数	md5（整文件，按字节）
dyn_em/module_initialize_real.F	9214	f37acbd0b368169c6dc211519f57aa24

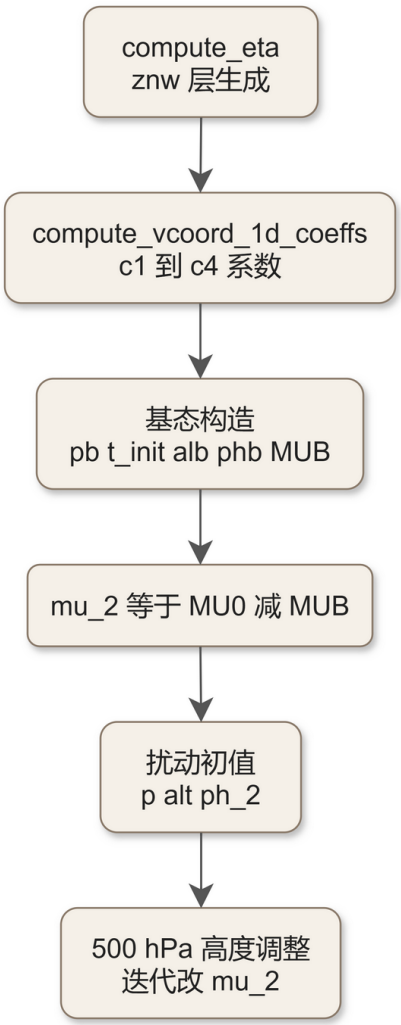
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。

- 收录分级（B）：**只收基态构造段**（init_domain_rk 的 L1584–L1620 与 L3768–L4108 五段）——9214 行中的其余单元（vert_interp/lagrange 插值族、sfcprs 系、filter_topo 系等）不属基态主线，注记见 § 五。教材锚点：ch07_real_namelist.md（real 流程）、ch10_vertical_coords_hydrostatics.md（基态公式）、`code_41_nest_init_utils.md`（znw/c1..c4 系数的生成）。

二、定位

一句话：**wrfinput 基态的出生地**——init_domain_rk（4891 行单体）里把 metgrid 场重整成 WRF 预报量的全套过程，本篇只取其中的静力基态构造：基态气压 pb/温度 t_init/比重容 alb/位势 phb、扰动 mu_2/p/ph_2 的初值，以及 500 hPa 高度对 MU_2 的迭代调整。

谁在调它：share/module_initialize（real.exe 的 init_modules）→ init_domain → init_domain_rk；rebalance（垂直嵌套反馈后重平衡）复用同一套公式。42 单元总表（42 units）不另列，分组见 § 五。



三、调用点：znw 与 c1..c4 系数的生成

```
!file: dyn_em/module_initialize_real.F    L1584-L1620
L01584              max_dz             = model_config_rec%max_dz
L01585              dzbot              = model_config_rec%dzbot
L01586              dzstretch_s        = model_config_rec%dzstretch_s
L01587              dzstretch_u        = model_config_rec%dzstretch_u
L01588              auto_levels_opt  = model_config_rec%auto_levels_opt
L01589
L01590              CALL compute_eta ( grid%znw , auto_levels_opt, &
L01591                                 eta_levels , max_eta , max_dz , dzbot , dzstretch_s , dzstretch_u , &
L01592                                 grid%p_top , g , p00 , cvpm , a , r_d , cp , &
L01593                                 t00 , p1000mb , t0 , tiso , p_strat , a_strat , &
L01594                                 ids , ide , jds , jde , kds , kde , &
L01595                                 ims , ime , jms , jme , kms , kme , &
L01596                                 its , ite , jts , jte , kts , kte )
L01597           END IF
L01598
L01599           !  For vertical coordinate, compute 1d arrays.
L01600
L01601           CALL compute_vcoord_1d_coeffs ( grid%ht, grid%etac, grid%znw, &
L01602                                           config_flags%hybrid_opt, &
L01603                                           r_d, g, p1000mb, &
L01604                                           grid%p_top, grid%p00, grid%t00, grid%tlp, &
L01605                                           ids, ide, jds, jde, kds, kde, &
L01606                                           ims, ime, jms, jme, kms, kme, &
L01607                                           its, ite, jts, jte, kts, kte, &
L01608                                           grid%znu, &
L01609                                           grid%c1f, grid%c2f, grid%c3f, grid%c4f, &
L01610                                           grid%c1h, grid%c2h, grid%c3h, grid%c4h )
L01611
L01612           IF ( config_flags%interp_theta ) THEN
L01613
L01614              !  The input field is temperature, we want potential temp.
L01615
L01616              CALL t_to_theta ( grid%t_gc , grid%p_gc , p00 , &
L01617                                ids , ide , jds , jde , 1   , num_metgrid_levels , &
L01618                                ims , ime , jms , jme , 1   , num_metgrid_levels , &
L01619                                its , ite , jts , jte , 1   , num_metgrid_levels )
L01620           END IF
```

讲解：①znw 的三种生成策略汇于 compute_eta 的 auto_levels_opt（namelist eta_levels 手工层 / auto_

levels 等 Δz 生成，max_dz/dzbot/dzstretch_s/u 四参数控制——namelist 对应 &dynamics 的 auto_levels_opt 等）；②compute_vcoord_1d_coeffs（code_41）随即生成 znu 与 c1..c4 八个一维系数（hybrid_opt 四选）；③interp_theta 时 metgrid 输入的温度场先转位温（t_to_theta）；④ 注意入参 grid%tlp——namelist 的 t00/a 组合在 Registry 里的别名。

四、基态构造五段

```
!file: dyn_em/module_initialize_real.F    L3768-L3834
L03768        !  Some of the many weird geopotential initializations that we'll see today: grid%ph0 is total,
L03769        !  and grid%ph_2 is a perturbation from the base state geopotential.  We set the base geopotential
L03770        !  at the lowest level to terrain elevation * gravity.
L03771
L03772        DO j=jts,jte
L03773           DO i=its,ite
L03774              IF ( skip_middle_points_t ( ids , ide , jds , jde , i , j , em_width , hold_ups ) ) CYCLE
L03775              grid%ph0(i,1,j) = grid%ht(i,j) * g
L03776              grid%ph_2(i,1,j) = 0.
L03777           END DO
L03778        END DO
L03779
L03780        !  Base state potential temperature and inverse density (alpha = 1/rho) from
L03781        !  the half eta levels and the base-profile surface pressure.  Compute 1/rho
L03782        !  from equation of state.  The potential temperature is a perturbation from t0.
L03783
L03784        DO j = jts, MIN(jte,jde-1)
L03785           DO i = its, MIN(ite,ide-1)
L03786
L03787              IF ( skip_middle_points_t ( ids , ide , jds , jde , i , j , em_width , hold_ups ) ) CYCLE
L03788
L03789              !  Base state pressure is a function of eta level and terrain, only, plus
L03790              !  the hand full of constants: p00 (sea level pressure, Pa), t00 (sea level
L03791              !  temperature, K), and A (temperature difference, from 1000 mb to 300 mb, K).
L03792
L03793              p_surf = p00 * EXP ( -t00/a + ( (t00/a)**2 - 2.*g*grid%ht(i,j)/a/r_d ) **0.5 )
L03794
L03795
L03796              DO k = 1, kte-1
L03797                 grid%php(i,k,j) = grid%c3f(k)*(p_surf - grid%p_top)+grid%c4f(k) + grid%p_top ! temporary, full lev base
↪  pressure
L03798                 grid%pb(i,k,j)  = grid%c3h(k)*(p_surf - grid%p_top)+grid%c4h(k) + grid%p_top
L03799                 temp = MAX ( tiso, t00 + A*LOG(grid%pb(i,k,j)/p00) )
L03800                 IF ( grid%pb(i,k,j) .LT. p_strat ) THEN
L03801                     temp = tiso + A_strat * LOG ( grid%pb(i,k,j)/p_strat )
L03802                 ENDIF
L03803  !              temp =             t00 + A*LOG(grid%pb(i,k,j)/p00)
L03804                 grid%t_init(i,k,j) = temp*(p00/grid%pb(i,k,j))**(r_d/cp) - t0
L03805                 grid%alb(i,k,j) = (r_d/p1000mb)*(grid%t_init(i,k,j)+t0)*(grid%pb(i,k,j)/p1000mb)**cvpm
L03806              END DO
L03807              grid%php(i,kte,j) = grid%p_top
L03808              !  Base state mu is defined as base state surface pressure minus grid%p_top
L03809              grid%MUB(i,j) = p_surf - grid%p_top
L03810              !  Dry surface pressure is defined as the following (this mu is from the input file
L03811              !  computed from the dry pressure).  Here the dry pressure is just reconstituted.
L03812              pd_surf = grid%MU0(i,j) + grid%p_top
L03813              !  Integrate base geopotential, starting at terrain elevation.  This assures that
L03814              !  the base state is in exact hydrostatic balance with respect to the model equations.
L03815              !  This field is on full levels.
L03816              grid%phb(i,1,j) = grid%ht(i,j) * g
L03817              IF (grid%hypsometric_opt == 1) THEN
L03818                 DO kk  = 2,kte
L03819                    k = kk-1
L03820                    grid%phb(i,kk,j) = grid%phb(i,kk-1,j) -
↪  grid%dnw(kk-1)*(grid%c1h(k)*grid%mub(i,j)+grid%c2h(k))*grid%alb(i,kk-1,j)
L03821                 END DO
L03822              ELSE IF (grid%hypsometric_opt == 2) THEN
L03823                 DO k = 2,kte
L03824                    pfu = grid%c3f(k  )*grid%MUB(i,j)+grid%c4f(k  ) + grid%p_top
L03825                    pfd = grid%c3f(k-1)*grid%MUB(i,j)+grid%c4f(k-1) + grid%p_top
L03826                    phm = grid%c3h(k-1)*grid%MUB(i,j)+grid%c4h(k-1) + grid%p_top
L03827                    grid%phb(i,k,j) = grid%phb(i,k-1,j) + grid%alb(i,k-1,j)*phm*LOG(pfd/pfu)
L03828                 END DO
L03829              ELSE
L03830                 CALL wrf_error_fatal( 'initialize_real: hypsometric_opt should be 1 or 2' )
L03831              END IF
L03832
L03833           END DO
L03834        END DO
```

讲解：**基态三件套**（「many weird geopotential initializations…today」的自嘲注释开场）：① **地面气压的 hypsometric 反解**：p_surf=p00 • exp(−t00/a+((t00/a)²−2g • ht/a/r_d)^0.5)——由常数对（p00 海平面气压、t00 海平面温度、a=1000→300 hPa 温差参数）与地形高度解析反解，无需迭代；② **基态气压剖面**：pb(k)=c3h(k) • (p_surf−p_top)+c4h(k)+p_top（混合坐标的 B(η) 加权，code_41 的 c3/c4 在此落地；php 是全层版临时量）；③ **两段式基态温度**：对流层 temp=max(tiso, t00+A • ln(pb/p00))（对数递减、tiso 折返防负递减），**pb<p_strat 切平流层** temp=tiso+A_strat • ln(pb/p_strat)（等温起步的递减）——ch10 基态公式的出处；t_init=temp • (p00/pb)$^{(r_d/cp)-t0\text{（扰动口径）}、alb=(r_d/p1000mb)(t_init+t0)(pb/p1000mb)}$cvpm（**cvpm=−cv/cp 口径的状态方程**，与 calc_p_rho_phi 静力分支同式）；④MUB=p_surf−p_top（基态干空气柱质量）；⑤phb 静水积分两式（hypsometric_opt=1 逐层 dnw • (c1h • MUB+c2h) • alb；=2 对数式——**与 calc_p_rho_phi 运行时完全同式**，注释自证「exact hydrostatic balance」）。

```
!file: dyn_em/module_initialize_real.F    L3879-L3902
L03879         !  Compute the total column perturbation dry pressure (grid%mub + grid%mu_2 + ptop = dry grid%psfc).
L03880
L03881         DO j = jts, min(jde-1,jte)
L03882            DO i = its, min(ide-1,ite)
L03883               IF ( skip_middle_points_t ( ids , ide , jds , jde , i , j , em_width , hold_ups ) ) CYCLE
L03884               grid%MU_2(i,j) = grid%MU0(i,j) - grid%MUB(i,j)
L03885            END DO
L03886         END DO
L03887
L03888         !  Fill in the outer rows and columns to allow us to be sloppy.
L03889
L03890         IF ( ite .EQ. ide ) THEN
L03891         i = ide
L03892         DO j = jts, MIN(jde-1,jte)
L03893            grid%MU_2(i,j) = grid%MU_2(i-1,j)
L03894         END DO
L03895         END IF
L03896
L03897         IF ( jte .EQ. jde ) THEN
L03898         j = jde
L03899         DO i = its, ite
L03900            grid%MU_2(i,j) = grid%MU_2(i,j-1)
L03901         END DO
L03902         END IF
```

讲解：**MU_2=MU0−MUB**——总干空气柱（MU0 来自 metgrid 的干气压积分）减基态即初始扰动；边界列复制（「allow us to be sloppy」——外圈 slop 填充的祖传注释）。

```
!file: dyn_em/module_initialize_real.F    L3904-L3943
L03904         lev500 = 0
L03905         DO j = jts, min(jde-1,jte)
L03906            DO i = its, min(ide-1,ite)
L03907               IF ( skip_middle_points_t ( ids , ide , jds , jde , i , j , em_width , hold_ups ) ) CYCLE
L03908
L03909               !  Assign the potential temperature (perturbation from t0) and qv on all the mass
L03910               !  point locations.
L03911
L03912               DO k =  1 , kde-1
L03913                  grid%t_2(i,k,j)          = grid%t_2(i,k,j) - t0
L03914               END DO
L03915
L03916               dpmu = 10001.
L03917               loop_count = 0
L03918
L03919               DO WHILE ( ( ABS(dpmu) .GT. 10. ) .AND. &
L03920                          ( loop_count .LT. 5 ) )
L03921
L03922                  loop_count = loop_count + 1
L03923
L03924                  !  Integrate the hydrostatic equation (from the RHS of the bigstep vertical momentum
L03925                  !  equation) down from the top to get the pressure perturbation.  First get the pressure
L03926                  !  perturbation, moisture, and inverse density (total and perturbation) at the top-most level.
L03927
L03928                  kk = kte-1
L03929                  k = kk+1
L03930
L03931                  qtot=0.
L03932                  DO im = PARAM_FIRST_SCALAR, num_3d_m
L03933                    qtot = qtot + moist(i,kk,j,im)
L03934                  ENDDO
L03935                  qvf2 = 1./(1.+qtot)
L03936                  qvf1 = qtot*qvf2
L03937
L03938                  grid%p(i,kk,j) = - 0.5*((grid%c1f(k)*grid%Mu_2(i,j))+qvf1*(grid%c1f(k)*grid%Mub(i,
↪  j)+grid%c2f(k)))/grid%rdnw(kk)/qvf2
L03939                  qvf = 1. + rvovrd*moist(i,kk,j,P_QV)
L03940                  grid%alt(i,kk,j) = (r_d/p1000mb)*(grid%t_2(i,kk,j)+t0)*qvf&
L03941                                  *(((grid%p(i,kk,j)+grid%pb(i,kk,j))/p1000mb)**cvpm)
L03942                  grid%al(i,kk,j) = grid%alt(i,kk,j) - grid%alb(i,kk,j)
L03943                  grid%p_hyd(i,kk,j) = grid%p(i,kk,j) + grid%pb(i,kk,j)
```

讲解：扰动初值：t_2 逐点减 t0 转扰动口径；**顶层起算的扰动气压** p(i,kte−1)=−0.5(c1f • mu_2+qvf1 • (c1f • Mub+c2f))/rdnw（qvf1=Σ 水物质/(1+Σ)——与 calc_p_rho_phi 静力分支同式、水量份额版本），向下积分同款；alt 全量比重容由状态方程、al=alt−alb、p_hyd=p+pb 存档。

```
!file: dyn_em/module_initialize_real.F    L3964-L3992
L03964  #if 1
L03965                  !  This is the hydrostatic equation used in the model after the small timesteps.  In
L03966                  !  the model, grid%al (inverse density) is computed from the geopotential.
L03967
L03968                  IF (grid%hypsometric_opt == 1) THEN
L03969                     DO kk  = 2,kte
L03970                        k = kk - 1
L03971                        grid%ph_2(i,kk,j) = grid%ph_2(i,kk-1,j) - &
L03972                                            grid%dnw(kk-1) * ( ((grid%c1h(k)*grid%mub(i,
↪  j)+grid%c2h(k))+(grid%c1h(k)*grid%mu_2(i,j)))*grid%al(i,kk-1,j) &
L03973                                          + (grid%c1h(k)*grid%mu_2(i,j))*grid%alb(i,kk-1,j) )
L03974                        grid%ph0(i,kk,j) = grid%ph_2(i,kk,j) + grid%phb(i,kk,j)
L03975                     END DO
L03976                  ELSE IF (grid%hypsometric_opt == 2) THEN
L03977                   ! Alternative hydrostatic eq.: dZ = -al*p*dLOG(p), where p is dry pressure.
L03978                   ! Note that al*p approximates Rd*T and dLOG(p) does z.
L03979                   ! Here T varies mostly linear with z, the first-order integration produces better result.
L03980
L03981                     grid%ph_2(i,1,j) = grid%phb(i,1,j)
L03982                     DO k = 2,kte
L03983                        pfu = grid%c3f(k  )*grid%MU0(i,j)+grid%c4f(k  ) + grid%p_top
L03984                        pfd = grid%c3f(k-1)*grid%MU0(i,j)+grid%c4f(k-1) + grid%p_top
L03985                        phm = grid%c3h(k-1)*grid%MU0(i,j)+grid%c4h(k-1) + grid%p_top
```

```
L03986                      grid%ph_2(i,k,j) = grid%ph_2(i,k-1,j) + grid%alt(i,k-1,j)*phm*LOG(pfd/pfu)
L03987                   END DO
L03988
L03989                   DO k = 1,kte
L03990                      grid%ph_2(i,k,j) = grid%ph_2(i,k,j) - grid%phb(i,k,j)
L03991                   END DO
L03992                END IF
```

讲解：**ph_2 的静水重建双式**：hypsometric_opt=1 用模型运行时的逐层式（注意 (c1h • mub+c2h)+(c1h • mu_2) 的合并写法——总柱质量分解为基态 +扰动）；=2 走 dZ=−al • p • dlnp 对数修订（注释：T 随 z 近线性、一阶积分更好），最后统一减 phb 转扰动口径。#if 0 段保留「直接用 WPS 三维高度场减 phb」的废弃路线。

```
!file: dyn_em/module_initialize_real.F    L4001-L4041
L04001                !  Recompute density, simlar to what the model does.
L04002
L04003                IF (grid%hypsometric_opt == 1) THEN
L04004                DO k=kts,kte-1
L04005                   grid%al(i,k,j)=-1./((grid%c1h(k)*grid%mub(i,j)+grid%c2h(k))+(grid%c1h(k)*grid%mu_2(i,
↪  j)))*(grid%alb(i,k,j)*(grid%c1h(k)*grid%mu_2(i,j))  &
L04006                                  +grid%rdnw(k)*(grid%ph_2(i,k+1,j)-grid%ph_2(i,k,j)))
L04007                ENDDO
L04008                ELSE IF (grid%hypsometric_opt == 2) THEN
L04009                DO k=kts,kte-1
L04010                   pfu = grid%c3f(k+1)*(grid%MUB(i,j)+grid%MU_2(i,j))+grid%c4f(k+1)+grid%p_top
L04011                   pfd = grid%c3f(k  )*(grid%MUB(i,j)+grid%MU_2(i,j))+grid%c4f(k  )+grid%p_top
L04012                   phm = grid%c3h(k  )*(grid%MUB(i,j)+grid%MU_2(i,j))+grid%c4h(k  )+grid%p_top
L04013                   qvf=-1./((grid%c1h(k)*grid%mub(i,j)+grid%c2h(k))+(grid%c1h(k)*grid%mu_2(i,j)))*(grid%alb(i,k,
↪  j)*(grid%c1h(k)*grid%mu_2(i,j))  &
L04014                                  +grid%rdnw(k)*(grid%ph_2(i,k+1,j)-grid%ph_2(i,k,j)))
L04015                   grid%al(i,k,j) = (grid%ph_2(i,k+1,j)-grid%ph_2(i,k,j)+grid%phb(i,k+1,j)-grid%phb(i,k,j)) &
L04016                                     /phm/LOG(pfd/pfu)-grid%alb(i,k,j)
L04017  #if 0
L04018  if ( internal_time_loop .EQ. 1 ) THEN
L04019  if (i.eq.its .and. j.eq.its)then
L04020  if (k.eq.kts)then
L04021  print *,'                                  k        old al          new al             alb            new alt           dz
↪  (m)       pres up          Pres mid        Pres down          c3 k           c3 k+1             c4 k            c4 k+1'
L04022  print *,' ================================================================================'
↪  ================================================================================================================================
↪  ================================'
L04023  endif
L04024  print *,'                  ',k,qvf,grid%al(i,k,j),grid%alb(i,k,j),grid%al(i,k,j)+grid%alb(i,k,j),(grid%ph_2(i,k+1,
↪  j)-grid%ph_2(i,k,j)+grid%phb(i,k+1,j)-grid%phb(i,k,j)),pfu,phm,pfd,grid%c3f(k),grid%c3f(k+1),grid%c4f(k),grid%c4f(k+1)
L04025  endif
L04026  endif
L04027  #endif
L04028
L04029                ENDDO
L04030                END IF
L04031
L04032                !  Compute pressure similarly to how computed within model.
L04033
L04034                DO k=kts,kte-1
L04035                   qvf = 1.+rvovrd*moist(i,k,j,P_QV)
L04036                   grid%p(i,k,j)=p1000mb*( (r_d*(t0+grid%t_2(i,k,j))*qvf)/                    &
L04037                                 (p1000mb*(grid%al(i,k,j)+grid%alb(i,k,j))) )**cpovcv  &
L04038                                 -grid%pb(i,k,j)
L04039                   grid%p_hyd(i,k,j) = grid%p(i,k,j) + grid%pb(i,k,j)
L04040                   grid%alt(i,k,j) = grid%al(i,k,j) + grid%alb(i,k,j)
L04041                ENDDO
```

讲解：「**Recompute density, similar to what the model does**」——al 不用状态方程值而由 ph_2 的几何差分反解（与 calc_p_rho_phi 非静力分支同式的 1/2 两版；opt=2 的 qvf 变量名复用是排版残留，式子正确），随后 p 再走一次状态方程（cpovcv 正口径）、p_hyd=p+pb、alt=al+alb——**保证初值与模式第一小步的算法逐位同源**（wrfinput 的 p/alt 与运行第一步诊断一致的关键）。#if 0 段是 TKW 调 opt=2 时的对账打印表。

```
!file: dyn_em/module_initialize_real.F    L4043-L4108
L04043                !  Adjust the column pressure so that the computed 500 mb height is close to the
L04044                !  input value (of course, not when we are doing hybrid input).
L04045
L04046                IF ( ( flag_metgrid .EQ. 1 ) .AND. ( i .EQ. i_valid ) .AND. ( j .EQ. j_valid ) ) THEN
L04047                   DO k = 1 , num_metgrid_levels
L04048                      IF ( ABS ( grid%p_gc(i,k,j) - 50000. ) .LT. 1. ) THEN
L04049                         lev500 = k
L04050                         EXIT
L04051                      END IF
L04052                   END DO
L04053                END IF
L04054
L04055                !  We only do the adjustment of height if we have the input data on pressure
L04056                !  surfaces, and folks have asked to do this option.
L04057
L04058                IF ( ( flag_metgrid .EQ. 1 ) .AND. &
L04059                     ( flag_ptheta  .EQ. 0 ) .AND. &
L04060                     ( config_flags%adjust_heights ) .AND. &
L04061                     ( lev500 .NE. 0 ) ) THEN
L04062
L04063                   DO k = 2 , kte-1
L04064
L04065                      !  Get the pressures on the full eta levels (grid%php is defined above as
L04066                      !  the full-lev base pressure, an easy array to use for 3d space).
L04067
L04068                      pl = grid%php(i,k  ,j) + &
L04069                           ( grid%p(i,k-1  ,j) * ( grid%znw(k    ) - grid%znu(k  ) ) + &
```

```
                              grid%p(i,k    ,j) * ( grid%znu(k-1  ) - grid%znw(k  ) ) / &
                            ( grid%znu(k-1  ) - grid%znu(k  ) )
                       pu = grid%php(i,k+1,j) + &
                            ( grid%p(i,k-1+1,j) * ( grid%znw(k  +1) - grid%znu(k+1) ) + &
                              grid%p(i,k  +1,j) * ( grid%znu(k-1+1) - grid%znw(k+1) ) ) / &
                            ( grid%znu(k-1+1) - grid%znu(k+1) )

                       !  If these pressure levels trap 500 mb, use them to interpolate
                       !  to the 500 mb level of the computed height.

                       IF ( ( pl .GE. 50000. ) .AND. ( pu .LT. 50000. ) ) THEN
                          zl = ( grid%ph_2(i,k  ,j) + grid%phb(i,k  ,j) ) / g
                          zu = ( grid%ph_2(i,k+1,j) + grid%phb(i,k+1,j) ) / g

                          z500 = ( zl * ( LOG(50000.) - LOG(pu    ) ) + &
                                   zu * ( LOG(pl    ) - LOG(50000.) ) ) / &
                                 ( LOG(pl) - LOG(pu) )
!                         z500 = ( zl * (    (50000.) -    (pu    ) ) + &
!                                  zu * (    (pl    ) -    (50000.) ) ) / &
!                                (    (pl) -    (pu) )

                          !  Compute the difference of the 500 mb heights (computed minus input), and
                          !  then the change in grid%mu_2.  The grid%php is still full-levels, base pressure.

                          dz500 = z500 - grid%ght_gc(i,lev500,j)
                          tvsfc = ((grid%t_2(i,1,j)+t0)*((grid%p(i,1,j)+grid%php(i,1,j))/p1000mb)**(r_d/cp)) * &
                                   (1.+0.6*moist(i,1,j,P_QV))
                          dpmu = ( grid%php(i,1,j) + grid%p(i,1,j) ) * EXP ( g * dz500 / ( r_d * tvsfc ) )
                          dpmu = dpmu - ( grid%php(i,1,j) + grid%p(i,1,j) )
                          grid%MU_2(i,j) = grid%MU_2(i,j) - dpmu
                          EXIT
                       END IF

                    END DO
                 ELSE
                    dpmu = 0.
                 END IF

              END DO
```

讲解：**500 hPa 高度迭代**（DO WHILE |dpmu|>10 且 <5 次）：adjust_heights=true 且输入在气压面时，找 metgrid 的 500 hPa 层（lev500），把计算柱在 500 hPa 的高度 z500（**ln(p) 对数插值**，注释保留线性旧式）与输入 ght_gc 之差 dz500，经地面虚温的 hypsometric 式 dpmu=p_sfc • exp(g • dz500/(r_d • Tv))−p_sfc 反推 MU_2 增量并 EXIT 重算整柱——**让初始 500 hPa 高度场与 metgrid 一致**的经典调整（教材 ch07 的 adjust_heights 语义在此落地）。

五、未录单元注记（42 单元分组）

1. **插值族**：vert_interp（752 行）/lagrange_setup/lagrange_interp/cubic_spline/TRIDIAGONAL_LINEAR_EQ——metgrid 层到模式层的垂直插值兵器库。
2. **干气压与转换**：p_dry/p_dts/p_dhs/find_p_top/t_to_theta/theta_to_t——干气压积分与温位温互转。
3. **水物质**：integ_moist/rh_to_mxrat1/2——RH→ 混合比（两套饱和式）。
4. **层生成**：compute_eta（本文件自己的 328 行版，含 dzstretch 拉伸与 prac_levels=59 的「silly levels」顶高估算——code_41 的 nest 版是同源简化）/levels。
5. **月平均场**：monthly_min_max/monthly_avg/monthly_interp_to_date/eightday_selector——SST/植被等月场插值。
6. **地面气压诊断**：sfcprs/sfcprs2/sfcprs3——诊断 2 m/10 m 场的静水外推。
7. **地形滤波**：filter_topo/filter_topo_old——topo_wind/GWD 用的高通滤波。
8. **杂项**：dry_stratos（平流层除湿，调用处被注释）/snowHires/make_IceNumber/skip_middle_points_t（亚域跳点谓词）。

六、改造入口

1. **基态廓线**：t00/a/tiso/p_strat/a_strat 五常数（namelist &dynamics 的 real 系）——理想基态敏感性实验。
2. **混合坐标**：hybrid_opt/etac（code_41）——本篇的 pb/phb/alb 全部自动跟随。
3. **500 hPa 调整**：adjust_heights=false 关闭迭代；dpmu 容差 10 Pa/5 次上限。
4. **初值同源性**：改 calc_p_rho_phi 的状态方程时必须同步本篇五段（「similar to what the model does」的对账关系）。

七、上下游链接

- 系数生成：`code_41_nest_init_utils.md`；运行时同式：`code_30_module_big_step_utilities_em.md`（calc_p_rho_phi）

- 常数底座：`code_01_module_model_constants.md`
- 教材正文：ch07_real_namelist.md、ch10_vertical_coords_hydrostatics.md

code_41 — 垂直坐标系数生成（dyn_em/nest_init_utils.F：compute_eta 与 compute_vcoord_1d_coeffs）

一、版本指纹

源文件	行数	md5（整文件，按字节）
dyn_em/nest_init_utils.F	1184	89b5f314f5c278d518df21bfe8303d87

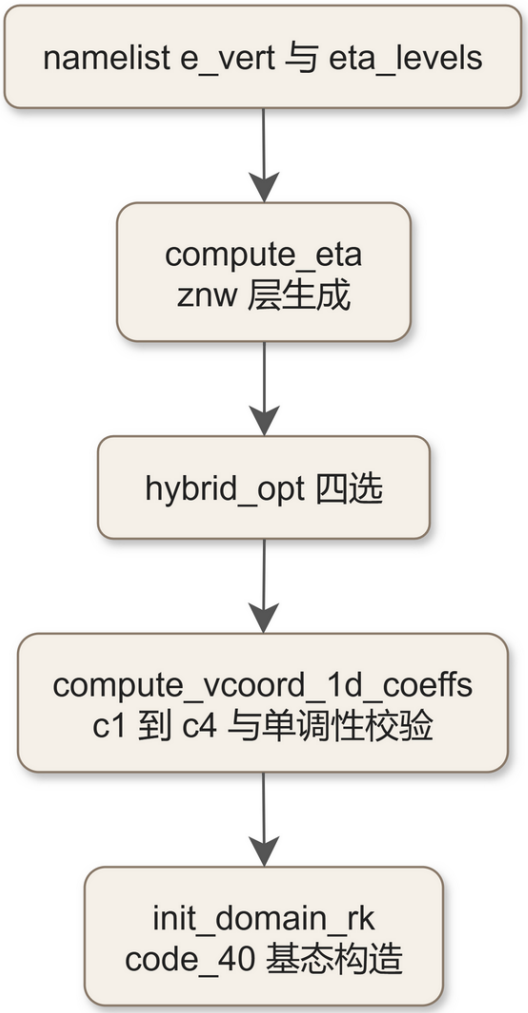
- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（B）：compute_vcoord_1d_coeffs 全段 + compute_eta 头段；其余 11 单元注记见 § 五。教材锚点：ch10_vertical_coords_hydrostatics.md（混合坐标公式）、`code_40_module_initialize_real.md`（消费方）。

二、定位

一句话：**垂直坐标的全部一维系数出生地**——znw（η 层面）由 compute_eta 生成（namelist 手工层或 auto_levels 等 Δz），compute_vcoord_1d_coeffs 随即按 hybrid_opt 四选生成混合坐标八系数 c1..c4（znu/c1f..c4f/c1h..c4h）并做参考气压单调性校验。

谁在调它：module_initialize_real 的 init_domain_rk（`code_40_module_initialize_real.md` § 三调用点）与 nest 的垂直嵌套初始化（init_domain_vert_nesting）；rebalance 也走同款。

13 单元表：init_domain_constants_em（L4–133）/init_domain_vert_nesting（L138–180）/vert_cor_vertical_nesting_integer（L187–267）/vert_cor_vertical_nesting_arbitrary（L271–427）/**compute_eta（L429–708）**/blend_terrain（L712–785）/copy_3d_field（L787–810）/adjust_tempqv（L812–890）/input_terrain_rsmas（L892–975）/update_after_feedback_em（L977–1031）/**compute_vcoord_1d_coeffs（L1033–1184）**。



三、compute_vcoord_1d_coeffs（全文）

```
!file: dyn_em/nest_init_utils.F    L1033-L1184
L01033  SUBROUTINE compute_vcoord_1d_coeffs ( ht, etac, znw, &
L01034                                        hybrid_opt, &
L01035                                        r_d, g, p1000mb, &
L01036                                        p_top, p00, t00, a, &
L01037                                        ids, ide, jds, jde, kds, kde, &
L01038                                        ims, ime, jms, jme, kms, kme, &
L01039                                        its, ite, jts, jte, kts, kte, &
L01040                                        znu, &
L01041                                        c1f, c2f, c3f, c4f, &
L01042                                        c1h, c2h, c3h, c4h )
L01043
L01044     IMPLICIT NONE
L01045
L01046     !  Input
L01047
L01048     INTEGER ,                       INTENT(IN ) :: hybrid_opt
L01049     REAL    ,                       INTENT(IN ) :: etac
L01050     REAL    ,                       INTENT(IN ) :: r_d, g, p1000mb, p_top, p00, t00, a
L01051     INTEGER ,                       INTENT(IN ) :: ids, ide, jds, jde, kds, kde, &
L01052                                                    ims, ime, jms, jme, kms, kme, &
L01053                                                    its, ite, jts, jte, kts, kte
L01054     REAL    , DIMENSION(kms:kme) ,           INTENT(IN ) :: znw
L01055     REAL    , DIMENSION(ims:ime , jms:jme) , INTENT(IN ) :: ht
L01056
L01057     !  Output
L01058
L01059     REAL    , DIMENSION(kms:kme) , INTENT(OUT) :: znu
L01060     REAL    , DIMENSION(kms:kme) , INTENT(OUT) :: c1f, c2f, c3f, c4f, &
L01061                                                   c1h, c2h, c3h, c4h
L01062
L01063     !  Local vars
L01064
L01065     INTEGER :: i, j, k
L01066     REAL    :: B1, B2, B3, B4, B5
L01067     REAL    :: p_surf, press_above, press_below
L01068     CHARACTER (LEN=256) :: a_message
L01069
L01070     DO k=kts, kte
L01071        IF      ( hybrid_opt .EQ. 0 ) THEN
```

```
L01072           c3f(k) = znw(k)
L01073        ELSE IF ( hybrid_opt .EQ. 1 ) THEN
L01074           c3f(k) = znw(k)
L01075        ELSE IF ( hybrid_opt .EQ. 2 ) THEN
L01076           B1 = 2. * etac**2 * ( 1. - etac )
L01077           B2 = -etac * ( 4. - 3. * etac - etac**3 )
L01078           B3 = 2. * ( 1. - etac**3 )
L01079           B4 = - ( 1. - etac**2 )
L01080           B5 = (1.-etac)**4
L01081           c3f(k) = ( B1 + B2*znw(k) + B3*znw(k)**2 + B4*znw(k)**3 ) / B5
L01082           IF ( znw(k) .LT. etac ) THEN
L01083              c3f(k) = 0.
L01084           END IF
L01085           IF ( k .EQ. kds ) THEN
L01086              c3f(k) = 1.
L01087           ELSE IF ( k .EQ. kde ) THEN
L01088              c3f(k) = 0.
L01089           END IF
L01090        ELSE IF ( hybrid_opt .EQ. 3 ) THEN
L01091           c3f(k) = znw(k)*sin(0.5*3.14159*znw(k))**2
L01092           IF      ( k .EQ. kds ) THEN
L01093              c3f(k) = 1.
L01094           ELSE IF ( k .EQ. kde ) THEN
L01095              c3f(kde) = 0.
L01096           END IF
L01097        ELSE
L01098           CALL wrf_message     ( 'ERROR: --- hybrid_opt' )
L01099           CALL wrf_message     ( 'ERROR: --- hybrid_opt=0    ==> Standard WRF terrain-following coordinate' )
L01100           CALL wrf_message     ( 'ERROR: --- hybrid_opt=1    ==> Standard WRF terrain-following coordinate, hybrid c1,
↪  c2, c3, c4' )
L01101           CALL wrf_message     ( 'ERROR: --- hybrid_opt=2    ==> Hybrid, Klemp polynomial' )
L01102           CALL wrf_message     ( 'ERROR: --- hybrid_opt=3    ==> Hybrid, sin^2' )
L01103           CALL wrf_error_fatal ( 'ERROR: --- Invalid option' )
L01104        END IF
L01105     END DO
L01106
L01107     !  c4 is a function of c3 and eta.
L01108
L01109     DO k=1, kde
L01110        c4f(k) = ( znw(k) - c3f(k) ) * ( p1000mb - p_top )
L01111     ENDDO
L01112
L01113     !  Now on half levels, just add up and divide by 2 (for c3h).  Use (eta-c3)*(p00-pt) for c4 on half levels.
L01114
L01115     DO k=1, kde-1
L01116        znu(k) = ( znw(k+1) + znw(k) ) * 0.5
L01117        c3h(k) = ( c3f(k+1) + c3f(k) ) * 0.5
L01118        c4h(k) = ( znu(k) - c3h(k) ) * ( p1000mb - p_top )
L01119     ENDDO
L01120
L01121     !  c1 = d(B)/d(eta).  We define c1f as c1 on FULL levels.  For a vertical difference,
L01122     !  we need to use B and eta on half levels.  The k-loop ends up referring to the
L01123     !  full levels, neglecting the top and bottom.
L01124
L01125     DO k=kds+1, kde-1
L01126        c1f(k) = ( c3h(k) - c3h(k-1) ) / ( znu(k) - znu(k-1) )
L01127     ENDDO
L01128
L01129     !  The boundary conditions to get the coefficients:
L01130     !  1) At k=kts: define d(B)/d(eta) = 1.  This gives us the same value of B and d(B)/d(eta)
L01131     !     when doing the sigma-only B=eta.
L01132     !  2) At k=kte: define d(B)/d(eta) = 0.  The curve B SMOOTHLY goes to zero, and at the very
L01133     !     top, B continues to SMOOTHLY go to zero.  Note that for almost all cases of non B=eta,
L01134     !     B is ALREADY=ZERO at the top, so this is a reasonable BC to assume.
L01135
L01136     c1f(kds) = 1.
L01137     IF ( ( hybrid_opt .EQ. 0 ) .OR. ( hybrid_opt .EQ. 1 ) ) THEN
L01138        c1f(kde) = 1.
L01139     ELSE
L01140        c1f(kde) = 0.
L01141     END IF
L01142
L01143     !  c2 = ( 1. - c1(k) ) * (p00 - pt).  There is no vertical differencing, so we can do the
L01144     !  full kds to kde looping.
L01145
L01146     DO k=kds, kde
L01147        c2f(k) = ( 1. - c1f(k) ) * ( p1000mb - p_top )
L01148     ENDDO
L01149
L01150     !  Now on half levels for c1 and c2.  The c1h will result from the full level c3 and full
L01151     !  level eta differences.  The c2 value use the half level c1(k).
L01152
L01153     DO k=1, kde-1
L01154        c1h(k) = ( c3f(k+1) - c3f(k) ) / ( znw(k+1) - znw(k) )
L01155        c2h(k) = ( 1. - c1h(k) ) * ( p1000mb - p_top )
L01156     ENDDO
L01157
L01158     !  With c3 and c4 on full levels, we can verify that we are maintaining monotonically
L01159     !  decreasing reference pressure.
L01160     !  P_dry_ref(k)  = c3f(k) * ( psurf - p_top ) + c4f(k) + p_top
L01161
L01162     DO j = jts, MIN(jde-1,jte)
L01163        DO i = its, MIN(ide-1,ite)
L01164           p_surf = p00 * EXP ( -t00/a + ( (t00/a)**2 - 2.*g*ht(i,j)/a/r_d ) **0.5 )
L01165           DO k=1, kde-1
L01166              press_above = c3f(k+1)*(p_surf-p_top) + c4f(k+1) + p_top
L01167              press_below = c3f(k  )*(p_surf-p_top) + c4f(k  ) + p_top
L01168              IF ( press_below - press_above .LE. 0.0 ) THEN
L01169                 CALL wrf_message ( '----- ERROR: The reference pressure is not monotonically decreasing' )
L01170                 CALL wrf_message ( '             This tends to be caused by very high topography')
L01171                 WRITE (a_message,*) '(i,j) = ',i,j,', topography = ',ht(i,j),' m'
L01172                 CALL wrf_message ( TRIM(a_message) )
L01173                 WRITE (a_message,*) 'k = ',k,', reference pressure = ',press_below,' Pa'
L01174                 CALL wrf_message ( TRIM(a_message) )
L01175                 WRITE (a_message,*) 'k = ',k+1,', reference pressure = ',press_above,' Pa'
L01176                 CALL wrf_message ( TRIM(a_message) )
L01177                 WRITE (a_message,*) 'In the dynamics namelist record, reduce etac from ',etac
L01178                 CALL wrf_error_fatal ( TRIM(a_message) )
```

```
L01179             END IF
L01180           ENDDO
L01181         ENDDO
L01182       ENDDO
L01183
L01184   END SUBROUTINE compute_vcoord_1d_coeffs
```

讲解：混合坐标 B(η) 与四系数：①**hybrid_opt 四选**——0/1 标准地形追随坐标（c3f=znw，即 B=σ；1 只是用 c1..c4 通路实现）、**2 Klemp 多项式**：c3f=(B1+B2 • η+B3 • η²+B4 • η³)/B5，五个系数只依赖 etac（B1=2ηc²(1−ηc)、B2=−ηc(4−3ηc−ηc³)、B3=2(1−ηc³)、B4=−(1−ηc²)、B5=(1−ηc)⁴），η<etac 时 c3f=0（纯等压面），边界处强制 c3f(kds)=1/c3f(kde)=0；**3 sin² 型**：c3f=η • sin²(π • η/2)——Klems 式之外的备选混合函数；②c4f=(η−c3) • (p00−pt)——**扰动气压 =η 份额与 B 份额的组合**：p_dry=c3 • (p_surf−pt)+c4+pt 的两系数分解（c3 权重地形气压、c4 补偿项）；③znu 半层 eta、c3h/c4h 半层平均；④**c1=dB/dη 用半层差分**（注释解释为什么用 B 与 η 的半层差而不是解析导数），边界条件：kds 处 c1f=1（与纯 σ 完全衔接）、kde 处非混合为 1、混合为 0（B 平滑归零）；c2f=(1−c1f) • (p00−pt)；c1h/c2h 半层版；⑤ **单调性校验**（逐列！）：参考气压 P_dry_ref(k)=c3f(k)(p_surf−pt)+c4f(k)+pt 必须逐层递减，p_surf 由 hypsometric 反解（与 code_40 同式），否则 **wrf_error_fatal「reduce etac」并打印地形高度与两层气压**——高地形 + etac 过大时 real 报错的经典出处（兰州个例 3 km 域遇过），解法就是降 etac 或加层。

四、compute_eta（头段）

```
!file: dyn_em/nest_init_utils.F    L429-L563
L00429  SUBROUTINE compute_eta ( znw , &
L00430                           eta_levels , max_eta , max_dz ,        &
L00431                           p_top_def , g_def , p00_def ,          &
L00432                           cvpm_def , a_def , r_d_def , cp_def ,  &
L00433                           t00_def , p1000mb_def , t0_def ,       &
L00434                           tiso_def , p_strat_def , a_strat_def , &
L00435                           ids , ide , jds , jde , kds , kde ,    &
L00436                           ims , ime , jms , jme , kms , kme ,    &
L00437                           its , ite , jts , jte , kts , kte )
L00438
L00439        !  Compute eta levels, either using given values from the namelist (hardly
L00440        !  a computation, yep, I know), or assuming a constant dz above the PBL,
L00441        !  knowing p_top and the number of eta levels.
L00442
L00443        IMPLICIT NONE
L00444
L00445        INTEGER , INTENT(IN)        :: ids , ide , jds , jde , kds , kde , &
L00446                                       ims , ime , jms , jme , kms , kme , &
L00447                                       its , ite , jts , jte , kts , kte
L00448        REAL , INTENT(IN)           :: max_dz
L00449        REAL , INTENT(IN)           :: p_top_def , g_def , p00_def , cvpm_def , &
L00450                                       a_def , r_d_def , cp_def , t00_def ,     &
L00451                                       p1000mb_def , t0_def , tiso_def ,        &
L00452                                       p_strat_def , a_strat_def
L00453        INTEGER , INTENT(IN)        :: max_eta
L00454        REAL , DIMENSION (max_eta)  :: eta_levels
L00455
L00456        REAL , DIMENSION (kts:kte) , INTENT(OUT) :: znw
L00457
L00458        !  Local vars
L00459
L00460        INTEGER :: k , kk
L00461        REAL(KIND=8) :: mub , t_init , p_surf , pb, ztop, ztop_pbl , dz , temp
L00462        REAL(KIND=8) , DIMENSION(kts:kte) :: dnw
L00463        REAL(KIND=8) :: p_top , g , p00 , cvpm , &
L00464                        a , r_d , cp , t00 ,     &
L00465                        p1000mb , t0 , tiso ,        &
L00466                        p_strat , a_strat
L00467
L00468        INTEGER , PARAMETER :: prac_levels = 59
L00469        INTEGER :: loop , loop1
L00470        REAL(KIND=8) , DIMENSION(prac_levels) :: znw_prac , znu_prac , dnw_prac
L00471        REAL(KIND=8) , DIMENSION(MAX(prac_levels,kde)) :: alb , phb
L00472        REAL(KIND=8) :: alb_max, t_init_max, pb_max, phb_max
L00473
L00474        CHARACTER(LEN=256) :: message
L00475
L00476        !  Compute top of the atmosphere with some silly levels.  We just want to
L00477        !  integrate to get a reasonable value for ztop.  We use the planned
L00478        !  PBL-esque
L00479        !  levels, and then just coarse resolution above that.  We know p_top,
L00480        !  and we
L00481        !  have the base state vars.
L00482
L00483        p_top   = p_top_def
L00484        g       = g_def
L00485        p00     = p00_def
L00486        cvpm    = cvpm_def
L00487        a       = a_def
L00488        r_d     = r_d_def
L00489        cp      = cp_def
L00490        t00     = t00_def
L00491        p1000mb = p1000mb_def
L00492        t0      = t0_def
L00493        tiso    = tiso_def
L00494        p_strat = p_strat_def
L00495        a_strat = a_strat_def
L00496
L00497        p_surf = p00
L00498
```

```
L00499        znw_prac = (/ 1.0000_8 , 0.9930_8 , 0.9830_8 , 0.9700_8 , 0.9540_8 , 0.9340_8 , 0.9090_8 , 0.8800_8 , &
L00500                      0.8500_8 , 0.8000_8 , 0.7500_8 , 0.7000_8 , 0.6500_8 , 0.6000_8 , 0.5500_8 , 0.5000_8 , &
L00501                      0.4500_8 , 0.4000_8 , 0.3500_8 , 0.3000_8 , 0.2500_8 , 0.2000_8 , 0.1500_8 , 0.1000_8 , &
L00502                      0.0800_8 , 0.0600_8 , 0.0400_8 , 0.0200_8 , &
L00503                      0.0150_8 , 0.0100_8 , 0.0090_8 , 0.0080_8 , 0.0070_8 , 0.0060_8 , 0.0050_8 , 0.0040_8 , &
L00504                      0.0035_8 , 0.0030_8 , &
L00505                      0.0028_8 , 0.0026_8 , 0.0024_8 , 0.0022_8 , 0.0020_8 , &
L00506                      0.0018_8 , 0.0016_8 , 0.0014_8 , 0.0012_8 , 0.0010_8 , &
L00507                      0.0009_8 , 0.0008_8 , 0.0007_8 , 0.0006_8 , 0.0005_8 , 0.0004_8 , 0.0003_8 , &
L00508                      0.0002_8 , 0.0001_8 , 0.00005_8, 0.0000_8 /)
L00509
L00510        DO k = 1 , prac_levels - 1
L00511          znu_prac(k) = ( znw_prac(k) + znw_prac(k+1) ) * 0.5_8
L00512          dnw_prac(k) = znw_prac(k+1) - znw_prac(k)
L00513        END DO
L00514
L00515        DO k = 1, prac_levels-1
L00516          pb = znu_prac(k)*(p_surf - p_top) + p_top
L00517          temp = MAX ( tiso, t00 + A*LOG(pb/p00) )
L00518          IF ( pb .LT. p_strat ) THEN
L00519            temp = tiso + A_strat*LOG(pb/p_strat)
L00520          END IF
L00521          t_init = temp*(p00/pb)**(r_d/cp) - t0
L00522          alb(k) = (r_d/p1000mb)*(t_init+t0)*(pb/p1000mb)**cvpm
L00523        END DO
L00524
L00525        !  Base state mu is defined as base state surface pressure minus p_top
L00526
L00527        mub = p_surf - p_top
L00528
L00529        !  Integrate base geopotential, starting at terrain elevation.
L00530
L00531        phb(1) = 0._8
L00532        DO k  = 2,prac_levels
L00533          phb(k) = phb(k-1) - dnw_prac(k-1)*mub*alb(k-1)
L00534        END DO
L00535
L00536        !  So, now we know the model top in meters.  Get the average depth above the PBL
L00537        !  of each of the remaining levels.  We are going for a constant delta z thickness.
L00538
L00539        ztop     = phb(prac_levels) / g
L00540        ztop_pbl = phb(8          ) / g
L00541        dz = ( ztop - ztop_pbl ) / REAL ( kde - 8 )
L00542
L00543        IF ( dz .GE. max_dz ) THEN
L00544          WRITE (message,FMT='("With a requested ",F7.1," Pa model top, the model lid will be about ",F7.1," m.")')
↪  p_top, ztop
L00545          CALL wrf_message ( message )
L00546          WRITE (message,FMT='("With ",I3," levels above the PBL, the level thickness will be about ",F6.1," m.")')
↪  kde-8, dz
L00547          CALL wrf_message ( message )
L00548          WRITE (message,FMT='("Thicknesses greater than ",F7.1," m are not recommended.")') max_dz
L00549          CALL wrf_message ( message )
L00550          CALL wrf_error_fatal ( 'Add more levels to namelist.input for e_vert' )
L00551        END IF
L00552
L00553        !  Standard levels near the surface so no one gets in trouble.
L00554
L00555        DO k = 1 , 8
L00556          eta_levels(k) = znw_prac(k)
L00557        END DO
L00558
L00559        !  Using d phb(k)/ d eta(k) = -mub * alb(k), eqn 2.9
L00560        !  Skamarock et al, NCAR TN 468.  Use full levels, so
L00561        !  use twice the thickness.
L00562
L00563        DO k = 8, kte-1-2
```

讲解：znw 层面的三条生成路线之 auto_levels 分支的前半：①**prac_levels=59 个「silly levels」**——一套固定精细 η 层（近地面 8 层 1.0→0.88、对流层 0.85→0.02、平流层 0.015→0.00005），用「planned PBL-esque levels + coarse above」积分出模型顶高 ztop——注释自嘲「silly levels, yep, I know」；② 基态温度廓线两段式（temp=max(tiso, t00+A·ln(pb/p00))、pb<p_strat 时 tiso+A_strat·ln(pb/p_strat)）——与 code_40 的基态同式；③mub=p_surf−p_top、phb 向上积分（dφ/dη=−mub·al，**注释引用 Skamarock et al. NCAR TN 468 式 2.9**——技术备忘录的正式出处）；④ztop=phb(59)/g、ztop_pbl=phb(8)/g（前 8 层视为 PBL 段），**dz=(ztop−ztop_pbl)/(kde−8)**——PBL 之上等厚分层；⑤**dz≥max_dz 时 FATAL「Add more levels to namelist.input for e_vert」**并打印模型顶高与层厚——e_vert 太少的经典报错出处（max_dz namelist 默认约 1100 m）；⑥ 前 8 层直接抄 znw_prac（「Standard levels near the surface so no one gets in trouble」）；后半（未录）是 dzstretch_s/u 的拉伸变体与 eta_levels 手工路径。

五、未录单元注记

1. **init_domain_constants_em（L4–133）**：map/lat-lon 等 2D 常数场的嵌套初始化框架。
2. **vert_cor_vertical_nesting_integer/arbitrary（两套，L150–427）**：嵌套域垂直层对齐（整数比与任意比两种映射——垂直嵌套的 eta 对齐策略；文件里同名子程序出现两次是嵌套两个调用语境）。
3. **blend_terrain（L712–785）**：嵌套边界带的地形平滑混合——feedback 后地形突变的缝合。
4. **adjust_tempqv（L812–890）**：嵌套后温湿的保守调整。
5. **input_terrain_rsmas（L892–975）**：RSMAS 高分辨率地形数据的读取。

6. **update_after_feedback_em（L977–1031）**：反馈后父域向子域的状态同步。

六、改造入口

1. **混合函数**：hybrid_opt=2 的五系数（etac 唯一形状参数）与 =3 sin² 型——加第四种混合函数（如 SKYHI 型）的挂点；单调性校验自动护航。
2. **层结构**：auto_levels 的 max_dz/dzbot/dzstretch_s/u——垂直分辨率设计（e_vert 配套）的解析生成。
3. **垂直嵌套对齐**：vert_cor 两套映射——e_vert 不同的父子域层对齐策略改造。

七、上下游链接

- 消费方：`code_40_module_initialize_real.md`（基态构造）、`code_30_module_big_step_utilities_em.md`（c1..c4 进状态方程）
- 常数底座：`code_01_module_model_constants.md`
- 教材正文：ch10_vertical_coords_hydrostatics.md、ch07_real_namelist.md

code_50 — solve_em 主循环导航页（dyn_em/solve_em.F：十四站的地图与调用点索引）

一、版本指纹

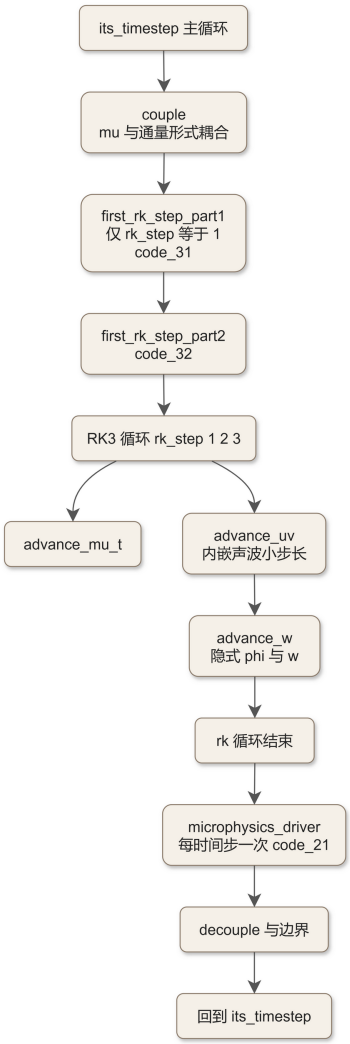
源文件	行数	md5（整文件，按字节）
dyn_em/solve_em.F	5197	9bb6a4d80dcce68a3ec90598b49f5a27

- 本篇为 **C 级导航页**：不给代码块（十四站的逐行走读由教材承载，见 ch21_solve_em_dynamics.md），只给主循环地图与**调用点行号索引**——按行号跳读 solve_em 用。

二、定位

一句话：ARW 动力核的总调度——单子程序 solve_em（约 5100 行）包一个 its_timestep 主循环：耦合/清账 → first_rk_step_part1/part2（仅 rk_step=1）→ RK3 三步（每步内嵌声波小步长循环）→ **RK 循环结束后调微物理**（每时间步一次）→ 解耦与边界。本页与文库各篇的接口：地图上的每个节点都有对应文库篇。

三、主循环地图



四、调用点行号索引（**4.8.0** 实测）

行号	调用	文库/教材锚点
L812	CALL first_rk_step_part1	code_31
L850	CALL first_rk_step_part2	code_32
L1284	CALL advance_uv	ch21 站位（水平动量平流）
L1397	CALL advance_mu_t	ch21（μ 与 θ 大步平流）
L1504	CALL advance_w	ch21（w-φ 垂直隐式解）
L3881	CALL microphysics_driver	code_21

讲解：六个锚点把两大总装件（part1/part2）与动力三巨头（mu_t/uv/w）和微物理钉在行号上；声波小步长的 advance_w 循环嵌在 advance_uv 与 advance_w 之间（RK 每步内 DO 小步）；**微物理在 RK 循环之后**——教材 ch13/ch21 的「微物理每时间步一次」判断的行号依据。其余站（Coriolis/diffusion/六阶扩散/Rayleigh 阻尼等）的调用位置随 config_flags 分派散布于 RK 循环内，明细见 ch21 的十四站表。

五、与文库各篇的对照

solve_em 位置	内容	文库篇
L812/L850	物理总装两件	code_31/32
RK 循环内	phy_prep/状态方程/rhs_ph/Coriolis/diffusion 工具	code_30
L3881	微物理分发	code_21（方案本体 code_07a/07b）
边界与耦合	couple/decouple/w_damp	code_30 § 七注记
常数	全部 USE 自	code_01
基态初值来源	wrfinput（real 生成）	code_40/41

六、上下游链接

- 上游：main/wrf.F → module_integrate（时间循环）→ solve_em；下游：本页地图所列各篇
- 教材正文：ch21_solve_em_dynamics.md（十四站逐行走读——本页不重复）

code_60 — 短波气溶胶光学预处理器（module_ra_aerosol.F：AOD@550 展开成方案波段）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_ra_aerosol.F	1470	e253705336af00334f01b6a1a8af8f49

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（C）：头部注记 + 两个主入口签名头段；十个单元的接口索引见 § 四。WRF-CMAQ/化学路线的气溶胶光学不在此文件（见 code_06 § 五的 RRTMG_SWRAD 内嵌段）。

二、定位

一句话：**aer_opt=2 路线的谱展开器**——把「550 nm AOD + Angstrom 指数 + SSA + 不对称因子」四个（多为 2D 的）输入量，按气溶胶类型与相对湿度展开成 Goddard 11 波段或 RRTMG_SW 14 波段的 3D 光学（tauaer/ssaaer/asyaer）；作者 Jose A. Ruiz-Arias（2012-11 起，2013-12 并入 WRF v3.6 正式版）。

谁在调它：radiation_driver 的 swrad_aerosol_select/swrad_aerosol_select2 两处（`code_20_module_radiation_driver.md` § 五），只服务 GODDARD_SW 与 RRTMG_SW 系方案。

三、头部注记与两个主入口头段

```
!file: phys/module_ra_aerosol.F    L1-L64
L00001  ! Aerosol optical properties from AOD at 550 nm, aerosol type and relative humidity
L00002  ! Author: Jose A. Ruiz-Arias
L00003  ! Init date: 11/2012
L00004  !
L00005  ! Changes:
L00006  !    12/2012, jararias: re-inception using aer_*_opt and aer_*_val options
L00007  ! 16/07/2013, jararias: Angstrom exponent limited to values greater than -0.3. Personal
L00008  !                       communication con Chris Gueymard
L00009  ! 16/07/2013, jararias: Added generation of mean values of angexp, ssa and asy between 0.4 um
L00010  !                       and 1 um to be promoted to the wrfout when parameterization is used (*_opt=3)
L00011  ! 12/2013, jararias: integration in the official WRFv3.6
L00012
L00013  module module_ra_aerosol
L00014
L00015  ! global variables
L00016
L00017  contains
L00018
L00019
L00020  !-----------------------------------------------------------------
L00021  !       INDICES CONVENTION
L00022  !-----------------------------------------------------------------
L00023  !       kms:kme define the range for full-level indices
L00024  !       kts:kte define the range for half-level indices
L00025  !
L00026  !       kms=1 is the first full level at surface
L00027  !       kts=1 is the first half level at surface
L00028  !
L00029  !       kme is the last full level at toa
L00030  !       kte is the last half level at toa
L00031  !
L00032  !       There is one more full level than half levels.
L00033  !       Therefore, kme=kte+1. I checked it in one of my
L00034  !       simulations:
L00035  !
L00036  !         namelist.input:
```

```
L00037  !             s_vert=1  e_vert=28
L00038  !         code:
L00039  !             kms= 1    kts= 1
L00040  !             kms=28    kte=27
L00041  !
L00042  !       In the vertical dimension there is no tiling for
L00043  !       parallelization as in the horizontal dimensions.
L00044  !       For i-dim and j-dim, the t-indices define the
L00045  !       range of indices over which each tile runs.
L00046  !-------------------------------------------------------------
L00047  !
L00048  !  namelist options:
L00049  !    aer_aod550_opt = [1,2] :
L00050  !        1 = input constant value for AOD at 550 nm from namelist.
L00051  !            In this case, the value is read from aer_aod550_val;
L00052  !        2 = input value from auxiliary input 15. It is a time-varying 2D grid in netcdf wrf-compatible
L00053  !            format. The default operation is aer_aod550_opt=1 and aer_aod550_val=0.12
L00054  !    aer_angexp_opt = [1,2,3] :
L00055  !        1 = input constant value for Angstrom exponent from namelist. In this case, the value is read
L00056  !            from aer_angexp_val;
L00057  !        2 = input value from auxiliary input 15, as in aer_aod550_opt;
L00058  !        3 = Angstrom exponent value estimated from the aerosol type defined in aer_type, and modulated
L00059  !            with the RH in WRF. Default operation is aer_angexp_opt = 1, and aer_angexp_val=1.3.
L00060  !    aer_ssa_opt and aer_asy_opt are similar to aer_angexp_opt.
L00061  !
L00062  !    aer_type = [1,2,3] : 1 for rural, 2 is urban and 3 is maritime.
L00063  !-------------------------------------------------------------
L00064
```

讲解：① 头部变更史即功能史：Angstrom 指数下限 −0.3（Gueymard 私人通信——负 Angstrom 意味着消光随波长增大，物理上罕见）、_opt=3 时把 angexp/ssa/asy 的 0.4–1 μm 均值提升到 wrfout（太阳能应用的验证通道）；② **垂直索引约定自查段**（kms/kme 整层 vs kts/kte 半层，kme=kte+1）——作者留了 e_vert=28 的实测验证，是 WRF 垂直索引教学的好例子；③namelist 全景：aer_aod550_opt=1（常数，默认 0.12）/2（auxinput15 时变 2D 场）、aer_angexp_opt/ssa_opt/asy_opt 三选（1 常数/2 auxinput/3 按类型 +RH 参数化）、**aer_type=1 农村/2 城市/3 海洋**——aer_type 只在 opt=3 的参数化分支用。

```
!file: phys/module_ra_aerosol.F    L65-L115
L00065  subroutine calc_aerosol_goddard_sw(ht,dz8w,p,t3d,qv3d,aer_type,                                  &
L00066                                     aer_aod550_opt, aer_angexp_opt, aer_ssa_opt, aer_asy_opt,  &
L00067                                     aer_aod550_val, aer_angexp_val, aer_ssa_val, aer_asy_val,  &
L00068                                     aod5502d, angexp2d, aerssa2d, aerasy2d,                    &
L00069                                     ims,ime,jms,jme,kms,kme,its,ite,jts,jte,kts,kte,           &
L00070                                     tauaer, ssaaer, asyaer                                     )
L00071
L00072      USE module_wrf_error , ONLY : wrf_err_message
L00073
L00074      implicit none
L00075
L00076      ! constants
L00077      integer, parameter :: N_BANDS=11
L00078
L00079      ! mid-band wavelength / 550. nm
L00080      real :: lower_wvl(N_BANDS),upper_wvl(N_BANDS)
L00081      integer :: kk
L00082      data (lower_wvl(kk),kk=1,N_BANDS) /0.175,0.225,0.245,0.280,0.295,0.310,0.325,0.400,0.700,1.220,2.270/
L00083      data (upper_wvl(kk),kk=1,N_BANDS) /0.225,0.245,0.280,0.295,0.310,0.325,0.400,0.700,1.220,2.270,10.00/
L00084
L00085      ! I/O variables
L00086      integer, intent(in) :: ims,ime,jms,jme,kms,kme, &
L00087                             its,ite,jts,jte,kts,kte
L00088      real, dimension(ims:ime, kms:kme, jms:jme), intent(in) :: p,     & ! pressure (Pa)
L00089                                                                t3d,   & ! temperature (K)
L00090                                                                dz8w,  & ! dz between full levels (m)
L00091                                                                qv3d     ! water vapor mixing ratio (kg/kg)
L00092      integer, intent(in) :: aer_type
L00093      integer, intent(in) :: aer_aod550_opt, aer_angexp_opt, aer_ssa_opt, aer_asy_opt
L00094      real, intent(in)    :: aer_aod550_val, aer_angexp_val, aer_ssa_val, aer_asy_val
L00095
L00096      real, dimension(ims:ime, jms:jme),                  intent(in)    :: ht
L00097      real, dimension(ims:ime, jms:jme),           optional, intent(inout) :: aod5502d, angexp2d, aerssa2d, aerasy2d
L00098      real, dimension(ims:ime, kms:kme, jms:jme, 1:N_BANDS), intent(inout) :: tauaer, ssaaer, asyaer
L00099
L00100      ! local variables
L00101      integer :: i,j,k,nb
L00102      real :: aod_rate,angexp_val,x,xy,xx
L00103      real, dimension(ims:ime, jms:jme, 1:N_BANDS) :: aod550spc
L00104      real, dimension(ims:ime, kms:kme, jms:jme)   :: rh  ! relative humidity
L00105
L00106      call calc_relative_humidity(p,t3d,qv3d,                &
L00107                                  ims,ime,jms,jme,kms,kme,   &
L00108                                  its,ite,jts,jte,kts,kte,   &
L00109                                  rh                         )
L00110
L00111      aer_aod550_opt_select: select case(aer_aod550_opt)
L00112         !case(0)
L00113         ! reserved for climatology
L00114         case(1)
L00115            if (aer_aod550_val .lt. 0 ) then
```

讲解：Goddard 短波入口头段：N_BANDS=11 与波段边界表（0.175–10 μm 的 11 段划分）；第一步统一调 calc_relative_humidity（从 p/t/qv 算 RH——RH 用于 opt=3 的吸湿增长参数化）；aer_aod550_opt 分派起点（case(1) 负值报错）。tauaer/ssaaer/asyaer 维度 (ims:ime, kms:kme, jms:jme, 1:11)——注意第四维起点是 1 不是 0，与 radiation_driver 的 allocate (…,1:11) 一致。

```
!file: phys/module_ra_aerosol.F    L397-L472
L00397  subroutine calc_aerosol_rrtmg_sw(ht,dz8w,p,t3d,qv3d,aer_type,                                  &
L00398                                   aer_aod550_opt, aer_angexp_opt, aer_ssa_opt, aer_asy_opt,  &
L00399                                   aer_aod550_val, aer_angexp_val, aer_ssa_val, aer_asy_val,  &
L00400                                   aod5502d, angexp2d, aerssa2d, aerasy2d,                    &
L00401                                   ims,ime,jms,jme,kms,kme,its,ite,jts,jte,kts,kte,           &
L00402                                   tauaer, ssaaer, asyaer, aod5503d                           )
L00403
L00404      USE module_wrf_error , ONLY : wrf_err_message
L00405
L00406      implicit none
L00407
L00408      ! constants
L00409      integer, parameter :: N_BANDS=14
L00410      ! local index variables
L00411      integer :: i,j,k,nb
L00412
L00413      real :: lower_wvl(N_BANDS),upper_wvl(N_BANDS)
L00414      data (lower_wvl(i),i=1,N_BANDS)
↪  /3.077,2.500,2.150,1.942,1.626,1.299,1.242,0.7782,0.6250,0.4415,0.3448,0.2632,0.2000,3.846/
L00415      data (upper_wvl(i),i=1,N_BANDS)
↪  /3.846,3.077,2.500,2.150,1.942,1.626,1.299,1.2420,0.7782,0.6250,0.4415,0.3448,0.2632,12.195/
L00416
L00417      ! I/O variables
L00418      real, dimension(ims:ime, kms:kme, jms:jme), intent(in) :: p,     & ! pressure (Pa)
L00419                                                                t3d,   & ! temperature (K)
L00420                                                                dz8w,  & ! dz between full levels (m)
L00421                                                                qv3d     ! water vapor mixing ratio (kg/kg)
L00422      integer, intent(in) :: ims,ime,jms,jme,kms,kme, &
L00423                             its,ite,jts,jte,kts,kte
L00424      integer, intent(in) :: aer_type
L00425      integer, intent(in) :: aer_aod550_opt, aer_angexp_opt, aer_ssa_opt, aer_asy_opt
L00426      real, intent(in)    :: aer_aod550_val, aer_angexp_val, aer_ssa_val, aer_asy_val
L00427
L00428      real, dimension(ims:ime, jms:jme), intent(in)    :: ht
L00429      real, dimension(ims:ime, jms:jme), optional, intent(inout) :: aod5502d, angexp2d, aerssa2d, aerasy2d
L00430      real, dimension(ims:ime, kms:kme, jms:jme, 1:N_BANDS), intent(inout) :: tauaer, ssaaer, asyaer
L00431
L00432      real, dimension(ims:ime, kms:kme, jms:jme), optional, intent(in) :: aod5503d   ! trude
L00433
L00434      ! local variables
L00435      real :: angexp_val,aod_rate,x,xy,xx
L00436      real, dimension(ims:ime, jms:jme, 1:N_BANDS) :: aod550spc
L00437      real, dimension(ims:ime, kms:kme, jms:jme, 1:N_BANDS) :: aod550spc3d    !  trude
L00438      real, dimension(ims:ime, kms:kme, jms:jme)   :: rh  ! relative humidity
L00439
L00440      call calc_relative_humidity(p,t3d,qv3d,                &
L00441                                  ims,ime,jms,jme,kms,kme,   &
L00442                                  its,ite,jts,jte,kts,kte,   &
L00443                                  rh                         )
L00444
L00445      aer_aod550_opt_select: select case(aer_aod550_opt)
L00446         !case(0)
L00447         ! reserved for climatology
L00448         case(1)
L00449            if (aer_aod550_val .lt. 0) then
L00450               write(wrf_err_message,'("aer_aod550_val must be positive. Negative value ",F7.4," found")') aer_aod550_val
L00451               call wrf_error_fatal(wrf_err_message)
L00452            end if
L00453            write( wrf_err_message, '("aer_aod550_opt=",I1,": AOD@550 nm fixed to value ",F6.3)')
↪  aer_aod550_opt,aer_aod550_val
L00454            call wrf_debug(100, wrf_err_message )
L00455            do j=jts,jte
L00456               do i=its,ite
L00457                  aod5502d(i,j)=aer_aod550_val
L00458               end do
L00459            end do
L00460
L00461         case(2)
L00462            if (.not.(present(aod5502d))) then
L00463               write(wrf_err_message,*) 'Expected gridded total AOD@550 nm, but it is not in the radiation driver'
L00464               call wrf_error_fatal(wrf_err_message)
L00465            end if
L00466            if (minval(aod5502d) .lt. 0) then
L00467               call wrf_error_fatal('AOD@550 must be positive. Negative value(s) found in auxinput')
L00468            end if
L00469            write( wrf_err_message, '("aer_aod550_opt=",I1,": AOD@550 nm read from auxinput (min=",F6.3," max=",F6.3,
↪  ")")') &
L00470                                    aer_aod550_opt,minval(aod5502d),maxval(aod5502d)
L00471            call wrf_debug(100, wrf_err_message )
L00472
```

讲解：RRTMG 短波入口头段：N_BANDS=14 与波段边界表（3.077–12.195 μm 的 14 段——即 RRTMG 波段 16–29 的波长界，逆序排列从长波端开始）；多一个 **aod5503d 可选入参（trude）**——aer_opt=3 的 3D AOD 通道（来自 gt_aod 的 QNWFA/QNIFA 预报，见 code_20 § 五），有它时 550 nm AOD 逐层给定而不用 2D 场；case(1)/case(2) 的输入校验（负值/缺失 auxinput 都 wrf_error_fatal）并打印 min/max 诊断。

四、单元接口索引（10 单元）

单元	行段	功能
calc_aerosol_goddard_sw	L65–L394	11 波段展开（Goddard SW 配套）
calc_aerosol_rrtmg_sw	L397–L766	14 波段展开（RRTMG_SW 配套，多 aod5503d 入口）

单元	行段	功能
calc_spectral_aod_goddard_sw	L769–L851	谱 AOD：Angstrom 律按波段中点波长展开
calc_spectral_ssa_goddard_sw	L853–L935	谱 SSA：按类型查表 +RH 吸湿修正
calc_spectral_asy_goddard_sw	L937–L1019	谱不对称因子：同上
calc_spectral_aod_rrtmg_sw	L1021–L1166	14 波段版谱 AOD（多 3D 入口）
calc_spectral_ssa_rrtmg_sw	L1168–L1274	14 波段版谱 SSA
calc_spectral_asy_rrtmg_sw	L1276–L1382	14 波段版谱不对称因子
aod_profiler	L1384–L1435	AOD 的垂直廓线分配（指数衰减型）
calc_relative_humidity	L1437–L1467	由 p/t/qv 算 RH

讲解：结构高度对称——两个主入口各自调三个 calc_spectral_*（AOD/SSA/ASY）与 aod_profiler；SSA/ASY 的「类型查表 +RH 修正」是 opt=3 的核心（rural/urban/maritime 三套表内嵌在 calc_spectral_ 系里）；aod_profiler 把柱 AOD 分配到层（气溶胶集中在边界层的指数型假设）。

五、改造入口

1. **加气溶胶类型**：aer_type 的三套谱表内嵌在 calc_spectral_ssa/asy 系——加第四类（如沙尘）改表 +分支。
2. **换垂直廓线**：aod_profiler 的指数型——用观测/模式气溶胶廓线替换的入口。
3. **谱展开律**：calc_spectral_aod_* 的 Angstrom 律与 −0.3 下限——用实测谱或 Winnokower 型参数化的挂点。
4. **aer_opt=3 联动**：aod5503d 入口与 radiation_driver 的 gt_aod（QNWFA/QNIFA）链路——沙尘/烟雾预报实验的通道。

六、上下游链接

- 上游驱动：`code_20_module_radiation_driver.md`（swrad_aerosol_select 两处挂点、gt_aod 的 3D AOD）
- 下游方案：`code_06_module_ra_rrtmg_sw.md`（tauaer_sw 14 波段消费方）、Goddard SW（11 波段）
- 教材正文：ch12_radiation_family.md、ch20_modifying_wrf.md

code_61 — Noah 的海冰与冰川件（module_sf_noah_seaice_drv.F + module_sf_noah_seaice.F + module_sf_noahlsm_glacial_only.F：C 级接口索引）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_sf_noah_seaice_drv.F	502	510c4ea03c8ee8d942375581d258ab6b
phys/module_sf_noah_seaice.F	1291	ca3bee0470b9f9127f8c8c9cd32b033a
phys/module_sf_noahlsm_glacial_only.F	1280	8d406e754b11825964f043f359fa2275

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（C）：三件各收头部（模块声明与主入口签名）；内部单元走同源差异注记。上游：`code_12a_module_sf_noahlsm.md`、`code_13_module_sf_noahdrv.md`（ICE 三分支的调用侧）。

二、定位

一句话：Noah 生态的两个**极地特化件**——海冰件（seaice_drv→seaice：带 5 cm 雪层的固定厚度冰面能量平衡）与冰川件（glacial_only：无土壤水的冰盖能量平衡）；两者都是 noahlsm 的**裁剪孪生**——CSNOW/HRTICE/PENMAN/SHFLX/SNOPAC/SNOWPACK/SNOWZ0/SNOW_NEW 同名单元直接复用或改写 noahlsm 的对应件，但常数经 `USE module_sf_noahlsm` 导入（RD/SIGMA/CPH2O/CPICE/LSUBF/EMISSI_S，seaice 还导入 HSTEP、glacial 导入 ROSR12 与 LVCOEF_DATA）——**三文件常数/求解器单点维护**的样板。

谁在调它们：seaice_noah 由 surface_driver 的 OML/海冰分支调（fractional_seaice 选项，code_24 § 二）；SFLX_GLACIAL 由 noahdrv 的 ICE=−1 分支调（code_13 §4.4）。

三、三件头部

```
!file: phys/module_sf_noah_seaice.F    L1-L35
L00001  MODULE module_sf_noah_seaice
L00002  #if defined(mpas)
L00003  use mpas_atmphys_constants,only: cp,R_D=>R_d,XLF,XLV,RHOWATER=>rho_w,STBOLT
L00004  use mpas_atmphys_utilities, only: physics_error_fatal
L00005  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00006  #else
L00007  use module_model_constants, only : CP, R_D, XLF, XLV, RHOWATER, STBOLT
L00008  use module_wrf_error
L00009  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00010  #endif
L00011    use module_sf_noahlsm, only : RD, SIGMA, CPH2O, CPICE, LSUBF, EMISSI_S, &
L00012         &                        HSTEP
L00013
L00014    PUBLIC  SFLX_SEAICE
L00015    PRIVATE CSNOW
L00016    PRIVATE HRTICE
L00017    PRIVATE PENMAN
L00018    PRIVATE SHFLX
L00019    PRIVATE SNOPAC
L00020    PRIVATE SNOWPACK
L00021    PRIVATE SNOWZ0
L00022    PRIVATE SNOW_NEW
L00023
L00024    INTEGER, PRIVATE :: ILOC
```

```
L00025    INTEGER, PRIVATE :: JLOC
L00026  !$omp threadprivate(iloc, jloc)
L00027
L00028    REAL, PARAMETER, PRIVATE :: TFREEZ = 273.15
L00029  !
L00030  CONTAINS
L00031  !
L00032    SUBROUTINE SFLX_SEAICE (IILOC, JJLOC, SEAICE_ALBEDO_OPT, SEAICE_ALBEDO_DEFAULT, &    !C
L00033         &                  SEAICE_SNOWDEPTH_OPT, SEAICE_SNOWDEPTH_MAX,      &    !C
L00034         &                  SEAICE_SNOWDEPTH_MIN,                            &    !C
L00035         &                  FFROZP,DT,ZLVL,NSOIL,                            &    !C
```

讲解：seaice 件头部三件事：①MPAS/WRF 双框架宏（与 noahlsm 同款）；②use module_sf_noahlsm, only: RD, SIGMA, CPH2O, CPICE, LSUBF, EMISSI_S, HSTEP——**常数与热推进求解器都从 noahlsm 导入**（HSTEP 指冰内温度层的推进——海冰按 4 层冰 +雪层处理但复用 Noah 的三对角推进）；③PUBLIC 只出 SFLX_SEAICE、其余八单元全 PRIVATE；TFREEZ=273.15 自带。主入口签名可见海冰五选项（SEAICE_ALBEDO_OPT/DEFAULT、SEAICE_SNOWDEPTH_OPT/MAX/MIN——namelist &physics 直通）。

```
!file: phys/module_sf_noah_seaice_drv.F    L1-L35
L00001  module module_sf_noah_seaice_drv
L00002  #if defined(mpas)
L00003  use mpas_atmphys_utilities, only: physics_message,physics_error_fatal
L00004  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00005  #define WRITE_MESSAGE(M) call physics_message( M )
L00006  #else
L00007  use module_wrf_error
L00008  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00009  #define WRITE_MESSAGE(M) call wrf_message( M )
L00010  #endif
L00011    use module_sf_noah_seaice
L00012    implicit none
L00013  contains
L00014    subroutine seaice_noah( SEAICE_ALBEDO_OPT, SEAICE_ALBEDO_DEFAULT, SEAICE_THICKNESS_OPT, &
L00015         &                  SEAICE_THICKNESS_DEFAULT, SEAICE_SNOWDEPTH_OPT,                &
L00016         &                  SEAICE_SNOWDEPTH_MAX, SEAICE_SNOWDEPTH_MIN,                    &
L00017         &                  T3D, QV3D, P8W3D, DZ8W, NUM_SOIL_LAYERS, DT, FRPCPN, SR,       &
L00018         &                  GLW, SWDOWN, RAINBL, SNOALB2D, QGH, XICE, XICE_THRESHOLD,      &
L00019         &                  ALBSI, ICEDEPTH, SNOWSI,                                       &
L00020         &                  TSLB, EMISS, ALBEDO, Z02D, TSK, SNOW, SNOWC, SNOWH2D, &
L00021         &                  CHS, CHS2, CQS2,                                               &
L00022         &                  RIB, ZNT, LH, HFX, QFX, POTEVP, GRDFLX, QSFC, ACSNOW,          &
L00023         &                  ACSNOM, SNOPCX, SFCRUNOFF, NOAHRES,                            &
L00024         &                  SF_URBAN_PHYSICS, B_T_BEP, B_Q_BEP, RHO,                       &
L00025         &                  IDS, IDE, JDS, JDE, KDS, KDE,                                  &
L00026         &                  IMS, IME, JMS, JME, KMS, KME,                                  &
L00027         &                  ITS, ITE, JTS, JTE, KTS, KTE  )
L00028  #if defined(wrfmodel)
L00029  #if (NMM_CORE != 1)
L00030      USE module_state_description, ONLY : NOAHUCMSCHEME
L00031      USE module_state_description, ONLY : BEPSCHEME
L00032      USE module_state_description, ONLY : BEP_BEMSCHEME
L00033  #endif
L00034  #endif
L00035      implicit none
```

讲解：**海冰驱动层**（seaice_noah，487 行单子程序）：签名即功能——五选项全收 + ICEDEPTH/SNOWSI/ALBSI 三个海冰专用态（冰厚/雪深/反照率，Registry 由 real 初始化 ifndicedepth/ifndalbsi 旗标控制）、XICE/XICE_THRESHOLD 分值冰判据；#if (NMM_CORE != 1) 下 USE state_description 的 NOAHUCMSCHEME/BEPSCHEME/BEP_BEMSCHEME——**海冰点在城市方案下的兼容处理**（B_T_BEP/B_Q_BEP 入参：海冰点的 BEP 源项要清零）。内部按格点判 XICE≥threshold 后调 SFLX_SEAICE（逐列）。

```
!file: phys/module_sf_noahlsm_glacial_only.F    L1-L40
L00001  MODULE module_sf_noahlsm_glacial_only
L00002  #if defined(mpas)
L00003  use mpas_atmphys_constants
L00004  use mpas_atmphys_utilities, only: physics_error_fatal
L00005  #define FATAL_ERROR(M) call physics_error_fatal( M )
L00006  #else
L00007  use module_model_constants
L00008  use module_wrf_error
L00009  #define FATAL_ERROR(M) call wrf_error_fatal( M )
L00010  #endif
L00011
L00012    USE module_sf_noahlsm, ONLY : RD, SIGMA, CPH2O, CPICE, LSUBF, EMISSI_S, ROSR12
L00013    USE module_sf_noahlsm, ONLY : LVCOEF_DATA
L00014
L00015    PRIVATE :: ALCALC
L00016    PRIVATE :: CSNOW
L00017    PRIVATE :: HRTICE
L00018    PRIVATE :: HSTEP
L00019    PRIVATE :: PENMAN
L00020    PRIVATE :: SHFLX
L00021    PRIVATE :: SNOPAC
L00022    PRIVATE :: SNOWPACK
L00023    PRIVATE :: SNOWZ0
L00024    PRIVATE :: SNOW_NEW
L00025
L00026    integer, private :: iloc, jloc
L00027  !$omp threadprivate(iloc, jloc)
L00028
L00029  CONTAINS
L00030
L00031    SUBROUTINE SFLX_GLACIAL (IILOC,JJLOC,ISICE,FFROZP,DT,ZLVL,NSOIL,SLDPTH,     &    !C
```

```
L00032         &                   LWDN,SOLNET,SFCPRS,PRCP,SFCTMP,Q2,           &    !F
L00033         &                   TH2,Q2SAT,DQSDT2,                            &    !I
L00034         &                   ALB, SNOALB,TBOT, Z0BRD, Z0, EMISSI, EMBRD,  &    !S
L00035         &                   T1,STC,SNOWH,SNEQV,ALBEDO,CH,                &    !H
L00036  ! ------------------------------------------------------------------
L00037  ! OUTPUTS, DIAGNOSTICS, PARAMETERS BELOW GENERALLY NOT NECESSARY WHEN
L00038  ! COUPLED WITH E.G. A NWP MODEL (SUCH AS THE NOAA/NWS/NCEP MESOSCALE ETA
L00039  ! MODEL).  OTHER APPLICATIONS MAY REQUIRE DIFFERENT OUTPUT VARIABLES.
L00040  ! ------------------------------------------------------------------
```

讲解：冰川件头部：USE noahlsm 取 RD/SIGMA/CPH2O/CPICE/LSUBF/EMISSI_S + **ROSR12（三对角求解器）+ LVCOEF_DATA（雪反照率老化系数）**——冰内温度层推进与雪反照率全套复用；PRIVATE 十单元、只出 SFLX_GLACIAL。主入口签名比 SFLX 精简一个量级：无土壤水/无蒸腾/无 RUNOFF2（对照 code_13 §4.4 的实参表——只有 RUNOFF1）。

四、同源单元差异注记（不重录）

单元	seaice（L544–L1288）	glacial（L412–L1199）	与 noahlsm 的关系
CSNOW	40 行	37 行	同 Dyachkova 式；冰面不 ×2 翻倍（对照 12a § 六——**翻倍系数只在陆地件**）
HRTICE	119 行	143 行	**HRT 的冰面特化**：冰层导热（无对流项、相变在界面）
PENMAN	73 行	64 行	精简版（无降水热容项的完整处理）
SHFLX	28 行	30 行	冰内热通量壳（调 HRTICE+HSTEP/自推进）
SNOPAC	265 行	218 行	雪盖路径裁剪版（无土壤水交互）
SNOWPACK/SNOWZ0/SNOW_NEW	126/30/50 行	124/—/—	与 noahlsm 同构（压密/粗糙度/新雪）
ALCALC	—	37 行	雪龄反照率（glacial 带、seaice 用 SEAICE_ALBEDO_OPT 常数表）
HSTEP	（USE noahlsm）	46 行	seaice 直接复用 noahlsm 的推进件

讲解：读法建议——先读 noahlsm（code_12a/12b）建立基准，再看两件的差异行即可；**海冰/冰川的能量守恒框架与陆地 Noah 同构**（PENMAN 潜势蒸发 →SNOPAC 雪盖 →SNOWPACK 压密），差异集中在：无土壤水储池、冰层导热用 HRTICE、雪/冰反照率走 SEAICE_ALBEDO_OPT 表或 ALCALC。

五、改造入口

1. **海冰能量平衡升级**（厚冰分类/冰内卤水传导）：SFLX_SEAICE 的固定厚度假设是第一改造点——SEAICE_THICKNESS_OPT 已是挂点。
2. **分值海冰**：seaice_noah 的 XICE_THRESHOLD（0.5/0.02 双口径，code_61 与 lsm_mosaic_init 的取值链）。
3. **冰川物质平衡**：SFLX_GLACIAL 只有 RUNOFF1——融雪出流直接流失，加冰面升华/再冻结账户在此。

六、上下游链接

- 调用链：`code_24_module_surface_driver.md`（seaice_noah）、`code_13_module_sf_noahdrv.md`（SFLX_GLACIAL 的 ICE 分支）
- 同源基准：`code_12a_module_sf_noahlsm.md`、`code_12b_module_sf_noahlsm.md`
- 教材正文：ch16_land_surface_family.md

code_62 — 次级驱动器三件头部（shallowcu_driver + module_fr_fire_driver_wrf.F + fddagd_driver：C 级接口索引）

一、版本指纹

源文件	行数	md5（整文件，按字节）
phys/module_shallowcu_driver.F	767	8b7d55b57a824915f5e32a3aa0e1955c
phys/module_fr_fire_driver_wrf.F	149	8db0ddd868c46e10e1b02d060a617554
phys/module_fddagd_driver.F	626	e068c0b8151240276176f2c08d5eaa96

- 抽取工具：workspace/scripts/code_extract.py v1.2，2026-10-12；块首指纹注释自带 range_md5。
- 收录分级（C）：三件头部（模块声明与主入口签名段）+ 单元索引；fire 包装层全文（149 行全收——它本身薄）。教材锚点：ch15_cumulus_family.md（独立浅对流）、ch21_solve_em_dynamics.md（调用位置）。

二、定位

一句话：part1 总装层里三个**次级驱动器**的接口索引——独立浅对流（shcu_physics，与主积云并行的第二积云通道）、火模式 WRF 包装层（ifire=2 的 fire_driver_em_step 与 rthfrten/rqvfrten 倾向出口）、格点 nudging（fddagd_driver，fdda3d/fdda2d 与 ndg_old/new 双时刻场的消费者）。

谁在调它们：first_rk_step_part1（`code_31_module_first_rk_step_part1.md` § 九/§ 十/§ 十一——调用序在此三段的末三站）。

三、独立浅对流 shallowcu_driver

```
!file: phys/module_shallowcu_driver.F    L1-L55
L00001  !WRF:MEDIATION_LAYER:PHYSICS
L00002  !
L00003
L00004  MODULE module_shallowcu_driver
L00005
L00006    implicit none
L00007
L00008    real, private, parameter :: QCLOUD_MIN = 0.0
L00009    real, private, parameter :: QVAPOR_MIN = 0.0
L00010
L00011  CONTAINS
L00012     SUBROUTINE shallowcu_driver(                                    &
L00013                   ! Order dependent args for domain, mem, and tile dims
L00014                        ids,ide, jds,jde, kds,kde                    &
L00015                       ,ims,ime, jms,jme, kms,kme                    &
L00016                       ,ips,ipe, jps,jpe, kps,kpe                    &
L00017                       ,i_start,i_end,j_start,j_end,kts,kte,num_tiles  &
L00018                   ! Order independent args (use VAR= in call)
L00019                   ! --Prognostic
L00020                       ,u,v,th,t                                     &
L00021                       ,p,pi,rho,moist                               &
L00022                   ! --Other arguments
L00023                       ,num_moist                                    &
L00024                       ,itimestep,dt,dx,dx2d,area2d                  &
L00025                       ,cudt,curr_secs,adapt_step_flag               &
L00026                       ,rainsh,pratesh,nca,rainshv                   &
L00027                       ,z,z_at_w,dz8w,mavail,pblh,p8w                &
L00028                       ,tke_pbl                                      &
L00029                       ,cldfra,cldfra_old,cldfra_old_mp,cldfra_conv  &
L00030                       ,cldfrash                                     &
L00031                       ,htop,hbot                                    &
L00032                   ! Package selection variables
L00033                       ,shcu_physics                                 &
```

```
L00034                   ! Optional moisture tracers
L00035                       ,qv_curr, qc_curr, qr_curr                      &
L00036                       ,qi_curr, qs_curr, qg_curr                      &
L00037                       ,qnc_curr,qni_curr                              &
L00038  #if ( WRF_CHEM == 1 )
L00039                       ,chem, chem_opt                                 &
L00040  #endif
L00041                   ! Optional output arguments for CAMZM scheme
L00042                       ,dlf, rliq, rliq2,dlf2  &
L00043                       ,cmfmc, cmfmc2       &
L00044                   ! Optional output arguments for CAMUW scheme
L00045                       ,cush, snowsh, icwmrsh, rprdsh, cbmf, cmfsl      &
L00046                       ,cmflq, evapcsh                                 &
L00047                   ! Optional moisture and other tendencies
L00048                       ,rqvshten,rqcshten,rqrshten                     &
L00049                       ,rqishten,rqsshten,rqgshten                     &
L00050                       ,rqcnshten,rqinshten                            &
L00051                       ,rqvblten,rqvften                               &
L00052                       ,rushten,rvshten                                &
L00053                       ,rthshten,rthraten,rthblten,rthften             &
L00054                   ! Optional moisture tracer flags
L00055                       ,f_qv,f_qc,f_qr                                 &
```

讲解：与 cumulus_driver 同族的调度器（682 行单子程序 + SPP 加/减对两件）：① **签名差异即语义差异**——rainsh/pratesh/rainshv（浅对流降水独立账本）、cldfrash（浅对流云量）、htop/hbot 共用积云的云顶底数组；②CAMZM/CAMUW 专属输出（dlf/rliq/cmfmc 与 cush/snowsh/icwmrsh/cbmf/cmfsl/cmflq/evapcsh——CAM 浅对流诊断全家）经 OPTIONAL 传入；③ 倾向出口 rqvshten/rqcshten/rqrshten/rqishten 系（**浅对流带雨/冰倾向**——比主积云的若干方案更全）；④QCLOUD_MIN/QVAPOR_MIN 模块级钳位。**与主积云的关系**：shcu_physics 与 cu_physics 可同时开（双积云并行的物理风险见 code_23 § 九），G3 的内嵌浅对流与这里的独立浅对流是两条线。SPP 对（Add/Remove_multi_perturb_shcu，L695–L765）与 radiation_driver 的同名件同构（code_20 § 八）。

四、火模式 WRF 包装层 module_fr_fire_driver_wrf.F（全文）

```
!file: phys/module_fr_fire_driver_wrf.F    L1-L149
L00001  ! WRF:MEDIATION_LAYER:FIRE_MODEL
L00002
L00003
L00004  #define DEBUG_OUT
L00005
L00006  module module_fr_fire_driver_wrf
L00007  ! wrf-specific driver
L00008
L00009  use module_fr_fire_driver
L00010  use module_fr_fire_atm
L00011  implicit none
L00012
L00013  contains
L00014
L00015  subroutine fire_driver_em_init (grid , config_flags               &
L00016              ,ids,ide, kds,kde, jds,jde                              &
L00017              ,ims,ime, kms,kme, jms,jme                              &
L00018              ,ips,ipe, kps,kpe, jps,jpe)
L00019
L00020      ! stub to call fire_driver_em with irun=0 and omit last 3 args
L00021
L00022      USE module_domain , only: domain , get_ijk_from_subgrid
L00023      USE module_configure , only : grid_config_rec_type
L00024      implicit none
L00025
L00026      TYPE(domain) , TARGET          :: grid   ! data
L00027      TYPE (grid_config_rec_type) , INTENT(IN)          :: config_flags
L00028      integer, intent(in):: &
L00029               ids,ide, kds,kde, jds,jde                              &
L00030              ,ims,ime, kms,kme, jms,jme                              &
L00031              ,ips,ipe, kps,kpe, jps,jpe
L00032
L00033      ! local
L00034      integer :: &  ! fire mesh sizes
L00035               ifds,ifde, kfds,kfde, jfds,jfde,                                  &
L00036               ifms,ifme, kfms,kfme, jfms,jfme,                                  &
L00037               ifps,ifpe, kfps,kfpe, jfps,jfpe
L00038      ! dummies
L00039
L00040      real,dimension(1,1,1)::rho,z_at_w,dz8w
L00041
L00042      call message('fire_driver_em_init: FIRE initialization start')
L00043
L00044      ! get fire mesh dimensions
L00045      CALL get_ijk_from_subgrid (  grid ,                   &
L00046                              ifds,ifde, jfds,jfde,kfds,kfde,                  &
L00047                              ifms,ifme, jfms,jfme,kfms,kfme,                  &
L00048                              ifps,ifpe, jfps,jfpe,kfps,kfpe)
L00049
L00050      call fire_driver_em ( grid , config_flags               &
L00051              ,1,2,0                                  & ! ifun start, end, test steps
L00052              ,ids,ide, kds,kde, jds,jde                              &
L00053              ,ims,ime, kms,kme, jms,jme                              &
L00054              ,ips,ipe, kps,kpe, jps,jpe                              &
L00055              ,ifds,ifde, jfds,jfde                                   &
L00056              ,ifms,ifme, jfms,jfme                                   &
L00057              ,ifps,ifpe, jfps,jfpe                                   &
L00058              )
L00059
L00060      call message('fire_driver_em_init: FIRE initialization complete')
L00061
```

```
L00062  end subroutine fire_driver_em_init
L00063
L00064  !
L00065  !***
L00066  !
L00067
L00068  subroutine fire_driver_em_step (grid , config_flags               &
L00069              ,ids,ide, kds,kde, jds,jde                             &
L00070              ,ims,ime, kms,kme, jms,jme                             &
L00071              ,ips,ipe, kps,kpe, jps,jpe                             &
L00072              ,rho,z_at_w,dz8w )
L00073
L00074      ! stub to call fire_driver_em
L00075
L00076      USE module_domain, only: domain , get_ijk_from_subgrid
L00077      USE module_configure , only : grid_config_rec_type
L00078      USE module_fr_fire_util, only : fire_test_steps
L00079      implicit none
L00080
L00081      TYPE(domain) , TARGET          :: grid   ! data
L00082      TYPE (grid_config_rec_type) , INTENT(IN)          :: config_flags
L00083      integer, intent(in):: &
L00084              ids,ide, kds,kde, jds,jde                              &
L00085             ,ims,ime, kms,kme, jms,jme                              &
L00086              ,ips,ipe, kps,kpe, jps,jpe
L00087      real,dimension(ims:ime, kms:kme, jms:jme),intent(in)::rho,z_at_w,dz8w
L00088
L00089      ! local
L00090      integer :: &  ! fire mesh sizes
L00091               ifds,ifde, kfds,kfde, jfds,jfde,                                  &
L00092               ifms,ifme, kfms,kfme, jfms,jfme,                                  &
L00093               ifps,ifpe, kfps,kfpe, jfps,jfpe
L00094      integer :: its,ite,jts,jte,kts,kte            ! atm tile
L00095      integer:: ij
L00096
L00097      call message('fire_driver_em_step: FIRE step start')
L00098
L00099      ! get fire mesh dimensions
L00100      CALL get_ijk_from_subgrid (  grid ,                  &
L00101                              ifds,ifde, jfds,jfde,kfds,kfde,                       &
L00102                              ifms,ifme, jfms,jfme,kfms,kfme,                       &
L00103                              ifps,ifpe, jfps,jfpe,kfps,kfpe)
L00104
L00105      call fire_driver_em ( grid , config_flags               &
L00106             ,3,6,fire_test_steps                                 &
L00107             ,ids,ide, kds,kde, jds,jde                             &
L00108             ,ims,ime, kms,kme, jms,jme                             &
L00109             ,ips,ipe, kps,kpe, jps,jpe                             &
L00110             ,ifds,ifde, jfds,jfde                                  &
L00111             ,ifms,ifme, jfms,jfme                                  &
L00112             ,ifps,ifpe, jfps,jfpe                                  &
L00113             ,rho,z_at_w,dz8w                                       &
L00114             )
L00115
L00116
L00117      ! --- add heat and moisture fluxes to tendency variables by postulated decay
L00118      do ij=1,grid%num_tiles
L00119         ! FIRE works on domain by 1 smaller, in last row&col winds are not set properly
L00120         its = grid%i_start(ij)          ! start atmospheric tile in i
L00121         ite = min(grid%i_end(ij),ide-1)    ! end atmospheric tile in i
L00122         jts = grid%j_start(ij)          ! start atmospheric tile in j
L00123         jte = min(grid%j_end(ij),jde-1)    ! end atmospheric tile in j
L00124         kts=kds
L00125         kte=kde
L00126
L00127         call fire_tendency(                &
L00128              ids,ide-1, kds,kde, jds,jde-1, & ! domain dimensions
L00129              ims,ime, kms,kme, jms,jme,      &
L00130              its,ite, kts,kte, jts,jte,      & !
L00131              grid%grnhfx,grid%grnqfx,grid%canhfx,grid%canqfx,        & ! fluxes on atm grid
L00132              config_flags%fire_ext_grnd,config_flags%fire_ext_crwn,config_flags%fire_crwn_hgt,              &
L00133              config_flags%fire_sfc_flx,config_flags%fire_heat_peak,config_flags%fire_tg_ub, &
L00134              grid%ht,z_at_w,dz8w,grid%mut,grid%c1h,grid%c2h,rho,          &
L00135              grid%rthfrten,grid%rqvfrten)                ! out
L00136
L00137       enddo
L00138
L00139         ! debug print to compare
L00140
L00141         call print_3d_stats(its,ite,kts,kte,jts,jte,ims,ime,kms,kme,jms,jme,grid%rthfrten,'fire_driver_phys:rthfrten')
L00142         call print_3d_stats(its,ite,kts,kte,jts,jte,ims,ime,kms,kme,jms,jme,grid%rqvfrten,'fire_driver_phys:rqvfrten')
L00143
L00144      call message('fire_driver_em_step: FIRE step complete')
L00145
L00146  end subroutine fire_driver_em_step
L00147
L00148  end module module_fr_fire_driver_wrf
L00149
```

讲解：**只有 149 行的薄包装**——WRF 大气与理想火模式（module_fr_fire_driver，77 KB 实体）之间的全部接口：①fire_driver_em_init：stub 调 fire_driver_em(irun=1,2,0)——**ifun 1/2 是初始化段**（火网格 ifds..ifpe 三层子格索引由 get_ijk_from_subgrid 取——火模式有自己的比大气更细的网格）；②fire_driver_em_step：stub 调 fire_driver_em(**irun=3,6**,fire_test_steps)——3–6 号任务步每步执行，入参只要 rho/z_at_w/dz8w 三个大气场；③ 段尾 **fire_tendency**：把火面热通量（grnhfx 地表/grnqfx/canhfx 树冠/canqfx）按 fire_ext_grnd/fire_ext_crwn/fire_crwn_hgt/fire_sfc_flx/fire_heat_peak/fire_tg_ub 六个 namelist 参数的「假定衰减廓线」注入 **grid%rthfrten/rqvfrten**——这就是 part2 的 update_phy_ten 实参表里 rthfrten/rqvfrten（注释「fire」）的来源（code_32 § 八）；④「FIRE works on domain by 1 smaller」——火网格边界一圈风场不可用的诚实注释。火物理本体（理想火蔓延、Level-Set/CFBM）在 module_fr_fire_driver.F，不在文库范围（ch20 案例库引用）。

五、格点 nudging fddagd_driver

```
!file: phys/module_fddagd_driver.F    L1-L45
L00001  !WRF:MEDIATION_LAYER:PHYSICS
L00002  !
L00003
L00004  MODULE module_fddagd_driver
L00005  CONTAINS
L00006
L00007  !-------------------------------------------------------------------
L00008     SUBROUTINE fddagd_driver(itimestep,dt,xtime,                    &
L00009                   curr_mins2, id,  &
L00010                   RUNDGDTEN,RVNDGDTEN,RTHNDGDTEN,                   &
L00011                   RPHNDGDTEN,RQVNDGDTEN,RMUNDGDTEN,                 &
L00012                   SDA_HFX, SDA_QFX,                                 & !fasdas
L00013                   HFX_FDDA,                                         & !fasdas
L00014                   u_ndg_old,v_ndg_old,t_ndg_old,ph_ndg_old,         &
L00015                   q_ndg_old,mu_ndg_old,       &
L00016                   u_ndg_new,v_ndg_new,t_ndg_new,ph_ndg_new,         &
L00017                   q_ndg_new,mu_ndg_new,       &
L00018                   u3d,v3d,th_phy,ph,rho,moist,                      &
L00019                   p_phy,pi_phy,p8w,t_phy,dz8w,z,z_at_w,             &
L00020                   grid,config_flags,DX,n_moist,                     &
L00021                   STEPFG,                                           &
L00022                   pblh,ht,regime,znt,                               &
L00023                   ids,ide, jds,jde, kds,kde,                        &
L00024                   ims,ime, jms,jme, kms,kme,                        &
L00025                   i_start,i_end, j_start,j_end, kts,kte, num_tiles, &
L00026                   u10, v10, th2, q2, &
L00027                   u10_ndg_old, v10_ndg_old, t2_ndg_old, th2_ndg_old, q2_ndg_old,  &
L00028                   rh_ndg_old, psl_ndg_old, ps_ndg_old, tob_ndg_old, odis_ndg_old, &
L00029                   u10_ndg_new, v10_ndg_new, t2_ndg_new, th2_ndg_new, q2_ndg_new,  &
L00030                   rh_ndg_new, psl_ndg_new, ps_ndg_new, tob_ndg_new, odis_ndg_new, &
L00031                   ips,ipe,jps,jpe,kps,kpe,                          &
L00032                   imsx,imex,jmsx,jmex,kmsx,kmex,                    &
L00033                   ipsx,ipex,jpsx,jpex,kpsx,kpex,                    &
L00034                   imsy,imey,jmsy,jmey,kmsy,kmey,                    &
L00035                   ipsy,ipey,jpsy,jpey,kpsy,kpey                     )
L00036  !-------------------------------------------------------------------
L00037     USE module_configure
L00038     USE module_state_description
L00039     USE module_model_constants
L00040     USE module_domain, ONLY : domain
L00041
L00042  ! *** add new modules of schemes here
L00043
L00044     USE module_fdda_psufddagd
L00045     USE module_fdda_spnudging
```

讲解：单子程序 618 行的 nudging 总装：① **双时刻场对**（u_ndg_old/new 五组 3D + u10_ndg_old/new 等十组 2D）——nudging 的分析场以 old/new 两时刻输入、时间线性插值到当前 xtime（fdda 前处理生成，Registry 态）；② **倾向出口六件** RUNDGDTEN/RVNDGDTEN/RTHNDGDTEN/RPHNDGDTEN/RQVNDGDTEN/RMUNDGDTEN（mu 的 nudging 倾向都有）——经 calculate_phy_tend/update_phy_ten 进总账（code_32 § 五/§ 八）；③SDA_HFX/SDA_QFX/HFX_FDDA（fasdas 同化与 nudging 的感热交互）；④STEPFG（nudging 自身的时间门控步长，与 grid_fdda 系 namelist 配套）；⑤wind/temp/moist/geo-p/sfc 五通道各自的开关与系数在 config_flags（grid_fdda/i/e 系）——内部按通道分派的散代码未录。

六、改造入口

1. **双积云叠加风险**：shcu+cu 同时开的对照实验（浅对流云量 cldfrash 是否与 cldfra_dp/sh 双计）。
2. **火-大气耦合强度**：fire_sfc_flx/fire_heat_peak 六参数与 rthfrten 注入廓线（fire_tendency 内）——火灾天气实验的强度旋钮。
3. **nudging 通道**：五通道开关/系数/时间窗——再分析拉回实验（ch19 的 ERA5 对照可扩展为 nudging 版）。

七、上下游链接

- 调用层：`code_31_module_first_rk_step_part1.md`；记账：`code_32_module_first_rk_step_part2.md`
- 同族：`code_23_module_cumulus_driver.md`（主积云对照）
- 教材正文：ch15_cumulus_family.md、ch21_solve_em_dynamics.md